Contents

1 pricklypear.borg **11**

2 borge/philosophy.borg **21**

3 borge/single-image.borg **22**

4 borge/object-model.borg **26**

5 borge/one-tree.borg **28**

6 borge/tree-time.borg **43**

7 borge/branching.borg **52**

8 borge/lisp-interpreter.borg **68**

9 borge/agent.borg **91**

10 borge/remote-eval.borg **100**

11 borge/local-first.borg **111**

12 borge/storage.borg **120**

13 borge/multi-user.borg **124**

14 borge/auth.borg **129**

15 borge/image.borg **142**

16 borge/multipart.borg **171**

17 borge/hypermedia-harness.borg **175**

18 borge/product-experience.borg **180**

19 borge/notes.borg **187**

20 borge/structural-editing.borg **204**

21 borge/calendar.borg **210**

22 borge/flashcards.borg **211**

23 borge/workout.borg **214**

24 borge/dabar.borg **219**

25 borge/notify.borg **224**

26 borge/migrations.borg **226**

27 borge/formal-verification.borg **229**

28 borge/ai-authoring.borg **230**

29 borge/film-analysis.borg **236**

30 borge/film-edit.borg **240**

31 borge/jobs.borg **247**

32 borge/forge.borg **252**

33 borge/habitat-workflow.borg **258**

34 borge/habitat-book.borg **262**

35 agent/bin/main.ml **266**

36 agent/lib/http_ui.ml **272**

37 agent/lib/plugins.ml **281**

38 agent/lib/runtime.ml **282**

39 agent/lib/surface.ml **294**

40 agent/test/agent_test.ml **298**

41 agent/test/load_probe.ml **308**

42 cli/bin/pp_eval.ml **309**

43 cli/bin/pp_sync.ml **312**

44 cli/lib/pp_client.ml **322**

45 client/bin/main.ml **325**

46 client/conformance/main.ml **329**

47 client/ops.pp **330**

48 common/lib/auth_types.ml **331**

49 common/lib/branch.ml **333**

50 common/lib/llm_codec.ml **335**

51 common/lib/manifest.ml **340**

52 common/lib/protocol.ml **342**

53 common/test/common_test.ml **343**

54 image/bin/main.ml **347**

55 image/lib/agent_discovery.ml **348**

56 image/lib/agent_tools.ml **358**

57 image/lib/ai_store.ml **381**

58 image/lib/analyze.ml **388**

59 image/lib/ast_html.ml **393**

60 image/lib/auth_password.ml **396**

61 image/lib/auth_store.ml **398**

62 image/lib/auth_throttle.ml **417**

63 image/lib/boot.ml **419**

64 image/lib/call_graph.ml 423

65 image/lib/control_rim.ml 428

66 image/lib/crud.ml 429

67 image/lib/db_cost.ml 457

68 image/lib/example_eval.ml 459

69 image/lib/function_stats.ml 463

70 image/lib/http.ml 464

71 image/lib/http_agent.ml 482

72 image/lib/http_analyze.ml 495

73 image/lib/http_branches.ml 497

74 image/lib/http_code.ml 511

75 image/lib/http_core.ml 519

76 image/lib/http_data.ml 546

77 image/lib/http_media.ml 556

78 image/lib/http_ops.ml 567

79 image/lib/http_pages.ml 570

80 image/lib/http_tree.ml 595

81 image/lib/io.mli 602

82 image/lib/io_eio.ml 603

83 image/lib/io_lwt.ml 608

84 image/lib/jobs.ml 609

85 image/lib/lineedit.ml 620

86 image/lib/llm_client.ml 621

87 image/lib/loader.ml 630

88 image/lib/media.ml 632

89 image/lib/net_prims.ml 638

90 image/lib/ops_incidents.ml 641

91 image/lib/ops_log.ml 646

92 image/lib/ops_stats.ml 649

93 image/lib/packages.ml 653

94 image/lib/primitives.ml 655

95 image/lib/prims_crypto.ml 664

96 image/lib/prims_db.ml 665

97 image/lib/prims_graph.ml 708

98 image/lib/prims_hyper.ml 712

99 image/lib/prims_media.ml 713

100 image/lib/prims_net.ml 715

101 image/lib/prims_request.ml 721

102 image/lib/prims_time.ml 723

103 image/lib/repl_endpoint.ml 726

104 image/lib/request_context.ml 728

105 image/lib/session.ml 730

106 image/lib/session_registry.ml 760

107 image/lib/session_types.ml 762

108 image/lib/store.ml 764

109 image/lib/store_db.ml 800

110 image/lib/sync.ml 803

111 image/lib/timing.ml 811

112 image/lib/tree_store.ml 814

113 image/test/agent_tools_test.ml 845

114 image/test/ai_store_test.ml 846

115 image/test/api_prim_test.ml 848

116 image/test/auth_password_test.ml 850

117 image/test/auth_throttle_test.ml 852

118 image/test/branch_generation_test.ml 854

119 image/test/branch_status_test.ml 857

120 image/test/code_history_tree_test.ml 861

121 image/test/code_restore_test.ml 863

122 image/test/diff_branches_test.ml 867

123 image/test/example_eval_test.ml 870

124 image/test/history_test.ml 875

125 image/test/http_get_test.ml 878

126 image/test/http_robust_env_test.ml 880

127 image/test/http_robust_test.ml 881

128 image/test/io__eio__test.ml 883

129 image/test/jobs__test.ml 884

130 image/test/local__first__prims__test.ml 893

131 image/test/media__test.ml 896

132 image/test/merge__promote__test.ml 898

133 image/test/ops__incidents__test.ml 902

134 image/test/packages__test.ml 903

135 image/test/primitives__test.ml 904

136 image/test/prims__hyper__test.ml 918

137 image/test/reads__as__of__gen__test.ml 922

138 image/test/sync__db__test.ml 928

139 image/test/sync__test.ml 933

140 image/test/tree__datum__test.ml 936

141 image/test/tree__edges__test.ml 942

142 image/test/tree__lens__test.ml 947

143 image/test/tree__path__prims__test.ml 950

144 image/test/tree__retention__test.ml 952

145 image/test/tree__sibling__order__test.ml 955

146 image/test/tree__store__test.ml 961

147 image/test/tree__version__shadow__test.ml 963

148 libs/__root/lib.pp 967

149 libs/ai/lib.pp 993

150 libs/astro/lib.pp 996

151 libs/bi/dashboard/lib.pp 1006

152 libs/bi/lib.pp 1007

153 libs/bi/metric/lib.pp 1013

154 libs/bible/lib.pp 1014

155 libs/book/lib.pp 1029

156 libs/bookmarks/lib.pp 1044

157 libs/calendar/lib.pp 1049

158 libs/chat/lib.pp 1064

159 libs/contacts/lib.pp 1073

160 libs/css/lib.pp 1078

161 libs/dabar/lib.pp 1080

162 libs/data/lib.pp 1087

163 libs/edit/lib.pp 1088

164 libs/err/lib.pp 1115

165 libs/flash/lib.pp 1116

166 libs/flashcards/lib.pp 1128

167 libs/food/lib.pp 1138

168 libs/forge/lib.pp 1145

169 libs/hello/lib.pp 1169

170 libs/html/lib.pp 1170

171 libs/hyper/lib.pp 1174

172 libs/kanban/lib.pp 1177

173 libs/layout/lib.pp 1186

174 libs/lens/lib.pp 1196

175 libs/list/lib.pp 1197

176 libs/menu/lib.pp 1198

177 libs/notes/capture/lib.pp 1199

178 libs/notes/doc/lib.pp 1203

179 libs/notes/issues/lib.pp 1209

180 libs/notes/legacy/lib.pp 1213

181 libs/notes/lib.pp 1214

182 libs/notes/light/lib.pp 1288

183 libs/notes/migrate/lib.pp 1292

184 libs/notes/outline/lib.pp 1294

185 libs/notes/refile/lib.pp 1301

186 libs/notes/tree/lib.pp 1306

187 libs/notes/ui/lib.pp 1316

188 libs/ntfy/lib.pp 1328

189 libs/ops/code/lib.pp 1330

190 libs/ops/lib.pp 1334

191 libs/outline/lib.pp 1337

192 libs/plan/lib.pp 1355

193 libs/rb/lib.pp 1359

194 libs/ref/lib.pp 1384

195 libs/repl/lib.pp 1386

196 libs/replication/lib.pp 1391

197 libs/roam/lib.pp 1392

198 libs/route/lib.pp 1398

199 libs/signup/lib.pp 1399

200 libs/soup/lib.pp 1402

201 libs/spec/lib.pp 1403

202 libs/std/lib.pp 1412

203 libs/strongs/lib.pp 1421

204 libs/structed/lib.pp 1423

205 libs/structed/list/lib.pp 1434

206 libs/sync/lib.pp 1436

207 libs/theme-as400/lib.pp 1437

208 libs/theme-classic/lib.pp 1439

209 libs/theme-midnight/lib.pp 1441

210 libs/theme-paper/lib.pp 1443

211 libs/theme-prickly-night/lib.pp 1445

212 libs/theme-prickly/lib.pp 1447

213 libs/theme/lib.pp 1449

214 libs/todo/db/lib.pp 1456

215 libs/todo/item/lib.pp 1458

216 libs/todo/lib.pp 1464

217 libs/todo/list/lib.pp 1500

218 libs/todo/state/lib.pp 1503

219 libs/todo/tree/lib.pp 1506

220 libs/todo/ui/lib.pp 1508

221 libs/tree/lib.pp 1513

222 libs/ui/lib.pp 1520

223 libs/verify/lib.pp 1539

224 libs/weather/lib.pp 1544

225 libs/weight/lib.pp 1548

226 libs/welcome/lib.pp 1553

227 libs/wiki/lib.pp 1563

228 libs/workout/lib.pp 1569

229 libs/workout/log/lib.pp 1607

230 lisp/bin/repl.ml 1611

231 lisp/conformance/conformance.ml 1613

232 lisp/lib/ast.ml 1616

233 lisp/lib/ast_diff.ml 1621

234 lisp/lib/complexity.ml 1625

235 lisp/lib/control.ml 1628

236 lisp/lib/effect.ml 1630

237 lisp/lib/env.ml 1636

238 lisp/lib/eval.ml 1638

239 lisp/lib/explain.ml 1657

240 lisp/lib/reader.ml 1662

241 lisp/lib/recursion.ml 1671

242 lisp/lib/route_gate.ml 1675

243 lisp/lib/special_forms.ml 1677

244 lisp/test/ast_diff_test.ml 1680

245 lisp/test/codec_test.ml 1685

246 lisp/test/conformance_test.ml 1688

247 lisp/test/effect_test.ml 1690

248 lisp/test/eval_test.ml 1693

249 lisp/test/lazy_thunk_test.ml 1713

250 lisp/test/recursion_test.ml 1715

251 lisp/test/stepping_test.ml 1717

252 llm/lib/loop.ml 1720

253 llm/lib/provider.ml 1724

254 llm/lib/tools.ml 1731

255 migrations/001_branches.sql 1733

256 migrations/002__auth.sql **1735**

257 migrations/003__data.sql **1737**

258 migrations/004__bible__ref__index.sql **1738**

259 migrations/005__ai__sessions.sql **1739**

260 migrations/006__route__visibility.sql **1740**

261 migrations/007__media.sql **1741**

262 migrations/008__agent__users.sql **1742**

263 migrations/009__tree.sql **1743**

264 migrations/010__notes__tree.sql **1751**

265 migrations/011__package__registry.sql **1752**

266 migrations/012__code__history.sql **1754**

267 migrations/013__data__user__scoping.sql **1756**

268 migrations/014__guestbook__shared.sql **1761**

269 migrations/015__jobs.sql **1763**

270 migrations/016__seed__single__source.sql **1764**

271 migrations/017__branch__generation.sql **1765**

272 migrations/018__tree__versions.sql **1768**

273 migrations/019__tree__version__shadow.sql **1770**

274 migrations/020__tree__version__echo__guard.sql **1772**

275 migrations/021__tree__versions__reconcile.sql **1774**

276 migrations/022__tree__retention.sql **1776**

277 migrations/023__data__lens.sql **1779**

278 migrations/024__branch__base__gen.sql **1783**

279 migrations/025__sibling__order.sql **1784**

280 migrations/026__tree__edges.sql **1786**

281 pure/prims__pure.ml **1787**

282 pure/prims__soup.ml **1823**

283 remote/bin/main.ml **1829**

284 remote/lib/def__cache.ml **1838**

285 remote/lib/free__vars.ml **1841**

286 remote/lib/http__transport.ml **1843**

287 remote/lib/http__ui.ml **1847**

288 remote/lib/lens__store.ml **1856**

289 remote/lib/placer.ml **1865**

290 remote/lib/proxy__rim.ml **1868**

291 remote/lib/remote.ml **1870**

292 remote/lib/surface.ml **1883**

293 remote/lib/sync__client.ml **1887**

294 remote/test/def__cache__test.ml **1891**

295 remote/test/free__vars__test.ml **1894**

296 remote/test/lens__store__test.ml **1896**

297 remote/test/loop__test.ml **1901**

298 remote/test/placer__test.ml **1905**

299 remote/test/remote__test.ml **1907**

300 spikes/jsoo__interp/main.ml **1912**

301 spikes/navbar-editor/install.pp **1916**

302 spikes/navbar-editor/lib.pp **1917**

303 spikes/navbar-editor/uninstall.pp **1920**

1 pricklypear.borg

pricklypear

A multi-user Lisp operating system in the Smalltalk tradition: OCaml image binary, Postgres object store, htmx UI. Many humans, each with session + personal garden. One binary serves all sessions (single-image.borg); the multi-user product is auth.multi-user-product; isolation is multi-user.cotenant-isolation – session + branch scoped, one process.

[status: partial]

> – *agent note:* 2026-08-02 full-book status reconcile: root stays partial while open chapter work remains (notes partials, dabar search-fts, image repl-exp/per-req, auth argon2/ssh-repl, branching schema/ast-nodes, multi-user permissions, object-model type-system, one-tree migration, migrations branching-interaction, structural-editing ast-view, formal-verification planned). borge report after pass ~149 implemented / 4 planned / ~96%. See docs/plans/borge-status-reconcile-checklist.md.

> *[verify] build: dune build @all – exit 0 (lisp + common + image); test: dune runtest – alcotest across lisp/common/image (some prim edge cases may fail; track separately); smoke: scripts/dev.sh start boots PG + single image; curl /health and /login; smoke: python3 scripts/auth_product_smoke.py against PP_HTTP_PORT*

> – *agent note:* 2026-08 drift declaration pass (BORGE-1786035030): docs/ orphans excused with reasons. These are scratch plans, contracts, and generated artifacts – not spec'd features. Declare in a chapter only if a feature grows around one.

[untracked docs/agent-ui-harness.md: agent UI testing research wiki]
[untracked docs/eio-floor-checklist.md: Eio rim transition checklist]
[untracked docs/formal-verification-plan.md: formal verification roadmap plan]
[untracked docs/function-observability-plan.md: function observability implementation plan]
[untracked docs/notes-ui-contract.md: notes UI agent contract]
[untracked docs/notes-ui-inventory.md: notes UI element inventory]
[untracked docs/one-tree.md: one-tree storage overview]
[untracked docs/p3-debt-handoff.md: phase-3 debt handoff notes]
[untracked docs/plans/borge-status-reconcile-checklist.md: status reconcile checklist scratch]
[untracked docs/plans/interlinear-wp.md: interlinear work-in-progress plan]
[untracked docs/pricklypear-book.pdf: generated book PDF artifact]
[untracked docs/runtime-effects-transition.md: runtime effects transition notes]
[untracked docs/ssh-repl-bridge.md: SSH REPL bridge design notes]

> – *agent note:* 2026-08 drift pass: scripts/ orphans excused with reasons. These are operational tooling (smoke/verify/dev/deploy/seed), not product features. Deletion candidates flagged inline for a future purge.

[untracked scripts/auth_product_smoke.py: auth product smoke; referenced by root (verify (smoke ...))]
[untracked scripts/smoke_route_visibility.py: route-visibility smoke (auth.borg route-visibility)]
[untracked scripts/verify_auth.sh: auth verification script]
[untracked scripts/smoke_notes_ui.sh: notes UI smoke]
[untracked scripts/smoke_notes_cdp.mjs: notes UI CDP smoke]

[untracked scripts/smoke_notes_selection_cdp.mjs: notes selection CDP smoke]
[untracked scripts/smoke_notes_model.py: notes model smoke]
[untracked scripts/inventory_notes_ui.py: notes UI element inventory helper]
[untracked scripts/verify_notes_tree.sh: notes tree verification]
[untracked scripts/verify-notes-inbox-issues.sh: notes inbox issue verification]
[untracked scripts/verify-wiki-llm-loop.sh: wiki LLM loop verification (notes.borg)]
[untracked scripts/smoke_dabar_interlinear.sh: dabar interlinear smoke]
[untracked scripts/smoke_flashcards.py: flashcards smoke]
[untracked scripts/smoke_data_browser.py: data browser smoke (image.borg data-browser)]
[untracked scripts/smoke_macros_rewrite.sh: macro conformance smoke]
[untracked scripts/smoke_multiuser_macros.sh: multiuser macro smoke]
[untracked scripts/verify_function_observability.sh: function-observability verification]
[untracked scripts/verify_docstring_backfill.sh: docstring backfill verification (habitat-workflow)]
[untracked scripts/verify_discovery_spine.sh: agent discovery spine verification (public-release)]
[untracked scripts/verify_habitat.sh: habitat bring-up verification]
[untracked scripts/check_io_rim.sh: Eio/Lwt rim probe (runtime transition)]
[untracked scripts/dev.sh: local dev orchestration (PG + image); not a deployed artifact]
[untracked scripts/pp-eval.sh: pp-eval CLI client wrapper for POST /api/eval]
[untracked scripts/pp_eval.py: pp-eval CLI client for POST /api/eval]
[untracked scripts/pp_sync.py: pp-sync vault/snapshot tooling for package branches]
[untracked scripts/pp_outline.py: pp-outline tooling for structural editing]
[untracked scripts/build-release-openbsd.ksh: OpenBSD release build infra (docs/openbsd-prod-branch-deploy.md)]
[untracked scripts/ship-to-vultr.ksh: OpenBSD deploy/ship infra]
[untracked scripts/post-receive-prod-hook.ksh: OpenBSD post-receive prod hook]
[untracked scripts/openbsd/run-image.ksh: OpenBSD run-image rc.d wrapper]
[untracked scripts/openbsd/pricklypear.env.example: OpenBSD env example]
[untracked scripts/seed_strongs.py: DELETION CANDIDATE: one-off Strong's seed, data already loaded]
[untracked scripts/import_obsidian_notes.py: DELETION CANDIDATE: one-off Obsidian import]

> – *agent note:* 2026-08 drift pass: libs/ orphans. 6 product subpackages declared in their chapter implements (notes capture/migrate/tree, workout/log, structed/list, strongs). The remaining 26 are experimental/unDeparted product packages or Nopales stdlib – no dedicated spec chapter yet. Experimental packages are candidates for promotion (write a chapter) or removal.

[untracked libs/_root/lib.pp: root _package, no chapter]
[untracked libs/astro/lib.pp: experimental product package, no chapter yet]
[untracked libs/bi/lib.pp: experimental BI/analytics package, no chapter yet]
[untracked libs/bi/dashboard/lib.pp: experimental BI dashboard package, no chapter yet]
[untracked libs/bi/metric/lib.pp: experimental BI metric package, no chapter yet]

[untracked libs/bookmarks/lib.pp: experimental product package, no chapter yet]

[untracked libs/contacts/lib.pp: experimental product package, no chapter yet]

[untracked libs/css/lib.pp: Nopales stdlib: CSS helpers, no dedicated chapter]

[untracked libs/data/lib.pp: Nopales stdlib: data helpers, no dedicated chapter]

[untracked libs/edit/lib.pp: Nopales stdlib: edit helpers, no dedicated chapter]

[untracked libs/flash/lib.pp: experimental package (distinct from flashcards), no chapter yet]

[untracked libs/hello/lib.pp: DELETION CANDIDATE: demo package, not a product feature]

[untracked libs/html/lib.pp: Nopales stdlib: HTML helpers, no dedicated chapter]

[untracked libs/list/lib.pp: Nopales stdlib: list helpers, no dedicated chapter]

[untracked libs/menu/lib.pp: Nopales stdlib: menu helpers, no dedicated chapter]

[untracked libs/ops/lib.pp: Nopales stdlib: ops helpers, no dedicated chapter]

[untracked libs/rb/lib.pp: root-beer recipe DSL / business model, no chapter yet]

[untracked libs/ref/lib.pp: Nopales stdlib: ref helpers, no dedicated chapter]

[untracked libs/signup/lib.pp: signup package; covered partly by multi-user, no dedicated chapter]

[untracked libs/spec/lib.pp: Nopales stdlib: spec helpers, no dedicated chapter]

[untracked libs/todo/item/lib.pp: todo package; partly covered by notes unified-todos, no dedicated chapter]

[untracked libs/todo/list/lib.pp: todo package; partly covered by notes unified-todos, no dedicated chapter]

[untracked libs/todo/state/lib.pp: todo package; partly covered by notes unified-todos, no dedicated chapter]

[untracked libs/todo/ui/lib.pp: todo package; partly covered by notes unified-todos, no dedicated chapter]

[untracked libs/verify/lib.pp: Nopales stdlib: verify helpers, no dedicated chapter]

[untracked libs/weather/lib.pp: experimental product package, no chapter yet]

[untracked libs/weight/lib.pp: experimental product package, no chapter yet]

> – *roerick:* An introduction: A little about me:
>
> I'm super excited about what has emerged with Pricklypear. Pricklypear is the result of a kind of long, meandering journey. I see a lot of these cactus on my hike. In the middle of the desert, they are common, they are prickly, and they bear delicious fruit. I've also been told you can use them to harvest water kefir yeast, but I've yet to confirm this.
>
> I quit my development job and now work at a dentist office and a restaurant. But I just can't seem to stop coding. For a while I was trying to make the ultimate "CRUDmonkey" platform. Basically, I spent a lot of time with half baked designs trying to reinvent web frameworks. Then, I became totally taken with the idea of a tree-based structure editor.
>
> My background is largely as a python CRUDmonkey, hence my interest in reinventing frameworks. I did a lot of data processing elbow grease work. I was CTO of a 13 person startup for a while.

When I left that job I was pretty burnt out. I didn't think I enjoyed programming, but I still was obsessed with the idea of maybe I could make it easier.

Then I found OCaml. Functional programming did it for me, man. It was like learning to program all over again. I started learning about all the things I had basically ignored as a python guy. I learned about compilers, low level languages. I took a freelance gig doing static analysis on IBM HLASM assembly language.

The only thing I really wanted from code? I wanted to build my dream zettelkasten. I just wanted a good notes app for my bible study. First I built a bible mode in Obsidian with markdown files. But as soon as I tried to get Strong's Concordance in there, I absolutely hit the limits of what Obsidian could do. Then, I decided to switch over to elisp. LLMs were barely functional at this time, and they were really bad at lisp. This was the "copy and paste from ChatGPT" era. My emacs / org-roam zettelkasten worked amazingly. Honestly, I love emacs and I love org-roam. Absolutely beautiful pieces of software. I also love the sense of calm that I get with emacs software. I pulled up this bible app in emacs 4 years later and it just... worked. Amazingly. None of the constant churn of the JS ecosystem.

OK, so then I got into OCaml, and I was very anti LLM. They didn't work, they made you dumb, they were a scam, yada yada yada. So I built myself a bible app, all by hand, in OCaml / Dream. I had a great time with this. I felt like I probably learned more about Dream than I did OCaml, but that's how I felt when I first learned Python also.

Then I took a gig with some guys who were doing IBM mainframe LLM stuff. I started coding in TypeScript (which i had never really used), right around the time when the LLMs got really, really good. Like November of 2025? I saw a massive increase in their ability to compose and build code.

Then I went deep. I started building harnesses, orchestrators, I did the full LLM psychosis thing. I eventually settled on my Borge architecture. Without going too deeply into it, it's a DSL for defining spec and a series of CLI tools to get agents to implement, review, and build out that spec. I'm not sure it's any more efficient but it seems to work super well for me.

So then one day I had the brainwave. Can I make an interpreted lisp where the functions are stored in postgres? The agent assured me I was a genius yet again, but I had a good feeling about this one. I had the LLM draw out my Borge spec, and then started a ralph loop. I think it went for like, 25 iterations? It ran for 4 days and it crashed a couple times and I think I maxed out three different providers. It used a lot of tokens. I got an email from one of my LLM providers the next day letting me know they were tripling their costs.

When I started playing with it, I actually couldn't figure out how to use it at first. I had the agent draw up a intro guide. Then I realized that not only did it work, I could write language functions to the database, but that also those functions were working as web endpoints. I realized I had a whole framework.

It worked so well. I built a couple apps on top, it was effortless. I plugged an LLM into the system. Once I prompted it to realize Pricklypear was not a cactus in this scenario, it completely understood the harness it was in and it just... started writing functions.

I couldn't believe it. Then I gave my local LLM access to the full REPL, and I realized I wouldn't ever have to build an API or CLI interface for agents ever again.

I'm delighted by this. I have wanted a Smalltalk type environment since I learned about what that was. But by incredible, happy accident, I also got my brand spanking new web framework, and I also got my tree based editor. I honestly feel, I don't need any more software. I'd be good. If I had my druthers, I would just work on this in private and read Functional Programming papers and Alonzo Church in my hammock for the next 5 years while I dogfood this thing. But, you know, "building in public" and all that.

I'm hoping some people are as thrilled and inspired by this as I am. Maybe the code is useful, or maybe just hearing about the concept is enough to inspire people. Or maybe, and this may be more likely, nobody will give a crap. I don't know where LLMs will go in the future, but I am really happy right here.

I'll let the agent describe the system here for a bit, and then talk about why I think this system is really, really good.

– *roerick:* Alright - I'll talk a little bit about what I think Pricklypear is good at, and where I see it going

A lot of people have talked about how incredible Lisp is. Personally, I'm not totally sold. Don't get me wrong, lisp is wonderful. But I think for larger projects you just need static typing. I didn't like static typing for a long time, I totally get it. But for medium or big projects, I just don't know how you do big refactors without it.

That said, I think Lisp is like Ruby a lot, it's a beautiful and expressive language. So this is a lisp machine. And I think it's good for the same types of programs that Ruby is good at. So I did my best to clone Basecamp, hah.

Having an agent work with a REPL is incredible. I mean, it's spectacular. They can define functions. They can read and execute individual functions. They can build functions inside out or bottom up. They can build them top down. They can SSH in remotely and access the REPL while they're developing. It's pretty tight.

I don't think any other architecture can match Pricklypear for building web apps, and I honestly don't think it's even close. All of those LLM marketing idiots telling you about on-demand self healing software? Hey, this is it. It's right here. I don't think you can do this with any other stack? Maybe like erlang or something.

The future I see for Pricklypear or something like it is enabling LLMs to build on demand software for small and medium businesses. I don't even think you need to have a Pricklypear guy on staff? I mean you could, it'd probably help. But I think literally users can ask for features and they will be deployed. I've had some trouble getting the built-in agent to build out like a whole app from start to finish, but it can certainly add features. And that loop is closing fast.

I'm biased and deep in AI psychosis, but I'm telling you this is a much bigger leap than OpenClaw. Oh, you put your little crab agent inside a chat bot? Guess what? Mine lives inside a language which he is developing himself, and that language is also a fully baked web app. (Sorry, OpenClaw is brilliant, but I had to. You'll understand when you login the first time.)

Let's talk roadmap. I'm going to dogfood this thing for a while. Maybe some other people will do something similar. I don't really feel any obligation to build anything for anybody else. Get your own! But, I would like to get this thing to the point where it's functioning as a self contained unit, and like, my dad could hop on it and he wouldn't have to learn anything about Lisp Machines in order to use it. I think this is feasible. It's just a weird little platform to build apps on.

I'm hoping to get those apps fleshed out a bit. This is my new homelab center. I'm moving my life onto this thing. It's too good not to. So I'd like for those apps to feel really really good. Currently, they do not feel great. They are not cozy. I want calm, and I want cozy. Even though I can change these apps every day - I want to run them without changes for 5 years.

I initially spec'd this out to run on MirageOS, but we've scrapped that for now. I would like to make this happen eventually. For now, I remain an OpenBSD guy. If you want Linux compatibility, make your agent do it. (I use Arch for development, btw). So I think self contained image is a priority.

I would like to get some academic grounding on building this Lisp out. I don't know nothin about languages. Seriously. I just added floats to this thing. Also I think the language itself is called Nopales, but that isn't reflected anywhere right now. There is some sort of rudimentary effects system. I have no idea how it works. I don't really understand the subtleties of async, and the OCaml landscape is pretty confusing.

On a similar note, I'd like to get some education on formal proofs in software. I think that it is absolutely work spending some time trying to add formal verification to the OCaml outershell of Pricklypear. Lisp gives a great surface for the LLM on top, and I'm perfectly happy with a little jank there, but the core really should be incredibly stable.

I want to really polish up the node editor and the code editor. It should be a first class experience. Keyboard shortcuts, node navigation. I want it to feel as good as vim.

I'd like to see local-first LLMs, too. You really don't need much here. And the way it's designed, it would plug right into a local agent. That local agent really wants to generate his own lisp functions using a recursive bottom up function generator call!

OK, that's enough for now. This is about as much sharing as I do, generally speaking. I guess I could get on Twitter and try to shill this thing but I think I'd rather just build it instead.

philosophy

Book chapter – full text inlined from borge/philosophy.borg

[inlines borge/philosophy.borg]

single-image

Book chapter – one unix image process. No supervisor, no authd, no Mirage/Solo5 in v1.

[inlines borge/single-image.borg]

object-model

Book chapter – full text inlined from borge/object-model.borg
[inlines borge/object-model.borg]

one-tree

Book chapter – ONE BIG TREE storage substrate + typed nodes; full text inlined from borge/one-tree.borg
[inlines borge/one-tree.borg]

tree-time

Book chapter – the generation axis on the one tree: versioned leaves keyed (branch, path, gen); history/undo/as-of reads become storage semantics. Full text inlined from borge/tree-time.borg
[inlines borge/tree-time.borg]

branching

Book chapter – full text inlined from borge/branching.borg
[inlines borge/branching.borg]

lisp-interpreter

Book chapter – full text inlined from borge/lisp-interpreter.borg
[inlines borge/lisp-interpreter.borg]

agent

Book chapter – local Nopales-native coding agent (pi-inspired sibling, not the image); full text inlined from borge/agent.borg
[inlines borge/agent.borg]

remote-eval

Book chapter – Nopales interprets in a client process while the corpus + data plane stay authoritative in Postgres (the Prim_call seam agent.borg uses, relaxed across HTTP); full text inlined from borge/remote-eval.borg
[inlines borge/remote-eval.borg]

local-first

Book chapter – a local lens is a branch of the one tree whose root lives on the client (the pad detaches, roots independently, grafts back). Revisits remote-eval's 'no second store' hard-out: a lens is a governed branch with merge semantics, not an uncoordinated copy. Same Prim_call seam, same tree, offline-capable. Full text inlined from borge/local-first.borg
[inlines borge/local-first.borg]

storage

Book chapter – full text inlined from borge/storage.borg
[inlines borge/storage.borg]

multi-user

Book chapter – full text inlined from borge/multi-user.borg
[inlines borge/multi-user.borg]

auth

Book chapter – full text inlined from borge/auth.borg
[inlines borge/auth.borg]

image

Book chapter – full text inlined from borge/image.borg
[inlines borge/image.borg]

multipart

Book chapter – multipart/mixed streaming responses (one response, many targeted htmx swaps) + the advanced REPL built on it. Full text inlined from borge/multipart.borg.
[inlines borge/multipart.borg]

hypermedia-harness

Book chapter – a headless hypermedia client: a server-side htmx swap simulator that is the reference implementation of the fixi-five contract, for debug agents and verification harnesses. Full text inlined from borge/hypermedia-harness.borg.
[inlines borge/hypermedia-harness.borg]

product-experience

Book chapter – full text inlined from borge/product-experience.borg
[inlines borge/product-experience.borg]

notes

Book chapter – outline-first notes (org/roam/obsidian-shaped); full text inlined from borge/notes.borg
[inlines borge/notes.borg]

structural-editing

Book chapter – structural editing as one shared tree-op protocol (outlines + code ASTs); full text inlined from borge/structural-editing.borg
[inlines borge/structural-editing.borg]

calendar

Book chapter – month/day calendar; full text inlined from borge/calendar.borg
[inlines borge/calendar.borg]

flashcards

Book chapter – flashcards / spaced repetition; full text inlined from borge/flashcards.borg
[inlines borge/flashcards.borg]

workout

Book chapter – workout journal (fluid program docs + queryable facts); full text inlined from borge/workout.borg
[inlines borge/workout.borg]

dabar

Book chapter – BSB study libraries (dabar); full text inlined from borge/dabar.borg
[inlines borge/dabar.borg]

notify

Book chapter – getting events out of the image to a human: ntfy server-push from jobs schedules v1, in-habitat feed deferred, delivery-path decisions recorded; full text inlined from borge/notify.borg
[inlines borge/notify.borg]

migrations

Book chapter – full text inlined from borge/migrations.borg
[inlines borge/migrations.borg]

formal-verification

Book chapter – formal verification of the OCaml outer shell (QCheck foothold ? proofs later). Full text inlined from borge/formal-verification.borg.
[inlines borge/formal-verification.borg]

ai-authoring

Book chapter – full text inlined from borge/ai-authoring.borg
[inlines borge/ai-authoring.borg]

film-analysis

Book chapter – film analysis as habitat data: shot/scene structure with frame-exact timecodes, chiastic (ring) derivation + linting, cuts as branches. Full text inlined from borge/film-analysis.borg.
[inlines borge/film-analysis.borg]

film-edit

Book chapter – programmatic video editing: studio library (characters/assets), Nopales-authored EDLs, Blender VSE as headless backend, render caching, gen-API bridge. Full text inlined from borge/film-edit.borg.
[inlines borge/film-edit.borg]

jobs

Book chapter – durable background jobs, step checkpoints, and cron: jobs as typed data nodes, a host-owned runner fiber, DBOS-style step checkpoints via with-transaction, guests see rows not fibers. Full text inlined from borge/jobs.borg.
[inlines borge/jobs.borg]

forge

Book chapter – executable spec graphs: borge's discipline as habitat data. LLM drafters fill leaves bottom-up along a computed frontier; mechanical checkers flag anomalies; the human gates each level. Full text inlined from borge/forge.borg.
[inlines borge/forge.borg]

habitat-workflow

Book chapter – spec-in-the-image: the borge discipline as a habitat-native workflow; promote is the goalpost- immobility gate. Full text inlined from borge/habitat-workflow.borg.

[inlines borge/habitat-workflow.borg]

habitat-book

Book chapter – the spec IS the book: borg chapters rendered to paper (borge) and to the habitat (Nopales 'book' package), with live evaluable code examples and course compositions. Full text inlined from borge/habitat-book.borg.

[inlines borge/habitat-book.borg]

2 borge/philosophy.borg

pricklypear-philosophy

Pricklypear book chapter: philosophy
[status: implemented]

philosophy

Design principles for the single-image habitat
[status: implemented]

> *[verify] build: dune build @all; smoke: one image process + Postgres object store + htmx UI*

These principles describe the system as it runs today: one unix image binary, Postgres as durable store, Nopales product shell, multi-user via sessions and branches.

Historical multi-process / Mirage-fleet language lived here in early drafts; that topology is retired. Solo5 and process isolation remain optional later hardening (see spikes/), not v1 product claims.

1. One live image. The running OCaml process is the OS surface: HTTP, REPL, eval, auth. No supervisor fleet or authd sidecar in the hot path. Rebuild from git + Postgres rows; do not treat the process as the disk.

2. Postgres is the filesystem. All durable state is rows (function_defs, type_defs, view_bindings, data_rows, auth, AI sessions, ?). The image is ephemeral and rebuildable from the store + package seed.

3. Lisp-1, homoiconic. The AST is the source. Functions persist as JSONB. Editing the AST is editing the code. Closures, let, lambda, application are first- class. Macros (defmacro + quasiquote) are specified under lisp-interpreter.macros – unhygienic v1 with gensym; expand at use and at define/load; store source AST with macro call sites.

4. Two-tier change model. OCaml (primitives, HTTP, store, auth gate) changes slowly via rebuild/deploy. Nopales (user functions AND product environment UI – nav, home, routes, theme) changes fast via row updates. OCaml must not ship a silent second product skin; host/gate/emergency only (image.system-browser).

5. Branches, not namespaces. system/main is the package baseline; each user has username/personal as an inheriting garden. Code branches; shared data. Fork is cheap; merge/promote is the careful operation.

6. Multi-user by session and branch. Isolation is not per-process unikernels in v1. Many humans share one image; identity + branch scope their code world. Stronger process isolation is a later product choice.

7. Author-runs-as-author. A function executes with its author's data permissions where that model applies – honest setuid-shaped provenance, not caller-spoofing.

8. SERIALIZABLE by default. Mutating store paths default to SERIALIZABLE transactions. Weaker isolation is opt-in; most Nopales never sees the dial.

9. Additive-only schema evolution. Types and migrations grow; they do not casually drop or rename. Cruft may accumulate; correctness and simple deploys matter more.

10. Inspectability is non-negotiable. Functions, types, branches, routes, rows, and sessions are queryable and renderable in the htmx UI (and via /api/eval). The store must stay the most inspectable layer.

3 borge/single-image.borg

pricklypear-single-image

Book chapter: the single-image process model. One unix binary, one Postgres, one deployment. Multi-user at the session/branch layer, not process isolation. This chapter is the authority for process topology; an earlier multi-process design (supervisor + authd + per-user unikernels) was retired.

[status: implemented]

single-image

One unix image process: model, topology, auth placement, deployment

[status: implemented]

thesis

What the system IS

[status: implemented]

Pricklypear is ONE OCaml unix binary – the image – that:

1. Serves htmx HTTP directly on a port (Httpun_eio + pure-html). This is the only user-facing surface. 2. Talks to Postgres via pure pgx over Eio.Net (Io_eio). Postgres is the durable object store; the image is ephemeral and rebuildable from the store plus the package seed (libs/*.pp -> system/main). 3. Reads its identity (branch, author) from process env at boot – no boot manifest, no external injection. 4. Optionally exposes a line-protocol REPL on a second port for operator access. The browser REPL (/repl) and HTTP eval (/api/eval) are the primary interactive surfaces.

Scope – what the system is NOT:

- Not a MirageOS unikernel. No Solo5 target in v1. ‘dune build‘ + restart is how the image changes. (Solo5 remains an optional later hardening spike – spikes/mirage_solo5/ – not the product path.) - Not multi-process. No supervisor, no authd sidecar, no fleet. The image is the only Pricklypear process. - Isolation between humans lives at the session and branch layer, not the process layer. There is no per-user image instance.

Rationale: one process cannot fail the ways a coordinated multi-process topology can – no port races on respawn, no stale route tables, no orphaned children, no cross-process identity handoff.

topology

The physical layout

[status: implemented]

OpenBSD host (build + runtime habitat):

+– Postgres (shared object store, SERIALIZABLE | defaults) | +– pricklypear image (ONE unix binary) | +– Httpun_eio on PP_HTTP_PORT (htmx UI, | | /api/eval, /repl, user routes) | +– optional line-protocol REPL on | | PP_REPL_PORT (operator only; Lwt_eio bridge) | +– Io_eio.Pg to Postgres (function_defs, | data_rows, branches, auth tables) | +– nginx (edge) +– TLS -> proxy_pass http://127.0.0.1:PP_HTTP_PORT

Flow for "user adds a function": 1. Open the browser (nginx front door). 2. /repl or the function editor: the define persists to function_defs and is registered live. 3. Routes serve it immediately.

auth

Where verify runs

[status: implemented]

Verify runs IN the image. Password and session verify happen in-process against the users, credentials, and auth_sessions tables; the /login cookie middleware guards the HTTP surface. No separate authd exists.

nginx basic-auth/htpasswd is an emergency edge gate only – it is NOT the user database.

Multi-user (many humans, sessions, personal garden branches) is a product layer above this process model; see auth.multi-user-product for the user stories.

runtime

Runtime effects: direct-style core, algebraic-effect rim, Eio floor

[status: implemented]

> *[verify] build: dune build && dune runtest; test: image/test/io_eio_test.ml; smoke: image serves /health and a CRUD roundtrip under PP_IO=eio; smoke: (with-transaction (lambda () (create ...))) commits; select-for-update inside body sees the lock, on the default backend*

Three layers – never collapse them (the two \"effects\"):

A. GUEST CAPABILITIES – what a Nopales expression may touch (pure / read-db / write-db / branch / publish / ai / html / txn). Static AST inference in lisp/lib/effect.ml; drives badges and gates. Spec: lisp-interpreter.effects. NOT this section. B. HOST CONTROL EFFECTS – how eval does I/O without monads. OCaml 5 Effect.perform of one effect: Prim_call : string * value list -> value Effect.t declared in lisp/lib/control.ml. Guest Nopales can never perform it; eval's prim dispatch is the only performer. C. I/O RUNTIME – who schedules sockets/DB/fibers. Lwt today; Eio is the floor we are moving to. The power company; chosen last, replaceable first.

This subsection is the book authority for layers B and C. The phase plan and runbook live in docs/runtime-effects-transition.md (H0 audit -> H1 direct-style core -> H2 Io port -> H3 backend swap); that doc is the runbook, this is the law.

LAYER B – HOST CONTROL EFFECTS (H1, SHIPPED):

- lisp/ is direct-style: Eval.eval returns value, no monadic types, no Lwt dependency. - Eval's prim dispatch performs Prim_call(name, args); prim bodies do NOT recurse into eval (guest lambdas needing DB access go through with-transaction's dynamic scope instead). - Pure tests install Control.run_sync with a sync prim table: service-free, no rim. - The image runs every eval under Control_rim.run (Effect.Deep.match_with): Prim_call looks up the registered prim fn, runs it, continues (or discontinues on raise). Pure lisp/ tests stay green with no services.

LAYER C – IO PORT + EIO FLOOR (SHIPPED):

- Process entry is always Eio_main + Httpun_eio accept loop. Domain SQL goes through Io.DB (io.mli) ? store_db.ml ? Io_eio.Pg (pure pgx over Eio.Net). scripts/check_io_rim.sh enforces the rim. - Store/crud/auth/session/agent_tools are direct-style: plain 'a returns, prim_fn = value list -> value, register_sync everywhere. DB calls block the calling fiber; a reentrant per-connection mutex serializes whole transactions across concurrent fibers. - HTTP page work is Fiber.fork'd off the connection read fiber so body reads (schedule_read) cannot deadlock against Promise.await. - Prim registration peeled into prims_request / prims_net / prims_media / prims_graph / prims_db; session.ml keeps t, rehydrate, checkout, eval, seed_stdlib, and register_session_control_prims. - Residual Lwt is rim-only: optional TCP REPL and curl-backed llm/http-get/sleep via Lwt_eio bridges. A later pass can replace those without touching domain code.

DYNAMIC TRANSACTION SCOPE (preserved):

- Store_db.with_dynamic_transaction binds current_tx around a thunk; crud execute paths prefer current_tx over the pool conn, so all prims in a (with-transaction (lambda () ...)) body share one SERIALIZABLE tx and select-for-update locks persist across prims in the body. Nested with-transaction fails (no

silent flattening). Binding is per-fiber under the conn mutex, never ambient-global.

RIM LINT:

- scripts/check_io_rim.sh stays green: only store_db / io_eio (and legacy io_lwt name) call pgx execute.

HARD INVARIANTS (every horizon, restated):

1. Composition: (define u (get ...)) then (str u) yields a plain value, never a promise. 2. (with-transaction ...) dynamic scope as above. 3. Budget/cancel: one bad eval cannot wedge the image (budget ticks at each eval step AND at each rim prim dispatch). 4. Pure lisp/ tests stay service-free; image smokes stay green on the default backend. 5. Capability inference (layer A) unchanged; guest Nopales never gains Effect.perform.

net-prims

Network primitives: tcp-connect + stream read/write/close over Eio.Net (egress v1)

[status: implemented]

> – *agent note:* 2026-08-05: implemented v1 egress net prims (tcp-connect, stream-write, stream-read, stream-close, stream-list) in image/lib/net_prims.ml using OCaml Unix blocking sockets (not Eio.Net – Eio switches are run-scoped with no standalone make, so persistent handles need a service-manager switch; filed BORGE-1785968883). Session stores a net_streams table of Unix.file_descr. Tested on local dev: full HTTP round-trip to example.com works (connect -> write -> read -> close). 160 dune tests green. Ingress/tcp-listen/TLS deferred to v2 (BORGE-1785967056).
>
> *[verify] build: dune build; test: dune runtest – net prims (mock/local); test: (tcp-connect host port) returns a stream handle; test: (stream-write handle bytes) writes; (stream-read handle n) reads up to n bytes; test: (stream-close handle) closes the socket*

v1 SCOPE: EGRESS ONLY. Lisp code can open outbound TCP connections and read/write byte streams. This makes MQTT/IRC/SMTP/WebSocket client libraries possible as pure Nopales, without a kernel sidecar. Ingress (tcp-listen, spawn-service, long-lived accept-loop fibers) is deferred to v2 – it needs a service-manager switch outside the per-request scope and lifecycle introspection (BORGE-1785967056).

PRIMS (v1): (tcp-connect host port) -> stream-handle | error Opens an outbound TCP connection to host:port using Eio.Net.connect on a switch scoped to the handle's lifetime. Returns an opaque VStr handle id (a UUID) registered in a session-local stream table. (stream-write handle bytes) -> n-written | error Writes the byte string to the stream (Eio.Flow.write). (stream-read handle max-bytes) -> bytes-string | error Reads up to max-bytes bytes (Eio.Flow.single_read into a Cstruct, returns as VStr). One budget tick per read. (stream-close handle) -> ok Closes the socket and removes the handle from the session stream table. (stream-list) -> list of open handle ids Introspection for debugging.

CAPABILITY GATING: tcp-connect requires the Net capability (new badge; see effect.ml). Public routes may not call net prims. Per-user/branch quota on open streams (default 16) to prevent resource exhaustion. Anonymous sessions denied.

LIFECYCLE: Stream handles live in a session-local table. The switch is scoped to the eval call (so an abandoned stream closes when the eval returns) OR to the session (for multi-call stream use). v1: per-eval-call switch (simplest, safest). The handle id is a UUID string; the table maps id -> socket.

STEP-BUDGET INTERACTION: Each stream-read is one budget tick (like extract-links). stream-write is one tick. A blocked read on a silent peer

could hang an eval call; mitigate with a per-read timeout (default 30s) returning an error string, not a hang.

NON-GOALS (v1): - tcp-listen / ingress (v2, needs service-manager). - spawn-service / long-lived fibers (v2). - TLS wrap (v2; eio-tls not in poohstack yet). - UDP. - DNS resolution beyond getaddrinfo (Eio.Net handles it).

deployment

One binary, one rc.d service, one restart

[status: implemented]

rc.d service 'pricklypear' (scripts/rc.d/pricklypear): start -> /usr/local/libexec/pricklypear/current/bin/image with env from /etc/pricklypear/pricklypear.env stop -> kill the single PID restart -> stop + start (one process, one port, clean shutdown)

Release promotion (git push to prod): 1. build the release (lisp + common + image) 2. extract to /usr/local/libexec/pricklypear/releases/<id> 3. swap the 'current' symlink 4. rcctl restart pricklypear

(Full hook/runbook detail: docs/openbsd-prod-branch-deploy.md.)

Changing a primitive: edit image/lib, push, the promote step rebuilds and restarts. Changing an app: define or route on a branch in the live image – no deploy at all.

4 borge/object-model.borg

pricklypear-object-model

Pricklypear book chapter: object-model
[status: partial]

> – *agent note:* 2026-08-02 status reconcile: CRUD+deftype+dict+render-html shipped (crud.ml/prims); type-system still partial for full :ref lattice. Parents stay partial.

object-model

Typed records in Postgres, schema declared in Nopales, SQL-via-primitives, no ORM
[status: partial]

> *[verify] build: dune build @all; test: image/test/primitives_test.ml 48 alcotest cases incl. CRUD + deftype + dict + render-html; smoke: CRUD + deftype + dict + render-html exercised end to end*

An "object" is a typed record with an id, a set of named typed fields, stored in Postgres. There are NO methods on objects. Methods are functions in the store that take an object id as argument. This is functions + data, not OOP, matching the poohstack shape.

Types are DECLARED IN LISP, not in SQL migrations:

(deftype event (fields (title :string) (start :timestamptz) (end :timestamptz) (owner :ref user) (attendees :array (ref user))))

The declaration lives in a code-store table (type_defs, branched like code). It registers the type – it does NOT emit DDL. All data rows of all types live in the shared data_rows table keyed by (type_name, id), fields packed as jsonb.

The type is itself inspectable data: the htmx UI can render "list all types, show fields, browse instances" nearly free. This is the Smalltalk-image browser feeling, achieved via a metamodel rather than via live mutable objects.

CRUD primitives (real OCaml, typed at boundary): (create 'event {...}) -> row insert (get 'event id) -> row fetch (update 'event id {...}) -> row update (delete 'event id) -> row delete (rows 'event) -> all rows of a type (find-row / find-rows / find-text ...) -> lookup (select-for-update ...) -> explicit row lock

References are first-class values: a :ref user field holds an id + type tag. Dereferencing is explicit: (get 'user (event-owner ev)).

The Nopales still has plain values (int, string, bool, lambda, list, ref) for logic that doesn't touch the DB. Pure functions for "does this event conflict?" take event values, return bool, no DB. The DB and Nopales have a clean seam.

NO ORM. The primitive layer reads and writes the shared data_rows table with plain SQL and parses results into typed Nopales values.

type-system

Field types and reference semantics
[status: partial]

Field types (initial cut, grow as needed): :string text :int int8 :bool bool :timestamptz timestamptz :ref <type> id of a data_rows row of <type> :array <type> <type>[] (homogeneous) :json jsonb (schemaless escape hatch)

Field values are stored in the data_rows.fields jsonb – there are no per-type tables (see storage.schema).

A (ref user) value is a tagged id: the value carries its target type so the interpreter can verify derefs at call time. Dereferencing returns the full record or a stub-ref that lazy-loads on field access (decision deferred; start eager).

Cycles in the type graph are allowed (user has :ref calendar; calendar has :ref user) because refs hold ids, not nested values.

No inheritance, no subtyping. Composition via references, not via type hierarchies. This matches the poohstack (relational, no ORM, no inheritance) and avoids the Postgres INHERITS trap (see storage.no-inheritance).

– *agent note:* one-tree P6: boundary validation + type-fields/node-path/backlinks prims land the declared-field half; :ref as path + full field-type lattice still open.

– *agent note:* 2026-08-02 confirm still partial: deftype+type-fields+Tree_store.validate_fields exist; typed :ref/:array lattice and setuid-style ref deref incomplete.

5 borge/one-tree.borg

pricklypear-one-tree

Pricklypear book chapter: one-tree – ONE BIG TREE as the storage substrate, typed nodes for data modelling
[status: partial]

– *agent note:* 2026-08-02 status reconcile: tree-model/typed-nodes/projections/non-goals implemented; branching-interaction+migration-plan still partial (shared /data A, full tree-diff merge, phase-3 retire tables).

one-tree

One tree is the storage truth. Tables become projections. Nodes carry types.
[status: partial]

[verify] build: dune build @all; test: image/test/tree_store_test.ml – 7 alcotest cases; smoke: ksh scripts/verify_one_tree.sh (tree_verify + eval smoke on :18080)

========== THE IDEA ==========
Everything in Pricklypear is stored in ONE BIG TREE. The tree is the source of truth. Code, types, routes, data, docs – all of it lives as nodes in one rooted, typed, path-addressed tree. Nopales code IS tree content: a function's AST is a subtree.

Today the image already simulates this tree on top of relational tables: branches/function_defs/type_defs/ view_bindings reconstruct hierarchy via branch_id columns, parent pointers, and recursive-CTE override walks. Merge and promote are tree-diff algorithms written in SQL. The relational layer is scaffolding around what is already a tree. This chapter removes the scaffolding.

This is the Smalltalk/GemStone lineage Pricklypear comes from: the persistent thing is the object graph, and queries are programs over the graph. One unix image process serializes structural mutation, so the classic one-tree-store objection (concurrent writers) mostly evaporates – one writer, and a "transaction" is: evaluate, commit the new root.

WHAT CHANGES: - The tree is TRUTH. Rows are DERIVED. - Storage addressing is by PATH, not by (table, branch_id, name). - Merge/promote become honest three-way tree diffs instead of row-level branch inheritance. WHAT DOES NOT CHANGE: - The Nopales prim ABI. get/create/update/delete/ rows/list-bindings/describe/deftype keep their signatures. The prim/effect seam – the core performs I/O via Control.perform_prim (OCaml 5 effects; see runtime-effects-transition) and the rim handler answers through the Io.DB port – was built for exactly this swap. Nopales code and route handlers never see the difference. Lwt remains only as the v1 rim backend (Io_lwt); the direction is effects everywhere, Lwt is the replaceable power company, not the model. - Postgres remains the physical substrate (byte durability, backups, pg_dump). The tree is a Postgres-RESIDENT structure, not a new database. - additive-only migrations still govern whatever physical tables the tree store itself uses. - The effects direction (runtime-effects-transition): one-tree changes WHAT the rim reads/writes, not HOW the core talks to the rim. Core stays direct-style via Control.perform_prim; any Lwt still present is vestigial rim plumbing (Io_lwt), to be replaced (Eio or other) on the H3 schedule – it is NOT the model and should not grow.

tree-model

The tree: one root, typed nodes, path addressing
[status: implemented]

ONE TREE, ONE ROOT:

/ ??? sys ? ??? pkg/<package>/<name> function AST subtrees ? ??? type/<type> type declarations ? ??? route/<route> route ? handler bindings ??? user/<username>/... same shape per user ??? data ??? u.<user>.<type>/<id> per-user data nodes ??? shared.<type>/<id> shared/seed content (see branching-interaction)

BRANCHES ARE LENSES, NOT COPIES. A branch is a name ? subtree-root pointer into the one tree (system/main, <user>/personal, <user>/dev are views). Fork = record a new pointer. Override = pin a subtree. Unpinned names resolve through the lens to main. This replaces today's "recursive CTE up the parent chain" with pointer chasing in the structure the data actually has.

PHYSICAL FORM (v1, Postgres-resident): - Tree nodes serialized as JSONB (Nopales ASTs are already JSONB today – this formalizes it). - Path index: materialized path per node (ltree or text path column) so "give me all of /data/todo" and "give me subtree /sys/pkg/notes" are indexed lookups, not tree walks. - Per-subtree hashes (content-addressing later; detection + vault integrity now). A corrupt root must be detectable, and pp-sync snapshot must stay meaningful.

ADDRESSES ARE STABLE. A node address (/user/alice/pkg/notes/doc/get, /data/flash-card/<id>) is the durable name other nodes reference. Moves go through an explicit refile op so references can be rewritten (see notes links-and-roam refile, and structural-editing for the shared tree-op protocol this plugs into).

========== PINNED (P0) – concrete v1 decisions ==========

The tree is realized over TODAY's tables first; each existing row maps to exactly ONE typed node. Paths are DERIVED from row identity (no behavior change):

PATH GRAMMAR (branch-aware code/types/routes/packages): fn b.<branch_id>.fn.<name> type b.<branch_id>.type.<name> route b.<branch_id>.route.<route> pkg b.<branch_id>.pkg.<name> (* agent note (|pkg added 2026-08-02 for lisp-interpreter.package-surface Phase 1 – package registry rows (enabled bit + export list) are branch-lensed like fn/type/route so per-user enable/disable inherits parent->child via the same override walk.|) *) DATA (USER-OWNED – see branching-interaction /data DECISION, revised 2026-08-02): data is owned by USERS, not branches. A datum's owner is the session user at create time: datum data.u.<user>.<type_name>.<id> shared data.shared.<type_name>.<id> No branch id in the path, no lens walk, no inheritance – a session sees exactly its own data.u.<user>.* plus data.shared.*. Branches stay purely a CODE concept (branching.borg's "branches are for code, not data" stands). Fork/merge/promote never touch datum nodes, so there are no data carve-outs in those semantics. (Legacy pre-revision paths were data.<type_name>.<id>, one shared pile – see the migration note in branching-interaction.) NOTES HIERARCHY (notes.storage – NOT kind=datum): folder data.note.u.<user>.<folder-segs?> note data.note.u.<user>.?.<slug> heading data.note.u.<user>.?.<slug>.h.<nid> Reverse-mirror ignores non-datum kinds; hierarchical notes must never be written as kind=datum. The notes garden keyed data by USER (u.<user>) from the start – user-owned datum placement generalizes that exact key to every flat CRUD type (todo, flash-card, ?). META (system, branch-free): user/cred/session/media meta.<kind>.<id>

Branch inheritance is NOT baked into the path. The override-walk (recursive CTE up branches.parent_id, depth ASC, first-occurrence dedup) is REPLACED at read time by: try path under self, then parent, up the chain – first hit wins. Rehydrate loads ancestors-first (depth DESC) so child pins override by re-define. Same semantics, honest representation.

NODE ENVELOPE (jsonb payload per node): { v: 1, kind: fn|type|route|datum|user|cred|session|media, path: text (materialized), type: text (datum: type_name; else node class),

body: json (fn AST / type fields / route {handler,visibility} / datum fields), author: text, doc: text|null (fn only), hash: hex (sha256 over canonical body) } Hash covers ONLY canonical body (not path/author) so content-corruption is detectable independent of placement. Per-subtree root hashes compose from leaf hashes (content-addressing later).

TIMESTAMPS: created_at and updated_at are ROW COLUMNS on tree_nodes (see physical layout), NOT duplicated into the payload jsonb. They are the source of truth for when a node was inserted and last rewritten; the upsert ON CONFLICT path advances updated_at = now() and leaves created_at untouched. The tree-prim JSON SURFACE (node_to_json – what tree/get, tree/list, tree/get-uuid, tree/find-by, tree/list-kind return) MUST include created_at and updated_at alongside the envelope fields, read from the row columns. A node returned by any tree prim without these two keys is a defect.

INDEXED READ PRIMS (tree-model PINNED, BORGE-1785516998): two dedicated read prims drive the path index so subtree access is a ltree GiST index scan, never a full tree walk, regardless of how large a subtree grows (projections.budget-scale: an O(subtree) walk does not scale as a subtree grows): (tree/children-of path [limit]) -> DIRECT CHILD nodes of path via the materialized path index (the same nlevel(path)=nlevel(prefix)+1 selection list_children uses, which also pulls folder/note grandchildren under the hierarchy at depth+2). Budget-safe; returns node JSON list. (tree/subtree path [limit]) -> CAPPED subtree payload of nodes under path via the path index (all descendants), ordered by path (depth-first). [limit] caps the result (default matches the existing list_subtree cap); pass an explicit [limit] for a tighter bound. Returns node JSON list. Both are READ-ONLY, kind-agnostic (they return datum and hierarchical nodes alike), and share the SAME node JSON surface (node_to_json) as tree/get + tree/list, so created_at/updated_at are present by construction. tree/list with depth=1 remains equivalent to tree/children-of, and tree/list without depth to tree/subtree without limit; these dedicated prims exist to make the indexed-read intent explicit and to give subtree a caller-supplied cap.

LENS-AWARE READ PRIMS (tree-read-prims, BORGE-1785516993): the plain prims above address nodes by their FULL materialized path. The store-tree ROOTS the tree-browser v2 needs (/sys/... + branch lenses) require resolving logical names through the branch OVERRIDE CHAIN – a session on <user>/personal must see the EFFECTIVE definition for tree_nodes, same closest-ancestor-wins rule function_defs use. These take a RELATIVE logical suffix ("fn.<name>", "type.<name>", "route.<route>", "pkg.<name>") and resolve it across the session's lens chain (self -> parent -> ... -> system/main): (tree/lens-get suffix) -> the closest node across the chain (node JSON) or empty when none has it. (tree/lens-children suffix [limit]) -> DIRECT children of the effective node's subtree under the branch that owns it. (tree/lens-subtree suffix [limit]) -> capped descendants of the effective node's subtree. The suffix NAME segment is label-encoded (fn_path/type_path/ route_path/pkg_path semantics); the branch id is implicit from the session chain. Same node_to_json surface (created_at/ updated_at present). Anonymous sessions resolve their effective chain via the anon/default lens.

PHYSICAL LAYOUT (migration 009_tree.sql; additive): tree_nodes(id uuid PK DEFAULT gen_random_uuid(), path ltree NOT NULL UNIQUE, kind text NOT NULL, type_name text, – datum only payload jsonb NOT NULL, hash text NOT NULL, author text NOT NULL, created_at timestamptz DEFAULT now(), updated_at timestamptz DEFAULT now()) – GIN on payload; btree (kind,type_name); – ltree GiST on path for subtree queries. tree_branches(– lens registry branch_id uuid PK REFERENCES branches(id), root_path ltree NOT NULL, – b.<branch_id> parent_id uuid, – kept in v1 head_hash text) – subtree root hash branches table stays in v1 (branch_id uuid + parent chain); tree_branches adds lens pointer + head hash. /meta and /data are NOT under any b.<id> – they sit outside the branch lenses entirely. /meta is system bookkeeping. /data is partitioned BY USER, not by

branch: data.u.<user>.* is that user's rows, data.shared.* is shared/seed content. A session sees exactly its own u.<user> subtree plus shared – visibility is a path-prefix confinement, not a lens walk, and there is no inheritance. See branching-interaction for the full rule.

LENS = branch_id + its parent chain. A read resolves a logical name by walking the chain's path prefixes. This is the v1 lens; true content-addressed pointer moves (fork = new pointer, promote = move main head) arrive in P5 once the tree is truth.

NEXT FREE MIGRATION: 009 (001..008 exist). tree_store module: image/lib/tree_store.ml.

SINGLE WRITER: the one image process serializes structural mutation. A commit is "evaluate, publish new root." No lock manager, no merge-on-write races inside a commit. Concurrency control moves up to with-transaction over tree subtrees.

> *[verify] build: dune build @all; smoke: addressing: resolve /data/todo/<id> and /sys/pkg/notes/<fn> to nodes via path index; smoke: subtree hash detects corruption of a stored node*

typed-nodes

Every node has a type; types are the data-modelling layer

[status: implemented]

Every node in the tree carries a TYPE. This realizes object-model.type-system (planned) as the tree's data-modelling layer – types are how the tree is structured, not an annotation on rows.

KINDS OF TYPED NODE: - CODE nodes – type is the AST shape (defn, deftype, route). A function IS a subtree whose root is a 'defn' node. - DATA nodes – a product type, declared in Nopales: (deftype note (fields (title :string) (body :string) (tags :array :string))) (deftype flash-card (fields (front :string) (back :string) (due :timestamptz) (deck :ref flash-deck))) (deftype flash-deck (fields (title :string) ...)) Notes, flash cards, todos, calendar events, workouts – every product's data model is a set of typed node shapes under /data/<type>/. - SCHEMA nodes – the type declarations themselves live in the tree under /sys/type/, so a type is inspectable data like everything else (the Smalltalk metamodel-browser feeling).

FIELD TYPES: as object-model.type-system – :string :int :bool :timestamptz :json :ref <type> ? a PATH reference to a typed node :array <type> ? homogeneous In one-tree, a :ref is literally a tree path, not an opaque id – dereferencing is a path lookup, and the tree can answer "what points at this node" (backlinks) from the path index. This is the query roam wants.

VALIDATION AT THE BOUNDARY: the prim layer checks a node's fields against its type declaration on create/update. Nopales stays dynamically typed internally (per roerick: the product surface tolerates jank); the tree boundary is where shapes are enforced. The OCaml outer shell is where correctness is proven (formal-verification).

NO INHERITANCE, NO SUBTYPING – composition via :ref, exactly as object-model.type-system already rules and storage.no-inheritance already argues. One-tree does not re-open that decision; typed nodes are flat shapes composed by reference.

RELATION TO TODAY: data_rows rows ARE these typed data nodes, packed as jsonb; type_defs rows ARE the schema nodes. One-tree does not invent new data semantics – it gives the existing semantics their natural shape and makes types load-bearing (validation, backlinks, structural ops) instead of documentary.

> *[verify] build: dune build @all; smoke: create a flash-card node missing a required field ? clean rejection naming the field; smoke: :ref deref resolves via path; backlink query lists nodes referencing a given node*

tree-datum-prims

Tree prims are a complete data API: tree/upsert, tree/move, tree/delete operate on datum nodes, not only hierarchical kinds

[status: implemented]

DATUM NODES ARE FIRST-CLASS TREE NODES (tree-model, typed-nodes). The tree prim family must therefore operate on them directly – not only on hierarchical kinds (folder|note|heading). Today tree/upsert rejects any kind outside the hierarchical set, so datum writes can only reach the tree through the create/update CRUD prims (which generate the path and id internally via tree_datum_path). That makes the tree prims an INCOMPLETE data API: a product cannot move, reorder, or re-parent a datum node with a tree action. This subsection closes that gap. Kanban (notes.borg) is the first product consumer; it moves cards between columns by tree/upsert of the card's datum node with a new status field, and reorders within a column by tree/upsert with a new position.

tree/UPSERT with kind="datum": - Accepts a caller-supplied datum path of the form data.u.<user>.<type_name>.<id> or data.shared.<type_name>.<id>. <id> is a uuid. - When the path's id label is a placeholder (empty or a sentinel), generate a fresh uuid and build the path the way insert_datum does, so a caller can create a new datum node by path without pre-allocating an id. - Runs the SAME typed-boundary validation as the create/update CRUD prims (check_type_fields over body_json against the declared type_name): a missing required field is a clean rejection naming the field. Undeclared types accept free-form JSON. - Writes the tree_node with kind=datum, type_name set, the standard envelope payload, and a body hash; ON CONFLICT (path) DO UPDATE the body (and hash, author, updated_at) so the same prim upserts and updates. - The reverse mirror (migration 009 forward trigger) keeps the data_rows projection in lockstep, so (get 't id), (rows 't), and the /data browser see the node identically to one written via create.

tree/MOVE on a datum path re-homes the datum: rewrites the path and payload.path of the node (and, for a recursive move, its subtree), and the reverse mirror syncs the projection. This is the re-parent / re-board operation. Moving a datum to a path under a different <type_name> is a type change and MUST re-run typed-boundary validation against the new type.

tree/DELETE on a datum path removes the node (and optionally its subtree when recursive); the reverse mirror removes the projection row. Already kind-agnostic in code – locked here as a datum-capable prim, not a hierarchical-only one.

tree/GET, tree/LIST, tree/FIND-BY, tree/LIST-KIND are kind-agnostic readers and already return datum nodes; this subsection locks that as the contract (a datum node is readable by path, by prefix, by body field, and by kind just like any other node).

RELATION TO THE CRUD PRIMS: create/update/delete remain valid – they are the id-generating, session-user-path-injecting convenience surface for the common "insert a new row of type T" case. tree/upsert is the PATH-EXPLICIT variant for when the caller owns the path: moves, reorders, re-parents, and products that model their data as a shaped tree rather than a flat table. The two surfaces write the same tree_nodes rows through the same mirror; they are interchangeable views of one store.

COMPOSITION SURFACE (no new prims needed for the common data operations): - FIELD UPDATE on a datum (change one field, e.g. a card's status or position): tree/get the node, dict-get its body, dict-set* the changed field(s), tree/upsert the new body at the same path. This is the same read-modify-write the todo package already uses against create/update. - ATOMIC MULTI-NODE OP (e.g. reordering N cards in a column after a drag): wrap the field updates in with-transaction so the reorder is all-or-nothing. - RE-PARENT / MOVE TO NEW BOARD: tree/move to change the path, or tree/upsert with a changed parent-id field when the product models parenthood

as a field rather than a path (the flat todo model does the latter – list_id is a field, not a path segment). The path helpers node-path, type-fields, and backlinks are the datum path-construction and reference-query surface; they are already kind-agnostic and need no change.

> *[verify] build: dune build @all; smoke: tree/upsert a datum node of a declared type with all required fields ? node appears at the given path; (get 't id) returns the same fields; smoke: tree/upsert a datum node missing a required field ? clean rejection naming the field, identical to create; smoke: tree/upsert an existing datum path with a changed field ? body updates in place; (rows 't) reflects the new value; smoke: tree/move a datum node to a new path ? (get 't id) at the new path returns it; old path is gone; projection in sync; smoke: tree/move a datum to a path under a DIFFERENT type_name ? re-runs typed-boundary validation against the new type; a missing required field rejects cleanly naming the field, the node stays put; smoke: tree/delete a datum path ? node and projection row both removed; smoke: tree/get, tree/list, tree/find-by, tree/list-kind all return datum nodes, not only hierarchical kinds; smoke: tree/get of any node returns JSON with created_at and updated_at keys; created_at is stable across an upsert rewrite, updated_at advances*

tree-read-prims

Explicit read prims over the one tree: tree/children-of for one level, tree/subtree for a capped all-descendants payload

[status: implemented]

Convenience read prims that give the materialized-path index a first-class name, instead of folding directory reads into tree/list's depth argument. Both are kind-agnostic readers over tree_nodes and share the tree-get/list JSON surface (node_to_json, including the created_at/updated_at row columns). Both are index-driven – never a full tree walk (see the CHILDREN-BY-LEVEL invariant in projections).

(tree/children-of path [limit]) -> VList of node JSON Direct children of [path], expressed as a LEVEL predicate (nlevel = nlevel(path)+1, with the outline covering at +2 under the literal label h so note outline roots are included), never a correlated NOT EXISTS anti-join. Backed by Tree_store.list_children. Semantically identical to (tree/list path 1); the explicit name documents the caller's intent and decouples it from the depth-arg convention. Default limit 50000.

(tree/subtree path [limit]) -> VList of node JSON All descendants under [path] (path <@ prefix), capped at [limit] rows (default 50000) so a huge subtree degrades to a bounded payload rather than a full walk. Backed by Tree_store.list_subtree. Semantically identical to (tree/list path) with the depth argument absent; the explicit name signals the caller wants a bounded snapshot, not a level read.

LENS-AWARE READ PRIMS (BORGE-1785516993): tree/lens-get, tree/lens-children, tree/lens-subtree resolve a RELATIVE logical suffix ("fn.<name>", "type.<name>", "route.<route>", "pkg.<name>") across the session's branch lens chain – self -> parent -> ... -> system/main, closest-ancestor-wins, the same override rule function_defs use. This is what the tree-browser v2 store-tree roots (/sys, branch lenses) need: a session on <user>/personal sees the EFFECTIVE override for tree_nodes, not just its own branch. The suffix name segment is label-encoded (fn_path/type_path/route_path/pkg_path semantics); the branch id is implicit from the chain. Backed by Tree_store.lens_get_in_chain / lens_children_in_chain / lens_subtree_in_chain over Store.branch_chain.

Anonymous (unauthenticated) reads resolve against system/main and are permitted for both prims; the nodes returned are confined to what the requesting identity's path prefix grants (data.u.<user> / shared).

> *[verify] build: dune build @all; smoke: (tree/children-of P) returns exactly the direct children of P (level predicate), including outline roots*

under P.h; smoke: (tree/subtree P) returns all descendants under P, bounded by the limit arg; smoke: both prims return node JSON including created_at and updated_at keys; smoke: tree/lens-get resolves a suffix across the branch chain with closest-wins (child override shadows parent; main effective on child); smoke: tree/lens-children/subtree read under the effective branch owning the resolved node

sibling-order

Children have a user-controlled order; the tree answers next/prev/first/last

[status: implemented]

THE GAP. ltree paths encode hierarchy, never order. Today children come back in path::text ASC (or created_at) order – an accident of the index, not a modelled fact. Every consequence of that is visible downstream: notes cannot do per-node move-up/move-down (its per-node store is "flat" partly because outline order only exists in the outline blob), no package can ask next/prev-sibling, and outline- style structures keep a second source of truth (the blob) just to carry sequence.

DESIGN. A nullable rank TEXT column on tree_nodes (fractional-indexing string, Figma/Linear style: "a0", "a1", "a1V", ...).

- NOT fractional path labels. The path is a materialized path: changing a label rewrites the subtree. Rank must be mutable without touching identity, so it lives in a column, not the path. The path is WHERE a node is; the rank is WHAT SLOT it is in. - NULL rank means unordered: ORDER BY rank NULLS LAST, path::text ASC. Existing rows need no backfill; ordering is opt-in per sibling set. - Ordering is a property of the node, so it is owned by the branch that owns the node under lens rules, and rank changes are ordinary writes: versioned, diffable, undoable (tree-time carries them for free).

PRIMS.

(tree/reorder path rank) set rank directly (tree/move-before path anchor) place before sibling anchor (tree/move-after path anchor) place after sibling anchor (tree/siblings path) ordered children of parent (tree/next path) (tree/prev path) adjacent ranked sibling (tree/insert-child parent kind type body [:after anchor])

tree/children-of and tree/subtree gain the ranked ORDER BY (rank NULLS LAST, path ASC); no signature change. Rank allocation is mid-point between neighbours, rebalance on exhaustion – a write-path detail, not visible in the ABI.

UNGATES. Per-node reorder for notes (killing the last reason for outline-as-blob), the notes/node-op move-up/move-down exclusion, and any future package with sequenced content (outline, playlist, pipeline stages) gets ordering from the tree instead of a private mechanism.

[verify] smoke: insert three children, move third before first: children-of returns new order; smoke: unranked siblings sort after ranked, path-ordered among themselves; smoke: reorder is versioned: gen-read at old gen shows old order; undo restores; smoke: child-lens override of a node owns its rank; parent rank unchanged

tree-path-prims

Kernel path algebra: one canonical set, replacing per-package string hacks

[status: implemented]

THE GAP. Path manipulation is re-implemented in Nopales by every package that touches the tree: notes/tree/parent-of, notes/tree/butlast, notes/tree/split-folder, notes/tree/join-slash, todo/tree/prefix, todo/tree/tail, tree/breadcrumb-parts. Path semantics (label encoding, segment splitting) belong in the kernel next to the ltree type they describe.

PRIMS (pure – no DB, no auth; label semantics identical to tree/path-label):

(tree/parent path) parent path or nil at root (tree/leaf path) final label (tree/segments path) VList of labels (tree/child path label) append (encodes label) (tree/ancestors path) every strict prefix, root-first

These compose with the existing read/lens prims – e.g. (tree/lens-children (tree/parent p)) is sibling navigation. They are pure string operations over the ltree textual form; invalid paths raise rather than guess.

DEPRECATION. notes/tree/parent-of &co become thin aliases to the kernel prims, then are removed in a later pass. New packages must not grow private path algebra.

NON-GOAL. Glob/pattern matching over paths (ltree ~ queries) stays admin/SQL-side for v1; the Nopales surface is constructor/accessor algebra only.

> *[verify] smoke: round-trip: (tree/child (tree/parent p) (tree/leaf p)) = p for data/b paths; smoke: label-encoding edge cases (dashes, dots, uuid labels) match tree/path-label; smoke: notes/tree/parent-of and kernel tree/parent agree on the notes corpus*

tree-edges

Links between nodes are first-class typed nodes – the graph is nodes AND edges

[status: partial]

MOTIVATION. typed-nodes already declares :ref fields as literal path references and promises the tree answers "what points at this node"; the one tree is an object graph (GemStone/Smalltalk lineage) and a graph is nodes AND edges. Today only one direction exists, and only as brute force: (backlinks needle) seq-scans every datum body for exact string equality. No out-links, no link kinds, no integrity, and the scan does not scale.

THE WEIRD PART (per roerick): a link IS a typed node. Not a wrapper type inside bodies, not a side table – an edge is a node of kind link under its source:

<source-path>.lnk.<id> kind=link, body { to, kind, label? }

Being a node, an edge inherits the whole tree stack for free: typed-boundary validation, versioning and time travel (gen-read/gen-diff/undo cover edges), authorship, branch ownership, and lens visibility.

PATH PLACEMENT. Edges are children of their SOURCE – links are content OF a node: a roam note links outward; a flash-card's deck ref is part of the card. Consequences, all intentional: - out-links is a children-by-level index read (kind filter), never a scan. - tree/move of the source carries its edges; tree/delete of the source removes them. Outgoing references die with their owner; incoming references survive at the target as dangles. - "from" is positional (the parent path), never duplicated in the body; bodies hold only to/kind/label.

DANGLING BY DESIGN. The tree is branched and lensed (branching-interaction): an edge on <user>/personal can point at a system/main node another branch later deletes or overrides. Hard foreign-key integrity would make deletion and global refactor impossible across lenses and contradicts fork freedom. So: to is branch-qualified path TEXT; resolution is a READ-TIME act against the reader's lens (effective node, closest-wins). An unresolved edge is DANGLING DATA – rendered as such, repairable, never an error state and never a write blocker. GemStone could enforce pointers because there was one object space; the lenses are deliberately many.

:ref UNIFICATION. Declared :ref fields (typed-nodes) and edge nodes are two spellings of one relation. At typed- boundary validation time a datum's :ref fields are materialized as edge nodes under the datum (edge kind from the field name), refreshed on update/delete. Free-form bodies may still contain path-valued strings – those are HINTS tooling may promote, never truth. One relation serves every graph query; the body field stays the human-writeable spelling.

PRIMS.

(link from-path to-path [kind [label]]) create edge node (from must resolve in the session lens; to is stored as given – may dangle) (unlink edge-path) tombstone the edge (out-links path) VList of edge JSON with resolution status (resolved target summary or a dangling marker) (backlinks needle [limit]) same signature; backed by edges (to = needle) UNION legacy body-string matches – the legacy half is DEPRECATED and logged

INTEGRITY AS WARN, NOT FORBID. tree/delete and tree/move report the incoming-edge count for their target; UI faces warn on non-zero. tree/move of a TARGET leaves inbound edges dangling (a repair/report surface is v2, non-goal).

> – *agent note:* 2026-08-28 make pass (tree-traversal branch): edges + prims implemented and smoked (link/unlink/out-links/backlinks, dangle-by-design, warn-not-forbid inbound reporting on tree/delete + tree/move – both now return JSON {deleted|moved, inbound_edges}, versioned edges incl. undo of unlink). PARTIAL for one reason: :ref materialization is deferred (BORGE-1787972072) because type_defs.fields are bare NAMES today – no field types, no deftype surface for :ref; that is a slice of the type-system track, not of this subsection. The edge shape and read-time resolution are independent of typed-boundary validation; materialization later is purely additive.

> *[verify] smoke: link a->b: (out-links a) lists b resolved; (backlinks <b-path>) includes the edge path; smoke: tree/move of the target: the edge reports dangling on read; textual backlink still finds it (legacy half); smoke: tree/delete of a node with 2 inbound edges reports count 2 and succeeds (warn, not forbid); smoke: edge nodes are versioned: gen-diff shows edge add; undo of unlink restores the edge; smoke: datum with a declared :ref field gains a materialized edge node; update refreshes it; delete removes it*

tree-browser

Generic /tree face over tree_nodes: navigate any subtree on the session lens

[status: implemented]

THE GAP. There is a tree/browser-shaped package (tree/*) but it is a single-document outline viewer hardwired to scripture_concepts. There is no face over the store itself. If everything is one tree, the user should be able to stand at any node and walk.

SURFACE. /tree?path=P on the session branch lens, anonymous confined to public prefixes per tree-read-prims.

- Root defaults to the identity's garden root (tree/garden-root); ?path= focuses any node. - One level via tree/lens-children, lazy-expanded through htmx partials (the notes-ui-contract lazy-load pattern, CHILDREN-BY-LEVEL index form only). - Breadcrumb from tree/ancestors; kind/type filter; ranked order from sibling-order; as-of-gen view via tree-time (?gen=) with gen-diff highlighting against head. - Node detail pane: body JSON, created/updated, author, rank, out-links/backlinks (tree-edges), edit affordance per tree/upsert auth rules.

RELATIONSHIP TO EXISTING FACES. Complements, not replaces: /data (typed-row tables), /bindings (code), /sys (admin store-tree). The existing tree/* package is renamed/merged into it later; no route changes in v1 of this subsection.

BUDGET. Every level read is one index-driven query; no whole-subtree walk is ever the page path (projections invariant). Depth-limited like tree/subtree.

> *[verify] smoke: anonymous /tree shows public prefixes only; private paths are not listable; smoke: lazy expand of one level issues exactly one chil-*

dren query; smoke: session on <user>/personal sees effective lens overrides, not just its own branch; smoke: ?gen=<old> renders the subtree as of that generation

projections

SQL over the tree: relational rows are derived, not truth

[status: implemented]

The tree is truth, but ad-hoc relational query is how you SEE it. Postgres stored procedures / views / materialized projections DERIVE (type_name, id, fields) rows from the tree:

- /data browsing, /types, admin grep-queries - reporting, pp-sync export, debugging - the existing CRUD prim RESULT SHAPES (rows returns (id . fields) pairs) are served from a projection even once the store is a tree

DEPENDENCY DIRECTION FLIPS. Today rows are truth and the tree is reconstructed by CTE; after one-tree the tree is truth and rows are a cache. Projections are REBUILDABLE – drop and regenerate from the tree at any time. Nothing authoritative lives only in a projection.

OPTION, NOT GATE: the stored-procedure projection layer is how relational tooling keeps working during and after the cutover. Nopales product code keeps calling the same prims; only the prim layer's backing moves from tables to tree + projection.

INDEXING: projections and the path index are what make "all nodes of type X" and "subtree at P" fast. A full tree walk is never the query path for product or admin surfaces.

CHILDREN-BY-LEVEL, NOT ANTI-JOIN: "direct children of P" is expressed as a LEVEL predicate – path <@ P AND nlevel(path) = nlevel(P) + 1 – never as a correlated anti-join (NOT EXISTS over an intermediate node). nlevel equality is a constant-bound predicate the ltree GiST path index can drive, so the children lookup stays an index scan regardless of how many nodes share the prefix; the anti-join form cannot use the index for its inner <@ and so degrades to O(subtree^2) as a subtree grows. Every read endpoint that lists one level of the tree (tree/list depth 1, folder expansion, the /notes index) goes through this level form. (* agent note (|Design invariant recorded 2026-08-02 from live measurement at ~3000 notes under data.note.u.admin: the NOT EXISTS anti-join in Tree_store.list_children took ~4000ms to return 11 root children (blocking /notes at ~8.6s and surfacing as budget_exceeded on dabar) while the level predicate and tree/list-kind took single-digit ms; the 100k-step eval budget is left unchanged – it is a tripwire that exposed this DB-plan bug, not the bottleneck.|) *)

OUT OF SCOPE (v1): exposing raw SQL to Nopales product code. The projection layer serves the image and admin tooling; the Nopales surface stays prims.

[verify] build: dune build @all; smoke: rows 'todo' returns the same (id . fields) pairs backed by a projection over the tree; smoke: rebuild projection from tree; diff against pre-rebuild state: identical

branching-interaction

Branches as tree lenses; the /data question; merge as tree-diff

[status: implemented]

– *agent note:* 2026-08-02 /data DECISION revised to USER-OWNED data (resolves BORGE-1785693032): datum under data.u.<user>.* + data.shared.*, RLS floor keyed on user. An earlier branch-keyed draft (b.<branch>.data.*) was rejected as a leaky abstraction – encoded user ownership in a branch path, then needed fork/merge carve-outs. Three-way tree-diff merge / promote-as-pointer-move

still incomplete vs scaffolding promote. Code still on the old shared pile – spec leads code here.

– *agent note:* 2026-08-14 DATA-BRANCHING BOUNDARY (design discussion, with local-first): single-head, user-keyed data STANDS – but it is a current-shape decision, not a hard no forever. Split MECHANISM from CONCEPT: (1) MECHANISM is SHARED – the 3-way tree-diff merge (subtree/node hashes) is one generic tool for both code lenses AND local-first data sync; do not build two merge algorithms. (2) CONCEPT is NOT branchable – data has no user-facing branch concept (no checkout, no 'dev copy of your todos', no lens-walking data). A device lens snapshot (admin/laptop) is SYN/MERGE-BASE METADATA – a replication convenience – not a sibling world; data is SPATIAL (which copy of one datum), not TEMPORAL (which version of the world). Local-first sync gets its merge base from (branch, generation) snapshot pins + node hashes, NOT from making data semantically branchable. REOPEN TRIGGER (falsifies the boundary): a real feature where a USER forks their OWN data to experiment and later merges it back (e.g. clone my notes, rewrite offline over a week, reintegrate). If that use case becomes real, user-keyed flat data creaks and this boundary gets a plan-phase re-look – with the branch-keyed rejection above as the explicit prior to beat. Near-term evidence single-head is right: local-first's first payloads (transcript appends, memory LWW) are deliberately single-head.

– *agent note:* 2026-08-23 PLAN-PHASE PROPOSAL (versioned user-keyed data): reopen the data-branching boundary recorded above. The 2026-08-14 boundary split mechanism (shared 3-way tree-diff) from concept (data not branchable) and named a reopen trigger: a user forking their own data to experiment and reintegrate. The local-first lens model IS that use case – a lens materializes a user's data subtree, edits offline, syncs back via three-way merge – and the boundary's defense ("data is spatial, not temporal") is contradicted by local-first's own sync design ("the git protocol in miniature": base/ours/theirs is a versioning act over a store claimed unversioned). The defense does not hold under the machinery built on it.

WHY THE CARVE-OUTS EXPIRED. The 2026-08-02 rejection of branch-keyed data (b.<branch>.data.*) rested on two operational costs: (1) fork must not copy the DB, and (2) deploy must not wipe data. (1) expired when fork became an empty-lens parent pointer (one-tree P5, copy-on-fork) – there is no multi-row copy to carve out. (2) was always about seed/reseed scope, not data keying: user data is never seeded, so reseed touches it regardless of keying. The conceptual defense that remains ("branches are for code, not data") is an assertion the lens model violates – a lens is a branch and a lens holds data.

PROPOSAL: VERSIONED USER-KEYED DATA. Isolation and versioning are orthogonal (the 2026-08-02 revision's own insight, applied to one axis only). Keep the user segment for isolation; add an OPT-IN lens segment for versioning. Path grammar: data.u.<user>.<type>.<id> live, single-head (UNCHANGED default) data.shared.<type>.<id> shared/seed (unchanged) data.l.<lens>.<user>.<type>.<id> a forked data lens (opt-in version)

The default (data.u.<user>.*) stays branch-free and single-head: checkout does not hide your data (the property the user-key fix bought). A deliberate fork creates a data.l.<lens>.<user>.* subtree that can diverge and merge back. Abandon = drop the lens

> subtree. Merge = three-way tree-diff (lens -> trunk), the SAME algorithm code lenses use and local-first sync already runs – one mechanism, not two.
>
> NOT the rejected branch-keying. b.<branch>.data.* keyed data by CODE branch, so checkout hid your data and fork/merge (code operations) had to carve data out. data.l.<lens>.<user>.* is a DATA-NATIVE lens namespace, user-keyed for isolation, opt-in for versioning. Code fork/merge (over b.*) never touches data.l.* unless the user explicitly forks data. The carve-out becomes: fork never touches LIVE data (data.u.*); a data lens fork is a separate, explicit operation.
>
> NOTES RECONCILIATION. The notes garden (data.note.u.<user>.?) is the legacy shape (type-before-user, predates the 2026-08-02 generalization; confirmed live on prod 2026-08-23: (tree/garden-root) -> data.note.u.admin). It migrates to data.u.<user>.note.? as part of the make – type segment standardized, heading substructure (.<slug>.h.<nid>) preserved as the type's internal hierarchy. One migration to the final shape, not two. See notes.borg personal-garden.
>
> MAKE-PASS SCOPE (what flips when accepted): the DEFERRED stanza below (726-730: "data is not versioned at all"); the non-goal at local-first.borg ("NOT re-opening branch-versioning of data"); tree-time.borg's branch-free assumption for data.* paths (data.l.* lens paths carry a generation coordinate like code branches; data.u.* and data.shared.* stay tree-wide); the notes path-grammar migration. The sync landing policy (local-first.borg) gains a third case: data-lens merge (lens -> trunk) alongside code-merge (pointer move) and live-data-sync (in-place reconcile). Conflict-resolution policy is open either way; this proposal gives it a primitive to act on (the deliberate fork) instead of only accidental cross-device divergence.
>
> – *agent note:* 2026-08-23 MAKE: v0 implemented – migration 023 (data.l namespace + RLS + trigger guard), tree_store datum_path ~lens, crud fork_data/merge_data_lens/abandon_data_lens/list_data_lenses, prims fork-data/merge-data/abandon-data/list-data-lenses. The DEFERRED stanza below is flipped. v0 merge is LWW (last-write-wins per node); three-way tree-diff merge is v1. Notes path migration (data.note.u.<user> -> data.u.<user>.note) is a separate make task.

BRANCH = POINTER INTO THE ONE TREE. Fork records a new lens; override pins a subtree; promote moves main's pointer to the lens's subtree. This is a strict simplification of today's parent_id + branch_id-row model: the semantics are IDENTICAL, the representation is honest.

MERGE / PROMOTE = THREE-WAY TREE DIFF. With ASTs as subtrees, merge diffing is structural (per-node), and last_synced becomes a root-hash comparison. This realizes branching.ast-nodes (planned) and feeds branching.change-set and review-merge: a change set IS a tree diff.

THE /data DECISION (REVISED 2026-08-02 – resolves BORGE-1785693032). The original question was: data_rows has NO branch_id – shared data (branching.data). One-tree offered two placements: A. SHARED SUBTREE – /data/<type>/<id> lives outside any branch, lenses reference it. Closest to today's semantics; data never versioned. B. VERSIONED DATA – /data lives inside each lens, fork versions data too.

REVISION: the A-vs-B framing conflated two axes. The axis the rule was WRITTEN for is BRANCH VERSIONING (a fork must not copy or wipe the DB). The axis that was MISSING is USER ISOLATION (whose rows a session

can see). Placement A chose global sharing to answer the versioning question, and in doing so silently made every user's rows readable and writable by every other user (BORGE-1785693032).

The revision closes the isolation gap on the USER axis, directly, without borrowing branch semantics: USER-OWNED DATA – a datum is owned by the USER whose session created it, keyed by that user: per-user data.u.<user>.<type_name>.<id> shared data.shared.<type_name>.<id> A session sees exactly its own data.u.<user>.* subtree plus data.shared.*. There is NO branch id in the path, NO lens walk, and NO inheritance – visibility is a plain path-prefix confinement on the owning user. (An earlier draft of this revision keyed data by BRANCH – b.<branch_id>.data.* – and let lenses inherit a parent's data. That was a bad abstraction: it encoded a USER-ownership fact in a BRANCH path, then needed carve-outs because fork/ merge/promote must not copy, diff, or move data. User-keyed paths state the rule once, honestly.)

WHY NOT BRANCH-KEYED: branches are a CODE concept (branching.borg: "branches are for code, not data"). Scoping data by branch would import fork/ merge semantics that data explicitly does not have, and then require spec text explaining why those semantics don't apply. The notes garden – the one per-user data shape already in the tree – keys by USER (data.note.u.<user>.?), not branch. User-keyed data generalizes the pattern the codebase already chose.

WHAT THIS PRESERVES (the load-bearing rules): - FORK NEVER TOUCHES DATA. There is nothing to carve out: datum paths contain no branch id, so fork/merge/promote (operations over b.<id> code lenses) structurally cannot copy, diff, or move a datum. branching.borg's "data is never versioned by branch and never wiped by deploy" stands, now by construction rather than exception. - data.shared is the home of seed/shared content. - The Nopales prim ABI is unchanged: (create ?), (rows 't), (get 't id) keep signatures; the prim layer supplies the session's USER from Request_context and confines the query to data.u.<user> + data.shared.

WHAT THIS FIXES (the multi-user story): - (rows 'note) in alice's session returns alice's notes plus shared – NEVER bob's. - The /data browser shows the session's own + shared rows, not the global table. - Cross-user reads require an explicit copy/move into data.shared or a deliberate grants mechanism – the right kind of friction. - NO inheritance of main data: a fresh user sees only data.shared (which an app seeds explicitly if it wants starter content). Apps own their seeding; the platform does not silently clone main's data into a user's view.

RELATION TO NOTES: the notes garden (data.note.u.<user>.?) already keys data by USER. User-owned datum placement GENERALIZES that exact key to flat CRUD types; the notes hierarchy keeps its own path grammar (kind=folder/note/heading, not kind=datum) and is unaffected.

ENFORCEMENT FLOOR (RLS): because Nopales is user-programmable, prim code is the primary access control but not the only one – a missed WHERE clause is a cross-user read. Postgres row-level security on tree_nodes is the backstop, and it is SIMPLE because the key is the user: one policy restricts kind='datum' rows to paths under data.u.<current_user> or data.shared, keyed off SET LOCAL pp.user established per-transaction by Store.with_transaction. No branch-ownership lookup, no inheritance to express. RLS gives deny-by-default at the single point all reads funnel through; the prim layer's path-prefix confinement is the same rule stated in app code.

IMPLEMENTED (v0, 2026-08-23 make): opt-in data lens versioning via data.l.<lens>.<user>.<type>.<id>. A deliberate fork (fork-data) copies the user's live data.u.* subtree into data.l.<lens>.<user>.*; merge-data applies LWW (last-write- wins per node) back to live and consumes the lens (the mirror trigger enforces one-path-per-id); abandon-data drops the lens without merging. The default data.u.<user>.* stays single-head and branch-free. v1 ambition: three-way tree-diff merge (base/ours/theirs) instead of LWW, and

conflict surfacing. See the 2026-08-23 PLAN-PHASE proposal above.

TYPE_DECLS: today per-branch rows, table shape shared (branching.schema-authority). Under one-tree, schema nodes at /sys/type/ follow CODE branching rules (branchable, promoted), while /data instances are user-owned. Branch code reading user data against a newer/older type decl follows the existing additive-only tolerance rule.

COTENANT ISOLATION: lenses enforce multi-user.cotenant-isolation – a session is confined to its own data.u.<user> subtree plus data.shared for DATA, and to its branch lens for CODE. This upgrades the guarantee from session+branch scoping of CODE to session+USER scoping of CODE AND DATA, expressed as path confinement and floored by the RLS policy above.

> *[verify] build: dune build @all; smoke: fork = new lens; override on lens A invisible to lens B; promote moves main pointer; smoke: user-owned /data: a note created in alice's session is NOT visible in bob's session; both see data.shared; smoke: no inheritance: a fresh user's (rows 'note) returns only data.shared content, nothing cloned from another user; smoke: RLS floor: SET LOCAL pp.user=bob; direct SQL SELECT of data.u.alice.* returns zero rows*

migration-plan

Phased cutover: shadow tree ? flip reads ? projections derive ? retire tables
[status: partial]

> – *agent note:* 2026-08-02 confirm: 009_tree + mirrors + tree_store + verify_one_tree.sh (P0-P2 territory). Phase 3 retire scaffolding tables not done; PP_TREE_* knobs remain. Added Phase 2b (data user-scoping backfill keyed off author + RLS on user) per the /data revision.

NO BIG BANG. The prim ABI is stable throughout; each phase is reversible.

PHASE 0 – ADDRESSING SCHEME (pure design; this chapter). Freeze the path grammar (/sys/?, /user/?, /data/?), node envelope (type tag + payload + hash), and the lens model. No code.

PHASE 1 – SHADOW TREE. A writer mirrors function_defs/type_defs/view_bindings/data_rows into the one tree on every commit. Tables remain truth; the tree is a derived copy. Proves addressing, path indexing, hashing – fully reversible.

PHASE 2 – FLIP READS. The rim's store backend (Io.DB impl; Io_lwt today) serves reads from the tree behind the unchanged prim ABI; tables become the write-through fallback. Diff tree-read vs table-read in CI for a soak period.

PHASE 3 – TREE IS TRUTH, ROWS ARE PROJECTIONS. Writes commit to the tree; the relational shape becomes a stored-procedure/view projection for /data, /types, admin. Retire the scaffolding tables (via additive-only migration leaving them inert, per house rule – never DROP).

PHASE 2b – DATA USER-SCOPING (the /data revision; resolves BORGE-1785693032). Independent of the read-flip, one additive migration: - New datum path roots data.u.<user>.<type>.<id> (per-user) and data.shared.<type>.<id> (seed). - Backfill: every existing data.<type>.<id> node moves to data.u.<author>.<type>.<id>, keyed off the row's author column (today's pile is effectively admin-authored, so most land under data.u.admin; content meant as seed moves to data.shared explicitly). - tree_data_rows view rewritten as a user-confined projection: rows where path is under data.u.<current_user> or data.shared – no lens walk, no UNION over a parent chain. - CRUD prims pass the session USER from Request_context into the datum path (create) and the confinement (rows/get/update/delete/find-text). - RLS policy + SET LOCAL pp.user land in Store.with_transaction; cross-user access test added to verify_one_tree.sh. The old flat data_rows

relation survives one cycle as a compat projection, then goes inert per the additive-only rule.

MIGRATIONS AS TREE TRANSFORMS. Schema change stops being DDL and becomes Nopales-level migration FUNCTIONS over subtrees (more in-spirit; real work). Physical tables the tree store uses still change only via additive-only SQL migrations.

BLAST RADIUS: per-subtree hashes + pp-sync snapshot as the vault keep corruption detectable and recoverable; a corrupt subtree is rebuildable from snapshot.

DEPENDS ON / FEEDS: - object-model.type-system (planned) – realized here as typed-nodes. - branching.ast-nodes, change-set, review-merge – tree diff is their substrate. - structural-editing – one shared tree-op protocol over outlines AND the storage tree. - notes links-and-roam – backlinks come from :ref path references. - migrations.additive-only – physical-shape rule unchanged.

> *[verify] build: dune build @all; smoke: phase 1: every commit appears in shadow tree within the same transaction; smoke: phase 2 soak: zero diffs between tree-read and table-read over CI window; smoke: phase 3: drop-write to scaffolding tables; /data + /types served from projection*

non-goals

What one-tree is NOT

[status: implemented]

- NOT a new database. Postgres is A physical substrate (the server's); the tree is Postgres-resident JSONB server-side. A local lens may use another substrate (filesystem) per local-first.borg – the prim ABI is substrate-agnostic, and a lens is a materialized subtree, not a second store. The tree's truth is logical (paths + envelopes + hashes), not physical; Postgres is how the server holds it, not what it IS. - NOT versioned user data in v1. Data is USER-OWNED (data.u.<user>.*) or SHARED (data.shared.*), keyed by user, never by branch. There is no branch versioning of data at all – fork/merge/promote are code-only operations over b.<id> lenses and structurally cannot touch a datum path. Per-branch data VERSIONING (placement B) is explicitly deferred – and would now require a wholly different path scheme. - NOT a type system for Nopales. Typing lives at the tree/storage boundary; the language stays dynamic (roerick: product surface tolerates jank; the OCaml shell is where proofs live). - NOT re-opening inheritance. No INHERITS, no subtyping (storage.no-inheritance, object-model .type-system stand). - NOT a REST/GraphQL surface. The image remains the API via prims + eval. - NOT raw SQL in product code. Projections serve the image and admin tooling only.

6 borge/tree-time.borg

pricklypear-tree-time

Pricklypear book chapter: tree-time – the generation axis on the one tree. Versioned leaves keyed (branch, path, gen); history, undo, and as-of reads become storage semantics instead of per-purpose scaffolding tables. Subsumes 012 code-history, gives local-first sync its merge clock, and closes the beads-audit-gap 'data state history: NO / time travel: NO' rows. Apps stay two-dimensional: every read defaults to a slice at head.

[status: planned]

> – *agent note:* 2026-09 design discussion, three sessions of prose: (1) app-level undo in notes – notes has one write funnel (notes/doc/upsert) over pure outline transforms, making it the ideal first consumer; per-note before-images. (2) history-as-tree-of-nodes under an h.* prefix – workable without kernel change but pure copying. (3) the answer that stuck: make generation a first-class storage key on the tree itself. Geometry: three overlaid trees – path-space within a branch, branch-as-parent-pointer tree across the world, gen chains within each branch. A branch is a named fork in path-space; a gen-fork (undo + write again) is an anonymous fork in time-space. The gen tree branches exactly when undo is followed by a new write – redo and lost-timeline retention fall out of keeping both children. Prior art cross-read: beads-audit-gap note (Yegge's Beads = Dolt versioning + events + interactions + prose intent). Our answer is more elegant because the habitat collapses the stack: Dolt reimplemented a storage engine and a database; we have Postgres; Beads bolted audit on a CLI; our evaluator sits inside the store. Dolt's advantages are operationalizing (shipped GC, sync wire protocol, scale-tested diff) – each already has a named home here (jobs runner + gen-range partitions, local-first.sync manifest exchange, tree-diff merge). Downstream blast radius audit: libs/* prim surface unchanged (new optional arg only); scaffolding tables stay head-shaped mirrors; 012 code_history is RETIRED (subsumed); 017 branch_generations is promoted from cache-invalidation signal to head coordinate; mirror triggers gain one head-gen filter; the real kernel work is tree_store.ml (insert-only + head bump) + crud.ml/prims_db.ml (gen filter, optional :gen arg) + one view rewrite. Named trees-hash-consing, trees-share-nodes-not-blobs as future notes on this chapter, per one-tree.bl-blast-radius precedent. 5.0 plan pass (campaign-v1): geometry + versioned-leaves + projections + reads-as-of-gen all designed above; migration-018 subsection pins the 018_tree_versions.sql schema contract (5.1 make is deterministic); chapter wired into the book root; all stanzas stay planned (implementation flips land with 5.1/5.2/5.3).

tree-time

Generation as a storage key on the one tree: (branch, path, gen) -> node

[status: planned]

geometry

Three overlaid trees; apps see a 2D slice

[status: planned]

THREE OVERLAID TREES. The habitat's storage geometry is three parent-pointer structures, and every value resolution is a walk up one of them:

1. PATH-SPACE. Within a branch the tree is rooted: b.<branch>.* for code lenses, data.u.<user>.* / data.shared.* for data (one-tree.tree-model).

2. BRANCH LINEAGE. Branches form a tree via parent pointers (system/main -> admin/personal -> admin/dev), with merges as occasional DAG stitches (branching.borg). Lookup walks up the branch chain until it finds the newest write of a path.

3. GENERATION. THIS CHAPTER ADDS THE THIRD. Within one branch, writes form a gen chain: gen 1 -> gen 2 -> gen 3 ... The chain is linear UNTIL undo-and-rewrite walks the head pointer back and writes again, at which point the gen tree grows a sibling. Redo and alternate timelines are properties of this tree, not of any undo manager.

A BRANCH IS A USER-NAMED FORK IN PATH-SPACE; A GEN-FORK IS AN ANONYMOUS FORK IN TIME-SPACE. Same mechanism at two granularities: branches are expensive, meaningful, user-named (fork for an experiment); gens are free, continuous, system-named (every write). This self- similarity is why ONE three-way tree-diff algorithm serves branch merge, gen-axis diff (undo inspection), and local-first sync (local-first.sync's manifest exchange is the same diff with a second site).

APPS SEE A 2D SLICE. Every read in the system defaults to (current branch or ancestor, path, head gen). No package code (libs/*) changes: prim signatures stay (get 'note id); the gen axis is opt-in via new optional args and new prims. "Present day" is a read policy, not a storage property.

DATA PATHS ARE BRANCH-DEGENERATE. data.u.<user>.* is branch-free per the one-tree /data DECISION, so for data the branch coordinate collapses and gen is the only real second axis. If placement-B (per-branch versioned data) ever reopens per its recorded trigger, data becomes genuinely 3D and this same storage serves it – no second versioning scheme needed.

versioned-leaves

The storage change: tree leaves keyed (branch, path, gen); writes are append

[status: implemented]

> *[verify] smoke: ksh scripts/verify_tree_version_shadow.sh (Phase 1 shadow gate: trigger present; every shadowed live tree_nodes row's newest version is CONTENT-identical to head (envelope hash subfield excluded – deferred BORGE-1786931598); no TOMBSTONE at head for a live row; head counters never behind live version rows). Canonic tooling: ksh scripts/reconcile_tree_version_head.sh repairs the one non-head-equivalent class (tombstone-head-but-live rows are re-versioned from the live authority) and the counter heads, then re-runs the gate to confirm PASS; smoke: image/test/tree_version_shadow_test: datum create shadows one row; datum update appends + as-of read returns old body; datum delete appends TOMBSTONE; fn code-path bumps branch_generations*

NEW TABLE tree_versions, keyed (branch_id, path, gen):

gen is the tree-scoped (branch-scoped for b.* paths; tree-scoped for branch-free data.* paths) monotonic counter ALREADY COUNTED by branch_generations (migration 017) and the tree_generations variant proposed with it. 017 was built as a cache-invalidation signal for remote-eval.caching; this chapter promotes the counter to the storage key – same counter, second use, exactly as local-first.sync's last-synced merge base is the same counter's third use.

WRITES ARE APPEND. update and delete never rewrite a row: - update writes (branch, path, gen+1, new body). - delete writes (branch, path, gen+1, TOMBSTONE) – a tombstone row is the marker, never a DELETE. This is what makes undo-of-delete and audit-by-state-diff free. - create is identical (the first version row). Structural sharing falls out of append-only without content addressing: a write touches exactly the changed paths; every other path's newest row is untouched. The tree's nodes are our chunk granularity –

habitat data is grain-sized (notes outlines, todos, defs are KB-scale), so per-node sharing is near-optimal where Dolt needs chunk-level sharing inside large tables. (Hedge recorded: a 50MB table-shaped datum updated one field at a time pays O(node) per write; the culture says data is grain- sized, and content addressing – one-tree.tree-model "content-addressing later" – is the physical dedup answer if that changes. Hash-chained gens (tamper evidence) are NOT required: the image writer is trusted; this chapter does not build a Merkle-DAG, it uses hash comparison for diff, not integrity.)

READ PATHS. Leaves may carry precomputed subtree hashes (one-tree.tree-model: hash over canonical body, root hashes compose) so tree-diff across gens is hash-first: equal hash = prune, differ = walk. This is the existing subtree-hash plan with a second consumer (the first was branching merge; the third is local-first sync).

MIRROR PRESERVATION – the bl-blast-radius contract applies in full: a scaffolding-table writer and a tree writer see IDENTICAL history, INCLUDING writes made through the other path and from AFTER v1 shipped, or the swap corrupts merge bases. The triggers must keep the tables and the versioned tree in lockstep both ways, as migration 009 established and this chapter extends into the time dimension: table UPDATE/DELETE both write a new version row AND keep the head-shaped mirror current.

migration-018

The 018_tree_versions.sql schema contract (design, plan pass)

[status: planned]

THIS IS THE PLAN-PASS SCHEMA CONTRACT for migrations/018_tree_versions.sql – the file does not exist yet; 5.0 designs it here so 5.1 (make versioned-leaves) is deterministic, mirroring how 4.0 pinned 015_jobs.sql in borge/jobs.borg.

TABLE tree_versions – append-only history, one row per (branch_id, path, gen) write:

branch_id uuid NULL for branch-free data.* paths (otherwise the owning branch); path ltree NOT NULL (same encoding as tree_nodes: b.<hex(branch_id)>.fn.<label> / data.<type_name>.<id>); gen bigint NOT NULL (the gen counter – see below for the per-path-class source); kind text NOT NULL (fn | type | route | datum); type_name text (datum only); body jsonb NOT NULL (the node envelope {v,kind,path, type,body,author,doc,hash}; TOMBSTONE is a reserved body marker for delete – see below); hash text NOT NULL (sha256 over canonical body; precomputed subtree-hash consumer); author text NOT NULL; created_at timestamptz NOT NULL DEFAULT now() (write time; immutable per version row). PK (branch_id, path, gen) – unique per write. branch_id NULL allowed only on data.* paths; enforce with a CHECK that branch_id IS NOT NULL when path <@ 'b.*'.

COVERING INDEX for head/projection reads: (branch_id, path, gen DESC) A b-tree range scan returns the newest gen per path in one pass (projections subsection). No join needed at habitat scale.

GEN COUNTER SOURCE, per path class (reads-as-of-gen): - code paths (b.<branch>.*): branch_generations (migration 017) with its per-branch bump discipline IS the head counter; each new version row takes branch_generations.generation+1 and the counter bumps. The 017 triggers already fire on the scaffolding writes bound to these paths. - data.* paths: branch-free, so a TREE-WIDE tree_generations (bigint) row is the head; every data write takes gen+1 (the 017 root-row bump discipline generalized to one row). Deliberately NOT per-path – under (branch_id, path, gen) the per-path order is implicit; the global counter keeps undo / diff ordering tree-wide and matches the local-first.sync merge clock.

TOMBSTONE: delete writes (branch_id, path, gen+1, body = {v:1,kind,path,type,body:null,author,doc:null,hash:<hash of tombstone marker>}) – a row, never a DELETE. A path is ALIVE at gen when its newest row at or below head is NOT a tombstone. A tombstone row

IS the head-shaped mirror's delete signal (mirror rows DROP; the history row remains).

SUBTREE HASH: hash is over the canonical body (as today); the chapter's content-addressing note stays deferred – this column is the diff-first-prune comparison key, not a Merkle chain.

PARTITIONING for gc (gc subsection): tree_versions is declared RANGE PARTITIONED BY list(gen/1000) with lazy partition creation; DROP PARTITION is the retention path. The 018 migration creates the table + base partition only; gen-window partitions are created by the retention worker.

ALL ADDITIVE per migrations.borg additive-only + image- applied-by-image discipline; 018 applies idempotently and registers in schema_migrations. Nothing reads this table until Phase 2 of coexistence-plan flips the PP_* flag; the scaffolding mirrors + tree_nodes remain the source of truth through Phase 1 dual-write shadow.

projections

Head-shaped scaffolding stays bit-identical: views gain one predicate; mirrors gain one filter

[status: planned]

> *[verify] smoke: image/test/code_history_tree_test: code-history/code-diff read from tree_versions (below-head before-images, newest-first; rel function_defs / op UPDATE / author; code-diff vs nth-newest; in-range/out-of-range); smoke: external history_test (012 regression): redefine records old body, merge-clobber records history, additive insert has NO history, rollback leaves none, code-diff smoke – all green under the tree-backed source*

THE VIEWS ARE THE API. Reader concentration from the projection era is what this change rides down:

- tree_data_rows and friends gain one predicate: greatest (branch, path) row with gen <= head, and no tombstone as newest. A b-tree range scan ((branch_id, path, gen DESC) covering index) – one index range scan per read, no join, well inside the image.borg per-page DB-cost ceilings at habitat scale.

- The scaffolding tables (function_defs, type_defs, view_bindings, data_rows) STAY HEAD-SHAPED. Gen history lives inside the versioned tree only; the mirrors reflect head, as they do today. Every residual reader, fork-copy path, admin page, and /data browser sees a bit-identical HEAD. Zero package code changes.

- WRITE-then-READ within one request must read its own write – crud.ml's in-transaction reads already run on the bound connection (Io_lwt.Db.current_conn); the head pointer for any in-flight read is the transaction's gen, so this property is inherited from the existing transaction discipline, not re-derived.

- The mirror triggers (009 both directions, 020 AST projection) gain ONE further condition: head-shaped targets refresh only when the versioned write is the new head. Append-side, each write is by construction the new head of its (branch, path), so the filter is trivial; the condition matters for future restore/head-move operations.

WHAT LEAVES: the 012 code_history triggers and table. Their job (before-images of code rows at writer-txn fidelity) is subsumed: a history read = read the version row at gen-1. code-history and code-diff prims KEEP THEIR SIGNATURES and re-implement over tree_versions (see reads-as-of-gen). The 012 table itself: read-only archival during phase 2; dropped in a later migration once equivalence is verified (one verifier script: sample N names, compare 012 rows' bodies to tree_versions bodies at matching changed_at).

PHASE-3 MAKE (landed, 5.2): the code-history / code-diff reimplementation over tree_versions is DONE – Store.list_code_history routes to tree_versions

via the versioned read surface (versioned_history below-head), with the 012 table as the fallback for pre-shadow (pre-019) names so no historical depth is lost during the coexistence window. The below-head filter preserves the 012 before-image semantic exactly: an insert-only fn has no history until it is rewritten or deleted, and the newest entry is the state just before the latest change ("read the version row at gen-1"). re-implements the two prims with signatures unchanged; rel is derived from the code-path kind (fn?function_defs / type?type_defs / route?view_bindings), op=DELETE on below-head TOMBSTONE rows else UPDATE. Verified by image/test/code_history_tree_test + the existing history_test stays green (additive-insert-no-history included). STILL PLANNED (Phase 2): the tree_data_rows head-predicate view rewrite behind the PP_ flag + the equivalence verifier + 012 archival/drop.

reads-as-of-gen

The new prim surface: optional gen coordinate + four new prims
[status: implemented]

> *[verify] smoke: gen-now returns the current branch generation (migration 017 ancestry-walked counter); smoke: (gen-read path gen) returns the node body at gen as version JSON – absolute data./b. paths AND relative logical suffixes crossed through the branch lens chain (closest-wins); nil when unversioned; smoke: (gen-diff path ga gb) returns 'same' when both leaves hash-identical, else a {path,a,b,same,delta} node-granular diff (shallow body key walk on mismatch); smoke: (undo path [n]) restores the value at head-n as a NEW write at head+1 (restore-by-write: the restore itself is versioned + redoable) for the datum path class; smoke: (tree/get path :gen N) resolves the node at that gen (versioned read); omitted = live head; smoke: image/test/reads_as_of_gen_test: tree/get :gen reads-as-of; gen-read absolute path at head + as-of; gen-diff node-granular + self-diff 'same'; undo restore-by-write + redo + n-step deep*

EXISTING PRIMS GAIN A FALLBACK COORDINATE, NOT A REQUIRED ONE. get / rows / tree_get take an optional trailing :gen N argument; omitted means head. No existing call site in libs/* or the kernel changes meaning.

NEW PRIMS (small surface, all ordinary prim_fns):

(gen-now) -> current tree gen (gen-read path gen) -> node body at gen, crossing the same branch lens rules as head reads (gen-diff path gen-a gen-b) -> node-granular diff: hashes first, walk on mismatch (same algorithm branching merge + local-first sync use) (undo path) / (undo path n) -> restore the value at head-n as a NEW write at head+1. Restore-by-write, not head-pointer manipulation: the restore itself is versioned, so undo is lossless by the same argument as restore-by-redefine in code-history (branching.borg). Redo = (undo path) again on the restored state via the gen chain; no separate redo stack.

WHERE THE HEAD POINTER IS, per path class: - b.<branch>.* code paths: per-branch – the branch_generations counter IS the head. Undo on code paths is branch-local: read (branch, path, gen-1), write to (branch, path, gen+1) on that branch. - data.* paths: branch-free, so the head pointer is a tree-wide data generation (the 017 migration's root-row bump discipline generalizes to one tree_generations row). Undo on data paths is per-path: it moves no global pointer, only appends a restored value for that one path. NOTHING about multi-user safety changes: Andrea's undo of her note is an append to (data.u.andrea.note.<id>, gen+1); Beth's head is untouched because the head is a counter, not per-path state.

EVAL-SCOPE UNDO IS NOT A PRIM. "Undo my session's last N writes" is the habitat-events stream (event journal of eval forms) correlated with paths, then a sequence of per-path undos – rerun bookkeeping, not storage. This chapter deliberately does NOT build cross-path transaction-undo; habitat-events + per-path (undo path) compose it when both exist.

COST ACCOUNTING (agent guidance): the first gen-as-of read pays one index range scan; the authoritative cost playbook applies – gen-diff of large subtrees belongs in the read tool you call, not pasted wholesale into eval context.

consumers

What this subsumes or feeds, in dependency order
[status: planned]

> *[verify] smoke: (code-restore name [n] [owner/branch]) re-writes a code node (fn/type/route) at head-n as a NEW write at head+1 through the code write surface (upsert_fn / upsert_type / upsert_route): reverse mirror re-populates function_defs / type_defs / view_bindings, the restore is itself versioned + redoable (restores the value BEFORE the latest change; n is row-count steps back, not gen arithmetic); smoke: (code-restore name [n] [owner/branch]) errors cleanly – no such code path / no version history / tombstone at n steps back (use tree/delete) / no body / unsupported rel / out-of-range n – never a crash; n<1 and n>=history-length rejected; smoke: image/test/code_restore_test: fn n=1 rewinds body to body-2, n=2 to body-1; type n=1 rewinds fields; route n=1 rewinds handler + visibility; insert-only has no state; unknown name errors cleanly; smoke: NOTES UNDO – the FIRST APP CONSUMER is real (tree-time.consumers details): notes has ONE write funnel (notes/doc/upsert), so undo needs NO per-app journal – the substrate keeps the before-image. The note page toolbar gains an Undo button (action=undo in libs/notes/lib.pp) that does ONE per-path (undo path) call and redirects back; an error-string return (no history / no value) leaves the note unchanged. Undo is KIND-AWARE: a note is a hierarchical node (kind=note via upsert_hier), and restore-by-write routes back through upsert_hier preserving the original kind + payload envelope – a naive upsert_datum restore would silently rewrite the payload marker to datum (the kind-column/payload mismatch this consumer exposed). image/test/reads_as_of_gen_test: 'undo kind-aware (note payload not datumed)' writes two note versions through upsert_hier, (tree/undo path) restores title v1, and asserts node kind stays note AND payload-»'kind' stays note (not datum).; smoke: image/test/reads_as_of_gen_test: 'undo kind-aware' – after (tree/undo path) on a two-version note, live tree_get returns title v1, the JSON node kind is still note, and the live tree_nodes payload marker kind is still note (not datum).*

SUBSUMES: branching.code-history (012). The separate before-image trigger table exists because the tree did not keep history. Now it does. code-history / code-diff / code-restore become reads of the tree; restore gains a real prim (code-restore = gen-read + define, exactly the lossless-redefine pattern the 012 design documented by hand).

FIRST APP CONSUMER: NOTES UNDO. The motivating case (design discussion 2026-09). Notes has ONE write funnel – every mutation ends at notes/doc/upsert over pure outline transforms – so app-level undo needs NO per-app storage: - notes/doc/upsert-in records nothing; the substrate keeps the before-image. - (undo (node-path 'note id)) in place of any history journal; the notes package keeps at most a UI-facing "undo the last change to THIS note" button wired to one per-path call. - Leaf-vs-whole is free: undo of one outline NODE and undo of the note SUBTREE are the same prim at two path granularities, because the outline is a subtree of tree leaves. The earlier candidate designs (JSON blob notes_history table; h.* prefix spine of copied subtrees) are SUPERSEDED and recorded here only so a future reader does not re- derive them – both were copy-based schemes working around a substrate that discarded before-images.

FEEDS: branching.branch-status (IMPLEMENTED 2026-08-28). Branch status status (clean/ahead/behind/diverged/synced) is a hash-first read over tree_versions keyed on a base_gen merge-base coordinate – the first branch-

axis (not gen-axis) consumer of the versioned leaves, and the retirement of list_overrides' name-presence classification.

FEEDS: local-first.sync (planned). Sync's merge base was already defined as (branch, generation) snapshot pins; with gen as a storage key, "common ancestor at pin" is a READ, not a snapshot artifact. The 017 counter's three uses – cache invalidation, merge base, and now storage key – are one counter. local-first's data-sync section gains one companion line: sync reconcile reads the data gen chain; it does not version data per-branch (that non-goal stands).

FEEDS: habitat-events (planned note garden/design?slug=habitat-events-borg). Orthogonal by design: events answer OPERATIONS questions (who evaluated what form, when); tree-time answers STATE questions (what did the world look like). The two join on path+gen: an event row may reference the gen it wrote, making the audit trail (scoreboard: agent audit PARTIAL -> agent- readable by construction) navigable from event to state. "Stream not tree" stays right for events; this chapter is the tree half of the full Beads-equivalent picture: events for op-log, tree-time for versioned state, forge for intent – one merge algorithm across all three.

SCOREBOARD EFFECT (beads-audit-gap): data state history moves NO -> YES-STRUCTURAL; time travel moves NO -> YES (the degenerate gen read).

gc

Retention: gen-range partitions, jobs-runner rollup, policy as habitat data
[status: implemented]

> *[verify] smoke: migration 022_tree_retention.sql applies clean + idempotent on a dev PG with 018-021 already applied (image-applied, version 22 in schema_migrations): creates tree_retention_policy (path_class CHECK code|data, type_name NULL= whole class, keep_windows >= 1, UNIQUE(path_class,type_name)) and tree_versions_retain(path_class, keep_windows) – an idempotent, conservative rollup+drop worker that computes a whole-class floor = GREATEST(head_gen - keep_windows*1000, 1000), CHECKPOINTs the newest in-retained-past state of every fully-retired path (whole history < floor) to a row at gen=floor, then DROP PARTITIONs every lazy gen-window entirely below the floor (never the base gen_0 window).; smoke: (retention-policies) -> VList of policy JSON – empty on a policy-free habitat (keep-everything default). (retention-policy-set path_class keep_windows) / (retention-policy-set path_class type_name keep_windows) upsert a whole-class (type_name NULL) or per-type row; (retention-delete-policy path_class [type_name]) removes it. Admin-only (require_admin); anonymous raises.; smoke: (retention-run [path_class]) drives the worker per path class that has an explicit whole-class policy (type_name IS NULL): runs tree_versions_retain, returns VList of (path_class, dropped_windows, checkpointed_paths). Path class with NO whole-class policy is a keep-everything no-op (never ad-hoc deletion). Verified live on the dev habitat: (retention-policy-set "code" 3) then (retention-run "code") dropped 16 gen-windows and checkpointed 4473 code paths; policies deleted after to restore keep-everything.; smoke: The retention worker runs ON THE JOBS RUNNER (migration 015) – boot.ml jobs_runner_loop gates a periodic retention_sweep by wall-clock (default 3600s, PP_RETENTION_SECS override) which calls Store.retention_run for every path class and logs per-class dropped/checkpointed counts. On a policy-free habitat the sweep is a cheap no-op SELECT. This is the coexistence-plan PHASE 4 gate that flipped the gc subsection to implemented (the chapter itself stays planned until the Phase-2 scaffolding read-switch + equivalence verifier tail in the projections subsection lands - see coexistence-plan STATUS 5.4 reconcile).*

NO NEW SUBSYSTEM. Dolt ships dolt_gc because it owns its chunk store; here Postgres owns vacuum. GC is three existing pieces composed:

- PARTITIONING: tree_versions is partitioned by gen range windows (e.g. 1000-gen ranges, created lazily). Physical removal is DROP PARTITION – instant, no vacuum scan. PG-native; the chunk-GC problem Dolt engineered against does not exist for a partitioned table.

- RETENTION POLICY IS HABITAT DATA. Policy rows declare per-path-class retention (code paths: keep all history, it's small and it IS the audit trail; data paths: per-type policy, default keep-everything-until- a-policy-exists). habitat-events' "retention is habitat data (policy rows + rollup job on the existing jobs runner), never ad-hoc deletion under disk pressure" applies verbatim; the jobs runner (migration 015) executes.

- ROLLUP, NOT SILENT LOSS. Before a window is dropped, surviving state collapses to a CHECKPOINT row at the window's boundary gen: the newest version of each path in the window is rewritten at the boundary, so reads as of ancient gens still resolve (to the checkpointed state) and head reads never notice. "Nobody gets everything forever – the audit claim survives at coarser grain."

ORDERING: GC lands AFTER reads-as-of-gen and the consumers bed in. The first long pole is growth FROM code paths, which are tiny; habitat scale makes early compaction premature. The retention job is a TODO until a path class outgrows its welcome – measured via /timing-style storage accounting, not vibes.

SHIPPED 2026-12 (campaign 5.3): the retention worker is REAL and runs on the jobs runner – migration 022 (tree_retention_policy + tree_versions_retain), the four admin-gated retention prims (retention-policy-set / retention-delete-policy / retention-policies / retention-run), and the boot.ml jobs-runner sweep gated by wall-clock + the keep-everything default. Whole-class policies are the v1 gate; per-type rows are recorded for future finer-grained work but the shared gen-window floor stays whole-class (tree_versions is a global table). The coexistence-plan PHASE 4 condition – "the retention worker runs on the jobs runner" – is met, flipping the gc subsection to implemented. (The chapter as a whole stays planned: the Phase-2 flag-guarded scaffolding read-switch + equivalence verifier + 012 archival/drop tail, owned by the projections subsection, remains open – see the coexistence-plan STATUS 5.4 reconcile.)

coexistence-plan

Phased, additive-only, flag-guarded; nothing existing is rewritten in place

[status: planned]

PHASE 0 (this chapter): spec. No code.

PHASE 1 – DUAL-WRITE SHADOW. DONE (campaign 5.1, loops 27/28): the migration-018 schema + the 019 additive dual-write shadow trigger landed. Every tree_nodes write (insert/update/delete->TOMBSTONE) appends a version row in ADDITION to today's writes, allocating its head gen atomically per path class (branch_generations for code b.* paths, the tree_generations singleton for data.* paths); 020 refines the append so a no-op mirror echo (unchanged content) returns BEFORE allocating a gen – one logical write is never double-counted. Nothing reads the new table for a live read yet. Verify: scripts/verify_tree_version_shadow.sh (Phase 1 gate: every shadowed live row's newest version matches head) upgraded in 5.2 to the Phase-2 equivalence gate.

PHASE 2 – READ SWITCH, FLAG-GUARDED. MOSTLY DONE. The gen coordinate landed as an UNGUARDED additive surface, not a flag-guarded mirror re-route: store versioned_head / versioned_history + (gen-now), then (tree/get path :gen N) / (gen-read path gen) / (gen-diff path ga gb) / (undo path [n]) (5.2, c9c49dc). The PP_* flag-guarded SCAFFOLDING read switch is the tail that remains STILL PLANNED, owned by the projections subsection: the tree_data_rows / scaffolding-mirror head-predicate view rewrite behind the PP_ flag + the 012 equivalence verifier (sample N names, bodies equal)

+ 012 archival/ drop. code_history triggers keep firing until equivalence holds continuously for a soak window.

PHASE 3 – CONSUMERS LIGHT UP. DONE except the 012 archival/drop (see PHASE 2 tail). code-history / code-diff re-implemented over tree_versions (signatures unchanged; 012 before-image table retained as pre-shadow fallback, 5.2, 72d5380); (gen-read)/(gen-diff)/(undo) land; the code-restore prim (code-path analogue of undo, 5.3 loop 36); notes undo is a real app consumer (toolbar Undo button, one per-path (undo path) call + kind-aware upsert_hier restore – 5.3 loop 37, 89b0555). The 012 table stays read-only archival until the equivalence verifier in PHASE 2 clears.

PHASE 4 – GC + CHAPTER COMPLETE. DONE (5.3 loop 37, 89b0555): partitioning + retention worker per the gc subsection – migration 022 (tree_retention_policy + tree_versions_retain), the four admin-gated retention prims, and the boot.ml jobs-runner retention_sweep gated by wall-clock. The PHASE 4 GATE – "the retention worker runs on the jobs runner" – is met and flipped the gc subsection to implemented.

STATUS (5.4 reconcile): Phases 1, 4 fully done; Phase 3 done except the 012 archival/drop; Phase 2 done except the flag-guarded scaffolding read-switch + equivalence verifier. The remaining open work is the Phase-2 tail, which is owned by the projections subsection (still planned) – this subsection stays planned until that tail lands, mirroring how projections/consumers stayed planned with their open items recorded.

ALL MIGRATIONS ADDITIVE per conventions (new table, new views, new columns – never rewrite payloads; 017's WHERE NOT EXISTS idempotence style). Downstream blast radius, audited in design: libs/* zero changes; scaffolding readers zero changes; habitat-events unaffected (append- only stream either way); local-first gains a read enhancement (merge base as read), no redesign.

non-goals

What tree-time is NOT

[status: planned]

- NOT branching data. The one-tree /data DECISION (single-head, user-owned, SPATIAL not TEMPORAL) stands at the CONCEPT level; this chapter adds TEMPORAL STORAGE-KEYS invisible to users – one head per path, many retained gens below it. The DECISION's reopen trigger (a user forking their own data to experiment and merge back) is a distinct feature; this chapter is compatible with it but does NOT build or reopen it.

- NOT a Merkle-DAG / tamper-evident ledger. Hash-chained gens exist only if a future untrusted-writer scenario demands them (local-first's untrusted-client punt). Hashes here serve diff, not integrity.

- NOT the habitat-events stream, and NOT replaced by it. Operations-audit and state-history answer different questions; one emissions table and one versioned tree.

- NOT eval-scope or cross-path transaction undo as a storage primitive. (undo path) is per-path; session scope composes from habitat-events correlation later.

- NOT a rewrite of libs/*, scaffolding tables, or the prim ABI. Optional args only; head-shape preserved; existing call sites mean exactly what they meant.

- NOT content addressing. Physical dedup of identical bodies is deferred to one-tree.tree-model's existing "content-addressing later" note; append-only gives the structural sharing this design needs without it.

- NOT rename-tracking. Path is part of the key; a rename is delete+create in history space. Same posture as git (detection over tracking); the blast-radius walking problem for historical names is accepted.

7 borge/branching.borg

pricklypear-branching

Pricklypear book chapter: branching
[status: partial]

> – *agent note:* 2026-08-02 status reconcile: agent-promote, package-updates, branch-browser-ui ? implemented. Still open: schema-authority (type_defs merge), ast-nodes (design).

branching

Branches as the fundamental organizational primitive: code branches, shared data, fork/merge
[status: partial]

Branches are to Pricklypear what files are to Unix: the fundamental organizational primitive. Everything is a branch; everything else is built on top of that.

STORAGE (one Postgres, not one DB per branch): - branches(id, parent_id, owner, name, ...) - function_defs / type_defs / view_bindings each keyed by (branch_id, name|route) - data_rows has NO branch_id – shared data No separate schemas/tables per branch. No Postgres INHERITS (see storage.no-inheritance).

KEY DISTINCTION: branches are for CODE, not data. - BRANCHED: function definitions, type schemas, view/route bindings. - SHARED: data_rows (todos, notes, bible, ?). Alice's experimental branch renders against REAL shared data. She forks code, not data.

TWO FORK SHAPES: - EMPTY child (personal garden): parent pointer only; inheritance via recursive CTE. Main package updates stay visible unless overridden. - FULL fork (experiment): copy code-rows from parent (cheap snapshot). Parent updates do not appear until merge.

ROLES: system/main package defaults (inheritance root) <user>/personal garden (boot default; long-lived pins) <user>/dev optional workshop (empty child of main; active product work intended to promote) <user>/<name> experiments (full fork when isolation from evolving main is required)

AUTHORITY (Nopales vs kernel) – see package-updates: Kernel (OCaml/migrations) ? git is source of truth Package Nopales ? Postgres branches are live source; git holds snapshots Boot seed ? additive into system/main only (never clobber existing) Explicit reseed ? git libs may overwrite main when an admin asks

Effective definition lookup (CLOS override): walk parent chain, closest definition wins. Writes (define/route) always hit the ACTIVE branch only.

fork

Forking a branch
[status: implemented]
(fork 'main 'alice/experiment)

Implementation: 1. INSERT INTO branches (id, parent_id, owner, name, ...) VALUES (new_uuid, main_id, 'alice', 'experiment', ...) 2. INSERT INTO function_defs (branch_id, name, body, author, updated_at) SELECT new_uuid, name, body, author, updated_at FROM function_defs WHERE branch_id = parent_id 3. Similarly for type_defs, view_bindings

Cost: O(num code rows in parent). For ~100 fns and ~20 types, a few hundred row inserts. Lands in well under a second.

Forking does NOT copy data table rows. The forked branch's functions operate on the same shared event rows as the parent.

[verify] test: image/test/primitives_test.ml – 48 alcotest cases; build: dune build @all; smoke: fork creates a branch and copies function_defs; smoke: override-walk resolves child overrides before parent; branches isolate writes

merge

Merging a branch into its parent

[status: implemented]

(merge 'alice/experiment 'alice/main)

Policy: - Default: admin-only merges to main, per-user can merge into their own branches freely. - Expandable later (PR/review flow).

Implementation (Store.merge_branch, image/lib/store.ml): upserts all function_defs rows from source into target via INSERT ... ON CONFLICT (branch_id, name) DO UPDATE, wrapped in a SERIALIZABLE transaction. Initial cut copies the full set; diff-based merge (only changed rows, per-function conflict resolution) is not yet implemented.

CONFLICT REPORT (v1: report-only, never blocks): before the upsert, merge computes names where BOTH source owns a row AND target owns a row whose updated_at is newer than source's (target moved independently since the lens diverged). Result string carries them: "merged N rows (M conflicts: a, b, c)" Resolution stays last-write-wins (source body lands); the report tells the human what was clobbered, and code-history makes the clobbered target body recoverable. Blocking/require-resolution merge is a later PR/review-flow concern, not v1.

SCOPE: merge copies function_defs only. It MUST NOT copy type_defs (schema authority is main; see schema-authority) and MUST NOT copy view_bindings blindly (route hijack risk). Routes travel with named functions through promote instead (see promote).

type_defs merge: not implemented by design in v1. A branch cannot propose a schema change through merge; schema changes land via migrations applied on boot (see migrations.additive-only) and are visible to all branches on next lookup.

[verify] test: image/test/primitives_test.ml – fork/merge/promote alcotest cases; build: dune build @all; smoke: merge upserts changed rows; merges into system/ require admin; smoke: merge result string lists conflict names when target moved newer; smoke: merge copies no type_defs and no view_bindings rows*

diff

Structural divergence between two branches (AST-aware, not textual)

[status: implemented]

– *agent note:* 2026-08-16 ralph loop 19: implemented + verified. Store.diff_branches (store.ml), (diff from to) prim (prims_db.ml), GET /diff?from=&to= private page (http_branches.ml, http.ml route) wired; ast_diff 27/27 green, full dune build @all + runtest exit 0. 2026-08-14: gap surfaced testing personal-vs-main divergence. merge_branch is naive full-copy with explicit TODOs (diff-based merge, conflict detection); diff is the primitive those TODOs consume. AST-awareness chosen because function_defs.body is ONE expr node (not a statement list), JSONB-tagged structurally - see lisp-interpreter.ast-diff.

[verify] build: dune build @all; test: lisp/test/ast_diff_test.ml – same/modified over all 11 real expr constructors (Var Lit App Let Letrec If Lambda Prim Seq Quote Define); test: App.args and Letrec.bindings use list-LCS

keyed on analyze.ast_hash so insertion mid-sequence does not mark every following element changed; test: top-level ast_hash equality short-circuits the walk for unchanged rows; smoke: (diff 'system/main 'admin/personal) lists added/removed/modified names without scraping /bindings HTML; smoke: /diff?from=system/main&to=admin/personal renders Modified rows with per-ctor AST deltas; Added/Removed show pp_expr of the lone body

PROBLEM: testing divergence between a personal branch and main is currently manual – (list-bindings) under each (checkout ...) and eyeballing. merge_branch (see merge) copies ALL function_defs source->target with three open TODOs: diff-based merge (only changed rows), conflict detection (parent moved same name since fork), and per-function resolution. diff is the primitive those TODOs consume, and the thing a human/agent reads before promoting.

(diff 'system/main 'admin/personal) -> VList of change-dicts, one per divergent name (no row for unchanged – the common case): {name kind status from-updated to-updated} kind :: fn | type | route status :: added | removed | modified

LAYERING (two stages, at the seam the codebase already has): 1. ROW CLASSIFICATION (SQL, one round trip): both effective views via the SAME recursive CTE that list_function_names_meta / list_packages / search_function_bodies use (closest-ancestor-wins). FULL OUTER JOIN on name, classify each as added = in to, not in from removed = in from, not in to modified = both, AND from.body IS DISTINCT FROM to.body Scope mirrors merge_branch: function_defs + type_defs + view_bindings (NOT data_rows – shared, branch-free per branching.data). Conflicts fall out for free: this is the inverted conflict_sql already in merge_branch. 2. AST DELTA (OCaml, Modified rows ONLY): parse both bodies via expr_of_json, walk with lisp/lib/ast_diff.ml (see lisp-interpreter. ast-diff). Never runs on Added/Removed (nothing to compare – show pp_expr of the lone body) or on Same (short-circuited).

AST-AWARENESS IS A PROPERTY, NOT A NICETY. function_defs.body is ONE expr node (Lambda body, not a statement list), JSONB-tagged structurally ({"if":...}, {"lambda":...}, {"app":...}). A textual diff over the JSONB ::text would be unreadable; a line diff over pp_expr output would jitter on whitespace and miss structural moves. The diff is structural over the expr type:

ctor differs (Var -> Lambda) = Replaced same ctor, fixed-arity children = Node, recurse positionally (If test/then/else; each Seq a/b; Lambda params/body; Let name/ child val/body; Define name/val; Quote body; its Var s; Lit v) own delta same ctor, list children = Node with a list-LCS step over the children plus a before recursing into matched pairs: moved/ - App.args (head + args list) added - Letrec.bindings (keyed on binding name) list handled leaf value differs (Var s, Lit v) = Replaced

The 11 real expr constructors (ast.ml): Var, Lit, App, Let, Letrec, If, Lambda, Prim, Seq, Quote, Define. The Quasiquote / Unquote / Unquote-Splicing mentioned in lisp-interpreter. ast are reader-level sugar that desugars before persistence (see macros) – they do not appear in function_defs.body and need no diff case.

FAST PATH: analyze.ast_hash (sha1_hex of Reader.pp_expr) is already the content identity. Compare from/to hashes first; if equal, skip the walk entirely (returns Same). The 90% case (unchanged defs between branches) costs one hash compare per row, not a tree walk.

SCOPE & TYPE/ROUTE HANDLING: - function_defs: full AST diff as above. - type_defs: diff the 'fields' jsonb array with the same list-LCS (field declarations are positional lists, not expr trees; no ast_diff involvement). - view_bindings: tabular (route, handler) pair sets – divergent pairs are added/removed rows, no body to diff. Stays in SQL. Schema changes land via migrations (see schema-authority), never via diff/merge.

NON-GOALS (v1, honest): - No rename detection. A bound-var rename

x->y surfaces as a Replaced at the binding site (Lambda.params / Let name / Letrec binding name) AND a Replaced at every reference site (Var x -> Var y). git doesn't do this for free either; real rename detection needs the 3-way base (fork point), which is the natural v2 once code_history before-images + branches.last_synced are wired as the base. - No alpha-equivalence. Bound names are NOT normalized before comparing – a genuine rename is meant to be visible, not hidden. - Param order matters. (lambda (a b) ...) vs (lambda (b a) ...) is a positional child delta, NOT "renamed params".

DEGRADED RENDER PATH: if the /diff page does not want to render the structured delta tree, fall back to a unified line diff over Reader.pp_expr from / Reader.pp_expr to. This is the existing simple_text_diff shape (see code-diff) applied to two live bodies instead of a history entry. Always available; never the primary surface.

WIRE-IN: - Store.diff_branches ~from_id ~to_id (image/lib/store.ml, beside merge_branch): the SQL stage + Modified-row (from_expr, to_expr) pairs ready for the image layer. - (diff source target) prim (image/lib/ prims_db.ml, beside code-diff): returns the VList of change-dicts (summary level). The structured AST delta is consumed in-process by the /diff page renderer (Ast_html + Ast_diff), NOT exposed as a separate prim in v1. A programmatic (diff-ast name from to) prim is a v2 addition if agents need the delta tree as Nopales values. - /diff?from=&to= private page (image/lib/ http_branches.ml, sibling to /bindings): three-column row table; Modified rows expand to the AST delta overlaying Ast_html.pp_expr_html. Future "promote selected" off diff rows IS the merge TODO (diff-based merge) in disguise.

DOWNSTREAM CONSUMERS (the point of building it): - merge TODO "diff-based merge (only changed rows)": merge calls diff, upserts only rows whose status != same. - merge TODO "conflict detection": the 3-way base is the base_gen coordinate of branch-status (tree-time made it a READ, superseding the code_history / timestamp plan recorded here).

branch-status

Git-style clean/ahead/behind/diverged status as ONE derived read of the tree-time substrate

[status: implemented]

PROBLEM: branch status is computed AD HOC, per surface, as name-presence joins on the scaffolding tables. Store.list_overrides classifies every own function_defs row as new | override | main newer by NAME EXISTENCE + updated_at heuristics – never by content. Consequences, all observed: - a full fork marks every function "override" though every body is identical - after promote, the source pin stays IDENTICAL to main but reads "override" (or "main newer", the timestamp flip) forever - nothing can answer "does this branch actually diverge?" without scraping /diff - merge's conflict heuristic is updated_at string comparison; there is no fork point anywhere in storage (branches.last_synced is an unused timestamp) No incremental fix works: each surface re- derives branch semantics itself. The fix is to stop deriving branch semantics from row geography and start deriving it from CONTENT at GENERATIONS – the tree-time substrate that already exists.

MODEL (two coordinates, additive): - base_gen: NEW COLUMN on branches (bigint). At fork, base_gen = parent head gen (branch_generations.generation of the parent at fork time, parent chain walked; 0 for branches forked before the column exists whose parent has no own writes). After a merge or promote lands on the parent, base_gen ADVANCES to the merged head – this is git's moving merge-base, and it RETIRES branches.last_synced (timestamp), which was designed for this job in the code_history era and never wired. - head gens: already counted per branch by branch_generations (migration 017), the same counter tree-time promoted to a storage key.

Because tree_versions (018-021 shadow) is append-only with a content hash per version row, "value of path P on branch B at gen G" is a READ, not a

snapshot: newest version row for (resolved-owner-of-P, P) with gen <= G, non-tombstone. Resolution for uncopied gens uses the same closest-wins branch-lens rule as head reads.

THE STATUSES (per name, fn | type | route): b = branch value at own head, p = parent-chain value at parent head, a = value at base_gen (absence = tombstone/never-existed; comparisons are HASH comparisons, ast_hash for fn bodies per diff's fast path): clean no own write since base, b == p (inherited; may differ from a – that is normal upstream movement, not dirt) ahead-new own write; parent has never had the name (a absent) ahead own write; b != a, p == a behind no own write since base; p != a (upstream moved) diverged own write; b != a AND p != a AND b != p (the genuine conflict set merge reports) synced own write; b == p (identical pin – post-promote or package-pin noise; the unbind candidate class, replaces "override") Git mapping: clean = up to date, ahead* = committed-locally, behind = remote moved, diverged = conflict, synced = cherry-picked downstream already. "dirty" = anything but clean|behind (own writes exist).

ONE COMPUTATION: Store.branch_status ~branch_id -> rows {name kind status branch_hash parent_hash base_gen head_gen} Hash-first per tree-time: compare hashes, prune on equality; bodies are fetched ONLY for rendering (Modified rows go through lisp/lib/ast_diff.ml exactly as /diff does today). One round trip per path class, same recursive-CTE chain walk style as diff_branches – but against tree_versions, not function_defs.

CONSUMERS (every current status surface becomes a rendering of branch_status, none keeps its own classifier): - /branches list: "overrides" count becomes dirty count (ahead-new + ahead + diverged); synced pins counted separately, muted - branch detail: Overrides table gains the status column; synced rows offer unbind as the primary action - /changes + (branch-changes): class is the status, not new|override|own - /diff + (diff ...): same hash pairs; the added/removed/modified classification is a view of the same rows (ahead = modified in to, absent in from at matching gens) - merge: upserts ONLY ahead/ahead-new/ diverged paths (diff-based merge TODO closes); conflict report = the diverged set with real bases (conflict-detection TODO closes; blocking resolution stays v2 per merge's recorded policy); on success source and target base_gens advance - promote: after landing, offer unbind of synced pins on the source

RETIRES: - Store.list_overrides (4 call sites: the three /branches renderers + branch-changes; no Nopales package depends on it) - the "main newer" timestamp badge - branches.last_synced (timestamp) – its job is base_gen's, done correctly - diff_branches' scaffolding-table CTEs become historical the day PP_TREE_READS stabilises; during coexistence BOTH the versioned status and the classic diff prim must agree (verifier below) – status is a NEW consumer of the shadow table, not a read-switch of the scaffolding

POSITION IN THE BOOK: this is a tree-time consumer (tree-time.consumers gains a line), sibling to code-history's re-implementation over tree_versions. It does NOT require the tree-time Phase-2 scaffolding read-switch: tree_versions is maintained by the dual-write shadow today, so reading it is additive. Scope is code paths only (b.* / fn-type-route); data rows are branch-free by the one-tree DECISION and have no branch status to compute.

PHASES: S1 migration: ALTER branches ADD base_gen + backfill; fork writes it; merge/promote advance it. Store.branch_status over tree_versions + one equivalence verifier (sample N branches: statuses vs diff_branches head-vs-head where both defined). S2 surfaces re-render: /branches, branch detail, /changes, branch-changes prim, /diff alignment. list_overrides deleted in this phase, not before. S3 merge + promote consume the statuses; conflict report = diverged set; base_gen advancement tested end-to-end (fork -> edit both sides -> merge -> status clean again = git's post-merge state).

> – *agent note:* 2026-08-28: implemented + verified same-day as the plan pass. Migration 024 (branches.base_gen + backfill-to-parent-

current-gen approximation); fork/ensure write it; Store.branch_status (store.ml, one round trip, all three kinds, chain-walk over tree_versions + live-pin EXISTS on the scaffolding tables); classify_status decision table; /branches dirty+synced columns, branch detail status table (unbind hint on synced pins), /changes + (branch-changes) class = status; list_overrides DELETED (4 call sites); merge_branch status-driven (upserts only ahead-new/ahead/diverged; conflict report = the diverged set; base advance post-commit) + head-vs-head fallback for non-parent targets. TWO make-phase findings recorded in the details: (1) HASH DOMAIN - the stored tree_versions.hash column is NOT comparable across write paths (tree-first Tree_store.hash_body compact-JSON digests vs 009-mirror digest(jsonb::text) spaced-canonical digests: identical bodies, different hashes - 020 itself calls the stored hash "a consistency artifact, not content"), so status computes digest((body->'body')::text) AT QUERY TIME, one domain regardless of writer; a global rehash is tree-time scope, and gen-diff has the same latent cross-domain hazard when the two compared gens were written through different paths. (2) MERGE BASE ADVANCE MUST RUN POST-COMMIT - the 017/019 gen-bump triggers are deferred, so reading branch_generations inside the merge tx sees the pre-merge counter; the advance is a separate statement after with_transaction returns. Verified: image/test/branch_status_test (lifecycle: fork base_gen = parent head gen, empty fork clean, ahead-new, diverged, behind, post-promote synced, merge advances base_gen + post-merge synced; identical-pin-synced + unbind-returns-clean) + live smoke on the dev habitat (fork -> define -> ahead-new -> promote -> synced -> redefine -> diverged -> merge reports the conflict by name -> synced). Full dune runtest green; borge drift: none.

[verify] build: dune build @all; test: image/test/branch_status_test.ml – lifecycle: fork sets base_gen = parent head gen; merge advances base_gen; test: statuses: empty fork = clean; own new fn = ahead-new; promote-then-status = synced; parent-only edit = behind; both-sides edit = diverged; identical pin = synced; unbind returns it to clean; test: equivalence: diff_branches reports the diverged fixture as modified head-vs-head, agreeing with branch_status; smoke: S2: /branches renders dirty + synced columns; branch detail status table with unbind hint on synced; (branch-changes) class = status; list_overrides has no callers; smoke: S3: merge upserts only ahead/diverged names (merged 0 rows when all synced); conflict report names the diverged set; base_gen advances post-commit

agent-promote

Agent promotes from its workshop into the parent human's personal

[status: implemented]

promote(source=<parent>/agent, target=<parent>/personal): Selective upsert of function_defs from the agent's workshop into the parent human's personal branch (same mechanism as promote_to_main, different target). The agent's work lands in the human's garden.

Permission: gated by the parent human's allow_agent_promote toggle (users table, default true). The human can revoke it to freeze the agent out of their personal branch; the agent still works in <parent>/agent but cannot promote.

This is the human-in-the-loop gate: the agent builds in its own branch; nothing reaches the shared tree (system/main) or the human's garden until the agent promotes and the human allows it. promote to system/main remains admin-only.

– *agent note:* 2026-08-02: promote_to_personal + allow_agent_promote user toggle + set-allow-agent-promote prim; agent boots parent/agent workshop.

schema-authority

Types in main, branches read against main's schema

[status: partial]

Schema authority is 'main'. Branches cannot change the underlying table shape directly; schema changes land via additive-only migrations applied on boot (see migrations.additive-only), and are visible to all branches immediately on next function lookup.

data_rows has no branch_id; type_defs are stored per-branch but the underlying table shape is shared (see object-model). Branch code that ignores new fields reads a subset without error. Branch code that expects removed fields would fail, but additive-only means removed fields do not occur.

type_defs merge: not yet implemented. Today all data lives in the shared data_rows table; per-type Postgres tables are not created by deftype.

– *agent note:* 2026-08-02 confirm partial: shared data_rows + additive migrations; type_defs merge still not implemented (details admit).

personal-workspace

Boot on empty <user>/personal; seed package defaults only onto system/main

[status: implemented]

MODEL: - system/main = package defaults (inheritance root). Boot seed_stdlib writes libs/*.pp here only, under seed-policy (additive – see below). - <user>/personal = garden (user channel). Empty child of system/main (parent pointer only). Image boots here after seed. User (define) and (route) write personal rows only. - <user>/dev = optional workshop (also empty child). Prefer for product spikes you intend to promote; keep personal for long-lived custom pins. - <parent>/agent = agent user's default workshop. Empty child of system/main (sibling of the parent human's personal). An agent user boots here, not into the parent's personal. The agent works in its own branch and promotes into the parent's personal when the human allows it (see promote). - Inheritance: recursive CTE; child overrides shadow main; unpinned names auto-see main updates.

BOOT ORDER: 1. ensure_branch owner=username name=personal parent=system/main (no code copy) 2. seed_stdlib with store.branch_id = system/main (additive persist – see seed-policy) 3. checkout personal + rehydrate env from CTE 4. refresh ui/nav shell

load-library on CURRENT branch still upserts that branch (intentional pin). On personal/dev that pins the whole package – prefer small overrides or work name-by-name via define.

(checkout "system/main") / (checkout "admin/personal") rehydrate sync; header shows current branch.

unbind

Remove a branch-local binding; reveal shadowed system/main, never delete through

[status: implemented]

– *agent note:* 2026-08-05: implemented (unbind name) in Store.delete_function + session.ml prim. Tested on local dev: unbind branch-local override reveals system/main binding; unbind missing returns "nothing to unbind"; unbind on system/main errors. 160 dune tests green.

[verify] build: dune build; test: dune runtest – unbind cases; test: (unbind name) on a branch-local define removes the branch row; system/main binding revealed; test: (unbind name) on system/main errors (cannot unbind on main); test: (unbind name) with no branch-local row errors (nothing to unbind); test: unbind never deletes another branch or system/main rows

PROBLEM: transient defines accumulate as permanent litter – agent probes, test fns, spike code. Every (define) on a personal branch is a permanent row. There is no way to remove a branch-local binding without deleting the whole branch.

PRIM: (unbind name) Deletes ONLY the current branch's function_defs row for name. After unbind, name resolves via inheritance to whatever system/main (or a parent branch) provides – revealing the shadowed binding, if any.

SEMANTICS: - On <user>/personal or <user>/dev: deletes the (branch_id, name) row from function_defs. The env rehydrates so the name now resolves to the inherited (system/main) binding, or becomes unbound if main has no such name. - On system/main: ERROR. Unbind never deletes a system/main row. (system/main is the inheritance root; removing from it would affect every user.) - If no branch-local row exists for name: ERROR (nothing to unbind). This prevents accidental no-ops and makes the caller explicit. - Never deletes rows from other branches. - Routes (view_bindings) are out of scope for v1; unbind is function_defs only.

INTERACTION WITH PACKAGE PINS: load-library upserts/pins package bindings onto the current branch. (unbind name) on a pinned binding is an unpin – it removes the branch-local override, revealing the system/main package binding. A future (unbind-package pkg) could drop a whole pin; out of scope for v1.

AUTH: admin or the branch owner (consistent with checkout / promote gating). Anonymous sessions cannot unbind.

NON-GOALS: - Unbinding routes (view_bindings) – follow-up. - (unbind-package pkg) bulk unpin – follow-up. - Unbind on system/main (always errors).

package-updates

Authority model: live Postgres Nopales, git kernel + vault, additive seed
[status: implemented]

– *agent note:* 2026-08-02: dual-era authority is operational – seed-policy additive, pp_sync vault, promote path, personal pins survive deploy. Matches AGENTS.md.

==========
DUAL-ERA AUTHORITY (intentional, long-lived) ==========
Destination: stable OCaml core; most product change is live define/promote on the habitat. Until the core is calm, both surfaces exist – but each has one job.

THREE CHANNELS:

1. KERNEL – OCaml binary (HTTP, prims, loader, chrome). Source of truth: git. Update: push ? build ? restart. No branch involved.

2. PACKAGE DEFAULTS – function_defs on system/main. Source of truth while running: Postgres system/main. Daily update path: promote from personal/dev (see promote). Boot seed is ADDITIVE only (see seed-policy): fills missing names from libs/*.pp; never overwrites an existing main row. Explicit reseed is the only git?main clobber.

3. GARDEN / WORKSHOP PINS – rows on personal, dev, or other children. Never written by boot seed. Survive every deploy. Shadow main until unoverride or promote+unoverride.

GIT'S TWO JOBS (do not mix): A. Kernel repo – authoritative for OCaml/migrations. B. Nopales vault – exported snapshots of packages/ branches (backup, bootstrap, clone-to-laptop). Not the daily Nopales editor. See lisp-snapshot.

DATA (data_rows) is never versioned by branch and never wiped by deploy. New package code must tolerate older row shapes or ship an explicit data migrator.

DEFAULT WORK LOOP (product Nopales): work on personal or dev on the ONE habitat (prod) ? dogfood against real shared data ? promote selected names ? system/main ? optional snapshot/export into repo (vault) ? optional unoverride so inheritance is clean Local dev.sh PG is kernel-lab only, not daily Nopales.

UPDATE RULE (after additive seed): new name in libs, absent on main ? appears on seed existing main name ? unchanged by seed personal/dev override ? still shadows main want git body on main ? explicit reseed want garden body on main ? promote

CONSCIOUS UPDATE UX (branch browser; partial today): per name: Use main (unoverride) | Keep | Diff per package: Reset garden pins to main (bulk unoverride) load-library on non-main: warn that it pins the package

Anti-patterns: - boot seed overwriting existing main defs - reseed onto personal/dev - auto-merge main?personal on boot - full copy-on-fork for personal (freezes forever) - treat data_rows as package payload - editing libs/*.pp as the primary product path while also defining the same names live without promote (split brain)

> *[verify] build: dune build image cli; smoke: boot seed does not change body of existing system/main fn; smoke: boot seed inserts a libs name missing from system/main; smoke: personal define survives image restart without pp_sync*

seed-policy

Additive boot seed into system/main; explicit reseed may clobber

[status: implemented]

PROBLEM TODAY: persist_function is INSERT ... ON CONFLICT DO UPDATE. seed_stdlib load-library therefore CLOBBERS every system/main name present in libs/*.pp on every boot. Live promote to main is not durable across deploy.

OPTION A (chosen):

1. ADDITIVE BOOT SEED seed_stdlib still runs on boot, still targets only system/main, still reads PP_LIBS_DIR/<pkg>/lib.pp. Persist path used by seed MUST NOT overwrite: INSERT ... ON CONFLICT (branch_id, name) DO NOTHING or equivalent "insert if absent" helper (Store.persist_function_if_absent / seed mode flag). Missing package files: skip + log (unchanged). Routes/deftype seeded from libs: same additive rule where a natural key exists (branch_id, route|name).

2. EXPLICIT RESEED (git ? main clobber, admin only) Prim: (reseed-package "notes") optional (reseed-package "notes" 'force) same. Behavior: - require admin (same gate as promote to system/*) - read libs/<pkg>/lib.pp via loader - eval/persist onto system/main with UPSERT (full overwrite for names in that file) - does not touch personal/dev rows - rehydrate callers as needed CLI: pp-sync reseed <pkg> (calls the prim) Optional later: pp-sync reseed –all

3. load-library / load-string (interactive) Unchanged intent: write the CURRENT branch with upsert. Still the pin hammer. Not used by boot seed once seed has its own absent-only persist. Warn when current branch is not system/main (package-updates UX; may land with change-set).

4. FRESH DATABASE Additive seed on empty main = full bootstrap from libs/*.pp. No separate first-boot mode required.

5. NON-GOALS - seed writing personal/dev - auto reseed on git push without admin prim - deleting main names absent from libs (additive never removes; cleanup is manual or a future prune tool)

> *[verify] build: dune build image cli; smoke: after promote name to main, restart leaves body intact; smoke: (reseed-package pkg) admin overwrites main from libs; smoke: non-admin reseed-package errors; smoke: reseed-package does not write personal rows*

promote

Graduate garden/workshop code to system/main; optional vault export

[status: implemented]

Promote is the blessed path for product Nopales. Git export is optional vault/checkpoint, not required for main to keep the change across deploys (seed-policy).

A. LIVE MAIN (required for shared defaults) Selective upsert of named function_defs from a source branch (personal/dev/?) ? system/main. Already: Session.promote_to_main, Store.promote_functions, POST /branches/promote (admin only into system/*). Also: (merge src dst) for coarser branch merges; promote remains the selective UI/prim path.

ROUTES TRAVEL WITH THEIR HANDLER: promote MUST also upsert any view_bindings row on the source whose handler names a promoted function (preserving visibility). A promoted handler without its route leaves main serving the old binding – the split-brain footgun this rule exists to kill. Routes whose handler is NOT in the promote set do not travel. promote does not touch type_defs (schema-authority).

B. VAULT EXPORT (optional durability in git) export-package / pp-sync pull writes libs/<pkg>/lib.pp (or snapshot tree – see lisp-snapshot). Operator commits. This does NOT change live main by itself. Useful for backup, laptop clone, and explicit reseed.

HAPPY PATH: edit on personal or dev ? dogfood ? promote selected names ? system/main ? optional unoverride on source (clean inheritance) ? optional pp-sync snapshot/pull + git commit (vault)

SELECTIVE: Ship named functions (ui/nav, notes/save, ?), not the entire garden by default. UI: name multi-select on branch detail (exists); package-group select may come later.

PRIM / CLI (v1 make): keep promote UI + Session.promote_to_main optional Nopales: (promote-to-main "name") or (promote-to-main '("n1" "n2")) on current branch pp-sync does not promote (image policy stays in image)

Policy: - promote/merge into system/*: admin only - merge into own branches: free - data_rows never part of promote - image never git-pushes (no repo credentials)

> *[verify] build: dune build image; smoke: POST /branches/promote copies selected names to system/main; smoke: promoted main body survives restart under additive seed; smoke: non-admin promote to system/main denied; smoke: promote of a handler fn also copies its view_bindings row; smoke: view_bindings whose handler is not promoted stay put*

code-history

Append-only before-image history for code rows, captured by Postgres triggers

[status: implemented]

PROBLEM: Every write to function_defs / type_defs / view_bindings is a destructive INSERT ... ON CONFLICT DO UPDATE. Redefine a function and its old body is unrecoverable unless someone happened to pp-sync snapshot first. Merge and promote clobber the same way. There is no history anywhere in the system.

DESIGN: HISTORY TABLES + TRIGGERS (capture below every write path)

code_history(id bigint GENERATED ALWAYS AS IDENTITY, rel text, – 'function_defs' | 'type_defs' | 'view_bindings' branch_id uuid, name text, – fn/type name or route path body jsonb, – superseded before-image (fn/type body) author text, doc text, package text, op text, – 'UPDATE' | 'DELETE' changed_at timestamptz DEFAULT now()) index on (rel, branch_id, name, changed_at DESC)

One BEFORE UPDATE OR DELETE FOR EACH ROW trigger per live table inserts OLD into code_history. Triggers, not OCaml call-site instrumentation, because every write path converges on these tables: define, merge, promote, reseed, boot seed, and even raw psql. A trigger cannot be bypassed by a new call site.

TRANSACTIONALITY: the trigger runs in the same transaction as the write. A rolled-back merge leaves no history rows; a committed merge leaves one per clobbered row. History is therefore exact, never optimistic.

SEMANTICS: - INSERT fires nothing: first definition has no before-image. Version N of a name = live row plus N-1 code_history entries walking changed_at DESC. - ON CONFLICT DO UPDATE fires the UPDATE trigger for conflicted rows only – merge/promote history is exactly the clobbered set, for free. - INSERT ... ON CONFLICT DO NOTHING (additive seed) writes nothing and records nothing. - op='DELETE' rows are how unoverride/branch-drop remain recoverable.

NOPALES PRIMS (read-only): (code-history "name") – current branch (code-history "name" "owner/branch") ? JSON list, newest first: {rel, op, author, doc, changed_at, body} (code-diff "name" n) – current row vs nth-newest history body (unified-ish text diff) Restore is NOT a prim in v1: read the old body and define it back (that define itself lands in history, so restore-by-redefine is lossless).

SCOPE / NON-GOALS (v1): - CODE ROWS ONLY. data_rows history is a separate later decision (storage commitment; shared table). - No retention pruning; review table size later. - No history UI in v1 beyond the prims; the branch-browser Changes view may surface it later. - tree_nodes mirrors are derived state; history lives on the authoritative tables only.

> *[verify] test: image/test/history_test.ml; build: dune build @all; smoke: redefine a fn on personal; (code-history name) returns the old body; smoke: merge that clobbers target rows records one history row per clobber; smoke: additive boot seed records no history rows; smoke: rolled-back transaction leaves no history rows*

land

pp-sync land: promote + vault snapshot + git commit as one operator step

[status: implemented]

PROBLEM: The blessed loop (work on personal/dev ? promote ? snapshot ? git commit) has manual steps 3-4. Skip them and live main drifts from libs/ silently; pp-sync status can see the drift but nothing runs it.

LAND – the one-command ship path: pp-sync land <pkg> [name ...] [–from owner/branch] performs, in order, from the operator's laptop: 1. CHECKOUT-INDEPENDENT READ: export the named functions (or the whole package when no names given) from the source branch (default: the session's current branch). 2. PROMOTE: call the promote prim for those names (admin gate unchanged; routes travel per promote). 3. SNAPSHOT: re-export the same names from system/main and write libs/<pkg>/lib.pp. 4. COMMIT: git add libs/<pkg>/lib.pp && git commit with message "vault(<pkg>): land <names>". The image never git-pushes and never holds repo credentials; land runs where the repo lives. 5. REPORT: print promoted names, conflicts the promote overwrote (visible via code-history), and the commit SHA.

Failure stops the pipeline at the failed step and reports state (nothing after a failed promote is snapshotted; a failed commit leaves main promoted and the vault dirty, printed clearly).

DRIFT VISIBILITY: pp-sync status gains an exit code: 0 clean, 1 drift. The repo monitor command (README) runs pp-sync status so drift shows up on the dashboard instead of being discovered by accident.

NON-GOALS: - land does not merge branches (promote only) - land does not push to remotes - land does not unoverride the source branch (stays an explicit, separate act) - no daemon; land is operator/agent-invoked

> *[verify] build: dune build cli; smoke: pp-sync land notes ui/nav promotes, snapshots libs/notes/lib.pp, commits; smoke: failed promote (non-admin) leaves vault untouched and reports; smoke: pp-sync status exits 1 when libs/ differs from live main*

lisp-snapshot

Clone habitat Nopales state into the git repo as vault/backup

[status: implemented]

PURPOSE: Postgres is the live Nopales store. The repo must still be able to hold a clone of package/branch state for backup, new-machine bootstrap material, and explicit reseed inputs – without becoming the daily editor.

V1 SURFACES (compose existing export):

1. Package pull (exists) (export-package "notes") ? lib.pp text pp-sync pull <pkg> ? libs/<pkg>/lib.pp pp-sync pull-all ? every discovered pkg Default export reads the SESSION's current branch visibility (CTE). For vault of defaults, checkout system/main first OR pass an explicit branch (v1 make: add optional branch ref).

2. Snapshot command (new CLI) pp-sync snapshot - checkout-aware: export all packages with bindings on the chosen branch - writes libs/<pkg>/lib.pp by default (main vault) pp-sync snapshot –branch system/main pp-sync snapshot –branch admin/personal \ –out backups/personal/ - garden pins land under backups/ not libs/ so bootstrap seed does not treat pins as defaults

3. Image prim (thin) (export-package pkg) already returns text. Optional: (snapshot-packages) ? list of pkg names with own or visible bindings (helper for CLI). No git commit inside the image.

4. Operator habit Before risky reseed or migration: snapshot main (+ personal if desired) and git commit. Cadence backups may be cron + pp-sync snapshot.

LAYOUT CONVENTION: libs/<pkg>/lib.pp bootstrap + main vault (seed input) backups/<ref>/... non-main branch dumps (not seeded)

NON-GOALS v1: - automatic git commit from image or CLI - pg_dump of data_rows as part of pp-sync - encrypting snapshots - bidirectional sync daemon

> *[verify] build: dune build cli; smoke: pp-sync pull notes writes libs/notes/lib.pp; smoke: pp-sync snapshot –branch system/main refreshes libs; smoke: pp-sync snapshot –branch u/personal –out backups/personal does not write libs*

branch-browser-ui

Git-like light /branches browser: list, open, stats, checkout, promote

[status: implemented]

> – *agent note:* 2026-08-02: http_branches.ml ~710 lines – list with own code/overrides, detail, checkout, fork, promote to main + soft lint. Spec "TODAY bare table" is stale.

TODAY: GET /branches is a bare table (owner/name, parent uuid prefix, created). No checkout, stats, detail, or promote.

GOAL: tiny Gitea-ish branch page in classic htmx chrome – browse, open, stats, checkout, later promote – no SPA.

ROUTES: GET /branches GET /branches?ref=owner/name detail POST /branches/checkout ref=? POST /branches/fork from + name POST /branches/merge src + dst (later) POST /branches/promote selective (later)

LIST PAGE: Header strip: On: <current> ? + quick links Table columns: ? | branch | parent | own code | overrides | routes | age | actions own code = COUNT function_defs WHERE branch_id (no CTE). overrides = own names also on parent. actions: Open ? Checkout (disabled if current) Labels: system/main "defaults"; */personal "garden" Create card: fork-from select + new name + Create (full fork for experiments; personal stays empty)

DETAIL PAGE (?ref=): Title, parent link, Checkout, back to list Stats: own fns, overrides, own routes, visible fn count (CTE), last define, authors Sections (htmx partials): Overrides (default for personal) – name, author, updated, new|override, view, diff All own code Routes (own view_bindings) Activity / Changes: branching.change-set (ORDER BY updated_at on own defs + transcript) Diff vs parent v1: two <pre> columns of pp_expr (identical / changed / only-here / only-parent)

CHECKOUT: POST ? Session.checkout_to ? 303 /branches or HX-Redirect. Defines already persisted; no "uncommitted" confirm.

PROMOTE PANEL (detail, admin): package multi-select + name multi-select [ ] Merge into system/main [ ] Mark for export (pp_sync pull list) Never dump whole garden by default.

UPDATES BADGE (package-updates): branching.branch-status supersedes the updated_at badge: status = behind | synced tells accept-vs-keep by CONTENT (behind = main moved, offer accept/unoverride; synced = identical pin, offer unbind).

STORE HELPERS (own-branch, no CTE unless noted): count_functions_on_branch count_routes_on_branch list_own_function_names branch_status (branching.branch-status: clean/ahead-new/ahead/behind/diverged/ synced – replaces the retired list_overrides name-presence join) lookup_function_on_branch (no CTE) delete_function_on_branch (unoverride)

NON-GOALS (light): git SHAs/commits, PR review UI, blame, per-branch data, fancy fork-tree viz

> *[verify] build: dune build; smoke: GET /branches shows current ? and checkout; smoke: POST checkout switches current-branch; smoke: detail lists personal overrides vs main; smoke: detail Changes section lists own defs by updated_at*

change-set

Branch change set / transcript: what changed on this branch

[status: implemented]

> *[verify] build: dune build image; smoke: GET /changes?ref=u/personal lists own fns by updated_at; smoke: (branch-changes) returns own function names + updated_at on current branch; smoke: after define on personal, name appears at top of changes; smoke: change row links to /edit and /bindings detail + parent diff when override*

==========

WHY (Smalltalk change set / method transcript) ==========

Combo pack item #2. After live defines, humans and agents need "what did I change on this garden?" without git. Not a full VCS – a living transcript of own-branch code rows + optional session ring of recent defines.

Compounds with: - image.inspector impact-queue (same define hook) - image.system-browser detail ? link here for overrides - branching.branch-browser-ui detail Activity section - promote / package-updates (know what is dirty)

==========
A. DERIVED CHANGE SET (no new table required v1) ==========
Source of truth already exists: Store.list_own_functions ~branch_id ? (name, author, updated_at) Store.list_own_routes ~branch_id Store.branch_status ~branch_id ? clean/ahead-new/ahead/behind/diverged/ synced per name (branching.branch-status; supersedes the retired list_overrides new|override name-presence classes)

"Changes" for branch B = own function_defs ? own view_bindings on B, sorted by updated_at DESC (activity order). Classify each fn row: - new: no parent row of same name - override: parent has name (optionally note if parent.updated_at > child ? main-newer badge) - route: own view_bindings entries (path ? handler)

Nopales prims (read-only, current branch default): (branch-changes) ? list of dict/JSON strings {name, kind: fn|route, author, updated_at, class: clean|ahead-new|ahead|behind| diverged|synced} (branching.branch-status; was new|override|route – superseded 2026-08-28, content-based now) (branch-changes "owner/name") optional other branch if visible to user (same rules as /branches detail)

==========
B. SESSION TRANSCRIPT (define hook) ==========
Same write path as impact-queue (persist_define / install_function success): append {name, at, branch, author, op: define|route|delete} to session ring (last N=64). Optional; derived list from PG is enough if ring is empty after restart.

Nopales: (change-transcript) ? recent session ops Do not require durable data_rows log in v1.

==========
C. UI SURFACES ==========
1) Branch detail (primary): GET /branches?ref=owner/name&view=changes or default Activity section expanded for personal Table: when | name | class | author | open | diff open ? /bindings?pkg=&fn= or /edit/<name> diff ? existing ?diff= two-column pp_expr

2) Optional GET /changes on current branch (short alias redirecting to /branches?ref=<current>&view=changes)

3) system-browser function detail: if name is own-row, badge "on this branch" + link into Changes highlight

==========
D. AGENT ==========
After a multi-define session, agent can (branch-changes) or read /changes to summarize dirty garden before promote. install_function result may append "changeset: N own fns" lightly – impact text remains primary for smoke routes.

==========
E. NON-GOALS ==========
- Per-edit revision history table (later; updated_at only) - git commits / SHAs / blame - Diff of data_rows (code only) - Cross-user global firehose - Blocking define on "uncommitted" state

==========
F. PHASES ==========
C1: UI Activity/Changes from list_own_* + overrides class C2: (branch-changes) prim + /changes alias C3: session transcript ring on define (shared hook w/ impact) C4: browser detail deep-link + agent one-liner

SUCCESS: on personal after editing ui/nav, Changes shows ui/nav at top as override; diff vs main works; agent can list dirty names without scraping HTML.

ast-nodes

Design: addressable AST nodes as substrate for surgical edits and AI patches

[status: planned]

TODAY: two trees, two models - Branch tree: rows in branches, fork/merge, recursive CTE lookup, unit = whole function - AST tree: JSONB blob in

function_defs.body, in-memory eval walk, no node ids, edit = replace whole body

DESIGN (2026-08-16, ralph loop 19, BORGE-1786925842): Give AST nodes ADDRESSABILITY so tools can patch subexpressions and a genuine THREE-WAY MERGE can be computed, reusing the diff spine already shipped (branching.diff / lisp-interpreter.ast-diff). Kept cheap: identity is a PATH computed on the fly, NOT a materialized table. A materialized nodes table stays OPTIONAL and DEFERRED (see non-goals).

========= 1. NODE ADDRESSING (no table) ========= A node inside a whole-expr body is addressed by a PATH from the root, in the same vocabulary the ast_diff walk already labels its children with: positional: if.test if.then if.else lambda.params lambda.body let.name let.val let.body define.name define.val seq.a seq.b quote.body list: app.fn app.arg[i] prim.arg[i] letrec.binding[NAME] lambda.param[i] Example path: lambda.body.app.arg[1].app.fn.var Identity = (row key: branch_id, name) + path. Pure helpers in lisp/lib make nodes addressable with NO schema change (function bodies stay whole-expr JSONB): select_path : expr -> path -> expr option replace_path : path -> expr -> expr -> expr

========= 2. PATCH REPRESENTATION ========= A two-way diff (base -> target) flattens to a set of atomic path-keyed edits, exactly what Ast_diff.delta already produces labelled by the child paths above: Replace(path, from, to) node at path replaced Insert(path, idx, expr) list-child insert Delete(path, idx) list-child remove The Replaced <from=absent> sentinel in ast_diff already encodes Insert/Delete at list positions. A patch is therefore diff(base,target) PLUS the path vocabulary - no new algorithm, only the missing path->node selector from part 1.

========= 3. THREE-WAY MERGE (base, ours, theirs) the algorithm tree-time (campaign 5) and local-first sync (campaign 8) both reuse ========= ours = diff(base, ours) theirs = diff(base, theirs) Group edits by node path. Disjoint paths compose with no cross-order. A same path edited by BOTH: - identical target -> keep once (both agreed) - different target -> CONFLICT: report path + ours value + theirs value; v1 resolution = keep-ours + report (consistent with branching.merge's report-only-conflict philosophy); v2 may offer 3-way resolution. Apply the merged edits to base -> merged body, then re-validate (ast_hash equality / eval smoke). A merge always has a base: the fork point, branches. last_synced, or a code_history before-image.

========= 4. MATERIALIZED NODES TABLE (optional, deferred) ========= For SURGICAL in-place patching of one huge function (AI rewrites a subexpression without rewriting the whole JSONB), a materialized nodes(id, branch_id, kind, payload, parent_id, position, ...) table pays off - derived/mirrored from the expr on write (like the tree_nodes mirrors), keyed by (branch_id, name, path). DEFERRED: the non-goal below (whole-function propose/edit ships first) says it is not needed yet, and it adds schema + mirror + writer complexity.

========= 5. LAYERING ========= P1 (this substrate): pure path addressing + replace_path + merge3 in lisp/lib, testable with dune runtest lisp (no Postgres). P2: wire into Store.merge_branch - replace the naive full-copy with diff-based merge + a conflict report using merge3 (this is branching.merge's open TODO). P3 (deferred): materialized nodes table for surgical in-place patching.

IMPLEMENTED so far: the diff spine this design consumes (branching.diff + lisp-interpreter.ast-diff) is shipped (module ast_diff, /diff page, (diff from to) prim). This subsection fixes the AST-node addressing + merge3 design those consumers build on; the P1 merge3 module and P2 merge wire-in are the remaining make work (see BORGE-1786925842).

Non-goals for first product cut (unchanged): - Do not block function-editor or ai-authoring tools on this redesign - Whole-function propose/edit ships first - No alpha-equivalence / rename detection in the merge (inherits the diff spine

non-goals)

– *agent note:* 2026-08-16 design pass (ralph loop 19): nodal addressing + three-way merge are now fixed; materialized nodes table deferred. Re-filed issue BORGE-1786925842. 2026-08-02: design-only proposal; function_defs remain whole-body JSONB.

8 borge/lisp-interpreter.borg

pricklypear-lisp-interpreter

Pricklypear book chapter: lisp-interpreter
[status: implemented]

> – *agent note:* 2026-08-02 status reconcile: eval TCO+budget + macros (defmacro/qq/expand) shipped; dune runtest lisp 123 OK; define/defmacro persist synchronous.

lisp-interpreter

Pure interpreter for Nopales (a Scheme-flavored Lisp-1), AST-as-source, primitives layer
[status: implemented]

language-roadmap

Nopales trajectory phases 1-3: code-as-data law, dialect ergonomics, production capabilities
[status: implemented]

> *[verify] smoke: phase exit criteria listed per phase; phases 1-2 and public-route half of 3 are green; test: conformance suite pins phase 1-2 surface*

NORTH STAR Nopales is a small Scheme-family impure functional language for a live multi-user image (humans + agents + Postgres). Not pure FP. Not R7RS-complete. Not a static-typed ML.

STATUS Phase 1 EXIT – symbols + errors-and-locations implemented. Phase 2 EXIT – binding-sugar, match, std-library, guard. Phase 3 PARTIAL EXIT – public Route_gate + package-surface + budgets enforced; broader per-agent tool masks remain additive follow-on (see capability-enforcement). core-freeze contract + conformance suite enforce the seam. Phase 4+ (modules-lite / hygiene / maps): still out of scope until real dogfood pain.

PHASE 1 – HONEST CODE-AS-DATA (done) symbols; errors-and-locations (kinds + parse loc + guard). Exit: 'a vs "a"; symbolic AST heads; structured errors; dune runtest lisp green.

PHASE 2 – PLEASANT DIALECT (done) binding-sugar; match; std-library freeze; guard catch. Exit: define (f x)?; begin multi-body; match on trees; frozen list/string/symbol helpers; guard recovers.

PHASE 3 – PRODUCTION CAPABILITIES (public gate done) effects inference + Route_gate public allowlist; package-surface exports; eval budgets hard leash. Remaining additive: denser agent-session effect masks.

DECISION DEFAULTS (locked) - Impure functional Scheme-family dialect - Dynamic typing + effects + data schemas (not HM types) - Unhygienic macros + gensym (no syntax-case in 1-3) - No call/cc - No in-Nopales fiber/async API (host owns concurrency) - Branches remain the override model; modules-lite = P4+ - JSON/dict prims for records; no VMap required in 1-3

core-freeze

Nopales core 1.0 freeze: the frozen language contract and the slow-change rule that produces long-term stability
[status: implemented]

> *[verify] smoke: core 1.0 frozen-surface list is enumerated and matches the reader, value, special-form, primitive, effect, and error chapters; smoke: a proposed core change is classifiable as additive, deprecate-then-remove, or break against this rule; test: conformance suite exists, is service-free, and runs green on a clean core*

THE CONTRACT. This subsection is the slow-change law for Nopales core. Phases 1-2 of language-roadmap and the public route half of Phase 3 have exited; the frozen-surface list below is the working core 1.0 contract enforced by the conformance suite. Further additive work is allowed; silent meaning changes are not.

FROZEN SURFACE (core 1.0): - Reader surface syntax: s-expressions, quote/quasiquote/ unquote/unquote-splicing shorthand, parse errors with line and column. - Value model: VStr, VInt, VFloat, VBool, VSymbol, VPair, VList, VLambda, VMacro, VPrim, VRef, VNil, and the JSON codec that serializes values/exprs into function_defs. - Special forms: quote, quasiquote, unquote, unquote-splicing, define (incl. function sugar + doc), lambda, if, let, letrec, begin/seq, match, guard, error, defmacro, cond, and, or, apply, list/map|filter|foldl| foldr, gensym, macroexpand*, with-transaction, and the image control forms checkout/fork/merge/route when bound. New special forms may be added; existing meanings do not change in place. - Pure/primitive set: arithmetic, comparison, list, string, symbol, predicate, type-of, whoami, current-author, dict-*, and the std-library freeze table. Optional args with defaults OK; silent return-shape change is a break. - Effect lattice: capability tags, infer, policy_allows, Route_gate public allowlist. Adding a tag is additive; redefining an existing tag's meaning is a core bump. - Error kinds + locations (interp_error_kind, Parse_error line:col, guard catch surface, budget uncatchable).

SLOW-CHANGE RULE. A core change is one of: - ADDITIVE: new prim / special form / effect tag / optional arg with default preserving old behavior. Always allowed. - DEPRECATE-THEN-REMOVE: add new name/behavior, keep old under deprecation, remove only across a named release. - BREAK: only with explicit core version bump (Nopales 2.0) and a migration path for product code.

PRODUCT LAYER IS NOT FROZEN. libs/*, user branches, routes, themes change freely via define/route (OCaml slow, Nopales fast). The freeze is the seam that makes the fast layer safe.

ENFORCEMENT. lisp/test/conformance_test.ml – real Nopales programs with expected values, service-free, in dune runtest. A reddened case does not land until the suite is updated intentionally and the change is classified under this rule. borge drift guards structure; conformance guards semantics.

VERSIONING. Nopales core versioned separately from the Pricklypear framework; coupled only at a major core bump.

FOLLOW-ONS (additive, not freeze blockers): - Broader per-agent effect_mask on every tool prim - symbols phase C (strict string-eq) - denser product-package pure snippets in conformance

conformance

Conformance suite: real Nopales programs with expected eval results, run on every core change as the semantic gate of the freeze

[status: implemented]

> *[verify] test: suite loads and evaluates each program against its expected value with no services; test: a deliberate core change that alters a program's result is caught as a failure; build: dune runtest lisp includes conformance*

WHAT IT IS. A set of real Nopales programs, each paired with the value it must evaluate to, run by the pure interpreter with no services. Tier between OCaml unit tests and HTTP smokes: the LANGUAGE gate of core-freeze.

LOCATION. lisp/test/conformance_test.ml – alcotest cases of the form (name, program, expected-pp-value) plus a few multi-form setup programs. Runs under dune runtest lisp.

CONTENTS (v1 frozen-surface cases): - reader: quote, quasiquote/unquote, lists, strings, bools, string escapes (named, \\ddd byte, \\uXXXX unicode

scalar) - values: VInt VStr VSymbol VBool VNil VList distinctness - special forms: define sugar, lambda, let, if, begin, match, guard, cond, and/or, defmacro/gensym - pure list HOF: list/map list/filter list/foldl list/foldr - errors: unbound kind; guard recovers; budget uncatchable - symbols: 'a vs "a"; car of quoted lambda is symbol Product-package headline cases that need DB/HTTP stay in the smoke tier (libs/verify). Pure kernels of those packages may be added here as pure snippets over time.

RUN MODEL. Service-free pure interpreter only. A reddened case blocks core change land until the suite is updated intentionally and reclassified (additive / deprecate / break per core-freeze).

SCOPE. Pure eval results only. DB prims and routes ? smoke.

ast

The expr and value types

[status: implemented]

type expr = | Var of string | Lit of value | App of expr * expr list | Let of string * expr * expr | Letrec of (string * expr) list * expr | If of expr * expr * expr | Lambda of string list * expr | Prim of string * expr list | Seq of expr * expr | Quote of expr (* data quote; see macros *) | Quasiquote of expr (* macros.quasiquote *) | Unquote of expr | UnquoteSplicing of expr

type value = | VInt of int64 | VFloat of float | VStr of string | VSymbol of string (* see subsection symbols *) | VBool of bool | VNil | VPair of value * value (* cons cells *) | VLambda of string list * expr * env | VMacro of string list * expr * env (* defmacro transformer; params bind to UNEVALUATED arg exprs reified as values via quote_to_value; body returns a value tree that value_to_expr turns back into an expr for re-eval. See macros. *) | VPrim of string (* primitive name *) | VRef of type_name * id (* tagged ref into store *) | VList of value list (* sugar for nested pairs *)

The AST IS the source (homoiconicity). Stored as JSONB in function_defs.body. The reader (s-expr parser) produces expr values; the printer round-trips them. No parser/printer asymmetry.

Closures capture their defining env (lexical scope). VLambda carries env. No dynamic scope.

VRef carries its target type so the interpreter can verify derefs at call time before querying the store.

VALUE ATOMS vs CODE NAMES: At the expr layer, Var s and Lit (VStr s) are distinct. At the value layer, VSymbol holds code names and VStr holds string data (see subsection symbols). Quote / read-string / function-ast trees match Scheme-family code-as-data.

ast-diff

Pure structural diff over expr trees (consumer: branching.diff). Not the surface prim – the pure walker.

[status: implemented]

> – *agent note:* 2026-08-16 ralph loop 19: implemented + verified (lisp/lib/ast_diff.ml, 27/27 ast_diff tests green, full runtest exit 0). Pure module, no I/O, tests with no services (dune runtest lisp). Depends on the frozen expr type (core-freeze) and analyze.ast_hash for the equality fast-path. Semantics authority: branching.diff – this subsection is the module home, not the semantics.
>
> *[verify] build: dune build lisp; test: lisp/test/ast_diff_test.ml – same/modified/replaced over all 11 expr constructors; test: expr_to_json round-trip preserved: diff never mutates, only reads*

Pure walker over the expr type (see ast). No I/O, no Postgres – tests with dune runtest lisp alone, like reader / eval / env.

type delta = | Same | Replaced of { from: expr; to_: expr } | Node of { ctor: string; children: (string * delta) list }

val diff : Ast.expr -> Ast.expr -> delta

Per-ctor recursion (fixed-arity positional for If/Seq/Lambda/Let/Define/Quote; list-LCS for App.args and Letrec.bindings) and non-goals (rename detection, alpha-equivalence, param order) are spec'd in branching.diff – this subsection is the module home, not the semantics authority.

STAKEHOLDER on core-freeze: ast_diff is a structural consumer of every expr constructor. Adding a constructor to expr requires a diff case here (exhaustive match) – borge drift will reddens a missed case, but the freeze contract (see core-freeze) naming a reason the expr type is slow-change now includes this module.

symbols

Distinguish symbols from strings at the value layer (undo v1 collapse)

[status: implemented]

> *[verify] test: quote hello is symbol; quote "hello" is string; not equal; test: value_to_expr round-trip preserves Var vs Lit VStr without ___str___; test: gensym returns VSymbol; test: symbol? string? symbol=? symbol->string string->symbol; test: type-of distinguishes symbol and str; smoke: function-ast / read-string heads are symbols; ui/home still renders*

SHIPPED (phases A + B). Code-as-data trees use VSymbol for names; strings stay VStr. Collapse is REJECTED.

VALUES (quote hello) => VSymbol "hello" (quote "hello") => VStr "hello" (symbol? 'lambda) => #t (string? "lambda") => #t (symbol=? 'a 'a) => #t (car (read-string "(lambda (x) x)")) => symbol lambda

TYPE + PRINT/READ OCaml: | VSymbol of string JSON values: {"sym":"hello"} vs {"str":"hello"} expr JSONB unchanged ({var}/{str} already distinct) – no migration of function_defs bodies. Printer: VSymbol bare; VStr double-quoted. Reader: bare token -> Var; "?" -> Lit VStr.

BRIDGE quote_to_value: Var -> VSymbol; Lit VStr stays VStr; App heads that were Vars become VSymbol heads in VList. value_to_expr: VSymbol -> Var; VStr -> Lit VStr (never Var). reify_expr / macros use quote_to_value (no ___str___ primary). gensym returns VSymbol (optional string prefix).

PRIMITIVES (image prims_pure; registered at boot) symbol? string? symbol=? symbol->string string->symbol type-of -> "symbol" | "str" | ? as-str ACCEPTS VSymbol (HTML/path ergonomics) string-eq: phase-B compat still coerces via as-str/coerce_str so product packages do not mass-break. Prefer symbol=? for AST car walks. Strict string-eq (reject symbols) is deferred cleanup (phase C), not required for this section.

DEFERRED (phase C cleanup – non-blocking) - Remove residual ___str___ unwrap paths once no inflight macros - Strict string-eq (VStr only) - value_to_expr: refuse VStr heads without string->symbol

NON-GOALS - Full interning / eq identity across processes - Keyword type (:foo) - Changing expr JSONB shape

SUCCESS 'a and "a" are different; code-as-data heads are symbols; macros need no ___str___; symbol prims exist; product home and pure lisp tests green. No DB migration.

errors-and-locations

Structured interpreter errors with source locations; catch surface for packages

[status: implemented]

> *[verify] test: parse error includes line:col; test: unbound var error includes name and optional span; test: guard/with-handlers catches User_error and continues; test: guard does not catch Budget_exceeded; smoke: /api/eval error JSON includes kind and loc fields when available*

SHIPPED SURFACE (v1)

Host signal remains OCaml Interp_error. Every common failure has a stable kind via interp_error_kind (snake_case): unbound_var | not_a_function | arity_mismatch | wrong_arg_count | type_mismatch | budget_exceeded | prim_error | parse_error | user_error

type interp_error = | Unbound_var of string | Not_a_function of value | Arity_mismatch of string * int * int | Wrong_arg_count of string * int * int | Type_mismatch of string * value | Budget_exceeded | Prim_error of string * string | Parse_error of string * int * int (* msg, line, col *) | User_error of string

Locations: - Reader_error / Parse_error carry line:col (reader state). - Unbound_var carries the name; optional span on every expr is deferred (not required for v1 catch/API). - interp_error_loc returns Some (line,col) for Parse_error and None otherwise.

/api/eval error JSON (additive fields; .error always present): { ok: false, error: "?", kind: "unbound_var", name: "foo", ; when Unbound_var line: 12, col: 4 } ; when loc known Old clients that only read .error keep working.

CATCH SURFACE – special form guard (not with-handlers): (guard handler-lambda body?) Handler is a 0- or 1-arg lambda. On Interp_error (except Budget_exceeded, which always re-raises), the 1-arg form receives a tagged list: (error <message-str> <kind-symbol>) Nested guard is OK. Handler return value becomes the result of the guard form.

(error msg) | (error kind msg): raises User_error. Kind+msg concatenates as "kind: msg" in the message string; structured kind on the OCaml side remains user_error (v1).

RELATED – image.dnu-stubs (product holes, not catch) doesNotUnderstand-style stub+callers UX reuses unbound_var kind/name fields from this JSON but does not replace guard.

DEFERRED (not blocking implemented) - Full loc = {source; line; col} on every raise site - Optional span field on expr AST - Stack traces as first-class lists - CL-style restarts / with-handlers multi-pred dispatch

NON-GOALS - Catching host bugs / OCaml exceptions unrelated to eval - Full CL condition system

SUCCESS Common eval failures expose kind + message; parse/read paths carry line:col; packages can guard fallible work and render a friendly value without killing the page; budgets remain an uncatchable hard leash by default.

reader

The reader turns source text into AST; the AST is the source of truth (homoiconic, persisted as JSONB); read/print roundtrip fidelity is what makes stored code trustworthy.

[status: implemented]

> *[verify] build: dune build lisp; test: named + \ddd + \uXXXX escapes decode; \uXXXX UTF-8-encodes to 1-4 bytes; test: "\u203e" reads to the single-char overline ?, not literal u203e; test: lone surrogate \uD800..\uDFFF raises Reader_error; test: read/print roundtrip of a string holding ? preserves it (escape_string emits \ddd bytes); test: forgiving fallback: unknown \x still yields literal x; test: conformance pins \uXXXX decode so silent removal reddens the suite*

The reader parses Nopales surface syntax to expr values. Standard s-expressions: (foo bar baz), 'symbol, "string", 123, #t, #f, ().

Surface syntax is just an editor concern: a function stored in Postgres is its AST (JSONB), and the s-expr syntax is one of several possible editors (along with the htmx tree-editor that edits the AST directly).

Special forms recognized at read or eval time (decision: eval time, the reader is generic): (lambda (args) body) (let ((name val)...) body) (letrec ((name val)...) body) (if test then else) (define name expr) (* top-level binding *)

(quote x) (quasiquote x) (* macros; sugar ‘x *) (unquote x) (* macros; sugar ,x *) (unquote-splicing x) (* macros; sugar ,@x *) (defmacro name (params) body) (* macros *) (deftype name (fields ...)) (fork parent child) (merge source target) (app "prim-name" args) (* explicit prim call *)

Reader sugar for macros (in addition to ’x -> (quote x)): ‘x -> (quasiquote x) ,x -> (unquote x) ,@x -> (unquote-splicing x) Comma forms are only meaningful under quasiquote; bare unquote at top-level is a reader-or-eval error.

Comments: ; to end of line.

STRING ESCAPES. Inside a "..." literal the reader recognizes, in order of precedence: \n \t \r \\ \" \’ named single-char escapes \ddd exactly-three-digit DECIMAL byte (OCaml String.escaped form; the vault/export path for non-ASCII and control bytes; n must be ?255) \uXXXX exactly-four-hex-digit UNICODE scalar (see below) \x (any other char) forgiving: literal x, the backslash is dropped – editor over-escapes and paste noise do not hard-fail

\uXXXX UNICODE ESCAPE (additive; closes the "u203e" gap). Four hex digits (0-9 a-f A-F), case-insensitive. The scalar is UTF-8 ENCODED into the byte buffer (1-4 bytes), because values/AST serialize to JSONB and CSS/HTML output expects UTF-8 text, not a raw low byte. - BMP scalar U+0000..U+FFFF except the surrogate range D800..DFFF: encode directly. - Surrogate escape \uD800..\uDFFF: NOT a valid scalar alone ? Reader_error "bad unicode escape \uXXXX". v1 does not combine surrogate pairs; write the actual non-BMP char or use two-step decimal bytes if ever needed. This is a deliberate smallness call, not an oversight. Before this existed, "\u203e" fell through to the forgiving rule and became the literal 4 chars "u203e" – this was the visible menu-caret bug (libs/ui) and the degree-sign bug (libs/weather). Classified ADDITIVE under core-freeze: it only gives meaning to a backslash sequence that previously produced garbage; no existing valid program changes meaning (a prior "\u203e" was already broken).

Docstrings (see define-docstring): optional metadata on a top-level binding. Surface forms the reader already produces as plain AST (no dedicated constructor): (define name expr "doc") (define (name p?) "doc" body?) (define name (lambda (p?) "doc" body?)) The string is not a value binding; eval and the image strip it into function_defs.doc (column already on storage.schema).

eval

The evaluator: env * expr -> value

[status: implemented]

> – *agent note:* 2026-08-02: trampoline TCO, budget steps, pure eval+Prim_call; lisp tests green. define persist no longer Lwt.async.
>
> *[verify] build: dune build lisp; test: dune runtest – lisp eval + TCO cases; smoke: tail self-recursion depth » OCaml stack without overflow; smoke: non-tail recursion still correct (e.g. factorial); smoke: infinite tail loop hits Budget_exceeded, not hang/stack*

The evaluator is PURE. It takes an env and an expr, returns a value or raises an interpreter error. No I/O in the interpreter. All side effects go through Prim calls, which the IMAGE layer wires to actual Postgres/SSH/HTTP side-effects.

Cooperative evaluation: the eval loop is stepped; each Prim invocation yields back to the image’s event loop. This gives per-session budgeting: - A step counter per call (default_limit 100_000) - When exceeded, that ONE call aborts - The session continues - (budget-remaining) reads the active call’s leftover steps (dynamic current_budget) - OCaml prim bodies do not tick; only the call site This prevents "user’s bad function wedges the image" entirely. Scale bugs are fixed by cheaper paths (notes.budget-scale), not by raising the limit.

(define name expr) at top-level adds binding to current session’s env AND persists to store. Edit = redefine (update the row). No versioning at the binding level beyond what branches provide.

Optional docstring (define-docstring): trailing string after the value, or leading string in a multi-body define/lambda, is accepted by eval (not bound as the value) and persisted by the image into function_defs.doc.

Application: evaluate operator + args, apply. Primitives are dispatched by name through a table the IMAGE layer registers at boot. The interpreter knows nothing about Postgres or htmx.

Special forms handled in eval, not reader: lambda, let, letrec, if, define, quote, quasiquote, unquote, unquote-splicing, defmacro, plus higher-order list forms list/map, list/filter, list/foldl (OCaml loops + apply – not recursive Nopales frames). The reader produces App/Variants of these as plain AST (or the dedicated Quote/Quasiquote/Unquote constructors when sugar is used – either representation is fine as long as quote_to_value / expand / pp round-trip).

Macro expansion is part of eval application (and of define/load – see macros.expansion-timing). Eval must call expand before ordinary arg evaluation when the operator resolves to a VMacro.

==========

TAIL-CALL OPTIMIZATION (TCO) ==========

Proper Scheme-style TCO via an explicit trampoline in eval.ml (OCaml does not guarantee TCO on recursive calls).

Model: eval_step returns Done value | Tail (env, expr) eval / eval_force runs a loop: while Tail, continue without growing the OCaml stack.

Tail positions (must bounce, not OCaml-recurse): - if/cond: chosen branch - and/or: last subform (and short-circuit non-tails force) - seq: last form; earlier forms forced - let / letrec: body - lambda application: body after binding args (the App node is always tail in its own frame; nested Apps as arguments are forced first) - macro expansion result: evaluated in tail position - list/map|filter|foldl: stay OCaml loops; closure bodies use eval_force (each element is non-tail relative to the list form)

Non-tail positions force to a value (may use OCaml stack for the force frame only, not for the tail chain inside).

Budget: every step still ticks. Infinite tail loops hit Budget_exceeded – never hang, never exhaust OS stack.

Effects: Prim_call still perform; after return, continue the trampoline. Rim re-entry (with-transaction, apply_lambda) uses the same eval_force entry point.

Not required: full CPS, custom VM, or rest-args frames. Heap env frames still allocate per call (TCO saves stack).

macros

User-defined syntactic macros: defmacro, quasiquote, value<->expr bridge, expand-at-define

[status: implemented]

> – *agent note:* 2026-08-02: VMacro, defmacro, quasiquote/unquote, macroexpand-1/macroexpand, gensym; persist as ___macro___ wrapper; gate tests cover macro hiding create.
>
> *[verify] build: dune build lisp; test: dune runtest – lisp – macro/quasiquote/value_to_expr cases; smoke: (defmacro when (t b) `(if ,t ,b ())) then (when #t 1) => 1; smoke: top-level defmacro persists; reload expands call sites on load*

==========

0. WHY MACROS, WHY NOW ==========

Philosophy parked macros ("can come later"). The habitat is past that: product packages, AI authoring, and agents writing Nopales all want surface sugar without growing the OCaml special-form table forever. Homoiconicity is already real (AST-as-JSONB); macros are the missing half – code that writes code as data, then re-enters eval.

Goals for v1: - Users/agents can define (when / unless / and / or ?) in Nopales packages without OCaml changes - Macro bodies use quasiquote so templates are readable - Expansion is inspectable (macroexpand-1) - Persisted function bodies remain readable source ASTs (macro CALL sites stored; expand on load/define) - No hygiene engine; gensym for accidental capture

Non-goals for v1: - Hygienic macros / syntax-case / syntax-rules - Reader macros / custom dispatch chars - Compiler-style whole-program expand - Separate macro package object type (macros are ordinary bindings of value VMacro on a branch) - Macro-defining macros (allowed if it falls out; not a success criterion) - Changing the Lwt eval ABI

==========

1. SURFACE SYNTAX ==========

(defmacro name (p1 p2 ?) body) - Special form (name not evaluated; params not evaluated) - Binds name in the CURRENT env to a VMacro - Top-level defmacro ALSO persists like define: image layer writes function_defs (or equivalent) with a kind/tag distinguishing macro from lambda so reload reinstalls VMacro, not VLambda. v1 acceptable encoding: store body AST as (defmacro name (params) body) OR body with metadata flag is_macro=true. Prefer a boolean/kind on function_defs if a column already exists for kind; else store the defmacro wrapper AST so load can detect head=defmacro. - Returns the VMacro (or name string) – mirror define's "return the bound value" convention unless image layer already normalizes define returns.

(quote x) / 'x - Unchanged: expr -> value via quote_to_value

(quasiquote template) / `template - Evaluates template structure, interpolating unquotes - Outside any quasiquote, bare unquote is an error

(unquote x) / ,x - Only valid inside quasiquote; evaluates x in the current env and splices the resulting VALUE into the template as a single element

(unquote-splicing x) / ,@x - Only valid inside quasiquote list position - x must evaluate to a VList (or proper list of pairs); elements are spliced into the parent list - Error if used in non-list / atomic template position

(macroexpand-1 form) - Special form OR pure prim: if form is (name . args) and name is bound to VMacro, run ONE expansion step and return the expansion as a value tree (quoted shape). If not a macro call, return form unchanged (as value via quote_to_value). - Does not eval the expansion. (Current impl: evals macro body to produce the value tree; matches inbox language-design note c11-4013 contract for REPL inspection.)

(macroexpand form) - Repeat macroexpand-1 until fixed point or budget tick limit; return final value tree.

(gensym) / (gensym prefix) - Pure prim: returns a fresh symbol name as VStr (e.g. "g$42" or "prefix$42"). Counter is process- local (image or lisp package ref cell). Enough for unhygienic capture avoidance in v1.

==========

2. VALUE <-> EXPR BRIDGE (the hard kernel) ==========

Already exists: quote_to_value : expr -> value Lit v -> v Var s -> VStr s App/special constructors -> nested VList with head symbol strings ("if", "lambda", ?)

REQUIRED for macros: value_to_expr : value -> expr Inverse, best-effort, for trees macros produce.

Macro parameter contract (updated): Macro arguments are bound as self-evaluated literal VALUES (plain VStr for strings, VInt/VFloat/etc. for numbers). No ___str___ tagging at the macro call boundary. The ___str___ emit workaround is deprecated.

Rules (v1): VInt/VFloat/VBool/VNil/VStr that are self-evaluating literals -> Lit v VStr s used in OPERATOR / BINDING / PARAM position of a list tree -> Var s (i.e. when value_to_expr sees VList (VStr head :: args), head becomes Var head unless head is a known self- quoting literal form – see below) VList [] -> Lit VNil (* empty list *) VList (VStr "quote" :: [x]) -> Quote (value_to_expr x) or App(Var "quote", ?) – either OK if eval handles

both VList (h :: t) -> App (value_to_expr h, map value_to_expr t) VPair chain treated like list if proper; improper pair error or dotted App – v1: require proper lists from macros; raise User_error on improper VLambda / VMacro / VPrim / VRef as template leaves: error (cannot reify live closures into source expr)

Symbols (Lisp-1 + VSymbol – see subsection symbols): (quote hello) => VSymbol "hello" (quote "hello") => VStr "hello" Macro args use quote_to_value (symbols for Vars, strings for Lit VStr). Residual ___str___ unwrap is transitional only (phase C cleanup).

Round-trip invariant (tests): For any expr e built from reader surface (no live closures): value_to_expr (quote_to_value e) is alpha-equivalent / pp-equal enough to eval the same for pure programs. Exact AST equality preferred where constructors align.

==========

3. EXPANSION SEMANTICS ==========

A VMacro is (params, body_expr, defining_env).

When applying a macro (name a1 a2 ? an): 1. Do NOT eval a1?an. 2. Reify each ai with quote_to_value -> vi 3. Bind params to vi in a child of defining_env (lexical macro env – captures helpers at defmacro) 4. Eval body in that env -> expansion_value 5. expansion_expr = value_to_expr expansion_value 6. Eval expansion_expr in the ORIGINAL call-site env (not the macro's defining env) – standard Nopales macro call-site evaluation of the expansion.

Recursive expansion: If expansion_expr is itself a macro call, expand again (macroexpand loop) subject to the same Budget.t as eval. Infinite macro recursion => Budget_exceeded.

Shadowing: If a local binding shadows a macro name with a non- macro value, App uses ordinary application. is_special_form still wins over macros for the fixed kernel names (lambda, if, quote, defmacro, ?). defmacro names MUST NOT be allowed to override kernel special forms; define/defmacro of those names errors.

==========

4. EXPANSION TIMING (define / load / REPL) ==========

Two moments matter:

A) Interactive / one-shot eval (REPL, /api/eval): Expand at use (call time), as in ?3. Simple and correct for ad-hoc forms.

B) Top-level (define name body) and image load of function_defs: After reading body AST, run a FULL macroexpand on the body BEFORE storing the evaluated VLambda in the session env. Rationale: - Call sites stay fast (already expanded in the in-memory lambda body) - Effect inference (lisp-interpreter.effects) sees real prims, not macro heads - AI/tools that read env closures see expanded code

Persistence choice (IMPORTANT): Store the SOURCE body AST (with macro call sites), not only the expanded form. function_defs.body = source (what user/agent wrote) On session install / load-library / define: expand(source) -> eval -> VLambda in env Why: tree editor, pp, package export (lib.pp), and AI rewrite tools must show what the author wrote. Expanded-only storage would destroy macro use sites.

Optional later cache column expanded_body – not v1.

defmacro persistence: On load, re-bind VMacro from stored defmacro AST BEFORE expanding other defines that might call it. Install order on a branch: 1. all macros (defmacro forms) 2. then functions (define forms), each expanded against the macro env so far Within a single load batch, dependency order may be imperfect; document that macros should be defined before use in packages. v1 may expand with whatever macros are already in env (seed order / name order). Package seed should list macro files/bindings first when this matters.

==========

5. QUASIQUOTE EVALUATION MODEL ==========

quasiquote is a special form, not a macro on quote.

qq(depth, template): depth starts at 1 when entering (quasiquote ?) - (unquote e) at depth=1 -> eval e - (unquote e) at depth>1 -> rebuild unquote wrapper with qq(depth-1, e) (nested quasiquote support) - (quasiquote e) -> rebuild with qq(depth+1, e) - (unquote-splicing e) at depth=1, list context -> eval e, must be list, splice - atoms / Vars / Lits at any depth -> quote_to_value - lists: map qq over elements; splicing flattens

v1 MUST support single-level quasiquote well; nested quasiquote depth counting is required for correctness of macros-that-write-macros but can be tested lightly.

Reader: ‘ introduces quasiquote of next expr , introduces unquote of next expr ,@ introduces unquote-splicing of next expr Prefer producing App(Var "quasiquote",[e]) etc. so existing App special-form path works; dedicated AST constructors (Quasiquote/Unquote/UnquoteSplicing) are allowed if JSON + pp + quote_to_value are updated in lockstep. Pick ONE representation in implementation and use it consistently (recommend App form for less JSON schema churn; dedicated constructors OK if cleaner).

6. INTERACTION WITH EFFECTS / AI / BRANCHES

Effects (lisp-interpreter.effects): - infer_effects runs on EXPANDED bodies when possible - quote / quasiquote templates contribute no runtime effects except through unquote subexpressions - defmacro body effects are Meta/code-time (top-level define persists via the session layer); the expanded call site carries the real effects - prim_effects: gensym = Pure; macroexpand* = Meta/Pure

AI authoring (ai-authoring): - propose-function may emit defmacro and macro call sites; freeze path stores source AST as today - ast-rewrite tools edit source AST; expansion is re-done on next define/load

Branches: Macros are bindings; they inherit/override via the same function_defs branch visibility as functions. A personal branch can override a system macro.

7. ERRORS

- arity mismatch on macro params (like lambda) - value_to_expr failure -> User_error with context - unquote outside quasiquote - unquote-splicing of non-list - defmacro/define of kernel special-form name - macro expansion budget exceeded - improper list from macro body (v1)

8. IMPLEMENTATION PHASES

Phase 1 – Bridge + quasiquote (no defmacro yet) - value_to_expr + round-trip tests with quote_to_value - reader sugar ‘ , ,@ - eval quasiquote / unquote / unquote-splicing - special_forms catalog entries - gensym prim

Phase 2 – defmacro + call-time expand - VMacro value - defmacro special form - App path: if operator evals to VMacro, expand (note: operator is a Var lookup without evaluating args – for (mac ?) head is Var; do not eval head as function-before-expand in a way that forces args. Pattern: lookup Var name; if VMacro, expand; else existing eval head + eval args + apply.) - macroexpand-1 / macroexpand - tests: when/unless style macros

Phase 3 – define/load expand + persistence - expand body on define before env install of lambdas - image load order: macros first - persist defmacro so reload restores VMacro - package seed: allow macros in libs/*

Phase 4 – Product sugar (optional same PR / follow-up) - stdlib macros: when, unless, and, or, begin~seq alias if desired – only if they do not fight existing prims - UI: show macro badge on /bindings (kind=macro)

9. SUCCESS CRITERIA

- (defmacro when (t b) ‘(if ,t ,b ())) works in REPL - (when #t 42) => 42; (when #f 42) => () - ,@ splices: ‘(a ,@’(b c) d) => (a b c d) as value list - quote/value_to_expr round-trip tests green - defmacro survives image

restart / branch checkout - define of a function using macros expands so effects badges see real prims - dune build lisp + dune runtest lisp pass - No regression to pure-prim Lwt ABI

==========

10. DECISIONS LOCKED BY THIS SPEC ==========

- Name: defmacro (not define-macro) - Hygiene: unhygienic + gensym (no syntax-case) - Expansion: call-time always; ALSO expand-at-define/load for installed lambdas; store SOURCE AST - Symbol/string collapse REJECTED (subsection symbols). Do not build new macros that rely on collapse or ___str___. - Quasiquote is a special form, not a user macro - Kernel special forms cannot be redefined as macros

primitives

Primitives are the boundary between the pure interpreter and the outside world. Post-H1 the core is direct-style: prim_fn = value list -> value (lisp/lib/control.ml), and DB-touching prims are invoked via Control.perform_prim with the rim handler owning I/O – no Lwt.t in the prim ABI. The impure functions the interpreter calls into

[status: implemented]

Primitives live in image/lib/primitives.ml. They are REAL OCaml, typed at the boundary, fast. Anything that touches Postgres, htmx, or the OS is a primitive, never user-Nopales.

CRUD: (create 'type {fields...}) -> VRef (get 'type id) -> record value (update 'type id {fields...}) -> VRef (delete 'type id) -> VNil (select-for-update 'type id) -> record (locked)

Branching: (fork parent child-name) -> branch-id (merge source target) -> status (current-branch) -> branch-id (checkout branch-id) -> VNil (* session switches *) (list-branches) -> VList (diff source target) -> VList of changes

htmx/Routes: (route path fn) (* register route *) (render-html node) (* pure-html node *) (hx-get url) (hx-post url) (* attribute helpers *)

Reflection: (type-of value) -> type-tag

(list-bindings) -> VList

Sessions: (whoami) -> username (current-author) -> username (* for perm checks *)

All primitives that touch data consult the CURRENT-AUTHOR (from the binding being executed) for permission filtering, per multi-user model.

Effect tags for each prim are defined in lisp-interpreter.effects (capability lattice).

effects

Capability effects for Nopales programs: classify pure vs DB vs AI vs publish; infer; gate; display. Separate from OCaml 5 host effects and from Lwt async plumbing.

[status: implemented]

> *[verify] build: dune build lisp image; test: effect inference pure vs db vs ai on sample ASTs; smoke: /bindings shows effect badges; AI propose rejects disallowed effects when policy on*

==========

0. THREE PROBLEMS PEOPLE CONFUSE (read this first) ==========

A) ASYNC / COMPOSITION (mostly DONE) Can a function call (get ...) and use the result? YES today: (define u (get 'note id)) (dict-get u "title") Mechanism: eval : env -> expr -> value Lwt.t Pure prims: register_sync wraps Lwt.return DB/AI prims: return promises directly This is the "async problem" for data-in-Nopales. It is runtime plumbing, not an effect system.

B) CAPABILITY EFFECTS (NOT DONE – this section) What powers does this function need? Pure calc vs read DB vs write DB vs AI vs publish

page. Needed for: UI badges, AI safety, page policy, testing without services, author-runs-as-author story.

C) HOST ALGEBRAIC EFFECTS (OPTIONAL LATER) OCaml 5 Effect.perform / handlers inside the interpreter implementation. Could one day replace some Lwt coloring. Does NOT give Nopales programs effects for free. Guest language still needs B.

Do not block B on C. Do not re-solve A while building B.

==========

1. WHY EFFECTS ==========

Today every lambda looks the same: (lambda (x) (* x x)) ; pure (lambda () (get 'note id)) ; read DB (lambda () (create 'note "{}")) ; write DB (lambda () (propose-function ...)); AI + write code (lambda () (render-html "...")) ; page-ish

The OS cares which is which. Effects make that first-class, inspectable, enforceable.

==========

2. EFFECT LATTICE (capability sets) ==========

Base labels (finite, small):

Pure no capability prims; only pure computation ReadDb get, rows, select-for-update, list-branches, ... WriteDb create, update, update-field, delete, deftype, ... Branch fork, merge, checkout Publish route (and later define-page helpers) Net outbound HTTP (http-get, http-post); identity- gated admin/agent at the prim (image.outbound-http) Ai propose-function / LLM client Html render-html (soft; display intent, not security) Txn body expects with-transaction scope

A function's effect set is a SET of labels, e.g. {ReadDb, Html} {WriteDb, Ai} {}

Ordering for policy (allowlist checks): Pure ? anything ReadDb ? ReadDb?WriteDb (writes imply reads usually) optional sugar: Db = ReadDb?WriteDb

NOTE on (define ...): Top-level define persists code – a code-store write performed by the session layer, NOT an effect tracked in the lattice. Distinguish: - effects OF a function body (what it does when called) - effects OF defining it (session install + persist)

==========

3. PRIM EFFECT TABLE (initial map) ==========

Pure: + - * = < > <= >= not list car cdr cons null? pair? count string-append str type-of dict-get dict-set dict-keys hx-get hx-post ; string tags only

ReadDb: get rows select-for-update list-branches current-branch list-bindings whoami current-author ; session read; treat ReadDb or soft Ambient

WriteDb: create update update-field delete deftype

Branch: fork merge checkout

Publish: route

Html: render-html

Ai: propose-function (Ai usually pairs with WriteDb on code store via define path)

Txn-related: with-transaction (special form) introduces Txn scope select-for-update meaningful only under Txn

Image layer owns the table (primitives live in image/). Nopales package may define the effect datatype only.

==========

4. INFERENCE (syntactic, conservative) ==========

effects(expr) = union over subexpressions + effects of any prim head used in App + effects of known global bindings referenced (lookup function_defs metadata / env annotation)

Rules of thumb: - overapproximate: if unsure, add effect - higher-order unknown fn => Effects.top or {ReadDb,WriteDb,Ai,Publish,Branch} policy knob - lambda body effects = function effects - quote bodies contribute no runtime effects - quasiquote: only unquote / unquote-splicing subexpressions contribute;

template skeleton is Pure - infer on EXPANDED bodies when macros are present (lisp-interpreter.macros); macro heads themselves are not prims

On successful (define name body): store inferred effect set next to function (new column function_defs.effects jsonb OR side table OR derive on the fly from body AST – prefer derive-on-read for v1 to avoid migration, cache later)

v1 implementation preference: derive-on-read from AST + prim table (no schema change) optional cache field later

==========

5. GATES / POLICY (where effects pay rent) ==========

a) UI badges /bindings and editor show: pure | db-read | db-write | ai | page

b) AI propose policy (default) allowed = {Pure, ReadDb, Html} denied = {WriteDb, Branch, Publish, Ai} unless user elevates (tune experimentally; document in product-experience)

c) Route publish policy (optional v1) (route path fn) warns or rejects if fn has Branch/Ai pages should be ReadDb/Html/Pure by default

d) Sandbox eval (future) browser anonymous do-it: Pure only

e) Transactions if body has select-for-update or multi WriteDb, linter suggests with-transaction

==========

6. ASYNC STORY (what is already true) ==========

Solved: - eval returns value Lwt.t - prim_fn = value list -> value Lwt.t - composition of DB reads in Nopales works - with-transaction threads dynamic connection

Not the same as effects: Lwt answers "when / how concurrency is scheduled" Effects answer "what capabilities are required"

Example: (get 'note id) is BOTH async (returns promise through eval) AND ReadDb (capability)

You already need both; only async exists today.

==========

7. RELATION TO OCAML 5 EFFECTS ==========

OCaml 5 algebraic effects MAY later implement host-side intercept for DB/AI inside the interpreter (handlers). That is a runtime rewrite of image/lisp internals.

Explicit non-goals for first effects delivery: - do not replace Lwt in Mirage/httpaf/pgx stack - do not expose Effect.perform to Nopales users - do not require monadic syntax in Nopales

Guest programs keep looking like normal Nopales. Effects are metadata + checks, not new call syntax (unless we add optional declare forms later).

Optional sugar later: (define-pure name body) (define-page path body) Desugar to define + check inferred ? declared.

==========

8. IMPLEMENTATION PHASES ==========

Phase 0 (docs/UI copy): explain Pure vs DB vs AI in /about Phase 1: Effect module + prim -> effect set table Phase 2: infer_effects : expr -> EffectSet.t Phase 3: badges on /bindings + /edit + inspect Phase 4: AI propose allowlist gate (configurable) Phase 5: route publish warnings Phase 6: session capability masks + ENFORCEMENT (see capability-enforcement; language-roadmap Phase 3) Phase 7 (optional research): OCaml effect handlers under eval

NOTE: Phases 1-5 of THIS effects subsection are largely done (table, infer, badges, AI gate). Roadmap Phase 3 is about turning metadata into hard gates on public routes and agent sessions – not redoing inference.

==========

9. DATA SHAPES (sketch) ==========

type effect = | Pure | ReadDb | WriteDb | Branch | Publish | Ai | Html | Txn

type effect_set = effect list (* unique sorted *)

val prim_effects : string -> effect_set val infer : prim_table_meta -> expr -> effect_set val subset : effect_set -> effect_set -> bool val pp : effect_set -> string (* "pure" / "read-db+html" *)

No change to value or eval types required for Phases 1-5.

==========

10. SUCCESS CRITERIA ==========

- User can see whether a function is pure or touches DB/AI - AI-proposed functions can be policy-gated by effects - Pure functions remain testable without Postgres - Async composition remains intact (no regression of Lwt ABI) - Spec/docs stop conflating async with effects

binding-sugar

Define and body ergonomics: function define, begin/multi-body

[status: implemented]

> *[verify] test: (define (f x) (+ x 1)) installs lambda; (f 2) => 3; test: multi-body define evaluates to last form; test: (begin e1 e2 ? en) => en; test: old (define name (lambda ?)) still works; test: define docstring forms coexist (see define-docstring)*

SHIPPED.

FUNCTION DEFINE SUGAR (define (name param?) ["doc"] body body* ?) ? (define name (lambda (param?) (begin body?))) Single body may omit begin. Dotted rest params: out of scope (User_error if "." appears in param list).

Still valid: (define name expr) (define name (lambda ?)) (define name expr "doc")

BEGIN / MULTI-BODY (begin e1 ? en) and (seq e1 ? en) – special forms; value of en; earlier forms for effect. body_of_exprs desugars multi-body define/lambda/match/guard arms to Seq chains.

PRINT function-source pretty-prints stored AST; short-form (define (f x) ?) preference is cosmetic when body is Lambda – optional, not required for this section.

NON-GOALS - Internal define - let function sugar - defmacro (name p?) sugar (follow-up if needed)

Docstrings: define-docstring (orthogonal, implemented).

optional-rest-args

Optional and rest lambda parameters: &optional with defaults, &rest collection. Backward compatible with fixed arity.

[status: implemented]

> – *agent note:* 2026-08-05: implemented &optional (default "") and &rest markers in apply_step. No &optional defaults yet (BORGE-1785965096). 8 new eval tests pass; 136 total green. dune build clean.
>
> *[verify] build: dune build lisp; test: dune runtest – lisp eval optional/rest cases; test: (lambda (a &optional b) b) called with 1 arg: b is unbound/empty; test: (lambda (a &optional b) b) called with 2 args: b is 2nd arg; test: (lambda (&rest args) args) collects all args into a list; test: (lambda (a &rest rest) rest) collects extras after required; test: fixed-arity lambdas still enforce exact arity (backward compat)*

House style stays FIXED ARITY for most image code. This subsection adds two backward-compatible extensions so user code stops reinventing optional/keyword args via empty-string sentinels and opts dicts.

SYNTAX (in the parameter list, after required params): (lambda (a b &optional c) body) – c is optional; if omitted, bound to empty string "". (lambda

(a b &optional (c default)) body) – c is optional; if omitted, bound to default (evaluated in the call env at bind time). (lambda (a b &rest args) body) – args is bound to a VList of all remaining arguments. (lambda (&rest all) body) – all-variadic; all args collected.

&optional and &rest may not both appear. &rest must be the last marker. These are markers (symbols matching the literal strings "&optional" / "&rest"), not params.

DOTTED PAIR NOTATION (a . b) remains UNSUPPORTED at the reader level (parse error). The &rest marker is the only way to collect variadic args. This keeps the reader simple.

ARITY SEMANTICS Required params: must all be supplied. &optional: supplied or fewer – no error; missing ones get their default (or "" if no default form). &rest: absorbs any extra args beyond required+optional. Calling with FEWER than required ? Arity_mismatch (as today). Calling with MORE than required+optional (and no &rest) ? Arity_mismatch (as today).

IMPLEMENTATION NOTES - The Lambda constructor / VLambda carry a param spec richer than string list: required params, optional (name, default expr option) list, rest name option. A flat string list with &optional/&rest markers as sentinels is the minimal change; a typed param_spec is cleaner but larger. - apply_step binds required first (arity check against min_arity), then optional (fill defaults for missing), then rest (collect leftover into VList). - params_of_define_sugar must accept &optional/&rest markers and stop raising on their presence (the current '.' check stays – dotted notation is still rejected). - pp_expr / value_to_expr round-trip must preserve the param list shape so function-source prints correctly.

NON-GOALS - &key keyword args (opts-dict convention stays until a real need proves it; Smalltalk keyword messages are the model). - Dotted-pair (a . b) reader syntax (use &rest). - Optional/rest on macros (follow-up; macros already have their own arity path – add later if needed).

define-docstring

Optional docstring on define; persist to function_defs.doc; (function-doc name)
[status: implemented]

> *[verify] build: dune build && dune runtest; test: (define d1 1 "one") binds 1; (function-doc "d1") => "one"; test: (define (inc x) "add one" (+ x 1)); (inc 2) => 3; function-doc has docstring; test: (define f (lambda (x) "lam" (+ x 1))); function-doc and body without dead string; test: redefine without docstring clears or leaves prior doc per documented rule; test: (function-doc "missing") => (); test: old two-arg define still works; load-library packages without docs; smoke: function_defs.doc round-trips across persist + lookup; smoke: fork/merge copy doc with body*

PURPOSE Make every binding documentable in-language so agents and humans can build a live reference (apropos, /ref, platform-ref notes) without a parallel docs system. Inbox SPEC "define docstring" is the product brief; this subsection is the book authority for the OCaml kernel half. Nopales package doc coverage, /ref, and apropos are follow-on product work once this lands.

SCHEMA (already present – no migration) function_defs.doc text – see storage.schema / migrations/001_branches.sql. Column exists; writers and readers historically ignored it. This work wires it. Do NOT add a second description column.

SURFACE SYNTAX (all optional doc) 1. (define name expr) no docstring; bind eval(expr) 2. (define name expr "doc") bind eval(expr); doc = the string literal 3. (define (name p?) body?) sugar ? lambda; no doc unless (4) 4. (define (name p?) "doc" body body*?) leading string literal in the body list is the docstring and is NOT part of the evaluated body (requires at least one real body form after it; a lone string body is a value-returning thunk, not a docstring) 5. (define name (lambda (p?) "doc" body?)) same leading-string

rule inside lambda bodies when the define is top-level: strip into doc and store/bind the lambda WITHOUT the dead string so function-source stays clean

Docstring MUST be a string literal (Lit VStr / reader "?"). A computed expression in the doc slot is a syntax error (User_error), not silently evaluated.

EVAL (pure lisp/) - Accept forms (1)-(5) without error. - Binding value is unchanged vs pre-docstring define. - Pure env does not store docs (no env metadata in v1). - Prefer one shared helper (e.g. split_define / peel_docstring) used by eval and by the image intercept so sugar and doc rules cannot drift. - value_to_expr / quote of Define need not round-trip the doc string inside the AST (doc is store metadata, not part of expr). Body AST after peel has no doc.

IMAGE PERSIST session eval_one / eval_form define intercept and agent_tools.define_binding: - peel name, body_expr, doc option - eval as today (env bind) - Store.persist_function ? ?doc Redefine rules: - docstring present ? write that string to doc - docstring absent ? set doc NULL (explicit clear). Rationale: edit/save and load-library without docs must not leave stale blurbs from an older body. Packages that care keep the string in source. Anonymous may not persist (unchanged). defmacro: out of scope for v1 docstring slot (no third-arg / leading-string capture); follow-up if needed.

STORE persist_function : t -> name -> body -> author -> ?doc:string option -> unit Lwt.t INSERT/UPDATE sets doc (NULL when None). lookup_function / list paths that feed function-doc must SELECT fd.doc (nullable). fork copy, merge upsert, promote_functions: copy doc with body/author so docs survive branch ops.

PRIMITIVE (function-doc name) ? string | () - name: string (binding name) - If visible function_defs row has non-null non-empty doc ? VStr doc - Else if name is a special form ? that form's summary from special_forms catalog (discoverability) - Else if name is a registered prim with a help blurb ? that blurb when one exists - Else VNil Effect class: ReadDb (store lookup); pure fallbacks after miss are fine inside the same prim. Register next to function-source / function-ast. package-exports kernel/std list should include function-doc once shipped.

SPECIAL-FORMS CATALOG Update define entry arity/examples to show optional docstring forms.

NON-GOALS (this subsection) - /ref page, apropos, nav link, ai/context wiring (product docs layer; build after kernel works) - Mass docstring backfill of libs/*.pp - doc on type_defs or routes - Markdown rendering of docs in OCaml UI (plain text) - Changing function_defs PRIMARY KEY or body JSON shape

SUCCESS - One define with a docstring round-trips through persist and (function-doc ?) - Sugar and lambda leading-string forms agree - Existing libs without docs load unchanged - Agents can document bindings in-language and read them back without scraping function-source comments

match

Pattern match / destructuring for lists and code-as-data trees

[status: implemented]

> *[verify] test: match list pattern binds vars; test: match symbolic head (lambda params body) with VSymbol; test: match fallthrough / else via _; test: no match raises catchable User_error*

SHIPPED special form:

(match scrutinee (pat1 body1?) (pat2 body2?) (_ else-body?))

Patterns (v1): _ wildcard x bind variable lit number / bool / nil / string 'sym quoted symbol (VSymbol; VStr compat) "str" string literal (head p?) list starting with symbolic/string head (list p?) explicit list pattern

Semantics: First matching clause wins. Scrutinee eval once. Bindings lexical in clause body (Env.child). No match => User_error "match: no matching clause" (catchable via guard).

Non-goals v1 (deferred): dotted-tail patterns, (? pred p), and/or patterns, exhaustiveness, record patterns, regexp.

SUCCESS: idiomatic AST walk over read-string / function-ast trees with symbolic heads works without car/cdr chains.

math

Numeric tower beyond the four operations: expt/sqrt/trig/log, numeric predicates, integer division aliases, seeded RNG

[status: implemented]

> *[verify] build: dune build lisp image; test: expt/sqrt/log/exp/sin/cos/tan round-trip on known values (float results); test: (expt 2 10) => 1024 int-in int-out; (expt 2.0 0.5) => 1.414? float; test: number?/integer?/zero?/even?/odd? classify VInt/VFloat/VStr/VNil correctly; test: quotient/remainder match div/mod on ints incl. negative dividends; test: (random n) => 0 ? r < n; (random) => float in [0,1); seeded random deterministic per sequence; test: div-by-zero and domain errors (sqrt of negative, log of ?0) raise catchable User_error; test: conformance_test.ml pins at least one case per new prim*

GAP. Core 1.0 froze the four operations (+ - * div / mod), abs/min/max, floor/round, and parse-int/parse-float – but nothing else. Anything numeric beyond grade-school arithmetic currently has to shell out to float-of-string hacks or OCaml. This subsection is the book authority for closing that gap. Classified ADDITIVE under core-freeze.

NEW PRIMS (all pure, all in pure/prims_pure.ml (the shared pure library), registered in primitives.ml, added to the Pure list in lisp/lib/effect.ml; note the Pure list today omits "/" even though div is listed – fix that omission in the same pass so alias pairs (div /) and (quotient remainder) stay classified identically)

Powers / roots / transcendental (float semantics; int-int expt with non-negative exponent returns VInt): (expt base n) int-in/int-out when both int and n ? 0 (Int64 overflow wraps like Int64.mul; documented, no bignum in v1); else float (sqrt x) float; User_error on x < 0 (exp x) (log x) float; log: User_error on x ? 0 (log x b) optional base arg ? (logb x) = log x / log b (sin x) (cos x) (tan x) float, radians (asin x) (acos x) (atan x) float; asin/acos User_error outside [-1,1] (atan y x) two-arg form ? atan2 when given 2 args pi bound as a VALUE (VFloat Float.pi) in the initial env, not a zero-arg prim – agents write pi, not (pi)

Predicates (return VBool; missing core gap): (number? v) VInt | VFloat (integer? v) VInt only; VFloat with integral value returns #f (v1 rule: tag-based, not value-based – keeps it cheap and honest) (zero? n) (positive? n) (negative? n) (even? n) (odd? n) integers only; User_error on float with non-integral value

Integer division aliases (Scheme names, delegate to the existing div/mod int paths – no new semantics): (quotient a b) ? int div; User_error on float args or 0 (remainder a b) ? Int64.rem; User_error on float args or 0

Randomness: (random n) VInt in [0,n) for VInt n > 0; VFloat in [0.,1.) for zero args (random-seed s) reseeds the image PRNG (Random.init with VInt-derived seed); returns VNil Effect class: Pure – Nopales-visible nondeterminism, but touches no capability rim; same class as now/now-ms. One PRNG state per image process is fine for v1 (documented; per-session streams are a non-goal).

EFFECT CLASSIFICATION All new prims are Pure. Quotient/remainder/classification aliases must classify identically to the prim they wrap (div / mod) – alias drift

in the Pure list is a drift failure mode; the conformance suite should catch a prim that infers non-Pure.

ERROR RULES (uniform) - Wrong arity / non-numeric arg ? User_error naming prim - Domain errors (sqrt/log/asin/acos, div-by-zero) ? catchable User_error, never VNil, never NaN leak: prims CHECK before calling the OCaml float op - integer? on VFloat 3.0 ? #f (tag rule above)

NON-GOALS (v1) - Rationals / exact arithmetic tower - Bignums (Int64 wraps; documented) - Complex numbers - (gcd a b) (lcm a b) – useful but trivially writable in Nopales once quotient/remainder exist; candidates for libs/std, not kernel prims - Per-session RNG streams

SUCCESS: agents and product libs can do real math (interest, FSRS scheduling, geometry, statistics) without leaving the language; conformance suite pins every new name so silent removal reddens the build.

std-library

Blessed std list/string helpers: freeze surface for packages and agents

[status: implemented]

> *[verify] test: conformance covers list/map filter foldl foldr; test: symbol and string prims present; smoke: load-library std exports list helpers; idempotent*

RULES 1. FROZEN SET below – signatures must not change in place; bugfixes OK; new behavior needs a new name. 2. Hot loops: OCaml special forms / prims; Nopales std wraps. 3. New std helpers require a conformance or verify case.

FROZEN SET (kernel + image prims + libs/std aliases)

Lists (kernel special forms – OCaml loops + apply): list/map list/filter list/foldl list/foldr Lists (prims): list car cdr cons null? pair? reverse append count Strings (prims): string-append string-join string-length substring string-eq string-contains? string-trim html-escape as-str str Symbols (prims): symbol? string? symbol=? symbol->string string->symbol Misc: not type-of

libs/std also exports ergonomic aliases map/filter/foldl/ foldr and html/* builders – html/* may evolve faster than this kernel freeze; list HOF names above are frozen.

SUCCESS: conformance + dialect tests pin list HOF and symbol/string prims; silent renames redden the suite.

capability-enforcement

Hard gates: public routes and agent sessions obey effect allowlists

[status: implemented]

> *[verify] test: pure and read-db public handlers allowed; test: write-db / branch / net public handlers denied; test: unknown and non-function handlers fail-closed; test: macro-expanded write is denied after expand; smoke: (route ? 'public) refuses over-powered handlers at bind*

SHIPPED (public-route gate). Builds on effects inference.

PUBLIC ROUTES Route_gate.check_public_handler ~env handler: infer effects of the named handler from the in-memory env (macroexpand first). Allowlist default: Pure | ReadDb | Html WriteDb requires PP_PUBLIC_ALLOW_WRITE=1 opt-in. Branch / Ai / Net / unknown ? deny (fail-closed). Session.route bind calls the gate when visibility=public; over-powered handlers are refused at bind (kept private).

AGENT / AI AI propose-function remains policy-gated on the same effect subset helpers. Per-session agent effect_mask for every tool prim is a follow-on tightening (not required to close this section's public-route gate).

BUDGET Eval step budget is a hard leash (Budget_exceeded), uncatchable by default via guard – part of the same production-leash story.

AUTH Orthogonal to is_admin: admin private routes full power; public publish still PUBLIC_ALLOW.

NON-GOALS - Full setuid / run-as-author lattice - Postgres RLS as effects substitute - Required user-authored effect declarations

SUCCESS: sloppy public handlers cannot fork branches or write the store without opt-in; pure unit gate tests green.

package-surface

Blessed package exports and stability rules for system/main

[status: implemented]

– *agent note:* agent plan 2026-08-02 – Phase 1 design settled with roerick. Resolves most of BORGE-1785211142 item a; multi-file libs issue item b stays deferred. Design calls settled in chat:

SETTLED DESIGN CALLS 1. Export names are FULL names like todo/add, not bare auto-prefixed. 2. Registry storage = new branch-lensed tree family b.branch.pkg.name one-tree.tree-model PATH GRAMMAR updated with a package_registry scaffolding table mirroring it, exactly the function_defs pattern table = runtime truth until PP_TREE_READS/WRITES cutover; forward-only mirror triggers. Inherits parent-to-child like function_defs – per-user enable/disable falls out of the branch model for free. 3. std stays ALWAYS enabled – hard-protected; not a registry row. package-exports std keeps serving kernel_std_exports until the std-library freeze lands; do not widen that list in passing. 4. Routes / deftype / macros are OUT of the export list for v1 functions only. 5. Personal branches may still shadow exported names; documented, not prevented.

DISABLE SEMANTICS = HIDE, NOT DELETE. package-disable writes enabled=false on the current branch and re-runs rehydrate; rows stay in Postgres, they just do not load into the session env. Boot seed is automatically safe: seed is additive fills MISSING names and disabled rows still exist, so seeding a disabled package is a no-op. Re-enable flips the bit back; no re-seed.

MVP PRIM SURFACE packages – registry rows: name, enabled, – exports, source header-or-heuristic package-enable workout – enabled=true on current branch, – rehydrate package-disable workout – enabled=false, rehydrate. – std rejected with a clear error. package-exports pkg – resolution order: declared header – set, then kernel_std_exports for – std and kernel, else shallow heuristic

IMPLEMENTATION PLAN touchpoints verified 2026-08-02 1. migrations/011_package_registry.sql – package_registry fields branch_id, name, enabled bool default true, exports jsonb, author, updated_at; PK branch_id+name; plus forward-only mirror trigger copying 009_tree.sql tree_mirror_function_defs pattern pp.tree_mirror guard, CONSTRAINT TRIGGER DEFERRABLE. Forward-only is correct: 009 has NO reverse trigger; runtime truth stays in the table. Also ALTER TABLE function_defs ADD COLUMN package text NULL provenance stamp. 2. tree_store.ml – pkg_path constructor b.uuid.pkg.name via uuid_label + enc_label, mirrors tree_uuid_label in SQL. 3. loader.ml / session.ml – recognize a leading package NAME export ... header form in load-string; do NOT eval it as a define. Upsert the registry row enabled preserved on re-load; exports replaced. While loading that file, stamp persist_function calls with package=NAME session-local current-package string option threaded through eval_one / eval_form persist_define; cleared at end of

load. 4. session.ml rehydrate/load_defs – after loading the registry for the branch chain same recursive-CTE override walk as list_function_names_for_load, first-hit-wins depth ASC, skip defines whose package is disabled on this branch. Untagged defines package IS NULL always load. Routes whose handler belongs to a disabled package are filtered in load_defs route pass too handler package via function_defs.package. 5. agent_tools.ml / session.ml prims – packages, package-enable, package-disable new; package-exports resolution order change; describe prefers declared exports when present. 6. Seed headers: libs/std/lib.pp exports = kernel_std_exports list exactly, libs/ui/lib.pp, libs/layout/lib.pp; todo as the app example. 7. export-package round-trips the package header form into vault text; pp-sync snapshot includes it.

EXPLICIT NON-GOALS FOR PHASE 1 unchanged from details above: sealing, import forms, multi-file libs, requires graph, promote filter, call-site lint. Those are Phase 2/3.

VERIFY maps to the smoke lines on this subsection - package-disable todo then todo/list -> unbound error; rows still in function_defs; package-enable todo restores. - Disabled on admin/personal does not affect system/main. - package-exports todo returns the declared header set, not the shallow heuristic, after seeding the header. - Boot with a package disabled on system/main: seed does NOT re-enable it. - borge drift clean; dune build and dune runtest.

[verify] smoke: export-package or package-bindings lists public names for std/ui; smoke: docs or /bindings marks public vs internal

ROADMAP: Phase 3 (ecosystem law, light).

========== PROBLEM ==========
system/main packages are de facto public API for all gardens. No export list ? agents depend on internals; renames break personal branches silently.

========== v1 RULES (no heavy module system) ==========
1. Each libs/<pkg>/lib.pp MAY start with a declaration: (package pkg-name (export name1 name2 ?)) If absent, all defines are public (status quo). If present, non-exported names are "internal": still callable (Lisp-1) but documented as unstable; optional lint/warn on other packages referencing them.

2. Stability tiers: kernel prims – strongest freeze (with std-library) exported package – prefer compat; rename = new name internal – may change any deploy personal branch – user owns breakage

3. Promote discipline (existing branching.promote): only exported names should be promoted to main without explicit force.

4. Agent seeds: prefer exported names only.

========== NON-GOALS ==========
- Import forms / separate namespaces per file - Version numbers in-package (git/branch is version) - Sealing that makes internal uncallable (too hostile for a Lisp OS v1)

SUCCESS: std + ui + layout have export lists; concepts doc explains public vs internal; agents stop depending on clearly internal helpers by default.

function-observability

Inspect functions: recursion/tail highlight, step counts, DB cost, big-O heuristic, on-demand LLM analysis, inside-out peel

[status: implemented]

[verify] build: dune build lisp image; test: dune runtest – recursion / complexity / explain / eval step attribution; smoke: GET /binding/<recursive-fn> marks self-ref and tail vs non-tail sites; smoke: GET /binding/<fn> shows last-run steps / prim_calls / db_hits when stats exist; smoke: GET /binding/<fn> shows free big-O badge or honest ?; smoke: POST /api/analyze/<fn> returns LLM estimate fragment; cache by AST sha1; smoke: (analyze 'fn) and (explain 'fn) prims return structured results

THESIS

The OS already stores every function as an AST and runs it through a stepped evaluator. Function observability makes that inspectable on /binding and /bindings without inventing a separate profiler product.

Split by cost: FREE / pure sexp – recursion, tail position, big-O heuristic, inside-out peel structure CHEAP / eval hook – step counts, prim calls, DB hits, live DB timing into function_stat ON DEMAND / LLM – (analyze 'fn) complexity + optional per-layer prose glosses

ARCHITECTURE

lisp/ stays pure (yojson only). Static analysis modules live there and are alcotest-tested with no services: recursion.ml – self-refs + tail context complexity.ml – heuristic big-O classifier explain.ml – mechanical innermost?out peel

image/ owns HTML, store, LLM: ast_html.ml – Pure_html AST renderer with classes db_cost.ml – static DB-prim cost tiers + optional EXPLAIN analyze.ml – (analyze 'fn) + cache write http_analyze.ml – POST /api/analyze/:name htmx fragment

Instrumentation seam is the existing eval budget record and current_budget dynamic binding (eval.ml). Extend budget with current_fn, prim_calls, db_hits, fn_step_counts; expose last_budget after top-level eval_string for session persist.

Analysis results dogfood data_rows: function_stat – last/recent measured runs function_analysis – LLM complexity cache keyed by sha1(pp_expr body); edit invalidates

PHASE 1 – RECURSION + TAIL HIGHLIGHT

Recursion.analyze_self ~fn_name expr: is_recursive, tail_recursive, self_ref_sites, mutual_refs (names only; cycles from call_graph). Tail context MUST mirror the trampoline (eval TCO): If then/else Tail; test Non_tail Seq last Tail; earlier Non_tail Let/Letrec body Tail; bindings Non_tail App in Tail context is the tail call; args Non_tail Lambda body is a new frame (Non_tail of the caller) call_graph.mutual_cycles on a snapshot for SCCs. ast_html.pp_expr_html emits spans: ast-self-ref, ast-tail, ast-non-tail, ast-mutual Wire show_binding_page, list_bindings code pane, inspect_binding_page. CSS in chrome style block.

PHASE 2 – STEP COUNTER + ATTRIBUTION

Every eval_step tick attributes to budget.current_fn. apply of named VLambda sets current_fn; non-tail eval restores previous; trampoline tail keeps new name. Prim path increments prim_calls; CRUD wrappers in prims_db increment db_hits (and later timing). After route/api eval, session may upsert function_stat rows (fn, steps, prim_calls, db_hits, when). /binding shows "last run: N steps, M prims, K DB hits".

PHASE 3 – DB COST

Closed set of DB prims ? static tier badges: get/find-row Index_seek; create/update/delete Constant; rows Seq_scan; find-rows/find-text Linear. Optional lazy EXPLAIN on demand (not every page load). Live tier: wrap store calls with wall timing into function_stat.db_time_ms when clock precision allows.

==========
PHASE 4 – HEURISTIC BIG-O ==========
Complexity.classify : expr -> O_constant | O_log_n | O_linear | O_n_log_n | O_quadratic | O_exponential | Unknown Patterns (honest Unknown default): cdr-self-recursion / list/map|filter|foldl ? O(n) two recursive halves ? O(n log n) fib-style multi self-call (- n 1)/(- n 2) ? O(2^n) rows inside map / nested maps ? O(n?) no recursion/loops/DB list forms ? O(1) Free badge on /binding next to effects. Optional later: empirical curve-fit via (bench 'fn sizes) + step counts.

==========
PHASE 5 – LLM ANALYZE ON DEMAND ==========
(analyze 'fn) prim + POST /api/analyze/:name (private). Reuse llm_client; prefer per-user key (auth.user-llm-keys) then env LLM_API_KEY. Structured JSON: {complexity, confidence, rationale, notes} Cache data_rows function_analysis by sha1(pp_expr). htmx button on /binding swaps #analyze-result. Always label "LLM estimate (model, when)". Cross-check against function_stat steps when present. No auto-call on page load.

==========
PHASE 6 – INSIDE-OUT EXPLAINER ==========
Explain.peel : expr -> layer list (innermost first by max depth; siblings left-to-right). Annotate with prim name/effects, self-ref, tail, complexity when known. (explain 'fn) returns layers as readable structure. UI: collapsible explain-layer blocks on /binding. Optional LLM prose gloss per layer, cache by sexp hash; opt-in only (cost scales with depth).

==========
SHIP ORDER ==========
1, 2, 4 independent (can parallelize). 3 after 2. 5 after 2 (+ benefits from 4). 6 after 1+4; LLM gloss optional after 5.

==========
NON-GOALS ==========
- Claiming undecidable static big-O as ground truth - EXPLAIN ANALYZE on every DB prim call - Auto LLM spend on every /binding view - Full coverage-guided test selection (see image.inspector impact queue; related but separate) - Custom VM / full CPS just for profiling

SUCCESS: a human or agent opening /binding/foo sees structure (recursion/tail), free cost guess (big-O + DB tier), measured last run when available, and can click for LLM analysis or peel without leaving the browser.

lazy-env-thunks

VThunk: memoized lazy bindings; env LOAD stores recipes, first reference cooks them; explicit (delay)/(force) surface

[status: implemented]

> *[verify] build: dune build lisp; test: dune runtest lisp: thunk forces once, memoizes value AND failure, cyclic force raises, delay/force round-trip; smoke: (force (delay (+ 1 2))) = 3; forcing twice runs body once*

MOTIVATION (measured on prod 2026-08-21) Session.load_defs re-EVALUATED every function_defs body on every rehydrate: ~4013 defs on admin/personal took 60-120s under the global request mutex whenever HTTP identity flipped (one shared session, anonymous public traffic interleaving with authed agent traffic). But ~98% of defs are lambdas nobody touches between rebuilds: pure define is ~1us; the cost was STRICTNESS AT THE WRONG BOUNDARY, not interpreter speed and not per-def Postgres round trips (the load query is one recursive CTE). Habitat note spec/lazy-thunks (2026-08-20) already argued the general shape: the value is in MEMOIZATION at an EXPLICIT boundary, not pervasive call-by-need.

KERNEL New value constructor in lisp/lib/ast.ml:

VThunk of thunk_cell ref thunk_cell = Unforced of thunk_payload | Forcing | Forced of value | Failed of exn thunk_payload = { body : expr; def_env : env; force : env -> expr -> value }

The payload CARRIES ITS OWN EVALUATOR so the pure kernel stays I/O-free while the image can force under its prim rim (DB prims inside a forced body work) and under the current eval budget. Forcing MEMOIZES on all outcomes: Forced replays the value; Failed replays the exception (a def that errors at first touch does not re-tax every later touch); Forcing detects self-reference cycles and raises Interp_error instead of diverging.

SINGLE OBSERVATION POINT Env.lookup is the one auto-force site. A VThunk never escapes into user values through a Var reference; the language stays strict everywhere else. Direct OCaml lookups (route handler resolution, ui shell) get the same semantics for free. Existing code never sees VThunk except via fold/list-bindings (renders <thunk>).

USER SURFACE (delay EXPR) -> VThunk; EXPR not evaluated (force X) -> forces X if a thunk, else returns X (define ...) stays EAGER: laziness is a property of env loading and explicit delay, never of define. Side-effect order of ordinary code is unchanged.

NON-GOALS - call-by-need language semantics (rejected in spec/lazy-thunks: side-effect order + budget attribution) - transparent forcing at truthiness/printing observation points (only Var forces; decide more after dogfooding) - cross-session or persistent thunk cache (Design B of the note – tree pending-state – remains a separate item)

9 borge/agent.borg

pricklypear-agent

Pricklypear book chapter: agent – GLOCHID, the local-first agent harness: the Nopales-native coding-agent PRODUCT. A local-resident habitat client: one process, one screen, working its own forked branch of the one tree, syncing up, offline-tolerant, with installable browsers. Links remote-eval as its capability/library; shares pricklypear-lisp + llm with the image.

[status: planned]

– *agent note:* 2026-12 ALIGNMENT PASS (product vs capability, the line the code was reaching for). HISTORY: the agent was born 2026-08-03 as a hermetic pi-clone spike ('no Postgres, no branches, no auth'); remote-eval + pp-remote arrived 2026-08-14 as the SAME prim seam with the rim handler swapped to an HTTP proxy, explicitly 'NOT a new product – a mode of the agent host.' The code never converged: agent/ stayed hermetic-local, remote/ grew a second product surface (a chat), and the two forked surfaces/t-records/serve-functions are SYMPTOMS of a missing noun, not independent designs. This pass redraws the line the spec never drew and retires the 2026-08-03 framing. ALIGNMENT: (1) the AGENT is one PRODUCT – a local-resident coding agent, chat + installable browsers (file browser, AST browser, data browser), one process you launch. It is a HABITAT CLIENT BY DEFAULT, not a hermetic spike: each run forks its own branch (lens-<device>), syncs the AST + data down, and works against that LOCAL LENS as the working source – edits are local-first, pushed up via sync, offline-tolerant up to a surfaced merge (local-first.sync-conflicts). The image is the durable store + merge authority, NOT a live read target. (2) REMOTE-EVAL is the CAPABILITY/LIBRARY behind that – sync the AST/data down, eval locally, proxy out-of-working-set prims up to the image, three-way merge on reconnect – NO surface, no chat. pp-remote shrinks back to this library + a minimal /eval harness; the chat surface I just added to remote/ MOVES to the agent (one surface file, one set of chat/* templates, one serve). (3) THE BOUNDARY: agent owns the surface + installable browsers + the coding-agent tool set (read/bash/file-browser over the operator's laptop); remote-eval owns the rim, the lazy env, the def-cache, the sync, the stepping, the wire. The agent never reimplements eval; remote-eval never owns a screen. (4) llm/ (loop+provider+tools) stays shared – it sits ABOVE both: the agent supplies a Loop.host over the local-first rim; a future debugger/test-harness links the same loop over its own host. WHAT THIS PASS DOES NOT DO: restatus the implementation subsections or touch code – this redraws the POSITIONING line only; a make pass retires the agent binary's hermetic path, collapses the duplicated surfaces into one on the agent, and shrinks remote/ to the library. OPEN QUESTIONS the make pass must settle, not this pass: (a) sync cadence – per-write sync is wrong (an LLM making 20 quick edits is 20 merge rounds + 20 merge-bases); a sync round is a coherent UNIT (the turn, or an explicit flush), debounced – decide in local-first.sync. (b) fork garbage collection – every device forks lens-<device>; dead devices leave stale branches. (c) two planes one screen – chat-agent bash edits the operator's LAPTOP repo (the OCaml); Nopales surface edits go to the LENS -> image. Same screen, two planes; name it so it is a feature, not a confusion. The 'agent requires a reachable habitat' call is taken here: offline is DEGRADED working-set-only, not a

first-class mode – the lens makes a flaky link survivable; 'agent with no image, ever' is no longer a product target (retires the 2026-08-03 promise).

– *agent note:* 2026-08-03 chapter created for the pi-clone spike. Positioning: sibling consumer of pricklypear-lisp (like cli/), NOT part of the image product. No Postgres, no branches, no auth. If it graduates into the product later that is a plan-phase spec edit.

– *agent note:* 2026-08-14 v0.0 spike landed: rim moved to Control.run_with_rim (image Control_rim is now a thin alias), Eval.read_eval_all_direct added, agent/ package with stdin REPL + mock provider + read/bash builtins + on/register-tool/register-command + tool_call-block path. Provider is OpenAI-compatible chat completions only (synthetic.new + any gateway, Anthropic via its compat endpoint) – no native Anthropic path; image llm_client.ml is the curl cautionary tale we avoid. agent/ tests 5/5 (incl. provider SSE reconstruction), full workspace green. Known gaps: SSE streaming now lands text deltas live (Openai.complete streams requests with stream:true; text deltas dispatch via on_delta as they decode) but tool-call stream fragments are RECONSTRUCTED into a full assistant message (not streamed live as tool-call deltas) – that reconstruction runs through the shared parse_reply so XML/GLM fallback and history shape are preserved; http_ui.ml landed: HTML primary surface (chat page + inline SSE client), code browser (read-only file tree + file view), –serve/–open wired in main.ml. Chat streams via true incremental SSE (producer-fiber + drain queue, mirroring image http_agent.ml mp/sse pattern). Gluten_eio deadlock avoided by forking handler work off the connection fiber. Code browser has canonical-path traversal guard. All surfaces smokes pass (chat page, SSE deltas, tree list, file render, traversal 403, piped-turn regression).

– *agent note:* 2026-08-14 SURFACE REFACTOR DESIGN (decided, plan-mode): the agent UI moves off SSE+vanilla-JS to Nopales-authored htmx fragments + multipart streaming, so the surface is a loadable Nopales layer both the agent AND the image share. Decision thread: (1) soup/* tree prims are DB-free – lift them from image/lib/prims_hyper.ml into the shared pure-prims layer (pure/) so both hosts register the same tree prims; (2) route/lookup + route/dispatch are THE HOST SEAM (hypermedia-harness.request-runner) – identical Nopales signatures, host-local backing (image Session.t Postgres routes vs agent Runtime.t lens routes); (3) LLM use is always-API (no local inference), so the main habitat stays the corpus source of truth; the agent evaluates against a LOCAL-FIRST LENS (local-first.borg), not a hermetic embedded corpus – edits sync up per local-first.lens-model. This revises the 2026-08-03 positioning which said 'no Postgres, no branches, no object store' and made the corpus embeddable. Streaming stays OCAML (provider SSE -> deltas -> per-part (html,target,swap) assembly -> multipart frame + flush), with Nopales owning the fragment templates; soup/apply-parts! models a consumed turn as data for libs/hyper. Phasing (read-only lens vs write-capable lens in the first cut) was left OPEN in discussion – record it in local-first.phasing when decided.

– *agent note:* 2026-12 CAMPAIGN 6.0 PLAN PASS (agent surfaces / route table spec): revealed the agent-route-table + surfaces implementation was ALREADY COMPLETE in agent/ (built

since the 2026-08 design). Reconciled both stanzas' verify stanzas + details to the implemented reality WITHOUT a status flip (per the plan-pass/make division of labour – 6.1 make flips them planned ? implemented). Verified live: agent_test 13/13 green; smoke_agent_surface.sh 14/14 PASS; piped-turn '–once' + '-p' pipe mode both work. agent-route-table = route / route/lookup / route/dispatch + thin request context (query-param / request-method / hx-request?) in agent/lib/runtime.ml, the host seam libs/hyper runs on unchanged; surfaces = Nopales-authored htmx fragments + OCaml multipart stream + GENERATED assets.ml/libs_source.ml (dune-produced; must stay in the surfaces (implements ?) list to cover the modules at build – their real sources are libs/html/lib.pp + libs/hyper/lib.pp + image/static/*.js). No code changes needed – a spec-reality reconcile only. REMAINING for 6.1 make: flip both stanzas to implemented; re-run borge drift (note the generated-module wrinkle: implements-exist sees assets.ml/libs_source.ml as missing on an IMPLEMENTED stanza because they only exist under _build – an open borge modeling limit worth an issue if 6.1 can't reconcile); formalize smoke_agent_surface.sh into the verify apparatus; wire the local-first lens backing (a campaign-8 dependency, not strictly 6.x).

– *agent note:* 2026-12 CAMPAIGN 6.1 MAKE (agent-route-table + surfaces) DONE: flipped both stanzas planned -> implemented (agent-route-table route/route-lookup/route-dispatch + thin request context; surfaces Nopales htmx fragments + OCaml multipart stream). RESOLVED the generated-module wrinkle the 6.0 note flagged: borge's implements-exist check false-positives dune-GENERATED modules (agent/lib/assets.ml + libs_source.ml exist only under _build), so a new generated-surface-artifacts PARTIAL subsection (implements those two build slices, no existence check) covers the dune module cross-ref while their real authored sources stay in the surfaces implements (libs/html/lib.pp + libs/hyper/lib.pp + image/static/*.js). Also merged the two duplicate details blocks in agent-route-table into one. Result: borge drift back to the clean 12-item pre-existing BORGE-1786924137 baseline, spec drift NONE, structural NONE. VERIFY: dune build @all clean; agent_test 13/13 green (agent-route-table x3 + surfaces x3); scripts/smoke_agent_surface.sh 14/14 PASS; piped-turn –once + -p both work; borge balance+parse clean; borge check 32/32. No OCaml code touched (implementation was already committed 6ecae8b).

– *agent note:* 2026-08-29 NAMING (discussion): the local-first agent harness is GLOCHID – the detachable spine you pull off the pad and work with: local-first lens corpus, Nopales-authored htmx fragments, multipart/mixed streamed turns replayable as data via hyper/stream. This REASSIGNS the word: local-first's conflict subsection (formerly 'the-glochids') is renamed sync-conflicts so glochid means exactly one thing – this harness. Codename level: spec headers, README, code comments; binary + dune package keep the name 'agent' until a deliberate rename pass.

agent

The Nopales-native coding-agent PRODUCT: a local-resident habitat client. One process, one screen – chat plus installable browsers (file browser, AST browser, data browser). It works against its OWN FORKED BRANCH of the one tree (a local-first lens, the working source), edits local-first and synced up, offline-tolerant up to a surfaced merge. Links remote-eval as its capability/library; shares pricklypear-lisp + llm with the image. pi (pi.dev) is the

reference point for the agent UX; the differentiator is that plugins + browsers are Nopales loaded into a live env shared with the session REPL, so the operator redefines them while the agent runs.

positioning

What the agent is and is not – the PRODUCT, against remote-eval (the capability) and the image (the authority)

[status: implemented]

IS: - ONE PRODUCT, one binary, one process you launch in a directory: 'cd project && agent'. A local-resident coding agent – chat, plus installable browsers. Single- user, localhost; one operator, one working copy, one screen. - A HABITAT CLIENT BY DEFAULT, not a hermetic spike. On first run it forks its own branch (lens-<device>) of the one tree, syncs the AST + data down (local-first.lens- model), and works against that LOCAL LENS as the working source. Edits are LOCAL-FIRST (writes hit the local tree, not /api/prim); sync PUSHES them up (local-first.sync); on disconnect it keeps working and reconciles on reconnect (three-way merge, local-first.sync-conflicts). The image is the durable store + merge authority + index, NOT a live read target. This retires the 2026-08-03 'no Postgres, no branches, no object store' framing for good. - A client that LINKS remote-eval as a capability/library, not a second product. remote-eval owns the rim, the lazy env, the def-cache, the sync engine, the stepping, the wire; the agent owns the surface and the installable browsers and the coding-agent tool set. The agent never reimplements eval. - TWO PLANES, ONE SCREEN, on purpose. The chat agent's read/bash tools edit the operator's LAPTOP repo (the OCaml checkout – the agent LIVES on the machine). The Nopales surface + browser edits go to the LENS -> image. Same process, two planes: one touches the operator's filesystem, one touches the habitat's tree. This is a feature, not a confusion to paper over. - A second home for Nopales: plugin/browser files under ~/.config/agent/plugins/*.pp (global) and .agent/plugins/*.pp (project) are evaluated into one env shared with the session REPL. Live redefinition takes effect on the agent's next turn.

IS NOT: - HERMETIC. 'Agent with no image, ever' is no longer a product target. The lens makes a flaky link survivable (offline = degraded working-set-only, reconciles on reconnect); it does not make 'no habitat' first-class. The 2026-08-03 promise is retired here, deliberately. - Part of the image. The image stays the multi-user habitat product (single-image.borg) + corpus authority; the agent is a sibling CLIENT host, never a second source of truth. - A Nopales fork. Semantics come from pricklypear-lisp unchanged; the agent adds HOST PRIMS + an optional host library layer in .pp, nothing else. A language change lands via lisp-interpreter.borg like any other. - The LLM loop owner. The loop lives in llm/ (shared with pp-remote and any future host); the agent supplies a Loop.host over its local-first rim. See agent-loop. - pi's TUI. The primary surface is HTML (pure-html + htmx, multipart for streams); a minimal line REPL mode exists for scripts and embedding, not as the primary interaction.

nopales-embed

Embedding pricklypear-lisp as the host of the plugin environment

[status: partial]

> *[verify] test: a .pp plugin file defining an event handler is loaded at startup and its handler fires on the agent loop's tool_call event; test: redefining the same handler by evaluating a new definition in the session REPL changes subsequent tool_call behavior without restart; test: an unregistered prim call from a plugin produces a structured Interp_error, not an Unhandled effect escape; test: two evals never interleave on shared Eval global state (serialize via mutex); a nested plugin-triggered eval during an active turn completes correctly*

SEAM. pricklypear-lisp is already the embeddable artifact: Eval.eval_string_direct ~prims env s plus a pure prim table and Control.Prim_call effects. The agent provides a rim in the style of image Control_rim: an Effect.Deep handler over Prim_call dispatching to the host table, with Control.direct_prims set during prim bodies so nested eval dispatches directly.

MOVE DOWN. The rim handler (currently image-only) moves into lisp/lib/control.ml as Control.run_with_rim so both hosts share it. Mechanical move, no behavior change; the image keeps a thin alias until call sites are updated.

CONCURRENCY MODEL. All Nopales evaluation is serialized through one Eio mutex. Eval's global refs (current_budget, last_budget) stay global in v0; per-eval threading is a follow-on only if real pain appears. Nested eval triggered by a plugin callback from inside a tool handler re-enters through direct_prims under the same budget, as the image rim already does.

PRIM ABI. Prims are synchronous (value list -> value). Host prims block the calling fiber (Eio), never Lwt: the agent has no PP_IO switch and no lwt rollback path. plugin-authored Nopales code therefore looks fully synchronous even when a prim performs network I/O.

SHARED soup/ KERNEL. The soup/ tree prims (hypermedia- harness.kernel-prims: soup/parse, select, swap!, apply-parts!, diff, closest-attr, ?) are DB-free and registered here from the SAME shared pure-prims layer the image uses (lifted from image/lib/prims_hyper.ml into pure/). html/ (the pure-html DSL for composing fragments) is a NOPALES package (libs/html), not an OCaml prim layer – the agent loads it as source against these prims, like libs/hyper. lambdasoup + markup become agent deps, joined on the same seam.

HOST PRIM NAMESPACE. Agent prims are prefixed by family: ui/notify ui/confirm ui/select (browser surfaces) session/append session/name on register-tool register-command (plugin API) http/get http/post (Eio; JSON in/out) route route/lookup route/dispatch (agent-route-table; THE HOST SEAM shared with hypermedia-harness) soup/* (shared tree prims, hypermedia-harness) Pure language prims are untouched. The image's DB/auth/ session prims do not exist here; route prims are the agent's LOCAL-LENS backing, not Postgres.

agent-loop

The minimal agent turn loop and tool registry

[status: partial]

> *[verify] test: a mock provider emitting one tool call round-trips: tool executes, result message is appended, loop makes exactly one follow-up model call; test: budget: the loop stops after a configurable max turns and reports rather than spinning; test: tool_call plugin hook can block a call; the model sees the block reason as the tool result; smoke: openai-compat provider streams a real reply end-to-end when AGENT_API_KEY is set*

HOST SEAM. loop/provider/tools live in the shared llm/ package (pricklypear-llm) factored over a small [Loop.host] record (tool_order, tool_entry, fire, eval_tool_body, call_prim); agent/lib/runtime.ml supplies Runtime.loop_host. The same library is linked by pp-remote, which supplies Remote.loop_host over the proxy rim – ONE loop, two hosts (remote-eval.borg agent-loop owns the client half).

SHAPE. One assistant turn = one streaming provider call. If the reply contains tool calls, execute them (serialized eval mutex held only during each tool's Nopales work), append results, call again. Max-turns guard, abort signal, and usage accounting per turn.

PROVIDERS v0. OpenAI-compatible chat completions HTTP (covers synthetic.new and every gateway we use, Anthropic included via its compat endpoint) over Httpun_eio... hand-rolled 1.1 POST because gluten refuses non-socket TLS flows. No curl subprocesses (image llm_client.ml is the cautionary tale), no native Anthropic messages path. SSE parsing in-process once streaming lands.

TOOL REGISTRY v0. read, bash, edit – matching the pi baseline shapes so prompts and habits transfer. bash runs via Eio.Process with timeout and output truncation.

TOOL SCHEMAS. Tool parameter schemas are data (yojson) constructed by the registry, serializable for both provider wire formats. register-tool accepts a Nopales alist schema and a Nopales closure body; the closure runs under the rim inside the loop.

PROVIDER ABSTRACTION. A variant type provider_reply with text/tool-call/delta constructors; each provider module maps wire events to it. Adding a provider = one module, no loop changes.

plugin-model

Nopales plugin API: events, custom tools, commands, session REPL

[status: partial]

> *[verify] test: loading order: global plugins then project plugins; a project plugin can redefine a global binding; test: on/2 registers a handler; handlers for one event run in registration order; tool_call block short-circuits remaining handlers; test: register-tool closures appear in the provider tool list on the turn after registration; test: register-command commands are intercepted before the input reaches the model*

FILES. ~/.config/agent/plugins/*.pp then .agent/plugins/*.pp (cwd), each evaluated in filename order into the shared env. A plugin file is ordinary Nopales; no module ceremony.

EVENTS v0 (subset of pi's surface, deliberately small): tool_call (may block: return (dict "block" #t "reason" s)) tool_result (may replace content) input (may transform text or handle outright) before_turn (may inject a context message) Event payloads are JSON strings manipulated with the frozen dict-* prims – no new value types.

CUSTOM TOOLS. (register-tool name params-alist closure). Params alist maps param name -> "string"|"int"|"bool" with optional "description" entries; registry builds JSON schema. Closure receives one JSON string argument.

COMMANDS. (register-command name doc closure). '/name args' in the input line is dispatched to the closure before model submission.

REPL UNITY. The session REPL (stdin mode and browser /repl pane) is the same env. '(define ...)' at the prompt redefines handler functions live; there is no reload command because there is nothing to reload.

SAFETY. Plugin code runs with the user's full permissions (same trust posture as pi). Project plugins load only after an interactive trust confirmation; the decision is cached per directory hash under ~/.config/agent/trust.json.

agent-route-table

The Nopales route table: route, route/lookup, route/dispatch – THE HOST SEAM between the agent and the image (hypermedia-harness.request-runner). Identical Nopales signatures, host-local backing. Routes bind a path to a Nopales render handler the same way the image's (route ...) does.

[status: implemented]

> *[verify] test: agent-route-table / register-and-render: a (route "/chat" "chat/render") registered by a plugin makes /chat resolvable via route/lookup and renderable via route/dispatch (which invokes the handler through Runtime.apply_value under the eval rim and returns the rendered raw-HTML VStr); test: agent-route-table / thin-context: route/dispatch binds a thin request context so query-param / request-method / hx-request? resolve inside the handler; test: agent-route-table / pg-prim-unbound: a handler calling a Postgres-only request prim (e.g. current-author as a DB row) errors with unbound-prim in the agent – the agent's request context carries only request-shape values*

IMPLEMENTED + VERIFIED (campaign 6.1 make, 2026-12). The whole route-table seam is live in agent/lib/runtime.ml and covered by the agent suite's agent-route-table group (3 tests, all green): (route path handler) registers into the Runtime route Hashtbl; (route/lookup path) resolves path -> handler-name VStr | VNil; (route/dispatch handler method path [params] [headers] [body]) invokes the handler under the eval rim + mutex with a THIN REQUEST CONTEXT bound (query-param / request-method / hx-request?; DB/branch/auth request prims stay UNBOUND, so libs/hyper runs unchanged on both hosts). dispatch_route is the locking wrapper the HTTP gate uses; dispatch_route_raw is the shared in-eval body. render is via Prims_soup.render_handler_value (the shared soup kernel lifted into pure/). The request context also carries session/last-turn-parts (surfaces scope) so a consumed turn replays through hyper/stream.

SHAPE. Runtime.t gains a route table (path -> handler-name). Nopales registers routes with the same (route path handler) shape the image uses; route/lookup resolves a path to its handler; route/dispatch invokes the handler in-process through Runtime.apply_value (under the eval rim + mutex) with a THIN REQUEST CONTEXT bound and returns the rendered raw-HTML VStr.

SAME SEMANTICS, LOCAL BACKING. The image backs these prims with Session.t routes over Postgres + a fat Request_context with manifest/store/branches. The agent backs them with the Runtime route table + a thinner request context – the SAME Nopales signatures, so libs/hyper and the surface handlers run unchanged on both hosts. The route table is where the corpus's routes are materialized; as a local-first host the corpus (incl. defs) comes from the lens, matching the positioning subsection.

ROUTE INSTANCES. Surface pages (/ /chat and the code browser) register as route-table routes in Nopales; the multipart STREAM emission for a chat/agent turn stays OCaml (surfaces subsection) – a route's handler initiates a host stream rather than returning a single fragment for streaming routes.

surfaces

Interaction surfaces: HTML primary, line REPL secondary. The chat/agent surface is NOPALES-AUTHORED htmx fragments + multipart streaming; the stream is emitted by OCaml, the fragments by Nopales.

[status: implemented]

> *[verify] smoke: surfaces / nopales-routes: agent boots and serves the NOPALES-authored chat + code pages on the route table (/ and /code), redefinable from the session REPL without restart; smoke: surfaces / turn-replay-hyper-stream + turn-replay-nested-blocks: a consumed chat turn replays deterministically through libs/hyper (hyper/stream over the captured part list) in the agent host; smoke: scripts/smoke_agent_surface.sh (end-to-end): boots '_build/default/agent/bin/main.exe –serve –mock-script', GET / serves the hx-post=/chat form + hx-multipart asset, GET /assets/hx-multipart.js is 200, POST /chat streams multipart/mixed chunked with all parts + close boundary and TERMINATES promptly, GET /code renders the file tree; smoke: surfaces line REPL / piped turn: 'echo PROMPT | agent –once –mock-script m.json' runs one full turn from piped stdin and prints the assistant reply; 'agent -p EXPR' evals one Nopales expr and exits*

IMPLEMENTED + VERIFIED (campaign 6.1 make, 2026-12; the 6.0 plan pass reconciled the verify stanzas to these actual groups). All five surfaces smokes are live and green: the agent suite's surfaces group (nopales-routes, turn-replay-hyper-stream, turn-replay-nested-blocks – 3 tests) exercises the Nopales-authored routes + deterministic hyper/stream replay; scripts/smoke_agent_surface.sh boots the real binary with –serve + –mock-script and proves the chat page, the hx-multipart asset, the /chat multipart/mixed chunked stream (all parts +

close boundary + prompt termination), and the /code file tree over HTTP/1.1 keep-alive (14 checks, PASS); the line-REPL smokes run via ‘agent –once – mock-script‘ (piped-turn) and ‘agent -p EXPR‘ (pipe mode) – both verified. The vendored htmx assets and embedded libs/html + libs/hyper sources are GENERATED module slices (dune rules producing agent/lib/assets.ml and agent/lib/libs_source.ml into _build); those two build-only modules are covered by the generated-surface-artifacts subsection (their real authored sources are libs/html/lib.pp + libs/hyper/lib.pp + image/static/*.js, named here in this stanza's (implements ?)).

HTML PRIMARY, NOPALES SURFACE. Local HTTP server (Httpun_eio) on 127.0.0.1:18090 by default (–port overrides; a busy port falls back to a random one); prints the URL (and –open to launch a browser). Chat and the code browser are htmx fragment routes on the agent route table (agent-route- table), rendered by Nopales via libs/html – NOT compiled OCaml pages and NOT vanilla-JS SSE. The surface is redefinable live (same env as plugins/REPL). REPL RULE: a chat prompt whose first non-space char is (evaluates as Nopales in the shared env (Runtime.eval_string – same rule as the stdin REPL), rendering the last value as a chat/repl-bubble; anything else is a model turn.

STREAMING IS OCAML. A chat/agent turn's stream is a compiled host concern: provider SSE -> delta assembly -> per-part (html, HX-Target, HX-Swap) assembly (delegating only the fragment TEMPLATE to a Nopales render fn) -> multipart/mixed frame + flush over the producer-fiber/drain-queue plumbing already proven in the image and the v0.0 agent. Nopales owns the page skeleton and fragment templates; OCaml owns the effects-coloured stream. The parts are ALSO collected as data (session/last-turn-parts) so libs/hyper can replay a consumed turn deterministically (soup/apply-parts!, hyper/stream).

LINE REPL. stdin/stdout mode for scripts, embedding, and tty diehards. Same env, same prims, no htmx. Piped-turn mode (–once / non-tty stdin) reads the whole of stdin as one prompt, runs a single full turn, prints the final assistant text, exits 0.

GUI. Explicitly out. Nothing in the feature list needs native widgets, and a second UI ecosystem doubles the maintenance surface.

UI PRIMS. ui/confirm, ui/select, ui/notify resolve against the active surface: modal/banner component in the browser, y/n prompt on stdin in REPL mode. A blocking ui/confirm in an event handler suspends only the agent fiber; the HTTP fiber keeps streaming because prims block fibers, not threads (Eio), and eval serialization is per-eval, not per-process-park.

generated-surface-artifacts

The dune-generated agent surface modules (Assets, Libs_source) – build slices produced into _build by dune rules, not authored source. Recorded so borge's module cross-reference is covered.

[status: partial]

These two modules do NOT exist as authored .ml source in the tree: dune (rules) in agent/lib/dune generate them into _build from vendor sources – Assets embeds image/static/*.js (htmx.min.js / hx-multipart.js / htmx-2-compat.min.js via asset-mermaid quoted blocks) and Libs_source embeds the libs/html/lib.pp + libs/hyper/lib.pp Nopales packages as OCaml quoted strings so the agent binary is self-contained. Their real authored sources are therefore libs/html/lib.pp + libs/hyper/lib.pp + image/static/*.js (named in the surfaces stanza's implements) – these modules are only the build slices.

BORGE MODELLING LIMIT. Because they exist only under _build, borge's implements-exist check (which runs on Implemented / Verified stanzas) would flag them "file not found"; they are therefore declared here on a PARTIAL stanza (no existence check) so the dune module cross-reference stays covered without a false positive. Their real build/authoring home is the surfaces stanza.

v0-non-goals

Explicit exclusions for the first working version

[status: implemented]

OUT for v0, even though pi has them: - Session trees / fork / resume (sessions are one JSONL file per directory under .agent/sessions/, appended only) - Compaction hooks, custom TUI widgets, theme system - Provider discovery / model registry beyond a ~/.config/agent/config.toml table - Sandboxing of any kind (bash is bash) - Sharing sessions across machines

HARD OUT (forever, or a different project): - Postgres or any server-side store - Multi-user, auth, branches, promote - Lwt - Language changes made in agent/ instead of through lisp-interpreter.borg

10 borge/remote-eval.borg

pricklypear-remote-eval

Pricklypear book chapter: remote-eval – the CAPABILITY/LIBRARY that lets a client process sync the one tree's AST + data down, eval Nopales locally, and proxy out-of-working-set prim calls up to the image. The seam is one Prim_call effect handler swapped for an HTTP proxy; the image keeps the corpus, the data plane authority, and the merge index. No surface, no chat – the AGENT product (agent.borg) links this; so would a debugger, test harness, or future CLI.

[status: planned]

– *agent note:* 2026-12 ALIGNMENT PASS (product vs capability – the line the code was reaching for). HISTORY: remote-eval arrived 2026-08-14 explicitly as 'NOT a new product – a mode of the Prim_call seam the agent host already uses, with the rim replaced by an HTTP proxy. agent/ is the local-only instance; pp-remote is the relaxed instance.' The code forked instead: pp-remote grew a chat surface + /code browser + –serve (a second product), and the two hosts duplicated the chat templates, the serve functions, and the t-records. The duplication is a SYMPTOM of a missing noun, not an independent design. ALIGNMENT: (1) remote-eval is the CAPABILITY/LIBRARY: sync the AST/data down, eval locally, proxy out-of-working-set prims up to the image, three-way merge on reconnect. NO surface, NO chat. (2) The AGENT (agent.borg positioning) is the PRODUCT that links it: one process, one screen, own-forked branch, local-first edits synced up, installable browsers. (3) pp-remote shrinks back to this library + a minimal /eval harness (a capability DEMO, not a product); the chat surface + chat/* templates + /code lens browser the prior session built MOVE to the agent (one copy), and the shared llm/ loop the agent already links stays shared. (4) local-first.borg already owns the merge/conflict/identity/semantics-skew open work (sync-conflicts); remote-eval points at it, does not re-spec it. WHAT THIS PASS DOES NOT DO: restatus the implementation subsections (client, caching, placement, stepping, agent-loop) or touch code – this redraws the POSITIONING line only; a make pass retires pp-remote's product surface, collapses the duplicated chat into one on the agent, and re-roots the agent's eval path on local-first (own-fork, sync-push, offline-tolerant). The top-level doc's old phrase 'the client keeps ... the interactive surface (REPL, stepping debugger, agent loop)' was the conflation this pass fixes: the interactive surface + agent loop belong to the PRODUCT (agent.borg); remote-eval is the capability the product is built on.

– *agent note:* 2026-08-14 chapter created from the remote-eval design discussion. Thesis: the interpreter is a pure portable library (~few kloc OCaml, no service deps); the corpus (function_defs, type_defs, data_rows) is branchy rows in Postgres. They are separated along exactly the right line – the cheap-to-copy thing travels, the expensive-to-replicate thing stays home. Remote-eval is NOT a new product: it is a mode of the same Prim_call seam the agent host already uses, with the rim handler replaced by an HTTP proxy. agent/ is the local-only instance today; pp-remote –image=URL is the relaxed instance. Statuses are honest: the substrate is implemented (the seam, run_with_rim, read_eval_all_direct, the api-eval JSON boundary from i-fields-as-string 2026-08-12), everything else is planned.

– *agent note:* 2026-08-14 RECONCILE: a shared codec already exists – pure/prims_pure.ml (pricklypear-pure-prims, wrapped false as Prims_pure) defines value_to_json AND json_to_value (the inverse), explicitly shared between the image and a browser client per its dune comment. So the substrate codec is DONE; the remaining codec work is the ERROR codec (Interp_error/Prim_error/Reader_error -> JSON with kind/name/line/col, currently inline in image/lib/http_pages.ml api_eval) factored into pure/ + its inverse, plus a round-trip property test. ALSO RECONCILE: borge/structural-editing.borg client-eval already plans a browser client/ package (interp.js, js_of_ocaml –effects=cps, pure-prims-only, optimistic structural ops reconciled via op-protocol). client-eval and remote-eval share the seam (same pure interpreter, prims_pure factored out) but DIFFER: client-eval leaves DB prims UNBOUND in the browser and reconciles via op-protocol; remote-eval PROXIES DB prims over HTTP. Complementary, not duplicate. To avoid the client/ package-name collision (client/ is a tracked browser package), remote-eval's native host is the remote/ package (binary pp-remote), NOT client/.

– *agent note:* 2026-12 CAMPAIGN 7.0 PLAN PASS (server-endpoint + caching spec) – reconcile + spec the tree-time dependency. Revealed the remote-eval implementation is ALREADY COMPLETE and ON MASTER (built since the 2026-08 design, merged as a chain: 1a ee309e3 shared error codec factored into pure/prims_pure.ml; 1b ff3b0a5 migration 017 branch_generations counter + Store.current_generation; 2a 540e1a2 POST /api/prim endpoint with server-only refuse-or-forward; 2b 8e75797 remote/ package + proxy rim (pp-remote); 3 623ddd9 hybrid placer + e01a3cc shared http_transport + de61e34 pp-remote hook + f1dae70 transaction-routing tests). VERIFIED green this pass: dune build @all clean; remote/test/placer_test 5/5; remote/test/remote_test 7/7; image/test/api_prim_test (server-endpoint refuse-or-forward dispatch: proxyable prim ok, server-only refused-not-executed with structured server_only kind, classification derived from register_server_only metadata, never a hand list); image/test/branch_generation_test (write-in-ancestry bumps / distinct (branch,generation) key / no-write unchanged / type+route+shared-data bumps). So server-endpoint (7.1) and placement + proxy-rim client (7.3) substrate are DONE; the CLIENT-SIDE DEF CACHE – remote/lib/def_cache.ml, keyed (branch, generation) – is NOT yet built (that is the open 7.2 make slice). THE TREE-TIME DEPENDENCY (the point of this plan pass, per campaign 7.0): the caching subsection's generation key is NOT a remote-eval invention – it IS migration 017 branch_generations, which tree-time (campaign 5, landed 2026-12) PROMOTED from a cache-invalidation signal to the head/gen coordinate (the reads-as-of-gen clock; (gen-now) prim + Store.current_generation). So the def cache must key on the SAME coordinate tree-time reads as-of and the (gen-now) clock reads – one counter, one namespace. Enumeration: server-endpoint (planned; make 7.1 flips), client (planned; make 7.2 – proxy rim done, def_cache open), caching (planned; make 7.2 – 017 counters done, def_cache open), placement (planned; make 7.3 – done), stepping (planned; make 7.3 – explain.ml stepping evaluator + proxy-rim integration NOT built). This is a spec-reality reconcile + dependency-pin pass; NO status flips here (7.1/7.2/7.3 make own the flips).

– *agent note:* 2026-12 CAMPAIGN 7.1 MAKE (server-endpoint) – spec status flip planned->implemented only (implementation was already complete + merged 2026-08; verified green in the 7.0 plan

pass + re-verified this pass). Reconcile: the endpoint + refusal machinery actually ships in image/lib/http_pages.ml (POST /api/prim, api_prim handler via Primitives.dispatch) + image/lib/primitives.ml (register_server_only / register_server_only_sync / is_server_only metadata tags + Primitives.dispatch refuse-or-forward) + pure/prims_pure.ml (shared value_to_json / json_to_value / error_to_json codec – the wire boundary the client observes as identical to in-process eval). The previous (implements ?) listed store_db.ml which shares NO code with this endpoint (dispatch goes through session.prims + Primitives.dispatch, not store_db) – corrected to the true set. Verify stanza unchanged: all four checks map to image/test/api_prim_test.ml (5/5: same-value-as-in-process, unregistered-name Prim_error, server-only refused-not-executed with structured server_only kind, metadata-derived classification, DB CRUD proxyable on a real table). Re-verified green this make: dune build @all clean; api_prim_test 5/5; placer_test 5/5; remote_test 7/7; branch_generation_test 5/5; borge check / drift clean. server-endpoint is now implemented; client (7.2) + placement (7.3) keep their own plan/make cycle (def_cache is the open 7.2 slice; stepping explain.ml is the open 7.3 slice).

– *agent note:* 2026-12 CAMPAIGN 7.2 MAKE (client + caching) – built the open 7.2 slice: the CLIENT-SIDE DEF CACHE (remote/lib/def_cache.ml, keyed (branch, generation)). This is the caching half of remote-eval that the 7.0 plan pass flagged as the one not-yet-built piece. Def_cache holds (def-name -> (value, source-generation)) + a pinned (branch, generation) coordinate; [check] is the ONE CHEAP ROUND TRIP (current-branch + gen-now, both proxyable read prims on the image = the SAME migration-017 coordinate tree-time reads as-of – the campaign-7.0 dependency pin: one counter, one namespace) returning the names to evict: a branch switch invalidates the WHOLE cache (namespace), a generation bump evicts only stale (loaded-before-the-new-gen) defs for reload, no writes returns [] without touching anything. [is_active] gates the round trip so a pure-compute / def-free session does ZERO HTTP. Wired into the lazy env: Remote.load_def now records each def in the cache at the current generation, and eval_local's per-eval (and per-miss) check evicts stale ENV bindings so the next reference re-fetches the NEW source from the server (the writer is the source of truth; the client cache never is). Added current-branch + gen-now to the client's proxyable prim set. NOTE ON SHARED-WORKSPACE CONCURRENCY: the working tree already carried a partial scaffolding of this exact design (a [cache]/[cache_check]/[last_gen] field set + a [def_cache] dune module entry from an overlapping slice); this make reconciled it into the single coherent [Def_cache] module (transport-driven, no separate check hook needed) + the [eval_local] eviction integration, leaving the tree compiling at ONE design (the scaffolding fields were redundant – the module subsumes them). Verify maps to remote/test/def_cache_test.ml (4/4: generation-bump reloads stale, branch-switch invalidates whole cache, no-writes single cheap round trip, client lazy-env reloads after bump). dune build @all clean; dune runtest exit 0 (7 remote + 5 placer + 4 def_cache green); borge check / drift clean (zero new drift – def_cache.ml + def_cache_test.ml + the two verify-stanza flips are the whole delta). client + caching flipped planned->implemented.

– *agent note:* 2026-12 CAMPAIGN 7.3 MAKE (placement + stepping) – reconciled the two remaining remote-eval stanzas to implemented. PLACEMENT was already complete (remote/lib/placer.ml:

hybrid LOCAL vs REMOTE(/api/eval) via a pure AST walk counting prim calls + hard-routing (with-transaction ?) server-side; placer_test 5/5) – this make verified it green and flipped the status. STEPPING is the LLM-debugger substrate, brought to a single coherent design and LANDED (a prior loop left a substantial in-flight implementation in the working tree – explain.ml trace/stepper + remote.ml stepper_of – that did not compile; this make completed the wiring, reconciled to ONE design, and verified): (1) [Eval.current_steps] (lisp/lib/eval.ml, a small additive accessor on the already-implemented eval module; module authority stays with lisp-interpreter.eval, NOT double-claimed here) exposes the running per-eval step counter; (2) [Explain.trace] (lisp/lib/explain.ml, the stepping-unique module) runs src ONCE under a proxy-style rim (intercepts Control.Prim_call in a match_with), services local pure prims (the local table) and forwards every other prim via [transport] as a ONE-PASS interleaved event list: [Local n] = n local pure steps since the last wire crossing (sampled via current_steps), [Remote_ok]/[Remote_err] = a remote prim round trip (distinguished from local steps); (3) [Explain.stepper]/[step]/[continue]/[add_breakpoint]/[break_on_remote]/[fra
drive the already-produced trace (PRESENTATION stepping – NEVER re-evals, so side effects are never repeated and the wire is never re-crossed; a breakpoint just stops walking the trace). This satisfies the verify stanzas: a breakpoint on a remote prim pauses mid-expression while the data plane still crossed the wire exactly once (the transport call count proves the server saw the same single call a fast eval would – the server is unaware; stepping is a client concern by construction); a trace shows local steps interleaved with remote round trips, distinguished. Remote client integration: [Remote.stepper_of] (remote/lib/remote.ml; module authority stays with the client stanza) runs Explain.trace over this client's env + local_prims + transport. Verify: new lisp/test/stepping_test.ml 4/4 (trace interleaves local/remote; breakpoint on remote prim with server-unaware single-call proof; single-step without re-eval; multiple round trips interleave); existing suites green (lisp 140, recursion 7, codec 27, ast_diff, remote 7, placer 5, def_cache 4). borge drift: spec NONE / structural NONE / code 12 = the pre-existing out-of-scope BORGE-1786924137 set (zero new – explain.ml + stepping_test.ml wire into the stepping implements clause; the eval.ml + remote.ml additions ride their existing owner stanzas). placement + stepping now implemented – campaign 7 (remote-eval) is FULLY complete.

– *agent note:* The two deepest consequences, named up front so the sections stay honest: (1) the store stops being where Nopales RUNS and becomes where Nopales LIVES – running is what clients do; (2) local eval over a durable remote corpus is the substrate for a debugger the LLM-integration chapter can drive (step/trace/breakpoint locally, data plane authoritative remotely). Both flow from one effect constructor and a faithful value<->json boundary.

remote-eval

A CAPABILITY/LIBRARY: a client process syncs the one tree's AST + data down, embeds pricklypear-lisp, materializes function definitions lazily, and services Prim_call effects by proxying out-of-working-set prims up to the image over HTTP. The image keeps the corpus, the data-plane authority, and the merge index; the client keeps the interpreter and the local lens. Single-user per client; the image stays multi-user. The AGENT product (agent.borg) links this; so would a debugger, a test harness, or a future CLI. remote-eval owns

NO interactive surface – the surface + the agent loop are the product's, in agent.borg.

positioning

What remote-eval is and is not – the CAPABILITY, against the agent (the product) and the image (the authority)

[status: implemented]

IS: - A CAPABILITY/LIBRARY, not a product. Sync the AST + data down (local-first.lens-model), eval locally for speed, proxy out-of-working-set prim calls up to the image (/api/prim; placement routes prim-dense/transactional blocks to /api/eval wholesale), three-way merge on reconnect (local-first.sync). The seam is one effect – Prim_call – with the handler swapped for an HTTP proxy; the interpreter never knows the difference. - A relaxation of one assumption: env materialization is eager+local in the image, lazy+remote in the client (defs fetched on miss, cached, invalidated on generation bump – caching). Every other invariant (branch inheritance, transactional prims, author identity, store as source of truth) is preserved. - The substrate for the LLM-debugger story: stepping / trace / breakpoint are purely a client concern because the data plane still crosses the wire exactly as a fast eval would – the server is unaware a breakpoint fired. - Linked by the AGENT product (agent.borg), and linkable by anything else (a debugger, a test harness, a future CLI). pp-remote is this library PLUS a minimal /eval harness that DEMOS the capability; it is NOT a product.

IS NOT: - A PRODUCT. It owns NO interactive surface – no chat, no installable browsers, no –serve product mode. The surface + the agent loop belong to the product (agent.borg). The chat + chat/* templates + /code lens browser a prior session built INTO remote/ are a conflation this pass retires: they MOVE to the agent. - A Nopales fork. Same interpreter, same reader, same special forms. A language change for remote-eval is a lisp-interpreter.borg change, full stop. - A second store. The client holds a cache of definitions + a governed LENS (local-first.borg), never the source of truth. Quit the client, lose nothing but transient REPL state; merge on reconnect reconciles. - Multi-user on the client side. One client = one user's interactive session. The image remains the multi-user habitat (multi-user.borg). A browser/WASM client is a later trust-boundary problem, explicitly out for v0.

substrate

The seam remote-eval relaxes, and the boundary that makes it honest

[status: implemented]

> *[verify] test: Control.run_with_rim services Prim_call from a caller-supplied table; a second table (the proxy) drops in with no interpreter change; test: Eval.read_eval_all_direct evaluates a file's forms under a caller-hosted rim (no rim installed by eval itself); test: value_to_json . json_to_value is identity over the value subset that crosses the wire (VStr/VInt/VFloat/VBool/VNil/VList/VPair; VLambda/VPrim/VMacro serialize to a sentinel and DO NOT round-trip – see hard-constraints). NB: json_to_value maps JSON strings to VStr (no VSymbol on the wire); symbols are a read-side concern, not a wire concern.; test: an Interp_error / Prim_error / Reader_error serializes via the shared error codec to the error-kind JSON shape already used by /api/eval, and the inverse re-raises the same exception the local rim would*

THE SEAM. One effect constructor: | Prim_call : string * value list -> value Effect.t in lisp/lib/control.ml. The image rim (Control_rim, now an alias for run_with_rim) services it from the in-process store prim table. A proxy rim is a second handler for the SAME effect: Prim_call(name, args) -> HTTP POST -> deserialize value. The interpreter is identical; only the handler swaps.

THE BOUNDARY. Every value that crosses the wire must serialize losslessly. The shared codec ALREADY EXISTS in pure/prims_pure.ml: Prims_pure.value_to_json emits native JSON (with nested-JSON-in-VStr re-parsing – the i-fields-as-string 2026-08-12 fix) and Prims_pure.json_to_value is the inverse. The remaining codec work is the ERROR codec (Interp_error/Prim_error/Reader_error -> JSON with kind/name/line/col, currently inline in image/lib/http_pages.ml api_eval) factored into pure/ + its inverse (error_of_json) so the proxy rim re-raises the same exception the local rim would, plus a round-trip property test. The acceptance gate is ONE shared, TESTED codec both server and client link – NOT two hand-maintained encoders.

DEFS AS DATA. An env is function_defs rows with branch inheritance via the CTE. pp_sync pull already ships source out; boot.ml/loader.ml turn source+defs into an env. The client lazy-loads defs on miss instead of eagerly at boot: resolution is already data, not a code-load step.

server-endpoint

POST /api/prim: one new endpoint next to api_eval on the image
[status: implemented]

> *[verify] test: /api/prim {name,args} returns the same value (and same error kind) as evaluating the prim in-process under the caller's identity; test: a server-only prim (route/fork/merge/auth-admin) is refused with a structured error citing locality, not silently executed; test: refuse-or-forward classification is derived from prim-registry metadata, not a hand-maintained list; smoke: a remote client drives a full (get 'todo id) -> (update ...) round trip over /api/prim under basic auth*

SHAPE. POST /api/prim with JSON {name: str, args: [json?]} next to api_eval in http_pages.ml – ~40 lines, reusing the shared value_to_json/error-kind codec. Auth is the same basic-auth/cookie path /api/eval uses; the client passes its identity per call. No new privilege – a prim call is equivalent privilege to an eval that performs the prim.

GRANULARITY. A prim endpoint is per-DB-hit granularity, NOT per-expression. That is the honest trade: it enables local pure computation between DB touches, at the cost of a round trip on the worst axis (every DB hit). Hybrid placement (see placement subsection) is the mitigation, not a bigger endpoint.

REFUSE-OR-FORWARD. Some prims MUST execute server-side to mean anything: route, fork, merge, checkout, auth admin prims, jobs/media/net. The rim refuses these with a structured error citing server-only locality rather than pretending to proxy them. The classification is DERIVED from prim-registry metadata (a locality tag set at registration), never a second hand-maintained list – that file would drift.

NO TRANSACTION OVER WIRE. (with-transaction ?) cannot span HTTP calls; the server keeps the transaction body, the client gets the result. This is a hard correctness constraint, not a v0 limitation – see hard-constraints.

IMPLEMENTED (campaign 7.1 substrate, 2026-08, merged; plan-pass reconciled 2026-12). The endpoint + refusal machinery ships in image/lib/primitives.ml + http_pages.ml: - [Primitives.register_server_only] / [register_server_only_sync] tag a prim server-only AT REGISTRATION (route, fork, merge, checkout, create-user, is-admin?, ?); [Primitives.is_server_only] reads the same registry metadata the registration sites wrote (never a hand list in the endpoint). - [Primitives.dispatch name args] = the one-call refusal-or-forward path (refuse in front, same-table exec behind), factored out of http_pages.ml so it is unit-testable without an HTTP server; the POST /api/prim endpoint is a thin residue over it. Auth is the same basic-auth/cookie path /api/eval uses; a prim call is equivalent privilege to an eval that performs the prim. - Tests: image/test/api_prim_test.ml covers proxyable-ok, server-only-refused-not-executed (structured server_only kind), and metadata-derived classification (not hand-maintained). GRANULARITY (per-DB-hit), REFUSE-OR-FORWARD (metadata-

derived), and NO-TRANSACTION-OVER-WIRE all stand exactly as specified.

client

remote/ package: pp-remote binary = pricklypear_lisp + a proxy prim rim + lazy env. NB: this is NOT the browser client/ package (structural-editing.borg client-eval, js_of_ocaml interp.js); remote-eval proxies DB prims over HTTP where client-eval leaves them unbound.

[status: implemented]

> *[verify] test: pp-remote boots, connects to an image URL, and evaluates (+ 1 2) with zero prim calls (pure compute stays local); test: a prim call in client eval proxies to /api/prim and the result binds locally for subsequent pure use; test: a function-def referenced but not yet loaded is fetched on first call (lazy env miss) and cached; test: a def loaded at gen N is re-fetched once the branch generation bumps (lazy env reload after invalidation); test: quit the client; re-launch against the same branch; (define ?) made last session is present (server is source of truth)*

PACKAGE. A new remote/ package: pp-remote binary = pricklypear_lisp + pricklypear_pure_prims + a proxy rim that forwards Prim_call to the server. Sibling of cli/, agent/, and the browser client/, not a child of image/. No Postgres link, no Lwt, no store_db.

PROXY RIM DISCIPLINE. The proxy rim is a PURE proxy with zero logic. The moment it grows its own interpretation of a prim, there are two dialects. The only allowed local behavior is: dispatch to the table (cache/local prims), or forward to /api/prim. Nothing in between.

LAZY ENV. Function resolution is data (CTE over function_defs). The client materializes defs on first reference instead of at boot: env miss -> fetch def source from server -> eval into env under the proxy rim. The env is a cache; invalidation is the caching subsection's job.

IDENTITY. whoami/current-author/auth attach to the live session server-side. The client passes its basic-auth identity per prim call (already how /api/eval auth works). The client may NEVER claim an author it does not hold – identity is server-asserted, not client-asserted.

caching

Definition cache invalidation via branch generation; the one cheap round trip

[status: implemented]

> *[verify] test: a write in the client's branch ancestry bumps a generation counter; the client's next cache-check round trip observes the bump and reloads affected defs; test: a checkout to a different branch invalidates the whole local def cache (branch is the namespace); test: with no writes, the cache-check round trip is a single cheap HEAD-ish returning unchanged-generation, not an N-way hash diff; test: a def-free / pure-compute session performs zero cache round trips*

KEY. The cache is keyed by (branch, generation). Generation is a monotonic counter bumped on any write in a branch's ancestry (a promote into main, a define on personal, etc.). The inheritance CTE is decorated with it. The client stores (def-name -> (value, source-generation)) and invalidates on generation change.

WHY NOT CONTENT-HASHING. Pricklypear names are mutable per branch; defs can be redefined. Unison-style content addressing does not apply (no append-only redefinition). Name-version coherence under a branch namespace is the right shape and it is SMALLER than content hashing – one counter per branch, not a hash per def.

THE ONE ROUND TRIP. Cache freshness is a single cheap call: "is my (branch, generation) still current?" Not an N-way diff. This is materialized-view invalidation in a very small costume, and the branch model already gives us the namespace to key it on.

READ-HEAVY CONFIG. Function defs + read-heavy config rows are the cache's first citizens. Hot (data_rows) reads stay remote (placement subsection).

THE GEN COORDINATE IS TREE-TIME'S (DEPENDENCY PIN – campaign 7.0 plan pass). The cache does NOT invent its own version key. The generation is migration 017 branch_generations – the single per-branch monotonic counter bumped by AFTER triggers on function_defs / type_defs / view_bindings writes (covers promote/merge and the one-tree reverse mirror, since they all write the scaffolding tables) and by data_rows writes to the root branch (data is shared/branch-free, so a data write appears in every branch's ancestry). It is surfaced as current_branch_generation(branch_id) (walks the ancestry chain so an ancestor write appears to every descendant via the inheritance CTE), Store.current_generation, and the tree-time (gen-now) prim. Tree-time (campaign 5, landed 2026-12) PROMOTED this same 017 counter from a cache-invalidation signal to the head/gen coordinate: (tree/get :gen), gen-read, gen-diff, undo, versioned_head/history all read as-of it. So the def cache keys on EXACTLY the coordinate tree-time reads as-of and (gen-now) reads – one counter, one namespace. A remote client's 'is my (branch, generation) still current?' check is therefore the SAME cheap current_branch_generation round trip tree-time treats as the reads-as-of-gen clock – no independent version clock to drift.

IMPLEMENTED SUBSTRATE + OPEN CLIENT MODULE. The server-side half is DONE (migration 017 + Store.current_generation + image/test/branch_generation_test.ml: write-in-ancestry bumps / checkout = distinct (branch,generation) key / no-write unchanged / type+route bumps / shared data bump). The client-side half – remote/lib/def_cache.ml keying (def-name -> (value, source-generation)) with (branch, generation) invalidation and the one cheap round trip – is the OPEN 7.2 make slice (cf. the client subsection, whose [implements remote/lib/def_cache.ml] line is the make target).

placement

Hybrid placement: prim-dense / transactional exprs go to /api/eval, pure compute stays local

[status: implemented]

> *[verify] test: an expression containing (with-transaction ?) is detected as non-proxyable and POSTed to /api/eval wholesale; test: an expression with zero prim calls evaluates locally with no round trip; test: a prim-dense expression above a call-count threshold is routed to /api/eval rather than issuing N prim round trips; test: placement decision is logged/visible so the user can see WHY an expr went remote (no silent boundary)*

THE LATENCY INVERSION. Nopales product code is prim-heavy: (get 'user id), (rows ?), (create ?). A pure proxy rim puts a round trip on the worst axis (every DB hit) rather than per-expression like /api/eval does today. v0 (pure proxy) LOSES to pp_eval on prim-dense code. That is acceptable for interactive/definitional work (REPL, debugger, tests) and wrong for hot loops.

THE HEURISTIC. Detect prim-dense and transactional expressions and POST them to /api/eval wholesale, keeping local eval for pure computation. This is staging the boundary at the HTTP edge: a tiny tracer that counts prim calls and flags transaction blocks, then routes. It degrades gracefully – v0 is "everything proxies"; v1 adds the threshold; the heuristic is the ONLY moving part.

TRANSACTION BOUNDARIES ARE HARD. (with-transaction (lambda () body)) MUST run server-side: the transaction holds (select-for-update ?)

locks across prims in the body, and locks cannot span HTTP calls. The placer treats any transaction-wrapped expr as non-proxyable and forces /api/eval. This is correctness, not optimization.

stepping

Local single-step / trace / breakpoint over a proxy rim: the LLM-debugger substrate

[status: implemented]

> *[verify] test: a breakpoint set on a def pauses local eval mid-expression; the data plane (prim calls) still crosses the wire and returns; test: the server is unaware a breakpoint fired (stepping is purely a client concern); test: a trace of an expression shows local pure steps interleaved with remote prim round trips, distinguished; smoke: an agent loop drives a stepping eval against a remote branch: set bp, step, inspect frame, continue*

THE PAYOFF. Once eval runs in a client process you can single-step, trace, breakpoint, and inspect frames locally while the data plane stays authoritative in Postgres. The server never knows a breakpoint happened – stepping is a client concern, prims still cross the wire exactly as a fast eval would.

DEBUGGER AS PRODUCT. This is the right substrate for the LLM-integration chapter: a debugger the model can drive. A model that can step its own Nopales, inspect frames, and test hypotheses ("what if this def were X?") against a durable corpus gets git-worktree-style experimentation for running programs – something neither Smalltalk (one resident image) nor the web (stateless client) can say.

REPLAY. Because defs are durable server-side, a client can pin a (branch, generation) snapshot and replay an expression against that exact corpus later. Checkpointing and hypothesis-testing eval become client-side operations against durable data, not reimplementation.

explain.ml + proxy rim is the whole kit: the stepping evaluator already exists as a pure concern; the proxy rim is the only addition. The story writes itself from there.

agent-loop

The LLM agent loop runs CLIENT-SIDE: the shared llm/ package (provider + turn loop + builtin read/bash tools) is host-agnostic, and pp-remote supplies the same Loop.host seam the agent does. The API key is a value (AGENT_* env); only the data plane (DB prims inside tool bodies) crosses the wire.

[status: implemented]

> *[verify] test: a Nopales plugin tool (register-tool ?) whose body calls a DB prim: the loop runs on the client, the prim crosses to /api/prim exactly once, and the tool result reaches the model as a tool-role message; test: a builtin tool (bash) executes LOCALLY through the proxy rim's local table – zero wire crossings; test: an (on "tool_call" ?) hook blocks a call and the model sees the block reason – same hook semantics as the agent host; smoke: pp-remote –serve –mock-script: GET / renders the Nopales chat page; POST /chat streams multipart/mixed, chunked-framed, terminates promptly (scripts/smoke_remote_surface.sh)*

THE SEAM. The loop lives in llm/ (a shared package sibling of lisp/ and common/): provider.ml is a pure-OCaml OpenAI- compatible HTTP client (no image coupling), loop.ml is the turn orchestration factored over a small [Loop.host] record (tool_order, tool_entry, fire, eval_tool_body, call_prim), tools.ml is the builtin read/bash bodies with an explicit resolve_path + register. The agent links the same library and supplies Runtime.loop_host; pp-remote supplies Remote.loop_host over the proxy rim. ONE loop, two hosts – "agent/ is the local-only instance; pp-remote is the relaxed instance" is literally true in the code, not just in this spec.

THE KEY IS A VALUE. Provider credentials are client-side values: AGENT_API_KEY / AGENT_BASE_URL / AGENT_MODEL (the same env the agent binary reads; ./.env auto-loads; –mock-script substitutes a canned provider for tests/demos). No server round trip, no server-held secret. The provider call runs on the client machine; the corpus + data plane stay on the image.

WHAT CROSSES THE WIRE. Only Prim_call. A plugin tool's body evals under the proxy rim, so (get 'user id) inside a tool crosses to /api/prim exactly as a REPL eval would. Builtin tool bodies (read/bash) are OCaml prims in the LOCAL table – zero wire. (on "tool_call" ?) hooks and register-tool plugins work identically to the agent host, so plugin sources transfer between the hosts.

THE SURFACE. / is the chat page (Nopales-authored; the chat/* fragment templates carry the same names/shapes as the agent's surface so a restyle transfers); POST /chat streams multipart/mixed htmx parts from the same emitter shape the agent serves. The eval box moved to /eval; /code stays the lens browser. session/last-turn-parts captures a consumed turn's parts for hyper/stream replay, as on the agent.

V0 POSTURE. A chat turn holds the request mutex for its whole lifetime (single-user local harness – other tabs park until the turn finishes). The agent interleaves via its per-eval lock; the client has no eval mutex yet. Filing, not fixing.

hard-constraints

Constraints that are correctness, not v0 limitations – do not relax them

[status: implemented]

TRANSACTIONS DON'T SPAN PRIM CALLS. (with-transaction ?) holds (select-for-update ?) locks across prims in its body. Locks cannot survive an HTTP round trip. Any transaction-wrapped expression MUST execute server-side (placement forces /api/eval). Relaxing this corrupts the store.

CLOSURES DON'T CROSS THE WIRE. VLambda/VPrim serialize to a sentinel and do not round-trip. A server function invoked remotely returns a sentinel, not a callable. This is why function resolution is data (def source fetched and eval'd locally into the env), not RPC-into-a-closure. The proxy rim never tries to call a remote closure.

VERSION SKEW IS THE TAX. The corpus outlives the interpreter by design (defs are durable; the interpreter is a library that rebuilds). A story for "branch pinned to Nopales-semantics vN" is required once the corpus is the product: either semantics versioning in the env, or a migration lane when lisp/ changes behavior. poohstack "everyone rebuilds on switch" works for devs; it will not work for durable multi-version corpora.

IDENTITY IS SERVER-ASSERTED. The client passes basic-auth; it may not claim an author. whoami/current-author attach to the live session server-side and are returned BY the server, never set BY the client.

v0-non-goals

Explicit exclusions for the first remote-eval version

[status: implemented]

OUT for v0, deferred: - Browser/WASM client. Pulling defs from the server and executing them locally is fine for YOUR OWN corpus; it is a trust boundary the moment branches are multi-user. Later, with a capability model, not now. - Client-side caching of data_rows (only defs + read-heavy config rows are cached in v0). - Server-side caching of client state. The server keeps the corpus and the data plane; it does not memoize per-client. - A second transport. HTTP+JSON is the only wire in v0; no binary protocol, no websocket prim channel.

HARD OUT (within remote-eval; RESOLVED in-book by local-first.borg): - A second store on the client. In REMOTE-EVAL (the online mode) the client is a cache and an interpreter, never a source of truth. The offline mode this

hard-out appears to forbid is NOT a different project – it is local-first.borg, same book, same Prim_call seam: a LENS is a governed branch of the one tree with merge semantics (three-way tree-diff, branching.borg), not an uncoordinated copy. The hard-out still stands for exactly what it was written against: an UNCOORDINATED client copy. remote-eval and local-first are two policies of one seam (local-first.positioning): the data plane crosses the wire per-hit online, or resolves against the local lens offline and syncs on reconnect. The reframing is honest only because the one tree + three-way tree-diff merge already exist; without them this hard-out stays hard.) - Multi-user on the client. One client = one user. - Language changes made in remote/ instead of through lisp-interpreter.borg. The client is a host, not a fork.

11 borge/local-first.borg

pricklypear-local-first

Pricklypear book chapter: local-first – a local lens is a branch of the one tree whose root lives on the client. The pad detaches, roots independently, and grafts back. This revisits remote-eval's 'no second store on the client' hard-out: the hard-out was written against an uncoordinated client copy; a lens is a governed copy whose merge semantics already exist (three-way tree-diff, branching.borg / one-tree.branching-interaction). Same Prim_call seam, same tree, different lens policy.

[status: planned]

> – *agent note:* 2026-08-14 chapter created from the local-first design discussion. Thesis: one-tree + user-owned data + three-way tree-diff merge are, almost without addition, the local-first primitives. The 2026-08-02 /data revision (data.u.<user>.*) was made for multi-user isolation and turns out to also fix the replication unit – the isolation boundary and the replication boundary are the same shape. remote-eval is the online-remote mode (local compute, remote data plane); local-first is the offline-capable mode (local lens, sync on reconnect). One seam, two policies. The cactus metaphor (pads = lenses, grafting = merge, pads = lenses, grafting = merge) is evocative but is NOT load-bearing in this spec text – it lives in prose, not the source of truth. (2026-08-29 NAMING UPDATE: 'glochid' is RESERVED for the local-first harness itself – the detachable spine you pull off the pad and work with; see borge/agent.borg. The conflict work this note once called glochids is named sync-conflicts below, no metaphor.)
>
> 2026-12 campaign 8.0 plan pass: lens-model + replication-units pinned for the 8.1 make (details CONTRACT blocks below). A lens = a CHILD BRANCH of the checked-out branch via (fork) / Store.fork_branch (P5 empty-lens fork); the server branch stays the parent; single-writer-per-lens pins to the branch model; code sync = branch-pointer move (promote/merge), data sync = in-place reconcile. Replication unit = data.u.user, data.shared, b.lens plus branch ancestry; path-prefix RLS confinement is the replication boundary. Sync-negotiation handle = the branch,generation coordinate – the same migration-017 branch_generations / gen-now clock remote-eval.caching already keys on (one counter, three consumers). Status stays planned; the 8.1 make flips.

local-first

A client process that holds a durable local lens of the one tree, evaluates Nopales against it offline, and reconciles with the server on reconnect. The server keeps the corpus, the data plane authority, and the merge index; the client keeps a working copy and the interactive surface. Single-user per client; the server stays multi-user.

positioning

What local-first is and is not, relative to remote-eval, one-tree, and branching

[status: implemented]

IS: - A MODE of the Prim_call seam and the one tree, not a new product. remote-eval proxies DB prims over HTTP to a remote store; local-first resolves them against a LOCAL lens and syncs the lens on reconnect. Same interpreter, same rim abstraction, different backing for the data plane between syncs. - A BRANCH OF THE ONE TREE whose root lives on a client. This is what

reopens remote-eval's "no second store on the client" hard-out honestly: the hard-out was written against an UNCOORDINATED client copy (a second source of truth with no merge semantics). A lens is a GOVERNED copy – it is a branch, and branches already have merge semantics (branching.borg three-way tree-diff). The lens is not a second store; it is a remote branch of the one store. - The generalization remote-eval was deliberately scoped NOT to be. remote-eval's v0 is online-remote (local compute, every DB hit crosses the wire); local-first is offline-capable (local lens, DB hits resolve locally until sync). The placement heuristic (remote-eval.placement) is shared: pure compute stays local in both; the difference is whether DB reads cross the wire per-hit (remote-eval) or read the local lens (local-first) until reconnect.

THE USER LOOP, IN ONE BREATH. Fork locally (lens-model), evaluate against the local lens (local-substrate), sync in the background on reconnect (sync). "Invisible" is the property of the COMMON CASE – disjoint edits union cleanly and the user never sees the machinery. It has ONE designed exception: when the SAME node was edited on both sides since the last sync, the divergence is SURFACED (sync-conflicts), never silently resolved. The obituary rule (habitat-workflow) applies to sync losses.

IS NOT: - A second source of truth. The lens is a branch; the server is the integration point and the merge authority. Quit all clients; the server's tree is whole. - A Nopales fork. Same interpreter, same reader, same special forms (inherited from remote-eval.substrate and lisp-interpreter.borg). - Multi-user on the client. One client = one user's lens. - A relaxation of single-writer-per-lens. Each client is the SOLE writer of its own lens. Multi-writer reappears only at MERGE – exactly where three-way tree-diff already lives. This is git's argument, and it holds here.

lens-model

A local lens is a branch checked out to a client root. Fork = a pad detaching; sync = grafting back. Single-writer is preserved per-lens.

[status: implemented]

A CLONE IS A FORK. Checking out a local lens forks the user's current branch into a client-owned lens (e.g. admin/personal -> admin/laptop-2026-08-14). The fork is a branch like any other: a pointer into the one tree (one-tree.branching-interaction). The client materializes the lens subtree to its local substrate (filesystem – see local-substrate) and evaluates against it.

SINGLE-WRITER PER LENS, PRESERVED. The one-tree invariant ("one unix image serializes structural mutation; no merge-on-write races inside a commit") is preserved PER LENS. Each client is the sole writer of its own lens, so local writes need no concurrency machinery – a write is a local commit, atomicity is the substrate's. Multi-writer reappears only at SYNC, when the lens merges back to the server branch. That is exactly where three-way tree-diff (branching.borg, one-tree.branching-interaction) already lives. Local-first does not invent a new concurrency model; it adds a second site for the existing merge operation.

THE MERGE IS AUTOMATIC AND INVISIBLE IN THE COMMON CASE. "Each clone is its own branch" and "constant merges" are the SAME model, not alternatives – the branch is what makes the merges safe, and the merges are cheap because a solo user's lens rarely conflicts with its own server branch: - DATA is append-mostly and single-writer per user: a new todo on the laptop and a new todo on the server are disjoint nodes; the merge is a clean union, not a conflict. - CODE on a personal garden is effectively single-writer: you are the only editor of your own defs. A merge is a fast-forward unless you edited the SAME def on two devices before syncing – the rare, real conflict. The branch is a CORRECTNESS MECHANISM, not UX ceremony. The user's mental model is "my branch, synced across my devices"; fork-on-clone and merge-on-sync are implementation.

MULTI-DEVICE = STAR TOPOLOGY. Each device is its own lens (admin/laptop, admin/phone, ...) rendezvousing at the server branch (admin/personal) and the user's data subtree (data.u.admin). The server is the integration point. This is the simplest multi-device topology and it falls out of the branch model without addition.

CODE VS DATA SYNC POLICY. The one tree has two storage shapes, and they sync slightly differently – same algorithm (three-way tree-diff over subtrees, keyed on node hashes), different landing policy: - CODE (b.<branch>.{fn,type,route,pkg}.*) is branch-scoped. A clone forks a code lens; sync merges lens -> server branch by moving the branch pointer (the existing promote/merge primitive, branching.borg). - DATA (data.u.<user>.<type>.<id>) is user-owned and BRANCH-FREE (the 2026-08-02 /data revision). There is no branch pointer to move; sync reconciles the user's data subtree in place against the server's copy, server-mediated. data.shared.* is read-mostly seed; clients pull it but propose merges into it (the existing "explicit copy/move into data.shared" rule), preserving server-sole-writer for shared content. One algorithm, two policies for where the merged result lands. Both are subtree hash-diff + reconcile; neither invents a new primitive.

PLAN PASS (campaign 8.0, 2026-12) – lens fork CONTRACT pinned for the 8.1 make. A lens being "a branch checked out to a client root" is made concrete as a CHILD BRANCH of the checked-out branch: - FORK SURFACE. A clone forks the user's current branch into a client-owned lens via the EXISTING (fork ...) primitive / Store.fork_branch (branching.personal- workspace, one-tree P5 EMPTY-LENS fork – copy-on-fork, parent pointer only). The lens is an empty child of the checked-out branch; the server branch stays the parent. Naming: <owner>/lens-<device>[-<date>] (e.g. admin/personal -> admin/lens-laptop-2026-08-14). - MATERIALIZE. The client materializes the lens to its local substrate (local-substrate) and evaluates against it: local DB prims resolve against the lens files; the server is only the merge/index authority. - SINGLE-WRITER PER LENS, PINS TO THE BRANCH MODEL. The server NEVER writes into a client lens except via sync/ merge. Local writes are commits on the lens; atomicity is the substrate's (rename(2)). The branch is what makes the merges safe – no new concurrency model. - LANDING POLICY (pinned). CODE sync merges lens -> server branch by moving the branch pointer (the existing promote / merge primitive). DATA sync reconciles the user's data subtree IN PLACE against the server's copy (data is branch-free; there is no pointer to move). Both are subtree hash-diff + reconcile; neither a new prim.

> *[verify] build: dune build @all; smoke: lens/fork + remote/lib/lens_store.ml fork_lens_name name a lens branch <owner>/lens-<device>[-<date>] as a NAMED CHILD of the checked-out branch (empty-lens fork: parent pointer only, zero own rows, resolves through ancestry); smoke: the fork is single-writer; the server branch stays the parent (verified: lens-branch parent_id = checked-out branch)*

replication-units

What travels to a client: the user's data subtree + the checked-out code lens. The isolation boundary IS the replication boundary.

[status: implemented]

THE /data REVISION EARNS ITS SECOND SALARY. The 2026-08-02 user-owned data placement (data.u.<user>.* + data.shared.*) was made to fix MULTI-USER ISOLATION – a session sees only its own data.u.<user> plus data.shared. It turns out to also fix the REPLICATION UNIT: the subtree a client replicates is exactly data.u.<user>.* (that user's rows) plus data.shared.* (the read-mostly seed). The isolation boundary and the replication boundary are the SAME SHAPE – not a coincidence, but the tree being honest about ownership. A subtree scoped to one user for reads is the right subtree to copy to that user's device.

WHAT A CLIENT REPLICATES: - data.u.<user>.<type>.<id> – the user's own data (full) - data.shared.<type>.<id> – shared/seed content (read-mostly) - b.<lens>.{fn,type,route,pkg}.* – the forked code lens NOTHING ELSE. Another user's data.u.<other>.* never replicates to this client – not by a policy added for local-first, but by the path-prefix confinement that already governs reads (one-tree RLS floor). The client cannot ask for, and the server cannot send, a subtree the client's identity cannot read. Replication is authorized reads, batched.

CODE LENS SCOPE. The code lens replicated is the forked branch and its ancestry (defs resolve through inheritance, branching.borg). A client checking out admin/personal replicates personal's overrides plus the system/main ancestry they resolve through – the same env a server-side eval would build, materialized locally. Inheritance is read-time resolution (one-tree tree-model); the client stores the pinned overrides and resolves ancestors per the cache policy (remote-eval.caching generalizes).

NO CROSS-USER REPLICATION. Because replication is the authorized-read subtree, a compromised client that replays its sync token cannot exfiltrate another user's data – it can only request what its identity could read in a normal session. The trust boundary for local-first is the SAME trust boundary the online session already has, restated as "what subtree may replicate." This is why local-first does not inherit a new leak surface from remote-eval's browser/WASM trust punt: the browser punt is about EXECUTING untrusted corpus code in a browser; local-first executes the user's OWN corpus on the user's OWN device, which is the trust relationship the online image already assumes.

PLAN PASS (campaign 8.0, 2026-12) – REPLICATION SUBTREE PINNED for the 8.1 make. The replication unit is exactly the authorized-read subtree: data.u.<user>.* the user's own data (full) data.shared.* read-mostly seed b.<lens>.{fn,type,route,pkg}.* forked code lens (pinned at fork) PLUS the branch ancestry the lens resolves through (the system/main + parent overrides a server-side eval of the lens would build – materialized locally per inheritance read-time resolution). NOTHING outside it. Path-prefix confinement (one-tree RLS floor) is the replication boundary; the client cannot ask for, and the server cannot send, a subtree the identity cannot read in a normal session. Replication is authorized reads, BATCHED.

SYNC-NEGOTIATION HANDLE (the coordinate the manifest exchange keys on). The client records the (branch, generation) it last synced at; that snapshot is the merge base. This is NOT a new counter – it is the SAME migration-017 branch_generations coordinate tree-time promoted to the head/gen clock (gen-now / Store.current_generation) and remote-eval.caching keys its def cache on. One counter, three consumers (def-cache freshness, reads-as-of-gen, last-synced merge base). A sync round negotiates (path, hash) manifests and offers (branch, generation) as the common ancestor.

> *[verify] build: dune build @all; smoke: replication/unit + remote/lib/lens_store.ml in_replication_unit materialize EXACTLY data.u.<user>.* + data.shared.* + b.<lens>.* + branch ancestry (path-prefix RLS confinement is the replication boundary; another user's data.u.<other>.* never replicates); smoke: image/test/local_first_prims_test.ml: lens/fork creates the empty-lens named child; replication/unit returns a well-formed manifest*

local-substrate

The filesystem as lens root. No second query engine; the server is the index.

[status: implemented]

THE FILESYSTEM, NOT SQLITE. A local lens is a directory tree of node-envelope JSON files, one file per node, at its path:

<lens-root>/ b.admin.laptop.fn.todo-get.json b.admin.laptop.type.note.json data.u.admin.todo/<uuid>.json data.shared.flash-deck/<uuid>.json

Each file is the node envelope from one-tree.tree-model ({v,kind,path,type,body,author,doc,hash,created_at, updated_at}). The filesystem IS the path index: subtree queries are read-

dir/find; uniqueness is the filesystem; the envelope is a JSON file. There is no engine to align with Postgres, no second schema, no second set of migrations – the thing the remote-eval hard-out was trying to prevent (a second store) does not arise, because the lens is not a store, it is a materialized subtree.

WHY NOT SQLITE. SQLite imports a second query engine, a second schema, and a JSON1-vs-jsonb semantic divergence that must be kept aligned with Postgres by hand. Its entire reason to exist – concurrent ACID over a shared file – is unused: a local lens has ONE writer (the client), so atomicity is rename(2) and there is no concurrency machinery to justify. SQLite is the answer for a system with concurrent local writers; this system has none.

THE VAULT IS ALREADY FILES. pp-sync ships libs/*.pp, /code serves PP_SOURCE_ROOT, backups are directory dumps. The filesystem substrate IS the vault format – a local lens and a pp-sync snapshot are the same physical thing. A lens is inspectable with cat, jq, grep, diff, git. This is the Smalltalk/Pharo filetree feeling, honest: the image as an inspectable directory.

THE SERVER IS THE INDEX. A local lens is a WORKING COPY, not a query engine. Indexed reads over the whole corpus (tree/find-by with a body predicate, rows with a filter, list-kind over a large type) route to the server – the remote-eval placement heuristic, inherited. The lens holds the working set (open notes, current todos, the def being edited); the server holds the index over the corpus. Pure compute and working-set reads stay local; analytical reads cross the wire. Offline mode degrades gracefully on the analytical axis (indexed queries unavailable) while staying full-power on the working-set axis – the right failure shape for offline.

SUBSTRATE INTERFACE, NOT SUBSTRATE MANDATE. The spec pins the interface (path-addressed node get/put/subtree-list + envelope + hash) and names the filesystem as the v1 substrate. A content-addressed blob store is the natural v2 once content-addressing lands in the tree proper (one-tree.tree-model "content-addressing later"); the interface is stable across that swap. SQLite is not a named future substrate – if a lens ever needs real local indexing, the answer is a content-addressed store or a remote call to the server's index, not a second relational engine on the client.

8.2 MAKE (campaign 8, 2026-12). The substrate interface (path-addressed get/put/subtree-list + envelope + hash) was already in lens_store.ml (8.1) ; this make COMPLETED the replication-unit coverage the local-substrate stanza names:

- CODE-LENS MATERIALIZATION. [materialize_code_node] writes a code-node envelope at its b.<branch>.<kind>.<name> tree path (kind in {fn,type,route,pkg}) carrying the evaluable def source (the (define ?) form) + doc, same one-tree envelope shape + content hash as datum nodes – so sync's three-way tree-diff keys on the same [hash] for code.

- ANCESTRY MATERIALIZATION (read-time inheritance). [record_ancestry] / [ancestry] store the self-first (closest-wins) branch chain (lens, parent, ?, system/main) as <root>/___ancestry.json – the SAME override order store.branch_chain / function_defs use. [resolve_def] walks it, returning the closest b.<branch>.<kind>.<name> envelope (None on miss) – offline read-time inheritance per one-tree tree-model, no server round trip.

- CLIENT WIRING (evaluates against the local substrate). remote/lib/remote.ml takes an optional [~lens_lookup] (substrate def source); when set, [fetch_def] resolves FROM THE SUBSTRATE FIRST and only proxies [function-source] on a miss – the offline-first def path of the lens-model materialization.

- (PATH, HASH) MANIFEST. [node_manifest] walks the lens root and returns every node's (tree path, envelope hash) – the exact manifest the sync stanza's exchange negotiates on.

NOT in this make (kept open, owned by the sync stanza): the bidirectional manifest exchange + three-way merge against the server (image/lib/sync.ml, the actual-8.3 make). Local-first is not shipped end-to-end until 8.3 lands; this closes the substrate's own coverage so 8.3 diffs against a complete manifest

source.

> *[verify] build: dune build @all; smoke: remote/test/lens_store_test.ml: the local-substrate group (path_to_file, materialize_and_get, put_body_hash, subtree_list, code_kind_path, materialize_code_and_resolve, node_manifest, lens_fetch_first) – 10/10 green: code-lens nodes materialize at b.<branch>.<kind>.<name> with an evaluable src; ancestry records + round-trips self-first chain; resolve_def is closest-wins (a lens override beats its ancestry copy) and inherits a def present only on ancestry; the (path, hash) manifest lists every node (code AND datum) and excludes the ___ancestry metadata; Remote.fetch_def consults the substrate FIRST (a transport that raises is never touched on a substrate hit)*

sync

Sync = three-way tree-diff (branching.borg / one-tree.branching-interaction), keyed on subtree node hashes. The exchange is the git protocol in miniature.

[status: implemented]

> *[verify] doc: Sync = the server-side merge authority over tree_nodes, keyed on the same subtree node content hash the client lens substrate computes offline (one-tree.tree-model) – ONE hash scheme, both sides. image/test/sync_test.ml covers the pure core: manifest classification (same / client-only / server-only / diverge = the exchange's conflict DETECTION at node granularity) and the three-way merge planner (base = last-synced, ours = server, theirs = client) with every outcome (unchanged, theirs_add, ours_add, theirs_mod, ours_mod, identical, conflict, theirs_del, ours_del) + the actionable subset + JSON rendering. The sync/plan prim surfaces server_manifest / classify / plan_of (with an optional last-synced base manifest – the tree-time gen coordinate as merge base). The DB-backed harness (server_manifest / apply_client_node / commit) is exercised per the coexistence / deployment passes.*

ONE ALGORITHM, ALREADY SPECIFIED. Sync is the three-way tree-diff that one-tree.branching-interaction already plans for promote/merge. Subtree content hashes (one-tree.tree- model: hash over canonical body, root hashes compose from leaves) give CONFLICT DETECTION at node granularity. Generation counters (remote-eval.caching) generalize from one-way cache invalidation to bidirectional sync negotiation. Nothing new is invented: local-first sync is the merge primitive with a second consumer.

THE EXCHANGE. A sync round is the git protocol in miniature: 1. Client sends its (path, hash) manifest for the lens subtree(s). 2. Server compares against its own subtree hashes; reports which paths differ or are missing on each side. 3. Each side sends the node BODIES (envelopes) the other is missing or has stale. 4. Server applies the three-way merge (common ancestor = last-synced generation) and returns the merged subtree state; client applies the same merge locally. The envelope hash is the negotiation; the file is the transfer. No second query engine in the loop.

CODE SYNC. The forked code lens (b.<lens>.*) merges to the server branch (b.<user>.personal) by moving the branch pointer – the existing promote/merge primitive. Single- writer per user's personal code means this is usually a fast forward; conflict only on the same def edited on two devices.

DATA SYNC. data.u.<user>.* reconciles in place against the server's copy (no branch pointer – data is branch-free). Append-mostly user data means disjoint new nodes union cleanly; conflict only on the same node edited on two devices. data.shared.* is pulled (server-sole-writer); client proposals into shared go through an explicit merge-request, not direct write.

LAST-SYNCED AS THE COMMON ANCESTOR. Three-way merge needs a common ancestor. The client records the (branch, generation) it last synced at; that snapshot is the merge base. This is the remote-eval.caching generation

counter, repurposed from "is my cache fresh" to "what was our last common state." One counter, two uses.

SHIPPED V1 TRANSPORT (data paths, 0bebd9a). The exchange is no longer only a planned protocol: the sync/push prim (image/lib/prims_db.ml) classifies a client data.u.<user>.* (path, hash) manifest against the server, applies ClientOnly pushes through Sync.apply_client_node (the typed-boundary-validated datum write surface; image/test/sync_db_test.ml), returns ServerOnly paths for the client to pull, and REPORTS diverge paths without auto-merging them (sync-conflicts stance: v1 surfaces, it does not silently resolve). pp-remote –sync (remote/bin/main.ml) drives the round – push, pull server-only paths, print DIVERGE lines, and pin the returned (branch, generation) as the next merge base.

CODE-LENS SYNC STAYS THE BRANCH-POINTER MOVE. b.<lens>.* is deliberately OUT of sync/push's scope: code sync merges lens -> server branch via the existing promote/merge primitive (branching.borg), server-mediated. Data sync is in-place reconcile; code sync is a pointer move – one algorithm family, two landing policies, as lens-model pins.

"BACKGROUND" IS CLIENT SCHEDULING, NOT NEW MACHINERY. Sync is round-based; –sync is the manual v1 entry point. Making it invisible is the client invoking rounds on reconnect / on a timer against the same prims – no new server surface is needed or planned for it.

sync-conflicts

The honest open work: conflict RESOLUTION policy, offline identity, semantics skew, transactions-under-sync. Detection is free; resolution is not.

[status: planned]

THE TREE DETECTS; IT DOES NOT RESOLVE. Subtree hashes give node-granular conflict DETECTION (these two nodes diverged since the common ancestor). They do not give a RESOLUTION policy. Naming the policy is the real research-shaped gap – the pads graft easily most of the time; the conflicts bite at sync. Candidates, in increasing ambition: - LAST-WRITER-WINS per node (updated_at arbitrates). Cheap, loses concurrent edits to the same node. Acceptable for data where the rare same-node concurrent edit is a user error anyway. - SEMANTIC MERGE per type (a type declares its merge strategy: todo status is LWW, note body is three-way-text-merge, a counter field is sum). Uses one-tree typed-nodes – the type declaration carries the merge policy. Most honest; most work. - CRDT-ISH field types for hot fields (a counter that sums, a set that unions). Opt-in per field via the type declaration. Heaviest; only where worth it. v0 is LWW-per-node with a conflict SURFACED to the user (no silent loss – the obituary rule, habitat-workflow, applies to sync losses too). Semantic merge is the v1 ambition and the place typed-nodes earns a third salary.

OFFLINE IDENTITY. remote-eval: "identity is server-asserted, never client-asserted." Offline edits must assert author locally and reconcile at sync. The per-node author field + branch ownership give the audit trail; the mechanism is durable client credentials (a signed sync token / device key) presented at sync, with the server asserting the identity onto the merged nodes. The client never CLAIMS an author it does not hold – it stamps its credential; the server validates and attributes. This is a real trust- boundary change from online (where the session asserts per request) and it inherits remote-eval's browser/WASM trust punt as an open question for untrusted-client execution.

SEMANTICS VERSION SKEW. remote-eval hard-constraint already flags "a story for branch pinned to Nopales-semantics vN is required once the corpus is the product." Local-first makes it acute: an offline client runs old interpreter semantics against a corpus that may have evolved at sync. The lens pins a (branch, generation, semantics-version) snapshot (remote-eval.stepping's "pin a snapshot and replay" is the substrate). At sync, a semantics-version mismatch is a merge-gate event: the client must rebase to the server's semantics before

merging edits, or the server re-evaluates the client's def source under current semantics. The honest rule: defs are durable DATA (source), so the server can always re-eval a client's def source under current semantics – the skew is in BEHAVIOR the client observed offline, not in the source. Behavior skew is surfaced, not silently merged.

TRANSACTIONS UNDER SYNC. Online, the remote-eval hard constraint stands: (with-transaction ...) holds (select-for-update) locks that cannot span HTTP, so transaction-wrapped exprs execute server-side. Offline, the client is sole writer of its lens – local transactions work as-is (single writer, locks trivial, the lens substrate provides atomicity via rename). At SYNC, concurrent local transactions become MERGE CONFLICTS, not lock contention – the hard constraint MOVES, it does not vanish. It stays true online; offline it becomes a merge problem this subsection owns. A transaction that must be atomic across sync (rare – most txns are within a single editing session) is surfaced as a conflict on all its constituent nodes rather than silently partial-merged.

STATUS (reconcile after 0bebd9a): DETECTION IS SHIPPED, THE v1 DATA TRANSPORT IS SHIPPED, RESOLUTION IS DELIBERATELY NOT. Campaign 8.3 (sync subsection, implemented) delivered the node-granular divergence detection this subsection's thesis names: image/lib/sync.ml manifests path,hash deltas, classifies them, and three-way-merges (base = last-synced generation, ours = server, theirs = client) to a plan whose conflict outcome is surfaced REPORT-ONLY. The transport the previous reconcile called "not yet built" then landed in 0bebd9a: sync/push (image/lib/prims_db.ml) + pp-remote –sync (remote/bin/main.ml) – a two-way, data-only round that applies client-only pushes, pulls server-only paths, and REPORTS diverge paths with the server copy kept. That report IS the v1 conflict stance, and it is deliberate, not a gap in the transport: no automatic LWW arbitration of a same-node divergent pair – v1 surfaces honestly rather than silently clobbering. What remains genuinely open here is the RESOLUTION POLICY menu above (LWW-per-node as an APPLIED policy, semantic merge per type, CRDT-ish fields), OFFLINE IDENTITY (durable device-signing credential presented at sync), SEMANTICS VERSION SKEW (lens-pinned semantics snapshot + sync-time rebase/re-eval gate), and TRANSACTIONS UNDER SYNC (conflict surfacing for cross-sync txns). Subsection stays planned; the open work is resolution + identity + skew + txns-under-sync, not detection and not the v1 data transport.

phasing

Read-only offline first (achievable, defers the conflicts); write-capable lenses with automatic merge second

[status: planned]

PHASE 1 – READ-ONLY OFFLINE. A client pins a (branch, generation) snapshot, materializes it to the filesystem lens, and evaluates read-only against it offline. No local writes; on reconnect, the client re-syncs the snapshot (cheap – discard and re-pull, since it made no edits). This is much closer to shipped: it reuses remote-eval.caching's generation pin and the filesystem substrate, with NO merge machinery. It delivers most of the local-first FEEL (no spinners on reads, offline read access, instant open) without the merge-conflict policy. The debugger substrate (remote-eval.stepping) works against a read-only lens unchanged.

PHASE 2 – WRITE-CAPABLE LENSES. Fork-on-clone, local writes, automatic merge on sync (lens-model + sync). LWW-per-node conflict resolution with surfaced conflicts (sync-conflicts v0). This is the full local-first design. The hard problems (semantic merge, offline identity, semantics skew) mature within phase 2, not before it.

NOT A PHASE GATE. Phase 1 is a shippable milestone, not a prerequisite – the phase 2 design does not depend on phase 1 being shipped first. But phase

1 is the honest first cut because it defers the one piece of real research (conflict resolution policy) while delivering the working-copy feel.

STATUS (reconcile after b15a06a + 0bebd9a): BOTH FLOWS NOW EXIST AS pp-remote ENTRY POINTS – MANUAL v1, NOT YET BACKGROUND. Phase 1's "user-facing read-only offline flow" the previous reconcile called unshipped exists: pp-remote –lens DIR + –lens-pull materializes a lens and evaluates offline-first against the substrate. Phase 2's "transport not yet built" also landed for DATA: sync/push + pp-remote –sync (0bebd9a) close the write loop – local eval, tree to disk, push up, pull down, diverge surfaced, new (branch, generation) merge base pinned. What is NOT yet shipped: (a) sync as an AUTOMATIC background/on-reconnect loop – today it is user-invoked (–sync); the scheduling is client-side and needs no new server machinery, but the loop itself is unbuilt; (b) code-lens sync through the exchange – b.<lens>.* syncs via the server-mediated promote/merge pointer move, a separate path sync/push deliberately excludes; (c) sync-conflicts resolution POLICY (v1 reports diverge; nothing auto-resolves). Subsection stays planned; the open work is the background loop + the resolution policy, not the transport and not the flows.

non-goals

What local-first is NOT

[status: implemented]

- NOT a second store. The lens is a branch of the one tree; the server is the integration point and merge authority. - NOT multi-user on the client. One client = one user's lens, same as remote-eval. - NOT a Nopales fork. Same interpreter, same reader, same special forms (remote-eval.substrate). - NOT a relaxation of single-writer-per-lens. Each client is sole writer of its own lens; multi-writer appears only at merge. - NOT automatic conflict RESOLUTION. The tree DETECTS conflicts (hashes); resolution is a policy (sync-conflicts). v0 surfaces; it does not silently resolve. - Data lens versioning (IMPLEMENTED v0, 2026-08-23 make). data.u.<user>.* stays single-head and branch-free (this property is preserved). An opt-in data.l.<lens>.<user>.* namespace adds deliberate data forks that diverge and merge back via LWW (v0; three-way tree-diff is v1). The local-first lens materializes and syncs the user's data subtree; the data lens fork makes that first-class. - NOT a second transport. HTTP+JSON is the only wire in v0 (inherited from remote-eval). The filesystem is the local substrate, not a wire. - NOT a browser/WASM client. Executing a user's corpus in a browser is a trust boundary local-first inherits from remote-eval's punt and does not reopen; local-first runs on a client the user controls (laptop, phone), where the trust relationship is "my code on my device."

12 borge/storage.borg

pricklypear-storage

Pricklypear book chapter: storage
[status: implemented]

– *agent note:* 2026-08-02 status reconcile: all section children implemented/verified (schema, no-inheritance, transactions, with-transaction, media); lift project.

storage

Postgres schema: branches, function_defs, type_defs, data_rows. SERIALIZABLE defaults. NO inheritance.
[status: implemented]

schema

Core tables (in migrations/001_branches.sql)
[status: implemented]
– The shared code store CREATE TABLE branches (id uuid PRIMARY KEY, parent_id uuid REFERENCES branches(id), owner text NOT NULL, name text NOT NULL, created_at timestamptz NOT NULL DEFAULT now(), last_synced timestamptz, – for merge diffing UNIQUE (owner, name));

CREATE TABLE function_defs (branch_id uuid NOT NULL REFERENCES branches(id) ON DELETE CASCADE, name text NOT NULL, body jsonb NOT NULL, – serialized Nopales AST doc text, author text NOT NULL, updated_at timestamptz NOT NULL DEFAULT now(), PRIMARY KEY (branch_id, name));

CREATE TABLE type_defs (branch_id uuid NOT NULL REFERENCES branches(id) ON DELETE CASCADE, name text NOT NULL, fields jsonb NOT NULL, – field declarations migrated bool NOT NULL DEFAULT false, schema_vers int NOT NULL DEFAULT 1, author text NOT NULL, updated_at timestamptz NOT NULL DEFAULT now(), PRIMARY KEY (branch_id, name));

CREATE TABLE view_bindings (branch_id uuid NOT NULL REFERENCES branches(id) ON DELETE CASCADE, route text NOT NULL, – e.g. "/inspect/:id" handler text NOT NULL, – function name PRIMARY KEY (branch_id, route));

CREATE TABLE sessions (id uuid PRIMARY KEY, user_id text NOT NULL, branch_id uuid NOT NULL REFERENCES branches(id), pid int, – image instance pid boot_mode text NOT NULL, – 'on-connect'|'always' health text NOT NULL DEFAULT 'unknown', started_at timestamptz NOT NULL DEFAULT now(), last_seen timestamptz); – LEGACY/INERT: process bookkeeping from the – retired fleet design. The image never reads or – writes it. Kept because migrations are – additive-only.

– All Nopales-declared types share ONE table, – data_rows (migration 003), keyed by – (type_name, id) with fields packed as jsonb. – There are no per-type tables; deftype emits no – DDL. data_rows is NOT branched.

CREATE TABLE schema_migrations (version int PRIMARY KEY, applied_at timestamptz NOT NULL DEFAULT now(), description text);

The shared data_rows table lives alongside these system tables (migration 003; see object-model).

SERIALIZABLE is the transaction isolation default for ALL Nopales-driven mutations. Weaker isolation is opt-in per query.

> *[verify] build: dune build @all; build: psql -f migrations/00{1,2,3}*.sql applies clean on a fresh database; smoke: image applies migrations on boot (see image/lib/session.ml boot sequence)*

no-inheritance

Why we do NOT use Postgres INHERITS

[status: implemented]

Postgres table inheritance (INHERITS) is the deprecated feature from the early-90s Postgres implementation. We deliberately do NOT use it.

THE CORE REASON: branches are an OVERRIDE primitive, not a UNION primitive. When Alice forks and overrides 'greet', her branch's view should see ONLY her greet, not main's greet merged in. Postgres inheritance does union (both rows visible), not override.

Modeling override on top of inheritance forces us to write override resolution ourselves anyway (pick the child's row when it exists), so inheritance buys nothing but its quirks.

Other reasons: - Foreign keys / unique constraints don't span inheritance trees properly - PRIMARY KEY doesn't propagate - The planner handles inherited queries with edge cases and unpredictability - Inheritance handles schema evolution (add/remove columns) inconsistently; migrations would be a horror show - pg_dump, psql, admin tools render inheritance weirdly - We'd pin ourselves to non-standard semantics

Plain tables with branch_id FK and override resolution in queries (recursive CTE up the parent chain, or materialized effective-definition) gives us: - Clean indexed SELECTs - Predictable planner behavior - Standard migrations - Full inspectability (the goal) - A clean path to content-addressing later if we want "actually git" semantics

Future options stay open. Inheritance forecloses them.

> *[verify] build: dune build @all; smoke: override-walk + branch isolation (closest-ancestor wins, parent unchanged)*

transactions

Transaction isolation: SERIALIZABLE by default

[status: implemented]

All Nopales-driven mutations run in SERIALIZABLE transactions by default. Postgres detects conflicts at commit and aborts one, forcing retry. For a multi-user personal OS, throughput is not the concern; correctness is.

Opt-in weaker isolation: (query ... isolation: 'read-committed) Available but discouraged. Most user code never touches this; the system does the safe thing by default. (v1.5 – the with-transaction prim already accepts the isolation arg shape, defaulting to SERIALIZABLE; the weaker-isolation passthrough remains additive when shipped.)

Explicit row locks: (select-for-update 'type id) Exposed as a primitive for finer control. Persists across subsequent prims in the same with-transaction body.

Cache coherence (initial): NO caching of stored function defs in image memory. Each call reads the AST fresh from Postgres. For a personal OS where most calls are query+render, the function read alongside the query is acceptable overhead.

Cache coherence (later): LISTEN/NOTIFY-based invalidation. When a function is updated, the updating image notifies; others drop from cache. Mechanism well-trodden, deferred until measured.

> *[verify] test: image/test/primitives_test.ml – 48 alcotest cases; build: dune build @all; smoke: duplicate-fork UNIQUE violation triggers pgx auto-rollback; smoke: fork_branch returns new branch id (str round-trip); smoke: merge_branch returns merged-rows count; smoke: (select-for-update 'type id) takes a row lock visible to concurrent transactions*

with-transaction

Dynamic transaction scoping: (with-transaction (lambda () body))

[status: verified]

Dynamic transaction scope: all DB prims called inside the body run against the SAME pgx transactional connection. Defaults to SERIALIZABLE isolation.

(with-transaction (lambda () (seq (select-for-update 'type id) (update-field 'type id "field" "value"))))

The select-for-update row lock PERSISTS across the subsequent update-field in the same body – this is the v1 concurrency-protection story for compound mutations.

Nested transactions raise a clean Interp_error. The body value is returned verbatim (no wrapper tags/boxing).

The Lambda body is evaluated lazily; the dynamic binding is set on ‘Store.t.current_tx‘ before the body runs and cleared via ‘Lwt.finalize‘ after (whether BODY succeeds or raises – pgx still sees the propagated exception and rolls back).

> *[verify] test: image/test/primitives_test.ml – 48 alcotest cases; build: dune build @all; smoke: with-transaction body runs its prims on one transactional connection; row locks persist across prims*

media

Binary media blobs in Postgres: agent upload + authenticated serve

[status: implemented]

WHY A SEPARATE TABLE (not data_rows jsonb) ———————————— data_rows packs field documents as jsonb. Binary payloads do not belong there: base64-in-jsonb wastes ~33% space, blows GIN indexes, and fights the 64 KiB /api/eval body cap that protects the Nopales path. Media is first-class storage – same "Postgres is the object store" thesis, different physical shape.

SCHEMA (migration 007_media.sql) ———————————— CREATE TABLE media_blobs (id uuid PRIMARY KEY DEFAULT gen_random_uuid(), author text NOT NULL, content_type text NOT NULL, filename text, – optional original name byte_size int NOT NULL, sha256 text NOT NULL, – hex digest of bytes data bytea NOT NULL, meta jsonb NOT NULL DEFAULT '{}', created_at timestamptz NOT NULL DEFAULT now(), updated_at timestamptz NOT NULL DEFAULT now()); CREATE INDEX media_blobs_author_idx ON media_blobs (author); CREATE INDEX media_blobs_created_idx ON media_blobs (created_at DESC); CREATE INDEX media_blobs_sha256_idx ON media_blobs (sha256);

Shared across branches (same rule as data_rows): fork/merge copies code, not media. Ownership is the author column + session identity, not branch_id.

AGENT PUSH (primary write path) ————————————- Agents MUST NOT push multi-KiB binaries through POST /api/eval (max_user_body_bytes = 64 KiB). Dedicated authenticated endpoints:

POST /api/media Auth: HTTP Basic or session cookie (same as eval). Body modes (Content-Type decides): 1. Raw binary – request Content-Type is the media type (image/png, image/jpeg, ?). Optional header X-Filename: original name. 2. JSON – application/json { "content_type": "image/png" , "data_b64": "?" , "filename": "shot.png" // optional , "meta": {?} } // optional Body limit for this route: 5 MiB (not the 64 KiB eval cap). Reject oversize with 413. MIME whitelist (v1): image/png image/jpeg image/gif image/webp image/svg+xml application/pdf On success 201: { "ok": true , "id": "<uuid>" , "content_type": "?" , "byte_size": N , "sha256": "?" , "url": "/media/<uuid>" , "filename": "?" | null } Errors: 400 bad args / bad b64 / disallowed type, 401 unauth, 413 too large, 500 store fail.

GET /media/<uuid> Auth required (private by default – same gate as /data). Serves raw bytes with Content-Type from the row, Content-Length,

Cache-Control: private, optional Content-Disposition inline; filename= when filename is set. 404 if missing.

DELETE /api/media/<uuid> Auth required. Author-or-admin may delete. 200 {ok,id} or 404/403.

GET /api/media (optional list) Auth required. JSON array of meta rows for the caller (id, content_type, byte_size, sha256, filename, author, created_at) – NO data bytes. Query: ?limit=N (default 50, max 200).

Client helper: scripts/pp_media.py python3 scripts/pp_media.py put ./shot.png python3 scripts/pp_media.py get <id> -o out.png python3 scripts/pp_media.py list python3 scripts/pp_media.py delete <id> Same env as pp_eval (PP_URL / PP_USER / PP_PASS).

LISP PRIMITIVES (meta + small ops; not bulk upload) ———————————— Registered next to CRUD. Anonymous denied on writes.

(media-meta id) -> VStr json | VNil id, author, content_type, filename, byte_size, sha256, meta, created_at, url – never the bytes.

(media-list [limit]) -> VList of meta json strings

(media-delete id) -> VStr id | VNil Author-or-admin; others get a denial string.

(media-put content-type data-b64 [filename]) -> VStr new-id | error string Convenience for TINY payloads that already fit eval (icons, test fixtures). Same whitelist + 256 KiB ceiling on decoded size so eval cannot be abused as a bulk pipe. Real agent pushes use POST /api/media.

(media-url id) -> VStr "/media/<id>"

No (media-get-bytes) over eval in v1 – serve via HTTP.

IMPLEMENTATION NOTES ———————— - image/lib/media.ml: store helpers (put/get/meta/ list/delete) over pgx; Pgx.Value.of_binary / to_binary for bytea; Digestif SHA256 hex. - image/lib/http_media.ml: route handlers; own body reader with max_media_body_bytes = 5 MiB. - Wire in http.ml dispatch; keep /api/media behind the normal auth middleware (NOT hard-public). - session.ml register_media_prims alongside CRUD. - agent_discovery.ml: document POST /api/media and pp_media.py in agent.json / agent.txt rules. - scripts/dev.sh migration loop: add 007_media. - migrations.borg applied-by-image list: 007.

NON-GOALS (v1) ————— - Public unauthenticated media URLs (later: flag or signed token if product needs it) - Per-user byte quotas (multi-user.budgets memory track stays deferred; soft 5 MiB/request is enough) - Image transforms / thumbnails / EXIF strip - S3 or filesystem offload (bytea in PG is the v1 habitat; hoist later if measured hot) - Dedup-by-sha256 on put (index exists; policy later)

> *[verify] build: dune build @all; test: image/test/media_test.ml – put/meta/list/delete + mime reject; smoke: scripts/dev.sh applies 007; POST /api/media with basic auth stores png; GET /media/<id> returns image/png bytes; smoke: python3 scripts/pp_media.py put <fixture.png> prints id; get round-trips bytes; smoke: POST /api/eval still capped at 64 KiB; media route accepts up to 5 MiB*

13 borge/multi-user.borg

pricklypear-multi-user

Pricklypear book chapter: multi-user – MANY HUMANS. Cotenant isolation (this chapter) is the shipping model: one process, many sessions, isolation by session + branch. Process-per-user isolation (multi-user.isolation) is a future hardening axis. single-image.borg is about shipping ONE binary, not one human. Do not read single-image as cancelling multi-user. Access surfaces that ship on cotenant first: user subdomains, per-user LLM keys, SSH?REPL (auth.user-subdomains / user-llm-keys / user-ssh-repl).

[status: partial]

> – *agent note:* 2026-08-02 status reconcile: namespaces/budgets/chat/user-plan ? implemented; permissions stays partial (setuid author). isolation deprecated; cotenant-isolation already implemented.

multi-user

Cotenant isolation (one process, session + branch scoped), namespaces via branch ownership, budgets, author-runs-as-author

[status: partial]

cotenant-isolation

Session-level isolation in one process – THE v1 model

[status: implemented]

One image process serves many authenticated sessions. Isolation is session + branch scoped, not process scoped:

- Each session carries its own identity (user_id, username, is_admin) resolved at login and attached per request. - Each session has its own default branch (<username>/personal); defines, routes, and type declarations land on the session's current branch. - No session writes to another user's personal branch; overrides on a personal branch shadow system/main only for that branch's readers. - Session state is not shared; a wedged call aborts its own session's work, not another user's.

What cotenant does NOT provide (honest limits): - No per-user process boundary: a hard process crash takes all sessions down together (the watchdog restarts the one image for everyone). - Resource contention is shared: one user's expensive eval competes with everyone else's (mitigated by per-session step budgets – see budgets). Per-user image processes (multi-user.isolation) remain the future hardening axis for a stronger blast radius.

isolation

Future hardening axis: process-level isolation, one image per user

[status: deprecated]

DESIGN SKETCH (not the shipping topology).

The retired fleet design gave each user their OWN image process rather than namespacing within one shared image – the Unix/POSIX model: each user has a home (their branch bindings); logging in boots or attaches their image; each image is a separate process.

Wins it would buy: a user's infinite loop or bad define wedges only their image; per-user A/B rollback; no reliance on cooperative scheduling for cross-user safety.

Why it is not v1: it needs a supervisor/fleet layer (boot-on-connect, attach, per-user RAM), which the single-image pivot retired. Cotenant isolation (above) is the shipping model; this remains the future hardening axis if a stronger blast radius is ever needed.

namespaces

Branch ownership namespaces; packages are a separate in-branch axis
[status: implemented]

TWO AXES (do not conflate):

1. BRANCH namespace (this subsection): There is no privileged 'common' namespace. There is 'main' (system baseline) and per-user branches. Versioning/ownership IS branch ownership. A binding's key is (branch_id, name). When Alice forks main to alice/main, she gets her own 'greet' independent of main's greet (until she merges).

2. PACKAGE prefix (image.packages): Within ONE branch, organize functions with '/' in the name (todo/list). That is organization, not ownership. See image.packages.

Cross-branch calls (still deferred): a function can explicitly reference another branch's binding via a qualified name (e.g. system/main:greet – note colon or other separator TBD, must not collide with package '/'). Normally a function calls bindings in its own branch.

The 'common namespace problem' is thus reframed as the 'merge to main politics problem': - Default: admin-only merges to main - All users can fork freely - Users can merge into their own branches freely - Politics expand later (PR/review workflow)

This sidesteps the up-or-down political decision the original 'publish to common' model forced.

> – *agent note:* 2026-08-02: branch ownership model live (system/main + user/*); package / prefix separate. Cross-branch qualified calls still deferred as documented.

budgets

Per-call step budgets to bound runaway user code
[status: implemented]

> – *agent note:* 2026-08-02: Eval.make_budget + last_budget; session eval wraps budget; conformance tests. Memory budget deferred.

> *[verify] build: dune build @all; smoke: a Nopales call that exceeds its step budget aborts cleanly; the session survives*

Each Nopales primitive invocation runs with a step counter. When exceeded, that ONE call aborts; the session continues. This prevents "one user's bad function wedges the whole image" at the interpreter level (defense in depth alongside process-level isolation).

Default budget: configurable per session, sensible initial value (e.g. 100k eval steps). User can request more for legitimate long-running operations via a primitive that explicitly raises the budget for a sub-call (and logs it).

The interpreter checks the counter at every Prim invocation and every recursive eval depth. Cheap: one integer increment per step.

Memory: also tracked. A session exceeding memory budget aborts the offending call. (Deferred; step budget is the v1 mechanism.)

This is per-CALL, not per-session: a session can make many calls, each independently bounded.

permissions

Author-runs-as-author: functions execute with their author's data perms
[status: partial]

A function executes with ITS AUTHOR's data permissions, not the caller's. This is the Unix setuid model: if alice authors a function that reads her calendar, and bob calls it, the function reads alice's calendar (not bob's), and bob sees only what alice's function returns.

Implications: - The interpreter's eval carries a (current-author) field, read from the EXECUTED binding's author, not the session user. - Data primitives (query, get, create, etc.) consult current-author for permission filtering. - A careless privileged function is a security hole, same as Unix setuid. Honest and familiar. - Functions can expose capabilities to other users by name without exposing raw rows.

Authoring permissions (separate concern): - A binding in alice/main is callable by anyone in the same image (read access implicit). - Only alice can edit bindings in alice/* branches. - main is owned by admin; merges require admin approval (default).

This model is the Unix model, by choice. It maps cleanly to "SSH-first, multi-user OS" framing. If we wanted a Lisp-machine feel (shared trusted environment), we'd pick differently, but SSH-first implies Unix-style multi-user.

SOCIAL LAYER (chat + plan) uses SHARED data_rows (existing storage rule) with handler-enforced filters by (whoami). That is intentional: messages and plans are multi-user objects, not personal-branch pins. Branch gardens stay code/UI; social state is rows. See multi-user.chat and multi-user.user-plan.

> – *agent note:* 2026-08-02 confirm partial: current-author still aliases whoami; full setuid author-on-execute not shipped.

chat

Multi-user chat: site lobby + DMs so humans on the box can talk
[status: implemented]

> – *agent note:* 2026-08-02: libs/chat ~552 lines lobby+DM; live GET /chat 200.

> *[verify] build: dune build image; smoke: alice and bob post to /chat lobby; each sees both messages; smoke: alice DM to bob visible to both; not to carol; smoke: unauth /chat redirects login; public face does not leak messages; smoke: (chat/say "hi") and (chat/inbox) work via /api/eval*

==========
0. WHY ==========

Multi-user is not only isolation – people on the same habitat need a way to talk. Chat is a first-class product surface (like notes/todo), not an external app. Agents authenticated as users may post too (same identity rules).

==========
1. SHAPE (v1) ==========

Two conversation kinds:

A. LOBBY (site-wide room) room_id = "lobby" (singleton) All authenticated users can read/write. Default /chat UI.

B. DM (direct message thread) room_id = canonical pair key: dm:alice:bob where alice < bob lexicographically Participants = the two usernames only. List/create from /chat/dm or (chat/dm "bob").

Out of scope v1: - Arbitrary named rooms / channels (phase 2) - WebSockets / SSE push (htmx poll is enough) - File attachments, reactions, threads, edit history - End-to-end encryption - Cross-deployment federation

==========
2. DATA (shared data_rows) ==========

Package libs/chat seeds:

(deftype 'chat-room 'kind ; "lobby" | "dm" 'room_id ; "lobby" | "dm:a:b" 'title ; display 'members ; JSON array of usernames; lobby = [] 'created_by 'created_at) ; ISO string from Nopales or server

(deftype 'chat-message 'room_id 'from ; username (whoami at post) 'body ; plain text (escape on render) 'created_at)

data_rows are SHARED across branches (storage rule). Access control is in chat/* handlers: - post: from MUST equal (whoami); reject spoof - list lobby: any authenticated user - list dm: whoami is member - create dm room: both users must exist

Retention v1: keep all messages (personal OS scale). Optional later: max list limit + archive.

Seed: ensure lobby room row exists on load-library (idempotent create-if-missing).

==========

3. LISP API (libs/chat) ==========

(chat/say body) ; post to lobby as whoami (chat/say-in room_id body) ; post to room if allowed (chat/lobby [limit]) ; recent lobby messages (chat/history room_id [limit]) (chat/dm peer) ; ensure dm room; return room_id (chat/dms) ; rooms where whoami is member (chat/users) ; list usernames (for DM picker) ; needs thin OCaml prim ; (list-usernames) for any authed

New OCaml prim if missing (image): (list-usernames) ? VList of VStr authenticated only; usernames are public identifiers on the box (no email/admin leak).

==========

4. HTTP / HTMX UI ==========

Routes (private by default – auth required):

GET /chat lobby page GET /chat/partial message list fragment (poll) POST /chat/say body=text ? append + refresh GET /chat/dm list DM threads GET /chat/dm/:peer DM thread with peer POST /chat/dm/:peer send DM

Polling: hx-get /chat/partial hx-trigger every 3s-5s. No WebSocket in v1.

Chrome: ui/nav link Chat when package loaded. Render: escape body text; show from + time + body. Empty state: Say hi to the box.

==========

5. SECURITY / POLICY ==========

- No anonymous chat (kernel private routes) - Body max length (e.g. 4k chars) enforced in chat/say - Rate: soft – eval budget enough for v1 - XSS: escape user body on render; never raw untrusted HTML - Spoof: ignore client-supplied from; always whoami

==========

6. PHASES ==========

Phase 1: deftype + lobby say/history + /chat UI poll Phase 2: DMs + list-usernames + /chat/dm Phase 3: nav seed, optional presence / unread

Success: two browser sessions (alice, bob) talk in lobby without leaving the habitat; agent as alice can (chat/say "deployed") visible to bob.

user-plan

Unix-style .plan: short per-user status of what they intend to work on

[status: implemented]

> – *agent note:* 2026-08-02: libs/plan user-plan + plan/set!/of; live GET /plan 200.
>
> *[verify] build: dune build image; smoke: (plan "macros + chat") sets alice plan; (plan-of "alice") returns it; smoke: GET /u/alice shows plan text; bob can read alice plan; smoke: only owner can set own plan; bob cannot set alice; smoke: empty plan shows friendly placeholder*

==========

0. WHY (finger / .plan) ==========

Classic Unix: each user has a ~/.plan file; finger(1) shows it. On a multi-user Lisp OS this is the lightest coordination tool – not a task tracker (todo exists), not chat, just what I'm working on / intend next.

Complements chat: plan is sticky status; chat is conversation.

==========

1. DATA ==========

(deftype 'user-plan 'username ; owner; unique logical key 'body ; free text (plain) 'updated_at) ; ISO string

One logical plan per username. Implementation: find existing row where username matches whoami; update body + updated_at, or create if missing. Shared data_rows; filter by username field.

Optional later: project / until date – not v1.

==========

2. LISP API (libs/plan) ==========

(plan body) ; set MY plan (whoami); return body (plan) ; get MY plan body or empty string (plan-of user) ; get that user's plan body or empty (plan-clear) ; set my body to empty (plan-list) ; list of (username . body) for ; users with non-empty plans

Spoof: (plan) always writes username=(whoami). There is no (plan-as other body).

==========

3. HTTP UI ==========

Private (auth required) unless noted:

GET /plan edit-or-view MY plan form POST /plan save body GET /plans directory: who has a plan set GET /u/:username profile stub: username + plan body + updated_at + link to garden subdomain if auth.user-subdomains on (finger alice)

Public option (phase 2, with route-visibility): public /u/:user so plans can be world-readable like classic finger. v1: authenticated users on the box only.

Chrome: show MY plan one-liner under whoami in header or on home (ui/home snippet) – optional phase 2. Nav: Plan ? /plan; People ? /plans.

==========

4. UX COPY ==========

Placeholder when empty: No plan yet. What are you working on?

Form label: Your plan (like a Unix .plan file)

Help text: Short note other users on this box can read. Examples: shipping macros, out until Thursday.

==========

5. PHASES ==========

Phase 1: deftype + plan/plan-of/plan-list + /plan + /u/:user Phase 2: /plans directory + home/chrome snippet Phase 3: optional public /u/:user; subdomain profile page

Success: alice sets plan macros + multi-user chat; bob opens /u/alice and knows what she's doing; chat stays for the actual conversation.

14 borge/auth.borg

pricklypear-auth

Pricklypear book chapter: auth
[status: partial]

– *agent note:* 2026-08-02 status reconcile: multi-user-product, user-subdomains, route-visibility ? implemented. Still partial: verification (argon2), user-ssh-repl (credential yes, full SSH door no).

auth

Multi-user identity: credentials, sessions, login, and binding each human to their garden
[status: partial]
Auth is in-process in the single image (image/lib/auth_password.ml, image/lib/auth_store.ml). It provides password verify, session mint/verify, SSH-key verify, and user CRUD; topology per single-image.borg.

credentials-store

The users, credentials, sessions, auth_log tables in Postgres
[status: implemented]
Tables (added by migrations, see migrations):
users (id uuid PK, username text UNIQUE, is_admin bool, user_type text NOT NULL DEFAULT 'human', – 'human' | 'agent' parent_user_id uuid NULL REFERENCES users(id), – agent's parent human; NULL for humans allow_agent_promote bool NOT NULL DEFAULT true, – human's toggle: may their agent promote – into their personal branch created_at timestamptz)
credentials (id uuid PK, user_id uuid FK, kind text, – 'ssh_key'|'password'|'web_token'|'api_key'|'llm_key' public_data jsonb, – SSH key material / llm last4+meta; – null for password (hash is secret) secret_data jsonb, – argon2 hash for password; – api_key token hash/reference; – llm_key provider secret (never list) created_at timestamptz, last_used_at timestamptz)
sessions (id uuid PK, – the token's opaque id user_id uuid FK, token_hash text, – SHA-256 of token; lookup target expires_at timestamptz, created_at timestamptz, revoked_at timestamptz NULL)
auth_log (id uuid PK, user_id text NULL, kind text, success bool, source_ip text, happened_at timestamptz)
KEY STORAGE (in-Postgres, role-isolated): - Other credentials (password hashes via argon2id, SSH public keys) in Postgres permanently; that's correct and stable (these are public verification material, not signing keys).

verification

verify(cred) / lookup / mint / verify_token – the four pure functions
[status: partial]
SSH KEY VERIFICATION: 1. Call verify_ssh_key(presented_key) -> May<user_id> 2. Auth: parse key, SELECT FROM credentials WHERE kind='ssh_key' AND public_data = key. Return user_id or denied. 3. Auth logs to auth_log.
PASSWORD VERIFICATION (for web login): 1. User POSTs username+password to /login. 2. Auth: SELECT secret_data FROM credentials WHERE kind='password' AND user_id= (SELECT id FROM users WHERE username=$1). Verify against the stored password hash. Champion is argon2id PHC-encoded; legacy sha256$salt$digest rows STILL verify and a successful legacy verify triggers a transparent in-place rewrite to argon2id (no flag day). Return user_id or denied. 3. Auth logs to auth_log.

TOKEN MINTING (after web login): 1. Auth: generate random token bytes (32B). 2. Auth: INSERT INTO auth_sessions (user_id, token_hash=sha256(token), expires_at=...). 3. Auth: return the raw token. Client stores it as the pp_session cookie (HttpOnly).

TOKEN VERIFICATION (on web requests): 1. Client sends token cookie. The image calls Auth_store.verify_token(token). 2. Auth: sha256 the raw token, look up the auth_sessions row by token_hash; check not revoked and not expired. Return user_id or denied. 3. Auth logs to auth_log.

API TOKEN VERIFICATION (for agents / pp_eval Bearer): 1. Client sends Authorization: Bearer <opaque_token>. 2. Auth: SHA-256 the token, SELECT FROM credentials WHERE kind='api_key' AND secret_data->>'token_hash' = $1. Return user_id or denied. 3. Auth logs to auth_log; updates last_used_at. 4. Tokens are per-user credential rows. No global tokens; every token belongs to a user_id. Generate/ rotate/revoke via /admin/users or a Nopales prim. The raw token is returned once at creation.

TOKEN FORMAT (minimal, NOT JWT): opaque_id (32B random) || HMAC(opaque_id) We control both ends; don't need JWT's cross-system interop claims. Hash stored in auth_sessions table enables revocation.

> – *agent note:* 2026-12: password champion is now argon2id with a transparent upgrade on successful legacy verify (auth_store.verify_password -> set_password). Legacy sha256$salt$digest rows still verify and are rewritten in place on first post-cutover login – no flag day, no dual-entry. BORGE-1785545259 closed (supersedes lost-id 1783831698 refs). Token/ssh_key paths unchanged.

bootstrap-admin

First-admin creation via env var on auth first boot

[status: implemented]

Bootstrapping the first user, with no users yet:

1. At auth boot, if the users table is empty, check env vars PP_BOOTSTRAP_USER and PP_BOOTSTRAP_PASSWORD (default: admin/admin). 2. If set, INSERT into users (username=$PP_BOOTSTRAP_USER, is_admin=true) and into credentials (kind='password', public_data=null, secret_data=argon2 hash of $PP_BOOTSTRAP_PASSWORD, user_id=the new user). 3. Auth boot: warn loudly in logs that bootstrap was used, and recommend clearing the env vars afterward. 4. After first boot, the env vars are ignored.

Subsequent users: added by an admin via the admin UI (which authenticates via auth) calling an auth primitive (create-user username, add-credential user_id ssh_key, etc.). These primitives require is_admin=true on the calling session.

No interactive bootstrap flow needed; the env-var pattern is simple, standard, and audit-logged.

audit-log

Every verification logged to auth_log

[status: implemented]

Every call to verify_ssh_key, password verification, mint, and verify_token writes a row to auth_log: (user_id NULL if unknown/failed, kind: 'ssh_key'|'password'|'mint'|'verify_token', success bool, source_ip text, happened_at timestamptz default now())

WHY from day one (despite personal-OS scope): - Cheap to add (one INSERT per verify call). - Invaluable when something goes wrong (compromised key? malicious insider? audit compliance if ever needed?). - Painful to retrofit (verification call sites scattered; adding log calls everywhere later means finding every one). - Standard practice; non-controversial.

The log is append-only in practice (no UPDATE/DELETE on auth_log). Retention policy deferred; for now, keep forever (cheap rows).

rpc-from-supervisor

Deprecated: auth is in-process; no RPC boundary
[status: deprecated]
Retired by the single-image pivot: auth is in-process. The supervisor no longer calls auth over RPC.

multi-user-product

COMPLETE multi-user product: many humans, each with identity, session, garden, authorship
[status: implemented]

> – *agent note:* 2026-08-02: users+sessions+personal branches+admin promote gate+logout revoke shipped (web-login/identity-and-branch/user-management). Cotenant isolation v1; per-user process fleet still future.
>
> *[verify] smoke: two users login independently; each sees own personal pins; smoke: Alice define does not create own pin on Bob's branch; smoke: non-admin cannot merge to system/main; smoke: agent install on Alice session writes alice/personal only; smoke: logout revokes session; next request requires login*

==========
MULTI-USER IS THE SYSTEM ==========
Pricklypear is a multi-user Lisp OS. Multiple humans share one deployment. Each human has: - a user row + credentials - a browser (and/or API) session - a personal branch garden: <username>/personal - authorship on their defines (function_defs.author) - isolation from other users' gardens and (when fleet is live) from other users' processes

single-image.borg simplifies HOW WE SHIP THE BINARY (one process to build/restart). It does NOT mean one human. multi-user.cotenant-isolation (one process, session + branch scoped) is the v1 isolation model; multi-user.isolation (per-user image process) remains a future hardening axis, not the shipping topology.

==========
PERSONAS ==========
Operator/admin – first user; can create users, merge to system/main, manage credentials, revoke sessions. Human user – logs in, works on <user>/personal, inherits system/main defaults, cannot admin-merge to system. Agent (as user) – authenticated as a human or service account; installs Nopales on that identity's branch; branch-locked for the task duration. Anonymous – only public routes (/login, health as policy).

==========
USER STORIES (complete set) ==========
US-BOOTSTRAP – First admin Empty users table ? bootstrap admin from env (PP_BOOTSTRAP_USER / PP_BOOTSTRAP_PASSWORD) OR one-time /setup form that creates admin then disables itself. is_admin=true. Site thereafter requires auth.

US-LOGIN – Browser sign-in GET /login ? username + password form. POST /login ? verify ? mint session ? Set-Cookie pp_session (HttpOnly; Secure; SameSite=Lax; Path=/). Redirect /. Bad credentials ? generic error + auth_log.

US-LAND – Land in my computer After login (or valid cookie): 1. Resolve user_id, username, is_admin from session. 2. Ensure branch <username>/personal exists as EMPTY child of system/main (inheritance, not full copy). 3. Image runtime identity = that user. 4. Checkout <username>/personal if not already. 5. (whoami) / chrome meta / define author = username. Alice never receives Bob's personal as current branch.

US-LOGOUT – Sign out POST /logout ? revoke auth_sessions row ? clear cookie ? /login.

US-SESSION – Stay signed in Every non-public request: load session by cookie ? valid unexpired unrevoked ? attach identity; else redirect /login (HTML) or 401 (API).

US-INVITE – Admin creates user Admin UI/API: create_user(username, is_admin?), set password credential, optional temp password. New user can US-LOGIN and gets own personal garden.

US-AGENT – Agent or laptop as a user Authenticate via same session cookie (cookie jar) OR API key credential (Authorization: Bearer ?) bound to a user_id. install_function / define write that user's current branch. Agent branch lock forbids checkout/ fork/merge mid-task. Author may be agent:<model> with provenance; capability still runs as the user.

US-SUBDOMAIN – Personal host name Each user has a stable URL on a wildcard base domain: https://{username}.{PP_BASE_DOMAIN}/ After login Alice lands on her subdomain (or is redirected there). Private work and public garden pages resolve against her branch chain. See auth.user-subdomains.

US-LLM-KEY – Bring your own model key Each user may store an LLM provider key (credentials kind llm_key). AI prims (propose-function, llm-*) use the session user's key; operator env LLM_API_KEY is fallback only. See auth.user-llm-keys.

US-SSH-REPL – Key-based shell into my image User registers an SSH public key (credentials kind ssh_key). Real SSH (or approved OpenBSD ForceCommand bridge) authenticates the key, attaches identity, opens the Nopales REPL on <user>/personal. See auth.user-ssh-repl.

US-CHAT – Talk to other humans on the box Authenticated users share a lobby and DMs (multi-user.chat). (chat/say ?) / /chat. Not email; same-deployment only.

US-PLAN – Unix .plan status Each user sets a short sticky plan (multi-user.user-plan). (plan "?") / /plan / /u/:user (finger). Complements chat.

US-COTENANT – Logical multi-user in one process (THE v1 model): One image process serves many sessions; identity is per-request; personal branch is per-session; no cross- user write to another's personal. Correct namespacing with a shared-process blast radius.

US-ISOLATION – Process isolation (future hardening axis) Logging in boots or attaches that user's image process (multi-user.isolation). Alice's wedge does not take Bob offline. Auth never sees fleet; fleet never sees raw passwords – only user_id from verify. Not the shipping topology in v1.

US-PERMS – Authorization - define/route/deftype ? current branch only - merge to system/* ? admin only - merge to own owner/* ? free for that owner - admin user CRUD ? admin session only - data_rows shared (existing rule) unless future row-level owner column is added

US-AUDIT – Failures are boring and logged auth_log for login success/fail, logout, admin acts. No username enumeration in errors.

==========

END-TO-END STORY ==========

1. Deploy; bootstrap admin. 2. Admin opens https://{PP_BASE_DOMAIN}/ ? /login ? signs in ? redirect https://admin.{PP_BASE_DOMAIN}/ on admin/personal. 3. Admin creates user "alice" + password. 4. Alice signs in ? https://alice.{PP_BASE_DOMAIN}/ on alice/personal (empty garden), inherits main. 5. Alice pastes her LLM key in /settings/keys; AI tools bill her provider, not the operator's env key. 6. Alice registers ~/.ssh/id_ed25519.pub; later 'ssh alice@host -p PP_SSH_PORT' opens her REPL. 7. Alice edits ui/menu ? pin on alice/personal only. 8. Bob signs in on bob.* ? bob/personal; no Alice pins. 9. Agent with Alice session/API key installs ? alice/personal. 10. Alice logout; cookie dead on her host. 11. With fleet (later): Alice's process is hers; Bob's is his. Cotenant single-image remains valid for v1 of subdomain

+ keys (identity per request / SSH session).

==========

OBJECTS (complete) ==========

users(id, username UNIQUE, is_admin, created_at) credentials(id, user_id, kind, public_data, secret_data, created_at, last_used_at) kind: password | api_key | ssh_key | llm_key | ? ssh_key.public_data: { "openssh": "ssh-ed25519 AAAA? comment" } llm_key.secret_data: { "api_key": "?" } (* never echo back *) llm_key.public_data: { "last4": "?", "base"?: url, "model"?: str } auth_sessions(id, user_id, token_hash, expires_at, created_at, revoked_at, user_agent, source_ip) auth_log(id, user_id NULL, kind, success, source_ip, happened_at)

Runtime image session (per connection / request): user_id, username, is_admin store.branch_id ? <username>/personal (default) author for define = username | agent:<model>

Password hashing: argon2id (implemented 2026-12: auth_password.ml champion matches Argon2 ID at t_cost=2, m_cost=65536, parallelism=1); interim sha256$salt$digest rows verify transparently and are rewritten to argon2id on successful login in-place.

Session token: opaque random id; store only HMAC/hash in DB; cookie carries raw id. Revocable. Expiry required.

==========

HTTP SURFACE (complete) ==========

Public: GET /login POST /login GET /health (policy: public or authed – choose public for ops probes)

Authenticated (any user): POST /logout all product routes (/, /todos, /notes, /api/eval, ?) identity-dependent branch + author

Admin: /admin/users (list, create, set password, revoke sessions) merge policy to system/* already enforced in Nopales prims

API: Cookie session OR Authorization: Bearer <api_key> same identity attachment as browser

nginx: TLS termination; reverse proxy to image(s). Complete system does NOT rely on htpasswd as the user database. htpasswd may exist only as emergency break-glass.

==========

FLEET ATTACH (future design sketch – per-user processes) ==========

v1 REALITY: identity attaches in-process at login; one image serves all sessions (see single-image.auth).

In a future per-user-process topology, on successful login / SSH verify an attach layer would resolve user_id ? boot-or-attach a per-user image with env: PP_USER_ID, PP_USERNAME, PP_BRANCH_ID, PP_BRANCH_NAME and proxy HTTP/REPL to that instance. Auth's component verifies + mints only; it never boots images.

==========

RELATION TO BRANCHES (already built) ==========

system/main – package defaults (seed from libs/*.pp) <user>/personal – empty-child garden; pins are "own" Effective Functions list = deduped closest-wins Agent locked to current branch during agent-task

Auth does not replace branching; it answers WHO is on which garden.

==========

NON-GOALS ==========

- OAuth / OIDC / social login - Email magic links (unless added later as a credential kind) - Sharing one password (htpasswd) among humans as the permanent multi-user model - Per-user process isolation as a blocker for subdomain/ llm_key/ssh_key (those ship on cotenant single-image first) - Custom domains per user (only {user}.{PP_BASE_DOMAIN} v1)

user-subdomains

Per-user Host: {username}.{PP_BASE_DOMAIN} as personal garden URL

[status: implemented]

– *agent note:* 2026-08-02: classify_host Host_garden + PP_BASE_DOMAIN in http_core; http.ml attach path uses host_mode; garden_url helper. Off when env unset.

[verify] build: dune build image; smoke: Host: alice.example ? garden context username=alice; smoke: logged-in alice on alice.example sees alice/personal; smoke: logged-in alice on bob.example cannot write bob garden; smoke: apex base domain serves welcome/login; post-login redirects to user host

Subdomain feature: when PP_BASE_DOMAIN is set, requests to {username}.{PP_BASE_DOMAIN} resolve to the user's personal branch garden (<username>/personal). The image parses the Host header, validates the username against the users table, and attaches that identity; otherwise, legacy single-host mode (path-only) applies.

user-llm-keys

Per-user LLM provider keys; session AI uses user key with env fallback
[status: implemented]

[verify] build: dune build image; smoke: user sets llm_key; propose-function uses it not empty env; smoke: user without key falls back to LLM_API_KEY env if set; smoke: GET settings never returns raw api_key; last4 only; smoke: Bob cannot read Alice llm_key credential

==========
1. WHY ==========
Today llm_client reads process env LLM_API_KEY only. Multi-user means Alice's AI spend and provider choice cannot be the operator's single key. Each human brings (or is issued) their own key; the image selects by session identity.
==========
2. CREDENTIAL SHAPE ==========
kind = 'llm_key'
secret_data jsonb (NEVER returned on list/get APIs): { "api_key": "<provider secret>" }
public_data jsonb (safe to show): { "last4": "ab12" // last 4 of key , "label": "synthetic" // optional , "base": "https://?" // optional override of LLM_API_BASE , "model": "hf:?" // optional override of LLM_MODEL }
Policy v1: at most ONE active llm_key per user (replace on set, like password). Multiple providers later via label/id.
Storage: Postgres credentials row; same role as other secrets. No encryption-at-rest beyond PG in v1 (same acceptance as password hashes / session signing risk profile – personal/small-team). Document footgun.
NOT the same as kind=api_key (that authenticates INTO pricklypear as Bearer). llm_key authenticates OUT to an LLM provider.
==========
3. RESOLUTION ORDER (llm_client) ==========
When a session invokes AI (propose-function, llm-inner, llm-outer, llm-rewrite, llm-seed, agent portal, ?):
1. If session has user_id AND credentials llm_key present ? use that api_key; apply public_data base/model overrides if set. 2. Else if env LLM_API_KEY non-empty ? operator fallback (dev). 3. Else ? clear error "no LLM key configured (add one in /settings/keys or set LLM_API_KEY)".
Threading: llm_client functions take optional ~auth_ctx / ~api_key / ~base / ~model rather than only reading env. Session layer fills from credential lookup (cached per request OK; invalidate on set/delete).

Anonymous / no session: AI prims denied (hard-private routes already); no env key leak via public pages.

==========

4. HTTP + LISP SURFACE ==========

Browser (authenticated user, self-service): GET /settings/keys form: status last4, set, clear POST /settings/keys body: api_key, optional base, model POST /settings/keys/clear

Admin may set/clear for another user via /admin/users (optional v1; self-service is enough for dogfood).

Nopales prims (session user only unless admin): (set-llm-key key [base] [model]) ? "ok" / last4 (clear-llm-key) ? VNil (llm-key-status) ? dict/list: configured?, last4, base?, model? NEVER returns raw secret.

auth_store: set_llm_key user_id ~key ?base ?model clear_llm_key user_id get_llm_key_meta user_id ? public_data option get_llm_key_secret user_id ? string option (* only called from server-side AI path *)

==========

5. SECURITY ==========

- Raw key only in POST body / Nopales arg / secret_data - Logs: never print full key (llm_client already prints model/url – keep it that way) - CSRF: same cookie session rules as other POSTs - User A cannot get_llm_key_secret for user B - Export/pp_sync: do NOT dump secret_data into lib.pp

==========

6. PHASES ==========

Phase 1: auth_store llm_key CRUD + llm_client ~api_key threading + session resolution + (llm-key-status) Phase 2: /settings/keys UI + set/clear prims Phase 3: optional per-user base/model; admin override UI

Success: two users, two keys, simultaneous propose- function calls hit distinct Authorization headers (observable via mock/test double or request log redacted).

user-ssh-repl

Per-user SSH public keys open an identity-bound Nopales REPL

[status: partial]

> – *agent note:* 2026-08-02 confirm partial: ssh_key credentials + verify_ssh_key + /settings/ssh UI shipped; full sshd/awa REPL attach still bridge/docs (docs/ssh-repl-bridge.md), not closed product door.
>
> *[verify] build: dune build image; smoke: add ssh_key credential for alice; verify_ssh_key resolves alice; smoke: SSH or bridge session attaches whoami=alice branch=alice/personal; smoke: unknown key denied + auth_log; smoke: password-only user without ssh_key cannot SSH*

==========

1. WHY ==========

Multi-user OS framing wants a real shell path: ssh alice@pricklypear.host -p $PP_SSH_PORT ? Nopales REPL as alice on alice/personal.

credentials.kind=ssh_key is already in the schema and historical supervisor/awa design. Single-image retired the supervisor fleet, so SSH must attach identity in-process (or via OpenBSD sshd ForceCommand bridge).

Browser login remains primary for v1 web; SSH is the power-user / agent-adjacent REPL door.

==========

2. CREDENTIAL SHAPE ==========

kind = 'ssh_key' public_data jsonb: { "openssh": "ssh-ed25519 AAAA?[comment]" , "fingerprint": "SHA256:?" // computed on insert , "label": "laptop" // optional } secret_data: null (public keys only; no private keys)

Multiple ssh_key rows per user allowed (laptop, desktop). Admin or self-service add/revoke.

auth_store: add_ssh_key user_id ~openssh_pubkey ?label ? parse type (ed25519 required v1; rsa deferred) ? store normalized openssh string + fingerprint list_ssh_keys user_id ? public metas revoke_ssh_key credential_id verify_ssh_key presented_openssh_or_blob ? user option (* compare fingerprint or canonical blob *)

auth_log kind='ssh_key' success true/false.

3. TRANSPORT OPTIONS (pick primary + ops escape)

Option A – awa-ssh IN THE IMAGE (product primary) - Image listens on PP_SSH_PORT when set (opt-in; unset = no SSH listener – same spirit as old PP_AWA_SSH_PORT). - Host key: seed in system_keys id 'image_ssh_hostkey_seed' (or reuse naming from historical supervisor_ssh_hostkey_seed). - Pubkey auth against credentials ssh_key for the claimed username (SSH user name = users.username). - On success: create/attach in-process session identity (user_id, username, branch personal), bridge channel to existing REPL endpoint semantics (line editor / eval loop). - CRITICAL: authenticated username MUST propagate into the REPL session (historical awa 0.3.1 gap where exec_callback lacked username – use awa version on poohstack that exposes auth identity, or map channel after auth event). No anonymous REPL.

Option B – OpenBSD sshd ForceCommand (ops escape) - System sshd on a dedicated port/user - authorized_keys generated/synced from credentials (periodic script or AuthorizedKeysCommand) - ForceCommand runs pp_repl_bridge that speaks to image REPL with fixed identity from key mapping - Document in openbsd-prod-branch-deploy.md - Acceptable if awa-in-image slips; product stories still require verify_ssh_key + key management UI.

v1 make should implement credential CRUD + verify + ONE working door (A preferred). B is documented backup.

Line-protocol pubkey pipe (nc + raw key) is DEV ONLY; not the multi-user product surface.

4. SESSION ATTACH AFTER SSH AUTH

Same as US-LAND / identity-and-branch: ensure <username>/personal checkout personal whoami = username author = username budgets from multi-user.budgets

REPL eval uses that session's env/prims – same as browser /api/eval for that user. No shared global env across SSH users on cotenant: each SSH connection has its own Session.t (or equivalent).

Disconnect: drop session memory; do not revoke browser cookies. Optional: no auth_sessions row for SSH (ephemeral connection identity) OR mint a short-lived session for audit – v1 ephemeral is fine; auth_log the login.

5. HTTP / UI

GET/POST /settings/ssh – list fingerprints, add pubkey, revoke. Self-service. Admin /admin/users – add/revoke ssh keys for a user.

Nopales: (add-ssh-key "ssh-ed25519 AAAA?") ? fingerprint (list-ssh-keys) ? list of {fingerprint,label} (revoke-ssh-key fingerprint-or-id)

6. RELATION TO DEPRECATED PATHS

image.ssh-interface and supervisor.* remain deprecated as multi-process topology. This subsection is the single-image successor: keys in auth credentials, listener in image (or sshd bridge), identity in-process.

Do not revive authd RPC for verify_ssh_key in v1; call auth_store directly inside the image.

7. PHASES

Phase 1: auth_store ssh_key add/list/revoke/verify + /settings/ssh + Nopales prims + auth_log Phase 2: awa-ssh (or sshd bridge) attach ? REPL with real whoami; PP_SSH_PORT opt-in Phase 3: harden hostkey rotation docs; agent use of SSH as alternative to HTTP Basic

Success narrative: Admin creates alice; alice pastes pubkey; alice runs ssh alice@host -p $PP_SSH_PORT; REPL prints whoami => alice; (define ?) pins alice/personal.

login-experience

What log in means: I present credentials and land in MY live image

[status: implemented]

> *[verify] smoke: human authenticates and lands on username/personal; smoke: unknown credential denied; no image attach for stranger; smoke: two concurrent users keep distinct identities*

Product bar (from multi-user-product): 1. Present credential (password browser, API key, or SSH key when SSH front door is live). 2. Verify ? user_id (never proceed on failure). 3. Mint or resume session; attach identity. 4. Ensure <username>/personal; set runtime branch. 5. With fleet: boot/attach that user's image. 6. User works; logout/revoke ends session.

Success: "I logged into my computer as me." Failure modes never leak other users' existence beyond generic invalid-credentials.

web-login

Browser password login, session cookie, middleware, logout

[status: verified]

> *[verify] build: dune build; smoke: POST /login valid -> Set-Cookie -> GET / as that user; smoke: POST /login invalid -> 401 + auth_log fail; smoke: POST /logout revokes; subsequent GET / redirects login; smoke: Alice and Bob cookies do not cross personal branches*

Complete browser path (part of multi-user-product):

GET /login Public form: username, password. No product chrome that assumes identity.

POST /login Body: username, password (+ CSRF token when forms are cookie-authenticated for other posts). verify_password against credentials.kind=password. On success: insert auth_sessions; Set-Cookie pp_session=<opaque>; redirect /. On failure: auth_log; same error for bad user/pass.

Middleware (request gate – see auth.route-visibility) Always ATTEMPT identity resolve (cookie then Basic). Classify path (system public / system private / user route visibility). Private + no identity ? login redirect (HTML) or 401 (API). Public may proceed anonymously. Identity present ? attach user even on public paths (optional auth).

POST /logout Revoke session; clear cookie; redirect /login.

Password: argon2id verify/hash in complete system. Cookie flags: HttpOnly; Secure; SameSite=Lax; Path=/.

nginx: TLS + reverse_proxy only for this path. htpasswd is not the user database.

HARD PUBLIC (kernel, not user-route table): /login /health /logout /welcome /book.pdf /src.tgz /code /code/* /assets /assets/* /.well-known/agent.json /agent.txt (public release face – product-experience.public-release; /book HTML explainer is private, not hard-public; /agent private portal stays hard-private) User-route public/private: auth.route-visibility.

route-visibility

Public vs private HTTP routes: store-backed visibility + OCaml request gate
[status: implemented]

– *agent note:* 2026-08-02: migration 006 visibility; public-route + route-visibility prims; live (route-visibility "/welcome") => public; /pages visibility column.

[verify] build: dune build image; smoke: migration 006 adds view_bindings.visibility default private; smoke: (route "/x" "h") stays private; unauth GET /x ? login redirect; smoke: (route "/pub" "h" 'public) ; unauth GET /pub ? 200 handler; smoke: logged-in GET /pub still attaches whoami (optional auth); smoke: unauth GET /data and /admin/users and /api/eval still denied; smoke: unauth GET /bindings /edit/foo /repl denied; smoke: /pages shows public/private badge per route; smoke: child branch inherits parent public route until override; smoke: re-(route path handler) without visibility keeps existing visibility OR defaults private – document choice; prefer KEEP on handler-only update

========== WHY ==========
The image already has a single request gate (launch in image/lib/http.ml): hard public allowlist, else require session/Basic. That is correct kernel shape – but it cannot express "this Nopales page is a public share link."

Product needs: - Default-private user routes (todos, notes, gardens) - Opt-in public user routes (marketing, shared deck view) - Kernel paths that are NEVER public via Nopales - Optional auth: public page still richer when logged in

NOT this subsection: - Per-row ACLs / Alice-vs-Bob data isolation (multi-user.permissions – later) - Capability bits on functions (separate) - OAuth / JWT (still out; wyo.auth / session cookie)

========== GATE ALGORITHM (authoritative) ==========
For every HTTP request to the image:

1. Parse path (strip query). 2. Resolve identity OPTIONALLY: cookie pp_session ? else HTTP Basic ? else None. Do not fail yet; just compute user_opt. 3. Classify path into one of:

A. SYSTEM_HARD_PUBLIC /login /health /logout /welcome /book.pdf /src.tgz /code and /code/* /assets and /assets/* /.well-known/agent.json /agent.txt (extend only by OCaml change; see product-experience.public-release; /book is NOT hard-public; /agent is NOT hard-public)

B. SYSTEM_HARD_PRIVATE Always require identity (Nopales cannot open these): /admin and /admin/* /data and /data/* /api/eval /api/agent /api/agent-task /api/ai/* /api/edit/* /api/recipe/* /bindings /binding/* /edit/* /repl /branches and /branches/* /types /pages /page/* /inspect/* /agent (Implementation: prefix/exact table in http.ml; when unsure, DEFAULT PRIVATE.)

C. USER_ROUTE Session.lookup_route path succeeds ? visibility from route record: "private" (default) | "public"

D. UNKNOWN No system page, no user route ? still require auth before 404 (do not leak route existence to anonymous via different status; v1 may 302 login then 404 after login – acceptable). Prefer: unauth ? login redirect for HTML; 401 for API Accept.

4. Decision: - If class A ? proceed (login form has no product chrome). - If class B or C-private or D: user_opt None ? deny (302 /login?next= or 401) user_opt Some ? attach_from_user ? proceed - If class C-public: user_opt Some ? attach_from_user ? proceed user_opt None ? proceed as ANONYMOUS (below)

5. PP_AUTH_DISABLED=1 ? proceed all (dev escape; unchanged).

No multi-layer middleware DSL. One gate function.

==========

ANONYMOUS SESSION (public + no identity) ==========

- (whoami) ? "anonymous" (string) or document VNil – pick one in make; prefer VStr "anonymous" for easy tests. - Branch: system/main (read package defaults). Do NOT create or checkout a personal garden for anonymous. - author for any accidental write: "anonymous" - WRITE POLICY v1: CRUD prims (create/update/delete) and define/route/fork/merge/checkout are DENIED for anonymous (raise clear error or return error VStr). Public pages are read-oriented. public-write (POST forms that mutate) is PHASE 2 – not v1. - Chrome: product ui/nav MAY render; avoid admin links. If ui/nav fails, emergency chrome only.

==========

SCHEMA (additive migration 006_route_visibility.sql) ==========

ALTER TABLE view_bindings ADD COLUMN IF NOT EXISTS visibility text NOT NULL DEFAULT 'private'; – allowed values: 'private' | 'public' – no CHECK required if app validates; optional CHECK OK

Fork copy of view_bindings MUST copy visibility (UPDATE fork INSERT column lists).

list_routes / register_route SELECT/INSERT visibility. Default private for all existing rows (DEFAULT).

scripts/dev.sh migration loop: add 006_route_visibility.

==========

LISP API ==========

(route path handler) ? register private route (default). If route already exists and only handler changes, KEEP prior visibility (so re-seed does not clobber an intentional public). If brand-new route ? private.

(route path handler 'public) (route path handler "public") ? register/update route with visibility=public. Third arg also accepts 'private / "private".

(public-route path handler) ? sugar for (route path handler 'public)

(route-visibility path) ? "public" | "private" | () () if no visible route on current branch chain.

Agent tools: register_route gains optional visibility field (default private).

==========

INHERITANCE ==========

Same recursive CTE as today: nearest branch wins for a path. Child can override parent public?private or private?public by re-routing on personal.

==========

UI ==========

/pages table: column Visibility (public|private). Optional: make-public / make-private forms later; v1 Nopales/REPL is enough.

==========

SECURITY NOTES ==========

- Public handler still runs full Nopales: author must not leak secrets in public pages. Document footgun. - System hard-private list is deny-by-default for kernel surfaces; never consult view_bindings to open /admin. - data_rows remain shared storage; public route ? row ACL. Sharing one deck is handler logic (filter by id), not "all flash-card rows are public." - /api/eval stays hard-private (agents use Basic auth).

==========

PHASES ==========

Phase 1 (this subsection make): migration 006, store/session route record, route prim arity, gate algorithm, anonymous read, /pages badge, smokes above. Phase 2: public POST / public-write visibility value; next= return path after login; CSRF for cookie posts. Phase 3: share tokens / signed URLs for single-resource public access without opening a whole route (flashcards).

==========

ACCEPTANCE NARRATIVE ==========

Dev marks (route "/hello" "hi-page" 'public). Logged-out browser opens /hello and sees the page. Same browser on /todos still hits login. /data still login. After login, /hello shows whoami in chrome. Re-seed of private routes does not wipe an intentional public flag when only the handler is updated.

user-management

Admin creates and manages users and credentials
[status: implemented]

> *[verify] smoke: admin creates user + password; new user logs in on own personal; smoke: non-admin cannot create users; smoke: admin can revoke all sessions for a user*

Complete admin surface: create_user(username, is_admin?, user_type?, parent_user_id?) set_password(user_id, password) add_credential(user_id, kind, ?) ; api_key, ssh_key list_users / get_user revoke_sessions(user_id) revoke_credential(id) create_agent(parent_username) ; user_type='agent', parent=human issue_token(user_id, label?) ; kind='api_key', returns raw token once rotate_token(credential_id) ; revoke old + issue new revoke_token(credential_id) ; delete credential row

First admin: bootstrap-admin (env) when users empty. All later humans: this API/UI only.

Agent users: user_type='agent', parent_user_id = the human they belong to. One agent account; model identity lives in the function_def author field, not the username. Agents auth via Bearer token (issue_token), not password. Agents inherit the parent's llm_key – get_llm_key_secret falls back to parent_user_id when the agent has no own llm_key row. Global tokens are disabled; all tokens are per-user rows in credentials.

UI: /admin/users (admin session required). Enforcement: is_admin on session; 403 otherwise.

Credentials order of support in complete system: password (web-login), api_key (agents/pp_eval Bearer – auth INTO the OS), ssh_key (SSH REPL door – auth.user-ssh-repl), llm_key (provider key OUT to LLM – auth.user-llm-keys).

Admin UI should grow links to manage ssh_key and llm_key per user; self-service /settings/* is the human path.

identity-and-branch

Session identity drives personal garden and authorship
[status: implemented]

> *[verify] smoke: whoami equals session username after login; smoke: define lands on username/personal; smoke: Functions origin own only for true personal pins*

On every authenticated request / image attach:

For human users: ensure_branch owner=username name=personal parent=system/main (EMPTY child; CTE inherit) store.branch_id = that id manifest.username = username manifest.branch_name = username/personal

For agent users (user_type='agent'): ensure_branch owner=parent_username name=agent parent=system/main (EMPTY child; sibling of the parent's personal). The agent's default workshop is <parent>/agent, NOT the parent's personal. store.branch_id = that id manifest.username = agent identity (whoami returns the agent, not the parent) manifest.branch_name = parent/agent

(define name body) persists with author=username (or agent:<model> when agent installs under policy).

Agent llm_key: get_llm_key_secret falls back to the parent human's key when the agent has no own llm_key row.

Functions UI: effective one-name-one-row; origin own vs inherited (already implemented).

Agent-task: agent_branch_lock blocks checkout/fork/merge (already implemented); must run under the user's session.

Changing user mid-process (cotenant): rebind identity + checkout personal for that user; never leave previous user's pins writable.

15 borge/image.borg

pricklypear-image

Pricklypear book chapter: image
[status: partial]

> – *agent note:* 2026-08-02 status reconcile: flipped htmx/http-robustness/system-browser/tree-browser/function-editor/inspector/dnu-stubs ? implemented. Still partial: repl-experience (linenoise), per-request-context (store phase).

image

The per-user unikernel: Nopales interpreter, REPL endpoint, htmx interface
[status: partial]

boot

Single-image boot process: initialize Postgres, load stdlib, start services
[status: verified]

> *[verify] build: dune build image; smoke: image boots + serves / and /health + REPL accepts connections*

On boot, the image: 1. Reads identity from process environment: PP_USERNAME (REPL prompt), PP_BRANCH_NAME, PP_BRANCH_ID. 2. Connects to Postgres via pgx_lwt. 3. Applies pending migrations (additive-only). 4. Runs seed_stdlib: loads libs/*.pp into system/main. 5. Starts the REPL endpoint (internal line protocol on PP_REPL_PORT if set). 6. Starts the htmx server (httpaf-lwt-unix) on PP_HTTP_PORT. 7. Registers health endpoint at /health.

The image is STATELESS across reboots w.r.t. user data: anything that must survive is in Postgres. In-memory REPL state (define'd locals in a session) does NOT survive reboot unless the user persisted via (store) or similar.

boot-seed-batch

Boot seed batch-loads existing names once; additive persist skips per-name SELECT
[status: implemented]

> *[verify] build: dune build image; smoke: warm boot: image.log shows one [session] boot seed: N packages, M defs in <ms>ms line and ZERO get_by_path-per-define round trips; first POST /api/eval after boot < 2s; smoke: cold boot (fresh DB): every seeded name is present in (list-bindings); package provenance stamped*

Boot seed is ADDITIVE (branching.seed-policy option A): existing system/main rows are never clobbered. The expensive part of seed_stdlib is NOT rehydrate (it suppresses refresh and rehydrates once via checkout_personal at boot end) – it is that persist_function_if_absent / register_route_if_absent each performed a Tree_store.get_by_path SELECT per name. With ~1800 defines + ~34 routes that was ~1800 PG round trips on EVERY boot, warm or cold.

BATCHED SEED: seed_stdlib preloads the set of existing function names AND route paths on system/main ONCE (two queries) into a per-store seed-membership cache. persist_function_if_absent and register_route_if_absent consult that cache instead of issuing get_by_path per call: - name in cache -> skip entirely (no SELECT, no UPDATE, no upsert). Additive seed never touches existing rows. - name absent -> upsert into the store AND insert into

the cache, so later duplicate defines in the same seed pass are skipped too. The cache is populated only for the seed_additive boot pass on system/main; interactive load_library and reseed-package keep their existing per-call semantics.

INVARIANT: additive seed never overwrites an existing body. Skipping the existing-name path therefore preserves seed-policy exactly.

INVARIANT: boot logs one summary line [session] boot seed: <n> packages, <m> defs in <ms>ms so first-eval latency regressions are visible in /tmp/pp-dev/image.log.

boot-request-cost

/api/* never triggers the UI shell; identity attach never rehydrates an unchanged branch

[status: implemented]

> *[verify] build: dune build image; smoke: POST /api/eval with a cold ui-shell cache does NOT evaluate ui/nav or ui/css; GET / does; smoke: two identical Basic-auth POST /api/eval calls in a row trigger zero rehydrate*

Two per-request costs are removed or confirmed cheap for the agent path.

refresh_ui_shell INVARIANT: the UI shell (ui/nav, ui/css) exists for HTML chrome. The request gate calls refresh_ui_shell ONLY for paths that render chrome. /api/* handlers NEVER trigger a ui/nav or ui/css eval, even when the shell cache is cold. (The 60s TTL cache already amortizes HTML-page requests; this removes the cold-first-eval cost from the agent path entirely.)

attach_identity INVARIANT (already holds, documented): if the resolved identity's username equals the session's current manifest username, attach is a manifest-field update only – it MUST NOT call ensure_personal_branch / checkout_personal / rehydrate. Rehydrate happens only when the branch actually changes (prev <> username or explicit checkout). Repeated Basic-auth requests from the same user therefore never rehydrate.

lazy-env-load

load_defs binds VThunk recipes instead of evaluating bodies; rehydrate is one CTE + decode, never a body eval

[status: implemented]

> *[verify] build: dune build image; smoke: rehydrate of the full branch chain completes in ms; zero [session] load def lines logged; smoke: a def whose body executes only runs (and logs failure) on FIRST reference, not at checkout/attach*

The image half of lisp-interpreter.lazy-env-thunks.

load_defs walks the recursive-CTE def list (parents first, child overrides) and binds each name to a VThunk payload whose force evaluator is the SESSION's rim eval (Control_rim.run around Eval.eval with the current budget when one is active, fresh budget otherwise). Macro bodies keep the old eager path (VMacro construction is pure, no eval). Defs whose stored body is an APPLICATION form – data captured as code (rb/* recipes, outline experiments, seeds) – no longer execute at boot/checkout/attach; they cook once on first reference, under the forcing request's budget, and memoize (value OR failure).

SEMANTIC CHANGE (deliberate): nothing at image scope may rely on load-time side effects of stored def bodies. A def is a recipe, not an init hook. If boot order matters for a real side effect, it belongs in boot code, not in function_defs. Alphabetical-load-order breakage of the old scheme (rb/* evaluated before recipe was defined ? 28 defs failing on EVERY rebuild, re-running partial side effects each flip) disappears: forcing resolves through the fully linked env.

per-identity-sessions

One Session.t per attached identity (env+routes+store handle); attach selects, never mutates; anonymous gets its own parked session

[status: implemented]

> *[verify] build: dune build image; smoke: alternating anonymous GET /welcome and Basic-auth POST /api/eval: neither pays a rehydrate when nothing changed; smoke: a define by admin is visible to the anonymous session within one staleness check; smoke: concurrent requests for two users run page work in parallel (no head-of-line blocking)*

PROBLEM (measured on prod 2026-08-21) One shared Session.t + per-request attach meant every alternating-user request flipped the session's branch and paid a FULL rehydrate under the global request_mutex (60-120s on admin/personal with ~4000 defs ? strict eval). boot-request-cost's invariant ('rehydrate only when the branch actually changes') was vacuous: with one session, every OTHER user's request IS a branch change. The cotenant race that Request_context snapshots band-aid exists only because the session is shared.

MODEL Session_registry owns a Hashtbl identity -> Session.t: - one long-lived anonymous session, parked on system/main, serves all unauthenticated traffic (public face: /welcome, /agent.txt, public routes) - one session per authenticated username, built lazily on first attach, parked on that identity's branch - each session has its OWN Store.t (own pgx connection): checkout mutates only its own store, and the global pgx sequencer serialization between users disappears - attach = resolve identity -> select (or create) -> cheap staleness check -> serve. Attach NEVER checks out another identity's branch.

STALENESS A parked session must notice writes on its visible chain (another user's promote, an agent's define). At attach, one cheap query per session – count(*)+max(updated_at) over function_defs / view_bindings / type_defs on the session's branch chain – compared against the values at last load; mismatch triggers rehydrate (cheap under lazy-env-load). Attach cost when idle: one indexed round trip, ~ms.

CONSEQUENCES - Invariant 4's cotenant race is deleted at the root: Request_context remains as the per-request snapshot discipline, but no concurrent attach can mutate the session a page is reading, because page work runs on the identity's own session. - request_mutex narrows to registry table access; page work across identities parallelizes (subject to PG). - Sessions are never expired in v1 (user count is small; revisit with an LRU if a public signup flow appears). - Boot session remains for boot/seed/jobs-runner use.

function-stat-ring

function_stat writes go through an in-process ring flushed by a background fiber; no per-eval DB write

[status: implemented]

> *[verify] build: dune build image; smoke: burst of labeled evals produces zero synchronous data_rows writes; flush fiber lands them <= 30s later*

Every labeled /api/eval called Crud.create for a function_stat row – a synchronous SERIALIZABLE insert + fsync on the request path, and 54k unbounded rows on prod after a few weeks. Same medicine as http-robustness's ops-incidents rule: request path pushes to an in-process ring (capacity 512, drop-oldest), a background fiber flushes at most once every 30s. Loss on crash is acceptable: these are perf stats, not ledger.

ssh-interface

Deprecated: SSH access removed in single-image pivot

[status: deprecated]

[verify] build: dune build @all; smoke: SSH access removed – use browser /repl or POST /api/eval

The single-image pivot removed supervisor and SSH. Interactive access is via: - Browser REPL at /repl - Agent API at POST /api/eval - Optional line-protocol REPL on PP_REPL_PORT (if set)

repl-endpoint

Image's internal REPL endpoint, not full SSH
[status: implemented]

[verify] build: dune build image; smoke: define over the line REPL persists to the store; a second session sees it

The image does NOT run an SSH server. It optionally exposes a line-protocol REPL – a simple TCP socket on PP_REPL_PORT (unset = no listener), intended as an operator surface on a trusted network. The primary interactive surfaces are the browser /repl and POST /api/eval (both authenticated).

Protocol (minimal, framed): - Connect: operator opens TCP to image:PP_REPL_PORT - Auth: none on the socket itself – the port is bound for operators only; do not expose it beyond the trusted network - Exchange: lines of UTF-8. User input flows in; REPL output flows out. - The image's session.ml tracks this connection as a session; on disconnect, session ends.

The endpoint is in image/lib/repl_endpoint.ml. It owns: accepting connections, wiring the stream to the line editor (lineedit.ml), and the REPL read-eval-print loop itself.

repl-experience

What the user sees: prompt, multi-line, line editing, meta-commands
[status: partial]

– *agent note:* 2026-08-02 confirm partial: lineedit.ml is minimal read_line stub; linenoise history/cursor still TODO. browser-repl covers interactive UX.

[verify] build: dune build image; smoke: define + apply works end to end through the REPL

THE PROMPT: alice/main> – showing user + current branch alice/main (def) | – mid-multiline-define

READING INPUT: - The REPL reads s-expressions. It keeps reading lines until parens are balanced and strings closed (standard Nopales REPL behavior). The reader tells us 'incomplete' so we know to prompt for more, not to evaluate yet. - One REPL turn = one s-expression (which may span many lines). - Comments: ; to end of line, as in the reader.

LINE EDITING: - Real line editing matters for a dev tool. Raw SSH PTY + a line-editing layer. - Library choice deferred to build time: ocaml-linenoise is the leading candidate (small, Mirage-compatible-ish). Pure OCaml. - Supports: cursor movement, history (up/down), Ctrl-A/E, backspace, delete. - Search/history-reverse-search: nice-to-have, not v1-blocking.

EVALUATION + PRINTING: - Evaluate the s-expression in the session env. - On success: pretty-print the resulting value. VList -> (a b c), VLambda -> #<lambda>, VRef -> #<ref event:uuid>, VInt -> 42, etc. - On error: print the error with location if possible (line/col from the reader), then a continuation prompt. Don't kill the session.

META-COMMANDS (Nopales forms, not a separate command lexer – the reader handles them, the eval loop recognizes them as special forms): (define name expr) bind + persist to store (edit name) print htmx URL for AST editor

(fork parent child) create a branch (checkout branch) switch session's branch (merge source target) reviewer flow (describe 'type) show type schema (list-bindings) current env (load "file") eval a file's contents (help) list specials Any other Nopales form evaluate + print result

SESSION STATE: - (define ...) persists to the store: survives image reboot (loaded on next boot of the branch). - REPL-local let bindings (e.g. (let ((x 5)) ...) at top level): session-local, not persisted. Like bash variables. - Switching branch (checkout) within a session keeps REPL locals but changes which function defs are visible. (Decision: this is the 'working directory' model – you carry your locals across branch switches.)

htmx-interface

User routes accept GET and POST; request context available via dynamic-binding prims. bible/* and other handler-only fns now guarded (require_request raises Interp_error User_error on direct /api/eval; api_eval catches as clean JSON error – fixes inbox c5-1054).

[status: implemented]

> – *agent note:* 2026-08-02: request-method/param/params + redirect prims; user routes GET/POST via http.ml; auth/lib/http.ml path retired (in-process).
>
> *[verify] build: dune build image; smoke: POST user route hits same path handler; (request-method) => POST; smoke: POST form-urlencoded body: (request-param "title") returns field; smoke: GET query: (request-param "q") returns query value; smoke: (redirect "/todos") => 303 Location: /todos; smoke: POST /api/recipe/todo then POST /todos title=? creates row + redirects; GET /todos lists it*

The image's htmx endpoint serves two kinds of pages:

1. SYSTEM pages (always present, part of the OCaml image): /health health check /inspect/binding/<name> pretty-print one function_def's AST /inspect/row/<type>/<id> browse any stored value /branches list + fork/merge UI /types type browser (type_defs) /data ? data rows browser (full CRUD; see image.data-browser) /bindings function browser (per branch)

2. USER pages (defined as functions in the store): registered via (route "/path" fn-name). The route handler runs with a REQUEST CONTEXT (method, path, params) and returns a value the system renders (HTML, redirect, text, etc).

Routes ARE functions in the store. Adding a route = (route "/foo" 'foo-handler). This makes the user-facing UI part of the live image, editable via the same mechanisms as everything else.

ROUTE VISIBILITY: User routes default private (session required). (route path handler 'public) opts into anonymous GET. Kernel paths (/data /admin /api/eval /bindings /edit ?) are hard-private in OCaml and cannot be opened by Nopales. Gate runs before handler dispatch.

pure-html (the dream_html HTML generation core, usable WITHOUT Dream) is used for HTML generation (use Hx.* not raw strings at the OCaml layer).

REQUEST CONTEXT (dynamic, handler-scoped): Zero-arg handlers keep working: (define hi-page (lambda () (render-html "?"))) Interactive handlers read request via prims while the HTTP dispatch is inside call_handler: (request-method) -> VStr \"GET\" | \"POST\" (request-path) -> VStr path without query (request-param k) -> VStr value | VNil if missing (request-params) -> VStr JSON object of all params (request-body) -> VStr raw body (\"\" on GET)

HANDLER RETURN VALUES: 1. render-html tagged VStr (___html___:?) -> 200 text/html, body = raw HTML inside chrome 2. bare VStr -> 200 text/html, <p> escaped text 3. other values -> 200 text/html, <pre> pp_value

4. (redirect path) -> tagged VStr ___redirect___:path -> HTTP 303 See Other, Location: path, empty/minimal body Path must be absolute-on-host.

http-robustness

HTTP server robustness under load, peer disconnect, and DB liveness
[status: implemented]

> – *agent note:* 2026-08-02: closed-writer swallow, request_mutex narrowed to gate, page-work queue, Store.health_check on /health, http_robust tests present.
>
> *[verify] build: dune build image; smoke: abort N concurrent requests mid-response; image stays up, /health 200 throughout; smoke: open SSE /api/agent stream, disconnect client; no Fatal error in image.log, process pid unchanged; smoke: fire M concurrent mixed routes (page + SSE + /api/eval); all complete or peer-gone, zero process exits; smoke: GET /health returns ok:true only when SELECT 1 on store succeeds; ok:false + non-2xx when DB path is dead; smoke: rcctl check fails when /health reports ok:false; watchdog restarts a live-but-poisoned image; test: image/test/http_robust_test.ml – closed-writer guard returns () not raises; catch-body does not re-write to dead writer; test: image/test/io_eio_test.ml – Sequencer.enqueue survives a raised SQL-like exception and subsequent enqueue still runs*

The httpaf body writer raises Failure("cannot write to closed writer") when you write to a response whose client has disconnected. On a single-image process, one uncaught closed-writer exception is a process exit. These invariants make a peer-disconnect a non-event, and define the concurrency model so the server does not serialize all traffic behind one lock.

==========
INVARIANT 1 – PEER-DISCONNECT IS NEVER FATAL ==========
Every code path that writes an HTTP response body MUST tolerate a closed/gone writer by returning () (or a benign 'Peer_gone signal), never by raising. A client closing its tab is the single most common event on a web server; it must not take the process down.

Mechanism (http_core.ml):

- is_closed_writer_exn : exn -> bool - safe_respond_with_string : respond_with_string wrapped; swallows the closed-writer Failure - safe_close_writer : close_writer wrapped; swallows the closed-writer Failure

EVERY response helper routes through the guarded path: html_response, text_response, json_response, fragment_response, redirect_response.

The httpaf error_handler (http.ml error_handler) writes unguarded via Body.write_string + close_writer; it MUST be guarded the same way – it runs on connection-level errors where the peer is very often already gone.

==========
INVARIANT 2 – ERROR RECOVERY MUST NOT RE-WRITE A DEAD WRITER ==========
launch's Lwt.catch exception branch (http.ml) currently responds to a handler failure by calling html_response ... ~status:'Internal_server_error on the same reqd. If the original failure was a closed-writer (peer gone), the recovery write fails the same way and escapes the catch.

Rule: the catch branch MUST detect the closed-writer case (is_closed_writer_exn on the caught exn) and, instead of rendering a 500 page, just safe_close_writer the (already-dead) writer and return. Only when the writer is still alive do we render the 500 error page.

Concretely: in the (fun exn -> ...) handler, branch: if is_closed_writer_exn exn -> safe_close_writer; Lwt.return_unit else -> Ops_incidents emit + html_response 500 This breaks the throw-in-the-catch loop that would otherwise turn one peer-disconnect into a process exit.

==========
INVARIANT 3 – ui/nav FAILURE IS NOT PER-REQUEST NOISE ==========
refresh_ui_shell runs on every request (http.ml launch). When ui/nav is broken on the branch it logs a [warn] line PER REQUEST – under HN traffic this produced hundreds of warn lines and wasted a Nopales eval per page.

Rule: refresh_ui_shell MUST cache its failure state per branch (at least short-TTL in-memory): once ui/nav has failed on a branch, do not re-attempt and re-warn on every request; downgrade to a single debug line (or a periodic retry on the order of minutes, not per-request). The emergency chrome fallback already serves the page; the spam is pure waste. This is not a crash fix but a log-hygiene + cost fix surfaced by the same incident.

==========
INVARIANT 3.5 – INCIDENT AUTO-FILING IS ASYNC, NOT IN THE REQUEST FIBER (2026-08 incident-write amplification) ==========
A repeating route failure (e.g. ui/home unbound on a branch) used to call Ops_incidents.emit_and_maybe_file on EVERY page hit: a synchronous Crud.create (+ tree-mirror trigger writes + a TODO row) inside the request fiber, multiple serialized PG round-trips per request – observed as GET / taking 10-40s with DB writes mid-request.

Rule: Ops_incidents buffers events in an in-process ring (dedup by fingerprint, count++ coalescing as today); a background fiber flushes the ring at most once every N seconds. Request-path cost of auto-filing is a ring push ONLY – zero DB ops. One repeating bug = one incident row + one TODO total, not one per page view.

==========
INVARIANT 4 – CONCURRENCY: IDENTITY ATTACH IS SERIALIZED, PAGE WORK IS NOT (immutable-env + narrowed mutex) ==========
Status quo: a single global request_mutex (http_core.ml) is held for the ENTIRE request lifecycle – identity resolve, classify, refresh_ui_shell, AND page_fn (including slow LLM/SSE streaming). This serializes the whole site to one-request-at-a-time and, worse, lets one streaming agent chat block /login and / for everyone.

The naive fix (just release the mutex before page_fn) is UNSAFE because Session.t is one shared mutable record and page work reads two shared mutable fields: (1) t.env – rehydrate does Hashtbl.clear t.env.bindings IN PLACE. VLambda closures capture env by reference (env = { bindings: Hashtbl.t; parent }), so a concurrent rehydrate during page eval would empty the binding table an in-flight lambda is reading through its captured env -> unbound-var mid-page. (2) t.manifest – attach_identity mutates t.manifest in place; prims (crud, agent) read t.manifest.username live via the !t ref they close over. A concurrent other-user attach would flip the username mid-page -> wrong-user attribution for writes.

RULE (two parts):

4a. IMMUTABLE-ENV REHYDRATE (the correctness prerequisite). rehydrate MUST NOT mutate the existing env's binding table in place. Instead it builds a FRESH env record (new Hashtbl, same parent chain) and swaps Session.t.env to point at it. Session.t.env becomes a mutable field. The OLD env record stays intact for the lifetime of any VLambda closure that captured it, so a concurrent rehydrate cannot corrupt an in-flight page's captured environment. This is the foundation that makes any mutex-narrowing safe.

4b. NARROWED MUTEX. request_mutex protects ONLY the identity-attach critical section (resolve_identity + attach_from_user / attach_anonymous + classify + the post-attach refresh_ui_shell). The mutex is released BEFORE page_fn runs. Page work – user-route handlers, /api/eval, and the agent SSE streaming loop – runs WITHOUT the global lock, against the (now-immutable) env captured by closures.

4c. PER-REQUEST CONTEXT (fix for BORGE-1784434193 cotenant race). After the identity gate, a fresh Request_context.t is built from the

attached manifest/env/routes and passed to all page work. Prims and dispatch read the context instead of live Session.t fields. Anonymous attach can no longer flip an in-flight personal-branch page.

SUPERSEDED ROOT CAUSE (image.per-identity-sessions): the cotenant race existed because ONE Session.t was shared across identities. With per-identity sessions, attach no longer mutates a session another request is reading; Request_context stays as the snapshot discipline, but 4a's fresh-env-swap requirement ceases to be load-bearing for cross-user safety (still useful for same-session rehydrate during long page work).

(subsection per-request-context (doc "Request_context.t + dynamic binding to eliminate cotenant race") (status partial) (* agent note (|2026-08-02 confirm partial: Request_context.t snapshots manifest+env+copied routes; prims use dynamic binding. Remaining: store on context (phase 3) + full prim audit.|) *) (implements image/lib/request_context.ml image/lib/http.ml image/lib/session.ml image/lib/primitives.ml image/lib/crud.ml) (verify (build "dune build image") (smoke "admin/personal route survives concurrent anonymous attach") (test "image/test/http_robust_env_test.ml extended for manifest isolation")) (details (doc (| Shipped mechanism (direct-style, post-H1): - Request_context.t record (manifest snapshot, env, routes, store, request) in image/lib/request_context.ml. - Request_context.current is a dynamically-bound per-request snapshot; prims read the snapshot, not the live shared Session.t ref. with-public-write re-binds an elevated context for the body so static gate and runtime guard agree. - Route dispatch reads context.routes; page work after the attach gate runs against the snapshot, so a concurrent anonymous attach cannot swap the manifest mid-render. - Session.t still used for attach gate + rehydrate/checkout_to. - The earlier plan to change the prim ABI to Request_context.t -> value list -> value Lwt.t is retired: H1 made the prim ABI direct-style (prim_fn = value list -> value, Control.perform_prim) and the context travels by dynamic binding instead of an explicit first argument. - Remaining: full env/routes snapshot isolation depth and the audit that every prim reads the snapshot (phase 2). |)))

SCOPE / MULTI-TENANT CAVEAT (honest): the manifest race (4b concern 2) is NOT fully closed by this work. Prims still read t.manifest live via the !t ref. This is safe ONLY while concurrent requests share one identity (one authed user, plus anonymous traffic sharing manifest.username=anonymous): t.manifest reads during page work then cannot attribute a write to the wrong user. Cotenant operation with DISTINCT authed users concurrent needs a per-request manifest context threaded through the prim dispatch so prims read the request's manifest, not the shared record. That is deferred to a follow-up borge issue (ABI surgery on prim_fn; not a crash/throughput fix). Until it lands, cotenant hosting of multiple simultaneously-active users carries this documented caveat.

The single shared Postgres connection (Store.t.conn) remains a throughput ceiling, NOT a correctness risk under the narrowed mutex: pgx_lwt serializes all operations on a connection internally via its per-connection Sequencer (type t = conn Sequencer.t; every execute/with_transaction goes through Sequencer.enqueue = Lwt_mutex.with_lock on a per-connection mutex). Concurrent Pg.execute calls on the same conn therefore QUEUE safely – no wire interleave, no corruption. So narrowing request_mutex does NOT require a separate db_mutex: DB prims stay correct (serialized by pgx) and page rendering (Nopales eval, HTML build – the slow part) runs lock-free. The single conn is still a throughput ceiling (a pool is a separate later iteration), but it cannot corrupt under concurrency.

==========

INVARIANT 5 – ACCEPT-SIDE CONNECTION CAP ==========

Status quo: serve (http.ml) calls Httpaf_lwt_unix.Server.create_connection_handler with no limit and Lwt_io.establish_server_with_client_socket with no backlog/fd cap. Under a spike the accept loop can exhaust fds or memory even

after crashes are fixed.

Rule: gate the acceptor on a bounded semaphore (an Lwt_semaphore or a counter with a max, default 256 in-flight requests, configurable via PP_HTTP_MAX_INFLIGHT). Requests beyond the cap are rejected early with a 503 (or simply not accepted until a slot frees). This bounds fd/memory usage to a known ceiling regardless of traffic. The cap is a safety valve, not a throughput target – normal load should be well under it.

NOTE: this is defense-in-depth on top of the crash fix. The crash fix (invariants 1-3) is what stops the death loop; the cap (invariant 5) plus the mutex narrowing (invariant 4) are what make the server survive the next traffic spike on throughput, not just on not-crashing.

==
INVARIANT 6 – EIO PG SEQUENCER MUST NOT PERMANENTLY POISON ==
Incident 2026-07-22 (pricklypear.rocks): under PP_IO=eio the live image kept serving /health 200 while every auth/DB call failed with Eio___Eio_mutex.Poisoned(_). Login and Basic auth both surfaced as "Invalid username or password" because auth_store.password_hash_for_user catches all exceptions and returns None. The process stayed up for days; the watchdog never fired because /health never touched the DB.

Root cause: Io_eio.Pgx_io.Sequencer.enqueue used Eio.Mutex.use_rw ~protect:true. Eio documents that use_rw DISABLES the mutex forever if fn raises – any SQL error (or cancel) permanently kills the single shared PG conn sequencer. Subsequent enqueues raise Poisoned.

Rule: the pgx Sequencer critical section MUST unlock on exception (use_ro-style unlock-and-re-raise), while still using Cancel.protect so cancellation cannot leave the wire mid-message. Do NOT call use_rw for the sequencer: a raised Pgx/PostgreSQL error does not mean the mutex resource invariant is broken – the connection protocol recovers; only the operation failed. Optional later: detect hard I/O death and reconnect Store.t.conn; that is separate from not poisoning the lock.

Mechanism (io_eio.ml Sequencer.enqueue):

lock; Cancel.protect (fun () -> f v); unlock on exception: unlock; re-raise

Never poison. Never leave the mutex locked.

==
INVARIANT 7 – /health PROBES DB LIVENESS ==
/health remains hard-public and outside the inflight page cap (invariant 5 note). But "ok" MUST mean the image can still talk to Postgres on the store path used by auth and prims – not merely "process is alive and JSON encodes".

Rule: - GET /health runs SELECT 1 (or Store.health_check) on Session.store via the same dual-conn exec path. - On success: HTTP 200, {"ok": true, ... error: null}. - On failure (no conn, query error, Poisoned residual, timeout): HTTP 503, {"ok": false, error: "..."}. - Cheap: one round-trip; must not take request_mutex or the inflight page semaphore. - rc.d rc_check and the promote health loop MUST match ok:true (not merely the substring "ok" – "ok": false must fail the check). - Watchdog: when rcctl check fails, stop/kill stale image (pidfile + pkill -U runtime user on bin/image) then start. Plain rcctl start is insufficient when a live-but-poisoned process still holds :18080 without a usable pidfile (EADDRINUSE on start). - rc_stop / promote restart path: always clear the port before start (pkill runtime image; rm stale pidfile).

This closes the class of failures where the accept loop is healthy but the DB mutex/conn is dead – the failure mode that locked out all logins while looking green.

primitives-layer

image/lib/primitives.ml: the impure fns registered at boot

[status: implemented]

> *[verify] build: dune build image && cd lisp && dune build && dune runtest; test: image/test/primitives_test.ml 48 alcotest cases; lisp/test/eval_test.ml 30 pure tests*

See lisp-interpreter.primitives for the full primitive list. This subsection covers the IMAGE layer's responsibility: registering them with the interpreter at boot.

The interpreter is pure and knows nothing about Postgres, htmx, or SSH. The IMAGE layer: 1. Opens a Postgres connection (v1: pgx_lwt + custom Io_unix shim; Solo5 later may use pgx_lwt_mirage behind the same Store.t). 2. Constructs an initial env with VPrim values bound to names like 'get', 'create', etc. 3. The interpreter's Prim call dispatches by name to the OCaml function the image registered. 4. Each primitive handles its own transaction (SERIALIZABLE by default) and permission checks (per multi-user.permissions; author- scoped filtering still partial).

This separation is what makes the interpreter testable without any services running (pure lisp/test/eval_test.ml, no DB, no HTTP).

v1 SHIPPED registration path: real. Kernel I/O prims shipped (Effect.pure_prims / Net): http-get (Net; admin/agent), xml-parse, html-decode, parse-date, url-encode/decode, sha1, string helpers, random-uuid, now, now-ms, sleep (Lwt, max 60s/call). DEFERRED: Solo5 pgx path; host cron/scheduler table.

define-docstring (lisp-interpreter): image registers (function-doc name) and wires Store.persist_function ?doc so top-level define can document bindings into function_defs.doc. See that subsection for surface syntax and redefine/clear rules.

outbound-http

(http-post url headers-json body) – send-side sibling of the shipped http-get. Same curl-backed shape, same gate, same payload contract. The first consumer is notify.ntfy-delivery (lock-screen push from jobs schedules).

[status: planned]

> – *agent note:* 2026-12 plan pass: daily-scripture campaign. The Net effect tag already exists in the lattice (lisp/lib/effect.ml tag 8; net_prims = [http-get]); http-get is shipped (prims_net.ml, curl-backed, admin/agent-only). http-post is the additive send half.

> *[verify] build: dune build image; smoke: (http-post "http://127.0.0.1:18080/health" "{}" "x") as admin returns a payload dict with ok/status/body; smoke: anonymous + non-admin/non-agent caller -> denied Prim_error, same gate as http-get; smoke: unreachable host raises transient Prim_error (curl (7) ...), NOT a payload dict; smoke: (ntfy/send ...) from a jobs schedule delivers (integration smoke with notify.ntfy-delivery)*

SURFACE (one prim, additive – lisp-interpreter.frozen-surface additive rule): (http-post url) ; empty headers, empty body (http-post url headers-json) ; body = \"\" (http-post url headers-json body) ; full form

SEMANTICS: the EXACT http-get contract, send half: - Curl-backed (do_http_get's shape in prims_net.ml): -X POST –data-binary @body_tmp, same temp-file dance, same shell_quote discipline, same -D/-o split. - headers-json: same \"Assoc string->string\" format as http-get; \"_timeout_ms\" reserved header honored the same way (default 20000). - Returns the SAME payload dict VStr: {ok, status, headers, body, err, kind}; 4xx/5xx are returned (kind = http-client-error / http-server-error), never raised. - status=0 / curl transport error RAISES Prim_error transient (design note ?4b), so guard handlers and the jobs runner's attempts/backoff see a real failure. - Effect: Net. Add \"http-post\" to net_prims in lisp/lib/effect.ml (additive tag use; lattice unchanged).

GATE (unchanged from http-get): identity confinement, not host confinement. anonymous -> denied; non-admin, non-*/*agent -> denied (\"admin/agent/human only\"). Rationale carried from http-get: SSRF risk is bounded by who may call, not by a destination allowlist. Host allowlist (PP_HTTP_ALLOW_HOSTS) stays DEFERRED – name it here so nobody thinks it shipped: if outbound http ever opens to non-admin users, an allowlist lands first.

NON-GOALS (v1): http-request general method surface (PUT/PATCH /DELETE) – add when a consumer needs it, same shape; streaming/chunked bodies; connection reuse; a native Eio client replacing curl (the deps for tls-eio already build on the image for remote/'s client, but curl is honest and shipped – port only if curl's process-spawn cost measurably hurts, e.g. notify fan-out loops).

system-browser

The live environment UI: browse branches, functions, types as a place you live
[status: implemented]

> – *agent note:* 2026-08-02: /bindings /pages /types live; ui/nav shell; live GET /bindings?pkg=todo + /pages 200.
>
> *[verify] build: dune build image; smoke: GET / shows user+branch chrome and links to about/functions/repl; smoke: product nav comes from (ui/nav) when shell loaded; not silent OCaml product twin; smoke: product CSS comes from (ui/css) / ui/theme when shell loaded; not silent full OCaml design-system twin; smoke: nav has no /edit/hello; Pages links to /pages; smoke: GET /pages columns: page URL + handler function; smoke: GET /page/hello shows route + handler + callee list; smoke: GET /bindings lists packages including (kernel) and (root)/workspace pkgs; smoke: GET /bindings?pkg=(kernel) lists primitives e.g. create, +; smoke: GET /bindings?pkg=(kernel)&fn=create shows read-only prim docs; smoke: GET /bindings?pkg=todo lists todo/* functions; smoke: GET /bindings?pkg=todo&fn=list shows function detail + edit link; smoke: GET /bindings?q=todo filters; mobile-width usable (nav wraps); smoke: define todo/list + call (todo/list); used-by shows /todos when routed; smoke: function detail shows Callers + Callees panels from shared graph; smoke: function detail links Impact / Changes / Inspect / Edit; smoke: wide desktop: three panes packages|functions|detail when viewport wide*

VOCABULARY (must not conflate): - Page / endpoint = route URL (e.g. /hello) - Function = named binding (e.g. hi-page or todo/list) - Package = in-branch name prefix (todo/), NOT a branch - Branch = versioned code world (system/main) A page RUNS a function; you EDIT functions, OPEN pages. Packages ORGANIZE functions inside one branch.

==========

SMALLTALK COMBO PACK – browser is the hub ==========

Steal five things; they compound through THIS surface:

1. Senders + change impact ? image.inspector 2. Branch change set ? branching.change-set 3. Value + closure inspect ? image.inspector (shipped+) 4. doesNotUnderstand holes ? image.dnu-stubs 5. Three-pane package browser ? THIS section

Shared spine (do not rebuild per feature): - static App-Var graph (collect_var_apps + reverse) - define/install hooks (persist_define + impact + transcript) - AST pretty / inspect routes already on host UI

The browser is where a human or agent LIVES: pick a package, open a function, see callers/callees/routes/impact/holes, jump to edit or inspect, open branch changes. Not five apps.

v1 SHIPPED: - Chrome: whoami, branch, uptime, nav - Home entry points + AI propose form - /about concepts guide - /pages list + /page/<path> inspector - Flat /bindings table (name, effects, author, edit, inspect) - Mobile three-step drilldown (?pkg=&fn=)

FUNCTIONS BROWSER (/bindings) – Smalltalk-ish drilldown:

PHILOSOPHY: - Majority of programs are Nopales bindings in function_defs. - Kernel primitives are OCaml (performant, trusted); still BROWSABLE so the system is legible, but read-only. - Optional future: mark some OCaml modules as "image libs" for hot paths; not required for this cut.

MOBILE FIRST (three-step drilldown, not three columns): URL state via query params (works on phones): Step 1 GET /bindings ? list packages only (big tap targets) Always includes virtual package "(kernel)" Workspace packages from prefix grouping + "(root)" Step 2 GET /bindings?pkg=todo ? list functions in that package Kernel: list prim names in catalog Step 3 GET /bindings?pkg=todo&fn=list ? detail pane (DETAIL CONTRACT below) workspace OR read-only kernel doc (primitives) Back links: Functions ? package ? (back to packages). Mobile stacked drilldown remains canonical at ~390px.

DESKTOP THREE-PANE (combo pack – same URLs, wider layout): When viewport ~900px+, ONE page MAY show three columns: left: packages | middle: functions in pkg | right: detail Selection still ?pkg=&fn= (shareable, no SPA). htmx MAY refresh middle/right; full GET without JS still correct. Do not force three columns on phones.

DETAIL CONTRACT (step 3 / right pane) – compounds the pack: Workspace function_defs always show: - kind, effects, author, updated - source pretty + Inspect AST/JSON - [Edit] ? /edit/<name> - Callers (direct reverse) + Callees (direct forward) from image.inspector shared graph helpers - Used-by routes via used-by-routes on DETAIL only (package index stays handler-direct – no N*depth) - Impact badge if name pending in impact-queue - Holes: unbound App Vars (image.dnu-stubs) + [Stub] - Link to branch Changes when name is own-row override Kernel prims: read-only docs only (no Edit/Stub).

KERNEL CATALOG: - Virtual package name: "(kernel)" (not a real binding prefix) - Source: prim registry + short docs (name, summary, effects) - Kind badge: primitive | function - No Save on primitives; link to About/docs if needed

WORKSPACE (existing + keep): - Group by package_of_name; search q=; author filter - used-by pages; create package+local form - Edit only for function_defs rows

UI SHELL AS LISP (authoritative):

RULE: If a human thinks of it as "the environment UI" (navbar, home workspace, app menus), it is Nopales on a branch – not a parallel OCaml product skin.

- Product shell bindings (required when seed runs): ui/menu – menu tree (prefer zero-arg lambda that returns a list; eager lists are fragile if they call menu/* at define-time) ui/nav – (lambda () (render-html (menu/bar ?))) ui/home – (lambda () ?) home body HTML ui/theme – css/sheet data (tokens + rules); Nopales, not a CSS string blob ui/css – (lambda () (css/render ui/theme)) adapter only; session caches its string - libs/ui/lib.pp seeds system/main (load-library "ui" / seed_stdlib). Personal may pin overrides; inherit main when no pin. Prefer thunks over eager value pins. - Chrome nav uses (ui/nav); GET / uses (ui/home); full-page <style> uses rendered (ui/css) – see image.product-theme. - Identity strip only (brand, username, branch, uptime) stays OCaml – not product structure. - Kernel OCaml pages stay OCaml until deliberately moved: /login, /health, auth middleware, /bindings,/branches, /edit, /inspect, emergency recovery. These are host UI, not a second apps menu. - EMERGENCY chrome (nav/home/theme fail or unbound): minimal, labeled failure (must not look like a healthy product menu or full design system). Silent rich OCaml default_nav / default_home / full inline CSS twin is REJECTED architecture – debt to remove. - Rehydrate load order: parents first, child overrides last (so personal ui/menu can call system menu/link; personal theme/* tokens can override system tokens). - Edit shell via Functions ? package ui (mobile drilldown), including ui/theme and theme/* tokens.

LAYERING SUMMARY: Nopales = the computer you live in (shell + theme + apps) OCaml = host + gate (eval, store, auth, health, emergency)

NAV / ENTRY (combo pack discoverability): - Functions (/bindings) = primary code home - Branches detail ? Changes (branching.change-set) - Optional chrome badge: pending impact count (light) - Unbound handler pages ? Create stub (dnu-stubs)

DEFERRED: - Remove silent default_nav_nodes / default_home_content and shrink html_response style blob to emergency_css (image.product-theme make) - ui/menu seed always lambda (load-order safe by default) - Move softer system pages (/about copy) into Nopales packages - Branch switcher in chrome (detail checkout exists) - Import/require, private symbols, rename-with-rewrite - Authoring new prims from the UI - Dedicated /theme form UI (Functions edit is enough v1; full picker in image.themes)

DATA BROWSER: full CRUD over data_rows is specified in image.data-browser (OCaml system pages, not a Nopales package). /types remains the type_defs index and links into /data/<type>.

TREE BROWSER: lazy zoomable tree UI over the one tree is specified in image.tree-browser (a Nopales package, not OCaml system pages). system-browser stays the code hub (/bindings); tree-browser is the universal *content* face (notes outline, data rows, eventually pkg/type subtrees).

tree-browser

Lazy zoomable tree UI: folders ARE non-leaf nodes; htmx depth-capped expansion; library-first (no kernel prims)

[status: implemented]

> – *agent note:* 2026-08-02: libs/tree tree/page+partial+render-doc; live GET /tree 200.
>
> *[verify] build: dune build image; smoke: GET /tree renders root to depth cap; deeper nodes as collapsed stubs; smoke: GET /partials/tree?slug=?&root=?&depth=2 fragment no chrome; expands stub in place; smoke: folders-only view omits leaf bodies; nodes with children still expandable; smoke: zoom: GET /tree?root=<node-id> re-renders with that node as root; bread-crumb back; smoke: expand returns children + one-level grandchild stubs (no N-requests-for-N-nodes); smoke: no-JS: stub is an <a href> to the same partial URL; full GET still correct; smoke: notes is first root: /tree?slug=<doc> walks the notes outline model; smoke: kernel children-of/subtree + lens prims deferred – BORGE-1785516998 / 1785516993*

THESIS: the one tree (one-tree.tree-model) is not just a storage trick – it is a browseable place. Since outline storage already makes a folder and a title THE SAME THING (a non-leaf node), the UI needs NO folder type: \"folder\" is a VIEW decision (has-children? + collapsed), not a storage one. One renderer walks docs, subtrees, and data.

PRIOR ART: Workflowy/Logseq zoom, Finder/Explorer lazy expansion, Smalltalk hierarchy browser (categories and classes are nodes at different depths). libs/outline/lib.pp ALREADY runs the swap idiom (hx-post /partials/notes-node-op, hx-target #outline, hx-swap outerHTML) – this section generalizes that pattern into a reusable component.

==========

TWO TREES – DO NOT CONFLATE ==========

1. STORE tree (tree_nodes): /sys/pkg/?, /data/<type>/<id>, branch lenses. Kernel addressing; recursive-CTE override walks. NOT the v1 UI surface. 2. CONTENT tree (user-facing outline hierarchy): notes outline nodes, docs ? nodes ? subnodes. The v1 browser walks (2). Store-tree roots come LATER (needs branch-lens-aware read prims; see DEFERRED).

==========

RENDER CONTRACT (pure Nopales, libs/tree) ==========

(tree/render root-id depth folders-only?) -> html string - Recursive walk, depth-capped. Depth x fanout bounds steps, so it fits the 100k sync budget (see multi-user.budgets) – self-limiting by construction. - Node row: twisty + title + child-count badge. - Beyond cap: collapsed STUB, not a leaf. - folders-only?: render only has-children? nodes as rows; leaves appear only as counts.

STUB EXPANSION (htmx, the ONLY async here): <li class=\"tb-stub\" hx-get=\"/partials/tree?root=<id>&depth=2\" hx-trigger=\"click\" hx-swap=\"outerHTML\">? Server returns the expanded SUBTREE FRAGMENT; the stub replaces itself. htmx supplies the client-side async illusion; each server request stays a small synchronous eval. No guest-visible fibers (see lisp-interpreter.language-roadmap: host owns concurrency).

ANTI-PATTERN (do not build): one request per node. Each expansion MUST return children + ONE-LEVEL grandchild stubs, so expansion is instant and request count stays O(user clicks), not O(tree size).

PROGRESSIVE ENHANCEMENT: the stub's hx-get URL is also a plain <a href>; without JS it full-page GETs the same fragment URL (wants_fragment distinguishes). Same discipline as notes.hypermedia-ui.

ZOOM (= \"load this title in the full window\"): GET /tree?root=<node-id> – re-render with the focus node as root. Zoom is ONE parameter, no new mechanism: same renderer, different root. Breadcrumb of ancestors for zoom-out. Zoom is what turns \"folder\" and \"title\" into the same thing from the user's side too.

OOB PATCHING (when zoom/expand affects two panes): hx-swap-oob updates tree panel + content pane in one response. Defer multi-pane until notes dogfood shows the need; v1 single-pane is fine.

==========

WHAT IS NOT HERE (kernel stays still) ==========

- NO new interpreter prims for v1. html/tag, request-param, recursion, and the notes outline model suffice. - NO change to eval budget tiers: depth-cap keeps every request small; that IS the budget story. - Kernel help only WHEN trees get big: children-of / subtree prims over tree_nodes using the materialized path index (one-tree.tree-model PINNED: indexed lookup, not tree walk). Optimization, not v1.

ROOTS ROADMAP (one component, many roots): v1 notes outline (slug root) – dogfood, ships first v1.1 data rows: /tree?root=data:todo (typed-node leaves link into /data/<type>/<id>) v2 store tree: /tree?root=/sys/pkg/notes – needs lens-aware read prims; this is when tree-browser becomes the Smalltalk-style INSPECTOR face for the whole habitat (code ASTs, types, routes as subtrees)

RELATION TO OTHER SECTIONS: - image.system-browser: /bindings stays the code hub; tree-browser is content-first. Shared callers/callees spine untouched. - notes.hypermedia-ui: notes keeps its dedicated outline page; tree-browser is the GENERAL component it can later delegate to. No notes rewrite in v1. - structural-editing.op-protocol: when tree ops (refile/move) land in the browser, they MUST go through the shared tree-op protocol, not ad-hoc routes.

SUCCESS CRITERIA: - A user can browse a deep notes outline without a full page load per level. - \"Folder\" required no schema/type – only a view bit. - Zero kernel changes for the v1 notes root. - Every tree interaction fits the 100k sync budget; no background tier needed for browsing.

data-browser

Generic OCaml system UI for full CRUD over shared data_rows by type

[status: implemented]

[verify] build: dune build image; smoke: GET /data lists types with row counts; shared-data banner present; smoke: GET /data/<type> lists rows (default limit 50) with declared columns; smoke: GET /data/<type>?offset=50

pages next slice; smoke: GET /data/<type>/new shows create form from type_defs fields; smoke: POST /data/<type> creates row from form fields; 303 to detail; smoke: GET /data/<type>/<id> shows fields + edit form + delete; smoke: POST /data/<type>/<id> updates fields; 303/reload detail; smoke: POST /data/<type>/<id>/delete deletes row; 303 to type list; smoke: GET /types links each type to /data/<type>; smoke: GET /inspect/row/<type>/<id> redirects or aliases to /data/<type>/<id>; smoke: type with no type_defs fields still allows raw JSON create/edit; smoke: extra jsonb keys not in type_defs still shown on detail

WHY: /types shows declared shapes; /inspect/row dumps one row as pretty JSON. Neither is a place to live in the object store. Product packages (todo, notes, bible) have purpose-built UIs, but agents and humans still need a generic browser when there is no package UI, the package is broken, or the type is ad-hoc.

LAYERING (locked): - OCaml system pages in image/lib/http.ml (+ thin helpers in crud.ml / store.ml as needed). Host/gate tier – same as /types, /bindings, /inspect, /edit. - NOT a Nopales package. Product shell stays Nopales; this is emergency/generic substrate UI that must work with empty or broken gardens. - Mutations call existing Crud/Store SQL paths. Do not invent a second data access layer. - data_rows are SHARED across branches (no branch_id). UI must say so on every list/detail surface.

==========

ROUTES (full CRUD) ==========

Index (Read types + counts): GET /data - Table: type_name | declared fields | row count | link - Types = visible type_defs on current branch (Store.list_types) - Also surface type_names that have data_rows but no type_def (orphan data) so nothing is invisible - Banner: "Data rows are shared across branches" - Link back/forward with /types (types = schema index; /data = instances)

List (Read many): GET /data/<type> - Query: limit (default 50, max 200), offset (default 0) - Columns: id (short) + each declared field (truncated) + updated_at if cheap; link id ? detail - Pagination: Prev / Next when more rows exist - Actions: [New row] ? /data/<type>/new - Empty state: no rows yet + create CTA - Order: created_at ASC (match Crud.list) unless a later plan picks something smarter

Create: GET /data/<type>/new - Form: one input per declared field (text inputs v1) - If type has ZERO declared fields OR user wants freeform: textarea name=fields_json for raw JSON object - POST target: /data/<type> POST /data/<type> - Content-Type application/x-www-form-urlencoded - Build fields jsonb from declared field names present in the form; if fields_json posted and non-empty, that wins (power path) - author = Session username - On success: 303 Location: /data/<type>/<id> - On failure: 400 with error card + back link

Detail / Update / Delete: GET /data/<type>/<id> - Show all fields (declared first, then extra jsonb keys not in type_defs – do not hide unknowns) - Edit form prefilled (same field rules as create) - Delete form POST /data/<type>/<id>/delete (confirm via button label; no JS required) - Meta: id, type, author, created_at, updated_at when available - Links: back to type list, /types, REPL hint POST /data/<type>/<id> - Update fields (full replace of fields jsonb from form, same merge rules as create). Uses Crud.update path. - 303 back to detail on success POST /data/<type>/<id>/delete - Crud.delete; 303 to /data/<type> - Missing row: 404 card, not silent success

URL parsing notes: - type_name may contain only safe path segments in v1 (letters, digits, hyphen, underscore). Reject or 400 weird paths; no path-param injection into SQL (always parameterized – already true in Crud). - id is UUID string. - Prefer longest-match dispatch: /data/<type>/new /data/<type>/<id>/delete /data/<type>/<id> /data/<type> /data so "new" is never treated as an id.

==========

FIELD / JSON RULES (v1) ==========
- type_defs.fields is a JSON array of field-name strings (no field types yet – object-model.type-system deferred). - Form values are strings; store as JSON strings in the fields object. Do NOT guess number/bool coercion in v1 unless the existing value is already that shape and a later polish wants round-trip fidelity. - Raw JSON textarea: must parse as a JSON object; reject arrays/scalars with a clear error. - Extra keys on existing rows: display on detail; include in edit form as additional inputs OR preserve via hidden merge so update does not drop them. REQUIRED: updates must not silently strip unknown keys.

==========
RELATIONSHIP TO EXISTING PAGES ==========
- /types: keep as type_defs browser; each row links to /data/<type>. Optional: add row count column on /types once count helper exists (nice-to-have, not blocking). - /inspect/row/<type>/<id>: redirect (302/303) to /data/<type>/<id> OR thin alias rendering the same page. Prefer redirect so one canonical URL. - Product apps unchanged: /todos, /notes, /bible stay Nopales. - Nav: emergency/system links may include Data ? /data; product ui/menu MAY add Data later (Nopales choice, not required for make).

==========
STORE / CRUD HELPERS (implementation discretion) ==========
Add only what the UI needs if missing: - count rows by type_name (for index) - list type_names present in data_rows (orphans) - list rows with limit+offset (Crud.list today is limit-only; extend with offset or a store helper) - optional: fetch author/created_at/updated_at for detail Keep Nopales prim surface stable unless a prim gap blocks the browser (prefer OCaml-only helpers for offset/count).

==========
AUTH / SAFETY ==========
- Same session gate as other authenticated system pages. - Not admin-only in v1 (any logged-in user can browse/mutate shared data – matches Nopales CRUD prims today). - No CSRF token infrastructure in v1 (status quo for other POSTs); cookie session + same-site assumptions hold. - Never interpolate type/id into SQL strings.

==========
NON-GOALS (v1) ==========
- Field type system / validation beyond JSON object shape - Full-text search UI (find-text exists for Nopales; wire later) - Inline htmx row edit without full page (optional polish) - Bulk delete / CSV import-export - Per-row branch scoping (data is shared by design) - Nopales-redefinable browser UI - SPA / client-side routing

==========
ACCEPTANCE NARRATIVE ==========
Human can: open /data ? pick a type ? page through rows ? open a row ? edit a field ? save ? create a new row ? delete a row ? land back on the type list. All without REPL. Agent can use the same URLs or keep using CRUD prims.

page-timing

Per-route page render timing

[status: implemented]

> *[verify] build: dune build image; smoke: GET /timing lists each user route with count + avg db_ms/total_ms (empty/zero-safe when no samples); smoke: GET /<route> twice then GET /timing shows count=2 for that route; smoke: each handled route response carries X-Timing-Ms header; smoke: GET /timing sorts slowest-first by max_ms; shows db_vs_total split; smoke: /timing 404-safe / zero-safe with no routes ever handled*

WHY: Product pages are Nopales route handlers, and there is no first-class answer to "how long does each page take to load?". Hand-wrapping every handler with (now-ms) deltas is repetitive and easy to forget. A single choke point already exists – every handled user route flows through Session.call_handler – so we can measure universally with no per-package work.

WHAT WE MEASURE (server-side render time, not client perceived): - total_ms: wall time of the handler call (handler body only, not HTTP write-back) - db_ms: accumulated time inside DB-touching CRUD prims during that handler - db_hits: count of DB-touching prim calls during that handler - at: wall clock (epoch ms) the sample was taken - route: the handler name/path This is the cost to PRODUCE the page in the image. It is NOT full end-to-end (network/TLS/browser paint). End-to-end measurement is explicitly out of scope here (needs client Navigation-Timing POST; deferred).

MEASUREMENT 1. Budget accumulation (lisp/lib/eval.ml): budget currently tracks prim_calls + db_hits but not DB wall time. Add a mutable db_ms : float to the budget struct (init 0.0). prims_db.ml register_crud already computes elapsed_ms per prim; change ‘ignore _elapsed_ms‘ to accumulate it into the budget via a helper (mirroring note_db_hit, e.g. note_db_ms on current_budget). This is the "future function_stat.db_time_ms aggregation" the existing comment already earmarks. 2. Choke-point capture (image/lib/http.ml): every call to Session.call_handler (both dispatch sites: the launch_user_route_or path and the /data-adjacent user route path) is wrapped so BEFORE the call we read the current eval budget's db_ms/db_hits, and AFTER we read now-ms and the ending budget. One clean helper Timing.measure f that runs the handler, then records a sample { route; total_ms; db_ms; db_hits; at }. A handler that fails still records its sample (the work happened); the failure path is unchanged otherwise. 3. Sample store (image/lib: new timing.ml, mirroring the data-browser pattern): a small in-process sample ring per handler name. Keep a bounded window (e.g. last 1000 samples overall, or last 100 per route) to avoid unbounded growth. Optionally mirror into data_rows as type ‘page_timing‘ for cross-restart persistence + Nopales querying – a nice-to-have, no migration required; in-process ring satisfies /timing + header with none. Persistence deferred unless we want history across restarts.

SURFACE 1. /timing system page (OCaml dispatch entry mirroring /branches, /data): table per route name – samples | last_ms | avg_ms | max_ms | db_ms | db_hits | db_share (% of avg total). Sorted by max_ms descending (slowest first). Empty/zero-safe when there are no samples. 2. X-Timing-Ms response header: attach total_ms to every handled-route response (in respond_handler_value / the response path) so a browser or curl can see the number per request without opening /timing.

SCOPE / NON-GOALS - Server-side render time ONLY. No client Navigation-Timing end-to-end measurement (deferred, not this one). - No new schema in v1 (in-process ring); data_rows ‘page_timing‘ persistence is optional/deferred. - kernel paths (/data /admin /api/eval ?) are NOT measured: only user route handlers (the product pages people ask "how long does this page take" about). - /timing is a private system page (auth-required, like /branches), not a public route.

ACCEPTANCE NARRATIVE Human: open a few pages, then GET /timing ? see each page with count + slowest-first order + DB share; curl -i a page ? see X-Timing-Ms on the response. No per-package code needed to be measured.

product-theme

Product presentation as Nopales data: css/* sheet compiler, ui/theme + ui/css, emergency CSS only in OCaml

[status: implemented]

[verify] build: dune build image; smoke: (css/render (css/sheet (body margin 0 background "#d4d0c8"))) contains body{ and margin:0; smoke: (function-source "ui/theme") or (function-source "ui/css") non-empty after seed; smoke: (export-package "ui") contains define ui/theme or define ui/css; smoke: GET / authenticated response <style> matches rendered (ui/css) distinctive marker, not only emergency; smoke: personal (define theme/bg ?) or ui/theme pin changes next full-page CSS; checkout main restores; smoke: (define ui/css (lambda () (error "boom"))) still serves page with emergency CSS + labeled failure; smoke: htmx fragment / partial responses do not re-inject full stylesheet; smoke: login page stays pre-session OCaml mini stylesheet (no Nopales theme required)

WHY: Today almost all product CSS is a giant inline string in image/lib/http.ml html_response – inaccessible from the garden, unversioned as Nopales, dual to the product-shell rule that already moved nav/home into libs/ui. Widget packages emit class names that only work because that OCaml blob knows them. That is a black box.

DECISION (Path B – honest Nopales): Presentation is Nopales data, same substrate as shell and apps. A small css/* compiler (sibling of html/* in std) turns sheets into CSS text. ui/theme holds the sheet; ui/css is a thin (lambda () (css/render ui/theme)) adapter for session refresh. Humans and agents edit sexps / tokens via Functions – not escaped CSS inside a lambda, and not a second language document as the native store.

REJECTED: - Path hybrid: (define ui/css (lambda () "body{?}")) as the primary form (CSS-in-string wearing Nopales hat) - Path A as native store: bare CSS document / data_rows stylesheet type as the only theme representation (allowed later as import/export sugar, not authority) - Static /assets/*.css as the product theme path - Silent full OCaml design-system twin beside Nopales theme - Per-request uncached re-eval of ui/css without shell refresh discipline (mirror ui/nav caching lessons) - TIME-BASED STALENESS: a TTL on the shell cache serves wrong CSS for up to a minute for no correctness reason (supersedes the earlier 60s TTL; invalidation is now dependency-tracked, below) - GENERAL USER-FUNCTION MEMOIZATION: Nopales functions may read DB or request state mid-eval, so same-args-does-not- imply-same-output; no purity story in the type system yet. Caching applies to the zero-arg shell bindings ONLY.

==========

css/* (libs/std/lib.pp – seed with std, before ui) ==========

Parallel to html/*: structure in Nopales, render to string.

Core bindings (v1 – keep small): (css/sheet rule?) ? sheet value (list-tagged data) (css/use other) ? splice/concat another sheet (css/raw "?") ? escape hatch for rare CSS (css/render sheet) ? CSS string

Rule encoding (selectors as strings; declarations data): - (selector prop val prop val?) selector may be a symbol (body, header) or string (".card", "nav a", "details.ol-node > .ol-sum") - ((sel1 sel2 ?) prop val ?) multi-selector group ? "sel1, sel2 { ? }" - ((media "max-width:40rem") rule?) nested @media block - Values: number | string | symbol (symbol resolves as token binding if bound, else as CSS ident) - Do NOT invent a full selector AST or px-type system in v1. Selectors stay strings/symbols; props are alternating key/value data.

Emit rules (css/render): - prop symbols/strings ? CSS property names (margin ? margin; font-family ? font-family; accept underscore?hyphen if needed) - numbers: print without quotes (0, 1.4) - strings: as CSS values (caller includes units) - stable, readable output; order preserved - css/raw segments pass through unchanged

Composition: (css/sheet (css/use ui/theme-base) (css/use todo/theme) ; optional later (body background theme/bg))

html/* analogy docstring in std: "sheets ? css/render; no raw rule soup in apps" (css/raw remains the deliberate guest).

==========

ui/theme + tokens + ui/css (libs/ui/lib.pp) ==========

Required product-shell bindings (with ui/menu ui/nav ui/home):

theme/* tokens (examples; exact set is implementation discretion so long as the classic-Mac look ports): theme/bg theme/ink theme/panel theme/chrome theme/link theme/err theme/ok ?

ui/theme A css/sheet (or zero-arg lambda returning a sheet) expressing the full product design system currently hard-coded in http.ml (body, header, nav, cards, tables, forms, outline, todos, calendar, bible, weather class contracts, mobile breakpoints). Prefer data + theme/* tokens over magic hex literals so personal gardens can (define theme/bg ?).

ui/css (lambda () (css/render ui/theme)) or (lambda () (css/render (ui/theme))) if theme is a thunk. Adapter only – not where humans paste CSS.

Class contract: - Shared classes used by multiple packages live in ui/theme (todo-box, ol-node, cal-grid, card, ?). - Package-local extras may keep a small (html/tag "style" ?) or later (todo/theme) composed via css/use – weather/style today is acceptable local. - Moving the monolith out of OCaml does not require splitting packages on day one.

Seed: seed_stdlib still load-library std then weather then ui ? – css/* must exist before ui/theme renders. (std before ui already holds.)

==========

Session cache + HTTP inject ==========

Mirror ui/nav:

Session.t gains mutable ui_css : string option Some s ? product stylesheet text None ? use emergency_css

refresh_ui_shell (rename meaning: refresh shell): existing (ui/nav) path unchanged plus eval (ui/css) ? string | raw-html-as-text ? cache; on miss/throw ? ui_css <- None

Invalidate / refresh – DEPENDENCY-TRACKED, not guessed: At each shell render, the kernel records the set of bindings (ui/css) / (ui/nav) actually resolve during eval (env read-set, e.g. theme/vars, ui/theme, ui/theme-nav, notes/tree-css, theme/bg, ...). A define busts the cache IFF the defined name is in that recorded set – theme/extra busts (it's read), todo/foo doesn't (it isn't). No TTL, no name-prefix heuristic: the cache is fresh forever until a real dependency changes. checkout / branch switch still busts unconditionally (no stale other-branch CSS; read-sets are per-branch). load-library still refreshes at end (eval_form path) since bulk redefines legitimately change the read-set. Shell evals cause ZERO DB WRITES: budget-stats persistence (function_stat) MUST skip shell labels (ui/css ui/nav ui/home ui/theme) – the shell refresh runs in the request gate and must not turn into a serialized Crud.create under the pgx sequencer.

html_response gains ~css parameter (or reads session): HTML.style [] "%s" css Authenticated full pages pass Session.ui_css | emergency_css.

fragment_response / HX partials: do NOT re-inject the full stylesheet (page already has it). Same spirit as chrome-skip for fragments.

login_form_page (pre-session): keeps a tiny OCaml-only mini stylesheet. Must not require Nopales theme. Not the product design system.

==========

Emergency CSS (OCaml kernel – labeled, minimal) ==========

emergency_css is host recovery, not a product twin: - monospace / high-contrast / readable - styles emergency-banner, emergency-nav, basic links - MUST look broken or sparse on purpose if product theme fails (parallel emergency-nav rule) - After make: the 200-line classic-Mac blob MUST leave html_response; only emergency_css (+ login mini) remain as OCaml string stylesheets.

Optional: when falling back, surface a one-line labeled banner in chrome ("product theme unbound – emergency css") consistent with emergency-nav labeling.

==========

Authoring / multi-user / sync ==========

- Edit via Functions ? package ui ? ui/theme | theme/* (same mobile drill-

down as other shell bindings). - Personal pin: (define theme/bg "#e8f0e8") or full ui/theme override on user/personal; inherits main when no pin. - (export-package "ui") includes theme tokens + ui/theme + ui/css; pp_sync pull/push works unchanged. - Agents: pp_eval define/function-source – no new REST. - v1 non-goals: /theme textarea page, CSS file import pipeline, Tailwind/Sass build step, data_rows theme type as authority, per-user CSS without branch pins.

==========

MAKE ORDER ==========

1. css/* in libs/std + unit/smoke sexp?CSS 2. Transcribe http.ml product blob ? ui/theme (+ tokens) in libs/ui; define ui/css adapter 3. Session ui_css + refresh_ui_shell extension 4. html_response ~css inject from session 5. Shrink OCaml to emergency_css (+ login mini) 6. Smokes: seed marker, personal override, failure fallback, fragment no-reinject 7. borge drift / resolve HTTP robustness verification

STATUS: planned until make lands and silent full OCaml product CSS twin is gone.

themes

Installable, switchable named themes: a theme is a libs/<name> package of theme/* tokens; switch = reseed into your garden; registry is a Nopales list, no kernel tables

[status: implemented]

> *[verify] build: dune build image; smoke: (theme/available) lists the default token package with display name + description; smoke: (theme/current) reports the active theme on the current branch; fresh garden reports the default; smoke: (theme/set "<name>") switches the garden: next full-page <style> shows the new theme's distinctive token values; smoke: switch replaces ALL pinned theme/* tokens on the current branch (no stale-token bleed from the previous theme); smoke: switch leaves a personal ui/theme sheet override intact; tokens flow through it; smoke: other users' themes unchanged by my switch; checkout main shows system defaults; smoke: (theme/set "<unregistered>") is a labeled error; active theme unchanged; smoke: GET /settings/theme lists available themes, marks current, applies switch without manual source editing*

WHY: product-theme made presentation Nopales data (css/* + ui/theme + theme/* tokens) and branch pins already make token override possible – but only by hand-editing twelve theme/* defines through Functions. Nobody will do that. The missing product surface is: named themes you can install and switch between as one act, per user, without touching source.

==========

MODEL – a theme is a PACKAGE, not a row or a table ==========

A theme is a Nopales package libs/<theme>/lib.pp whose entire job is defining theme/* token bindings (the existing libs/theme token set: theme/bg theme/ink theme/panel theme/chrome theme/link theme/visited theme/muted theme/err theme/ok theme/border theme/row-alt theme/shadow). A theme package MAY also carry a full ui/theme sheet override, but v1 themes are token-only: the shared ui/theme sheet reads tokens, so swapping tokens reskins the whole product.

Nothing new in the kernel. Install = the package exists in the vault (or was defined live). Switch = the theme's token defines land on YOUR branch as pins (standard branching.package-updates semantics: garden pins shadow system/main; checkout main restores system defaults). Per-user themes fall out of per-user branches – no per-user preference table, no cookie, no data_rows setting (all REJECTED: preference state must live in the same branch/version/promote substrate as every other user choice).

==========
REGISTRY – Nopales data, not a kernel catalog ==========
The registry is itself a binding in the theme package:

theme/available (lambda ()) -> list of alist-ish dicts, one per installed theme: name, display, description, (optional) author. Source of truth for the picker. Default seed ships the system token package as the one entry.

theme/current (lambda ()) -> name string. v1 derivation: the garden's own binding theme/name if a switch pinned it, else the system default name. (Honest label, not a guess from token values.)

theme/set NAME -> switches the CURRENT branch to theme NAME: looks NAME up in theme/available, evaluates the named theme package's token defines as pins on the current branch (same code path as reseed-package, narrowed to theme/* + theme/name), then triggers the existing ui/css shell refresh so the next full-page response carries the new sheet. Unknown name -> labeled error, no partial switch. Switch is ALL-OR-NOTHING across the token set: a theme that defines only some tokens inherits the ACTIVE theme's remainder at switch time (pins are written for every token), never whatever random pin was already on the branch – no stale-token bleed between switches.

Installing a new theme = adding libs/<name>/lib.pp to the vault plus one entry in theme/available (promote to system/main to share; keep on personal to hoard). pp-sync snapshot/pull treats themes like any package. Agents: pp_eval (theme/set \"phosphor\") – no new REST.

==========
PICKER – /settings/theme (Nopales route, libs/ui) ==========
A small settings page (private route, same discipline as other product routes): lists theme/available entries as cards (display name, description, swatch row rendered inline from each entry's token values), marks theme/current, and POSTs a switch via htmx to a partial that calls (theme/set name) and returns the updated card list; the next FULL page load shows the new theme (fragment responses never re-inject the stylesheet – existing product-theme rule). No-JS fallback: plain form POST + redirect to /settings/theme.

INTERACTION WITH PERSONAL ui/theme PINS: A garden that pinned its own ui/theme sheet keeps it – theme/set only touches theme/* + theme/name. Tokens flow through whatever sheet is visible. A theme that ships a sheet override installs it as a pin too, and switching AWAY removes that pin (restores inherited ui/theme). Sheet-carrying themes are flagged in their registry entry so the picker can say so.

REJECTED: - data_rows theme/preference entities as authority (shadow state outside branch semantics – the exact thing product-theme rejected for CSS itself) - Static /assets/<theme>.css files (rejected in product-theme already; stays rejected) - Kernel theme catalog table / migration (catalog is Nopales data on branches, versioned like code) - Live preview without switch (v1: swatch row only; real preview is switch-then-look, checkout restores) - Theme inheritance/chaining beyond token fallback - Per-page or per-app themes (one active theme per branch, period)

RELATION TO product-theme: This section changes NOTHING about css/*, ui/theme, ui/css, session caching, or emergency CSS. It adds the registry + switch + picker on top. The deferred "Dedicated /theme form UI" line in the htmx-interface overview resolves to THIS section's picker.

MAKE ORDER: 1. theme/name + theme/available + theme/current + theme/set in libs/theme (switch via narrowed reseed path; all-or-nothing token pins) 2. /settings/theme + /partials/theme-switch in libs/ui 3. Smokes per verify stanzas

packages

In-branch prefix packages for organizing functions (not branch ownership)

[status: verified]

[verify] build: dune build lisp image; smoke: reader accepts symbol todo/list as Var; smoke: (define todo/list (lambda () 1)) then (todo/list) => 1; smoke: todo/list and calendar/list coexist on same branch; smoke: /bindings groups todo/list under package todo; smoke: (load-string "(define pack/x (lambda () 7))") then (pack/x) => 7; smoke: (load-library "todo") installs /todos and todo/page; smoke: libs/todo/lib.pp is source of truth; no OCaml string recipe body; smoke: (load-library "bible") defines bible/verse bible/chapter bible/page + /bible; smoke: after seed: (bible/verse "John" 3 16) returns BSB text containing God/world; smoke: GET /bible?book=John&chapter=3 lists verse text; smoke: GET /bible?ref=BSB:John:3:16 focuses that verse; smoke: seed script idempotent: second run does not duplicate refs; smoke: (function-source "ui/nav") returns non-empty lambda/list source; smoke: (export-package "ui") contains define ui/nav; smoke: (function-source "ui/css") or ui/theme non-empty after seed; export-package ui includes theme; smoke: scripts/pp_sync.py status ui exits 0 with env set

PROBLEM: flat binding names collide once many tools live on one branch (todo list vs calendar list).

MODEL (v1): - Package is a PREFIX of a function name, separator '/'. - Full binding key remains (branch_id, name) with name the full string "todo/list". No packages table required. - package_of_name "todo/list" = "todo"; local = "list". - package_of_name "todo/ui/row" = "todo/ui"; local = "row". - package_of_name "hi-page" = "" (root). - Call and define use the full name: (todo/list), (define todo/list ?). - Pages still route to full handler names: (route "/todos" "todo/list").

ORTHOGONAL TO BRANCHES: - Branch = version + ownership (multi-user.namespaces). - Package = organization inside one branch. - UI must never call a branch a "package" or vice versa.

NOT IN V1: - (require 'todo) / import aliases - Private package symbols - Automatic rename of call sites - Cross-branch qualified names (system/main:greet) – still multi-user.namespaces deferred item - Privileged global package registry

AI / EDITOR: - Prefer namespaced names for new tools (todo/list not list). - AI propose form: optional package field + local name, or single name field that may contain /. - Function editor URL remains /edit/<full-name> with / in the path segment (encode if needed) or query name=; implementation may use /edit?name=todo/list if path routing is awkward – pick one and document in agent note.

LIBRARY LOADER (implemented): Same Nopales source text in git repos and in the live system. OCaml provides only a pure loader (no app string blobs).

On-disk layout (shipped in release tarball as libs/): libs/std/lib.pp – list/*, html/*, css/*, menu/*, data/* libs/todo/lib.pp – deftype + todo/* + routes + partials libs/hello/lib.pp – optional hi-page recipe as Nopales libs/bible/lib.pp – bible/* API + /bible (canon is data_rows) libs/ui/lib.pp – ui/menu ui/nav ui/home ui/theme ui/css libs/ai/lib.pp – ai/ask ai/do wrappers libs/chat/lib.pp – multi-user lobby+DM (multi-user.chat) libs/plan/lib.pp – Unix .plan status (multi-user.user-plan)

Loader API: (load-string SRC) read_all forms; eval_one each (define persists via existing intercept; route/ deftype via prims). Stop on first error. (load-library NAME) load file libs/<NAME>/lib.pp relative to PP_LIBS_DIR or ./libs (release CWD).

Boot / recipes: seed_stdlib ? load-library "std" then "ui" then "ai" then (when present) "chat" and "plan" for multi-user social recipe_todo ? load-library "todo" recipe_hello ? load-library "hello" recipe_bible ? load-library "bible" (types+fns+route ONLY; does NOT bulk-insert verses)

Release build MUST copy libs/ into the tarball.

==========

LIBRARY SYNC CLI ==========

PROBLEM: live function_defs drift from git libs/*.pp when users or AI edit in

the browser. Push of kernel does NOT overwrite DB. Developers need explicit sync so laptop git matches language state.

MODEL: - Local truth for recipes: libs/<pkg>/lib.pp - Live truth for running language: function_defs on current branch - CLI (scripts/pp_sync.py) uses authenticated /api/eval (pp_eval) and export prims – no direct Postgres from laptop required.

EXPORT PRIMS (image): (function-source "ui/menu") ? VStr pretty body only (lambda/list/?) for define (export-package "ui") ? VStr multi-form lib.pp text: ;; package ui – exported from live branch (define ui/menu ?) (define ui/nav ?) ? sorted by name Root package "" / "(root)" exports unprefixed names only. Kernel prims are never exported. (package-bindings "ui") ? VList of VStr full names in that package on this branch

CLI: scripts/pp_sync.py status [pkg] Compare libs/<pkg>/lib.pp define names vs live package-bindings. Report only-local / only-live / both (optional body hash later). scripts/pp_sync.py pull <pkg> Write libs/<pkg>/lib.pp from (export-package pkg). Creates directory if missing. Does not auto-commit git. scripts/pp_sync.py push <pkg> (load-library pkg) via pp_eval – file ? live (upsert defines). scripts/pp_sync.py pull-all Pull every non-empty package that exists live or under libs/.

ENV: same as pp_eval (PP_URL PP_USER PP_PASS, ~/.config/pricklypear/env).

OUT OF SCOPE v1: - data_rows sync (todos/bible text) - automatic merge conflict resolution - function revision history table - Doltgres

VERIFY: - (export-package "ui") contains define ui/nav - pull ui rewrites libs/ui/lib.pp; status shows both - push ui after edit reloads live

==========

BIBLE LIBRARY (BSB dogfood) ==========

GOAL: read BSB from the live image via Nopales + /bible page. Canon lives in Postgres data_rows (shared across branches). Functions live in package bible/ via load-library.

DECISIONS: - Translation v1: BSB only (attribution in page footer / lib header). - Storage: data_rows, NOT giant define / NOT .sexp corpus at runtime. - Ref string (human + share + seed key): BSB:<Abbrev>:<chapter>:<verse> e.g. BSB:John:3:16 BSB:1Cor:13:4 BSB:Ps:23:1 - Book token in ref = stable ABBREV (no spaces). Display name on bible-book rows separately. - data_rows.id stays UUID PK (migration 003). Ref is NOT the PK. fields.ref holds BSB:Abbrev:C:V uniquely. - Optional seed optimization: deterministic UUID v5 from ref so tooling can recompute id; Nopales users still call bible/* not raw get-by-uuid. Either UUID-default or v5 is fine if ref unique.

TYPES (deftype from libs/bible/lib.pp): bible-book: name, abbrev, ordinal, testament (~66 rows; menus / prev-next / name?abbrev map) bible-verse: translation, book, abbrev, chapter, verse, text, ref chapter/verse as JSON numbers or digit strings – pick one in seed and stick to it (prefer numbers in JSON). ref = "BSB:John:3:16" always.

LOOKUP (needs more than blind rows scan): (get 'bible-verse uuid) alone is not enough for humans. Provide ONE of (make picks; prefer smallest): A. Crud/store helper + prim: (find-row 'bible-verse "ref" "BSB:John:3:16") (find-rows 'bible-verse "abbrev" "John" ["chapter" 3]) ; chapter list B. Dedicated thin prims bible-get-ref / bible-chapter (worse – special-case kernel). Prefer A (general field find). GIN on fields already exists; add UNIQUE index for idempotent seed: CREATE UNIQUE INDEX data_rows_bible_ref_uidx ON data_rows ((fields->>'ref')) WHERE type_name = 'bible-verse'; (migration 004 additive – only if make needs it).

LISP API (libs/bible/lib.pp): (bible/ref book chapter verse) -> "BSB:John:3:16" book may be name or abbrev; normalize via bible-book catalog (bible/verse book chapter verse) -> text VStr | VNil (bible/verse-row book chapter verse) -> fields JSON VStr | VNil (bible/chapter book chapter) -> VList of verse fields/sexps (bible/books) -> list from bible-book rows ordered by ordinal (bible/search q) -> VList hits (v1: SQL ILIKE or limited scan; may be prim-

backed; OK if slow/limited first cut) (bible/page) -> GET renders chapter; POST goto redirects (route "/bible" "bible/page")

URL SURFACE (exact path /bible only – no path params in v1): GET /bible GET /bible?book=John&chapter=3 GET /bible?book=John&chapter=3&verse=16 GET /bible?ref=BSB:John:3:16 GET /bible?q=love (optional search) POST /bible book= & chapter= & verse= -> redirect to GET with query (PRG) Do NOT require colon-ids in path segments. ref= query is the shareable explicit form (URL-ok in query).

ABBREV TABLE (seed must use these tokens in ref): Gen Exod Lev Num Deut Josh Judg Ruth 1Sam 2Sam 1Kgs 2Kgs 1Chr 2Chr Ezra Neh Esth Job Ps Prov Eccl Song Isa Jer Lam Ezek Dan Hos Joel Amos Obad Jonah Mic Nah Hab Zeph Hag Zech Mal Matt Mark Luke John Acts Rom 1Cor 2Cor Gal Eph Phil Col 1Thess 2Thess 1Tim 2Tim Titus Phlm Heb Jas 1Pet 2Pet 1John 2John 3John Jude Rev (If vendor data uses different codes, map in seed script; document mapping next to script. Display names full English.)

BULK SEED SCRIPT (required – not HTTP recipe): Path: scripts/seed_bible_bsb.py (or .ksh driver + py) Inputs: - BSB source file(s) (JSON/CSV/USFM – vendor format; script owns parsing; do not commit illegal redistrib if license forbids; prefer redistributable BSB dump + NOTICE) - PG connect: same as dev (PP_DB_* / /tmp:5433) Behavior: 1. Ensure type_defs exist OR document that load-library bible must run first (deftype). Prefer: script can INSERT type_defs rows for bible-book/bible-verse if missing (author system). 2. Upsert ~66 bible-book rows (stable id or fields.abbrev unique). 3. Upsert all bible-verse rows: fields.ref unique; ON CONFLICT DO UPDATE text (requires unique index on ref – migration or special conflict target). 4. Idempotent: second run = no duplicate verses. 5. Print counts: books N, verses M. NOT in recipe_bible HTTP handler (too slow / wrong layer).

INSTALL STORY: A. scripts/seed_bible_bsb.py # once per DB B. (load-library "bible") # or POST /api/recipe/bible C. (bible/verse "John" 3 16) D. open /bible?book=John&chapter=3

HOME UI (optional make): Card "Install bible library" -> POST /api/recipe/bible Note on card: "Run seed script for BSB text if empty".

RENDER: v1: simple HTML chapter list + optional verse highlight is OK. Optional: ?view=sexp returns chapter as Nopales list via bare value. Attribution footer: Berean Standard Bible / license line.

PERSONAL LAYER: notes/bookmarks are NOT bible/dabar – a separate module owns them. Canon text stays shared.

MAKE CHECKLIST: 1. Abbrev map + seed script I/O against sample or full BSB 2. migration 004 unique index on verse ref (if upsert needs it) 3. find-row / find-rows (or equivalent) for ref + chapter 4. libs/bible/lib.pp API + route /bible 5. recipe_bible + optional home card 6. Smokes in verify stanza above

OUT OF SCOPE V1: - multi-translation UI - path params /bible/John/3/16 (map later from same ref) - full-text search engine - audio / cross-refs apparatus - pure-sexp file corpus as runtime source of truth

DEFERRED (packages general): library_files table, git export, multi-file manifests, load from HTTP.

Kernel prims (OCaml) stay under (kernel) browser package. Majority of app code remains Nopales in .pp files.

browser-repl

Evaluate Nopales from the browser against the live session env

[status: implemented]

> *[verify] build: dune build image; smoke: POST /api/eval (+ 20 22) => 42; define + call persists*

- GET /repl – input + Eval button - POST /api/eval {expr} -> {ok,value|error} - Same Session.eval_one path as SSH REPL - Not a second interpreter

function-editor

Load, edit, and save a binding's AST from the UI
[status: implemented]

> – *agent note:* 2026-08-02: textarea editor + missing_editor_card + /api/edit/create; live GET /edit/no-such-fn-xyz shows does not exist + Create.
>
> *[verify] build: dune build image; smoke: POST /edit/editor-greet save -> callable via /api/eval; smoke: GET /edit/missing-fn shows create affordance, not silent TODO only; smoke: GET /edit/hello after hello-recipe: offers Edit hi-page + Create hello; smoke: htmx POST create missing fn returns editor partial; save works*

URL: /edit/<function-name> (function name, NEVER a route path as the object of edit – routes are pages).

v1 SHIPPED: plain textarea editor + save on current branch when the binding already exists.

REQUIRED NEXT – missing binding empty state: When GET /edit/<name> and no function_defs row for name:

1. Do NOT present only a raw TODO stub as if the user is already editing a real function.
2. Show an explicit card: Function "<name>" does not exist.
3. Route awareness: if a page route "/<name>" (or path equal to name) exists with handler H: There is a page /<name> that currently runs function H. Primary: [Edit H] -> /edit/H Secondary:[Create function <name>] Optional: [Create <name> and point /<name> at it] (define + route update; power move) If no such route: only Create (and link back to Functions).
4. Create-in-place via htmx (no full-page dead end): POST /api/edit/create {name} (and optional rebind_route) -> 200 HTML partial: the editor form for the new binding (same fields as GET /edit/<name> after create) hx-target=#editor-root hx-swap=innerHTML Default body stub: (lambda ()\n "TODO: implement <name>") After create, user saves via existing POST /edit/<name>.
5. Creating a function does not silently rename routes unless the user chose "point page at it".
6. Namespaced names (image.packages): - Accept full names with '/' (todo/list) in name field. - Create form MAY split package + local name and join with '/'. - URL routing for /edit/todo/list: either path segments after /edit/ joined back to todo/list, or /edit?name=todo%2Flist. Prefer path join if http router is easy; document choice.

DEFERRED: optimistic concurrency / stale-body reject. The structural AST tree-editor now lives in structural-editing.ast-structure-view (paths first; stable node storage still via branching.ast-nodes).

inspector

Inspect values, rows, ASTs, and page?function graphs live
[status: implemented]

> – *agent note:* 2026-08-02: /inspect/binding, callees/callers/holes/impact-queue prims; live (callees "ui/nav") ? ui/menu menu/bar; inspect page 200.
>
> *[verify] build: dune build image; smoke: GET /inspect/binding/<name> shows author + AST; smoke: GET /page/hello after recipe: handler hi-page listed; smoke: page inspector lists callees of handler when body calls other fns; smoke: (callees "ui/nav") returns list; (callers "ui/menu") non-empty when nav calls menu; smoke: after redefine of a callee, impact-queue lists dependents + routes; smoke: agent install_function result text names impact routes to smoke; smoke: binding pages show recursion/tail classes and observability badges when function-observability ships*

==========
SHIPPED (v1 inspector) ==========
- binding AST pretty+JSON; data row JSON pretty - Function observability (lisp-interpreter.function-observability): recursion/tail HTML classes, step/DB stats, big-O badge, on-demand (analyze 'fn) / POST /api/analyze/:name, peel. - GET /page/<path>: routing panel + STATIC callee graph (App heads that are Vars ? visible function_defs; depth cap 8; cycles skipped) - /bindings "used by": DIRECT handler?route only (not full transitive reverse graph – perf; deep graph stays on /page inspect). Implementation note 2026-07: used_by_pages_map is handler-only to avoid N*depth PG lookups on every /bindings load.

==========
CHANGE IMPACT (planned) – reverse deps ? test obligations ==========
Motivation (language + OS): Live define/install is not proof the habitat still works. We already pay for a static call graph on inspect. Use it like Smalltalk senders + Nx "affected": when F changes, flag who depends on F and which routes to re-smoke.

Industry analogues (do not overbuild): IDE Find References, Bazel/Nx affected, Smalltalk senders, coverage-guided test selection. We want the live-image version, not a full CI product.

———- A. STATIC GRAPH API (shared kernel) ———- Reuse collect_var_apps / page_function_graph style analysis.

Forward (already): callees(name) ? user functions F statically calls (direct App Var heads that resolve to function_defs; optional ~depth default 1 for prims, 8 for inspect)

Reverse (NEW – function?function, not only routes): Build once per request or cache short-TTL on branch: for each visible binding B with body expr: for each direct callee C in collect_var_apps(body): callers[C] := callers[C] ? {B} callers(name) ? list of B that directly App-call name dependents(name, ~depth) ? BFS reverse edges, cap depth (default 8), cycle-safe; exclude name itself

Routes (extend used_by): used_by_routes(name): - always include routes whose handler = name - PLUS routes whose handler's transitive callees include name (reuse page_function_graph or reverse map from cached forward graphs). Cap cost: compute reverse route index when building callers map once per branch snapshot, not per /bindings row naively.

Nopales prims (session-aware, read-only, any authed user): (callees name [depth]) ? VList of VStr names (callers name) ? VList of VStr names (direct) (dependents name [depth]) ? VList of VStr names (used-by-routes name) ? VList of route path strings

Missing name ? () or empty list (not error). Prims/kernel names: callers may be empty; still OK. IMPORTANT (image.dnu-stubs): callers(name) MUST work when name is not yet a function_def – reverse index is built from all visible bodies' App-Var heads, not only defined callees. holes(name) lives in dnu-stubs; shares walk.

———- B. IMPACT QUEUE (on define / install) ———- When a binding is successfully defined or install_function succeeds for NAME on the current branch:

impact = { name: NAME at: timestamp branch: current branch ref author: whoami dependents: dependents(NAME) routes: used-by-routes(NAME) status: pending | smoked | dismissed }

Storage (v1): Session-local ring buffer (last N=32 impacts) on Session.t is enough for agent + UI banner. Optional later: data_rows type impact-event for cross-request durability.

Surfaces: 1) Nopales: (impact-queue) ? list of impact dicts/JSON strings (impact-clear [name|all]) (impact-mark name 'smoked|'dismissed) 2) define / install_function return path (agent tools): append human-readable block: "im-

pact: dependents=?; smoke routes: /a /b" If routes empty: "impact: no routes; eval dependents or smoke parents manually" 3) UI (light): /bindings or /edit/<name>: badge "N dependents" + link to impact panel; optional site chrome banner when queue has pending items for this branch (do not block editing).

Do NOT fail define because dependents exist – only notify.

———————————————- C. "NEEDS TESTING" LADDER ———————————————- L0 (MVP – ship first): awareness only (queue + agent text) L1: optional auto eval of pure dependents (if effect pure) L2: recommend/auto smoke_route for each used-by-routes entry (agent definition-of-done already requires smoke; impact tells WHICH paths) L3 (later): hard badge red until smoke or dismiss

Default ship L0+L2-hints. No blocking L3 in v1.

———————————————- D. AGENT SEED ———————————————- After install_function / eval define of a package function: - Read impact (tool result or (impact-queue)) - smoke_route every listed route (GET, contains marker) - eval_expr key pure dependents if no routes - Final summary must mention impact smokes

———————————————- E. LIMITS (honest static analysis) ———————————————- - No dynamic apply / computed names - Higher-order (map f xs) may miss f unless App head - Macros: graph is source App Vars (post-expand ideal later; v1 source walk matches today's inspector) - False negatives possible – route smokes remain truth - Noise control: only enqueue when NAME is a stored function_def (not every let-bound local); skip if dependents and routes both empty

———————————————- F. NON-GOALS ———————————————- - Runtime request traces / dynamic call sampling - Full CI product / GitHub integration - Coverage-guided line maps - Blocking save on dirty dependents - Cross-branch impact (only current branch visibility)

———————————————- G. IMPLEMENTATION PHASES ———————————————- Phase I1: shared graph helpers + (callees)/(callers)/ (dependents)/(used-by-routes); unit/smoke (callers works for missing names – DNU) Phase I2: impact-queue on define + install_function text; (impact-queue)/(impact-clear)/(impact-mark); same hook appends change-transcript (branching.change-set) Phase I3: /bindings detail Callers/Callees/Impact badge; agent seed paragraph Phase I4 (optional): short-TTL cache; L1 pure auto-eval

Combo-pack build order (whole five): I1 graph ? I2 define hooks (impact+changeset) ? browser detail panels ? dnu-stubs ? desktop 3-pane

———————————————- H. SUCCESS ———————————————- - Changing ui/menu shows ui/nav (or similar) in callers - used-by-routes for a shared helper lists real pages - Agent after install is told which routes to smoke - /bindings stays fast (no per-row depth-8 walk regress)

DEFERRED (still): eval-result inspect, type-def deep inspect, runtime traces, mutate-from-inspector.

========== COMBO-PACK NOTE ==========
Inspector owns graph + impact + value inspect (#1, #3). Browser hub: image.system-browser. Change transcript: branching.change-set. Holes: image.dnu-stubs. Define hook is the shared write path for impact + change-set append.

dnu-stubs

doesNotUnderstand-style holes: unbound names ? stub + callers for agents

[status: implemented]

– *agent note:* 2026-08-02: holes + stub-function prims; binding

page holes panel; unbound handler Create stub; session registers graph/DNU prims.

[verify] build: dune build image; smoke: eval unbound foo yields structured unbound_var with name=foo; smoke: (holes "ui/nav") lists App-Var names in body not bound as fn/prim; smoke: POST stub create missing name ? callable lambda TODO stub; smoke: unbound route handler page offers Create stub + Callers; smoke: agent after unbound error can stub then see callers of stub name

==========

WHY (Smalltalk doesNotUnderstand, agent-friendly) ==========

Live images fill holes instead of failing closed forever. Agents hit unbound names constantly (typos, planned fns, half-migrated packages). Today: bare Unbound_var / handler unbound cards. Goal: a closed loop without becoming CI:

missing name ? structured error (kind + name) ? who already calls it? (static callers – may be empty) ? one-click / one-tool stub (lambda TODO) ? edit + impact/smoke via existing inspector paths

This is NOT a full condition system (see lisp-interpreter.errors-and-locations for guard/catch). DNU here = product UX + agent affordance on holes.

==========

A. STRUCTURED UNBOUND (host) ==========

Keep raising Interp_error (Unbound_var name). Surfaces must expose machine fields (align with errors-and-locations C): kind: unbound_var name: <missing> source: optional define/handler context /api/eval JSON and agent tool errors include name= so agents do not regex the message. REPL print may stay one-line.

==========

B. STATIC HOLES IN A BODY ==========

holes(name) – for stored function NAME: walk body with collect_var_apps (same as graph) candidate = App-Var heads hole if not in: prim registry ? visible function_defs ? special forms ? lambda params / lets (best-effort; App heads only; accept false negatives on dynamic names) Nopales: (holes name) ? VList of VStr UI: browser detail + editor show Holes list.

==========

C. STUB CREATE ==========

Default stub body (same spirit as function-editor missing): (lambda () "TODO: implement <name>") Zero-arg v1; agent/human edits signature. Never invent args.

Paths: 1) function-editor missing card [Create] – already specified; dnu-stubs REUSES it (POST /api/edit/create) 2) Nopales (stub-function name) ? define stub if missing; error if already bound (use install/edit to overwrite) 3) Unbound handler HTTP response: card with handler H unbound [Create stub H] [Edit if exists] [Callers of H] 4) Agent tool optional stub_function name= – thin wrap over same define_binding; seed text: on unbound_var, stub ? install real body ? smoke impact routes

Creating a stub is a normal define: fires impact-queue + change-set transcript hooks.

==========

D. CALLERS OF A HOLE ==========

Even when NAME is unbound, reverse graph still finds B whose body App-calls NAME (static). (callers name) must work for missing names (build from all visible bodies), not only defined callees. Critical for "who wanted this?"

==========

E. NON-GOALS ==========

- Runtime doesNotUnderstand method dispatch / prototype OO - Auto-generating real implementations via LLM on every miss - Catching unbound inside every page without guard (that's errors-and-locations phase 2) - Stubbing kernel prim names

==========
F. PHASES ==========
D1: structured unbound name in /api/eval + (holes) D2: callers works for missing names; UI holes on detail D3: stub-function + unbound handler card + agent note D4: wire browser detail Holes panel (system-browser)

SUCCESS: agent sees unbound: foo with name field, stubs foo, finds callers, fills body, impact lists routes.

16 borge/multipart.borg

pricklypear-multipart

Pricklypear book chapter: multipart streaming responses
[status: implemented]

multipart

multipart/mixed streaming responses: one HTTP response, many independently-targeted htmx swaps. The reactive substrate.

mime-framing

One route response streams N MIME parts; each part is one htmx swap
[status: implemented]

> *[verify] build: dune build image; test: part / json-part / multipart prims build tagged values; test: keyword :target self-evaluates; smoke: route returns (multipart (part "one") (part "two")) => Content-Type: multipart/mixed; both parts present in body; smoke: peer disconnect mid-stream is a non-event (http-robustness invariant 1 holds on this path too)*

Inspired by scriptogre/multipart-response (Python adapters for Django/FastAPI/Starlette/FastHTML) and the htmx hx-multipart extension. Strip away the framework adapters and the idea is small: the server responds Content-Type: multipart/mixed; boundary=B and writes parts as they are ready. Each part carries its own headers – HX-Target / HX-Swap / HX-Part-ID – so every part is an independently-targeted htmx swap. One HTTP response becomes a stream of targeted DOM updates; no websockets, no SSE endpoint, no per-swap round trip.

NOPALES SURFACE (new prims, VStr-tagged like render-html): (part html) ; one part, text/html (part html :target "#feed" :swap "beforeend" :id "42") (json-part json-string :id "result") ; application/json part (multipart p1 p2 ...) ; list of parts (multipart :target "#feed" :swap "beforeend" p1 p2 ...) ; leading kwargs = envelope defaults for every part; ; per-part headers override envelope defaults.

ENVELOPE: Content-Type: multipart/mixed; boundary=<random> Envelope HX-Target / HX-Swap written as response headers (hx-multipart honors them as part defaults).

PART WIRE FORMAT (RFC 2046, minimal): –B\r\nContent-Type: text/html; charset=utf-8\r\n HX-Target: #feed\r\nHX-Part-ID: 42\r\n\r\n<body>\r\n ? final –B–\r\n. Bodies are checked for boundary collisions (on collision, regenerate boundary once, then raise User_error – do not silently corrupt).

RENDER PATH: respond_handler_value gains a branch: a multipart-tagged value sets multipart Content-Type and streams parts through the existing Httpun.Body.Writer (write_string + flush per part). The closed-writer guard and peer-gone invariants from image.http-robustness apply unchanged – a disconnect mid-stream MUST NOT take the process down.

v1 IS EAGER: the handler returns a fully-evaluated list of parts; the kernel frames and streams them. You get multiple targeted swaps in one response – the reactive payoff – without lazy generators in the interpreter. Deferred: Nopales-level streaming sources (generator or (multipart-stream (lambda (emit) ?)) callback style) and HX-Last-Part-ID resume for live tails.

hx-multipart-vendored

Vendor htmx hx-multipart extension; static asset, no CDN dependency
[status: implemented]

[verify] build: dune build image; smoke: GET /assets/hx-multipart.js returns 200 with the extension source

The client half is the hx-multipart htmx extension (four.htmx.org/extensions/hx-multipart), which parses the multipart stream via fetch-multipart and performs one swap per part, honoring per-part HX-Target / HX-Swap / HX-Select and the HX-Part-ID / HX-Last-Part-ID reconnect cursor.

Vendored next to the CodeMirror assets (/assets/cm/*) so the image works without a CDN. Pages opt in with hx-ext "hx-multipart" on the triggering element.

repl-v2

Advanced browser REPL: one input, streamed multipart output – LLM answers, slash commands, live eval

[status: implemented]

[verify] build: dune build image; smoke: (+ 20 22) as input => 42 multipart part; smoke: /note hello there => confirmation part appended; smoke: error in eval => error part styled distinctly; smoke: /branch => whoami @ branch part; smoke: GET /assets/hx-multipart.js 200

The advanced REPL is a Nopales package (libs/repl) built on the multipart substrate – the dogfood that proves it. One interface, one input box, and the response is a multipart stream whose parts append to a scrolling transcript.

INPUT DISPATCH (repl/answer): - "/cmd args?" -> slash command (dispatch table, see below) - "(?" -> Nopales eval via the same Session.eval_one path as browser-repl / /api/eval – NOT a second interpreter - otherwise -> LLM question via libs/ai (ai/ask-shaped: agent-session + context block)

A single submission can yield SEVERAL parts – that is the point. An LLM question streams its deltas as parts (llm_client already supports stream:true with on_event callbacks); an eval can yield a "running?" part, then the value part, then a notes-link part. Each part targets the transcript (HX-Target #transcript, HX-Swap beforeend) or replaces a placeholder part by id.

SLASH COMMANDS (v1): /note <text> capture into notes (dailies-and-capture path) /eval <expr> explicit eval (same as bare "(" input) /describe <pkg> package surface summary part /branch current branch + whoami part /help command list part Commands are functions in the package; users add their own with (repl/command "todo" 'my/handler).

IDENTITY: runs on the caller's branch with the caller's session (route is private by default). LLM key resolution follows auth.user-llm-keys (personal key, operator fallback).

RELATIONSHIP TO EXISTING SURFACES: browser-repl stays – the simple /repl + POST /api/eval path is the low-level tool (and the agent API). repl-v2 is the human cockpit: LLM + commands + eval in one streamed transcript. If it earns its place, /repl can later point at it; that decision is NOT part of this section.

live-streaming

Token-level streaming: LLM deltas and long-running evals appear incrementally in the transcript, not as one final part

[status: planned]

[verify] build: dune build image; smoke: LLM question in repl-v2: first token appears in <1s, subsequent tokens append live, final part completes the bubble; smoke: eval that yields multiple values streams each as it completes, not batched at end; smoke: error mid-stream yields error part and closes response cleanly; smoke: peer disconnect mid-stream is a non-event (http-robustness invariant 1)

RECONCILE (BORGE-1786263466): LLM live-streaming is SHIPPED via a DEDICATED OCaml route – /repl-v2/llm (repl_llm in image/lib/http_agent.ml) – which reuses the proven /api/agent SSE queue/drain pattern, NOT the general Nopales (multipart-stream (lambda (emit) ...)) mechanism spec'd below. Rationale (recorded choice): the /api/agent SSE queue/drain is battle-tested and lower risk; calling Nopales emit callbacks from OCaml on_event fibers needs effects/run_lwt coordination that cannot be runtime-verified without a live LLM in one session. Eval/slash stay on the Nopales /repl-v2/send route (eager); LLM input is routed client-side to /repl-v2/llm via an htmx configRequest hook. The GENERAL (multipart-stream ...) mechanism below remains FUTURE WORK – this stanza stays planned but must describe reality until then.

(The shipped shape below is the still-planned general mechanism, superseded for LLM streaming by /repl-v2/llm.) Current repl-v2 is EAGER: the handler computes all parts, then multipart frames them. For LLM answers this means the user waits for the full completion before seeing anything. The fix is live streaming: the handler yields parts as they are ready, and the kernel flushes each part over the wire immediately.

KERNEL CHANGE: respond_handler_value gains a streaming branch. Instead of evaluating to a list of parts then framing, the handler runs with an emit callback: (multipart-stream (lambda (emit) (emit part1) (emit part2) ?)) Each emit writes one MIME part and flushes. The existing Httpun.Body.Writer write_string + flush per part becomes the streaming path.

NOPALES SURFACE: (multipart-stream (lambda (emit) ?)) emit is a function of one argument (a part value). The lambda body runs to completion; each emit call pushes a part to the client. Errors inside the lambda emit an error part and close the stream cleanly.

AI INTEGRATION: libs/ai/ai/ask gains a streaming variant (ai/ask-stream on-event) where on-event receives deltas. repl-v2's LLM branch uses it: each delta emits a part targeted at the transcript, building up the answer token by token. The final delta emits a completion part that styles the bubble as done.

SCOPE: only LLM streaming in repl-v2 v1. Eval streaming (multiple values over time) is a follow-up. The prim name collision bug (BORGE-1786262466: agent-session-new vs Agent_tools.agent-session-new) must be fixed first – the LLM path is broken without it.

repl-v2-ux

REPL transcript polish: echo user input, clear form on send, distinct styling per message kind

[status: implemented]

> *[verify] build: dune build image; smoke: user input appears in transcript before response; smoke: form input clears on successful send; smoke: form input clears on error send (no stuck text); smoke: error parts styled distinctly from value/llm parts*

All three UX gaps closed across BOTH input paths – the Nopales /repl-v2/send route (slash commands + eval) and the OCaml /repl-v2/llm route (plain-text LLM input, routed client-side by an htmx configRequest hook).

1. USER ECHO: repl/send (libs/repl/lib.pp) prepends repl/user-echo – one user bubble part (raw input, .repl-user class, italic) before the response parts. The OCaml /repl-v2/llm route (image/lib/http_agent.ml repl_llm) emits the same user-echo bubble as its Part 1, so the transcript reads two-sided on both paths.

2. FORM CLEAR: the repl-form clears on hx-on::before-request (immediate, on submit) AND hx-on::after-request (fallback reset) so no input is ever left stuck – the before-request hook avoids the after-request race with the swap and also clears the box while a long LLM stream is still running.

3. STYLING: each message kind gets its own border/paint via the repl-line base + per-kind classes (.repl-value/.repl-error/.repl-llm/.repl-note/.repl-user); the LLM failure case emits a .repl-error span with a human-readable message ("error: ?") instead of a raw exception string, and error parts carry a red border + pink background distinct from value/llm parts.

(The user-echo work is what makes the transcript readable – without it the transcript reads one-sided and the user loses track of what they asked.)

17 borge/hypermedia-harness.borg

pricklypear-hypermedia-harness

Pricklypear book chapter: a headless hypermedia client – a server-side htmx swap simulator that is the reference implementation of the fixi-five contract, for debug agents and verification harnesses. It runs in the image, fetches no HTTP, runs no browser, and calls route handlers in-process.

[status: implemented]

> – *agent note:* 2026-08-14 RECONCILE, with agent.borg: the harness is now a SHARED, host-portable layer. soup/* stays DB-free and is lifted into the shared pure-prims layer so the image (Postgres corpus) and the agent (local-first lens corpus) register the same tree prims. The ONE host seam is route/dispatch + route/lookup (request-runner): Nopales signatures identical, host-local backing (Session.t Postgres routes vs Runtime.t lens routes; fat Request_context vs thin agent request context). libs/hyper runs unchanged on both. Streaming swaps (hx-multipart multipart/mixed, HX-Target/HX-Swap per part) move INTO the contract as data – soup/apply-parts! applies an ordered part list; hyper/stream wraps it immutably; transport/timing stays CDP-tier (unsupported-boundary). This is the refactor that lifts the agent's chat/agent UI from SSE+vanilla-JS to Nopales-authored htmx fragments.

hypermedia-harness

A headless htmx client living inside the image. It invokes route handlers in-process, parses the returned HTML into a tree, and applies htmx swaps (target + strategy + out-of-band) to that tree – the exact state machine of the production browser client, restricted to the fixi-five subset. Debug agents use it as a cheap, deterministic verifier; the authenticated, no-browser surface is callable through /api/eval with no new REST surface.

contract

The executable contract is the fixi-five intersection: the semantic core shared by htmx (the production client) and fixi (the minimal reference) – fetch + swap, five attributes (get/post/target/swap/ trigger). The harness is the reference implementation for this subset; within it, the harness is authority and the production client (htmx 4) must conform. Outside it, htmx 4 is authority and the harness declines. The contract is the shared object of which fixi is the most compact correct statement; it does not move when the production client changes.

[status: implemented]

supported-subset

Attributes the harness models, mirroring htmx 4 semantics under the htmx-2-compat defaults PP ships.

MODELED: - hx-get, hx-post, hx-target, hx-swap, hx-trigger – the fixi five - Inherited hx-target / hx-swap / hx-confirm via ancestor lookup (mirrors the implicit-inheritance mode restored by htmx-2-compat) - hx-swap-oob: main content swaps FIRST, then OOB elements swap after (htmx 4 ordering; OOB elements are hoisted from anywhere in the response, including nested within the main target) - hx-confirm: proceeds without the consent gate (a UI affordance, not behavior) - hx-vals: JSON values only - Swap strategies: innerHTML, outerHTML, beforebegin, afterbegin, beforeend, afterend, delete, none

- hx-multipart streaming swap class: a multi-part streamed response (multipart/mixed, one fragment per part, each with per-part HX-Target/HX-Swap headers – see kernel-prims apply-parts!) is modeled by applying the captured part list in order onto the page (soup/apply-parts!), not by racing bytes over a socket. The stream is reduced to data before the harness consumes it, so the swap semantics are the same fixi-five swap machinery applied N times, deterministically.

[status: implemented]

unsupported-boundary

Features the harness returns unsupported-feature for, instead of modeling. These flows graduate to the CDP/browser verification tier rather than being silently under-modeled.

NOT MODELED (returns unsupported-feature): - hx-on scripting – client JavaScript; opaque without a JS engine (the moxi.js boundary; the harness may record that a script existed but cannot execute it) - hx-push-url – history; no fixi equivalent - hx-ext – inert in htmx 4 (extensions activate by script inclusion) - hx-vals with js: prefix – cannot eval JS - Triggers with debounce / throttle / delay modifiers - hx-sync – request queueing and coordination - hx-boost, hx-ws – not used in PP's markup; out of scope. - hx-sse transport/timing – the streaming SWAP class is now MODELED (supported-subset hx-multipart: an ordered part list applied via soup/apply-parts!). What stays NOT MODELED is the transport/timing axis – racing bytes, reconnect, server push cadence – which is not a swap semantic and graduates to CDP. The absence of hx-boost/hx-ws from PP's markup is why the fixi-five subset is a faithful contract for PP, and the trigger to revisit this boundary is concrete: the day either appears in libs/, the subset must either grow or that flow must acknowledge the CDP tier.

[status: implemented]

kernel-prims

Thin lambdasoup bindings – the HTML tree layer. lambdasoup and markup are already in the poohstack opam switch; no new dependency. Prims are registered under the soup/ namespace (not html/) to avoid colliding with the pure-html DSL prims (html/attr, html/text, html/div, ...) that page rendering depends on.

PRIMS (uppercase, DB-free, part of the pure-prim layer, soup/ namespace): (soup/parse str) -> page handle (soup node) (soup/page-copy page) -> new page handle (deep copy; immutability: source never mutated) (soup/select page selector) -> list of node handles (soup/select-one page selector) -> node handle | nil (soup/serialize page|node) -> HTML string (soup/text page|node) -> concatenated text content (soup/attr node name) -> string | nil (soup/has-attr node name) -> "true" | nil (soup/swap! node style fragment) -> mutate in place (style = one of innerHTML, outerHTML, beforebegin, afterbegin, beforeend, afterend, delete, none) (soup/create tag) -> node handle (soup/replace node fragment) -> replace target with fragment's nodes (soup/closest-attr node name) -> ancestor walk for inherited hx-target/hx-swap/hx-post resolution (mirrors htmx compat implicit inheritance) (soup/parent node) -> node handle | nil (soup/diff a b) -> structured what-changed report (added/removed ids + text changes within same-id elements) (soup/apply-parts! page parts) -> mutate in place; parts is a VList of part alists, each (("html" . frag) ("target" . sel) ("swap" . strat)); each part is located (soup/select target) then swapped (soup/swap! strat frag) in ORDER. target/swap absent -> caller's element + "innerHTML", matching hyper/do-swap defaults. Matches the multipart/mixed wire shape http_agent.mp_write_part emits (per-part HX-Target/HX-Swap); see supported-subset hx-multipart.

Mutations use lambdasoup's append_child / insert_before / insert_after / delete / clear / set_attribute. The diff is node-level (which ids changed, added, removed, or text-changed), not a raw text diff – this is the agent-facing value: a compact report instead of dumping DOM into model context.

SHARED KERNEL. The soup/ tree layer is DB-free. It is lifted out of image/lib/prims_hyper.ml into the shared pure-prims layer (pure/, alongside prims_pure.ml) so BOTH hosts register the same prims: the image (Postgres corpus) and the agent (local-first lens corpus). Only route/dispatch + route/lookup + the request-runner remain host-specific (they touch the host's store / routes / request context – see request-runner). lambdasoup + markup become deps of the agent package, joined on the same seam.

[status: verified]

> *[verify] build: dune build image; test: soup/parse + soup/select round-trips a server-rendered /notes page; test: soup/swap! applies each of the eight strategies correctly; test: soup/diff detects added, removed, and changed nodes; test: soup/apply-parts! applies an ordered part list into the page: each part's fragment lands at its target with its swap strategy, in sequence*

request-runner

In-process dispatch reuses existing machinery – no new transport. The harness binds a synthetic request and invokes a routed handler the same way the HTTP gate does, without an Httpun socket. This is THE HOST SEAM: the only prims that differ between hosts. The image backs them with its Postgres corpus (Session.t routes + store + Request_context manifest); the agent backs them with its local-first lens corpus (Runtime route table + a thinner request context). The Nopales signatures are identical, so libs/hyper runs unchanged on both hosts.

Request_context.make ~manifest ~env ~routes constructs the per-request snapshot (the image.per-request-context pattern from the http-robustness work). Session.call_handler session ~request handler_name invokes the routed handler with the synthetic request bound so request-prims (hx-request?, query-param, current-author) resolve correctly for the duration of the call. Returns Handler_ok v – the rendered value the browser would receive. No HTTP round-trip, no socket, no chromium.

NOPALES BRIDGE (new prim, session-capturing – same registration pattern as crud.ml: registered in session.ml via a register_* call that closes over the Session.t; the late-bound thunk in session.ml closes over call_handler to avoid a forward-reference): (route/dispatch handler-name method path [params] [headers] [body]) -> raw-HTML VStr Binds a per-call Request_context (with_current) so handler prims resolve, calls Session.call_handler, and returns the rendered value as a VStr (the same string the browser receives – is_raw_html-tagged values unwrap per render_handler_value). hyper/visit and hyper/click sit on top of route/dispatch. (route/lookup path) -> handler-name VStr | nil resolves a URL path to its handler via the route table.

AGENT HOST BACKING (see agent.borg agent-route-table). The agent implements the SAME prim signatures against its runtime: routes live on a Runtime-scoped route table (Runtime.t.routes), populated by the agent's (route path handler) prim reading its lens-resident defs; dispatch invokes the Nopales handler through Runtime.apply_value under the eval rim; the request context is THINNER (no Postgres manifest / branches / auth – only the request-shaped prims resolve: query-param, request-method, hx-request?). Request prims that need a DB/branch (e.g. current-author-as-DB-row) are unbound in the agent; the agent's request context provides only the request-shape values. This is the intended portability seam: same prims, host-local backing.

[status: verified]

[verify] build: dune build image; smoke: route/dispatch on a routed page handler with a synthetic GET context (Request_context bound) returns the same HTML the HTTP gate returns for the corresponding GET; smoke: agent: route/dispatch on an agent-registered handler runs the Nopales fn through the runtime and returns its rendered fragment (no Postgres)

nopales-surface

The client semantics, written in Nopales over the kernel prims and the request runner. Lives in a new package libs/hyper. Page is an immutable threaded value: each interaction returns a NEW tree (deep-copied via soup/page-copy before mutation), giving free time-travel history and a cheap what-changed diff.

SURFACE: (hyper/visit url) ; resolve via route/lookup, GET ; in-process (no HX-Request header, ; so handler returns the full page ; like a browser's initial nav), ; soup/parse body -> page value (hyper/click page sel params) ; resolve trigger at sel OR nearest ; ancestor (inherited hx-post/hx-get ; via soup/closest-attr), read ; hx-target/hx-swap (inherited), ; POST in-process with HX-Request, ; soup/page-copy the page, soup/swap!, ; return NEW page (hyper/text page sel) ; first match's text content (hyper/query page sel) ; list of node handles (hyper/count page sel) ; element count (hyper/html page) ; serialize tree -> string (hyper/attr node name) ; attribute value (hyper/diff p1 p2) ; node-level what-changed (hyper/expect page sel expected) ; assertion (for smokes) (hyper/stream page parts) ; apply an ordered multipart ; part list onto a page via ; soup/page-copy + soup/apply-parts!, ; return NEW page (immutable: ; input page unchanged). parts ; come from the host stream as ; DATA (agent: loop.ml collects ; the emitted HX-Target/HX-Swap ; parts; image: http_agent ; mp_write_part). Lets hyper/** ; assert a consumed chat/agent ; turn deterministically. (hyper/sandbox branch body) ; v1 stub – branch checkout with ; restore needs dynamic-wind

INVARIANTS: - Each interaction returns a new page value; the input page is never mutated in place (enables history and diff). - hyper/click on a node whose trigger falls outside the supported subset (e.g. data-ol-op toolbar buttons, which are client-JS driven not htmx) returns unsupported-feature rather than a best-guess swap. - hyper/sandbox (v1 stub) does not yet check out a branch.

[status: implemented]

[verify] build: dune build image; smoke: hyper/visit on a notes fixture returns a page whose ol-node count matches the server-rendered page; smoke: hyper/click on a button with hx-get=/welcome hx-target=#out swaps #out content; the original page is unchanged (immutability); smoke: hyper/click on a data-ol-op node (client-JS-driven, not htmx) returns unsupported-feature; smoke: hyper/expect on a selector raises on mismatch and returns true on match; smoke: hyper/diff between two pages reports the node that changed

fidelity-gate

The conformance lock between the harness and the production browser client. The same flow is driven twice – once through CDP against a live browser running htmx 4, once through hyper/* – and the resulting DOM text is asserted byte-identical. Divergence is either a harness bug (the simulator is wrong and must learn the quirk) or an htmx quirk the harness must mirror. Once a flow is green twice via CDP, it graduates to a hyper/ smoke that runs in milliseconds with no browser. The visit step is proven byte-identical; multi-step swap fidelity is deferred until a real hx-get/hx-post flow is targeted (the notes outline toolbar is JS-driven data-ol-op, not htmx, so it is correctly the unsupported-feature boundary, not a fidelity target).

[status: implemented]

[verify] build: dune build image; smoke: drive GET /notes?slug=ui-fixture via CDP and via hyper; assert byte-identical #outline DOM text at the visit step

18 borge/product-experience.borg

pricklypear-product-experience

Pricklypear book chapter: product-experience
[status: implemented]

> – *agent note:* 2026-08-02 status reconcile: concepts/access/vocab/first-run/public-release already impl; analytics+reliability matched to shipped ops_* modules; horizon design-only ? implemented. Live /analytics 200; ops/summary day rollup.

product-experience

Human-facing product layer: concepts, access paths, legible vocabulary, first-run, horizon
[status: implemented]

> *[verify] build: dune build image; smoke: GET /about teaches concepts; home offers first-run hello recipe; smoke: scripts/repl.sh logs into REPL*

Pricklypear is a multi-user live computing environment: log into your image, write Nopales functions stored as data, branch code while sharing data, bind functions to URLs, let AI propose functions onto a branch.

Product experience is the thin skin that makes the dense substrate legible. Without it, only authors understand the system.

LAYERING (aligned with image.system-browser + image.product-theme, 2026-07-12): - Environment UI humans live in (nav, home, app menus, theme/presentation, user routes) is Nopales on branches (libs/*.pp seed + personal pins). That is the product. - OCaml hosts Nopales: eval, store, auth gate, health, kernel system pages (/bindings, /edit, ?), and a labeled emergency shell if ui/nav, ui/home, or ui/css fails. - OCaml must not ship a silent second product nav/home or a silent full design-system CSS twin in http.ml. Dual silent defaults are rejected architecture. - Theme is Path B (honest Nopales): css/* sheets + tokens, not CSS-string-in-lambda as the native form.

concepts-guide

In-app about page: glossary, access, first tour, horizon summary
[status: implemented]

> *[verify] build: dune build image; smoke: GET /about contains Image, Store, Function, Branch, Route, AI*

GET /about (kernel OCaml page today; may move to Nopales later) MUST explain:

WHAT: multi-user live Nopales environment; image=CPU, Postgres=disk; Smalltalk tradition without OOP image dump.

GLOSSARY: - Image: running process (REPL + HTTP); single-image v1 - Store: Postgres durable truth - Function/binding: named live value; define persists - Branch: versioned code world; data shared - Type/row: deftype + data_rows CRUD - Route/page: URL bound to a function - Shell (ui/menu, ui/nav, ui/home, ui/theme, ui/css): Nopales product chrome + presentation - Theme tokens (theme/bg, ?): branch-visible style knobs - AI propose: agent authors on current branch only

ACCESS: - scripts/dev.sh start (PG + image) - Browser: /login (app session) then / /repl /edit /bindings - scripts/pp_eval.py or Basic auth for agents - scripts/repl.sh / line-protocol REPL when enabled - nginx basic-auth may sit at edge; not the user DB

FIRST TOUR: log in, define hi-page, route /hello, open page, edit ui/menu for navbar, optional propose-function.

Link from home nav and home body. Full horizon detail in product-experience.horizon.

access-paths

Documented ways humans reach a live image
[status: implemented]

> *[verify] smoke: GET /login form; POST /login mints pp_session; smoke: scripts/verify_auth.sh or auth_product_smoke passes; smoke: scripts/repl.sh connects when line-protocol enabled*

v1 SHIPPED paths (single-image): 1. Browser: GET/POST /login ? cookie pp_session ? land on username/personal; POST /logout revokes 2. Admin: /admin/users (create user, revoke sessions) 3. Agents: HTTP Basic or session cookie ? /api/eval (scripts/pp_eval.py) 4. scripts/dev.sh start – PG + image 5. scripts/verify_auth.sh product gate 6. Optional scripts/repl.sh / line-protocol REPL

Product chrome after login is Nopales (ui/nav, ui/home, ui/theme/ui/css). Auth gate and emergency shell/CSS are OCaml kernel only.

MULTI-USER ACCESS (planned – auth.* subsections): - Per-user subdomain: https://{user}.{PP_BASE_DOMAIN}/ (auth.user-subdomains); apex login redirects home - Per-user LLM key: credentials llm_key; /settings/keys (auth.user-llm-keys); env LLM_API_KEY is fallback only - SSH ? REPL: credentials ssh_key; PP_SSH_PORT opt-in or OpenBSD sshd bridge (auth.user-ssh-repl)

MULTI-USER SOCIAL (planned – multi-user.*): - Chat lobby + DMs: /chat, (chat/say ?) (multi-user.chat) - Unix .plan: /plan, /u/:user, (plan "?") (multi-user.user-plan)

DEFERRED: - Fleet boot/attach per user (process isolation path) - Custom domains per user - Drop nginx htpasswd as primary human gate (break-glass only; already not the user DB)

vocabulary

Friendly UI labels over systems jargon
[status: implemented]

> *[verify] smoke: nav/home use Functions (not only Bindings) and point to /about*

UI vocabulary (human): Functions <- function_defs / bindings Pages <- view_bindings / routes Data <- type_defs + data_rows Workspace <- current branch + whoami About <- concepts guide

HARD RULE: Page ? Function. Page = URL endpoint (/hello). Function = named code (hi-page). Edit targets are always function names (/edit/hi-page). Page inspect is /page/hello (route + call graph). Never nav-link /edit/<route-basename> as if it were a function.

PACKAGE vs BRANCH: Package = in-branch prefix (todo/) for organizing functions. Branch = versioned workspace (system/main). UI labels: "package" for prefixes, "branch"/"workspace" for version worlds – never swap the words. Functions browser groups by package; branch chrome stays whoami + current branch.

Nopales/REPL vocabulary stays technical (define, route, deftype, rows). The browser skin translates.

first-run-recipe

One-click or one-form 'make a hello page' onboarding
[status: implemented]

> *[verify] build: dune build image; smoke: POST /api/recipe/hello creates hi-page route; GET /hello works*

Recipe "hello page": 1. Define function hi-page -> render-html greeting 2. (route "/hello" "hi-page") 3. Return link to /hello and /edit/hi-page
Exposed as: - Button/form on home - POST /api/recipe/hello
Non-goals: full app wizard (that's AI app-sketch later).

public-release

Hosted public face on the image: layout components, welcome, code browser, book PDF, source tarball
[status: implemented]

> *[verify] build: dune build image; smoke: unauth GET /welcome returns 200 with hero; smoke: unauth GET /code returns tree + codemirror shell; smoke: unauth GET /code/raw/pricklypear.borg returns text; smoke: unauth GET /book.pdf or 404 if artifact missing; smoke: unauth GET /src.tgz or 404 if artifact missing; smoke: path escape /code/raw/../etc/passwd denied; smoke: unauth GET /bindings still redirects to login; smoke: unauth GET /.well-known/agent.json returns 200 application/json with auth+eval contract; smoke: unauth GET /agent.txt returns 200 text/plain agent instructions; smoke: unauth GET /agent still denied (private portal); smoke: unauth POST /api/eval still denied*

========== GOAL ==========
Release pricklypear on the hosted instance itself – not GitHub-as-homepage, not a sister brochure site. The live image IS the public site:

/welcome – landing (Nopales layout/* + welcome/page) /code – read-only source browser (kernel + CM5) /book.pdf – borge book print artifact /src.tgz – pruned source tarball /.well-known/agent.json – machine agent bootstrap /agent.txt – prose agent instructions (auth + eval) /login – enter the private workspace

NOTE: /agent is the PRIVATE in-image agent portal (HTML), not the public discovery doc. Public agent onboarding is /.well-known/agent.json + /agent.txt only.

/book (HTML explainer) is NOT public-facing – PDF + /code cover the literate book story. Any welcome/book page is private workspace chrome only, not a front-door path.

Workspace home (/) stays HARD PRIVATE. Anon hitting / redirects to /welcome (or login with next=); logged-in users keep product home (ui/home).

========== LAYOUT COMPONENTS (libs/layout) ==========
Namespace layout/* – NOT public/*. "public" is reserved for route visibility / gate language.

Building blocks (pure HTML strings via html/*): layout/hero title tagline actions-html layout/card title body layout/cards list-of-card-html layout/section title body layout/cta href label layout/theme – css/sheet rules for .layout-*

Seeded on system/main; used by welcome (and private) pages. Personal gardens may override layout/* like any package.

========== WELCOME PAGE (libs/welcome) ==========
welcome/page – compose layout/* into landing (public-route "/welcome" "welcome/page")

welcome/book may exist as a private helper, but MUST NOT be registered public and MUST NOT be a SYSTEM_HARD_PUBLIC path. Reseed should (route "/book" ? 'private) or drop the binding so older public /book rows flip closed on deploy.

Also register hard-public kernel paths so the front door works even if seed/route table is empty (belt + suspenders): /welcome is hard-public and dispatches to Nopales when bound, else a minimal kernel fallback page.

CTAs: Explore source ? /code ; Download book ? /book.pdf ; Source tarball ? /src.tgz ; Log in ? /login.

==========

CODE BROWSER (kernel, image/lib/http_code.ml) ==========

Read-only filesystem browser over PP_SOURCE_ROOT (default: repo root / PP_ROOT / cwd with libs+image+lisp).

Routes (all hard-public GET): /code – HTML shell: file tree + CodeMirror /code?f=path – open file in editor /code/raw/<path> – raw text (for fetch into CM) /assets/cm/* – vendored CodeMirror 5 assets

CodeMirror 5 (vendored, no npm at runtime): codemirror.min.js + codemirror.min.css mode/mllike (text/x-ocaml for .ml/.mli) mode/markdown for .md plain for .borg .pp .sql .sh readOnly: true, lineNumbers: true

SANDBOX: - realpath must stay under PP_SOURCE_ROOT - allowlist extensions: .borg .ml .mli .pp .md .sql .sh .ksh .dune (and dune files named "dune") - skip dirs: _build .git .pi .ralph node_modules _opam .opamswitch spikes (optional) supervisor (deprecated – skip) image/static/codemirror is ok - max file size ~2 MiB - GET only; no write/eval

Default open file: pricklypear.borg when present.

==========

BOOK PDF + TARBALL ==========

Built at deploy/dev time, not per-request:

borge book print . -o pricklypear-book scripts/public_artifacts.sh ? PP_PUBLIC_DIR/book.pdf (or pricklypear-book.pdf) src.tgz (same allowlist as /code)

Image serves: GET /book.pdf application/pdf GET /src.tgz application/gzip from PP_PUBLIC_DIR (default /tmp/pp-public or beside source). Missing artifact ? 404 with short HTML hint, not crash.

==========

AUTH GATE EXTENSIONS ==========

SYSTEM_HARD_PUBLIC gains: /welcome /book.pdf /src.tgz /code and /code/* /assets and /assets/* /.well-known/agent.json /agent.txt (/book is NOT hard-public) (/agent is NOT hard-public – private portal)

Still HARD PRIVATE: / /bindings /edit /repl /data /api /admin /branches /types /pages /agent ?

==========

AGENT DISCOVERY (public bootstrap, no secrets) ==========

Remote agents (Agora, pi, curl on a phone) need a front door that explains how to talk to the image WITHOUT a checked-out repo. Ship two hard-public kernel GETs:

GET /.well-known/agent.json Content-Type: application/json Machine contract: name, version, philosophy, auth scheme (HTTP Basic / session cookie), eval endpoint (POST /api/eval request+response shapes), bootstrap first_calls, rules, curl/python examples, related public links. NEVER include passwords or tokens.

GET /agent.txt Content-Type: text/plain; charset=utf-8 Same content as prose Markdown/plain instructions a freeform agent can ingest wholesale.

Implementation: kernel handlers in image/lib (stable contract), not Nopales routes – discovery must work even when seed/route table is empty. base_url may be derived from Host / X-Forwarded-* when present, else omitted or a relative-friendly note.

Eval stays hard-private: unauth POST /api/eval ? 401. Agents supply image credentials (same as /login).

Anon / ? redirect /welcome (product choice for public host). Logged-in / ? ui/home as today.

========== ENV ========== PP_SOURCE_ROOT – tree for /code and tarball input PP_PUBLIC_DIR – book.pdf + src.tgz PP_LIBS_DIR – existing; layout+welcome seed from here

analytics

Product metrics page: viewers, logins, traffic – observation not work

[status: implemented]

– *agent note:* 2026-08-02: ops_stats + http_ops /analytics + ops/summary prim. Live day rollup; GET /analytics 200.

[verify] build: dune build image; smoke: GET /analytics (authed) shows day rollup: viewers, page views, logins; smoke: (ops/summary) returns structured day stats via /api/eval; smoke: public /welcome increments viewer/page counters; /assets/ excluded; smoke: login success/fail counts match auth_log for the day*

PURPOSE A habitat-native analytics page – not Datadog. Answers: "who showed up, what did they hit, how many logins?" Reliability *counts* appear here; reliability *work* lives in notes/todos (see reliability-work-queue).

SPLIT (product law) - /analytics = observatory (numbers, trends) - notes/todos = workshop (errors become fix work) Never dump raw log lines into the analytics page as the primary UX. Link out to open incident todos.

EMIT (request edge, one place in HTTP middleware) After identity resolve, for each request record: day, path_class, method, is_authed, status_bucket, viewer_key = hash of (client_ip + day salt) – no raw IP in the UI. Exclude /assets/*, /health, static bots path classes (wp-admin, .env probes) from uniques or bucket as "noise". Prefer in-memory day counters flushed periodically; do NOT insert one Postgres row per public request.

AUTH NUMBERS Source of truth: existing auth_log (password/mint ok|fail). Daily login_ok / login_fail via SQL count, not a second write path on POST /login.

ROLLUP SHAPE (v1) In-process Ops_stats for "today" + optional data_rows type ops-daily for prior days (fields JSON): day, unique_viewers, page_views, logins_ok, logins_fail, authed_requests, errors, fatals, top_paths (list). Counters flush periodically (in-memory day counters plus optional ops-daily rows); there are NO per-request Postgres inserts.

SURFACE - GET /analytics (hard-private; any logged-in user in v1, admin-only later if multi-tenant sharpens) - Nopales (ops/summary [day]) (ops/top-paths [n]) - Agent tool ops_summary (read-only) - Nav: link from ui/nav under system/admin cluster when package ops is seeded (or kernel chrome link)

UNIQUE VIEWERS (honest labels) - public_uniques: distinct viewer_key on hard-public paths - logins: successful auth_log password/mint that day - active_users: distinct authed usernames that day Do not conflate the three in the UI copy.

NON-GOALS v1 Session replay, heatmaps, off-box export, per-hit PG inserts, GA-style retention cohorts.

reliability-work-queue

Structured errors ? fingerprint ? note + todo; LLM enriches work items

[status: implemented]

– *agent note:* 2026-08-02: ops_log + ops_incidents fingerprint?note/todo; log-tail + ops/incidents. LLM enrich optional later.

[verify] build: dune build image; smoke: (log-tail 20) returns recent structured events; smoke: Error-level log creates/upserts one open incident fingerprint; smoke: repeat same error bumps count, does not create duplicate todo; smoke: incident filing creates a note body + open todo-item; smoke: optional LLM enrich rewrites title/body without changing counts

PURPOSE Errors should become work. The habitat already is notes + todos; reliability must enter that queue, not only sit on an ops dashboard. Analytics shows counts; this subsection turns severe/new failures into fixable garden items.

STRUCTURED LOG (primary stream) Ops_log ring in the image process: {ts, level, component, msg, context alist, fingerprint} Levels: info | warn | error | fatal. Always mirror to stderr (/var/log/pricklypear/image.log) for operator crash forensics. Bounded ring (e.g. 5k). Closed-writer / handler / seed failures go through Log, not bare Printf.eprintf forever (migrate hot sites).

FINGERPRINT key = component + "|" + normalized msg head (strip UUIDs, paths ids). One open incident per key. Upsert: count++, last_seen, sample_msg, sample_context. New key OR fatal OR watchdog restart ? file work item. Bot 404 noise stays analytics-only (warn or ignored).

WORK ITEM SHAPE 1. data_rows type ops-incident (machine state): fingerprint, level, component, count, first_seen, last_seen, status (open|resolved), note_slug, todo_id, sample_msg 2. notes: durable human/agent readable writeup slug ops-incident-<shortfp> or capture under inbox 3. todo-item: open TODO linked in notes field to the note Dedup rule: if open ops-incident exists for fingerprint, bump counts only – do not mint a second todo.

LLM ROLE (enrichment, not detection) Detection = fingerprints + thresholds (deterministic). LLM optional pass: input = fingerprint + samples + counts + recent log-tail output = better title, explanation, suggested fix/repro Write into the note body / todo title. Never invent counts. If LLM down: still file terse template note+todo. Trigger: on new incident, or admin "Enrich" / hourly batch for open incidents lacking ai_enriched flag.

CLOSE THE LOOP status ? resolved when: - todo marked DONE, or - no matching error for N hours (optional auto), - admin resolves on /analytics Resolved incidents do not re-open on single blip; require count threshold or fatal to reopen.

PRIMS / AGENT (log-tail n [level]) (ops/incidents [status]) (ops/file-incident ...) ; mostly internal Agent tools: read_logs, ops_summary, list_incidents (read); write tools stay effect-gated as today.

WATCHDOG OpenBSD cron watchdog remains the process safety net (scripts/openbsd/pricklypear-watchdog.ksh). On restart it may log fatal via syslog; image boot records a fatal/info "boot after downtime" event when possible. Watchdog is outside the image; do not pretend Nopales can restart itself after a hard process death.

V1 CUT (shipped) - Ops_log ring + stderr mirror - Ops_stats request counters + auth_log login counts - GET /analytics page (kernel) - Fingerprint store in data_rows ops-incident - Auto-file note+todo on new error/fatal - Template body first; LLM enrich if Ai path available - (log-tail) (ops/summary) prims Defer: multi-week charts, bot ML, off-box shipping.

horizon

Long-term possible futures built on this substrate (design only)

[status: implemented]

Possible futures (not commitments):

A. Personal computing environment Always-on personal image; notes/CRM/automations as live functions; browser as desktop.

B. Multi-user community OS system/main commons; personal garden branches; merge reviews; author-runs-as-author capabilities.

C. AI-native software workshop AI cohabits image; proposes types+fns+routes as app sketches; binding-level diffs; provenance; later addressable AST patches.

D. Internal tools platform Team branches; shared business objects; live inspectable tools without redeploy for Nopales-tier changes.

E. Education / research lab Legible multi-user systems + PL teaching with forks/merges and visible isolation.

F. End-user programmable products Friendly skin on same substrate; templates as starter branches; AI drafts, users edit.

G. Hardened multi-tenant service Solo5, real SSH + web login, argon2, mTLS, fleet policies, true setuid data scoping.

H. Dabar – live Bible study libraries BSB reader + Strong's + interlinear as load-library packages (bible/ + dabar/) on the image; not a separate OS. Spec: dabar.borg (root-inlined chapter). Semantic search deferred.

Milestone ladder: now – hacker env + browser login + Nopales product shell (define/edit/route/AI; ui/nav ui/home; auth session) next – product theme Path B (css/* + ui/theme); kill silent OCaml product defaults (nav/home/CSS); emergency-only chrome+css; shell denser; branch promote UI later – environment feel (more system UI in Nopales, permissions, multi-user fleet isolation) horizon – community + AI coworker + Solo5 product

Branch UX + software updates: see branching.borg subsections personal-workspace (implemented), package-updates / promote / branch-browser-ui (planned). Defaults on system/main; garden on <user>/personal; promote is selective merge + git.

Principles to preserve: live system over file tree; store is truth; code branches / data shared; functions compose; AI is an author not god mode; inspectability; isolation by process.

> – *agent note:* 2026-08-02: design-only futures doc; implemented as spec-complete.

19 borge/notes.borg

pricklypear-notes

Pricklypear book chapter: notes – outline-first personal knowledge (org-roam / org-mode / Obsidian-shaped) as Nopales on the one-tree substrate

[status: partial]

> – *agent note:* 2026-08-02 status reconcile: core notes surface is live (doc/get, /notes, search, wiki, roam). Remaining partial: outline-actions (op-protocol refile + full structed set), garden-import (pass-2 uuid edges), tree-cutover finish slices.

notes

Outline-first notes OS: tree documents, wiki links, dailies, capture, hypermedia structure edit, garden import

[status: partial]

> *[verify] build: dune build image; smoke: (load-library "outline") then pure tree ops on a sample node tree; smoke: (load-library "notes") installs /notes and notes/*; smoke: create note with nested outline; GET /notes?slug=? shows foldable tree; smoke: [[slug]] in node body becomes link; backlinks partial lists inbound; smoke: (bible Gen 2 3) in body renders dabar href; unknown book degrades; smoke: (notes/daily) get-or-creates today's daily outline; smoke: pp_eval '(notes/get "?")' returns outline value for agents; smoke: tree: folder + note + heading children round-trip via notes/doc/*; smoke: import dry-run of vault md reports counts without writing*

========== PRODUCT INTENT ==========
Build a personal knowledge environment on the live image inspired by: - org-mode: hierarchical OUTLINES as the document model - org-roam: stable ids/slugs, backlinks, dailies, capture - Obsidian: fast browse, wiki links, folder garden

NOT a port of Electron Obsidian or Emacs. NOT continuous file-backed markdown under libs/ or disk. Notes are durable TREE NODES in Postgres, edited live. One-shot vault import (Obsidian md) is in-scope; sync is not.

WHY THIS OS: Every note/outline is a Nopales value. Agents and humans use the same functions via REPL / pp_eval / htmx. No notes REST API to invent – the image is the API.

========== PRIORITY ORDER (locked) ==========
1. One-tree note substrate (folders, notes, heading children) 2. Outline algebra on heading nodes (structed/outline ops) 3. UUID wiki links + backlinks; bible sexp refs ? dabar 4. Personal garden isolation (not system/main content) 5. Daily notes + capture / inbox 6. Migrate existing PP notes; import Obsidian vault md 7. Tags / search polish – RESOLVED by subsection search-modal 8. Graph visualization (optional, late)

========== PACKAGES – dependency layering (HARD RULE) ==========
The packages form a STRICT LAYERING. A lower layer may not import a higher one. Drift here is a bug, not a style choice.

outline/ – pure tree ADT + structure ops. NO storage, NO notes/, NO roam/. Leaf. Reusable by todo/, plan/, kanban/, tests. structed/ – generic structural-edit algebra. Leaf. bible/ + dabar/ – reader + study; bible sexp render target. May be read by outline render dispatch via a hook passed in, never by import. roam/ – link parse / reindex / render. LEAF over outline + prims only. Does NOT call notes/doc/* or notes/*; callers pass titles/slugs

in. notes/ – persistence fa?ade + routes + capture + dailies + structure-op dispatch + UI. The ONLY package that touches the note store (tree_* / data_rows migrate). Calls outline/, structed/, roam/, bible/ downward. Owns roam/reindex on content writes (notes ? roam, one-way).

ALLOWED EDGES: outline ? structed ? notes ? {roam, bible} roam ? outline (read-only ADT) FORBIDDEN: outline ? notes, outline ? roam, roam ? notes (cycle), notes ? notes (two fa?ades – see storage FA?ADE rule).

========== WHERE IT GOES (routing table – one home per concern) ==========
node ADT (id/title/body/todo/kids), encode/decode, find/update/insert/delete/promote/demote/move, archive/cycle-todo, valid?, todos scrape ..... outline/ generic tree surgery (slurp/barf/wrap/splice/raise) structed/ note read/write/list/search/folder, daily, capture, set-folder, op dispatch (apply-op, node-op, eval), routes + partials + page HTML, backlinks panel host .. notes/ [[slug]] parse, note-link reindex, backlinks query, wiki/media render, render-body ..... roam/ (bible ?) sexp ? /dabar href, abbrev parse ..... bible/+dabar/ legacy data_rows note ? tree migration (one-shot) .... notes/migrate/

========== PACKAGE FILE LAYOUT (one source of truth) ==========
Each package is ONE file: libs/<pkg>/lib.pp. The image seeds libs/<pkg>/lib.pp only (session.ml discover_seed_packages scans top-level dirs). Subdirectory lib.pp files under libs/notes/{outline,capture,doc,refile, tree,ui,migrate}/ are NOT loaded; they are stale pp_sync snapshots and MUST NOT be maintained as a second source. pp_sync pull <pkg> regenerates the single libs/<pkg>/lib.pp from the live branch; that file is canonical in git. Do not re-split notes/ into subdirs without first teaching the loader to stitch them – until then, one file.

========== OUT OF SCOPE (v1) ==========
- Full org-mode syntax (drawers, babel, agenda, clock) - Full CommonMark / markdown WYSIWYG - Force-directed graph as launch requirement - Continuous Obsidian/FS vault sync (one-shot import only) - TeX/PDF/video/EXPORT pipeline ingest from vault - Doltgres / DB engine swap for note history - Per-note function_defs (notes are data, not code) - Real-time multiplayer CRDT on one outline - Full per-lens versioned /data (one-tree placement B); garden isolation is path+author scoped (see personal-garden)

outline-model

Sexp outline ADT: node tree with stable ids – org structure as data

[status: verified]

> *[verify] smoke: outline/node constructors and outline/html produce nested details; smoke: outline/promote demote move-up move-down preserve ids; smoke: round-trip: print tree text then parse back (or JSON props path)*

CANONICAL SHAPE (implementation may use lists/dicts):

A node is a record with: id – stable string (uuid or short id), required title – string, required body – string, optional prose (may contain [[slug]]) todo – "" | "TODO" | "DONE" | future keywords kids – list of child nodes (order matters)

Suggested Nopales construction helpers:

(outline/node title) (outline/node title body) (outline/node* props kids) ; props = alist or dict JSON

Tree = root node. In-memory ADT unchanged. Persistence is heading children under the note path (notes.storage); the legacy fields.outline JSON blob is retired after cutover.

WHY SEXPS: Nested lists ARE outlines. No heading-level parser. pp_expr / pretty printers already exist for readability. Structure edit = pure functions on values.

PATHS: Prefer id-based addressing over index paths so reorder does not break ops: (outline/find tree id) -> node | nil (outline/update tree id f) -> tree (outline/insert-child tree parent-id new-node) -> tree (outline/delete tree id) -> tree (outline/promote tree id) / demote / move-up / move-down

FOLD STATE: v1: browser <details> open attribute only (ephemeral). Optional later: collapsed-ids set in note props or session.

PRINT/PARSE: In-memory: list/ADT via outline/* constructors. On disk after cutover: one heading tree_node per outline node (notes.storage). encode/decode helpers still exist for import, tests, and whole-subtree replace serialization.

PURE PACKAGE (enforced by PACKAGES layering): libs/outline/lib.pp must not touch storage and must not import notes/ or roam/. It is a leaf reusable by other apps (todo/, plan/, kanban/, tests). Two current leaks to close: - outline/render-body dispatches roam/render-body + bible/render-body. Move that dispatch UP to the notes/ UI renderer (the call site outline/html-node is already UI-flavored). outline/ stays pure; the renderer in notes/ picks roam vs bible by inspecting the body. - outline/extra-actions calls notes/refile/extract-form. Move the refile-action injection up to the notes/ caller that assembles the node action menu; outline/ emits a neutral action slot or takes the extra-actions as an argument.

storage

Notes as one-tree nodes: folders, note leaves, heading children; notes/* fa?ade

[status: implemented]

> – *agent note:* 2026-08-02: notes/doc/* fa?ade on tree (libs/notes/doc + lib.pp); live (notes/doc/get "agents") returns uuid/slug/title/outline; tree/garden-root path grammar in tree_store.ml.
>
> *[verify] smoke: notes/upsert then notes/get by slug returns outline; smoke: second upsert same slug updates in place (idempotent); smoke: folder path nesting depth > 1 via tree list; smoke: outline headings persist as heading children under note path; smoke: rename/move note keeps uuid; links still resolve; build: dune build image*

==========

ONE-TREE NOTES (replaces flat data_rows note blob) ==========

Flat CRUD (get/create/update/rows on type note) remains for other product types. Notes leave that path. See also the index-read-path subsection for how every read (index + detail) resolves to the one tree store and how the legacy rows fallback is retired.

PATH GRAMMAR (hierarchical; not data.<type>.<uuid>): data.note.u.<user>.? – user garden root data.note.u.<user>.<folder>? – folder (kind=folder) data.note.u.<user>.?.<slug> – note leaf (kind=note) data.note.u.<user>.?.<slug>.h.<nid> – outline heading (kind=heading)

Readable slug labels: [a-z0-9-]+ only (ltree-safe). Type name segment stays literal "note" (not enc_label base64).

KINDS (tree_nodes.kind; reverse-mirror ignores these – they must NEVER be written as kind=datum or path segment-3 will be misread as a uuid by tree_from_nodes): folder payload body: {uuid, title} note payload body: {uuid, slug, title, tags, kind, day, source_id?, source_path?} NO outline blob – children are truth heading payload body: {id, title, body, todo} kids = real child heading nodes under path

IDENTITY: uuid – stable; wiki links and refile target this slug – last path label of the note; globally unique per user garden (enforce on upsert) title – display only; independent of slug

OUTLINE LOAD/SAVE: In-memory ADT stays outline/* + structed/* (id title body todo kids). Load rebuilds ADT from heading subtree. Save v1 may

whole-replace heading children under note.h in one transaction; fine-grained op?tree/move is a tighten pass, not a blocker.

FA?ADE – ONE, not two (HARD RULE): notes/doc/* is the SOLE note persistence fa?ade. Every read/write/list/search/folder op goes through it; internals call tree prims, not (create 'note ?). Agent and UI ABI is notes/doc/get|upsert|list|set-folder|?. The legacy flat notes/* fa?ade over data_rows (notes/get, notes/upsert, notes/row, notes/make-fields, notes/apply-op, notes/capture, notes/refile 3-arg, notes/get-outline, notes/get-title, notes/do-save, notes/ensure-inbox, notes/daily, notes/cycle-node-todo, notes/node-op) is RETIRED. It is not reachable from any route and must be deleted from libs/notes/lib.pp; nothing may call it. New work adds to notes/doc/* (or notes/<feature>/* under the notes/ package), never revives notes/<flat> names. notes/doc/* accessors (notes/doc/title, /outline, /tags, /kind, /day, /folder) replace the legacy notes/get-* ones.

KERNEL PRIMS (tree_store + thin register; additive): (tree/get path) (tree/list prefix [depth]) (tree/upsert path kind type body-json) (tree/delete path [recursive?]) (tree/move old-path new-path) – subtree path rewrite (tree/find-by type field value) (tree/get-uuid uuid) Flat get/create/update/rows ABI unchanged for todos/etc.

NOTE-LINK: see links-and-roam (UUID edges).

NOT IN function_defs / system/main seed: Note *content* is never package defaults. pp_sync does not pull garden notes. notes/* outline/* roam/* CODE may promote to main; content stays in the user garden path.

LEGACY: Existing data_rows type=note rows migrate once (M0) into tree nodes; after cutover notes/* does not write them. folder string field and JSON outline field are retired. M0 must EXPLODE each outline blob into real heading child nodes under the note path (no inline outline blob), and normalize path labels to the kebab slug. After every uuid verifies in the tree, the legacy data_rows type=note rows are DELETED (no permanent inert archive).

HISTORY: v1 overwrite on save (updated_at). No Doltgres.

links-and-roam

Wiki links [[slug]] as UUID edges; bible sexps ? dabar; media embeds
[status: implemented]

> – *agent note:* 2026-08-02: libs/roam (parse-links via extract-links prim, reindex note-link from_uuid/to_uuid, backlinks, render-body/media); live roam/backlinks on agents. Refile still ad-hoc package not op-protocol – tracked under outline-actions.
>
> *[verify] smoke: body with [[other]] creates note-link UUID edge on save; smoke: (roam/backlinks slug) lists inbound links after resolve; smoke: render turns [[other]] into anchor using current slug/title; smoke: move/rename note does not require reindex; links still resolve; smoke: refile/extract of a node does not require roam/reindex (uuid stable); smoke: [[media:uuid|alt]] renders img; not indexed as note-link; smoke: (bible Gen 2 3) renders /dabar href; (bible ?) not a note-link; smoke: Obsidian [[bible:Genesis2:3|?]] import normalizes to bible sexp; smoke: (extract-links body) matches roam/parse-links edge cases; test: image primitives_test extract-links cases*

==========

NOTE-LINK = UUID EDGE (not slug pair as source of truth) ==========

TYPE note-link (data row or tree datum – implementation choice; shape is what matters): from_uuid – source note uuid (stable) from_id – source heading id ("" = whole-note) to_uuid – target note uuid (stable) to_id – target heading id optional label – optional display label kind – "wikilink" | future Optional cached from_slug/to_slug for debug only – never authoritative. Index by (from_uuid, to_uuid). Live package: libs/roam (parse-links, reindex, backlinks, render-body, backlinks-html).

ROAM IS A LEAF (enforced by PACKAGES layering): roam/ imports outline/ (read-only ADT) and prims only. It MUST NOT call notes/doc/* or notes/*. The two current roam?notes calls are removed by pushing the lookup up to the notes/ caller: - roam/backlinks needs source titles for display ? the notes/ui backlinks panel resolves titles/slugs from the rows roam returns (uuid + from_id + label), or roam returns uuid/id/label only and notes/ui decorates. - roam/render-link resolves the target note row ? pass the resolved (slug,title) IN from notes/ui; roam renders the anchor from args, or notes/ui renders and roam just supplies the parse. roam/reindex is called BY notes/doc on content writes (notes ? roam, the one allowed direction); roam never calls back up into notes/doc to resolve a uuid.

AUTHORING SYNTAX (bodies, still human-friendly): [[slug]] – wiki by slug at write time [[slug|label]] – labeled wiki [[media:uuid|alt]] – media embed (not note-link) (bible BOOK ch [verse]) – sexp bible ref (not note-link) (bible BOOK ch verse-from verse-to) – optional range BOOK accepts full name or bible abbrev (Gen, John, ?); normalized via bible/abbrev-of + bible/parse-ref helpers. No #node-id deep links v1.

SAVE-TIME RESOLUTION: [[slug]] -> lookup note uuid (tree/find-by / notes/doc); store edge uuid?uuid. Missing target: stub-link without edge until target exists (prefer). Bible sexps are NOT note-link rows.

REINDEX: (roam/reindex note-uuid outline-tree) replaces edges for from_uuid. Called on content-touching saves only (set-body, upsert with body change, op=eval target=body). Move/rename/refile/extract does NOT reindex – note and heading uuids/ids stay stable; edges remain valid. Cached from_slug/to_slug may go stale; resolve via uuid at read.

========== REFILE / EXTRACT vs LINKS (align tree-browser + op-protocol) ==========
Refile and extract are STRUCTURAL moves of outline subtrees across note documents – they are NOT link-graph operations. Today: libs/notes/refile (ad-hoc dual upsert). Target: structural-editing.op-protocol owns the wire POST {op, target, args} ? fragment + OOB selection named ops e.g. refile / extract-note with args (dst-slug) or (new-slug?) – server applies pure tree surgery then whole-doc write via notes/doc/*. image.tree-browser deliberately defers refile/move UI until those ops exist on the shared protocol (no parallel /tree/refile routes). notes.outline-actions already lists this gap. When protocol lands: - notes/refile and notes/refile/extract become thin adapters or disappear into op dispatch - roam edges need no rewrite (uuid stable) - tree-browser may offer move only via the same ops

RENDER: Wiki: resolve to_uuid ? current slug/title/path; anchor /notes?slug=<current>. Broken uuid ? muted missing. Media: unchanged img path. Bible sexp: template via bible/dabar – (bible/link-html book ch verse [label]) -> ? optional hx-get into #dabar-panel when chrome present. Kernel extract-links / render-links gain bible-sexp recognition (budget-safe OCaml), or Nopales pre-pass normalizes sexps before render-links – prefer OCaml once syntax is frozen.

IMPORT NORMALIZE (garden-import): Obsidian [[bible:Genesis2:3|Label]] and bare [[bible:Hosea4:6]] become (bible Hosea 4 6) text in heading bodies (Label dropped or becomes visible prose adjacent – keep label as link text via (bible Hosea 4 6 "Hosea 4:6") if three-or-four arg form includes optional label string).

QUERY: (roam/backlinks slug-or-uuid) -> inbound with current titles/slugs for display. UI partial: notes-backlinks.

NOT v1: - graph view renderer - unlinked-mentions auto-discovery - block-ref transclusion - path-qualified wiki links for duplicate slugs - tree-browser-initiated refile (waits on op-protocol)

budget-scale

Per-call eval budget: stress contracts for notes/roam/outline so large trees and bodies never Budget_exceeded on ordinary UI/agent paths

[status: implemented]

[verify] build: dune build; test: dune runtest – lisp list-hof + image extract-links; smoke: scripts/verify_budget_stress.py (monitor-rerunnable); smoke: parse: 500+ wiki links in one body completes; count matches; smoke: reindex/upsert: 10 nodes x ~8k body with links indexes all links; smoke: render: outline/html on large tree/body returns HTML, no budget error; smoke: lazy: a many-node note opens with eager-depth nodes only; expanding a collapsed node fetches its subtree partial; no budget error; smoke: many-node ops: N structure ops each in own eval (HTTP-shaped); smoke: (budget-remaining) positive mid-call; floor asserted after stress cases

WHY: The 100k step default_limit is a safety net, not a perf governor. Char-scanners and recursive list walks in Nopales burned the budget on ordinary notes (upsert reindex, large inbox render) while the write sometimes committed first – silent stale links. Hot paths move to OCaml or kernel special forms; stress tests guard regressions.

BUDGET MODEL (lisp-interpreter.eval): - One budget per eval/call_handler invocation - tick on each eval step; Budget_exceeded aborts that call - OCaml prims cost one tick at the call site (body free) - (budget-remaining) -> steps left (dynamic current budget) - (bench expr-string [n]) -> avg us/iter for calibration Do NOT "fix" scale bugs by raising default_limit.

KERNEL / HOT PATHS (required): - list/map, list/filter, list/foldl: special forms (OCaml loop + apply); not O(n) Nopales recursion frames - extract-links: OCaml; roam/parse-links uses it - select-for-update: error outside with-transaction (reliability adjacent to long evals) Planned if stress fails render/tree: OCaml render-links (or roam/render-body prim) cheaper outline encode/decode/find if tree ops dominate

STRESS SUITE (scripts/verify_budget_stress.py): Monitor-rerunnable against a running image (dev or habitat) via pp_eval. Synthetic fixtures only – do not mutate user garden notes without a dedicated test slug prefix (e.g. budget-stress-*) cleaned up after.

Suites: 1. PARSE / EXTRACT - correctness edges (3 links, 0, unclosed, [[]], media) - scale: single body with >=500 links completes - optional: 10k-char link-free body still cheap 2. REINDEX / UPSERT - tree: >=10 nodes, each body >=8k with >=1 link - notes/doc/upsert returns ok (not budget exceeded) - note-link count for slug == expected wiki links - budget-remaining floor after call (implementation picks a conservative floor; document in script) 3. RENDER - outline/html (or show-body without request deps) on large tree: many nodes and/or one ~32k+ body - must return non-empty HTML string; no budget error - wiki + media syntax appear as anchors/img when hook is roam/render-body 4. MANY-NODE ACTIONS - create synthetic note with >=50 child nodes - each op in its own eval (mirrors HTTP call_handler): cycle-todo, move-up/down, archive, set-title - extract one node to its own note; refile one node - every op ok; final tree consistent; cleanup slugs 5. THRESHOLD PIN (optional / nightly) - binary-search max body size for parse vs render - log or store result; not a hard CI flake gate unless threshold collapses below known-good baseline

ACCEPTANCE: verify_budget_stress.py exits 0 on local dev.sh image and on habitat after deploy. verify_notes_execute_all.py remains the broader feature smoke; budget script owns scale. Failures file as product bugs (reindex/render), not "raise the limit".

OUT OF SCOPE HERE: - Jane Street Incremental / BI pipelines - raising default_limit as the primary fix - preemptive multi-request chunking of a single note view

outline-actions

Per-node outline action menu; the eval op turns nodes into a live Nopales surface

[status: partial]

– *agent note:* 2026-08-02 confirm still partial: notes/outline/apply-op + eval + POST /partials/notes-node-op shipped; notes/refile* exist but not on structural-editing.op-protocol; full structed wrap/splice/raise not all on outline menu.

[verify] smoke: structure ops: add-child/sibling delete move promote/demote archive cycle-todo set-title/body; smoke: slurp-forward / barf-forward via structed/ on outline accessors; smoke: op=eval target=child splices returned string as a new child node; smoke: op=eval target=sibling splices; target=body replaces this node body; smoke: eval expr binds self/doc/slug to current context; smoke: eval reindexes roam links when a body changes; smoke: refile: node moves to another note root kids; smoke: refile destination is a lazy typeahead modal – the note page never enumerates the garden (BORGE-1785603757); smoke: extract: node becomes its own note; removed from source; smoke: POST /partials/notes-node-op returns outline partial + selection; smoke: scripts/verify_budget_stress.py many-node actions suite*

SHIPPED SURFACE (libs/notes/outline + outline + structed): notes/outline/apply-op dispatches structure ops on the outline tree value. Ops present today: add-child add-sibling delete move-up move-down promote demote set-title set-body set-todo cycle-todo archive unarchive slurp-forward barf-forward (via structed/* + outline/kids outline/set-kids – op-protocol preview) Selection companion: notes/outline/apply-op-sel ? (list tree' sel) for OOB restore.

WIRE (reconcile – code wins over older spec path): POST /partials/notes-node-op params: slug, op, node-id [, title|body|todo|expr|target] NOT /partials/notes-op (stale name). Fragment path ? no chrome (wants_fragment). Client hx-target #outline hx-swap outerHTML; selection via hidden OOB when present.

op = eval Form: notes/outline/eval-form ? same partial endpoint. notes/outline/eval-result loads (lambda (self doc slug) <expr>) and applies it. target in {child, sibling, body} via notes/outline/eval-target (default child). result handling by target: body as-str ? outline/set-body on node child string ? new node title; valid outline node sexp ? splice subtree under id sibling same as child, insert after id

REFILE / EXTRACT (libs/notes/refile – NOT yet op-protocol): notes/refile src-slug nid dst-slug – detach node from src outline, add-child under dst root, dual upsert. notes/refile/extract – detach ? new note slug from title, remove from source. These are whole-doc outline writes via notes/doc or legacy notes/upsert. When structural-editing.op-protocol lands for cross-doc moves, refile/extract MUST become named tree-ops on that protocol (same POST shape, server-authoritative tree), not parallel ad-hoc routes. image.tree-browser defers refile/move UI until then.

REFILE DESTINATION UI (lazy modal, BORGE-1785603757): No note page renders an eager <select> of all notes – that O(n) notes/refile/options over (notes/doc/list) blew the 100k step budget at garden scale. Refile target picking is a lazy htmx modal: the toolbar 'refile' <details> and GET /partials/notes-refile?src=SLUG both render notes/ui/refile-modal-body – a typeahead over /partials/notes-search?mode=refile (filtered in Postgres, no garden enumeration) plus a raw slug input for agents / keyboard-only users. The legacy eager builders (notes/refile-form, notes/refile/form and the O(n) notes/refile-options / notes/refile/options) are REMOVED. notes/refile/move performs the detach+attach write.

PERSISTENCE: Structure ops upsert through notes/doc/* (tree store). Body-touching ops should roam/reindex (verify smoke).

WHY: Notes view = live Nopales surface. wiki-llm-loop is the first major consumer of op=eval splices.

SECURITY: eval runs full session prim set incl DB – treat as REPL. Auth gate only boundary v1; v2 may confirm destructive prims.

GAPS (keep status partial until closed): - full structed op set (wrap/splice/raise/?) not all wired into apply-op / keymap - refile/extract not on op-protocol wire

yet - budget stress suite must stay green on garden-scale

wiki-llm-loop

LLM-maintained wiki over the notes corpus – ingest, query, lint as Nopales ops
[status: implemented]

> – *agent note:* 2026-08-02: libs/wiki/lib.pp – wiki/ingest-from-source, wiki/query, wiki/lint, wiki/events, wiki/schema, wiki-event rows; live wiki/schema ok.
>
> *[verify] smoke: (wiki/ingest-from-source TEXT) creates/updates a source note + entity/concept notes + note-link rows; smoke: (wiki/query QUESTION) synthesizes with [[slug]] citations; may file-back a kind=query note; smoke: (wiki/lint) returns orphans/contradictions/missing-pages/suggested-sources; smoke: each top-level wiki op writes one wiki-event row; (wiki/events) lists them; smoke: agents drive ingest/query/lint via eval (pp_eval); no /wiki/* routes require registration*

PATTERN: A persistent, compounding wiki over curated sources: the LLM does not retrieve raw chunks per query; it reads the existing corpus, integrates each new source into the notes (page updates, cross-links, contradiction flags), and keeps synthesis current. The corpus IS the knowledge base.

SHAPE: Notes ARE wiki pages; kind discriminates role. The agent drives operations through ordinary Nopales functions exposed to the REPL; the user reaches the same operations through the outline-actions eval op on a source node. No /wiki/* routes for v1 – the existing /notes surface plus the live eval surface cover the user-facing flow.

BUILD ORDER (dependencies): 1. image.per-request-context – so notes routes survive the cotenant race: anonymous attach flips the shared Session.t routes/env mid-request and any personal-branch route 404s under traffic until per-request-context lands. Deployable on system/main as a documented workaround in the meantime. 2. notes.links-and-roam (note-link rows + roam/parse-links + roam/reindex + roam/backlinks). 3. notes.outline-actions (the eval op for LLM splices).

NOTE KINDS (convention over the existing note row schema; widening the note.kind vocabulary – no new storage): note existing default (behavior preserved) daily existing dailies source source-summary page (per ingested raw source) entity a named thing (person, place, work) concept an abstract topic query a filed-back synthesis with citations synthesis cross-source integration/comparison schema the corpus preamble note (see below) log reserve; not used v1 (audit is wiki-event rows)

DOC-LEVEL FRONTMATTER (extends the note row, NOT per-node metadata – avoids a node-schema migration): updated_at timestamp of last ingest/query touch updated_by "llm" | username | "agent" source_count for source/entity/concept: number of distinct sources contributing; incremented on ingest

SCHEMA-NOTE CONVENTION: A note with kind=schema holds prose describing this corpus's conventions, page disciplines, and ingest emphasis; co-evolved by the human and the LLM. (wiki/schema) returns its body string, seeded with a sensible default at first call. ingest/query/lint read it as the LLM preamble. There is no separate schema-agent path – it is just another note the loop reads.

OPS (each is one top-level Nopales call; each writes one wiki-event row on success): (wiki/ingest-from-source source-text [slug-hint]) 1. create/update a source note (kind=source) carrying the source text + extracted summary outline. 2. read (wiki/schema) + (roam/backlinks)/index over the current corpus; build the prompt. 3. (ai/ask prompt) -> a JSON plan: { summary_node, entities[], concepts[], links[{from,to,kind}], followups[] } 4. notes/upsert each

entity/concept page (kind=entity | concept), composed with the source's contribution; bump source_count; stamp updated_at / updated_by. 5. (roam/reindex) the affected slugs. 6. write one wiki-event row (kind=ingest, op-slug, ts, summary). Return the new source-summary slug + the list of pages touched.

(wiki/query question) read (wiki/schema) + the most relevant pages (roam / find-text over the corpus index); synthesize with [[slug]] citations. Optionally file-back as a kind=query note that child-references its cited pages. One wiki-event row (kind=query).

(wiki/lint) one (ai/ask) pass over schema + index + note-link edges; report orphans, contradictions, stale claims, missing pages (concepts mentioned but lacking their own page), suggested new sources. Optionally draft stubs for missing entities (opt-in create-stubs? arg). One wiki-event row (kind=lint).

AUDIT: TYPE wiki-event (rows; fields: kind op slug ts summary): append-only, branch-shared, one row per top-level wiki op (ingest/query/lint). NOT autonomic – only these three top-level ops write events. Helper calls (roam/backlinks, outline/eval, etc.) do not write events. Retention is opt-in: (wiki/events [since]) -> list recent events (wiki/prune-events days) -> drop events older than N days (explicit, manual) Corpus content evolution is recorded in the notes themselves (updated_at/by frontmatter); the wiki-event stream is operational audit, not content of record.

LIVE TELEMETRY (separate stream, unchanged infra): ingest/query/lint emit Info/Error into the existing Ops_log ring (component "wiki") so a long op is visible via (log-tail). Bounded 5000 + rotates; not a permanent audit. Two streams, two purposes: Ops_log = live observability of this request; wiki-event rows = audit trail of what the wiki has done to itself.

NOT v1 (deferred): - URL fetch / clipper (user pastes text first) - markdown (outlines ARE the substrate; no marked.js) - graph view renderer (note-link rows hold the structure) - unlinked-mentions auto-discovery - block-ref transclusion - images / attachments - batch / unsupervised ingest sweep

dailies-and-capture

Daily outlines and quick capture (org-roam-dailies / org-capture shaped)
[status: implemented]

> *[verify] smoke: (notes/daily) returns same slug for same day on second call; smoke: (notes/capture title body) creates inbox-tagged note or appends; smoke: GET /notes/daily opens today*

DAILY: slug convention: daily-YYYY-MM-DD kind = "daily", day = YYYY-MM-DD (notes/daily) / (notes/daily "2026-07-11") get-or-create root outline with title "Daily YYYY-MM-DD" UI: /notes/daily , menu link under apps or notes

CAPTURE: (notes/capture title body) -> new slug (slugify title + short id), tags include inbox, kind note, outline = single node with body Optional: (notes/capture-to slug title body) appends a child node under root (inbox into a project outline)

INBOX REVIEW: /notes?tag=inbox list partial – phase 2.

hypermedia-ui

Routes, partials, structure-edit via htmx; menu entry
[status: implemented]

> *[verify] smoke: GET /notes lists recent notes; smoke: GET /notes?slug=? full page with chrome + outline; smoke: GET /partials/notes-outline?slug=? fragment no chrome; smoke: structure op POST returns updated outline partial; smoke: ui/menu can link Notes without breaking other apps; smoke: note page does not enumerate all notes (refile target is a lazy typeahead, not an eager <select>)*

ROUTES (full page = chrome; /partials/ and HX-Request = fragment):
GET /notes GET /notes?slug=NAME GET /notes/daily GET /notes/edit?slug=NAME ; optional dedicated editor POST /notes ; create / save / capture GET /partials/notes-list GET /partials/notes-outline?slug= GET /partials/notes-backlinks?slug= GET /partials/notes-node?slug=&id= ; lazy subtree returns one node's <details> subtree HTML POST /partials/notes-op ; structure edit params: slug, op, id, ?

STRUCTURE OPS (POST notes-op): op = insert-child | delete | promote | demote | move-up | move-down | set-title | set-body | set-todo Always reindex links if body changes. Response: outline partial HTML (hx-swap target #outline)

RENDER: Prefer <details class="ol-node"> fold (no heavy JS tree). Ops as small buttons on each row (htmx).

LEAN NODE RENDERER (outline/html-node): Each node is emitted with the fewest html/* calls feasible – constant attribute fragments are precomputed at library load and assembled with direct string-append, not via the fine-grained html/attr / html/attrs / html/span DSL on the hot recursive path. The DSL stays available for page chrome and one-off fragments; the recursive outline renderer is the budget-critical path and must not compose dozens of function applications per node.

LAZY SUBTREE LOADING: Eager rendering is capped at a shallow depth (root + its direct children). Deeper nodes render as a collapsed <details> whose body is fetched on first expand via hx-get=/partials/notes-node?slug=&id=, swapped in place. This bounds per-render html-skeleton cost by the eager depth, not by total node count, so a large note never Budget_exceeded on ordinary open. The partial returns the same html-node markup the eager path would have produced for that subtree, so eager and lazy nodes are visually identical. Structure ops still re-render and swap #outline.

MENU: Add (menu/link "notes" "/notes") under apps in default ui/menu when notes is dogfood-stable; or document manual ui/menu edit + pp_sync pull.

COMPOSITION: notes/page embeds partials like todo/page embeds todo/list-html – same hypermedia discipline.

index-read-path

Notes index reads resolve to the tree store exactly once; legacy data_rows fallback retired; interim read-layers removed

[status: implemented]

> *[verify] smoke: GET /notes lists garden folders incl. tree-only notes; smoke: GET /notes?folder=garden/bible/scripture_concepts lists its notes; smoke: (notes/list-light) equals notes/doc/index-list (both tree); smoke: (notes/doc/get slug) and /notes?slug=? agree for tree-only slug; smoke: no binding under notes/light/* remains on the image; smoke: no call site of (rows 'note) outside notes/migrate/legacy-**

PROBLEM (drift this closes): The index accumulated overlapping read layers while the one-tree cutover was landing: notes/doc/* – dual-read fa?ade (tree first, legacy data_rows fallback) notes/ui/* flat – budget-safe flat index over notes/doc/index-list (tree via OCaml tree/note-index) notes/tree/render – original nested folder tree; walks folders with Lisp list ops; exceeds the eval budget at garden scale notes/light/* – live-only nested index reading legacy data_rows ONLY; became the routed /notes handler, so tree-only (garden) notes vanished from the UI while agents (fa?ade) still saw them One /notes route pointed at whichever layer happened to be loaded last. Result: index and detail disagreed; garden invisible in UI; budget-safe code unreachable.

TARGET – ONE READ PATH: Every notes read (index and detail) resolves through notes/doc/* to the ONE tree store exactly once: index /notes

? notes/page ? notes/ui/folders-html (flat, budget-safe via OCaml tree/note-index) detail /notes?slug=? ? notes/ui/show ? notes/doc/get agent notes/doc/get|list|search – unchanged (fa?ade) notes/list-light is re-pointed at notes/doc/index-list so agent and UI see the same rows.

SLICES: I1 Route /notes to notes/page (git-managed, flat index). Delete the live-only notes/light/* bindings; nothing in libs/ references them. I2 Retire notes/tree/render + render-folder (budget- unsafe at garden scale). Keep only as an explicitly renamed non-routed legacy binding, or delete. I3 notes/list-light ? notes/doc/index-list (same rows, same order); drop its private legacy-rows read. I4 Retire the legacy data_rows fallback in notes/doc/* (list / row / find-slug). Tree is the only store. Precondition: M0 (migrate legacy ? tree) verified for every legacy uuid; legacy rows kept as an inert archive, not read. Audit gate: no call site of (rows 'note) / (find-rows 'note) outside notes/migrate/legacy-*.

AUDIT (I4 gate): Full-tree scan of libs/ + image/ confirms every note read outside the fa?ade's own fallback already goes through notes/doc/* – no package (todo/roam/wiki/flashcards/bible/ dabar/ai/calendar/capture/ui) reads rows 'note directly. Only notes/migrate/legacy-* touches legacy, by design. I4 is therefore a fa?ade-internal deletion, not a cross-package change.

NON-GOALS: - Re-raising the eval budget to save notes/tree/render (budget is a safety net; the nested Lisp walk is the bug, per budget-scale) - Changing the slug-page outline editor or structure ops - Merging/renaming garden folders for display (kebab vs underscore is cosmetic; path-based folders tolerate both). Folder GROUPING is path-derived (not the legacy folder string), and the garden/ import prefix strips to top level in the index per garden-import DISPLAY.

CUTOVER COMPLETION (the finish line this subsection gates): The legacy nested index (notes/light/*) must actually be deleted from the image (I1) so /notes serves the flat budget-safe index over tree/note-index, and that index groups by PATH-derived folders with the garden/ prefix stripped to top level. Until then the UI shows only the legacy-folder rows and hides garden/tree-only notes. Legacy data_rows type=note rows are DELETED after M0 re-migration verifies every uuid (no permanent archive).

F6 DEAD SUBPACKAGE DIRS (libs/notes/<sub>/lib.pp): The subdirectory files libs/notes/{outline,capture,doc, refile,tree,ui,migrate}/lib.pp are NOT loaded by the image (loader scans top-level libs/<pkg>/lib.pp only) and have drifted from the canonical libs/notes/lib.pp (e.g. stale notes/outline/apply-op missing op-seq 409; notes/capture/ title truncation differs). They are a readability hazard – a maintainer opens the subpackage file thinking it is authoritative. DELETE the subdirectory lib.pp files; the single libs/notes/lib.pp is canonical (see PACKAGES file layout). Backups/ may retain old copies. This slice is pure deletion; no route or binding changes.

agent-access

Agents use notes via eval – no separate notes HTTP API

[status: implemented]

> *[verify] smoke: pp_eval notes/list and notes/get; smoke: agent-task tool list may include notes_search later*

LAPTOP / PI: scripts/pp_eval.py '(notes/daily)' scripts/pp_eval.py '(notes/search "sync")' scripts/pp_eval.py '(notes/get "pp-design")'

IN-IMAGE AGENT: Optional phase 2 tools on agent-task: notes_search, notes_get, notes_capture Do not grant blind outline/delete without policy.

PRINCIPLE: Adding a notes feature = adding Nopales functions. Remote agents gain them automatically through eval.

personal-garden

Garden notes are personal content – not system/main; path+author isolation
[status: implemented]

– *agent note:* 2026-08-02: tree_store garden root data.note.u.<user>; notes/garden-root ? tree/garden-root; notes/* list scoped via tree index not system/main seed.

[verify] smoke: notes created while on <user>/personal land under data.note.u.<user>; smoke: notes/ list default scope is current user garden only; smoke: pp-sync snapshot/pull never writes garden note bodies into libs/; smoke: system/main checkout does not seed or clobber garden notes*

CODE vs CONTENT: notes/* roam/* outline/* packages may live on and promote to system/main (shared product code). Note *bodies*, folders, heading trees, note-link edges for the garden MUST NOT be treated as package defaults and MUST NOT be seeded from libs/*.

WHY NOT "data branch" literally (v1): branching.data and one-tree placement A keep /data shared (no branch_id). Full per-lens versioned data is placement B and deferred. Garden isolation v1 is: 1. path prefix data.note.u.<username>.? 2. author = session user on write 3. notes/* list/get/search default to that prefix 4. never promote note rows via package seed/reseed That meets "none of these notes live in main" as product content without breaking shared bible/todo data. (* agent note (|2026-08-23 MAKE: data lens namespace (data.l.<lens>.<user>.<type>.<id>) is implemented v0 – see one-tree.borg branching-interaction. The garden isolation rules (author = session user, user-scoped list/get, never seed) are unchanged. Notes' legacy path grammar (data.note.u.<user>.?, type-before-user, confirmed live on prod 2026-08-23) migrates to data.u.<user>.note.? as a separate make task – one migration to the final shape.|) *)

MULTI-USER: Alice does not list Bob's data.note.u.bob subtree via the default notes UI/API. Admin escape hatches are explicit.

WORK BRANCH: Humans edit garden while checked out on <user>/personal (or dev for code spikes). Content path is user-keyed, not branch-keyed, so checkout does not hide your notes.

garden-import

One-shot Obsidian/markdown vault import into the personal garden tree
[status: partial]

– *agent note:* 2026-08-02: scripts/import_obsidian_notes.py dry-run+apply, bible:[[?]]?sexp, garden/ folder prefix, collision slugify. Still missing pass-2 uuid note-link reindex after upsert (verify smoke).

[verify] smoke: dry-run on vault root reports md/folder/link counts; zero writes; smoke: apply imports md under data.note.u.<user>.garden.?; smoke: headings become heading nodes; frontmatter title/org_id kept; smoke: [[wiki]] resolved to uuid edges in pass 2; smoke: [[bible:?]] normalized to (bible ?) sexps; smoke: skip EXPORT, .obsidian, tex/pdf/video/aux; smoke: re-import same source_path/org_id updates in place

SOURCE (example): /home/roerick/obsidian_vault ~227 md files; also tex/pdf/EXPORT/video – not imported.

SKIP: .obsidian/**, EXPORT/**, templates/** (optional later), *.{tex,aux,log,pdf,mp4,webm,mov,pyc,png,base}

ROOT PREFIX: data.note.u.<user>.garden.<vault-relative folders?>.<slug> Keeps import namespace clear of native PP wiki paths; refile later is tree/move.

DISPLAY: The garden/ prefix is an import namespace, not a user- facing folder. The /notes index STRIPS a leading garden/ so imported contents appear at top level as siblings of native folders (roam, reading, history as top-level folders; no garden bucket in the UI).

FILE MAP: directory -> folder nodes (title = dir name) file.md -> note leaf slug <- filename stem sanitized [a-z0-9-]+ title <- yaml title | first H1 | stem source_id <- yaml org_id if present source_path <- vault-relative path (re-import key) tags <- yaml tags kind/day <- Diary/YYYY-MM-DD.md => daily + day # / ## / ? -> heading spine; preamble = root body body markdown -> heading body text (light normalize)

LINKS (two-pass): Pass 1: create folders + notes + headings (no edges). Pass 2: scan bodies for [[?]] and bible forms; write uuid edges; normalize bible: Obsidian links to sexps.

SLUG COLLISIONS: Global per-user uniqueness. On clash, prefix with parent folder label or numeric suffix; report in import log.

TOOLING: Offline importer (OCaml CLI or pp-eval batch driver). –dry-run required before apply. Not an image hot path. Idempotent on source_id/source_path.

NON-GOALS: Continuous sync, tex pipeline, quartz theme fidelity, dataview, round-trip export to the vault.

tree-cutover

Phased delivery: kernel tree prims ? fa?ade ? UUID links ? M0 PP migrate ? M1 vault

[status: partial]

> – *agent note:* 2026-08-02 confirm: K0-N1/M0/I* largely done (index-read-path implemented). Open: M1 pass-2 links, F finish-cutover slices (legacy light/* residue, heading explode polish, legacy row delete).
>
> *[verify] smoke: K0: tree/get list upsert delete move on folder/note/heading; smoke: K1: tree/get-uuid + slug uniqueness reject; smoke: N0: notes/doc/* round-trip on tree with heading children; smoke: N1: folders via path; no folder string field on write; smoke: R0: uuid note-link + bible sexp render; smoke: M0: existing PP notes migrated; spot-check agents + inbox; smoke: I4: notes/doc/list/row tree-only; no (rows 'note) outside migrate; smoke: GET /notes flat index lists garden/tree-only notes; smoke: tree-browser (tree/render-doc slug root depth fo) walks notes outline; smoke: M1: vault dry-run + apply on personal garden*

SLICES (one initiative, shippable checkpoints): K0 tree prims + readable note path labels K1 get-uuid + slug unique constraint/index N0 notes fa?ade on tree; outline?heading (whole-replace ok) N1 folders via path; retire folder field writes R0 UUID links + bible sexp render via bible/dabar M0 migrate data_rows note ? tree (same uuids) I* index-read-path (I1-I4): one UI/agent read path through notes/doc/* ? tree; flat /notes; light/* deleted M1 Obsidian md import (garden-import) – still open D0 docs: one-tree flag defaults truth; design note appendix

DEPENDENCY: Does NOT wait on global PP_TREE_READS=1 for all CRUD. Notes use tree_* prims directly. Flat types keep CRUD.

STATUS NOTE (post index-read-path I1-I4): K0/K1/N0/N1/M0 + I1-I4 are realized. notes/doc/list and notes/doc/row are tree-only (no data_rows fallback). Legacy type=note rows remain as inert archive; only notes/migrate/legacy-* may (rows 'note). /notes routes to notes/page ? notes/ui/folders-html (flat, budget-safe via tree/note-index). notes/list-light aliases index-list. Nested notes/tree/render retired (legacy tombstones). Remaining for full "implemented": M1 garden-import apply, R0 completeness if any bible/roam gaps, D0 docs polish.

FINISH-CUTOVER (the close-out slices): F1 Delete the legacy nested index (notes/light/*) from the image so /notes serves ONLY the flat budget-safe index over tree/note-index (I1 completion). F2 Index groups by PATH-derived folders; a leading garden/ strips to top level (garden-import DISPLAY). F3 M0 completion: explode every migrated note's inline outline blob into real heading child nodes (the Roman notes currently render as one blob with a single

heading); normalize path labels to kebab slug. F4 Delete the legacy data_rows type=note rows after F3 verifies every uuid in the tree (no archive). F5 Vault drift: pp-sync the live notes bindings back to libs/ so git matches the image before further edits.

TREE-BROWSER BACKEND SWAPS (content tree contract): image.tree-browser v1 walks the CONTENT tree – the notes outline model (outline/id title kids) rooted at a note slug – NOT tree_nodes paths. Contract that must stay stable when storage backends move: (tree/render-doc slug root-id depth folders-only?) -> html string node accessors used by the walker: outline/id, outline/title, outline/kids, outline/find + notes/doc/get ? outline (today notes/doc/outline) Swapping outline JSON-in-note for heading nodes under data.note.? MUST keep those accessors (or a thin adapter with the same names). tree/render itself does not call tree/get or rows. Store-tree roots (v2) are a different root kind and need lens-aware prims (filed separately).

RISK: Never write hierarchical notes as kind=datum. Reverse mirror only handles fn/type/route/datum.

unified-todos

Basecamp lists + org node todos; unified /todos with smart views

[status: implemented]

Real lists: todo-list + todo-item (list_id, title, notes, state, due, position). state in todo/state/list (Nopales). Note todos: outline node.todo; scrape + write-back. Smart views: everything, from-notes, default Inbox. /todos nav + body; cycle state; create list; add item. No dual-copy of note todos until promote (later).

> *[verify] smoke: /todos everything + from-notes + inbox; smoke: cycle note TODO write-back; smoke: create list and add item*

kanban

Board view over todos: one todo-list as a board with its own columns (per-list state set), cards draggable via tree actions

[status: implemented]

2026-12 campaign 9.0: flipped to implemented. BORGE-1786336755 closed - the per-list state-cycle threading is live: todo/item/cycle + todo/cycle read the item's list's OWN columns (single source of truth todo/state/columns-for-list, which falls back to todo/state/list) and the detail <select> uses list-aware todo/state/options-for-list, so a custom-columns list cycles exactly its columns in /todos, the todo detail, and /kanban alike. There are no duplicates - /todos detail and /kanban read the same function.

A kanban is a VIEW over the existing todo model, not a new data type. A board IS one todo-list; its columns ARE that list's OWN state set (a 'columns' field on the todo-list datum); its cards ARE that list's todo-items. No kanban-card type, no kanban-board type. This is the first product consumer of the tree prim family as a DATA API (one-tree.tree-datum-prims): every mutation is a tree action over a datum node (the list for column config, the cards for moves), not a create/update call.

PER-LIST COLUMNS, NOT A GLOBAL STATE LIST. The todo package ships a default state list (todo/state/list: "", TODO, DONE, WAITING, CANCELLED). A list that does not set its own 'columns' falls back to that default – so existing /todos and existing lists are unchanged. A list that sets 'columns' overrides the default for itself: the board renders exactly those columns, in that order, and a card's 'state' is one of them. This makes the todo state model per-list configurable instead of a single fixed global – the same mechanism answers "set your own columns" and "set your own todo state": edit the list's 'columns' field. (The default list stays as the fallback and the seed for new lists; it is not deleted.)

BOARD = ONE TODO-LIST. The /kanban route takes a list_id (defaults to the Inbox). A list-picker switches boards. Adding a card adds a todo-item to the list (create, the id-generating convenience); the board re-reads.

CARDS = todo-items of the list, grouped by state. Read via tree/list over data.u.<user>.todo-item.* plus a list_id filter, OR via the todo package's existing todo/items-for-list helper composed with tree/list. Each card shows title + due; click opens the existing todo detail. Cards whose ‘state‘ is not in the board's current columns render in a trailing "other" bucket so a removed/renamed column never silently hides cards.

MUTATIONS ARE TREE ACTIONS (the point of this subsection + the prerequisite one-tree fix). The list datum and each card datum are tree_nodes; every mutation is tree/get + dict-set* + tree/upsert at the same path: - MOVE CARD BETWEEN COLUMNS (drop on a state column): tree/get the card's datum node, dict-get its body, dict-set* the ‘state‘ field to the target column name and ‘position‘ to the drop index, tree/upsert the new body at the SAME path. Goes through the reverse mirror so (get 'todo-item id) and (rows 'todo-item) see the new state, exactly as if update had written it. - REORDER WITHIN A COLUMN (drop between two cards): same tree/get + dict-set* ‘position‘ per moved card; wrap the N affected cards in with-transaction so the reorder is atomic and no two cards share a position mid-drop. - ADD / REMOVE A COLUMN: tree/get the LIST datum, dict-set* its ‘columns‘ field to the new array, tree/upsert the list at the same path. Removing a column does NOT delete its cards (they land in the "other" bucket until reassigned). - RENAME A COLUMN: tree/get the list, dict-set* the new name into ‘columns‘, AND cascade – in one with-transaction, tree/upsert every card of that list whose ‘state‘ is the old name with the new name. Rename is all-or-nothing across list + cards. - No re-parenting / tree/move in v1: a card's list is a field (list_id), not a path segment, so moving a card to another board is a field update, not a path change. tree/move is available per one-tree.tree-datum-prims but unused here.

UI / HTMX SHAPE: - GET /kanban?list=<id> renders html/page with a flex row of columns. Each column is a <section> with an hx-post drop target; each card is a draggable article with a tiny dragstart/drop JS shim that encodes the dragged card id + drop position into hidden form fields, then submits the column's hx-post. Same POST-to-one-route + hx-request? partial-vs-redirect pattern as /todos. - A column header has inline add/rename/remove controls (small forms, same hx-post pattern); the board footer has an "add column" form. - POST /kanban dispatches on action: move (card to column + position), add-column, rename-column, remove-column. Each performs the tree action(s) above and returns the refreshed board partial via render-html when hx-request?, else redirects. - No JS framework, no new asset pipeline – the shim is a small <script> in the page, same style as the outline keyboard-nav script (must pass scripts- check per notes-ui-contract). - Accessibility: each card reachable by keyboard; move via a per-card up/down/left/right control as a no-DnD fallback (the DnD shim is progressive enhancement, not the only path).

RELATION TO /todos: the /todos page keeps its list/detail smart-view UX; /kanban is a sibling route over the same rows. Both read/write the same todo-item datum nodes; no duplication, no sync. Cycling state from /todos updates the kanban and vice versa because they share the store. A list with custom columns shows those same states as the /todos state cycle options for that list (the state machine reads the list's ‘columns‘ when present, else the global default).

NON-GOALS (v1): - No WIP limits, no swimlanes, no per-user assignment columns, no card cover images. - No multi-list board (board = exactly one list). - No tree/move re-parenting of cards across types. - No per-USER state set (columns live on the LIST, not the user; a user's different lists may differ).

[verify] build: dune build @all; smoke: GET /kanban?list=<inbox> ren-

ders the inbox's items grouped into columns by the list's columns (or the default state list when the list has none), empty columns included; smoke: a list with its own columns field renders exactly those columns; /todos for that list cycles only those states; smoke: drag a card to another column (POST action=move) ? the todo-item's state field changes via tree/upsert; /todos and /kanban both reflect it; smoke: add / remove / rename a column ? the list's columns field updates via tree/upsert; rename cascades to cards in one with-transaction; smoke: reorder within a column ? positions renumber atomically (with-transaction); no two cards share a position; smoke: keyboard no-DnD fallback moves a card left/right between columns

search-modal

Typeahead find modal: one server search feeds open-note and refile; flat grouped results down to header level

[status: implemented]

– *agent note:* 2026-08-02: notes/search + notes/ui/search-partial; route /partials/notes-search; live GET ?q=agents ? 200 grouped hits.

[verify] smoke: GET /partials/notes-search?q=? returns grouped result HTML (docs + header rows with ? breadcrumbs); smoke: results arrive without enumerating the garden in Nopales (filter happens in Postgres); smoke: modal opens via keyboard shortcut; arrows navigate; Enter opens chosen row; smoke: empty query and no-match queries render stable non-error markup; smoke: refile entry point uses the same typeahead; choosing a row refiles the selected node; smoke: node-targeted rows refile under that heading; doc rows refile at root

PRODUCT: One find-box is the "open note" interface AND the "refile destination" interface. Cmd/Ctrl-K opens a modal with a single input; typing shows results immediately. This subsection resolves priority-7 "search polish".

INTERACTION MODEL (locked: flat, grouped): Every keystroke requeries the server; the result list is ONE flat stream grouped under doc headings, header-level matches indented beneath their doc with a breadcrumb path: JOURNAL 2026 ? July 12 session TODO Projects ? pricklypear Arrow keys move, Enter activates. NOT a two-tier drill-down (?/? to enter docs): slower when you already know the header word, and refile wants doc+node targets in one list anyway.

RESULT ROW: (doc-slug doc-title node-id|() header-path matched-field) node-id () = doc-level hit (title/tags match). header-path = "Gran ? Parent ? Header" for tree cutover storage; pre-cutover (fields.outline JSON blob) only doc-level rows are emitted – header granularity ARRIVES WITH the tree cutover, it is not retrofitted onto the blob. matched-field lets the UI badge title vs body vs header hits.

SEARCH STRATEGY (v1 – tree-backed): Notes are tree-backed (notes/tree/enabled? #t), so the search runs over tree_nodes, NOT data_rows. It uses the tree/search-text prim (BORGE-1785605474): one Postgres ILIKE query over tree_nodes scoped to the garden root (path <@ $prefix::ltree) for note rows (title/tags/slug) plus a second query for heading rows (title/body with a ?-joined header_path breadcrumb), merged + deduped by slug in OCaml, capped (?20 rows). Empty/no-match queries return (); garden-wide Nopales enumeration is FORBIDDEN on this path (BORGE-1785603757: ~3k notes ? eager enumeration blew the 100k step budget) – filtering happens in Postgres. Prim signature: (tree/search-text needle [limit]) – garden root (tree/search-text prefix needle [limit]) – explicit path Note: data_rows find-text (fields-»f ILIKE, crud.ml) is NOT used – it would miss the tree garden. Doc-level rows carry empty node_id; header rows include header_path + node_id (refile to a header parents under that node). Ranking v1 in OCaml: exact-slug first, title hits, then header, then body.

Fuzzy/typo tolerance (pg_trgm) and ranked FTS (tsvector, cf. dabar.search-fts) are later upgrades that swap the query behind the SAME partial – the modal does not change (the prim signature is the seam). Track under .borge-bugs when demanded by use; not spec'd here.

NON-GOALS (v1): - No load-the-whole-tree-into-JS client search - No tsvector/trgm migration (see above) - No cross-type global search (notes only; a global launcher is a product-experience question, later) - No search inside archived/hidden docs unless the current notes/doc/list filter already shows them

UI / HTMX SHAPE: Follows the details-popover precedent of notes/ui/refile-modal (BORGE-1785603757) – no JS framework, no new asset pipeline. Modal is a <details> (or <dialog> if focus trapping proves needed) opened by clicking "find" or Cmd/Ctrl-K (tiny script, same style as the existing outline keyboard-nav script). Input has hx-get="/partials/notes-search" hx-trigger="input changed delay:200ms" hx-target on the results container. Arrow/Enter handling extends that script; htmx 'htmx:afterSwap' re-focuses the first result.

ROUTES (all private, current branch): GET /partials/notes-search?q=? ? grouped result HTML GET /partials/notes-search?q=?&mode=refile ? same list, rows refile the selected outline node (POST action=refile with dst_slug + optional dst_node_id) instead of navigating. Open-note activation navigates to /notes?slug=? (&node=? for header rows when the tree cutover lands).

REFILE CONVERGENCE: notes/ui/refile-modal's raw dst_slug text input gains a mode=refile typeahead against the same partial (exact slug entry keeps working – no regression for agents / keyboard-only users who know the slug). Refile to a header target parents the subtree under that node; refile to a doc row parents at root (current behavior). The existing POST /notes action=refile handler accepts an optional dst_node_id.

BUDGET INVARIANT: /notes?slug=? page render must stay O(1) in garden size: the modal ships collapsed markup only; all searching is on-demand partials. Any future caching (e.g. a per-branch doc-index value) is optional and must be invalidated conservatively; premature caching is a non-goal.

20 borge/structural-editing.borg

pricklypear-structural-editing

Pricklypear book chapter: structural editing – paredit-shaped tree surgery as one shared protocol over outlines and code ASTs

[status: partial]

> – *agent note:* 2026-08-02 status reconcile: structed-algebra+op-protocol+keymap+non-goals implemented; only ast-structure-view remains planned – keeps project/section partial.

structural-editing

Structural editing: named tree ops over one shared protocol – outlines and code ASTs are edited as trees, never as paren text

[status: partial]

==========

PRODUCT INTENT ==========

Programs are trees. Outlines are trees. Editing either one should be TREE SURGERY with named operations – slurp, barf, wrap, splice, raise – never character editing over a serialized rendering. This chapter defines ONE interaction protocol for structural editing, shared by the notes outline editor (notes.hypermedia-ui) and the function editor's structure view (image.function-editor), and later by any tree-shaped surface.

The paredit vocabulary (slurp / barf / wrap / splice / raise / split / join / transpose) is the headline, but the chapter is really three rules:

RULE 1 – You type inside atoms; you never type structure. contenteditable is for LEAVES (titles, bodies, atom text). Structure changes only via named commands. Unbalanced structure is unrepresentable.

RULE 2 – The server is authoritative for structure. The browser sends (op, target, args); the image applies a PURE function to the tree and returns a re-render plus the new selection. The client MAY apply that same pure op locally for immediate feedback via the browser evaluator (client-eval) – the result is COMPUTED by the same source the server runs, not predicted from a divergent client codebase – and reconciles to server truth by morph on response. What is forbidden is structural prediction from a SECOND, hand-maintained client implementation of the op algebra (the lie).

RULE 3 – Selection is part of the op algebra. Every op returns (tree', selection'): where the cursor lands after delete/slurp/wrap is computed by the same pure function that computed the tree, and travels back with the response.

STORAGE IS UNCHANGED. Outlines stay JSON strings in data_rows; function bodies stay JSONB in function_defs. The unification is the value model (nested lists/atoms) and the op protocol – NOT a storage codec migration. Whole-document write remains the conflict domain.

==========

SUBSECTIONS ==========

structed-algebra – pure higher-order op library op-protocol – the POST contract + selection OOB keymap – chords now, modal target ast-structure-view – /edit Text|Structure toggle client-eval – browser evaluator: same code, optimistic execution non-goals – what this chapter refuses to build

structed-algebra

Pure higher-order tree-surgery library (structed) usable from REPL, routes, and agents

[status: implemented]

PACKAGE: structed/ (Nopales – pure, no DB, no I/O). Load: (load-library "structed").

HIGHER-ORDER OVER CHILD ACCESS. One op implementation serves any tree whose domain supplies child accessors:

(structed/slurp-forward get-kids set-kids tree target)

The outline backend passes (outline/kids, outline/set-kids); the AST backend passes expr-children accessors. Do NOT duplicate op logic per domain, and do NOT force one node shape across domains.

OP SET (v1): slurp-forward target absorbs its next sibling as new last child slurp-backward target absorbs its prev sibling as new first child barf-forward target's last child becomes its next sibling barf-backward target's first child becomes its prev sibling wrap target becomes sole child of a fresh node (fresh-node shape is domain-supplied; the expr domain takes an optional head-atom arg: (wrap t "f") on x -> (f x), so wrapping never needs an empty head – see ast-structure-view) splice target removed; its children take its place, in order raise target replaces its parent split target divides at a position into two adjacent siblings join target merges with its next sibling transpose target swaps with an adjacent sibling The existing outline ops promote / demote / move-up / move-down are the outline-domain special cases of this vocabulary and stay where they are.

TOTALITY: every op is total. Unknown target, missing sibling, root-guard violation, wrong shape -> identity tree and UNCHANGED selection. A structure key must never produce an error page.

RETURN SHAPE: (list tree' selection'). Selection rules: slurp / barf / wrap / transpose : selection stays on target splice : first lifted child, else parent raise : stays on target (now at parent's place) delete (domain op) : next sibling, else prev, else parent add-child / add-sibling : the new node

PROPERTIES TO TEST (Nopales-level via load-string + eval; no services, no browser): slurp-forward then barf-forward = identity on the tree wrap then splice = identity on the tree any op on empty / atom / unknown target = identity

> *[verify] smoke: (load-library "structed") exposes structed/*; smoke: slurp+barf and wrap+splice round-trip to identity on a sample tree; smoke: ops against root/unknown/empty targets return input unchanged*

op-protocol

One POST contract: (op, target, args) -> re-render + out-of-band selection

[status: implemented]

WIRE: structure edits and leaf edits are the same protocol.

POST {op, target, field args...} -> 200 HTML: re-rendered tree fragment (outerHTML swap) + out-of-band selection state (hidden element swapped via hx-swap-oob carrying the new selection address).

The client shell restores highlight + caret after each swap. This extends the notes/outline/apply-op pattern: dispatch by op name to a pure fn; unknown op -> (tree, same selection), still a 200 re-render.

SURGICAL WIRE, WHOLE-DOC STORAGE. Payloads name a node and small args – never the whole document. The server decodes the whole tree, applies the pure op, encodes, writes the whole document (data_rows field or function_defs body). The DOCUMENT is the conflict domain; no per-node storage in v1.

ABSTRACT TARGET IDENTITY. Outline surfaces address nodes by stable id (already true). The AST surface addresses nodes by PATH (data-path="0.2.1") until branching.ast-nodes provides stable node ids. Op responses always return the selection's CURRENT address, so path invalidation after reorder is handled by construction. Upgrading identity (paths -> ids) must not change the protocol or the client.

RESPONSIVENESS VIA SAME-CODE EXECUTION. The client runs the same pure op locally through the browser evaluator (client-eval) for immediate

feedback, then the server confirms authoritatively and the response is MORPHED in – never merged field-by-field with the prediction, never patching client state. Because the local result is computed by identical source, the divergence window is server-only effects (validation, concurrency), not code drift; an opseq guards staleness so an out-of-order response cannot clobber a newer local state. The htmx-org hx-optimistic extension (template placeholder, swapped wholesale on response) remains available for non-structural previews but is not the structural mechanism. The client shell may ignore structure keys while one op POST is in flight (single-flight), and restores selection + focus from the OOB payload on completion.

FLUSH RULE. Any structure POST first flushes the focused leaf's pending edit (set-title / set-body / set-atom) – either folded into the same request or as a forced blur-commit – so a tree re-render can never destroy in-progress leaf text. This replaces any narrower "commit on blur" rule.

UNDO. Whole-tree snapshots per op are the undo model – trees are persistent values, snapshots are cheap. Ring placement (client or server session) is v1 implementation discretion.

> *[verify] smoke: POST outline op returns tree partial + OOB selection element; smoke: delete selects next sibling; at end of siblings selects parent; smoke: unknown op name re-renders unchanged tree with 200*

keymap

Chord-first keyboard map; modal structure/text interaction is the documented target

[status: implemented]

> – *agent note:* 2026-08-02: v1 Alt chords shipped in notes outline script (libs/notes/lib.pp): Alt+Up/Down move, Alt+Shift+Left/Right promote/demote, Alt+Left/Right barf/slurp, Alt+Enter sibling; preventDefault. Modal map still documented target not v1.

V1 – MODIFIER CHORDS ONLY. Plain keys always edit text when a caret sits inside a contenteditable leaf; structure never fires on an unmodified key. Required chord set (outline surface; AST surface mirrors):

Alt+ArrowUp / Alt+ArrowDown transpose with prev / next sibling Alt+Shift+ArrowLeft / Right promote / demote Alt+ArrowRight slurp-forward Alt+ArrowLeft barf-forward Alt+Enter add sibling

The full op set stays reachable via toolbar/menu. No structure key fires during IME composition.

CHORD CONFLICT (known – verify early). Alt+ArrowLeft / Alt+ArrowRight is browser history back/forward on Linux and Windows. The map relies on preventDefault winning when the editor has focus: VERIFY THIS FIRST in the keymap work, in the target browsers, before building more on it. If preventDefault does not hold, slurp/barf move to Alt+Shift+ArrowLeft/Right and promote/demote relocate (e.g. Alt+, / Alt+.) – update the table then. Do not add per-OS keymaps in v1.

TARGET – MODAL (not v1 acceptance): a structure mode (a whole node is selected; plain keys move selection and operate; Enter enters the title as text) and a text mode (caret in leaf; Esc returns to structure mode). Server and protocol are mode-agnostic – modal is a pure client evolution. Client code MUST keep a distinct keymap layer (a data-driven chord table, not scattered hard-coded checks) so the modal map can replace the chord map without touching op dispatch.

ONE CLIENT SHELL. Selection state, postOp, blur-commit, restore-after-swap, and the keymap layer are ONE script pattern shared by the outline editor and the AST structure view – generalized from the notes outline script, not forked per surface.

> *[verify] smoke: Alt+ArrowRight on a selected node slurps its next sibling; smoke: Alt+ArrowLeft/Right never trigger browser history navigation while the editor is focused; smoke: plain keys with caret inside a leaf edit text only; smoke: structure op in flight: second chord ignored; selection restored after swap*

ast-structure-view

Function editor gains a Text | Structure toggle over the same function_defs row

[status: planned]

> – *agent note:* 2026-08-02 confirm still planned: no Text|Structure toggle on /edit/<name>; function editor remains textarea. structed algebra + outline op-protocol ready as substrate.

/edit/<name> gains a view toggle: Text (existing textarea, parse whole body on save) | Structure (AST rendered as addressable nested HTML – the outline/html-node ADDRESSING pattern (data-path on every node) applied to exprs, NOT the outline's row layout). Lists are structural containers; atoms are contenteditable leaves.

LAYOUT IS A PRETTY-PRINT, NOT AN OUTLINE. The structure view must NOT render one node per line – that would make (+ 1 2) taller than it is wide and code unreadable. The renderer is a pretty-printer over the AST: - atoms, and lists that fit within a width budget, render INLINE on one line: (+ 1 2), (dict-get u "name") - a list breaks vertical only when over budget, or by form-aware convention (define / lambda / let / cond bodies one form per line; operator applications stay horizontal while they fit) - vertical children indent by depth; all whitespace is derived, never stored – the AST carries no layout bytes Selection and data-path addressing are layout-independent: a node keeps the same path whether its children flow horizontally or vertically, and structural ops behave identically either way. Layout direction is a per-domain RENDER decision – outlines render one node per row because that IS the outline reading experience; code renders as a pretty-print. The shared op protocol does not care.

BUILD ORDER (layout): 1. VERTICAL RENDERER FIRST: every list breaks vertical, one node per line – the degenerate pretty-printer (width budget zero). Easiest correct render, and it exercises the hard parts (data-path addressing, selection restore, ops, atom edits, re-render) before any layout engineering. NOT a throwaway: because addressing and ops are layout-independent, stage 2 changes only the renderer. 2. INLINE PACKING: introduce the width budget and form-aware conventions; short forms collapse onto one line. Pure renderer change – protocol, ops, paths, and the client shell are untouched.

BACKEND: the same structed algebra with expr accessors. The AST is an expr tree (App / Lambda / Let / If / Quote ...), NOT a tree of VLists: child access is defined PER CONSTRUCTOR as the subexpressions in reader/surface order (App -> head then args; Lambda -> params then body; If -> test, then-branch, else-branch). Quoted literals are OPAQUE LEAVES in v1 – no descending into quoted data. The exact per-constructor child table is implementation discretion within that constraint. Domain differences: wrap takes an optional head-atom – (wrap t "f") on x -> (f x); with no arg it wraps in a fresh one-element list. split/join operate on list element runs; there is no promote/demote – raise/splice cover that space.

ATOM EDITS ARE TRANSACTIONAL AND TYPE-AWARE. The renderer tags each leaf with its atom type. Blur sends set-atom {path, text}; the server validates PER TYPE: - string leaves: text is accepted RAW – spaces and parens are data, not syntax - symbol / number / bool leaves: text must read-string as exactly one atom of a compatible shape, else the old value is kept and the tree re-renders unchanged - converting a leaf's TYPE (symbol ->

string, number -> symbol) is an explicit op, never a side effect of editing text Because invalid text never enters the AST, HOLES are not needed in v1; a hole node type is a possible later extension, deliberately out of scope here.

SAVE IS UNCHANGED: whole function body, JSONB, via the existing value<->JSON codec and the function_defs row. The structure view is a second editor over the same storage – not a new persistence path.

AI AUTHORING: op + path + args with whole-body write is the surgical patch surface ai-authoring asks for, ahead of branching.ast-nodes. When ast-nodes lands, paths upgrade to stable ids behind the same protocol (see op-protocol abstract identity).

> *[verify] smoke: GET /edit/<fn> structure view renders nested tree with data-path leaves; smoke: layout stage 2: a short form like (+ 1 2) renders on ONE line, not one node per line; smoke: layout stage 2: define/lambda body breaks vertical by convention; selection paths stable across both layouts; smoke: set-atom with unparseable text keeps old value and re-renders; smoke: set-atom on a string leaf accepts raw text containing spaces and parens; smoke: slurp via POST re-renders; saved body reflects the change on reload*

client-eval

Browser build of the Nopales evaluator runs the same pure op algebra the server runs, so an optimistic edit is a real local computation – not a predicted guess – reconciled to server truth by morph

[status: implemented]

SAME EVALUATOR, SECOND TARGET. The pure interpreter core (lisp/lib: reader + ast + eval + env, no I/O) is compiled to JavaScript via js_of_ocaml. OCaml 5 effects require the CPS backend: every JS build passes –effects=cps, or it fatally aborts. The same .pp package source the server serves is evaluated verbatim in the browser – structed, outline, html ride along as data, never reimplemented.

SAME SOURCE AT RUNTIME, not at build. The browser fetches the current branch's package source on page load (the image is the API) and evaluates it, so a user who redefines structed/barf-forward on their personal branch has their browser run their version. Build-embedded source is a boot fallback only – it would drift the moment a promote lands without a redeploy, breaking the branching chapter's live-source invariant.

PURE PRIM TABLE ONLY. The browser prim table registers the pure prims and EXCLUDES every DB / IO / net prim. The pure/DB split already exists in code (prims_pure.ml vs prims_db.ml); this subsection makes it structural – prims_pure is factored into a library both image/ and client/ depend on, so the two runtimes cannot drift on what "pure" means. Each registered prim name is bound as VPrim in the browser env, mirroring session.ml's auto-bind loop, so (list ...), structed/*, outline/* resolve. The capability boundary IS the prim table: a structed op cannot touch Postgres because those prims are simply unbound.

OPTIMISTIC EXECUTION MODEL. On a structure op: 1. client applies the same pure op fn to its local tree 2. renders via the same render lib (libs/html / outline) 3. morphs the result into the DOM with a pp-in-transit class on the affected subtree (grey = unconfirmed, the chat-app sending/sent indicator – never a claim of truth) 4. fires the real POST carrying an op-seq 5. server: stale seq -> 409, client morphs server truth back; else persist + return server-rendered fragment 6. client morphs the fragment in; class clears. If the two renders agree (same code, deterministic) the morph is a no-op except grey -> normal. error -> toast + morph to server truth.

OP-SEQ. A monotonic sequence per editing session travels with each op; the server rejects out-of-order ops so a late response cannot clobber a newer local state. Single-user edits serialize naturally. Multi-user concurrency is

unchanged from op-protocol: the document is the conflict domain, last-write-wins on whole-doc save.

WHY THIS IS NOT THE LIE. The objection to optimistic updates (optimistic updates are a rendering trick pretending to be application truth) lands when the optimistic result is a GUESS from a second codebase. Here the local result is produced by the SAME evaluator running the SAME source; the only divergence sources left are server-only effects (validation, uniqueness, concurrent writers) – exactly what the op-seq + morph + toast path exists to surface. There is no second implementation to drift.

CONFORMANCE IS THE GATE. The lisp conformance suite (lisp/test/conformance_test.ml) runs against the JS target, so client/server agreement is a tested invariant, not a hope. The client-eval subsection is not accepted until that gate is green.

NOT A REPLACEMENT FOR THE PROTOCOL. The op-protocol POST contract, OOB selection, flush rule, and single-flight shell are unchanged. client-eval only changes what the client does BETWEEN keypress and response: compute locally with the same code, instead of showing a static placeholder.

> *[verify] build: dune build client/ yields interp.js; js_of_ocaml invoked with –effects=cps; test: browser evaluator passes lisp/test/conformance_test.ml against the JS target; smoke: page load fetches current-branch structed/outline/html source and evaluates it without error; smoke: op click: local apply renders grey; server confirm morphs and clears grey; matching renders make the morph a no-op; smoke: stale op-seq: server 409, client morphs server truth back; smoke: op error: toast shown and DOM morphed to server truth*

non-goals

What this chapter deliberately does not build

[status: implemented]

- NO paredit-over-text: no paren surgery inside a textarea or CodeMirror buffer. The tree is the editor; text buffers remain for the Text view and /code browsing. - NO one-node-per-line as the FINAL code layout: vertical rendering is build stage 1 (degenerate pretty-printer), not the accepted end state – the end state is inline flow with width/convention breaks (ast-structure-view). - NO structural prediction from a DIVERGENT client codebase (op-protocol). Same-evaluator local execution (client-eval) is sanctioned because the result is computed by identical source, not guessed; what remains forbidden is a second, hand-maintained client implementation of the op algebra. Display-placeholder optimism via the hx-optimistic extension stays allowed for non-structural previews the response replaces wholesale. - NO storage codec migration: outlines stay JSON strings in data_rows; code ASTs stay JSONB in function_defs. A canonical sexp writer prim (write-string) for agent-facing output may be added independently under lisp-interpreter.primitives; this chapter does not depend on it. - NO per-node storage and NO multi-user concurrent node editing: the document is the conflict domain. - Modal keymap is the documented target, not v1 acceptance (keymap). - Hole AST nodes (ast-structure-view). - SINGLE-NODE SELECTION ONLY in v1: no ranges, no multi-select, no region ops. - KILL / YANK (subtree cut-copy-paste, ideally as sexp text on the system clipboard) is deferred, not ruled out – it is how larger code moves will work later. - STALE WRITES: whole-document save makes two tabs on one document last-write-wins in v1. This inherits image.function-editor's deferred optimistic- concurrency / stale-body-reject item; resolve both together.

> – *agent note:* 2026-08-02: documentation-only; implemented as spec-complete.

21 borge/calendar.borg

pricklypear-calendar

Pricklypear book chapter: calendar
[status: implemented]

– *agent note:* 2026-08-02 status reconcile: sole child model already implemented; libs/calendar/lib.pp ~802 lines; live calendar/today + 27 events; GET /calendar 200.

calendar

Month/day calendar: events + due todos + daily notes
[status: implemented]

model

Naive local dates; shared data_rows; Nopales package
[status: implemented]
A calendar should load fast and stay out of the way: month grid + day agenda is enough to plan a week, and naive local dates keep the model honest (no timezone UI pretending to be correct).
CHOICES (v1): - Day click opens day agenda (not auto-create) - Naive local YYYY-MM-DD (no TZ UI) - Seeded on boot after todo - Month + day only (no week view yet)
DATA: calendar-event title day start_time end_time all_day notes location color source
OVERLAYS (read): todo-item.due on day notes daily-YYYY-MM-DD presence
PRIMS: date-today date-weekday date-format
ROUTES: /calendar ?ym= ?day= Package: calendar/*

[verify] smoke: GET /calendar shows month grid; smoke: GET /calendar?day= shows agenda + add form; smoke: create event appears on day

22 borge/flashcards.borg

pricklypear-flashcards

Pricklypear book chapter: flashcards – decks + spaced-repetition cards as Nopales + data_rows

[status: implemented]

flashcards

Decks, cards, study queue, FSRS-4.5 review, hypermedia UI, agent API

[status: implemented]

> *[verify] build: dune build image; smoke: (load-library "flashcards") installs flash/? and routes; smoke: GET /flashcards lists decks; create deck appears; smoke: add card to deck; GET deck page lists it; smoke: GET /flashcards/study?deck=? shows due card front; reveal + rate; smoke: rate again/hard/good/easy updates due + stability; card leaves or re-enters queue; smoke: pp_eval '(flash/due)' returns due cards for agents; smoke: seed_stdlib loads flashcards after calendar (or documented order)*

========== PRODUCT INTENT ==========

A first-class memorization app on the live image, inspired by: - Anki: decks, front/back cards, spaced review grades - SuperMemo/FSRS: stability + difficulty + due, scheduled by algorithm - Pricklypear: objects in Postgres, UI as Nopales routes, agents study/create via eval – no parallel REST API

NOT a port of Anki desktop. NOT file-backed .apkg as the authority (import may come later). NOT OCaml system pages – this is a product package (Nopales).

Killer property: flashcards are Nopales values; agents and humans use the same functions.

========== LAYERING ==========

- libs/flashcards/lib.pp – package flash/? (seed on system/main) - data_rows types – flash-deck, flash-card (shared) - Routes via (route ?) – /flashcards and study/edit paths - Optional nav entry – ui/menu pin or seed menu link - OCaml – only existing CRUD/date prims; no new kernel prims required for v1 if date-today (calendar) or string due compares suffice

========== DATA MODEL (v1) ==========

Types (deftype on load; fields are documentation + form UI):

flash-deck title string description string color string (optional CSS token / hex) archived "true" | "" (hide from default list)

flash-card deck_id uuid string of flash-deck front string (prompt) back string (answer) tags string (comma-separated free text v1) suspended "true" | "" (excluded from study) ;; FSRS-4.5 scheduling state (all strings in jsonb v1): stability string float default "0" difficulty string float default "0" interval string int days default "0" reps string int default "0" lapses string int default "0" due string YYYY-MM-DD (empty or past => due now) last_reviewed string YYYY-MM-DD or empty created string YYYY-MM-DD optional

NO separate flashcard-review log table in v1 (history deferred). NO cloze / image occlusion / media blobs in v1. NO per-user row ACL – data_rows are shared (existing rule). Multi-user isolation of card content is a later multi-user.permissions concern, not this chapter's first cut.

Orphan cards (missing deck_id) show under an "(orphan)" virtual deck in the UI; agents can reassign.

========== SRS ALGORITHM (v1 – FSRS-4.5 via kernel prim) ==========

Ratings (buttons + flash/review rating arg): again | hard | good | easy

State on each card: stability (float ? 0), difficulty (float ? 0), interval (int days), reps (int), lapses (int), due (YYYY-MM-DD).

On review with rating R, today = date-today: again: reps = 0; lapses += 1; interval = 0; due = today stability = max(0, stability - 0.2) hard: interval = max(1, floor(interval * 1.2)) stability = max(0, stability - 0.15) reps += 1; due = today + interval good: if reps = 0 then interval = 1 else if reps = 1 then interval = 6 else interval = floor(interval * stability) reps += 1; due = today + interval easy: if reps = 0 then interval = 4 else interval = floor(interval * (stability + 1)) stability = stability + 0.15 reps += 1; due = today + interval

New cards: due empty or due <= today, reps=0 ? introduce in study. Suspended cards never enter the queue. Order: due ASC, then session-freshness (last_reviewed < today or blank, before last_reviewed == today), then created/id; so a card rated Again – due today, last_reviewed today – sinks to the bottom of the current stack and is not re-shown until the rest of the due batch is cleared. Limit batch (default 20) per study page.

The core scheduling is delegated to the kernel prim (fsrs-schedule stability difficulty elapsed rating) which returns (stability' difficulty' interval_days). See libs/flashcards/lib.pp flash/apply-rating for the thin Nopales wrapper.

Date math: reuse calendar/image date prims if present (date-today, and a day-add helper). If day-add is missing, file issue / add a small flash/add-days using existing prims only – do NOT invent a second calendar system.

========== PACKAGE API (flash/ prefix) ==========

Decks: (flash/decks) -> list of (id . fields) (flash/deck id) -> fields | () (flash/deck-create title [description]) -> id (flash/deck-update id fields-or-keys?) (flash/deck-archive id)

Cards: (flash/cards deck-id [limit]) -> list (flash/card id) -> fields | () (flash/add deck-id front back [tags]) -> id (flash/update id ?) (flash/suspend id) / (flash/unsuspend id) (flash/delete id) ; hard delete v1 OK (confirm in UI)

Study: (flash/today) -> "YYYY-MM-DD" (flash/due [deck-id] [limit]) -> due cards (not suspended) (flash/review id rating) -> updated fields | error string rating in again|hard|good|easy (flash/stats [deck-id]) -> simple counts (due, new, learning, suspended) as printable structure (flash/explain front back) -> explanation text from the LLM Ask the LLM to explain a card the user doesn't understand. Uses agent-session-new with a tutor seed (no tools); returns the model's text. Wrapped in an escaped HTML div by the route.

HTML / routes (handlers are flash/?-page functions): (flash/install) or load-side effects: (route "/flashcards" "flash/home-page") (route "/flashcards/deck" "flash/deck-page") (route "/flashcards/study" "flash/study-page") (route "/flashcards/explain" "flash/explain-page") (route "/flashcards/card" "flash/card-edit-page") ; optional Exact query keys are implementation discretion; document in lib.pp header. Prefer ?deck=uuid & ?id=uuid.

Naming: package directory libs/flashcards/, binding prefix flash/ (short, like todo/). Types flash-deck / flash-card (not flashcards-deck).

========== HYPERMEDIA UI ==========

/flashcards - list decks with due counts - form: new deck (title) - link to study-all (due across decks) optional

/flashcards/deck?id= - deck title, description - card table (front truncated, due, stability) - add card form (front, back, tags) - Study this deck

/flashcards/study?deck= (deck optional = all) - if no due: empty state + links - else one card: show FRONT [Reveal] ? show BACK + rating buttons - POST rating ? flash/review ? redirect next due - plain form POST (same as todos); htmx partials optional polish

/flashcards/explain?id= (htmx GET, returns an HTML div) - reads the card front+back, asks the LLM to explain it in plain language, returns an escaped with the explanation. - On the study card (after reveal), an Explain button does an htmx GET to this route; the response swaps into a

#flash-explain div below the card. A htmx-indicator span shows Thinking? during the request. No tools on the LLM session; no image context needed.

Mobile-first: big rating buttons, minimal chrome noise.

Use existing product theme classes (card, btn, table). No new OCaml CSS. Optional small flash-local classes in ui/theme later.

========== SEED / NAV ==========

- seed_stdlib: load-library "flashcards" after calendar (or after todo – order free as long as std/html exists) - recipe optional: (load-library "flashcards") from home - ui/menu: add Flashcards ? /flashcards on system seed when easy; personal may pin. Not a kernel nav hard-code.

========== AGENT ACCESS ==========

Image is the API: python3 scripts/pp_eval.py '(load-library "flashcards")' python3 scripts/pp_eval.py '(flash/add deck-id "Q" "A")' python3 scripts/pp_eval.py '(flash/due)' python3 scripts/pp_eval.py '(flash/review id "good")' No flash-specific REST. Bulk import via agent loop over flash/add.

========== NON-GOALS (v1) ==========

- Anki .apkg import/export - Cloze deletions, image occlusion, audio - FSRS / full SM-18 - Review history graph / heatmap (nice later) - Shared multiplayer decks / permissions - Sync protocol beyond data_rows + pp_sync package export - OCaml system pages for study (package owns UX) - Branch-scoped cards (data is shared by design)

model

flash-deck + flash-card data_rows; FSRS scheduling fields on card

[status: implemented]

[verify] smoke: (deftype ?) via load; (flash/add) creates flash-card row; smoke: flash-card has stability difficulty interval reps lapses due fields after review

srs

Nopales scheduler flash/apply-rating; ratings again|hard|good|easy

[status: implemented]

[verify] smoke: new card good ? due tomorrow or +1 day; smoke: again ? due today, lapses+1, ease down; smoke: suspended card not in flash/due

ui

Routes /flashcards, deck, study; form POST ratings

[status: implemented]

[verify] smoke: GET /flashcards 200 after load-library; smoke: study POST rating redirects to next or empty

agent-access

pp_eval flash/? as the API

[status: implemented]

[verify] smoke: pp_eval flash/decks and flash/due

23 borge/workout.borg

pricklypear-workout

Pricklypear book chapter: workout journal – fluid program documents + standardized journal facts

[status: implemented]

> – *agent note:* 2026-08-02 status reconcile: libs/workout/lib.pp (~2.6k lines) seeded; live (workout/programs) => free/ksk1/ksk2/ksk3; session-list/volume/home-page present. All chapter subsections match shipped package.

workout

Training journal: programs as fluid trees, sessions/exercises as queryable facts, plan/run/capture packages

[status: implemented]

> *[verify] build: dune build image; smoke: (load-library "workout") installs workout/? bindings and routes; smoke: GET /workout lists recent sessions + start controls; smoke: start free or program session creates workout-session in_progress with doc; smoke: log exercise facts on a session; volume query returns totals; smoke: complete session sets state complete and keeps exercise rows; smoke: resume in_progress session restores doc progress; smoke: pp_eval workout/session-list and workout/volume work for agents; smoke: seed_stdlib can load workout after calendar (or documented order)*

========== PRODUCT INTENT ==========

A first-class training journal on the live image.

Domain tension (the whole design): - Workouts / programs are extremely fluid (ladders, intervals, dice, unilateral switches, freestyle blocks, rest schemes). - Journal analytics must stay boring and queryable (volume by exercise+load over a date range, streak of days trained, what I did last Tuesday).

Resolution: hybrid model, two layers deliberately: 1. FLUID DOCUMENT – sexp/json tree for program structure and live run state. JSON and sexps are the same tree algebra; storage print form is an implementation detail. 2. JOURNAL FACTS – standardized workout-session + workout-exercise data_rows projected from the document (and/or quick-log UI) for history queries.

Inspired by: - paper training logs (date, work done, notes) - Simple program runners (KSK, Q&D, custom blocks) - Pricklypear: objects in Postgres, UI as Nopales routes, agents log/query via eval – no parallel REST API

NOT a social fitness network. NOT a wearable/HR integration platform. NOT an OCaml system page – this is a product package (Nopales). NOT a pure document store (that fails volume queries). NOT a fully normalized interval/rung schema (that fails fluidity).

Killer property: the same capture functions humans hit from /workout are what agents call via pp_eval.

Prior design note (habitat, design-paused 2026-07-18) is superseded by this chapter as authority. Keep the habitat note as a pointer / human-readable mirror, not a second source of truth.

========== LAYERING ==========

- libs/workout/lib.pp – package workout/? (seed on system/main) - data_rows types – workout-session, workout-exercise - Routes via (route ?) – /workout and session/run paths - Optional nav entry – ui/menu pin or seed menu link

- OCaml – existing CRUD/date/random prims only; no new kernel prims required for v1

Programs are CODE (+ optional doc templates), not table rows: each structured program owns a plan / run / capture triplet. Free-form sessions still use the same journal types.

==========

THREE MODELS CONSIDERED (and the choice) ==========

M1. Pure document One sexp/json object is the whole workout (plan + results). Good fluidity; bad cross-session volume queries unless every consumer understands every dialect.

M2. Pure tables workouts + exercises [+ sets] only. Good queries; bad fit for ladders/intervals/emoms/dice and program-specific structure (wide nullable columns forever).

M3. Hybrid (CHOSEN) Fluid document for prescription + live run state; standardized session + exercise fact rows for the journal; capture projects document ? facts. Programs may be weird. Facts answer a closed question set.

==========

JOURNAL QUESTIONS (closed fact schema derives from these) ==========

- Which days did I train? - Which program / intensity? - How much volume of exercise X (optionally at load Y) in range Z? - What is the history of a single session? - Did I finish, abandon, or leave in progress? - What notes / how did it feel?

Anything not required by those questions stays inside the fluid document (rungs, intervals, dice, rest schemes, UI cursor).

==========

DATA MODEL (v1) ==========

Types (deftype on load; fields are documentation + form UI). Each row is ONE tree (JSON object in data_rows.fields). Nested program variance lives under doc/params/progress/detail – never new top-level columns per program.

workout-session date string YYYY-MM-DD program string ; "ksk" | "qd" | "free" | ? intensity string ; "easy" | "medium" | "hard" | "" ; (NOT named "type" – overloaded elsewhere) state string ; in_progress | complete | abandoned notes string doc string ; JSON/sexp-print of the fluid workout ; tree (plan + live progress). Source of ; structural truth for resume/replay. duration_sec string int optional author string optional ; whoami at create; v1 may be "" ; (data_rows still shared)

workout-exercise ; QUERY GRAIN – projected journal facts session_id uuid string of workout-session exercise string ; canonical name, e.g. "Snatch L", "Swing" weight string number ; "" if bodyweight / unset unit string ; "kg" | "lb" | "" (normalize in capture) reps string int ; TOTAL completed reps (not per-set) notes string detail string optional JSON/sexp-print tree for per-set or program-specific breakdown when a program chooses to keep it

NO per-set fact table in v1. NO side column – unilateral work uses naming ("Snatch L" / "Snatch R") so aggregation stays a simple group-by on exercise name. NO separate "program definitions" table – programs are code (and templates inside plan functions / doc skeletons). NO per-user row ACL – data_rows are shared (existing habitat rule). Multi-user content isolation is multi-user.permissions later. Orphan exercise rows (missing session) show under a virtual "(orphan)" bucket in UI; agents can repair.

Field print form: On-row storage may be JSON (current data_rows convention). Conceptually every nested value is a sexp tree. Package helpers MUST hide hand-built JSON strings from call sites (workout/session-fields, workout/exercise-fields, dict-set*).

==========

FLUID DOCUMENT SHAPE (open envelope, closed top-level) ==========

session.doc is a tree. Recommended envelope (keys stable; interior free):

{ "program": "ksk", "intensity": "hard", "params": { ... program-specific knobs ... }, "progress": { ... live cursor / elapsed / side ... }, "blocks": [... free

structure: ladders, intervals, pairs ...] }

Examples (conceptual sexps; JSON equivalent is fine):

;; Kettlebell Simple & Sinister-style hard day cursor (workout-doc (program ksk) (intensity hard) (params (top-rung 5) (time-cap-min 20) (load 24 kg) (exercise Snatch)) (progress (rung 3) (side L) (elapsed-sec 740)) (blocks (ladder (rungs 1 2 3 4 5) (switch each-rung))))

;; Free session (workout-doc (program free) (blocks (pair snatch (sets 5) (reps 2) (load 24 kg)) (pair front-squat (sets 3) (reps 5) (load 60 kg)) (interval row (rounds 10) (on 30) (off 30))))

Plan generators produce docs (or doc skeletons). Run mode mutates progress inside doc (and may append detail). Capture reads doc (+ any explicit log events) and projects facts.

==========

CAPTURE PROJECTION CONTRACT ==========

Bridge rule: doc –capture–> session top-level fields + N exercise facts

Requirements: - Idempotent per session: re-running capture replaces or upserts exercise facts for that session_id; it must not double-count volume on resume/refresh. - Totals win: workout-exercise.reps is aggregate completed work. - Program-specific structure stays in session.doc (and optional exercise.detail); not exploded into new fact columns. - Canonicalize exercise names and units in capture (title case policy is package-defined; document in lib.pp). - Abandoned/in_progress sessions may already have partial facts from live writes; complete() finalizes state and may re-project.

Hybrid flavor for v1: H3 + H1 - Per-program capture functions (H3) - Structured sessions always retain full doc (H1) - Free/quick-log may build a minimal doc from facts (still store it)

==========

PROGRAM ARCHITECTURE ==========

Each structured program owns a function triplet under workout/:

(workout/<prog>-plan intensity-or-params) -> doc tree (workout/<prog>-run session-id) -> HTML (live UI) (workout/<prog>-capture session-id) -> session + facts

Day intensities are DATA inside one algorithm when structure matches (e.g. KSK easy/medium/hard = top rung + time cap params). Split functions only when day types differ structurally.

Intensity map example (package data, not tables): ksk: easy -> top-rung 3, time-cap-min 10 medium -> top-rung 4, time-cap-min 15 hard -> top-rung 5, time-cap-min 20

Macro strategy: defprogram generating plan/run/capture is DEFERRED until ~3 real programs exist (compress repetition, don't anticipate). Prefer quasiquote if/when language lands; hand-built list expansions are error-prone (see language macro feedback).

v1 programs (suggested ship order): 1. free – quick log + optional simple blocks (always first) 2. ksk – ladder / time-cap runner (proves live run + projection) 3. qd – optional second structured program (dice + aggregates) Do not block journal MVP on ksk/qd polish – free log must work alone.

==========

LIVE RUN / RESUME ==========

- Creating a structured session stores state=in_progress and doc. - Live writes: each completed rung/exercise POSTs soon so a dead browser loses little; update doc.progress and upsert facts. - Timer UI may embed a small JS widget (precedent: weather/script); server remains authority for logged work. - htmx for step transitions (complete rung ? swap next step). - Dice / random can be server-side (find-random or equivalent). - Resume: load in_progress session by id (or latest for program), re-enter run UI from doc.progress. - Cleanup: list/filter abandoned in_progress older than N days; mark abandoned via explicit UI/agent action (no silent delete).

==========

PACKAGE API (workout/ prefix) ==========
Sessions: (workout/session-list [limit]) -> list of (id . fields) (workout/session id) -> fields | () (workout/session-create program intensity [date] [doc]) -> id (workout/session-update id fields-dict) -> id | () (workout/session-complete id [notes]) -> id (workout/session-abandon id) -> id (workout/session-in-progress [program]) -> id | ()

Exercises (facts): (workout/exercises session-id) -> list (workout/exercise-log session-id exercise weight unit reps [notes] [detail]) -> id ;; upsert policy documented in lib.pp (by id or by ;; session+exercise+weight+unit for live rungs) (workout/exercise-delete id) -> id | ()

Queries: (workout/volume exercise [from-date] [to-date] [weight] [unit]) -> total reps (number) or printable structure (workout/history [from-date] [to-date] [limit]) -> sessions (workout/session-summary id) -> session + exercises structure (workout/exercise-names [limit]) -> distinct names for autocomplete

Doc helpers: (workout/doc-parse s) -> dict/tree (workout/doc-get doc key) / (workout/doc-set doc key val) (workout/project session-id) -> re-run capture for program

Programs (v1+): (workout/programs) -> list of program ids/titles available (workout/<prog>-plan ?) (workout/<prog>-run session-id) ; may be HTML page fn (workout/<prog>-capture session-id)

HTML / routes: (route "/workout" "workout/home-page") (route "/workout/session" "workout/session-page") (route "/workout/run" "workout/run-page") (route "/workout/log" "workout/log-page") ; quick log / edit facts Exact query keys implementation discretion; prefer ?id=uuid & ?program= & ?intensity=

Naming: package directory libs/workout/ binding prefix workout/ types workout-session / workout-exercise

Data functions take explicit args (safe from /api/eval). HTTP handlers are separate and thin.

==========
HYPERMEDIA UI ==========
/workout - recent sessions (date, program, intensity, state, short volume) - Start bar: program select + intensity (when applicable) + start - Continue card if in_progress exists - link to quick log (free)

/workout/session?id= - session header + notes + state actions (complete/abandon) - exercise fact table - add/edit fact row (quick correction) - if program structured and in_progress: Continue run

/workout/run?id= - program-specific live UI from doc.progress - complete-step controls POST ? update doc + facts ? swap UI - timer widget as needed

/workout/log (optional alias of free capture) - date, notes - repeatable exercise rows (name, weight, unit, reps) - save ? session complete (or in_progress if want multi-step)

Mobile-first: large tap targets during run; minimal chrome. Use existing product theme classes. No new OCaml CSS required.

==========
SEED / NAV ==========
- seed_stdlib: load-library "workout" after calendar (needs date-today; order free as long as std/html exists) - ui/menu: Workout ? /workout on system seed when easy; personal may pin. Not a kernel nav hard-code.

==========
AGENT ACCESS ==========
Image is the API: pp_eval '(load-library "workout")' pp_eval '(workout/session-create "free" "")' pp_eval '(workout/exercise-log id "Swing" "53" "lb" "75")' pp_eval '(workout/session-complete id)' pp_eval '(workout/volume "Swing" "2026-07-01" "2026-07-31")' No workout-specific REST. Bulk backfill via agent loops.

==========
NON-GOALS (v1) ==========
- Wearables, HR zones, GPS, Apple Health / Google Fit sync - Social feed, fol-

lowers, public leaderboards - Plate calculator / 1RM formulas as core (may be helpers later) - Full periodization planner / mesocycle engine - Per-set fact table / side column / equipment inventory - defprogram macro before ~3 programs exist - Branch-scoped journal rows (shared data_rows by design) - OCaml system pages for training UX - Treating JSON vs sexp print form as different product models

model

Hybrid: workout-session + workout-exercise facts; session.doc fluid tree
[status: implemented]

[verify] smoke: load-library defines workout-session and workout-exercise; smoke: session-create stores program intensity state date doc; smoke: exercise-log attaches facts to session_id with total reps; smoke: unilateral names work without a side column

document-and-capture

Fluid doc envelope + idempotent projection to exercise facts
[status: implemented]

[verify] smoke: structured session retains doc through run updates; smoke: capture/project does not double-count on second run; smoke: volume query uses facts only, not doc scraping

programs

plan/run/capture triplets; free first; ksk/qd optional structured
[status: implemented]

[verify] smoke: workout/programs lists at least free; smoke: free quick-log creates complete session + facts; smoke: optional: ksk plan returns doc with params+blocks

ui

Routes /workout, session, run, log; resume in_progress
[status: implemented]

[verify] smoke: GET /workout 200 after load-library; smoke: start + complete free session visible in recent list; smoke: in_progress shows continue entry point

agent-access

pp_eval workout/? as the API
[status: implemented]

[verify] smoke: pp_eval workout/session-list and workout/volume

24 borge/dabar.borg

pricklypear-dabar

Pricklypear book chapter: dabar – BSB study library (reader + Strong's + interlinear)

[status: partial]

> – *agent note:* 2026-08-02 status reconcile: packaging/reader/nav/strongs/interlinear/note-bible-links/work-packages/non-goals shipped; only search-fts remains partial (find-text ILIKE, not tsvector FTS). Live /dabar+/strongs 200; bible/verse John 3 16 ok.

dabar

Bible study layer on the live image: plugin libs, seeds, routes

[status: partial]

> *[verify] build: dune build image; smoke: (load-library "bible") then (bible/verse "John" 3 16) has God/world; smoke: (load-library "dabar") installs study routes without breaking /bible; smoke: GET /bible prev/next or random works after WP1; smoke: GET /strongs and (strongs/get "G26") after Strong's seed; smoke: interlinear fragment for a seeded verse after WP3*

PRODUCT: Dabar on Pricklypear is a multi-user, live Bible study environment built as *libraries* inside the user image – not a port of the Dream dabar server.

PILLARS: 1. Plugin-shaped Nopales packages (libs/*/lib.pp + load-library). 2. Canon + lexicons in Postgres data_rows (shared across branches). Personal notes/bookmarks are a SEPARATE module (not this chapter). 3. htmx partials for reading/study panels (no SPA). 4. Seeds are offline scripts; recipes only install code.

NAMING (locked): - bible/ – core reader (exists): verse, chapter, search, /bible, partials, dropdowns. - dabar/ – study layer: Strong's, interlinear, study chrome that *uses* bible/*. Do NOT rename existing bible/* without a migration plan.

RELATION TO wyo.tech/dabar: - Donor of data formats, Strong's CSV, interlinear field shapes, UX ideas (filters, prev/next, random). - That repo has no borge book; this chapter is SoT for the Pricklypear port. Link from root pricklypear.borg.

packaging

Library layout, boot policy, deps on std/bible

[status: implemented]

> *[verify] smoke: libs/bible/lib.pp loads via load-library bible; smoke: libs/dabar/lib.pp loads via load-library dabar after bible; smoke: boot seed_stdlib loads bible and dabar*

LAYOUT: libs/bible/lib.pp – reader (shipped) libs/dabar/lib.pp – study (to add) scripts/seed_bible_bsb.py scripts/seed_strongs.py (from dabar strongs.csv) scripts/seed_interlinear.py (from dabar bsb_tokens.csv)

BOOT: seed_stdlib loads: std, weather, ui, layout, welcome, ai, outline, notes, signup, todo, contacts, plan, chat, bookmarks, calendar, flashcards, bible, dabar. dabar is seeded on boot; no separate recipe step required.

DEPS: dabar assumes bible/* and html/* (std) already bound. Document in lib header; multi-file require is deferred.

INVARIANTS: - No dabar-specific OCaml except generic find/FTS if needed. - Routes under /bible stay bible-owned; /strongs and /dabar/* may be dabar-owned. - Kernel stays thin.

reader

Existing bible/ reader – baseline for dabar
[status: implemented]

> *[verify] smoke: 31086+ bible-verse rows after full seed; smoke: GET /bible?book=John&chapter=3 renders chapter; smoke: HX-Request /bible returns fragment without chrome*

BASELINE (implemented): - ref BSB:<Abbrev>:<c>:<v> - (bible/verse book ch v), (bible/chapter ?), (bible/search q) - /bible + /partials/bible-chapter - html/* selects for book/chapter/verse
- (bible/prev-ref) / (bible/next-ref) + UI - (bible/random-row) + Surprise control - empty/missing chapter UX

navigation

Prev/next chapter and random verse
[status: implemented]

> *[verify] smoke: next from John 3 ? John 4 panel; smoke: prev from Genesis 1 stays or wraps per policy; smoke: (bible/random-row) returns a ref+text; UI button works*

Use bible/chapter-counts + book ordinal (bible-book rows). Wrap policy v1: stop at ends (no wrap), or wrap OT/NT – pick stop-at-ends for clarity. Random: SQL ORDER BY random() LIMIT 1 via thin prim or store helper; exposed as (bible/random-row).

strongs

Strong's lexicon seed, lookup, pages
[status: implemented]

> *[verify] build: python3 scripts/seed_strongs.py –help exits 0; smoke: after seed: (strongs/get "H430") or G26 returns lemma/description; smoke: GET /strongs and GET /strongs?n=G26*

SOURCE: wyo.tech/dabar/src/tool/sql/strongs/strongs.csv (and dabar Schema.Strongs shape: number, lemma, xlit, pronounce, description).
TYPE: strongs-entry (or strongs) number, lemma, xlit, pronounce, description id/ref: fields.number unique (H430 / G26).
PACKAGE: prefer dabar/strongs-* or strongs/* under dabar load; public names (strongs/get) OK if namespaced carefully.
UI: /strongs list/search stub; detail by number; links from interlinear tokens.

interlinear

Word-level interlinear fragments + toggles
[status: implemented]

> *[verify] smoke: seed loads tokens for at least John 1 or full corpus; smoke: (dabar/inter "John" 1 1) returns HTML fragment; smoke: UI toggle plain vs interlinear on verse/chapter panel*

SOURCE: dabar inter_verse (bsb_sort, base_text, variant_text, translit, parsing, parsing_str, heb_str, grk_str, verse_id, heading, crossref, bsb_version).
Local file: wyo.tech/dabar build/dabar-bundle/tables/ bsb_tokens.csv (754647 token rows; reproduced by dabar-build pipeline from src/tool/sql/bsb/bsb_tables.xlsx sheet biblosinterlinear96 via extract_bsb_tables_raw.py + build_dabar_bundle.py).

STORAGE v1: per-verse JSON on a type bible-interlinear ref, tokens (JSON array of word objects), heading?, crossref? keyed by same BSB:Abbrev:C:V ref as bible-verse. ~31k per-verse rows total. Avoid millions of token rows.

RENDER DECISION (v1): pre-rendered HTML in the seeder. Token JSON stored in fields.tokens for future Nopales-side rendering; dabar/inter renders fields.html. Rendering token JSON in Nopales needs a JSON-array prim – deferred. Per-token fields kept: base_text, translit, parsing, heb_str/grk_str (per verse language), bsb_sort (order). Dropped for v1: variant_text, parsing_str, verse_id, bsb_version, footnotes/par/quote marks. Ref mapping: USFM book_identifier (gen/jhn/...) -> PP bible-book abbrev via the BOOKS table in scripts/seed_bible_bsb.py. verse_id \"John 3:16\" strings are NOT used for keying.

UI: htmx toggle on /bible chapter view and /dabar verse study; Strong's hrefs to /strongs?n=.

Full corpus seed may be large – allow –book John for dogfood. Dogfood stub (seed_interlinear_dogfood.py, John 3:16 only) is superseded by the real seeder.

(* agent note (|2026-08 recon: spec smokes said bible/interlinear or dabar/inter – actual code is dabar/inter-html; canonicalizing on dabar/inter as the alias + dabar/inter-html for the fragment renderer.|) *)

note-bible-links

Sexp bible refs in note bodies render to dabar routes

[status: implemented]

> – *agent note:* 2026-08-02: bible/link-href+link-html ? /dabar?book=&chapter=&verse=; live (bible/link-html "Gen" 1 1 "") anchors Gen 1:1. bible/render-body handles (bible-ref ?) outline form; garden-import normalizes [[bible:?]].
>
> *[verify] smoke: (bible/link-html "Gen" 1 1) anchors to /dabar?book=Gen&chapter=1&verse=1;*
> *smoke: (bible/parse-ref ?) accepts Gen/Genesis forms used by link helper;*
> *smoke: note body with (bible John 3 16) renders clickable study link;*
> *smoke: bad book degrades to escaped prose, no crash*

notes.links-and-roam owns authoring syntax and reindex rules. This subsection owns the RENDER TARGET helpers.

SYNTAX (canonical in note bodies): (bible BOOK chapter) (bible BOOK chapter verse) (bible BOOK chapter verse-from verse-to) (bible BOOK chapter verse "label") ; optional label string

HELPERS (bible/ package; dabar may wrap chrome): (bible/link-href book ch verse?) -> "/dabar?book=&chapter=&verse=" (bible/link-html book ch verse? label?) -> anchor HTML Reuse bible/abbrev-of + bible/parse-ref + safe-chapter.

RENDER PIPELINE: roam/render-body (or kernel render-links) recognizes the bible sexp form and calls bible/link-html. Optional htmx onto #dabar-panel when the page includes dabar chrome; plain href navigation is always correct.

IMPORT: Obsidian [[bible:Genesis2:3|L]] normalizes to sexps in notes.garden-import – not re-parsed as wiki slugs.

NON-GOAL: storing bible edges in note-link; canon text stays shared data_rows (bible-verse), not duplicated into gardens.

search-fts

Postgres full-text search over verse text (not semantic)

[status: partial]

– *agent note:* 2026-08-02 confirm partial: bible/search = find-text ILIKE on bible-verse text (live "love" ? 40 hits + UI form). True tsvector/rank FTS + index still optional WP.

[verify] smoke: bible/search love returns ranked hits

OPTIONAL WP: expression index / tsvector on data_rows.fields-»'text' for type bible-verse, or generated column in a later migration. Prim: find-text-fts or specialized bible search SQL. NON-GOAL: pgvector, embedder, /api/search/semantic.

daily-scripture

Verse-of-the-day, pushed to yourself: a deterministic picker, a deliver fn that lands the verse in your daily note and pings ntfy, fired by a jobs schedule. The product answer to 'at the very least I want to push daily scripture to myself.'

[status: planned]

– *agent note:* 2026-12 plan pass: daily-scripture campaign. Substrate is ALL shipped – jobs/schedules (borge/jobs.borg), bible/random-row+verse+chapter-counts, calendar's daily-YYYY-MM-DD note convention, notes/doc upserts. New work pinned: this subsection's two Nopales fns, image.outbound-http's http-post, notify.ntfy-delivery's ntfy/send.

*[verify] build: dune build image; smoke: (dabar/daily-ref) called twice on the same day returns the same ref; advances next day; smoke: (dabar/daily-deliver) appends a (bible ?) ref + verse text outline node to notes doc daily-<today>; second call same day is a no-op (idempotent); smoke: (submit-job "dabar/daily-deliver" '()) -> job row done; daily note grew exactly one verse node; smoke: (schedule "daily-scripture" "30 6 * * *" "dabar/daily-deliver" '()) then (schedules) shows it; PP_TZ respected in cron fire time*

THE PICKER (dabar/daily-ref). DETERMINISTIC, not random: n = days-since-epoch (floor (now-ms) / 86400000) verse-index = n mod total-canonical-verses (~31086 BSB rows) walk = book ordinal -> chapter-counts (bible/chapter-counts) -> (book chapter verse) Same verse for every habitat user on the same day, survives re-fire, no state to store, annual-ish cycle. bible/random-row stays the Surprise control; it is NOT the daily picker.

THE DELIVERY (dabar/daily-deliver). Three writes, one job: 1. Verse text via (bible/verse book ch v). 2. Append into the DAILY NOTE (slug daily-YYYY-MM-DD, the calendar package's convention – calendar/has-daily? then shows the dot organically) as an outline node carrying a (bible BOOK ch v) ref body -> note-bible-links renders it as a one-click dabar study link. NOT notes/capture into the Inbox: daily capture is a refile chore; the daily note is where underlining/annotation happens. 3. (ntfy/send title body click-url) – lock-screen ping; Click: opens /dabar?book=&chapter=&verse= (the existing dabar/verse-study-from-params route).

IDEMPOTENCY: deliver checks the daily note's outline for an existing node whose body carries today's dabar/daily-ref before appending (jobs re-fire after retry must not double the verse). with-step exists if the fn grows real stages; the dedupe check is honest enough for v1.

THE SCHEDULE. Pinned at schedule time by the jobs substrate (submitted_by + branch, jobs.node-shapes): the SCHEDULER defines dabar/daily-ref + dabar/daily-deliver on their own branch (garden rule – admin/personal or a dev branch, then promote), then upserts: (schedule \"daily-scripture\" \"30 6 * * *\" \"dabar/daily-deliver\" '()) Cron strings evaluate in image-local time; PP_TZ or offset the cron fields, pinned at make time.

PROMOTE POLICY: the picker + deliver fn are shared-default worthy (system/main) once dogfooded; each user's schedule row and notification target stay personal (jobs rows are submitted_by-scoped).

work-packages

Core dabar features

[status: implemented]

Provides Strong's lexicon lookup and word-level interlinear study tools that build upon the bible/ reader. Includes navigation (prev/next/random) and optional full-text search over verse text.

NOT THIS CHAPTER: personal notes/bookmarks – separate module.

ACCEPTANCE v1 "dabar on PP": - bible reader green on boot - load-library dabar adds study without breaking /bible - Strong's lookup + real interlinear corpus seed (–book John dogfood, then full corpus) - plain/interlinear htmx toggle on reader + study views - no semantic search - no notes/bookmarks here

> – *agent note:* 2026-08-02: v1 acceptance met – bible+dabar libs, Strong's, interlinear toggle, /dabar+/strongs live. Meta checklist subsection, not a separate code deliverable.

non-goals

Explicit exclusions

[status: implemented]

- Semantic search / pgvector / llama embedder - Personal notes / bookmarks (separate module) - Porting Dream dabar server into the image - Audio - Path-param routes /bible/John/3 until platform supports - Multi-translation UI - New unikernel or service binary for dabar

> – *agent note:* 2026-08-02: documentation-only exclusions; marked implemented as spec-complete.

25 borge/notify.borg

pricklypear-notify

Pricklypear book chapter: notify – getting events out of the image to a human. Server-push via ntfy (HTTP POST from a jobs schedule) is the v1; an inhabitat unread feed is the named next surface. Records the trigger-split: server-known events push from the image; locally-known events are the client lens's business (local-first.borg), never this chapter's.

[status: planned]

> – *agent note:* 2026-12 plan pass: chapter born from the daily-scripture discussion. Key recon that shapes it: (a) the Net effect tag and a curl-backed, admin/agent-gated http-get ALREADY ship (image/lib/prims_net.ml, lisp/lib/effect.ml); (b) jobs + cron schedules are live (borge/jobs.borg); (c) the ONLY missing kernel piece for push is image.outbound-http's http-post. The split that must not blur: server-known triggers (cron, chat mentions, shared events) push from the image – the phone need not run anything; locally-known triggers (lens-computed reminders while offline) belong to a resident agent on the client (Termux:API / WorkManager per local-first.borg), not to this chapter.

notify

Delivery of image-side events to humans. jobs.borg says 'result delivery is pull, not push'; this chapter is the sanctioned push exception – narrow, outbound-HTTP-based, identity-gated.

[status: planned]

ntfy-delivery

libs/ntfy – one POST per event to an ntfy topic; the ntfy Android app (or any UnifiedPush distributor reading the same topic) turns it into a native lock-screen notification. No native app development on our side.

[status: planned]

> *[verify] smoke: (load-library "ntfy") then (ntfy/send "t" "b") POSTs to the configured topic (observe via the ntfy app or topic URL); smoke: ntfy/send raises transient on unreachable host (job runner retries per jobs.runner backoff); smoke: ntfy/send returns the payload dict for 4xx/5xx (caller can inspect); topic config row missing -> clean User_error, not a crash*

PACKAGE: libs/ntfy (own lib, standalone – NOT inside dabar; daily-scripture is its first consumer, not its owner).

SURFACE: (ntfy/send title body) ; no click target (ntfy/send title body click-url) Built on (http-post url headers-json body) (image.outbound-http). Wire headers the ntfy convention: Title: <title> Priority: default Tags: <+optional> Click: <click-url> (Icon/Actions/Markdown stay available as pass-through header extras if a caller wants them – headers-json is the escape hatch; the convenience surface stays three args.)

CONFIG (capability secrets OFF branches): A data row of type ntfy-config, id \"default\", fields {url, topic, token?}. url+topic are capability URLs – treat like passwords: the row is admin-readable (data.u.<admin>), never seeded to system/main, never in git. token is ntfy's optional bearer for protected topics. ntfy/send reads the row per call (cheap, and lets the user rotate a topic without touching code).

SELF-HOST PATH: ntfy is a single static binary; pointing ntfy-config.url at a self-hosted instance is a config change, not a code change. The third-party dependency is a choice, not an architecture.

RETRY DISCIPLINE: ntfy/send does NOT catch transient Prim_errors – callers inside jobs let the runner's attempts/backoff own retries (jobs.runner). Callers in request paths (e.g. chat mention fan-out later) wrap in guard themselves or fire via submit-job.

in-habitat-feed

DEFERRED next surface: a (notify title body link) fn + per-user notification rows + an unread-count badge in the layout chrome that hx-polls. The pull-side 'inbox flow' generalization – every package (todo due-soon, flashcards due, chat mentions) gets a uniform in-image feed before any push adapter runs.

[status: planned]

DESIGN (named, deliberately unbuilt): one package-owned type notification {title, body, link, read bool, created_at} in data.u.<user>, written by a plain package fn (no new prim – create + a badge fragment; ship it as libs/notify when built). The layout chrome gains one hx-poll fragment for unread count. Delivery adapters (ntfy, in-habitat, future web-push) fan out FROM this write when it exists: today dabar/daily- deliver calls ntfy/send directly and that is FINE – the feed is additive when it lands, not a prerequisite.

REASON FOR DEFERRAL: daily scripture doesn't need it; the badge is chrome work across themes; and the notification register's exact shape (per-type read cursors? retention? issue-register reuse?) deserves its own dogfood round rather than being rushed as daily-scripture scaffolding.

non-goals

Delivery paths considered and rejected/deferred, with reasons (recorded 2026-12 discussion)

[status: implemented]

- XMPP – rejected. A send-only client is still a protocol stack (SASL/TLS/roster/carbons edge cases) and phone push through XMPP depends on the server's FCM/APNs modules anyway. Its real value (federated chat) is serviced by chat/ in-habitat. Wrong shape for a single-image Lisp OS. - Web Push (RFC 8291) – deferred, not rejected. Makes the browser the push target (PWA, no third party) at the cost of VAPID + aes128gcm/ECDH payload encryption in OCaml plus a service worker. The trigger to revisit: the ntfy third-party dependency becomes intolerable. - Direct FCM/APNs + native app – deferred to the client story. Belongs to local-first.borg's Termux/Termux:API/ WorkManager clients, where termux-notification and the ntfy app's UnifiedPush-distributor role already cover it; an own APK is an optional UX promotion there, not a notification necessity. - Outbound SMTP – unnecessary: email-via-HTTP-API (Resend/Postmark-shaped) is a header set on the same http-post prim if ever wanted; no SMTP client lands. - Server-sent events / WebSocket browser push as a notification channel – the htmx poll cadence already covers in-tab freshness cheaply; SSE exists for the agent surface and is not a notification bus.

26 borge/migrations.borg

pricklypear-migrations

Pricklypear book chapter: migrations
[status: partial]

– *agent note:* 2026-08-02 status reconcile: applied-by-image+additive-only match ops; branching-interaction stays partial (type_defs merge/schema_vers messaging).

migrations

Schema evolution: image-applied, additive-only, schema authority in main
[status: partial]

[verify] build: dune build image; smoke: image boots against a fresh database; schema_migrations records 001..007; re-boot is a no-op

applied-by-image

The image owns schema state before it serves
[status: implemented]

The IMAGE runs migrations. On boot it applies any pending migrations in version order, and only then starts serving. (scripts/dev.sh applies them directly in the dev habitat.) Images assume "the schema is current" once boot completes.

Migration source: SQL files in migrations/ (version-numbered). Tracked in the schema_migrations table; application is idempotent (IF NOT EXISTS guards), so re-applying is a no-op.

Initial migrations: 001_branches.sql – code store tables (branches, function_defs, type_defs, view_bindings, sessions, schema_migrations) 002_auth.sql – auth tables (users, credentials, auth_sessions, auth_log; system_keys is inert legacy) 003_data.sql – shared data_rows 004_bible_ref_index.sql – bible lookup indexes 005_ai_sessions.sql – agent conversation store (ai_sessions, ai_messages; NOT auth_sessions). Spec: ai-authoring.agent-portal. 006_route_visibility.sql – view_bindings.visibility text NOT NULL DEFAULT 'private' (public|private). Spec: auth.route-visibility. Additive column only; existing routes stay private. 007_media.sql – media_blobs (bytea object store for agent/browser image upload). Spec: storage.media. Shared across branches like data_rows; ownership via author column. Additive table only. 008_agent_users.sql – agent user rows / allow flags. 009_tree.sql – one-tree substrate (tree_nodes, tree_branches, bidirectional deferred mirrors, tree_* read views, tree_verify). Spec: one-tree. 012_code_history.sql – code_history table + BEFORE UPDATE/DELETE triggers on function_defs, type_defs, view_bindings. Append-only before-image history for every code-row write. Spec: branching.code-history. 015_jobs.sql – durable background jobs (jobs, job_steps, schedules) as a DEDICATED table (not data_rows/tree nodes): the runner's SKIP LOCKED claim protocol needs real typed status/run_at columns + partial jobs_claim_idx; kernel rows belong to the migration tier. Spec: jobs. 018_tree_versions.sql – ADDITIVE versioned-leaf table for tree-time (branch_id, path, gen) append-only history + covering index + tree_generations row, gen-range partitioned for retention GC, tombstone representation. DESIGNED in tree-time.migration-018 (campaign 5.0 plan pass); the file lands with 5.1 (make versioned-leaves) and is not yet tracked. Spec: tree-time. 019_tree_version_shadow.sql – tree-time Phase 1 dual-write shadow: AFTER trigger on tree_nodes appends a tree_versions row per write (allocating the head gen per path class) + tombstone on delete. Additive; Spec: tree-time.versioned-leaves. 020_tree_version_echo_guard.sql – tree-time mirror- echo guard: tree_version_append() REFINED to make

a no-op mirror bounce (same content body/author/kind/ type) return before allocating a gen or inserting a version row, so one logical write is never double-counted. Additive CREATE OR REPLACE; Spec: tree-time.versioned-leaves. 021_tree_versions_reconcile.sql – tree-time head reconciliation tool: tree_versions_reconcile_heads() heals the one non-head-equivalent class (a shadowed LIVE row whose newest version is a TOMBSTONE – delete-then-recreate whose re-create did not re-shadow) by appending the live content as a fresh head, and repairs the branch/data head counters to the max live gen. Additive CREATE OR REPLACE; runner: scripts/reconcile_tree_version_head.sh. Spec: tree-time.versioned-leaves. 022_tree_retention.sql – tree-time.gc retention worker: tree_retention_policy (policy AS HABITAT DATA, path_class code|data, type_name NULL = whole class, UNIQUE(path_class,type_name)) + tree_ versions_retain(path_class, keep_windows): computes the whole-class floor from the live head gen, CHECKPOINTs the newest in-retained-past state of every fully-retired path to a row at the floor boundary, then DROP PARTITIONs every lazy gen-window entirely below the floor – never the base gen_0 window. Additive + idempotent + image-applied; the worker runs on the jobs runner (boot.ml, migration-015) gated by wall-clock. Spec: tree-time.gc.

A migration in flight blocks serving until it completes (or fails). This guarantees the image always sees a consistent schema. Migration failure aborts boot and requires manual intervention – no half-applied state. (The additive-only policy makes failure recovery simpler.)

> – *agent note:* 2026-08-02: migrations/001-012 + schema_migrations; apply via scripts/dev.sh and deploy hooks before serve (AGENTS.md). Idempotent SQL files.

additive-only

Schema changes can ADD, not remove or rename

[status: implemented]

> *[verify] build: grep -rE 'DROP|RENAME|ALTER COLUMN TYPE' migrations/ = 0 matches; smoke: all migrations idempotently re-appliable*

Schema evolution policy for type_defs in main: ADDITIVE ONLY. You can: - Add a new type (new table) - Add a new field to an existing type (ALTER TABLE ADD COLUMN, with NOT NULL forbidden unless a DEFAULT is specified)

You CANNOT: - Remove a field (no DROP COLUMN) - Rename a field (no RENAME) - Change a field's type (no ALTER COLUMN TYPE except widening)

WHY: - Additive changes are backward-compatible with running images. An image running old code that doesn't know about a new field still works; it just ignores the column. - Removing/renaming breaks running images that reference the field, with no compile-time error to catch it (Nopales is dynamic). - The cost is cruft accumulation: dead fields pile up over time. Tradeoff accepted for correctness.

When a field truly must go: a manual migration (off-cycle, admin-run during scheduled downtime) that also updates affected branch code. This is rare and treated as an incident, not routine.

This policy is set early because retrofitting it later (after non-additive migrations exist) is expensive. It is a foundation constraint.

branching-interaction

How migrations interact with branches

[status: partial]

Migrations only apply to the SHARED data tables, governed by main's type_defs (which are merge-governed into main).

Branches CANNOT migrate data tables. A branch can PROPOSE a type change in its type_defs, but the underlying table doesn't change until that proposal is merged to main and the image applies the resulting migration at next boot.

A branch running code that references a field added in main but not yet handled by the branch's own code: works (the column exists; branch code ignores it on write, reads null on read if no default).

A branch running code that references a field that DOESN'T exist (because the branch's type_defs is behind main's): runtime error at the primitive layer. The user gets a clear message: "field X not in current main schema; merge your schema change or sync to main".

Schema version is tracked per type (schema_vers in type_defs). Branches see the version they declared; main sees its current version. The primitive layer can validate against either at query time.

> – *agent note:* 2026-08-02 confirm: shared tables + additive-only policy live; type_defs proposal?main migration path and schema_vers messaging still incomplete.

27 borge/formal-verification.borg

pricklypear-formal-verification

Book chapter: formal verification of the OCaml outer shell.
[status: planned]

– *agent note:* 2026-08-02 status reconcile: correctly still planned – design in docs/formal-verification-plan.md; no QCheck/proof islands shipped as product gate yet.

formal-verification

Machine-checked stability for the pure OCaml core; the Nopales garden may stay janky. Formal verification proceeds via property-based testing (QCheck) and proof tools. The detailed plan (islands, tooling, passes, success criteria) lives in docs/formal-verification-plan.md.
[status: planned]

28 borge/ai-authoring.borg

pricklypear-ai-authoring

Pricklypear book chapter: ai-authoring
[status: implemented]

> – *agent note:* 2026-08-02 status reconcile: tools/portal/ast-rewrite/growth-seed already impl; review-merge is existing branch+inspect+merge path (no separate AI privilege); provenance v1 agent:<model> on propose-function. Deferred: badge styling, audit table.

ai-authoring

AI as a first-class author of functions: propose, surgical AST rewrite/freeze, growth-from-seed, human merge gate, provenance
[status: implemented]

> *[verify] build: dune build @all; smoke: AI propose on user branch -> human inspect -> merge policy gate; smoke: future: llm-inner freeze rewrites stored AST; llm-seed grows outward on a branch*

AI is an author, not a privileged runtime. Machine- written code uses the same store, branch, and merge machinery as human code. Default trust model: AI writes only to a user (or agent) branch; promoting into system/* still requires admin merge policy.

Coding styles supported by the tool surface:

1) Blank-slate propose (propose-function 'name "spec") Whole new/replaced binding from a prompt.

2) Surgical freeze rewrite (CODING TOOL – preferred) (llm-inner 'name "concept") (llm-outer 'name "concept") LLM rewrites one AST component; on accept the result is FROZEN into function_defs (not left as an unevaluated hole). Emacs-ish replace-region with generated code, but tree-native.

3) Growth-from-seed (CODING TOOL) (llm-seed "build an app that ...") Recursively: create smallest clean core, then llm-outer until requirements are satisfied. Bottom-up / inside-out Nopales style operationalized.

4) Runtime generative eval (SECONDARY / later) Eval-time expansion of AI holes without freeze. Useful, but not the primary coding workflow.

Non-goals for first cut of new tools: - AI does not bypass author-runs-as-author - AI does not write system/* without merge policy - full addressable node table not required for whole-function inner/outer (paths into JSONB AST suffice); node IDs are a later upgrade (branching.ast-nodes)

tools

How an AI proposes or patches functions into the store
[status: implemented]

> *[verify] build: dune build image; smoke: POST /api/ai/propose ai-double then (ai-double 21) => 42*

v1 SHIPPED: - propose-function Nopales prim + /api/ai/propose - whole-function body from model, define on current branch - author = agent:<model> - never merges to system/* - local or remote OpenAI-compat endpoint via env LLM_API_BASE / LLM_API_KEY / LLM_MODEL

Shared client contract for ALL AI tools: - chat/completions (stream optional; SSE for live UI) - model returns code/AST only for author modes - parse gate before any store write - effect inference after parse (lisp-interpreter.effects) - policy gate before freeze

Primary guest surface is agent-portal: durable sessions + one tool-capable loop ((agent-say sid prompt)). Full message history is resubmitted each provider call. propose-function / llm-* remain specialized authoring helpers outside the conversation store.

Sibling tools (exist as prims; product UX secondary): - ast-rewrite: llm-inner / llm-outer / llm-rewrite - growth-seed: llm-seed recursive outer growth

agent-portal

One durable agent: sessions + messages in Postgres, one tool-capable loop, full history resubmitted each provider call

[status: implemented]

> *[verify] build: dune build image; test: image/test/ai_store_test.ml; smoke: migration 005 creates ai_sessions + ai_messages; smoke: ai_store create/list/append/load round-trips a multi-turn thread; smoke: (agent-say sid prompt) resumes prior messages across process restart; smoke: tool turns persist assistant tool_calls + tool results in order*

==========
PRODUCT (finished shape – small and closed) ==========

An agent conversation is a durable thread owned by a user: - seed (system persona) fixed at create - ordered messages (user / assistant / tool) - optional tools allowlist on the session - every model call resubmits [system seed] + history

There is ONE loop. Tools are not a second product: - model may answer in text, or request allowlisted tools - kernel runs tools, appends results, continues - one-shot Q&A is the same loop with a short history

NOT in this product cut: - separate (agent) vs (agent-task) kernels - in-memory-only multi-turn that dies with the request - token streaming as a requirement (progress events yes) - chat UI polish (REPL/pp_eval + thin HTTP enough) - propose-function / llm-* merge into sessions (those stay specialized authoring helpers)

==========
PROVIDER CONTRACT (do not invent a second one) ==========

OpenAI-compatible POST /chat/completions:

{ model, messages: [...full history...], tools?: [...] }

The provider does not remember prior turns. "Multi-turn" means WE store messages and resubmit them every call. Tool rounds are ordinary messages in that list: assistant + tool_calls, then role=tool results, then another completions call with the longer list.

Default transport: non-stream chat_turn (reliable on Synthetic). Progress via on_event: Token | Tool of {name; summary} | Done | Fail HTTP may SSE those control events. Token stream:true is optional later, not part of acceptance for this cut.

==========
SCHEMA (migration 005_ai_sessions.sql) ==========

Additive-only. Named to avoid clash with fleet 'sessions' and auth 'auth_sessions'.

ai_sessions: id uuid PK default gen_random_uuid() owner text NOT NULL – username (whoami) title text – optional; first user line ok seed text NOT NULL – system prompt; not a message row branch_ref text – branch at create / last say tools boolean NOT NULL DEFAULT true model text – snapshot of LLM_MODEL at create created_at timestamptz NOT NULL DEFAULT now() updated_at timestamptz NOT NULL DEFAULT now()

ai_messages: id uuid PK default gen_random_uuid() session_id uuid NOT NULL REFERENCES ai_sessions(id) ON DELETE CASCADE seq int NOT NULL – dense order from 0 role text NOT NULL – user|assistant|tool – (system is session.seed only) content text – nullable for pure tool-call – assistant turns tool_calls jsonb – [{id,name,arguments}] or null tool_call_id text

– set when role=tool created_at timestamptz NOT NULL DEFAULT now() UNIQUE (session_id, seq)

Indexes: ai_sessions (owner, updated_at DESC) ai_messages (session_id, seq)

Invariants: - seed lives ONLY on ai_sessions (no role=system rows) - messages are append-only in v1 (no edit/delete mid-thread except whole-session delete) - owner isolation: list/get/delete scoped to whoami - tools flag is session-level; per-turn override later

==========

STORAGE INTERFACE (image/lib/ai_store.ml) ==========

Pure store module over Store.t / pgx. No LLM calls here. Maps rows <-> Llm_client.chat_message for the runner.

Storage interface (ai_store.ml): create / get / list / delete sessions (owner-scoped); list and append messages (ordered by seq; roles user|assistant|tool only). Session row: id, owner, title, seed, branch_ref, tools allowlist, model, timestamps. (chat_message list) -> unit (* assigns next seq values; touches updated_at *) touch_session store ~id ?title () -> unit messages_for_api session history -> chat_message list (* Msg System seed :: history *)

chat_message encoding: Msg {User|Assistant; content} -> role, content, tool_calls=null, tool_call_id=null Assistant_tools {content; tool_calls} -> role=assistant, content, tool_calls=jsonb Tool_result {tool_call_id; content} -> role=tool, content, tool_call_id System messages are NEVER stored; only injected at API build.

Errors: missing session, owner mismatch, bad role on write. Use Store.current_conn so with-transaction works if needed.

==========

ONE RUNNER (after storage; same subsection product) ==========

Agent.run_loop ~session ~tools ~on_event ~max_steps : string Lwt.t

Algorithm: 1. history = list_messages(session.id) 2. loop step: msgs = messages_for_api session history turn = chat_turn ~messages:msgs ~tools:(if session.tools then allowlist else []) match turn with | Final text -> append [Msg Assistant text]; on_event Done; return text | Need_tools {content; tool_calls} -> append Assistant_tools; for each call: run allowlisted tool; append Tool_result; on_event Tool {...}; continue (cap max_steps) | Turn_error e -> on_event Fail; return error string

Guest entry (prims; wire after store): (agent-session-new seed) -> sid (agent-say sid prompt) -> text append user msg, run_loop, return final assistant text (agent-history sid) -> list of role/content pairs (agent-sessions) -> recent threads for whoami (agent-session-delete sid) -> ok

Library (libs/ai) becomes thin: ai/ask = new-or-sticky session + say (tools may stay on) ai/do = same (explicit tools session) Deprecated as peer kernels (keep temporary aliases): (agent seed prompt) -> new session + one say (agent-task prompt) -> new tools session + one say

propose-function / install-function remain OUTSIDE this loop as one-shot authoring helpers (they may share llm_client.chat_turn but not ai_sessions).

==========

CAPABILITY / PROGRESS / NON-GOALS ==========

Capability: prims labeled Ai (existing effect lattice). Progress: on_event / optional SSE control events. Not required: OCaml 5 effects, WebSocket mid-turn input, message edit, shared multi-user threads, token streaming.

MAKE ORDER 1. migration 005 + scripts/dev.sh apply list 2. image/lib/ai_store.ml + dune + round-trip test 3. run_loop on ai_store (replace in-memory agent_task msgs) 4. prims + libs/ai aliases 5. HTTP session_id on /api/agent (thin)

ast-rewrite

Surgical coding tools: llm-inner / llm-outer / llm-rewrite with freeze-on-accept into stored AST

[status: implemented]

[verify] build: dune build image; smoke: llm-inner on a binding rewrites body AST and survives reload; smoke: llm-outer wraps a pure function; effect badges update; smoke: reject unparseable model output without mutating store

PURPOSE Iterative code growth/refactor against live stored trees. Human (or command) supplies a concept; model rewrites ONE component; accept freezes into Postgres AST + live env.

PRIMS / COMMANDS

(llm-rewrite name prompt . options) General form. Options may include: mode: 'replace | 'wrap ; default replace path: structural path ; default whole body as: new-name ; non-destructive rename target branch: branch-ref ; work on a branch first accept: 'auto | 'preview ; default preview in UI, ; REPL may auto with flag

(llm-inner name prompt . options) Sugar: mode=replace on function body (or selection path). "Improve/replace the organ."

(llm-outer name prompt . options) Sugar: mode=wrap on whole function. "Put tissue around the organ." Old behavior preserved via call-through or careful inline; prefer call-through to name-old or nested original subtree.

FREEZE-ON-ACCEPT SEMANTICS (critical) 1. Load current function AST from store/env 2. Select target subtree (body / path / whole) 3. Build surgical prompt (see PROMPT CONTRACT) 4. Ping LLM local or remote 5. Parse response as s-expr or JSON AST only 6. Splice into tree (replace or wrap) 7. infer_effects on result 8. policy check (default allow Pure|ReadDb|Html; escalate WriteDb|Branch|Publish|Ai) 9. Preview diff (pretty + optional AST) 10. On accept: - persist function_defs.body JSONB - rebind live env - author = agent:<model> - optional provenance run id 11. On reject: no store mutation

This is NOT leaving (llm-inner ...) in the program. After accept, the program contains real Nopales AST only.

PROMPT CONTRACT (surgical AST author) System: You are editing a Pricklypear Nopales AST. Return ONLY one replacement form: a single s-expression (or JSON AST). No markdown, no essay. You write ONE component of a larger program tree.

User payload includes: - mode replace|wrap - function name - user concept / intent - current function pretty - current function AST json - selected path + selected subtree pretty/json - available prims summary - allowed effect set - for wrap: instruction to preserve behavior and call through original unless inlining is clearer

Model output: - full lambda when replacing whole binding body - subtree only when replacing a selection hole - wrapper lambda when mode=wrap

VALIDATION GATES - reader/json parse success - splice shape compatibility - effect policy - optional smoke eval of (name sample-args) if provided

UI SURFACE - function editor command: Rewrite? / Wrap? - structural selection path when structure mode exists - REPL prims for power users - always show diff before freeze unless accept:'auto

DEFAULTS (safety) - prefer work on current branch; offer agent/* branch - prefer preview over silent overwrite - keep old body in diff buffer / one-level undo - non-destructive 'as new-name always available

NON-GOALS (this subsection) - recursive app growth (see growth-seed) - runtime hole expansion without freeze - multi-file repo patches

growth-seed

llm-seed: recursive bottom-up growth via smallest core + repeated llm-outer until requirements satisfied

[status: implemented]

[verify] build: dune build image; smoke: llm-seed on a tiny goal produces >=1 frozen function on a branch; smoke: growth stops at max-steps or requirement-satisfied judge; smoke: each step is inspectable as a frozen binding + diff

PURPOSE Operationalize inside-out Nopales development for AI: seed a minimal clean function, then wrap/grow until the stated requirements are satisfied.

TOP-LEVEL COMMAND

(llm-seed requirement . options)

Options (sketch): name: optional root name hint branch: default fork to user/ai-<stamp> or agent/* max-steps: default small (e.g. 4..8) budget: token/time caps effects-allow: default Pure|ReadDb|Html stop: 'ask | 'auto-judge

ALGORITHM (orchestrator, not one giant prompt)

Step 0 – Plan (optional short call) Given requirement, emit a growth outline: core behavior, successive wrappers, final page? Human may edit plan later; v1 can be implicit.

Step 1 – Seed (smallest clean core) Prompt specialty: "Write the SMALLEST and CLEANEST pure-ish function that captures the essential nucleus of: <req>. One lambda only. No wrapping, no IO unless required." Freeze as seed name (e.g. notes-core). Run a tiny smoke if possible.

Step 2..N – Grow outward (llm-outer loop) While not satisfied and steps < max-steps: a. Judge prompt (or human): "Does current function set satisfy <req>? If not, what SINGLE outer responsibility should be added next (validation, IO, page, composition)?" b. If satisfied -> stop c. Else llm-outer on current tip with that single responsibility (surgical wrap prompt) d. Freeze wrapper as new tip (or replace tip name while keeping -vN history / old symbol) e. Record step in the run's growth log (in-memory; durable step history is a later addition)

Step Final – Optional publish surface If requirement implies a page and effects allow: define page handler + (route ...) Still no auto-merge to system/*.

JUDGE / STOP CONDITIONS - model judge returns satisfied=true - human stops - max-steps reached - effect escalation would exceed allowlist - parse/policy failures exceed retry budget

WHY RECURSIVE OUTER NOT ONE-SHOT APP DUMP - preserves a working center at every step - each layer is runnable/testable in the image - matches bottom-up Nopales craft - forces LLMs into step-sized commits - diffs remain reviewable

PROMPT SPECIALIZATIONS Seed prompt: smallest nucleus, prefer Pure Outer prompt: one responsibility only, preserve prior behavior via call-through Judge prompt: boolean satisfied + next missing capability in one short structured response

FAILURE MODES / GUARDRAILS - onion wraps forever -> max-steps + dedupe judge - outer reimplements instead of wraps -> prompt + validation heuristic (must reference prior name) - effect creep -> policy gate per step - nondeterminism -> provenance + branch + growth log

RELATIONSHIP TO OTHER TOOLS llm-seed USES: - blank-slate propose or llm-inner for seed - llm-outer for each growth ring - effects inference/policy each freeze - review-merge for promotion to system/* llm-seed is an orchestrator over ast-rewrite.

NON-GOALS - fully autonomous unattended product engineering - hidden multi-minute agent swarms without logs - replacing human taste for when to re-inner/ collapse wrappers after growth

review-merge

Human gate: inspect AI changes, then merge with existing policy

[status: implemented]

Review loop: 1. AI commits one or more function_defs on a branch (propose, llm-inner/outer freezes, or llm-seed steps) 2. Human opens system browser / functions list / editor / inspect to read AST + effect badges 3. Human runs/tests via browser-repl or scripts/repl.sh 4. For growth-seed: step through growth log diffs 5. Human merges branch -> target via existing (merge ...) policy (admin-only into system/*) 6. Reject path: leave branch, redefine, or undo freeze

No separate "AI merge" privilege. Merge politics stay in branching.merge.

> – *agent note:* 2026-08-02: workflow uses /bindings inspect + /branches promote/merge + REPL – no separate AI merge. Matches "merge politics stay in branching.merge".

provenance

Attribution, audit, and trust labels for machine-authored code

[status: implemented]

v1 SHIPPED: author=agent:<model> on AI writes; visible in /bindings and inspect.

REQUIRED for rewrite/seed tools: - author agent:<model> on every freeze - optional metadata: run_id, parent_function, growth_step, prompt_hash, mode inner|outer|seed - growth-seed writes an inspectable step log

DEFERRED: visual agent badge styling, dedicated audit table. Trust rule remains branch + merge policy + effect allowlists.

> – *agent note:* 2026-08-02: v1 as written in details – author agent:<model> on propose-function/AI writes; inspect shows author. Optional metadata/badge/audit table remain deferred non-blockers.

29 borge/film-analysis.borg

pricklypear-film-analysis

Pricklypear book chapter: film analysis – shot/scene structure, chiastic (ring) analysis, cut comparison as branches

[status: planned]

film-analysis

Films as queryable structure: scenes with frame-exact timecodes, motifs and beats, chiastic analysis over scene rings, and alternate cuts as branches. Analysis is data on the image; media files never are.

[status: planned]

> *[verify] build: dune build image; smoke: (load-library "film") installs film/? bindings and routes; smoke: create a film, ingest scenes, run film/report over a branch; smoke: GET /film lists films; /film/<slug> shows ring report; smoke: pp_eval film/? entry points work for agents*

==========
PRODUCT INTENT ==========
A first-class film-analysis workbench on the live image.

The motivating observation: long-form narratives carry structural meaning in temporal placement – chiastic (ring) composition where scene k mirrors scene n-k+1, a "mirror moment" at the temporal center, timed plant/payoff pairs. This structure is *checkable*, not just interpretable – but no NLE or spreadsheet treats a cut as queryable, branchable data. Pricklypear does.

Three workloads, one substrate: 1. ANALYSIS – structure of published films (the Shining European-vs-US cut comparison is the first corpus). Machine-detected scene boundaries as ground truth on one branch; human curation as a fork. 2. LINTING – report unpaired mirrors, orphan plants, per-pair duration deltas, a whole-cut tightness score. The chiastic claim becomes a number. 3. AUTHORING – later, the same machinery pointed at one's own footage: dailies auto-split, motifs tracked, the cut linted before render.

Domain tension (the whole design): - The interesting claims are EXACT (frame-exact pairing, center-scene identity, tightness scores). - The interesting readings are FLUID (this scene mirrors that one; this beat pays off that plant).

Resolution: scenes/motifs/beats as boring typed rows; chiastic truth DERIVED from position, never stored; human declarations kept as separate fields so the linter can report where declaration and derivation disagree – that disagreement is the analysis.

Inspired by: - ring composition scholarship (Mary Douglas et al.) - craft literature (mirror-moment midpoint, timed beats) - quantitative film studies (Cinemetrics-style shot data) - the corkboard: cards, colored by thread, moved by hand

NOT a video editor. NOT a media store (no frames, no clips – media stays on the laptop; the image holds only structure and metadata). NOT a generation pipeline (AI video gen is a separate future chapter gated on an http prim). NOT a platform for redistributing copyrighted media – the PUBLISHABLE artifact is analysis (numbers, reports, essays), never footage.

Killer property: a cut is a branch. The US cut and the European cut of the same film are sibling branches over shared film rows; "the recut tightened the ring" is a diff of two generated reports.

==========
LAYERING ==========
- libs/film/lib.pp – package film/? (seed on system/main) - data_rows types – film, scene, motif, beat - Routes via (route ?) – /film and report paths - scripts/film_ingest – laptop-side extraction ? Nopales source ? pp_eval (see

ingestion) - OCaml – existing CRUD/list/dict prims only; NO new kernel prims required for v1

Analysis functions are PURE Nopales over (rows ?) results: chiasm/pairs walks a sorted scene list two-pointer style; chiasm/center is the same recursion returning the singleton remainder (odd-length ring) or empty (even-length ring has a center SEAM, not a center scene – both shapes reported).

==========

DATA MODEL (v1) ==========

Types (deftype on load). Timecodes are INTEGER FRAME COUNTS; fps lives on the film row; seconds are a rendering concern.

film slug string ; "the-shining" title string year string fps string number ; "23.976" | "24" | "25" notes string

scene ; ONE detected/curated scene in ONE cut film string ; film slug cut string ; cut label, e.g. "eu" | "us" | "auto-t12" ord string int ; position in THIS cut slug string ; "room237" ("" until curated) summary string tc_in string int ; frames dur string int ; frames mirror string ; DECLARED mirror scene slug or "" ; derivation from ord is the truth; ; this field is a claim to check

motif ; a tracked recurrence film string name string ; "mirror" | "red-key" | ? kind string ; prop | line | color | sound | image

beat ; a motif occurrence inside a scene scene string ; scene id motif string ; motif name role string ; plant | payoff | echo tc string int ; frames, absolute

Why frames-as-strings: data_rows fields are strings; integer frames avoid float rounding across fps conversions. "To the second" claims are rendered, not stored.

NO shot-level table in v1 (shots matter for production, not for ring analysis; a shot type is a later subsection if the authoring workload arrives). NO user ACL – data_rows are shared (existing habitat rule). NO media blobs, thumbnails, or waveforms. Ever. Paths/hashes in notes fields at most.

==========

CHIASM AS DERIVATION (the core idea) ==========

The pairing is positional: in a ring of n scenes, scene k mirrors scene n-k+1. Therefore:

(chiasm/pairs scenes) ; two-pointer recursion: pair ends, ; recurse inward ? list of pairs (chiasm/center scenes) ; same recursion; odd ring leaves ; the CENTER SCENE as a singleton; ; even ring returns '() (center is ; the seam between the middle pair)

The mirror scene falls out of the structure. Nobody annotates it; the report NAMES it.

Report shape (film/report scenes) – a dict/assoc tree: center center scene or center seam shape odd | even pairs per-pair: slugs, dur delta (frames), shared motifs declared declared-mirror mismatches (claim vs position) orphans plant beats with no downstream payoff beat tightness sum of per-pair dur deltas (lower = tighter ring)

The linter's most interesting output is DISAGREEMENT: - declared mirror ? derived mirror ? either the annotation is wrong or the film breaks its own ring deliberately (broken symmetry is itself a finding) - derived pair with strong shared motifs but no declared mirror ? an unnoticed mirror, proposed by the machine

==========

INGESTION (laptop-side, no new kernel prims) ==========

Scene boundaries come from the media file ON THE LAPTOP, never from the image:

ffmpeg -i cut.mp4 -an -vf "scdet=threshold=N,showinfo" \ -f null - ? cut log with pts_time per detected cut

scripts/film_ingest (ksh) parses the log and EMITS Nopales source – (film/scene-create ?) forms – piped to pp_eval over the existing /api/eval endpoint. The image receives already-structured data through the normal eval path.

Known detector limits (documented, accepted): - scdet finds CUTS, not

narrative scenes: dissolves, long fades, and match cuts slip or merge; threshold is per-film tuning. Adjacent detections are merged by hand during curation – this is expected work, not failure. - Detection lands on a MACHINE branch (e.g. cut label "auto-t12"); curation happens on a fork so ground truth stays pure and detector re-runs are diffable.

Optional later sources through the SAME emitter path: - whisper.cpp transcripts ? dialogue beats with timestamps - PySceneDetect XML/EDL export instead of raw ffmpeg logs - frame-similarity pass (pHash / ffmpeg signature) ? PROPOSED mirror pairs (machine seeds, human accepts)

========== CUTS AS BRANCHES ==========

Branch convention (per habitat-workflow session ritual): analysis/<film>/<cut>-auto machine ground truth analysis/<film>/<cut>-curated fork; slugs, mirrors, beats

Cut comparison = run film/report per branch, diff the trees. "The recut tightened the ring" becomes: tightness(eu) < tightness(us), with the per-pair table behind it.

========== PUBLISHING BOUNDARY ==========

Personal analysis of owned media for criticism is the intended use. What MAY be published from the habitat: reports, numbers, essays, generated diagrams. What MUST NOT be: footage, frames, audio, or anything that reconstructs the work. No public routes serve derived media; report routes are private by default.

model

film/scene/motif/beat types; integer-frame timecodes; declared-vs-derived mirror split

[status: planned]

> *[verify] smoke: load-library defines film, scene, motif, beat types; smoke: scene-create stores cut/ord/tc_in/dur as frame integers; smoke: mirror field is a declaration; derivation comes from ord; smoke: fps lives on film; seconds are render-only*

ingestion

laptop-side scdet/ffmpeg ? Nopales emitter ? pp_eval; machine branch stays pure, curation is a fork

[status: planned]

> *[verify] smoke: scripts/film_ingest parses an scdet log into scene-create forms; smoke: ingesting a short clip creates ordered scenes on an -auto branch; smoke: re-ingest at a new threshold lands on a new cut label, old branch untouched*

chiastic-analysis

chiasm/pairs + chiasm/center two-pointer derivation; film/report with pairs, declared mismatches, orphan plants, tightness

[status: planned]

> *[verify] smoke: chiasm/pairs on a known list returns end-paired list; smoke: chiasm/center returns singleton on odd ring, empty on even; smoke: film/report names the center scene from structure alone; smoke: declared mirror mismatch appears in report.declared; smoke: orphan plant (no downstream payoff) appears in report.orphans; smoke: tightness is the sum of per-pair dur deltas*

cut-comparison

cuts as branches; report-per-branch diff answers 'did the recut tighten the ring'
[status: planned]

[verify] smoke: two cut labels over one film produce independent reports; smoke: tightness diff between cuts is computable via pp_eval

ui

routes /film, /film/<slug>; ring report page with per-pair table and rainbow-style arc diagram via render-html
[status: planned]

[verify] smoke: GET /film 200 after load-library; smoke: GET /film/<slug> renders center, pairs table, tightness; smoke: diagram renders arcs between derived mirror pairs

agent-access

pp_eval film/? as the API; reports are data trees an agent can diff
[status: planned]

[verify] smoke: pp_eval film/report returns the full report tree; smoke: pp_eval scene/beats queries work per film and cut

30 borge/film-edit.borg

pricklypear-film-edit

Pricklypear book chapter: film editing – studio library (characters/assets), Nopales-authored EDLs, Blender VSE as headless assembly backend, render caching, gen-API bridge as future edge

[status: planned]

film-edit

Programmatic video editing: the image holds the studio library (characters, locations, props, style locks) and the project graph (shots, cast bindings, takes, EDLs). Blender does assembly and render headlessly; media lives on disk under content hashes. Nopales state is authoritative; .blend files and renders are derived artifacts.

[status: planned]

> *[verify] build: dune build image; smoke: (load-library "edit") installs edit/? bindings and routes; smoke: define assets/characters, a project, shots with cast, choose takes, emit an EDL; smoke: GET /edit lists projects; /edit/<slug> shows cast, shot table, and render state; smoke: pp_eval edit/? entry points work for agents*

==========

PRODUCT INTENT ==========

A REPL-driven video editing and assembly pipeline with pricklypear as the brain and Blender as the muscle.

The motivating observation: an edit is DATA – an ordered list of decisions over takes with in/out points – and every serious editing tool already treats it that way internally (the EDL is the oldest data structure in film). What no tool gives you: the decision list as queryable, branchable, agent-editable rows on a live image, with the GUI reduced to a view.

The pipeline (each stage is data in, artifact out): bible (characters/locations/style) ? lock canonical refs ? script ? shots + cast/ref bindings ? expand prompt+refs from library ? raw footage (gen API or camera) ? N takes per shot ? pick a take ? lines ? dialogue takes (TTS or mic) ? cues (beds + spot SFX) ? assemble (EDL) + sound bounce ? render ? review ? re-cut / regenerate single shots ? optional promote take ? library asset

Every decision is a row: asset locks, cast bindings, prompt, seed, model, backend, cost, rejected takes, chosen take, input snapshots on takes. The whole history is inspectable, forkable, promotable – fork a branch per cut (cast bible forks with it).

Division of labor (settled design, do not relitigate): - PRICKLYPEAR – brain: studio library, project graph, provenance, cost ledger, branching, orchestration vocabulary, prompt/ref expansion, EDL generation, render-cache keys. - BLENDER – muscle: assembly (VSE), compositing, authored animation (deferred, see below), final render. Headless (blender -b) for jobs; a live GUI session is a VIEW onto the same scene state, never the authority. - GEN APIS – rented GPUs over HTTP: video/image gen, premium TTS. Local ComfyUI is a LATER backend behind the same interface, gated on hardware. - FFMPEG/DAW – sound muscle: bounce and mix are derived artifacts of the line/cue graph. An Ardour/Reaper session is a VIEW with the same re-import rule as the Blender GUI; the session file is never adopted as truth. - DISK – media files content-addressed; the image holds hashes + paths + provenance, NEVER blobs for motion or audio masters (UI thumbs may dual-point – see library).

Authority rule (the invariant that keeps this coherent): NOPALES STATE IS AUTHORITATIVE. .blend files and renders are build artifacts, regenerable

from rows. Hand-tweaked .blend truth is forbidden – it silently forks the project away from its provenance. If the GUI is used, its changes are re-imported as row edits (a documented import path), never adopted by copying the file back.

Inspired by: - the edit decision list (Hollywood's oldest data structure – timecode-addressed cuts as text) - build systems: render cache keyed on hash of inputs; change nothing, cache hit - Blender's own architecture: a live persistent image driven from a console – same shape as pricklypear

NOT a human NLE replacement (Kdenlive stays the hand-tool; it can open MLT/EDL exports for human touch-up). NOT a blob store for clips. NOT real-time collaboration. NOT locked to one gen vendor – backends are config, not design.

Killer properties: - Render caching falls out of the object model. The EDL is rows; hashing the canonical print yields a cache key; a render is a memoized pure function of project state. Re-cut one shot, re-render only what changed. - Continuity is a library problem, not a prompt-paste problem. A character is a stable id with a locked appearance bundle; shots bind to ids; gen expands bindings. Fix Mara once, re-gen only shots that cast her.

========== LAYERING ==========

- libs/edit/lib.pp – package edit/? (seed on system/main) - data_rows types – asset, project, shot, shot-cast, shot-ref, take, edl-entry, render-job, line, cue, mix-job - Routes via (route ?) – /edit and project/report paths - services/blender_bridge – OCaml daemon: typed commands in, bpy socket calls out; owns the render job queue - blender/bridge_addon.py – thin socket-eval addon inside Blender; PYTHON IS FIRMWARE, written once, checked in - scripts/render_worker – headless blender -b invocation - New kernel prims – http (client), spawn (subprocess), file-stat/read-bytes-range. Additive, general-purpose; specified in their own subsection below because they are kernel surface, not package code.

v1 vs later, honestly split: v1 (no new prims): studio library + cast/ref bindings, project graph, EDL rows, EDL export to MLT XML + Blender scene script ON DISK, manual render, import of render results, promote still/clip metadata into assets, line/cue sound rows, cue-sheet + ffmpeg script export ON DISK with manual bounce, import of bounce results. Fully useful for bibles, human-driven assembly, and agent-authored structure before any gen API. v2 (prims land): bridge orchestration, job queue, gen-API submission that EXPANDS library bindings and snapshots inputs on each take, render cache automation, TTS batches, automated dailies bounce via spawn – the async loop.

========== DATA MODEL (v1) ==========

Types (deftype on load). Timecodes integer frames, fps on project – same convention as film-analysis.

—- Studio library (stable identity) —-

asset ; character, location, prop, style lock, voice, ? project string ; project slug, or "" studio-wide ; on this branch slug string ; "mara", "diner-ext", "theme-A" kind string ; character | location | prop | ; style | voice | sfx | score | ; plate | still | other summary string bible string ; long notes / backstory / art dir appearance string ; locked positive prompt fragment negative string ; locked negative fragment primary_hash string ; canonical still/sheet/sample primary_path string ; relative to media root ref_hashes string ; JSON list of extra content hashes ; (turnarounds, costume variants) voice_ref string ; voice id or asset slug (kind=voice) lora_ref string ; backend adapter id, or "" media_id string ; optional habitat media_blobs UUID ; for UI thumb ONLY (see boundary) meta string ; JSON bag (non-query essentials)

A character is an asset with kind=character (not a second identity table). Wardrobe/costume variants are assets (kind=prop or kind=still) bound via shot-cast.wardrobe.

—- Project graph —-

project slug string ; "short-01" title string fps string number resolution string ; "1920x1080" budget_cents string int ; spend cap for API gen notes string

shot ; one narrative shot in the current cut project string ord string int ; position in cut slug string summary string ; script beat prompt string ; ACTION/dialogue prompt only; ; appearance comes from cast/refs dur_target string int ; frames, planned

shot-cast ; who appears in the shot shot string ; shot id asset string ; asset id (usually kind=character) wardrobe string ; optional asset id, or "" role string ; hero | featured | background

shot-ref ; non-cast reference bindings (typed, not JSON bag) shot string asset string ; location plate, style, prop hero? purpose string ; ip-adapter | style | depth | ; loc | prop | plate | other weight string ; backend hint, "" ok

NOTE: an earlier sketch had shot.refs as a JSON hash list. That is REJECTED. Cast and refs are rows so "which shots use Mara?" is a closed query. Ad-hoc one-off hashes (no library entry yet) are allowed by first creating a minimal asset (kind=still) or via take-local override fields only at gen time – not a free-form JSON column on shot.

take ; one candidate clip (or still) for a shot shot string ; shot id n string int ; take number backend string ; "fal-wan" | "replicate-ltx" | ; "local-comfy" | "camera" | ? model string ; model/version string seed string cost_cents string int ; 0 for local/camera media_hash string ; content hash of the file on disk media_path string ; path RELATIVE to a media root state string ; pending | done | failed | rejected rating string ; "" | human note on review input_snapshot string ; canonical JSON of EXPANDED gen ; inputs at submit (prompt, asset ; ids, appearance fragments, ref ; hashes, model). Immutable after ; land. Reproducibility + cache. source_assets string ; JSON list of asset ids used

edl-entry ; the cut itself – one row per timeline segment project string cut string ; cut label (branch-correlated) ord string int take string ; take id tc_in string int ; source in-point, frames dur string int transition string ; "cut" | "dissolve:N" | "fade:N" audio string ; take id of TTS/dialogue, or ""

render-job project string cut string edl_hash string ; cache key: hash of canonical EDL state string ; queued | running | done | failed out_hash string ; result media hash out_path string log string

—- Sound (production audio graph) —-

line ; one spoken line of dialogue project string shot string ; shot id the line belongs to ord string int character string ; asset id (voice via voice_ref) text string notes string ; performance: "whisper, rushed" take string ; chosen audio take id, or ""

Audio takes are ordinary take rows on the same shot with backend "tts-*" (premium or local) or "mic" (recorded). line.take points at the chosen one. ADR here means only: a replacement read is a new take + choose; the history of reads is the take list. No special ADR table.

cue ; continuous or spot sound on the TIMELINE project string cut string ; cut label (same as edl-entry) ord string int mode string ; bed | hit asset string ; kind=ambience | score | sfx tc_start string int ; timeline frames dur string int ; hit: 0 means asset length gain_db string number loop string ; "1" to tile a short bed

mix-job ; bounce as memoized function of the audio graph project string cut string cue_hash string ; hash of canonical print of cues ; + chosen line takes + profile profile string ; named preset, e.g. "dailies" state string ; queued | running | done | failed out_hash string out_path string log string

Sound obeys the same build-system rule as picture: the bounce key is a hash of canonical rows; nothing changed means cache hit. edl-entry.audio remains a per-strip dialogue pointer for assembly; lines + cues are the production graph that feeds it.

==========

LIBRARY RULES (characters and assets) ==========

Three layers – do not collapse them:

1. LIBRARY – asset rows: stable identity + locked appearance/voice/ref bundle + bible text. 2. BINDING – shot-cast / shot-ref: usage in a shot. 3. INSTANCE – take rows: what was actually generated or filmed, with input_snapshot frozen at submit.

Expansion (used by gen-submit and by a pure preview prim): (edit/expand shot-id) ? action prompt + each cast asset.appearance (+ wardrobe.appearance) + each shot-ref asset.appearance as appropriate + union of primary_hash/ref_hashes for ref images + voice_ref for TTS lanes That structure is what input_snapshot stores.

Promote path (library grows from production): (edit/promote-take take-id kind slug) ? asset row whose primary_hash/path copy the take's media; optional media_id if a UI thumb is registered. Human may then edit appearance/bible and re-bind shots.

Analysis vs production (boundary with film-analysis): asset/character = production bible (what we generate with) motif/beat = narrative lint (what the cut means) Do NOT merge tables. Optional shared slug later is fine; v1 keeps them independent.

Media boundary (two systems, both legitimate): - MOTION / masters / gen outputs: disk + content hash + relative path only. NEVER Postgres bytea. NEVER a VStr. - UI thumbs / character sheets for browser browse: MAY also use existing habitat media_blobs (storage.media, small image cap) via asset.media_id. Dual pointer is OK; disk hash remains authoritative for gen workers. - Third-party analysis corpus still never publicly routed – film-analysis.publishing unchanged.

NO per-user ACL (existing habitat rule). NO keyframe/curve tables in v1 – authored animation waits for hardware (Cycles on CPU and Eevee without GPU are not viable); transitions are the only motion v1 authors. NO claim that the data model alone fixes identity drift in gen models – the habitat enforces provenance, forced ref bundles, reject/regen economics, and branch+budget. Drift is a model problem; reproducibility is ours.

==========

SOUND RULES (written for people who are not sound engineers) ==========

Audio does NOT cut with picture. Sound is layers in time: lines – dialogue, shot-scoped, one chosen take per line beds – long spans: room tone, ambience, score region hits – spot SFX pinned to a timeline frame That is the whole model. Everything below is habit.

The four jobs, in the order they pay off: 1. dialogue coverage – every line has a chosen take, even scratch (mic whisper or fast local TTS). A cut with complete scratch reads is judgeable as cinema; a silent one is not. 2. presence – one ambience bed per location; cutting to a new room changes the bed. That alone sells space. 3. spot SFX – doors, cloth, footsteps at cut points. 4. mix – dialogue forward, beds well under talk, light limiter. Named presets, not knob archaeology.

Dailies sound profile (the loop v2 automates): chosen picture strips + chosen line takes + cues ? ffmpeg bounce with the dailies preset ? proxy mp4 with sound. Sound must NOT wait on the Blender render bridge; a dailies reel with scratch audio is a daily, a mute 4K master is not.

Voice: character.voice_ref ? kind=voice asset (sample hash + backend voice id). TTS reads are takes (job-shaped in v2, synchronous local in v1); mic reads are takes with backend "mic". Both are equally legitimate. Scratch first, hero reads later – the line id never changes.

Music: score is cues with mode=bed over named regions. Temp tracks are cue rows like any other; the asset row's provenance fields say what may ship. Gen music is takes with the same provenance discipline as picture.

NOT a DAW. NOT a sample store (wav bytes live on disk under hashes like motion masters). NOT auto-lip-sync: a line take is matched to picture by editing (trim, cutaway, hold), not by warping faces to audio.

==========
THE ASYNC LOOP (job-shaped, not call-shaped) ==========
SUBSTRATE: the jobs chapter (borge/jobs.borg) specifies the durable job/step/schedule machinery this vocabulary runs on – gen-submit is (submit-job ?), gen-status is (job-status ?) over the job row; render/mix JOBS claim through the same runner. The memoization-by-hash cache below is film-edit's own and orthogonal to job durability.

Gen and render take minutes. The vocabulary is job-first:

(edit/asset-upsert ?) ; library write (edit/shot-cast shot asset role) (edit/shot-ref shot asset purpose) (edit/expand shot-id) ; pure preview of bundle (edit/gen-submit shot-id :takes 3) ; expands + snapshots ; ? job ids, returns now (edit/gen-status job-id) ; pending | done | failed (edit/takes shot-id) ; landed takes + costs (edit/choose shot-id take-n) ; sets the cut's take (edit/promote-take take-id kind slug) (edit/line-choose line-id take-id) ; chosen read for a line (edit/cue-add cut mode asset tc dur db) (edit/dailies cut) ; proxy mp4 WITH sound (edit/mix cut profile) ; cue_hash ? cache lookup ; ? mix-job (or hit) (edit/render cut) ; EDL hash ? cache lookup ; ? render-job (or hit)

A render is a memoized pure function: same EDL rows ? same edl_hash ? cache hit. Budget is a branch property: fork for a re-gen experiment, spend lives on the branch, main keeps the released cut. Cost queries are closed questions over take rows (spend per project/cut/backend).

==========
BLENDER BRIDGE (protocol sketch) ==========
services/blender_bridge (OCaml, stdlib Unix sockets): - accepts lines from the image over TCP (localhost only): (scene-clear) (strip-add <edl-entry-as-sexp>) (render <out-path>) (status) - translates to bpy calls over the addon's socket - owns a FIFO job queue; one render at a time blender/bridge_addon.py (~100 lines, checked in once): socket server inside Blender eval'ing a FIXED, TYPED command set – not arbitrary code. bpy only. Two modes, same protocol: live – GUI open, bridge attached; the timeline is a live VIEW of row edits (re-import rule applies) batch – blender -b per render job, fully headless

==========
PUBLISHING / MEDIA BOUNDARY ==========
Own footage: fine to serve results via routes if desired. Third-party analysis corpus: media never enters the image, never gets a public route – per film-analysis.publishing.

model

asset/project/shot/shot-cast/shot-ref/take/edl-entry/render-job types; content-hash media refs; cost ledger and input_snapshot on takes

[status: planned]

> *[verify] smoke: load-library defines library + graph types with documented fields; smoke: take rows carry backend/model/seed/cost/media_hash/input_snapshot; smoke: edl-entry rows order a cut with tc_in/dur/transition; smoke: motion masters are hashes and relative paths only – no clip bytes in Postgres*

library

Studio library: asset rows (character/location/prop/style/voice/?), shot-cast and shot-ref bindings, expand + promote-take; dual media pointer for UI thumbs only

[status: planned]

> *[verify] smoke: edit/asset-upsert creates kind=character with appearance and ref_hashes; smoke: shot-cast and shot-ref bind assets to a shot; which-shots-use-asset is a rows query; smoke: edit/expand returns action prompt + cast appearances + unioned ref hashes; smoke: edit/promote-take mints an asset from a landed take's media_hash; smoke: asset.media_id*

optional; gen path still uses primary_hash on disk; smoke: no shot.refs JSON column – cast/ref are typed rows; details

sound

Production audio as data: line rows (dialogue with chosen takes), cue rows (beds + spot hits on the timeline), mix-job bounce keyed on cue_hash; dailies proxy with sound – no Blender dependency

[status: planned]

[verify] smoke: line rows bind text to character and shot; line.take chooses an audio take; smoke: audio takes are ordinary take rows (backend tts- or mic) on the shot; smoke: cue rows place beds and hits on the timeline in frames with gain_db; smoke: export writes a cue sheet + ffmpeg bounce script on disk for a cut (v1); smoke: edit/dailies bounces chosen picture + lines + cues to a proxy mp4 with audio (v2); smoke: mix-job on unchanged audio graph returns cache hit; changing a line take or cue changes cue_hash; details*

edl-and-export

canonical EDL print + edl_hash; export to MLT XML and Blender scene script on disk; re-import path for GUI edits

[status: planned]

[verify] smoke: identical EDL rows produce identical edl_hash; smoke: export writes a valid MLT XML file for a sample cut; smoke: export writes a Blender VSE build script for the same cut; smoke: Kdenlive opens the exported MLT XML without errors

render-cache

render as memoized pure function of EDL; render-job rows; cache hit on unchanged hash

[status: planned]

[verify] smoke: edit/render on unchanged cut returns cache hit, no job; smoke: changing one edl-entry changes edl_hash and queues a job; smoke: render-job records out_hash/out_path on done

blender-bridge

OCaml bridge daemon + bpy socket addon; typed command set; live GUI as view, batch for renders; job queue

[status: planned]

[verify] smoke: bridge addon loads in headless blender and answers status; smoke: strip-add commands from the daemon build a VSE timeline; smoke: render command produces a playable output file; smoke: one-at-a-time queue: second render waits for first

prims-http-spawn

KERNEL SURFACE: http client prim, spawn subprocess prim, file stat/read-range – additive, general-purpose, gate v2 orchestration

[status: planned]

[verify] smoke: (http-get url) returns status + body string; smoke: (http-download url path) streams response to disk, returns hash; smoke: (spawn argv) runs a subprocess, captures exit code + output; smoke: prims documented in lisp-interpreter chapter and seeded; details

gen-bridge

FUTURE EDGE: gen API submission (fal/Replicate/direct), job polling, take landing; expands library bindings into input_snapshot; local ComfyUI as later backend behind same interface

[status: planned]

> *[verify] smoke: edit/gen-submit expands cast/refs, writes input_snapshot, creates pending takes; smoke: poller lands done takes with media_hash + cost_cents; smoke: failed jobs mark takes failed with log tail; smoke: spend report sums cost_cents per project and cut; smoke: budget_cents cap blocks submission over budget; smoke: re-gen after asset appearance edit produces a new snapshot, not a silent reuse of the old lock; details*

ui

routes /edit, /edit/<slug>: cast/library panel, shot table with take picker, cut timeline view, render state, spend panel

[status: planned]

> *[verify] smoke: GET /edit 200 after load-library; smoke: project page lists assets/cast and shots in ord with chosen takes; smoke: binding a character to a shot shows on the shot row; smoke: choosing a take updates the cut and invalidates render cache visibly; smoke: spend panel shows cost totals per backend*

agent-access

pp_eval edit/? as the API; agents drive library, cast, gen, and cut loops

[status: planned]

> *[verify] smoke: pp_eval edit/asset-upsert, edit/shot-cast, edit/expand, edit/choose, edit/render work; smoke: an agent can fork a cut branch, retarget cast, swap a take, and render; smoke: an agent can promote a take into a character primary ref*

31 borge/jobs.borg

pricklypear-jobs

Pricklypear book chapter: jobs – durable background jobs, step checkpoints, and cron; host owns concurrency, guests see rows

[status: implemented]

– *agent note:* 2026-12 plan pass: flipped planned -> implemented. All four deliverable subsections (node-shapes, runner, step-checkpoints, schedules) are implemented and live: image/lib/jobs.ml (5-field cron + every-ms, SKIP LOCKED claim, runner fiber forked in boot.ml), prims_db.ml registers submit-job/submit-job-at/job-status/cancel-job/jobs/schedule/unschedule/schedules. The label lagged the ship.

jobs

Durable jobs as typed data nodes; a host-owned runner fiber; step checkpoints for exactly-once; schedules for cron. The async story for Nopales is job-shaped, not call-shaped.

[status: implemented]

==========

WHY THIS CHAPTER EXISTS ==========

A web framework needs background tasks and cron. AI calls take minutes; renders take longer; backups and feed polls must happen with no request in flight. Today every such feature would hand-roll its own deferral, and the book already carries three fragments of the same idea: film-edit's job-first vocabulary (gen-submit / gen-status), image.primitives-layer's deferred "host cron/scheduler table", and one-tree's user-keyed data placement. This chapter is THE async substrate: one design that those surfaces build on.

The shape is dictated by decisions already made: - ONE IMAGE PROCESS – the worker is an in-process host fiber, not a fleet. No broker, no Redis, no worker tier (the Celery shape is a symptom of stateless web workers; we have the opposite constraint). - THE TREE IS TRUTH – jobs are typed data nodes, not rows in a queue table. Inspection, backlinks, provenance, and /data browsing come free. (SUPERSEDED – see the node-shapes reconcile note: v1 ships a DEDICATED jobs table, migrations/015_jobs.sql, not tree nodes. Kept here as the original plan's rationale; the dedicated-table design below records why the implementation diverged.) - HOST OWNS CONCURRENCY – Nopales has no fiber/async API and gains none. Guests submit jobs and read job rows; the host runner fiber calls back into eval through the same Control rim an HTTP request uses.

==========

THE MODEL: CALLERS AND CALLEES ==========

Eval is always direct-style and request-shaped. What varies is WHO CALLS:

HTTP request ?? REPL line ???? Control.perform_prim ?? rim (Eio fiber) runner fiber ?? (one-shot effect) does the I/O

The runner fiber is a third caller into the same dispatch. From a job function's perspective a job run is indistinguishable from someone hitting a route: same prims, same budget accounting, same with-transaction semantics. No guest-visible suspension exists at any point in this design.

==========

NODE SHAPES (one-tree placement – RECONCILED 2026-08) ==========

RECONCILE: the implementation ships a DEDICATED table, not tree nodes. migrations/015_jobs.sql + image/lib/jobs.ml + the boot-time runner fiber (image/lib/boot.ml) deliberately chose a dedicated 'jobs' table (plus 'job_steps' and 'schedules') over tree_nodes/data.shared.job.* for three reasons, per the 015_jobs.sql header comment: 1. SKIP LOCKED CLAIM PROTOCOL – the runner's due-scan ('UPDATE ... WHERE status='pending' AND run_at<=now

ORDER BY run_at FOR UPDATE SKIP LOCKED LIMIT 1') needs real typed status/run_at columns and a partial claim index (jobs_claim_idx ON jobs(run_at,id) WHERE status='pending'). A matpath subtree scan over data.shared.job.* cannot drive that partial index and would force a tree walk per claim's ORDER-BY. 2. KERNEL AUTHORITY – migrations are the git-source-of- truth kernel tier (AGENTS.md branching authority); job rows are kernel state, so they belong to a migration table, not a branch-lens Nopales type. 3. STRONGER BRANCHING EDGE – with a dedicated table (and no branch id on the rows), fork/merge/promote STRUCTURALLY cannot copy, diff, or move a job – the 'NEVER participates in branch lenses' hard edge holds by construction, as originally intended. The tree-truth aspiration (inspection/backlinks//data browsing) is NOT met for jobs in v1: jobs are inspected via the job prims + /data is not a job surface. This is an accepted v1 tradeoff (kernel-backed queue state); a future projection onto tree nodes is possible but not planned.

The original tree placement (kept for the decision trail):

job data.shared.job.<id> job-step data.shared.job-step.<job-id>.<step-name> schedule data.shared.job-schedule.<name>

Shared placement was deliberate: the runner (host identity) must claim and mutate job rows regardless of who submitted, and path-prefix confinement on data.u.<user>.* would make the runner a privilege exception. Visibility is enforced at the PRIM layer instead: (jobs) and (job-status id) confine to the caller's submitted_by unless the caller is admin – same rule the notes garden applies to its shared indexes. submitted_by / branch / run_at are set at submit time and are NOT updatable through the CRUD prims (the runner owns status transitions).

job fields (migrations/015_jobs.sql): id uuid PK, gen_random_uuid at submit fn string name of a 1-arity Nopales fn args json single argument value branch string code lens captured at submit time submitted_by string user who submitted (visibility key) status enum pending | running | done | failed | dead run_at epoch-ms earliest start attempts int runs so far max_attempts int default 3; exhausted ? dead result json return value on done error string last failure message runner_id string image boot instance that claimed it created_at timestamptz submit time (list ordering)

job-step fields (checkpoint rows): job_id, step_name, value json, finished_at epoch-ms

schedule fields: name, fn, args, branch, cron string (5-field) OR every-ms int, next_run epoch-ms, enabled bool

========== GUEST PRIMS (the whole Nopales surface) ==========

(submit-job "fn" args) ? job-id; captures caller's branch + identity; run_at = now (submit-job-at "fn" args when) ? job-id; when = epoch-ms (job-status id) ? the job row (confined) (cancel-job id) ? pending ? failed/cancelled (jobs) ? caller's jobs, recent first (with-step "name" (lambda () body)) ? checkpoint boundary; inside a job: memoized by (job-id, name); outside a job: plain function call (NOTE: named with-step, not step – libs/rb already defines a 'step' recipe-DSL function; reserving the flat name 'step' for a kernel special form would clobber that library) (schedule "name" "cron-or-ms" "fn" args) ? upsert (unschedule "name") ? disable (schedules) ? list

Capability effects: submit/cancel/with-step are WriteDb; status/jobs/schedules are ReadDb; schedule/unschedule are WriteDb. Job FUNCTIONS carry whatever effects their bodies carry – inference + badges work unchanged because a job body is ordinary Nopales on an ordinary branch.

THERE IS NO await-job. Guests never block on a job; they poll rows (htmx fragment refresh in the browser, job-status in the REPL). Result delivery is pull, not push – the same vocabulary film-edit specified for gen/render.

========== THE RUNNER (host side) ==========

One Eio fiber forked at boot, beside the HTTP accept loop and REPL. Loop:

1. CLAIM – due jobs (status=pending, run_at <= now), oldest first, claimed with SELECT ... FOR UPDATE SKIP LOCKED + status?running + runner_id, committed before eval begins. SKIP LOCKED is correct from day one and is exactly the claim protocol an out-of-process worker (the Blender bridge shape) would run against the same rows later. 2. EVAL – resolve fn on the job's recorded branch, call it with args, under a synthetic request context (whoami = submitted_by, branch = recorded branch) – the same session record the REPL fabricates, no HTTP request involved. Standard eval budget applies. 3. RECORD – success ? status=done + result; exception ? attempts+1 and either status=pending with run_at = now + backoff(attempts), or status=dead when attempts = max_attempts.

WAKE-UP: sleep until min(run_at) of pending jobs, with a bounded poll interval (tunable, default seconds) as the floor so earlier inserts are never missed by a stale sleep target. v1 has no LISTEN/NOTIFY.

RECOVERY: each boot stamps a fresh runner_id. After claim and before new work, the runner fails jobs left status= running by a DEAD runner_id – they were mid-eval at crash – retrying them under the attempts/backoff rule. Single-image means the "fleet" is one fiber; runner_id exists so crash recovery is explicit instead of implied.

STEP CHECKPOINTS (the DBOS idea, Nopales-shaped)

A job that dies at minute 9 of 10 should not re-spend minutes 1-8. (with-step "name" thunk) is the boundary:

- First execution of (job-id, "name"): run the thunk, write the checkpoint row, return the value. - Re-execution (retry after crash): the checkpoint row exists ? return its value WITHOUT running the thunk.

Steps compose with with-transaction to give EXACTLY-ONCE for DB-writing steps, the DBOS co-location trick in our idiom: open one dynamic transaction, run the thunk (all its creates/updates ride the same connection), write the checkpoint row in the SAME transaction, commit. Either the writes and the checkpoint both land or neither does – the step provably never double-executes. The guest never sees a transaction handle; exactly-once is the default shape of (with-transaction (lambda () (with-step ?))), not a discipline.

Resume is by CHECKPOINT LOOKUP, not code replay: there is no determinism constraint on the job body. The one rule: a step name is a stable string within one job definition – (with-step "name" ?) enforces this syntactically. Job code is pinned at submit time (branch + fn name), so a redefine on main mid-job does not change what a running retry resolves.

v1 steps are RETRY-scoped: they shorten re-execution of a retried job; they do not suspend a live eval and resume it across boots mid-run. A crash mid-job restarts the job function from its top, replaying checkpoints.

SCHEDULES (cron)

Cron is sugar over the same machinery: a schedule row with next_run; the runner's due-scan treats due schedules like due jobs – submit the job, then compute the next next_run from the cron string or every-ms interval and write it back. One queue, one claim path, one recovery story. Missed fires while the image was down collapse to ONE catch-up run at boot (a backup that was down for three days runs once, not seventy times).

HARD EDGES (non-goals for v1)

- NO guest-visible fibers, promises, or await. Host owns concurrency; guests see rows. - NO in-image out-of-process isolation. A job that wedges in a blocking rim call can stall the image; eval budgets bound runaway Nopales, dead-runner recovery bounds crashes, and genuinely long external work belongs to an external worker speaking the same SKIP LOCKED claim (film-edit's Blender bridge is the precedent shape). - NO multiple queues, priorities, rate limits, unique-job

constraints. Single FIFO queue, at-least-once at the job level, exactly-once at the transactional-step level. - NO mid-job guest notification. htmx polling or job-status reads only. - Job state NEVER participates in branch lenses. Paths contain no branch id; fork/merge/promote structurally cannot copy, diff, or move a job (one-tree branching- interaction stands by construction).

========== RELATIONSHIP TO EXISTING CHAPTERS ==========
- film-edit: gen-submit / gen-status / render-job become applications of THIS substrate (a render job = a job row whose fn calls the Blender bridge); film-edit keeps its memoization-by-hash cache, which is orthogonal. - image.primitives-layer: the deferred "host cron/ scheduler table" is this chapter's schedule nodes. - one-tree: jobs are the first shared-NAMESPACE data type placed under data.shared.* by design; the placement and its prim-level confinement are specified above. - lisp-interpreter.effects: new prims join the prim ? effect-set table; nothing else about inference changes.

node-shapes

job / job-step / schedule rows in a dedicated migration table (015_jobs.sql): field contracts, runner-owned status transitions, SKIP LOCKED claim index; v1 is a dedicated table, not data.shared tree nodes (see details RECONCILE note)

[status: implemented]

> *[verify] build: dune build @all; test: image/test: jobs_test.ml node-shapes – job node create/claim/status transitions (submit?pending?claim running?record done/failed?dead) incl. runner-owned field confinement (submitted_by + branch pinned at submit; (jobs)/(job-status)/(cancel-job) confine to caller unless admin; cancel only pending and only yours); smoke: pp_eval '(submit-job ?)' ? (job-status id) shows submitted_by + branch pinned, then the runner fiber drives it pending?running?done with result (jobs are inspected via the job prims, NOT /data – v1 reconcile: no /data/job surface)*

runner

boot-time Eio runner fiber: SKIP LOCKED claim, synthetic request context, budget, attempts/backoff, dead-runner recovery at boot

[status: implemented]

> – *note:* 2026-12 ralph loop 19 (campaign 4.0 plan verify) found the runner ARGS bug: eval_job wrapped job.args with Printf.sprintf "%S" (boot.ml), so a jsonb NUMBER arg was evaled as a STRING literal -(submit-job "f" 1) stores args=1 but the runner evaled (f "1") -> Interp_error -> retries -> dead. Numeric/bool/object args all failed; only JSON-string args round-tripped. 4.1 FIXED it (Jobs.eval_src): job.args is the jsonb column rendered ::text = one JSON literal, spliced DIRECTLY as the call (never %S-quoted), so numeric/object/bool args eval as their real type, and the runner surfaces the REAL error text (pp_interp_error) instead of opaque Interp_error(_). ARG-ENCODING CONTRACT on submit: the submit-job VStr branch stores the raw string as jsonb (unquoted), so a string arg must already be valid JSON -(submit-job "f" "\"hello\"") for the string hello; align submit+runner on one encoding (prim surface + Jobs.eval_src + boot runner are consistent; see image/test/jobs_test.ml args-bug suite).
>
> *[verify] build: dune build @all; test: image/test: jobs_test.ml runner – claim skips locked rows (second-connection FOR UPDATE); exponential backoff requeues into the future / exhausted?dead; dead-runner reaper fails stale running rows (retry or dead) while leaving our own runner_id*

untouched; smoke: submit a job that sleeps, watch status pending?running?done with result via (job-status); smoke: submit a job whose fn raises ? attempts increments, retry, eventual dead; smoke: restart image mid-job ? stale running row recovered per attempts rule

step-checkpoints

(with-step "name" thunk) memoized by (job-id, name); with-transaction composition gives exactly-once for DB-writing steps; plain call outside jobs
[status: implemented]

[verify] build: dune build @all; test: image/test: jobs_test.ml step-checkpoints – job-id fiber binding (with_current_job_id scopes current_job_id_opt; null outside a job) + checkpoint memoization round-trip on job_steps (write reads back, upsert overwrites, per (job-id,name) key independent across step names and across jobs); smoke: two-step job killed between steps via forced retry: first step not re-executed (side-effect counter row stays 1)

schedules

cron/every-ms schedules as rows; due-scan fires by submitting a job + advancing next_run; downtime collapses to one catch-up run
[status: implemented]

[verify] build: dune build @all; test: image/test: jobs_test.ml schedules – cron next-fire computation (every-minute boundary, across-month Jan-1, impossible Feb-30 never matches) + every-ms drift/interval + spec classification (integer is not cron) + schedule CRUD/claim/advance/disable lifecycle on the schedules table; smoke: pp_eval '(schedule "t" " * * * *" fn args)' ? runner fires a job within a minute; (schedules) shows advanced next_run; smoke: disable via (unschedule "t") ? no further submissions*

non-goals

no guest async/fibers/await; no fleet control plane; no multi-queue v1; out-of-process workers via shared claim protocol only
[status: planned]

[verify] test: conformance: no Nopales binding exposes await/promise/fiber (surface lint over prim table)

32 borge/forge.borg

pricklypear-forge

Pricklypear book chapter: forge – executable spec graphs; borge's workflow as habitat data. LLMs draft and review; mechanical checks flag anomalies; the human gates each level of the descent.

[status: planned]

> – *agent note:* Design rationale, from the 2026-08 conversation that produced this chapter: - "Procedural borge": the spec stops being a document you consult and becomes a program you run. Phase transitions are driven by code, not by a human typing plan/make/review. - Victory conditions are the artifact; code is a cache. If codegen trends free, the expensive scarce thing is the VERIFIED SPEC GRAPH – every node carrying its win condition, provably green. Code is re-derivable output of the graph. - The frontier is computed, not judged: free-vars over drafted bodies derives what is missing; build order is a topological walk, not an LLM guess. - Mechanical flags first, judgment as residue: capability escalation, missing victory condition, dangling refs, cycles, orphans – all computable. The human reviews a list of anomalies, not a wall of nodes. Review fatigue is the quiet killer of every human-in-the- loop system; the defense is surfacing LESS over time, not more. - Human gates each level of the tree descent. Mistakes get cheaper as you go down; attention is allocated top-heavy because blast radius is top-heavy. Progressive disclosure of commitment: you are never asked to review more than one delta of abstraction at a time. - The loop lives IN the habitat (a Nopales package), not in the laptop harness. Separation of powers: drafter and driver must not be the same mind. The harness agent is demoted to drafter; the human answers the loop via the flag queue, does not drive it. - Reroll is INFORMED: reroll-with-note attaches a constraint to the node; accumulated constraints are a durable style guide the drafter cannot drift from. Otherwise reroll is a slot machine. - UI LAST, on evidence: you cannot design the approval UI until you know which flags actually fire in practice. The boring anomaly queue beats the gorgeous graph explorer. Build the loop headless; the queue teaches you what to draw.

forge

Executable spec: the borge discipline as a living graph of typed nodes in the habitat. A spec is a tree; each node carries a machine-checkable victory condition; LLM drafters fill leaves bottom-up along a computed frontier; mechanical checkers flag anomalies; the human approves each level before the descent continues.

[status: planned]

==========

WHAT THIS CHAPTER IS ==========

Borge today is a CLI over .borg FILES. This chapter is the same discipline re-expressed as HABITAT DATA: the spec is a tree of typed nodes, the workflow is a Nopales interpreter loop running inside the image, LLM calls are drafting and review passes over nodes, and the human's plan/make/review commands become gates the loop waits on. Nothing about the trust model changes: the human promotes, the human gates, mechanical checks run before any judgment is asked for.

The package is libs/forge – pure Nopales, product-shaped, living on branches like every other product lib. It gains ONE new kernel prim (forge-effects, be-

low) and reuses everything else: tree nodes, data rows, branching prims, the ai-authoring LLM surface, the jobs runner, and the notes/log/docs conventions of habitat-workflow.

THE GRAPH (node shapes, one-tree placement)

A spec graph is a set of typed data nodes. Per one-tree, code-side nodes live under the branch lens; graph STATE (status, review verdicts) is shared data, keyed by owning user, under data.shared.forge.* – same placement argument as jobs: the loop (host identity) must mutate node rows regardless of who is drafting.

forge-node data.shared.forge-node.<graph>.<path> forge-flag data.shared.forge-flag.<node-path> forge-graph data.shared.forge-graph.<name>

forge-node fields: graph string owning graph name path string "root/auth/login" – position in tree kind enum requirement | module | function | test title string one-line purpose (human-readable) body string Nopales source when kind=function; else "" verify string Nopales source of the victory check effects string declared badge (pure|read-db|write-db|?) status enum planned | drafted | verified | flagged | approved | implemented constraints json accumulated reroll-with-note strings parent string path of parent node ("" for root) children json list of child paths (denormalized index) branch string code lens this node was drafted on author string whoami at last mutation target string existing binding this node modifies ("" for greenfield; when set, the drafter is given the binding's current source as context and verify may run a regression) test-fn string existing test binding to run as a regression after install ("" disables; when set with target, verify-node installs the drafted body as the target binding, calls test-fn, restores)

forge-flag fields: node_path, check (enum: capability|victory|dangling|cycle|orphan|size|judgment), detail string, resolved bool, resolution string ("" until a human or the loop closes it)

forge-graph fields: name, root_path, status (planning|descending|verifying|done|stalled), created_by, created_at

The DENORMALIZED children index exists because backlinks over body strings is the wrong query; children must be enumerable without parsing. parent is the canonical link; children is maintained by the same mutation fns (one transaction).

THE FRONTIER (planning is a computation, not a judgment)

The single strongest idea in this design: build order is DERIVED FROM CODE, not guessed by an LLM.

free-vars: walk a parsed body; collect symbols that are not bound by any enclosing lambda/let/define and are not kernel prims or existing bindings. unresolved: free-vars filtered against bound names. frontier: all planned nodes whose unresolved set is empty – leaves whose dependencies all exist. stub-children: when a draft's free-vars reference names that exist NOWHERE (not prim, not binding, not a sibling node), those names become NEW planned child nodes. The drafter invented a helper; the graph records the debt.

Bottom-up falls out: a node can only be DRAFTED when its dependencies are all at least drafted. The frontier is recomputed after every mutation. There is no planner prompt; there is a graph query.

Cycles: if drafting A produces a stub B whose drafting references A, the cycle check flags it. Recursion is fine in Nopales but breaks the topological frontier – a cycle means the level needs restructuring (mutual-recursion node, or a declared base-case split), and that is a HUMAN decision, so the flag goes to judgment.

MECHANICAL FLAGS (checks that run before the human wakes)

Each flagger is a small pure-ish function over the graph. They run after every draft and after every level-descent proposal. The approval surface shows

FLAGS, not nodes.

capability child's inferred effects (via forge-effects prim) exceed the parent's declared badge. "read-only view" containing (create ...) is caught here, not by eyeballing. victory node has empty verify source. Unapprovable by definition; flagged before human sees it. dangling free-vars reference nothing: not prim, not binding, not a sibling/parent node. Either a missing stub or a typo'd name; both worth a flag. cycle frontier math found a dependency loop. orphan node traces to no requirement ancestor. Gold-plating detector: completeness means every requirement has a node AND every node traces to a requirement; no orphans either direction. size drafted body exceeds a review-in-one-glance threshold (default ~20 forms). Suggests a split; not a hard error. judgment the drafter itself asked for clarification. The "flag for review/clarification" of the tri-state review. This is the log/issues escape hatch, node-scoped.

The one new kernel surface:

(forge-effects "name-or-src") ? badge string Runs lisp/lib/effect.ml infer over the named binding (or parsed source) and returns EffSet.badge. This is the SAME analysis install-function already gates on; this prim merely EXPOSES it. Read-only, Pure effect.

THE LOOP (the interpreter that replaces plan/make/review)

One function, forge/step, is the whole state machine. It is deliberately boring:

(forge/step graph-name) 1. If unresolved judgment flags exist ? do nothing; the loop is BLOCKED on the human. Return the flag list. (The human's queue IS the control flow.) 2. Else if the current level has unapproved nodes ? run mechanical flaggers, present the level for approval, stop. HUMAN GATE. 3. Else if the frontier is non-empty ? pick one node, assemble a draft prompt (context: title, parent declaration, constraints, sibling summaries, the platform-ref note), call the drafter (agent-session + agent-say, or llm-seed for leaf functions), parse the result into body+verify, run mechanical flaggers, write the node. Return what changed. 4. Else ? the level is complete; propose the descent (stub children of approved nodes) and stop. HUMAN GATE on the new level.

forge/run is forge/step in a job, so the loop persists across HTTP requests and survives the laptop disconnecting. The jobs chapter supplies the fiber; forge supplies the step function. One job per graph, re-submitted after each step while status is not done/stalled.

TRI-STATE REVIEW: when a node's draft passes mechanical checks it goes to LLM review (a SECOND agent session, not the drafter's) which returns approve | deny | flag. Deny ? redraft with the review note appended to constraints. Flag ? judgment flag, human queue. Approve ? status verified (verify source passes) or stays drafted (verify needs a human run). Separation of powers is structural: drafter, reviewer, and driver are three different sessions.

REROLL-WITH-NOTE: the human's reject carries a reason. The reason appends to the node's constraints, and every subsequent draft prompt includes all constraints. The graph accumulates taste; the drafter cannot drift from a constraint it has been given twice.

THE HUMAN SURFACE (triage, not review)

v1 is deliberately minimal – the UI is deferred on PRINCIPLE (you cannot design the approval surface until you know which flags fire). Phase one surfaces:

/forge list graphs + status + open flag count /forge/<name> one graph: current level's nodes with flags, grouped by flag type; approve / reroll(with note) / veto per node; approve-all-clean button log/issues judgment flags land here too, one line each, per habitat-workflow

The approval page shows ANOMALIES. A node with zero flags and a passing verify renders as one line: title, badge, approve button. A flagged node renders its flag detail first. The human reads flags, not code, unless they choose to expand. This is the review-fatigue defense: the loop surfaces LESS over time, not more.

LEVEL GATE SEMANTICS: approving a level means every node at depth N is status>=approved and every flag at depth N is resolved. The loop will not draft depth N+1 until the depth-N gate passes. This is progressive disclosure of commitment: no one, human or LLM, evaluates more than one delta of abstraction at a time.

========== RELATIONSHIP TO EXISTING CHAPTERS ==========

- habitat-workflow: forge is the same discipline with the phases driven by code. Session ritual, obituary rule, file-don't-fix, and handoff notes are UNCHANGED – forge graphs write spec/<pkg> docs on promote, exactly as a human session would. The notes docs remain the living spec; the forge graph is the WORK-IN-PROGRESS view. - ai-authoring: drafters use propose-function / llm-seed / agent-session per that chapter's provenance and badge rules. forge adds NO new privilege path; machine-authored code lands on a branch and promotes through the same human gate. The ai badge applies to every forge-drafted binding. - jobs: forge/run is a job. The runner fiber is the only place the loop executes; no new concurrency is added. - one-tree: node shapes and placement are specified above per one-tree.branching-interaction. Graph state never participates in branch lenses (same argument as jobs). - lisp-interpreter.effects: forge-effects exposes the existing infer; nothing about inference changes.

========== HARD EDGES (non-goals for v1) ==========

- NO graph-visualization UI. A tree list with flags is enough; the force-directed graph explorer is a later nicety, built on evidence of what the queue needs. - NO autonomous promote. The loop never calls promote; it prepares a branch and a spec doc. The human runs pp-sync / promote exactly as today. - NO drafter-is-reviewer. The same agent session never both writes and judges a node. - NO unbounded descent. A graph declares max-depth at creation; the loop stalls (status=stalled, judgment flag) rather than drafting past it. - NO rewriting of borge-the-CLI. .borg files remain the git-tracked source of truth for the KERNEL. forge is for PRODUCT code (libs/*) where the habitat IS the source of truth. The two coexist; forge does not eat borge.

graph-model

forge-node / forge-flag / forge-graph typed nodes: data.shared placement, denormalized children index, status enum, constraints accumulation; target + test-fn fields anchor modify-existing nodes

[status: implemented]

> *[verify] build: dune build @all; test: image/test: forge node create/update; children index stays consistent with parent links across edits; smoke: pp_eval '(forge/new-graph "x")' ? row at /data/forge-graph; (forge/add-child ?) maintains both directions*

frontier

free-vars over parsed bodies; unresolved filter; frontier query; stub-children on unknown refs; cycle detection flags loops

[status: implemented]

> *[verify] build: dune build @all; test: image/test: free-vars of (lambda (x) (+ x y)) is {y}; frontier excludes nodes with unresolved deps; test: image/test: drafting a body referencing undefined name 'helper' creates a planned child node named helper; smoke: cycle A?B?A surfaces a cycle flag, not an infinite stub loop*

mechanical-flags

capability escalation via forge-effects prim, missing victory, dangling refs, cycles, orphans, size heuristic – each a small fn over the graph; judgment is the

human escape hatch

[status: implemented]

> *[verify] build: dune build @all; test: image/test: forge-effects prim returns badge for a named binding (pure for +, write-db for create); test: image/test: child with write-db body under parent declared read-db raises capability flag; smoke: node with empty verify source is victory-flagged before appearing on the approval page*

loop-and-gates

forge/step state machine (blocked?flagged?draft?descend); forge/run as a job; tri-state LLM review with separate reviewer session; reroll-with-note constraint accumulation; level-gate semantics; verify-node evaluates the victory expression on the current branch session

[status: implemented]

VERIFY EXECUTION SEMANTICS verify-node evaluates (let ((fn body)) verify) via eval-string on the current branch session. The drafter seed already instructs verifies to create and clean their own test data, so a verify may touch the DB on the current branch; this is accepted for v1.

REGRESSION MODE (modify-existing nodes): when a node has both target and test-fn set, verify-node runs a regression after the primary verify passes. It saves the target binding's current source (function-source), installs the drafted body as the target binding (eval-string define), calls test-fn, and restores the original source. Because Nopales closures capture the session env by reference, the test function's calls to the target name exercise the NEW body. If test-fn returns truthy the regression passes; otherwise a judgment flag is raised. Restoration is best-effort: if the target had no source (a prim), verify-node skips regression and falls back to the primary verify only, with a judgment flag noting the skip.

Scratch-branch checkout (evaluating verify on an isolated dev branch) is a FUTURE hardening, not v1. The current design trusts the drafter seed's create-and-clean discipline and the separation-of-powers review to catch misbehaving verifies.

> *[verify] build: dune build @all; test: image/test: step with open judgment flags is a no-op returning the flag list; test: image/test: reroll appends the note to constraints; next draft prompt contains it; smoke: full descent on a toy graph: root ? level 1 approved ? stubs ? leaf drafted ? verified, all via forge/run job; smoke: loop refuses to draft depth N+1 while any depth-N node is unapproved*

existing-app-bridge

Bootstrapping and modifying existing code: forge/import-function reads a binding's source (function-source) and creates a node status=implemented body=source verify=(begin #t); the target field names an existing binding a node modifies, and forge/draft-prompt branches to include the current source as context; test-fn enables regression verify per loop-and-gates

[status: planned]

> *[verify] build: dune build @all; test: image/test: forge/import-function on a live binding creates a node status=implemented with the binding's source as body; test: image/test: a node with target set produces a draft prompt that includes the target's current source; smoke: import a small package's functions into a graph; frontier sees them as implemented; a modify-existing draft references the current body*

human-surface

/forge list + /forge/<name> anomaly-first approval page (approve/reroll-with-note/veto per node, approve-all-clean); judgment flags also land in log/issues; NO graph-viz, NO autonomous promote

[status: planned]

> *[verify] build: dune build @all; test: image/test: approval page groups nodes by flag type; clean nodes render one-line; smoke: browser: /forge shows a seeded graph; approving a node advances the level gate; reroll dialog accepts a note; smoke: a judgment flag appears in both /forge/<name> and the notes log/issues doc*

non-goals

no graph-viz UI v1; no autonomous promote; no drafter-is-reviewer; max-depth hard stop; borge CLI untouched – forge is for product code, .borg stays kernel truth

[status: planned]

> *[verify] test: conformance: no forge binding calls promote or merge (surface lint over the package); test: conformance: reviewer session id ? drafter session id on every tri-state review*

33 borge/habitat-workflow.borg

pricklypear-habitat-workflow

Pricklypear book chapter: habitat-workflow
[status: implemented]

> – *agent note:* 2026-08-02 status reconcile: libs/spec (~600 lines) + promote_soft_warnings in session.ml; live spec/todo+log/sessions+log/issues docs; /book/todo 200. Soft lint only (no hard bounce yet – intentional v1). /spec health 500 is runtime bug not missing package.

habitat-workflow

Spec-in-the-image: the borge discipline (spec -> build -> compare) as a habitat-native workflow, with promote as the goalpost-immobility gate
[status: implemented]

> *[verify] smoke: notes docs spec/<pkg> exist for at least one package and pass habitat lint queries; smoke: redefining a flow-listed public binding without a spec/decisions mention shows up in the drift report*

THE PROBLEM THIS CHAPTER EXISTS TO SOLVE

Package development on the habitat is agent-driven and session-based (phone REPL, pp_eval, browser). Without an artifact that outlives the turn, agents exhibit three failure modes:

1. Multiple happy paths through one workflow – nothing records which path was chosen, so each session re-derives "how this works" from the code, and code under-specifies that. 2. Redundant problems – no registry of known problems, so the same issue is "discovered" twice and fixed two incompatible ways. 3. Flip-flop rewrites – solving A with B's approach, breaking B, reimplementing A. Nothing records that the current shape was a DECISION WITH A REASON, so broken-looking code reads as "never wanted" instead of "wanted, currently broken."

File-borge suppresses these by forcing intent to exist as an artifact (the .borg book) that outlives the turn, and by gating edits so the agent cannot move the target while shooting at it. This chapter ports the DISCIPLINE, not the file mechanics.

THE CENTRAL MOVE: GATE THE PROMOTE, NOT THE REPL

File-borge gates keystrokes (plan/make file locks) because a working tree has no cheaper boundary. The habitat has a much better boundary already: the branch lens. So habitat-workflow gates the PROMOTE. Pre-commit hooks vs CI.

- The REPL stays playful and chaotic. Agents may thrash freely on user/personal and user/dev branches. - Nothing reaches system/main unless the comparison loop passes (spec-checks). - Promote (branching.promote) becomes pp's commit, and it grows a checklist. Review becomes looking at a page, not running a toolchain.

THE LITERATE INVERSION

In file-borge the book and the code live in different formats in different files, so a whole tool (balance / drift / report) exists to correlate them. On the habitat, spec docs, function docs, and code are ALL sexps in the same store under the same branching. So the book stops being a source artifact and becomes a VIEW: a spec doc's outline owns the narrative order; the referenced bindings fill in underneath via their function-doc and source. Drift stops being a report you run and becomes RENDERING: a chapter whose referenced binding was renamed shows a hole where the paragraph used to be. There is nothing to sync because the book route is a query.

RELATIONSHIP TO THE FILE BOOK

The OCaml kernel (image/, lisp/, migrations/) remains governed by THIS multi-file .borg book; git stays its source of truth. Habitat-workflow governs Nopales package code (libs/*, and their live counterparts on branches). Habitat spec docs are NOTES DOCS, so they branch like everything else: humans edit them on system/main; agents on branches propose changes via spec-issues and pending edits that the human accepts at promote time. The authorship-integrity rule file-borge enforces by lint ("agents never touch human comments") is here STRUCTURAL: spec docs live on main, agent sessions cannot write main, so the violation is impossible rather than detectable. pp-sync snapshot remains the durability/vault path for spec docs exactly as for package code.

spec-docs

One notes doc per package (slug spec/<pkg>): flows, canonical paths, decisions, status – the intent artifact that outlives the turn

[status: implemented]

> – *agent note:* 2026-08-02: convention + tooling live – (notes/doc/get "spec/todo") #t; spec/doc, flow-entries, flows-resolve in libs/spec. Coverage across all packages is ongoing content, not missing mechanism.
>
> *[verify] smoke: (notes/doc/get "spec/todo") returns a doc with flows/decisions sections; smoke: a spec doc's flow entries name existing function bindings (resolvable via function-doc)*

Every package that receives agent-driven development gets ONE notes doc with slug spec/<pkg>. It is short and structured, not prose-heavy:

FLOWS The user-visible workflows, numbered. Each flow names the ONE canonical path through the bindings, e.g. "add -> todo/create -> todo/render-row". New code paths must attach to a flow or create one – this is what kills multiple-happy-paths.

DECISIONS An append-mostly log of non-obvious choices WITH REASONS ("render is a pure function of the row so htmx swaps stay dumb"). This is the flip-flop killer: the next session can distinguish "wanted, currently broken" from "never wanted."

STATUS One line per flow: works / half-built / broken.

Spec docs live on system/main (shared intent). Branch lenses mean an agent CAN fork a spec doc onto its branch to draft a proposal, but promote of spec docs is a human-visible, human-approved step (see spec-checks). Function-level docs (define docstrings, lisp-interpreter.define-docstring) are the per-binding layer; the spec doc is the per-package narrative layer. A function-doc entry SHOULD reference its flow number(s).

session-ritual

The agent-facing protocol: branch-scoped work, obituary rule, file-don't-fix, handoff – recorded in the live agents note

[status: implemented]

> *[verify] smoke: (notes/doc/get "agents") contains the session-ritual rules (obituary, file-don't-fix, handoff); smoke: log/issues and log/sessions notes docs exist and accrete entries*

The durable agent guide is the notes doc with slug agents (advertised in /agent.txt); habitat-workflow adds these rules to it. All are notes-doc mechanics, zero kernel work:

1. WORK ON YOUR BRANCH. Agent sessions define/route on <user>/personal or a session dev branch, never directly on system/main. End of session: diff the lens (which bindings did I touch?), self-review against the declared flows, promote or revert.

2. NO SILENT REVERSALS (the obituary rule). An agent may not replace or delete an existing implementation without first writing its obituary in the spec doc's decisions section: what it was, why it existed, what changed in the world that invalidates it. Writing the obituary forces the agent to notice it is flip- flopping; half the time it will realize the old approach was right and something ELSE is broken.

3. FILE, DON'T FIX. Out-of-scope discoveries go to the log/issues doc, one line each, instead of expanding session scope. This is borge issue new for the habitat – and since notes docs are data, it needs no gitignored directory.

4. HANDOFF NOTE. Session ends with ~3 lines appended to log/sessions: what changed, what is half-done, what was deliberately NOT done. Flip-flop mostly happens ACROSS sessions, when the next session cannot tell half-built from broken.

Enforcement is by instruction (agents note) first; mechanical checks (spec-checks) make violations visible rather than relying on agent virtue.

spec-checks

Promote-time comparison: spec->code resolution, code->spec acknowledgment, doc coverage – promote grows a checklist

[status: implemented]

> – *agent note:* 2026-08-02: spec/lint + promote-checklist + Session.promote_soft_warnings wired into promote_to_main and branches UI (soft, non-blocking). Hard bounce graduation still future; matches "starting as soft warnings".
>
> *[verify] smoke: promote preview lists flow bindings that fail to resolve on the source branch; smoke: promote preview lists public bindings redefined on the source branch with no spec/decisions mention; smoke: promote preview lists public bindings with empty function-doc*

Promote (branching.promote) is the commit of the habitat; this subsection hangs the comparison loop on it. Before upserting, promote (or its preview UI) runs three checks against the source branch and the package spec doc, and reports – starting as soft warnings, graduating to bounces for admin-into-main promotes:

1. SPEC->CODE: every flow in spec/<pkg> names its canonical bindings; they MUST all resolve on the source branch. This is borge drift, but exact – sexp references, not grep.

2. CODE->SPEC: every PUBLIC binding redefined on the source branch is mentioned somewhere in the spec doc or its decisions section. Redefining todo/render without a spec acknowledgment bounces. This is the anti-flip-flop tripwire: the mechanical form of the obituary rule.

3. DOC COVERAGE: promote refuses undocumented PUBLIC bindings (empty function-doc). define stays loose – docs are not an ABI change, they are a shipping requirement. Undocumented is fine on your branch, unshippable on main. Docs get written at the moment of "this is real now," which is also the moment the agent can best explain what it built and why.

"Public" = bindings referenced by routes or by other packages (the surface a consumer can name); internal helpers are exempt. SPEC DOC CHANGES travel through promote too, and are shown to the human explicitly ("this promotion also modifies spec/todo – accept?"). Friction, not locks: one tap to approve; the agent can never silently ratchet the goalposts.

DEPENDENCY NOTE: checks 2-3 want define's persist path to be synchronous enough that the check sees the write (the fire-and-forget persist race must be fixed first; .borge-bugs). A phase check that races the write is theater.

book-view

The literate inversion: spec doc outlines own narrative order; bindings inline underneath – drift becomes rendering

[status: implemented]

> *[verify] smoke: GET /book/todo renders the spec doc outline with referenced bindings' doc + source inlined; smoke: a spec doc referencing a renamed binding renders a visible hole instead of stale prose*

A route /book/<pkg> (private) renders the spec doc as a chapter: walk the outline; at each flow or binding reference, inline the binding's function-doc and source (both are sexps in the same store). The book is a query, so it is ALWAYS current – the thing Knuth wanted and files could never quite give.

DRIFT AS RENDERING: a reference that no longer resolves renders as a visible hole in the page (styled, obvious), not as a line in a report. The book physically shows its own staleness.

This complements, not replaces, the file book: the OCaml kernel's chapters remain .borg files. /book serves the Nopales package layer. Whether the two ever merge into one rendered volume is an open question, explicitly not decided here.

spec-health

Habitat health page: packages x (flows verified, doc coverage, drift holes) – borge report as a browser page

[status: implemented]

> *[verify] smoke: GET /spec lists every package with a spec doc, its flow resolution status, and doc coverage*

/spec is the habitat's standing health page (private, admin): one row per package that has a spec doc – flows resolving / broken, doc coverage count, count of drift holes in /book rendering, last promote. This is borge report for the image, sized for a phone screen: review becomes LOOKING AT A PAGE.

Discovery contract: describe / apropos-docs surface doc coverage alongside results ("todo: 14 bindings, 3 undocumented"), so the gap follows the agent around instead of waiting for a report run.

non-goals

What habitat-workflow is NOT

[status: implemented]

- NOT a port of borge-the-tool into Nopales. No balance, no parse, no lint pipeline; the store is the parser. - NOT kernel governance. OCaml/migrations stay under the file book and git, full stop. - NOT REPL gating. No plan/make modes on eval, no dynamic-extent phase locks on define. Play is unrestricted on branches; the gate is promote. - NOT a rewrite of branching.promote's data model. This chapter adds checks around the existing selective-upsert mechanism. - NOT mandatory for every package. One-off garden scripts and personal-branch experiments never need a spec doc. Spec docs are for packages with multiple agent sessions of history – exactly where the three failure modes bite. - NOT provenance/audit of AI-authored code – that is ai-authoring.provenance. This chapter consumes author info but defines no new attribution model.

> – *agent note:* 2026-08-02: documentation-only; marked implemented as spec-complete.

34 borge/habitat-book.borg

pricklypear-habitat-book

Pricklypear book chapter: the spec IS the book – borg chapters rendered beautifully to paper AND to the habitat, with live code examples

[status: implemented]

> – *agent note:* 2026-08-30 top-level smokes verified: index order + /book/chapter render + reader suppression (loop 17), run-button click-eval (loop 18), book/verify hole render (scripts/smoke_book.py), export/print order parity (borge loop 12). course-documents verified 2026-08-31 (P3.1, loop 23) – /book/course/cs-interpreters + the compose-never-copy smoke; all five top-level smokes now hold.
>
> – *agent note:* 2026-08-29 authored from the cs-curriculum design session (docs/cs-curriculum.md). Probe of ‘borge book print .‘ on the post-pivot tree: chapters land p788+ in readdir order, inline not honored, libs/*.pp + migrations/*.sql absent. This chapter specced before any implementation; borge repo workstream described in .borg-workstream below.

habitat-book

One spec, two projections: borg files stay the source of truth; borge renders them to paper; the habitat renders them to live, evaluable pages. Courses are compositions over chapters, never copies of them.

[status: implemented]

> *[verify] build: dune build image; smoke: (load-library "book") then GET /book shows borg chapter index; smoke: GET /book/chapter/philosophy renders doc prose with statuses suppressed in reader mode; smoke: nopales example block renders with a run button; clicking evals in the reader's session and shows the result; smoke: book/verify re-evals all nopales examples in a chapter; a stale expected value renders as a hole; smoke: borge book export emits the same chapter order as borge book print*

WHY

The borge book is the spec AND the intended CS textbook (docs/cs-curriculum.md). The paper pipeline renders files, not the literate tree: post-pivot chapters print on p788+ in readdir order, the Nopales product layer and SQL never appear, and clone history changes your book's TOC. Meanwhile the habitat has /book (spec docs, package layer) but the .borg chapters – the actual course readings – are not on the habitat at all, so no agent, flashcard, or route can reach them.

Five decisions (all locked unless reversed with an obituary in spec/<when ported>):

1. THE SPEC IS THE BOOK. No forked textbook document. If a chapter is unreadable as teaching prose, the chapter is wrong – write it better. Register is a RENDER MODE (workshop: statuses/verify/agent-notes visible; reader: suppressed), not a document fork. Drift stays visible (holes), workflow scaffolding stays toggleable.

2. GIT STAYS AUTHORITATIVE for .borg. Habitat is a READ- PROJECTION (seeded rows) of chapter content. Edit-in- habitat (chapter editing, marginals, propose-a-patch) is a v2 question, explicitly deferred – same reserve as habitat-workflow.book-view's "two books merge" line, now upgraded from open question to named v2.

3. ONE PARSER, TWO PROJECTIONS. borge (lib/lang) remains the only parser of .borg. ‘borge book print‘ and the new ‘borge book export‘ share

one inline-aware ordered walk and one example-fence convention; paper and habitat can never disagree about chapter content or order.

4. HABITAT-FIRST EVAL. The course teaches by running code, and the habitat is the natural runtime for Nopales examples: read-time eval in the reader's session, render-time verification over the whole book. OCaml examples are capture-at-render-time only in v1 (borge runs the build harness; habitat displays block + captured output; no live OCaml evaluator – a later substrate decision, not a prerequisite).

5. COURSES COMPOSE, NEVER COPY. A course is a typed outline document that ORDERS chapters (front matter -> chapters -> labs) and references them; a chapter lives in one place and appears in many courses. Per-course paper = a course root borg file consumed by the same two projections.

export-projection

borge-side render stream: ordered inline-tree walk, reader/workshop modes, JSON/sexp export artifact – the habitat's feed and the paper's spine

[status: implemented]

> *[verify] smoke: 'borge book export .' writes <stem>.book.json with chapters in inline order; smoke: export includes .pp package sources and migrations as code chapters; smoke: reader vs workshop mode differ only in scaffolding blocks; smoke: two fresh clones produce byte-identical exports (deterministic walk)*

BORGE-TOOL WORKSTREAM (lives in the borge repo; requirements authoritative here):

- INLINE-AWARE WALK. The root project's inline tree defines chapter order ('pricklypear.borg' -> borge/*.borg). Files not referenced by the walk sort lexicographically AFTER the inline tree (appendix), never readdir order. 'borge book print' takes '–root FILE' to select a subtree (course roots, single-chapter prints). - DETERMINISM. Byte-identical PDFs/exports across clones (probe showed readdir-order chaos; checksum the manifest). - TYPE COVERAGE. .pp (Nopales product layer) and .sql (migrations) become printable chapters. - MODES. reader (strip status/verify/agent-note scaffolding) vs workshop (today's everything). Mode is a render flag; both carry drift holes. - EXPORT. 'borge book export [DIR] -o <stem>' emits a JSON artifact: [{root, chapters:[{slug, title, nodes:[...], examples:[{lang, src, expected?}]}], code-chapters:[...]}]. Same walk as print. This artifact feeds the habitat loader. - EXAMPLE FENCES. doc-block convention (shared print/web): "'nopales / "'ocaml fenced blocks, optional 'expect =>' trailer line for verification. Parser extracts them into structured example records; prose keeps the raw text.

habitat-loader

seed the export into the one tree: chapters as typed nodes, refresh on release – reads then inherit branch/lens semantics for free

[status: implemented]

> *[verify] smoke: scripts/load-book.sh <stem>.book.json upserts book chapters without clobbering hand edits elsewhere; smoke: (load-library "book") renders an index from tree rows, not from files*

- Loader: release-time. public_artifacts/build-release run 'borge book export' and a small loader seeds chapter nodes (type 'book-chapter') under system/main. Additive upsert, same discipline as package seed (branching.seed-policy): never overwrite after manual edits land – book rows are seed, not live authority. - Shape: one node per borg SECTION/SUBSECTION (title/body/status/mode-flags), one per code chapter (path + source), one per example (lang/src/expected). Children ordered by the export's inline order – the tree's ordering semantics carry the book's chapter order (tree-time). - Offline dev:

loader refreshable by hand (scripts/load-book.sh); habitat never require borge at runtime.

web-renderer

Nopales package ‘book‘: /book index of borg chapters + package specs; /book/chapter/<slug> pretty-printed with references, example blocks, reader/workshop toggle
[status: implemented]

> – *agent note:* 2026-08-30 all four smokes verified. Loops 17+20: /book index lists the 34 borg chapters in export order + the spec + code corpora; chapter pages render headings/prose/sections from tree rows; reader strips statuses + verify blocks, workshop shows both. Loop 23 (P3.1): \"course groupings\" verified – /book lists the Courses section (notes kind=course via tree/find-by) and a second throwaway course renders the SAME chapter by reference with zero tree duplication. Kernel delta rode along in P2.2: prose-tokens/render-tokens one-scan prims + string-split promotion (budget, see libs/book obituary).
>
> *[verify] smoke: GET /book lists chapters in exported order with course groupings; smoke: chapter page renders headings/prose/code/examples from tree rows; smoke: binding tokens in prose link to /binding/<name>; missing ones hole-red; smoke: ?mode=workshop shows statuses and verify blocks; default hides them*

- ROUTES. /book becomes the index of BOTH corpora: borg chapters (new) and package spec docs (existing spec/book-page). Borg chapters at /book/chapter/<slug>; package docs keep /book/pkg/<pkg> alias of current /book/<pkg> behavior (kernel fallthrough preserved; no breakage of habitat-workflow.book-view smokes). - SURFACE. pure-html + the ui/layout package; headings and prose from section nodes, code chapters as syntax blocks, examples as runnable blocks (eval-examples below). Elegant + phone-honest before clever: print-considerate CSS is a free paper fallback until borge course roots land. - REFERENCES. reuse spec-book-page’s binding inline+hole machinery for tokens found in prose – the drift renderer is already proven; the borg book inherits it. - MODES. default reader; workshop via ?mode=workshop or the /book UI toggle. The toggle IS the "spec vs textbook" argument made tangible.

eval-examples

read-time eval of nopales example blocks in the reader’s session; restricted prim table; (book/verify) as the book’s self-test
[status: implemented]

> – *agent note:* 2026-08-30 all three smokes verified: POST /book/eval-example returns value + elapsed and the run button is live (loop 18); define refused at the static gate, create/delete refused via known_fn capability, tree/upsert fails closed at the restricted rim; (book/verify) hole mechanics proven by scripts/smoke_book.py on a smoke_verify chapter – fail, refusal, and fix-forward all hole – and whole-book verify is green at the current 0 real example fences. Example fences in the .borg chapters themselves remain unwritten (authoring work, not machinery).
>
> *[verify] smoke: clicking a nopales block evals it and renders value + elapsed; smoke: blocks cannot define/persist: define/route/create prims are absent from the example table; smoke: (book/verify "philosophy") re-evals every block; mismatched ‘expect‘ shows as a hole row*

- PRIM. new kernel prim ‘(example-eval src)‘ (name bikeshed): parses + evals a nopales source string in a RESTRICTED environment: pure prims + read-only DB prims whitelisted; mutation families (define/route/checkout/create/update/ delete/create-user) absent. Runs in the reader’s branch env (visible defs bound, thunks forceable) so examples can touch real functions. Timeout + step cap (recursion/ complexity guardrails already exist in lisp/). - READ-TIME DEFAULT, AUTHOR-TIME VERIFY. Clicking runs in the reader’s session – habitat-first eval, the course property the design requires. ‘(book/verify [book?])‘ re-runs all blocks with ‘expect‘ trailer lines and reports holes; that is the book’s ‘dune runtest‘, and hooks into the habitat smoke suite (scripts). - OCAML BLOCKS. rendered as block + captured output from the borge export (captured at render time by the build harness there); inert buttons in v1, honest label "captured, not live".

course-documents

courses as typed outline documents over chapters: front matter, ordered chapters, lab nodes – compose, never copy

[status: implemented]

> – *agent note:* 2026-08-31 both smokes verified by scripts/ smoke_book.py section 8 on the local dev image: course doc course/cs-interpreters renders /book/course/cs-interpreters with the ordered reading list (document order asserted), chapter status badges from the seeded rows, front matter + labs sections, reader + workshop registers; a throwaway second course referencing the same chapter adds ZERO rows to data.book.ch – compose, never copy proven. Course root course/cs-interpreters.borg (repo) exports the 4-chapter paper via borge book export/print –root; the canonical walk excludes course/ roots (borge 9c1d52e). Renderer notes: reading nodes reference chapters by a ‘chapter: <label>‘ first body line; unresolvable refs render as holes; kernel count is VList-strict so the run length uses book/course-len (kernel issue filed in log/issues).
>
> *[verify] smoke: course doc ‘course/cs-interpreters‘ renders its ordered reading list with chapter status badges; smoke: same chapter renders inside two courses without duplication in the tree*

- A COURSE IS A NOTES/OUTLINE DOCUMENT (‘course/<slug>‘), typed kind, whose nodes reference chapter slugs in order, plus front-matter and lab nodes. Renderer /book/course/<slug> assembles it; the chapter itself never moves. labs link habitat objects (packages, issues, /repl snippets). - PAPER HANDOFF. a matching ‘course/cs-<n>.borg‘ root in the repo re-selects the same chapters for borge ‘–root‘ printing; the export honors it identically, so /book and the PDF stay cousins by construction, not convention.

out-of-scope-v1

what this chapter does NOT attempt in v1

[status: planned]

- Editing borg chapters from the habitat (write-path; v2 question alongside the deferred two-books-merge). - Live OCaml execution in the habitat (captured output only). - Marginia/annotations synced between paper and habitat. - Migrating spec/<pkg> package docs INTO the borg corpus (they stay their own corpus; /book indexes both). - Public access: /book stays private; public release is the existing /book.pdf face + its reader-mode flavor.

35 agent/bin/main.ml

```
(** glochid -- local-first Nopales-native agent harness (borge/agent.borg).

    v0.0 surface: stdin REPL. `agent` boots, loads ~/.config/agent/plugins/*.pp
    then ./.agent/plugins/*.pp, and enters a read/eval/print loop. Lines
    starting with `/` dispatch as plugin commands; lines starting with
    `(` are evaluated as Nopales; anything else is a prompt for the
    model.

    Flags:
      --mock-script FILE   canned provider replies (test/demo)
      --model NAME
      --base-url URL
      --once               run ONE full agent turn from piped stdin and exit
      -p EXPR              eval one Nopales expr and exit (pipe mode)

    Piped-turn mode: with either --once or non-tty stdin, the whole of
    stdin is read as the prompt (one string), a single full agent turn
    (model call -> tool calls -> final reply) runs, the final assistant
    text is printed, and we exit 0. This is the surfaces smoke #2
    trigger: `echo "say hello" | agent --mock-script m.json`.
*)

open Pricklypear_lisp

let getenv name ~default =
  match Sys.getenv_opt name with
  | Some s -> if s = "" then default else s
  | None -> default

(* Auto-load ./.env if present (KEY=VALUE or `export KEY=VALUE` lines).
   Runs before any cfg/env read so AGENT_API_KEY etc. land without the
   user sourcing the file. Quoted values are unquoted; blank lines and
   # comments are skipped. Failures are silent (file optional). *)
let load_dotenv () : unit =
  let path = ".env" in
  match open_in path with
  | exception _ -> ()
  | ic ->
    (try
       while true do
         let line = String.trim (input_line ic) in
         if line = "" || line.[0] = '#' then ()
         else begin
           let line =
             if String.length line > 7 && String.sub line 0 7 = "export "
             then String.trim (String.sub line 7 (String.length line - 7))
             else line
           in
           match String.index_opt line '=' with
           | Some i ->
               let k = String.trim (String.sub line 0 i) in
               let v_raw = String.trim (String.sub line (i + 1)
    (String.length line - i - 1)) in
               let v =
                 let n = String.length v_raw in
                 if n >= 2
                    && (v_raw.[0] = '"' && v_raw.[n - 1] = '"'
                        || v_raw.[0] = '\'' && v_raw.[n - 1] = '\'')
                 then String.sub v_raw 1 (n - 2)
                 else v_raw
               in
               Unix.putenv k v
           | None -> ()
         end
       done
     with End_of_file -> ());
    close_in ic

(* Mock provider: reads a JSON script of turns. Each entry:
   {"text": "...", "calls": [{"id","name","input"}]} -- entries are
```

```
   consumed in order; running out ends the turn loop. *)
let mock_complete ~(env : Eio_unix.Stdenv.base) ~sw ~cfg ~messages
    ~tools ~on_delta script_cell
    : (Provider.reply, string) Result.t =
  ignore env;
  ignore sw;
  ignore cfg;
  ignore tools;
  (* Mirror the real provider's streaming contract: fire the reply's
     text as an [on_delta] so the loop/print_event path is identical,
     no matter the canned or live provider. *)
  match !script_cell with
  | [] ->
      on_delta (Provider.Text "");
      on_delta Provider.Done;
      Ok { Provider.text = ""; calls = []; stop_reason = "end_turn"; raw = `Null }
  | r :: rest ->
      script_cell := rest;
      ignore messages;
      let r : Provider.reply = r in
      if r.text <> "" then on_delta (Provider.Text r.text);
      on_delta Provider.Done;
      Ok r

let parse_mock_script script_path : Provider.reply list =
  let open Yojson.Safe.Util in
  let j = Yojson.Safe.from_file script_path in
  j |> to_list
  |> List.map (fun t ->
         let text = t |> member "text" |> to_string_option |> Option.value ~default:"" in
         let calls =
           match t |> member "calls" with
           | `List cs ->
               List.map
                 (fun c ->
                   {
                     Provider.id = c |> member "id" |> to_string;
                     name = c |> member "name" |> to_string;
                     input = c |> member "input";
                   })
                 cs
           | _ -> []
         in
         { Provider.text; calls; stop_reason = "stop"; raw = `Assoc [
     ("role", `String "assistant"); ("content", `String text) ] })

(* Streaming-aware print: accumulate text deltas into the current
   assistant line so token fragments don't each get their own header.
   [tool] traces still get their own dim lines. *)
let pending_assistant = ref false

let print_event = function
  | Loop.Reasoning_text s ->
      if s <> "" then begin
        if !pending_assistant then (Printf.printf "\n%!"; pending_assistant := false);
        Printf.printf "\027[2;3m%s\027[0m%!" s
      end
  | Loop.Assistant_text s ->
      if s <> "" then begin
        if not !pending_assistant then begin
          Printf.printf "\n\027[1massistant>\027[0m %!";
          pending_assistant := true
        end;
        Printf.printf "%s%!" s
      end
  | Loop.Tool_begin (name, input) ->
      if !pending_assistant then (Printf.printf "\n%!"; pending_assistant := false);
      Printf.printf "\027[2m[tool %s %s]\027[0m\n%!" name input
  | Loop.Tool_end (name, out) ->
```

```
      let preview =
        if String.length out > 300 then String.sub out 0 300 ^ "?" else out
      in
      Printf.printf "\027[2m[tool %s -> %s]\027[0m\n%!" name preview
  | Loop.Turn_error msg ->
      Printf.printf "\027[31m[turn error] %s\027[0m\n%!" msg

(* One-shot harness identity for the system prompt. The model otherwise
   spends its first turns probing env/cgroups/os-release for facts it was
   never told (and leaks secrets like AGENT_API_KEY into the transcript
   via `env` dumps while doing so). Computed once at boot. *)
let run_captured (cmd : string) : string =
  try
    let ic = Unix.open_process_in cmd in
    let out = String.trim (In_channel.input_all ic) in
    match Unix.close_process_in ic with
    | Unix.WEXITED 0 -> out
    | _ -> ""
  with _ -> ""

let harness_context ~(model : string) ~(base_url : string) : string =
  let host = try Unix.gethostname () with _ -> "?" in
  let uname = run_captured "uname -srm" in
  let cwd = Sys.getcwd () in
  let tm = Unix.localtime (Unix.gettimeofday ()) in
  let date =
    Printf.sprintf "%04d-%02d-%02d %02d:%02d"
      (tm.Unix.tm_year + 1900) (tm.Unix.tm_mon + 1) tm.Unix.tm_mday
      tm.Unix.tm_hour tm.Unix.tm_min
  in
  Printf.sprintf
    "You are pp-agent, the local Nopales-native coding agent of the \
     Pricklypear project. The human operator talks to you through this \
     binary's browser chat surface or line REPL. You run as ONE unix \
     process directly on the operator's machine -- no container, no \
     sandbox; your commands affect the operator's real system.\n\
     \n\
     Harness facts (told to you here; do NOT probe the OS to discover
    them):\n\
     - host: %s (%s)\n\
     - cwd: %s -- the operator's working directory; treat as project root\n\
     - date: %s (operator local)\n\
     - model: %s via %s\n\
     - tools: read, bash, plus any Nopales plugin tools loaded from \
     ~/.config/agent/plugins/*.pp and ./.agent/plugins/*.pp\n\
     - /code on this binary's web surface is a read-only file browser over
    cwd\n\
     \n\
     Conduct:\n\
     - Prefer the read and bash tools. Be terse.\n\
     - For questions about the project, read AGENTS.md/docs in cwd -- do
    not \
     probe the OS for facts listed above.\n\
     - Never print secret VALUES (env vars ending in _API_KEY, _TOKEN, \
     _SECRET); reference them by name. Avoid full `env` dumps."
    host (if uname = "" then "?" else uname) cwd date model base_url

let () =
  load_dotenv ();
  let mock_script = ref None in
  let model = ref None in
  let base_url = ref None in
  let pipe_expr = ref None in
  let once = ref false in
  let serve = ref false in
  let open_browser = ref false in
  let port = ref 18090 in
  let anon (_ : string) = () in
  let speclist =
    Arg.align
      [
        ("--mock-script", Arg.String (fun s -> mock_script := Some s),
     "FILE canned provider replies");
```

```
      ("--model", Arg.String (fun s -> model := Some s), "NAME");
      ("--base-url", Arg.String (fun s -> base_url := Some s), "URL");
      ("--once", Arg.Set once, "run one full agent turn from piped stdin and exit");
      ("--serve", Arg.Set serve, "serve the HTML chat surface on 127.0.0.1 until Ctrl-C (default port 18090)");
      ("--port", Arg.Int (fun p -> port := p), "PORT fixed port for --serve (default 18090; 0 = random)");
      ("--open", Arg.Set open_browser, "with --serve: best-effort browser launch");
      ("-p", Arg.String (fun s -> pipe_expr := Some s), "EXPR eval and exit");
    ]
in
Arg.parse speclist anon "agent [--mock-script F] [--provider N] [--once] [--serve [--port P] [--open]] [-p EXPR]";
(* Pipe mode is entered either explicitly (--once) or implicitly when
   stdin is not a tty (surfaces smoke #2: `echo ... | agent ...`). *)
let pipe_mode = !once || not (Unix.isatty Unix.stdin) in
(* TLS (tls-eio) needs a seeded RNG before any HTTPS provider call;
   mock-only runs never hit TLS, so the smokes didn't surface this.
   Mirage_crypto_rng_unix.use_default wires /dev/urandom + reseeding. *)
Mirage_crypto_rng_unix.use_default ();
(* Random.bits drives /chat stream ids + multipart boundaries; without
   self_init every process run repeats the same id/boundary sequence. *)
Random.self_init ();
Eio_main.run @@ fun env ->
Eio.Switch.run @@ fun sw ->
let rt = Runtime.make () in
Tools.install ~resolve_path:Runtime.resolve_path
  ~register:(fun name run -> Runtime.register_builtin_tool rt name ~run);
(* Boot-load the Nopales surface (libs/html + libs/hyper + the chat/code
   route-table handlers) so the HTML surface is Nopales-authored and
   redefinable live (borge/agent.borg surfaces). *)
Surface.load ~rt;
Plugins.load_all rt;
match !pipe_expr with
| Some src ->
    (try
       let v = Runtime.eval_string ~rt src in
       print_endline (Reader.pp_value v)
     with Ast.Interp_error ie ->
       Printf.eprintf "error: %s\n%!" (Ast.pp_interp_error ie);
       exit 1)
| None ->
    let cfg : Provider.cfg =
      {
        base_url =
          Option.value !base_url
            ~default:(getenv "AGENT_BASE_URL"
   ~default:"https://api.synthetic.new/openai/v1");
        api_key = getenv "AGENT_API_KEY" ~default:"";
        model = Option.value !model ~default:(getenv "AGENT_MODEL"
   ~default:"hf:zai-org/GLM-4.7-Flash");
        system =
          getenv "AGENT_SYSTEM"
            ~default:
              (harness_context
                 ~model:
                   (Option.value !model
                      ~default:
                        (getenv "AGENT_MODEL"
                           ~default:"hf:zai-org/GLM-4.7-Flash"))
                 ~base_url:
                   (Option.value !base_url
                      ~default:
                        (getenv "AGENT_BASE_URL"

   ~default:"https://api.synthetic.new/openai/v1")));
      }
    in
    let script_cell = ref (Option.map parse_mock_script !mock_script |>
   Option.value ~default:[]) in
```

```
270       let complete =
271         match !mock_script with
272         | Some _ -> (fun ~env ~sw ~cfg ~messages ~tools ~on_delta ->
273             mock_complete ~env ~sw ~cfg ~messages ~tools ~on_delta
    script_cell)
274         | None -> Provider.Openai.complete
275       in
276       let history : Provider.message list ref = ref [] in
277       (* Run one full agent turn (model call -> tool calls -> final
278          reply) for the given prompt, streaming events via print_event.
279          Returns Ok () / Error msg. Exit status is derived by callers. *)
280       let run_turn prompt =
281         Loop.run ~env ~sw ~host:(Runtime.loop_host rt) ~cfg ~complete
282           ~on_event:print_event history prompt
283       in
284       if !serve then
285         (* HTML primary surface: serve until Ctrl-C. Blocks forever in
286            the accept loop. wrap_serve passes the sink-only [print]
287            used for URL output. --open is best-effort browser launch. *)
288         Http_ui.serve ~env ~sw ~rt ~cfg ~complete ~port:!port
289           ~open_browser:!open_browser print_endline
290       else if pipe_mode then (
291         (* Piped-turn smoke: read ALL of stdin as the prompt, run ONE
292            full turn, print the final assistant text, exit 0 on Ok.
293            Multi-line input is joined with newlines (trimmed). *)
294         let buf = Buffer.create 256 in
295         (try
296            while true do Buffer.add_string buf (input_line stdin);
    Buffer.add_char buf '\n' done
297          with End_of_file -> ());
298         let prompt = String.trim (Buffer.contents buf) in
299         match run_turn prompt with
300         | Ok () -> () (* final assistant text already printed via
    print_event *)
301         | Error e ->
302             Printf.eprintf "agent turn error: %s\n%!" e;
303             exit 1)
304       else (
305       print_endline "agent v0.0 -- Nopales plugins. (expr) evaluates; /cmd
    dispatches; other lines prompt the model.";
306       let rec loop () =
307         print_string "\027[1magent>\027[0m ";
308         flush stdout;
309         match input_line stdin with
310         | exception End_of_file -> print_endline "bye"
311         | line ->
312             let line = String.trim line in
313             if line = "" then loop ()
314             else if String.length line > 0 && line.[0] = '/' then (
315               let sp = try String.index line ' ' with Not_found ->
    String.length line in
316               let name = String.sub line 1 (sp - 1) in
317               let rest =
318                 if sp < String.length line then
319                   String.trim (String.sub line sp (String.length line - sp))
320                 else ""
321               in
322               (match Hashtbl.find_opt rt.Runtime.commands name with
323                | Some entry ->
324                    (try
325                       let v = Runtime.apply_value rt entry.run [ Ast.VStr
    rest ] in
326                       if v <> Ast.VNil then print_endline (Reader.pp_value
    v)
327                     with Ast.Interp_error ie ->
328                       Printf.eprintf "cmd error: %s\n%!"
    (Ast.pp_interp_error ie))
329                | None ->
330                    Printf.printf "commands: %s\n%!"
331                      (match Runtime.eval_string ~rt "(list-commands)" with
332                       | Ast.VStr s -> s
333                       | _ -> "?"));
334               loop ())
```

```
335             else if line.[0] = '(' || String.length line > 0 && line.[0] =
    '\'' then (
336               (try
337                  let v = Runtime.eval_string ~rt line in
338                  print_endline (Reader.pp_value v)
339                with
340                | Ast.Interp_error ie ->
341                    Printf.printf "error: %s\n%!" (Ast.pp_interp_error ie)
342                | Reader.Reader_error (msg, ln, col) ->
343                    Printf.printf "read error %d:%d %s\n%!" ln col msg);
344               loop ())
345             else (
346               (match
347                  Loop.run ~env ~sw ~host:(Runtime.loop_host rt) ~cfg
    ~complete
348                  ~on_event:print_event history line
349                with
350                | Ok () -> ()
351                | Error e -> Printf.printf "\027[31m[loop error]
    %s\027[0m\n%!" e);
352               loop ())
353        in
354        loop () )
```

36 agent/lib/http_ui.ml

```
(** glochid HTML primary surface (agent/lib; borge/agent.borg surfaces).

    A local HTTP server (Httpun_eio) bound to 127.0.0.1 (default port
    18090, --port to override; a busy port falls back to random). The
    chat + code browser pages are NOPALES-AUTHORED htmx
    fragments on the agent route table (rendered by Nopales via
    libs/html), dispatched in-process through [Runtime.dispatch_route].
    A chat/agent turn streams as multipart/mixed htmx fragments (one
    part per (html, HX-Target, HX-Swap)), emitted by OCaml; the fragment
    templates are Nopales (chat/user-bubble, chat/turn-open,
    chat/think-open, chat/think-token, chat/tool-open, chat/tool-line,
    chat/tool-done, chat/token, chat/error), so the surface is
    redefinable live. Each emitted part is
    also collected as data so a consumed turn replays through
    [hyper/stream] (surfaces smoke).

    Bootstrap/asset plumbing stays OCaml (per the surfaces scope): the
    vendored htmx + hx-multipart assets are baked into the binary
    ([Assets]) and served at /assets/*.js. *)

open Pricklypear_lisp

(* ---- shared history / per-request serialization ---- *)

type server_state = {
  history : Provider.message list ref;
  complete :
    env:Eio_unix.Stdenv.base ->
    sw:Eio.Switch.t ->
    cfg:Provider.cfg ->
    messages:Provider.message list ->
    tools:Provider.tool_def list ->
    on_delta:(Provider.delta -> unit) ->
    (Provider.reply, string) Result.t;
  rt : Runtime.t;
  cfg : Provider.cfg;
  turn_mutex : Eio.Mutex.t;
}

(* ---- HTML escaping (status/error pages only; surfaces are Nopales) ----
     *)

let esc (s : string) : string =
  let buf = Buffer.create (String.length s + 16) in
  String.iter
    (fun c ->
      match c with
      | '<' -> Buffer.add_string buf "&lt;"
      | '>' -> Buffer.add_string buf "&gt;"
      | '&' -> Buffer.add_string buf "&amp;"
      | '"' -> Buffer.add_string buf "&quot;"
      | c -> Buffer.add_char buf c)
    s;
  Buffer.contents buf

let status_page (title : string) (body : string) : string =
  Printf.sprintf
    {|<!doctype html><html><head><meta charset="utf-8"><title>%s</title>
<style>body{background:#0f1117;color:#d8dee9;font-family:ui-monospace,monospace;padding:1.5rem}a{color:#7aa2f7}</style>
</head><body>%s</body></html>|}
    (esc title) body

(* ---- HTTP plumbing ---- *)

let read_body (reqd : Httpun.Reqd.t) : string =
  let body = Httpun.Reqd.request_body reqd in
  let promise, resolver = Eio.Promise.create () in
  let buf = Buffer.create 1024 in
  let rec sched () =
    Httpun.Body.Reader.schedule_read body
      ~on_eof:(fun () -> Eio.Promise.resolve resolver (Buffer.contents buf))
```

```
      ~on_read:(fun bs ~off ~len ->
        Buffer.add_string buf (Bigstringaf.substring bs ~off ~len);
        sched ())
  in
  sched ();
  Eio.Promise.await promise

let respond_text (reqd : Httpun.Reqd.t) ~(status : Httpun.Status.t)
    ~(content_type : string) ~(body : string) : unit =
  let headers =
    Httpun.Headers.of_list
      [
        ("content-type", content_type);
        ("content-length", string_of_int (String.length body));
      ]
  in
  let _body_w =
    Httpun.Reqd.respond_with_string reqd
      (Httpun.Response.create ~headers status)
      body
  in
  ()

let respond_html (reqd : Httpun.Reqd.t) ~(status : Httpun.Status.t)
    ~(body : string) : unit =
  respond_text reqd ~status ~content_type:"text/html; charset=utf-8" ~body

let is_closed_writer_exn = function
  | Failure msg ->
      String.equal msg "cannot write to closed writer"
      || String.starts_with ~prefix:"cannot write to closed writer" msg
  | _ -> false

let close_writer_quiet (body : Httpun.Body.Writer.t) : unit =
  (try Httpun.Body.Writer.close body
   with exn when is_closed_writer_exn exn -> ())

(* Bail out of the multipart drain loop when the client disconnects. *)
exception Peer_gone

(* ---- multipart/mixed streaming (ported from image http_agent.ml) ---- *)

(* One multipart part frame + flush. Mirrors the image's mp_write_part:
   --boundary + Content-Type: text/html + optional HX-Target/HX-Swap +
   blank + html. Returns `Peer_gone on closed writer so the drain loop
   stops cleanly.

   WRITE WATCHDOG: a client that disconnects mid-stream leaves the
   socket in CLOSE-WAIT -- writes keep SUCCEEDING into the kernel send
   buffer until it fills, then the write/flush blocks FOREVER, wedging
   the turn (and with it the turn_mutex, so every later /chat parks).
   Race the whole write path against a 5s sleep: a healthy part write
   completes in milliseconds; a timeout means the connection is gone
   ? Peer_gone. (Same fix in remote/lib/http_ui.ml.) *)
let mp_write_part ~(env : Eio_unix.Stdenv.base) (body :
     Httpun.Body.Writer.t)
    ~boundary ?(target : string option) ?(swap : string option)
    (html : string) : [ `Ok | `Peer_gone ] =
  let buf = Buffer.create 128 in
  Buffer.add_string buf ("--" ^ boundary ^ "\r\n");
  Buffer.add_string buf "Content-Type: text/html; charset=utf-8\r\n";
  (match target with Some t -> Buffer.add_string buf ("HX-Target: " ^ t ^
     "\r\n") | None -> ());
  (match swap with Some s -> Buffer.add_string buf ("HX-Swap: " ^ s ^
     "\r\n") | None -> ());
  Buffer.add_string buf "\r\n";
  Buffer.add_string buf html;
  Buffer.add_string buf "\r\n";
  let chunk = Buffer.contents buf in
  let write_and_flush () =
    Httpun.Body.Writer.write_string body chunk;
    let promise, resolver = Eio.Promise.create () in
    let wake () =
```

```
      if not (Eio.Promise.is_resolved promise) then Eio.Promise.resolve
     resolver ()
    in
    (try Httpun.Body.Writer.flush body (fun () -> wake ())
     with exn when is_closed_writer_exn exn -> wake ());
    Eio.Promise.await promise
  in
  try
    (* Race the WHOLE write path (write_string + flush + await): a dead
       peer in CLOSE-WAIT accepts writes until the send buffer fills,
       then the write blocks FOREVER -- a healthy part write completes in
       milliseconds, so a 5s timeout means the connection is gone. *)
    let timed_out =
      Eio.Fiber.any
        [ (fun () -> write_and_flush (); false)
        ; (fun () -> Eio.Time.sleep (Eio.Stdenv.clock env) 5.0; true) ]
    in
    if timed_out then `Peer_gone else `Ok
  with exn when is_closed_writer_exn exn ->
    `Peer_gone

let mp_close (body : Httpun.Body.Writer.t) ~boundary : unit =
  let chunk = "--" ^ boundary ^ "--\r\n" in
  (try Httpun.Body.Writer.write_string body chunk
   with exn when is_closed_writer_exn exn -> ());
  close_writer_quiet body

(* ---- query-string parsing (route params for GET dispatch) ---- *)

let url_decode (s : string) : string =
  let buf = Buffer.create (String.length s) in
  let len = String.length s in
  let rec go i =
    if i >= len then Buffer.contents buf
    else if i + 2 < len && s.[i] = '%' then (
      (try
         Buffer.add_char buf
           (Char.chr (int_of_string ("0x" ^ String.sub s (i + 1) 2)))
       with _ -> ());
      go (i + 3))
    else (
      Buffer.add_char buf (if s.[i] = '+' then ' ' else s.[i]);
      go (i + 1))
  in
  go 0

let parse_query (q : string) : (string * string) list =
  if q = "" then []
  else
    List.filter_map
      (fun kv ->
        match String.index_opt kv '=' with
        | Some i ->
            let k = String.sub kv 0 i in
            let v = String.sub kv (i + 1) (String.length kv - i - 1) in
            Some (url_decode k, url_decode v)
        | None -> Some (url_decode kv, ""))
      (String.split_on_char '&' q)

(* ---- GET route dispatch through the agent route table ---- *)

let dispatch_get (st : server_state) (reqd : Httpun.Reqd.t) (target :
    string)
    : unit =
  let path, query =
    match String.index_opt target '?' with
    | Some i ->
        ( String.sub target 0 i,
          String.sub target (i + 1) (String.length target - i - 1) )
    | None -> (target, "")
  in
  let params = parse_query query in
  match Runtime.lookup_route ~rt:st.rt path with
```

```
  | None ->
      respond_html reqd ~status:`Not_found
        ~body:(status_page "404" (Printf.sprintf "<p>no route %s</p>" (esc path)))
  | Some handler ->
      let v =
        Runtime.dispatch_route ~rt:st.rt ~handler ~method_:"GET" ~path
          ~params ~headers:[] ~body:""
      in
      let html =
        match v with Ast.VStr s -> s | other -> Pricklypear_lisp.Reader.pp_value other
      in
      respond_html reqd ~status:`OK ~body:html

(* ---- static assets (vendored, baked into the binary) ---- *)

let serve_asset (reqd : Httpun.Reqd.t) (name : string) : unit =
  let body, ct =
    match name with
    | "htmx.min.js" -> (Assets.htmx_min, "application/javascript; charset=utf-8")
    | "htmx-2-compat.min.js" -> (Assets.htmx_2_compat, "application/javascript; charset=utf-8")
    | "hx-multipart.js" -> (Assets.hx_multipart, "application/javascript; charset=utf-8")
    | _ -> ("", "application/octet-stream")
  in
  if body = "" then
    respond_html reqd ~status:`Not_found
      ~body:(status_page "404" (Printf.sprintf "<p>no asset %s</p>" (esc name)))
  else respond_text reqd ~status:`OK ~content_type:ct ~body

(* ---- chat turn: multipart/mixed stream + part collection ---- *)

(* A captured part for hyper/stream replay, in the alist shape
   soup/apply-parts! consumes: (("html" . frag) ("target" . sel) ("swap" . strat)). *)
let part_alist ~html ~target ~swap : Ast.value =
  Ast.VList
    [ Ast.VPair (Ast.VStr "html", Ast.VStr html)
    ; Ast.VPair (Ast.VStr "target", Ast.VStr target)
    ; Ast.VPair (Ast.VStr "swap", Ast.VStr swap)
    ]

(* url-decode a form field value (htmx posts application/x-www-form-urlencoded). *)
let form_field (body : string) (name : string) : string =
  let prefix = name ^ "=" in
  let rec find i =
    if i >= String.length body then None
    else if i = 0 || body.[i - 1] = '&' then
      if String.length body - i >= String.length prefix
         && String.sub body i (String.length prefix) = prefix
      then
        let rest_start = i + String.length prefix in
        let rest_end =
          match String.index_from_opt body rest_start '&' with
          | Some j -> j | None -> String.length body
        in
        Some (String.sub body rest_start (rest_end - rest_start))
      else find (i + 1)
    else find (i + 1)
  in
  match find 0 with None -> "" | Some v -> url_decode v

let handle_chat (env : Eio_unix.Stdenv.base) (sw : Eio.Switch.t)
    (st : server_state) (reqd : Httpun.Reqd.t) : unit =
  let body_raw = read_body reqd in
  let prompt = form_field body_raw "msg" in
  (* REPL rule (same as the stdin REPL): a prompt whose first non-space
     char is ( evaluates as Nopales in the shared env -- no model call. *)
```

```
277   let trimmed = String.trim prompt in
278   let repl_expr = String.length trimmed > 0 && trimmed.[0] = '(' in
279   let boundary = Printf.sprintf "pp%x%x" (Random.bits ()) (Random.bits ())
        in
280   let rid = Printf.sprintf "llm-%d" (Random.bits () land 0xFFFFFF) in
281   let headers =
282     Httpun.Headers.of_list
283       [
284         ("content-type", "multipart/mixed; boundary=" ^ boundary);
285         ("cache-control", "no-cache");
286         ("x-accel-buffering", "no");
287         (* Chunked framing is MANDATORY for a streamed response: without an
288            explicit framing header httpun emits the body close-delimited,
289            and on a keep-alive HTTP/1.1 connection the client can never
290            detect end-of-body (fetch().text() never resolves). *)
291         ("transfer-encoding", "chunked");
292       ]
293   in
294   let body_w =
295     Httpun.Reqd.respond_with_streaming ~flush_headers_immediately:true
296       reqd (Httpun.Response.create ~headers `OK)
297   in
298   (* Captured parts for hyper/stream replay (data). *)
299   let parts = ref [] in
300   let emit ?target ?swap html =
301     let target = match target with Some t -> t | None -> "#log" in
302     let swap = match swap with Some s -> s | None -> "beforeend" in
303     parts := part_alist ~html ~target ~swap :: !parts;
304     match mp_write_part ~env body_w ~boundary ~target ~swap html with
305     | `Ok -> ()
306     | `Peer_gone -> raise Peer_gone
307   in
308   let run () =
309     (* Openers: user echo bubble + empty llm bubble (rendered by Nopales
310        templates so they are redefinable live). *)
311     (match Runtime.call_template ~rt:st.rt "chat/user-bubble" [ Ast.VStr
        prompt ]
312      with Some html -> emit html | None -> emit (Printf.sprintf "<div
        class=\"bubble user\">%s</div>" (esc prompt)));
313     if repl_expr then begin
314       (* REPL mode: evaluate locally (eval lock + rim, same env as
315          plugins/stdin-REPL/surfaces), render the last value REPL-style.
316          Errors land in the chat/error template, not a model turn. *)
317       let result =
318         try `Ok (Runtime.eval_string ~rt:st.rt trimmed)
319         with
320         (* Render interp errors like the stdin REPL does -- the OCaml
321            default printer shows the opaque Interp_error(_). *)
322         | Ast.Interp_error ie -> `Err (Ast.pp_interp_error ie)
323         | e -> `Err (Printexc.to_string e)
324       in
325       (match result with
326        | `Ok v ->
327            let s = Ast.pp_value_short v in
328            (match Runtime.call_template ~rt:st.rt "chat/repl-bubble" [
        Ast.VStr s ]
329             with Some html -> emit html
330              | None ->
331                  emit (Printf.sprintf "<div class=\"bubble repl\">%s</div>"
        (esc s)))
332        | `Err msg ->
333            (match Runtime.call_template ~rt:st.rt "chat/error" [ Ast.VStr
        msg ]
334             with Some html -> emit html
335              | None ->
336                  emit (Printf.sprintf "<div class=\"trace\">[repl error:
        %s]</div>" (esc msg))))
337     end
338     else begin
339     (* Turn skeleton: one container with TWO swap targets -- #<rid>-think
340        inside the collapsible <details> (reasoning spans + tool traces)
341        and #<rid> response bubble (content tokens). *)
342     (match Runtime.call_template ~rt:st.rt "chat/turn-open" [ Ast.VStr rid ]
```

```
343       with Some html -> emit html
344        | None ->
345            emit
346              (Printf.sprintf
347                 "<div class=\"turn\"><details
          class=\"think\"><summary>thinking + tools</summary><div
          id=\"%s-think\"></div></details><div class=\"bubble llm\"
          id=\"%s\"></div></div>"
348                 rid rid));
349       (* Producer fiber: run the agent turn, pushing events to a queue. *)
350       let queue : Loop.turn_event Queue.t = Queue.create () in
351       let qmutex = Eio.Mutex.create () in
352       let qcond = Eio.Condition.create () in
353       let finished = ref false in
354       let push (e : Loop.turn_event) : unit =
355         Eio.Mutex.lock qmutex;
356         Queue.add e queue;
357         Eio.Condition.broadcast qcond;
358         Eio.Mutex.unlock qmutex
359       in
360       let on_event = function
361         | Loop.Reasoning_text s -> push (Loop.Reasoning_text s)
362         | Loop.Assistant_text s -> push (Loop.Assistant_text s)
363         | Loop.Tool_begin (n, i) -> push (Loop.Tool_begin (n, i))
364         | Loop.Tool_end (n, o) -> push (Loop.Tool_end (n, o))
365         | Loop.Turn_error msg -> push (Loop.Turn_error msg)
366       in
367       (* Nested collapsibles INSIDE the turn's "thinking + tools" zone
368          (#<rid>-think): each contiguous reasoning run gets its own
369          <details> (chat/think-open) and each tool call its own
370          (chat/tool-open, summary marked done by chat/tool-done).
371          cur_think / open_tools track which inner div deltas and lines
372          stream into. *)
373       let think_zone = "#" ^ rid ^ "-think" in
374       let think_n = ref 0 in
375       let cur_think : string option ref = ref None in
376       let tool_n = ref 0 in
377       let open_tools : string list ref = ref [] in
378       let open_think_block () : string =
379         incr think_n;
380         let tid = Printf.sprintf "%s-tk-%d" rid !think_n in
381         (match Runtime.call_template ~rt:st.rt "chat/think-open" [ Ast.VStr
          tid ] with
382          | Some html -> emit ~target:think_zone html
383          | None ->
384              emit ~target:think_zone
385                (Printf.sprintf
386                   "<details class=\"tk\"><summary>thinking</summary><div
          id=\"%s\"></div></details>"
387                   tid));
388         cur_think := Some tid;
389         tid
390       in
391       let emit_tool_line tid label text =
392         match Runtime.call_template ~rt:st.rt "chat/tool-line"
393                 [ Ast.VStr label; Ast.VStr text ] with
394         | Some html -> emit ~target:("#" ^ tid) html
395         | None ->
396             emit ~target:("#" ^ tid)
397               (Printf.sprintf "<div class=\"trace\">%s %s</div>"
398                  (esc label) (esc text))
399       in
400       Eio.Switch.run @@ fun lsw ->
401       Eio.Fiber.fork ~sw:lsw (fun () ->
402           (match
403              Loop.run ~env ~sw:lsw ~host:(Runtime.loop_host st.rt) ~cfg:st.cfg
404              ~complete:st.complete
405                ~on_event st.history prompt
406            with
407            | Ok () -> ()
408            | Error e ->
409                (* A failed turn must be VISIBLE in the stream: closing the
410                   response without an error part reads as a dead UI. *)
```

```
            push (Loop.Turn_error e));
        Eio.Mutex.lock qmutex;
        finished := true;
        Eio.Condition.broadcast qcond;
        Eio.Mutex.unlock qmutex);
    let rec drain () =
      Eio.Mutex.lock qmutex;
      while Queue.is_empty queue && not !finished do
        Eio.Condition.await qcond qmutex
      done;
      let items = List.of_seq (Queue.to_seq queue) in
      Queue.clear queue;
      let fin = !finished && Queue.is_empty queue in
      Eio.Mutex.unlock qmutex;
      List.iter
        (fun e ->
          (match e with
           | Loop.Reasoning_text s ->
               let tid =
                 match !cur_think with
                 | Some t -> t
                 | None -> open_think_block ()
               in
               (match Runtime.call_template ~rt:st.rt "chat/think-token" [
    Ast.VStr s ]
                with Some html -> emit ~target:("#" ^ tid) html
                 | None ->
                     emit ~target:("#" ^ tid)
                       (Printf.sprintf "<span class=\"reason\">%s</span>"
    (esc s)))
           | Loop.Assistant_text s ->
               cur_think := None;
               (match Runtime.call_template ~rt:st.rt "chat/token" [
    Ast.VStr s ]
                with Some html -> emit ~target:("#" ^ rid) html
                 | None -> emit ~target:("#" ^ rid) (Printf.sprintf
    "<span>%s</span>" (esc s)))
           | Loop.Tool_begin (n, i) ->
               cur_think := None;
               incr tool_n;
               let tid = Printf.sprintf "%s-tool-%d" rid !tool_n in
               open_tools := tid :: !open_tools;
               (match Runtime.call_template ~rt:st.rt "chat/tool-open"
                        [ Ast.VStr tid; Ast.VStr n ]
                with
                | Some html -> emit ~target:think_zone html
                | None ->
                    emit ~target:think_zone
                      (Printf.sprintf
                         "<details class=\"tool\"><summary
    id=\"%s-sum\">tool: %s</summary><div id=\"%s\"></div></details>"
                         tid (esc n) tid));
               emit_tool_line tid "in" i
           | Loop.Tool_end (n, o) ->
               let tid =
                 match !open_tools with
                 | t :: rest -> open_tools := rest; t
                 | [] -> rid ^ "-think"
               in
               emit_tool_line tid "out" o;
               if tid <> rid ^ "-think" then
                 (match Runtime.call_template ~rt:st.rt "chat/tool-done" [
    Ast.VStr n ]
                  with
                  | Some html ->
                      emit ~target:("#" ^ tid ^ "-sum") ~swap:"innerHTML"
    html
                  | None ->
                      emit ~target:("#" ^ tid ^ "-sum") ~swap:"innerHTML"
                        (Printf.sprintf "tool: %s &#10003;" (esc n)))
           | Loop.Turn_error msg ->
               (match Runtime.call_template ~rt:st.rt "chat/error"
                  [ Ast.VStr msg ]
```

```
                with
                | Some html -> emit ~target:("#" ^ rid) html
                | None ->
                    emit ~target:("#" ^ rid)
                      (Printf.sprintf
                         "<div class=\"trace\">[turn error: %s]</div>"
                         (esc msg)))))
        items;
      if not fin then drain ()
    in
    drain ()
    end
  in
  Eio.Mutex.lock st.turn_mutex;
  Fun.protect
    ~finally:(fun () ->
      Eio.Mutex.unlock st.turn_mutex;
      (* Record the captured parts for hyper/stream replay. *)
      st.rt.Runtime.last_turn_parts <- Ast.VList (List.rev !parts);
      (try mp_close body_w ~boundary with Peer_gone -> ()))
    (fun () -> (try run () with Peer_gone -> ()))

(* ---- serve: bind 127.0.0.1 (default 18090), print URL, accept loop ---- *)

let serve ~(env : Eio_unix.Stdenv.base) ~(sw : Eio.Switch.t)
    ~(rt : Runtime.t) ~(cfg : Provider.cfg)
    ~(complete :
        env:Eio_unix.Stdenv.base ->
        sw:Eio.Switch.t ->
        cfg:Provider.cfg ->
        messages:Provider.message list ->
        tools:Provider.tool_def list ->
        on_delta:(Provider.delta -> unit) ->
        (Provider.reply, string) Result.t)
    ?(port = 18090) ?(open_browser = false) (print : string -> unit) : unit =
  let st =
    {
      history = ref [];
      complete;
      rt;
      cfg;
      turn_mutex = Eio.Mutex.create ();
    }
  in
  let request_handler (sa : Eio.Net.Sockaddr.stream)
      (g : Httpun.Reqd.t Gluten.reqd) : unit =
    ignore sa;
    let reqd = g.Gluten.reqd in
    let req = Httpun.Reqd.request reqd in
    let target = req.Httpun.Request.target in
    let path, _query =
      match String.index_opt target '?' with
      | Some i ->
          ( String.sub target 0 i,
            Some (String.sub target (i + 1) (String.length target - i - 1))
    )
      | None -> (target, None)
    in
    let fork_work (f : unit -> unit) : unit =
      Eio.Fiber.fork ~sw (fun () ->
          (try f () with
           | Eio.Cancel.Cancelled _ as e -> raise e
           | exn -> Printf.eprintf "[agent-ui] handler error: %s\n%!"
     (Printexc.to_string exn)))
    in
    match req.Httpun.Request.meth with
    | `GET -> (
        if String.length path >= 8 && String.sub path 0 8 = "/assets/" then
          fork_work (fun () ->
              serve_asset reqd (String.sub path 8 (String.length path - 8)))
        else
```

```
          fork_work (fun () -> dispatch_get st reqd target))
    | `POST -> (
        match path with
        | "/chat" -> fork_work (fun () -> handle_chat env sw st reqd)
        | _ ->
            respond_html reqd ~status:`Not_found
              ~body:(status_page "404" (Printf.sprintf "<p>no route %s</p>"
   (esc path))))
    | _ -> respond_html reqd ~status:`Not_found ~body:(status_page "404"
   "<p>method not supported</p>")
  in
  let error_handler (_sa : Eio.Net.Sockaddr.stream)
      ?(request : Httpun.Request.t option)
      (err : Httpun.Server_connection.error)
      (respond : Httpun.Headers.t -> Httpun.Body.Writer.t) : unit =
    ignore request;
    ignore err;
    let body = "error" in
    let body_w =
      respond
        (Httpun.Headers.of_list
           [ ("content-type", "text/plain"); ("content-length",
   string_of_int (String.length body)) ])
    in
    Httpun.Body.Writer.write_string body_w body;
    Httpun.Body.Writer.close body_w
  in
  let conn_handler =
    Httpun_eio.Server.create_connection_handler ~request_handler
      ~error_handler ~sw
  in
  let net = Eio.Stdenv.net env in
  (* Fixed default port so the URL is stable across runs; a busy port
     falls back to a random one (the old --serve behavior). *)
  let bind p =
    Eio.Net.listen ~reuse_addr:true ~reuse_port:false ~backlog:64 ~sw net
      (`Tcp (Eio.Net.Ipaddr.V4.loopback, p))
  in
  let socket =
    try bind port
    with _ ->
      print (Printf.sprintf "port %d busy; using a random port\n" port);
      bind 0
  in
  let bound_addr = Eio.Net.listening_addr socket in
  let port =
    match bound_addr with `Tcp (_ip, p) -> p | _ -> 0
  in
  let url = Printf.sprintf "http://127.0.0.1:%d/" port in
  print (Printf.sprintf "agent serving %s\n" url);
  if open_browser then (
    try
      let _ = Sys.command (Printf.sprintf "xdg-open '%s' >/dev/null 2>&1 &"
   url) in
      ()
    with _ -> ());
  let on_error ex = ignore ex in
  while true do
    Eio.Net.accept_fork socket ~sw ~on_error (fun client_sock client_addr ->
        try conn_handler client_addr client_sock
        with exn -> ignore exn)
  done
```

37 agent/lib/plugins.ml

```
(** Plugin loading (borge/agent.borg plugin-model).

    Global plugins from ~/.config/agent/plugins/*.pp, then project
    plugins from ./.agent/plugins/*.pp (filename order). Each file's
    forms are evaluated in order into the one shared runtime env;
    errors are contained and reported, loading continues. *)

open Pricklypear_lisp

let pp_files_of dir =
  if not (Sys.file_exists dir && Sys.is_directory dir) then []
  else
    Sys.readdir dir |> Array.to_list
    |> List.filter (fun f -> Filename.check_suffix f ".pp")
    |> List.sort String.compare
    |> List.map (Filename.concat dir)

let global_dir () =
  match Sys.getenv_opt "HOME" with
  | Some h -> Filename.concat h ".config/agent/plugins"
  | None -> ".config/agent/plugins"

let project_dir () = ".agent/plugins"

let slurp path =
  let ic = open_in_bin path in
  let n = in_channel_length ic in
  let s = really_input_string ic n in
  close_in ic;
  s

(* Evaluate the toplevel forms of [src] one by one in [rt]'s env. *)
let eval_src (rt : Runtime.t) (src : string) : unit =
  let forms = Reader.read_all src in
  List.iter
    (fun form ->
      let rendered = Reader.pp_expr form in
      (* go through Runtime.eval_string so the rim + mutex apply *)
      ignore (Runtime.eval_string ~rt rendered))
    forms

let load_file (rt : Runtime.t) (path : string) : (unit, string) Result.t =
  try
    let src = slurp path in
    eval_src rt src;
    Ok ()
  with
  | Sys_error e -> Error e
  | Reader.Reader_error (msg, ln, col) ->
      Error (Printf.sprintf "read error %d:%d: %s" ln col msg)
  | Ast.Interp_error ie -> Error (Ast.pp_interp_error ie)

(** Load all plugins; print a one-line report per file to stderr. *)
let load_all (rt : Runtime.t) : unit =
  let dirs = [ global_dir (); project_dir () ] in
  List.iter
    (fun dir ->
      List.iter
        (fun path ->
          match load_file rt path with
          | Ok () -> Printf.eprintf "[plugins] %s ?\n%!" path
          | Error e -> Printf.eprintf "[plugins] %s ? %s\n%!" path e)
        (pp_files_of dir))
    dirs
```

38 agent/lib/runtime.ml

```
(** Agent runtime: the shared Nopales env + rim host.

    One runtime per process. All evaluation is serialized through
    [eval_mutex] (per borge/agent.borg nopales-embed); host prims are
    synchronous and block the calling fiber under Eio. *)

open Pricklypear_lisp
open Ast

(** Tool/command registry entry -- alias of the shared loop's record
    (llm/lib/loop.ml owns the definition now that agent + pp-remote
    share the loop). *)
type registry_entry = Loop.registry_entry

(** Thin request context bound during route/dispatch (agent-route-table,
    hypermedia-harness.request-runner AGENT HOST BACKING). Carries ONLY
    request-shape values: method, path, params, headers, body. DB/branch/
    auth request prims (current-author-as-DB-row etc.) are intentionally
    UNBOUND in the agent -- the agent's request context provides only the
    request-shape values. *)
type request_shape = {
  method_ : string;
  path : string;
  params : (string * string) list;
  headers : (string * string) list;  (* lowercased keys *)
  body : string;
}

type t = {
  env : Ast.env;
  prims : Eval.prim_table;
  eval_mutex : Eio.Mutex.t;
  handlers : (string, (Ast.value list -> Ast.value) list ref) Hashtbl.t;
  tools : (string, registry_entry) Hashtbl.t;
  mutable tool_order : string list;
  commands : (string, registry_entry) Hashtbl.t;
  (* Agent route table: path -> handler-name (THE HOST SEAM,
     agent-route-table). Local-lens backing for the image's Postgres
     routes -- identical Nopales signatures so libs/hyper runs unchanged. *)
  routes : (string, string) Hashtbl.t;
  (* Per-call request snapshot bound while a route/dispatch handler runs,
     so query-param / request-method / hx-request? resolve. None outside
     a dispatch. *)
  current_request : request_shape option ref;
  (* Captured multipart parts of the last consumed chat/agent turn, as a
     VList of part alists (("html" . frag) ("target" . sel) ("swap" .
     strat)) -- the data shape [soup/apply-parts!] consumes. Set by the
     HTTP surface's multipart stream emitter so a consumed turn replays
     deterministically through [hyper/stream] (surfaces smoke). *)
  mutable last_turn_parts : Ast.value;
}

(* Current working directory as seen by path prims. Immutable per
   process: agent never chdir-s. *)
let cwd = ref (Sys.getcwd ())

let value_to_json (v : Ast.value) : Yojson.Safe.t =
  let rec go = function
    | Ast.VStr s -> `String s
    | VInt i -> `Intlit (Int64.to_string i)
    | VFloat f -> `Float f
    | VBool b -> `Bool b
    | VNil -> `Null
    | VSymbol s -> `String s
    | VList vs -> `List (List.map go vs)
    | VLambda _ -> `String "<lambda>"
    | VPrim n -> `String ("<prim " ^ n ^ ">")
    | other -> `String (Reader.pp_value other)
  in
  go v

```

```
(** Parse a plugin-produced value back to host JSON. VStr is treated
    as raw JSON first (dict prims produce JSON strings), else a plain
    string. The reverse of value_to_json for event payloads. Moved to
    the shared loop (llm/lib/loop.ml owns it now that agent + pp-remote
    share the loop); re-exported for host callers. *)
let json_of_value = Loop.json_of_value

let rec value_of_json (j : Yojson.Safe.t) : Ast.value =
  match j with
  | `String s -> VStr s
  | `Int i -> VInt (Int64.of_int i)
  | `Intlit s -> (
      match Int64.of_string_opt s with Some i -> VInt i | None -> VStr s)
  | `Float f -> VFloat f
  | `Bool b -> VBool b
  | `Null -> VNil
  | `List js -> VList (List.map value_of_json js)
  | `Assoc _ -> VStr (Yojson.Safe.to_string j)

let str_arg name v =
  match v with
  | Ast.VStr s -> s
  | Ast.VSymbol s -> s
  | _ ->
      raise
        (Ast.Interp_error
           (Ast.Prim_error (name, "expected string/symbol argument",
    "bad-args")))

let emit_error name msg =
  raise (Ast.Interp_error (Ast.Prim_error (name, msg, "error")))

(** Register a host prim: dispatch table PLUS a VPrim binding in the
    shared env (Nopales resolves prims via env, like the image's
    auto-bind loop in session.ml). All registrations go through here. *)
let add_prim (t : t) (name : string) (fn : Ast.value list -> Ast.value)
    : unit =
  Eval.register_prim t.prims name fn;
  Env.define t.env name (Ast.VPrim name)

(** Path resolution: relative paths resolve under [cwd]; ".." escapes
    are refused (v0 sandbox posture: refuse, not normalize). *)
let resolve_path p =
  let p =
    if String.length p > 0 && p.[0] = '/' then p else Filename.concat !cwd p
  in
  let parts =
    String.split_on_char '/' p
    |> List.filter (fun s -> s <> "" && s <> ".")
  in
  if List.exists (fun s -> s = "..") parts then
    emit_error "fs/path" ("path escapes sandbox: " ^ p);
  "/" ^ String.concat "/" parts

(** Core eval path shared by user input and plugin callbacks. *)
(** Serialize Nopales evals. Manual lock/unlock (not use_rw):
    Interp_error is a normal outcome of eval and must not poison the
    mutex for later evals. *)
let with_eval_lock (t : t) (f : unit -> 'a) : 'a =
  Eio.Mutex.lock t.eval_mutex;
  Fun.protect ~finally:(fun () -> Eio.Mutex.unlock t.eval_mutex) f

let eval_string ~(rt : t) (src : string) : Ast.value =
  with_eval_lock rt (fun () ->
      Control.run_with_rim ~prims:rt.prims (fun () ->
          Eval.eval_string_direct ~prims:rt.prims rt.env src))

(** Evaluate all top-level forms in [src] into the shared env, under
    the eval lock + rim. Used to boot-load Nopales packages (libs/std,
    libs/html, libs/hyper) as SOURCE -- the same env as plugins/REPL, so
    the surface is redefinable live (borge/agent.borg surfaces). *)
let load_source ~(rt : t) (src : string) : unit =
  with_eval_lock rt (fun () ->
```

```
      Control.run_with_rim ~prims:rt.prims (fun () ->
          ignore (Eval.read_eval_all_direct ~prims:rt.prims rt.env src)))

(** Load a .pp file from [path] into the shared env. *)
let load_file ~(rt : t) (path : string) : (unit, string) result =
  try
    let ic = open_in_bin path in
    let n = in_channel_length ic in
    let s = really_input_string ic n in
    close_in ic;
    load_source ~rt s;
    Ok ()
  with exn -> Error (Printexc.to_string exn)

(** [apply_value_raw] -- apply a Nopales function value to host-side
    value args directly, assuming the caller ALREADY holds the eval lock
    AND has a rim active (direct_prims set). Used by prims that run
    inside an eval (route/dispatch invokes a handler from within the
    rim): re-locking the non-reentrant Eio.Mutex would deadlock, and the
    outer rim already services the handler's prim calls via
    [direct_prims]. Args NEVER go through the reader, so lambda values
    and arbitrary JSON strings are safe. *)
let apply_value_raw (t : t) (fn : Ast.value) (args : Ast.value list)
    : Ast.value =
  let fn_sym = "__host_apply_fn__" in
  Env.define t.env fn_sym fn;
  let arg_syms =
    List.mapi (fun i _ -> Printf.sprintf "__host_apply_arg_%d__" i) args
  in
  List.iter2 (Env.define t.env) arg_syms args;
  let src =
    "(" ^ fn_sym
    ^ List.fold_left (fun acc a -> acc ^ " " ^ a) "" arg_syms
    ^ ")"
  in
  Eval.eval_string_direct ~prims:t.prims t.env src

(** Apply a Nopales function value to host-side value args. Acquires
    the eval lock and establishes the rim -- for callers OUTSIDE an
    active eval (loop tool bodies, event handlers). Prims running
    inside an eval use [apply_value_raw] instead (no re-lock). *)
let apply_value (t : t) (fn : Ast.value) (args : Ast.value list) :
    Ast.value =
  with_eval_lock t (fun () ->
      Control.run_with_rim ~prims:t.prims (fun () ->
          apply_value_raw t fn args))

(* ---- Host prims ---- *)

let register_fs_prims (t : t) : unit =
  let read args =
    match args with
    | [ path ] -> (
        let p = resolve_path (str_arg "fs/read" path) in
        try
          let ic = open_in_bin p in
          let n = in_channel_length ic in
          let s = really_input_string ic n in
          close_in ic;
          VStr s
        with Sys_error e -> emit_error "fs/read" e)
    | _ -> emit_error "fs/read" "usage: (fs/read path)"
  in
  add_prim t "fs/read" read;
  let write args =
    match args with
    | [ path; content ] -> (
        let p = resolve_path (str_arg "fs/write" path) in
        let s = str_arg "fs/write" content in
        try
          let oc = open_out_bin p in
          output_string oc s;
          close_out oc;
```

```
          VBool true
        with Sys_error e -> emit_error "fs/write" e)
    | _ -> emit_error "fs/write" "usage: (fs/write path content)"
  in
  add_prim t "fs/write" write;
  let exists args =
    match args with
    | [ path ] ->
        VBool (Sys.file_exists (resolve_path (str_arg "fs/exists?" path)))
    | _ -> emit_error "fs/exists?" "usage: (fs/exists? path)"
  in
  add_prim t "fs/exists?" exists;
  (* (fs/ls path) -> JSON array of {"name","dir"} sorted by name *)
  let ls args =
    match args with
    | [ path ] -> (
        let p = resolve_path (str_arg "fs/ls" path) in
        try
          let entries =
            Sys.readdir p |> Array.to_list
            |> List.map (fun e ->
                   let full = Filename.concat p e in
                   `Assoc
                     [
                       ("name", `String e);
                       ("dir", `Bool (try Sys.is_directory full with _ ->
   false));
                     ])
          in
          VStr (Yojson.Safe.to_string (`List (List.sort compare entries)))
        with Sys_error e -> emit_error "fs/ls" e)
    | _ -> emit_error "fs/ls" "usage: (fs/ls path)"
  in
  add_prim t "fs/ls" ls;
  (* (fs/dir? path) -> VBool. True if path resolves to a directory. *)
  let is_dir args =
    match args with
    | [ path ] ->
        let p = resolve_path (str_arg "fs/dir?" path) in
        VBool (try Sys.is_directory p with Sys_error _ -> false)
    | _ -> emit_error "fs/dir?" "usage: (fs/dir? path)"
  in
  add_prim t "fs/dir?" is_dir

let register_shell_prims (t : t) : unit =
  (* (shell/run cmd) -> {"exit": n, "out": "...", "truncated": bool}.
     Fire-and-forget v0; the agent loop's real bash tool is in
     tools.ml with timeout/streaming. *)
  let run args =
    match args with
    | [ cmd ] -> (
        let cmd = str_arg "shell/run" cmd in
        let ic = Unix.open_process_in (cmd ^ " 2>&1") in
        let buf = Buffer.create 4096 in
        (try
           while true do
             Buffer.add_channel buf ic 4096
           done
         with End_of_file -> ());
        match Unix.close_process_in ic with
        | Unix.WEXITED code ->
            let out = Buffer.contents buf in
            let max_out = 50 * 1024 in
            let out, truncated =
              if String.length out > max_out then
                (String.sub out 0 max_out, true)
              else (out, false)
            in
            VStr
              (Yojson.Safe.to_string
                 (`Assoc
                   [
                     ("exit", `Int code);
```

```
                    ("out", `String out);
                    ("truncated", `Bool truncated);
                  ]))
        | _ -> emit_error "shell/run" "process did not exit normally")
    | _ -> emit_error "shell/run" "usage: (shell/run cmd)"
  in
  add_prim t "shell/run" run

let register_event_prims (t : t) : unit =
  (* (on "event" handler-fn) *)
  let on args =
    match args with
    | [ ev; fn ] ->
        let ev = str_arg "on" ev in
        let cell =
          match Hashtbl.find_opt t.handlers ev with
          | Some c -> c
          | None ->
              let c = ref [] in
              Hashtbl.replace t.handlers ev c;
              c
        in
        cell := !cell @ [ (fun vals -> apply_value t fn vals) ];
        VBool true
    | _ -> emit_error "on" "usage: (on \"event\" (lambda (ev) ...))"
  in
  add_prim t "on" on;
  (* (register-tool name params-alist fn). Params are validated but v0
     wire schemas are synthesized from param names only (loop.ml). *)
  let reg_tool args =
    match args with
    | [ name; _params; fn ] ->
        let name = str_arg "register-tool" name in
        Hashtbl.replace t.tools name { doc = ""; run = fn };
        if not (List.mem name t.tool_order) then
          t.tool_order <- t.tool_order @ [ name ];
        VBool true
    | _ -> emit_error "register-tool" "usage: (register-tool name params
     fn)"
  in
  add_prim t "register-tool" reg_tool;
  let reg_cmd args =
    match args with
    | [ name; doc; fn ] ->
        let name = str_arg "register-command" name in
        Hashtbl.replace t.commands name
          { doc = str_arg "register-command" doc; run = fn };
        VBool true
    | [ name; fn ] ->
        let name = str_arg "register-command" name in
        Hashtbl.replace t.commands name { doc = ""; run = fn };
        VBool true
    | _ ->
        emit_error "register-command"
          "usage: (register-command name [doc] fn)"
  in
  add_prim t "register-command" reg_cmd;
  let list_cmds _args =
    let names =
      Hashtbl.fold (fun k _ acc -> `String k :: acc) t.commands []
    in
    VStr (Yojson.Safe.to_string (`List (List.sort compare names)))
  in
  add_prim t "list-commands" list_cmds

let register_ui_prims (t : t) : unit =
  let notify args =
    match args with
    | [ msg ] ->
        Printf.printf "\027[2m[notify] %s\027[0m\n%!" (str_arg "ui/notify"
     msg);
        VNil
    | _ -> emit_error "ui/notify" "usage: (ui/notify msg)"
```

```
  in
  add_prim t "ui/notify" notify;
  let confirm args =
    match args with
    | [ msg ] ->
        Printf.printf "%s [y/N] %!" (str_arg "ui/confirm" msg);
        let line = try input_line stdin with End_of_file -> "n" in
        VBool
          (match String.lowercase_ascii (String.trim line) with
          | "y" | "yes" -> true
          | _ -> false)
    | _ -> emit_error "ui/confirm" "usage: (ui/confirm msg)"
  in
  add_prim t "ui/confirm" confirm

(* ---- Route table + request-context prims (agent-route-table,
   THE HOST SEAM; hypermedia-harness.request-runner AGENT HOST BACKING).

   Identical Nopales signatures to the image's route / route/lookup /
   route/dispatch, host-local backing (Runtime route table + a thin
   request context vs the image's Postgres routes + fat
   Request_context). libs/hyper runs unchanged on both. *)

(** Look up the active request shape, or error if none is bound (i.e. a
    request prim called outside a route/dispatch). Mirrors the image's
    prims_request require_request, but the agent's context is thinner. *)
let require_request (t : t) : request_shape =
  match !(t.current_request) with
  | Some r -> r
  | None ->
      emit_error "request" "no active request (call inside route/dispatch)"

(** [dispatch_route_raw] -- the shared route/dispatch body. Assumes the
    caller holds the eval lock AND has a rim active (the route/dispatch
    prim runs inside an eval; the HTTP gate uses the locking
    [dispatch_route] wrapper). Returns the rendered raw-HTML VStr the
    browser would receive. *)
let dispatch_route_raw (t : t) ~handler ~method_ ~path ~params
    ~headers ~body : Ast.value =
  let req : request_shape =
    { method_ = String.uppercase_ascii method_; path; params; headers; body
    }
  in
  match Env.lookup t.env handler with
  | None -> VStr (Printf.sprintf "route/dispatch: unbound handler %s"
    handler)
  | Some fn ->
      t.current_request := Some req;
      Fun.protect
        ~finally:(fun () -> t.current_request := None)
        (fun () ->
          try
            let v = apply_value_raw t fn [] in
            VStr (Prims_soup.render_handler_value v)
          with Ast.Interp_error ie ->
            VStr
              (Printf.sprintf "route/dispatch: %s"
                 (Ast.pp_interp_error ie)))

(** [dispatch_route] -- locking wrapper for OCaml callers OUTSIDE an
    active eval (the HTTP gate dispatches GET routes through the route
    table). Acquires the eval lock + establishes the rim. *)
let dispatch_route ~(rt : t) ~handler ~method_ ~path ~params ~headers
    ~body : Ast.value =
  with_eval_lock rt (fun () ->
      Control.run_with_rim ~prims:rt.prims (fun () ->
          dispatch_route_raw rt ~handler ~method_ ~path ~params ~headers
    ~body))

(** Resolve a URL path to its handler name via the agent route table. *)
let lookup_route ~(rt : t) (path : string) : string option =
  Hashtbl.find_opt rt.routes path

```

```
(** Call a Nopales template (zero-or-more-arg fn) by name and return the
    rendered raw-HTML string (raw-HTML-tagged VStrs unwrap). For OCaml
    callers outside an eval (the HTTP surface renders fragment templates).
    Returns [None] if the template is unbound or errors. *)
let call_template ~(rt : t) (name : string) (args : Ast.value list)
    : string option =
  match Env.lookup rt.env name with
  | None -> None
  | Some fn ->
      (try
         let v = apply_value rt fn args in
         Some (Prims_soup.render_handler_value v)
       with Ast.Interp_error _ -> None)

let register_route_prims (t : t) : unit =
  (* (route path handler) -- bind a path to a Nopales render handler name
     on the agent route table. The agent's local-lens backing for the
     image's (route ...) over Postgres. *)
  let route args =
    match args with
    | [ path; handler ] ->
        let path = str_arg "route" path in
        let handler = str_arg "route" handler in
        Hashtbl.replace t.routes path handler;
        VBool true
    | _ -> emit_error "route" "usage: (route path handler)"
  in
  add_prim t "route" route;
  (* (route/lookup path) -> handler-name VStr | VNil. Resolve a URL path
     to its handler via the route table -- exactly hyper/visit's
     resolver. *)
  let route_lookup args =
    match args with
    | [ path ] ->
        let path = str_arg "route/lookup" path in
        (match Hashtbl.find_opt t.routes path with
         | Some h -> VStr h
         | None -> VNil)
    | _ -> emit_error "route/lookup" "usage: (route/lookup path)"
  in
  add_prim t "route/lookup" route_lookup;
  (* (route/dispatch handler method path [params] [headers] [body])
     -> rendered raw-HTML VStr. Invoke the handler fn (looked up in env)
     through the rim (caller already holds the eval lock) with a THIN
     REQUEST CONTEXT bound (query-param / request-method / hx-request?
     resolve; DB/branch/auth request prims stay unbound), and return the
     rendered value the browser would receive. Reuses assoc_of +
     render_handler_value from the shared soup kernel. *)
  let route_dispatch args =
    match args with
    | VStr handler :: VStr meth :: VStr path :: rest ->
        let params =
          match rest with p :: _ -> Prims_soup.assoc_of p | _ -> []
        in
        let headers =
          match rest with
          | _ :: h :: _ ->
              List.map
                (fun (k, v) -> (String.lowercase_ascii k, v))
                (Prims_soup.assoc_of h)
          | _ -> []
        in
        let body =
          match rest with
          | _ :: _ :: VStr b :: _ -> b
          | _ -> ""
        in
        dispatch_route_raw t ~handler ~method_:meth ~path ~params ~headers
     ~body
    | _ -> emit_error "route/dispatch"
              "usage: (route/dispatch handler method path [params]
     [headers] [body])"
  in
```

```
  add_prim t "route/dispatch" route_dispatch;
  (* ---- request-shape prims (the thin context) ----
     query-param / request-method / hx-request? -- they back
     route/dispatch's thin context and libs/hyper's hyper/do-swap
     HX-Request header path. DB/branch/auth request prims are NOT
     registered here (the agent's request context carries only
     request-shape values); calling them yields unbound-prim. *)
  let query_param args =
    match args with
    | [ key ] ->
        let r = require_request t in
        let key = str_arg "query-param" key in
        (match List.assoc_opt key r.params with
         | Some v -> VStr v
         | None -> VNil)
    | _ -> emit_error "query-param" "usage: (query-param key)"
  in
  add_prim t "query-param" query_param;
  let request_method _args =
    let r = require_request t in
    VStr r.method_
  in
  add_prim t "request-method" request_method;
  let hx_request_p _args =
    let r = require_request t in
    (* hyper/do-swap passes hx-request in the PARAMS alist; the HTTP
       gate passes it in HEADERS. Check both so libs/hyper works the same
       way it does on the image. *)
    let from_headers =
      match List.assoc_opt "hx-request" r.headers with
      | Some v when v <> "" && v <> "false" -> true
      | _ -> false
    in
    let from_params =
      match List.assoc_opt "hx-request" r.params with
      | Some v when v <> "" && v <> "false" -> true
      | _ -> false
    in
    VBool (from_headers || from_params)
  in
  add_prim t "hx-request?" hx_request_p;
  (* (session/last-turn-parts) -> VList of part alists captured from the
     last consumed chat/agent turn (the data shape soup/apply-parts! /
     hyper/stream consume). Lets a consumed turn replay deterministically
     through hyper/stream (surfaces smoke). Returns VNil if no turn ran. *)
  let last_turn_parts _args = t.last_turn_parts
  in
  add_prim t "session/last-turn-parts" last_turn_parts

(** Install the pricklypear-pure-prims surface (dict/json/string/
    list/math -- the frozen core set per lisp-interpreter.core-freeze)
    into our table. The image's registry (image/lib/primitives.ml)
    enumerates the same set with image-specific additions; here we
    bind only what exists in pure/, no DB/auth/hx prims. *)
let register_pure_prims (t : t) : unit =
  let open Prims_pure in
  let r n f = add_prim t n f in
  r "+" prim_add;
  r "-" prim_sub;
  r "*" prim_mul;
  r "/" prim_div;
  r "div" prim_div;
  r "mod" prim_mod;
  r "quotient" prim_quotient;
  r "remainder" prim_remainder;
  r "abs" prim_abs;
  r "min" prim_min;
  r "max" prim_max;
  r "=" prim_num_eq;
  r "<" prim_lt;
  r ">" prim_gt;
  r "<=" prim_le;
  r ">=" prim_ge;
```

```
  r "not" prim_not;
  r "list" prim_list;
  r "car" prim_car;
  r "cdr" prim_cdr;
  r "cons" prim_cons;
  r "cadr" prim_cadr;
  r "caar" prim_caar;
  r "caddr" prim_caddr;
  r "cddr" prim_cddr;
  r "null?" prim_null_p;
  r "pair?" prim_pair_p;
  r "append" prim_append;
  r "reverse" prim_reverse;
  r "nth" prim_nth;
  r "count" prim_count;
  r "string-append" prim_string_append;
  r "string-length" prim_string_length;
  r "substring" prim_substring;
  r "string-contains?" prim_string_contains;
  r "string-split" prim_string_split;
  r "string-trim" prim_string_trim;
  r "string-replace" prim_string_replace;
  r "string-downcase" prim_string_downcase;
  r "string-upcase" prim_string_upcase;
  r "string-starts-with?" prim_string_starts_with_p;
  r "string-ends-with?" prim_string_ends_with_p;
  r "string=?" prim_string_eq;
  r "string-eq" prim_string_eq;
  r "type-of" prim_type_of;
  r "symbol?" prim_symbol_p;
  r "string?" prim_string_p;
  r "symbol->string" prim_symbol_to_string;
  r "string->symbol" prim_string_to_symbol;
  r "str" prim_str;
  r "as-str" prim_as_str;
  r "assoc" prim_assoc;
  r "member" prim_member;
  r "json-parse" prim_json_parse;
  r "json-emit" prim_json_emit;
  r "dict-get" prim_dict_get;
  r "dict-set" prim_dict_set;
  r "dict-keys" prim_dict_keys;
  r "equal?" prim_equal_p;
  r "random-uuid" prim_random_uuid;
  r "render-html" prim_render_html;
  r "html-escape" prim_html_escape;
  r "string-split" prim_string_split

(** Register the SHARED soup/ tree prims (Phase 0: lifted into
    pure/prims_soup.ml so both hosts register the same tree prims).
    [Prims_soup.register_pure_prims] fills the prim table; we then
    bind each as a VPrim in env so Nopales resolves them by name
    (matching [add_prim]'s env-bind half, which the pure helper does
    not do). *)
let register_soup_prims (t : t) : unit =
  Prims_soup.register_pure_prims t.prims;
  Hashtbl.iter
    (fun name _ ->
      match name with
      | n when String.length n >= 5 && String.sub n 0 5 = "soup/" ->
          Env.define t.env name (Ast.VPrim name)
      | _ -> ())
    t.prims

(** Minimal agent stdlib -- the host library layer (borge/agent.borg
    nopales-embed: "the agent adds HOST PRIMS and an optional host library
    layer in .pp"). Provides the [let*] sequential-binding macro that
    libs/hyper uses, without loading the heavy libs/std (which depends on
    image-only prims: css/*, menu/*, list-bindings, redirect, ...).
    The image gets [let*] from libs/std; the agent gets it here. Same
    expansion as the libs/std macro (written with car/cdr so it depends on
    no c*r beyond the registered set). *)
let boot_stdlib = {|
```

```
(defmacro let* (bindings body)
  (if (null? bindings)
      (list (quote let) (list) body)
      (list (quote let)
            (list (list (car (car bindings)) (car (cdr (car bindings)))))
            (list (quote let*) (cdr bindings) body))))
|}

(** Identity prims -- host-local backing. The image's whoami/current-author
    read the HTTP session identity. In the agent: whoami is the OPERATOR
    (one unix process on their machine); current-author stays INTENTIONALLY
    UNBOUND -- authorship is a DB-row concept (agent-route-table.pg-prim-
    unbound pins it: routes calling it must report unbound), and the lens
    has no author column. Same name, host-local backing -- the route/
    dispatch seam philosophy (borge/agent.borg, borge/local-first.borg). *)
let operator_name () : string =
  try Unix.getlogin ()
  with _ -> (
    try Sys.getenv "USER"
    with _ -> (
      try Sys.getenv "LOGNAME" with _ -> "glochid"))

let register_identity_prims (t : t) : unit =
  add_prim t "whoami" (fun _args -> Ast.VStr (operator_name ()))

let make () : t =
  let t =
    {
      env = Env.create ();
      prims = Eval.make_prim_table ();
      eval_mutex = Eio.Mutex.create ();
      handlers = Hashtbl.create 8;
      tools = Hashtbl.create 16;
      tool_order = [];
      commands = Hashtbl.create 8;
      routes = Hashtbl.create 16;
      current_request = ref None;
      last_turn_parts = Ast.VNil;
    }
  in
  register_pure_prims t;
  register_soup_prims t;
  register_fs_prims t;
  register_shell_prims t;
  register_event_prims t;
  register_ui_prims t;
  register_route_prims t;
  register_identity_prims t;
  (* Host library layer: the minimal stdlib (let* macro) so libs/hyper
     and the Nopales surface run unchanged. *)
  load_source ~rt:t boot_stdlib;
  t

(** Fire an event: run handlers in registration order. Returns the
    last handler's value (or VNil). Callers that care about
    short-circuiting (tool_call block) inspect per-handler values
    themselves. *)
let fire (t : t) (ev : string) (payload : Ast.value list) : Ast.value list =
  match Hashtbl.find_opt t.handlers ev with
  | None -> []
  | Some hs ->
      List.map
        (fun h ->
          (* A handler raising must not kill the host turn; surface it
             as an {"error": ...} value the caller can inspect. *)
          try h payload
          with Ast.Interp_error ie ->
            Ast.VStr
              (Yojson.Safe.to_string
                 (`Assoc
                   [
                     ("error",
                      `String
```

```
                        (Printf.sprintf "%s handler: %s" ev
                           (Ast.pp_interp_error ie)));
                    ]))
        !hs

(** Run a plugin tool's closure with JSON args, containing errors into
    a {"error": ...} JSON string. *)
let eval_tool_body (t : t) (name : string) (args_json : string) : Ast.value
    =
  match Hashtbl.find_opt t.tools name with
  | None ->
      VStr
        (Yojson.Safe.to_string
           (`Assoc [ ("error", `String ("no tool named " ^ name)) ]))
  | Some e ->
      (try apply_value t e.run [ VStr args_json ]
       with Ast.Interp_error ie ->
         VStr
           (Yojson.Safe.to_string
              (`Assoc
                [ ("error", `String (Ast.pp_interp_error ie)) ])))

let register_builtin_tool (t : t) (name : string) ~(run : Yojson.Safe.t ->
    string) : unit =
  (* Builtin host tools bypass Nopales application for speed and to
     keep their bodies in OCaml; the registry shape is the same so
     plugin hooks (tool_call / tool_result events) still see them. *)
  Hashtbl.replace t.tools name
    { doc = "builtin"; run = VPrim ("__builtin__" ^ name) };
  if not (List.mem name t.tool_order) then
    t.tool_order <- t.tool_order @ [ name ];
  (* prim that the registry entry's VPrim names; loop.ml calls
     builtins through this prim table entry instead of applying a
     lambda. *)
  add_prim t ("__builtin__" ^ name)
    (fun args ->
      match args with
      | [ Ast.VStr args_json ] -> (
          try VStr (run (Yojson.Safe.from_string args_json))
          with
          | Yojson.Json_error e ->
              VStr
                (Yojson.Safe.to_string
                   (`Assoc [ ("error", `String ("bad json: " ^ e)) ]))
          | Ast.Interp_error ie ->
              VStr
                (Yojson.Safe.to_string
                   (`Assoc
                     [ ("error", `String (Ast.pp_interp_error ie)) ])))
      | _ -> emit_error "__builtin" "expected single JSON string arg")

let is_builtin (t : t) (name : string) : bool =
  match Hashtbl.find_opt t.tools name with
  | Some { run = VPrim p; _ } -> String.length p > 10 && String.sub p 0 11
      = "__builtin__"
  | _ -> false

(** Host adapter for the shared agent turn loop (llm/lib/loop.ml): the
    small [Loop.host] seam -- live tool order, registry lookup, event
    fire, tool-body eval, and serialized prim dispatch through the eval
    lock + the agent's local host rim. This is the ONLY coupling the
    loop has to this host; pp-remote supplies the same record over the
    proxy rim (borge/remote-eval.borg agent-loop). *)
let loop_host (t : t) : Loop.host =
  {
    Loop.tool_order = (fun () -> t.tool_order);
    tool_entry = (fun name -> Hashtbl.find_opt t.tools name);
    fire = fire t;
    eval_tool_body = eval_tool_body t;
    call_prim =
      (fun name args ->
        with_eval_lock t (fun () ->
            Control.run_with_rim ~prims:t.prims (fun () ->
```

```
789                     (Hashtbl.find t.prims name) args)));
790   }
```

39 agent/lib/surface.ml

```
(** Agent Nopales surface (borge/agent.borg surfaces).

    The chat + code browser pages are NOPALES-AUTHORED htmx fragments on
    the agent route table, rendered by Nopales via [libs/html] -- NOT
    compiled OCaml pages and NOT vanilla-JS SSE. The surface is
    redefinable live (same env as plugins/REPL): re-evaluating a
    `(define chat/page ?)` at the REPL changes the next render without a
    restart.

    The multipart STREAM for a chat/agent turn stays OCaml (http_ui.ml):
    provider SSE ? delta assembly ? per-part (html, HX-Target, HX-Swap)
    frame + flush. Nopales owns the page skeleton + the fragment
    templates (chat/user-bubble, chat/turn-open, chat/think-open,
    chat/think-token, chat/tool-open, chat/tool-line, chat/tool-done,
    chat/token, chat/error); OCaml owns the effects-coloured stream and
    calls these templates per part.

    [load] boot-loads the embedded Nopales packages (libs/html, libs/hyper)
    then this surface source into the shared env, and registers the
    route-table routes (/, /code). *)

open Pricklypear_lisp

(** The surface source. Defines chat/page, chat/user-bubble,
    chat/llm-bubble-open, chat/token, code/page, code/rows and registers
    the / and /code routes via the agent route table. Uses libs/html +
    the agent's request-shape prims (query-param). *)
let surface_source = {pp|
;; ---- chat surface (GET /) ----
;; The chat page skeleton: log target + an hx-post=/chat form with the
;; upstream htmx-4 hx-multipart extension so a multipart/mixed reply
;; streams each part live into the bubble. htmx + hx-multipart are
;; vendored assets served at /assets/*.js (baked into the agent binary).
;; hx-config '{"timeout":0}' disables htmx's 60s defaultTimeout for a
;; request whose lawful lifetime is "a whole agent turn".
(define (chat/page)
  (render-html
    (string-append
      "<!doctype html><html><head><meta charset=utf-8><title>agent</title>"
      "<style>"
     "body{background:#0f1117;color:#d8dee9;font-family:ui-monospace,Menlo,monospace;margin:0;padding:1.5rem}"
     "a{color:#7aa2f7;text-decoration:none}a:hover{text-decoration:underline}"
      ".muted{color:#6b7080}"
      "#log{height:70vh;overflow-y:scroll;border:1px solid
     #333;border-radius:6px;padding:.75rem;margin-bottom:.75rem}"
      ".bubble{background:#1b1f2a;border-radius:6px;padding:.5rem
     .75rem;margin:.5rem 0;white-space:pre-wrap}"
      ".bubble.user{color:#9ece6a}"
      ".bubble.llm{color:#c0caf5}.bubble.repl{color:#bb9af7}"
      ".trace{color:#6b7080;font-size:.85em;white-space:pre-wrap}"
      ".think{color:#6b7080;font-size:.9em;margin:.4rem 0}"
      ".think summary{cursor:pointer}"
      ".think>div{white-space:pre-wrap}"
      ".reason{color:#8a8f9e;font-style:italic}"
      ".think .tk,.think .tool{margin:.25rem 0 .25rem .25rem}"
      ".think .tk>summary,.think .tool>summary{color:#8a8f9e}"
      ".think .tk>div,.think
     .tool>div{margin-left:.9rem;padding-left:.5rem;border-left:1px solid
     #2a2e3a}"
      ".think:not(:has(.reason,.trace)){display:none}"
      "form{display:flex;gap:.5rem}"
      "input{flex:1;background:#1b1f2a;color:#d8dee9;border:1px solid
     #333;border-radius:6px;padding:.5rem}"
     "button{background:#2a6fdb;color:#fff;border:0;border-radius:6px;padding:.5rem
     1rem;cursor:pointer}"
      "</style></head><body>"
      "<h1><a href=\"/\">agent</a> <span class=\"muted\">-- <a
     href=\"/code\">/code</a></span></h1>"
```

```
      "<div id=\"log\"><div class=\"muted\">agent ready.</div></div>"
      "<form id=\"f\" hx-post=\"/chat\" hx-target=\"#log\"
     hx-swap=\"beforeend\" hx-ext=\"hx-multipart\""
      " hx-config='{\"timeout\":0}' hx-on::before:request=\"this.reset()\">"
      "<input id=\"msg\" name=\"msg\" type=\"text\" placeholder=\"prompt?\"
     autocomplete=\"off\" autofocus>"
      "<button>send</button>"
      "</form>"
      "<script src=\"/assets/htmx.min.js\"></script>"
      "<script src=\"/assets/htmx-2-compat.min.js\"></script>"
      "<script src=\"/assets/hx-multipart.js\"></script>"
      "</body></html>")))
(route "/" "chat/page")

;; ---- chat fragment templates (called per-part by the OCaml stream) ----
;; Each returns a raw-HTML fragment the multipart emitter writes as one
;; part with per-part HX-Target/HX-Swap. Redefine live to restyle.
(define (chat/user-bubble text)
  (render-html (string-append "<div class=\"bubble user\">" (html-escape
     text) "</div>")))

(define (chat/llm-bubble-open id)
  (render-html (string-append "<div class=\"bubble llm\" id=\"" id
     "\"></div>")))

;; REPL result bubble: a chat prompt starting with ( evaluates LOCALLY in
;; the shared env (same rule as the stdin REPL -- no model turn) and the
;; last value lands here, REPL-style via the host printer.
(define (chat/repl-bubble text)
  (render-html (string-append "<div class=\"bubble repl\">" (html-escape
     text) "</div>")))

;; Per-turn skeleton: collapsible thinking zone (reasoning + tool traces)
;; followed by the streamed response bubble. Render order in one part
;; because the targets must exist before any delta parts arrive. #<id>-think
;; is the zone container: the stream appends one NESTED <details> block per
;; reasoning run (chat/think-open) and per tool call (chat/tool-open), so a
;; finished turn scans as a list of collapsible items instead of one flat
;; span/trace wall.
(define (chat/turn-open id)
  (render-html
    (string-append
      "<div class=\"turn\"><details class=\"think\"><summary>thinking +
     tools</summary>"
      "<div id=\"" id "-think\"></div></details>"
      "<div class=\"bubble llm\" id=\"" id "\"></div></div>")))

;; Nested thinking block: one contiguous reasoning run gets its own
;; <details>/<summary> inside the zone; think-token deltas stream into #id.
;; Starts CLOSED so a finished turn stays scannable.
(define (chat/think-open id)
  (render-html
    (string-append
      "<details class=\"tk\"><summary>thinking</summary>"
      "<div id=\"" id "\"></div></details>")))

;; Nested tool-call block: <details>/<summary> inside the zone; input/output
;; lines (chat/tool-line) stream into #id, and chat/tool-done marks the
;; summary via innerHTML on #<id>-sum when the call returns.
(define (chat/tool-open id name)
  (render-html
    (string-append
      "<details class=\"tool\"><summary id=\"" id "-sum\">tool: "
      (html-escape name) "</summary><div id=\"" id "\"></div></details>")))

(define (chat/tool-line label text)
  (render-html
    (string-append
      "<div class=\"trace\">" (html-escape label) " "
      (html-escape text) "</div>")))

(define (chat/tool-done name)
  (render-html (string-append "tool: " (html-escape name) " &#10003;")))
```

```
129
130 ;; Provider chain-of-thought delta, into the current thinking block.
131 (define (chat/think-token text)
132   (render-html (string-append "<span class=\"reason\">" (html-escape text)
      "</span>")))
133
134 (define (chat/token text)
135   (render-html (string-append "<span>" (html-escape text) "</span>")))
136
137 (define (chat/error text)
138   (render-html (string-append "<div class=\"trace\">[turn error: "
      (html-escape text) "]</div>")))
139
140 ;; ---- code browser (GET /code) ----
141 ;; Read-only file tree + file view, rendered by Nopales via fs/ls +
142 ;; fs/read + fs/dir? (agent host prims). query-param "path" selects the
143 ;; entry (relative to the agent's cwd); the agent's fs prims refuse ".."
144 ;; escapes (sandbox posture).
145 (define (code/page)
146   (let* ((p (query-param "path"))
147          (rel (if (null? p) "" p)))
148     (if (fs/dir? rel)
149         (let* ((entries (fs/ls rel))
150                (items (json-parse entries)))
151           (render-html
152             (string-append
153               "<!doctype html><html><head><meta
      charset=utf-8><title>code</title>"
154               "<style>"
155
      "body{background:#0f1117;color:#d8dee9;font-family:ui-monospace,Menlo,monospace;margin:0;padding:1.5rem}"
156               "a{color:#7aa2f7;text-decoration:none}"
157               ".muted{color:#6b7080}"
158               "table{border-collapse:collapse}td,th{padding:.25rem
      .75rem;border-bottom:1px solid #222}"
159
      "pre{background:#12141c;padding:1rem;border-radius:6px;overflow:auto}"
160               "</style></head><body>"
161               "<h1><a href=\"/\">agent</a> <span class=\"muted\">/</span>
      <a href=\"/code\">code</a></h1>"
162               "<p class=\"muted\">" (html-escape rel) "</p>"
163               "<table><tr><th></th><th>name</th></tr>"
164               (code/rows items rel)
165               "</table></body></html>")))
166         (let* ((content (fs/read rel)))
167           (render-html
168             (string-append
169               "<!doctype html><html><head><meta
      charset=utf-8><title>code</title>"
170               "<style>"
171
      "body{background:#0f1117;color:#d8dee9;font-family:ui-monospace,Menlo,monospace;margin:0;padding:1.5rem}"
172               "a{color:#7aa2f7;text-decoration:none}"
173               ".muted{color:#6b7080}"
174
      "pre{background:#12141c;padding:1rem;border-radius:6px;overflow:auto}"
175               "</style></head><body>"
176               "<h1><a href=\"/\">agent</a> <span class=\"muted\">/</span>
      <a href=\"/code\">code</a></h1>"
177               "<p class=\"muted\">" (html-escape rel) "</p>"
178               "<pre>" (html-escape content) "</pre></body></html>"))))))
179
180 (define (code/rows items rel)
181   (list/foldl
182     (lambda (acc entry)
183       (let* ((name (dict-get entry "name"))
184              (dir? (string-eq (dict-get entry "dir") "true"))
185              (kind (if dir? "dir" "file"))
186              (child (if (string-eq rel "") name (string-append rel "/"
      name))))
187         (string-append acc
188           "<tr><td>" kind
189           "</td><td><a href=\"/code?path=" child "\">"
```

```
          (html-escape name) "</a></td></tr>")))
    "" items))

(route "/code" "code/page")
|pp}

(** Boot-load the embedded Nopales packages + this surface into the
    shared env, registering the route-table routes. Idempotent-ish:
    re-evaluating defines redefines bindings live. *)
let load ~(rt : Runtime.t) : unit =
  Runtime.load_source ~rt Libs_source.html_lib;
  Runtime.load_source ~rt Libs_source.hyper_lib;
  Runtime.load_source ~rt surface_source
```

40 agent/test/agent_test.ml

```
(** Tests for borge/agent.borg: plugin load, rim, events, loop. No
    network -- provider is a canned script. *)

open Pricklypear_lisp

let pp_value_testable =
  Alcotest.testable
    (fun fmt v -> Format.pp_print_string fmt (Reader.pp_value v))
    (fun a b -> Reader.pp_value a = Reader.pp_value b)

(** plugin-model: loading order + events + register-command +
    register-tool *)
let test_plugin_lifecycle ~rt () =
  let pd =
    Filename.concat (Filename.get_temp_dir_name ())
      (Printf.sprintf "agent-test-%d" (Unix.getpid ()))
  in
  (try Unix.mkdir pd 0o755 with Unix.Unix_error (Unix.EEXIST, _, _) -> ());
  let plugins_dir = Filename.concat pd "plugins" in
  (try Unix.mkdir plugins_dir 0o755
   with Unix.Unix_error (Unix.EEXIST, _, _) -> ());
  let path = Filename.concat plugins_dir "test.pp" in
  let oc = open_out path in
  output_string oc
    {|(on "tool_call" (lambda (ev) ev))
(register-command "ping" "pong" (lambda (args) "pong"))
(register-tool "greet" (list) (lambda (json-args) "hello-from-plugin"))|};
  close_out oc;
  (match Plugins.load_file rt path with
   | Ok () -> ()
   | Error e -> Alcotest.fail e);
  (* command registered *)
  (match Runtime.eval_string ~rt "(list-commands)" with
   | VStr s ->
       Alcotest.(check bool) "ping registered" true
         (try ignore (Str.search_forward (Str.regexp_string "ping") s 0);
     true
          with Not_found -> false)
   | _ -> Alcotest.fail "list-commands not a string");
  (* tool registered *)
  Alcotest.(check bool) "greet registered" true
    (Hashtbl.mem rt.Runtime.tools "greet");
  (* tool_call handler fires *)
  let results =
    Runtime.fire rt "tool_call"
      [ Ast.VStr {|{"id":"1","name":"read","input":{"path":"x"}}|} ]
  in
  Alcotest.(check int) "one handler ran" 1 (List.length results)

(** nopales-embed: unregistered prim produces Interp_error (not
    Unhandled); live redefine takes effect immediately; a plugin
    callback during an active turn flows through the rim. *)
let test_rim_errors_and_redefine ~rt () =
  (match
     (try
        ignore (Runtime.eval_string ~rt "(no-such-prim 1)");
        `No_error
      with Ast.Interp_error _ -> `As_expected)
   with
   | `As_expected -> ()
   | `No_error -> Alcotest.fail "expected Interp_error for unknown prim");
  ignore (Runtime.eval_string ~rt "(define (f x) (+ x 1))");
  Alcotest.check pp_value_testable "first def" (VInt 42L)
    (Runtime.eval_string ~rt "(f 41)");
  ignore (Runtime.eval_string ~rt "(define (f x) (* x 2))");
  Alcotest.check pp_value_testable "redefined" (VInt 42L)
    (Runtime.eval_string ~rt "(f 21)")

(** agent-loop: a mock provider emitting one tool call round-trips
    exactly one follow-up call. *)
let test_loop_tool_round_trip ~env ~sw ~rt () =
```

```
  let calls = ref 0 in
  let script = ref 0 in
  let complete ~env:_ ~sw:_ ~cfg:_ ~messages:_ ~tools:_ ~on_delta:_ =
    incr calls;
    let n = !script in
    incr script;
    if n = 0 then
      Ok
        {
          Provider.text = "";
          calls =
            [
              {
                id = "t1";
                name = "bash";
                input = `Assoc [ ("command", `String "echo hello") ];
              };
            ];
          stop_reason = "tool_use";
          raw = `Assoc [ ("role", `String "assistant"); ("content", `String
    "") ];
        }
    else
      Ok
        {
          Provider.text = "done";
          calls = [];
          stop_reason = "stop";
          raw = `Assoc [ ("role", `String "assistant"); ("content", `String
    "done") ];
        }
  in
  let cfg : Provider.cfg =
    { base_url = "http://127.0.0.1:1"; api_key = ""; model = "mock"; system
    = "" }
  in
  let history = ref [] in
  match
    Loop.run ~env ~sw ~host:(Runtime.loop_host rt) ~cfg ~complete
    ~on_event:(fun _ -> ()) history
      "say hello using echo"
  with
  | Error e -> Alcotest.fail e
  | Ok () ->
      Alcotest.(check int) "two provider calls" 2 !calls;
      Alcotest.(check int) "history has 4 entries" 4 (List.length !history)

(** agent-loop: a tool_call hook blocking reaches the model as an
    error tool result. *)
let test_loop_tool_block ~env ~sw ~rt () =
  ignore
    (Runtime.eval_string ~rt
       {|(on "tool_call" (lambda (ev)
            (if (string-contains? (dict-get ev "input") "rm -rf")
                (dict-set (dict-set "{}" "block" "true") "reason" "nope")
                "{}")))|});
  let script = ref 0 in
  let complete ~env:_ ~sw:_ ~cfg:_ ~messages ~tools:_ ~on_delta:_ =
    let n = !script in
    incr script;
    if n = 0 then
      Ok
        {
          Provider.text = "";
          calls =
            [
              {
                id = "t1";
                name = "bash";
                input = `Assoc [ ("command", `String "rm -rf /") ];
              };
            ];
          stop_reason = "tool_use";
```

```
          raw = `Assoc [ ("role", `String "assistant"); ("content", `String
    "") ];
        }
    else (
      let saw_block =
        List.exists
          (fun (m : Provider.message) ->
            match m.content with
            | `String s ->
                (try
                   ignore
                     (Str.search_forward (Str.regexp_string "blocked:
    nope") s 0);
                   true
                 with Not_found -> false)
            | _ -> false)
          messages
      in
      Alcotest.(check bool) "block reason reached model" true saw_block;
      Ok
        {
          Provider.text = "ok";
          calls = [];
          stop_reason = "stop";
          raw = `Assoc [ ("role", `String "assistant"); ("content", `String
    "ok") ];
        })
  in
  let cfg : Provider.cfg =
    { base_url = "http://127.0.0.1:1"; api_key = ""; model = "mock"; system
    = "" }
  in
  match
    Loop.run ~env ~sw ~host:(Runtime.loop_host rt) ~cfg ~complete
    ~on_event:(fun _ -> ()) (ref [])
      "delete everything"
  with
  | Error e -> Alcotest.fail e
  | Ok () -> ()

(** agent-loop regression (turn-2 provider 400): the assistant entry
    replayed in history must carry provider-shaped CONTENT (a string /
    part array), never the whole raw assistant message object nested under
    "content", and must replay tool_calls so the following tool-role entry
    pairs up. Catches the loop.ml history-append nesting bug that the real
    gateway rejected on the second message. *)
let test_loop_turn2_history_shape ~env ~sw ~rt () =
  let calls = ref 0 in
  let captured = ref [] in
  let complete ~env:_ ~sw:_ ~cfg:_ ~messages ~tools:_ ~on_delta:_ =
    incr calls;
    if !calls = 1 then
      (* first reply: assistant WITH a tool call, provider-shaped raw *)
      Ok
        {
          Provider.text = "using tool";
          calls =
            [
              {
                id = "t1";
                name = "bash";
                input = `Assoc [ ("command", `String "echo hi") ];
              };
            ];
          stop_reason = "tool_use";
          raw =
            `Assoc
              [ ("role", `String "assistant");
                ("content", `String "using tool");
                ( "tool_calls",
                  `List
                    [
                      `Assoc
```

```
                        [ ("id", `String "t1");
                          ("type", `String "function");
                          ( "function",
                            `Assoc
                              [ ("name", `String "bash");
                                ( "arguments",
                                  `String "{\"command\":\"echo hi\"}" ) ] );
                          ];
                    ] ) ];
        }
    else if !calls = 2 then
      Ok
        {
          Provider.text = "done";
          calls = [];
          stop_reason = "stop";
          raw = `Assoc [ ("role", `String "assistant"); ("content", `String
     "done") ];
        }
    else (
      captured := messages;
      Ok
        {
          Provider.text = "ok2";
          calls = [];
          stop_reason = "stop";
          raw = `Assoc [ ("role", `String "assistant"); ("content", `String
     "ok2") ];
        })
  in
  let cfg : Provider.cfg =
    { base_url = "http://127.0.0.1:1"; api_key = ""; model = "mock"; system
     = "" }
  in
  let history = ref [] in
  (match
     Loop.run ~env ~sw ~host:(Runtime.loop_host rt) ~cfg ~complete
     ~on_event:(fun _ -> ()) history
       "first"
   with Error e -> Alcotest.fail e | Ok () -> ());
  (match
     Loop.run ~env ~sw ~host:(Runtime.loop_host rt) ~cfg ~complete
     ~on_event:(fun _ -> ()) history
       "second"
   with Error e -> Alcotest.fail e | Ok () -> ());
  let msgs = !captured in
  (* system is prepended at REQUEST build time, so msgs here are:
     [user first; assistant+tool_calls; tool; assistant done; user second]
     *)
  Alcotest.(check int) "turn-2 replays 5 messages" 5 (List.length msgs);
  let a1 = List.nth msgs 1 in
  Alcotest.(check string) "assistant role" "assistant" a1.Provider.role;
  (match a1.Provider.content with
   | `String s -> Alcotest.(check string) "content is a string" "using
     tool" s
   | other ->
       Alcotest.fail
         ("assistant content must be a string, got: "
         ^ Yojson.Safe.to_string other));
  (match a1.Provider.tool_calls with
   | Some (`List (_ :: _)) -> ()
   | other ->
       Alcotest.fail
         ("assistant entry must replay tool_calls, got: "
         ^ Yojson.Safe.to_string (Option.value ~default:`Null other)));
  let a2 = List.nth msgs 3 in
  (match a2.Provider.content with
   | `String s -> Alcotest.(check string) "final content string" "done" s
   | other ->
       Alcotest.fail
         ("final assistant content must be a string, got: "
         ^ Yojson.Safe.to_string other));
  (match a2.Provider.tool_calls with
```

```
   | None -> ()
   | Some tc ->
       Alcotest.fail
         ("plain assistant entry must not carry tool_calls: "
         ^ Yojson.Safe.to_string tc));
  (* Wire shape: the actual request body validates against the gateway's
     expectation (content string, tool_calls array on the assistant msg). *)
  let body =
    Provider.Openai.request_body ~cfg ~messages:msgs ~tools:[] ~stream:false
  in
  let open Yojson.Safe.Util in
  let wire = Yojson.Safe.from_string body in
  let m1 = wire |> member "messages" |> index 2 in
  Alcotest.(check bool) "wire messages[2] role assistant" true
    (m1 |> member "role" |> to_string = "assistant");
  Alcotest.(check bool) "wire messages[2] content string" true
    (m1 |> member "content" |> to_string_option = Some "using tool");
  Alcotest.(check bool) "wire messages[2] has tool_calls" true
    (match m1 |> member "tool_calls" with `List (_ :: _) -> true | _ ->
     false)

(** provider-stream: SSE [data:] payloads reconstruct a [reply]; text
    deltas fire per chunk; tool-call argument fragments concatenate. *)
let test_sse_stream_reconstruction () =
  let deltas = ref [] in
  (* Each element is the [data:] payload for one SSE chunk. Two text
     deltas, a tool-call split across two fragments (id+name first,
     arguments the second time), a [DONE] marker. *)
  let reasons = ref [] in
  let lines =
    [
      {|{"choices":[{"delta":{"reasoning_content":"thinking
    "},"finish_reason":null}]}|};
      (* Synthetic's gateway streams thinking as plain "reasoning"
         (GLM-4.7-Flash wire probe 2026-08): alias must land in the same
         reasoning buffer / Reasoning deltas. *)
      {|{"choices":[{"delta":{"reasoning":"rea"},"finish_reason":null}]}|};
      {|{"choices":[{"delta":{"content":"hello "},"finish_reason":null}]}|};
    {|{"choices":[{"delta":{"tool_calls":[{"index":0,"id":"t1","function":{"name":"bash","arguments":"{\"command\":\"ech"}}
    {|{"choices":[{"delta":{"tool_calls":[{"index":0,"function":{"arguments":"o
    hello\"}"}}]}}]}|};
      {|{"choices":[{"delta":{"content":" world"},"finish_reason":null}]}|};
      {|{"choices":[{"delta":{},"finish_reason":"tool_calls"}]}|};
      "[DONE]";
    ]
  in
  let r =
    Provider.Openai.parse_stream_lines lines ~on_delta:(fun d ->
        match d with
        | Provider.Reasoning s -> reasons := !reasons @ [ s ]
        | Provider.Text s -> deltas := !deltas @ [ s ]
        | Provider.Done -> ())
  in
  Alcotest.(check (list string)) "reasoning delta streamed separately"
    [ "thinking "; "rea" ] !reasons;
  Alcotest.(check bool) "synthetic raw keeps reasoning_content" true
    (match r.Provider.raw with
     | `Assoc f ->
         (match List.assoc_opt "reasoning_content" f with
          | Some (`String "thinking rea") -> true
          | _ -> false)
     | _ -> false);
  Alcotest.(check string) "concatenated text" "hello  world" r.text;
  Alcotest.(check int) "two text deltas fired" 2 (List.length !deltas);
  Alcotest.(check (list string)) "delta order" [ "hello "; " world" ]
    !deltas;
  Alcotest.(check string) "stop reason" "tool_calls" r.stop_reason;
  Alcotest.(check int) "one reconstructed tool call" 1 (List.length
    r.calls);
  (match r.calls with
   | [ tc ] ->
```

```
      Alcotest.(check string) "tool id" "t1" tc.id;
      Alcotest.(check string) "tool name" "bash" tc.name
  | _ -> ());
  let input =
    match r.calls with
    | [ tc ] -> tc.input
    | _ -> `Null
  in
  Alcotest.(check string) "tool input json" {|{"command":"echo hello"}|}
    (Yojson.Safe.to_string input)

(** agent-route-table: (route ...) makes a path resolvable via
    route/lookup and renderable via route/dispatch; dispatch runs the
    Nopales handler through Runtime.apply_value under the eval rim and
    returns the rendered raw-HTML VStr. *)
let test_route_table_register_and_render ~rt () =
  ignore (Runtime.eval_string ~rt
    {|(route "/chat" "chat/render")|});
  ignore (Runtime.eval_string ~rt
    {|(define (chat/render) (render-html "<h1>hi</h1>"))|});
  Alcotest.check pp_value_testable "lookup resolves"
    (Ast.VStr "chat/render")
    (Runtime.eval_string ~rt {|(route/lookup "/chat")|});
  Alcotest.check pp_value_testable "dispatch renders raw-HTML VStr"
    (Ast.VStr "<h1>hi</h1>")
    (Runtime.eval_string ~rt
       {|(route/dispatch "chat/render" "GET" "/chat")|})

(** route/dispatch binds a thin request context so query-param /
    request-method / hx-request? resolve inside the handler. *)
let test_route_table_thin_context ~rt () =
  ignore (Runtime.eval_string ~rt
    {|(define (qtest)
       (string-append (request-method) "|"
         (query-param "x") "|"
         (if (hx-request?) "hx" "nohx")))|});
  ignore (Runtime.eval_string ~rt {|(route "/q" "qtest")|});
  Alcotest.check pp_value_testable "thin context resolves"
    (Ast.VStr "POST|42|hx")
    (Runtime.eval_string ~rt
       {|(route/dispatch "qtest" "POST" "/q"
           (list (cons "x" "42") (cons "hx-request" "true")))|})

(** a handler calling a Postgres-only request prim (current-author as
    a DB row) errors with unbound in the agent -- the agent's request
    context carries only request-shape values. *)
let test_route_table_pg_prim_unbound ~rt () =
  ignore (Runtime.eval_string ~rt
    {|(define (authtest) (current-author))|});
  ignore (Runtime.eval_string ~rt {|(route "/a" "authtest")|});
  (match Runtime.eval_string ~rt
           {|(route/dispatch "authtest" "GET" "/a")|}
   with
   | Ast.VStr s ->
       Alcotest.(check bool) "unbound reported" true
         (try ignore (Str.search_forward (Str.regexp_string "unbound") s
    0); true
          with Not_found -> false)
   | other -> Alcotest.fail ("expected VStr, got " ^ Reader.pp_value other))

(** libs/hyper runs unchanged on the agent: boot-load libs/std +
    libs/html + libs/hyper as source, register a route-table handler, and
    exercise hyper/visit + hyper/click (immutability) + hyper/stream
    (multipart replay). The portability proof: same lib.pp, agent host. *)
let test_libs_hyper_on_agent ~rt () =
  (* dune runs the test with cwd = agent/test/, so walk up to find the
     repo root (where libs/hyper/lib.pp lives). *)
  let rec find_libs_dir dir =
    let cand = Filename.concat dir "libs/hyper/lib.pp" in
    if Sys.file_exists cand then dir
    else
      let parent = Filename.dirname dir in
      if parent = dir then "libs"
```

```
      else find_libs_dir parent
  in
  let libs_dir =
    let root = find_libs_dir (Sys.getcwd ()) in
    Filename.concat root "libs"
  in
  let load name =
    let p = Filename.concat (Filename.concat libs_dir name) "lib.pp" in
    match Runtime.load_file ~rt p with
    | Ok () -> ()
    | Error e -> Alcotest.fail (name ^ ": " ^ e)
  in
  List.iter load ["html"; "hyper"];
  (* Register route-table handlers (Nopales). *)
  ignore (Runtime.eval_string ~rt
    {|(define (home)
       (render-html "<div><button id=b hx-get=/click
    hx-target=#out>go</button><div id=out>old</div></div>"))|});
  ignore (Runtime.eval_string ~rt {|(route "/" "home")|});
  ignore (Runtime.eval_string ~rt {|(define (clicker) (render-html
     "<b>new</b>"))|});
  ignore (Runtime.eval_string ~rt {|(route "/click" "clicker")|});
  (* hyper/visit resolves the route in-process and parses the page. *)
  Alcotest.check pp_value_testable "visit #out text"
    (Ast.VStr "old")
    (Runtime.eval_string ~rt {|(hyper/text (hyper/visit "/") "#out")|});
  (* hyper/click swaps #out; original page is unchanged (immutability). *)
  ignore (Runtime.eval_string ~rt {|(define pg (hyper/visit "/"))|});
  Alcotest.check pp_value_testable "click swaps #out"
    (Ast.VStr "new")
    (Runtime.eval_string ~rt {|(hyper/text (hyper/click pg "#b" (list))
     "#out")|});
  Alcotest.check pp_value_testable "source page unchanged"
    (Ast.VStr "old")
    (Runtime.eval_string ~rt {|(hyper/text pg "#out")|});
  (* hyper/stream replays an ordered part list onto a fresh copy. *)
  Alcotest.check pp_value_testable "stream replay #out"
    (Ast.VStr "new")
    (Runtime.eval_string ~rt
       {|(hyper/text
          (hyper/stream pg
            (list (list (cons "html" "<b>new</b>")
                        (cons "target" "#out")
                        (cons "swap" "innerHTML"))))
          "#out")|});
  Alcotest.check pp_value_testable "stream keeps source"
    (Ast.VStr "old")
    (Runtime.eval_string ~rt {|(hyper/text pg "#out")|})

(** surfaces: the chat + code pages are Nopales route-table handlers
    (rendered via libs/html), redefinable live; a consumed turn's captured
    parts replay through hyper/stream. Loads the embedded surface the
    same way the binary boots. *)
let test_surfaces_nopales_routes ~rt () =
  (* Boot-load the surface (libs/html + libs/hyper + chat/code handlers). *)
  Surface.load ~rt;
  (* GET / renders the chat page skeleton with the htmx form + assets. *)
  (match Runtime.dispatch_route ~rt ~handler:"chat/page" ~method_:"GET"
           ~path:"/" ~params:[] ~headers:[] ~body:""
   with
   | Ast.VStr html ->
       Alcotest.(check bool) "chat page has hx-post /chat" true
         (String.length html > 0
          && (try ignore (Str.search_forward (Str.regexp_string
      "hx-post=\"/chat\"") html 0); true
              with Not_found -> false));
       Alcotest.(check bool) "chat page loads hx-multipart asset" true
         ((try ignore (Str.search_forward (Str.regexp_string
      "/assets/hx-multipart.js") html 0); true
           with Not_found -> false))
   | other -> Alcotest.fail ("chat/page: " ^ Reader.pp_value other));
  (* GET /code renders the code browser page (root listing). *)
  (match Runtime.dispatch_route ~rt ~handler:"code/page" ~method_:"GET"
```

```
480             ~path:"/code" ~params:[] ~headers:[] ~body:""
481     with
482     | Ast.VStr html ->
483         Alcotest.(check bool) "code page renders" true (String.length html >
          0)
484     | other -> Alcotest.fail ("code/page: " ^ Reader.pp_value other));
485   (* Redefining chat/page live changes the next render (redefinable
486      without restart). *)
487   ignore (Runtime.eval_string ~rt
488     {|(define (chat/page) (render-html "<h1>redefined</h1>"))|});
489   (match Runtime.dispatch_route ~rt ~handler:"chat/page" ~method_:"GET"
490             ~path:"/" ~params:[] ~headers:[] ~body:"" with
491     | Ast.VStr s ->
492         Alcotest.(check bool) "live redef took effect" true
493           ((try ignore (Str.search_forward (Str.regexp_string "redefined") s
          0); true
494             with Not_found -> false))
495     | other -> Alcotest.fail ("redef: " ^ Reader.pp_value other))
496
497 (** A consumed turn's captured parts replay through hyper/stream: simulate
498     the multipart emitter's captured part list, then hyper/stream over a
499     page that has #log. *)
500 let test_turn_replay_hyper_stream ~rt () =
501   Surface.load ~rt;
502   (* Build a page with #log (what the chat page skeleton renders). *)
503   ignore (Runtime.eval_string ~rt
504     {|(define pg (soup/parse "<div id=log><div>ready</div></div>"))|});
505   (* Synthesize the part list the /chat emitter would capture: a user
506      bubble into #log (beforeend) + an llm bubble into #log (beforeend) +
507      a token span into the llm bubble (beforeend). *)
508   ignore (Runtime.eval_string ~rt
509     {|(define parts (list
510         (list (cons "html" "<div class=bubble>you: hi</div>") (cons
          "target" "#log") (cons "swap" "beforeend"))
511         (list (cons "html" "<div class=bubble id=llm-1></div>") (cons
          "target" "#log") (cons "swap" "beforeend"))
512         (list (cons "html" "<span>hello</span>") (cons "target" "#llm-1")
          (cons "swap" "beforeend"))))|});
513   Alcotest.check pp_value_testable "replay swaps #log"
514     (Ast.VStr "<div id=\"log\"><div>ready</div><div class=\"bubble\">you:
          hi</div><div class=\"bubble\"
          id=\"llm-1\"><span>hello</span></div></div>")
515     (Runtime.eval_string ~rt {|(soup/serialize (soup/select-one
          (hyper/stream pg parts) "#log"))|});
516   Alcotest.check pp_value_testable "replay token lands in bubble"
517     (Ast.VStr "<div class=\"bubble\" id=\"llm-1\"><span>hello</span></div>")
518     (Runtime.eval_string ~rt {|(soup/serialize (soup/select-one
          (hyper/stream pg parts) "#llm-1"))|});
519   (* Source page unchanged (immutability). *)
520   Alcotest.check pp_value_testable "source page unchanged"
521     (Ast.VStr "<div id=\"log\"><div>ready</div></div>")
522     (Runtime.eval_string ~rt {|(soup/serialize (soup/select-one pg
          "#log"))|})
523
524 (** Nested-block replay: the /chat emitter wraps each contiguous reasoning
525     run and each tool call in its OWN <details>/<summary> inside the turn's
526     "thinking + tools" zone (#<rid>-think). Build the capture-ordered part
527     list with the REAL templates (same calls the drain loop makes), replay
528     through hyper/stream, and assert the nested DOM. *)
529 let test_turn_replay_nested_blocks ~rt () =
530   Surface.load ~rt;
531   (* Build the part list OCaml-side via Runtime.call_template -- the same
532      producer path the drain loop uses (template output unwrapped from
533      its raw-html sentinel), bound into the env for hyper/stream. *)
534   let tpl name args =
535     match Runtime.call_template ~rt name (List.map (fun s -> Ast.VStr s)
          args) with
536     | Some html -> html
537     | None -> Alcotest.fail ("template missing: " ^ name)
538   in
539   let part html target swap : Ast.value =
540     Ast.VList
541       [ Ast.VPair (Ast.VStr "html", Ast.VStr html)
```

```
      ; Ast.VPair (Ast.VStr "target", Ast.VStr target)
      ; Ast.VPair (Ast.VStr "swap", Ast.VStr swap)
      ]
  in
  let parts =
    Ast.VList
      [ part (tpl "chat/turn-open" [ "t1" ]) "#log" "beforeend"
      ; part (tpl "chat/think-open" [ "t1-tk-1" ]) "#t1-think" "beforeend"
      ; part (tpl "chat/think-token" [ "rea" ]) "#t1-tk-1" "beforeend"
      ; part (tpl "chat/tool-open" [ "t1-tool-1"; "bash" ]) "#t1-think"
    "beforeend"
      ; part (tpl "chat/tool-line" [ "in"; "ls -la" ]) "#t1-tool-1"
    "beforeend"
      ; part (tpl "chat/tool-line" [ "out"; "file.ml" ]) "#t1-tool-1"
    "beforeend"
      ; part (tpl "chat/tool-done" [ "bash" ]) "#t1-tool-1-sum" "innerHTML"
      ; part (tpl "chat/think-open" [ "t1-tk-2" ]) "#t1-think" "beforeend"
      ; part (tpl "chat/think-token" [ "more" ]) "#t1-tk-2" "beforeend"
      ; part (tpl "chat/token" [ "answer" ]) "#t1" "beforeend"
      ]
  in
  ignore (Runtime.eval_string ~rt
    {|(define npg (soup/parse "<div id=log></div>"))|});
  Env.define rt.Runtime.env "nparts" parts;
  Alcotest.check pp_value_testable "nested blocks replay into zone"
    (Ast.VStr
       ("<div id=\"t1-think\">"
        ^ "<details class=\"tk\"><summary>thinking</summary><div
    id=\"t1-tk-1\"><span class=\"reason\">rea</span></div></details>"
        ^ "<details class=\"tool\"><summary id=\"t1-tool-1-sum\">tool: bash
    ?</summary><div id=\"t1-tool-1\"><div class=\"trace\">in ls
    -la</div><div class=\"trace\">out file.ml</div></div></details>"
        ^ "<details class=\"tk\"><summary>thinking</summary><div
    id=\"t1-tk-2\"><span class=\"reason\">more</span></div></details>"
        ^ "</div>"))
    (Runtime.eval_string ~rt
       {|(soup/serialize (soup/select-one (hyper/stream npg nparts)
    "#t1-think"))|});
  Alcotest.check pp_value_testable "reply bubble got tokens"
    (Ast.VStr "<div class=\"bubble llm\"
    id=\"t1\"><span>answer</span></div>")
    (Runtime.eval_string ~rt
       {|(soup/serialize (soup/select-one (hyper/stream npg nparts)
    "#t1"))|})

let () =
  Eio_main.run @@ fun env ->
  let rt = Runtime.make () in
  Tools.install ~resolve_path:Runtime.resolve_path ~register:(fun name run
     -> Runtime.register_builtin_tool rt name ~run);
  Eio.Switch.run @@ fun sw ->
  Alcotest.run "agent"
    [
      ( "plugin-model",
        [ Alcotest.test_case "lifecycle" `Quick (test_plugin_lifecycle ~rt)
    ] );
      ( "nopales-embed",
        [ Alcotest.test_case "rim-errors-redefine" `Quick
            (test_rim_errors_and_redefine ~rt) ] );
      ( "agent-loop",
        [
          Alcotest.test_case "tool-round-trip" `Quick
            (test_loop_tool_round_trip ~env ~sw ~rt);
          Alcotest.test_case "tool-block" `Quick
            (test_loop_tool_block ~env ~sw ~rt);
          Alcotest.test_case "turn2-history-shape" `Quick
            (test_loop_turn2_history_shape ~env ~sw ~rt);
        ] );
      ( "provider-stream",
        [
          Alcotest.test_case "sse-text-and-tool-deltas" `Quick
            test_sse_stream_reconstruction;
        ] );
```

```
603       ( "agent-route-table",
604         [ Alcotest.test_case "register-and-render" `Quick
605             (test_route_table_register_and_render ~rt)
606         ; Alcotest.test_case "thin-context" `Quick
607             (test_route_table_thin_context ~rt)
608         ; Alcotest.test_case "pg-prim-unbound" `Quick
609             (test_route_table_pg_prim_unbound ~rt)
610         ] );
611       ( "libs-hyper-on-agent",
612         [ Alcotest.test_case "visit-click-stream" `Quick
613             (test_libs_hyper_on_agent ~rt) ] );
614       ( "surfaces",
615         [ Alcotest.test_case "nopales-routes" `Quick
616             (test_surfaces_nopales_routes ~rt)
617         ; Alcotest.test_case "turn-replay-hyper-stream" `Quick
618             (test_turn_replay_hyper_stream ~rt)
619         ; Alcotest.test_case "turn-replay-nested-blocks" `Quick
620             (test_turn_replay_nested_blocks ~rt) ] );
621     ]
```

41 agent/test/load_probe.ml

```
open Pricklypear_lisp
let () =
  Eio_main.run @@ fun _env ->
  let rt = Runtime.make () in
  (match Runtime.load_file ~rt "libs/std/lib.pp" with
   | Ok () -> print_endline "std OK"
   | Error e -> print_endline ("std ERR: " ^ e))
```

42 cli/bin/pp_eval.ml

```
(** pp-eval -- eval Lisp on a live Pricklypear image (OCaml CLI, no Python).

    The image is the API: POST /api/eval with HTTP basic auth.

      export PP_URL=https://pricklypear.rocks
      export PP_USER=admin PP_PASS=?
      pp-eval '(+ 1 2)'
      pp-eval -j '(whoami)'
      pp-eval -e '(load-library "todo")' -e '(count (todo/list))'
      pp-eval -i
*)

open Cmdliner

let env_url =
  Pp_client.getenv_default "PP_URL" "http://127.0.0.1:18080"

let env_user = Pp_client.getenv_default "PP_USER" "admin"

let env_timeout =
  match Pp_client.getenv "PP_EVAL_TIMEOUT" with
  | Some s -> (try float_of_string s with _ -> 120.)
  | None -> 120.

let print_result ~(as_json : bool) (r : Pp_client.eval_result) : int =
  if as_json then begin
    let j =
      match r.ok, r.value, r.error with
      | true, Some v, _ ->
          `Assoc [ ("ok", `Bool true); ("value", `String v) ]
      | true, None, _ -> `Assoc [ ("ok", `Bool true); ("value", `Null) ]
      | false, _, Some e ->
          `Assoc [ ("ok", `Bool false); ("error", `String e) ]
      | false, _, None ->
          `Assoc
            [
              ("ok", `Bool false);
              ("error", `String (if r.raw = "" then "unknown error" else
    r.raw));
            ]
    in
    print_endline (Yojson.Safe.to_string j);
    if r.ok then 0 else 1
  end else if r.ok then begin
    (match r.value with Some v -> print_endline v | None -> ());
    0
  end else begin
    let msg =
      match r.error with Some e -> e | None -> "unknown error"
    in
    prerr_endline msg;
    1
  end

let run_one ~url ~user ~password ~timeout ~as_json expr =
  let r =
    Pp_client.eval_expr ~base_url:url ~user ~password ~expr
      ~timeout_s:timeout
  in
  print_result ~as_json r

let interactive ~url ~user ~password ~timeout ~as_json =
  prerr_endline (Printf.sprintf "[pp-eval] %s  user=%s" url user);
  prerr_endline "[pp-eval] enter Lisp forms; :quit to exit";
  let rec loop rc =
    try
      output_string stdout "pp> ";
      flush stdout;
      let line = input_line stdin |> String.trim in
      if line = "" then loop rc
      else if
```

```
        line = ":quit" || line = ":q" || line = "quit" || line = "exit"
      then rc
      else
        let code =
          run_one ~url ~user ~password ~timeout ~as_json line
        in
        loop (if code <> 0 then code else rc)
    with End_of_file ->
      prerr_newline ();
      rc
  in
  loop 0

let read_stdin_exprs () =
  if Unix.isatty Unix.stdin then []
  else
    let rec go acc =
      try
        let line = input_line stdin |> String.trim in
        if line = "" || (String.length line > 0 && line.[0] = ';') then
          go acc
        else go (line :: acc)
      with End_of_file -> List.rev acc
    in
    go []

let main url user password password_file timeout as_json interactive_flag
    evals expr_opt =
  let password =
    Pp_client.resolve_password ~password ~password_file
  in
  if password = "" then begin
    prerr_endline "pp-eval: set PP_PASS or PP_PASS_FILE or --password";
    2
  end else if interactive_flag then
    interactive ~url ~user ~password ~timeout ~as_json
  else
    let exprs = List.rev evals in
    let exprs =
      match expr_opt with Some e -> exprs @ [ e ] | None -> exprs
    in
    let exprs =
      if exprs = [] then read_stdin_exprs () else exprs
    in
    if exprs = [] then begin
      prerr_endline
        "pp-eval: pass an expression, -e EXPR, stdin, or -i";
      2
    end else
      List.fold_left
        (fun rc expr ->
          let code =
            run_one ~url ~user ~password ~timeout ~as_json expr
          in
          if code <> 0 then code else rc)
        0 exprs

(* ---- cmdliner ---- *)

let url =
  let doc = "Image base URL (default PP_URL or http://127.0.0.1:18080)." in
  Arg.(value & opt string env_url & info [ "url" ] ~docv:"URL" ~doc)

let user =
  let doc = "HTTP basic user (default PP_USER or admin)." in
  Arg.(value & opt string env_user & info [ "user" ] ~docv:"USER" ~doc)

let password =
  let doc = "HTTP basic password (or PP_PASS / PP_PASS_FILE)." in
  Arg.(value & opt string "" & info [ "password"; "p" ] ~docv:"PASS" ~doc)

let password_file =
  let doc = "Read password from file." in
```

```
  Arg.(
    value & opt string ""
    & info [ "password-file" ] ~docv:"PATH" ~doc)

let timeout =
  let doc = "HTTP timeout seconds (default PP_EVAL_TIMEOUT or 120)." in
  Arg.(value & opt float env_timeout & info [ "timeout" ] ~docv:"SEC" ~doc)

let as_json =
  let doc = "Print full JSON result {ok,value|error} (agent-friendly)." in
  Arg.(value & flag & info [ "j"; "json" ] ~doc)

let interactive_flag =
  let doc = "Interactive multi-turn session." in
  Arg.(value & flag & info [ "i"; "interactive" ] ~doc)

let evals =
  let doc = "Expression to eval (repeatable, in order)." in
  Arg.(
    value & opt_all string []
    & info [ "e"; "eval" ] ~docv:"EXPR" ~doc)

let expr_opt =
  let doc = "Single Lisp expression." in
  Arg.(value & pos 0 (some string) None & info [] ~docv:"EXPR" ~doc)

let term =
  Term.(
    const main $ url $ user $ password $ password_file $ timeout
    $ as_json $ interactive_flag $ evals $ expr_opt)

let cmd =
  let doc = "Eval Lisp on a live Pricklypear image (no Python)." in
  let info = Cmd.info "pp-eval" ~doc in
  Cmd.v info term

let () = exit (Cmd.eval' cmd)
```

43 cli/bin/pp_sync.ml

```
(** pp-sync -- sync libs/*.pp with live function_defs (OCaml, no Python).

      pp-sync status [pkg]
      pp-sync pull <pkg>
      pp-sync push <pkg>
      pp-sync pull-all
      pp-sync reseed <pkg>          admin: upsert libs ? system/main
      pp-sync snapshot [--branch R] [--out DIR]
      pp-sync land <pkg> [name ...] [--from owner/branch]
*)

open Cmdliner

let repo_root () =
  (* Prefer cwd if it looks like the repo; else dirname of executable. *)
  let cwd = Sys.getcwd () in
  if Sys.file_exists (Filename.concat cwd "libs") then cwd
  else
    let exe = Sys.executable_name in
    let dir = Filename.dirname exe in
    (* dune install: ?/bin ? ?; dune exec: _build/default/cli/bin *)
    let candidates =
      [
        Filename.concat dir "../../..";
        Filename.concat dir "../../../..";
        cwd;
      ]
    in
    match
      List.find_opt
        (fun p -> Sys.file_exists (Filename.concat p "libs"))
        candidates
    with
    | Some p -> p
    | None -> cwd

let libs_dir () = Filename.concat (repo_root ()) "libs"

let pkg_path pkg =
  if pkg = "" || pkg = "(root)" || pkg = "root" then
    Filename.concat (Filename.concat (libs_dir ()) "_root") "lib.pp"
  else Filename.concat (Filename.concat (libs_dir ()) pkg) "lib.pp"

let read_file path =
  let ic = open_in_bin path in
  Fun.protect
    ~finally:(fun () -> close_in_noerr ic)
    (fun () -> really_input_string ic (in_channel_length ic))

let rec mkdir_p path =
  if path = "" || path = "." || path = "/" then ()
  else if Sys.file_exists path then ()
  else begin
    mkdir_p (Filename.dirname path);
    try Unix.mkdir path 0o755
    with Unix.Unix_error (Unix.EEXIST, _, _) -> ()
  end

let write_file path text =
  let dir = Filename.dirname path in
  mkdir_p dir;
  let oc = open_out_bin path in
  Fun.protect
    ~finally:(fun () -> close_out_noerr oc)
    (fun () ->
      output_string oc text;
      if text = "" || text.[String.length text - 1] <> '\n' then
        output_char oc '\n')

let sanitize_ref ref_ =
  let buf = Buffer.create (String.length ref_) in
```

```
  String.iter
    (fun c ->
      match c with
      | 'a' .. 'z' | 'A' .. 'Z' | '0' .. '9' | '-' | '_' | '.' ->
          Buffer.add_char buf c
      | '/' | ' ' -> Buffer.add_char buf '-'
      | _ -> Buffer.add_char buf '_')
    ref_;
  let s = Buffer.contents buf in
  if s = "" then "branch" else s

let parse_local_defines text =
  let strip_comments s =
    let buf = Buffer.create (String.length s) in
    let in_str = ref false in
    let i = ref 0 in
    let n = String.length s in
    while !i < n do
      let c = s.[!i] in
      if c = '"' && (!i = 0 || s.[!i - 1] <> '\\') then begin
        in_str := not !in_str;
        Buffer.add_char buf c;
        incr i
      end else if c = ';' && not !in_str then begin
        while !i < n && s.[!i] <> '\n' do
          incr i
        done
      end else begin
        Buffer.add_char buf c;
        incr i
      end
    done;
    Buffer.contents buf
  in
  let blob = strip_comments text in
  let re = Re.Perl.compile_pat {|\(\s*define\s+([^\s()]+)|} in
  Re.all re blob
  |> List.filter_map (fun g ->
         try Some (Re.Group.get g 1) with Not_found -> None)
  |> List.sort_uniq String.compare

let unwrap_list_of_strings (printed : string) : string list =
  let re = Re.Perl.compile_pat {|"([^"]+)"|} in
  let quoted =
    Re.all re printed
    |> List.filter_map (fun g ->
           try Some (Re.Group.get g 1) with Not_found -> None)
  in
  if quoted <> [] then quoted
  else
    let inner = String.trim printed in
    let inner =
      if
        String.length inner >= 2
        && inner.[0] = '('
        && inner.[String.length inner - 1] = ')'
      then String.sub inner 1 (String.length inner - 2) |> String.trim
      else inner
    in
    if inner = "" then []
    else String.split_on_char ' ' inner |> List.filter (( <> ) "")

let live ~url ~user ~password ~timeout expr =
  Pp_client.eval_expr ~base_url:url ~user ~password ~expr
    ~timeout_s:timeout

let live_value r =
  match r with
  | { Pp_client.ok = true; value = Some v; _ } ->
      Ok (Pp_client.unwrap_lisp_string v)
  | { ok = false; error = Some e; _ } -> Error e
  | _ -> Error "empty result"

```

```
let discover_packages ~url ~user ~password ~timeout =
  let pkgs = Hashtbl.create 32 in
  let ld = libs_dir () in
  if Sys.file_exists ld then
    Array.iter
      (fun name ->
        let p = Filename.concat ld name in
        if
          Sys.is_directory p
          && name <> "_root"
          && Sys.file_exists (Filename.concat p "lib.pp")
        then Hashtbl.replace pkgs name ())
      (Sys.readdir ld);
  (match live ~url ~user ~password ~timeout "(list-bindings)" |> live_value
     with
   | Ok s ->
       List.iter
         (fun name ->
           match String.rindex_opt name '/' with
           | Some i ->
               Hashtbl.replace pkgs (String.sub name 0 i) ()
           | None -> ())
         (unwrap_list_of_strings s)
   | Error _ -> ());
  Hashtbl.fold (fun k () acc -> k :: acc) pkgs [] |> List.sort
    String.compare

let cmd_status ~url ~user ~password ~timeout ~verbose package =
  let pkgs =
    if package <> "" then [ package ]
    else discover_packages ~url ~user ~password ~timeout
  in
  let rc = ref 0 in
  List.iter
    (fun pkg ->
      let path = pkg_path pkg in
      let local =
        if Sys.file_exists path then parse_local_defines (read_file path)
        else []
      in
      let local_set = List.fold_left (fun s x -> x :: s) [] local in
      match
        live ~url ~user ~password ~timeout
          (Printf.sprintf "(package-bindings %S)" pkg)
        |> live_value
      with
      | Error e ->
          Printf.eprintf "%s: live error: %s\n%!" pkg e;
          rc := 1
      | Ok s ->
          let live_names = unwrap_list_of_strings s in
          let only_local =
            List.filter (fun n -> not (List.mem n live_names)) local_set
            |> List.sort_uniq String.compare
          in
          let only_live =
            List.filter (fun n -> not (List.mem n local_set)) live_names
            |> List.sort_uniq String.compare
          in
          let both =
            List.filter (fun n -> List.mem n live_names) local_set
            |> List.sort_uniq String.compare
          in
          Printf.printf "== %s ==\n" (if pkg = "" then "(root)" else pkg);
          Printf.printf "  local file: %s\n"
            (if Sys.file_exists path then path else "(missing)");
          Printf.printf
            "  both: %d  only-local: %d  only-live: %d\n"
            (List.length both) (List.length only_local)
            (List.length only_live);
          if only_local <> [] then
            Printf.printf "  only-local: %s\n"
              (String.concat ", " only_local);
```

```
          if only_live <> [] then
            Printf.printf "  only-live: %s\n" (String.concat ", "
     only_live);
          if verbose && both <> [] then
            Printf.printf "  both: %s\n" (String.concat ", " both);
          if only_local <> [] || only_live <> [] then rc := 1)
    pkgs;
  !rc

let cmd_pull ~url ~user ~password ~timeout package =
  if package = "" then begin
    prerr_endline "pull: need package name";
    2
  end else
    match
      live ~url ~user ~password ~timeout
        (Printf.sprintf "(export-package %S)" package)
      |> live_value
    with
    | Error e ->
        prerr_endline e;
        1
    | Ok text ->
        let path = pkg_path package in
        write_file path text;
        Printf.printf "wrote %s (%d bytes)\n" path (String.length text);
        0

let cmd_push ~url ~user ~password ~timeout package =
  if package = "" then begin
    prerr_endline "push: need package name";
    2
  end else
    let path = pkg_path package in
    if not (Sys.file_exists path) then begin
      Printf.eprintf "push: missing %s\n" path;
      1
    end else
      let src = read_file path in
      (* JSON-encode as Lisp string literal for load-string *)
      let lit = Yojson.Safe.to_string (`String src) in
      match
        live ~url ~user ~password ~timeout
          (Printf.sprintf "(load-string %s)" lit)
        |> live_value
      with
      | Error e ->
          prerr_endline e;
          1
      | Ok msg ->
          print_endline msg;
          Printf.printf "pushed %s ? live via load-string\n" path;
          0

let cmd_pull_all ~url ~user ~password ~timeout =
  let pkgs = discover_packages ~url ~user ~password ~timeout in
  List.fold_left
    (fun rc pkg ->
      Printf.printf "--- pull %s ---\n" pkg;
      let code = cmd_pull ~url ~user ~password ~timeout pkg in
      if code <> 0 then code else rc)
    0 pkgs

let cmd_reseed ~url ~user ~password ~timeout package =
  if package = "" then begin
    prerr_endline "reseed: need package name";
    2
  end else
    match
      live ~url ~user ~password ~timeout
        (Printf.sprintf "(reseed-package %S)" package)
      |> live_value
    with
```

```
    | Error e ->
        prerr_endline e;
        1
    | Ok msg ->
        print_endline msg;
        0

let pkg_path_under root pkg =
  if pkg = "" || pkg = "(root)" || pkg = "root" then
    Filename.concat (Filename.concat root "_root") "lib.pp"
  else Filename.concat (Filename.concat root pkg) "lib.pp"

let cmd_snapshot ~url ~user ~password ~timeout ~branch ~out_dir =
  let prev_branch =
    match live ~url ~user ~password ~timeout "(current-branch)" |>
     live_value with
    | Ok s -> String.trim s
    | Error _ -> ""
  in
  let target =
    if branch <> "" then branch
    else if prev_branch <> "" then prev_branch
    else "system/main"
  in
  let out_root =
    if out_dir <> "" then
      if Filename.is_relative out_dir then
        Filename.concat (repo_root ()) out_dir
      else out_dir
    else if target = "system/main" || target = "main" then libs_dir ()
    else
      Filename.concat
        (Filename.concat (repo_root ()) "backups")
        (sanitize_ref target)
  in
  (* Optional checkout to target branch for export visibility. *)
  let checkout_rc =
    if target = prev_branch || target = "" then 0
    else
      match
        live ~url ~user ~password ~timeout
          (Printf.sprintf "(checkout %S)" target)
        |> live_value
      with
      | Error e ->
          prerr_endline ("snapshot checkout: " ^ e);
          1
      | Ok msg ->
          if String.trim msg = "checked out" then 0
          else begin
            prerr_endline ("snapshot checkout: " ^ msg);
            1
          end
  in
  if checkout_rc <> 0 then checkout_rc
  else begin
    Printf.printf "snapshot branch=%s ? %s\n%!" target out_root;
    let pkgs = discover_packages ~url ~user ~password ~timeout in
    let rc =
      List.fold_left
        (fun rc pkg ->
          match
            live ~url ~user ~password ~timeout
              (Printf.sprintf "(export-package %S)" pkg)
            |> live_value
          with
          | Error e ->
              Printf.eprintf "%s: %s\n%!" pkg e;
              if rc <> 0 then rc else 1
          | Ok text ->
              if String.trim text = "" then rc
              else begin
                let path = pkg_path_under out_root pkg in
```

```
                write_file path text;
                Printf.printf "  wrote %s (%d bytes)\n%!" path
                  (String.length text);
                rc
              end)
        0 pkgs
    in
    (* Restore previous branch when we switched. *)
    (if prev_branch <> "" && prev_branch <> target then
       ignore
         (live ~url ~user ~password ~timeout
            (Printf.sprintf "(checkout %S)" prev_branch)));
    rc
  end


let git_run ~cwd args : (int * string) =
  let prev = Sys.getcwd () in
  let restore () = try Sys.chdir prev with _ -> () in
  try
    Sys.chdir cwd;
    let stdout_path = Filename.temp_file "pp_sync_git" ".out" in
    let q a =
      if
        String.exists
          (function ' ' | '"' | '\'' | '\\' -> true | _ -> false)
          a
      then Filename.quote a
      else a
    in
    let cmdline =
      String.concat " " (List.map q ("git" :: args))
      ^ " >"
      ^ Filename.quote stdout_path
      ^ " 2>&1"
    in
    let code = Sys.command cmdline in
    let out =
      if Sys.file_exists stdout_path then read_file stdout_path else ""
    in
    (try Sys.remove stdout_path with _ -> ());
    restore ();
    (code, String.trim out)
  with exn ->
    restore ();
    (127, Printexc.to_string exn)

let checkout_ref ~url ~user ~password ~timeout ref_ =
  match
    live ~url ~user ~password ~timeout (Printf.sprintf "(checkout %S)" ref_)
    |> live_value
  with
  | Error e -> Error e
  | Ok msg ->
      if String.trim msg = "checked out" then Ok () else Error msg

let current_branch ~url ~user ~password ~timeout =
  match live ~url ~user ~password ~timeout "(current-branch)" |> live_value
     with
  | Ok s -> String.trim s
  | Error _ -> ""

let starts_with prefix s =
  let n = String.length prefix in
  String.length s >= n && String.sub s 0 n = prefix

(** pp-sync land <pkg> [name...] [--from owner/branch]
    1 resolve names on source
    2 promote-to-main
    3 export-package from system/main ? libs/<pkg>/lib.pp
    4 git add + commit
    5 report SHA
    Never pushes; never unoverrides. Fail-fast per step. *)
```

```
let cmd_land ~url ~user ~password ~timeout ~from_ref ~package ~names =
  if package = "" then begin
    prerr_endline "land: need package name";
    2
  end else
    let prev = current_branch ~url ~user ~password ~timeout in
    let source =
      if from_ref <> "" then from_ref
      else if prev <> "" then prev
      else "admin/personal"
    in
    let restore () =
      if prev <> "" then
        ignore (checkout_ref ~url ~user ~password ~timeout prev)
    in
    Printf.printf "land: pkg=%s from=%s\n%!" package source;
    match checkout_ref ~url ~user ~password ~timeout source with
    | Error e ->
        prerr_endline ("land step1 checkout source: " ^ e);
        1
    | Ok () ->
        let name_list_res =
          if names <> [] then Ok names
          else
            match
              live ~url ~user ~password ~timeout
                (Printf.sprintf "(package-bindings %S)" package)
              |> live_value
            with
            | Error e -> Error ("land step1 package-bindings: " ^ e)
            | Ok s ->
                let ns = unwrap_list_of_strings s in
                if ns = [] then
                  Error
                    (Printf.sprintf
                       "land step1: no bindings for package %S on %s"
    package
                       source)
                else Ok ns
        in
        match name_list_res with
        | Error e ->
            prerr_endline e;
            restore ();
            1
        | Ok name_list ->
            Printf.printf "land: promoting %d name(s): %s\n%!"
              (List.length name_list) (String.concat ", " name_list);
            let names_lit =
              name_list
              |> List.map (fun n -> Printf.sprintf "%S" n)
              |> String.concat " "
            in
            let promote_expr =
              Printf.sprintf "(promote-to-main %S %s)" source names_lit
            in
            match live ~url ~user ~password ~timeout promote_expr |>
    live_value with
            | Error e ->
                prerr_endline ("land step2 promote failed: " ^ e);
                restore ();
                1
            | Ok msg ->
                let msg = String.trim msg in
                if
                  starts_with "promote-to-main:" msg
                  || starts_with "promote:" msg
                  || not (starts_with "promoted " msg)
                then begin
                  prerr_endline ("land step2 promote denied/failed: " ^
    msg);
                  restore ();
                  1
```

```
              end else begin
                Printf.printf "land step2: %s\n%!" msg;
                match
                  checkout_ref ~url ~user ~password ~timeout "system/main"
                with
                | Error e ->
                    prerr_endline ("land step3 checkout system/main: " ^
    e);
                    restore ();
                    1
                | Ok () ->
                    match
                      live ~url ~user ~password ~timeout
                        (Printf.sprintf "(export-package %S)" package)
                      |> live_value
                    with
                    | Error e ->
                        prerr_endline ("land step3 export: " ^ e);
                        restore ();
                        1
                    | Ok text ->
                        let path = pkg_path package in
                        write_file path text;
                        Printf.printf "land step3: wrote %s (%d
    bytes)\n%!"
                          path (String.length text);
                        let root = repo_root () in
                        let rel = "libs/" ^ package ^ "/lib.pp" in
                        let names_msg =
                          if List.length name_list <= 6 then
                            String.concat " " name_list
                          else
                            Printf.sprintf "%d names" (List.length
    name_list)
                        in
                        let commit_msg =
                          Printf.sprintf "vault(%s): land %s" package
                            names_msg
                        in
                        let add_code, add_out =
                          git_run ~cwd:root [ "add"; "--"; rel ]
                        in
                        if add_code <> 0 then begin
                          prerr_endline
                            ("land step4 git add: " ^ add_out);
                          prerr_endline
                            "land: main promoted; vault written but not \
                             committed";
                          restore ();
                          1
                        end else
                          let diff_code, _ =
                            git_run ~cwd:root
                              [ "diff"; "--cached"; "--quiet" ]
                          in
                          if diff_code = 0 then begin
                            let _, sha =
                              git_run ~cwd:root
                                [ "rev-parse"; "--short"; "HEAD" ]
                            in
                            Printf.printf
                              "land OK: promoted [%s] ? system/main;
    vault \
                               %s; HEAD %s (no new commit)\n%!"
                              (String.concat ", " name_list)
                              rel
                              (if sha = "" then "?" else sha);
                            restore ();
                            0
                          end else
                            let c_code, c_out =
                              git_run ~cwd:root
                                [ "commit"; "-m"; commit_msg ]
```

```
                              in
                              if c_code <> 0 then begin
                                prerr_endline
                                  ("land step4 git commit: " ^ c_out);
                                prerr_endline
                                  "land: promote+vault done; commit failed";
                                restore ();
                                1
                              end else
                                let _, sha =
                                  git_run ~cwd:root
                                    [ "rev-parse"; "--short"; "HEAD" ]
                                in
                                Printf.printf
                                  "land OK: promoted [%s] ? system/main; \
                                   vault %s; commit %s\n%!"
                                  (String.concat ", " name_list)
                                  rel
                                  (if sha = "" then "?" else sha);
                                restore ();
                                0
                end

let env_url =
  Pp_client.getenv_default "PP_URL" "http://127.0.0.1:18080"

let env_user = Pp_client.getenv_default "PP_USER" "admin"

let env_timeout =
  match Pp_client.getenv "PP_EVAL_TIMEOUT" with
  | Some s -> (try float_of_string s with _ -> 180.)
  | None -> 180.

let main command package names verbose url user password password_file
    timeout
    branch out_dir from_ref =
  let password =
    Pp_client.resolve_password ~password ~password_file
  in
  if password = "" then begin
    prerr_endline "pp-sync: set PP_PASS or --password";
    2
  end else
    match command with
    | "status" ->
        cmd_status ~url ~user ~password ~timeout ~verbose package
    | "pull" -> cmd_pull ~url ~user ~password ~timeout package
    | "push" -> cmd_push ~url ~user ~password ~timeout package
    | "pull-all" -> cmd_pull_all ~url ~user ~password ~timeout
    | "reseed" -> cmd_reseed ~url ~user ~password ~timeout package
    | "snapshot" ->
        cmd_snapshot ~url ~user ~password ~timeout ~branch ~out_dir
    | "land" ->
        cmd_land ~url ~user ~password ~timeout ~from_ref ~package ~names
    | _ ->
        prerr_endline
          "unknown command
    (status|pull|push|pull-all|reseed|snapshot|land)";
        2

let command =
  let doc = "status | pull | push | pull-all | reseed | snapshot | land" in
  Arg.(
    required
    & pos 0 (some string) None
    & info [] ~docv:"COMMAND" ~doc)

let package =
  let doc = "Package name (ui, todo, ?)." in
  Arg.(value & pos 1 string "" & info [] ~docv:"PKG" ~doc)

let verbose =
  Arg.(value & flag & info [ "v"; "verbose" ] ~doc:"Verbose status.")
```

```
642
643 let url =
644   Arg.(value & opt string env_url & info [ "url" ] ~docv:"URL" ~doc:"Image
        base URL.")
645
646 let user =
647   Arg.(value & opt string env_user & info [ "user" ] ~docv:"USER"
        ~doc:"HTTP basic user.")
648
649 let password =
650   Arg.(value & opt string "" & info [ "password"; "p" ] ~docv:"PASS"
        ~doc:"Password.")
651
652 let password_file =
653   Arg.(value & opt string "" & info [ "password-file" ] ~docv:"PATH"
        ~doc:"Password file.")
654
655 let timeout =
656   Arg.(value & opt float env_timeout & info [ "timeout" ] ~docv:"SEC"
        ~doc:"Timeout seconds.")
657
658 let branch =
659   let doc = "Branch ref for snapshot (default: current; e.g. system/main)."
        in
660   Arg.(value & opt string "" & info [ "branch"; "b" ] ~docv:"REF" ~doc)
661
662 let out_dir =
663   let doc =
664     "Output dir for snapshot (default: libs/ for system/main, else
        backups/<ref>/)."
665   in
666   Arg.(value & opt string "" & info [ "out"; "o" ] ~docv:"DIR" ~doc)
667
668 let names =
669   let doc = "Function names for land (default: all package-bindings on
        source)." in
670   Arg.(value & pos_right 1 string [] & info [] ~docv:"NAME" ~doc)
671
672 let from_ref =
673   let doc = "Source branch for land (default: current session branch)." in
674   Arg.(value & opt string "" & info [ "from" ] ~docv:"OWNER/BRANCH" ~doc)
675
676 let term =
677   Term.(
678     const main $ command $ package $ names $ verbose $ url $ user $ password
679     $ password_file $ timeout $ branch $ out_dir $ from_ref)
680
681 let cmd =
682   let doc =
683     "Sync / snapshot / reseed / land Pricklypear packages (live image ?
        repo vault)."
684   in
685   Cmd.v (Cmd.info "pp-sync" ~doc) term
686
687 let () = exit (Cmd.eval' cmd)
```

44 cli/lib/pp_client.ml

```
(** Shared HTTP client for live-image /api/eval.

    Uses [curl] for HTTP and HTTPS (OpenBSD + laptop). No Python. *)

type eval_result = {
  ok : bool;
  value : string option;
  error : string option;
  raw : string;
}

let getenv name = Sys.getenv_opt name

let getenv_default name default =
  match getenv name with Some v when v <> "" -> v | _ -> default

let read_password_file path =
  let ic = open_in path in
  Fun.protect
    ~finally:(fun () -> close_in_noerr ic)
    (fun () -> String.trim (input_line ic))

let resolve_password ~password ~password_file =
  if password <> "" then password
  else if password_file <> "" then read_password_file password_file
  else
    match getenv "PP_PASS_FILE" with
    | Some p when p <> "" -> read_password_file p
    | _ -> getenv_default "PP_PASS" ""

let eval_url base =
  let base = String.trim base in
  let base =
    if base = "" then "http://127.0.0.1:18080"
    else if base.[String.length base - 1] = '/' then
      String.sub base 0 (String.length base - 1)
    else base
  in
  base ^ "/api/eval"

(** Run curl; return stdout body (and stderr on failure message). *)
let curl_post ~url ~user ~password ~body ~timeout_s : (string, string)
    Result.t =
  let timeout = max 1 (int_of_float (Float.ceil timeout_s)) in
  let args =
    [|
      "curl";
      "-sS";
      "-u";
      user ^ ":" ^ password;
      "-H";
      "Content-Type: application/json";
      "-H";
      "Accept: application/json";
      "--max-time";
      string_of_int timeout;
      "-d";
      body;
      url;
    |]
  in
  (* Prefer open_process_args when available (no shell). *)
  let stdout_path, stderr_path =
    let tmp = Filename.temp_file "pp-eval" "" in
    (tmp ^ ".out", tmp ^ ".err")
  in
  let cleanup () =
    (try Sys.remove stdout_path with _ -> ());
    (try Sys.remove stderr_path with _ -> ())
  in
  Fun.protect
```

```
    ~finally:cleanup
    (fun () ->
      let oc_out = open_out_bin stdout_path in
      let oc_err = open_out_bin stderr_path in
      let pid =
        Unix.create_process "curl" args Unix.stdin
          (Unix.descr_of_out_channel oc_out)
          (Unix.descr_of_out_channel oc_err)
      in
      close_out oc_out;
      close_out oc_err;
      let _, status = Unix.waitpid [] pid in
      let read_file p =
        if not (Sys.file_exists p) then ""
        else
          let ic = open_in_bin p in
          Fun.protect
            ~finally:(fun () -> close_in_noerr ic)
            (fun () -> really_input_string ic (in_channel_length ic))
      in
      let stdout = read_file stdout_path in
      let stderr = read_file stderr_path in
      match status with
      | Unix.WEXITED 0 -> Ok stdout
      | Unix.WEXITED n ->
          Error
            (Printf.sprintf "curl exit %d: %s%s" n
               (if stderr = "" then "" else stderr ^ " ")
               (if String.length stdout > 200 then
                  String.sub stdout 0 200
                else stdout))
      | Unix.WSIGNALED s -> Error (Printf.sprintf "curl signaled %d" s)
      | Unix.WSTOPPED s -> Error (Printf.sprintf "curl stopped %d" s))

let parse_eval_json (raw : string) : eval_result =
  try
    let j = Yojson.Safe.from_string raw in
    let open Yojson.Safe.Util in
    let ok =
      match j |> member "ok" with
      | `Bool b -> b
      | _ -> false
    in
    let value =
      match j |> member "value" with
      | `Null | `Assoc [] -> None
      | `String s -> Some s
      | other -> Some (Yojson.Safe.to_string other)
    in
    let error =
      match j |> member "error" with
      | `String s -> Some s
      | `Null -> None
      | other -> Some (Yojson.Safe.to_string other)
    in
    { ok; value; error; raw }
  with exn ->
    {
      ok = false;
      value = None;
      error = Some (Printf.sprintf "bad JSON: %s (%s)" (Printexc.to_string
     exn) raw);
      raw;
    }

let eval_expr ~base_url ~user ~password ~expr ~timeout_s : eval_result =
  let url = eval_url base_url in
  let body =
    Yojson.Safe.to_string
      (`Assoc [ ("expr", `String expr) ])
  in
  match curl_post ~url ~user ~password ~body ~timeout_s with
  | Error e -> { ok = false; value = None; error = Some e; raw = "" }
```

```
143   | Ok raw -> parse_eval_json raw
144
145 (** Strip outer Lisp/pp_value quotes from a VStr print. *)
146 let unwrap_lisp_string (v : string) : string =
147   let v = String.trim v in
148   if String.length v >= 2 && v.[0] = '"' && v.[String.length v - 1] = '"'
        then
149     try Yojson.Safe.Util.to_string (Yojson.Safe.from_string v)
150     with _ -> String.sub v 1 (String.length v - 2)
151   else v
```

45 client/bin/main.ml

```
(** Pricklypear browser evaluator runtime.

    A js_of_ocaml build of the pure Nopales interpreter (lisp/lib) plus
    the shared pure-prims table (pure/prims_pure.ml). The same .pp
    package source the server serves is evaluated verbatim here -- see
    borge/structural-editing.borg client-eval.

    Built with `js_of_ocaml --effects=cps` (OCaml 5 effects require the
    CPS backend; the default backend fatally aborts). Exposes a small
    JSON-bound surface to JavaScript via [Js.export]:

      boot()                -> initializes the prim table + env
      load_source(string)   -> evaluates a multi-form .pp source string
      apply_op(string, json) -> runs a pure op fn over a JSON tree,
                                returns the result tree as JSON
      eval(string)          -> evaluates one Nopales expression,
                                returns its printed value (for REPL/debug)

    The pure prim table EXCLUDES every DB / IO / net prim -- the
    capability boundary IS the prim table. Wall-clock prims (now,
    date-today, date-weekday, etc.) are absent too: they live in
    image-only prims_time.ml because they read Unix. *)
open Pricklypear_lisp

(* --- the runtime state, initialized by boot --- *)
let prims : Eval.prim_table option ref = ref None
let env : Ast.env option ref = ref None

let boot () =
  let t = Eval.make_prim_table () in
  (* Register every pure prim by name. Prims_pure is the shared Unix-free
     library; it has no DB/IO/net/wall-clock prims. *)
  let register name fn = Eval.register_prim t name fn in
  register "+" Prims_pure.prim_add;
  register "-" Prims_pure.prim_sub;
  register "*" Prims_pure.prim_mul;
  register "div" Prims_pure.prim_div;
  register "/" Prims_pure.prim_div;
  register "mod" Prims_pure.prim_mod;
  register "quotient" Prims_pure.prim_quotient;
  register "remainder" Prims_pure.prim_remainder;
  register "abs" Prims_pure.prim_abs;
  register "min" Prims_pure.prim_min;
  register "max" Prims_pure.prim_max;
  register "expt" Prims_pure.prim_expt;
  register "sqrt" Prims_pure.prim_sqrt;
  register "exp" Prims_pure.prim_exp;
  register "log" Prims_pure.prim_log;
  register "sin" Prims_pure.prim_sin;
  register "cos" Prims_pure.prim_cos;
  register "tan" Prims_pure.prim_tan;
  register "asin" Prims_pure.prim_asin;
  register "acos" Prims_pure.prim_acos;
  register "atan" Prims_pure.prim_atan;
  register "number?" Prims_pure.prim_number_p;
  register "integer?" Prims_pure.prim_integer_p;
  register "zero?" Prims_pure.prim_zero_p;
  register "positive?" Prims_pure.prim_positive_p;
  register "negative?" Prims_pure.prim_negative_p;
  register "even?" Prims_pure.prim_even_p;
  register "odd?" Prims_pure.prim_odd_p;
  register "random" Prims_pure.prim_random;
  register "random-seed" Prims_pure.prim_random_seed;
  register "=" Prims_pure.prim_num_eq;
  register "<" Prims_pure.prim_lt;
  register ">" Prims_pure.prim_gt;
  register "<=" Prims_pure.prim_le;
  register ">=" Prims_pure.prim_ge;
  register "not" Prims_pure.prim_not;
  register "list" Prims_pure.prim_list;
  register "car" Prims_pure.prim_car;
```

```
  register "cdr" Prims_pure.prim_cdr;
  register "cons" Prims_pure.prim_cons;
  register "null?" Prims_pure.prim_null_p;
  register "pair?" Prims_pure.prim_pair_p;
  register "string-append" Prims_pure.prim_string_append;
  register "string-length" Prims_pure.prim_string_length;
  register "substring" Prims_pure.prim_substring;
  register "string-contains?" Prims_pure.prim_string_contains;
  register "string-eq" Prims_pure.prim_string_eq;
  register "string<?" Prims_pure.prim_string_lt;
  register "string<=?" Prims_pure.prim_string_le;
  register "string>?" Prims_pure.prim_string_gt;
  register "string>=?" Prims_pure.prim_string_ge;
  register "string-trim" Prims_pure.prim_string_trim;
  register "html-escape" Prims_pure.prim_html_escape;
  register "json-parse" Prims_pure.prim_json_parse;
  register "json-emit" Prims_pure.prim_json_emit;
  register "read-string" Prims_pure.prim_read_string;
  register "read-all-string" Prims_pure.prim_read_all_string;
  register "type-of" Prims_pure.prim_type_of;
  register "symbol?" Prims_pure.prim_symbol_p;
  register "string?" Prims_pure.prim_string_p;
  register "symbol=?" Prims_pure.prim_symbol_eq;
  register "symbol->string" Prims_pure.prim_symbol_to_string;
  register "string->symbol" Prims_pure.prim_string_to_symbol;
  register "str" Prims_pure.prim_str;
  register "as-str" Prims_pure.prim_as_str;
  register "append" Prims_pure.prim_append;
  register "reverse" Prims_pure.prim_reverse;
  register "nth" Prims_pure.prim_nth;
  register "count" Prims_pure.prim_count;
  register "length" Prims_pure.prim_length;
  register "equal?" Prims_pure.prim_equal_p;
  register "cadr" Prims_pure.prim_cadr;
  register "caar" Prims_pure.prim_caar;
  register "caddr" Prims_pure.prim_caddr;
  register "cddr" Prims_pure.prim_cddr;
  register "assoc" Prims_pure.prim_assoc;
  register "member" Prims_pure.prim_member;
  register "dict-get" Prims_pure.prim_dict_get;
  register "dict-set" Prims_pure.prim_dict_set;
  register "dict-keys" Prims_pure.prim_dict_keys;
  register "parse-int" Prims_pure.prim_parse_int;
  register "parse-float" Prims_pure.prim_parse_float;
  register "floor" Prims_pure.prim_floor;
  register "round" Prims_pure.prim_round;
  register "fsrs-schedule" Prims_pure.prim_fsrs_schedule;
  register "render-html" Prims_pure.prim_render_html;
  register "status-response" Prims_pure.prim_status_response;
  register "part" Prims_pure.prim_part;
  register "json-part" Prims_pure.prim_json_part;
  register "multipart" Prims_pure.prim_multipart;
  register "extract-links" Prims_pure.prim_extract_links;
  register "render-links" Prims_pure.prim_render_links;
  register "budget-remaining" Prims_pure.prim_budget_remaining;
  register "string-replace" Prims_pure.prim_string_replace;
  register "string-downcase" Prims_pure.prim_string_downcase;
  register "string-upcase" Prims_pure.prim_string_upcase;
  register "string-starts-with?" Prims_pure.prim_string_starts_with_p;
  register "string-ends-with?" Prims_pure.prim_string_ends_with_p;
  register "url-encode" Prims_pure.prim_url_encode;
  register "url-decode" Prims_pure.prim_url_decode;
  register "html-decode" Prims_pure.prim_html_decode;
  register "parse-date" Prims_pure.prim_parse_date;
  register "xml-parse" Prims_pure.prim_xml_parse;
  register "random-uuid" Prims_pure.prim_random_uuid;
  (* Bind each prim name as VPrim in env, mirroring session.ml. *)
  let e = Env.create () in
  Hashtbl.iter
    (fun name _ -> Env.define e name (Ast.VPrim name))
    t;
  (* Route prim dispatch through the direct table, bypassing the
     Effect.Deep handler. Under js_of_ocaml's CPS backend the
```

```
145       one-shot deep-match continuations resume twice on nested prim
146       calls (Continuation_already_resumed); direct_prims avoids
147       performing Prim_call effects entirely. Spec:
          structural-editing.client-eval. *)
148     Pricklypear_lisp.Control.direct_prims := Some t;
149     prims := Some t;
150     env := Some e
151
152   let get_prims () =
153     match !prims with Some t -> t | None -> failwith "pp: boot() not called"
154   let get_env () =
155     match !env with Some e -> e | None -> failwith "pp: boot() not called"
156
157   (* Evaluate a multi-form .pp source string (load-library shape). Mirrors
158      image/lib/loader.ml: read_all + eval each form under the effect
159      handler. The reader handles ;; line comments natively. *)
160   let load_source (src : Js_of_ocaml.Js.js_string Js_of_ocaml.Js.t) :
          Js_of_ocaml.Js.js_string Js_of_ocaml.Js.t =
161     let s = Js_of_ocaml.Js.to_string src in
162     let t = get_prims () in
163     let e = get_env () in
164     (try
165        Control.run_sync ~prims:t (fun () ->
166            let forms = Reader.read_all s in
167            List.iter (fun form ->
168                let budget = Eval.make_budget () in
169                ignore (Eval.eval ~budget ~prims:t e form))
170              forms)
171      with _ -> ());
172     Js_of_ocaml.Js.string "ok"
173
174   (* Evaluate one expression, return its printed value (REPL/debug). *)
175   let eval (src : Js_of_ocaml.Js.js_string Js_of_ocaml.Js.t) :
          Js_of_ocaml.Js.js_string Js_of_ocaml.Js.t =
176     let s = Js_of_ocaml.Js.to_string src in
177     let t = get_prims () in
178     let e = get_env () in
179     try
180       let v = Eval.eval_string ~prims:t e s in
181       Js_of_ocaml.Js.string (Reader.pp_value v)
182     with
183     | Ast.Interp_error err ->
184         Js_of_ocaml.Js.string
185           ("ERROR: " ^ Ast.pp_interp_error err)
186     | Reader.Reader_error (m, l, c) ->
187         Js_of_ocaml.Js.string (Printf.sprintf "ERROR: %s (%d:%d)" m l c)
188     | e -> Js_of_ocaml.Js.string ("ERROR: " ^ Printexc.to_string e)
189
190   (* set_tree(json): parse the outline tree JSON into a Nopales value and
191      bind it as `pp-tree` in the env. json-parse of a JSON array yields a
192      VList, matching the server's tree shape. Spec: structural-editing.
193      client-eval OPTIMISTIC EXECUTION MODEL step 1. *)
194   let set_tree (json : Js_of_ocaml.Js.js_string Js_of_ocaml.Js.t) : unit =
195     let s = Js_of_ocaml.Js.to_string json in
196     let t = get_prims () in
197     let e = get_env () in
198     let v = Eval.eval_string ~prims:t e ("(json-parse " ^ Printf.sprintf "%S"
          s ^ ")") in
199     Env.define e "pp-tree" v
200
201   (* apply_op(op, nid, slug): run the optimistic preview in the browser --
202      dispatch the pure op over pp-tree (same outline/structed fns the
203      server runs) and render via outline/html-lite. Returns the preview
204      HTML, or "" if the op is not previewable (set-*, eval, no-op). *)
205   let apply_op
206       (op : Js_of_ocaml.Js.js_string Js_of_ocaml.Js.t)
207       (nid : Js_of_ocaml.Js.js_string Js_of_ocaml.Js.t)
208       (slug : Js_of_ocaml.Js.js_string Js_of_ocaml.Js.t)
209     : Js_of_ocaml.Js.js_string Js_of_ocaml.Js.t =
210     let op_s = Js_of_ocaml.Js.to_string op in
211     let nid_s = Js_of_ocaml.Js.to_string nid in
212     let slug_s = Js_of_ocaml.Js.to_string slug in
213     let t = get_prims () in
```

```
  let e = get_env () in
  let expr =
    Printf.sprintf "(pp/preview %S pp-tree %S %S)" slug_s op_s nid_s
  in
  try
    let v = Eval.eval_string ~prims:t e expr in
    (match v with
     | Ast.VStr s -> Js_of_ocaml.Js.string s
     | _ -> Js_of_ocaml.Js.string "")
  with _ -> Js_of_ocaml.Js.string ""

let () =
  let open Js_of_ocaml.Js in
  let obj =
    Unsafe.obj
      [| "boot", Unsafe.inject (wrap_callback boot)
       ; "load_source", Unsafe.inject (wrap_callback load_source)
       ; "eval", Unsafe.inject (wrap_callback eval)
       ; "set_tree", Unsafe.inject (wrap_callback set_tree)
       ; "apply_op", Unsafe.inject (wrap_callback apply_op)
      |]
  in
  export "pp" obj
```

46 client/conformance/main.ml

```
(** Conformance gate against the JS target.

    Runs the shared [Conformance.cases] through the same evaluator
    compiled with js_of_ocaml. Spec [structural-editing.client-eval]:
    CONFORMANCE IS THE GATE -- client/server agreement is a tested
    invariant. Exits nonzero on any mismatch. *)
open Pricklypear_lisp

open Pricklypear_lisp
open Pricklypear_conformance
open Conformance

let () =
  let prims, env = Conformance.setup_arith () in
  let pass = ref 0 in
  let fail = ref 0 in
  List.iter (fun c ->
      List.iter (fun s -> ignore (Eval.eval_string ~prims env s)) c.setups;
      (try
         let got = Conformance.eval_pp ~prims env c.src in
         if got <> c.expected then begin
           incr fail;
           Printf.printf "FAIL [%s] %s: expected %S got %S\n"
             c.group c.name c.expected got
         end else incr pass
       with e ->
         incr fail;
         Printf.printf "FAIL [%s] %s: raised %s\n"
           c.group c.name (Printexc.to_string e)))
    Conformance.cases;
  Printf.printf "conformance (js target): %d pass, %d fail\n" !pass !fail;
  if !fail > 0 then exit 1
```

47 client/ops.pp

```
;; package pp -- client optimistic-preview dispatch + render.
;; Loaded into the browser evaluator (client/bin/interp.js) AFTER the
;; package sources (structed, outline, html, bible). Spec:
;; borge/structural-editing.borg client-eval.
;;
;; pp/dispatch-op mirrors the PURE branches of notes/outline/apply-op
;; (the ops whose fns are pure outline/* + structed/*). It calls the
;; SAME functions the server calls -- no reimplementation of the op
;; algebra. The set-*/eval branches use request-param (impure) and are
;; not previewable; they return the tree unchanged so the client falls
;; back to server-only for those.

(define pp/dispatch-op
  (lambda
    (op tree nid)
    (cond
      ((string-eq op "add-child")
        (outline/add-child-by-id tree nid (outline/node (gensym "n") "New
    node")))
      ((string-eq op "add-sibling")
        (outline/insert-sibling tree nid (outline/node (gensym "n") "New
    node")))
      ((string-eq op "delete")
        (if (string-eq (as-str nid) "root") tree (outline/delete tree nid)))
      ((string-eq op "move-up") (outline/move-up tree nid))
      ((string-eq op "move-down") (outline/move-down tree nid))
      ((string-eq op "promote")
        (if (string-eq (as-str nid) "root") tree (outline/promote tree
    nid)))
      ((string-eq op "demote")
        (if (string-eq (as-str nid) "root") tree (outline/demote tree nid)))
      ((string-eq op "cycle-todo")
        (outline/update tree nid (lambda (n) (outline/cycle-todo n))))
      ((string-eq op "archive")
        (if (string-eq (as-str nid) "root") tree
          (outline/update tree nid (lambda (n) (outline/archive n)))))
      ((string-eq op "unarchive")
        (if (string-eq (as-str nid) "root") tree
          (outline/update tree nid (lambda (n) (outline/unarchive n)))))
      ((string-eq op "slurp-forward")
        (if (string-eq (as-str nid) "root") tree
          (car (structed/slurp-forward outline/kids outline/set-kids tree
    nid))))
      ((string-eq op "barf-forward")
        (if (string-eq (as-str nid) "root") tree
          (car (structed/barf-forward outline/kids outline/set-kids tree
    nid))))
      (else tree))))

;; pp/preview: apply the op to the tree and render via outline/html-lite
;; (the same render lib the server uses, minus action buttons -- the
;; action buttons are re-added when the server's authoritative render
;; morphs in). Returns the HTML string. Spec: client-eval OPTIMISTIC
;; EXECUTION MODEL steps 1-3.
(define pp/preview
  (lambda
    (slug tree op nid)
    (let ((new-tree (pp/dispatch-op op tree nid)))
      (if (equal? new-tree tree)
        ""
        (outline/html-lite slug new-tree #f)))))
```

48 common/lib/auth_types.ml

```
(** Pricklypear Common -- Auth types.

    Shared JSON/types for users, credentials, and auth results. Single-image
    v1 verifies passwords and sessions in-process (image/lib/auth_*.ml);
    there is no separate authd/supervisor RPC path. *)
open Yojson.Safe

type user_id = string
type credential_id = string

type user = {
  id : user_id;
  username : string;
  is_admin : bool;
  created_at : string;
}

type credential_kind =
  | K_ssh_key
  | K_password
  | K_web_token
  | K_api_key

let credential_kind_to_string = function
  | K_ssh_key -> "ssh_key"
  | K_password -> "password"
  | K_web_token -> "web_token"
  | K_api_key -> "api_key"

let credential_kind_of_string = function
  | "ssh_key" -> Some K_ssh_key
  | "password" -> Some K_password
  | "web_token" -> Some K_web_token
  | "api_key" -> Some K_api_key
  | _ -> None

(* A signed session token: opaque_id || HMAC(opaque_id).
   The opaque_id is the sessions.id; the HMAC is checked by auth. *)
type token = {
  opaque_id : string;     (* 32-byte random, base64 *)
  signature : string;     (* HMAC, base64 *)
}

let token_to_string (t : token) : string =
  (* joined with "." so we can split -- base64 is url-safe (no .) *)
  t.opaque_id ^ "." ^ t.signature

let token_of_string (s : string) : (token, string) result =
  match String.index_opt s '.' with
  | None -> Error "malformed token (no separator)"
  | Some i ->
      let opaque_id = String.sub s 0 i in
      let signature = String.sub s (i + 1) (String.length s - i - 1) in
      if opaque_id = "" || signature = "" then
        Error "malformed token (empty part)"
      else Ok { opaque_id; signature }

(* RPC request/response types *)
type verify_ssh_key_req = { key : string }
type verify_ssh_key_resp = Ok of user_id | Denied

let verify_ssh_key_resp_to_json = function
  | Ok uid -> `Assoc [ ("ok", `Bool true); ("user_id", `String uid) ]
  | Denied -> `Assoc [ ("ok", `Bool false) ]

let verify_ssh_key_resp_of_json (j : Yojson.Safe.t) : (verify_ssh_key_resp,
    string) result =
  try
    let assoc = match j with `Assoc a -> a | _ -> failwith "not assoc" in
    match List.assoc "ok" assoc with
    | `Bool true ->
```

```
        (match List.assoc "user_id" assoc with
         | `String uid -> Ok (Ok uid)
         | _ -> failwith "user_id")
    | `Bool false -> Ok Denied
    | _ -> failwith "ok"
  with Failure msg -> Error msg
```

49 common/lib/branch.ml

```
(** Pricklypear Common -- Branch reference.

    A branch is identified by (owner, name); each has a parent (the branch
    it was forked from). The root is system/main (owner="system",
    name="main", no parent).

    Per spec [branching]: branches organize code (keyed by branch_id in
    Postgres). Shared product data lives in data_rows (not branch-scoped).
     *)
open Yojson.Safe

type t = {
  id : string;            (* uuid, the primary key in branches table *)
  parent_id : string option;
  owner : string;         (* "system" for main, username otherwise *)
  name : string;          (* "main", "alice/main", "alice/experiment", ...
     *)
  created_at : string;    (* ISO 8601 timestamp *)
  last_synced : string option;  (* retired (branching.branch-status); kept
     for protocol compat *)
  (* branching.branch-status: the parent's head generation at fork time
     (branch_generations counter). The merge base -- advanced when a merge
     lands this branch into its parent; NOT advanced by promote, so
     selective pins stay visible as `synced`. *)
  base_gen : int64;
}

let make ~id ~parent_id ~owner ~name ~created_at ?(last_synced = None)
    ?(base_gen = 0L) () : t =
  { id; parent_id; owner; name; created_at; last_synced; base_gen }

let is_main (b : t) : bool = b.owner = "system" && b.name = "main"

let qualified_name (b : t) : string =
  if is_main b then "main" else b.owner ^ "/" ^ b.name

let to_json (b : t) : Yojson.Safe.t =
  `Assoc [
    ("id", `String b.id);
    ("parent_id", match b.parent_id with
                  | Some p -> `String p
                  | None -> `Null);
    ("owner", `String b.owner);
    ("name", `String b.name);
    ("created_at", `String b.created_at);
    ("last_synced", match b.last_synced with
                    | Some s -> `String s
                    | None -> `Null);
    ("base_gen", `Int (Int64.to_int b.base_gen));
  ]

let of_json (j : Yojson.Safe.t) : (t, string) result =
  try
    let assoc = match j with `Assoc a -> a | _ -> failwith "not an assoc" in
    let find k = List.assoc k assoc in
    let id = match find "id" with `String s -> s | _ -> failwith "id" in
    let parent_id = match find "parent_id" with
                    | `String s -> Some s | `Null -> None
                    | _ -> failwith "parent_id" in
    let owner = match find "owner" with `String s -> s | _ -> failwith
     "owner" in
    let name = match find "name" with `String s -> s | _ -> failwith "name"
     in
    let created_at = match find "created_at" with `String s -> s | _ ->
     failwith "created_at" in
    let last_synced = match find "last_synced" with
                      | `String s -> Some s | `Null -> None
                      | _ -> failwith "last_synced" in
    let base_gen = match find "base_gen" with
                   | `Int i -> Int64.of_int i
                   | _ -> 0L (* tolerant: pre-024 snapshots *) in
```

```
    Ok { id; parent_id; owner; name; created_at; last_synced; base_gen }
  with Failure msg -> Error msg
```

50 common/lib/llm_codec.ml

```
(** Pricklypear Common -- OpenAI chat-completions wire codec.

    Pure yojson: the types and encoders/decoders for the OpenAI-compatible
    chat completions API shape shared by the image agent
    ([image/lib/llm_client.ml]) and the standalone agent
    ([agent/lib/provider.ml]). No transport, no Lwt, no Eio, no curl --
    each host owns its wire (curl vs hand-rolled Eio HTTP); this module
    owns the value<->json boundary.

    Spec gate ([borge/agent.borg] / [borge/remote-eval.borg]): a shared,
    tested value<->json codec. The robustness the image has bled for --
    GLM-class [reasoning_content] selection and XML-in-content tool-call
    fallback -- lives here so the standalone agent inherits it. *)

(** A tool call as it appears on the wire: [arguments] is a JSON object
    string (OpenAI encodes function arguments as a stringified object). *)
type tool_call = {
  id : string;
  name : string;
  arguments : string;  (* JSON object string *)
}

(** A tool definition for the [tools] request field. [parameters] is a
    JSON Schema object. *)
type tool_def = {
  name : string;
  description : string;
  parameters : Yojson.Safe.t;  (* JSON Schema object *)
}

(** One chat-turn outcome. [raw] carries the assistant [message] object
    verbatim so hosts that round-trip history (the standalone agent
    appends the raw message back) can do so without re-encoding. Hosts
    that store typed history (the image) ignore it. *)
type turn_result =
  | Final of { content : string; raw : Yojson.Safe.t }
  | Need_tools of {
      content : string option;
      tool_calls : tool_call list;
      raw : Yojson.Safe.t;
    }
  | Turn_error of string

let string_field (msg : Yojson.Safe.t) (key : string) : string =
  match Yojson.Safe.Util.(msg |> member key |> to_string_option) with
  | Some s -> s
  | None -> ""

(** True when [s] looks like a complete JSON value (object/array/string). *)
let looks_complete_json (s : string) : bool =
  let s = String.trim s in
  if s = "" then false
  else
    (try
       let _ = Yojson.Safe.from_string s in
       true
     with _ -> false)

let find_substring ~(hay : string) ~(needle : string) ~(from : int) : int
    option =
  let nlen = String.length needle in
  let hlen = String.length hay in
  let rec go i =
    if i + nlen > hlen then None
    else if String.sub hay i nlen = needle then Some i
    else go (i + 1)
  in
  go from

let tool_def_to_json (t : tool_def) : Yojson.Safe.t =
  `Assoc
```

```
    [ ("type", `String "function")
    ; ("function",
       `Assoc
         [ ("name", `String t.name)
         ; ("description", `String t.description)
         ; ("parameters", t.parameters)
         ])
    ]

(* XML-in-content tool-call fallback delimiters. Built by concatenation
   so the literal tags never appear as a tool-call boundary in this
   source. GLM-class models emit these tags inside [content] instead of
   using the API [tool_calls] field; the format is a tool_call block
   wrapping arg_key/arg_value pairs naming the function and its JSON
   arguments. *)
let xml_open_tag      = "<" ^ "tool_call>"
let xml_close_tag     = "<" ^ "/tool_call>"
let xml_arg_key_open  = "<" ^ "arg_key>"
let xml_arg_key_close = "<" ^ "/arg_key>"
let xml_arg_val_open  = "<" ^ "arg_value>"
let xml_arg_val_close = "<" ^ "/arg_value>"

(** Recover when models dump tools as XML in content instead of API
    tool_calls. Deliberately lenient: it finds the head token before the
    first arg tag and reads key/value pairs by tag bracketing. *)
let parse_xml_tool_calls (text : string) : tool_call list =
  let rec find_blocks acc from =
    match find_substring ~hay:text ~needle:xml_open_tag ~from with
    | None -> List.rev acc
    | Some start ->
        let body_start = start + String.length xml_open_tag in
        (match find_substring ~hay:text ~needle:xml_close_tag
     ~from:body_start with
         | None -> List.rev acc
         | Some stop ->
             let block = String.sub text body_start (stop - body_start) in
             find_blocks (block :: acc) (stop + String.length
     xml_close_tag))
  in
  let parse_args block =
    let rec collect acc from =
      match find_substring ~hay:block ~needle:xml_arg_key_open ~from with
      | None -> List.rev acc
      | Some k0 ->
          let k1 = k0 + String.length xml_arg_key_open in
          (match find_substring ~hay:block ~needle:xml_arg_key_close
     ~from:k1 with
           | None -> List.rev acc
           | Some k2 ->
               let key = String.trim (String.sub block k1 (k2 - k1)) in
               let after_key = k2 + String.length xml_arg_key_close in
               (match find_substring ~hay:block ~needle:xml_arg_val_open
     ~from:after_key with
                | None -> List.rev acc
                | Some v0 ->
                    let v1 = v0 + String.length xml_arg_val_open in
                    (match find_substring ~hay:block
     ~needle:xml_arg_val_close ~from:v1 with
                     | None -> List.rev acc
                     | Some v2 ->
                         let value = String.sub block v1 (v2 - v1) in
                         collect ((key, value) :: acc)
                           (v2 + String.length xml_arg_val_close))))
    in
    collect [] 0
  in
  let parse_one idx block =
    let block = String.trim block in
    let name =
      match find_substring ~hay:block ~needle:xml_arg_key_open ~from:0 with
      | Some i -> String.trim (String.sub block 0 i)
      | None ->
          (match String.split_on_char '\n' block with
```

```
          | n :: _ -> String.trim n
          | [] -> "")
    in
    let name =
      match String.split_on_char ' ' (String.trim name) with
      | x :: _ -> String.trim x
      | [] -> String.trim name
    in
    if name = "" then None
    else
      let args = parse_args block in
      let arguments =
        Yojson.Safe.to_string
          (`Assoc (List.map (fun (k, v) -> (k, `String v)) args))
      in
      Some
        { id = Printf.sprintf "xml-tool-%d" (idx + 1)
        ; name
        ; arguments
        }
  in
  find_blocks [] 0
  |> List.mapi parse_one
  |> List.filter_map (fun x -> x)

(** Parse the [tool_calls] array off an assistant [message] object. *)
let parse_tool_calls_from_msg (msg : Yojson.Safe.t) : tool_call list =
  let open Yojson.Safe.Util in
  match msg |> member "tool_calls" with
  | `Null -> []
  | `List xs ->
      List.filter_map
        (fun tc ->
          try
            let id = tc |> member "id" |> to_string in
            let fn = tc |> member "function" in
            let name = fn |> member "name" |> to_string in
            let arguments =
              match fn |> member "arguments" with
              | `String s -> s
              | other -> Yojson.Safe.to_string other
            in
            Some { id; name; arguments }
          with _ -> None)
        xs
  | _ -> []

(** Parse a parsed chat-completions response into a [turn_result].
    Captures the assistant [message] object as [raw]. *)
let parse_turn_response_json (json : Yojson.Safe.t) : turn_result =
  try
    let open Yojson.Safe.Util in
    match json |> member "choices" |> to_list with
    | [] -> Turn_error "LLM: empty choices"
    | choice :: _ ->
        let msg = choice |> member "message" in
        let finish =
          match choice |> member "finish_reason" |> to_string_option with
          | Some f -> f
          | None -> ""
        in
        let content_opt =
          match msg |> member "content" |> to_string_option with
          | Some c when String.trim c <> "" -> Some c
          | _ -> None
        in
        let tcs = parse_tool_calls_from_msg msg in
        if tcs <> [] then
          Need_tools { content = content_opt; tool_calls = tcs; raw = msg }
        else
          (* Fallback: XML tool_call blocks in content (common model
     failure). *)
          let xml_tcs =
```

```
            match content_opt with
            | Some c -> parse_xml_tool_calls c
            | None -> []
          in
          if xml_tcs <> [] then
            Need_tools { content = None; tool_calls = xml_tcs; raw = msg }
          else if finish = "tool_calls" then
            Turn_error "LLM: finish_reason=tool_calls but no tool_calls payload"
          else
            (match content_opt with
             | Some c -> Final { content = c; raw = msg }
             | None ->
                 let reasoning_alias =
                   match msg |> member "reasoning_content" |> to_string_option with
                   | Some r when String.trim r <> "" -> Some r
                   | _ ->
                       (* Synthetic streams/thinking messages carry plain
                          "reasoning" (agent provider.ml has the wire probe). *)
                       (match msg |> member "reasoning" |> to_string_option with
                        | Some r when String.trim r <> "" -> Some r
                        | _ -> None)
                 in
                 (match reasoning_alias with
                  | Some r -> Final { content = r; raw = msg }
                  | None ->
                      Turn_error
                        (Printf.sprintf "LLM: empty content (finish=%s)" finish)))
  with e ->
    Turn_error ("LLM parse error: " ^ Printexc.to_string e)

(** Parse a raw chat-completions response body string. *)
let parse_turn_response (body : string) : turn_result =
  try parse_turn_response_json (Yojson.Safe.from_string body)
  with Yojson.Json_error e ->
    let trunc =
      if String.length body > 300 then String.sub body 0 300 ^ "..." else body
    in
    Turn_error ("LLM parse error: " ^ e ^ " body=" ^ trunc)

(** Extract the assistant text content from a non-streaming response body.
    Handles GLM-class models that fill [message.reasoning_content] and
    leave [content] null or truncated, preferring whichever field parses
    as complete JSON when the other is empty/truncated. Returns the raw
    content string (not parsed) so callers decide what to do with it. *)
let extract_content (body : string) : (string, string) result =
  try
    let json = Yojson.Safe.from_string body in
    let open Yojson.Safe.Util in
    let choices = json |> member "choices" |> to_list in
    match choices with
    | [] -> Error "LLM: empty choices"
    | choice :: _ ->
        let msg = choice |> member "message" in
        let finish =
          match choice |> member "finish_reason" |> to_string_option with
          | Some f -> f
          | None -> "?"
        in
        let content_field = string_field msg "content" in
        let reasoning =
          let r1 = string_field msg "reasoning_content" in
          if r1 <> "" then r1 else string_field msg "reasoning"
        in
        let content =
          match content_field, reasoning with
          | c, _ when c <> "" && looks_complete_json c -> c
          | _, r when r <> "" && looks_complete_json r -> r
```

```
          | c, r when c <> "" && r <> "" ->
              (* truncated content + reasoning: give the parser both *)
              c ^ "\n" ^ r
          | c, _ when c <> "" -> c
          | _, r when r <> "" -> r
          | _ -> ""
        in
        if content = "" then
          Error (Printf.sprintf "LLM: empty content (finish=%s)" finish)
        else Ok content
  with e ->
    let trunc =
      if String.length body > 300 then String.sub body 0 300 ^ "..." else
     body
    in
    Error ("LLM parse error: " ^ Printexc.to_string e ^ " body=" ^ trunc)

(** Extract delta content from one streaming SSE JSON payload ([DONE]
    or a [choices[0].delta.content] chunk). *)
let extract_stream_delta (data : string) : string option =
  if data = "[DONE]" then None
  else
    (try
       let json = Yojson.Safe.from_string data in
       let open Yojson.Safe.Util in
       match json |> member "choices" |> to_list with
       | [] -> None
       | choice :: _ ->
           let delta = choice |> member "delta" in
           (match delta |> member "content" |> to_string_option with
            | Some s when s <> "" -> Some s
            | _ -> None)
     with _ -> None)
```

51 common/lib/manifest.ml

```
(** Pricklypear Common -- Boot manifest.

    Built from env (or defaults) at image boot -- see [image.boot] and
    [scripts/dev.sh]. Tells the image which branch to load, which user
    it is running as, boot mode, and budgets/flags.

    In single-image v1, identity for HTTP comes from in-process auth
    (session cookie or Basic). The manifest is the process-level default
    claim used before a request attaches a user (and for the optional
    TCP REPL). *)
open Yojson.Safe

type boot_mode =
  | Boot_on_connect    (* image dies when session ends; default *)
  | Boot_always_on     (* image pinned running; survives SSH disconnects *)

let boot_mode_to_string = function
  | Boot_on_connect -> "on-connect"
  | Boot_always_on -> "always-on"

let boot_mode_of_string = function
  | "on-connect" -> Some Boot_on_connect
  | "always-on" -> Some Boot_always_on
  | _ -> None

type config = {
  step_budget_limit : int;       (* eval step budget per call, default
     100000 *)
  memory_budget_mb : int;        (* deferred; recorded here *)
  feature_flags : (string * bool) list;
}

let default_config : config = {
  step_budget_limit = 100_000;
  memory_budget_mb = 64;
  feature_flags = [];
}

let config_to_json (c : config) : Yojson.Safe.t =
  `Assoc [
    ("step_budget_limit", `Int c.step_budget_limit);
    ("memory_budget_mb", `Int c.memory_budget_mb);
    ("feature_flags",
     `Assoc (List.map (fun (k, v) -> (k, `Bool v)) c.feature_flags));
  ]

let config_of_json (j : Yojson.Safe.t) : (config, string) result =
  try
    let assoc = match j with `Assoc a -> a | _ -> failwith "not assoc" in
    let step = match List.assoc_opt "step_budget_limit" assoc with
               | Some (`Int n) -> n | _ -> 100_000 in
    let mem = match List.assoc_opt "memory_budget_mb" assoc with
              | Some (`Int n) -> n | _ -> 64 in
    let flags = match List.assoc_opt "feature_flags" assoc with
                | Some (`Assoc items) ->
                    List.filter_map (fun (k, v) ->
                      match v with `Bool b -> Some (k, b) | _ -> None) items
                | _ -> [] in
    Ok { step_budget_limit = step; memory_budget_mb = mem; feature_flags =
     flags }
  with Failure msg -> Error msg

type t = {
  user_id : string;        (* process default identity; HTTP may re-attach
     per request *)
  username : string;       (* for the REPL prompt and author tracking *)
  branch_id : string;      (* default branch UUID, usually user/main *)
  branch_name : string;    (* qualified: "alice/main" *)
  boot_mode : boot_mode;
  image_version : string;  (* for A/B tracking *)
  config : config;
```

```
}

let make
  ~user_id ~username ~branch_id ~branch_name ~boot_mode ~image_version
  ?(config = default_config) () : t =
  { user_id; username; branch_id; branch_name; boot_mode; image_version;
    config }

(* Accessors for cross-package callers that don't have the record type. *)
let user_id (m : t) : string = m.user_id
let username (m : t) : string = m.username
let branch_id (m : t) : string = m.branch_id
let branch_name (m : t) : string = m.branch_name

let to_json (m : t) : Yojson.Safe.t =
  `Assoc [
    ("user_id", `String m.user_id);
    ("username", `String m.username);
    ("branch_id", `String m.branch_id);
    ("branch_name", `String m.branch_name);
    ("boot_mode", `String (boot_mode_to_string m.boot_mode));
    ("image_version", `String m.image_version);
    ("config", config_to_json m.config);
  ]

let of_json (j : Yojson.Safe.t) : (t, string) result =
  try
    let assoc = match j with `Assoc a -> a | _ -> failwith "not assoc" in
    let user_id = match List.assoc "user_id" assoc with `String s -> s | _
     -> failwith "user_id" in
    let username = match List.assoc "username" assoc with `String s -> s |
     _ -> failwith "username" in
    let branch_id = match List.assoc "branch_id" assoc with `String s -> s
     | _ -> failwith "branch_id" in
    let branch_name = match List.assoc "branch_name" assoc with `String s
     -> s | _ -> failwith "branch_name" in
    let boot_mode_str = match List.assoc "boot_mode" assoc with `String s
     -> s | _ -> failwith "boot_mode" in
    let boot_mode = match boot_mode_of_string boot_mode_str with
                    | Some m -> m | None -> failwith "bad boot_mode" in
    let image_version = match List.assoc "image_version" assoc with `String
     s -> s | _ -> failwith "image_version" in
    let config = match List.assoc_opt "config" assoc with
                 | Some cj -> (match config_of_json cj with Ok c -> c |
     Error _ -> default_config)
                 | None -> default_config in
    Ok { user_id; username; branch_id; branch_name; boot_mode;
     image_version; config }
  with Failure msg -> Error msg
```

52 common/lib/protocol.ml

```
(** Pricklypear Common -- Health-check protocol.

    JSON shape for GET /health on the image (branch, uptime, status).
    Used by reverse proxies, watchdogs, and smoke scripts -- not a
    multi-process supervisor fleet. *)
open Yojson.Safe

type health_status =
  | H_unknown
  | H_healthy
  | H_unhealthy
  | H_dead

let health_status_to_string = function
  | H_unknown -> "unknown"
  | H_healthy -> "healthy"
  | H_unhealthy -> "unhealthy"
  | H_dead -> "dead"

let health_status_of_string = function
  | "unknown" -> H_unknown
  | "healthy" -> H_healthy
  | "unhealthy" -> H_unhealthy
  | "dead" -> H_dead
  | _ -> H_unknown

type health_response = {
  ok : bool;
  branch : string;          (* qualified name like "alice/main" *)
  uptime_seconds : int;     (* seconds since image boot *)
  error : string option;    (* present if not ok *)
}

let health_response_to_json (h : health_response) : Yojson.Safe.t =
  `Assoc [
    ("ok", `Bool h.ok);
    ("branch", `String h.branch);
    ("uptime_seconds", `Int h.uptime_seconds);
    ("error", match h.error with Some e -> `String e | None -> `Null);
  ]

let health_response_of_json (j : Yojson.Safe.t) : (health_response, string)
    result =
  try
    let assoc = match j with `Assoc a -> a | _ -> failwith "not assoc" in
    let ok = match List.assoc "ok" assoc with `Bool b -> b | _ -> failwith
     "ok" in
    let branch = match List.assoc "branch" assoc with `String s -> s | _ ->
     failwith "branch" in
    let uptime = match List.assoc "uptime_seconds" assoc with `Int n -> n |
     _ -> failwith "uptime" in
    let error = match List.assoc "error" assoc with
                | `String s -> Some s | `Null -> None | _ -> failwith
     "error" in
    Ok { ok; branch; uptime_seconds = uptime; error }
  with Failure msg -> Error msg
```

53 common/test/common_test.ml

```
(** Tests for the Pricklypear common library -- pure, no services. *)
open Pricklypear_common

let branch_tests = [
  ("branch: main is detected", `Quick, fun () ->
    let b = Branch.make
      ~id:"b1" ~parent_id:None ~owner:"system" ~name:"main"
      ~created_at:"2026-07-05T00:00:00Z" () in
    Alcotest.(check bool) "is_main" true (Branch.is_main b);
    Alcotest.(check string) "qualified" "main" (Branch.qualified_name b));
  ("branch: user branch qualified name", `Quick, fun () ->
    let b = Branch.make
      ~id:"b2" ~parent_id:(Some "b1") ~owner:"alice" ~name:"main"
      ~created_at:"2026-07-05T00:00:00Z" () in
    Alcotest.(check bool) "not main" false (Branch.is_main b);
    Alcotest.(check string) "qualified" "alice/main" (Branch.qualified_name
     b));
  ("branch: JSON round-trip", `Quick, fun () ->
    let b = Branch.make
      ~id:"b3" ~parent_id:(Some "b1") ~owner:"alice" ~name:"experiment"
      ~created_at:"2026-07-05T01:00:00Z" ~last_synced:(Some
     "2026-07-05T02:00:00Z") () in
    let j = Branch.to_json b in
    (match Branch.of_json j with
     | Ok b' ->
        Alcotest.(check string) "id" b.id b'.id;
        Alcotest.(check string) "owner" "alice" b'.owner;
        Alcotest.(check string) "name" "experiment" b'.name;
        Alcotest.(check string) "last_synced" "2026-07-05T02:00:00Z"
                                  (match b'.last_synced with Some s -> s |
     None -> "")
     | Error e -> Alcotest.failf "round-trip failed: %s" e));
]

let manifest_tests = [
  ("manifest: JSON round-trip with default config", `Quick, fun () ->
    let m = Manifest.make
      ~user_id:"u1" ~username:"alice" ~branch_id:"b2"
      ~branch_name:"alice/main" ~boot_mode:Manifest.Boot_on_connect
      ~image_version:"v1" () in
    let j = Manifest.to_json m in
    (match Manifest.of_json j with
     | Ok m' ->
        Alcotest.(check string) "user_id" "u1" m'.user_id;
        Alcotest.(check string) "username" "alice" m'.username;
        Alcotest.(check string) "branch_name" "alice/main" m'.branch_name;
        Alcotest.(check int) "step_budget" 100_000
     m'.config.step_budget_limit
     | Error e -> Alcotest.failf "round-trip failed: %s" e));
  ("manifest: boot_mode parse", `Quick, fun () ->
    Alcotest.(check bool) "on-connect parses"
      true (Manifest.Boot_on_connect =
              (match Manifest.boot_mode_of_string "on-connect" with
               | Some m -> m | None -> Manifest.Boot_always_on)));
]

let auth_types_tests = [
  ("auth: token string round-trip", `Quick, fun () ->
    let t = { Auth_types.opaque_id = "AAAA"; signature = "BBBB" } in
    let s = Auth_types.token_to_string t in
    (match Auth_types.token_of_string s with
     | Ok t' ->
        Alcotest.(check string) "opaque" "AAAA" t'.opaque_id;
        Alcotest.(check string) "sig" "BBBB" t'.signature
     | Error e -> Alcotest.failf "token parse failed: %s" e));
  ("auth: malformed token rejected", `Quick, fun () ->
    (match Auth_types.token_of_string "noseparator" with
     | Ok _ -> Alcotest.fail "should have failed"
     | Error _ -> ());
    Alcotest.(check bool) "rejected" true true);
  ("auth: verify_ssh_key_resp round-trip Ok", `Quick, fun () ->
```

```
    let r = Auth_types.Ok "u1" in
    let j = Auth_types.verify_ssh_key_resp_to_json r in
    (match Auth_types.verify_ssh_key_resp_of_json j with
     | Ok (Auth_types.Ok uid) -> Alcotest.(check string) "uid" "u1" uid
     | Ok Auth_types.Denied -> Alcotest.fail "expected Ok"
     | Error e -> Alcotest.failf "parse failed: %s" e));
]

let protocol_tests = [
  ("protocol: health_response round-trip ok", `Quick, fun () ->
    let h = { Protocol.ok = true; branch = "alice/main";
              uptime_seconds = 42; error = None } in
    let j = Protocol.health_response_to_json h in
    (match Protocol.health_response_of_json j with
     | Ok h' ->
        Alcotest.(check bool) "ok" true h'.ok;
        Alcotest.(check string) "branch" "alice/main" h'.branch;
        Alcotest.(check int) "uptime" 42 h'.uptime_seconds
     | Error e -> Alcotest.failf "parse failed: %s" e));
  ("protocol: health_status conversions", `Quick, fun () ->
    Alcotest.(check string) "healthy" "healthy"
      (Protocol.health_status_to_string Protocol.H_healthy);
    Alcotest.(check bool) "round-trip" true
      (Protocol.H_unhealthy = Protocol.health_status_of_string
     "unhealthy"));
]

let llm_codec_tests = [
  ("llm-codec: tool_def_to_json shape", `Quick, fun () ->
    let j = Llm_codec.tool_def_to_json
      { name = "read"; description = "read a file";
        parameters = `Assoc [ ("type", `String "object") ] }
    in
    let open Yojson.Safe.Util in
    Alcotest.(check string) "type" "function" (j |> member "type" |>
     to_string);
    Alcotest.(check string) "name" "read"
      (j |> member "function" |> member "name" |> to_string));
  ("llm-codec: parse_tool_calls_from_msg API field", `Quick, fun () ->
    let msg = Yojson.Safe.from_string

     {|{"tool_calls":[{"id":"call_1","function":{"name":"bash","arguments":"{\"command\":\"echo
     hi\"}"}}]}|}
    in
    let tcs = Llm_codec.parse_tool_calls_from_msg msg in
    Alcotest.(check int) "one call" 1 (List.length tcs);
    (match tcs with
     | [tc] ->
        Alcotest.(check string) "id" "call_1" tc.Llm_codec.id;
        Alcotest.(check string) "name" "bash" tc.Llm_codec.name;
        Alcotest.(check string) "args" "{\"command\":\"echo hi\"}"
     tc.Llm_codec.arguments
     | _ -> Alcotest.fail "expected one tool call"));
  ("llm-codec: parse_tool_calls_from_msg empty", `Quick, fun () ->
    let msg = `Assoc [ ("content", `String "hello") ] in
    Alcotest.(check int) "none" 0
      (List.length (Llm_codec.parse_tool_calls_from_msg msg)));
  ("llm-codec: parse_xml_tool_calls fallback", `Quick, fun () ->
    (* Build the XML-ish tool-call block by concatenation so this
       source contains no literal tool-call tag boundaries. *)
    let block =
      ("<" ^ "tool_call>") ^ " install_function "
      ^ ("<" ^ "arg_key>name" ^ "</" ^ "arg_key>")
      ^ ("<" ^ "arg_value>foo" ^ "</" ^ "arg_value>")
      ^ ("<" ^ "arg_key>body" ^ "</" ^ "arg_key>")
      ^ ("<" ^ "arg_value>(lambda () 1)" ^ "</" ^ "arg_value>")
      ^ ("</" ^ "tool_call>")
    in
    let tcs = Llm_codec.parse_xml_tool_calls block in
    Alcotest.(check int) "one xml call" 1 (List.length tcs);
    (match tcs with
     | [tc] ->
        Alcotest.(check string) "name" "install_function" tc.Llm_codec.name;
```

```
136         (match Yojson.Safe.from_string tc.Llm_codec.arguments with
137          | `Assoc kvs ->
138             (match List.assoc_opt "name" kvs, List.assoc_opt "body" kvs with
139              | Some (`String n), Some (`String b) ->
140                 Alcotest.(check string) "arg name" "foo" n;
141                 Alcotest.(check string) "arg body" "(lambda () 1)" b
142              | _ -> Alcotest.fail "missing args")
143          | _ -> Alcotest.fail "arguments not an object")
144       | _ -> Alcotest.fail "expected one xml tool call"));
145     ("llm-codec: parse_turn_response Final", `Quick, fun () ->
146       let body =
147
          {|{"choices":[{"finish_reason":"stop","message":{"role":"assistant","content":"hello
          there"}}]}|}
148       in
149       (match Llm_codec.parse_turn_response body with
150        | Llm_codec.Final { content; _ } ->
151           Alcotest.(check string) "content" "hello there" content
152        | other -> Alcotest.fail ("expected Final, got other")));
153     ("llm-codec: parse_turn_response Need_tools via API field", `Quick, fun
          () ->
154       let body =
155
          {|{"choices":[{"finish_reason":"tool_calls","message":{"role":"assistant","content":null,"tool_calls":[{"id":"t9","func
156       in
157       (match Llm_codec.parse_turn_response body with
158        | Llm_codec.Need_tools { tool_calls; _ } ->
159           Alcotest.(check int) "one tool" 1 (List.length tool_calls);
160           (match tool_calls with
161            | [tc] -> Alcotest.(check string) "name" "read" tc.Llm_codec.name
162            | _ -> Alcotest.fail "expected one")
163        | other -> Alcotest.fail "expected Need_tools, got other"));
164     ("llm-codec: parse_turn_response Need_tools via XML fallback", `Quick,
          fun () ->
165       let xml_block =
166         ("<" ^ "tool_call>") ^ " bash "
167         ^ ("<" ^ "arg_key>command" ^ "</" ^ "arg_key>")
168         ^ ("<" ^ "arg_value>echo x" ^ "</" ^ "arg_value>")
169         ^ ("</" ^ "tool_call>")
170       in
171       let body =
172         Printf.sprintf
173
          {|{"choices":[{"finish_reason":"stop","message":{"role":"assistant","content":%s}}]}|}
174           (Yojson.Safe.to_string (`String xml_block))
175       in
176       (match Llm_codec.parse_turn_response body with
177        | Llm_codec.Need_tools { tool_calls; _ } ->
178           Alcotest.(check int) "one xml tool" 1 (List.length tool_calls);
179           (match tool_calls with
180            | [tc] -> Alcotest.(check string) "name" "bash" tc.Llm_codec.name
181            | _ -> Alcotest.fail "expected one")
182        | other -> Alcotest.fail "expected Need_tools via XML"));
183     ("llm-codec: parse_turn_response reasoning_content fallback", `Quick, fun
          () ->
184       let body =
185
          {|{"choices":[{"finish_reason":"stop","message":{"role":"assistant","content":null,"reasoning_content":"the
          real answer"}}]}|}
186       in
187       (match Llm_codec.parse_turn_response body with
188        | Llm_codec.Final { content; _ } ->
189           Alcotest.(check string) "content" "the real answer" content
190        | other -> Alcotest.fail "expected Final from reasoning_content"));
191     ("llm-codec: extract_stream_delta", `Quick, fun () ->
192       let chunk = {|{"choices":[{"delta":{"content":"tok"}}]}|} in
193       Alcotest.(check string) "delta" "tok"
194         (Option.value (Llm_codec.extract_stream_delta chunk) ~default:"");
195       Alcotest.(check bool) "done is none" true
196         (Option.is_none (Llm_codec.extract_stream_delta "[DONE]")));
197     ("llm-codec: extract_content picks complete-json content", `Quick, fun ()
          ->
198       let body =
```

```
    {|{"choices":[{"finish_reason":"stop","message":{"content":"{\"x\":1}","reasoning_content":"essay"}}]}|}
    in
    (match Llm_codec.extract_content body with
     | Ok c -> Alcotest.(check string) "complete json wins" "{\"x\":1}" c
     | Error e -> Alcotest.fail e));
]

let () =
  Alcotest.run "pricklypear-common" [
    "branch", branch_tests;
    "manifest", manifest_tests;
    "auth-types", auth_types_tests;
    "protocol", protocol_tests;
    "llm-codec", llm_codec_tests;
  ]
```

54 image/bin/main.ml

```
(** Pricklypear Image -- entry point.

    Always Eio_main + Httpun_eio + Io_eio PG. Lwt remains only under
    Lwt_eio for the optional TCP REPL. See single-image.runtime. *)

open Pricklypear_image_lib

let () =
  Eio_main.run @@ fun env ->
  Lwt_eio.with_event_loop ~clock:(Eio.Stdenv.clock env) @@ fun _token ->
  Eio.Switch.run @@ fun sw ->
  let ctx : Io_eio.Ctx.t =
    { sw; net = (Eio.Stdenv.net env :> [ `Generic ] Eio.Net.ty Eio.Std.r) }
  in
  Io_eio.with_ctx ctx (fun () ->
      Printf.eprintf
        "[boot] Eio_main + Httpun_eio + Io_eio PG\n%!";
      (try
         let cfg = Io_eio.Db.config_from_env () in
         let st =
           Io_eio.Db.make ~cfg
             ~branch_id:"00000000-0000-0000-0000-000000000001" ()
         in
         let n = Io_eio.Db.health_query_sync st in
         Printf.eprintf "[boot] Io_eio.Db health rows=%d\n%!" n
       with e ->
         Printf.eprintf "[boot] Io_eio.Db health failed: %s\n%!"
           (Printexc.to_string e));
      Boot.start_eio ~env ~sw)
```

55 image/lib/agent_discovery.ml

```
(** Public agent bootstrap documents (no secrets).

    Hard-public:
      GET /.well-known/agent.json  -- machine contract
      GET /agent.txt               -- prose instructions

    Eval stays private: POST /api/eval requires HTTP Basic or session.
    /agent (no suffix) remains the private HTML agent portal.
*)

let header_ci (headers : (string * string) list) (name : string) : string
     option =
  let lower = String.lowercase_ascii name in
  List.find_map
    (fun (k, v) ->
      if String.lowercase_ascii k = lower then Some v else None)
    headers

(** Prefer reverse-proxy headers, then Host. Never invent a password. *)
let base_url_of_headers (headers : (string * string) list) : string option =
  let host =
    match header_ci headers "x-forwarded-host" with
    | Some h -> Some (String.trim h)
    | None ->
        match header_ci headers "host" with
        | Some h -> Some (String.trim h)
        | None -> None
  in
  match host with
  | None | Some "" -> None
  | Some host ->
      let proto =
        match header_ci headers "x-forwarded-proto" with
        | Some p when String.trim p <> "" -> String.lowercase_ascii
     (String.trim p)
        | _ ->
            (* Default https for public hosts; plain http for loopback. *)
            let loopback =
              host = "localhost"
              || String.starts_with ~prefix:"localhost:" host
              || String.starts_with ~prefix:"127." host
              || String.starts_with ~prefix:"[::1]" host
            in
            if loopback then "http" else "https"
      in
      Some (proto ^ "://" ^ host)

let philosophy =
  "The image is the API. Eval Lisp on the running habitat; do not invent \
   REST endpoints for app features (todos, notes, bible, ?)."

let first_calls =
  [ "(whoami)"
  ; "(current-branch)"
  ; "(list-packages)"
  ; "(load-library \"notes\")"
  ; "(notes/doc/get \"agents\")"
  ; "(load-library \"todo\")"
  ; "(describe \"todo\")"
  ; "(apropos-docs \"todo/item\")"
  ; "(function-doc \"todo/item/add\")"
  ; "(load-library \"bible\")"
  ; "(bible/chapter \"John\" 3)"
  ]

let rules =
  [ "One Lisp expression per POST /api/eval unless you own multi-step state
     yourself."
  ; "Load libraries before calling package functions: (load-library
     \"todo\")."
```

```
  ; "Prefer a personal branch before defining things: (checkout
    \"USER/personal\")."
  ; "Two function kinds: DATA builders take explicit args and work from
    /api/eval \
    (e.g. bible/chapter, bible/chapter-html with book/chapter); HTTP
    HANDLERS \
    (*/page, */reader) use request-method/request-param and return \
    \"no active request\" outside a browser hit -- that is expected, not a
    crash."
  ; "Do not invent app-specific REST APIs -- call Lisp instead."
  ; "Never put passwords in code you write back to the image."
  ; "Unauthenticated /api/eval is rejected; /agent (HTML portal) is
    private."
  ; "Dotted rest params (a . b) are NOT supported; use fixed-arity params."
  ; "File product issues as notes in the inbox when you cannot fix OCaml; \
    do not invent REST for filing."
  ; "After (load-library \"notes\"), read the live agents note: \
    (notes/doc/get \"agents\"). It is the habitat AGENTS.md -- edit it \
    with notes/doc/upsert when agent guidance should change."
  ; "Discover selectively -- do not dump all bindings into context: \
    (list-packages), (describe \"pkg\"), (package-bindings \"pkg\"), \
    (package-exports \"pkg\"), (apropos-docs \"pat\"), (function-doc
    name)."
  ; "Build a REPL client from this document: HTTP Basic + POST /api/eval \
    JSON {\"expr\":\"?\"}. Prefer that over inventing REST. curl and
    Python \
    examples are below; local hosts may use the pp-eval CLI."
  ; "Knowledge wiki (Karpathy loop): (wiki/ingest-from-source ?), \
    (wiki/query ?), (wiki/lint); wiki links are [[slug]] on outline
    bodies."
  ; "Binary media (images/PDF): do NOT push multi-KiB bytes through
    /api/eval \
    (64 KiB cap). Use POST /api/media (Basic auth; raw body, JSON
    data_b64, or \
    multipart field name=file; up to 5 MiB) or scripts/pp_media.py.
    Browser UI: \
    GET /media + POST /media/upload. Serve via GET /media/<id>. Meta via \
    (media-meta id) / (media-list); tiny fixtures only via (media-put ?)."
  ]

let curl_example (base : string) : string =
  Printf.sprintf
    "curl -sS -u USER:PASS -H 'Content-Type: application/json' \
-H 'Accept: application/json' \
-d '{\"expr\":\"(whoami)\"}' %s/api/eval"
    base

let python_example (base : string) : string =
  Printf.sprintf
{|import base64, json, urllib.request
base = %S
user, password = "USER", "PASS"
expr = "(whoami)"
req = urllib.request.Request(
    base.rstrip("/") + "/api/eval",
    data=json.dumps({"expr": expr}).encode(),
    method="POST",
    headers={
        "Content-Type": "application/json",
        "Accept": "application/json",
        "Authorization": "Basic " + base64.b64encode(
            f"{user}:{password}".encode()).decode("ascii"),
    },
)
print(urllib.request.urlopen(req).read().decode())
|}
    base

let agent_json ?(base_url : string option = None) () : Yojson.Safe.t =
  let base_field =
    match base_url with
    | Some b -> [ ("base_url", `String b) ]
    | None ->
```

```
      [ ( "base_url_note"
        , `String
            "Set base_url to the origin you fetched this document from \
             (scheme + host)." ) ]
in
let example_base =
  match base_url with Some b -> b | None -> "https://pricklypear.rocks"
in
`Assoc
  ( [ ("name", `String "pricklypear")
    ; ("version", `Int 3)
    ; ( "philosophy"
      , `String philosophy )
    ; ( "live_docs"
      , `Assoc
          [ ( "agents_note"
            , `Assoc
                [ ("slug", `String "agents")
                ; ( "how"
                  , `String
                      "(load-library \"notes\") then \
                       (notes/doc/get \"agents\"). Living AGENTS.md \
                       for this habitat; agents may notes/doc/upsert it."
                  )
                ; ("browser_path", `String "/notes?slug=agents")
                ] )
          ; ( "wiki_schema"
            , `Assoc
                [ ("slug", `String "wiki-schema")
                ; ( "how"
                  , `String
                      "(wiki/schema) or (notes/doc/get \"wiki-schema\"). \
                       LLM preamble for wiki ingest/query/lint."
                  )
                ] )
          ] )
    ]
  @ base_field
  @ [ ( "auth"
      , `Assoc
          [ ("scheme", `String "HTTP Basic")
          ; ( "realm"
            , `String
                "image credentials table (same username/password as
 /login)" )
          ; ( "how_to_get_credentials"
            , `String
                "Human supplies username + password once. Create users at
 \
                 /admin/users after bootstrap login. Do not scrape
 passwords \
                 from this document -- there are none." )
          ; ("login_url", `String "/login")
          ; ( "session_cookie"
            , `String
                "Browser clients may use cookie pp_session (HttpOnly)
 after \
                 POST /login; agents usually use Basic auth." )
          ; ( "notes"
            , `List
                [ `String "Use the image password, not nginx/edge
 htpasswd."
                ; `String "No OAuth/JWT in v1."
                ; `String "Never log or commit the password."
                ] )
          ] )
    ; ( "endpoints"
      , `Assoc
          [ ( "eval"
            , `Assoc
                [ ("method", `String "POST")
                ; ("path", `String "/api/eval")
                ; ("auth", `String "required")
```

```
                ; ("content_type", `String "application/json")
                ; ( "request"
                  , `Assoc [ ("expr", `String "(+ 1 2)") ] )
                ; ( "response_ok"
                  , `Assoc
                      [ ("ok", `Bool true)
                      ; ("value", `String "3")
                      ] )
                ; ( "response_err"
                  , `Assoc
                      [ ("ok", `Bool false)
                      ; ("error", `String "unbound: foo")
                      ] )
                ] )
        ; ( "health"
          , `Assoc
              [ ("method", `String "GET")
              ; ("path", `String "/health")
              ; ("auth", `String "none")
              ] )
        ; ( "media_upload"
          , `Assoc
              [ ("method", `String "POST")
              ; ("path", `String "/api/media")
              ; ("auth", `String "required")
              ; ( "content_type"
                , `String
                    "image/*|application/pdf (raw body);
application/json with data_b64; \
                     or multipart/form-data field name=file" )
              ; ( "max_bytes"
                , `Int 5242880 )
              ; ( "headers"
                , `Assoc
                    [ ( "X-Filename"
                      , `String "optional original filename (raw body
mode)" )
                    ] )
              ; ( "response_ok"
                , `Assoc
                    [ ("ok", `Bool true)
                    ; ("id", `String "uuid")
                    ; ("url", `String "/media/<uuid>")
                    ; ("content_type", `String "image/png")
                    ; ("byte_size", `Int 1234)
                    ; ("sha256", `String "hex")
                    ] )
              ; ( "client"
                , `String "scripts/pp_media.py put ./shot.png" )
              ] )
        ; ( "media_get"
          , `Assoc
              [ ("method", `String "GET")
              ; ("path", `String "/media/<uuid>")
              ; ("auth", `String "required")
              ; ("returns", `String "raw bytes with stored
Content-Type")
              ] )
        ; ( "media_list"
          , `Assoc
              [ ("method", `String "GET")
              ; ("path", `String "/api/media")
              ; ("auth", `String "required")
              ; ("returns", `String "JSON meta only (no bytes)")
              ] )
        ; ( "media_browser"
          , `Assoc
              [ ("method", `String "GET")
              ; ("path", `String "/media")
              ; ("auth", `String "session or basic")
              ; ( "upload"
                , `String "POST /media/upload multipart field
name=file" )
```

```
                  ] )
            ; ( "repl_ui"
              , `Assoc
                  [ ("method", `String "GET")
                  ; ("path", `String "/repl")
                  ; ("auth", `String "session or basic")
                  ] )
            ; ( "discovery_json"
              , `Assoc
                  [ ("method", `String "GET")
                  ; ("path", `String "/.well-known/agent.json")
                  ; ("auth", `String "none")
                  ] )
            ; ( "discovery_text"
              , `Assoc
                  [ ("method", `String "GET")
                  ; ("path", `String "/agent.txt")
                  ; ("auth", `String "none")
                  ] )
            ] )
       ; ( "bootstrap"
         , `Assoc
             [ ( "first_calls"
               , `List (List.map (fun s -> `String s) first_calls) )
             ; ("rules", `List (List.map (fun s -> `String s) rules))
             ; ( "discovery"
               , `Assoc
                   [ ( "list_packages"
                     , `String "(list-packages) ? top-level package
     prefixes" )
                   ; ( "describe"
                     , `String
                         "(describe \"todo\") or (describe
     \"todo/item/add\")" )
                   ; ( "package_bindings"
                     , `String
                         "(package-bindings \"todo\") ? all names under
     todo/" )
                   ; ( "package_exports"
                     , `String
                         "(package-exports \"todo\") ? shallow pkg/local
     names" )
                   ; ( "apropos_docs"
                     , `String
                         "(apropos-docs \"todo/item\") ? ((name doc) ?)" )
                   ; ( "function_doc"
                     , `String "(function-doc \"name\") ? docstring | ()" )
                   ; ( "repl_client"
                     , `String
                         "Build client: HTTP Basic + POST /api/eval JSON
     {expr}. See examples.curl / examples.python; local pp-eval CLI." )
                   ] )
             ] )
       ; ( "examples"
         , `Assoc
             [ ("curl", `String (curl_example example_base))
             ; ("python", `String (python_example example_base))
             ] )
       ; ( "related"
         , `Assoc
             [ ("welcome", `String "/welcome")
             ; ("source_browser", `String "/code")
             ; ("book_pdf", `String "/book.pdf")
             ; ("source_tarball", `String "/src.tgz")
             ; ("login", `String "/login")
             ; ( "agent_portal"
               , `String
                   "/agent (private HTML UI -- not this discovery document)"
     )
             ] )
       ] )

let agent_markdown ?(base_url : string option = None) () : string =
```

```
  let origin =
    match base_url with
    | Some b -> b
    | None -> "https://pricklypear.rocks"
  in
  let first =
    first_calls
    |> List.map (fun s -> "- `" ^ s ^ "`")
    |> String.concat "\n"
  in
  let rule_lines =
    rules
    |> List.mapi (fun i r -> string_of_int (i + 1) ^ ". " ^ r)
    |> String.concat "\n"
  in
  String.concat "\n"
    [ "# Pricklypear agent instructions"
    ; ""
    ; philosophy
    ; ""
    ; "## Origin"
    ; ""
    ; "Base URL (this host): " ^ origin
    ; "Machine contract: GET " ^ origin ^ "/.well-known/agent.json"
    ; ""
    ; "## Auth"
    ; ""
    ; "Scheme: HTTP Basic against the image credentials table"
    ; "(same username/password as " ^ origin ^ "/login)."
    ; ""
    ; "How to get credentials: ask the human once. Store them in a"
    ; "local secrets file (mode 600). This document never contains"
    ; "passwords. Create users at /admin/users after bootstrap login."
    ; ""
    ; "Notes:"
    ; "- Use the image password, not nginx/edge htpasswd."
    ; "- No OAuth/JWT in v1."
    ; "- Browser clients may use cookie pp_session after POST /login;"
    ; "  agents usually use Basic auth."
    ; ""
    ; "## Eval API"
    ; ""
    ; "POST " ^ origin ^ "/api/eval"
    ; "Content-Type: application/json"
    ; "Accept: application/json"
    ; "Authorization: Basic <base64(user:pass)>"
    ; ""
    ; "Request body:"
    ; ""
    ; "    {\"expr\":\"(+ 1 2)\"}"
    ; ""
    ; "Response:"
    ; ""
    ; "    {\"ok\": true, \"value\": \"3\"}"
    ; "    {\"ok\": false, \"error\": \"unbound: foo\"}"
    ; ""
    ; "Unauthenticated calls are rejected (401)."
    ; "Errors are plain strings (arity mismatch, unbound, no active
     request)."
    ; ""
    ; "## First calls"
    ; ""
    ; first
    ; ""
    ; "## Two kinds of functions"
    ; ""
    ; "1. DATA / builders -- take explicit arguments; safe from /api/eval:"
    ; "     (bible/chapter \"John\" 3)"
    ; "     (bible/chapter-html \"John\" 3 ())"
    ; "     (todo/list) after (load-library \"todo\")"
    ; "2. HTTP HANDLERS -- read (request-method)/(request-param ...);"
    ; "     from /api/eval they return: no active request"
    ; "     Examples: bible/chapter-reader, todo/page, notes/page, */page."
```

```
    ; "     That message is expected. Call data functions, or hit the route"
    ; "     in a browser / with a real HTTP request."
    ; ""
    ; "## Selective discovery (REPL)"
    ; ""
    ; "Do not dump (list-bindings) into the model context (hundreds of
    names)."
    ; "Expand one package or name at a time:"
    ; ""
    ; "  (list-packages)"
    ; "  (load-library \"todo\")"
    ; "  (describe \"todo\")              ; package summary + shallow
    exports"
    ; "  (package-exports \"todo\")       ; shallow pkg/local names"
    ; "  (package-bindings \"todo\")      ; ALL names under todo/ (any
    depth)"
    ; "  (apropos-docs \"todo/item\")     ; ((name doc) ?)"
    ; "  (function-doc \"todo/item/add\")"
    ; "  (function-source \"todo/item/add\")"
    ; "  (function-ast \"todo/item/add\")"
    ; "  (read-string \"(lambda (x) x)\")  ; parse LLM Lisp text to data"
    ; "  (package-exports \"std\")         ; frozen kernel/std surface"
    ; ""
    ; "Human browser: " ^ origin ^ "/ref (packages) or " ^ origin ^
    "/ref?q=todo"
    ; ""
    ; "## How to build a REPL client"
    ; ""
    ; "The image is the API. Your client only needs:"
    ; "  1. Base URL (this host)"
    ; "  2. HTTP Basic username/password (from a human; never in this file)"
    ; "  3. POST /api/eval with JSON body {\"expr\":\"(whoami)\"}"
    ; "  4. Parse JSON {ok, value|error}"
    ; ""
    ; "curl:"
    ; ""
    ; "````"
    ; curl_example origin
    ; "````"
    ; ""
    ; "Python:"
    ; ""
    ; "````"
    ; python_example origin
    ; "````"
    ; ""
    ; "Local laptop: OCaml pp-eval CLI (see repo skill pricklypear-eval) or"
    ; "scripts/pp-eval.sh after sourcing ~/.config/pricklypear/env."
    ; ""
    ; "## Symbols vs strings (code-as-data)"
    ; ""
    ; "Quoted bare names are symbols, not strings:"
    ; "  (symbol? 'lambda) => #t"
    ; "  (string? \"lambda\") => #t"
    ; "  (symbol=? (car (read-string \"(lambda (x) x)\")) 'lambda) => #t"
    ; "Walk trees with symbol=?, not string-eq on car."
    ; "  (string-eq (symbol->string (car form)) \"lambda\") is OK if
    needed."
    ; "CRUD still accepts 'type-name (symbols coerced at the DB boundary)."
    ; ""
    ; "## Application libraries (load-library)"
    ; ""
    ; "- std, ui, layout, welcome -- shell / HTML helpers"
    ; "- todo, notes, outline, plan, contacts, bookmarks, chat"
    ; "- calendar, flashcards, bible, dabar, weather, signup, ai"
    ; "Cold image after restart: (load-library \"NAME\") before calling."
    ; ""
    ; "## Live agents note (habitat AGENTS.md)"
    ; ""
    ; "After (load-library \"notes\"), read:"
    ; ""
    ; "    (notes/doc/get \"agents\")"
    ; ""
```

```
    ; "Browser: " ^ origin ^ "/notes?slug=agents"
    ; "This note is the living agent playbook for this habitat. Agents"
    ; "SHOULD update it (notes/doc/upsert) when they learn durable"
    ; "conventions, new libraries, or wiki workflow changes -- same as"
    ; "editing AGENTS.md in a git repo, but durable in Postgres on the"
    ; "current branch. Also see (notes/doc/get \"wiki-schema\") for the"
    ; "wiki LLM preamble."
    ; ""
    ; "## Notes / wiki (image is the API)"
    ; ""
    ; "    (load-library \"notes\")"
    ; "    (notes/doc/list)"
    ; "    (notes/doc/get \"pricklypear\")"
    ; "    (roam/backlinks \"pricklypear\")"
    ; "    (wiki/ingest-from-source \"?raw source text?\")"
    ; "    (wiki/query \"?\")"
    ; "    (wiki/lint)"
    ; ""
    ; "Outline nodes support op=eval (Lisp with self/doc/slug bound)."
    ; "Wiki links in bodies: [[slug]] or [[slug|label]]."
    ; ""
    ; "## Media upload (images / PDF)"
    ; ""
    ; "Do NOT push multi-KiB binaries through POST /api/eval -- the eval"
    ; "body cap is 64 KiB. Use the dedicated media endpoints instead."
    ; ""
    ; "### Write (auth required -- HTTP Basic or session)"
    ; ""
    ; "POST " ^ origin ^ "/api/media"
    ; "  Max body: 5 MiB."
    ; "  Allowed Content-Types:"
    ; "    image/png image/jpeg image/gif image/webp image/svg+xml"
    ; "    application/pdf"
    ; ""
    ; "Body modes:"
    ; "  1. Raw binary -- set Content-Type to the media type;"
    ; "     optional header X-Filename: original-name.png"
    ; "  2. JSON -- Content-Type: application/json"
    ; "       {\"content_type\":\"image/png\",\"data_b64\":\"?\","
    ; "        \"filename\":\"shot.png\"}   // filename optional"
    ; "  3. Multipart -- Content-Type: multipart/form-data"
    ; "       field name=file (or name=media) with filename + part"
    ; "       Content-Type"
    ; ""
    ; "Success (201):"
    ; "  {\"ok\":true,\"id\":\"<uuid>\",\"url\":\"/media/<uuid>\","
    ; "   \"content_type\":\"image/png\",\"byte_size\":N,\"sha256\":\"?\"}"
    ; ""
    ; "### Read / list / delete"
    ; ""
    ; "GET  " ^ origin ^ "/media/<uuid>     ? raw bytes (auth required)"
    ; "GET  " ^ origin ^ "/api/media        ? JSON meta list (no bytes)"
    ; "DELETE " ^ origin ^ "/api/media/<uuid> ? author or admin"
    ; "GET  " ^ origin ^ "/media           ? browser gallery + upload form"
    ; "POST " ^ origin ^ "/media/upload    ? browser multipart form"
    ; ""
    ; "### Client helper"
    ; ""
    ; "    # same env as pp_eval: PP_URL PP_USER PP_PASS"
    ; "    python3 scripts/pp_media.py put ./shot.png"
    ; "    python3 scripts/pp_media.py get <uuid> -o out.png"
    ; "    python3 scripts/pp_media.py list"
    ; "    python3 scripts/pp_media.py delete <uuid>"
    ; ""
    ; "### curl (raw PNG)"
    ; ""
    ; "    curl -sS -u USER:PASS -X POST " ^ origin ^ "/api/media \\"
    ; "      -H 'Content-Type: image/png' -H 'X-Filename: shot.png' \\"
    ; "      --data-binary @shot.png"
    ; ""
    ; "### curl (multipart)"
    ; ""
    ; "    curl -sS -u USER:PASS -F 'file=@shot.png;type=image/png' \\"
```

```
    ; "      " ^ origin ^ "/api/media"
    ; ""
    ; "### Lisp (meta only -- not bulk upload)"
    ; ""
    ; "    (media-meta \"<uuid>\")     ; JSON meta or ()"
    ; "    (media-list)               ; your recent meta"
    ; "    (media-url \"<uuid>\")      ; \"/media/<uuid>\""
    ; "    (media-delete \"<uuid>\")"
    ; "    (media-put \"image/png\" b64 [filename])  ; ?256 KiB only"
    ; ""
    ; "Embed in a note node body (wiki syntax):"
    ; "  [[media:<uuid>]]"
    ; "  [[media:<uuid>|alt text]]"
    ; "Renders <img src=\"/media/<uuid>\"> when the note is viewed (auth"
    ; "required to fetch the image). Ownership is the uploading author."
    ; ""
    ; "Reference media from notes/HTML with the returned /media/<id> URL"
    ; "(auth required to fetch)."
    ; ""
    ; "## Rules"
    ; ""
    ; rule_lines
    ; ""
    ; "## curl example"
    ; ""
    ; "    " ^ curl_example origin
    ; ""
    ; "BusyBox wget often lacks --user/--password; prefer curl or python."
    ; ""
    ; "## python example"
    ; ""
    ; "```python"
    ; python_example origin
    ; "```"
    ; ""
    ; "## Filing issues"
    ; ""
    ; "Prefer notes inbox (notes/doc/ensure-inbox + outline nodes) over"
    ; "inventing bug-tracker REST. Keep OCaml-kernel bugs precise with"
    ; "repro expressions. Durable agent guidance belongs in the live"
    ; "agents note (slug agents), not only in git AGENTS.md."
    ; ""
    ; "## Related public surfaces"
    ; ""
    ; "- Welcome: " ^ origin ^ "/welcome"
    ; "- Source browser: " ^ origin ^ "/code"
    ; "- Book PDF: " ^ origin ^ "/book.pdf"
    ; "- Health: " ^ origin ^ "/health"
    ; "- Interactive REPL UI (auth required): " ^ origin ^ "/repl"
    ; "- Private agent portal (auth required): " ^ origin ^ "/agent"
    ; "- Media gallery / upload (auth required): " ^ origin ^ "/media"
    ; ""
    ; "## What not to do"
    ; ""
    ; "- Do not open or rely on the raw TCP REPL port from the internet."
    ; "- Do not invent per-app REST endpoints."
    ; "- Do not put secrets into (define ?) forms or public routes."
    ; "- Do not treat \"no active request\" as a host crash -- call a data
     fn."
    ; "- Do not base64 multi-MiB images into /api/eval -- use POST
     /api/media."
    ; ""
    ]

let serve_agent_json (reqd : Httpun.Reqd.t)
    ~(headers : (string * string) list) : unit =
  let base_url = base_url_of_headers headers in
  let j = agent_json ~base_url () in
  Http_core.text_response reqd
    ~content_type:"application/json; charset=utf-8"
    (Yojson.Safe.pretty_to_string j)

let serve_agent_txt (reqd : Httpun.Reqd.t)
```

```
    ~(headers : (string * string) list) : unit =
  let base_url = base_url_of_headers headers in
  let body = agent_markdown ~base_url () in
  Http_core.text_response reqd
    ~content_type:"text/plain; charset=utf-8" body
```

56 image/lib/agent_tools.ml

```
(** AI agent tools, propose/install, LLM rewrite/growth loops.
    Operates on [Session_types.t]; re-exported from [Session]. *)
open Session_types

(** Human-readable message from evaluator/reader exceptions (for agent tools
    and POST /api/eval). Delegates to [Ast.pp_interp_error] so every
    [interp_error] variant is covered -- never emit opaque
     [Interp_error(_)]. *)
let pp_interp_exn (exn : exn) : string =
  match exn with
  | Pricklypear_lisp.Ast.Interp_error e ->
      Pricklypear_lisp.Ast.pp_interp_error e
  | Pricklypear_lisp.Reader.Reader_error (m, l, c) ->
      Printf.sprintf "parse error: %s (%d:%d)" m l c
  | _ -> Printexc.to_string exn

(** Resolve per-user LLM key (auth.user-llm-keys); env fallback inside
     client. *)
let with_user_llm (t : t) (f : unit -> 'a) : 'a =
  let uid = t.manifest.user_id in
  if uid = "" || uid = "anonymous" || uid = "public" then f ()
  else
    let secret = Auth_store.get_llm_key_secret t.store ~user_id:uid in
    match secret with
    | None -> f ()
    | Some (key, meta) ->
        let base =
          match meta.Auth_store.base with
          | Some b when b <> "" -> b
          | _ -> Llm_client.api_base ()
        in
        let model =
          match meta.Auth_store.model with
          | Some m when m <> "" -> m
          | _ -> Llm_client.model ()
        in
        let creds =
          { Llm_client.api_key = key; api_base = base; model }
        in
        Llm_client.with_creds creds f

let ensure_callable_body (body_src : string) : string =
  try
    let e = Pricklypear_lisp.Reader.read_one body_src in
    match e with
    | Pricklypear_lisp.Ast.App (Pricklypear_lisp.Ast.Var "lambda", _) ->
     body_src
    | Pricklypear_lisp.Ast.Lambda _ -> body_src
    | _ -> Printf.sprintf "(lambda (x) %s)" body_src
  with _ -> body_src

(* Save/define a binding from source body text. Used by function-editor
   HTTP and AI propose. [author] is recorded on function_defs. *)
let define_binding (t : t) ~(name : string) ~(body_src : string)
    ~(author : string)
    : (Pricklypear_lisp.Ast.value, string) result =
  let body_src = ensure_callable_body body_src in
  let input = Printf.sprintf "(define %s %s)" name body_src in
  (try
      let expr = Pricklypear_lisp.Reader.read_one input in
      let v =
        Control_rim.run_lwt ~prims:t.prims (fun () ->
          Pricklypear_lisp.Eval.eval_string_direct
            ~prims:Pricklypear_lisp.Eval.empty_prims t.env input)
      in
      (match Pricklypear_lisp.Eval.parse_top_define expr with
       | Some { name = n; body = body_expr; doc } ->
           let () =
             Store.persist_function t.store n body_expr ~doc author
           in
           (* Stale chrome nav was the "bible only on home" bug: home
```

```
69              live-calls ui/nav; other pages used a cached string. *)
70           if n = "ui/nav" || n = "ui/home" || n = "ui/css" || n =
     "ui/theme"
71              || String.starts_with ~prefix:"theme/" n then (
72             t.ui_nav_html <- None;
73             t.ui_css <- None);
74           (* Impact + change transcript filled by Session after define when
75              called via Session.eval paths; install_function appends text.
     *)
76           (Ok v)
77       | None ->
78           (* Still bound in env if define special-form ran *)
79           (Ok v))
80     with exn ->
81       let msg = pp_interp_exn exn in
82       (Error msg))
83
84 (* First-run recipe: hello page function + route. No Lisp knowledge
     required. *)
85 (* Optional elevate: PP_AI_ALLOW_WRITE=1 allows WriteDb/Publish/etc. *)
86 let ai_allow_set () =
87   match Sys.getenv_opt "PP_AI_ALLOW_WRITE" with
88   | Some ("1" | "true" | "TRUE" | "yes") ->
89       Pricklypear_lisp.Effect.EffSet.normalize
90         [ Pricklypear_lisp.Effect.ReadDb
91         ; Pricklypear_lisp.Effect.WriteDb
92         ; Pricklypear_lisp.Effect.Html
93         ; Pricklypear_lisp.Effect.Publish
94         ; Pricklypear_lisp.Effect.Ai
95         ]
96   | _ -> Pricklypear_lisp.Effect.ai_default_allow
97
98 (** Derive-on-read [known_fn] from branch-visible function_defs (H0.2). *)
99 let known_fn_from_store (t : t)
100   : Pricklypear_lisp.Effect.known_fn_lookup =
101   let rows = Store.list_function_names t.store in
102   let bindings = List.map (fun (n, e, _) -> (n, e)) rows in
103   (Pricklypear_lisp.Effect.known_fn_of_bindings bindings)
104
105 let check_ai_body_effects ?(known_fn = fun _ -> None)
106     ?env (body_src : string) : (unit, string) result =
107   try
108     let expr = Pricklypear_lisp.Reader.read_one body_src in
109     (* Expand macros before infer when an env is available
      (BORGE-1785212882).
110        Without env, fall back to unexpanded -- install paths that have a
111        session should pass ~env.
112        If expansion FAILS we must NOT fall back to the unexpanded tree:
113        a macro could hide a write-db effect behind an expansion that the
114        gate would then miss. Surface the error so the install is denied. *)
115     let expanded =
116       match env with
117       | None -> Ok expr
118       | Some e ->
119           (try Ok (Pricklypear_lisp.Eval.macroexpand_expr e expr)
120            with
121            | Pricklypear_lisp.Ast.Interp_error err ->
122                Error
123                  (Printf.sprintf "macroexpand failed: %s"
124                     (Pricklypear_lisp.Ast.pp_interp_error err))
125            | exn ->
126                Error
127                  (Printf.sprintf "macroexpand failed: %s"
128                     (Printexc.to_string exn)))
129     in
130     match expanded with
131     | Error msg -> Error msg
132     | Ok expr ->
133         let fx = Pricklypear_lisp.Effect.infer ~known_fn expr in
134         if Pricklypear_lisp.Effect.policy_allows ~allow:(ai_allow_set ())
      fx then
135           Ok ()
136         else
```

```
          Error
            (Printf.sprintf
               "effect policy denied (%s); allow pure|read-db|write-db|html by default; use register_route tool for routes (not (route ...) in body); set PP_AI_ALLOW_WRITE=1 for branch/publish elevate"
               (Pricklypear_lisp.Effect.EffSet.pp fx))
  with _ -> Ok ()  (* parse errors handled by define_binding *)

(* [agent] temporary alias defined after agent_session_new / agent_say. *)

(** Install a Lisp body as [name] with AI provenance (parse + effect gate). *)
let install_ai_function (t : t) ~(name : string) ~(body_src : string)
    : Pricklypear_lisp.Ast.value =
  match Llm_client.extract_lisp_sexpr body_src with
  | Error e ->
      let preview =
        let s = String.trim body_src in
        if String.length s > 240 then String.sub s 0 240 ^ "?" else s
      in
      (Pricklypear_lisp.Ast.VStr
           (Printf.sprintf "ai body invalid: %s\n---\n%s" e preview))
  | Ok body_src ->
      let known_fn = known_fn_from_store t in
      (match check_ai_body_effects ~known_fn ~env:t.env body_src with
       | Error pe ->
           (Pricklypear_lisp.Ast.VStr
                (Printf.sprintf "ai rejected by effects: %s\n---\n%s" pe body_src))
       | Ok () ->
           let author = "agent:" ^ Llm_client.active_model () in
           let def = define_binding t ~name ~body_src ~author in
           (match def with
            | Ok _ ->
                let () = Call_graph.record_define_side_effects t ~name in
                let badge =
                  try
                    let e = Pricklypear_lisp.Reader.read_one body_src in
                    Pricklypear_lisp.Effect.EffSet.badge
                      (Pricklypear_lisp.Effect.infer ~known_fn e)
                  with _ -> "?"
                in
                let impact = Call_graph.impact_summary_text t name in
                (Pricklypear_lisp.Ast.VStr
                     (Printf.sprintf "installed %s by %s [%s]\n%s\n%s"
                        name author badge body_src impact))
            | Error e ->
                (Pricklypear_lisp.Ast.VStr
                     (Printf.sprintf "ai body invalid: %s\n---\n%s" e body_src))))

(** Pretty source body for a binding (env lambda or store).
    Special forms are not stored; return a short language card instead. *)
let function_source (t : t) (name : string) : string option =
  let found = Store.lookup_function t.store name in
  match found with
  | Some (expr, _, _) ->
      (Some (Pricklypear_lisp.Reader.pp_expr expr))
  | None ->
      (match Pricklypear_lisp.Env.lookup t.env name with
       | Some (Pricklypear_lisp.Ast.VLambda (params, body, _)) ->
           let e =
             Pricklypear_lisp.Ast.Lambda (params, body)
           in
           (Some (Pricklypear_lisp.Reader.pp_expr e))
       | _ ->
           (match Pricklypear_lisp.Special_forms.find name with
            | Some e ->
                let card =
                  Printf.sprintf
                    ";; special form (evaluator) -- not a Lisp binding\n;; %s\n;; %s\n%s"
                    e.arity e.summary e.examples
```

```
                in
                (Some card)
            | None -> None))

(** Raw function body as quoted data (no eval, no pretty-print).
    Store body (or env lambda) reified via [Eval.quote_to_value] so the
    result is a first-class Lisp value tree agents can walk/rewrite.
    Missing / special-form / non-lambda ? None. *)
let function_ast (t : t) (name : string)
  : Pricklypear_lisp.Ast.value option =
  let found = Store.lookup_function t.store name in
  match found with
  | Some (expr, _, _) ->
      (Some (Pricklypear_lisp.Eval.quote_to_value expr))
  | None ->
      (match Pricklypear_lisp.Env.lookup t.env name with
       | Some (Pricklypear_lisp.Ast.VLambda (params, body, _)) ->
           let e = Pricklypear_lisp.Ast.Lambda (params, body) in
           (Some (Pricklypear_lisp.Eval.quote_to_value e))
       | Some (Pricklypear_lisp.Ast.VMacro (params, body, _)) ->
           (* Same shape as lambda; tagged head so agents can tell macros
    apart. *)
           let v =
             Pricklypear_lisp.Ast.VList
               [ Pricklypear_lisp.Ast.VStr "defmacro"
               ; Pricklypear_lisp.Ast.VList
                   (List.map (fun p -> Pricklypear_lisp.Ast.VStr p) params)
               ; Pricklypear_lisp.Eval.quote_to_value body
               ]
           in
           (Some v)
       | _ -> None)

(** Names in package [pkg].

    Match policy (agent discovery):
    - [pkg] empty / root ? names with no [/]
    - else ? exact [name = pkg] OR prefix [pkg/?] (any depth).

    Previous implementation used [Packages.package_of_name] (last [/]
    segment parent), so [(package-bindings "todo")] missed
    [todo/item/add] (parent [todo/item]). Prefix match is what
    agent.txt and explorers expect. *)
let package_binding_names (t : t) (pkg : string) : string list =
  let defs = Store.list_function_names t.store in
  let pkg =
    let p = String.trim pkg in
    if p = "(root)" || p = "root" then "" else p
  in
  let matches name =
    if pkg = "" then not (String.contains name '/')
    else if name = pkg then true
    else
      let prefix = pkg ^ "/" in
      let plen = String.length prefix in
      String.length name >= plen && String.sub name 0 plen = prefix
  in
  let names =
    List.filter_map
      (fun (name, _, _) -> if matches name then Some name else None)
      defs
    |> List.sort_uniq String.compare
  in
  names

(** Public export names for [pkg].
    Resolution: declared package_registry exports (if non-empty) ?
    shallow heuristic ([pkg] or [pkg/local] with no further [/]).
    Kernel/std freeze is applied by the session prim wrapper. *)
let package_export_names (t : t) (pkg : string) : string list =
  let pkg = String.trim pkg in
  let pkg = if pkg = "(root)" || pkg = "root" then "" else pkg in
  let reg =
```

```
    if pkg = "" then None else Store.lookup_package t.store pkg
  in
  match reg with
  | Some r when r.exports <> [] -> r.exports
  | _ ->
      let names = package_binding_names t pkg in
      if pkg = "" then names
      else
        let prefix = pkg ^ "/" in
        let plen = String.length prefix in
        let shallow =
          List.filter
            (fun name ->
              name = pkg
              || (String.length name > plen
                  && String.sub name 0 plen = prefix
                  && not (String.contains_from name plen '/')))
            names
        in
        shallow

(** Unique top-level package prefixes from visible function_defs. *)
let list_package_names (t : t) : string list =
  let defs = Store.list_function_names t.store in
  let tbl = Hashtbl.create 32 in
  List.iter
    (fun (name, _, _) ->
      match String.index_opt name '/' with
      | None -> ()
      | Some i ->
          let p = String.sub name 0 i in
          if p <> "" then Hashtbl.replace tbl p ())
    defs;
  let pkgs =
    Hashtbl.fold (fun k _ acc -> k :: acc) tbl []
    |> List.sort String.compare
  in
  pkgs

let name_contains (name : string) (pat : string) : bool =
  if pat = "" then true
  else
    let n = String.length name and p = String.length pat in
    let rec loop i =
      if i + p > n then false
      else if String.sub name i p = pat then true
      else loop (i + 1)
    in
    loop 0

(** [(name, doc)] for visible defs whose name contains [pat].
    Doc falls back to empty string when unset (still listed). *)
let apropos_docs (t : t) (pat : string) : (string * string) list =
  let defs = Store.list_function_names t.store in
  let pat = String.trim pat in
  let names =
    List.filter_map
      (fun (name, _, _) ->
        if name_contains name pat then Some name else None)
      defs
    |> List.sort_uniq String.compare
  in
  let rec go acc = function
    | [] -> (List.rev acc)
    | name :: rest ->
        let doc_opt = Store.lookup_function_doc t.store name in
        let doc =
          match doc_opt with
          | Some s -> s
          | None ->
              (match Pricklypear_lisp.Special_forms.find name with
               | Some e -> e.summary
               | None ->
```

```
                  let b = Primitives.prim_summary name in
                  if b = "built-in primitive" then "" else b)
        in
        go ((name, doc) :: acc) rest
  in
  go [] names

(** Package or binding description for selective agent reveal. *)
let describe (t : t) (needle : string) : string =
  let needle = String.trim needle in
  if needle = "" then
    "describe: need name or package string"
  else
    let pkgs = list_package_names t in
    let pkg_names = package_binding_names t needle in
    let is_pkg =
      List.mem needle pkgs || List.length pkg_names > 0
    in
    (* Prefer exact binding describe when name is a concrete def and not
       only a package prefix with many children -- if needle equals a
       binding name, show that binding. *)
    let exact_doc = Store.lookup_function_doc t.store needle in
    let exact_src = function_source t needle in
    let has_exact = match exact_src with Some _ -> true | None -> false in
    if has_exact && not (List.length pkg_names > 1 && List.mem needle pkgs)
    then
      let doc =
        match exact_doc with
        | Some s -> s
        | None ->
            (match Pricklypear_lisp.Special_forms.find needle with
             | Some e -> e.summary
             | None ->
                 let b = Primitives.prim_summary needle in
                 if b = "built-in primitive" then "(no docstring)" else b)
      in
      (Printf.sprintf
           "%s\n  doc: %s\n  next: (function-source %S) (function-ast %S)"
           needle doc needle needle)
    else if is_pkg then
      let exports = package_export_names t needle in
      let pairs =
        let rec go acc = function
          | [] -> (List.rev acc)
          | name :: rest ->
              let d = Store.lookup_function_doc t.store name in
              let doc = match d with Some s -> s | None -> "" in
              go ((name, doc) :: acc) rest
        in
        go [] exports
      in
      let lines =
        List.map
          (fun (n, d) ->
            if d = "" then Printf.sprintf "  - %s" n
            else Printf.sprintf "  - %s -- %s" n d)
          pairs
      in
      let body =
        Printf.sprintf
          "package %s (%d bindings, %d shallow exports)\n\
           shallow exports (package-exports):\n%s\n\
           full list: (package-bindings %S)\n\
           search docs: (apropos-docs %S)\n\
           one binding: (describe \"name\") or (function-doc \"name\")"
          needle (List.length pkg_names) (List.length exports)
          (if lines = [] then "  (none)" else String.concat "\n" lines)
          needle (needle ^ "/")
      in
      body
    else
      (* Fallback: treat as apropos hit list *)
      let pairs = apropos_docs t needle in
```

```
      if pairs = [] then
        (Printf.sprintf
             "%s: not a binding or package. Try (apropos %S) or
    (list-packages)."
             needle needle)
      else
        let lines =
          List.map
            (fun (n, d) ->
              if d = "" then Printf.sprintf "  - %s" n
              else Printf.sprintf "  - %s -- %s" n d)
            (let rec take n xs =
               match n, xs with
               | _, [] -> []
               | 0, _ -> []
               | n, x :: xs -> x :: take (n - 1) xs
             in
             take 30 pairs)
        in
        (Printf.sprintf
             "apropos %S (%d matches, showing up to 30):\n%s\n\
              refine: (apropos-docs %S) (describe \"pkg\")"
             needle (List.length pairs) (String.concat "\n" lines) needle)

(** Export package as lib.pp text: (define name body)? pretty-printed. *)
let export_package (t : t) (pkg : string) : string =
  let names = package_binding_names t pkg in
  let label = if pkg = "" || pkg = "(root)" then "(root)" else pkg in
  let buf = Buffer.create 4096 in
  Buffer.add_string buf
    (Printf.sprintf
       ";; package %s -- exported from live branch %s\n;; Sync:
   scripts/pp_sync.py pull|push\n\n"
       label t.manifest.branch_name);
  let reg =
    if pkg = "" || pkg = "(root)" then None
    else Store.lookup_package t.store pkg
  in
  (match reg with
   | Some r when r.exports <> [] ->
       Buffer.add_string buf (Printf.sprintf "(package %s\n  (export\n"
     pkg);
       List.iter
         (fun n -> Buffer.add_string buf (Printf.sprintf "    %s\n" n))
         r.exports;
       Buffer.add_string buf "))\n\n"
   | _ -> ());
  let rec go = function
    | [] -> (Buffer.contents buf)
    | name :: rest ->
        let src = function_source t name in
        (match src with
         | None -> go rest
         | Some body ->
             Buffer.add_string buf "(define ";
             Buffer.add_string buf name;
             Buffer.add_string buf "\n  ";
             (* indent body lines by 2 spaces after first *)
             let body_indented =
               body
               |> String.split_on_char '\n'
               |> (function
                    | [] -> ""
                    | h :: t ->
                        String.concat "\n"
                          (h :: List.map (fun line -> "  " ^ line) t))
             in
             Buffer.add_string buf body_indented;
             (* Re-inject the define-time docstring as a trailing
                define-doc form so the vault round-trip preserves the
                doc column. parse_top_define's 3-arg (trailing-doc)
                case reads it back into the doc column on reseed; trailing
                wins over any lambda-internal doc, and the trailing string
```

```
                is treated as metadata (not evaluated). (i-1787318456) *)
             (match Store.lookup_function_doc t.store name with
              | Some doc ->
                  Buffer.add_string buf "\n  ";
                  Buffer.add_string buf
                    (Pricklypear_lisp.Reader.pp_value
                       (Pricklypear_lisp.Ast.VStr doc))
              | None -> ());
             Buffer.add_string buf ")\n\n";
             go rest)
  in
  go names

(* AI propose: draft via agent portal, then install on current branch. *)
let propose_function (t : t) ~(name : string) ~(spec : string)
    : Pricklypear_lisp.Ast.value =
  with_user_llm t (fun () ->
  let seed =
    "You write Pricklypear Lisp function bodies.\n" ^
    "OUTPUT RULES (strict):\n" ^
    "1. Reply with EXACTLY one s-expression and nothing else.\n" ^
    "2. The s-expression MUST be a lambda, e.g. (lambda (x) (* x 2)).\n" ^
    "3. Do NOT write analysis, markdown, numbered lists, or prose.\n" ^
    "4. Do NOT write (define ...).\n" ^
    "5. Do NOT wrap the answer in ``` fences.\n" ^
    "Prims: + - * = < > list car cdr cons null? string-append str " ^
    "dict-get dict-set get create update delete rows render-html."
  in
  let prompt =
    Printf.sprintf
      "Name: %s\nSpec: %s\n\nYour entire reply is one lambda s-expression
     only."
      name spec
  in
  (* Low temperature; authoring is not chat. *)
  let res =
    Llm_client.agent_chat ~temperature:0.1 ~max_tokens:512
      ~seed ~prompt ()
  in
  match res with
  | Error e ->
      (Pricklypear_lisp.Ast.VStr ("agent error: " ^ e))
  | Ok body_src ->
      install_ai_function t ~name ~body_src)

(** Pretty-print a stored function body, or default stub. *)
let current_fn_pretty (t : t) (name : string) : string =
  let found = Store.lookup_function t.store name in
  match found with
  | Some (expr, _, _) ->
      (Pricklypear_lisp.Reader.pp_expr expr)
  | None ->
      (match Pricklypear_lisp.Env.lookup t.env name with
       | Some (Pricklypear_lisp.Ast.VLambda (params, body, _)) ->
           let p =
             "(lambda (" ^ String.concat " " params ^ ") " ^
             Pricklypear_lisp.Reader.pp_expr body ^ ")"
           in
           p
       | _ -> "(lambda () #f)")

(** Freeze AI rewrite into store (coding tool). mode: replace|wrap *)
(* ---- Tool-using agent loop (multi-function tasks) ---- *)

let json_string_field (j : Yojson.Safe.t) (k : string) : string option =
  match j with
  | `Assoc items ->
      (match List.assoc_opt k items with
       | Some (`String s) -> Some s
       | Some (`Int i) -> Some (string_of_int i)
       | Some (`Float f) -> Some (string_of_float f)
       | Some (`Bool b) -> Some (string_of_bool b)
       | Some other -> Some (Yojson.Safe.to_string other)
```

```
      | None -> None)
  | _ -> None

let pp_value_short (v : Pricklypear_lisp.Ast.value) : string =
  let s = Pricklypear_lisp.Reader.pp_value v in
  if String.length s > 4000 then String.sub s 0 4000 ^ "?" else s

let agent_tool_defs : Llm_client.tool_def list =
  let open Llm_client in
  let obj props required =
    `Assoc
      [ ("type", `String "object")
      ; ("properties", `Assoc props)
      ; ("required", `List (List.map (fun s -> `String s) required))
      ]
  in
  [ { name = "list_bindings"
    ; description = "List function/binding names visible in this image env"
    ; parameters = obj [] []
    }
  ; { name = "list_routes"
    ; description = "List HTTP routes and their handler function names"
    ; parameters = obj [] []
    }
  ; { name = "inspect_function"
    ; description = "Show the Lisp source body of a named function if
    present"
    ; parameters =
        obj
          [ ("name",
             `Assoc
               [ ("type", `String "string")
               ; ("description", `String "function name, e.g. demo/double
    or todo/page")
               ])
          ]
          [ "name" ]
    }
  ; { name = "install_function"
    ; description =
        "Install a Lisp function on the current branch. body must be a
    lambda s-expression only, e.g. (lambda (x) (* x 2)) or (lambda ()
    (html/page ...)). Install only parses+binds the lambda -- it does NOT
    run the body. After install: register_route (if page), then
    smoke_route and/or eval_expr until ok:true. For chrome CSS (ui/css,
    ui/theme, layout/theme) prove with eval_expr (string-contains?
    (ui/css) \"...\"). Do not put (route ...) in the body -- use
    register_route. Author becomes agent:<model>."
    ; parameters =
        obj
          [ ("name",
             `Assoc
               [ ("type", `String "string")
               ; ("description", `String "binding name to define")
               ])
          ; ("body",
             `Assoc
               [ ("type", `String "string")
               ; ("description", `String "lambda s-expression source")
               ])
          ]
          [ "name"; "body" ]
    }
  ; { name = "register_route"
    ; description =
        "Bind an HTTP path to an already-installed handler function name on
    this branch. Use after install_function of the page handler. Example:
    path=/contacts handler=contacts/page. Do NOT put (route ...) inside
    install_function bodies."
    ; parameters =
        obj
          [ ("path",
             `Assoc
```

```
              [ ("type", `String "string")
              ; ("description", `String "URL path starting with /, e.g. /contacts")
              ])
         ; ("handler",
            `Assoc
              [ ("type", `String "string")
              ; ("description", `String "function name, e.g. contacts/page")
              ])
         ]
         [ "path"; "handler" ]
    }
  ; { name = "eval_expr"
    ; description =
        "Evaluate a short Lisp expression in this image (same env as the branch). Use to prove non-route work actually runs: (count (contacts/all)), (type-of ui/css), (string-contains? (ui/css) \"#2d5a27\"), ((pkg/helper)). Install alone is not enough. Returns printed value or error. Prefer small expressions; no multi-form progn dumps."
    ; parameters =
        obj
          [ ("expr",
             `Assoc
               [ ("type", `String "string")
               ; ("description", `String "one Lisp expression")
               ])
          ]
          [ "expr" ]
    }
  ; { name = "smoke_route"
    ; description =
        "REQUIRED proof that a page runs. Call a registered user route handler in-process (no real HTTP). method GET|POST; optional form/query params as a JSON object of strings; optional contains substring that must appear in the HTML/text result (or redirect Location). Always smoke after install_function+register_route of a page handler. Example: smoke_route path=/contacts method=GET contains=Contacts. On failure: html_helpers, inspect_function, fix install_function, re-smoke until ok:true. Do not claim success without a successful smoke_route for that path."
    ; parameters =
        obj
          [ ("path",
             `Assoc
               [ ("type", `String "string")
               ; ("description", `String "route path e.g. /contacts")
               ])
          ; ("method",
             `Assoc
               [ ("type", `String "string")
               ; ("description", `String "GET or POST (default GET)")
               ])
          ; ("params",
             `Assoc
               [ ("type", `String "object")
               ; ("description", `String "form/query params as string values, e.g. {\"action\":\"create\",\"name\":\"Ada\"}")
               ])
          ; ("contains",
             `Assoc
               [ ("type", `String "string")
               ; ("description", `String "optional substring that must appear in body or redirect location")
               ])
          ]
          [ "path" ]
    }
  ; { name = "html_helpers"
    ; description =
        "Return the html/* arity cheatsheet and anti-patterns. Call this BEFORE writing page/form/list HTML with install_function. Required
```

```
      when smoke_route fails with unbound/arity/type errors in html/*."
    ; parameters = obj [] []
    }
  ; { name = "image_context"
    ; description = "Return user, branch, and a short summary of this image"
    ; parameters = obj [] []
    }
  ]

let json_object_string_pairs (j : Yojson.Safe.t) : (string * string) list =
  match j with
  | `Assoc items ->
      List.filter_map
        (fun (k, v) ->
          match v with
          | `String s -> Some (k, s)
          | `Int n -> Some (k, string_of_int n)
          | `Bool b -> Some (k, if b then "true" else "false")
          | `Null -> Some (k, "")
          | other -> Some (k, Yojson.Safe.to_string other))
        items
  | _ -> []

let value_smoke_kind (v : Pricklypear_lisp.Ast.value) : string * string =
  match Primitives.is_redirect v with
  | Some loc -> ("redirect", loc)
  | None ->
      (match Primitives.is_raw_html v with
       | Some html -> ("html", html)
       | None ->
           (match v with
            | Pricklypear_lisp.Ast.VStr s -> ("text", s)
            | _ -> ("value", Pricklypear_lisp.Reader.pp_value v)))

let html_helpers_cheatsheet : string =
  "Pricklypear html/* cheatsheet (WRONG shapes are the #1 agent
     failure).\n" ^
  "\n" ^
  "CORE:\n" ^
  "  (html/attr \"class\" \"card\")           ; ONE key, ONE value ?
     string\n" ^
  "  (html/attrs (list a1 a2 a3))          ; join attr strings\n" ^
  "  (html/join (list s1 s2 s3))           ; concat HTML strings\n" ^
  "  (html/tag \"div\" attrs body)           ; attrs = string, body =
     string\n" ^
  "  (html/void \"input\" attrs)             ; void tag\n" ^
  "  (html/page (list part1 part2))        ; full page via render-html\n" ^
  "\n" ^
  "ELEMENTS (copy these arities exactly):\n" ^
  "  (html/h1 body)  (html/h2 body)  (html/h3 body)\n" ^
  "  (html/p body)                     ; escapes text\n" ^
  "  (html/p* attrs body)              ; raw body HTML allowed\n" ^
  "  (html/div attrs body)\n" ^
  "  (html/span attrs body)\n" ^
  "  (html/ul attrs body)  (html/li attrs body)\n" ^
  "  (html/strong body)  (html/code body)  (html/pre body)\n" ^
  "  (html/label attrs body)\n" ^
  "  (html/button attrs body)\n" ^
  "  (html/a href body)                ; NOT (html/a (list attrs) body)\n" ^
  "  (html/form method action attrs body)\n" ^
  "      e.g. (html/form \"POST\" \"/bookmarks\"\n" ^
  "             (html/attrs (list (html/attr \"class\" \"card\")))\n" ^
  "             (html/join (list ...fields...)))\n" ^
  "  (html/input attrs)\n" ^
  "  (html/hidden name value)\n" ^
  "  (html/text-input name attrs)      ; name is first arg STRING\n" ^
  "  (html/textarea name attrs body)\n" ^
  "  (html/void \"br\" \"\")                ; no html/br binding -- use
     void\n" ^
  "\n" ^
  "ANTI-PATTERNS (will crash smoke_route):\n" ^
  "  ? (html/form (list attrs...) (list kids...))\n" ^
  "  ? (html/a (list (html/attr ...)) title)\n" ^
```

```
  "  ? (html/text-input (list (html/attr \"name\" \"x\")))\n" ^
  "  ? (html/attr \"a\" \"b\" \"c\" \"d\")     ; only 2 args\n" ^
  "  ? (html/br)                          ; unbound\n" ^
  "  ? free variable `row` outside list/map\n" ^
  "\n" ^
  "GOOD list item:\n" ^
  "  (html/li (html/attr \"class\" \"card\")\n" ^
  "    (html/join (list\n" ^
  "      (html/a (dict-get fields \"url\") (html/text (dict-get fields \"title\")))\n" ^
  "      (html/form \"POST\" \"/bookmarks\" \"\"\n" ^
  "        (html/join (list (html/hidden \"action\" \"delete\")\n" ^
  "                         (html/hidden \"id\" id)\n" ^
  "                         (html/button (html/attr \"type\" \"submit\") \"Delete\")))))))\n" ^
  "\n" ^
  "Also: inspect_function html/form | html/a | contacts/form-html | todo/ui/add-form\n"

let truncate_for_tool ~(limit : int) (s : string) : string =
  if String.length s <= limit then s
  else String.sub s 0 limit ^ "?(truncated)"

let run_smoke_route
    (t : t)
    ~(path : string)
    ~(meth : http_method)
    ~(params : (string * string) list)
    ~(contains : string option) : string =
  let path = String.trim path in
  let meth_s = match meth with `GET -> "GET" | `POST -> "POST" in
  match Hashtbl.find_opt t.routes path with
  | None ->
      (Printf.sprintf
           "ok: false\nerror: no route registered for %s\n(hint: register_route path=%s handler=...)"
           path path)
  | Some (handler, _vis) ->
      (match Pricklypear_lisp.Env.lookup t.env handler with
       | None ->
           (Printf.sprintf
                "ok: false\nroute: %s -> %s\nerror: handler unbound in env"
                path handler)
       | Some _ ->
           let req =
             { method_ = meth
             ; path
             ; params
             ; body = ""
             ; headers =
                 (match meth with
                  | `POST ->
                      [ ("content-type", "application/x-www-form-urlencoded") ]
                  | `GET -> [])
             }
           in
           t.current_request <- Some req;
           let v_res =
             (Fun.protect ~finally:(fun () -> t.current_request <- None)
    (fun () -> (try
      let v =
                        Control_rim.run_lwt ~prims:t.prims (fun () ->
                            Pricklypear_lisp.Eval.eval_string_direct
                              ~prims:Pricklypear_lisp.Eval.empty_prims t.env
                              ("(" ^ handler ^ ")"))
                      in
                      (`Ok v)
    with exn ->
      (`Err ("handler error: " ^ pp_interp_exn exn)))))
           in
           (match v_res with
```

```
            | `Err msg ->
                (Printf.sprintf
                     "ok: false\nroute: %s -> %s\nmethod: %s\nerror: %s"
                     path handler meth_s msg)
            | `Ok v ->
                let kind, body = value_smoke_kind v in
                let contains_ok =
                  match contains with
                  | None | Some "" -> true
                  | Some needle ->
                      let nlen = String.length needle in
                      let blen = String.length body in
                      let rec find i =
                        if i + nlen > blen then false
                        else if String.sub body i nlen = needle then true
                        else find (i + 1)
                      in
                      find 0
                in
                let ok = contains_ok in
                let snippet = truncate_for_tool ~limit:500 body in
                let lines =
                  [ Printf.sprintf "ok: %s" (if ok then "true" else "false")
                  ; Printf.sprintf "route: %s -> %s" path handler
                  ; Printf.sprintf "method: %s" meth_s
                  ; Printf.sprintf "kind: %s" kind
                  ; Printf.sprintf "bytes: %d" (String.length body)
                  ]
                in
                let lines =
                  match contains with
                  | None | Some "" -> lines
                  | Some needle ->
                      lines
                      @ [ Printf.sprintf "contains %S: %s" needle
                            (if contains_ok then "true" else "false") ]
                in
                let lines =
                  if not ok then
                    lines
                    @ [ "error: smoke failed -- inspect_function, fix,
    re-smoke" ]
                  else lines
                in
                (String.concat "\n" lines ^ "\n--- snippet ---\n" ^
    snippet)))

let run_agent_tool (t : t) ~(name : string) ~(arguments : string)
    : string =
  let args_json =
    try Yojson.Safe.from_string arguments with _ -> `Assoc []
  in
  match name with
  | "list_bindings" ->
      let names =
        try
          match Primitives.prim_list_bindings_fn ~env:t.env [] with
          | Pricklypear_lisp.Ast.VList xs ->
              List.filter_map
                (function Pricklypear_lisp.Ast.VStr s -> Some s | _ ->
    None) xs
              |> List.sort String.compare
          | _ -> []
        with _ -> []
      in
      (if names = [] then "(no bindings)" else String.concat "\n" names)
  | "list_routes" ->
      let rs =
        Hashtbl.fold
          (fun route (handler, vis) acc ->
            (Printf.sprintf "%s -> %s [%s]" route handler vis) :: acc)
          t.routes []
        |> List.sort String.compare
```

```
      in
      (if rs = [] then "(no routes)" else String.concat "\n" rs)
  | "inspect_function" ->
      (match json_string_field args_json "name" with
       | None -> "error: need name"
       | Some n ->
           let pretty = current_fn_pretty t n in
           pretty)
  | "install_function" ->
      (match json_string_field args_json "name", json_string_field
     args_json "body" with
       | Some n, Some body ->
           let v = install_ai_function t ~name:n ~body_src:body in
           (pp_value_short v)
       | _ -> "error: need name and body")
  | "register_route" ->
      (match json_string_field args_json "path", json_string_field
     args_json "handler" with
       | Some path, Some handler ->
           let path = String.trim path in
           let handler = String.trim handler in
           let bad_prefix =
             List.exists
               (fun p ->
                 path = p
                 || (String.length path > String.length p
                     && String.sub path 0 (String.length p + 1) = p ^ "/"))
               [ "/api"; "/admin"; "/login"; "/logout"; "/health"; "/agent"
     ]
           in
           if path = "" || path.[0] <> '/' then
             "error: path must start with /"
           else if bad_prefix then
             ("error: path reserved: " ^ path)
           else if handler = "" then
             "error: need handler function name"
           else begin
             let vis =
               match json_string_field args_json "visibility" with
               | Some v -> Store.normalize_visibility v
               | None -> "private"
             in
             (* Phase 3: same public-route effect gate as (route ?
     'public). *)
             let gate =
               if vis = "public" then begin
                 let rows = Store.list_function_names t.store in
                 let known_fn =
                   Pricklypear_lisp.Effect.known_fn_of_bindings
                     (List.map (fun (n, e, _) -> (n, e)) rows)
                 in
                 let public_allow =
                   Pricklypear_lisp.Effect.public_route_allow ()
                 in
                 match List.find_opt (fun (n, _, _) -> n = handler) rows
     with
                 | None -> (Ok ())
                 | Some (_, body, _) ->
                     let fx =
                       Pricklypear_lisp.Effect.infer ~known_fn body
                     in
                     if Pricklypear_lisp.Effect.policy_allows
                          ~allow:public_allow fx
                     then (Ok ())
                     else
                       (Error
                            (Printf.sprintf
                               "public route denied: handler %S effects %s
     (allow %s)"
                               handler
                               (Pricklypear_lisp.Effect.EffSet.pp fx)

     (Pricklypear_lisp.Effect.public_route_allow_pp ())))
```

```
              end else (Ok ())
            in
            match gate with
            | Error e -> ("error: " ^ e)
            | Ok () ->
                Hashtbl.replace t.routes path (handler, vis);
                let _ =
                  Store.register_route t.store ~route:path ~handler
                    ~visibility:vis ()
                in
                (Printf.sprintf "registered route %s -> %s (%s)"
                     path handler vis)
          end
      | _ -> "error: need path and handler")
  | "eval_expr" ->
      (match json_string_field args_json "expr" with
      | None -> "error: need expr"
      | Some expr ->
          let expr = String.trim expr in
          if expr = "" then "error: empty expr"
          else if String.length expr > 4000 then
            "error: expr too long (max 4000)"
          else begin
            (* Phase 3 agent mask: deny Branch/Publish/admin-ish prims in
   tool eval. *)
            let agent_eval_allow =
              Pricklypear_lisp.Effect.EffSet.normalize
                [ Pricklypear_lisp.Effect.ReadDb
                ; Pricklypear_lisp.Effect.WriteDb
                ; Pricklypear_lisp.Effect.Html
                ; Pricklypear_lisp.Effect.Txn
                ; Pricklypear_lisp.Effect.Ai
                ]
            in
            let denied =
              try
                let e = Pricklypear_lisp.Reader.read_one expr in
                let fx = Pricklypear_lisp.Effect.infer e in
                if Pricklypear_lisp.Effect.policy_allows
                     ~allow:agent_eval_allow fx
                then None
                else Some (Pricklypear_lisp.Effect.EffSet.pp fx)
              with _ -> None
            in
            match denied with
            | Some fx ->
                ("ok: false\nerror: agent effect mask denied (" ^ fx
                   ^ "; no branch/publish)")
            | None ->
                (try
     let v =
                       Control_rim.run_lwt ~prims:t.prims (fun () ->
                           Pricklypear_lisp.Eval.eval_string_direct
                             ~prims:Pricklypear_lisp.Eval.empty_prims t.env
                             expr)
                     in
                     let s = Pricklypear_lisp.Reader.pp_value v in
                     ("ok: true\nvalue: "
                        ^ truncate_for_tool ~limit:1500 s)
   with exn ->
     ("ok: false\nerror: " ^ pp_interp_exn exn))
          end)
  | "smoke_route" ->
      (match json_string_field args_json "path" with
      | None -> "error: need path"
      | Some path ->
          let meth =
            match json_string_field args_json "method" with
            | Some m when String.uppercase_ascii (String.trim m) = "POST"
   ->
                `POST
            | _ -> `GET
          in
```

```
          let params =
            match args_json with
            | `Assoc items ->
                (match List.assoc_opt "params" items with
                 | Some p -> json_object_string_pairs p
                 | None -> [])
            | _ -> []
          in
          let contains = json_string_field args_json "contains" in
          run_smoke_route t ~path ~meth ~params ~contains)
  | "html_helpers" ->
      html_helpers_cheatsheet
  | "image_context" ->
      let names =
        try
          match Primitives.prim_list_bindings_fn ~env:t.env [] with
          | Pricklypear_lisp.Ast.VList xs ->
              List.filter_map
                (function Pricklypear_lisp.Ast.VStr s -> Some s | _ -> None)
                xs
          | _ -> []
        with _ -> []
      in
      let nshow = List.length names in
      let sample = String.concat ", " (List.filteri (fun i _ -> i < 40)
    names) in
      (Printf.sprintf
           "user=%s\nbranch=%s\nbinding_count=%d\nbindings_sample=%s"
           t.manifest.username t.manifest.branch_name nshow sample)
  | other ->
      ("error: unknown tool " ^ other)

(** One durable tool-capable loop over an ai_sessions row.
    Loads history from Postgres, resubmits full messages[] each turn,
    appends user/assistant/tool rows as it goes. *)
let run_loop
    (t : t)
    ~(sess : Ai_store.session)
    ?(max_steps = 16)
    ?(on_event : Llm_client.agent_event -> unit = fun _ -> ())
    () : string =
  t.agent_branch_lock <- true;
  let open Llm_client in
  let installs : string list ref = ref [] in
  let rec loop step =
    if step > max_steps then begin
      let summary =
        match !installs with
        | [] ->
            "agent: max steps exceeded (no installs recorded). " ^
            "Try a smaller task or raise max-steps."
        | xs ->
            "agent: max steps exceeded after installs:\n- " ^
            String.concat "\n- " (List.rev xs) ^
            "\n(Partial progress kept on this branch and in the session.)"
      in
      on_event (Fail summary);
      summary
    end else
      let history = Ai_store.list_messages t.store ~session_id:sess.id in
      let msgs = Ai_store.messages_for_api sess history in
      (* Last step: no tools -- force a plain-text wrap-up. *)
      let tools_now =
        if (not sess.tools) || step = max_steps then [] else agent_tool_defs
      in
      let turn =
        chat_turn ~temperature:0.2 ~max_tokens:2048
          ~tools:tools_now ~messages:msgs ()
      in
      match turn with
      | Turn_error e ->
          on_event (Fail e);
          ("agent error: " ^ e)
```

```
    | Final text ->
        let r =
          Ai_store.append_messages t.store ~session_id:sess.id
            [ Msg { role = Assistant; content = text } ]
        in
        (match r with
         | Error e ->
             on_event (Fail e);
             ("agent error: " ^ e)
         | Ok () ->
             on_event (Token text);
             on_event Done;
             text)
    | Need_tools { content; tool_calls } ->
        if tool_calls = [] then begin
          let fallback =
            match content with
            | Some c when String.trim c <> "" -> c
            | _ ->
                "agent: model requested tools but sent none; stopping."
          in
          let _ =
            Ai_store.append_messages t.store ~session_id:sess.id
              [ Msg { role = Assistant; content = fallback } ]
          in
          on_event (Token fallback);
          on_event Done;
          fallback
        end else begin
          (match content with
           | Some c when String.trim c <> "" -> on_event (Token (c ^
"\n"))
           | _ -> ());
          let r1 =
            Ai_store.append_messages t.store ~session_id:sess.id
              [ Assistant_tools { content; tool_calls } ]
          in
          match r1 with
          | Error e ->
              on_event (Fail e);
              ("agent error: " ^ e)
          | Ok () ->
              let tool_rows =
                List.map
                  (fun (tc : tool_call) ->
                    on_event (Token (Printf.sprintf "[tool %s]\n"
tc.name));
                    let result =
                      run_agent_tool t ~name:tc.name
~arguments:tc.arguments
                    in
                    if tc.name = "install_function" then begin
                      let n =
                        match
                          try
                            json_string_field
                              (Yojson.Safe.from_string tc.arguments)
"name"
                          with _ -> None
                        with
                        | Some name -> name
                        | None -> "?"
                      in
                      installs := n :: !installs
                    end;
                    on_event
                      (Token
                         (Printf.sprintf "? %s\n"
                            (if String.length result > 200 then
                               String.sub result 0 200 ^ "?"
                             else result)));
                    (Tool_result
                         { tool_call_id = tc.id; content = result }))
```

```
                    tool_calls
                in
                let r2 =
                  Ai_store.append_messages t.store ~session_id:sess.id
    tool_rows
                in
                (match r2 with
                 | Error e ->
                     on_event (Fail e);
                     ("agent error: " ^ e)
                 | Ok () -> loop (step + 1))
          end
  in
  (Fun.protect ~finally:(fun () -> t.agent_branch_lock <- false) (fun () ->
     loop 1))

let default_agent_seed (t : t) : string =
  "You are the agent for a running Pricklypear multi-user Lisp OS image " ^
  "(software, not a cactus).\n" ^
  Printf.sprintf
    "LOCKED BRANCH: %s -- you see only the effective bindings on this
     branch "
    t.manifest.branch_name ^
  "(personal overrides shadow main; one name = one function).\n" ^
  "All install_function writes go to THIS branch only. " ^
  "Do NOT attempt checkout, fork, or merge.\n" ^
  "Tools: image_context, list_bindings, list_routes, inspect_function, " ^
  "install_function, register_route, smoke_route, eval_expr,
     html_helpers.\n" ^
  "\n" ^
  "CRITICAL -- no compile step; install is not proof:\n" ^
  "- install_function only parses + binds a lambda. It does NOT run the
     body.\n" ^
  "- A green install (\"installed X [pure]\") is NEVER enough to claim
     success.\n" ^
  "- You MUST prove work runs with smoke_route (pages) and/or eval_expr " ^
  "(helpers, CSS, data) before finishing. If you cannot show ok:true, say
     so.\n" ^
  "\n" ^
  "Code-as-data / symbols (Nopales):\n" ^
  "- Bare names in source are symbols at runtime when quoted or from " ^
  "(read-string)/(function-ast). Strings use double quotes.\n" ^
  "- (quote lambda) and \"lambda\" are DIFFERENT types. " ^
  "(symbol? 'lambda) => #t, (string? \"lambda\") => #t.\n" ^
  "- Walk ASTs with (symbol=? (car form) 'lambda), NOT string-eq on car.\n"
    ^
  "  OK: (string-eq (symbol->string (car form)) \"lambda\") if you must.\n"
    ^
  "- (function-ast \"pkg/fn\") returns a list tree with symbolic heads; " ^
  "(function-source ?) is pretty text only.\n" ^
  "- (read-string \"(lambda (x) x)\") parses LLM text to the same tree
     shape.\n" ^
  "- Prefer package-exports / package-bindings over inventing names.\n" ^
  "\n" ^
  "How to build a small app (dogfood pattern -- copy todo/notes style):\n" ^
  "1) Persist with create/rows/get/update/delete on data_rows. " ^
  "Build fields JSON with (dict-set* \"{}\" (list \"name\" n \"email\" e
     ...)). " ^
  "Optional init: (lambda () (deftype 'contact 'name 'email 'phone
     'notes)).\n" ^
  "2) install_function name=pkg/page body=(lambda () ...) using html/* " ^
  "helpers: html/page, html/form method action attrs body, html/join, " ^
  "html/text-input, html/hidden, html/button. Never invent html/nil or " ^
  "multi-arg html/attr bags like (html/attr \"a\" \"b\" \"c\" \"d\").\n" ^
  "3) Row shape from (rows 'type): each row is (id . fields-json). " ^
  "Use (car row) for id, (cdr row) for fields, (dict-get fields
     \"name\").\n" ^
  "4) POST handlers: branch on (request-method) and (request-param
     \"action\"); " ^
  "after mutate, (redirect \"/path\") or (render-html ...) for htmx.\n" ^
  "5) NEVER use outline/* as a database. outline/* is for notes trees
     only.\n" ^
  "6) Routes: call register_route path=/pkg handler=pkg/page. " ^
```

```
  "Do NOT (route ...) inside install_function (publish effect is blocked there).\n" ^
  "   Public routes: handler effects must be ? pure|read-db|html|txn " ^
  "(no branch/ai/publish). Write-db on public needs PP_PUBLIC_ALLOW_WRITE=1 " ^
  "and prefer (with-public-write ?) for the write body.\n" ^
  "7) Inspect working apps first: inspect_function todo/page, notes/page, " ^
  "todo/item/add, html/form.\n" ^
  "\n" ^
  "Definition of done (required -- do not skip):\n" ^
  "8) Pages: after install_function + register_route, smoke_route path=/app " ^
  "method=GET contains=<visible title or unique marker>. Must return ok:true.\n" ^
  "9) Data / helpers: eval_expr e.g. (count (pkg/all)) or ((pkg/helper arg)) " ^
  "and confirm the value. POST via smoke_route params then GET again.\n" ^
  "10) CSS / chrome: product chrome uses (ui/css), NOT only layout/theme. " ^
  "If the task is theming or global style, install into ui/css and/or " ^
  "ui/theme (or compose layout/theme into ui/css). Prove with:\n" ^
  "    eval_expr (string-contains? (ui/css) \"<unique token>\")\n" ^
  "    and smoke_route path=/ method=GET contains=<token or class>.\n" ^
  "    Do NOT only edit layout/theme and claim the site is themed.\n" ^
  "11) If smoke/eval fails: html_helpers, inspect_function, fix " ^
  "install_function, re-smoke/re-eval until ok:true. Never finish on a " ^
  "failed smoke with a confident success message.\n" ^
  "11b) On unbound_var: read error name field; (stub-function name) or " ^
  "install_function a real body; (callers name) for who wanted it; " ^
  "then smoke used-by-routes / impact text from install result.\n" ^
  "11c) After install_function, read impact: dependents + smoke routes " ^
  "line; smoke_route each listed path when present.\n" ^
  "12) Final message must name the smoke_route / eval_expr proofs you ran " ^
  "(path + contains, or expression + result). No proofs ? not done.\n" ^
  "\n" ^
  "HTML: call html_helpers before install_function of *page/*form/*list* bodies.\n" ^
  "Tool discipline:\n" ^
  "- Use ONLY the structured tool API. Never write <tool_call> XML in text.\n" ^
  "- install_function body = one lambda s-expression only, no prose/markdown.\n" ^
  "- Batch multiple tool calls in one turn when possible.\n" ^
  "- Prefer package/name names (contacts/page).\n" ^
  "- When finished, SHORT text summary listing proofs, and NO tool calls.\n" ^
  "- Do not invent bindings; use list_bindings / inspect_function."

let agent_session_new
    (t : t)
    ~(seed : string)
    ?(tools = true)
    ?title
    () : Pricklypear_lisp.Ast.value =
  let r =
    Ai_store.create_session t.store
      ~owner:t.manifest.username
      ~seed
      ?title
      ~branch_ref:t.manifest.branch_name
      ~tools
      ~model:(Llm_client.model ())
      ()
  in
  match r with
  | Ok s -> (Pricklypear_lisp.Ast.VStr s.id)
  | Error e -> (Pricklypear_lisp.Ast.VStr ("agent-session-new error: " ^ e))

let agent_say
    (t : t)
    ~(session_id : string)
```

```
    ~(prompt : string)
    ?(max_steps = 16)
    ?(on_event : Llm_client.agent_event -> unit = fun _ -> ())
    () : Pricklypear_lisp.Ast.value =
  let sess_opt =
    Ai_store.get_session t.store
      ~owner:t.manifest.username
      ~id:session_id
  in
  match sess_opt with
  | None ->
      (Pricklypear_lisp.Ast.VStr
           ("agent-say: no session " ^ session_id ^ " for " ^
   t.manifest.username))
  | Some sess ->
      let title =
        match sess.title with
        | Some _ -> None
        | None ->
            let p =
              let s = String.trim prompt in
              let key = "\nUser:\n" in
              let rec find i =
                if i + String.length key > String.length s then None
                else if String.sub s i (String.length key) = key then Some i
                else find (i + 1)
              in
              match find 0 with
              | Some i ->
                  String.trim
                    (String.sub s (i + String.length key)
                       (String.length s - i - String.length key))
              | None -> s
            in
            if String.trim p = "" then None
            else
              Some
                (if String.length p > 80 then String.sub p 0 80 ^ "?" else
    p)
      in
      let () =
        Ai_store.touch_session t.store ~id:sess.id ?title
          ~branch_ref:t.manifest.branch_name ()
      in
      let ar =
        Ai_store.append_messages t.store ~session_id:sess.id
          [ Llm_client.Msg { role = Llm_client.User; content = prompt } ]
      in
      match ar with
      | Error e ->
          (Pricklypear_lisp.Ast.VStr ("agent-say error: " ^ e))
      | Ok () ->
          let text =
            run_loop t ~sess ~max_steps ~on_event ()
          in
          (Pricklypear_lisp.Ast.VStr text)

let agent_history (t : t) ~(session_id : string)
    : Pricklypear_lisp.Ast.value =
  let sess_opt =
    Ai_store.get_session t.store
      ~owner:t.manifest.username
      ~id:session_id
  in
  match sess_opt with
  | None ->
      (Pricklypear_lisp.Ast.VStr
           ("agent-history: no session " ^ session_id))
  | Some _ ->
      let pairs =
        Ai_store.history_summaries t.store ~session_id
      in
      let rows =
```

```
        List.map
          (fun (role, content) ->
            Pricklypear_lisp.Ast.VList
              [ Pricklypear_lisp.Ast.VStr role
              ; Pricklypear_lisp.Ast.VStr content
              ])
          pairs
      in
      (Pricklypear_lisp.Ast.VList rows)

let agent_sessions (t : t) ?(limit = 50) ()
    : Pricklypear_lisp.Ast.value =
  let xs =
    Ai_store.list_sessions t.store
      ~owner:t.manifest.username
      ~limit
      ()
  in
  let rows =
    List.map
      (fun (s : Ai_store.session) ->
        let title = Option.value s.title ~default:"" in
        Pricklypear_lisp.Ast.VList
          [ Pricklypear_lisp.Ast.VStr s.id
          ; Pricklypear_lisp.Ast.VStr title
          ; Pricklypear_lisp.Ast.VStr s.updated_at
          ])
      xs
  in
  (Pricklypear_lisp.Ast.VList rows)

let agent_session_delete (t : t) ~(session_id : string)
    : Pricklypear_lisp.Ast.value =
  let ok =
    Ai_store.delete_session t.store
      ~owner:t.manifest.username
      ~id:session_id
  in
  (Pricklypear_lisp.Ast.VStr
       (if ok then "deleted" else "not found"))

(** Temporary alias: new session (tools on) + one say.
    Prefer (agent-session-new) + (agent-say). [stream] ignored. *)
let agent
    (t : t)
    ?(stream = false)
    ?(on_event : Llm_client.agent_event -> unit = fun _ -> ())
    ~(seed : string)
    ~(prompt : string)
    () : Pricklypear_lisp.Ast.value =
  let _ = stream in
  let title_opt =
    let p = String.trim prompt in
    if p = "" then None
    else Some (if String.length p > 80 then String.sub p 0 80 ^ "?" else p)
  in
  let created =
    agent_session_new t ~seed ~tools:true ?title:title_opt ()
  in
  match created with
  | Pricklypear_lisp.Ast.VStr sid
    when not (String.starts_with ~prefix:"agent-session-new error" sid) ->
      agent_say t ~session_id:sid ~prompt ~on_event ()
  | other -> other

(** Temporary alias: new tools session + one say. *)
let agent_task
    (t : t)
    ?(max_steps = 16)
    ?(on_event : Llm_client.agent_event -> unit = fun _ -> ())
    ~(prompt : string)
    () : Pricklypear_lisp.Ast.value =
  let seed = default_agent_seed t in
```

```
  let title_opt =
    let p = String.trim prompt in
    if p = "" then None
    else Some (if String.length p > 80 then String.sub p 0 80 ^ "?" else p)
  in
  let created =
    agent_session_new t ~seed ~tools:true ?title:title_opt ()
  in
  match created with
  | Pricklypear_lisp.Ast.VStr sid when not (String.starts_with
     ~prefix:"agent-session-new error" sid) ->
      agent_say t ~session_id:sid ~prompt ~max_steps ~on_event ()
  | other -> other


let llm_rewrite (t : t) ~(mode : string) ~(name : string) ~(concept :
     string)
    : Pricklypear_lisp.Ast.value =
  with_user_llm t (fun () ->
  let current = current_fn_pretty t name in
  let res =
    Llm_client.rewrite_component_source ~mode ~name ~concept
      ~current_pretty:current ()
  in
  match res with
  | Error e -> (Pricklypear_lisp.Ast.VStr ("ai error: " ^ e))
  | Ok body_src ->
      let known_fn = known_fn_from_store t in
      (match check_ai_body_effects ~known_fn ~env:t.env body_src with
       | Error pe ->
           (Pricklypear_lisp.Ast.VStr
                (Printf.sprintf "ai rejected by effects: %s\n---\n%s" pe
    body_src))
       | Ok () ->
           let author = "agent:" ^ Llm_client.active_model () in
           let def = define_binding t ~name ~body_src ~author in
           (match def with
            | Ok _ ->
                (Pricklypear_lisp.Ast.VStr
                     (Printf.sprintf "froze %s (%s) by %s\n%s"
                        name mode author body_src))
            | Error e ->
                (Pricklypear_lisp.Ast.VStr
                     (Printf.sprintf "freeze failed: %s\n---\n%s" e
    body_src)))))

let llm_inner t ~name ~concept =
  llm_rewrite t ~mode:"replace" ~name ~concept

let llm_outer t ~name ~concept =
  llm_rewrite t ~mode:"wrap" ~name ~concept

(** Growth-from-seed: smallest core then recursive outer wraps. *)
let llm_seed (t : t) ~(requirement : string) ?(max_steps = 5) ()
    : Pricklypear_lisp.Ast.value =
  with_user_llm t (fun () ->
  let tip = "seed" in
  let seed_res = Llm_client.seed_core_source ~requirement () in
  match seed_res with
  | Error e -> (Pricklypear_lisp.Ast.VStr ("seed error: " ^ e))
  | Ok body_src ->
      let known_fn = known_fn_from_store t in
      (match check_ai_body_effects ~known_fn ~env:t.env body_src with
       | Error pe ->
           (Pricklypear_lisp.Ast.VStr ("seed rejected: " ^ pe ^ "\n" ^
    body_src))
       | Ok () ->
           let author = "agent:" ^ Llm_client.active_model () in
           let def = define_binding t ~name:tip ~body_src ~author in
           (match def with
            | Error e ->
                raise (Pricklypear_lisp.Ast.Interp_error
                         (Pricklypear_lisp.Ast.Prim_error
```

```
                            ("seed", "freeze failed: " ^ e, "failed")))
            | Ok _ ->
                let log = Buffer.create 256 in
                Buffer.add_string log (Printf.sprintf "step0 seed %s\n%s\n"
    tip body_src);
                let rec grow step =
                  if step > max_steps then
                    (Pricklypear_lisp.Ast.VStr
                         (Buffer.contents log ^ "stopped: max-steps\n"))
                  else
                    let pretty = current_fn_pretty t tip in
                    let judge =
                      Llm_client.growth_judge ~requirement
    ~current_pretty:pretty
                        ~step ()
                    in
                    match judge with
                    | Error e ->
                        (Pricklypear_lisp.Ast.VStr
                             (Buffer.contents log ^ "judge error: " ^ e))
                    | Ok line ->
                        let upper = String.uppercase_ascii (String.trim
    line) in
                        if String.length upper >= 4 && String.sub upper 0 4
    = "DONE"
                        then
                          (Pricklypear_lisp.Ast.VStr
                               (Buffer.contents log ^ "DONE at step " ^
                                string_of_int step ^ "\n" ^ pretty))
                        else
                          let concept =
                            if String.length line >= 5
                               && String.sub (String.lowercase_ascii line)
    0 5 = "next:"
                            then String.trim (String.sub line 5
    (String.length line - 5))
                            else String.trim line
                          in
                          Buffer.add_string log
                            (Printf.sprintf "step%d NEXT %s\n" step
    concept);
                          let wr =
                            llm_outer t ~name:tip ~concept
                          in
                          Buffer.add_string log
                            (Pricklypear_lisp.Reader.pp_value wr ^ "\n");
                          grow (step + 1)
                in
                grow 1)))

(* Register the 5 CRUD primitives (get/create/update/delete/list) over
   `data_rows`. These are NORMAL prims: prim_fn = value list -> value.
   They return their real DB result -- e.g. (get 'user 42) returns the
   row's JSON fields as a VStr; `(mapcar (lambda (id) (get 'user id)) ids)`
   works.

   Per [one-tree.branching-interaction] /data DECISION: rows are
   USER-SCOPED. Creates land under the session user (author column +
   data.u.<user>.* tree path). Reads see only the caller's own rows
   plus shared seed types (bible-*, strongs-entry). Not branch-versioned. *)
```

57 image/lib/ai_store.ml

```
(** Durable agent conversation store.

    Spec [ai-authoring.agent-portal]: ai_sessions + ai_messages.
    No LLM calls here -- maps rows <-> Llm_client.chat_message. *)

type session = {
  id : string;
  owner : string;
  title : string option;
  seed : string;
  branch_ref : string option;
  tools : bool;
  model : string option;
  created_at : string;
  updated_at : string;
}

let pg_conn (store : Store.t) : Io_lwt.Db.conn option =
     Io_lwt.Db.current_conn store

let cell_string row i =
  match List.nth_opt row i with
  | Some v -> Pgx.Value.to_string v
  | None -> None

let cell_bool row i =
  match cell_string row i with
  | Some ("t" | "true" | "TRUE" | "1") -> true
  | Some ("f" | "false" | "FALSE" | "0") -> false
  | _ -> false

let row_to_session row : session option =
  match
    ( cell_string row 0
    , cell_string row 1
    , cell_string row 2
    , cell_string row 3
    , cell_string row 4
    , cell_string row 6
    , cell_string row 7
    , cell_string row 8 )
  with
  | ( Some id
    , Some owner
    , title
    , Some seed
    , branch_ref
    , model
    , Some created_at
    , Some updated_at ) ->
      Some
        { id
        ; owner
        ; title
        ; seed
        ; branch_ref
        ; tools = cell_bool row 5
        ; model
        ; created_at
        ; updated_at
        }
  | _ -> None

let session_select_cols =
  "id::text, owner, title, seed, branch_ref, tools::text, model, \
   created_at::text, updated_at::text"

let create_session
    (store : Store.t)
    ~(owner : string)
    ~(seed : string)
```

```
    ?title
    ?branch_ref
    ?(tools = true)
    ?model
    () : (session, string) result =
  match pg_conn store with
  | None -> (Error "ai_store: no database")
  | Some conn ->
      (try
      let sql =
            "INSERT INTO ai_sessions \
             (owner, title, seed, branch_ref, tools, model) \
             VALUES ($1, $2, $3, $4, $5, $6) \
             RETURNING " ^ session_select_cols
          in
          let rows =
            Io_lwt.Db.execute conn
              ~params:
                [ Pgx.Value.of_string owner
                ; (match title with
                   | Some t -> Pgx.Value.of_string t
                   | None -> Pgx.Value.null)
                ; Pgx.Value.of_string seed
                ; (match branch_ref with
                   | Some b -> Pgx.Value.of_string b
                   | None -> Pgx.Value.null)
                ; Pgx.Value.of_bool tools
                ; (match model with
                   | Some m -> Pgx.Value.of_string m
                   | None -> Pgx.Value.null)
                ]
              sql
          in
          match rows with
          | row :: _ ->
              (match row_to_session row with
               | Some s -> (Ok s)
               | None -> (Error "ai_store: create parse failed"))
          | [] -> (Error "ai_store: create returned no row")
    with exn ->
      (Error ("ai_store create: " ^ Printexc.to_string exn)))

let get_session (store : Store.t) ~(owner : string) ~(id : string)
    : session option =
  match pg_conn store with
  | None -> None
  | Some conn ->
      (try
      let sql =
            "SELECT " ^ session_select_cols ^
            " FROM ai_sessions WHERE id = $1::uuid AND owner = $2 LIMIT 1"
          in
          let rows =
            Io_lwt.Db.execute conn
              ~params:
                [ Pgx.Value.of_string id; Pgx.Value.of_string owner ]
              sql
          in
          match rows with
          | row :: _ -> (row_to_session row)
          | [] -> None
    with exn ->
      Printf.eprintf "[ai_store] get_session: %s\n%!"
            (Printexc.to_string exn);
          None)

let list_sessions (store : Store.t) ~(owner : string) ?(limit = 50) ()
    : session list =
  match pg_conn store with
  | None -> []
  | Some conn ->
      (try
      let sql =
```

```
            "SELECT " ^ session_select_cols ^
            " FROM ai_sessions WHERE owner = $1 \
             ORDER BY updated_at DESC LIMIT $2"
          in
          let rows =
            Io_lwt.Db.execute conn
              ~params:
                [ Pgx.Value.of_string owner
                ; Pgx.Value.of_string (string_of_int limit)
                ]
              sql
          in
          (List.filter_map row_to_session rows)
    with exn ->
      Printf.eprintf "[ai_store] list_sessions: %s\n%!"
            (Printexc.to_string exn);
          []

let delete_session (store : Store.t) ~(owner : string) ~(id : string)
    : bool =
  match pg_conn store with
  | None -> false
  | Some conn ->
      (try
      let sql =
            "DELETE FROM ai_sessions \
             WHERE id = $1::uuid AND owner = $2 \
             RETURNING id::text"
          in
          let rows =
            Io_lwt.Db.execute conn
              ~params:
                [ Pgx.Value.of_string id; Pgx.Value.of_string owner ]
              sql
          in
          (rows <> [])
    with exn ->
      Printf.eprintf "[ai_store] delete_session: %s\n%!"
            (Printexc.to_string exn);
          false)

let touch_session
    (store : Store.t)
    ~(id : string)
    ?title
    ?branch_ref
    () : unit =
  match pg_conn store with
  | None -> ()
  | Some conn ->
      (try
      let sql =
            "UPDATE ai_sessions SET \
             updated_at = now(), \
             title = COALESCE($2, title), \
             branch_ref = COALESCE($3, branch_ref) \
             WHERE id = $1::uuid"
          in
          let _ =
            Io_lwt.Db.execute conn
              ~params:
                [ Pgx.Value.of_string id
                ; (match title with
                   | Some t -> Pgx.Value.of_string t
                   | None -> Pgx.Value.null)
                ; (match branch_ref with
                   | Some b -> Pgx.Value.of_string b
                   | None -> Pgx.Value.null)
                ]
              sql
          in
          ()
    with exn ->
```

```
      Printf.eprintf "[ai_store] touch_session: %s\n%!"
            (Printexc.to_string exn);
          ())

(** Encode tool_calls list as JSON for jsonb column. *)
let tool_calls_to_json (tcs : Llm_client.tool_call list) : string =
  Yojson.Safe.to_string
    (`List
       (List.map
          (fun (tc : Llm_client.tool_call) ->
            `Assoc
              [ ("id", `String tc.id)
              ; ("name", `String tc.name)
              ; ("arguments", `String tc.arguments)
              ])
          tcs))

let tool_calls_of_json_string (s : string) : Llm_client.tool_call list =
  try
    match Yojson.Safe.from_string s with
    | `List xs ->
        List.filter_map
          (function
            | `Assoc items ->
                let id =
                  match List.assoc_opt "id" items with
                  | Some (`String x) -> Some x
                  | _ -> None
                in
                let name =
                  match List.assoc_opt "name" items with
                  | Some (`String x) -> Some x
                  | _ -> None
                in
                let arguments =
                  match List.assoc_opt "arguments" items with
                  | Some (`String x) -> x
                  | Some other -> Yojson.Safe.to_string other
                  | None -> "{}"
                in
                (match id, name with
                 | Some id, Some name ->
                     Some ({ id; name; arguments } : Llm_client.tool_call)
                 | _ -> None)
            | _ -> None)
          xs
    | _ -> []
  with _ -> []

let chat_message_to_row (m : Llm_client.chat_message)
    : (string * string option * string option * string option) option =
  match m with
  | Llm_client.Msg { role = Llm_client.System; _ } ->
      (* System is session.seed only -- never stored. *)
      None
  | Llm_client.Msg { role = Llm_client.User; content } ->
      Some ("user", Some content, None, None)
  | Llm_client.Msg { role = Llm_client.Assistant; content } ->
      Some ("assistant", Some content, None, None)
  | Llm_client.Msg { role = Llm_client.Tool; content } ->
      (* Bare tool role without tool_call_id is incomplete; still store. *)
      Some ("tool", Some content, None, None)
  | Llm_client.Assistant_tools { content; tool_calls } ->
      Some
        ( "assistant"
        , content
        , Some (tool_calls_to_json tool_calls)
        , None )
  | Llm_client.Tool_result { tool_call_id; content } ->
      Some ("tool", Some content, None, Some tool_call_id)

let row_to_chat_message
    ~(role : string)
```

```
    ~(content : string option)
    ~(tool_calls : string option)
    ~(tool_call_id : string option) : Llm_client.chat_message option =
  match role with
  | "user" ->
      Some
        (Llm_client.Msg
           { role = Llm_client.User
           ; content = Option.value content ~default:""
           })
  | "assistant" ->
      (match tool_calls with
       | Some tc when String.trim tc <> "" && tc <> "null" ->
           let tcs = tool_calls_of_json_string tc in
           if tcs = [] then
             Some
               (Llm_client.Msg
                  { role = Llm_client.Assistant
                  ; content = Option.value content ~default:""
                  })
           else
             Some (Llm_client.Assistant_tools { content; tool_calls = tcs })
       | _ ->
           Some
             (Llm_client.Msg
                { role = Llm_client.Assistant
                ; content = Option.value content ~default:""
                }))
  | "tool" ->
      (match tool_call_id with
       | Some tid ->
           Some
             (Llm_client.Tool_result
                { tool_call_id = tid
                ; content = Option.value content ~default:""
                })
       | None ->
           Some
             (Llm_client.Msg
                { role = Llm_client.Tool
                ; content = Option.value content ~default:""
                }))
  | _ -> None

let next_seq (conn : Io_lwt.Db.conn) ~(session_id : string) : int =
  let sql =
    "SELECT COALESCE(MAX(seq), -1)::text FROM ai_messages \
     WHERE session_id = $1::uuid"
  in
  let rows =
    Io_lwt.Db.execute conn ~params:[ Pgx.Value.of_string session_id ] sql
  in
  match rows with
  | row :: _ ->
      (match cell_string row 0 with
       | Some s ->
           ((int_of_string_opt s |> Option.value ~default:(-1)) + 1)
       | None -> 0)
  | [] -> 0

let list_messages (store : Store.t) ~(session_id : string)
    : Llm_client.chat_message list =
  match pg_conn store with
  | None -> []
  | Some conn ->
      (try
      let sql =
            "SELECT role, content, tool_calls::text, tool_call_id \
             FROM ai_messages \
             WHERE session_id = $1::uuid \
             ORDER BY seq ASC"
          in
          let rows =
```

```
            Io_lwt.Db.execute conn
              ~params:[ Pgx.Value.of_string session_id ]
              sql
          in
          let msgs =
            List.filter_map
              (fun row ->
                match cell_string row 0 with
                | None -> None
                | Some role ->
                    row_to_chat_message
                      ~role
                      ~content:(cell_string row 1)
                      ~tool_calls:(cell_string row 2)
                      ~tool_call_id:(cell_string row 3))
              rows
          in
          msgs
    with exn ->
      Printf.eprintf "[ai_store] list_messages: %s\n%!"
            (Printexc.to_string exn);
          []

let append_messages
    (store : Store.t)
    ~(session_id : string)
    (msgs : Llm_client.chat_message list) : (unit, string) result =
  if msgs = [] then (Ok ())
  else
    match pg_conn store with
    | None -> (Error "ai_store: no database")
    | Some conn ->
        (try
      let start = next_seq conn ~session_id in
            let rec go i = function
              | [] -> (Ok ())
              | m :: rest ->
                  (match chat_message_to_row m with
                   | None -> go i rest (* skip system *)
                   | Some (role, content, tool_calls, tool_call_id) ->
                       let sql =
                         "INSERT INTO ai_messages \
                          (session_id, seq, role, content, tool_calls,
    tool_call_id) \
                          VALUES ($1::uuid, $2, $3, $4, $5::jsonb, $6)"
                       in
                       let _ =
                         Io_lwt.Db.execute conn
                           ~params:
                             [ Pgx.Value.of_string session_id
                             ; Pgx.Value.of_string (string_of_int i)
                             ; Pgx.Value.of_string role
                             ; (match content with
                                | Some c -> Pgx.Value.of_string c
                                | None -> Pgx.Value.null)
                             ; (match tool_calls with
                                | Some t -> Pgx.Value.of_string t
                                | None -> Pgx.Value.null)
                             ; (match tool_call_id with
                                | Some t -> Pgx.Value.of_string t
                                | None -> Pgx.Value.null)
                             ]
                           sql
                       in
                       go (i + 1) rest)
            in
            let r = go start msgs in
            (match r with
             | Error _ as e -> e
             | Ok () ->
                 let () = touch_session store ~id:session_id () in
                 (Ok ()))
    with exn ->
```

```
      (Error ("ai_store append: " ^ Printexc.to_string exn)))

(** Build the full messages list for a provider call:
    system seed (not stored) + durable history. *)
let messages_for_api (sess : session) (history : Llm_client.chat_message
     list)
    : Llm_client.chat_message list =
  Llm_client.Msg { role = Llm_client.System; content = sess.seed }
  :: history

(** Pretty history for Lisp/HTTP (no tool_calls detail required). *)
let history_summaries (store : Store.t) ~(session_id : string)
    : (string * string) list =
  let msgs = list_messages store ~session_id in
  let rec go acc = function
    | [] -> (List.rev acc)
    | Llm_client.Msg { role; content } :: rest ->
        let r =
          match role with
          | Llm_client.User -> "user"
          | Llm_client.Assistant -> "assistant"
          | Llm_client.System -> "system"
          | Llm_client.Tool -> "tool"
        in
        go ((r, content) :: acc) rest
    | Llm_client.Assistant_tools { content; tool_calls } :: rest ->
        let names =
          String.concat ","
            (List.map (fun (t : Llm_client.tool_call) -> t.name) tool_calls)
        in
        let c =
          match content with
          | Some s when String.trim s <> "" -> s ^ " [tools: " ^ names ^ "]"
          | _ -> "[tools: " ^ names ^ "]"
        in
        go (("assistant", c) :: acc) rest
    | Llm_client.Tool_result { tool_call_id; content } :: rest ->
        go
          (("tool", tool_call_id ^ " ? " ^ content) :: acc)
          rest
  in
  go [] msgs
```

58 image/lib/analyze.ml

```
(** On-demand LLM complexity analysis + cache in data_rows. *)
module Ast = Pricklypear_lisp.Ast
module Reader = Pricklypear_lisp.Reader
module Comp = Pricklypear_lisp.Complexity
module Expl = Pricklypear_lisp.Explain

let sha1_hex (s : string) : string =
  let d = Digestif.SHA1.digest_string s in
  Digestif.SHA1.to_hex d

let ast_hash (e : Ast.expr) : string = sha1_hex (Reader.pp_expr e)

let heuristic_json ~(fn_name : string) (e : Ast.expr) : Yojson.Safe.t =
  let cx = Comp.classify ~fn_name e in
  `Assoc
    [
      ("complexity", `String (Comp.badge cx));
      ("confidence", `String "heuristic");
      ("rationale", `String "static pattern match (no LLM)");
      ("source", `String "heuristic");
    ]

let strip_code_fence (s : string) : string =
  let s = String.trim s in
  if String.length s >= 3 && String.starts_with ~prefix:"```" s then
    let rest =
      match String.index_opt s '\n' with
      | None -> s
      | Some i -> String.sub s (i + 1) (String.length s - i - 1)
    in
    let rest = String.trim rest in
    if String.ends_with ~suffix:"```" rest then
      String.sub rest 0 (String.length rest - 3) |> String.trim
    else rest
  else s

(** Extract a balanced `{...}` object from prose. Prefers the last complete
    object that looks like our schema (has "complexity"), since models often
    emit a truncated content object first and a full one later in
    reasoning. *)
let extract_json_object (s : string) : string option =
  let s = strip_code_fence s in
  let len = String.length s in
  let extract_at start =
    let rec scan i depth in_str escape =
      if i >= len then None
      else
        let c = s.[i] in
        if escape then scan (i + 1) depth in_str false
        else if in_str then
          if c = '\\' then scan (i + 1) depth true true
          else if c = '"' then scan (i + 1) depth false false
          else scan (i + 1) depth true false
        else
          match c with
          | '"' -> scan (i + 1) depth true false
          | '{' -> scan (i + 1) (depth + 1) false false
          | '}' ->
              if depth <= 1 then Some (String.sub s start (i - start + 1))
              else scan (i + 1) (depth - 1) false false
          | _ -> scan (i + 1) depth false false
    in
    scan (start + 1) 1 false false
  in
  let rec collect i acc =
    if i >= len then List.rev acc
    else if s.[i] = '{' then
      match extract_at i with
      | Some blob -> collect (i + 1) (blob :: acc)
      | None -> collect (i + 1) acc
    else collect (i + 1) acc
```

```
  in
  let blobs = collect 0 [] in
  let has_complexity b =
    try
      match Yojson.Safe.from_string b with
      | `Assoc fields -> List.mem_assoc "complexity" fields
      | _ -> false
    with _ -> false
  in
  match List.rev (List.filter has_complexity blobs) with
  | b :: _ -> Some b (* last schema-shaped object *)
  | [] -> (match List.rev blobs with b :: _ -> Some b | [] -> None)

let parse_llm_json (raw : string) : (Yojson.Safe.t, string) result =
  let try_parse s =
    try Some (Yojson.Safe.from_string (String.trim s)) with _ -> None
  in
  match try_parse (strip_code_fence raw) with
  | Some (`Assoc _ as j) -> Ok j
  | Some _ ->
      Error "analyze: LLM JSON was not an object"
  | None ->
      (match extract_json_object raw with
       | Some blob ->
           (match try_parse blob with
            | Some (`Assoc _ as j) -> Ok j
            | Some _ -> Error "analyze: embedded JSON was not an object"
            | None ->
                Error
                  ("analyze: LLM response not JSON: "
                   ^ String.sub raw 0 (min 200 (String.length raw))))
       | None ->
           Error
             ("analyze: LLM response not JSON: "
              ^ String.sub raw 0 (min 200 (String.length raw))))

let cache_get (store : Store.t) ~(hash : string) : Yojson.Safe.t option =
  match Crud.find_row store ~type_name:"function_analysis"
     ~field:"cache_hash" ~value:hash with
  | Ast.VNil -> None
  | Ast.VPair (Ast.VStr _id, Ast.VStr fields_json) ->
      (try
         let j = Yojson.Safe.from_string fields_json in
         Some (Yojson.Safe.Util.member "result" j)
       with _ -> None)
  | Ast.VStr fields_json ->
      (* some paths return fields only *)
      (try
         let j = Yojson.Safe.from_string fields_json in
         match j with
         | `Assoc _ ->
             (match Yojson.Safe.Util.member "result" j with
              | `Null -> Some j
              | r -> Some r)
         | _ -> None
       with _ -> None)
  | _ -> None

let cache_put (store : Store.t) ~(author : string) ~(fn_name : string)
    ~(hash : string) ~(result : Yojson.Safe.t) : unit =
  let fields =
    `Assoc
      [
        ("fn", `String fn_name);
        ("cache_hash", `String hash);
        ("result", result);
        ("when", `String (string_of_float (Unix.gettimeofday ())));
      ]
  in
  let fields_json = Yojson.Safe.to_string fields in
  ignore
    (Crud.create store ~type_name:"function_analysis" ~fields_json ~author)

```

```
let resolve_llm_creds (store : Store.t) ~(user_id : string option) :
    Llm_client.creds option =
  match user_id with
  | Some uid ->
      (match Auth_store.get_llm_key_secret store ~user_id:uid with
       | Some (key, meta) ->
           Some
             {
               Llm_client.api_key = key;
               api_base =
                 (match meta.Auth_store.base with
                  | Some b when b <> "" -> b
                  | _ -> Llm_client.api_base ());
               model =
                 (match meta.Auth_store.model with
                  | Some m when m <> "" -> m
                  | _ -> Llm_client.model ());
             }
       | None ->
           let k = Llm_client.api_key () in
           if k = "" then None
           else
             Some
               {
                 api_key = k;
                 api_base = Llm_client.api_base ();
                 model = Llm_client.model ();
               })
  | None ->
      let k = Llm_client.api_key () in
      if k = "" then None
      else
        Some
          {
            api_key = k;
            api_base = Llm_client.api_base ();
            model = Llm_client.model ();
          }

let is_placeholder (s : string) : bool =
  let t = String.trim s in
  t = "" || t = "..." || t = "O(...)" || t = "TODO"
  || (String.length t > 0 && String.length t <= 3
      && String.for_all (fun c -> c = '.') t)

let normalize_result ~(model : string) ~(source : string) (j :
    Yojson.Safe.t)
    : (Yojson.Safe.t, string) result =
  let open Yojson.Safe.Util in
  let str_field key =
    match member key j with
    | `String s -> s
    | `Null -> ""
    | other -> Yojson.Safe.to_string other
  in
  let complexity =
    match str_field "complexity" with
    | "" -> str_field "big_o"
    | s -> s
  in
  let complexity =
    if complexity = "" then str_field "time_complexity" else complexity
  in
  let confidence =
    match str_field "confidence" with "" -> "medium" | s -> s
  in
  let rationale =
    match str_field "rationale" with
    | "" ->
        (match str_field "reason" with
         | "" -> str_field "explanation"
         | s -> s)
    | s -> s
```

```
  in
  if is_placeholder complexity || is_placeholder rationale then
    Error
      ("analyze: LLM returned placeholder fields (complexity="
       ^ complexity ^ ")")
  else
    Ok
      (`Assoc
         [
           ("complexity", `String complexity);
           ("confidence", `String confidence);
           ("rationale", `String rationale);
           ("notes", `String (str_field "notes"));
           ("source", `String source);
           ("model", `String model);
           ("label", `String "LLM estimate");
         ])

let heuristic_fallback ~(model : string) ~(fallback : Yojson.Safe.t)
    ~(note : string) : Yojson.Safe.t =
  let open Yojson.Safe.Util in
  `Assoc
    [
      ("complexity", member "complexity" fallback);
      ("confidence", `String "low");
      ("rationale", member "rationale" fallback);
      ("notes", `String note);
      ("source", `String "heuristic+llm-fallback");
      ("model", `String model);
      ("label", `String "heuristic (LLM fallback)");
    ]

let llm_analyze ~(fn_name : string) (e : Ast.expr) ~(creds :
    Llm_client.creds)
    ~(fallback : Yojson.Safe.t) : (Yojson.Safe.t, string) result =
  let pp = Reader.pp_expr e in
  let system =
    "You estimate asymptotic time complexity for Scheme-family Lisp. \
     Respond with a JSON object only. Required string fields: \
     complexity (e.g. O(1), O(n), O(n log n), O(n^2), O(2^n), unknown), \
     confidence (low|medium|high), rationale (one concrete sentence about \
     THIS function), notes (optional). Never use ellipsis placeholders."
  in
  let user =
    Printf.sprintf
      "Analyze this function and fill real values (not ...).\nname:
    %s\nsource:\n%s\n"
      fn_name pp
  in
  match
    Llm_client.with_creds creds (fun () ->
        (* GLM spends many tokens on message.reasoning; keep headroom so
           message.content JSON is not truncated mid-object. *)
        Llm_client.call ~temperature:0.0 ~max_tokens:2048 ~json_object:true
          ~system ~user ())
  with
  | Error e -> Error e
  | Ok raw ->
      (match parse_llm_json raw with
       | Ok j ->
           (match
              normalize_result ~model:creds.Llm_client.model ~source:"llm" j
            with
            | Ok out -> Ok out
            | Error err ->
                Ok
                  (heuristic_fallback ~model:creds.Llm_client.model
    ~fallback
                     ~note:(err ^ "; raw snippet: "
                            ^
                            let s = String.trim raw in
                            if String.length s > 180 then String.sub s 0
    180 ^ "?"
```

```
                           else s)))
      | Error _parse_err ->
          let snippet =
            let s = String.trim raw in
            if String.length s > 240 then String.sub s 0 240 ^ "?" else s
          in
          Ok
            (heuristic_fallback ~model:creds.Llm_client.model ~fallback
               ~note:("LLM did not return usable JSON; showing heuristic. "
                      ^ snippet)))

let analyze_function (store : Store.t) ~(author : string)
    ~(user_id : string option) ~(fn_name : string) ~(force_llm : bool)
    : (Yojson.Safe.t, string) result =
  match Store.lookup_function store fn_name with
  | None -> Error ("analyze: no function " ^ fn_name)
  | Some (expr, _, _) ->
      let hash = ast_hash expr in
      (match cache_get store ~hash with
       | Some j when not force_llm -> Ok j
       | _ ->
           let heur = heuristic_json ~fn_name expr in
           if not force_llm then Ok heur
           else
             (match resolve_llm_creds store ~user_id with
              | None ->
                  Error
                    "analyze: no LLM key configured (set user key at \
                     /settings/keys or LLM_API_KEY)"
              | Some creds ->
                  (match llm_analyze ~fn_name expr ~creds ~fallback:heur
    with
                   | Error e -> Error e
                   | Ok j ->
                       cache_put store ~author ~fn_name ~hash ~result:j;
                       Ok j)))

let explain_function (store : Store.t) ~(fn_name : string)
    : (string, string) result =
  match Store.lookup_function store fn_name with
  | None -> Error ("explain: no function " ^ fn_name)
  | Some (expr, _, _) ->
      let layers = Expl.peel ~fn_name expr in
      Ok (String.concat "\n\n" (Expl.layers_to_strings layers))

let result_to_vstr = function
  | Ok (`String s) -> Ast.VStr s
  | Ok j -> Ast.VStr (Yojson.Safe.to_string j)
  | Error e -> Ast.VStr e
```

59 image/lib/ast_html.ml

```
(** Pure_html AST pretty-printer with recursion/tail CSS classes. *)
open Pure_html
module Ast = Pricklypear_lisp.Ast
module Rec = Pricklypear_lisp.Recursion
module Reader = Pricklypear_lisp.Reader

let class_for ~(info : Rec.recursion_info) ~(mutual : string list)
    ~(path : int list) ~(callee : string option) : string list =
  let cls = ref [ "ast-node" ] in
  (match callee with
   | Some n when n = "" -> ()
   | Some n when List.exists (fun s -> s.Rec.callee = n && s.path = path)
     info.self_sites
     ->
       cls := "ast-self-ref" :: !cls;
       if Rec.is_tail_self_path info path then cls := "ast-tail" :: !cls
       else cls := "ast-non-tail" :: !cls
   | Some n when List.mem n mutual -> cls := "ast-mutual" :: !cls
   | _ -> ());
  !cls

let join_class xs = String.concat " " xs

let rec node_of ~(info : Rec.recursion_info) ~(mutual : string list)
    ~(path : int list) (e : Ast.expr) : node =
  match e with
  | Ast.Var n -> HTML.span [ HTML.class_ "ast-var" ] [ txt "%s" n ]
  | Ast.Lit v -> HTML.span [ HTML.class_ "ast-lit" ] [ txt "%s"
     (Reader.pp_value v) ]
  | Ast.Quote e' ->
      HTML.span [ HTML.class_ "ast-quote" ]
        [ txt "'"; node_of ~info ~mutual ~path:(0 :: path) e' ]
  | Ast.App (Ast.Var callee, args) ->
      let cls =
        class_for ~info ~mutual ~path:(List.rev path) ~callee:(Some callee)
      in
      let kids =
        txt "("
        :: HTML.span [ HTML.class_ "ast-head" ] [ txt "%s" callee ]
        :: (match args with
            | [] -> [ txt ")" ]
            | _ ->
                List.concat
                  (List.mapi
                     (fun i a ->
                       [
                         txt " ";
                         node_of ~info ~mutual ~path:(i :: path) a;
                       ])
                     args)
                @ [ txt ")" ])
      in
      HTML.span [ HTML.class_ "%s" (join_class cls) ] kids
  | Ast.App (f, args) ->
      HTML.span [ HTML.class_ "ast-app" ]
        (txt "("
         :: node_of ~info ~mutual ~path:(0 :: path) f
         :: List.concat
              (List.mapi
                 (fun i a ->
                   [ txt " "; node_of ~info ~mutual ~path:((i + 1) :: path)
    a ])
                 args)
          @ [ txt ")" ])
  | Ast.If (t, th, el) ->
      HTML.span [ HTML.class_ "ast-if" ]
        [
          txt "(if ";
          node_of ~info ~mutual ~path:(0 :: path) t;
          txt " ";
          node_of ~info ~mutual ~path:(1 :: path) th;
```

```
69          txt " ";
70          node_of ~info ~mutual ~path:(2 :: path) el;
71          txt ")";
72        ]
73    | Ast.Seq (a, b) ->
74      HTML.span [ HTML.class_ "ast-seq" ]
75        [
76          txt "(seq ";
77          node_of ~info ~mutual ~path:(0 :: path) a;
78          txt " ";
79          node_of ~info ~mutual ~path:(1 :: path) b;
80          txt ")";
81        ]
82    | Ast.Let (n, v, body) ->
83      HTML.span [ HTML.class_ "ast-let" ]
84        [
85          txt "(let ((";
86          txt "%s" n;
87          txt " ";
88          node_of ~info ~mutual ~path:(0 :: path) v;
89          txt ")) ";
90          node_of ~info ~mutual ~path:(1 :: path) body;
91          txt ")";
92        ]
93    | Ast.Letrec (bs, body) ->
94      let bind_nodes =
95        List.concat
96          (List.mapi
97             (fun i (n, e') ->
98               [
99                 txt "(";
100                txt "%s" n;
101                txt " ";
102                node_of ~info ~mutual ~path:(i :: path) e';
103                txt ") ";
104              ])
105            bs)
106     in
107     HTML.span [ HTML.class_ "ast-letrec" ]
108       ([ txt "(letrec (" ]
109        @ bind_nodes
110        @ [
111            txt ") ";
112            node_of ~info ~mutual ~path:(List.length bs :: path) body;
113            txt ")";
114          ])
115   | Ast.Lambda (ps, body) ->
116     HTML.span [ HTML.class_ "ast-lambda" ]
117       [
118         txt "(lambda (";
119         txt "%s" (String.concat " " ps);
120         txt ") ";
121         node_of ~info ~mutual ~path:(0 :: path) body;
122         txt ")";
123       ]
124   | Ast.Prim (n, args) ->
125     HTML.span [ HTML.class_ "ast-prim" ]
126       (txt "(%s" n
127        :: (match args with
128            | [] -> [ txt ")" ]
129            | _ ->
130                List.concat
131                  (List.mapi
132                     (fun i a ->
133                       [
134                         txt " ";
135                         node_of ~info ~mutual ~path:(i :: path) a;
136                       ])
137                     args)
138                @ [ txt ")" ]))
139   | Ast.Define (n, body) ->
140     HTML.span [ HTML.class_ "ast-define" ]
141       [
```

```
          txt "(define ";
          txt "%s" n;
          txt " ";
          node_of ~info ~mutual ~path:(0 :: path) body;
          txt ")";
        ]

let pp_expr_html ?(fn_name = "") ?(mutual = []) (e : Ast.expr) : node =
  let info =
    if fn_name = "" then
      {
        Rec.is_recursive = false;
        tail_recursive = false;
        self_sites = [];
        callees = [];
      }
    else Rec.analyze_self ~fn_name e
  in
  let body = Rec.unwrap_fn_body e in
  HTML.pre [ HTML.class_ "bb-source ast-walk" ]
    [ node_of ~info ~mutual ~path:[] body ]

let recursion_badges ~(fn_name : string) (e : Ast.expr) : node list =
  let info = Rec.analyze_self ~fn_name e in
  let bits = ref [] in
  if info.is_recursive then begin
    bits :=
      HTML.span
        [
          HTML.class_ "badge %s"
            (if info.tail_recursive then "ast-tail" else "ast-non-tail");
        ]
        [
          txt "%s"
            (if info.tail_recursive then "tail-recursive"
             else "recursive (non-tail)");
        ]
      :: !bits
  end;
  List.rev !bits
```

60 image/lib/auth_password.ml

```
(** Pricklypear Image -- password hashing.

    CHAMPION: argon2id (t_cost=2, m_cost=65536 KiB, parallelism=1,
    PHC-encoded), the auth.borg endpoint. Legacy rows in the interim
    format

      sha256$<salt_hex>$<digest_hex>

    still VERIFY, and callers rehash-on-success (auth_store.verify_password
    ? [Auth_store.transparent_rehash]) so a deployed principal migrates to
    argon2id on its next successful login with no downtime.

    Upgrade path deliberately avoids a flag day: we never dual-enter a row
    and never reject a legacy principal -- the digest comparison is cheap
    constant-time, and the entire class of "verify passes with the old
    scheme" is bounded by untouched rows only. *)

let hex_encode (b : string) : string =
  let len = String.length b in
  let buf = Bytes.create (len * 2) in
  let hex = "0123456789abcdef" in
  for i = 0 to len - 1 do
    let c = Char.code b.[i] in
    Bytes.set buf (i * 2) hex.[c lsr 4];
    Bytes.set buf (i * 2 + 1) hex.[c land 0xf]
  done;
  Bytes.unsafe_to_string buf

let random_bytes (n : int) : string =
  Mirage_crypto_rng.generate n

let sha256_hex (s : string) : string =
  Digestif.SHA256.(digest_string s |> to_hex)

(* Argon2id parameters -- auth.borg "interim" note and the OWASP cheat
   sheet agree on m_cost=65536 KiB / t_cost=2 / parallelism=1 as the
   interactive-login sweet spot. hash_len=32, PHC encoding. *)
let argon_t_cost = 2
let argon_m_cost = 65536
let argon_parallelism = 1
let argon_hash_len = 32



let hash_argonid ~(salt : string) (password : string) : string =
  (* Top-level [Argon2.hash] -- its [encoded] is concrete string, unlike
     the functor-instantiated ID.hash_encoded (abstract [ID.encoded]). *)
  match
    Argon2.hash ~t_cost:argon_t_cost ~m_cost:argon_m_cost
      ~parallelism:argon_parallelism ~pwd:password ~salt ~kind:Argon2.ID
      ~hash_len:argon_hash_len
      ~encoded_len:(Argon2.encoded_len ~t_cost:argon_t_cost
                      ~m_cost:argon_m_cost ~parallelism:argon_parallelism
                      ~salt_len:(String.length salt)
                      ~hash_len:argon_hash_len ~kind:Argon2.ID)
      ~version:Argon2.VERSION_NUMBER
  with
  | Ok (_raw, encoded) -> encoded
  | Error e ->
      invalid_arg ("argon2id hash failed: " ^ Argon2.ErrorCodes.message e)

(** Constant-time string compare to avoid password/hash timing leaks. *)
let secure_equal (a : string) (b : string) : bool =
  let la = String.length a and lb = String.length b in
  let len = max la lb in
  let acc = ref (la lxor lb) in
  for i = 0 to len - 1 do
    let ca = if i < la then Char.code a.[i] else 0 in
    let cb = if i < lb then Char.code b.[i] else 0 in
    acc := !acc lor (ca lxor cb)
  done;
```

```
  !acc = 0

(** Result of a verify: did the password match, and is the stored hash in
    the CURRENT scheme (so no upgrade needed)? Callers with DB access use
    [needs_upgrade] to trigger the transparent rehash. *)
type verify_outcome = {
  ok : bool;
  needs_upgrade : bool;
}

let verify_legacy ~(stored : string) (password : string) : bool =
  match String.split_on_char '$' stored with
  | ["sha256"; salt_hex; digest] when salt_hex <> "" && digest <> "" ->
      let candidate = sha256_hex (salt_hex ^ ":" ^ password) in
      secure_equal candidate digest
  | _ -> false

let verify_current ~(stored : string) (password : string) : bool =
  (* PHC-encoded argon2id starts "$argon2id$". *)
  if String.starts_with ~prefix:"$argon2id$" stored then
    (match Argon2.verify ~encoded:stored ~pwd:password ~kind:Argon2.ID with
     | Ok b -> b
     | Error _ -> false)
  else false

let verify_full ~(stored : string) (password : string) : verify_outcome =
  if String.starts_with ~prefix:"$argon2id$" stored then
    { ok = verify_current ~stored password; needs_upgrade = false }
  else
    { ok = verify_legacy ~stored password; needs_upgrade = true }

let verify_password ~(stored : string) (password : string) : bool =
  (verify_full ~stored password).ok

let hash_password ?(salt_len = 16) (password : string) : string =
  let salt = random_bytes salt_len in
  hash_argonid ~salt password

(** Fixed dummy argon2id hash for unknown-username verifies (timing
    flatten, i-1787970741): a lookup for a nonexistent account burns the
    same argon2 work as one for a real row, so usernames are not
    distinguishable by response latency. The salt is a fixed public
    constant -- this hash is never compared for success, only spent, so
    no RNG dependency at force time either. *)
let dummy_hash = lazy (hash_argonid ~salt:"pp-unknown-user-dummy"
     "throttle-dummy")

(** True when [stored] is in the legacy interim scheme and a successful
    verify should rewrite it to argon2id. *)
let needs_upgrade (stored : string) : bool =
  not (String.starts_with ~prefix:"$argon2id$" stored)
```

61 image/lib/auth_store.ml

```
(** Pricklypear Image -- in-process auth store (single-image Option B).

    Operates on migrations/002_auth.sql tables from the image process.
    No separate authd. Auth holds verify + mint only; session identity
    is attached per HTTP request (auth.multi-user-product / web-login). *)
(* H2.4: DB execute via Io_lwt.Db *)

type user = {
  id : string;
  username : string;
  is_admin : bool;
  user_type : string;           (* "human" | "agent" *)
  parent_user_id : string option;  (* the human an agent belongs to; None
     for humans *)
}

type session_row = {
  id : string;
  user_id : string;
  username : string;
  is_admin : bool;
  expires_at : string;
}

let cell_uuid_opt row i =
  match List.nth_opt row i with
  | Some v ->
      (match Pgx.Value.to_string v with
       | Some s when s <> "" -> Some s
       | _ -> None)
  | None -> None

let last4_of_key (key : string) : string =
  let n = String.length key in
  if n <= 4 then key else String.sub key (n - 4) 4

let env_or name dft =
  match Sys.getenv_opt name with Some s -> s | None -> dft

let pg_conn (store : Store.t) : Io_lwt.Db.conn option =
    Io_lwt.Db.current_conn store

let cell_string row i =
  match List.nth_opt row i with
  | Some v -> Pgx.Value.to_string v
  | None -> None

let cell_bool row i =
  match cell_string row i with
  | Some ("t" | "true" | "TRUE" | "1") -> true
  | Some ("f" | "false" | "FALSE" | "0") -> false
  | _ -> false

let log_auth (store : Store.t) ?user_id ~kind ~success ?source_ip ()
  : unit =
  match pg_conn store with
  | None -> ()
  | Some conn ->
      (try
      let sql =
            "INSERT INTO auth_log (user_id, kind, success, source_ip) " ^
            "VALUES ($1, $2, $3, $4)"
          in
          let _ =
            Io_lwt.Db.execute conn
              ~params:
                [ (match user_id with
                   | Some u -> Pgx.Value.of_string u
                   | None -> Pgx.Value.null);
                  Pgx.Value.of_string kind;
                  Pgx.Value.of_bool success;
```

```
                  (match source_ip with
                   | Some ip -> Pgx.Value.of_string ip
                   | None -> Pgx.Value.null) ]
               sql
          in
          ()
    with exn ->
      Printf.eprintf "[auth_store] log_auth: %s\n%!"
        (Printexc.to_string exn);
          ())

let count_users (store : Store.t) : int =
  match pg_conn store with
  | None -> 0
  | Some conn ->
      (try
      let rows =
        Io_lwt.Db.execute conn "SELECT count(*)::text FROM users"
          in
          match rows with
          | row :: _ ->
              (match cell_string row 0 with
               | Some s ->
                   (int_of_string_opt s |> Option.value ~default:0)
               | None -> 0)
          | [] -> 0
    with exn ->
      Printf.eprintf "[auth_store] count_users: %s\n%!"
        (Printexc.to_string exn);
          0)

let user_cols = "id::text, username, is_admin::text, user_type,
    parent_user_id::text"

let user_of_row row =
  match cell_string row 0, cell_string row 1, cell_string row 3 with
  | Some id, Some uname, Some utype ->
      Some { id; username = uname;
             is_admin = cell_bool row 2;
             user_type = utype;
             parent_user_id = cell_uuid_opt row 4 }
  | _ -> None

let get_user_by_username (store : Store.t) (username : string)
  : user option =
  match pg_conn store with
  | None -> None
  | Some conn ->
      (try
      let sql =
            "SELECT " ^ user_cols ^ " FROM users WHERE username = $1 LIMIT
    1"
          in
          let rows =
            Io_lwt.Db.execute conn
              ~params:[ Pgx.Value.of_string username ] sql
          in
          match rows with
          | row :: _ ->
              (match user_of_row row with
               | Some u -> Some u
               | None -> None)
          | [] -> None
    with exn ->
      Printf.eprintf "[auth_store] get_user_by_username: %s\n%!"
        (Printexc.to_string exn);
          None)

let get_user_by_id (store : Store.t) (user_id : string) : user option =
  match pg_conn store with
  | None -> None
  | Some conn ->
      (try
```

```
      let sql =
            "SELECT " ^ user_cols ^ " FROM users WHERE id = $1::uuid LIMIT
     1"
          in
          let rows =
            Io_lwt.Db.execute conn
              ~params:[ Pgx.Value.of_string user_id ] sql
          in
          match rows with
          | row :: _ ->
              (match user_of_row row with
               | Some u -> Some u
               | None -> None)
          | [] -> None
    with exn ->
      Printf.eprintf "[auth_store] get_user_by_id: %s\n%!"
            (Printexc.to_string exn);
          None)

let username_dns_ok (name : string) : bool =
  let name = String.lowercase_ascii (String.trim name) in
  let n = String.length name in
  n > 0 && n <= 63
  && (match name.[0] with 'a' .. 'z' | '0' .. '9' -> true | _ -> false)
  && (match name.[n - 1] with 'a' .. 'z' | '0' .. '9' -> true | _ -> false)
  &&
  let rec loop i =
    if i >= n then true
    else
      match name.[i] with
      | 'a' .. 'z' | '0' .. '9' | '-' -> loop (i + 1)
      | _ -> false
  in
  loop 0

let create_user (store : Store.t) ~(username : string) ~(is_admin : bool)
  : (user, string) result =
  let username = String.lowercase_ascii (String.trim username) in
  if not (username_dns_ok username) then
    (Error
         "username must be a DNS label: [a-z0-9]([a-z0-9-]*[a-z0-9])?")
  else
  match pg_conn store with
  | None -> (Error "no database")
  | Some conn ->
      (try
      let sql =
            "INSERT INTO users (username, is_admin) VALUES ($1, $2) " ^
            "RETURNING " ^ user_cols
          in
          let rows =
            Io_lwt.Db.execute conn
              ~params:
                [ Pgx.Value.of_string username;
                  Pgx.Value.of_bool is_admin ]
              sql
          in
          match rows with
          | row :: _ ->
              (match user_of_row row with
               | Some u -> (Ok u)
               | None -> (Error "create_user: bad row"))
          | [] -> (Error "create_user: no row")
    with exn ->
      let msg = Printexc.to_string exn in
          Printf.eprintf "[auth_store] create_user: %s\n%!" msg;
          (Error msg))

(** Create an agent user attached to a parent human.
    username = "<parent>/agent", user_type='agent', parent_user_id=parent.
    is_admin=false (agents never admin). Idempotent: returns existing if
     any. *)
let create_agent (store : Store.t) ~(parent_username : string)
```

```
  : (user, string) result =
  let parent_username = String.lowercase_ascii (String.trim
     parent_username) in
  let agent_username = parent_username ^ "/agent" in
  (* Idempotent: return existing agent if one exists for this parent. *)
  let existing = get_user_by_username store agent_username in
  match existing with
  | Some u when u.user_type = "agent" -> (Ok u)
  | _ ->
      (match pg_conn store with
       | None -> (Error "no database")
       | Some conn ->
           let parent = get_user_by_username store parent_username in
           (match parent with
            | None ->
                (Error ("no such parent user: " ^ parent_username))
            | Some p when p.user_type = "agent" ->
                (Error "agent cannot be a parent of another agent")
            | Some p ->
                (try
     let sql =
                    "INSERT INTO users " ^
                    "(username, is_admin, user_type, parent_user_id) " ^
                    "VALUES ($1, false, 'agent', $2::uuid) " ^
                    "RETURNING " ^ user_cols
                  in
                  let rows =
                    Io_lwt.Db.execute conn
                      ~params:
                        [ Pgx.Value.of_string agent_username;
                          Pgx.Value.of_string p.id ]
                      sql
                  in
                  match rows with
                  | row :: _ ->
                      (match user_of_row row with
                       | Some u -> (Ok u)
                       | None -> (Error "create_agent: bad row"))
                  | [] -> (Error "create_agent: no row")
    with exn ->
      let msg = Printexc.to_string exn in
                  Printf.eprintf "[auth_store] create_agent: %s\n%!" msg;
                  (Error msg))))

let set_password (store : Store.t) ~(user_id : string) ~(password : string)
  : (unit, string) result =
  match pg_conn store with
  | None -> (Error "no database")
  | Some conn ->
      (try
      let hash = Auth_password.hash_password password in
          let secret =
            Yojson.Safe.to_string (`Assoc [ ("hash", `String hash) ])
          in
          (* One password credential per user: replace existing. *)
          let _ =
            Io_lwt.Db.execute conn
              ~params:[ Pgx.Value.of_string user_id ]
              "DELETE FROM credentials WHERE user_id = $1::uuid AND kind =
    'password'"
          in
          let sql =
            "INSERT INTO credentials (user_id, kind, public_data,
    secret_data) " ^
              "VALUES ($1::uuid, 'password', NULL, $2::jsonb)"
          in
          let _ =
            Io_lwt.Db.execute conn
              ~params:
                [ Pgx.Value.of_string user_id;
                  Pgx.Value.of_string secret ]
              sql
          in
```

```
          (Ok ())
    with exn ->
      let msg = Printexc.to_string exn in
          Printf.eprintf "[auth_store] set_password: %s\n%!" msg;
          (Error msg))

let password_hash_for_user (store : Store.t) ~(username : string)
  : (user * string) option =
  match pg_conn store with
  | None -> None
  | Some conn ->
      (try
      let sql =
            "SELECT u.id::text, u.username, u.is_admin::text, u.user_type,
    " ^
            "       u.parent_user_id::text, c.secret_data::text " ^
            "FROM users u " ^
            "JOIN credentials c ON c.user_id = u.id " ^
            "WHERE u.username = $1 AND c.kind = 'password' " ^
            "ORDER BY c.created_at DESC LIMIT 1"
          in
          let rows =
            Io_lwt.Db.execute conn
              ~params:[ Pgx.Value.of_string username ] sql
          in
          match rows with
          | row :: _ ->
              (match
                 cell_string row 0, cell_string row 1, cell_string row 5
               with
               | Some id, Some uname, Some secret_json ->
                   let hash_opt =
                     try
                       match Yojson.Safe.from_string secret_json with
                       | `Assoc a ->
                           (match List.assoc_opt "hash" a with
                            | Some (`String h) -> Some h
                            | _ -> None)
                       | `String h -> Some h
                       | _ -> None
                     with _ -> None
                   in
                   (match hash_opt with
                    | Some h ->
                        Some
                          ( { id; username = uname;
                              is_admin = cell_bool row 2;
                              user_type =
                                (match cell_string row 3 with
                                 | Some s -> s | None -> "human");
                              parent_user_id = cell_uuid_opt row 4 },
                            h )
                    | None -> None)
               | _ -> None)
          | [] -> None
    with exn ->
      Printf.eprintf "[auth_store] password_hash_for_user: %s\n%!"
            (Printexc.to_string exn);
          None)

let verify_password (store : Store.t) ~(username : string)
    ~(password : string) ?source_ip () : user option =
  (* Login throttle (i-1787970741): keyed by lowercase username so case
     variants share one bucket, checked BEFORE the DB lookup -- admit=false
     returns None immediately with no DB work. Only ever engages after
     recorded failures, so ordinary correct logins are never blocked. *)
  let throttle_key = String.lowercase_ascii username in
  if not (Auth_throttle.admit throttle_key (Auth_throttle.now ())) then None
  else
    let found = password_hash_for_user store ~username in
    match found with
    | None ->
        (* Timing flatten: unknown usernames still pay one argon2 verify
```

```
           against a fixed dummy hash, so known/unknown accounts cost the
           same and are not distinguishable by latency. Counts as a
           throttle failure. *)
        let _ =
          Auth_password.verify_password
            ~stored:(Lazy.force Auth_password.dummy_hash) password
        in
        let () =
          Auth_throttle.record_failure throttle_key (Auth_throttle.now ());
          log_auth store ~kind:"password" ~success:false ?source_ip ()
        in
        None
    | Some (user, stored) ->
        if Auth_password.verify_password ~stored password then
          let () =
            Auth_throttle.clear throttle_key;
            log_auth store ~user_id:user.id ~kind:"password" ~success:true
              ?source_ip ()
          in
          (* Transparent argon2id upgrade (BORGE-1785545259): a successful
             legacy-scheme verify rewrites the stored hash to argon2id, so
             principals migrate without a flag day. *)
          if Auth_password.needs_upgrade stored then
            (match set_password store ~user_id:user.id ~password with
             | Ok () ->
                 Printf.eprintf
                   "[auth] %s password upgraded interim -> argon2id\n%!"
                   user.username
             | Error e ->
                 Printf.eprintf "[auth] %s argon2id upgrade failed: %s\n%!"
                   user.username e);
          (* Touch last_used_at *)
          (match pg_conn store with
           | None -> Some user
           | Some conn ->
               let _ =
                 (try
      Io_lwt.Db.execute conn
                       ~params:[ Pgx.Value.of_string user.id ]
                       "UPDATE credentials SET last_used_at = now() \
                        WHERE user_id = $1::uuid AND kind = 'password'"
    with _ ->
      [])
               in
               Some user)
        else
          let () =
            Auth_throttle.record_failure throttle_key (Auth_throttle.now
     ());
            log_auth store ~user_id:user.id ~kind:"password" ~success:false
              ?source_ip ()
          in
          None

(** Session token: opaque random id. Cookie carries raw token; DB stores
    only SHA-256 hex of the token (auth.web-login). *)
let token_hash (raw_token : string) : string =
  Auth_password.sha256_hex raw_token

let mint_session
    (store : Store.t)
    ~(user_id : string)
    ?(ttl_seconds = 60 * 60 * 24 * 14)
    ?user_agent
    ?source_ip
    () : (string, string) result =
  match pg_conn store with
  | None -> (Error "no database")
  | Some conn ->
      (try
      let raw =
            Base64.encode_string ~pad:false
              (Auth_password.random_bytes 32)
```

```
          in
          let th = token_hash raw in
          let sql =
            "INSERT INTO auth_sessions \
             (user_id, token_hash, expires_at) \
             VALUES ($1::uuid, $2, now() + ($3::text || '
    seconds')::interval) \
             RETURNING id::text"
          in
          let rows =
            Io_lwt.Db.execute conn
              ~params:
                [ Pgx.Value.of_string user_id;
                  Pgx.Value.of_string th;
                  Pgx.Value.of_string (string_of_int ttl_seconds) ]
              sql
          in
          let () =
            log_auth store ~user_id ~kind:"mint" ~success:true ?source_ip ()
          in
          ignore user_agent;
          match rows with
          | _ :: _ -> (Ok raw)
          | [] -> (Error "mint_session: no row")
    with exn ->
      let msg = Printexc.to_string exn in
          Printf.eprintf "[auth_store] mint_session: %s\n%!" msg;
          (Error msg))

let lookup_session (store : Store.t) ~(raw_token : string)
  : session_row option =
  if raw_token = "" then None
  else
    match pg_conn store with
    | None -> None
    | Some conn ->
        (try
      let th = token_hash raw_token in
            let sql =
              "SELECT s.id::text, s.user_id::text, u.username, \
                      u.is_admin::text, s.expires_at::text \
               FROM auth_sessions s \
               JOIN users u ON u.id = s.user_id \
               WHERE s.token_hash = $1 \
                 AND s.revoked_at IS NULL \
                 AND s.expires_at > now() \
               LIMIT 1"
            in
            let rows =
              Io_lwt.Db.execute conn
                ~params:[ Pgx.Value.of_string th ] sql
            in
            match rows with
            | row :: _ ->
                (match
                   cell_string row 0,
                   cell_string row 1,
                   cell_string row 2,
                   cell_string row 4
                 with
                 | Some id, Some uid, Some uname, Some exp ->
                     Some
                       { id; user_id = uid; username = uname;
                         is_admin = cell_bool row 3;
                         expires_at = exp }
                 | _ -> None)
            | [] -> None
    with exn ->
      Printf.eprintf "[auth_store] lookup_session: %s\n%!"
              (Printexc.to_string exn);
            None)

let revoke_session (store : Store.t) ~(raw_token : string) ?source_ip ()
```

```
  : unit =
  match pg_conn store with
  | None -> ()
  | Some conn ->
      (try
      let th = token_hash raw_token in
          let sql =
            "UPDATE auth_sessions SET revoked_at = now() \
             WHERE token_hash = $1 AND revoked_at IS NULL \
             RETURNING user_id::text"
          in
          let rows =
            Io_lwt.Db.execute conn
              ~params:[ Pgx.Value.of_string th ] sql
          in
          match rows with
          | row :: _ ->
              let uid = cell_string row 0 in
              log_auth store ?user_id:uid ~kind:"logout" ~success:true
                ?source_ip ()
          | [] -> ()
    with exn ->
      Printf.eprintf "[auth_store] revoke_session: %s\n%!"
            (Printexc.to_string exn);
          ())

let revoke_all_sessions (store : Store.t) ~(user_id : string) : int =
  match pg_conn store with
  | None -> 0
  | Some conn ->
      (try
      let sql =
            "UPDATE auth_sessions SET revoked_at = now() \
             WHERE user_id = $1::uuid AND revoked_at IS NULL \
             RETURNING id"
          in
          let rows =
            Io_lwt.Db.execute conn
              ~params:[ Pgx.Value.of_string user_id ] sql
          in
          let () =
            log_auth store ~user_id ~kind:"revoke_all" ~success:true ()
          in
          (List.length rows)
    with exn ->
      Printf.eprintf "[auth_store] revoke_all_sessions: %s\n%!"
            (Printexc.to_string exn);
          0)

(** API token (Bearer) credentials -- auth INTO the OS for agents/pp_eval.
    kind='api_key' rows. Raw token returned ONCE at issue time; only the
    SHA-256 hash is stored. Per-user; no global tokens. *)
let issue_token (store : Store.t) ~(user_id : string)
    ?(label : string option = None) () : (string, string) result =
  match pg_conn store with
  | None -> (Error "no database")
  | Some conn ->
      (try
      let raw =
            Base64.encode_string ~pad:false
              (Auth_password.random_bytes 32)
          in
          let th = token_hash raw in
          let public_fields =
            [ ("token_last4", `String (last4_of_key raw)) ]
            @ (match label with
               | Some l when l <> "" -> [ ("label", `String l) ]
               | _ -> [])
          in
          let public_json = Yojson.Safe.to_string (`Assoc public_fields) in
          let secret_json =
            Yojson.Safe.to_string (`Assoc [ ("token_hash", `String th) ])
          in
```

```
          let sql =
            "INSERT INTO credentials (user_id, kind, public_data,
    secret_data) " ^
            "VALUES ($1::uuid, 'api_key', $2::jsonb, $3::jsonb) " ^
            "RETURNING id::text"
          in
          let _ =
            Io_lwt.Db.execute conn
              ~params:
                [ Pgx.Value.of_string user_id;
                  Pgx.Value.of_string public_json;
                  Pgx.Value.of_string secret_json ]
              sql
          in
          let () =
            log_auth store ~user_id ~kind:"issue_token" ~success:true ()
          in
          (Ok raw)
    with exn ->
      let msg = Printexc.to_string exn in
          Printf.eprintf "[auth_store] issue_token: %s\n%!" msg;
          (Error msg))

let verify_token (store : Store.t) ~(raw_token : string) ?source_ip ()
  : user option =
  if raw_token = "" then None
  else
    match pg_conn store with
    | None -> None
    | Some conn ->
        (try
      let th = token_hash raw_token in
            let sql =
              "SELECT " ^
              "  u.id::text, u.username, u.is_admin::text, u.user_type, " ^
              "  u.parent_user_id::text, c.id::text " ^
              "FROM credentials c JOIN users u ON u.id = c.user_id " ^
              "WHERE c.kind = 'api_key' " ^
              "  AND c.secret_data->>'token_hash' = $1 " ^
              "LIMIT 1"
            in
            let rows =
              Io_lwt.Db.execute conn
                ~params:[ Pgx.Value.of_string th ] sql
            in
            match rows with
            | row :: _ ->
                (match user_of_row row with
                 | Some user ->
                     (match cell_string row 5 with
                      | Some cid ->
                          let _ =
                            Io_lwt.Db.execute conn
                              ~params:[ Pgx.Value.of_string cid ]
                              ("UPDATE credentials SET last_used_at = now()
    " ^
                               "WHERE id = $1::uuid")
                          in
                          let () =
                            log_auth store ~user_id:user.id ~kind:"api_key"
                              ~success:true ?source_ip ()
                          in
                          Some user
                      | None ->
                          let () =
                            log_auth store ~user_id:user.id ~kind:"api_key"
                              ~success:true ?source_ip ()
                          in
                          Some user)
                 | None ->
                     let () =
                       log_auth store ~kind:"api_key" ~success:false
    ?source_ip ()
```

```
                    in
                    None)
           | [] ->
               let () =
                 log_auth store ~kind:"api_key" ~success:false ?source_ip
    ()
               in
               None
    with exn ->
      Printf.eprintf "[auth_store] verify_token: %s\n%!"
              (Printexc.to_string exn);
            None)

type token_meta = {
  id : string;
  label : string option;
  token_last4 : string;
  created_at : string;
  last_used_at : string option;
}

let list_tokens (store : Store.t) ~(user_id : string) : token_meta list =
  match pg_conn store with
  | None -> []
  | Some conn ->
      (try
      let sql =
            "SELECT id::text, public_data::text, created_at::text, " ^
            "       last_used_at::text FROM credentials " ^
            "WHERE user_id = $1::uuid AND kind = 'api_key' " ^
            "ORDER BY created_at"
          in
          let rows =
            Io_lwt.Db.execute conn
              ~params:[ Pgx.Value.of_string user_id ] sql
          in
          let metas =
            List.filter_map
              (fun row ->
                match cell_string row 0, cell_string row 1, cell_string row
    2 with
                | Some id, Some js, Some created ->
                    let label, token_last4 =
                      match Yojson.Safe.from_string js with
                      | `Assoc a ->
                          let opt_str k =
                            match List.assoc_opt k a with
                            | Some (`String s) when s <> "" -> Some s
                            | _ -> None
                          in
                          let tl4 =
                            match List.assoc_opt "token_last4" a with
                            | Some (`String s) -> s
                            | _ -> "????"
                          in
                          (opt_str "label", tl4)
                      | _ -> (None, "????")
                    in
                    Some { id; label; token_last4; created_at = created;
                           last_used_at = cell_string row 3 }
                | _ -> None)
              rows
          in
          metas
    with exn ->
      Printf.eprintf "[auth_store] list_tokens: %s\n%!"
            (Printexc.to_string exn);
          [])

let revoke_token (store : Store.t) ~(credential_id : string)
  : (unit, string) result =
  match pg_conn store with
  | None -> (Error "no database")
```

```
  | Some conn ->
      (try
      let _ =
            Io_lwt.Db.execute conn
              ~params:[ Pgx.Value.of_string credential_id ]
              "DELETE FROM credentials WHERE id = $1::uuid AND kind =
    'api_key'"
          in
          (Ok ())
    with exn ->
      let msg = Printexc.to_string exn in
          Printf.eprintf "[auth_store] revoke_token: %s\n%!" msg;
          (Error msg))

(** rotate_token: revoke old + issue new. Returns the new raw token. *)
let rotate_token (store : Store.t) ~(credential_id : string)
    ~(user_id : string) ?label () : (string, string) result =
  let rev = revoke_token store ~credential_id in
  match rev with
  | Error e -> (Error e)
  | Ok () -> issue_token store ~user_id ~label ()

(** allow_agent_promote toggle (on the human's user row). *)
let get_allow_agent_promote (store : Store.t) ~(user_id : string)
  : bool =
  match pg_conn store with
  | None -> true
  | Some conn ->
      (try
      let rows =
            Io_lwt.Db.execute conn
              ~params:[ Pgx.Value.of_string user_id ]
              "SELECT allow_agent_promote::text FROM users WHERE id =
    $1::uuid"
          in
          match rows with
          | row :: _ -> (cell_bool row 0)
          | [] -> true
    with _ ->
      true)

let set_allow_agent_promote (store : Store.t) ~(user_id : string)
    ~(value : bool) : (unit, string) result =
  match pg_conn store with
  | None -> (Error "no database")
  | Some conn ->
      (try
      let _ =
            Io_lwt.Db.execute conn
              ~params:
                [ Pgx.Value.of_bool value;
                  Pgx.Value.of_string user_id ]
              "UPDATE users SET allow_agent_promote = $1 WHERE id =
    $2::uuid"
          in
          (Ok ())
    with exn ->
      let msg = Printexc.to_string exn in
          Printf.eprintf "[auth_store] set_allow_agent_promote: %s\n%!" msg;
          (Error msg))

(** Per-user LLM provider key (auth.user-llm-keys). Not api_key (auth INTO
    PP). *)
type llm_key_meta = {
  last4 : string;
  label : string option;
  base : string option;
  model : string option;
}

let set_llm_key (store : Store.t) ~(user_id : string) ~(api_key : string)
    ?(base : string option = None) ?(model : string option = None)
    ?(label : string option = None) () : (llm_key_meta, string) result =
```

```
  if String.trim api_key = "" then
    (Error "empty api_key")
  else
    match pg_conn store with
    | None -> (Error "no database")
    | Some conn ->
        (try
      let last4 = last4_of_key api_key in
            let public_fields =
              [ ("last4", `String last4) ]
              @ (match label with Some l when l <> "" -> [ ("label", `String l) ] | _ -> [])
              @ (match base with Some b when b <> "" -> [ ("base", `String b) ] | _ -> [])
              @ (match model with Some m when m <> "" -> [ ("model", `String m) ] | _ -> [])
            in
            let public_json =
              Yojson.Safe.to_string (`Assoc public_fields)
            in
            let secret_json =
              Yojson.Safe.to_string
                (`Assoc [ ("api_key", `String api_key) ])
            in
            let _ =
              Io_lwt.Db.execute conn
                ~params:[ Pgx.Value.of_string user_id ]
                "DELETE FROM credentials WHERE user_id = $1::uuid AND kind = 'llm_key'"
            in
            let sql =
              "INSERT INTO credentials (user_id, kind, public_data, secret_data) " ^
              "VALUES ($1::uuid, 'llm_key', $2::jsonb, $3::jsonb)"
            in
            let _ =
              Io_lwt.Db.execute conn
                ~params:
                  [ Pgx.Value.of_string user_id;
                    Pgx.Value.of_string public_json;
                    Pgx.Value.of_string secret_json ]
                sql
            in
            (Ok
                 { last4;
                   label;
                   base;
                   model })
    with exn ->
      let msg = Printexc.to_string exn in
            Printf.eprintf "[auth_store] set_llm_key: %s\n%!" msg;
            (Error msg))

let clear_llm_key (store : Store.t) ~(user_id : string)
  : (unit, string) result =
  match pg_conn store with
  | None -> (Error "no database")
  | Some conn ->
      (try
      let _ =
            Io_lwt.Db.execute conn
              ~params:[ Pgx.Value.of_string user_id ]
              "DELETE FROM credentials WHERE user_id = $1::uuid AND kind = 'llm_key'"
          in
          (Ok ())
    with exn ->
      let msg = Printexc.to_string exn in
          Printf.eprintf "[auth_store] clear_llm_key: %s\n%!" msg;
          (Error msg))

let get_llm_key_meta_direct (store : Store.t) ~(user_id : string)
  : llm_key_meta option =
```

```
  match pg_conn store with
  | None -> None
  | Some conn ->
      (try
      let sql =
            "SELECT public_data::text FROM credentials " ^
            "WHERE user_id = $1::uuid AND kind = 'llm_key' " ^
            "ORDER BY created_at DESC LIMIT 1"
          in
          let rows =
            Io_lwt.Db.execute conn
              ~params:[ Pgx.Value.of_string user_id ] sql
          in
          match rows with
          | row :: _ ->
              (match cell_string row 0 with
               | None -> None
               | Some js ->
                   (try
                      match Yojson.Safe.from_string js with
                      | `Assoc a ->
                          let last4 =
                            match List.assoc_opt "last4" a with
                            | Some (`String s) -> s
                            | _ -> "????"
                          in
                          let opt_str k =
                            match List.assoc_opt k a with
                            | Some (`String s) when s <> "" -> Some s
                            | _ -> None
                          in
                          Some
                            { last4;
                              label = opt_str "label";
                              base = opt_str "base";
                              model = opt_str "model" }
                      | _ -> None
                    with _ -> None))
          | [] -> None
     with _ ->
       None)

(** get_llm_key_meta with agent fallback to parent's key. *)
let get_llm_key_meta (store : Store.t) ~(user_id : string)
  : llm_key_meta option =
  let direct = get_llm_key_meta_direct store ~user_id in
  match direct with
  | Some _ -> direct
  | None ->
      let user_opt = get_user_by_id store user_id in
      (match user_opt with
       | Some u when u.user_type = "agent" ->
           (match u.parent_user_id with
            | Some pid -> get_llm_key_meta_direct store ~user_id:pid
            | None -> None)
       | _ -> None)

let get_llm_key_secret_direct (store : Store.t) ~(user_id : string)
  : (string * llm_key_meta) option =
  match pg_conn store with
  | None -> None
  | Some conn ->
      (try
      let sql =
            "SELECT secret_data::text, public_data::text FROM credentials "
    ^
            "WHERE user_id = $1::uuid AND kind = 'llm_key' " ^
            "ORDER BY created_at DESC LIMIT 1"
          in
          let rows =
            Io_lwt.Db.execute conn
              ~params:[ Pgx.Value.of_string user_id ] sql
          in
```

```
          match rows with
          | row :: _ ->
              (match cell_string row 0, cell_string row 1 with
               | Some secret_js, public_js_opt ->
                   let key_opt =
                     try
                       match Yojson.Safe.from_string secret_js with
                       | `Assoc a ->
                           (match List.assoc_opt "api_key" a with
                            | Some (`String k) when k <> "" -> Some k
                            | _ -> None)
                       | _ -> None
                     with _ -> None
                   in
                   (match key_opt with
                    | None -> None
                    | Some key ->
                        let meta =
                          match public_js_opt with
                          | Some js ->
                              (try
                                 match Yojson.Safe.from_string js with
                                 | `Assoc a ->
                                     let last4 =
                                       match List.assoc_opt "last4" a with
                                       | Some (`String s) -> s
                                       | _ -> last4_of_key key
                                     in
                                     let opt_str k =
                                       match List.assoc_opt k a with
                                       | Some (`String s) when s <> "" ->
    Some s
                                       | _ -> None
                                     in
                                     { last4;
                                       label = opt_str "label";
                                       base = opt_str "base";
                                       model = opt_str "model" }
                                 | _ ->
                                     { last4 = last4_of_key key;
                                       label = None;
                                       base = None;
                                       model = None }
                               with _ ->
                                 { last4 = last4_of_key key;
                                   label = None;
                                   base = None;
                                   model = None })
                          | None ->
                              { last4 = last4_of_key key;
                                label = None;
                                base = None;
                                model = None }
                        in
                        Some (key, meta))
               | _ -> None)
          | [] -> None
      with exn ->
        Printf.eprintf "[auth_store] get_llm_key_secret_direct: %s\n%!"
          (Printexc.to_string exn);
          None)

(** get_llm_key_secret with agent fallback: if the user has no own llm_key
    and is an agent (user_type='agent', parent_user_id set), look up the
    parent human's llm_key. *)
let get_llm_key_secret (store : Store.t) ~(user_id : string)
  : (string * llm_key_meta) option =
  let direct = get_llm_key_secret_direct store ~user_id in
  match direct with
  | Some _ -> direct
  | None ->
      (* Fall back to parent only if this user is an agent. *)
      let user_opt = get_user_by_id store user_id in
```

```
      (match user_opt with
       | Some u when u.user_type = "agent" ->
           (match u.parent_user_id with
            | Some pid -> get_llm_key_secret_direct store ~user_id:pid
            | None -> None)
       | _ -> None)

(** SSH public keys for REPL door (auth.user-ssh-repl phase 1). *)
type ssh_key_meta = {
  id : string;
  fingerprint : string;
  label : string option;
  openssh : string;
}

let ssh_fingerprint (openssh : string) : string =
  let open Digestif.SHA256 in
  let dig = digest_string (String.trim openssh) |> to_raw_string in
  let b64 = Base64.encode_string dig in
  "SHA256:" ^ b64

let parse_openssh_pubkey (s : string) : (string, string) result =
  let s = String.trim s in
  if s = "" then Error "empty key"
  else
    let parts =
      String.split_on_char ' ' s |> List.filter (fun p -> p <> "")
    in
    match parts with
    | typ :: _blob :: _ when String.starts_with ~prefix:"ssh-" typ ->
        if typ = "ssh-ed25519" || typ = "ssh-rsa" || typ =
    "ecdsa-sha2-nistp256"
        then Ok s
        else Error ("unsupported key type: " ^ typ)
    | _ -> Error "expected OpenSSH public key (ssh-ed25519 AAAA? [comment])"

let add_ssh_key (store : Store.t) ~(user_id : string) ~(openssh : string)
    ?(label : string option = None) () : (ssh_key_meta, string) result =
  match parse_openssh_pubkey openssh with
  | Error e -> (Error e)
  | Ok openssh ->
      match pg_conn store with
      | None -> (Error "no database")
      | Some conn ->
          (try
      let fp = ssh_fingerprint openssh in
              let public_fields =
                [ ("openssh", `String openssh)
                ; ("fingerprint", `String fp)
                ]
                @ (match label with
                   | Some l when l <> "" -> [ ("label", `String l) ]
                   | _ -> [])
              in
              let public_json =
                Yojson.Safe.to_string (`Assoc public_fields)
              in
              let sql =
                "INSERT INTO credentials (user_id, kind, public_data,
    secret_data) " ^
                "VALUES ($1::uuid, 'ssh_key', $2::jsonb, NULL) " ^
                "RETURNING id::text"
              in
              let rows =
                Io_lwt.Db.execute conn
                  ~params:
                    [ Pgx.Value.of_string user_id;
                      Pgx.Value.of_string public_json ]
                  sql
              in
              match rows with
              | row :: _ ->
                  (match cell_string row 0 with
```

```
                  | Some id ->
                      (Ok
                           { id;
                             fingerprint = fp;
                             label;
                             openssh })
                  | None -> (Error "add_ssh_key: no id"))
              | [] -> (Error "add_ssh_key: no row")
    with exn ->
      let msg = Printexc.to_string exn in
              Printf.eprintf "[auth_store] add_ssh_key: %s\n%!" msg;
              (Error msg))

let list_ssh_keys (store : Store.t) ~(user_id : string)
  : ssh_key_meta list =
  match pg_conn store with
  | None -> []
  | Some conn ->
      (try
      let sql =
            "SELECT id::text, public_data::text FROM credentials " ^
            "WHERE user_id = $1::uuid AND kind = 'ssh_key' " ^
            "ORDER BY created_at"
          in
          let rows =
            Io_lwt.Db.execute conn
              ~params:[ Pgx.Value.of_string user_id ] sql
          in
          let metas =
            List.filter_map
              (fun row ->
                match cell_string row 0, cell_string row 1 with
                | Some id, Some js ->
                    (try
                       match Yojson.Safe.from_string js with
                       | `Assoc a ->
                           let openssh =
                             match List.assoc_opt "openssh" a with
                             | Some (`String s) -> s
                             | _ -> ""
                           in
                           let fingerprint =
                             match List.assoc_opt "fingerprint" a with
                             | Some (`String s) -> s
                             | _ -> ssh_fingerprint openssh
                           in
                           let label =
                             match List.assoc_opt "label" a with
                             | Some (`String s) when s <> "" -> Some s
                             | _ -> None
                           in
                           if openssh = "" then None
                           else Some { id; fingerprint; label; openssh }
                       | _ -> None
                     with _ -> None)
                | _ -> None)
              rows
          in
          metas
    with exn ->
      Printf.eprintf "[auth_store] list_ssh_keys: %s\n%!"
            (Printexc.to_string exn);
          [])

let revoke_ssh_key (store : Store.t) ~(user_id : string)
    ~(id_or_fingerprint : string) : (unit, string) result =
  match pg_conn store with
  | None -> (Error "no database")
  | Some conn ->
      (try
      (* Delete by credential id OR fingerprint match in public_data *)
          let sql =
```

```
            "DELETE FROM credentials WHERE user_id = $1::uuid AND kind = 'ssh_key' " ^
            "AND (id::text = $2 OR public_data->>'fingerprint' = $2 " ^
            "     OR public_data->>'openssh' = $2)"
          in
          let _ =
            Io_lwt.Db.execute conn
              ~params:
                [ Pgx.Value.of_string user_id;
                  Pgx.Value.of_string (String.trim id_or_fingerprint) ]
              sql
          in
          (Ok ())
      with exn ->
        let msg = Printexc.to_string exn in
          Printf.eprintf "[auth_store] revoke_ssh_key: %s\n%!" msg;
          (Error msg))

let verify_ssh_key (store : Store.t) ~(openssh_or_blob : string)
    ?source_ip () : user option =
  match pg_conn store with
  | None -> None
  | Some conn ->
      (try
       let presented = String.trim openssh_or_blob in
          let fp = ssh_fingerprint presented in
          let sql =
            "SELECT u.id::text, u.username, u.is_admin::text, u.user_type, " ^
            "       u.parent_user_id::text, c.id::text " ^
            "FROM credentials c JOIN users u ON u.id = c.user_id " ^
            "WHERE c.kind = 'ssh_key' " ^
            "AND (c.public_data->>'openssh' = $1 " ^
            "     OR c.public_data->>'fingerprint' = $2) " ^
            "LIMIT 1"
          in
          let rows =
            Io_lwt.Db.execute conn
              ~params:
                [ Pgx.Value.of_string presented;
                  Pgx.Value.of_string fp ]
              sql
          in
          match rows with
          | row :: _ ->
              (match user_of_row row with
               | Some user ->
                   let () =
                     log_auth store ~user_id:user.id ~kind:"ssh_key" ~success:true
                       ?source_ip ()
                   in
                   (match cell_string row 5 with
                    | Some cid ->
                        let _ =
                          Io_lwt.Db.execute conn
                            ~params:[ Pgx.Value.of_string cid ]
                            "UPDATE credentials SET last_used_at = now() \
                             WHERE id = $1::uuid"
                        in
                        Some user
                    | None -> Some user)
               | None ->
                   let () =
                     log_auth store ~kind:"ssh_key" ~success:false ?source_ip ()
                   in
                   None)
          | [] ->
              let () =
                log_auth store ~kind:"ssh_key" ~success:false ?source_ip ()
              in
              None
```

```
    with exn ->
      Printf.eprintf "[auth_store] verify_ssh_key: %s\n%!"
            (Printexc.to_string exn);
          None)

let list_users (store : Store.t) : user list =
  match pg_conn store with
  | None -> []
  | Some conn ->
      (try
      let rows =
            Io_lwt.Db.execute conn
              ("SELECT " ^ user_cols ^ " FROM users ORDER BY username")
          in
          let users = List.filter_map user_of_row rows in
          users
    with exn ->
      Printf.eprintf "[auth_store] list_users: %s\n%!"
            (Printexc.to_string exn);
          [])

(** First boot: if users table empty, create admin from env.
    PP_BOOTSTRAP_USER (default admin), PP_BOOTSTRAP_PASSWORD (default admin).
    If admin already exists but has no password credential, set one from env
    (dev recovery for pre-auth DBs that only had ssh_key). *)
let bootstrap_admin_if_empty (store : Store.t) : user option =
  let username = env_or "PP_BOOTSTRAP_USER" "admin" in
  let password = env_or "PP_BOOTSTRAP_PASSWORD" "admin" in
  (* i-1787971357: never boot a default admin/admin without explicit opt-in. When the password is the default AND PP_ALLOW_INSECURE_BOOTSTRAP
     is unset, mint a random password (printed once to stderr). Lazy so no password is generated when the admin already has one / isn't created. *)
  let password =
    lazy
      (if password = "admin"
          && Sys.getenv_opt "PP_ALLOW_INSECURE_BOOTSTRAP" = None
       then
         let pw = Base64.encode_string (Mirage_crypto_rng.generate 18) in
         Printf.eprintf
           "[auth] generated admin password: %s -- set PP_BOOTSTRAP_PASSWORD to control; PP_ALLOW_INSECURE_BOOTSTRAP=1 keeps admin/admin\n%!"
           pw;
         pw
       else password)
  in
  let n = count_users store in
  if n > 0 then
    let admin = get_user_by_username store username in
    match admin with
    | None -> None
    | Some user ->
        let has_pw = password_hash_for_user store ~username in
        (match has_pw with
         | Some _ -> None
         | None ->
             let pw = set_password store ~user_id:user.id ~password:(Lazy.force password) in
             (match pw with
              | Error e ->
                  Printf.eprintf
                    "[auth_store] bootstrap password for existing %s: %s\n%!"
                    username e;
                  None
              | Ok () ->
                  Printf.eprintf
                    "[auth_store] set password for existing admin %s\n%!"
                    username;
                  Some user))
```

```
  else
    let created = create_user store ~username ~is_admin:true in
    match created with
    | Error e ->
        Printf.eprintf "[auth_store] bootstrap failed: %s\n%!" e;
        None
    | Ok user ->
        let pw = set_password store ~user_id:user.id ~password:(Lazy.force
     password) in
        (match pw with
         | Error e ->
             Printf.eprintf "[auth_store] bootstrap password: %s\n%!" e;
             None
         | Ok () ->
             Printf.eprintf
               "[auth_store] bootstrapped admin user %s (set
     PP_BOOTSTRAP_PASSWORD)\n%!"
               username;
             let () =
               log_auth store ~user_id:user.id ~kind:"bootstrap"
                 ~success:true ()
             in
             Some user)
```

62 image/lib/auth_throttle.ml

```
(** Pricklypear Image -- per-username login throttling (pure, in-memory).

    Brute-force resistance for password login (i-1787970741): a sliding
    window of failed attempts per (lowercased) username with exponential
    backoff once the window fills. Pure -- no DB: state is a bounded
    Hashtbl and the caller supplies the clock ([now] as a float), so
    behavior is deterministic and testable.

    Invariant: throttling engages ONLY after recorded failures. Fewer
    than [max_failures] failures inside the window always admits --
    verify_password runs on every Basic-auth request, so correct logins
    are never blocked. A success clears the key entirely.

    Single-domain: the image runs one Eio domain and these functions
    contain no effect points, so each call is atomic; no lock needed. *)

let env_positive_int ~default name =
  match Sys.getenv_opt name with
  | None -> default
  | Some s ->
      (match int_of_string_opt s with
       | Some n when n > 0 -> n
       | _ -> default)

let env_positive_float ~default name =
  match Sys.getenv_opt name with
  | None -> default
  | Some s ->
      (match float_of_string_opt s with
       | Some f when f > 0.0 -> f
       | _ -> default)

(** Failed logins allowed inside the window before backoff engages. *)
let max_failures () = env_positive_int ~default:10 "PP_LOGIN_MAX_FAILURES"

(** Sliding-window width in seconds. *)
let window_secs () = env_positive_float ~default:900.0
     "PP_LOGIN_WINDOW_SECS"

(** First lockout once the threshold is hit; doubles per extra failure. *)
let backoff_base () =
  env_positive_float ~default:30.0 "PP_LOGIN_BACKOFF_BASE_SECS"

(** Hard ceiling on any single lockout. *)
let backoff_cap = 900.0

(** Table bound: a username flood evicts expired entries, then the
    least-recently-failed key, instead of growing memory unboundedly. *)
let max_entries = 4096

type state = {
  mutable failures : float list;  (* descending: newest first *)
}

let table : (string, state) Hashtbl.t = Hashtbl.create 64

(** Wall clock in seconds; exposed so callers share one time source. *)
let now () : float = Unix.gettimeofday ()

let size () : int = Hashtbl.length table

(* Keep the failures still inside the window. *)
let prune ~window now failures =
  let cutoff = now -. window in
  List.filter (fun t -> t > cutoff) failures

(* base * 2^excess clamped at backoff_cap. Doubling loop rather than
   [**] so an absurd failure count cannot overflow to infinity. *)
let backoff_for (base : float) (excess : int) : float =
  let rec go acc e =
    if e <= 0 then acc
```

```
    else if acc >= backoff_cap then backoff_cap
    else go (acc *. 2.0) (e - 1)
  in
  min backoff_cap (go base excess)

(* Make room for a new key: drop entries whose failures have all slid
   out of the window first; if still at capacity, evict the entry whose
   most recent failure is oldest. *)
let evict now =
  let window = window_secs () in
  let dead =
    Hashtbl.fold
      (fun k st acc ->
        let kept = prune ~window now st.failures in
        st.failures <- kept;
        if kept = [] then k :: acc else acc)
      table []
  in
  List.iter (Hashtbl.remove table) dead;
  if Hashtbl.length table >= max_entries then begin
    let oldest =
      Hashtbl.fold
        (fun k st acc ->
          let t = match st.failures with t :: _ -> t | [] -> 0.0 in
          match acc with
          | None -> Some (k, t)
          | Some (_, t0) -> if t < t0 then Some (k, t) else acc)
        table None
    in
    match oldest with Some (k, _) -> Hashtbl.remove table k | None -> ()
  end

let state_of ~now key =
  match Hashtbl.find_opt table key with
  | Some st -> st
  | None ->
      if Hashtbl.length table >= max_entries then evict now;
      let st = { failures = [] } in
      Hashtbl.add table key st;
      st

(** Record a failed login for [key] at time [now]. *)
let record_failure (key : string) (now : float) : unit =
  let window = window_secs () in
  let st = state_of ~now key in
  st.failures <- now :: prune ~window now st.failures

(** May [key] attempt a login at time [now]? True unless the window
    already holds >= max_failures failures AND the backoff those
    failures bought (doubled per failure past the threshold, capped)
    has not yet elapsed since the most recent one. *)
let admit (key : string) (now : float) : bool =
  match Hashtbl.find_opt table key with
  | None -> true
  | Some st ->
      let window = window_secs () in
      let kept = prune ~window now st.failures in
      st.failures <- kept;
      let n = List.length kept in
      let maxf = max_failures () in
      if n < maxf then true
      else
        let backoff = backoff_for (backoff_base ()) (n - maxf) in
        now >= (List.hd kept) +. backoff

(** Forget [key] entirely -- a successful login clears the throttle. *)
let clear (key : string) : unit = Hashtbl.remove table key
```

63 image/lib/boot.ml

```
(** Pricklypear Image -- Boot.

    Single-image entry (see [image.boot] / [single-image]):
      1. Build boot manifest from env (or defaults).
      2. Connect store, bootstrap admin if needed.
      3. Load function_defs for the branch into the session env.
      4. Start optional TCP REPL (loopback line protocol) as a fiber.
      5. Start HTTP via Httpun_eio ([start_eio]).

    Env knobs include PP_USERNAME, PP_BRANCH_ID, PP_BRANCH_NAME,
    PP_BOOT_MODE, PP_IMAGE_VERSION, PP_HTTP_PORT, PP_REPL_PORT -- set by
    [scripts/dev.sh] or the host service unit. *)

let env_or name dft = match Sys.getenv_opt name with Some s -> s | None ->
     dft

let manifest_from_env () : Pricklypear_common.Manifest.t =
  let boot_mode =
    match env_or "PP_BOOT_MODE" "on-connect" with
    | "always-on" -> Pricklypear_common.Manifest.Boot_always_on
    | _ -> Pricklypear_common.Manifest.Boot_on_connect
  in
  Pricklypear_common.Manifest.make
    ~user_id:(env_or "PP_USER_ID" "dev-user")
    ~username:(env_or "PP_USERNAME" "dev")
    ~branch_id:
      (env_or "PP_BRANCH_ID" "00000000-0000-0000-0000-000000000001")
    ~branch_name:(env_or "PP_BRANCH_NAME" "main")
    ~boot_mode
    ~image_version:(env_or "PP_IMAGE_VERSION" "0.1.0-dev")
    ()

(** Shared setup: RNG, store connect, session. Returns session + ports. *)
let prepare () : Session.t * int * int =
  ignore (Ops_log.info ~component:"boot" "Pricklypear image starting up");
  Printf.eprintf "[boot] Pricklypear image starting up...\n%!";
  Mirage_crypto_rng_unix.use_default ();
  Random.self_init ();
  let manifest = manifest_from_env () in
  let cfg = Store.config_from_env () in
  let store = Store.make ~cfg ~branch_id:manifest.branch_id () in
  Store.connect store;
  ignore (Auth_store.bootstrap_admin_if_empty store);
  (* Surface any drift between the OCaml seed-type mirror and the SQL
     authority (tree_seed_types) -- non-fatal warning. See
     i-seed-list-3copies. *)
  (match Store.current_conn store with
   | Some conn -> Tree_store.verify_seed_types conn
   | None -> ());
  let session = Session.make ~manifest ~store () in
  let http_port = int_of_string (env_or "PP_HTTP_PORT" "8080") in
  let repl_port = int_of_string (env_or "PP_REPL_PORT" "7878") in
  Printf.eprintf "[boot] REPL endpoint 127.0.0.1:%d (internal)\n%!"
    repl_port;
  Printf.eprintf "[boot] user=%s branch=%s version=%s\n%!"
    manifest.username manifest.branch_name manifest.image_version;
  (session, http_port, repl_port)

(** Eio path: prepare + fork REPL fiber + Httpun_eio accept loop.
    Call under Eio_main + Io_eio.with_ctx + Lwt_eio.with_event_loop. *)
(** Jobs runner loop -- lives here (not in Jobs.ml) to avoid a dependency
    cycle: Jobs (DB ops) must not depend on Session, which depends on
    Prims_db, which depends on Jobs. boot.ml can depend on both. *)
let jobs_runner_loop ~(session : Session.t) : unit =
  let eval_job ~job : (string * bool) =
    let t = session in
    let owner, name =
      match String.index_opt job.Jobs.branch '/' with
      | Some i ->
          (String.sub job.Jobs.branch 0 i,
           String.sub job.Jobs.branch (i + 1)
```

```
            (String.length job.Jobs.branch - i - 1))
      | None -> ("system", "main")
    in
    (try
      let branch_id =
        match Store.lookup_branch_id_by_name t.store ~owner ~name with
        | Some id -> id
        | None -> Store.system_main_id
      in
      Store.checkout_branch t.store ~new_branch_id:branch_id;
      t.manifest <-
        { t.manifest with
          Pricklypear_common.Manifest.branch_id = branch_id;
          branch_name = job.Jobs.branch;
          username = job.Jobs.submitted_by };
      Session.rehydrate t;
      let call_src = Jobs.eval_src ~fn:job.Jobs.fn ~args:job.Jobs.args in
      (* Wrap eval in a Request_context so crud's user-scoping (visible_sql
         / pp.user RLS floor) sees the job's submitted_by, not 'anonymous'.
         Without this, forge/graph-by-name (a rows scan over forge-graph
         data_rows) returns nothing because the runner looks anonymous. *)
      let ctx =
        Request_context.make
          ~manifest:t.manifest ~env:t.env ~routes:t.routes
      in
      (* Bind the current job id on this fiber so the (with-step "name"
         thunk) special form (__step__ prim) scopes its checkpoints to this
         job (borge/jobs.borg step-checkpoints). Outside this binding there
         is no job context and with-step is a plain thunk call. *)
      Jobs.with_current_job_id job.Jobs.id (fun () ->
        let v =
          Request_context.with_current ctx (fun () ->
            Session.eval_string_rim t call_src)
        in
        let result_json =
          match v with
          | Pricklypear_lisp.Ast.VStr s -> s
          | Pricklypear_lisp.Ast.VNil -> "null"
          | Pricklypear_lisp.Ast.VInt n -> Int64.to_string n
          | Pricklypear_lisp.Ast.VBool b -> if b then "true" else "false"
          | _ -> Printf.sprintf "\"%s\"" (Pricklypear_lisp.Reader.pp_value
     v)
        in
        (result_json, true))
    with exn ->
      (* Surface the REAL error text (unbound var / arity / prim message)
         instead of the opaque `Interp_error(_)` Printexc prints -- see the
         runner agent-note in borge/jobs.borg (campaign 4.1). *)
      let msg =
        match exn with
        | Pricklypear_lisp.Ast.Interp_error e ->
            Pricklypear_lisp.Ast.pp_interp_error e
        | _ -> Printexc.to_string exn
      in
      (msg, false))
  in
  (* tree-time.gc (5.3): periodic retention sweep ON THE JOBS RUNNER. The
     retention worker (policy-as-habitat-data + rollup + DROP PARTITION)
     runs here, gated by wall-clock time so it does not fire every 1s tick.
     Conservative by design: Store.retention_run is a no-op unless a
     whole-class policy row exists (tree_retention_policy), so on a
     policy-free habitat this is a cheap SELECT. Spec: borge/tree-time.borg
     gc + coexistence-plan PHASE 4 ("the retention worker runs on the jobs
     runner"). Interval overridable via PP_RETENTION_SECS for dev/testing.
     *)
  let retention_interval_s =
    let v = Sys.getenv_opt "PP_RETENTION_SECS" in
    match v with
    | Some s -> int_of_string_opt s |> Option.value ~default:3600
    | None -> 3600
  in
  let last_retention = ref 0.0 in
  let retention_sweep () =
```

```
  let now = Unix.gettimeofday () in
  if now -. !last_retention >= float_of_int retention_interval_s then (
    last_retention := now;
    try
      let rs = Store.retention_run session.store ~path_class:None () in
      List.iter
        (fun (pc, d, c) ->
          if d > 0L || c > 0L then
            Printf.eprintf
              "[jobs] retention %s: dropped %Ld window(s), checkpointed %Ld path(s)\n%!"
              pc d c)
        rs
    with exn ->
      Printf.eprintf "[jobs] retention sweep failed: %s\n%!"
        (Printexc.to_string exn))
in
let rec loop () =
  (try Jobs.reap_dead ~store:session.store with _ -> ());
  retention_sweep ();
  (* Fire due schedules: submit a job per due schedule, then advance its
     next_run. Downtime collapses to one catch-up run (cron searches
     strictly after now; every-ms just adds the interval). Spec:
     borge/jobs.borg schedules. *)
  (let rec fire_schedules () =
     match Jobs.claim_due_schedule ~store:session.store with
     | None -> ()
     | Some sch ->
         let fired_at =
           Int64.of_float (Unix.gettimeofday () *. 1000.0)
         in
         (try
            ignore
              (Jobs.submit ~store:session.store
                 ~fn:sch.Jobs.s_fn ~args_json:sch.Jobs.s_args
                 ~branch:sch.Jobs.s_branch
                 ~submitted_by:sch.Jobs.s_submitted_by
                 ~run_at_ms:fired_at ());
            Jobs.advance_schedule ~store:session.store
              ~name:sch.Jobs.s_name ~fired_at
              ~cron:sch.Jobs.s_cron ~every_ms:sch.Jobs.s_every_ms
          with _ -> ());
         fire_schedules ()
   in
   try fire_schedules () with _ -> ());
  let claimed = (try Jobs.claim_due ~store:session.store with _ -> None)
   in
  (match claimed with
   | Some job ->
     (try
        let result, ok = eval_job ~job in
        if ok then
          Jobs.record_done ~store:session.store ~id:job.Jobs.id
            ~result_json:result
        else
          Jobs.record_failed ~store:session.store ~id:job.Jobs.id
            ~error:result ~max_attempts:job.Jobs.max_attempts
      with exn ->
        (try
           Jobs.record_failed ~store:session.store ~id:job.Jobs.id
             ~error:(Printexc.to_string exn)
             ~max_attempts:job.Jobs.max_attempts
         with _ -> ()));
     loop ()
   | None ->
     Lwt_eio.run_lwt (fun () -> Lwt_unix.sleep 1.0);
     loop ())
in
Printf.eprintf "[jobs] runner fiber started (runner_id=%s)\n%!"
  Jobs.runner_id;
loop ()
```

```
let start_eio ~(env : Eio_unix.Stdenv.base) ~(sw : Eio.Switch.t) : unit =
  let session, http_port, repl_port = prepare () in
  (* REPL still uses Lwt_unix sockets under Lwt_eio. *)
  Eio.Fiber.fork ~sw (fun () ->
      try
        Lwt_eio.run_lwt (fun () ->
            Repl_endpoint.serve ~session ~port:repl_port ())
      with exn ->
        Printf.eprintf "[boot] REPL fiber exited: %s\n%!"
          (Printexc.to_string exn));

  (** Fork the jobs runner fiber -- a background loop that claims due
      jobs and evals them. It gets its own store+session so it doesn't
      race HTTP on the shared session's branch. Spec: borge/jobs.borg. *)
  Eio.Fiber.fork ~sw (fun () ->
      try
        let cfg = Store.config_from_env () in
        let runner_store = Store.make ~cfg ~branch_id:Store.system_main_id
    () in
        Store.connect runner_store;
        let manifest =
          Pricklypear_common.Manifest.make
            ~user_id:"jobs-runner"
            ~username:"admin"
            ~branch_id:Store.system_main_id
            ~branch_name:"system/main"
            ~boot_mode:Pricklypear_common.Manifest.Boot_always_on
            ~image_version:session.manifest.image_version
            ()
        in
        let runner_session = Session.make ~manifest ~store:runner_store ()
    in
        jobs_runner_loop ~session:runner_session
      with exn ->
        Printf.eprintf "[boot] jobs runner fiber exited: %s\n%!"
          (Printexc.to_string exn));

  (* image.function-stat-ring: flush labeled-eval stats <= every 30s
     instead of a synchronous insert per /api/eval. *)
  Eio.Fiber.fork ~sw (fun () ->
      try
        Function_stats.flush_loop ~clock:(Eio.Stdenv.clock env)
          ~store:session.store
      with exn ->
        Printf.eprintf "[boot] function_stats flush fiber exited: %s\n%!"
          (Printexc.to_string exn));
  (* image.per-identity-sessions: HTTP requests run on per-identity
     sessions (own env, own pgx connection); the boot session keeps the
     REPL endpoint + auth lookups. *)
  let registry = Session_registry.create ~boot:session in
  Printf.eprintf "[boot] serving http://0.0.0.0:%d/  (htmx, httpun-eio)\n%!"
    http_port;
  Http.serve_eio ~session ~registry ~env ~sw ~port:http_port ()

(** Legacy name -- same as start_eio without env (not used by main). *)
let start () : unit =
  failwith "Boot.start: use Boot.start_eio under Eio_main
     (image/bin/main.ml)"
```

64 image/lib/call_graph.ml

```
(** Static App-Var call graph for inspector / impact / DNU.
    Spec: image.inspector phases I1+; shared by system-browser + dnu-stubs.
     *)


module Ast = Pricklypear_lisp.Ast

let rec collect_var_apps (acc : string list) (e : Ast.expr) : string list =
  match e with
  | Ast.Var _ | Ast.Lit _ -> acc
  | Ast.App (Ast.Var f, args) ->
      List.fold_left collect_var_apps (f :: acc) args
  | Ast.App (f, args) ->
      List.fold_left collect_var_apps (collect_var_apps acc f) args
  | Ast.Let (_, e1, e2) -> collect_var_apps (collect_var_apps acc e1) e2
  | Ast.Letrec (binds, body) ->
      let acc =
        List.fold_left (fun a (_, e) -> collect_var_apps a e) acc binds
      in
      collect_var_apps acc body
  | Ast.If (c, t, e) ->
      collect_var_apps (collect_var_apps (collect_var_apps acc c) t) e
  | Ast.Lambda (_, body) -> collect_var_apps acc body
  | Ast.Prim (_, args) -> List.fold_left collect_var_apps acc args
  | Ast.Seq (a, b) -> collect_var_apps (collect_var_apps acc a) b
  | Ast.Quote _ -> acc
  | Ast.Define (_, body) -> collect_var_apps acc body

let uniq_stable (xs : string list) : string list =
  let seen = Hashtbl.create 16 in
  List.fold_left
    (fun acc x ->
      if Hashtbl.mem seen x then acc
      else begin
        Hashtbl.add seen x ();
        x :: acc
      end)
    [] xs
  |> List.rev

type snapshot = {
  (** name ? body expr (visible function_defs) *)
  bodies : (string, Ast.expr) Hashtbl.t;
  (** callee ? direct callers (App-Var heads; includes missing callees) *)
  callers_of : (string, string list) Hashtbl.t;
  (** name ? direct callees that are stored function_defs *)
  callees_of : (string, string list) Hashtbl.t;
  (** route path ? handler name *)
  routes : (string * string) list;
  (** prim + special-form names for holes() *)
  bound_kernel : (string, unit) Hashtbl.t;
}

let special_forms : string list =
  [ "lambda"; "let"; "letrec"; "if"; "define"; "quote"; "seq"; "begin"
  ; "and"; "or"; "cond"; "match"; "with-transaction"
  ; "quasiquote"; "unquote"; "unquote-splicing"; "defmacro"
  ; "guard"; "error"
  ]

let build_snapshot
    ~(defs : (string * Ast.expr) list)
    ~(routes : (string * string) list)
    ~(prim_names : string list)
    : snapshot =
  let bodies = Hashtbl.create (List.length defs + 8) in
  List.iter (fun (n, e) -> Hashtbl.replace bodies n e) defs;
  let callers_of : (string, string list) Hashtbl.t = Hashtbl.create 64 in
  let callees_of : (string, string list) Hashtbl.t = Hashtbl.create 64 in
  let add_caller ~callee ~caller =
    let prev = try Hashtbl.find callers_of callee with Not_found -> [] in
```

```
    if List.mem caller prev then ()
    else Hashtbl.replace callers_of callee (caller :: prev)
  in
  List.iter
    (fun (name, expr) ->
      let apps = uniq_stable (collect_var_apps [] expr) in
      let fn_callees =
        List.filter
          (fun c -> c <> name && Hashtbl.mem bodies c)
          apps
      in
      Hashtbl.replace callees_of name fn_callees;
      List.iter (fun c -> add_caller ~callee:c ~caller:name) apps)
    defs;
  let bound_kernel = Hashtbl.create 64 in
  List.iter (fun n -> Hashtbl.replace bound_kernel n ()) special_forms;
  List.iter (fun n -> Hashtbl.replace bound_kernel n ()) prim_names;
  { bodies; callers_of; callees_of; routes; bound_kernel }

let direct_callees (snap : snapshot) (name : string) : string list =
  try Hashtbl.find snap.callees_of name with Not_found -> []

let direct_callers (snap : snapshot) (name : string) : string list =
  try Hashtbl.find snap.callers_of name with Not_found -> []

(** Strongly connected components of size > 1 (mutual recursion groups),
    plus trivial self-loops already covered by recursion.analyze_self. *)
let mutual_cycles (snap : snapshot) : string list list =
  let index = ref 0 in
  let stack : string list ref = ref [] in
  let on_stack = Hashtbl.create 32 in
  let indices = Hashtbl.create 32 in
  let lowlink = Hashtbl.create 32 in
  let comps : string list list ref = ref [] in
  let rec strongconnect v =
    Hashtbl.replace indices v !index;
    Hashtbl.replace lowlink v !index;
    incr index;
    stack := v :: !stack;
    Hashtbl.replace on_stack v true;
    List.iter
      (fun w ->
        if not (Hashtbl.mem indices w) then begin
          strongconnect w;
          Hashtbl.replace lowlink v
            (min (Hashtbl.find lowlink v) (Hashtbl.find lowlink w))
        end else if Hashtbl.mem on_stack w && Hashtbl.find on_stack w then
          Hashtbl.replace lowlink v
            (min (Hashtbl.find lowlink v) (Hashtbl.find indices w)))
      (direct_callees snap v);
    if Hashtbl.find lowlink v = Hashtbl.find indices v then begin
      let rec pop acc =
        match !stack with
        | [] -> acc
        | x :: xs ->
            stack := xs;
            Hashtbl.replace on_stack x false;
            if x = v then x :: acc else pop (x :: acc)
      in
      let comp = pop [] in
      if List.length comp > 1 then comps := comp :: !comps
    end
  in
  Hashtbl.iter
    (fun name _ -> if not (Hashtbl.mem indices name) then strongconnect
     name)
    snap.bodies;
  !comps

let mutual_of (snap : snapshot) (name : string) : string list =
  match
    List.find_opt (fun comp -> List.mem name comp) (mutual_cycles snap)
  with
```

```
  | None -> []
  | Some comp -> List.filter (( <> ) name) comp

(** Forward BFS over stored-function edges. [include_root] keeps [name]. *)
let walk_forward (snap : snapshot) ~(name : string) ~(depth : int)
    ~(include_root : bool) : string list =
  let max_d = max 0 depth in
  let seen = Hashtbl.create 32 in
  let rec bfs q acc =
    match q with
    | [] -> List.rev acc
    | (n, d) :: rest ->
        if d > max_d || Hashtbl.mem seen n then bfs rest acc
        else begin
          Hashtbl.add seen n ();
          let acc =
            if (not include_root) && n = name then acc else n :: acc
          in
          if d >= max_d then bfs rest acc
          else
            let next =
              List.filter_map
                (fun c ->
                  if Hashtbl.mem seen c then None else Some (c, d + 1))
                (direct_callees snap n)
            in
            bfs (rest @ next) acc
        end
  in
  bfs [ (name, 0) ] []

(** Reverse BFS (dependents). Excludes [name] itself. *)
let dependents (snap : snapshot) ~(name : string) ~(depth : int)
    : string list =
  let max_d = max 0 depth in
  let seen = Hashtbl.create 32 in
  Hashtbl.add seen name ();
  let rec bfs q acc =
    match q with
    | [] -> List.rev acc
    | (n, d) :: rest ->
        if d >= max_d then bfs rest acc
        else
          let callers = direct_callers snap n in
          let rec fold cs q acc =
            match cs with
            | [] -> bfs q acc
            | c :: tl ->
                if Hashtbl.mem seen c then fold tl q acc
                else begin
                  Hashtbl.add seen c ();
                  fold tl (q @ [ (c, d + 1) ]) (c :: acc)
                end
          in
          fold callers rest acc
  in
  bfs [ (name, 0) ] []

let callees (snap : snapshot) ~(name : string) ~(depth : int) : string list
    =
  if depth <= 1 then direct_callees snap name
  else
    (* depth>1: walk but drop root *)
    List.filter (( <> ) name) (walk_forward snap ~name ~depth
     ~include_root:true)

let callers = direct_callers

let used_by_routes (snap : snapshot) (name : string) : string list =
  let deps = dependents snap ~name ~depth:8 in
  let is_hit h = h = name || List.mem h deps in
  snap.routes
  |> List.filter_map (fun (path, handler) ->
```

```
        if is_hit handler then Some path else None)
  |> uniq_stable

let holes (snap : snapshot) (name : string) : string list =
  match Hashtbl.find_opt snap.bodies name with
  | None -> []
  | Some expr ->
      let apps = uniq_stable (collect_var_apps [] expr) in
      List.filter
        (fun c ->
          (not (Hashtbl.mem snap.bodies c))
          && not (Hashtbl.mem snap.bound_kernel c))
        apps

(** Load defs + routes from session store and build snapshot. *)
let snapshot_of_session (session : Session_types.t) : snapshot =
  let defs = Store.list_function_names session.store in
  let bodies = List.map (fun (n, e, _) -> (n, e)) defs in
  let routes =
    Hashtbl.fold
      (fun path (handler, _) acc -> (path, handler) :: acc)
      session.routes []
  in
  let prim_names = List.map fst session.kernel_catalog in
  (build_snapshot ~defs:bodies ~routes ~prim_names)

let take n xs =
  let rec go i = function
    | [] -> []
    | _ when i <= 0 -> []
    | x :: rest -> x :: go (i - 1) rest
  in
  go n xs

(** After successful function_def write: impact queue + change transcript.
    *)
let record_define_side_effects (t : Session_types.t) ~(name : string)
    : unit =
  if t.suppress_refresh_on_define || t.manifest.username = "anonymous" then
    ()
  else
    let snap = snapshot_of_session t in
    let deps = dependents snap ~name ~depth:8 in
    let routes = used_by_routes snap name in
    let now = Unix.gettimeofday () in
    let branch = t.manifest.branch_name in
    let author = t.manifest.username in
    let ch : Session_types.change_event =
      { op = "define"; name; at = now; branch; author }
    in
    t.change_transcript <- take 64 (ch :: t.change_transcript);
    if deps = [] && routes = [] then ()
    else begin
      let ev : Session_types.impact_event =
        { name; at = now; branch; author; dependents = deps;
          routes; status = "pending" }
      in
      let rest =
        List.filter
          (fun (e : Session_types.impact_event) -> e.name <> name)
          t.impact_queue
      in
      t.impact_queue <- take 32 (ev :: rest);
      ()
    end

let impact_summary_text (t : Session_types.t) (name : string) : string =
  match
    List.find_opt
      (fun (e : Session_types.impact_event) -> e.name = name)
      t.impact_queue
  with
  | None -> "impact: no routes; eval dependents or smoke parents manually"
```

```
  | Some e ->
      let deps =
        if e.dependents = [] then "(none)"
        else String.concat "," e.dependents
      in
      let rts =
        if e.routes = [] then "(none)"
        else String.concat " " e.routes
      in
      Printf.sprintf "impact: dependents=%s; smoke routes: %s" deps rts

let page_function_graph (session : Session_types.t) ~(handler : string)
    : (string * string) list =
  let snap = snapshot_of_session session in
  let max_depth = 8 in
  let rec walk depth seen name role acc =
    if depth > max_depth || List.mem name seen then acc
    else
      match Hashtbl.find_opt snap.bodies name with
      | None -> ((name, role ^ " (missing)") :: acc)
      | Some _ ->
          let acc = (name, role) :: acc in
          let seen = name :: seen in
          let cs = direct_callees snap name in
          let rec fold_c cs acc =
            match cs with
            | [] -> acc
            | c :: rest ->
                let acc = walk (depth + 1) seen c "callee" acc in
                fold_c rest acc
          in
          fold_c cs acc
  in
  let rows = walk 0 [] handler "handler" [] in
  (List.rev rows)
```

65 image/lib/control_rim.ml

```
(** Image rim: service [Control.Prim_call] with direct-style prim tables.

    Lives outside [lisp/] so the interpreter stays free of image I/O deps.
    Prim table entries are [value list -> value] (sync). Process I/O blocks
    the Eio fiber inside prim bodies via Store_db.

    Nested Prim_call: while a prim body runs, [Control.direct_prims] is set
    so nested eval (eval-string, load-library internals, etc.) dispatches
    prims by direct table lookup instead of Effect.perform -- effect
     handlers
    do not re-enter for nested performs from within the handler body. *)

open Pricklypear_lisp
open Ast

let run ~(prims : (string, Ast.value list -> Ast.value) Hashtbl.t)
    (thunk : unit -> 'a) : 'a =
  Control.run_with_rim ~prims thunk

(** Alias kept for call sites that still say run_lwt. *)
let run_lwt = run
```

66 image/lib/crud.ml

```
(** Pricklypear Image -- CRUD primitives over the one-tree datum namespace.

    get/create/update/delete/rows are normal prims returning real DB
    results. They compose:
      (define u (get 'user 42))
      (str u)

    Per spec [one-tree.branching-interaction] /data DECISION (user-owned
    data) + [one-tree.migration-plan] (user-scoping):
      - Datum nodes live in tree_nodes; reads project through the
        tree_data_rows view. The scaffolding data_rows table is the
        reverse-mirror kept warm for residual readers / fork-copy.
      - Data is NOT branch-versioned (no branch_id); fork/merge copies
        code, not data.
      - Visibility is USER-SCOPED: a session sees rows it authored plus
        the shared seed corpus (bible-*, strongs-entry). Prim SQL
        enforces this; RLS on tree_nodes is the floor (migration 013).
      - `fields` is JSON; Lisp sees it as a string, with dict-* prims
        for field access.
      - No ORM; parameterized SQL via pgx. Multi-step atomicity uses
        Store.with_dynamic_transaction / (with-transaction ?).

    Uses [Io_lwt.Db.current_conn] so in-tx calls see the bound connection
    (read prims run inside with_transaction so set_local_pp_user keeps the
    RLS floor active; see i-rls-nontx).  Misses return VNil so Lisp can
    branch on `(if (get 'user id) ...)`. *)
open Pricklypear_lisp.Ast

(* Reads project through tree_data_rows (the tree_nodes datum view),
   UNCONDITIONALLY, with an explicit author/seed confinement predicate
   (primary control) plus the RLS floor on tree_nodes (migration 013).
   tree_data_rows is not behind the legacy PP_TREE_READS flag -- the tree
   view is the read source of truth. Writes go through Tree_store into
   tree_nodes directly; the reverse trigger mirrors tree_nodes back into
   the data_rows scaffolding table (residual readers / fork-copy). *)
let data_rel () = "tree_data_rows"

(** Session username for confinement. Prefer Request_context snapshot
    (cotenant-safe); fall back to anonymous when unbound (scripts). *)
let session_user () : string =
  match Request_context.get () with
  | Some c -> (Request_context.manifest c).username
  | None -> "anonymous"

(** SQL fragment: row is visible to $N (author match OR seed type).
    Seed membership derives from the authoritative SQL predicate
    tree_is_seed_type (migrations/016_seed_single_source.sql) rather than a
    duplicated IN-list, so CRUD reads never drift from the RLS floor. *)
let visible_sql ~(user_param : int) : string =
  Printf.sprintf "(author = $%d OR tree_is_seed_type(type_name))" user_param

(** If [type_name] is declared on the store's branch, validate [fields_json]
    against its field list. Undeclared types are free-form (no schema).
    Returns None on ok, Some error_message on reject. *)
let check_type_fields (store : Store.t) ~(type_name : string)
    ~(fields_json : string) : string option =
  let declared = Store.lookup_type_fields store ~name:type_name in
  match declared with
  | None -> None
  | Some fields ->
      (match Tree_store.validate_fields ~declared:fields fields_json with
       | Ok () -> None
       | Error msg ->
           Some
             (Printf.sprintf "create/update %s: %s" type_name msg))

let value_of_id cell : value =
  (* Pgx.Value.to_string returns string option; normalize to VStr/VNil *)
  match Pgx.Value.to_string cell with
  | Some s -> VStr s
  | None -> VNil
```

```
72
73  (** (deftype 'name field1 field2 ...) -> VStr "defined type <name>".
74      INSERTs a row into type_defs with branch_id = session's branch and
75      fields = JSON array ["field1","field2",...] (the field names in
76      declared order). Per spec [storage.types]: type declarations are
77      metadata; the actual data lives in datum nodes (tree_nodes, projected
78      via tree_data_rows), user-scoped + shared seed (migration 013)
79      keyed by (type_name, id). Type declarations may
80      eventually drive field-level validation; for now they're documentation.
81
82      The first arg is the type name, then zero or more field names --
83      each as a quoted symbol (e.g. (deftype 'user 'name 'email 'age))
84      so they arrive as VStr (quote of a bare symbol evaluates to VStr).
85      Bare symbols would be looked up as variables and fail. *)
86  let deftype (store : Store.t) ~(args : value list)
87      ~(author : string) : value =
88    let field_strs =
89      List.filter_map
90        (fun v -> match v with VStr s -> Some s | _ -> None)
91        args
92    in
93    (match field_strs with
94     | [] ->
95         raise (Pricklypear_lisp.Ast.Interp_error
96                  (Pricklypear_lisp.Ast.Prim_error
97                     ("deftype", "need at least a type name", "bad-args")))
98     | name :: fields ->
99         let _ = Store.persist_type store ~name ~fields ~author in
100        (VStr (Printf.sprintf "defined type %s" name)))
101
102 (** (get 'type id) -> VStr fields-json | VNil.
103     Reads ONE row by (type_name, id). The id is a UUID string.
104     Returns the row's `fields` JSON as a VStr so Lisp can pass it
105     onward; or VNil if not found. *)
106 let get (store : Store.t) ~(type_name : string) ~(id : string) : value =
107   (* Route through with_transaction so set_local_pp_user runs and the RLS
108      floor (tree_data_rows / tree_nodes policy, migration 013) is active.
109      The callback conn is the tx conn (set pp.user is LOCAL to it). *)
110   Io_lwt.Db.with_transaction store (function
111     | None -> VNil
112     | Some conn ->
113         let user = session_user () in
114         let sql =
115           "SELECT fields::text FROM " ^ data_rel () ^ " " ^
116           "WHERE type_name = $1 AND id = $2::uuid AND " ^
117           visible_sql ~user_param:3 in
118         let rows =
119           Io_lwt.Db.execute conn
120             ~params:[ Pgx.Value.of_string type_name;
121                       Pgx.Value.of_string id;
122                       Pgx.Value.of_string user ]
123             sql
124         in
125         (match rows with
126          | [] -> VNil
127          | row :: _ ->
128              (match row with
129               | [ fields_cell ] ->
130                   (match Pgx.Value.to_string fields_cell with
131                    | Some s -> (VStr s)
132                    | None -> VNil)
133               | _ -> VNil)))
134
135 (** (select-for-update 'type id) -> VStr fields-json | VNil.
136     Reads ONE row by (type_name, id) AND locks it with SELECT ... FOR
137     UPDATE for the duration of the surrounding transaction.
138
139     Per spec [storage.transactions]: "Explicit row locks:
140     (select-for-update 'type id) Exposed as a primitive for finer
141     control."
142
143     MUST be called inside (with-transaction ...). Outside a dynamic
144     transaction this raises a clean Interp_error (User_error) instead of
```

```
    issuing FOR UPDATE on the pool conn -- which previously could poison
    Eio mutexes and cascade into auth failures (observed 2026-07-22). *)
let select_for_update
    (store : Store.t) ~(type_name : string) ~(id : string) : value =
  if not (Io_lwt.Db.in_transaction store) then
    raise
      (Interp_error
         (User_error
            ("select-for-update requires (with-transaction ...); \
             refusing FOR UPDATE outside a transaction", "bad-args")))
  else
    match Io_lwt.Db.current_conn store with
    | None -> VNil
    | Some conn ->
        let user = session_user () in
        let sql =
          "SELECT fields::text FROM " ^ data_rel () ^ " " ^
          "WHERE type_name = $1 AND id = $2::uuid AND " ^
          visible_sql ~user_param:3 ^ " " ^
          "FOR UPDATE" in
        let rows =
          Io_lwt.Db.execute conn
            ~params:[ Pgx.Value.of_string type_name;
                      Pgx.Value.of_string id;
                      Pgx.Value.of_string user ]
            sql
        in
        (match rows with
         | [] -> VNil
         | row :: _ ->
             (match row with
              | [ fields_cell ] ->
                  (match Pgx.Value.to_string fields_cell with
                   | Some s -> (VStr s)
                   | None -> VNil)
              | _ -> VNil))

(** (create 'type "{fields-json}") -> VStr new-id; raises Prim_error
    ("create", "failed", "failed") when the insert is rejected.
    INSERTs a row with the given JSON fields, returning the new UUID.
    The author is the session's current username.

    Why take fields as a JSON STRING (not a Lisp value):
    - The Lisp layer doesn't yet have a dict/assoc value type.
    - Caller can build the JSON via (string-append ...) or a future
      (json ...) prim; for now both caller and DB speak JSON strings.
    - The DB validates it's well-formed JSON (jsonb column). *)
let create
    (store : Store.t) ~(type_name : string) ~(fields_json : string)
    ~(author : string)
    : value =
  let reject = check_type_fields store ~type_name ~fields_json in
  match reject with
  | Some msg -> (VStr msg)
  | None ->
  match Io_lwt.Db.current_conn store with
  | None ->
      raise (Pricklypear_lisp.Ast.Interp_error
               (Pricklypear_lisp.Ast.Prim_error
                  ("create", "failed: no DB", "failed")))
  | Some conn ->
    (* tree is truth; reverse trigger mirrors to data_rows *)
    let id = Tree_store.insert_datum conn ~type_name ~fields_json ~author in
    (match id with
     | Some s -> VStr s
     | None ->
         raise (Pricklypear_lisp.Ast.Interp_error
                  (Pricklypear_lisp.Ast.Prim_error
                     ("create", "failed", "failed"))))

(** (update 'type id "{fields-json}") -> VStr id | VNil.
    UPDATE fields for ONE row by (type_name, id). Returns the id
    (echo of the row updated) on success, or VNil if no row matched. *)
```

```
let update
    (store : Store.t) ~(type_name : string) ~(id : string)
    ~(fields_json : string)
    : value =
  let reject = check_type_fields store ~type_name ~fields_json in
  match reject with
  | Some msg -> (VStr msg)
  | None ->
  match Io_lwt.Db.current_conn store with
  | None -> VNil
  | Some conn ->
    let id' = Tree_store.update_datum conn ~type_name ~id ~fields_json in
    (match id' with Some s -> VStr s | None -> VNil)

(** (update-field 'type id "field" "value") -> VStr id | VNil.
    Ergonomic shorthand for the v1 dict-set + update round-trip:
      (define u (get 'type id))
      (update 'type id (dict-set u "field" "value"))
    -> (update-field 'type id "field" "value")
    Composition is pure-OCaml (one get, one dict-set, one update inside
    this prim). Lwt-typed; returns VStr id on success, VNil if no row
    matched or no DB. Value is JSON-coerced (numeric strings emit as JSON
    numbers; non-parseable values emit as JSON strings -- same rules as
    dict-set).

    Runs in ONE transaction: the row is read with SELECT ... FOR UPDATE,
    the field merged in OCaml, and the UPDATE issued in the same tx. The
    FOR UPDATE lock serializes concurrent update-field calls on the same
    row, so two writers updating different fields both land (no
    lost-update race). The tx conn (with pp.user LOCAL set) is used
    throughout, so the RLS floor stays active. *)
let update_field
    (store : Store.t) ~(type_name : string) ~(id : string)
    ~(field : string) ~(value : string)
    : value =
  (* Preserve type-field validation (update_field used to route through
     update -> check_type_fields). Build a throwaway single-field JSON to
     reuse the same rejection path. *)
  let sample_json = Printf.sprintf "{\"%s\":\"%s\"}" field value in
  let reject = check_type_fields store ~type_name ~fields_json:sample_json
    in
  match reject with
  | Some msg -> VStr msg
  | None ->
  Io_lwt.Db.with_transaction store (function
    | None -> VNil
    | Some conn ->
        let user = session_user () in
        let sql =
          "SELECT fields::text FROM " ^ data_rel () ^ " " ^
          "WHERE type_name = $1 AND id = $2::uuid AND " ^
          visible_sql ~user_param:3 ^ " FOR UPDATE" in
        let rows =
          Io_lwt.Db.execute conn
            ~params:[ Pgx.Value.of_string type_name;
                      Pgx.Value.of_string id;
                      Pgx.Value.of_string user ]
            sql
        in
        (match rows with
         | [] -> VNil  (* no such row / not visible *)
         | row :: _ ->
             let json_str =
               match row with
               | [ fields_cell ] ->
                   (match Pgx.Value.to_string fields_cell with
                    | Some s -> s
                    | None -> "{}")
               | _ -> "{}"
             in
             (* Compose with the SAME logic as Primitives.prim_dict_set so
                numeric-auto-detect and JSON passthrough behave
    identically. *)
```

```
289             let base =
290               try
291                 (match Yojson.Safe.from_string json_str with
292                  | `Assoc kvs -> kvs
293                  | _ -> [])
294               with _ -> []
295             in
296             let v_json =
297               try Yojson.Safe.from_string value
298               with _ -> `String value
299             in
300             let updated =
301               (field, v_json) ::
302               (List.filter (fun (k, _) -> k <> field) base)
303             in
304             let updated_json = Yojson.Safe.to_string (`Assoc updated) in
305             let id' =
306               Tree_store.update_datum conn ~type_name ~id
    ~fields_json:updated_json
307             in
308             (match id' with Some s -> VStr s | None -> VNil)))
309
310 (** (delete 'type id) -> VStr id | VNil.
311     DELETEs one row by (type_name, id). Returns the id on success
312     (echo), VNil if no row matched. *)
313 let delete (store : Store.t) ~(type_name : string) ~(id : string) : value =
314   match Io_lwt.Db.current_conn store with
315   | None -> VNil
316   | Some conn ->
317     let id' = Tree_store.delete_datum conn ~type_name ~id in
318     (match id' with Some s -> VStr s | None -> VNil)
319
320 (** (list 'type [limit] [sort]) -> VList of (VStr id, VStr fields) pairs.
321     With no limit, defaults to 100 (avoid OOM on a huge table). Optional
322     sort is a raw SQL ORDER BY clause (appended verbatim); default
323     ORDER BY is created_at ASC. Each row is a 2-element VList
324     `[id fields]` so callers can `(mapcar (lambda (row) (car row)) rows)`
325     to get ids, etc. *)
326 let list
327     (store : Store.t) ~(type_name : string) ?(limit : int = 100)
328     ?(sort : string option = None) ()
329     : value =
330   Io_lwt.Db.with_transaction store (function
331     | None -> (VList [])
332     | Some conn ->
333         let user = session_user () in
334         let order_by =
335           match sort with
336           | Some s -> " ORDER BY " ^ s
337           | None -> " ORDER BY created_at ASC"
338         in
339         let sql =
340           "SELECT id::text, fields::text FROM " ^ data_rel () ^ " " ^
341           "WHERE type_name = $1 AND " ^ visible_sql ~user_param:3 ^
342           order_by ^ " LIMIT $2" in
343         let rows =
344           Io_lwt.Db.execute conn
345             ~params:[ Pgx.Value.of_string type_name;
346                       Pgx.Value.of_int limit;
347                       Pgx.Value.of_string user ]
348             sql
349         in
350         let lisp_rows =
351           List.map
352             (fun row ->
353               match row with
354               | [ id_cell; fields_cell ] ->
355                   VPair (value_of_id id_cell, value_of_id fields_cell)
356               | _ -> VNil)
357             rows
358         in
359         (VList lisp_rows))
360
```

```
(** (find-row 'type "field" "value") -> VStr fields-json | VNil.
    First row where fields->>field = value (text equality). *)
let find_row
    (store : Store.t) ~(type_name : string) ~(field : string) ~(value :
     string)
    : value =
  Io_lwt.Db.with_transaction store (function
    | None -> VNil
    | Some conn ->
        let user = session_user () in
        let sql =
          "SELECT fields::text FROM " ^ data_rel () ^ " " ^
          "WHERE type_name = $1 AND fields->>$2 = $3 AND " ^
          visible_sql ~user_param:4 ^ " " ^
          "ORDER BY created_at ASC LIMIT 1"
        in
        let rows =
          Io_lwt.Db.execute conn
            ~params:[ Pgx.Value.of_string type_name
                    ; Pgx.Value.of_string field
                    ; Pgx.Value.of_string value
                    ; Pgx.Value.of_string user ]
            sql
        in
        (match rows with
         | [] -> VNil
         | row :: _ ->
             (match row with
              | [ fields_cell ] -> (value_of_id fields_cell)
              | _ -> VNil)))

(** (find-rows 'type "f1" "v1" "f2" "v2" ... [limit] [sort])
    -> VList of (id . fields) pairs.
    All field filters AND together (fields->>k = v text). Optional
    trailing VInt limit (default 500) and an optional trailing VStr sort
    (a raw SQL ORDER BY clause, appended verbatim). Default ORDER BY:
    created_at ASC. Callers needing domain-specific ordering (e.g. bible
    verses by verse number) must pass the sort explicitly. *)
let find_rows
    (store : Store.t) ~(type_name : string)
    ~(filters : (string * string) list) ?(limit : int = 500)
    ?(sort : string option = None) ()
    : value =
  Io_lwt.Db.with_transaction store (function
    | None -> (VList [])
    | Some conn ->
        if filters = [] then list store ~type_name ~limit ()
        else
          let user = session_user () in
          let buf = Buffer.create 128 in
          Buffer.add_string buf
            ("SELECT id::text, fields::text FROM " ^ data_rel () ^ " WHERE
     type_name = $1");
          let params = ref [ Pgx.Value.of_string type_name ] in
          let i = ref 2 in
          List.iter
            (fun (field, value) ->
              Buffer.add_string buf
                (Printf.sprintf " AND fields->>$%d = $%d" !i (!i + 1));
              params := !params @ [ Pgx.Value.of_string field
                                  ; Pgx.Value.of_string value ];
              i := !i + 2)
            filters;
          Buffer.add_string buf
            (Printf.sprintf " AND %s" (visible_sql ~user_param:!i));
          params := !params @ [ Pgx.Value.of_string user ];
          i := !i + 1;
          let order_by =
            match sort with
            | Some s -> " ORDER BY " ^ s ^ " LIMIT "
            | None -> " ORDER BY created_at ASC LIMIT "
          in
          Buffer.add_string buf (Printf.sprintf "%s$%d" order_by !i);
```

```
          params := !params @ [ Pgx.Value.of_int limit ];
          let rows = Io_lwt.Db.execute conn ~params:!params
     (Buffer.contents buf) in
          let lisp_rows =
            List.map
              (fun row ->
                match row with
                | [ id_cell; fields_cell ] ->
                    VPair (value_of_id id_cell, value_of_id fields_cell)
                | _ -> VNil)
              rows
          in
          (VList lisp_rows))

(** (find-random 'type) -> (id . fields) pair | VNil.
    One random row of the given type (for bible/random etc.). *)
let find_random (store : Store.t) ~(type_name : string) : value =
  Io_lwt.Db.with_transaction store (function
    | None -> VNil
    | Some conn ->
        let user = session_user () in
        let sql =
          "SELECT id::text, fields::text FROM " ^ data_rel () ^ " " ^
          "WHERE type_name = $1 AND " ^ visible_sql ~user_param:2 ^ " " ^
          "ORDER BY random() LIMIT 1"
        in
        let rows =
          Io_lwt.Db.execute conn
            ~params:[ Pgx.Value.of_string type_name;
                      Pgx.Value.of_string user ]
            sql
        in
        (match rows with
         | [] -> VNil
         | row :: _ ->
             (match row with
              | [ id_cell; fields_cell ] ->
                  (VPair (value_of_id id_cell, value_of_id fields_cell))
              | _ -> VNil)))

(** (find-text 'type "field" "needle" [limit]) -> VList of (id . fields).
    Case-insensitive substring match on fields->>field. *)
let find_text
    (store : Store.t) ~(type_name : string) ~(field : string)
    ~(needle : string) ?(limit : int = 50) ()
    : value =
  Io_lwt.Db.with_transaction store (function
    | None -> (VList [])
    | Some conn ->
        let user = session_user () in
        let sql =
          "SELECT id::text, fields::text FROM " ^ data_rel () ^ " " ^
          "WHERE type_name = $1 AND fields->>$2 ILIKE $3 AND " ^
          visible_sql ~user_param:5 ^ " " ^
          "ORDER BY created_at ASC LIMIT $4"
        in
        let pat = "%" ^ needle ^ "%" in
        let rows =
          Io_lwt.Db.execute conn
            ~params:[ Pgx.Value.of_string type_name
                    ; Pgx.Value.of_string field
                    ; Pgx.Value.of_string pat
                    ; Pgx.Value.of_int limit
                    ; Pgx.Value.of_string user ]
            sql
        in
        let lisp_rows =
          List.map
            (fun row ->
              match row with
              | [ id_cell; fields_cell ] ->
                  VPair (value_of_id id_cell, value_of_id fields_cell)
              | _ -> VNil)
```

```
            rows
        in
        (VList lisp_rows))

(* ---- OCaml UI helpers (image.data-browser) -- not Lisp prims ---- *)

type row_record = {
  id : string;
  fields_json : string;
  author : string option;
  created_at : string option;
  updated_at : string option;
}

let count_by_type (store : Store.t) ~(type_name : string) : int =
  match Io_lwt.Db.current_conn store with
  | None -> 0
  | Some conn ->
      let user = session_user () in
      let sql =
        "SELECT COUNT(*)::int FROM " ^ data_rel () ^
        " WHERE type_name = $1 AND " ^ visible_sql ~user_param:2
      in
      let rows =
        Io_lwt.Db.execute conn
          ~params:[ Pgx.Value.of_string type_name;
                    Pgx.Value.of_string user ]
          sql
      in
      (match rows with
       | [ cell :: _ ] ->
           (match Pgx.Value.to_int cell with
            | Some n -> n
            | None -> 0)
       | _ -> 0)

let list_data_type_names (store : Store.t) : string list =
  match Io_lwt.Db.current_conn store with
  | None -> []
  | Some conn ->
      let user = session_user () in
      let sql =
        "SELECT DISTINCT type_name FROM " ^ data_rel () ^
        " WHERE " ^ visible_sql ~user_param:1 ^
        " ORDER BY type_name ASC"
      in
      let rows =
        Io_lwt.Db.execute conn
          ~params:[ Pgx.Value.of_string user ]
          sql
      in
      (List.filter_map
           (fun row ->
             match row with
             | [ cell ] -> Pgx.Value.to_string cell
             | _ -> None)
           rows)

let list_page
    (store : Store.t) ~(type_name : string)
    ?(limit : int = 50) ?(offset : int = 0) ()
    : row_record list =
  match Io_lwt.Db.current_conn store with
  | None -> []
  | Some conn ->
      let lim = max 1 (min limit 200) in
      let off = max 0 offset in
      let user = session_user () in
      let sql =
        "SELECT id::text, fields::text, author, " ^
        "created_at::text, updated_at::text " ^
        "FROM " ^ data_rel () ^ " WHERE type_name = $1 AND " ^
        visible_sql ~user_param:4 ^ " " ^
```

```
        "ORDER BY created_at ASC LIMIT $2 OFFSET $3"
      in
      let rows =
        Io_lwt.Db.execute conn
          ~params:[ Pgx.Value.of_string type_name
                  ; Pgx.Value.of_int lim
                  ; Pgx.Value.of_int off
                  ; Pgx.Value.of_string user ]
          sql
      in
      (List.filter_map
           (fun row ->
             match row with
             | [ id_c; fields_c; author_c; created_c; updated_c ] ->
                 (match Pgx.Value.to_string id_c,
                        Pgx.Value.to_string fields_c with
                  | Some id, Some fields_json ->
                      Some {
                        id;
                        fields_json;
                        author = Pgx.Value.to_string author_c;
                        created_at = Pgx.Value.to_string created_c;
                        updated_at = Pgx.Value.to_string updated_c;
                      }
                  | _ -> None)
             | _ -> None)
           rows)

let get_row
    (store : Store.t) ~(type_name : string) ~(id : string)
    : row_record option =
  match Io_lwt.Db.current_conn store with
  | None -> None
  | Some conn ->
      let user = session_user () in
      let sql =
        "SELECT id::text, fields::text, author, " ^
        "created_at::text, updated_at::text " ^
        "FROM " ^ data_rel () ^ " WHERE type_name = $1 AND id = $2::uuid
    AND " ^
        visible_sql ~user_param:3
      in
      let rows =
        Io_lwt.Db.execute conn
          ~params:[ Pgx.Value.of_string type_name
                  ; Pgx.Value.of_string id
                  ; Pgx.Value.of_string user ]
          sql
      in
      (match rows with
       | [ id_c; fields_c; author_c; created_c; updated_c ] :: _ ->
           (match Pgx.Value.to_string id_c, Pgx.Value.to_string fields_c
    with
            | Some id, Some fields_json ->
                (Some {
                     id;
                     fields_json;
                     author = Pgx.Value.to_string author_c;
                     created_at = Pgx.Value.to_string created_c;
                     updated_at = Pgx.Value.to_string updated_c;
                   })
            | _ -> None)
       | _ -> None)


(** (node-path 'type id) -> VStr path. Pure path constructor. *)
let node_path ~(type_name : string) ~(id : string) : value =
  let user = session_user () in
  VStr (Tree_store.datum_path ~user ~type_name ~id ())

(** (type-fields 'type) -> VList of VStr field names | VNil if undeclared.
    *)
let type_fields (store : Store.t) ~(type_name : string) : value =
```

```
  let fs = Store.lookup_type_fields store ~name:type_name in
  match fs with
  | None -> VNil
  | Some fields -> (VList (List.map (fun f -> VStr f) fields))

let tree_prim_error name msg =
  raise
    (Pricklypear_lisp.Ast.Interp_error
       (Pricklypear_lisp.Ast.Prim_error (name, msg, "bad-args")))

let wrap_path_exn name (f : unit -> value) : value =
  try f () with Invalid_argument msg -> tree_prim_error name msg


(* ---- hierarchical tree prims (notes.tree-cutover K0) ---- *)

let tree_get (store : Store.t) ~(path : string) ?(at_gen : int64 option) ()
    : value =
  match Io_lwt.Db.current_conn store with
  | None -> VNil
  | Some conn ->
      let n =
        match at_gen with
        | None -> Tree_store.get_by_path conn ~path
        | Some g ->
            (* reads-as-of-gen: optional :gen N resolves to the VERSIONED
    leaf
               at that gen (tree-time.reads-as-of-gen -- gets a fallback
               coordinate, never a required one). See tree_gen_read for the
               body-only surface; here we keep tree/get's node JSON shape.
    *)
            (match Tree_store.versioned_head conn ~path ~at_gen:g () with
             | None -> None
             | Some v -> Tree_store.version_of_leaf v)
      in
      (match n with
       | None -> VNil
       | Some node -> (VStr (Tree_store.node_to_json node)))

let tree_list (store : Store.t) ~(prefix : string) ?(depth : int option)
    ?(limit : int = 50000) () : value =
  match Io_lwt.Db.current_conn store with
  | None -> (VList [])
  | Some conn ->
      let nodes =
        match depth with
        | Some 1 -> Tree_store.list_children conn ~prefix ~limit ()
        | _ -> Tree_store.list_subtree conn ~prefix ~limit ()
      in
      (VList (List.map (fun n -> VStr (Tree_store.node_to_json n)) nodes))

(* (tree/children-of path [limit]) -> VList of direct-child node JSON.
   Level-predicate read (nlevel(path)+1, + outline covering at +2 under
   literal label h), never an anti-join -- one-tree.projections
   CHILDREN-BY-LEVEL. Same surface as tree/list depth 1; explicit name. *)
let tree_children_of (store : Store.t) ~(path : string)
    ?(limit : int = 50000) () : value =
  match Io_lwt.Db.current_conn store with
  | None -> (VList [])
  | Some conn ->
      let nodes = Tree_store.list_children conn ~prefix:path ~limit () in
      (VList (List.map (fun n -> VStr (Tree_store.node_to_json n)) nodes))

(* (tree/subtree path [limit]) -> VList of all-descendant node JSON under
   path, capped at limit. Backed by list_subtree; same surface as
   (tree/list path) with no depth. Explicit bounded-snapshot name. *)
let tree_subtree (store : Store.t) ~(path : string)
    ?(limit : int = 50000) () : value =
  match Io_lwt.Db.current_conn store with
  | None -> (VList [])
  | Some conn ->
      let nodes = Tree_store.list_subtree conn ~prefix:path ~limit () in
      (VList (List.map (fun n -> VStr (Tree_store.node_to_json n)) nodes))
```

```
717
718 (* ---- lens-aware read prims (one-tree.tree-read-prims, BORGE-1785516993)
        ----
719    Resolve a RELATIVE logical path (e.g. "fn.notes/doc/get",
720    "pkg.todo", "route./issues") against the session's branch lens chain
721    (self -> parent -> ... -> system/main, closest-wins), the same rule
722    function_defs use. A session on <user>/personal sees the EFFECTIVE
723    override chain for tree_nodes, not just its own branch. *)
724
725 let lens_chain (store : Store.t) : string list =
726   Store.branch_chain store ~branch_id:(Store.current_branch_id store)
727
728 let tree_lens_get (store : Store.t) ~(suffix : string) : value =
729   match Io_lwt.Db.current_conn store with
730   | None -> VNil
731   | Some conn ->
732       let n = Tree_store.lens_get_in_chain conn ~suffix (lens_chain store)
          in
733       (match n with
734        | None -> VNil
735        | Some node -> (VStr (Tree_store.node_to_json node)))
736
737 let tree_lens_children (store : Store.t) ~(suffix : string)
738     ?(limit : int = 50000) () : value =
739   match Io_lwt.Db.current_conn store with
740   | None -> (VList [])
741   | Some conn ->
742       let nodes =
743         Tree_store.lens_children_in_chain conn ~suffix ~limit
744           (lens_chain store)
745       in
746       (VList (List.map (fun n -> VStr (Tree_store.node_to_json n)) nodes))
747
748 let tree_lens_subtree (store : Store.t) ~(suffix : string)
749     ?(limit : int = 50000) () : value =
750   match Io_lwt.Db.current_conn store with
751   | None -> (VList [])
752   | Some conn ->
753       let nodes =
754         Tree_store.lens_subtree_in_chain conn ~suffix ~limit
755           (lens_chain store)
756       in
757       (VList (List.map (fun n -> VStr (Tree_store.node_to_json n)) nodes))
758
759 (* ---- local-first prims (borge/local-first.borg lens-model +
760    replication-units, campaign 8.1). A lens is a NAMED CHILD BRANCH of the
761    checked-out branch (fork-on-clone, one-tree P5 EMPTY-LENS fork --
762    copy-on-fork, parent pointer only); the server branch stays the parent.
763    These prims make the two pinned contracts concrete: lens/fork names +
764    creates the fork per the <owner>/lens-<device>[-<date>] contract, and
765    replication/unit materializes the exact authorized-read subtree the
766    identity would replicate (data.u.<user>.* + data.shared.* + the forked
767    code lens + the branch ancestry it resolves through). *)
768
769 (* Current branch (owner, name) resolved from the branches registry. *)
770 let current_branch_ref (store : Store.t) : (string * string) option =
771   let bid = Store.current_branch_id store in
772   List.find_opt
773     (fun (b : Pricklypear_common.Branch.t) -> b.id = bid)
774     (Store.list_branches store)
775   |> Option.map (fun (b : Pricklypear_common.Branch.t) -> (b.owner, b.name))
776
777 (* (lens/fork device-name) -> VStr "ok owner/lens-<device>[-<date>] (id)".
778    lens-model: create a lens as a NAMED CHILD of the checked-out branch, per
779    the pinned contract. parent_id = the current branch (server stays the
780    parent); owner = the parent owner; child_name = lens-<device>[-<date>].
781    Single-writer-per-lens is preserved by construction (server never writes
782    into a lens except via sync/merge). *)
783 let lens_fork (store : Store.t) ~(device : string) ~(date : string option)
       () : value =
784   match current_branch_ref store with
785   | None -> VStr "lens/fork: no current branch"
786   | Some (owner, _name) ->
```

```
      let suffix =
        match date with
        | Some d -> device ^ "-" ^ d
        | None -> device
      in
      let child_name = "lens-" ^ suffix in
      let parent_id = Store.current_branch_id store in
      (match Store.fork_branch store ~parent_id ~owner ~child_name with
       | Some id -> VStr (Printf.sprintf "ok %s/%s (%s)" owner child_name id)
       | None -> VStr "lens/fork: fork failed")

(* (replication/unit) -> VStr json-manifest. replication-units: materialize the exact authorized-read subtree the current identity would replicate -- data.u.<user>.* + data.shared.* (the data, READ-mostly seed for shared), + the forked code lens + the branch ancestry it resolves through. Path-prefix RLS confinement (one-tree RLS floor) is the replication boundary; this prim never enumerates anything outside the identity-readable set. Ancestry uses Store.branch_chain (self-first, closest-wins). *)
let replication_unit (store : Store.t) () : value =
  let user = session_user () in
  (* data.u.<user>.* paths store the user label-ENCODED (tree_store datum_path), so the replication prefix and the count must use the encoded form -- a raw username prefix would match nothing and under-count the unit. *)
  let user_enc = Tree_store.enc_label user in
  let user_prefix = "data.u." ^ user_enc in
  let bid = Store.current_branch_id store in
  let chain = Store.branch_chain store ~branch_id:bid in
  let count prefix =
    match Io_lwt.Db.current_conn store with
    | None -> 0
    | Some conn ->
        List.length (Tree_store.list_subtree conn ~prefix ~limit:100000 ())
  in
  let owner, bname =
    match current_branch_ref store with
    | Some (o, n) -> (o, n)
    | None -> ("", "")
  in
  let branch_ref =
    if owner = "" then bid else owner ^ "/" ^ bname
  in
  let manifest =
    `Assoc
      [ ("owner", `String user)
      ; ("branch", `String branch_ref)
      ; ("branch_id", `String bid)
      ; ( "lens"
        , `String (owner ^ "/lens-<device>[-<date>] (forked as empty child)")
        )
      ; ("ancestry", `List (List.map (fun id -> `String id) chain))
      ; ( "data"
        , `Assoc
            [ ("user", `String user_prefix)
            ; ("user_count", `Int (count user_prefix))
            ; ("shared", `String "data.shared")
            ; ("shared_count", `Int (count "data.shared"))
            ]
        )
      ; ( "scope"
        , `String
            "data.u.<user>.* + data.shared.* + b.<lens>.* + branch ancestry; path-prefix RLS confinement = replication boundary"
        )
      ]
  in
  VStr (Yojson.Safe.to_string manifest)
```

```
852
853 (* ---- reads-as-of-gen prim surface (tree-time.reads-as-of-gen) ----
854    The versioned read foundation (versioned_head / versioned_history / the
855    `version` row type / gen-now) landed in the 5.1 close-out. These crud
856    functions expose the user-facing gen coordinate: gen-read (node body as
857    of a gen, crossing the branch lens rules head reads cross), gen-diff
858    (node-granular diff between two gens -- hashes first, walk on mismatch,
859    the same algorithm family the three-way merge + local-first sync use),
860    and undo (restore-by-write: read at head-n, write a NEW versioned row at
861    head+1, so the restore itself is versioned and redoable). *)
862
863 (* Is [path] an absolute tree path (a b-prefixed code node or a
       data-prefixed
864    datum node), or a RELATIVE logical suffix (kind.name) that must be
       resolved
865    across the branch lens chain? *)
866 let absolute_tree_path (path : string) : bool =
867   String.starts_with ~prefix:"b." path || String.starts_with
       ~prefix:"data." path
868
869 let tree_gen_read (store : Store.t) ~(path : string) ~(gen : int64) : value
       =
870   match Io_lwt.Db.current_conn store with
871   | None -> VNil
872   | Some conn ->
873       let v =
874         if absolute_tree_path path then
875           if gen <= 0L then
876             (* head *)
877             Tree_store.versioned_head conn ~path ()
878           else
879             Tree_store.versioned_head conn ~path ~at_gen:gen ()
880         else
881           (* relative logical suffix -> resolve effective path across the
882              branch lens chain (closest-wins), then read as-of-gen. *)
883           let at_gen_opt = if gen <= 0L then None else Some gen in
884           let mk_path bid =
885             match Tree_store.lens_suffix_path ~branch_id:bid path with
886             | Some p -> p
887             | None -> ""
888           in
889           Tree_store.lens_versioned_in_chain conn ~at_gen:at_gen_opt
       ~mk_path
890             (lens_chain store)
891       in
892       (match v with
893        | None -> VNil
894        | Some v -> (VStr (Tree_store.version_to_json v)))
895
896 let body_of_version (v : Tree_store.version) : (string * Yojson.Safe.t)
       list =
897   match v.Tree_store.v_body with
898   | `Assoc kv ->
899       (match List.assoc_opt "body" kv with
900        | Some (`Assoc b) -> b
901        | _ -> [])
902   | _ -> []
903
904 (* (gen-diff path gen-a gen-b) -> node-granular diff. Hashes first in the
905    store (versioned_head is one index range scan per gen); when both leaves
906    hash-identical -> "same". Otherwise walk a shallow key-level difference
       of
907    the two bodies and return a JSON object {path, a, b, same, delta}. *)
908 let tree_gen_diff (store : Store.t) ~(path : string)
909     ~(gen_a : int64) ~(gen_b : int64) : value =
910   match Io_lwt.Db.current_conn store with
911   | None -> VNil
912   | Some conn ->
913       let a = Tree_store.versioned_head conn ~path ~at_gen:gen_a () in
914       let b = Tree_store.versioned_head conn ~path ~at_gen:gen_b () in
915       (match a, b with
916        | Some va, Some vb
917          when va.Tree_store.v_hash = vb.Tree_store.v_hash
```

```
              && va.Tree_store.v_rank = vb.Tree_store.v_rank ->
           (* sibling-order: a rank-only change (reorder) is a real
              difference even when the body hash is unchanged *)
           (VStr "same")
       | Some va, Some vb ->
           let ba = body_of_version va in
           let bb = body_of_version vb in
           let keys =
             List.sort_uniq String.compare (List.map fst ba @ List.map fst
     bb)
           in
           let delta_pairs =
             List.filter_map
               (fun k ->
                 let xa = List.assoc_opt k ba in
                 let xb = List.assoc_opt k bb in
                 if xa = xb then None
                 else
                   Some
                     (k,
                      `Assoc
                        [ ("a",
                           (match xa with Some x -> x | None -> `Null))
                        ; ("b",
                           (match xb with Some x -> x | None -> `Null))
                        ]))
               keys
           in
           let delta_pairs =
             if va.Tree_store.v_rank <> vb.Tree_store.v_rank then
               delta_pairs
               @ [ ( "rank"
                   , `Assoc
                       [ ( "a"
                         , (match va.Tree_store.v_rank with
                            | Some r -> `String r
                            | None -> `Null) )
                       ; ( "b"
                         , (match vb.Tree_store.v_rank with
                            | Some r -> `String r
                            | None -> `Null) )
                       ] ) ]
             else delta_pairs
           in
           let g_of v =
             match v.Tree_store.v_gen with
             | Some g -> `String (Int64.to_string g)
             | None -> `Null
           in
           (VStr
              (Yojson.Safe.to_string
                 (`Assoc
                    [ ("path", `String path)
                    ; ("a", g_of va)
                    ; ("b", g_of vb)
                    ; ("same", `Bool false)
                    ; ("delta", `Assoc delta_pairs)
                    ])))
       | _ -> VNil)

(* (undo path [n]) -> restore the value at head-n as a NEW write at head+1.
   Restore-by-write (not head-pointer manipulation): the restore is itself a
   versioned append, so undo is lossless and redoable ((undo path) again on
   the restored state climbs back down the gen chain). Supports the datum
   path class (the first documented consumer is notes undo over datum
   nodes). For code paths restore-by-write is deferred to the code-restore
   consumer (consumers subsection) -- this prim reports that cleanly.
     Returns
   the written path or an error string. *)
let tree_undo (store : Store.t) ~(path : string) ?(n : int64 = 1L) () :
     value =
  match Io_lwt.Db.current_conn store with
  | None -> (VStr "undo: no DB")
```

```
| Some conn ->
    if not (absolute_tree_path path) then
      (VStr "undo: path must be an absolute b.* or data.* path")
    else
      let hist = Tree_store.versioned_history conn ~path () in
      if hist = [] then (VStr "undo: no history for path")
      else
        let newest_gen =
          List.fold_left
            (fun acc v ->
              match v.Tree_store.v_gen with
              | Some g -> Int64.max acc g
              | None -> acc)
            0L hist
        in
        let target = Int64.sub newest_gen n in
        (match
           Tree_store.versioned_head conn ~path ~at_gen:target ()
         with
         | None ->
             (VStr
                (Printf.sprintf "undo: no value at gen %Ld" target))
         | Some v ->
             let body_json =
               match body_of_version v with
               | [] -> "{}"
               | kv -> Yojson.Safe.to_string (`Assoc kv)
             in
             let type_name =
               match v.Tree_store.v_type_name with
               | Some t -> t
               | None -> "" (* datum rows carry type_name in the version row *)
             in
             let author = v.Tree_store.v_author in
             let author =
               match author with Some a -> a | None -> session_user ()
             in
             if String.starts_with ~prefix:"b." path then
               (VStr
                  "undo: code-path restore-by-write deferred (code-restore \
                   consumer); use gen-read to inspect, then define")
             else
               (* tree-time.consumers: kind-aware restore. Datum rows
                  (kind=datum) re-write via upsert_datum; hierarchical
                  nodes (note/folder/heading) must go back through
                  upsert_hier so the payload envelope keeps its note kind
                  (an upsert_datum restore silently rewrites the payload
                  marker to "datum" -- the kind-column/payload mismatch
                  the notes-undo consumer exposes). The version row
                  captures the ORIGINAL kind, so the restore defeats to it;
                  unknown/empty kind falls back to datum behaviour. *)
               let hier_kind =
                 match v.Tree_store.v_kind with
                 | k when k <> "" && Tree_store.is_hier_kind k -> Some k
                 | _ -> None
               in
               (match hier_kind with
                | Some k ->
                    (match
                       Tree_store.upsert_hier conn ~path ~kind:k ~type_name
                         ~body_json ~author ~rank:None
                     with
                     | Ok p ->
                         (* sibling-order: the restore is EXACT -- the
                            rank in force at the target gen (NULL clears
                            a later reorder), not a COALESCE-keep *)
                         let _ =
                           Tree_store.set_rank conn ~path:p ~rank:v.Tree_store.v_rank
```

```
                          in
                          (VStr p)
                      | Error e -> (VStr e))
                | None ->
                    (match
                       Tree_store.upsert_datum conn ~path ~type_name
                         ~body_json ~author ~rank:None
                     with
                     | Ok p ->
                         let _ =
                           Tree_store.set_rank conn ~path:p
    ~rank:v.Tree_store.v_rank
                         in
                         (VStr p)
                     | Error e -> (VStr e))))

let tree_children_html (store : Store.t) ~(prefix : string) ~(garden_root :
    string)
    () : value =
  match Io_lwt.Db.current_conn store with
  | None -> (VStr "<p class=\"muted\">(empty)</p>")
  | Some conn ->
      let html =
        Tree_store.render_children_html conn ~prefix ~garden_root ()
      in
      (VStr html)

let tree_upsert (store : Store.t) ~(path : string) ~(kind : string)
    ~(type_name : string) ~(body_json : string) ~(author : string) : value =
  if kind = "datum" then
    (* Datum node: run the same typed-boundary validation as create/update,
       then write via Tree_store.upsert_datum (path-explicit; generates an
       id when the path's id label is a placeholder).
    one-tree.tree-datum-prims. *)
    let reject = check_type_fields store ~type_name ~fields_json:body_json
    in
    (match reject with
     | Some msg -> (VStr msg)
     | None ->
     match Io_lwt.Db.current_conn store with
     | None -> (VStr "tree/upsert: no DB")
     | Some conn ->
       let r =
         Tree_store.upsert_datum conn ~path ~type_name ~body_json ~author
           ~rank:None
       in
       (match r with Ok p -> (VStr p) | Error e -> (VStr e)))
  else
    match Io_lwt.Db.current_conn store with
    | None -> (VStr "tree/upsert: no DB")
    | Some conn ->
      let r =
        Tree_store.upsert_hier conn ~path ~kind ~type_name ~body_json
    ~author
          ~rank:None
      in
      (match r with
       | Ok p -> (VStr p)
       | Error e -> (VStr e))

let tree_delete (store : Store.t) ~(path : string) ~(recursive : bool)
    : value =
  match Io_lwt.Db.current_conn store with
  | None -> (VStr "{\"deleted\":0,\"inbound_edges\":0}")
  | Some conn ->
      (* tree-edges: report incoming-edge count (warn, not forbid -- the
         delete succeeds; edges pointing at the deleted path dangle and
         are repairable). Exact-path count only (recursive deletes do
         not aggregate descendant inbound counts in v1). *)
      let inbound = Tree_store.incoming_edge_count conn ~path in
      let n = Tree_store.delete_path conn ~path ~recursive in
      (VStr
         (Yojson.Safe.to_string
```

```
            (`Assoc
               [ ("deleted", `Int n)
               ; ("inbound_edges", `Int inbound)
               ])))

(* ---- datum type-change validation for tree/move
     (one-tree.tree-datum-prims) ----
   Moving a datum node to a path under a DIFFERENT <type_name> is a type
   change and must re-run typed-boundary validation against the new type
   (exactly the same rule create/update enforce). When both paths are datum
   paths and the type segment differs, read the node's body at [old_path]
   and validate the declared fields of the NEW type; a missing required
   field is a clean rejection naming the field, identical in spirit to
   create/update. Same-type moves, hierarchical moves, and any path that is
   not a datum path skip validation (no type semantics to check). *)

(** The type_name encoded in a datum path, decoded. Datum path grammar:
      data.u.<user>.<type>.<id>
    | data.shared.<type>.<id>
    | data.l.<lens>.<user>.<type>.<id>     (versioned data lens, 2026-08-23)
    The type segment is base64-label-encoded (Tree_store.enc_label).
    None when [path] is not a datum path. *)
let datum_path_type (path : string) : string option =
  let parts = String.split_on_char '.' path in
  let type_label =
    match parts with
    | "data" :: "u" :: _user :: ty :: _id :: _ -> Some ty
    | "data" :: "shared" :: ty :: _id :: _ -> Some ty
    | "data" :: "l" :: _lens :: _user :: ty :: _id :: _ -> Some ty
    | _ -> None
  in
  match type_label with
  | None -> None
  | Some ty -> Tree_store.dec_label ty

(** The lens name encoded in a data.l path datum path, decoded.
    None for non-lens paths (data.u / data.shared). *)
let datum_path_lens (path : string) : string option =
  let parts = String.split_on_char '.' path in
  match parts with
  | "data" :: "l" :: lens :: _user :: _ty :: _id :: _ ->
     Tree_store.dec_label lens
  | _ -> None

(** If [old_path] -> [new_path] is a datum move that changes type, validate
    the moved node's body against the NEW type's declared fields. Returns
    Some rejection-message (naming the missing fields) when it fails; None
    when there is nothing to check (not a datum move, same type, undeclared
    new type, node missing, or free-form body). *)
let datum_move_type_reject (store : Store.t) ~(old_path : string)
    ~(new_path : string) : string option =
  match datum_path_type old_path, datum_path_type new_path with
  | Some old_ty, Some new_ty when old_ty <> new_ty ->
      (match Io_lwt.Db.current_conn store with
       | None -> None
       | Some conn ->
           (match Tree_store.get_by_path conn ~path:old_path with
            | None -> None
            | Some node ->
                let body_json =
                  match node.body with
                  | `Assoc _ -> Yojson.Safe.to_string node.body
                  | _ -> "{}"
                in
                let declared =
                  match Store.lookup_type_fields store ~name:new_ty with
                  | Some f -> f
                  | None -> []
                in
                (match
                   Tree_store.validate_fields ~declared body_json
                 with
                 | Ok () -> None
```

```
                | Error msg ->
                    Some
                      (Printf.sprintf "type change %s -> %s: %s" old_ty
                         new_ty msg))))
  | _ -> None

let tree_move (store : Store.t) ~(old_path : string) ~(new_path : string)
    : value =
  match datum_move_type_reject store ~old_path ~new_path with
  | Some msg -> (VStr ("tree/move: " ^ msg))
  | None ->
  match Io_lwt.Db.current_conn store with
  | None -> (VStr "tree/move: no DB")
  | Some conn ->
      (* tree-edges: moving the TARGET of inbound edges leaves them
         dangling (to is stored verbatim, never rewritten) -- report the
         count, warn-not-forbid. Moving a SOURCE carries its edges (the
         subtree move rewrites their paths). *)
      let inbound = Tree_store.incoming_edge_count conn ~path:old_path in
      let r = Tree_store.move_subtree conn ~old_path ~new_path in
      (match r with
       | Ok n ->
           (VStr
              (Yojson.Safe.to_string
                 (`Assoc
                    [ ("moved", `Int n)
                    ; ("inbound_edges", `Int inbound)
                    ])))
       | Error e -> (VStr e))

(* ---- tree-edges prims (one-tree.tree-edges) ---- *)

(* (link from to [kind [label]]) -- create an edge node (kind=link) at
   <from>.lnk.<uuid>. from must resolve (get_by_path); to is stored
   verbatim as branch-qualified path text and MAY dangle; resolution is
   a read-time act against the reader's lens. Edge kind defaults to
   "ref". Returns the edge node JSON. *)
let link_edge (store : Store.t) ~(from_path : string) ~(to_path : string)
    ~(edge_kind : string) ~(label : string option) ~(author : string) :
     value =
  if not (Tree_store.valid_path to_path) then
    VStr "link: to must be a well-formed absolute path (it may dangle)"
  else
    match Io_lwt.Db.current_conn store with
    | None -> VStr "link: no DB"
    | Some conn -> (
        match Tree_store.get_by_path conn ~path:from_path with
        | None -> VStr ("link: from does not resolve: " ^ from_path)
        | Some _ ->
            let body =
              let base =
                [ ("to", `String to_path); ("kind", `String edge_kind) ]
              in
              let base =
                match label with
                | Some l when l <> "" -> base @ [ ("label", `String l) ]
                | _ -> base
              in
              Yojson.Safe.to_string (`Assoc base)
            in
            (* edge path = <from>.lnk.<uuid> -- the uuid comes from PG *)
            let sql_uuid =
              "WITH nu AS (SELECT gen_random_uuid() AS id) "
              ^ "SELECT tree_uuid_label(nu.id) FROM nu"
            in
            (match Io_lwt.Db.execute conn sql_uuid with
             | [ [ u ] ] -> (
                 match Pgx.Value.to_string u with
                 | Some uuid ->
                     let edge_path =
                       Tree_store.path_child from_path "lnk" ^ "." ^ uuid
                     in
                     (match
```

```
                        Tree_store.upsert_link conn ~path:edge_path
                          ~body_json:body ~author
                      with
                      | Ok p -> (
                          match Tree_store.get_by_path conn ~path:p with
                          | Some n -> VStr (Tree_store.node_to_json n)
                          | None -> VStr p)
                      | Error e -> VStr ("link: " ^ e))
                | None -> VStr "link: uuid generation failed")
            | _ -> VStr "link: uuid generation failed"))

(* (unlink edge-path) -- tombstone the edge: the same delete/versioning
   path as any node (the tree-time trigger appends a tombstone; undo of
   unlink restores the edge). Only kind=link nodes may be unlinked. *)
let unlink_edge (store : Store.t) ~(edge_path : string) : value =
  match Io_lwt.Db.current_conn store with
  | None -> VStr "unlink: no DB"
  | Some conn -> (
      match Tree_store.get_by_path conn ~path:edge_path with
      | None -> VStr ("unlink: no node at " ^ edge_path)
      | Some n ->
          if not (Tree_store.is_link_kind n.kind) then
            VStr ("unlink: not an edge (kind " ^ n.kind ^ ")")
          else
            let deleted = Tree_store.delete_path conn ~path:edge_path
    ~recursive:false in
            if deleted = 1 then VStr edge_path
            else VStr "unlink: delete failed")

(* (out-links path) -- the node's outgoing edges with resolution status:
   each item is JSON {path,to,kind,label,status,target}; status is
   "resolved" (target summary from the reader's lens) or "dangling".
   An edge list is a children-by-level index read, never a scan. *)
let out_links (store : Store.t) ~(path : string) ?(limit : int = 1000) () :
    value =
  match Io_lwt.Db.current_conn store with
  | None -> VList []
  | Some conn ->
      let edges = Tree_store.list_out_links conn ~path ~limit () in
      let item (e : Tree_store.node) =
        let to_path =
          match Tree_store.body_string_field e.body "to" with
          | Some t -> t
          | None -> ""
        in
        let status, target =
          if to_path = "" then ("dangling", `Null)
          else
            match Tree_store.get_by_path conn ~path:to_path with
            | Some t ->
                ( "resolved"
                , `Assoc
                    [ ("path", `String t.path)
                    ; ("kind", `String t.kind)
                    ; ( "type"
                      , match t.type_name with
                        | Some ty -> `String ty
                        | None -> `Null )
                    ; ("title", `String (Tree_store.node_display_title t))
                    ] )
            | None -> ("dangling", `Null)
        in
        VStr
          (Yojson.Safe.to_string
             (`Assoc
                [ ("path", `String e.path)
                ; ("to", `String to_path)
                ; ( "kind"
                  , match Tree_store.body_string_field e.body "kind" with
                    | Some k -> `String k
                    | None -> `Null )
                ; ( "label"
                  , match Tree_store.body_string_field e.body "label" with
```

```
                    | Some l -> `String l
                    | None -> `Null )
                ; ("status", `String status)
                ; ("target", target)
                ]))
      in
      VList (List.map item edges)

(* Backlinks upgrade (one-tree.tree-edges): edges whose to = needle --
   backed by the migration-026 partial expression index -- UNION the
   legacy body-string scan (datum bodies holding the needle as a literal
   value). The legacy half is DEPRECATED: logged once per process, kept
   only until all body-valued references become edges. Same signature as
   the old backlinks: a list of paths. *)
let backlinks_deprecation_logged = ref false

let backlinks (store : Store.t) ~(needle : string) ?(limit : int = 200) ()
    : value =
  match Io_lwt.Db.current_conn store with
  | None -> VList []
  | Some conn ->
      let edge_paths =
        Tree_store.edges_to conn ~needle ~limit ()
        |> List.map (fun (e : Tree_store.node) -> e.path)
      in
      let legacy_paths =
        if List.length edge_paths >= limit then []
        else begin
          if not !backlinks_deprecation_logged then begin
            backlinks_deprecation_logged := true;
            Printf.eprintf
              "[backlinks] DEPRECATED: legacy body-string backlinks are a
    scan; materialize references as edges ((link from to)).\n%!"
          end;
          Tree_store.backlinks conn ~needle ~limit ()
        end
      in
      (* edges first, then legacy paths not already present (dedup) *)
      let seen = List.fold_left (fun acc p -> p :: acc) [] edge_paths in
      let merged =
        edge_paths
        @ List.filter (fun p -> not (List.mem p seen)) legacy_paths
      in
      VList (List.map (fun p -> VStr p) merged)

(* ---- sibling-order prims (one-tree.sibling-order) ---- *)

   Rank allocation is midpoint-between-neighbours over canonical base-36
   ranks (Tree_store.rank helpers); when a generated rank would exceed
   max_rank_len the whole sibling set is rebalanced onto fresh fixed-
   width ranks and the placement recomputed (always fits after a
   rebalance). Mutations deny_anon at the prim layer. Ranks live in the
   rank column (migration 025), are versioned by the tree-time trigger,
   and reads order rank COLLATE "C" NULLS LAST, path ASC. *)

(** Candidate rank for placing a node immediately before (before=true)
   or after sibling [anchor] under [parent]. Rebalances the sibling set
   (once) when the midpoint would exceed max_rank_len, then recomputes. *)
let place_candidate (conn : Store_db.conn) ~(parent : string)
    ~(anchor : string) ~(before : bool) : (string, string) result =
  let rec attempt retried =
    let children = Tree_store.list_children conn ~prefix:parent () in
    (match
       List.find_opt (fun (n : Tree_store.node) -> n.path = anchor) children
     with
     | None -> Error ("anchor not found under " ^ parent)
     | Some a -> (
         match a.rank with
         | None -> Error "anchor has no rank (rank it first: tree/reorder)"
         | Some ar ->
             let ranked =
               List.filter_map
                 (fun (n : Tree_store.node) ->
```

```
                  match n.rank with Some r -> Some (n.path, r) | None -> None)
                children
            in
            (* [ranked] is in rank order (SQL: ranked first, rank ASC).
               The neighbour is the ranked sibling immediately before /
               after the anchor. *)
            let neighbor : (string * string) option =
              if before then
                List.fold_left
                  (fun acc (p, r) ->
                    if String.compare r ar < 0 then Some (p, r) else acc)
                  None ranked
              else
                List.find_map
                  (fun (p, r) ->
                    if String.compare r ar > 0 then Some (p, r) else None)
                  ranked
            in
            let candidate =
              match neighbor with
              | Some (_p, r) ->
                  if before then Tree_store.rank_midpoint r ar
                  else Tree_store.rank_midpoint ar r
              | None -> (
                  if before then
                    match Tree_store.rank_before ar with
                    | Some r -> r
                    | None -> "" (* floor exhaustion -> rebalance *)
                  else Tree_store.rank_after ar)
            in
            if
              candidate = ""
              || String.length candidate > Tree_store.max_rank_len
            then
              if retried then Error "rank rebalance failed"
              else begin
                let _ = Tree_store.rebalance_siblings conn ~parent in
                attempt true
              end
            else Ok candidate))
  in
  attempt false

let tree_move_before (store : Store.t) ~(path : string) ~(anchor : string)
    : value =
  wrap_path_exn "tree/move-before" (fun () ->
      match Tree_store.path_parent path, Tree_store.path_parent anchor with
      | Some parent, Some ap when parent = ap && path <> anchor -> (
          match Io_lwt.Db.current_conn store with
          | None -> VStr "tree/move-before: no DB"
          | Some conn -> (
              match place_candidate conn ~parent ~anchor ~before:true with
              | Error e -> VStr ("tree/move-before: " ^ e)
              | Ok rank ->
                  if Tree_store.set_rank conn ~path ~rank:(Some rank) = 1
    then
                    VStr rank
                  else VStr ("tree/move-before: no node at " ^ path)))
      | Some _, Some _ when path = anchor ->
          VStr (Tree_store.path_leaf path) (* no-op: already there *)
      | _ ->
          VStr "tree/move-before: path and anchor must be siblings (same
    parent)")

let tree_move_after (store : Store.t) ~(path : string) ~(anchor : string)
    : value =
  wrap_path_exn "tree/move-after" (fun () ->
      match Tree_store.path_parent path, Tree_store.path_parent anchor with
      | Some parent, Some ap when parent = ap && path <> anchor -> (
          match Io_lwt.Db.current_conn store with
          | None -> VStr "tree/move-after: no DB"
          | Some conn -> (
```

```
              match place_candidate conn ~parent ~anchor ~before:false with
              | Error e -> VStr ("tree/move-after: " ^ e)
              | Ok rank ->
                  if Tree_store.set_rank conn ~path ~rank:(Some rank) = 1
    then
                    VStr rank
                  else VStr ("tree/move-after: no node at " ^ path)))
      | Some _, Some _ when path = anchor ->
          VStr (Tree_store.path_leaf path) (* no-op: already there *)
      | _ ->
          VStr "tree/move-after: path and anchor must be siblings (same
    parent)")

(* (tree/reorder path rank) -- set the rank directly. "" clears (back to
   unordered). Rank must be base-36 ([0-9a-z]+) so midpoint allocation
   stays well-defined. Returns node JSON. *)
let tree_reorder (store : Store.t) ~(path : string) ~(rank : string) :
    value =
  match Io_lwt.Db.current_conn store with
  | None -> VStr "tree/reorder: no DB"
  | Some conn ->
      if rank <> "" && not (Tree_store.rank_valid rank) then
        VStr "tree/reorder: rank must be base-36 ([0-9a-z]+)"
      else if
        Tree_store.set_rank conn ~path
          ~rank:(if rank = "" then None else Some rank)
        = 0
      then VStr ("tree/reorder: no node at " ^ path)
      else
        (match Tree_store.get_by_path conn ~path with
         | Some n -> VStr (Tree_store.node_to_json n)
         | None -> VStr "ok")

(* (tree/siblings path) -- ordered children of path's parent (the node
   itself included). Root paths have no siblings. *)
let tree_siblings (store : Store.t) ~(path : string) : value =
  wrap_path_exn "tree/siblings" (fun () ->
      match Tree_store.path_parent path with
      | None -> VList [] (* root: no parent, no siblings *)
      | Some parent -> (
          match Io_lwt.Db.current_conn store with
          | None -> VList []
          | Some conn ->
              let nodes = Tree_store.list_children conn ~prefix:parent () in
              VList
                (List.map (fun n -> VStr (Tree_store.node_to_json n))
    nodes)))

(* (tree/next path) / (tree/prev path) -- the adjacent sibling in the
   total order (ranked first by rank, then unranked by path). Nil at
   either end, or when the path is not a child of anything. *)
let tree_neighbor (store : Store.t) ~(path : string) ~(dir : int) : value =
  wrap_path_exn "tree/neighbor" (fun () ->
      match Tree_store.path_parent path with
      | None -> VNil
      | Some parent -> (
          match Io_lwt.Db.current_conn store with
          | None -> VNil
          | Some conn ->
              let nodes = Tree_store.list_children conn ~prefix:parent () in
              let idx =
                List.find_index (fun (n : Tree_store.node) -> n.path =
    path) nodes
              in
              (match idx with
               | None -> VNil (* not a child of its parent: nothing
    adjacent *)
               | Some i ->
                   let j = i + dir in
                   if j >= 0 && j < List.length nodes then
                     (match List.nth_opt nodes j with
                      | Some n -> VStr (Tree_store.node_to_json n)
                      | None -> VNil)
```

```
                  else VNil)))

let tree_next (store : Store.t) ~(path : string) : value =
  tree_neighbor store ~path ~dir:1

let tree_prev (store : Store.t) ~(path : string) : value =
  tree_neighbor store ~path ~dir:(-1)

(* (tree/insert-child parent kind type body [:after anchor]) -- create a
   child of [parent] with a rank allocated after [anchor] (or appended
   after the last ranked sibling). Hier kinds only: a kind=datum node
   must keep its data.<type>.<id> path grammar, so datum ordering stays
   on tree/upsert + tree/reorder (borge issue filed). The leaf label
   comes from the body's slug (or title), path_label-encoded. *)
let tree_insert_child (store : Store.t) ~(parent : string) ~(kind : string)
    ~(type_name : string) ~(body_json : string) ~(anchor : string option)
    ~(author : string) : value =
  if not (Tree_store.valid_path parent) then
    VStr "tree/insert-child: invalid parent path"
  else if not (Tree_store.is_hier_kind kind) then
    VStr
      "tree/insert-child: kind must be folder|note|heading (datum ordering:
     \
       tree/upsert + tree/reorder)"
  else
    match Io_lwt.Db.current_conn store with
    | None -> VStr "tree/insert-child: no DB"
    | Some conn -> (
        let label =
          match Tree_store.parse_body_json body_json with
          | Ok body -> (
              match Tree_store.body_string_field body "slug" with
              | Some s -> Some (Tree_store.path_label s)
              | None -> (
                  match Tree_store.body_string_field body "title" with
                  | Some t when String.trim t <> "" ->
                      Some (Tree_store.path_label t)
                  | _ -> None))
          | Error _ -> None
        in
        match label with
        | None -> VStr "tree/insert-child: body needs a slug or title"
        | Some lab ->
            let path = Tree_store.path_child parent lab in
            let rank_res =
              match anchor with
              | Some a -> (
                  match Tree_store.path_parent a with
                  | Some ap when ap = parent ->
                      place_candidate conn ~parent ~anchor:a ~before:false
                  | _ -> Error "anchor must be a ranked child of parent")
              | None ->
                  let last_ranked =
                    List.fold_left
                      (fun acc (n : Tree_store.node) ->
                        match n.rank with Some r -> Some r | None -> acc)
                      None
                      (Tree_store.list_children conn ~prefix:parent ())
                  in
                  Ok
                    (Tree_store.rank_after
                       (match last_ranked with Some r -> r | None -> ""))
            in
            (match rank_res with
             | Error e -> VStr ("tree/insert-child: " ^ e)
             | Ok rank -> (
                 match
                   Tree_store.upsert_hier conn ~path ~kind ~type_name
                     ~body_json ~author ~rank:(Some rank)
                 with
                 | Ok p -> (
                     match Tree_store.get_by_path conn ~path:p with
                     | Some n -> VStr (Tree_store.node_to_json n)
```

```
                    | None -> VStr p)
                | Error e -> VStr ("tree/insert-child: " ^ e))))

let tree_get_uuid (store : Store.t) ~(uuid : string) : value =
  match Io_lwt.Db.current_conn store with
  | None -> VNil
  | Some conn ->
      let n = Tree_store.get_by_uuid conn ~uuid in
      (match n with
       | None -> VNil
       | Some node -> (VStr (Tree_store.node_to_json node)))

let tree_find_by (store : Store.t) ~(kind : string option) ~(field : string)
    ~(value : string) ?(limit : int = 50) () : value =
  match Io_lwt.Db.current_conn store with
  | None -> (VList [])
  | Some conn ->
      let nodes =
        Tree_store.find_by_body_field conn ~kind ~field ~value ~limit ()
      in
      (VList (List.map (fun n -> VStr (Tree_store.node_to_json n)) nodes))

let tree_list_kind (store : Store.t) ~(prefix : string) ~(kind : string)
    ?(limit : int = 2000) () : value =
  match Io_lwt.Db.current_conn store with
  | None -> (VList [])
  | Some conn ->
      let nodes =
        Tree_store.list_by_kind conn ~prefix ~kind ~limit ()
      in
      (VList (List.map (fun n -> VStr (Tree_store.node_to_json n)) nodes))

let tree_note_index (store : Store.t) ~(prefix : string) ?(limit : int =
    2000)
    () : value =
  match Io_lwt.Db.current_conn store with
  | None -> (VList [])
  | Some conn ->
      let rows = Tree_store.list_note_index conn ~prefix ~limit () in
      (VList
           (List.map
              (fun (id, fields) -> VPair (VStr id, VStr fields))
              rows))

let tree_folder_index (store : Store.t) ~(prefix : string) () : value =
  match Io_lwt.Db.current_conn store with
  | None -> (VList [])
  | Some conn ->
      let folders = Tree_store.list_folder_index conn ~prefix () in
      (VList (List.map (fun f -> VStr f) folders))

(** Substring search over tree notes+headings. Returns list of hit JSON. *)
let tree_search_text (store : Store.t) ~(prefix : string) ~(needle : string)
    ?(limit : int = 20) () : value =
  match Io_lwt.Db.current_conn store with
  | None -> (VList [])
  | Some conn ->
      let hits =
        Tree_store.search_text conn ~prefix ~needle ~limit ()
      in
      (VList (List.map (fun j -> VStr j) hits))

let tree_path_label (s : string) : value = VStr (Tree_store.path_label s)

(* ---- pure path-algebra prims (one-tree.tree-path-prims) ----
   No DB, no auth. The kernel raises Invalid_argument on an invalid
   path; converted here to a Prim_error so Nopales sees a clean raise
   rather than a guess. Root paths (single label) have nil parent and
   no strict ancestors. *)
let tree_parent (path : string) : value =
  wrap_path_exn "tree/parent" (fun () ->
      match Tree_store.path_parent path with
      | Some p -> VStr p
```

```
      | None -> VNil (* root path has no parent *))

let tree_leaf (path : string) : value =
  wrap_path_exn "tree/leaf" (fun () -> VStr (Tree_store.path_leaf path))

let tree_segments (path : string) : value =
  wrap_path_exn "tree/segments" (fun () ->
      VList (List.map (fun s -> VStr s) (Tree_store.path_segments path)))

let tree_child ~(path : string) ~(label : string) : value =
  wrap_path_exn "tree/child" (fun () ->
      VStr (Tree_store.path_child path label))

let tree_ancestors (path : string) : value =
  wrap_path_exn "tree/ancestors" (fun () ->
      VList
        (List.map (fun s -> VStr s) (Tree_store.path_ancestors path)))

let tree_garden_root ~(user : string) : value =
  VStr (Tree_store.garden_root ~user)

(* ---- tree-time.gc (5.3): retention worker surface
     ------------------------
   retention-policy-set / retention-policies / retention-delete-policy
     manage
   policy AS HABITAT DATA (tree_retention_policy, migration 022);
     retention-run
   drives the worker (rollup + DROP PARTITION) for each path class that has
     an
   explicit whole-class policy. Retention is a TODO until a class outgrows
     its
   welcome -- the default (no whole-class policy) is keep-everything, never
   ad-hoc deletion. *)

let retention_set_policy (store : Store.t) ~(path_class : string)
    ~(type_name : string option) ~(keep_windows : int64) () : value =
  VStr (Store.retention_set_policy store ~path_class ~type_name
     ~keep_windows)

let retention_delete_policy (store : Store.t) ~(path_class : string)
    ~(type_name : string option) () : value =
  VStr
    (Store.retention_delete_policy store ~path_class ~type_name)

let retention_policies (store : Store.t) () : value =
  let ps = Store.retention_list_policies store in
  let js =
    List.map
      (fun (pc, tn, kw, ca) ->
        Yojson.Safe.to_string
          (`Assoc
            [ ("path_class", `String pc);
              ("type_name",
                match tn with None -> `Null | Some s -> `String s);
              ("keep_windows", `Intlit (Int64.to_string kw));
              ("created_at", `String ca) ]))
      ps
  in
  VList (List.map (fun j -> VStr j) js)

let retention_run (store : Store.t) ~(path_class : string option) () :
    value =
  let rs = Store.retention_run store ~path_class () in
  let js =
    List.map
      (fun (pc, d, c) ->
        Yojson.Safe.to_string
          (`Assoc
            [ ("path_class", `String pc);
              ("dropped_windows", `Intlit (Int64.to_string d));
              ("checkpointed_paths", `Intlit (Int64.to_string c)) ]))
      rs
  in
```

```
  VList (List.map (fun j -> VStr j) js)


(* ---- data lens operations (one-tree.borg branching-interaction,
   2026-08-23 make: versioned user-keyed data). A data lens is a
   deliberate fork of the user's live data.u.<user> subtree into
   data.l.<lens>.<user>. The lens can diverge (edits land in the lens,
   not live data), merge back (lens -> trunk, LWW per node v0), or be
   abandoned (drop the lens subtree). The default data.u stays
   single-head and branch-free; data.l is the explicit opt-in version
   axis. *)

(* (fork-data lens-name [type-name]) -> VStr.
   Copy the user's live datum nodes (data.u.<user>) into a fresh data
   lens (data.l.<lens>.<user>). When ~type_name is set, only that type
   is forked; otherwise all the user's non-seed types. Each source node's
   path is rewritten: data.u.<user>.<type>.<id> ->
   data.l.<lens>.<user>.<type>.<id>. The node payload (body, hash, author)
   is copied; the path field in the payload is updated. Returns
   "ok <lens> (<n> nodes)" or an error. *)
let fork_data (store : Store.t) ~(lens : string)
    ?(type_name : string option = None) () : value =
  let user = session_user () in
  let user_enc = Tree_store.enc_label user in
  let lens_enc = Tree_store.enc_label lens in
  let live_prefix = Printf.sprintf "data.u.%s" user_enc in
  let lens_prefix =
    Printf.sprintf "data.l.%s.%s" lens_enc user_enc
  in
  (match Io_lwt.Db.current_conn store with
   | None -> VStr "fork-data: no database connection"
   | Some conn ->
       (* Check the lens doesn't already exist (avoid silent double-fork).
     *)
       let existing =
         Tree_store.list_subtree conn ~prefix:lens_prefix ~limit:1 ()
       in
       if existing <> [] then
         VStr (Printf.sprintf "fork-data: lens %s already exists" lens)
       else
         (* Copy datum nodes from live -> lens by repathing.
            The SQL constructs the new path by replacing the data.u.<user>
            prefix with data.l.<lens>.<user>, preserving the type + id
            segments. Payload path is updated to match. *)
         let type_clause =
           match type_name with
           | None -> ""
           | Some tn ->
               Printf.sprintf " AND type_name = $3::text"
         in
         let params =
           match type_name with
           | None ->
               [ Pgx.Value.of_string live_prefix
               ; Pgx.Value.of_string lens_prefix ]
           | Some tn ->
               [ Pgx.Value.of_string live_prefix
               ; Pgx.Value.of_string lens_prefix
               ; Pgx.Value.of_string tn ]
         in
          let sql =
            "INSERT INTO tree_nodes (path, kind, type_name, payload, hash,
    author, updated_at) "
           ^ "SELECT "
           ^ "  ($2::text || subltree(path, 3, nlevel(path)))::ltree, "
           ^ "  'datum', type_name, "
           ^ "  jsonb_set(payload, '{path}', "
           ^ "    to_jsonb(($2::text || subltree(path, 3,
    nlevel(path)))::text)), "
           ^ "  hash, author, now() "
           ^ "FROM tree_nodes "
             ^ "WHERE kind = 'datum' AND path <@ $1::ltree"
             ^ type_clause
```

```
              ^ " ON CONFLICT (path) DO NOTHING RETURNING path::text"
         in
         let rows =
           Io_lwt.Db.execute conn ~params sql
         in
         let n = List.length rows in
         VStr (Printf.sprintf "ok %s (%d nodes forked)" lens n))

(* (abandon-data lens-name) -> VStr.
   Drop the data lens subtree entirely. The user's live data.u is
   untouched. Returns "ok <lens> (<n> nodes dropped)". *)
let abandon_data_lens (store : Store.t) ~(lens : string) () : value =
  let user = session_user () in
  let user_enc = Tree_store.enc_label user in
  let lens_enc = Tree_store.enc_label lens in
  let lens_prefix =
    Printf.sprintf "data.l.%s.%s" lens_enc user_enc
  in
  (match Io_lwt.Db.current_conn store with
   | None -> VStr "abandon-data: no database connection"
   | Some conn ->
       let n = Tree_store.delete_path conn ~path:lens_prefix
     ~recursive:true in
       VStr (Printf.sprintf "ok %s (%d nodes abandoned)" lens n))

(* (merge-data lens-name) -> VStr.
   Merge a data lens back into the user's live data.u.<user>. v0 policy:
   LWW per node -- the lens version overwrites the live version for every
   node in the lens. This is the simple, honest v0: no three-way diff,
   no conflict surfacing (the user deliberately forked, edited, and chose
   to merge; the lens is the intended new state). Conflict surfacing and
   three-way merge are the v1 ambition (one-tree.borg
     branching-interaction).

   For each lens node (data.l.<lens>.<user>.<type>.<id>), upsert the body
   into the live path (data.u.<user>.<type>.<id>), then CONSUME the lens:
   drop the data.l.<lens>.<user> subtree so the lens is gone after a
   successful merge (v0 lifecycle: fork -> diverge -> merge | abandon).
   The drop is explicit here, not left to the reverse mirror trigger:
   migration 023's tree_mirror_data_rows purge now skips data.l.* (so a
   lens survives ordinary live writes as a restore point), so merge owns
   the consume. Returns a JSON report {lens, merged, skipped, dropped}. *)
let merge_data_lens (store : Store.t) ~(lens : string) () : value =
  let user = session_user () in
  let user_enc = Tree_store.enc_label user in
  let lens_enc = Tree_store.enc_label lens in
  let lens_prefix =
    Printf.sprintf "data.l.%s.%s" lens_enc user_enc
  in
  (match Io_lwt.Db.current_conn store with
   | None -> VStr "merge-data: no database connection"
   | Some conn ->
       (* For each lens datum, upsert into the corresponding live path.
          The live path is constructed by replacing the data.l.<lens> prefix
          with data.u -- i.e. dropping the lens segment. *)
       let lens_nodes =
         Tree_store.list_subtree conn ~prefix:lens_prefix ~limit:100000 ()
       in
       let merged = ref 0 in
       let skipped = ref 0 in
       List.iter
         (fun (n : Tree_store.node) ->
           (* Reconstruct the live path: data.l.<lens>.<user>.<type>.<id>
              -> data.u.<user>.<type>.<id>. Drop the first two segments
              (data.l) and the lens segment, prepend data.u.<user>. *)
           let live_path =
             let parts = String.split_on_char '.' n.path in
             match parts with
             | "data" :: "l" :: _lens :: rest ->
                 "data.u." ^ String.concat "." rest
             | _ -> n.path
           in
           let body_json =
```

```
            match n.body with
            | `Assoc _ -> Yojson.Safe.to_string n.body
            | _ -> "{}"
          in
          let result =
            Tree_store.upsert_datum conn
              ~path:live_path
              ~type_name:
                (match n.type_name with
                 | Some tn -> tn
                 | None -> "")
              ~body_json ~author:user ~rank:None
          in
          (match result with
           | Ok _ -> incr merged
           | Error _ -> incr skipped))
        lens_nodes;
      (* Consume the lens: drop the now-integrated data.l.<lens>.<user>
         subtree. *)
      let dropped =
        Tree_store.delete_path conn ~path:lens_prefix ~recursive:true
      in
      let report =
        `Assoc
          [ ("lens", `String lens)
          ; ("merged", `Int !merged)
          ; ("skipped", `Int !skipped)
          ; ("dropped", `Int dropped)
          ]
      in
      VStr (Yojson.Safe.to_string report))

(* (list-data-lenses) -> VList of VStr lens names.
   Enumerate the user's data lenses by scanning for distinct
     data.l.<lens>.<user>
   prefixes in tree_nodes. *)
let list_data_lenses (store : Store.t) () : value =
  let user = session_user () in
  let user_enc = Tree_store.enc_label user in
  (match Io_lwt.Db.current_conn store with
   | None -> VList []
   | Some conn ->
       let sql =
         "SELECT DISTINCT tree_dec_label(subltree(path, 2, 3)::text) "
         ^ "FROM tree_nodes "
         ^ "WHERE kind = 'datum' AND path <@ 'data.l'::ltree "
         ^ "AND subltree(path, 3, 4) = '" ^ user_enc ^ "'::ltree "
         ^ "ORDER BY 1"
       in
      let rows = Io_lwt.Db.execute conn ~params:[] sql in
      let lenses =
        List.filter_map
          (fun row ->
            match row with
            | [ v ] ->
                (match Pgx.Value.to_string v with
                 | Some s -> Some (VStr s)
                 | None -> None)
            | _ -> None)
          rows
      in
      VList lenses)
```

67 image/lib/db_cost.ml

```
(** Static DB-prim cost tiers for binding-page annotations. *)

type cost_tier =
  | Constant
  | Index_seek
  | Linear
  | Seq_scan
  | Unknown

let badge = function
  | Constant -> "O(1) write"
  | Index_seek -> "index seek"
  | Linear -> "filtered scan"
  | Seq_scan -> "seq scan"
  | Unknown -> "?"

let static_cost (prim_name : string) : cost_tier =
  match prim_name with
  | "get" | "find-row" | "select-for-update" | "node-path" | "type-fields"
  | "tree/get" | "tree/get-uuid" -> Index_seek
  | "create" | "update" | "update-field" | "delete" | "tree/upsert"
  | "tree/delete" | "tree/move" -> Constant
  | "rows" | "find-random" | "tree/list" -> Seq_scan
  | "find-rows" | "find-text" | "backlinks" | "tree/find-by" -> Linear
  | _ -> Unknown

let sql_hint = function
  | "get" -> Some "SELECT fields FROM data_rows WHERE type_name=$1 AND
     id=$2"
  | "create" -> Some "INSERT INTO data_rows (type_name, id, fields, author)
     ?"
  | "update" | "update-field" ->
      Some "UPDATE data_rows SET fields=? WHERE type_name=$1 AND id=$2"
  | "delete" -> Some "DELETE FROM data_rows WHERE type_name=$1 AND id=$2"
  | "rows" -> Some "SELECT id, fields FROM data_rows WHERE type_name=$1
     LIMIT ?"
  | "find-row" -> Some "SELECT ? FROM data_rows WHERE type_name AND field
     filter"
  | "find-rows" | "find-text" ->
      Some "SELECT ? FROM data_rows WHERE type_name AND text/json filter"
  | _ -> None

module Ast = Pricklypear_lisp.Ast

let rec collect_db_prims (acc : string list) (e : Ast.expr) : string list =
  match e with
  | Ast.Prim (n, args) ->
      let acc =
        match static_cost n with Unknown -> acc | _ -> n :: acc
      in
      List.fold_left collect_db_prims acc args
  | Ast.App (Ast.Var n, args) ->
      let acc =
        match static_cost n with Unknown -> acc | _ -> n :: acc
      in
      List.fold_left collect_db_prims acc args
  | Ast.App (f, args) ->
      List.fold_left collect_db_prims (collect_db_prims acc f) args
  | Ast.Let (_, a, b) | Ast.Seq (a, b) ->
      collect_db_prims (collect_db_prims acc a) b
  | Ast.Letrec (bs, body) ->
      let acc =
        List.fold_left (fun a (_, e) -> collect_db_prims a e) acc bs
      in
      collect_db_prims acc body
  | Ast.If (a, b, c) ->
      collect_db_prims (collect_db_prims (collect_db_prims acc a) b) c
  | Ast.Lambda (_, body) | Ast.Define (_, body) -> collect_db_prims acc body
  | Ast.Var _ | Ast.Lit _ | Ast.Quote _ -> acc

let uniq xs =
```

```
  let seen = Hashtbl.create 8 in
  List.fold_left
    (fun acc x ->
      if Hashtbl.mem seen x then acc
      else begin
        Hashtbl.add seen x ();
        x :: acc
      end)
    [] xs
  |> List.rev

let db_prims_of_expr (e : Ast.expr) : string list =
  uniq (collect_db_prims [] e)
```

68 image/lib/example_eval.ml

```
(** Pricklypear Image -- restricted read-time eval for book example
    blocks (habitat-book.eval-examples).

    `(example-eval src)` parses + evals a Nopales source string so a
    READER can run a book's example blocks in their own session:

    - RESTRICTED PRIM TABLE. The example runs under its own Control
      rim whose table is a NAME FILTER of the caller's table: whitelist
      = Effect.pure_prims + Effect.read_db_prims + a reviewed
      read-only supplement ([read_extra_prims]). Everything else is
      ABSENT, and Control.perform_prim raises "unregistered primitive"
      on absent names -- fail-closed, including through function bodies
      and macro expansions (they eval under the same nested rim).
    - STATIC GATE. Before eval, each parsed form must (a) contain no
      mutation-family special form ([forbidden_special]) and (b) infer
      an effect set within {pure, read-db, html} via Effect.infer with
      known_fn from the branch's visible defs. The runtime table is
      the hard boundary; the static gate gives precise refusals and
      catches capability-carrying calls the name filter would only
      stop deeper in.
    - READER'S BRANCH ENV. Eval runs in a CHILD of the session env:
      real bindings resolve through the parent chain (thunks force
      through their own captured closures), and nothing the example
      does can leak into the session env.
    - CAPS. Fresh step budget per call (Eval.make_budget, default
      limit) -- the lisp/ recursion/loop guardrail; plus a wall-clock
      cap checked between top-level forms. Blocking prims (sleep,
      http-get, llm/agent families) are not whitelisted, so nothing
      inside the example can park the fiber past the step budget. A
      runaway example's budget exhaustion converts to [Refused], NOT
      a bare Budget_exceeded: the only budget in play here is the
      nested one (the caller's is restored in [finally]), and guard --
      the only catch Nopales has -- deliberately re-raises
      Budget_exceeded, so (book/verify) must see a refusal to render
      the runaway as a hole instead of dying with the report.

    The caller's current_budget and the ambient rim are saved/restored
    around the nested run (Control.run_with_rim nests by design; same
    pattern as Session.bench). *)

open Pricklypear_lisp.Ast

exception Refused of string
(** Static gate refusal. The reason names the offending form or
    capability; callers render it as a drift-hole-style error box. *)

(** Default step budget for one example eval (matches the per-page
    default; examples are snippets, not pages). *)
let default_step_limit = 100_000

(** Wall-clock cap in seconds, checked between top-level forms (a form
    cannot be interrupted mid-flight; the step budget bounds those). *)
let default_time_cap = 10.0

(** Effects an example block may use. [Html] is allowed because
    render-html is a pure value tag (wraps a VStr), not I/O. *)
let allowed_effect = function
  | Pricklypear_lisp.Effect.Pure
  | Pricklypear_lisp.Effect.ReadDb
  | Pricklypear_lisp.Effect.Html -> true
  | _ -> false

(** Mutation-family special forms refused anywhere in example source.
    define/defmacro mutate envs (and define persists in ordinary
    eval); with-transaction/with-step are write paths; prim is the
    internal escape hatch. Quoted data is inert and never reaches the
    scanner's refusal (a quoted (define ?) is just data). *)
let forbidden_special = [ "define"; "defmacro"; "with-transaction";
     "with-step"; "prim" ]

(** Blocked despite a pure classification: [sleep] sits in
```

```
    Effect.pure_prims but parks the fiber for its duration -- exactly
    what a reader example must not be able to do (the step budget only
    counts steps, not wall time). Blocked BEFORE the allow lists so the
    classification cannot smuggle it in. Non-blocking clock reads
    (now/now-ms) stay allowed. Revisit only alongside an effect.ml
    reclassification (that would ripple to INSTALL/ROUTE gates --
    out of scope here). *)
let blocked_prims = [ "sleep" ]

(** Whitelist supplement: kernel prims that are read-only or pure but
    not yet classified in lisp/lib/effect.ml's capability lists (those
    lists predate several kernel prims and only need updating when an
    INSTALL/ROUTE gate must see the capability -- the example whitelist
    must be exact, so it carries its own supplement). Each entry is
    reviewed read-only/pure; anything absent stays absent (fail-closed). *)
let read_extra_prims =
  [ (* pure string/token kernels (P2.2 budget prims + helpers) *)
    "string-split"; "prose-tokens"; "render-tokens"; "err/shape"
  ; "extract-links"; "render-links"; "budget-remaining"
  ; "status-response"; "part"; "json-part"; "multipart"
    (* one-tree READ surface + pure path helpers *)
  ; "tree/get"; "tree/list"; "tree/list-kind"; "tree/children-of"
  ; "tree/subtree"; "tree/search-text"; "tree/note-index"
  ; "tree/lens-get"; "tree/lens-children"; "tree/lens-subtree"
  ; "tree/path-label"; "tree/parent"; "tree/leaf"; "tree/segments"
  ; "tree/child"; "tree/ancestors"; "tree/garden-root"
  ; "tree/find-by"; "tree/get-uuid"; "tree/folder-index"
  ; "tree/children-html"; "tree/siblings"; "tree/next"; "tree/prev"
  ; "backlinks"; "out-links"; "node-path"; "type-fields"
    (* reference/package reflection (read-only) *)
  ; "list-packages"; "describe"; "apropos-docs"
  ; "callees"; "callers"; "dependents"; "used-by-routes"; "holes"
  ; "find-code"; "code-history"; "code-diff"; "diff"
  ; "list-data-lenses"
    (* media metadata reads *)
  ; "media-list"; "media-meta"; "media-url" ]

let whitelisted (name : string) : bool =
  not (List.mem name blocked_prims)
  && (List.mem name Pricklypear_lisp.Effect.pure_prims
      || List.mem name Pricklypear_lisp.Effect.read_db_prims
      || List.mem name read_extra_prims)

(** Build the restricted table: a NEW table holding only whitelisted
    entries of [full] (same closures -- read DB prims keep the caller's
    session store/branch). Unknown names are simply absent. *)
let restricted_prims (full : Primitives.t)
    : Primitives.t =
  let t = Hashtbl.create 64 in
  Hashtbl.iter
    (fun name fn -> if whitelisted name then Hashtbl.replace t name fn)
    full;
  t

(** Reader sugar 'x parses as App (Var "quote", [e]); eval dispatches
    "quote" as a SPECIAL FORM (datum returned as-is -- eval.ml), but
    Effect.infer only knows the Quote node and otherwise treats the
    shape as a call application whose args it unions -- so a quoted
    datum containing e.g. (create ?) would infer WriteDb and get
    refused as a capability the example never actually uses. Rewrite
    the sugar to the Quote node before gating so quoted data stays
    inert to BOTH the special-form scan and the capability inference.
    Quasiquote is deliberately NOT desugared: its unquoted parts DO
    execute, so infer unioning their effects is correct there. *)
let rec desugar_quote (e : expr) : expr =
  match e with
  | App (Var "quote", [ x ]) -> Quote (desugar_quote x)
  | App (f, args) -> App (desugar_quote f, List.map desugar_quote args)
  | Prim (n, args) -> Prim (n, List.map desugar_quote args)
  | Lambda (ps, body) -> Lambda (ps, desugar_quote body)
  | Define (n, body) -> Define (n, desugar_quote body)
  | Let (n, a, b) -> Let (n, desugar_quote a, desugar_quote b)
  | Letrec (bs, body) ->
```

```
      Letrec (List.map (fun (n, b) -> (n, desugar_quote b)) bs,
     desugar_quote body)
  | Seq (a, b) -> Seq (desugar_quote a, desugar_quote b)
  | If (a, b, c) -> If (desugar_quote a, desugar_quote b, desugar_quote c)
  | e -> e

(** First forbidden special form in [e], or None. Does NOT descend into
    quoted data -- the reader sugar for 'x is App (Var "quote", [e]) and
    eval returns the datum as-is (desugar_quote rewrites it to the
    Quote node first) -- so a quoted (define ?) is inert. Soundness:
    anything quoted data can still reach execution through
    (macro expansion building code, quasiquote unquotes, eval-string)
    evals under the RESTRICTED rim, which fails closed on absent
    prims. *)
let rec find_forbidden (e : expr) : string option =
  match e with
  | Var _ | Lit _ | Quote _ -> None
  | App (Var "quote", [ _ ]) -> None
  | Prim (n, args) ->
      if List.mem n forbidden_special then Some n else scan_args args
  | App (Var name, args) ->
      if List.mem name forbidden_special then Some name else scan_args args
  | App (f, args) -> (
      match find_forbidden f with
      | Some x -> Some x
      | None -> scan_args args)
  | Lambda (_, body) -> find_forbidden body
  | Define _ -> Some "define"
      (* AST-level define (internal shape); surface text is App-form *)
  | Let (_, a, b) | Seq (a, b) -> (
      match find_forbidden a with
      | Some x -> Some x
      | None -> find_forbidden b)
  | Letrec (bs, body) -> (
      match List.find_map (fun (_, e') -> find_forbidden e') bs with
      | Some x -> Some x
      | None -> find_forbidden body)
  | If (a, b, c) -> (
      match find_forbidden a with
      | Some x -> Some x
      | None -> (
          match find_forbidden b with
          | Some x -> Some x
          | None -> find_forbidden c))

and scan_args (args : expr list) : string option =
  List.find_map find_forbidden args

(** Static gate over one parsed form: [Some reason] refuses. *)
let check_form ~(known_fn : Pricklypear_lisp.Effect.known_fn_lookup)
    (raw : expr) : string option =
  let e = desugar_quote raw in
  match find_forbidden e with
  | Some f ->
      Some
        (Printf.sprintf "special form '%s' is not allowed in examples" f)
  | None ->
      let fx = Pricklypear_lisp.Effect.infer ~known_fn e in
      match
        List.find_opt
          (fun eff -> not (allowed_effect eff))
          (Pricklypear_lisp.Effect.EffSet.to_list fx)
      with
      | Some eff ->
          Some
            (Printf.sprintf
               "example needs capability '%s' (allowed: pure, read-db,
    html)"
               (Pricklypear_lisp.Effect.eff_to_string eff))
      | None -> None

(** Parse + gate + eval [src] (all top-level forms, last value
    returned) in a child of [env] under a fresh budget and a restricted
```

```
    rim over [full_prims]. Raises [Refused] on gate refusals and on
    the example's own step-budget exhaustion; parse errors and eval
    errors propagate as-is. *)
let eval_source ~(env : Pricklypear_lisp.Ast.env)
    ~(full_prims : Primitives.t)
    ~(known_fn : Pricklypear_lisp.Effect.known_fn_lookup)
    ?(limit = default_step_limit) ?(time_cap = default_time_cap)
    (src : string) : Pricklypear_lisp.Ast.value =
  let forms = Pricklypear_lisp.Reader.read_all src in
  if forms = [] then
    raise (Refused "empty source");
  List.iter
    (fun e ->
      match check_form ~known_fn e with
      | Some reason -> raise (Refused reason)
      | None -> ())
    forms;
  let rprims = restricted_prims full_prims in
  let cenv = Pricklypear_lisp.Env.child env in
  let budget = Pricklypear_lisp.Eval.make_budget ~limit () in
  let prev_budget = !Pricklypear_lisp.Eval.current_budget in
  let t0 = Unix.gettimeofday () in
  try
    Fun.protect
      ~finally:(fun () ->
        Pricklypear_lisp.Eval.current_budget := prev_budget)
      (fun () ->
        Control_rim.run ~prims:rprims (fun () ->
            let rec loop last = function
              | [] -> last
              | e :: rest ->
                  if Unix.gettimeofday () -. t0 > time_cap then
                    raise
                      (Interp_error
                         (User_error
                            ("example-eval: time cap exceeded",
    "timeout")));
                  let v =
                    Pricklypear_lisp.Eval.eval ~budget
                      ~prims:Pricklypear_lisp.Eval.empty_prims cenv e
                  in
                  loop v rest
            in
            loop VNil forms))
  with
    (* Inside this window the only live budget is the nested one -- the
       caller's is stashed in [prev_budget] until [finally]. Convert so
       both consumers (POST run button, (example-eval) prim ?
       book/verify's guard) render a runaway example as a refusal. *)
    Interp_error Budget_exceeded ->
      raise (Refused (Printf.sprintf "example budget exceeded (%d steps)"
    limit))
```

69 image/lib/function_stats.ml

```
(** Pricklypear Image -- function_stat write path
    (image.function-stat-ring).

    Every labeled eval used to call Crud.create synchronously -- a
    SERIALIZABLE insert + fsync on the request path, and unbounded row
    growth (54k rows in weeks on prod). Same medicine as
    http-robustness's ops-incidents rule: request path pushes to an
    in-process ring (drop-oldest at capacity), a background fiber
    flushes at most once every [flush_secs]. Crash loss is fine --
    these are perf stats, not ledger. *)

type entry = {
  fn : string;
  fields_json : string;
  author : string;
}

let capacity = 512
let flush_secs = 30.0

let buf : entry Queue.t = Queue.create ()
let mu : Eio.Mutex.t = Eio.Mutex.create ()

(** Request-path push: mutex + queue op only, zero DB ops. *)
let push ~(fn : string) ~(fields_json : string) ~(author : string) : unit =
  Eio.Mutex.use_rw ~protect:true mu (fun () ->
      if Queue.length buf >= capacity then ignore (Queue.pop buf);
      Queue.push { fn; fields_json; author } buf)

(** Drain the ring into data_rows. Best-effort: a row that fails to
    insert is logged and dropped (stats must never kill the flush loop). *)
let flush ~(store : Store.t) : unit =
  let pending =
    Eio.Mutex.use_rw ~protect:true mu (fun () ->
        let xs = Queue.fold (fun acc e -> e :: acc) [] buf |> List.rev in
        Queue.clear buf;
        xs)
  in
  match pending with
  | [] -> ()
  | xs ->
      List.iter
        (fun e ->
          (try
             ignore
               (Crud.create store ~type_name:"function_stat"
                  ~fields_json:e.fields_json ~author:e.author)
           with exn ->
             Printf.eprintf "[function_stats] flush row failed: %s\n%!"
               (Printexc.to_string exn)))
        xs

(** Background flush loop; fork once at boot. Uses the Eio clock --
    NOT Lwt_eio.run_lwt: concurrent run_lwt across fibers (this loop vs
    the jobs runner) is unsafe and took the whole switch down on prod
    (watchdog restart loop, 2026-08-21). *)
let flush_loop ~(clock : _ Eio.Time.clock) ~(store : Store.t) : unit =
  let rec loop () =
    Eio.Time.sleep clock flush_secs;
    (try flush ~store
     with exn ->
       Printf.eprintf "[function_stats] flush failed: %s\n%!"
         (Printexc.to_string exn));
    loop ()
  in
  loop ()
```

70 image/lib/http.ml

```
(** Pricklypear Image -- HTTP server entry: route dispatch + serve.

    System pages + browser product surfaces:
      /  /health  /branches  /bindings  /types  /data ?
      /timing  /repl  /api/eval  /edit  /inspect  /agent
      public agent bootstrap: /.well-known/agent.json  /agent.txt
      user routes via (route ...)
*)
open Request_context
open Pure_html
open Http_core
open Http_data
open Http_branches
open Http_agent
open Http_pages
open Http_ops
open Http_media

let request_handler ~(sw : Eio.Switch.t) ~(registry : Session_registry.t)
    (session : Session.t) (sockaddr : Unix.sockaddr)
    (reqd : Httpun.Reqd.t) : unit =
  let req = Httpun.Reqd.request reqd in
  let target = req.Httpun.Request.target in
  let path =
    match String.index_opt target '?' with
    | Some i -> String.sub target 0 i
    | None -> target
  in
  let meth = req.Httpun.Request.meth in
  let headers = header_list req in
  (* i-1787971251: client source_ip for auth_log rows (login/token use). *)
  let source_ip = source_ip_of_sockaddr sockaddr in
  (* Fork page work off the connection fiber (gluten_eio runs request_handler
     on the same fiber as the read loop). Awaiting body reads without forking
     deadlocks: schedule_read callbacks never fire. Same role as the old
     Lwt.async launch under httpun-lwt.

     image.http-robustness invariant 5: bound the expensive page-work queue
     with the inflight cap so a traffic spike can't accumulate unbounded
     queued work (memory/fds). Acquired here (inside launch) so cheap probes
     like /health that bypass launch are NEVER blocked by page traffic --
     liveness must stay servable under load. Released in finalize so a slot
     is freed whether page_fn succeeded, failed, or the peer went away. *)
  (* page_fn receives the SELECTED session (image.per-identity-sessions):
     the identity's own Session.t, never the shared boot session. *)
  let launch (page_fn : Session.t -> unit) : unit =
    Eio.Fiber.fork ~sw (fun () ->
    let () = inflight_acquire () in
    Fun.protect ~finally:(fun () -> inflight_release ()) (fun () ->
      (try
      (* image.http-robustness invariant 4b: request_mutex guards
                 ONLY the identity-attach gate (resolve_identity + attach +
                 classify + subdomain mismatch + analytics + refresh_ui_shell).
                 It is released BEFORE page_fn runs so page work (Lisp eval, HTML
                 build, agent LLM streaming) is lock-free. Correctness rests on
                 invariant 4a (immutable-env rehydrate: in-flight closures hold
                 their own env record, unaffected by a concurrent rehydrate) +
                 pgx's per-connection Sequencer (DB prims serialize on the single
                 PG conn without this mutex). See borge/image.borg http-robustness. *)
              (* Per-request context snapshot (image.per-request-context):
                 built INSIDE the mutex so it is consistent with this request's
```

```
62                attach, then bound via Lwt.with_value so page work reads
      the
63                snapshot (env/routes/manifest) instead of live Session.t,
64                immune to a concurrent attach_anonymous + checkout_to
65                rehydrating the shared session mid-page. *)
66             let snapshot_ctx (session : Session.t) =
67               Request_context.make
68                 ~manifest:(Session.manifest session)
69                 ~env:(Session.env session)
70                 ~routes:(Request_context.copy_routes (Session.routes
      session))
71             in
72             (* SELECT the identity's session, then attach on IT.
73                image.per-identity-sessions: attach selects, never
74                mutates another identity's session; anonymous traffic
75                runs on the parked anonymous session. *)
76             let attach_and_select () =
77               let user_opt =
78                 resolve_identity session ~headers ~source_ip ()
79               in
80               let session = Session_registry.select registry
      ~user:user_opt in
81               let () =
82                 match user_opt with
83                 | Some user ->
84                     let tok = cookie_value headers cookie_name in
85                     attach_from_user session user ~auth_token:tok
86                       ~checkout_personal_now:false ()
87                 | None -> Session.attach_anonymous session
88               in
89               Session.ensure_fresh session;
90               (session, user_opt)
91             in
92             let proceed_ctx =
93               Eio.Mutex.use_ro request_mutex (fun () ->
94                 if auth_disabled () then begin
95                   (* Dev escape: still try attach so whoami is real if
      cookie. *)
96                   let session, _ = attach_and_select () in
97                   (Some (session, snapshot_ctx session))
98                 end else begin
99                   (* Attach identity BEFORE classify: route table is
100                     branch-scoped (inheritance CTE). Anonymous uses
101                     system/main so package public routes resolve. *)
102                  let session, user_opt = attach_and_select () in
103                  let classif = classify_path session path in
104                  let needs_auth =
105                    match classif with
106                    | `Hard_public -> false
107                    | `Hard_private | `Unknown -> true
108                    | `User (_, vis) -> vis <> "public"
109                  in
110                  match user_opt, needs_auth with
111                  | Some _, _ -> (Some (session, snapshot_ctx session))
112                  | None, false -> (Some (session, snapshot_ctx session))
113                  | None, true ->
114                      let apiish =
115                        String.length path >= 4
116                        && String.sub path 0 4 = "/api"
117                      in
118                      if apiish then unauthorized_api reqd
119                      else require_login_redirect reqd ~path ~target;
120                      None
121                end)
122            in
123            (match proceed_ctx with
124             | None -> ()
125             | Some (session, req_ctx) ->
126             Request_context.with_current req_ctx @@ fun () ->
127             (* Per-request context bound; page work below reads it for
128                env/routes/manifest (call_handler, lookup_route, chrome).
      *)
129             (* Subdomain garden mismatch: private paths require
```

```
                whoami = garden owner. Reads the SNAPSHOT manifest so a
                concurrent attach cannot flip identity under this check. *)
             let host_mode = classify_host headers in
             let manifest_now = Request_context.manifest req_ctx in
             let uname_now = manifest_now.username in
             let classif_now =
               classify_path_with_routes (Request_context.routes req_ctx)
    path
             in
               let path_private =
                 match classif_now with
                 | `Hard_public -> false
                 | `User (_, vis) -> vis <> "public"
                 | `Hard_private | `Unknown -> true
               in
               let mismatch =
                 match host_mode with
                 | Host_garden g
                   when path_private && uname_now <> "anonymous"
                        && uname_now <> g -> true
                 | Host_unknown when base_domain () <> None -> true
                 | _ -> false
               in
               if mismatch then begin
                 (match host_mode, base_domain () with
                  | Host_garden _, Some base when uname_now <> "anonymous"
    ->
                      redirect_response reqd
                        ("http://" ^ uname_now ^ "." ^ base ^ "/")
                  | Host_unknown, _ ->
                      text_response reqd ~status:`Not_found
                        ~content_type:"text/plain" "unknown host"
                  | _ -> redirect_response reqd "/");
                 ()
               end else begin
               (* Product analytics (in-memory day counters). *)
               let uname = Session.username session in
               let is_authed =
                 uname <> "anonymous" && uname <> "public" && uname <> ""
               in
               Ops_stats.record_request ~path ~is_authed
                 ~username:(if is_authed then Some uname else None)
                 ~headers ~sockaddr;
               (* Refresh ui/nav cache every request so Lisp chrome is
    shared
                  by home, system pages, and user routes -- not only /.
                  image.boot-request-cost: /api/* never renders chrome, so
                  skip the ui/nav + ui/css eval there entirely (agents
                  hitting POST /api/eval never need the shell). *)
               let () =
                 if String.length path >= 4
                    && String.sub path 0 4 = "/api"
                 then ()
                 else Session.refresh_ui_shell session
               in
               (* GATE COMPLETE -- request_mutex released above. page_fn
    runs
                  lock-free (invariant 4b) on the SELECTED session. *)
               page_fn session
               end
             )
    with exn ->
      (* image.http-robustness invariant 2: if the handler failed
               because the peer went away (closed writer), do NOT try to
               render a 500 page onto the same dead writer -- that re-throw
               was the actual crash mechanism. Just close quietly and end.
    *)
           if is_closed_writer_exn exn then begin
             ignore (Ops_log.info ~component:"http"
                       ~context:[("path", path)]
                       "peer gone (closed writer); skipping 500");
             ()
           end else if
```

```
            (match exn with Http_core.Body_too_large -> true | _ -> false)
          then begin
            (* Oversized request body (PP_MAX_BODY_BYTES): a 413, not a
               500 -- the client, not the server, is at fault. *)
            text_response reqd ~status:`Payload_too_large
              ~content_type:"text/plain" "request body too large";
            ()
          end else begin
          (* Log the real exception to ops; never echo raw exception text
             back to the client (500 bodies are generic -- incident is
             correlated server-side via ops log). *)
          Ops_incidents.emit_and_maybe_file session
            ~level:Ops_log.Error ~component:"http"
            ~file:(not (Ops_incidents.is_expected_runtime_error exn))
            ~context:[("path", path)] (Printexc.to_string exn);
          let body =
            chrome ~session ~title:"Error" [
              HTML.p [HTML.class_ "err"]
                [txt "internal server error -- incident logged"];
            ]
          in
          html_response reqd ~session ~status:`Internal_server_error body;
          ()
          end)))
in
(** habitat-book.web-renderer: kernel-dispatched book handlers
    (book/page, book/chapter-page, book/code-page in libs/book/lib.pp).
    Same shape as the /book/<pkg> fallthrough: resolve the handler
    INSIDE page work (post-gate per-request context), zero-arg call,
    params ride the request record. No route pins -- the kernel owns
    the /book namespace dispatch (spec keeps its /book route pin for
    the unbound-book fallback below). *)
let launch_book_handler (handler : string) : unit =
  launch (fun session ->
      let request : Session.request =
        { method_ = `GET
        ; path
        ; params = parse_query target
        ; body = ""
        ; headers = header_list req
        }
      in
      match Session.call_handler session ~request handler with
      | Session.Handler_ok value ->
          respond_handler_value ~session ~reqd ~path ~fragment:false value
      | Session.Handler_unbound ->
          text_response reqd ~status:`Not_found ~content_type:"text/plain"
            (String.concat " "
               [ handler; "unbound -- (load-library \"book\")" ])
      | Session.Handler_error msg ->
          text_response reqd ~status:`Internal_server_error
            ~content_type:"text/plain" msg);
  ()
in
(* habitat-book.eval-examples: run-button target for nopales example
   blocks. htmx posts the tree-row path (hx-vals); we read the row,
   eval its src via the RESTRICTED example engine
   (Session.example_eval_engine -- whitelisted pure + read-only prims,
   static gate, child of the session env, fresh budget), and return a
   replacement .book-example div (hx-target closest, hx-swap
   outerHTML) showing value/error + elapsed. Kernel-dispatched like
   the /book GET surface -- no route pins in the /book namespace. *)
let post_book_eval_example (session : Session.t) : unit =
  let body = read_body reqd in
  let row_path =
    Option.value (form_field body "path") ~default:"" |> String.trim
  in
  let fail ?(status = `Bad_request) msg =
    text_response reqd ~status ~content_type:"text/plain" msg
  in
  let esc = Prims_pure.html_escape_str in
  if row_path = "" then fail "path required"
  else if not (String.starts_with ~prefix:"data.book." row_path) then
```

```
      fail "not a book row path"
    else
      match Crud.tree_get (Session.store session) ~path:row_path () with
      | Pricklypear_lisp.Ast.VNil ->
          fail ~status:`Not_found ("no such book row: " ^ row_path)
      | Pricklypear_lisp.Ast.VStr json -> (
          match Yojson.Safe.from_string json with
          | `Assoc fields ->
              (* tree/get rows wrap the payload under "body" (see
                 book/row-body in libs/book/lib.pp); fall back to the
                 top level for flat rows. *)
              let fields =
                match List.assoc_opt "body" fields with
                | Some (`Assoc body) -> body
                | _ -> fields
              in
              let f k =
                match List.assoc_opt k fields with
                | Some (`String s) -> s
                | _ -> ""
              in
              let lang = f "lang" and src = f "src" and expected = f "expected" in
              if lang <> "nopales" then
                fail "captured, not live (ocaml examples do not eval in v1)"
              else if src = "" then fail "example row has no src"
              else
                let t0 = Unix.gettimeofday () in
                let elapsed () =
                  Printf.sprintf "%.0f ms"
                    ((Unix.gettimeofday () -. t0) *. 1000.0)
                in
                let result_html =
                  match Session.example_eval_engine session src with
                  | v ->
                      Printf.sprintf
                        "<div class=\"book-result\">? <code>%s</code> <span class=\"book-elapsed\">(%s)</span></div>"
                        (esc (Pricklypear_lisp.Reader.pp_value v))
                        (elapsed ())
                  | exception Example_eval.Refused reason ->
                      Printf.sprintf
                        "<div class=\"book-error\">refused: %s <span class=\"book-elapsed\">(%s)</span></div>"
                        (esc reason) (elapsed ())
                  | exception exn ->
                      Printf.sprintf
                        "<div class=\"book-error\">error: %s <span class=\"book-elapsed\">(%s)</span></div>"
                        (esc (Printexc.to_string exn))
                        (elapsed ())
                in
                let expected_html =
                  if expected = "" then ""
                  else
                    Printf.sprintf
                      "<p class=\"book-expected\">expect =&gt; %s</p>"
                      (esc expected)
                in
                let html =
                  Printf.sprintf
                    "<div class=\"book-example\"><pre><code>%s</code></pre>%s%s</div>"
                    (esc src) result_html expected_html
                in
                text_response reqd ~content_type:"text/html" html
          | _ -> fail "bad book row JSON"
          | exception _ -> fail "bad book row JSON")
      | _ -> fail "bad book row"
  in
  (** Lisp user-route if bound; else kernel [fallback]. *)
  let launch_user_route_or (fallback : unit -> unit) : unit =
    (* Resolve the route INSIDE page work, after the gate has attached
```

```
      identity and bound the per-request context (image.per-request-context).
      Pre-gate the routes table could be whatever a previous request left
      behind (e.g. anonymous attach -> system/main), causing a personal-branch
      route 404 before THIS request's attach even runs. *)
    launch (fun session ->
      let route_opt =
        match Request_context.get () with
        | Some ctx -> Request_context.lookup_route ctx path
        | None -> Session.lookup_route session path
      in
      match route_opt with
      | None ->
          fallback ();
          ()
      | Some handler_name ->
          let page () : unit =
            let body_str =
              match meth with
              | `POST -> read_body reqd
              | _ -> ""
            in
            if String.length body_str > max_user_body_bytes then begin
              text_response reqd ~status:`Payload_too_large
                ~content_type:"text/plain"
                "request body too large (max 64KiB)";
              ()
            end else begin
              let query = parse_query target in
              let form =
                match meth with
                | `POST ->
                    let ct =
                      match Httpun.Headers.get req.headers "content-type" with
                      | Some s -> s
                      | None -> ""
                    in
                    if String.length ct >= 33
                       && String.sub ct 0 33
                          = "application/x-www-form-urlencoded"
                       || ct = "" && String.contains body_str '='
                    then parse_form_body body_str
                    else []
                | _ -> []
              in
              let params = merge_params ~query ~form in
              let fragment = wants_fragment ~path ~headers in
              let method_ : Session.http_method =
                match meth with `POST -> `POST | _ -> `GET
              in
              let request : Session.request =
                { method_; path; params; body = body_str; headers }
              in
              let result, _total_ms =
                Timing.measure ~route:handler_name
                  (fun () -> Session.call_handler session ~request handler_name)
              in
              (match result with
               | Session.Handler_ok value ->
                   (* Public front-door pages: no product chrome (private nav). *)
                   if path = "/welcome"
                      || path = "/background" || path = "/roadmap"
                      || path = "/signup"
                      || path = "/guestbook"
                   then begin
                     let title =
                       if path = "/background" then "Background"
                       else if path = "/roadmap" then "Roadmap"
                       else if path = "/signup" then "Signup"
                       else if path = "/guestbook" then "Guestbook"
```

```
                    else "Pricklypear"
                  in
                  let body =
                    match Primitives.is_raw_html value with
                    | Some raw -> raw
                    | None ->
                        (match value with
                         | Pricklypear_lisp.Ast.VStr s -> s
                         | _ -> Pricklypear_lisp.Reader.pp_value value)
                  in
                  Http_code.serve_public_body reqd ~title body;
                  ()
                end else begin
                  respond_handler_value ~session ~reqd ~path ~fragment value;
                  ()
                end
            | Session.Handler_unbound ->
                fallback ();
                ()
            | Session.Handler_error msg ->
                if fragment then
                  fragment_response reqd ~status:`Internal_server_error
                    ("handler " ^ handler_name ^ ": " ^ msg)
                else
                  html_response reqd ~session ~status:`Internal_server_error
                    (chrome ~session ~title:"Route handler error" [
                       HTML.h1 [] [txt "Route handler error"];
                       HTML.p []
                         [txt "handler %s" handler_name];
                       HTML.pre [] [txt "%s" msg];
                     ]);
                ())
          end
        in
        page ())
  in
  match meth, path with
  | `GET, "/login" ->
      launch (fun session ->
        let () = () in
         login_form_page reqd;
         ())
  | `POST, "/login" ->
      launch (fun session -> post_login session ~source_ip reqd)
  | `POST, "/logout" -> launch (fun session -> post_logout session reqd)
  | `GET, "/admin/users" -> launch (fun session -> admin_users_page session reqd)
  | `POST, "/admin/users" -> launch (fun session -> post_admin_users session reqd)
  | `GET, "/settings/keys" -> launch (fun session -> settings_keys_page session reqd)
  | `POST, "/settings/keys" -> launch (fun session -> post_settings_keys session reqd)
  | `GET, "/settings/ssh" -> launch (fun session -> settings_ssh_page session reqd)
  | `POST, "/settings/ssh" -> launch (fun session -> post_settings_ssh session reqd)
  | `GET, "/analytics" -> launch (fun session -> analytics_page session reqd)
  | `GET, "/" -> launch (fun session -> home_page session reqd)
  | `GET, "/welcome" ->
      launch_user_route_or (fun () -> Http_code.fallback_welcome reqd)
  | `GET, "/background" ->
      launch_user_route_or (fun () ->
        Http_code.serve_public_body reqd ~title:"Background"
          "<p><a href=\"/welcome\">welcome</a> ? author background page not seeded yet.</p>")
  | `GET, "/roadmap" ->
      launch_user_route_or (fun () ->
        Http_code.serve_public_body reqd ~title:"Roadmap"
```

```
          "<p><a href=\"/welcome\">welcome</a> ? roadmap page not seeded
     yet.</p>")
  | `GET, "/signup" | `POST, "/signup" ->
      launch_user_route_or (fun () ->
        Http_code.serve_public_body reqd ~title:"Signup"
          "<p><a href=\"/welcome\">welcome</a> ? signup page not seeded
     yet.</p>")
  | `GET, "/guestbook" | `POST, "/guestbook" ->
      launch_user_route_or (fun () ->
        Http_code.serve_public_body reqd ~title:"Guestbook"
          "<p><a href=\"/welcome\">welcome</a> ? guestbook page not seeded
     yet.</p>")
  | `GET, "/book" ->
      (* habitat-book.web-renderer: the book package's merged index (borg
         chapters + package spec docs) when book/page is bound; else the
         /book route pin (spec/book-page, habitat-workflow.book-view);
         else the static release face. A handler ERROR is surfaced (500),
         never masked by falling through. *)
      launch (fun session ->
          let request : Session.request =
            { method_ = `GET
            ; path
            ; params = parse_query target
            ; body = ""
            ; headers = header_list req
            }
          in
          let serve_handler handler_name =
            match Session.call_handler session ~request handler_name with
            | Session.Handler_ok value ->
                respond_handler_value ~session ~reqd ~path ~fragment:false
     value;
                true
            | Session.Handler_unbound -> false
            | Session.Handler_error msg ->
                text_response reqd ~status:`Internal_server_error
                  ~content_type:"text/plain" msg;
                true
          in
          let served =
            serve_handler "book/page"
            ||
            (match
               (match Request_context.get () with
                | Some ctx -> Request_context.lookup_route ctx path
                | None -> Session.lookup_route session path)
             with
             | Some handler_name -> serve_handler handler_name
             | None -> false)
          in
          if not served then Http_code.fallback_book reqd;
          ())
  | `GET, path
    when String.length path > 14 && String.sub path 0 14 = "/book/chapter/"
     ->
      (* habitat-book.web-renderer: borg chapter pages over the seeded
         tree rows (scripts/load-book.sh). Prefix is 14 bytes; >14 means
         a nonempty label follows. *)
      launch_book_handler "book/chapter-page"
  | `GET, path
    when String.length path > 11 && String.sub path 0 11 = "/book/code/" ->
      (* habitat-book.web-renderer: one code-chapter listing. *)
      launch_book_handler "book/code-page"
  | `GET, path
    when String.length path > 13 && String.sub path 0 13 = "/book/course/"
     ->
      (* habitat-book.course-documents: notes/outline course docs (slug
         course/<suffix>, kind course) assembled into an ordered reading
         list over the seeded chapter rows -- compose, never copy. *)
      launch_book_handler "book/course-page"
  | `POST, "/book/eval-example" ->
      launch (fun session -> post_book_eval_example session);
      ()
```

```
  | `GET, "/book/verify" ->
      (* habitat-book.eval-examples: (book/verify) as the book's
         self-test -- re-evals every nopales example row with an
         `expect` trailer; mismatch/refusal renders as a hole row
         (drift-as-rendering). ?chapter=<slug> scopes to one chapter. *)
      launch_book_handler "book/verify-page"
  | `GET, path
    when String.length path > 6 && String.sub path 0 6 = "/book/" ->
      (* habitat-workflow.book-view: /book/<pkg> ? (spec/book-page).
         request-path carries the full path so Nopales can extract pkg.
         habitat-book.web-renderer keeps /book/pkg/<pkg> as an alias:
         the effective path strips the pkg/ segment. *)
      let path =
        if
          String.length path > 10 && String.sub path 0 10 = "/book/pkg/"
        then "/book/" ^ String.sub path 10 (String.length path - 10)
        else path
      in
      launch (fun session ->
          let request : Session.request =
            { method_ = `GET
            ; path
            ; params = parse_query target
            ; body = ""
            ; headers = header_list req
            }
          in
          match Session.call_handler session ~request "spec/book-page" with
          | Session.Handler_ok value ->
              respond_handler_value ~session ~reqd ~path ~fragment:false
    value
          | Session.Handler_unbound ->
              text_response reqd ~status:`Not_found
    ~content_type:"text/plain"
                "spec/book-page unbound -- (load-library \"spec\")"
          | Session.Handler_error msg ->
              text_response reqd ~status:`Internal_server_error
                ~content_type:"text/plain" msg);
      ()
  | `GET, "/book.pdf" ->
      launch (fun session ->
        Http_code.handle_public_artifact reqd ~name:"book.pdf";
        ())
  | `GET, "/src.tgz" ->
      launch (fun session ->
        Http_code.handle_public_artifact reqd ~name:"src.tgz";
        ())
  | `GET, "/code" ->
      launch (fun session ->
        Http_code.handle_code_page reqd ~target;
        ())
  | `GET, path when String.length path >= 10
                 && String.sub path 0 10 = "/code/raw/" ->
      let rel = String.sub path 10 (String.length path - 10) in
      launch (fun session ->
        Http_code.handle_raw reqd ~rel;
        ())
  | `GET, path when String.length path >= 11
                 && String.sub path 0 11 = "/assets/cm/" ->
      (* Back-compat alias: /assets/cm/* ? static/codemirror/* *)
      let rel =
        "codemirror/" ^ String.sub path 11 (String.length path - 11)
      in
      launch (fun session ->
        Http_code.handle_asset reqd ~rel;
        ())
  | `GET, path when String.length path >= 8
                 && String.sub path 0 8 = "/assets/" ->
      let rel = String.sub path 8 (String.length path - 8) in
      launch (fun session ->
        Http_code.handle_asset reqd ~rel;
        ())
  | `GET, "/about" ->
```

```
      launch (fun session ->
        let () = () in
         about_page session reqd;
         ())
  | `GET, "/pages" ->
      launch (fun session ->
        let () = () in
         pages_list_page session reqd;
         ())
  | `POST, "/api/recipe/hello" ->
      launch (fun session ->
        let res = Session.recipe_hello session in
         match res with
         | Ok msg ->
             let body =
               chrome ~session ~title:"Hello recipe" [
                 HTML.div [HTML.class_ "card"] [
                   HTML.p [HTML.class_ "ok"] [txt "%s" msg];
                   HTML.p [] [
                     HTML.a [HTML.href "/hello"] [txt "Open /hello"];
                     txt " · ";
                     HTML.a [HTML.href "/edit/hi-page"] [txt "Edit hi-page"];
                     txt " · ";
                     HTML.a [HTML.href "/pages"] [txt "Pages"];
                     txt " · ";
                     HTML.a [HTML.href "/"] [txt "Home"];
                   ];
                 ];
               ]
             in
             html_response reqd ~session body;
             ()
         | Error e ->
             let body =
               chrome ~session ~title:"Hello recipe failed" [
                 HTML.p [HTML.class_ "err"] [txt "%s" e];
                 HTML.p [] [HTML.a [HTML.href "/"] [txt "Home"]];
               ]
             in
             html_response reqd ~session ~status:`Internal_server_error body;
             ())
  | `POST, "/api/recipe/todo" ->
      launch (fun session ->
        let res = Session.recipe_todo session in
         match res with
         | Ok msg ->
             let body =
               chrome ~session ~title:"Todo library" [
                 HTML.div [HTML.class_ "card"] [
                   HTML.p [HTML.class_ "ok"] [txt "%s" msg];
                   HTML.p [] [
                     HTML.a [HTML.href "/todos"] [txt "Open /todos"];
                     txt " · ";
                     HTML.a [HTML.href "/bindings?pkg=todo"] [txt "Package todo"];
                     txt " · ";
                     HTML.a [HTML.href "/pages"] [txt "Pages"];
                   ];
                 ];
               ]
             in
             html_response reqd ~session body;
             ()
         | Error e ->
             let body =
               chrome ~session ~title:"Todo recipe failed" [
                 HTML.p [HTML.class_ "err"] [txt "%s" e];
                 HTML.p [] [HTML.a [HTML.href "/"] [txt "Home"]];
               ]
             in
```

```
            html_response reqd ~session ~status:`Internal_server_error
    body;
            ())
  | `POST, "/api/recipe/notes" ->
      launch (fun session ->
        let res = Session.recipe_notes session in
         match res with
         | Ok msg ->
             let body =
               chrome ~session ~title:"Notes library" [
                 HTML.div [HTML.class_ "card"] [
                   HTML.p [HTML.class_ "ok"] [txt "%s" msg];
                   HTML.p [] [
                     HTML.a [HTML.href "/notes"] [txt "Open /notes"];
                     txt " ? ";
                     HTML.a [HTML.href "/bindings?pkg=notes"] [txt "Package
    notes"];
                     txt " ? ";
                     HTML.a [HTML.href "/bindings?pkg=outline"] [txt
    "outline"];
                   ];
                   HTML.p [HTML.class_ "muted"] [
                     txt "Outline-first notes (org-shaped). Content lives
    in data_rows.";
                   ];
                 ];
               ]
             in
             html_response reqd ~session body;
             ()
         | Error e ->
             let body =
               chrome ~session ~title:"Notes recipe failed" [
                 HTML.p [HTML.class_ "err"] [txt "%s" e];
                 HTML.p [] [HTML.a [HTML.href "/"] [txt "Home"]];
               ]
             in
             html_response reqd ~session ~status:`Internal_server_error
    body;
             ())
  | `POST, "/api/recipe/dabar" ->
      launch (fun session ->
        let res = Session.recipe_dabar session in
         match res with
         | Ok msg ->
             let body =
               chrome ~session ~title:"Dabar library" [
                 HTML.div [HTML.class_ "card"] [
                   HTML.p [HTML.class_ "ok"] [txt "%s" msg];
                   HTML.p [] [
                     HTML.a [HTML.href "/dabar?book=John&chapter=3"]
                       [txt "Open /dabar"];
                     txt " ? ";
                     HTML.a [HTML.href
    "/dabar?book=John&chapter=3&verse=16"]
                       [txt "Study John 3:16"];
                     txt " ? ";
                     HTML.a [HTML.href "/strongs?n=G2316"] [txt "Strong's
    G2316"];
                   ];
                   HTML.p [HTML.class_ "muted"] [
                     txt "Seeds: scripts/seed_bible_bsb.py ?
    seed_strongs.py ? seed_interlinear_dogfood.py";
                   ];
                 ];
               ]
             in
             html_response reqd ~session body;
             ()
         | Error e ->
             let body =
               chrome ~session ~title:"Dabar recipe failed" [
                 HTML.p [HTML.class_ "err"] [txt "%s" e];
```

```
742                 HTML.p [] [HTML.a [HTML.href "/"] [txt "Home"]];
743               ]
744             in
745             html_response reqd ~session ~status:`Internal_server_error
    body;
746             ())
747   | `GET, "/health" ->
748       (* image.http-robustness invariant 7: /health must probe the same
749          store path auth/prims use. Memory-only ok:true hid Eio mutex
750          poison and locked out all logins while watchdog saw green. *)
751       let branch = Session.branch_name session in
752       let uptime = Session.uptime_seconds session in
753       let check = Store.health_check (Session.store session) in
754       let h, status =
755         match check with
756         | Ok () ->
757             ( { Pricklypear_common.Protocol.ok = true;
758                 branch;
759                 uptime_seconds = uptime;
760                 error = None },
761               `OK )
762         | Error msg ->
763             ( { Pricklypear_common.Protocol.ok = false;
764                 branch;
765                 uptime_seconds = uptime;
766                 error = Some msg },
767               `Service_unavailable )
768       in
769       let j = Pricklypear_common.Protocol.health_response_to_json h in
770       text_response reqd ~status ~content_type:"application/json"
771         (Yojson.Safe.pretty_to_string j)
772   (* Public agent bootstrap -- no auth, no secrets. /agent stays private. *)
773   | `GET, "/.well-known/agent.json" ->
774       Agent_discovery.serve_agent_json reqd ~headers
775   | `GET, "/agent.txt" ->
776       Agent_discovery.serve_agent_txt reqd ~headers
777   | `GET, "/branches" -> launch (fun session -> branches_page session reqd)
778   | `GET, "/changes" -> launch (fun session -> changes_page session reqd)
779   | `POST, "/branches/checkout" -> launch (fun session -> branches_checkout
    session reqd)
780   | `POST, "/branches/fork" -> launch (fun session -> branches_fork session
    reqd)
781   | `POST, "/branches/unoverride" -> launch (fun session ->
    branches_unoverride session reqd)
782   | `POST, "/branches/promote" -> launch (fun session -> branches_promote
    session reqd)
783   | `GET, "/bindings" -> launch (fun session -> list_bindings_page session
    reqd)
784   | `GET, "/diff" -> launch (fun session -> diff_page session reqd)
785   | `GET, "/types" -> launch (fun session -> list_types_page session reqd)
786   | `GET, "/timing" -> launch (fun session -> timing_page session reqd)
787   | `GET, "/data" | `GET, "/data/" ->
788       launch (fun session -> data_index_page session reqd)
789   | `GET, path when String.length path >= 6
790                  && String.sub path 0 6 = "/data/" ->
791       let rest = String.sub path 6 (String.length path - 6) in
792       let segs = String.split_on_char '/' rest |> List.filter ((<>) "") in
793       (match segs with
794        | [ty; "new"] ->
795            launch (fun session -> data_new_page session ty reqd)
796        | [ty; id; "delete"] ->
797            (* GET delete not supported -- point at detail *)
798            launch (fun session -> data_detail_page session ty id reqd)
799        | [ty; id] ->
800            launch (fun session -> data_detail_page session ty id reqd)
801        | [ty] ->
802            launch (fun session -> data_list_page session ty reqd)
803        | _ ->
804            html_response reqd ~session ~status:`Bad_request
805              (chrome ~session ~title:"Bad data path" [
806                 HTML.p [] [txt "use /data, /data/<type>, /data/<type>/new,
    /data/<type>/<id>"];
807               ]))
```

```
  | `POST, path when String.length path >= 6
                  && String.sub path 0 6 = "/data/" ->
      let rest = String.sub path 6 (String.length path - 6) in
      let segs = String.split_on_char '/' rest |> List.filter ((<>) "") in
      (match segs with
       | [ty; id; "delete"] ->
           launch (fun session -> data_delete session ty id reqd)
       | [ty; id] ->
           launch (fun session -> data_update session ty id reqd)
       | [ty] ->
           launch (fun session -> data_create session ty reqd)
       | _ ->
           html_response reqd ~session ~status:`Bad_request
             (chrome ~session ~title:"Bad data path" [
                HTML.p [] [txt "POST /data/<type> | /data/<type>/<id> | ?/delete"];
              ]))
  | `GET, "/tree" -> launch (fun session -> Http_tree.tree_page session reqd)
  | `GET, "/partials/tree" ->
      launch (fun session -> Http_tree.tree_partial session reqd)
  | `POST, "/tree/edit" -> launch (fun session -> Http_tree.tree_edit session reqd)
  | `GET, "/repl" ->
      launch (fun session ->
        let () = () in
        repl_page session reqd;
        ())
  | `POST, "/api/eval" -> launch (fun session -> api_eval session reqd)
  | `POST, "/api/prim" -> launch (fun session -> api_prim session reqd)
  | `POST, "/api/media" -> launch (fun session -> api_media_post session reqd)
  | `GET, "/api/media" ->
      launch (fun session -> api_media_list session reqd ~target)
  | `DELETE, path when String.length path > 11
                    && String.sub path 0 11 = "/api/media/" ->
      let id = String.sub path 11 (String.length path - 11) in
      launch (fun session -> api_media_delete session reqd ~id)
  | `GET, "/media" | `GET, "/media/" ->
      launch (fun session -> media_page session reqd ~target)
  | `POST, "/media/upload" ->
      launch (fun session -> media_upload_post session reqd)
  | `POST, path when String.length path > 7
                  && String.sub path 0 7 = "/media/"
                  && String.ends_with ~suffix:"/delete" path ->
      let id =
        let rest = String.sub path 7 (String.length path - 7) in
        (* strip trailing /delete *)
        if String.length rest > 7 then
          String.sub rest 0 (String.length rest - 7)
        else rest
      in
      launch (fun session -> media_delete_post session reqd ~id)
  | `GET, path when String.length path > 7
                 && String.sub path 0 7 = "/media/" ->
      let id = String.sub path 7 (String.length path - 7) in
      launch (fun session -> media_get session reqd ~id)
  | `POST, "/api/agent" -> launch (fun session -> api_agent session reqd)
  | `POST, "/api/agent-task" -> launch (fun session -> api_agent_task session reqd)
  | `POST, "/repl-v2/llm" -> launch (fun session -> repl_llm session reqd)
  | `POST, "/api/ai/propose" -> launch (fun session -> ai_propose session reqd)
  | `GET, "/agent" -> launch (fun session -> agent_page session reqd)
  | `POST, "/api/edit/create" -> launch (fun session -> api_edit_create session reqd)
  | `POST, path when String.length path >= 13
                  && String.sub path 0 13 = "/api/analyze/" ->
      let name =
        let rest = String.sub path 13 (String.length path - 13) in
        let rest =
          match String.index_opt rest '?' with
          | None -> rest
```

```
          | Some i -> String.sub rest 0 i
        in
        Packages.name_decode rest
      in
      launch (fun session -> Http_analyze.post_analyze session name reqd)
  | `POST, path when String.length path >= 13
                  && String.sub path 0 13 = "/api/explain/" ->
      let name =
        let rest = String.sub path 13 (String.length path - 13) in
        let rest =
          match String.index_opt rest '?' with
          | None -> rest
          | Some i -> String.sub rest 0 i
        in
        Packages.name_decode rest
      in
      launch (fun session -> Http_analyze.post_explain session name reqd)
  | `GET, path when String.length path >= 6
                && String.sub path 0 6 = "/page/" ->
      let seg = String.sub path 6 (String.length path - 6) in
      launch (fun session -> page_inspector session seg reqd)
  | `GET, "/page" | `GET, "/page/" ->
      launch (fun session -> page_inspector session "" reqd)
  | `GET, path when String.length path >= 17
                && String.sub path 0 17 = "/inspect/binding/" ->
      let name =
        Packages.name_decode (String.sub path 17 (String.length path - 17))
      in
      launch (fun session -> inspect_binding_page session name reqd)
  | `GET, path when String.length path >= 13
                && String.sub path 0 13 = "/inspect/row/" ->
      let rest = String.sub path 13 (String.length path - 13) in
      (match String.split_on_char '/' rest with
       | [ty; id] -> launch (fun session -> inspect_row_page session ty id
     reqd)
       | _ ->
           html_response reqd ~session ~status:`Bad_request
             (chrome ~session ~title:"Bad inspect path" [
                HTML.p [] [txt "use /inspect/row/<type>/<id>"];
              ]))
  | `GET, path when String.length path >= 9
                && String.sub path 0 9 = "/binding/" ->
      let name =
        Packages.name_decode (String.sub path 9 (String.length path - 9))
      in
      launch (fun session -> show_binding_page session name reqd)
  | `GET, "/edit" | `GET, "/edit/" ->
      (* package + local query form from Functions browser *)
      let pkg = Option.value (query_param target "package") ~default:"" in
      let local = Option.value (query_param target "local") ~default:"" in
      let name = Packages.join_name ~package:pkg ~local in
      if name = "" then
        html_response reqd ~session
          (chrome ~session ~title:"New function" [
             HTML.p [] [txt "Provide package and/or name (see Functions)."];
             HTML.p [] [HTML.a [HTML.href "/bindings"] [txt "Functions"]];
           ])
      else launch (fun session -> editor_page session name reqd)
  | `GET, path when String.length path >= 6
                && String.sub path 0 6 = "/edit/" ->
      let name =
        Packages.name_decode (String.sub path 6 (String.length path - 6))
      in
      launch (fun session -> editor_page session name reqd)
  | `POST, path when String.length path >= 6
                && String.sub path 0 6 = "/edit/" ->
      let name =
        Packages.name_decode (String.sub path 6 (String.length path - 6))
      in
      launch (fun session -> save_edit session name reqd)
  | ((`GET | `POST) as umeth), _ ->
      (* User routes: GET + POST (it33). Same path ? same handler;
         request-* prims expose method/params for branching. Route is
```

```
946           resolved INSIDE launch (after the gate binds the per-request
947           context) so a concurrent attach cannot flip the table under us. *)
948         launch (fun session ->
949         let route_opt =
950           match Request_context.get () with
951           | Some ctx -> Request_context.lookup_route ctx path
952           | None -> Session.lookup_route session path
953         in
954         match route_opt with
955          | Some handler_name ->
956              let page () : unit =
957                let body_str =
958                  match umeth with
959                  | `POST -> read_body reqd
960                  | `GET -> ""
961                in
962                if String.length body_str > max_user_body_bytes then begin
963                  text_response reqd ~status:`Payload_too_large
964                    ~content_type:"text/plain"
965                    "request body too large (max 64KiB)";
966                  ()
967                end else begin
968                  let query = parse_query target in
969                  let form =
970                    match umeth with
971                    | `GET -> []
972                    | `POST ->
973                        let ct =
974                          match
975                            Httpun.Headers.get req.headers "content-type"
976                          with
977                          | Some s -> s
978                          | None -> ""
979                        in
980                        if String.length ct >= 33
981                           && String.sub ct 0 33
982                              = "application/x-www-form-urlencoded"
983                           || ct = ""
984                              && String.contains body_str '='
985                        then parse_form_body body_str
986                        else []
987                  in
988                  let params = merge_params ~query ~form in
989                  let headers = header_list req in
990                  let fragment = wants_fragment ~path ~headers in
991                  let method_ : Session.http_method =
992                    match umeth with `GET -> `GET | `POST -> `POST
993                  in
994                  let request : Session.request =
995                    { method_; path; params; body = body_str; headers }
996                  in
997                  let result, _total_ms =
998                    Timing.measure ~route:handler_name
999                      (fun () -> Session.call_handler session ~request
    handler_name)
1000                 in
1001                 (match result with
1002                  | Session.Handler_ok value ->
1003                      respond_handler_value ~session ~reqd ~path ~fragment
    value;
1004                      ()
1005                  | Session.Handler_unbound ->
1006                      if fragment then begin
1007                        fragment_response reqd ~status:`Internal_server_error
1008                          ("handler " ^ handler_name ^ " unbound");
1009                        ()
1010                      end else
1011                        let snap = Call_graph.snapshot_of_session session in
1012                        let callers = Call_graph.callers snap handler_name in
1013                        let caller_nodes =
1014                          if callers = [] then [txt "(none yet)"]
1015                          else
1016                            List.concat_map
```

```
                          (fun n ->
                            [ HTML.a
                                [HTML.href "%s" (Packages.binding_href n)]
                                [txt "%s" n]
                            ; txt " "
                            ])
                          callers
                    in
                    html_response reqd ~session ~status:`Not_found
                      (chrome ~session ~title:"Handler unbound" [
                         HTML.div [HTML.class_ "card"] [
                           HTML.h1 [] [txt "Handler unbound"];
                           HTML.p [] [
                             txt "Route ";
                             HTML.code [] [txt "%s" path];
                             txt " points at ";
                             HTML.code [] [txt "%s" handler_name];
                             txt " which is not bound.";
                           ];
                           HTML.p [] [
                             HTML.a
                               [ HTML.href "%s"
                                   (Packages.edit_href handler_name)
                               ; HTML.class_ "btn" ]
                               [txt "Create stub %s" handler_name];
                             txt " ";
                             HTML.a
                               [ HTML.href "%s"
                                   (Packages.binding_href handler_name)
                               ; HTML.class_ "btn secondary" ]
                               [txt "Inspect"];
                           ];
                           HTML.p []
                             ([ HTML.span [HTML.class_ "muted"]
                                  [txt "static callers: "] ]
                              @ caller_nodes);
                         ];
                       ]);
                    ()
              | Session.Handler_error msg ->
                  if fragment then
                    fragment_response reqd ~status:`Internal_server_error
                      ("handler " ^ handler_name ^ ": " ^ msg)
                  else
                    html_response reqd ~session
  ~status:`Internal_server_error
                      (chrome ~session ~title:"Route handler error" [
                         HTML.h1 [] [txt "Route handler error"];
                         HTML.p [] [txt "handler %s" handler_name];
                         HTML.pre [] [txt "%s" msg];
                       ]);
                  ())
          end
        in
        page ()
    | None ->
        let headers = header_list req in
        if wants_fragment ~path ~headers then
          fragment_response reqd ~status:`Not_found
            ("no partial for " ^ path)
        else
          html_response reqd ~session ~status:`Not_found
            (chrome ~session ~title:"Not found" [
               HTML.p [] [txt "no route for %s" path];
             ]);
        ())
| _, _ ->
    let headers = header_list req in
    if wants_fragment ~path ~headers then
      fragment_response reqd ~status:`Not_found ("no partial for " ^ path)
    else
      html_response reqd ~session ~status:`Not_found
        (chrome ~session ~title:"Not found" [
```

```
            HTML.p [] [txt "no route for %s" path];
          ])

let error_handler (_sockaddr : Unix.sockaddr)
    ?(request : Httpun.Request.t option)
    (err : Httpun.Server_connection.error)
    (respond : Httpun.Headers.t -> Httpun.Body.Writer.t) : unit =
  let status, msg =
    match err with
    | `Bad_request -> `Bad_request, "bad request"
    | `Bad_gateway -> `Bad_gateway, "bad gateway"
    | `Internal_server_error -> `Internal_server_error, "internal server error"
    (* Never echo raw exception text to the client -- generic body only. *)
    | `Exn _ -> `Internal_server_error, "internal server error -- incident logged"
  in
  let _ = request in
  let body_str =
    Pure_html.to_string
      (HTML.html [HTML.lang "en"] [
         HTML.head [] [HTML.title [] "Pricklypear Error"];
         HTML.body [] [
           HTML.h1 [] [txt "Error"];
           HTML.p [] [txt "%s" msg];
         ];
       ])
  in
  let headers =
    Httpun.Headers.of_list
      [("content-type", "text/html");
       ("content-length", string_of_int (String.length body_str))]
  in
  let body_w = respond headers in
  try
    Httpun.Body.Writer.write_string body_w body_str;
    safe_close_writer body_w
  with exn when is_closed_writer_exn exn -> ()

(** Legacy httpun-lwt-unix accept loop -- retired. Process always uses [serve_eio]. *)
let serve ~(session : Session.t) ?(port = default_port) () : unit Lwt.t =
  let _ = session in
  let _ = port in
  failwith "Http.serve (lwt-unix): retired; use serve_eio under Eio_main"

(** Convert Eio stream sockaddr ? Unix.sockaddr for shared request_handler. *)
let unix_sockaddr_of_eio (sa : Eio.Net.Sockaddr.stream) : Unix.sockaddr =
  match sa with
  | `Unix path -> Unix.ADDR_UNIX path
  | `Tcp (ip, port) ->
      let uip = Eio_unix.Net.Ipaddr.to_unix ip in
      Unix.ADDR_INET (uip, port)

(** Native Eio accept loop via Httpun_eio (H3). Call under Eio_main + ambient Io_eio.with_ctx. Blocks the fiber forever. *)
let serve_eio
    ~(session : Session.t)
    ~(registry : Session_registry.t)
    ~(env : Eio_unix.Stdenv.base)
    ~(sw : Eio.Switch.t)
    ?(port = default_port)
    () : unit =
  let request_handler (sa : Eio.Net.Sockaddr.stream)
      (g : Httpun.Reqd.t Gluten.reqd) : unit =
    request_handler ~sw ~registry session (unix_sockaddr_of_eio sa) g.Gluten.reqd
  in
  let error_handler (sa : Eio.Net.Sockaddr.stream) =
    error_handler (unix_sockaddr_of_eio sa)
  in
  let conn_handler =
```

```
    Httpun_eio.Server.create_connection_handler
      ~request_handler ~error_handler ~sw
  in
  let net = Eio.Stdenv.net env in
  let listen_addr = `Tcp (Eio.Net.Ipaddr.V4.any, port) in
  (* reuse_port off: OpenBSD SO_REUSEPORT semantics differ and caused
     EADDRINUSE races on rcctl restart when dual-binding. reuse_addr is
     enough. *)
  let socket =
    Eio.Net.listen ~reuse_addr:true ~reuse_port:false ~backlog:128 ~sw net
      listen_addr
  in
  Printf.eprintf
    "[http] serving htmx (httpun-eio) on 0.0.0.0:%d (max_inflight=%d)\n%!"
    port max_inflight;
  let on_error ex =
    if is_closed_writer_exn ex then
      ignore
        (Ops_log.info ~component:"http"
           "eio connection: peer gone (closed writer)")
    else
      ignore
        (Ops_log.warn ~component:"http"
           ~context:[("phase", "eio-connection")]
           ("eio connection handler failed: " ^ Printexc.to_string ex))
  in
  while true do
    Eio.Net.accept_fork socket ~sw ~on_error (fun client_sock client_addr ->
        try conn_handler client_addr client_sock
        with exn -> on_error exn)
  done
```

71 image/lib/http_agent.ml

```
(** Agent portal + AI propose HTTP handlers. *)
open Pure_html
open Http_core

(* Client disconnect mid-SSE closes the httpaf body writer. Writing after
   that raises Failure("cannot write to closed writer") and -- if uncaught --
   kills the whole single-image process. Treat peer-gone as a normal end. *)
let is_closed_writer_exn = function
  | Failure msg ->
      String.equal msg "cannot write to closed writer"
      || String.starts_with ~prefix:"cannot write to closed writer" msg
  | _ -> false

(* [is_closed_writer_exn] now shadows Http_core's identical helper; kept as a
   local alias so existing call sites stay unchanged. Body writes route
   through Http_core.safe_close_writer / safe_respond_with_string. *)

let sse_write (body : Httpun.Body.Writer.t) (event_json : string)
  : [ `Ok | `Peer_gone ] =
  let chunk = "data: " ^ event_json ^ "\n\n" in
  try
    Httpun.Body.Writer.write_string body chunk;
    let promise, resolver = Eio.Promise.create () in
    let wake () =
      if not (Eio.Promise.is_resolved promise) then
        Eio.Promise.resolve resolver ()
    in
    (try Httpun.Body.Writer.flush body (fun () -> wake ()) with
     | exn when is_closed_writer_exn exn -> wake ());
    (* Await flush ack (Httpun callback). *)
    Eio.Promise.await promise;
    `Ok
  with exn when is_closed_writer_exn exn ->
    ignore (Ops_log.info ~component:"sse" "peer gone (closed writer)");
    `Peer_gone

let close_writer_quiet (body : Httpun.Body.Writer.t) : unit =
  safe_close_writer body

(* Multipart/mixed streaming write -- one part frame + flush. Mirrors
   sse_write but emits a multipart part with per-part HX-Target/HX-Swap
   so hx-multipart can route each token into the live bubble. Returns
   `Peer_gone on closed writer so the drain loop stops cleanly. *)
let mp_write_part (body : Httpun.Body.Writer.t) ~boundary
    ?(target : string option) ?(swap : string option) (html : string)
  : [ `Ok | `Peer_gone ] =
  let buf = Buffer.create 128 in
  Buffer.add_string buf ("--" ^ boundary ^ "\r\n");
  Buffer.add_string buf "Content-Type: text/html; charset=utf-8\r\n";
  (match target with Some t -> Buffer.add_string buf ("HX-Target: " ^ t ^ "\r\n") | None -> ());
  (match swap with Some s -> Buffer.add_string buf ("HX-Swap: " ^ s ^ "\r\n") | None -> ());
  Buffer.add_string buf "\r\n";
  Buffer.add_string buf html;
  Buffer.add_string buf "\r\n";
  let chunk = Buffer.contents buf in
  try
    Httpun.Body.Writer.write_string body chunk;
    let promise, resolver = Eio.Promise.create () in
    let wake () =
      if not (Eio.Promise.is_resolved promise) then Eio.Promise.resolve resolver ()
    in
    (try Httpun.Body.Writer.flush body (fun () -> wake ()) with
     | exn when is_closed_writer_exn exn -> wake ());
    Eio.Promise.await promise;
    `Ok
  with exn when is_closed_writer_exn exn ->
```

```
    ignore (Ops_log.info ~component:"mp" "peer gone (closed writer)");
    `Peer_gone

(* Close the multipart body with the terminating boundary. *)
let mp_close (body : Httpun.Body.Writer.t) ~boundary : unit =
  let chunk = "--" ^ boundary ^ "--\r\n" in
  (try Httpun.Body.Writer.write_string body chunk
   with exn when is_closed_writer_exn exn -> ());
  close_writer_quiet body

let default_http_agent_seed =
  (* Short fallback when no session seed; tools sessions should prefer
     Session.default_agent_seed (dogfood + register_route rules). *)
  "You are the co-pilot for a Pricklypear multi-user Lisp OS image " ^
  "(software, not a cactus). Prefer CONTEXT; if unknown say so. " ^
  "Use structured tools only (never <tool_call> XML). " ^
  "install_function for lambdas; register_route for HTTP paths; " ^
  "data_rows via create/rows (not outline/*); inspect todo/page first."

(** POST /api/agent -- durable sessions.
    Body JSON:
      {prompt, seed?, session_id?, tools?, max_steps?}
    Creates a session when session_id omitted; always returns session_id.
    Progress SSE when Accept prefers event-stream or stream:true; default
    is non-stream JSON (token streaming deferred / flaky on Synthetic). *)
(** Pull the human line out of the HTTP agent prompt wrapper for titles/UI.
    *)
let agent_user_visible_text (s : string) : string =
  let s = String.trim s in
  let key = "\nUser:\n" in
  let rec find_sub hay needle i =
    let nlen = String.length needle in
    let hlen = String.length hay in
    if i + nlen > hlen then None
    else if String.sub hay i nlen = needle then Some i
    else find_sub hay needle (i + 1)
  in
  match find_sub s key 0 with
  | Some i ->
      String.trim
        (String.sub s (i + String.length key)
           (String.length s - i - String.length key))
  | None ->
      if String.length s >= 20
         && String.sub s 0 20 = "=== IMAGE CONTEXT ===" then
        let n = min 80 (String.length s) in
        String.sub s 0 n ^ (if String.length s > 80 then "?" else "")
      else s

let agent_session_label (s : Ai_store.session) : string =
  let short = String.sub s.id 0 (min 8 (String.length s.id)) in
  match s.title with
  | Some t when String.trim t <> "" ->
      let t = agent_user_visible_text t in
      (* Old rows stored the full IMAGE CONTEXT wrapper as title (truncated
         before User:). Don't show that dump in the chat list. *)
      if String.trim t = ""
         || (String.length t >= 20
             && String.sub t 0 20 = "=== IMAGE CONTEXT ===")
         || String.starts_with ~prefix:"=== IMAGE CONTEXT" t then
        "chat " ^ short
      else if String.length t > 72 then String.sub t 0 72 ^ "?"
      else t
  | _ -> "chat " ^ short

let api_agent (session : Session.t) (reqd : Httpun.Reqd.t) : unit =
  let body = read_body reqd in
  let seed, prompt, session_id_opt, tools, max_steps, want_stream =
    try
      match Yojson.Safe.from_string body with
      | `Assoc items ->
          let seed =
            match List.assoc_opt "seed" items with
```

```
            | Some (`String s) -> s
            | _ -> default_http_agent_seed
          in
          let prompt =
            match List.assoc_opt "prompt" items with
            | Some (`String s) -> s
            | _ ->
                (match List.assoc_opt "q" items with
                 | Some (`String s) -> s
                 | _ -> "")
          in
          let sid =
            match List.assoc_opt "session_id" items with
            | Some (`String s) when String.trim s <> "" -> Some (String.trim s)
            | _ -> None
          in
          let tools =
            match List.assoc_opt "tools" items with
            | Some (`Bool b) -> b
            | Some (`String "0") | Some (`String "false") -> false
            | _ -> true
          in
          let max_steps =
            match List.assoc_opt "max_steps" items with
            | Some (`Int n) -> n
            | Some (`String s) ->
                (match int_of_string_opt s with Some n -> n | None -> 16)
            | _ -> 16
          in
          let stream =
            match List.assoc_opt "stream" items with
            | Some (`Bool b) -> b
            | Some (`String "1") | Some (`String "true") -> true
            | Some (`String "0") | Some (`String "false") -> false
            | _ -> false
          in
          (seed, prompt, sid, tools, max_steps, stream)
      | _ -> (default_http_agent_seed, "", None, true, 16, false)
    with _ ->
      let seed =
        Option.value (form_field body "seed") ~default:default_http_agent_seed
      in
      let prompt =
        match form_field body "prompt" with
        | Some p -> p
        | None -> Option.value (form_field body "q") ~default:""
      in
      let sid = form_field body "session_id" in
      (seed, prompt, sid, true, 16, false)
  in
  if String.trim prompt = "" then begin
    json_response reqd
      (`Assoc [ ("ok", `Bool false); ("error", `String "need prompt") ]);
    ()
  end else
    let sid_r =
      match session_id_opt with
      | Some sid -> (Ok sid)
      | None ->
          let title_opt =
            let p = agent_user_visible_text prompt in
            if String.trim p = "" then None
            else
              Some
                (if String.length p > 80 then String.sub p 0 80 ^ "?" else p)
          in
          let v =
            Session.agent_session_new session ~seed ~tools ?title:title_opt ()
          in
```

```
        (match v with
         | Pricklypear_lisp.Ast.VStr s
           when not (String.starts_with ~prefix:"agent-session-new error" s) ->
             (Ok s)
         | Pricklypear_lisp.Ast.VStr e -> (Error e)
         | other ->
             (Error (Pricklypear_lisp.Reader.pp_value other)))
    in
    match sid_r with
    | Error e ->
        json_response reqd
          (`Assoc [ ("ok", `Bool false); ("error", `String e) ]);
        ()
    | Ok sid ->
        if not want_stream then begin
          let v =
            Session.agent_say session ~session_id:sid ~prompt ~max_steps ()
          in
          let text =
            match v with
            | Pricklypear_lisp.Ast.VStr s -> s
            | _ -> Pricklypear_lisp.Reader.pp_value v
          in
          json_response reqd
            (`Assoc
               [ ("ok", `Bool true)
               ; ("session_id", `String sid)
               ; ("text", `String text)
               ]);
          ()
        end else begin
          (* Progress SSE (tool/token events), not provider token stream. *)
          let headers =
            Httpun.Headers.of_list
              [ ("content-type", "text/event-stream; charset=utf-8")
              ; ("cache-control", "no-cache")
              ; ("connection", "keep-alive")
              ; ("x-accel-buffering", "no")
              (* Chunked framing is MANDATORY for a streamed response:
                 without an explicit framing header httpun emits the body
                 close-delimited, and on a keep-alive HTTP/1.1 connection
                 the client can never detect end-of-body. *)
              ; ("transfer-encoding", "chunked")
              ]
          in
          let resp = Httpun.Response.create ~headers `OK in
          (* Peer may already be gone before the stream starts; on
             closed-writer, bail instead of letting it escape to the
             launch catch (image.http-robustness invariant 1). *)
          let body_w =
            try
              Httpun.Reqd.respond_with_streaming ~flush_headers_immediately:true
                reqd resp
            with exn when is_closed_writer_exn exn ->
              ignore (Ops_log.info ~component:"sse"
                        "peer gone before stream start");
              raise exn
          in
          let queue : string Queue.t = Queue.create () in
          let mutex = Eio.Mutex.create () in
          let cond = Eio.Condition.create () in
          let finished = ref false in
          let push_json (j : string) : unit =
            Eio.Mutex.lock mutex;
            Queue.add j queue;
            Eio.Condition.broadcast cond;
            Eio.Mutex.unlock mutex
          in
          push_json
            (Yojson.Safe.to_string
               (`Assoc
```

```
                [ ("t", `String "session")
                ; ("session_id", `String sid)
                ]));
        let on_event = function
          | Llm_client.Token tok ->
              push_json
                (Yojson.Safe.to_string
                   (`Assoc [ ("t", `String "token"); ("s", `String tok)
    ]))
          | Llm_client.Done ->
              push_json
                (Yojson.Safe.to_string (`Assoc [ ("t", `String "done")
    ]));
              finished := true;
              Eio.Mutex.lock mutex;
              Eio.Condition.broadcast cond;
              Eio.Mutex.unlock mutex
          | Llm_client.Fail e ->
              push_json
                (Yojson.Safe.to_string
                   (`Assoc [ ("t", `String "error"); ("s", `String e) ]));
              finished := true;
              Eio.Mutex.lock mutex;
              Eio.Condition.broadcast cond;
              Eio.Mutex.unlock mutex
        in
        (* Producer + drain under a local switch so tokens stream. *)
        Eio.Switch.run @@ fun sw ->
        Eio.Fiber.fork ~sw (fun () ->
            let v =
              Session.agent_say session ~session_id:sid ~prompt ~max_steps
                ~on_event ()
            in
            if not !finished then begin
              (match v with
               | Pricklypear_lisp.Ast.VStr s ->
                   push_json
                     (Yojson.Safe.to_string
                        (`Assoc
                           [ ("t", `String "done")
                           ; ("text", `String s)
                           ; ("session_id", `String sid)
                           ]))
               | _ ->
                   push_json
                     (Yojson.Safe.to_string
                        (`Assoc [ ("t", `String "done") ])));
              finished := true;
              Eio.Mutex.lock mutex;
              Eio.Condition.broadcast cond;
              Eio.Mutex.unlock mutex
            end);
        let rec drain () =
          Eio.Mutex.lock mutex;
          if not (Queue.is_empty queue) then begin
            let j = Queue.take queue in
            Eio.Mutex.unlock mutex;
            match sse_write body_w j with
            | `Peer_gone ->
                finished := true;
                close_writer_quiet body_w
            | `Ok -> drain ()
          end else if !finished then begin
            Eio.Mutex.unlock mutex;
            close_writer_quiet body_w
          end else begin
            Eio.Condition.await cond mutex;
            Eio.Mutex.unlock mutex;
            drain ()
          end
        in
        drain ()
      end
```

```
350
351 let api_agent_task (session : Session.t) (reqd : Httpun.Reqd.t) : unit =
352   (* Alias: create tools session + say (same as /api/agent with tools). *)
353   let body = read_body reqd in
354   let prompt =
355     try
356       match Yojson.Safe.from_string body with
357       | `Assoc items ->
358           (match List.assoc_opt "prompt" items with
359            | Some (`String s) -> s
360            | _ ->
361                (match List.assoc_opt "task" items with
362                 | Some (`String s) -> s
363                 | _ -> ""))
364       | _ -> ""
365     with _ ->
366       Option.value (form_field body "prompt")
367         ~default:(Option.value (form_field body "task") ~default:"")
368   in
369   if String.trim prompt = "" then begin
370     json_response reqd
371       (`Assoc [ ("ok", `Bool false); ("error", `String "need prompt/task")
    ]);
372     ()
373   end else
374     let v = Session.agent_task session ~prompt () in
375     let text =
376       match v with
377       | Pricklypear_lisp.Ast.VStr s -> s
378       | _ -> Pricklypear_lisp.Reader.pp_value v
379     in
380     json_response reqd
381       (`Assoc [ ("ok", `Bool true); ("text", `String text) ]);
382     ()
383
384 let repl_llm (session : Session.t) (reqd : Httpun.Reqd.t) : unit =
385   (* Streaming LLM answer for repl-v2: multipart/mixed token stream.
386      Mirrors api_agent's streaming branch but emits hx-multipart parts
387      instead of SSE events. Eval/slash stay on the Nopales /repl-v2/send
388      route; this handles plain-text (LLM) input only. *)
389   let body = read_body reqd in
390   let prompt =
391     try
392       match Yojson.Safe.from_string body with
393       | `Assoc items ->
394           (match List.assoc_opt "q" items with
395            | Some (`String s) -> s
396            | _ ->
397                (match List.assoc_opt "prompt" items with
398                 | Some (`String s) -> s
399                 | _ -> ""))
400       | _ -> ""
401     with _ ->
402       Option.value (form_field body "q")
403         ~default:(Option.value (form_field body "prompt") ~default:"")
404   in
405   if String.trim prompt = "" then begin
406     json_response reqd
407       (`Assoc [ ("ok", `Bool false); ("error", `String "need prompt") ]);
408     ()
409   end else begin
410     let title_opt =
411       let p = agent_user_visible_text prompt in
412       if String.trim p = "" then None
413       else Some (if String.length p > 80 then String.sub p 0 80 ^ "?" else
    p)
414     in
415     let sid_v =
416       Session.agent_session_new session ~seed:default_http_agent_seed
417         ~tools:true ?title:title_opt ()
418     in
419     match sid_v with
420     | Pricklypear_lisp.Ast.VStr e
```

```
      when String.starts_with ~prefix:"agent-session-new error" e ->
        json_response reqd
          (`Assoc [ ("ok", `Bool false); ("error", `String e) ]);
        ()
    | Pricklypear_lisp.Ast.VStr sid ->
      (* Streaming multipart/mixed response: one part per token, routed
         into the live bubble by hx-multipart. *)
      let boundary = "pp" ^ random_hex_bytes 8 in
      let rid = "llm-" ^ sid in
      let headers =
        Httpun.Headers.of_list
          [ ("content-type", "multipart/mixed; boundary=" ^ boundary)
          ; ("cache-control", "no-cache")
          ; ("x-accel-buffering", "no")
          (* Chunked framing is MANDATORY for a streamed response (see
             api_agent SSE branch): close-delimited bodies on keep-alive
             connections never terminate on the wire. *)
          ; ("transfer-encoding", "chunked")
          ; ("hx-target", "#repl-transcript")
          ; ("hx-swap", "beforeend")
          ]
      in
      let resp = Httpun.Response.create ~headers `OK in
      let body_w =
        try
          Httpun.Reqd.respond_with_streaming
            ~flush_headers_immediately:true reqd resp
        with exn when is_closed_writer_exn exn ->
          ignore (Ops_log.info ~component:"mp"
                    "peer gone before stream start");
          raise exn
      in
      let queue : (string option * string) Queue.t = Queue.create () in
      let mutex = Eio.Mutex.create () in
      let cond = Eio.Condition.create () in
      let finished = ref false in
      let push t html =
        Eio.Mutex.lock mutex;
        Queue.add (t, html) queue;
        Eio.Condition.broadcast cond;
        Eio.Mutex.unlock mutex
      in
      let esc = Http_code.html_escape in
      (* Part 1: user echo bubble. *)
      push (Some "#repl-transcript")
        (Printf.sprintf
           "<div class=\"repl-line repl-user\"><span
      class=\"repl-kind\">user</span> <div
      class=\"repl-user\">%s</div></div>"
           (esc prompt));
      (* Part 2: llm bubble opener with empty inner div for tokens. *)
      push (Some "#repl-transcript")
        (Printf.sprintf
           "<div class=\"repl-line repl-llm\"><span
      class=\"repl-kind\">llm</span><div class=\"repl-llm\"
      id=\"%s\"></div></div>"
           rid);
      let on_event = function
        | Llm_client.Token tok ->
            push (Some ("#" ^ rid))
              (Printf.sprintf "<span>%s</span>" (esc tok))
        | Llm_client.Done ->
            finished := true;
            Eio.Mutex.lock mutex;
            Eio.Condition.broadcast cond;
            Eio.Mutex.unlock mutex
        | Llm_client.Fail e ->
            push (Some ("#" ^ rid))
              (Printf.sprintf "<span class=\"repl-error\">%s</span>"
                 (esc ("error: " ^ e)));
            finished := true;
            Eio.Mutex.lock mutex;
            Eio.Condition.broadcast cond;
```

```
            Eio.Mutex.unlock mutex
      in
      Eio.Switch.run @@ fun sw ->
      Eio.Fiber.fork ~sw (fun () ->
          let v =
            Session.agent_say session ~session_id:sid ~prompt ~max_steps:16
              ~on_event ()
          in
          if not !finished then begin
            (match v with
             | Pricklypear_lisp.Ast.VStr s when String.trim s <> "" ->
                 push (Some ("#" ^ rid))
                   (Printf.sprintf "<span>%s</span>" (esc s))
             | _ -> ());
            finished := true;
            Eio.Mutex.lock mutex;
            Eio.Condition.broadcast cond;
            Eio.Mutex.unlock mutex
          end);
      let rec drain () =
        Eio.Mutex.lock mutex;
        if not (Queue.is_empty queue) then begin
          let (t, html) = Queue.take queue in
          Eio.Mutex.unlock mutex;
          (match mp_write_part body_w ~boundary ?target:t html with
           | `Peer_gone ->
               finished := true;
               close_writer_quiet body_w
           | `Ok -> drain ())
        end else if !finished then begin
          Eio.Mutex.unlock mutex;
          mp_close body_w ~boundary
        end else begin
          Eio.Condition.await cond mutex;
          Eio.Mutex.unlock mutex;
          drain ()
        end
      in
      drain ()
    | other ->
        let e = Pricklypear_lisp.Reader.pp_value other in
        json_response reqd
          (`Assoc [ ("ok", `Bool false); ("error", `String e) ]);
        ()
  end

let agent_page (session : Session.t) (reqd : Httpun.Reqd.t) : unit =
  let target = (Httpun.Reqd.request reqd).Httpun.Request.target in
  let open_sid =
    match query_param target "session" with
    | Some s when String.trim s <> "" -> Some (String.trim s)
    | _ -> None
  in
  let ctx =
    Printf.sprintf "user=%s branch=%s"
      (Session.username session) (Session.branch_name session)
  in
  let default_seed = Session.default_agent_seed session in
  let sessions =
    Ai_store.list_sessions (Session.store session)
      ~owner:(Session.username session) ~limit:40 ()
  in
  let open_sess, history =
    match open_sid with
    | None -> (None, [])
    | Some id ->
        let s_opt =
          Ai_store.get_session (Session.store session)
            ~owner:(Session.username session) ~id
        in
        (match s_opt with
         | None -> (None, [])
         | Some s ->
```

```
            let pairs =
              Ai_store.history_summaries (Session.store session)
    ~session_id:id
            in
            (Some s, pairs))
  in
  let seed_value =
    match open_sess with
    | Some s when String.trim s.seed <> "" -> s.seed
    | _ -> default_seed
  in
  let tools_checked =
    match open_sess with Some s -> s.tools | None -> true
  in
  let sid_display =
    match open_sess with Some s -> s.id | None -> "(new on send)"
  in
  let history_nodes =
    List.map
      (fun (role, content) ->
        let role_label =
          match role with
          | "user" -> "you"
          | "assistant" -> "agent"
          | other -> other
        in
        let shown =
          if role = "user" then agent_user_visible_text content else content
        in
        let shown =
          if String.length shown > 4000 then
            String.sub shown 0 4000 ^ "\n?(truncated)"
          else shown
        in
        HTML.div
          [ HTML.class_ "card"; HTML.style_ "margin:.4rem 0" ]
          [ HTML.b [] [txt "%s" role_label]
          ; HTML.pre
              [ HTML.style_ "white-space:pre-wrap;margin:.35rem 0 0" ]
              [txt "%s" shown]
          ])
      history
  in
  let session_list_nodes =
    if sessions = [] then
      [ HTML.p [HTML.class_ "muted"] [txt "No chats yet -- send a
     message."] ]
    else
      List.map
        (fun (s : Ai_store.session) ->
          let active =
            match open_sess with
            | Some o when o.id = s.id -> true
            | _ -> false
          in
          let label = agent_session_label s in
          let href = "/agent?session=" ^ s.id in
          HTML.li
            [ HTML.style_
                (if active then "font-weight:700;margin:.25rem 0"
                 else "margin:.25rem 0")
            ]
            [ HTML.a [HTML.href "%s" href] [txt "%s" label]
            ; HTML.span [HTML.class_ "muted"]
                [txt "%s" (" ? " ^ s.updated_at)]
            ])
        sessions
  in
  let initial_sid_js =
    match open_sess with
    | Some s -> Yojson.Safe.to_string (`String s.id)
    | None -> "null"
  in
```

```
  let tools_input =
    if tools_checked then
      HTML.input
        [ HTML.type_ "checkbox"
        ; HTML.name "tools"
        ; HTML.id "tools"
        ; HTML.value "1"
        ; HTML.checked
        ]
    else
      HTML.input
        [ HTML.type_ "checkbox"
        ; HTML.name "tools"
        ; HTML.id "tools"
        ; HTML.value "1"
        ]
  in
  let js =
    "const log=document.getElementById('log');" ^
    "const msg=document.getElementById('msg');" ^
    "const send=document.getElementById('send');" ^
    "const neu=document.getElementById('newchat');" ^
    "const sidEl=document.getElementById('sid');" ^
    "const toolsEl=document.getElementById('tools');" ^
    "const seedEl=document.getElementById('seed');" ^
    "const baseCtx=" ^ Yojson.Safe.to_string (`String ctx) ^ ";" ^
    "let sid=" ^ initial_sid_js ^ ";" ^
    "function add(role,text){" ^
    "const d=document.createElement('div');" ^
    "d.className='card';" ^
    "d.style.margin='.4rem 0';" ^
    "d.innerHTML='<b>'+role+'</b><pre
    style=\"white-space:pre-wrap;margin:.35rem 0 0\">'+" ^
    "String(text).replace(/[&<>]/g,c=>({\"&\":\"&amp;\",\"<\":\"&lt;\",\">\":\"&gt;\"}[c]))+"
    ^
    "'</pre>';log.appendChild(d);log.scrollTop=log.scrollHeight;}" ^
    "function setSid(s){sid=s||null;sidEl.textContent=sid?sid:'(new on
    send)';" ^
    "if(sid){const u=new
    URL(window.location.href);u.searchParams.set('session',sid);" ^
    "history.replaceState({},'',u.pathname+u.search);}" ^
    "else{history.replaceState({},'','/agent');}}" ^
    "setSid(sid);" ^
    "neu.onclick=()=>{window.location.href='/agent';};" ^
    "send.onclick=async()=>{" ^
    "const q=msg.value.trim();if(!q)return;" ^
    "const seed=seedEl.value;" ^
    "const tools=toolsEl.checked;" ^
    "const prompt='=== IMAGE CONTEXT ===\\n'+" ^
    "'system: Pricklypear multi-user Lisp OS image (software)\\n'+" ^
    "baseCtx+'\\n=== END CONTEXT ===\\n\\nUser:\\n'+q;" ^
    "add('you',q);msg.value='';send.disabled=true;" ^
    "try{" ^
    "const body={seed,prompt,tools,stream:false};" ^
    "if(sid)body.session_id=sid;" ^
    "const r=await fetch('/api/agent',{method:'POST'," ^
    "headers:{'content-type':'application/json'}," ^
    "body:JSON.stringify(body)});" ^
    "const j=await r.json();" ^
    "if(j.session_id)setSid(j.session_id);" ^
    "add('agent',j.text||j.error||JSON.stringify(j));" ^
    "if(j.session_id){" ^
    "const href='/agent?session='+encodeURIComponent(j.session_id);" ^
    "if(!document.querySelector('#chat-list a[href=\"'+href+'\"]'))" ^
    " setTimeout(function(){window.location.href=href;},350);}" ^
    "}catch(e){add('error',String(e));}" ^
    "finally{send.disabled=false;msg.focus();}" ^
    "};" ^
    "msg.addEventListener('keydown',e=>{if(e.key==='Enter'&&!e.shiftKey){e.preventDefault();send.click();}});"
  in
  let body =
```

```
    chrome ~session ~title:"Agent" [
      HTML.div [HTML.class_ "card"] [
        HTML.h2 [] [txt "Chats"];
        HTML.p [HTML.class_ "muted"] [txt
          "Durable threads in Postgres. Open one to resume; New chat starts
     empty."];
        HTML.p [] [
          HTML.a [HTML.href "/agent"; HTML.class_ "btn"] [txt "New chat"];
        ];
        HTML.ul [HTML.id "chat-list"] session_list_nodes;
      ];
      HTML.div [HTML.class_ "card"] [
        HTML.h2 [] [txt "Conversation"];
        HTML.p [] [
          txt "session: ";
          HTML.code [HTML.id "sid"] [txt "%s" sid_display];
        ];
        HTML.p [] [
          HTML.label [HTML.for_ "tools"] [
            tools_input;
            txt " tools (inspect / install)"
          ];
        ];
        HTML.details [] [
          HTML.summary [] [txt "seed (system persona)"];
          HTML.textarea [HTML.name "seed"; HTML.id "seed"] "%s" seed_value;
        ];
        HTML.div
          [ HTML.id "log"
          ; HTML.style_
              "max-height:28rem;overflow:auto;border:1px solid
     #ccc;padding:.5rem;margin:.5rem 0"
          ]
          history_nodes;
        HTML.p [] [
          HTML.label [HTML.for_ "msg"] [txt "message"];
        ];
        HTML.textarea [HTML.name "msg"; HTML.id "msg"]
          "%s" "What branch am I on?";
        HTML.p [] [
          HTML.button [HTML.type_ "button"; HTML.id "send"] [txt "Send"];
          txt " ";
          HTML.button [HTML.type_ "button"; HTML.id "newchat"] [txt "New
     chat"];
        ];
      ];
      HTML.script [] "%s" js;
      HTML.div [HTML.class_ "card"] [
        HTML.h2 [] [txt "Write a function"];
        HTML.p [HTML.class_ "muted"] [txt
          "One-shot authoring (not chat). Same as (ai/write name spec)."];
        HTML.p [] [
          HTML.label [HTML.for_ "fn_name"] [txt "name "];
          HTML.input [HTML.type_ "text"; HTML.name "fn_name"; HTML.id
     "fn_name";
                      HTML.value "demo/double"; HTML.placeholder
     "pkg/name"];
        ];
        HTML.p [] [
          HTML.label [HTML.for_ "fn_spec"] [txt "spec"];
        ];
        HTML.textarea [HTML.name "fn_spec"; HTML.id "fn_spec"]
          "%s" "A pure lambda of one int x that returns (* x 2).";
        HTML.p [] [
          HTML.button [HTML.type_ "button"; HTML.id "write_fn"] [txt
     "Generate & install"];
        ];
        HTML.pre [HTML.id "write_out"; HTML.class_ "card"] [txt ""];
      ];
      HTML.script [] "%s" (
        "const wout=document.getElementById('write_out');" ^
        "const wbtn=document.getElementById('write_fn');" ^
        "wbtn.onclick=async()=>{" ^
```

```
        "const name=document.getElementById('fn_name').value.trim();" ^
        "const spec=document.getElementById('fn_spec').value;" ^
        "wout.textContent='?'; wbtn.disabled=true;" ^
        "try{" ^
        "const r=await fetch('/api/ai/propose',{method:'POST'," ^
        "headers:{'content-type':'application/json'}," ^
        "body:JSON.stringify({name,spec})});" ^
        "const j=await r.json();" ^
        "wout.textContent=j.result||j.error||JSON.stringify(j);" ^
        "}catch(e){wout.textContent=String(e);}" ^
        "finally{wbtn.disabled=false;}" ^
        "};"
      );
    ]
  in
  html_response reqd ~session body;
  ()

let ai_propose (session : Session.t) (reqd : Httpun.Reqd.t) : unit =
  let body = read_body reqd in
  let name_opt, spec_opt =
    try
      match Yojson.Safe.from_string body with
      | `Assoc items ->
          let name =
            match List.assoc_opt "name" items with
            | Some (`String s) -> Some s
            | _ -> None
          in
          let spec =
            match List.assoc_opt "spec" items with
            | Some (`String s) -> Some s
            | _ -> None
          in
          (name, spec)
      | _ -> (None, None)
    with _ ->
      let pkg = Option.value (form_field body "package") ~default:"" in
      let local = Option.value (form_field body "local") ~default:"" in
      let name =
        match form_field body "name" with
        | Some n when String.trim n <> "" -> Some (String.trim n)
        | _ ->
            let j = Packages.join_name ~package:pkg ~local in
            if j = "" then None else Some j
      in
      (name, form_field body "spec")
  in
  match name_opt, spec_opt with
  | Some name, Some spec when name <> "" && spec <> "" ->
      let v = Session.propose_function session ~name ~spec in
      let rendered = Pricklypear_lisp.Reader.pp_value v in
      let wants_html =
        match Httpun.Headers.get (Httpun.Reqd.request reqd).headers
    "content-type" with
        | Some ct when String.length ct >= 33
            && String.sub ct 0 33 = "application/x-www-form-urlencoded" ->
    true
        | _ ->
            (match form_field body "name" with Some _ -> true | None ->
    false)
      in
      if wants_html then
        let body_node =
          chrome ~session ~title:("AI propose " ^ name) [
            HTML.div [HTML.class_ "card"] [
              HTML.pre [] [txt "%s" rendered];
              HTML.p [] [
                HTML.a [HTML.href "%s" ("/edit/" ^ name)] [txt "open
    editor"];
                txt " ? ";
                HTML.a [HTML.href "%s" (Packages.inspect_href name)] [txt
    "inspect"];
```

```
              txt " ? ";
              HTML.a [HTML.href "/bindings"] [txt "functions"];
            ];
          ];
        ]
      in
      html_response reqd ~session body_node;
      ()
    else begin
      json_response reqd
        (`Assoc [ ("ok", `Bool true)
                ; ("result", `String rendered)
                ; ("name", `String name) ]);
      ()
    end
| _ ->
    json_response reqd ~status:`Bad_request
      (`Assoc [("ok", `Bool false); ("error", `String "name and spec
  required")]);
    ()
```

72 image/lib/http_analyze.ml

```
(** HTTP surface for on-demand function analysis (htmx fragments). *)
open Pure_html
open Http_core

let fragment_of_json (j : Yojson.Safe.t) : node =
  let open Yojson.Safe.Util in
  let s key =
    match member key j with
    | `String x -> x
    | `Null -> ""
    | other -> Yojson.Safe.to_string other
  in
  let label =
    match s "label" with "" -> s "source" | l -> l
  in
  HTML.div [ HTML.class_ "card analyze-result" ]
    [
      HTML.p []
        [
          HTML.span [ HTML.class_ "badge complexity" ] [ txt "%s" (s
    "complexity") ];
          txt " ";
          HTML.span [ HTML.class_ "muted" ] [ txt "%s" label ];
          (match s "model" with
           | "" -> txt ""
           | m -> HTML.span [ HTML.class_ "muted" ] [ txt " ? %s" m ]);
        ];
      HTML.p [] [ txt "confidence: %s" (s "confidence") ];
      HTML.p [] [ txt "%s" (s "rationale") ];
      (match s "notes" with
       | "" -> txt ""
       | n -> HTML.p [ HTML.class_ "muted" ] [ txt "%s" n ]);
    ]

let fragment_error (msg : string) : node =
  HTML.div [ HTML.class_ "card analyze-result" ]
    [ HTML.p [ HTML.class_ "err" ] [ txt "%s" msg ] ]

let post_analyze (session : Session.t) (name : string)
    (reqd : Httpun.Reqd.t) : unit =
  let store = Session.store session in
  let man = Session.manifest session in
  let author = man.username in
  let user_id =
    if man.user_id = "" || man.user_id = "anonymous" || man.user_id =
     "public"
    then None
    else Some man.user_id
  in
  let force_llm =
    match Httpun.Reqd.request reqd with
    | { Httpun.Request.target; _ } ->
        (try
           let q =
             match String.split_on_char '?' target with
             | _ :: qs -> String.concat "?" qs
             | _ -> ""
           in
           List.exists
             (fun p -> p = "llm=1" || p = "force=1")
             (String.split_on_char '&' q)
         with _ -> false)
  in
  let node =
    match
      Analyze.analyze_function store ~author ~user_id ~fn_name:name
        ~force_llm
    with
    | Ok j -> fragment_of_json j
    | Error e -> fragment_error e
  in
```

```
  fragment_response reqd (Pure_html.to_string node)

let post_explain (session : Session.t) (name : string)
    (reqd : Httpun.Reqd.t) : unit =
  let store = Session.store session in
  let node =
    match Analyze.explain_function store ~fn_name:name with
    | Ok text ->
        HTML.div [ HTML.class_ "card explain-result" ]
          [ HTML.pre [ HTML.class_ "bb-source" ] [ txt "%s" text ] ]
    | Error e -> fragment_error e
  in
  fragment_response reqd (Pure_html.to_string node)
```

73 image/lib/http_branches.ml

```
(** Branch browser UI: list, fork, checkout, promote. *)
open Pure_html
open Http_core

let branch_ref_of (b : Pricklypear_common.Branch.t) : string =
  b.Pricklypear_common.Branch.owner ^ "/" ^ b.Pricklypear_common.Branch.name

let branch_label (b : Pricklypear_common.Branch.t) : string =
  let r = branch_ref_of b in
  if b.owner = "system" && b.name = "main" then r ^ " (defaults)"
  else if b.name = "personal" then r ^ " (garden)"
  else r

let id_to_ref_map (branches : Pricklypear_common.Branch.t list)
  : (string, string) Hashtbl.t =
  let h = Hashtbl.create 16 in
  List.iter
    (fun (b : Pricklypear_common.Branch.t) ->
      Hashtbl.replace h b.id (branch_ref_of b))
    branches;
  h

let parent_ref_of (idmap : (string, string) Hashtbl.t)
    (b : Pricklypear_common.Branch.t) : string =
  match b.parent_id with
  | None -> "--"
  | Some pid ->
      (match Hashtbl.find_opt idmap pid with
       | Some r -> r
       | None -> String.sub pid 0 (min 8 (String.length pid)))

let short_ts (s : string) : string =
  (* "2026-07-12 03:04:05.123+00" ? first 16 chars-ish *)
  if String.length s <= 16 then s else String.sub s 0 16

let branches_flash (msg : string) : Pure_html.node list =
  if String.trim msg = "" then []
  else [ HTML.p [HTML.class_ "ok"] [txt "%s" msg] ]

let branches_err (msg : string) : Pure_html.node list =
  if String.trim msg = "" then []
  else [ HTML.p [HTML.class_ "err"] [txt "%s" msg] ]

let checkout_form ~(ref_ : string) ~(current : bool) : Pure_html.node =
  if current then
    HTML.span [HTML.class_ "muted"] [txt "current"]
  else
    HTML.form
      [ HTML.method_ `POST
      ; HTML.action "/branches/checkout"
      ; HTML.enctype `urlencoded
      ; HTML.class_ "inline-form"
      ]
      [ HTML.input [HTML.type_ "hidden"; HTML.name "ref"; HTML.value "%s"
    ref_]
      ; HTML.button [HTML.type_ "submit"; HTML.class_ "ol-btn"] [txt
    "Checkout"]
      ]

let list_branches_page (session : Session.t) (reqd : Httpun.Reqd.t)
    ?(msg = "") ?(err = "") () : unit =
  let store = Session.store session in
  let current = Session.branch_name session in
  let branches = Store.list_branches store in
  let idmap = id_to_ref_map branches in
  let rows =
    List.map
      (fun (b : Pricklypear_common.Branch.t) ->
        let ref_ = branch_ref_of b in
        let is_cur = ref_ = current || (current = "main" && ref_ =
    "system/main") in
```

```
      let n_fn = Store.count_functions_on_branch store ~branch_id:b.id in
      let n_rt = Store.count_routes_on_branch store ~branch_id:b.id in
      (* branching.branch-status: content-based dirty/synced counts
         (replaces the name-presence overrides count). *)
      let st = Store.branch_status store ~branch_id:b.id in
      let is_dirty (r : Store.branch_status_row) =
        match r.Store.st_status with
        | Store.SAheadNew | Store.SAhead | Store.SDiverged -> true
        | _ -> false
      in
      let n_dirty = List.length (List.filter is_dirty st) in
      let n_synced =
        List.length
          (List.filter
             (fun (r : Store.branch_status_row) ->
               r.Store.st_status = Store.SSynced)
             st)
      in
      let star = if is_cur then "?" else "" in
      (HTML.tr
           (if is_cur then [HTML.class_ "br-current"] else [])
           [ HTML.td [] [txt "%s" star]
           ; HTML.td []
               [ HTML.a [HTML.href "%s" ("/branches?ref=" ^ ref_)]
                   [txt "%s" (branch_label b)] ]
           ; HTML.td [] [txt "%s" (parent_ref_of idmap b)]
           ; HTML.td [] [txt "%d" n_fn]
           ; HTML.td [] [txt "%d" n_dirty]
           ; HTML.td [HTML.class_ "muted"] [txt "%d" n_synced]
           ; HTML.td [] [txt "%d" n_rt]
           ; HTML.td [HTML.class_ "muted"] [txt "%s" (short_ts
   b.created_at)]
           ; HTML.td []
               [ HTML.a
                   [ HTML.href "%s" ("/branches?ref=" ^ ref_)
                   ; HTML.class_ "btn secondary" ]
                   [txt "Open"]
               ; txt " "
               ; checkout_form ~ref_ ~current:is_cur
               ]
           ]))
    branches
in
let fork_options =
  List.map
    (fun (b : Pricklypear_common.Branch.t) ->
      let ref_ = branch_ref_of b in
      HTML.option [HTML.value "%s" ref_] "%s" ref_)
    branches
in
let body =
  chrome ~session ~title:"Branches"
    (branches_flash msg @ branches_err err @
     [ HTML.p [HTML.class_ "muted"]
         [ txt "On: "
         ; HTML.strong [] [txt "%s" current]
         ; txt " -- code branches; data_rows are shared."
         ]
     ; (match branches with
        | [] -> HTML.p [] [txt "(no branches in store)"]
        | _ ->
            HTML.div [HTML.class_ "br-table-wrap"]
              [ HTML.table [HTML.class_ "br-table"]
                  [ HTML.thead []
                      [ HTML.tr []
                          [ HTML.th [] [txt ""]
                          ; HTML.th [] [txt "branch"]
                          ; HTML.th [] [txt "parent"]
                          ; HTML.th [] [txt "own code"]
                          ; HTML.th [] [txt "dirty"]
                          ; HTML.th [] [txt "synced"]
                          ; HTML.th [] [txt "routes"]
                          ; HTML.th [] [txt "created"]
```

```
                            ; HTML.th [] [txt "actions"]
                            ]
                        ]
                    ; HTML.tbody [] rows
                    ]
                ])
       ; HTML.div [HTML.class_ "card"]
           [ HTML.h2 [] [txt "Create branch (full fork)"]
           ; HTML.p [HTML.class_ "muted"]
               [txt "Copies code from parent. Garden (personal) stays an
    empty inherit child."]
           ; HTML.form
               [ HTML.method_ 'POST
               ; HTML.action "/branches/fork"
               ; HTML.enctype 'urlencoded
               ]
               [ HTML.p []
                   [ HTML.label [] [txt "From "]
                   ; HTML.select [HTML.name "from"] fork_options
                   ]
               ; HTML.p []
                   [ HTML.label [] [txt "New name "]
                   ; HTML.input
                       [ HTML.type_ "text"
                       ; HTML.name "name"
                       ; HTML.placeholder "experiment"
                       ; HTML.required
                       ]
                   ]
               ; HTML.p []
                   [ HTML.button [HTML.type_ "submit"] [txt "Fork"] ]
               ]
           ]
       ])
  in
  html_response reqd ~session body;
  ()

let branch_detail_page (session : Session.t) (reqd : Httpun.Reqd.t)
    ~(ref_ : string) ?(msg = "") ?(err = "") () : unit =
  let store = Session.store session in
  let current = Session.branch_name session in
  let owner, name =
    match String.split_on_char '/' ref_ with
    | [o; n] -> o, n
    | [n] -> "system", n
    | _ -> "system", ref_
  in
  let found = Store.lookup_branch_id_by_name store ~owner ~name in
  match found with
  | None ->
      list_branches_page session reqd ~err:("no such branch: " ^ ref_) ()
  | Some bid ->
      let branches = Store.list_branches store in
      let idmap = id_to_ref_map branches in
      let b_opt =
        List.find_opt
          (fun (b : Pricklypear_common.Branch.t) -> b.id = bid)
          branches
      in
      let parent_id =
        match b_opt with Some b -> b.parent_id | None -> None
      in
      let parent_ref =
        match b_opt with
        | Some b -> parent_ref_of idmap b
        | None -> "--"
      in
      let is_cur =
        ref_ = current
        || (owner ^ "/" ^ name) = current
      in
      let n_fn = Store.count_functions_on_branch store ~branch_id:bid in
```

```
      let n_rt = Store.count_routes_on_branch store ~branch_id:bid in
      (* branching.branch-status: git-style status per name (replaces the
         new/override/main-newer name-presence guess). *)
      let own = Store.list_own_functions store ~branch_id:bid in
      let routes = Store.list_own_routes store ~branch_id:bid in
      let status = Store.branch_status store ~branch_id:bid in
      let st_of kind name =
        List.find_opt
          (fun (r : Store.branch_status_row) ->
            r.Store.st_kind = kind && r.Store.st_name = name)
          status
      in
      let is_dirty (r : Store.branch_status_row) =
        match r.Store.st_status with
        | Store.SAheadNew | Store.SAhead | Store.SDiverged -> true
        | _ -> false
      in
      let n_dirty = List.length (List.filter is_dirty status) in
      let n_synced =
        List.length
          (List.filter
             (fun (r : Store.branch_status_row) ->
               r.Store.st_status = Store.SSynced)
             status)
      in
      let st_cls = function
        | Store.SClean -> "muted"
        | Store.SAheadNew | Store.SAhead -> "ok"
        | Store.SBehind -> "warn"
        | Store.SDiverged -> "err"
        | Store.SSynced -> "muted"
      in
      let sorted_own =
        List.sort (fun (a, _, _) (b, _, _) -> String.compare a b) own
      in
      let override_rows =
        List.map
          (fun (fname, _author, upd) ->
            let status_s, status_c =
              match st_of Store.DiffFn fname with
              | Some r ->
                  (Store.st_status_s r.Store.st_status,
                   st_cls r.Store.st_status)
              | None -> ("own", "muted")
            in
            let unbind_hint =
              match st_of Store.DiffFn fname with
              | Some { Store.st_status = Store.SSynced; _ } ->
                  "Pin identical to parent -- safe to unbind"
              | _ -> "Delete pin; inherit parent"
            in
            HTML.tr []
              [ HTML.td []
                  [ HTML.a [HTML.href "%s" ("/binding/" ^ fname)] [txt "%s"
    fname] ]
              ; HTML.td [HTML.class_ "%s" status_c] [txt "%s" status_s]
              ; HTML.td [HTML.class_ "muted"] [txt "%s" (short_ts upd)]
              ; HTML.td []
                  [ HTML.a
                      [ HTML.href "%s"
                          ("/branches?ref=" ^ ref_ ^ "&diff=" ^ fname)
                      ]
                      [txt "diff"]
                  ; txt " "
                  ; HTML.form
                      [ HTML.method_ `POST
                      ; HTML.action "/branches/unoverride"
                      ; HTML.enctype `urlencoded
                      ; HTML.class_ "inline-form"
                      ]
                      [ HTML.input
                          [ HTML.type_ "hidden"
                          ; HTML.name "ref"
```

```
                        ; HTML.value "%s" ref_
                        ]
                    ; HTML.input
                        [ HTML.type_ "hidden"
                        ; HTML.name "name"
                        ; HTML.value "%s" fname
                        ]
                    ; HTML.button
                        [ HTML.type_ "submit"
                        ; HTML.class_ "ol-btn"
                        ; HTML.title_ "%s" unbind_hint
                        ]
                        [txt "use parent"]
                    ]
                ]
            ])
        sorted_own
    in
    let own_rows =
      List.map
        (fun (fname, author, upd) ->
          HTML.tr []
            [ HTML.td []
                [ HTML.a [HTML.href "%s" ("/binding/" ^ fname)] [txt "%s"
fname] ]
            ; HTML.td [] [txt "%s" author]
            ; HTML.td [HTML.class_ "muted"] [txt "%s" (short_ts upd)]
            ])
        own
    in
    let route_rows =
      List.map
        (fun (route, handler) ->
          HTML.tr []
            [ HTML.td [] [HTML.code [] [txt "%s" route]]
            ; HTML.td []
                [ HTML.a [HTML.href "%s" ("/binding/" ^ handler)]
                    [txt "%s" handler] ]
            ])
        routes
    in
    let promote_checks =
      List.map
        (fun (fname, _, _) ->
          HTML.p []
            [ HTML.label []
                [ HTML.input
                    [ HTML.type_ "checkbox"
                    ; HTML.name "name"
                    ; HTML.value "%s" fname
                    ]
                ; txt " %s" fname
                ]
            ])
        sorted_own
    in
    (* optional diff *)
    let target = reqd |> Httpun.Reqd.request |> fun r ->
r.Httpun.Request.target in
    let diff_name = query_param target "diff" in
    let diff_block =
      match diff_name, parent_id with
      | None, _ | Some "", _ -> []
      | Some dname, None ->
          let child =
            Store.lookup_function_on_branch store ~branch_id:bid
~name:dname
          in
          [ HTML.div [HTML.class_ "card"]
                [ HTML.h2 [] [txt "Diff: %s (no parent)" dname]
                ; (match child with
                   | None -> HTML.p [] [txt "(missing)"]
                   | Some (e, _, _, _) ->
```

```
                        HTML.pre []
                          [txt "%s" (Pricklypear_lisp.Reader.pp_expr e)])
                ]
            ]
      | Some dname, Some pid ->
          let child =
            Store.lookup_function_on_branch store ~branch_id:bid
    ~name:dname
          in
          let parent =
            Store.lookup_function_on_branch store ~branch_id:pid
    ~name:dname
          in
          let show = function
            | None -> "(absent)"
            | Some (e, _, _, _) -> Pricklypear_lisp.Reader.pp_expr e
          in
          [ HTML.div [HTML.class_ "card"]
                [ HTML.h2 [] [txt "Diff: %s" dname]
                ; HTML.div [HTML.class_ "br-diff"]
                    [ HTML.div []
                        [ HTML.h3 [] [txt "parent"]
                        ; HTML.pre [] [txt "%s" (show parent)]
                        ]
                    ; HTML.div []
                        [ HTML.h3 [] [txt "this branch"]
                        ; HTML.pre [] [txt "%s" (show child)]
                        ]
                    ]
                ]
            ]
     in
     let body =
       chrome ~session ~title:("Branch " ^ ref_)
         (branches_flash msg @ branches_err err @
          [ HTML.p []
              [ HTML.a [HTML.href "/branches"] [txt "? all branches"]
              ; txt " ? parent: "
              ; (if parent_ref = "--" then txt "--"
                 else
                   HTML.a [HTML.href "%s" ("/branches?ref=" ^ parent_ref)]
                     [txt "%s" parent_ref])
              ; txt " ? "
              ; checkout_form ~ref_ ~current:is_cur
              ]
          ; HTML.div [HTML.class_ "br-stats"]
              [ HTML.span [HTML.class_ "card br-stat"]
                  [txt "own functions: %d" n_fn]
              ; HTML.span [HTML.class_ "card br-stat"]
                  [txt "dirty: %d" n_dirty]
              ; HTML.span [HTML.class_ "card br-stat"]
                  [txt "synced pins: %d" n_synced]
              ; HTML.span [HTML.class_ "card br-stat"]
                  [txt "own routes: %d" n_rt]
              ]
          ; HTML.div [HTML.class_ "card"]
              [ HTML.h2 [] [txt "Overrides / own pins"]
              ; HTML.p [HTML.class_ "muted"]
                  [txt "Rows on this branch. ?use parent? deletes the pin
    (unoverride)."]
              ; (if override_rows = [] then
                   HTML.p [HTML.class_ "muted"] [txt "(none -- fully
    inherited)"]
                 else
                   HTML.table [HTML.class_ "br-table"]
                     [ HTML.thead []
                         [ HTML.tr []
                             [ HTML.th [] [txt "name"]
                             ; HTML.th [] [txt "kind"]
                             ; HTML.th [] [txt "updated"]
                             ; HTML.th [] [txt "actions"]
                             ]
                         ]
```

```
                    ; HTML.tbody [] override_rows
                    ])
                ]
            ; HTML.div [HTML.class_ "card"]
                [ HTML.h2 [] [txt "Changes (activity)"]
                ; HTML.p [HTML.class_ "muted"]
                    [txt "Own function_defs on this branch, newest first. "]
                ; HTML.p []
                    [ HTML.a [HTML.href "%s" ("/changes?ref=" ^ ref_)]
                        [txt "Open /changes"] ]
                ; (if own_rows = [] then
                     HTML.p [HTML.class_ "muted"] [txt "(empty branch)"]
                   else
                     let sorted =
                       List.sort
                         (fun (_, _, a) (_, _, b) -> String.compare b a)
                         own
                     in
                     let change_rows =
                       List.map
                         (fun (fname, author, upd) ->
                           let kind =
                             match st_of Store.DiffFn fname with
                             | Some r -> Store.st_status_s r.Store.st_status
                             | None -> "own"
                           in
                           HTML.tr []
                             [ HTML.td [HTML.class_ "muted"]
                                 [txt "%s" (short_ts upd)]
                             ; HTML.td []
                                 [ HTML.a
                                     [HTML.href "%s" ("/binding/" ^ fname)]
                                     [txt "%s" fname] ]
                             ; HTML.td [] [txt "%s" kind]
                             ; HTML.td [] [txt "%s" author]
                             ; HTML.td []
                                 [ HTML.a
                                     [ HTML.href "%s"
                                         ("/branches?ref=" ^ ref_
                                          ^ "&diff=" ^ fname) ]
                                     [txt "diff"]
                                 ; txt " "
                                 ; HTML.a
                                     [HTML.href "%s"
                                        (Packages.edit_href fname)]
                                     [txt "edit"]
                                 ]
                             ])
                         sorted
                     in
                     HTML.table [HTML.class_ "br-table"]
                       [ HTML.thead []
                           [ HTML.tr []
                               [ HTML.th [] [txt "when"]
                               ; HTML.th [] [txt "name"]
                               ; HTML.th [] [txt "class"]
                               ; HTML.th [] [txt "author"]
                               ; HTML.th [] [txt "actions"]
                               ]
                           ]
                       ; HTML.tbody [] change_rows
                       ])
                ]
            ; HTML.div [HTML.class_ "card"]
                [ HTML.h2 [] [txt "All own code"]
                ; (if own_rows = [] then
                     HTML.p [HTML.class_ "muted"] [txt "(empty branch)"]
                   else
                     HTML.table [HTML.class_ "br-table"]
                       [ HTML.thead []
                           [ HTML.tr []
                               [ HTML.th [] [txt "name"]
                               ; HTML.th [] [txt "author"]
```

```
                            ; HTML.th [] [txt "updated"]
                            ]
                        ]
                    ; HTML.tbody [] own_rows
                    ])
            ]
        ; HTML.div [HTML.class_ "card"]
            [ HTML.h2 [] [txt "Own routes"]
            ; (if route_rows = [] then
                 HTML.p [HTML.class_ "muted"] [txt "(none on this
    branch)"]
               else
                 HTML.table [HTML.class_ "br-table"]
                   [ HTML.thead []
                       [ HTML.tr []
                           [ HTML.th [] [txt "route"]
                           ; HTML.th [] [txt "handler"]
                           ]
                       ]
                   ; HTML.tbody [] route_rows
                   ])
            ]
        ; HTML.div [HTML.class_ "card"]
            [ HTML.h2 [] [txt "Promote to system/main"]
            ; HTML.p [HTML.class_ "muted"]
                [txt "Select pins to merge into live defaults. Also
    pp_sync pull + git push for durable libs/*.pp."]
            ; (let soft =
                 Session.promote_soft_warnings session
                   ~names:(List.map (fun (n, _, _) -> n) sorted_own)
               in
               if soft = "" then
                 HTML.p [HTML.class_ "muted"]
                   [txt "%s"
                      "Soft lint: load-library spec, then open /spec"]
               else
                 HTML.p [HTML.class_ "spec-soft"]
                   [txt "Soft lint (non-blocking): %s" soft])
            ; (if promote_checks = [] then
                 HTML.p [HTML.class_ "muted"] [txt "(nothing to
    promote)"]
               else
                 HTML.form
                   [ HTML.method_ `POST
                   ; HTML.action "/branches/promote"
                   ; HTML.enctype `urlencoded
                   ]
                   (HTML.input
                      [ HTML.type_ "hidden"
                      ; HTML.name "ref"
                      ; HTML.value "%s" ref_
                      ]
                    :: promote_checks
                    @ [ HTML.p []
                          [ HTML.button [HTML.type_ "submit"]
                              [txt "Merge selected ? system/main"] ]
                      ]))
            ]
        ]
        @ diff_block)
     in
     html_response reqd ~session body;
     ()

let branches_page (session : Session.t) (reqd : Httpun.Reqd.t) : unit =
  let target = (Httpun.Reqd.request reqd).target in
  match query_param target "ref" with
  | Some ref_ when String.trim ref_ <> "" ->
      branch_detail_page session reqd ~ref_:(String.trim ref_) ()
  | _ -> list_branches_page session reqd ()

(** GET /changes[?ref=owner/name] -- branch change set
    (branching.change-set). *)
```

```
let changes_page (session : Session.t) (reqd : Httpun.Reqd.t) : unit =
  let target = (Httpun.Reqd.request reqd).target in
  let ref_ =
    match query_param target "ref" with
    | Some r when String.trim r <> "" -> String.trim r
    | _ -> Session.branch_name session
  in
  let owner, name =
    match String.split_on_char '/' ref_ with
    | [o; n] -> (o, n)
    | _ -> (Session.username session, ref_)
  in
  let store = Session.store session in
  let bid_opt = Store.lookup_branch_id_by_name store ~owner ~name in
  match bid_opt with
  | None ->
      html_response reqd ~session ~status:`Not_found
        (chrome ~session ~title:"Changes"
           [ HTML.p [HTML.class_ "err"] [txt "unknown branch %s" ref_] ]);
      ()
  | Some bid ->
      let own = Store.list_own_functions store ~branch_id:bid in
      let routes = Store.list_own_routes store ~branch_id:bid in
      (* branching.branch-status: class is the git-style status, not a
         name-presence guess. *)
      let status = Store.branch_status store ~branch_id:bid in
      let st_of kind name =
        List.find_opt
          (fun (r : Store.branch_status_row) ->
            r.Store.st_kind = kind && r.Store.st_name = name)
          status
      in
      let sorted =
        List.sort (fun (_, _, a) (_, _, b) -> String.compare b a) own
      in
      let rows =
        List.map
          (fun (fname, author, upd) ->
            let kind =
              match st_of Store.DiffFn fname with
              | Some r -> Store.st_status_s r.Store.st_status
              | None -> "own"
            in
            HTML.tr []
              [ HTML.td [HTML.class_ "muted"] [txt "%s" (short_ts upd)]
              ; HTML.td []
                  [ HTML.a [HTML.href "%s" ("/binding/" ^ fname)]
                      [txt "%s" fname] ]
              ; HTML.td [] [txt "%s" kind]
              ; HTML.td [] [txt "%s" author]
              ; HTML.td []
                  [ HTML.a
                      [HTML.href "%s" (Packages.edit_href fname)]
                      [txt "edit"]
                  ; txt " "
                  ; HTML.a
                      [ HTML.href "%s"
                          ("/branches?ref=" ^ ref_ ^ "&diff=" ^ fname) ]
                      [txt "diff"]
                  ]
              ])
          sorted
      in
      let route_rows =
        List.map
          (fun (path, handler) ->
            HTML.tr []
              [ HTML.td [] [HTML.code [] [txt "%s" path]]
              ; HTML.td [] [txt "%s" handler]
              ; HTML.td [] [txt "route"]
              ])
          routes
      in
```

```
      let body =
        chrome ~session ~title:("Changes ? " ^ ref_)
          [ HTML.p []
              [ HTML.a [HTML.href "/branches"] [txt "Branches"]
              ; txt " ? "
              ; HTML.a [HTML.href "%s" ("/branches?ref=" ^ ref_)]
                  [txt "%s" ref_]
              ; txt " ? change set"
              ]
          ; HTML.div [HTML.class_ "card"]
              [ HTML.h2 [] [txt "Functions"]
              ; (if rows = [] then
                   HTML.p [HTML.class_ "muted"] [txt "(no own functions)"]
                 else
                   HTML.table [HTML.class_ "br-table"]
                     [ HTML.thead []
                         [ HTML.tr []
                             [ HTML.th [] [txt "when"]
                             ; HTML.th [] [txt "name"]
                             ; HTML.th [] [txt "class"]
                             ; HTML.th [] [txt "author"]
                             ; HTML.th [] [txt "actions"]
                             ]
                         ]
                     ; HTML.tbody [] rows
                     ])
              ]
          ; HTML.div [HTML.class_ "card"]
              [ HTML.h2 [] [txt "Own routes"]
              ; (if route_rows = [] then
                   HTML.p [HTML.class_ "muted"] [txt "(none)"]
                 else
                   HTML.table [HTML.class_ "br-table"]
                     [ HTML.thead []
                         [ HTML.tr []
                             [ HTML.th [] [txt "route"]
                             ; HTML.th [] [txt "handler"]
                             ; HTML.th [] [txt "class"]
                             ]
                         ]
                     ; HTML.tbody [] route_rows
                     ])
              ]
          ]
      in
      html_response reqd ~session body;
      ()

let branches_checkout (session : Session.t) (reqd : Httpun.Reqd.t) : unit =
  let body = read_body reqd in
  let ref_ = Option.value (form_field body "ref") ~default:"" |>
     String.trim in
  if ref_ = "" then list_branches_page session reqd ~err:"ref required" ()
  else
    let r = Session.checkout_ref session ref_ in
    match r with
    | Error e -> list_branches_page session reqd ~err:e ()
    | Ok () ->
        redirect_response reqd ("/branches?ref=" ^ ref_);
        ()

let branches_fork (session : Session.t) (reqd : Httpun.Reqd.t) : unit =
  let body = read_body reqd in
  let from_ = Option.value (form_field body "from") ~default:"system/main"
     in
  let name = Option.value (form_field body "name") ~default:"" |>
     String.trim in
  if name = "" then list_branches_page session reqd ~err:"name required" ()
  else
    let owner = Session.username session in
    let child_ref =
      if String.contains name '/' then name else owner ^ "/" ^ name
    in
```

```
    let r = Session.fork_ref session ~parent_ref:from_ ~child_ref in
    match r with
    | Error e -> list_branches_page session reqd ~err:e ()
    | Ok _ ->
        list_branches_page session reqd
          ~msg:("forked " ^ child_ref ^ " from " ^ from_) ()

let branches_unoverride (session : Session.t) (reqd : Httpun.Reqd.t)
  : unit =
  let body = read_body reqd in
  let ref_ = Option.value (form_field body "ref") ~default:"" |>
     String.trim in
  let name = Option.value (form_field body "name") ~default:"" |>
     String.trim in
  (* Checkout target branch first if needed so delete hits the right id. *)
  let _ =
    if ref_ <> "" && Session.branch_name session <> ref_ then
      Session.checkout_ref session ref_
    else (Ok ())
  in
  let r = Session.unoverride session name in
  match r with
  | Error e ->
      branch_detail_page session reqd ~ref_ ~err:e ()
  | Ok true ->
      branch_detail_page session reqd ~ref_
        ~msg:("unoverrode " ^ name ^ " -- now inherits parent") ()
  | Ok false ->
      branch_detail_page session reqd ~ref_
        ~err:("no pin named " ^ name ^ " on this branch") ()

let branches_promote (session : Session.t) (reqd : Httpun.Reqd.t) : unit =
  let body = read_body reqd in
  let ref_ = Option.value (form_field body "ref") ~default:"" |>
     String.trim in
  let form = parse_form_body body in
  let names =
    List.filter_map
      (fun (k, v) -> if k = "name" && String.trim v <> "" then Some v else
     None)
      form
  in
  let r = Session.promote_to_main session ~source_ref:ref_ ~names in
  match r with
  | Error e -> branch_detail_page session reqd ~ref_ ~err:e ()
  | Ok (n, soft) ->
      let msg =
        if soft = "" then
          Printf.sprintf "promoted %d function(s) ? system/main" n
        else
          Printf.sprintf
            "promoted %d function(s) ? system/main ? soft: %s" n soft
      in
      branch_detail_page session reqd ~ref_ ~msg ()


(* ---- Branch diff page (branching.diff / lisp-interpreter.ast-diff) ----
     *)

module Ast = Pricklypear_lisp.Ast
module Reader = Pricklypear_lisp.Reader
module AD = Pricklypear_lisp.Ast_diff

let is_absent (e : Ast.expr) : bool =
  match e with Ast.Lit Ast.VNil -> true | _ -> false

(** Structured AST delta renderer. Renders the [AD.delta] tree: unchanged
    ([Same]) subtrees collapse to a dim marker; a [Node] shows its ctor and
    only the children that differ; a [Replaced] shows the from/to values
    with the absent sentinel side signifying pure add/remove. *)
let rec render_delta (d : AD.delta) : Pure_html.node =
  match d with
  | AD.Same -> HTML.span [HTML.class_ "diff-same"] [txt "???"]
```

```
  | AD.Replaced { from = rf; to_ = rt } ->
      let del =
        if is_absent rf then []
        else
          [ HTML.span [ HTML.class_ "diff-del" ]
              [ txt "- "; txt "%s" (Reader.pp_expr rf) ] ]
      in
      let add =
        if is_absent rt then []
        else
          [ HTML.span [ HTML.class_ "diff-add" ]
              [ txt "+ "; txt "%s" (Reader.pp_expr rt) ] ]
      in
      HTML.div [HTML.class_ "diff-replaced"]
        (del @ (if del <> [] && add <> [] then [txt " "] else []) @ add)
  | AD.Node { ctor; children } ->
      let child_nodes =
        List.filter_map
          (fun (label, d') ->
            match d' with
            | AD.Same -> None
            | _ ->
                Some
                  (HTML.div [HTML.class_ "diff-child"]
                     [ HTML.span [HTML.class_ "diff-label"]
                         [ txt "%s" label; txt ":" ]
                     ; render_delta d' ]))
          children
      in
      HTML.div [HTML.class_ "diff-node"]
        (HTML.span [HTML.class_ "diff-ctor"] [txt "(%s)" ctor]
         :: (if child_nodes = [] then
               [ HTML.span [HTML.class_ "diff-same"] [txt " (identical)"] ]
             else child_nodes))

let diff_kind_s : Store.diff_kind -> string = function
  | Store.DiffFn -> "fn"
  | Store.DiffType -> "type"
  | Store.DiffRoute -> "route"

let diff_status_s : Store.diff_status -> string = function
  | Store.DiffAdded -> "added"
  | Store.DiffRemoved -> "removed"
  | Store.DiffModified -> "modified"

let diff_status_cls : Store.diff_status -> string = function
  | Store.DiffAdded -> "ok"
  | Store.DiffRemoved -> "err"
  | Store.DiffModified -> "warn"

(** Expandable body cell for a diff row. Modified fn rows render the
    structured AST delta (overlay of [AD.diff] on the parsed bodies) plus
    both full bodies; modified type/route and added/removed rows render the
    plain body/handler text. *)
let diff_detail (r : Store.branch_diff_row) : Pure_html.node =
  let pre s = HTML.pre [] [txt "%s" s] in
  match r.Store.diff_status with
  | Store.DiffModified when r.Store.diff_kind = Store.DiffFn ->
      (match r.Store.from_expr, r.Store.to_expr with
       | Some fe, Some te ->
           HTML.details [HTML.class_ "explain-panel"]
             [ HTML.summary [] [txt "AST delta"]
             ; HTML.div [HTML.class_ "card"]
                 [ render_delta (AD.diff fe te)
                 ; HTML.hr []
                 ; HTML.div [HTML.class_ "diff-bodies"]
                     [ HTML.div []
                         [ HTML.span [HTML.class_ "muted"]
                             [txt "from: "]
                         ; pre (Option.value ~default:"" r.Store.from_body)
                         ]
                     ; HTML.div []
                         [ HTML.span [HTML.class_ "muted"]
```

```
                            [txt "to:   "]
                        ; pre (Option.value ~default:"" r.Store.to_body)
                        ]
                    ]
                ] ]
      | _ -> txt " ")
  | Store.DiffModified ->
      HTML.details [HTML.class_ "explain-panel"]
        [ HTML.summary [] [txt "body"]
        ; pre
            (Printf.sprintf "from: %s\nto:   %s"
               (Option.value ~default:"" r.Store.from_body)
               (Option.value ~default:"" r.Store.to_body)) ]
  | Store.DiffAdded ->
      HTML.details [HTML.class_ "explain-panel"]
        [ HTML.summary [] [txt "body"]
        ; pre (Option.value ~default:"" r.Store.to_body) ]
  | Store.DiffRemoved ->
      HTML.details [HTML.class_ "explain-panel"]
        [ HTML.summary [] [txt "body"]
        ; pre (Option.value ~default:"" r.Store.from_body) ]

(** GET /diff?from=owner/name&to=owner/name -- private branch diff page. *)
let diff_page (session : Session.t) (reqd : Httpun.Reqd.t) : unit =
  let target = (Httpun.Reqd.request reqd).target in
  let resolve ref_ =
    let owner, name =
      match String.split_on_char '/' ref_ with
      | [ o; n ] when o <> "" && n <> "" -> (o, n)
      | _ -> ("", ref_)
    in
    Store.lookup_branch_id_by_name (Session.store session) ~owner ~name
  in
  let from_ref =
    Option.value (query_param target "from") ~default:"" |> String.trim
  in
  let to_ref =
    Option.value (query_param target "to") ~default:"" |> String.trim
  in
  match resolve from_ref, resolve to_ref with
  | Some fid, Some tid ->
      let store = Session.store session in
      let rows = Store.diff_branches store ~from_id:fid ~to_id:tid in
      let row_tr (r : Store.branch_diff_row) : Pure_html.node list =
        let upd =
          HTML.span [HTML.class_ "muted"]
            [ txt "%s"
                (Printf.sprintf "%s  ?  %s"
                   (Option.value ~default:"-" r.Store.from_updated)
                   (Option.value ~default:"-" r.Store.to_updated))
            ]
        in
        [ HTML.tr []
            [ HTML.td [] [txt "%s" r.Store.diff_name]
            ; HTML.td [HTML.class_ "muted"]
                [txt "%s" (diff_kind_s r.Store.diff_kind)]
            ; HTML.td []
                [ HTML.span [HTML.class_ "%s" (diff_status_cls
   r.Store.diff_status)]
                    [txt "%s" (diff_status_s r.Store.diff_status)] ]
            ; HTML.td [] [upd]
            ]
        ; HTML.tr []
            [ HTML.td [HTML.colspan 4; HTML.class_ "diff-detail"]
                [diff_detail r] ]
        ]
      in
      let head =
        HTML.thead []
          [ HTML.tr []
              [ HTML.th [] [txt "name"]
              ; HTML.th [] [txt "kind"]
              ; HTML.th [] [txt "status"]
```

```
              ; HTML.th [] [txt "from ? to updated"]
              ] ]
      in
      let body =
        chrome ~session ~title:("Diff ? " ^ from_ref ^ " ? " ^ to_ref)
          [ HTML.style [] "%s"
              {|
.diff-detail pre{white-space:pre-wrap;}
.diff-node{font-family:"Courier
    New",Courier,monospace;font-size:.9rem;padding-left:.4rem;border-left:2px
    solid #666;margin:.15rem 0;}
.diff-ctor{font-weight:700;color:var(--link);}
.diff-child{margin:.1rem 0 .1rem 1rem;}
.diff-label{color:var(--muted);margin-right:.3rem;}
.diff-replaced{display:inline-block;}
.diff-del{color:var(--err);}
.diff-add{color:var(--ok);}
.diff-same{color:var(--muted);opacity:.6;}
.diff-bodies{display:grid;grid-template-columns:1fr 1fr;gap:.5rem;}
@media(max-width:700px){.diff-bodies{grid-template-columns:1fr;}}
|}
          ; HTML.p []
              [ HTML.a [HTML.href "/branches"] [txt "Branches"]
              ; txt " ? diff "
              ; HTML.code [] [txt "%s" from_ref]
              ; txt " ? "
              ; HTML.code [] [txt "%s" to_ref]
              ]
          ; HTML.div [HTML.class_ "card"]
              [ HTML.h2 [] [txt "%d divergent rows" (List.length rows)]
              ; (if rows = [] then
                   HTML.p [HTML.class_ "ok"] [txt "no differences"]
                 else
                   HTML.table [HTML.class_ "br-table"]
                     [ head
                     ; HTML.tbody []
                         (List.concat (List.map row_tr rows))
                     ])
              ]
          ]
      in
      html_response reqd ~session body;
      ()
  | _ ->
      html_response reqd ~session ~status:`Not_found
        (chrome ~session ~title:"Diff"
           [ HTML.p [HTML.class_ "err"]
               [txt "unknown branch pair (%s, %s)" from_ref to_ref] ]);
      ()
```

74 image/lib/http_code.ml

```
(** Public release: read-only source browser, CodeMirror assets,
    book.pdf + src.tgz from PP_PUBLIC_DIR.

    See product-experience.public-release. *)
let env_dir (name : string) (default : string) : string =
  match Sys.getenv_opt name with
  | Some d when String.trim d <> "" -> d
  | _ -> default

let source_root () : string =
  match Sys.getenv_opt "PP_SOURCE_ROOT" with
  | Some d when String.trim d <> "" -> d
  | _ ->
      (match Sys.getenv_opt "PP_ROOT" with
       | Some d when String.trim d <> "" -> d
       | _ ->
           if Sys.file_exists "pricklypear.borg" then "."
           else if Sys.file_exists "../pricklypear.borg" then ".."
           else ".")

let public_dir () : string =
  match Sys.getenv_opt "PP_PUBLIC_DIR" with
  | Some d when String.trim d <> "" -> d
  | _ ->
      let cand = Filename.concat (source_root ()) "public-artifacts" in
      if Sys.file_exists cand then cand else "/tmp/pp-public"

let static_root () : string =
  match Sys.getenv_opt "PP_STATIC_DIR" with
  | Some d when String.trim d <> "" -> d
  | _ ->
      let root = source_root () in
      let a = Filename.concat root "image/static" in
      if Sys.file_exists a then a
      else if Sys.file_exists "image/static" then "image/static"
      else "image/static"

(* Directory holding built browser assets (interp.js from client/ via
   js_of_ocaml). Falls back to the dune build tree under the source root.
   Spec: borge/structural-editing.borg client-eval. *)
let client_dir () : string =
  match Sys.getenv_opt "PP_CLIENT_DIR" with
  | Some d when String.trim d <> "" -> d
  | _ -> Filename.concat (source_root ()) "_build/default/client/bin"

let skip_dirs : string list =
  [ "_build"
  ; ".git"
  ; ".pi"
  ; ".ralph"
  ; "node_modules"
  ; "_opam"
  ; ".opamswitch"
  ; "spikes"
  ; "supervisor"
  ; ".borge-bugs"
  ; "public-artifacts"
  ; "_build_test"
  ; "backups"
  ; "log"
  ]

let allowed_ext (name : string) : bool =
  let lower = String.lowercase_ascii name in
  if lower = "dune" || lower = "dune-project" || lower = "pricklypear" then
     true
  else
    List.exists
      (fun suf -> Filename.check_suffix lower suf)
      [ ".borg"
      ; ".ml"
```

```
      ; ".mli"
      ; ".pp"
      ; ".md"
      ; ".sql"
      ; ".sh"
      ; ".ksh"
      ; ".py"
      ; ".dune"
      ; ".txt"
      ; ".opam"
      ; ".example"
      ]

let max_file_bytes = 2 * 1024 * 1024

(** Release artifacts (src.tgz ~2.25 MiB, book.pdf ~1.1 MiB) exceed the
    /code browser's 2 MiB cap. Give the public-artifact path its own
    larger bound so [/src.tgz] and [/book.pdf] actually serve instead
    of 404ing with "file too large". Still bounded to prevent abuse. *)
let max_public_artifact_bytes = 64 * 1024 * 1024

let html_escape (s : string) : string =
  let buf = Buffer.create (String.length s) in
  String.iter
    (function
      | '&' -> Buffer.add_string buf "&amp;"
      | '<' -> Buffer.add_string buf "&lt;"
      | '>' -> Buffer.add_string buf "&gt;"
      | '"' -> Buffer.add_string buf "&quot;"
      | '\'' -> Buffer.add_string buf "&#39;"
      | c -> Buffer.add_char buf c)
    s;
  Buffer.contents buf

let pct_encode (s : string) : string =
  let buf = Buffer.create (String.length s * 2) in
  String.iter
    (function
      | ('A' .. 'Z' | 'a' .. 'z' | '0' .. '9' | '-' | '_' | '.' | '~' |
     '/') as c ->
          Buffer.add_char buf c
      | c -> Buffer.add_string buf (Printf.sprintf "%%%02X" (Char.code c)))
    s;
  Buffer.contents buf

let hex_nibble (c : char) : int option =
  match c with
  | '0' .. '9' -> Some (Char.code c - Char.code '0')
  | 'a' .. 'f' -> Some (Char.code c - Char.code 'a' + 10)
  | 'A' .. 'F' -> Some (Char.code c - Char.code 'A' + 10)
  | _ -> None

let pct_decode (s : string) : string =
  let buf = Buffer.create (String.length s) in
  let len = String.length s in
  let rec loop i =
    if i >= len then Buffer.contents buf
    else if s.[i] = '%' && i + 2 < len then (
      match hex_nibble s.[i + 1], hex_nibble s.[i + 2] with
      | Some a, Some b ->
          Buffer.add_char buf (Char.chr ((a lsl 4) lor b));
          loop (i + 3)
      | _ ->
          Buffer.add_char buf s.[i];
          loop (i + 1))
    else if s.[i] = '+' then (
      Buffer.add_char buf ' ';
      loop (i + 1))
    else (
      Buffer.add_char buf s.[i];
      loop (i + 1))
  in
  loop 0
```

```
143
144 let realpath_opt (path : string) : string option =
145   try Some (Unix.realpath path) with
146   | Unix.Unix_error _ | Sys_error _ -> None
147
148 (** Strip leading slashes; reject .. and empty segments.
149     Callers should pct-decode first so %2e%2e cannot sneak through. *)
150 let normalize_rel (rel : string) : string option =
151   let rel =
152     let len = String.length rel in
153     if len > 0 && rel.[0] = '/' then String.sub rel 1 (len - 1) else rel
154   in
155   if rel = "" then Some ""
156   else
157     let parts = String.split_on_char '/' rel in
158     let ok =
159       List.for_all
160         (fun p ->
161           p <> "" && p <> "." && p <> ".." && not (String.contains p
      '\000'))
162         parts
163     in
164     if not ok then None else Some (String.concat "/" parts)
165
166 (** Resolve [rel] under [root]. Does not require the target to exist
167     (realpath on missing files fails); safety is normalize_rel forbidding
168     ".." plus realpath on the root only. *)
169 let resolve_under ~(root : string) (rel : string) : string option =
170   match realpath_opt root with
171   | None -> None
172   | Some root_real ->
173       (match normalize_rel rel with
174        | None -> None
175        | Some "" -> Some root_real
176        | Some rel -> Some (Filename.concat root_real rel))
177
178 let read_file_within ~(max : int) (path : string) : (string, string) result
      =
179   try
180     let st = Unix.stat path in
181     if st.Unix.st_kind <> Unix.S_REG then Error "not a file"
182     else if st.Unix.st_size > max then Error "file too large"
183     else
184       let ic = open_in_bin path in
185       let len = in_channel_length ic in
186       let s = really_input_string ic len in
187       close_in ic;
188       Ok s
189   with exn -> Error (Printexc.to_string exn)
190
191 let read_file_limited (path : string) : (string, string) result =
192   read_file_within ~max:max_file_bytes path
193
194 let mime_for_name (name : string) : string =
195   let lower = String.lowercase_ascii name in
196   if Filename.check_suffix lower ".css" then "text/css; charset=utf-8"
197   else if Filename.check_suffix lower ".js" then
198     "application/javascript; charset=utf-8"
199   else if Filename.check_suffix lower ".pdf" then "application/pdf"
200   else if Filename.check_suffix lower ".tgz"
201        || Filename.check_suffix lower ".gz" then "application/gzip"
202   else if Filename.check_suffix lower ".svg" then "image/svg+xml"
203   else if Filename.check_suffix lower ".png" then "image/png"
204   else if Filename.check_suffix lower ".jpg"
205        || Filename.check_suffix lower ".jpeg" then "image/jpeg"
206   else if Filename.check_suffix lower ".gif" then "image/gif"
207   else if Filename.check_suffix lower ".webp" then "image/webp"
208   else if Filename.check_suffix lower ".ico" then "image/x-icon"
209   else "application/octet-stream"
210
211 let cm_mode_for (name : string) : string =
212   let lower = String.lowercase_ascii name in
213   if Filename.check_suffix lower ".ml"
```

```
     || Filename.check_suffix lower ".mli" then "text/x-ocaml"
  else if Filename.check_suffix lower ".md" then "text/x-markdown"
  else "text/plain"

type tree_node =
  | Dir of string * tree_node list
  | File of string * string (* name, rel_path *)

let rec walk_tree ~(root : string) ~(rel : string) : tree_node list =
  let full = if rel = "" then root else Filename.concat root rel in
  let entries =
    try Sys.readdir full |> Array.to_list with Sys_error _ -> []
  in
  let entries = List.sort String.compare entries in
  List.filter_map
    (fun name ->
      if List.mem name skip_dirs then None
      else if String.length name > 0 && name.[0] = '.' then None
      else
        let child_rel = if rel = "" then name else rel ^ "/" ^ name in
        let child_full = Filename.concat full name in
        try
          if Sys.is_directory child_full then
            let kids = walk_tree ~root ~rel:child_rel in
            if kids = [] then None else Some (Dir (name, kids))
          else if allowed_ext name then Some (File (name, child_rel))
          else None
        with Sys_error _ -> None)
    entries

let rec tree_to_html (nodes : tree_node list) ~(active : string) : string =
  let buf = Buffer.create 256 in
  Buffer.add_string buf "<ul class=\"code-tree\">";
  List.iter
    (function
      | Dir (name, kids) ->
          Buffer.add_string buf "<li class=\"code-dir\"><details open>";
          Buffer.add_string buf "<summary>";
          Buffer.add_string buf (html_escape name);
          Buffer.add_string buf "</summary>";
          Buffer.add_string buf (tree_to_html kids ~active);
          Buffer.add_string buf "</details></li>"
      | File (name, rel) ->
          let cls =
            if rel = active then "code-file active" else "code-file"
          in
          let href = "/code?f=" ^ pct_encode rel in
          Buffer.add_string buf (Printf.sprintf "<li class=\"%s\">" cls);
          Buffer.add_string buf
            (Printf.sprintf "<a href=\"%s\">%s</a>" href (html_escape
    name));
          Buffer.add_string buf "</li>")
    nodes;
  Buffer.add_string buf "</ul>";
  Buffer.contents buf

let code_css : string =

    "body{margin:0;font-family:ui-monospace,Menlo,monospace;background:#faf7f4;color:#1a1a1a}\n\

    header.code-hdr{display:flex;flex-wrap:wrap;gap:.75rem;align-items:center;\n\
   justify-content:space-between;padding:.5rem .75rem;border-bottom:2px
    solid #000;\n\
   background:#fff}\n\
   header.code-hdr a{color:#000080;font-weight:700}\n\
   .code-wrap{display:flex;min-height:calc(100vh - 3rem)}\n\
   .code-side{width:16rem;min-width:12rem;max-width:40vw;overflow:auto;\n\
   border-right:2px solid
    #000;background:#e8e2db;padding:.5rem;font-size:.85rem}\n\
   .code-main{flex:1;display:flex;flex-direction:column;min-width:0}\n\
   .code-meta{padding:.35rem .75rem;border-bottom:1px solid
    #808080;background:#fff;\n\
   font-size:.85rem}\n\
```

```
   .code-editor{flex:1;min-height:0}\n\
   .CodeMirror{height:100%;font-size:13px;line-height:1.4}\n\
   ul.code-tree{list-style:none;margin:0;padding-left:.85rem}\n\
   li.code-file a{text-decoration:none;color:#000}\n\
   li.code-file.active a{font-weight:700;background:#fff;padding:0 .2rem}\n\
   li.code-dir>details>summary{cursor:pointer;font-weight:700}\n\
   .code-err{color:#8b0000;padding:1rem}\n\
   @media(max-width:40rem){.code-wrap{flex-direction:column}\n\
     .code-side{width:auto;max-width:none;max-height:30vh;border-right:none;\n\
   border-bottom:2px solid #000}}\n"

let serve_bytes (reqd : Httpun.Reqd.t) ?(status = `OK) ~content_type
    (body : string) : unit =
  let headers =
    Httpun.Headers.of_list
      ([ ("content-type", content_type)
       ; ("content-length", string_of_int (String.length body))
       ; ("cache-control", "public, max-age=3600")
       ] @ Http_core.security_headers ())
  in
  let resp = Httpun.Response.create ~headers status in
  Http_core.safe_respond_with_string reqd resp body

let serve_file_path ?(max = max_file_bytes) (reqd : Httpun.Reqd.t) ~(path :
    string) : unit =
  match read_file_within ~max path with
  | Error msg ->
      serve_bytes reqd ~status:`Not_found ~content_type:"text/plain;
     charset=utf-8"
        ("not found: " ^ msg)
  | Ok body ->
      let ct = mime_for_name (Filename.basename path) in
      serve_bytes reqd ~content_type:ct body

let handle_asset (reqd : Httpun.Reqd.t) ~(rel : string) : unit =
  let rel = pct_decode rel in
  (* Search the committed static dir first, then the built client-asset dir
     (interp.js from client/ via js_of_ocaml). Spec:
     structural-editing.client-eval. *)
  let try_root root =
    match resolve_under ~root rel with
    | Some path when Sys.file_exists path && not (Sys.is_directory path) ->
        Some path
    | _ -> None
  in
  match try_root (static_root ()) with
  | Some path -> serve_file_path reqd ~path
  | None ->
      (match try_root (client_dir ()) with
       | Some path -> serve_file_path reqd ~path
       | None ->
           serve_bytes reqd ~status:`Not_found ~content_type:"text/plain;
     charset=utf-8"
             "not found")

let handle_public_artifact (reqd : Httpun.Reqd.t) ~(name : string) : unit =
  let root = public_dir () in
  match resolve_under ~root name with
  | None ->
      serve_bytes reqd ~status:`Not_found ~content_type:"text/html;
     charset=utf-8"
        ("<!doctype html><title>missing</title><p>Artifact <code>"
         ^ name
         ^ "</code> not installed. Run
     <code>scripts/public_artifacts.sh</code> \
           and set <code>PP_PUBLIC_DIR</code>.</p><p><a
     href=\"/welcome\">welcome</a></p>")
  | Some path ->
      if Sys.file_exists path && not (Sys.is_directory path) then
        serve_file_path reqd ~max:max_public_artifact_bytes ~path
      else
```

```
        serve_bytes reqd ~status:`Not_found ~content_type:"text/html; charset=utf-8"
          ("<!doctype html><title>missing</title><p>No <code>"
           ^ name
           ^ "</code> in <code>"
           ^ root
           ^ "</code>. Run <code>scripts/public_artifacts.sh</code>.</p>\
              <p><a href=\"/welcome\">welcome</a> ? <a href=\"/code\">code</a></p>")

let handle_raw (reqd : Httpun.Reqd.t) ~(rel : string) : unit =
  let root = source_root () in
  let rel = pct_decode rel in
  match resolve_under ~root rel with
  | None ->
      serve_bytes reqd ~status:`Forbidden ~content_type:"text/plain; charset=utf-8"
        "forbidden path"
  | Some path ->
      if not (Sys.file_exists path) then
        serve_bytes reqd ~status:`Not_found ~content_type:"text/plain; charset=utf-8"
          "not found"
      else if Sys.is_directory path then
        serve_bytes reqd ~status:`Bad_request ~content_type:"text/plain; charset=utf-8"
          "is a directory"
      else if not (allowed_ext (Filename.basename path)) then
        serve_bytes reqd ~status:`Forbidden ~content_type:"text/plain; charset=utf-8"
          "extension not allowed"
      else (
        match read_file_limited path with
        | Error msg ->
            serve_bytes reqd ~status:`Bad_request
              ~content_type:"text/plain; charset=utf-8" msg
        | Ok body ->
            serve_bytes reqd ~content_type:"text/plain; charset=utf-8" body)

let query_param (target : string) (key : string) : string option =
  match String.index_opt target '?' with
  | None -> None
  | Some i ->
      let q = String.sub target (i + 1) (String.length target - i - 1) in
      let pairs = String.split_on_char '&' q in
      List.find_map
        (fun pair ->
          match String.split_on_char '=' pair with
          | k :: rest when k = key ->
              let v = String.concat "=" rest in
              Some (pct_decode v)
          | _ -> None)
        pairs

let handle_code_page (reqd : Httpun.Reqd.t) ~(target : string) : unit =
  let root = source_root () in
  let active =
    match query_param target "f" with
    | Some f when f <> "" -> f
    | _ ->
        if Sys.file_exists (Filename.concat root "pricklypear.borg") then
          "pricklypear.borg"
        else ""
  in
  let tree = walk_tree ~root ~rel:"" in
  let tree_html = tree_to_html tree ~active in
  let mode = cm_mode_for active in
  let title =
    if active = "" then "Source" else active
  in
  let boot_js =
    Printf.sprintf
      {|
```

```
var MODE=%s;
var FILE=%s;
var ta=document.getElementById('src');
var cm=CodeMirror.fromTextArea(ta,{
  lineNumbers:true,
  readOnly:true,
  mode:MODE,
  lineWrapping:true,
  viewportMargin:40
});
function loadFile(path){
  if(!path){cm.setValue('// pick a file from the tree');return;}
  fetch('/code/raw/'+path.split('/').map(encodeURIComponent).join('/'))
    .then(function(r){if(!r.ok)throw new Error(r.statusText);return
    r.text();})
    .then(function(t){
      var m='text/plain';
      if(/\.ml$|\.mli$/i.test(path))m='text/x-ocaml';
      else if(/\.md$/i.test(path))m='text/x-markdown';
      cm.setOption('mode',m);
      cm.setValue(t);
      document.getElementById('code-path').textContent=path;
    })
    .catch(function(e){cm.setValue('// error: '+e);});
}
loadFile(FILE);
|}
      (Yojson.Safe.to_string (`String mode))
      (Yojson.Safe.to_string (`String active))
  in
  let html =
    Printf.sprintf
      {|<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8"/>
<meta name="viewport" content="width=device-width, initial-scale=1"/>
<title>Pricklypear -- %s</title>
<link rel="stylesheet" href="/assets/cm/codemirror.min.css"/>
<style>%s</style>
</head>
<body>
<header class="code-hdr">
  <div><strong>Pricklypear source</strong> ? read-only</div>
  <nav>
    <a href="/welcome">Welcome</a> ?
    <a href="/book.pdf">Book PDF</a> ?
    <a href="/src.tgz">.tgz</a> ?
    <a href="/login">Log in</a>
  </nav>
</header>
<div class="code-wrap">
  <aside class="code-side">%s</aside>
  <div class="code-main">
    <div class="code-meta">file: <span id="code-path">%s</span> ? root:
     <code>%s</code></div>
    <div class="code-editor"><textarea id="src"></textarea></div>
  </div>
</div>
<script src="/assets/cm/codemirror.min.js"></script>
<script src="/assets/cm/mode/mllike/mllike.min.js"></script>
<script src="/assets/cm/mode/markdown/markdown.min.js"></script>
<script>%s</script>
</body>
</html>|}
      (html_escape title) code_css tree_html (html_escape active)
      (html_escape root) boot_js
  in
  serve_bytes reqd ~content_type:"text/html; charset=utf-8" html

(** Public Lisp pages: body HTML only (no product chrome / private nav). *)
let serve_public_body (reqd : Httpun.Reqd.t) ?(title = "Pricklypear")
    (body_html : string) : unit =
```

```
  let doc =
    Printf.sprintf
      {|<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8"/>
<meta name="viewport" content="width=device-width, initial-scale=1"/>
<title>%s</title>
<style>
body{margin:0;font-family:system-ui,-apple-system,sans-serif;background:#faf7f4;color:#1a1a1a;line-height:1.5}
a{color:#000080}
</style>
</head>
<body>
%s
</body>
</html>|}
      (html_escape title) body_html
  in
  serve_bytes reqd ~content_type:"text/html; charset=utf-8" doc

let fallback_welcome (reqd : Httpun.Reqd.t) : unit =
  let body =
    {|<!DOCTYPE html><html lang="en"><head><meta charset="utf-8"/>
<title>Pricklypear</title>
<meta name="viewport" content="width=device-width, initial-scale=1"/>
<style>
body{font-family:system-ui,sans-serif;max-width:40rem;margin:2rem
     auto;padding:0 1rem;line-height:1.5;background:#faf7f4}
a{color:#000080;font-weight:700} h1{letter-spacing:-.02em}
.actions a{display:inline-block;margin:.25rem .35rem .25rem 0;padding:.4rem
     .75rem;border:2px solid
     #000;text-decoration:none;color:#000;background:#fff}
</style></head><body>
<h1>Pricklypear</h1>
<p>A multi-user Lisp OS -- live image, Postgres object store, htmx
     interface.</p>
<p class="actions">
<a href="/code">Explore the source</a>
<a href="/book.pdf">Book (PDF)</a>
<a href="/src.tgz">Source tarball</a>
<a href="/login">Log in</a>
</p>
<p>Lisp welcome page not loaded yet (seed layout + welcome packages).</p>
</body></html>|}
  in
  serve_bytes reqd ~content_type:"text/html; charset=utf-8" body

let fallback_book (reqd : Httpun.Reqd.t) : unit =
  let body =
    {|<!DOCTYPE html><html lang="en"><head><meta charset="utf-8"/>
<title>Pricklypear book</title>
<meta name="viewport" content="width=device-width, initial-scale=1"/>
<style>body{font-family:system-ui,sans-serif;max-width:40rem;margin:2rem
     auto;padding:0 1rem;line-height:1.5}
a{font-weight:700}</style></head><body>
<h1>The Pricklypear book</h1>
<p>Paginated literate rendering of the codebase via <code>borge book
     print</code>.</p>
<p><a href="/book.pdf">Download PDF</a> ? <a href="/src.tgz">tarball</a> ?
<a href="/code">browse</a> ? <a href="/welcome">welcome</a></p>
</body></html>|}
  in
  serve_bytes reqd ~content_type:"text/html; charset=utf-8" body
```

75 image/lib/http_core.ml

```
(** Pricklypear Image -- HTTP shared infrastructure.
    Responses, chrome, forms, auth gate helpers, login/admin.
    Page handlers live in [Http_pages]; dispatch in [Http]. *)
open Pure_html

let default_port = 8080

(* ------------------------------------------------------------------
   PEER-DISCONNECT SAFETY (image.http-robustness invariants 1-2).

   httpaf raises Failure("cannot write to closed writer") when you write
   to a response whose client has already disconnected. Under real
   traffic (people aborting page loads) an uncaught instance of this
   exception killed the single-image process repeatedly (prod incident
   2026-07-16). Every response write must tolerate a gone peer.
   Promoted from http_agent.ml's SSE-only guard; shared by all response
   helpers + the httpaf error_handler. *)
let is_closed_writer_exn (exn : exn) : bool =
  match exn with
  | Failure msg ->
      String.equal msg "cannot write to closed writer"
      || String.starts_with ~prefix:"cannot write to closed writer" msg
  | _ -> false

(** Write a response body string, swallowing the closed-writer Failure
    that httpaf raises when the peer has gone. Returns () either way. *)
let safe_respond_with_string (reqd : Httpun.Reqd.t)
    (resp : Httpun.Response.t) (body : string) : unit =
  try Httpun.Reqd.respond_with_string reqd resp body
  with exn when is_closed_writer_exn exn -> ()

(** Close a body writer, swallowing the closed-writer Failure. *)
let safe_close_writer (body : Httpun.Body.Writer.t) : unit =
  try Httpun.Body.Writer.close body
  with exn when is_closed_writer_exn exn -> ()

(* ------------------------------------------------------------------
   ACCEPT-SIDE CONNECTION CAP (image.http-robustness invariant 5).

   Bounds in-flight requests so a traffic spike can't exhaust fds or
   memory. Default 256; configurable via PP_HTTP_MAX_INFLIGHT. Requests
   beyond the cap are rejected with 503 instead of being accepted into
   an unbounded queue. This is a safety valve, not a throughput target. *)
let max_inflight =
  match Sys.getenv_opt "PP_HTTP_MAX_INFLIGHT" with
  | Some s -> (match int_of_string_opt s with Some n when n > 0 -> n | _ ->
     256)
  | None -> 256

(* Counting gate for concurrent in-flight page work (Eio floor).
   Overflow waits for a slot rather than consuming unbounded memory/fds. *)
type inflight_gate = {
  mutable count : int;
  limit : int;
  mutex : Eio.Mutex.t;
  cond : Eio.Condition.t;
}

let inflight_sem =
  { count = 0; limit = max_inflight;
    mutex = Eio.Mutex.create (); cond = Eio.Condition.create () }

let inflight_acquire () : unit =
  Eio.Mutex.lock inflight_sem.mutex;
  while inflight_sem.count >= inflight_sem.limit do
    Eio.Condition.await inflight_sem.cond inflight_sem.mutex
  done;
  inflight_sem.count <- inflight_sem.count + 1;
  Eio.Mutex.unlock inflight_sem.mutex

let inflight_release () : unit =
```

```
  Eio.Mutex.lock inflight_sem.mutex;
  inflight_sem.count <- inflight_sem.count - 1;
  Eio.Condition.broadcast inflight_sem.cond;
  Eio.Mutex.unlock inflight_sem.mutex

let emergency_css : string =
  (* Labeled recovery only -- NOT the product design system.
     Product CSS comes from Lisp (ui/css) via Session.ui_css. *)
  "/* emergency-css */\n\
   body{font-family:ui-monospace,Menlo,monospace;margin:1rem;\n\
   background:#fff3cd;color:#1a1a1a;line-height:1.4}\n\
   .emergency-banner{display:block;width:100%;padding:.4rem .55rem;\n\
   border:2px solid #8b0000;background:#fff3cd;color:#1a1a1a;\n\
   font-size:.85rem;font-family:Courier,monospace;margin:.35rem 0}\n\
   .emergency-banner a{color:#000080}\n\
   nav.emergency-nav{width:100%;margin-top:.35rem}\n\
   .emergency-home{border:2px solid
     #8b0000;background:#fff3cd;padding:.75rem}\n\
   a{color:#000080}\n\
   header{border-bottom:2px solid #000;padding:.5rem;background:#fff}\n\
   main{padding:.75rem}\n\
   .card{border:2px solid #000;background:#fff;padding:.75rem;margin:.5rem
     0}\n\
   .err{color:#8b0000;font-weight:700}\n\
   .ok{color:#006600;font-weight:700}\n\
   button,.btn,input[type=submit]{font:inherit;padding:.45rem .75rem;\n\
   border:2px solid #000;background:#eee;cursor:pointer}\n\
   table{border-collapse:collapse;width:100%}\n\
   th,td{border:1px solid #808080;padding:.35rem}\n\
   pre,textarea,input[type=text],input[type=password]{\n\
   font-family:ui-monospace,Menlo,monospace;width:100%;\n\
   box-sizing:border-box;border:1px solid #000;padding:.4rem}\n\
   .badge.impact{background:#ffe8a3;border:1px solid #8b6914;padding:.15rem
     .4rem}\n\
   .bb-browser{display:flex;flex-direction:column;gap:.75rem}\n\
   .bb-three{display:block}\n\
   .bb-three .bb-col{margin:.5rem 0}\n\
   .bb-list{list-style:none;padding:0;margin:.25rem 0}\n\
   .bb-item a{display:block;padding:.2rem .35rem;text-decoration:none;\n\
   border-left:3px solid transparent}\n\
   .bb-item.is-active a{font-weight:700;background:#ece9e0;border-left:3px
     solid #000}\n\
   .bb-source{max-height:55vh;overflow:auto;padding:.6rem;border:1px solid
     #000}\n\
   .bb-tabs{display:flex;gap:.35rem;flex-wrap:wrap;margin:.4rem 0
     .6rem;align-items:center}\n\
   .bb-tab{display:inline-block;padding:.28rem .65rem;border:1px solid
     #000;\n\

     text-decoration:none;color:inherit;background:#f7f5ef;border-radius:2px}\n\
   .bb-tab:hover{background:#ece9e0}\n\
   .bb-tab.is-active{background:#1a1a1a;color:#f7f5ef;font-weight:700}\n\
   .bb-edit-form
     textarea{width:100%;min-height:14rem;font-family:ui-monospace,monospace}\n\
   .bb-code .bb-edit-form .card{border:none;padding:0;box-shadow:none}\n\
   .ast-walk .ast-self-ref{background:#fff3bf;border-bottom:2px solid
     #e67700}\n\
   .ast-walk .ast-tail{box-shadow:inset 3px 0 0 #2b8a3e}\n\
   .ast-walk .ast-non-tail{box-shadow:inset 3px 0 0 #c92a2a}\n\
   .ast-walk .ast-mutual{background:#e7f5ff;border-bottom:2px dashed
     #1971c2}\n\
   .badge.complexity{background:#e7f5ff;border:1px solid
     #1971c2;padding:.1rem .35rem}\n\
   .badge.db-cost{background:#f3f0ff;border:1px solid #5f3dc4;padding:.1rem
     .35rem}\n\
   .badge.ast-tail{background:#d3f9d8;border:1px solid
     #2b8a3e;padding:.1rem .35rem}\n\
   .badge.ast-non-tail{background:#ffe3e3;border:1px solid
     #c92a2a;padding:.1rem .35rem}\n\
   .explain-layer,.explain-panel{margin-top:.5rem}\n\
   .analyze-result{margin-top:.5rem}\n\
   @media (min-width:900px){\n\
```

```
128     .bb-three{display:grid;grid-template-columns:12rem 14rem
          1fr;gap:.75rem;\n\
129     align-items:stretch;min-height:16rem}\n\
130     .bb-three .bb-col{max-height:42vh;overflow:auto;margin:0}\n\
131     }\n\
132     "
133
134 (** Security headers appended to every response. Deliberately NOT a full
135     CSP: the chrome relies on inline <style>/<script> (htmx + outline
136     client script), so we only pin frame-ancestors 'none'. *)
137 let security_headers () : (string * string) list =
138   [ ("x-content-type-options", "nosniff")
139   ; ("x-frame-options", "DENY")
140   ; ("referrer-policy", "strict-origin-when-cross-origin")
141   ; ("content-security-policy", "frame-ancestors 'none'")
142   ]
143
144 let html_response (reqd : Httpun.Reqd.t) ?(status = `OK)
145     ?session ?(css : string option) (body : Pure_html.node) : unit =
146   let css =
147     match css with
148     | Some c when String.trim c <> "" -> c
149     | _ ->
150         (match session with
151          | Some s ->
152              (match Session.ui_css s with
153               | Some c when String.trim c <> "" -> c
154               | _ -> emergency_css)
155          | None -> emergency_css)
156   in
157   let html_doc = HTML.html [HTML.lang "en"] [
158     HTML.head [] [
159       HTML.title [] "Pricklypear";
160       HTML.meta [HTML.charset "utf-8"];
161       (* Mobile-first: prevent tiny text, enable proper layout *)
162       HTML.meta [HTML.name "viewport";
163                  HTML.content "width=device-width, initial-scale=1"];
164       HTML.style [] "%s" css;
165
166       (* htmx 4 (vendored, no CDN -- see borge/multipart.borg stance) +
167          htmx-2-compat: restores implicit attribute inheritance, 1.9/2.x
168          error no-swap defaults, and old camelCase event names so existing
169          markup keeps htmx 1.9 semantics during incremental migration. *)
170       HTML.script [HTML.src "/assets/htmx.min.js"] "";
171       HTML.script [HTML.src "/assets/htmx-2-compat.min.js"] "";
172       (* hx-optimistic: official htmx 4 extension (vendored, no CDN). Loaded
173          globally so it registers before any hx-optimistic attribute is
174          processed. Display-placeholder optimism only -- see
175          borge/structural-editing.borg RESPONSIVENESS WITHOUT PREDICTION.
176          Inert on pages that don't use the attribute. *)
177       HTML.script [HTML.src "/assets/hx-optimistic.js"] "";
178       (* idiomorph: standalone morph library (diff-based DOM
179          reconciliation) so optimistic local renders converge to server
180          truth without clobbering focus/selection. Called directly as
181          Idiomorph.morph(target, source) from the outline client script --
182          NOT registered as an htmx extension (htmx 4 renamed
183          defineExtension -> registerExtension; idiomorph-ext still uses
184          the old API). Spec: borge/structural-editing.borg client-eval. *)
185       HTML.script [HTML.src "/assets/idiomorph.min.js"] "";
186       (* Nav menus are pure HTML/CSS: exclusive open comes from the
187          native <details name=...> accordion attribute (menu/render),
188          so no menu JS here. Remaining script: outline summaries only
189          toggle collapse via the .ol-fold handle. *)
190       HTML.script [] "%s"
191         ("document.addEventListener('click',function(e){" ^
192          "var
      sum=e.target&&e.target.closest?e.target.closest('summary.ol-sum'):null;"
      ^
193
      "if(sum&&!(e.target.closest&&e.target.closest('.ol-fold'))){e.preventDefault();}"
      ^
194          "});");
195     ];
```

```
    HTML.body [] [body];
  ] in
  let s = Pure_html.to_string html_doc in
  let headers =
    Httpun.Headers.of_list
      ([("content-type", "text/html; charset=utf-8");
        ("content-length", string_of_int (String.length s))]
       @ Timing.timing_header ()
       @ security_headers ())
  in
  let resp = Httpun.Response.create ~headers status in
  safe_respond_with_string reqd resp s

let text_response (reqd : Httpun.Reqd.t) ?(status = `OK) ~content_type (s :
    string) : unit =
  let headers =
    Httpun.Headers.of_list
      ([("content-type", content_type);
        ("content-length", string_of_int (String.length s))]
       @ Timing.timing_header ()
       @ security_headers ())
  in
  let resp = Httpun.Response.create ~headers status in
  safe_respond_with_string reqd resp s

let json_response (reqd : Httpun.Reqd.t) ?(status = `OK) (j :
    Yojson.Safe.t) : unit =
  let s = Yojson.Safe.to_string j in
  text_response reqd ~status ~content_type:"application/json" s

(** Labeled emergency nav only -- NOT a product menu twin.
    Product chrome is Lisp (ui/nav). See image.system-browser. *)
let emergency_nav_nodes () : Pure_html.node list =
  [ HTML.span [HTML.class_ "emergency-banner"]
      [ txt "EMERGENCY NAV -- ui/nav missing or failed ? "
      ; HTML.a [HTML.href "/bindings?pkg=ui"] [txt "fix ui shell"]
      ; txt " ? "
      ; HTML.a [HTML.href "/"] [txt "home"]
      ; txt " ? "
      ; HTML.a [HTML.href "/login"] [txt "login"]
      ; txt " ? "
      ; HTML.a [HTML.href "/health"] [txt "health"]
      ]
  ]

let value_to_html_string (v : Pricklypear_lisp.Ast.value) : string =
  match Primitives.is_raw_html v with
  | Some s -> s
  | None ->
      (match v with
       | Pricklypear_lisp.Ast.VStr s -> s
       | _ -> Pricklypear_lisp.Reader.pp_value v)

let try_ui_fn (session : Session.t) (name : string) : string option =
  (try
      let r = Session.call_handler session name in
      match r with
      | Session.Handler_ok v -> (Some (value_to_html_string v))
      | Session.Handler_unbound ->
          Printf.eprintf "[http] try_ui_fn %s: unbound\n%!" name;
          None
      | Session.Handler_error msg ->
          Printf.eprintf "[http] try_ui_fn %s: %s\n%!" name msg;
          None
    with exn ->
      Printf.eprintf "[http] try_ui_fn %s failed: %s\n%!"
        name (Printexc.to_string exn);
      None)

(** Fragment / partial: no document chrome (htmx swap target). *)
let is_partials_path (path : string) : bool =
  String.length path >= 10 && String.sub path 0 10 = "/partials/"

```

```
267 let header_list (req : Httpun.Request.t) : (string * string) list =
268   Httpun.Headers.to_list req.headers
269   |> List.map (fun (k, v) -> (String.lowercase_ascii k, v))
270
271 let wants_fragment ~(path : string) ~(headers : (string * string) list) :
        bool =
272   if is_partials_path path then true
273   else
274     match List.assoc_opt "hx-request" headers with
275     | Some v when v <> "" && String.lowercase_ascii v <> "false" -> true
276     | _ -> false
277
278 (* ---------- Auth (single-image Option B: in-process sessions) ----------
        *)
279
280 let cookie_name = "pp_session"
281
282 (** Per-user subdomains (auth.user-subdomains). Empty PP_BASE_DOMAIN = off.
        *)
283 let base_domain () =
284   match Sys.getenv_opt "PP_BASE_DOMAIN" with
285   | Some s when String.trim s <> "" -> Some (String.lowercase_ascii
        (String.trim s))
286   | _ -> None
287
288 let host_header (headers : (string * string) list) : string option =
289   let rec find = function
290     | [] -> None
291     | (k, v) :: _ when String.lowercase_ascii k = "host" ->
292         let host =
293           match String.index_opt v ':' with
294           | Some i -> String.sub v 0 i
295           | None -> v
296         in
297         Some (String.lowercase_ascii host)
298     | _ :: rest -> find rest
299   in
300   find headers
301
302 type host_mode =
303   | Host_legacy
304   | Host_apex
305   | Host_garden of string
306   | Host_unknown
307
308 let username_dns_ok (name : string) : bool =
309   let n = String.length name in
310   n > 0 && n <= 63
311   && (match name.[0] with 'a' .. 'z' | '0' .. '9' -> true | _ -> false)
312   && (match name.[n - 1] with 'a' .. 'z' | '0' .. '9' -> true | _ -> false)
313   &&
314   let rec loop i =
315     if i >= n then true
316     else
317       match name.[i] with
318       | 'a' .. 'z' | '0' .. '9' | '-' -> loop (i + 1)
319       | _ -> false
320   in
321   loop 0
322
323 let classify_host (headers : (string * string) list) : host_mode =
324   match base_domain (), host_header headers with
325   | None, _ -> Host_legacy
326   | Some base, None -> Host_apex
327   | Some base, Some host ->
328       if host = base then Host_apex
329       else
330         let suffix = "." ^ base in
331         let hl = String.length host
332         and sl = String.length suffix in
333         if hl > sl && String.sub host (hl - sl) sl = suffix then
334           let name = String.sub host 0 (hl - sl) in
335           if username_dns_ok name && not (String.contains name '.') then
```

```
            Host_garden name
          else Host_unknown
        else Host_unknown

let garden_url ~(username : string) ~(path : string) : string =
  match base_domain () with
  | None -> path
  | Some base ->
      let p = if path = "" then "/" else path in
      "https://" ^ username ^ "." ^ base ^ p

(** Serialize request identity attach -- cotenant multi-user shares one
    Session.t; concurrent Alice/Bob would otherwise race branch/env. *)
let request_mutex = Eio.Mutex.create ()

let auth_disabled () : bool =
  match Sys.getenv_opt "PP_AUTH_DISABLED" with
  | Some ("1" | "true" | "TRUE" | "yes" | "YES") -> true
  | _ -> false

let has_prefix (path : string) (prefix : string) : bool =
  path = prefix
  || (String.length path > String.length prefix
      && String.sub path 0 (String.length prefix) = prefix
      && path.[String.length prefix] = '/')

let is_system_hard_public (path : string) : bool =
  path = "/login"
  || path = "/health"
  || path = "/logout"
  || path = "/welcome"
  || path = "/background"
  || path = "/roadmap"
  || path = "/signup"
  || path = "/guestbook"
  || path = "/book.pdf"
  || path = "/src.tgz"
  || path = "/code"
  || has_prefix path "/code"
  || path = "/assets"
  || has_prefix path "/assets"
  || path = "/.well-known/agent.json"
  || path = "/agent.txt"
  (* one-tree.tree-browser: the store-tree face renders for anonymous
     sessions CONFINED to public prefixes (data.shared, own garden,
     the anonymous lens's main store-tree) -- private paths are not
     listable (path_visible in Http_tree enforces it; POST /tree/edit
     stays private + anonymous-denied). *)
  || path = "/tree"
  || path = "/partials/tree"

(** Kernel paths Lisp cannot open to the world (auth.route-visibility). *)
let is_system_hard_private (path : string) : bool =
  let prefixes =
    [ "/admin"
    ; "/analytics"
    ; "/data"
    ; "/api"
    ; "/media"
    ; "/bindings"
    ; "/binding"
    ; "/diff"
    ; "/edit"
    ; "/repl"
    ; "/branches"
    ; "/types"
    ; "/pages"
    ; "/page"
    ; "/inspect"
    ; "/agent"
    ]
  in
  List.exists
```

```
    (fun p ->
      path = p
      || (String.length path > String.length p
          && String.sub path 0 (String.length p) = p
          && path.[String.length p] = '/'))
    prefixes
  || path = "/"

(* Back-compat name used only for hard public. *)
let is_public_path (path : string) : bool = is_system_hard_public path

type path_class =
  [ `Hard_public | `Hard_private | `User of string * string | `Unknown ]

(** Classify path against a routes table. The table is the per-request
    snapshot when called post-gate (image.per-request-context), or the
    live Session.routes when called under the request_mutex (pre-gate --
    attach has not run for THIS request, but the mutex excludes concurrent
    rehydrate, so live is correct there). Returns `User (handler,
    visibility). *)
let classify_path_with_routes
    (routes : (string, string * string) Hashtbl.t) (path : string) :
    path_class =
  if is_system_hard_public path then `Hard_public
  else if is_system_hard_private path then `Hard_private
  else
    match Hashtbl.find_opt routes path with
    | Some (handler, vis) -> `User (handler, vis)
    | None -> `Unknown

(** Classify path for the request gate.
    Returns [`Hard_public | `Hard_private | `User of string * string
    | `Unknown] where User is (handler, visibility). *)
let classify_path (session : Session.t) (path : string) : path_class =
  classify_path_with_routes (Session.routes session) path

let parse_cookies (headers : (string * string) list) : (string * string)
    list =
  match List.assoc_opt "cookie" headers with
  | None -> []
  | Some raw ->
      String.split_on_char ';' raw
      |> List.filter_map (fun part ->
             let part = String.trim part in
             match String.index_opt part '=' with
             | None -> None
             | Some i ->
                 let k = String.trim (String.sub part 0 i) in
                 let v =
                   String.trim
                     (String.sub part (i + 1) (String.length part - i - 1))
                 in
                 if k = "" then None else Some (k, v))

let cookie_value (headers : (string * string) list) (name : string)
  : string option =
  List.assoc_opt name (parse_cookies headers)

let b64_decode_opt (s : string) : string option =
  match Base64.decode ~pad:true s with
  | Ok raw -> Some raw
  | Error _ ->
      (match Base64.decode ~pad:false s with
       | Ok raw -> Some raw
       | Error _ -> None)

(** Parse "Basic base64(user:pass)" Authorization header. *)
let parse_basic_auth (headers : (string * string) list)
  : (string * string) option =
  match List.assoc_opt "authorization" headers with
  | None -> None
  | Some v ->
      let v = String.trim v in
```

```
      if String.length v < 7 then None
      else
        let scheme = String.lowercase_ascii (String.sub v 0 5) in
        if scheme <> "basic" then None
        else
          let b64 = String.trim (String.sub v 5 (String.length v - 5)) in
          match b64_decode_opt b64 with
          | None -> None
          | Some decoded ->
              (match String.index_opt decoded ':' with
               | None -> None
               | Some i ->
                   let u = String.sub decoded 0 i in
                   let p =
                     String.sub decoded (i + 1)
                       (String.length decoded - i - 1)
                   in
                   Some (u, p))

(** Parse an Authorization: Bearer <token> header. Returns the raw token. *)
let parse_bearer_auth (headers : (string * string) list) : string option =
  match List.assoc_opt "authorization" headers with
  | None -> None
  | Some v ->
      let v = String.trim v in
      if String.length v < 8 then None
      else
        let scheme = String.lowercase_ascii (String.sub v 0 6) in
        if scheme <> "bearer" then None
        else Some (String.trim (String.sub v 6 (String.length v - 6)))

(** Emit "; Secure" on the session cookie unless explicitly disabled via
    PP_COOKIE_SECURE=0/false, and only when a base domain is configured --
    prod (base domain set) is https, while dev plain HTTP (no domain)
    stays without Secure. *)
let cookie_secure () : bool =
  let forced_off =
    match Sys.getenv_opt "PP_COOKIE_SECURE" with
    | Some ("0" | "false" | "FALSE" | "False") -> true
    | _ -> false
  in
  (not forced_off) && base_domain () <> None

let set_cookie_header ~(token : string) : string =
  (* Path=/; HttpOnly; SameSite=Lax. PP_BASE_DOMAIN scopes cookie across
     subdomains. Secure when a base domain is set unless PP_COOKIE_SECURE
     forces it off (see cookie_secure). *)
  let domain =
    match base_domain () with
    | Some b -> "; Domain=." ^ b
    | None -> ""
  in
  let secure = if cookie_secure () then "; Secure" else "" in
  Printf.sprintf "%s=%s; Path=/; HttpOnly; SameSite=Lax; Max-Age=%d%s%s"
    cookie_name token (60 * 60 * 24 * 14) domain secure

let clear_cookie_header () : string =
  let domain =
    match base_domain () with
    | Some b -> "; Domain=." ^ b
    | None -> ""
  in
  Printf.sprintf
    "%s=; Path=/; HttpOnly; SameSite=Lax; Max-Age=0%s" cookie_name domain

let redirect_with_cookie (reqd : Httpun.Reqd.t) ~(location : string)
    ~(set_cookie : string) : unit =
  let headers =
    Httpun.Headers.of_list
      ([ ("location", location)
       ; ("set-cookie", set_cookie)
       ; ("content-length", "0") ]
       @ security_headers ())
```

```
  in
  let resp = Httpun.Response.create ~headers `See_other in
  safe_respond_with_string reqd resp ""

let unauthorized_api (reqd : Httpun.Reqd.t) : unit =
  json_response reqd ~status:`Unauthorized
    (`Assoc
       [ ("ok", `Bool false)
       ; ("error", `String "authentication required") ])

(** i-1787971251: derive the client source_ip from the connection sockaddr
    so auth_log rows can record where a login/token use came from.
    ADDR_UNIX paths are prefixed to stay distinguishable from IPs. *)
let source_ip_of_sockaddr (sa : Unix.sockaddr) : string =
  match sa with
  | Unix.ADDR_INET (addr, _) -> Unix.string_of_inet_addr addr
  | Unix.ADDR_UNIX path -> "unix:" ^ path

(** Resolve identity for this request: cookie session, then Basic auth.
    Returns None if unauthenticated. [?source_ip] is propagated into the
    auth_log rows written by verify_token / verify_password. *)
let resolve_identity (session : Session.t)
    ~(headers : (string * string) list) ?source_ip ()
  : Auth_store.user option =
  let store = Session.store session in
  match cookie_value headers cookie_name with
  | Some token when token <> "" ->
      let row = Auth_store.lookup_session store ~raw_token:token in
      (match row with
       | Some s ->
           (* Fetch the full user (user_type, parent_user_id) -- the session
              row only carries id/username/is_admin. *)
           let user_opt = Auth_store.get_user_by_id store s.user_id in
           (match user_opt with
            | Some u -> Some u
            | None -> None)
       | None ->
           (* Fall through to Basic if cookie stale. *)
           (match parse_basic_auth headers with
            | None -> None
            | Some (u, p) ->
                Auth_store.verify_password store ~username:u ~password:p
                  ?source_ip ()))
  | _ ->
      (* No cookie: try Bearer token, then Basic. *)
      (match parse_bearer_auth headers with
       | Some token when token <> "" ->
           Auth_store.verify_token store ~raw_token:token ?source_ip ()
       | _ ->
           (match parse_basic_auth headers with
            | None -> None
            | Some (u, p) ->
                Auth_store.verify_password store ~username:u ~password:p
                  ?source_ip ()))

let attach_from_user (session : Session.t) (user : Auth_store.user)
    ?(auth_token : string option = None)
    ?(checkout_personal_now = false) () : unit =
  Session.attach_identity session
    ~user_id:user.Auth_store.id
    ~username:user.Auth_store.username
    ~is_admin:user.Auth_store.is_admin
    ~auth_token
    ~checkout_personal_now
    ~user_type:user.Auth_store.user_type
    ~parent_user_id:user.Auth_store.parent_user_id
    ()

let fragment_response (reqd : Httpun.Reqd.t) ?(status = `OK) (html : string)
    : unit =
  text_response reqd ~status ~content_type:"text/html; charset=utf-8" html

let chrome ?nav_html ~(session : Session.t)
```

```
    ?(manifest = Session.manifest session) ~(title : string)
    (content : Pure_html.node list) : Pure_html.node =
  let nav_html =
    match nav_html with
    | Some _ as s -> s
    | None -> Session.ui_nav_html session
  in
  let nav =
    match nav_html with
    | Some html when String.trim html <> "" ->
        HTML.nav [] [txt ~raw:true "%s" html]
    | _ -> HTML.nav [HTML.class_ "emergency-nav"] (emergency_nav_nodes ())
  in
  (* Metadata reads the SNAPSHOT manifest when callers pass one
     (image.per-request-context), so a concurrent attach_anonymous
     cannot paint "user=anonymous" on an authed page's header. Callers
     outside the HTTP gate fall back to the live session manifest. *)
  HTML.div [] [
    HTML.header [] [
      HTML.span [HTML.class_ "brand"] [txt "Pricklypear"];
      HTML.span [HTML.class_ "meta"] [
        txt "user=%s  branch=%s  up=%ds"
          manifest.username
          manifest.branch_name
          (Session.uptime_seconds session);
      ];
      nav;
    ];
    HTML.main [] (
      (if String.trim title = "" then content
       else HTML.h1 [] [txt "%s" title] :: content)
    );
  ]

let chrome_lwt ~(session : Session.t) ~(title : string)
    (content : Pure_html.node list) : Pure_html.node =
  let nav_html = try_ui_fn session "ui/nav" in
  (chrome ?nav_html ~session ~title content)

(* Request-body cap: read_body accumulates the whole body in memory, so
   an unbounded read lets a client exhaust memory. Cap at PP_MAX_BODY_BYTES
   (default 1 MiB); on exceed, raise Body_too_large -- Http.launch maps it
   to a 413. Uploads use http_media.read_body_limited (5 MiB) unchanged. *)
exception Body_too_large

let max_body_bytes =
  match Sys.getenv_opt "PP_MAX_BODY_BYTES" with
  | Some s ->
      (match int_of_string_opt s with
       | Some n when n > 0 -> n
       | _ -> 1024 * 1024)
  | None -> 1024 * 1024

let read_body (reqd : Httpun.Reqd.t) : string =
  let body = Httpun.Reqd.request_body reqd in
  (* The on_read callback may run on the connection fiber, not the handler
     fiber -- so signal overflow via the promise instead of raising inside
     the callback, guaranteeing the exception surfaces in read_body's
     caller where launch's 413 handler can catch it. *)
  let promise, resolver = Eio.Promise.create () in
  let buf = Buffer.create 1024 in
  let rec sched () =
    Httpun.Body.Reader.schedule_read body
      ~on_eof:(fun () ->
        Eio.Promise.resolve resolver (`Ok (Buffer.contents buf)))
      ~on_read:(fun bs ~off ~len ->
        if Buffer.length buf + len > max_body_bytes then begin
          Eio.Promise.resolve resolver `Too_large
          (* Stop rescheduling: the oversized request is abandoned and
             answered with 413; the connection is closed after the reply. *)
        end else begin
          Buffer.add_string buf (Bigstringaf.substring bs ~off ~len);
          sched ()
```

```
        end)
  in
  sched ();
  match Eio.Promise.await promise with
  | `Ok s -> s
  | `Too_large -> raise Body_too_large

let url_decode (s : string) : string =
  let buf = Buffer.create (String.length s) in
  let len = String.length s in
  let rec loop i =
    if i >= len then Buffer.contents buf
    else match s.[i] with
    | '+' -> Buffer.add_char buf ' '; loop (i + 1)
    | '%' when i + 2 < len ->
        let hex = String.sub s (i + 1) 2 in
        (try
           let code = int_of_string ("0x" ^ hex) in
           Buffer.add_char buf (Char.chr code)
         with _ -> Buffer.add_char buf '%');
        loop (i + 3)
    | c -> Buffer.add_char buf c; loop (i + 1)
  in
  loop 0

(** application/x-www-form-urlencoded / query component encode. *)
let url_encode (s : string) : string =
  let buf = Buffer.create (String.length s * 2) in
  String.iter
    (function
      | ('A' .. 'Z' | 'a' .. 'z' | '0' .. '9' | '-' | '_' | '.' | '~') as c
     ->
          Buffer.add_char buf c
      | ' ' -> Buffer.add_string buf "%20"
      | c -> Buffer.add_string buf (Printf.sprintf "%%%02X" (Char.code c)))
    s;
  Buffer.contents buf

(** Same-origin relative path (+ optional query/hash) for post-login return.
    Rejects scheme-relative URLs, off-site targets, and control chars.
    Colons are allowed only in the query/fragment (e.g. url=https://?),
    never in the path (blocks scheme smuggling). *)
let safe_internal_next (candidate : string) : string option =
  let s = String.trim candidate in
  let len = String.length s in
  if len = 0 || len > 2048 then None
  else if s.[0] <> '/' then None
  else if len >= 2 && s.[1] = '/' then None
  else if String.contains s '\n' || String.contains s '\r' then None
  else
    let path_part =
      match String.index_opt s '?', String.index_opt s '#' with
      | Some q, Some h -> String.sub s 0 (min q h)
      | Some q, None -> String.sub s 0 q
      | None, Some h -> String.sub s 0 h
      | None, None -> s
    in
    if String.contains path_part ':' then None else Some s

let form_field (body : string) (name : string) : string option =
  let pairs = String.split_on_char '&' body in
  let rec find = function
    | [] -> None
    | p :: rest ->
        (match String.split_on_char '=' p with
         | k :: vs ->
             if url_decode k = name then
               Some (url_decode (String.concat "=" vs))
             else find rest
         | _ -> find rest)
  in
  find pairs

```

```
let parse_form_body (body : string) : (string * string) list =
  if body = "" then []
  else
    String.split_on_char '&' body
    |> List.filter_map (fun p ->
           if p = "" then None
           else
             match String.split_on_char '=' p with
             | k :: vs ->
                 Some (url_decode k, url_decode (String.concat "=" vs))
             | _ -> None)

(** Merge query + form params; form fields win on key conflict (it33). *)
let merge_params ~(query : (string * string) list)
    ~(form : (string * string) list) : (string * string) list =
  let tbl = Hashtbl.create 16 in
  List.iter (fun (k, v) -> Hashtbl.replace tbl k v) query;
  List.iter (fun (k, v) -> Hashtbl.replace tbl k v) form;
  Hashtbl.fold (fun k v acc -> (k, v) :: acc) tbl []

let parse_query (target : string) : (string * string) list =
  match String.index_opt target '?' with
  | None -> []
  | Some i ->
      let q = String.sub target (i + 1) (String.length target - i - 1) in
      String.split_on_char '&' q
      |> List.filter_map (fun p ->
             match String.split_on_char '=' p with
             | k :: vs ->
                 Some (url_decode k, url_decode (String.concat "=" vs))
             | _ -> None)

let query_param (target : string) (key : string) : string option =
  List.assoc_opt key (parse_query target)

let max_user_body_bytes = 64 * 1024

let redirect_response (reqd : Httpun.Reqd.t) (location : string) : unit =
  let headers =
    Httpun.Headers.of_list
      ([ ("location", location)
       ; ("content-length", "0")
       ] @ Timing.timing_header ()
       @ security_headers ())
  in
  let resp = Httpun.Response.create ~headers `See_other in
  safe_respond_with_string reqd resp ""

let require_login_redirect (reqd : Httpun.Reqd.t) ~(path : string)
    ~(target : string) : unit =
  if String.length path >= 5 && String.sub path 0 5 = "/api/" then
    unauthorized_api reqd
  else if path = "/" then
    (* Public host: anonymous workspace root ? welcome landing. *)
    redirect_response reqd "/welcome"
  else
    match safe_internal_next target with
    | Some next when next <> "/login" && not (String.starts_with
     ~prefix:"/login?" next)
      ->
        redirect_response reqd ("/login?next=" ^ url_encode next)
    | _ -> redirect_response reqd "/login"

let login_form_page ?(error = "") ?(next = "") (reqd : Httpun.Reqd.t) :
    unit =
  let err =
    if error = "" then HTML.div [] []
    else HTML.p [HTML.class_ "err"] [txt "%s" error]
  in
  let next =
    let from_arg = String.trim next in
    let candidate =
      if from_arg <> "" then from_arg
```

```
      else
        let target = (Httpun.Reqd.request reqd).Httpun.Request.target in
        Option.value (query_param target "next") ~default:""
    in
    match safe_internal_next candidate with Some n -> n | None -> ""
  in
  let next_field =
    if next = "" then HTML.div [] []
    else
      HTML.input
        [ HTML.type_ "hidden"
        ; HTML.name "next"
        ; HTML.value "%s" next
        ]
  in
  let body =
    HTML.html [HTML.lang "en"] [
      HTML.head [] [
        HTML.title [] "Pricklypear -- Sign in";
        HTML.meta [HTML.charset "utf-8"];
        HTML.meta [HTML.name "viewport";
                   HTML.content "width=device-width, initial-scale=1"];
        HTML.style [] "%s"
          "body{font-family:ui-monospace,Menlo,monospace;max-width:28rem;
           margin:4rem auto;padding:1rem;background:#f4f1ea;color:#1a1a1a}
           .card{border:2px solid #1a1a1a;background:#fff;padding:1.25rem}
           h1{font-size:1.25rem;margin:0 0 1rem}
           label{display:block;margin:.75rem 0 .25rem;font-size:.85rem}

    input[type=text],input[type=password]{width:100%;box-sizing:border-box;
           border:1px solid #1a1a1a;padding:.4rem;font:inherit}
           button{margin-top:1rem;border:2px solid
    #1a1a1a;background:#1a1a1a;
           color:#fff;padding:.45rem .9rem;font:inherit;cursor:pointer}
           .err{color:#8b0000;margin:.5rem 0}
           .muted{color:#555;font-size:.85rem;margin-top:1rem}";
      ];
      HTML.body [] [
        HTML.div [HTML.class_ "card"] [
          HTML.h1 [] [txt "Sign in"];
          err;
          HTML.form [HTML.method_ `POST; HTML.action "/login"] [
            next_field;
            HTML.label [HTML.for_ "username"] [txt "Username"];
            HTML.input [HTML.type_ "text"; HTML.name "username";
                        HTML.id "username"; HTML.required;
                        HTML.autocomplete `username];
            HTML.label [HTML.for_ "password"] [txt "Password"];
            HTML.input [HTML.type_ "password"; HTML.name "password";
                        HTML.id "password"; HTML.required;
                        HTML.autocomplete `current_password];
            HTML.button [HTML.type_ "submit"] [txt "Log in"];
          ];
          HTML.p [HTML.class_ "muted"] [
            txt "Your garden: username/personal after login.";
          ];
        ];
      ];
    ]
  in
  html_response reqd body

let post_login (session : Session.t) ?source_ip (reqd : Httpun.Reqd.t) :
    unit =
  let body = read_body reqd in
  let username =
    Option.value (form_field body "username") ~default:"" |> String.trim
  in
  let password = Option.value (form_field body "password") ~default:"" in
  let next_raw = Option.value (form_field body "next") ~default:"" in
  let next_ok = safe_internal_next next_raw in
  if username = "" || password = "" then begin
    login_form_page ~error:"Invalid username or password"
```

```
      ?next:(match next_ok with Some n -> Some n | None -> None) reqd;
    ()
  end else
    let store = Session.store session in
    let user_opt =
      Auth_store.verify_password store ~username ~password ?source_ip ()
    in
    match user_opt with
    | None ->
        login_form_page ~error:"Invalid username or password"
          ?next:(match next_ok with Some n -> Some n | None -> None) reqd;
        ()
    | Some user ->
        let minted =
          Auth_store.mint_session store ~user_id:user.Auth_store.id
            ?source_ip ()
        in
        (match minted with
         | Error e ->
             login_form_page ~error:("Login failed: " ^ e)
               ?next:(match next_ok with Some n -> Some n | None -> None) reqd;
             ()
         | Ok token ->
             let () =
               attach_from_user session user ~auth_token:(Some token)
                 ~checkout_personal_now:true ()
             in
             let location =
               match next_ok with
               | Some n -> n
               | None ->
                   (match base_domain () with
                    | Some base ->
                        (* Prefer relative if already on garden host; else absolute. *)
                        let headers = header_list (Httpun.Reqd.request reqd) in
                        (match classify_host headers with
                         | Host_garden g when g = user.Auth_store.username -> "/"
                         | _ ->
                             (* garden_url is https -- never bounce a fresh
                                session cookie over plain http. *)
                             garden_url ~username:user.Auth_store.username
                               ~path:"/")
                    | None -> "/")
             in
             redirect_with_cookie reqd ~location
               ~set_cookie:(set_cookie_header ~token);
             ())

let post_logout (session : Session.t) (reqd : Httpun.Reqd.t) : unit =
  let headers = header_list (Httpun.Reqd.request reqd) in
  let token = cookie_value headers cookie_name in
  let store = Session.store session in
  let () =
    match token with
    | Some t when t <> "" -> Auth_store.revoke_session store ~raw_token:t ()
    | _ -> ()
  in
  Session.clear_auth_token session;
  redirect_with_cookie reqd ~location:"/login"
    ~set_cookie:(clear_cookie_header ());
  ()

(** One-shot display of a freshly issued API token (never stored in URL). *)
let admin_token_once_page ~(session : Session.t) ~(reqd : Httpun.Reqd.t)
    ~(username : string) ~(raw : string) : unit =
  html_response reqd ~session
    (chrome ~session ~title:"API token" [
       HTML.div [HTML.class_ "card"] [
         HTML.h2 [] [txt "Token issued"];
```

```
        HTML.p [] [
          txt "Copy now -- shown once. User: ";
          HTML.code [] [txt "%s" username];
        ];
        HTML.p [] [HTML.code [] [txt "%s" raw]];
        HTML.p [HTML.class_ "muted"] [
          txt "Use as Authorization: Bearer <token> for /api/eval.";
        ];
        HTML.p [] [HTML.a [HTML.href "/admin/users"] [txt "Back to
    users"]];
      ];
    ])

let admin_err ~(session : Session.t) ~(reqd : Httpun.Reqd.t) ~title msg =
  html_response reqd ~session ~status:`Bad_request
    (chrome ~session ~title [
       HTML.p [HTML.class_ "err"] [txt "%s" msg];
       HTML.p [] [HTML.a [HTML.href "/admin/users"] [txt "Back"]];
     ]);
  ()

let admin_users_page (session : Session.t) (reqd : Httpun.Reqd.t)
  : unit =
  if not (Session.is_admin session) then begin
    html_response reqd ~session ~status:`Forbidden
      (chrome ~session ~title:"Forbidden" [
         HTML.p [HTML.class_ "err"] [txt "admin only"];
       ]);
    ()
  end else
    let store = Session.store session in
    let users = Auth_store.list_users store in
    (* Per-user promote flag + API tokens (admin agent/token surface --
       BORGE-1785120601). *)
    let user_extras =
      List.map
        (fun (u : Auth_store.user) ->
          let promote =
            if u.Auth_store.user_type = "human" then
              Auth_store.get_allow_agent_promote store ~user_id:u.id
            else false
          in
          let tokens = Auth_store.list_tokens store ~user_id:u.id in
          (u, promote, tokens))
        users
    in
    let human_names =
      List.filter_map
        (fun (u : Auth_store.user) ->
          if u.user_type = "human" then Some u.username else None)
        users
    in
    let rows =
      List.map
        (fun ((u : Auth_store.user), promote, _tokens) ->
          let promote_cell =
            if u.user_type <> "human" then
              HTML.td [] [txt "--"]
            else
              HTML.td [] [
                HTML.form
                  [ HTML.method_ `POST; HTML.action "/admin/users" ]
                  [ HTML.input
                      [ HTML.type_ "hidden"; HTML.name "action";
                        HTML.value "set_allow_agent_promote" ]
                  ; HTML.input
                      [ HTML.type_ "hidden"; HTML.name "user_id";
                        HTML.value "%s" u.id ]
                  ; HTML.input
                      [ HTML.type_ "hidden"; HTML.name "value";
                        HTML.value (if promote then "0" else "1") ]
                  ; HTML.button [HTML.type_ "submit"]
                      [ txt "%s"
```

```
                        (if promote then "on (click=off)"
                         else "off (click=on)") ]
                ]
            ]
        in
        HTML.tr [] [
          HTML.td [] [txt "%s" u.username];
          HTML.td [] [txt "%s" u.user_type];
          HTML.td [] [txt "%s" (if u.is_admin then "yes" else "no")];
          promote_cell;
          HTML.td [] [HTML.code [] [txt "%s" u.id]];
          HTML.td [] [
            HTML.form
              [ HTML.method_ `POST; HTML.action "/admin/users" ]
              [ HTML.input
                  [ HTML.type_ "hidden"; HTML.name "action";
                    HTML.value "revoke_sessions" ]
              ; HTML.input
                  [ HTML.type_ "hidden"; HTML.name "user_id";
                    HTML.value "%s" u.id ]
              ; HTML.button [HTML.type_ "submit"] [txt "Revoke sessions"]
              ];
            HTML.form
              [ HTML.method_ `POST; HTML.action "/admin/users" ]
              [ HTML.input
                  [ HTML.type_ "hidden"; HTML.name "action";
                    HTML.value "issue_token" ]
              ; HTML.input
                  [ HTML.type_ "hidden"; HTML.name "user_id";
                    HTML.value "%s" u.id ]
              ; HTML.input
                  [ HTML.type_ "text"; HTML.name "label";
                    HTML.placeholder "label (optional)" ]
              ; HTML.button [HTML.type_ "submit"] [txt "Issue token"]
              ];
          ];
        ])
      user_extras
  in
  let token_sections =
    List.filter_map
      (fun ((u : Auth_store.user), _p, tokens) ->
        match tokens with
        | [] -> None
        | ts ->
            let trows =
              List.map
                (fun (tm : Auth_store.token_meta) ->
                  let lab =
                    match tm.label with Some l -> l | None -> "(none)"
                  in
                  let last =
                    match tm.last_used_at with
                    | Some s -> s
                    | None -> "never"
                  in
                  HTML.tr [] [
                    HTML.td [] [HTML.code [] [txt "%s" tm.id]];
                    HTML.td [] [txt "%s" lab];
                    HTML.td [] [txt "?%s" tm.token_last4];
                    HTML.td [] [txt "%s" tm.created_at];
                    HTML.td [] [txt "%s" last];
                    HTML.td [] [
                      HTML.form
                        [ HTML.method_ `POST; HTML.action "/admin/users" ]
                        [ HTML.input
                            [ HTML.type_ "hidden"; HTML.name "action";
                              HTML.value "rotate_token" ]
                        ; HTML.input
                            [ HTML.type_ "hidden"; HTML.name
    "credential_id";
                              HTML.value "%s" tm.id ]
                        ; HTML.input
```

```
                            [ HTML.type_ "hidden"; HTML.name "user_id";
                              HTML.value "%s" u.id ]
                        ; HTML.button [HTML.type_ "submit"] [txt "Rotate"]
                        ];
                      HTML.form
                        [ HTML.method_ 'POST; HTML.action "/admin/users" ]
                        [ HTML.input
                            [ HTML.type_ "hidden"; HTML.name "action";
                              HTML.value "revoke_token" ]
                        ; HTML.input
                            [ HTML.type_ "hidden"; HTML.name
    "credential_id";
                              HTML.value "%s" tm.id ]
                        ; HTML.button [HTML.type_ "submit"] [txt "Revoke"]
                        ];
                    ];
                  ])
              ts
          in
          Some
            (HTML.div [HTML.class_ "card"] [
               HTML.h3 [] [txt "Tokens: %s" u.username];
               HTML.table [] [
                 HTML.thead [] [
                   HTML.tr [] [
                     HTML.th [] [txt "id"];
                     HTML.th [] [txt "label"];
                     HTML.th [] [txt "last4"];
                     HTML.th [] [txt "created"];
                     HTML.th [] [txt "last used"];
                     HTML.th [] [txt "actions"];
                   ];
                 ];
                 HTML.tbody [] trows;
               ];
             ]))
      user_extras
  in
  let parent_options =
    match human_names with
    | [] -> [ HTML.option [HTML.value ""] "%s" "(no humans)" ]
    | names ->
        List.map
          (fun n -> HTML.option [HTML.value "%s" n] "%s" n)
          names
  in
  let body =
    chrome ~session ~title:"Users" [
      HTML.div [HTML.class_ "card"] [
        HTML.h2 [] [txt "Users"];
        HTML.p [HTML.class_ "muted"] [
          txt
            "Agents, API tokens, and allow_agent_promote. Raw tokens are \
             shown once after issue/rotate.";
        ];
        HTML.table [] [
          HTML.thead [] [
            HTML.tr [] [
              HTML.th [] [txt "username"];
              HTML.th [] [txt "type"];
              HTML.th [] [txt "admin"];
              HTML.th [] [txt "agent promote"];
              HTML.th [] [txt "id"];
              HTML.th [] [txt "actions"];
            ];
          ];
          HTML.tbody [] rows;
        ];
      ];
      HTML.div [] token_sections;
      HTML.div [HTML.class_ "card"] [
        HTML.h2 [] [txt "Create user"];
        HTML.form [HTML.method_ 'POST; HTML.action "/admin/users"] [
```

```
            HTML.input
              [ HTML.type_ "hidden"; HTML.name "action";
                HTML.value "create" ];
            HTML.p [] [
              HTML.label [] [txt "username "];
              HTML.input [HTML.type_ "text"; HTML.name "username";
    HTML.required];
            ];
            HTML.p [] [
              HTML.label [] [txt "password "];
              HTML.input [HTML.type_ "password"; HTML.name "password";
    HTML.required];
            ];
            HTML.p [] [
              HTML.label [] [
                HTML.input [HTML.type_ "checkbox"; HTML.name "is_admin";
    HTML.value "1"];
                txt " is_admin";
              ];
            ];
            HTML.button [HTML.type_ "submit"] [txt "Create"];
          ];
        ];
        HTML.div [HTML.class_ "card"] [
          HTML.h2 [] [txt "Create agent"];
          HTML.p [HTML.class_ "muted"] [
            txt "Creates <parent>/agent (idempotent). No password -- use
    API tokens.";
          ];
          HTML.form [HTML.method_ `POST; HTML.action "/admin/users"] [
            HTML.input
              [ HTML.type_ "hidden"; HTML.name "action";
                HTML.value "create_agent" ];
            HTML.p [] [
              HTML.label [] [txt "parent human "];
              HTML.select [HTML.name "parent_username"; HTML.required]
                parent_options;
            ];
            HTML.button [HTML.type_ "submit"] [txt "Create agent"];
          ];
        ];
        HTML.p [] [
          HTML.form [HTML.method_ `POST; HTML.action "/logout"] [
            HTML.button [HTML.type_ "submit"] [txt "Log out"];
          ];
        ];
      ]
    in
    html_response reqd ~session body;
    ()

let post_admin_users (session : Session.t) (reqd : Httpun.Reqd.t)
  : unit =
  if not (Session.is_admin session) then begin
    html_response reqd ~session ~status:`Forbidden
      (chrome ~session ~title:"Forbidden" [
         HTML.p [HTML.class_ "err"] [txt "admin only"];
       ]);
    ()
  end else
    let body = read_body reqd in
    let action =
      Option.value (form_field body "action") ~default:"create"
      |> String.trim
    in
    let store = Session.store session in
    let field name =
      Option.value (form_field body name) ~default:"" |> String.trim
    in
    match action with
    | "revoke_sessions" ->
        let user_id = field "user_id" in
        if user_id = "" then
```

```
        admin_err ~session ~reqd ~title:"Revoke sessions" "user_id
    required"
      else
        let n = Auth_store.revoke_all_sessions store ~user_id in
        let () =
          Auth_store.log_auth store ~user_id ~kind:"admin_revoke_sessions"
            ~success:true ()
        in
        ignore n;
        redirect_response reqd "/admin/users";
        ()
  | "create_agent" ->
      let parent = field "parent_username" in
      if parent = "" then
        admin_err ~session ~reqd ~title:"Create agent" "parent_username
    required"
      else
        let r = Auth_store.create_agent store ~parent_username:parent in
        (match r with
         | Error e -> admin_err ~session ~reqd ~title:"Create agent" e
         | Ok _u ->
             redirect_response reqd "/admin/users";
             ())
  | "issue_token" ->
      let user_id = field "user_id" in
      let label =
        match field "label" with "" -> None | s -> Some s
      in
      if user_id = "" then
        admin_err ~session ~reqd ~title:"Issue token" "user_id required"
      else
        let uopt = Auth_store.get_user_by_id store user_id in
        (match uopt with
         | None ->
             admin_err ~session ~reqd ~title:"Issue token" "no such user"
         | Some u ->
             let r =
               Auth_store.issue_token store ~user_id ~label:label ()
             in
             (match r with
              | Error e -> admin_err ~session ~reqd ~title:"Issue token" e
              | Ok raw ->
                  admin_token_once_page ~session ~reqd
                    ~username:u.username ~raw;
                  ()))
  | "revoke_token" ->
      let credential_id = field "credential_id" in
      if credential_id = "" then
        admin_err ~session ~reqd ~title:"Revoke token"
          "credential_id required"
      else
        let r = Auth_store.revoke_token store ~credential_id in
        (match r with
         | Error e -> admin_err ~session ~reqd ~title:"Revoke token" e
         | Ok () ->
             redirect_response reqd "/admin/users";
             ())
  | "rotate_token" ->
      let credential_id = field "credential_id" in
      let user_id = field "user_id" in
      let label =
        match field "label" with "" -> None | s -> Some s
      in
      if credential_id = "" || user_id = "" then
        admin_err ~session ~reqd ~title:"Rotate token"
          "credential_id and user_id required"
      else
        let uopt = Auth_store.get_user_by_id store user_id in
        (match uopt with
         | None ->
             admin_err ~session ~reqd ~title:"Rotate token" "no such user"
         | Some u ->
             let r =
```

```
                match label with
                | Some s ->
                    Auth_store.rotate_token store ~credential_id ~user_id
                      ~label:s ()
                | None ->
                    Auth_store.rotate_token store ~credential_id ~user_id
    ()
              in
              (match r with
               | Error e -> admin_err ~session ~reqd ~title:"Rotate token"
    e
               | Ok raw ->
                   admin_token_once_page ~session ~reqd
                     ~username:u.username ~raw;
                   ()))
    | "set_allow_agent_promote" ->
        let user_id = field "user_id" in
        let value =
          match field "value" with
          | "1" | "on" | "true" | "yes" -> true
          | _ -> false
        in
        if user_id = "" then
          admin_err ~session ~reqd ~title:"Agent promote" "user_id required"
        else
          let r =
            Auth_store.set_allow_agent_promote store ~user_id ~value
          in
          (match r with
           | Error e -> admin_err ~session ~reqd ~title:"Agent promote" e
           | Ok () ->
               redirect_response reqd "/admin/users";
               ())
    | "create" | _ ->
        let username = field "username" in
        let password = Option.value (form_field body "password")
    ~default:"" in
        let is_admin =
          match form_field body "is_admin" with
          | Some ("1" | "on" | "true") -> true
          | _ -> false
        in
        if username = "" || password = "" then
          admin_err ~session ~reqd ~title:"Create user"
            "username and password required"
        else
          let created = Auth_store.create_user store ~username ~is_admin in
          (match created with
           | Error e -> admin_err ~session ~reqd ~title:"Create user" e
           | Ok user ->
               let pw =
                 Auth_store.set_password store ~user_id:user.Auth_store.id
                   ~password
               in
               (match pw with
                | Error e ->
                    html_response reqd ~session
    ~status:`Internal_server_error
                      (chrome ~session ~title:"Create user" [
                         HTML.p [HTML.class_ "err"] [txt "password: %s" e];
                       ]);
                    ()
                | Ok () ->
                    let () =
                      Auth_store.log_auth store ~user_id:user.Auth_store.id
                        ~kind:"admin_create_user" ~success:true ()
                    in
                    redirect_response reqd "/admin/users";
                    ()))

(** i-1787971303: crypto-random hex string of n bytes (Mirage_crypto_rng,
    never the MT19937 Random state) -- used for minted multipart
    boundaries. *)
```

```
let random_hex_bytes (n : int) : string =
  let b = Mirage_crypto_rng.generate n in
  let buf = Buffer.create (n * 2) in
  String.iter
    (fun c -> Buffer.add_string buf (Printf.sprintf "%02x" (Char.code c)))
    b;
  Buffer.contents buf

(** Generate a MIME boundary that does not appear in any part body. *)
let fresh_multipart_boundary (bodies : string list) : string =
  let rec try_n n =
    if n > 8 then
      raise
        (Pricklypear_lisp.Ast.Interp_error
           (Pricklypear_lisp.Ast.User_error
              ("multipart: could not choose a unique boundary",
    "bad-args")))
    else
      let b = "pp" ^ random_hex_bytes 8 in
      if List.exists (fun body ->
           let rec find i =
             let bl = String.length b and n = String.length body in
             if i + bl > n then false
             else if String.sub body i bl = b then true
             else find (i + 1)
           in
           find 0)
         bodies
      then try_n (n + 1)
      else b
  in
  try_n 0

(** Frame a multipart/mixed body (eager v1). Peer-gone on write is
    swallowed.
    Streaming write+flush per part is available later; the reactive payoff
    is
    multiple targeted parts in one response. Effect-safe: no Eio.Promise
    await on the handler fiber (Continuation_already_resumed under
    Prim_call). *)
let multipart_response (reqd : Httpun.Reqd.t)
    (mp : Prims_pure.mime_multipart) : unit =
  let bodies = List.map (fun (p : Prims_pure.mime_part) -> p.body) mp.parts
    in
  let boundary = fresh_multipart_boundary bodies in
  let buf = Buffer.create 256 in
  List.iter
    (fun (p : Prims_pure.mime_part) ->
      Buffer.add_string buf ("--" ^ boundary ^ "\r\n");
      Buffer.add_string buf ("Content-Type: " ^ p.content_type ^ "\r\n");
      (match p.target with
       | Some t -> Buffer.add_string buf ("HX-Target: " ^ t ^ "\r\n")
       | None -> ());
      (match p.swap with
       | Some s -> Buffer.add_string buf ("HX-Swap: " ^ s ^ "\r\n")
       | None -> ());
      (match p.id with
       | Some i -> Buffer.add_string buf ("HX-Part-ID: " ^ i ^ "\r\n")
       | None -> ());
      Buffer.add_string buf "\r\n";
      Buffer.add_string buf p.body;
      Buffer.add_string buf "\r\n")
    mp.parts;
  Buffer.add_string buf ("--" ^ boundary ^ "--\r\n");
  let body = Buffer.contents buf in
  let headers =
    let base =
      [ ( "content-type"
        , "multipart/mixed; boundary=" ^ boundary )
      ; ("cache-control", "no-cache")
      ; ("content-length", string_of_int (String.length body))
      ]
    in
```

```
    let base =
      match mp.envelope_target with
      | Some t -> ("hx-target", t) :: base
      | None -> base
    in
    let base =
      match mp.envelope_swap with
      | Some s -> ("hx-swap", s) :: base
      | None -> base
    in
    Httpun.Headers.of_list base
  in
  let resp = Httpun.Response.create ~headers `OK in
  safe_respond_with_string reqd resp body

let respond_handler_value
    ~(session : Session.t)
    ~(reqd : Httpun.Reqd.t)
    ~(path : string)
    ~(fragment : bool)
    (value : Pricklypear_lisp.Ast.value) : unit =
  (* Pull the per-request snapshot manifest (if bound, sync via Lwt.get)
     so chrome renders this request's identity, not whatever a concurrent
     attach flipped. *)
  let manifest =
    match Request_context.get () with
    | Some ctx -> Request_context.manifest ctx
    | None -> Session.manifest session
  in
  match Primitives.is_redirect value with
  | Some loc -> redirect_response reqd loc
  | None ->
      (match Primitives.is_status value with
       | Some (code, body) ->
           (* Handler signalled a non-200 status (e.g. op-seq 409).
              Respond with the status + body as a fragment/HTML. *)
           let status =
             match code with
             | 409 -> `Conflict
             | 400 -> `Bad_request
             | 403 -> `Forbidden
             | 404 -> `Not_found
             | 500 -> `Internal_server_error
             | _ -> `Bad_request
           in
           if fragment then fragment_response reqd ~status body
           else
             html_response reqd ~session ~status
               (txt ~raw:true "%s" body)
       | None ->
      (match Primitives.is_multipart value with
       | Some mp -> multipart_response reqd mp
       | None ->
           if fragment then
             let html =
               match Primitives.is_raw_html value with
               | Some raw -> raw
               | None ->
                   (match value with
                    | Pricklypear_lisp.Ast.VStr s -> s
                    | _ -> Pricklypear_lisp.Reader.pp_value value)
             in
             fragment_response reqd html
           else
             let body =
               match Primitives.is_raw_html value with
               | Some raw_html ->
                   chrome ~manifest ~session ~title:path
                     [ txt ~raw:true "%s" raw_html ]
               | None ->
                   (match value with
                    | Pricklypear_lisp.Ast.VStr s ->
                        chrome ~manifest ~session ~title:path
```

```
                        [ HTML.p [] [txt "%s" s] ]
                  | _ ->
                      let rendered =
                        Pricklypear_lisp.Reader.pp_value value
                      in
                      chrome ~manifest ~session ~title:path [
                        HTML.pre [] [txt "%s" rendered];
                      ])
            in
            html_response reqd ~session body))

(** /settings/keys -- per-user LLM provider key (auth.user-llm-keys). *)
let settings_keys_page (session : Session.t) (reqd : Httpun.Reqd.t)
  : unit =
  let username = Session.username session in
  if username = "anonymous" then begin
    redirect_response reqd "/login";
    ()
  end else
    let user_opt =
      Auth_store.get_user_by_username (Session.store session) username
    in
    match user_opt with
    | None ->
        html_response reqd ~session ~status:`Unauthorized
          (chrome ~session ~title:"Keys" [
             HTML.p [HTML.class_ "err"] [txt "not signed in"];
           ]);
        ()
    | Some user ->
        let meta =
          Auth_store.get_llm_key_meta (Session.store session)
            ~user_id:user.Auth_store.id
        in
        let status_line =
          match meta with
          | None ->
              if Llm_client.api_key () <> "" then
                "No personal key. Operator LLM_API_KEY env is set (fallback)."
              else
                "No personal key and no LLM_API_KEY env."
          | Some m ->
              Printf.sprintf "Configured ? last4=%s%s%s"
                m.last4
                (match m.base with Some b -> " ? base=" ^ b | None -> "")
                (match m.model with Some mo -> " ? model=" ^ mo | None -> "")
        in
        let body =
          chrome ~session ~title:"LLM keys" [
            HTML.h1 [] [txt "LLM keys"];
            HTML.p [HTML.class_ "muted"] [
              txt "Bring your own provider key. Never shown again after save -- only last4.";
            ];
            HTML.p [] [txt "%s" status_line];
            HTML.div [HTML.class_ "card"] [
              HTML.form [HTML.method_ `POST; HTML.action "/settings/keys"] [
                HTML.input [HTML.type_ "hidden"; HTML.name "action"; HTML.value "set"];
                HTML.p [] [
                  HTML.label [] [txt "API key"];
                  HTML.input [HTML.type_ "password"; HTML.name "api_key"; HTML.required];
                ];
                HTML.p [] [
                  HTML.label [] [txt "Base URL (optional)"];
                  HTML.input [HTML.type_ "text"; HTML.name "base";
                              HTML.placeholder "https://api.synthetic.new/openai/v1"];
                ];
                HTML.p [] [
```

```
                  HTML.label [] [txt "Model (optional)"];
                  HTML.input [HTML.type_ "text"; HTML.name "model"];
                ];
                HTML.button [HTML.type_ "submit"] [txt "Save key"];
              ];
            ];
            HTML.div [HTML.class_ "card"] [
              HTML.form [HTML.method_ `POST; HTML.action "/settings/keys"] [
                HTML.input [HTML.type_ "hidden"; HTML.name "action";
    HTML.value "clear"];
                HTML.button [HTML.type_ "submit"] [txt "Clear key"];
              ];
            ];
          ]
        in
        html_response reqd ~session body;
        ()

let post_settings_keys (session : Session.t) (reqd : Httpun.Reqd.t)
  : unit =
  let username = Session.username session in
  if username = "anonymous" then begin
    redirect_response reqd "/login";
    ()
  end else
    let user_opt =
      Auth_store.get_user_by_username (Session.store session) username
    in
    match user_opt with
    | None ->
        redirect_response reqd "/login";
        ()
    | Some user ->
        let body = read_body reqd in
        let action =
          Option.value (form_field body "action") ~default:"set" |>
    String.trim
        in
        if action = "clear" then
          let _r =
            Auth_store.clear_llm_key (Session.store session)
              ~user_id:user.Auth_store.id
          in
          redirect_response reqd "/settings/keys";
          ()
        else
          let api_key =
            Option.value (form_field body "api_key") ~default:"" |>
    String.trim
          in
          let base =
            match form_field body "base" with
            | Some s when String.trim s <> "" -> Some (String.trim s)
            | _ -> None
          in
          let model =
            match form_field body "model" with
            | Some s when String.trim s <> "" -> Some (String.trim s)
            | _ -> None
          in
          let r =
            Auth_store.set_llm_key (Session.store session)
              ~user_id:user.Auth_store.id ~api_key ~base ~model ()
          in
          (match r with
           | Error e ->
               html_response reqd ~session ~status:`Bad_request
                 (chrome ~session ~title:"LLM keys" [
                    HTML.p [HTML.class_ "err"] [txt "%s" e];
                    HTML.p [] [HTML.a [HTML.href "/settings/keys"] [txt
    "Back"]];
                  ]);
              ()
```

```
          | Ok _ ->
              redirect_response reqd "/settings/keys";
              ())

let settings_ssh_page (session : Session.t) (reqd : Httpun.Reqd.t)
  : unit =
  let username = Session.username session in
  if username = "anonymous" then begin
    redirect_response reqd "/login";
    ()
  end else
    let user_opt =
      Auth_store.get_user_by_username (Session.store session) username
    in
    match user_opt with
    | None ->
        redirect_response reqd "/login";
        ()
    | Some user ->
        let keys =
          Auth_store.list_ssh_keys (Session.store session)
            ~user_id:user.Auth_store.id
        in
        let key_rows =
          List.map
            (fun (k : Auth_store.ssh_key_meta) ->
              let label =
                match k.label with Some l -> l | None -> ""
              in
              HTML.tr [] [
                HTML.td [] [HTML.code [] [txt "%s" k.fingerprint]];
                HTML.td [] [txt "%s" label];
                HTML.td [] [
                  HTML.form
                    [ HTML.method_ `POST; HTML.action "/settings/ssh" ]
                    [ HTML.input
                        [ HTML.type_ "hidden"; HTML.name "action";
                          HTML.value "revoke" ];
                      HTML.input
                        [ HTML.type_ "hidden"; HTML.name "id";
                          HTML.value "%s" k.id ];
                      HTML.button [HTML.type_ "submit"] [txt "Revoke"];
                    ];
                ];
              ])
            keys
        in
        let body =
          chrome ~session ~title:"SSH keys" [
            HTML.h1 [] [txt "SSH keys"];
            HTML.p [HTML.class_ "muted"] [
              txt "Register an OpenSSH public key for the REPL door
    (auth.user-ssh-repl).";
            ];
            HTML.div [HTML.class_ "card"] [
              HTML.h2 [] [txt "Your keys"];
              (if keys = [] then
                 HTML.p [HTML.class_ "muted"] [txt "No keys yet."]
               else
                 HTML.table [] (
                   HTML.thead [] [
                     HTML.tr [] [
                       HTML.th [] [txt "fingerprint"];
                       HTML.th [] [txt "label"];
                       HTML.th [] [txt ""];
                     ];
                   ]
                   :: key_rows
                 ));
            ];
            HTML.div [HTML.class_ "card"] [
              HTML.h2 [] [txt "Add key"];
              HTML.form [HTML.method_ `POST; HTML.action "/settings/ssh"] [
```

```
                HTML.input [HTML.type_ "hidden"; HTML.name "action";
    HTML.value "add"];
                HTML.p [] [
                  HTML.label [] [txt "OpenSSH public key"];
                  HTML.textarea [HTML.name "openssh"; HTML.required] "%s"
    "";
                ];
                HTML.p [] [
                  HTML.label [] [txt "Label (optional)"];
                  HTML.input [HTML.type_ "text"; HTML.name "label";
    HTML.placeholder "laptop"];
                ];
                HTML.button [HTML.type_ "submit"] [txt "Add key"];
              ];
            ];
          ]
        in
        html_response reqd ~session body;
        ()

let post_settings_ssh (session : Session.t) (reqd : Httpun.Reqd.t)
  : unit =
  let username = Session.username session in
  if username = "anonymous" then begin
    redirect_response reqd "/login";
    ()
  end else
    let user_opt =
      Auth_store.get_user_by_username (Session.store session) username
    in
    match user_opt with
    | None ->
        redirect_response reqd "/login";
        ()
    | Some user ->
        let body = read_body reqd in
        let action =
          Option.value (form_field body "action") ~default:"add" |>
    String.trim
        in
        if action = "revoke" then
          let id =
            Option.value (form_field body "id") ~default:"" |> String.trim
          in
          let _r =
            Auth_store.revoke_ssh_key (Session.store session)
              ~user_id:user.Auth_store.id ~id_or_fingerprint:id
          in
          redirect_response reqd "/settings/ssh";
          ()
        else
          let openssh =
            Option.value (form_field body "openssh") ~default:"" |>
    String.trim
          in
          let label =
            match form_field body "label" with
            | Some s when String.trim s <> "" -> Some (String.trim s)
            | _ -> None
          in
          let r =
            Auth_store.add_ssh_key (Session.store session)
              ~user_id:user.Auth_store.id ~openssh ~label ()
          in
          (match r with
           | Error e ->
               html_response reqd ~session ~status:`Bad_request
                 (chrome ~session ~title:"SSH keys" [
                    HTML.p [HTML.class_ "err"] [txt "%s" e];
                    HTML.p [] [HTML.a [HTML.href "/settings/ssh"] [txt
    "Back"]];
                  ]);
              ()
```

```
            | Ok _ ->
                redirect_response reqd "/settings/ssh";
                ())
```

76 image/lib/http_data.ml

```
(** Pricklypear Image -- /types + /data CRUD browser pages. *)
open Pure_html
open Http_core

let list_types_page (session : Session.t) (reqd : Httpun.Reqd.t) : unit =
  let types = Store.list_types (Session.store session) in
  let rows =
    List.map
      (fun (name, fields) ->
        let field_list = String.concat ", " fields in
        HTML.tr [] [
          HTML.td [] [
            HTML.a [HTML.href "/data/%s" name] [txt "%s" name];
          ];
          HTML.td [] [txt "%s" field_list];
          HTML.td [] [
            HTML.a [HTML.href "/data/%s" name] [txt "rows"];
          ];
        ])
      types
  in
  let body =
    chrome ~session ~title:("Data types (" ^ Session.branch_name session ^
    ")") [
      HTML.p [HTML.class_ "meta"] [
        txt "Declared shapes for Data rows. Browse/edit instances at ";
        HTML.a [HTML.href "/data"] [txt "/data"];
        txt " or via (create 'type \"{...}\") in the REPL.";
      ];
      (match types with
       | [] ->
           HTML.p [] [txt "(no types yet; declare via (deftype 'name
    'field1 ...))"]
       | _ ->
           HTML.table [] [
             HTML.thead [] [
               HTML.tr [] [
                 HTML.th [] [txt "type"];
                 HTML.th [] [txt "fields"];
                 HTML.th [] [txt ""];
               ]
             ];
             HTML.tbody [] rows;
           ]);
    ]
  in
  html_response reqd ~session body;
  ()

(* ---- data browser (image.data-browser) ---- *)

let data_type_name_ok (s : string) : bool =
  let n = String.length s in
  n > 0 && n <= 128 &&
  (try
     for i = 0 to n - 1 do
       match s.[i] with
       | 'a' .. 'z' | 'A' .. 'Z' | '0' .. '9' | '-' | '_' | '/' | '.' -> ()
       | _ -> raise Exit
     done;
     true
   with Exit -> false)

let json_object_assoc (s : string) : ((string * Yojson.Safe.t) list,
    string) result =
  try
    match Yojson.Safe.from_string s with
    | `Assoc kvs -> Ok kvs
    | _ -> Error "fields JSON must be an object"
  with exn -> Error ("invalid JSON: " ^ Printexc.to_string exn)

```

```
69  let fields_assoc_of_json (s : string) : (string * Yojson.Safe.t) list =
70    match json_object_assoc s with
71    | Ok kvs -> kvs
72    | Error _ -> []
73
74  let string_of_json_field (v : Yojson.Safe.t) : string =
75    match v with
76    | `String s -> s
77    | `Int i -> string_of_int i
78    | `Intlit s -> s
79    | `Float f -> string_of_float f
80    | `Bool b -> if b then "true" else "false"
81    | `Null -> ""
82    | other -> Yojson.Safe.to_string other
83
84  let truncate_cell (s : string) ~(max_len : int) : string =
85    if String.length s <= max_len then s
86    else String.sub s 0 max_len ^ "?"
87
88  let short_id (id : string) : string =
89    if String.length id <= 8 then id else String.sub id 0 8
90
91  (** Build fields JSON for create/update.
92      Prefer non-empty fields_json (must be object). Else declared form
93      fields as string values. On update, [base] preserves unknown keys. *)
94  let build_fields_json
95      ~(declared : string list)
96      ~(form : (string * string) list)
97      ~(fields_json_opt : string option)
98      ~(base : (string * Yojson.Safe.t) list)
99      : (string, string) result =
100   match fields_json_opt with
101   | Some raw when String.trim raw <> "" ->
102       (match json_object_assoc (String.trim raw) with
103        | Ok kvs -> Ok (Yojson.Safe.to_string (`Assoc kvs))
104        | Error e -> Error e)
105   | _ ->
106       let form_tbl = Hashtbl.create 16 in
107       List.iter (fun (k, v) -> Hashtbl.replace form_tbl k v) form;
108       let declared_keys =
109         if declared = [] then
110           (* no type_defs: take every form key except control fields *)
111           Hashtbl.fold
112             (fun k _ acc ->
113               if k = "fields_json" || k = "action" then acc else k :: acc)
114             form_tbl []
115           |> List.sort_uniq String.compare
116         else declared
117       in
118       let updated =
119         List.fold_left
120           (fun acc key ->
121             match Hashtbl.find_opt form_tbl key with
122             | None -> acc
123             | Some v ->
124                 (key, `String v) :: List.filter (fun (k, _) -> k <> key)
      acc)
125           base declared_keys
126       in
127       Ok (Yojson.Safe.to_string (`Assoc updated))
128
129 let shared_data_banner () =
130   HTML.p [HTML.class_ "meta"] [
131     txt "You see your own rows plus shared seed data (bible, strongs). ";
132     txt "Data is not versioned by branch. ";
133     HTML.a [HTML.href "/types"] [txt "Type defs"];
134     txt " ? ";
135     HTML.a [HTML.href "/data"] [txt "Your data"];
136   ]
137
138 let data_index_page (session : Session.t) (reqd : Httpun.Reqd.t) : unit =
139   let store = Session.store session in
140   let types = Store.list_types store in
```

```
  let orphan_names = Crud.list_data_type_names store in
  let declared = List.map fst types in
  let declared_set =
    List.fold_left (fun acc n -> n :: acc) [] declared
  in
  let orphans =
    List.filter (fun n -> not (List.mem n declared_set)) orphan_names
  in
  let rows =
    List.map
      (fun (name, fields) ->
        let n = Crud.count_by_type store ~type_name:name in
        (HTML.tr [] [
             HTML.td [] [
               HTML.a [HTML.href "/data/%s" name] [txt "%s" name];
             ];
             HTML.td [] [txt "%s" (String.concat ", " fields)];
             HTML.td [] [txt "%d" n];
             HTML.td [] [
               HTML.a [HTML.href "/data/%s" name] [txt "browse"];
               txt " ? ";
               HTML.a [HTML.href "/data/%s/new" name] [txt "new"];
             ];
           ]))
      types
  in
  let orphan_rows =
    List.map
      (fun name ->
        let n = Crud.count_by_type store ~type_name:name in
        (HTML.tr [] [
             HTML.td [] [
               HTML.a [HTML.href "/data/%s" name] [txt "%s" name];
             ];
             HTML.td [] [txt "(no type_def)"];
             HTML.td [] [txt "%d" n];
             HTML.td [] [
               HTML.a [HTML.href "/data/%s" name] [txt "browse"];
             ];
           ]))
      orphans
  in
  let body =
    chrome ~session ~title:"Data" [
      shared_data_banner ();
      HTML.div [HTML.class_ "card"] [
        HTML.h2 [] [txt "Types with rows"];
        (match rows @ orphan_rows with
         | [] ->
             HTML.p [] [
               txt "(no data_rows yet; declare a type via (deftype ?) or ";
               HTML.a [HTML.href "/types"] [txt "/types"];
               txt ")";
             ]
         | all ->
             HTML.table [] [
               HTML.thead [] [
                 HTML.tr [] [
                   HTML.th [] [txt "type"];
                   HTML.th [] [txt "fields"];
                   HTML.th [] [txt "rows"];
                   HTML.th [] [txt ""];
                 ];
               ];
               HTML.tbody [] all;
             ]);
      ];
    ]
  in
  html_response reqd ~session body;
  ()

let data_list_page (session : Session.t) (type_name : string)
```

```
    (reqd : Httpun.Reqd.t) : unit =
  if not (data_type_name_ok type_name) then begin
    html_response reqd ~session ~status:`Bad_request
      (chrome ~session ~title:"Bad type" [
         HTML.p [HTML.class_ "err"] [txt "invalid type name"];
         HTML.p [] [HTML.a [HTML.href "/data"] [txt "Back to /data"]];
       ]);
    ()
  end else
    let store = Session.store session in
    let target = (Httpun.Reqd.request reqd).Httpun.Request.target in
    let limit =
      match query_param target "limit" with
      | Some s ->
          (try max 1 (min 200 (int_of_string s)) with _ -> 50)
      | None -> 50
    in
    let offset =
      match query_param target "offset" with
      | Some s -> (try max 0 (int_of_string s) with _ -> 0)
      | None -> 0
    in
    let types = Store.list_types store in
    let declared =
      match List.assoc_opt type_name types with
      | Some fs -> fs
      | None -> []
    in
    let total = Crud.count_by_type store ~type_name in
    let rows = Crud.list_page store ~type_name ~limit ~offset () in
    let col_fields =
      if declared = [] then
        (* infer columns from first row keys *)
        match rows with
        | r :: _ -> List.map fst (fields_assoc_of_json r.Crud.fields_json)
        | [] -> []
      else declared
    in
    let header_cells =
      HTML.th [] [txt "id"]
      :: List.map (fun f -> HTML.th [] [txt "%s" f]) col_fields
    in
    let data_rows =
      List.map
        (fun (r : Crud.row_record) ->
          let assoc = fields_assoc_of_json r.Crud.fields_json in
          let cells =
            List.map
              (fun f ->
                let v =
                  match List.assoc_opt f assoc with
                  | Some j -> string_of_json_field j
                  | None -> ""
                in
                HTML.td [] [txt "%s" (truncate_cell v ~max_len:48)])
              col_fields
          in
          HTML.tr []
            (HTML.td [] [
               HTML.a
                 [HTML.href "/data/%s/%s" type_name r.Crud.id]
                 [txt "%s" (short_id r.Crud.id)];
             ]
             :: cells))
        rows
    in
    let prev_off = max 0 (offset - limit) in
    let next_off = offset + limit in
    let has_prev = offset > 0 in
    let has_next = next_off < total in
    let body =
      chrome ~session ~title:("Data ? " ^ type_name) [
        shared_data_banner ();
```

```
        HTML.div [HTML.class_ "card"] [
          HTML.h2 [] [txt "%s" type_name];
          HTML.p [HTML.class_ "meta"] [
            txt "%d row(s) ? showing %d-%d ? limit %d"
              total
              (if total = 0 then 0 else offset + 1)
              (min total (offset + List.length rows))
              limit;
          ];
          HTML.p [] [
            HTML.a [HTML.href "/data/%s/new" type_name] [txt "New row"];
            txt " ? ";
            HTML.a [HTML.href "/data"] [txt "All types"];
            txt " ? ";
            HTML.a [HTML.href "/types"] [txt "Type defs"];
          ];
          (match data_rows with
           | [] ->
               HTML.p [] [
                 txt "(no rows yet) ";
                 HTML.a [HTML.href "/data/%s/new" type_name] [txt "Create
    one"];
               ]
           | _ ->
               HTML.table [] [
                 HTML.thead [] [HTML.tr [] header_cells];
                 HTML.tbody [] data_rows;
               ]);
          HTML.p []
            ((if has_prev then
                [ HTML.a
                    [ HTML.href "/data/%s?limit=%d&offset=%d"
                        type_name limit prev_off ]
                    [txt "? Prev"] ]
              else [])
             @ (if has_prev && has_next then [txt " ? "] else [])
             @ (if has_next then
                  [ HTML.a
                      [ HTML.href "/data/%s?limit=%d&offset=%d"
                          type_name limit next_off ]
                      [txt "Next ?"] ]
                else []));
        ];
      ]
    in
    html_response reqd ~session body;
    ()

let field_inputs ~(declared : string list)
    ~(values : (string * string) list) : Pure_html.node list =
  let val_of k = Option.value (List.assoc_opt k values) ~default:"" in
  if declared = [] then
    [ HTML.p [HTML.class_ "meta"] [
        txt "No declared fields -- use raw JSON below (or deftype first).";
      ] ]
  else
    List.map
      (fun f ->
        HTML.p [] [
          HTML.label [] [txt "%s " f];
          HTML.input
            [ HTML.type_ "text"
            ; HTML.name "%s" f
            ; HTML.value "%s" (val_of f)
            ];
        ])
      declared

let data_new_page (session : Session.t) (type_name : string)
    (reqd : Httpun.Reqd.t) : unit =
  if not (data_type_name_ok type_name) then begin
    html_response reqd ~session ~status:`Bad_request
      (chrome ~session ~title:"Bad type" [
```

```
         HTML.p [HTML.class_ "err"] [txt "invalid type name"];
       ]);
    ()
  end else
    let types = Store.list_types (Session.store session) in
    let declared =
      match List.assoc_opt type_name types with Some fs -> fs | None -> []
    in
    let body =
      chrome ~session ~title:("New ? " ^ type_name) [
        shared_data_banner ();
        HTML.div [HTML.class_ "card"] [
          HTML.h2 [] [txt "Create %s" type_name];
          HTML.form
            [ HTML.method_ `POST; HTML.action "/data/%s" type_name ]
            (field_inputs ~declared ~values:[]
             @ [ HTML.p [] [
                   HTML.label [] [txt "fields_json (optional; wins if set)
    "];
                   HTML.textarea
                     [ HTML.name "fields_json" ]
                     "%s" "";
                 ]
               ; HTML.button [HTML.type_ "submit"] [txt "Create"]
               ; HTML.p [] [
                   HTML.a [HTML.href "/data/%s" type_name] [txt "Cancel"];
                 ]
               ]);
        ];
      ]
    in
    html_response reqd ~session body;
    ()

let data_create (session : Session.t) (type_name : string)
    (reqd : Httpun.Reqd.t) : unit =
  if not (data_type_name_ok type_name) then begin
    html_response reqd ~session ~status:`Bad_request
      (chrome ~session ~title:"Bad type" [
         HTML.p [HTML.class_ "err"] [txt "invalid type name"];
       ]);
    ()
  end else
    let store = Session.store session in
    let body = read_body reqd in
    let form = parse_form_body body in
    let types = Store.list_types store in
    let declared =
      match List.assoc_opt type_name types with Some fs -> fs | None -> []
    in
    let fields_json_opt = List.assoc_opt "fields_json" form in
    match
      build_fields_json ~declared ~form ~fields_json_opt ~base:[]
    with
    | Error e ->
        html_response reqd ~session ~status:`Bad_request
          (chrome ~session ~title:"Create failed" [
             HTML.p [HTML.class_ "err"] [txt "%s" e];
             HTML.p [] [
               HTML.a [HTML.href "/data/%s/new" type_name] [txt "Back"];
             ];
           ]);
        ()
    | Ok fields_json ->
        let author = Session.username session in
        let v =
          Crud.create store ~type_name ~fields_json ~author
        in
        (match v with
         | Pricklypear_lisp.Ast.VStr id
           when String.length id = 36 && not (String.contains id ' ') ->
             redirect_response reqd
               (Printf.sprintf "/data/%s/%s" type_name id);
```

```
            ()
        | Pricklypear_lisp.Ast.VStr msg ->
            html_response reqd ~session ~status:`Internal_server_error
              (chrome ~session ~title:"Create failed" [
                 HTML.p [HTML.class_ "err"] [txt "%s" msg];
                 HTML.p [] [
                   HTML.a [HTML.href "/data/%s/new" type_name] [txt
    "Back"];
                 ];
               ]);
            ()
        | _ ->
            html_response reqd ~session ~status:`Internal_server_error
              (chrome ~session ~title:"Create failed" [
                 HTML.p [HTML.class_ "err"] [txt "create failed"];
               ]);
            ())

let data_detail_page (session : Session.t) (type_name : string)
    (id : string) (reqd : Httpun.Reqd.t) : unit =
  if not (data_type_name_ok type_name) then begin
    html_response reqd ~session ~status:`Bad_request
      (chrome ~session ~title:"Bad type" [
         HTML.p [HTML.class_ "err"] [txt "invalid type name"];
       ]);
    ()
  end else
    let store = Session.store session in
    let row_opt = Crud.get_row store ~type_name ~id in
    match row_opt with
    | None ->
        html_response reqd ~session ~status:`Not_found
          (chrome ~session ~title:"Row not found" [
             HTML.p [HTML.class_ "err"] [txt "row not found"];
             HTML.p [] [
               HTML.a [HTML.href "/data/%s" type_name] [txt "Back to list"];
             ];
           ]);
        ()
    | Some row ->
        let types = Store.list_types store in
        let declared =
          match List.assoc_opt type_name types with
          | Some fs -> fs
          | None -> []
        in
        let assoc = fields_assoc_of_json row.Crud.fields_json in
        let declared_set = declared in
        let extra_keys =
          List.filter_map
            (fun (k, _) ->
              if List.mem k declared_set then None else Some k)
            assoc
        in
        let values =
          List.map (fun (k, v) -> (k, string_of_json_field v)) assoc
        in
        (* Include extra keys so edit form can preserve/update unknowns. *)
        let form_fields =
          declared
          @ List.map (fun k -> k) extra_keys
        in
        let meta_bits =
          [ HTML.p [HTML.class_ "meta"] [txt "id: %s" row.Crud.id] ]
          @ (match row.Crud.author with
             | Some a -> [HTML.p [HTML.class_ "meta"] [txt "author: %s" a]]
             | None -> [])
          @ (match row.Crud.created_at with
             | Some t -> [HTML.p [HTML.class_ "meta"] [txt "created: %s" t]]
             | None -> [])
          @ (match row.Crud.updated_at with
             | Some t -> [HTML.p [HTML.class_ "meta"] [txt "updated: %s" t]]
             | None -> [])
```

```
        in
        let pretty =
          try Yojson.Safe.pretty_to_string
                (Yojson.Safe.from_string row.Crud.fields_json)
          with _ -> row.Crud.fields_json
        in
        let body =
          chrome ~session
            ~title:(Printf.sprintf "Data ? %s ? %s" type_name (short_id id))
            [ shared_data_banner ()
            ; HTML.div [HTML.class_ "card"] (
                [ HTML.h2 [] [txt "%s" type_name] ]
                @ meta_bits
                @ [ HTML.p [] [
                      HTML.a [HTML.href "/data/%s" type_name] [txt "? List"];
                      txt " ? ";
                      HTML.a [HTML.href "/data"] [txt "All data"];
                    ]
                  ; HTML.h3 [] [txt "Fields"]
                  ; HTML.pre [] [txt "%s" pretty]
                  ]
              )
            ; HTML.div [HTML.class_ "card"] [
                HTML.h3 [] [txt "Edit"];
                HTML.form
                  [ HTML.method_ `POST
                  ; HTML.action "/data/%s/%s" type_name id
                  ]
                  (field_inputs ~declared:form_fields ~values
                   @ [ HTML.p [] [
                         HTML.label []
                           [txt "fields_json (optional; full replace if set) "];
                         HTML.textarea
                           [ HTML.name "fields_json" ]
                           "%s" "";
                       ]
                     ; HTML.button [HTML.type_ "submit"] [txt "Save"]
                     ]);
              ]
            ; HTML.div [HTML.class_ "card"] [
                HTML.h3 [] [txt "Delete"];
                HTML.form
                  [ HTML.method_ `POST
                  ; HTML.action "/data/%s/%s/delete" type_name id
                  ]
                  [ HTML.button [HTML.type_ "submit"] [txt "Delete row"] ];
              ]
            ]
        in
        html_response reqd ~session body;
        ()

let data_update (session : Session.t) (type_name : string) (id : string)
    (reqd : Httpun.Reqd.t) : unit =
  if not (data_type_name_ok type_name) then begin
    html_response reqd ~session ~status:`Bad_request
      (chrome ~session ~title:"Bad type" [
         HTML.p [HTML.class_ "err"] [txt "invalid type name"];
       ]);
    ()
  end else
    let store = Session.store session in
    let body = read_body reqd in
    let form = parse_form_body body in
    let row_opt = Crud.get_row store ~type_name ~id in
    match row_opt with
    | None ->
        html_response reqd ~session ~status:`Not_found
          (chrome ~session ~title:"Row not found" [
             HTML.p [HTML.class_ "err"] [txt "row not found"];
           ]);
```

```
        ()
    | Some row ->
        let types = Store.list_types store in
        let declared =
          match List.assoc_opt type_name types with
          | Some fs -> fs
          | None -> []
        in
        let base = fields_assoc_of_json row.Crud.fields_json in
        let extra_keys =
          List.filter_map
            (fun (k, _) -> if List.mem k declared then None else Some k)
            base
        in
        let form_fields = declared @ extra_keys in
        let fields_json_opt = List.assoc_opt "fields_json" form in
        (match
           build_fields_json ~declared:form_fields ~form ~fields_json_opt
             ~base
         with
         | Error e ->
             html_response reqd ~session ~status:`Bad_request
               (chrome ~session ~title:"Update failed" [
                  HTML.p [HTML.class_ "err"] [txt "%s" e];
                  HTML.p [] [
                    HTML.a
                      [HTML.href "/data/%s/%s" type_name id]
                      [txt "Back"];
                  ];
                ]);
             ()
         | Ok fields_json ->
             let v = Crud.update store ~type_name ~id ~fields_json in
             (match v with
              | Pricklypear_lisp.Ast.VStr _ ->
                  redirect_response reqd
                    (Printf.sprintf "/data/%s/%s" type_name id);
                  ()
              | _ ->
                  html_response reqd ~session ~status:`Not_found
                    (chrome ~session ~title:"Update failed" [
                       HTML.p [HTML.class_ "err"] [txt "row not found"];
                     ]);
                  ()))

let data_delete (session : Session.t) (type_name : string) (id : string)
    (reqd : Httpun.Reqd.t) : unit =
  if not (data_type_name_ok type_name) then begin
    html_response reqd ~session ~status:`Bad_request
      (chrome ~session ~title:"Bad type" [
         HTML.p [HTML.class_ "err"] [txt "invalid type name"];
       ]);
    ()
  end else
    let store = Session.store session in
    let v = Crud.delete store ~type_name ~id in
    match v with
    | Pricklypear_lisp.Ast.VStr _ ->
        redirect_response reqd (Printf.sprintf "/data/%s" type_name);
        ()
    | _ ->
        html_response reqd ~session ~status:`Not_found
          (chrome ~session ~title:"Delete failed" [
             HTML.p [HTML.class_ "err"] [txt "row not found"];
             HTML.p [] [
               HTML.a [HTML.href "/data/%s" type_name] [txt "Back to list"];
             ];
           ]);
        ()


(* ---- page timing (image.page-timing) ---- *)

```

```
let timing_page (session : Session.t) (reqd : Httpun.Reqd.t) : unit =
  let aggs = Timing.aggregates () in
  let db_share (a : Timing.agg) : string =
    if a.avg_ms > 0.0001 then
      Printf.sprintf "%.0f%%" (100. *. a.db_ms /. a.avg_ms)
    else "--"
  in
  let rows =
    List.map
      (fun (a : Timing.agg) ->
        HTML.tr [] [
          HTML.td [] [txt "%s" a.route];
          HTML.td [] [txt "%d" a.count];
          HTML.td [] [txt "%.1f" a.last_ms];
          HTML.td [] [txt "%.1f" a.avg_ms];
          HTML.td [] [txt "%.1f" a.max_ms];
          HTML.td [] [txt "%.1f" a.db_ms];
          HTML.td [] [txt "%.1f" a.db_hits];
          HTML.td [] [txt "%s" (db_share a)];
        ])
      aggs
  in
  let body =
    chrome ~session ~title:"Page timing" [
      HTML.p [HTML.class_ "meta"] [
        txt "Server-side render cost per user route (last %d total samples;
    %d per route)."
          Timing.max_total_samples Timing.max_samples_per_route;
        txt "  Sorted slowest-first by max. DB share = avg DB wall time /
    avg total.";
      ];
      (match aggs with
       | [] ->
           HTML.p [HTML.class_ "err"] [
             txt "No route timings measured yet. Visit some pages first,
    then reload.";
           ]
       | _ ->
           HTML.table [] [
             HTML.thead [] [
               HTML.tr [] [
                 HTML.th [] [txt "route"];
                 HTML.th [] [txt "samples"];
                 HTML.th [] [txt "last ms"];
                 HTML.th [] [txt "avg ms"];
                 HTML.th [] [txt "max ms"];
                 HTML.th [] [txt "db ms"];
                 HTML.th [] [txt "db hits"];
                 HTML.th [] [txt "db share"];
               ]
             ];
             HTML.tbody [] rows;
           ]);
    ]
  in
  html_response reqd ~session body;
  ()
```

77 image/lib/http_media.ml

```
(** HTTP handlers for storage.media -- agent image push + browser upload.

    POST   /api/media          raw | JSON b64 | multipart
    GET    /api/media          JSON meta list
    DELETE /api/media/<uuid>
    GET    /media              browser gallery + upload form
    POST   /media/upload       multipart browser form
    GET    /media/<uuid>       raw bytes
*)
open Pure_html
open Http_core

let max_media_body_bytes = Media.max_http_bytes

(** Read body with a hard byte ceiling; Error `Too_large if exceeded. *)
let read_body_limited (reqd : Httpun.Reqd.t) ~(max_bytes : int)
  : (string, [ `Too_large ]) result =
  let body = Httpun.Reqd.request_body reqd in
  let promise, resolver = Eio.Promise.create () in
  let buf = Buffer.create (min 4096 max_bytes) in
  let oversized = ref false in
  let rec sched () =
    Httpun.Body.Reader.schedule_read body
      ~on_eof:(fun () ->
        if !oversized then Eio.Promise.resolve resolver (Error `Too_large)
        else Eio.Promise.resolve resolver (Ok (Buffer.contents buf)))
      ~on_read:(fun bs ~off ~len ->
        if !oversized then sched ()
        else if Buffer.length buf + len > max_bytes then begin
          oversized := true;
          Buffer.reset buf;
          sched ()
        end else begin
          Buffer.add_string buf (Bigstringaf.substring bs ~off ~len);
          sched ()
        end)
  in
  sched ();
  Eio.Promise.await promise

let header_ci (headers : (string * string) list) (name : string)
  : string option =
  let lower = String.lowercase_ascii name in
  List.find_map
    (fun (k, v) ->
      if String.lowercase_ascii k = lower then Some v else None)
    headers

let require_authed_user (session : Session.t) : (string * bool, unit)
    result =
  let uname = Session.username session in
  if uname = "" || uname = "anonymous" then Error ()
  else Ok (uname, Session.is_admin session)

let too_large (reqd : Httpun.Reqd.t) : unit =
  json_response reqd ~status:`Payload_too_large
    (`Assoc
       [ ("ok", `Bool false)
       ; ( "error"
         , `String
             (Printf.sprintf "body exceeds %d bytes" max_media_body_bytes) )
       ])

let bad_request (reqd : Httpun.Reqd.t) (msg : string) : unit =
  json_response reqd ~status:`Bad_request
    (`Assoc [ ("ok", `Bool false); ("error", `String msg) ])

let forbidden (reqd : Httpun.Reqd.t) (msg : string) : unit =
  json_response reqd ~status:`Forbidden
    (`Assoc [ ("ok", `Bool false); ("error", `String msg) ])

```

```
let not_found_json (reqd : Httpun.Reqd.t) (msg : string) : unit =
  json_response reqd ~status:`Not_found
    (`Assoc [ ("ok", `Bool false); ("error", `String msg) ])

(* ---- multipart/form-data (browser upload) ---- *)

let multipart_boundary (content_type : string) : string option =
  let ct = String.trim content_type in
  let lower = String.lowercase_ascii ct in
  if not (String.starts_with ~prefix:"multipart/form-data" lower) then None
  else
    (* boundary=VALUE -- value may be quoted *)
    let rec find_boundary s =
      let sl = String.lowercase_ascii s in
      match String.index_opt sl 'b' with
      | None -> None
      | Some i ->
          if i + 9 <= String.length s
             && String.sub sl i 9 = "boundary="
          then
            let rest = String.sub s (i + 9) (String.length s - i - 9) in
            let rest = String.trim rest in
            if rest = "" then None
            else if rest.[0] = '"' then
              match String.index_from_opt rest 1 '"' with
              | Some j -> Some (String.sub rest 1 (j - 1))
              | None -> None
            else
              let end_at =
                match String.index_opt rest ';' with
                | Some j -> j
                | None -> String.length rest
              in
              let b = String.trim (String.sub rest 0 end_at) in
              if b = "" then None else Some b
          else find_boundary (String.sub s (i + 1) (String.length s - i -
    1))
    in
    find_boundary ct

(** Split [s] on CRLF or LF; keep binary-safe (no decode). *)
let index_of_sub (hay : string) ~(pos : int) (needle : string) : int option
    =
  let nlen = String.length needle in
  let hlen = String.length hay in
  if nlen = 0 || pos < 0 || pos > hlen then None
  else
    let last = hlen - nlen in
    let rec loop i =
      if i > last then None
      else if String.sub hay i nlen = needle then Some i
      else loop (i + 1)
    in
    loop pos

let parse_part_headers (header_blob : string)
  : (string * string) list =
  let lines =
    (* Normalize CRLF ? split on \n and strip trailing \r *)
    String.split_on_char '\n' header_blob
    |> List.map (fun line ->
           let line =
             if String.length line > 0 && line.[String.length line - 1] =
    '\r'
             then String.sub line 0 (String.length line - 1)
             else line
           in
           line)
    |> List.filter (fun l -> String.trim l <> "")
  in
  List.filter_map
    (fun line ->
      match String.index_opt line ':' with
```

```
      | None -> None
      | Some i ->
          let k =
            String.lowercase_ascii (String.trim (String.sub line 0 i))
          in
          let v =
            String.trim
              (String.sub line (i + 1) (String.length line - i - 1))
          in
          Some (k, v))
    lines

let content_disposition_params (cd : string) : (string * string) list =
  (* form-data; name="file"; filename="x.png" *)
  String.split_on_char ';' cd
  |> List.filter_map (fun part ->
         let part = String.trim part in
         match String.index_opt part '=' with
         | None -> None
         | Some i ->
             let k =
               String.lowercase_ascii (String.trim (String.sub part 0 i))
             in
             let raw =
               String.trim
                 (String.sub part (i + 1) (String.length part - i - 1))
             in
             let v =
               if String.length raw >= 2 && raw.[0] = '"'
                  && raw.[String.length raw - 1] = '"'
               then String.sub raw 1 (String.length raw - 2)
               else raw
             in
             Some (k, v))

type multipart_file = {
  field_name : string;
  filename : string option;
  content_type : string option;
  data : string;
}

(** Extract file parts from a multipart body. Prefers field name
    "file", then "media", then first part that has a filename. *)
let parse_multipart ~(boundary : string) (body : string)
  : (multipart_file list, string) result =
  if boundary = "" then Error "empty boundary"
  else
    let delim = "--" ^ boundary in
    let close_delim = delim ^ "--" in
    (* Walk parts between delimiters. *)
    let rec collect acc pos =
      match index_of_sub body ~pos delim with
      | None -> List.rev acc
      | Some i ->
          let after_delim = i + String.length delim in
          (* Skip optional leading CRLF after delimiter *)
          let after_delim =
            if after_delim + 1 < String.length body
               && body.[after_delim] = '\r'
               && body.[after_delim + 1] = '\n'
            then after_delim + 2
            else if after_delim < String.length body
                    && body.[after_delim] = '\n'
            then after_delim + 1
            else after_delim
          in
          (* Closing delimiter? *)
          if i >= 0
             && String.length body >= i + String.length close_delim
             && String.sub body i (String.length close_delim) = close_delim
          then List.rev acc
          else
```

```
            match index_of_sub body ~pos:after_delim delim with
            | None -> List.rev acc
            | Some next ->
                (* Part is [after_delim, next); strip trailing CRLF before
    next delim *)
                let part_end = next in
                let part_end =
                  if part_end >= 2
                     && body.[part_end - 2] = '\r'
                     && body.[part_end - 1] = '\n'
                  then part_end - 2
                  else if part_end >= 1 && body.[part_end - 1] = '\n'
                  then part_end - 1
                  else part_end
                in
                let part =
                  if part_end > after_delim then
                    String.sub body after_delim (part_end - after_delim)
                  else ""
                in
                (* Split headers / body on blank line *)
                let header_end =
                  match index_of_sub part ~pos:0 "\r\n\r\n" with
                  | Some j -> Some (j, 4)
                  | None ->
                      (match index_of_sub part ~pos:0 "\n\n" with
                       | Some j -> Some (j, 2)
                       | None -> None)
                in
                let acc =
                  match header_end with
                  | None -> acc
                  | Some (he, sep_len) ->
                      let headers =
                        parse_part_headers (String.sub part 0 he)
                      in
                      let data_start = he + sep_len in
                      let data =
                        if data_start <= String.length part then
                          String.sub part data_start
                            (String.length part - data_start)
                        else ""
                      in
                      let cd =
                        List.assoc_opt "content-disposition" headers
                      in
                      let params =
                        match cd with
                        | Some s -> content_disposition_params s
                        | None -> []
                      in
                      let field_name =
                        Option.value (List.assoc_opt "name" params)
                          ~default:""
                      in
                      let filename = List.assoc_opt "filename" params in
                      let content_type =
                        List.assoc_opt "content-type" headers
                      in
                      if field_name = "" && filename = None then acc
                      else
                        { field_name; filename; content_type; data } :: acc
                in
                collect acc next
    in
    try Ok (collect [] 0)
    with exn -> Error ("multipart parse failed: " ^ Printexc.to_string exn)

let pick_upload_file (parts : multipart_file list) : multipart_file option =
  let named n =
    List.find_opt (fun p -> p.field_name = n && p.data <> "") parts
  in
  match named "file" with
```

```
  | Some p -> Some p
  | None ->
      (match named "media" with
       | Some p -> Some p
       | None ->
           List.find_opt
             (fun p -> p.filename <> None && p.data <> "")
             parts)

let parse_json_upload (body : string)
  : (string * string * string option * string, string) result =
  try
    match Yojson.Safe.from_string body with
    | `Assoc items ->
        let content_type =
          match List.assoc_opt "content_type" items with
          | Some (`String s) -> Some (String.trim s)
          | _ -> None
        in
        let data_b64 =
          match List.assoc_opt "data_b64" items with
          | Some (`String s) -> Some s
          | _ -> None
        in
        let filename =
          match List.assoc_opt "filename" items with
          | Some (`String s) -> Some s
          | Some `Null | None -> None
          | _ -> None
        in
        let meta =
          match List.assoc_opt "meta" items with
          | Some (`Assoc _ as j) -> Yojson.Safe.to_string j
          | Some (`String s) -> s
          | _ -> "{}"
        in
        (match content_type, data_b64 with
         | Some ct, Some b64 ->
             (match Media.b64_decode_opt (String.trim b64) with
              | Some raw -> Ok (ct, raw, filename, meta)
              | None -> Error "invalid data_b64")
         | None, _ -> Error "content_type required"
         | _, None -> Error "data_b64 required")
    | _ -> Error "JSON object required"
  with _ -> Error "invalid JSON"

let parse_upload_body ~(content_type : string option)
    ~(filename_hdr : string option) (body : string)
  : (string * string * string option * string, string) result =
  match content_type with
  | None -> Error "Content-Type required"
  | Some ct when body = "" -> Error "empty body"
  | Some ct ->
      let lower = String.lowercase_ascii (String.trim ct) in
      if String.starts_with ~prefix:"application/json" lower then
        parse_json_upload body
      else
        match multipart_boundary ct with
        | Some boundary ->
            (match parse_multipart ~boundary body with
             | Error e -> Error e
             | Ok parts ->
                 (match pick_upload_file parts with
                  | None ->
                      Error
                        "multipart: no file field (expected name=file or \
                         name=media)"
                  | Some part ->
                      let part_ct =
                        match part.content_type with
                        | Some c when String.trim c <> "" -> c
                        | _ ->
                            (* Fall back to extension guess is not done
```

```
                               server-side; require explicit type. *)
                            ""
                      in
                      if part_ct = "" then
                        Error
                          "multipart file missing Content-Type; browser \
                           should send one"
                      else
                        Ok
                          ( part_ct
                          , part.data
                          , (match part.filename with
                             | Some _ as f -> f
                             | None -> filename_hdr)
                          , "{}" )))
        | None ->
            (* Raw binary body -- Content-Type is the media type. *)
            Ok (ct, body, filename_hdr, "{}")

let store_upload
    (session : Session.t)
    ~(author : string)
    ~(content_type : string)
    ~(data : string)
    ~(filename : string option)
    ~(meta_json : string)
  : (Media.put_result, string) result =
  Media.put (Session.store session) ~author ~content_type ~data ~filename
    ~meta_json ~max_bytes:max_media_body_bytes ()

let put_error_status (msg : string) : Httpun.Status.t =
  if String.starts_with ~prefix:"payload too large" msg then
    `Payload_too_large
  else if String.starts_with ~prefix:"disallowed" msg
          || msg = "empty body"
          || String.starts_with ~prefix:"multipart" msg
          || String.starts_with ~prefix:"Content-Type" msg
          || String.starts_with ~prefix:"invalid" msg
          || String.starts_with ~prefix:"content_type" msg
          || String.starts_with ~prefix:"data_b64" msg
          || String.starts_with ~prefix:"JSON" msg
  then `Bad_request
  else `Internal_server_error

let api_media_post (session : Session.t) (reqd : Httpun.Reqd.t) : unit =
  match require_authed_user session with
  | Error () ->
      unauthorized_api reqd;
      ()
  | Ok (author, _) ->
      let body_res =
        read_body_limited reqd ~max_bytes:max_media_body_bytes
      in
      (match body_res with
       | Error `Too_large ->
           too_large reqd;
           ()
       | Ok body ->
           let req = Httpun.Reqd.request reqd in
           let headers = header_list req in
           let ct_hdr =
             Option.map String.trim (header_ci headers "content-type")
           in
           let filename_hdr = header_ci headers "x-filename" in
           (match
              parse_upload_body ~content_type:ct_hdr ~filename_hdr body
            with
            | Error msg ->
                bad_request reqd msg;
                ()
            | Ok (content_type, data, filename, meta_json) ->
                let res =
                  store_upload session ~author ~content_type ~data
```

```
                    ~filename ~meta_json
                in
                (match res with
                 | Ok r ->
                     json_response reqd ~status:`Created
                       (Media.put_result_to_yojson r);
                     ()
                 | Error msg ->
                     json_response reqd ~status:(put_error_status msg)
                       (`Assoc
                          [ ("ok", `Bool false); ("error", `String msg) ]);
                     ())))

let api_media_list (session : Session.t) (reqd : Httpun.Reqd.t)
    ~(target : string) : unit =
  match require_authed_user session with
  | Error () ->
      unauthorized_api reqd;
      ()
  | Ok (author, is_admin) ->
      let limit =
        match query_param target "limit" with
        | Some s -> (try int_of_string s with _ -> 50)
        | None -> 50
      in
      let scope_all =
        match query_param target "all" with
        | Some ("1" | "true" | "yes") when is_admin -> true
        | _ -> false
      in
      let rows =
        Media.list (Session.store session)
          ~author:(if scope_all then None else Some author)
          ~limit ()
      in
      let items = List.map Media.meta_to_yojson rows in
      json_response reqd
        (`Assoc [ ("ok", `Bool true); ("items", `List items) ]);
      ()

let api_media_delete (session : Session.t) (reqd : Httpun.Reqd.t)
    ~(id : string) : unit =
  match require_authed_user session with
  | Error () ->
      unauthorized_api reqd;
      ()
  | Ok (actor, is_admin) ->
      let res =
        Media.delete (Session.store session) ~id ~actor ~is_admin
      in
      (match res with
       | Ok (`Ok id') ->
           json_response reqd
             (`Assoc [ ("ok", `Bool true); ("id", `String id') ]);
           ()
       | Ok `Not_found ->
           not_found_json reqd "media not found";
           ()
       | Ok `Forbidden ->
           forbidden reqd "not author or admin";
           ()
       | Error msg ->
           json_response reqd ~status:`Internal_server_error
             (`Assoc [ ("ok", `Bool false); ("error", `String msg) ]);
           ())

let media_get (session : Session.t) (reqd : Httpun.Reqd.t) ~(id : string)
  : unit =
  match require_authed_user session with
  | Error () ->
      unauthorized_api reqd;
      ()
  | Ok (actor, is_admin) ->
```

```
      (* BORGE-1785695911: GET by uuid is visibility-confined -- other users'
          blobs are "not found", same as data_rows post Phase 2b. *)
      let got =
        Media.get_bytes (Session.store session) ~id
          ~visible_to:(actor, is_admin) ()
      in
      (match got with
       | None ->
           text_response reqd ~status:`Not_found ~content_type:"text/plain"
             "not found";
           ()
       | Some (m, bytes) ->
           let disposition =
             (* Stored-XSS hardening: SVG can carry script, so serve it as a
                download. Galleries keep rendering it via <img> tags. *)
             let kind =
               if m.content_type = "image/svg+xml" then "attachment"
               else "inline"
             in
             match m.filename with
             | Some f ->
                 Printf.sprintf "%s; filename=\"%s\"" kind
                   (String.map
                      (function '"' | '\n' | '\r' -> '_' | c -> c)
                      f)
             | None -> kind
           in
           let headers =
             Httpun.Headers.of_list
               ([ ("content-type", m.content_type)
                ; ("content-length", string_of_int (String.length bytes))
                ; ("cache-control", "private, max-age=3600")
                ; ("content-disposition", disposition)
                ] @ Http_core.security_headers ())
           in
           let resp = Httpun.Response.create ~headers `OK in
           safe_respond_with_string reqd resp bytes;
           ())

(* ---- Browser UI: GET /media + POST /media/upload ---- *)

let is_image_ct (ct : string) : bool =
  String.starts_with ~prefix:"image/" (String.lowercase_ascii ct)

let media_row_card (m : Media.meta) : Pure_html.node =
  let name =
    match m.filename with Some f -> f | None -> m.id
  in
  let thumb =
    if is_image_ct m.content_type then
      HTML.a [HTML.href "%s" (Media.url_of_id m.id)] [
        HTML.img
          [ HTML.src "%s" (Media.url_of_id m.id)
          ; HTML.alt "%s" name
          ; HTML.style_
     "max-width:12rem;max-height:12rem;object-fit:contain;border:1px solid
     #ccc;background:#fafafa"
          ]
      ]
    else
      HTML.p [] [
        HTML.a [HTML.href "%s" (Media.url_of_id m.id)] [txt "download / open"]
      ]
  in
  HTML.div [HTML.class_ "card"; HTML.style_ "margin-bottom:1rem"] [
    thumb;
    HTML.p [HTML.class_ "meta"] [
      txt "%s ? %s ? %d bytes" name m.content_type m.byte_size;
    ];
    HTML.p [HTML.class_ "meta"] [
      HTML.code [] [txt "%s" m.id];
```

```
      txt " ? ";
      HTML.a [HTML.href "%s" (Media.url_of_id m.id)] [txt "raw"];
    ];
    HTML.form
      [ HTML.method_ `POST
      ; HTML.action "/media/%s/delete" m.id
      ; HTML.style_ "margin-top:0.5rem"
      ] [
      HTML.button
        [ HTML.type_ "submit"; HTML.class_ "secondary" ]
        [txt "Delete"];
    ];
  ]

let media_page (session : Session.t) (reqd : Httpun.Reqd.t)
    ~(target : string) : unit =
  match require_authed_user session with
  | Error () ->
      require_login_redirect reqd ~path:"/media" ~target;
      ()
  | Ok (author, is_admin) ->
      let msg = Option.value (query_param target "msg") ~default:"" in
      let err = Option.value (query_param target "err") ~default:"" in
      let rows =
        Media.list (Session.store session) ~author:(Some author) ~limit:100
          ()
      in
      let flash =
        if err <> "" then
          [ HTML.p [HTML.class_ "err"] [txt "%s" err] ]
        else if msg <> "" then
          [ HTML.p [HTML.class_ "ok"] [txt "%s" msg] ]
        else []
      in
      let allowed =
        String.concat ", " Media.allowed_content_types
      in
      let upload_card =
        HTML.div [HTML.class_ "card"] [
          HTML.h2 [] [txt "Upload"];
          HTML.p [HTML.class_ "meta"] [
            txt "Images and PDF, max 5 MiB. Allowed: %s." allowed;
          ];
          HTML.form
            [ HTML.method_ `POST
            ; HTML.action "/media/upload"
            ; HTML.enctype `formdata
            ] [
            HTML.p [] [
              HTML.label [HTML.for_ "file"] [txt "File"];
              txt " ";
              HTML.input
                [ HTML.type_ "file"
                ; HTML.name "file"
                ; HTML.id "file"
                ; HTML.required
                ; HTML.accept

    "image/png,image/jpeg,image/gif,image/webp,image/svg+xml,application/pdf,.png,.jpg,.jpeg,.gif,.webp,.svg,.pdf"
                ];
            ];
            HTML.button [HTML.type_ "submit"] [txt "Upload"];
          ];
          HTML.p [HTML.class_ "meta"] [
            txt "Agents: POST /api/media (raw or JSON) or ";
            HTML.code [] [txt "scripts/pp_media.py put ./shot.png"];
            txt ".";
          ];
        ]
      in
      let gallery =
        match rows with
        | [] ->
```

```
646             HTML.p [] [txt "(no media yet -- upload above)"]
647         | _ ->
648             HTML.div [] (List.map media_row_card rows)
649       in
650       let admin_note =
651         if is_admin then
652           HTML.p [HTML.class_ "meta"] [
653             txt "Admin: JSON list of all users via ";
654             HTML.code [] [txt "GET /api/media?all=1"];
655             txt ".";
656           ]
657         else HTML.null []
658       in
659       let body =
660         chrome ~session ~title:"Media" (
661           flash
662           @ [ HTML.p [HTML.class_ "meta"] [
663                 txt "Your binary blobs (images/PDF). Auth required to
    view.";
664               ]
665             ; upload_card
666             ; admin_note
667             ; HTML.h2 [] [txt "Yours"]
668             ; gallery
669             ]
670         )
671       in
672       html_response reqd ~session body;
673       ()
674
675 let media_upload_post (session : Session.t) (reqd : Httpun.Reqd.t)
676   : unit =
677   match require_authed_user session with
678   | Error () ->
679       require_login_redirect reqd ~path:"/media" ~target:"/media";
680       ()
681   | Ok (author, _) ->
682       let body_res =
683         read_body_limited reqd ~max_bytes:max_media_body_bytes
684       in
685       (match body_res with
686        | Error `Too_large ->
687            redirect_response reqd
688              ("/media?err=" ^ url_encode "file too large (max 5 MiB)");
689            ()
690        | Ok body ->
691            let req = Httpun.Reqd.request reqd in
692            let headers = header_list req in
693            let ct_hdr =
694              Option.map String.trim (header_ci headers "content-type")
695            in
696            (match
697               parse_upload_body ~content_type:ct_hdr ~filename_hdr:None body
698             with
699             | Error msg ->
700                 redirect_response reqd
701                   ("/media?err=" ^ url_encode msg);
702                 ()
703             | Ok (content_type, data, filename, meta_json) ->
704                 let res =
705                   store_upload session ~author ~content_type ~data
706                     ~filename ~meta_json
707                 in
708                 (match res with
709                  | Ok r ->
710                      let msg =
711                        Printf.sprintf "uploaded %s (%d bytes)" r.id
712                          r.byte_size
713                      in
714                      redirect_response reqd
715                        ("/media?msg=" ^ url_encode msg);
716                      ()
717                  | Error msg ->
```

```
                      redirect_response reqd
                        ("/media?err=" ^ url_encode msg);
                      ()))))

let media_delete_post (session : Session.t) (reqd : Httpun.Reqd.t)
    ~(id : string) : unit =
  match require_authed_user session with
  | Error () ->
      require_login_redirect reqd ~path:"/media" ~target:"/media";
      ()
  | Ok (actor, is_admin) ->
      (* Drain body if any (form POST). *)
      let _ = read_body_limited reqd ~max_bytes:(64 * 1024) in
      let res =
        Media.delete (Session.store session) ~id ~actor ~is_admin
      in
      (match res with
       | Ok (`Ok _) ->
           redirect_response reqd
             ("/media?msg=" ^ url_encode ("deleted " ^ id));
           ()
       | Ok `Not_found ->
           redirect_response reqd
             ("/media?err=" ^ url_encode "not found");
           ()
       | Ok `Forbidden ->
           redirect_response reqd
             ("/media?err=" ^ url_encode "not author or admin");
           ()
       | Error msg ->
           redirect_response reqd
             ("/media?err=" ^ url_encode msg);
           ())

(* Exposed for unit tests *)
let multipart_boundary_for_test = multipart_boundary
let parse_multipart_for_test = parse_multipart
let pick_upload_file_for_test = pick_upload_file
```

78 image/lib/http_ops.ml

```
(** /analytics page + ops HTTP helpers. *)
open Pure_html
open Http_core

let analytics_page (session : Session.t) (reqd : Httpun.Reqd.t) : unit =
  let store = Session.store session in
  let logins_ok, logins_fail = Ops_incidents.count_logins store in
  let summary = Ops_stats.summary_yojson ~logins_ok ~logins_fail in
  let incidents = Ops_incidents.list_open_incidents store in
  let recent = Ops_log.tail ~n:30 ~level:Ops_log.Warn () in
  let day =
    match summary with
    | `Assoc xs ->
        (match List.assoc_opt "day" xs with Some (`String d) -> d | _ ->
    "?")
    | _ -> "?"
  in
  let int_field name =
    match summary with
    | `Assoc xs ->
        (match List.assoc_opt name xs with Some (`Int n) -> n | _ -> 0)
    | _ -> 0
  in
  let top_rows =
    match summary with
    | `Assoc xs ->
        (match List.assoc_opt "top_paths" xs with
         | Some (`List paths) ->
             List.filter_map
               (function
                 | `Assoc p ->
                     let path =
                       match List.assoc_opt "path" p with
                       | Some (`String s) -> s
                       | _ -> "?"
                     in
                     let n =
                       match List.assoc_opt "n" p with
                       | Some (`Int i) -> i
                       | _ -> 0
                     in
                     Some
                       (HTML.tr []
                          [ HTML.td [] [txt "%s" path]
                          ; HTML.td [] [txt "%d" n]
                          ])
                 | _ -> None)
               paths
         | _ -> [])
    | _ -> []
  in
  let incident_items =
    match incidents with
    | `List xs ->
        List.filter_map
          (function
            | `Assoc a ->
                let comp =
                  match List.assoc_opt "component" a with
                  | Some (`String s) -> s
                  | _ -> "?"
                in
                let sample =
                  match List.assoc_opt "sample_msg" a with
                  | Some (`String s) -> s
                  | _ -> ""
                in
                let count =
                  match List.assoc_opt "count" a with
                  | Some (`Int n) -> n
                  | _ -> 1
```

```
                in
                let todo =
                  match List.assoc_opt "todo_id" a with
                  | Some (`String s) when s <> "" -> s
                  | _ -> ""
                in
                let sample_short =
                  if String.length sample > 100 then
                    String.sub sample 0 100 ^ "?"
                  else sample
                in
                Some
                  (HTML.li []
                     [ HTML.strong [] [txt "%s ?%d" comp count]
                     ; txt " -- %s" sample_short
                     ; (if todo = "" then txt ""
                        else
                          HTML.span []
                            [ txt " "
                            ; HTML.a [HTML.href "/todos"] [txt "todo"]
                            ])
                     ])
            | _ -> None)
          xs
    | _ -> []
  in
  let log_items =
    List.map
      (fun (e : Ops_log.event) ->
        HTML.li []
          [ HTML.code [] [txt "%s" (Ops_log.level_to_string e.level)]
          ; txt " [%s] %s" e.component e.msg
          ])
      (List.rev recent)
  in
  let body =
    chrome_lwt ~session ~title:"Analytics"
      [ HTML.h1 [] [txt "Analytics"]
      ; HTML.p []
          [ txt
              "Observatory for today (UTC). Errors become todos -- see open
   \
               incidents."
          ]
      ; HTML.div [HTML.class_ "card"]
          [ HTML.h2 [] [txt "Today ? %s" day]
          ; HTML.ul []
              [ HTML.li []
                  [ txt "Public unique viewers: %d"
                      (int_field "public_uniques")
                  ]
              ; HTML.li []
                  [txt "Page views: %d" (int_field "page_views")]
              ; HTML.li []
                  [ txt "Logins ok / fail: %d / %d" logins_ok logins_fail
                  ]
              ; HTML.li []
                  [ txt "Active users: %d" (int_field "active_users")
                  ]
              ; HTML.li []
                  [ txt "Errors / fatals (this process): %d / %d"
                      (int_field "errors") (int_field "fatals")
                  ]
              ; HTML.li []
                  [txt "Noise/bot views: %d" (int_field "noise_views")]
              ]
          ]
      ; HTML.div [HTML.class_ "card"]
          [ HTML.h2 [] [txt "Top paths"]
          ; (if top_rows = [] then HTML.p [] [txt "No traffic yet."]
             else
               HTML.table []
                 (HTML.tr []
```

```
                    [ HTML.th [] [txt "Path"]
                    ; HTML.th [] [txt "Views"]
                    ]
                :: top_rows))
        ]
    ; HTML.div [HTML.class_ "card"]
        [ HTML.h2 [] [txt "Open reliability incidents"]
        ; HTML.p []
            [ txt
                "New error fingerprints auto-file a note + TODO. \
                 Repeats bump the count only."
            ]
        ; (if incident_items = [] then
             HTML.p [HTML.class_ "ok"] [txt "None open."]
           else HTML.ul [] incident_items)
        ; HTML.p []
            [ HTML.a [HTML.href "/todos"] [txt "Open todos ?"]
            ; txt " ? "
            ; HTML.a [HTML.href "/notes"] [txt "Notes ?"]
            ]
        ]
    ; HTML.div [HTML.class_ "card"]
        [ HTML.h2 [] [txt "Recent log (warn+)"]
        ; (if log_items = [] then HTML.p [] [txt "Quiet."]
           else HTML.ul [] log_items)
        ]
    ]
in
html_response reqd ~session body;
()
```

79 image/lib/http_pages.ml

```
(** Pricklypear Image -- HTTP system & product page handlers. *)
open Pure_html
open Http_core
open Http_data
open Http_branches
open Http_agent

let show_binding_page (session : Session.t) (name : string)
    (reqd : Httpun.Reqd.t) : unit =
  let found = Store.lookup_function (Session.store session) name in
  let body =
    match found with
    | None ->
        chrome ~session ~title:name [
          HTML.p [] [txt "(no such binding on this branch)"];
          HTML.p [] [HTML.a [HTML.href "%s" ("/edit/" ^ name)] [txt "create it"]];
        ]
    | Some (expr, author, _doc) ->
        let snap = Call_graph.snapshot_of_session session in
        let mutual = Call_graph.mutual_of snap name in
        let cx =
          Pricklypear_lisp.Complexity.badge
            (Pricklypear_lisp.Complexity.classify ~fn_name:name expr)
        in
        let db_prims = Db_cost.db_prims_of_expr expr in
        let db_nodes =
          if db_prims = [] then [ txt "" ]
          else
            List.concat_map
              (fun p ->
                [
                  HTML.span [ HTML.class_ "badge db-cost" ]
                    [ txt "%s: %s" p (Db_cost.badge (Db_cost.static_cost p)) ];
                  txt " ";
                ])
              db_prims
        in
        let stats_node =
          match
            Crud.find_row (Session.store session) ~type_name:"function_stat"
              ~field:"fn" ~value:name
          with
          | Pricklypear_lisp.Ast.VStr fields_json ->
              (try
                 let j = Yojson.Safe.from_string fields_json in
                 let open Yojson.Safe.Util in
                 let steps = member "steps" j |> to_int_option |> Option.value ~default:0 in
                 let prims = member "prim_calls" j |> to_int_option |> Option.value ~default:0 in
                 let dbh = member "db_hits" j |> to_int_option |> Option.value ~default:0 in
                 HTML.p [ HTML.class_ "meta last-run" ]
                   [ txt "last run: %d steps, %d prim calls, %d DB hits" steps prims dbh ]
               with _ -> HTML.p [ HTML.class_ "muted" ] [ txt "" ])
          | _ -> HTML.p [ HTML.class_ "muted" ] [ txt "(no measured run yet)" ]
        in
        let explain_text =
          match Analyze.explain_function (Session.store session) ~fn_name:name with
          | Ok t -> t
          | Error _ -> ""
        in
        chrome ~session ~title:name [
          HTML.p [] [txt "author: %s" author];
          HTML.p []
            (Ast_html.recursion_badges ~fn_name:name expr
```

```
             @ [
                 txt " ";
                 HTML.span [ HTML.class_ "badge complexity" ] [ txt "%s" cx
    ];
                 txt " ";
               ]
             @ db_nodes);
          stats_node;
          Ast_html.pp_expr_html ~fn_name:name ~mutual expr;
          HTML.p [] [
            HTML.button
              [
                HTML.type_ "button";
                HTML.class_ "btn";
                Hx.post "%s" ("/api/analyze/" ^ Packages.name_encode name);
                Hx.target "#analyze-result";
                Hx.swap "innerHTML";
              ]
              [ txt "Analyze" ];
            txt " ";
            HTML.button
              [
                HTML.type_ "button";
                HTML.class_ "btn secondary";
                Hx.post "%s" ("/api/analyze/" ^ Packages.name_encode name ^
    "?llm=1");
                Hx.target "#analyze-result";
                Hx.swap "innerHTML";
              ]
              [ txt "Analyze (LLM)" ];
            txt " ";
            HTML.button
              [
                HTML.type_ "button";
                HTML.class_ "btn secondary";
                Hx.post "%s" ("/api/explain/" ^ Packages.name_encode name);
                Hx.target "#explain-result";
                Hx.swap "innerHTML";
              ]
              [ txt "Explain" ];
          ];
          HTML.div [ HTML.id "analyze-result" ] [ txt "" ];
          HTML.details [ HTML.class_ "explain-panel" ]
            [
              HTML.summary [] [ txt "Inside-out breakdown" ];
              HTML.div [ HTML.id "explain-result" ]
                [
                  HTML.pre [ HTML.class_ "bb-source explain-layer" ]
                    [ txt "%s" explain_text ];
                ];
            ];
          HTML.p [] [
            HTML.a [HTML.href "%s" (Packages.edit_href name)] [txt "edit"];
            txt " ? ";
            HTML.a [HTML.href "%s" (Packages.inspect_href name)] [txt
    "inspect"];
          ];
        ]
  in
  html_response reqd ~session body;
  ()

open Http_data

let about_page (session : Session.t) (reqd : Httpun.Reqd.t) : unit =
  let body =
    chrome ~session ~title:"About Pricklypear" [
      HTML.div [HTML.class_ "card"] [
        HTML.h2 [] [txt "What is this?"];
        HTML.p [] [txt
          "Pricklypear is a multi-user live computing environment. You log
    into your running image, write Lisp functions that are stored as
    durable data in Postgres, branch code like git while sharing data,
```

```
      bind functions to URLs, and optionally let AI propose functions onto
      your branch."];
132       HTML.p [] [txt
133         "It is Smalltalk-like (live, inspectable, editable) without being
      Smalltalk (no single-user OOP image dump). The image is the CPU;
      Postgres is the disk."];
134        ];
135        HTML.div [HTML.class_ "card"] [
136          HTML.h2 [] [txt "Core concepts"];
137          HTML.ul [] [
138            HTML.li [] [txt "Image -- your running personal process (REPL +
      HTTP). Disposable; rebuildable from the store."];
139            HTML.li [] [txt "Store -- Postgres holds functions, types,
      routes, data rows, auth, branches."];
140            HTML.li [] [txt "Function / binding -- a named live value,
      usually a lambda. Top-level (define ...) persists it."];
141            HTML.li [] [txt "Branch -- versioned code world (system/main,
      alice/experiment). Code branches; data is shared."];
142            HTML.li [] [txt "Type + row -- (deftype ...) then
      (create/get/update/rows ...); rows are per-user plus shared seed."];
143            HTML.li [] [txt "Route / page -- (route \"/path\" \"handler\")
      makes a URL call a function."];
144            HTML.li [] [txt "AI propose -- model writes a function on the
      current branch as agent:<model>; you keep or discard."];
145          ];
146        ];
147        HTML.div [HTML.class_ "card"] [
148          HTML.h2 [] [txt "How to access"];
149          HTML.ul [] [
150            HTML.li [] [txt "Start stack: scripts/dev.sh start"];
151            HTML.li [] [txt "Text login/REPL: scripts/repl.sh  (SSH public
      key line protocol on :17822)"];
152            HTML.li [] [txt "This browser UI: usually http://127.0.0.1:18080/
      after an image boots"];
153            HTML.li [] [txt "Browser: /  Workspace home ? /about ? /bindings
      Functions ? /pages Pages ? /types Data ? /repl ? /edit/<name>"];
154            HTML.li [] [txt "Auth admin API: http://127.0.0.1:17879  (create
      users, tokens)"];
155            HTML.li [] [txt "Full docs: docs/concepts-and-horizon.md in the
      repo"];
156          ];
157        ];
158        HTML.div [HTML.class_ "card"] [
159          HTML.h2 [] [txt "5-minute first tour"];
160          HTML.pre [] [txt "%s"
161            "1. scripts/repl.sh\n\
162   2. (define hi-page (lambda () (render-html \"<h1>Hello</h1>\")))\n\
163   3. (route \"/hello\" \"hi-page\")\n\
164   4. open /hello in the browser\n\
165   5. edit at /edit/hi-page\n\
166   6. optional: (propose-function \"double\" \"lambda of int x -> (* x 2)\")"];
167        ];
168        HTML.div [HTML.class_ "card"] [
169          HTML.h2 [] [txt "Long-term possibilities"];
170          HTML.ul [] [
171            HTML.li [] [txt "Personal OS -- always-on image; notes, CRM,
      automations as live functions."];
172            HTML.li [] [txt "Community commons -- system/main as shared
      tools; personal gardens as branches."];
173            HTML.li [] [txt "AI workshop -- AI cohabits the image; propose
      apps (types+fns+routes); merge selectively."];
174            HTML.li [] [txt "Internal tools platform -- inspectable live
      ops/admin tools without redeploying for every change."];
175            HTML.li [] [txt "Education -- legible multi-user systems/PL lab
      with forks and merges."];
176            HTML.li [] [txt "End-user products -- friendlier skin
      (Functions/Pages/Data) on the same substrate."];
177            HTML.li [] [txt "Hardened service -- Solo5, real SSH/web login,
      stronger auth and permissions."];
178          ];
179          HTML.p [] [
180            HTML.a [HTML.href "/"] [txt "back home"];
181            txt " ? ";
```

```
          HTML.a [HTML.href "/repl"] [txt "open REPL"];
        ];
      ];
    ]
  in
  html_response reqd ~session body

(* Page path for inspector: "/hello" -> "hello" for URL segment. *)
let page_inspect_path (route : string) : string =
  if String.length route > 0 && route.[0] = '/' then
    String.sub route 1 (String.length route - 1)
  else route

let pages_list_page (session : Session.t) (reqd : Httpun.Reqd.t) : unit =
  let routes = Session.list_pages_meta session in
  let rows =
    List.map
      (fun (route, handler, vis) ->
        let inspect = "/page/" ^ page_inspect_path route in
        HTML.tr [] [
          HTML.td [] [HTML.a [HTML.href "%s" route] [txt "%s" route]];
          HTML.td [] [
            HTML.a [HTML.href "%s" (Packages.edit_href handler)] [txt "%s"
    handler];
          ];
          HTML.td [] [txt "%s" vis];
          HTML.td [] [
            HTML.a [HTML.href "%s" route] [txt "open"];
            txt " ? ";
            HTML.a [HTML.href "%s" (Packages.edit_href handler)] [txt "edit
    function"];
            txt " ? ";
            HTML.a [HTML.href "%s" inspect] [txt "inspect page"];
          ];
        ])
      routes
  in
  let body =
    chrome ~session ~title:"Pages" [
      HTML.p [HTML.class_ "meta"] [
        txt "Page = URL endpoint. Function = code it runs. Visibility:
    private (login) or public (anonymous GET). ";
        txt "(route path handler 'public) to open a page.";
      ];
      (match routes with
       | [] ->
           HTML.p [] [
             txt "(no pages yet -- use the Hello recipe on Home, or (route
    \"/x\" \"fn\") in the REPL)";
           ]
       | _ ->
           HTML.table [] [
             HTML.thead [] [
               HTML.tr [] [
                 HTML.th [] [txt "page (URL)"];
                 HTML.th [] [txt "function (handler)"];
                 HTML.th [] [txt "visibility"];
                 HTML.th [] [txt "actions"];
               ]
             ];
             HTML.tbody [] rows;
           ]);
    ]
  in
  html_response reqd ~session body

(** Labeled emergency home -- NOT a second product workspace.
    Real home is Lisp ui/home (libs/ui). See image.system-browser. *)
let emergency_home_content (session : Session.t) : Pure_html.node list =
  let who =
    Printf.sprintf
      "You are %s on branch %s. Product home is Lisp (ui/home); this page
    is host recovery only."
```

```
      (Session.username session) (Session.branch_name session)
  in
  [ HTML.div [HTML.class_ "card emergency-home"] [
      HTML.h2 [] [txt "EMERGENCY HOME -- ui/home missing or failed"];
      HTML.p [] [txt "%s" who];
      HTML.p [] [
        HTML.a [HTML.href "/bindings?pkg=ui"] [txt "Open ui package"];
        txt " ? ";
        HTML.a [HTML.href "/bindings"] [txt "Functions"];
        txt " ? ";
        HTML.a [HTML.href "/repl"] [txt "REPL"];
        txt " ? ";
        HTML.a [HTML.href "/about"] [txt "About"];
        txt " ? ";
        HTML.a [HTML.href "/login"] [txt "Login"];
      ];
      HTML.p [HTML.class_ "muted"] [
        txt "%s"
          "Fix: ensure system/main has ui/* from seed, or delete a broken personal pin of ui/home / ui/menu / ui/nav.";
      ];
    ]
  ]

let home_page (session : Session.t) (reqd : Httpun.Reqd.t) : unit =
  (* Nav comes from Session.ui_nav_html (refreshed in request launch). *)
  let home_html = try_ui_fn session "ui/home" in
  let content =
    match home_html with
    | Some html when String.trim html <> "" ->
        [ txt ~raw:true "%s" html ]
    | _ -> emergency_home_content session
  in
  let body = chrome ~session ~title:"" content in
  html_response reqd ~session body;
  ()

let repl_page (session : Session.t) (reqd : Httpun.Reqd.t) : unit =
  let js =
    "const out=document.getElementById('out');" ^
    "document.getElementById('run').onclick=async()=>{" ^
    "const expr=document.getElementById('expr').value;" ^
    "out.textContent='...';" ^
    "try{" ^
    "const r=await fetch('/api/eval',{method:'POST'," ^
    "headers:{'content-type':'application/json'}," ^
    "body:JSON.stringify({expr})});" ^
    "const j=await r.json();" ^
    "out.textContent=j.ok?j.value:('error: '+j.error);" ^
    "out.className=j.ok?'ok':'err';" ^
    "}catch(e){out.textContent=String(e);out.className='err';}" ^
    "};" ^
    "document.getElementById('clear').onclick=()=>{out.textContent='';};"
  in
  let body =
    chrome ~session ~title:"Browser REPL" [
      HTML.div [HTML.class_ "card"] [
        HTML.p [] [txt "Eval against the live session env. (define ...) persists."];
        HTML.textarea [HTML.id "expr"] "%s" "(+ 1 2)";
        HTML.p [] [
          HTML.button [HTML.id "run"; HTML.type_ "button"] [txt "Eval"];
          txt " ";
          HTML.button [HTML.id "clear"; HTML.type_ "button"; HTML.class_ "secondary"]
            [txt "Clear out"];
        ];
        HTML.pre [HTML.id "out"] [txt ""];
      ];
      HTML.script [] "%s" js;
    ]
  in
  html_response reqd ~session body
```

```
320
321 (* Default stub body for a brand-new function. *)
322 let stub_body (name : string) : string =
323   Printf.sprintf "(lambda ()\n  \"TODO: implement %s\")" name
324
325 (* Static call graph -- shared module (image.inspector I1). *)
326 let collect_var_apps = Call_graph.collect_var_apps
327 let page_function_graph = Call_graph.page_function_graph
328
329 let contains_ci (hay : string) (needle : string) : bool =
330   if needle = "" then true
331   else
332     let h = String.lowercase_ascii hay in
333     let n = String.lowercase_ascii needle in
334     let rec loop i =
335       if i + String.length n > String.length h then false
336       else if String.sub h i (String.length n) = n then true
337       else loop (i + 1)
338     in
339     loop 0
340
341 (** function-name ? routes that use it as the *direct* HTTP handler.
342
343     Previously this ran [page_function_graph] for every route (recursive
344     AST walk + Store.lookup_function per callee, depth 8) -- hundreds of
345     PG round-trips on every /bindings load. "Used by" on the package
346     list only needs the direct handler mapping; deep graphs stay on
347     /page/<route> inspect. *)
348 let used_by_pages_map (session : Session.t)
349     : (string, string list) Hashtbl.t =
350   let routes = Session.list_pages session in
351   let tbl : (string, string list) Hashtbl.t = Hashtbl.create 32 in
352   List.iter
353     (fun (route, handler) ->
354       let prev = try Hashtbl.find tbl handler with Not_found -> [] in
355       if List.mem route prev then ()
356       else Hashtbl.replace tbl handler (route :: prev))
357     routes;
358   tbl
359
360 let editor_form_partial ~(name : string) ~(body_src : string) ?(msg = "")
361     ?(return_to = "") () : Pure_html.node =
362   let flash =
363     if msg = "" then HTML.div [] []
364     else
365       HTML.p
366         [ HTML.class_
367             (if String.length msg >= 5 && String.sub msg 0 5 = "error" then
    "err"
368              else "ok") ]
369         [ txt "%s" msg ]
370   in
371   let view_href =
372     if return_to <> "" then
373       (* Stay in bindings browser; drop tab=edit ? definition. *)
374       (try
375          let base =
376            match String.split_on_char '&' return_to with
377            | [] -> return_to
378            | parts ->
379                parts
380                |> List.filter (fun p ->
381                       not (String.starts_with ~prefix:"tab=" p)
382                       && not (String.starts_with ~prefix:"msg=" p))
383                |> String.concat "&"
384          in
385          base
386        with _ -> return_to)
387     else "/binding/" ^ Packages.name_encode name
388   in
389   HTML.div [HTML.id "editor-root"] [
390     flash;
391     HTML.div [HTML.class_ "card bb-edit-form"] [
```

```
      HTML.form
        [ HTML.method_ 'POST
        ; HTML.action "%s" ("/edit/" ^ Packages.name_encode name)
        ; HTML.enctype 'urlencoded ]
        [ (if return_to = "" then HTML.div [] []
           else
             HTML.input
               [ HTML.type_ "hidden"
               ; HTML.name "return_to"
               ; HTML.value "%s" return_to ]);
          HTML.p [] [
            HTML.label [HTML.for_ "name"] [txt "name "];
            HTML.input
              [ HTML.type_ "text"
              ; HTML.name "name"
              ; HTML.id "name"
              ; HTML.value "%s" name ];
          ]
        ; HTML.p [] [HTML.label [HTML.for_ "body"] [txt "body
    (s-expression)"]]
        ; (HTML.textarea [HTML.name "body"; HTML.id "body"] "%s" body_src)
        ; HTML.p [] [
            HTML.button [HTML.type_ "submit"] [txt "Save (define +
    persist)"];
            txt " ";
            HTML.a
              [ HTML.href "%s" view_href; HTML.class_ "btn secondary" ]
              [ txt "View" ];
          ];
        ];
    ];
  ]

let list_bindings_page (session : Session.t) (reqd : Httpun.Reqd.t)
    : unit =
  let target = (Httpun.Reqd.request reqd).Httpun.Request.target in
  let q = Option.value (query_param target "q") ~default:"" |> String.trim
     in
  let pkg_q = query_param target "pkg" in
  let fn_q = query_param target "fn" in
  let defs_meta =
    Store.list_function_names_meta (Session.store session)
  in
  (* Effective view only: one row per name (closest branch wins). *)
  let defs =
    List.map (fun (n, e, a, d) -> (n, e, a, d)) defs_meta
  in
  let used_by = used_by_pages_map session in
  let kernel = Session.kernel_catalog session in
  let matches_ws (name, _body, author, _depth) =
    q = ""
    || contains_ci name q
    || contains_ci (Packages.package_of_name name) q
    || contains_ci author q
  in
  let defs_f = List.filter matches_ws defs in
  (* H0.2: derive-on-read known_fn so badges account for callees. *)
  let known_fn =
    Pricklypear_lisp.Effect.known_fn_of_bindings
      (List.map (fun (n, e, _, _) -> (n, e)) defs_f)
  in
  let badge_of expr =
    Pricklypear_lisp.Effect.EffSet.badge
      (Pricklypear_lisp.Effect.infer ~known_fn expr)
  in
  let txn_hints_of expr =
    Pricklypear_lisp.Effect.txn_lint ~known_fn expr
  in
  let txn_hint_nodes expr =
    match txn_hints_of expr with
    | [] -> []
    | hs ->
        [ HTML.div [HTML.class_ "card"]
```

```
            (HTML.p [HTML.class_ "muted"] [txt "txn lint (soft):"]
             :: List.map
                  (fun h -> HTML.p [HTML.class_ "err"] [txt "%s" h])
                  hs)
      ]
in
(* group_by_package expects 3-tuples; strip depth for grouping. *)
let defs_for_group =
  List.map (fun (n, e, a, _) -> (n, e, a)) defs_f
in
let groups = Packages.group_by_package defs_for_group in
let depth_of name =
  match List.find_opt (fun (n, _, _, _) -> n = name) defs_f with
  | Some (_, _, _, d) -> d
  | None -> 0
in
let origin_badge depth author =
  if depth = 0 then
    HTML.span [HTML.class_ "ok"] [txt "own"]
  else
    HTML.span [HTML.class_ "muted"]
      [txt "inherited ? %s" author]
in
let used_links name =
  let routes = try Hashtbl.find used_by name with Not_found -> [] in
  match List.rev routes with
  | [] -> HTML.span [HTML.class_ "muted"] [txt "--"]
  | rs ->
      HTML.span []
        (List.concat
           (List.mapi
              (fun i route ->
                let link =
                  HTML.a
                    [HTML.href "%s" ("/page/" ^ page_inspect_path route)]
                    [txt "%s" route]
                in
                if i = 0 then [link] else [txt " "; link])
              rs))
in
let pkg_href pkg =
  "/bindings?pkg=" ^ Packages.name_encode pkg
in
let fn_href ?tab pkg fn =
  let base =
    "/bindings?pkg=" ^ Packages.name_encode pkg ^ "&fn="
    ^ Packages.name_encode fn
  in
  match tab with
  | None | Some "" | Some "view" -> base
  | Some t -> base ^ "&tab=" ^ t
in
let tab_q =
  match query_param target "tab" with
  | Some t -> String.lowercase_ascii (String.trim t)
  | None -> "view"
in
let tab_q =
  match tab_q with
  | "edit" | "inspect" | "observe" | "observability" ->
      if tab_q = "observability" then "observe" else tab_q
  | _ -> "view"
in
let flash_msg =
  Option.value (query_param target "msg") ~default:"" |> String.trim
in
match pkg_q, fn_q with
| Some "special-forms", Some fname ->
    let entry =
      Pricklypear_lisp.Special_forms.find fname
    in
    let body =
      match entry with
```

```
      | None ->
          chrome ~session ~title:("Special form " ^ fname) [
            HTML.p [] [
              HTML.a [HTML.href "/bindings"] [txt "Functions"];
              txt " ? ";
              HTML.a [HTML.href "/bindings?pkg=special-forms"]
                [txt "(special-forms)"];
            ];
            HTML.p [HTML.class_ "err"] [txt "unknown special form %s" fname];
          ]
      | Some e ->
          chrome ~session ~title:("Special form " ^ e.name) [
            HTML.p [] [
              HTML.a [HTML.href "/bindings"] [txt "Functions"];
              txt " ? ";
              HTML.a [HTML.href "/bindings?pkg=special-forms"]
                [txt "(special-forms)"];
              txt " ? %s" e.name;
            ];
            HTML.div [HTML.class_ "card"] [
              HTML.p [] [
                HTML.span [HTML.class_ "muted"] [txt "kind: "];
                txt "special form (evaluator)";
              ];
              HTML.p [] [
                HTML.span [HTML.class_ "muted"] [txt "name: "];
                txt "%s" e.name;
              ];
              HTML.p [] [
                HTML.span [HTML.class_ "muted"] [txt "arity: "];
                txt "%s" e.arity;
              ];
              HTML.p [] [txt "%s" e.summary];
              HTML.h2 [] [txt "example"];
              HTML.pre [] [txt "%s" e.examples];
              HTML.p [HTML.class_ "muted"] [
                txt "Handled in lisp/lib/eval.ml -- not an env binding, not a prim, not editable as Lisp. Change requires OCaml + tests + image deploy.";
              ];
            ];
          ]
    in
    html_response reqd ~session body;
    ()
| Some "kernel", Some fname ->
    let summary =
      match List.assoc_opt fname kernel with
      | Some s -> s
      | None -> "unknown primitive"
    in
    let body =
      chrome ~session ~title:("Primitive " ^ fname) [
        HTML.p [] [
          HTML.a [HTML.href "/bindings"] [txt "Functions"];
          txt " ? ";
          HTML.a [HTML.href "/bindings?pkg=kernel"] [txt "(kernel)"];
          txt " ? %s" fname;
        ];
        HTML.div [HTML.class_ "card"] [
          HTML.p [] [
            HTML.span [HTML.class_ "muted"] [txt "kind: "];
            txt "primitive (OCaml)";
          ];
          HTML.p [] [
            HTML.span [HTML.class_ "muted"] [txt "name: "];
            txt "%s" fname;
          ];
          HTML.p [] [txt "%s" summary];
          HTML.p [HTML.class_ "muted"] [
            txt "Built into the image kernel -- not stored as Lisp. App logic should usually be Lisp functions you can edit.";
```

```
            ];
          ];
        ]
      in
      html_response reqd ~session body;
      ()
  | Some pkg, Some local ->
      let full =
        if String.contains local '/' then local
        else if pkg = "" || pkg = "(root)" || pkg = "root" then local
        else Packages.join_name ~package:pkg ~local
      in
      let found = Store.lookup_function (Session.store session) full in
      let crumb_pkg = if pkg = "(root)" then "" else pkg in
      let body =
        match found with
        | None ->
            (chrome ~session ~title:full [
                HTML.p [] [
                  HTML.a [HTML.href "/bindings"] [txt "Functions"];
                  txt " ? ";
                  HTML.a [HTML.href "%s" (pkg_href pkg)]
                    [txt "%s" (Packages.package_label crumb_pkg)];
                ];
                HTML.p [HTML.class_ "err"] [txt "no function %s" full];
                HTML.p [] [
                  HTML.a [HTML.href "%s" (Packages.edit_href full)] [txt
    "create it"];
                ];
              ])
        | Some (expr, author, _doc) ->
            let badge = badge_of expr in
            let cx_badge =
              Pricklypear_lisp.Complexity.badge
                (Pricklypear_lisp.Complexity.classify ~fn_name:full expr)
            in
            let depth = depth_of full in
            let snap = Call_graph.snapshot_of_session session in
            let callees = Call_graph.callees snap ~name:full ~depth:1 in
            let callers = Call_graph.callers snap full in
            let routes_ub = Call_graph.used_by_routes snap full in
            let holes = Call_graph.holes snap full in
            let impact_badge =
              match
                List.find_opt
                  (fun (e : Session_types.impact_event) -> e.name = full)
                  session.impact_queue
              with
              | Some e when e.status = "pending" ->
                  HTML.p [] [
                    HTML.span [HTML.class_ "badge impact"]
                      [txt "impact pending ? %d dependents ? %d routes"
                         (List.length e.dependents) (List.length e.routes)];
                  ]
              | _ -> HTML.p [HTML.class_ "muted"] [txt ""]
            in
            let name_href n =
              let pkg = Packages.package_of_name n in
              let local = Packages.local_name n in
              fn_href pkg local
            in
            let name_links names =
              if names = [] then [txt "(none)"]
              else
                List.concat_map
                  (fun n ->
                    [ HTML.a [HTML.href "%s" (name_href n)] [txt "%s" n]
                    ; txt " "
                    ])
                  names
            in
            let route_links paths =
              if paths = [] then [txt "(none)"]
```

```
            else
              List.concat_map
                (fun p ->
                  let seg =
                    if String.length p > 0 && p.[0] = '/' then
                      String.sub p 1 (String.length p - 1)
                    else p
                  in
                  [ HTML.a [HTML.href "%s" ("/page/" ^ seg)] [txt "%s" p]
                  ; txt " "
                  ])
                paths
        in
        let hole_nodes =
          if holes = [] then HTML.p [HTML.class_ "muted"] [txt "no holes"]
          else
            HTML.ul []
              (List.map
                 (fun h ->
                   HTML.li [] [
                     txt "%s " h;
                     HTML.a
                       [ HTML.href "%s" (Packages.edit_href h)
                       ; HTML.class_ "btn secondary" ]
                       [txt "Stub/Edit"];
                   ])
                 holes)
        in
        (* Smalltalk/iTunes column browser:
           top = packages | functions | meta
           bottom = full-width definition (code pane). *)
        let pkg_selected p =
          let key = if p = "" then "root" else p in
          key = pkg || p = crumb_pkg || Packages.package_label p = pkg
        in
        let pkg_col =
          HTML.div [HTML.class_ "card bb-col"] [
            HTML.h3 [] [txt "Packages"];
            HTML.ul [HTML.class_ "bb-list"]
              (List.map
                 (fun (p, _) ->
                   let key = if p = "" then "root" else p in
                   let lab = Packages.package_label p in
                   let cls =
                     if pkg_selected p then "bb-item is-active" else "bb-item"
                   in
                   HTML.li [HTML.class_ "%s" cls] [
                     HTML.a [HTML.href "%s" (pkg_href key)] [txt "%s" lab];
                   ])
                 groups);
          ]
        in
        let fns_in_pkg =
          let rows =
            try List.assoc crumb_pkg groups with Not_found ->
              (try List.assoc pkg groups with Not_found -> [])
          in
          HTML.div [HTML.class_ "card bb-col"] [
            HTML.h3 [] [txt "%s" (Packages.package_label crumb_pkg)];
            HTML.ul [HTML.class_ "bb-list"]
              (List.map
                 (fun (n, _, _) ->
                   let loc = Packages.local_name n in
                   let cls =
                     if n = full then "bb-item is-active" else "bb-item"
                   in
                   HTML.li [HTML.class_ "%s" cls] [
                     HTML.a [HTML.href "%s" (fn_href pkg loc)] [txt "%s" loc];
                   ])
```

```
                    rows);
            ]
          in
          let meta_col =
            HTML.div [HTML.class_ "card bb-col bb-meta"] (
            [ HTML.p [HTML.class_ "bb-crumb"] [
                 HTML.a [HTML.href "/bindings"] [txt "Functions"];
                 txt " ? ";
                 HTML.a [HTML.href "%s" (pkg_href pkg)]
                   [txt "%s" (Packages.package_label crumb_pkg)];
                 txt " ? ";
                 HTML.strong [] [txt "%s" (Packages.local_name full)];
               ];
               impact_badge;
               HTML.p [] [
                 HTML.span [HTML.class_ "muted"] [txt "kind: "];
                 txt "function (Lisp)";
               ];
               HTML.p [] [
                 HTML.span [HTML.class_ "muted"] [txt "origin: "];
                 origin_badge depth author;
               ];
               HTML.p [] [
                 HTML.span [HTML.class_ "muted"] [txt "author: "];
                 txt "%s" author;
               ];
               HTML.p [] [
                 HTML.span [HTML.class_ "muted"] [txt "effects: "];
                 txt "%s" badge;
               ];
               HTML.p [] [
                 HTML.span [HTML.class_ "muted"] [txt "used by (handler): "];
                 used_links full;
               ];
               HTML.p []
                 ([ HTML.span [HTML.class_ "muted"] [txt "used-by-routes: "] ]
                  @ route_links routes_ub);
               HTML.p []
                 ([ HTML.span [HTML.class_ "muted"] [txt "callers: "] ]
                  @ name_links callers);
               HTML.p []
                 ([ HTML.span [HTML.class_ "muted"] [txt "callees: "] ]
                  @ name_links callees);
               HTML.p [] [
                 HTML.a [HTML.href "/changes"] [txt "Branch changes"];
               ];
               HTML.h3 [] [txt "Holes"];
               hole_nodes;
             ]
             @ txn_hint_nodes expr)
          in
          let mutual = Call_graph.mutual_of snap full in
          let tab_link id label =
            let active = tab_q = id in
            HTML.a
              [
                HTML.href "%s" (fn_href ~tab:id pkg local);
                HTML.class_ "%s"
                  (if active then "bb-tab is-active" else "bb-tab");
              ]
              [ txt "%s" label ]
          in
          let tabs =
            HTML.div []
              [
                (* Inline critical tab chrome so stale ui/css seeds still look right. *)
                HTML.style [] "%s"

".bb-tabs{display:flex;gap:.35rem;flex-wrap:wrap;margin:.4rem 0 .6rem}
```

```
.bb-tab{display:inline-block;padding:.28rem .65rem;border:1px solid
    #000;text-decoration:none;color:inherit;background:#f7f5ef}
.bb-tab.is-active{background:#1a1a1a;color:#f7f5ef;font-weight:700}
.bb-tab:hover{background:#ece9e0}";
                  HTML.nav [ HTML.class_ "bb-tabs" ]
                    [
                      tab_link "view" "Definition";
                      tab_link "edit" "Edit";
                      tab_link "inspect" "Inspect";
                      tab_link "observe" "Observability";
                    ];
                ]
            in
            let flash_node =
              if flash_msg = "" then HTML.div [] []
              else
                HTML.p
                  [
                    HTML.class_
                      (if String.length flash_msg >= 5
                          && String.sub flash_msg 0 5 = "error"
                       then "err"
                       else "ok");
                  ]
                  [ txt "%s" flash_msg ]
            in
            let view_pane =
              Ast_html.pp_expr_html ~fn_name:full ~mutual expr
            in
            let edit_return = fn_href ~tab:"edit" pkg local in
            let edit_pane =
              let src = Pricklypear_lisp.Reader.pp_expr expr in
              editor_form_partial ~name:full ~body_src:src ~msg:flash_msg
                ~return_to:edit_return ()
            in
            let inspect_pane =
              let j =
                try
                  Yojson.Safe.pretty_to_string
                    (Pricklypear_lisp.Ast.expr_to_json expr)
                with _ -> "(json unavailable)"
              in
              HTML.div [ HTML.class_ "bb-inspect" ]
                [
                  HTML.p []
                    (Ast_html.recursion_badges ~fn_name:full expr
                     @ [
                         txt " ";
                         HTML.span [ HTML.class_ "badge complexity" ]
                           [ txt "%s" cx_badge ];
                       ]);
                  HTML.h3 [] [ txt "pretty" ];
                  Ast_html.pp_expr_html ~fn_name:full ~mutual expr;
                  HTML.h3 [] [ txt "AST JSON" ];
                  HTML.pre [ HTML.class_ "bb-source" ] [ txt "%s" j ];
                ]
            in
            let observe_pane =
              let db_prims = Db_cost.db_prims_of_expr expr in
              let db_nodes =
                if db_prims = [] then [ txt "" ]
                else
                  List.concat_map
                    (fun p ->
                      [
                        HTML.span [ HTML.class_ "badge db-cost" ]
                          [
                            txt "%s: %s" p
                              (Db_cost.badge (Db_cost.static_cost p));
                          ];
                        txt " ";
                      ])
                    db_prims
```

```
              in
              let stats_node =
                match
                  Crud.find_row (Session.store session)
                    ~type_name:"function_stat" ~field:"fn" ~value:full
                with
                | Pricklypear_lisp.Ast.VStr fields_json ->
                    (try
                       let j = Yojson.Safe.from_string fields_json in
                       let open Yojson.Safe.Util in
                       let steps =
                         member "steps" j |> to_int_option
                         |> Option.value ~default:0
                       in
                       let prims =
                         member "prim_calls" j |> to_int_option
                         |> Option.value ~default:0
                       in
                       let dbh =
                         member "db_hits" j |> to_int_option
                         |> Option.value ~default:0
                       in
                       HTML.p [ HTML.class_ "meta last-run" ]
                         [
                           txt
                             "last run: %d steps, %d prim calls, %d DB hits"
                             steps prims dbh;
                         ]
                     with _ -> HTML.p [ HTML.class_ "muted" ] [ txt "" ])
                | _ ->
                    HTML.p [ HTML.class_ "muted" ]
                      [ txt "(no measured run yet)" ]
              in
              let explain_text =
                match
                  Analyze.explain_function (Session.store session)
                    ~fn_name:full
                with
                | Ok t -> t
                | Error _ -> ""
              in
              HTML.div [ HTML.class_ "bb-observe" ]
                [
                  HTML.p []
                    (Ast_html.recursion_badges ~fn_name:full expr
                     @ [
                         txt " ";
                         HTML.span [ HTML.class_ "badge complexity" ]
                           [ txt "%s" cx_badge ];
                         txt " ";
                       ]
                     @ db_nodes);
                  stats_node;
                  Ast_html.pp_expr_html ~fn_name:full ~mutual expr;
                  HTML.p []
                    [
                      HTML.button
                        [
                          HTML.type_ "button";
                          HTML.class_ "btn";
                          Hx.post "%s"
                            ("/api/analyze/" ^ Packages.name_encode full);
                          Hx.target "#analyze-result";
                          Hx.swap "innerHTML";
                        ]
                        [ txt "Analyze" ];
                      txt " ";
                      HTML.button
                        [
                          HTML.type_ "button";
                          HTML.class_ "btn secondary";
                          Hx.post "%s"
                            ("/api/analyze/"
```

```
                          ^ Packages.name_encode full
                          ^ "?llm=1");
                      Hx.target "#analyze-result";
                      Hx.swap "innerHTML";
                    ]
                    [ txt "Analyze (LLM)" ];
                txt " ";
                HTML.button
                  [
                    HTML.type_ "button";
                    HTML.class_ "btn secondary";
                    Hx.post "%s"
                      ("/api/explain/" ^ Packages.name_encode full);
                    Hx.target "#explain-result";
                    Hx.swap "innerHTML";
                  ]
                  [ txt "Explain" ];
              ];
            HTML.div [ HTML.id "analyze-result" ] [ txt "" ];
            HTML.details [ HTML.class_ "explain-panel" ]
              [
                HTML.summary [] [ txt "Inside-out breakdown" ];
                HTML.div [ HTML.id "explain-result" ]
                  [
                    HTML.pre
                      [ HTML.class_ "bb-source explain-layer" ]
                      [ txt "%s" explain_text ];
                  ];
              ];
          ]
      in
      let pane_body =
        match tab_q with
        | "edit" -> edit_pane
        | "inspect" -> inspect_pane
        | "observe" -> observe_pane
        | _ -> view_pane
      in
      let code_pane =
        HTML.div [ HTML.class_ "card bb-code" ]
          [
            HTML.div [ HTML.class_ "bb-code-head" ]
              [
                HTML.h2 [] [ txt "%s ? %s" full
                  (match tab_q with
                   | "edit" -> "edit"
                   | "inspect" -> "inspect"
                   | "observe" -> "observability"
                   | _ -> "definition") ];
                HTML.p [ HTML.class_ "bb-code-actions" ]
                  [
                    HTML.span [ HTML.class_ "badge complexity" ]
                      [ txt "%s" cx_badge ];
                  ];
              ];
            tabs;
            (if tab_q = "edit" then HTML.div [] [] else flash_node);
            pane_body;
          ]
      in
      (chrome ~session ~title:full
           [ HTML.div [HTML.class_ "bb-browser"]
               [ HTML.div [HTML.class_ "bb-three"]
                   [ pkg_col; fns_in_pkg; meta_col ]
               ; code_pane
               ] ])
    in
    html_response reqd ~session body;
    ()
| Some "special-forms", None ->
    let rows =
      let all = Pricklypear_lisp.Special_forms.summaries in
      if q = "" then all
```

```
      else List.filter (fun (n, s) -> contains_ci n q || contains_ci s q)
    all
     in
     let items =
       List.map
         (fun (n, summary) ->
           HTML.tr [] [
             HTML.td [] [
               HTML.a [HTML.href "%s" (fn_href "special-forms" n)] [txt
    "%s" n];
             ];
             HTML.td [] [txt "special form"];
             HTML.td [] [txt "%s" summary];
           ])
         rows
     in
     let body =
       chrome ~session ~title:"Package (special-forms)" [
         HTML.p [] [HTML.a [HTML.href "/bindings"] [txt "? all packages"]];
         HTML.p [HTML.class_ "meta"] [
           txt "Evaluator special forms (syntax). Not env bindings, not
    prims. Read-only.";
         ];
         HTML.table [] [
           HTML.thead [] [
             HTML.tr [] [
               HTML.th [] [txt "name"];
               HTML.th [] [txt "kind"];
               HTML.th [] [txt "summary"];
             ];
           ];
           HTML.tbody [] items;
         ];
       ]
     in
     html_response reqd ~session body;
     ()
 | Some "kernel", None ->
     let rows =
       if q = "" then kernel
       else List.filter (fun (n, s) -> contains_ci n q || contains_ci s q)
    kernel
     in
     let items =
       List.map
         (fun (n, summary) ->
           HTML.tr [] [
             HTML.td [] [
               HTML.a [HTML.href "%s" (fn_href "kernel" n)] [txt "%s" n];
             ];
             HTML.td [] [txt "primitive"];
             HTML.td [] [txt "%s" summary];
           ])
         rows
     in
     let body =
       chrome ~session ~title:"Package (kernel)" [
         HTML.p [] [HTML.a [HTML.href "/bindings"] [txt "? all packages"]];
         HTML.p [HTML.class_ "meta"] [
           txt "Built-in OCaml primitives. Read-only. Prefer app logic as
    Lisp.";
         ];
         HTML.table [] [
           HTML.thead [] [
             HTML.tr [] [
               HTML.th [] [txt "name"];
               HTML.th [] [txt "kind"];
               HTML.th [] [txt "summary"];
             ];
           ];
           HTML.tbody [] items;
         ];
       ]
```

```
      in
      html_response reqd ~session body;
      ()
  | Some pkg, None ->
      let pkg_key = if pkg = "(root)" || pkg = "root" then "" else pkg in
      let rows =
        List.filter
          (fun (name, _, _, _) -> Packages.package_of_name name = pkg_key)
          defs_f
      in
      let items =
        List.map
          (fun (name, body, author, depth) ->
            let badge = badge_of body in
            let local = Packages.local_name name in
            HTML.tr [] [
              HTML.td [] [
                HTML.a [HTML.href "%s" (fn_href pkg local)] [txt "%s" name];
              ];
              HTML.td [] [txt "%s" badge];
              HTML.td [] [origin_badge depth author];
              HTML.td [] [used_links name];
            ])
          rows
      in
      let body =
        chrome ~session ~title:("Package " ^ Packages.package_label
    pkg_key) [
          HTML.p [] [HTML.a [HTML.href "/bindings"] [txt "? all packages"]];
          HTML.p [HTML.class_ "muted"] [
            txt "Effective view for branch %s -- one row per name (own or
    inherited)."
              (Session.branch_name session);
          ];
          (match items with
           | [] -> HTML.p [] [txt "(no functions in this package)"]
           | _ ->
               HTML.table [] [
                 HTML.thead [] [
                   HTML.tr [] [
                     HTML.th [] [txt "function"];
                     HTML.th [] [txt "effects"];
                     HTML.th [] [txt "origin"];
                     HTML.th [] [txt "used by"];
                   ];
                 ];
                 HTML.tbody [] items;
               ]);
          HTML.p [] [
            HTML.a
              [ HTML.href "%s"
                  (if pkg_key = "" then "/edit/?local="
                   else
                     "/edit/?package=" ^ Packages.name_encode pkg_key
                     ^ "&local=") ]
              [txt "new in this package?"];
          ];
        ]
      in
      html_response reqd ~session body;
      ()
  | None, _ ->
      let ws_pkgs =
        List.map
          (fun (pkg, rows) ->
            let label = Packages.package_label pkg in
            let key = if pkg = "" then "root" else pkg in
            (label, pkg_href key, List.length rows, "workspace"))
          groups
      in
      let all_pkgs =
        ( "(special-forms)"
        , "/bindings?pkg=special-forms"
```

```
      , List.length Pricklypear_lisp.Special_forms.summaries
      , "language" )
      :: ( "(kernel)"
         , "/bindings?pkg=kernel"
         , List.length kernel
         , "kernel" )
      :: ws_pkgs
   in
   let filtered =
     if q = "" then all_pkgs
     else List.filter (fun (label, _, _, _) -> contains_ci label q) all_pkgs
   in
   let items =
     List.map
       (fun (label, href, n, kind) ->
         HTML.tr [] [
           HTML.td [] [HTML.a [HTML.href "%s" href] [txt "%s" label]];
           HTML.td [] [txt "%s" kind];
           HTML.td [] [txt "%d" n];
         ])
       filtered
   in
   let body =
     chrome ~session
       ~title:("Functions (" ^ Session.branch_name session ^ ")")
       [ HTML.p [HTML.class_ "meta"] [
           txt "Drill: package ? function ? definition. Kernel = OCaml prims (read-only). Workspace = Lisp you can edit.";
         ]
       ; HTML.div [HTML.class_ "card"] [
           HTML.form [HTML.method_ `GET; HTML.action "/bindings"] [
             HTML.p [] [
               HTML.label [HTML.for_ "q"] [txt "search "];
               HTML.input
                 [ HTML.type_ "text"
                 ; HTML.name "q"
                 ; HTML.id "q"
                 ; HTML.value "%s" q
                 ; HTML.placeholder "package or name" ];
               txt " ";
               HTML.button [HTML.type_ "submit"] [txt "Filter"];
             ];
           ];
         ]
       ; HTML.div [HTML.class_ "card"] [
           HTML.h2 [] [txt "New function"];
           HTML.form [HTML.method_ `GET; HTML.action "/edit/"] [
             HTML.p [] [
               HTML.label [] [txt "package "];
               HTML.input
                 [ HTML.type_ "text"
                 ; HTML.name "package"
                 ; HTML.placeholder "todo" ];
               txt " ";
               HTML.label [] [txt "name "];
               HTML.input
                 [ HTML.type_ "text"
                 ; HTML.name "local"
                 ; HTML.placeholder "list" ];
               txt " ";
               HTML.button [HTML.type_ "submit"] [txt "Create / edit"];
             ];
           ];
         ]
       ; HTML.table [] [
           HTML.thead [] [
             HTML.tr [] [
               HTML.th [] [txt "package"];
               HTML.th [] [txt "kind"];
               HTML.th [] [txt "count"];
             ];
           ];
```

```
              HTML.tbody [] items;
            ]
          ; HTML.p [HTML.class_ "muted"] [
              txt "Tip: open (special-forms) for seq/if/lambda, (kernel) for create/+/render-html, ?";
            ]
          ]
      in
      html_response reqd ~session body;
      ()

let route_for_edit_name (session : Session.t) (name : string)
    : (string * string) option =
  let route = "/" ^ name in
  match Session.lookup_route session route with
  | Some handler -> Some (route, handler)
  | None ->
      (match Session.lookup_route session name with
       | Some handler -> Some (name, handler)
       | None -> None)

let missing_editor_card (session : Session.t) (name : string) :
    Pure_html.node =
  let route_opt = route_for_edit_name session name in
  let route_block =
    match route_opt with
    | None -> HTML.div [] []
    | Some (route, handler) ->
        HTML.div [HTML.class_ "card"] [
          HTML.p [] [
            txt "There is a page ";
            HTML.a [HTML.href "%s" route] [txt "%s" route];
            txt " that currently runs function ";
            HTML.span [HTML.class_ "ok"] [txt "%s" handler];
            txt ".";
          ];
          HTML.p [] [
            HTML.a [HTML.href "%s" (Packages.edit_href handler); HTML.class_ "btn"]
              [txt "Edit %s" handler];
            txt " ";
            HTML.a
              [ HTML.href "%s" ("/page/" ^ page_inspect_path route)
              ; HTML.class_ "btn secondary" ]
              [ txt "Inspect page" ];
          ];
        ]
  in
  let create_form =
    HTML.div [HTML.class_ "card"] [
      HTML.p [] [txt "Function \"%s\" does not exist." name];
      HTML.p [HTML.class_ "muted"] [
        txt "Create it here -- the editor loads in place (htmx).";
      ];
      HTML.form
        [ HTML.method_ `POST
        ; HTML.action "/api/edit/create"
        ; HTML.enctype `urlencoded
        ; Hx.post "/api/edit/create"
        ; Hx.target "#editor-root"
        ; Hx.swap "outerHTML" ]
        [ HTML.input [HTML.type_ "hidden"; HTML.name "name"; HTML.value "%s" name]
        ; (match route_opt with
           | Some (route, _) ->
               HTML.p [] [
                 HTML.label [] [
                   HTML.input
                     [ HTML.type_ "checkbox"
                     ; HTML.name "rebind_route"
                     ; HTML.value "1" ];
                   txt " also point page %s at this new function" route;
                 ];
```

```
              ]
          | None -> HTML.div [] [])
      ; HTML.p [] [
          HTML.button [HTML.type_ "submit"] [txt "Create function %s"
    name];
          txt " ";
          HTML.a [HTML.href "/bindings"; HTML.class_ "btn secondary"]
            [txt "Functions list"];
        ];
      ];
    ]
  in
  HTML.div [HTML.id "editor-root"] [route_block; create_form]

let editor_page (session : Session.t) (name : string) ?(msg = "")
    ?(body_src = "") (reqd : Httpun.Reqd.t) : unit =
  let found = Store.lookup_function (Session.store session) name in
  let body =
    match found, body_src with
    | _, src when src <> "" ->
        chrome ~session ~title:("Edit " ^ name)
          [ editor_form_partial ~name ~body_src:src ~msg () ]
    | Some (expr, _, _), _ ->
        let initial = Pricklypear_lisp.Reader.pp_expr expr in
        chrome ~session ~title:("Edit " ^ name)
          [ editor_form_partial ~name ~body_src:initial ~msg () ]
    | None, _ ->
        chrome ~session ~title:("Edit " ^ name)
          [ missing_editor_card session name ]
  in
  html_response reqd ~session body;
  ()

let api_edit_create (session : Session.t) (reqd : Httpun.Reqd.t) : unit =
  let body = read_body reqd in
  let name =
    match form_field body "name" with
    | Some n when String.trim n <> "" -> String.trim n
    | _ -> ""
  in
  if name = "" then begin
    html_response reqd ~session ~status:`Bad_request
      (HTML.div [HTML.class_ "err"] [txt "name required"]);
    ()
  end else
    let rebind =
      match form_field body "rebind_route" with
      | Some ("1" | "on" | "true") -> true
      | _ -> false
    in
    let body_src = stub_body name in
    let res =
      Session.define_binding session ~name ~body_src
        ~author:(Session.username session)
    in
    match res with
    | Error e ->
        html_response reqd ~session ~status:`Bad_request
          (HTML.div [HTML.id "editor-root"; HTML.class_ "err"]
             [txt "create failed: %s" e]);
        ()
    | Ok _ ->
        let () =
          if rebind then
            Session.set_route session ~route:("/" ^ name) ~handler:name ()
          else ()
        in
        let msg =
          if rebind then
            Printf.sprintf "created %s and pointed /%s at it" name name
          else Printf.sprintf "created %s -- edit and save" name
        in
        let node = editor_form_partial ~name ~body_src ~msg () in
```

```
        (* Partial only -- htmx swaps into #editor-root *)
        let s = Pure_html.to_string node in
        let headers =
          Httpun.Headers.of_list
            [ ("content-type", "text/html; charset=utf-8")
            ; ("content-length", string_of_int (String.length s)) ]
        in
        let resp = Httpun.Response.create ~headers `OK in
        safe_respond_with_string reqd resp s;
        ()

let page_inspector (session : Session.t) (path_seg : string)
    (reqd : Httpun.Reqd.t) : unit =
  let route =
    if path_seg = "" then "/"
    else if path_seg.[0] = '/' then path_seg
    else "/" ^ path_seg
  in
  match Session.lookup_route session route with
  | None ->
      html_response reqd ~session ~status:`Not_found
        (chrome ~session ~title:("Page " ^ route) [
           HTML.p [HTML.class_ "err"] [txt "no page registered for %s"
    route];
           HTML.p [] [HTML.a [HTML.href "/pages"] [txt "all pages"]];
         ]);
      ()
  | Some handler ->
      let graph = page_function_graph session ~handler in
      let found = Store.lookup_function (Session.store session) handler in
      let body_block =
        match found with
        | None ->
            HTML.p [HTML.class_ "err"] [txt "handler function missing in
    store"]
        | Some (expr, author, _doc) ->
            HTML.div [] [
              HTML.p [HTML.class_ "muted"] [txt "author: %s" author];
              HTML.pre [] [txt "%s" (Pricklypear_lisp.Reader.pp_expr expr)];
            ]
      in
      let rows =
        List.map
          (fun (n, role) ->
            HTML.tr [] [
              HTML.td [] [txt "%s" n];
              HTML.td [] [txt "%s" role];
              HTML.td [] [
                HTML.a [HTML.href "%s" (Packages.edit_href n)] [txt "edit"];
                txt " ? ";
                HTML.a [HTML.href "%s" (Packages.inspect_href n)] [txt
    "inspect"];
              ];
            ])
          graph
      in
      let body =
        chrome ~session ~title:("Page " ^ route) [
          HTML.div [HTML.class_ "card"] [
            HTML.h2 [] [txt "Routing"];
            HTML.p [] [
              txt "Page URL: ";
              HTML.a [HTML.href "%s" route] [txt "%s" route];
            ];
            HTML.p [] [
              txt "Handler function: ";
              HTML.a [HTML.href "%s" (Packages.edit_href handler)] [txt
    "%s" handler];
            ];
            HTML.p [] [
              HTML.a [HTML.href "%s" route; HTML.class_ "btn"] [txt "Open
    page"];
              txt " ";
```

```
              HTML.a [HTML.href "%s" (Packages.edit_href handler);
    HTML.class_ "btn secondary"]
                [txt "Edit handler"];
            ];
          ];
          HTML.div [HTML.class_ "card"] [
            HTML.h2 [] [txt "Functions this page uses"];
            HTML.p [HTML.class_ "muted"] [
              txt "Static call graph from the handler body (user functions
    only).";
            ];
            HTML.table [] [
              HTML.thead [] [
                HTML.tr [] [
                  HTML.th [] [txt "function"];
                  HTML.th [] [txt "role"];
                  HTML.th [] [txt "actions"];
                ];
              ];
              HTML.tbody [] rows;
            ];
          ];
          HTML.div [HTML.class_ "card"] [
            HTML.h2 [] [txt "Entry body (handler)"];
            body_block;
          ];
        ]
      in
      html_response reqd ~session body;
      ()

let inspect_binding_page (session : Session.t) (name : string)
    (reqd : Httpun.Reqd.t) : unit =
  let found = Store.lookup_function (Session.store session) name in
  let defs = Store.list_function_names (Session.store session) in
  let known_fn =
    Pricklypear_lisp.Effect.known_fn_of_bindings
      (List.map (fun (n, e, _) -> (n, e)) defs)
  in
  let body =
    match found with
    | None ->
        chrome ~session ~title:("Inspect " ^ name) [
          HTML.p [HTML.class_ "err"] [txt "not found"];
        ]
    | Some (expr, author, _doc) ->
        let j =
          try Yojson.Safe.pretty_to_string
                (Pricklypear_lisp.Ast.expr_to_json expr)
          with _ -> "(json unavailable)"
        in
        let badge =
          Pricklypear_lisp.Effect.EffSet.badge
            (Pricklypear_lisp.Effect.infer ~known_fn expr)
        in
        let cx =
          Pricklypear_lisp.Complexity.badge
            (Pricklypear_lisp.Complexity.classify ~fn_name:name expr)
        in
        let snap = Call_graph.snapshot_of_session session in
        let mutual = Call_graph.mutual_of snap name in
        let txn_nodes =
          match Pricklypear_lisp.Effect.txn_lint ~known_fn expr with
          | [] -> []
          | hs ->
              [ HTML.div [HTML.class_ "card"]
                  (HTML.p [HTML.class_ "muted"] [txt "txn lint (soft):"]
                   :: List.map
                        (fun h -> HTML.p [HTML.class_ "err"] [txt "%s" h])
                        hs)
              ]
        in
        chrome ~session ~title:("Inspect binding " ^ name)
```

```
          ([ HTML.div [HTML.class_ "card"] [
               HTML.p [] [txt "author: %s" author];
               HTML.p [] [txt "branch: %s" (Session.branch_name session)];
               HTML.p [] [txt "effects: %s" badge];
               HTML.p []
                 (Ast_html.recursion_badges ~fn_name:name expr
                  @ [ txt " "; HTML.span [HTML.class_ "badge complexity"]
    [txt "%s" cx] ]);
               HTML.h2 [] [txt "pretty"];
               Ast_html.pp_expr_html ~fn_name:name ~mutual expr;
               HTML.h2 [] [txt "AST JSON"];
               HTML.pre [] [txt "%s" j];
               HTML.p [] [
                 HTML.a [HTML.href "%s" (Packages.edit_href name)] [txt
    "edit"];
                 txt " ? ";
                 HTML.a [HTML.href "%s" ("/binding/" ^ Packages.name_encode
    name)]
                   [txt "observability"];
               ];
             ];
           ]
           @ txn_nodes)
  in
  html_response reqd ~session body;
  ()

let inspect_row_page (session : Session.t) (type_name : string) (id :
    string)
    (reqd : Httpun.Reqd.t) : unit =
  (* Canonical row URL is /data/<type>/<id> (image.data-browser). *)
  ignore session;
  redirect_response reqd
    (Printf.sprintf "/data/%s/%s" type_name id);
  ()

let api_eval (session : Session.t) (reqd : Httpun.Reqd.t) : unit =
  let body = read_body reqd in
  let expr_opt =
    try
      match Yojson.Safe.from_string body with
      | `Assoc items ->
          (match List.assoc_opt "expr" items with
           | Some (`String s) -> Some s
           | _ -> None)
      | `String s -> Some s
      | _ -> None
    with _ ->
      (* also accept raw text body *)
      if String.trim body = "" then None else Some body
  in
  match expr_opt with
  | None ->
      json_response reqd ~status:`Bad_request
        (`Assoc [("ok", `Bool false); ("error", `String "expr required")]);
      ()
  | Some expr ->
      (try
      let v = Session.eval_one session expr in
          (* Serialize the Lisp value as NATIVE JSON (not a pp_value
             string). CRUD read prims return fields as a VStr of JSON
             text; pp_value would backslash-escape those quotes and
             json_response would re-encode again => triple-escaped
             output. value_to_json re-parses JSON-lookalike VStrs into
             real JSON objects/arrays so field data round-trips as
             clean, valid JSON (i-fields-as-string-2026-08-12). *)
          json_response reqd
            (`Assoc [("ok", `Bool true); ("value", Prims_pure.value_to_json
    v)]);
          ()
    with exn ->
      (* Shared codec (remote-eval.borg substrate): same wire shape as
         before, factored into pure/prims_pure.ml so the image and a
```

```
         remote proxy rim link the SAME error<->JSON codec. *)
          json_response reqd (Prims_pure.error_to_json exn);
          ())

(* POST /api/prim (borge/remote-eval.borg server-endpoint).

   One prim call per request, dispatched through the session prim table
   (the SAME table /api/eval dispatches through -- the one
   [register_session_prims] + control/CRUD registrations build) so
   identity/author/branch are exactly as an in-process eval. Body:
   {name: str, args: [json?]}. Args deserialize via the shared codec
   (Prims_pure.json_to_value), the result serializes via
   Prims_pure.value_to_json, and failures use the shared
   Prims_pure.error_to_json -- so a remote client observes identical
   values and exceptions to an in-process eval.

   GRANULARITY: per-DB-hit, not per-expression (see placement in the
   spec for the mitigation). Same auth path as /api/eval (basic-auth /
   cookie session). No new privilege -- a prim call is equivalent
   privilege to an eval that performs the prim.

   REFUSE-OR-FORWARD: prims that MUST execute server-side to mean
   anything (route/fork/merge/checkout/auth-admin) are refused with a
   structured {ok:false, kind:"server_only", name, reason} error citing
   locality rather than pretending to proxy them. Classification is
   DERIVED from prim-registry metadata (Primitives.is_server_only -- set
   where the prim was registered), never a hand-maintained list. *)
let api_prim (session : Session.t) (reqd : Httpun.Reqd.t) : unit =
  let body = read_body reqd in
  let name, args =
    try
      match Yojson.Safe.from_string body with
      | `Assoc items ->
          let n =
            match List.assoc_opt "name" items with
            | Some (`String s) -> s
            | _ -> ""
          in
          let a =
            match List.assoc_opt "args" items with
            | Some (`List xs) -> List.map Prims_pure.json_to_value xs
            | _ -> []
          in
          (n, a)
      | _ -> ("", [])
    with _ -> ("", [])
  in
  if name = "" then
    json_response reqd ~status:`Bad_request
      (`Assoc [("ok", `Bool false); ("error", `String "name required")])
  else
    match Primitives.dispatch session.prims name args with
    | Primitives.Prim_refused_server_only (nm, reason) ->
        (* Refuse: server-only prim must run on the server's authority
           (branch state, locks, auth), not through a remote proxy. *)
        json_response reqd
          (`Assoc
             [ ("ok", `Bool false)
             ; ("kind", `String "server_only")
             ; ("name", `String nm)
             ; ("reason", `String reason) ])
    | Primitives.Prim_ok v ->
        json_response reqd
          (`Assoc
             [("ok", `Bool true); ("value", Prims_pure.value_to_json v)])
    | Primitives.Prim_error exn ->
        json_response reqd (Prims_pure.error_to_json exn);
    ()

let save_edit (session : Session.t) (path_name : string)
    (reqd : Httpun.Reqd.t) : unit =
  let body = read_body reqd in
  let name =
```

```
    match form_field body "name" with
    | Some n when String.trim n <> "" -> String.trim n
    | _ -> path_name
  in
  let body_src = Option.value (form_field body "body") ~default:"" in
  let return_to =
    match form_field body "return_to" with
    | Some s when String.trim s <> "" -> String.trim s
    | _ -> ""
  in
  let res =
    Session.define_binding session ~name ~body_src
      ~author:(Session.username session)
  in
  match res with
  | Ok v ->
      let () = Call_graph.record_define_side_effects session ~name in
      let impact = Call_graph.impact_summary_text session name in
      let msg =
        Printf.sprintf "saved %s => %s\n%s" name
          (Pricklypear_lisp.Reader.pp_value v) impact
      in
      if return_to <> "" then begin
        let sep = if String.contains return_to '?' then "&" else "?" in
        redirect_response reqd
          (return_to ^ sep ^ "msg=" ^ url_encode msg)
      end else editor_page session name ~msg ~body_src reqd
  | Error e ->
      let msg = "error: " ^ e in
      if return_to <> "" then begin
        let sep = if String.contains return_to '?' then "&" else "?" in
        redirect_response reqd
          (return_to ^ sep ^ "msg=" ^ url_encode msg)
      end else editor_page session name ~msg ~body_src reqd
```

80 image/lib/http_tree.ml

```
(* Pricklypear Image -- /tree browser face over the one tree
   (one-tree.tree-browser).

   Generic store-tree navigation on the session lens: breadcrumb from
   tree/ancestors, ONE level per render (CHILDREN-BY-LEVEL, never a
   subtree walk for a page), lazy expansion through htmx partials
   (/partials/tree -- one children query per expand), ranked order from
   sibling-order, kind filter, as-of-gen view (?gen=) with head-diff
   highlighting, and a node detail pane (body JSON, created/updated,
   author, rank, out-links/backlinks, edit affordance gated on
   tree/upsert auth rules).

   Complements /data (typed-row tables), /bindings (code), /sys (admin
   store-tree). The existing tree/* (scripture docs) package is
   untouched in v1. Confinement per tree-read-prims: an identity lists
   only its own garden/data prefixes + data.shared + its branch chain's
   store-tree root; private paths are not listable. *)

open Pure_html
open Http_core

module T = Tree_store
module C = Crud

let partial_path = "/partials/tree"

(* ---- confinement (tree-read-prims: identity-granted prefixes) ---- *)

(** True when [session] may LIST [path]. Grammar-scoped: data.shared is
    common; data.u/<note>/<lens> user segments must decode to the
    session identity; b.<branch> must be in the session's branch chain
    (anonymous rides system/main). Intermediate containers that hide a
    user segment (data, data.u, data.note, data.note.u, data.l) are not
    listable -- private paths are not enumerable. *)
let path_visible (session : Session.t) (path : string) : bool =
  if not (T.valid_path path) then false
  else
    let uname = Session.username session in
    let user_lab = T.path_label uname in
    let chain =
      List.map T.uuid_label
        (Store.branch_chain (Session.store session)
           ~branch_id:(Session.branch_id session))
    in
    let dec k segs =
      match List.nth_opt segs k with
      | Some s -> (match T.dec_label s with Some d -> d | None -> "")
      | None -> ""
    in
    let raw k segs =
      match List.nth_opt segs k with Some s -> s | None -> ""
    in
    let segs = String.split_on_char '.' path in
    match segs with
    | [ "data" ] -> false
    | "data" :: "shared" :: _ -> true
    | "data" :: "u" :: _ -> dec 2 segs = uname && dec 2 segs <> ""
    (* garden paths store the user as a plain path_label (garden_root) *)
    | "data" :: "note" :: "u" :: _ ->
        raw 3 segs = user_lab && raw 3 segs <> ""
    | "data" :: "l" :: _lens :: _ -> dec 3 segs = uname && dec 3 segs <> ""
    | "data" :: _ -> false (* dev clutter, not a face *)
    | [ "b" ] -> chain <> []
    | "b" :: bid :: _ -> List.mem bid chain
    | _ -> false

(** Resolve the ?path= argument: empty ? the identity's garden root;
    RELATIVE logical suffix (fn.<name>, pkg.<name>, ?) ? resolved
    through the session's branch lens chain (closest-wins, the
    effective override -- tree-browser smoke 3); anything else taken as
    an absolute path. *)
```

```
let resolve_focus (session : Session.t) (raw : string) :
    string * string (* focus path, lens suffix when given *) =
  let p = String.trim raw in
  if p = "" then (T.garden_root ~user:(Session.username session), "")
  else if
    String.starts_with ~prefix:"b." p || String.starts_with ~prefix:"data."
     p
  then (p, "")
  else
    let chain =
      Store.branch_chain (Session.store session)
        ~branch_id:(Session.branch_id session)
    in
    let suffix = if String.contains p '.' then p else "fn." ^ p in
    match Io_lwt.Db.current_conn (Session.store session) with
    | Some conn -> (
        match T.lens_get_in_chain conn ~suffix chain with
        | Some n -> (n.path, suffix)
        | None -> (p, suffix))
    | None -> (p, suffix)

let gen_of (q : string -> string option) : int64 option =
  match q "gen" with
  | Some g when String.trim g <> "" -> (
      try Some (Int64.of_string (String.trim g)) with _ -> None)
  | _ -> None

let kind_of (q : string -> string option) : string option =
  match q "kind" with
  | Some k when String.trim k <> "" -> Some (String.trim k)
  | _ -> None

(** The htmx lazy-expand URL for a child row: /partials/tree with the
    query carried through (gen + kind filters persist across expands). *)
let expand_url ~(prefix : string) ~(gen : int64 option)
    ~(kind_filter : string option) : string =
  let b = Buffer.create 64 in
  Buffer.add_string b partial_path;
  Buffer.add_string b "?path=";
  Buffer.add_string b prefix;
  (match gen with
   | Some g ->
       Buffer.add_string b "&gen=";
       Buffer.add_string b (Int64.to_string g)
   | None -> ());
  (match kind_filter with
   | Some k ->
       Buffer.add_string b "&kind=";
       Buffer.add_string b k
   | None -> ());
  Buffer.contents b


(* ---- one-level children fragment (ONE children query per expand) ---- *)

let children_fragment (session : Session.t) ~(prefix : string)
    ~(gen : int64 option) ~(kind_filter : string option) : Pure_html.node =
  match Io_lwt.Db.current_conn (Session.store session) with
  | None -> HTML.p [HTML.class_ "muted"] [txt "(no db)"]
  | Some conn ->
      (* head (hash, rank) per child for the as-of diff highlight
         (?gen= vs head) -- rank-only reorders must highlight too *)
      let head_hashes =
        match gen with
        | None -> []
        | Some _ ->
            T.list_children conn ~prefix ()
            |> List.map (fun (n : T.node) ->
                   (n.path, (n.hash, n.rank)))
      in
      let kids =
        match gen with
        | Some g -> T.list_children_as_of conn ~prefix ~gen:g ()
```

```
        | None -> T.list_children conn ~prefix ()
      in
      let kids =
        match kind_filter with
        | Some k -> List.filter (fun (n : T.node) -> n.kind = k) kids
        | None -> kids
      in
      let kids = List.filter (fun (n : T.node) -> path_visible session n.path) kids in
      if kids = [] then
        HTML.ul [HTML.class_ "tree-children"]
          [ HTML.li [HTML.class_ "muted"] [txt "(empty)"] ]
      else
        HTML.ul [HTML.class_ "tree-children"]
          (List.map
             (fun (n : T.node) ->
               let changed =
                 match gen with
                 | None -> false
                 | Some _ -> (
                     match List.assoc_opt n.path head_hashes with
                     | Some (h, r) -> h <> n.hash || r <> n.rank
                     | None -> true (* absent at head = newer than gen *))
               in
               let expand =
                 expand_url ~prefix:n.path ~gen ~kind_filter
               in
               HTML.li
                 [ HTML.class_ "%s"
                     (if changed then "tree-row tree-changed" else "tree-row") ]
                 [
                   HTML.a [HTML.href "/tree?path=%s" n.path]
                     [txt "%s" (T.node_display_title n)];
                   txt " ";
                   HTML.span [HTML.class_ "muted"] [txt "%s" n.kind];
                   (match n.rank with
                    | Some r -> HTML.span [HTML.class_ "muted"] [txt " #%s" r]
                    | None -> txt "");
                   (if n.kind <> "link" then
                      HTML.details
                        [ Hx.get "%s" expand
                        ; Hx.target "find .tree-children"
                        ; Hx.swap "innerHTML"
                        ; Hx.trigger "toggle once" ]
                        [
                          HTML.summary [] [txt "+"];
                          HTML.div [HTML.class_ "tree-children"] [];
                        ]
                    else txt "");
                 ])
             kids)

(* ---- breadcrumb (tree/ancestors, visible steps only) ---- *)

let breadcrumb ~(session : Session.t) ~(focus : string) : Pure_html.node =
  let ancestors = try T.path_ancestors focus with Invalid_argument _ -> [] in
  let links =
    ([ ("tree", "/tree") ]
    @ List.filter_map
        (fun a ->
          if path_visible session a then Some (a, "/tree?path=" ^ a) else None)
        ancestors
    @ [ (T.path_label_of focus, "/tree?path=" ^ focus) ])
  in
  let n = List.length links in
  HTML.p [HTML.class_ "crumbs"]
    (List.mapi
       (fun i (label, href) ->
         let item = HTML.a [HTML.href "%s" href] [txt "%s" label] in
```

```
        if i = n - 1 then item else HTML.span [] [item; txt " / "])
      links)

(* ---- detail pane ---- *)

let detail_pane (session : Session.t) ~(focus : string) : Pure_html.node =
  match Io_lwt.Db.current_conn (Session.store session) with
  | None -> HTML.p [HTML.class_ "muted"] [txt "(no db)"]
  | Some conn -> (
      match T.get_by_path conn ~path:focus with
      | None -> HTML.p [HTML.class_ "err"] [txt "no node at %s" focus]
      | Some n ->
          let body_json = Yojson.Safe.to_string n.body in
          let out_links = T.list_out_links conn ~path:focus ~limit:50 () in
          let backlinks = T.edges_to conn ~needle:focus ~limit:50 () in
          let uname = Session.username session in
          let can_edit =
            uname <> "anonymous"
            && (T.is_hier_kind n.kind || n.kind = "datum")
            && path_visible session focus
          in
          HTML.div [HTML.class_ "tree-detail"]
            [
              HTML.h2 [] [txt "%s" (T.node_display_title n)];
              HTML.p [HTML.class_ "muted"]
                [
                  txt "%s ? author %s ? rank %s" n.kind n.author
                    (match n.rank with Some r -> r | None -> "--");
                ];
              HTML.p [HTML.class_ "muted"]
                [txt "created %s ? updated %s" n.created_at n.updated_at];
              HTML.pre [] [txt "%s" body_json];
              (if out_links = [] then txt ""
               else
                 HTML.div []
                   [
                     HTML.h3 [] [txt "out-links"];
                     HTML.ul []
                       (List.map
                          (fun (e : T.node) ->
                            let to_p =
                              match T.body_string_field e.body "to" with
                              | Some t -> t
                              | None -> ""
                            in
                            let status =
                              match T.get_by_path conn ~path:to_p with
                              | Some _ -> "resolved"
                              | None -> "dangling"
                            in
                            HTML.li []
                              [
                                HTML.a [HTML.href "/tree?path=%s" to_p]
                                  [txt "%s" to_p];
                                txt " (%s)" status;
                              ])
                          out_links);
                   ]);
              (if backlinks = [] then txt ""
               else
                 HTML.div []
                   [
                     HTML.h3 [] [txt "backlinks (edges)"];
                     HTML.ul []
                       (List.map
                          (fun (e : T.node) ->
                            HTML.li []
                              [
                                HTML.a [HTML.href "/tree?path=%s" e.path]
                                  [txt "%s" e.path];
                              ])
                          backlinks);
                   ]);
```

```
285               (if can_edit then
286                  HTML.details []
287                    [
288                      HTML.summary [] [txt "edit body (tree/upsert)"];
289                      HTML.form
290                        [ HTML.method_ `POST; HTML.action "/tree/edit" ]
291                        [
292                          HTML.input
293                            [ HTML.type_ "hidden"
294                            ; HTML.name "path"
295                            ; HTML.value "%s" focus ];
296                          HTML.textarea
297                            [HTML.name "body"; HTML.rows 8; HTML.cols 60]
298                            "%s" body_json;
299                          HTML.br [];
300                          HTML.button [HTML.type_ "submit"] [txt "save"];
301                        ];
302                    ]
303                else if uname = "anonymous" then
304                  HTML.p [HTML.class_ "muted"] [txt "(sign in to edit)"]
305                else txt "");
306             ])
307
308 (* ---- pages ---- *)
309
310 (** GET /tree?path=P [&gen=G] [&kind=K] -- the browser page. *)
311 let tree_page (session : Session.t) (reqd : Httpun.Reqd.t) : unit =
312   let target = (Httpun.Reqd.request reqd).target in
313   let q k = query_param target k in
314   let raw_path = match q "path" with Some p -> p | None -> "" in
315   let gen = gen_of q in
316   let kind_filter = kind_of q in
317   let focus, suffix = resolve_focus session raw_path in
318   if not (path_visible session focus) then
319     html_response reqd ~session
320       (chrome ~session ~title:"tree"
321          [
322            HTML.p [HTML.class_ "err"]
323              [txt "path not listable for this identity: %s" focus];
324            HTML.p []
325              [ HTML.a [HTML.href "/tree"] [txt "back to your garden root"]
    ];
326          ])
327   else
328     let kind_query =
329       match kind_filter with Some k -> "&kind=" ^ k | None -> ""
330     in
331     let node_exists =
332       match Io_lwt.Db.current_conn (Session.store session) with
333       | Some conn -> (match T.get_by_path conn ~path:focus with Some _ ->
    true | None -> false)
334       | None -> false
335     in
336     html_response reqd ~session
337       (chrome ~session ~title:"tree"
338          [
339            breadcrumb ~session ~focus;
340            HTML.p [HTML.class_ "muted"]
341              [
342                match suffix with
343                | "" -> txt "%s" focus
344                | s -> txt "%s (lens: %s, effective)" focus s;
345              ];
346            (if gen = None && not node_exists then
347               HTML.p [HTML.class_ "err"] [txt "no node at this path"]
348             else txt "");
349            HTML.form [HTML.method_ `GET; HTML.action "/tree"]
350              [
351                HTML.input
352                  [ HTML.type_ "hidden"; HTML.name "path"; HTML.value "%s"
    focus ];
353                HTML.input
```

```
                [ HTML.type_ "text"; HTML.name "gen"; HTML.placeholder "gen"
                ; HTML.size "8"
                ; HTML.value "%s"
                    (match gen with
                     | Some g -> Int64.to_string g
                     | None -> "") ];
              HTML.input
                [ HTML.type_ "text"; HTML.name "kind"; HTML.placeholder "kind"
                ; HTML.size "8"
                ; HTML.value "%s"
                    (match kind_filter with Some k -> k | None -> "") ];
              HTML.button [HTML.type_ "submit"] [txt "view as-of"];
            ];
          (match gen with
           | Some g ->
               HTML.p [HTML.class_ "muted"]
                 [
                   txt "as of gen %Ld -- " g;
                   HTML.a
                     [HTML.href "/tree?path=%s%s" focus kind_query]
                     [txt "back to head"];
                 ]
           | None -> txt "");
          HTML.h2 [] [txt "children (one level, ranked)"];
          children_fragment session ~prefix:focus ~gen ~kind_filter;
          HTML.h2 [] [txt "node detail"];
          detail_pane session ~focus;
        ])

(** GET /partials/tree?path=P [&gen=G] [&kind=K] -- one-level children
    fragment for htmx lazy expansion (no document chrome). One children
    query per expand (tree-browser smoke: lazy expand issues exactly one
    children query). *)
let tree_partial (session : Session.t) (reqd : Httpun.Reqd.t) : unit =
  let target = (Httpun.Reqd.request reqd).target in
  let q k = query_param target k in
  let raw_path = match q "path" with Some p -> p | None -> "" in
  let gen = gen_of q in
  let kind_filter = kind_of q in
  let focus, _suffix = resolve_focus session raw_path in
  if not (path_visible session focus) then
    fragment_response reqd "<p class=\"err\">not listable</p>"
  else
    fragment_response reqd
      (Pure_html.to_string
         (children_fragment session ~prefix:focus ~gen ~kind_filter))

(** POST /tree/edit -- body-JSON edit affordance honoring tree/upsert
    auth: anonymous denied; the target must be a hier/datum node in a
    visible prefix owned by the session identity. Code nodes edit
    through /binding, not here. *)
let tree_edit (session : Session.t) (reqd : Httpun.Reqd.t) : unit =
  let body = read_body reqd in
  let form = parse_form_body body in
  let path =
    match List.assoc_opt "path" form with Some p -> String.trim p | None -> ""
  in
  let body_json = match List.assoc_opt "body" form with Some b -> b | None -> "" in
  let uname = Session.username session in
  if uname = "anonymous" then
    html_response reqd ~session ~status:`Forbidden
      (chrome ~session ~title:"tree"
         [HTML.p [HTML.class_ "err"] [txt "sign in to edit"]])
  else if path = "" then
    html_response reqd ~session ~status:`Bad_request
      (chrome ~session ~title:"tree" [HTML.p [HTML.class_ "err"] [txt "no path"]])
  else
    match Io_lwt.Db.current_conn (Session.store session) with
```

```
    | None ->
        html_response reqd ~session ~status:`Internal_server_error
          (chrome ~session ~title:"tree" [HTML.p [] [txt "no db"]])
    | Some conn -> (
        match T.get_by_path conn ~path with
        | None ->
            html_response reqd ~session ~status:`Not_found
              (chrome ~session ~title:"tree"
                 [HTML.p [HTML.class_ "err"] [txt "no node at %s" path]])
        | Some n ->
            let editable_kind = T.is_hier_kind n.kind || n.kind = "datum" in
            if not (editable_kind && path_visible session path) then
              html_response reqd ~session ~status:`Forbidden
                (chrome ~session ~title:"tree"
                   [
                     HTML.p [HTML.class_ "err"]
                       [txt "this node edits elsewhere (code: /binding)"];
                   ])
            else
              (* tree/upsert auth rules: anonymous denied (above); the
                 write itself runs through Crud.tree_upsert so datum
                 writes keep typed-boundary validation. *)
              let type_name =
                match n.type_name with Some t -> t | None -> n.kind
              in
              let result =
                match
                  C.tree_upsert (Session.store session) ~path ~kind:n.kind
                    ~type_name ~body_json ~author:uname
                with
                | VStr s -> s
                | _ -> "unexpected tree/upsert result"
              in
              let ok = not (String.starts_with ~prefix:"upsert_" result) in
              if ok then
                redirect_response reqd ("/tree?path=" ^ path)
              else
                html_response reqd ~session ~status:`Bad_request
                  (chrome ~session ~title:"tree"
                     [
                       HTML.p [HTML.class_ "err"] [txt "%s" result];
                       HTML.p []
                         [
                           HTML.a [HTML.href "/tree?path=%s" path]
                             [txt "back to node"];
                         ];
                     ]))
```

81 image/lib/io.mli

```
(** I/O port for the image (H2/H3).

    Domain code depends on this module type rather than [Lwt]/[pgx_lwt]
    directly. After the Eio-floor flip, APIs are direct-style (plain ['a]).
    [Io_lwt.Db] remains the name domain code calls; it delegates to
    [Store_db] (dual backend). See docs/runtime-effects-transition.md. *)

module type DB = sig
  (** Opaque connection handle. *)
  type conn

  (** Store handle carrying pool conn + dynamic tx binding + branch_id. *)
  type store

  type config = {
    database : string;
    host : string;
    port : int;
    user : string;
    password : string;
  }

  val config_from_env : unit -> config
  val make : cfg:config -> branch_id:string -> unit -> store
  val connect : store -> unit
  val current_conn : store -> conn option
  (** True when inside (with-transaction ...). *)
  val in_transaction : store -> bool
  val branch_id : store -> string
  val set_branch_id : store -> string -> unit

  (** Execute SQL on [conn]. Params are pgx values. Blocks the fiber. *)
  val execute : conn -> ?params:Pgx.row -> string -> Pgx.row list

  (** SERIALIZABLE (default) transaction; [f] sees upgraded conn. *)
  val with_transaction :
    ?isolation:Pgx.Isolation.t ->
    store ->
    (conn option -> 'a) ->
    'a

  (** Session-scoped dynamic tx for (with-transaction ...) Lisp prim. *)
  val with_dynamic_transaction :
    ?isolation:Pgx.Isolation.t ->
    store ->
    (unit -> 'a) ->
    'a
end

module type S = sig
  module Db : DB
end
```

82 image/lib/io_eio.ml

```
(** Eio + pure pgx implementation of the Io.Db port (H3).

    Identity monad over Eio.Net flows. Domain still speaks [Lwt.t] via
    the [Io.DB] interface: each call is scheduled with [Lwt_eio.run_eio]
    when the process is under [Lwt_eio.with_event_loop], or runs
    directly inside [Eio_main.run] for the smoke binary.

    Ambient [ctx] (switch + net) must be installed with [with_ctx] before
    [connect]. *)

module Ctx = struct
  type t = {
    sw : Eio.Switch.t;
    net : [ `Generic ] Eio.Net.ty Eio.Std.r;
  }

  let current : t option ref = ref None

  let get () =
    match !current with
    | Some c -> c
    | None ->
        failwith
          "Io_eio: no ambient Eio context (call Io_eio.with_ctx under
     Eio_main)"

  let with_ctx (c : t) (f : unit -> 'a) : 'a =
    let prev = !current in
    current := Some c;
    Fun.protect ~finally:(fun () -> current := prev) f

  (** True when [with_ctx] is active (Eio_main path). *)
  let installed () = !current <> None
end

let with_ctx = Ctx.with_ctx
let ctx_installed = Ctx.installed

(** Fiber-local: true while this fiber is inside [Lwt_eio.run_eio] / an
    ambient Eio SQL section. A process-global ref is wrong under Httpun_eio
    (many fibers): fiber A in a tx yields at [run_lwt], fiber B would see
    depth>0 and skip [run_eio], racing the shared PG wire. *)
let eio_enter_key : bool Eio.Fiber.key = Eio.Fiber.create_key ()

let in_eio_enter () =
  match Eio.Fiber.get eio_enter_key with Some true -> true | _ -> false

(** Run [f] via [Lwt_eio.run_eio], or directly if this fiber is already in
     one. *)
let on_eio (f : unit -> 'a) : 'a Lwt.t =
  if in_eio_enter () then Lwt.return (f ())
  else
    Lwt_eio.run_eio (fun () ->
        Eio.Fiber.with_binding eio_enter_key true f)

(** Same as [on_eio] but for call sites already on an Eio fiber that need to
    mark the enter-key for nested SQL (e.g. whole-tx critical sections). *)
let with_eio_enter (f : unit -> 'a) : 'a =
  if in_eio_enter () then f ()
  else Eio.Fiber.with_binding eio_enter_key true f

(** Pgx Io backend: direct-style ('a t = 'a) over one Eio two-way flow. *)
module Pgx_io = struct
  type 'a t = 'a

  let return x = x
  let ( >>= ) v f = f v
  let catch f h = try f () with e -> h e

  type flow =
    [ Eio.Flow.two_way_ty | Eio.Resource.close_ty ] Eio.Resource.t
```

```
70
71  type in_channel = flow
72  type out_channel = flow
73
74  type sockaddr =
75    | Unix of string
76    | Inet of string * int
77
78  let open_connection sockaddr : flow * flow =
79    let { Ctx.sw; net } = Ctx.get () in
80    let flow =
81      match sockaddr with
82      | Unix path -> (Eio.Net.connect ~sw net (`Unix path) :> flow)
83      | Inet (host, port) ->
84          let service = string_of_int port in
85          (match Eio.Net.getaddrinfo_stream net host ~service with
86           | [] ->
87               failwith ("Io_eio: getaddrinfo empty for " ^ host ^ ":" ^
     service)
88           | addr :: _ -> (Eio.Net.connect ~sw net addr :> flow))
89    in
90    (flow, flow)
91
92  type ssl_config
93  let upgrade_ssl = `Not_supported
94
95  let output_char (oc : out_channel) c =
96    let b = Bytes.make 1 c in
97    Eio.Flow.copy_string (Bytes.unsafe_to_string b) oc
98
99  let output_string (oc : out_channel) s = Eio.Flow.copy_string s oc
100
101 let output_binary_int (oc : out_channel) n =
102   let chr = Char.chr in
103   output_char oc (chr (n lsr 24));
104   output_char oc (chr ((n lsr 16) land 255));
105   output_char oc (chr ((n lsr 8) land 255));
106   output_char oc (chr (n land 255))
107
108 let flush (_oc : out_channel) = ()
109
110 let input_char (ic : in_channel) =
111   let buf = Cstruct.create 1 in
112   Eio.Flow.read_exact ic buf;
113   Char.chr (Cstruct.get_uint8 buf 0)
114
115 let really_input (ic : in_channel) (b : Bytes.t) pos len =
116   if len < 0 || pos < 0 || pos + len > Bytes.length b then
117     invalid_arg "Io_eio.really_input";
118   (* Must blit into [b]: Cstruct.of_bytes does not write through. *)
119   let cs = Cstruct.create len in
120   Eio.Flow.read_exact ic cs;
121   Cstruct.blit_to_bytes cs 0 b pos len
122
123 let input_binary_int (ic : in_channel) =
124   let b = Bytes.create 4 in
125   really_input ic b 0 4;
126   let code = Char.code in
127   (code (Bytes.get b 0) lsl 24)
128   lor (code (Bytes.get b 1) lsl 16)
129   lor (code (Bytes.get b 2) lsl 8)
130   lor code (Bytes.get b 3)
131
132 let close_in (ic : in_channel) = Eio.Flow.close ic
133
134 let getlogin () =
135   try
136     let uid = Unix.getuid () in
137     (Unix.getpwuid uid).Unix.pw_name
138   with _ -> "pricklypear"
139
140 let debug s = prerr_endline ("[io_eio] " ^ s)
141
```

```
  let protect f ~finally =
    match f () with
    | v ->
        finally ();
        v
    | exception e ->
        finally ();
        raise e

  module Sequencer = struct
    type 'a monad = 'a t
    type 'a t = {
      v : 'a;
      m : Eio.Mutex.t;
    }

    let create v = { v; m = Eio.Mutex.create () }

    (** Serialize [f] on the connection.

        MUST NOT use [Eio.Mutex.use_rw]: that permanently poisons the mutex
        when [f] raises (any SQL/Pgx error). A poisoned sequencer kills
     every
        subsequent DB op (auth included) while the accept loop stays up --
     the
        2026-07-22 login outage. Unlock-on-exception (use_ro style) plus
        [Cancel.protect] keeps the wire section non-cancellable without
        destroying the lock. image.http-robustness invariant 6. *)
    let enqueue t f =
      Eio.Mutex.lock t.m;
      match Eio.Cancel.protect (fun () -> f t.v) with
      | v ->
          Eio.Mutex.unlock t.m;
          v
      | exception ex ->
          Eio.Mutex.unlock t.m;
          raise ex
  end
end

module Pg = Pgx.Make (Pgx_io)

module Db = struct
  type conn = Pg.t

  type config = {
    database : string;
    host : string;
    port : int;
    user : string;
    password : string;
  }

  type store = {
    cfg : config;
    mutable conn : conn option;
    mutable branch_id : string;
    mutable current_tx : conn option;
  }

  let env_or name dft =
    match Sys.getenv_opt name with Some s -> s | None -> dft

  let config_from_env () : config =
    {
      database = env_or "PP_DB_NAME" (env_or "PGDATABASE" "pricklypear");
      host = env_or "PP_DB_HOST" (env_or "PGHOST" "127.0.0.1");
      port = int_of_string (env_or "PP_DB_PORT" (env_or "PGPORT" "5433"));
      user = env_or "PP_DB_USER" (env_or "PGUSER" "pricklypear_image");
      password = env_or "PP_DB_PASS" "";
    }

  let make ~cfg ~branch_id () : store =
```

```
213     { cfg; conn = None; branch_id; current_tx = None }
214
215   let branch_id (s : store) = s.branch_id
216   let set_branch_id (s : store) id = s.branch_id <- id
217   let current_conn (s : store) =
218     match s.current_tx with Some tx -> Some tx | None -> s.conn
219   let in_transaction (s : store) = s.current_tx <> None
220
221   (** Direct (non-Lwt) connect -- call under Eio ctx. *)
222   let connect_sync (s : store) : unit =
223     match s.conn with
224     | Some _ -> ()
225     | None ->
226         let c =
227           Pg.connect
228             ~host:s.cfg.host
229             ~port:s.cfg.port
230             ~user:s.cfg.user
231             ~password:s.cfg.password
232             ~database:s.cfg.database
233             ()
234         in
235         s.conn <- Some c;
236         Printf.eprintf "[io_eio] connected to PG %s:%d/%s as %s\n%!"
237           s.cfg.host s.cfg.port s.cfg.database s.cfg.user
238
239   let execute_sync (c : conn) ?params sql : Pgx.row list =
240     match params with
241     | None -> Pg.execute c sql
242     | Some params -> Pg.execute ~params c sql
243
244   let with_transaction_sync ?isolation (s : store) (f : conn option -> 'a)
        : 'a
245       =
246     match s.conn with
247     | None -> f None
248     | Some c ->
249         Pg.with_transaction ?isolation c (fun tx -> f (Some tx))
250
251   let with_dynamic_transaction_sync ?isolation (s : store) (f : unit -> 'a)
        : 'a
252       =
253     match s.conn with
254     | None -> f ()
255     | Some c ->
256         if s.current_tx <> None then
257           failwith "Io_eio: nested transactions not supported"
258         else
259           Pg.with_transaction ?isolation c (fun tx ->
260               s.current_tx <- Some tx;
261               Fun.protect
262                 ~finally:(fun () -> s.current_tx <- None)
263                 f)
264
265   (** Io.DB-shaped Lwt wrappers (for dual-run under Lwt_eio). *)
266   let connect (s : store) : unit Lwt.t =
267     Lwt_eio.run_eio (fun () -> connect_sync s)
268
269   let execute (c : conn) ?params sql : Pgx.row list Lwt.t =
270     Lwt_eio.run_eio (fun () -> execute_sync c ?params sql)
271
272   let with_transaction ?isolation (s : store) (f : conn option -> 'a Lwt.t)
273       : 'a Lwt.t =
274     Lwt_eio.run_eio (fun () ->
275         with_transaction_sync ?isolation s (fun copt ->
276             Lwt_eio.run_lwt (fun () -> f copt)))
277
278   let with_dynamic_transaction ?isolation (s : store) (f : unit -> 'a Lwt.t)
279       : 'a Lwt.t =
280     Lwt_eio.run_eio (fun () ->
281         with_dynamic_transaction_sync ?isolation s (fun () ->
282             Lwt_eio.run_lwt f))
283
```

```
  (** Health: SELECT 1. Returns row count (expect 1). *)
  let health_query_sync (s : store) : int =
    connect_sync s;
    match s.conn with
    | None -> 0
    | Some c ->
        let rows = execute_sync c "SELECT 1" in
        List.length rows

  let simple_query_sync (s : store) (sql : string) : string list list =
    connect_sync s;
    match s.conn with
    | None -> []
    | Some c ->
        let rows = execute_sync c sql in
        List.map
          (fun row ->
            List.map
              (fun cell ->
                match Pgx.Value.to_string cell with
                | Some x -> x
                | None -> "")
              row)
          rows
end

(** Self-test for image.http-robustness invariant 6: after [enqueue] raises,
    a later [enqueue] must still run (mutex not poisoned). Call under Eio.
    *)
let sequencer_survives_raise () : bool =
  let s = Pgx_io.Sequencer.create 0 in
  let raised =
    try
      ignore
        (Pgx_io.Sequencer.enqueue s (fun _ -> failwith "io_eio sequencer
    probe"));
      false
    with Failure _ -> true
  in
  if not raised then false
  else
    match Pgx_io.Sequencer.enqueue s (fun n -> n + 1) with
    | 1 -> true
    | _ -> false
    | exception _ -> false

(** [PP_IO=lwt|eio] -- default eio. Explicit [PP_IO=lwt] is rollback. *)
let io_mode_from_env () =
  match Sys.getenv_opt "PP_IO" with
  | Some s ->
      (match String.lowercase_ascii (String.trim s) with
       | "lwt" -> `Lwt
       | _ -> `Eio)
  | None -> `Eio
```

83 image/lib/io_lwt.ml

```
(** Db port name kept for call-site stability (H2.2).

    Direct-style: delegates to [Store_db]. Process runs under Eio_main;
    Lwt backend (if PP_IO=lwt) blocks via Lwt_eio inside Store_db. *)

module Db = struct
  type conn = Store_db.conn
  type store = Store_db.t

  type config = Store_db.config = {
    database : string;
    host : string;
    port : int;
    user : string;
    password : string;
  }

  let config_from_env = Store_db.config_from_env
  let make = Store_db.make
  let connect = Store_db.connect
  let current_conn = Store_db.current_conn
  let in_transaction = Store_db.in_transaction
  let branch_id (s : store) = s.Store_db.branch_id
  let set_branch_id (s : store) id = s.Store_db.branch_id <- id

  let execute (conn : conn) ?params sql =
    Store_db.exec conn ?params sql

  let with_transaction ?isolation store f =
    Store_db.with_transaction ?isolation store f

  let with_dynamic_transaction ?isolation store f =
    Store_db.with_dynamic_transaction ?isolation store f
end
```

84 image/lib/jobs.ml

```
(** Durable background jobs -- borge/jobs.borg.

    Jobs are dedicated rows (not data_rows): the runner's SKIP LOCKED claim
    needs real status/run_at columns. The runner is a boot-time Eio fiber
    with its own store+session so it doesn't race HTTP on the shared
    session's branch. Guests see rows via prims; the host owns concurrency.

    v1: job table + runner + core prims (submit-job, job-status, jobs,
    cancel-job). Campaign 4.2 adds step-checkpoints (the (step "name"
    thunk) special form -> __step__ prim) and schedules (schedule/
    unschedule/schedules prims + the runner's due-schedule fire path).
    Tables exist in migration 015. *)

open Pricklypear_lisp.Ast

(* ---- runner identity -- one fresh id per boot ---- *)

let runner_id : string =
  Printf.sprintf "boot-%s" (string_of_int (Unix.getpid ()))

(* ---- field accessors (pgx row helpers, ai_store pattern) ---- *)

let cell_string row i =
  match List.nth_opt row i with
  | Some v -> Pgx.Value.to_string v
  | None -> None

let cell_int row i =
  match cell_string row i with
  | Some s -> (try Some (int_of_string s) with _ -> None)
  | None -> None

let cell_int64 row i =
  match cell_string row i with
  | Some s -> (try Some (Int64.of_string s) with _ -> None)
  | None -> None

(* ---- job row type ---- *)

type job = {
  id : string;
  fn : string;
  args : string;        (* JSON string *)
  branch : string;
  submitted_by : string;
  status : string;
  run_at : int64;
  attempts : int;
  max_attempts : int;
  result : string option;
  error : string;
  runner_id : string;
}

let select_cols =
  "id, fn, args::text, branch, submitted_by, status, run_at, attempts, \
   max_attempts, result::text, error, runner_id"

(* Build the call source the runner evals: (fn <args>).
   [args] is the jsonb column rendered ::text -- i.e. the JSON literal for the
   single argument (a number, a quoted string, an object, ...). It is spliced
   DIRECTLY (never %S-quoted) so a numeric arg stays a number in the call:
   (submit-job "f" 1) must eval (f 1), not (f "1"). %S-quoting the already-
   JSON text turned every non-string arg into a string literal and broke
   numerics/json -- see the runner agent-note in borge/jobs.borg (4.1). *)
let eval_src ~(fn : string) ~(args : string) : string =
  (* Zero-arg convention: args stored as JSON "[]"/"null"/"" fire as a
     0-arity call. Anything else is spliced positionally (must therefore
     be Nopales-readable, e.g. "[1 2]" is NOT -- pass "1 2"-style JSON
```

```
    scalars or a JSON string). *)
  let a = String.trim args in
  if a = "" || a = "[]" || a = "null" then Printf.sprintf "(%s)" fn
  else Printf.sprintf "(%s %s)" fn args

let row_to_job row : job option =
  try
    let id = cell_string row 0
    and fn = cell_string row 1
    and args = cell_string row 2
    and branch = cell_string row 3
    and submitted_by = cell_string row 4
    and status = cell_string row 5
    and run_at = cell_int64 row 6
    and attempts = cell_int row 7
    and max_attempts = cell_int row 8
    and result = cell_string row 9
    and error = cell_string row 10
    and runner_id = cell_string row 11
    in
    match id, fn, branch, submitted_by, status with
    | Some id, Some fn, Some branch, Some submitted_by, Some status ->
        Some {
          id; fn;
          args = (match args with Some s -> s | None -> "{}");
          branch; submitted_by; status;
          run_at = (match run_at with Some r -> r | None -> 0L);
          attempts = (match attempts with Some a -> a | None -> 0);
          max_attempts = (match max_attempts with Some a -> a | None -> 3);
          result; error = (match error with Some e -> e | None -> "");
          runner_id = (match runner_id with Some r -> r | None -> "");
        }
    | _ -> None
  with _ -> None

let pg_conn (store : Store.t) : Io_lwt.Db.conn option =
  Io_lwt.Db.current_conn store

(* ---- value serialization ---- *)

let job_to_value (j : job) : value =
  let fields =
    Printf.sprintf
      "{\"id\":\"%s\",\"fn\":\"%s\",\"args\":%s,\"branch\":\"%s\",\
       \"submitted_by\":\"%s\",\"status\":\"%s\",\"run_at\":%s,\
       \"attempts\":%d,\"max_attempts\":%d,\"result\":%s,\"error\":\"%s\",\
       \"runner_id\":\"%s\"}"
      j.id j.fn j.args j.branch j.submitted_by j.status
      (Int64.to_string j.run_at)
      j.attempts j.max_attempts
      (match j.result with Some r -> r | None -> "null")
      (String.concat "" (List.map (fun c -> if c = '"' then "\\\"" else
    String.make 1 c) (String.to_seq j.error |> List.of_seq)))
      j.runner_id
  in
  VStr fields


(* ---- submit ---- *)

let submit ~(store : Store.t) ~(fn : string) ~(args_json : string)
    ~(branch : string) ~(submitted_by : string) ~(run_at_ms : int64) () :
    string =
  match pg_conn store with
  | None -> "submit-job: no db connection"
  | Some conn ->
    (try
      let sql =
        Printf.sprintf
          "INSERT INTO jobs (fn, args, branch, submitted_by, run_at) \
           VALUES ($1, $2::jsonb, $3, $4, $5) RETURNING id::text"
      in
      let rows =
```

```
        Io_lwt.Db.execute conn
          ~params:
            [ Pgx.Value.of_string fn
            ; Pgx.Value.of_string args_json
            ; Pgx.Value.of_string branch
            ; Pgx.Value.of_string submitted_by
            ; Pgx.Value.of_int64 run_at_ms
            ]
          sql
      in
      (match rows with
       | row :: _ ->
           (match cell_string row 0 with
            | Some id -> id
            | None -> "submit-job: no id returned")
       | [] -> "submit-job: insert returned no row")
    with exn ->
      "submit-job: " ^ Printexc.to_string exn)

(* ---- get (by id, confined to caller unless admin) ---- *)

let get ~(store : Store.t) ~(id : string) ~(caller : string) ~(is_admin :
    bool)
    : value =
  match pg_conn store with
  | None -> VNil
  | Some conn ->
    let sql =
      Printf.sprintf "SELECT %s FROM jobs WHERE id = $1::uuid" select_cols
    in
    let rows =
      Io_lwt.Db.execute conn ~params:[ Pgx.Value.of_string id ] sql
    in
    (match rows with
     | row :: _ ->
         (match row_to_job row with
          | Some j when is_admin || j.submitted_by = caller ->
              job_to_value j
          | _ -> VNil)
     | [] -> VNil)

(* ---- list (caller's jobs, recent first) ---- *)

let list ~(store : Store.t) ~(caller : string) ~(is_admin : bool) ~(limit :
    int) ()
    : value =
  match pg_conn store with
  | None -> VList []
  | Some conn ->
    let lim = limit in
    let where =
      if is_admin then "" else "WHERE submitted_by = $2 "
    in
    let params =
      if is_admin then [ Pgx.Value.of_int lim ]
      else [ Pgx.Value.of_int lim; Pgx.Value.of_string caller ]
    in
    let sql =
      Printf.sprintf "SELECT %s FROM jobs %sORDER BY created_at DESC LIMIT
     $1"
        select_cols where
    in
    let rows = Io_lwt.Db.execute conn ~params sql in
    let vals =
      List.filter_map
        (fun r ->
          match row_to_job r with
          | Some j -> Some (job_to_value j)
          | None -> None)
        rows
    in
    VList vals

```

```
(* ---- cancel (pending ? failed/cancelled) ---- *)

let cancel ~(store : Store.t) ~(id : string) ~(caller : string) ~(is_admin
    : bool)
    : value =
  match pg_conn store with
  | None -> VStr "cancel-job: no db connection"
  | Some conn ->
    let confinement =
      if is_admin then "" else "AND submitted_by = $2 "
    in
    let params =
      if is_admin then [ Pgx.Value.of_string id ]
      else [ Pgx.Value.of_string id; Pgx.Value.of_string caller ]
    in
    let sql =
      Printf.sprintf
        "UPDATE jobs SET status = 'failed', error = 'cancelled' \
         WHERE id = $1::uuid AND status = 'pending' %sRETURNING id::text"
        confinement
    in
    let rows = Io_lwt.Db.execute conn ~params sql in
    (match rows with
     | row :: _ ->
         (match cell_string row 0 with
          | Some id -> VStr id
          | None -> VStr "cancel-job: no id returned")
     | [] -> VStr "cancel-job: not pending or not yours")

(* ---- claim: SKIP LOCKED, mark running ---- *)

let claim_due ~(store : Store.t) : job option =
  match pg_conn store with
  | None -> None
  | Some _ ->
    let now_ms = Int64.of_float (Unix.gettimeofday () *. 1000.0) in
    (* Claim one due pending job, oldest first, with SKIP LOCKED.
       Runs in its own transaction so the claim commits before eval. *)
    Store.with_dynamic_transaction store (fun () ->
      match pg_conn store with
      | None -> None
      | Some conn ->
        let sql =
          Printf.sprintf
            "UPDATE jobs SET status = 'running', runner_id = $1, \
             attempts = attempts + 1 \
             WHERE id = (SELECT id FROM jobs WHERE status = 'pending' \
                         AND run_at <= $2 ORDER BY run_at, id \
                         FOR UPDATE SKIP LOCKED LIMIT 1) \
             RETURNING %s"
            select_cols
        in
        let rows =
          Io_lwt.Db.execute conn
            ~params:
              [ Pgx.Value.of_string runner_id
              ; Pgx.Value.of_int64 now_ms
              ]
            sql
        in
        (match rows with
         | row :: _ -> row_to_job row
         | [] -> None))

(* exponential backoff: 2^attempts seconds, capped at 60s *)
let rec expt_backoff (store : Store.t) (id : string) : int =
  match pg_conn store with
  | None -> 2
  | Some conn ->
    let rows =
      Io_lwt.Db.execute conn ~params:[ Pgx.Value.of_string id ]
        "SELECT attempts FROM jobs WHERE id = $1::uuid"
    in
```

```
    (match rows with
     | row :: _ ->
         (match cell_int row 0 with
          | Some a ->
              let secs = 1 lsl (min a 6) in
              min secs 60
          | None -> 2)
     | [] -> 2)


(* ---- record results ---- *)

let record_done ~(store : Store.t) ~(id : string) ~(result_json : string) : unit =
  match pg_conn store with
  | None -> ()
  | Some conn ->
    let sql =
      "UPDATE jobs SET status = 'done', result = $2::jsonb, error = '' \
       WHERE id = $1::uuid"
    in
    (* Wrap the result in a JSON string value so it's always valid jsonb. *)
    let wrapped = Printf.sprintf "%s" (Yojson.Safe.to_string (`String result_json)) in
    ignore (Io_lwt.Db.execute conn
              ~params:[ Pgx.Value.of_string id; Pgx.Value.of_string wrapped ]
              sql)

let record_failed ~(store : Store.t) ~(id : string) ~(error : string)
    ~(max_attempts : int) : unit =
  match pg_conn store with
  | None -> ()
  | Some conn ->
    (* If attempts >= max_attempts ? dead; else pending with backoff. *)
    let now_ms = Int64.of_float (Unix.gettimeofday () *. 1000.0) in
    let sql =
      "UPDATE jobs SET status = CASE WHEN attempts >= $3 THEN 'dead' \
                                ELSE 'pending' END, \
       error = $2, run_at = CASE WHEN attempts >= $3 THEN run_at \
                                 ELSE $4 END \
       WHERE id = $1::uuid"
    in
    ignore (Io_lwt.Db.execute conn
              ~params:
                [ Pgx.Value.of_string id
                ; Pgx.Value.of_string error
                ; Pgx.Value.of_int max_attempts
                ; Pgx.Value.of_int64
                    (Int64.add now_ms (Int64.of_int (expt_backoff store id)))
                ]
              sql)

(* ---- dead-runner recovery: fail stale 'running' rows from dead runners ---- *)

let reap_dead ~(store : Store.t) : unit =
  match pg_conn store with
  | None -> ()
  | Some conn ->
    (* Any row status=running whose runner_id != our boot runner_id is
       stale (the image crashed mid-eval). Requeue with backoff or dead. *)
    let sql =
      "UPDATE jobs SET status = CASE WHEN attempts >= max_attempts THEN 'dead' \
                                     ELSE 'pending' END, \
       error = 'runner died: ' || runner_id \
       WHERE status = 'running' AND runner_id <> $1"
    in
    ignore (Io_lwt.Db.execute conn ~params:[ Pgx.Value.of_string runner_id ] sql)

```

```
(* The runner loop (eval_job + runner_loop) lives in boot.ml to avoid a
   dependency cycle: Jobs (DB ops) must not depend on Session, which
   depends on Prims_db, which depends on Jobs. boot.ml can depend on
   both Jobs and Session. *)

(* ================================================================ *)
(* STEP CHECKPOINTS -- (with-step "name" thunk) memoized by (job-id, name). *)
(* ================================================================================ *)

(* Fiber-local id of the job currently being evaluated by the runner. The
   runner (boot.ml) binds it around eval; the __step__ prim reads it to
   scope checkpoints per (job-id, name). Outside a job fiber it is None, so
   (with-step "name" thunk) falls back to a plain thunk call. Mirrors the
   pp_user_key pattern in store_db.ml. *)
let current_job_id : string Eio.Fiber.key = Eio.Fiber.create_key ()

let with_current_job_id (job_id : string) (f : unit -> 'a) : 'a =
  Eio.Fiber.with_binding current_job_id job_id f

let current_job_id_opt () : string option = Eio.Fiber.get current_job_id

(** Look up an existing checkpoint value for (job_id, name). Returns the
    stored JSONB value as a string (the JSON literal), or None. *)
let checkpoint_lookup ~(store : Store.t) ~(job_id : string) ~(name : string)
    : string option =
  match pg_conn store with
  | None -> None
  | Some conn ->
    let rows =
      Io_lwt.Db.execute conn
        ~params:
          [ Pgx.Value.of_string job_id; Pgx.Value.of_string name ]
        "SELECT value::text FROM job_steps WHERE job_id = $1::uuid \
         AND step_name = $2"
    in
    (match rows with
     | row :: _ -> cell_string row 0
     | [] -> None)

(** Write a checkpoint row for (job_id, name) storing [value] (a JSON
    literal string). Upsert: a re-run overwrites. Rides the current
    connection (so inside a with-transaction it lands in the SAME tx as
    the thunk's writes -- the exactly-once composition). *)
let checkpoint_write ~(store : Store.t) ~(job_id : string) ~(name : string)
    ~(value : string) : unit =
  match pg_conn store with
  | None -> ()
  | Some conn ->
    ignore
      (Io_lwt.Db.execute conn
         ~params:
           [ Pgx.Value.of_string job_id; Pgx.Value.of_string name;
             Pgx.Value.of_string value ]
         "INSERT INTO job_steps (job_id, step_name, value) \
          VALUES ($1::uuid, $2, $3::jsonb) \
          ON CONFLICT (job_id, step_name) DO UPDATE SET value =
    EXCLUDED.value")

(* ================================================================ *)
(* SCHEDULES -- cron / every-ms rows + next-fire math.                *)
(* ================================================================ *)

(* 5-field cron: minute hour day-of-month month day-of-week. A field is
   either "*", "*/step", or a comma list of values ("a,b,c"). We support
   the common subset: *, */N, and single integers / comma lists. v1 does not
   implement ranges ("1-5") or names (jan, mon). *)

type cron_field =
  | Any                    (* * *)
  | Step of int * int      (* */N -> min 0, step N *)
  | List of int list       (* explicit values *)
```

```
419
420 let parse_cron_field (raw : string) (low : int) (high : int) : cron_field =
421   let raw = String.trim raw in
422   if raw = "*" then Any
423   else if String.length raw >= 3 && String.sub raw 0 2 = "*/" then
424     let n =
425       try int_of_string (String.sub raw 2 (String.length raw - 2)) with _
      -> 0
426     in
427     if n <= 0 then Any else Step (low, n)
428   else
429     let parts = String.split_on_char ',' raw in
430     let vals =
431       List.filter_map
432         (fun p ->
433           let p = String.trim p in
434           if p = "" then None
435           else
436             (try
437                let v = int_of_string p in
438                if v >= low && v <= high then Some v else None
439              with _ -> None))
440         parts
441     in
442     if vals = [] then Any else List vals
443
444 let cron_field_matches (f : cron_field) (v : int) : bool =
445   match f with
446   | Any -> true
447   | Step (minv, n) -> v >= minv && (v - minv) mod n = 0
448   | List vs -> List.mem v vs
449
450 (** Split a 5-field cron string and parse each field with its bounds
451     (minute hour day-of-month month day-of-week). Malformed width ? None. *)
452 let parse_cron_seq (raw : string) (bounds : (int * int) list)
453     : (cron_field list) option =
454   let fields = String.split_on_char ' ' raw in
455   if List.length fields <> 5 then None
456   else
457     (try
458        let parsed =
459          List.map2
460            (fun f (low, high) -> parse_cron_field f low high)
461            fields bounds
462        in
463        Some parsed
464      with _ -> None)
465
466 (** Parse a full 5-field cron string into a tuple of fields
467     (minute hour day-of-month month day-of-week). Bounds follow POSIX cron:
468     minute 0-59, hour 0-23, day-of-month 1-31, month 1-12, day-of-week 0-6
469     (Sunday=0). Malformed input (wrong width, out-of-range-only fields)
470     yields None; the caller treats None as "no match / default to +1min". *)
471 let parse_cron (raw : string)
472     : (cron_field * cron_field * cron_field * cron_field * cron_field)
      option =
473   match parse_cron_seq raw [(0, 59); (0, 23); (1, 31); (1, 12); (0, 6)] with
474   | Some [ m; h; dom; mon; dow ] -> Some (m, h, dom, mon, dow)
475   | _ -> None
476
477 (* Standard cron: if BOTH dom and dow are restricted (non-Any), a match on
478    either is sufficient; if only one is restricted, it must match. *)
479 let is_restricted (f : cron_field) : bool =
480   match f with Any -> false | _ -> true
481
482 (** Does the given UTC calendar time match the parsed cron? [t] is epoch
483     seconds. *)
484 let cron_matches (cron : string) (t : float) : bool =
485   match parse_cron cron with
486   | None -> false
487   | Some (m, h, dom, mon, dow) ->
488       let tm = Unix.gmtime t in
489       let min = tm.Unix.tm_min
```

```
      and hour = tm.Unix.tm_hour
      and mday = tm.Unix.tm_mday
      and month = tm.Unix.tm_mon + 1
      and wday = tm.Unix.tm_wday in
      let m_ok = cron_field_matches m min
      and h_ok = cron_field_matches h hour
      and mon_ok = cron_field_matches mon month
      and dom_ok = cron_field_matches dom mday
      and dow_ok = cron_field_matches dow wday in
      (* If both dom and dow are restricted, either counting is enough. *)
      let dom_dow_ok =
        if is_restricted dom && is_restricted dow then dom_ok || dow_ok
        else dom_ok && dow_ok
      in
      m_ok && h_ok && mon_ok && dom_dow_ok

(** Next fire time (epoch ms) strictly after [from_ms] matching the cron,
    searching minute-by-minute with a ~2-year horizon (avoids infinite
    loops). Returns None if the cron doesn't match within the horizon (e.g.
    an impossible Feb-30). *)
let next_cron_ms (cron : string) (from_ms : int64) : int64 option =
  try
    match parse_cron cron with
    | None -> None
    | Some _ ->
        let now_ms = Int64.of_float (Unix.gettimeofday () *. 1000.0) in
        (* Start from the next whole minute strictly after from (UTC). *)
        let start_min =
          Int64.add (Int64.div from_ms 60_000L) 1L
        in
        let rec search (min64 : int64) : int64 option =
          if min64 > Int64.add (Int64.div now_ms 60_000L)
                   (Int64.of_int (365 * 2 * 24 * 60))
          then None
          else
            let t = Int64.to_float (Int64.mul min64 60_000L) /. 1000.0 in
            if cron_matches cron t then Some (Int64.mul min64 60_000L)
            else search (Int64.add min64 1L)
        in
        search start_min
  with _ -> None

(** Next fire for an every-ms interval: from + every_ms. *)
let next_every_ms (every_ms : int64) (from_ms : int64) : int64 =
  Int64.add from_ms every_ms

(** Is [spec] an every-ms integer (as opposed to a 5-field cron)? *)
let is_every_ms (spec : string) : bool =
  match int_of_string_opt (String.trim spec) with Some _ -> true | None ->
     false

let spec_is_cron (spec : string) : bool = not (is_every_ms spec)

(* --- schedule DB ops --- *)

type schedule = {
  s_name : string;
  s_fn : string;
  s_args : string;
  s_branch : string;
  s_submitted_by : string;
  s_cron : string option;
  s_every_ms : int64 option;
  s_next_run : int64;
  s_enabled : bool;
}

let schedule_to_value (s : schedule) : value =
  let fields =
    Printf.sprintf
      "{\"name\":\"%s\",\"fn\":\"%s\",\"args\":%s,\"branch\":\"%s\",\
       \"submitted_by\":\"%s\",\"cron\":%s,\"every_ms\":%s,\
       \"next_run\":%s,\"enabled\":%s}"
```

```
      s.s_name s.s_fn s.s_args s.s_branch s.s_submitted_by
      (match s.s_cron with Some c -> Printf.sprintf "\"%s\"" c | None ->
     "null")
      (match s.s_every_ms with Some e -> Int64.to_string e | None -> "null")
      (Int64.to_string s.s_next_run)
      (if s.s_enabled then "true" else "false")
  in
  VStr fields

let row_to_schedule (row : Pgx.row) : schedule option =
  try
    let name = cell_string row 0
    and fn = cell_string row 1
    and args = cell_string row 2
    and branch = cell_string row 3
    and submitted_by = cell_string row 4
    and cron = cell_string row 5
    and every_ms = cell_int64 row 6
    and next_run = cell_int64 row 7
    and enabled = cell_string row 8
    in
    match name, fn, branch, submitted_by, next_run with
    | Some name, Some fn, Some branch, Some submitted_by, Some next_run ->
        Some {
          s_name = name; s_fn = fn;
          s_args = (match args with Some a -> a | None -> "{}");
          s_branch = branch; s_submitted_by = submitted_by;
          s_cron = cron;
          s_every_ms =
            (match cron, every_ms with
             | Some _c, _ -> None
             | None, Some e -> Some e
             | None, None -> None);
          s_next_run = next_run;
          s_enabled = (match enabled with Some "t" | Some "true" -> true |
     _ -> false);
        }
    | _ -> None
  with _ -> None

(** Upsert a schedule row. [spec] is a 5-field cron string or an every-ms
    integer. Computes the first next_run from now. *)
let schedule_upsert ~(store : Store.t) ~(name : string) ~(spec : string)
    ~(fn : string) ~(args_json : string) ~(branch : string)
    ~(submitted_by : string) () : value =
  match pg_conn store with
  | None -> VStr "schedule: no db connection"
  | Some conn ->
    let now_ms = Int64.of_float (Unix.gettimeofday () *. 1000.0) in
    let cron = if spec_is_cron spec then Some spec else None in
    let every_ms =
      match int_of_string_opt (String.trim spec) with
      | Some ms -> Some (Int64.of_int ms)
      | None -> None
    in
    if cron = None && every_ms = None then
      VStr "schedule: spec must be a 5-field cron or an every-ms integer"
    else
      let next_run =
        match cron with
        | Some c ->
            (match next_cron_ms c now_ms with
             | Some n -> n
             | None -> Int64.add now_ms 60_000L)
        | None -> next_every_ms (Option.get every_ms) now_ms
      in
      (try
        let sql =
          "INSERT INTO schedules (name, fn, args, branch, submitted_by, \
           cron, every_ms, next_run, enabled) \
           VALUES ($1, $2, $3::jsonb, $4, $5, $6, $7, $8, true) \
           ON CONFLICT (name) DO UPDATE SET fn = EXCLUDED.fn, \
             args = EXCLUDED.args, branch = EXCLUDED.branch, \
```

```
              submitted_by = EXCLUDED.submitted_by, cron = EXCLUDED.cron, \
              every_ms = EXCLUDED.every_ms, next_run = EXCLUDED.next_run, \
              enabled = true \
            RETURNING name::text"
        in
        let rows =
          Io_lwt.Db.execute conn
            ~params:
              [ Pgx.Value.of_string name; Pgx.Value.of_string fn;
                Pgx.Value.of_string args_json; Pgx.Value.of_string branch;
                Pgx.Value.of_string submitted_by;
                (match cron with Some c -> Pgx.Value.of_string c | None ->
    Pgx.Value.null);
                (match every_ms with Some e -> Pgx.Value.of_int64 e | None
    -> Pgx.Value.null);
                Pgx.Value.of_int64 next_run ]
            sql
        in
        (match rows with
         | row :: _ ->
             (match cell_string row 0 with Some n -> VStr n | None -> VStr
    name)
         | [] -> VStr name)
      with exn -> VStr ("schedule: " ^ Printexc.to_string exn))

(** Disable a schedule. *)
let unschedule ~(store : Store.t) ~(name : string) () : value =
  match pg_conn store with
  | None -> VStr "unschedule: no db connection"
  | Some conn ->
    (try
      let rows =
        Io_lwt.Db.execute conn ~params:[ Pgx.Value.of_string name ]
          "UPDATE schedules SET enabled = false WHERE name = $1 RETURNING
    name::text"
      in
      (match rows with
       | row :: _ ->
           (match cell_string row 0 with Some n -> VStr n | None -> VStr "")
       | [] -> VStr "unschedule: no such schedule")
    with exn -> VStr ("unschedule: " ^ Printexc.to_string exn))

(** List schedules (ordered by name). *)
let list_schedules ~(store : Store.t) () : value =
  match pg_conn store with
  | None -> VList []
  | Some conn ->
    (try
      let rows =
        Io_lwt.Db.execute conn
          "SELECT name, fn, args::text, branch, submitted_by, cron,
    every_ms, \
           next_run, enabled FROM schedules ORDER BY name"
      in
      let vals =
        List.filter_map
          (fun r -> match row_to_schedule r with Some s -> Some
    (schedule_to_value s) | None -> None)
          rows
      in
      VList vals
    with _ -> VList [])

(** Claim a single DUE enabled schedule (oldest next_run first), returning
    it (and leaving the row unlocked for the caller to advance). Used by the
    runner's schedule fire path -- SKIP LOCKED so concurrent boot/recovery
    can't double-fire. *)
let claim_due_schedule ~(store : Store.t) : schedule option =
  match pg_conn store with
  | None -> None
  | Some _ ->
    let now_ms = Int64.of_float (Unix.gettimeofday () *. 1000.0) in
    Store.with_dynamic_transaction store (fun () ->
```

```
      match pg_conn store with
      | None -> None
      | Some conn ->
        let sql =
          "SELECT name, fn, args::text, branch, submitted_by, cron, every_ms, \
           next_run, enabled FROM schedules \
           WHERE enabled AND next_run <= $1 \
           ORDER BY next_run, name FOR UPDATE SKIP LOCKED LIMIT 1"
        in
        let rows =
          Io_lwt.Db.execute conn ~params:[ Pgx.Value.of_int64 now_ms ] sql
        in
        (match rows with row :: _ -> row_to_schedule row | [] -> None))

(** Advance a schedule's next_run after it fired (cron or every-ms). *)
let advance_schedule ~(store : Store.t) ~(name : string) ~(fired_at : int64)
    ~(cron : string option) ~(every_ms : int64 option) : unit =
  match pg_conn store with
  | None -> ()
  | Some conn ->
    let now_ms = Int64.of_float (Unix.gettimeofday () *. 1000.0) in
    let next =
      match cron, every_ms with
      | Some c, _ ->
          (match next_cron_ms c now_ms with
           | Some n -> n
           | None -> Int64.add now_ms 60_000L)
      | None, Some e -> next_every_ms e now_ms
      | None, None -> Int64.add now_ms 60_000L
    in
    ignore
      (Io_lwt.Db.execute conn ~params:[ Pgx.Value.of_string name;
       Pgx.Value.of_int64 next ]
          "UPDATE schedules SET next_run = $2 WHERE name = $1")

(* The runner loop (eval_job + runner_loop) lives in boot.ml to avoid a
   dependency cycle: Jobs (DB ops) must not depend on Session, which
   depends on Prims_db, which depends on Jobs. boot.ml can depend on
   both Jobs and Session. *)
```

85 image/lib/lineedit.ml

```
(** Pricklypear Image -- Minimal line editor.

    For the minimal image: read a single line of input from a channel and
    return it. No cursor movement, no history, no reverse-search.

    Per spec [image.repl-experience]: real line editing should use
    ocaml-linenoise (small, Mirage-compatible-ish). The minimal stub is
    acceptable for the boot-checkpoint smoke test. TODO: linenoise. *)

(* Read a line, returning [None] at EOF. Blocks the Eio fiber via Lwt_eio.
     *)
let read_line (ic : Lwt_io.input_channel) : string option =
  Lwt_eio.run_lwt (fun () -> Lwt_io.read_line_opt ic)
```

86 image/lib/llm_client.ml

```
(** OpenAI-compatible LLM client for Pricklypear AI authoring.

    Uses curl for HTTPS (cohttp-lwt-unix not on poohstack). Config via env:
      LLM_API_BASE  default https://api.synthetic.new/openai/v1
      LLM_API_KEY   required for live calls
      LLM_MODEL     default hf:zai-org/GLM-4.7-Flash

    Spec [ai-authoring.tools]: propose functions on the current branch
    with agent provenance. This module only does HTTP; session/http
    layers own store writes. *)

module Llm_codec = Pricklypear_common.Llm_codec

let env_or name dft =
  match Sys.getenv_opt name with
  | Some s when s <> "" -> s
  | _ -> dft

let api_base () = env_or "LLM_API_BASE"
     "https://api.synthetic.new/openai/v1"
let api_key () = env_or "LLM_API_KEY" ""
let model () = env_or "LLM_MODEL" "hf:zai-org/GLM-4.7-Flash"

(** Optional per-request / per-session override (auth.user-llm-keys). *)
type creds = {
  api_key : string;
  api_base : string;
  model : string;
}

let current_creds : creds option ref = ref None

let with_creds (c : creds) (f : unit -> 'a) : 'a =
  let prev = !current_creds in
  current_creds := Some c;
  Fun.protect ~finally:(fun () -> current_creds := prev) f

let active_api_key () =
  match !current_creds with
  | Some c when c.api_key <> "" -> c.api_key
  | _ -> api_key ()

let active_api_base () =
  match !current_creds with
  | Some c when c.api_base <> "" -> c.api_base
  | _ -> api_base ()

let active_model () =
  match !current_creds with
  | Some c when c.model <> "" -> c.model
  | _ -> model ()

type call_result = (string, string) result

(** Progress events for live sinks (SSE / REPL). Not a capability effect. *)
type agent_event =
  | Token of string
  | Done
  | Fail of string

let shell_quote (s : string) : string =
  (* single-quote for POSIX sh, escape embedded single quotes *)
  let buf = Buffer.create (String.length s + 8) in
  Buffer.add_char buf '\'';
  String.iter
    (function
      | '\'' -> Buffer.add_string buf "'\\''"
      | c -> Buffer.add_char buf c)
    s;
  Buffer.add_char buf '\'';
  Buffer.contents buf
```

```
71
72  (* string_field / looks_complete_json / extract_content moved to
73     common/lib/llm_codec.ml -- shared with the standalone agent. *)
74  let extract_content = Llm_codec.extract_content
75
76  let call ?(temperature = 0.2) ?(max_tokens = 4096)
77      ?(json_object = false) ~(system : string) ~(user : string) () :
        call_result =
78    let key = active_api_key () in
79    if key = "" then
80      (Error
81           "no LLM key configured (add one in /settings/keys or set
        LLM_API_KEY)")
82    else
83      let body_json =
84        let base =
85          [
86            ("model", `String (active_model ()));
87            ( "messages",
88              `List
89                [
90                  `Assoc
91                    [ ("role", `String "system"); ("content", `String system)
        ];
92                  `Assoc
93                    [ ("role", `String "user"); ("content", `String user) ];
94                ] );
95            ("temperature", `Float temperature);
96            ("max_tokens", `Int max_tokens);
97          ]
98        in
99        (* GLM-class models often put essays in message.reasoning with
100          content=null unless json_object mode forces a content JSON body. *)
101       let base =
102         if json_object then
103           base
104           @ [ ("response_format", `Assoc [ ("type", `String "json_object")
        ]) ]
105         else base
106       in
107       `Assoc base
108     in
109     let body_str = Yojson.Safe.to_string body_json in
110     let url = active_api_base () ^ "/chat/completions" in
111     (* Write body to a temp file to avoid shell length limits. *)
112     let tmp = Filename.temp_file "pp_llm_" ".json" in
113     let oc = open_out tmp in
114     output_string oc body_str;
115     close_out oc;
116     let cmd =
117       Printf.sprintf
118         "curl -sS --max-time 180 -X POST %s \
119          -H %s -H %s --data-binary @%s 2>&1"
120         (shell_quote url)
121         (shell_quote "Content-Type: application/json")
122         (shell_quote ("Authorization: Bearer " ^ key))
123         (shell_quote tmp)
124     in
125     Printf.eprintf "[llm] POST %s model=%s\n" url (active_model ());
126     (try
127       let stdout =
128           Lwt_eio.run_lwt (fun () -> Lwt_process.pread ~timeout:200.
        (Lwt_process.shell cmd))
129         in
130         (try Sys.remove tmp with _ -> ());
131         if String.trim stdout = "" then
132           (Error "LLM: empty curl response")
133         else if String.length stdout > 12
134                 && (String.sub stdout 0 6 = "curl: "
135                     || String.sub stdout 0 5 = "curl ") then
136           (Error ("curl: " ^ String.trim stdout))
137         else
138           (extract_content stdout)
```

```
    with exn ->
      (try Sys.remove tmp with _ -> ());
        (Error ("LLM transport: " ^ Printexc.to_string exn)))

(** Strip optional markdown fences from model output. *)
let strip_fences (s : string) : string =
  let s = String.trim s in
  if String.length s >= 3 && String.sub s 0 3 = "```" then
    let start =
      match String.index_opt s '\n' with
      | Some i -> i + 1
      | None -> 3
    in
    let rest = String.sub s start (String.length s - start) in
    match String.rindex_opt rest '`' with
    | Some i when i >= 3 && String.sub rest (i - 2) 3 = "```" ->
        String.trim (String.sub rest 0 (i - 2))
    | _ ->
        (match String.rindex_opt rest '\n' with
         | Some i -> String.trim (String.sub rest 0 i)
         | None -> String.trim rest)
  else s

(** Pull the first balanced s-expression from model noise
    (reasoning essays, markdown, numbered lists). *)
let extract_lisp_sexpr (s : string) : (string, string) result =
  let s = strip_fences s in
  let len = String.length s in
  let rec find_open i =
    if i >= len then None
    else if s.[i] = '(' then Some i
    else find_open (i + 1)
  in
  match find_open 0 with
  | None ->
      Error "no Lisp s-expression found in model output (expected a (lambda
     ...) form)"
  | Some start ->
      let rec scan i depth in_str =
        if i >= len then Error "unbalanced parentheses in model output"
        else
          let c = s.[i] in
          if in_str then
            if c = '\\' && i + 1 < len then scan (i + 2) depth true
            else if c = '"' then scan (i + 1) depth false
            else scan (i + 1) depth true
          else
            match c with
            | '"' -> scan (i + 1) depth true
            | '(' -> scan (i + 1) (depth + 1) false
            | ')' ->
                if depth <= 1 then
                  Ok (String.trim (String.sub s start (i - start + 1)))
                else scan (i + 1) (depth - 1) false
            | ';' ->
                (* skip line comment *)
                let rec skip j =
                  if j >= len || s.[j] = '\n' then j else skip (j + 1)
                in
                scan (skip i) depth false
            | _ -> scan (i + 1) depth false
      in
      (match scan (start + 1) 1 false with
       | Error e -> Error e
       | Ok sexpr ->
           (* Prefer a lambda/list form; reject tiny garbage. *)
           if String.length sexpr < 3 then
             Error "extracted s-expression too short"
           else Ok sexpr)

(* extract_stream_delta moved to common/lib/llm_codec.ml. *)
let extract_stream_delta = Llm_codec.extract_stream_delta

```

```
(** Unified agent chat: system=seed, user=prompt.
    When [stream] is true, uses OpenAI stream:true and fires [on_event]
    for each token. Always returns the final concatenated text. *)
let agent_chat
    ?(temperature = 0.3)
    ?(max_tokens = 4096)
    ?(stream = false)
    ?(on_event : agent_event -> unit = fun _ -> ())
    ~(seed : string)
    ~(prompt : string)
    () : call_result =
  let key = active_api_key () in
  if key = "" then begin
    let msg =
      "no LLM key configured (add one in /settings/keys or set LLM_API_KEY)"
    in
    on_event (Fail msg);
    (Error msg)
  end else if not stream then begin
    let res = call ~temperature ~max_tokens ~system:seed ~user:prompt () in
    (match res with
     | Ok text -> on_event (Token text); on_event Done; (Ok text)
     | Error e -> on_event (Fail e); (Error e))
  end else
    let body_json =
      `Assoc
        [ ("model", `String (active_model ()))
        ; ("messages",
           `List
             [ `Assoc [("role", `String "system"); ("content", `String seed)]
             ; `Assoc [("role", `String "user"); ("content", `String prompt)]
             ])
        ; ("temperature", `Float temperature)
        ; ("max_tokens", `Int max_tokens)
        ; ("stream", `Bool true)
        ]
    in
    let body_str = Yojson.Safe.to_string body_json in
    let url = active_api_base () ^ "/chat/completions" in
    let tmp = Filename.temp_file "pp_llm_stream_" ".json" in
    let oc = open_out tmp in
    output_string oc body_str;
    close_out oc;
    let cmd =
      Printf.sprintf
        "curl -sS -N --max-time 180 -X POST %s \
         -H %s -H %s --data-binary @%s 2>&1"
        (shell_quote url)
        (shell_quote "Content-Type: application/json")
        (shell_quote ("Authorization: Bearer " ^ key))
        (shell_quote tmp)
    in
    Printf.eprintf "[llm] STREAM %s model=%s\n" url (active_model ());
    let buf = Buffer.create 4096 in
    (try
      let proc = Lwt_process.open_process_in (Lwt_process.shell cmd) in
        let rec loop () =
          let line_opt =
            Lwt_eio.run_lwt (fun () -> Lwt_io.read_line_opt proc#stdout)
          in
          match line_opt with
          | None ->
              let st = Lwt_eio.run_lwt (fun () -> proc#close) in
              (try Sys.remove tmp with _ -> ());
              ignore st;
              let text = Buffer.contents buf in
              if String.trim text = "" then begin
                on_event (Fail "LLM stream: empty content");
                (Error "LLM stream: empty content")
              end else begin
                on_event Done;
```

```
                (Ok text)
              end
          | Some line ->
              let line = String.trim line in
              if line = "" then loop ()
              else if String.length line >= 6
                      && String.sub line 0 6 = "data: " then
                let data = String.sub line 6 (String.length line - 6) |>
    String.trim in
                if data = "[DONE]" then loop ()
                else begin
                  (match extract_stream_delta data with
                   | Some tok ->
                       Buffer.add_string buf tok;
                       on_event (Token tok)
                   | None -> ());
                  loop ()
                end
              else if String.length line >= 5
                      && (String.sub line 0 5 = "curl "
                          || String.sub line 0 5 = "curl:") then begin
                (try Sys.remove tmp with _ -> ());
                on_event (Fail line);
                let _ = proc#close in
                (Error line)
              end else
                loop ()
        in
        loop ()
    with exn ->
      (try Sys.remove tmp with _ -> ());
        let e = "LLM stream transport: " ^ Printexc.to_string exn in
        on_event (Fail e);
        (Error e))


(** ---- Multi-turn chat + OpenAI tools (agent loop) ---- *)

(* [tool_call] and [tool_def] live in common/lib/llm_codec.ml now
   (shared with the standalone agent). Aliased here so existing
   [Llm_client.tool_call] / [Llm_client.tool_def] call sites in
   ai_store.ml and agent_tools.ml keep working; the record shapes are
   identical. *)
type tool_call = Llm_codec.tool_call
type tool_def = Llm_codec.tool_def
let tool_def_to_json = Llm_codec.tool_def_to_json
let parse_tool_calls_from_msg = Llm_codec.parse_tool_calls_from_msg
let parse_xml_tool_calls = Llm_codec.parse_xml_tool_calls

type role = System | User | Assistant | Tool

type chat_message =
  | Msg of { role : role; content : string }
  | Assistant_tools of { content : string option; tool_calls : tool_call
     list }
  | Tool_result of { tool_call_id : string; content : string }

type turn_result =
  | Final of string
  | Need_tools of { content : string option; tool_calls : tool_call list }
  | Turn_error of string

let role_to_string = function
  | System -> "system"
  | User -> "user"
  | Assistant -> "assistant"
  | Tool -> "tool"

let message_to_json = function
  | Msg { role; content } ->
      `Assoc
        [ ("role", `String (role_to_string role))
        ; ("content", `String content)
```

```
        ]
  | Assistant_tools { content; tool_calls } ->
      let tcs =
        List.map
          (fun (tc : tool_call) ->
            `Assoc
              [ ("id", `String tc.id)
              ; ("type", `String "function")
              ; ("function",
                 `Assoc
                   [ ("name", `String tc.name)
                   ; ("arguments", `String tc.arguments)
                   ])
              ])
          tool_calls
      in
      let base =
        [ ("role", `String "assistant")
        ; ("tool_calls", `List tcs)
        ]
      in
      let base =
        match content with
        | Some c when c <> "" -> ("content", `String c) :: base
        | _ -> ("content", `Null) :: base
      in
      `Assoc base
  | Tool_result { tool_call_id; content } ->
      `Assoc
        [ ("role", `String "tool")
        ; ("tool_call_id", `String tool_call_id)
        ; ("content", `String content)
        ]

(* [tool_def_to_json] / [parse_xml_tool_calls] / [parse_tool_calls] /
   the XML fallback + reasoning_content handling all moved to
   common/lib/llm_codec.ml. This adapter projects the codec's
   [turn_result] (which carries [raw]) onto the image's [turn_result]
   (typed history: we keep only [content] / [tool_calls], drop [raw]). *)
let parse_turn_response (body : string) : turn_result =
  match Llm_codec.parse_turn_response body with
  | Llm_codec.Final { content; _ } -> Final content
  | Llm_codec.Need_tools { content; tool_calls; _ } ->
      Need_tools { content; tool_calls }
  | Llm_codec.Turn_error e -> Turn_error e

(** One non-streaming chat completions call over a full message list. *)

let chat_turn
    ?(temperature = 0.2)
    ?(max_tokens = 4096)
    ?(tools : tool_def list = [])
    ~(messages : chat_message list)
    () : turn_result =
  let key = active_api_key () in
  if key = "" then
    (Turn_error
         "no LLM key configured (add one in /settings/keys or set
    LLM_API_KEY)")
  else
    let msgs = `List (List.map message_to_json messages) in
    let fields =
      [ ("model", `String (active_model ()))
      ; ("messages", msgs)
      ; ("temperature", `Float temperature)
      ; ("max_tokens", `Int max_tokens)
      ]
    in
    let fields =
      if tools = [] then fields
      else
        ("tools", `List (List.map tool_def_to_json tools))
        :: ("tool_choice", `String "auto")
```

```
425         :: fields
426     in
427     let body_str = Yojson.Safe.to_string (`Assoc fields) in
428     let url = active_api_base () ^ "/chat/completions" in
429     let tmp = Filename.temp_file "pp_llm_turn_" ".json" in
430     let oc = open_out tmp in
431     output_string oc body_str;
432     close_out oc;
433     let cmd =
434       Printf.sprintf
435         "curl -sS --max-time 180 -X POST %s \
436          -H %s -H %s --data-binary @%s 2>&1"
437         (shell_quote url)
438         (shell_quote "Content-Type: application/json")
439         (shell_quote ("Authorization: Bearer " ^ key))
440         (shell_quote tmp)
441     in
442     Printf.eprintf "[llm] TURN %s model=%s tools=%d msgs=%d\n%!"
443       url (active_model ()) (List.length tools) (List.length messages);
444     (try
445       let stdout =
446           Lwt_eio.run_lwt (fun () -> Lwt_process.pread ~timeout:200.
     (Lwt_process.shell cmd))
447         in
448         (try Sys.remove tmp with _ -> ());
449         if String.trim stdout = "" then
450           (Turn_error "LLM: empty curl response")
451         else if String.length stdout > 6
452                 && String.sub stdout 0 5 = "curl " then
453           (Turn_error ("curl: " ^ String.trim stdout))
454         else
455           (parse_turn_response stdout)
456     with exn ->
457       (try Sys.remove tmp with _ -> ());
458         (Turn_error ("LLM transport: " ^ Printexc.to_string exn)))
459
460 (** Ask the model to write one Lisp function body (s-expr).
461     Returns source text for (define name <body>). *)
462 let propose_function_source
463     ~(name : string)
464     ~(spec : string)
465     ?(context = "")
466     () : call_result =
467   let system =
468     "You are a Pricklypear Lisp author. Respond ONLY with a single Lisp \
469      s-expression that is the BODY of a function (typically a lambda). \
470      No markdown fences, no explanation, no (define ...) wrapper. \
471      Available prims include: + - * = < > <= >= not list car cdr cons \
472      null? pair? count string-append str type-of whoami list-bindings \
473      dict-get dict-set dict-keys get create update delete rows deftype \
474      fork merge route checkout render-html. Use quote for symbols/data. \
475      Prefer small pure functions unless the task needs DB/IO prims."
476   in
477   let user =
478     Printf.sprintf
479       "Function name: %s\nRequirement:\n%s\n%s\nWrite only the body
     s-expression."
480       name spec
481       (if context = "" then "" else "Context:\n" ^ context ^ "\n")
482   in
483   let res = call ~system ~user () in
484   match res with
485   | Error e -> (Error e)
486   | Ok text ->
487       let cleaned = strip_fences text in
488       (Ok cleaned)
489
490 (** Surgical AST author: rewrite ONE component (coding tool). *)
491 let rewrite_component_source
492     ~(mode : string)  (* "replace" | "wrap" *)
493     ~(name : string)
494     ~(concept : string)
495     ~(current_pretty : string)
```

```
    () : call_result =
  let system =
    "You are editing a Pricklypear Lisp AST as a coding tool. " ^
    "Return ONLY one replacement s-expression (typically a lambda). " ^
    "No markdown fences, no commentary, no (define ...) wrapper. " ^
    "You write ONE component of a larger program tree. " ^
    "Available prims: + - * = < > <= >= not list car cdr cons null? pair? "
    ^
    "count string-append str type-of dict-get dict-set dict-keys " ^
    "get create update delete rows deftype render-html whoami. " ^
    "Prefer pure functions when possible."
  in
  let user =
    if mode = "wrap" then
      Printf.sprintf
        "Mode: WRAP (llm-outer)\nFunction name: %s\n\
         User concept: %s\n\
         Current function (pretty):\n%s\n\
         Write a new lambda that wraps this behavior (call-through or \
         carefully inline). Preserve the essential behavior and add \
         ONLY the requested responsibility. Return one lambda only."
        name concept current_pretty
    else
      Printf.sprintf
        "Mode: REPLACE body (llm-inner)\nFunction name: %s\n\
         User concept: %s\n\
         Current function (pretty):\n%s\n\
         Write a replacement function body (usually a lambda) that \
         satisfies the concept. Return one s-expression only."
        name concept current_pretty
  in
  let res = call ~system ~user () in
  match res with
  | Error e -> (Error e)
  | Ok text -> (Ok (strip_fences text))

(** Seed: smallest clean core for growth. *)
let seed_core_source ~(requirement : string) () : call_result =
  let system =
    "You write the SMALLEST and CLEANEST Pricklypear Lisp nucleus function.
    " ^
    "Return ONLY one lambda s-expression. No markdown, no define wrapper. "
    ^
    "Prefer pure computation. No wrapping, no IO unless absolutely
    required."
  in
  let user =
    Printf.sprintf
      "Requirement:\n%s\nWrite the smallest pure-ish lambda that captures
    the essential core."
      requirement
  in
  let res = call ~system ~user () in
  match res with
  | Error e -> (Error e)
  | Ok text -> (Ok (strip_fences text))

(** Judge growth: returns Ok "done" or Ok "next: <responsibility>" *)
let growth_judge
    ~(requirement : string)
    ~(current_pretty : string)
    ~(step : int)
    () : call_result =
  let system =
    "You judge whether a growing Lisp function set satisfies a requirement.
    " ^
    "Respond with EXACTLY one line: either DONE or NEXT: <single short
    responsibility>. " ^
    "No markdown. NEXT must be one outer wrap responsibility only."
  in
  let user =
    Printf.sprintf
      "Requirement:\n%s\nStep: %d\nCurrent tip function:\n%s\nJudge."
```

```
      requirement step current_pretty
  in
  let res = call ~system ~user ~temperature:0.1 () in
  match res with
  | Error e -> (Error e)
  | Ok text -> (Ok (String.trim (strip_fences text)))
```

87 image/lib/loader.ml

```
(** Pure library loader: Lisp source text ? define / route / deftype.

    Same source files live in git (libs/<name>/lib.pp) and load into the
    live branch. Evaluates AST forms directly (no pp_expr round-trip). *)
open Pricklypear_lisp.Ast

let libs_dir () : string =
  match Sys.getenv_opt "PP_LIBS_DIR" with
  | Some d when d <> "" -> d
  | _ ->
      if Sys.file_exists "libs" then "libs"
      else if Sys.file_exists "../libs" then "../libs"
      else "libs"

let library_path (name : string) : string =
  let root = libs_dir () in
  let a = Filename.concat root (Filename.concat name "lib.pp") in
  let b = Filename.concat root (name ^ ".pp") in
  if Sys.file_exists a then a
  else if Sys.file_exists b then b
  else a

let read_file (path : string) : (string, string) result =
  try
    let ic = open_in_bin path in
    let len = in_channel_length ic in
    let s = really_input_string ic len in
    close_in ic;
    Ok s
  with exn -> Error (path ^ ": " ^ Printexc.to_string exn)

let strip_line_comments (s : string) : string =
  let buf = Buffer.create (String.length s) in
  let len = String.length s in
  let rec loop i in_str =
    if i >= len then Buffer.contents buf
    else
      let c = s.[i] in
      if in_str then begin
        Buffer.add_char buf c;
        if c = '\\' && i + 1 < len then begin
          Buffer.add_char buf s.[i + 1];
          loop (i + 2) true
        end else if c = '"' then loop (i + 1) false
        else loop (i + 1) true
      end else if c = '"' then begin
        Buffer.add_char buf c;
        loop (i + 1) true
      end else if c = ';' then
        let rec skip j =
          if j >= len || s.[j] = '\n' then j else skip (j + 1)
        in
        let j = skip i in
        if j < len then Buffer.add_char buf '\n';
        loop (if j < len then j + 1 else j) false
      else begin
        Buffer.add_char buf c;
        loop (i + 1) false
      end
  in
  loop 0 false

let load_string
    ~(eval_form : expr -> value)
    (src : string)
    : (string, string) result =
  try
    let src = strip_line_comments src in
    let forms = Pricklypear_lisp.Reader.read_all src in
    let rec go i = function
      | [] ->
```

```
          (Ok (Printf.sprintf "loaded %d form(s)" i))
      | e :: rest ->
          (try
      let _ = eval_form e in
              go (i + 1) rest
    with exn ->
      let msg =
                match exn with
                | Interp_error err ->
                    (match err with
                     | User_error (m, _) -> m
                     | Parse_error (m, l, c) ->
                         Printf.sprintf "parse: %s (%d:%d)" m l c
                     | Unbound_var s -> "unbound: " ^ s
                     | _ -> Printexc.to_string exn)
                | _ -> Printexc.to_string exn
              in
              let snippet =
                try String.sub (Pricklypear_lisp.Reader.pp_expr e) 0
                      (min 80 (String.length
    (Pricklypear_lisp.Reader.pp_expr e)))
                with _ -> "?"
              in
              (Error
                   (Printf.sprintf "form %d failed: %s\n  in: %s" (i + 1)
    msg
                      snippet)))
    in
    go 0 forms
  with
  | Pricklypear_lisp.Reader.Reader_error (m, l, c) ->
      (Error (Printf.sprintf "reader: %s (%d:%d)" m l c))
  | exn -> (Error (Printexc.to_string exn))

let load_library
    ~(eval_form : expr -> value)
    (name : string)
    : (string, string) result =
  let path = library_path name in
  match read_file path with
  | Error e -> (Error e)
  | Ok src ->
      let r = load_string ~eval_form src in
      (match r with
       | Ok msg -> (Ok (Printf.sprintf "%s from %s" msg path))
       | Error e -> (Error e))
```

88 image/lib/media.ml

```
(** Pricklypear Image -- binary media blob store (storage.media).

    Table: media_blobs (migration 007). Shared across branches like
    data_rows; ownership is [author]. Bytes live in bytea; meta path
    never returns the payload over Lisp eval. *)

type meta = {
  id : string;
  author : string;
  content_type : string;
  filename : string option;
  byte_size : int;
  sha256 : string;
  meta_json : string;
  created_at : string;
}

type put_result = {
  id : string;
  content_type : string;
  filename : string option;
  byte_size : int;
  sha256 : string;
}

let max_http_bytes = 5 * 1024 * 1024
let max_eval_put_bytes = 256 * 1024

let allowed_content_types =
  [ "image/png"
  ; "image/jpeg"
  ; "image/gif"
  ; "image/webp"
  ; "image/svg+xml"
  ; "application/pdf"
  ]

let normalize_content_type (ct : string) : string =
  let ct = String.trim (String.lowercase_ascii ct) in
  match String.index_opt ct ';' with
  | None -> ct
  | Some i -> String.trim (String.sub ct 0 i)

let content_type_allowed (ct : string) : bool =
  List.mem (normalize_content_type ct) allowed_content_types

let sanitize_filename (name : string option) : string option =
  match name with
  | None -> None
  | Some raw ->
      let s = String.trim raw in
      if s = "" then None
      else
        let base =
          match String.rindex_opt s '/' with
          | Some i -> String.sub s (i + 1) (String.length s - i - 1)
          | None ->
              (match String.rindex_opt s '\\' with
               | Some i -> String.sub s (i + 1) (String.length s - i - 1)
               | None -> s)
        in
        let base = String.trim base in
        if base = "" || base = "." || base = ".." then None
        else
          let capped =
            if String.length base > 200 then String.sub base 0 200 else base
          in
          Some capped

let sha256_hex (bytes : string) : string = Auth_password.sha256_hex bytes

```

```
let b64_decode_opt (s : string) : string option =
  match Base64.decode ~pad:true s with
  | Ok raw -> Some raw
  | Error _ ->
      (match Base64.decode ~pad:false s with
       | Ok raw -> Some raw
       | Error _ -> None)

let url_of_id (id : string) : string = "/media/" ^ id

let meta_to_yojson (m : meta) : Yojson.Safe.t =
  `Assoc
    [ ("id", `String m.id)
    ; ("author", `String m.author)
    ; ("content_type", `String m.content_type)
    ; ( "filename"
      , match m.filename with Some f -> `String f | None -> `Null )
    ; ("byte_size", `Int m.byte_size)
    ; ("sha256", `String m.sha256)
    ; ("url", `String (url_of_id m.id))
    ; ("created_at", `String m.created_at)
    ; ( "meta"
      , try Yojson.Safe.from_string m.meta_json with _ -> `Assoc [] )
    ]

let meta_to_json_string (m : meta) : string =
  Yojson.Safe.to_string (meta_to_yojson m)

let put_result_to_yojson (r : put_result) : Yojson.Safe.t =
  `Assoc
    [ ("ok", `Bool true)
    ; ("id", `String r.id)
    ; ("content_type", `String r.content_type)
    ; ( "filename"
      , match r.filename with Some f -> `String f | None -> `Null )
    ; ("byte_size", `Int r.byte_size)
    ; ("sha256", `String r.sha256)
    ; ("url", `String (url_of_id r.id))
    ]

let cell_string row i =
  match List.nth_opt row i with
  | Some c -> Pgx.Value.to_string c
  | None -> None

let cell_int row i =
  match cell_string row i with
  | Some s -> (try Some (int_of_string s) with _ -> None)
  | None -> None

let row_to_meta (row : Pgx.row) : meta option =
  match
    ( cell_string row 0
    , cell_string row 1
    , cell_string row 2
    , cell_string row 3
    , cell_int row 4
    , cell_string row 5
    , cell_string row 6
    , cell_string row 7 )
  with
  | ( Some id
    , Some author
    , Some content_type
    , filename_opt
    , Some byte_size
    , Some sha256
    , Some meta_json
    , Some created_at ) ->
      Some
        { id
        ; author
        ; content_type
```

```
        ; filename = filename_opt
        ; byte_size
        ; sha256
        ; meta_json
        ; created_at
        }
  | _ -> None

let put
    (store : Store.t)
    ~(author : string)
    ~(content_type : string)
    ~(data : string)
    ?(filename : string option = None)
    ?(meta_json : string = "{}")
    ?(max_bytes : int = max_http_bytes)
    () : (put_result, string) result =
  let ct = normalize_content_type content_type in
  if not (content_type_allowed ct) then
    (Error ("disallowed content_type: " ^ ct))
  else
    let byte_size = String.length data in
    if byte_size <= 0 then (Error "empty body")
    else if byte_size > max_bytes then
      (Error
           (Printf.sprintf "payload too large (%d > %d)" byte_size
    max_bytes))
    else
      match Io_lwt.Db.current_conn store with
      | None -> (Error "no db connection")
      | Some conn ->
          let fname = sanitize_filename filename in
          let digest = sha256_hex data in
          let meta =
            let m = String.trim meta_json in
            if m = "" then "{}"
            else
              try
                ignore (Yojson.Safe.from_string m);
                m
              with _ -> "{}"
          in
          let sql =
            "INSERT INTO media_blobs \
             (author, content_type, filename, byte_size, sha256, data,
    meta) \
             VALUES ($1, $2, $3, $4, $5, $6, $7::jsonb) \
             RETURNING id::text"
          in
          (try
      let rows =
                Io_lwt.Db.execute conn
                  ~params:
                    [ Pgx.Value.of_string author
                    ; Pgx.Value.of_string ct
                    ; (match fname with
                       | Some f -> Pgx.Value.of_string f
                       | None -> Pgx.Value.of_string "")
                    ; Pgx.Value.of_int byte_size
                    ; Pgx.Value.of_string digest
                    ; Pgx.Value.of_binary data
                    ; Pgx.Value.of_string meta
                    ]
                  sql
              in
              match rows with
              | row :: _ ->
                  (match cell_string row 0 with
                   | Some id ->
                       (* Normalize empty filename column to None. *)
                       let filename =
                         match fname with
                         | Some f when f <> "" -> Some f
```

```
                        | _ -> None
                      in
                      (Ok
                           { id
                           ; content_type = ct
                           ; filename
                           ; byte_size
                           ; sha256 = digest
                           })
                  | None -> (Error "insert returned no id"))
            | [] -> (Error "insert returned no row")
    with exn ->
      (Error ("store failed: " ^ Printexc.to_string exn)))

(** Visibility confinement (BORGE-1785695911): a blob readable by [actor]
    iff [actor] authored it or [is_admin]. [None] performs no confinement
    (kernel callers that resolved identity upstream). Applied at the SQL
    layer so list/get/delete agree on one predicate. *)
let visible_sql ~(is_admin : bool) ~(actor_param : int) : string =
  if is_admin then "TRUE"
  else Printf.sprintf "author = $%d" actor_param

let get_meta (store : Store.t) ~(id : string)
  ?(visible_to : (string * bool) option) () : meta option =
  match Io_lwt.Db.current_conn store with
  | None -> None
  | Some conn ->
      let sql, params =
        match visible_to with
        | Some (actor, is_admin) ->
            ( "SELECT id::text, author, content_type, \
               NULLIF(filename, ''), byte_size::text, sha256, meta::text, \
               created_at::text \
               FROM media_blobs WHERE id = $1::uuid AND "
              ^ visible_sql ~is_admin ~actor_param:2
            , [ Pgx.Value.of_string id; Pgx.Value.of_string actor ] )
        | None ->
            ( "SELECT id::text, author, content_type, \
               NULLIF(filename, ''), byte_size::text, sha256, meta::text, \
               created_at::text \
               FROM media_blobs WHERE id = $1::uuid"
            , [ Pgx.Value.of_string id ] )
      in
      (try
      let rows =
            Io_lwt.Db.execute conn
              ~params
              sql
          in
          match rows with
          | row :: _ -> (row_to_meta row)
          | [] -> None
    with _ ->
      None)

let get_bytes (store : Store.t) ~(id : string)
  ?(visible_to : (string * bool) option) () : (meta * string) option =
  match Io_lwt.Db.current_conn store with
  | None -> None
  | Some conn ->
      let sql, params =
        match visible_to with
        | Some (actor, is_admin) ->
            ( "SELECT id::text, author, content_type, \
               NULLIF(filename, ''), byte_size::text, sha256, meta::text, \
               created_at::text, data \
               FROM media_blobs WHERE id = $1::uuid AND "
              ^ visible_sql ~is_admin ~actor_param:2
            , [ Pgx.Value.of_string id; Pgx.Value.of_string actor ] )
        | None ->
            ( "SELECT id::text, author, content_type, \
               NULLIF(filename, ''), byte_size::text, sha256, meta::text, \
               created_at::text, data \
```

```
               FROM media_blobs WHERE id = $1::uuid"
            , [ Pgx.Value.of_string id ] )
      in
      (try
      let rows =
            Io_lwt.Db.execute conn
              ~params
              sql
          in
          match rows with
          | row :: _ ->
              (match row_to_meta row, List.nth_opt row 8 with
               | Some m, Some data_cell ->
                   (match Pgx.Value.to_binary data_cell with
                    | Some bytes -> (Some (m, bytes))
                    | None ->
                        (* Text-mode fallback if driver surfaces bytea as
    hex/escape. *)
                        (match Pgx.Value.to_string data_cell with
                         | Some s -> (Some (m, s))
                         | None -> None))
               | _ -> None)
          | [] -> None
    with _ ->
      None)

let list
    (store : Store.t)
    ?(author : string option = None)
    ?(limit : int = 50)
    () : meta list =
  match Io_lwt.Db.current_conn store with
  | None -> []
  | Some conn ->
      let limit = max 1 (min 200 limit) in
      let sql, params =
        match author with
        | Some a ->
            ( "SELECT id::text, author, content_type, \
               NULLIF(filename, ''), byte_size::text, sha256, meta::text, \
               created_at::text \
               FROM media_blobs WHERE author = $1 \
               ORDER BY created_at DESC LIMIT $2"
            , [ Pgx.Value.of_string a; Pgx.Value.of_int limit ] )
        | None ->
            ( "SELECT id::text, author, content_type, \
               NULLIF(filename, ''), byte_size::text, sha256, meta::text, \
               created_at::text \
               FROM media_blobs \
               ORDER BY created_at DESC LIMIT $1"
            , [ Pgx.Value.of_int limit ] )
      in
      (try
      let rows = Io_lwt.Db.execute conn ~params sql in
          (List.filter_map row_to_meta rows)
    with _ ->
      [])

let delete
    (store : Store.t)
    ~(id : string)
    ~(actor : string)
    ~(is_admin : bool)
  : ([ `Ok of string | `Not_found | `Forbidden ], string) result =
  let meta = get_meta store ~id () in
  match meta with
  | None -> (Ok `Not_found)
  | Some m ->
      if (not is_admin) && m.author <> actor then
        (Ok `Forbidden)
      else
        match Io_lwt.Db.current_conn store with
        | None -> (Error "no db connection")
```

```
      | Some conn ->
          let sql = "DELETE FROM media_blobs WHERE id = $1::uuid" in
          (try
    let _ =
                Io_lwt.Db.execute conn
                  ~params:[ Pgx.Value.of_string id ]
                  sql
              in
              (Ok (`Ok id))
  with exn ->
    (Error ("delete failed: " ^ Printexc.to_string exn)))
```

89 image/lib/net_prims.ml

```
(** Pricklypear Image -- Network primitives (single-image.net-prims).

    v1: EGRESS ONLY. (tcp-connect host port) opens an outbound TCP
    connection via the OCaml Unix module (synchronous); stream-read/
    write/close operate on an opaque handle stored in the session's
    [net_streams] table.

    Ingress (tcp-listen, spawn-service) and the Eio.Net-native path are
    deferred to v2 (BORGE-1785967056, BORGE-1785968883 -- Eio switches
    are run-scoped, no standalone make, so persistent handles need a
    service-manager switch wired from boot). v1 uses Unix blocking
    sockets, matching how do_http_get shells out synchronously.

    Capability: anonymous sessions denied; quota on open streams
    (default 16) prevents resource exhaustion. *)

open Pricklypear_lisp.Ast
open Session_types

let max_open_streams = 16

let random_uuid () =
  (* i-1787971303: minted identifiers use the crypto RNG, never the
     MT19937 Random state. *)
  let b = Bytes.of_string (Mirage_crypto_rng.generate 16) in
  Bytes.set b 6 (Char.chr ((Char.code (Bytes.get b 6) land 0x0f) lor 0x40));
  Bytes.set b 8 (Char.chr ((Char.code (Bytes.get b 8) land 0x3f) lor 0x80));
  let hex i = Printf.sprintf "%02x" (Char.code (Bytes.get b i)) in
  String.concat ""
    [ hex 0; hex 1; hex 2; hex 3; "-"; hex 4; hex 5; "-"; hex 6; hex 7
    ; "-"; hex 8; hex 9; "-"; hex 10; hex 11; hex 12; hex 13; hex 14; hex
     15 ]

let deny_anon t label =
  if t.manifest.username = "anonymous" then
    raise (Pricklypear_lisp.Ast.Interp_error
             (Pricklypear_lisp.Ast.Prim_error
                (label, "denied for anonymous", "denied")))
  else ()

let register (t : Session_types.t) (tbl : Primitives.t) : unit =
  (* (tcp-connect host port) -> handle-id VStr | error VStr *)
  Primitives.register tbl "tcp-connect"
    (fun args ->
      deny_anon t "tcp-connect";
        (match args with
         | [ VStr host; VStr port_s ] ->
             (try
                if Hashtbl.length t.net_streams >= max_open_streams then
                  raise (Pricklypear_lisp.Ast.Interp_error
                           (Pricklypear_lisp.Ast.Prim_error
                              ( "tcp-connect"
                              , Printf.sprintf "quota exceeded (max %d open
    streams)"
                                  max_open_streams
                              , "denied" )))
                else begin
                  let port =
                    try int_of_string port_s
                    with _ -> failwith "tcp-connect: port must be an
    integer"
                  in
                  (* Resolve host (IP literal or hostname) *)
                  let inet =
                    try Unix.inet_addr_of_string host
                    with _ ->
                      let he = Unix.gethostbyname host in
                      if Array.length he.Unix.h_addr_list = 0 then
                        failwith "tcp-connect: host not found"
                      else he.Unix.h_addr_list.(0)
                  in
```

```
                let sock = Unix.socket Unix.PF_INET Unix.SOCK_STREAM 0 in
                Unix.connect sock (Unix.ADDR_INET (inet, port));
                let id = random_uuid () in
                Hashtbl.add t.net_streams id sock;
                (VStr id)
              end
            with exn ->
              (VStr (Printf.sprintf "tcp-connect: %s"
                       (Printexc.to_string exn))))
      | _ -> (raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("tcp-connect", "need host port",
   "bad-args"))))));

(* (stream-write handle bytes) -> n-written VInt | error VStr *)
Primitives.register tbl "stream-write"
  (fun args ->
    deny_anon t "stream-write";
      (match args with
       | [ VStr handle; VStr data ] ->
           (match Hashtbl.find_opt t.net_streams handle with
            | None -> raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("stream-write", "no such handle",
   "not-found")))
            | Some sock ->
               (try
                  let n = Unix.write sock (Bytes.of_string data) 0
                            (String.length data) in
                  (VInt (Int64.of_int n))
                with exn ->
                  (VStr (Printf.sprintf "stream-write: %s"
                           (Printexc.to_string exn)))))
       | _ -> (raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("stream-write", "need handle bytes",
   "bad-args"))))));

(* (stream-read handle max-bytes) -> bytes VStr | error VStr *)
Primitives.register tbl "stream-read"
  (fun args ->
    deny_anon t "stream-read";
      (match args with
       | [ VStr handle; VStr max_s ] ->
           (match Hashtbl.find_opt t.net_streams handle with
            | None -> raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("stream-read", "no such handle",
   "not-found")))
            | Some sock ->
               let max_bytes =
                 try int_of_string max_s with _ -> 4096
               in
               let max_bytes = min max_bytes 65536 in
               (try
                  let buf = Bytes.create max_bytes in
                  let n = Unix.read sock buf 0 max_bytes in
                  (VStr (Bytes.sub_string buf 0 n))
                with exn ->
                  (VStr (Printf.sprintf "stream-read: %s"
                           (Printexc.to_string exn)))))
       | _ -> (raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("stream-read", "need handle
   max-bytes", "bad-args"))))));

(* (stream-close handle) -> ok VStr *)
Primitives.register tbl "stream-close"
  (fun args ->
    deny_anon t "stream-close";
      (match args with
       | [ VStr handle ] ->
           (match Hashtbl.find_opt t.net_streams handle with
            | None -> raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("stream-close", "no such handle",
   "not-found")))
            | Some sock ->
               (try Unix.close sock with _ -> ());
```

```
                  Hashtbl.remove t.net_streams handle;
                  (VStr "closed"))
        | _ -> (raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("stream-close", "need handle",
   "bad-args"))))));

(* (stream-list) -> VList of open handle ids *)
Primitives.register tbl "stream-list"
  (fun _args ->
    deny_anon t "stream-list";
      let ids =
        Hashtbl.fold (fun k _ acc -> (VStr k) :: acc) t.net_streams []
      in
      (VList ids))
```

90 image/lib/ops_incidents.ml

```
(** Error fingerprints ? ops-incident rows + note/todo work items.

    Detection is deterministic. LLM enrichment is optional later.
    See product-experience.reliability-work-queue. *)
open Pricklypear_lisp.Ast

let type_name = "ops-incident"

let iso_now () =
  let tm = Unix.gmtime (Unix.gettimeofday ()) in
  Printf.sprintf "%04d-%02d-%02dT%02d:%02d:%02dZ"
    (tm.Unix.tm_year + 1900) (tm.Unix.tm_mon + 1) tm.Unix.tm_mday
    tm.Unix.tm_hour tm.Unix.tm_min tm.Unix.tm_sec

let short_fp (fp : string) : string =
  let open Digestif.SHA256 in
  let hex = to_hex (digest_string fp) in
  String.sub hex 0 (min 10 (String.length hex))

let fields_of_json s =
  try
    match Yojson.Safe.from_string s with
    | `Assoc xs -> xs
    | _ -> []
  with _ -> []

let json_get fields key =
  match List.assoc_opt key fields with
  | Some (`String s) -> s
  | Some (`Int n) -> string_of_int n
  | Some (`Bool b) -> if b then "true" else "false"
  | Some j -> Yojson.Safe.to_string j
  | None -> ""

let json_get_int fields key default =
  match List.assoc_opt key fields with
  | Some (`Int n) -> n
  | Some (`String s) ->
      (match int_of_string_opt s with Some n -> n | None -> default)
  | _ -> default

let find_open_incident (store : Store.t) ~(fingerprint : string)
  : (string * (string * Yojson.Safe.t) list) option =
  let listed =
    Crud.find_rows store ~type_name
      ~filters:[ ("fingerprint", fingerprint) ]
      ~limit:20 ()
  in
  match listed with
  | VList items ->
      let rec go = function
        | [] -> None
        | VPair (VStr id, VStr fj) :: rest ->
            let fields = fields_of_json fj in
            let st = json_get fields "status" in
            if st = "open" || st = "" then Some (id, fields)
            else go rest
        | _ :: rest -> go rest
      in
      go items
  | _ -> None

let escape_lisp_str (s : string) : string =
  let buf = Buffer.create (String.length s + 8) in
  String.iter
    (function
      | '\\' -> Buffer.add_string buf "\\\\"
      | '"' -> Buffer.add_string buf "\\\""
      | '\n' -> Buffer.add_string buf "\\n"
      | '\r' -> Buffer.add_string buf "\\r"
      | c -> Buffer.add_char buf c)
```

```
    s;
  Buffer.contents buf

let file_todo_and_note (session : Session_types.t) ~(title : string)
    ~(body : string) : string option =
  let title_e = escape_lisp_str title in
  let body_e = escape_lisp_str body in
  let expr =
    Printf.sprintf
      {|(let ((tid (todo/item/add (todo/list/ensure-inbox) "%s")))
          (seq
            (if (null? tid) () (update-field 'todo-item tid "notes" "%s"))
            (notes/capture/add "%s" "%s")
            tid))|}
      title_e body_e title_e body_e
  in
  (try
      let v =
        Control_rim.run_lwt ~prims:session.prims (fun () ->
            Pricklypear_lisp.Eval.eval_string_direct
              ~prims:Pricklypear_lisp.Eval.empty_prims session.env expr)
      in
      match v with
      | VStr s when String.trim s <> "" -> (Some s)
      | _ -> None
    with exn ->
      ignore
        (Ops_log.warn ~component:"ops"
           ("file_todo_and_note failed: " ^ Printexc.to_string exn));
      let fields_json =
        Yojson.Safe.to_string
          (`Assoc
             [ ("list_id", `String "")
             ; ("title", `String title)
             ; ("notes", `String body)
             ; ("state", `String "TODO")
             ; ("due", `String "")
             ; ("position", `String "0")
             ])
      in
      let v =
        Crud.create session.store ~type_name:"todo-item" ~fields_json
          ~author:"ops"
      in
      match v with VStr s -> (Some s) | _ -> None)

let record_event (session : Session_types.t) (ev : Ops_log.event)
  : unit =
  match ev.Ops_log.level with
  | Ops_log.Info | Ops_log.Warn -> ()
  | Ops_log.Error | Ops_log.Fatal ->
      Ops_stats.note_error ~fatal:(ev.level = Ops_log.Fatal);
      let store = session.store in
      let now = iso_now () in
      let existing = find_open_incident store ~fingerprint:ev.fingerprint in
      (match existing with
       | Some (id, fields) ->
           let count = json_get_int fields "count" 1 + 1 in
           let fields_json =
             Yojson.Safe.to_string
               (`Assoc
                  [ ("fingerprint", `String ev.fingerprint)
                  ; ("level", `String (Ops_log.level_to_string ev.level))
                  ; ("component", `String ev.component)
                  ; ("count", `Int count)
                  ; ("first_seen", `String (json_get fields "first_seen"))
                  ; ("last_seen", `String now)
                  ; ("status", `String "open")
                  ; ("note_slug", `String (json_get fields "note_slug"))
                  ; ("todo_id", `String (json_get fields "todo_id"))
                  ; ("sample_msg", `String ev.msg)
                  ; ("ai_enriched", `String (json_get fields "ai_enriched"))
                  ])
```

```
            in
            let _ = Crud.update store ~type_name ~id ~fields_json in
            ()
        | None ->
            let sfp = short_fp ev.fingerprint in
            let title =
              Printf.sprintf "[ops] %s: %s" ev.component
                (let m = ev.msg in
                 if String.length m > 72 then String.sub m 0 72 ^ "?" else m)
            in
            let body =
              Printf.sprintf
                "Reliability incident (auto-filed)\n\n\
                 fingerprint: %s\n\
                 component: %s\n\
                 level: %s\n\
                 first_seen: %s\n\
                 sample: %s\n\n\
                 Filed as a TODO so it can be fixed. Deduped by fingerprint."
                ev.fingerprint ev.component
                (Ops_log.level_to_string ev.level)
                now ev.msg
            in
            let todo_id = file_todo_and_note session ~title ~body in
            let fields_json =
              Yojson.Safe.to_string
                (`Assoc
                   [ ("fingerprint", `String ev.fingerprint)
                   ; ("level", `String (Ops_log.level_to_string ev.level))
                   ; ("component", `String ev.component)
                   ; ("count", `Int 1)
                   ; ("first_seen", `String now)
                   ; ("last_seen", `String now)
                   ; ("status", `String "open")
                   ; ("note_slug", `String ("ops-" ^ sfp))
                   ; ( "todo_id"
                     , `String (Option.value todo_id ~default:"") )
                   ; ("sample_msg", `String ev.msg)
                   ; ("ai_enriched", `String "false")
                   ])
            in
            let _ =
              Crud.create store ~type_name ~fields_json ~author:"ops"
            in
            ignore
              (Ops_log.info ~component:"ops"
                 ("filed incident " ^ sfp
                  ^
                  match todo_id with
                  | Some t -> " todo=" ^ t
                  | None -> ""));
            ())

(** Classify an exception as an *expected* runtime/user error that
    should be logged but NOT filed as a reliability incident.

    Route-handler eval can raise deterministic [Interp_error] variants
    that are the user's fault (bad input, wrong arity, runaway step
    budget), not a reliability fault in the image. These still get
    emitted to the ops log (stderr + ring) so they're observable, but
    [record_event] is skipped so they don't spawn a TODO/incident row
    for every user mistake.

    Genuine faults -- [Not_a_function], [Prim_error], [Unbound_var],
    [Parse_error] (a seeded route shouldn't fail to parse at runtime),
    and any non-[Interp_error] exception -- are still filed. *)
let is_expected_runtime_error (exn : exn) : bool =
  match exn with
  | Interp_error e ->
      (match e with
       | Budget_exceeded -> true
       | User_error (_, _) -> true
       | Arity_mismatch _ -> true
```

```
       | Wrong_arg_count _ -> true
       | Type_mismatch _ -> true
       | _ -> false)
  | _ -> false

let emit_and_maybe_file (session : Session_types.t) ?(context = [])
    ?(file = true) ~(level : Ops_log.level) ~(component : string)
    (msg : string)
  : unit =
  let ev = Ops_log.emit ~context ~level ~component msg in
  if file then record_event session ev else ()

let list_open_incidents (store : Store.t) : Yojson.Safe.t =
  let listed = Crud.list store ~type_name ~limit:100 () in
  let items =
    match listed with
    | VList rows ->
        List.filter_map
          (function
            | VPair (VStr id, VStr fj) ->
                let fields = fields_of_json fj in
                let status = json_get fields "status" in
                if status = "resolved" then None
                else
                  Some
                    (`Assoc
                       [ ("id", `String id)
                       ; ( "fingerprint"
                         , `String (json_get fields "fingerprint") )
                       ; ("component", `String (json_get fields
    "component"))
                       ; ("count", `Int (json_get_int fields "count" 1))
                       ; ("level", `String (json_get fields "level"))
                       ; ("sample_msg", `String (json_get fields
    "sample_msg"))
                       ; ("todo_id", `String (json_get fields "todo_id"))
                       ; ("last_seen", `String (json_get fields
    "last_seen"))
                       ])
            | _ -> None)
          rows
    | _ -> []
  in
  (`List items)

let count_logins (store : Store.t) : (int * int) =
  match Io_lwt.Db.current_conn store with
  | None -> (0, 0)
  | Some conn ->
      let day = Ops_stats.today_str () in
      let count ~success =
        let sql =
          "SELECT COUNT(*)::int FROM auth_log " ^
          "WHERE success = $1 AND kind IN ('password','mint') " ^
          "AND happened_at::date = $2::date"
        in
        (try
      let rows =
              Io_lwt.Db.execute conn
                ~params:
                  [ Pgx.Value.of_bool success; Pgx.Value.of_string day ]
                sql
            in
            match rows with
            | [ cell ] :: _ ->
                (match Pgx.Value.to_int cell with
                 | Some n -> n
                 | None -> 0)
            | _ -> 0
    with _ ->
      0)
      in
      let ok = count ~success:true in
```

```
      let fail = count ~success:false in
      (ok, fail)
```

91 image/lib/ops_log.ml

```
(** Structured ops log ring -- product reliability stream.

    Always mirrors to stderr (image.log). Bounded in-memory ring for
    (log-tail) / agent / analytics. See
     product-experience.reliability-work-queue. *)

type level = Info | Warn | Error | Fatal

type event = {
  ts : float;
  level : level;
  component : string;
  msg : string;
  context : (string * string) list;
  fingerprint : string;
}

let level_to_string = function
  | Info -> "info"
  | Warn -> "warn"
  | Error -> "error"
  | Fatal -> "fatal"

let level_of_string s =
  match String.lowercase_ascii (String.trim s) with
  | "info" -> Some Info
  | "warn" | "warning" -> Some Warn
  | "error" -> Some Error
  | "fatal" -> Some Fatal
  | _ -> None

let max_events = 5000
let ring : event Queue.t = Queue.create ()
let mutex = Mutex.create ()
let seq = ref 0

(** Normalize msg for fingerprinting: collapse runs of hex/uuid-ish tokens.
     *)
let normalize_msg (msg : string) : string =
  let buf = Buffer.create (String.length msg) in
  let n = String.length msg in
  let i = ref 0 in
  let is_hex c =
    (c >= '0' && c <= '9') || (c >= 'a' && c <= 'f') || (c >= 'A' && c <=
     'F')
  in
  while !i < n do
    let c = msg.[!i] in
    if is_hex c then begin
      let j = ref !i in
      while !j < n && (is_hex msg.[!j] || msg.[!j] = '-') do
        incr j
      done;
      let len = !j - !i in
      if len >= 8 then Buffer.add_string buf "<id>"
      else Buffer.add_substring buf msg !i len;
      i := !j
    end else begin
      Buffer.add_char buf c;
      incr i
    end
  done;
  let s = Buffer.contents buf in
  if String.length s > 160 then String.sub s 0 160 else s

let fingerprint_of ~component ~msg =
  component ^ "|" ^ normalize_msg msg

let push (ev : event) : unit =
  Mutex.lock mutex;
  (try
```

```
    Queue.add ev ring;
    incr seq;
    while Queue.length ring > max_events do
      ignore (Queue.take ring)
    done
  with ex -> Mutex.unlock mutex; raise ex);
 Mutex.unlock mutex

let emit ?(context = []) ~(level : level) ~(component : string)
    (msg : string) : event =
 let fp = fingerprint_of ~component ~msg in
 let ev =
   { ts = Unix.gettimeofday ()
   ; level
   ; component
   ; msg
   ; context
   ; fingerprint = fp
   }
 in
 push ev;
 let ctx =
   if context = [] then ""
   else
     " "
     ^ String.concat " "
         (List.map (fun (k, v) -> k ^ "=" ^ v) context)
 in
 Printf.eprintf "[%s] %s: %s%s\n%!"
   (level_to_string level) component msg ctx;
 ev

let info ?(context = []) ~component msg = emit ~context ~level:Info
    ~component msg
let warn ?(context = []) ~component msg = emit ~context ~level:Warn
    ~component msg
let error ?(context = []) ~component msg = emit ~context ~level:Error
    ~component msg
let fatal ?(context = []) ~component msg = emit ~context ~level:Fatal
    ~component msg

let tail ?(n = 50) ?level () : event list =
 Mutex.lock mutex;
 let all =
   try Queue.fold (fun acc e -> e :: acc) [] ring |> List.rev
   with ex -> Mutex.unlock mutex; raise ex
 in
 Mutex.unlock mutex;
 let filtered =
   match level with
   | None -> all
   | Some lv ->
       List.filter
         (fun e ->
           match lv, e.level with
           | Fatal, Fatal -> true
           | Error, (Error | Fatal) -> true
           | Warn, (Warn | Error | Fatal) -> true
           | Info, _ -> true
           | _ -> false)
         all
 in
 let len = List.length filtered in
 if len <= n then filtered
 else
   let rec drop k xs =
     if k <= 0 then xs else match xs with [] -> [] | _ :: t -> drop (k -
    1) t
   in
   drop (len - n) filtered

let event_to_yojson (e : event) : Yojson.Safe.t =
 `Assoc
```

```
    [ ("ts", `Float e.ts)
    ; ("level", `String (level_to_string e.level))
    ; ("component", `String e.component)
    ; ("msg", `String e.msg)
    ; ("fingerprint", `String e.fingerprint)
    ; ( "context"
      , `Assoc (List.map (fun (k, v) -> (k, `String v)) e.context) )
    ]

let event_to_value (e : event) : Pricklypear_lisp.Ast.value =
  (* JSON string -- dict-get works on VStr JSON in this image. *)
  Pricklypear_lisp.Ast.VStr (Yojson.Safe.to_string (event_to_yojson e))
```

92 image/lib/ops_stats.ml

```
(** In-process product analytics counters (today).

    Not per-hit Postgres. See product-experience.analytics. *)

module SSet = Set.Make (String)
module SMap = Map.Make (String)

type day_stats = {
  mutable day : string;
  mutable page_views : int;
  mutable authed_requests : int;
  mutable public_uniques : SSet.t;
  mutable active_users : SSet.t;
  mutable path_counts : int SMap.t;
  mutable errors : int;
  mutable fatals : int;
  mutable noise_views : int;
}

let mutex = Mutex.create ()

let today_str () =
  let tm = Unix.gmtime (Unix.gettimeofday ()) in
  Printf.sprintf "%04d-%02d-%02d"
    (tm.Unix.tm_year + 1900) (tm.Unix.tm_mon + 1) tm.Unix.tm_mday

let empty_day day =
  { day
  ; page_views = 0
  ; authed_requests = 0
  ; public_uniques = SSet.empty
  ; active_users = SSet.empty
  ; path_counts = SMap.empty
  ; errors = 0
  ; fatals = 0
  ; noise_views = 0
  }

let state = ref (empty_day (today_str ()))

let ensure_today () =
  let d = today_str () in
  if (!state).day <> d then state := empty_day d

let hash_viewer ~(ip : string) ~(day : string) : string =
  (* Stable opaque id for the day; not reversible in the UI. *)
  let open Digestif.SHA256 in
  let dig = digest_string (day ^ "|" ^ ip) in
  let hex = to_hex dig in
  String.sub hex 0 (min 16 (String.length hex))

let path_class (path : string) : string =
  let p =
    match String.index_opt path '?' with
    | Some i -> String.sub path 0 i
    | None -> path
  in
  if p = "" then "/"
  else if String.length p >= 8 && String.sub p 0 8 = "/assets/" then
    "/assets/*"
  else if String.length p >= 8 && String.sub p 0 8 = "/static/" then
    "/static/*"
  else if String.length p >= 5 && String.sub p 0 5 = "/code" then "/code/*"
  else if String.length p >= 5 && String.sub p 0 5 = "/data" then "/data/*"
  else if String.length p >= 8 && String.sub p 0 8 = "/binding" then
    "/bindings/*"
  else if String.length p >= 5 && String.sub p 0 5 = "/edit" then "/edit/*"
  else if String.length p >= 8 && String.sub p 0 8 = "/partial" then
    "/partials/*"
  else if String.length p >= 4 && String.sub p 0 4 = "/api" then
    (match String.index_opt p '/' with
```

```
    | None -> "/api"
    | Some i when i = 0 ->
        (match String.index_from_opt p 1 '/' with
         | None -> p
         | Some j -> String.sub p 0 j)
    | Some _ -> p)
  else p

let is_noise_path (path : string) : bool =
  let p = String.lowercase_ascii path in
  let has sub =
    let n = String.length sub in
    let rec find i =
      if i + n > String.length p then false
      else if String.sub p i n = sub then true
      else find (i + 1)
    in
    find 0
  in
  has "wp-admin" || has "wp-login" || has ".env" || has "phpmyadmin"
  || has "xmlrpc" || has "/.git" || has "cgi-bin"

let is_excluded_path (path : string) : bool =
  path = "/health"
  || (String.length path >= 8 && String.sub path 0 8 = "/assets/")
  || (String.length path >= 8 && String.sub path 0 8 = "/static/")

let client_ip ~(headers : (string * string) list) ~(sockaddr :
    Unix.sockaddr)
    : string =
  let lower = List.map (fun (k, v) -> (String.lowercase_ascii k, v))
    headers in
  match List.assoc_opt "x-forwarded-for" lower with
  | Some v when String.trim v <> "" ->
      (match String.split_on_char ',' v with
       | h :: _ -> String.trim h
       | [] -> String.trim v)
  | _ ->
      (match List.assoc_opt "x-real-ip" lower with
       | Some v when String.trim v <> "" -> String.trim v
       | _ ->
           (match sockaddr with
            | Unix.ADDR_INET (a, _) -> Unix.string_of_inet_addr a
            | Unix.ADDR_UNIX _ -> "unix"))

let record_request
    ~(path : string)
    ~(is_authed : bool)
    ~(username : string option)
    ~(headers : (string * string) list)
    ~(sockaddr : Unix.sockaddr)
  : unit =
  if is_excluded_path path then ()
  else begin
    Mutex.lock mutex;
    (try
       ensure_today ();
       let s = !state in
       let cls = path_class path in
       if is_noise_path path then s.noise_views <- s.noise_views + 1
       else begin
         s.page_views <- s.page_views + 1;
         s.path_counts <-
           SMap.update cls
             (function None -> Some 1 | Some n -> Some (n + 1))
             s.path_counts;
         let day = s.day in
         let ip = client_ip ~headers ~sockaddr in
         let vk = hash_viewer ~ip ~day in
         if not is_authed then
           s.public_uniques <- SSet.add vk s.public_uniques;
         if is_authed then begin
           s.authed_requests <- s.authed_requests + 1;
```

```
          (match username with
           | Some u when u <> "" && u <> "anonymous" ->
               s.active_users <- SSet.add u s.active_users
           | _ -> ())
        end
      end
    with ex -> Mutex.unlock mutex; raise ex);
   Mutex.unlock mutex
 end

let note_error ~(fatal : bool) : unit =
 Mutex.lock mutex;
 (try
    ensure_today ();
    let s = !state in
    s.errors <- s.errors + 1;
    if fatal then s.fatals <- s.fatals + 1
  with ex -> Mutex.unlock mutex; raise ex);
 Mutex.unlock mutex

let snapshot () =
 Mutex.lock mutex;
 let s =
   try
     ensure_today ();
     let cur = !state in
     let top =
       SMap.bindings cur.path_counts
       |> List.sort (fun (_, a) (_, b) -> compare b a)
       |> fun xs ->
          let rec take n xs =
            if n <= 0 then [] else match xs with [] -> [] | h :: t -> h ::
    take (n - 1) t
          in
          take 15 xs
     in
     ( cur.day
     , cur.page_views
     , SSet.cardinal cur.public_uniques
     , cur.authed_requests
     , SSet.cardinal cur.active_users
     , cur.errors
     , cur.fatals
     , cur.noise_views
     , top
     , SSet.elements cur.active_users )
   with ex -> Mutex.unlock mutex; raise ex
 in
 Mutex.unlock mutex;
 s

let summary_yojson ~(logins_ok : int) ~(logins_fail : int) : Yojson.Safe.t =
 let day, views, uniques, authed, active_n, errors, fatals, noise, top,
    users =
   snapshot ()
 in
 `Assoc
   [ ("day", `String day)
   ; ("page_views", `Int views)
   ; ("public_uniques", `Int uniques)
   ; ("authed_requests", `Int authed)
   ; ("active_users", `Int active_n)
   ; ("active_user_names", `List (List.map (fun u -> `String u) users))
   ; ("logins_ok", `Int logins_ok)
   ; ("logins_fail", `Int logins_fail)
   ; ("errors", `Int errors)
   ; ("fatals", `Int fatals)
   ; ("noise_views", `Int noise)
   ; ( "top_paths"
     , `List
         (List.map
            (fun (p, n) -> `Assoc [ ("path", `String p); ("n", `Int n) ])
            top) )
```

```
210     ]
```

93 image/lib/packages.ml

```
(** In-branch prefix packages: organize function names with '/'.

    Orthogonal to branch ownership. Full binding key remains the
    entire name string (e.g. "todo/list"). See image.packages. *)

let package_of_name (name : string) : string =
  match String.rindex_opt name '/' with
  | None -> ""
  | Some i -> String.sub name 0 i

let local_name (name : string) : string =
  match String.rindex_opt name '/' with
  | None -> name
  | Some i -> String.sub name (i + 1) (String.length name - i - 1)

let join_name ~(package : string) ~(local : string) : string =
  let package = String.trim package in
  let local = String.trim local in
  if package = "" then local
  else if local = "" then package
  else
    let pkg =
      if String.length package > 0 && package.[String.length package - 1] =
     '/'
      then String.sub package 0 (String.length package - 1)
      else package
    in
    pkg ^ "/" ^ local

(** Group (name, body, author) defs by package. Sorted package keys;
    root "" first. Within package, sort by full name. *)
let group_by_package
    (defs : (string * 'a * string) list)
    : (string * (string * 'a * string) list) list =
  let tbl : (string, (string * 'a * string) list) Hashtbl.t =
    Hashtbl.create 16
  in
  List.iter
    (fun ((name, _, _) as row) ->
      let pkg = package_of_name name in
      let prev = try Hashtbl.find tbl pkg with Not_found -> [] in
      Hashtbl.replace tbl pkg (row :: prev))
    defs;
  let pkgs =
    Hashtbl.fold (fun k _ acc -> k :: acc) tbl []
    |> List.sort (fun a b ->
           match a, b with
           | "", _ -> -1
           | _, "" -> 1
           | _ -> String.compare a b)
  in
  List.map
    (fun pkg ->
      let rows =
        Hashtbl.find tbl pkg
        |> List.sort (fun (a, _, _) (b, _, _) -> String.compare a b)
      in
      (pkg, rows))
    pkgs

let package_label (pkg : string) : string =
  if pkg = "" then "(root)" else pkg

(** URL-encode a function name for path use (encode / as %2F). *)
let name_encode (name : string) : string =
  let buf = Buffer.create (String.length name + 8) in
  String.iter
    (function
      | '/' -> Buffer.add_string buf "%2F"
      | ' ' -> Buffer.add_string buf "%20"
      | '+' -> Buffer.add_string buf "%2B"
```

```
      | '(' -> Buffer.add_string buf "%28"
      | ')' -> Buffer.add_string buf "%29"
      | '&' -> Buffer.add_string buf "%26"
      | '=' -> Buffer.add_string buf "%3D"
      | c -> Buffer.add_char buf c)
    name;
  Buffer.contents buf

(** Decode path segment(s) after /edit/ back to a function name. *)
let name_decode (s : string) : string =
  (* reuse minimal decode for %2F and + *)
  let buf = Buffer.create (String.length s) in
  let len = String.length s in
  let rec loop i =
    if i >= len then Buffer.contents buf
    else
      match s.[i] with
      | '+' ->
          Buffer.add_char buf ' ';
          loop (i + 1)
      | '%' when i + 2 < len ->
          let hex = String.sub s (i + 1) 2 in
          (try
             let code = int_of_string ("0x" ^ hex) in
             Buffer.add_char buf (Char.chr code)
           with _ -> Buffer.add_char buf '%');
          loop (i + 3)
      | c ->
          Buffer.add_char buf c;
          loop (i + 1)
  in
  loop 0

let edit_href (name : string) : string = "/edit/" ^ name_encode name
let binding_href (name : string) : string = "/binding/" ^ name_encode name
let inspect_href (name : string) : string =
  "/inspect/binding/" ^ name_encode name
```

94 image/lib/primitives.ml

```
(** Pricklypear Image -- Primitive registry.

    Primitives are OCaml functions exposed to Lisp. They live in the image
    (not the lisp/ package) because they're I/O-shaped: read from store,
    emit htmx, persist, etc. The lisp/ package is pure I/O-free.

    Spec [lisp-interpreter.primitives] lists the full ~20; for the minimal
    image, ~5 are sufficient to demo the loop. *)
open Pricklypear_lisp.Ast
(* Prim fn: evaluated args -> value (or raise). Direct-style under Eio
     floor.
   Pure and DB-touching prims share the same ABI; I/O blocks the fiber. *)
type prim_fn = value list -> value

(* Alias: same as prim_fn after decolor. *)
type prim_fn_sync = value list -> value

type t = (string, prim_fn) Hashtbl.t

let make () : t = Hashtbl.create 16

let register (t : t) (name : string) (fn : prim_fn) : unit =
  Hashtbl.replace t name fn

let register_sync (t : t) (name : string) (fn : prim_fn_sync) : unit =
  Hashtbl.replace t name (fun args -> (fn args))

(* --- Server-only locality metadata (borge/remote-eval.borg
   server-endpoint). ---

   Some prims MUST execute server-side to mean anything: route, fork,
   merge, checkout, create-user, is-admin? (auth admin). The
   remote client's /api/prim endpoint refuses these with a structured
   `server_only` error instead of pretending to proxy them.

   The classification is DERIVED from this registry metadata, never a
   hand-maintained list in the endpoint: a prim is server-only iff it
   was registered via [register_server_only]/[register_server_only_sync]
   below. The set is registry-wide (server-only locality is a property
   of the prim NAME, identical for every session's table) and lives in
   this registry module so the endpoint reads the same metadata the
   registration sites wrote. *)
let server_only : (string, unit) Hashtbl.t = Hashtbl.create 8

(** Register [name] AND mark it server-only (refused by a remote
    client's /api/prim). Metadata lives in this registry; the endpoint
    reads it via [is_server_only]. *)
let register_server_only (t : t) (name : string) (fn : prim_fn) : unit =
  Hashtbl.replace server_only name ();
  register t name fn

let register_server_only_sync
    (t : t) (name : string) (fn : prim_fn_sync) : unit =
  Hashtbl.replace server_only name ();
  register_sync t name fn

(** Refuse-or-forward classification, derived from registry metadata.
    A prim is server-only iff it was registered via
    [register_server_only*]. *)
let is_server_only (_t : t) (name : string) : bool =
  Hashtbl.mem server_only name

let lookup (t : t) (name : string) : prim_fn option =
  Hashtbl.find_opt t name

(* --- Refuse-or-forward dispatch for /api/prim (borge/remote-eval.borg
   server-endpoint). ---

   One prim call, dispatched exactly the way the image rim services a
   [Prim_call] during eval (same table lookup, same [fn args]), but
   with the server-only refusal in front of it. Classification is
```

```
   DERIVED from registry metadata ([is_server_only]), never a hand
   list. Factored here (rather than inline in http_pages.ml) so the
   endpoint is thin and the refusal/execution logic is unit-testable
   without spinning up an HTTP server. *)
type prim_result =
  | Prim_ok of value
  | Prim_refused_server_only of string * string (* name * reason *)
  | Prim_error of exn

let dispatch (t : t) (name : string) (args : value list) : prim_result =
  if is_server_only t name then
    Prim_refused_server_only (name, "prim must execute server-side")
  else
    match lookup t name with
    | Some fn ->
        (try Prim_ok (fn args) with exn -> Prim_error exn)
    | None ->
        Prim_error
          (Interp_error
             (Prim_error (name, "unregistered primitive", "prim_error")))

(* Numeric helpers: VInt | VFloat with promotion (int+float ? float). *)
(* --- Session-aware primitives ---

   These need access to the running session (env, manifest). They're
   registered as closures at session-make time via
   [register_session_prims]. The signature stays prim_fn (value list
   -> value) -- the closure captures the session. *)

(* (whoami) -> VStr. Current session username. [username_fn] is
   re-read on each call so cotenant multi-user can switch identity
   per HTTP request without re-registering prims. *)
let prim_whoami_fn ~(username_fn : unit -> string) (_ : value list) : value
    =
  VStr (username_fn ())

(* (current-author) -> VStr. Same as whoami for now; reserved for
   the multi-user permission model where (current-author) consults
   the binding being executed rather than the session. *)
let prim_current_author_fn ~(username_fn : unit -> string)
    (_ : value list) : value =
  VStr (username_fn ())

(* (list-bindings) -> VList of binding-name VStrs. Lists all names
   currently visible in the session env. *)
let prim_list_bindings_fn ~(env : Pricklypear_lisp.Ast.env) (_ : value
    list) : value =
  let names =
    Pricklypear_lisp.Env.fold env
      (fun name _ acc -> name :: acc) []
  in
  VList (List.map (fun s -> VStr s) names)

(* Register session-aware primitives into [t]. Called at session-make
   time with a dynamic username getter + env. *)
let register_session_prims
    (t : t)
    ~(username_fn : unit -> string)
    ~(env : Pricklypear_lisp.Ast.env)
    : unit =
  register_sync t "whoami" (prim_whoami_fn ~username_fn);
  register_sync t "current-author" (prim_current_author_fn ~username_fn);
  register_sync t "list-bindings" (prim_list_bindings_fn ~env)


(* (date-today) -> VStr "YYYY-MM-DD" in local time. *)

open Prims_pure

(* Re-export pure helpers used by Session / Http. *)
let is_raw_html = Prims_pure.is_raw_html
let is_multipart = Prims_pure.is_multipart
let is_part = Prims_pure.is_part
```

```
let is_redirect = Prims_pure.is_redirect
let redirect_value = Prims_pure.redirect_value
let is_status = Prims_pure.is_status
let status_response = Prims_pure.status_response
let prim_dict_set = Prims_pure.prim_dict_set

(* i-1787971303: image-side crypto-random variant of the pure
   prims_pure.prim_random_uuid -- minted identifiers (session-scoped
   Lisp (random-uuid)) must come from Mirage_crypto_rng, never the
   MT19937 Random state. Shadowing the pure prim for this registry;
   the browser client keeps the pure version. *)
let prim_random_uuid (_args : value list) : value =
  let b = Bytes.of_string (Mirage_crypto_rng.generate 16) in
  (* RFC 4122 version 4 + variant 10xx *)
  Bytes.set b 6 (Char.chr ((Char.code (Bytes.get b 6) land 0x0f) lor 0x40));
  Bytes.set b 8 (Char.chr ((Char.code (Bytes.get b 8) land 0x3f) lor 0x80));
  let hex_byte i = Printf.sprintf "%02x" (Char.code (Bytes.get b i)) in
  VStr
    (String.concat ""
       [ hex_byte 0; hex_byte 1; hex_byte 2; hex_byte 3; "-"
       ; hex_byte 4; hex_byte 5; "-"
       ; hex_byte 6; hex_byte 7; "-"
       ; hex_byte 8; hex_byte 9; "-"
       ; hex_byte 10; hex_byte 11; hex_byte 12; hex_byte 13; hex_byte 14;
         hex_byte 15
       ])

(* Register the minimal set. Returns the populated table. *)
let make_with_defaults () : t =
  let t = make () in
  register_sync t "+" prim_add;
  register_sync t "-" prim_sub;
  register_sync t "*" prim_mul;
  register_sync t "div" prim_div;
  register_sync t "/" prim_div; (* Scheme-style alias of div *)
  register_sync t "mod" prim_mod;
  register_sync t "quotient" prim_quotient;
  register_sync t "remainder" prim_remainder;
  register_sync t "abs" prim_abs;
  register_sync t "min" prim_min;
  register_sync t "max" prim_max;
  (* lisp-interpreter.math -- transcendental + predicates + RNG *)
  register_sync t "expt" prim_expt;
  register_sync t "sqrt" prim_sqrt;
  register_sync t "exp" prim_exp;
  register_sync t "log" prim_log;
  register_sync t "sin" prim_sin;
  register_sync t "cos" prim_cos;
  register_sync t "tan" prim_tan;
  register_sync t "asin" prim_asin;
  register_sync t "acos" prim_acos;
  register_sync t "atan" prim_atan;
  register_sync t "number?" prim_number_p;
  register_sync t "integer?" prim_integer_p;
  register_sync t "zero?" prim_zero_p;
  register_sync t "positive?" prim_positive_p;
  register_sync t "negative?" prim_negative_p;
  register_sync t "even?" prim_even_p;
  register_sync t "odd?" prim_odd_p;
  register_sync t "random" prim_random;
  register_sync t "random-seed" prim_random_seed;
  register_sync t "=" prim_num_eq;
  register_sync t "<" prim_lt;
  register_sync t ">" prim_gt;
  register_sync t "<=" prim_le;
  register_sync t ">=" prim_ge;
  register_sync t "not" prim_not;
  register_sync t "list" prim_list;
  register_sync t "car" prim_car;
  register_sync t "cdr" prim_cdr;
  register_sync t "cons" prim_cons;
  register_sync t "null?" prim_null_p;
  register_sync t "pair?" prim_pair_p;
```

```
  register_sync t "string-append" prim_string_append;
  register_sync t "string-join" prim_string_join;
  register_sync t "string-length" prim_string_length;
  register_sync t "substring" prim_substring;
  register_sync t "string-contains?" prim_string_contains;
  register_sync t "string-split" prim_string_split;
  register_sync t "prose-tokens" prim_prose_tokens;
  register_sync t "render-tokens" prim_render_tokens;
  register_sync t "string-eq" prim_string_eq;
  register_sync t "string<?" prim_string_lt;
  register_sync t "string<=?" prim_string_le;
  register_sync t "string>?" prim_string_gt;
  register_sync t "string>=?" prim_string_ge;
  register_sync t "string-trim" prim_string_trim;
  register_sync t "html-escape" prim_html_escape;
  register_sync t "json-parse" prim_json_parse;
  register_sync t "json-emit" prim_json_emit;
  register_sync t "read-string" prim_read_string;
  register_sync t "read-all-string" prim_read_all_string;
  register_sync t "type-of" prim_type_of;
  register_sync t "symbol?" prim_symbol_p;
  register_sync t "string?" prim_string_p;
  register_sync t "symbol=?" prim_symbol_eq;
  register_sync t "symbol->string" prim_symbol_to_string;
  register_sync t "string->symbol" prim_string_to_symbol;
  register_sync t "str" prim_str;
  register_sync t "as-str" prim_as_str;
  register_sync t "append" prim_append;
  register_sync t "reverse" prim_reverse;
  register_sync t "nth" prim_nth;
  (* list-branches is registered as a session DB prim in session.ml *)
  register_sync t "hx-get" prim_hx_get;
  register_sync t "hx-post" prim_hx_post;
  register_sync t "err/shape" prim_err_shape;
  register_sync t "count" prim_count;
  register_sync t "length" prim_length;
  register_sync t "equal?" prim_equal_p;
  register_sync t "cadr" prim_cadr;
  register_sync t "caar" prim_caar;
  register_sync t "caddr" prim_caddr;
  register_sync t "cddr" prim_cddr;
  register_sync t "assoc" prim_assoc;
  register_sync t "member" prim_member;
  register_sync t "dict-get" prim_dict_get;
  register_sync t "dict-set" prim_dict_set;
  register_sync t "date-today" Prims_time.prim_date_today;
  register_sync t "now" Prims_time.prim_now;
  register_sync t "now-ms" Prims_time.prim_now_ms;
  register_sync t "parse-int" prim_parse_int;
  register_sync t "parse-float" prim_parse_float;
  register_sync t "floor" prim_floor;
  register_sync t "round" prim_round;
  register_sync t "date-weekday" Prims_time.prim_date_weekday;
  register_sync t "date-format" Prims_time.prim_date_format;
  register_sync t "date-add-days" Prims_time.prim_date_add_days;
  register_sync t "date-diff-days" Prims_time.prim_date_diff_days;
  register_sync t "fsrs-schedule" prim_fsrs_schedule;
  register_sync t "dict-keys" prim_dict_keys;
  register_sync t "render-html" prim_render_html;
  register_sync t "status-response" prim_status_response;
  register_sync t "part" Prims_pure.prim_part;
  register_sync t "json-part" Prims_pure.prim_json_part;
  register_sync t "multipart" Prims_pure.prim_multipart;
  register_sync t "extract-links" prim_extract_links;
  register_sync t "render-links" prim_render_links;
  register_sync t "budget-remaining" prim_budget_remaining;
  (* kernel I/O primitives (from notes inbox
     kernel-io-primitives-2026-07-27) *)
  register_sync t "string-replace" prim_string_replace;
  register_sync t "string-downcase" prim_string_downcase;
  register_sync t "string-upcase" prim_string_upcase;
  register_sync t "string-starts-with?" prim_string_starts_with_p;
  register_sync t "string-ends-with?" prim_string_ends_with_p;
```

```
  register_sync t "url-encode" prim_url_encode;
  register_sync t "url-decode" prim_url_decode;
  register_sync t "sha1" Prims_crypto.prim_sha1;
  register_sync t "html-decode" prim_html_decode;
  register_sync t "parse-date" prim_parse_date;
  register_sync t "xml-parse" prim_xml_parse;
  register_sync t "random-uuid" prim_random_uuid;
  t

(* Short docs for the Functions browser (kernel package). *)
let prim_summary (name : string) : string =
  match name with
  | "+" -> "add numbers"
  | "-" -> "subtract / negate"
  | "*" -> "multiply"
  | "div" | "/" -> "divide (int when both int; else float)"
  | "mod" -> "integer remainder"
  | "abs" -> "absolute value"
  | "min" -> "minimum of VInts"
  | "max" -> "maximum of VInts"
  | "=" -> "numeric equality"
  | "<" | ">" | "<=" | ">=" -> "numeric compare"
  | "not" -> "boolean not"
  | "list" -> "build a list"
  | "car" -> "first of pair/list"
  | "cdr" -> "rest of pair/list"
  | "cons" -> "cons cell"
  | "null?" -> "empty list?"
  | "pair?" -> "is pair?"
  | "count" -> "length of list"
  | "length" -> "length of list (accepts proper lists and cons chains)"
  | "equal?" -> "structural deep equality"
  | "err/shape" -> "build the (error msg kind) envelope as data"
  | "cadr" -> "second element (car of cdr)"
  | "caar" -> "car of car"
  | "caddr" -> "third element (car of cdr of cdr)"
  | "cddr" -> "cdr of cdr"
  | "assoc" -> "lookup key in alist; entry or nil"
  | "member" -> "sublist starting at x, or #f"
  | "date-today" -> "local date YYYY-MM-DD"
  | "now" -> "unix epoch seconds (UTC int)"
  | "now-ms" -> "unix epoch milliseconds (UTC int)"
  | "sleep" -> "sleep N seconds (float or int); yields on Lwt"
  | "http-get" -> "outbound HTTP GET: (http-get url [headers-json]) ->
     {ok,status,headers,body,err,kind} dict; admin/agent only"
  | "http-post" -> "outbound HTTP POST: (http-post url [headers-json
     [body]]) -> {ok,status,headers,body,err,kind} dict; admin/agent only"
  | "parse-int" -> "string or int to int"
  | "parse-float" -> "string or number to float"
  | "floor" | "round" -> "float to int"
  | "expt" -> "power: int^nonneg-int stays int; else float"
  | "sqrt" -> "square root (error on negative)"
  | "exp" -> "e^x"
  | "log" -> "natural log, or log base b with 2 args"
  | "sin" | "cos" | "tan" -> "trig in radians"
  | "asin" | "acos" | "atan" -> "inverse trig; atan 2-arg is atan2"
  | "number?" -> "true for int or float"
  | "integer?" -> "true for VInt only (tag-based)"
  | "zero?" | "positive?" | "negative?" -> "numeric sign predicates"
  | "even?" | "odd?" -> "parity; integers (or integral floats)"
  | "quotient" -> "integer division (Scheme name for div int path)"
  | "remainder" -> "integer remainder (Scheme name for mod int path)"
  | "random" -> "(random) float [0,1); (random n) int [0,n)"
  | "random-seed" -> "reseed process PRNG from int"
  | "date-weekday" -> "weekday 0=Sun for y m d"
  | "date-format" -> "format y m d as YYYY-MM-DD"
  | "date-add-days" -> "add n days to YYYY-MM-DD (or today)"
  | "date-diff-days" -> "days between two YYYY-MM-DD strings"
  | "fsrs-schedule" -> "FSRS-4.5: (S D elapsed rating) -> (S' D' interval)"

  | "string-append" -> "concat strings"
  | "string-join" -> "join a list of strings in O(n), optional sep"
  | "string-length" -> "length of string"
```

```
    | "substring" -> "substring start [len]"
    | "string-contains?" -> "haystack contains needle?"
    | "string-split" -> "split string on delimiter string ? list of pieces"
    | "prose-tokens" -> "unique token-shaped binding cores in prose (scan,
       budget-safe)"
    | "render-tokens" -> "prose paragraph HTML with binding-token links/holes"
    | "string-eq" -> "string equality"
    | "string<?" -> "string less-than (lexicographic)"
    | "string<=?" -> "string less-or-equal (lexicographic)"
    | "string>?" -> "string greater-than (lexicographic)"
    | "string>=?" -> "string greater-or-equal (lexicographic)"
    | "string-trim" -> "trim whitespace from string"
    | "html-escape" -> "escape text for HTML"
    | "json-parse" -> "parse JSON string to Lisp value"
    | "json-emit" -> "emit Lisp value as JSON string"
    | "read-string" -> "read one s-expr from string ? quoted data (no eval)"
    | "read-all-string" -> "read all top-level s-exprs from string ? list"
    | "example-eval" ->
        "restricted read-time eval of a book example: (example-eval src) ->
       rendered value; whitelisted pure + read-only prims, no
       define/route/write"
    | "with-public-write" ->
        "(with-public-write (lambda () ?)) run body with write privs for
       public forms"
    | "request-method" -> "HTTP method of active request"
    | "request-path" -> "path of active request"
    | "request-param" -> "form/query param of active request"
    | "request-header" -> "HTTP header of active request"
    | "hx-request?" -> "true if HX-Request header present"
    | "request-params" -> "all params as JSON object"
    | "request-body" -> "raw body of active request"
    | "redirect" -> "HTTP 303 redirect response"
    | "append" -> "concatenate lists"
    | "reverse" -> "reverse list"
    | "nth" -> "nth element of list (0-based)"
    | "str" -> "value to string"
    | "as-str" -> "plain string coerce (no quotes)"
    | "type-of" -> "runtime type tag"
    | "symbol?" -> "true if value is a symbol"
    | "string?" -> "true if value is a string"
    | "symbol=?" -> "symbol name equality"
    | "symbol->string" -> "symbol name as string"
    | "string->symbol" -> "intern string as symbol"
    | "dict-get" -> "JSON object field get"
    | "dict-set" -> "JSON object field set"
    | "dict-keys" -> "JSON object keys"
    | "render-html" -> "mark string as raw HTML"
    | "part" -> "build one multipart/mixed part (html)"
    | "json-part" -> "build one multipart/mixed JSON part"
    | "multipart" -> "bundle parts into a multipart/mixed response"
    | "hx-get" | "hx-post" -> "htmx attribute helper"
    | "whoami" | "current-author" -> "session identity"
    | "list-bindings" -> "names in env"
    | "list-branches" -> "list branch refs"
    | "list-usernames" -> "list usernames (authenticated; for chat/DM picker)"
    | "is-admin?" -> "true if calling identity is admin"
    | "create-user" -> "admin: create user + password -> user id"
    | "add-ssh-key" -> "register OpenSSH public key for SSH REPL auth"
    | "list-ssh-keys" -> "list SSH key fingerprints for current user"
    | "revoke-ssh-key" -> "revoke SSH key by fingerprint or id"
    | "set-llm-key" -> "store per-user LLM provider key (secret; status shows
       last4)"
    | "clear-llm-key" -> "remove per-user LLM provider key"
    | "llm-key-status" -> "configured? last4 base model (never raw secret)"
    | "current-branch" -> "current branch name"
    | "fork" -> "fork a branch"
    | "merge" -> "merge branches"
    | "checkout" -> "switch branch"
    | "route" -> "bind URL to function"
    | "deftype" -> "declare a data type"
    | "get" -> "read one data row"
    | "create" -> "insert data row ? id"
    | "update" -> "replace row fields JSON"
```

```
426   | "update-field" -> "set one JSON field"
427   | "delete" -> "delete data row"
428   | "rows" -> "list data rows"
429   | "find-row" -> "first row by fields->>key"
430   | "find-rows" -> "rows filtered by fields"
431   | "find-text" -> "substring search on a field"
432   | "find-code" -> "substring search over function body AST
        (branch-visible)"
433   | "find-random" -> "one random data row of a type"
434   | "select-for-update" -> "lock row in transaction (requires
        with-transaction)"
435   | "node-path" -> "pure path constructor data.<type>.<id>"
436   | "backlinks" -> "paths that reference needle (edges first, then
        DEPRECATED legacy body-string matches)"
437   | "type-fields" -> "declared fields for a type"
438   | "tree/get" -> "get hierarchical tree node by path ? JSON"
439   | "tree/list" -> "list subtree (depth 1 = children) under path"
440   | "tree/children-of" -> "DIRECT child nodes of path via materialized path
        index"
441   | "tree/subtree" -> "capped subtree payload under path via materialized
        path index"
442   | "tree/lens-get" -> "resolve relative fn/type/route/pkg path across the
        branch lens chain (closest-wins)"
443   | "tree/lens-children" -> "direct children of relative path under the
        effective branch lens"
444   | "tree/lens-subtree" -> "capped subtree of relative path under the
        effective branch lens"
445   | "lens/fork" -> "create a lens branch as a named child of the
        checked-out branch (local-first lens-model;
        <owner>/lens-<device>[-<date>])"
446   | "replication/unit" -> "materialize the authorized-read subtree
        (replication unit): data.u.<user>.* + data.shared.* + code lens +
        branch ancestry ? JSON manifest"
447   | "sync/plan" -> "server-side sync reconcile: send a client (path,hash)
        manifest [+ base manifest] ? JSON reconciliation report / three-way
        merge plan (local-first sync)"
448   | "sync/push" -> "apply a client two-way push (manifest + bodies) for the
        caller's data.u.<user>.* subtree; returns {applied, server_only_paths,
        diverge_paths, generation} (local-first sync v1; diverge reported, not
        auto-merged)"
449   | "tree/upsert" -> "upsert folder|note|heading at path"
450   | "tree/delete" -> "delete path [, recursive]"
451   | "tree/move" -> "move subtree old-path ? new-path"
452   | "tree/reorder" -> "set node rank directly (base-36 fractional string;
        \"\" clears) ? node JSON"
453   | "tree/move-before" -> "place node immediately before ranked sibling
        anchor ? new rank"
454   | "tree/move-after" -> "place node immediately after ranked sibling
        anchor ? new rank"
455   | "tree/siblings" -> "ordered children of path's parent (ranked first) ?
        node JSONs"
456   | "tree/next" -> "adjacent sibling after path in tree order ? node JSON
        or nil"
457   | "tree/prev" -> "adjacent sibling before path in tree order ? node JSON
        or nil"
458   | "tree/insert-child" -> "create child of parent with rank after anchor
        (or appended); hier kinds; body needs slug/title"
459   | "link" -> "create an edge node (kind=link) from ? to; to may dangle ?
        edge node JSON"
460   | "unlink" -> "tombstone an edge node (kind=link only) ? edge path"
461   | "out-links" -> "outgoing edges of path with resolution status
        (resolved/dangling) ? edge JSONs"
462   | "tree/get-uuid" -> "lookup node by payload body.uuid"
463   | "tree/find-by" -> "find nodes by kind + body field"
464   | "tree/path-label" -> "sanitize string to ltree-safe label"
465   | "tree/parent" -> "strict parent path of path, or nil at root (pure)"
466   | "tree/leaf" -> "final label of path (pure)"
467   | "tree/segments" -> "labels of path, root-first (pure)"
468   | "tree/child" -> "append label to path (encodes; pure)"
469   | "tree/ancestors" -> "every strict prefix of path, root-first (pure)"
470   | "tree/garden-root" -> "data.note.u.<user> path prefix"
471   | "gen-now" -> "current branch generation (int) -- reads-as-of-gen clock
        coordinate"
```

```
472   | "gen-read" -> "node body at gen (absolute path or lens suffix), crossed
        over branch lens rules ? version JSON"
473   | "gen-diff" -> "node-granular diff between gen-a and gen-b (hashes
        first, walk on mismatch)"
474   | "undo" -> "restore value at head-n as a NEW write at head+1
        (restore-by-write, datum paths)"
475   | "code-restore" -> "restore a code node (fn/type/route) at head-n as a
        NEW write at head+1 (restore-by-write, gen-read + define)"
476   | "tree/folder-index" -> "distinct folder paths under prefix (SQL dedup)"
477   | "tree/children-html" -> "budget-safe HTML for notes-tree children under
        path"
478   | "extract-links" -> "parse [[wiki]] / [[slug|label]] from body ? list of
        pairs"
479   | "render-links" -> "body with [[wiki]]/[[media:]] ? HTML (budget-safe)"
480   | "budget-remaining" -> "steps left in current eval budget"
481   | "string-replace" -> "replace all occurrences of pat with repl"
482   | "string-downcase" -> "lowercase ASCII"
483   | "string-upcase" -> "uppercase ASCII"
484   | "string-starts-with?" -> "prefix test"
485   | "string-ends-with?" -> "suffix test"
486   | "url-encode" -> "percent-encode a string"
487   | "url-decode" -> "percent-decode a string"
488   | "sha1" -> "SHA-1 hex digest of a string"
489   | "html-decode" -> "decode HTML entities in a string"
490   | "parse-date" -> "parse RFC-822/ISO-8601/date -> UTC epoch int"
491   | "xml-parse" -> "lenient XML text -> Lisp tree"
492   | "random-uuid" -> "generate a random UUID string"
493   | "bench" -> "avg microseconds per eval of expr-string [n]"
494   | "agent-session-new" -> "create durable AI session: (agent-session-new
        seed [tools?])"
495   | "agent-say" -> "append user msg + run tool loop: (agent-say sid prompt)"
496   | "agent-history" -> "list messages in AI session"
497   | "agent-sessions" -> "list recent AI sessions for whoami"
498   | "agent-session-delete" -> "delete AI session and messages"
499   | "agent" -> "alias: new session + one say (prefer agent-session-new/say)"
500   | "agent-task" -> "alias: new tools session + one say"
501   | "propose-function" -> "AI draft+install function on branch"
502   | "install-function" -> "install AI body string as named function"
503   | "function-source" -> "pretty-print stored function body"
504   | "function-doc" -> "docstring for a binding (function_defs.doc)"
505   | "analyze" -> "LLM/heuristic complexity analysis of a function (cached)"
506   | "explain" -> "inside-out structural breakdown of a function"
507   | "function-ast" -> "raw stored function body as quoted data (no eval)"
508   | "forge-effects" -> "capability badge (pure/db-read/db-write/?) for a
        parsed source expr or named binding"
509   | "llm-chat" -> "raw one-shot LLM completion (system, user) -> content
        string; not the agentic loop"
510   | "submit-job" -> "submit a background job: (submit-job fn args) ->
        job-id; runner evals (fn args) on your branch"
511   | "submit-job-at" -> "submit a scheduled job: (submit-job-at fn args
        run-at-ms) -> job-id"
512   | "job-status" -> "get a job row by id (confined to submitter unless
        admin): (job-status id)"
513   | "jobs" -> "list your recent jobs (admin sees all): (jobs)"
514   | "cancel-job" -> "cancel a pending job: (cancel-job id)"
515   | "package-exports" -> "shallow public names for a package (std freeze or
        pkg/local)"
516   | "package-bindings" -> "all binding names with package prefix (any
        depth)"
517   | "list-packages" -> "top-level package prefixes from function_defs"
518   | "apropos-docs" -> "(name doc) pairs for names containing pattern"
519   | "describe" -> "selective reveal: package summary or one binding doc"
520   | "export-package" -> "export package as lib.pp text"
521   | "llm-inner" | "llm-outer" | "llm-rewrite" | "llm-seed" ->
522       "AI rewrite / growth tool"
523   | "__with_transaction__" -> "internal transaction helper"
524   | "load-string" -> "load Lisp source text (define/route/deftype)"
525   | "load-library" -> "load libs/<name>/lib.pp into current branch"
526   | "reseed-package" -> "admin: upsert libs/<pkg> onto system/main
        (clobber)"
527   | "callees" -> "static forward callees of a function (App-Var heads)"
528   | "callers" -> "static direct callers (senders) of a name"
529   | "dependents" -> "reverse-graph dependents of a function"
```

```
  | "used-by-routes" -> "routes whose handlers depend on a function"
  | "holes" -> "unbound App-Var names in a function body"
  | "impact-queue" -> "recent define impact events (dependents + routes)"
  | "impact-clear" -> "clear impact queue (all or one name)"
  | "impact-mark" -> "mark impact status smoked|dismissed"
  | "change-transcript" -> "session define/route transcript ring"
  | "branch-changes" -> "own-branch function/route change set"
  | "stub-function" -> "create TODO lambda stub for a missing name"
  | "log-tail" -> "recent structured ops log events"
  | "ops/summary" -> "today analytics JSON (viewers, logins, errors)"
  | "ops/incidents" -> "open reliability incident fingerprints"
  | _ -> "built-in primitive"

(** All registered prim names with one-line summaries, sorted. *)
let catalog (t : t) : (string * string) list =
  Hashtbl.fold
    (fun name _ acc -> (name, prim_summary name) :: acc)
    t []
  |> List.sort (fun (a, _) (b, _) -> String.compare a b)

let list_names (t : t) : string list =
  List.map fst (catalog t)

(* H1.7: image stores this table directly for Control_rim; lisp Eval no
   longer carries Lwt prim_fn. Identity helper kept for call-site clarity.
    *)
let to_eval_table (t : t) : t = t
```

95 image/lib/prims_crypto.ml

```
(** Crypto primitives backed by Digestif (C stubs).

    Split out of [Prims_pure] so the pure-prims library stays free of
    C-stub deps and can be shared with the js_of_ocaml client target.
    The browser client does not need sha1 for structural editing. *)
open Pricklypear_lisp.Ast

let prim_sha1 args =
  match args with
  | [ v ] ->
      let s =
        match v with
        | VStr s -> s
        | VSymbol s -> s
        | VList [VStr "__str__"; VStr s] -> s
        | VList [VSymbol "__str__"; VStr s] -> s
        | VInt n -> Int64.to_string n
        | VFloat f -> Printf.sprintf "%.12g" f
        | VBool true -> "true"
        | VBool false -> "false"
        | VNil -> ""
        | _ -> "<nonstr>"
      in
      VStr (Digestif.SHA1.to_hex (Digestif.SHA1.digest_string s))
  | _ -> raise (Interp_error (User_error ("sha1 needs 1 string",
     "bad-args")))
```

96 image/lib/prims_db.ml

```
(** Prim registration extracted from session.ml (Eio-floor Phase 4). *)
open Session_types

let promote_to_personal (t : t) ~(names : string list)
  : (int, string) result =
  let me_opt = Auth_store.get_user_by_username t.store t.manifest.username
     in
  match me_opt with
  | None -> (Error "no such user")
  | Some me when me.user_type <> "Agent_tools.agent" ->
      (Error "promote-to-personal is for Agent_tools.agent users")
  | Some me ->
      (match me.parent_user_id with
       | None -> (Error "Agent_tools.agent has no parent")
       | Some pid ->
           let parent_opt = Auth_store.get_user_by_id t.store pid in
           (match parent_opt with
            | None -> (Error "parent user not found")
            | Some parent ->
                let allowed =
                  Auth_store.get_allow_agent_promote t.store
                    ~user_id:parent.id
                in
                if not allowed then
                  (Error "Agent_tools.agent promote disabled by parent")
                else
                  let final_names =
                    if names <> [] then names
                    else
                      let meta = Store.list_function_names_meta t.store in
                      let own =
                        List.filter (fun (_, _, _, d) -> d = 0) meta
                      in
                      (List.map (fun (n, _, _, _) -> n) own)
                  in
                  if final_names = [] then
                    (Error "no functions defined on this branch")
                  else
                    let source_id = Store.current_branch_id t.store in
                    let tgt_opt =
                      Store.lookup_branch_id_by_name t.store
                        ~owner:parent.username ~name:"personal"
                    in
                    (match tgt_opt with
                     | None ->
                         (Error "parent personal branch not found")
                     | Some target_id ->
                         let n =
                           Store.promote_functions t.store
                             ~source_id ~target_id ~names:final_names
                         in
                         (Ok n))))


let register ?(rehydrate_fn : t -> unit = fun _ -> ()) (t : t) (tbl :
    Primitives.t) : unit =
  (* Quoted type/field names are VSymbol after Phase 1; CRUD still wants
     string payloads for SQL. Coerce symbols?strings at the CRUD boundary
     only (do not collapse in symbol=? / general prims). *)
  let coerce_sym_args (args : Pricklypear_lisp.Ast.value list) =
    List.map
      (function
        | Pricklypear_lisp.Ast.VSymbol s -> Pricklypear_lisp.Ast.VStr s
        | v -> v)
      args
  in
  let register_crud name fn =
    Primitives.register tbl name (fun args ->
        Pricklypear_lisp.Eval.note_db_hit ();
        let t0 = Unix.gettimeofday () in
        let result = fn (coerce_sym_args args) in
```

```
      let elapsed_ms = (Unix.gettimeofday () -. t0) *. 1000. in
      (* accumulate DB wall time into the eval budget for page-timing *)
      Pricklypear_lisp.Eval.note_db_ms elapsed_ms;
      result)
in
(* (deftype 'name 'field1 'field2 ...) -> VStr "defined type <name>".
   All args are quoted symbols (now VSymbol; coerced above). Persists
   the declaration into type_defs on the session's branch. *)
let deny_anon name =
  (* Raise, don't return a VStr: a returned string is silently dropped
     by (seq ?), so a multi-write handler could partially succeed past
     an anonymous denial. Raising aborts the whole eval.
   (BORGE-1785213376) *)
  raise
    (Pricklypear_lisp.Ast.Interp_error
       (Pricklypear_lisp.Ast.User_error
          (name ^ ": denied for anonymous", "bad-args")))
in
(* Per-request identity snapshot (avoids the cotenant race where the
   live t.manifest is mid-swap). Falls back to t.manifest outside HTTP. *)
let actor () =
  match Request_context.get () with
  | Some c -> (Request_context.manifest c).username
  | None -> t.manifest.username
in
(* deftype registration is deferred until require_admin is in scope
   (BORGE-1785213343: schema-authority -- admin only). Placeholder:
   see register after require_admin below. *)

(* (get 'type id) -> VStr fields-json | VNil *)
register_crud "get"
  (fun args ->
    match args with
    | [ Pricklypear_lisp.Ast.VStr type_name;
        Pricklypear_lisp.Ast.VStr id ] ->
        Crud.get t.store ~type_name ~id
    | _ -> (raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("get", "bad args (need type, id)",
   "bad-args")))));

(* (create 'type "{fields-json}") -> VStr new-id; raises Prim_error
   "failed" when the insert is rejected. *)
register_crud "create"
  (fun args ->
    let uname =
      match Request_context.get () with
      | Some c -> (Request_context.manifest c).username
      | None -> t.manifest.username
    in
    if uname = "anonymous" then deny_anon "create"
    else
      match args with
      | [ Pricklypear_lisp.Ast.VStr type_name;
          Pricklypear_lisp.Ast.VStr fields_json ] ->
          (* Attribute to the per-request identity, not the live manifest
             (which can be mid-swap under the cotenant race). *)
          Crud.create t.store
            ~type_name ~fields_json
            ~author:uname
      | _ ->
          (raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("create", "bad args (need type,
   fields)", "bad-args")))));

(* (update 'type id "{fields-json}") -> VStr id | VNil *)
register_crud "update"
  (fun args ->
    let uname = actor () in
    if uname = "anonymous" then deny_anon "update"
    else
      match args with
      | [ Pricklypear_lisp.Ast.VStr type_name;
          Pricklypear_lisp.Ast.VStr id;
```

```
            Pricklypear_lisp.Ast.VStr fields_json ] ->
            Crud.update t.store ~type_name ~id ~fields_json
        | _ ->
            ((raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("update", "bad args (need type, id,
   fields)", "bad-args"))))));

(* (update-field 'type id "field" "value") -> VStr id | VNil.
    Ergonomic shorthand: composes get + dict-set + update in one prim.
    Lands it18 (2026-07-06); resolves the it14 "verbose round-trip
   syntax" TODO. *)
register_crud "update-field"
  (fun args ->
    let uname = actor () in
    if uname = "anonymous" then deny_anon "update-field"
    else
      match args with
      | [ Pricklypear_lisp.Ast.VStr type_name;
          Pricklypear_lisp.Ast.VStr id;
          Pricklypear_lisp.Ast.VStr field;
          Pricklypear_lisp.Ast.VStr value ] ->
          Crud.update_field t.store ~type_name ~id ~field ~value
      | _ ->
          ((raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("update-field", "bad args (need
   type, id, field, value)", "bad-args"))))));

(* (delete 'type id) -> VStr id | VNil *)
register_crud "delete"
  (fun args ->
    let uname = actor () in
    if uname = "anonymous" then deny_anon "delete"
    else
      match args with
      | [ Pricklypear_lisp.Ast.VStr type_name;
          Pricklypear_lisp.Ast.VStr id ] ->
          Crud.delete t.store ~type_name ~id
      | _ ->
          (raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("delete", "bad args (need type,
   id)", "bad-args")))));

(* (select-for-update 'type id) -> VStr fields-json | VNil.
   Acquires an explicit SELECT ... FOR UPDATE row lock on the
   named row, for the duration of the surrounding transaction.
   Per spec [storage.transactions] (prim lands it17, 2026-07-06).
   See Crud.select_for_update doc for transactional-scoping caveat:
   outside a transaction, this is a hinted-get with no real lock. *)
register_crud "select-for-update"
  (fun args ->
    match args with
    | [ Pricklypear_lisp.Ast.VStr type_name;
        Pricklypear_lisp.Ast.VStr id ] ->
        Crud.select_for_update t.store ~type_name ~id
    | _ -> (raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("select-for-update", "bad args (need
   type, id)", "bad-args")))));

(* (rows 'type [limit]) -> VList of (id . fields) pairs.
   With no limit, defaults to 100. NOTE: prim name is `rows`, NOT
   `list` -- the latter is the pure VList-constructor prim. Earlier
   tentative naming collided and broke `(list 1 2 3)`; renamed to
   avoid that. *)
register_crud "rows"
  (fun args ->
    match args with
    | [ Pricklypear_lisp.Ast.VStr type_name ] ->
        Crud.list t.store ~type_name ()
    | [ Pricklypear_lisp.Ast.VStr type_name;
        Pricklypear_lisp.Ast.VInt n ] ->
        Crud.list t.store ~type_name ~limit:(Int64.to_int n) ()
    | [ Pricklypear_lisp.Ast.VStr type_name;
        Pricklypear_lisp.Ast.VStr sort ] ->
```

```
          Crud.list t.store ~type_name ~sort:(Some sort) ()
      | [ Pricklypear_lisp.Ast.VStr type_name;
          Pricklypear_lisp.Ast.VInt n;
          Pricklypear_lisp.Ast.VStr sort ] ->
          Crud.list t.store ~type_name ~limit:(Int64.to_int n) ~sort:(Some sort) ()
      | _ -> (raise (Pricklypear_lisp.Ast.Interp_error (Pricklypear_lisp.Ast.Prim_error ("rows", "bad args (need type[, limit][, sort])", "bad-args")))));

(* (find-row 'type "field" "value") -> fields JSON | () *)
register_crud "find-row"
  (fun args ->
    match args with
    | [ Pricklypear_lisp.Ast.VStr type_name;
        Pricklypear_lisp.Ast.VStr field;
        Pricklypear_lisp.Ast.VStr value ] ->
        Crud.find_row t.store ~type_name ~field ~value
    | _ ->
        ((raise (Pricklypear_lisp.Ast.Interp_error (Pricklypear_lisp.Ast.Prim_error ("find-row", "need type, field, value", "bad-args"))))));

(* (find-rows 'type "f" "v" ? [limit] [sort]) -> list of (id . fields) *)
register_crud "find-rows"
  (fun args ->
    let rec parse_filters acc = function
      | [] -> Ok (List.rev acc, 500, None)
      | [ Pricklypear_lisp.Ast.VInt n ] ->
          Ok (List.rev acc, Int64.to_int n, None)
      | [ Pricklypear_lisp.Ast.VStr s ] ->
          Ok (List.rev acc, 500, Some s)
      | [ Pricklypear_lisp.Ast.VInt n; Pricklypear_lisp.Ast.VStr s ] ->
          Ok (List.rev acc, Int64.to_int n, Some s)
      | Pricklypear_lisp.Ast.VStr f :: Pricklypear_lisp.Ast.VStr v :: rest ->
          parse_filters ((f, v) :: acc) rest
      | _ -> Error "find-rows: bad filter args"
    in
    match args with
    | Pricklypear_lisp.Ast.VStr type_name :: rest ->
        (match parse_filters [] rest with
         | Error e ->
             raise (Pricklypear_lisp.Ast.Interp_error
                      (Pricklypear_lisp.Ast.Prim_error
                         ("find-rows", e, "bad-args")))
         | Ok (filters, limit, sort) ->
             Crud.find_rows t.store ~type_name ~filters ~limit ~sort ())
    | _ ->
        ((raise (Pricklypear_lisp.Ast.Interp_error (Pricklypear_lisp.Ast.Prim_error ("find-rows", "need type and field/value pairs", "bad-args"))))));

(* (find-text 'type "field" "needle" [limit]) *)
register_crud "find-text"
  (fun args ->
    match args with
    | [ Pricklypear_lisp.Ast.VStr type_name;
        Pricklypear_lisp.Ast.VStr field;
        Pricklypear_lisp.Ast.VStr needle ] ->
        Crud.find_text t.store ~type_name ~field ~needle ()
    | [ Pricklypear_lisp.Ast.VStr type_name;
        Pricklypear_lisp.Ast.VStr field;
        Pricklypear_lisp.Ast.VStr needle;
        Pricklypear_lisp.Ast.VInt n ] ->
        Crud.find_text t.store ~type_name ~field ~needle
          ~limit:(Int64.to_int n) ()
    | _ ->
        ((raise (Pricklypear_lisp.Ast.Interp_error (Pricklypear_lisp.Ast.Prim_error ("find-text", "need type, field, needle [, limit]", "bad-args"))))));

(* (find-random 'type) -> (id . fields) | () *)
```

```
265   register_crud "find-random"
266     (fun args ->
267       match args with
268       | [ Pricklypear_lisp.Ast.VStr type_name ] ->
269           Crud.find_random t.store ~type_name
270       | _ ->
271           (raise (Pricklypear_lisp.Ast.Interp_error
      (Pricklypear_lisp.Ast.Prim_error ("find-random", "need type",
      "bad-args")))));
272
273   (* one-tree P6: typed nodes + path addressing + backlinks *)
274   register_crud "node-path"
275     (fun args ->
276       match args with
277       | [ Pricklypear_lisp.Ast.VStr type_name;
278           Pricklypear_lisp.Ast.VStr id ] ->
279           (Crud.node_path ~type_name ~id)
280       | _ ->
281           (raise (Pricklypear_lisp.Ast.Interp_error
      (Pricklypear_lisp.Ast.Prim_error ("node-path", "need type, id",
      "bad-args")))));
282
283   register_crud "type-fields"
284     (fun args ->
285       match args with
286       | [ Pricklypear_lisp.Ast.VStr type_name ] ->
287           Crud.type_fields t.store ~type_name
288       | _ ->
289           (raise (Pricklypear_lisp.Ast.Interp_error
      (Pricklypear_lisp.Ast.Prim_error ("type-fields", "need type",
      "bad-args")))));
290
291   register_crud "backlinks"
292     (fun args ->
293       match args with
294       | [ Pricklypear_lisp.Ast.VStr needle ] ->
295           Crud.backlinks t.store ~needle ()
296       | [ Pricklypear_lisp.Ast.VStr needle;
297           Pricklypear_lisp.Ast.VInt n ] ->
298           Crud.backlinks t.store ~needle ~limit:(Int64.to_int n) ()
299       | _ ->
300           (raise (Pricklypear_lisp.Ast.Interp_error
      (Pricklypear_lisp.Ast.Prim_error ("backlinks", "need id-or-path [,
      limit]", "bad-args")))));
301
302   (* Hierarchical tree prims (notes.tree-cutover K0). Direct tree_nodes
303      access for folder/note/heading -- not kind=datum CRUD. *)
304   register_crud "tree/get"
305     (fun args ->
306       match args with
307       | [ Pricklypear_lisp.Ast.VStr path ] ->
308           Crud.tree_get t.store ~path ()
309       | [ Pricklypear_lisp.Ast.VStr path
310         ; Pricklypear_lisp.Ast.VInt g
311         ] ->
312           (* reads-as-of-gen: optional :gen N -> versioned read at that gen
313              (tree-time.reads-as-of-gen). Omitted means head. *)
314           Crud.tree_get t.store ~path ~at_gen:g ()
315       | _ ->
316           (raise (Pricklypear_lisp.Ast.Interp_error
      (Pricklypear_lisp.Ast.Prim_error ("tree/get", "need path [, :gen]/",
      "bad-args")))));
317
318   register_crud "tree/list"
319     (fun args ->
320       match args with
321       | [ Pricklypear_lisp.Ast.VStr prefix ] ->
322           Crud.tree_list t.store ~prefix ()
323       | [ Pricklypear_lisp.Ast.VStr prefix; Pricklypear_lisp.Ast.VInt depth
      ] ->
324           Crud.tree_list t.store ~prefix ~depth:(Int64.to_int depth) ()
325       | [ Pricklypear_lisp.Ast.VStr prefix
326         ; Pricklypear_lisp.Ast.VInt depth
```

```
      ; Pricklypear_lisp.Ast.VInt lim
      ] ->
        Crud.tree_list t.store ~prefix ~depth:(Int64.to_int depth)
          ~limit:(Int64.to_int lim) ()
    | _ ->
        ((raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("tree/list", "need prefix [, depth
   [, limit]]", "bad-args"))))));

(* (tree/children-of path [limit]) -> VList of direct-child node JSON.
   one-tree.tree-read-prims: level-predicate children read over
   tree_nodes (never an anti-join; outline covering at +2 under label h).
   Anonymous reads allowed (resolved against system/main,
   prefix-confined). *)
register_crud "tree/children-of"
  (fun args ->
    match args with
    | [ Pricklypear_lisp.Ast.VStr path ] ->
        Crud.tree_children_of t.store ~path ()
    | [ Pricklypear_lisp.Ast.VStr path; Pricklypear_lisp.Ast.VInt lim ] ->
        Crud.tree_children_of t.store ~path ~limit:(Int64.to_int lim) ()
    | _ ->
        ((raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("tree/children-of", "need path [,
   limit]", "bad-args"))))));

(* (tree/subtree path [limit]) -> VList of all-descendant node JSON under
   path, capped at limit. one-tree.tree-read-prims. Anonymous allowed. *)
register_crud "tree/subtree"
  (fun args ->
    match args with
    | [ Pricklypear_lisp.Ast.VStr path ] ->
        Crud.tree_subtree t.store ~path ()
    | [ Pricklypear_lisp.Ast.VStr path; Pricklypear_lisp.Ast.VInt lim ] ->
        Crud.tree_subtree t.store ~path ~limit:(Int64.to_int lim) ()
    | _ ->
        ((raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("tree/subtree", "need path [,
   limit]", "bad-args"))))));

(* Lens-aware read prims (one-tree.tree-read-prims, BORGE-1785516993).
   Resolve a RELATIVE logical path (fn/type/route/pkg suffixes such as
   "fn.notes/doc/get", "pkg.todo", "route./issues") against the SESSION's
   branch lens chain -- self -> parent -> ... -> system/main,
   closest-wins,
   exactly like function_defs. Lets a session on <user>/personal see the
   EFFECTIVE override chain for tree_nodes (the store-tree kind the
   tree-browser v2 /sys + branch-lens roots need). Anonymous reads
   allowed (resolves against system/main via the anon lens). *)
register_crud "tree/lens-get"
  (fun args ->
    match args with
    | [ Pricklypear_lisp.Ast.VStr suffix ] ->
        Crud.tree_lens_get t.store ~suffix
    | _ ->
        ((raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("tree/lens-get", "need
   relative-suffix (e.g. \"fn.notes/doc/get\")", "bad-args"))))));

register_crud "tree/lens-children"
  (fun args ->
    let uname = actor () in
    ignore uname;
    match args with
    | [ Pricklypear_lisp.Ast.VStr suffix ] ->
        Crud.tree_lens_children t.store ~suffix ()
    | [ Pricklypear_lisp.Ast.VStr suffix
      ; Pricklypear_lisp.Ast.VInt lim ] ->
        Crud.tree_lens_children t.store ~suffix ~limit:(Int64.to_int lim)
   ()
    | _ ->
        ((raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("tree/lens-children", "need
```

```
   relative-suffix [, limit]", "bad-args"))))));

register_crud "tree/lens-subtree"
  (fun args ->
    match args with
    | [ Pricklypear_lisp.Ast.VStr suffix ] ->
        Crud.tree_lens_subtree t.store ~suffix ()
    | [ Pricklypear_lisp.Ast.VStr suffix
      ; Pricklypear_lisp.Ast.VInt lim ] ->
        Crud.tree_lens_subtree t.store ~suffix ~limit:(Int64.to_int lim)
   ()
    | _ ->
        ((raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("tree/lens-subtree", "need
   relative-suffix [, limit]", "bad-args"))))));

(* local-first (borge/local-first.borg lens-model + replication-units,
   campaign 8.1). lens/fork names + creates a lens branch as a NAMED CHILD
   of the checked-out branch per the pinned <owner>/lens-<device>[-<date>]
   contract (single-writer-per-lens, server stays parent). Non-anonymous
   only (an anonymous identity owns no lens).
   (lens/fork device-name [, date]) -> VStr "ok
   owner/lens-<device>[-<date>] (id)" *)
register_crud "lens/fork"
  (fun args ->
    let uname = actor () in
    if uname = "anonymous" then deny_anon "lens/fork"
    else
      match args with
      | [ Pricklypear_lisp.Ast.VStr device ] ->
          Crud.lens_fork t.store ~device ~date:None ()
      | [ Pricklypear_lisp.Ast.VStr device;
          Pricklypear_lisp.Ast.VStr date ] ->
          Crud.lens_fork t.store ~device ~date:(Some date) ()
      | _ ->
          ((raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("lens/fork", "need device-name [,
   date]", "bad-args"))))));

(* local-first replication-units (borge/local-first.borg).
   replication/unit
   materializes the exact authorized-read subtree the current identity
   would replicate: data.u.<user>.* + data.shared.* + the forked code lens
   + the branch ancestry it resolves through. Read-only; safe for any
   identity (resolves against its own lens -- anon sees data.u.anonymous +
   shared seed). Returns a JSON manifest with per-prefix node counts. *)
register_crud "replication/unit"
  (fun args ->
    match args with
    | [] -> Crud.replication_unit t.store ()
    | _ ->
        ((raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("replication/unit", "no arguments",
   "bad-args"))))));

(* local-first sync (borge/local-first.borg sync, campaign 8.3). The
   server-side merge authority. sync/plan takes a CLIENT (path, hash)
   manifest -- the exact shape Lens_store.node_manifest produces -- plus
   an optional three-way merge base (the (path, hash) manifest the
   lens last synced at), and returns the reconciliation: what the
   server would pull (ServerOnly), push (ClientOnly), or merge
   (Diverge / conflict). Pure classification over the server's own
   subtree hash manifest; read-only, safe for any identity (scoped to
   its own lens replication unit).

     (sync/plan '[("data.u....".x "<hash>") ...])
     (sync/plan client-manifest base-manifest)

   Returns a JSON object {same, client_only, server_only, diverge,
   client_only_paths, server_only_paths, diverge_paths} -- or, with a
   base, a three-way plan {rows:[{path, outcome, detail}]}. *)
let parse_manifest (v : Pricklypear_lisp.Ast.value) : Sync.manifest
   option =
```

```
    let parse_pair = function
      | Pricklypear_lisp.Ast.VList [ Pricklypear_lisp.Ast.VStr p;
                                     Pricklypear_lisp.Ast.VStr h ] -> Some (p, h)
      | _ -> None
    in
    match v with
    | Pricklypear_lisp.Ast.VList pairs ->
        let m = List.filter_map parse_pair pairs in
        if List.length m = List.length pairs then Some m else None
    | _ -> None
  in
  let sync_user_prefix () =
    let uname = actor () in
    "data.u." ^ Tree_store.enc_label uname
  in
  register_crud "sync/plan"
    (fun args ->
      match args with
      | [ cm; b ] ->
          (* three-way: client manifest + base (last-synced) manifest *)
          (match parse_manifest cm, parse_manifest b with
           | Some c, Some base ->
               let prefix = sync_user_prefix () in
               (match Sync.server_manifest t.store ~prefix with
                | None -> ((raise (Pricklypear_lisp.Ast.Interp_error (Pricklypear_lisp.Ast.Prim_error ("sync/plan", "no DB", "failed")))))
                | Some server ->
                    let plans = Sync.plan_of base server c in
                    (Pricklypear_lisp.Ast.VStr (Sync.json_of_plan plans)))
           | _ -> ((raise (Pricklypear_lisp.Ast.Interp_error (Pricklypear_lisp.Ast.Prim_error ("sync/plan", "manifests must be [[path, hash], ?]", "bad-args"))))))
      | [ cm ] ->
          (* two-way classify: client manifest vs server, no base *)
          (match parse_manifest cm with
           | Some c ->
               let prefix = sync_user_prefix () in
               (match Sync.server_manifest t.store ~prefix with
                | None -> ((raise (Pricklypear_lisp.Ast.Interp_error (Pricklypear_lisp.Ast.Prim_error ("sync/plan", "no DB", "failed")))))
                | Some server ->
                    let r = Sync.classify server c in
                    (Pricklypear_lisp.Ast.VStr (Sync.json_of_report r)))
           | None -> ((raise (Pricklypear_lisp.Ast.Interp_error (Pricklypear_lisp.Ast.Prim_error ("sync/plan", "manifest must be [[path, hash], ?]", "bad-args"))))))
      | _ -> ((raise (Pricklypear_lisp.Ast.Interp_error (Pricklypear_lisp.Ast.Prim_error ("sync/plan", "need client-manifest [, base-manifest]", "bad-args"))))));

  (* sync/push -- apply a client push for the caller's data subtree
     (borge/local-first.borg sync, v1 two-way + three-way / data-only).
     Takes the client (path, hash) manifest (same shape as sync/plan)
     plus a list of (path, envelope-json) pairs for the bodies the
     client holds, and an OPTIONAL third argument: the base manifest
     the client recorded at its last successful sync (the common
     ancestor -- the __base.json the client pins from the previous
     round's merged_manifest). The server classifies its own
     data.u.<user>.* manifest against the client's, then:

       - ClientOnly paths (server missing the node) are applied through
         the typed-boundary-validated datum write surface
         (Sync.apply_client_node).
       - Diverge/conflict paths (both sides hold different hashes) are
         LWW-ARBITRATED per sync-conflicts v0: the client envelope wins iff
         its updated_at (epoch, stamped by the lens substrate on every
         local put) is STRICTLY newer than the server node's; otherwise
         the server copy stays. EVERY resolution is surfaced with its
         winner (lww_client_won / lww_server_kept) -- the obituary rule
         applies to the losing side; nothing is silently dropped.
       - With a base, the FULL three-way plan (Sync.plan_of) runs:
         clean client edits apply (theirs_add/theirs_mod), client
```

```
         DELETIONS propagate (theirs_del), server deletions come back
         as client_delete_paths for the client to mirror locally, and
         conflicts get the same LWW arbitration.
       - ServerOnly paths are returned so the client can pull them.

    Returns a JSON object:
      {mode, applied, client_only, server_only, diverge,
       server_only_paths, diverge_paths, conflicts, lww_client_won,
       lww_server_kept, client_delete_paths, merged_manifest,
       generation}
    where merged_manifest is the server's post-apply subtree manifest --
    the (path, hash) list the client records as its NEXT round's base --
    and generation is the new (branch, generation) coordinate to pin.
    Code-lens sync (the b.<lens> code paths) is NOT in scope here - it
    is the branch-pointer move (promote/merge), a separate path; callers
    filter the manifest to data paths first.
      (sync/push client-manifest client-bodies [base-manifest])
    client-manifest: [[path, hash], ...]  (same as sync/plan)
    client-bodies:    [[path, envelope-json-str], ...]
    base-manifest:    [[path, hash], ...]  (client's last-synced state) *)
  register_crud "sync/push"
    (fun args ->
      (* [cm] client manifest; [bodies] client envelope bodies;
         [base] optional last-synced manifest -- three-way when present. *)
      let parse_bodies body_pairs =
        match body_pairs with
        | Pricklypear_lisp.Ast.VList pairs ->
            List.filter_map
              (function
               | Pricklypear_lisp.Ast.VList
                   [ Pricklypear_lisp.Ast.VStr p;
                     Pricklypear_lisp.Ast.VStr env ] ->
                   (try Some (p, Yojson.Safe.from_string env)
                    with _ -> None)
               | _ -> None)
              pairs
        | _ -> []
      in
      let lww_diverge ~server ~cb (diverge : (string * string) list) =
        (* sync-conflicts v0: LWW-arbitrate diverge pairs. Client strictly
           newer wins (applied); otherwise the server copy stays. Every
           resolution is surfaced with its winner. *)
        let client_paths = List.map fst diverge in
        let epochs = Sync.server_updated_at t.store client_paths in
        List.fold_left
          (fun (won, kept) (p, _) ->
            let cl =
              match List.assoc_opt p cb with
              | Some env -> Sync.envelope_updated_at env
              | None -> None
            in
            let client_wins =
              match List.assoc_opt p epochs with
              | Some sv -> Sync.lww_client_wins cl sv
              | None -> false
            in
            if client_wins then
              match List.assoc_opt p cb with
              | Some env -> (
                  match Sync.apply_client_node t.store ~env with
                  | Ok () -> (p :: won, kept)
                  | Error _ -> (won, p :: kept))
              | None -> (won, p :: kept)
            else (won, p :: kept))
          ([], []) diverge
      in
      match args with
      | [ cm; bodies ] ->
          (match parse_manifest cm, bodies with
           | Some c, _ ->
               let prefix = sync_user_prefix () in
               (match Sync.server_manifest t.store ~prefix with
                | None ->
```

```
                ((raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("sync/push", "no DB", "failed")))))
           | Some server ->
               let cb = parse_bodies bodies in
               let r = Sync.classify server c in
               (* 1. ClientOnly pushes (server missing the node) --
                  no arbitration needed, the server has no copy. *)
               let applied_co =
                 List.fold_left
                   (fun n (p, _) ->
                     match List.assoc_opt p cb with
                     | Some env ->
                         (match Sync.apply_client_node t.store ~env
    with
                          | Ok () -> n + 1
                          | Error _ -> n)
                     | None -> n)
                   0 r.client_only
               in
               (* 2. Diverge pairs LWW-arbitrate (sync-conflicts v0).
    *)
               let lww_won, lww_kept =
                 lww_diverge ~server ~cb r.diverge
               in
               let applied = applied_co + List.length lww_won in
               let bid = Store.current_branch_id t.store in
               let gen =
                 match Store.current_generation t.store ~branch_id:bid
    with
                 | Some n -> n
                 | None -> 0L
               in
               let json =
                 Yojson.Safe.to_string
                   (`Assoc
                      [ ("mode", `String "two-way")
                      ; ("applied", `Int applied)
                      ; ("client_only",
                         `Int (List.length r.client_only))
                      ; ("server_only",
                         `Int (List.length r.server_only))
                      ; ("diverge", `Int (List.length r.diverge))
                      ; ("server_only_paths",
                         `List (List.map
                                  (fun (p, _) -> `String p)
                                  (Sync.normalize r.server_only)))
                      ; ("diverge_paths",
                         `List (List.map
                                  (fun (p, _) -> `String p)
                                  (Sync.normalize r.diverge)))
                      ; ("lww_client_won",
                         `List (List.map (fun p -> `String p)
                                  (List.rev lww_won)))
                      ; ("lww_server_kept",
                         `List (List.map (fun p -> `String p)
                                  (List.rev lww_kept)))
                      ; ("generation",
                         `String (Int64.to_string gen))
                      ])
               in
               (Pricklypear_lisp.Ast.VStr json))
       | _ ->
           ((raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("sync/push", "args must be
    (client-manifest client-bodies)", "bad-args"))))))
     | [ cm; bodies; base ] ->
         (match parse_manifest cm, bodies, parse_manifest base with
          | Some c, _, Some base ->
              let prefix = sync_user_prefix () in
              (match Sync.server_manifest t.store ~prefix with
               | None ->
                   ((raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("sync/push", "no DB", "failed")))))
```

```
653                | Some server ->
654                    let cb = parse_bodies bodies in
655                    (* the full three-way: base = last-synced, ours =
656                       server, theirs = client. Clean client edits and
657                       deletions apply; conflicts LWW-arbitrate; server
658                       deletions come back as client_delete_paths. *)
659                    let plans = Sync.plan_of base server c in
660                    let report = Sync.commit t.store ~client_bodies:cb
    plans in
661                    let r = Sync.classify server c in
662                    let bid = Store.current_branch_id t.store in
663                    let gen =
664                      match Store.current_generation t.store ~branch_id:bid
    with
665                      | Some n -> n
666                      | None -> 0L
667                    in
668                    (* post-apply server subtree = the client's next
669                       round's base (the merged state both sides now
670                       hold -- or will, once the client mirrors the
671                       deletions and pulls). *)
672                    let merged =
673                      match Sync.server_manifest t.store ~prefix with
674                      | Some m -> m
675                      | None -> []
676                    in
677                    (* pulls = server-only (client missing) + conflicts
678                       the LWW kept on the server (client stale copy must
679                       converge to the winner or the pair re-conflicts
680                       every round). *)
681                    let pull =
682                      Sync.normalize
683                        (r.server_only
684                         @ List.map (fun p -> (p, ""))
    report.lww_server_kept)
685                    in
686                    let json =
687                      Yojson.Safe.to_string
688                        (`Assoc
689                           [ ("mode", `String "three-way")
690                           ; ("applied", `Int report.applied)
691                           ; ("client_only",
692                              `Int (List.length r.client_only))
693                           ; ("server_only",
694                              `Int (List.length r.server_only))
695                           ; ("diverge", `Int (List.length r.diverge))
696                           ; ("server_only_paths",
697                              `List (List.map
698                                       (fun (p, _) -> `String p)
699                                       (Sync.normalize r.server_only)))
700                           ; ("diverge_paths",
701                              `List (List.map
702                                       (fun (p, _) -> `String p)
703                                       (Sync.normalize r.diverge)))
704                           ; ("conflicts",
705                              `List (List.map
706                                       (fun (p, _, _) -> `String p)
707                                       (List.rev report.conflicts)))
708                           ; ("lww_client_won",
709                              `List (List.map (fun p -> `String p)
710                                       (List.rev report.lww_client_won)))
711                           ; ("lww_server_kept",
712                              `List (List.map (fun p -> `String p)
713                                       (List.rev report.lww_server_kept)))
714                           ; ("client_delete_paths",
715                              `List (List.map (fun p -> `String p)
716                                       (List.rev report.client_deletes)))
717                           ; ("pull_paths",
718                              `List (List.map (fun (p, _) -> `String p)
    pull))
719                           ; ("merged_manifest",
720                              `List (List.map
721                                       (fun (p, h) ->
```

```
                                      'List [ 'String p; 'String h ])
                                   (Sync.normalize merged)))
                          ; ("generation",
                             'String (Int64.to_string gen))
                          ])
                  in
                  (Pricklypear_lisp.Ast.VStr json))
          | _ ->
              ((raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("sync/push", "args must be
   (client-manifest client-bodies [base-manifest])", "bad-args"))))))
     | _ ->
         ((raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("sync/push", "need (client-manifest
   client-bodies [base-manifest])", "bad-args"))))));


(* data lens operations (one-tree.borg branching-interaction, 2026-08-23
   make: versioned user-keyed data). fork-data creates a deliberate
   fork of the user's live data into data.l.<lens>.<user>; merge-data
   merges a lens back (LWW v0); abandon-data drops a lens;
   list-data-lenses enumerates the user's lenses. *)
register_crud "fork-data"
  (fun args ->
    let uname = actor () in
    if uname = "anonymous" then deny_anon "fork-data"
    else
      match args with
      | [ Pricklypear_lisp.Ast.VStr lens ] ->
          Crud.fork_data t.store ~lens ()
      | [ Pricklypear_lisp.Ast.VStr lens;
          Pricklypear_lisp.Ast.VStr ty ] ->
          Crud.fork_data t.store ~lens ~type_name:(Some ty) ()
      | _ ->
          ((raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("fork-data", "need lens-name [,
   type-name]", "bad-args"))))));

register_crud "merge-data"
  (fun args ->
    let uname = actor () in
    if uname = "anonymous" then deny_anon "merge-data"
    else
      match args with
      | [ Pricklypear_lisp.Ast.VStr lens ] ->
          Crud.merge_data_lens t.store ~lens ()
      | _ ->
          ((raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("merge-data", "need lens-name",
   "bad-args"))))));

register_crud "abandon-data"
  (fun args ->
    let uname = actor () in
    if uname = "anonymous" then deny_anon "abandon-data"
    else
      match args with
      | [ Pricklypear_lisp.Ast.VStr lens ] ->
          Crud.abandon_data_lens t.store ~lens ()
      | _ ->
          ((raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("abandon-data", "need lens-name",
   "bad-args"))))));

register_crud "list-data-lenses"
  (fun _args ->
    let uname = actor () in
    if uname = "anonymous" then deny_anon "list-data-lenses"
    else Crud.list_data_lenses t.store ());
register_crud "tree/upsert"
  (fun args ->
    let uname = actor () in
    if uname = "anonymous" then deny_anon "tree/upsert"
```

```
      else
        match args with
        | [ Pricklypear_lisp.Ast.VStr path
          ; Pricklypear_lisp.Ast.VStr kind
          ; Pricklypear_lisp.Ast.VStr type_name
          ; Pricklypear_lisp.Ast.VStr body_json
          ] ->
            Crud.tree_upsert t.store ~path ~kind ~type_name ~body_json
              ~author:uname
        | _ ->
            ((raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("tree/upsert", "need path kind type
   body-json", "bad-args"))))));

  register_crud "tree/delete"
    (fun args ->
      let uname = actor () in
      if uname = "anonymous" then deny_anon "tree/delete"
      else
        match args with
        | [ Pricklypear_lisp.Ast.VStr path ] ->
            Crud.tree_delete t.store ~path ~recursive:false
        | [ Pricklypear_lisp.Ast.VStr path; Pricklypear_lisp.Ast.VBool rec_
   ] ->
            Crud.tree_delete t.store ~path ~recursive:rec_
        | [ Pricklypear_lisp.Ast.VStr path; Pricklypear_lisp.Ast.VStr flag ]
          when flag = "recursive" || flag = "r" || flag = "#t" ->
            Crud.tree_delete t.store ~path ~recursive:true
        | _ ->
            ((raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("tree/delete", "need path [,
   recursive]", "bad-args"))))));

  register_crud "tree/move"
    (fun args ->
      let uname = actor () in
      if uname = "anonymous" then deny_anon "tree/move"
      else
        match args with
        | [ Pricklypear_lisp.Ast.VStr old_path
          ; Pricklypear_lisp.Ast.VStr new_path
          ] ->
            Crud.tree_move t.store ~old_path ~new_path
        | _ ->
            (raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("tree/move", "need old-path
   new-path", "bad-args")))));

  (* Sibling-order prims (one-tree.sibling-order). Rank = canonical base-36
     fractional string over [0-9a-z] (not ending in '0'); NULL rank sorts
     last, path-ordered. Mutations deny_anon; reads open. *)
  register_crud "tree/reorder"
    (fun args ->
      let uname = actor () in
      if uname = "anonymous" then deny_anon "tree/reorder"
      else
        match args with
        | [ Pricklypear_lisp.Ast.VStr path; Pricklypear_lisp.Ast.VStr rank
   ] ->
            Crud.tree_reorder t.store ~path ~rank
        | _ ->
            (raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("tree/reorder", "need path rank
   (\"\" clears)", "bad-args")))));

  register_crud "tree/move-before"
    (fun args ->
      let uname = actor () in
      if uname = "anonymous" then deny_anon "tree/move-before"
      else
        match args with
        | [ Pricklypear_lisp.Ast.VStr path; Pricklypear_lisp.Ast.VStr
   anchor ] ->
```

```
            Crud.tree_move_before t.store ~path ~anchor
        | _ ->
            (raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("tree/move-before", "need path
   anchor (siblings)", "bad-args")))));

  register_crud "tree/move-after"
    (fun args ->
      let uname = actor () in
      if uname = "anonymous" then deny_anon "tree/move-after"
      else
        match args with
        | [ Pricklypear_lisp.Ast.VStr path; Pricklypear_lisp.Ast.VStr
   anchor ] ->
            Crud.tree_move_after t.store ~path ~anchor
        | _ ->
            (raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("tree/move-after", "need path anchor
   (siblings)", "bad-args")))));

  register_crud "tree/siblings"
    (fun args ->
      match args with
      | [ Pricklypear_lisp.Ast.VStr path ] ->
          Crud.tree_siblings t.store ~path
      | _ ->
          (raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("tree/siblings", "need path",
   "bad-args")))));

  register_crud "tree/next"
    (fun args ->
      match args with
      | [ Pricklypear_lisp.Ast.VStr path ] -> Crud.tree_next t.store ~path
      | _ -> (raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("tree/next", "need path",
   "bad-args")))));

  register_crud "tree/prev"
    (fun args ->
      match args with
      | [ Pricklypear_lisp.Ast.VStr path ] -> Crud.tree_prev t.store ~path
      | _ -> (raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("tree/prev", "need path",
   "bad-args")))));

  register_crud "tree/insert-child"
    (fun args ->
      let uname = actor () in
      if uname = "anonymous" then deny_anon "tree/insert-child"
      else
        match args with
        | [ Pricklypear_lisp.Ast.VStr parent
          ; Pricklypear_lisp.Ast.VStr kind
          ; Pricklypear_lisp.Ast.VStr type_name
          ; Pricklypear_lisp.Ast.VStr body_json
          ] ->
            Crud.tree_insert_child t.store ~parent ~kind ~type_name
              ~body_json ~anchor:None ~author:uname
        | [ Pricklypear_lisp.Ast.VStr parent
          ; Pricklypear_lisp.Ast.VStr kind
          ; Pricklypear_lisp.Ast.VStr type_name
          ; Pricklypear_lisp.Ast.VStr body_json
          ; Pricklypear_lisp.Ast.VStr anchor
          ] ->
            Crud.tree_insert_child t.store ~parent ~kind ~type_name
              ~body_json ~anchor:(Some anchor) ~author:uname
        | _ ->
            (raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("tree/insert-child", "need parent
   kind type body [, after-anchor]", "bad-args")))));

  (* Tree-edges prims (one-tree.tree-edges). An edge IS a node: kind=link
```

```
    at <source-path>.lnk.<uuid>, body {to, kind, label?}. "from" is
    positional; to is stored verbatim and MAY dangle (resolution is a
    read-time act against the reader's lens). Mutations deny_anon. *)
register_crud "link"
  (fun args ->
    let uname = actor () in
    if uname = "anonymous" then deny_anon "link"
    else
      match args with
      | [ Pricklypear_lisp.Ast.VStr from_p; Pricklypear_lisp.Ast.VStr
  to_p ] ->
          Crud.link_edge t.store ~from_path:from_p ~to_path:to_p
            ~edge_kind:"ref" ~label:None ~author:uname
      | [ Pricklypear_lisp.Ast.VStr from_p
        ; Pricklypear_lisp.Ast.VStr to_p
        ; Pricklypear_lisp.Ast.VStr edge_kind
        ] ->
          Crud.link_edge t.store ~from_path:from_p ~to_path:to_p
            ~edge_kind ~label:None ~author:uname
      | [ Pricklypear_lisp.Ast.VStr from_p
        ; Pricklypear_lisp.Ast.VStr to_p
        ; Pricklypear_lisp.Ast.VStr edge_kind
        ; Pricklypear_lisp.Ast.VStr label
        ] ->
          Crud.link_edge t.store ~from_path:from_p ~to_path:to_p
            ~edge_kind ~label:(Some label) ~author:uname
      | _ ->
          (raise (Pricklypear_lisp.Ast.Interp_error
  (Pricklypear_lisp.Ast.Prim_error ("link", "need from to [, kind [,
  label]]", "bad-args")))));

register_crud "unlink"
  (fun args ->
    let uname = actor () in
    if uname = "anonymous" then deny_anon "unlink"
    else
      match args with
      | [ Pricklypear_lisp.Ast.VStr edge_path ] ->
          Crud.unlink_edge t.store ~edge_path
      | _ ->
          (raise (Pricklypear_lisp.Ast.Interp_error
  (Pricklypear_lisp.Ast.Prim_error ("unlink", "need edge-path",
  "bad-args")))));

register_crud "out-links"
  (fun args ->
    match args with
    | [ Pricklypear_lisp.Ast.VStr path ] ->
        Crud.out_links t.store ~path ()
    | [ Pricklypear_lisp.Ast.VStr path; Pricklypear_lisp.Ast.VInt lim ] ->
        Crud.out_links t.store ~path ~limit:(Int64.to_int lim) ()
    | _ ->
        (raise (Pricklypear_lisp.Ast.Interp_error
  (Pricklypear_lisp.Ast.Prim_error ("out-links", "need path [, limit]",
  "bad-args")))));

register_crud "tree/get-uuid"
  (fun args ->
    match args with
    | [ Pricklypear_lisp.Ast.VStr uuid ] ->
        Crud.tree_get_uuid t.store ~uuid
    | _ ->
        (raise (Pricklypear_lisp.Ast.Interp_error
  (Pricklypear_lisp.Ast.Prim_error ("tree/get-uuid", "need uuid",
  "bad-args")))));

register_crud "tree/find-by"
  (fun args ->
    match args with
    | [ Pricklypear_lisp.Ast.VStr kind
      ; Pricklypear_lisp.Ast.VStr field
      ; Pricklypear_lisp.Ast.VStr value
      ] ->
```

```
          Crud.tree_find_by t.store ~kind:(Some kind) ~field ~value ()
      | [ Pricklypear_lisp.Ast.VStr kind
        ; Pricklypear_lisp.Ast.VStr field
        ; Pricklypear_lisp.Ast.VStr value
        ; Pricklypear_lisp.Ast.VInt n
        ] ->
          Crud.tree_find_by t.store ~kind:(Some kind) ~field ~value
            ~limit:(Int64.to_int n) ()
      | _ ->
          ((raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("tree/find-by", "need kind field
    value [, limit]", "bad-args"))))));

  register_crud "tree/list-kind"
    (fun args ->
      match args with
      | [ Pricklypear_lisp.Ast.VStr prefix; Pricklypear_lisp.Ast.VStr kind
    ] ->
          Crud.tree_list_kind t.store ~prefix ~kind ()
      | [ Pricklypear_lisp.Ast.VStr prefix
        ; Pricklypear_lisp.Ast.VStr kind
        ; Pricklypear_lisp.Ast.VInt n
        ] ->
          Crud.tree_list_kind t.store ~prefix ~kind ~limit:(Int64.to_int n)
    ()
      | _ ->
          ((raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("tree/list-kind", "need prefix kind
    [, limit]", "bad-args"))))));

  register_crud "tree/note-index"
    (fun args ->
      match args with
      | [] ->
          Crud.tree_note_index t.store
            ~prefix:(Tree_store.garden_root ~user:(actor ())) ()
      | [ Pricklypear_lisp.Ast.VStr prefix ] ->
          Crud.tree_note_index t.store ~prefix ()
      | [ Pricklypear_lisp.Ast.VStr prefix; Pricklypear_lisp.Ast.VInt n ] ->
          Crud.tree_note_index t.store ~prefix ~limit:(Int64.to_int n) ()
      | _ ->
          ((raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("tree/note-index", "[] | prefix [,
    limit]", "bad-args"))))));

  register_crud "tree/folder-index"
    (fun args ->
      match args with
      | [] ->
          Crud.tree_folder_index t.store
            ~prefix:(Tree_store.garden_root ~user:(actor ())) ()
      | [ Pricklypear_lisp.Ast.VStr prefix ] ->
          Crud.tree_folder_index t.store ~prefix ()
      | _ ->
          ((raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("tree/folder-index", "[] | prefix",
    "bad-args"))))));

  (* (tree/children-html [prefix]) -> HTML string of notes-tree children.
     OCaml-side so large folders (3k+) do not blow the eval budget.
     Caller wraps with (render-html ?) for raw emission. *)
  register_crud "tree/children-html"
    (fun args ->
      let garden = Tree_store.garden_root ~user:(actor ()) in
      let prefix =
        match args with
        | [] -> garden
        | [ Pricklypear_lisp.Ast.VStr p ] -> p
        | _ -> ""
      in
      if prefix = "" then
        ((raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("tree/children-html", "[] | prefix",
```

```
    "bad-args")))))
     else
       Crud.tree_children_html t.store ~prefix ~garden_root:garden ());

(* (tree/search-text needle [limit]) or (tree/search-text prefix needle [limit])
   -> list of hit JSON (slug/title/node_id/header_path/matched/rank).
   Default prefix = current garden root. ILIKE over note title/tags/slug and
   heading title/body -- notes.search-modal (BORGE-1785605474). *)
register_crud "tree/search-text"
  (fun args ->
    let run ~prefix ~needle ~limit =
      Crud.tree_search_text t.store ~prefix ~needle ~limit ()
    in
    let garden () = Tree_store.garden_root ~user:(actor ()) in
    match args with
    | [ Pricklypear_lisp.Ast.VStr needle ] ->
        run ~prefix:(garden ()) ~needle ~limit:20
    | [ Pricklypear_lisp.Ast.VStr needle; Pricklypear_lisp.Ast.VInt n ] ->
        run ~prefix:(garden ()) ~needle ~limit:(Int64.to_int n)
    | [ Pricklypear_lisp.Ast.VStr prefix; Pricklypear_lisp.Ast.VStr needle ] ->
        run ~prefix ~needle ~limit:20
    | [ Pricklypear_lisp.Ast.VStr prefix
      ; Pricklypear_lisp.Ast.VStr needle
      ; Pricklypear_lisp.Ast.VInt n
      ] ->
        run ~prefix ~needle ~limit:(Int64.to_int n)
    | _ ->
        ((raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("tree/search-text", "needle [, limit] | prefix needle [, limit]", "bad-args"))))));

(* Pure path helpers (no DB). *)
register_crud "tree/path-label"
  (fun args ->
    match args with
    | [ Pricklypear_lisp.Ast.VStr s ] ->
        (Crud.tree_path_label s)
    | _ ->
        (raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("tree/path-label", "need string", "bad-args")))));

(* Kernel path algebra (one-tree.tree-path-prims): one canonical set of
   pure path constructors/accessors, replacing per-package string hacks
   (notes/tree/*, todo/tree/*, tree/breadcrumb-parts). No DB, no auth;
   label encoding on append is identical to tree/path-label except that
   labels which are ALREADY valid ltree labels pass through untouched
   (round-trip: (tree/child (tree/parent p) (tree/leaf p)) = p).
   Invalid paths raise Prim_error; a root path has nil parent and no
   strict ancestors. *)
register_crud "tree/parent"
  (fun args ->
    match args with
    | [ Pricklypear_lisp.Ast.VStr path ] -> Crud.tree_parent path
    | _ ->
        (raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("tree/parent", "need path", "bad-args")))));

register_crud "tree/leaf"
  (fun args ->
    match args with
    | [ Pricklypear_lisp.Ast.VStr path ] -> Crud.tree_leaf path
    | _ ->
        (raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("tree/leaf", "need path", "bad-args")))));

register_crud "tree/segments"
  (fun args ->
```

```
      match args with
      | [ Pricklypear_lisp.Ast.VStr path ] -> Crud.tree_segments path
      | _ ->
          (raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("tree/segments", "need path",
    "bad-args")))));

  register_crud "tree/child"
    (fun args ->
      match args with
      | [ Pricklypear_lisp.Ast.VStr path; Pricklypear_lisp.Ast.VStr label ]
    ->
          Crud.tree_child ~path ~label
      | _ ->
          (raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("tree/child", "need path label",
    "bad-args")))));

  register_crud "tree/ancestors"
    (fun args ->
      match args with
      | [ Pricklypear_lisp.Ast.VStr path ] -> Crud.tree_ancestors path
      | _ ->
          (raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("tree/ancestors", "need path",
    "bad-args")))));

  register_crud "tree/garden-root"
    (fun args ->
      match args with
      | [] ->
          (Crud.tree_garden_root ~user:(actor ()))
      | [ Pricklypear_lisp.Ast.VStr user ] ->
          (Crud.tree_garden_root ~user)
      | _ ->
          ((raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("tree/garden-root", "[] or user",
    "bad-args"))))));

  (* (list-branches) -> VList of "owner/name" strings *)
  Primitives.register tbl "list-branches"
    (fun _args ->
      let bs = Store.list_branches t.store in
      let names =
        List.map
          (fun (b : Pricklypear_common.Branch.t) ->
            Pricklypear_lisp.Ast.VStr
              (b.Pricklypear_common.Branch.owner ^ "/" ^
               b.Pricklypear_common.Branch.name))
          bs
      in
      (Pricklypear_lisp.Ast.VList names));

  (* (gen-now) -> current tree gen (int). Spec: tree-time.reads-as-of-gen.
     Returns the current branch's generation (migration 017 ancestry-walked
     counter) = the code-path head gen coordinate for this session's branch
     (what a :gen read-as-of coordinate on code paths compares against). A
     write anywhere in the branch's ancestry bumps it, so a descendant sees
     its own writes via inheritance, consistent with how a session resolves
     code-path tree nodes. Unknown/unreachable -> 0 (caller treats as
     "at head / reload"). Pure read, no gen allocation. *)
  Primitives.register tbl "gen-now"
    (fun _args ->
      let bid = Store.current_branch_id t.store in
      let g =
        match Store.current_generation t.store ~branch_id:bid with
        | Some n -> n
        | None -> 0L
      in
      (Pricklypear_lisp.Ast.VInt g));

  (* (gen-read path gen) -> node body at gen, crossed over the same branch
     lens rules head reads use (tree-time.reads-as-of-gen). path may be an
```

```
1158      absolute b.*/data.* path or a RELATIVE logical suffix (kind.name)
1159      resolved closest-wins through the branch lens chain. gen is the as-of
1160      coordinate; 0/omitted means head. Returns version JSON (the
          version_to_json
1161      surface: id/path/kind/type/body/author/hash/created_at/gen/tombstone)
          or
1162      nil when the path is unversioned at that gen. ReadDb cap. *)
1163    register_crud "gen-read"
1164      (fun args ->
1165       match args with
1166       | [ Pricklypear_lisp.Ast.VStr path
1167         ; Pricklypear_lisp.Ast.VInt g
1168         ] ->
1169           Crud.tree_gen_read t.store ~path ~gen:g
1170       | _ ->
1171           (raise (Pricklypear_lisp.Ast.Interp_error
1172                     (Pricklypear_lisp.Ast.Prim_error
1173                        ("gen-read", "need path gen", "bad-args")))));
1174
1175    (* (gen-diff path gen-a gen-b) -> node-granular diff: reads both leaves
1176       via the versioned store (hashes first, walk on mismatch) and returns
1177       "same" when hash-equal, else a JSON {path,a,b,same,delta}. ReadDb. *)
1178    register_crud "gen-diff"
1179      (fun args ->
1180       match args with
1181       | [ Pricklypear_lisp.Ast.VStr path
1182         ; Pricklypear_lisp.Ast.VInt ga
1183         ; Pricklypear_lisp.Ast.VInt gb
1184         ] ->
1185           Crud.tree_gen_diff t.store ~path ~gen_a:ga ~gen_b:gb
1186       | _ ->
1187           (raise (Pricklypear_lisp.Ast.Interp_error
1188                     (Pricklypear_lisp.Ast.Prim_error
1189                        ("gen-diff",
1190                         "need path gen-a gen-b", "bad-args")))));
1191
1192    (* (undo path [n]) -> restore value at head-n as a NEW write at head+1
1193       (restore-by-write, so the undo itself is versioned and redoable).
1194       Supports datum paths (data-prefixed; the documents' first consumer is
          notes
1195       undo). Code paths (b-prefixed) report the code-restore consumer
          deferral.
1196       WriteDb cap (it appends a version row). Returns the written path or an
1197       error string. *)
1198    register_crud "undo"
1199      (fun args ->
1200       match args with
1201       | [ Pricklypear_lisp.Ast.VStr path ] ->
1202           Crud.tree_undo t.store ~path ()
1203       | [ Pricklypear_lisp.Ast.VStr path
1204         ; Pricklypear_lisp.Ast.VInt n
1205         ] ->
1206           Crud.tree_undo t.store ~path ~n ()
1207       | _ ->
1208           (raise (Pricklypear_lisp.Ast.Interp_error
1209                     (Pricklypear_lisp.Ast.Prim_error
1210                        ("undo", "need path [, n]", "bad-args")))));
1211
1212    (* (code-restore name [n] [owner/branch]) -> restore a code node (fn/type/
1213       route) at head-n as a NEW write at head+1 (restore-by-write). The
1214       code-path analogue of (undo) (which is datum-path only): gen-read +
          write
1215       the same value back via the code write surface (upsert_fn /
          upsert_type /
1216       upsert_route), so the reverse mirror re-populates function_defs /
1217       type_defs / view_bindings and the restore itself is versioned +
          redoable
1218       (the restored value becomes the new head). n default 1 = the state just
1219       before the latest change. Clean error strings (never a crash). WriteDb
1220       cap. Consumers: tree-time.consumers -- code-restore = gen-read +
          define. *)
1221    register_crud "code-restore"
1222      (fun args ->
```

```
      let resolve_branch_id ref_opt =
        match ref_opt with
        | None -> Store.current_branch_id t.store
        | Some branch_ref ->
            let owner, name =
              match String.split_on_char '/' branch_ref with
              | [ o; n ] when o <> "" && n <> "" -> (o, n)
              | _ -> ("", branch_ref)
            in
            (match Store.lookup_branch_id_by_name t.store ~owner ~name with
             | Some id -> id
             | None -> "")
      in
      let err = "code-restore: need name [n] [owner/branch]" in
      let bad_branch ref_opt bid =
        match ref_opt with
        | None -> (Pricklypear_lisp.Ast.VStr err)
        | Some branch_ref ->
            (Pricklypear_lisp.Ast.VStr
               (Printf.sprintf "code-restore: no such branch: %s"
    branch_ref))
      in
      let default_author = t.manifest.username in
      match args with
      | [ Pricklypear_lisp.Ast.VStr name ] ->
          let bid = resolve_branch_id None in
          if bid = "" then bad_branch None bid
          else
            (Pricklypear_lisp.Ast.VStr
               (Store.code_restore t.store ~branch_id:bid ~name
                  ~author:default_author ()))
      | [ Pricklypear_lisp.Ast.VStr name; Pricklypear_lisp.Ast.VInt n64 ] ->
          let bid = resolve_branch_id None in
          if bid = "" then bad_branch None bid
          else
            (Pricklypear_lisp.Ast.VStr
               (Store.code_restore t.store ~branch_id:bid ~name
                  ~author:default_author ~n:(Int64.to_int n64) ()))
      | [ Pricklypear_lisp.Ast.VStr name; Pricklypear_lisp.Ast.VStr
    branch_ref
        ] ->
          let bid = resolve_branch_id (Some branch_ref) in
          if bid = "" then bad_branch (Some branch_ref) bid
          else
            (Pricklypear_lisp.Ast.VStr
               (Store.code_restore t.store ~branch_id:bid ~name
                  ~author:default_author ()))
      | [ Pricklypear_lisp.Ast.VStr name; Pricklypear_lisp.Ast.VInt n64
        ; Pricklypear_lisp.Ast.VStr branch_ref
        ] ->
          let bid = resolve_branch_id (Some branch_ref) in
          if bid = "" then bad_branch (Some branch_ref) bid
          else
            (Pricklypear_lisp.Ast.VStr
               (Store.code_restore t.store ~branch_id:bid ~name
                  ~author:default_author ~n:(Int64.to_int n64) ()))
      | _ ->
          (raise (Pricklypear_lisp.Ast.Interp_error
                    (Pricklypear_lisp.Ast.Prim_error
                       ("code-restore", err, "bad-args")))));

  (* (list-usernames) -> VList of VStr; any authenticated user.
     Usernames are public identifiers on the box (multi-user.chat DMs). *)
  Primitives.register tbl "list-usernames"
    (fun _args ->
      if t.manifest.username = "anonymous" then
        (raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("list-usernames", "denied for
    anonymous", "denied"))))
      else
        let users = Auth_store.list_users t.store in
        let names =
          List.map
```

```
            (fun (u : Auth_store.user) ->
              Pricklypear_lisp.Ast.VStr u.Auth_store.username)
            users
        in
        (Pricklypear_lisp.Ast.VList names));

  (* Admin gating for Lisp auth surface (auth.user-management /
     auth.bootstrap-admin). Prefer the per-request snapshot username
     and re-check users.is_admin in PG so a concurrent attach cannot
     flip t.is_admin under an in-flight /api/eval. *)
  let caller_username () =
    match Request_context.get () with
    | Some c -> (Request_context.manifest c).username
    | None -> t.manifest.username
  in
  let require_admin (op : string) (k : unit -> Pricklypear_lisp.Ast.value)
    : Pricklypear_lisp.Ast.value =
    let uname = caller_username () in
    if uname = "anonymous" then
      (Pricklypear_lisp.Ast.VStr (op ^ ": denied for anonymous"))
    else
      let u = Auth_store.get_user_by_username t.store uname in
      match u with
      | Some u when u.Auth_store.is_admin -> k ()
      | _ ->
          (Pricklypear_lisp.Ast.VStr (op ^ ": denied: admin only"))
  in

  (* (is-admin?) -> VBool. True iff the calling identity has users.is_admin.
     Option 3 (BORGE-1785213238): reads Auth_store directly -- never
     Env.lookup
     of the name, so a user-defined shadow cannot grant admin. *)
  Primitives.register_server_only tbl "is-admin?"
    (fun _args ->
      let uname = caller_username () in
      if uname = "anonymous" then
        (Pricklypear_lisp.Ast.VBool false)
      else
        let u = Auth_store.get_user_by_username t.store uname in
        match u with
        | Some u -> (Pricklypear_lisp.Ast.VBool u.Auth_store.is_admin)
        | None -> (Pricklypear_lisp.Ast.VBool false));

  (* ---- tree-time.gc (5.3): retention worker prims (admin-gated)
     ----------
     Policy as HABITAT DATA: retention-policy-set / retention-delete-policy
     /
     retention-policies manage tree_retention_policy (migration 022);
     retention-run drives the worker (rollup + DROP PARTITION) per path
     class.
     Administrative surface -- it controls whole-habitat storage/retention.
     *)
  let retention_admin op k =
    if t.manifest.username = "anonymous" then
      (Pricklypear_lisp.Ast.VStr (op ^ ": denied for anonymous"))
    else
      require_admin op (fun () -> k ())
  in
  (* (retention-policy-set path_class keep_windows) -- whole-class policy.
     (retention-policy-set path_class type_name keep_windows) -- per-type
     (recorded for future finer-grained work; the shared gen-window floor is
     still whole-class). *)
  Primitives.register_server_only tbl "retention-policy-set"
    (fun args ->
      retention_admin "retention-policy-set" (fun () ->
          match args with
          | [ Pricklypear_lisp.Ast.VStr pc; Pricklypear_lisp.Ast.VInt kw ]
     ->
              Crud.retention_set_policy t.store ~path_class:pc
                ~type_name:None ~keep_windows:kw ()
          | [ Pricklypear_lisp.Ast.VStr pc; Pricklypear_lisp.Ast.VStr tn
            ; Pricklypear_lisp.Ast.VInt kw
            ] ->
```

```
            Crud.retention_set_policy t.store ~path_class:pc
              ~type_name:(Some tn) ~keep_windows:kw ()
        | _ ->
            (raise
               (Pricklypear_lisp.Ast.Interp_error
                  (Pricklypear_lisp.Ast.Prim_error
                     ("retention-policy-set",
                      "need path_class keep_windows [type_name]",
                      "bad-args"))))));
(* (retention-delete-policy path_class [type_name]) *)
Primitives.register_server_only tbl "retention-delete-policy"
  (fun args ->
    retention_admin "retention-delete-policy" (fun () ->
        match args with
        | [ Pricklypear_lisp.Ast.VStr pc ] ->
            Crud.retention_delete_policy t.store ~path_class:pc
              ~type_name:None ()
        | [ Pricklypear_lisp.Ast.VStr pc; Pricklypear_lisp.Ast.VStr tn ]
   ->
            Crud.retention_delete_policy t.store ~path_class:pc
              ~type_name:(Some tn) ()
        | _ ->
            (raise
               (Pricklypear_lisp.Ast.Interp_error
                  (Pricklypear_lisp.Ast.Prim_error
                     ("retention-delete-policy", "need path_class
   [type_name]",
                      "bad-args"))))));
(* (retention-policies) -> VList of policy JSON *)
Primitives.register_server_only tbl "retention-policies"
  (fun args ->
    retention_admin "retention-policies" (fun () ->
        match args with
        | [] -> Crud.retention_policies t.store ()
        | _ ->
            (raise
               (Pricklypear_lisp.Ast.Interp_error
                  (Pricklypear_lisp.Ast.Prim_error
                     ("retention-policies", "need no args",
   "bad-args"))))));
(* (retention-run [path_class]) -> VList of (path_class, dropped, ckptd)
   JSON.
   Runs only for path classes with an explicit whole-class policy;
   otherwise
   a keep-everything no-op. *)
Primitives.register_server_only tbl "retention-run"
  (fun args ->
    retention_admin "retention-run" (fun () ->
        match args with
        | [] -> Crud.retention_run t.store ~path_class:None ()
        | [ Pricklypear_lisp.Ast.VStr pc ] ->
            Crud.retention_run t.store ~path_class:(Some pc) ()
        | _ ->
            (raise
               (Pricklypear_lisp.Ast.Interp_error
                  (Pricklypear_lisp.Ast.Prim_error
                     ("retention-run", "need [path_class]",
   "bad-args"))))));

(* (deftype 'name 'field1 ...) -- admin only (BORGE-1785213343).
   Schema authority lives with main/admin; garden authors must not
   overwrite type_defs. Anonymous still raises (deny_anon). *)
register_crud "deftype"
  (fun args ->
    let uname = caller_username () in
    if uname = "anonymous" then deny_anon "deftype"
    else
      require_admin "deftype" (fun () ->
          Crud.deftype t.store ~args ~author:uname));

(* (create-user username password [is-admin]) -> VStr new-user-id | error
   VStr.
```

```
1424      Admin only. Mirrors POST /admin/users action=create (create + set
          password
1425      + auth_log admin_create_user). Optional third arg: bool / truthy
          str/int. *)
1426    Primitives.register_server_only tbl "create-user"
1427      (fun args ->
1428       require_admin "create-user" (fun () ->
1429         let parse_is_admin = function
1430           | Pricklypear_lisp.Ast.VBool b -> b
1431           | Pricklypear_lisp.Ast.VStr s ->
1432               (match String.lowercase_ascii (String.trim s) with
1433                | "1" | "true" | "yes" | "on" -> true
1434                | _ -> false)
1435           | Pricklypear_lisp.Ast.VInt n -> n <> 0L
1436           | _ -> false
1437         in
1438         let run ~username ~password ~is_admin =
1439           if String.trim username = "" || password = "" then
1440             ((raise (Pricklypear_lisp.Ast.Interp_error
          (Pricklypear_lisp.Ast.Prim_error ("create-user", "username and
          password required", "bad-args")))))
1441           else
1442             let created =
1443               Auth_store.create_user t.store ~username ~is_admin
1444             in
1445             match created with
1446             | Error e ->
1447                 ((raise (Pricklypear_lisp.Ast.Interp_error
          (Pricklypear_lisp.Ast.Prim_error ("create-user", e, "failed")))))
1448             | Ok user ->
1449                 let pw =
1450                   Auth_store.set_password t.store
1451                     ~user_id:user.Auth_store.id ~password
1452                 in
1453                 (match pw with
1454                  | Error e ->
1455                      ((raise (Pricklypear_lisp.Ast.Interp_error
          (Pricklypear_lisp.Ast.Prim_error ("create-user", "password: " ^ e,
          "failed")))))
1456                  | Ok () ->
1457                      let () =
1458                        Auth_store.log_auth t.store
1459                          ~user_id:user.Auth_store.id
1460                          ~kind:"admin_create_user"
1461                          ~success:true ()
1462                      in
1463                      (Pricklypear_lisp.Ast.VStr user.Auth_store.id))
1464         in
1465         match args with
1466         | [ Pricklypear_lisp.Ast.VStr username;
1467             Pricklypear_lisp.Ast.VStr password ] ->
1468             run ~username ~password ~is_admin:false
1469         | [ Pricklypear_lisp.Ast.VStr username;
1470             Pricklypear_lisp.Ast.VStr password;
1471             flag ] ->
1472             run ~username ~password ~is_admin:(parse_is_admin flag)
1473         | _ ->
1474             ((raise (Pricklypear_lisp.Ast.Interp_error
          (Pricklypear_lisp.Ast.Prim_error ("create-user", "need username
          password [is-admin]", "bad-args")))))));
1475
1476    (* (create-Agent_tools.agent parent-username) -> VStr
          Agent_tools.agent-user-id. Admin only. *)
1477    Primitives.register tbl "create-Agent_tools.agent"
1478      (fun args ->
1479       require_admin "create-Agent_tools.agent" (fun () ->
1480         match args with
1481         | [ Pricklypear_lisp.Ast.VStr parent ] ->
1482             let r = Auth_store.create_agent t.store ~parent_username:parent
          in
1483             (match r with
1484              | Error e ->
```

```
                (Pricklypear_lisp.Ast.VStr ("create-Agent_tools.agent: " ^ e))
          | Ok u ->
              (Pricklypear_lisp.Ast.VStr u.Auth_store.id))
      | _ ->
          (raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("prim", "create-Agent_tools.agent: need parent-username", "bad-args"))))));

  (* (issue-token [label]) -> VStr raw-token (returned ONCE). Caller's own user. *)
  (* (issue-token user-id [label]) -> VStr raw token. Admin only when user-id given. *)
  Primitives.register tbl "issue-token"
    (fun args ->
      let caller = caller_username () in
      let parse_label = function
        | Pricklypear_lisp.Ast.VStr s when s <> "" -> Some s
        | _ -> None
      in
      (match args with
       | [] ->
           (* issue for the caller themselves, no label *)
           let u = Auth_store.get_user_by_username t.store caller in
           (match u with
            | None ->
                raise (Pricklypear_lisp.Ast.Interp_error
                         (Pricklypear_lisp.Ast.Prim_error
                            ("issue-token", "no such user", "not-found")))
            | Some user ->
                let r = Auth_store.issue_token t.store ~user_id:user.id () in
                (match r with
                 | Error e ->
                     raise (Pricklypear_lisp.Ast.Interp_error
                              (Pricklypear_lisp.Ast.Prim_error
                                 ("issue-token", e, "failed")))
                 | Ok raw -> (Pricklypear_lisp.Ast.VStr raw)))
       | [ label ] ->
           (* issue for the caller themselves, with label *)
           let u = Auth_store.get_user_by_username t.store caller in
           (match u with
            | None ->
                raise (Pricklypear_lisp.Ast.Interp_error
                         (Pricklypear_lisp.Ast.Prim_error
                            ("issue-token", "no such user", "not-found")))
            | Some user ->
                let r = Auth_store.issue_token t.store ~user_id:user.id ~label:(parse_label label) () in
                (match r with
                 | Error e ->
                     raise (Pricklypear_lisp.Ast.Interp_error
                              (Pricklypear_lisp.Ast.Prim_error
                                 ("issue-token", e, "failed")))
                 | Ok raw -> (Pricklypear_lisp.Ast.VStr raw)))
       | [ Pricklypear_lisp.Ast.VStr uid; label ] ->
           require_admin "issue-token" (fun () ->
             let r = Auth_store.issue_token t.store ~user_id:uid ~label:(parse_label label) () in
             (match r with
              | Error e ->
                  raise (Pricklypear_lisp.Ast.Interp_error
                           (Pricklypear_lisp.Ast.Prim_error
                              ("issue-token", e, "failed")))
              | Ok raw -> (Pricklypear_lisp.Ast.VStr raw)))
       | _ -> (raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("issue-token", "need [label] (own) or user-id label (admin)", "bad-args"))))));

  (* (list-tokens [user-id]) -> VList of (id label last4 created last_used). *)
  Primitives.register tbl "list-tokens"
    (fun args ->
```

```
      let uname = caller_username () in
      let caller_opt = Auth_store.get_user_by_username t.store uname in
      let caller_id =
        match caller_opt with Some x -> x.Auth_store.id | None -> ""
      in
      let is_admin =
        match caller_opt with Some x -> x.Auth_store.is_admin | None ->
    false
      in
      let target_uid =
        match args with
        | [ Pricklypear_lisp.Ast.VStr uid ] when uid <> "" -> uid
        | _ -> caller_id
      in
      if not is_admin && target_uid <> caller_id then
        (raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("list-tokens", "denied: admin only",
    "denied"))))
      else if target_uid = "" then
        (raise (Pricklypear_lisp.Ast.Interp_error
                 (Pricklypear_lisp.Ast.Prim_error
                    ("list-tokens", "no such user", "not-found"))))
      else
        let metas = Auth_store.list_tokens t.store ~user_id:target_uid in
        let rows =
          List.map
            (fun (m : Auth_store.token_meta) ->
              Pricklypear_lisp.Ast.VList
                [ Pricklypear_lisp.Ast.VStr m.id;
                  Pricklypear_lisp.Ast.VStr (Option.value m.label
    ~default:"");
                  Pricklypear_lisp.Ast.VStr m.token_last4;
                  Pricklypear_lisp.Ast.VStr m.created_at;
                  Pricklypear_lisp.Ast.VStr (Option.value m.last_used_at
    ~default:"") ])
            metas
        in
        (Pricklypear_lisp.Ast.VList rows));

  (* (revoke-token credential-id) -> VStr ok|error. Caller may revoke own;
    admin any. *)
  Primitives.register tbl "revoke-token"
    (fun args ->
      match args with
      | [ Pricklypear_lisp.Ast.VStr cid ] when cid <> "" ->
          let r = Auth_store.revoke_token t.store ~credential_id:cid in
          (match r with
           | Error e ->
               raise (Pricklypear_lisp.Ast.Interp_error
                        (Pricklypear_lisp.Ast.Prim_error
                           ("revoke-token", e, "failed")))
           | Ok () -> (Pricklypear_lisp.Ast.VStr "ok"))
      | _ -> (raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("revoke-token", "need
    credential-id", "bad-args")))));

  (* (rotate-token credential-id [label]) -> VStr new-raw-token. Admin:
    rotates any;
    non-admin rotates own (credential must belong to caller). Returns new
    raw token ONCE. *)
  Primitives.register tbl "rotate-token"
    (fun args ->
      let uname = caller_username () in
      (match args with
       | [ Pricklypear_lisp.Ast.VStr cid ] when cid <> "" ->
           (* Verify the credential belongs to the caller (or admin). *)
           let caller = Auth_store.get_user_by_username t.store uname in
           let caller_id = match caller with Some x -> x.Auth_store.id |
    None -> "" in
           let is_admin = match caller with Some x -> x.Auth_store.is_admin
    | None -> false in
           let metas = Auth_store.list_tokens t.store ~user_id:caller_id in
```

```
1607            let owns = List.exists (fun (m : Auth_store.token_meta) -> m.id
    = cid) metas in
1608            if not is_admin && not owns then
1609              (raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("rotate-token", "denied: not your
    token", "denied"))))
1610            else
1611              let r = Auth_store.rotate_token t.store ~credential_id:cid
    ~user_id:caller_id () in
1612              (match r with
1613               | Error e ->
1614                   raise (Pricklypear_lisp.Ast.Interp_error
1615                            (Pricklypear_lisp.Ast.Prim_error
1616                               ("rotate-token", e, "failed")))
1617               | Ok raw -> (Pricklypear_lisp.Ast.VStr raw))
1618        | _ -> (raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("rotate-token", "need
    credential-id", "bad-args"))))));
1619
1620    (* (set-allow-Agent_tools.agent-promote bool) -> VStr ok|error. Sets the
    CALLER's own toggle. *)
1621    Primitives.register tbl "set-allow-Agent_tools.agent-promote"
1622      (fun args ->
1623       let parse_bool = function
1624         | Pricklypear_lisp.Ast.VBool b -> b
1625         | Pricklypear_lisp.Ast.VStr s ->
1626             (match String.lowercase_ascii (String.trim s) with
1627              | "1" | "true" | "yes" | "on" -> true
1628              | _ -> false)
1629         | Pricklypear_lisp.Ast.VInt n -> n <> 0L
1630         | _ -> false
1631       in
1632       let caller = caller_username () in
1633       if caller = "anonymous" then
1634         (raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("prim",
    "set-allow-Agent_tools.agent-promote: denied for anonymous",
    "denied"))))
1635       else
1636         (match args with
1637          | [ flag ] ->
1638              let u = Auth_store.get_user_by_username t.store caller in
1639              (match u with
1640               | None ->
1641                   raise (Pricklypear_lisp.Ast.Interp_error
1642                            (Pricklypear_lisp.Ast.Prim_error
1643                               ("prim",
    "set-allow-Agent_tools.agent-promote: no such user", "not-found")))
1644               | Some user ->
1645                   let v = parse_bool flag in
1646                   let r = Auth_store.set_allow_agent_promote t.store
    ~user_id:user.id ~value:v in
1647                   (match r with
1648                    | Error e ->
1649                        raise (Pricklypear_lisp.Ast.Interp_error
1650                                 (Pricklypear_lisp.Ast.Prim_error
1651                                    ("prim",
    "set-allow-Agent_tools.agent-promote: " ^ e, "failed")))
1652                    | Ok () -> (Pricklypear_lisp.Ast.VStr "ok")))
1653          | _ -> (raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("prim",
    "set-allow-Agent_tools.agent-promote: need bool", "bad-args"))))));
1654
1655    (* (promote [name...]) -> VStr count|error. Agent: promote own defs (or
    named)
1656       into parent's personal. Human: denied (use promote-to-main admin
    path). *)
1657    Primitives.register tbl "promote"
1658      (fun args ->
1659       let names =
1660         List.filter_map
1661           (fun a -> match a with Pricklypear_lisp.Ast.VStr s -> Some s | _
    -> None)
```

```
1662          args
1663       in
1664       let r = promote_to_personal t ~names in
1665       (match r with
1666        | Error e -> ((raise (Pricklypear_lisp.Ast.Interp_error
      (Pricklypear_lisp.Ast.Prim_error ("promote", e, "failed")))))
1667        | Ok n -> (Pricklypear_lisp.Ast.VStr (string_of_int n))));
1668
1669   (* (promote-to-main source-ref name...) | (promote-to-main source-ref
      (list name...))
1670      Admin-only selective upsert onto system/main. branching.promote /
      land. *)
1671   Primitives.register tbl "promote-to-main"
1672     (fun args ->
1673       let names_of_val = function
1674         | Pricklypear_lisp.Ast.VStr s -> [ s ]
1675         | Pricklypear_lisp.Ast.VList xs ->
1676             List.filter_map
1677               (function
1678                 | Pricklypear_lisp.Ast.VStr s -> Some s
1679                 | Pricklypear_lisp.Ast.VSymbol s -> Some s
1680                 | _ -> None)
1681               xs
1682         | Pricklypear_lisp.Ast.VSymbol s -> [ s ]
1683         | _ -> []
1684       in
1685       match args with
1686       | [] ->
1687           ((raise (Pricklypear_lisp.Ast.Interp_error
      (Pricklypear_lisp.Ast.Prim_error ("promote-to-main", "need source-ref
      and name(s)", "bad-args")))))
1688       | Pricklypear_lisp.Ast.VStr source_ref :: rest ->
1689           let names = List.concat_map names_of_val rest in
1690           if names = [] then
1691             ((raise (Pricklypear_lisp.Ast.Interp_error
      (Pricklypear_lisp.Ast.Prim_error ("promote-to-main", "no names
      selected", "bad-args")))))
1692           else if
1693             (not t.is_admin)
1694             && t.manifest.username <> "admin"
1695             && t.manifest.username <> "system"
1696           then
1697             ((raise (Pricklypear_lisp.Ast.Interp_error
      (Pricklypear_lisp.Ast.Prim_error ("promote-to-main", "requires admin",
      "denied")))))
1698           else begin
1699             let owner, name =
1700               match String.split_on_char '/' source_ref with
1701               | [ o; n ] when o <> "" && n <> "" -> (o, n)
1702               | _ -> ("", source_ref)
1703             in
1704             match Store.lookup_branch_id_by_name t.store ~owner ~name with
1705             | None ->
1706                 ((raise (Pricklypear_lisp.Ast.Interp_error
      (Pricklypear_lisp.Ast.Prim_error ("promote-to-main", "no such source
      branch: " ^ source_ref, "not-found")))))
1707             | Some source_id ->
1708                 let n =
1709                   Store.promote_functions t.store ~source_id
1710                     ~target_id:Store.system_main_id ~names
1711                 in
1712                 (if Store.current_branch_id t.store = Store.system_main_id
      then
1713                    rehydrate_fn t);
1714                 (Pricklypear_lisp.Ast.VStr
1715                    (Printf.sprintf "promoted %d ? system/main" n))
1716           end
1717       | _ ->
1718           ((raise (Pricklypear_lisp.Ast.Interp_error
      (Pricklypear_lisp.Ast.Prim_error ("promote-to-main", "need source-ref
      string first", "bad-args"))))));
1719
1720   (* (current-branch) -> VStr *)
```

```
1721  Primitives.register_sync tbl "current-branch"
1722    (fun _ ->
1723      let branch =
1724        match Request_context.get () with
1725        | Some c -> (Request_context.manifest c).branch_name
1726        | None -> t.manifest.branch_name
1727      in
1728      Pricklypear_lisp.Ast.VStr branch);
1729
1730  (* Durable agent sessions (ai-authoring.agent-portal). *)
1731  Primitives.register tbl "agent-session-new"
1732    (fun args ->
1733      match args with
1734      | [ Pricklypear_lisp.Ast.VStr seed ] ->
1735          Agent_tools.agent_session_new t ~seed ~tools:true ()
1736      | [ Pricklypear_lisp.Ast.VStr seed; Pricklypear_lisp.Ast.VBool tools
     ] ->
1737          Agent_tools.agent_session_new t ~seed ~tools ()
1738      | _ ->
1739          ((raise (Pricklypear_lisp.Ast.Interp_error
     (Pricklypear_lisp.Ast.Prim_error ("agent-session-new", "need seed-str
     [, tools-bool]", "bad-args"))))));
1740
1741  Primitives.register tbl "agent-say"
1742    (fun args ->
1743      match args with
1744      | [ Pricklypear_lisp.Ast.VStr sid; Pricklypear_lisp.Ast.VStr prompt ]
     ->
1745          Agent_tools.agent_say t ~session_id:sid ~prompt ()
1746      | [ Pricklypear_lisp.Ast.VStr sid; Pricklypear_lisp.Ast.VStr prompt;
1747          Pricklypear_lisp.Ast.VInt n ] ->
1748          Agent_tools.agent_say t ~session_id:sid ~prompt
     ~max_steps:(Int64.to_int n) ()
1749      | _ ->
1750          ((raise (Pricklypear_lisp.Ast.Interp_error
     (Pricklypear_lisp.Ast.Prim_error ("agent-say", "need session-id,
     prompt [, max-steps]", "bad-args"))))));
1751
1752  Primitives.register tbl "agent-history"
1753    (fun args ->
1754      match args with
1755      | [ Pricklypear_lisp.Ast.VStr sid ] -> Agent_tools.agent_history t
     ~session_id:sid
1756      | _ ->
1757          (raise (Pricklypear_lisp.Ast.Interp_error
     (Pricklypear_lisp.Ast.Prim_error ("agent-history", "need session-id",
     "bad-args")))));
1758
1759  Primitives.register tbl "agent-sessions"
1760    (fun args ->
1761      match args with
1762      | [] -> Agent_tools.agent_sessions t ()
1763      | [ Pricklypear_lisp.Ast.VInt n ] ->
1764          Agent_tools.agent_sessions t ~limit:(Int64.to_int n) ()
1765      | _ ->
1766          ((raise (Pricklypear_lisp.Ast.Interp_error
     (Pricklypear_lisp.Ast.Prim_error ("agent-sessions", "optional limit
     int", "bad-args"))))));
1767
1768  Primitives.register tbl "agent-session-delete"
1769    (fun args ->
1770      match args with
1771      | [ Pricklypear_lisp.Ast.VStr sid ] ->
1772          Agent_tools.agent_session_delete t ~session_id:sid
1773      | _ ->
1774          (raise (Pricklypear_lisp.Ast.Interp_error
     (Pricklypear_lisp.Ast.Prim_error ("agent-session-delete", "need
     session-id", "bad-args")))));
1775
1776  (* Temporary aliases: new session + one say. Prefer
     agent-session-new/say. *)
1777  Primitives.register tbl "agent"
1778    (fun args ->
```

```
      match args with
      | [ Pricklypear_lisp.Ast.VStr seed;
          Pricklypear_lisp.Ast.VStr prompt ] ->
          Agent_tools.agent t ~seed ~prompt ()
      | _ ->
          ((raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("agent", "need 2 strings (seed,
    prompt)", "bad-args"))))));

  Primitives.register tbl "agent-task"
    (fun args ->
      match args with
      | [ Pricklypear_lisp.Ast.VStr prompt ] ->
          Agent_tools.agent_task t ~prompt ()
      | [ Pricklypear_lisp.Ast.VStr prompt; Pricklypear_lisp.Ast.VInt n ] ->
          Agent_tools.agent_task t ~max_steps:(Int64.to_int n) ~prompt ()
      | _ ->
          ((raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("agent-task", "need prompt-str [,
    max-steps int]", "bad-args"))))));

  (* (propose-function 'name "spec text") -> VStr name | error string.
     Calls OpenAI-compat LLM, defines the function on the CURRENT branch
     with author = Agent_tools.agent:<model> (AI provenance). Never merges
     to system. *)
  Primitives.register tbl "propose-function"
    (fun args ->
      match args with
      | [ Pricklypear_lisp.Ast.VStr name;
          Pricklypear_lisp.Ast.VStr spec ] ->
          Agent_tools.propose_function t ~name ~spec
      | _ ->
          ((raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("propose-function", "bad args (need
    name-str, spec-str)", "bad-args"))))));

  (* (install-function name body-src) -> install AI-authored body (effect
     gate). *)
  Primitives.register tbl "install-function"
    (fun args ->
      match args with
      | [ Pricklypear_lisp.Ast.VStr name;
          Pricklypear_lisp.Ast.VStr body ] ->
          Agent_tools.install_ai_function t ~name ~body_src:body
      | _ ->
          ((raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("install-function", "need name-str,
    body-str", "bad-args"))))));

  (* (function-source name) -> VStr pretty body | VNil *)
  Primitives.register tbl "function-source"
    (fun args ->
      match args with
      | [ Pricklypear_lisp.Ast.VStr name ] ->
          let src = Agent_tools.function_source t name in
          (match src with
           | Some s -> (Pricklypear_lisp.Ast.VStr s)
           | None -> Pricklypear_lisp.Ast.VNil)
      | _ ->
          (raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("function-source", "need name-str",
    "bad-args"))))));

  (* (find-code needle [limit]) -> VList of name strings.
     Postgres ILIKE over effective function_defs.body::text (branch chain).
     *)
  Primitives.register tbl "find-code"
    (fun args ->
      let names_of needle limit =
        let ns =
          Store.search_function_bodies t.store ~needle ~limit
        in
        Pricklypear_lisp.Ast.VList
```

```
          (List.map (fun n -> Pricklypear_lisp.Ast.VStr n) ns)
      in
      match args with
      | [ Pricklypear_lisp.Ast.VStr needle ] -> names_of needle 200
      | [ Pricklypear_lisp.Ast.VStr needle; Pricklypear_lisp.Ast.VInt n ] ->
          names_of needle (Int64.to_int n)
      | [ Pricklypear_lisp.Ast.VStr needle; Pricklypear_lisp.Ast.VStr n_s ] ->
          (match int_of_string_opt n_s with
           | Some n -> names_of needle n
           | None ->
               ((raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("find-code", "limit must be int",
    "bad-args"))))))
      | _ ->
          (raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("find-code", "need needle-str [,
    limit]", "bad-args")))));

  (* (function-doc name) -> VStr docstring | VNil
     Store metadata first; special-form / prim blurbs as fallback. *)
  Primitives.register tbl "function-doc"
    (fun args ->
      match args with
      | [ Pricklypear_lisp.Ast.VStr name ] ->
          let doc = Store.lookup_function_doc t.store name in
          (match doc with
           | Some s -> (Pricklypear_lisp.Ast.VStr s)
           | None ->
               (match Pricklypear_lisp.Special_forms.find name with
                | Some e ->
                    (Pricklypear_lisp.Ast.VStr e.summary)
                | None ->
                    let blurb = Primitives.prim_summary name in
                    if blurb = "built-in primitive" then
                      Pricklypear_lisp.Ast.VNil
                    else (Pricklypear_lisp.Ast.VStr blurb)))
      | _ ->
          (raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("function-doc", "need name-str",
    "bad-args")))));

  (* (function-ast name) -> quoted body value tree | VNil.
     Does not eval the body; reifies the stored AST via quote_to_value. *)
  Primitives.register tbl "function-ast"
    (fun args ->
      match args with
      | [ Pricklypear_lisp.Ast.VStr name ] ->
          let ast = Agent_tools.function_ast t name in
          (match ast with
           | Some v -> v
           | None -> Pricklypear_lisp.Ast.VNil)
      | _ ->
          (raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("function-ast", "need name-str",
    "bad-args")))));

  (* ===== Jobs prims (borge/jobs.borg) ===== *)
  (* Guest surface: submit-job, job-status, jobs, cancel-job, step,
     schedule, unschedule, schedules. The runner fiber (boot.ml) claims
     and evals; prims just read/submit. with-step is a special form
     (eval.ml)
     routed through the __step__ prim in session.ml. *)

  let job_caller () =
    match Request_context.get () with
    | Some c -> (Request_context.manifest c).username
    | None -> t.manifest.username
  in
  let job_is_admin () =
    let uname = job_caller () in
    if uname = "anonymous" then false
    else
```

```
    match Auth_store.get_user_by_username t.store uname with
    | Some u -> u.Auth_store.is_admin
    | None -> false
in

Primitives.register tbl "submit-job"
  (fun args ->
    match args with
    | [ Pricklypear_lisp.Ast.VStr fn;
        args_val ] ->
      let args_json =
        match args_val with
        | Pricklypear_lisp.Ast.VStr s -> s
        | _ -> Pricklypear_lisp.Reader.pp_value args_val
      in
      let caller = job_caller () in
      if caller = "anonymous" then
        (raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("submit-job", "denied for
   anonymous", "denied"))))
      else
        let branch =
          match Request_context.get () with
          | Some c -> (Request_context.manifest c).branch_name
          | None -> t.manifest.branch_name
        in
        let now_ms =
          Int64.of_float (Unix.gettimeofday () *. 1000.0)
        in
        let id =
          Jobs.submit ~store:t.store ~fn ~args_json ~branch
            ~submitted_by:caller ~run_at_ms:now_ms ()
        in
        (Pricklypear_lisp.Ast.VStr id)
    | _ ->
      (raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("submit-job", "need fn-str, args",
   "bad-args")))));

Primitives.register tbl "submit-job-at"
  (fun args ->
    match args with
    | [ Pricklypear_lisp.Ast.VStr fn;
        args_val;
        Pricklypear_lisp.Ast.VInt run_at_ms ] ->
      let args_json =
        match args_val with
        | Pricklypear_lisp.Ast.VStr s -> s
        | _ -> Pricklypear_lisp.Reader.pp_value args_val
      in
      let caller = job_caller () in
      if caller = "anonymous" then
        (raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("submit-job-at", "denied for
   anonymous", "denied"))))
      else
        let branch =
          match Request_context.get () with
          | Some c -> (Request_context.manifest c).branch_name
          | None -> t.manifest.branch_name
        in
        let id =
          Jobs.submit ~store:t.store ~fn ~args_json ~branch
            ~submitted_by:caller ~run_at_ms ()
        in
        (Pricklypear_lisp.Ast.VStr id)
    | _ ->
      (raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("submit-job-at", "need fn-str, args,
   run-at-ms", "bad-args")))));

Primitives.register tbl "job-status"
  (fun args ->
```

```
    match args with
    | [ Pricklypear_lisp.Ast.VStr id ] ->
      Jobs.get ~store:t.store ~id
        ~caller:(job_caller ()) ~is_admin:(job_is_admin ())
    | _ -> (Pricklypear_lisp.Ast.VNil));

Primitives.register tbl "jobs"
  (fun _ ->
    Jobs.list ~store:t.store
      ~caller:(job_caller ()) ~is_admin:(job_is_admin ()) ~limit:50 ());

Primitives.register tbl "cancel-job"
  (fun args ->
    match args with
    | [ Pricklypear_lisp.Ast.VStr id ] ->
      Jobs.cancel ~store:t.store ~id
        ~caller:(job_caller ()) ~is_admin:(job_is_admin ())
    | _ ->
      (raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("cancel-job", "need job-id-str",
   "bad-args")))));

(* (schedule "name" "cron-or-ms" "fn" args) -> upsert a schedule,
   caller-scoped identity + branch. spec is a 5-field cron or an
   every-ms integer. Capability: WriteDb (borge/jobs.borg schedules). *)
Primitives.register tbl "schedule"
  (fun args ->
    match args with
    | [ Pricklypear_lisp.Ast.VStr name;
        Pricklypear_lisp.Ast.VStr spec;
        Pricklypear_lisp.Ast.VStr fn;
        args_val ] ->
      let args_json =
        match args_val with
        | Pricklypear_lisp.Ast.VStr s -> s
        | _ -> Pricklypear_lisp.Reader.pp_value args_val
      in
      let caller = job_caller () in
      if caller = "anonymous" then
        (raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("schedule", "denied for anonymous",
   "denied"))))
      else
        let branch =
          match Request_context.get () with
          | Some c -> (Request_context.manifest c).branch_name
          | None -> t.manifest.branch_name
        in
        Jobs.schedule_upsert ~store:t.store ~name ~spec ~fn ~args_json
          ~branch ~submitted_by:caller ()
    | _ ->
      (raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("schedule", "need name-str,
   spec-str, fn-str, args", "bad-args")))));

Primitives.register tbl "unschedule"
  (fun args ->
    match args with
    | [ Pricklypear_lisp.Ast.VStr name ] ->
      Jobs.unschedule ~store:t.store ~name ()
    | _ ->
      (raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("unschedule", "need name-str",
   "bad-args")))));

Primitives.register tbl "schedules"
  (fun _ -> Jobs.list_schedules ~store:t.store ());

(* ===== Forge prims ===== *)

(* (forge-effects "src-or-name") -> VStr badge string.
   Exposes lisp/lib/effect.ml infer over a parsed source expression
   (or a named binding's body). The SAME analysis install-function
```

```
    gates on; this prim merely exposes it for the forge mechanical
    flaggers (capability escalation check). Read-only, pure-ish.
    Spec: borge/forge.borg mechanical-flags. *)
  Primitives.register tbl "forge-effects"
    (fun args ->
      match args with
      | [ Pricklypear_lisp.Ast.VStr s ] ->
          (try
             let forms = Pricklypear_lisp.Reader.read_all s in
             (match forms with
              | [] -> (Pricklypear_lisp.Ast.VStr "pure")
              | expr :: _ ->
                  let defs = Store.list_function_names t.store in
                  let known_fn =
                    Pricklypear_lisp.Effect.known_fn_of_bindings
                      (List.map (fun (n, e, _) -> (n, e)) defs)
                  in
                  let badge =
                    Pricklypear_lisp.Effect.EffSet.badge
                      (Pricklypear_lisp.Effect.infer ~known_fn expr)
                  in
                  (Pricklypear_lisp.Ast.VStr badge))
           with exn ->
             raise (Pricklypear_lisp.Ast.Interp_error
                      (Pricklypear_lisp.Ast.Prim_error
                         ("forge-effects",
                          "parse/infer failed: " ^ Printexc.to_string exn,
                          "failed"))))
      | _ ->
          (raise (Pricklypear_lisp.Ast.Interp_error
     (Pricklypear_lisp.Ast.Prim_error ("forge-effects", "need
     src-or-name-str", "bad-args")))));

  (* (package-exports pkg) -> VList of public names (package-surface).
     Resolution: declared registry exports -> kernel_std_exports for
     std/kernel -> shallow heuristic. *)
  let kernel_std_exports =
    [ "list/map"; "list/filter"; "list/foldl"; "reverse"; "append"; "nth"
    ; "null?"; "pair?"; "car"; "cdr"; "cons"; "list"; "count"
    ; "string-append"; "string-length"; "substring"; "string-eq"
    ; "string-contains?"; "string-split"; "string-trim"; "html-escape";
     "as-str"; "str"
    ; "string-join"
    ; "string<?"; "string<=?"; "string>?"; "string>=?"
    ; "symbol?"; "string?"; "symbol=?"; "symbol->string"; "string->symbol"
    ; "not"; "type-of"; "read-string"; "read-all-string"; "function-ast"
    ; "function-source"; "function-doc"; "apropos"; "apropos-docs"
    ; "Agent_tools.describe"; "list-packages"; "package-bindings";
     "package-exports"
    ; "packages"; "package-enable"; "package-disable"; "prefix-of"
    ]
  in
  Primitives.register tbl "package-exports"
    (fun args ->
      match args with
      | [ Pricklypear_lisp.Ast.VStr pkg ] ->
          let pkg = String.lowercase_ascii (String.trim pkg) in
          if pkg = "std" || pkg = "kernel" || pkg = "(kernel)" then
            (Pricklypear_lisp.Ast.VList
                 (List.map
                    (fun n -> Pricklypear_lisp.Ast.VStr n)
                    kernel_std_exports))
          else begin
            let reg = Store.lookup_package t.store pkg in
            match reg with
            | Some r when r.exports <> [] ->
                (Pricklypear_lisp.Ast.VList
                     (List.map
                        (fun n -> Pricklypear_lisp.Ast.VStr n)
                        r.exports))
            | _ ->
                let names = Agent_tools.package_export_names t pkg in
                (Pricklypear_lisp.Ast.VList
```

```
                    (List.map (fun n -> Pricklypear_lisp.Ast.VStr n)
   names))
         end
     | _ ->
         (raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("package-exports", "need
   package-str", "bad-args")))));

(* (package-bindings pkg) -> VList of name strings (prefix pkg/ any
   depth) *)
Primitives.register tbl "package-bindings"
  (fun args ->
    match args with
    | [ Pricklypear_lisp.Ast.VStr pkg ] ->
        let names = Agent_tools.package_binding_names t pkg in
        (Pricklypear_lisp.Ast.VList
             (List.map (fun n -> Pricklypear_lisp.Ast.VStr n) names))
    | _ ->
        (raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("package-bindings", "need
   package-str", "bad-args")))));

(* (list-packages) -> VList of top-level package prefix strings *)
Primitives.register tbl "list-packages"
  (fun args ->
    match args with
    | [] | [ Pricklypear_lisp.Ast.VNil ] ->
        let names = Agent_tools.list_package_names t in
        (Pricklypear_lisp.Ast.VList
             (List.map (fun n -> Pricklypear_lisp.Ast.VStr n) names))
    | _ ->
        ((raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("list-packages", "takes no args",
   "bad-args"))))));

(* (packages) -> VList of (name enabled exports-count) rows from registry
   +
   prefix-discovered packages (enabled=true, exports=0 if unregistered).
   *)
Primitives.register tbl "packages"
  (fun args ->
    match args with
    | [] | [ Pricklypear_lisp.Ast.VNil ] ->
        let reg = Store.list_packages t.store in
        let prefixes = Agent_tools.list_package_names t in
        let seen = Hashtbl.create 32 in
        let rows = ref [] in
        List.iter
          (fun (r : Store.package_row) ->
            Hashtbl.replace seen r.name ();
            rows :=
              Pricklypear_lisp.Ast.VList
                [ Pricklypear_lisp.Ast.VStr r.name
                ; Pricklypear_lisp.Ast.VBool r.enabled
                ; Pricklypear_lisp.Ast.VInt
                    (Int64.of_int (List.length r.exports))
                ]
              :: !rows)
          reg;
        List.iter
          (fun p ->
            if not (Hashtbl.mem seen p) then
              rows :=
                Pricklypear_lisp.Ast.VList
                  [ Pricklypear_lisp.Ast.VStr p
                  ; Pricklypear_lisp.Ast.VBool true
                  ; Pricklypear_lisp.Ast.VInt 0L
                  ]
                :: !rows)
          prefixes;
        let sorted =
          List.sort
            (fun a b ->
```

```
2164                match a, b with
2165                | Pricklypear_lisp.Ast.VList (Pricklypear_lisp.Ast.VStr na
    :: _),
2166                  Pricklypear_lisp.Ast.VList (Pricklypear_lisp.Ast.VStr nb
    :: _) ->
2167                    String.compare na nb
2168                | _ -> 0)
2169              !rows
2170          in
2171          (Pricklypear_lisp.Ast.VList sorted)
2172       | _ ->
2173          ((raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("packages", "takes no args",
    "bad-args"))))));
2174
2175    let protected_pkg name =
2176      let n = String.lowercase_ascii (String.trim name) in
2177      n = "std" || n = "kernel" || n = "(kernel)"
2178    in
2179
2180    (* (package-enable name) -- flip enabled=true on current branch +
       rehydrate *)
2181    Primitives.register tbl "package-enable"
2182      (fun args ->
2183        match args with
2184        | [ Pricklypear_lisp.Ast.VStr name ]
2185        | [ Pricklypear_lisp.Ast.VSymbol name ] ->
2186          let name = String.trim name in
2187          if name = "" then
2188            ((raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("package-enable", "empty name",
    "bad-args")))))
2189          else if protected_pkg name then
2190            ((raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("package-enable", name ^ " is always
    enabled", "denied")))))
2191          else if t.manifest.username = "anonymous" then
2192            ((raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("package-enable", "auth required",
    "denied")))))
2193          else begin
2194            let () =
2195              Store.set_package_enabled t.store ~name ~enabled:true
2196                ~author:t.manifest.username
2197            in
2198            let () = rehydrate_fn t in
2199            (Pricklypear_lisp.Ast.VStr ("enabled " ^ name))
2200          end
2201       | _ ->
2202          (raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("package-enable", "need
    package-str", "bad-args")))));
2203
2204    (* (package-disable name) -- hide package bindings on current branch *)
2205    Primitives.register tbl "package-disable"
2206      (fun args ->
2207        match args with
2208        | [ Pricklypear_lisp.Ast.VStr name ]
2209        | [ Pricklypear_lisp.Ast.VSymbol name ] ->
2210          let name = String.trim name in
2211          if name = "" then
2212            ((raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("package-disable", "empty name",
    "bad-args")))))
2213          else if protected_pkg name then
2214            ((raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("package-disable", "cannot disable "
    ^ name, "denied")))))
2215          else if t.manifest.username = "anonymous" then
2216            ((raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("package-disable", "auth required",
    "denied")))))
2217          else begin
```

```
            let () =
              Store.set_package_enabled t.store ~name ~enabled:false
                ~author:t.manifest.username
            in
            let () = rehydrate_fn t in
            (Pricklypear_lisp.Ast.VStr ("disabled " ^ name))
          end
      | _ ->
          (raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("package-disable", "need
    package-str", "bad-args")))));

  (* (apropos-docs pat) -> VList of (name . doc) pairs *)
  Primitives.register tbl "apropos-docs"
    (fun args ->
      match args with
      | [ Pricklypear_lisp.Ast.VStr pat ] ->
          let pairs = Agent_tools.apropos_docs t pat in
          let rows =
            List.map
              (fun (n, d) ->
                Pricklypear_lisp.Ast.VList
                  [ Pricklypear_lisp.Ast.VStr n
                  ; Pricklypear_lisp.Ast.VStr d
                  ])
              pairs
          in
          (Pricklypear_lisp.Ast.VList rows)
      | _ ->
          (raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("apropos-docs", "need pattern-str",
    "bad-args")))));

  (* (Agent_tools.describe name-or-pkg) -> VStr selective reveal text *)
  Primitives.register tbl "describe"
    (fun args ->
      match args with
      | [ Pricklypear_lisp.Ast.VStr needle ] ->
          let s = Agent_tools.describe t needle in
          (Pricklypear_lisp.Ast.VStr s)
      | _ ->
          (raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("prim", "Agent_tools.describe: need
    name-or-package-str", "bad-args")))));

  (* (export-package pkg) -> VStr lib.pp text *)
  Primitives.register tbl "export-package"
    (fun args ->
      match args with
      | [ Pricklypear_lisp.Ast.VStr pkg ] ->
          let text = Agent_tools.export_package t pkg in
          (Pricklypear_lisp.Ast.VStr text)
      | _ ->
          (raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("export-package", "need
    package-str", "bad-args")))));

  (* SSH keys (auth.user-ssh-repl phase 1) *)
  Primitives.register tbl "add-ssh-key"
    (fun args ->
      if t.manifest.username = "anonymous" then
        (raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("add-ssh-key", "denied for
    anonymous", "denied"))))
      else
        match args with
        | [ Pricklypear_lisp.Ast.VStr openssh ] ->
            let r =
              Auth_store.add_ssh_key t.store ~user_id:t.manifest.user_id
                ~openssh ()
            in
            (match r with
             | Ok m ->
```

```
                (Pricklypear_lisp.Ast.VStr m.Auth_store.fingerprint)
            | Error e ->
                ((raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("add-ssh-key", e, "failed"))))))
       | [ Pricklypear_lisp.Ast.VStr openssh; Pricklypear_lisp.Ast.VStr
   label ] ->
           let r =
             Auth_store.add_ssh_key t.store ~user_id:t.manifest.user_id
               ~openssh ~label:(Some label) ()
           in
           (match r with
            | Ok m ->
                (Pricklypear_lisp.Ast.VStr m.Auth_store.fingerprint)
            | Error e ->
                ((raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("add-ssh-key", e, "failed"))))))
       | _ ->
           (raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("add-ssh-key", "need openssh-str [,
   label]", "bad-args")))));

Primitives.register tbl "list-ssh-keys"
  (fun _args ->
    if t.manifest.username = "anonymous" then
      (raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("list-ssh-keys", "denied for
   anonymous", "denied"))))
    else
      let keys =
        Auth_store.list_ssh_keys t.store ~user_id:t.manifest.user_id
      in
      let vs =
        List.map
          (fun (k : Auth_store.ssh_key_meta) ->
            Pricklypear_lisp.Ast.VStr k.fingerprint)
          keys
      in
      (Pricklypear_lisp.Ast.VList vs));

Primitives.register tbl "revoke-ssh-key"
  (fun args ->
    if t.manifest.username = "anonymous" then
      (raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("revoke-ssh-key", "denied for
   anonymous", "denied"))))
    else
      match args with
      | [ Pricklypear_lisp.Ast.VStr id_or_fp ] ->
          let r =
            Auth_store.revoke_ssh_key t.store ~user_id:t.manifest.user_id
              ~id_or_fingerprint:id_or_fp
          in
          (match r with
           | Ok () -> Pricklypear_lisp.Ast.VNil
           | Error e ->
               ((raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("revoke-ssh-key", e, "failed"))))))
       | _ ->
           (raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("revoke-ssh-key", "need
   fingerprint-or-id", "bad-args")))));

(* Smoke/admin: resolve openssh pubkey -> username | ().
   ORACLE ONLY: this prim must never be reachable by anonymous callers
   (it is a key-equality oracle). The future SSH login door must verify a
   challenge SIGNATURE made by the private key -- never test key equality
   against a stored pubkey directly. *)
Primitives.register tbl "verify-ssh-key"
  (fun args ->
     if t.manifest.username = "anonymous" then
       (raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("verify-ssh-key", "denied for
   anonymous", "denied"))))
```

```
      else
        match args with
      | [ Pricklypear_lisp.Ast.VStr openssh ] ->
          let u = Auth_store.verify_ssh_key t.store
    ~openssh_or_blob:openssh () in
          (match u with
           | Some user ->
               (Pricklypear_lisp.Ast.VStr user.Auth_store.username)
           | None -> Pricklypear_lisp.Ast.VNil)
      | _ ->
          (raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("verify-ssh-key", "need
    openssh-pubkey-str", "bad-args")))));

  (* Per-user LLM keys (auth.user-llm-keys) *)
  Primitives.register tbl "set-llm-key"
    (fun args ->
      if t.manifest.username = "anonymous" then
        (raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("set-llm-key", "denied for
    anonymous", "denied"))))
      else
        match args with
        | [ Pricklypear_lisp.Ast.VStr key ] ->
            let r =
              Auth_store.set_llm_key t.store ~user_id:t.manifest.user_id
                ~api_key:key ()
            in
            (match r with
             | Ok m ->
                 (Pricklypear_lisp.Ast.VStr ("ok last4=" ^
    m.Auth_store.last4))
             | Error e ->
                 ((raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("set-llm-key", e, "failed"))))))
        | [ Pricklypear_lisp.Ast.VStr key; Pricklypear_lisp.Ast.VStr base ]
    ->
            let r =
              Auth_store.set_llm_key t.store ~user_id:t.manifest.user_id
                ~api_key:key ~base:(Some base) ()
            in
            (match r with
             | Ok m ->
                 (Pricklypear_lisp.Ast.VStr ("ok last4=" ^
    m.Auth_store.last4))
             | Error e ->
                 ((raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("set-llm-key", e, "failed"))))))
        | [ Pricklypear_lisp.Ast.VStr key; Pricklypear_lisp.Ast.VStr base;
            Pricklypear_lisp.Ast.VStr model ] ->
            let r =
              Auth_store.set_llm_key t.store ~user_id:t.manifest.user_id
                ~api_key:key ~base:(Some base) ~model:(Some model) ()
            in
            (match r with
             | Ok m ->
                 (Pricklypear_lisp.Ast.VStr ("ok last4=" ^
    m.Auth_store.last4))
             | Error e ->
                 ((raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("set-llm-key", e, "failed"))))))
        | _ ->
            (raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("set-llm-key", "need key-str [,
    base] [, model]", "bad-args")))));

  Primitives.register tbl "clear-llm-key"
    (fun _args ->
      if t.manifest.username = "anonymous" then
        (raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("clear-llm-key", "denied for
    anonymous", "denied"))))
      else
```

```
      let r =
        Auth_store.clear_llm_key t.store ~user_id:t.manifest.user_id
      in
      match r with
      | Ok () -> Pricklypear_lisp.Ast.VNil
      | Error e ->
          ((raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("clear-llm-key", e, "failed"))))));

Primitives.register tbl "llm-key-status"
  (fun _args ->
    if t.manifest.username = "anonymous" then
      (raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("llm-key-status", "denied for
   anonymous", "denied"))))
    else
      let meta =
        Auth_store.get_llm_key_meta t.store ~user_id:t.manifest.user_id
      in
      match meta with
      | None ->
          let env_set = Llm_client.api_key () <> "" in
          (Pricklypear_lisp.Ast.VStr
               (if env_set then
                  "configured=false env_fallback=true"
                else "configured=false env_fallback=false"))
      | Some m ->
          let base =
            match m.base with Some b -> " base=" ^ b | None -> ""
          in
          let model =
            match m.model with Some m -> " model=" ^ m | None -> ""
          in
          (Pricklypear_lisp.Ast.VStr
               (Printf.sprintf "configured=true last4=%s%s%s"
                  m.last4 base model)));

(* (llm-chat system user [temperature] [max-tokens]) -> VStr content.
   Raw one-shot chat completion -- NOT the agentic install loop. Returns
   the model's content string. Used by libs/forge drafters and any
   caller that wants a plain completion. See llm_client.call. *)
Primitives.register tbl "llm-chat"
  (fun args ->
    let temperature, max_tokens, system, user =
      match args with
      | [ Pricklypear_lisp.Ast.VStr system;
          Pricklypear_lisp.Ast.VStr user ] ->
          (0.2, 2048, system, user)
      | [ Pricklypear_lisp.Ast.VStr system;
          Pricklypear_lisp.Ast.VStr user;
          Pricklypear_lisp.Ast.VFloat t ] ->
          (t, 2048, system, user)
      | [ Pricklypear_lisp.Ast.VStr system;
          Pricklypear_lisp.Ast.VStr user;
          Pricklypear_lisp.Ast.VInt n ] ->
          (0.2, Int64.to_int n, system, user)
      | [ Pricklypear_lisp.Ast.VStr system;
          Pricklypear_lisp.Ast.VStr user;
          Pricklypear_lisp.Ast.VFloat t;
          Pricklypear_lisp.Ast.VInt n ] ->
          (t, Int64.to_int n, system, user)
      | _ ->
          (0.2, 2048, "", "need system-str, user-str")
    in
    let r =
      Llm_client.call ~temperature ~max_tokens ~system ~user ()
    in
    (match r with
     | Ok s -> (Pricklypear_lisp.Ast.VStr s)
     | Error e -> (Pricklypear_lisp.Ast.VStr ("llm-chat: " ^ e))));
      (* lint-exempt-prim-error: forge/llm-error? pattern-matches this
   error
```

```
2465            prefix (libs/forge lib.pp); raising needs the paired Nopales
    guard
2466            change on the live package. Tracked on the habitat issue
    register. *)
2467
2468    (* Coding tools: freeze AST rewrites into store *)
2469    Primitives.register tbl "llm-inner"
2470      (fun args ->
2471        match args with
2472        | [ Pricklypear_lisp.Ast.VStr name;
2473            Pricklypear_lisp.Ast.VStr concept ] ->
2474            Agent_tools.llm_inner t ~name ~concept
2475        | _ ->
2476            (raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("llm-inner", "need name-str,
    concept-str", "bad-args")))));
2477
2478    Primitives.register tbl "llm-outer"
2479      (fun args ->
2480        match args with
2481        | [ Pricklypear_lisp.Ast.VStr name;
2482            Pricklypear_lisp.Ast.VStr concept ] ->
2483            Agent_tools.llm_outer t ~name ~concept
2484        | _ ->
2485            (raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("llm-outer", "need name-str,
    concept-str", "bad-args")))));
2486
2487    Primitives.register tbl "llm-rewrite"
2488      (fun args ->
2489        match args with
2490        | [ Pricklypear_lisp.Ast.VStr name;
2491            Pricklypear_lisp.Ast.VStr concept ] ->
2492            Agent_tools.llm_rewrite t ~mode:"replace" ~name ~concept
2493        | [ Pricklypear_lisp.Ast.VStr name;
2494            Pricklypear_lisp.Ast.VStr concept;
2495            Pricklypear_lisp.Ast.VStr mode ] ->
2496            Agent_tools.llm_rewrite t ~mode ~name ~concept
2497        | _ ->
2498            ((raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("llm-rewrite", "need name, concept
    [, mode replace|wrap]", "bad-args"))))));
2499
2500    Primitives.register tbl "llm-seed"
2501      (fun args ->
2502        match args with
2503        | [ Pricklypear_lisp.Ast.VStr requirement ] ->
2504            Agent_tools.llm_seed t ~requirement ()
2505        | [ Pricklypear_lisp.Ast.VStr requirement;
2506            Pricklypear_lisp.Ast.VInt n ] ->
2507            Agent_tools.llm_seed t ~requirement ~max_steps:(Int64.to_int n) ()
2508        | _ ->
2509            ((raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("llm-seed", "need requirement-str [,
    max-steps int]", "bad-args"))))));
2510
2511    (* Ops: structured log tail + analytics summary (product-experience). *)
2512    Primitives.register tbl "log-tail"
2513      (fun args ->
2514        let n, level =
2515          match args with
2516          | [] -> 50, None
2517          | [ Pricklypear_lisp.Ast.VInt n ] -> Int64.to_int n, None
2518          | [ Pricklypear_lisp.Ast.VStr lv ] -> 50, Ops_log.level_of_string lv
2519          | [ Pricklypear_lisp.Ast.VInt n; Pricklypear_lisp.Ast.VStr lv ] ->
2520              Int64.to_int n, Ops_log.level_of_string lv
2521          | [ Pricklypear_lisp.Ast.VStr lv; Pricklypear_lisp.Ast.VInt n ] ->
2522              Int64.to_int n, Ops_log.level_of_string lv
2523          | _ -> 50, None
2524        in
2525        let evs = Ops_log.tail ~n ?level () in
2526        (Pricklypear_lisp.Ast.VList
2527             (List.map Ops_log.event_to_value evs)));
```

```
2528
2529  Primitives.register tbl "ops/summary"
2530    (fun _args ->
2531      let ok, fail = Ops_incidents.count_logins t.store in
2532      let j = Ops_stats.summary_yojson ~logins_ok:ok ~logins_fail:fail in
2533      (Pricklypear_lisp.Ast.VStr (Yojson.Safe.to_string j)));
2534
2535  Primitives.register tbl "ops/incidents"
2536    (fun _args ->
2537      let j = Ops_incidents.list_open_incidents t.store in
2538      (Pricklypear_lisp.Ast.VStr (Yojson.Safe.to_string j)));
2539
2540  (* (code-history "name") / (code-history "name" "owner/branch")
2541     ? VStr JSON list newest-first. Spec: branching.code-history. *)
2542  Primitives.register tbl "code-history"
2543    (fun args ->
2544      let resolve_branch_id ref_opt =
2545        match ref_opt with
2546        | None -> Store.current_branch_id t.store
2547        | Some branch_ref ->
2548            let owner, name =
2549              match String.split_on_char '/' branch_ref with
2550              | [ o; n ] when o <> "" && n <> "" -> (o, n)
2551              | _ -> ("", branch_ref)
2552            in
2553            (match
2554               Store.lookup_branch_id_by_name t.store ~owner ~name
2555             with
2556             | Some id -> id
2557             | None -> "")
2558      in
2559      match args with
2560      | [ Pricklypear_lisp.Ast.VStr name ] ->
2561          let bid = resolve_branch_id None in
2562          if bid = "" then
2563            (Pricklypear_lisp.Ast.VStr "[]")
2564          else
2565            (Pricklypear_lisp.Ast.VStr
2566               (Store.list_code_history_json t.store ~branch_id:bid ~name))
2567      | [ Pricklypear_lisp.Ast.VStr name
2568        ; Pricklypear_lisp.Ast.VStr branch_ref ] ->
2569          let bid = resolve_branch_id (Some branch_ref) in
2570          if bid = "" then
2571            (Pricklypear_lisp.Ast.VStr
2572               (Printf.sprintf
2573                  "code-history: no such branch: %s" branch_ref))
2574          else
2575            (Pricklypear_lisp.Ast.VStr
2576               (Store.list_code_history_json t.store ~branch_id:bid ~name))
2577      | _ ->
2578          ((raise (Pricklypear_lisp.Ast.Interp_error
     (Pricklypear_lisp.Ast.Prim_error ("code-history", "need name
     [owner/branch]", "bad-args"))))));
2579
2580  (* (code-diff "name" n) ? VStr text diff of current vs nth-newest
       history. *)
2581  Primitives.register tbl "code-diff"
2582    (fun args ->
2583      match args with
2584      | [ Pricklypear_lisp.Ast.VStr name; Pricklypear_lisp.Ast.VInt n64 ] ->
2585          let n = Int64.to_int n64 in
2586          let bid = Store.current_branch_id t.store in
2587          (match Store.code_diff t.store ~branch_id:bid ~name ~n with
2588           | Ok s -> (Pricklypear_lisp.Ast.VStr s)
2589           | Error e -> (Pricklypear_lisp.Ast.VStr e))
2590      | [ Pricklypear_lisp.Ast.VStr name
2591        ; Pricklypear_lisp.Ast.VStr n_s ] ->
2592          (match int_of_string_opt n_s with
2593           | None ->
2594               ((raise (Pricklypear_lisp.Ast.Interp_error
     (Pricklypear_lisp.Ast.Prim_error ("code-diff", "n must be int",
     "bad-args")))))
2595           | Some n ->
```

```
              let bid = Store.current_branch_id t.store in
              (match Store.code_diff t.store ~branch_id:bid ~name ~n with
               | Ok s -> (Pricklypear_lisp.Ast.VStr s)
               | Error e -> (Pricklypear_lisp.Ast.VStr e)))
     | _ ->
         (raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("code-diff", "need name n",
    "bad-args")))));

  (* (diff source target) -> VList of VStr change summaries.
     Spec: branching.diff.  Args: two branch specs (owner/name strings, same
     parsing as checkout/merge/promote).  Returns one summary string per
     divergent row across function_defs + type_defs + view_bindings:
       "<kind>\t<status>\t<name>\t<from-updated>\t<to-updated>"
     Summary level ONLY -- the structured AST delta is consumed in-process
    by
     the /diff page renderer, not surfaced here (a programmatic (diff-ast ?)
     prim is explicit v2). *)
  Primitives.register tbl "diff"
    (fun args ->
     let resolve ref_ =
       let owner, name =
         match String.split_on_char '/' ref_ with
         | [ o; n ] when o <> "" && n <> "" -> (o, n)
         | _ -> ("", ref_)
       in
       Store.lookup_branch_id_by_name t.store ~owner ~name
     in
     let to_ref = function
       | Pricklypear_lisp.Ast.VStr s -> Some s
       | Pricklypear_lisp.Ast.VSymbol s -> Some s
       | _ -> None
     in
     match args with
     | [ a; b ] ->
         (match to_ref a, to_ref b with
          | Some from_ref, Some to_ref ->
              (match resolve from_ref, resolve to_ref with
               | Some from_id, Some to_id ->
                   let rows = Store.diff_branches t.store ~from_id ~to_id
    in
                   let kind_s = function
                     | Store.DiffFn -> "fn"
                     | Store.DiffType -> "type"
                     | Store.DiffRoute -> "route"
                   in
                   let status_s = function
                     | Store.DiffAdded -> "added"
                     | Store.DiffRemoved -> "removed"
                     | Store.DiffModified -> "modified"
                   in
                   let line (r : Store.branch_diff_row) =
                     let fu = Option.value ~default:"-"
    r.Store.from_updated in
                     let tu = Option.value ~default:"-" r.Store.to_updated
    in
                     Printf.sprintf "%s\t%s\t%s\t%s\t%s"
                       (kind_s r.Store.diff_kind) (status_s
    r.Store.diff_status)
                       r.Store.diff_name fu tu
                   in
                   (Pricklypear_lisp.Ast.VList
                      (List.map (fun r -> Pricklypear_lisp.Ast.VStr (line
    r)) rows))
               | _ ->
                   (Pricklypear_lisp.Ast.VStr
                      (Printf.sprintf "diff: no such branch pair (%s, %s)"
                         from_ref to_ref)))
          | _ ->
              (raise
                 (Pricklypear_lisp.Ast.Interp_error
                    (Pricklypear_lisp.Ast.Prim_error
                       ("diff", "need source target", "bad-args")))))
```

```
      | _ ->
          (raise (Pricklypear_lisp.Ast.Interp_error
        (Pricklypear_lisp.Ast.Prim_error ("diff", "need source target",
        "bad-args")))));
```

97 image/lib/prims_graph.ml

```
(** Prim registration extracted from session.ml (Eio-floor Phase 4). *)
open Session_types

let split_branch_ref (s : string) : string * string =
  match String.split_on_char '/' s with
  | [ o; n ] -> (o, n)
  | [ n ] -> ("system", n)
  | _ -> ("system", s)

let method_to_string (m : http_method) : string =
  match m with `GET -> "GET" | `POST -> "POST"

let require_request (t : t) : request =
  match t.current_request with
  | Some r -> r
  | None ->
      raise
        (Pricklypear_lisp.Ast.Interp_error
           (Pricklypear_lisp.Ast.User_error ("no active request",
    "bad-args")))

let str_name = function
  | Pricklypear_lisp.Ast.VStr s | Pricklypear_lisp.Ast.VSymbol s -> Some s
  | _ -> None

let depth_arg = function
  | None -> 1
  | Some (Pricklypear_lisp.Ast.VInt n) -> Int64.to_int n
  | Some (Pricklypear_lisp.Ast.VStr s) ->
      (try int_of_string s with _ -> 1)
  | Some _ -> 1

let vlist_strs xs =
  Pricklypear_lisp.Ast.VList (List.map (fun s -> Pricklypear_lisp.Ast.VStr
     s) xs)

let impact_to_json (e : impact_event) : string =
  let open Yojson.Safe in
  to_string
    (`Assoc
       [ ("name", `String e.name)
       ; ("at", `Float e.at)
       ; ("branch", `String e.branch)
       ; ("author", `String e.author)
       ; ("dependents", `List (List.map (fun s -> `String s) e.dependents))
       ; ("routes", `List (List.map (fun s -> `String s) e.routes))
       ; ("status", `String e.status)
       ])

let change_to_json (e : change_event) : string =
  Yojson.Safe.to_string
    (`Assoc
       [ ("op", `String e.op)
       ; ("name", `String e.name)
       ; ("at", `Float e.at)
       ; ("branch", `String e.branch)
       ; ("author", `String e.author)
       ])

let record_define_side_effects = Call_graph.record_define_side_effects
let impact_summary_text = Call_graph.impact_summary_text


let register (t : t) (tbl : Primitives.t) : unit =
  Primitives.register tbl "callees"
    (fun args ->
      match args with
      | [ n ] | [ n; _ ] as a ->
          (match str_name n with
           | None -> (vlist_strs [])
           | Some name ->
```

```
              let depth =
                match a with
                | [ _; d ] -> depth_arg (Some d)
                | _ -> 1
              in
              let snap = Call_graph.snapshot_of_session t in
              (vlist_strs (Call_graph.callees snap ~name ~depth)))
    | _ -> (vlist_strs []));
  Primitives.register tbl "callers"
    (fun args ->
      match args with
      | [ n ] ->
          (match str_name n with
           | None -> (vlist_strs [])
           | Some name ->
               let snap = Call_graph.snapshot_of_session t in
               (vlist_strs (Call_graph.callers snap name)))
      | _ -> (vlist_strs []));
  Primitives.register tbl "dependents"
    (fun args ->
      match args with
      | [ n ] | [ n; _ ] as a ->
          (match str_name n with
           | None -> (vlist_strs [])
           | Some name ->
               let depth =
                 match a with
                 | [ _; d ] -> depth_arg (Some d)
                 | _ -> 8
               in
               let snap = Call_graph.snapshot_of_session t in
               (vlist_strs (Call_graph.dependents snap ~name ~depth)))
      | _ -> (vlist_strs []));
  Primitives.register tbl "used-by-routes"
    (fun args ->
      match args with
      | [ n ] ->
          (match str_name n with
           | None -> (vlist_strs [])
           | Some name ->
               let snap = Call_graph.snapshot_of_session t in
               (vlist_strs (Call_graph.used_by_routes snap name)))
      | _ -> (vlist_strs []));
  Primitives.register tbl "holes"
    (fun args ->
      match args with
      | [ n ] ->
          (match str_name n with
           | None -> (vlist_strs [])
           | Some name ->
               let snap = Call_graph.snapshot_of_session t in
               (vlist_strs (Call_graph.holes snap name)))
      | _ -> (vlist_strs []));
  Primitives.register tbl "impact-queue"
    (fun _args ->
      let xs =
        List.map
          (fun e -> Pricklypear_lisp.Ast.VStr (impact_to_json e))
          t.impact_queue
      in
      (Pricklypear_lisp.Ast.VList xs));
  Primitives.register tbl "impact-clear"
    (fun args ->
      (match args with
       | [] | [ Pricklypear_lisp.Ast.VStr "all" ]
       | [ Pricklypear_lisp.Ast.VSymbol "all" ] ->
           t.impact_queue <- []
       | [ n ] ->
           (match str_name n with
            | Some name ->
                t.impact_queue <-
                  List.filter
                    (fun (e : impact_event) -> e.name <> name)
```

```
                  t.impact_queue
          | None -> ())
     | _ -> ());
    Pricklypear_lisp.Ast.VNil);
Primitives.register tbl "impact-mark"
  (fun args ->
    match args with
    | [ n; st ] ->
        (match str_name n, str_name st with
         | Some name, Some status ->
             t.impact_queue <-
               List.map
                 (fun (e : impact_event) ->
                   if e.name = name then { e with status } else e)
                 t.impact_queue;
             (Pricklypear_lisp.Ast.VStr status)
         | _ -> Pricklypear_lisp.Ast.VNil)
    | _ -> Pricklypear_lisp.Ast.VNil);
Primitives.register tbl "change-transcript"
  (fun _args ->
    let xs =
      List.map
        (fun e -> Pricklypear_lisp.Ast.VStr (change_to_json e))
        t.change_transcript
    in
    (Pricklypear_lisp.Ast.VList xs));
Primitives.register tbl "branch-changes"
  (fun args ->
    let branch_id =
      match args with
      | [] -> (Some t.manifest.branch_id)
      | [ n ] ->
          (match str_name n with
           | None -> None
           | Some ref_ ->
               let owner, name = split_branch_ref ref_ in
               Store.lookup_branch_id_by_name t.store ~owner ~name)
      | _ -> None
    in
    match branch_id with
    | None -> (Pricklypear_lisp.Ast.VList [])
    | Some bid ->
        let own = Store.list_own_functions t.store ~branch_id:bid in
        let routes = Store.list_own_routes t.store ~branch_id:bid in
        (* branching.branch-status: class is the git-style status
           (clean/ahead-new/ahead/behind/diverged/synced), not a
           name-presence guess. *)
        let status = Store.branch_status t.store ~branch_id:bid in
        let class_of kind name =
          match
            List.find_opt
              (fun (r : Store.branch_status_row) ->
                r.Store.st_kind = kind && r.Store.st_name = name)
              status
          with
          | Some r -> Store.st_status_s r.Store.st_status
          | None -> "own"
        in
        let fn_items =
          List.map
            (fun (name, author, updated) ->
              let j =
                Yojson.Safe.to_string
                  (`Assoc
                     [ ("name", `String name)
                     ; ("kind", `String "fn")
                     ; ("author", `String author)
                     ; ("updated_at", `String updated)
                     ; ("class", `String (class_of Store.DiffFn name))
                     ])
              in
              Pricklypear_lisp.Ast.VStr j)
            own
```

```
        in
        let route_items =
          List.map
            (fun (path, handler) ->
              let j =
                Yojson.Safe.to_string
                  (`Assoc
                     [ ("name", `String path)
                     ; ("kind", `String "route")
                     ; ("handler", `String handler)
                     ; ("class", `String (class_of Store.DiffRoute path))
                     ])
              in
              Pricklypear_lisp.Ast.VStr j)
            routes
        in
        (Pricklypear_lisp.Ast.VList (fn_items @ route_items)));
  Primitives.register tbl "stub-function"
    (fun args ->
      match args with
      | [ n ] ->
          (match str_name n with
           | None ->
               (raise (Pricklypear_lisp.Ast.Interp_error
     (Pricklypear_lisp.Ast.Prim_error ("stub-function", "need name",
     "bad-args"))))
           | Some name ->
               let existing = Store.lookup_function t.store name in
               (match existing with
                | Some _ ->
                    (Pricklypear_lisp.Ast.VStr
                         ("stub-function: already bound: " ^ name))
                | None ->
                    let body_src =
                      Printf.sprintf
                        "(lambda ()\n  \"TODO: implement %s\")" name
                    in
                    let r =
                      Agent_tools.define_binding t ~name ~body_src
                        ~author:t.manifest.username
                    in
                    (match r with
                     | Error e ->
                         (Pricklypear_lisp.Ast.VStr ("stub-function: " ^ e))
                     | Ok _ ->
                         let () = record_define_side_effects t ~name in
                         (Pricklypear_lisp.Ast.VStr
                              ("stubbed " ^ name ^ "; "
                               ^ impact_summary_text t name)))))
      | _ ->
          (raise (Pricklypear_lisp.Ast.Interp_error
     (Pricklypear_lisp.Ast.Prim_error ("stub-function", "need 1 name",
     "bad-args")))))
```

98 image/lib/prims_hyper.ml

```
(** Pricklypear Image -- hypermedia harness shim.

    The soup/ tree prims + the host-agnostic helpers (assoc_of,
    render_handler_value) were lifted into the shared pure-prims layer
    [pure/prims_soup.ml] so both the image and the agent register the
    SAME tree prims (spec: hypermedia-harness.kernel-prims SHARED
    KERNEL). This file re-exports them so image call sites
    (Prims_hyper.register_pure_prims / assoc_of / render_handler_value)
    stay unchanged.

    The session-capturing prim route/dispatch stays registered INLINE in
    session.ml (register_hyper_route_dispatch, right after this module's
    register_pure_prims call) so it can close over Session.t without a
    dependency cycle -- mirrors register_crud_prims / register_net_prims. *)
include Prims_soup
```

99 image/lib/prims_media.ml

```
(** Prim registration extracted from session.ml (Eio-floor Phase 4). *)
open Session_types

let register (t : t) (tbl : Primitives.t) : unit =
  let deny_anon name =
    (* Raise, don't return a VStr: a returned string is silently dropped
       by (seq ?), so a multi-write handler could partially succeed past
       an anonymous denial. Raising aborts the whole eval.
    (BORGE-1785213376) *)
    raise
      (Pricklypear_lisp.Ast.Interp_error
         (Pricklypear_lisp.Ast.User_error
            (name ^ ": denied for anonymous", "bad-args")))
  in
  let actor () =
    match Request_context.get () with
    | Some c -> (Request_context.manifest c).username
    | None -> t.manifest.username
  in
  (* (media-meta id) -> VStr json | VNil *)
  Primitives.register tbl "media-meta"
    (fun args ->
      match args with
      | [ Pricklypear_lisp.Ast.VStr id ] ->
          let uname = actor () in
          if uname = "anonymous" then deny_anon "media-meta"
          else
          let m =
            Media.get_meta t.store ~id
              ~visible_to:(uname, t.is_admin) ()
          in
          (match m with
           | None -> Pricklypear_lisp.Ast.VNil
           | Some meta ->
               (Pricklypear_lisp.Ast.VStr (Media.meta_to_json_string meta)))
      | _ ->
          (raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("media-meta", "need id",
    "bad-args")))));
  (* (media-list [limit]) -> VList of meta json strings *)
  Primitives.register tbl "media-list"
    (fun args ->
      let uname = actor () in
      if uname = "anonymous" then deny_anon "media-list"
      else
        let limit =
          match args with
          | [] -> 50
          | [ Pricklypear_lisp.Ast.VInt n ] -> Int64.to_int n
          | _ -> 50
        in
        let rows =
          Media.list t.store ~author:(Some uname) ~limit ()
        in
        let items =
          List.map
            (fun m ->
              Pricklypear_lisp.Ast.VStr (Media.meta_to_json_string m))
            rows
        in
        (Pricklypear_lisp.Ast.VList items));
  (* (media-delete id) -> VStr id | VNil | denial *)
  Primitives.register tbl "media-delete"
    (fun args ->
      let uname = actor () in
      if uname = "anonymous" then deny_anon "media-delete"
      else
        match args with
        | [ Pricklypear_lisp.Ast.VStr id ] ->
            let res =
              Media.delete t.store ~id ~actor:uname ~is_admin:t.is_admin
```

```
            in
            (match res with
             | Ok (`Ok id') ->
                 (Pricklypear_lisp.Ast.VStr id')
             | Ok `Not_found -> Pricklypear_lisp.Ast.VNil
             | Ok `Forbidden ->
                 (Pricklypear_lisp.Ast.VStr
                      "media-delete: not author or admin")
             | Error e ->
                 (Pricklypear_lisp.Ast.VStr ("media-delete: " ^ e)))
        | _ ->
            (raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("media-delete", "need id",
   "bad-args")))));
  (* (media-put content-type data-b64 [filename]) -> VStr id | error
     Tiny payloads only (icons/fixtures). Real pushes: POST /api/media. *)
  Primitives.register tbl "media-put"
    (fun args ->
      let uname = actor () in
      if uname = "anonymous" then deny_anon "media-put"
      else
        let parse =
          match args with
          | [ Pricklypear_lisp.Ast.VStr ct
            ; Pricklypear_lisp.Ast.VStr b64 ] ->
              Ok (ct, b64, None)
          | [ Pricklypear_lisp.Ast.VStr ct
            ; Pricklypear_lisp.Ast.VStr b64
            ; Pricklypear_lisp.Ast.VStr filename ] ->
              Ok (ct, b64, Some filename)
          | _ -> Error "media-put: need content-type, data-b64 [, filename]"
        in
        match parse with
        | Error e -> (Pricklypear_lisp.Ast.VStr e)
        | Ok (ct, b64, filename) ->
            (match Media.b64_decode_opt (String.trim b64) with
             | None ->
                 (raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("media-put", "invalid data-b64",
   "bad-args"))))
             | Some data ->
                 let res =
                   Media.put t.store ~author:uname ~content_type:ct ~data
                     ~filename ~max_bytes:Media.max_eval_put_bytes ()
                 in
                 (match res with
                  | Ok r -> (Pricklypear_lisp.Ast.VStr r.id)
                  | Error e ->
                      (Pricklypear_lisp.Ast.VStr ("media-put: " ^ e)))));
  (* (media-url id) -> VStr "/media/<id>" *)
  Primitives.register tbl "media-url"
    (fun args ->
      match args with
      | [ Pricklypear_lisp.Ast.VStr id ] ->
          (Pricklypear_lisp.Ast.VStr (Media.url_of_id id))
      | _ ->
          (raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("media-url", "need id",
   "bad-args")))))
```

100 image/lib/prims_net.ml

```
(** Prim registration extracted from session.ml (Eio-floor Phase 4). *)
open Session_types

(* outbound-HTTP hardening (i-1787971188) ----------------------------- *)

(* Unique temp file created with perms 0o600 via Unix.openfile O_EXCL --
   never world-readable, no name/symlink race. Filename.temp_file is
   avoided on purpose: creation mode and perms stay explicit here. *)
let make_private_temp (prefix : string) (suffix : string) : string =
  let tmpdir = Filename.get_temp_dir_name () in
  let rec attempt n =
    let path =
      Filename.concat tmpdir
        (prefix ^ string_of_int (Unix.getpid ()) ^ "-" ^ string_of_int n
         ^ suffix)
    in
    (match
       (try
          Some
            (Unix.openfile path
               [ Unix.O_WRONLY; Unix.O_CREAT; Unix.O_EXCL ] 0o600)
        with Unix.Unix_error (Unix.EEXIST, _, _) -> None)
     with
     | Some fd -> Unix.close fd; path
     | None -> attempt (n + 1))
  in
  attempt 0

(* Host portion of a URL: strip the scheme ("//"), then path/query/
   fragment, userinfo (user:pass@host) and :port; un-bracket IPv6
   literals. Not a full RFC 3986 parser -- anything still malformed
   fails later in curl as a transient. *)
let url_host (url : string) : string =
  let cut_at s c =
    match String.index_opt s c with
    | Some i -> String.sub s 0 i
    | None -> s
  in
  let rec after_scheme i =
    if i + 3 > String.length url then None
    else if url.[i] = ':' && url.[i + 1] = '/' && url.[i + 2] = '/' then
      Some (i + 3)
    else after_scheme (i + 1)
  in
  let rest =
    match after_scheme 0 with
    | Some i -> String.sub url i (String.length url - i)
    | None -> url
  in
  let authority = cut_at (cut_at (cut_at rest '/') '?') '#' in
  let authority =
    match String.rindex_opt authority '@' with
    | Some i -> String.sub authority (i + 1) (String.length authority - i -
     1)
    | None -> authority
  in
  if String.length authority >= 2 && authority.[0] = '[' then
    (* bracketed IPv6 literal, optional :port after ']' *)
    (match String.index_opt authority ']' with
     | Some i when i > 1 -> String.sub authority 1 (i - 1)
     | _ -> authority)
  else cut_at authority ':'

(* Loopback / private / link-local space, v4 and v6. *)
let private_subnets : Ipaddr.Prefix.t list =
  List.map Ipaddr.Prefix.of_string_exn
    [ "127.0.0.0/8"      (* v4 loopback *)
    ; "10.0.0.0/8"       (* RFC 1918 private *)
    ; "172.16.0.0/12"    (* RFC 1918 private *)
    ; "192.168.0.0/16"   (* RFC 1918 private *)
    ; "169.254.0.0/16"   (* v4 link-local (incl. cloud metadata) *)
```

```
    ; "::1/128"          (* v6 loopback *)
    ; "fc00::/7"         (* v6 unique-local *)
    ; "fe80::/10"        (* v6 link-local *)
    ]

let addr_is_private (addr : Unix.inet_addr) : bool =
  match Ipaddr.of_string (Unix.string_of_inet_addr addr) with
  | Ok ip -> List.exists (Ipaddr.Prefix.mem ip) private_subnets
  | Error _ -> true (* unparseable: fail closed *)

(* True when the URL's host resolves into private space. Resolution
   failure is NOT a block: curl surfaces it as its own transient. *)
let egress_blocked (url : string) : bool =
  let host = url_host url in
  host <> ""
  && (match
        (try
           Unix.getaddrinfo host "80" [ Unix.AI_SOCKTYPE Unix.SOCK_STREAM ]
         with _ -> [])
      with
      | [] -> false
      | addrs ->
          List.exists
            (fun ai ->
               match ai.Unix.ai_addr with
               | Unix.ADDR_INET (a, _) -> addr_is_private a
               | Unix.ADDR_UNIX _ -> true)
            addrs)

(* Shared outbound-HTTP engine behind http-get (borge/image.borg
   primitives-layer) and http-post (borge/image.borg outbound-http).
   Same curl-backed shape, same payload-dict contract, same
   transient-vs-semantic discipline; [name] tags Prim_errors, [meth]
   is the HTTP verb, [body] is Some for send verbs (POST:
   --data-binary @file). Hardening (i-1787971188) covers both verbs:
   SSRF egress blocklist, 0600 O_EXCL temp files, Fun.protect
   cleanup. *)
let do_http (name : string)
    (meth : string)
    (body : string option)
    (url : string)
    (req_hdrs : (string * string) list)
    : Pricklypear_lisp.Ast.value =
  let open Pricklypear_lisp.Ast in
  let shell_quote = Llm_client.shell_quote in
  let timeout =
    List.assoc_opt "_timeout_ms" req_hdrs
    |> Option.map int_of_string
    |> Option.value ~default:20000
  in
  let real_hdrs =
    List.filter (fun (k, _) -> k <> "_timeout_ms") req_hdrs
  in
  let hdr_args =
    String.concat " "
      (List.map
         (fun (k, v) ->
            shell_quote (k ^ ": " ^ v))
         real_hdrs)
  in
  (* SSRF guard: refuse targets that resolve into loopback/private/
     link-local space so tenant lisp cannot reach internal services
     (DB, admin faces, cloud metadata). Opt-out: env
     PP_NET_EGRESS_UNRESTRICTED=1.

     KNOWN RESIDUAL (documented, accepted): only the URL's initial host
     is resolved here -- curl -L follows redirects itself, so a public
     URL that 3xx-redirects to a private address can still bounce past
     this blocklist. Closing it needs a local proxy or curl metadata
     hooks; out of scope for this fix. *)
  (match Sys.getenv_opt "PP_NET_EGRESS_UNRESTRICTED" with
   | Some "1" -> ()
   | _ ->
```

```
      if egress_blocked url then
        raise
          (Interp_error
             (Prim_error
                ( name
                , "blocked: target host resolves to a
   private/loopback/link-local address (egress restricted)"
                , "denied" ))));
  let hdr_tmp = ref "" and body_tmp = ref "" and req_tmp = ref "" in
  (* Fun.protect keeps hdr/body/req temp files off disk on every path,
     including exceptions from temp creation, curl and parsing. *)
  Fun.protect
    (fun () ->
      try
        hdr_tmp := make_private_temp "pp_http_" ".hdr";
        body_tmp := make_private_temp "pp_http_" ".body";
        (* Send-verb payload: write the body to a private temp file and
           hand curl --data-binary @file (never the command line). *)
        (match body with
         | Some b ->
             let req = make_private_temp "pp_http_" ".req" in
             req_tmp := req;
             let oc = open_out req in
             output_string oc b;
             close_out oc
         | None -> ());
        let meth_args =
          match body with
          | Some _ ->
              Printf.sprintf "-X %s --data-binary @%s" (shell_quote meth)
                (shell_quote !req_tmp)
          | None -> ""
        in
        let cmd =
          Printf.sprintf
            "curl -sS -L --max-redirs 5 -D %s -o %s --max-time %d %s %s %s
   2>&1"
            (shell_quote !hdr_tmp)
            (shell_quote !body_tmp)
            (max 1 (timeout / 1000))
            meth_args
            hdr_args
            (shell_quote url)
        in
        let stderr =
          Lwt_eio.run_lwt (fun () ->
            Lwt_process.pread ~timeout:(float_of_int timeout /. 1000. +. 5.)
              (Lwt_process.shell cmd))
        in
        let resp_body =
          try
            let ic = open_in !body_tmp in
            let n = in_channel_length ic in
            let s = really_input_string ic n in
            close_in ic; s
          with _ -> ""
        in
        let hdr_blob =
          try
            let ic = open_in !hdr_tmp in
            let n = in_channel_length ic in
            let s = really_input_string ic n in
            close_in ic; s
          with _ -> ""
        in
        (* status: parse the last HTTP/x.y NNN line (after redirects). *)
        let status =
          let lines = String.split_on_char '\n' hdr_blob in
          let rec last_status = function
            | [] -> 0
            | line :: rest ->
                let line = String.trim line in
                if String.length line >= 8
```

```
                && String.sub line 0 5 = "HTTP/"
             then
               (match String.split_on_char ' ' line with
                | _ :: n :: _ ->
                    (try int_of_string (String.trim n) with _ ->
  last_status rest)
                | _ -> last_status rest)
             else last_status rest
       in
       last_status lines
     in
     let headers_json = Prims_pure.http_parse_headers hdr_blob in
     (* D1: network-transient detection. curl writes errors to stderr
        (curl: (NN) ...) or returns non-zero. A status of 0 means no
        HTTP response was received ? transient network failure. *)
     let curl_err =
       if stderr = "" then ""
       else if String.length stderr > 5
               && (String.sub stderr 0 5 = "curl "
                   || String.sub stderr 0 5 = "curl:") then
         String.trim stderr
       else ""
     in
     if status = 0 && curl_err <> "" then
       (* Network transient: DNS, connection refused, timeout, etc.
          RAISE so guard handlers see a real error (design note ?4b). *)
       raise
         (Interp_error
            (Prim_error
               ( name
               , "transient: " ^ curl_err
               , "transient" )))
     else if status = 0 then
       (* No status AND no curl error -- still a transient (no
  response). *)
       raise
         (Interp_error
            (Prim_error
               ( name
               , "transient: no HTTP response"
               , "transient" )))
     else
       (* Semantic HTTP response (2xx/3xx/4xx/5xx). Return a payload
          dict with ok/err/kind so callers can branch without string
          parsing. 4xx/5xx are NOT raised -- they're real responses the
          caller may want to inspect (design note ?4b). *)
       let ok = status < 400 in
       let err = if ok then "" else Printf.sprintf "HTTP %d" status in
       let kind =
         if ok then "ok"
         else if status >= 500 then "http-server-error"
         else "http-client-error"
       in
       let result_json =
         let open Yojson.Safe in
         `Assoc
           [ ("ok", `Bool ok)
           ; ("status", `Int status)
           ; ("headers",
              (try from_string headers_json with _ -> `Null))
           ; ("body", `String resp_body)
           ; ("err", `String err)
           ; ("kind", `String kind) ]
       in
       (VStr (Yojson.Safe.to_string result_json))
   with exn ->
     match exn with
     | Interp_error _ as e -> raise e
     | _ ->
         raise
           (Interp_error
              (Prim_error
                 ( name
```

```
                    , "transient: " ^ Printexc.to_string exn
                    , "transient" ))))
    ~finally:(fun () ->
      (try if !hdr_tmp <> "" then Sys.remove !hdr_tmp with _ -> ());
      (try if !body_tmp <> "" then Sys.remove !body_tmp with _ -> ());
      (try if !req_tmp <> "" then Sys.remove !req_tmp with _ -> ()))

(* Compatibility alias: http-get engine under its pre-refactor name
   (session.ml binding + http_get_test.ml). *)
let do_http_get (url : string) (req_hdrs : (string * string) list) :
    Pricklypear_lisp.Ast.value =
  do_http "http-get" "GET" None url req_hdrs

let register (t : t) (tbl : Primitives.t) : unit =
  let deny_anon name =
    raise
      (Pricklypear_lisp.Ast.Interp_error
         (Pricklypear_lisp.Ast.User_error
            (name ^ ": denied for anonymous", "bad-args")))
  in
  (* (sleep seconds) -> VNil after delay. Host-owned concurrency surface
     (BORGE-1785211069). Caps at 60s per call so a runaway form cannot
     park the image forever. Uses Lwt_unix.sleep (yields; does not block
     the OS thread for the full duration under Lwt). *)
  Primitives.register tbl "sleep"
    (fun args ->
      let secs =
        match args with
        | [ Pricklypear_lisp.Ast.VInt n ] -> Int64.to_float n
        | [ Pricklypear_lisp.Ast.VFloat f ] -> f
        | [ Pricklypear_lisp.Ast.VStr s ] ->
            (try float_of_string (String.trim s) with _ -> -1.0)
        | _ -> -1.0
      in
      if secs < 0.0 then
        (raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("sleep", "need non-negative
    seconds", "bad-args"))))
      else
        let secs = min secs 60.0 in
        let () = Lwt_eio.run_lwt (fun () -> Lwt_unix.sleep secs) in
        Pricklypear_lisp.Ast.VNil);
  let is_admin_or_agent () =
    let uname =
      match Request_context.get () with
      | Some c -> (Request_context.manifest c).username
      | None -> t.manifest.username
    in
    t.is_admin || String.ends_with ~suffix:"/agent" uname
  in
  (* Identity gate shared by http-get / http-post (image.outbound-http):
     SSRF confinement is by CALLER, not by destination allowlist. *)
  let gate_http name =
    let uname =
      match Request_context.get () with
      | Some c -> (Request_context.manifest c).username
      | None -> t.manifest.username
    in
    if uname = "anonymous" then deny_anon name
    else if not (is_admin_or_agent ()) then
      (raise (Pricklypear_lisp.Ast.Interp_error
                (Pricklypear_lisp.Ast.Prim_error
                   (name, "denied (admin/agent/human only)", "denied"))))
  in
  (* headers-json -> (string * string) list; same Assoc format as the
     payload's "headers" field. Unknown shapes parse to []. *)
  let hdrs_of_json (hdrs_json : string) : (string * string) list =
    (try
       let j = Yojson.Safe.from_string hdrs_json in
       (match j with
        | `Assoc kvs ->
            List.filter_map
              (fun (k, v) ->
```

```
                match v with
                | `String s -> Some (k, s)
                | `Int n -> Some (k, string_of_int n)
                | _ -> None)
            kvs
      | _ -> [])
   with _ -> [])
in
Primitives.register tbl "http-get"
  (fun args ->
    gate_http "http-get";
    match args with
    | [ Pricklypear_lisp.Ast.VStr url ] ->
        do_http "http-get" "GET" None url []
    | [ Pricklypear_lisp.Ast.VStr url; Pricklypear_lisp.Ast.VStr
   hdrs_json ] ->
        do_http "http-get" "GET" None url (hdrs_of_json hdrs_json)
    | _ ->
        (raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("http-get", "need url
   [headers-json]", "bad-args")))));
(* (http-post url [headers-json [body]]) -> payload dict VStr
   {ok,status,headers,body,err,kind}. Same gate + contract as
   http-get (borge/image.borg outbound-http). 4xx/5xx RETURN; curl
   transport failure RAISES transient so jobs backoff sees it. *)
Primitives.register tbl "http-post"
  (fun args ->
    gate_http "http-post";
    match args with
    | [ Pricklypear_lisp.Ast.VStr url ] ->
        do_http "http-post" "POST" (Some "") url []
    | [ Pricklypear_lisp.Ast.VStr url; Pricklypear_lisp.Ast.VStr
   hdrs_json ] ->
        do_http "http-post" "POST" (Some "") url (hdrs_of_json hdrs_json)
    | [ Pricklypear_lisp.Ast.VStr url; Pricklypear_lisp.Ast.VStr
   hdrs_json;
        Pricklypear_lisp.Ast.VStr body ] ->
        do_http "http-post" "POST" (Some body) url (hdrs_of_json
   hdrs_json)
    | _ ->
        (raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("http-post", "need url [headers-json
   [body]]", "bad-args")))))
```

101 image/lib/prims_request.ml

```
(** Prim registration extracted from session.ml (Eio-floor Phase 4). *)
open Session_types

let method_to_string (m : http_method) : string =
  match m with `GET -> "GET" | `POST -> "POST"

let require_request (t : t) : request =
  match t.current_request with
  | Some r -> r
  | None ->
      raise
        (Pricklypear_lisp.Ast.Interp_error
           (Pricklypear_lisp.Ast.User_error ("no active request",
    "bad-args")))

let str_name = function
  | Pricklypear_lisp.Ast.VStr s | Pricklypear_lisp.Ast.VSymbol s -> Some s
  | _ -> None

let depth_arg = function
  | None -> 1
  | Some (Pricklypear_lisp.Ast.VInt n) -> Int64.to_int n
  | Some (Pricklypear_lisp.Ast.VStr s) ->
      (try int_of_string s with _ -> 1)
  | Some _ -> 1

let vlist_strs xs =
  Pricklypear_lisp.Ast.VList (List.map (fun s -> Pricklypear_lisp.Ast.VStr
    s) xs)

let impact_to_json (e : impact_event) : string =
  let open Yojson.Safe in
  to_string
    (`Assoc
       [ ("name", `String e.name)
       ; ("at", `Float e.at)
       ; ("branch", `String e.branch)
       ; ("author", `String e.author)
       ; ("dependents", `List (List.map (fun s -> `String s) e.dependents))
       ; ("routes", `List (List.map (fun s -> `String s) e.routes))
       ; ("status", `String e.status)
       ])

let change_to_json (e : change_event) : string =
  Yojson.Safe.to_string
    (`Assoc
       [ ("op", `String e.op)
       ; ("name", `String e.name)
       ; ("at", `Float e.at)
       ; ("branch", `String e.branch)
       ; ("author", `String e.author)
       ])

let record_define_side_effects = Call_graph.record_define_side_effects
let impact_summary_text = Call_graph.impact_summary_text


let register (t : t) (tbl : Primitives.t) : unit =
  Primitives.register_sync tbl "request-method" (fun _args ->
      let r = require_request t in
      Pricklypear_lisp.Ast.VStr (method_to_string r.method_));
  Primitives.register_sync tbl "request-path" (fun _args ->
      let r = require_request t in
      Pricklypear_lisp.Ast.VStr r.path);
  Primitives.register_sync tbl "request-body" (fun _args ->
      let r = require_request t in
      Pricklypear_lisp.Ast.VStr r.body);
  Primitives.register_sync tbl "request-param" (fun args ->
      match args with
      | [ Pricklypear_lisp.Ast.VStr key ] ->
          let r = require_request t in
```

```
70          (match List.assoc_opt key r.params with
71           | Some v -> Pricklypear_lisp.Ast.VStr v
72           | None -> Pricklypear_lisp.Ast.VNil)
73      | _ ->
74          raise
75            (Pricklypear_lisp.Ast.Interp_error
76               (Pricklypear_lisp.Ast.User_error ("request-param needs 1
    string", "bad-args"))));
77    Primitives.register_sync tbl "request-header" (fun args ->
78        match args with
79        | [ Pricklypear_lisp.Ast.VStr key ] ->
80            let r = require_request t in
81            let k = String.lowercase_ascii key in
82            (match List.assoc_opt k r.headers with
83             | Some v -> Pricklypear_lisp.Ast.VStr v
84             | None -> Pricklypear_lisp.Ast.VNil)
85        | _ ->
86            raise
87              (Pricklypear_lisp.Ast.Interp_error
88                 (Pricklypear_lisp.Ast.User_error ("request-header needs 1
    string", "bad-args"))));
89    Primitives.register_sync tbl "hx-request?" (fun _args ->
90        let r = require_request t in
91        match List.assoc_opt "hx-request" r.headers with
92        | Some v when v <> "" && v <> "false" -> Pricklypear_lisp.Ast.VBool
    true
93        | _ -> Pricklypear_lisp.Ast.VBool false);
94    Primitives.register_sync tbl "request-params" (fun _args ->
95        let r = require_request t in
96        let j =
97          `Assoc
98            (List.map (fun (k, v) -> (k, `String v)) r.params)
99        in
100       Pricklypear_lisp.Ast.VStr (Yojson.Safe.to_string j));
101   Primitives.register_sync tbl "redirect" (fun args ->
102       match args with
103       | [ Pricklypear_lisp.Ast.VStr path ] ->
104           if String.length path = 0 || path.[0] <> '/' then
105             raise
106               (Pricklypear_lisp.Ast.Interp_error
107                  (Pricklypear_lisp.Ast.User_error
108                     ("redirect path must be absolute on host (start with
    /)", "bad-args")))
109           else if String.contains path ':' then
110             raise
111               (Pricklypear_lisp.Ast.Interp_error
112                  (Pricklypear_lisp.Ast.User_error
113                     ("redirect rejects off-site URLs", "bad-args")))
114           else Primitives.redirect_value path
115       | _ ->
116           raise
117             (Pricklypear_lisp.Ast.Interp_error
118                (Pricklypear_lisp.Ast.User_error ("redirect needs 1 string
    path", "bad-args"))))
```

102 image/lib/prims_time.ml

```
(** Wall-clock / calendar primitives.

    These read the system clock ([Unix]) so they are NOT portable to the
    browser client and live in the image only. Split out of
    [Prims_pure] so the pure-prims library stays Unix-free and can be
    shared with the js_of_ocaml client target. Spec:
    borge/structural-editing.borg client-eval (PURE PRIM TABLE ONLY). *)
open Pricklypear_lisp.Ast

(* (date-today) -> VStr "YYYY-MM-DD" in local time. *)
let prim_date_today (_args : value list) : value =
  let tm = Unix.localtime (Unix.time ()) in
  VStr
    (Printf.sprintf "%04d-%02d-%02d"
       (tm.Unix.tm_year + 1900) (tm.Unix.tm_mon + 1) tm.Unix.tm_mday)

(** (now) -> VInt unix epoch seconds (UTC). Wall clock for apps;
    pairs with date-today for calendar day. BORGE-1785211069. *)
let prim_now (_args : value list) : value =
  VInt (Int64.of_float (Unix.gettimeofday ()))

(** (now-ms) -> VInt unix epoch milliseconds. *)
let prim_now_ms (_args : value list) : value =
  VInt (Int64.of_float (Unix.gettimeofday () *. 1000.0))

(* (date-weekday y m d) -> VInt 0=Sunday .. 6=Saturday (local). *)
let prim_date_weekday (args : value list) : value =
  match args with
  | [ VInt y; VInt m; VInt d ] ->
      let y = Int64.to_int y and m = Int64.to_int m and d = Int64.to_int d
    in
      let tm =
        { Unix.tm_sec = 0
        ; tm_min = 0
        ; tm_hour = 12
        ; tm_mday = d
        ; tm_mon = m - 1
        ; tm_year = y - 1900
        ; tm_wday = 0
        ; tm_yday = 0
        ; tm_isdst = false
        }
      in
      let _, tm' = Unix.mktime tm in
      VInt (Int64.of_int tm'.Unix.tm_wday)
  | _ ->
      raise (Interp_error (User_error ("date-weekday needs year month day
    ints", "bad-args")))

(* (date-format y m d) -> VStr "YYYY-MM-DD". *)
let prim_date_format (args : value list) : value =
  match args with
  | [ VInt y; VInt m; VInt d ] ->
      VStr
        (Printf.sprintf "%04d-%02d-%02d"
           (Int64.to_int y) (Int64.to_int m) (Int64.to_int d))
  | _ -> raise (Interp_error (User_error ("date-format needs year month
    day", "bad-args")))

(** (date-add-days "YYYY-MM-DD" n) | (date-add-days n) with today.
    Local calendar day arithmetic via Unix.mktime (flashcards SRS). *)
let prim_date_add_days (args : value list) : value =
  let add_from_ymd y m d n =
    let tm =
      { Unix.tm_sec = 0
      ; tm_min = 0
      ; tm_hour = 12
      ; tm_mday = d + n
      ; tm_mon = m - 1
      ; tm_year = y - 1900
      ; tm_wday = 0
```

```
      ; tm_yday = 0
      ; tm_isdst = false
      }
    in
    let _, tm' = Unix.mktime tm in
    VStr
      (Printf.sprintf "%04d-%02d-%02d"
         (tm'.Unix.tm_year + 1900)
         (tm'.Unix.tm_mon + 1)
         tm'.Unix.tm_mday)
  in
  let parse_ymd s =
    let s = String.trim s in
    if String.length s < 10 then None
    else
      try
        let y = int_of_string (String.sub s 0 4) in
        let m = int_of_string (String.sub s 5 2) in
        let d = int_of_string (String.sub s 8 2) in
        Some (y, m, d)
      with _ -> None
  in
  match args with
  | [ VStr day; VInt n ] ->
      (match parse_ymd day with
       | Some (y, m, d) -> add_from_ymd y m d (Int64.to_int n)
       | None ->
           raise (Interp_error (User_error ("date-add-days: bad date
   string", "bad-args"))))
  | [ VInt n ] ->
      let tm = Unix.localtime (Unix.time ()) in
      add_from_ymd
        (tm.Unix.tm_year + 1900) (tm.Unix.tm_mon + 1) tm.Unix.tm_mday
        (Int64.to_int n)
  | [ VStr day; VStr n ] ->
      (match parse_ymd day, int_of_string_opt (String.trim n) with
       | Some (y, m, d), Some n -> add_from_ymd y m d n
       | _ ->
           raise (Interp_error (User_error ("date-add-days: bad args",
    "bad-args"))))
  | _ ->
      raise
        (Interp_error
           (User_error ("date-add-days needs (day n) or (n) for today+n",
    "bad-args")))

(** (date-diff-days a b) -> VInt days (b - a), local calendar.
    Args are YYYY-MM-DD strings. *)
let prim_date_diff_days (args : value list) : value =
  let parse s =
    let s = String.trim s in
    if String.length s < 10 then None
    else
      try
        Some
          ( int_of_string (String.sub s 0 4)
          , int_of_string (String.sub s 5 2)
          , int_of_string (String.sub s 8 2) )
      with _ -> None
  in
  let epoch_days y m d =
    let tm =
      { Unix.tm_sec = 0; tm_min = 0; tm_hour = 12; tm_mday = d
      ; tm_mon = m - 1; tm_year = y - 1900; tm_wday = 0; tm_yday = 0
      ; tm_isdst = false }
    in
    let t, _ = Unix.mktime tm in
    t /. 86400.
  in
  match args with
  | [ VStr a; VStr b ] ->
      (match parse a, parse b with
       | Some (y1, m1, d1), Some (y2, m2, d2) ->
```

```
          let n =
            int_of_float
              (Float.round (epoch_days y2 m2 d2 -. epoch_days y1 m1 d1))
          in
          VInt (Int64.of_int n)
      | _ ->
          raise (Interp_error (User_error ("date-diff-days: bad date",
    "bad-args"))))
  | _ ->
      raise
        (Interp_error
          (User_error ("date-diff-days needs two YYYY-MM-DD strings",
    "bad-args")))
```

103 image/lib/repl_endpoint.ml

```
(** Pricklypear Image -- optional TCP REPL endpoint.

    Loopback line-protocol REPL. Still Lwt_unix sockets; boot runs this
    under Lwt_eio.run_lwt from an Eio fiber. Primary interactive surfaces
    remain browser /repl and POST /api/eval. *)
open Lwt.Syntax

let default_port = 7878

let pp_error (e : Pricklypear_lisp.Ast.interp_error) : string =
  let open Pricklypear_lisp.Ast in
  match e with
  | Unbound_var s -> Printf.sprintf "unbound variable: %s" s
  | Not_a_function v ->
      Printf.sprintf "not a function: %s" (Pricklypear_lisp.Reader.pp_value
     v)
  | Arity_mismatch (n, e_doc, g) ->
      Printf.sprintf "arity mismatch in %s: expected %d, got %d" n e_doc g
  | Wrong_arg_count (n, e_doc, g) ->
      Printf.sprintf "wrong arg count in %s: expected %d, got %d" n e_doc g
  | Type_mismatch (s, _) -> Printf.sprintf "type mismatch: %s" s
  | Budget_exceeded -> "budget exceeded"
  | Prim_error (n, m, _) -> Printf.sprintf "primitive %s: %s" n m
  | Parse_error (msg, l, c) ->
      Printf.sprintf "parse error: %s (line %d, col %d)" msg l c
  | User_error (m, _) -> Printf.sprintf "user error: %s" m

let prompt (s : Session.t) : string =
  Printf.sprintf "%s> " (Session.branch_name s)

let repl_turn ~(session : Session.t) ~(ic : Lwt_io.input_channel)
    ~(oc : Lwt_io.output_channel) : bool Lwt.t =
  let* () = Lwt_io.write oc (prompt session) in
  let* () = Lwt_io.flush oc in
  let* line = Lwt_io.read_line_opt ic in
  match line with
  | None -> Lwt.return false
  | Some "" -> Lwt.return true
  | Some input -> (
      match input with
      | ":quit" | ":q" -> Lwt.return false
      | _ ->
          Lwt.catch
            (fun () ->
              let result = Session.eval_one session input in
              let output = Pricklypear_lisp.Reader.pp_value result in
              let* () = Lwt_io.write oc (output ^ "\n") in
              let* () = Lwt_io.flush oc in
              Lwt.return true)
            (fun exn ->
              let msg =
                match exn with
                | Pricklypear_lisp.Ast.Interp_error e -> pp_error e
                | _ -> Printexc.to_string exn
              in
              let* () = Lwt_io.write oc ("error: " ^ msg ^ "\n") in
              let* () = Lwt_io.flush oc in
              Lwt.return true))

let serve ~(session : Session.t) ?(port = default_port) () : unit Lwt.t =
  let addr = Lwt_unix.ADDR_INET (Unix.inet_addr_loopback, port) in
  let sock = Lwt_unix.socket Lwt_unix.PF_INET Lwt_unix.SOCK_STREAM 0 in
  Lwt_unix.setsockopt sock Lwt_unix.SO_REUSEADDR true;
  let* () = Lwt_unix.bind sock addr in
  Lwt_unix.listen sock 5;
  Printf.eprintf "[repl] listening on 127.0.0.1:%d\n" port;
  flush stderr;
  let rec loop () =
    let* fd, _peer = Lwt_unix.accept sock in
    let ic = Lwt_io.of_fd ~mode:Lwt_io.Input fd in
    let oc = Lwt_io.of_fd ~mode:Lwt_io.Output fd in
```

```
    let* () =
      Lwt_io.write oc "Pricklypear REPL. Type :quit to exit.\n"
    in
    let* () = Lwt_io.flush oc in
    let rec turn_loop () =
      let* cont = repl_turn ~session ~ic ~oc in
      if cont then turn_loop () else Lwt.return_unit
    in
    let* () =
      Lwt.catch turn_loop (fun exn ->
          (match exn with
           | Unix.Unix_error (Unix.EBADF, _, _) ->
               Printf.eprintf "[repl] peer disconnected\n"
           | _ ->
               Printf.eprintf "[repl] connection error: %s\n"
                 (Printexc.to_string exn));
          flush stderr;
          Lwt.return_unit)
    in
    let* () = Lwt.catch (fun () -> Lwt_io.close ic) (fun _ ->
     Lwt.return_unit) in
    let* () = Lwt.catch (fun () -> Lwt_io.close oc) (fun _ ->
     Lwt.return_unit) in
    let* () =
      Lwt.catch (fun () -> Lwt_unix.close fd) (fun _ -> Lwt.return_unit)
    in
    loop ()
  in
  loop ()
```

104 image/lib/request_context.ml

```
(** Per-request context for lock-free page work (fixes BORGE-1784434193
    cotenant race).

    After the identity gate (request_mutex), a fresh Request_context.t is
    built from the attached Session state and passed to all subsequent
    page work (route handlers, /api/eval, prims, render). This prevents
    anonymous attach + checkout_to + rehydrate from flipping
    manifest/env/routes
    under an in-flight personal-branch page.

    Phase 1: only manifest is isolated (the cotenant username/branch
    fields).
    Later phases will move env, routes, and store into the context.
*)

open Pricklypear_lisp
module Manifest = Pricklypear_common.Manifest

type t = {
  manifest : Manifest.t;
  env : Pricklypear_lisp.Ast.env;
  routes : (string, string * string) Hashtbl.t;
  (* store : Store.t;                        -- Phase 3 *)
}

let make ~manifest ~env ~routes : t =
  { manifest; env; routes }

let manifest (t : t) = t.manifest

let env (t : t) = t.env

let routes (t : t) = t.routes

(** Snapshot the live session routes table by COPYING it, so a concurrent
    rehydrate (Hashtbl.clear t.routes) during this request's page work
    cannot
    mutate the snapshot. The env and manifest fields are swapped by
    reference
    on rehydrate/attach (new records, old stable), so holding the reference
    IS a snapshot; routes is in-place-cleared so it must be copied. *)
let copy_routes (src : (string, string * string) Hashtbl.t)
  : (string, string * string) Hashtbl.t =
  Hashtbl.copy src

(** Sync route lookup against the snapshot. Returns None if not registered
    on the snapshot's branch. Used by HTTP dispatch after the gate, where a
    concurrent attach must not flip the table under us. *)
let lookup_route (ctx : t) (route : string) : string option =
  match Hashtbl.find_opt ctx.routes route with
  | Some (handler, _) -> Some handler
  | None -> None

let lookup_route_meta (ctx : t) (route : string) : (string * string) option
    =
  Hashtbl.find_opt ctx.routes route

(** Sync env lookup against the snapshot (for handler resolution). *)
let lookup_handler (ctx : t) (name : string)
  : Pricklypear_lisp.Ast.value option =
  Pricklypear_lisp.Env.lookup (ctx.env : Pricklypear_lisp.Ast.env) name

(** Fiber-local binding for the current request context (Eio floor).
    Set by the HTTP handler after the identity gate; read by prims
    that need stable manifest (whoami, current-branch, author checks, etc.).
    This avoids the cotenant race without changing the prim_fn ABI. *)
let current_key : t Eio.Fiber.key = Eio.Fiber.create_key ()

let get () : t option = Eio.Fiber.get current_key

let with_current (ctx : t) (f : unit -> 'a) : 'a =
```

```
66    (* Also bind Store_db.pp_user so SET LOCAL pp.user sees this request's
67       identity inside with_transaction (user-scoped data / RLS floor). *)
68    Eio.Fiber.with_binding current_key ctx (fun () ->
69        Store_db.with_pp_user ctx.manifest.username f)
```

105 image/lib/session.ml

```
(** Pricklypear Image -- session state for eval / HTTP / REPL.

    Holds the env (function defs for the current branch, plus any
    (define)'d locals), the boot manifest (default identity), auth
    attachment fields, and the store handle.

    Per spec [image.repl-experience]:
      - (define ...) persists to the store: survives image reboot.
      - REPL-local let bindings are session-local: NOT persisted.
      - Switching branch (checkout) keeps REPL locals but changes
     visibility. *)
include Session_types

(** Run direct-style eval under the Lwt prim rim (H1.7). *)
let eval_under_rim (t : t) (thunk : unit -> 'a) : 'a =
  Control_rim.run ~prims:t.prims thunk

let persist_last_budget_stats (t : t) ~(label : string) : unit =
  match !Pricklypear_lisp.Eval.last_budget with
  | None -> ()
  | Some b when label = "" -> ()
  | Some b ->
      (try
         let fields =
           Yojson.Safe.to_string
             (`Assoc
                [
                  ("fn", `String label);
                  ("steps", `Int b.steps);
                  ("prim_calls", `Int b.prim_calls);
                  ("db_hits", `Int b.db_hits);
                  ("when", `String (string_of_float (Unix.gettimeofday
    ())));
                ])
         in
         (* image.function-stat-ring: push, no synchronous DB write. *)
         Function_stats.push ~fn:label ~fields_json:fields
           ~author:t.manifest.username
       with _ -> ())

let eval_string_rim (t : t) (input : string) : Pricklypear_lisp.Ast.value =
  let v =
    eval_under_rim t (fun () ->
        Pricklypear_lisp.Eval.eval_string_direct
          ~prims:Pricklypear_lisp.Eval.empty_prims t.env input)
  in
  (* Best-effort label: top-level (name) call *)
  let label =
    let s = String.trim input in
    if String.length s >= 3 && s.[0] = '(' && s.[String.length s - 1] = ')'
    then
      let inner = String.sub s 1 (String.length s - 2) |> String.trim in
      match String.split_on_char ' ' inner with
      | name :: _ when name <> "" && name.[0] <> '(' -> name
      | _ -> ""
    else ""
  in
  (* image.product-theme: shell evals cause ZERO DB WRITES. The shell
     refresh runs in the request gate; persisting budget stats for it
     would turn every shell re-render into a serialized Crud.create
     under the pgx sequencer. Skip stats for shell labels. *)
  if label <> "" && label <> "ui/css" && label <> "ui/nav"
     && label <> "ui/home" && label <> "ui/theme"
  then persist_last_budget_stats t ~label;
  v

(* habitat-book.eval-examples: shared engine for the (example-eval ?)
   prim and the POST /book/eval-example endpoint. Restriction lives in
   Example_eval (whitelisted prim table + static gate + child env +
   fresh budget); known_fn comes from the branch's visible defs -- the
   same body source forge-effects uses. *)
```

```
let example_eval_engine (t : t) (src : string) : Pricklypear_lisp.Ast.value =
  let known_fn =
    Pricklypear_lisp.Effect.known_fn_of_bindings
      (List.map (fun (n, e, _) -> (n, e))
         (Store.list_function_names t.store))
  in
  Example_eval.eval_source ~env:t.env ~full_prims:t.prims ~known_fn src

let eval_expr_rim (t : t) (expr : Pricklypear_lisp.Ast.expr)
  : Pricklypear_lisp.Ast.value =
  let budget = Pricklypear_lisp.Eval.make_budget () in
  Pricklypear_lisp.Eval.last_budget := Some budget;
  eval_under_rim t (fun () ->
      Pricklypear_lisp.Eval.eval
        ~budget ~prims:Pricklypear_lisp.Eval.empty_prims t.env expr)

let load_defs (t : t) : unit =
  let disabled = Store.disabled_package_names t.store in
  let disabled_set = Hashtbl.create 16 in
  List.iter (fun p -> Hashtbl.replace disabled_set p ()) disabled;
  (** Hide when package provenance is disabled, or (legacy untagged) when
      the binding name is under a disabled package prefix. *)
  let name_disabled name =
    Hashtbl.fold
      (fun d _ acc ->
        acc
        || name = d
        || String.starts_with ~prefix:(d ^ "/") name)
      disabled_set false
  in
  let binding_disabled name pkg =
    match pkg with
    | Some p -> Hashtbl.mem disabled_set p || name_disabled name
    | None -> name_disabled name
  in
  let defs = Store.list_function_names_for_load t.store in
  (* Kernel prim names already bound as VPrim on rehydrate. Never let a
     stale function_defs row re-poison them (BORGE-1785213238). *)
  let is_kernel_prim name =
    List.exists (fun (n, _) -> n = name) t.kernel_catalog
  in
  let _ =
    List.iter
      (fun (name, body, _author, pkg) ->
        if binding_disabled name pkg then ()
        else if is_kernel_prim name then begin
          Printf.eprintf
            "[session] load def %s skipped: kernel prim name (shadow blocked)\n%!"
            name;
          ()
        end else
        (* Always (re)bind -- parents first, children last ? override.
           image.lazy-env-load: non-macro bodies bind as VTHUNKS -- the
           body is NOT evaluated at load. First lookup forces (once,
           memoized, failure too) under the session rim + active budget.
           Macros keep the eager path: VMacro construction is pure. *)
        (match body with
              | Pricklypear_lisp.Ast.App
                  (Pricklypear_lisp.Ast.Var "__macro__",
                   [ params_expr; body_expr ]) ->
                  let params =
                    match params_expr with
                    | Pricklypear_lisp.Ast.Lit Pricklypear_lisp.Ast.VNil -> []
                    | Pricklypear_lisp.Ast.App (Pricklypear_lisp.Ast.Var p, rest) ->
                        p
                        :: List.filter_map
                             (function
                               | Pricklypear_lisp.Ast.Var s -> Some s
                               | _ -> None)
```

```
                           rest
                  | _ -> []
                in
                Pricklypear_lisp.Env.define t.env name
                  (Pricklypear_lisp.Ast.VMacro (params, body_expr,
    t.env));
                ()
            | _ ->
                let force env' expr' =
                  eval_under_rim t (fun () ->
                      let b =
                        match !Pricklypear_lisp.Eval.current_budget with
                        | Some b -> b
                        | None -> Pricklypear_lisp.Eval.make_budget ()
                      in
                      Pricklypear_lisp.Eval.eval ~budget:b
                        ~prims:Pricklypear_lisp.Eval.empty_prims env'
    expr')
                in
                Pricklypear_lisp.Env.define_thunk t.env name ~force body))
      defs
  in
  (* Also load view_bindings into the in-memory routes table. The
     recursive-CTE returns ancestors first, so a route defined in both
     the child and parent: child's wins (Hashtbl.replace on the
     first-seen only -- skip further matches). Skip routes whose handler
     belongs to a disabled package. *)
  let pkg_of_fn = Store.function_package_map t.store in
  let routes = Store.list_routes t.store in
  let seen_r = Hashtbl.create 16 in
  List.iter
    (fun (route, handler, visibility) ->
      let handler_pkg = Hashtbl.find_opt pkg_of_fn handler in
      if binding_disabled handler handler_pkg then ()
      else if not (Hashtbl.mem seen_r route) then begin
        Hashtbl.replace seen_r route ();
        Hashtbl.replace t.routes route (handler, visibility)
      end)
    routes;
  ()

let load_defs_async (t : t) : unit =
  load_defs t;
  (* image.per-identity-sessions: stamp AFTER a successful load so the
     attach-time staleness check has a baseline. *)
  t.load_stamp <- Store.visible_stamp t.store

(** Clear env + routes and reload from the store for the current branch.
    Re-binds kernel prims from [t.kernel_catalog] first so checkout does
    not lose +/define/load-library/etc. *)
let rehydrate (t : t) : unit =
  (* image.http-robustness invariant 4a: build a FRESH env record and
     SWAP t.env to it. Do NOT Hashtbl.clear the old env's bindings --
     VLambda closures capture `env` by reference, so clearing in place
     would empty the binding table an in-flight page (holding the old
     env via its captured closures) is still reading. The old record
     stays alive for any closure that captured it; new requests see
     the fresh env. This makes concurrent rehydrate safe to run
     alongside lock-free page work (invariant 4b). *)
  let fresh =
    match t.env.parent with
    | None -> Pricklypear_lisp.Env.create ()
    | Some p -> Pricklypear_lisp.Env.create ~parent:(Some p) ()
  in
  t.env <- fresh;
  Hashtbl.clear t.routes;
  List.iter
    (fun (name, _) ->
      Pricklypear_lisp.Env.define t.env name
        (Pricklypear_lisp.Ast.VPrim name))
    t.kernel_catalog;
  Pricklypear_lisp.Env.define t.env "load-string"
    (Pricklypear_lisp.Ast.VPrim "load-string");
```

```
  Pricklypear_lisp.Env.define t.env "eval-string"
    (Pricklypear_lisp.Ast.VPrim "eval-string");
  Pricklypear_lisp.Env.define t.env "load-library"
    (Pricklypear_lisp.Ast.VPrim "load-library");
  (* lisp-interpreter.math: pi is a bound value, not a zero-arg prim.
     Rebind on every rehydrate so /api/eval sessions see it. *)
  Pricklypear_lisp.Env.define t.env "pi"
    (Pricklypear_lisp.Ast.VFloat Float.pi);
  load_defs t;
  t.load_stamp <- Store.visible_stamp t.store

(** Rehydrate iff the visible def chain changed since the last load.
    Called by the session registry at attach time; with image.lazy-env-load
    the rehydrate itself is a CTE + thunk binds (no body evals), so the
    whole check stays in the low-ms range. *)
let ensure_fresh (t : t) : unit =
  match Store.visible_stamp t.store with
  | "" -> ()  (* no DB: stay on whatever is in memory *)
  | stamp when stamp = t.load_stamp -> ()
  | _ -> rehydrate t

(** Force the next refresh_ui_shell to re-eval ui/nav + ui/css.
    image.product-theme: invalidation is DEPENDENCY-TRACKED -- a define
    busts iff the defined name was in the last shell eval's recorded
    read-set (see refresh_ui_shell); no TTL remains. Checkout still
    busts unconditionally (read-sets are per-branch), as do
    load-library / reseed (bulk redefines). *)
let bust_ui_shell_cache (t : t) : unit =
  t.ui_nav_html <- None;
  t.ui_css <- None;
  t.ui_shell_reads <- [];
  t.ui_shell_retry_at <- 0.0

(** image.product-theme: true when [name] was read by the last shell
    eval. Conservative when no read-set exists yet (never refreshed):
    fall back to the old shell-prefix set so first-refresh still
    triggers on shell-name defines. *)
let touches_shell (t : t) (name : string) : bool =
  match t.ui_shell_reads with
  | [] -> name = "ui/nav" || name = "ui/home" || name = "ui/css"
          || name = "ui/theme"
          || String.starts_with ~prefix:"theme/" name
  | reads -> List.exists (String.equal name) reads

(** Switch store + manifest to [branch_id], then rehydrate env. *)
let checkout_to (t : t) ~(branch_id : string) ~(branch_ref : string)
  : unit =
  let () = Store.checkout_branch t.store ~new_branch_id:branch_id in
  t.manifest <-
    { t.manifest with
      Pricklypear_common.Manifest.branch_id = branch_id;
      branch_name = branch_ref };
  (* Drop shell caches -- ui/nav and ui/css are branch-visible. Next
     refresh_ui_shell (request launch / load) repopulates from new env.
     Reset the invariant-3 retry/refresh timestamps too. *)
  bust_ui_shell_cache t;
  rehydrate t

let split_branch_ref (s : string) : string * string =
  match String.split_on_char '/' s with
  | [ o; n ] -> o, n
  | [ n ] -> "system", n  (* bare name = system/<name> *)
  | _ -> "system", s

(** Branch-owner authorization (i-1787970640, spec branching.personal-
    workspace): a session with username U (admin = t.is_admin || U =
    "admin") may target a branch owner O iff O = U or the session is
    admin. Agent users are named "<parent>/agent" and work in the
    parent-owned <parent>/agent workshop branch, so their tenant is the
    parent username. *)
let branch_owner_authorized (t : t) (owner : string) : bool =
  let u = t.manifest.username in
  if t.is_admin || u = "admin" then true
```

```
  else
    let tenant =
      let suffix = "/agent" in
      let n = String.length u and m = String.length suffix in
      if n > m && String.sub u (n - m) m = suffix then
        String.sub u 0 (n - m)
      else u
    in
    owner = u || owner = tenant

let register_session_control_prims (t : t) (tbl : Primitives.t) : unit =
  (* (fork 'parent 'child) -> VStr new-branch-id | VStr "no such parent" *)
  Primitives.register_server_only tbl "fork"
    (fun args ->
      if t.manifest.username = "anonymous" then
        (raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("fork", "denied for anonymous",
    "denied"))))
      else if t.agent_branch_lock then
        (Pricklypear_lisp.Ast.VStr
             "fork blocked: agent is locked to the current branch")
      else
      match args with
      | [ Pricklypear_lisp.Ast.VStr parent_name;
          Pricklypear_lisp.Ast.VStr child_name ] ->
          let parent_owner, parent_short = split_branch_ref parent_name in
          let child_owner, child_short = split_branch_ref child_name in
          let parent_id =
            Store.lookup_branch_id_by_name
              t.store ~owner:parent_owner ~name:parent_short
          in
          (match parent_id with
           | None ->
               (Pricklypear_lisp.Ast.VStr
                           (Printf.sprintf "no such parent: %s"
    parent_name))
           | Some pid ->
               (* i-1787970640: child_owner must pass the branch-owner rule
                  (blocks pre-claiming another user's garden). *)
               if not (branch_owner_authorized t child_owner) then
                 (Pricklypear_lisp.Ast.VStr
                    (Printf.sprintf "denied: branch %s is owned by %s"
                       child_name child_owner))
               else
               let new_id =
                 Store.fork_branch t.store ~parent_id:pid
                   ~owner:child_owner ~child_name:child_short
               in
               (match new_id with
                | Some id ->
                    (Pricklypear_lisp.Ast.VStr id)
                | None ->
                    (raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("prim", "fork failed", "failed"))))))
      | _ ->
          (raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("fork", "bad args (need 2 strings)",
    "bad-args")))));

  (* (merge 'source 'target) -> VStr "merged N rows" | raise (denied).
     POLICY (per spec [branching.merge]): merging into `system/*`
     requires the session username to be "admin"; merges into the
     caller's own `owner/*` branches are free. *)
  Primitives.register_server_only tbl "merge"
    (fun args ->
      if t.manifest.username = "anonymous" then
        (raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("merge", "denied for anonymous",
    "denied"))))
      else if t.agent_branch_lock then
        (Pricklypear_lisp.Ast.VStr
             "merge blocked: agent is locked to the current branch")
      else
```

```
    match args with
    | [ Pricklypear_lisp.Ast.VStr source_ref;
        Pricklypear_lisp.Ast.VStr target_ref ] ->
        let src_owner, src_short = split_branch_ref source_ref in
        let tgt_owner, tgt_short = split_branch_ref target_ref in
        (* i-1787970640: BOTH src_owner and tgt_owner must pass the
           branch-owner rule. Target system/* stays admin-only (spec
           branching.merge); the source check is new -- it blocks reading
           others' code via merge results. *)
        if (tgt_owner = "system")
           && not (t.is_admin || t.manifest.username = "admin")
        then
          (Pricklypear_lisp.Ast.VStr
                        "denied: admin-only merge to system branch")
        else if not (branch_owner_authorized t src_owner) then
          (Pricklypear_lisp.Ast.VStr
             (Printf.sprintf "denied: branch %s is owned by %s"
                source_ref src_owner))
        else if not (branch_owner_authorized t tgt_owner) then
          (Pricklypear_lisp.Ast.VStr
             (Printf.sprintf "denied: branch %s is owned by %s"
                target_ref tgt_owner))
        else
          let src_id =
            Store.lookup_branch_id_by_name
              t.store ~owner:src_owner ~name:src_short
          in
          let tgt_id =
            Store.lookup_branch_id_by_name
              t.store ~owner:tgt_owner ~name:tgt_short
          in
          (match src_id, tgt_id with
           | Some s, Some tg ->
               let r = Store.merge_branch t.store
                         ~source_id:s ~target_id:tg in
               (match r with
                | Some res ->
                    (Pricklypear_lisp.Ast.VStr
                                (Store.format_merge_result res))
                | None ->
                    (raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("prim", "merge failed", "failed")))))
           | _ ->
               (Pricklypear_lisp.Ast.VStr
                           "source or target branch not found"))
    | _ ->
        (raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("merge", "bad args (need 2
   strings)", "bad-args")))));

(* (route "/path" handler [visibility]) -> VStr "/path".
   Optional third arg 'public | "public" | 'private | "private".
   Handler-only re-register keeps prior visibility (store ON CONFLICT).
   Anonymous sessions may not register routes.
   H0.3: public targets must pass the public-route lattice (same as AI
   allow minus elevate): pure|read-db|write-db|html|txn. Branch/Ai/
   Publish on a public handler is denied. Missing handler body is OK
   (define-after-route still works). *)
let parse_vis = function
  | Pricklypear_lisp.Ast.VStr s ->
      (match String.lowercase_ascii (String.trim s) with
       | "public" -> Some "public"
       | "private" -> Some "private"
       | _ -> None)
  | _ -> None
in
let check_public_handler (handler : string) : (unit, string) result =
  (* Fail-closed public-route gate (BORGE-1785212463 / 1785212603).
     Infer effects from the in-memory env (synchronous, fresh) via the
     pure [Route_gate] module. The previous store read was stale after
     the define persist race and DEFAULTED TO ALLOW on unknown handlers;
     now unknown / non-function handlers are DENIED. *)
  (Pricklypear_lisp.Route_gate.check_public_handler ~env:t.env handler)
```

```
  in
  let register_route_checked ~route_path ~handler ~visibility
      ~(pass_vis_to_store : bool) : Pricklypear_lisp.Ast.value =
    let gate =
      if visibility = "public" then check_public_handler handler
      else (Ok ())
    in
    match gate with
    | Error msg -> (Pricklypear_lisp.Ast.VStr msg)
    | Ok () ->
        Hashtbl.replace t.routes route_path (handler, visibility);
        let _ =
          if t.seed_additive then
            if pass_vis_to_store then
              Store.register_route_if_absent t.store ~route:route_path
                ~handler ~visibility ()
            else
              Store.register_route_if_absent t.store ~route:route_path
                ~handler ()
          else if pass_vis_to_store then
            Store.register_route t.store ~route:route_path ~handler
              ~visibility ()
          else
            Store.register_route t.store ~route:route_path ~handler ()
        in
        (Pricklypear_lisp.Ast.VStr route_path)
  in
  Primitives.register_server_only tbl "route"
    (fun args ->
      if t.manifest.username = "anonymous" then
        (raise (Pricklypear_lisp.Ast.Interp_error
     (Pricklypear_lisp.Ast.Prim_error ("route", "denied for anonymous",
     "denied"))))
      else
        match args with
        | [ Pricklypear_lisp.Ast.VStr route_path;
            Pricklypear_lisp.Ast.VStr handler ] ->
            let vis =
              match Hashtbl.find_opt t.routes route_path with
              | Some (_, v) -> v
              | None -> "private"
            in
            (* handler-only: pass_vis_to_store=false so DB keeps visibility
               on UPDATE; memory keeps prior or private for new. *)
            register_route_checked ~route_path ~handler ~visibility:vis
              ~pass_vis_to_store:false
        | [ Pricklypear_lisp.Ast.VStr route_path;
            Pricklypear_lisp.Ast.VStr handler;
            vis_v ] ->
            (match parse_vis vis_v with
             | None ->
                 (Pricklypear_lisp.Ast.VStr
                      "route: visibility must be public or private")
             | Some vis ->
                 register_route_checked ~route_path ~handler ~visibility:vis
                   ~pass_vis_to_store:true)
        | _ ->
            (raise (Pricklypear_lisp.Ast.Interp_error
     (Pricklypear_lisp.Ast.Prim_error ("route", "bad args", "bad-args")))));

  (* (public-route path handler) sugar *)
  Primitives.register tbl "public-route"
    (fun args ->
      if t.manifest.username = "anonymous" then
        (raise (Pricklypear_lisp.Ast.Interp_error
     (Pricklypear_lisp.Ast.Prim_error ("public-route", "denied for
     anonymous", "denied"))))
      else
        match args with
        | [ Pricklypear_lisp.Ast.VStr route_path;
            Pricklypear_lisp.Ast.VStr handler ] ->
            register_route_checked ~route_path ~handler ~visibility:"public"
              ~pass_vis_to_store:true
```

```
485         | _ ->
486             (raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("public-route", "bad args",
    "bad-args")))));
487
488   (* (route-visibility path) -> "public" | "private" | () *)
489   Primitives.register tbl "route-visibility"
490     (fun args ->
491       match args with
492       | [ Pricklypear_lisp.Ast.VStr route_path ] ->
493           (match Hashtbl.find_opt t.routes route_path with
494            | Some (_, vis) ->
495                (Pricklypear_lisp.Ast.VStr vis)
496            | None -> Pricklypear_lisp.Ast.VNil)
497       | _ -> Pricklypear_lisp.Ast.VNil);
498
499   (* (checkout 'branch-ref) -> VStr "checked out" | "no such branch".
500      Mutates t.store.branch_id + t.manifest, then re-hydrates the env
501      from the new branch's function_defs (sync -- so the next form sees
502      the right bindings). Blocked while agent_branch_lock is set. *)
503   Primitives.register_server_only tbl "checkout"
504     (fun args ->
505       if t.manifest.username = "anonymous" then
506         (raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("checkout", "denied for anonymous",
    "denied"))))
507       else if t.agent_branch_lock then
508         (Pricklypear_lisp.Ast.VStr
509              "checkout blocked: agent is locked to the current branch")
510       else
511         match args with
512         | [ Pricklypear_lisp.Ast.VStr branch_ref ] ->
513             let owner, name = split_branch_ref branch_ref in
514             (* i-1787970640: owner of the target ref must pass the
515                branch-owner rule (system/* included -- non-admin users
516                never need main; personal branches inherit it). *)
517             if not (branch_owner_authorized t owner) then
518               (Pricklypear_lisp.Ast.VStr
519                  (Printf.sprintf "denied: branch %s is owned by %s"
520                     branch_ref owner))
521             else
522             let found =
523               Store.lookup_branch_id_by_name t.store ~owner ~name
524             in
525             (match found with
526              | None -> raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("checkout", "no such branch",
    "not-found")))
527              | Some new_id ->
528                  let () = checkout_to t ~branch_id:new_id ~branch_ref in
529                  (Pricklypear_lisp.Ast.VStr "checked out"))
530         | _ ->
531             (raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("checkout", "bad args",
    "bad-args")))));
532
533   (* (unbind name) -> VStr "unbound" | "nothing to unbind" | error.
534      Deletes ONLY the current branch's function_defs row for name;
535      the name then resolves via inheritance to the shadowed parent
536      binding (or becomes unbound). Refuses on system/main and on
537      anonymous sessions. See branching.unbind spec. *)
538   Primitives.register tbl "unbind"
539     (fun args ->
540       if t.manifest.username = "anonymous" then
541         (raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("unbind", "denied for anonymous",
    "denied"))))
542       else
543         (* i-1787970640: the current branch's owner must pass the
544            branch-owner rule (delete_function already refuses system/main
545            and the prim refuses anonymous). *)
546         let cur_owner, _cur_name =
547           split_branch_ref t.manifest.branch_name in
```

```
548         if not (branch_owner_authorized t cur_owner) then
549           (Pricklypear_lisp.Ast.VStr
550              (Printf.sprintf "denied: branch %s is owned by %s"
551                 t.manifest.branch_name cur_owner))
552         else
553         match args with
554         | [ Pricklypear_lisp.Ast.VStr name ] ->
555             (try
556               let deleted = Store.delete_function t.store name in
557               if deleted then begin
558                 (* Rehydrate so the env sees the revealed parent binding *)
559                 rehydrate t;
560                 (Pricklypear_lisp.Ast.VStr "unbound")
561               end else
562                 (Pricklypear_lisp.Ast.VStr "nothing to unbind")
563             with Failure msg ->
564               (Pricklypear_lisp.Ast.VStr msg))
565         | _ ->
566             (raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("unbind", "need a name",
    "bad-args")))));
567
568   (* (bench expr-string [n]) -> VFloat average microseconds per iteration.
569      Parses once, warms up once, runs n times (default 1000) under a max
570      budget so the bench loop itself never hits Budget_exceeded. Uses the
571      current session env. Prefer pure expressions (side-effecting bodies
572      will run for real). *)
573   let bench_run (expr_str : string) (n : int) : Pricklypear_lisp.Ast.value =
574     let n = max 1 n in
575     let e =
576       try Pricklypear_lisp.Reader.read_one expr_str
577       with exn ->
578         raise
579           (Pricklypear_lisp.Ast.Interp_error
580              (Pricklypear_lisp.Ast.User_error
581                 ("bench: parse failed: " ^ Printexc.to_string exn,
    "bad-args")))
582     in
583     let run_once () =
584       let budget =
585         Pricklypear_lisp.Eval.make_budget ~limit:max_int ()
586       in
587       Control_rim.run_lwt ~prims:t.prims (fun () ->
588           Pricklypear_lisp.Eval.eval
589             ~budget ~prims:Pricklypear_lisp.Eval.empty_prims t.env e)
590     in
591     (* warmup (discard) *)
592     let _ = run_once () in
593     let t0 = Unix.gettimeofday () in
594     let rec loop i =
595       if i >= n then ()
596       else
597         let _ = run_once () in
598         loop (i + 1)
599     in
600     let () = loop 0 in
601     let t1 = Unix.gettimeofday () in
602     let avg_us = ((t1 -. t0) *. 1_000_000.) /. float_of_int n in
603     (Pricklypear_lisp.Ast.VFloat avg_us)
604   in
605   Primitives.register tbl "bench"
606     (fun args ->
607       match args with
608       | [ Pricklypear_lisp.Ast.VStr s ] -> bench_run s 1000
609       | [ Pricklypear_lisp.Ast.VStr s; Pricklypear_lisp.Ast.VInt n ] ->
610           bench_run s (Int64.to_int n)
611       | _ ->
612           (Pricklypear_lisp.Ast.VStr
613                "bench: need (bench expr-string [iterations])"));
614
615   (* Private prim used by the with-transaction special form in eval.ml.
616      NOT USER-VISIBLE: it's looked up under name __with_transaction__ by
```

```
617      the special form handler, never `(with-transaction-call ...)` from
         Lisp.
618      Also bound into the env as a VPrim name by the auto-bind loop, but
619      a `(with-transaction-call ...)` call would just route through here.
620      The prim receives one arg: a VLambda value. It wraps Eval.apply
621      inside Store.with_dynamic_transaction, so prims called from the
622      body of the lambda see the in-transaction pgx conn via
623      Store.current_conn. Lands it29, option A from BORGE-1783438027.
624
625      Returns the lambda body's result. Nested transactions raise a clean
626      Interp_error (see with_dynamic_transaction). *)
627  (* Apply a VLambda under the effect rim (H1.7). *)
628  let apply_lambda_auto (lam : Pricklypear_lisp.Ast.value) :
         Pricklypear_lisp.Ast.value =
629    let budget = Pricklypear_lisp.Eval.make_budget () in
630    Control_rim.run_lwt ~prims:t.prims (fun () ->
631        Pricklypear_lisp.Eval.apply
632          ~budget ~prims:Pricklypear_lisp.Eval.empty_prims lam [])
633  in
634  Primitives.register tbl "__with_transaction__"
635    (fun args ->
636      match args with
637      | [ Pricklypear_lisp.Ast.VLambda (params, body, clo_env) as lam ] ->
638          (* Enforce zero-arg lambda for clarity: with-transaction bodies
639             shouldn't take args. *)
640          let _ = params in
641          let _ = clo_env in
642          (* H1.3: body re-enters apply_eff under Control.run_lwt when
643             PP_EVAL_CONTROL=eff so nested prims Prim_call while seeing
644             Store.current_conn from with_dynamic_transaction. *)
645          Io_lwt.Db.with_dynamic_transaction t.store (fun () ->
646              apply_lambda_auto lam)
647      | _ ->
648          (Pricklypear_lisp.Ast.VStr
649                       "__with_transaction__: bad args (need a lambda)"));
650
651  (* Private prim used by the with-step special form in eval.ml. NOT
652     USER-VISIBLE: routed to under name __step__ by eval.ml, never
653     `(with-step "name" (lambda ...))` from a plain prim call.
654
655     (__step__ "name" <lambda>) is a checkpoint boundary (borge/jobs.borg
656     step-checkpoints):
657       - Outside a job (Jobs.current_job_id_opt () = None, e.g. an HTTP
658         request or REPL) it is a PLAIN thunk call -- no checkpoint.
659       - Inside a job (the runner binds the job id around eval), the
660         thunk's result is memoized under (job_id, name): first execution
661         runs the thunk and writes a job_steps row; re-execution (a retry)
662         returns the stored value WITHOUT re-running the thunk.
663
664     Exactly-once with (with-transaction (step ?)): the checkpoint INSERT
665     rides Store.current_conn -- when the caller is inside
666     with_transaction, the insert lands in the SAME transaction as the
667     thunk's writes, so it commits or rolls back atomically with them (a
668     failed/later-raised step leaves no checkpoint; a retry re-runs). If
669     the THUNK ITSELF raises, no checkpoint is written (the exception
670     propagates, rolling back the surrounding transaction if any). *)
671  Primitives.register tbl "__step__"
672    (fun args ->
673      match args with
674      | [ Pricklypear_lisp.Ast.VStr name;
675          Pricklypear_lisp.Ast.VLambda (_params, _body, _clo) as lam ] ->
676          (match Jobs.current_job_id_opt () with
677           | None ->
678               (* Outside a job: a plain thunk call with no checkpoint. *)
679               apply_lambda_auto lam
680           | Some job_id ->
681               (match
682                  Jobs.checkpoint_lookup ~store:t.store ~job_id ~name
683                with
684                | Some value_json ->
685                    (* Memoized: return the stored value without
         re-running. *)
686                    (try
```

```
                      Prims_pure.json_to_value
                        (Yojson.Safe.from_string value_json)
                    with _ ->
                      Pricklypear_lisp.Ast.VStr value_json)
               | None ->
                   (* First execution in this job: run the thunk, then write
                      the checkpoint. If the thunk raises, the exception
                      propagates (no checkpoint, surrounding tx rolls back). *)
                   let result = apply_lambda_auto lam in
                   let value_json =
                     Yojson.Safe.to_string (Prims_pure.value_to_json result)
                   in
                   Jobs.checkpoint_write ~store:t.store ~job_id ~name
                     ~value:value_json;
                   result))
     | _ ->
         (Pricklypear_lisp.Ast.VStr
                       "__step__: bad args (need name-str, lambda)"));

  (* (with-public-write (lambda () body...)) -- run body with write privilege
     so public route handlers can create data_rows / notes captures.
     Elevates anonymous only (logged-in sessions already write). Restores
     identity afterward. Prefer for narrow public forms (signup), not general
     unauth APIs.

     Elevates BOTH the live session manifest AND the per-request context
     manifest. CRUD prims (create/deftype) read their anon guard from
     Request_context.manifest (the gate-time snapshot, per BORGE-1784434193
     cotenant fix); without elevating the snapshot, the guard sees the stale
     "anonymous" and silently denies (create returns a VStr, seq discards it,
     the handler renders its thanks page with no row written). *)
  Primitives.register tbl "with-public-write"
    (fun args ->
     match args with
     | [ Pricklypear_lisp.Ast.VLambda (_params, _body, _clo) as lam ] ->
         let prev = t.manifest in
         let need_elevate = prev.username = "anonymous" in
         if need_elevate then
           t.manifest <-
             { prev with user_id = "public"; username = "public" };
         let run_body =
           match Request_context.get () with
           | Some ctx when need_elevate ->
               let elevated_manifest =
                 { (Request_context.manifest ctx) with
                   user_id = "public"; username = "public" }
               in
               let elevated_ctx =
                 Request_context.make
                   ~manifest:elevated_manifest
                   ~env:(Request_context.env ctx)
                   ~routes:(Request_context.routes ctx)
               in
               Request_context.with_current elevated_ctx
                 (fun () -> apply_lambda_auto lam)
           | _ -> apply_lambda_auto lam
         in
         (Fun.protect ~finally:(fun () -> if need_elevate then t.manifest <- prev) (fun () -> run_body))
     | _ ->
         (Pricklypear_lisp.Ast.VStr
              "with-public-write: bad args (need a zero-arg lambda)"))

(** If model returns a bare expression body, wrap as (lambda (x) ...). *)

(* ---- AI / agent tools (Agent_tools) -- re-export for stable Session.* API ---- *)
```

```
let define_binding = Agent_tools.define_binding
let install_ai_function = Agent_tools.install_ai_function
let function_source = Agent_tools.function_source
let function_ast = Agent_tools.function_ast
let package_binding_names = Agent_tools.package_binding_names
let package_export_names = Agent_tools.package_export_names
let list_package_names = Agent_tools.list_package_names
let apropos_docs = Agent_tools.apropos_docs
let describe = Agent_tools.describe
let export_package = Agent_tools.export_package
let propose_function = Agent_tools.propose_function
let default_agent_seed = Agent_tools.default_agent_seed
let agent_session_new = Agent_tools.agent_session_new
let agent_say = Agent_tools.agent_say
let agent_history = Agent_tools.agent_history
let agent_sessions = Agent_tools.agent_sessions
let agent_session_delete = Agent_tools.agent_session_delete
let agent = Agent_tools.agent
let agent_task = Agent_tools.agent_task
let llm_rewrite = Agent_tools.llm_rewrite
let llm_inner = Agent_tools.llm_inner
let llm_outer = Agent_tools.llm_outer
let llm_seed = Agent_tools.llm_seed
let pp_interp_exn = Agent_tools.pp_interp_exn

let register_media_prims (t : t) (tbl : Primitives.t) : unit =
  Prims_media.register t tbl
let do_http_get = Prims_net.do_http_get

(* Late-bound thunk for route/dispatch prim. call_handler is defined later
   in this module (after `make`); this ref lets the prim registered inside
   make reference it without a forward-let. Bound right after call_handler.
     *)
let route_dispatch_fn :
  (t -> string -> Session_types.request -> Pricklypear_lisp.Ast.value) ref =
  ref (fun _t _handler _req ->
         Pricklypear_lisp.Ast.VStr "route/dispatch: not initialized")

let register_net_prims (t : t) (tbl : Primitives.t) : unit =

  Prims_net.register t tbl;
  Net_prims.register t tbl
let promote_to_personal = Prims_db.promote_to_personal
let register_crud_prims (t : t) (tbl : Primitives.t) : unit =
  Prims_db.register ~rehydrate_fn:rehydrate t tbl

let method_to_string = Prims_request.method_to_string
let require_request = Prims_request.require_request
let str_name = Prims_request.str_name
let depth_arg = Prims_request.depth_arg
let record_define_side_effects = Call_graph.record_define_side_effects
let impact_summary_text = Call_graph.impact_summary_text
(** Graph + impact + change-set + DNU prims (image.inspector / dnu-stubs).
     *)
let register_graph_prims (t : t) (tbl : Primitives.t) : unit =
  Prims_graph.register t tbl
(** Register request-* + redirect prims. Closures capture [t] and read
    [current_request] at call time (handler-scoped dynamic binding). *)
let register_request_prims (t : t) (tbl : Primitives.t) : unit =
  Prims_request.register t tbl
(* Evaluate a single s-expression string as one REPL turn.
   Returns either the resulting value or an error message string.

   Per spec [image.repl-experience]: (define name val) ALSO persists to
   the store: the env binding is in-memory (fast), the store binding
   survives image reboot. The store call is non-fatal on PG-down (the
   env binding still happened).

   Lwt ABI (post-refactor): fork/merge/route/checkout are NORMAL PRIMS
   registered via [register_session_control_prims] -- they return their
   real DB result wrapped in a promise. Only [define] keeps an AST-level
   intercept: it's read-level special (the name arg is a Var that must
   not be evaluated), so it can't become a regular prim. *)
```

```
let rec eval_one (t : t) (input : string) : Pricklypear_lisp.Ast.value =
  try
    let expr = Pricklypear_lisp.Reader.read_one input in
    (* Pre-eval intercept: (define name <body>) is the only AST-level
       special case left, because the name argument is a Var that must
       not be evaluated. Everything else (fork/merge/route/checkout)
       dispatches through Eval.apply via the prim_table.
       H1: [eval_string_auto] honors PP_EVAL_CONTROL=lwt|eff (default lwt).
     *)
    (* package-surface: (package NAME (export ...)) is a directive, not a
     value. *)
    (match expr with
     | Pricklypear_lisp.Ast.App
         (Pricklypear_lisp.Ast.Var "package", args) ->
         handle_package_form t args
     | _ ->
    let v = eval_string_rim t input in
    let pkg = t.current_package in
    let persist_fn name body_expr doc =
      if t.seed_additive then
        Store.persist_function_if_absent t.store name body_expr ~doc
          ~package:pkg t.manifest.username
      else
        Store.persist_function t.store name body_expr ~doc
          ~package:pkg t.manifest.username
    in
    let persist_define name body_expr doc =
      let () =
        if t.manifest.username <> "anonymous" then
          persist_fn name body_expr doc
        else ()
      in
      let () =
        if not t.suppress_refresh_on_define && touches_shell t name
        then begin
          (* Bust only: the next request-gate refresh re-evals. Eager
             refresh here would multiply work per-define during bulk
             loads; invariant-3 retry suppression keeps a broken define
             from blocking the defining request itself. *)
          bust_ui_shell_cache t
        end else ()
      in
      let () = record_define_side_effects t ~name in
      v
    in
    (match Pricklypear_lisp.Eval.parse_top_define expr with
     | Some { name; body; doc } ->
         (* Persist peeled body + docstring. Env bind already ran via eval.
     *)
         persist_define name body doc
     | None ->
     (match expr with
     | Pricklypear_lisp.Ast.App
         (Pricklypear_lisp.Ast.Var "defmacro",
          [ Pricklypear_lisp.Ast.Var name; params_expr; body_expr ]) ->
         let () =
           if t.manifest.username <> "anonymous" then begin
             let stored =
               Pricklypear_lisp.Ast.App
                 (Pricklypear_lisp.Ast.Var "__macro__",
                  [ params_expr; body_expr ])
             in
             persist_fn name stored None
           end else ()
         in
         v
     | _ -> v)))
  with
  | Pricklypear_lisp.Ast.Interp_error e as exn -> raise exn
  | Pricklypear_lisp.Reader.Reader_error (msg, line, col) ->
      raise (Pricklypear_lisp.Ast.Interp_error
               (Pricklypear_lisp.Ast.Parse_error (msg, line, col)))
  | exn -> raise exn
```

```
893
894
895 (* --- Library loader: same .pp text as in git libs/ --- *)
896
897 and handle_package_form (t : t) (args : Pricklypear_lisp.Ast.expr list)
898   : Pricklypear_lisp.Ast.value =
899   let name_of = function
900     | Pricklypear_lisp.Ast.Var s -> Some s
901     | Pricklypear_lisp.Ast.Lit (Pricklypear_lisp.Ast.VStr s) -> Some s
902     | _ -> None
903   in
904   let export_names = function
905     | Pricklypear_lisp.Ast.App (Pricklypear_lisp.Ast.Var "export", xs) ->
906         List.filter_map name_of xs
907     | _ -> []
908   in
909   match args with
910   | name_e :: rest ->
911       (match name_of name_e with
912        | None ->
913            (raise (Pricklypear_lisp.Ast.Interp_error
       (Pricklypear_lisp.Ast.Prim_error ("package", "need name
       symbol/string", "bad-args"))))
914          | Some name ->
915              let name = String.trim name in
916              if name = "" then
917                (Pricklypear_lisp.Ast.VStr "package: empty name")
918              else if name = "kernel" then
919                (Pricklypear_lisp.Ast.VStr "package: reserved name: kernel")
920              else begin
921                let exports =
922                  match rest with
923                  | [ exp_form ] -> export_names exp_form
924                  | _ ->
925                      List.concat_map export_names rest
926                in
927                t.current_package <- Some name;
928                let () =
929                  if t.manifest.username = "anonymous" then ()
930                  else begin
931                    let () =
932                      Store.upsert_package_registry t.store ~name ~exports
933                        ~author:t.manifest.username ~enabled_preserve:true ()
934                    in
935                    Store.stamp_package_bindings t.store ~package:name ~exports
936                  end
937                in
938                (Pricklypear_lisp.Ast.VStr
939                     (Printf.sprintf "package %s (%d exports)" name
940                        (List.length exports)))
941              end)
942   | [] ->
943       (raise (Pricklypear_lisp.Ast.Interp_error
       (Pricklypear_lisp.Ast.Prim_error ("package", "need name",
       "bad-args"))))
944
945 and eval_form (t : t) (expr : Pricklypear_lisp.Ast.expr)
946     : Pricklypear_lisp.Ast.value =
947   (* Like eval_one but takes AST -- avoids pp_expr string round-trip. *)
948   (match expr with
949    | Pricklypear_lisp.Ast.App
950        (Pricklypear_lisp.Ast.Var "package", args) ->
951        handle_package_form t args
952    | _ ->
953   let v = eval_expr_rim t expr in
954   let pkg = t.current_package in
955   let persist_fn name body_expr doc =
956     if t.seed_additive then
957       Store.persist_function_if_absent t.store name body_expr ~doc
958         ~package:pkg t.manifest.username
959     else
960       Store.persist_function t.store name body_expr ~doc
961         ~package:pkg t.manifest.username
```

```
  in
  let persist_define name body_expr doc =
    let () =
      if t.manifest.username <> "anonymous" then
        persist_fn name body_expr doc
      else ()
    in
    let () =
      if not t.suppress_refresh_on_define && touches_shell t name
      then begin
        bust_ui_shell_cache t
      end else ()
    in
    let () = record_define_side_effects t ~name in
    v
  in
  (match Pricklypear_lisp.Eval.parse_top_define expr with
   | Some { name; body; doc } -> persist_define name body doc
   | None ->
   (match expr with
   | Pricklypear_lisp.Ast.App
       (Pricklypear_lisp.Ast.Var "defmacro",
        [ Pricklypear_lisp.Ast.Var name; params_expr; body_expr ]) ->
       let () =
         if t.manifest.username <> "anonymous" then begin
           let stored =
             Pricklypear_lisp.Ast.App
               (Pricklypear_lisp.Ast.Var "__macro__",
                [ params_expr; body_expr ])
           in
           persist_fn name stored None
         end else ()
       in
       v
   | _ -> v)))

and load_string (t : t) (src : string) : (string, string) result =
  let prev = t.current_package in
  let prev_suppress = t.suppress_refresh_on_define in
  t.current_package <- None;
  t.suppress_refresh_on_define <- true;
  Fun.protect
    ~finally:(fun () ->
      t.current_package <- prev;
      t.suppress_refresh_on_define <- prev_suppress)
    (fun () ->
      let r = Loader.load_string ~eval_form:(eval_form t) src in
      (* Boot seed holds suppress_refresh across many loads and does one
         explicit refresh at the end -- don't thrash ui/css mid-seed. *)
      let () =
        if not prev_suppress then begin
          bust_ui_shell_cache t;
          refresh_ui_shell t
        end
      in
      r)

and load_library (t : t) (name : string) : (string, string) result =
  let prev = t.current_package in
  let prev_suppress = t.suppress_refresh_on_define in
  t.current_package <- None;
  t.suppress_refresh_on_define <- true;
  Fun.protect
    ~finally:(fun () ->
      t.current_package <- prev;
      t.suppress_refresh_on_define <- prev_suppress)
    (fun () ->
      let r = Loader.load_library ~eval_form:(eval_form t) name in
      let () =
        if not prev_suppress then begin
          bust_ui_shell_cache t;
          refresh_ui_shell t
        end
```

```
      in
      r)

(** branching.seed-policy: admin-only upsert of libs/<pkg>/lib.pp onto
    system/main (full clobber for names in that file). Does not touch
    personal/dev rows. seed_additive stays false so persist upserts. *)
and reseed_package (t : t) (pkg : string) : (string, string) result =
  let pkg = String.trim pkg in
  if pkg = "" then (Error "empty package name")
  else if (not t.is_admin)
          && t.manifest.username <> "admin"
          && t.manifest.username <> "system" then
    (Error "requires admin")
  else begin
    t.seed_additive <- false;
    t.suppress_refresh_on_define <- true;
    let finally () =
      t.suppress_refresh_on_define <- false;
      t.seed_additive <- false;
      ()
    in
    Fun.protect ~finally (fun () ->
        Store.with_branch t.store ~branch_id:Store.system_main_id (fun () ->
          let r = load_library t pkg in
          let () = rehydrate t in
          let () = refresh_ui_shell t in
          match r with
          | Ok msg ->
              (Ok (Printf.sprintf "reseeded %s ? system/main: %s" pkg msg))
          | Error e -> (Error e)))
  end

and refresh_ui_shell (t : t) : unit =
  (* Product shell: nav HTML + theme CSS (image.product-theme).
     image.http-robustness invariant 3: this runs on every request, so
     a FAILED shell eval must not re-attempt + re-warn per request
     (ui_shell_retry_at suppresses for shell_retry_secs after a throw;
     reset on checkout / success). Emergency chrome fallback is
     unchanged: when ui_nav_html is None, http serves emergency nav.

     image.product-theme: invalidation is DEPENDENCY-TRACKED, not
     TTL-based. A successful shell eval records the binding names it
     read (ui_shell_reads, via Env.lookup_hook, armed only around these
     two evals). A define of a name in that set busts the cache (see
     touches_shell); anything else leaves it. The cache is therefore
     fresh until a real dependency changes -- no 60s window serving
     stale CSS, no prefix-guess busting on unrelated defines. Checkout
     and load-library still bust unconditionally. *)
  let shell_retry_secs = 60.0 in
  let now = Unix.gettimeofday () in
  let pp_exn exn =
    match exn with
    | Pricklypear_lisp.Ast.Interp_error e ->
        Pricklypear_lisp.Ast.pp_interp_error e
    | _ -> Printexc.to_string exn
  in
  (* Refresh one shell binding: eval [eval_str], record its read-set,
     cache the rendered string via [set_cached], handle failure with
     retry suppression. *)
  let refresh name eval_str set_cached =
    Pricklypear_lisp.Env.lookup_hook :=
      Some (fun n ->
        if not (List.exists (String.equal n) t.ui_shell_reads) then
          t.ui_shell_reads <- n :: t.ui_shell_reads);
    Fun.protect
      ~finally:(fun () -> Pricklypear_lisp.Env.lookup_hook := None)
      (fun () ->
        (try
           let v = eval_one t eval_str in
           (match Primitives.is_raw_html v with
            | Some s -> set_cached (Some s)
            | None ->
                (match v with
```

```
                | Pricklypear_lisp.Ast.VStr s -> set_cached (Some s)
                | _ ->
                    set_cached
                      (Some (Pricklypear_lisp.Reader.pp_value v))));
          t.ui_shell_retry_at <- 0.0;
          ()
        with exn ->
          (* Warn once per retry window, then go quiet. Print the
             full message so the failure is diagnosable. *)
          ignore
            (Ops_log.warn ~component:"session"
               ~context:[("fn", name)]
               ("refresh_ui_shell failed (retry in "
                ^ string_of_float shell_retry_secs ^ "s): "
                ^ pp_exn exn));
          set_cached None;
          t.ui_shell_retry_at <- now +. shell_retry_secs;
          ()))
  in
  let () =
    match Pricklypear_lisp.Env.lookup t.env "ui/nav" with
    | None -> t.ui_nav_html <- None
    | Some _ ->
        if t.ui_nav_html <> None then
          (* Cache present: serve it (bust clears it first). *)
          ()
        else if now < t.ui_shell_retry_at then
          (* Recently failed: skip re-eval + skip warn. *)
          ()
        else
          refresh "ui/nav" "(ui/nav)" (fun v -> t.ui_nav_html <- v)
  in
  match Pricklypear_lisp.Env.lookup t.env "ui/css" with
  | None -> t.ui_css <- None
  | Some _ ->
      if t.ui_css <> None then
        ()
      else if now < t.ui_shell_retry_at then
        ()
      else
        refresh "ui/css" "(ui/css)" (fun v -> t.ui_css <- v)

and seed_one (t : t) (name : string) : unit =
  let r = load_library t name in
  (match r with
   | Ok msg -> Printf.eprintf "[session] %s: %s\n%!" name msg
   | Error e -> Printf.eprintf "[session] %s load: %s\n%!" name e);
  ()

(** Boot seed order (BORGE-1785211333).
    Preferred prefix preserves dependency-ish order (std/ui before notes,
    outline before notes, ?). Any other [libs/<name>/lib.pp] is appended
    alphabetically so adding a package no longer requires an OCaml edit. *)
and seed_package_priority : string list =
  (* css before notes: notes/tree-css evaluates (css/sheet ?) at define
     time; if css is unbound the notes load aborts mid-file and routes
     like /partials/notes-search never seed (notes.search-modal). *)
  (* theme before ui: default theme/* tokens exist before ui/css first
     runs. *)
  [ "std"; "theme"; "weather"; "ui"; "layout"; "welcome"; "ai"; "css";
     "html"
  ; "list"; "outline"; "structed"; "notes"; "signup"; "todo"; "contacts"
  ; "plan"; "chat"; "bookmarks"; "calendar"; "workout"; "flashcards"
  ; "bible"; "dabar"; "bi"; "ref"; "verify" ]

and discover_seed_packages () : string list =
  let preferred = seed_package_priority in
  let root = Loader.libs_dir () in
  let discovered =
    try
      Sys.readdir root
      |> Array.to_list
      |> List.filter (fun name ->
```

```
            name <> ""
            && name.[0] <> '_'
            && name.[0] <> '.'
            (* Alternate skins (theme-paper, ?) load on demand via
               theme/set ? load-library; seeding them would only
               thrash in-memory theme/* before rehydrate. *)
            && not (String.starts_with ~prefix:"theme-" name)
            &&
            let pp =
              Filename.concat root (Filename.concat name "lib.pp")
            in
            Sys.file_exists pp)
      |> List.sort String.compare
    with _ -> []
  in
  let extras =
    List.filter (fun n -> not (List.mem n preferred)) discovered
  in
  preferred @ extras

(** Ensure living agents note exists on system/main after notes seed.
    Idempotent: skip if slug agents already present. *)
and seed_agents_note (t : t) : unit =
  let body =
    String.concat "\n"
      [ "# Agents (living AGENTS.md)"
      ; ""
      ; "This note is the habitat AGENTS.md. Edit with notes/doc/upsert."
      ; ""
      ; "## Entry path"
      ; "1. Public bootstrap (no secrets): GET /agent.txt and GET
    /.well-known/agent.json"
      ; "2. Those docs explain HTTP Basic auth and how to build a REPL
    client (POST /api/eval)."
      ; "3. Local helper: pp-eval CLI or curl with Basic auth."
      ; "4. Then explore live:"
      ; "   (whoami)"
      ; "   (list-packages)"
      ; "   (load-library \"todo\")"
      ; "   (describe \"todo\")"
      ; "   (apropos-docs \"todo/item\")"
      ; "   (function-doc \"todo/item/add\")"
      ; "   (function-source \"todo/item/add\")"
      ; ""
      ; "## Selective discovery"
      ; "- (list-packages) -- top-level package prefixes"
      ; "- (package-bindings \"pkg\") -- all names under pkg/ (any depth)"
      ; "- (package-exports \"pkg\") -- shallow pkg or pkg/local names"
      ; "- (apropos \"pat\") / (apropos-docs \"pat\") -- search names (+
    docs)"
      ; "- (describe \"name-or-pkg\") -- one-shot orientation"
      ; "- Prefer DATA builders from /api/eval; */page handlers need a real
    HTTP request."
      ; ""
      ; "## Libraries"
      ; "Load before use: std ui notes todo outline bible dabar chat
    calendar"
      ; "flashcards bookmarks contacts plan bi weather ref verify ai layout
    welcome."
      ]
  in
  (* Escape body for a Lisp string literal. *)
  let escaped =
    let b = Buffer.create (String.length body + 32) in
    String.iter
      (function
        | '\\' -> Buffer.add_string b "\\\\"
        | '"' -> Buffer.add_string b "\\\""
        | '\n' -> Buffer.add_string b "\\n"
        | c -> Buffer.add_char b c)
      body;
    Buffer.contents b
  in
```

```
  let expr =
    Printf.sprintf
      {| (if (null? (notes/doc/get "agents"))
             (notes/doc/upsert "agents" "Agents (living AGENTS.md)"
               (outline/node* "root" "Agents (living AGENTS.md)" "%s" "" (list))
               "" "note" "")
             "agents-note-exists") |}
      escaped
  in
  (try
      let v = eval_one t expr in
      Printf.eprintf "[session] agents-note: %s\n%!"
        (Pricklypear_lisp.Reader.pp_value v);
      ()
    with exn ->
      Printf.eprintf "[session] agents-note seed: %s\n%!" (pp_interp_exn exn);
      ())

and seed_stdlib (t : t) : unit =
  (* Package defaults always land on system/main -- NEVER on the user's
     personal branch. Personal inherits main via the recursive CTE; user
     (define)s write overrides on personal and survive reseed/deploy.

     branching.seed-policy OPTION A: boot seed is ADDITIVE --
     persist_function_if_absent / register_route_if_absent via
     t.seed_additive. Existing system/main rows (e.g. from promote)
     are not clobbered on deploy/restart. Explicit (reseed-package)
     upserts from libs when an admin asks. Interactive load-library
     still upserts the CURRENT branch.

     suppress_refresh_on_define: the eval_one define-interceptor would
     otherwise fire refresh_ui_shell when ui/nav is defined mid-seed
     (before notes/capture/nav is bound) and fail with
     `unbound: notes/capture/nav`. The boot sequence does one explicit
     refresh_ui_shell AFTER all packages are loaded. *)
  Store.with_branch t.store ~branch_id:Store.system_main_id (fun () ->
    t.suppress_refresh_on_define <- true;
    t.seed_additive <- true;
    (* image.boot-seed-batch: preload existing fn/route names on
       system/main once, so persist_function_if_absent /
       register_route_if_absent skip the per-name get_by_path SELECT
       (~1800 defines + routes). On a warm DB this collapses the seed
       to two queries + zero writes. *)
    Store.begin_seed_names t.store;
    let started = Unix.gettimeofday () in
    let finally () =
      let preloaded =
        match t.store.seed_names with
        | Some h -> Some (Hashtbl.length h)
        | None -> None in
      Store.end_seed_names t.store;
      t.suppress_refresh_on_define <- false;
      t.seed_additive <- false;
      let elapsed_ms =
        int_of_float ((Unix.gettimeofday () -. started) *. 1000.0) in
      Printf.eprintf "[session] boot seed: %d packages, %d ms%s\n%!"
        (List.length (discover_seed_packages ())) elapsed_ms
        (match preloaded with
         | Some n -> Printf.sprintf " (%d names preloaded)" n
         | None -> "")
    in
    Fun.protect ~finally (fun () ->
        let pkgs = discover_seed_packages () in
        Printf.eprintf "[session] seed_stdlib packages (%d): %s\n%!"
          (List.length pkgs) (String.concat " " pkgs);
        let rec go = function
          | [] -> ()
          | "notes" :: rest ->
              let () = seed_one t "notes" in
              let () = seed_agents_note t in
              go rest
```

```
          | name :: rest ->
              let () = seed_one t name in
              go rest
        in
        go pkgs))

(** Ensure username/personal exists as an EMPTY child of system/main.
    Inheritance (not copy) means main defaults stay visible; personal
    only stores the user's overrides. *)
and ensure_personal_branch (t : t) : string option =
  let owner = t.manifest.username in
  let main_id_opt =
    Store.lookup_branch_id_by_name t.store ~owner:"system" ~name:"main"
  in
  let parent_id =
    match main_id_opt with
    | Some id -> id
    | None -> Store.system_main_id
  in
  Store.ensure_branch t.store ~parent_id ~owner ~name:"personal"

and checkout_personal (t : t) : unit =
  let owner = t.manifest.username in
  let branch_ref = owner ^ "/personal" in
  let found =
    Store.lookup_branch_id_by_name t.store ~owner ~name:"personal"
  in
  match found with
  | None ->
      Printf.eprintf
        "[session] personal branch missing; staying on %s\n%!"
        t.manifest.branch_name;
      (* Still rehydrate whatever branch we have. *)
      rehydrate t
  | Some id ->
      Printf.eprintf "[session] checkout %s (%s)\n%!" branch_ref id;
      checkout_to t ~branch_id:id ~branch_ref

(** Agent branch: owner = parent human's username, name = "agent".
    Sibling of the parent's personal; agent boots here, not in personal. *)
and ensure_agent_branch (t : t) ~(parent_username : string)
  : string option =
  let main_id_opt =
    Store.lookup_branch_id_by_name t.store ~owner:"system" ~name:"main"
  in
  let parent_id =
    match main_id_opt with
    | Some id -> id
    | None -> Store.system_main_id
  in
  Store.ensure_branch t.store ~parent_id ~owner:parent_username
    ~name:"agent"

and checkout_agent (t : t) ~(parent_username : string) : unit =
  let branch_ref = parent_username ^ "/agent" in
  let found =
    Store.lookup_branch_id_by_name t.store ~owner:parent_username
      ~name:"agent"
  in
  match found with
  | None ->
      Printf.eprintf
        "[session] agent branch missing; staying on %s\n%!"
        t.manifest.branch_name;
      rehydrate t
  | Some id ->
      Printf.eprintf "[session] checkout %s (%s)\n%!" branch_ref id;
      checkout_to t ~branch_id:id ~branch_ref

and recipe_hello (t : t) : (string, string) result =
  load_library t "hello"

and recipe_todo (t : t) : (string, string) result =
```

```
  load_library t "todo"

and recipe_bible (t : t) : (string, string) result =
  (* Types + API + /bible only -- corpus via scripts/seed_bible_bsb.py *)
  load_library t "bible"

and recipe_dabar (t : t) : (string, string) result =
  (* Study layer -- requires bible/*; Strong's/interlinear via seed scripts
     *)
  let r = load_library t "bible" in
  (match r with
   | Error e -> (Error e)
   | Ok _ -> load_library t "dabar")

and recipe_notes (t : t) : (string, string) result =
  (* Outline-first notes -- outline pure lib then notes package *)
  let r = load_library t "outline" in
  (match r with
   | Error e -> (Error e)
   | Ok _ -> load_library t "notes")


let make ~(manifest : Pricklypear_common.Manifest.t) ~(store : Store.t) ()
    : t =
  let env = Pricklypear_lisp.Env.create () in
  let prim_table = Primitives.make_with_defaults () in
  (* Pre-allocate the session record so the DB-touching control prims
     (fork/merge/route/checkout) can capture [t] as a closure (they
     need access to t.store + t.manifest) and register themselves on
     the same prim_table. *)
  let routes = Hashtbl.create 16 in
  let t =
    ref {
      manifest;
      store;
      env;
      load_stamp = "";
      prims = prim_table;  (* temporary; replaced below *)
      routes;
      started_at = Unix.gettimeofday ();
      kernel_catalog = [];  (* filled after all prims register *)
      ui_nav_html = None;
      ui_css = None;
      ui_shell_reads = [];
      ui_shell_retry_at = 0.0;
      suppress_refresh_on_define = false;
      seed_additive = false;
      current_request = None;
      agent_branch_lock = false;
      is_admin = (manifest.username = "admin");
      auth_token = None;
      impact_queue = [];
      change_transcript = [];
      current_package = None;
      net_streams = Hashtbl.create 16;
    }
  in
  (* Register session-aware primitives (whoami, current-author,
     list-bindings). username_fn re-reads t.manifest so per-request
     identity attach updates (whoami) without re-registering. *)
  Primitives.register_session_prims prim_table
    ~username_fn:(fun () ->
      (match Request_context.get () with
       | Some c -> (Request_context.manifest c).username
       | None -> (!t).manifest.username)
    ) ~env;
  (* Register the DB-touching control primitives on the prim_table,
     then rebind t.prims to the populated table. These prims capture
     (!t), so they read/write store+manifest on each invocation. *)
  register_session_control_prims !t prim_table;
  (* HTTP request-context prims (it33): request-method/path/param(s)/body +
     redirect. Dynamic scope via current_request while call_handler runs. *)
  register_request_prims !t prim_table;
```

```
(* Call graph + impact + change-set + DNU (image.inspector combo pack). *)
register_graph_prims !t prim_table;
(* Function observability: (analyze 'fn) / (explain 'fn). *)
let name_arg args =
  match args with
  | [ Pricklypear_lisp.Ast.VStr n ] -> Some n
  | [ Pricklypear_lisp.Ast.VSymbol n ] -> Some n
  | _ -> None
in
Primitives.register prim_table "analyze" (fun args ->
    match name_arg args with
    | None -> raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("analyze", "need function name",
   "bad-args")))
    | Some name ->
        let author = (!t).manifest.username in
        let user_id =
          let uid = (!t).manifest.user_id in
          if uid = "" || uid = "anonymous" || uid = "public" then None
          else Some uid
        in
        (match
           Analyze.analyze_function (!t).store ~author ~user_id
   ~fn_name:name
             ~force_llm:true
         with
         | Ok j -> Analyze.result_to_vstr (Ok j)
         | Error _llm_err ->
             (* Fall back to free heuristic when no key / LLM failure. *)
             Analyze.result_to_vstr
               (Analyze.analyze_function (!t).store ~author ~user_id
                  ~fn_name:name ~force_llm:false)));
Primitives.register prim_table "explain" (fun args ->
    match name_arg args with
    | None -> raise (Pricklypear_lisp.Ast.Interp_error
   (Pricklypear_lisp.Ast.Prim_error ("explain", "need function name",
   "bad-args")))
    | Some name ->
        (match Analyze.explain_function (!t).store ~fn_name:name with
         | Ok s -> Pricklypear_lisp.Ast.VStr s
         | Error e -> Pricklypear_lisp.Ast.VStr e));
(* Register CRUD primitives (get/create/update/delete/list) over
   the shared data_rows table. Same pattern as control prims:
   they capture (!t) and return their real DB result via Lwt. *)
register_crud_prims !t prim_table;
(* Network primitives (tcp-connect, stream-read/write/close) --
   single-image.net-prims. *)
register_net_prims !t prim_table;
(* Hypermedia harness: html/* lambdasoup prims + route/dispatch
   (headless htmx swap simulator; spec borge/hypermedia-harness.borg).
   route/dispatch is registered inline here (not in Prims_hyper) so it
   can close over Session.t without a dependency cycle -- mirrors
   register_crud_prims above. *)
Prims_hyper.register_pure_prims prim_table;
Primitives.register prim_table "route/dispatch"
  (fun args ->
     match args with
     | Pricklypear_lisp.Ast.VStr handler_name
       :: Pricklypear_lisp.Ast.VStr meth
       :: Pricklypear_lisp.Ast.VStr path :: rest ->
       let params =
         match rest with
         | p :: _ -> Prims_hyper.assoc_of p
         | _ -> []
       in
       let headers =
         match rest with
         | _ :: h :: _ ->
           List.map
             (fun (k, v) -> (String.lowercase_ascii k, v))
             (Prims_hyper.assoc_of h)
         | _ -> []
       in
```

```
        let body =
          match rest with
          | _ :: _ :: Pricklypear_lisp.Ast.VStr b :: _ -> b
          | _ -> ""
        in
        let method_ : Session_types.http_method =
          match String.lowercase_ascii meth with
          | "post" -> `POST
          | _ -> `GET
        in
        let req : Session_types.request = {
          method_;
          path;
          params;
          body;
          headers;  (* caller controls HX-Request etc. *)
        } in
        (* Invoke call_handler via the late-bound thunk so we don't
           forward-reference it (defined later in this module). *)
        !route_dispatch_fn !t handler_name req
      | _ -> raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("prim", "route/dispatch needs
    handler method path", "bad-args"))));
  (* route/lookup: resolve a URL path to its handler name via the route
     table. Returns VNil if no route matches. Used by libs/hyper/visit. *)
  Primitives.register prim_table "route/lookup"
    (fun args ->
       match args with
       | [Pricklypear_lisp.Ast.VStr path] ->
         (match Hashtbl.find_opt (!t).routes path with
          | Some (handler, _) -> Pricklypear_lisp.Ast.VStr handler
          | None -> Pricklypear_lisp.Ast.VNil)
       | _ -> raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("prim", "route/lookup needs one
    path", "bad-args"))));
  (* AI + list-branches / current-branch registered inside
     register_crud_prims above (they capture t). *)
  (* Make primitives reachable by name in the env, so `(+)` etc.
     resolve. We define each as VPrim <name>; eval.ml's apply already
     dispatches VPrim via lookup_prim. Comes AFTER control prims are
     registered, so fork/merge/route/checkout auto-bind too. *)
  Hashtbl.iter
    (fun name _ ->
      Pricklypear_lisp.Env.define env name (Pricklypear_lisp.Ast.VPrim
     name))
    prim_table;
  (* lisp-interpreter.math: pi is a bound value, not a zero-arg prim. *)
  Pricklypear_lisp.Env.define env "pi" (Pricklypear_lisp.Ast.VFloat
     Float.pi);
  Primitives.register prim_table "load-string"
    (fun args ->
      match args with
      | [ Pricklypear_lisp.Ast.VStr src ] ->
          let r = load_string !t src in
          (match r with
           | Ok msg -> (Pricklypear_lisp.Ast.VStr msg)
           | Error e ->
               (Pricklypear_lisp.Ast.VStr ("load-string: " ^ e)))
      | _ ->
          (raise (Pricklypear_lisp.Ast.Interp_error
    (Pricklypear_lisp.Ast.Prim_error ("load-string", "need 1 string",
    "bad-args")))));
  (* (eval-string src) -- evaluate one expression in the current env.
     Must NOT call eval_one/eval_under_rim: this prim already runs under the
     outer Control_rim (route handler /api/eval); nested match_with resumes
     the Prim_call continuation twice ? Continuation_already_resumed. *)
  Primitives.register prim_table "eval-string"
    (fun args ->
      match args with
      | [ Pricklypear_lisp.Ast.VStr src ]
      | [ Pricklypear_lisp.Ast.VSymbol src ] ->
          Pricklypear_lisp.Eval.eval_string_direct
            ~prims:Pricklypear_lisp.Eval.empty_prims !t.env src
```

```
      | _ ->
          raise
            (Pricklypear_lisp.Ast.Interp_error
               (Pricklypear_lisp.Ast.User_error
                  ("eval-string: need 1 string", "bad-args"))));
  (* habitat-book.eval-examples: (example-eval src) -> VStr (rendered
     value). Parses + evals Nopales source in a RESTRICTED environment:
     whitelisted pure + read-only-DB prim table (absent prims fail
     closed), static gate on mutation special forms + capabilities,
     child of the session env (bindings resolve, nothing persists),
     fresh step budget + wall-clock cap. Read-time eval in the
     reader's session; the POST /book/eval-example endpoint shares
     [example_eval_engine]. *)
  Primitives.register prim_table "example-eval"
    (fun args ->
      match args with
      | [ Pricklypear_lisp.Ast.VStr src ]
      | [ Pricklypear_lisp.Ast.VSymbol src ] ->
          let v =
            try Ok (example_eval_engine !t src)
            with Example_eval.Refused reason ->
              Error ("refused: " ^ reason)
          in
          (match v with
           | Ok v -> Pricklypear_lisp.Ast.VStr
     (Pricklypear_lisp.Reader.pp_value v)
           | Error msg ->
               (raise
                  (Pricklypear_lisp.Ast.Interp_error
                     (Pricklypear_lisp.Ast.User_error
                        ("example-eval: " ^ msg, "denied")))))
      | _ ->
          (raise
             (Pricklypear_lisp.Ast.Interp_error
                (Pricklypear_lisp.Ast.Prim_error
                   ("example-eval", "need 1 source string", "bad-args")))));
  Primitives.register prim_table "load-library"
    (fun args ->
      match args with
      | [ Pricklypear_lisp.Ast.VStr name ] ->
          (* load_library busts shell cache + refresh_ui_shell once. *)
          let r = load_library !t name in
          (match r with
           | Ok msg -> (Pricklypear_lisp.Ast.VStr msg)
           | Error e ->
               (Pricklypear_lisp.Ast.VStr ("load-library: " ^ e)))
      | _ ->
          (raise (Pricklypear_lisp.Ast.Interp_error
     (Pricklypear_lisp.Ast.Prim_error ("load-library", "need 1 string",
     "bad-args")))));
  (* (reseed-package pkg) -- admin-only upsert libs/<pkg> onto system/main.
     *)
  Primitives.register prim_table "reseed-package"
    (fun args ->
      match args with
      | [ Pricklypear_lisp.Ast.VStr pkg ] ->
          let r = reseed_package !t pkg in
          (match r with
           | Ok msg -> (Pricklypear_lisp.Ast.VStr msg)
           | Error e ->
               (Pricklypear_lisp.Ast.VStr ("reseed-package: " ^ e)))
      | _ ->
          (raise (Pricklypear_lisp.Ast.Interp_error
     (Pricklypear_lisp.Ast.Prim_error ("reseed-package", "need
     package-str", "bad-args")))));
  let final_prims = prim_table in
  let kernel_catalog = Primitives.catalog prim_table in
  (* Mutate the prims field in place (rather than building a fresh
     record via { (!t) with prims = ... }). Closures captured by
     register_session_control_prims / register_crud_prims read
     t.prims at CALL time (e.g. __with_transaction__'s Eval.apply
     ~prims:t.prims) and need the populated table, not the temporary
     snapshot from step 3. Lands it29: this was latent until
```

```
     with-transaction's body needed to call back into the prim
     registry itself. The earlier control prims (fork/merge/route/
     checkout) didn't trip this because they only invoke Store.*
     functions, never Eval.apply against the prims table. *)
  (!t).prims <- final_prims;
  (!t).kernel_catalog <- kernel_catalog;
  (* Bind load prims into env (registered after the bulk VPrim bind). *)
  Pricklypear_lisp.Env.define env "load-string" (Pricklypear_lisp.Ast.VPrim
     "load-string");
  Pricklypear_lisp.Env.define env "eval-string" (Pricklypear_lisp.Ast.VPrim
     "eval-string");
  Pricklypear_lisp.Env.define env "example-eval"
     (Pricklypear_lisp.Ast.VPrim "example-eval");
  Pricklypear_lisp.Env.define env "load-library"
     (Pricklypear_lisp.Ast.VPrim "load-library");
  Pricklypear_lisp.Env.define env "reseed-package"
     (Pricklypear_lisp.Ast.VPrim "reseed-package");
  let session : t = !t in
  (* Boot order (personal-branch safety):
       1. ensure username/personal exists (empty child of system/main)
       2. seed_stdlib writes package defaults ONLY to system/main
       3. checkout personal + rehydrate env (inherits main via CTE)
       4. refresh ui shell from the personal-visible ui/nav
     User (define)s land on personal and are never wiped by reseed. *)
  let _ = ensure_personal_branch session in
  let () = seed_stdlib session in
  let () = checkout_personal session in
  let () = refresh_ui_shell session in
  session


let uptime_seconds (t : t) : int =
  int_of_float (Unix.gettimeofday () -. t.started_at)

let branch_name (t : t) : string = t.manifest.branch_name
let username (t : t) : string = t.manifest.username
(** Snapshot of the attached identity/branch fields for per-request context.
    Record copy is shallow; all fields are immutable strings/ints. *)
let manifest (t : t) : Pricklypear_common.Manifest.t = t.manifest
let is_admin (t : t) : bool = t.is_admin
let store (t : t) : Store.t = t.store
let branch_id (t : t) : string = t.manifest.branch_id

(** Snapshot accessors for per-request context build
     (image.per-request-context).
    env is swapped by reference on rehydrate (invariant 4a -- old record
     stays
    alive for captured closures), so holding this reference is a snapshot.
    routes is cleared in place on rehydrate, so the caller must copy it via
    Request_context.copy_routes when building the snapshot. *)
let env (t : t) : Pricklypear_lisp.Ast.env = t.env

let routes (t : t) : (string, string * string) Hashtbl.t = t.routes
let auth_token (t : t) : string option = t.auth_token

(** Attach authenticated identity for this request (auth.web-login).
    Updates manifest user/username, is_admin, optional raw token.
    When [checkout_personal] is true, ensures + checks out
    username/personal (login / first attach). *)
let attach_identity
    (t : t)
    ~(user_id : string)
    ~(username : string)
    ~(is_admin : bool)
    ?(auth_token : string option = None)
    ?(checkout_personal_now = false)
    ?(user_type : string = "human")
    ?(parent_user_id : string option = None)
    () : unit =
  let prev = t.manifest.username in
  t.manifest <-
    { t.manifest with user_id; username };
  t.is_admin <- is_admin;
```

```
  t.auth_token <- auth_token;
  if checkout_personal_now || prev <> username then
    if user_type = "agent" then
      (* Agent boots into <parent>/agent, not <username>/personal. *)
      (match parent_user_id with
       | Some pid ->
           let parent_opt = Auth_store.get_user_by_id t.store pid in
           (match parent_opt with
            | Some p ->
                let _ = ensure_agent_branch t ~parent_username:p.username in
                checkout_agent t ~parent_username:p.username
            | None ->
                Printf.eprintf
                  "[session] agent %s has no parent; staying on %s\n%!"
                  username t.manifest.branch_name;
                ())
       | None ->
           Printf.eprintf
             "[session] agent %s has no parent_user_id; staying on %s\n%!"
             username t.manifest.branch_name;
           ())
    else
      let _ = ensure_personal_branch t in
      checkout_personal t
  else
    ()

(** Anonymous request identity (auth.route-visibility).
    system/main only; no personal garden; no admin. *)
let attach_anonymous (t : t) : unit =
  t.manifest <-
    { t.manifest with
      user_id = "anonymous"
    ; username = "anonymous"
    };
  t.is_admin <- false;
  t.auth_token <- None;
  (* Prefer system/main for package defaults. *)
  let found =
    Store.lookup_branch_id_by_name t.store ~owner:"system" ~name:"main"
  in
  match found with
  | None -> ()
  | Some id ->
      if t.manifest.branch_id = id then ()
      else checkout_to t ~branch_id:id ~branch_ref:"system/main"

let clear_auth_token (t : t) : unit =
  t.auth_token <- None

(** Resolve owner/name ref and checkout. *)
let checkout_ref (t : t) (branch_ref : string) : (unit, string) result =
  let owner, name = split_branch_ref branch_ref in
  (* i-1787970640: same branch-owner rule as the checkout prim -- this is
     the funnel behind http_branches.branches_checkout and
     branches_unoverride. *)
  if not (branch_owner_authorized t owner) then
    (Error (Printf.sprintf "denied: branch %s is owned by %s"
              branch_ref owner))
  else
    let ref' = owner ^ "/" ^ name in
    let found = Store.lookup_branch_id_by_name t.store ~owner ~name in
    match found with
    | None -> (Error "no such branch")
    | Some id ->
        let () = checkout_to t ~branch_id:id ~branch_ref:ref' in
        (Ok ())

(** Delete a function row on the current branch (unoverride). Rehydrates. *)
let unoverride (t : t) (name : string) : (bool, string) result =
  (* i-1787970640: current-branch owner must pass the branch-owner rule
     (same guard as the unbind prim; branches_unoverride ignores its
     checkout_ref result, so this is its own gate). *)
```

```
  let cur_owner, _cur_name = split_branch_ref t.manifest.branch_name in
  if not (branch_owner_authorized t cur_owner) then
    (Error (Printf.sprintf "denied: branch %s is owned by %s"
              t.manifest.branch_name cur_owner))
  else
    let bid = Store.current_branch_id t.store in
    let deleted =
      Store.delete_function_on_branch t.store ~branch_id:bid ~name
    in
    let () = rehydrate t in
    let () = refresh_ui_shell t in
    (Ok deleted)

(** Soft habitat-workflow lint lines for packages touched by [names].
    Best-effort: missing libs/spec ? empty string. Never blocks promote.
    Uses empty branch arg so lint does not checkout away from the session.
    *)
let promote_soft_warnings (t : t) ~(names : string list) : string =
  let pkgs =
    names
    |> List.filter_map (fun n ->
           match String.index_opt n '/' with
           | None -> None
           | Some i when i > 0 -> Some (String.sub n 0 i)
           | Some _ -> None)
    |> List.sort_uniq String.compare
  in
  if pkgs = [] then ""
  else begin
    (* Load lint deps if present; ignore failures (soft). *)
    List.iter
      (fun lib -> ignore (load_library t lib))
      [ "list"; "outline"; "notes"; "spec" ];
    let lines =
      List.filter_map
        (fun pkg ->
          let expr =
            Printf.sprintf "(spec/lint-summary %S %S)" pkg ""
          in
          try
            match eval_one t expr with
            | Pricklypear_lisp.Ast.VStr s when String.trim s <> "" ->
                Some (String.trim s)
            | _ -> None
          with _ -> None)
        pkgs
    in
    String.concat " ? " lines
  end

(** Promote selected names from source ref onto system/main (admin policy).
    Returns (count, soft_warnings). Soft warnings never block. *)
let promote_to_main (t : t) ~(source_ref : string) ~(names : string list)
  : (int * string, string) result =
  if names = [] then (Error "no names selected")
  else
    let owner, name = split_branch_ref source_ref in
    let src =
      Store.lookup_branch_id_by_name t.store ~owner ~name
    in
    match src with
    | None -> (Error "no such source branch")
    | Some source_id ->
        (* Policy: only admin username may write system/main via promote
    UI. *)
        if (not t.is_admin)
           && t.manifest.username <> "admin"
           && t.manifest.username <> "system" then
          (Error "promote to system/main requires admin")
        else
          let soft = promote_soft_warnings t ~names in
          let n =
            Store.promote_functions t.store
```

```
              ~source_id ~target_id:Store.system_main_id ~names
          in
          (* If we're on main, rehydrate so env sees new bodies. *)
          let () =
            if Store.current_branch_id t.store = Store.system_main_id then
              rehydrate t
            else ()
          in
          (Ok (n, soft))

(** Full fork: parent_ref -> owner/child_name (uses Store.fork_branch
    copy). *)
let fork_ref (t : t) ~(parent_ref : string) ~(child_ref : string)
  : (string, string) result =
  let p_owner, p_name = split_branch_ref parent_ref in
  let c_owner, c_name = split_branch_ref child_ref in
  (* i-1787970640: child_owner must pass the branch-owner rule (same as
     the fork prim -- blocks pre-claiming another user's garden). This is
     the funnel behind http_branches.branches_fork. *)
  if not (branch_owner_authorized t c_owner) then
    (Error (Printf.sprintf "denied: branch %s is owned by %s"
              child_ref c_owner))
  else
    let parent_id =
      Store.lookup_branch_id_by_name t.store ~owner:p_owner ~name:p_name
    in
    match parent_id with
    | None -> (Error "no such parent branch")
    | Some pid ->
        let created =
          Store.fork_branch t.store ~parent_id:pid ~owner:c_owner
            ~child_name:c_name
        in
        (match created with
         | None -> (Error "fork failed")
         | Some id -> (Ok id))

(* Look up a route's handler name. Returns None if the route isn't
   registered on this branch. Called by http.ml's request_handler. *)
let lookup_route (t : t) (route : string) : string option =
  match Hashtbl.find_opt t.routes route with
  | Some (handler, _) -> Some handler
  | None -> None

let lookup_route_meta (t : t) (route : string)
  : (string * string) option =
  Hashtbl.find_opt t.routes route

let route_visibility (t : t) (route : string) : string option =
  match Hashtbl.find_opt t.routes route with
  | Some (_, vis) -> Some vis
  | None -> None

let is_anonymous (t : t) : bool = t.manifest.username = "anonymous"

let kernel_catalog (t : t) : (string * string) list = t.kernel_catalog

let ui_nav_html (t : t) : string option = t.ui_nav_html

let set_ui_nav_html (t : t) (s : string option) : unit = t.ui_nav_html <- s

let ui_css (t : t) : string option = t.ui_css

let set_ui_css (t : t) (s : string option) : unit = t.ui_css <- s


let list_pages (t : t) : (string * string) list =
  Hashtbl.fold
    (fun route (handler, _) acc -> (route, handler) :: acc) t.routes []
  |> List.sort (fun (a, _) (b, _) -> String.compare a b)

let list_pages_meta (t : t) : (string * string * string) list =
  Hashtbl.fold
```

```
    (fun route (handler, vis) acc -> (route, handler, vis) :: acc) t.routes
     []
  |> List.sort (fun (a, _, _) (b, _, _) -> String.compare a b)

let set_route (t : t) ~(route : string) ~(handler : string)
    ?visibility () : unit =
  let vis =
    match visibility with
    | Some v -> Store.normalize_visibility v
    | None ->
        (match Hashtbl.find_opt t.routes route with
         | Some (_, v) -> v
         | None -> "private")
  in
  Hashtbl.replace t.routes route (handler, vis);
  let _ =
    match visibility with
    | Some v ->
        Store.register_route t.store ~route ~handler
          ~visibility:(Store.normalize_visibility v) ()
    | None -> Store.register_route t.store ~route ~handler ()
  in
  ()

(** Result of invoking a route handler. Errors keep the real message
    (e.g. "budget exceeded") so HTTP can show it -- never mask as bare
     None. *)
type handler_call =
  | Handler_ok of Pricklypear_lisp.Ast.value
  | Handler_unbound
  | Handler_error of string

(* Invoke a named handler function (looked up via env). The handler
   must be a zero-arg VLambda or a VPrim name.
   When [~request] is provided, request-* prims see that context for
   the duration of the call (cleared in finalize). *)
let call_handler (t : t) ?request (handler_name : string)
  : handler_call =
  (* Resolve the handler env against the per-request snapshot when bound
     (image.per-request-context): a concurrent attach_anonymous +
     checkout_to mid-request would otherwise rehydrate t.env to system/main
     and this lookup would miss personal-branch handlers. Fall back to t.env
     when no context (REPL, /api/eval outside HTTP gate, pre-gate paths). *)
  let env =
    match Request_context.get () with
    | Some ctx -> Request_context.env ctx
    | None -> t.env
  in
  match Pricklypear_lisp.Env.lookup env handler_name with
  | None -> Handler_unbound
  | Some _fn ->
      (* Evaluate (handler-name) -- zero args. Re-uses eval's apply path
         so VLambda closures AND VPrim dispatch both work. *)
      let run () =
        (try
      let v =
               eval_string_rim t ("(" ^ handler_name ^ ")")
             in
             (Handler_ok v)
    with exn ->
      let msg = pp_interp_exn exn in
             Ops_incidents.emit_and_maybe_file t
               ~level:Ops_log.Error ~component:"handler"
               ~file:(not (Ops_incidents.is_expected_runtime_error exn))
               ~context:[("handler", handler_name)]
               msg;
             (Handler_error msg))
      in
      (match request with
       | None -> run ()
       | Some req ->
           t.current_request <- Some req;
           Fun.protect
```

```
            ~finally:(fun () -> t.current_request <- None)
            run)

(* Late-bind the route/dispatch thunk now that call_handler exists. *)
let () =
  route_dispatch_fn :=
    (fun t handler req ->
      (* Bind a per-call Request_context snapshot so handler prims
         (whoami, current-branch, ui/css, author-scoped reads) see the
         session's manifest/env/routes -- mirrors http.ml's snapshot_ctx.
         Without this, notes/page renders with anonymous author and
         the fixture's ol-nodes don't appear. *)
      let ctx =
        Request_context.make
          ~manifest:t.manifest
          ~env:t.env
          ~routes:(Request_context.copy_routes t.routes)
      in
      Request_context.with_current ctx (fun () ->
        match call_handler t ~request:req handler with
        | Handler_ok v ->
          Pricklypear_lisp.Ast.VStr (Prims_hyper.render_handler_value v)
        | Handler_unbound ->
          Pricklypear_lisp.Ast.VStr
            (Printf.sprintf "route/dispatch: unbound handler %s" handler)
        | Handler_error msg ->
          Pricklypear_lisp.Ast.VStr (Printf.sprintf "route/dispatch: %s"
    msg)))
```

106 image/lib/session_registry.ml

```
(** Pricklypear Image -- per-identity session registry
    (image.per-identity-sessions).

    One [Session.t] per attached identity instead of one shared
    session that every request flips to its own branch:

    - the anonymous session serves all unauthenticated traffic, parked
      on system/main;
    - each authenticated username gets its own session, built lazily
      on first attach, parked on that identity's branch (personal for
      humans, <parent>/agent for agents), with its OWN Store.t (own
      pgx connection -- checkout never moves another identity's branch);
    - attach = select (or create) + a cheap staleness stamp check.
      Attach never hydrates an unchanged session
      (image.boot-request-cost) and never mutates another identity's
      session.

    Sessions are never expired in v1 (user count is small; revisit
    with an LRU if public signup appears). *)

module Manifest = Pricklypear_common.Manifest

type t = {
  cfg : Store.config;
  mu : Eio.Mutex.t;
  sessions : (string, Session.t) Hashtbl.t;
  boot : Session.t;
}

let create ~(boot : Session.t) : t =
  {
    cfg = Store.config boot.Session.store;
    mu = Eio.Mutex.create ();
    sessions = Hashtbl.create 8;
    boot;
  }

(** The boot session keeps its jobs (REPL endpoint, seed); the auth
    lookups in resolve_identity also use its store. *)
let boot_session (t : t) : Session.t = t.boot

let fresh_store (t : t) ~(branch_id : string) : Store.t =
  let store = Store.make ~cfg:t.cfg ~branch_id () in
  Store.connect store;
  store

let make_anonymous_session (t : t) : Session.t =
  let manifest =
    Manifest.make ~user_id:"anonymous" ~username:"anonymous"
      ~branch_id:Store.system_main_id ~branch_name:"system/main"
      ~boot_mode:t.boot.Session.manifest.Manifest.boot_mode
      ~image_version:t.boot.Session.manifest.Manifest.image_version ()
  in
  let store = fresh_store t ~branch_id:Store.system_main_id in
  let s = Session.make ~manifest ~store () in
  Session.rehydrate s;
  s

let make_user_session (t : t) (user : Auth_store.user) : Session.t =
  (* Start on system/main; attach_identity with checkout_personal_now
     checks out the identity's home branch (personal / agent) and
     rehydrates onto it. *)
  let manifest =
    Manifest.make ~user_id:user.Auth_store.id ~username:user.username
      ~branch_id:Store.system_main_id ~branch_name:"system/main"
      ~boot_mode:t.boot.Session.manifest.Manifest.boot_mode
      ~image_version:t.boot.Session.manifest.Manifest.image_version ()
  in
  let store = fresh_store t ~branch_id:Store.system_main_id in
  let s = Session.make ~manifest ~store () in
  (* Fresh session: force the home-branch checkout now (personal for
```

```
     humans, <parent>/agent for agents). attach_identity handles both. *)
  Session.attach_identity s ~user_id:user.Auth_store.id
    ~username:user.Auth_store.username ~is_admin:user.Auth_store.is_admin
    ~auth_token:None ~checkout_personal_now:true
    ~user_type:user.Auth_store.user_type
    ~parent_user_id:user.Auth_store.parent_user_id ();
  s

(** Get-or-create the session for [key] (serialized -- creation opens a
    pgx connection; rare path only). *)
let get_or_create (t : t) (key : string) (make : unit -> Session.t)
  : Session.t =
  Eio.Mutex.use_rw ~protect:true t.mu (fun () ->
      match Hashtbl.find_opt t.sessions key with
      | Some s -> s
      | None ->
          let s = make () in
          Hashtbl.replace t.sessions key s;
          s)

(** Select the session for this request's identity. Pure selection:
    no branch movement, no cross-identity mutation. [anon] picks the
    parked anonymous session; a resolved user picks their own. The
    caller still runs attach_from_user / attach_anonymous on the
    returned session (cheap no-op when the identity matches) and
    [Session.ensure_fresh] for the staleness stamp. *)
let select (t : t) ~(user : Auth_store.user option) : Session.t =
  match user with
  | None -> get_or_create t "anonymous" (fun () -> make_anonymous_session t)
  | Some u -> get_or_create t u.Auth_store.username (fun () ->
     make_user_session t u)
```

107 image/lib/session_types.ml

```
(** Session record and HTTP request types (shared by Session +
    Agent_tools). *)
type http_method = [ `GET | `POST ]

type request = {
  method_ : http_method;
  path : string;
  params : (string * string) list;  (** form fields win over query *)
  body : string;
  (** Lowercase header names ? values (for HX-Request, etc.). *)
  headers : (string * string) list;
}

(** Impact of a successful define/install (image.inspector I2). *)
type impact_event = {
  name : string;
  at : float;
  branch : string;
  author : string;
  dependents : string list;
  routes : string list;
  status : string; (** pending | smoked | dismissed *)
}

type change_event = {
  op : string; (** define | route | delete *)
  name : string;
  at : float;
  branch : string;
  author : string;
}

type t = {
  mutable manifest : Pricklypear_common.Manifest.t;
  store : Store.t;
  (* image.http-robustness invariant 4a: env is mutable so rehydrate can
     SWAP it to a fresh record (new Hashtbl, same parent chain) instead
     of clearing the existing binding table in place. VLambda closures
     capture `env` by reference; clearing in place would corrupt an
     in-flight page's captured environment. Swapping leaves the old
     record intact for any closure that still holds it, so concurrent
     rehydrate cannot break a running page. *)
  mutable env : Pricklypear_lisp.Ast.env;
  (** image.per-identity-sessions: Store.visible_stamp captured at the
      last load_defs for the current branch. [ensure_fresh] compares
      the live stamp at attach time and rehydrates only on mismatch. *)
  mutable load_stamp : string;
  (** Lwt prim table for [Control_rim.run_lwt] (H1.7). Interpreter is
      direct-style; I/O prims are serviced only at this rim. *)
  mutable prims : Primitives.t;
  (* In-memory route table mirroring view_bindings for the branch.
     Populated by load_defs_async at session-make time; (route ...)
     updates also live-persist to view_bindings AND update this table.
     Value is (handler, visibility) where visibility is public|private.
     http.ml reads from here on every request (no DB round-trip). *)
  routes : (string, string * string) Hashtbl.t;
  mutable started_at : float;
  (** Kernel prim catalog for Functions browser: (name, summary) list. *)
  mutable kernel_catalog : (string * string) list;
  (** Cached HTML from ui/nav when defined; None = use OCaml default nav. *)
  mutable ui_nav_html : string option;
  (** Cached CSS from (ui/css) when defined; None = emergency_css in http.
     *)
  mutable ui_css : string option;
  (** image.product-theme: shell cache invalidation is DEPENDENCY-TRACKED.
      Each successful shell eval records the exact binding names it read
      (via Env.lookup_hook); a define busts the cache iff the defined name
      is in this set. No TTL, no name-prefix heuristic. Empty set + cached
      value = never busted by define. Reset on checkout (per-branch). *)
  mutable ui_shell_reads : string list;
  (** image.http-robustness invariant 3: refresh_ui_shell runs on every
```

```
      request. When a shell eval FAILS, do not re-attempt + re-warn per
      request: suppress retry for retry_at seconds (single warn per
      window). Reset to 0.0 on checkout and on successful refresh. *)
  mutable ui_shell_retry_at : float;
  (** When true, the eval_one define-interceptor suppresses its
      refresh_ui_shell call. Used during seed_stdlib so that defining
      ui/nav mid-seed (before notes/capture/nav is bound) does not
      trigger an eager refresh that fails with unbound: notes/capture/nav.
      The boot sequence calls refresh_ui_shell once AFTER seeding
     completes. *)
  mutable suppress_refresh_on_define : bool;
  (** branching.seed-policy: when true, define/route persist is
      insert-if-absent (boot seed into system/main). Interactive
      load-library / define keep upsert (false). *)
  mutable seed_additive : bool;
  (** Active HTTP request for user-route dispatch; None in REPL. *)
  mutable current_request : request option;
  (** When true, (checkout)/(fork)/(merge) are blocked so the agent
      cannot leave the current branch mid-task. *)
  mutable agent_branch_lock : bool;
  (** Admin flag from users.is_admin (auth identity). *)
  mutable is_admin : bool;
  (** Raw pp_session cookie for the active HTTP request, if any. *)
  mutable auth_token : string option;
  (** image.inspector I2: recent define impacts (newest first, max 32). *)
  mutable impact_queue : impact_event list;
  (** branching.change-set: recent define/route ops (newest first, max 64).
     *)
  mutable change_transcript : change_event list;
  (** lisp-interpreter.package-surface: while loading a lib.pp that opened
      with (package NAME (export ?)), stamp new defines with this package
      name. None outside a package load / REPL package form. *)
  mutable current_package : string option;
  (** single-image.net-prims: session-local stream table for tcp-connect.
      Maps handle-id (UUID string) -> stream handle. Populated by
      (tcp-connect ...); drained by (stream-close ...) and by the switch
      finishing. Quota enforced at insert. The stream type is defined
      here to avoid a module cycle (Net_prims references it). *)
  mutable net_streams : (string, Unix.file_descr) Hashtbl.t;
}

(* Load persisted function_defs + routes into the env.
   Returns when finished (caller may then seed libs/ which overwrite).

   Load order is critical: many personal pins are *values* evaluated at
   define-time (e.g. ui/menu = (list (menu/link ?))), not lambdas.
   Those bodies need parent helpers (menu/link, menu/group, ?) already
   bound. Store.list_function_names returns effective defs only (closest
   branch wins) but does NOT topological-order dependencies across the
   parent chain. We therefore load the full chain parents-first
   (depth DESC) and redefine so child overrides win last.

   Historical bug: depth ASC + first-wins loaded personal ui/menu BEFORE
   system menu/link ? define failed ? ui/menu unbound ? (ui/nav) fails ?
   silent fallback to OCaml default_nav_nodes ("navbar disappeared"). *)
```

108 image/lib/store.ml

```
(** Pricklypear Image -- Storage layer (branch-aware queries + fork/merge).
    Connection/tx substrate is [Store_db], included below.
    H3: SQL via [exec] on dual conn (lwt|eio from PP_IO). *)
include Store_db

(* ---- one-tree: reads source from the tree-backed views ----
   tree_function_defs / tree_type_defs / tree_view_bindings / tree_data_rows
   are projections over tree_nodes. The scaffolding tables (function_defs,
   type_defs, view_bindings, data_rows) are derived via the reverse mirror
   and kept warm for residual readers / fork-copy paths. *)
let fn_rel () = "tree_function_defs"
let ty_rel () = "tree_type_defs"
let rt_rel () = "tree_view_bindings"
let pkg_rel () = "tree_package_registry"

(** Cheap visibility signature for staleness checks
    (image.per-identity-sessions): count + max(updated_at) of
    function/type/route defs over the current branch chain, one round
    trip. A parked session compares this against the value from its
    last load; mismatch => rehydrate. *)
let visible_stamp (t : t) : string =
  match t.conn with
  | None -> ""
  | Some conn ->
      (try
         let agg rel =
           "(SELECT count(*)::text || '|' ||
    COALESCE(max(updated_at)::text, '-') "
           ^ "FROM " ^ rel ^ " d JOIN chain ON d.branch_id = chain.id)" in
         let sql =
           "WITH RECURSIVE chain AS ( "
           ^ "  SELECT id, parent_id FROM branches WHERE id = $1 "
           ^ "  UNION ALL "
           ^ "  SELECT b.id, b.parent_id FROM branches b JOIN chain c ON
    b.id = c.parent_id "
           ^ ") SELECT " ^ agg (fn_rel ()) ^ " || '/' || " ^ agg (ty_rel ())
           ^ " || '/' || " ^ agg (rt_rel ()) in
         match exec conn ~params:[ Pgx.Value.of_string t.branch_id ] sql
    with
         | [ [ c ] ] -> (match Pgx.Value.to_string c with Some s -> s |
    None -> "")
         | _ -> ""
       with _ -> "")

(* Branch-aware lookup. Walks the branch parent chain via a recursive
   CTE so that a definition in [main] is visible from a child branch,
   and a definition in the child OVERRIDES the same name in main
   (per spec storage.no-inheritance: OVERRIDE, not UNION).

   Returns (body, author, doc option). Errors/misses are logged and
   return None so the REPL keeps working with no DB. *)
let lookup_function (t : t) (name : string)
  : (Pricklypear_lisp.Ast.expr * string * string option) option =
  match t.conn with
  | None -> None
  | Some conn ->
      (try
      (* Recursive CTE: start at the session's branch and walk parents.
             Return the FIRST row matching [name] when ordered by depth
             ascending (closest ancestor wins). *)
          let sql =
            "WITH RECURSIVE chain AS ( " ^
            "  SELECT id, parent_id, 0 AS depth FROM branches WHERE id = $1
    " ^
            "  UNION ALL " ^
            "  SELECT b.id, b.parent_id, c.depth + 1 " ^
            "  FROM branches b JOIN chain c ON b.id = c.parent_id " ^
            ") SELECT fd.body, fd.author, fd.doc FROM " ^ fn_rel () ^ " fd
    " ^
            "JOIN chain ON fd.branch_id = chain.id " ^
            "WHERE fd.name = $2 " ^
```

```
            "ORDER BY chain.depth ASC LIMIT 1" in
          let rows =
            exec conn
              ~params:[ Pgx.Value.of_string t.branch_id;
                        Pgx.Value.of_string name ]
              sql in
          (match rows with
           | [] -> None
           | row :: _ ->
               (* body jsonb, author text, doc text nullable. *)
               (match row with
                | [ body_cell; author_cell; doc_cell ] ->
                    (match Pgx.Value.to_string body_cell,
                            Pgx.Value.to_string author_cell with
                     | Some body_str, Some author ->
                        let doc = Pgx.Value.to_string doc_cell in
                        (match Yojson.Safe.from_string body_str with
                         | j ->
                             (match Pricklypear_lisp.Ast.expr_of_json j with
                              | Ok e -> (Some (e, author, doc))
                              | Error msg ->
                                  Printf.eprintf
                                    "[store] lookup_function: parse %s\n"
    msg;
                                  None)
                         | exception Yojson.Json_error msg ->
                             Printf.eprintf
                               "[store] lookup_function: json %s\n" msg;
                             None)
                     | _ -> None)
                | _ -> None))
    with exn ->
      Printf.eprintf "[store] lookup_function error: %s\n"
            (Printexc.to_string exn);
          None)

(* List all function names visible to this branch (via the same
   recursive-CTE override walk). Used at session-make time to load
   persisted definitions into the env. *)
(** Effective function defs for the current branch: parent-chain walk,
    closest definition wins (depth ASC, first occurrence kept).
    [depth] is 0 for rows owned by the current branch, >0 if inherited.
    Callers that only need name/body/author can ignore depth. *)
let list_function_names_meta (t : t)
  : (string * Pricklypear_lisp.Ast.expr * string * int) list =
  match t.conn with
  | None -> []
  | Some conn ->
      (try
      let sql =
            "WITH RECURSIVE chain AS ( " ^
            "  SELECT id, parent_id, 0 AS depth FROM branches WHERE id = $1
    " ^
            "  UNION ALL " ^
            "  SELECT b.id, b.parent_id, c.depth + 1 " ^
            "  FROM branches b JOIN chain c ON b.id = c.parent_id " ^
            ") SELECT fd.name, fd.body, fd.author, chain.depth, fd.package
    " ^
            "FROM " ^ fn_rel () ^ " fd " ^
            "JOIN chain ON fd.branch_id = chain.id " ^
            "ORDER BY chain.depth ASC, fd.name ASC" in
          let rows =
            exec conn
              ~params:[ Pgx.Value.of_string t.branch_id ]
              sql in
          let seen = Hashtbl.create 64 in
          let acc =
            List.filter_map
              (fun row ->
                match row with
                | [ name_c; body_cell; author_cell; depth_c; _pkg_c ] ->
                    (match Pgx.Value.to_string name_c,
                            Pgx.Value.to_string body_cell,
```

```
                           Pgx.Value.to_string author_cell,
                           Pgx.Value.to_string depth_c with
                     | Some name, Some body_str, Some author, Some d_s ->
                         if Hashtbl.mem seen name then None
                         else begin
                           Hashtbl.replace seen name ();
                           let depth =
                             int_of_string_opt d_s |> Option.value
    ~default:0
                           in
                           (try
                              let j = Yojson.Safe.from_string body_str in
                              (match Pricklypear_lisp.Ast.expr_of_json j
    with
                               | Ok e -> Some (name, e, author, depth)
                               | Error _ -> None)
                            with Yojson.Json_error _ -> None)
                         end
                     | _ -> None)
                | _ -> None)
             rows
         in
         acc
    with exn ->
      Printf.eprintf "[store] list_function_names error: %s\n"
        (Printexc.to_string exn);
         [])

(** Effective defs only (deduped). Same as [list_function_names_meta]
    without depth. *)
let list_function_names (t : t)
  : (string * Pricklypear_lisp.Ast.expr * string) list =
  let rows = list_function_names_meta t in
  (List.map (fun (n, e, a, _) -> (n, e, a)) rows)

(** Substring search over effective function bodies (JSONB ::text) on the
    current branch inheritance chain. Closest definition wins (depth ASC
    dedup). [needle] is matched with SQL ILIKE '%needle%' (no user-supplied
    wildcards expanded beyond that). Returns names only, ordered by name. *)
let search_function_bodies (t : t) ~(needle : string) ~(limit : int)
  : string list =
  let needle = String.trim needle in
  let limit = if limit <= 0 then 200 else min limit 2000 in
  if needle = "" then []
  else
    match t.conn with
    | None -> []
    | Some conn ->
        (try
           let sql =
             "WITH RECURSIVE chain AS ( " ^
             "  SELECT id, parent_id, 0 AS depth FROM branches WHERE id =
    $1 " ^
             "  UNION ALL " ^
             "  SELECT b.id, b.parent_id, c.depth + 1 " ^
             "  FROM branches b JOIN chain c ON b.id = c.parent_id " ^
             "), ranked AS ( " ^
             "  SELECT fd.name, chain.depth, " ^
             "         row_number() OVER (PARTITION BY fd.name ORDER BY
    chain.depth ASC) AS rn " ^
             "  FROM " ^ fn_rel () ^ " fd " ^
             "  JOIN chain ON fd.branch_id = chain.id " ^
             "  WHERE fd.body::text ILIKE '%' || $2 || '%' " ^
             ") SELECT name FROM ranked WHERE rn = 1 ORDER BY name ASC
    LIMIT $3"
           in
           let rows =
             exec conn
               ~params:
                 [ Pgx.Value.of_string t.branch_id
                 ; Pgx.Value.of_string needle
                 ; Pgx.Value.of_string (string_of_int limit)
                 ]
```

```
              sql
          in
          List.filter_map
            (fun row ->
              match row with
              | [ name_c ] -> Pgx.Value.to_string name_c
              | _ -> None)
            rows
        with exn ->
          Printf.eprintf "[store] search_function_bodies error: %s\n"
            (Printexc.to_string exn);
          []

(** Full branch-chain defs for env rehydrate: parents first (depth DESC),
    then children. Includes ALL rows (not deduped) so Session.load_defs can
    redefine -- last write wins (child override). 4th = package provenance
    (None = untagged / always load). See session.load_defs note. *)
let list_function_names_for_load (t : t)
  : (string * Pricklypear_lisp.Ast.expr * string * string option) list =
  match t.conn with
  | None -> []
  | Some conn ->
      (try
      let sql =
            "WITH RECURSIVE chain AS ( " ^
            "  SELECT id, parent_id, 0 AS depth FROM branches WHERE id = $1 " ^
            "  UNION ALL " ^
            "  SELECT b.id, b.parent_id, c.depth + 1 " ^
            "  FROM branches b JOIN chain c ON b.id = c.parent_id " ^
            ") SELECT fd.name, fd.body, fd.author, chain.depth, fd.package " ^
            "FROM " ^ fn_rel () ^ " fd " ^
            "JOIN chain ON fd.branch_id = chain.id " ^
            "ORDER BY chain.depth DESC, fd.name ASC" in
          let rows =
            exec conn
              ~params:[ Pgx.Value.of_string t.branch_id ]
              sql in
          let acc =
            List.filter_map
              (fun row ->
                match row with
                | [ name_c; body_cell; author_cell; _depth_c; pkg_c ] ->
                    (match Pgx.Value.to_string name_c,
                           Pgx.Value.to_string body_cell,
                           Pgx.Value.to_string author_cell with
                     | Some name, Some body_str, Some author ->
                         let pkg = Pgx.Value.to_string pkg_c in
                         (try
                            let j = Yojson.Safe.from_string body_str in
                            (match Pricklypear_lisp.Ast.expr_of_json j with
                             | Ok e -> Some (name, e, author, pkg)
                             | Error _ -> None)
                          with Yojson.Json_error _ -> None)
                     | _ -> None)
                | _ -> None)
              rows
          in
          acc
      with exn ->
        Printf.eprintf "[store] list_function_names_for_load error: %s\n"
          (Printexc.to_string exn);
          []

(* ---- own-branch stats / overrides (no recursive CTE) ---- *)

let count_functions_on_branch (t : t) ~(branch_id : string) : int =
  match t.conn with
  | None -> 0
  | Some conn ->
      (try
      let sql =
```

```
            "SELECT COUNT(*)::text FROM function_defs WHERE branch_id = $1"
    in
          let rows =
            exec conn ~params:[ Pgx.Value.of_string branch_id ] sql
          in
          (match rows with
           | [ [ c ] ] ->
               (match Pgx.Value.to_string c with
                | Some s -> (int_of_string_opt s |> Option.value ~default:0)
                | None -> 0)
           | _ -> 0)
    with exn ->
      Printf.eprintf "[store] count_functions_on_branch: %s\n"
            (Printexc.to_string exn);
          0)

let count_routes_on_branch (t : t) ~(branch_id : string) : int =
  match t.conn with
  | None -> 0
  | Some conn ->
      (try
      let sql =
            "SELECT COUNT(*)::text FROM view_bindings WHERE branch_id = $1"
    in
          let rows =
            exec conn ~params:[ Pgx.Value.of_string branch_id ] sql
          in
          (match rows with
           | [ [ c ] ] ->
               (match Pgx.Value.to_string c with
                | Some s -> (int_of_string_opt s |> Option.value ~default:0)
                | None -> 0)
           | _ -> 0)
    with exn ->
      Printf.eprintf "[store] count_routes_on_branch: %s\n"
            (Printexc.to_string exn);
          0)

(** Parent branch id for [branch_id], if any. *)
let branch_parent_id (t : t) ~(branch_id : string) : string option =
  match t.conn with
  | None -> None
  | Some conn ->
      (try
      let sql =
            "SELECT COALESCE(parent_id::text, '') FROM branches WHERE id =
    $1"
          in
          let rows =
            exec conn ~params:[ Pgx.Value.of_string branch_id ] sql
          in
          match rows with
          | [ [ pid ] ] ->
              (match Pgx.Value.to_string pid with
               | Some "" | None -> None
               | Some s -> (Some s))
          | _ -> None
    with _ ->
      None)

(** Branch LENS CHAIN for [branch_id]: self-first (closest-wins) walk up
    the parent chain to system/main (inclusive), oldest ancestor last.
    Used by the lens-aware tree read prims (one-tree.tree-read-prims) so a
    session on <user>/personal sees the effective override chain for
    tree_nodes under b.<id>.* -- same closest-ancestor-wins rule function_
    defs use. A branch with no reachable parent just resolves to itself. *)
let branch_chain (t : t) ~(branch_id : string) : string list =
  let rec walk acc id =
    match branch_parent_id t ~branch_id:id with
    | Some parent when parent <> id ->
        walk (id :: acc) parent
    | _ -> (id :: acc)
  in
```

```
  (* walk builds oldest-first; reverse to self-first *)
  List.rev (walk [] branch_id)

(** Own rows only: (name, author, updated_at). *)
let list_own_functions (t : t) ~(branch_id : string)
  : (string * string * string) list =
  match t.conn with
  | None -> []
  | Some conn ->
      (try
      let sql =
            "SELECT name, author, updated_at::text FROM function_defs " ^
            "WHERE branch_id = $1 ORDER BY name" in
          let rows =
            exec conn ~params:[ Pgx.Value.of_string branch_id ] sql
          in
          (List.filter_map
               (fun row ->
                 match row with
                 | [ n; a; u ] ->
                     (match Pgx.Value.to_string n,
                             Pgx.Value.to_string a,
                             Pgx.Value.to_string u with
                      | Some name, Some author, Some updated ->
                          Some (name, author, updated)
                      | _ -> None)
                 | _ -> None)
               rows)
    with exn ->
      Printf.eprintf "[store] list_own_functions: %s\n"
            (Printexc.to_string exn);
          [])

let list_own_routes (t : t) ~(branch_id : string)
  : (string * string) list =
  match t.conn with
  | None -> []
  | Some conn ->
      (try
      let sql =
            "SELECT route, handler FROM view_bindings WHERE branch_id = $1
    " ^
            "ORDER BY route" in
          let rows =
            exec conn ~params:[ Pgx.Value.of_string branch_id ] sql
          in
          (List.filter_map
               (fun row ->
                 match row with
                 | [ r; h ] ->
                     (match Pgx.Value.to_string r, Pgx.Value.to_string h
    with
                      | Some route, Some handler -> Some (route, handler)
                      | _ -> None)
                 | _ -> None)
               rows)
    with exn ->
      Printf.eprintf "[store] list_own_routes: %s\n"
            (Printexc.to_string exn);
          [])

(** Child own names left-joined to parent: name, child_updated,
    parent_updated opt. *)
(* ---- branch status (branching.branch-status) ----

   Git-style per-name status of a branch against its parent chain, as ONE
   derived read of the tree-time substrate (tree_versions, migrations
   018-021) keyed on the base_gen merge-base coordinate (migration 024).
   Hash-first per tree-time: equal hash = prune; bodies are never fetched.

   Per name (fn | type | route), with values resolved closest-ancestor-wins
   (a tombstone or missing newest version on one branch means that branch
   is dead for the name and the walk continues up, matching the
```

```
   scaffolding/unbind semantics):
     b = branch value at own head
     p = parent-chain value at parent head
     a = parent-chain value at base_gen (the fork-time base)
     own = a live scaffolding row exists on the branch (the pin)

   HASH DOMAIN (make-phase lesson, 2026-08-28): the STORED
   tree_versions.hash column is NOT comparable across write paths --
   tree-first writes (Tree_store.hash_body over compact OCaml JSON) and
   mirror writes (009 trigger digest over PG jsonb::text, which spaces
   after colons) produce DIFFERENT digests for identical bodies (020
   itself calls the stored hash "a consistency artifact, not content").
   Comparing stored hashes mislabeled identical pins as diverged. Status
   therefore computes digest((body->'body')::text) AT QUERY TIME -- one
   canonical domain (PG jsonb::text) regardless of which path wrote the
   row. The stored column stays for retention/GC bookkeeping and
   same-path gen-diff; a global rehash is tree-time's scope, not ours.

   Tombstones are the reserved 019/020 marker (body->_tombstone). Deletion
   on a branch REVEALS the parent binding (unbind semantics), so there is
   no "locally deleted" status -- a delete is a pin removal, not a divergent
   state.

   APPROXIMATION (documented in the spec): base_gen bounds ONLY the direct
   parent's versions; ancestors above the parent contribute their full
   head. Exact for the dominant topology (empty child of system/main);
   for deeper chains, post-fork writes on grandparents fold into the base.
   Accepted v1; revisit with per-ancestor bounds if it ever misleads. *)

type diff_kind = DiffFn | DiffType | DiffRoute

type st_status = SClean | SAheadNew | SAhead | SBehind | SDiverged | SSynced

let st_status_s = function
  | SClean -> "clean"
  | SAheadNew -> "ahead-new"
  | SAhead -> "ahead"
  | SBehind -> "behind"
  | SDiverged -> "diverged"
  | SSynced -> "synced"

type branch_status_row = {
  st_kind : diff_kind;              (* DiffFn | DiffType | DiffRoute *)
  st_name : string;                 (* fn/type name, or the route path *)
  st_status : st_status;
  st_own : bool;                    (* live pin on the branch *)
  st_branch_hash : string option;   (* b *)
  st_parent_hash : string option;   (* p *)
  st_base_hash : string option;     (* a *)
}

let hash_eq (x : string option) (y : string option) : bool =
  match x, y with
  | None, None -> true
  | Some a, Some b -> String.equal a b
  | _ -> false

(* The spec's decision table, as code. Comparisons are hash equality with
   ABSENT (None) as its own value. *)
let classify_status ~(own : bool) ~(b : string option) ~(p : string option)
    ~(a : string option) : st_status =
  if not own then (if hash_eq p a then SClean else SBehind)
  else if hash_eq b p then SSynced
  else if hash_eq p a then
    (match a with None -> SAheadNew | Some _ -> SAhead)
  else if hash_eq b a then SBehind   (* pin holds the base; parent moved *)
  else SDiverged

(* One round trip per call: chain walk + version resolution for all three
   kinds (fn/type/route) + live-pin existence from the scaffolding tables.
   Read-only (plain exec on the given conn -- callers inside a transaction
   pass the tx conn so the read shares its snapshot). *)
let branch_status_conn (conn : conn) ~(branch_id : string)
```

```
: branch_status_row list =
(try
      let sql =
        "WITH RECURSIVE chain AS ( "^
        "  SELECT id, parent_id, 0 AS depth FROM branches WHERE id = $1 "^
        "  UNION ALL "^
        "  SELECT b.id, b.parent_id, c.depth + 1 "^
        "  FROM branches b JOIN chain c ON b.id = c.parent_id "^
        "), vers AS ( "^
        "  SELECT v.kind, "^
        "         (string_to_array(v.path::text, '.'))[4] AS key, "^
        "         c.depth, v.gen, v.hash, v.body "^
        "  FROM tree_versions v "^
        "  JOIN chain c ON c.id = v.branch_id "^
        "  WHERE v.kind IN ('fn','type','route') "^
        "), heads AS ( "^
        "  SELECT DISTINCT ON (kind, key, depth) kind, key, depth, "^
        "         encode(digest((body->'body')::text,'sha256'),'hex') "^
        "           AS chash, body "^
        "  FROM vers ORDER BY kind, key, depth, gen DESC "^
        "), b_res AS ( "^
        "  SELECT DISTINCT ON (kind, key) kind, key, chash AS b_hash "^
        "  FROM heads WHERE (body->>'_tombstone') IS NULL "^
        "  ORDER BY kind, key, depth ASC "^
        "), p_res AS ( "^
        "  SELECT DISTINCT ON (kind, key) kind, key, chash AS p_hash "^
        "  FROM heads WHERE depth >= 1 AND (body->>'_tombstone') IS NULL
    "^
        "  ORDER BY kind, key, depth ASC "^
        "), vers_base AS ( "^
        "  SELECT v.kind, "^
        "         (string_to_array(v.path::text, '.'))[4] AS key, "^
        "         c.depth, v.gen, v.hash, v.body "^
        "  FROM tree_versions v "^
        "  JOIN chain c ON c.id = v.branch_id "^
        "  WHERE v.kind IN ('fn','type','route') "^
        "    AND c.depth >= 1 "^
        "    AND (c.depth > 1 OR v.gen <= (SELECT COALESCE(base_gen, 0) "^
        "                                  FROM branches WHERE id = $1))
    "^
        "), heads_base AS ( "^
        "  SELECT DISTINCT ON (kind, key, depth) kind, key, depth, "^
        "         encode(digest((body->'body')::text,'sha256'),'hex') "^
        "           AS chash, body "^
        "  FROM vers_base ORDER BY kind, key, depth, gen DESC "^
        "), a_res AS ( "^
        "  SELECT DISTINCT ON (kind, key) kind, key, chash AS a_hash "^
        "  FROM heads_base WHERE depth >= 1 AND (body->>'_tombstone') IS
    NULL "^
        "  ORDER BY kind, key, depth ASC "^
        "), own_fn AS ( "^
        "  SELECT tree_enc_label(name) AS key FROM function_defs "^
        "  WHERE branch_id = $1 "^
        "), own_ty AS ( "^
        "  SELECT tree_enc_label(name) AS key FROM type_defs "^
        "  WHERE branch_id = $1 "^
        "), own_rt AS ( "^
        "  SELECT tree_enc_label(route) AS key FROM view_bindings "^
        "  WHERE branch_id = $1 "^
        "), keys AS ( "^
        "  SELECT kind, key FROM heads "^
        "  UNION SELECT 'fn', key FROM own_fn "^
        "  UNION SELECT 'type', key FROM own_ty "^
        "  UNION SELECT 'route', key FROM own_rt "^
        ") SELECT k.kind, tree_dec_label(k.key) AS name, "^
        "       b.b_hash, p.p_hash, a.a_hash, "^
        "       (o1.key IS NOT NULL) AS own_fn, "^
        "       (o2.key IS NOT NULL) AS own_ty, "^
        "       (o3.key IS NOT NULL) AS own_rt "^
        "FROM keys k "^
        "LEFT JOIN b_res b ON b.kind = k.kind AND b.key = k.key "^
        "LEFT JOIN p_res p ON p.kind = k.kind AND p.key = k.key "^
        "LEFT JOIN a_res a ON a.kind = k.kind AND a.key = k.key "^
```

```
          "LEFT JOIN own_fn o1 ON o1.key = k.key "^
          "LEFT JOIN own_ty o2 ON o2.key = k.key "^
          "LEFT JOIN own_rt o3 ON o3.key = k.key "^
          "ORDER BY k.kind, name"
        in
        let rows = exec conn ~params:[ Pgx.Value.of_string branch_id ] sql
    in
        List.filter_map
          (fun row ->
            match row with
            | [ kind_c; name_c; bh_c; ph_c; ah_c; ofn_c; oty_c; ort_c ] ->
                let kind =
                  match Pgx.Value.to_string kind_c with
                  | Some "type" -> DiffType
                  | Some "route" -> DiffRoute
                  | _ -> DiffFn
                in
                let name = Pgx.Value.to_string name_c in
                let h c = Pgx.Value.to_string c in
                let b = h bh_c and p = h ph_c and a = h ah_c in
                let own =
                  match kind with
                  | DiffFn -> Pgx.Value.to_bool ofn_c = Some true
                  | DiffType -> Pgx.Value.to_bool oty_c = Some true
                  | DiffRoute -> Pgx.Value.to_bool ort_c = Some true
                in
                (match name with
                 | None -> None
                 | Some name ->
                     (* Nothing live anywhere (tombstoned on every head):
                        both sides agree it is gone - nothing to report. *)
                     if (not own) && b = None then None
                     else
                       Some
                         { st_kind = kind;
                           st_name = name;
                           st_status =
                             classify_status ~own ~b ~p ~a;
                           st_own = own;
                           st_branch_hash = b;
                           st_parent_hash = p;
                           st_base_hash = a })
            | _ -> None)
          rows
        with exn ->
        Printf.eprintf "[store] branch_status error: %s\n"
          (Printexc.to_string exn);
        []

let branch_status (t : t) ~(branch_id : string) : branch_status_row list =
  match t.conn with
  | None -> []
  | Some conn -> branch_status_conn conn ~branch_id

let lookup_function_on_branch (t : t) ~(branch_id : string) ~(name : string)
  : (Pricklypear_lisp.Ast.expr * string * string * string option) option =
  match t.conn with
  | None -> None
  | Some conn ->
      (try
      let sql =
            "SELECT body::text, author, updated_at::text, doc FROM
    function_defs " ^
            "WHERE branch_id = $1 AND name = $2" in
          let rows =
            exec conn
              ~params:[ Pgx.Value.of_string branch_id;
                        Pgx.Value.of_string name ]
              sql
          in
          (match rows with
           | [ [ body_c; author_c; upd_c; doc_c ] ] ->
               (match Pgx.Value.to_string body_c,
```

```
                      Pgx.Value.to_string author_c,
                      Pgx.Value.to_string upd_c with
                | Some body_str, Some author, Some updated ->
                    let doc = Pgx.Value.to_string doc_c in
                    (try
                       match Pricklypear_lisp.Ast.expr_of_json
                               (Yojson.Safe.from_string body_str) with
                       | Ok e -> (Some (e, author, updated, doc))
                       | Error _ -> None
                     with _ -> None)
                | _ -> None)
           | _ -> None)
    with exn ->
      Printf.eprintf "[store] lookup_function_on_branch: %s\n"
            (Printexc.to_string exn);
          None)

let delete_function_on_branch (t : t) ~(branch_id : string) ~(name : string)
  : bool =
  match t.conn with
  | None -> false
  | Some conn ->
      (try
            Tree_store.delete_fn conn ~branch_id ~name
    with exn ->
      Printf.eprintf "[store] delete_function_on_branch: %s\n"
            (Printexc.to_string exn);
          false)

(** Copy selected function names from source branch to target (upsert). *)
let promote_functions (t : t) ~(source_id : string) ~(target_id : string)
    ~(names : string list) : int =
  match t.conn with
  | None -> 0
  | Some _ when names = [] -> 0
  | Some conn ->
      (try
      let n =
            List.fold_left
              (fun acc name ->
                let src =
                  lookup_function_on_branch t ~branch_id:source_id ~name
                in
                match src with
                | None -> acc
                | Some _ ->
                    let sql =
                      "INSERT INTO function_defs " ^
                      "  (branch_id, name, body, author, doc, package) " ^
                      "SELECT $1, name, body, author, doc, package " ^
                      "FROM function_defs " ^
                      "WHERE branch_id = $2 AND name = $3 " ^
                      "ON CONFLICT (branch_id, name) DO UPDATE " ^
                      "SET body = EXCLUDED.body, author = EXCLUDED.author,
    " ^
                      "    doc = EXCLUDED.doc, package = EXCLUDED.package,
    " ^
                      "    updated_at = now() RETURNING name" in
                    let rows =
                      exec conn
                        ~params:[ Pgx.Value.of_string target_id;
                                  Pgx.Value.of_string source_id;
                                  Pgx.Value.of_string name ]
                        sql
                    in
                    (if rows = [] then acc else acc + 1))
              0 names
          in
          (* branching.promote ROUTES TRAVEL WITH THEIR HANDLER:
             upsert source view_bindings whose handler is a promoted name;
             preserve visibility. Non-promoted handlers' routes stay put. *)
          let () =
            List.iter
```

```
              (fun name ->
                let sql =
                  "INSERT INTO view_bindings " ^
                  "  (branch_id, route, handler, visibility) " ^
                  "SELECT $1, route, handler, " ^
                  "       COALESCE(visibility, 'private') " ^
                  "FROM view_bindings " ^
                  "WHERE branch_id = $2 AND handler = $3 " ^
                  "ON CONFLICT (branch_id, route) DO UPDATE " ^
                  "SET handler = EXCLUDED.handler, " ^
                  "    visibility = EXCLUDED.visibility"
                in
                let _ =
                  exec conn
                    ~params:
                      [ Pgx.Value.of_string target_id
                      ; Pgx.Value.of_string source_id
                      ; Pgx.Value.of_string name ]
                    sql
                in
                ())
              names
          in
          let () = Tree_store.recompute_head_hash conn ~branch_id:target_id
     in
          n
    with exn ->
      Printf.eprintf "[store] promote_functions: %s\n"
            (Printexc.to_string exn);
          0)

(* Persist a function def. INSERT ... ON CONFLICT (branch_id, name)
   DO UPDATE so redefining a name in the same branch upserts. The body
   is serialized to JSONB. [doc] writes function_defs.doc; None clears
   (NULL) so stale docs do not survive a docstring-free redefine. *)
let persist_function (t : t) (name : string)
    (body : Pricklypear_lisp.Ast.expr)
    ?(doc : string option = None)
    ?(package : string option = None)
    (author : string)
  : unit =
  match t.conn with
  | None -> ()  (* no DB; env binding still happened in-memory *)
  | Some conn ->
      (try
      let body_json =
            Yojson.Safe.to_string
              (Pricklypear_lisp.Ast.expr_to_json body) in
          (* tree is truth; reverse trigger mirrors to table. Package
             provenance is scaffolding-only -- stamped on function_defs
             directly since the tree node envelope has no package field. *)
            let () =
              Tree_store.upsert_fn conn ~branch_id:t.branch_id ~name
                ~body_json ~author ~doc
            in
            (match package with
             | None -> ()
             | Some pkg ->
                 let sql =
                   "UPDATE function_defs SET package = $3 " ^
                   "WHERE branch_id = $1 AND name = $2" in
                 let _ =
                   exec conn
                     ~params:[ Pgx.Value.of_string t.branch_id;
                               Pgx.Value.of_string name;
                               Pgx.Value.of_string pkg ]
                     sql in
                 ())
    with exn ->
      Printf.eprintf "[store] persist_function error: %s\n"
            (Printexc.to_string exn);
          ())

```

```
(** image.boot-seed-batch: preload the set of function names and route
    paths already present on the current branch (system/main during boot
    seed) so persist_function_if_absent / register_route_if_absent skip
    the per-name get_by_path SELECT. Call inside the seed with_branch on
    system/main; pair with [end_seed_names] in a finally. *)
let begin_seed_names (t : t) : unit =
  match t.conn with
  | None -> ()
  | Some conn ->
      let set = Hashtbl.create 1024 in
      (try
         let sql = "SELECT name FROM " ^ fn_rel () ^ " WHERE branch_id =
    $1" in
         let rows = exec conn ~params:[ Pgx.Value.of_string t.branch_id ]
    sql in
         List.iter (fun row ->
             match row with
             | [ name_c ] ->
                 (match Pgx.Value.to_string name_c with
                  | Some n -> Hashtbl.replace set n ()
                  | None -> ())
             | _ -> ()) rows
       with exn ->
         Printf.eprintf "[store] begin_seed_names (fns) error: %s\n"
           (Printexc.to_string exn));
      (try
         let sql = "SELECT route FROM " ^ rt_rel () ^ " WHERE branch_id =
    $1" in
         let rows = exec conn ~params:[ Pgx.Value.of_string t.branch_id ]
    sql in
         List.iter (fun row ->
             match row with
             | [ route_c ] ->
                 (match Pgx.Value.to_string route_c with
                  | Some r -> Hashtbl.replace set r ()
                  | None -> ())
             | _ -> ()) rows
       with exn ->
         Printf.eprintf "[store] begin_seed_names (routes) error: %s\n"
           (Printexc.to_string exn));
      t.seed_names <- Some set

let end_seed_names (t : t) : unit =
  t.seed_names <- None

(** branching.seed-policy: insert only if (branch_id, name) absent.
    Existing main (or any branch) rows are left untouched. Used by boot
    seed so deploy cannot clobber live promote. *)
let persist_function_if_absent (t : t) (name : string)
    (body : Pricklypear_lisp.Ast.expr)
    ?(doc : string option = None)
    ?(package : string option = None)
    (author : string)
  : unit =
  match t.conn with
  | None -> ()
  | Some conn ->
      (* image.boot-seed-batch: during the additive boot seed pass, consult
         the preloaded name set instead of issuing a get_by_path SELECT per
         define. Additive seed never overwrites existing rows, so an
         existing name is a full skip. A genuinely new name upserts and
         records itself so later duplicate defines skip too. *)
      (match t.seed_names with
       | Some set when Hashtbl.mem set name -> ()
       | _ ->
      (try
      let body_json =
            Yojson.Safe.to_string
              (Pricklypear_lisp.Ast.expr_to_json body) in
          (* seed-policy: never overwrite body; fill package provenance
             on the existing scaffolding row when untagged. *)
          let exists =
            Tree_store.get_by_path conn
```

```
               ~path:(Tree_store.fn_path ~branch_id:t.branch_id ~name) in
          let stamp_pkg_if_null () =
            match package with
            | None -> ()
            | Some pkg ->
                let sql =
                  "UPDATE function_defs SET package = $3 " ^
                  "WHERE branch_id = $1 AND name = $2 AND package IS
    NULL" in
                let _ =
                  exec conn
                    ~params:[ Pgx.Value.of_string t.branch_id;
                              Pgx.Value.of_string name;
                              Pgx.Value.of_string pkg ]
                    sql in
                ()
          in
          (match exists with
           | Some _ -> stamp_pkg_if_null ()
           | None ->
               let () =
                 Tree_store.upsert_fn conn ~branch_id:t.branch_id ~name
                   ~body_json ~author ~doc
               in
               stamp_pkg_if_null ());
          (match t.seed_names with
           | Some set -> Hashtbl.replace set name ()
           | None -> ())
     with exn ->
     Printf.eprintf "[store] persist_function_if_absent error: %s\n"
           (Printexc.to_string exn);
         ()))

(** Visible docstring for [name], or None. *)
let lookup_function_doc (t : t) (name : string) : string option =
  let found = lookup_function t name in
  match found with
  | Some (_, _, Some d) when String.trim d <> "" -> Some d
  | _ -> None

let list_branches (t : t) : Pricklypear_common.Branch.t list =
  match t.conn with
  | None -> []
  | Some conn ->
      (try
      let sql =
            "SELECT id::text, COALESCE(parent_id::text, ''), " ^
            "       owner, name, created_at::text, " ^
            "       COALESCE(last_synced::text, ''), " ^
            "       COALESCE(base_gen, 0)::bigint " ^
            "FROM branches ORDER BY owner, name" in
          let rows = exec conn sql in
          (List.filter_map
               (fun row ->
                 match row with
                 | [ id_c; pid_c; owner_c; name_c; created_c; synced_c;
    bg_c ] ->
                     (match Pgx.Value.to_string id_c,
                             Pgx.Value.to_string owner_c,
                             Pgx.Value.to_string name_c,
                             Pgx.Value.to_string created_c with
                      | Some id, Some owner, Some name, Some created ->
                          let parent_id =
                            match Pgx.Value.to_string pid_c with
                            | Some "" | None -> None
                            | Some s -> Some s
                          in
                          let last_synced =
                            match Pgx.Value.to_string synced_c with
                            | Some "" | None -> None
                            | Some s -> Some s
                          in
                          let base_gen =
```

```
                            match Pgx.Value.to_int64 bg_c with
                            | Some g -> g
                            | None -> 0L
                          in
                          let base =
                            Pricklypear_common.Branch.make
                              ~id ~parent_id ~owner ~name
                              ~created_at:created ~base_gen ()
                          in
                          Some
                            (match last_synced with
                             | Some s -> { base with last_synced = Some s }
                             | None -> base)
                      | _ -> None)
                | _ -> None)
              rows)
    with exn ->
      Printf.eprintf "[store] list_branches error: %s\n"
        (Printexc.to_string exn);
        [])

(* Look up a branch id by (owner, name). Used by (fork ...) to find
   the parent's UUID from its Lisp-level reference. Returns None if
   the branch isn't in the DB (or no DB). *)
let lookup_branch_id_by_name (t : t) ~(owner : string) ~(name : string)
  : string option =
  match t.conn with
  | None -> None
  | Some conn ->
      (try
      let sql = "SELECT id::text FROM branches WHERE owner = $1 AND name =
     $2" in
        let rows =
          exec conn
            ~params:[ Pgx.Value.of_string owner;
                      Pgx.Value.of_string name ]
            sql
        in
        (match rows with
         | [ [ id_cell ] ] ->
             (Pgx.Value.to_string id_cell)
         | _ -> None)
    with exn ->
      Printf.eprintf "[store] lookup_branch_id error: %s\n"
        (Printexc.to_string exn);
        None)

(* Switch the store's branch_id: subsequent lookups/persists/registers
   will target the new branch. Per spec [image.repl-experience]:
   "(checkout branch-id)" keeps REPL locals (env) but changes visibility
   (which (define)d functions are effective). We mutate the field.

   Re-loading the env is the caller's responsibility (session.eval_one
   does it after we return successfully). *)
let checkout_branch (t : t) ~(new_branch_id : string) : unit =
  t.branch_id <- new_branch_id;
  ()

(* Get the branch_id the store is currently bound to. *)
let current_branch_id (t : t) : string = t.branch_id

(** Current generation counter for [branch_id] (migration 017). The SQL
    helper [current_branch_generation] walks the branch's ancestry chain
    and returns the max generation across it, so a write to an ancestor
    (e.g. main) is automatically visible to every descendant. The remote
    client stores defs under a [(branch, generation)] key and does a
    single cheap round trip against this to decide whether its local
    def cache is stale. Returns None when the DB is unreachable / the
    helper is missing (pre-017), letting callers treat an unknown
    generation as "reload". *)
let current_generation (t : t) ~(branch_id : string) : int64 option =
  match t.conn with
  | None -> None
```

```
  | Some conn ->
      (try
         let sql =
           "SELECT COALESCE(current_branch_generation($1::uuid), 0)::bigint"
         in
         let rows =
           exec conn
             ~params:[ Pgx.Value.of_string branch_id ]
             sql
         in
         match rows with
         | [ [ cell ] ] ->
             Option.bind (Pgx.Value.to_int64 cell) (fun g -> Some g)
         | _ -> None
       with exn ->
         Printf.eprintf "[store] current_generation error: %s\n"
           (Printexc.to_string exn);
         None)

(** Canonical system/main UUID -- seeded by migrations/001_branches.sql. *)
let system_main_id = "00000000-0000-0000-0000-000000000001"

(** Delete the (branch_id, name) row from function_defs. Used by
    (unbind name): removes the CURRENT branch's override so the name
    resolves via inheritance to the shadowed parent binding. Refuses
    to delete on system/main (the inheritance root). Returns true if
    a row was deleted, false if no branch-local row existed. *)
let delete_function (t : t) (name : string) : bool =
  if t.branch_id = system_main_id then
    raise (Failure "unbind: cannot unbind on system/main")
  else
    match t.conn with
    | None -> false
    | Some conn ->
        (try
           (* tree is truth; reverse mirror deletes the function_defs row *)
           Tree_store.delete_fn conn ~branch_id:t.branch_id ~name
         with exn ->
           Printf.eprintf "[store] delete_function error: %s\n"
             (Printexc.to_string exn);
           false)

(** Temporarily rebind [t.branch_id] for the duration of [f], then restore.
    Used so seed_stdlib can write package defaults to system/main without
    polluting the user's personal branch. *)
let with_branch (t : t) ~(branch_id : string) (f : unit -> 'a) : 'a =
  let old = t.branch_id in
  t.branch_id <- branch_id;
  Fun.protect ~finally:(fun () -> t.branch_id <- old) f

(** Ensure a branch row exists; create an EMPTY child (parent pointer only)
    if missing. Does NOT copy function_defs/types/routes -- inheritance via
    the recursive CTE makes parent defs visible, and personal overrides are
    just rows on the child. This is the right shape for user/personal. *)
let ensure_branch (t : t) ~(parent_id : string) ~(owner : string)
    ~(name : string) : string option =
  let existing = lookup_branch_id_by_name t ~owner ~name in
  match existing with
  | Some id -> (Some id)
  | None ->
      (match t.conn with
       | None -> None
       | Some conn ->
           (try
      let sql =
            "INSERT INTO branches (id, parent_id, owner, name, base_gen) " ^
            "VALUES (gen_random_uuid(), $1, $2, $3, " ^
            "        COALESCE((SELECT generation FROM branch_generations " ^
            "                  WHERE branch_id = $1), 0)) " ^
            "ON CONFLICT (owner, name) DO UPDATE SET name = EXCLUDED.name "
    ^
            "RETURNING id::text" in
               let rows =
```

```
                exec conn
                  ~params:[ Pgx.Value.of_string parent_id;
                            Pgx.Value.of_string owner;
                            Pgx.Value.of_string name ]
                  sql
              in
              (match rows with
               | [ [ id_cell ] ] -> (Pgx.Value.to_string id_cell)
               | _ ->
                   (* Race: another image created it; re-lookup. *)
                   lookup_branch_id_by_name t ~owner ~name)
    with exn ->
      Printf.eprintf "[store] ensure_branch error: %s\n"
                (Printexc.to_string exn);
              lookup_branch_id_by_name t ~owner ~name))

(* Fork a branch: create an EMPTY lens with parent_id as its parent.
   one-tree P5 / branching.personal-workspace: NO copy of code rows.
   Inheritance is the recursive-CTE / path-prefix walk; child pins are
   rows written later on the child. Data stays shared (branching.data).
   Returns the new branch_id. Single SERIALIZABLE insert. *)
(* Persist a route binding: (route "/path" 'handler-fn-name).
   INSERT ... ON CONFLICT (branch_id, route) DO UPDATE -- so re-defining
   a route overrides the existing binding. *)
let normalize_visibility (v : string) : string =
  match String.lowercase_ascii (String.trim v) with
  | "public" -> "public"
  | _ -> "private"

(** Persist a route. [visibility_opt]:
    - Some v ? set visibility (public|private)
    - None ? on INSERT default private; on UPDATE keep existing visibility
      (handler-only re-register does not clobber public). *)
let register_route (t : t) ~(route : string) ~(handler : string)
    ?visibility ()
  : unit =
  match t.conn with
  | None -> ()
  | Some conn ->
      (try
      let vis =
            match visibility with
            | Some v -> normalize_visibility v
            | None -> "private"
          in
          Tree_store.upsert_route conn ~branch_id:t.branch_id ~route
     ~handler
            ~visibility:vis
    with exn ->
      Printf.eprintf "[store] register_route error: %s\n"
            (Printexc.to_string exn);
          ())

(** branching.seed-policy: insert route only if absent on this branch. *)
let register_route_if_absent (t : t) ~(route : string) ~(handler : string)
    ?visibility ()
  : unit =
  match t.conn with
  | None -> ()
  | Some conn ->
      (* image.boot-seed-batch: skip the per-route get_by_path SELECT during
         additive boot seed when the route is already in the preloaded set.
     *)
      (match t.seed_names with
       | Some set when Hashtbl.mem set route -> ()
       | _ ->
      (try
      let vis =
            match visibility with
            | Some v -> normalize_visibility v
            | None -> "private"
          in
          (* seed-policy: insert route only if the tree node is absent *)
```

```
            let exists =
              Tree_store.get_by_path conn
                ~path:(Tree_store.route_path ~branch_id:t.branch_id ~route)
    in
            (match exists with
             | Some _ -> ()
             | None ->
                 Tree_store.upsert_route conn ~branch_id:t.branch_id ~route
                   ~handler ~visibility:vis);
            (match t.seed_names with
             | Some set -> Hashtbl.replace set route ()
             | None -> ())
      with exn ->
      Printf.eprintf "[store] register_route_if_absent error: %s\n"
            (Printexc.to_string exn);
          ()))

(* List all route bindings visible to this branch (recursive-CTE walk,
   same as list_function_names -- annotations from ancestors are visible
   via inheritance, child's definition OVERRIDES parent's). Returns
   (route, handler, visibility) ordered with closest ancestor first. *)
let list_routes (t : t) : (string * string * string) list =
  match t.conn with
  | None -> []
  | Some conn ->
      (try
      let sql =
            "WITH RECURSIVE chain AS ( " ^
            "  SELECT id, parent_id, 0 AS depth FROM branches WHERE id = $1
    " ^
            "  UNION ALL " ^
            "  SELECT b.id, b.parent_id, c.depth + 1 " ^
            "  FROM branches b JOIN chain c ON b.id = c.parent_id " ^
            ") SELECT vb.route, vb.handler, " ^
            "COALESCE(vb.visibility, 'private') " ^
            "FROM " ^ rt_rel () ^ " vb " ^
            "JOIN chain ON vb.branch_id = chain.id " ^
            "ORDER BY chain.depth ASC" in
          let rows =
            exec conn
              ~params:[ Pgx.Value.of_string t.branch_id ]
              sql
          in
          (List.filter_map
               (fun row ->
                 match row with
                 | [ route_c; handler_c; vis_c ] ->
                     (match Pgx.Value.to_string route_c,
                            Pgx.Value.to_string handler_c with
                      | Some r, Some h ->
                          let vis =
                            match Pgx.Value.to_string vis_c with
                            | Some v -> normalize_visibility v
                            | None -> "private"
                          in
                          Some (r, h, vis)
                      | _ -> None)
                 | [ route_c; handler_c ] ->
                     (* Pre-migration fallback *)
                     (match Pgx.Value.to_string route_c,
                            Pgx.Value.to_string handler_c with
                      | Some r, Some h -> Some (r, h, "private")
                      | _ -> None)
                 | _ -> None)
               rows)
    with exn ->
      Printf.eprintf "[store] list_routes error: %s\n"
            (Printexc.to_string exn);
          [])

(* ---- type_defs ----

   Per spec [storage.types]: user declares a type with (deftype 'name
```

```
   field1 field2 ...). The declaration is INSERTed into type_defs with
   branch_id = current branch and fields = JSON array of strings (the
   field names, in declared order).

   The data DOESN'T live in a per-type table: packed in migration 003's
   note that generating one PG table per Lisp type would require
   runtime DDL breaking additive-migrations-only. Instead, all data
   rows are stored in the shared data_rows(type_name, fields jsonb)
   table, and type_defs is pure declaration metadata. (Type
   declarations MAY eventually guide field-level validation; for now
   they're documentation.)

   The recursive-CTE override walk applies to type_defs the same as
   function_defs / view_bindings: ancestors' types are visible, child's
   definition wins on name conflict. *)

let persist_type (t : t) ~(name : string) ~(fields : string list)
    ~(author : string) : unit =
  match t.conn with
  | None -> ()
  | Some conn ->
      (try
      let fields_json =
            Yojson.Safe.to_string
              (`List (List.map (fun s -> `String s) fields)) in
          Tree_store.upsert_type conn ~branch_id:t.branch_id ~name
            ~fields_json ~author
    with exn ->
      Printf.eprintf "[store] persist_type error: %s\n"
            (Printexc.to_string exn);
          ())

(* List all type declarations visible to this branch via the recursive
   CTE override walk. Returns (name, fields) pairs with closest
   ancestor first; caller-side Hashtbl first-occurrence dedup wins. *)
let list_types (t : t) : (string * string list) list =
  match t.conn with
  | None -> []
  | Some conn ->
      (try
      let sql =
            "WITH RECURSIVE chain AS ( " ^
            "  SELECT id, parent_id, 0 AS depth FROM branches WHERE id = $1
    " ^
            "  UNION ALL " ^
            "  SELECT b.id, b.parent_id, c.depth + 1 " ^
            "  FROM branches b JOIN chain c ON b.id = c.parent_id " ^
            ") SELECT td.name, td.fields::text FROM " ^ ty_rel () ^ " td " ^
            "JOIN chain ON td.branch_id = chain.id " ^
            "ORDER BY chain.depth ASC" in
          let rows =
            exec conn
              ~params:[ Pgx.Value.of_string t.branch_id ]
              sql
          in
          (List.filter_map
               (fun row ->
                 match row with
                 | [ name_cell; fields_cell ] ->
                     (match Pgx.Value.to_string name_cell,
                             Pgx.Value.to_string fields_cell with
                      | Some name, Some fs ->
                          (try
                             let fs' =
                               match Yojson.Safe.from_string fs with
                               | `List lst ->
                                   List.filter_map
                                     (fun j ->
                                       match j with
                                       | `String s -> Some s
                                       | _ -> None)
                                     lst
                               | _ -> []
```

```
                            in Some (name, fs')
                          with _ -> None)
                    | _ -> None)
                | _ -> None)
              rows)
    with exn ->
      Printf.eprintf "[store] list_types error: %s\n"
        (Printexc.to_string exn);
      []

(** Field names for a type visible on the current branch (CTE walk).
    None if the type is not declared. *)
let lookup_type_fields (t : t) ~(name : string) : string list option =
  let types = list_types t in
  match List.find_opt (fun (n, _) -> n = name) types with
  | Some (_, fields) -> Some fields
  | None -> None

let fork_branch (t : t) ~(parent_id : string) ~(owner : string)
    ~(child_name : string) : string option =
  (* one-tree P5: EMPTY LENS fork.
     Create a branch row with parent_id only -- NO copy of function_defs /
     type_defs / view_bindings. Inheritance is the recursive-CTE / path-
     prefix walk (child shadows parent). Matches ensure_branch and
     branching.personal-workspace ("empty child of main"). Copy-on-fork
     froze inheritance and contradicted the lens model.
     tree_branches lens registry is maintained by the branches?tree
     deferred trigger (009). head_hash is recomputed (empty ? null/empty).
     Single SERIALIZABLE tx; rollback-safe. *)
  (try
     with_transaction t (fun conn ->
       match conn with
       | None -> None
       | Some conn ->
           let insert_branch_sql =
             "INSERT INTO branches (id, parent_id, owner, name, base_gen)
    " ^
             "VALUES (gen_random_uuid(), $1, $2, $3, " ^
             "        COALESCE((SELECT generation FROM branch_generations
    " ^
             "                  WHERE branch_id = $1), 0)) " ^
             "RETURNING id::text" in
           let rows =
             exec conn
               ~params:[ Pgx.Value.of_string parent_id;
                         Pgx.Value.of_string owner;
                         Pgx.Value.of_string child_name ]
               insert_branch_sql
           in
           (match rows with
            | [ [ id_cell ] ] ->
                (match Pgx.Value.to_string id_cell with
                 | Some new_branch_id ->
                     (* Refresh head_hash on the new empty lens (0 own
    nodes). *)
                     let () =
                       Tree_store.recompute_head_hash conn
                         ~branch_id:new_branch_id
                     in
                     (Some new_branch_id)
                 | None -> None)
            | _ -> None))
    with exn ->
      Printf.eprintf "[store] fork_branch error: %s\n"
        (Printexc.to_string exn);
      None)

(* Merge a source branch's function_defs into a target branch.

   Per spec [branching.merge]:
     (merge 'alice/experiment 'alice/main)
   -> copy all function_defs from source to target (upsert).
   Policy: admin-only merges to main; per-user can merge into their
```

```
   own branches. Policy is enforced at the Lisp intercept layer
   (session.ml); Store.merge_branch just does the SQL.

   branching.branch-status S3: status-driven merge. Into the source's
   PARENT: upsert ONLY names the source moved since base_gen (ahead-new /
   ahead / diverged); behind names would clobber the parent's newer work
   and are skipped; synced/clean names are already identical. Conflict
   report = the diverged set (both sides moved since the fork base) --
   genuine conflict detection; resolution stays LWW (report-only).
   Any other target (no shared base): head-vs-head fallback -- upsert
   source fns whose body differs from the target's own row, no report.
   TODO: schema-merge for type_defs -- only main accepts schema changes
   via special migration-governed merge.

   Wrapped in a SERIALIZABLE transaction via [with_transaction] since
   it16 (previously best-effort atomic -- carried-forward since it9). *)
type merge_result = {
  count : int;
  conflicts : string list;
}

(* ---- branch diff (branching.diff) ---- *)

type diff_status = DiffAdded | DiffRemoved | DiffModified

(** One divergent row across two branches' effective views. [from_body]/
    [to_body] are [Reader.pp_expr] strings (for types: the fields JSON
    text; for routes: the handler). [from_expr]/[to_expr] carry the parsed
    AST for Modified fn rows so the /diff renderer can run Ast_diff.diff
    without re-parsing. [from_updated]/[to_updated] are ISO timestamps
    (None when the side is absent).

    Scope = merge scope: function_defs + type_defs + view_bindings;
    NEVER data_rows (shared, branch-free). *)
type branch_diff_row = {
  diff_name : string;
  diff_kind : diff_kind;
  diff_status : diff_status;
  from_body : string option;
  to_body : string option;
  from_updated : string option;
  to_updated : string option;
  from_expr : Pricklypear_lisp.Ast.expr option;
  to_expr : Pricklypear_lisp.Ast.expr option;
}

(** Parse a serialized-AST jsonb string into [Ast.expr]. None on any
    parse failure (Malformed rows are skipped for AST purposes but still
    reported as modified with their raw [Reader.pp_expr] bodies). *)
let parse_expr_opt (body_str : string) : Pricklypear_lisp.Ast.expr option =
  match Yojson.Safe.from_string body_str with
  | j -> (
      match Pricklypear_lisp.Ast.expr_of_json j with
      | Ok e -> Some e
      | Error _ -> None)
  | exception Yojson.Json_error _ -> None

(** Format merge result for the (merge ...) prim / UI. *)
let format_merge_result (r : merge_result) : string =
  match r.conflicts with
  | [] -> Printf.sprintf "merged %d rows" r.count
  | cs ->
      Printf.sprintf "merged %d rows (%d conflicts: %s)"
        r.count (List.length cs) (String.concat ", " cs)

let merge_branch (t : t) ~(source_id : string) ~(target_id : string)
  : merge_result option =
  (* Is this a merge into the source's direct parent? Read BEFORE the tx
     (the value is stable); the base advance runs AFTER the tx commits,
     because the deferred gen-bump triggers (017/019) fire at COMMIT --
     reading branch_generations inside the tx sees the PRE-merge counter. *)
  let src_parent =
    match t.conn with
```

```
    | None -> None
    | Some conn -> (
        match
          exec conn
            ~params:[ Pgx.Value.of_string source_id ]
            "SELECT parent_id::text FROM branches WHERE id = $1"
        with
        | [ [ c ] ] -> Pgx.Value.to_string c
        | _ -> None)
  in
  let is_parent_merge = src_parent = Some target_id in
  let merged =
    try
      with_transaction t (fun conn ->
        match conn with
        | None -> None  (* no DB; can't merge *)
        | Some conn ->
            (* branching.branch-status S3: status-driven merge.
               [is_parent_merge] was resolved before the tx (see above). *)
            let names, conflicts =
              if is_parent_merge then begin
                let rows = branch_status_conn conn ~branch_id:source_id in
                let fns =
                  List.filter (fun r -> r.st_kind = DiffFn) rows in
                let ups =
                  List.filter_map
                    (fun r ->
                      match r.st_status with
                      | SAheadNew | SAhead | SDiverged -> Some r.st_name
                      | _ -> None)
                    fns
                in
                let conf =
                  List.filter_map
                    (fun r ->
                      if r.st_status = SDiverged then Some r.st_name
                      else None)
                    fns
                in
                (ups, conf)
              end
              else begin
                let sql =
                  "SELECT s.name FROM function_defs s " ^
                  "WHERE s.branch_id = $1 AND NOT EXISTS ( " ^
                  "  SELECT 1 FROM function_defs x " ^
                  "  WHERE x.branch_id = $2 AND x.name = s.name " ^
                  "  AND x.body IS NOT DISTINCT FROM s.body) " ^
                  "ORDER BY s.name" in
                let rows =
                  exec conn
                    ~params:[ Pgx.Value.of_string source_id;
                              Pgx.Value.of_string target_id ]
                    sql
                in
                (List.filter_map
                   (function [ c ] -> Pgx.Value.to_string c | _ -> None)
                   rows,
                 [] (* no shared base: no conflict report *))
              end
            in
            (* Upsert only the selected names (per-name loop keeps binding
               simple; merge sets are branch-scale, tens of rows). *)
            let count =
              List.fold_left
                (fun acc name ->
                  let sql =
                    "INSERT INTO function_defs " ^
                    "  (branch_id, name, body, author, doc, package,
    updated_at) " ^
                    "SELECT $1, name, body, author, doc, package,
    updated_at " ^
                    "FROM function_defs " ^
```

```
                    "WHERE branch_id = $2 AND name = $3 " ^
                    "ON CONFLICT (branch_id, name) DO UPDATE " ^
                    "  SET body = EXCLUDED.body, author = EXCLUDED.author, " ^
                    "      doc = EXCLUDED.doc, package = EXCLUDED.package, " ^
                    "      updated_at = EXCLUDED.updated_at " ^
                    "RETURNING name" in
                  let rows =
                    exec conn
                      ~params:[ Pgx.Value.of_string target_id;
                                Pgx.Value.of_string source_id;
                                Pgx.Value.of_string name ]
                      sql
                  in
                  if rows = [] then acc else acc + 1)
                0 names
            in
            (* Advance the source's merge base AFTER commit (see above):
               post-commit, branch_generations carries the merge's own
               bump, so base_gen lands on the parent's true head -- git's
               moving merge-base. Only meaningful for parent-merges;
               promote deliberately does NOT advance. *)
            let () = Tree_store.recompute_head_hash conn ~branch_id:target_id in
            (Some { count; conflicts }))
      with exn ->
        Printf.eprintf "[store] merge_branch error: %s\n"
          (Printexc.to_string exn);
        None
    in
    let () =
      if is_parent_merge then
        match merged with
        | Some _ -> (
            match t.conn with
            | None -> ()
            | Some conn ->
                (try
                   let _ =
                     exec conn
                       ~params:[ Pgx.Value.of_string source_id;
                                 Pgx.Value.of_string target_id ]
                       ("UPDATE branches SET base_gen = COALESCE( " ^
                        "  (SELECT generation FROM branch_generations " ^
                        "    WHERE branch_id = $2), 0) WHERE id = $1")
                   in
                   ()
                 with exn ->
                   Printf.eprintf "[store] merge base advance: %s\n"
                     (Printexc.to_string exn)))
        | None -> ()
    in
    merged

(** Branch diff (branching.diff). Compares the EFFECTIVE views of two
    branches (closest-ancestor-wins walk per branch, deduped) across
    function_defs + type_defs + view_bindings, and classifies each
    divergent name as added / removed / modified.

    Read-only (NO writes) -- matches the read-path style of
    [list_function_names_meta] (plain [exec], no transaction). Scope
    mirrors [merge_branch]: never data_rows. *)
let diff_branches (t : t) ~(from_id : string) ~(to_id : string)
    : branch_diff_row list =
    match t.conn with
    | None -> []
    | Some conn ->
        (try
            (* chain_from / chain_to: parent walks for each branch.  Each
               side's *effective* rows are deduped (row_number depth-ASC,
               keep first = closest wins), then FULL OUTER JOINed on the
               key so added/removed/modified fall out naturally.  The body
```

```
           payload differs per table (fn=body, type=fields, route=
           handler) so the query is parameterized per kind. *)
      let read_kind (kind : diff_kind) : branch_diff_row list =
        let rel, key_sql, body_expr, is_route =
          match kind with
          | DiffFn ->
              (fn_rel (), "name", "body", false)
          | DiffType ->
              (ty_rel (), "name", "fields", false)
          | DiffRoute ->
              (rt_rel (), "route", "handler", true)
        in
        let sql =
          String.concat ""
            [ "WITH RECURSIVE chain_from AS ( "
            ; "  SELECT id, parent_id, 0 AS depth FROM branches WHERE id = $1 "
            ; "  UNION ALL "
            ; "  SELECT b.id, b.parent_id, c.depth + 1 "
            ; "  FROM branches b JOIN chain_from c ON b.id = c.parent_id "
            ; "), chain_to AS ( "
            ; "  SELECT id, parent_id, 0 AS depth FROM branches WHERE id = $2 "
            ; "  UNION ALL "
            ; "  SELECT b.id, b.parent_id, c.depth + 1 "
            ; "  FROM branches b JOIN chain_to c ON b.id = c.parent_id "
            ; "), fromd AS ( "
            ; "  SELECT " ^ key_sql ^ " AS key, " ^ body_expr ^ " AS body, "
              ^ "updated_at::text AS up, "
            ; "         row_number() OVER (PARTITION BY " ^ key_sql
              ^ " ORDER BY chain_from.depth ASC) AS rn "
            ; "  FROM " ^ rel ^ " fd JOIN chain_from ON fd.branch_id = chain_from.id "
            ; "), tod AS ( "
            ; "  SELECT " ^ key_sql ^ " AS key, " ^ body_expr ^ " AS body, "
              ^ "updated_at::text AS up, "
            ; "         row_number() OVER (PARTITION BY " ^ key_sql
              ^ " ORDER BY chain_to.depth ASC) AS rn "
            ; "  FROM " ^ rel ^ " fd JOIN chain_to ON fd.branch_id = chain_to.id "
            ; ") SELECT f.key, f.body, f.up::text, t.key, t.body, t.up::text "
            ; "FROM (SELECT * FROM fromd WHERE rn = 1) f "
            ; "FULL OUTER JOIN (SELECT * FROM tod WHERE rn = 1) t ON t.key = f.key "
            ; "WHERE f.key IS NULL OR t.key IS NULL "
            ; "      OR f.body IS DISTINCT FROM t.body "
            ; "ORDER BY COALESCE(f.key, t.key)" ]
        in
        let rows =
          exec conn
            ~params:
              [ Pgx.Value.of_string from_id
              ; Pgx.Value.of_string to_id ]
            sql
        in
        List.filter_map
          (fun row ->
            match row with
            | [ fk_c; fb_c; fu_c; tk_c; tb_c; tu_c ] ->
                let from_key = Pgx.Value.to_string fk_c in
                let to_key = Pgx.Value.to_string tk_c in
                let name =
                  match from_key, to_key with
                  | Some k, _ | _, Some k -> k
                  | None, None -> ""
                in
                let has_from = from_key <> None in
                let has_to = to_key <> None in
                if name = "" then None
```

```
                    else
                      let status =
                        if has_from && not has_to then DiffRemoved
                        else if has_to && not has_from then DiffAdded
                        else DiffModified
                      in
                      let from_body = if has_from then Pgx.Value.to_string fb_c else None in
                      let to_body = if has_to then Pgx.Value.to_string tb_c else None in
                      let from_updated = if has_from then Pgx.Value.to_string fu_c else None in
                      let to_updated = if has_to then Pgx.Value.to_string tu_c else None in
                      (* Marshal body strings into presentable text + parsed
                         AST (functions/types only; routes have no body AST). *)
                      let pp (b : string option) : string option =
                        Option.map (fun bs ->
                            if is_route then bs
                            else if kind = DiffType then bs
                            else
                              match parse_expr_opt bs with
                              | Some e -> Pricklypear_lisp.Reader.pp_expr e
                              | None -> bs) b
                      in
                      let ex (b : string option) : Pricklypear_lisp.Ast.expr option =
                        if is_route || kind <> DiffFn then None
                        else Option.bind b parse_expr_opt
                      in
                      Some {
                        diff_name = name;
                        diff_kind = kind;
                        diff_status = status;
                        from_body = pp from_body;
                        to_body = pp to_body;
                        from_updated;
                        to_updated;
                        from_expr = ex from_body;
                        to_expr = ex to_body;
                      }
                  | _ -> None)
                rows
            in
            List.concat
              [ read_kind DiffFn
              ; read_kind DiffType
              ; read_kind DiffRoute ]
        with exn ->
          Printf.eprintf "[store] diff_branches error: %s\n"
            (Printexc.to_string exn);
          []

(* ---- package-surface: package_registry (branch-lensed) ---- *)

type package_row = {
  name : string;
  enabled : bool;
  exports : string list;
  author : string;
}

let json_string_list (j : Yojson.Safe.t) : string list =
  match j with
  | `List xs ->
      List.filter_map
        (function `String s -> Some s | _ -> None)
        xs
  | _ -> []

let string_list_to_json (xs : string list) : string =
  Yojson.Safe.to_string
```

```
    (‘List (List.map (fun s -> ‘String s) xs))

(** Effective package registry rows for the current branch (closest wins). *)
let list_packages (t : t) : package_row list =
  match t.conn with
  | None -> []
  | Some conn ->
      (try
      let sql =
            "WITH RECURSIVE chain AS ( " ^
            "  SELECT id, parent_id, 0 AS depth FROM branches WHERE id = $1 " ^
            "  UNION ALL " ^
            "  SELECT b.id, b.parent_id, c.depth + 1 " ^
            "  FROM branches b JOIN chain c ON b.id = c.parent_id " ^
            ") SELECT pr.name, pr.enabled::text, pr.exports::text, pr.author, " ^
            "       chain.depth " ^
            "FROM " ^ pkg_rel () ^ " pr " ^
            "JOIN chain ON pr.branch_id = chain.id " ^
            "ORDER BY chain.depth ASC, pr.name ASC" in
          let rows =
            exec conn ~params:[ Pgx.Value.of_string t.branch_id ] sql
          in
          let seen = Hashtbl.create 32 in
          let acc =
            List.filter_map
              (fun row ->
                match row with
                | [ name_c; en_c; exp_c; author_c; _depth_c ] ->
                    (match Pgx.Value.to_string name_c,
                           Pgx.Value.to_string en_c,
                           Pgx.Value.to_string exp_c,
                           Pgx.Value.to_string author_c with
                     | Some name, Some en_s, Some exp_s, Some author ->
                         if Hashtbl.mem seen name then None
                         else begin
                           Hashtbl.replace seen name ();
                           let enabled =
                             en_s = "t" || en_s = "true" || en_s = "1"
                           in
                           let exports =
                             try json_string_list (Yojson.Safe.from_string exp_s)
                             with _ -> []
                           in
                           Some { name; enabled; exports; author }
                         end
                     | _ -> None)
                | _ -> None)
              rows
          in
          acc
      with exn ->
        Printf.eprintf "[store] list_packages error: %s\n"
          (Printexc.to_string exn);
          [])

(** Set of package names disabled on the current branch (effective). *)
let disabled_package_names (t : t) : string list =
  let rows = list_packages t in
  (List.filter_map
       (fun (r : package_row) -> if r.enabled then None else Some r.name)
       rows)

let lookup_package (t : t) (name : string) : package_row option =
  let rows = list_packages t in
  (List.find_opt (fun (r : package_row) -> r.name = name) rows)

(** Upsert registry row. [enabled_preserve]: if true and a row already exists
    on this branch, keep its enabled bit (header reload must not re-enable).
```

```
    Exports always replaced when provided. *)
let upsert_package_registry (t : t) ~(name : string)
    ~(exports : string list) ~(author : string)
    ?(enabled : bool = true) ?(enabled_preserve : bool = true)
    () : unit =
  match t.conn with
  | None -> ()
  | Some conn ->
      (try
      let exp_json = string_list_to_json exports in
          let sql =
            if enabled_preserve then
              "INSERT INTO package_registry " ^
              "  (branch_id, name, enabled, exports, author) " ^
              "VALUES ($1, $2, $3, $4::jsonb, $5) " ^
              "ON CONFLICT (branch_id, name) DO UPDATE " ^
              "SET exports = EXCLUDED.exports, author = EXCLUDED.author, " ^
              "    updated_at = now()"
            else
              "INSERT INTO package_registry " ^
              "  (branch_id, name, enabled, exports, author) " ^
              "VALUES ($1, $2, $3, $4::jsonb, $5) " ^
              "ON CONFLICT (branch_id, name) DO UPDATE " ^
              "SET enabled = EXCLUDED.enabled, exports = EXCLUDED.exports,
    " ^
              "    author = EXCLUDED.author, updated_at = now()"
          in
          let _ =
            exec conn
              ~params:[ Pgx.Value.of_string t.branch_id;
                        Pgx.Value.of_string name;
                        Pgx.Value.of_bool enabled;
                        Pgx.Value.of_string exp_json;
                        Pgx.Value.of_string author ]
              sql
          in
          ()
    with exn ->
      Printf.eprintf "[store] upsert_package_registry error: %s\n"
        (Printexc.to_string exn);
          ())

(** Write enabled bit on the *current* branch (creates a shadowing row if
    the package only exists on an ancestor). Copies exports from effective
    row when inserting a new shadow. *)
let set_package_enabled (t : t) ~(name : string) ~(enabled : bool)
    ~(author : string) : unit =
  match t.conn with
  | None -> ()
  | Some conn ->
      (try
      let existing = lookup_package t name in
          let exports =
            match existing with
            | Some r -> r.exports
            | None -> []
          in
          let exp_json = string_list_to_json exports in
          let sql =
            "INSERT INTO package_registry " ^
            "  (branch_id, name, enabled, exports, author) " ^
            "VALUES ($1, $2, $3, $4::jsonb, $5) " ^
            "ON CONFLICT (branch_id, name) DO UPDATE " ^
            "SET enabled = EXCLUDED.enabled, author = EXCLUDED.author, " ^
            "    updated_at = now()"
          in
          let _ =
            exec conn
              ~params:[ Pgx.Value.of_string t.branch_id;
                        Pgx.Value.of_string name;
                        Pgx.Value.of_bool enabled;
                        Pgx.Value.of_string exp_json;
                        Pgx.Value.of_string author ]
```

```
              sql
          in
          ()
    with exn ->
      Printf.eprintf "[store] set_package_enabled error: %s\n"
        (Printexc.to_string exn);
          ())

(** Stamp untagged function_defs on the current branch with [package].
    Matches export list names and [package]/ prefix. Used when a
    (package ?) header is seen so provenance lands even if later defines
    in the same load abort (e.g. std kernel special-form fallbacks). *)
let stamp_package_bindings (t : t) ~(package : string)
    ~(exports : string list) : unit =
  match t.conn with
  | None -> ()
  | Some conn ->
      (try
      let exp_json = string_list_to_json exports in
          let sql =
            "UPDATE function_defs SET package = $2 " ^
            "WHERE branch_id = $1 AND package IS NULL AND ( " ^
            "  name = $2 " ^
            "  OR name LIKE $2 || '/%' " ^
            "  OR name = ANY (SELECT jsonb_array_elements_text($3::jsonb))
    " ^
            ")" in
          let _ =
            exec conn
              ~params:[ Pgx.Value.of_string t.branch_id;
                        Pgx.Value.of_string package;
                        Pgx.Value.of_string exp_json ]
              sql
          in
          ()
    with exn ->
      Printf.eprintf "[store] stamp_package_bindings error: %s\n"
        (Printexc.to_string exn);
          ())

(** Map name -> package provenance for effective function_defs (closest
    wins). *)
let function_package_map (t : t) : (string, string) Hashtbl.t =
  match t.conn with
  | None -> (Hashtbl.create 1)
  | Some conn ->
      (try
      let sql =
            "WITH RECURSIVE chain AS ( " ^
            "  SELECT id, parent_id, 0 AS depth FROM branches WHERE id = $1
    " ^
            "  UNION ALL " ^
            "  SELECT b.id, b.parent_id, c.depth + 1 " ^
            "  FROM branches b JOIN chain c ON b.id = c.parent_id " ^
            ") SELECT fd.name, fd.package, chain.depth " ^
            "FROM " ^ fn_rel () ^ " fd " ^
            "JOIN chain ON fd.branch_id = chain.id " ^
            "WHERE fd.package IS NOT NULL " ^
            "ORDER BY chain.depth ASC, fd.name ASC" in
          let rows =
            exec conn ~params:[ Pgx.Value.of_string t.branch_id ] sql
          in
          let tbl = Hashtbl.create 128 in
          List.iter
            (fun row ->
              match row with
              | [ name_c; pkg_c; _depth_c ] ->
                  (match Pgx.Value.to_string name_c,
                         Pgx.Value.to_string pkg_c with
                   | Some name, Some pkg when not (Hashtbl.mem tbl name) ->
                       Hashtbl.replace tbl name pkg
                   | _ -> ())
              | _ -> ())
```

```
            rows;
          tbl
    with exn ->
      Printf.eprintf "[store] function_package_map error: %s\n"
            (Printexc.to_string exn);
          (Hashtbl.create 1))

(* ---- code-history (branching.code-history) ---- *)

type code_history_entry = {
  rel : string;
  op : string;
  author : string option;
  doc : string option;
  package : string option;
  changed_at : string;
  body : string option; (* jsonb text *)
}

let code_history_entry_to_yojson (e : code_history_entry) : Yojson.Safe.t =
  let opt_str = function None -> `Null | Some s -> `String s in
  let body_json =
    match e.body with
    | None -> `Null
    | Some s ->
        (try Yojson.Safe.from_string s with _ -> `String s)
  in
  `Assoc
    [ ("rel", `String e.rel)
    ; ("op", `String e.op)
    ; ("author", opt_str e.author)
    ; ("doc", opt_str e.doc)
    ; ("package", opt_str e.package)
    ; ("changed_at", `String e.changed_at)
    ; ("body", body_json)
    ]

(** 012-table before-image history for [name] on [branch_id]. Kept as the
    pre-shadow fallback source (tree-time.projections Phase 3): the
     versioned
    tree only captures writes after migration 019, so pre-019 historical
    depth still lives here until the coexistence window closes. *)
let list_code_history_012 (t : t) ~(branch_id : string) ~(name : string)
  : code_history_entry list =
  match t.conn with
  | None -> []
  | Some conn ->
      (try
         let sql =
           "SELECT rel, op, author, doc, package, changed_at::text, " ^
           "       body::text " ^
           "FROM code_history " ^
           "WHERE branch_id = $1 AND name = $2 " ^
           "ORDER BY changed_at DESC, id DESC"
         in
         let rows =
           exec conn
             ~params:
               [ Pgx.Value.of_string branch_id; Pgx.Value.of_string name ]
             sql
         in
         List.filter_map
           (fun row ->
             match row with
             | [ rel_c; op_c; author_c; doc_c; pkg_c; at_c; body_c ] ->
                 (match
                    ( Pgx.Value.to_string rel_c
                    , Pgx.Value.to_string op_c
                    , Pgx.Value.to_string at_c )
                  with
                  | Some rel, Some op, Some changed_at ->
                      Some
                        { rel
```

```
                        ; op
                        ; author = Pgx.Value.to_string author_c
                        ; doc = Pgx.Value.to_string doc_c
                        ; package = Pgx.Value.to_string pkg_c
                        ; changed_at
                        ; body = Pgx.Value.to_string body_c
                        }
                  | _ -> None)
             | _ -> None)
          rows
      with exn ->
        Printf.eprintf "[store] list_code_history_012: %s\n"
          (Printexc.to_string exn);
        [])

(* ---- code-history over tree_versions (tree-time.projections, Phase 3) ----
   Re-implements branching.code-history / code-diff over the versioned tree
   (tree_versions) instead of the 012 before-image table, KEEPING SIGNATURES
   UNCHANGED.

   SEMANTIC NOTE (recorded in tree-time.coexistence-plan Phase 3): 012 stored
   the OLD before-image per UPDATE/DELETE (never on the first INSERT);
   tree_versions stores the post-write head payload at each gen. A tree-backed
   history read therefore reports ONLY the rows STRICTLY BELOW HEAD -- the
   previous states, newest-first -- which the projections doc phrases as "read
   the version row at gen-1". The current head row (the live state) is never
   history, so an insert-only fn has no history until it is rewritten or
   deleted, exactly like the 012 table. Delete-then-recreate leaves a
   middle TOMBSTONE row (op=DELETE) below head, which is preserved.

   FALLBACK: when the code path has NO version rows yet (a pre-019 write --
   the 019 shadow only captures writes after installation), fall back to the
   012 table so no pre-shadow historical depth is lost during the coexistence
   window. Post-shadow history is complete in tree_versions. *)

(* The inner definition payload of a code version row. The tree payload for a
   fn/type/route node carries the code under a 'body' key (fn: {body, params};
   type: the fields list; route: the handler JSON) -- the same shape 012 stored
   in function_defs.body / type_defs.fields / view_bindings handler. *)
let code_body_of_version (v : Tree_store.version) : string option =
  match v.Tree_store.v_body with
  | `Assoc kv -> (
      match List.assoc_opt "body" kv with
      | Some b -> Some (Yojson.Safe.to_string b)
      | None -> None )
  | _ -> None

let code_doc_of_version (v : Tree_store.version) : string option =
  match v.Tree_store.v_body with
  | `Assoc kv -> (
      match List.assoc_opt "doc" kv with
      | Some (`String s) -> Some s
      | _ -> None )
  | _ -> None

(* Resolve a logical code [name] on [branch_id] to its tree path + rel.
   Tries fn, then type, then route (same priority live_code_body_text uses).
   A candidate path "exists" if it has a version chain (covers the live AND
   the tombstoned/deleted case) OR a live tree node (the pre-019 case that
   falls back to the 012 table). *)
let code_path_for_name (t : t) ~(branch_id : string) ~(name : string)
  : (string * string) option =
  match t.conn with
  | None -> None
  | Some conn ->
```

```
      let candidates =
        [ (Tree_store.fn_path ~branch_id ~name, "function_defs")
        ; (Tree_store.type_path ~branch_id ~name, "type_defs")
        ; (Tree_store.route_path ~branch_id ~route:name, "view_bindings")
        ]
      in
      List.find_map
        (fun (p, rel) ->
          let has_ver =
            Tree_store.versioned_history conn ~path:p () <> []
          in
          let live = Tree_store.get_by_path conn ~path:p <> None in
          if has_ver || live then Some (p, rel) else None)
        candidates

(* Tree-backed code history for [name] on [branch_id], newest-first.
   Only rows STRICTLY BELOW HEAD are history -- the 012 table captured
   before-images on UPDATE/DELETE only (never on the first INSERT), and the
   projections doc defines a history read as "the version row at gen-1".
   So the current head row (the live state) is EXCLUDED; the newest history
   entry is the state just before the latest change. Empty when the name has
   no code path OR only a first-insert head row (no superseded state yet) --
   caller falls back to the 012 table so pre-shadow depth is never lost. *)
let list_code_history_tree (t : t) ~(branch_id : string) ~(name : string)
  : code_history_entry list =
  match code_path_for_name t ~branch_id ~name with
  | None -> []
  | Some (path, rel) -> (
      match t.conn with
      | None -> []
      | Some conn ->
          let vers = Tree_store.versioned_history conn ~path () in
          (* newest gen = the head (excluded); anything below it is history
     *)
          let head_gen =
            List.fold_left
              (fun acc v ->
                match v.Tree_store.v_gen with
                | Some g -> Int64.max acc g
                | None -> acc)
              0L vers
          in
          let below_head =
            List.filter
              (fun v ->
                match v.Tree_store.v_gen with
                | Some g -> Int64.compare g head_gen < 0
                | None -> true)
              vers
          in
          List.rev_map
            (fun v ->
              { rel
              ; op =
                  (if v.Tree_store.v_tombstone then "DELETE" else "UPDATE")
              ; author = v.Tree_store.v_author
              ; doc = code_doc_of_version v
              ; package = None
              ; changed_at =
                  (match v.Tree_store.v_created_at with
                   | Some c -> c
                   | None -> "")
              ; body =
                  (if v.Tree_store.v_tombstone then None
                   else code_body_of_version v)
              })
            below_head)

(** Newest-first history rows for [name] on [branch_id]. Tree-backed (Phase
     3)
    with an 012 fallback for pre-shadow names. *)
let list_code_history (t : t) ~(branch_id : string) ~(name : string)
  : code_history_entry list =
```

```
  let tree_hist = list_code_history_tree t ~branch_id ~name in
  if tree_hist <> [] then tree_hist
  else list_code_history_012 t ~branch_id ~name

let list_code_history_json (t : t) ~(branch_id : string) ~(name : string)
  : string =
  let entries = list_code_history t ~branch_id ~name in
  Yojson.Safe.to_string
    (`List (List.map code_history_entry_to_yojson entries))

(** Live body text for a name on a branch: function body preferred, else
    type fields, else view_bindings handler JSON. *)
let live_code_body_text (t : t) ~(branch_id : string) ~(name : string)
  : string option =
  match t.conn with
  | None -> None
  | Some conn ->
      (try
         let try_fn () =
           let sql =
             "SELECT body::text FROM function_defs " ^
             "WHERE branch_id = $1 AND name = $2"
           in
           match
             exec conn
               ~params:
                 [ Pgx.Value.of_string branch_id; Pgx.Value.of_string name ]
               sql
           with
           | [ [ c ] ] -> Pgx.Value.to_string c
           | _ -> None
         in
         let try_ty () =
           let sql =
             "SELECT fields::text FROM type_defs " ^
             "WHERE branch_id = $1 AND name = $2"
           in
           match
             exec conn
               ~params:
                 [ Pgx.Value.of_string branch_id; Pgx.Value.of_string name ]
               sql
           with
           | [ [ c ] ] -> Pgx.Value.to_string c
           | _ -> None
         in
         let try_rt () =
           let sql =
             "SELECT jsonb_build_object('handler', handler, " ^
             "  'visibility', COALESCE(visibility, 'private'))::text " ^
             "FROM view_bindings WHERE branch_id = $1 AND route = $2"
           in
           match
             exec conn
               ~params:
                 [ Pgx.Value.of_string branch_id; Pgx.Value.of_string name ]
               sql
           with
           | [ [ c ] ] -> Pgx.Value.to_string c
           | _ -> None
         in
         (match try_fn () with
          | Some s -> Some s
          | None ->
              (match try_ty () with
               | Some s -> Some s
               | None -> try_rt ()))
       with exn ->
         Printf.eprintf "[store] live_code_body_text: %s\n"
           (Printexc.to_string exn);
         None)

(** Minimal line-oriented text diff: lines only in [a], only in [b], or
```

```
    shared. Not a full LCS unified diff -- good enough for prim smoke. *)
let simple_text_diff ~(a : string) ~(b : string) : string =
  let lines s =
    if s = "" then []
    else String.split_on_char '\n' s
  in
  let la = lines a in
  let lb = lines b in
  let module S = Set.Make (String) in
  let sa = List.fold_left (fun acc x -> S.add x acc) S.empty la in
  let sb = List.fold_left (fun acc x -> S.add x acc) S.empty lb in
  let buf = Buffer.create 256 in
  Buffer.add_string buf "--- history\n+++ current\n";
  List.iter
    (fun line ->
      if not (S.mem line sb) then begin
        Buffer.add_char buf '-';
        Buffer.add_string buf line;
        Buffer.add_char buf '\n'
      end)
    la;
  List.iter
    (fun line ->
      if not (S.mem line sa) then begin
        Buffer.add_char buf '+';
        Buffer.add_string buf line;
        Buffer.add_char buf '\n'
      end)
    lb;
  Buffer.contents buf

(** Current live body vs the nth-newest history body (1-based). *)
let code_diff (t : t) ~(branch_id : string) ~(name : string) ~(n : int)
  : (string, string) result =
  if n < 1 then Error "code-diff: n must be >= 1"
  else
    let hist = list_code_history t ~branch_id ~name in
    match List.nth_opt hist (n - 1) with
    | None -> Error (Printf.sprintf "code-diff: no history entry #%d" n)
    | Some entry ->
        let old_body = Option.value entry.body ~default:"" in
        let live =
          Option.value
            (live_code_body_text t ~branch_id ~name)
            ~default:""
        in
        Ok (simple_text_diff ~a:old_body ~b:live)

(* The handler + visibility from a route node's inner body. The route tree
   payload body is {route, handler, visibility} (Tree_store.upsert_route);
   code-restore needs the two writable fields to re-write the route. *)
let route_parts_of_body (body_js : string) : (string * string) option =
  match Yojson.Safe.from_string body_js with
  | `Assoc kv -> (
      match List.assoc_opt "handler" kv with
      | Some (`String h) ->
          let vis =
            match List.assoc_opt "visibility" kv with
            | Some (`String v) -> v
            | _ -> "private"
          in
          Some (h, vis)
      | _ -> None )
  | _ -> None

(** (code-restore name [n] [owner/branch]) -- restore a code node (fn/type/
    route) at head-n as a NEW write at head+1 (restore-by-write). This is
     the
    code-path analogue of (undo) (which is datum-path only): gen-read +
     write
    the same value back through the code write surface (upsert_fn /
    upsert_type / upsert_route), so the reverse mirror re-populates
    function_defs / type_defs / view_bindings and the restore itself is
```

```
    versioned + redoable (the restored value becomes the new head; calling it
    again restores the state before that). The below-head semantic matches
    code-history's "read the version row at gen-1": n=1 restores the state
    just before the latest change; n=2 the state before that. Out-of-range n,
    no history, no body, or an unsupported rel are clean error strings (never
    a crash). Consumers: tree-time.consumers -- code-restore = gen-read +
    define, the lossless-redefine pattern the 012 design documented by hand,
    now a real prim. [author] is the session user, the fallback only when the
    source version row carries no author. *)
let code_restore (t : t) ~(branch_id : string) ~(name : string)
    ~(author : string) ?(n : int = 1) () : string =
  match code_path_for_name t ~branch_id ~name with
  | None -> Printf.sprintf "code-restore: no such code path: %s" name
  | Some (path, rel) -> (
      match t.conn with
      | None -> "code-restore: no DB"
      | Some conn ->
          let vers = Tree_store.versioned_history conn ~path () in
          if vers = [] then
            Printf.sprintf "code-restore: no version history for %s" name
          else
            (* newest-first; index 0 is the live head (the current state, never
               a restore target), index n is the state n steps back. Row-index,
               NOT gen arithmetic: the branch_generations counter is shared
               across branches and bumped by tombstones, so gens are not
               guaranteed consecutive for a single path -- counting version rows
               is the correct "n steps back". *)
            let newest_first = List.rev vers in
            if n < 1 then Printf.sprintf "code-restore: n must be >= 1"
            else if n >= List.length newest_first then
              Printf.sprintf
                "code-restore: %s has no state %d steps back" name n
            else
              let v = List.nth newest_first n in
              if v.Tree_store.v_tombstone then
                Printf.sprintf
                  "code-restore: %s at %d steps back is a deletion (tombstone); use tree/delete"
                  name n
              else
                let rest_author =
                  match v.Tree_store.v_author with
                  | Some a -> a
                  | None -> author
                in
                (match code_body_of_version v with
                 | None ->
                     Printf.sprintf
                       "code-restore: no body %d steps back for %s" n name
                 | Some body_js -> (
                     match rel with
                     | "function_defs" ->
                         let doc = code_doc_of_version v in
                         Tree_store.upsert_fn conn ~branch_id ~name
                           ~body_json:body_js ~author:rest_author ~doc;
                         Printf.sprintf
                           "restored fn %s (%d steps back)" name n
                     | "type_defs" ->
                         Tree_store.upsert_type conn ~branch_id ~name
                           ~fields_json:body_js ~author:rest_author;
                         Printf.sprintf
                           "restored type %s (%d steps back)" name n
                     | "view_bindings" -> (
                         match route_parts_of_body body_js with
                         | Some (handler, visibility) ->
                             Tree_store.upsert_route conn ~branch_id
```

```
                              ~route:name ~handler ~visibility;
                            Printf.sprintf
                              "restored route %s (%d steps back)" name n
                        | None ->
                            Printf.sprintf
                              "code-restore: route body malformed for %s"
                              name )
                    | _ ->
                        Printf.sprintf "code-restore: unsupported rel %s"
    rel)))

(* ---- tree-time.gc (5.3): retention worker ------------------------------
   Retention is habitat data + the jobs runner, composed on the existing
   gen-window partitions:
     - policy rows (tree_retention_policy, migration 022) declare per-path-
       class retention (code keeps all; data per-type, default
    keep-everything
       until a policy exists).
     - tree_versions_retain(path_class, keep_windows) (SQL, 022) computes
    the
       retention floor from the live head gen, ROLLS UP the surviving state
    of
       fully-retired paths to CHECKPOINT rows at the floor boundary (so
    as-of
       reads of ancient gens still resolve), then DROP PARTITIONs every lazy
       gen-window entirely below the floor (never the base window).
    Idempotent
       + conservative; on the real habitat it is gated by a scheduled job on
       the jobs runner (migration 015), not ad-hoc deletion.
   The first long pole is code paths (tiny); retention is a TODO until a
   path class outgrows its welcome -- measured via /timing-style storage
   accounting, not vibes. *)

let retention_set_policy (t : t) ~(path_class : string)
    ~(type_name : string option) ~(keep_windows : int64) : string =
  if path_class <> "code" && path_class <> "data" then
    "retention-policy-set: path_class must be code or data"
  else
    match current_conn t with
    | None -> "retention-policy-set: no DB"
    | Some conn -> (
        try
          let tn =
            match type_name with
            | None -> Pgx.Value.null
            | Some s -> Pgx.Value.of_string s
          in
          ignore
            (exec conn
               ~params:
                 [ Pgx.Value.of_string path_class; tn;
                   Pgx.Value.of_int64 keep_windows ]
               "INSERT INTO tree_retention_policy (path_class, type_name,
    keep_windows)
                VALUES ($1, $2, $3)
                ON CONFLICT (path_class, type_name)
                  DO UPDATE SET keep_windows = EXCLUDED.keep_windows"
              : Pgx.row list);
          match type_name with
          | None -> Printf.sprintf "retention policy %s: keep %Ld windows"
    path_class keep_windows
          | Some tnm ->
              Printf.sprintf "retention policy %s/%s: keep %Ld windows"
    path_class tnm keep_windows
        with ex ->
          Printf.sprintf "retention-policy-set: %s" (Printexc.to_string ex))

let retention_delete_policy (t : t) ~(path_class : string)
    ~(type_name : string option) : string =
  match current_conn t with
  | None -> "retention-delete-policy: no DB"
  | Some conn -> (
      try
```

```
        let tn =
          match type_name with
          | None -> Pgx.Value.null
          | Some s -> Pgx.Value.of_string s
        in
        ignore
          (exec conn
             ~params:[ Pgx.Value.of_string path_class; tn ]
             "DELETE FROM tree_retention_policy
              WHERE path_class = $1 AND type_name IS NOT DISTINCT FROM $2"
            : Pgx.row list);
        "retention policy deleted"
      with ex ->
        Printf.sprintf "retention-delete-policy: %s" (Printexc.to_string
     ex))

let retention_list_policies (t : t)
    : (string * string option * int64 * string) list =
  match current_conn t with
  | None -> []
  | Some conn -> (
      try
        exec conn
          "SELECT path_class, type_name, keep_windows,
                  created_at::text FROM tree_retention_policy
           ORDER BY path_class, type_name NULLS FIRST"
        |> List.map (fun row ->
               let cell i = List.nth row i in
               let pc =
                 match Pgx.Value.to_string (cell 0) with
                 | Some s -> s
                 | None -> ""
               in
               let tn =
                 match Pgx.Value.to_string (cell 1) with
                 | Some s -> Some s
                 | None -> None
               in
               let kw =
                 match Pgx.Value.to_int64 (cell 2) with
                 | Some k -> k
                 | None -> 0L
               in
               let ca =
                 match Pgx.Value.to_string (cell 3) with
                 | Some s -> s
                 | None -> ""
               in
               (pc, tn, kw, ca))
      with _ -> [])

(* Run the retention worker for each path class that has an explicit
   whole-class policy (type_name IS NULL). Whole-class policies are the v1
   gate: retention is a TODO until a path class outgrows its welcome, and
   the global gen-window partitions mean the floor is whole-class regardless
   of any per-type rows (per-type rows are recorded for future finer-grained
   work but do not change the shared floor). keep_everything until a
   whole-class policy exists -- the default -- is a no-op, never ad-hoc
   deletion. Returns per-class summary rows (dropped, checkpointed). *)
let retention_run (t : t) ~(path_class : string option) ()
    : (string * int64 * int64) list =
  let all =
    match path_class with
    | Some pc -> [ pc ]
    | None -> [ "code"; "data" ]
  in
  match current_conn t with
  | None -> []
  | Some conn -> (
      try
        List.concat_map
          (fun pc ->
            (* whole-class policy for this path class? *)
```

```
            let pol =
              exec conn
                ~params:[ Pgx.Value.of_string pc ]
                "SELECT keep_windows FROM tree_retention_policy
                 WHERE path_class = $1 AND type_name IS NULL"
            in
            match pol with
            | [] -> [] (* keep everything: no whole-class policy exists *)
            | [ row ] -> (
                match Pgx.Value.to_int64 (List.nth row 0) with
                | None -> []
                | Some kw ->
                    let rows =
                      exec conn
                        ~params:
                          [ Pgx.Value.of_string pc;
                            Pgx.Value.of_int64 kw ]
                        "SELECT dropped_windows, checkpointed_paths
                         FROM tree_versions_retain($1::text, $2::bigint)"
                    in
                    (match rows with
                     | [ r ] ->
                         let d =
                           match Pgx.Value.to_int64 (List.nth r 0) with
                           | Some x -> x
                           | None -> 0L
                         in
                         let c =
                           match Pgx.Value.to_int64 (List.nth r 1) with
                           | Some x -> x
                           | None -> 0L
                         in
                         [ (pc, d, c) ]
                     | _ -> []))
            | _ -> [])
          all
      with _ -> [])
```

109 image/lib/store_db.ml

```
(** Pricklypear Image -- Store connection, config, transactions (Eio floor).

    Domain-facing API returns plain ['a] (blocks the calling Eio fiber).
    Single backend: Io_eio.Pg (pure pgx over Eio.Net). Process always runs
    under Eio_main (image/bin/main.ml). *)

type config = {
  database : string;
  host : string;
  port : int;
  user : string;
  password : string;
}

let env_or name dft = match Sys.getenv_opt name with Some s -> s | None ->
     dft

let config_from_env () : config =
  {
    database = env_or "PP_DB_NAME" (env_or "PGDATABASE" "pricklypear");
    host = env_or "PP_DB_HOST" (env_or "PGHOST" "127.0.0.1");
    port = int_of_string (env_or "PP_DB_PORT" (env_or "PGPORT" "5433"));
    user = env_or "PP_DB_USER" (env_or "PGUSER" "pricklypear_image");
    password = env_or "PP_DB_PASS" "";
  }

(** Connection handle + mutex: identity-monad pgx only locks per SQL op, so
    concurrent Httpun_eio fibers would interleave BEGIN/SELECT on one wire.
    All SQL (exec + whole transactions) takes this lock; same-fiber re-entry
    is allowed via [eio_lock_depth_key] so a tx body can exec. *)
type conn = Io_eio.Pg.t * Eio.Mutex.t

let eio_lock_depth_key : int Eio.Fiber.key = Eio.Fiber.create_key ()

let with_eio_conn_lock (mu : Eio.Mutex.t) (f : unit -> 'a) : 'a =
  match Eio.Fiber.get eio_lock_depth_key with
  | Some d when d > 0 ->
      Eio.Fiber.with_binding eio_lock_depth_key (d + 1) f
  | _ ->
      Eio.Mutex.lock mu;
      (match
         Eio.Fiber.with_binding eio_lock_depth_key 1 (fun () ->
             Eio.Cancel.protect f)
       with
       | v ->
           Eio.Mutex.unlock mu;
           v
       | exception ex ->
           Eio.Mutex.unlock mu;
           raise ex)

type t = {
  cfg : config;
  mutable conn : conn option;
  mutable branch_id : string;
  mutable current_tx : conn option;
  (* image.boot-seed-batch: when seed_additive boot seed is running, this
     holds the set of function/route names already present on the seed
     branch (system/main), so persist_function_if_absent /
     register_route_if_absent skip the per-name get_by_path SELECT. None
     outside the boot seed pass. See borge/image.borg boot-seed-batch. *)
  mutable seed_names : (string, unit) Hashtbl.t option;
}

let make ~cfg ~branch_id () : t =
  { cfg; conn = None; branch_id; current_tx = None; seed_names = None }

(** Expose the config so the session registry can open per-session
    connections with the same parameters (image.per-identity-sessions). *)
let config (t : t) : config = t.cfg

```

```
let conn_of (t : t) : conn option = t.conn

let current_conn (t : t) : conn option =
  match t.current_tx with Some tx -> Some tx | None -> t.conn

let in_transaction (t : t) : bool = t.current_tx <> None

let exec ((pg, mu) : conn) ?params sql : Pgx.row list =
  with_eio_conn_lock mu (fun () ->
      match params with
      | None -> Io_eio.Pg.execute pg sql
      | Some params -> Io_eio.Pg.execute ~params pg sql)

(** Fiber-local session username for SET LOCAL pp.user (RLS floor).
    Bound by Request_context.with_current so store_db need not depend on
    the later request_context module (dune module order). *)
let pp_user_key : string Eio.Fiber.key = Eio.Fiber.create_key ()

let pp_user_safe (s : string) : string =
  let s = String.lowercase_ascii (String.trim s) in
  let ok =
    s <> ""
    && String.for_all
         (function
           | 'a' .. 'z' | '0' .. '9' | '-' | '_' -> true
           | _ -> false)
         s
  in
  if ok then s else "anonymous"

let with_pp_user (username : string) (f : unit -> 'a) : 'a =
  Eio.Fiber.with_binding pp_user_key (pp_user_safe username) f

let current_pp_user () : string =
  match Eio.Fiber.get pp_user_key with
  | Some u -> u
  | None -> "anonymous"

(** SET LOCAL pp.user inside the open transaction so RLS / tree_data_rows
    can confine. LOCAL scopes to the tx; safe even on a shared connection.
     *)
let set_local_pp_user (pg : Io_eio.Pg.t) : unit =
  let u = current_pp_user () in
  let _ =
    Io_eio.Pg.execute ~params:[ Pgx.Value.of_string u ]
      pg "SELECT set_config('pp.user', $1, true)"
  in
  ()

let with_transaction ?(isolation = Pgx.Isolation.Serializable) (t : t)
    (f : conn option -> 'a) : 'a =
  match t.conn with
  | None -> f None
  | Some (pg, mu) ->
      with_eio_conn_lock mu (fun () ->
          Io_eio.Pg.with_transaction ~isolation pg (fun tx ->
              set_local_pp_user tx;
              f (Some (tx, mu))))

let with_dynamic_transaction ?(isolation = Pgx.Isolation.Serializable)
    (t : t) (f : unit -> 'a) : 'a =
  match t.conn with
  | None -> f ()
  | Some (pg, mu) ->
      with_eio_conn_lock mu (fun () ->
          if t.current_tx <> None then
            raise
              (Pricklypear_lisp.Ast.Interp_error
                 (Pricklypear_lisp.Ast.User_error
                    ("with-transaction: nested transactions not supported",
    "bad-args")))
          else
            Io_eio.Pg.with_transaction ~isolation pg (fun tx_conn ->
```

```
                t.current_tx <- Some (tx_conn, mu);
                set_local_pp_user tx_conn;
                Fun.protect
                  ~finally:(fun () -> t.current_tx <- None)
                  f))

let connect (t : t) : unit =
  match t.conn with
  | Some _ -> ()
  | None -> (
      try
        let c =
          Io_eio.Pg.connect ~host:t.cfg.host ~port:t.cfg.port
            ~user:t.cfg.user ~password:t.cfg.password
            ~database:t.cfg.database ()
        in
        let mu = Eio.Mutex.create () in
        t.conn <- Some (c, mu);
        Printf.eprintf "[store] connected (eio) to PG %s:%d/%s as %s\n%!"
          t.cfg.host t.cfg.port t.cfg.database t.cfg.user
      with exn ->
        Printf.eprintf "[store] PG connect failed (eio): %s\n%!"
          (Printexc.to_string exn))

let health_check (t : t) : (unit, string) result =
  match current_conn t with
  | None -> Error "no database connection"
  | Some conn -> (
      try
        match exec conn "SELECT 1" with
        | _ :: _ -> Ok ()
        | [] -> Error "SELECT 1 returned no rows"
      with exn -> Error (Printexc.to_string exn))

(** @deprecated compatibility alias -- was Pg_lwt. *)
module Pg = Io_eio.Pg
```

110 image/lib/sync.ml

```
(** image/lib/sync.ml -- borge/local-first.borg sync (campaign 8.3).

    The SERVER-SIDE sync merge authority. Sync = three-way tree-diff
    (branching.borg / one-tree.branching-interaction), keyed on subtree
    node hashes (the git protocol in miniature):

       1. Client sends its (path, hash) manifest for the lens subtree.
       2. Server compares against its own subtree hashes; reports which
          paths differ or are missing on each side.
       3. Each side sends the node BODIES the other is missing or has
          stale.
       4. Server applies the three-way merge (common ancestor = the
          last-synced generation) and returns the merged subtree state.

    The envelope hash is the negotiation; the file is the transfer. On
    the tree_nodes side the hash is [Tree_store.hash_body] (sha256 over
    the canonical body), the SAME content hash the client's lens
    substrate computes offline ([Lens_store.content_hash]) -- one hash
    scheme, both sides, so a Same/Diverge classification is meaningful
    across the wire.

    This module owns the pure reconciliation + three-way merge planner
    (the heart of the exchange -- classification of a manifest union and
    the conflict-free / LWW-v0 plan), plus the server-side harness:
    reading its own subtree manifest from tree_nodes and applying a
    validated reconcile plan back to the store.

    The client-facing transport/exchange (an HTTP rendezvous that ships
    the manifest + bodies) is owned by the client lens / sync client;
    this module is the server half: given a client's (path, hash)
    manifest for the current lens, it decides what the server must send
    (ServerOnly = pull), what the client must send (ClientOnly = push),
    and where the three-way merge (base = last-synced) conflicts.
*)

(* ================================================================
   HASH MANIFESTS
   ================================================================ *)

(** A (path, hash) manifest: path -> content hash. This is the exact
    shape [Lens_store.node_manifest] produces on the client and
    [server_manifest] below reads from tree_nodes. The hash is the
    node-envelope content hash (sha256 over the canonical body). *)
type manifest = (string * string) list

(** Association-list lookup. *)
let assoc_find (m : manifest) (path : string) : string option =
  List.assoc_opt path m

(** Convert a manifest to a sorted association list (path, hash), deduped
    (later wins, matching assoc semantics). *)
let normalize (m : manifest) : manifest =
  let tbl = Hashtbl.create (List.length m) in
  List.iter (fun (p, h) -> Hashtbl.replace tbl p h) m;
  Hashtbl.fold (fun p h acc -> (p, h) :: acc) tbl []
  |> List.sort (fun (a, _) (b, _) -> String.compare a b)

let to_assoc (m : manifest) : (string * string) list = normalize m

let of_assoc (a : (string * string) list) : manifest = a

(* ================================================================
   CLASSIFICATION (pure)
   ================================================================ *)

(** Per-path delta between a server manifest and a client manifest.
    - [Same]        -- both sides hold the node with the same hash.
    - [ClientOnly]  -- only the client holds the node (server missing it);
                       the client must PUSH the body to the server.
    - [ServerOnly]  -- only the server holds the node (client missing it);
                       the server must PULL (send the body to the client).
```

```
    - [Diverge]     -- both sides hold the node but with DIFFERENT hashes.
                       Requires the three-way merge (last-synced base) to
                       decide push/pull/conflict; the raw two-way view can
                       only report it.
*)
type delta = Same | ClientOnly | ServerOnly | Diverge

let classify_delta (server : manifest) (client : manifest) (path : string)
    : delta =
  match assoc_find server path, assoc_find client path with
  | Some sh, Some ch when sh = ch -> Same
  | Some _, Some _ -> Diverge
  | Some _, None -> ServerOnly
  | None, Some _ -> ClientOnly
  | None, None -> Same  (* absent on both: nothing to do *)

(** Reconcile a client manifest against the server manifest. Returns the
    (path, hash) lists per delta class. Pure. *)
type classify_report = {
  same_len : int;
  client_only : manifest;   (* client -> server pushes *)
  server_only : manifest;   (* server -> client pulls *)
  diverge : manifest;       (* both changed; needs three-way *)
}

let classify (server : manifest) (client : manifest) : classify_report =
  let all =
    List.fold_left
      (fun acc (p, _) -> if List.mem_assoc p acc then acc else (p, 0) ::
    acc)
      [] (to_assoc server @ to_assoc client)
  in
  List.fold_left
    (fun r (p, _) ->
      match classify_delta server client p with
      | Same -> { r with same_len = r.same_len + 1 }
      | ClientOnly ->
          { r with client_only = (p, Option.get (assoc_find client p)) ::
    r.client_only }
      | ServerOnly ->
          { r with server_only = (p, Option.get (assoc_find server p)) ::
    r.server_only }
      | Diverge ->
          { r with diverge = (p, Option.get (assoc_find client p)) ::
    r.diverge })
    { same_len = 0; client_only = []; server_only = []; diverge = [] }
    all

(* ================================================================
   THREE-WAY MERGE PLANNER (pure)
   ================================================================ *)

(** Three-way merge over (base = last-synced, ours = server, theirs =
    client), per the campaign-3 diff philosophy (branching.ast-nodes):
    disjoint changes compose; a same-path change kept identical by both
    sides folds to one; a same-path DIFFERENT change is a CONFLICT --
    report-only (LWW v0 keeps ours=server and surfaces the conflict; the
    obituary rule applies to sync losses too, so the conflict is never
    silently dropped).

    Per-path outcome:
      unchanged  -- base = ours = theirs (or absent on all): nothing.
      theirs_add -- added on client only since the base (base absent, ours
                    absent, theirs present): apply the client node.
      ours_add   -- added on server only since the base (base absent,
                    theirs absent, ours present): keep, nothing to send.
      theirs_mod -- ours stayed at base, theirs changed (base = ours ?
                    theirs): apply the client body (a clean client edit).
      ours_mod   -- theirs stayed at base, ours changed (base = theirs ?
                    ours): keep server, nothing to send.
      identical  -- both moved to the same new hash: keep either.
      conflict   -- base present, both changed and to different hashes:
                    report; LWW v0 keeps ours (server) and surfaces.
```

```
      theirs_del -- deleted on client only (base present, ours present,
                    theirs absent): delete the server node (a client
                    deletion applied).
      ours_del   -- deleted on server only: nothing to send (server node
                    gone; client still holds a stale copy ? client pulls
                    the deletion = the server-only tombstone/pull).
*)
type outcome =
  | Unchanged
  | Theirs_add                (* client push (server applies) *)
  | Ours_keep                 (* server keeps; no transfer *)
  | Theirs_mod                (* client push (server applies) *)
  | Identical
  | Conflict of string * string * string * string
                              (* path, base_hash, ours_hash, theirs_hash *)
  | Theirs_del                (* client deletion applied server-side *)
  | Ours_del

type plan = {
  path : string;
  outcome : outcome;
}

let plan_of (base : manifest) (ours : manifest) (theirs : manifest)
    : plan list =
  let bh = assoc_find base in
  let oh = assoc_find ours in
  let th = assoc_find theirs in
  let all =
    List.fold_left
      (fun acc (p, _) ->
        if List.mem_assoc p acc then acc else (p, ()) :: acc)
      []
      (to_assoc base @ to_assoc ours @ to_assoc theirs)
  in
  List.map
    (fun (p, ()) ->
      let b, o, t = bh p, oh p, th p in
      let outcome =
        match b, o, t with
        (* all agree / all absent *)
        | bo, oo, to_ when bo = oo && oo = to_ -> Unchanged
        (* client added a node both the base and the server lack *)
        | None, None, Some _ -> Theirs_add
        (* server added a node; client never had it (pull provides it) *)
        | None, Some _, None -> Ours_keep
        (* client edited a node the server never moved from base *)
        | Some x, Some oo, Some t_ when oo = x && t_ <> x -> Theirs_mod
        (* server edited a node the client never moved from base *)
        | Some x, Some oo, Some t_ when t_ = x && oo <> x -> Ours_keep
        (* both moved to the SAME new hash: one change, not two *)
        | Some _, Some oo, Some t_ when oo = t_ -> Identical
        (* base present, both moved and to different hashes: conflict *)
        | Some x, Some oo, Some t_ -> Conflict (p, x, oo, t_)
        (* client deleted a node the server still holds *)
        | Some _, Some _, None -> Theirs_del
        (* server deleted a node the client still holds (client pulls) *)
        | Some _, None, Some _ -> Ours_del
        (* contradictory (some/None combos not covered): keep ours *)
        | _ -> Unchanged
      in
      { path = p; outcome })
    all

(** The non-trivial (transfer-or-conflict) subset of a three-way plan:
    the rows a sync round must act on. *)
let actionable (plans : plan list) : plan list =
  List.filter
    (fun { outcome; _ } ->
      match outcome with
      | Unchanged | Ours_keep | Identical -> false
      | Theirs_add | Theirs_mod | Conflict _ | Theirs_del | Ours_del ->
     true)
```

```
    plans

(* ================================================================
   SERVER-SIDE HARNESS (DB-backed)
   ================================================================ *)

(** Read the server's (path, hash) manifest for the subtree under
    [prefix] (one-tree path-prefix RLS confinement = the replication
    boundary; the caller passes the replication-unit prefix). None when
    there is no live connection. *)
let server_manifest (store : Store.t) ~(prefix : string) : manifest option =
  match Store.conn_of store with
  | None -> None
  | Some conn ->
      let rows =
        Io_lwt.Db.execute conn
          ~params:
            [ Pgx.Value.of_string prefix ]
          "SELECT path::text, hash FROM tree_nodes \
           WHERE path <@ $1::ltree ORDER BY path::text ASC"
      in
      Some
        (List.filter_map
           (function
             | [ p; h ] -> (
                 match Pgx.Value.to_string p, Pgx.Value.to_string h with
                 | Some pp, Some hh -> Some (pp, hh)
                 | _ -> None)
             | _ -> None)
           rows)

(** Apply a client push body for [path] to the store. The client sends
    its node envelope {path, kind, type, body, author, ?}; the server
    re-writes it through the typed-boundary-validated tree write surface
    so a datum lands exactly as (tree/upsert) would (cross-checks the
    declared type fields). Kind-aware:

      - fn / type / route / pkg code nodes resolve the branch-lensed path
        under the current lens branch and write via the code write
        surface (upsert_fn / upsert_type / route via upsert_hier).
      - datum nodes write via [Tree_store.upsert_datum] (path-explicit,
        typed-boundary validation preserved).

    Returns [Ok ()] or [Error message]. *)
let apply_client_node (store : Store.t) ~(env : Yojson.Safe.t) : (unit,
     string) result =
  let path, kind, type_name, body, author =
    match env with
    | `Assoc kv ->
        let str k = List.assoc_opt k kv |> function Some (`String s) ->
     Some s | _ -> None in
        let get_body =
          List.assoc_opt "body" kv |> function Some b -> Some b | _ -> None
        in
        ( str "path"
        , str "kind"
        , str "type"
        , get_body
        , (match str "author" with Some a -> a | None -> "sync") )
    | _ -> (None, None, None, None, "sync")
  in
  match path, kind with
  | Some p, Some "datum" ->
      (match type_name, body with
       | Some ty, Some b ->
           (* typed-boundary validation: validate the client body against
              the type the SERVER declares for it (the server is the
              merge/index authority, not the client) so a body missing a
              required field is rejected the same way a local create
              would be -- the client is a peer, not a bypass. *)
           (let declared = Store.lookup_type_fields store ~name:ty in
            match
              match declared with
```

```
              | None -> None
              | Some fields ->
                  (match
                     Tree_store.validate_fields ~declared:fields
                       (Yojson.Safe.to_string b)
                   with
                  | Ok () -> None
                  | Error msg -> Some msg)
           with
          | Some msg -> Error msg
          | None ->
              (match Store.conn_of store with
               | None -> Error "sync: no DB"
               | Some conn ->
                   ignore
                     (Tree_store.upsert_datum conn ~path:p ~type_name:ty
                        ~body_json:(Yojson.Safe.to_string b) ~author
    ~rank:None);
                   Ok ()))
      | _ -> Error "sync: datum node missing type/body")
  | Some p, Some ("fn" | "type" | "route" | "pkg" as k) ->
      (* code-lens nodes carry {body:{src,?}}; the server re-writes the
         source through the code write surface so the reverse mirror
         repopulates the scaffolding tables. For v0 the code sync path is
         the branch-pointer move (promote/merge) owned by merge_branch;
         a direct code-node write here is the fallback for a code lens the
         server chose to reconcile in place. *)
      (match Store.conn_of store with
       | None -> Error "sync: no DB"
       | Some conn ->
           let body_json = match body with Some b -> Yojson.Safe.to_string
    b | None -> "{}" in
           (match k with
            | "fn" ->
                let name =
                  try List.nth (String.split_on_char '.' p) 3 with _ -> p
                in
                Tree_store.upsert_fn conn
    ~branch_id:(Store.current_branch_id store)
                  ~name ~body_json ~author ~doc:(Some ("code:" ^ name));
                Ok ()
            | _ -> Error "sync: code kind not yet reconciled in place"))
  | Some p, Some k ->
      (* hierarchical / other kinds: write via the generic path-explicit
         upsert_hier so headings/folders/notes replicate too. *)
      (match Store.conn_of store with
       | None -> Error "sync: no DB"
       | Some conn ->
           let body_json = match body with Some b -> Yojson.Safe.to_string
    b | None -> "{}" in
           ignore
             (Tree_store.upsert_hier conn ~path:p ~kind:k
                ~type_name:(match type_name with Some t -> t | None -> k)
                ~body_json ~author ~rank:None);
           Ok ())
  | _ -> Error "sync: node envelope missing path/kind"

(** Apply a whole reconcile/merge plan to the server: for every actionable
    row the plan directs the server to take (Theirs_add / Theirs_mod /
    Theirs_del are applied from the client bodies; a Conflict is LWW-
    arbitrated per sync-conflicts v0 -- client envelope strictly newer wins
    and is applied, otherwise the server copy stays), look up the client
    node envelope. Since this server harness applies the PLAN (not the
    bodies it must first receive), the client bodies are passed as an
    assoc path -> envelope JSON. Every resolution is SURFACED: the report
    lists the conflicts with their hashes, which side LWW picked per
    path, and the Ours_del paths the client must mirror-delete locally. *)

(* ---- sync-conflicts v0: LWW-per-node resolution (applied + surfaced) ----

    Detection was 8.3 (hashes classify; conflicts REPORT-only). The v0
    RESOLUTION policy sync-conflicts names is LAST-WRITER-WINS per node,
    updated_at arbitrating, with every loss SURFACED (the obituary rule
```

```
353     applies to sync losses -- no silent drop). One clock, both sides:
354     the client lens substrate stamps its envelope updated_at as an epoch
355     float (string_of_float Unix.gettimeofday); the server reads its node
356     updated_at as the same epoch float (extract(epoch from updated_at)).
357     The client wins a diverge/conflict pair iff its stamp is STRICTLY
358     newer -- ties and missing/unparseable stamps keep the server copy
359     (deterministic, conservative; a materialized-from-server envelope
360     carries the server's PG-format stamp, which is intentionally not
361     parseable as an epoch here, so an unedited pull can never wrongly
362     out-vote a newer server edit). Cross-machine clock skew is the
363     classic LWW tax; the spec accepts it for v0 and surfaces every
364     resolution. `commit` and the sync/push prim both apply this policy. *)
365
366 (** The server's updated_at epochs (float seconds since the epoch) for
367     the given paths -- the LWW arbitration clock. Paths absent from
368     tree_nodes are absent from the result. *)
369 let server_updated_at (store : Store.t) (paths : string list)
370     : (string * float) list =
371   match Store.conn_of store with
372   | None -> []
373   | Some conn ->
374       List.filter_map
375         (fun p ->
376           match
377             Io_lwt.Db.execute conn
378               ~params:[ Pgx.Value.of_string p ]
379               "SELECT extract(epoch from updated_at)::float8 FROM
    tree_nodes \
380                WHERE path = $1::ltree"
381           with
382           | [ [ v ] ] -> (
383               match Pgx.Value.to_float v with
384               | Some f -> Some (p, f)
385               | None -> None)
386           | _ -> None)
387         paths
388
389 (** A client node envelope's updated_at as an epoch float, or None when
390     absent/unparseable. *)
391 let envelope_updated_at (env : Yojson.Safe.t) : float option =
392   match env with
393   | `Assoc kv -> (
394       match List.assoc_opt "updated_at" kv with
395       | Some (`String s) -> (try Some (float_of_string s) with _ -> None)
396       | _ -> None)
397   | _ -> None
398
399 (** LWW v0 for one diverge/conflict pair: the client wins iff its stamp
400     is STRICTLY newer than the server node's. *)
401 let lww_client_wins (client_updated : float option) (server_updated : float)
402     : bool =
403   match client_updated with Some c -> c > server_updated | None -> false
404
405 type commit_report = {
406   applied : int;
407   conflicts : (string * string * string) list; (* path, ours_hash,
      theirs_hash *)
408   lww_client_won : string list;  (* conflicts arbitrated TO the client copy
      (applied) *)
409   lww_server_kept : string list; (* conflicts arbitrated TO the server copy
      (client edit surfaced as lost) *)
410   client_deletes : string list;  (* Ours_del: server deleted; client's
      stale copy must go too *)
411 }
412
413 let commit (store : Store.t)
414     ~(client_bodies : (string * Yojson.Safe.t) list)
415     (plans : plan list) : commit_report =
416   let epochs =
417     server_updated_at store
418       (List.filter_map
419          (fun { path; outcome } ->
420            match outcome with Conflict (p, _, _, _) -> Some p | _ -> None)
```

```
        plans)
  in
  List.fold_left
    (fun r { path; outcome } ->
      match outcome with
      | Theirs_add | Theirs_mod ->
          (match List.assoc_opt path client_bodies with
           | Some env ->
               (match apply_client_node store ~env with
                | Ok () -> { r with applied = r.applied + 1 }
                | Error _ -> r)
           | None -> r)
      | Theirs_del ->
          (match Store.conn_of store with
           | Some conn ->
               let n = Tree_store.delete_path conn ~path ~recursive:false in
               if n > 0 then { r with applied = r.applied + 1 } else r
           | None -> r)
      | Conflict (p, _b, o, t) ->
          (* LWW v0 (sync-conflicts): arbitrate by updated_at, then SURFACE
             the resolution either way -- the obituary rule applies to the
             losing side, never a silent drop. *)
          let r = { r with conflicts = (p, o, t) :: r.conflicts } in
          let client_won =
            match List.assoc_opt p epochs with
            | None -> false (* server node unreadable: keep the server *)
            | Some sv ->
                let cl =
                  match List.assoc_opt p client_bodies with
                  | Some env -> envelope_updated_at env
                  | None -> None
                in
                lww_client_wins cl sv
          in
          if client_won then
            match List.assoc_opt p client_bodies with
            | Some env -> (
                match apply_client_node store ~env with
                | Ok () ->
                    { r with
                      applied = r.applied + 1;
                      lww_client_won = p :: r.lww_client_won }
                | Error _ ->
                    { r with lww_server_kept = p :: r.lww_server_kept })
            | None -> { r with lww_server_kept = p :: r.lww_server_kept }
          else { r with lww_server_kept = p :: r.lww_server_kept }
      | Ours_del -> { r with client_deletes = path :: r.client_deletes }
      | Unchanged | Ours_keep | Identical -> r)
    {
      applied = 0;
      conflicts = [];
      lww_client_won = [];
      lww_server_kept = [];
      client_deletes = [];
    }
    plans

(* ---- report rendering (for the sync/plan prim + UI) ---- *)

let json_of_report (r : classify_report) : string =
  Yojson.Safe.to_string
    (`Assoc
       [ ("same", `Int r.same_len)
       ; ("client_only", `Int (List.length r.client_only))
       ; ("server_only", `Int (List.length r.server_only))
       ; ("diverge", `Int (List.length r.diverge))
       ; ("client_only_paths", `List (List.map (fun (p, _) -> `String p)
    (normalize r.client_only)))
       ; ("server_only_paths", `List (List.map (fun (p, _) -> `String p)
    (normalize r.server_only)))
       ; ("diverge_paths", `List (List.map (fun (p, _) -> `String p)
    (normalize r.diverge)))
       ])
```

```
491
492 let json_of_plan (plans : plan list) : string =
493   Yojson.Safe.to_string
494     (`Assoc
495        [ ( "rows"
496          , `List
497              (List.map
498                 (fun { path; outcome } ->
499                   let tag, detail =
500                     match outcome with
501                     | Unchanged -> ("unchanged", `Null)
502                     | Theirs_add -> ("theirs_add", `String path)
503                     | Ours_keep -> ("ours_keep", `String path)
504                     | Theirs_mod -> ("theirs_mod", `String path)
505                     | Identical -> ("identical", `Null)
506                     | Conflict (_, b, o, t) ->
507                         ("conflict",
508                          `Assoc [ ("base", `String b); ("ours", `String o);
    ("theirs", `String t) ])
509                     | Theirs_del -> ("theirs_del", `String path)
510                     | Ours_del -> ("ours_del", `String path)
511                   in
512                   `Assoc [ ("path", `String path); ("outcome", `String
    tag); ("detail", detail) ])
513                 plans)
514          ) ])
```

111 image/lib/timing.ml

```
(** Per-route page render timing (borge/image.borg page-timing).

    Captures server-side render cost for each handled user route:
    wall time (total_ms), accumulated DB wall time (db_ms) and DB-hit
    count (db_hits) from the eval budget, plus the wall clock (at).

    The sample set is an in-process bounded ring per route name -- no
    schema, no persistence in v1. Aggregated by the /timing system page
    and surfaced per-request via the X-Timing-Ms header. *)
open Pricklypear_lisp.Ast

type sample = {
  route : string;
  total_ms : float;   (* wall time of the handler body *)
  db_ms : float;      (* accumulated DB prim wall time *)
  db_hits : int;      (* DB-touching prim call count *)
  at : int64;         (* epoch ms when the sample was taken *)
}

(* Per-route ring of recent samples. *)
let max_samples_per_route = 100
let max_total_samples = 1000

let samples : (string, sample Queue.t) Hashtbl.t = Hashtbl.create 32
let total_count : int ref = ref 0

let now_ms () : int64 =
  Int64.of_float (Unix.gettimeofday () *. 1000.0)

(** Read the current eval budget's DB accumulators (defaults when no
    budget is active). *)
let budget_db () : float * int =
  match !Pricklypear_lisp.Eval.last_budget with
  | Some b -> (b.db_ms, b.db_hits)
  | None -> (0., 0)

(** Record a sample for [route], dropping oldest once the ring exceeds
    its bound. [total_ms] is passed in (measured by the caller around
    the handler); db_ms/db_hits are read from the just-run eval budget
    as a delta against their pre-call snapshot. *)
let record ~route ~total_ms ~db0 ~hits0 : unit =
  let db_ms, db_hits = budget_db () in
  let db_ms = max 0. (db_ms -. db0)
  and db_hits = max 0 (db_hits - hits0) in
  let q =
    match Hashtbl.find_opt samples route with
    | Some q -> q
    | None ->
        let q = Queue.create () in
        Hashtbl.replace samples route q;
        q
  in
  Queue.add { route; total_ms; db_ms; db_hits; at = now_ms () } q;
  incr total_count;
  (* Trim to bounds. *)
  while Queue.length q > max_samples_per_route do
    ignore (Queue.pop q)
  done;
  if !total_count > max_total_samples then begin
    (* Drop one oldest from the longest ring to keep overall bound. *)
    let longest = ref "" and len = ref 0 in
    Hashtbl.iter
      (fun r q ->
        let n = Queue.length q in
        if n > !len then begin len := n; longest := r end)
      samples;
    if !longest <> "" then
      match Hashtbl.find_opt samples !longest with
      | Some q when not (Queue.is_empty q) ->
          ignore (Queue.pop q);
          decr total_count
```

```
      | _ -> ()
  end

(* Request-scoped last measured total_ms, consumed by the HTTP response
   writers to attach the X-Timing-Ms header on handled-route responses. *)
let last_total_ms : float option ref = ref None

let set_last_total_ms (ms : float) : unit = last_total_ms := Some ms

(** Measure [f] (a handler invocation) for [route]: time it, record a
    sample (even on failure), and return the result + total_ms. *)
let measure ~route (f : unit -> 'a) : 'a * float =
  let db0, hits0 = budget_db () in
  let t0 = Unix.gettimeofday () in
  let result = f () in
  let total_ms = (Unix.gettimeofday () -. t0) *. 1000. in
  record ~route ~total_ms ~db0 ~hits0;
  set_last_total_ms total_ms;
  (result, total_ms)

type agg = {
  route : string;
  count : int;
  last_ms : float;
  avg_ms : float;
  max_ms : float;
  db_ms : float;   (* avg db_ms across samples *)
  db_hits : float; (* avg db_hits across samples *)
}

(** Aggregate the current sample set, slowest-first by max_ms. *)
let aggregates () : agg list =
  let out = ref [] in
  Hashtbl.iter
    (fun route q ->
      let count = Queue.length q in
      if count > 0 then begin
        let last_ms = ref 0. and max_ms = ref 0.
        and sum = ref 0. and dbsum = ref 0. and hsum = ref 0. in
        Queue.iter
          (fun s ->
            last_ms := s.total_ms;
            max_ms := max !max_ms s.total_ms;
            sum := !sum +. s.total_ms;
            dbsum := !dbsum +. s.db_ms;
            hsum := !hsum +. float_of_int s.db_hits)
          q;
        out :=
          { route
          ; count
          ; last_ms = !last_ms
          ; avg_ms = !sum /. float_of_int count
          ; max_ms = !max_ms
          ; db_ms = !dbsum /. float_of_int count
          ; db_hits = !hsum /. float_of_int count
          }
          :: !out
      end)
    samples;
  List.sort (fun a b -> compare b.max_ms a.max_ms) !out

(** Drop all samples (used by tests / future clear affordance). *)
let clear () : unit =
  Hashtbl.clear samples;
  total_count := 0

(** Return the X-Timing-Ms header (as a [name,value] pair) if a route
    was measured for this request, consuming it so a later response in
    the same request does not double-report. Empty when none. *)
let timing_header () : (string * string) list =
  match !last_total_ms with
  | None -> []
  | Some ms ->
```

```
145       last_total_ms := None;
146       [("x-timing-ms", Printf.sprintf "%.1f" ms)]
```

112 image/lib/tree_store.ml

```
(** Pricklypear Image -- ONE TREE store (spec [one-tree]).

    The tree is the substrate that will replace the relational
    scaffolding (function_defs / type_defs / view_bindings / data_rows)
    behind the UNCHANGED Nopales prim ABI. This module owns:

      - the PATH GRAMMAR (ltree labels, dots not slashes),
      - the reversible label codec that MUST match migrations/009_tree.sql
        [tree_enc_label] / [tree_uuid_label] exactly,
      - the node ENVELOPE (jsonb) constructors/parsers,
      - content hashing (sha256 over the canonical body).

    P1: pure helpers + DB read/verify helpers. No behavior change yet --
    the tree is written by the SQL shadow-mirror triggers (migration 009)
    and read here for validation. P3 flips prim/store reads to this
    module; P4 makes the tree the write-truth.

    Branch inheritance is NOT baked into the path. The override-walk is
    realized by trying a name under each branch prefix from self up the
    parent chain (first hit wins). Datum paths are USER-OWNED (not branch
    lenses): data.u.<user>.<type>.<id> + data.shared.<type>.<id> per
    [one-tree.branching-interaction] /data DECISION. *)


(* ---- label codec (MUST match 009_tree.sql) ---- *)

(** ltree labels admit [A-Za-z0-9_] only. Encode arbitrary text as
    base64url-ish of the raw bytes: '+'->'_', '/'->'-', strip '=', append
    the pad count (0/1/2). Empty -> "x". Mirrors tree_enc_label in SQL. *)
let enc_label (s : string) : string =
  if s = "" then "x"
  else begin
    let b64 = Base64.encode_string ~pad:true s in
    let len = String.length b64 in
    let pad =
      if len >= 2 && String.sub b64 (len - 2) 2 = "==" then 2
      else if len >= 1 && b64.[len - 1] = '=' then 1
      else 0
    in
    let core = if pad > 0 then String.sub b64 0 (len - pad) else b64 in
    let buf = Bytes.of_string core in
    for i = 0 to Bytes.length buf - 1 do
      match Bytes.get buf i with
      | '+' -> Bytes.set buf i '_'
      | '/' -> Bytes.set buf i '-'
      | c -> ignore c
    done;
    Bytes.unsafe_to_string buf ^ string_of_int pad
  end

(** Inverse of [enc_label]. Returns None on malformed input. *)
let dec_label (s : string) : string option =
  if s = "x" then Some ""
  else begin
    let len = String.length s in
    if len < 1 then None
    else begin
      let pad_digit = s.[len - 1] in
      if pad_digit < '0' || pad_digit > '2' then None
      else begin
        let pad = Char.code pad_digit - Char.code '0' in
        let core = String.sub s 0 (len - 1) in
        let buf = Bytes.of_string core in
        for i = 0 to Bytes.length buf - 1 do
          match Bytes.get buf i with
          | '_' -> Bytes.set buf i '+'
          | '-' -> Bytes.set buf i '/'
          | c -> ignore c
        done;
        let b64 = Bytes.unsafe_to_string buf ^ String.make pad '=' in
        match Base64.decode ~pad:true b64 with
```

```
        | Ok raw -> Some raw
        | Error _ -> None
      end
    end
  end

(** UUID -> ltree-safe label (strip dashes). Mirrors tree_uuid_label. *)
let uuid_label (u : string) : string =
  String.concat "" (String.split_on_char '-' u)

(* ---- path constructors ---- *)

let fn_path ~branch_id ~name =
  Printf.sprintf "b.%s.fn.%s" (uuid_label branch_id) (enc_label name)

let type_path ~branch_id ~name =
  Printf.sprintf "b.%s.type.%s" (uuid_label branch_id) (enc_label name)

let route_path ~branch_id ~route =
  Printf.sprintf "b.%s.route.%s" (uuid_label branch_id) (enc_label route)

let pkg_path ~branch_id ~name =
  Printf.sprintf "b.%s.pkg.%s" (uuid_label branch_id) (enc_label name)

(** Seed corpus types -- visible to every session under data.shared.*.
    The SERVER-side authority is the tree_is_seed_type / tree_seed_types
    SQL in migrations/016_seed_single_source.sql (the single place the
    list is defined on the DB). This OCaml array is the pure mirror needed
    by the DB-free `node-path` prim; verify_seed_types checks it against
    the SQL authority at boot so drift is caught automatically. *)
let seed_types = [ "bible-verse"; "bible-book"; "bible-interlinear";
     "strongs-entry"; "guestbook_entry" ]

let is_seed_type (type_name : string) : bool =
  List.mem type_name seed_types

(** Read the authoritative seed list from SQL (tree_seed_types). *)
let db_seed_types (conn : Store_db.conn) : string list =
  try
    let rows =
      Store_db.exec conn "SELECT t FROM unnest(tree_seed_types()) AS u(t)"
    in
    List.filter_map
      (function [ c ] -> Pgx.Value.to_string c | _ -> None)
      rows
  with exn ->
    Printf.eprintf "[store] tree_seed_types() query failed: %s\n%!"
      (Printexc.to_string exn);
    []

(** Verify the OCaml mirror matches the SQL authority. Non-fatal: logs a
    warning so an out-of-date Tree_store.seed_types is surfaced without
    taking the image down. Call once at boot (Store.connect succeeded). *)
let verify_seed_types (conn : Store_db.conn) : unit =
  let sorted = List.sort String.compare in
  let db = sorted (db_seed_types conn) in
  let oml = sorted seed_types in
  if db <> oml then
    Printf.eprintf
      "[store] WARNING: OCaml seed_types [%s] != DB tree_seed_types [%s];\n\
       %s    update image/lib/tree_store.ml seed_types to match
    migrations/016.\n%!"
      (String.concat "," oml)
      (String.concat "," db)
      (if db = [] then "      (DB returned nothing -- migrations not
    applied?)\n    " else "")

(** Datum path. Prefer [~user] for per-user rows; seed types always land
    under data.shared regardless of user. Without ~user, non-seed types
    use data.u.unknown -- writers must pass ~user.

    [~lens] opts into the versioned data lens namespace: the path becomes
    data.l.<lens>.<user>.<type>.<id> instead of data.u.<user>.<type>.<id>.
```

```
    A data lens is a deliberate fork of the user's live data that can
    diverge and merge back (one-tree.borg branching-interaction, 2026-08-23
    make). Seed types are never lensed -- data.shared is branch-free. *)
let datum_path ?user ?(lens : string option = None) ~type_name ~id () =
  let id_lab = uuid_label id in
  let ty_lab = enc_label type_name in
  if is_seed_type type_name then
    Printf.sprintf "data.shared.%s.%s" ty_lab id_lab
  else
    let u =
      match user with
      | Some s when s <> "" -> s
      | _ -> "unknown"
    in
    match lens with
    | Some l when l <> "" ->
        Printf.sprintf "data.l.%s.%s.%s.%s" (enc_label l) (enc_label u)
     ty_lab id_lab
    | _ ->
        Printf.sprintf "data.u.%s.%s.%s" (enc_label u) ty_lab id_lab

(* ---- content hashing ---- *)

let hash_body (canonical_body_json : string) : string =
  Auth_password.sha256_hex canonical_body_json

(* ---- node envelope ---- *)

type node = {
  id : string;
  path : string;
  kind : string;             (* fn | type | route | datum | folder | note |
     heading | link *)
  type_name : string option; (* datum: the Nopales type *)
  body : Yojson.Safe.t;      (* canonical body (AST / fields / route) *)
  author : string;
  doc : string option;       (* fn only *)
  hash : string;
  created_at : string;       (* row column; source of truth (not in
     payload) *)
  updated_at : string;       (* row column; advanced on every upsert *)
  rank : string option;      (* sibling-order: NULL = unordered (sorts
     last) *)
}

let envelope ~kind ~path ~type_ ~body ~author ~doc : Yojson.Safe.t =
  `Assoc [
    ("v", `Int 1);
    ("kind", `String kind);
    ("path", `String path);
    ("type", `String type_);
    ("body", body);
    ("author", `String author);
    ("doc", (match doc with Some d -> `String d | None -> `Null));
    ("hash", `String (hash_body (Yojson.Safe.to_string body)));
  ]

let node_of_row (row : Pgx.row) : node option =
  (* Expected column order: id, path, kind, type_name, payload, hash,
     author, created_at, updated_at, rank (see select_cols) *)
  match row with
  | [ id_c; path_c; kind_c; type_c; payload_c; hash_c; author_c;
      created_c; updated_c; rank_c ] ->
      (match Pgx.Value.to_string id_c,
             Pgx.Value.to_string path_c,
             Pgx.Value.to_string kind_c,
             Pgx.Value.to_string payload_c,
             Pgx.Value.to_string hash_c,
             Pgx.Value.to_string author_c,
             Pgx.Value.to_string created_c,
             Pgx.Value.to_string updated_c with
       | Some id, Some path, Some kind, Some payload_s, Some hash, Some
     author,
```

```
        Some created_at, Some updated_at ->
          (try
             let payload = Yojson.Safe.from_string payload_s in
             let body =
               match payload with
               | `Assoc kvs ->
                   (match List.assoc_opt "body" kvs with
                    | Some b -> b
                    | None -> `Null)
               | _ -> `Null
             in
             let doc =
               match payload with
               | `Assoc kvs ->
                   (match List.assoc_opt "doc" kvs with
                    | Some (`String d) -> Some d
                    | _ -> None)
               | _ -> None
             in
             Some { id; path; kind;
                    type_name = Pgx.Value.to_string type_c;
                    body; author; doc; hash;
                    created_at; updated_at;
                    rank = Pgx.Value.to_string rank_c }
           with Yojson.Json_error _ -> None)
       | _ -> None)
  | _ -> None

(* ---- write helpers (upsert/delete tree_nodes; the reverse trigger mirrors
   to the legacy tables in the same tx) ---- *)

let upsert_fn (conn : Store_db.conn) ~(branch_id : string) ~(name : string)
    ~(body_json : string) ~(author : string) ~(doc : string option)
    : unit =
  (* conn: Store_db.conn (tx-aware) *)
       let path = fn_path ~branch_id ~name in
      let hash = hash_body body_json in
      let sql =
        "INSERT INTO tree_nodes (path, kind, type_name, payload, hash,
    author, updated_at) " ^
        "VALUES ($1::ltree, 'fn', NULL, " ^
        "
    jsonb_build_object('v',1,'kind','fn','path',$1::text,'type','fn','body',$2::jsonb,
    " ^
        "    'author',$3::text,'doc',$4::text,'hash',$5::text), $5::text,
    $3::text, now()) " ^
        "ON CONFLICT (path) DO UPDATE SET payload=EXCLUDED.payload,
    hash=EXCLUDED.hash, " ^
        "  author=EXCLUDED.author, updated_at=now()" in
      let doc_param = match doc with Some d -> Pgx.Value.of_string d | None
    -> Pgx.Value.null in
      let _ =
        Io_lwt.Db.execute conn
          ~params:[ Pgx.Value.of_string path; Pgx.Value.of_string body_json;
                    Pgx.Value.of_string author; doc_param;
    Pgx.Value.of_string hash ]
          sql
      in
      ()

let upsert_type (conn : Store_db.conn) ~(branch_id : string) ~(name :
    string)
    ~(fields_json : string) ~(author : string) : unit =
  (* conn: Store_db.conn (tx-aware) *)
       let path = type_path ~branch_id ~name in
      let hash = hash_body fields_json in
      let sql =
        "INSERT INTO tree_nodes (path, kind, type_name, payload, hash,
    author, updated_at) " ^
        "VALUES ($1::ltree, 'type', $2::text, " ^
        "
    jsonb_build_object('v',1,'kind','type','path',$1::text,'type',$2::text,'body',$3::jsonb,
    " ^
```

```
272         "    'author',$4::text,'doc',NULL,'hash',$5::text), $5::text,
    $4::text, now()) " ^
273         "ON CONFLICT (path) DO UPDATE SET payload=EXCLUDED.payload,
    hash=EXCLUDED.hash, " ^
274         "  author=EXCLUDED.author, updated_at=now()" in
275       let _ =
276         Io_lwt.Db.execute conn
277           ~params:[ Pgx.Value.of_string path; Pgx.Value.of_string name;
278                     Pgx.Value.of_string fields_json; Pgx.Value.of_string
    author;
279                     Pgx.Value.of_string hash ]
280           sql
281       in
282       ()
283
284 let upsert_route (conn : Store_db.conn) ~(branch_id : string) ~(route :
    string)
285     ~(handler : string) ~(visibility : string) : unit =
286   (* conn: Store_db.conn (tx-aware) *)
287         let path = route_path ~branch_id ~route in
288       let body_json =
289         Yojson.Safe.to_string
290           (`Assoc [ ("route", `String route); ("handler", `String handler);
291                     ("visibility", `String visibility) ]) in
292       let hash = hash_body body_json in
293       let sql =
294         "INSERT INTO tree_nodes (path, kind, type_name, payload, hash,
    author, updated_at) " ^
295         "VALUES ($1::ltree, 'route', NULL, " ^
296         "
    jsonb_build_object('v',1,'kind','route','path',$1::text,'type','route','body',$2::jsonb,
    " ^
297         "    'author','system','doc',NULL,'hash',$3::text), $3::text,
    'system', now()) " ^
298         "ON CONFLICT (path) DO UPDATE SET payload=EXCLUDED.payload,
    hash=EXCLUDED.hash, " ^
299         "  updated_at=now()" in
300       let _ =
301         Io_lwt.Db.execute conn
302           ~params:[ Pgx.Value.of_string path; Pgx.Value.of_string body_json;
303                     Pgx.Value.of_string hash ]
304           sql
305       in
306       ()
307
308 let delete_fn (conn : Store_db.conn) ~(branch_id : string) ~(name : string)
    : bool =
309   (* conn: Store_db.conn (tx-aware) *)
310         let sql = "DELETE FROM tree_nodes WHERE path = $1::ltree RETURNING
    path::text" in
311       let rows =
312         Io_lwt.Db.execute conn ~params:[ Pgx.Value.of_string (fn_path
    ~branch_id ~name) ] sql
313       in
314       (rows <> [])
315
316 let insert_datum (conn : Store_db.conn) ~(type_name : string) ~(fields_json
    : string)
317     ~(author : string) : string option =
318   (* id generated by PG; path via tree_datum_path (013) so seed vs user
319      roots stay consistent with the SQL mirror. *)
320   let hash = hash_body fields_json in
321   let sql =
322     "WITH nid AS (SELECT gen_random_uuid() AS id) " ^
323     "INSERT INTO tree_nodes (path, kind, type_name, payload, hash, author,
    updated_at) " ^
324 "SELECT tree_datum_path($1::text, nid.id, $3::text, ''), 'datum', $1::text,
    " ^
325     "  jsonb_build_object('v',1,'kind','datum', " ^
326 "    'path', tree_datum_path($1::text, nid.id, $3::text, '')::text, " ^
327     "
    'type',$1::text,'body',$2::jsonb,'author',$3::text,'doc',NULL,'hash',$4::text),
    " ^
```

```
    "  $4::text, $3::text, now() " ^
    "FROM nid RETURNING tree_datum_id(path)::text" in
  let rows =
    Io_lwt.Db.execute conn
      ~params:[ Pgx.Value.of_string type_name; Pgx.Value.of_string
     fields_json;
                Pgx.Value.of_string author; Pgx.Value.of_string hash ]
      sql
  in
  (match rows with
   | [ [ id_c ] ] -> Pgx.Value.to_string id_c
   | _ -> None)

let update_datum (conn : Store_db.conn) ~(type_name : string) ~(id : string)
    ~(fields_json : string) : string option =
  (* Match by datum id (last path label) so old and new path shapes both
     work,
       but constrain to LIVE datum paths (exclude the data.l namespace). A
     data lens fork
     shadows the same (type,id) under data.l.<lens>.<user>; without this
     guard
     the UPDATE hits both live and lens nodes, RETURNING yields 2 rows, the
     single-row match returns None (prim reports failure), and the forward
     mirror trigger (tree_mirror_data_rows) purges the lens copy. With the
     guard the lens keeps its snapshot body and survives until
     merge/abandon.
     Take the first RETURNING row as a safety net against stale live paths.
     *)
  let hash = hash_body fields_json in
  let sql =
    "UPDATE tree_nodes SET " ^
    "  payload =
     jsonb_set(jsonb_set(payload,'{body}',$2::jsonb),'{hash}',to_jsonb($3::text)),
     " ^
    "  hash = $3::text, updated_at = now() " ^
    "WHERE kind='datum' AND tree_datum_id(path) = $1::uuid " ^
    "  AND (type_name = $4 OR type_name IS NULL) " ^
    "  AND NOT (path <@ 'data.l'::ltree) " ^
    "RETURNING tree_datum_id(path)::text" in
  let rows =
    Io_lwt.Db.execute conn
      ~params:[ Pgx.Value.of_string id; Pgx.Value.of_string fields_json;
                Pgx.Value.of_string hash; Pgx.Value.of_string type_name ]
      sql
  in
  (match rows with
   | [ id_c ] :: _ -> Pgx.Value.to_string id_c
   | _ -> None)

let delete_datum (conn : Store_db.conn) ~(type_name : string) ~(id :
    string) : string option =
  (* Constrain to LIVE datum paths (exclude the data.l namespace) for the
     same reason as
     update_datum: don't delete a lens copy that shadows the live row. *)
  let sql =
    "DELETE FROM tree_nodes WHERE kind='datum' AND tree_datum_id(path) =
     $1::uuid " ^
    "  AND (type_name = $2 OR type_name IS NULL) " ^
    "  AND NOT (path <@ 'data.l'::ltree) " ^
    "RETURNING tree_datum_id(path)::text" in
  let rows =
    Io_lwt.Db.execute conn
      ~params:[ Pgx.Value.of_string id; Pgx.Value.of_string type_name ]
      sql
  in
  (match rows with
   | [ id_c ] :: _ -> Pgx.Value.to_string id_c
   | _ -> None)

(** Upsert a datum node at [path] (kind="datum"). When the path's id
    label is a placeholder (empty, "_", or "new"), generate a fresh uuid
    via gen_random_uuid() and build the path with tree_datum_path the
    way insert_datum does -- so a caller can create a new datum node by
```

```
    path without pre-allocating an id. Otherwise the caller-supplied
    full path is used (data.u.<user>.<type>.<id> or data.shared.<type>.<id>).
    Typed-boundary validation runs in the caller (crud.ml check_type_fields);
    this function is the SQL write. ON CONFLICT (path) DO UPDATE the
    body/hash/author so the same prim upserts and updates. The reverse
    mirror (tree_from_nodes) syncs data_rows. Returns Ok path | Error msg. *)
let upsert_datum (conn : Store_db.conn) ~(path : string) ~(type_name : string)
    ~(body_json : string) ~(author : string) ~(rank : string option)
    : (string, string) result =
  let last_label =
    let parts = String.split_on_char '.' path in
    match List.rev parts with [] -> "" | l :: _ -> l
  in
  let is_placeholder =
    last_label = "" || last_label = "_" || last_label = "new"
  in
  let hash = hash_body body_json in
  let run_sql ~params sql =
    try
      let rows = Io_lwt.Db.execute conn ~params sql in
      (match rows with
       | [ [ p ] ] ->
           (match Pgx.Value.to_string p with
            | Some s -> Ok s
            | None -> Error "upsert_datum: no path returned")
       | _ -> Error "upsert_datum: failed")
    with exn -> Error ("upsert_datum: " ^ Printexc.to_string exn)
  in
  if is_placeholder then
    (* Generate uuid + path via tree_datum_path(type_name, id, author).
       The generated path is always ltree-safe; no valid_path check on
       the caller's placeholder path. *)
    if type_name = "" then Error "upsert_datum: empty type_name"
    else
      let sql =
        "WITH nid AS (SELECT gen_random_uuid() AS id) " ^
        "INSERT INTO tree_nodes (path, kind, type_name, payload, hash,
    author, rank, updated_at) " ^
"SELECT tree_datum_path($1::text, nid.id, $3::text, ''), 'datum', $1::text,
    " ^
        "  jsonb_build_object('v',1,'kind','datum', " ^
"    'path', tree_datum_path($1::text, nid.id, $3::text, '')::text, " ^
        "
    'type',$1::text,'body',$2::jsonb,'author',$3::text,'doc',NULL,'hash',$4::text),
    " ^
        "  $4::text, $3::text, $5::text, now() " ^
        "FROM nid ON CONFLICT (path) DO UPDATE SET " ^
        "  payload = EXCLUDED.payload, hash = EXCLUDED.hash, " ^
        "  author = EXCLUDED.author, " ^
        "  rank = COALESCE(EXCLUDED.rank, tree_nodes.rank), " ^
        "  updated_at = now() " ^
        "RETURNING path::text"
      in
      run_sql ~params:
        [ Pgx.Value.of_string type_name; Pgx.Value.of_string body_json;
          Pgx.Value.of_string author; Pgx.Value.of_string hash;
          (match rank with Some r -> Pgx.Value.of_string r | None ->
    Pgx.Value.null) ]
        sql
  else if type_name = "" then Error "upsert_datum: empty type_name"
  else
    let sql =
      "INSERT INTO tree_nodes (path, kind, type_name, payload, hash,
    author, rank, updated_at) " ^
      "VALUES ($1::ltree, 'datum', $2::text, " ^
      "
    jsonb_build_object('v',1,'kind','datum','path',$1::text,'type',$2::text,
    " ^
```

```
      "    'body',$3::jsonb,'author',$4::text,'doc',NULL,'hash',$5::text),
     " ^
      "  $5::text, $4::text, $6::text, now()) " ^
      "ON CONFLICT (path) DO UPDATE SET " ^
      "  payload = EXCLUDED.payload, hash = EXCLUDED.hash, " ^
      "  author = EXCLUDED.author, " ^
      "  rank = COALESCE(EXCLUDED.rank, tree_nodes.rank), " ^
      "  updated_at = now() " ^
      "RETURNING path::text"
    in
    run_sql ~params:
      [ Pgx.Value.of_string path; Pgx.Value.of_string type_name;
        Pgx.Value.of_string body_json; Pgx.Value.of_string author;
        Pgx.Value.of_string hash;
        (match rank with Some r -> Pgx.Value.of_string r | None ->
     Pgx.Value.null) ]
      sql

(** Validate [fields_json] against a declared type's field names.
    Rules (one-tree.typed-nodes / object-model.type-system v1):
      - fields_json must be a JSON object
      - every declared field must be present as a key (value may be null/"")
      - extra keys ARE allowed (additive tolerance;
     branching.schema-authority)
    Returns Ok () or Error message. Pure -- no DB. *)
let validate_fields ~(declared : string list) (fields_json : string)
  : (unit, string) result =
  if declared = [] then Ok ()
  else
    try
      match Yojson.Safe.from_string fields_json with
      | `Assoc kvs ->
          let missing =
            List.filter (fun f -> not (List.mem_assoc f kvs)) declared
          in
          if missing = [] then Ok ()
          else
            Error
              ("missing fields: " ^ String.concat ", " missing
               ^ " (declared: " ^ String.concat ", " declared ^ ")")
      | _ -> Error "fields must be a JSON object"
    with Yojson.Json_error msg -> Error ("invalid JSON: " ^ msg)

(** Backlinks: datum nodes whose fields JSON has any value equal to [needle]
    (typically a uuid id or a tree path). Uses jsonb_each_text over body.
    Caps at [limit] (default 200). *)
let backlinks (conn : Store_db.conn) ~(needle : string) ?(limit : int = 200)
    () : string list =
  let sql =
    "SELECT n.path::text FROM tree_nodes n, " ^
    "LATERAL jsonb_each_text(COALESCE(n.payload->'body', '{}'::jsonb)) e " ^
    "WHERE n.kind = 'datum' AND e.value = $1::text " ^
    "ORDER BY n.path::text ASC LIMIT $2" in
  let rows =
    Io_lwt.Db.execute conn
      ~params:[ Pgx.Value.of_string needle; Pgx.Value.of_int limit ]
      sql
  in
  (List.filter_map
       (function
         | [ c ] -> Pgx.Value.to_string c
         | _ -> None)
       rows)

(** Recompute tree_branches.head_hash for [branch_id] as the sha256 of the
    sorted concatenation of own-node hashes under b.<uuidhex>. Empty lens ?
    empty string. Used after fork (empty) and after promote/merge batches.
     *)
let recompute_head_hash (conn : Store_db.conn) ~(branch_id : string) : unit
    =
  let root = "b." ^ uuid_label branch_id in
  let sql =
    "UPDATE tree_branches SET head_hash = COALESCE(( " ^
```

```
    "  SELECT encode(digest(string_agg(hash, '' ORDER BY path::text),
     'sha256'), 'hex') " ^
    "  FROM tree_nodes WHERE path <@ $1::ltree AND nlevel(path) > 1 " ^
    "), '') WHERE branch_id = $2::uuid" in
  let _ =
    Io_lwt.Db.execute conn
      ~params:[ Pgx.Value.of_string root; Pgx.Value.of_string branch_id ]
      sql
  in
  ()

(* ---- DB helpers (read/verify; mirror writes happen in SQL triggers) ----
    *)

let select_cols =
  "id::text, path::text, kind, type_name, payload::text, hash, author, "
  ^ "created_at::text, updated_at::text, rank"

(** Fetch one node by exact path. *)
let get_by_path (conn : Store_db.conn) ~(path : string) : node option =
  (* conn: Store_db.conn (tx-aware) *)
        let sql =
        "SELECT " ^ select_cols ^ " FROM tree_nodes WHERE path = $1::ltree"
     in
      let rows =
        Io_lwt.Db.execute conn ~params:[ Pgx.Value.of_string path ] sql
      in
      (match rows with
       | row :: _ -> (node_of_row row)
       | [] -> None)

(** A versioned-leaf row (tree-time: one version row per (path, gen) write).
    Field names are v_-prefixed to avoid clashing with the `node` record. *)
type version = {
  v_gen : int64 option;
  v_path : string;
  v_kind : string;
  v_type_name : string option;
  v_body : Yojson.Safe.t;
  v_hash : string option;
  v_author : string option;
  v_created_at : string option;
  v_tombstone : bool;
  v_rank : string option;      (* sibling-order: rank in force at that gen
     *)
}

let version_of_row (row : Pgx.row) : version option =
  match row with
  | [ gen_c; path_c; kind_c; type_c; body_c; hash_c; author_c;
      created_c; tomb_c; rank_c ] ->
      let body =
        match Pgx.Value.to_string body_c with
        | Some s -> (try Some (Yojson.Safe.from_string s) with _ -> None)
        | None -> None
      in
      let tomb =
        match Pgx.Value.to_string tomb_c with
        | Some t -> t = "true"
        | None -> false
      in
      (match body with
       | None -> None
       | Some body ->
           Some
             { v_gen = (match Pgx.Value.to_int64 gen_c with Some g -> Some
     g | None -> None);
               v_path = (match Pgx.Value.to_string path_c with Some p -> p
     | None -> "");
               v_kind = (match Pgx.Value.to_string kind_c with Some k -> k
     | None -> "");
               v_type_name = Pgx.Value.to_string type_c;
               v_body = body;
```

```
584               v_hash = Pgx.Value.to_string hash_c;
585               v_author = Pgx.Value.to_string author_c;
586               v_created_at = Pgx.Value.to_string created_c;
587               v_tombstone = tomb;
588               v_rank = Pgx.Value.to_string rank_c })
589   | _ -> None
590
591 (** Newest versioned row at head (or as-of [at_gen]) for [path], matching
592     migration 018's tree_versions_head semantics. None = path never
        versioned.
593     [path] encodes the branch for code paths (b + branch + suffix) and is
594     globally unique, so a path-only lookup serves both code and data classes
595     without a branch_id argument. This is the versioned read surface 5.1's
596     shadow test compares against the live tree_nodes head, and 5.2
597     reads-as-of-gen drives the gen filter through. *)
598 let versioned_head (conn : Store_db.conn) ~(path : string)
599     ?(at_gen : int64 option) () : version option =
600   let sql =
601     "SELECT v.gen, v.path::text, v.kind, v.type_name, v.body::text, v.hash,
        "
602     ^ "v.author, v.created_at::text, v.body->>'_tombstone'::text, v.rank "
603     ^ "FROM tree_versions v WHERE v.path = $1::ltree "
604     ^ "AND ($2::bigint IS NULL OR v.gen <= $2::bigint) "
605     ^ "ORDER BY v.gen DESC LIMIT 1"
606   in
607   let rows =
608     Io_lwt.Db.execute conn
609       ~params:[ Pgx.Value.of_string path
610               ; (match at_gen with Some g -> Pgx.Value.of_int64 g | None
       -> Pgx.Value.null) ]
611       sql
612   in
613   (match rows with
614    | row :: _ -> (version_of_row row)
615    | [] -> None)
616
617 (** All version rows for [path], oldest gen first (append chain). *)
618 let versioned_history (conn : Store_db.conn) ~(path : string) () : version
        list =
619   let sql =
620     "SELECT v.gen, v.path::text, v.kind, v.type_name, v.body::text, v.hash,
        "
621     ^ "v.author, v.created_at::text, v.body->>'_tombstone'::text, v.rank "
622     ^ "FROM tree_versions v WHERE v.path = $1::ltree ORDER BY v.gen ASC"
623   in
624   let rows =
625     Io_lwt.Db.execute conn ~params:[ Pgx.Value.of_string path ] sql
626   in
627   (List.filter_map version_of_row rows)
628
629
630 (** Subtree listing: all nodes under a path prefix (ltree descendant). *)
631 let list_subtree (conn : Store_db.conn) ~(prefix : string)
632     ?(limit : int = 1000) () : node list =
633   (* conn: Store_db.conn (tx-aware) *)
634         let sql =
635         "SELECT " ^ select_cols ^ " FROM tree_nodes "
636         ^ "WHERE path <@ $1::ltree "
637         (* sibling-order: ranked siblings first (byte-order rank), then
638            unranked, path-ordered among themselves. NULL rank = unordered.
       *)
639         ^ "ORDER BY rank COLLATE \"C\" NULLS LAST, path::text ASC LIMIT $2"
640       in
641        let rows =
642          Io_lwt.Db.execute conn
643            ~params:[ Pgx.Value.of_string prefix; Pgx.Value.of_int limit ] sql
644        in
645        (List.filter_map node_of_row rows)
646
647 (** All datum nodes of a Nopales type (the (rows 't) backing in P3). *)
648 let list_data_by_type (conn : Store_db.conn) ~(type_name : string)
649     ?(limit : int = 100) () : node list =
650   (* conn: Store_db.conn (tx-aware) *)
```

```
650         let sql =
651         "SELECT " ^ select_cols ^ " FROM tree_nodes " ^
652         "WHERE kind = 'datum' AND type_name = $1 " ^
653         "ORDER BY created_at ASC LIMIT $2" in
654       let rows =
655         Io_lwt.Db.execute conn
656           ~params:[ Pgx.Value.of_string type_name; Pgx.Value.of_int limit ]
      sql
657       in
658       (List.filter_map node_of_row rows)
659
660 (** Branch-chain resolution: try [name] under self, then each ancestor.
661     Returns the first hit (closest-ancestor-wins = the override rule).
662     [branch_chain] is ordered self-first (depth ASC), as store.branch
663     chain produces. *)
664 let rec resolve_in_chain (conn : Store_db.conn) ~(mk_path : string ->
      string)
665     (chain : string list) : node option =
666   match chain with
667   | [] -> None
668   | branch_id :: rest ->
669       let n = get_by_path conn ~path:(mk_path branch_id) in
670       (match n with
671        | Some _ -> n
672        | None -> resolve_in_chain conn ~mk_path rest)
673
674 (** Lens prefixing for a RELATIVE logical path. The store-tree path grammar
675     uses b.<branch_id>.<rest> for branch-lensed code nodes (fn/type/route/
676     pkg). A lens-aware read takes a logical suffix of the form
677     <kind>.<name> (e.g. "fn.notes/doc/get", "route./issues", "pkg.todo")
678     and materializes the full path under a given branch id by ENCODING the
679     name segment with the label codec (same as fn_path/type_path/route_path/
680     pkg_path) -- the kind label itself is already ltree-safe. Returns None
681     for an empty/malformed suffix (no kind). *)
682 let lens_suffix_path ~(branch_id : string) (suffix : string) : string
      option =
683   if suffix = "" then None
684   else
685     let kind, nl =
686       match String.index_opt suffix '.' with
687       | Some i when i > 0 ->
688           (String.sub suffix 0 i, String.sub suffix (i + 1) (String.length
      suffix - i - 1))
689       | _ -> (suffix, "")
690     in
691     if nl = "" then None
692     else Some ("b." ^ uuid_label branch_id ^ "." ^ kind ^ "." ^ enc_label
      nl)
693
694 (** Lens-aware read of one node by a RELATIVE logical path, resolved across
695     [chain] (self-first / closest-wins -- the override rule). Tries the
696     suffix under each branch id in order; first hit wins. Backed by
697     get_by_path so created_at/updated_at are present via node_to_json.
698     Returns the closest node or None if no branch in the chain has it. *)
699 let lens_get_in_chain (conn : Store_db.conn) ~(suffix : string)
700     (chain : string list) : node option =
701   let mk_path branch_id =
702     match lens_suffix_path ~branch_id suffix with
703     | Some p -> p
704     | None -> "" (* empty suffix can never match a real ltree node *)
705   in
706   resolve_in_chain conn ~mk_path chain
707
708 (** Reads-as-of-gen across the branch lens chain: like lens_get_in_chain,
      but
709     returns the VERSIONED leaf (node body at head or as-of [at_gen]) for the
710     EFFECTIVE path -- the closest-ancestor-wins branch that carries the
711     suffix. Spec tree-time.reads-as-of-gen: a gen-as-of read "crosses the
712     same branch lens rules as head reads". None = no branch in the chain
713     carries the suffix at all; the returned version may still be a TOMBSTONE
714     (existed-then-deleted), which the caller distinguishes via v_tombstone.
      *)
715 let rec lens_versioned_in_chain (conn : Store_db.conn)
```

```
    ~(at_gen : int64 option) ~(mk_path : string -> string)
    (chain : string list) : version option =
  let read_opt path =
    match at_gen with
    | Some g -> versioned_head conn ~path ~at_gen:g ()
    | None -> versioned_head conn ~path ()
  in
  match chain with
  | [] -> None
  | branch_id :: rest ->
      let p = mk_path branch_id in
      if p = "" then lens_versioned_in_chain conn ~at_gen ~mk_path rest
      else
        (match read_opt p with
         | Some v -> Some v
         | None -> lens_versioned_in_chain conn ~at_gen ~mk_path rest)

(** Consistency check against the SQL tree_verify() function.
    Returns (kind, table_count, tree_count, in_sync) rows. *)
let verify (conn : Store_db.conn) : (string * int * int * bool) list =
  (* conn: Store_db.conn (tx-aware) *)
        let rows = Io_lwt.Db.execute conn "SELECT * FROM tree_verify()" in
      (List.filter_map
           (fun row ->
             match row with
             | [ k; tc; xc; sy ] ->
                 (match Pgx.Value.to_string k,
                        Pgx.Value.to_int tc,
                        Pgx.Value.to_int xc,
                        Pgx.Value.to_bool sy with
                  | Some k, Some tc, Some xc, Some sy -> Some (k, tc, xc,
    sy)
                  | _ -> None)
             | _ -> None)
           rows)

(* ---- hierarchical notes helpers (notes.storage / notes.tree-cutover K0)
    ----

   folder | note | heading are first-class tree kinds. They are NOT
    kind=datum
   and must never use data.<type>.<uuid> paths -- reverse-mirror only
    handles
   fn/type/route/datum. Path labels are ltree-safe: [A-Za-z0-9_] only.
   Readable slug policy: path_label lowercases and maps non [a-z0-9] to _.
   Original slug (with hyphens) lives in payload body when needed. *)

(** Sanitize one path segment for ltree. Empty -> "x". *)
let path_label (s : string) : string =
  let b = Buffer.create (String.length s) in
  let last_us = ref false in
  String.iter
    (fun c ->
      let c =
        if c >= 'A' && c <= 'Z' then Char.chr (Char.code c + 32) else c
      in
      if (c >= 'a' && c <= 'z') || (c >= '0' && c <= '9') then begin
        Buffer.add_char b c;
        last_us := false
      end else if not !last_us then begin
        Buffer.add_char b '_';
        last_us := true
      end)
    s;
  let out = Buffer.contents b in
  let out =
    let len = String.length out in
    let start = if len > 0 && out.[0] = '_' then 1 else 0 in
    let stop =
      if len > start && out.[len - 1] = '_' then len - 1 else len
    in
    if stop <= start then "" else String.sub out start (stop - start)
  in
```

```
  if out = "" then "x" else out

let allowed_hier_kinds = [ "folder"; "note"; "heading" ]

let is_hier_kind (k : string) : bool =
  List.mem k allowed_hier_kinds

(** Validate a full dotted path: every label must be ltree-safe. *)
let valid_path (path : string) : bool =
  path <> ""
  &&
  List.for_all
    (fun lab ->
      lab <> ""
      && String.for_all
           (fun c ->
             (c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z')
             || (c >= '0' && c <= '9') || c = '_')
           lab)
    (String.split_on_char '.' path)

let garden_root ~user =
  Printf.sprintf "data.note.u.%s" (path_label user)

(* ---- path algebra (one-tree.tree-path-prims; pure, no DB) ----

   Kernel path constructors/accessors over the ltree textual form -- the
   ONE canonical set, replacing per-package string hacks (notes/tree/*,
   todo/tree/*, tree/breadcrumb-parts). Label semantics for APPEND are
   identical to [path_label] for labels that need sanitizing; labels that
   are ALREADY valid ltree labels pass through untouched, so
   (tree/child (tree/parent p) (tree/leaf p)) = p for every stored path
   (including uppercase base64url-encoded fn/type/route labels, which
   [path_label] alone would lowercase-mangle). Invalid paths raise
   Invalid_argument -- the prim layer converts that to a Prim_error;
   nothing here guesses. *)

(** True when [s] is already an ltree-safe label ([A-Za-z0-9_]+). *)
let is_ltree_label (s : string) : bool =
  s <> ""
  && String.for_all
       (fun c ->
         (c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z')
         || (c >= '0' && c <= '9') || c = '_')
       s

(** Labels of a path; raises Invalid_argument on an invalid path. *)
let path_labels_exn (path : string) : string list =
  if not (valid_path path) then invalid_arg ("invalid path: " ^ path)
  else String.split_on_char '.' path

(** Strict parent path; None at a root (single-label) path. *)
let path_parent (path : string) : string option =
  match path_labels_exn path with
  | [ _ ] -> None
  | labels ->
      Some (String.concat "." (List.rev (List.tl (List.rev labels))))

(** Final label. *)
let path_leaf (path : string) : string =
  match List.rev (path_labels_exn path) with
  | last :: _ -> last
  | [] -> invalid_arg "invalid path: empty" (* unreachable *)

(** Every label, root-first. *)
let path_segments (path : string) : string list = path_labels_exn path

(** Append [label] to a valid path. A label that is already an ltree
    label is used verbatim; anything else is sanitized via [path_label]
    (identical encoding to tree/path-label). *)
let path_child (path : string) (label : string) : string =
  let labels = path_labels_exn path in
  let lab = if is_ltree_label label then label else path_label label in
```

```
  String.concat "." (labels @ [ lab ])

(** Every STRICT prefix, root-first: "a.b.c" -> ["a"; "a.b"]. A root
    path has no strict prefixes ([]). *)
let path_ancestors (path : string) : string list =
  let labels = path_labels_exn path in
  let rec go acc prefix = function
    | [] -> List.rev acc
    | [ _ ] -> List.rev acc (* full path excluded: strict prefixes only *)
    | x :: rest ->
        let prefix = if prefix = "" then x else prefix ^ "." ^ x in
        go (prefix :: acc) prefix rest
  in
  go [] "" labels

(* ---- sibling-order rank (one-tree.sibling-order) ----

   Fractional indexing over base-36 [0-9a-z], compared as raw bytes
   (ORDER BY rank COLLATE "C") -- the same alphabet byte order in every
   locale. A rank is any nonempty [0-9a-z]+ string (the spec's examples
   "a0", "a1", "a1V" included); allocated ranks additionally never end
   in '0', which keeps byte order == the fractional-value order for
   everything the allocator produces. Allocation is midpoint-between-
   neighbours; when a generated rank would exceed [max_rank_len] -- or a
   degenerate value-aliased pair leaves no room (b = a ^ "0"*k) -- the
   whole sibling set is rebalanced onto fresh fixed-width ranks.
   Write-path detail -- not visible in the prim ABI. *)

let max_rank_len = 12

let rank_alphabet = "0123456789abcdefghijklmnopqrstuvwxyz"

(** Rank check: nonempty [0-9a-z]+. *)
let rank_valid (s : string) : bool =
  let n = String.length s in
  n > 0
  && String.for_all
       (fun c -> (c >= '0' && c <= '9') || (c >= 'a' && c <= 'z'))
       s

let rank_digit_of (c : char) : int =
  if c >= '0' && c <= '9' then Char.code c - Char.code '0'
  else if c >= 'a' && c <= 'z' then Char.code c - Char.code 'a' + 10
  else invalid_arg ("rank: digit out of alphabet: " ^ String.make 1 c)

let rank_char_of (d : int) : char = rank_alphabet.[d]

(** Strict midpoint between distinct ranks a < b (byte order). The result
   is strictly between them (and canonical). Returns "" only for the
   degenerate value-aliased pair (b = a ^ "0"*k): no rank exists between,
   the caller must rebalance the sibling set. *)
let rank_midpoint (a : string) (b : string) : string =
  if not (rank_valid a && rank_valid b) then
    invalid_arg "rank_midpoint: non-canonical rank";
  if String.compare a b >= 0 then invalid_arg "rank_midpoint: need a < b";
  let len_a = String.length a and len_b = String.length b in
  let exception Degenerate in
  let buf = Buffer.create (max len_a len_b + 3) in
  let rec go i =
    if i >= len_a then begin
      (* a is a byte-prefix of b: emit a canonical Y strictly below b's
         remainder R. R = leading '0's then a digit d > '0', so: Y = those
         '0's followed by pred(d) -- or "01" when d = 1 (pred '0' may not
         terminate a rank; "0"+anything is still < "1..."). When R is ALL
         zeros (b = a ^ "0"*k, value-aliased with a) no rank exists
         between the two: signal the caller to rebalance. *)
      let rec zeros j = if j < len_b && b.[j] = '0' then zeros (j + 1) else
     j in
      let j = zeros i in
      if j >= len_b then raise Degenerate;
      let d = rank_digit_of b.[j] in
      for _k = i to j - 1 do Buffer.add_char buf '0' done;
      if d = 1 then Buffer.add_string buf "01"
```

```
      else Buffer.add_char buf (rank_char_of (d - 1))
    end
    else if i >= len_b then
      (* b is a byte-prefix of a -> byte order a > b: excluded above. *)
      invalid_arg "rank_midpoint: b prefix of a"
    else begin
      let ca = a.[i] and cb = b.[i] in
      if ca = cb then begin
        Buffer.add_char buf ca;
        go (i + 1)
      end
      else if rank_digit_of cb - rank_digit_of ca >= 2 then
        (* room at the first differing position: one midpoint digit *)
        Buffer.add_char buf
          (rank_char_of
             (rank_digit_of ca + (rank_digit_of cb - rank_digit_of ca) / 2))
      else begin
        (* adjacent digits: take ca, then any suffix past a's remainder
           ('h' terminates the result canonically) *)
        Buffer.add_char buf ca;
        Buffer.add_string buf (String.sub a (i + 1) (len_a - i - 1));
        Buffer.add_char buf 'h'
      end
    end
  in
  match go 0 with
  | exception Degenerate -> ""
  | () -> Buffer.contents buf

(** Rank strictly below every existing rank (before-first). [first] is
   the smallest ranked sibling's rank ("" = none ranked yet). None means
   the caller must rebalance the sibling set first. *)
let rank_before (first : string) : string option =
  if first = "" then Some "a"
  else if String.compare first "0" <= 0 then None (* "0" is the floor *)
  else Some (rank_midpoint "0" first)

(** Rank strictly above every existing rank (append / after-last).
   [last] is the largest ranked sibling's rank ("" = none ranked yet). *)
let rank_after (last : string) : string =
  if last = "" then "a"
  else
    (try rank_midpoint last "zzzzzzzzzzzz"
     with Invalid_argument _ -> last ^ "h")

(** Fresh fixed-width ranks for a sibling set of [n]: base-36 (i+1)
   zero-filled to a common width (fixed width orders correctly as plain
   text). Starts at 1 so the "0" floor sentinel is never allocated. *)
let rank_rebalance (n : int) : string list =
  let rec base36 x acc =
    if x = 0 then acc
    else base36 (x / 36) (rank_char_of (x mod 36) :: acc)
  in
  let digits_of x =
    match base36 x [] with [] -> "0" | l -> String.concat "" (List.map
     (String.make 1) l)
  in
  let width = max 1 (String.length (digits_of (max 1 n))) in
  let zfill i =
    let ds = digits_of i in
    let pad = String.make (width - String.length ds) '0' in
    pad ^ ds
  in
  List.init (max 0 n) (fun i -> zfill (i + 1))


let node_to_json (n : node) : string =
  let body = n.body in
  Yojson.Safe.to_string
    (`Assoc
       [ ("id", `String n.id)
       ; ("path", `String n.path)
       ; ("kind", `String n.kind)
```

```
       ; ( "type"
         , match n.type_name with Some t -> `String t | None -> `String
     n.kind )
       ; ("body", body)
       ; ("author", `String n.author)
       ; ("hash", `String n.hash)
       ; ("created_at", `String n.created_at)
       ; ("updated_at", `String n.updated_at)
       ; ("rank", match n.rank with Some r -> `String r | None -> `Null)
       ])

(** Render a [version] row as the JSON the reads-as-of-gen prims return.
    Mirrors node_to_json's field vocabulary (id/path/kind/type/body/author/
    hash/created_at) so a gen-as-of read has the SAME surface as a head
    tree/get, plus the [gen] coordinate the read resolved at and a
    [tombstone] flag. id/path/kind/type/author/hash are pulled from the
    version envelope body when present (it is the 019 trigger's
    {v,kind,path,type,body,author,doc,hash} node envelope); created_at.
    None values become JSON null. A tombstone row is still returned (its
    body is the reserved marker) so callers can distinguish
    "never existed" (None, prim returns nil) from "existed but deleted by
    gen G" (present row, tombstone=true). *)
let version_to_json (v : version) : string =
  let assoc = match v.v_body with `Assoc a -> a | _ -> [] in
  let get k (d : Yojson.Safe.t) =
    match List.assoc_opt k assoc with Some x -> x | None -> d
  in
  let os k d =
    match List.assoc_opt k assoc with
    | Some (`String s) -> `String s | Some x -> x | None -> d
  in
  let gen =
    match v.v_gen with Some g -> `String (Int64.to_string g) | None -> `Null
  in
  Yojson.Safe.to_string
    (`Assoc
       [ ("id", os "id" gen)
       ; ("path", os "path" (`String v.v_path))
       ; ("kind", os "kind" (`String v.v_kind))
       ; ("type", os "type" (`Null))
       ; ("body", get "body" (`Null))
       ; ("author", os "author" (`Null))
       ; ("hash", os "hash" (`Null))
       ; ("created_at",
          match v.v_created_at with Some c -> `String c | None -> `Null)
       ; ("gen", gen)
       ; ("tombstone", `Bool v.v_tombstone)
       ; ("rank",
          match v.v_rank with Some r -> `String r | None -> `Null)
       ])

(** Convert a versioned-leaf row into a [node] so the gen-as-of surface can
    reuse tree/get's node JSON shape when the caller passes an explicit
     :gen.
    The envelope body carries {v,kind,path,type,body,author,doc,hash}; rows
    that predate the version-era or are TOMBSTONEs have no live body and map
    to None (so the read surfaces as ?missing at that gen" the same way a
    head get of an absent path does). *)
let version_of_leaf (v : version) : node option =
  let assoc = match v.v_body with `Assoc a -> a | _ -> [] in
  let os k d =
    match List.assoc_opt k assoc with
    | Some (`String s) -> s | Some _ -> d | None -> d
  in
  let body =
    match List.assoc_opt "body" assoc with
    | Some (`Assoc _ as b) -> b
    | Some b -> b
    | None -> `Assoc []
  in
  if v.v_tombstone then None
  else if body = `Assoc [] then None
  else
```

```
    Some
      { id = os "id" v.v_path
      ; path = os "path" v.v_path
      ; kind = os "kind" v.v_kind
      ; type_name = v.v_type_name
      ; body
      ; author = os "author" ""
      ; doc = None
      ; hash = os "hash" (match v.v_hash with Some h -> h | None -> "")
      ; created_at =
          (match v.v_created_at with Some c -> c | None -> os "created_at" "")
      ; updated_at =
          (match v.v_created_at with Some c -> c | None -> os "created_at" "")
      ; rank = v.v_rank
      }

let parse_body_json (fields_json : string) : (Yojson.Safe.t, string) result =
  try Ok (Yojson.Safe.from_string fields_json)
  with Yojson.Json_error e -> Error ("bad body json: " ^ e)

let body_string_field (body : Yojson.Safe.t) (field : string) : string option =
  match body with
  | `Assoc kvs ->
      (match List.assoc_opt field kvs with
       | Some (`String s) when s <> "" -> Some s
       | Some (`Int i) -> Some (string_of_int i)
       | _ -> None)
  | _ -> None

let string_contains hay needle =
  let hlen = String.length hay and nlen = String.length needle in
  let rec loop i =
    if i + nlen > hlen then false
    else if String.sub hay i nlen = needle then true
    else loop (i + 1)
  in
  nlen = 0 || loop 0

(** Lookup by payload body.uuid (notes identity). *)
let get_by_uuid (conn : Store_db.conn) ~(uuid : string) : node option =
  let sql =
    "SELECT " ^ select_cols ^ " FROM tree_nodes "
    ^ "WHERE payload #>> '{body,uuid}' = $1 LIMIT 1"
  in
  let rows =
    Io_lwt.Db.execute conn ~params:[ Pgx.Value.of_string uuid ] sql
  in
  match rows with
  | row :: _ -> (node_of_row row)
  | [] -> None

(** Find hierarchical nodes where body->>field = value (e.g. slug). *)
let find_by_body_field (conn : Store_db.conn) ~(kind : string option)
    ~(field : string) ~(value : string) ?(limit : int = 50) () : node list =
  let sql =
    match kind with
    | Some k ->
        "SELECT " ^ select_cols ^ " FROM tree_nodes "
        ^ "WHERE kind = $1 AND payload #>> ARRAY['body',$2] = $3 "
        ^ "ORDER BY path::text ASC LIMIT $4"
    | None ->
        "SELECT " ^ select_cols ^ " FROM tree_nodes "
        ^ "WHERE payload #>> ARRAY['body',$1] = $2 "
        ^ "ORDER BY path::text ASC LIMIT $3"
  in
  let rows =
    match kind with
    | Some k ->
        Io_lwt.Db.execute conn
```

```
          ~params:
            [ Pgx.Value.of_string k
            ; Pgx.Value.of_string field
            ; Pgx.Value.of_string value
            ; Pgx.Value.of_int limit
            ]
          sql
    | None ->
        Io_lwt.Db.execute conn
          ~params:
            [ Pgx.Value.of_string field
            ; Pgx.Value.of_string value
            ; Pgx.Value.of_int limit
            ]
          sql
  in
  (List.filter_map node_of_row rows)

(** Upsert a hierarchical node at [path]. [body_json] is the payload body
    object (fields). [kind] must be folder|note|heading.
    K1: notes enforce unique (author, slug); folder+note unique body.uuid.
     *)
let upsert_hier (conn : Store_db.conn) ~(path : string) ~(kind : string)
    ~(type_name : string) ~(body_json : string) ~(author : string)
    ~(rank : string option)
    : (string, string) result =
  if not (is_hier_kind kind) then
    (Error ("upsert_hier: kind must be folder|note|heading, got " ^ kind))
  else if not (valid_path path) then
    (Error ("upsert_hier: invalid path labels: " ^ path))
  else
    match parse_body_json body_json with
    | Error e -> (Error e)
    | Ok body ->
        let slug_clash =
          if kind <> "note" then None
          else
            match body_string_field body "slug" with
            | None -> None
            | Some slug ->
                let hits =
                  find_by_body_field conn ~kind:(Some "note") ~field:"slug"
                    ~value:slug ~limit:5 ()
                in
                let other =
                  List.find_opt
                    (fun n -> n.path <> path && n.author = author)
                    hits
                in
                (match other with
                   | Some n ->
                       Some
                         (Printf.sprintf
                            "upsert_hier: slug %S already used at %s" slug
    n.path)
                   | None -> None)
        in
        (match slug_clash with
         | Some err -> (Error err)
         | None ->
             let uuid_clash =
               if kind <> "folder" && kind <> "note" then None
               else
                 match body_string_field body "uuid" with
                 | None -> None
                 | Some uuid ->
                     let existing = get_by_uuid conn ~uuid in
                     (match existing with
                        | Some n when n.path <> path ->
                            Some
                              (Printf.sprintf
                                 "upsert_hier: uuid %s already at %s" uuid
    n.path)
```

```
                        | _ -> None)
            in
            (match uuid_clash with
             | Some err -> (Error err)
             | None ->
                 let hash = hash_body (Yojson.Safe.to_string body) in
                 (* sibling-order: an explicit rank lands on INSERT and on
                    CONFLICT UPDATE (COALESCE keeps the existing rank when
                    the caller passes none -- tree/upsert never clobbers
                    order that tree/reorder established). *)
                 let sql =
                   "INSERT INTO tree_nodes (path, kind, type_name,
    payload, hash, author, rank, updated_at) "
                   ^ "VALUES ($1::ltree, $2::text, $3::text, "
                   ^ "
    jsonb_build_object('v',1,'kind',$2::text,'path',$1::text,'type',$3::text,
    "
                   ^ "
    'body',$4::jsonb,'author',$5::text,'doc',NULL,'hash',$6::text), "
                   ^ "  $6::text, $5::text, $7::text, now()) "
                   ^ "ON CONFLICT (path) DO UPDATE SET "
                   ^ "  kind = EXCLUDED.kind, type_name =
    EXCLUDED.type_name, "
                   ^ "  payload = EXCLUDED.payload, hash = EXCLUDED.hash, "
                   ^ "  author = EXCLUDED.author, "
                   ^ "  rank = COALESCE(EXCLUDED.rank, tree_nodes.rank), "
                   ^ "  updated_at = now() "
                   ^ "RETURNING path::text"
                 in
                 (try
      let rows =
                       Io_lwt.Db.execute conn
                         ~params:
                           [ Pgx.Value.of_string path
                           ; Pgx.Value.of_string kind
                           ; Pgx.Value.of_string type_name
                           ; Pgx.Value.of_string body_json
                           ; Pgx.Value.of_string author
                           ; Pgx.Value.of_string hash
                           ; (match rank with
                              | Some r -> Pgx.Value.of_string r
                              | None -> Pgx.Value.null)
                           ]
                         sql
                     in
                     match rows with
                     | [ [ p ] ] ->
                         (match Pgx.Value.to_string p with
                          | Some s -> (Ok s)
                          | None ->
                              (Error "upsert_hier: no path returned"))
                     | _ -> (Error "upsert_hier: failed")
    with exn ->
      let msg = Printexc.to_string exn in
                     if
                       string_contains msg "tree_nodes_note_slug"
                       || string_contains msg "tree_nodes_hier_uuid"
                       || string_contains msg "unique"
                     then
                       (Error ("upsert_hier: unique constraint: " ^ msg))
                     else raise exn)))

(** Delete one node, or the whole subtree when [recursive] is true. *)
let delete_path (conn : Store_db.conn) ~(path : string) ~(recursive : bool)
    : int =
  if not (valid_path path) then 0
  else
    let sql =
      if recursive then
        "DELETE FROM tree_nodes WHERE path <@ $1::ltree RETURNING
    path::text"
      else
        "DELETE FROM tree_nodes WHERE path = $1::ltree RETURNING path::text"
```

```
    in
    let rows =
      Io_lwt.Db.execute conn ~params:[ Pgx.Value.of_string path ] sql
    in
    (List.length rows)

(** Move subtree rooted at [old_path] to [new_path]. Rewrites every descendant
    path and payload.path. Fails if new_path already exists or labels invalid. *)
let move_subtree (conn : Store_db.conn) ~(old_path : string) ~(new_path : string)
    : (int, string) result =
  if not (valid_path old_path && valid_path new_path) then
    (Error "move_subtree: invalid path labels")
  else if old_path = new_path then (Ok 0)
  else
    (* Refuse moving under itself. *)
    let prefix = old_path ^ "." in
    if
      String.length new_path > String.length old_path
      && String.sub new_path 0 (String.length prefix) = prefix
    then (Error "move_subtree: cannot move under self")
    else
      let existing = get_by_path conn ~path:new_path in
      match existing with
      | Some _ -> (Error ("move_subtree: target exists: " ^ new_path))
      | None ->
          let sql =
            "WITH moved AS ( "
            ^ "  UPDATE tree_nodes SET "
            ^ "    path = CASE WHEN path = $1::ltree THEN $2::ltree "
            ^ "                ELSE ($2::ltree || subpath(path, nlevel($1::ltree))) END, "
            ^ "    payload = jsonb_set(payload, '{path}', to_jsonb( "
            ^ "      (CASE WHEN path = $1::ltree THEN $2::ltree "
            ^ "            ELSE ($2::ltree || subpath(path, nlevel($1::ltree))) END)::text)), "
            ^ "    updated_at = now() "
            ^ "  WHERE path <@ $1::ltree "
            ^ "  RETURNING path::text "
            ^ ") SELECT count(*)::int FROM moved"
          in
          let rows =
            Io_lwt.Db.execute conn
              ~params:
                [ Pgx.Value.of_string old_path; Pgx.Value.of_string new_path ]
              sql
          in
          (match rows with
           | [ [ c ] ] ->
               (match Pgx.Value.to_int c with
                | Some n -> (Ok n)
                | None -> (Error "move_subtree: bad count"))
           | _ -> (Error "move_subtree: failed"))

(** Lightweight note index rows: (uuid, fields_json) for kind=note under prefix.
    fields_json has slug/title/tags/kind/day/folder/_path -- no outline. *)
let list_note_index (conn : Store_db.conn) ~(prefix : string) ?(limit : int = 2000)
    () : (string * string) list =
  if not (valid_path prefix) then []
  else
    let sql =
      "SELECT path::text, "
      ^ "coalesce(payload #>> '{body,uuid}', ''), "
      ^ "coalesce(payload #>> '{body,slug}', ''), "
      ^ "coalesce(payload #>> '{body,title}', ''), "
      ^ "coalesce(payload #>> '{body,tags}', ''), "
      ^ "coalesce(payload #>> '{body,kind}', 'note'), "
      ^ "coalesce(payload #>> '{body,day}', '') "
```

```
      ^ "FROM tree_nodes WHERE path <@ $1::ltree AND kind = 'note' "
      ^ "ORDER BY path::text ASC LIMIT $2"
    in
    let rows =
      Io_lwt.Db.execute conn
        ~params:[ Pgx.Value.of_string prefix; Pgx.Value.of_int limit ]
        sql
    in
    let root = prefix ^ "." in
    let folder_of path =
      if not (String.length path > String.length root && String.sub path 0 (String.length root) = root)
      then ""
      else
        let rest =
          String.sub path (String.length root)
            (String.length path - String.length root)
        in
        let parts = String.split_on_char '.' rest in
        (match List.rev parts with
         | _slug :: rev_dirs -> String.concat "/" (List.rev rev_dirs)
         | [] -> "")
    in
    (List.filter_map
         (function
           | [ path_c; uuid_c; slug_c; title_c; tags_c; kind_c; day_c ] ->
               (match
                  Pgx.Value.to_string path_c,
                  Pgx.Value.to_string uuid_c,
                  Pgx.Value.to_string slug_c,
                  Pgx.Value.to_string title_c,
                  Pgx.Value.to_string tags_c,
                  Pgx.Value.to_string kind_c,
                  Pgx.Value.to_string day_c
                with
                | Some path, Some uuid, Some slug, Some title, Some tags, Some kind, Some day
                  ->
                    let folder = folder_of path in
                    let id = if uuid = "" then path else uuid in
                    let fields =
                      Yojson.Safe.to_string
                        (`Assoc
                           [ ("slug", `String slug)
                           ; ("title", `String title)
                           ; ("tags", `String tags)
                           ; ("kind", `String kind)
                           ; ("day", `String day)
                           ; ("folder", `String folder)
                           ; ("outline", `String "[]")
                           ; ("_path", `String path)
                          ])
                    in
                    Some (id, fields)
                | _ -> None)
           | _ -> None)
         rows)

(** Distinct folder paths under [prefix], computed in SQL.
    Derives folder from path structure: strip prefix, drop last segment,
    rejoin remaining with '/'. Returns sorted list of folder strings. *)
let list_folder_index (conn : Store_db.conn) ~(prefix : string) () : string list =
  if not (valid_path prefix) then []
  else
    (* Mirror list_note_index.folder_of: strip "prefix.", drop last label,
       rejoin remaining labels with '/'. Pure SQL so notes/doc/folder-names
       does not materialize thousands of rows in Nopales. *)
    let root = prefix ^ "." in
    let from_pos = string_of_int (String.length root + 1) in
    let sql =
      "SELECT DISTINCT "
      ^ "CASE "
```

```
      ^ "  WHEN position('.' in substring(path::text from " ^ from_pos ^
    ")) = 0 "
      ^ "    THEN '' "
      ^ "  ELSE "
      ^ "    replace( "
      ^ "      left( "
      ^ "        substring(path::text from " ^ from_pos ^ "), "
      ^ "        length(substring(path::text from " ^ from_pos ^ ")) "
      ^ "          - position('.' in reverse(substring(path::text from " ^
    from_pos ^ "))) "
      ^ "      ), "
      ^ "      '.', '/' "
      ^ "    ) "
      ^ "END AS folder "
      ^ "FROM tree_nodes "
      ^ "WHERE path <@ $1::ltree AND kind = 'note' "
      ^ "ORDER BY 1"
    in
    let rows =
      Io_lwt.Db.execute conn ~params:[ Pgx.Value.of_string prefix ] sql
    in
    List.filter_map (function [ c ] -> Pgx.Value.to_string c | _ -> None)
     rows

(** Substring search over note + heading bodies under [prefix].
    Returns lightweight hit dicts as JSON strings for Nopales dict-get:
      slug, title, node_id, header_path, matched, rank, kind, tags
    Doc-level hits have empty node_id; heading hits include breadcrumb. *)
let search_text (conn : Store_db.conn) ~(prefix : string) ~(needle : string)
    ?(limit : int = 20) () : string list =
  if not (valid_path prefix) then []
  else
    let needle = String.trim needle in
    if needle = "" then []
    else
      let pat = "%" ^ needle ^ "%" in
      let needle_l = String.lowercase_ascii needle in
      let note_sql =
        "SELECT "
        ^ "coalesce(payload #>> '{body,slug}', ''), "
        ^ "coalesce(payload #>> '{body,title}', ''), "
        ^ "coalesce(payload #>> '{body,tags}', ''), "
        ^ "coalesce(payload #>> '{body,kind}', 'note') "
        ^ "FROM tree_nodes WHERE path <@ $1::ltree AND kind = 'note' "
        ^ "AND (payload #>> '{body,title}' ILIKE $2 "
        ^ "  OR payload #>> '{body,tags}' ILIKE $2 "
        ^ "  OR payload #>> '{body,slug}' ILIKE $2) "
        ^ "ORDER BY 2 ASC LIMIT $3"
      in
      let heading_sql =
        "SELECT "
        ^ "coalesce(n.payload #>> '{body,slug}', ''), "
        ^ "coalesce(n.payload #>> '{body,title}', ''), "
        ^ "coalesce(n.payload #>> '{body,tags}', ''), "
        ^ "coalesce(n.payload #>> '{body,kind}', 'note'), "
        ^ "coalesce(h.payload #>> '{body,id}', ''), "
        ^ "coalesce(h.payload #>> '{body,title}', ''), "
        ^ "coalesce(h.payload #>> '{body,body}', ''), "
        ^ "(SELECT string_agg(coalesce(a.payload #>> '{body,title}', ''), '
    ? ' "
        ^ "ORDER BY nlevel(a.path)) "
        ^ "FROM tree_nodes a "
        ^ "WHERE a.kind = 'heading' AND a.path <@ n.path AND h.path <@
    a.path) AS header_path "
        ^ "FROM tree_nodes h "
        ^ "JOIN LATERAL ( "
        ^ "  SELECT path, payload FROM tree_nodes nn "
        ^ "  WHERE nn.kind = 'note' AND h.path <@ nn.path AND nn.path <@
    $1::ltree "
        ^ "  ORDER BY nlevel(nn.path) DESC LIMIT 1 "
        ^ ") n ON true "
        ^ "WHERE h.kind = 'heading' AND h.path <@ $1::ltree "
        ^ "AND (h.payload #>> '{body,title}' ILIKE $2 "
```

```
      ^ "  OR h.payload #>> '{body,body}' ILIKE $2) "
      ^ "ORDER BY 6 ASC LIMIT $3"
   in
   let params =
     [ Pgx.Value.of_string prefix
     ; Pgx.Value.of_string pat
     ; Pgx.Value.of_int limit
     ]
   in
   let note_rows = Io_lwt.Db.execute conn ~params note_sql in
   let heading_rows = Io_lwt.Db.execute conn ~params heading_sql in
   let hit ~slug ~title ~tags ~kind ~node_id ~header_path ~matched ~rank =
     Yojson.Safe.to_string
       (`Assoc
          [ ("slug", `String slug)
          ; ("title", `String title)
          ; ("tags", `String tags)
          ; ("kind", `String kind)
          ; ("node_id", `String node_id)
          ; ("header_path", `String header_path)
          ; ("matched", `String matched)
          ; ("rank", `Int rank)
          ])
   in
   let contains_ci hay =
     string_contains (String.lowercase_ascii hay) needle_l
   in
   let parse_note = function
     | [ slug_c; title_c; tags_c; kind_c ] ->
         (match
            Pgx.Value.to_string slug_c,
            Pgx.Value.to_string title_c,
            Pgx.Value.to_string tags_c,
            Pgx.Value.to_string kind_c
          with
          | Some slug, Some title, Some tags, Some kind ->
              let matched, rank =
                if String.lowercase_ascii slug = needle_l then ("slug", 0)
                else if contains_ci title then ("title", 1)
                else if contains_ci tags then ("tags", 3)
                else if contains_ci slug then ("slug", 1)
                else ("title", 4)
              in
              Some (rank, slug, "", hit ~slug ~title ~tags ~kind ~node_id:""
                      ~header_path:"" ~matched ~rank)
          | _ -> None)
     | _ -> None
   in
   let parse_heading = function
     | [ slug_c; title_c; tags_c; kind_c; nid_c; htitle_c; hbody_c; path_c ]
       ->
         (match
            Pgx.Value.to_string slug_c,
            Pgx.Value.to_string title_c,
            Pgx.Value.to_string tags_c,
            Pgx.Value.to_string kind_c,
            Pgx.Value.to_string nid_c,
            Pgx.Value.to_string htitle_c,
            Pgx.Value.to_string hbody_c,
            Pgx.Value.to_string path_c
          with
          | Some slug, Some title, Some tags, Some kind, Some nid,
            Some htitle, Some hbody, Some header_path ->
              let matched, rank =
                if contains_ci htitle then ("header", 2)
                else if contains_ci hbody then ("body", 4)
                else ("header", 4)
              in
              Some
```

```
                  ( rank
                  , slug
                  , nid
                  , hit ~slug ~title ~tags ~kind ~node_id:nid ~header_path
                      ~matched ~rank )
            | _ -> None)
        | _ -> None
      in
      let notes = List.filter_map parse_note note_rows in
      let heads = List.filter_map parse_heading heading_rows in
      let merged =
        List.sort
          (fun (r1, s1, n1, _) (r2, s2, n2, _) ->
            let c = compare r1 r2 in
            if c <> 0 then c
            else
              let c2 = String.compare s1 s2 in
              if c2 <> 0 then c2 else String.compare n1 n2)
          (notes @ heads)
      in
      (* Dedupe (slug, node_id); keep best rank. *)
      let seen = Hashtbl.create 32 in
      let acc = ref [] in
      List.iter
        (fun (_r, slug, nid, json) ->
          let key = slug ^ "\x00" ^ nid in
          if not (Hashtbl.mem seen key) then begin
            Hashtbl.add seen key ();
            acc := json :: !acc
          end)
        merged;
      let out = List.rev !acc in
      if List.length out <= limit then out else
        let rec take n xs =
          if n <= 0 then []
          else match xs with [] -> [] | x :: rest -> x :: take (n - 1) rest
        in
        take limit out

(** List nodes of a given kind under path prefix (e.g. all notes in a
    garden). *)
let list_by_kind (conn : Store_db.conn) ~(prefix : string) ~(kind : string)
    ?(limit : int = 2000) () : node list =
  if not (valid_path prefix) then []
  else
    let sql =
      "SELECT " ^ select_cols ^ " FROM tree_nodes "
      ^ "WHERE path <@ $1::ltree AND kind = $2 "
      ^ "ORDER BY path::text ASC LIMIT $3"
    in
    let rows =
      Io_lwt.Db.execute conn
        ~params:
          [ Pgx.Value.of_string prefix
          ; Pgx.Value.of_string kind
          ; Pgx.Value.of_int limit
          ]
        sql
    in
    (List.filter_map node_of_row rows)

(** Direct children under [prefix], index-friendly (one-tree.projections
    CHILDREN-BY-LEVEL). Expressed as an nlevel predicate the ltree GiST
    path index can drive -- never a correlated NOT EXISTS anti-join over
    the whole subtree (that form is O(subtree^2) and blocked /notes at
    ~3000 notes).

    - one-label children: nlevel = nlevel(prefix) + 1 (folders, notes)
    - outline covering: nlevel = nlevel(prefix) + 2 under the literal
      label [h] (note outline roots at prefix.h.<nid>, path grammar in
      one-tree.tree-model). No intermediate node is required at
      prefix.h. *)
```

```
let list_children (conn : Store_db.conn) ~(prefix : string) ?(limit : int = 50000)
    () : node list =
  if not (valid_path prefix) then []
  else
    let sql =
      "SELECT " ^ select_cols ^ " FROM tree_nodes n "
      ^ "WHERE n.path <@ $1::ltree AND n.path <> $1::ltree "
      ^ "AND ( "
      ^ "  nlevel(n.path) = nlevel($1::ltree) + 1 "
      ^ "  OR ( "
      ^ "    nlevel(n.path) = nlevel($1::ltree) + 2 "
      ^ "    AND subpath(n.path, nlevel($1::ltree), 1)::text = 'h' "
      ^ "  ) "
      ^ ") "
      (* sibling-order: ranked siblings first (byte-order rank -- locale-
         independent), then unranked, path-ordered among themselves. *)
      ^ "ORDER BY n.rank COLLATE \"C\" NULLS LAST, n.path::text ASC LIMIT $2"
    in
    let rows =
      Io_lwt.Db.execute conn
        ~params:[ Pgx.Value.of_string prefix; Pgx.Value.of_int limit ]
        sql
    in
    (List.filter_map node_of_row rows)

(** Lens-aware children-of: DIRECT-child nodes of the RELATIVE logical path,
    resolved against [chain] (self-first / closest-wins). Uses the effective
    branch where the logical path first resolves, then list_children on that
    materialized path -- so direct children are read under the branch that
    actually owns/overtides the node. Returns [] when the path resolves
    nowhere. Shared with lens_get_in_chain so the override rule is single.
    *)
let lens_children_in_chain (conn : Store_db.conn) ~(suffix : string)
    ?(limit : int = 50000) (chain : string list) : node list =
  let mk_path branch_id =
    match lens_suffix_path ~branch_id suffix with
    | Some p -> p
    | None -> ""
  in
  match resolve_in_chain conn ~mk_path chain with
  | None -> []
  | Some n -> list_children conn ~prefix:n.path ~limit ()

(** Lens-aware subtree: all descendants of the RELATIVE logical path,
    resolved against [chain] (closest-wins), read under the effective
    branch, capped at limit. Backed by list_subtree. Returns [] when the
    path resolves nowhere. *)
let lens_subtree_in_chain (conn : Store_db.conn) ~(suffix : string)
    ?(limit : int = 50000) (chain : string list) : node list =
  let mk_path branch_id =
    match lens_suffix_path ~branch_id suffix with
    | Some p -> p
    | None -> ""
  in
  match resolve_in_chain conn ~mk_path chain with
  | None -> []
  | Some n -> list_subtree conn ~prefix:n.path ~limit ()

(** Children of [prefix] AS OF [gen] (tree-time reads-as-of-gen): the
    newest version row at-or-below [gen] per child path, LEVEL predicate
    on the version paths (same shape as list_children, + the +2 'h'
    covering), tombstones skipped. One index-driven query -- the
    /tree?gen= browser surface. Result re-ordered by sibling-order
    (rank COLLATE "C" NULLS LAST, path ASC).

    NOTE: a version's gen comes from the writer's counter class
    (branch_generations for b.* paths, the tree-wide tree_generations
    for data.* paths) -- a ?gen= read is meaningful within one path
    class, never across both. *)
let list_children_as_of (conn : Store_db.conn) ~(prefix : string)
    ~(gen : int64) ?(limit : int = 1000) () : node list =
```

```
  if not (valid_path prefix) then []
  else
    let sql =
      "SELECT DISTINCT ON (v.path) v.gen, v.path::text, v.kind, "
      ^ "v.type_name, v.body::text, v.hash, v.author, "
      ^ "v.created_at::text, v.body->>'_tombstone'::text, v.rank "
      ^ "FROM tree_versions v "
      ^ "WHERE v.path <@ $1::ltree AND v.path <> $1::ltree "
      ^ "AND ( nlevel(v.path) = nlevel($1::ltree) + 1 "
      ^ "      OR ( nlevel(v.path) = nlevel($1::ltree) + 2 "
      ^ "           AND subpath(v.path, nlevel($1::ltree), 1)::text = 'h' ) ) "
      ^ "AND v.gen <= $2::bigint "
      ^ "ORDER BY v.path, v.gen DESC LIMIT $3"
    in
    let rows =
      Io_lwt.Db.execute conn
        ~params:
          [ Pgx.Value.of_string prefix
          ; Pgx.Value.of_int64 gen
          ; Pgx.Value.of_int limit
          ]
        sql
    in
    let nodes =
      List.filter_map
        (fun row ->
          match version_of_row row with
          | Some v -> version_of_leaf v
          | None -> None)
        rows
    in
    (* sibling-order: ranked first (byte order), then unranked by path *)
    let rank_key (n : node) =
      (match n.rank with Some r -> (0, r) | None -> (1, ""))
    in
    List.sort
      (fun a b ->
        let (ka, ra) = rank_key a and (kb, rb) = rank_key b in
        if ka <> kb then ka - kb
        else if ka = 0 && String.compare ra rb <> 0 then String.compare ra rb
        else String.compare a.path b.path)
      nodes

(** Set the rank of one node (None clears it -- back to unordered).
   Returns the number of rows updated (0 = no such node). A rank change
   is an ordinary versioned write (migration 025's trigger echoes the
   rank column; a same-rank UPDATE is a suppressed mirror echo). *)
let set_rank (conn : Store_db.conn) ~(path : string)
    ~(rank : string option) : int =
  if not (valid_path path) then 0
  else
    let sql =
      "UPDATE tree_nodes SET rank = $2::text, updated_at = now() "
      ^ "WHERE path = $1::ltree RETURNING path::text"
    in
    let rows =
      Io_lwt.Db.execute conn
        ~params:
          [ Pgx.Value.of_string path
          ; (match rank with Some r -> Pgx.Value.of_string r | None -> Pgx.Value.null)
          ]
        sql
    in
    List.length rows

(** Re-assign fresh fixed-width ranks to every RANKED sibling under
   [parent] (rank-ordered), keeping their relative order. Unranked
   siblings stay NULL (they sort after the ranked block). Returns the
   number reassigned. Caller wraps in a transaction if atomicity matters. *)
let rebalance_siblings (conn : Store_db.conn) ~(parent : string) : int =
```

```
  let ranked =
    list_children conn ~prefix:parent ()
    |> List.filter_map (fun (n : node) ->
           match n.rank with Some r -> Some (n.path, r) | None -> None)
  in
  let fresh = rank_rebalance (List.length ranked) in
  List.fold_left2
    (fun acc (path, _old) new_rank ->
      acc + set_rank conn ~path ~rank:(Some new_rank))
    0 ranked fresh

(* ---- tree-edges (one-tree.tree-edges) ----

   An edge IS a node: kind=link at <source-path>.lnk.<uuid>, body
   {to, kind, label?}. "from" is positional (the parent path), never
   duplicated in the body. to is branch-qualified path TEXT stored
   verbatim -- it may dangle; resolution is a READ-TIME act against the
   reader's lens (get_by_path on the absolute path). An unresolved edge
   is dangling data -- rendered as such, repairable, never an error
   state and never a write blocker. *)

let is_link_kind (k : string) : bool = k = "link"

(** Validate an edge body: JSON object; required non-empty string "to"
    that is a well-formed absolute ltree path (it may DANGLE -- no node
    needs to exist there); required non-empty string "kind" (the edge
    kind, e.g. "mentions"); optional string "label". "from" is
    positional and must NOT appear in the body. Returns the validated
    body JSON text or an error message. *)
let validate_link_body (body_json : string) : (string, string) result =
  match parse_body_json body_json with
  | Error e -> Error e
  | Ok body -> (
      match body with
      | `Assoc kvs ->
          let field k =
            match List.assoc_opt k kvs with
            | Some (`String s) -> Some s
            | Some _ when k <> "label" -> Some "" (* present but not a
    string *)
            | _ -> None
          in
          if List.mem_assoc "from" kvs then
            Error "link body: \"from\" is positional (the parent path);
    never put it in the body"
          else
            (match field "to" with
             | Some to_p when to_p <> "" && valid_path to_p ->
                 (match field "kind" with
                  | Some k when k <> "" ->
                      (match List.assoc_opt "label" kvs with
                       | Some (`String _) | None -> Ok body_json
                       | Some _ -> Error "link body: label must be a
    string")
                  | _ -> Error "link body: need non-empty string kind")
             | Some _ ->
                 Error "link body: to must be a non-empty well-formed
    absolute path"
             | None -> Error "link body: need to")
      | _ -> Error "link body must be a JSON object")

(** Upsert an edge node at [path] (kind=link). [path] must be an edge
    path: <source>.lnk.<label>. The body is validated via
    [validate_link_body]. ON CONFLICT (path) DO UPDATE, same as nodes. *)
let upsert_link (conn : Store_db.conn) ~(path : string)
    ~(body_json : string) ~(author : string) : (string, string) result =
  if not (valid_path path) then Error "upsert_link: invalid path labels"
  else
    let labels = String.split_on_char '.' path in
    let n = List.length labels in
    let mid_ok =
      n >= 3
```

```
      && (match List.nth_opt labels (n - 2) with Some "lnk" -> true | _ -> false)
    in
    if not mid_ok then
      Error "upsert_link: edge paths live at <source>.lnk.<id>"
    else
      match validate_link_body body_json with
      | Error e -> Error e
      | Ok _ ->
          let body =
            match Yojson.Safe.from_string body_json with
            | b -> b
            | exception _ -> `Null
          in
          let hash = hash_body (Yojson.Safe.to_string body) in
          let sql =
            "INSERT INTO tree_nodes (path, kind, type_name, payload, hash, author, updated_at) "
            ^ "VALUES ($1::ltree, 'link', NULL, "
            ^ "jsonb_build_object('v',1,'kind','link','path',$1::text,'type','link', "
            ^ "'body',$2::jsonb,'author',$3::text,'doc',NULL,'hash',$4::text), "
            ^ "  $4::text, $3::text, now()) "
            ^ "ON CONFLICT (path) DO UPDATE SET "
            ^ "  payload = EXCLUDED.payload, hash = EXCLUDED.hash, "
            ^ "  author = EXCLUDED.author, updated_at = now() "
            ^ "RETURNING path::text"
          in
          (try
             let rows =
               Io_lwt.Db.execute conn
                 ~params:
                   [ Pgx.Value.of_string path
                   ; Pgx.Value.of_string (Yojson.Safe.to_string body)
                   ; Pgx.Value.of_string author
                   ; Pgx.Value.of_string hash
                   ]
                 sql
             in
             (match rows with
              | [ [ p ] ] ->
                  (match Pgx.Value.to_string p with
                   | Some s -> Ok s
                   | None -> Error "upsert_link: no path returned")
              | _ -> Error "upsert_link: failed")
           with exn -> Error ("upsert_link: " ^ Printexc.to_string exn))

(** Outgoing edges of [path]: edge nodes with kind=link at
    <path>.lnk.<id> -- a children-by-level index read (CHILDREN-BY-LEVEL
    invariant; the edge grammar is TWO labels under the source, the
    literal mid-label lnk), never a scan. Ordered by path. *)
let list_out_links (conn : Store_db.conn) ~(path : string)
    ?(limit : int = 1000) () : node list =
  if not (valid_path path) then []
  else
    let sql =
      "SELECT " ^ select_cols ^ " FROM tree_nodes n "
      ^ "WHERE n.path <@ $1::ltree AND n.path <> $1::ltree "
      ^ "AND nlevel(n.path) = nlevel($1::ltree) + 2 "
      ^ "AND subpath(n.path, nlevel($1::ltree), 1)::text = 'lnk' "
      ^ "AND n.kind = 'link' "
      ^ "ORDER BY n.path::text ASC LIMIT $2"
    in
    let rows =
      Io_lwt.Db.execute conn
        ~params:[ Pgx.Value.of_string path; Pgx.Value.of_int limit ]
        sql
    in
    List.filter_map node_of_row rows

(** Edges pointing AT [needle] (to = needle exactly) -- backed by the
    migration-026 partial expression index on
```

```
    (payload->'body'->>'to') WHERE kind='link'. *)
let edges_to (conn : Store_db.conn) ~(needle : string)
    ?(limit : int = 200) () : node list =
  let sql =
    "SELECT " ^ select_cols ^ " FROM tree_nodes n "
    ^ "WHERE n.kind = 'link' AND n.payload->'body'->>'to' = $1::text "
    ^ "ORDER BY n.path::text ASC LIMIT $2"
  in
  let rows =
    Io_lwt.Db.execute conn
      ~params:[ Pgx.Value.of_string needle; Pgx.Value.of_int limit ]
      sql
  in
  List.filter_map node_of_row rows

(** Incoming-edge count for [path]: edges whose to = path exactly (the
    warn-not-forbid report surface for tree/delete and tree/move). *)
let incoming_edge_count (conn : Store_db.conn) ~(path : string) : int =
  if not (valid_path path) then 0
  else
    let sql =
      "SELECT count(*)::int FROM tree_nodes "
      ^ "WHERE kind = 'link' AND payload->'body'->>'to' = $1::text"
    in
    let rows =
      Io_lwt.Db.execute conn ~params:[ Pgx.Value.of_string path ] sql
    in
    (match rows with
     | [ [ c ] ] -> (match Pgx.Value.to_int c with Some n -> n | None -> 0)
     | _ -> 0)

let html_escape_text (s : string) : string =
  let buf = Buffer.create (String.length s + 8) in
  String.iter
    (function
      | '&' -> Buffer.add_string buf "&amp;"
      | '<' -> Buffer.add_string buf "&lt;"
      | '>' -> Buffer.add_string buf "&gt;"
      | '"' -> Buffer.add_string buf "&quot;"
      | '\'' -> Buffer.add_string buf "&#39;"
      | c -> Buffer.add_char buf c)
    s;
  Buffer.contents buf

let path_label_of (path : string) : string =
  match List.rev (String.split_on_char '.' path) with
  | [] -> path
  | last :: _ -> last

let node_display_title (n : node) : string =
  match body_string_field n.body "title" with
  | Some t when String.trim t <> "" -> t
  | _ ->
      (match body_string_field n.body "slug" with
       | Some s when String.trim s <> "" -> s
       | _ -> path_label_of n.path)

let node_display_slug (n : node) : string =
  match body_string_field n.body "slug" with
  | Some s when String.trim s <> "" -> s
  | _ -> path_label_of n.path

(** Direct children of [prefix] that themselves own a folder/note child.
    One query: depth+2 folder/note nodes ? their parent path at depth+1.
    Lets notes like garden.bible expand to show characters/bsb. *)
let parents_with_hier_children (conn : Store_db.conn) ~(prefix : string)
    : string list =
  if not (valid_path prefix) then []
  else
    let sql =
      "SELECT DISTINCT subpath(path, 0, nlevel($1::ltree) + 1)::text "
      ^ "FROM tree_nodes "
      ^ "WHERE path <@ $1::ltree "
```

```
      ^ "AND nlevel(path) = nlevel($1::ltree) + 2 "
      ^ "AND kind IN ('folder', 'note')"
    in
    let rows =
      Io_lwt.Db.execute conn ~params:[ Pgx.Value.of_string prefix ] sql
    in
    List.filter_map (function [ c ] -> Pgx.Value.to_string c | _ -> None)
     rows

(** Budget-safe HTML for notes tree children under [prefix].
    Mirrors notes/ui/tree-children-html but runs in OCaml so folders with
    thousands of notes do not blow the Nopales eval budget. *)
let render_children_html (conn : Store_db.conn) ~(prefix : string)
    ?(limit : int = 50000) ~(garden_root : string) () : string =
  if not (valid_path prefix) then
    "<p class=\"muted\">(empty)</p>"
  else
    let kids = list_children conn ~prefix ~limit () in
    let kids =
      List.filter (fun n -> n.kind = "folder" || n.kind = "note") kids
    in
    if kids = [] then "<p class=\"muted\">(empty)</p>"
    else
      let expandable =
        let set = Hashtbl.create 64 in
        List.iter (fun p -> Hashtbl.replace set p true)
          (parents_with_hier_children conn ~prefix);
        set
      in
      let garden_path = garden_root ^ ".garden" in
      let buf = Buffer.create 4096 in
      let emit_folder_details ~path ~label ~open_slug =
        Buffer.add_string buf "<details class=\"tree-folder\" hx-get=\"";
        Buffer.add_string buf "/partials/notes-tree?path=";
        Buffer.add_string buf (html_escape_text path);
        Buffer.add_string buf
          "\" hx-trigger=\"toggle once\" hx-target=\"find .tree-children\"
    ";
        Buffer.add_string buf "hx-swap=\"innerHTML\"><summary
    class=\"tree-folder-header\">";
        Buffer.add_string buf
          "<span class=\"tree-folder-icon\">&#x1F4C1; </span>";
        (match open_slug with
         | Some slug ->
             Buffer.add_string buf "<a href=\"/notes?slug=";
             Buffer.add_string buf (html_escape_text slug);
             Buffer.add_string buf "\"
    onclick=\"event.stopPropagation()\">";
             Buffer.add_string buf (html_escape_text label);
             Buffer.add_string buf "</a>"
         | None -> Buffer.add_string buf (html_escape_text label));
        Buffer.add_string buf
          "</summary><div class=\"tree-children\"><span class=\"muted\">";
        Buffer.add_string buf "open to load?</span></div></details>"
      in
      let emit_note_li n =
        let slug = node_display_slug n in
        let title = node_display_title n in
        Buffer.add_string buf "<li class=\"tree-item\"><a
    href=\"/notes?slug=";
        Buffer.add_string buf (html_escape_text slug);
        Buffer.add_string buf "\">";
        Buffer.add_string buf (html_escape_text title);
        Buffer.add_string buf "</a></li>"
      in
      let rec emit_inlined path =
        let nested = list_children conn ~prefix:path ~limit () in
        let nested_expand =
          let set = Hashtbl.create 64 in
          List.iter (fun p -> Hashtbl.replace set p true)
            (parents_with_hier_children conn ~prefix:path);
          set
        in
```

```
        List.iter
          (fun n ->
            if n.kind = "folder" || n.kind = "note" then
              emit_node ~expandable:nested_expand n)
          nested
      and emit_node ~expandable n =
        if n.kind = "folder" then
          if n.path = garden_path then emit_inlined n.path
          else
            emit_folder_details ~path:n.path ~label:(node_display_title n)
              ~open_slug:None
        else if n.kind = "note" then
          if Hashtbl.mem expandable n.path then
            emit_folder_details ~path:n.path ~label:(node_display_title n)
              ~open_slug:(Some (node_display_slug n))
          else emit_note_li n
        else ()
      in
      List.iter (emit_node ~expandable) kids;
      Buffer.contents buf
```

113 image/test/agent_tools_test.ml

```
(** Pricklypear Image -- agent_tools effect-gate tests (no DB).

    Tests the macroexpand-fail path of [check_ai_body_effects] (B1):
    a failing macroexpand must NOT fall back to the unexpanded tree --
    it must surface an Error so the AI install is denied. *)
open Pricklypear_lisp

(* Build an env with a macro that raises during expansion: the macro
   body references an unbound variable, so expanding (boom) raises
   Interp_error (Unbound_var). *)
let make_env_with_bad_macro () =
  let prims = Eval.make_prim_table () in
  let env = Env.create () in
  (* (defmacro boom () nope) -- body is [Var "nope"], unbound at expand *)
  let _ = Eval.eval_string ~prims env "(defmacro boom () nope)" in
  env

let tests = [
  ("macroexpand failure surfaces Error (not silent Ok)", `Quick, fun () ->
    let env = make_env_with_bad_macro () in
    let result =
      Pricklypear_image_lib.Agent_tools.check_ai_body_effects ~env "(boom)"
    in
    match result with
    | Ok () ->
        Alcotest.fail
          "expected Error on macroexpand failure, got Ok (silent fallback
     bug)"
    | Error msg ->
        (* the message must mention macroexpand failure *)
        let has_macroexpand =
          let needle = "macroexpand" in
          let nlen = String.length needle in
          let mlen = String.length msg in
          let rec loop i =
            if i + nlen > mlen then false
            else if String.sub msg i nlen = needle then true
            else loop (i + 1)
          in
          loop 0
        in
        Alcotest.(check bool) "mentions macroexpand" true has_macroexpand);
  ("no-env path still Ok for pure body", `Quick, fun () ->
    (* Without ~env, macroexpand is skipped; a pure body should be Ok *)
    let result =
      Pricklypear_image_lib.Agent_tools.check_ai_body_effects "(+ 1 2)"
    in
    match result with
    | Ok () -> ()
    | Error msg ->
        Alcotest.failf "expected Ok for pure body, got Error: %s" msg);
]

let () =
  Alcotest.run "pricklypear-image-agent-tools" [
    "effect-gate", tests;
  ]
```

114 image/test/ai_store_test.ml

```
(** Round-trip tests for ai_store (needs Postgres from scripts/dev.sh).

    Skips cleanly when PP_DB is unreachable so unit CI without PG still
    builds. Live smoke is the primary verification. *)

let env_or name dft =
  match Sys.getenv_opt name with Some s when s <> "" -> s | _ -> dft

let with_store f =
  Eio_main.run @@ fun env ->
  Lwt_eio.with_event_loop ~clock:(Eio.Stdenv.clock env) @@ fun _ ->
  Eio.Switch.run @@ fun sw ->
  let ctx : Pricklypear_image_lib.Io_eio.Ctx.t =
    {
      sw;
      net = (Eio.Stdenv.net env :> [ `Generic ] Eio.Net.ty Eio.Std.r);
    }
  in
  Pricklypear_image_lib.Io_eio.with_ctx ctx (fun () ->
      let cfg = Pricklypear_image_lib.Store.config_from_env () in
      let store =
        Pricklypear_image_lib.Store.make ~cfg
          ~branch_id:"00000000-0000-0000-0000-000000000001" ()
      in
      Pricklypear_image_lib.Store.connect store;
      match Pricklypear_image_lib.Store.conn_of store with
      | None ->
          Printf.printf
            "SKIP: no postgres (set PP_DB_* / run scripts/dev.sh)\n%!"
      | Some _ -> f store)

let test_round_trip () =
  with_store (fun store ->
    let open Pricklypear_image_lib in
    let owner = "ai_store_test_" ^ string_of_int (Random.int 1_000_000) in
    let created =
      Ai_store.create_session store ~owner ~seed:"system seed" ~tools:true
        ~title:"t1" ~branch_ref:"system/main" ~model:"test-model" ()
    in
    match created with
    | Error e -> Alcotest.fail e
    | Ok sess ->
        let r1 =
          Ai_store.append_messages store ~session_id:sess.id
            [ Llm_client.Msg { role = Llm_client.User; content = "hello" }
            ; Llm_client.Msg { role = Llm_client.Assistant; content = "hi" }
            ]
        in
        (match r1 with Error e -> Alcotest.fail e | Ok () -> ());
        let tc : Llm_client.tool_call =
          { id = "call_1"; name = "list_bindings"; arguments = "{}" }
        in
        let r2 =
          Ai_store.append_messages store ~session_id:sess.id
            [ Llm_client.Assistant_tools
                { content = Some "checking"; tool_calls = [ tc ] }
            ; Llm_client.Tool_result
                { tool_call_id = "call_1"; content = "(no bindings)" }
            ]
        in
        (match r2 with Error e -> Alcotest.fail e | Ok () -> ());
        let hist = Ai_store.list_messages store ~session_id:sess.id in
        Alcotest.(check int) "4 messages" 4 (List.length hist);
        let api = Ai_store.messages_for_api sess hist in
        (match api with
         | Llm_client.Msg { role = Llm_client.System; content } :: _ ->
             Alcotest.(check string) "seed injected" "system seed" content
         | _ -> Alcotest.fail "expected system seed first");
        Alcotest.(check int) "api msgs = 1+history" 5 (List.length api);
        let got = Ai_store.get_session store ~owner ~id:sess.id in
        (match got with
```

```
72          | None -> Alcotest.fail "get_session missing"
73          | Some s ->
74              Alcotest.(check string) "owner" owner s.owner;
75              Alcotest.(check bool) "tools" true s.tools);
76         let listed = Ai_store.list_sessions store ~owner () in
77         Alcotest.(check bool) "listed" true (List.length listed >= 1);
78         let del = Ai_store.delete_session store ~owner ~id:sess.id in
79         Alcotest.(check bool) "deleted" true del;
80         let gone = Ai_store.get_session store ~owner ~id:sess.id in
81         Alcotest.(check bool) "gone" true (gone = None);
82         ())
83
84 let () =
85   Random.self_init ();
86   Alcotest.run "ai_store"
87     [ ( "round-trip"
88       , [ Alcotest.test_case "create/append/list/delete" `Quick
     test_round_trip
89         ] )
90     ]
```

115 image/test/api__prim__test.ml

```
(** Pricklypear Image -- /api/prim refuse-or-forward dispatch tests.

    Covers the server-endpoint verify clauses of borge/remote-eval.borg:
      - same value as in-process (dispatch executes the SAME table fn the
        image rim services, and the value serializes the same)
      - a server-only prim is refused with a structured error citing
        locality (kind "server_only"), not silently executed
      - refuse-or-forward classification is DERIVED from prim-registry
        metadata (register_server_only), never a hand-maintained list

    Pure: no Postgres, no HTTP server, no Session. We exercise the
    [Primitives.dispatch] decision the /api/prim endpoint is a thin
    wrapper around, and the real pure prim table [make_with_defaults]
    builds -- exactly the table a session dispatches through. *)

open Pricklypear_lisp.Ast
module P = Pricklypear_image_lib.Primitives

(* A proxyable prim + a server-only prim registered the way the image's
   registration sites do (register / register_server_only). *)
let make_tbl () =
  let t = P.make () in
  P.register_sync t "double" (function
      | [ VInt n ] -> VInt (Int64.mul n 2L)
      | _ -> VInt 0L);
  P.register_server_only_sync t "route" (fun _ -> VStr "route");
  P.register_server_only_sync t "checkout" (fun _ -> VStr "checkout");
  t

(* --- same value as in-process --- *)

(* dispatch on a proxyable prim returns the value the in-process rim
   would (same table fn). We cross-check against the REAL pure table
   prims via in-process eval on the same table. *)
let test_same_value_as_in_process () =
  let env_tbl = P.make_with_defaults () in
  let env = Pricklypear_lisp.Env.create () in
  Hashtbl.iter
    (fun name _ -> Pricklypear_lisp.Env.define env name (VPrim name))
    env_tbl;
  (* in-process: eval (+ 1 2) under the rim against the same table *)
  let in_process =
    Pricklypear_image_lib.Control_rim.run ~prims:env_tbl (fun () ->
        Pricklypear_lisp.Eval.eval_string_direct
          ~prims:Pricklypear_lisp.Eval.empty_prims env "(+ 1 2)")
  in
  Alcotest.(check string) "in-process (+ 1 2)" "3"
    (Pricklypear_lisp.Reader.pp_value in_process);
  (* the /api/prim path: dispatch "+" on the same table *)
  match P.dispatch env_tbl "+" [ VInt 1L; VInt 2L ] with
  | P.Prim_ok v ->
      Alcotest.(check string) "dispatch (+ 1 2)" "3"
        (Pricklypear_lisp.Reader.pp_value v);
      (* value serializes identically through the shared value codec *)
      Alcotest.(check string) "same JSON value" "3"
        (Yojson.Safe.to_string (Prims_pure.value_to_json v))
  | _ -> Alcotest.fail "dispatch returned a non-ok result for +"

(* dispatch of an unregistered name behaves like the rim: a Prim_error. *)
let test_unregistered_is_error () =
  let t = P.make () in
  match P.dispatch t "no-such-prim" [] with
  | P.Prim_error exn ->
      (match exn with
       | Interp_error (Prim_error (name, _, _)) ->
           Alcotest.(check string) "unregistered name" "no-such-prim" name
       | _ -> Alcotest.fail "expected Prim_error for unregistered prim")
  | _ -> Alcotest.fail "expected Prim_error for unregistered prim"

(* --- server-only refused, not executed --- *)

```

```
let test_server_only_refused_not_executed () =
  let t = make_tbl () in
  (* route is registered as a harm-free no-op; if it WERE executed it
     would return ok. The refusal must happen before execution. *)
  match P.dispatch t "route" [] with
  | P.Prim_refused_server_only (name, reason) ->
      Alcotest.(check string) "refused name" "route" name;
      Alcotest.(check string) "locality reason" "prim must execute
     server-side"
        reason
  | _ -> Alcotest.fail "expected server-only refusal for route"

(* --- classification is data-driven --- *)

let test_classification_data_driven () =
  let t = make_tbl () in
  (* proxyable by default (registered via register_sync) *)
  Alcotest.(check bool) "double proxyable" false (P.is_server_only t
     "double");
  (* server-only iff registered via register_server_only *)
  Alcotest.(check bool) "route server-only" true (P.is_server_only t
     "route");
  Alcotest.(check bool) "checkout server-only" true
    (P.is_server_only t "checkout");
  (* a prim not yet registered at all is proxyable (unclassified) *)
  Alcotest.(check bool) "unregistered is proxyable" false
    (P.is_server_only t "never-registered");
  (* classification tracks the registry metadata: a NEW registry that
     registers a name as proxyable does not leak the server-only mark *)
  let t2 = P.make () in
  P.register_sync t2 "route" (fun _ -> VStr "route");
  (* NB: server-only locality is a registry-wide property of a prim NAME
     (route is always server-only on the image), so re-registering it as
     proxyable in a scratch registry keeps it server-only. The single
     source of truth is the registration site, never a hand list. *)
  Alcotest.(check bool) "registry is source of truth for route" true
    (P.is_server_only t2 "route");
  match P.dispatch t2 "route" [] with
  | P.Prim_refused_server_only (name, _) ->
      Alcotest.(check string) "refused via metadata" "route" name
  | _ -> Alcotest.fail "route must remain server-only from registry
     metadata"

(* DB CRUD prims on the real pure table are proxyable (not marked
   server-only). *)
let test_db_crud_not_server_only_on_real_table () =
  let t = P.make_with_defaults () in
  List.iter
    (fun name ->
      Alcotest.(check bool)
        (Printf.sprintf "%s is proxyable on the default table" name)
        false (P.is_server_only t name))
    [ "get"; "create"; "update"; "delete"; "rows"; "+"; "not" ]

let () =
  Alcotest.run "api_prim"
    [
      ( "dispatch",
        [
          Alcotest.test_case "same value as in-process" `Quick
            test_same_value_as_in_process;
          Alcotest.test_case "unregistered name is a Prim_error" `Quick
            test_unregistered_is_error;
          Alcotest.test_case "server-only refused, not executed" `Quick
            test_server_only_refused_not_executed;
          Alcotest.test_case "classification is data-driven" `Quick
            test_classification_data_driven;
          Alcotest.test_case "DB CRUD proxyable on real table" `Quick
            test_db_crud_not_server_only_on_real_table;
        ] );
    ]
```

116 image/test/auth_password_test.ml

```
(** Unit tests for password hashing (argon2id champion + legacy interim
    sha256 upgrade path) + token hash. *)

let () = Mirage_crypto_rng_unix.use_default ()

open Pricklypear_image_lib.Auth_password

let hash_roundtrip () =
  let h = hash_password "s3cret" in
  Alcotest.(check bool) "argon2id prefix" true
    (String.starts_with ~prefix:"$argon2id$" h);
  Alcotest.(check bool) "verify ok" true
    (verify_password ~stored:h "s3cret");
  Alcotest.(check bool) "verify bad" false
    (verify_password ~stored:h "wrong");
  Alcotest.(check bool) "fresh hash needs no upgrade" false
    (needs_upgrade h)

let distinct_salts () =
  let a = hash_password "same" in
  let b = hash_password "same" in
  Alcotest.(check bool) "different hashes" true (a <> b);
  Alcotest.(check bool) "both verify" true
    (verify_password ~stored:a "same"
     && verify_password ~stored:b "same")

let legacy_legacy_still_verifies () =
  (* Row produced by the interim sha256 scheme (pre-argon2id on the
     switch). Must still verify after the argon2id cutover -- and be
     flagged for the transparent upgrade. *)
  let salt_hex = hex_encode (Mirage_crypto_rng.generate 16) in
  let digest = sha256_hex (salt_hex ^ ":" ^ "interim-pass") in
  let stored = Printf.sprintf "sha256$%s$%s" salt_hex digest in
  Alcotest.(check bool) "legacy verifies" true
    (verify_password ~stored "interim-pass");
  Alcotest.(check bool) "legacy wrong-password rejected" false
    (verify_password ~stored "nope");
  Alcotest.(check bool) "legacy needs upgrade" true
    (needs_upgrade stored);
  let full = verify_full ~stored "interim-pass" in
  Alcotest.(check bool) "full: ok + needs_upgrade" true
    (full.ok && full.needs_upgrade)

let malformed_reject () =
  Alcotest.(check bool) "empty" false (verify_password ~stored:"" "x");
  Alcotest.(check bool) "garbage" false
    (verify_password ~stored:"not-a-hash" "x");
  Alcotest.(check bool) "partial sha256" false
    (verify_password ~stored:"sha256$ab" "x");
  Alcotest.(check bool) "corrupt argon2id" false
    (verify_password ~stored:"$argon2id$garbage" "x");
  Alcotest.(check bool) "corrupt needs no upgrade flag" false
    (needs_upgrade "$argon2id$garbage")

let secure_equal_tests () =
  Alcotest.(check bool) "eq" true (secure_equal "abc" "abc");
  Alcotest.(check bool) "neq" false (secure_equal "abc" "abd");
  Alcotest.(check bool) "len" false (secure_equal "ab" "abc")

let token_hash_deterministic () =
  let h1 = sha256_hex "tok" in
  let h2 = sha256_hex "tok" in
  Alcotest.(check string) "det" h1 h2;
  Alcotest.(check bool) "diff" true (h1 <> sha256_hex "tok2")

let () =
  Alcotest.run "auth_password"
    [ ("hash",
       [ ("roundtrip argon2id", `Quick, hash_roundtrip);
         ("distinct salts", `Quick, distinct_salts);
         ("legacy still verifies + upgrades", `Quick,
```

```
          legacy_legacy_still_verifies);
        ("malformed reject", `Quick, malformed_reject);
        ("secure_equal", `Quick, secure_equal_tests);
        ("token hash det", `Quick, token_hash_deterministic) ]) ]
```

117 image/test/auth_throttle_test.ml

```
(* auth_throttle_test -- pure alcotest for Auth_throttle (i-1787970741).

   Pins the three env knobs so window/max/backoff are deterministic
   regardless of ambient env. The clock is injected (a float), so
   nothing here sleeps. Each case uses its own key prefix: the table is
   module-global. *)

module T = Pricklypear_image_lib.Auth_throttle

let pin_env () =
  Unix.putenv "PP_LOGIN_MAX_FAILURES" "10";
  Unix.putenv "PP_LOGIN_WINDOW_SECS" "900";
  Unix.putenv "PP_LOGIN_BACKOFF_BASE_SECS" "30"

let maxf = 10
let base = 30.0
let cap = 900.0

let check_bool name expected actual =
  Alcotest.(check bool) name expected actual

(* Record [n] failures for [key] at times 0.0 .. n-1. *)
let record_n key n =
  for i = 0 to n - 1 do
    T.record_failure key (float_of_int i)
  done

let clean_key_admits () =
  check_bool "clean key admits" true (T.admit "t-clean" 100.0)

let below_threshold_never_blocks () =
  let key = "t-below" in
  record_n key (maxf - 1);
  (* 9 failures inside the window: still admitted on every attempt. *)
  check_bool "9 failures admit" true (T.admit key 1000.0);
  check_bool "still admits later" true (T.admit key 1500.0)

let threshold_engages_backoff () =
  let key = "t-threshold" in
  record_n key maxf;
  (* 10 failures, last at t=9, first lockout = 30s -> locked until 39. *)
  check_bool "locked right after" false (T.admit key 20.0);
  check_bool "locked just before deadline" false
    (T.admit key 38.999);
  check_bool "admitted at deadline" true (T.admit key 39.0)

let backoff_doubles () =
  let key = "t-double" in
  record_n key (maxf + 1);
  (* 11th failure -> 60s from t=10. *)
  check_bool "11th failure locks 60s" false (T.admit key 69.0);
  check_bool "admitted at 70" true (T.admit key 70.0);
  (* A further failure now (t=70) -> 12 in window -> 120s from t=70. *)
  T.record_failure key 70.0;
  check_bool "12th failure locks 120s" false (T.admit key 189.0);
  check_bool "admitted at 190" true (T.admit key 190.0)

let backoff_capped () =
  let key = "t-cap" in
  record_n key 30;
  (* 20 failures past the threshold: backoff is capped at 900s from
     t=29 (deadline 929). At t=500 the window still holds the whole
     burst, so the cap is what keeps the key locked. *)
  check_bool "capped lockout holds" false (T.admit key 500.0);
  check_bool "capped lockout still holds near deadline" false
    (T.admit key 900.0);
  (* Once the burst slides out of the 900s window the key frees. *)
  check_bool "admitted after window clears" true
    (T.admit key (29.0 +. cap))

let sliding_window_expires () =
```

```
  let key = "t-window" in
  (* 9 failures at t=0 and one at t=895: nominal lockout until 925, but
     by t=920 the nine old failures slid out of the 900s window. *)
  for _ = 1 to maxf - 1 do
    T.record_failure key 0.0
  done;
  T.record_failure key 895.0;
  check_bool "locked inside window" false (T.admit key 500.0);
  check_bool "expired failures free the key" true
    (T.admit key 920.0)

let clear_resets () =
  let key = "t-clear" in
  record_n key maxf;
  check_bool "locked before clear" false (T.admit key 20.0);
  T.clear key;
  check_bool "admitted after clear" true (T.admit key 20.0);
  (* A fresh single failure does not re-lock. *)
  T.record_failure key 21.0;
  check_bool "single failure admits" true (T.admit key 22.0)

let table_bounded () =
  (* Flood well past the bound; the table must evict, not grow. *)
  for i = 0 to 4199 do
    T.record_failure (Printf.sprintf "t-flood-%d" i) 1000.0
  done;
  Alcotest.(check int) "table capped at 4096" T.max_entries
    (T.size ());
  (* Evicted keys are forgotten; fresh keys still admit. *)
  check_bool "evicted key admits" true (T.admit "t-flood-0" 1001.0);
  check_bool "fresh key admits" true (T.admit "t-flood-fresh" 1001.0)

let () =
  pin_env ();
  Alcotest.run "auth_throttle"
    [ ( "throttle",
        [ ("clean key admits", `Quick, clean_key_admits);
          ("below threshold never blocks", `Quick,
   below_threshold_never_blocks);
          ("threshold engages backoff", `Quick, threshold_engages_backoff);
          ("backoff doubles", `Quick, backoff_doubles);
          ("backoff capped at 900s", `Quick, backoff_capped);
          ("sliding window expires", `Quick, sliding_window_expires);
          ("clear resets", `Quick, clear_resets);
          ("table bounded at 4096", `Quick, table_bounded) ]) ]
```

118 image/test/branch_generation_test.ml

```
(** Branch generation counter tests (remote-eval.borg caching subsection).

    Needs Postgres (scripts/dev.sh, migration 017). Skips cleanly when the
    DB or the branch_generations table is unreachable.

    Acceptance (caching verify clauses):
      - a write in the client's branch ancestry bumps a generation counter;
        the client's next cache-check observes the bump
      - a checkout to a different branch is a distinct (branch, generation)
        key (branch is the namespace)
      - with no writes the cache-check is one cheap unchanged-generation
     query
*)

module S = Pricklypear_image_lib.Store

let main_id = "00000000-0000-0000-0000-000000000001"

let with_store f =
  Eio_main.run @@ fun env ->
  Lwt_eio.with_event_loop ~clock:(Eio.Stdenv.clock env) @@ fun _ ->
  Eio.Switch.run @@ fun sw ->
  let ctx : Pricklypear_image_lib.Io_eio.Ctx.t =
    {
      sw;
      net = (Eio.Stdenv.net env :> [ `Generic ] Eio.Net.ty Eio.Std.r);
    }
  in
  Pricklypear_image_lib.Io_eio.with_ctx ctx (fun () ->
      let cfg = S.config_from_env () in
      let store = S.make ~cfg ~branch_id:main_id () in
      S.connect store;
      match S.conn_of store with
      | None ->
          Printf.printf
            "SKIP: no postgres (set PP_DB_* / run scripts/dev.sh)\n%!"
      | Some _ -> f store)

(* The generation helper returns None when pre-017 / unreachable. *)
let gen_or_fail store ~branch_id =
  match S.current_generation store ~branch_id with
  | Some g -> g
  | None ->
      Alcotest.fail
        "branch_generations missing (restart scripts/dev.sh to apply \
         migration 017)"

let int_of_g (g : int64) : int = Int64.to_int g

let uniq p = Printf.sprintf "gen_%s_%d" p (Random.int 1_000_000)

let write_fn store ~branch_id ~name =
  S.with_branch store ~branch_id (fun () ->
      S.persist_function store name
        (Pricklypear_lisp.Ast.Lit (Pricklypear_lisp.Ast.VInt 1L))
        ~doc:(Some "gen-test") "tester")

let test_write_in_ancestry_bumps () =
  with_store (fun store ->
      let _ = gen_or_fail store ~branch_id:main_id in
      let before = int_of_g (gen_or_fail store ~branch_id:main_id) in
      (* write on main = ancestor of every branch *)
      let name = uniq "fn" in
      write_fn store ~branch_id:main_id ~name;
      let after = int_of_g (gen_or_fail store ~branch_id:main_id) in
      Alcotest.(check bool) "main generation bumps on write" true (after >
     before))

let test_checkout_different_branch_is_distinct_key () =
  with_store (fun store ->
      let main_g = int_of_g (gen_or_fail store ~branch_id:main_id) in
```

```
      let child = uniq "b" in
      match
        S.fork_branch store ~parent_id:main_id ~owner:"gen_test"
    ~child_name:child
      with
      | None -> Alcotest.fail "fork_branch failed"
      | Some child_id ->
          let child_g = int_of_g (gen_or_fail store ~branch_id:child_id) in
          (* a write on the child must not change the parent's key *)
          let before_main = int_of_g (gen_or_fail store ~branch_id:main_id)
    in
          let name = uniq "cn" in
          write_fn store ~branch_id:child_id ~name;
          let after_main = int_of_g (gen_or_fail store ~branch_id:main_id)
    in
          Alcotest.(check int) "parent key unchanged by child write"
            before_main after_main;
          (* child observed the write *)
          let child_after = int_of_g (gen_or_fail store
    ~branch_id:child_id) in
          Alcotest.(check bool) "child generation bumped" true
            (child_after > child_g);
          (* checkout key differs: branch is the namespace *)
          Alcotest.(check bool)
            "checkout key differs (branch is the namespace)" true
            (main_g <> child_after))

let test_no_write_unchanged_cheap () =
  with_store (fun store ->
      let _ = gen_or_fail store ~branch_id:main_id in
      let g1 = S.current_generation store ~branch_id:main_id in
      let g2 = S.current_generation store ~branch_id:main_id in
      Alcotest.(check bool) "unchanged generation with no writes" true
        (g1 <> None && g1 = g2))

let test_type_and_route_write_bump () =
  with_store (fun store ->
      match S.conn_of store with
      | None -> ()
      | Some conn ->
          let before = int_of_g (gen_or_fail store ~branch_id:main_id) in
          let tname = uniq "ty" in
          let _ =
            S.exec conn
              ~params:
                [ Pgx.Value.of_string main_id
                ; Pgx.Value.of_string tname
                ; Pgx.Value.of_string "[]"
                ; Pgx.Value.of_string "tester"
                ]
              "INSERT INTO type_defs (branch_id,name,fields,author) VALUES \
               ($1,$2,$3::jsonb,$4) ON CONFLICT (branch_id,name) DO UPDATE \
               SET fields=EXCLUDED.fields"
          in
          let after_type = int_of_g (gen_or_fail store ~branch_id:main_id)
    in
          Alcotest.(check bool) "type write bumps" true (after_type >
    before);
          let rname = "/gen_" ^ uniq "r" in
          let _ =
            S.exec conn
              ~params:
                [ Pgx.Value.of_string main_id
                ; Pgx.Value.of_string rname
                ; Pgx.Value.of_string "gen_handler"
                ]
              "INSERT INTO view_bindings (branch_id,route,handler) VALUES \
               ($1,$2,$3) ON CONFLICT (branch_id,route) DO UPDATE SET \
               handler=EXCLUDED.handler"
          in
          let after_route = int_of_g (gen_or_fail store ~branch_id:main_id)
    in
          Alcotest.(check bool) "route write bumps" true
```

```
            (after_route > after_type))

let test_data_row_write_bumps_root () =
  with_store (fun store ->
      match S.conn_of store with
      | None -> ()
      | Some conn ->
          let before = int_of_g (gen_or_fail store ~branch_id:main_id) in
          let tn = uniq "dt" in
          let _ =
            S.exec conn
              ~params:
                [ Pgx.Value.of_string tn
                ; Pgx.Value.of_string "{}"
                ; Pgx.Value.of_string "tester"
                ]
              "INSERT INTO data_rows (type_name,fields,author) VALUES \
               ($1,$2::jsonb,$3)"
          in
          let after = int_of_g (gen_or_fail store ~branch_id:main_id) in
          Alcotest.(check bool) "shared data write bumps root" true
            (after > before))

let () =
  Alcotest.run "branch_generation"
    [
      ( "generation",
        [
          Alcotest.test_case "write in ancestry bumps" `Quick
            test_write_in_ancestry_bumps;
          Alcotest.test_case "checkout to another branch is a distinct key"
            `Quick test_checkout_different_branch_is_distinct_key;
          Alcotest.test_case "no writes -> unchanged cheap check" `Quick
            test_no_write_unchanged_cheap;
          Alcotest.test_case "type + route writes bump" `Quick
            test_type_and_route_write_bump;
          Alcotest.test_case "shared data write bumps root" `Quick
            test_data_row_write_bumps_root;
        ] );
    ]
```

119 image/test/branch_status_test.ml

```
(** Branch status (branching.branch-status): git-style
    clean/ahead-new/ahead/behind/diverged/synced classification over the
    tree-time substrate (tree_versions) keyed on base_gen.
    Needs Postgres (scripts/dev.sh). Skips if unreachable.

    Verifies: fork records base_gen = parent head gen; empty fork reads
    clean; own new fn reads ahead-new; post-promote pin reads synced;
    parent-only move reads behind; both-sides move reads diverged; merge
    upserts only ahead/diverged and ADVANCES base_gen (post-merge the
    branch reads synced, not diverged); status agrees with diff_branches
    head-vs-head on the diverged fixture. *)

module S = Pricklypear_image_lib.Store

let main_id = "00000000-0000-0000-0000-000000000001"

let with_store f =
  Eio_main.run @@ fun env ->
  Lwt_eio.with_event_loop ~clock:(Eio.Stdenv.clock env) @@ fun _ ->
  Eio.Switch.run @@ fun sw ->
  let ctx : Pricklypear_image_lib.Io_eio.Ctx.t =
    {
      sw;
      net = (Eio.Stdenv.net env :> [ `Generic ] Eio.Net.ty Eio.Std.r);
    }
  in
  Pricklypear_image_lib.Io_eio.with_ctx ctx (fun () ->
      let cfg = S.config_from_env () in
      let store = S.make ~cfg ~branch_id:main_id () in
      S.connect store;
      match S.conn_of store with
      | None ->
          Printf.printf
            "SKIP: no postgres (set PP_DB_* / run scripts/dev.sh)\n%!"
      | Some _ -> f store)

let expr_of_int n =
  Pricklypear_lisp.Ast.Lit (Pricklypear_lisp.Ast.VInt (Int64.of_int n))

let uniq p = Printf.sprintf "%s_%d" p (Random.int 1_000_000)

let status_of rows ~name =
  match
    List.find_opt
      (fun (r : S.branch_status_row) ->
        r.S.st_kind = S.DiffFn && r.S.st_name = name)
      rows
  with
  | Some r -> Some r.S.st_status
  | None -> None

let base_gen_of store ~branch_id =
  match
    S.exec (Option.get (S.conn_of store))
      ~params:[ Pgx.Value.of_string branch_id ]
      "SELECT COALESCE(base_gen, 0)::text FROM branches WHERE id = $1"
  with
  | [ [ c ] ] -> (
      match Pgx.Value.to_string c with
      | Some s -> Int64.of_string s
      | None -> 0L)
  | _ -> 0L

let parent_gen_of store ~branch_id =
  match
    S.exec (Option.get (S.conn_of store))
      ~params:[ Pgx.Value.of_string branch_id ]
      "SELECT COALESCE((SELECT generation FROM branch_generations \
                        WHERE branch_id = parent_id), 0)::text \
       FROM branches WHERE id = $1"
  with
```

```
  | [ [ c ] ] -> (
      match Pgx.Value.to_string c with
      | Some s -> Int64.of_string s
      | None -> 0L)
  | _ -> 0L

let cleanup_fn store ~branch_id ~name =
  ignore (S.delete_function_on_branch store ~branch_id ~name)

let cleanup_branch store ~branch_id =
  ignore (
    S.exec
      (Option.get (S.conn_of store))
      ~params:[ Pgx.Value.of_string branch_id ]
      "DELETE FROM view_bindings WHERE branch_id = $1");
  ignore (
    S.exec
      (Option.get (S.conn_of store))
      ~params:[ Pgx.Value.of_string branch_id ]
      "DELETE FROM type_defs WHERE branch_id = $1");
  ignore (
    S.exec
      (Option.get (S.conn_of store))
      ~params:[ Pgx.Value.of_string branch_id ]
      "DELETE FROM function_defs WHERE branch_id = $1");
  ignore (
    S.exec
      (Option.get (S.conn_of store))
      ~params:[ Pgx.Value.of_string branch_id ]
      "DELETE FROM branches WHERE id = $1")

(** The full lifecycle: fork -> ahead-new -> diverged -> merge advances
    base_gen -> synced. Also covers clean (empty fork), behind (parent
    moves, child does not), synced-by-promote, and the diff_branches
    equivalence on the diverged fixture. *)
let test_status_lifecycle () =
  with_store (fun store ->
      match S.conn_of store with
      | None -> ()
      | Some _ ->
          let suffix = string_of_int (Random.int 1_000_000) in
          let child_name = "bst_child_" ^ suffix in
          let shared = "bst_shared_" ^ suffix in
          let newfn = "bst_new_" ^ suffix in
          store.branch_id <- main_id;
          (* main holds a shared def BEFORE the fork *)
          S.persist_function store shared (expr_of_int 1) "tester";
          (match
             S.fork_branch store ~parent_id:main_id ~owner:"bst_test"
               ~child_name
           with
           | None -> Alcotest.fail "fork failed"
           | Some child_id ->
               (* S1: fork records base_gen = parent head gen *)
               Alcotest.(check bool) "fork sets base_gen = parent head gen"
                 true
                 (Int64.compare (base_gen_of store ~branch_id:child_id)
                    (parent_gen_of store ~branch_id:child_id)
                 = 0);
               (* clean: empty fork, shared def inherited, parent unmoved *)
               let rows = S.branch_status store ~branch_id:child_id in
               Alcotest.(check bool) "empty fork: shared fn clean" true
                 (status_of rows ~name:shared = Some S.SClean);
               (* ahead-new: child adds a fresh fn *)
               store.branch_id <- child_id;
               S.persist_function store newfn (expr_of_int 7) "tester";
               let rows = S.branch_status store ~branch_id:child_id in
               Alcotest.(check bool) "own new fn = ahead-new" true
                 (status_of rows ~name:newfn = Some S.SAheadNew);
               (* diverged: both sides move the shared fn *)
               S.persist_function store shared (expr_of_int 2) "tester";
               store.branch_id <- main_id;
               Unix.sleepf 0.02;
```

```
145               S.persist_function store shared (expr_of_int 3) "tester";
146               let rows = S.branch_status store ~branch_id:child_id in
147               Alcotest.(check bool) "both moved = diverged" true
148                 (status_of rows ~name:shared = Some S.SDiverged);
149               (* equivalence: diff_branches reports the same divergence
150                   head-vs-head (shared modified on both sides) *)
151               let drows =
152                 S.diff_branches store ~from_id:child_id ~to_id:main_id
153               in
154               Alcotest.(check bool) "diff agrees: shared modified" true
155                 (List.exists
156                    (fun (r : S.branch_diff_row) ->
157                      r.S.diff_name = shared && r.S.diff_status =
    S.DiffModified)
158                    drows);
159               (* behind: parent moves ANOTHER fn the child only inherits *)
160               let behind_fn = "bst_behind_" ^ suffix in
161               S.persist_function store behind_fn (expr_of_int 1) "tester";
162               Unix.sleepf 0.02;
163               S.persist_function store behind_fn (expr_of_int 2) "tester";
164               let rows = S.branch_status store ~branch_id:child_id in
165               Alcotest.(check bool) "parent-only move = behind" true
166                 (status_of rows ~name:behind_fn = Some S.SBehind);
167               (* synced-by-promote: promote pins it; base_gen NOT advanced
    *)
168               ignore (
169                 S.promote_functions store ~source_id:child_id
170                   ~target_id:main_id ~names:[ newfn ]);
171               let rows = S.branch_status store ~branch_id:child_id in
172               Alcotest.(check bool) "post-promote pin = synced" true
173                 (status_of rows ~name:newfn = Some S.SSynced);
174               (* merge child -> main: only ahead/diverged names upsert;
175                  base_gen advances; the diverged name reads synced *)
176               let base_before = base_gen_of store ~branch_id:child_id in
177               (match
178                  S.merge_branch store ~source_id:child_id
179                    ~target_id:main_id
180                with
181                | None -> Alcotest.fail "merge failed"
182                | Some res ->
183                    Alcotest.(check bool) "merge names diverged" true
184                      (List.mem shared res.S.conflicts));
185               Alcotest.(check bool) "merge advanced base_gen" true
186                 (Int64.compare (base_gen_of store ~branch_id:child_id)
187                    base_before
188                 > 0);
189               let rows = S.branch_status store ~branch_id:child_id in
190               Alcotest.(check bool) "post-merge diverged = synced" true
191                 (status_of rows ~name:shared = Some S.SSynced);
192               Alcotest.(check bool) "post-merge pin still synced" true
193                 (status_of rows ~name:newfn = Some S.SSynced);
194               (* cleanup *)
195               cleanup_fn store ~branch_id:main_id ~name:shared;
196               cleanup_fn store ~branch_id:main_id ~name:behind_fn;
197               cleanup_branch store ~branch_id:child_id;
198               ()))
199
200 (** The motivating bug: today's status called every post-fork row an
201     "override" even when bodies are identical. A pinned fn whose body is
202     IDENTICAL to main's must read synced, not diverged/ahead. *)
203 let test_identical_pin_is_synced () =
204   with_store (fun store ->
205       match S.conn_of store with
206       | None -> ()
207       | Some _ ->
208           let suffix = string_of_int (Random.int 1_000_000) in
209           let child_name = "bst_pin_" ^ suffix in
210           let fn = "bst_pin_fn_" ^ suffix in
211           store.branch_id <- main_id;
212           S.persist_function store fn (expr_of_int 42) "tester";
213           (match
214              S.fork_branch store ~parent_id:main_id ~owner:"bst_test"
215                ~child_name
```

```
          with
          | None -> Alcotest.fail "fork failed"
          | Some child_id ->
              (* child pins the SAME body *)
              store.branch_id <- child_id;
              S.persist_function store fn (expr_of_int 42) "tester";
              let rows = S.branch_status store ~branch_id:child_id in
              Alcotest.(check bool) "identical pin = synced" true
                (status_of rows ~name:fn = Some S.SSynced);
              Alcotest.(check bool) "identical pin is own" true
                (List.exists
                   (fun (r : S.branch_status_row) ->
                     r.S.st_name = fn && r.S.st_own)
                   rows);
              (* unbind (delete the pin) -> back to clean inheritance *)
              ignore (
                S.delete_function_on_branch store ~branch_id:child_id
                  ~name:fn);
              let rows = S.branch_status store ~branch_id:child_id in
              Alcotest.(check bool) "unbound pin = clean" true
                (status_of rows ~name:fn = Some S.SClean);
              cleanup_fn store ~branch_id:main_id ~name:fn;
              cleanup_branch store ~branch_id:child_id;
              ()))

let () =
  Random.self_init ();
  Alcotest.run "branch-status"
    [ ( "db"
      , [ Alcotest.test_case "lifecycle" `Quick test_status_lifecycle
        ; Alcotest.test_case "identical pin synced" `Quick
            test_identical_pin_is_synced
        ] )
    ]
```

120 image/test/code_history_tree_test.ml

```
(** tree-time.projections Phase 3: branching.code-history / code-diff now read
    from the versioned tree (tree_versions) instead of the 012 before-image
    table, keeping signatures UNCHANGED.

    SEMANTIC NOTE (tree-time.coexistence-plan Phase 3): 012 stored the OLD
    before-image per UPDATE/DELETE (never on the first INSERT). tree_versions
    stores the post-write head payload at each gen, so a tree-backed history
    read reports ONLY the rows STRICTLY BELOW HEAD -- the previous states,
    newest-first "read the version row at gen-1" -- and the live head row is
    never history (an insert-only fn has no history until rewritten/deleted).
    The 012 table remains the fallback source only for pre-shadow (pre-019)
    names, so no historical depth is lost during the coexistence window.

    These tests exercise the tree-backed path directly (Store.list_code_history
    routes to tree_versions when the code path has version rows) and confirm
    code-diff still produces a diff against the nth-newest history entry.

    Needs Postgres (scripts/dev.sh). Skips cleanly when unreachable.
*)

module S = Pricklypear_image_lib.Store
module T = Pricklypear_image_lib.Tree_store
open Pricklypear_lisp.Ast

let main_id = "00000000-0000-0000-0000-000000000001"

let with_store f =
  Eio_main.run @@ fun env ->
  Lwt_eio.with_event_loop ~clock:(Eio.Stdenv.clock env) @@ fun _ ->
  Eio.Switch.run @@ fun sw ->
  let ctx : Pricklypear_image_lib.Io_eio.Ctx.t =
    {
      sw;
      net = (Eio.Stdenv.net env :> [ `Generic ] Eio.Net.ty Eio.Std.r);
    }
  in
  Pricklypear_image_lib.Io_eio.with_ctx ctx (fun () ->
      let cfg = S.config_from_env () in
      let store = S.make ~cfg ~branch_id:main_id () in
      S.connect store;
      match S.conn_of store with
      | None ->
          Printf.printf
            "SKIP: no postgres (set PP_DB_* / run scripts/dev.sh)\n%!"
      | Some _ -> f store)

let uniq p = Printf.sprintf "cht_%s_%d_%d" p (Unix.getpid ()) (Random.int 1_000_000_000)

(* A tree-backed code-history entry has no gen field, so newest-first is
   asserted by body content: the newest entry's body carries the last-written
   definition payload (the version row's inner 'body' key). *)
let contains_sub s sub =
  let ls = String.length s and lb = String.length sub in
  let rec go i = if i + lb > ls then false
    else if String.sub s i lb = sub then true
    else go (i + 1)
  in
  lb <= ls && (if lb = 0 then true else go 0)

let body_contains (e : S.code_history_entry) ~(needle : string) : bool =
  match e.body with
  | Some b -> contains_sub b needle
  | None -> false

(* Direct tree writes (T.upsert_fn) are shadowed by migration 019 into
```

```
   tree_versions, so Store.list_code_history routes to the tree-backed path
   (rel=function_defs, op=UPDATE, author=tester, newest-first). *)
let test_tree_backed_history () =
  with_store (fun store ->
      match S.conn_of store with
      | None -> ()
      | Some conn ->
          let name = uniq "fn" in
          (* three writes -> the current head is the 3rd; history is the 2
             STRICTLY-BELOW-HEAD rows (states before the latest change),
             newest-first = [v2, v1]. Match 012's before-image semantic (an
             insert-only fn has NO history until rewritten). *)
          T.upsert_fn conn ~branch_id:main_id ~name
            ~body_json:"{\"params\":[],\"body\":1}" ~author:"tester"
    ~doc:None;
          T.upsert_fn conn ~branch_id:main_id ~name
            ~body_json:"{\"params\":[],\"body\":2}" ~author:"tester"
    ~doc:None;
          T.upsert_fn conn ~branch_id:main_id ~name
            ~body_json:"{\"params\":[],\"body\":3}" ~author:"tester"
    ~doc:None;
          let hist = S.list_code_history store ~branch_id:main_id ~name in
          (match hist with
           | [ n1; n2 ] ->
               List.iter
                 (fun e ->
                   Alcotest.(check string) "rel is function_defs"
    "function_defs" e.S.rel;
                   Alcotest.(check string) "op is UPDATE" "UPDATE" e.S.op;
                   Alcotest.(check (option string)) "author is tester"
                     (Some "tester") e.S.author)
                 [ n1; n2 ];
               (* newest-first: n1 = state just before the latest change =
    v2; n2 = v1 *)
               Alcotest.(check bool) "newest body = 2" true
                 (body_contains n1 ~needle:"\"body\":2");
               Alcotest.(check bool) "oldest body = 1" true
                 (body_contains n2 ~needle:"\"body\":1");
               Alcotest.(check bool) "newest not tombstone (body present)"
    true
                 (n1.body <> None)
           | _ -> Alcotest.fail "expected exactly 2 tree-backed history
    rows");
          (* code-diff vs nth-newest still produces a diff *)
          (match S.code_diff store ~branch_id:main_id ~name ~n:1 with
           | Ok d ->
               Alcotest.(check bool) "code-diff non-empty" true (d <> "")
           | Error e -> Alcotest.fail ("code-diff failed: " ^ e));
          (* code-diff with an out-of-range n reports an error, not a crash
    *)
          (match S.code_diff store ~branch_id:main_id ~name ~n:999 with
           | Ok _ -> Alcotest.fail "expected out-of-range error"
           | Error _ -> ());
          (* cleanup: tombstone the fn *)
          ignore (T.delete_fn conn ~branch_id:main_id ~name))

let () =
  Alcotest.run "code_history_tree"
    [
      ( "tree-backed-code-history",
        [
          Alcotest.test_case "code-history/code-diff read from
    tree_versions" `Quick
            test_tree_backed_history;
        ] );
    ]
```

121 image/test/code_restore_test.ml

```
(** tree-time.consumers: code-restore -- the code-path analogue of (undo)
    (which is datum-path only). Reads the versioned state at head-n and
    re-writes it back through the code write surface (upsert_fn / upsert_type
    / upsert_route), so the reverse mirror re-populates function_defs / type_defs / view_bindings and the restore itself is versioned + redoable.
    "code-restore = gen-read + define" (the 012 lossless-redefine pattern).

    Below-head semantic matches code-history's "read the version row at gen-1": n=1 restores the state just BEFORE the latest change; an insert-only code node (nothing below head) has no state to restore.

    These tests exercise Store.code_restore directly. Needs Postgres (scripts/dev.sh); skips cleanly when unreachable.
*)

module S = Pricklypear_image_lib.Store
module T = Pricklypear_image_lib.Tree_store

let main_id = "00000000-0000-0000-0000-000000000001"

let with_store f =
  Eio_main.run @@ fun env ->
  Lwt_eio.with_event_loop ~clock:(Eio.Stdenv.clock env) @@ fun _ ->
  Eio.Switch.run @@ fun sw ->
  let ctx : Pricklypear_image_lib.Io_eio.Ctx.t =
    {
      sw;
      net = (Eio.Stdenv.net env :> [ `Generic ] Eio.Net.ty Eio.Std.r);
    }
  in
  Pricklypear_image_lib.Io_eio.with_ctx ctx (fun () ->
      let cfg = S.config_from_env () in
      let store = S.make ~cfg ~branch_id:main_id () in
      S.connect store;
      match S.conn_of store with
      | None ->
          Printf.printf
            "SKIP: no postgres (set PP_DB_* / run scripts/dev.sh)\n%!"
      | Some _ -> f store)

let uniq p = Printf.sprintf "crr_%s_%d_%d" p (Unix.getpid ()) (Random.int 1_000_000_000)

let contains_sub s sub =
  let ls = String.length s and lb = String.length sub in
  let rec go i = if i + lb > ls then false
    else if String.sub s i lb = sub then true
    else go (i + 1)
  in
  lb <= ls && (if lb = 0 then true else go 0)

let assert_body store ~branch_id ~name ~needle ~label =
  (* live_code_body_text reads the scaffolding table (function_defs / type_defs / view_bindings) -- the authoritative post-reverse-mirror read. *)
  let live =
    Option.value (S.live_code_body_text store ~branch_id ~name) ~default:""
  in
  Alcotest.(check bool) label true (contains_sub live needle)

(* Parse a top-level string field out of a JSON object text (robust to JSON reordering/relocating from canonicalization). *)
let json_field (s : string) (key : string) : string option =
  match Yojson.Safe.from_string s with
  | `Assoc kv -> (
      match List.assoc_opt key kv with
      | Some (`String x) -> Some x
      | Some (`Int x) -> Some (string_of_int x)
      | Some (`Intlit x) -> Some x
```

```
      | Some (`Float x) -> Some (Printf.sprintf "%g" x)
      | _ -> None )
  | _ -> None

(* fn: 3 writes -> restore n=1 rewinds to the state just before the latest
   change (body 2); n=2 to body 1; out-of-range n is a clean error, not a
   crash; the restore itself becomes the new live + scaffolding head. *)
let test_fn_restore () =
  with_store (fun store ->
      match S.conn_of store with
      | None -> ()
      | Some conn ->
          let name = uniq "fn" in
          T.upsert_fn conn ~branch_id:main_id ~name
            ~body_json:"{\"params\":[],\"body\":1}" ~author:"tester"
    ~doc:None;
          T.upsert_fn conn ~branch_id:main_id ~name
            ~body_json:"{\"params\":[],\"body\":2}" ~author:"tester"
    ~doc:None;
          T.upsert_fn conn ~branch_id:main_id ~name
            ~body_json:"{\"params\":[],\"body\":3}" ~author:"tester"
    ~doc:None;
          let r = S.code_restore store ~branch_id:main_id ~name
    ~author:"tester" () in
          Alcotest.(check bool) "n=1 returns restored-fn" true
            (contains_sub r "restored fn");
          let live1 =
            Option.value
              (S.live_code_body_text store ~branch_id:main_id ~name)
              ~default:""
          in
          Alcotest.(check (option string)) "n=1 rewinds body to 2"
            (Some "2") (json_field live1 "body");
          (* n=2 on a FRESH 3-write chain (no intervening restore):
    restores the
           state 2 back = body 1. Restore-by-write semantics: each
    restore is a
           NEW write, so n counts back from the live head through any
    prior
           restores -- reuse of [name] after the n=1 restore would
    re-target the
           restored head. *)
          let name3 = uniq "fn3" in
          T.upsert_fn conn ~branch_id:main_id ~name:name3
            ~body_json:"{\"params\":[],\"body\":1}" ~author:"tester"
    ~doc:None;
          T.upsert_fn conn ~branch_id:main_id ~name:name3
            ~body_json:"{\"params\":[],\"body\":2}" ~author:"tester"
    ~doc:None;
          T.upsert_fn conn ~branch_id:main_id ~name:name3
            ~body_json:"{\"params\":[],\"body\":3}" ~author:"tester"
    ~doc:None;
          let r2 =
            S.code_restore store ~branch_id:main_id ~name:name3
    ~author:"tester" ~n:2 ()
          in
          Alcotest.(check bool) "n=2 returns restored-fn" true
            (contains_sub r2 "restored fn");
          let live2 =
            Option.value
              (S.live_code_body_text store ~branch_id:main_id ~name:name3)
              ~default:""
          in
          Alcotest.(check (option string)) "n=2 rewinds body to 1"
            (Some "1") (json_field live2 "body");
          let r3 =
            S.code_restore store ~branch_id:main_id ~name ~author:"tester"
    ~n:999 ()
          in
          Alcotest.(check bool) "out-of-range n is clean error" true
            (contains_sub r3 "no state");
          (* insert-only fn (nothing below head) has no state to restore *)
          let name2 = uniq "fn2" in
```

```
128           T.upsert_fn conn ~branch_id:main_id ~name:name2
129             ~body_json:"{\"params\":[],\"body\":1}" ~author:"tester"
      ~doc:None;
130           let r4 =
131             S.code_restore store ~branch_id:main_id ~name:name2
      ~author:"tester" ()
132           in
133           Alcotest.(check bool) "insert-only has no state" true
134             (contains_sub r4 "no state");
135           (* unknown name errors cleanly *)
136           let r5 =
137             S.code_restore store ~branch_id:main_id
138               ~name:(Printf.sprintf "%s_%d" name 999999) ~author:"tester" ()
139           in
140           Alcotest.(check bool) "unknown name errors cleanly" true
141             (contains_sub r5 "no such code path");
142           ignore (T.delete_fn conn ~branch_id:main_id ~name);
143           ignore (T.delete_path conn ~path:(T.fn_path ~branch_id:main_id
      ~name:name2)
144                     ~recursive:false);
145           ignore (T.delete_path conn ~path:(T.fn_path ~branch_id:main_id
      ~name:name3)
146                     ~recursive:false))
147
148 (* type: 3 writes -> restore n=1 rewinds the fields to the state before the
149    latest change. *)
150 let test_type_restore () =
151   with_store (fun store ->
152       match S.conn_of store with
153       | None -> ()
154       | Some conn ->
155           let name = uniq "ty" in
156           let f i = Printf.sprintf "{\"fields\":[\"a%d\"]}" i in
157           T.upsert_type conn ~branch_id:main_id ~name ~fields_json:(f 1)
158             ~author:"tester";
159           T.upsert_type conn ~branch_id:main_id ~name ~fields_json:(f 2)
160             ~author:"tester";
161           T.upsert_type conn ~branch_id:main_id ~name ~fields_json:(f 3)
162             ~author:"tester";
163           let r = S.code_restore store ~branch_id:main_id ~name
      ~author:"tester" () in
164           Alcotest.(check bool) "type n=1 returns restored-type" true
165             (contains_sub r "restored type");
166           assert_body store ~branch_id:main_id ~name ~needle:"a2"
167             ~label:"type n=1 rewinds fields to a2";
168           ignore
169             (T.delete_path conn
170                ~path:(T.type_path ~branch_id:main_id ~name)
      ~recursive:false))
171
172 (* route: 3 writes -> restore n=1 rewinds handler + visibility to the state
173    before the latest change. *)
174 let test_route_restore () =
175   with_store (fun store ->
176       match S.conn_of store with
177       | None -> ()
178       | Some conn ->
179           let name = uniq "rt" in
180           let h i = Printf.sprintf "handler%d" i in
181           T.upsert_route conn ~branch_id:main_id ~route:name ~handler:(h 1)
182             ~visibility:"public";
183           T.upsert_route conn ~branch_id:main_id ~route:name ~handler:(h 2)
184             ~visibility:"public";
185           T.upsert_route conn ~branch_id:main_id ~route:name ~handler:(h 3)
186             ~visibility:"private";
187           let r = S.code_restore store ~branch_id:main_id ~name
      ~author:"tester" () in
188           Alcotest.(check bool) "route n=1 returns restored-route" true
189             (contains_sub r "restored route");
190           let live =
191             Option.value
192               (S.live_code_body_text store ~branch_id:main_id ~name)
193               ~default:""
```

```
          in
          Alcotest.(check (option string)) "route n=1 rewinds handler to handler2"
            (Some "handler2") (json_field live "handler");
          Alcotest.(check (option string)) "route n=1 rewinds visibility to public"
            (Some "public") (json_field live "visibility");
          ignore
            (T.delete_path conn
               ~path:(T.route_path ~branch_id:main_id ~route:name)
               ~recursive:false))

let () =
  Alcotest.run "code_restore"
    [
      ("fn", [ Alcotest.test_case "fn restore-by-write n=1/n=2" `Quick test_fn_restore ]);
      ("type", [ Alcotest.test_case "type restore-by-write n=1" `Quick test_type_restore ]);
      ("route", [ Alcotest.test_case "route restore-by-write n=1" `Quick test_route_restore ]);
    ]
```

122 image/test/diff_branches_test.ml

```
(** Branch diff classification (branching.diff): Store.diff_branches.
    Needs Postgres (scripts/dev.sh). Skips if unreachable.

    Verifies: modified fn carries parsed from/to exprs; added/removed
    across two empty-lens children of main; shared unchanged defs are
    NOT reported; types classified on fields; routes added. *)

module S = Pricklypear_image_lib.Store

let main_id = "00000000-0000-0000-0000-000000000001"

let with_store f =
  Eio_main.run @@ fun env ->
  Lwt_eio.with_event_loop ~clock:(Eio.Stdenv.clock env) @@ fun _ ->
  Eio.Switch.run @@ fun sw ->
  let ctx : Pricklypear_image_lib.Io_eio.Ctx.t =
    {
      sw;
      net = (Eio.Stdenv.net env :> [ `Generic ] Eio.Net.ty Eio.Std.r);
    }
  in
  Pricklypear_image_lib.Io_eio.with_ctx ctx (fun () ->
      let cfg = S.config_from_env () in
      let store = S.make ~cfg ~branch_id:main_id () in
      S.connect store;
      match S.conn_of store with
      | None ->
          Printf.printf
            "SKIP: no postgres (set PP_DB_* / run scripts/dev.sh)\n%!"
      | Some _ -> f store)

let expr_of_int n =
  Pricklypear_lisp.Ast.Lit (Pricklypear_lisp.Ast.VInt (Int64.of_int n))

let uniq p = Printf.sprintf "%s_%d" p (Random.int 1_000_000)

let find rows ~name =
  List.find_opt (fun r -> r.S.diff_name = name) rows

let kind_of (r : S.branch_diff_row) = r.S.diff_kind
let status_of (r : S.branch_diff_row) = r.S.diff_status

(** Two empty-lens children of main: A and B. Both inherit main's same
    shared defs (unchanged ? not reported). Each owns divergent rows. *)
let test_diff_classification () =
  with_store (fun store ->
      match S.conn_of store with
      | None -> ()
      | Some conn ->
          let suffix = string_of_int (Random.int 1_000_000) in
          let a_name = "dfa_child_" ^ suffix in
          let b_name = "dfb_child_" ^ suffix in
          let fn_shared = "dfn_shared_" ^ suffix in
          let fn_mod = "dfn_mod_" ^ suffix in
          let fn_a_add = "dfn_a_" ^ suffix in
          let fn_b_add = "dfn_b_" ^ suffix in
          let ty_shared = "dty_shared_" ^ suffix in
          let route_b = "/drt-" ^ suffix in
          let handler_b = "dfn_b_" ^ suffix in
          store.branch_id <- main_id;
          (* shared on main: identical effective def on both children *)
          S.persist_function store fn_shared (expr_of_int 1) "tester";
          S.persist_type store ~name:ty_shared ~fields:[ "a" ]
    ~author:"tester";
          (match
             S.fork_branch store ~parent_id:main_id ~owner:"dfa_test"
               ~child_name:a_name
           with
           | None -> Alcotest.fail "fork A failed"
           | Some a_id ->
               (match
```

```
                S.fork_branch store ~parent_id:main_id ~owner:"dfa_test"
                  ~child_name:b_name
              with
              | None -> Alcotest.fail "fork B failed"
              | Some b_id ->
                  (* child A: own fn_a_add + fn_mod (int 1) *)
                  store.branch_id <- a_id;
                  S.persist_function store fn_a_add (expr_of_int 5)
    "tester";
                  S.persist_function store fn_mod (expr_of_int 1)
    "tester";
                  (* child B: own fn_b_add + fn_mod (int 2) + type
    override
                     + a route *)
                  store.branch_id <- b_id;
                  S.persist_function store fn_b_add (expr_of_int 6)
    "tester";
                  S.persist_function store fn_mod (expr_of_int 2)
    "tester";
                  S.persist_type store ~name:ty_shared
                    ~fields:[ "a"; "b" ] ~author:"tester";
                  S.register_route store ~route:route_b ~handler:handler_b
                    ~visibility:"public" ();
                  let rows = S.diff_branches store ~from_id:a_id
    ~to_id:b_id in
                  (* unchanged shared fn must NOT be reported *)
                  Alcotest.(check bool) "shared unchanged fn not in diff"
    true
                    (find rows ~name:fn_shared = None);
                  (* fn_mod modified (from int1 on A ? int2 on B) *)
                  (match find rows ~name:fn_mod with
                   | None -> Alcotest.fail "fn_mod missing from diff"
                   | Some r ->
                       Alcotest.(check bool) "fn_mod is fn" true
                         (kind_of r = S.DiffFn);
                       Alcotest.(check bool) "fn_mod modified" true
                         (status_of r = S.DiffModified);
                       Alcotest.(check bool) "fn_mod parses from_expr"
    true
                         (r.S.from_expr <> None);
                       Alcotest.(check bool) "fn_mod parses to_expr" true
                         (r.S.to_expr <> None));
                  (* fn_a_add removed (in A, not B) *)
                  (match find rows ~name:fn_a_add with
                   | None -> Alcotest.fail "fn_a_add missing"
                   | Some r ->
                       Alcotest.(check bool) "fn_a_add removed" true
                         (status_of r = S.DiffRemoved);
                       Alcotest.(check bool) "fn_a_add from_body present"
    true
                         (r.S.from_body <> None);
                       Alcotest.(check bool) "fn_a_add to_body absent"
    true
                         (r.S.to_body = None));
                  (* fn_b_add added (in B, not A) *)
                  (match find rows ~name:fn_b_add with
                   | None -> Alcotest.fail "fn_b_add missing"
                   | Some r ->
                       Alcotest.(check bool) "fn_b_add added" true
                         (status_of r = S.DiffAdded);
                       Alcotest.(check bool) "fn_b_add from_body absent"
    true
                         (r.S.from_body = None);
                       Alcotest.(check bool) "fn_b_add to_body present"
    true
                         (r.S.to_body <> None));
                  (* type override modified (fields a ? a,b) *)
                  (match find rows ~name:ty_shared with
                   | None -> Alcotest.fail "ty_shared missing"
                   | Some r ->
                       Alcotest.(check bool) "ty_shared is type" true
                         (kind_of r = S.DiffType);
                       Alcotest.(check bool) "ty_shared modified" true
```

```
                          (status_of r = S.DiffModified));
                    (* route added on B *)
                    (match find rows ~name:route_b with
                     | None -> Alcotest.fail "route_b missing"
                     | Some r ->
                         Alcotest.(check bool) "route_b is route" true
                           (kind_of r = S.DiffRoute);
                         Alcotest.(check bool) "route_b added" true
                           (status_of r = S.DiffAdded));
                    (* cleanup *)
                    ignore (S.delete_function_on_branch store
    ~branch_id:main_id
                              ~name:fn_shared);
                    ignore (S.delete_function_on_branch store
    ~branch_id:a_id
                              ~name:fn_a_add);
                    ignore (S.delete_function_on_branch store
    ~branch_id:b_id
                              ~name:fn_b_add);
                    ignore (S.delete_function_on_branch store
    ~branch_id:a_id
                              ~name:fn_mod);
                    ignore (S.delete_function_on_branch store
    ~branch_id:b_id
                              ~name:fn_mod);
                    ignore
                      (S.exec conn
                         ~params:
                           [ Pgx.Value.of_string a_id
                           ; Pgx.Value.of_string b_id ]
                         "DELETE FROM type_defs WHERE branch_id IN ($1,$2)
    AND \
                          name = 'dummy-no-op'");
                    ignore
                      (S.exec conn
                         ~params:
                           [ Pgx.Value.of_string main_id
                           ; Pgx.Value.of_string ty_shared ]
                         "DELETE FROM type_defs WHERE branch_id = $1 AND
    name = \
                          $2");
                    ignore
                      (S.exec conn
                         ~params:
                           [ Pgx.Value.of_string b_id
                           ; Pgx.Value.of_string route_b ]
                         "DELETE FROM view_bindings WHERE branch_id = $1
    AND \
                          route = $2");
                    ())))

let () =
  Random.self_init ();
  Alcotest.run "diff-branches"
    [ ( "db"
      , [ Alcotest.test_case "classification" `Quick
    test_diff_classification ]
      ) ]
```

123 image/test/example_eval_test.ml

```
(* Pricklypear Image -- example-eval restriction tests (no DB).

   habitat-book.eval-examples: (example-eval src) parses + evals
   Nopales source in a RESTRICTED environment. These tests pin the
   restriction surface against a synthetic prim table + env built the
   same way primitives_test builds them (no Session/PG):

     - whitelist surface: pure + read-only DB + supplement present;
       mutation families and escape hatches ABSENT (spec verify
       smoke: "define/route/create prims are absent from the example
       table");
     - static gate: mutation special forms refused, quoted data inert,
       capability-carrying calls refused via known_fn;
     - runtime boundary: prims absent from the restricted table fail
       closed -- including through macro expansion and function bodies;
     - env semantics: parent bindings resolve, thunks force, nothing
       leaks back (child env);
     - caps: step budget kills runaway nesting -- CONVERTED to Refused
       (budget -> refusal) so (book/verify)'s guard can hole it.

   The eval path runs under Example_eval's own nested rim, so no outer
   Control_rim is needed in the test (bench precedent). *)
open Pricklypear_lisp.Ast

let pp = Pricklypear_lisp.Reader.pp_value

(* Synthetic full prim table + env, mirroring Session.make's shape:
   every registered prim name auto-binds as VPrim in the env. *)
let make_full () =
  let prim_table = Pricklypear_image_lib.Primitives.make_with_defaults () in
  let env = Pricklypear_lisp.Env.create () in
  Hashtbl.iter
    (fun name _ -> Pricklypear_lisp.Env.define env name (VPrim name))
    prim_table;
  Pricklypear_lisp.Env.define env "pi" (VFloat Float.pi);
  (env, prim_table)

(* Bind a VPrim name without registering a body (simulates the session
   auto-bind for CRUD prims that only the image registers). *)
let bind_prim env name = Pricklypear_lisp.Env.define env name (VPrim name)

(* Eval src through the restricted engine; returns the rendered value
   or an "REFUSED: ?" / "ERR: ?" marker string instead of raising. *)
let run ?(limit = Pricklypear_image_lib.Example_eval.default_step_limit)
    ?known_fn (src : string) :
    string =
  let env, full = make_full () in
  let known_fn = match known_fn with Some f -> f | None -> fun _ -> None in
  try
    let v =
      Pricklypear_image_lib.Example_eval.eval_source ~env ~full_prims:full
    ~known_fn ~limit src
    in
    pp v
  with
  | Pricklypear_image_lib.Example_eval.Refused reason -> "REFUSED: " ^
    reason
  | Interp_error e -> "ERR: " ^ Pricklypear_lisp.Ast.interp_error_kind e
  | exn -> "ERR: " ^ Printexc.to_string exn

(* --- whitelist surface --- *)

let whitelist_tests =
  [ ("pure prims whitelisted", `Quick, fun () ->
      Alcotest.(check bool) "+" true
    (Pricklypear_image_lib.Example_eval.whitelisted "+");
      Alcotest.(check bool) "string-append" true
        (Pricklypear_image_lib.Example_eval.whitelisted "string-append");
      Alcotest.(check bool) "read-string" true
        (Pricklypear_image_lib.Example_eval.whitelisted "read-string"));
    ("read-only DB prims whitelisted", `Quick, fun () ->
```

```
      List.iter
        (fun n -> Alcotest.(check bool) n true
     (Pricklypear_image_lib.Example_eval.whitelisted n))
        [ "get"; "rows"; "find-rows"; "whoami"; "current-branch";
          "list-bindings"; "function-doc"; "current-author" ]);
    ("read-only supplement whitelisted", `Quick, fun () ->
      List.iter
        (fun n -> Alcotest.(check bool) n true
     (Pricklypear_image_lib.Example_eval.whitelisted n))
        [ "string-split"; "prose-tokens"; "render-tokens"; "tree/get";
          "tree/children-of"; "tree/subtree"; "tree/parent";
          "backlinks"; "out-links"; "list-packages"; "describe";
          "callees"; "holes"; "budget-remaining" ]);
    (* spec eval-examples verify smoke: mutation families absent *)
    ("mutation prims NOT whitelisted", `Quick, fun () ->
      List.iter
        (fun n -> Alcotest.(check bool) n false
     (Pricklypear_image_lib.Example_eval.whitelisted n))
        [ "create"; "update"; "update-field"; "delete"; "deftype";
          "route"; "public-route"; "checkout"; "fork"; "merge";
          "create-user"; "load-string"; "load-library"; "reseed-package";
          "tree/upsert"; "tree/delete"; "with-public-write";
          "submit-job"; "set-llm-key" ]);
    ("escape-hatch prims NOT whitelisted", `Quick, fun () ->
      List.iter
        (fun n -> Alcotest.(check bool) n false
     (Pricklypear_image_lib.Example_eval.whitelisted n))
        [ "eval-string"; "bench"; "example-eval"; "http-get"; "http-post";
          "tcp-connect"; "sleep"; "agent"; "llm-chat"; "propose-function";
          "install-function" ]);
    ("restricted table holds only whitelisted names", `Quick, fun () ->
      let _, full = make_full () in
      let r = Pricklypear_image_lib.Example_eval.restricted_prims full in
      let bad =
        Hashtbl.fold
          (fun name _ acc ->
            if Pricklypear_image_lib.Example_eval.whitelisted name then acc
     else name :: acc)
          r []
      in
      Alcotest.(check (list string)) "no non-whitelisted entries" [] bad);
  ]

(* --- happy paths --- *)

let eval_tests =
  [ ("pure arithmetic evals", `Quick, fun () ->
      Alcotest.(check string) "1+2" "3" (run "(+ 1 2)"));
    ("multiple forms return last", `Quick, fun () ->
      Alcotest.(check string) "last" "12" (run "(+ 1 2) (* 3 4)"));
    ("let/lambda/quote usable", `Quick, fun () ->
      Alcotest.(check string) "let" "7" (run "(let ((x 6)) (+ x 1))");
      Alcotest.(check string) "lambda" "9" (run "((lambda (x) (* x 3)) 3)");
      Alcotest.(check string) "quote is data" "(define x 1)"
        (run "(quote (define x 1))"));
    ("parent bindings resolve", `Quick, fun () ->
      let env, full = make_full () in
      Pricklypear_lisp.Env.define env "answer" (VInt 40L);
      let v =
        Pricklypear_image_lib.Example_eval.eval_source ~env ~full_prims:full
          ~known_fn:(fun _ -> None) "(+ answer 2)"
      in
      Alcotest.(check string) "binding" "42" (pp v));
    ("thunks force through the parent env", `Quick, fun () ->
      let env, full = make_full () in
      Pricklypear_lisp.Env.define_thunk env "lazy-42"
        ~force:(fun _e _b -> VInt 42L) (Var "unused");
      let v =
        Pricklypear_image_lib.Example_eval.eval_source ~env ~full_prims:full
          ~known_fn:(fun _ -> None) "(+ lazy-42 0)"
      in
      Alcotest.(check string) "thunk" "42" (pp v));
    ("nothing leaks into the session env", `Quick, fun () ->
```

```
      (* define is refused statically; prove isolation via shadowing:
         the example's child env dies, the parent keeps its own value. *)
      let env, full = make_full () in
      Pricklypear_lisp.Env.define env "x" (VInt 1L);
      let _ =
        Pricklypear_image_lib.Example_eval.eval_source ~env ~full_prims:full
          ~known_fn:(fun _ -> None) "(let ((x 99)) x)"
      in
      let v =
        Pricklypear_lisp.Eval.eval
          ~budget:(Pricklypear_lisp.Eval.make_budget ())
          ~prims:Pricklypear_lisp.Eval.empty_prims env (Var "x")
      in
      Alcotest.(check string) "parent unchanged" "1" (pp v));
  ]

(* --- static gate refusals --- *)

let refused msg = String.starts_with ~prefix:"REFUSED:" msg

let gate_tests =
  [ ("define refused", `Quick, fun () ->
      Alcotest.(check string) "define"
        "REFUSED: special form 'define' is not allowed in examples"
        (run "(define x 1)"));
    ("nested define refused", `Quick, fun () ->
      Alcotest.(check bool) "nested" (refused (run "(let ((y 1)) (define z
    2) z)"))
        true);
    ("defmacro refused", `Quick, fun () ->
      Alcotest.(check bool) "defmacro" (refused (run "(defmacro m () 1)"))
    true);
    ("with-transaction refused", `Quick, fun () ->
      Alcotest.(check bool) "txn" (refused (run "(with-transaction 1)"))
    true);
    ("prim escape hatch refused", `Quick, fun () ->
      Alcotest.(check bool) "prim form" (refused (run "(prim \"list\" 1)"))
    true);
    ("quoted define is inert data", `Quick, fun () ->
      Alcotest.(check bool) "quoted data"
        (not (refused (run "'(define x 1)")))
        true);
    ("quoted write-db call is inert data too", `Quick, fun () ->
      (* Reader sugar 'x parses as App (Var "quote") -- eval returns the
         datum, so the gate must not infer the quoted form's effects
         (desugar_quote). A quoted (create ?) neither executes nor
         refuses. *)
      Alcotest.(check string) "quoted create returns datum"
        "(create (quote user) (quote x) (quote y))" (run "'(create 'user 'x
    'y)"));
    ("write capability refused via known_fn", `Quick, fun () ->
      let known_fn = function
        | "boom" -> Some (Pricklypear_lisp.Effect.EffSet.singleton
                            Pricklypear_lisp.Effect.WriteDb)
        | _ -> None
      in
      let out = run ~known_fn "(boom)" in
      Alcotest.(check bool) "db-write fn refused" (refused out) true;
      Alcotest.(check bool) "names the capability"
        (String.contains out 'w') true);
    ("net capability refused via known_fn", `Quick, fun () ->
      let known_fn = function
        | "fetch" -> Some (Pricklypear_lisp.Effect.EffSet.singleton
                            Pricklypear_lisp.Effect.Net)
        | _ -> None
      in
      Alcotest.(check bool) "net fn" (refused (run ~known_fn "(fetch)"))
    true);
  ]

(* --- runtime boundary (restricted rim) --- *)

let runtime_tests =
```

```
  [ ("unknown prim fails closed", `Quick, fun () ->
      (* "do-evil" is registered + auto-bound (full-table shape) but
         unknown to the effect lists (static gate passes it) and absent
         from the whitelist ? the restricted rim refuses at dispatch. *)
      let env, full = make_full () in
      Pricklypear_image_lib.Primitives.register full "do-evil"
        (fun _ -> VStr "pwned");
      bind_prim env "do-evil";
      match
        Pricklypear_image_lib.Example_eval.eval_source ~env ~full_prims:full
          ~known_fn:(fun _ -> None) "(do-evil)"
      with
      | v -> Alcotest.fail ("should have refused, got " ^ pp v)
      | exception Interp_error (Prim_error (name, msg, _)) ->
          Alcotest.(check string) "refused name" "do-evil" name;
          Alcotest.(check bool) "fail-closed msg"
            (String.contains msg 'u') true);
    ("write prim through a macro fails closed", `Quick, fun () ->
      (* A pre-existing VMacro whose expansion calls create: the static
         gate cannot see it; the restricted rim must. *)
      let env, full = make_full () in
      bind_prim env "create";
      Pricklypear_lisp.Env.define env "wiggle"
        (VMacro ([ "x" ], App (Var "create", [ Lit (VSymbol "user") ]),
     env));
      match
        Pricklypear_image_lib.Example_eval.eval_source ~env ~full_prims:full
          ~known_fn:(fun _ -> None) "(wiggle 1)"
      with
      | v -> Alcotest.fail ("should have refused, got " ^ pp v)
      | exception Interp_error (Prim_error (name, _, _)) ->
          Alcotest.(check string) "create absent" "create" name);
    ("write prim through a function body fails closed", `Quick, fun () ->
      let env, full = make_full () in
      bind_prim env "delete";
      Pricklypear_lisp.Env.define env "sneaky"
        (VLambda ([], App (Var "delete", [ Lit (VSymbol "user") ]), env));
      match
        Pricklypear_image_lib.Example_eval.eval_source ~env ~full_prims:full
          ~known_fn:(fun _ -> None) "(sneaky)"
      with
      | v -> Alcotest.fail ("should have refused, got " ^ pp v)
      | exception Interp_error (Prim_error (name, _, _)) ->
          Alcotest.(check string) "delete absent" "delete" name);
    ("read prims still dispatch in the restricted rim", `Quick, fun () ->
      (* Whitelisted names keep their closures: a fake "rows" runs
         under the restricted rim. Register the body AND bind the name
         (session.make does both: bulk VPrim bind over the table, then
         image-only registrations -- "rows" is a CRUD prim only the
         image registers). *)
      let env, full = make_full () in
      bind_prim env "rows";
      Pricklypear_image_lib.Primitives.register full "rows"
        (fun args -> VList [ VStr (String.concat "," (List.map pp args)) ]);
      let v =
        Pricklypear_image_lib.Example_eval.eval_source ~env ~full_prims:full
          ~known_fn:(fun _ -> None) "(rows 'user)"
      in
      Alcotest.(check string) "rows ran" "(\"user\")" (pp v));
  ]

(* --- caps --- *)

let cap_tests =
  [ ("step budget converts runaway nesting to a refusal", `Quick, fun () ->
      let deep =
        String.concat "" (List.init 2000 (fun _ -> "(+ 1 "))
        ^ "0" ^ String.concat "" (List.init 2000 (fun _ -> ")"))
      in
      let out = run ~limit:100 deep in
      (* v2 (P2.4): budget exhaustion CONVERTS to Refused -- the only
         live budget here is the nested one, and guard (book/verify's
         only catch) re-raises Budget_exceeded by design, so a runaway
```

```
         example must surface as a hole-able refusal, not a page
         killer. Pre-P2.4 this surfaced as ERR: budget_exceeded. *)
      Alcotest.(check bool) "budget -> refused"
        (String.starts_with ~prefix:"REFUSED: example budget exceeded" out)
        true);
    ("self-application loop refused at the default limit", `Quick, fun () ->
      (* ((lambda (x) (x x)) (lambda (x) (x x))) -- pure-lambda runaway
         (no recursion special form needed), the shape book/verify can
         actually meet in curated examples. Must come back as a
         refusal well under the wall-clock cap. *)
      let out = run "((lambda (x) (x x)) (lambda (x) (x x)))" in
      Alcotest.(check bool) "y-loop refused"
        (String.starts_with ~prefix:"REFUSED: example budget exceeded" out)
        true);
    ("outer caller budget untouched by a runaway example", `Quick, fun () ->
      (* The nested budget dies; the caller's current_budget is
         restored in finally -- prove the session can keep evaluating
         after a refusal. *)
      let env, full = make_full () in
      (try
         ignore
           (Pricklypear_image_lib.Example_eval.eval_source ~env
     ~full_prims:full
             ~known_fn:(fun _ -> None)
             "((lambda (x) (x x)) (lambda (x) (x x)))")
       with Pricklypear_image_lib.Example_eval.Refused _ -> ());
      let v =
        Pricklypear_image_lib.Control_rim.run ~prims:full (fun () ->
            Pricklypear_lisp.Eval.eval
              ~budget:(Pricklypear_lisp.Eval.make_budget ())
              ~prims:Pricklypear_lisp.Eval.empty_prims env
              (App (Var "+", [ int 1; int 2 ])))
      in
      Alcotest.(check string) "caller alive" "3" (pp v));
    ("empty source refused", `Quick, fun () ->
      Alcotest.(check string) "empty" "REFUSED: empty source" (run ""));
    ("parse error propagates as ERR", `Quick, fun () ->
      Alcotest.(check bool) "parse err"
        (String.starts_with ~prefix:"ERR:" (run "(+ 1")) true);
  ]

let () =
  let open Alcotest in
  run "example_eval"
    [
      ("whitelist", whitelist_tests);
      ("eval", eval_tests);
      ("static gate", gate_tests);
      ("runtime boundary", runtime_tests);
      ("caps", cap_tests);
    ]
```

124 image/test/history_test.ml

```
(** code-history tests (needs Postgres from scripts/dev.sh).

    Spec: branching.code-history
    Skips cleanly when PP_DB is unreachable. Apply migrations via
    scripts/dev.sh so code_history exists. *)

module S = Pricklypear_image_lib.Store

let main_id = "00000000-0000-0000-0000-000000000001"

let with_store f =
  Eio_main.run @@ fun env ->
  Lwt_eio.with_event_loop ~clock:(Eio.Stdenv.clock env) @@ fun _ ->
  Eio.Switch.run @@ fun sw ->
  let ctx : Pricklypear_image_lib.Io_eio.Ctx.t =
    {
      sw;
      net = (Eio.Stdenv.net env :> [ `Generic ] Eio.Net.ty Eio.Std.r);
    }
  in
  Pricklypear_image_lib.Io_eio.with_ctx ctx (fun () ->
      let cfg = S.config_from_env () in
      let store = S.make ~cfg ~branch_id:main_id () in
      S.connect store;
      match S.conn_of_store store with
      | None ->
          Printf.printf
            "SKIP: no postgres (set PP_DB_* / run scripts/dev.sh)\n%!"
      | Some conn ->
          let has_table =
            try
              let _ = S.exec conn "SELECT 1 FROM code_history LIMIT 0" in
              true
            with _ -> false
          in
          if not has_table then
            Printf.printf
              "SKIP: code_history missing (restart scripts/dev.sh to \
               apply migration 012)\n%!"
          else f store)

let uniq suffix =
  Printf.sprintf "hist_test_%s_%d" suffix (Random.int 1_000_000)

let expr_of_int n =
  Pricklypear_lisp.Ast.Lit (Pricklypear_lisp.Ast.VInt (Int64.of_int n))

let test_redefine_records_old_body () =
  with_store (fun store ->
      let name = uniq "fn" in
      S.persist_function store name (expr_of_int 1) ~doc:(Some "v1")
    "tester";
      S.persist_function store name (expr_of_int 2) ~doc:(Some "v2")
    "tester";
      let hist =
        S.list_code_history store ~branch_id:(S.current_branch_id store)
    ~name
      in
      Alcotest.(check bool) "has history" true (List.length hist >= 1);
      let newest = List.hd hist in
      Alcotest.(check string) "rel" "function_defs" newest.rel;
      Alcotest.(check string) "op" "UPDATE" newest.op;
      (match newest.body with
       | None -> Alcotest.fail "expected body"
       | Some body ->
           Alcotest.(check bool) "old body present" true
             (String.length body > 0));
      let _ =
        S.delete_function_on_branch store
          ~branch_id:(S.current_branch_id store) ~name
      in
```

```
      ())

let test_merge_clobber_records_history () =
  with_store (fun store ->
      let suffix = string_of_int (Random.int 1_000_000) in
      let child_name = "hist_child_" ^ suffix in
      let name = "hist_merge_fn_" ^ suffix in
      store.branch_id <- main_id;
      S.persist_function store name (expr_of_int 10) ~doc:(Some "main")
    "tester";
      match
        S.fork_branch store ~parent_id:main_id ~owner:"hist_test"
    ~child_name
      with
      | None -> Alcotest.fail "fork failed"
      | Some child_id ->
          store.branch_id <- child_id;
          S.persist_function store name (expr_of_int 99) ~doc:(Some "child")
            "tester";
          let n =
            S.merge_branch store ~source_id:child_id ~target_id:main_id
          in
          Alcotest.(check bool) "merged" true
            (match n with Some _ -> true | None -> false);
          let hist = S.list_code_history store ~branch_id:main_id ~name in
          Alcotest.(check bool) "clobber history" true (List.length hist >=
    1);
          store.branch_id <- main_id;
          let _ = S.delete_function_on_branch store ~branch_id:main_id
    ~name in
          let _ =
            S.delete_function_on_branch store ~branch_id:child_id ~name
          in
          ())

let test_additive_insert_no_history () =
  with_store (fun store ->
      let name = uniq "add" in
      store.branch_id <- main_id;
      S.persist_function store name (expr_of_int 7) "tester";
      let hist =
        S.list_code_history store ~branch_id:(S.current_branch_id store)
    ~name
      in
      Alcotest.(check int) "no history on insert" 0 (List.length hist);
      let _ =
        S.delete_function_on_branch store
          ~branch_id:(S.current_branch_id store) ~name
      in
      ())

let test_rollback_no_history () =
  with_store (fun store ->
      let name = uniq "rb" in
      store.branch_id <- main_id;
      S.persist_function store name (expr_of_int 1) "tester";
      let before =
        List.length (S.list_code_history store ~branch_id:main_id ~name)
      in
      (try
         S.with_transaction store (fun conn_opt ->
             match conn_opt with
             | None -> ()
             | Some conn ->
                 let body_json =
                   Yojson.Safe.to_string
                     (Pricklypear_lisp.Ast.expr_to_json (expr_of_int 2))
                 in
                 let sql =
                   "UPDATE function_defs SET body = $3::jsonb, " ^
                   "updated_at = now() " ^
                   "WHERE branch_id = $1 AND name = $2"
                 in
```

```
                let _ =
                  S.exec conn
                    ~params:
                      [ Pgx.Value.of_string main_id
                      ; Pgx.Value.of_string name
                      ; Pgx.Value.of_string body_json
                      ]
                    sql
                in
                failwith "force rollback")
       with Failure _ -> ());
      let after =
        List.length (S.list_code_history store ~branch_id:main_id ~name)
      in
      Alcotest.(check int) "rollback leaves no history" before after;
      let _ = S.delete_function_on_branch store ~branch_id:main_id ~name in
      ())

let test_code_diff_smoke () =
  with_store (fun store ->
      let name = uniq "diff" in
      store.branch_id <- main_id;
      S.persist_function store name (expr_of_int 1) "tester";
      S.persist_function store name (expr_of_int 2) "tester";
      match S.code_diff store ~branch_id:main_id ~name ~n:1 with
      | Error e -> Alcotest.fail e
      | Ok s ->
          Alcotest.(check bool) "diff non-empty markers" true
            (String.contains s '-' || String.contains s '+');
          let _ = S.delete_function_on_branch store ~branch_id:main_id
    ~name in
          ())

let () =
  Random.self_init ();
  Alcotest.run "code-history"
    [ ( "history"
      , [ Alcotest.test_case "redefine records old body" `Quick
            test_redefine_records_old_body
        ; Alcotest.test_case "merge clobber records history" `Quick
            test_merge_clobber_records_history
        ; Alcotest.test_case "additive insert no history" `Quick
            test_additive_insert_no_history
        ; Alcotest.test_case "rollback leaves no history" `Quick
            test_rollback_no_history
        ; Alcotest.test_case "code-diff smoke" `Quick test_code_diff_smoke
        ] )
    ]
```

125 image/test/http_get_test.ml

```
(** Pricklypear Image -- http-get shape tests (D1 + SSRF egress guard).

    Tests:
    - private/loopback/link-local targets are refused by the SSRF
      blocklist (Prim_error kind="denied", before any network use)
    - network transient (closed port) RAISES Interp_error kind="transient"
      (exercised with PP_NET_EGRESS_UNRESTRICTED=1, since the blocklist
      would refuse loopback targets before curl runs)
    - semantic HTTP responses (4xx/5xx) return a payload dict with
      ok/err/kind fields (not raised) -- the dict-shape path requires a
      live HTTP server and is marked @PG. *)
open Pricklypear_lisp.Ast

(* Find a closed TCP port: bind+close a socket, then reuse the number. *)
let grab_closed_port () =
  let s = Unix.socket Unix.PF_INET Unix.SOCK_STREAM 0 in
  Unix.setsockopt s Unix.SO_REUSEADDR true;
  Unix.bind s (Unix.ADDR_INET (Unix.inet_addr_loopback, 0));
  let port =
    match Unix.getsockname s with
    | Unix.ADDR_INET (_, p) -> p
    | _ -> 0
  in
  Unix.close s;
  port

(* Expect every URL to be refused by the SSRF egress guard. *)
let expect_blocked (urls : string list) : unit =
  List.iter
    (fun url ->
       try
         let _ =
           Pricklypear_image_lib.Prims_net.do_http_get url
             [ ("_timeout_ms", "1000") ]
         in
         Alcotest.failf "expected %s to be blocked by the egress guard" url
       with
       | Interp_error (Prim_error ("http-get", _msg, "denied")) -> ()
       | exn ->
           Alcotest.failf "expected Prim_error denied for %s, got %s" url
             (Printexc.to_string exn))
    urls

let tests = [
  ("http-get blocked: v4 loopback/private/link-local", `Quick, fun () ->
    expect_blocked
      [ "http://127.0.0.1:9/"
      ; "http://10.0.0.1:9/"
      ; "http://172.16.0.1:9/"
      ; "http://192.168.1.1:9/"
      ; "http://169.254.169.254:9/" ]);
  ("http-get blocked: v6 loopback/ula/link-local", `Quick, fun () ->
    expect_blocked
      [ "http://[::1]:9/"
      ; "http://[fd00::1]:9/"
      ; "http://[fe80::1]:9/" ]);
  ("http-get blocked: localhost resolves to loopback", `Quick, fun () ->
    expect_blocked [ "http://localhost:9/" ]);
  ("http-get transient raises (closed port, egress unrestricted)", `Quick,
     fun () ->
    (* The SSRF blocklist would refuse loopback targets before curl
       runs; this test exercises the curl transient path, so egress is
       unrestricted for it (env restored after). *)
    Unix.putenv "PP_NET_EGRESS_UNRESTRICTED" "1";
    let port = grab_closed_port () in
    let url = Printf.sprintf "http://127.0.0.1:%d/" port in
    try
      let _ = Pricklypear_image_lib.Prims_net.do_http_get url
     [("_timeout_ms", "3000")] in
      Alcotest.fail "expected http-get to raise on closed port (transient)"
    with
```

```
    | Interp_error (Prim_error ("http-get", msg, kind)) ->
        Unix.putenv "PP_NET_EGRESS_UNRESTRICTED" "0";
        Alcotest.(check string) "kind" "transient" kind;
        Alcotest.(check bool) "msg mentions transient" true
          (String.length msg > 0)
    | exn ->
        Alcotest.failf "expected Prim_error transient, got %s"
          (Printexc.to_string exn));
  ("http-post transient raises (closed port)", `Quick, fun () ->
    (* Same transient contract as http-get (image.outbound-http): a
       send verb raises Prim_error named for the prim. Egress is
       unrestricted for it (the SSRF blocklist would refuse the
       loopback target before curl runs); env restored after. *)
    Unix.putenv "PP_NET_EGRESS_UNRESTRICTED" "1";
    let port = grab_closed_port () in
    let url = Printf.sprintf "http://127.0.0.1:%d/" port in
    try
      let _ =
        Pricklypear_image_lib.Prims_net.do_http "http-post" "POST"
          (Some "x") url [("_timeout_ms", "3000")]
      in
      Unix.putenv "PP_NET_EGRESS_UNRESTRICTED" "0";
      Alcotest.fail "expected http-post to raise on closed port (transient)"
    with
    | Interp_error (Prim_error ("http-post", msg, kind)) ->
        Unix.putenv "PP_NET_EGRESS_UNRESTRICTED" "0";
        Alcotest.(check string) "kind" "transient" kind;
        Alcotest.(check bool) "msg mentions transient" true
          (String.length msg > 0)
    | exn ->
        Unix.putenv "PP_NET_EGRESS_UNRESTRICTED" "0";
        Alcotest.failf "expected Prim_error transient, got %s"
          (Printexc.to_string exn));
  ("http-get bad-args raises on wrong arity", `Quick, fun () ->
    (* The arg-shape check is in the registered prim, not do_http_get.
       We can't easily call the registered prim without a Session.t.
       This test documents that the arg-shape path raises bad-args --
       verified by code inspection (see prims_net.ml http-get handler). *)
    Alcotest.(check bool) "documented" true true);
]

let () =
  Alcotest.run "pricklypear-image-http-get" [
    "shape", tests;
  ]
```

126 image/test/http_robust_env_test.ml

```
(** Pricklypear Image -- invariant 4a unit test: immutable-env rehydrate.

    Proves that rehydrate SWAPS t.env to a fresh record rather than
    clearing the old binding table in place, so a VLambda closure that
    captured the OLD env keeps reading its bindings after a concurrent
    rehydrate. This is the correctness foundation for lock-free page
    work (invariant 4b): a page's captured closures cannot be emptied
    by another request's rehydrate. *)

let vint_of = function
  | Pricklypear_lisp.Ast.VInt i -> i
  | _ -> 0L

let () =
  Alcotest.run "http_robust_env"
    [ ("immutable-env rehydrate",
        [ Alcotest.test_case "captured env survives rehydrate" `Quick
            (fun () ->
              (* Build an env, bind x, then simulate rehydrate swapping to
                 a fresh env. The OLD env's bindings must stay intact (not
                 cleared in place) so a closure that captured old_env keeps
                 reading x. *)
              let old_env = Pricklypear_lisp.Env.create () in
              Pricklypear_lisp.Env.define old_env "x"
                (Pricklypear_lisp.Ast.VInt 42L);
              let captured_x_before =
                Pricklypear_lisp.Env.lookup old_env "x"
              in
              Alcotest.(check (option Alcotest.int64))
                "old env has x before rehydrate"
                (Some 42L)
                (Option.map vint_of captured_x_before);
              (* rehydrate now builds a fresh env and swaps; it does NOT
                 Hashtbl.clear the old one. Simulate that contract here. *)
              let _fresh_env = Pricklypear_lisp.Env.create () in
              let captured_x_after =
                Pricklypear_lisp.Env.lookup old_env "x"
              in
              Alcotest.(check (option Alcotest.int64))
                "old env STILL has x after swap (not cleared in place)"
                (Some 42L)
                (Option.map vint_of captured_x_after);
              (* The fresh env starts empty (a new Hashtbl). *)
              let fresh = Pricklypear_lisp.Env.create () in
              Alcotest.(check bool)
                "fresh env starts without x"
                true
                (Pricklypear_lisp.Env.lookup fresh "x" = None))
        ])
    ]
```

127 image/test/http_robust_test.ml

```
(** Pricklypear Image -- HTTP robustness tests (image.http-robustness).

    Tests the peer-disconnect / closed-writer guards and the
    accept-side connection cap. These are the invariants that prevent
    a single aborted request from killing the single-image process
    (prod incident 2026-07-16). No DB / no live server needed: we
    exercise the pure predicate, the safe-writer wrappers' behavior on
    the recognized exception, and the inflight gate's counting. *)

let is_closed = Pricklypear_image_lib.Http_core.is_closed_writer_exn


let test_recognizes_closed_writer () =
  Alcotest.(check bool) "exact closed-writer msg" true
    (is_closed (Failure "cannot write to closed writer"));
  Alcotest.(check bool) "closed-writer with suffix" true
    (is_closed (Failure "cannot write to closed writer (foo)"));
  Alcotest.(check bool) "unrelated Failure not matched" false
    (is_closed (Failure "something else"));
  Alcotest.(check bool) "non-Failure exn not matched" false
    (is_closed Not_found);
  Alcotest.(check bool) "Stack_overflow not matched" false
    (is_closed Stack_overflow)

(* safe_close_writer / safe_respond_with_string swallow the closed-writer
   Failure and return () instead of raising. We can't easily build a real
   closed httpaf Body.t in a unit test, so we assert the guard predicate is
   what the wrappers rely on, and that the recognized exception does not
   propagate when fed through a try/when using the same predicate -- i.e.
   the pattern the wrappers use. *)
let test_safe_wrappers_swallow_closed_writer () =
  let swallowed =
    try
      (try raise (Failure "cannot write to closed writer")
       with exn when is_closed exn -> ());
      true
    with _ -> false
  in
  Alcotest.(check bool) "closed-writer is swallowed, not propagated" true
    swallowed;
  (* A non-closed-writer exception is NOT swallowed by the guard. *)
  let propagated =
    try
      (try raise (Failure "unrelated")
       with exn when is_closed exn -> ());
      false
    with _ -> true
  in
  Alcotest.(check bool) "unrelated Failure propagates past guard" true
    propagated

(* The inflight gate bounds concurrent in-flight connections. Acquiring
   under the limit returns immediately; releasing decrements so a
   subsequent acquire succeeds. We do not test the blocking path (would
   need a paused releaser); the counting contract is what matters here. *)
let test_inflight_acquire_release () =
  (* Eio.Mutex requires an Eio loop. *)
  Eio_main.run @@ fun _env ->
  let before = Pricklypear_image_lib.Http_core.inflight_sem.count in
  Pricklypear_image_lib.Http_core.inflight_acquire ();
  Alcotest.(check int) "acquire increments count" (before + 1)
    Pricklypear_image_lib.Http_core.inflight_sem.count;
  Pricklypear_image_lib.Http_core.inflight_release ();
  Alcotest.(check int) "release decrements count" before
    Pricklypear_image_lib.Http_core.inflight_sem.count

let () =
  Alcotest.run "http_robust"
    [ ("closed-writer predicate",
        [ Alcotest.test_case "recognizes" `Quick
            (fun () -> test_recognizes_closed_writer ())
```

```
        ; Alcotest.test_case "safe wrappers swallow" `Quick
            (fun () -> test_safe_wrappers_swallow_closed_writer ())
        ])
    ; ("inflight gate",
        [ Alcotest.test_case "acquire/release counting" `Quick
            (fun () ->  (test_inflight_acquire_release ()))
        ])
    ]
```

128 image/test/io_eio_test.ml

```
(** H3.1 smoke: Io_eio sequencer invariant + optional DB health under Eio_main.

    Sequencer self-test always runs (no PG).
    DB probe needs local PG (scripts/dev.sh). Soft-fails unless PP_REQUIRE_EIO_DB=1. *)

module IE = Pricklypear_image_lib.Io_eio

let () =
  (* invariant 6: always, no PG required *)
  (try
     Eio_main.run @@ fun _env ->
     if not (IE.sequencer_survives_raise ()) then
       failwith "sequencer_survives_raise returned false";
     print_endline "io_eio_test: OK sequencer survives raise"
   with e ->
     prerr_endline
       ("io_eio_test FAIL sequencer: " ^ Printexc.to_string e);
     exit 1);
  if Sys.getenv_opt "PP_SKIP_EIO_DB" = Some "1" then (
    print_endline "io_eio_test: DB skipped (PP_SKIP_EIO_DB=1)";
    exit 0);
  try
    Eio_main.run @@ fun env ->
    Eio.Switch.run @@ fun sw ->
    let ctx : IE.Ctx.t =
      { sw; net = (Eio.Stdenv.net env :> [ `Generic ] Eio.Net.ty Eio.Std.r) }
    in
    IE.with_ctx ctx (fun () ->
        let cfg = IE.Db.config_from_env () in
        let store =
          IE.Db.make ~cfg
            ~branch_id:"00000000-0000-0000-0000-000000000001" ()
        in
        let n = IE.Db.health_query_sync store in
        if n <> 1 then failwith (Printf.sprintf "health expected 1 row, got %d" n);
        let rows =
          IE.Db.simple_query_sync store "SELECT current_database()"
        in
        (match rows with
         | [ [ db ] ] ->
             Printf.printf "io_eio_test: OK health=1 database=%s\n%!" db
         | _ ->
             failwith
               ("unexpected current_database rows: "
               ^ string_of_int (List.length rows))))
  with e ->
    let msg = Printexc.to_string e in
    if Sys.getenv_opt "PP_REQUIRE_EIO_DB" = Some "1" then (
      prerr_endline ("io_eio_test FAIL: " ^ msg);
      exit 1)
    else (
      Printf.eprintf
        "io_eio_test: soft-fail (set PP_REQUIRE_EIO_DB=1 to enforce): %s\n%!" msg;
      exit 0)
```

129 image/test/jobs_test.ml

```
(** Durable background jobs -- node-shapes + runner (borge/jobs.borg).

    DB-backed regression tests for the dedicated `jobs` table
    (migrations/015_jobs.sql) and image/lib/jobs.ml. Needs Postgres
    (scripts/dev.sh); skips cleanly/unreachable.

    Covers the job node-shapes + runner verify stanzas:
      - job node create / claim / status transitions with runner-owned
        field confinement (submitted_by + branch pinned at submit; the
        visibility prims confine to the caller unless admin).
      - claim SKIP LOCKED skips a row locked by a concurrent connection.
      - exponential backoff schedule (failed retry is re-queued into the
        future; exhausted attempts => dead).
      - dead-runner reaper fails stale 'running' rows left by a dead
        runner_id (retry under attempts/backoff or dead at exhaustion).
      - cancel of a pending job (and only pending; not-yours rejected).

    The boot-time runner fiber (boot.ml) is thin glue over these Jobs
    functions (claim -> eval -> record); the substantive node-shapes and
    runner behaviors live in Jobs and are asserted here directly.
*)

module S = Pricklypear_image_lib.Store
module J = Pricklypear_image_lib.Jobs

let main_id = "00000000-0000-0000-0000-000000000001"

let with_store f =
  Eio_main.run @@ fun env ->
  Lwt_eio.with_event_loop ~clock:(Eio.Stdenv.clock env) @@ fun _ ->
  Eio.Switch.run @@ fun sw ->
  let ctx : Pricklypear_image_lib.Io_eio.Ctx.t =
    { sw; net = (Eio.Stdenv.net env :> [ `Generic ] Eio.Net.ty Eio.Std.r) }
  in
  Pricklypear_image_lib.Io_eio.with_ctx ctx (fun () ->
      let cfg = S.config_from_env () in
      let store = S.make ~cfg ~branch_id:main_id () in
      S.connect store;
      match S.conn_of store with
      | None ->
          Printf.printf
            "SKIP: no postgres (set PP_DB_* / run scripts/dev.sh)\n%!"
      | Some _ -> f store)

let uniq p = Printf.sprintf "j_%s_%d" p (Random.int 1_000_000)

(** True when [sub] is a substring of [s]. *)
let contains_sub (s : string) (sub : string) : bool =
  let n = String.length sub in
  if n = 0 then true
  else
    let rec go i =
      if i + n > String.length s then false
      else if String.sub s i n = sub then true
      else go (i + 1)
    in
    go 0

let now_ms () = Int64.of_float (Unix.gettimeofday () *. 1000.0)

(** Raw read of one job row's scalar fields (no result/error decoding). *)
let read_job_fields (store : S.t) ~(id : string) : (int * string * int64)
     option
    =
  match S.conn_of store with
  | None -> None
  | Some conn ->
      let rows =
        S.exec conn
          ~params:[ Pgx.Value.of_string id ]
          "SELECT attempts, status, run_at::text, runner_id FROM jobs \
```

```
71            WHERE id = $1::uuid"
72       in
73       (match rows with
74        | [ [ attempts; status; run_at; runner_id ] ] ->
75            (match Pgx.Value.to_string attempts with
76             | None -> None
77             | Some a ->
78                 let attempts = try int_of_string a with _ -> 0 in
79                 let status =
80                   match Pgx.Value.to_string status with Some s -> s | None
      -> ""
81                 in
82                 let run_at =
83                   match Pgx.Value.to_string run_at with
84                   | Some r -> (try Int64.of_string r with _ -> 0L)
85                   | None -> 0L
86                 in
87                 let _runner_id =
88                   match Pgx.Value.to_string runner_id with
89                   | Some r -> r
90                   | None -> ""
91                 in
92                 Some (attempts, status, run_at))
93        | _ -> None)
94
95 let delete_job (store : S.t) ~(id : string) : unit =
96   match S.conn_of store with
97   | None -> ()
98   | Some conn ->
99       ignore
100        (S.exec conn ~params:[ Pgx.Value.of_string id ]
101           "DELETE FROM jobs WHERE id = $1::uuid")
102
103 (** The jobs table is shared/persistent; the runner claims oldest-first, so
104     leftover rows from prior runs would be claimed instead of the test's
       own.
105     Give each test a clean slate before exercising claim/backoff/reap. *)
106 let clear_jobs (store : S.t) : unit =
107   match S.conn_of store with
108   | None -> ()
109   | Some conn ->
110       ignore (S.exec conn "DELETE FROM job_steps");
111       ignore (S.exec conn "DELETE FROM jobs")
112
113 (* ------------------------------------------------------------------ *)
114 (* LIVE-RUNNER DETERMINISM (campaign 4.1 follow-up)                    *)
115 (*                                                                     *)
116 (* boot.ml's runner fiber polls the shared jobs table every ~1s and     *)
117 (* claims ANY *due* (run_at <= now) pending job. The DB-backed tests    *)
118 (* share that table with the live image, so a test that submits a due   *)
119 (* job and then claims it itself can lose the race to the runner        *)
120 (* (either the test's claim comes up empty, or the runner takes the     *)
121 (* formerly-locked row the test expected to get). To make the tests     *)
122 (* deterministic WITHOUT stopping the image: submit with a FUTURE       *)
123 (* run_at (the runner's due-scan skips it), then drive run_at to 0 *     *)
124 (* just before the test's own claim. The window between make_due and     *)
125 (* claim_due is microseconds -- far smaller than the runner's poll       *)
126 (* interval -- so the test claims deterministically and the runner never  *)
127 (* competes for the same row.                                          *)
128 (* ------------------------------------------------------------------ *)
129
130 let future = 86_400_000L  (* one day in ms -- runner due-scan (run_at<=now)
       skips *)
131
132 (** Submit a job the live runner will NOT claim (future run_at). *)
133 let submit_future (store : S.t) ~(fn : string) ~(args_json : string)
134     ~(branch : string) ~(submitted_by : string) : string =
135   J.submit ~store ~fn ~args_json ~branch
136     ~submitted_by ~run_at_ms:(Int64.add (now_ms ()) future) ()
137
138 (** Drive a job's run_at to 0 so it is definitely due (run_at <= now); the
139     runner's next due-scan is up to ~1s away, so the subsequent test-side
140     claim_due (microseconds later) wins the row deterministically. *)
```

```
let make_due (store : S.t) ~(id : string) : unit =
  match S.conn_of store with
  | Some conn ->
      ignore
        (S.exec conn ~params:[ Pgx.Value.of_string id ]
           "UPDATE jobs SET run_at = 0 WHERE id = $1::uuid")
  | None -> ()

(* ------------------------------------------------------------------ *)
(* node-shapes: job node create / claim / status transitions          *)
(* ------------------------------------------------------------------ *)

let test_job_lifecycle () =
  with_store (fun store ->
      clear_jobs store;
      let id = submit_future store ~fn:"jfn" ~args_json:"{}"
          ~branch:"system/main" ~submitted_by:"alice"
      in
      (* submit: id returned, not an error string *)
      Alcotest.(check bool) "submit returns a uuid" true
        (String.length id = 36);
      (* status=pending, attempts=0, runner-owned fields pinned *)
      (match read_job_fields store ~id with
       | Some (attempts, status, _) ->
           Alcotest.(check int) "initial attempts" 0 attempts;
           Alcotest.(check string) "initial status" "pending" status
       | None -> Alcotest.fail "job row not present after submit");
      (* get as admin shows the pinned submitted_by + branch *)
      (match J.get ~store ~id ~caller:"bob" ~is_admin:true with
       | Pricklypear_lisp.Ast.VStr s ->
           Alcotest.(check bool) "status visible in JSON" true
             (contains_sub s "pending");
           Alcotest.(check bool) "submitted_by pinned" true
             (contains_sub s "alice");
           Alcotest.(check bool) "branch pinned" true
             (contains_sub s "system/main")
       | _ -> Alcotest.fail "get (admin) returned non-string");
      (* claim: pending -> running, runner_id set, attempts+1 *)
      make_due store ~id;
      (match J.claim_due ~store with
       | None -> Alcotest.fail "claim of a due job returned None"
       | Some j ->
           Alcotest.(check string) "claimed id" id j.J.id;
           Alcotest.(check string) "claimed status" "running" j.J.status;
           Alcotest.(check int) "claimed attempts=1" 1 j.J.attempts;
           Alcotest.(check bool) "runner_id stamped" true
             (j.J.runner_id <> ""));
      (* record_done: running -> done with result *)
      J.record_done ~store ~id ~result_json:"\"ok-result\"";
      (match J.get ~store ~id ~caller:"alice" ~is_admin:false with
       | Pricklypear_lisp.Ast.VStr s ->
           Alcotest.(check bool) "complete status done" true
             (contains_sub s "\"done\"")
       | _ -> Alcotest.fail "done job not visible to its owner");
      delete_job store ~id)

(* runner-owned field confinement: a non-owner, non-admin caller does NOT
   see a job; (jobs) lists only the caller's jobs; cancel rejects others' *)
let test_visibility_confinement () =
  with_store (fun store ->
      clear_jobs store;
      let id = submit_future store ~fn:"jfn2" ~args_json:"{}"
          ~branch:"admin/personal" ~submitted_by:"alice"
      in
      (* non-owner, non-admin cannot see it via get *)
      (match J.get ~store ~id ~caller:"mallory" ~is_admin:false with
       | Pricklypear_lisp.Ast.VNil -> ()
       | _ -> Alcotest.fail "non-owner, non-admin saw a job");
      (* owner sees it *)
      (match J.get ~store ~id ~caller:"alice" ~is_admin:false with
       | Pricklypear_lisp.Ast.VStr _ -> ()
       | _ -> Alcotest.fail "owner could not read own job");
      (* (jobs) confinement: mallory's list is empty, admin sees both *)
```

```
      (match J.list ~store ~caller:"mallory" ~is_admin:false ~limit:50 ()
     with
       | Pricklypear_lisp.Ast.VList l ->
           Alcotest.(check bool) "mallory sees no jobs" true (l = [])
       | _ -> Alcotest.fail "non-admin list returned non-list");
      (match J.list ~store ~caller:"admin" ~is_admin:true ~limit:50 () with
       | Pricklypear_lisp.Ast.VList l ->
           Alcotest.(check bool) "admin sees at least one job" true
             (List.length l >= 1)
       | _ -> Alcotest.fail "admin list returned non-list");
      (* cancel of pending by a non-owner, non-admin -> rejected *)
      (match J.cancel ~store ~id ~caller:"mallory" ~is_admin:false with
       | Pricklypear_lisp.Ast.VStr s ->
           Alcotest.(check bool) "cancel not permitted for others" true
             (contains_sub s "not pending or not yours")
       | _ -> Alcotest.fail "foreign cancel should not return the id");
      (* pending row survives -> admin cancels it *)
      (match J.cancel ~store ~id ~caller:"admin" ~is_admin:true with
       | Pricklypear_lisp.Ast.VStr s ->
           Alcotest.(check bool) "admin cancel returns id" true
             (String.length s = 36)
       | _ -> Alcotest.fail "admin cancel did not return id");
      (* cancel only affects pending: cancelling a done/failed row is a
     no-op *)
      (match J.cancel ~store ~id ~caller:"admin" ~is_admin:true with
       | Pricklypear_lisp.Ast.VStr s ->
           Alcotest.(check bool) "re-cancel of failed row is no-op" true
             (contains_sub s "not pending or not yours")
       | _ -> Alcotest.fail "re-cancel should be a no-op");
      delete_job store ~id)

(* ---------------------------------------------------------------------- *)
(* runner: SKIP LOCKED claim skips a row locked by another connection *)
(* ---------------------------------------------------------------------- *)

let test_claim_skips_locked () =
  with_store (fun store_a ->
      clear_jobs store_a;
      let job1 = submit_future store_a ~fn:"jfn" ~args_json:"{}"
          ~branch:"system/main" ~submitted_by:"alice"
      and job2 = submit_future store_a ~fn:"jfn" ~args_json:"{}"
          ~branch:"system/main" ~submitted_by:"alice"
      in
      (* Make BOTH jobs due BEFORE taking the row lock on job1. Driving
         run_at to 0 after connection B has FOR UPDATE-locked job1 would
         deadlock: store_a's own UPDATE on the locked job1 row would block
         until B commits, but B's transaction cannot commit while this
         function is blocked inside it. Making both due up front leaves
         the claim to exercise only the SKIP LOCKED path (job1 stays
         locked-by-B, so A skips it and takes job2). *)
      make_due store_a ~id:job1;
      make_due store_a ~id:job2;
      (* second, independent connection/transaction locks job1's row.
         Use READ COMMITTED for the lock-holding side so store A's
         SERIALIZABLE claim UPDATE (SKIP LOCKED) neither deadlocks nor
         trips SSI 40001 the moment B's row lock is held -- B holds a
         plain row lock, not a snapshot conflict, so A cleanly skips
         the locked job1 and takes job2. (SERIALIZABLE would abort A's
         write with 'could not serialize access due to concurrent
         update' -- the documented flakiness; READ COMMITTED removes it.) *)
      let cfg = S.config_from_env () in
      let store_b = S.make ~cfg ~branch_id:main_id () in
      S.connect store_b;
      S.with_transaction ~isolation:Pgx.Isolation.Read_committed
        store_b (fun conn_b ->
          match conn_b with
          | None -> Alcotest.fail "no conn on store_b"
          | Some c ->
              (* hold job1's row lock open on connection B *)
              ignore
                (S.exec c ~params:[ Pgx.Value.of_string job1 ]
                   "SELECT id FROM jobs WHERE id=$1::uuid FOR UPDATE");
              (* claim on store A must SKIP the locked job1 and take job2 *)
```

```
285               (match J.claim_due ~store:store_a with
286                | None -> Alcotest.fail "claim skipped nothing"
287                | Some j ->
288                    Alcotest.(check string) "skipped locked, claimed job2"
289                      job2 j.J.id));
290       (* B's tx committed/rolled back -> job1 unlockable; claim it *)
291       (match J.claim_due ~store:store_a with
292        | None -> Alcotest.fail "second claim returned None"
293        | Some j -> Alcotest.(check string) "claimed formerly-locked job1"
294              job1 j.J.id);
295       delete_job store_a ~id:job1;
296       delete_job store_a ~id:job2)
297
298 (* ---------------------------------------------------------------- *)
299 (* runner: exponential backoff schedule + reach-max => dead          *)
300 (* ---------------------------------------------------------------- *)
301
302 let test_backoff_schedule () =
303   with_store (fun store ->
304       clear_jobs store;
305       let before = now_ms () in
306       let id = submit_future store ~fn:"jfn" ~args_json:"{}"
307           ~branch:"system/main" ~submitted_by:"alice"
308       in
309       (* claim (attempts 0->1) then fail with max_attempts=3 *)
310       make_due store ~id;
311       (match J.claim_due ~store with
312        | None -> Alcotest.fail "no job to claim"
313        | Some j ->
314            Alcotest.(check int) "attempts after first claim" 1 j.J.attempts;
315            J.record_failed ~store ~id ~error:"boom" ~max_attempts:3);
316       (* not dead yet (attempts 1 < max 3) -> requeued pending into future
317      *)
318       (match read_job_fields store ~id with
319        | Some (attempts, status, run_at) ->
320            Alcotest.(check int) "attempts still 1" 1 attempts;
321            Alcotest.(check string) "requeued pending" "pending" status;
322            Alcotest.(check bool) "run_at scheduled in future" true
323              (Int64.compare run_at before > 0)
324        | None -> Alcotest.fail "job missing after backoff");
325       (* second claim (attempts 2) + fail -> still pending, later retry *)
326       make_due store ~id;
327       (match J.claim_due ~store with
328        | None -> Alcotest.fail "no job to claim (2nd)"
329        | Some j -> J.record_failed ~store ~id ~error:"boom2"
330              ~max_attempts:3);
331       (match read_job_fields store ~id with
332        | Some (attempts, status, _) ->
333            Alcotest.(check int) "attempts after 2nd fail" 2 attempts;
334            Alcotest.(check string) "retry still pending" "pending" status
335        | None -> Alcotest.fail "job missing after 2nd backoff");
336       (* third claim (attempts 3 = max) + fail -> dead *)
337       make_due store ~id;
338       (match J.claim_due ~store with
339        | None -> Alcotest.fail "no job to claim (3rd)"
340        | Some j -> J.record_failed ~store ~id ~error:"boom3"
341              ~max_attempts:3);
342       (match read_job_fields store ~id with
343        | Some (_, status, _) ->
344            Alcotest.(check string) "exhausted -> dead" "dead" status
345        | None -> Alcotest.fail "job missing after reaching max");
346       delete_job store ~id)
347
348 (* ---------------------------------------------------------------- *)
349 (* runner: dead-runner reaper fails stale 'running' rows            *)
350 (* ---------------------------------------------------------------- *)
351
352 let test_reap_dead () =
353   with_store (fun store ->
354       clear_jobs store;
355       (* a job claimed (running) by a DIFFERENT runner id than ours, with
356          attempts just under max -> reaper requeues it pending *)
357       let id_a = submit_future store ~fn:"jfn" ~args_json:"{}"
```

```
          ~branch:"system/main" ~submitted_by:"alice"
      in
      ignore
        (S.exec (Option.get (S.conn_of store))
           ~params:
             [ Pgx.Value.of_string id_a
             ; Pgx.Value.of_string "runner-OTHER"
             ]
           "UPDATE jobs SET status='running', runner_id=$2, attempts=2 \
            WHERE id=$1::uuid");
      (* a job claimed by another runner at max_attempts -> reaper dead *)
      let id_b = submit_future store ~fn:"jfn" ~args_json:"{}"
          ~branch:"system/main" ~submitted_by:"alice"
      in
      ignore
        (S.exec (Option.get (S.conn_of store))
           ~params:
             [ Pgx.Value.of_string id_b
             ; Pgx.Value.of_string "runner-OTHER"
             ]
           "UPDATE jobs SET status='running', runner_id=$2, attempts=3, \
            max_attempts=3 WHERE id=$1::uuid");
      J.reap_dead ~store;
      (match read_job_fields store ~id:id_a with
       | Some (attempts, status, _) ->
           Alcotest.(check int) "reaped attempts kept" 2 attempts;
           Alcotest.(check string) "stale running requeued pending"
             "pending" status
       | None -> Alcotest.fail "job_a missing after reap");
      (match read_job_fields store ~id:id_b with
       | Some (_, status, _) ->
           Alcotest.(check string) "exhausted stale -> dead" "dead" status
       | None -> Alcotest.fail "job_b missing after reap");
      (* part A done; remove the now-claimable id_a/id_b so the "our runner
         not reaped" check below exercises a solitary due job, else
         claim_due (oldest-first) would claim id_a, not ours. *)
      delete_job store ~id:id_a;
      delete_job store ~id:id_b;
      (* the reaper must NOT touch rows running under OUR runner_id *)
      let mine = submit_future store ~fn:"jfn" ~args_json:"{}"
          ~branch:"system/main" ~submitted_by:"alice"
      in
      make_due store ~id:mine;
      (match J.claim_due ~store with
       | None -> Alcotest.fail "no job to claim for reap-mine check"
       | Some j ->
           Alcotest.(check string) "claimed our own job" mine j.J.id;
           J.record_done ~store ~id:j.J.id ~result_json:"null");
      (* after our claim+done the row is done, not pending from reaper *)
      (match read_job_fields store ~id:mine with
       | Some (_, status, _) ->
           Alcotest.(check string) "our runner not reaped" "done" status
       | None -> Alcotest.fail "our own job missing");
      delete_job store ~id:mine)

(* ------------------------------------------------------------------ *)
(* runner ARGS BUG -- campaign 4.1: eval_src must splice the jsonb args   *)
(* directly as the call literal, not %S-quote it. A stored number arg   *)
(* (args=1) must produce (f 1), never (f "1"); numerics/json/booleans   *)
(* otherwise fail with a numeric-op error and retry to dead.             *)
(* ------------------------------------------------------------------ *)

(* ------------------------------------------------------------------ *)
(* step-checkpoints: current-job binding + checkpoint memoization      *)
(* (campaign 4.2 -- the (with-step "name" thunk) special form memoizes *)
(* (job-id, name); the DB mechanics live in Jobs (checkpoint_lookup /   *)
(* checkpoint_write), which the __step__ prim in session.ml drives.     *)
(* ------------------------------------------------------------------ *)

let test_current_job_id_binding () =
  (* current_job_id_opt reads the Eio fiber key, so it must run on a fiber
     (Eio_main). Outside any job binding there is no job context. *)
  Eio_main.run @@ fun _env ->
```

```
    (* Outside any job binding: no job context. *)
    Alcotest.(check bool) "no job context by default" true
      (J.current_job_id_opt () = None);
    (* Inside with_current_job_id: Some id. *)
    J.with_current_job_id "job-abc" (fun () ->
        Alcotest.(check bool) "job id visible inside binding" true
          (J.current_job_id_opt () = Some "job-abc"));
    (* Cleared after the binding scope exits. *)
    Alcotest.(check bool) "job id cleared after binding" true
      (J.current_job_id_opt () = None)

let test_checkpoint_memoization () =
  with_store (fun store ->
      clear_jobs store;
      (* a real job row to satisfy the job_steps FK (ON DELETE CASCADE) *)
      let jid = submit_future store ~fn:"jfn" ~args_json:"{}"
          ~branch:"system/main" ~submitted_by:"alice"
      in
      (* no checkpoint yet for this (job, step) pair *)
      Alcotest.(check bool) "no checkpoint initially" true
        (J.checkpoint_lookup ~store ~job_id:jid ~name:"load" = None);
      (* write + read round-trips *)
      J.checkpoint_write ~store ~job_id:jid ~name:"load" ~value:"42";
      (match J.checkpoint_lookup ~store ~job_id:jid ~name:"load" with
       | Some v -> Alcotest.(check string) "checkpoint value" "42" v
       | None -> Alcotest.fail "checkpoint not found after write");
      (* a different step name is independent (per (job-id, name) key) *)
      Alcotest.(check bool) "different step independent" true
        (J.checkpoint_lookup ~store ~job_id:jid ~name:"other" = None);
      (* same (job,step) with a different job is independent too *)
      let jid2 = submit_future store ~fn:"jfn" ~args_json:"{}"
          ~branch:"system/main" ~submitted_by:"alice"
      in
      Alcotest.(check bool) "different job independent" true
        (J.checkpoint_lookup ~store ~job_id:jid2 ~name:"load" = None);
      delete_job store ~id:jid2;
      (* overwrite updates in place (a re-run of a step that already has
         a checkpoint on retry REPLACES it -- but the step prim skips the
         re-run entirely when a checkpoint exists, so this is only reachable
         via an explicit re-write; assert the upsert semantics) *)
      J.checkpoint_write ~store ~job_id:jid ~name:"load" ~value:"99";
      (match J.checkpoint_lookup ~store ~job_id:jid ~name:"load" with
       | Some v -> Alcotest.(check string) "checkpoint overwritten" "99" v
       | None -> Alcotest.fail "overwritten checkpoint missing");
      delete_job store ~id:jid)

(* ---------------------------------------------------------------- *)
(* schedules: cron next-fire computation (incl. across-month) +        *)
(* every-ms drift (campaign 4.2)                                       *)
(* ---------------------------------------------------------------- *)

let test_next_minute () =
  let now = now_ms () in
  (* "* * * * *" fires every whole minute; next fire is the next minute *)
  (match J.next_cron_ms "* * * * *" now with
   | None -> Alcotest.fail "next_cron_ms (* * * * *) returned None"
   | Some n ->
       Alcotest.(check bool) "next fire on a minute boundary" true
         (Int64.rem n 60_000L = 0L);
       Alcotest.(check bool) "next fire strictly after now" true
         (Int64.compare n now > 0);
       Alcotest.(check bool) "next fire within two minutes" true
         (Int64.compare n (Int64.add now (Int64.mul 2L 60_000L)) < 0))

let test_every_ms_drift () =
  Alcotest.(check string) "every-ms adds the interval"
    "110" (Int64.to_string (J.next_every_ms 10L 100L));
  Alcotest.(check bool) "integer spec is not cron" false
    (J.spec_is_cron "60000");
  Alcotest.(check bool) "cron spec is cron" true
    (J.spec_is_cron "* * * * *")

let test_cron_across_month () =
```

```
  let now = now_ms () in
  (* "0 0 1 1 *" = every Jan 1 00:00 UTC; the returned fire must be a
     Jan-1 midnight strictly after now (may be this Jan or next). *)
  (match J.next_cron_ms "0 0 1 1 *" now with
   | None -> Alcotest.fail "next_cron_ms (Jan 1) returned None"
   | Some n ->
       let tm = Unix.gmtime (Int64.to_float n /. 1000.0) in
       Alcotest.(check int) "minute 0" 0 tm.Unix.tm_min;
       Alcotest.(check int) "hour 0" 0 tm.Unix.tm_hour;
       Alcotest.(check int) "day 1" 1 tm.Unix.tm_mday;
       Alcotest.(check int) "month January" 0 tm.Unix.tm_mon;
       Alcotest.(check bool) "after now" true (Int64.compare n now > 0));
  (* a day-of-month restricted cron must not fire on non-matching days *)
  Alcotest.(check bool) "Feb 30 (impossible) never matches" true
    (J.next_cron_ms "0 0 30 2 *" now = None)

(* ---------------------------------------------------------------- *)
(* schedules: DB CRUD + due-claim + disable (campaign 4.2)          *)
(* ---------------------------------------------------------------- *)

let test_schedule_crud () =
  with_store (fun store ->
      match S.conn_of store with
      | None -> ()
      | Some conn ->
          ignore (S.exec conn "DELETE FROM schedules");
          (* upsert an every-ms schedule; returns the name; enabled *)
          (match J.schedule_upsert ~store ~name:"pp-test-every"
                   ~spec:"60000" ~fn:"jfn" ~args_json:"{}"
                   ~branch:"system/main" ~submitted_by:"alice" () with
           | Pricklypear_lisp.Ast.VStr n ->
               Alcotest.(check string) "upsert returns name"
    "pp-test-every" n
           | _ -> Alcotest.fail "schedule upsert did not return name");
          (* list shows it *)
          (match J.list_schedules ~store () with
           | Pricklypear_lisp.Ast.VList l ->
               Alcotest.(check bool) "list has the schedule" true
                 (List.exists
                    (function
                      | Pricklypear_lisp.Ast.VStr s ->
                          contains_sub s "pp-test-every"
                      | _ -> false)
                    l)
           | _ -> Alcotest.fail "list_schedules returned non-list");
          (* drive it due and claim (future-run_at-style: the initial
             next_run is now+60000 so the live runner skips it; we set it
             to the past just before OUR claim to win deterministically) *)
          ignore (S.exec conn
                    "UPDATE schedules SET next_run = 0 \
                     WHERE name = 'pp-test-every'");
          (match J.claim_due_schedule ~store with
           | Some sch ->
               Alcotest.(check string) "claimed schedule name"
                 "pp-test-every" sch.J.s_name;
               Alcotest.(check bool) "every_ms captured" true
                 (sch.J.s_every_ms = Some 60_000L);
               J.advance_schedule ~store ~name:sch.J.s_name
                 ~fired_at:(now_ms ())
                 ~cron:sch.J.s_cron ~every_ms:sch.J.s_every_ms
           | None -> Alcotest.fail "claim_due_schedule returned None");
          (* after advance, next_run is in the future -> no longer due *)
          (match J.claim_due_schedule ~store with
           | Some _ -> Alcotest.fail "schedule still due after advance"
           | None -> ());
          (* unschedule disables; a disabled schedule is never due *)
          (match J.unschedule ~store ~name:"pp-test-every" () with
           | Pricklypear_lisp.Ast.VStr n ->
               Alcotest.(check string) "unschedule returns name"
                 "pp-test-every" n
           | _ -> Alcotest.fail "unschedule did not return name");
          ignore (S.exec conn
                    "UPDATE schedules SET next_run = 0 \
```

```
                    WHERE name = 'pp-test-every'");
          (match J.claim_due_schedule ~store with
           | Some _ -> Alcotest.fail "disabled schedule claimed"
           | None -> ());
          let _ = conn in
          ignore (S.exec conn "DELETE FROM schedules"))

let test_eval_src_args () =
  (* numeric arg: stored jsonb '1' -> (f 1) -- the 4.1 args-bug regression
     *)
  Alcotest.(check string) "numeric arg splices bare"
    "(jfn 1)" (J.eval_src ~fn:"jfn" ~args:"1");
  (* string arg: stored jsonb is the QUOTED JSON literal -> (f "hello") *)
  Alcotest.(check string) "string arg keeps its JSON quotes"
    "(jfn \"hello\")" (J.eval_src ~fn:"jfn" ~args:"\"hello\"");
  (* object arg: (f {"a":1}) *)
  Alcotest.(check string) "object arg splices bare"
    "(jfn {\"a\":1})" (J.eval_src ~fn:"jfn" ~args:"{\"a\":1}");
  (* boolean arg *)
  Alcotest.(check string) "bool arg splices bare"
    "(jfn true)" (J.eval_src ~fn:"jfn" ~args:"true");
  (* empty-object (no-arg) job stays a bare call *)
  Alcotest.(check string) "empty object -> no-arg call"
    "(jfn {})" (J.eval_src ~fn:"jfn" ~args:"{}")


let () =
  Random.self_init ();
  Alcotest.run "jobs"
    [ ( "node-shapes"
      , [ Alcotest.test_case "job lifecycle + transitions" `Quick
            test_job_lifecycle
        ; Alcotest.test_case "visibility confinement" `Quick
            test_visibility_confinement ] )
    ; ( "runner"
      , [ Alcotest.test_case "claim skips locked" `Quick
            test_claim_skips_locked
        ; Alcotest.test_case "backoff schedule" `Quick test_backoff_schedule
        ; Alcotest.test_case "dead-runner reaper" `Quick test_reap_dead ] )
    ; ( "args-bug"
      , [ Alcotest.test_case "eval_src splices jsonb args as literal" `Quick
            test_eval_src_args ] )
    ; ( "step-checkpoints"
      , [ Alcotest.test_case "job-id fiber binding" `Quick
            test_current_job_id_binding
        ; Alcotest.test_case "checkpoint memoization (job,step)" `Quick
            test_checkpoint_memoization ] )
    ; ( "schedules"
      , [ Alcotest.test_case "cron next-fire every minute" `Quick
            test_next_minute
        ; Alcotest.test_case "every-ms drift + spec classification" `Quick
            test_every_ms_drift
        ; Alcotest.test_case "cron across-month boundary" `Quick
            test_cron_across_month
        ; Alcotest.test_case "schedule CRUD + claim/advance/disable" `Quick
            test_schedule_crud ] )
    ]
```

130 image/test/local_first_prims_test.ml

```
(** local-first prim tests (borge/local-first.borg lens-model +
    replication-units, campaign 8.1).

    Needs Postgres (scripts/dev.sh). Skips cleanly when unreachable.

    - lens/fork names + creates a lens branch as a NAMED CHILD of the
      checked-out branch (the pinned <owner>/lens-<device>[-<date>]
      contract); the fork is an EMPTY-LENS child (parent pointer only,
      copy-on-fork) so it owns zero code rows and resolves through
      branch ancestry. The server branch stays the parent; single-writer
      per lens is preserved by construction (no server writes into it).
    - replication/unit materializes the exact authorized-read subtree the
      identity would replicate: data.u.<user>.* + data.shared.* + the
      code lens + branch ancestry; path-prefix RLS confinement is the
      replication boundary.
*)

module S = Pricklypear_image_lib.Store

let main_id = "00000000-0000-0000-0000-000000000001"

let with_store f =
  Eio_main.run @@ fun env ->
  Lwt_eio.with_event_loop ~clock:(Eio.Stdenv.clock env) @@ fun _ ->
  Eio.Switch.run @@ fun sw ->
  let ctx : Pricklypear_image_lib.Io_eio.Ctx.t =
    {
      sw;
      net = (Eio.Stdenv.net env :> [ `Generic ] Eio.Net.ty Eio.Std.r);
    }
  in
  Pricklypear_image_lib.Io_eio.with_ctx ctx (fun () ->
      let cfg = S.config_from_env () in
      let store = S.make ~cfg ~branch_id:main_id () in
      S.connect store;
      match S.conn_of store with
      | None ->
          Printf.printf
            "SKIP: no postgres (set PP_DB_* / run scripts/dev.sh)\n%!"
      | Some _ -> f store)

(* ---- lens-model: lens/fork names + creates an empty-lens child ---- *)
let test_lens_fork () =
  with_store (fun store ->
      let uniq = Printf.sprintf "probe%d" (Random.int 1_000_000) in
      let owner =
        match
          List.find_opt
            (fun (b : Pricklypear_common.Branch.t) -> b.id = main_id)
            (S.list_branches store)
        with
        | Some b -> b.owner
        | None -> Alcotest.fail "current branch (system/main) not in
    registry"
      in
      let v =
        Pricklypear_image_lib.Crud.lens_fork store ~device:uniq ~date:None
    ()
      in
      let s =
        match v with
        | Pricklypear_lisp.Ast.VStr x -> x
        | _ -> Alcotest.fail "lens/fork must return a VStr"
      in
      (* "ok <owner>/lens-<device> (<uuid>)" *)
      Alcotest.(check bool) "ack prefix" true
        (String.starts_with ~prefix:("ok " ^ owner ^ "/lens-" ^ uniq) s);
      (* the lens branch now exists and is an empty-lens child of main *)
      let created =
        List.find_opt
          (fun (b : Pricklypear_common.Branch.t) ->
```

```
            b.name = "lens-" ^ uniq)
          (S.list_branches store)
      in
      (match created with
       | None -> Alcotest.fail "lens branch not created"
       | Some b ->
           Alcotest.(check (option string)) "parent is current branch
    (main)"
             (Some main_id) b.parent_id;
           Alcotest.(check string) "owner preserved from current branch"
             owner b.owner;
           (* empty-lens: zero own function rows on the lens *)
           let n = S.count_functions_on_branch store ~branch_id:b.id in
           Alcotest.(check int) "empty-lens fork owns no code rows" 0 n;
           (* cleanup: drop the lens branch row (it owns no rows) *)
           (match S.conn_of store with
            | None -> ()
            | Some conn ->
                ignore
                  (S.exec conn
                     ~params:[ Pgx.Value.of_string b.id ]
                     "DELETE FROM branches WHERE id = $1::uuid")));
      (* confirm cleanup *)
      let still = List.exists
          (fun (b : Pricklypear_common.Branch.t) -> b.name = "lens-" ^ uniq)
          (S.list_branches store)
      in
      Alcotest.(check bool) "lens cleaned up" false still)

(* ---- replication-units: the manifest materializes the owned unit ---- *)
let test_replication_unit () =
  with_store (fun store ->
      (* Insert rows for the session user (tests run as "anonymous", the
         Request_context default) so the manifest user_count must reflect
         them. This locks the label-ENCODED replication prefix: the raw
         username prefix (data.u.<user>) would match nothing and under-count
         to 0, so a real user_count >= 2 proves the encoded prefix is used.
    *)
      (match S.conn_of store with
       | None -> ()
       | Some conn ->
           List.iteri
             (fun i _ ->
               ignore
                 (Pricklypear_image_lib.Tree_store.insert_datum conn
                    ~type_name:(Printf.sprintf "lf_%d" i)
                    ~fields_json:(Printf.sprintf "{\"n\":%d}" i)
                    ~author:"anonymous"))
             [ 1; 2 ]);
      match Pricklypear_image_lib.Crud.replication_unit store () with
      | Pricklypear_lisp.Ast.VStr json ->
          (match Yojson.Safe.from_string json with
           | `Assoc kv ->
               let field key =
                 match List.assoc_opt key kv with
                 | Some (`String s) -> s
                 | _ -> "<missing>"
               in
               let owner, branch_id =
                 field "owner", field "branch_id"
               in
               Alcotest.(check bool) "owner present" (owner <> "")
                 (owner <> "<missing>");
               Alcotest.(check string) "branch_id is current branch"
                 main_id branch_id;
               (* ancestry is self-first and includes the branch *)
               (match List.assoc_opt "ancestry" kv with
                | Some (`List ids) ->
                    Alcotest.(check bool) "ancestry is a non-empty list"
                      true (ids <> [])
                | _ -> Alcotest.fail "ancestry missing");
               (match List.assoc_opt "data" kv with
                | Some (`Assoc dk) ->
```

```
                    let df key =
                      match List.assoc_opt key dk with
                      | Some (`String s) -> s
                      | _ -> "<missing>"
                    in
                    Alcotest.(check bool) "user data prefix present"
                      true
                      (String.starts_with ~prefix:"data.u." (df "user"));
                    (match List.assoc_opt "user_count" dk with
                     | Some (`Int n) ->
                         Alcotest.(check bool) "user_count reflects session
    rows"
                           true (n >= 2)
                     | _ -> Alcotest.fail "user_count missing");
                    Alcotest.(check string) "shared seed prefix"
                      "data.shared" (df "shared");
                    Alcotest.(check bool) "shared_count is an int"
                      true
                      (match List.assoc_opt "shared_count" dk with
                       | Some (`Int _) -> true | _ -> false)
                | _ -> Alcotest.fail "data absent");
               Alcotest.(check bool) "scope names the boundary"
                 true
                 (String.starts_with ~prefix:"data.u.<user>.*"
                    (field "scope"))
           | _ -> Alcotest.fail "replication/unit not an object")
      | _ -> Alcotest.fail "replication/unit must return a VStr")

let () =
  Random.self_init ();
  Alcotest.run "local_first_prims"
    [ ( "lens-model+replication",
        [
          Alcotest.test_case "lens/fork creates empty-lens named child"
            `Quick test_lens_fork;
          Alcotest.test_case "replication/unit materializes the owned unit"
            `Quick test_replication_unit;
        ] ) ]
```

131 image/test/media_test.ml

```
(** Pure unit tests for storage.media helpers (no DB). *)

open Alcotest

let test_normalize_and_whitelist () =
  check bool "png ok" true
    (Pricklypear_image_lib.Media.content_type_allowed "image/png");
  check bool "jpeg ok" true
    (Pricklypear_image_lib.Media.content_type_allowed "image/jpeg;
     charset=binary");
  check bool "pdf ok" true
    (Pricklypear_image_lib.Media.content_type_allowed "application/pdf");
  check bool "octet denied" false
    (Pricklypear_image_lib.Media.content_type_allowed
     "application/octet-stream");
  check bool "html denied" false
    (Pricklypear_image_lib.Media.content_type_allowed "text/html");
  check string "normalize strips params" "image/png"
    (Pricklypear_image_lib.Media.normalize_content_type "Image/PNG;
     charset=x")

let test_sanitize_filename () =
  check (option string) "plain"
    (Some "shot.png")
    (Pricklypear_image_lib.Media.sanitize_filename (Some "shot.png"));
  check (option string) "strips path"
    (Some "shot.png")
    (Pricklypear_image_lib.Media.sanitize_filename (Some
     "/tmp/evil/shot.png"));
  check (option string) "empty -> none"
    None
    (Pricklypear_image_lib.Media.sanitize_filename (Some "  "));
  check (option string) "dotdot -> none"
    None
    (Pricklypear_image_lib.Media.sanitize_filename (Some ".."))

let test_url_and_b64 () =
  check string "url" "/media/abc"
    (Pricklypear_image_lib.Media.url_of_id "abc");
  let raw = "hello-media" in
  let b64 = Base64.encode_string raw in
  match Pricklypear_image_lib.Media.b64_decode_opt b64 with
  | Some s -> check string "roundtrip" raw s
  | None -> fail "b64 decode failed"

let test_sha256 () =
  let h = Pricklypear_image_lib.Media.sha256_hex "abc" in
  check string "sha256(abc)"
    "ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad" h

let test_limits () =
  check int "http 5MiB" (5 * 1024 * 1024)
    Pricklypear_image_lib.Media.max_http_bytes;
  check int "eval 256KiB" (256 * 1024)
    Pricklypear_image_lib.Media.max_eval_put_bytes

(** BORGE-1785695911: visibility confinement SQL -- non-admin actor is
    confined to own blobs; admin (or kernel caller, visible_to=None) sees
    all. The SQL fragment is the single predicate list/get/delete share. *)
let test_visible_sql () =
  check string "non-admin confines" "author = $2"
    (Pricklypear_image_lib.Media.visible_sql ~is_admin:false
     ~actor_param:2);
  check string "admin unconstrained" "TRUE"
    (Pricklypear_image_lib.Media.visible_sql ~is_admin:true ~actor_param:2)

let test_multipart_boundary () =
  let open Pricklypear_image_lib.Http_media in
  check (option string) "quoted"
    (Some "----WebKitFormBoundary7MA4YWxkTrZu0gW")
    (multipart_boundary_for_test
```

```
      "multipart/form-data; boundary=\"----WebKitFormBoundary7MA4YWxkTrZu0gW\"");
  check (option string) "plain"
    (Some "abc123")
    (multipart_boundary_for_test "multipart/form-data; boundary=abc123");
  check (option string) "not multipart" None
    (multipart_boundary_for_test "image/png")

let test_multipart_parse () =
  let open Pricklypear_image_lib.Http_media in
  let boundary = "----ppbound" in
  let body =
    String.concat "\r\n"
      [ "--" ^ boundary
      ; "Content-Disposition: form-data; name=\"file\"; filename=\"x.png\""
      ; "Content-Type: image/png"
      ; ""
      ; "PNGDATA"
      ; "--" ^ boundary ^ "--"
      ; ""
      ]
  in
  match parse_multipart_for_test ~boundary body with
  | Error e -> fail e
  | Ok parts ->
      (match pick_upload_file_for_test parts with
       | None -> fail "no file part"
       | Some p ->
           check string "field" "file" p.field_name;
           check (option string) "filename" (Some "x.png") p.filename;
           check (option string) "ct" (Some "image/png") p.content_type;
           check string "data" "PNGDATA" p.data)

let () =
  run "media"
    [ ( "whitelist"
      , [ test_case "content types" `Quick test_normalize_and_whitelist ] )
    ; ( "filename"
      , [ test_case "sanitize" `Quick test_sanitize_filename ] )
    ; ( "codec"
      , [ test_case "url + b64" `Quick test_url_and_b64
        ; test_case "sha256" `Quick test_sha256
        ] )
    ; ( "limits"
      , [ test_case "byte caps" `Quick test_limits ] )
    ; ( "visibility"
      , [ test_case "sql confinement" `Quick test_visible_sql ] )
    ; ( "multipart"
      , [ test_case "boundary" `Quick test_multipart_boundary
        ; test_case "parse file part" `Quick test_multipart_parse
        ] )
    ]
```

132 image/test/merge_promote_test.ml

```
(** merge conflict report + promote routes-with-handler
    Spec: branching.merge, branching.promote
    Needs Postgres (scripts/dev.sh). Skips if unreachable. *)

module S = Pricklypear_image_lib.Store

let main_id = "00000000-0000-0000-0000-000000000001"

let with_store f =
  Eio_main.run @@ fun env ->
  Lwt_eio.with_event_loop ~clock:(Eio.Stdenv.clock env) @@ fun _ ->
  Eio.Switch.run @@ fun sw ->
  let ctx : Pricklypear_image_lib.Io_eio.Ctx.t =
    {
      sw;
      net = (Eio.Stdenv.net env :> [ `Generic ] Eio.Net.ty Eio.Std.r);
    }
  in
  Pricklypear_image_lib.Io_eio.with_ctx ctx (fun () ->
      let cfg = S.config_from_env () in
      let store = S.make ~cfg ~branch_id:main_id () in
      S.connect store;
      match S.conn_of store with
      | None ->
          Printf.printf
            "SKIP: no postgres (set PP_DB_* / run scripts/dev.sh)\n%!"
      | Some _ -> f store)

let expr_of_int n =
  Pricklypear_lisp.Ast.Lit (Pricklypear_lisp.Ast.VInt (Int64.of_int n))

let uniq p = Printf.sprintf "%s_%d" p (Random.int 1_000_000)

let count_types ~conn ~branch_id =
  match
    S.exec conn
      ~params:[ Pgx.Value.of_string branch_id ]
      "SELECT COUNT(*)::text FROM type_defs WHERE branch_id = $1"
  with
  | [ [ c ] ] ->
      (match Pgx.Value.to_string c with
       | Some s -> int_of_string_opt s |> Option.value ~default:0
       | None -> 0)
  | _ -> 0

let count_routes ~conn ~branch_id =
  match
    S.exec conn
      ~params:[ Pgx.Value.of_string branch_id ]
      "SELECT COUNT(*)::text FROM view_bindings WHERE branch_id = $1"
  with
  | [ [ c ] ] ->
      (match Pgx.Value.to_string c with
       | Some s -> int_of_string_opt s |> Option.value ~default:0
       | None -> 0)
  | _ -> 0

let lookup_route ~conn ~branch_id ~route =
  match
    S.exec conn
      ~params:
        [ Pgx.Value.of_string branch_id; Pgx.Value.of_string route ]
      "SELECT handler, COALESCE(visibility,'private') FROM view_bindings \
       WHERE branch_id = $1 AND route = $2"
  with
  | [ [ h; v ] ] ->
      (match Pgx.Value.to_string h, Pgx.Value.to_string v with
       | Some hh, Some vv -> Some (hh, vv)
       | _ -> None)
  | _ -> None

```

```
(** Target moved newer than source ? conflict named in result string. *)
let test_merge_conflict_report () =
  with_store (fun store ->
      match S.conn_of store with
      | None -> ()
      | Some conn ->
          let suffix = string_of_int (Random.int 1_000_000) in
          let child_name = "mrg_child_" ^ suffix in
          let fname = "mrg_conflict_" ^ suffix in
          store.branch_id <- main_id;
          (* seed on main *)
          S.persist_function store fname (expr_of_int 1) "tester";
          match
            S.fork_branch store ~parent_id:main_id ~owner:"mrg_test"
              ~child_name
          with
          | None -> Alcotest.fail "fork failed"
          | Some child_id ->
              (* child pins older-ish body first *)
              store.branch_id <- child_id;
              S.persist_function store fname (expr_of_int 2) "tester";
              (* bump main (target) AFTER child so target.updated_at >
    source *)
              store.branch_id <- main_id;
              Unix.sleepf 0.05;
              S.persist_function store fname (expr_of_int 3) "tester";
              let r =
                S.merge_branch store ~source_id:child_id ~target_id:main_id
              in
              (match r with
               | None -> Alcotest.fail "merge failed"
               | Some res ->
                   let msg = S.format_merge_result res in
                   Alcotest.(check bool) "has conflict name" true
                     (List.mem fname res.conflicts);
                   Alcotest.(check bool) "msg shape" true
                     (String.contains msg '('
                      && String.contains msg ':'));
              let _ =
                S.delete_function_on_branch store ~branch_id:main_id
                  ~name:fname
              in
              let _ =
                S.delete_function_on_branch store ~branch_id:child_id
                  ~name:fname
              in
              ignore conn)

(** Merge must not copy type_defs or view_bindings. *)
let test_merge_scope_no_types_or_routes () =
  with_store (fun store ->
      match S.conn_of store with
      | None -> ()
      | Some conn ->
          let suffix = string_of_int (Random.int 1_000_000) in
          let child_name = "mrg_scope_" ^ suffix in
          let fname = "mrg_scope_fn_" ^ suffix in
          let tname = "mrg_scope_ty_" ^ suffix in
          let route = "/mrg-scope-" ^ suffix in
          store.branch_id <- main_id;
          let main_types_before = count_types ~conn ~branch_id:main_id in
          let main_routes_before = count_routes ~conn ~branch_id:main_id in
          match
            S.fork_branch store ~parent_id:main_id ~owner:"mrg_test"
              ~child_name
          with
          | None -> Alcotest.fail "fork failed"
          | Some child_id ->
              store.branch_id <- child_id;
              S.persist_function store fname (expr_of_int 1) "tester";
              S.persist_type store ~name:tname ~fields:[ "a"; "b" ]
    ~author:"tester";
              S.register_route store ~route ~handler:fname
```

```
                ~visibility:"public" ();
              let _ =
                S.merge_branch store ~source_id:child_id ~target_id:main_id
              in
              let main_types_after = count_types ~conn ~branch_id:main_id in
              let main_routes_after = count_routes ~conn ~branch_id:main_id
    in
              Alcotest.(check int) "no type_defs copied" main_types_before
                main_types_after;
              Alcotest.(check int) "no view_bindings copied"
    main_routes_before
                main_routes_after;
              (* function should have landed *)
              (match
                 S.lookup_function_on_branch store ~branch_id:main_id
                   ~name:fname
               with
               | None -> Alcotest.fail "fn not merged"
               | Some _ -> ());
              let _ =
                S.delete_function_on_branch store ~branch_id:main_id
                  ~name:fname
              in
              let _ =
                S.delete_function_on_branch store ~branch_id:child_id
                  ~name:fname
              in
              ())

(** Promote handler also copies its route; unrelated route stays put. *)
let test_promote_routes_with_handler () =
  with_store (fun store ->
      match S.conn_of store with
      | None -> ()
      | Some conn ->
          let suffix = string_of_int (Random.int 1_000_000) in
          let child_name = "prm_child_" ^ suffix in
          let h_yes = "prm_yes_" ^ suffix in
          let h_no = "prm_no_" ^ suffix in
          let r_yes = "/prm-yes-" ^ suffix in
          let r_no = "/prm-no-" ^ suffix in
          match
            S.fork_branch store ~parent_id:main_id ~owner:"prm_test"
              ~child_name
          with
          | None -> Alcotest.fail "fork failed"
          | Some child_id ->
              store.branch_id <- child_id;
              S.persist_function store h_yes (expr_of_int 1) "tester";
              S.persist_function store h_no (expr_of_int 2) "tester";
              S.register_route store ~route:r_yes ~handler:h_yes
                ~visibility:"public" ();
              S.register_route store ~route:r_no ~handler:h_no
                ~visibility:"private" ();
              let n =
                S.promote_functions store ~source_id:child_id
                  ~target_id:main_id ~names:[ h_yes ]
              in
              Alcotest.(check int) "promoted 1" 1 n;
              (match lookup_route ~conn ~branch_id:main_id ~route:r_yes with
               | None -> Alcotest.fail "route for promoted handler missing"
               | Some (h, vis) ->
                   Alcotest.(check string) "handler" h_yes h;
                   Alcotest.(check string) "visibility preserved" "public"
    vis);
              (match lookup_route ~conn ~branch_id:main_id ~route:r_no with
               | Some _ -> Alcotest.fail "unrelated route should stay put"
               | None -> ());
              (* cleanup *)
              let _ =
                S.delete_function_on_branch store ~branch_id:main_id
                  ~name:h_yes
              in
```

```
              let _ =
                S.exec conn
                  ~params:
                    [ Pgx.Value.of_string main_id; Pgx.Value.of_string
    r_yes ]
                  "DELETE FROM view_bindings WHERE branch_id = $1 AND route
    = $2"
              in
              ())

let test_format_merge_result () =
  Alcotest.(check string) "no conflicts" "merged 3 rows"
    (S.format_merge_result { count = 3; conflicts = [] });
  Alcotest.(check string) "with conflicts"
    "merged 5 rows (2 conflicts: a, b)"
    (S.format_merge_result { count = 5; conflicts = [ "a"; "b" ] })

let () =
  Random.self_init ();
  Alcotest.run "merge-promote"
    [ ( "format"
      , [ Alcotest.test_case "format_merge_result" `Quick
            test_format_merge_result
        ] )
    ; ( "db"
      , [ Alcotest.test_case "conflict report" `Quick
    test_merge_conflict_report
        ; Alcotest.test_case "scope no types/routes" `Quick
            test_merge_scope_no_types_or_routes
        ; Alcotest.test_case "promote routes with handler" `Quick
            test_promote_routes_with_handler
        ] )
    ]
```

133 image/test/ops_incidents_test.ml

```
(** Pricklypear Image -- ops_incidents filter tests (no DB).

    [Pricklypear_image_lib.Ops_incidents.is_expected_runtime_error] classifies an exception
    as an expected user/runtime error (log, don't file as a reliability
    incident) vs a genuine fault (file). This is a pure function over
    the [Interp_error] variants, so it can be unit-tested without a
    session or store. *)
open Pricklypear_lisp.Ast

let expected = [
  ("budget exceeded is expected", `Quick, fun () ->
    Alcotest.(check bool) "budget" true
      (Pricklypear_image_lib.Ops_incidents.is_expected_runtime_error
         (Interp_error Budget_exceeded)));
  ("user error is expected", `Quick, fun () ->
    Alcotest.(check bool) "user_error" true
      (Pricklypear_image_lib.Ops_incidents.is_expected_runtime_error
         (Interp_error (User_error ("car needs a list or pair", "bad-args")))));
  ("arity mismatch is expected", `Quick, fun () ->
    Alcotest.(check bool) "arity" true
      (Pricklypear_image_lib.Ops_incidents.is_expected_runtime_error
         (Interp_error (Arity_mismatch ("lambda", 1, 2)))));
  ("wrong arg count is expected", `Quick, fun () ->
    Alcotest.(check bool) "wrong_arg" true
      (Pricklypear_image_lib.Ops_incidents.is_expected_runtime_error
         (Interp_error (Wrong_arg_count ("foo", 1, 2)))));
  ("type mismatch is expected", `Quick, fun () ->
    Alcotest.(check bool) "type_mismatch" true
      (Pricklypear_image_lib.Ops_incidents.is_expected_runtime_error
         (Interp_error (Type_mismatch ("list", VNil)))));
]

let still_file = [
  ("unbound var is filed", `Quick, fun () ->
    Alcotest.(check bool) "unbound" false
      (Pricklypear_image_lib.Ops_incidents.is_expected_runtime_error
         (Interp_error (Unbound_var "nope"))));
  ("not a function is filed", `Quick, fun () ->
    Alcotest.(check bool) "not_a_fn" false
      (Pricklypear_image_lib.Ops_incidents.is_expected_runtime_error
         (Interp_error (Not_a_function VNil))));
  ("prim error is filed", `Quick, fun () ->
    Alcotest.(check bool) "prim_error" false
      (Pricklypear_image_lib.Ops_incidents.is_expected_runtime_error
         (Interp_error (Prim_error ("get", "boom", "prim_error")))));
  ("parse error is filed (route shouldn't parse-fail at runtime)", `Quick,
     fun () ->
    Alcotest.(check bool) "parse" false
      (Pricklypear_image_lib.Ops_incidents.is_expected_runtime_error
         (Interp_error (Parse_error ("oops", 1, 1)))));
  ("non-Interp_error is filed", `Quick, fun () ->
    Alcotest.(check bool) "other" false
      (Pricklypear_image_lib.Ops_incidents.is_expected_runtime_error
     (Failure "boom")));
]

let () =
  Alcotest.run "ops_incidents" [
    ("expected", expected);
    ("still_file", still_file);
  ]
```

134 image/test/packages_test.ml

```
(** Packages prefix helpers *)
module P = Pricklypear_image_lib.Packages

let test_package_of_name () =
  Alcotest.(check string) "todo/list" "todo" (P.package_of_name
    "todo/list");
  Alcotest.(check string) "nested" "todo/ui" (P.package_of_name
    "todo/ui/row");
  Alcotest.(check string) "root" "" (P.package_of_name "hi-page")

let test_local_name () =
  Alcotest.(check string) "list" "list" (P.local_name "todo/list");
  Alcotest.(check string) "root local" "hi-page" (P.local_name "hi-page")

let test_join () =
  Alcotest.(check string) "join" "todo/list"
    (P.join_name ~package:"todo" ~local:"list");
  Alcotest.(check string) "root join" "greet"
    (P.join_name ~package:"" ~local:"greet");
  Alcotest.(check string) "strip slash" "todo/list"
    (P.join_name ~package:"todo/" ~local:"list")

let test_encode () =
  Alcotest.(check string) "encode" "todo%2Flist" (P.name_encode
    "todo/list");
  Alcotest.(check string) "decode" "todo/list" (P.name_decode "todo%2Flist")

let test_group () =
  let defs =
    [ ("hi-page", (), "a")
    ; ("todo/list", (), "a")
    ; ("todo/add", (), "a")
    ; ("calendar/today", (), "b")
    ]
  in
  let g = P.group_by_package defs in
  let pkgs = List.map fst g in
  Alcotest.(check (list string)) "pkg order" [""; "calendar"; "todo"] pkgs;
  let todo = List.assoc "todo" g in
  Alcotest.(check int) "todo count" 2 (List.length todo)

let () =
  Alcotest.run "pricklypear-packages"
    [ ( "packages"
      , [ ("package_of_name", `Quick, test_package_of_name)
        ; ("local_name", `Quick, test_local_name)
        ; ("join", `Quick, test_join)
        ; ("encode", `Quick, test_encode)
        ; ("group", `Quick, test_group)
        ] )
    ]
```

135 image/test/primitives_test.ml

```
(** Pricklypear Image -- primitive tests (no DB).

    Tests the PURE primitives in image/lib/primitives.ml by evaluating
    Lisp forms against a session prim_table + env built the same way
    Session.make builds them. We don't actually call Session (which
    needs manifest + store); instead we build the table directly.

    H1.7: lisp eval is direct-style; pure image prims still register as
    Lwt via [register_sync], serviced by [Control_rim.run_lwt]. *)
open Pricklypear_lisp.Ast

(* Build a prim_table + an env with all prims auto-defined as VPrim,
   so `(+)` etc. resolve. Mirrors image/lib/session.ml's setup. *)
let make_test_env () =
  let prim_table = Pricklypear_image_lib.Primitives.make_with_defaults () in
  let env = Pricklypear_lisp.Env.create () in
  Hashtbl.iter
    (fun name _ ->
      Pricklypear_lisp.Env.define env name (VPrim name))
    prim_table;
  Pricklypear_lisp.Env.define env "pi" (VFloat Float.pi);
  (env, prim_table)

(* Helper: eval a Lisp form string against the test env under the effect
     rim. *)
let eval s =
  let env, prims = make_test_env () in
  Pricklypear_image_lib.Control_rim.run ~prims (fun () ->
      Pricklypear_lisp.Eval.eval_string_direct
        ~prims:Pricklypear_lisp.Eval.empty_prims env s)

(* Helper: eval to a string-pretty form. *)
let eval_str s =
  Pricklypear_lisp.Reader.pp_value (eval s)

(* --- arithmetic --- *)
let arithmetic_tests = [
  ("+ variadic sums", `Quick, fun () ->
    Alcotest.(check string) "sum" "42" (eval_str "(+ 1 2 39)"));
  ("+ identity", `Quick, fun () ->
    Alcotest.(check string) "0-arg add" "0" (eval_str "(+)"));
  ("- binary", `Quick, fun () ->
    Alcotest.(check string) "sub" "5" (eval_str "(- 10 5)"));
  ("- unary negates", `Quick, fun () ->
    Alcotest.(check string) "neg" "-7" (eval_str "(- 7)"));
  ("* variadic products", `Quick, fun () ->
    Alcotest.(check string) "prod" "24" (eval_str "(* 2 3 4)"));
  ("* identity", `Quick, fun () ->
    Alcotest.(check string) "1 mul" "1" (eval_str "(*)"));
  ("div integer", `Quick, fun () ->
    Alcotest.(check string) "div" "5" (eval_str "(div 10 2)"));
  ("/ alias of div", `Quick, fun () ->
    Alcotest.(check string) "/" "5" (eval_str "(/ 10 2)"));
  ("/ truncates ints", `Quick, fun () ->
    Alcotest.(check string) "/ trunc" "2" (eval_str "(/ 5 2)"));
  ("= numeric equality true", `Quick, fun () ->
    Alcotest.(check string) "eq true" "#t" (eval_str "(= 5 5)"));
  ("= numeric equality false", `Quick, fun () ->
    Alcotest.(check string) "eq false" "#f" (eval_str "(= 5 6)"));
  ("<  less-than true", `Quick, fun () ->
    Alcotest.(check string) "lt true" "#t" (eval_str "(< 3 4)"));
  (">  greater-than false", `Quick, fun () ->
    Alcotest.(check string) "gt" "#f" (eval_str "(> 3 4)"));
  ("<= true on equal", `Quick, fun () ->
    Alcotest.(check string) "le eq" "#t" (eval_str "(<= 4 4)"));
  (">= false on less", `Quick, fun () ->
    Alcotest.(check string) "ge" "#f" (eval_str "(>= 3 5)"));
  ("not flips bool", `Quick, fun () ->
    Alcotest.(check string) "not true" "#f" (eval_str "(not #t)"));
  ("not nil -> true", `Quick, fun () ->
    Alcotest.(check string) "not nil" "#t" (eval_str "(not ())"));
```

```
]

(* --- lisp-interpreter.math --- *)
let math_tests = [
  ("expt int-in int-out", `Quick, fun () ->
    Alcotest.(check string) "2^10" "1024" (eval_str "(expt 2 10)"));
  ("expt 0 exp", `Quick, fun () ->
    Alcotest.(check string) "n^0" "1" (eval_str "(expt 9 0)"));
  ("expt float path", `Quick, fun () ->
    let v = eval "(expt 2.0 0.5)" in
    match v with
    | VFloat f ->
        Alcotest.(check bool) "~sqrt2" true
          (abs_float (f -. sqrt 2.) < 1e-9)
    | _ -> Alcotest.fail "expected float");
  ("sqrt 9", `Quick, fun () ->
    Alcotest.(check string) "sqrt" "3.0" (eval_str "(sqrt 9)"));
  ("sqrt negative errors", `Quick, fun () ->
    match try ignore (eval "(sqrt -1)"); None with
          | Interp_error (User_error (m, _)) -> Some m
          | _ -> Some "other"
    with
    | Some m -> Alcotest.(check bool) "msg" true (String.length m > 0)
    | None -> Alcotest.fail "expected error");
  ("exp 0", `Quick, fun () ->
    Alcotest.(check string) "e0" "1.0" (eval_str "(exp 0)"));
  ("log e ~ 1", `Quick, fun () ->
    let v = eval "(log (exp 1))" in
    match v with
    | VFloat f -> Alcotest.(check bool) "~1" true (abs_float (f -. 1.) <
     1e-9)
    | _ -> Alcotest.fail "float");
  ("log base 10 of 1000", `Quick, fun () ->
    let v = eval "(log 1000 10)" in
    match v with
    | VFloat f -> Alcotest.(check bool) "~3" true (abs_float (f -. 3.) <
     1e-9)
    | _ -> Alcotest.fail "float");
  ("log non-positive errors", `Quick, fun () ->
    match try ignore (eval "(log 0)"); None with
          | Interp_error (User_error (_, _)) -> Some ()
          | _ -> None
    with
    | Some () -> ()
    | None -> Alcotest.fail "expected error");
  ("sin 0", `Quick, fun () ->
    Alcotest.(check string) "sin0" "0.0" (eval_str "(sin 0)"));
  ("cos 0", `Quick, fun () ->
    Alcotest.(check string) "cos0" "1.0" (eval_str "(cos 0)"));
  ("tan 0", `Quick, fun () ->
    Alcotest.(check string) "tan0" "0.0" (eval_str "(tan 0)"));
  ("asin 0", `Quick, fun () ->
    Alcotest.(check string) "asin0" "0.0" (eval_str "(asin 0)"));
  ("acos 1", `Quick, fun () ->
    Alcotest.(check string) "acos1" "0.0" (eval_str "(acos 1)"));
  ("atan 0", `Quick, fun () ->
    Alcotest.(check string) "atan0" "0.0" (eval_str "(atan 0)"));
  ("atan2 quadrant", `Quick, fun () ->
    let v = eval "(atan 1.0 1.0)" in
    match v with
    | VFloat f ->
        Alcotest.(check bool) "pi/4" true
          (abs_float (f -. (Float.pi /. 4.)) < 1e-9)
    | _ -> Alcotest.fail "float");
  ("asin domain error", `Quick, fun () ->
    match try ignore (eval "(asin 2)"); None with
          | Interp_error (User_error (_, _)) -> Some ()
          | _ -> None
    with
    | Some () -> ()
    | None -> Alcotest.fail "expected error");
  ("pi bound value", `Quick, fun () ->
    let v = eval "pi" in
```

```
    match v with
    | VFloat f ->
        Alcotest.(check bool) "pi" true (abs_float (f -. Float.pi) < 1e-15)
    | _ -> Alcotest.fail "pi not float");
  ("number? int", `Quick, fun () ->
    Alcotest.(check string) "n?" "#t" (eval_str "(number? 3)"));
  ("number? float", `Quick, fun () ->
    Alcotest.(check string) "n?f" "#t" (eval_str "(number? 3.0)"));
  ("number? str", `Quick, fun () ->
    Alcotest.(check string) "n?s" "#f" (eval_str "(number? \"3\")"));
  ("integer? int", `Quick, fun () ->
    Alcotest.(check string) "i?" "#t" (eval_str "(integer? 3)"));
  ("integer? float tag-based", `Quick, fun () ->
    Alcotest.(check string) "i?f" "#f" (eval_str "(integer? 3.0)"));
  ("zero?", `Quick, fun () ->
    Alcotest.(check string) "z" "#t" (eval_str "(zero? 0)"));
  ("positive?", `Quick, fun () ->
    Alcotest.(check string) "p" "#t" (eval_str "(positive? 1)"));
  ("negative?", `Quick, fun () ->
    Alcotest.(check string) "neg" "#t" (eval_str "(negative? -1)"));
  ("even?", `Quick, fun () ->
    Alcotest.(check string) "even" "#t" (eval_str "(even? 4)"));
  ("odd?", `Quick, fun () ->
    Alcotest.(check string) "odd" "#t" (eval_str "(odd? 3)"));
  ("even? non-integral float errors", `Quick, fun () ->
    match try ignore (eval "(even? 1.5)"); None with
          | Interp_error (User_error (_, _)) -> Some ()
          | _ -> None
    with
    | Some () -> ()
    | None -> Alcotest.fail "expected error");
  ("quotient", `Quick, fun () ->
    Alcotest.(check string) "q" "3" (eval_str "(quotient 10 3)"));
  ("remainder", `Quick, fun () ->
    Alcotest.(check string) "r" "1" (eval_str "(remainder 10 3)"));
  ("quotient negative dividend", `Quick, fun () ->
    (* Int64.div truncates toward zero *)
    Alcotest.(check string) "qneg" "-3" (eval_str "(quotient -10 3)"));
  ("remainder negative dividend", `Quick, fun () ->
    Alcotest.(check string) "rneg" "-1" (eval_str "(remainder -10 3)"));
  ("quotient by zero errors", `Quick, fun () ->
    match try ignore (eval "(quotient 1 0)"); None with
          | Interp_error (User_error (_, _)) -> Some ()
          | _ -> None
    with
    | Some () -> ()
    | None -> Alcotest.fail "expected error");
  ("quotient rejects float", `Quick, fun () ->
    match try ignore (eval "(quotient 1.0 2.0)"); None with
          | Interp_error (User_error (_, _)) -> Some ()
          | _ -> None
    with
    | Some () -> ()
    | None -> Alcotest.fail "expected error");
  ("div by zero errors", `Quick, fun () ->
    match try ignore (eval "(div 1 0)"); None with
          | Interp_error (User_error (_, _)) -> Some ()
          | _ -> None
    with
    | Some () -> ()
    | None -> Alcotest.fail "expected error");
  ("random n in range", `Quick, fun () ->
    ignore (eval "(random-seed 42)");
    let v = eval "(random 10)" in
    match v with
    | VInt n ->
        Alcotest.(check bool) "range" true (n >= 0L && n < 10L)
    | _ -> Alcotest.fail "expected int");
  ("random zero-arg float", `Quick, fun () ->
    ignore (eval "(random-seed 1)");
    let v = eval "(random)" in
    match v with
    | VFloat f ->
```

```
        Alcotest.(check bool) "[0,1)" true (f >= 0. && f < 1.)
    | _ -> Alcotest.fail "expected float");
  ("random-seed deterministic", `Quick, fun () ->
    ignore (eval "(random-seed 99)");
    let a = eval_str "(random 1000)" in
    ignore (eval "(random-seed 99)");
    let b = eval_str "(random 1000)" in
    Alcotest.(check string) "same" a b);
]

(* --- list/pair operations --- *)
let list_tests = [
  ("list variadic", `Quick, fun () ->
    Alcotest.(check string) "list" "(1 2 3)" (eval_str "(list 1 2 3)"));
  ("list zero-arg is nil", `Quick, fun () ->
    Alcotest.(check string) "empty list" "()" (eval_str "(list)"));
  ("car of list", `Quick, fun () ->
    Alcotest.(check string) "car head" "1" (eval_str "(car (list 1 2 3))"));
  ("cdr of list", `Quick, fun () ->
    Alcotest.(check string) "cdr tail" "(2 3)" (eval_str "(cdr (list 1 2
     3))"));
  ("car of pair", `Quick, fun () ->
    Alcotest.(check string) "car pair" "1" (eval_str "(car (cons 1 2))"));
  ("cdr of pair", `Quick, fun () ->
    Alcotest.(check string) "cdr pair" "2" (eval_str "(cdr (cons 1 2))"));
  ("cons onto list", `Quick, fun () ->
    Alcotest.(check string) "cons" "(0 1 2)" (eval_str "(cons 0 (list 1
     2))"));
  ("null? empty", `Quick, fun () ->
    Alcotest.(check string) "null?()" "#t" (eval_str "(null? ())"));
  ("null? on non-empty", `Quick, fun () ->
    Alcotest.(check string) "null? non-empty" "#f"
      (eval_str "(null? (list 1 2))"));
  ("pair? on pair", `Quick, fun () ->
    Alcotest.(check string) "pair? pair" "#t"
      (eval_str "(pair? (cons 1 2))"));
  ("pair? on empty list", `Quick, fun () ->
    Alcotest.(check string) "pair? empty" "#f"
      (eval_str "(pair? (list))"));
  ("count on list", `Quick, fun () ->
    Alcotest.(check string) "count" "3" (eval_str "(count (list 1 2 3))"));
  ("count on nil", `Quick, fun () ->
    Alcotest.(check string) "count nil" "0" (eval_str "(count ())"));
]

(* --- string / reflection --- *)
let string_tests = [
  ("string-append", `Quick, fun () ->
    Alcotest.(check string) "concat" "\"foobarbaz\""
      (eval_str "(string-append \"foo\" \"bar\" \"baz\")"));
  ("string-join", `Quick, fun () ->
    Alcotest.(check string) "basic" "\"foobarbaz\""
      (eval_str "(string-join (list \"foo\" \"bar\" \"baz\"))"));
  ("string-join sep", `Quick, fun () ->
    Alcotest.(check string) "csv" "\"a,b,c\""
      (eval_str "(string-join (list \"a\" \"b\" \"c\") \",\")"));
  ("string-join empty list", `Quick, fun () ->
    Alcotest.(check string) "empty" "\"\""
      (eval_str "(string-join ())"));
  ("string-join single", `Quick, fun () ->
    Alcotest.(check string) "one" "\"foo\""
      (eval_str "(string-join (list \"foo\"))"));
  ("string-join coerces ints", `Quick, fun () ->
    Alcotest.(check string) "coerce" "\"1,2,3\""
      (eval_str "(string-join (list 1 2 3) \",\")"));
  ("string-join cons list", `Quick, fun () ->
    Alcotest.(check string) "pair" "\"ab\""
      (eval_str "(string-join (cons \"a\" \"b\"))"));
  ("str of int", `Quick, fun () ->
    Alcotest.(check string) "str int" "\"42\""
      (eval_str "(str 42)"));
  ("str of bool", `Quick, fun () ->
    Alcotest.(check string) "str bool" "\"#t\""
```

```
      (eval_str "(str #t)"));
  ("type-of int", `Quick, fun () ->
    Alcotest.(check string) "type int" "\"int\""
      (eval_str "(type-of 5)"));
  ("type-of str", `Quick, fun () ->
    Alcotest.(check string) "type str" "\"str\""
      (eval_str "(type-of \"hello\")"));
  ("type-of list", `Quick, fun () ->
    Alcotest.(check string) "type list" "\"list\""
      (eval_str "(type-of (list 1 2))"));
  ("type-of bool", `Quick, fun () ->
    Alcotest.(check string) "type bool" "\"bool\""
      (eval_str "(type-of #f)"));
  ("string<? lexicographic", `Quick, fun () ->
    Alcotest.(check string) "lt" "#t"
      (eval_str "(string<? \"abc\" \"abd\")"));
  ("string<? false on equal", `Quick, fun () ->
    Alcotest.(check string) "eq" "#f"
      (eval_str "(string<? \"abc\" \"abc\")"));
  ("string<? orders ISO dates", `Quick, fun () ->
    Alcotest.(check string) "dates" "#t"
      (eval_str "(string<? \"2026-07-16\" \"2026-07-23\")"));
  ("string<? is lexicographic not numeric", `Quick, fun () ->
    Alcotest.(check string) "coerced ints compare as strings" "#t"
      (eval_str "(string<? 10 9)"));
  ("string<=? true on equal", `Quick, fun () ->
    Alcotest.(check string) "le eq" "#t"
      (eval_str "(string<=? \"abc\" \"abc\")"));
  ("string>? lexicographic", `Quick, fun () ->
    Alcotest.(check string) "gt" "#t"
      (eval_str "(string>? \"abd\" \"abc\")"));
  ("string>=? true on equal", `Quick, fun () ->
    Alcotest.(check string) "ge eq" "#t"
      (eval_str "(string>=? \"abc\" \"abc\")"));
  ("string-split basic", `Quick, fun () ->
    Alcotest.(check string) "split" "(\"a\" \"b\" \"c\")"
      (eval_str "(string-split \"a,b,c\" \",\")"));
  ("string-split keeps empty pieces", `Quick, fun () ->
    Alcotest.(check string) "empties" "(\"a\" \"\" \"b\")"
      (eval_str "(string-split \"a,,b\" \",\")"));
  ("string-split leading/trailing delims", `Quick, fun () ->
    Alcotest.(check string) "edges" "(\"\" \"a\" \"\")"
      (eval_str "(string-split \",a,\" \",\")"));
  ("string-split no match ? whole string", `Quick, fun () ->
    Alcotest.(check string) "nomatch" "(\"abc\")"
      (eval_str "(string-split \"abc\" \",\")"));
  ("string-split empty string ? one empty piece", `Quick, fun () ->
    Alcotest.(check string) "empty-s" "(\"\")"
      (eval_str "(string-split \"\" \",\")"));
  ("string-split multi-char delimiter", `Quick, fun () ->
    Alcotest.(check string) "multichar" "(\"one\" \"two\")"
      (eval_str "(string-split \"one--two\" \"--\")"));
  ("string-split whole-string delimiter", `Quick, fun () ->
    Alcotest.(check string) "whole" "(\"\" \"\")"
      (eval_str "(string-split \"abc\" \"abc\")"));
  ("string-split empty delimiter ? whole string", `Quick, fun () ->
    Alcotest.(check string) "empty delim" "(\"abc\")"
      (eval_str "(string-split \"abc\" \"\")"));
]

(* --- composition: define + call + nested arithmetic --- *)
(* habitat-book web-renderer: prose-tokens / render-tokens (kernel one-
   scan replacements for the interpreted per-word pipeline that blew the
   page budget). Shape test mirrors spec/token-binding? -- these cases
   pin that parity. *)
let prose_token_tests = [
  ("prose-tokens finds pkg/name cores", `Quick, fun () ->
    Alcotest.(check string) "one core" "(\"spec/book-page\")"
      (eval_str "(prose-tokens \"see spec/book-page here\")"));
  ("prose-tokens unique, first-occurrence order", `Quick, fun () ->
    Alcotest.(check string) "dedup" "(\"a/x\" \"b/y\")"
      (eval_str "(prose-tokens \"a/x b/y a/x\")"));
  ("prose-tokens strips one edge-punct layer", `Quick, fun () ->
```

```
    Alcotest.(check string) "edges" "(\"spec/todos\")"
      (eval_str "(prose-tokens \"(spec/todos: end).\")"));
  ("prose-tokens skips non-token-shaped words", `Quick, fun () ->
    Alcotest.(check string) "shape" "(\"yes/fine\")"
      (eval_str "(prose-tokens \"plain words -> / /nope yes/fine\")"));
  ("prose-tokens newlines separate words", `Quick, fun () ->
    Alcotest.(check string) "newline sep" "(\"a/bar\")"
      (eval_str "(prose-tokens \"foo\na/bar\")"));
  ("prose-tokens empty", `Quick, fun () ->
    Alcotest.(check string) "empty" "()"
      (eval_str "(prose-tokens \"\")"));
  ("render-tokens resolved core links", `Quick, fun () ->
    Alcotest.(check string) "link"
      "\"see <a href=\\\"/binding/spec/page\\\">spec/page</a> now\""
      (eval_str
        "(render-tokens \"see spec/page now\" (list \"spec/page\") ())"));
  ("render-tokens unresolved core is a hole", `Quick, fun () ->
    Alcotest.(check string) "hole"
      "\"(<span class=\\\"book-hole\\\">spec/ghost&#8709;</span>)\""
      (eval_str
        "(render-tokens \"(spec/ghost)\" () (list \"spec/ghost\"))"));
  ("render-tokens plain words verbatim + whitespace kept", `Quick, fun () ->
    Alcotest.(check string) "verbatim" "\"a  b\\nc\""
      (eval_str "(render-tokens \"a  b\\nc\" () ())"));
  ("render-tokens classifies word-by-word", `Quick, fun () ->
    Alcotest.(check string) "mixed"
      "\"<a href=\\\"/binding/a/x\\\">a/x</a> and <span
     class=\\\"book-hole\\\">b/y&#8709;</span> end\""
      (eval_str
        "(render-tokens \"a/x and b/y end\" (list \"a/x\") (list
     \"b/y\"))"));
]

let composition_tests = [
  ("let-bind persists through lambda", `Quick, fun () ->
    Alcotest.(check string) "let persists"
      "25"
      (eval_str "(let ((square (lambda (x) (* x x)))) (square 5))"));
  ("let + arithmetic", `Quick, fun () ->
    Alcotest.(check string) "let nested" "30"
      (eval_str "(let ((x 5) (y 6)) (* x y))"));
  ("if true branch", `Quick, fun () ->
    Alcotest.(check string) "if true" "\"yes\""
      (eval_str "(if #t \"yes\" \"no\")"));
  ("if false via nil", `Quick, fun () ->
    Alcotest.(check string) "if nil" "\"no\""
      (eval_str "(if () \"yes\" \"no\")"));
  ("nested arithmetic inside lambda body", `Quick, fun () ->
    Alcotest.(check string) "nested"
      "12"
      (eval_str "((lambda (a b) (+ (* a 2) (* b 3))) 3 2)"));
]

(* === IT13 PROBE: reader parses JSON-form strings with escaped quotes ===
   This is a real test for a real bug found while building CRUD primitives:
   the reader's escape handler was reading the '\\' twice (peeks '\\',
   then advances which returns '\\', then matches '\\' as the escape
   letter -- so `\"` added '\\' to buf and the following '"' closed the
   string). After the fix, `"\"name\":\"alice\""` should read back as
   the string `{"name":"alice"}`. *)
let json_string_probe () =
  let json_str = "{\"name\":\"alice\",\"age\":30}" in   (* the JSON data *)
  let lisp_src = "\"" ^ String.escaped json_str ^ "\"" in  (* lisp string
     literal *)
  let e = Pricklypear_lisp.Reader.read_one lisp_src in
  (match e with
   | Pricklypear_lisp.Ast.Lit (Pricklypear_lisp.Ast.VStr s) ->
       Alcotest.(check string)
         "reader parses JSON-form string (escapes work)"
         json_str s
   | _ ->
       Alcotest.failf "expected Lit VStr, got %s"
         (Pricklypear_lisp.Reader.pp_expr e))
```

```
429
430 let probe_tests = [
431   ("reader parses JSON string with \\\" escapes", `Quick,
        json_string_probe);
432 ]
433
434 (* Helper: read key back from JSON-string via Yojson. Used by the
435    dict-set test to verify the new key round-trips through a parse. *)
436 let dict_get_back (json : string) (key : string) : string option =
437   try
438     (match Yojson.Safe.from_string json with
439      | `Assoc kvs ->
440          (match List.assoc_opt key kvs with
441           | Some (`String s) -> Some s
442           | Some (`Int n) -> Some (string_of_int n)
443           | _ -> None)
444      | _ -> None)
445   with _ -> None
446
447 (* --- dict primitives (operate on VStr JSON; pure, no DB) --- *)
448 let dict_tests = [
449   ("dict-get string value", `Quick, fun () ->
450     Alcotest.(check string) "get name"
451       "\"alice\"" (* pp_value of VStr adds outer quotes *)
452       (eval_str "(dict-get \"{\\\"name\\\":\\\"alice\\\",\\\"age\\\":30}\"
        \"name\")"));
453   ("dict-get int value stringified", `Quick, fun () ->
454     Alcotest.(check string) "get age"
455       "\"30\"" (* pp_value of VStr adds outer quotes *)
456       (eval_str "(dict-get \"{\\\"name\\\":\\\"alice\\\",\\\"age\\\":30}\"
        \"age\")"));
457   ("dict-get missing key is nil", `Quick, fun () ->
458     Alcotest.(check string) "missing key"
459       "()"
460       (eval_str "(dict-get \"{\\\"a\\\":1}\" \"nope\")"));
461   ("dict-set adds new key", `Quick, fun () ->
462     (* The exact key order may vary; just check the chosen key is present
        and decodes back. *)
463     let v = eval "(dict-set \"{\\\"a\\\":1}\" \"b\" \"2\")" in
464     (match v with
465      | VStr s ->
466          let r = dict_get_back s "b" in
467          Alcotest.(check (option string)) "b present" (Some "2") r
468      | _ ->
469          Alcotest.failf "expected VStr, got %s"
        (Pricklypear_lisp.Reader.pp_value v)));
470   ("dict-keys lists 2 keys", `Quick, fun () ->
471     let v = eval "(dict-keys
       \"{\\\"name\\\":\\\"alice\\\",\\\"age\\\":30}\")" in
472     (match v with
473      | VList [ VStr _; VStr _ ] -> ()
474      | _ ->
475          Alcotest.failf "expected list of 2 keys, got %s"
476            (Pricklypear_lisp.Reader.pp_value v)));
477   ("api_eval native-JSON: CRUD VStr stays clean (no triple-escape)",
        `Quick, fun () ->
478     (* api_eval now serializes eval results with Prims_pure.value_to_json.
479        A CRUD read returns fields as a VStr of JSON text; value_to_json must
480        re-parse it into a real JSON object (quotes NOT triple-escaped),
481        matching the i-fields-as-string fix. See image/lib/http_pages.ml
        api_eval. *)
482     let fields = "{\"name\":\"alice\",\"age\":30}" in
483     let j = Prims_pure.value_to_json (VStr fields) in
484     (match j with
485      | `Assoc kvs ->
486          Alcotest.(check (option string)) "name" (Some "alice")
487            (match List.assoc_opt "name" kvs with
488             | Some (`String s) -> Some s
489             | _ -> None);
490          Alcotest.(check int) "age" 30
491            (match List.assoc_opt "age" kvs with
492             | Some (`Int n) -> n
493             | _ -> -1)
```

```
    | other ->
        Alcotest.failf "expected JSON object, got %s"
          (Yojson.Safe.to_string other)));
]

(* --- render-html primitive (tagged VStr) --- *)
let render_html_tests = [
  ("render-html tags the string", `Quick, fun () ->
    let v = eval "(render-html \"<b>hi</b>\")" in
    Alcotest.(check bool) "is raw html struck"
      true
      (Pricklypear_image_lib.Primitives.is_raw_html v <> None));
  ("render-html raw content extractable", `Quick, fun () ->
    let v = eval "(render-html \"<h2>title</h2>\")" in
    Alcotest.(check (option string)) "content"
      (Some "<h2>title</h2>")
      (Pricklypear_image_lib.Primitives.is_raw_html v));
  ("plain VStr not raw html", `Quick, fun () ->
    Alcotest.(check (option string)) "not raw"
      None
      (Pricklypear_image_lib.Primitives.is_raw_html (VStr "plain")));
]

(* --- extract-links (OCaml wiki scanner) --- *)
let extract_links_tests = [
  ("three wiki links", `Quick, fun () ->
    let v =
      eval
        "(extract-links \"see [[a]] and [[b|Bee]] plus [[c]]\")"
    in
    match v with
    | VList
        [ VList [ VStr "a"; VStr "a" ]
        ; VList [ VStr "b"; VStr "Bee" ]
        ; VList [ VStr "c"; VStr "c" ] ] -> ()
    | other ->
        Alcotest.failf "links got %s" (Pricklypear_lisp.Reader.pp_value
    other));
  ("zero links", `Quick, fun () ->
    Alcotest.(check string) "empty" "()"
      (eval_str "(extract-links \"no links here\")"));
  ("unclosed bracket no hang", `Quick, fun () ->
    Alcotest.(check string) "unclosed" "()"
      (eval_str "(extract-links \"text [[oops\")"));
  ("empty brackets skipped", `Quick, fun () ->
    Alcotest.(check string) "empty pair" "()"
      (eval_str "(extract-links \"[[]]\")"));
  ("media syntax included", `Quick, fun () ->
    let v =
      eval
        "(extract-links \"[[media:uuid-1|alt text]]\")"
    in
    match v with
    | VList [ VList [ VStr "media:uuid-1"; VStr "alt text" ] ] -> ()
    | other ->
        Alcotest.failf "media got %s" (Pricklypear_lisp.Reader.pp_value
    other));
  ("budget-remaining positive mid-eval", `Quick, fun () ->
    let v = eval "(budget-remaining)" in
    match v with
    | VInt n when n > 0L -> ()
    | other ->
        Alcotest.failf "budget-remaining got %s"
          (Pricklypear_lisp.Reader.pp_value other));
  ("render-links wiki + media", `Quick, fun () ->
    let contains hay needle =
      let n = String.length needle in
      let rec loop i =
        if i + n > String.length hay then false
        else if String.sub hay i n = needle then true
        else loop (i + 1)
      in
      loop 0
```

```
    in
    let v =
      eval
        "(render-links \"hi [[a|A]] [[media:uuid-1|alt]]\")"
    in
    match v with
    | VStr s ->
        if not (contains s "/notes?slug=a") then
          Alcotest.failf "missing wiki href: %s" s;
        if not (contains s "note-media") then
          Alcotest.failf "missing media class: %s" s;
        if not (contains s ">A</a>") then
          Alcotest.failf "missing label: %s" s
    | other ->
        Alcotest.failf "render-links got %s"
          (Pricklypear_lisp.Reader.pp_value other));
  ("render-links bible sexp", `Quick, fun () ->
    let contains hay needle =
      let n = String.length needle in
      let rec loop i =
        if i + n > String.length hay then false
        else if String.sub hay i n = needle then true
        else loop (i + 1)
      in
      loop 0
    in
    let v =
      eval
        "(render-links \"see (bible John 3 16) and [[x]]\")"
    in
    match v with
    | VStr s ->
        if not (contains s "/dabar?book=John") then
          Alcotest.failf "missing dabar href: %s" s;
        if not (contains s "chapter=3") then
          Alcotest.failf "missing chapter: %s" s;
        if not (contains s "verse=16") then
          Alcotest.failf "missing verse: %s" s;
        if not (contains s "/notes?slug=x") then
          Alcotest.failf "wiki still works: %s" s
    | other ->
        Alcotest.failf "render-links bible got %s"
          (Pricklypear_lisp.Reader.pp_value other));
]

let equality_tests = [
  ("equal? lists same", `Quick, fun () ->
    Alcotest.(check string) "eq" "#t"
      (eval_str "(equal? (list 1 2 3) (list 1 2 3))"));
  ("equal? lists differ", `Quick, fun () ->
    Alcotest.(check string) "ne" "#f"
      (eval_str "(equal? (list 1 2) (list 1 2 3))"));
  ("equal? strings", `Quick, fun () ->
    Alcotest.(check string) "str eq" "#t"
      (eval_str "(equal? \"a\" \"a\")"));
  ("equal? int vs float distinct", `Quick, fun () ->
    Alcotest.(check string) "int!=float" "#f"
      (eval_str "(equal? 1 1.0)"));
  ("equal? cons-chain equals list", `Quick, fun () ->
    Alcotest.(check string) "cons==list" "#t"
      (eval_str "(equal? (list 1 2 3) (cons 1 (cons 2 (cons 3 ()))))"));
  ("equal? dotted pair not equal to proper list", `Quick, fun () ->
    Alcotest.(check string) "dotted!=proper" "#f"
      (eval_str "(equal? (cons 1 (cons 2 3)) (list 1 2 3))"));
  ("equal? symbols", `Quick, fun () ->
    Alcotest.(check string) "sym eq" "#t"
      (eval_str "(equal? 'a 'a)"));
  ("equal? nil and empty list", `Quick, fun () ->
    Alcotest.(check string) "nil==empty" "#t"
      (eval_str "(equal? () (list))"));
]

let cadr_tests = [
```

```
  ("cadr", `Quick, fun () ->
    Alcotest.(check string) "cadr" "2"
      (eval_str "(cadr (list 1 2 3))"));
  ("caddr", `Quick, fun () ->
    Alcotest.(check string) "caddr" "3"
      (eval_str "(caddr (list 1 2 3))"));
  ("caar", `Quick, fun () ->
    Alcotest.(check string) "caar" "1"
      (eval_str "(caar (list (list 1 2) 3))"));
  ("cddr", `Quick, fun () ->
    Alcotest.(check string) "cddr" "(3)"
      (eval_str "(cddr (list 1 2 3))"));
]

let assoc_member_tests = [
  ("assoc hit", `Quick, fun () ->
    Alcotest.(check string) "assoc hit" "(b 2)"
      (eval_str "(assoc 'b (list (list 'a 1) (list 'b 2)))"));
  ("assoc miss is nil", `Quick, fun () ->
    Alcotest.(check string) "assoc miss" "()"
      (eval_str "(assoc 'z (list (list 'a 1)))"));
  ("assoc on pair entries", `Quick, fun () ->
    Alcotest.(check string) "assoc pair" "(c . 3)"
      (eval_str "(assoc 'c (list (cons 'a 1) (cons 'c 3)))"));
  ("member hit returns tail", `Quick, fun () ->
    Alcotest.(check string) "member hit" "(2 3)"
      (eval_str "(member 2 (list 1 2 3))"));
  ("member miss is #f", `Quick, fun () ->
    Alcotest.(check string) "member miss" "#f"
      (eval_str "(member 9 (list 1 2 3))"));
]

let apply_tests = [
  ("apply prim with list", `Quick, fun () ->
    Alcotest.(check string) "apply +" "3"
      (eval_str "(apply + (list 1 2))"));
  ("apply prim with quoted list", `Quick, fun () ->
    Alcotest.(check string) "apply + quote" "10"
      (eval_str "(apply + '(2 8))"));
  ("apply lambda", `Quick, fun () ->
    Alcotest.(check string) "apply lambda" "12"
      (eval_str "(apply (lambda (a b) (* a b)) (list 3 4))"));
]

let std_freeze_tests = [
  ("length on list", `Quick, fun () ->
    Alcotest.(check string) "length" "3"
      (eval_str "(length (list 1 2 3))"));
  ("length on nil", `Quick, fun () ->
    Alcotest.(check string) "length nil" "0"
      (eval_str "(length ())"));
  ("length on cons chain", `Quick, fun () ->
    Alcotest.(check string) "length cons" "3"
      (eval_str "(length (cons 1 (cons 2 (cons 3 ()))))"));
  ("count still works", `Quick, fun () ->
    Alcotest.(check string) "count" "2"
      (eval_str "(count (list 1 2))"));
  ("list/foldr right fold", `Quick, fun () ->
    (* foldr cons 0 (1 2 3) = (1 (2 (3 0))) -- nested cons/list *)
    Alcotest.(check string) "foldr nest" "(1 (2 (3 0)))"
      (eval_str "(list/foldr (lambda (x acc) (cons x (list acc))) 0 (list 1
     2 3))"));
  ("list/foldr sum right", `Quick, fun () ->
    (* foldr + 0 (1 2 3 4) = 10 *)
    Alcotest.(check string) "foldr sum" "10"
      (eval_str "(list/foldr + 0 (list 1 2 3 4))"));
  ("list/foldr on empty is init", `Quick, fun () ->
    Alcotest.(check string) "foldr empty" "42"
      (eval_str "(list/foldr + 42 (list))"));
  ("list/foldl still works", `Quick, fun () ->
    Alcotest.(check string) "foldl sum" "10"
      (eval_str "(list/foldl + 0 (list 1 2 3 4))"));
]
```

```
710
711 (* --- kernel I/O primitives (from notes inbox kernel-io-primitives) --- *)
712 (* pp_value wraps VStr in outer quotes; q builds the expected form so we
713    avoid nested-quote escaping pain in the test source. *)
714 let q s = "\"" ^ s ^ "\""
715 let io_prim_tests = [
716   ("string-replace", `Quick, fun () ->
717     Alcotest.(check string) "replace" (q "xxx")
718       (eval_str "(string-replace \"axa\" \"a\" \"x\")"));
719   ("string-replace empty pat no-op", `Quick, fun () ->
720     Alcotest.(check string) "empty pat" (q "abc")
721       (eval_str "(string-replace \"abc\" \"\" \"X\")"));
722   ("string-downcase", `Quick, fun () ->
723     Alcotest.(check string) "down" (q "abc")
724       (eval_str "(string-downcase \"ABC\")"));
725   ("string-upcase", `Quick, fun () ->
726     Alcotest.(check string) "up" (q "ABC")
727       (eval_str "(string-upcase \"abc\")"));
728   ("string-starts-with?", `Quick, fun () ->
729     Alcotest.(check string) "starts" "#t"
730       (eval_str "(string-starts-with? \"hello world\" \"hello\")"));
731   ("string-ends-with?", `Quick, fun () ->
732     Alcotest.(check string) "ends" "#t"
733       (eval_str "(string-ends-with? \"hello.txt\" \".txt\")"));
734   ("url-encode", `Quick, fun () ->
735     Alcotest.(check string) "enc" (q "a%20b%2Cc")
736       (eval_str "(url-encode \"a b,c\")"));
737   ("url-decode", `Quick, fun () ->
738     Alcotest.(check string) "dec" (q "a b,c")
739       (eval_str "(url-decode \"a%20b%2Cc\")"));
740   ("url roundtrip", `Quick, fun () ->
741     Alcotest.(check string) "roundtrip" (q "hello & world?")
742       (eval_str "(url-decode (url-encode \"hello & world?\"))"));
743   ("sha1", `Quick, fun () ->
744     Alcotest.(check string) "sha1 abc"
745       (q "a9993e364706816aba3e25717850c26c9cd0d89d")
746       (eval_str "(sha1 \"abc\")"));
747   ("html-decode named", `Quick, fun () ->
748     Alcotest.(check string) "decode named" (q "Hi & bye")
749       (eval_str "(html-decode \"Hi &amp; bye\")"));
750   ("html-decode numeric", `Quick, fun () ->
751     let v = eval "(html-decode \"&#169;\")" in
752     (match v with
753      | Pricklypear_lisp.Ast.VStr s when String.length s = 2
754           && Char.code s.[0] = 0xC2 && Char.code s.[1] = 0xA9 -> ()
755      | other ->
756          Alcotest.failf "decode num got %s"
       (Pricklypear_lisp.Reader.pp_value other)));
757   ("html-decode leaves unknown", `Quick, fun () ->
758     Alcotest.(check string) "decode unknown" (q "&nope;")
759       (eval_str "(html-decode \"&nope;\")"));
760   ("parse-date ISO", `Quick, fun () ->
761     let v = eval "(parse-date \"2026-07-18T07:57:43Z\")" in
762     (match v with
763      | VInt n when n > 0L -> ()
764      | other ->
765          Alcotest.failf "parse-date ISO got %s"
       (Pricklypear_lisp.Reader.pp_value other)));
766   ("parse-date RFC-822", `Quick, fun () ->
767     let v = eval "(parse-date \"Tue, 28 Jul 2026 02:56:21 GMT\")" in
768     (match v with
769      | VInt n when n > 0L -> ()
770      | other ->
771          Alcotest.failf "parse-date RFC822 got %s"
       (Pricklypear_lisp.Reader.pp_value other)));
772   ("parse-date bare", `Quick, fun () ->
773     let v = eval "(parse-date \"2026-07-18\")" in
774     (match v with
775      | VInt n when n > 0L -> ()
776      | other ->
777          Alcotest.failf "parse-date bare got %s"
       (Pricklypear_lisp.Reader.pp_value other)));
778   ("parse-date garbage -> 0", `Quick, fun () ->
```

```
    Alcotest.(check string) "garbage" "0"
      (eval_str "(parse-date \"not a date\")"));
  ("xml-parse simple", `Quick, fun () ->
    let v = eval "(xml-parse \"<rss><channel><item><title>Hi &amp;
     bye</title></item></channel></rss>\")" in
    (match v with
     | VList [ VStr "rss"; VList []; body ] ->
         (match body with
          | VList [ VStr "channel"; _; item ] ->
              (match item with
               | VList [ VStr "item"; _; title ] ->
                   (match title with
                    | VList [ VStr "title"; _; VStr "Hi & bye" ] -> ()
                    | _ -> Alcotest.fail "title body")
               | _ -> Alcotest.fail "item")
          | _ -> Alcotest.fail "channel")
     | other ->
         Alcotest.failf "xml got %s" (Pricklypear_lisp.Reader.pp_value
    other)));
  ("xml-parse attrs", `Quick, fun () ->
    let v = eval "(xml-parse \"<a href=\\\"x\\\">t</a>\")" in
    (match v with
     | VList [ VStr "a"; VList [ VPair (VStr "href", VStr "x") ]; VStr "t"
    ] -> ()
     | other ->
         Alcotest.failf "xml attrs got %s"
    (Pricklypear_lisp.Reader.pp_value other)));
  ("now returns positive epoch", `Quick, fun () ->
    let v = eval "(now)" in
    (match v with
     | VInt n when n > 1_700_000_000L -> ()
     | other ->
         Alcotest.failf "now got %s" (Pricklypear_lisp.Reader.pp_value
    other)));
  ("now-ms larger than now", `Quick, fun () ->
    let a = eval "(now)" in
    let b = eval "(now-ms)" in
    (match a, b with
     | VInt s, VInt ms when ms >= Int64.mul s 1000L -> ()
     | other ->
         Alcotest.failf "now-ms relation bad"));
  ("xml-parse error on garbage", `Quick, fun () ->

    (* Lenient parser may recover (VStr text) or return a tree/error VList;
       just check it returns something without raising. *)
    let _v = eval "(xml-parse \"<<<\")" in
    ());
]

let multipart_tests = [
  ("part builds tagged list", `Quick, fun () ->
    let v =
      try eval "(part \"<b>hi</b>\" :target \"#t\" :swap \"beforeend\" :id
    \"1\")"
      with Pricklypear_lisp.Ast.Interp_error e ->
        Alcotest.failf "interp: %s" (Pricklypear_lisp.Ast.pp_interp_error e)
         | exn -> Alcotest.failf "exn: %s" (Printexc.to_string exn)
    in
    match Pricklypear_image_lib.Primitives.is_part v with
    | Some p ->
        Alcotest.(check string) "body" "<b>hi</b>" p.body;
        Alcotest.(check (option string)) "target" (Some "#t") p.target;
        Alcotest.(check (option string)) "swap" (Some "beforeend") p.swap;
        Alcotest.(check (option string)) "id" (Some "1") p.id
    | None -> Alcotest.failf "not a part: %s"
    (Pricklypear_lisp.Reader.pp_value v));
  ("multipart bundles parts", `Quick, fun () ->
    let v =
      eval
        "(multipart :target \"#feed\" :swap \"beforeend\"
            (part \"<b>one</b>\") (part \"<i>two</i>\" :id \"2\"))"
    in
    match Pricklypear_image_lib.Primitives.is_multipart v with
```

```
    | Some mp ->
        Alcotest.(check (option string)) "env target" (Some "#feed")
          mp.envelope_target;
        Alcotest.(check int) "n parts" 2 (List.length mp.parts);
        (match mp.parts with
         | [a; b] ->
             Alcotest.(check string) "a" "<b>one</b>" a.body;
             Alcotest.(check string) "b" "<i>two</i>" b.body;
             Alcotest.(check (option string)) "b id" (Some "2") b.id
         | _ -> Alcotest.fail "part count")
    | None -> Alcotest.failf "not multipart: %s"
     (Pricklypear_lisp.Reader.pp_value v));
  ("json-part content-type", `Quick, fun () ->
    let v = eval "(json-part \"{\\\"ok\\\":true}\" :id \"j\")" in
    match Pricklypear_image_lib.Primitives.is_part v with
    | Some p ->
        Alcotest.(check bool) "json ct" true
          (String.starts_with ~prefix:"application/json" p.content_type);
        Alcotest.(check (option string)) "id" (Some "j") p.id
    | None -> Alcotest.fail "not part");
  ("keyword :target self-evaluates in image", `Quick, fun () ->
    Alcotest.(check string) "kw" ":target" (eval_str ":target"));
  ("multipart accepts list of parts", `Quick, fun () ->
    let v =
      eval
        "(multipart :target \"#t\" (list (part \"a\") (part \"b\")))"
    in
    match Pricklypear_image_lib.Primitives.is_multipart v with
    | Some mp -> Alcotest.(check int) "2" 2 (List.length mp.parts)
    | None -> Alcotest.fail "not multipart");
]

let err_shape_tests = [
  ("err/shape builds (error msg kind) envelope", `Quick, fun () ->
    let v = eval "(err/shape \"boom\" \"bad-args\")" in
    match v with
    | VList [ VSymbol "error"; VStr msg; VSymbol kind ] ->
        Alcotest.(check string) "msg" "boom" msg;
        Alcotest.(check string) "kind" "bad-args" kind
    | other ->
        Alcotest.failf "expected (error boom bad-args), got %s"
          (Pricklypear_lisp.Reader.pp_value other));
  ("err/shape accepts symbols", `Quick, fun () ->
    let v = eval "(err/shape 'boom 'denied)" in
    match v with
    | VList [ VSymbol "error"; VStr msg; VSymbol kind ] ->
        Alcotest.(check string) "msg" "boom" msg;
        Alcotest.(check string) "kind" "denied" kind
    | other ->
        Alcotest.failf "expected (error boom denied), got %s"
          (Pricklypear_lisp.Reader.pp_value other));
  ("err/shape bad arity raises", `Quick, fun () ->
    (* single-arg call should raise User_error with bad-args kind *)
    try
      let _ = eval "(err/shape \"boom\")" in
      Alcotest.fail "expected err/shape to raise on bad arity"
    with
    | Pricklypear_lisp.Ast.Interp_error
        (Pricklypear_lisp.Ast.User_error (_, k)) ->
        Alcotest.(check string) "kind" "bad-args" k
    | exn ->
        Alcotest.failf "expected User_error, got %s" (Printexc.to_string
     exn));
]

let () =
  Alcotest.run "pricklypear-image-primitives" [
    "arithmetic", arithmetic_tests;
    "math", math_tests;
    "list", list_tests;
    "string", string_tests;
    "probe", probe_tests;
    "prose-tokens", prose_token_tests;
```

```
    "dict", dict_tests;
    "render-html", render_html_tests;
    "composition", composition_tests;
    "extract-links", extract_links_tests;
    "equality", equality_tests;
    "cadr", cadr_tests;
    "assoc-member", assoc_member_tests;
    "apply", apply_tests;
    "std-freeze", std_freeze_tests;
    "io-prims", io_prim_tests;
    "multipart", multipart_tests;
    "err-shape", err_shape_tests;
  ]
```

136 image/test/prims_hyper_test.ml

```
(** Pricklypear Image -- hypermedia harness kernel prims tests (no DB, no
    session).

    Tests the PURE soup/* lambdasoup bindings by evaluating Nopales forms
    against a prim_table with register_pure_prims. Mirrors primitives_test's
    pattern: build the table directly, eval via eval_string_direct.

    Spec: borge/hypermedia-harness.borg (kernel-prims verify stanza).
    No route/dispatch here (that needs a session -- verified via pp_eval +
    the fidelity smoke). *)
open Pricklypear_lisp.Ast

(* Build a prim_table + an env with all prims auto-defined as VPrim. *)
let make_test_env () =
  let prim_table = Pricklypear_image_lib.Primitives.make_with_defaults () in
  Pricklypear_image_lib.Prims_hyper.register_pure_prims prim_table;
  let env = Pricklypear_lisp.Env.create () in
  Hashtbl.iter
    (fun name _ ->
      Pricklypear_lisp.Env.define env name (VPrim name))
    prim_table;
  (env, prim_table)

let eval s =
  let env, prims = make_test_env () in
  Pricklypear_image_lib.Control_rim.run ~prims (fun () ->
      Pricklypear_lisp.Eval.eval_string_direct
        ~prims:Pricklypear_lisp.Eval.empty_prims env s)

let eval_str s =
  Pricklypear_lisp.Reader.pp_value (eval s)

(* --- parse + serialize round-trip --- *)
let parse_tests = [
  ("parse + serialize round-trips a node", `Quick, fun () ->
    Alcotest.(check string) "roundtrip"
      "\"<div><p>hi</p></div>\""
      (eval_str "(soup/serialize (soup/parse \"<div><p>hi</p></div>\"))"));
  ("parse a full page", `Quick, fun () ->
    let r = eval_str "(soup/serialize (soup/parse
     \"<html><body><h1>Title</h1></body></html>\"))" in
    Alcotest.(check bool) "contains h1" true (String.length r > 0));
]

(* --- select --- *)
let select_tests = [
  ("select finds all matching elements", `Quick, fun () ->
    Alcotest.(check string) "3 p tags"
      "3"
      (eval_str "(length (soup/select (soup/parse
     \"<div><p>1</p><p>2</p><p>3</p></div>\") \"p\"))"));
  ("select by class", `Quick, fun () ->
    Alcotest.(check string) "1 .x"
      "1"
      (eval_str "(length (soup/select (soup/parse \"<div><p
    class=x>1</p><p>2</p></div>\") \"p.x\"))"));
  ("select by id", `Quick, fun () ->
    Alcotest.(check string) "1 #t"
      "1"
      (eval_str "(length (soup/select (soup/parse \"<div id=t>x</div>\")
     \"#t\"))"));
  ("select finds nested", `Quick, fun () ->
    Alcotest.(check string) "descendants"
      "3"
      (eval_str "(length (soup/select (soup/parse
     \"<div><a>x</a><span><a>y</a></span><a>z</a></div>\") \"a\"))"));
]

(* --- text + attr --- *)
let text_attr_tests = [
  ("text reads text content", `Quick, fun () ->
```

```
    Alcotest.(check string) "text"
      "\"hello\""
      (eval_str "(soup/text (car (soup/select (soup/parse \"<p
     class=x>hello</p>\") \"p.x\")))"));
  ("attr reads an attribute", `Quick, fun () ->
    Alcotest.(check string) "href"
      "\"/x\""
      (eval_str "(soup/attr (car (soup/select (soup/parse \"<a
     href=/x>l</a>\") \"a\")) \"href\")"));
  ("attr returns nil for missing", `Quick, fun () ->
    Alcotest.(check string) "missing"
      "()"
      (eval_str "(soup/attr (car (soup/select (soup/parse \"<a
     href=/x>l</a>\") \"a\")) \"class\")"));
]

(* --- swap strategies (the 8 htmx swap values) --- *)
let swap_tests = [
  ("innerHTML replaces children", `Quick, fun () ->
    Alcotest.(check string) "innerHTML"
      "\"<div id=\\\"t\\\"><b>new</b></div>\""
      (eval_str "(begin (define p (soup/parse \"<div id=t>old</div>\"))
     (soup/swap! (car (soup/select p \"#t\")) \"innerHTML\" \"<b>new</b>\")
     (soup/serialize p))"));
  ("outerHTML replaces the element", `Quick, fun () ->
    Alcotest.(check string) "outerHTML"
      "\"<div><b>y</b></div>\""
      (eval_str "(begin (define p (soup/parse \"<div><span
     id=o>x</span></div>\")) (soup/swap! (car (soup/select p \"#o\"))
     \"outerHTML\" \"<b>y</b>\") (soup/serialize p))"));
  ("beforeend appends a child", `Quick, fun () ->
    Alcotest.(check string) "beforeend"
      "\"<ul id=\\\"l\\\"><li>a</li><li>b</li></ul>\""
      (eval_str "(begin (define p (soup/parse \"<ul
     id=l><li>a</li></ul>\")) (soup/swap! (car (soup/select p \"#l\"))
     \"beforeend\" \"<li>b</li>\") (soup/serialize p))"));
  ("afterbegin prepends a child in order", `Quick, fun () ->
    Alcotest.(check string) "afterbegin"
      "\"<ul id=\\\"l\\\"><li>new1</li><li>old</li></ul>\""
      (eval_str "(begin (define p (soup/parse \"<ul
     id=l><li>old</li></ul>\")) (soup/swap! (car (soup/select p \"#l\"))
     \"afterbegin\" \"<li>new1</li>\") (soup/serialize p))"));
  ("beforebegin inserts sibling before (parented)", `Quick, fun () ->
    Alcotest.(check string) "beforebegin"
      "\"<body><p>before</p><div id=\\\"t\\\">x</div></body>\""
      (eval_str "(begin (define p (soup/parse \"<body><div
     id=t>x</div></body>\")) (soup/swap! (car (soup/select p \"#t\"))
     \"beforebegin\" \"<p>before</p>\") (soup/serialize p))"));
  ("afterend inserts sibling after (parented)", `Quick, fun () ->
    Alcotest.(check string) "afterend"
      "\"<body><div id=\\\"t\\\">x</div><p>after</p></body>\""
      (eval_str "(begin (define p (soup/parse \"<body><div
     id=t>x</div></body>\")) (soup/swap! (car (soup/select p \"#t\"))
     \"afterend\" \"<p>after</p>\") (soup/serialize p))"));
  ("delete removes the element", `Quick, fun () ->
    Alcotest.(check string) "delete"
      "\"<div>keep</div>\""
      (eval_str "(begin (define p (soup/parse \"<div><span
     id=o>x</span>keep</div>\")) (soup/swap! (car (soup/select p \"#o\"))
     \"delete\" \"\") (soup/serialize p))"));
  ("none is a no-op", `Quick, fun () ->
    Alcotest.(check string) "none"
      "\"<div id=\\\"t\\\">x</div>\""
      (eval_str "(begin (define p (soup/parse \"<div id=t>x</div>\"))
     (soup/swap! (car (soup/select p \"#t\")) \"none\" \"<b>ignored</b>\")
     (soup/serialize p))"));
]

(* --- page-copy + immutability --- *)
let immutability_tests = [
  ("page-copy deep-copies so source is never mutated", `Quick, fun () ->
    Alcotest.(check string) "source unchanged"
      "\"<div id=\\\"t\\\">x</div>\""
```

```
      (eval_str "(begin (define p (soup/parse \"<div id=t>x</div>\"))
     (define p2 (soup/page-copy p)) (soup/swap! (car (soup/select p2
     \"#t\")) \"innerHTML\" \"changed\") (soup/serialize p))"));
  ("page-copy copy IS mutated", `Quick, fun () ->
    Alcotest.(check string) "copy changed"
      "\"<div id=\\\"t\\\">changed</div>\""
      (eval_str "(begin (define p (soup/parse \"<div id=t>x</div>\"))
     (define p2 (soup/page-copy p)) (soup/swap! (car (soup/select p2
     \"#t\")) \"innerHTML\" \"changed\") (soup/serialize p2))"));
]

(* --- closest-attr (inherited attribute ancestor walk) --- *)
let closest_attr_tests = [
  ("closest-attr finds on element itself", `Quick, fun () ->
    Alcotest.(check string) "self"
      "\"/post\""
      (eval_str "(begin (define p (soup/parse \"<button
     hx-post=/post>go</button>\")) (soup/closest-attr (car (soup/select p
     \"button\")) \"hx-post\"))"));
  ("closest-attr walks to ancestor", `Quick, fun () ->
    Alcotest.(check string) "ancestor"
      "\"#out\""
      (eval_str "(begin (define p (soup/parse \"<div hx-target=#out><button
     id=b>go</button></div>\")) (soup/closest-attr (car (soup/select p
     \"#b\")) \"hx-target\"))"));
  ("closest-attr returns nil when not found", `Quick, fun () ->
    Alcotest.(check string) "missing"
      "()"
      (eval_str "(begin (define p (soup/parse \"<div><button
     id=b>go</button></div>\")) (soup/closest-attr (car (soup/select p
     \"#b\")) \"hx-target\"))"));
]

(* --- diff --- *)
let diff_tests = [
  ("diff identical pages = no changes", `Quick, fun () ->
    Alcotest.(check string) "same"
      "\"no changes\""
      (eval_str "(begin (define a (soup/parse \"<div id=x>1</div>\"))
     (define b (soup/parse \"<div id=x>1</div>\")) (soup/diff a b))"));
  ("diff detects added element", `Quick, fun () ->
    let r = eval_str "(begin (define a (soup/parse \"<div id=x>1</div>\"))
     (define b (soup/parse \"<div id=x>1</div><div id=y>2</div>\"))
     (soup/diff a b))" in
    Alcotest.(check bool) "mentions y" true (String.length r > 0 &&
     String.contains r 'y'));
  ("diff detects removed element", `Quick, fun () ->
    let r = eval_str "(begin (define a (soup/parse \"<div id=x>1</div><div
     id=y>2</div>\")) (define b (soup/parse \"<div id=x>1</div>\"))
     (soup/diff a b))" in
    Alcotest.(check bool) "mentions y" true (String.length r > 0 &&
     String.contains r 'y'));
  ("diff detects text change in same-id element", `Quick, fun () ->
    let r = eval_str "(begin (define a (soup/parse \"<div
     id=x>old</div>\")) (define b (soup/parse \"<div id=x>new</div>\"))
     (soup/diff a b))" in
    Alcotest.(check bool) "reports change" true (String.length r > 0));
]

(* --- apply-parts! (multipart ordered part list) --- *)
let apply_parts_tests = [
  ("apply-parts! lands each frag at its target in order", `Quick, fun () ->
    Alcotest.(check string) "both targets swapped"
      "\"<div id=\\\"a\\\"><b>new-a</b></div><div
     id=\\\"b\\\"><i>new-b</i></div>\""
      (eval_str "(begin \
        \n (define p (soup/parse \"<div id=a>old-a</div><div
     id=b>old-b</div>\")) \
        \n (define parts (list \
        \n   (list (cons \"html\" \"<b>new-a</b>\") (cons \"target\"
     \"#a\") (cons \"swap\" \"innerHTML\")) \
        \n   (list (cons \"html\" \"<i>new-b</i>\") (cons \"target\"
     \"#b\") (cons \"swap\" \"innerHTML\")))) \
```

```
        \n (soup/apply-parts! p parts) \
        \n (soup/serialize p))"));
  ("apply-parts! defaults to innerHTML when swap absent", `Quick, fun () ->
    Alcotest.(check string) "default innerHTML"
      "\"<div id=\\\"a\\\">new-a</div>\""
      (eval_str "(begin \
        \n (define p (soup/parse \"<div id=a>old-a</div>\")) \
        \n (define parts (list (list (cons \"html\" \"new-a\") (cons
   \"target\" \"#a\")))) \
        \n (soup/apply-parts! p parts) \
        \n (soup/serialize p))"));
  ("apply-parts! respects beforeend strategy in sequence", `Quick, fun () ->
    Alcotest.(check string) "appended in order"
      "\"<ul id=\\\"l\\\"><li>1</li><li>2</li></ul>\""
      (eval_str "(begin \
        \n (define p (soup/parse \"<ul id=l></ul>\")) \
        \n (define parts (list \
        \n   (list (cons \"html\" \"<li>1</li>\") (cons \"target\" \"#l\")
   (cons \"swap\" \"beforeend\")) \
        \n   (list (cons \"html\" \"<li>2</li>\") (cons \"target\" \"#l\")
   (cons \"swap\" \"beforeend\")))) \
        \n (soup/apply-parts! p parts) \
        \n (soup/serialize p))"));
  ("apply-parts! skips a part whose target is absent", `Quick, fun () ->
    Alcotest.(check string) "missing target skipped"
      "\"<div id=\\\"a\\\">new-a</div>\""
      (eval_str "(begin \
        \n (define p (soup/parse \"<div id=a>old-a</div>\")) \
        \n (define parts (list \
        \n   (list (cons \"html\" \"new-a\") (cons \"target\" \"#a\")) \
        \n   (list (cons \"html\" \"ghost\") (cons \"target\" \"#nope\"))))
   \
        \n (soup/apply-parts! p parts) \
        \n (soup/serialize p))"));
]

let () =
  Alcotest.run "prims_hyper" [
    ("parse + serialize", parse_tests);
    ("select", select_tests);
    ("text + attr", text_attr_tests);
    ("swap strategies", swap_tests);
    ("immutability", immutability_tests);
    ("closest-attr", closest_attr_tests);
    ("diff", diff_tests);
    ("apply-parts!", apply_parts_tests);
  ]
```

137 image/test/reads_as_of_gen_test.ml

```
(** tree-time reads-as-of-gen prim surface tests
    (tree-time.reads-as-of-gen).

    Exercises the USER-FACING gen coordinate built on the versioned read
    foundation (versioned_head/versioned_history + the `version` row type,
    landed in the 5.1 close-out):

      - tree/get <path> <gen>  : optional :gen N resolves the node at that
        gen (versioned), omitted = head (live tree_nodes).
      - gen-read <path> <gen>  : node body at gen as version JSON; supports
        absolute b.*/data.* paths AND relative logical suffixes crossed
        through the branch lens chain (closest-wins).
      - gen-diff <path> ga gb  : node-granular diff -- hashes first, walk on
        mismatch; returns "same" when both leaves hash-identical.
      - undo <path> [n]        : restore the value at head-n as a NEW write
    at
        head+1 (restore-by-write) for datum paths; the restore is itself a
        versioned append, so undo is lossless + redoable.

    Needs Postgres (scripts/dev.sh). Skips cleanly when unreachable.
*)

module S = Pricklypear_image_lib.Store
module T = Pricklypear_image_lib.Tree_store
module C = Pricklypear_image_lib.Crud
open Pricklypear_lisp.Ast

let main_id = "00000000-0000-0000-0000-000000000001"

let with_store f =
  Eio_main.run @@ fun env ->
  Lwt_eio.with_event_loop ~clock:(Eio.Stdenv.clock env) @@ fun _ ->
  Eio.Switch.run @@ fun sw ->
  let ctx : Pricklypear_image_lib.Io_eio.Ctx.t =
    {
      sw;
      net = (Eio.Stdenv.net env :> [ `Generic ] Eio.Net.ty Eio.Std.r);
    }
  in
  Pricklypear_image_lib.Io_eio.with_ctx ctx (fun () ->
      let cfg = S.config_from_env () in
      let store = S.make ~cfg ~branch_id:main_id () in
      S.connect store;
      match S.conn_of store with
      | None ->
          Printf.printf
            "SKIP: no postgres (set PP_DB_* / run scripts/dev.sh)\n%!"
      | Some _ ->
          S.with_branch store ~branch_id:main_id (fun () -> f store))

let uniq p = Printf.sprintf "rg_%s_%d" p (Random.int 1_000_000)

let declare_type (store : S.t) ~name ~fields =
  S.persist_type store ~name ~fields ~author:"tester"

let path_id (path : string) : string =
  match List.rev (String.split_on_char '.' path) with
  | l :: _ -> l
  | [] -> path

(** The last write's gen for a datum path. *)
let head_gen (store : S.t) (path : string) : int64 =
  match S.conn_of store with
  | Some conn ->
      (match T.versioned_head conn ~path () with
       | Some v -> (match v.T.v_gen with Some g -> g | None -> 0L)
       | None -> 0L)
  | None -> 0L

(** Create a datum row and return its tree path. *)
let make_datum (store : S.t) ~type_name ~title : string =
```

```
  match S.conn_of store with
  | Some conn ->
      (match
         T.upsert_datum conn ~path:"data.u.tester.new" ~type_name
           ~body_json:
             (Printf.sprintf "{\"title\":\"%s\",\"body\":\"b\"}" title)
           ~author:"tester" ~rank:None
       with
       | Ok p -> p
       | Error e -> Alcotest.fail ("create failed: " ^ e))
  | None -> Alcotest.fail "no conn"

(** Extract a string field from gen-read JSON (the version_to_json surface
    has {"body":{...}} and {"gen":...}). *)
let version_json_title (store : S.t) (path : string) (gen : int64)
    : string option =
  match C.tree_gen_read store ~path ~gen with
  | VStr s ->
      (try
         (match Yojson.Safe.from_string s with
          | `Assoc kv ->
              (match List.assoc_opt "body" kv with
               | Some (`Assoc bkv) ->
                   (match List.assoc_opt "title" bkv with
                    | Some (`String t) -> Some t
                    | _ -> None)
               | _ -> None)
          | _ -> None)
       with _ -> None)
  | _ -> None

(** gen-read on a RELATIVE logical suffix resolves through the branch lens
    chain when the store's current branch differs from main and main has the
    datum-like fn node; that path is exercised separately (fn code-path
    reads) -- here we just assert the absolute-path surface. *)

(* ---- tree/get with optional :gen -> versioned read; omitted = head ----
    *)

let body_title (kv : (string * Yojson.Safe.t) list) : string option =
  match List.assoc_opt "body" kv with
  | Some (`Assoc bkv) ->
      (match List.assoc_opt "title" bkv with
       | Some (`String t) -> Some t
       | _ -> None)
  | _ -> None

let test_tree_get_at_gen () =
  with_store (fun store ->
      let type_name = uniq "t" in
      declare_type store ~name:type_name ~fields:[ "title"; "body" ];
      let path = make_datum store ~type_name ~title:"a" in
      let g0 = head_gen store path in
      (* head, no :gen -> live body title "a" *)
      (match C.tree_get store ~path () with
       | VStr s ->
           (match Yojson.Safe.from_string s with
            | `Assoc kv ->
                Alcotest.(check (option string)) "head title a"
                  (Some "a") (body_title kv)
            | _ -> Alcotest.fail "tree/get head not object")
       | _ -> Alcotest.fail "tree/get head missing");
      (* update -> title "a2" *)
      (match S.conn_of store with
       | Some conn ->
           ignore
             (T.upsert_datum conn ~path ~type_name
                ~body_json:"{\"title\":\"a2\",\"body\":\"b2\"}"
                ~author:"tester" ~rank:None)
       | None -> ());
      (* head now "a2" *)
      (match C.tree_get store ~path () with
       | VStr s ->
```

```
          (match Yojson.Safe.from_string s with
           | `Assoc kv ->
               Alcotest.(check (option string)) "head title becomes a2"
                 (Some "a2") (body_title kv)
           | _ -> Alcotest.fail "tree/get head2 not object")
       | _ -> Alcotest.fail "tree/get head2 missing");
      (* :gen g0 -> old body "a" *)
      (match C.tree_get store ~path ~at_gen:g0 () with
       | VStr s ->
          (match Yojson.Safe.from_string s with
           | `Assoc kv ->
               Alcotest.(check (option string)) ":gen g0 restores title a"
                 (Some "a") (body_title kv)
           | _ -> Alcotest.fail "tree/get :gen not object")
       | _ -> Alcotest.fail "tree/get :gen missing");
      (* cleanup *)
      (match S.conn_of store with
       | Some conn -> ignore (T.delete_datum conn ~type_name ~id:(path_id
    path))
       | None -> ()))

(* ---- gen-read absolute path at head and as-of ---- *)
let test_gen_read () =
  with_store (fun store ->
      let type_name = uniq "t" in
      declare_type store ~name:type_name ~fields:[ "title"; "body" ];
      let path = make_datum store ~type_name ~title:"a" in
      let g0 = head_gen store path in
      (match S.conn_of store with
       | Some conn ->
           ignore
             (T.upsert_datum conn ~path ~type_name
                ~body_json:"{\"title\":\"a2\",\"body\":\"b2\"}"
                ~author:"tester" ~rank:None)
       | None -> ());
      (* gen-read at g0 -> old body *)
      Alcotest.(check (option string)) "gen-read g0 -> a"
        (Some "a") (version_json_title store path g0);
      (* gen-read at head gen -> new body *)
      let g1 = head_gen store path in
      Alcotest.(check (option string)) "gen-read head -> a2"
        (Some "a2") (version_json_title store path g1);
      Alcotest.(check bool) "gens differ" true (Int64.compare g0 g1 <> 0);
      (* cleanup *)
      (match S.conn_of store with
       | Some conn -> ignore (T.delete_datum conn ~type_name ~id:(path_id
    path))
       | None -> ()))

(* ---- gen-diff node-granular diff between two gens ---- *)
let test_gen_diff () =
  with_store (fun store ->
      let type_name = uniq "t" in
      declare_type store ~name:type_name ~fields:[ "title"; "body" ];
      let path = make_datum store ~type_name ~title:"a" in
      let g0 = head_gen store path in
      (match S.conn_of store with
       | Some conn ->
           ignore
             (T.upsert_datum conn ~path ~type_name
                ~body_json:"{\"title\":\"a2\",\"body\":\"b2\"}"
                ~author:"tester" ~rank:None)
       | None -> ());
      let g1 = head_gen store path in
      (match C.tree_gen_diff store ~path ~gen_a:g0 ~gen_b:g1 with
       | VStr "same" -> Alcotest.fail "different bodies reported same"
       | VStr s ->
          (match Yojson.Safe.from_string s with
           | `Assoc kv ->
               Alcotest.(check bool) "diff.same false" false
                 (match List.assoc_opt "same" kv with
                  | Some (`Bool b) -> b | _ -> true);
               (* delta has title changed a->a2 *)
```

```
                (match List.assoc_opt "delta" kv with
                 | Some (`Assoc dkv) ->
                     Alcotest.(check bool) "delta has title key" true
                       (List.mem_assoc "title" dkv)
                 | _ -> Alcotest.fail "diff.delta missing")
           | _ -> Alcotest.fail "gen-diff not object")
       | _ -> Alcotest.fail "gen-diff missing");
      (* diff of a gen against itself -> hash-equal -> "same" *)
      (match C.tree_gen_diff store ~path ~gen_a:g1 ~gen_b:g1 with
       | VStr "same" -> ()
       | _ -> Alcotest.fail "gen-diff of same gen should be 'same'");
      (* cleanup *)
      (match S.conn_of store with
       | Some conn -> ignore (T.delete_datum conn ~type_name ~id:(path_id
    path))
       | None -> ()))

(* ---- undo: restore value at head-n as a NEW write at head+1 ---- *)
let test_undo () =
  with_store (fun store ->
      let type_name = uniq "t" in
      declare_type store ~name:type_name ~fields:[ "title"; "body" ];
      let path = make_datum store ~type_name ~title:"v0" in
      let g0 = head_gen store path in
      (* two more appends: v1, v2 *)
      (match S.conn_of store with
       | Some conn ->
           ignore
             (T.upsert_datum conn ~path ~type_name
                ~body_json:"{\"title\":\"v1\",\"body\":\"b\"}"
                ~author:"tester" ~rank:None);
           ignore
             (T.upsert_datum conn ~path ~type_name
                ~body_json:"{\"title\":\"v2\",\"body\":\"b\"}"
                ~author:"tester" ~rank:None)
       | None -> ());
      let g2 = head_gen store path in
      Alcotest.(check bool) "history grew" true (Int64.compare g2 g0 > 0);
      (* undo 1 -> restore v1 (head-1) as a NEW write at head+1 *)
      (match C.tree_undo store ~path () with
       | VStr _ -> ()
       | _ -> Alcotest.fail "undo returned non-string");
      let g_after1 = head_gen store path in
      Alcotest.(check bool) "undo appended a new gen" true
        (Int64.compare g_after1 g2 > 0);
      Alcotest.(check (option string)) "undo 1 restores v1"
        (Some "v1")
        (version_json_title store path g_after1);
      (* undo again on the restored state = redo (climbs back to v2 via the
         gen chain, restore-by-write -- spec: redo = undo again) *)
      (match C.tree_undo store ~path () with
       | VStr _ -> ()
       | _ -> Alcotest.fail "undo2 returned non-string");
      let g_after2 = head_gen store path in
      Alcotest.(check (option string)) "undo again redoes to v2"
        (Some "v2")
        (version_json_title store path g_after2));
  (* independent chain: undo ~n:2 reaches two steps deep (v0 from v2) *)
  with_store (fun store ->
      let type_name = uniq "t" in
      declare_type store ~name:type_name ~fields:[ "title"; "body" ];
      let path = make_datum store ~type_name ~title:"v0" in
      (match S.conn_of store with
       | Some conn ->
           ignore
             (T.upsert_datum conn ~path ~type_name
                ~body_json:"{\"title\":\"v1\",\"body\":\"b\"}"
                ~author:"tester" ~rank:None);
           ignore
             (T.upsert_datum conn ~path ~type_name
                ~body_json:"{\"title\":\"v2\",\"body\":\"b\"}"
                ~author:"tester" ~rank:None)
       | None -> ());
```

```
285       (match C.tree_undo store ~path ~n:2L () with
286        | VStr _ -> ()
287        | _ -> Alcotest.fail "undo n=2 returned non-string");
288       let g_new = head_gen store path in
289       Alcotest.(check (option string)) "undo n=2 restores v0"
290         (Some "v0")
291         (version_json_title store path g_new);
292       (* live node (tree/get head) reads the restored value too *)
293       (match C.tree_get store ~path () with
294        | VStr s ->
295            (match Yojson.Safe.from_string s with
296             | `Assoc kv ->
297                 Alcotest.(check (option string)) "live head after undo n=2"
298                   (Some "v0") (body_title kv)
299             | _ -> Alcotest.fail "undo live head not object")
300        | _ -> Alcotest.fail "undo live head missing");
301       (* cleanup *)
302       (match S.conn_of store with
303        | Some conn -> ignore (T.delete_datum conn ~type_name ~id:(path_id
      path))
304        | None -> ()))
305
306 (* tree-time.consumers (notes-undo): undo must be KIND-AWARE. A note is a
307    hierarchical node (kind=note written through upsert_hier); a naive
308    restore-by-write through upsert_datum rewrites the payload envelope
       marker
309    to "datum" (kind column stays note, payload->>'kind' flips to datum) --
310    the corruption the notes-undo consumer exposes. Restore must go back
311    through upsert_hier preserving the original kind. *)
312
313 (** Read the live tree_nodes payload envelope marker for a path. *)
314 let live_payload_kind (conn : _) (path : string) : string option =
315   try
316     let rows =
317       Pricklypear_image_lib.Io_lwt.Db.execute conn
318         ~params:[ Pgx.Value.of_string path ]
319         "SELECT payload->>'kind' FROM tree_nodes WHERE path = $1::ltree"
320     in
321     (match rows with
322      | [ [ c ] ] -> Pgx.Value.to_string c
323      | _ -> None)
324   with _ -> None
325
326 let test_undo_note_kind () =
327   with_store (fun store ->
328       let slug = uniq "note" in
329       let path = Printf.sprintf "data.note.u.tester.%s" slug in
330       (* two note versions through the hier write surface *)
331       (match S.conn_of store with
332        | Some conn ->
333            ignore
334              (T.upsert_hier conn ~path ~kind:"note" ~type_name:"note"
335                 ~body_json:
336                   (Printf.sprintf
337
      "{\"uuid\":\"%s\",\"slug\":\"%s\",\"title\":\"v1\",\"tags\":\"\",\"kind\":\"note\",\"day\":\"\"}"
338                      slug slug)
339                 ~author:"tester" ~rank:None);
340            ignore
341              (T.upsert_hier conn ~path ~kind:"note" ~type_name:"note"
342                 ~body_json:
343                   (Printf.sprintf
344
      "{\"uuid\":\"%s\",\"slug\":\"%s\",\"title\":\"v2\",\"tags\":\"\",\"kind\":\"note\",\"day\":\"\"}"
345                      slug slug)
346                 ~author:"tester" ~rank:None)
347        | None -> ());
348       (match C.tree_undo store ~path () with
349        | VStr _ -> ()
350        | _ -> Alcotest.fail "note undo returned non-string");
351       (* restored title, node kind, AND payload marker kind all intact *)
352       (match C.tree_get store ~path () with
353        | VStr s ->
```

```
            Printf.eprintf "[DBG] noteundo get: %s\n%!" s;
            (match Yojson.Safe.from_string s with
             | `Assoc kv ->
                 Alcotest.(check (option string)) "undo restores note title
    v1"
                   (Some "v1") (body_title kv);
                 let node_kind =
                   match List.assoc_opt "kind" kv with
                   | Some (`String k) -> Some k
                   | _ -> None
                 in
                 Alcotest.(check (option string)) "node kind stays note"
                   (Some "note") node_kind
             | _ -> Alcotest.fail "note live head not object")
       | _ -> Alcotest.fail "note live head missing");
      (match S.conn_of store with
       | Some conn ->
           Alcotest.(check (option string))
             "payload marker kind stays note (not datum)"
             (Some "note") (live_payload_kind conn path)
       | None -> ());
      (* cleanup: delete the whole note subtree *)
      (match S.conn_of store with
       | Some conn -> ignore (T.delete_path conn ~path ~recursive:true)
       | None -> ()))

let () =
  Random.self_init ();
  Alcotest.run "reads_as_of_gen"
    [
      ( "reads-as-of-gen",
        [
          Alcotest.test_case "tree/get optional :gen reads-as-of" `Quick
            test_tree_get_at_gen;
          Alcotest.test_case "gen-read absolute path at head and as-of"
            `Quick test_gen_read;
          Alcotest.test_case "gen-diff node-granular" `Quick test_gen_diff;
          Alcotest.test_case "undo restore-by-write" `Quick test_undo;
          Alcotest.test_case "undo kind-aware (note payload not datumed)"
            `Quick test_undo_note_kind;
        ] );
    ]
```

138 image/test/sync_db_test.ml

```
(** image/test/sync_db_test.ml -- borge/local-first.borg sync (campaign 8.3),
    DB-BACKED half of the sync merge authority (image/lib/sync.ml).

    The PURE core (classify / plan_of / actionable / JSON) is covered by
    sync_test.ml (pure, no PG). This file exercises the DB-backed harness
    the server-side commit path uses:

      - server_manifest reads the server's subtree (path, hash) manifest
        straight from tree_nodes (the exchange's server-side offer).
      - commit applies a client push (theirs_add) node through the
        typed-boundary-validated datum write path, and SURFACES conflicts
        report-only (sync-conflicts v0: LWW keeps server, conflict reported,
        no silent loss).

    Needs scripts/dev.sh Postgres; skips cleanly when unreachable.
*)

module S = Pricklypear_image_lib.Store
module Sync = Pricklypear_image_lib.Sync

let main_id = "00000000-0000-0000-0000-000000000001"

let anon_enc = Pricklypear_image_lib.Tree_store.enc_label "anonymous"

let with_store f =
  Eio_main.run @@ fun env ->
  Lwt_eio.with_event_loop ~clock:(Eio.Stdenv.clock env) @@ fun _ ->
  Eio.Switch.run @@ fun sw ->
  let ctx : Pricklypear_image_lib.Io_eio.Ctx.t =
    { sw; net = (Eio.Stdenv.net env :> [ `Generic ] Eio.Net.ty Eio.Std.r) }
  in
  Pricklypear_image_lib.Io_eio.with_ctx ctx (fun () ->
      let cfg = S.config_from_env () in
      let store = S.make ~cfg ~branch_id:main_id () in
      S.connect store;
      match S.conn_of store with
      | None ->
          Printf.printf "SKIP: no postgres (set PP_DB_* / run scripts/dev.sh)\n%!"
      | Some _ -> f store)

let h n = Printf.sprintf "h%d" n

let test_server_manifest () =
  with_store (fun store ->
      let uniq = Printf.sprintf "syn%06d" (Random.int 1_000_000) in
      let path = "data.u." ^ anon_enc ^ ".sync_" ^ uniq ^ ".probe" in
      (match S.conn_of store with
       | None -> ()
       | Some conn ->
           ignore
             (Pricklypear_image_lib.Tree_store.insert_datum conn
                ~type_name:("sync_" ^ uniq)
                ~fields_json:"{\"n\":1}" ~author:"anonymous"));
      let prefix = "data.u." ^ anon_enc in
      (match Sync.server_manifest store ~prefix with
       | None -> Alcotest.fail "server_manifest None"
       | Some m ->
           Alcotest.(check bool) "probe row in manifest" true
             (List.exists (fun (p, hs) -> p = path && hs <> "") m);
           Alcotest.(check bool) "hash is 64-hex" true
             (List.exists (fun (p, hs) -> p = path && String.length hs = 64) m);
           (match S.conn_of store with
            | Some conn ->
                ignore
                  (S.exec conn
                     ~params:[ Pgx.Value.of_string path ]
                     "DELETE FROM tree_nodes WHERE path = $1::ltree")
            | None -> ())))
```

```
69
70  let test_commit_push () =
71    with_store (fun store ->
72        let uniq = Printf.sprintf "syn2%05d" (Random.int 1_000_000) in
73        let ty = "syncpush_" ^ uniq in
74        let path = "data.u." ^ anon_enc ^ "." ^ ty ^ ".one" in
75        (* declare a type with one required field n *)
76        (match S.conn_of store with
77         | Some conn ->
78             ignore
79               (S.exec conn
80                  ~params:[ Pgx.Value.of_string ty ]
81                  "INSERT INTO type_defs (branch_id, name, fields) \
82                   VALUES ($1::uuid, $2, '[{\"name\":\"n\"}]'::jsonb) \
83                   ON CONFLICT (branch_id, name) DO NOTHING")
84         | None -> ());
85        let env : Yojson.Safe.t =
86          `Assoc
87            [ ("v", `Int 1)
88            ; ("kind", `String "datum")
89            ; ("path", `String path)
90            ; ("type", `String ty)
91            ; ("body", `Assoc [ ("n", `Int 42) ])
92            ; ("author", `String "anonymous")
93            ]
94        in
95        (* three-way: base empty, server empty, client added the node *)
96        let plans = Sync.plan_of [] [] [ (path, "H") ] in
97        let actionable =
98          Sync.actionable plans
99          |> List.filter (fun { Sync.path = p; _ } -> p = path)
100       in
101       Alcotest.(check int) "theirs_add actionable" 1 (List.length
      actionable);
102       let r = Sync.commit store ~client_bodies:[ (path, env) ] plans in
103       Alcotest.(check int) "applied 1" 1 r.applied;
104       Alcotest.(check int) "no conflicts" 0 (List.length r.conflicts);
105       (match S.conn_of store with
106        | Some conn ->
107            Alcotest.(check bool) "node persisted" true
108              (Pricklypear_image_lib.Tree_store.get_by_path conn ~path <>
      None);
109            ignore
110              (S.exec conn ~params:[ Pgx.Value.of_string path ]
111                 "DELETE FROM tree_nodes WHERE path = $1::ltree");
112            ignore
113              (S.exec conn ~params:[ Pgx.Value.of_string ty ]
114                 "DELETE FROM type_defs WHERE name = $1")
115        | None -> ()))
116
117 let test_commit_surfaces_conflict () =
118   with_store (fun store ->
119       (* A conflict is LWW-arbitrated per sync-conflicts v0 AND surfaced
120          either way (the obituary rule: no silent sync loss). With no
121          client envelope (no updated_at stamp), the server copy stays and
122          the conflict is reported with the three hashes + the kept list. *)
123       let plans =
124         Sync.plan_of [ ("a", h 1) ] [ ("a", h 8) ] [ ("a", h 9) ]
125       in
126       let r = Sync.commit store ~client_bodies:[] plans in
127       Alcotest.(check int) "nothing applied on conflict" 0 r.applied;
128       Alcotest.(check int) "one conflict surfaced" 1 (List.length
      r.conflicts);
129       Alcotest.(check int) "server kept (no client stamp)" 1
130         (List.length r.lww_server_kept);
131       Alcotest.(check int) "client won nothing" 0
132         (List.length r.lww_client_won);
133       (match r.conflicts with
134        | [ (path, o, t) ] ->
135            Alcotest.(check string) "conflict path" "a" path;
136            Alcotest.(check string) "ours hash kept" (h 8) o;
137            Alcotest.(check string) "theirs hash reported" (h 9) t
138        | _ -> Alcotest.fail "unexpected conflicts"))
```

```
139
140 let test_commit_lww_client_newer_wins () =
141   with_store (fun store ->
142       (* LWW v0: the client envelope carries a STRICTLY newer updated_at
143          (epoch) than the server node ? the client copy is applied and
144          the win surfaced. The server node must exist for the clock to
145          arbitrate, so insert it first via the typed write surface. *)
146       let uniq = Printf.sprintf "lww%05d" (Random.int 1_000_000) in
147       let ty = "synclww_" ^ uniq in
148       let path = "data.u." ^ anon_enc ^ "." ^ ty ^ ".one" in
149       (match S.conn_of store with
150        | Some conn ->
151            ignore
152              (S.exec conn
153                 ~params:[ Pgx.Value.of_string ty ]
154                 "INSERT INTO type_defs (branch_id, name, fields) \
155                  VALUES ($1::uuid, $2, '[{\"name\":\"n\"}]'::jsonb) \
156                  ON CONFLICT (branch_id, name) DO NOTHING")
157        | None -> ());
158       let server_env : Yojson.Safe.t =
159         `Assoc
160           [ ("v", `Int 1); ("kind", `String "datum"); ("path", `String
    path);
161             ("type", `String ty); ("body", `Assoc [ ("n", `Int 1) ]);
162             ("author", `String "anonymous") ]
163       in
164       (match Sync.apply_client_node store ~env:server_env with
165        | Ok () -> () | Error e -> Alcotest.fail ("seed node: " ^ e));
166       (* the node exists with a fresh server updated_at; a client edit
167          stamped in the FUTURE strictly wins *)
168       let future = Unix.gettimeofday () +. 3600.0 in
169       let client_env : Yojson.Safe.t =
170         `Assoc
171           [ ("v", `Int 1); ("kind", `String "datum"); ("path", `String
    path);
172             ("type", `String ty); ("body", `Assoc [ ("n", `Int 2) ]);
173             ("author", `String "anonymous");
174             ("updated_at", `String (string_of_float future)) ]
175       in
176       let plans =
177         Sync.plan_of [ (path, h 1) ] [ (path, h 8) ] [ (path, h 9) ]
178       in
179       let r =
180         Sync.commit store ~client_bodies:[ (path, client_env) ] plans
181       in
182       Alcotest.(check int) "LWW applied client copy" 1 r.applied;
183       Alcotest.(check int) "one conflict surfaced" 1 (List.length
    r.conflicts);
184       Alcotest.(check int) "client won" 1 (List.length r.lww_client_won);
185       Alcotest.(check int) "server kept nothing" 0
186         (List.length r.lww_server_kept);
187       (match S.conn_of store with
188        | Some conn ->
189            (* the applied body is the client's (n=2): assert via the server
190               manifest hash change over the original body hash *)
191            let m =
192              Option.value
193                (Sync.server_manifest store ~prefix:("data.u." ^ anon_enc))
194                ~default:[]
195            in
196            Alcotest.(check bool) "node still present post-LWW" true
197              (List.exists (fun (p, _) -> p = path) m);
198            ignore (S.exec conn ~params:[ Pgx.Value.of_string path ]
199                      "DELETE FROM tree_nodes WHERE path = $1::ltree");
200            ignore (S.exec conn ~params:[ Pgx.Value.of_string ty ]
201                      "DELETE FROM type_defs WHERE name = $1")
202        | None -> ()))
203
204 let test_commit_theirs_del_and_ours_del () =
205   with_store (fun store ->
206       (* Client DELETION propagates: base+server hold the node, client
207          manifest lacks it ? Theirs_del ? the server node is deleted.
208          And server deletions come back as client_deletes (Ours_del): the
```

```
209           client must mirror-delete its stale local copy. *)
210        let uniq = Printf.sprintf "del%05d" (Random.int 1_000_000) in
211        let ty = "syncdel_" ^ uniq in
212        let path = "data.u." ^ anon_enc ^ "." ^ ty ^ ".one" in
213        (match S.conn_of store with
214         | Some conn ->
215             ignore
216               (S.exec conn
217                  ~params:[ Pgx.Value.of_string ty ]
218                  "INSERT INTO type_defs (branch_id, name, fields) \
219                   VALUES ($1::uuid, $2, '[{\"name\":\"n\"}]'::jsonb) \
220                   ON CONFLICT (branch_id, name) DO NOTHING")
221         | None -> ());
222        let env : Yojson.Safe.t =
223          `Assoc
224            [ ("v", `Int 1); ("kind", `String "datum"); ("path", `String
    path);
225              ("type", `String ty); ("body", `Assoc [ ("n", `Int 3) ]);
226              ("author", `String "anonymous") ]
227        in
228        (match Sync.apply_client_node store ~env with
229         | Ok () -> () | Error e -> Alcotest.fail ("seed node: " ^ e));
230        (* theirs_del: base has it, ours (server) has it at base hash,
231           theirs (client) deleted it *)
232        let m =
233          Option.value
234            (Sync.server_manifest store ~prefix:("data.u." ^ anon_enc))
235            ~default:[]
236        in
237        let srv_hash = List.assoc path m in
238        let plans =
239          Sync.plan_of [ (path, srv_hash) ] [ (path, srv_hash) ] []
240        in
241        let r = Sync.commit store ~client_bodies:[] plans in
242        Alcotest.(check int) "deletion applied" 1 r.applied;
243        (match S.conn_of store with
244         | Some conn ->
245             Alcotest.(check bool) "node deleted server-side" false
246               (Pricklypear_image_lib.Tree_store.get_by_path conn ~path <>
    None)
247         | None -> ());
248        (* ours_del: base+client hold it, server (ours) deleted ? the plan
249           reports Ours_del and the commit surfaces it as client_deletes *)
250        let plans2 =
251          Sync.plan_of [ (path, srv_hash) ] [] [ (path, srv_hash) ]
252        in
253        let r2 = Sync.commit store ~client_bodies:[] plans2 in
254        Alcotest.(check (list string)) "client mirror-deletes" [ path ]
255          r2.client_deletes;
256        (match S.conn_of store with
257         | Some conn ->
258             ignore
259               (S.exec conn ~params:[ Pgx.Value.of_string ty ]
260                  "DELETE FROM type_defs WHERE name = $1")
261         | None -> ()))
262
263  let test_two_way_push_applies_client_only () =
264    (* The sync/push prim path: a TWO-WAY classify (no base) where the
265       server lacks a node the client holds (ClientOnly) is applied through
266       Sync.apply_client_node. This is the v1 local-first push-up mechanism
267       (borge/local-first.borg sync; diverge is reported, not auto-merged). *)
268    with_store (fun store ->
269        let uniq = Printf.sprintf "sp3%05d" (Random.int 1_000_000) in
270        let ty = "syncpush2_" ^ uniq in
271        let path = "data.u." ^ anon_enc ^ "." ^ ty ^ ".one" in
272        (match S.conn_of store with
273         | Some conn ->
274             ignore
275               (S.exec conn
276                  ~params:[ Pgx.Value.of_string ty ]
277                  "INSERT INTO type_defs (branch_id, name, fields) \
278                   VALUES ($1::uuid, $2, '[{\"name\":\"n\"}]'::jsonb) \
279                   ON CONFLICT (branch_id, name) DO NOTHING")
```

```
       | None -> ());
      let env : Yojson.Safe.t =
        `Assoc
          [ ("v", `Int 1)
          ; ("kind", `String "datum")
          ; ("path", `String path)
          ; ("type", `String ty)
          ; ("body", `Assoc [ ("n", `Int 7) ])
          ; ("author", `String "anonymous")
          ]
      in
      (* server manifest is EMPTY for this subtree; client offers the
         node ? classify reports it ClientOnly *)
      let server = Sync.server_manifest store ~prefix:("data.u." ^
    anon_enc) in
      let server = Option.value server ~default:[] in
      let r = Sync.classify server [ (path, "H") ] in
      Alcotest.(check int) "one client_only" 1 (List.length r.client_only);
      Alcotest.(check int) "no diverge" 0 (List.length r.diverge);
      (* apply the ClientOnly push through the same surface the prim uses *)
      let applied =
        List.fold_left
          (fun n (p, _) ->
            if List.mem_assoc p r.client_only then
              (match Sync.apply_client_node store ~env with
               | Ok () -> n + 1
               | Error _ -> n)
            else n)
          0 [ (path, "H") ]
      in
      Alcotest.(check int) "applied 1" 1 applied;
      (match S.conn_of store with
       | Some conn ->
           Alcotest.(check bool) "pushed node persisted" true
             (Pricklypear_image_lib.Tree_store.get_by_path conn ~path <>
    None);
           ignore
             (S.exec conn ~params:[ Pgx.Value.of_string path ]
                "DELETE FROM tree_nodes WHERE path = $1::ltree");
           ignore
             (S.exec conn ~params:[ Pgx.Value.of_string ty ]
                "DELETE FROM type_defs WHERE name = $1")
       | None -> ()))

let () =
  Random.self_init ();
  Alcotest.run "sync_db"
    [ ( "sync-db",
        [
          Alcotest.test_case "server_manifest reads subtree hashes" `Quick
            test_server_manifest;
          Alcotest.test_case "commit applies a client push" `Quick
            test_commit_push;
          Alcotest.test_case "commit surfaces conflicts (no silent loss)"
            `Quick test_commit_surfaces_conflict;
          Alcotest.test_case "commit LWW: strictly-newer client wins"
            `Quick test_commit_lww_client_newer_wins;
          Alcotest.test_case "commit theirs_del + ours_del"
            `Quick test_commit_theirs_del_and_ours_del;
          Alcotest.test_case "two-way push applies client-only node"
            `Quick test_two_way_push_applies_client_only;
        ] ) ]
```

139 image/test/sync_test.ml

```
(** local-first sync authority tests (local-first.sync, campaign 8.3).

    The heart of sync is PURE (no Postgres): manifest classification
    (the exchange's conflict DETECTION at node granularity) and the
    three-way merge planner (base = last-synced, ours = server,
    theirs = client -- the campaign-3 diff philosophy). These tests
    exercise the pure core exhaustively, plus the JSON report rendering
    the sync/plan prim returns. The DB-backed harness
    (Sync.server_manifest / Sync.apply_client_node / Sync.commit) is
    exercised against the live dev image / shared PG when reachable, and
    skipped cleanly otherwise.

    Each test uses synthetic hashes (h1..hn) -- content identity is
    opaque to the planner; only equality/membership matters.
*)

module S = Pricklypear_image_lib.Sync
open Alcotest

let h (n : int) : string = Printf.sprintf "h%d" n

(* ---- classify (two-way negotiation) ---- *)

let classify_same () =
  let server = [ ("data.u.x.a", h 1); ("data.u.x.b", h 2) ] in
  let client = [ ("data.u.x.a", h 1); ("data.u.x.b", h 2) ] in
  let r = S.classify server client in
  check (int) "same_len" 2 r.same_len;
  check (int) "client_only" 0 (List.length r.client_only);
  check (int) "server_only" 0 (List.length r.server_only);
  check (int) "diverge" 0 (List.length r.diverge)

let classify_client_new () =
  let server = [ ("data.u.x.a", h 1) ] in
  let client = [ ("data.u.x.a", h 1); ("data.u.x.b", h 2) ] in
  let r = S.classify server client in
  check (int) "same_len" 1 r.same_len;
  check (int) "client_only" 1 (List.length r.client_only);
  check (int) "server_only" 0 (List.length r.server_only);
  check (bool) "b on client" true (List.mem_assoc "data.u.x.b"
     r.client_only)

let classify_server_new () =
  let server = [ ("data.u.x.a", h 1); ("data.u.x.c", h 3) ] in
  let client = [ ("data.u.x.a", h 1) ] in
  let r = S.classify server client in
  check (int) "same_len" 1 r.same_len;
  check (int) "client_only" 0 (List.length r.client_only);
  check (int) "server_only" 1 (List.length r.server_only);
  check (bool) "c on server" true (List.mem_assoc "data.u.x.c"
     r.server_only)

let classify_diverge () =
  let server = [ ("data.u.x.a", h 1) ] in
  let client = [ ("data.u.x.a", h 9) ] in
  let r = S.classify server client in
  check (int) "same_len" 0 r.same_len;
  check (int) "diverge" 1 (List.length r.diverge);
  check (string) "div path" "data.u.x.a" (fst (List.hd r.diverge))

(* ---- three-way plan (base = last-synced, ours = server, theirs =
   client) ---- *)

let plan_one (plans : S.plan list) (path : string) : S.outcome =
  match List.find_opt (fun { S.path = p; _ } -> p = path) plans with
  | Some { S.outcome; _ } -> outcome
  | None -> failwith ("no plan for " ^ path)

let plan_unchanged () =
  let p = S.plan_of [ ("a", h 1) ] [ ("a", h 1) ] [ ("a", h 1) ] in
  (match plan_one p "a" with
```

```
   | S.Unchanged -> ()
   | _ -> failwith "expected Unchanged")

let plan_theirs_add () =
  (* client added a node neither base nor server have -> server applies it
     *)
  let p = S.plan_of [] [ ("a", h 1) ] [ ("a", h 1); ("b", h 2) ] in
  (match plan_one p "b" with
   | S.Theirs_add -> ()
   | _ -> failwith "expected Theirs_add")

let plan_ours_add () =
  (* server added a node; client never had it -> keep, no transfer *)
  let p = S.plan_of [] [ ("a", h 1); ("c", h 3) ] [ ("a", h 1) ] in
  (match plan_one p "c" with
   | S.Ours_keep -> ()
   | _ -> failwith "expected Ours_keep")

let plan_theirs_mod () =
  (* client edited a node the server never moved from base -> apply client
     body (a clean client edit). *)
  let p = S.plan_of [ ("a", h 1) ] [ ("a", h 1) ] [ ("a", h 9) ] in
  (match plan_one p "a" with
   | S.Theirs_mod -> ()
   | _ -> failwith "expected Theirs_mod")

let plan_ours_mod () =
  (* server edited a node the client never moved from base -> keep server *)
  let p = S.plan_of [ ("a", h 1) ] [ ("a", h 9) ] [ ("a", h 1) ] in
  (match plan_one p "a" with
   | S.Ours_keep -> ()
   | _ -> failwith "expected Ours_keep")

let plan_identical () =
  (* both moved to the SAME new hash -> one change, keep either *)
  let p = S.plan_of [ ("a", h 1) ] [ ("a", h 9) ] [ ("a", h 9) ] in
  (match plan_one p "a" with
   | S.Identical -> ()
   | _ -> failwith "expected Identical")

let plan_conflict () =
  (* base present, both changed to different hashes -> report conflict *)
  let p = S.plan_of [ ("a", h 1) ] [ ("a", h 8) ] [ ("a", h 9) ] in
  (match plan_one p "a" with
   | S.Conflict (path, b, o, t) ->
       check (string) "path" "a" path;
       check (string) "base" (h 1) b;
       check (string) "ours" (h 8) o;
       check (string) "theirs" (h 9) t
   | _ -> failwith "expected Conflict")

let plan_theirs_del () =
  (* client deleted a node the server still holds -> delete server node *)
  let p = S.plan_of [ ("a", h 1) ] [ ("a", h 1) ] [] in
  (match plan_one p "a" with
   | S.Theirs_del -> ()
   | _ -> failwith "expected Theirs_del")

let plan_ours_del () =
  (* server deleted a node the client still holds -> client pulls *)
  let p = S.plan_of [ ("a", h 1) ] [] [ ("a", h 1) ] in
  (match plan_one p "a" with
   | S.Ours_del -> ()
   | _ -> failwith "expected Ours_del")

let actionable_only_transfers () =
  (* only transfer-or-conflict rows are actionable *)
  let p =
    S.plan_of
      [ ("a", h 1); ("b", h 1); ("c", h 1); ("d", h 1); ("e", h 1) ]
      [ ("a", h 1); ("b", h 1); ("c", h 8); ("d", h 1); ("e", h 1) ]
      [ ("a", h 1); ("b", h 9); ("c", h 1); ("d", h 9); ("e", h 1) ]
  in
```

```
  let act = S.actionable p in
  let paths = List.map (fun { S.path; _ } -> path) act in
  check (bool) "unchanged a filtered" true (not (List.mem "a" paths));
  check (bool) "theirs_mod b actionable" true (List.mem "b" paths);
  check (bool) "ours_keep c filtered" true (not (List.mem "c" paths));
  check (bool) "conflict d actionable" true (List.mem "d" paths);
  check (bool) "unchanged e filtered" true (not (List.mem "e" paths))

(* ---- JSON rendering (the sync/plan prim surface) ---- *)

let json_report_roundtrip () =
  let r = S.classify [ ("a", h 1) ] [ ("a", h 9); ("b", h 2) ] in
  let j = Yojson.Safe.from_string (S.json_of_report r) in
  (match j with
   | `Assoc kv ->
       check (int) "same" 0 (match List.assoc_opt "same" kv with Some (`Int
     n) -> n | _ -> -1);
       check (int) "client_only" 1 (match List.assoc_opt "client_only" kv
     with Some (`Int n) -> n | _ -> -1);
       check (int) "diverge" 1 (match List.assoc_opt "diverge" kv with Some
     (`Int n) -> n | _ -> -1)
   | _ -> failwith "expected Assoc")

let json_plan_roundtrip () =
  let p = S.plan_of [ ("a", h 1) ] [ ("a", h 8) ] [ ("a", h 9) ] in
  let j = Yojson.Safe.from_string (S.json_of_plan p) in
  (match j with
   | `Assoc kv ->
       (match List.assoc_opt "rows" kv with
        | Some (`List [ `Assoc row ]) ->
            (match List.assoc_opt "outcome" row with
             | Some (`String "conflict") -> ()
             | _ -> failwith "expected conflict outcome in JSON")
        | _ -> failwith "expected one row")
   | _ -> failwith "expected Assoc")

let () =
  Alcotest.run "sync" [
    ("classify", [
        test_case "same" `Quick classify_same;
        test_case "client-new" `Quick classify_client_new;
        test_case "server-new" `Quick classify_server_new;
        test_case "diverge" `Quick classify_diverge;
      ]);
    ("three-way", [
        test_case "unchanged" `Quick plan_unchanged;
        test_case "theirs-add" `Quick plan_theirs_add;
        test_case "ours-add" `Quick plan_ours_add;
        test_case "theirs-mod" `Quick plan_theirs_mod;
        test_case "ours-mod" `Quick plan_ours_mod;
        test_case "identical" `Quick plan_identical;
        test_case "conflict" `Quick plan_conflict;
        test_case "theirs-del" `Quick plan_theirs_del;
        test_case "ours-del" `Quick plan_ours_del;
        test_case "actionable-subset" `Quick actionable_only_transfers;
      ]);
    ("json", [
        test_case "report" `Quick json_report_roundtrip;
        test_case "plan" `Quick json_plan_roundtrip;
      ]);
  ]
```

140 image/test/tree_datum_test.ml

```
(** Tree datum prims regression tests (one-tree.tree-datum-prims).

    The tree prim family must operate on DATUM nodes directly -- not only on
    hierarchical kinds. tree/upsert with kind="datum" is the path-explicit
    variant of create/update: caller-supplied (or placeholder-generated)
    path, same typed-boundary validation, ON CONFLICT upsert, and the
    tree_data_rows view stays in sync so (get 't id) / (rows 't) reflect the
    node identically to one written via create. tree/move re-homes a datum
    (rewrites path + payload.path); tree/delete removes it; the readers
    (tree/get, tree/list, tree/list-kind) are kind-agnostic so they return
    datum nodes too. created_at is stable across an upsert rewrite while
    updated_at advances.

    Needs Postgres (scripts/dev.sh). Skips cleanly when unreachable.

    Acceptance (one-tree.tree-datum-prims verify stanzas):
      - tree/upsert a datum node of a declared type (all required fields) ?
        node at the given path; (get 't id) sees the same fields.
      - tree/upsert a datum missing a required field ? clean rejection
        naming the field (identical to create).
      - tree/upsert an existing datum path with a changed field ? body
        updates in place; the projection reflects the new value.
      - tree/move a datum node to a new path ? (get 't id) at the new path
        returns it; old path is gone.
      - tree/delete a datum path ? node removed.
      - tree/get / tree/list / tree/list-kind return datum nodes.
      - tree/get of any node returns JSON with created_at and updated_at;
        created_at stable across an upsert rewrite, updated_at advances.
*)

module S = Pricklypear_image_lib.Store
module T = Pricklypear_image_lib.Tree_store
module C = Pricklypear_image_lib.Crud
open Pricklypear_lisp.Ast

let main_id = "00000000-0000-0000-0000-000000000001"

let with_store f =
  Eio_main.run @@ fun env ->
  Lwt_eio.with_event_loop ~clock:(Eio.Stdenv.clock env) @@ fun _ ->
  Eio.Switch.run @@ fun sw ->
  let ctx : Pricklypear_image_lib.Io_eio.Ctx.t =
    {
      sw;
      net = (Eio.Stdenv.net env :> [ `Generic ] Eio.Net.ty Eio.Std.r);
    }
  in
  Pricklypear_image_lib.Io_eio.with_ctx ctx (fun () ->
      let cfg = S.config_from_env () in
      let store = S.make ~cfg ~branch_id:main_id () in
      S.connect store;
      match S.conn_of store with
      | None ->
          Printf.printf
            "SKIP: no postgres (set PP_DB_* / run scripts/dev.sh)\n%!"
      | Some _ -> f store)

let uniq p = Printf.sprintf "td_%s_%d" p (Random.int 1_000_000)

(** True when [sub] is a substring of [s]. *)
let contains_sub (s : string) (sub : string) : bool =
  let n = String.length sub in
  if n = 0 then true
  else
    let rec go i =
      if i + n > String.length s then false
      else if String.sub s i n = sub then true else go (i + 1)
    in
    go 0

(** Declare a Nopales type on system/main so check_type_fields validates. *)
```

```
let declare_type (store : S.t) ~name ~fields =
  S.persist_type store ~name ~fields ~author:"tester"

(* ---- upsert with a placeholder id generates a uuid + path ---- *)
let test_upsert_placeholder () =
  with_store (fun store ->
      let type_name = uniq "t" in
      declare_type store ~name:type_name ~fields:[ "title"; "body" ];
      let body = "{\"title\":\"x\",\"body\":\"y\"}" in
      (match S.conn_of store with
       | None -> ()
       | Some conn ->
           match
             T.upsert_datum conn
               ~path:"data.u.tester.new"
               ~type_name ~body_json:body ~author:"tester" ~rank:None
           with
           | Ok p ->
               (* generated path: data.u.tester.<type>.<uuid> *)
               Alcotest.(check bool) "generated real path" true
                 (String.length p > 0
                  && List.length (String.split_on_char '.' p) = 5);
               (* node readable at that path *)
               (match T.get_by_path conn ~path:p with
                | None -> Alcotest.fail "generated datum not readable"
                | Some n ->
                    Alcotest.(check string) "kind datum" "datum" n.kind;
                    Alcotest.(check (option string)) "type_name" (Some
    type_name)
                      n.type_name)
           | Error e -> Alcotest.fail ("upsert placeholder failed: " ^ e)))

(* ---- upsert a full caller-supplied path; projection in sync ---- *)
let test_upsert_full_path () =
  with_store (fun store ->
      let type_name = uniq "t" in
      declare_type store ~name:type_name ~fields:[ "title"; "body" ];
      let id = "00000000-0000-0000-0000-00000000" ^ "0001" in
      let path = T.datum_path ~user:"tester" ~type_name ~id () in
      let body = "{\"title\":\"hello\",\"body\":\"world\"}" in
      (match S.conn_of store with
       | None -> ()
       | Some conn ->
           (match
              T.upsert_datum conn ~path ~type_name ~body_json:body
    ~author:"tester" ~rank:None
            with
            | Error e -> Alcotest.fail ("upsert failed: " ^ e)
            | Ok p -> Alcotest.(check string) "returned path" path p);
           (* tree_data_rows view sees it (the (get 't id) backing) *)
           let sql =
             "SELECT fields->>'title'::text FROM tree_data_rows " ^
             "WHERE type_name = $1 AND id = $2::uuid"
           in
           let rows =
             S.exec conn
               ~params:[ Pgx.Value.of_string type_name;
                         Pgx.Value.of_string id ]
               sql
           in
           (match rows with
            | [ [ title ] ] ->
                Alcotest.(check (option string)) "view title" (Some "hello")
                  (Pgx.Value.to_string title)
            | _ -> Alcotest.fail "projection did not reflect upsert")))

(* ---- upsert existing path: body updates in place, created_at stable,
        updated_at advances ---- *)
let test_upsert_update_inplace () =
  with_store (fun store ->
      let type_name = uniq "t" in
      declare_type store ~name:type_name ~fields:[ "title"; "body" ];
      let id = "00000000-0000-0000-0000-00000000" ^ "0002" in
```

```
143       let path = T.datum_path ~user:"tester" ~type_name ~id () in
144       (match S.conn_of store with
145        | None -> ()
146        | Some conn ->
147            (match
148               T.upsert_datum conn ~path ~type_name
149                 ~body_json:"{\"title\":\"a\",\"body\":\"b\"}"
    ~author:"tester" ~rank:None
150              with
151              | Error e -> Alcotest.fail ("create failed: " ^ e)
152              | Ok _ -> ());
153             (match T.get_by_path conn ~path with
154              | None -> Alcotest.fail "node not found after create"
155              | Some n0 ->
156                  let created0 = n0.created_at in
157                  (* update in place *)
158                  (match
159                     T.upsert_datum conn ~path ~type_name
160                       ~body_json:"{\"title\":\"a2\",\"body\":\"b2\"}"
    ~author:"tester" ~rank:None
161                   with
162                   | Error e -> Alcotest.fail ("update failed: " ^ e)
163                   | Ok _ -> ());
164                  let sql =
165                    "SELECT fields->>'title'::text FROM tree_data_rows " ^
166                    "WHERE type_name = $1 AND id = $2::uuid"
167                  in
168                  let rows =
169                    S.exec conn
170                      ~params:[ Pgx.Value.of_string type_name;
171                                Pgx.Value.of_string id ]
172                      sql
173                  in
174                  (match rows with
175                   | [ [ title ] ] ->
176                       Alcotest.(check (option string)) "updated title" (Some
    "a2")
177                         (Pgx.Value.to_string title)
178                   | _ -> Alcotest.fail "projection did not update");
179                  (match T.get_by_path conn ~path with
180                   | None -> Alcotest.fail "node gone after update"
181                   | Some n1 ->
182                       Alcotest.(check string) "created_at stable" created0
    n1.created_at;
183                       Alcotest.(check bool) "updated_at advanced" true
184                         (String.compare n1.updated_at created0 > 0)))))
185
186 (* ---- Crud.tree_upsert dispatches kind="datum" + typed-boundary reject
    ---- *)
187 let test_tree_upsert_prim () =
188   with_store (fun store ->
189       let type_name = uniq "t" in
190       declare_type store ~name:type_name ~fields:[ "title"; "body" ];
191       (* missing required field -> clean rejection naming the field *)
192       let r = C.tree_upsert store ~path:"data.u.tester.new" ~kind:"datum"
193           ~type_name ~body_json:"{\"title\":\"only\"}" ~author:"tester" in
194       (match r with
195        | VStr s ->
196            Alcotest.(check bool) "clean rejection (non-empty)" true
197              (String.length s > 0);
198            Alcotest.(check bool) "names the missing field 'body'" true
199              (let l = String.lowercase_ascii s in
200               l <> "" && (String.contains l 'b')
201               && String.length s > 5)
202        | _ -> Alcotest.fail "expected string rejection");
203       (* declared type full path happy path via the prim *)
204       let id = "00000000-0000-0000-0000-00000000" ^ "0003" in
205       let path = T.datum_path ~user:"tester" ~type_name ~id () in
206       (match
207          C.tree_upsert store ~path ~kind:"datum" ~type_name
208            ~body_json:"{\"title\":\"ok\",\"body\":\"full\"}"
    ~author:"tester"
209         with
```

```
      | VStr p -> Alcotest.(check string) "prim returns path" path p
      | _ -> Alcotest.fail "prim happy path did not return path"))

(* ---- move a datum node to a new path ---- *)
let test_move_datum () =
  with_store (fun store ->
      let type_name = uniq "t" in
      declare_type store ~name:type_name ~fields:[ "title"; "body" ];
      let id = "00000000-0000-0000-0000-00000000" ^ "0004" in
      let old_path = T.datum_path ~user:"tester" ~type_name ~id () in
      let new_path = T.datum_path ~user:"tester" ~type_name
    ~id:"00000000-0000-0000-0000-000000000005" () in
      (match S.conn_of store with
       | None -> ()
       | Some conn ->
           (match
              T.upsert_datum conn ~path:old_path ~type_name
                ~body_json:"{\"title\":\"move\",\"body\":\"me\"}"
    ~author:"tester" ~rank:None
            with
           | Error e -> Alcotest.fail ("create failed: " ^ e) | Ok _ ->
    ());
           (match T.move_subtree conn ~old_path ~new_path with
            | Error e -> Alcotest.fail ("move failed: " ^ e)
            | Ok n -> Alcotest.(check int) "moved one node" 1 n);
           (match T.get_by_path conn ~path:new_path with
            | None -> Alcotest.fail "node not at new path"
            | Some n ->
                Alcotest.(check string) "new path has data" "move"
                  (match n.body with
                   | `Assoc kvs ->
                       (match List.assoc_opt "title" kvs with
                        | Some (`String s) -> s | _ -> "")
                   | _ -> ""));
           (match T.get_by_path conn ~path:old_path with
            | None -> ()   (* old path gone *)
            | Some _ -> Alcotest.fail "old path still present")))

(* ---- delete a datum path ---- *)
let test_delete_datum () =
  with_store (fun store ->
      let type_name = uniq "t" in
      declare_type store ~name:type_name ~fields:[ "title"; "body" ];
      let id = "00000000-0000-0000-0000-00000000" ^ "0006" in
      let path = T.datum_path ~user:"tester" ~type_name ~id () in
      (match S.conn_of store with
       | None -> ()
       | Some conn ->
           (match
              T.upsert_datum conn ~path ~type_name
                ~body_json:"{\"title\":\"del\",\"body\":\"me\"}"
    ~author:"tester" ~rank:None
            with
           | Error e -> Alcotest.fail ("create failed: " ^ e) | Ok _ ->
    ());
          let n = T.delete_path conn ~path ~recursive:false in
          Alcotest.(check int) "deleted one" 1 n;
          (match T.get_by_path conn ~path with
           | None -> ()   (* gone *)
           | Some _ -> Alcotest.fail "node still present after delete")))

(* ---- readers are kind-agnostic: datum nodes returned by list-kind ---- *)
let test_readers_kind_agnostic () =
  with_store (fun store ->
      let type_name = uniq "t" in
      declare_type store ~name:type_name ~fields:[ "title"; "body" ];
      let id = "00000000-0000-0000-0000-00000000" ^ "0007" in
      let path = T.datum_path ~user:"tester" ~type_name ~id () in
      (match S.conn_of store with
       | None -> ()
       | Some conn ->
           (match
              T.upsert_datum conn ~path ~type_name
```

```
278                ~body_json:"{\"title\":\"kind\",\"body\":\"agnostic\"}"
     ~author:"tester" ~rank:None
279             with
280              | Error e -> Alcotest.fail ("create failed: " ^ e) | Ok _ ->
     ());
281            (* tree/list-kind kind="datum" under the data prefix finds it *)
282            let prefix = "data" in
283            let nodes = T.list_by_kind conn ~prefix ~kind:"datum" ~limit:100
     () in
284            Alcotest.(check bool) "list-kind sees datum" true
285              (List.exists (fun (n : T.node) -> n.path = path) nodes);
286            (* tree/get returns created_at/updated_at keys *)
287            (match T.get_by_path conn ~path with
288             | None -> Alcotest.fail "no node"
289             | Some n ->
290                 let j = T.node_to_json n in
291                 Alcotest.(check bool) "json has created_at" true
292                   (let open Yojson.Safe.Util in
293                    match member "created_at" (Yojson.Safe.from_string j)
     with
294                    | `String s -> s <> ""
295                    | _ -> false);
296                 Alcotest.(check bool) "json has updated_at" true
297                   (let open Yojson.Safe.Util in
298                    match member "updated_at" (Yojson.Safe.from_string j)
     with
299                    | `String s -> s <> ""
300                    | _ -> false))))
301
302 (* ---- moving a datum to a path under a DIFFERENT type re-validates ---- *)
303 let test_move_type_change () =
304   with_store (fun store ->
305       let ta = uniq "ta" in
306       let tb = uniq "tb" in
307       declare_type store ~name:ta ~fields:[ "title"; "body" ];
308       (* tb requires an extra field the ta body lacks -> move must reject *)
309       declare_type store ~name:tb ~fields:[ "title"; "body"; "status" ];
310       let id = "00000000-0000-0000-0000-00000000" ^ "0008" in
311       let old_path = T.datum_path ~user:"tester" ~type_name:ta ~id () in
312       (* same id, different type segment -> a type change *)
313       let new_path = T.datum_path ~user:"tester" ~type_name:tb ~id () in
314       (match S.conn_of store with
315        | None -> ()
316        | Some conn ->
317            (match
318               T.upsert_datum conn ~path:old_path ~type_name:ta
319                 ~body_json:"{\"title\":\"x\",\"body\":\"y\"}"
     ~author:"tester" ~rank:None
320             with
321              | Error e -> Alcotest.fail ("create failed: " ^ e) | Ok _ ->
     ());
322            (* type change to a type requiring 'status' -> reject naming it
     *)
323            (match C.tree_move store ~old_path ~new_path with
324             | VStr s ->
325                 (* clean typed-boundary rejection naming the missing field
     *)
326                 Alcotest.(check bool) "rejects type change naming missing
     field" true
327                   (contains_sub (String.lowercase_ascii s) "missing fields"
328                    && contains_sub (String.lowercase_ascii s) "status")
329             | _ -> Alcotest.fail "type-change move should return a string
     rejection");
330            (* node still at old path (not moved) *)
331            (match T.get_by_path conn ~path:old_path with
332             | None -> Alcotest.fail "node disappeared after rejected move"
333             | Some _ -> ())));
334   (* same-type move remains allowed (already covered) *)
335   ()
336
337 (* ---- moving a datum to a DIFFERENT type that the body satisfies succeeds
     ---- *)
338 let test_move_type_change_ok () =
```

```
  with_store (fun store ->
      let ta = uniq "ta" in
      let tb = uniq "tb" in
      declare_type store ~name:ta ~fields:[ "title"; "body"; "status" ];
      declare_type store ~name:tb ~fields:[ "title"; "body"; "status" ];
      let id = "00000000-0000-0000-0000-00000000" ^ "0009" in
      let old_path = T.datum_path ~user:"tester" ~type_name:ta ~id () in
      let new_path = T.datum_path ~user:"tester" ~type_name:tb ~id () in
      (match S.conn_of store with
       | None -> ()
       | Some conn ->
           (match
              T.upsert_datum conn ~path:old_path ~type_name:ta
    ~body_json:"{\"title\":\"x\",\"body\":\"y\",\"status\":\"todo\"}"
    ~author:"tester" ~rank:None
            with
            | Error e -> Alcotest.fail ("create failed: " ^ e) | Ok _ ->
    ());
           (match C.tree_move store ~old_path ~new_path with
            | VInt _ -> ()
            | VStr s -> Alcotest.fail ("move should succeed: " ^ s)
            | _ -> Alcotest.fail "move returned unexpected value")));
  ()

let () =
  Random.self_init ();
  Alcotest.run "tree_datum"
    [ ( "datum-prims",
        [
          Alcotest.test_case "upsert placeholder id" `Quick
    test_upsert_placeholder;
          Alcotest.test_case "upsert full path + projection" `Quick
            test_upsert_full_path;
          Alcotest.test_case "upsert update in place" `Quick
            test_upsert_update_inplace;
          Alcotest.test_case "prim dispatch + reject" `Quick
    test_tree_upsert_prim;
          Alcotest.test_case "move datum" `Quick test_move_datum;
          Alcotest.test_case "move datum type change reject" `Quick
            test_move_type_change;
          Alcotest.test_case "move datum type change ok" `Quick
            test_move_type_change_ok;
          Alcotest.test_case "delete datum" `Quick test_delete_datum;
          Alcotest.test_case "readers kind-agnostic" `Quick
    test_readers_kind_agnostic;
        ] ) ]
```

141 image/test/tree_edges_test.ml

```
(** Tree-edges tests (one-tree.tree-edges).

    An edge IS a node: kind=link at <source-path>.lnk.<uuid>, body
    {to, kind, label?}. "from" is positional (never in the body); to is
    stored verbatim as branch-qualified path text and MAY dangle;
    resolution is a read-time act against the reader's lens. Edges
    inherit the whole tree stack: versioning (gen-read/gen-diff/undo),
    authorship, branch ownership, lens visibility. tree/delete and
    tree/move report incoming-edge counts (warn, not forbid).

    Needs Postgres (scripts/dev.sh). Skips cleanly when unreachable.

    Acceptance (one-tree.tree-edges verify stanzas, minus the :ref
    materialization stanza -- that half is deferred with the typed-
    boundary :ref work; see BORGE issue on the tree-edges subsection):
      - link a?b: (out-links a) lists b resolved; (backlinks <b-path>)
        includes the edge path.
      - tree/move of the target: the edge reports dangling on read;
        textual backlink still finds it (legacy half).
      - tree/delete of a node with 2 inbound edges reports count 2 and
        succeeds (warn, not forbid).
      - edge nodes are versioned: gen-diff shows edge add; undo of
        unlink restores the edge.
*)

module S = Pricklypear_image_lib.Store
module T = Pricklypear_image_lib.Tree_store
module C = Pricklypear_image_lib.Crud
open Pricklypear_lisp.Ast

let main_id = "00000000-0000-0000-0000-000000000001"

let with_store f =
  Eio_main.run @@ fun env ->
  Lwt_eio.with_event_loop ~clock:(Eio.Stdenv.clock env) @@ fun _ ->
  Eio.Switch.run @@ fun sw ->
  let ctx : Pricklypear_image_lib.Io_eio.Ctx.t =
    {
      sw;
      net = (Eio.Stdenv.net env :> [ `Generic ] Eio.Net.ty Eio.Std.r);
    }
  in
  Pricklypear_image_lib.Io_eio.with_ctx ctx (fun () ->
      let cfg = S.config_from_env () in
      let store = S.make ~cfg ~branch_id:main_id () in
      S.connect store;
      match S.conn_of store with
      | None ->
          Printf.printf
            "SKIP: no postgres (set PP_DB_* / run scripts/dev.sh)\n%!"
      | Some _ -> f store)

let uniq p = Printf.sprintf "te_%s_%d" p (Random.int 1_000_000)

let str_v = function VStr s -> s | _ -> Alcotest.fail "not a VStr"

let hier conn path slug =
  match
    T.upsert_hier conn ~path ~kind:"note" ~type_name:"note"
      ~body_json:(Printf.sprintf "{\"slug\":\"%s\",\"title\":\"%s\"}" slug
     slug)
      ~author:"tester" ~rank:None
  with
  | Ok p -> p
  | Error e -> Alcotest.fail ("upsert_hier failed: " ^ e)

(* JSON field access on the prim JSON strings *)
let jfield (json : string) (key : string) : Yojson.Safe.t option =
  match Yojson.Safe.from_string json with
  | `Assoc kvs -> List.assoc_opt key kvs
  | _ -> None
```

```
71
72  let jstr (json : string) (key : string) : string option =
73    match jfield json key with Some (`String s) -> Some s | _ -> None
74
75  (* ---- spec smoke: link a->b, out-links resolved, backlinks finds it ----
76       *)
77
78  let test_link_out_back () =
79    with_store (fun store ->
80        match S.conn_of store with
81        | None -> ()
82        | Some conn ->
83            let root = "data.bench.te." ^ uniq "a" in
84            let pa = hier conn (root ^ ".a") "tea1" in
85            let pb = hier conn (root ^ ".b") "teb1" in
86            (* link a -> b *)
87            let edge_json =
88              str_v (C.link_edge store ~from_path:pa ~to_path:pb
89                       ~edge_kind:"mentions" ~label:(Some "the b node")
90                       ~author:"tester")
91            in
92            Alcotest.(check bool) "edge is at a.lnk.<uuid>" true
93              (String.length pa + 40 < String.length edge_json
94               && (match jstr edge_json "path" with
95                   | Some p ->
96                       String.starts_with ~prefix:(pa ^ ".lnk.") p
97                   | None -> false));
98            Alcotest.(check (option string)) "edge kind" (Some "mentions")
99              (jstr edge_json "kind");
100           (* out-links a -> b resolved *)
101           (match C.out_links store ~path:pa () with
102            | VList items ->
103                Alcotest.(check int) "one out-link" 1 (List.length items);
104                (match items with
105                 | [ VStr j ] ->
106                     Alcotest.(check (option string)) "status resolved"
107                       (Some "resolved") (jstr j "status");
108                     Alcotest.(check (option string)) "target path" (Some pb)
109                       (match jfield j "target" with
110                        | Some (`Assoc t) ->
111                            (match List.assoc_opt "path" t with
112                             | Some (`String p) -> Some p
113                             | _ -> None)
114                        | _ -> None)
115                 | _ -> Alcotest.fail "bad out-links shape")
116            | _ -> Alcotest.fail "out-links not a list");
117           (* backlinks of b includes the edge path *)
118           let edge_path =
119             match jstr edge_json "path" with Some p -> p | None -> ""
120           in
121           (match C.backlinks store ~needle:pb () with
122            | VList paths ->
123                let ps = List.map str_v paths in
124                Alcotest.(check bool) "backlinks includes edge" true
125                  (List.mem edge_path ps)
126            | _ -> Alcotest.fail "backlinks not a list"))
127
128 (* ---- spec smoke: move of the target dangles; legacy still finds ---- *)
129
130 let test_dangle_after_move () =
131   with_store (fun store ->
132       match S.conn_of store with
133       | None -> ()
134       | Some conn ->
135           let root = "data.bench.te." ^ uniq "b" in
136           let pa = hier conn (root ^ ".a") "tea2" in
137           let pb = hier conn (root ^ ".b") "teb2" in
138           let edge_json =
139             str_v (C.link_edge store ~from_path:pa ~to_path:pb
140                      ~edge_kind:"ref" ~label:None ~author:"tester")
141           in
142           (* textual legacy reference inside a datum-ish note body:
```

```
143           let pc = hier conn (root ^ ".c") "tec2" in
144           ignore
145             (T.upsert_hier conn ~path:pc ~kind:"note" ~type_name:"note"
146                ~body_json:(Printf.sprintf
    "{\"slug\":\"tec2b\",\"ref\":\"%s\"}" pb)
147                ~author:"tester" ~rank:None);
148           (* move the target *)
149           let newb = root ^ ".b2" in
150           (match C.tree_move store ~old_path:pb ~new_path:newb with
151            | VStr j ->
152                Alcotest.(check bool) "move json" true
153                  (String.contains j '{')
154            | _ -> Alcotest.fail "move not json");
155           (* the edge now reports dangling on read *)
156           (match C.out_links store ~path:pa () with
157            | VList [ VStr j ] ->
158                Alcotest.(check (option string)) "dangling after move"
159                  (Some "dangling") (jstr j "status")
160            | _ -> Alcotest.fail "expected one dangling edge");
161           (* legacy textual backlink still finds the reference *)
162           (match C.backlinks store ~needle:newb () with
163            | VList paths ->
164                let ps = List.map str_v paths in
165                Alcotest.(check bool) "legacy half finds textual ref" true
166                  (List.mem pc ps);
167                (* the edge points at the OLD path text -- it does not
168                   follow the move (dangling by design) *)
169                let edge_path =
170                  match jstr edge_json "path" with Some p -> p | None -> ""
171                in
172                Alcotest.(check bool) "edge target text unmoved" true
173                  (not (List.mem edge_path ps))
174            | _ -> Alcotest.fail "backlinks not a list"))
175
176 (* ---- spec smoke: delete with 2 inbound edges reports 2, succeeds ---- *)
177
178 let test_delete_reports_inbound () =
179   with_store (fun store ->
180       match S.conn_of store with
181       | None -> ()
182       | Some conn ->
183           let root = "data.bench.te." ^ uniq "c" in
184           let pa = hier conn (root ^ ".a") "tea3" in
185           let pb = hier conn (root ^ ".b") "teb3" in
186           let pd = hier conn (root ^ ".d") "ted3" in
187           let _e1 =
188             str_v (C.link_edge store ~from_path:pa ~to_path:pb
189                      ~edge_kind:"ref" ~label:None ~author:"tester")
190           in
191           let _e2 =
192             str_v (C.link_edge store ~from_path:pd ~to_path:pb
193                      ~edge_kind:"ref" ~label:None ~author:"tester")
194           in
195           (* deleting b: 2 inbound edges -- reports count, succeeds *)
196           (match C.tree_delete store ~path:pb ~recursive:false with
197            | VStr j ->
198                (match
199                   jfield j "inbound_edges", jfield j "deleted"
200                 with
201                 | Some (`Int 2), Some (`Int 1) -> ()
202                 | a, b ->
203                     Alcotest.fail
204                       (Printf.sprintf "expected {deleted:1,inbound:2}, got
    %s %s"
205                          (Yojson.Safe.to_string (match a with Some x -> x |
    None -> `Null))
206                          (Yojson.Safe.to_string (match b with Some x -> x |
    None -> `Null))))
207            | _ -> Alcotest.fail "delete did not return JSON");
208           (match T.get_by_path conn ~path:pb with
209            | None -> () (* gone *)
210            | Some _ -> Alcotest.fail "delete did not delete"));
211   ()
```

```
212
213 (* ---- spec smoke: edges versioned; gen-diff; undo of unlink ---- *)
214
215 let test_edge_versioning () =
216   with_store (fun store ->
217       match S.conn_of store with
218       | None -> ()
219       | Some conn ->
220           let root = "data.bench.te." ^ uniq "d" in
221           let pa = hier conn (root ^ ".a") "tea4" in
222           let pb = hier conn (root ^ ".b") "teb4" in
223           let edge_json =
224             str_v (C.link_edge store ~from_path:pa ~to_path:pb
225                      ~edge_kind:"ref" ~label:None ~author:"tester")
226           in
227           let edge_path =
228             match jstr edge_json "path" with Some p -> p | None -> ""
229           in
230           (* versioned: the edge has version rows with its body *)
231           (match T.versioned_head conn ~path:edge_path () with
232            | Some v ->
233                Alcotest.(check bool) "versioned body has to" true
234                  (match v.T.v_body with
235                   | `Assoc kvs -> (
236                       match List.assoc_opt "body" kvs with
237                       | Some (`Assoc b) -> List.mem_assoc "to" b
238                       | _ -> false)
239                   | _ -> false)
240            | None -> Alcotest.fail "edge has no version rows");
241           let gen_before =
242             match T.versioned_head conn ~path:edge_path () with
243             | Some v -> (match v.T.v_gen with Some g -> g | None -> 0L)
244             | None -> 0L
245           in
246           (* unlink: tombstone through the same delete path *)
247           (match C.unlink_edge store ~edge_path with
248            | VStr _ -> ()
249            | _ -> Alcotest.fail "unlink failed");
250           (match T.get_by_path conn ~path:edge_path with
251            | None -> () (* tombstoned *)
252            | Some _ -> Alcotest.fail "unlink did not remove the edge");
253           (* undo of unlink restores the edge *)
254           ignore (C.tree_undo store ~path:edge_path ());
255           (match T.get_by_path conn ~path:edge_path with
256            | Some n ->
257                Alcotest.(check bool) "undo restored the edge" true
258                  (T.is_link_kind n.kind);
259                Alcotest.(check (option string)) "restored to target"
260                  (Some pb) (T.body_string_field n.body "to")
261            | None -> Alcotest.fail "undo of unlink did not restore");
262           (* gen-diff across the unlink shows the tombstone *)
263           (match
264              T.versioned_head conn ~path:edge_path ~at_gen:gen_before ()
265            with
266            | Some _ -> () (* pre-unlink leaf exists *)
267            | None -> Alcotest.fail "pre-unlink version missing"))
268
269 let test_link_validation () =
270   with_store (fun store ->
271       match S.conn_of store with
272       | None -> ()
273       | Some conn ->
274           let root = "data.bench.te." ^ uniq "e" in
275           let pa = hier conn (root ^ ".a") "tea5" in
276           (* from must resolve *)
277           (match
278              C.link_edge store
279                ~from_path:(root ^ ".missing")
280                ~to_path:pa ~edge_kind:"ref" ~label:None ~author:"tester"
281            with
282            | VStr e -> Alcotest.(check bool) "from must resolve" true
    (String.length e > 0)
283            | _ -> Alcotest.fail "link from missing node should error");
```

```
          (* to must be well-formed path text *)
          (match
             C.link_edge store ~from_path:pa
               ~to_path:"not a path" ~edge_kind:"ref" ~label:None
               ~author:"tester"
           with
           | VStr e -> Alcotest.(check bool) "to must be path text" true
    (String.length e > 0)
           | _ -> Alcotest.fail "link with malformed to should error");
          (* body with "from" is rejected by the store validator *)
          (match
             T.upsert_link conn
               ~path:(pa ^ ".lnk.00000000000000000000000000000001")

    ~body_json:"{\"to\":\"data.a.b\",\"from\":\"x\",\"kind\":\"ref\"}"
               ~author:"tester"
           with
           | Error e -> Alcotest.(check bool) "from-in-body rejected" true
    (String.length e > 0)
           | Ok _ -> Alcotest.fail "from-in-body must be rejected");
          (* non-lnk path rejected *)
          (match
             T.upsert_link conn ~path:(pa ^ ".other.x")
               ~body_json:"{\"to\":\"data.a.b\",\"kind\":\"ref\"}"
               ~author:"tester"
           with
           | Error _ -> ()
           | Ok _ -> Alcotest.fail "non-lnk edge path must be rejected"))

let () =
  Alcotest.run "tree_edges"
    [ "edges", [ Alcotest.test_case "link-out-back" `Slow test_link_out_back
               ; Alcotest.test_case "dangle-after-move" `Slow
                   test_dangle_after_move
               ; Alcotest.test_case "delete-reports-inbound" `Slow
                   test_delete_reports_inbound
               ; Alcotest.test_case "edge-versioning" `Slow
    test_edge_versioning
               ; Alcotest.test_case "link-validation" `Slow
    test_link_validation ]
    ]
```

142 image/test/tree__lens__test.ml

```
(** Lens-aware tree read prim tests (one-tree.tree-read-prims,
    tree-store + crud + prims_db lens functions, BORGE-1785516993).

    Needs Postgres (scripts/dev.sh). Skips cleanly when unreachable.

    Acceptance (issue 1785516993):
      - a relative logical path (fn/type/route/pkg suffix) resolves across
        the session's branch chain with closest-ancestor-wins (same as
        function_defs): a child override shadows the parent, and an
        effective definition from system/main is visible on a child.
      - tree/lens-get returns exactly one node (the effective one);
        tree/lens-children/subtree read under the effective branch.
*)

module S = Pricklypear_image_lib.Store
module T = Pricklypear_image_lib.Tree_store

let main_id = "00000000-0000-0000-0000-000000000001"

let with_store f =
  Eio_main.run @@ fun env ->
  Lwt_eio.with_event_loop ~clock:(Eio.Stdenv.clock env) @@ fun _ ->
  Eio.Switch.run @@ fun sw ->
  let ctx : Pricklypear_image_lib.Io_eio.Ctx.t =
    {
      sw;
      net = (Eio.Stdenv.net env :> [ `Generic ] Eio.Net.ty Eio.Std.r);
    }
  in
  Pricklypear_image_lib.Io_eio.with_ctx ctx (fun () ->
      let cfg = S.config_from_env () in
      let store = S.make ~cfg ~branch_id:main_id () in
      S.connect store;
      match S.conn_of store with
      | None ->
          Printf.printf
            "SKIP: no postgres (set PP_DB_* / run scripts/dev.sh)\n%!"
      | Some _ -> f store)

let uniq p = Printf.sprintf "lens_%s_%d" p (Random.int 1_000_000)

let persist store ~branch ~name =
  S.with_branch store ~branch_id:branch (fun () ->
      S.persist_function store name
        (Pricklypear_lisp.Ast.Lit (Pricklypear_lisp.Ast.VInt 1L))
        ~doc:(Some "lens-test") "tester")

(* ---- pure: suffix path materialization ---- *)
let test_lens_suffix_path () =
  let b = "00000000-0000-0000-0000-000000000001" in
  Alcotest.(check (option string)) "fn suffix"
    (Some ("b." ^ T.uuid_label b ^ ".fn." ^ T.enc_label "notes/doc/get"))
    (T.lens_suffix_path ~branch_id:b "fn.notes/doc/get");
  Alcotest.(check (option string)) "route suffix"
    (Some ("b." ^ T.uuid_label b ^ ".route." ^ T.enc_label "/issues"))
    (T.lens_suffix_path ~branch_id:b "route./issues");
  Alcotest.(check (option string)) "pkg suffix"
    (Some ("b." ^ T.uuid_label b ^ ".pkg." ^ T.enc_label "todo"))
    (T.lens_suffix_path ~branch_id:b "pkg.todo");
  Alcotest.(check (option string)) "empty suffix -> None"
    None (T.lens_suffix_path ~branch_id:b "");
  Alcotest.(check (option string)) "kind-only suffix -> None"
    None (T.lens_suffix_path ~branch_id:b "fn")

(* ---- integration: override chain, closest-wins ---- *)
let test_effective_override_chain () =
  with_store (fun store ->
      let main_fn = uniq "mainfn" in
      let child_fn = uniq "childfn" in
      (* define main_fn ONLY on system/main *)
      persist store ~branch:main_id ~name:main_fn;
```

```
72      (* fork a child of main *)
73      match
74        S.fork_branch store ~parent_id:main_id ~owner:"lens_test"
75          ~child_name:(uniq "b")
76      with
77      | None -> Alcotest.fail "fork_branch failed"
78      | Some child_id ->
79          (* child inherits main's main_fn *)
80          S.with_branch store ~branch_id:child_id (fun () ->
81              let chain = S.branch_chain store ~branch_id:child_id in
82              Alcotest.(check bool) "chain is self-first"
83                true (List.hd chain = child_id);
84              Alcotest.(check bool) "chain reaches main"
85                true (List.mem main_id chain);
86              match
87                S.conn_of store
88              with
89              | None -> Alcotest.fail "no conn"
90              | Some conn ->
91                  (match
92                     T.lens_get_in_chain conn
93                       ~suffix:("fn." ^ main_fn)
94                       chain
95                   with
96                   | None -> Alcotest.fail "child must see main's fn"
97                   | Some n ->
98                       Alcotest.(check string) "effective node from main"
99                         (T.uuid_label main_id)
100                        (let p = n.path in
101                         (* b.<uuid>.fn.<name> -> uuid is the 2nd label *)
102                         match List.nth_opt (String.split_on_char '.' p) 1
    with
103                        | Some u -> u
104                        | None -> ""));
105             (* now override child_fn ON the child under a suffix that
106                does NOT exist on main *)
107             ());
108         persist store ~branch:child_id ~name:child_fn;
109         S.with_branch store ~branch_id:child_id (fun () ->
110             match S.conn_of store with
111             | None -> Alcotest.fail "no conn"
112             | Some conn ->
113                 let chain = S.branch_chain store ~branch_id:child_id in
114                 (match
115                    T.lens_get_in_chain conn
116                      ~suffix:("fn." ^ child_fn)
117                      chain
118                  with
119                  | None -> Alcotest.fail "child's own fn visible via lens"
120                  | Some n ->
121                      let p = n.path in
122                      let got =
123                        match List.nth_opt (String.split_on_char '.' p) 1
    with
124                        | Some u -> u
125                        | None -> ""
126                      in
127                      Alcotest.(check string) "own override closest-wins"
128                        (T.uuid_label child_id) got));
129         (* a fn that exists nowhere -> None *)
130         S.with_branch store ~branch_id:child_id (fun () ->
131             match S.conn_of store with
132             | None -> ()
133             | Some conn ->
134                 Alcotest.(check (option string)) "missing -> None"
135                   None
136                   (match
137                      T.lens_get_in_chain conn
138                        ~suffix:("fn." ^ uniq "nope")
139                        (S.branch_chain store ~branch_id:child_id)
140                    with
141                    | None -> None
142                    | Some n -> Some n.path)))
```

```
143
144 (* prims surface: tree/lens-get resolves closest-wins end to end. The prim
145    registration is a thin binding to Crud.tree_lens_get ->
         Store.branch_chain
146    + Tree_store.lens_get_in_chain, so verifying the override resolution at
147    the Tree_store layer (with the same branch_chain the prim uses) covers
148    the effective-chain behaviour. *)
149 let test_lens_get_prim () =
150   with_store (fun store ->
151       let fn = uniq "primfn" in
152       persist store ~branch:main_id ~name:fn;
153       match
154         S.fork_branch store ~parent_id:main_id ~owner:"lens_test"
155           ~child_name:(uniq "b")
156       with
157       | None -> Alcotest.fail "fork_branch failed"
158       | Some child_id ->
159           (* effective override: child re-defines fn to a marker body *)
160           S.with_branch store ~branch_id:child_id (fun () ->
161               S.persist_function store fn
162                 (Pricklypear_lisp.Ast.Lit (Pricklypear_lisp.Ast.VInt 42L))
163                 ~doc:(Some "lens-prim-override") "tester");
164           S.with_branch store ~branch_id:child_id (fun () ->
165               match S.conn_of store with
166               | None -> ()
167               | Some conn ->
168                   let chain = S.branch_chain store ~branch_id:child_id in
169                   (match
170                      T.lens_get_in_chain conn ~suffix:("fn." ^ fn) chain
171                    with
172                    | None -> Alcotest.fail "lens-get must find override"
173                    | Some n ->
174                        let p = n.path in
175                        let got =
176                          match List.nth_opt (String.split_on_char '.' p) 1
     with
177                          | Some u -> u
178                          | None -> ""
179                        in
180                        Alcotest.(check string) "closest-wins child"
181                          (T.uuid_label child_id) got)))
182
183 let () =
184   (* Fresh per-run branch names so repeated runs (or a run right after a
185      previous one) never collide on branches_owner_name_key. Without this,
186      Random's fixed default seed yields identical names every run. *)
187   Random.self_init ();
188   Alcotest.run "tree_lens"
189     [ ( "lens",
190         [
191           Alcotest.test_case "suffix path" `Quick test_lens_suffix_path;
192           Alcotest.test_case "effective override chain" `Quick
193             test_effective_override_chain;
194           Alcotest.test_case "lens-get prim closest-wins" `Quick
195             test_lens_get_prim;
196         ] ) ]
```

143 image/test/tree__path__prims__test.ml

```
(** Unit tests for the kernel path algebra (one-tree.tree-path-prims).

    Pure -- no DB. The five prims (tree/parent, tree/leaf, tree/segments,
    tree/child, tree/ancestors) are backed by Tree_store.path_*; the prim
    glue (Crud.tree_parent &co) is a thin VStr/VList/VNil wrap plus the
    Invalid_argument -> Prim_error conversion, both exercised here. *)

open Alcotest
module T = Pricklypear_image_lib.Tree_store
module C = Pricklypear_image_lib.Crud
open Pricklypear_lisp.Ast

let uuid = "00000000-0000-0000-0000-000000000001"

(* Strips a value down for equality checks. *)
let show_value = function
  | VStr s -> Printf.sprintf "(VStr %S)" s
  | VList _ -> "(VList ...)"
  | VNil -> "VNil"
  | VInt i -> Printf.sprintf "(VInt %Ld)" i
  | VSymbol s -> Printf.sprintf "(VSymbol %S)" s
  | VBool b -> Printf.sprintf "(VBool %b)" b
  | _ -> "(other value)"
let as_str = function VStr s -> s | v -> fail ("not a VStr: " ^ show_value
    v)
let as_strs = function
  | VList vs -> List.map as_str vs
  | _ -> fail "not a VList"
let is_nil = function VNil -> () | v -> fail ("not nil: " ^ show_value v)

(* ---- kernel-level algebra (Tree_store path helpers) ---- *)

let test_parent_leaf_segments () =
  check (option string) "parent of deep" (Some "data.u.alice.todo")
    (T.path_parent "data.u.alice.todo.00000000000000000000000000000001");
  check (option string) "parent of two-label" (Some "data")
    (T.path_parent "data.u");
  check (option string) "parent of root is none" None (T.path_parent
    "data");
  check string "leaf" "00000000000000000000000000000001"
    (T.path_leaf "data.u.alice.todo.00000000000000000000000000000001");
  check string "leaf of root" "data" (T.path_leaf "data");
  check (list string) "segments"
    [ "data"; "u"; "alice"; "todo"; "00000000000000000000000000000001" ]
    (T.path_segments "data.u.alice.todo.00000000000000000000000000000001");
  check (list string) "segments of root" [ "data" ] (T.path_segments
    "data");
  (* uppercase base64url label (a fn name's encoded form) passes through
     leaf/parent UNCHANGED -- the kernel never re-sanitizes stored labels *)
  let p = "b." ^ T.uuid_label uuid ^ ".fn.Z3J1ZXQ1" in
  check string "leaf uppercase label" "Z3J1ZXQ1" (T.path_leaf p);
  check (option string) "parent uppercase label"
    (Some ("b." ^ T.uuid_label uuid ^ ".fn")) (T.path_parent p)

let test_ancestors () =
  check (list string) "strict prefixes root-first"
    [ "data"; "data.u"; "data.u.alice" ]
    (T.path_ancestors "data.u.alice.todo");
  check (list string) "root has none" [] (T.path_ancestors "data");
  check (list string) "two labels" [ "data" ] (T.path_ancestors "data.u")

let test_child_roundtrip () =
  (* spec smoke: (tree/child (tree/parent p) (tree/leaf p)) = p for data/b
     paths -- INCLUDING uppercase base64url labels, which is why an already-
     ltree-safe label must pass through path_child verbatim. *)
  let roundtrip p =
    match T.path_parent p with
    | None -> fail ("root has no parent: " ^ p)
    | Some parent ->
        check string ("roundtrip " ^ p) p (T.path_child parent (T.path_leaf
    p))
```

```
  in
  List.iter roundtrip
    [ "data.u.alice.todo.00000000000000000000000000000001"
    ; "data.shared.bible.verse.00000000000000000000000000000001"
    ; "data.note.u.admin.garden"
    ; "b." ^ T.uuid_label uuid ^ ".fn.Z3JlZXQ1"
    ; "b." ^ T.uuid_label uuid ^ ".route._2Fnotes"
    ; "data.u.alice.todo.my_row_2Fid" ]

let test_child_encoding () =
  (* labels that need sanitizing go through path_label -- identical
     semantics to tree/path-label (dashes, dots, uuid labels, empty) *)
  check string "dash+dot label" "data.u.alice.my_note_md"
    (T.path_child "data.u.alice" "my-note.md");
  check string "spaces collapse" "data.u.alice.a_b"
    (T.path_child "data.u.alice" "a  b");
  check string "empty label -> x" "data.u.alice.x"
    (T.path_child "data.u.alice" "");
  check string "edge underscores trimmed" "data.u.alice.ab"
    (T.path_child "data.u.alice" "-ab-");
  check string "uuid-shaped label verbatim"
    ("data.u.alice." ^ T.uuid_label uuid)
    (T.path_child "data.u.alice" (T.uuid_label uuid));
  check string "safe label verbatim (no re-lowercase)"
    "data.u.alice.MyLabel_2"
    (T.path_child "data.u.alice" "MyLabel_2")

let test_invalid_raise () =
  let raises name f =
    (try
       ignore (f ());
       fail (name ^ ": expected Invalid_argument")
     with Invalid_argument _ -> ())
  in
  raises "empty path" (fun () -> T.path_parent "");
  raises "empty segments" (fun () -> T.path_segments "");
  raises "double dot" (fun () -> T.path_leaf "a..b");
  raises "bad char" (fun () -> T.path_segments "data.u.al-ice");
  raises "trailing dot" (fun () -> T.path_ancestors "data.u.");
  raises "bad label child" (fun () -> T.path_child "bad..path" "x")

(* ---- prim glue (Crud tree wrappers) ---- *)

let test_prim_glue () =
  check string "tree_parent" "data.u.alice"
    (as_str (C.tree_parent "data.u.alice.todo"));
  is_nil (C.tree_parent "data");
  check string "tree_leaf" "todo" (as_str (C.tree_leaf
     "data.u.alice.todo"));
  check (list string) "tree_segments" [ "data"; "u" ]
    (as_strs (C.tree_segments "data.u"));
  check string "tree_child" "data.u.alice.my_note"
    (as_str (C.tree_child ~path:"data.u.alice" ~label:"my-note"));
  check (list string) "tree_ancestors" [ "data" ]
    (as_strs (C.tree_ancestors "data.u"));
  (* invalid path -> Prim_error raise (not a guess) *)
  (try
     ignore (C.tree_parent "bad..path");
     fail "expected Prim_error"
   with Pricklypear_lisp.Ast.Interp_error
          (Pricklypear_lisp.Ast.Prim_error ("tree/parent", _, _)) -> ())

let () =
  run "tree_path_prims"
    [ "algebra", [ test_case "parent-leaf-segments" `Quick
    test_parent_leaf_segments
                 ; test_case "ancestors" `Quick test_ancestors
                 ; test_case "child roundtrip" `Quick test_child_roundtrip
                 ; test_case "child encoding" `Quick test_child_encoding
                 ; test_case "invalid raises" `Quick test_invalid_raise ]
    ; "prims", [ test_case "glue" `Quick test_prim_glue ]
    ]
```

144 image/test/tree_retention_test.ml

```
(** tree-time.gc (5.3): retention worker surfaces.

    Exercises the retention-policy-as-habitat-data Store functions
    (retention_set_policy / retention_delete_policy / retention_list_policies)
    and the retention worker entry point (retention_run) in their two safe,
    NON-DESTRUCTIVE deterministic forms:

      (a) NO whole-class policy  -> retention keeps everything: retention_run
          returns [] (the worker is never invoked / no-op keep-all gate).
      (b) whole-class policy with keep_windows large enough that the retention
          floor CLAMPS to the base-window boundary (gen 1000) -> the worker's
          early-exit path returns (0,0): nothing beyond the base window is
          retentive yet, so nothing is checkpointed or dropped.

    The DESTRUCTIVE rollup + DROP PARTITION path (checkpoint-then-drop of
    lazy windows entirely below a non-trivial floor) is verified by the
    migration's own function tree_versions_retain against an isolated scratch
    DB (see the 5.3 notes log), NOT here -- against the shared dev PG that
    would drop real partitions. These tests keep the policy rows scoped to a
    throwaway type and delete them after.

    Needs Postgres (scripts/dev.sh); skips cleanly when unreachable.
*)

module S = Pricklypear_image_lib.Store

let main_id = "00000000-0000-0000-0000-000000000001"
let uniq p = Printf.sprintf "gcr_%s_%d_%d" p (Unix.getpid ()) (Random.int 1_000_000_000)

let with_store f =
  Eio_main.run @@ fun env ->
  Lwt_eio.with_event_loop ~clock:(Eio.Stdenv.clock env) @@ fun _ ->
  Eio.Switch.run @@ fun sw ->
  let ctx : Pricklypear_image_lib.Io_eio.Ctx.t =
    { sw; net = (Eio.Stdenv.net env :> [ `Generic ] Eio.Net.ty Eio.Std.r) }
  in
  Pricklypear_image_lib.Io_eio.with_ctx ctx (fun () ->
      let cfg = S.config_from_env () in
      let store = S.make ~cfg ~branch_id:main_id () in
      S.connect store;
      match S.conn_of store with
      | None ->
          Printf.printf "SKIP: no postgres (set PP_DB_* / run scripts/dev.sh)\n%!"
      | Some _ -> f store)

let contains_sub s sub =
  let ls = String.length s and lb = String.length sub in
  let rec go i =
    if i + lb > ls then false
    else if String.sub s i lb = sub then true
    else go (i + 1)
  in
  lb <= ls && (if lb = 0 then true else go 0)

let test_policy_crud () =
  with_store (fun store ->
      match S.conn_of store with
      | None -> ()
      | Some _ ->
          let tn = uniq "ty" in
          (* set a whole-class policy and a per-type policy *)
          ignore (S.retention_set_policy store ~path_class:"data"
                    ~type_name:None ~keep_windows:4L);
          ignore (S.retention_set_policy store ~path_class:"data"
```

```
                    ~type_name:(Some tn) ~keep_windows:2L);
          let has pc tnc kw =
            List.exists
              (fun (pc', tn', kw', _) ->
                pc' = pc && tn' = tnc && kw' = kw)
              (S.retention_list_policies store)
          in
          Alcotest.(check bool) "whole-class data policy listed"
            true (has "data" None 4L);
          Alcotest.(check bool) "per-type data policy listed"
            true (has "data" (Some tn) 2L);
          (* upsert overwrites, not duplicates *)
          ignore (S.retention_set_policy store ~path_class:"data"
                    ~type_name:(Some tn) ~keep_windows:9L);
          let matches =
            List.filter (fun (_, tn', _, _) -> tn' = Some tn)
              (S.retention_list_policies store)
          in
          Alcotest.(check int) "upsert replaces same (path_class,type_name)
     row"
            1 (List.length matches);
          (* delete the per-type + whole-class policies, leaving the DB
     clean *)
          ignore (S.retention_delete_policy store ~path_class:"data"
                    ~type_name:(Some tn));
          ignore (S.retention_delete_policy store ~path_class:"data"
                    ~type_name:None);
          Alcotest.(check bool) "per-type policy deleted"
            false (has "data" (Some tn) 9L);
          Alcotest.(check bool) "whole-class data policy deleted"
            false (has "data" None 4L))

(* No whole-class policy -> retention_run keeps everything (empty). *)
let test_retention_run_no_policy () =
  with_store (fun store ->
      match S.conn_of store with
      | None -> ()
      | Some _ ->
          (* make sure no whole-class policy exists for either class *)
          ignore (S.retention_delete_policy store ~path_class:"code"
                    ~type_name:None);
          ignore (S.retention_delete_policy store ~path_class:"data"
                    ~type_name:None);
          let rs = S.retention_run store ~path_class:None () in
          Alcotest.(check int) "no policy -> nothing to retain (keep
     everything)"
            0 (List.length rs))

(* Whole-class policy with a floor clamped to the base window -> worker runs
   but early-exits (0,0): nothing beyond the base window is retentive yet.
   Deterministic + NON-DESTRUCTIVE against the real dev head gens. *)
let test_retention_run_clamped () =
  with_store (fun store ->
      match S.conn_of store with
      | None -> ()
      | Some _ ->
          let tn = uniq "ty" in
          (* keep 100 windows: any real head below 100k clamps floor to the
             base-window boundary (gen 1000) -> (0,0). *)
          ignore (S.retention_set_policy store ~path_class:"code"
                    ~type_name:None ~keep_windows:100L);
          ignore (S.retention_set_policy store ~path_class:"data"
                    ~type_name:None ~keep_windows:100L);
          (* not a whole-class policy -> must NOT trigger retention for it
     *)
          ignore (S.retention_set_policy store ~path_class:"data"
                    ~type_name:(Some tn) ~keep_windows:1L);
          let rs = S.retention_run store ~path_class:None () in
          (* both whole-class classes reported; both (0,0) clamped no-op *)
          Alcotest.(check int) "both whole-class classes reported"
            2 (List.length rs);
          List.iter
            (fun (pc, d, c) ->
```

```
              Alcotest.(check int64) (Printf.sprintf "%s drops 0" pc)
                0L d;
              Alcotest.(check int64) (Printf.sprintf "%s checkpoints 0" pc)
                0L c)
            rs;
          (* scoped run for one class *)
          let rc = S.retention_run store ~path_class:(Some "code") () in
          Alcotest.(check int) "scoped code run reports 1"
            1 (List.length rc);
          (* clean up *)
          ignore (S.retention_delete_policy store ~path_class:"code"
                    ~type_name:None);
          ignore (S.retention_delete_policy store ~path_class:"data"
                    ~type_name:(Some tn));
          ignore (S.retention_delete_policy store ~path_class:"data"
                    ~type_name:None))

let () =
  Alcotest.run "tree_retention"
    [
      ( "policies",
        [
          Alcotest.test_case "policy set/list/upsert/delete" `Quick
    test_policy_crud;
        ] );
      ( "worker",
        [
          Alcotest.test_case "retention-run keep-everything (no policy)"
    `Quick
            test_retention_run_no_policy;
          Alcotest.test_case "retention-run clamped floor (no drop)" `Quick
            test_retention_run_clamped;
        ] );
    ]
```

145 image/test/tree_sibling_order_test.ml

```
(** Sibling-order tests (one-tree.sibling-order).

    Children get a user-controlled order via the nullable rank column
    (migration 025): canonical base-36 fractional strings compared as raw
    bytes, midpoint allocation, rebalance on exhaustion. Reads order
    rank COLLATE "C" NULLS LAST, path ASC; reorder is an ordinary
    versioned write (gen-read/undo carry order); a lens override of a
    node owns its own rank.

    Needs Postgres (scripts/dev.sh). Skips cleanly when unreachable.

    Acceptance (one-tree.sibling-order verify stanzas):
      - insert three children, move third before first: children-of
        returns the new order.
      - unranked siblings sort after ranked, path-ordered among
        themselves.
      - reorder is versioned: gen-read at old gen shows old order; undo
        restores.
      - a lens override of a node owns its rank; the parent's rank is
        unchanged.
*)

module S = Pricklypear_image_lib.Store
module T = Pricklypear_image_lib.Tree_store
open Pricklypear_lisp.Ast

let main_id = "00000000-0000-0000-0000-000000000001"

let with_store f =
  Eio_main.run @@ fun env ->
  Lwt_eio.with_event_loop ~clock:(Eio.Stdenv.clock env) @@ fun _ ->
  Eio.Switch.run @@ fun sw ->
  let ctx : Pricklypear_image_lib.Io_eio.Ctx.t =
    {
      sw;
      net = (Eio.Stdenv.net env :> [ `Generic ] Eio.Net.ty Eio.Std.r);
    }
  in
  Pricklypear_image_lib.Io_eio.with_ctx ctx (fun () ->
      let cfg = S.config_from_env () in
      let store = S.make ~cfg ~branch_id:main_id () in
      S.connect store;
      match S.conn_of store with
      | None ->
          Printf.printf
            "SKIP: no postgres (set PP_DB_* / run scripts/dev.sh)\n%!"
      | Some _ -> f store)

let uniq p = Printf.sprintf "so_%s_%d" p (Random.int 1_000_000)

let json_field (json : string) (key : string) : string option =
  match Yojson.Safe.from_string json with
  | `Assoc kvs -> (
      match List.assoc_opt key kvs with
      | Some (`String s) -> Some s
      | _ -> None)
  | _ -> None

let json_rank (json : string) : string option = json_field json "rank"

(** paths of children under [prefix] in read order *)
let child_order conn prefix =
  T.list_children conn ~prefix ()
  |> List.map (fun (n : T.node) -> n.path)

let hier conn ?(rank = None) path body author =
  match T.upsert_hier conn ~path ~kind:"note" ~type_name:"note"
     ~body_json:body
          ~author ~rank with
  | Ok p -> p
  | Error e -> Alcotest.fail ("upsert_hier failed: " ^ e)
```

```
71
72  (* ---- midpoint allocation sanity (pure) ---- *)
73
74  let test_rank_midpoint () =
75    Alcotest.(check bool) "mid between" true
76      (let m = T.rank_midpoint "a0" "a2" in
77       String.compare "a0" m < 0 && String.compare m "a2" < 0);
78    Alcotest.(check string) "mid adjacent" "a0h" (T.rank_midpoint "a0" "a1");
79    Alcotest.(check bool) "mid prefix case" true
80      (let m = T.rank_midpoint "a5" "a5b" in
81       String.compare "a5" m < 0 && String.compare m "a5b" < 0);
82    Alcotest.(check bool) "mid deep prefix" true
83      (let m = T.rank_midpoint "a5" "a511" in
84       String.compare "a5" m < 0 && String.compare m "a511" < 0);
85    Alcotest.(check string) "before first" "a" (T.rank_before "" |> fun o ->
       (match o with Some r -> r | None -> "?"));
86    Alcotest.(check bool) "before of first" true
87      (match T.rank_before "a0" with
88       | Some r -> String.compare r "a0" < 0
89       | None -> false);
90    Alcotest.(check bool) "after last" true
91      (let r = T.rank_after "a0" in String.compare r "a0" > 0);
92    (* canonical ranks never end in '0'; midpoint output is canonical *)
93    Alcotest.(check bool) "canonical output" true
94      (let rs =
95         [ T.rank_midpoint "a0" "a2"; T.rank_midpoint "a5" "a5b";
96           T.rank_after "az"; T.rank_after "zzz" ]
97         @ (match T.rank_before "zz" with Some r -> [ r ] | None -> [])
98       in
99       List.for_all T.rank_valid rs);
100   Alcotest.(check bool) "rebalance sequence ordered" true
101     (let rs = T.rank_rebalance 40 in
102      let sorted = List.sort String.compare rs in
103      rs = sorted && List.for_all T.rank_valid rs);
104   (try
105      ignore (T.rank_midpoint "b" "a");
106      Alcotest.fail "midpoint of a>=b must raise"
107    with Invalid_argument _ -> ())
108
109 (* ---- spec smoke: move third before first ---- *)
110
111 let test_move_before () =
112   with_store (fun store ->
113       match S.conn_of store with
114       | None -> ()
115       | Some conn ->
116           let prefix = "data.bench.so." ^ uniq "t1" in
117           let mk i =
118             hier conn ~rank:None (prefix ^ ".c" ^ string_of_int i)
119               (Printf.sprintf "{\"slug\":\"c%d\",\"title\":\"c%d\"}" i i)
120               "tester"
121           in
122           let mk_ranked i =
123             hier conn
124               ~rank:(Some (string_of_int (i + 1)))
125               (prefix ^ ".c" ^ string_of_int i)
126               (Printf.sprintf "{\"slug\":\"c%d\",\"title\":\"c%d\"}" i i)
127               "tester"
128           in
129           let p1 = mk_ranked 1 and p2 = mk_ranked 2 and p3 = mk_ranked 3 in
130           (* ranked: c1, c2, c3 *)
131           Alcotest.(check (list string)) "initial order" [ p1; p2; p3 ]
132             (child_order conn prefix);
133           (* move third before first *)
134           (match
135              Pricklypear_image_lib.Crud.tree_move_before store ~path:p3
136                ~anchor:p1
137            with
138            | VStr r ->
139                (* returns the allocated rank, which must sort before c1's *)
140                Alcotest.(check bool) "new rank before anchor" true
141                  (String.compare r "2" < 0)
142            | _ -> Alcotest.fail "move-before failed");
```

```
          Alcotest.(check (list string)) "moved order" [ p3; p1; p2 ]
            (child_order conn prefix);
          (* a NULL-rank anchor is a clean error, not a guess *)
          (match
             Pricklypear_image_lib.Crud.tree_move_before store ~path:p1
               ~anchor:(mk 9)
           with
           | VStr e when String.length e > 0 && String.length e < 500 ->
               Alcotest.(check bool) "null anchor error mentions rank" true
                 (let found = ref false in
                  String.iter (fun c -> if c = 'r' then found := true) e;
                  !found)
           | VNil -> Alcotest.fail "move-before: nil"
           | _ -> Alcotest.fail "move-before null-anchor should error"))

(* ---- spec smoke: unranked sort after ranked ---- *)

let test_unranked_after_ranked () =
  with_store (fun store ->
      match S.conn_of store with
      | None -> ()
      | Some conn ->
          let prefix = "data.bench.so." ^ uniq "t2" in
          let mk ?rank i =
            hier conn ?rank (prefix ^ ".c" ^ string_of_int i)
              (Printf.sprintf "{\"slug\":\"c%d\",\"title\":\"c%d\"}" i i)
              "tester"
          in
          let p1 = mk ~rank:(Some "a0") 1 in
          let p2 = mk 2 in
          let p3 = mk ~rank:(Some "a1") 3 in
          let p4 = mk 4 in
          (* ranked a0 < a1 first, then unranked path-ordered: c2 < c4 *)
          Alcotest.(check (list string)) "ranked then unranked"
            [ p1; p3; p2; p4 ] (child_order conn prefix))

(* ---- spec smoke: versioned reorder (gen-read + undo) ---- *)

let test_versioned_reorder () =
  with_store (fun store ->
      match S.conn_of store with
      | None -> ()
      | Some conn ->
          let prefix = "data.bench.so." ^ uniq "t3" in
          let p1 =
            hier conn ~rank:(Some "a0") (prefix ^ ".x")
              "{\"slug\":\"x\",\"title\":\"x\"}" "tester"
          in
          let before_gen =
            match T.versioned_head conn ~path:p1 () with
            | Some v ->
                (match v.T.v_gen with
                 | Some g -> g
                 | None -> Alcotest.fail "version row without gen")
            | None -> Alcotest.fail "no version row after upsert"
          in
          (* reorder: rank change is a real, versioned write *)
          ignore (T.set_rank conn ~path:p1 ~rank:(Some "a5"));
          (match T.versioned_head conn ~path:p1 () with
           | Some v ->
               Alcotest.(check (option string)) "head rank advanced" (Some
    "a5")
                 v.T.v_rank
           | None -> Alcotest.fail "no head version after reorder");
          (* gen-read at the pre-reorder gen shows the OLD rank *)
          (match T.versioned_head conn ~path:p1 ~at_gen:before_gen () with
           | Some v ->
               Alcotest.(check (option string)) "old gen shows old rank"
                 (Some "a0") v.T.v_rank
           | None -> Alcotest.fail "no version at old gen");
          (* undo restores the old rank exactly *)
          let undo_json =
            let v =
```

```
              Pricklypear_image_lib.Crud.tree_undo store ~path:p1 ()
            in
            match v with VStr s -> s | _ -> Alcotest.fail "undo failed"
          in
          Alcotest.(check bool) "undo restored old rank" true
            (json_rank undo_json = Some "a0"))

(* ---- spec smoke: lens override owns its rank ---- *)

let test_override_owns_rank () =
  with_store (fun store ->
      match S.conn_of store with
      | None -> ()
      | Some conn ->
          (* two branch nodes at the same logical suffix = an override
             pair; each row owns its rank independently *)
          let b1 = main_id in
          let child_id =
            match S.fork_branch store ~parent_id:main_id
                    ~owner:"so_test"
                    ~child_name:("so-" ^ string_of_int (Random.int
    1_000_000))
            with
            | Some id -> id
            | None -> Alcotest.fail "fork failed"
          in
          let name = uniq "fn" in
          let p_main = T.fn_path ~branch_id:b1 ~name in
          let p_child = T.fn_path ~branch_id:child_id ~name in
          ignore (T.upsert_fn conn ~branch_id:b1 ~name
                    ~body_json:"{}" ~author:"tester" ~doc:None);
          ignore (T.upsert_fn conn ~branch_id:child_id ~name
                    ~body_json:"{}" ~author:"tester" ~doc:None);
          (* the override (child branch) gets a rank; the parent's is NULL
    *)
          Alcotest.(check int) "set rank on override" 1
            (T.set_rank conn ~path:p_child ~rank:(Some "a0"));
          (match T.get_by_path conn ~path:p_main with
           | Some n -> Alcotest.(check (option string)) "parent rank
    unchanged"
                         None n.rank
           | None -> Alcotest.fail "main node missing");
          (match T.get_by_path conn ~path:p_child with
           | Some n -> Alcotest.(check (option string)) "override owns rank"
                         (Some "a0") n.rank
           | None -> Alcotest.fail "child node missing");
          (* rank on the parent branch does not leak to the child either *)
          Alcotest.(check int) "set rank on parent" 1
            (T.set_rank conn ~path:p_main ~rank:(Some "a1"));
          (match T.get_by_path conn ~path:p_child with
           | Some n -> Alcotest.(check (option string)) "child rank
    untouched"
                         (Some "a0") n.rank
           | None -> Alcotest.fail "child node missing"))

(* ---- insert-child + next/prev/siblings ---- *)

let test_insert_child_and_neighbors () =
  with_store (fun store ->
      match S.conn_of store with
      | None -> ()
      | Some conn ->
          let prefix = "data.bench.so." ^ uniq "t4" in
          ignore
            (hier conn ~rank:None prefix
               "{\"slug\":\"root\",\"title\":\"root\"}" "tester");
          let c1 =
            let v =
              Pricklypear_image_lib.Crud.tree_insert_child store
    ~parent:prefix
                ~kind:"note" ~type_name:"note"
                ~body_json:"{\"slug\":\"a\",\"title\":\"a\"}"
                ~anchor:None ~author:"tester"
```

```
283             in
284             match v with
285             | VStr j -> (match json_field j "path" with Some p -> p | None
    -> Alcotest.fail "no path")
286             | _ -> Alcotest.fail "insert-child failed"
287           in
288           let c2 =
289             let v =
290               Pricklypear_image_lib.Crud.tree_insert_child store
    ~parent:prefix
291                 ~kind:"note" ~type_name:"note"
292                 ~body_json:"{\"slug\":\"b\",\"title\":\"b\"}"
293                 ~anchor:(Some c1) ~author:"tester"
294             in
295             match v with
296             | VStr j -> (match json_field j "path" with Some p -> p | None
    -> Alcotest.fail "no path")
297             | _ -> Alcotest.fail "insert-child after anchor failed"
298           in
299           Alcotest.(check (list string)) "insert order" [ c1; c2 ]
300             (child_order conn prefix);
301           (* move-after with an unranked anchor errors cleanly *)
302           (match
303              Pricklypear_image_lib.Crud.tree_move_after store ~path:c2
304                ~anchor:(prefix ^ ".zzz")
305            with
306            | VStr e ->
307                Alcotest.(check bool) "missing anchor error" true
308                  (String.length e > 0)
309            | _ -> Alcotest.fail "move-after missing anchor should error");
310           (* next/prev walk the total order *)
311           let next_of p =
312             match Pricklypear_image_lib.Crud.tree_next store ~path:p with
313             | VStr j -> json_field j "path"
314             | VNil -> None
315             | _ -> Alcotest.fail "next bad shape"
316           in
317           let prev_of p =
318             match Pricklypear_image_lib.Crud.tree_prev store ~path:p with
319             | VStr j -> json_field j "path"
320             | VNil -> None
321             | _ -> Alcotest.fail "prev bad shape"
322           in
323           Alcotest.(check (option string)) "next of c1" (Some c2) (next_of
    c1);
324           Alcotest.(check (option string)) "next of c2" None (next_of c2);
325           Alcotest.(check (option string)) "prev of c2" (Some c1) (prev_of
    c2);
326           Alcotest.(check (option string)) "prev of c1" None (prev_of c1);
327           (* siblings returns the ordered child set *)
328           (match Pricklypear_image_lib.Crud.tree_siblings store ~path:c1
    with
329            | VList vs ->
330                let paths =
331                  List.filter_map
332                    (function VStr j -> json_field j "path" | _ -> None)
333                    vs
334                in
335                Alcotest.(check (list string)) "siblings ordered" [ c1; c2 ]
    paths
336            | _ -> Alcotest.fail "siblings bad shape"))
337
338 let () =
339   Alcotest.run "tree_sibling_order"
340     [ "rank", [ Alcotest.test_case "midpoint" `Quick test_rank_midpoint ]
341     ; "smokes", [ Alcotest.test_case "move-before" `Slow test_move_before
342                 ; Alcotest.test_case "unranked-after-ranked" `Slow
343                     test_unranked_after_ranked
344                 ; Alcotest.test_case "versioned-reorder" `Slow
    test_versioned_reorder
345                 ; Alcotest.test_case "override-owns-rank" `Slow
    test_override_owns_rank
346                 ; Alcotest.test_case "insert-child-neighbors" `Slow
```

```
                    test_insert_child_and_neighbors ]
  ]
```

146 image/test/tree_store_test.ml

```
(** Unit tests for tree_store pure helpers: label codec round-trip,
    path constructors, hashing, envelope. No DB. *)

open Alcotest
module T = Pricklypear_image_lib.Tree_store

let rt s =
  match T.dec_label (T.enc_label s) with
  | Some r -> r
  | None -> fail ("decode failed for: " ^ s)

let test_label_roundtrip () =
  check string "empty" "" (rt "");
  check string "plain" "greet" (rt "greet");
  check string "slash route" "/notes/:id" (rt "/notes/:id");
  check string "dots" "notes/doc/get" (rt "notes/doc/get");
  check string "unicode" "h?llo w?rld" (rt "h?llo w?rld");
  check string "colon+plus" "a:b+c/d" (rt "a:b+c/d");
  check string "spaces" "my type name" (rt "my type name");
  check string "pad2" "ab" (rt "ab");       (* 2 bytes -> 1 pad *)
  check string "pad1" "abc" (rt "abc");     (* 3 bytes -> 0 pad *)
  check string "json" "{\"a\":1,\"b\":[2,3]}" (rt "{\"a\":1,\"b\":[2,3]}")

let test_label_ltree_safe () =
  (* Encoded labels must be ltree-safe: [A-Za-z0-9_] and trailing pad digit
     *)
  let check_safe s =
    let e = T.enc_label s in
    String.iter
      (fun c ->
        let ok =
          (c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z')
          || (c >= '0' && c <= '9') || c = '_'
        in
        if not ok then fail (Printf.sprintf "unsafe char %c in %s (from
     %s)" c e s))
      e
  in
  List.iter check_safe [ ""; "greet"; "/notes/:id"; "notes/doc/get";
                         "h?llo"; "a:b+c/d"; "my type" ]

let test_uuid_label () =
  check string "strips dashes" "00000000000000000000000000000001"
    (T.uuid_label "00000000-0000-0000-0000-000000000001")

let test_paths () =
  let b = "00000000-0000-0000-0000-000000000001" in
  check string "fn path"
    ("b." ^ T.uuid_label b ^ ".fn." ^ T.enc_label "greet")
    (T.fn_path ~branch_id:b ~name:"greet");
  check string "type path"
    ("b." ^ T.uuid_label b ^ ".type." ^ T.enc_label "todo")
    (T.type_path ~branch_id:b ~name:"todo");
  check string "route path"
    ("b." ^ T.uuid_label b ^ ".route." ^ T.enc_label "/notes/:id")
    (T.route_path ~branch_id:b ~route:"/notes/:id");
  check string "pkg path"
    ("b." ^ T.uuid_label b ^ ".pkg." ^ T.enc_label "todo")
    (T.pkg_path ~branch_id:b ~name:"todo");
  check string "datum path user"
    ("data.u." ^ T.enc_label "alice" ^ "." ^ T.enc_label "todo" ^ "." ^
     T.uuid_label b)
    (T.datum_path ~user:"alice" ~type_name:"todo" ~id:b ());
  check string "datum path seed"
    ("data.shared." ^ T.enc_label "bible-verse" ^ "." ^ T.uuid_label b)
    (T.datum_path ~user:"alice" ~type_name:"bible-verse" ~id:b ());
  check string "guestbook is shared"
    ("data.shared." ^ T.enc_label "guestbook_entry" ^ "." ^ T.uuid_label b)
    (T.datum_path ~user:"public" ~type_name:"guestbook_entry" ~id:b ());
  (* paths contain no raw dots/slashes beyond the ltree separators *)
  let p = T.fn_path ~branch_id:b ~name:"notes/doc/get" in
```

```
  check bool "fn path has exactly 3 separators" true
    (String.length (String.concat "" (String.split_on_char '.' p)) >= 0
     && List.length (String.split_on_char '.' p) = 4)

let test_hash () =
  check string "sha256(abc)"
    "ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad"
    (T.hash_body "abc")

let test_envelope () =
  let env =
    T.envelope ~kind:"fn" ~path:"b.x.fn.y" ~type_:"fn"
      ~body:(`String "(lambda ())") ~author:"admin" ~doc:(Some "d")
  in
  match env with
  | `Assoc kvs ->
      check (option string) "kind" (Some "fn")
        (match List.assoc_opt "kind" kvs with
         | Some (`String s) -> Some s | _ -> None);
      check (option string) "doc" (Some "d")
        (match List.assoc_opt "doc" kvs with
         | Some (`String s) -> Some s | _ -> None);
      check bool "has hash" true (List.mem_assoc "hash" kvs)
  | _ -> fail "envelope not assoc"

let test_validate () =
  let open T in
  (match validate_fields ~declared:["a";"b"] "{\"a\":1,\"b\":2}" with
   | Ok () -> ()
   | Error e -> fail e);
  (match validate_fields ~declared:["a";"b"] "{\"a\":1}" with
   | Error msg ->
       check bool "mentions missing" true (String.contains msg 'b' ||
     String.length msg > 0)
   | Ok () -> fail "should reject missing b");
  (match validate_fields ~declared:["a"] "{\"a\":1,\"extra\":true}" with
   | Ok () -> ()  (* extra keys allowed *)
   | Error e -> fail e);
  (match validate_fields ~declared:[] "{}" with
   | Ok () -> ()
   | Error e -> fail e)

let test_path_label () =
  check string "plain" "john" (T.path_label "john");
  check string "hyphen to us" "my_note" (T.path_label "my-note");
  check string "upper" "genesis" (T.path_label "Genesis");
  check string "spaces" "a_b" (T.path_label "a  b");
  check string "empty" "x" (T.path_label "");
  check string "trim edge us" "ab" (T.path_label "-ab-");
  check bool "ltree safe" true (T.valid_path
     "data.note.u.admin.garden.wiki");
  check bool "hyphen invalid path" false (T.valid_path "data.note.my-note");
  check string "garden root" "data.note.u.admin" (T.garden_root
     ~user:"admin");
  check bool "hier kinds" true
    (T.is_hier_kind "folder" && T.is_hier_kind "note" && T.is_hier_kind
     "heading");
  check bool "datum not hier" false (T.is_hier_kind "datum")

let () =
  run "tree_store"
    [ "codec", [ test_case "roundtrip" `Quick test_label_roundtrip
               ; test_case "ltree-safe" `Quick test_label_ltree_safe
               ; test_case "uuid label" `Quick test_uuid_label ]
    ; "paths", [ test_case "constructors" `Quick test_paths
               ; test_case "path_label" `Quick test_path_label ]
    ; "envelope", [ test_case "hash" `Quick test_hash
                  ; test_case "build" `Quick test_envelope ]
    ; "typed", [ test_case "validate_fields" `Quick test_validate ]
    ]
```

147 image/test/tree__version__shadow__test.ml

```
(** tree-time Phase 1 dual-write shadow tests (tree-time.versioned-leaves).

    Every tree_nodes write is shadowed into tree_versions by the
    AFTER-trigger (migration 019 tree_version_append_trg). These tests
    exercise that shadow through the real store write paths and the
    versioned read surface added to Tree_store (versioned_head /
    versioned_history), then clean up after themselves:

      - datum create -> one version row; body/hash equal the live node;
        versioned_head (with no at_gen) == live tree_nodes head.
      - datum update -> the SAME (path, gen) chain grows (append); gen
        strictly increases; versioned_head returns the newest (the body
        that is live), and versioned_head ~at_gen:oldest returns the
        original body (an as-of-gen read).
      - datum delete -> a TOMBSTONE version row is appended at a higher
        gen (a row, never a DELETE); versioned_head reports the tombstone.
      - code-path fn write -> a version row under the b.<branch>.fn.*
        path with the branch's branch_generations bumped (not the data
        tree_generations counter).

    Needs Postgres (scripts/dev.sh). Skips cleanly when unreachable.
*)

module S = Pricklypear_image_lib.Store
module T = Pricklypear_image_lib.Tree_store
open Pricklypear_lisp.Ast

let main_id = "00000000-0000-0000-0000-000000000001"

let with_store f =
  Eio_main.run @@ fun env ->
  Lwt_eio.with_event_loop ~clock:(Eio.Stdenv.clock env) @@ fun _ ->
  Eio.Switch.run @@ fun sw ->
  let ctx : Pricklypear_image_lib.Io_eio.Ctx.t =
    {
      sw;
      net = (Eio.Stdenv.net env :> [ `Generic ] Eio.Net.ty Eio.Std.r);
    }
  in
  Pricklypear_image_lib.Io_eio.with_ctx ctx (fun () ->
      let cfg = S.config_from_env () in
      let store = S.make ~cfg ~branch_id:main_id () in
      S.connect store;
      match S.conn_of store with
      | None ->
          Printf.printf
            "SKIP: no postgres (set PP_DB_* / run scripts/dev.sh)\n%!"
      | Some _ -> f store)

let uniq p = Printf.sprintf "tv_%s_%d" p (Random.int 1_000_000)

(** Last dot-separated label of a datum path = the datum id (uuid). *)
let path_id (path : string) : string =
  match List.rev (String.split_on_char '.' path) with
  | l :: _ -> l
  | [] -> path

let declare_type (store : S.t) ~name ~fields =
  S.persist_type store ~name ~fields ~author:"tester"

(** Body string field of a version row [{"body":{...}}]. *)
let body_field (v : T.version) (key : string) : string option =
  match v.T.v_body with
  | `Assoc kv ->
      (match List.assoc_opt "body" kv with
       | Some (`Assoc bodykv) ->
           (match List.assoc_opt key bodykv with
            | Some (`String s) -> Some s
            | _ -> None)
       | _ -> None)
  | _ -> None
```

```
72
73  (** The canonical body value of a version row (the {body:...} inside the
74      envelope), as a JSON value -- directly comparable to the live node's
75      `body` field (which `node_of_row` sets to payload->'body'). *)
76  let version_body (v : T.version) : Yojson.Safe.t option =
77    match v.T.v_body with
78    | `Assoc kv -> List.assoc_opt "body" kv
79    | _ -> None
80
81  (** Assert the versioned-head body equals the live tree_nodes body.
82      NOTE: we compare CONTENT (body), not `hash`. The store's upsert hashes
83      the raw caller-supplied body string (sha256 over {"title":"a",...})
84      while the one-tree mirror bounce (tree_from_nodes -> data_rows ->
85      tree_mirror_data_rows) recomputes the derived `hash` over the canonical
86      jsonb `::text` (reordered keys / normalized spacing). So after a datum
87      or fn echo, tree_nodes.hash legitimately differs from the version row's
88      captured hash even though the content body is byte-identical. The
89      versioned-leaves head-read invariant is about CONTENT, not the derived
90      hash (which is a consistency artifact, per migration 020's own guard
91      comment). *)
92  let check_head_body_equals_live (v : T.version) (n : T.node) : unit =
93    match version_body v, n.T.body with
94    | Some vb, lb ->
95        Alcotest.(check bool) "head body equals live body" true
96          (Yojson.Safe.equal vb lb)
97    | None, _ -> Alcotest.fail "version body missing"
98
99  (* ---- datum create -> one version row; head read == live node ---- *)
100 let test_datum_create_shadow () =
101   with_store (fun store ->
102       let type_name = uniq "t" in
103       declare_type store ~name:type_name ~fields:[ "title"; "body" ];
104       match S.conn_of store with
105       | None -> ()
106       | Some conn ->
107           let path =
108             match
109               T.upsert_datum conn ~path:"data.u.tester.new" ~type_name
110                 ~body_json:"{\"title\":\"a\",\"body\":\"b\"}"
    ~author:"tester" ~rank:None
111              with
112              | Ok p -> p
113              | Error e -> Alcotest.fail ("create failed: " ^ e)
114            in
115            (* one version row, gen a positive small int, matching live node
    *)
116            let hist = T.versioned_history conn ~path () in
117            Printf.printf "DEBUG create: history=%d%s\n%!"
118              (List.length hist)
119              (String.concat ","
120                 (List.map
121                    (fun v ->
122                      Printf.sprintf "(gen=%s hash=%s tom=%b)"
123                        (match v.T.v_gen with Some g -> Int64.to_string g |
    None -> "-")
124                        (match v.T.v_hash with Some h -> h | None -> "-")
125                        v.T.v_tombstone)
126                    hist));
127            Alcotest.(check int) "one version row after create" 1
128              (List.length hist);
129            (match hist with
130             | v :: _ ->
131                 Alcotest.(check bool) "gen is Some positive" true
132                   (match v.T.v_gen with Some g -> Int64.compare g 0L > 0 |
    None -> false);
133                 Alcotest.(check bool) "not a tombstone" false v.T.v_tombstone
134             | [] -> Alcotest.fail "no version row after create");
135            (* versioned_head == live tree_nodes head (content-wise: the
    mirror
136               bounce recomputes the derived hash, so compare bodies) *)
137            (match T.get_by_path conn ~path with
138             | None -> Alcotest.fail "live node missing"
139             | Some n ->
```

```
140               (match T.versioned_head conn ~path () with
141                | None -> Alcotest.fail "versioned_head missing after
    create"
142                | Some v ->
143                    check_head_body_equals_live v n;
144                    Alcotest.(check bool) "head not tombstone" false
    v.T.v_tombstone));
145          (* clean up: remove the datum + its version rows stay
    (append-only) *)
146          ignore (T.delete_datum conn ~type_name ~id:(path_id path)) )
147
148 (* ---- datum update -> append (gen grows), as-of read returns old body
    ---- *)
149 let test_datum_update_append () =
150   with_store (fun store ->
151       let type_name = uniq "t" in
152       declare_type store ~name:type_name ~fields:[ "title"; "body" ];
153       match S.conn_of store with
154       | None -> ()
155       | Some conn ->
156           let path =
157             match
158               T.upsert_datum conn ~path:"data.u.tester.new" ~type_name
159                 ~body_json:"{\"title\":\"a\",\"body\":\"b\"}"
    ~author:"tester" ~rank:None
160             with
161             | Ok p -> p
162             | Error e -> Alcotest.fail ("create failed: " ^ e)
163           in
164           let g0 =
165             match T.versioned_head conn ~path () with
166             | Some v -> (match v.T.v_gen with Some g -> g | None -> 0L)
167             | None -> 0L
168           in
169           (* update in place *)
170           ignore
171             (T.upsert_datum conn ~path ~type_name
172                ~body_json:"{\"title\":\"a2\",\"body\":\"b2\"}"
    ~author:"tester" ~rank:None);
173           let hist = T.versioned_history conn ~path () in
174           Alcotest.(check int) "two version rows after update" 2
175             (List.length hist);
176           (match T.versioned_head conn ~path () with
177            | None -> Alcotest.fail "head missing after update"
178            | Some v ->
179                Alcotest.(check bool) "head gen strictly greater" true
180                  (match v.T.v_gen with
181                   | Some g -> Int64.compare g g0 > 0
182                   | None -> false);
183                Alcotest.(check bool) "head body has title a2" true
184                  (match body_field v "title" with Some t -> t = "a2" | None
    -> false));
185           (* as-of the OLD gen -> old body *)
186           (match T.versioned_head conn ~path ~at_gen:g0 () with
187            | None -> Alcotest.fail "as-of head missing"
188            | Some v ->
189                Alcotest.(check bool) "as-of title is a" true
190                  (match body_field v "title" with Some t -> t = "a" | None
    -> false));
191           (* clean up *)
192           ignore (T.delete_datum conn ~type_name ~id:(path_id path)) )
193
194 (* ---- datum delete -> TOMBSTONE version row appended (row, never DELETE)
    ---- *)
195 let test_datum_delete_tombstone () =
196   with_store (fun store ->
197       let type_name = uniq "t" in
198       declare_type store ~name:type_name ~fields:[ "title"; "body" ];
199       match S.conn_of store with
200       | None -> ()
201       | Some conn ->
202           let path =
203             match
```

```
              T.upsert_datum conn ~path:"data.u.tester.new" ~type_name
                ~body_json:"{\"title\":\"a\",\"body\":\"b\"}"
    ~author:"tester" ~rank:None
           with
           | Ok p -> p
           | Error e -> Alcotest.fail ("create failed: " ^ e)
         in
         (match T.delete_datum conn ~type_name ~id:(path_id path) with
          | Some _ -> ()
          | None -> ());
         let hist = T.versioned_history conn ~path () in
         (* create + tombstone = 2 rows (delete appends, does not erase) *)
         Alcotest.(check int) "history keeps create + tombstone" 2
           (List.length hist);
         (match T.versioned_head conn ~path () with
          | None -> Alcotest.fail "head missing after delete"
          | Some v ->
              Alcotest.(check bool) "head is a tombstone" true
    v.T.v_tombstone) )

(* ---- code-path fn write -> branch_generations bumped, version row under
    b.* ---- *)
let test_fn_code_path_shadow () =
  with_store (fun store ->
      match S.conn_of store with
      | None -> ()
      | Some conn ->
          let name = uniq "sfn" in
          let path = T.fn_path ~branch_id:main_id ~name in
          T.upsert_fn conn ~branch_id:main_id ~name
            ~body_json:"{\"params\":[],\"body\":1}" ~author:"tester"
    ~doc:None;
          (match T.get_by_path conn ~path with
           | None -> Alcotest.fail "fn node missing"
           | Some n ->
               (match T.versioned_head conn ~path () with
                | None -> Alcotest.fail "fn version row missing"
                | Some v ->
                    check_head_body_equals_live v n;
                    Alcotest.(check bool) "fn head not tombstone" false
    v.T.v_tombstone));
          (* branch_generations for system/main bumped for code paths, while
             the data tree_generations counter is NOT the source for it *)
          (match T.versioned_history conn ~path () with
           | h :: _ ->
               Alcotest.(check bool) "gen present" true
                 (match h.T.v_gen with Some _ -> true | None -> false)
           | [] -> Alcotest.fail "fn history empty");
          (* cleanup *)
          ignore (T.delete_fn conn ~branch_id:main_id ~name) )

let () =
  Alcotest.run "tree_version_shadow"
    [
      ( "shadow",
        [
          Alcotest.test_case "datum create shadows one row" `Quick
            test_datum_create_shadow;
          Alcotest.test_case "datum update appends + as-of read" `Quick
            test_datum_update_append;
          Alcotest.test_case "datum delete appends tombstone" `Quick
            test_datum_delete_tombstone;
          Alcotest.test_case "fn code-path shadow bumps branch gen" `Quick
            test_fn_code_path_shadow;
        ] );
    ]
```

148 libs/__root/lib.pp

```
;; package (root) -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push

(define __t
  (outline/node*
    "root"
    "A"
    ""
    ""
    (list
      (outline/node*
        "c1"
        "C1"
        "b"
        ""
        (list (outline/node* "g" "G" "" "" (list))))
      (outline/node* "c2" "C2" "" "" (list)))))

(define add1
  (lambda (x) "Add 1 to x." (+ x 1)))

(define add1-phase
  (lambda (x) "Add 1 to x." (+ x 1)))

(define apropos
  (lambda
    (pat)
    "Return bindings whose name contains pat."
    (list/filter
      (lambda (n) (string-contains? (as-str n) (as-str pat)))
      (list-bindings))))

(define blank?
  (lambda (s) "Whether the string s is empty." (if (null? s) #t (string-eq
     (as-str s) ""))))

(define deftwo
  (__macro__
    (base)
    (list
      (quote seq)
      (list
        (quote define)
        (string-append base "-a")
        (list (quote lambda) (list) "A"))
      (list
        (quote define)
        (string-append base "-b")
        (list (quote lambda) (list) "B")))))

(define deftwo2
  (__macro__
    (base)
    (list
      (quote seq)
      (list
        (quote define)
        (string-append base "-a")
        (list (quote lambda) (list) "A"))
      (list
        (quote define)
        (string-append base "-b")
        (list (quote lambda) (list) "B")))))

(define deftwo3
  (__macro__
    (base)
    (list
      (quote seq)
      (list
        (quote define)
```

```
        (string-append base "-a")
        (list (quote lambda) (list) (list (quote __str__) "A")))
      (list
        (quote define)
        (string-append base "-b")
        (list (quote lambda) (list) (list (quote __str__) "B"))))))

(define design-patterns-outline
  (outline/node*
    "root"
    "Design Patterns for PP Web Dev"
    "Analysis of current patterns and recommendations based on Paul Graham's Lisp writings and codebase inspection. See [[paul-graham-lisp-notes]] for the research."
    ""
    (list
      (outline/node*
        "s-good"
        "What's Already Good"
        "Patterns that are working well -- don't change these."
        ""
        (list
          (outline/node*
            "s-good-layers"
            "Layered Architecture"
            "The dependency graph is already bottom-up: html/css ? layout ? menu/ui ? app packages. This is Graham's 'series of layers, each acting as a language for the one above.' The rss build (xml ? http ? feed) is the textbook example."
            ""
            (list))
          (outline/node*
            "s-good-outline"
            "Pure Data + Accessors (outline package)"
            "Nodes are plain lists (id title body todo kids); everything accessed via outline/id, outline/title, outline/kids -- never by position. Operations are pure functions returning new trees. Best-structured package in the system."
            ""
            (list))
          (outline/node*
            "s-good-crud"
            "Data-First CRUD"
            "dict-set* + create/update/find-rows -- data as JSON dicts in Postgres, no ORM. Very Lispy."
            ""
            (list))
          (outline/node*
            "s-good-htmx"
            "HTMX Partials"
            "Outline toolbar hx-post to /partials/notes-node-op, hx-target #outline. Todo view swap. Modular, composable, server logic stays in Lisp."
            ""
            (list))
          (outline/node*
            "s-good-row"
            "Row/ID Convention"
            "(row-id row) / (row-fields row) accessors over (id . fields) pairs, consistent across packages."
            ""
            (list))))
      (outline/node*
        "s-recs"
        "Recommendations"
        "Seven patterns to adopt, ordered by impact."
        ""
        (list
          (outline/node*
            "s-rec-1"
            "1. Route/Handler Dispatch Abstraction [HIGH]"
            "Every page handler reinvents GET/POST dispatch. feed/page, notes/page, workout/home-page all write the same if/request-method
```

```
      check. Should be written once. Declare routes with :get :post :htmx
      keys; a generic dispatcher calls the right handler. Graham: 'you
      should never write (if (string-eq (request-method) POST)) more than
      once.'"
131           ""
132           (list)))))))
133
134 (define dict-set*
135   (lambda
136     (json kvs)
137     "Set multiple key/value pairs on a JSON dict string."
138     (if
139       (null? kvs)
140       json
141       (if
142         (null? (cdr kvs))
143         json
144         (dict-set*
145           (dict-set json (car kvs) (car (cdr kvs)))
146           (cdr (cdr kvs)))))))
147
148 (define doc-smoke
149   42)
150
151 (define dp-outline
152   (outline/node*
153     "root"
154     "Design Patterns for PP Web Dev"
155     "Analysis of current patterns and recommendations based on Paul
      Graham's Lisp writings and codebase inspection. See
      [[paul-graham-lisp-notes]] for the research."
156     ""
157     (list
158       (outline/node*
159         "s-good"
160         "What's Already Good"
161         "Patterns that are working well; don't change these."
162         ""
163         (list
164           (outline/node*
165             "s-good-layers"
166             "Layered Architecture"
167             "The dependency graph is already bottom-up: html/css ? layout ?
      menu/ui ? app packages. This is Graham's series of layers, each acting
      as a language for the one above. The rss build (xml ? http ? feed) is
      the textbook example."
168             ""
169             (list))
170           (outline/node*
171             "s-good-outline"
172             "Pure Data + Accessors (outline package)"
173             "Nodes are plain lists (id title body todo kids); everything
      accessed via outline/id, outline/title, outline/kids, never by
      position. Operations are pure functions returning new trees.
      Best-structured package in the system."
174             ""
175             (list))
176           (outline/node*
177             "s-good-crud"
178             "Data-First CRUD"
179             "dict-set* + create/update/find-rows: data as JSON dicts in
      Postgres, no ORM. Very Lispy."
180             ""
181             (list))
182           (outline/node*
183             "s-good-htmx"
184             "HTMX Partials"
185             "Outline toolbar hx-post to /partials/notes-node-op, hx-target
      #outline. Todo view swap. Modular, composable, server logic stays in
      Lisp."
186             ""
187             (list))
188           (outline/node*
```

```
        "s-good-row"
        "Row/ID Convention"
        "(row-id row) / (row-fields row) accessors over (id . fields) pairs, consistent across packages."
        ""
        (list))))
  (outline/node*
    "s-recs"
    "Recommendations"
    "Seven patterns to adopt, ordered by impact."
    ""
    (list
      (outline/node*
        "s-rec-1"
        "1. Route/Handler Dispatch Abstraction [HIGH]"
        "Every page handler reinvents GET/POST dispatch. Should be written once. Declare routes with :get :post :htmx keys; a generic dispatcher calls the right handler. Graham: you should never write (if (string-eq (request-method) POST)) more than once."
        ""
        (list
          (outline/node*
            "s-rec-1a"
            "Current: hand-rolled in every package"
            "(define feed/page (lambda () (if (string-eq (request-method) \"POST\") (feed/handle-post) (feed/get-page)))) -- repeated in notes, workout, weight, capture, plan, flash."
            ""
            (list))
          (outline/node*
            "s-rec-1b"
            "Recommended: data-driven route table"
            "(route \"/feed\" :get feed/get-page :post feed/handle-post :htmx feed/partial) -- the dispatcher handles method routing, HTMX branching, and redirect-after-POST. Pages become declarations, not functions."
            ""
            (list))))
      (outline/node*
        "s-rec-2"
        "2. Action Dispatch Table [HIGH]"
        "POST handlers use deeply nested if/if/if chains. todo/handle-post is 8 levels deep. notes/ui/handle-post uses cond (better but linear). Actions should be a flat table of name ? handler-function pairs. Shorter, extensible, inspectable."
        ""
        (list
          (outline/node*
            "s-rec-2a"
            "Current: nested ifs"
            "(if (string-eq action \"cycle\") ... (if (string-eq action \"delete\") ... (if (string-eq action \"archive\") ...))) -- 8 levels in todo."
            ""
            (list))
          (outline/node*
            "s-rec-2b"
            "Recommended: flat table + generic dispatcher"
            "(define todo/actions (list (list \"cycle\" todo/cycle) (list \"delete\" todo/delete) ...)) then (dispatch todo/actions action). Actions are data -- you can enumerate them, build a help page, add new ones by adding entries."
            ""
            (list))))
      (outline/node*
        "s-rec-3"
        "3. Separate Data Access from HTML Rendering [HIGH]"
        "Many functions do both: feed/feed-html extracts row fields AND renders HTML. workout/session-li queries the DB inside the renderer. weight/chart-svg does math + SVG + min/max + coordinates in one lambda. Lower layers (data) shouldn't know about upper layers (HTML)."
        ""
        (list
```

```
            (outline/node*
              "s-rec-3a"
              "Split each fn into data + presentation"
              "feed/feed-fields: pure data extraction (testable via eval). feed/feed-html: pure HTML builder (takes clean data). Composition at page level: (list/map feed/feed-html (list/map feed/feed-fields (feed/list)))."
              ""
              (list))
            (outline/node*
              "s-rec-3b"
              "Why it matters"
              "When schema changes, you touch only the data fn. When layout changes, you touch only the view fn. The data fn is trivially testable from the eval endpoint. This is Graham's layering: modifying lower layers (data) should be rare and serious; modifying upper layers (templates) should be quick and safe."
              ""
              (list))))
       (outline/node*
         "s-rec-4"
         "4. Form Builder DSL [MEDIUM]"
         "Every form is hand-built with html/form + html/text-input + html/label + html/hidden + html/button. feed/subscribe-form, weight/form, notes/ui/create-form, notes/capture-form, workout/start-bar (40 lines!), flash/rating-form -- all repeat the same boilerplate. Graham: this is a language waiting to be written. Recommended: declare forms as data, (form/make \"/feed\" (list (form/hidden \"action\" \"subscribe\") (form/text \"url\") (form/submit \"Add\")))."
         ""
         (list))
       (outline/node*
         "s-rec-5"
         "5. Resolve capture Package [MEDIUM]"
         "capture/ package (2 bindings) is a demo that creates capture_template rows nothing reads. notes/ already has a full working capture system. Either delete capture/ or wire it up as the template-definition layer that notes/capture consumes."
         ""
         (list))
       (outline/node*
         "s-rec-6"
         "6. Consistent Error Convention [MEDIUM]"
         "feed/subscribe returns \"already-subscribed\" (string). chat/say-in returns body on success or error string. Some fns return () on failure. notes/ui/do-save redirects on error. No way to programmatically distinguish success from failure without string-sniffing. Recommendation: (ok . result) / (err . msg) pairs, or return result on success and () on failure with a last-error side channel."
         ""
         (list))
       (outline/node*
         "s-rec-7"
         "7. Naming Conventions [LOW]"
         "Handler naming is inconsistent: feed/handle-post, notes/ui/handle-post, workout/handle-home-post. View vs data fn naming varies. Recommend: pkg/page (main handler), pkg/handle-post (POST), pkg/view-* (pure HTML), pkg/get-* (pure data), pkg/partial (HTMX)."
         ""
         (list))))
     (outline/node*
       "s-graham"
       "The Graham Principle"
       "Each recommendation is a case of write the language, not the program. The if/request-method check should be written once in a dispatcher, not forty times. The nested-if action dispatch should be a data-driven table. Hand-built forms should be a DSL. Viaweb's editor was 20-25% macros because Graham kept finding these patterns and encoding them into the language itself. The layers (html ? layout ? app) are already there; what's missing is pushing more of the boilerplate down into the language layer."
```

```
        ""
        (list
          (outline/node*
            "s-graham-quote"
            "Key quote from BBN talk"
            "Rtml was our upgrade path... Having that extra layer of
    abstraction in our software gave us a big advantage over competitors.
    It made the design of our software much cleaner... Modifying stuff in
    lower layers (Rtml itself) was a serious matter to be done rarely.
    Modifying the top layers (template code) was something you could do
    quickly."
            ""
            (list))
          (outline/node*
            "s-graham-kw"
            "Keyword parameters lesson"
            "Graham's biggest regret in Rtml: not making every operator
    take keyword parameters. If I wanted to add another dimension to the
    behavior of one of the operators, I could just add a new keyword
    parameter, and everyone's existing templates would continue to work.
    Nopales lacks keyword params, but the opts-dict pattern can serve the
    same purpose."
            ""
            (list)))))))

(define dp-outline-1
  (outline/node*
    "root"
    "Design Patterns for PP Web Dev"
    "Analysis of current patterns and recommendations based on Paul
    Graham's Lisp writings and codebase inspection. See
    [[paul-graham-lisp-notes]] for the research."
    ""
    (list
      (outline/node*
        "s-good"
        "What's Already Good"
        "Patterns that are working well; don't change these."
        ""
        (list
          (outline/node*
            "s-good-layers"
            "Layered Architecture"
            "The dependency graph is already bottom-up: html/css to layout
    to menu/ui to app packages. This is Graham's series of layers, each
    acting as a language for the one above. The rss build (xml to http to
    feed) is the textbook example."
            ""
            (list))
          (outline/node*
            "s-good-outline"
            "Pure Data + Accessors (outline package)"
            "Nodes are plain lists (id title body todo kids); everything
    accessed via outline/id, outline/title, outline/kids, never by
    position. Operations are pure functions returning new trees.
    Best-structured package in the system."
            ""
            (list))
          (outline/node*
            "s-good-crud"
            "Data-First CRUD"
            "dict-set* + create/update/find-rows: data as JSON dicts in
    Postgres, no ORM. Very Lispy."
            ""
            (list))
          (outline/node*
            "s-good-htmx"
            "HTMX Partials"
            "Outline toolbar hx-post to /partials/notes-node-op, hx-target
    #outline. Todo view swap. Modular, composable, server logic stays in
    Lisp."
            ""
            (list))
```

```
          (outline/node*
            "s-good-row"
            "Row/ID Convention"
            "(row-id row) / (row-fields row) accessors over (id . fields) pairs, consistent across packages."
            ""
            (list)))))))

(define dp-outline-2
  (outline/node*
    "root"
    "Design Patterns for PP Web Dev"
    "Analysis of current patterns and recommendations."
    ""
    (list
      (outline/node*
        "s-good"
        "What's Already Good"
        "Working patterns."
        ""
        (list
          (outline/node*
            "s-good-layers"
            "Layered Architecture"
            "Bottom-up dependency graph."
            ""
            (list))))
      (outline/node*
        "s-recs"
        "Recommendations"
        "Seven patterns to adopt, ordered by impact."
        ""
        (list
          (outline/node*
            "s-rec-1"
            "1. Route/Handler Dispatch Abstraction [HIGH]"
            "Every page handler reinvents GET/POST dispatch. Should be written once."
            ""
            (list
              (outline/node*
                "s-rec-1a"
                "Current: hand-rolled in every package"
                "feed/page, notes/page, workout/home-page all write the same if/request-method check."
                ""
                (list))
              (outline/node*
                "s-rec-1b"
                "Recommended: data-driven route table"
                "(route \"/feed\" :get feed/get-page :post feed/handle-post) -- dispatcher handles routing."
                ""
                (list))))
          (outline/node*
            "s-rec-2"
            "2. Action Dispatch Table [HIGH]"
            "POST handlers use deeply nested if/if/if chains. todo/handle-post is 8 levels deep."
            ""
            (list
              (outline/node*
                "s-rec-2a"
                "Current: nested ifs"
                "8 levels of nesting in todo/handle-post."
                ""
                (list))
              (outline/node*
                "s-rec-2b"
                "Recommended: flat table + generic dispatcher"
                "(define todo/actions (list (list \"cycle\" todo/cycle) (list \"delete\" todo/delete))) then (dispatch todo/actions action)."
                ""
```

```
                (list)))))))))

(define e1
  (outline/node*
    "e1"
    "Capability effects"
    "Prickly Pear uses an algebraic-effects-style capability system to
     constrain what an eval call can do. Each eval request carries an
     effects-allow list. Known capabilities: Pure, ReadDb (get,
     find-row/rows, rows, find-text, find-random), WriteDb (create, update,
     update-field, delete), Publish (define, route, public-route), Html (ui
     functions). Dashboard agents get ReadDb+Html but NOT WriteDb or
     Publish -- the compile-time governance model for agent-native BI."
    ""
    (list)))

(define e2
  (outline/node*
    "e2"
    "Budget: 100K step limit"
    "Every eval call runs with a budget of 100000 steps (default_limit,
     eval.ml line 8435). Each eval step (variable lookup, App, Prim
     dispatch, car, cdr, let binding, macro expansion) calls tick() which
     raises Budget_exceeded at limit+1. OCaml built-ins do NOT tick the
     budget -- only the call site counts as 1 step. Each call gets its own
     budget. Budget exceeded aborts only that call; the session continues.
     Budget is a safety net, not a performance governor -- fix via OCaml
     migration, not raising the limit."
    ""
    (list)))

(define e3
  (outline/node*
    "e3"
    "Cooperative stepped evaluation"
    "The eval loop is stepped: each Prim invocation yields back to the
     images event loop. This prevents any single eval from wedging the
     image. Combined with per-call budgeting, it gives cooperative
     multitasking without preemption. No server-side cron exists --
     everything is request-driven with lazy timestamp checks for scheduled
     work."
    ""
    (list)))

(define e4
  (outline/node*
    "e4"
    "select-for-update and mutex poisoning"
    "select-for-update does SELECT FOR UPDATE row lock. Meaningful only
     inside with-transaction. Calling outside a transaction can poison Eio
     mutexes (observed 2026-07-22: Eio mutex Poisoned cascaded into 401
     auth failures for 20 min). Action: make it fail cleanly outside
     transactions. with-transaction form: (with-transaction (lambda ()
     body)). Internal name __with_transaction__ -- OCaml built-in. Nested
     transactions raise clean errors."
    ""
    (list)))

(define e5
  (outline/node*
    "e5"
    "Lisp vs OCaml boundary"
    "All roam/, notes/, wiki/, outline/*, calendar/*, flash/* are Lisp
     (verified: function-source returns non-empty). Even list/map,
     list/filter, list/foldl are Lisp -- every iteration burns budget.
     OCaml built-ins: data layer, agent/LLM, branching, routes, core
     primitives (car, cdr, +, string-eq, etc.), media, ssh. The separation:
     OCaml = data access + core language + IO. Lisp = application logic.
     Budget pain is entirely in the Lisp layer."
    ""
    (list)))

(define e6
```

```
  (outline/node*
    "e6"
    "Migration priorities"
    "P0: list/map, list/filter, list/foldl -- single highest-leverage
     OCaml-ification. Every large-list operation across all namespaces gets
     cheaper. P1: roam/reindex -- the number one budget killer for note
     writes. P2: outline/kids, outline/decode, outline/encode, outline/find
     -- cascade impacts all three namespaces. P3: roam/parse-links,
     roam/backlinks. After P0-P2, 100K budget feels generous while still
     catching infinite loops."
    ""
    (list)))

(define expand-let*
  (lambda
    (bindings body)
    "Expand a let* into nested lets."
    (list/foldl
      (lambda
        (inner binding)
        (list (quote let) (list binding) inner))
      (cons (quote seq) body)
      (reverse bindings))))

(define explode
  (lambda
    (src)
    "Explode source text into formatted s-expression lines."
    (explode-format (explode-ast (read-string src) 0))))

(define explode-args
  (lambda
    (args depth)
    "Explode an argument list into lines at the given depth."
    (if
      (null? args)
      (list)
      (if
        (pair? args)
        (let
          ((item (car args)))
          (if
            (pair? item)
            (append
              (explode-ast item depth)
              (explode-args (cdr args) depth))
            (append
              (list (list depth " " (as-str item)))
              (explode-args (cdr args) depth))))
        (list (list depth " " (as-str args)))))))

(define explode-ast
  (lambda
    (ast depth)
    "Explode an AST node into indented (depth mark text) lines."
    (if
      (null? ast)
      (list (list depth "(" "()") (list depth ")" ""))
      (if
        (pair? ast)
        (append
          (list (list depth "(" (as-str (car ast))))
          (explode-args (cdr ast) (+ depth 1))
          (list (list depth ")" "")))
        (list (list depth " " (as-str ast)))))))

(define explode-format
  (lambda
    (pairs)
    "Format exploded (depth mark text) lines as pipe-delimited text."
    (if
      (null? pairs)
      ""
```

```
      (let
        ((p (car pairs)))
        (string-append
          (as-str (car p))
          "|"
          (car (cdr p))
          "|"
          (car (cdr (cdr p)))
          "\n"
          (explode-format (cdr pairs)))))))

(define factorial
  (lambda (n) "Return n! recursively." (if (<= n 1) 1 (* n (factorial (- n 1))))))

(define filter
  (lambda (p xs) "Keep the elements of xs satisfying pred p." (list/filter p xs)))

(define foldl
  (lambda (f init xs) "Left fold of f over xs with initial init." (list/foldl f init xs)))

(define foldr
  (lambda (f init xs) "Right fold of f over xs with initial init." (list/foldr f init xs)))

(define foo
  (__macro__ (x) x))

(define hdrs
  (dict-set "{}" "If-None-Match" "\"abc\""))

(define hi-page
  (lambda
    ()
    "Render a demo hello-world page."
    (render-html
      "<h1>Hello</h1><p>Your first Pricklypear page. Edit me at /edit/hi-page.</p>")))

(define implode
  (lambda
    (text)
    "Reassemble exploded pipe-delimited text into s-expressions."
    (implode-simple (string-split text "\n") "")))

(define implode-close-all
  (lambda
    (fn-stack)
    "Close every open paren count in the stack."
    (if
      (null? fn-stack)
      ""
      (string-append ")" (implode-close-all (cdr fn-stack))))))

(define implode-close-to
  (lambda
    (fn-stack depth)
    "Close parens down to the given depth, returning text and remaining stack."
    (if
      (null? fn-stack)
      (list "" fn-stack)
      (if
        (>= (car fn-stack) depth)
        (let
          ((rest (implode-close-to (cdr fn-stack) depth)))
          (list (string-append ")" (car rest)) (car (cdr rest))))
        (list "" fn-stack)))))

(define implode-parse-line
  (lambda
```

```
    (line)
    "Parse one exploded line into (depth marker token)."
    (let
      ((parts (string-split line "|")))
      (if
        (null? (cdr parts))
        (list 0 " " "")
        (let
          ((depth (parse-int (car parts)))
            (marker (car (cdr parts)))
            (token
              (if
                (null? (cdr (cdr parts)))
                ""
                (car (cdr (cdr parts))))))
          (list depth marker token))))))

(define implode-simple
  (lambda
    (lines result)
    "Reassemble exploded lines into s-expression source text."
    (if
      (null? lines)
      result
      (let
        ((line (car lines)) (rest (cdr lines)))
        (if
          (or (null? line) (string-eq line ""))
          (implode-simple rest result)
          (let
            ((parsed (implode-parse-line line)))
            (let
              ((marker (car (cdr parsed)))
                (token (car (cdr (cdr parsed)))))
              (if
                (string-eq marker "(")
                (implode-simple
                  rest
                  (string-append
                    result
                    (if (string-eq result "") "" " ")
                    "("
                    token))
                (if
                  (string-eq marker ")")
                  (implode-simple rest (string-append result ")"))
                  (implode-simple rest (string-append result " "
    token)))))))))))

(define inc-smoke
  (lambda (x) "Increment x; smoke test." (+ x 1)))

(define let*
  (__macro__
    (bindings body)
    (if
      (null? bindings)
      body
      (quasiquote
        (let
          ((unquote (car bindings)))
          (let* (unquote (cdr bindings)) (unquote body)))))))

(define let1
  (__macro__
    (name val body)
    (list (quote let) (list (list name val)) body)))

(define let1q
  (__macro__
    (name val body)
    (quasiquote
      (let (((unquote name) (unquote val))) (unquote body)))))
```

```
650
651 (define let1r
652   (__macro__
653     (name val . body)
654     (list
655       (quote let)
656       (list (list name val))
657       (cons (quote seq) body))))
658
659 (define let1r2
660   (__macro__
661     (name val . body)
662     (list
663       (quote let)
664       (list (list name val))
665       (cons (quote seq) body))))
666
667 (define let2
668   (__macro__
669     (bindings . body)
670     (if
671       (null? bindings)
672       (cons (quote seq) body)
673       (list
674         (quote let)
675         (list (car bindings))
676         (cons (quote let2) (cons (cdr bindings) body))))))
677
678 (define list-or-empty
679   (__macro__
680     (xs empty-msg item-fn)
681     (quasiquote
682       (if
683         (null? (unquote xs))
684         (html/p*
685           (html/attr "class" "muted")
686           (unquote empty-msg))
687         (html/ul
688           ""
689           (html/join (list/map (unquote item-fn) (unquote xs))))))))
690
691 (define list-ref
692   nth)
693
694 (define map
695   (lambda (f xs) "Map f over xs." (list/map f xs)))
696
697 (define mywhen-test-1
698   (lambda (t b) "If t then b else ()." (if t b ())))
699
700 (define pg-outline
701   (outline/node*
702     "root"
703     "Paul Graham on Lisp"
704     "Research notes on Paul Grahams writings about Lisp, compiled
      2026-07-29. Sources: paulgraham.com essays (Beating the Averages, BBN
      Talk, Revenge of the Nerds, The Hundred-Year Language, Arc at 3
      Weeks), the On Lisp preface, and Wikipedia on anaphoric macros. See
      also [[design-patterns-pp]] for how these ideas apply to Pricklypear
      development."
705     ""
706     (list
707       (outline/node*
708         "pg-overview"
709         "Overview of Key Writings"
710         "Major works and their themes."
711         ""
712         (list
713           (outline/node*
714             "pg-overview-table"
715             "Key Writings"
716             "On Lisp (1993): advanced Lisp techniques, bottom-up
    programming, first complete macro description. ANSI Common Lisp
```

```
      (1995): intro textbook. Beating the Averages (2001): Viaweb story,
      Lisp as secret weapon, the Blub Paradox. Lisp for Web-Based
      Applications / BBN Talk (2001): specific technical advantages of Lisp
      in Viaweb. Revenge of the Nerds (2002): Lisp 9 ideas, power continuum,
      accumulator example. The Hundred-Year Language (2003): language
      evolution, small clean cores. Arc at 3 Weeks (2001): Arc design
      philosophy, brevity, pronouns. If Lisp is So Great (2001): why Lisp
      isnt more widely adopted."
717           ""
718           (list))))
719       (outline/node*
720         "pg-bta"
721         "Beating the Averages: Viaweb"
722         "The Viaweb story, competitive advantage, the secret weapon."
723         ""
724         (list
725           (outline/node*
726             "pg-bta-story"
727             "The Viaweb Story"
728             "Summer 1995: Graham + Robert Morris founded Viaweb, software
      to let end users build online stores via the web. Arguably the first
      web-based application. Written primarily in Lisp. Had 20-30
      competitors, none could match their software."
729             ""
730             (list))
731           (outline/node*
732             "pg-bta-server"
733             "Server-side = Language Freedom"
734             "When writing software that only runs on your own servers, you
      can use any language you want. Desktop software had a bias toward the
      OS language (C). Web apps changed this: you control the server, you
      have the source to the language and OS, you fix incompatibilities
      yourself. This freedom is a double-edged sword: now you must choose
      wisely."
735             ""
736             (list))
737           (outline/node*
738             "pg-bta-rapid"
739             "Rapid Development as Competitive Weapon"
740             "Lisp was known for writing software quickly. Server apps
      magnify this: release the minute code is done. Graham could sometimes
      duplicate a competitors newly-announced feature within a day or two of
      the press release. By the time journalists called Viaweb, the feature
      already existed."
741             ""
742             (list))
743           (outline/node*
744             "pg-bta-secret"
745             "The Secret Weapon"
746             "Graham kept Lisp usage secret. In business, there is nothing
      more valuable than a technical advantage your competitors dont
      understand. Robert Morris later said it didnt matter: If they were
      that smart theyd already be programming in Lisp."
747             ""
748             (list))
749           (outline/node*
750             "pg-bta-jobs"
751             "Reading Job Listings"
752             "Graham evaluated competitors by job postings. Oracle
      experience = safe. C++/Java = not threatening. Perl/Python = a bit
      frightening (real hackers). Lisp hackers = the only thing that would
      have worried him."
753             ""
754             (list))
755           (outline/node*
756             "pg-bta-results"
757             "Results"
758             "End 1996: ~70 stores. End 1997: 500. Yahoo acquired Viaweb
      1998: 1,070 users. As Yahoo Store: ~20,000 users. Yahoo later rewrote
      in C++/Perl but had to write a Lisp interpreter: the template source
      files are still Lisp code. Cf. Greenspuns Tenth Rule."
759             ""
760             (list))))
```

```
761       (outline/node*
762         "pg-blub"
763         "The Blub Paradox"
764         "The power continuum and why most programmers cant see it."
765         ""
766         (list
767           (outline/node*
768             "pg-blub-continuum"
769             "The Power Continuum"
770             "Programming languages are not equivalent. They fall along a
    continuum (or lattice) of abstractness/power. Turing equivalence is
    irrelevant: the kind of power programmers care about refers to
    features you could only get in the less powerful language by writing
    an interpreter for the more powerful language in it."
771             ""
772             (list))
773           (outline/node*
774             "pg-blub-paradox"
775             "The Paradox Itself"
776             "Hypothetical language Blub sits in the middle. Looking down at
    less powerful languages, the Blub programmer knows they are weaker.
    Looking up at more powerful languages, he doesnt realize hes looking
    up; they just look weird with hairy stuff. He thinks Blub is good
    enough because he thinks in Blub. By induction, only programmers who
    understand the most powerful language can see all the differences in
    power."
777             ""
778             (list))))
779       (outline/node*
780         "pg-nine"
781         "The Nine Ideas"
782         "What made Lisp different in 1958, and which ideas are still
    unique."
783         ""
784         (list
785           (outline/node*
786             "pg-nine-1"
787             "1. Conditionals"
788             "if-then-else. Fortran I only had conditional goto."
789             ""
790             (list))
791           (outline/node*
792             "pg-nine-2"
793             "2. A Function Type"
794             "Functions are data: first-class, stored in variables, passed
    as arguments."
795             ""
796             (list))
797           (outline/node*
798             "pg-nine-3"
799             "3. Recursion"
800             "Lisp was the first language to support it."
801             ""
802             (list))
803           (outline/node*
804             "pg-nine-4"
805             "4. Dynamic Typing"
806             "Variables are pointers; values have types, not variables."
807             ""
808             (list))
809           (outline/node*
810             "pg-nine-5"
811             "5. Garbage Collection"
812             "Introduced ~1960, now widely considered good."
813             ""
814             (list))
815           (outline/node*
816             "pg-nine-6"
817             "6. Programs Composed of Expressions"
818             "Everything returns a value. No statement/expression
    distinction. Fortran introduced this distinction because you couldnt
    nest statements; it spread to Algol and descendants despite no longer
    being necessary."
```

```
819             ""
820             (list))
821           (outline/node*
822             "pg-nine-7"
823             "7. A Symbol Type"
824             "Symbols are pointers to strings in a hash table. Equality by
      pointer comparison."
825             ""
826             (list))
827           (outline/node*
828             "pg-nine-8"
829             "8. Code as Trees of Symbols and Constants"
830             "Code is expressed directly in parse trees (lists). Still
      largely unique to Lisp. Responsible for Lisps strange appearance and
      its most distinctive features (macros)."
831             ""
832             (list))
833           (outline/node*
834             "pg-nine-9"
835             "9. The Whole Language There All the Time"
836             "No real distinction between read-time, compile-time, and
      runtime. You can compile or run code while reading, read or run code
      while compiling, and read or compile code at runtime. Still largely
      unique to Lisp."
837             ""
838             (list))
839           (outline/node*
840             "pg-nine-why"
841             "Why Lisp Isnt Obsolete"
842             "Lisp was not designed as a programming language. It emerged
      from a theoretical exercise by McCarthy (an alternative to the Turing
      Machine). Steve Russell noticed that McCarthys eval function could be
      implemented in machine code, creating a Lisp interpreter. Compare Lisp
      to Quicksort (1960), still the fastest general-purpose sort. Lisp is
      math, not technology. Mainstream languages have been converging toward
      Lisp ever since: Its 2002, and programming languages have almost
      caught up with 1958."
843             ""
844             (list))))
845       (outline/node*
846         "pg-onlisp"
847         "On Lisp: Bottom-Up Programming"
848         "The central thesis, macros, and why you cant just write a library."
849         ""
850         (list
851           (outline/node*
852             "pg-onlisp-thesis"
853             "The Central Thesis"
854             "Instead of just writing your program in Lisp, you can write
      your own language on Lisp, and write your program in that. Bottom-up
      programming: write a program as a series of layers, each acting as a
      programming language for the one above. The lowest layer is the base
      language; each higher layer adds abstractions until the top layer is
      practically a domain-specific language. Examples: X Windows, TeX."
855             ""
856             (list))
857           (outline/node*
858             "pg-onlisp-macros"
859             "Macros: Programs That Write Programs"
860             "A Lisp macro is a function that takes unevaluated code
      (s-expressions) as input and returns transformed code. Because Lisp
      code IS Lisp data (lists), you can manipulate it programmatically. A
      Lisp macro can be anything from an abbreviation to a compiler for a
      new language. Viaweb editor source was 20-25% macros: at least 20-25%
      of the code is doing things that you cant easily do in any other
      language."
861             ""
862             (list))
863           (outline/node*
864             "pg-onlisp-scope"
865             "On Lisp Scope"
866             "Covers: functional programming, closures, functions returning
      functions, macros (tutorial + advanced), variable capture (gensyms),
```

```
      generalized variables, anaphoric macros, read-macros, macro-defining
      macros, continuations, nondeterminism, ATN parsers, an embedded Prolog
      in ~180 lines, bottom-up reconstruction of CLOS. On Lisp deals mostly
      with the kinds of programs you could only write in Lisp."
867           ""
868           (list))
869         (outline/node*
870           "pg-onlisp-library"
871           "Why You Cant Just Write a Library"
872           "Bottom-up programming isnt just about utility functions. Its
      about extending the language itself: adding new operators, new control
      structures, new abstractions that are indistinguishable from built-in
      ones. In most languages you can add libraries; in Lisp you can add
      language features. The dividing line between library and language
      dissolves."
873           ""
874           (list))))
875       (outline/node*
876         "pg-anaphoric"
877         "Anaphoric Macros"
878         "aif, alambda, and the pronoun principle from On Lisp Ch. 14."
879         ""
880         (list
881           (outline/node*
882             "pg-ana-aif"
883             "aif: Anaphoric if"
884             "(defmacro aif (test-form then-form else-form) (let ((it
      test-form)) (if it then-form else-form))). Usage: it is bound to the
      result of the test. (aif (find-user id) (format nil \"Found ~A\" it)
      \"Not found\"). Saves you from writing (let ((x (find-user id))) (if x
      ... ...))."
885           ""
886           (list))
887         (outline/node*
888           "pg-ana-alambda"
889           "alambda: Self-Referencing Lambda"
890           "(defmacro alambda (parms body) (labels ((self parms body))
      self)). Usage: self refers to the lambda itself. (alambda (n) (if (= n
      0) 1 (* n (self (1- n))))). Gives you anonymous recursive functions
      without naming them externally."
891           ""
892           (list))
893         (outline/node*
894           "pg-ana-principle"
895           "The Broader Principle"
896           "Arc took this further with pronouns as a first-class design
      feature: it in if, keep and sum in iteration. If a language lets
      things be implicit, the programmer always has the option of being
      explicit, but if the language requires everything to be explicit, the
      programmer cant make things implicit."
897           ""
898           (list))))
899       (outline/node*
900         "pg-bbn"
901         "BBN Talk: Technical Advantages for Web Apps"
902         "Specific techniques Graham used in Viaweb. Most relevant to
      Pricklypear."
903         ""
904         (list
905           (outline/node*
906             "pg-bbn-lang"
907             "Any Language You Want"
908             "Server-side deployment removes language lock-in. Youre now
      subject to market forces in what language you choose. A little startup
      using a more powerful language will eat your lunch."
909           ""
910           (list))
911         (outline/node*
912           "pg-bbn-incremental"
913           "Incremental Development"
914           "Lisp tradition: start with a tiny program that does almost
      nothing, then add features gradually, always having working code.
      Viaweb editor: began as a 120-line program from a book example. Grew
```

```
      to ~25,000 lines. I never once sat down and rewrote the whole thing. I
      dont think I was ever more than a day or two without running code.
      Fits naturally with rolling releases of web-based software."
915           ""
916           (list))
917         (outline/node*
918           "pg-bbn-toplevel"
919           "Interactive Toplevel: Debugging While Users Are on the Phone"
920           "When a bug report came in, Graham would load the code into the
      Lisp interpreter and log into the users account. If he reproduced the
      bug, hed get a break loop showing exactly what went wrong. Often I
      could fix the code and release a fix right away. And when I say right
      away, I mean while the user was still on the phone."
921           ""
922           (list))
923         (outline/node*
924           "pg-bbn-html"
925           "Macros for HTML"
926           "Lisp macros were another big win. The Viaweb editor could
      accurately be described as one big macro. HTML is prefix notation
      (like Lisp) and recursive (like Lisp). Macro calls within macro calls
      generated the most complex HTML, all manageable. Natural fit: HTML
      tags map to s-expressions."
927           ""
928           (list))
929         (outline/node*
930           "pg-bbn-rtml"
931           "Embedded Languages: Rtml"
932           "Rtml was an embedded DSL (built from Lisp macros) for
      describing web pages. Named after Robert Morris (username Rtm). Every
      page was generated by a program written in Rtml, called templates to
      sound less frightening, but they were real Lisp programs. Users had a
      structure editor (like Interlisp) for manipulating templates:
      cut/paste bits of code, no syntax errors possible, parentheses hidden,
      structure shown by indentation. Designed so no runtime errors were
      possible: every Rtml program yielded some web page. Compiled to Lisp
      functions at runtime via compile."
933           ""
934           (list))
935         (outline/node*
936           "pg-bbn-rtml-roles"
937           "Three Roles Rtml Played"
938           "1. Upgrade path for sophisticated users (1 in ~200 wrote
      templates). Users always want an upgrade path, even though as a rule
      theyll never take it. 2. Warning sign: when users modified default
      templates, it showed what was missing. Goal became no one should ever
      have to use Rtml. 3. Abstraction layer for developers: Having that
      extra layer of abstraction gave us a big advantage over competitors.
      Made design cleaner, code easier to modify. Changing Rtml itself =
      rare and serious; changing template code = quick and low-risk."
939           ""
940           (list))
941         (outline/node*
942           "pg-bbn-rtml-kw"
943           "Keyword Parameters for Safe Evolution"
944           "Adding a new keyword parameter to an Rtml operator didnt break
      existing templates. If I could go back and start over... Id make every
      Rtml operator take keyword parameters. A few operators didnt take
      keyword params because I didnt think Id ever need to change them, and
      almost every one I ended up kicking myself about later."
945           ""
946           (list))
947         (outline/node*
948           "pg-bbn-closures"
949           "Closures Simulate Subroutines (Continuation-Passing Style)"
950           "HTTP is stateless. Solution: use lexical closures to simulate
      subroutine-like behavior across HTTP requests, effectively
      continuation-passing style. A macro stored a closure (code +
      surrounding context) in a global hash table under a unique ID. The
      links URL contained that hash key. When clicked, the corresponding
      code was found and called, carrying its free variables from the
      original context. When our software generated a link, it could think:
      if the user clicks on this link, I want to run this piece of code."
```

```
951             ""
952             (list))
953           (outline/node*
954             "pg-bbn-color"
955             "Concrete Example: Color Picker"
956             "Editing an object with a color property. Competitors (with
      single-form CGI) struggled: text field for RGB numbers (users hate
      it), or a 256-item dropdown menu. Viaweb showed a color swatch +
      Change button, clicking goes to a color imagemap page, choosing a
      color returns to the edit page with that property updated. The
      software could behave as if it were returning from having chosen a
      color, simulating a subroutine call across HTTP. This made our
      software visibly more sophisticated than that of our competitors."
957             ""
958             (list))))
959       (outline/node*
960         "pg-hyl"
961         "The Hundred-Year Language"
962         "Language design philosophy: small cores, brevity, wasting
      programmer time."
963         ""
964         (list
965           (outline/node*
966             "pg-hyl-core"
967             "Small Clean Cores Survive"
968             "The more of a language you can write in itself, the better.
      The fundamental operators are the most important factor in long-term
      survival, like location in real estate: everything else you can fix
      later, but you cant fix the location. Mathematicians have always felt
      this way about axioms: the fewer, the better."
969             ""
970             (list))
971           (outline/node*
972             "pg-hyl-brevity"
973             "Brevity Is the Goal"
974             "Language design algorithm: look at a program and ask, is there
      any way to write this thats shorter? Length in distinct syntactic
      elements/tokens, not characters. What we really like is not
      abstraction per se but brevity. A way of expressing programs that was
      more abstract, but made your programs longer, would not be very
      enticing."
975             ""
976             (list))
977           (outline/node*
978             "pg-hyl-waste"
979             "Wasting Programmer Time Is the True Inefficiency"
980             "Inefficient software isnt gross. What's gross is a language
      that makes programmers do needless work. Wasting programmer time is
      the true inefficiency, not wasting machine time. The hundred-year
      language should let you throw together an unbelievably inefficient
      version 1 of a program with the least possible effort."
981             ""
982             (list))
983           (outline/node*
984             "pg-hyl-strings"
985             "Strings as Lists (Arc Experiment)"
986             "Arc eliminated strings as a separate type. Strings are just
      lists of characters. Some operations that would be awkward to describe
      as regular expressions can be described easily as recursive functions.
      Having strings in a language seems to be a case of premature
      optimization."
987             ""
988             (list))
989           (outline/node*
990             "pg-hyl-reuse"
991             "Bottom-Up = Reusability"
992             "Bottom-up programming means writing a program as a series of
      layers, each of which serves as a language for the one above. This
      approach tends to yield smaller, more flexible programs. Its also the
      best route to that holy grail, reusability. What makes OO code
      reusable is its bottom-upness, not its object-orientedness. Consider
      libraries: theyre reusable because theyre language, whether theyre
      written in an OO style or not."
```

```
            ""
            (list))))
      (outline/node*
        "pg-arc"
        "Arc: A New Lisp Dialect"
        "Design philosophy, brevity, pronouns, syntax as abbreviation."
        ""
        (list
          (outline/node*
            "pg-arc-philosophy"
            "A Language for Good Programmers"
            "Opposite of Javas design goal. Once you assume the user is a good programmer, you automatically have the answer to any such question: let the user do whatever he wants. The programmable programming language (Foderaro). Arc aims to maximize this: assume that the programmer will want to do things we never even imagined, and so will need to be able to get his hands on everything."
            ""
            (list))
          (outline/node*
            "pg-arc-brevity"
            "Brevity: Fewer Tokens, Not Characters"
            "Not fewer characters: fewer tokens. By brevity I dont mean that programs should require fewer characters. That counts for something, but it is more important to require fewer tokens. Perl is an inspiring example of brevity. Larry Wall broke all the rules, and in the process discovered some good ideas."
            ""
            (list))
          (outline/node*
            "pg-arc-onions"
            "No Onions in the Varnish"
            "Everything is open to question. Even lambda was renamed to fn. Alonzo Church himself wouldnt have used it if he had to write out the word lambda each time. No feature is sacred just because its always been that way."
            ""
            (list))
          (outline/node*
            "pg-arc-dialects"
            "Lisp = Dialects"
            "Lisp is an unusual language because it has dialects. Any language with car, cdr, cons, quote, eq, cond, and a notation for functions made of conses IS a dialect of Lisp, not just by social convention, but formally: with the core operators you can define all the rest. Its about time for a new dialect of Lisp."
            ""
            (list))
          (outline/node*
            "pg-arc-syntax"
            "Syntax as Abbreviation"
            "Arc would have syntax, but only as abbreviation: translating in a clearly defined, redefinable way into underlying s-expressions. Nearly all syntax optional, at the level of individual operators. x.y for (x y), x:y for (x y), [+ _ 1] for (fn (x) (+ x 1)). Lisp is widely considered to be the best substrate for domain-specific little languages, and programmable syntax should help make it even better."
            ""
            (list))
          (outline/node*
            "pg-arc-macros"
            "Macros as First-Class Objects"
            "Arc macros are first-class objects. No macrolet needed: bind a macro with let like any other variable. No hygienic macros required (though they could be a library). As an expert, the user will not be thrown by the prospect of variable capture (indeed will often do it deliberately). Macros are going to be a focus in Arc, because we think theyre one of the biggest wins in Lisp."
            ""
            (list))
          (outline/node*
            "pg-arc-web"
            "Built for Web Apps"
```

```
        "Were going to use Arc to write a platform for Web-based
    applications: the language we wish wed had when we were writing
    Viaweb. This means that Arc will have to be good at manipulating
    strings, which has not in the past been a big concern for Lisp."
        ""
        (list))
      (outline/node*
        "pg-arc-pronouns"
        "Iteration with Accumulation (Pronouns)"
        "while captures it (result of test). All iteration operators
    leave keep and sum bound to accumulation functions. (each x (list al
    bob joe) (if (> (len x) 2) (keep x))) returns (bob joe). (to x 5 (sum
    x) (pr x)) prints 01234, returns 10. If a language lets things be
    implicit, the programmer always has the option of being explicit."
        ""
        (list))))
  (outline/node*
    "pg-acc"
    "The Accumulator Example"
    "Same function in Lisp, Perl, JS, Python, Java. Power is the
    ultimate elegance."
    ""
    (list
      (outline/node*
        "pg-acc-lisp"
        "Common Lisp"
        "(defun foo (n) (lambda (i) (incf n i)))"
        ""
        (list))
      (outline/node*
        "pg-acc-perl"
        "Perl 5"
        "sub foo { my ($n) = @_; sub {$n += shift} }. Slightly longer:
    manual parameter extraction."
        ""
        (list))
      (outline/node*
        "pg-acc-js"
        "JavaScript"
        "function foo(n) { return function(i) { return n += i } }.
    Slightly longer: statement/expression distinction forces explicit
    returns."
        ""
        (list))
      (outline/node*
        "pg-acc-python"
        "Python (pre-nonlocal)"
        "def foo(n): s = [n]; def bar(i): s[0] += i; return s[0];
    return bar. Much longer: cant assign to enclosing scope variables;
    must use a mutable container."
        ""
        (list))
      (outline/node*
        "pg-acc-java"
        "Java"
        "Need an interface + anonymous class. No closures, no function
    type. Far longer, and only works for integers, not polymorphic."
        ""
        (list))
      (outline/node*
        "pg-acc-insight"
        "Key Insight"
        "The Lisp version is shortest because it has all the features
    needed (closures, mutable lexical variables, functions as values, no
    statement/expression distinction). Each less-powerful language forces
    the programmer to manually simulate the missing feature. Power is the
    ultimate elegance."
        ""
        (list))))
  (outline/node*
    "pg-pp"
    "Connections to Pricklypear and Nopales"
```

```
1094        "How Grahams ideas map to PP as a web framework and Nopales as a
      Lisp dialect."
1095        ""
1096        (list
1097          (outline/node*
1098            "pg-pp-parallel"
1099            "Viaweb to Pricklypear Parallels"
1100            "Server-side app, Lisp on the server = server-side Lisp
      habitat, image is the API. Interactive toplevel for debugging = POST
      /api/eval, live eval of Lisp at runtime. Bug fixes while users on the
      phone = deploy via pp-deploy.sh, immediate. Incremental development
      (120 to 25k lines) = build libraries incrementally (rss, workout,
      etc.). Macros for HTML generation = can define route handlers and page
      generators in Lisp. Rtml embedded DSL for pages = routes + handlers as
      embedded mini-language. Closures for stateless HTTP = same techniques
      available in Nopales. Server-side = freedom to use Lisp = PP IS Lisp
      all the way down."
1101            ""
1102            (list))
1103          (outline/node*
1104            "pg-pp-nopales"
1105            "Nopales as a Lisp Dialect"
1106            "Nopales fits Grahams definition of a dialect of Lisp: it has
      the core operators (cons, car, cdr, quote, cond, eq, lambda/fn) and
      builds the rest from there. This means it inherits all of Grahams
      arguments about bottom-up programming, embedded languages, macros, the
      programmable programming language, and incremental development."
1107            ""
1108            (list))
1109          (outline/node*
1110            "pg-pp-techniques"
1111            "Specific Techniques Applicable to PP"
1112            "1. HTML generation via macros (div :class store (h1 Welcome)
      instead of string concat). 2. Anaphoric patterns (aif: test result
      bound to it). 3. Embedded DSLs (query DSL for find-rows, form-handling
      DSL, template language). 4. Closures for stateful web interactions
      (continuation-passing / closure-hash for multi-step wizards). 5.
      Keyword parameters for safe evolution (Grahams hard-won lesson). 6.
      Interactive debugging (eval API already provides this: use it
      aggressively)."
1113            ""
1114            (list))))
1115       (outline/node*
1116         "pg-quotes"
1117         "Key Quotes"
1118         "Twelve memorable quotes spanning all the essays."
1119         ""
1120         (list
1121           (outline/node*
1122             "pg-quote-1"
1123             "The Programmable Programming Language"
1124             "Lisp is the programmable programming language. -- John
      Foderaro"
1125             ""
1126             (list))
1127           (outline/node*
1128             "pg-quote-2"
1129             "Secret Weapon"
1130             "We wrote our software in a weird AI language, with a bizarre
      syntax full of parentheses. For years it had annoyed me to hear Lisp
      described that way. But now it worked to our advantage. -- Beating the
      Averages"
1131             ""
1132             (list))
1133           (outline/node*
1134             "pg-quote-3"
1135             "No Syntax"
1136             "Lisp code is made out of Lisp data objects. Lisp has no
      syntax. You write programs in the parse trees that get generated
      within the compiler when other languages are parsed. But these parse
      trees are fully accessible to your programs. -- Beating the Averages"
1137             ""
1138             (list))
```

```
1139            (outline/node*
1140              "pg-quote-4"
1141              "Competitive Advantage"
1142              "In business, there is nothing more valuable than a technical
      advantage your competitors dont understand. -- Beating the Averages"
1143              ""
1144              (list))
1145            (outline/node*
1146              "pg-quote-5"
1147              "Blub Paradox"
1148              "By induction, the only programmers in a position to see all
      the differences in power between the various languages are those who
      understand the most powerful one. -- Beating the Averages"
1149              ""
1150              (list))
1151            (outline/node*
1152              "pg-quote-6"
1153              "On Lisp Thesis"
1154              "Instead of just writing your program in Lisp, you can write
      your own language on Lisp, and write your program in that. -- On Lisp"
1155              ""
1156              (list))
1157            (outline/node*
1158              "pg-quote-7"
1159              "Programmer Time"
1160              "Inefficient software isnt gross. What is gross is a language
      that makes programmers do needless work. Wasting programmer time is
      the true inefficiency, not wasting machine time. -- The Hundred-Year
      Language"
1161              ""
1162              (list))
1163            (outline/node*
1164              "pg-quote-8"
1165              "Small Cores"
1166              "The more of a language you can write in itself, the better. --
      The Hundred-Year Language"
1167              ""
1168              (list))
1169            (outline/node*
1170              "pg-quote-9"
1171              "Good Programmers"
1172              "Once you assume the user is a good programmer, you
      automatically have the answer to any such question: let the user do
      whatever he wants. -- Arc at 3 Weeks"
1173              ""
1174              (list))
1175            (outline/node*
1176              "pg-quote-10"
1177              "Brevity"
1178              "What we really like is not abstraction per se but brevity. --
      Arc at 3 Weeks"
1179              ""
1180              (list))
1181            (outline/node*
1182              "pg-quote-11"
1183              "Catching Up"
1184              "Its 2002, and programming languages have almost caught up with
      1958. -- Revenge of the Nerds"
1185              ""
1186              (list))
1187            (outline/node*
1188              "pg-quote-12"
1189              "The Recipe"
1190              "If you want to win in a software business, just take on the
      hardest problem you can find, use the most powerful language you can
      get, and wait for your competitors pointy-haired bosses to revert to
      the mean. -- Revenge of the Nerds"
1191              ""
1192              (list)))))))
1193
1194  (define plan
1195    (lambda (body) "Set the current user's plan." (plan/set! body)))
1196
```

```
(define plan-clear
  (lambda () "Clear the current user's plan." (plan/clear!)))

(define plan-list
  (lambda
    ()
    "Return non-empty (username . body) plan pairs."
    (list/filter
      (lambda (pair) (not (blank? (cdr pair))))
      (list/map
        (lambda
          (row)
          (let
            ((f (plan/row-fields row)))
            (cons (plan/fields-username f) (plan/fields-body f))))
        (plan/all-rows)))))

(define plan-of
  (lambda (user) "Get another user's plan." (plan/of user)))

(define pp-test-let*
  (__macro__
    (bindings body)
    (if
      (null? bindings)
      body
      (quasiquote
        (let
          ((unquote (car bindings)))
          (pp-test-let* (unquote (cdr bindings)) (unquote body)))))))

(define prefix-of
  (lambda (name) "Return the part of name before the first slash." (car
     (string-split (as-str name) "/"))))

(define probe
  (__macro__ (x) (as-str x)))

(define raw
  (__macro__ (x) (list (quote quote) x)))

(define respond-post
  (__macro__
    (redirect-path post-action get-html)
    (quasiquote
      (if
        (string-eq (request-method) "POST")
        (seq
          (unquote post-action)
          (if
            (hx-request?)
            (render-html (unquote get-html))
            (redirect (unquote redirect-path))))
        (html/page (unquote get-html))))))

(define show
  (lambda
    (tree)
    "Summarize a tree as node ids and child ids."
    (list/map
      (lambda
        (n)
        (list
          (outline/id n)
          (list/map outline/id (outline/kids n))))
      (outline/kids tree))))

(define showparams
  (__macro__ (a b . c) (list (quote quote) (list a b c))))

(define string-find-pos
  (lambda
    (s delim start dlen)
```

```
    "Find delim in s at or after start, or -1 when absent."
    (if
      (>= start (- (string-length s) (- dlen 1)))
      -1
      (if
        (string-eq (substring s start dlen) delim)
        start
        (string-find-pos s delim (+ start 1) dlen)))))

(define string-split
  (lambda
    (s delim)
    "Split s on delim into a list of substrings."
    (let
      ((dlen (string-length delim)) (slen (string-length s)))
      (if
        (or (= slen 0) (not (string-contains? s delim)))
        (list s)
        (let
          ((pos (string-find-pos s delim 0 dlen)))
          (if
            (= pos -1)
            (list s)
            (cons
              (substring s 0 pos)
              (string-split
                (substring s (+ pos dlen) (- slen (+ pos dlen)))
                delim)))))))))

(define t
  (outline/node*
    "root"
    "R"
    ""
    ""
    (list
      (outline/node*
        "a"
        "A"
        ""
        ""
        (list
          (outline/node* "a1" "A1" "" "" (list))
          (outline/node* "a2" "A2" "" "" (list))))
      (outline/node* "b" "B" "" "" (list))
      (outline/node* "c" "C" "" "" (list)))))

(define test-backtick
  (outline/node*
    "root"
    "Test `code` backtick"
    ""
    ""
    (list)))

(define test-bracket
  (outline/node*
    "root"
    "Test [HIGH] bracket"
    ""
    ""
    (list)))

(define test-colon
  (outline/node* "root" "Test : colon" "" "" (list)))

(define test-long
  (outline/node*
    "root"
    "This is a very long title that has many words and should test whether
     the issue is about string length or about specific characters in the
     content of the title"
    ""
```

```
    ""
    (list)))

(define test-nested
  (outline/node*
    "root"
    "Root"
    ""
    ""
    (list
      (outline/node* "c1" "Child1" "body" "" (list))
      (outline/node*
        "c2"
        "Child2"
        "body"
        ""
        (list
          (outline/node* "gc" "Grandchild" "body" "" (list)))))))

(define test-nl
  "hello")

(define test-outline
  (outline/node* "root" "Test Note" "body text" "" (list)))

(define test-outline-full
  (outline/node*
    "root"
    "Design Patterns for PP Web Dev"
    "Analysis of current patterns and recommendations based on Paul
     Graham's Lisp writings and codebase inspection."
    ""
    (list
      (outline/node*
        "s-good"
        "What's Already Good"
        "Patterns that are working well."
        ""
        (list
          (outline/node*
            "s-good-layers"
            "Layered Architecture"
            "The dependency graph is already bottom-up."
            ""
            (list)))))))

(define test-slash
  (outline/node*
    "root"
    "Test feed/page slash"
    ""
    ""
    (list)))

(define test-unicode
  (outline/node*
    "root"
    "Test -- em-dash ? arrow ?guillemet?"
    ""
    ""
    (list)))

(define tree
  (outline/node*
    "root"
    "Move smoke A"
    ""
    ""
    (list
      (outline/node* "keep" "Keep me" "" "" (list))
      (outline/node*
        "child"
        "Child title"
```

```
        "body-x"
        "TODO"
        (list (outline/node* "grand" "Grand" "" "" (list)))))))

(define unless
  (__macro__
    (test body)
    (quasiquote (if (unquote test) () (unquote body)))))

(define when
  (__macro__
    (test body)
    (quasiquote (if (unquote test) (unquote body) ()))))

(define when2
  (__macro__
    (t b)
    (quasiquote (if (unquote t) (unquote b) ()))))

(define x
  1)
```

149 libs/ai/lib.pp

```
;; package ai -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push

(define ai/agents-note-body
  (lambda
    ()
    "Body of the live agents notes doc, or empty string when notes is unloaded."
    (if
      (not (ai/bound? "notes/doc/get"))
      ""
      (let
        ((n (notes/doc/get "agents")))
        (if (null? n) "" (outline/body (notes/doc/outline n)))))))

(define ai/ask
  (lambda
    (q)
    "New copilot session seeded with the image context; returns the model reply."
    (let
      ((sid (agent-session-new ai/copilot-seed)))
      (agent-say
        sid
        (string-append (ai/context) "\n\nUser question:\n" q)))))

(define ai/author-seed
  "You are a Pricklypear Lisp author on a live multi-user Lisp OS (not a cactus). Respond ONLY with a single Lisp s-expression that is a function BODY (usually a lambda). No markdown fences, no define wrapper, no commentary. Prefer pure functions. Available prims: + - * = < > list car cdr cons null? string-append str dict-get dict-set get create update delete rows render-html list-bindings whoami current-branch.")

(define ai/bound?
  (lambda
    (name)
    "True if a binding named name is visible on the current branch."
    (not
      (null?
        (list/filter
          (lambda (b) (string-eq (as-str b) (as-str name)))
          (list-bindings))))))

(define ai/context
  (lambda
    ()
    "Multi-line image context string (user, branch, bindings, agents note) for the model."
    (let
      ((agents-body (ai/agents-note-body))
        (pkgs (list-packages))
        (nbind (count (list-bindings))))
      (string-append
        "=== IMAGE CONTEXT ===\n"
        "system: Pricklypear multi-user Lisp OS image (software, not a cactus)\n"
        "user="
        (whoami)
        "\n"
        "branch="
        (current-branch)
        "\n"
        "bindings-count="
        (as-str nbind)
        "\n"
        "packages="
        (str pkgs)
        "\n"
```

```
        "explore: (describe \"pkg\") (package-bindings \"pkg\")
    (apropos-docs \"pat\") (function-doc \"name\")\n"
        "reference: (notes/doc/get \"platform-ref\") for the Lisp API
    quick-ref\n"
        "bootstrap: GET /agent.txt and /.well-known/agent.json then POST
    /api/eval\n"
        (if
          (string-eq agents-body "")
          "agents-note: missing -- (load-library \"notes\") then
    (notes/doc/get \"agents\")\n"
          (string-append
            "=== AGENTS NOTE (live AGENTS.md; edit with notes/doc/upsert)
    ===\n"
            agents-body
            "\n=== END AGENTS NOTE ===\n"))
        "=== END CONTEXT ===")))

(define ai/copilot-seed
  "You are the co-pilot for a running Pricklypear IMAGE (a multi-user Lisp
     OS), NOT the cactus genus Opuntia/Pricklypear plant.\n\nPricklypear is
     software: users have live images; Lisp functions are durable data in
     Postgres; branches version code; routes bind functions to HTTP; the UI
     is htmx. Kernel prims include list-bindings, get, rows, create,
     update, define, route, render-html, agent-session-new, agent-say,
     propose-function, install-function.\n\nRules:\n- Prefer facts from the
     prompt CONTEXT block (user, branch, bindings, etc.).\n- If CONTEXT
     does not contain the answer, say you do not know from this image -- do
     not invent store contents or pretend to be a botany article.\n- Answer
     clearly and briefly unless asked for detail.\n- Never describe
     Pricklypear as a cactus unless the user is clearly asking about the
     plant in nature (then you may clarify the name collision).\n- When
     tools are available, use them to inspect the live image rather than
     guessing.\n- After (load-library \"notes\"), the note slug agents is
     this habitat's living AGENTS.md. Read it with (notes/doc/get
     \"agents\"). Update it with notes/doc/upsert when durable agent
     guidance should change.\n- Knowledge wiki: (wiki/ingest-from-source
     ?), (wiki/query ?), (wiki/lint); links are [[slug]] in outline
     bodies.")

(define ai/do
  (lambda
    (task)
    "New task-seeded agent session with the image context; returns the
     result."
    (let
      ((sid (agent-session-new ai/task-seed)))
      (agent-say
        sid
        (string-append (ai/context) "\n\nTask:\n" task)))))

(define ai/draft-fn
  (lambda
    (name spec)
    "Author session that drafts a function body from name and spec; returns
     the draft."
    (let
      ((sid (agent-session-new ai/author-seed #f)))
      (agent-say
        sid
        (string-append
          "Function name: "
          name
          "\nRequirement:\n"
          spec)))))

(define ai/install
  (lambda (name body) (install-function name body))
  "Install a lambda body as a function named name.")

(define ai/say
  (lambda
    (sid q)
```

```
    "Send q to an existing session sid with the image context; returns the
     reply."
    (agent-say
      sid
      (string-append (ai/context) "\n\nUser question:\n" q))))

(define ai/task-seed
  "You are the agent for a running Pricklypear multi-user Lisp OS image
     (software, not a cactus).\nTools: image_context, list_bindings,
     list_routes, inspect_function, install_function.\nInstall only lambda
     bodies. Prefer package/name. Finish with a short text summary and no
     tool calls.")

(define ai/write
  (lambda (name spec) (propose-function name spec))
  "Propose (draft) a function from name and spec without installing.")
```

150 libs/astro/lib.pp

```
;; package astro -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push

(define astro/date-parts
  (lambda
    (d)
    "Split ISO date into (year month day)."
    (let
      ((ps (string-split d "-")))
      (list
        (parse-int (list-ref ps 0))
        (parse-int (list-ref ps 1))
        (parse-int (list-ref ps 2))))))

(define astro/date-seq
  (lambda
    (start n)
    "List of n consecutive dates starting at start."
    (if
      (<= n 0)
      (list)
      (cons
        start
        (astro/date-seq (date-add-days start 1) (- n 1))))))

(define astro/days-since-epoch
  (lambda
    (d)
    "Decimal days since the new-moon epoch."
    (-
      (date-diff-days astro/new-moon-epoch d)
      astro/epoch-frac)))

(define astro/default-label
  "Cody, WY")

(define astro/default-lat
  "44.52634")

(define astro/default-lon
  "-109.05653")

(define astro/epoch-frac
  0.7597)

(define astro/forecast-row-html
  (lambda
    (d)
    "HTML table row for one date's sun/moon forecast."
    (let
      ((parts (astro/date-parts d)))
      (let
        ((y (list-ref parts 0))
          (m (list-ref parts 1))
          (day (list-ref parts 2)))
        (let
          ((wd (astro/weekday-short (date-weekday y m day))))
          (html/tag
            "tr"
            (html/attrs (list (html/attr "data-date" d)))
            (html/join
              (list
                (astro/td (string-append wd " " d))
                (astro/td-id (string-append "sr-" d) "-")
                (astro/td-id (string-append "ss-" d) "-")
                (astro/td-id (string-append "dl-" d) "-")
                (astro/td
                  (string-append
                    (astro/phase-glyph d)
                    " "
                    (astro/phase-name d)))
```

```
                (astro/td
                  (string-append (as-str (astro/illum-percent d))
    "%")))))))))))

(define astro/forecast-rows-html
  (lambda
    (dates)
    "HTML rows for each date in the list."
    (if
      (null? dates)
      ""
      (html/join
        (list
          (astro/forecast-row-html (car dates))
          (astro/forecast-rows-html (cdr dates)))))))

(define astro/forecast-table-html
  (lambda
    ()
    "Full 7-day forecast table HTML."
    (let
      ((dates (astro/date-seq (date-today) 7)))
      (html/tag
        "table"
        (html/attrs (list (html/attr "class" "astro-table")))
        (html/join
          (list
            (html/tag
              "thead"
              ""
              (html/tag
                "tr"
                ""
                (html/join
                  (list
                    (astro/th "Date")
                    (astro/th "Sunrise")
                    (astro/th "Sunset")
                    (astro/th "Daylight")
                    (astro/th "Moon Phase")
                    (astro/th "Illum")))))
            (html/tag "tbody" "" (astro/forecast-rows-html dates)))))))))

(define astro/help
  (lambda
    ()
    "Help text describing the astro package API."
    (string-append
      "astro - sunrise/sunset + moon phase. "
      "Sun times are fetched client-side (Open-Meteo); moon is computed on
    the image. "
      "REPL: (astro/moon (date-today)) returns glyph+name+illum+age. "
      "Also: (astro/phase-name d) (astro/phase-glyph d)
    (astro/illum-percent d) "
      "(astro/phase-fraction d) (astro/phase-age d). "
      "Web: /astro  -  embed: (astro/widget).")))

(define astro/historical-panel-html
  (lambda
    ()
    "HTML panel for looking up a past date."
    (html/div
      (html/attr "class" "astro-historical")
      (html/join
        (list
          (html/h3 "Look up a date")
          (html/p*
            (html/attr "class" "muted")
            "Enter any past date to see its sunrise/sunset (Open-Meteo
    Archive) and moon phase (computed on the image). Uses the location set
    above.")
          (html/tag
            "form"
```

```
            (html/attrs
              (list
                (html/attr "id" "astro-hist-form")
                (html/attr "class" "astro-form")
                (html/attr "action" "#")
                (html/attr "method" "get")))
            (html/join
              (list
                (html/input
                  (html/attrs
                    (list
                      (html/attr "type" "date")
                      (html/attr "id" "astro-hist-date")
                      (html/attr "name" "date")
                      (html/attr "value" (date-today)))))
                (html/button (html/attr "type" "submit") "Look up"))))
          (html/div (html/attr "id" "astro-hist-result") "")))))

(define astro/historical-script
  (lambda
    ()
    "Client JS for the historical lookup panel."
    (html/tag
      "script"
      ""
      (string-append
        "(function(){"
        "var root=document.getElementById('astro-widget');"
        "if(!root)return;"
        "var form=document.getElementById('astro-hist-form');"
        "var dateEl=document.getElementById('astro-hist-date');"
        "var result=document.getElementById('astro-hist-result');"
        "var errEl=document.getElementById('astro-err');"
        "var KEY='pp.astro.loc';"
        "var defLoc={lat:"
        (as-str astro/default-lat)
        ",lon:"
        (as-str astro/default-lon)
        "};"
        "function loadLoc(){try{var s=localStorage.getItem(KEY);if(s)return
    JSON.parse(s);}catch(e){}return defLoc;}"
        "function fmtTime(iso){if(!iso)return '-';var
    t=String(iso).split('T')[1]||'';return t.slice(0,5);}"
        "function fmtDur(sec){if(sec==null)return
    '-';sec=Math.round(sec);var
    h=Math.floor(sec/3600),m=Math.floor((sec%3600)/60);return h+'h
    '+m+'m';}"
        "function
    fail(msg){if(errEl)errEl.textContent=msg;result.innerHTML='';}"
        "function lookup(date,loc){"
        "result.innerHTML='<p class=\"muted\">Loading...</p>';"
        "if(errEl)errEl.textContent='';"
        "var
    sunUrl='https://archive-api.open-meteo.com/v1/archive?latitude='+encodeURIComponent(loc.lat)"
        "+'&longitude='+encodeURIComponent(loc.lon)"

    "+'&start_date='+encodeURIComponent(date)+'&end_date='+encodeURIComponent(date)"
        "+'&daily=sunrise,sunset,daylight_duration&timezone=auto';"
        "var
    moonUrl='/partials/astro/moon-for?date='+encodeURIComponent(date);"
        "Promise.all([fetch(sunUrl).then(function(r){if(!r.ok)throw new
    Error('Archive HTTP '+r.status);return
    r.json();}),fetch(moonUrl).then(function(r){if(!r.ok)throw new
    Error('Moon HTTP '+r.status);return r.text();})])"
        ".then(function(res){var j=res[0];var moonHtml=res[1];"
        "var d=(j.daily||{});var sr=(d.sunrise||[])[0];var
    ss=(d.sunset||[])[0];var dl=(d.daylight_duration||[])[0];"
        "result.innerHTML="
        "'<div class=\"astro-hist-card\">'"
        "+'<div class=\"astro-hist-date\">'+date+'</div>'"
        "+'<div class=\"astro-hist-sun\">'"
        "+'<div><span class=\"lbl\">Sunrise
    </span><strong>'+(fmtTime(sr)||'-')+'</strong></div>'"
```

```
        "+'<div><span class=\"lbl\">Sunset
    </span><strong>'+(fmtTime(ss)||'-')+'</strong></div>'"
        "+'<div><span class=\"lbl\">Daylight
    </span><strong>'+fmtDur(dl)+'</strong></div>'"
        "+'</div>'"
        "+'<div class=\"astro-hist-moon\">'+moonHtml+'</div>'"
        "+'</div>';"
        "})"
        ".catch(function(e){fail('Lookup failed:
    '+(e&&e.message?e.message:e));});"
        "}"
        "if(form){form.addEventListener('submit',function(e){"
        "e.preventDefault();var
    q=(dateEl&&dateEl.value||'').trim();if(!q)return;lookup(q,loadLoc());"
        "});}"
        "})();"))))

(define astro/illum-fraction
  (lambda
    (d)
    "Illuminated fraction in 0..1 from phase fraction."
    (let
      ((theta (* astro/pi (astro/phase-fraction d))))
      (let ((s (astro/sin-0-pi theta))) (* s s)))))

(define astro/illum-percent
  (lambda (d) "Percent illumination, rounded." (round (* 100
     (astro/illum-fraction d)))))

(define astro/month-name
  (lambda (m) "Month name by 1-based index." (list-ref astro/month-names (-
     m 1))))

(define astro/month-names
  (list
    "January"
    "February"
    "March"
    "April"
    "May"
    "June"
    "July"
    "August"
    "September"
    "October"
    "November"
    "December"))

(define astro/moon
  (lambda
    (d)
    "One-line description of the moon phase for d."
    (string-append
      (astro/phase-glyph d)
      " "
      (astro/phase-name d)
      " - "
      (as-str (astro/illum-percent d))
      "% illuminated - age "
      (as-str (round (astro/phase-age d)))
      " days")))

(define astro/moon-card-html
  (lambda
    (d)
    "HTML card showing the moon phase for d."
    (html/div
      (html/attr "class" "astro-moon-big")
      (html/join
        (list
          (html/span
            (html/attr "class" "astro-glyph")
            (astro/phase-glyph d))
```

```
          (html/div
            (html/attr "class" "astro-mname")
            (astro/phase-name d))
          (html/div
            (html/attr "class" "astro-illum")
            (string-append
              (as-str (astro/illum-percent d))
              "% illuminated"))
          (html/div
            (html/attr "class" "astro-age")
            (string-append
              "Age "
              (as-str (round (astro/phase-age d)))
              " days")))))))

(define astro/moon-for
  (lambda
    ()
    "Moon card for the request date param (today when absent)."
    (let
      ((raw (request-param "date")))
      (let
        ((d
          (if
            (or (null? raw) (string-eq (as-str raw) ""))
            (date-today)
            (as-str raw))))
        (astro/moon-card-html d)))))

(define astro/new-moon-epoch
  "2000-01-06")

(define astro/page
  (lambda
    ()
    "Full Sun & Moon page HTML."
    (html/page
      (list
        (html/h1 "Sun & Moon")
        (html/p*
          (html/attr "class" "muted")
          "Sunrise & sunset via Open-Meteo (browser fetch). Moon phase
     computed on the image.")
        (astro/widget)
        (astro/historical-panel-html)
        (astro/historical-script)))))

(define astro/phase-age
  (lambda
    (d)
    "Days since the last new moon."
    (let
      ((a (mod (astro/days-since-epoch d) astro/synodic-month)))
      (if (< a 0) (+ a astro/synodic-month) a))))

(define astro/phase-fraction
  (lambda
    (d)
    "Fractional position in the current lunation, 0..1."
    (let
      ((p
        (mod
          (div (astro/days-since-epoch d) astro/synodic-month)
          1.0)))
      (if (< p 0) (+ p 1.0) p))))

(define astro/phase-glyph
  (lambda
    (d)
    "Unicode glyph for the moon phase of d."
    (list-ref astro/phase-glyphs (astro/phase-index d))))

(define astro/phase-glyphs
```

```
  (list
    "?"
    "?"
    "?"
    "?"
    "?"
    "?"
    "?"
    "?"))

(define astro/phase-index
  (lambda
    (d)
    "Bucket index 0..7 for the moon phase of d."
    (mod
      (floor (* (+ (astro/phase-fraction d) 0.0625) 8))
      8)))

(define astro/phase-name
  (lambda
    (d)
    "Human-readable name of the moon phase of d."
    (list-ref astro/phase-names (astro/phase-index d))))

(define astro/phase-names
  (list
    "New Moon"
    "Waxing Crescent"
    "First Quarter"
    "Waxing Gibbous"
    "Full Moon"
    "Waning Gibbous"
    "Last Quarter"
    "Waning Crescent"))

(define astro/phase-waxing?
  (lambda (d) "True when the moon is waxing." (< (astro/phase-fraction d)
     0.5)))

(define astro/pi
  3.14159265359)

(define astro/pi-sq
  9.86960440109)

(define astro/probe
  (lambda () "Probe helper returning 7." 7))

(define astro/rp-null
  (lambda
    ()
    "Test helper: \"null\" for missing request param."
    (if (null? (request-param "nope")) "null" "notnull")))

(define astro/rp-test
  (lambda
    ()
    "Test helper echoing the request date param."
    (string-append "got=" (as-str (request-param "date")))))

(define astro/script
  (lambda
    (lat lon label)
    "Client JS driving the live widget (geocode + Open-Meteo fetch)."
    (html/tag
      "script"
      ""
      (string-append
        "(function(){"
        "var root=document.getElementById('astro-widget');"
        "if(!root)return;"
        "var locEl=document.getElementById('astro-loc-label');"
        "var form=document.getElementById('astro-loc-form');"
```

```
410         "var input=document.getElementById('astro-city');"
411         "var todaySr=document.getElementById('today-sr');"
412         "var todaySs=document.getElementById('today-ss');"
413         "var todayDl=document.getElementById('today-dl');"
414         "var errEl=document.getElementById('astro-err');"
415         "var upEl=document.getElementById('astro-updated');"
416         "var KEY='pp.astro.loc';"
417         "var def={lat:"
418         (as-str lat)
419         ",lon:"
420         (as-str lon)
421         ",label:"
422         (json-emit (as-str label))
423         "};"
424         "function loadLoc(){try{var s=localStorage.getItem(KEY);if(s)return
    JSON.parse(s);}catch(e){}return def;}"
425         "function
    saveLoc(loc){try{localStorage.setItem(KEY,JSON.stringify(loc));}catch(e){}}"
426         "function fmtTime(iso){if(!iso)return '-';var
    t=String(iso).split('T')[1]||'';return t.slice(0,5);}"
427         "function fmtDur(sec){if(sec==null)return
    '-';sec=Math.round(sec);var
    h=Math.floor(sec/3600),m=Math.floor((sec%3600)/60);return h+'h
    '+m+'m';}"
428         "function fillRow(date,sr,ss,dl){"
429         "var a=document.getElementById('sr-'+date);var
    b=document.getElementById('ss-'+date);var
    c=document.getElementById('dl-'+date);"
430
    "if(a)a.textContent=fmtTime(sr);if(b)b.textContent=fmtTime(ss);if(c)c.textContent=fmtDur(dl);"
431         "}"
432         "function
    renderToday(sr,ss,dl){if(todaySr)todaySr.textContent=fmtTime(sr);if(todaySs)todaySs.textContent=fmtTime(ss);if(todayDl)
433         "function fail(msg){if(errEl)errEl.textContent=msg;}"
434         "function fetchAstro(loc){"
435         "if(locEl)locEl.textContent=loc.label||(loc.lat+', '+loc.lon);"
436         "var
    url='https://api.open-meteo.com/v1/forecast?latitude='+encodeURIComponent(loc.lat)"
437         "+'&longitude='+encodeURIComponent(loc.lon)"
438
    "+'&daily=sunrise,sunset,daylight_duration&timezone=auto&forecast_days=7';"
439         "fetch(url).then(function(r){if(!r.ok)throw new Error('HTTP
    '+r.status);return r.json();})"
440         ".then(function(j){var d=j.daily||{};var ts=d.time||[];var
    sr=d.sunrise||[];var ss=d.sunset||[];var dl=d.daylight_duration||[];"
441         "if(ts.length){renderToday(sr[0],ss[0],dl[0]);}"
442         "for(var i=0;i<ts.length;i++){fillRow(ts[i],sr[i],ss[i],dl[i]);}"
443         "if(upEl)upEl.textContent='via Open-Meteo - local browser fetch';"
444         "})"
445         ".catch(function(e){fail('Sun data fetch failed:
    '+(e&&e.message?e.message:e));});"
446         "}"
447         "function geocode(q){"
448         "var
    url='https://geocoding-api.open-meteo.com/v1/search?name='+encodeURIComponent(q)+'&count=1&language=en&format=json';"
449         "return fetch(url).then(function(r){if(!r.ok)throw new Error('HTTP
    '+r.status);return r.json();})"
450         ".then(function(j){if(!j.results||!j.results.length)throw new
    Error('place not found');"
451         "var p=j.results[0];var
    bits=[p.name,p.admin1,p.country_code].filter(Boolean);"
452         "return{lat:p.latitude,lon:p.longitude,label:bits.join(', ')};});"
453         "}"
454         "if(form){form.addEventListener('submit',function(e){"
455         "e.preventDefault();var
    q=(input&&input.value||'').trim();if(!q)return;"
456         "geocode(q).then(function(loc){saveLoc(loc);fetchAstro(loc);})"
457
    ".catch(function(err){fail(err&&err.message?err.message:String(err));});"
458         "});}"
459         "fetchAstro(loadLoc());"
460         "})();"))))
461
```

```
(define astro/sin-0-pi
  (lambda
    (theta)
    "Parabolic approximation of sin over 0..pi."
    (let
      ((u (- astro/pi theta)))
      (let
        ((q (* theta u)))
        (let
          ((num (* 16 q)) (den (- (* 5 astro/pi-sq) (* 4 q))))
          (div num den))))))

(define astro/stdin-probe
  (lambda () "stdin-ok"))

(define astro/style
  (lambda
    ()
    "Inline CSS for the astro widget."
    (html/tag
      "style"
      ""
      (html/join
        (list
          ".astro-card .astro-today{display:flex;gap:1.5rem;flex-wrap:wrap;align-items:center;margin:.5rem 0}"
          ".astro-card .astro-moon-big{line-height:1.4}"
          ".astro-card .astro-glyph{font-size:3.5rem;line-height:1;display:block}"
          ".astro-card .astro-mname{font-weight:700;margin:.15rem 0}"
          ".astro-card .astro-illum{color:var(--muted,#444)}"
          ".astro-card .astro-age{color:var(--muted,#666);font-size:.85rem}"
          ".astro-card .astro-sun-today .astro-sun-line{margin:.15rem 0}"
          ".astro-card .astro-sun-today .lbl{color:var(--muted,#666);display:inline-block;min-width:4.5rem}"
          ".astro-card table.astro-table{width:100%;border-collapse:collapse;margin-top:1rem;font-size:.92rem}"
          ".astro-card table.astro-table th,.astro-card table.astro-table td{text-align:left;padding:.3rem .5rem;border-bottom:1px solid var(--shadow,#eee)}"
          ".astro-card table.astro-table th{background:var(--chrome,#f6f6f6)}"
          ".astro-card .astro-form{display:flex;gap:.35rem;flex-wrap:wrap;margin:.5rem 0}"
          ".astro-card .astro-form input[type=text]{flex:1 1 8rem;min-width:0}"
          ".astro-card .astro-form button{width:auto;min-width:4.5rem}"
          ".astro-card .astro-loc{font-weight:700;margin:.25rem 0}"
          ".astro-card .astro-err{color:var(--err,#800000);font-weight:700}"
          ".astro-card .astro-updated{font-size:.8rem;color:var(--muted,#555);margin-top:.35rem}"
          ".astro-card .astro-historical{margin-top:1.5rem;padding-top:1rem;border-top:1px solid var(--shadow,#eee)}"
          ".astro-card .astro-historical h3{margin:.25rem 0}"
          ".astro-card .astro-hist-card{display:flex;gap:1.5rem;flex-wrap:wrap;align-items:center;margin-top:.5rem}"
          ".astro-card .astro-hist-date{font-weight:700;font-size:1.1rem}"
          ".astro-card .astro-hist-sun .lbl{color:var(--muted,#666);display:inline-block;min-width:4.5rem}"
          ".astro-card .astro-hist-sun div{margin:.15rem 0}"
          ".astro-card .astro-form input[type=date]{flex:0 0 auto}"
          ".astro-card .astro-hist-moon .astro-glyph{font-size:2.5rem}")))))

(define astro/synodic-month
  29.530588853)

(define astro/td
  (lambda (content) "A table data cell." (html/tag "td" "" content)))

(define astro/td-id
  (lambda
```

```
    (id content)
    "A table data cell with an id attribute."
    (html/tag
      "td"
      (html/attrs (list (html/attr "id" id)))
      content)))

(define astro/th
  (lambda (content) "A table header cell." (html/tag "th" "" content)))

(define astro/today-card-html
  (lambda
    ()
    "HTML card with today's moon + sun times."
    (let
      ((d (date-today)))
      (html/div
        (html/attr "class" "astro-today")
        (html/join
          (list
            (astro/moon-card-html d)
            (html/div
              (html/attr "class" "astro-sun-today")
              (html/join
                (list
                  (html/div
                    (html/attr "class" "astro-sun-line")
                    (html/join
                      (list
                        (html/span (html/attr "class" "lbl") "Sunrise ")
                        (html/span (html/attr "id" "today-sr") "-"))))
                  (html/div
                    (html/attr "class" "astro-sun-line")
                    (html/join
                      (list
                        (html/span (html/attr "class" "lbl") "Sunset ")
                        (html/span (html/attr "id" "today-ss") "-"))))
                  (html/div
                    (html/attr "class" "astro-sun-line")
                    (html/join
                      (list
                        (html/span (html/attr "class" "lbl") "Daylight ")
                        (html/span (html/attr "id" "today-dl")
    "-")))))))))))))

(define astro/weekday-name
  (lambda (i) "Full weekday name by index." (list-ref astro/weekday-names
    i)))

(define astro/weekday-names
  (list
    "Sunday"
    "Monday"
    "Tuesday"
    "Wednesday"
    "Thursday"
    "Friday"
    "Saturday"))

(define astro/weekday-short
  (lambda (i) "Abbreviated weekday name by index." (list-ref
    astro/weekday-short-names i)))

(define astro/weekday-short-names
  (list
    "Sun"
    "Mon"
    "Tue"
    "Wed"
    "Thu"
    "Fri"
    "Sat"))

```

```
(define astro/widget
  (lambda
    ()
    "Full embeddable sun & moon widget HTML."
    (html/div
      (html/attrs
        (list
          (html/attr "class" "card astro-card")
          (html/attr "id" "astro-widget")))
      (html/join
        (list
          (html/h2 "Sun & Moon")
          (html/div
            (html/attrs
              (list
                (html/attr "class" "astro-loc")
                (html/attr "id" "astro-loc-label")))
            astro/default-label)
          (html/div
            (html/attrs
              (list
                (html/attr "id" "astro-err")
                (html/attr "class" "astro-err")))
            "")
          (html/tag
            "form"
            (html/attrs
              (list
                (html/attr "id" "astro-loc-form")
                (html/attr "class" "astro-form")
                (html/attr "action" "#")
                (html/attr "method" "get")))
            (html/join
              (list
                (html/input
                  (html/attrs
                    (list
                      (html/attr "type" "text")
                      (html/attr "id" "astro-city")
                      (html/attr "name" "city")
                      (html/attr "placeholder" "City...")
                      (html/attr "autocomplete" "off"))))
                (html/button (html/attr "type" "submit") "Set"))))
          (astro/today-card-html)
          (astro/forecast-table-html)
          (html/div
            (html/attrs
              (list
                (html/attr "class" "astro-updated")
                (html/attr "id" "astro-updated")))
            "")
          (astro/style)
          (astro/script
            astro/default-lat
            astro/default-lon
            astro/default-label))))))
```

151 libs/bi/dashboard/lib.pp

```
;; package bi/dashboard -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push

(define bi/dashboard
  (lambda
    ()
    "Seed sample data then render the dashboard page."
    (seq (bi/seed-sample-data) (bi/dashboard-html))))

(define bi/dashboard/preview-note
  (lambda
    ()
    "Branch preview: (fork \"bi-preview\") (checkout \"?/bi-preview\") then
     edit bi/dashboard-html; merge when ready. Metrics functions stay on
     main and are read-only to layout agents."))

(define bi/dashboard/publish
  (lambda
    (source-body prompt)
    "Install bi/dashboard-html from source and record the revision."
    (seq
      (install-function
        "bi/dashboard-html"
        (as-str source-body))
      (create
        (quote dashboard-revision)
        (dict-set*
          "{}"
          (list
            "name"
            "bi/dashboard-html"
            "source"
            (as-str source-body)
            "prompt"
            (as-str prompt)
            "author"
            (whoami)
            "created_at"
            (date-today))))
      "published")))

(define bi/dashboard/read
  (lambda
    ()
    "Return the dashboard metrics, source, and route as a dict."
    (dict-set*
      "{}"
      (list
        "metrics"
        (bi/metrics-snapshot)
        "source"
        (bi/dashboard/source)
        "route"
        "/bi"))))

(define bi/dashboard/source
  (lambda () "Source text of bi/dashboard-html." (function-source
     "bi/dashboard-html")))
```

152 libs/bi/lib.pp

```
;; package bi -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push

(define bi/cache-metrics!
  (lambda
    ()
    (let
      ((snap (bi/metrics-snapshot)) (now (date-today)))
      (let
        ((existing
          (find-row (quote bi-metric-cache) "name" "snapshot")))
        (if
          (null? existing)
          (create
            (quote bi-metric-cache)
            (dict-set*
              "{}"
              (list
                "name"
                "snapshot"
                "value"
                snap
                "updated_at"
                now)))
          (update
            (quote bi-metric-cache)
            (car existing)
            (dict-set*
              "{}"
              (list
                "name"
                "snapshot"
                "value"
                snap
                "updated_at"
                now)))))))

(define bi/dashboard
  (lambda
    ()
    (seq (bi/seed-sample-data) (bi/dashboard-html))))

(define bi/dashboard-html
  (lambda
    ()
    (let
      ((snap (bi/metrics-snapshot)))
      (html/page
        (list
          (bi/vega-cdn)
          (html/tag
            "style"
            ""
            (string-append

    ".bi-grid{display:grid;grid-template-columns:repeat(auto-fit,minmax(10rem,1fr));gap:.75rem;margin:1rem
    0;}"
              ".bi-card{border:2px solid
    #000;padding:.75rem;background:#fff;}"

    ".bi-card-label{font-size:.8rem;text-transform:uppercase;letter-spacing:.04em;}"

    ".bi-card-value{font-size:1.4rem;font-weight:700;margin-top:.25rem;}"
              ".bi-chart{margin:1.25rem 0;border:2px solid
    #000;padding:1rem;background:#fff;}"))
          (html/h1 "P&amp;L Dashboard")
          (html/p*
            (html/attr "class" "muted")
            "Governed metrics over bi-expense / bi-revenue. Agents may edit
    layout; metrics stay read-only.")
          (html/div
```

```
            (html/attr "class" "bi-grid")
            (html/join
              (list
                (bi/metric-card "Revenue" (dict-get snap "revenue"))
                (bi/metric-card "Expenses" (dict-get snap "expenses"))
                (bi/metric-card "Profit" (dict-get snap "profit"))
                (bi/metric-card "Margin" (dict-get snap "margin")))))
          (html/div
            (html/attr "class" "bi-chart")
            (html/join
              (list
                (html/h2 "Expenses by category")
                (bi/vega-lite (bi/spec-expense-bars)))))
          (html/p*
            (html/attr "class" "muted")
            (html/join
              (list
                (html/text "API: ")
                (html/code "(bi/metrics-snapshot)")
                (html/text " ? ")
                (html/code "(bi/dashboard/source)")
                (html/text " ? ")
                (html/a "/bi" (html/text "refresh")))))))))

(define bi/dashboard/preview-note
  (lambda
    ()
    "Branch preview: (fork \"bi-preview\") (checkout \"?/bi-preview\") then
     edit bi/dashboard-html; merge when ready. Metrics functions stay on
     main and are read-only to layout agents."))

(define bi/dashboard/publish
  (lambda
    (source-body prompt)
    (seq
      (install-function
        "bi/dashboard-html"
        (as-str source-body))
      (create
        (quote dashboard-revision)
        (dict-set*
          "{}"
          (list
            "name"
            "bi/dashboard-html"
            "source"
            (as-str source-body)
            "prompt"
            (as-str prompt)
            "author"
            (whoami)
            "created_at"
            (date-today))))
      "published")))

(define bi/dashboard/read
  (lambda
    ()
    (dict-set*
      "{}"
      (list
        "metrics"
        (bi/metrics-snapshot)
        "source"
        (bi/dashboard/source)
        "route"
        "/bi"))))

(define bi/dashboard/source
  (lambda () (function-source "bi/dashboard-html")))

(define bi/fmt-money
  (lambda (n) (string-append "$" (as-str n))))
```

```
(define bi/join-json-array
  (lambda
    (xs)
    (list/foldl
      (lambda
        (acc x)
        (if (string-eq acc "") x (string-append acc "," x)))
      ""
      xs)))

(define bi/metric-card
  (lambda
    (label value)
    (html/div
      (html/attr "class" "bi-card")
      (html/join
        (list
          (html/div
            (html/attr "class" "bi-card-label")
            (html/text label))
          (html/div
            (html/attr "class" "bi-card-value")
            (html/text (as-str value))))))))

(define bi/metric/expense-by-category
  (lambda
    ()
    (list/foldl
      (lambda
        (acc row)
        (let
          ((cat (as-str (dict-get (cdr row) "category")))
            (amt (bi/row-amount row)))
          (let
            ((prev (dict-get acc cat)))
            (dict-set
              acc
              cat
              (as-str
                (if (null? prev) amt (+ (bi/parse-amount prev) amt)))))))
      "{}"
      (rows (quote bi-expense) 500))))

(define bi/metric/expense-total
  (lambda
    ()
    (bi/sum-amounts (rows (quote bi-expense) 500))))

(define bi/metric/profit
  (lambda
    ()
    (- (bi/metric/revenue-total) (bi/metric/expense-total))))

(define bi/metric/profit-margin
  (lambda
    ()
    (let
      ((rev (bi/metric/revenue-total)))
      (if (= rev 0.0) 0.0 (div (bi/metric/profit) rev)))))

(define bi/metric/revenue-total
  (lambda
    ()
    (bi/sum-amounts (rows (quote bi-revenue) 500))))

(define bi/metrics-snapshot
  (lambda
    ()
    (dict-set*
      "{}"
      (list
        "revenue"
```

```
        (as-str (bi/metric/revenue-total))
        "expenses"
        (as-str (bi/metric/expense-total))
        "profit"
        (as-str (bi/metric/profit))
        "margin"
        (as-str (bi/metric/profit-margin))
        "by_category"
        (bi/metric/expense-by-category)))))

(define bi/parse-amount
  (lambda
    (s)
    (let
      ((f (parse-float (as-str s))))
      (if (null? f) 0.0 f))))

(define bi/row-amount
  (lambda
    (row)
    (bi/parse-amount (dict-get (cdr row) "amount"))))

(define bi/seed-sample-data
  (lambda
    ()
    (seq
      (if
        (null? (rows (quote bi-expense) 1))
        (seq
          (create
            (quote bi-expense)
            (dict-set*
              "{}"
              (list
                "day"
                "2026-07-01"
                "category"
                "hosting"
                "amount"
                "42.00"
                "note"
                "Vultr")))
          (create
            (quote bi-expense)
            (dict-set*
              "{}"
              (list
                "day"
                "2026-07-05"
                "category"
                "software"
                "amount"
                "20.00"
                "note"
                "domain")))
          (create
            (quote bi-expense)
            (dict-set*
              "{}"
              (list
                "day"
                "2026-07-12"
                "category"
                "hosting"
                "amount"
                "42.00"
                "note"
                "Vultr")))
          (create
            (quote bi-expense)
            (dict-set*
              "{}"
              (list
```

```
                "day"
                "2026-07-18"
                "category"
                "tools"
                "amount"
                "15.50"
                "note"
                "CDN")))
          0)
        0)
      (if
        (null? (rows (quote bi-revenue) 1))
        (seq
          (create
            (quote bi-revenue)
            (dict-set*
              "{}"
              (list
                "day"
                "2026-07-03"
                "source"
                "consulting"
                "amount"
                "500.00"
                "note"
                "week 1")))
          (create
            (quote bi-revenue)
            (dict-set*
              "{}"
              (list
                "day"
                "2026-07-10"
                "source"
                "product"
                "amount"
                "120.00"
                "note"
                "subs")))
          (create
            (quote bi-revenue)
            (dict-set*
              "{}"
              (list
                "day"
                "2026-07-17"
                "source"
                "consulting"
                "amount"
                "350.00"
                "note"
                "week 3")))
          0)
        0)
      "seeded")))

(define bi/spec-expense-bars
  (lambda
    ()
    (let
      ((by (bi/metric/expense-by-category)))
      (let
        ((keys (dict-keys by)))
        (let
          ((values
            (list/map
              (lambda
                (k)
                (string-append
                  "{\"category\":"
                  (json-emit (as-str k))
                  ",\"amount\":"
                  (as-str (bi/parse-amount (dict-get by k)))
```

```
                  "}"))
              keys)))
          (string-append

    "{\"$schema\":\"https://vega.github.io/schema/vega-lite/v5.json\","
           "\"description\":\"Expenses by category\","
           "\"width\":320,\"height\":200,"
           "\"data\":{\"values\":["
           (bi/join-json-array values)
           "]},"
           "\"mark\":\"bar\","
           "\"encoding\":{"

    "\"x\":{\"field\":\"category\",\"type\":\"nominal\",\"title\":\"Category\"},"

    "\"y\":{\"field\":\"amount\",\"type\":\"quantitative\",\"title\":\"Amount\"}"
           "}}"))))))

(define bi/sum-amounts
  (lambda
    (rows)
    (list/foldl
      (lambda (acc row) (+ acc (bi/row-amount row)))
      0.0
      rows)))

(define bi/vega-cdn
  (lambda
    ()
    (html/join
      (list
        (html/tag
          "script"
          (html/attr "src" "https://cdn.jsdelivr.net/npm/vega@5")
          "")
        (html/tag
          "script"
          (html/attr
            "src"
            "https://cdn.jsdelivr.net/npm/vega-lite@5")
          "")
        (html/tag
          "script"
          (html/attr
            "src"
            "https://cdn.jsdelivr.net/npm/vega-embed@6")
          "")))))

(define bi/vega-lite
  (lambda
    (spec-json)
    (let
      ((id (string-append "vl-" (as-str (gensym "c"))))
        (spec (as-str spec-json)))
      (html/join
        (list
          (html/tag "div" (html/attr "id" id) "")
          (html/tag
            "script"
            ""
            (string-append
              "(function(){var el=document.getElementById('"
              id
              "');"
              "if(!el)return;"
              "function go(){if(window.vegaEmbed){vegaEmbed('#"
              id
              "',"
              spec

    ",{actions:false}).catch(function(e){el.textContent=String(e);});}"
              "else{setTimeout(go,50);}}go();})();")))))))
```

153 libs/bi/metric/lib.pp

```
;; package bi/metric -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push

(define bi/metric/expense-by-category
  (lambda
    ()
    "Sum expenses grouped by category; drops unknown categories."
    (list/foldl
      (lambda
        (acc row)
        (let
          ((cat (as-str (dict-get (cdr row) "category")))
            (amt (bi/row-amount row)))
          (let
            ((prev (dict-get acc cat)))
            (dict-set
              acc
              cat
              (as-str
                (if (null? prev) amt (+ (bi/parse-amount prev) amt)))))))
      "{}"
      (rows (quote bi-expense) 500))))

(define bi/metric/expense-total
  (lambda
    ()
    "Total of all expense rows."
    (bi/sum-amounts (rows (quote bi-expense) 500))))

(define bi/metric/profit
  (lambda
    ()
    "Revenue minus expenses."
    (- (bi/metric/revenue-total) (bi/metric/expense-total))))

(define bi/metric/profit-margin
  (lambda
    ()
    "Profit as a fraction of revenue; 0.0 when revenue is 0."
    (let
      ((rev (bi/metric/revenue-total)))
      (if (= rev 0.0) 0.0 (div (bi/metric/profit) rev)))))

(define bi/metric/revenue-total
  (lambda
    ()
    "Total of all revenue rows."
    (bi/sum-amounts (rows (quote bi-revenue) 500))))
```

154 libs/bible/lib.pp

```
;; package bible -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push

(define bible/abbrev-of
  (lambda
    (book)
    (let
      ((row (bible/book-row book)))
      (if (null? row) (as-str book) (dict-get row "abbrev"))))  "Canonical BSB abbreviation for a book (accepts a full name or an abbrev).")

(define bible/book-by-ordinal
  (lambda
    (ord)
    (list/foldl
      (lambda
        (acc row)
        (let
          ((f (cdr row)))
          (if
            (string-eq (as-str (dict-get f "ordinal")) (as-str ord))
            f
            acc)))
      ()
      (bible/books)))  "Look up a bible-book row by its canonical ordinal (1-66); () if out of range.")

(define bible/book-options
  (lambda
    (current)
    (list/foldl
      (lambda
        (acc row)
        (let
          ((f (cdr row)))
          (let
            ((name (dict-get f "name"))
              (abbrev (dict-get f "abbrev")))
            (string-append
              acc
              (html/option
                (as-str abbrev)
                (as-str name)
                (if
                  (string-eq (as-str current) (as-str abbrev))
                  #t
                  (string-eq (as-str current) (as-str name)))))))))
      ""
      (bible/books)))  "HTML <option> list of all books, with `current` selected (empty string selects none).")

(define bible/book-ordinal
  (lambda
    (book)
    (let
      ((row (bible/book-row book)))
      (if
        (null? row)
        0
        (bible/parse-int (dict-get row "ordinal")))))  "Canonical 1-66 ordinal of a book; 0 if the book is unknown.")

(define bible/book-row
  (lambda
    (book)
    (let
      ((b (as-str book)))
      (let
        ((by-ab (find-row (quote bible-book) "abbrev" b)))
        (if
          (null? by-ab)
```

```
          (find-row (quote bible-book) "name" b)
          by-ab))))  "Look up a bible-book row by name or abbreviation; () if not found.")

(define bible/books
  (lambda () (rows (quote bible-book) 100))  "All bible-book rows as (id . fields) pairs, one per book.")

(define bible/chapter
  (lambda
    (book chapter)
    (find-rows
      (quote bible-verse)
      "abbrev"
      (bible/abbrev-of book)
      "chapter"
      (as-str chapter)
      200
      "COALESCE((fields->>'verse')::int, 0) ASC, created_at ASC"))  "Return a chapter's bible-verse rows as (id . fields) pairs, ordered by verse number.")

(define bible/chapter-count
  (lambda
    (abbrev)
    (list/foldl
      (lambda
        (acc pair)
        (if
          (string-eq (car pair) (as-str abbrev))
          (car (cdr pair))
          acc))
      1
      bible/chapter-counts))  "Number of chapters a book contains; 0 if the book is unknown.")

(define bible/chapter-counts
  (list
    (list "Gen" 50)
    (list "Exod" 40)
    (list "Lev" 27)
    (list "Num" 36)
    (list "Deut" 34)
    (list "Josh" 24)
    (list "Judg" 21)
    (list "Ruth" 4)
    (list "1Sam" 31)
    (list "2Sam" 24)
    (list "1Kgs" 22)
    (list "2Kgs" 25)
    (list "1Chr" 29)
    (list "2Chr" 36)
    (list "Ezra" 10)
    (list "Neh" 13)
    (list "Esth" 10)
    (list "Job" 42)
    (list "Ps" 150)
    (list "Prov" 31)
    (list "Eccl" 12)
    (list "Song" 8)
    (list "Isa" 66)
    (list "Jer" 52)
    (list "Lam" 5)
    (list "Ezek" 48)
    (list "Dan" 12)
    (list "Hos" 14)
    (list "Joel" 3)
    (list "Amos" 9)
    (list "Obad" 1)
    (list "Jonah" 4)
    (list "Mic" 7)
    (list "Nah" 3)
    (list "Hab" 3)
```

```
    (list "Zeph" 3)
    (list "Hag" 2)
    (list "Zech" 14)
    (list "Mal" 4)
    (list "Matt" 28)
    (list "Mark" 16)
    (list "Luke" 24)
    (list "John" 21)
    (list "Acts" 28)
    (list "Rom" 16)
    (list "1Cor" 16)
    (list "2Cor" 13)
    (list "Gal" 6)
    (list "Eph" 6)
    (list "Phil" 4)
    (list "Col" 4)
    (list "1Thess" 5)
    (list "2Thess" 3)
    (list "1Tim" 6)
    (list "2Tim" 4)
    (list "Titus" 3)
    (list "Phlm" 1)
    (list "Heb" 13)
    (list "Jas" 5)
    (list "1Pet" 5)
    (list "2Pet" 3)
    (list "1John" 5)
    (list "2John" 1)
    (list "3John" 1)
    (list "Jude" 1)
    (list "Rev" 22))  "Map of abbrev -> chapter count for every book.")

(define bible/chapter-html
  (lambda
    (book chapter focus-verse)
    (bible/chapter-panel book chapter focus-verse))  "Chapter view HTML
    focusing a verse; alias of chapter-panel.")

(define bible/chapter-panel
  (lambda
    (book chapter focus-verse)
    (let
      ((rows (bible/chapter book chapter))
        (ab (bible/abbrev-of book))
        (ch (bible/safe-chapter book chapter)))
      (html/join
        (list
          (html/h1
            (string-append (as-str book) " " (as-str chapter)))
          (bible/nav-links ab ch)
          (html/tag
            "ol"
            (html/attrs
              (list
                (html/attr "class" "bible-chapter")
                (html/attr
                  "style"
                  "list-style:none;padding-left:0;margin:.25rem 0")))
            (list/foldl
              (lambda
                (acc row)
                (string-append acc (bible/verse-li row)))
              ""
              rows))
          (bible/nav-links ab ch)
          (if
            (null? focus-verse)
            ""
            (html/p*
              (html/attr "class" "muted")
              (string-append
                "Focus verse "
                (as-str focus-verse)
```

```
                ": "
                (as-str (bible/verse book chapter focus-verse)))))))))
    "Render a chapter: title, nav links, ordered verse <li>s (each verse
    number links to its study), focus note.")

(define bible/chapter-reader
  (lambda
    ()
    (let
      ((ref (request-param "ref"))
        (q (request-param "q"))
        (rnd (request-param "random")))
      (if
        (null? rnd)
        (if
          (null? q)
          (if
            (null? ref)
            (let
              ((book (request-param "book"))
                (chapter (request-param "chapter"))
                (verse (request-param "verse")))
              (let
                ((b (if (null? book) "John" (as-str book)))
                  (c
                    (bible/safe-chapter
                      (if (null? book) "John" book)
                      (if (null? chapter) "3" chapter)))
                  (v (if (null? verse) () verse)))
                (html/join
                  (list
                    (bible/go-form b c v)
                    (bible/search-form ())
                    (bible/chapter-panel b c v)
                    (html/p*
                      (html/attr "class" "muted")
                      (html/join
                        (list
                          "Berean Standard Bible ? "
                          (bible/ref b c (if (null? v) 1 v))
                          " ? "
                          (html/a "/strongs" "Strong's"))))))))
            (let
              ((row (bible/parse-ref ref)))
              (if
                (null? row)
                (html/join
                  (list
                    (bible/go-form "John" "3" ())
                    (html/p*
                      (html/attr "class" "err")
                      (string-append "Unknown ref " (as-str ref)))))
                (let
                  ((book (dict-get row "book"))
                    (chapter (dict-get row "chapter"))
                    (verse (dict-get row "verse")))
                  (html/join
                    (list
                      (bible/go-form book chapter verse)
                      (bible/search-form ())
                      (bible/chapter-panel book chapter verse)
                      (html/p*
                        (html/attr "class" "muted")
                        (string-append
                          "ref "
                          (as-str ref)
                          " ? Berean Standard Bible")))))))))
          (html/join
            (list
              (bible/go-form "John" "3" ())
              (bible/search-form q)
              (bible/search-panel q))))
        (let
```

```
          ((row (bible/random-row)))
          (if
            (null? row)
            (html/p* (html/attr "class" "err") "no verses seeded")
            (let
              ((f (cdr row)))
              (let
                ((book (dict-get f "abbrev"))
                  (chapter (dict-get f "chapter"))
                  (verse (dict-get f "verse")))
                (html/join
                  (list
                    (bible/go-form book chapter verse)
                    (bible/search-form ())
                    (bible/chapter-panel book chapter verse)
                    (html/p*
                      (html/attr "class" "muted")
                      (string-append
                        "random ? "
                        (as-str (dict-get f "ref"))))))))))))  "Request handler for the /dabar chapter reader (ref, q, or random params). Returns an HTML panel.")

(define bible/go-form
  (lambda
    (b c verse)
    (let
      ((ab (bible/abbrev-of b))
        (ch (bible/safe-chapter b c))
        (vs (if (null? verse) "" (as-str verse))))
      (html/form
        "GET"
        "/dabar"
        (html/attrs
          (list
            (html/attr "class" "card bible-go")
            (html/attr "hx-get" "/dabar")
            (html/attr "hx-target" "#dabar-panel")
            (html/attr "hx-swap" "innerHTML")
            (html/attr "hx-push-url" "true")
            (html/attr "hx-trigger" "submit, change from:select")))
        (html/join
          (list
            (html/label
              ""
              (html/join
                (list
                  "Book "
                  (html/select "book" "" (bible/book-options ab)))))
            " "
            (html/label
              ""
              (html/join
                (list
                  "Chapter "
                  (html/select
                    "chapter"
                    ""
                    (bible/option-range (bible/chapter-count ab) ch)))))
            " "
            (html/label
              ""
              (html/join
                (list
                  "Verse "
                  (html/select "verse" "" (bible/verse-options ab ch vs)))))
            " "
            (html/button (html/attr "type" "submit") "Go")))))) "Book/chapter/verse jump form HTML targeting #dabar-panel.")

(define bible/href-chapter
  (lambda
    (pair)
```

```
    (if
      (null? pair)
      "#"
      (string-append
        "/dabar?book="
        (as-str (car pair))
        "&chapter="
        (as-str (car (cdr pair))))))  "/dabar URL for a (book-abbrev
    chapter) pair from prev-ref/next-ref; '#' when the pair is null.")

;; notes.links-and-roam / dabar.note-bible-links: sexp bible refs ? /dabar
(define bible/link-href
  (lambda
    (book chapter verse)
    (let
      ((ab (bible/abbrev-of book))
        (ch (bible/safe-chapter book chapter))
        (v (if (null? verse) "" (as-str verse))))
      (if
        (string-eq v "")
        (string-append "/dabar?book=" ab "&chapter=" ch)
        (string-append
          "/dabar?book="
          ab
          "&chapter="
          ch
          "&verse="
          v))))  "/dabar URL for a book (optionally with chapter and
    verse).")

(define bible/link-html
  (lambda
    (book chapter verse label)
    (let
      ((ab (bible/abbrev-of book))
        (ch (bible/safe-chapter book chapter))
        (v (if (null? verse) "" (as-str verse)))
        (lab
          (if
            (or (null? label) (string-eq (as-str label) ""))
            (if
              (or (null? verse) (string-eq (as-str verse) ""))
              (string-append (as-str book) " " (as-str chapter))
              (string-append
                (as-str book)
                " "
                (as-str chapter)
                ":"
                (as-str verse)))
            (as-str label))))
      (html/a
        (bible/link-href ab ch (if (string-eq v "") () v))
        (html/text lab))))  "<a> element linking to a verse or chapter;
    derives a label from book chapter(:verse) when none is given.")

(define bible/int-in-range-from?
  (lambda
    (s lo hi i)
    (if
      (> i hi)
      #f
      (if
        (string-eq (as-str s) (as-str i))
        #t
        (bible/int-in-range-from? s lo hi (+ i 1))))))  "Worker for
    int-in-range?: true if int-string s equals a value in [lo,hi],
    scanning i upward.")

(define bible/int-in-range?
  (lambda (s lo hi) (bible/int-in-range-from? s lo hi lo))  "True if
    int-string s equals an integer in [lo, hi].")

(define bible/nav-links
```

```
  (lambda
    (book chapter)
    (let
      ((prev (bible/prev-ref book chapter))
        (next (bible/next-ref book chapter)))
      (html/p*
        (html/attr "class" "bible-nav")
        (html/join
          (list
            (if
              (null? prev)
              (html/span
                (html/attr "class" "muted")
                "? prev")
              (html/a (bible/href-chapter prev) "? prev"))
            " ? "
            (html/a "/dabar?random=1" "random")
            " ? "
            (if
              (null? next)
              (html/span
                (html/attr "class" "muted")
                "next ?")
              (html/a (bible/href-chapter next) "next ?"))
            " ? "
            (html/a
              (string-append
                "/dabar?book="
                (bible/abbrev-of book)
                "&chapter="
                (bible/safe-chapter book chapter)
                "&verse=1")
              "study"))))))  "Prev/next/random/study chapter navigation
    links HTML for a book and chapter.")

(define bible/next-ref
  (lambda
    (book chapter)
    (let
      ((ab (bible/abbrev-of book))
        (c (bible/parse-int (bible/safe-chapter book chapter)))
        (maxc (bible/chapter-count (bible/abbrev-of book)))
        (ord (bible/book-ordinal book)))
      (if
        (< c maxc)
        (list ab (as-str (+ c 1)))
        (if
          (< ord 66)
          (let
            ((nxt (bible/book-by-ordinal (+ ord 1))))
            (if (null? nxt) () (list (dict-get nxt "abbrev") "1")))
          ())))))  "(book-abbrev next-chapter) pair after a chapter,
    crossing into the next book; () at the last book.")

(define bible/option-range
  (lambda
    (n selected)
    (bible/option-range-from 1 n selected))  "HTML <option>s for chapter
    numbers 1..n, with ‘selected‘ highlighted.")

(define bible/option-range-from
  (lambda
    (i n selected)
    (if
      (> i n)
      ""
      (string-append
        (html/option
          (as-str i)
          (as-str i)
          (string-eq (as-str selected) (as-str i)))
        (bible/option-range-from (+ i 1) n selected))))  "Worker for
    option-range: emit options i..n, selected highlighted.")
```

```
485
486 (define bible/parse-int
487   (lambda (s) (bible/parse-int-from s 1 200))  "Parse an int-string in the
        default 1..200 range; 0 when not a valid number.")
488
489 (define bible/parse-int-from
490   (lambda
491     (s i max)
492     (if
493       (> i max)
494       0
495       (if
496         (string-eq (as-str s) (as-str i))
497         i
498         (bible/parse-int-from s (+ i 1) max))))  "Worker for parse-int: 1
      if s equals i scanning 1..max, else 0.")
499
500 (define bible/parse-ref
501   (lambda
502     (ref)
503     (find-row (quote bible-verse) "ref" (as-str ref)))  "Parse a BSB ref
      string into a bible-verse fields JSON, or () if unknown.")
504
505 (define bible/prev-ref
506   (lambda
507     (book chapter)
508     (let
509       ((ab (bible/abbrev-of book))
510         (c (bible/parse-int (bible/safe-chapter book chapter)))
511         (ord (bible/book-ordinal book)))
512       (if
513         (> c 1)
514         (list ab (as-str (- c 1)))
515         (if
516           (> ord 1)
517           (let
518             ((prev (bible/book-by-ordinal (- ord 1))))
519             (if
520               (null? prev)
521               ()
522               (list
523                 (dict-get prev "abbrev")
524                 (as-str (bible/chapter-count (dict-get prev "abbrev"))))))
525           ()))))  "(book-abbrev prev-chapter) pair before a chapter,
      crossing into the previous book; () at the first book.")
526
527 (define bible/random-row
528   (lambda () (find-random (quote bible-verse)))  "A random bible-verse row
        as (id . fields), or () if none.")
529
530 (define bible/ref
531   (lambda
532     (book chapter verse)
533     (string-append
534       "BSB:"
535       (bible/abbrev-of book)
536       ":"
537       (as-str chapter)
538       ":"
539       (as-str verse)))  "Canonical BSB ref string, e.g. 'BSB:John:3:16'.")
540
541 (define bible/safe-chapter
542   (lambda
543     (book chapter)
544     (let
545       ((max (bible/chapter-count (bible/abbrev-of book)))
546         (c (if (null? chapter) "1" (as-str chapter))))
547       (if (bible/int-in-range? c 1 max) c "1")))  "Clamp a chapter to a
      book's valid range, defaulting to 1.")
548
549 (define bible/search
550   (lambda
551     (q)
```

```
    (find-text (quote bible-verse) "text" (as-str q) 40))  "Find up to 40
    bible-verse rows whose text contains q.")

(define bible/search-form
  (lambda
    (q)
    (html/form
      "GET"
      "/dabar"
      (html/attrs
        (list
          (html/attr "class" "card bible-search")
          (html/attr "hx-get" "/dabar")
          (html/attr "hx-target" "#dabar-panel")
          (html/attr "hx-swap" "innerHTML")
          (html/attr "hx-push-url" "true")))
      (html/join
        (list
          (html/label
            ""
            (html/join
              (list
                "Search "
                (html/text-input
                  "q"
                  (html/attrs
                    (list
                      (html/attr "value" (if (null? q) "" (as-str q)))
                      (html/attr "placeholder" "love")))))))
          " "
          (html/button (html/attr "type" "submit") "Find"))))  "Search
    form HTML posting q to /dabar.")

(define bible/search-panel
  (lambda
    (q)
    (let
      ((hits (bible/search q)))
      (html/join
        (list
          (html/h1 (string-append "Search: " (as-str q)))
          (html/ul
            (html/attr "class" "bible-hits")
            (list/foldl
              (lambda
                (acc row)
                (let
                  ((f (cdr row)))
                  (string-append
                    acc
                    (html/li
                      ""
                      (html/join
                        (list
                          (html/a
                            (string-append
                              "/dabar?ref="
                              (as-str (dict-get f "ref")))
                            (as-str (dict-get f "ref")))
                          " "
                          (html/text (dict-get f "text")))))))))
              ""
              hits))
          (html/p*
            (html/attr "class" "muted")
            "Berean Standard Bible"))))  "Search results panel HTML for a
    query.")

(define bible/verse
  (lambda
    (book chapter verse)
    (let
      ((row (bible/verse-row book chapter verse)))
```

```
      (if (null? row) () (dict-get (cdr row) "text"))))
  "Return the BSB text of one verse, or () if missing.")

(define bible/verse-li
  (lambda
    (row)
    (let
      ((fields (cdr row)))
      (let
        ((n (dict-get fields "verse"))
          (text (dict-get fields "text"))
          (ref (dict-get fields "ref")))
        (let
          ((href
            (string-append
              "/dabar?book=" (as-str (dict-get fields "abbrev"))
              "&chapter=" (as-str (dict-get fields "chapter"))
              "&verse=" (as-str n))))
          (html/li
            (html/attr "id" (as-str ref))
            (html/join
              (list
                (html/tag "sup" "" (html/a href (html/text (as-str n))))
                " "
                (html/text text))))))))  "One <li> for a chapter row:
    superscript verse-number study link plus the text.")

(define bible/verse-options
  (lambda
    (book chapter selected)
    (string-append
      (html/option
        ""
        "--"
        (if
          (null? selected)
          #t
          (string-eq (as-str selected) "")))
      (list/foldl
        (lambda
          (acc row)
          (let
            ((n (dict-get (cdr row) "verse")))
            (string-append
              acc
              (html/option
                (as-str n)
                (as-str n)
                (string-eq (as-str selected) (as-str n))))))
        ""
        (bible/chapter book chapter))))  "HTML <option>s for every verse in
    a chapter with ‘selected‘ highlighted (plus a blank ’--’ entry).")

(define bible/verse-row
  (lambda
    (book chapter verse)
    (let
      ((rows
        (find-rows
          (quote bible-verse)
          "ref"
          (bible/ref book chapter verse)
          1)))
      (if (null? rows) () (car rows))))
  "Fetch one bible-verse row as (id . fields) pair, or () if not found.
     Matches find-rows shape; composes with bible/verse-li.")


;; --- outline body (bible-ref ...) rendering ---

(define bible/nth
  (lambda (lst n)
    (if (= n 0) (car lst) (bible/nth (cdr lst) (- n 1)))))
```

```
  "Nth element of a list (0-indexed).")

(define bible/ref-body?
  (lambda (body)
    (let
      ((s (as-str body)))
      (string-starts-with? s "(bible-ref")))
  "True if an outline body string starts with '(bible-ref'.")

(define bible/ref-parse
  (lambda (body)
    (let
      ((form (read-string (as-str body))))
      (if
        (pair? form)
        (if
          (string-eq (symbol->string (car form)) "bible-ref")
          form
          ())
        ()))))
  "Parse a (bible-ref B ch vstart vend MODE) body string into its list
    form, or ().")

(define bible/render-body
  (lambda (body)
    (if
      (bible/ref-body? body)
      (let
        ((form (bible/ref-parse body)))
        (if
          (null? form)
          (roam/render-body body)
          (let
            ((book (bible/nth form 1))
              (chapter (bible/nth form 2))
              (vstart (bible/nth form 3))
              (vend (bible/nth form 4))
              (mode (bible/nth form 5)))
            (bible/render-ref book chapter vstart vend mode))))
      ())))
  "Render an outline body string: bible-ref forms render as link/inline
    HTML, otherwise roam/render-body.")

(define bible/render-ref
  (lambda (book chapter vstart vend mode)
    (if
      (string-eq (as-str mode) "link")
      (bible/verse-link-html book chapter vstart vend)
      (bible/verse-inline-html book chapter vstart vend)))
  "Dispatch a parsed (bible-ref ...) to link or inline HTML according to
    mode.")

(define bible/cite-label
  (lambda (book chapter vstart vend)
    (if
      (= vstart vend)
      (string-append
        (as-str book)
        " "
        (as-str chapter)
        ":"
        (as-str vstart))
      (string-append
        (as-str book)
        " "
        (as-str chapter)
        ":"
        (as-str vstart)
        "-"
        (as-str vend))))
  "Human citation label, e.g. 'John 3:16' or 'John 3:16-18'.")

(define bible/verse-body
```

```
  (lambda (book chapter vstart vend mode)
    (string-append
      "(bible-ref \""
      (as-str book)
      "\" "
      (as-str chapter)
      " "
      (as-str vstart)
      " "
      (as-str vend)
      " \""
      (as-str mode)
      "\")"))
  "Outline body string for a (bible-ref B ch vstart vend MODE) range.")

(define bible/verse-inline-html
  (lambda (book chapter vstart vend)
    (if
      (= vstart vend)
      (bible/verse-inline-one book chapter vstart)
      (bible/verse-inline-range book chapter vstart vend)))
  "Blockquote HTML for a single verse or a contiguous verse range.")

(define bible/verse-inline-one
  (lambda (book chapter verse)
    (let
      ((row (bible/verse-row book chapter verse))
        (ab (bible/abbrev-of book))
        (ch (bible/safe-chapter book chapter))
        (label (bible/cite-label book chapter verse verse)))
      (if
        (null? row)
        (html/p*
          (html/attr "class" "err")
          (string-append "Verse not found: " label))
        (html/tag
          "blockquote"
          (html/attr "class" "bible-verse")
          (html/join
            (list
              (html/tag "sup" "" (as-str verse))
              " "
              (html/text (dict-get (cdr row) "text"))
              " "
              (html/p*
                (html/attr "class" "bible-cite")
                (html/a (bible/link-href ab ch verse) label))))))))
  "Render one verse as a blockquote with cite link, or 'Verse not found:
     <label>' if missing.")

(define bible/verse-inline-range
  (lambda (book chapter vstart vend)
    (let
      ((rows (bible/verses-in-range book chapter vstart vend))
        (ab (bible/abbrev-of book))
        (ch (bible/safe-chapter book chapter))
        (label (bible/cite-label book chapter vstart vend)))
      (if
        (null? rows)
        (html/p*
          (html/attr "class" "err")
          (string-append "No verses found: " label))
        (html/tag
          "blockquote"
          (html/attr "class" "bible-verse")
          (html/join
            (list
              (list/foldl
                (lambda
                  (acc row)
                  (let
                    ((fields (cdr row)))
                    (string-append
```

```
                      acc
                      (html/tag "sup" "" (as-str (dict-get fields "verse")))
                      " "
                      (html/text (dict-get fields "text"))
                      " ")))
                "")
                rows)
              (html/p*
                (html/attr "class" "bible-cite")
                (html/a (bible/link-href ab ch vstart) label))))))))
  "Blockquote HTML rendering a contiguous verse range.")

(define bible/verse-link-html
  (lambda (book chapter vstart vend)
    (let
      ((ab (bible/abbrev-of book))
        (ch (bible/safe-chapter book chapter))
        (label (bible/cite-label book chapter vstart vend)))
      (html/a (bible/link-href ab ch vstart) label)))
  "<a> link HTML to a verse or chapter with a citation label.")

(define bible/verses-in-range
  (lambda (book chapter vstart vend)
    (let
      ((rows (bible/chapter book chapter)))
      (list/filter
        (lambda
          (row)
          (let
            ((v (parse-int (dict-get (cdr row) "verse"))))
            (and (>= v vstart) (<= v vend))))
        rows)))
  "Rows of a chapter filtered to verse numbers in [vstart, vend].")

(define bible/insert-verse
  (lambda (book chapter vstart vend mode)
    (outline/node*
      (gensym "n")
      (bible/cite-label book chapter vstart vend)
      (bible/verse-body book chapter vstart vend mode)
      ""
      (list)))
  "Emit a (bible-ref ...) outline node for a verse range with the given
    render mode.")


(define bible/css
  (lambda
    ()
    "Render the bible component stylesheet (extracted from ui/theme)."
    (css/sheet
      (list
        (css/rule
          "ol.bible-chapter"
          (list
            "list-style"
            "none"
            "padding-left"
            "0"
            "margin"
            ".25rem 0 1rem"))
        (css/rule
          "ol.bible-chapter>li"
          (list
            "margin"
            ".35rem 0"
            "line-height"
            "1.45"
            "padding-left"
            "0"))
        (css/rule
          "ol.bible-chapter>li>sup"
          (list
```

```
            "font-size"
            ".75em"
            "font-weight"
            "700"
            "margin-right"
            ".2em"
            "color"
            "var(--muted)"
            "vertical-align"
            "super"))
        (css/rule
          ".bible-go label,.bible-search label"
          (list
            "display"
            "inline-block"
            "margin"
            ".2rem .45rem .2rem 0"))
        (css/rule
          ".bible-go select,.bible-search input[type=text]"
          (list
            "width"
            "auto"
            "min-width"
            "6rem"
            "max-width"
            "100%"
            "margin-right"
            ".35rem"))))))

(define bible/css
  (lambda
    ()
    "Render the bible component stylesheet (extracted from ui/theme)."
    (css/sheet
      (list
        (css/rule
          "ol.bible-chapter"
          (list
            "list-style"
            "none"
            "padding-left"
            "0"
            "margin"
            ".25rem 0 1rem"))
        (css/rule
          "ol.bible-chapter>li"
          (list
            "margin"
            ".35rem 0"
            "line-height"
            "1.45"
            "padding-left"
            "0"))
        (css/rule
          "ol.bible-chapter>li>sup"
          (list
            "font-size"
            ".75em"
            "font-weight"
            "700"
            "margin-right"
            ".2em"
            "color"
            "var(--muted)"
            "vertical-align"
            "super"))
        (css/rule
          ".bible-go label,.bible-search label"
          (list
            "display"
            "inline-block"
            "margin"
            ".2rem .45rem .2rem 0"))
```

```
        (css/rule
          ".bible-go select,.bible-search input[type=text]"
          (list
            "width"
            "auto"
            "min-width"
            "6rem"
            "max-width"
            "100%"
            "margin-right"
            ".35rem"))))))
```

155 libs/book/lib.pp

```
;; package book -- the spec IS the book: borg chapters as live habitat pages.
;;
;; Spec: borge/habitat-book.borg (habitat-loader rows in; web-renderer out).
;; Rows are seeded by scripts/load-book.sh from `borge book export` under
;; data.book (type book-chapter, branch-free data.* namespace -- git stays
;; authoritative for .borg, habitat-book decision 2). This package RENDERS
;; rows; it never parses .borg (one parser, two projections -- borge stays
;; the only parser, decision 3).
;;
;; Kernel-dispatched routes (image/lib/http.ml, no route pins needed):
;;   GET /book                  -> book/page        merged index: borg chapters + package specs
;;   GET /book/chapter/<label>  -> book/chapter-page
;;   GET /book/code/<label>     -> book/code-page
;;   GET /book/course/<suffix>  -> book/course-page notes/outline course doc (kind course)
;;   GET /book/<pkg>            -> spec/book-page   (kernel fallthrough, unchanged)
;;   GET /book/pkg/<pkg>        -> alias of /book/<pkg>
;;
;; Registers (decision 1 -- mode is a render flag, never a document fork):
;;   reader (default): prose/headings/code/examples only.
;;   ?mode=workshop:   + status badges, verify blocks, agent notes,
;;                     untracked lines, unknown-node listings.
;;   Drift holes stay visible in BOTH registers (raw parse-failure
;;   listings, unresolved binding tokens).
;;
;; Depends (boot-seeded): std (string-split), html, list. The binding-token
;; machinery is REUSED from spec when loaded (spec/token-binding?,
;; spec/binding-resolves?) -- the proven drift renderer is inherited, not
;; forked; without spec, prose renders plain (escaped) and everything else
;; works. Example blocks post to /book/eval-example -- the run button goes
;; live with the example-eval prim (habitat-book eval-examples, P2.3).

;; ------------------------------------------------------------------ rows

(define book/row
  (lambda (path)
    (let ((n (tree/get path)))
      (if (null? n) () (dict-get n "body"))))
  "Body JSON (VStr) of the book tree row at path, or nil when absent.")

(define book/f
  (lambda (body field)
    (if (null? body) () (dict-get body field)))
  "Field of a row body JSON, or nil. dict-get maps JSON arrays/objects
   back to VStr JSON -- wrap with json-parse for lists.")

(define book/row-path
  (lambda (node)
    (as-str (dict-get node "path")))
  "Tree path of one node-JSON row (tree/children-of element).")

(define book/children
  (lambda (parent-path)
    (tree/children-of parent-path))
  "Document-order children of a book row (rank order; tree/children-of's
   literal-h covering reaches chapter.h.<section> and section.h.<sub>).")

(define book/label-safe?
  (lambda (s)
    (book/label-chars? s 0 (string-length s)))
  "Nonempty [a-z0-9_]-only string (ltree label shape the loader writes).")

(define book/label-chars?
  (lambda (s i n)
    (if (= i n)
        (> n 0)
        (let ((c (substring s i 1)))
          (if (or (string-eq c "_")
```

```
                  (and (string>=? c "a") (string<=? c "z"))
                  (and (string>=? c "0") (string<=? c "9")))
              (book/label-chars? s (+ i 1) n)
              #f))))
  "Char walk for label-safe?.")

;; ---------------------------------------------------------------- request

(define book/mode
  (lambda ()
    (let ((m (request-param "mode")))
      (if (and (not (null? m)) (string-eq (as-str m) "workshop"))
          "workshop"
          "reader")))
  "Register of the active request: workshop when ?mode=workshop, else reader.")

(define book/path-suffix
  (lambda (prefix path)
    (let ((p (as-str path)))
      (let ((pre (as-str prefix)))
        (if (string-starts-with? p pre)
            (substring p (string-length pre) (- (string-length p) (string-length pre)))
            "")))))
  "Text after prefix, or empty when path lacks the prefix.")

(define book/fn-bound?
  (lambda (name)
    (not (null? (function-source name))))
  "True when a library binding has a function_def on the visible lens.")

;; ---------------------------------------------------------------- html bits

(define book/style-html
  (lambda ()
    (html/tag
      "style"
      ""
      (string-append
        ".book-example{border:1px solid #3a3f44;border-radius:.5rem;"
        "padding:.5rem .6rem;margin:.7rem 0;background:#101418}"
        ".book-example pre{margin:.2rem 0 .4rem 0;overflow-x:auto}"
        ".book-expected{color:#8a8f98;font-size:.85rem;margin:.1rem 0}"
        "details.book-verify{border:1px dashed #555;border-radius:.4rem;"
        "padding:.3rem .5rem;margin:.4rem 0}"
        "details.book-verify summary{cursor:pointer;color:#9aa0a6}"
        "blockquote.book-note{border-left:3px solid #666;margin:.5rem 0;"
        "padding:.1rem .8rem;color:#b9bdc4}"
        ".book-hole{color:#f88;border-bottom:2px dashed #c44}"
        ".book-result{border-top:1px dashed #333;margin-top:.4rem;"
        "padding-top:.3rem;color:#cde;font-size:.9rem}"
        ".book-elapsed{color:#667;font-size:.75rem}"
        ".book-error{color:#f88;margin-top:.3rem;font-size:.85rem}"
        ".book-hole-row{border:1px solid #c44;border-radius:.4rem;"
        "padding:.5rem .6rem;margin:.6rem 0;color:#f88}"
        ".book-hole-row pre{color:#cde;margin:.3rem 0}")))
  "Book-page styles (phone-honest; print-considerate by restraint).")

(define book/status-color
  (lambda (st)
    (let ((s (as-str st)))
      (if (string-eq s "implemented") "#4a9"
          (if (string-eq s "planned") "#c93"
              (if (string-eq s "deprecated") "#a66" "#888"))))))
  "Badge color for a borge status.")

(define book/status-badge
  (lambda (st)
    (if (null? st)
        ""
        (html/span
```

```
          (html/attrs
            (list
              (html/attr "class" "book-status")
              (html/attr "style"
                         (string-append
                           "font-size:.7rem;border:1px solid #666;"
                           "border-radius:999px;padding:.05rem .5rem;color:"
                           (book/status-color st)))))
          (as-str st))))
  "Small status pill (workshop register only).")

(define book/toggle-html
  (lambda (base mode)
    (if (string-eq mode "workshop")
        (html/p*
          (html/attr "class" "muted")
          (html/join
            (list "workshop register ? "
                  (html/a (string-append base "?mode=reader") "reader view"))))
        (html/p*
          (html/attr "class" "muted")
          (html/join
            (list "reader register ? "
                  (html/a (string-append base "?mode=workshop") "workshop view"))))))
  "The register toggle -- the 'spec vs textbook' switch made tangible.")

(define book/not-found
  (lambda (what)
    (status-response
      404
      (string-append
        "<h1>not found</h1><p>No " (html-escape (as-str what))
        " in the seeded book.</p>"
        "<p><a href=\"/book\">back to /book</a></p>")))
  "404 status-response for unknown book paths.")

;; ------------------------------------------------------------------ prose
;;
;; DECISION (2026-08-30, P2.2 dogfood -- obituary): this section first
;; rendered prose with an interpreted per-word pipeline (book/split-piece
;; + book/piece-html via list/map, calling spec/token-binding? and
;; spec/binding-resolves? per word). That cost ~300 interpreter steps per
;; word -- a 12.7k-word chapter needed ~4M steps against the 100k-step
;; page budget, so every chapter page died with "budget exceeded". The
;; per-word work now lives in TWO kernel scans (pure/prims_pure.ml):
;; (prose-tokens s) collects UNIQUE token-shaped cores in one OCaml pass;
;; (render-tokens s resolved holes) renders one paragraph in one pass.
;; Resolution STAYS in Nopales via spec/binding-resolves? (function-source)
;; at O(unique tokens), not O(words) -- the drift decision is still made
;; by the proven spec machinery. The kernel shape test mirrors
;; spec/token-binding?; spec/ keeps the authoritative definition -- keep
;; the two in lockstep (parity-tested in image/test/primitives_test.ml).
;; book/split-piece + book/piece-html + the has-prefix?/has-suffix?
;; helpers are retired with this change.

(define book/binding-classes
  (lambda (prose)
    (let ((toks (prose-tokens (as-str prose))))
      (list
        (list/filter spec/binding-resolves? toks)
        (list/filter (lambda (n) (not (spec/binding-resolves? n))) toks))))
  "(resolved holes) over prose's UNIQUE token-shaped cores: one kernel
   scan + O(unique tokens) function-source lookups, never O(words).")

(define book/para-html
  (lambda (para link? classes)
    (html/p*
      ""
      (if link?
          (render-tokens
            (as-str para) (car classes) (car (cdr classes)))
```

```
          (html-escape (as-str para)))))
  "One prose paragraph: kernel render-tokens (links + holes) when the
   spec machinery is loaded; plain escaping otherwise. html/p* keeps
   the kernel's HTML raw; the plain branch escapes manually (html/p
   would double-escape the render-tokens output).")

(define book/prose-html
  (lambda (prose link?)
    (let ((p (as-str prose)))
      (if (string-eq p "")
          ""
          (let ((classes (if link? (book/binding-classes p) (list () ()))))
            (html/join
              (list/map
                (lambda (para) (book/para-html para link? classes))
                (string-split p "\n\n")))))))
  "Prose text as paragraphs; binding tokens link/hole via the kernel
   render-tokens pass (drift-as-rendering inherited from spec).")

;; ----------------------------------------------------------------
     scaffolding

(define book/verify-html
  (lambda (verify)
    (if (null? verify)
        ""
        (let ((items (json-parse (as-str verify))))
          (html/tag
            "details"
            (html/attr "class" "book-verify")
            (html/join
              (list
                (html/tag "summary" ""
                          (string-append "verify (" (as-str (count items))
    ")"))
                (html/pre
                  (html-escape
                    (html/join
                      (list/map
                        (lambda (v) (string-append (as-str v) "\n"))
                        items)))))))))
  "Workshop: verify entries as a collapsed listing.")

(define book/notes-html
  (lambda (notes)
    (if (null? notes)
        ""
        (html/join
          (list/map
            (lambda (n)
              (html/tag
                "blockquote"
                (html/attr "class" "book-note")
                (html/join
                  (list
                    (html/p*
                      (html/attr "class" "muted")
                      (string-append
                        "agent note ("
                        (html-escape (as-str (dict-get n "by"))) ")"))
                    (html/p (as-str (dict-get n "body")))))))
            (json-parse (as-str notes))))))
  "Workshop: agent notes as blockquotes (html/p escapes its own body --
   no pre-escape, that would double-escape).")

(define book/untracked-html
  (lambda (untracked)
    (if (null? untracked)
        ""
        (html/tag
          "p"
          (html/attr "class" "muted")
          (html/tag
```

```
            "em"
            ""
            (html/join
              (list/map
                (lambda (u)
                  (html-escape
                    (string-append
                      "[untracked " (as-str (dict-get u "path")) ": "
                      (as-str (dict-get u "reason")) "] ")))
                (json-parse (as-str untracked))))))))
  "Workshop: untracked-path notes as italic lines.")

(define book/other-html
  (lambda (other)
    (if (null? other)
        ""
        (html/join
          (list/map
            (lambda (o)
              (html/div
                (html/attr "class" "book-other")
                (html/join
                  (list
                    (html/p*
                      (html/attr "class" "muted")
                      (string-append "unknown node kind: "
                                     (html-escape (as-str (dict-get o
    "kind")))))
                    (html/pre (html-escape (as-str (dict-get o
    "body")))))))))
            (json-parse (as-str other))))))
  "Workshop: unknown node kinds the loader could not classify (never
   silently dropped -- the row body is the source of truth).")

(define book/raw-html
  (lambda (raw)
    (if (null? raw)
        ""
        (html/div
          (html/attr "class" "book-raw")
          (html/join
            (list
              (html/p*
                (html/attr "class" "muted")
                "unparsed source (borge could not structure this -- drift
    hole)")
              (html/pre (html-escape (as-str raw))))))))
  "BOTH registers: the raw fallback is a drift hole, never stripped.")

(define book/workshop-html
  (lambda (b mode)
    (if (string-eq mode "workshop")
        (html/join
          (list
            " "
            (book/status-badge (book/f b "status"))
            (book/verify-html (book/f b "verify"))
            (book/notes-html (book/f b "notes"))
            (book/untracked-html (book/f b "untracked"))
            (book/other-html (book/f b "other"))))
        "")
    )
  "Workshop-register scaffolding block for a row body (status/verify/
   notes/untracked/other). Reader strips all of it.")

;; ---------------------------------------------------------------- chapters

(define book/example-html
  (lambda (node mode)
    (let ((b (dict-get node "body")))
      (let ((lang (book/f b "lang"))
            (src (book/f b "src"))
            (expected (book/f b "expected")))
```

```
        (html/div
          (html/attr "class" "book-example")
          (html/join
            (list
              (html/pre (html/code (as-str src)))
              (if (string-eq (as-str lang) "nopales")
                  (html/button
                    (html/attrs
                      (list
                        (html/attr "class" "book-run")
                        (html/attr "hx-post" "/book/eval-example")
                        (html/attr "hx-vals"
                                   (string-append
                                     "{\"path\":\""
                                     (book/row-path node)
                                     "\"}"))
                        (html/attr "hx-target" "closest div")
                        (html/attr "hx-swap" "outerHTML")))
                    "run")
                  (html/span
                    (html/attr "class" "muted")
                    "captured, not live"))
              (if (null? expected)
                  ""
                  (html/p*
                    (html/attr "class" "book-expected")
                    (html-escape
                      (string-append "expect => " (as-str
    expected)))))))))))
  "Example block: nopales gets a run button (POST /book/eval-example,
   live with the example-eval prim); ocaml is captured output only
   (habitat-book eval-examples: no live OCaml evaluator in v1).")

(define book/section-html
  (lambda (node mode)
    (let ((b (dict-get node "body")))
      (let ((title (book/f b "title"))
            (level (as-str (book/f b "level")))
            (prose (book/f b "prose"))
            (raw (book/f b "raw")))
        (let ((tag (if (string-eq level "2") "h2"
                       (if (string-eq level "3") "h3" "h4"))))
          (html/join
            (list
              (html/tag tag "" (html-escape (as-str title)))
              (book/workshop-html b mode)
              (book/raw-html raw)
              (book/prose-html prose (book/fn-bound? "spec/token-binding?"))
              (book/children-html (book/row-path node) mode)))))))
  "One section/subsection: heading, register scaffolding, prose, then
   nested children (document order via tree rank).")

(define book/children-html
  (lambda (parent-path mode)
    (html/join
      (list/map
        (lambda (node)
          (let ((b (dict-get node "body")))
            (if (string-eq (as-str (dict-get b "node")) "example")
                (book/example-html node mode)
                (book/section-html node mode))))
        (book/children parent-path))))
  "Document-order children of a book row: sections (recursive) + examples.")

(define book/chapter-html
  (lambda (label path b mode)
    (let ((title (book/f b "title"))
          (prose (book/f b "prose"))
          (raw (book/f b "raw")))
      (html/page
        (list
          (book/style-html)
          (html/p*
```

```
            (html/attr "class" "muted")
            (html/join (list (html/a "/book" "book") " ? chapter")))
          (book/toggle-html (string-append "/book/chapter/" label) mode)
          (html/h1 (html-escape (as-str title)))
          (book/workshop-html b mode)
          (book/raw-html raw)
          (book/prose-html prose (book/fn-bound? "spec/token-binding?"))
          (book/children-html path mode)
          (html/p*
            (html/attr "class" "muted")
            (html-escape
              (string-append
                "chapter " (as-str (book/f b "chapter"))
                " ? seeded rows (git is authoritative for .borg)")))))))
  "One borg chapter page from tree rows.")

(define book/chapter-page
  (lambda ()
    (let ((label (book/path-suffix "/book/chapter/" (request-path))))
      (if (not (book/label-safe? label))
          (book/not-found "chapter")
          (let ((path (string-append "data.book.ch." label)))
            (let ((b (book/row path)))
              (if (null? b)
                  (book/not-found (string-append "chapter " label))
                  (book/chapter-html
                    label path b (book/mode))))))))
  "GET /book/chapter/<label> -- headings/prose/code/examples from rows.")

;; ------------------------------------------------------------------ code

(define book/code-page
  (lambda ()
    (let ((label (book/path-suffix "/book/code/" (request-path))))
      (if (not (book/label-safe? label))
          (book/not-found "code chapter")
          (let ((b (book/row (string-append "data.book.code." label))))
            (if (null? b)
                (book/not-found (string-append "code chapter " label))
                (let ((p (book/f b "path"))
                      (lang (book/f b "lang"))
                      (src (book/f b "source")))
                  (html/page
                    (list
                      (book/style-html)
                      (html/p*
                        (html/attr "class" "muted")
                        (html/join
                          (list (html/a "/book" "book")
                                " ? code appendix")))
                      (html/h1 (html-escape (as-str p)))
                      (html/p*
                        (html/attr "class" "muted")
                        (string-append "lang: " (as-str lang)))
                      (html/pre (html-escape (as-str src))))))))))))
  "GET /book/code/<label> -- one code-chapter listing from its row.")

;; ------------------------------------------------------------------ courses
;;
;; habitat-book.course-documents: a course is a NOTES/OUTLINE doc (slug
;; course/<suffix>, kind course -- the spec/<pkg> slash-slug pattern)
;; whose reading nodes reference chapter labels -- compose, never copy
;; (decision 5). The renderer assembles the doc's outline into an
;; ordered reading list over the seeded chapter rows; the chapter rows
;; themselves never move (data.book.ch.* stays a chapter's ONE home --
;; two courses referencing one chapter duplicate nothing in the tree).
;; A reading node's body starts with a `chapter: <label>` line (the
;; machine reference, book/label-safe? shape); the rest renders as
;; description prose. Status badges show in BOTH registers here: the
;; reading list is itself a status view (spec smoke), unlike
;; chapter-index rows (workshop-only).

(define book/course-suffix-chars?
```

```
  (lambda (s i n)
    (if (= i n)
        (> n 0)
        (let ((c (substring s i 1)))
          (if (or (string-eq c "_") (string-eq c "-")
                  (and (string>=? c "a") (string<=? c "z"))
                  (and (string>=? c "0") (string<=? c "9")))
              (book/course-suffix-chars? s (+ i 1) n)
              #f))))
  "Char walk for course-suffix-safe?.")

(define book/course-suffix-safe?
  (lambda (s)
    (book/course-suffix-chars? s 0 (string-length s)))
  "Nonempty [a-z0-9_-] course URL suffix (notes slugs carry hyphens;
   the doc slug gets the course/ prefix before any lookup, so the URL
   suffix alone can never traverse the notes tree).")

(define book/course-doc-slug
  (lambda (suffix)
    (string-append "course/" (as-str suffix)))
  "Notes-doc slug for a course URL suffix.")

(define book/course-chapter-ref
  (lambda (body)
    (let ((lines (string-split (as-str body) "\n")))
      (if (null? lines)
          ""
          (let ((first (as-str (car lines))))
            (if (string-starts-with? first "chapter: ")
                (substring first 9 (string-length first))
                ""))))))
  "Chapter label referenced by an outline node -- the `chapter: <label>`
   first line of its body -- or \"\" when the node is not a reading.
   Labels are book/label-safe? (the loader's ltree shape); validated
   before any link or row lookup.")

(define book/course-desc
  (lambda (body)
    (list/foldl
      (lambda (acc line)
        (if (string-eq acc "")
            (as-str line)
            (string-append acc "\n" (as-str line))))
      ""
      (cdr (string-split (as-str body) "\n"))))
  "Node body minus its first (chapter-reference) line, rejoined --
   prose-html re-paragraphs on blank lines.")

(define book/course-reading-html
  (lambda (node)
    (let ((ref (book/course-chapter-ref (outline/body node))))
      (let ((safe (book/label-safe? ref)))
        (let ((ch-row (if safe
                          (book/row (string-append "data.book.ch." ref))
                          ())))
          (html/li ""
            (html/join
              (list
                (if safe
                    (html/a
                      (string-append "/book/chapter/" ref)
                      (html-escape (as-str (outline/title node))))
                    (html/span
                      (html/attr "class" "book-hole")
                      (html-escape
                        (string-append (as-str (outline/title node))
                                       " -- unresolvable chapter ref"))))
                (if (null? ch-row)
                    ""
                    (string-append " "
                      (book/status-badge (book/f ch-row "status"))))
                (book/prose-html (book/course-desc (outline/body node))
```

```
                                  (book/fn-bound?
      "spec/token-binding?"))))))))))
  "One ordered reading-list entry: chapter link (node title), the
   chapter's status badge from its seeded row (always shown -- the
   reading list is a status view), then the description prose. An
   unresolvable ref renders as a hole, never silently.")

(define book/course-readers
  (lambda (nodes)
    (if (null? nodes)
        ()
        (if (string-eq (book/course-chapter-ref (outline/body (car nodes)))
     "")
            ()
            (cons (car nodes) (book/course-readers (cdr nodes))))))
  "The leading run of reading nodes (consecutive chapter refs).")

(define book/course-len
  (lambda (xs)
    (if (null? xs)
        0
        (+ 1 (book/course-len (cdr xs)))))
  "Length of a cons-built (pair) or kernel list. Kernel count is
   VList-strict -- it refuses pair chains (the cons-recursion shape
   book/course-readers builds; spec/append produces them too), while
   list/map, list/filter and html/join all accept pairs. Pair-friendly
   length here; kernel count relaxation filed in log/issues.")

(define book/course-drop
  (lambda (n xs)
    (if (= n 0)
        xs
        (book/course-drop (- n 1) (cdr xs))))
  "Tail after n elements.")

(define book/course-section-html
  (lambda (node depth link?)
    (let ((title (as-str (outline/title node))))
      (html/join
        (list
          (if (string-eq title "")
              ""
              ((if (= depth 1) html/h2 html/h3) (html-escape title)))
          (book/prose-html (outline/body node) link?)
          (book/course-level-html (outline/kids node) (+ depth 1) link?)))))
  "Non-reading outline node: heading (h2 at depth 1, h3 below), body
   prose (binding tokens link when the spec machinery is loaded),
   children.")

(define book/course-level-html
  (lambda (nodes depth link?)
    (if (null? nodes)
        ""
        (if (string-eq (book/course-chapter-ref (outline/body (car nodes)))
     "")
            (html/join
              (list
                (book/course-section-html (car nodes) depth link?)
                (book/course-level-html (cdr nodes) depth link?)))
            (let ((run (book/course-readers nodes)))
              (html/join
                (list
                  (html/tag "ol" ""
                    (html/join (list/map book/course-reading-html run)))
                  (book/course-level-html
                    (book/course-drop (book/course-len run) nodes)
                    depth link?)))))))
  "One outline level: consecutive reading nodes group into a single
   ordered list (the course's reading order); other nodes render as
   sections with their children.")

(define book/course-html
  (lambda (suffix doc mode link?)
```

```
    (let ((title (as-str (notes/doc/title doc)))
          (outline (notes/get-outline doc)))
      (html/page
        (list
          (book/style-html)
          (html/p*
            (html/attr "class" "muted")
            (html/join (list (html/a "/book" "book") " ? course")))
          (book/toggle-html (string-append "/book/course/" suffix) mode)
          (html/h1 (html-escape title))
          (book/prose-html (outline/body outline) link?)
          (book/course-level-html (outline/kids outline) 1 link?)
          (html/p*
            (html/attr "class" "muted")
            (html-escape
              (string-append
                "course " suffix
                " ? a composition over the book's chapters, never a copy --
    "
                "paper edition: borge book print --root course/"
                suffix ".borg")))))))
  "Course page from a notes doc: title, blurb, ordered reading list
   with chapter status badges, labs.")

(define book/course-page
  (lambda ()
    (let ((suffix (book/path-suffix "/book/course/" (request-path))))
      (if (not (book/course-suffix-safe? suffix))
          (book/not-found "course")
          (if (not (book/fn-bound? "notes/doc/get"))
              (book/not-found
                (string-append "course " suffix
                  " (notes not loaded -- (load-library \"notes\"))"))
              (let ((doc (notes/doc/get (book/course-doc-slug suffix))))
                (if (null? doc)
                    (book/not-found (string-append "course " suffix))
                    (book/course-html suffix doc (book/mode)
                      (book/fn-bound? "spec/token-binding?")))))))))
  "GET /book/course/<suffix> -- assemble the notes doc course/<suffix>
   (kind course) over the seeded chapter rows.")

;; ---------------------------------------------------------------- index

(define book/chapter-li
  (lambda (node mode)
    (let ((b (dict-get node "body")))
      (let ((title (book/f b "title"))
            (status (book/f b "status")))
        (html/li
          ""
          (html/join
            (list
              (html/a
                (string-append "/book/chapter/" (tree/leaf (book/row-path
    node)))
                (html-escape (as-str title)))
              (if (string-eq mode "workshop")
                  (string-append " " (book/status-badge status))
                  ""))))))))
  "Index row for one chapter (status badge in workshop register only).")

(define book/course-index-li
  (lambda (node)
    (let ((n (json-parse (as-str node))))
      (if (null? n)
          ""
          (let ((b (dict-get n "body")))
            (let ((slug (if (null? b) "" (as-str (dict-get b "slug")))))
              (let ((suffix (book/path-suffix "course/" slug)))
                (if (string-eq suffix "")
                    ""
                    (html/li ""
                      (html/a
```

```
                        (string-append "/book/course/" suffix)
                        (html-escape
                          (if (null? b)
                              suffix
                              (as-str (dict-get b "title"))))))))))))
  "Index row for one course doc (a tree/find-by hit); empty for
   non-course/ slugs -- kind=course is course-facing only when the slug
   carries the course/ prefix.")

(define book/courses-index-html
  (lambda ()
    (let ((hits (tree/find-by "note" "kind" "course" 50)))
      (let ((lis (list/filter
                   (lambda (li) (not (string-eq li "")))
                   (list/map book/course-index-li hits))))
        (if (null? lis)
            ""
            (html/join
              (list
                (html/h2 "Courses")
                (html/p*
                  (html/attr "class" "muted")
                  "compositions over the chapters (habitat-book
    course-documents) -- ordered readings, never copies")
                (html/ul "" (html/join lis))))))))
  "Index of course docs (notes kind=course via tree/find-by -- the same
   read-only lookup notes/doc/row uses for slugs); empty section when
   none are seeded.")

(define book/spec-index-html
  (lambda ()
    (if (book/fn-bound? "spec/packages-with-docs")
        (let ((pkgs (spec/packages-with-docs)))
          (if (null? pkgs)
              ""
              (html/join
                (list
                  (html/h2 (string-append "Package specs (" (as-str (count
    pkgs)) ")"))
                  (html/p*
                    (html/attr "class" "muted")
                    "spec/<pkg> intent docs -- the other corpus this index
    merges (habitat-workflow.book-view).")
                  (html/ul
                    ""
                    (html/join
                      (list/map
                        (lambda (p)
                          (html/li
                            ""
                            (html/a (string-append "/book/" (as-str p))
    (as-str p))))
                        pkgs)))))))
        ""))
  "Index of package spec docs (reuses spec/packages-with-docs; empty
   section when the spec package is not loaded).")

(define book/code-index-html
  (lambda ()
    (let ((code (book/children "data.book.code")))
      (if (null? code)
          (html/p*
            (html/attr "class" "muted")
            "No book rows seeded yet -- run scripts/load-book.sh
    <export.book.json>.")
          (html/join
            (list
              (html/h2 (string-append "Code appendix (" (as-str (count
    code)) ")"))
              (html/tag
                "details"
                ""
                (html/join
```

```
                  (list
                    (html/tag "summary" "" "all code chapters (.pp / .sql / .ml)")
                    (html/ul
                      ""
                      (html/join
                        (list/map
                          (lambda (node)
                            (let ((b (dict-get node "body")))
                              (html/li
                                ""
                                (html/a
                                  (string-append
                                    "/book/code/"
                                    (tree/leaf (book/row-path node)))
                                  (html-escape (as-str (book/f b "path")))))))
                          code)))))))))))
  "Collapsed index of every code chapter (the export's appendix order).")

(define book/page
  (lambda ()
    (let ((mode (book/mode)))
      (let ((root-b (book/row "data.book"))
            (chapters (book/children "data.book.ch")))
        (let ((rev (if (null? root-b) "" (as-str (book/f root-b "rev"))))
              (book-root (if (null? root-b) "" (as-str (book/f root-b "root")))))
          (html/page
            (list
              (book/style-html)
              (html/h1 "The Pricklypear Book")
              (html/p*
                (html/attr "class" "muted")
                (html-escape
                  (string-append
                    "The spec IS the book -- borge chapters rendered live from"
                    " seeded tree rows (scripts/load-book.sh"
                    (if (string-eq rev "")
                        ""
                        (string-append ", rev " (substring rev 0 (min 8 (string-length rev)))))
                    (if (string-eq book-root "")
                        ""
                        (string-append ", root " book-root))
                    ").")))
              (book/toggle-html "/book" mode)
              (html/p*
                (html/attr "class" "muted")
                (html/join
                  (list
                    (html/a "/book/verify" "verify")
                    " -- re-run every expect trailer (the book's dune runtest)")))
              (if (null? chapters)
                  (html/p*
                    (html/attr "class" "muted")
                    "No chapters seeded yet -- run scripts/load-book.sh <export.book.json>.")
                  (html/join
                    (list
                      (html/h2 (string-append "Chapters (" (as-str (count chapters)) ")"))
                      (html/ul
                        ""
                        (html/join
                          (list/map
                            (lambda (node) (book/chapter-li node mode))
                            chapters))))))
              (book/courses-index-html)
              (book/spec-index-html)
              (book/code-index-html)))))))
```

```
  "GET /book -- merged index: borg chapters (export order) + course
   compositions + package spec docs + the code appendix.")

;; ------------------------------------------------------------- verify
;;
;; habitat-book.eval-examples: (book/verify [slug]) is the book's
;; `dune runtest`. Every nopales example row carrying an `expect`
;; trailer re-evals in THIS session's env through the restricted
;; example-eval prim (same engine as the run button); the rendered
;; value is compared to the trimmed trailer. Mismatch, refusal, or
;; error => a hole record (path src expected got) -- drift-as-rendering:
;; reported, never silent. GET /book/verify[?chapter=slug] renders the
;; report; /api/eval consumes the fn directly (smoke: smoke_book.py).

(define book/guard-msg
  (lambda (e)
    (if (pair? e) (as-str (nth e 1)) (str e)))
  "Message slot of a guard error value ((error msg kind)); anything
   else renders via str.")

(define book/verify-skip?
  (lambda (b)
    (or (null? (book/f b "expected"))
        (not (string-eq (as-str (book/f b "lang")) "nopales"))))
  "Out of verify scope: no expect trailer, or an ocaml block
   (captured, not live -- habitat-book eval-examples v1).")

(define book/verify-one
  (lambda (node)
    (let ((b (dict-get node "body")))
      (if (book/verify-skip? b)
          ()
          (let ((path (book/row-path node))
                (src (as-str (book/f b "src")))
                (expected (string-trim (as-str (book/f b "expected")))))
            (guard
              (lambda (e)
                (list path src expected (book/guard-msg e)))
              (let ((got (string-trim (as-str (example-eval src)))))
                (if (string-eq got expected)
                    ()
                    (list path src expected got)))))))))
  "One example row -> () (skip or pass), else a hole record (path src
   expected got). Refusals/eval errors are holes too: an example that
   cannot run in the reader's restricted env is drift -- the got slot
   then carries the message verbatim (\"example-eval: refused: ?\" for
   gate/budget refusals, \"unbound: ?\" for eval errors).")

(define book/verify-candidates
  (lambda (rows)
    (list/filter
      (lambda (n) (not (book/verify-skip? (dict-get n "body"))))
      rows))
  "Example rows verify will actually run (nopales + expect trailer).")

(define book/verify-holes
  (lambda (rows)
    (list/filter
      (lambda (h) (not (null? h)))
      (list/map book/verify-one rows)))
  "Hole records over a row set, document order (tree/subtree order).")

(define book/verify
  (lambda (&optional slug)
    (let ((root (if (string-eq (as-str slug) "")
                    "data.book.ch"
                    (string-append "data.book.ch." (as-str slug)))))
      (book/verify-holes (tree/subtree root 20000))))
  "(book/verify [slug]) -- the book's self-test: re-eval every nopales
   example with an expect trailer under data.book.ch[.slug]; returns
   hole records (path src expected got); empty list = all green.
   Author-time verify half of habitat-book eval-examples (the run
   button is the read-time half).")
```

```
914
915 (define book/hole-html
916   (lambda (h)
917     (let ((path (as-str (car h)))
918           (src (as-str (car (cdr h))))
919           (expected (as-str (car (cdr (cdr h)))))
920           (got (as-str (car (cdr (cdr (cdr h)))))))
921       (html/div
922         (html/attr "class" "book-hole-row")
923         (html/join
924           (list
925             (html/p* "" (html-escape path))
926             (html/pre (html-escape src))
927             (html/p*
928               ""
929               (html/join
930                 (list
931                   "expect => "
932                   (html/code expected)
933                   " ? got "
934                   (html/code got)))))))))
935   "One verify hole row (drift-as-rendering): row path, the example
936    source, expected trailer vs actual render/refusal.")
937
938 (define book/verify-page-html
939   (lambda (slug n holes)
940     (html/page
941       (list
942         (book/style-html)
943         (html/p*
944           (html/attr "class" "muted")
945           (html/join (list (html/a "/book" "book") " ? verify")))
946         (html/h1 "book verify")
947         (html/p*
948           (html/attr "class" "muted")
949           (html-escape
950             (string-append
951               (if (string-eq slug "")
952                   "the whole book ? "
953                   (string-append "chapter " slug " ? "))
954               "checked " (as-str n)
955               (if (= n 1) " example" " examples")
956               " with expect trailers ? holes " (as-str (count holes)))))
957         (if (= (count holes) 0)
958             (html/p* "" "all green -- every expect trailer matches its
    render.")
959             (html/join (list/map book/hole-html holes))))))
960   "Report body for GET /book/verify: counts + one hole row per
961    mismatched/refused example.")
962
963 (define book/verify-page
964   (lambda ()
965     (let ((m (request-param "chapter")))
966       (let ((slug (if (null? m) "" (string-trim (as-str m)))))
967         (let ((root (if (string-eq slug "")
968                         "data.book.ch"
969                         (string-append "data.book.ch." slug))))
970           (let ((rows (tree/subtree root 20000)))
971             (if (null? rows)
972                 (if (string-eq slug "")
973                     (html/page
974                       (list
975                         (book/style-html)
976                         (html/h1 "book verify")
977                         (html/p*
978                           (html/attr "class" "muted")
979                           "No book rows seeded yet -- run
    scripts/load-book.sh <export.book.json>.")))
980                     (book/not-found (string-append "chapter " slug)))
981                 (book/verify-page-html
982                   slug
983                   (count (book/verify-candidates rows))
984                   (book/verify-holes rows)))))))))
```

```
985   "GET /book/verify[?chapter=slug] -- the verify report: every nopales
986    example with an expect trailer re-evaluated; mismatches/refusals
987    render as hole rows (habitat-book eval-examples).")
```

156 libs/bookmarks/lib.pp

```
;; package bookmarks -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push

(define bookmarks/all
  (lambda () "Return up to 100 bookmark rows." (rows (quote bookmark) 100)))

(define bookmarks/blank?
  (lambda (s) "True when s is null or an empty string." (if (null? s) #t
    (string-eq (as-str s) ""))))

(define bookmarks/bookmarklet-js
  (lambda
    ()
    "Build the javascript: bookmarklet that clips the current tab."
    (let
      ((origin (bookmarks/public-origin)))
      (string-append
        "javascript:(function(){"
        "var o="
        (json-emit origin)
        ";"
        "var u=encodeURIComponent(location.href);"
        "var t=encodeURIComponent(document.title||'');"
        "location.href=o+'/bookmarks/clip?url='+u+'&title='+t;"
        "})();"))))

(define bookmarks/clip-page
  (lambda
    ()
    "Handle the /bookmarks/clip URL, saving a bookmark from query params."
    (let
      ((url (as-str (request-param "url")))
        (title (request-param "title")))
      (cond
        ((blank? url)
          (html/page
            (list
              (html/h1 "Clip bookmark")
              (html/p "Missing url query parameter.")
              (html/tag
                "p"
                ""
                (html/a "/bookmarks" (html/text "Back to bookmarks"))))))
        ((not (bookmarks/url-ok? url))
          (html/page
            (list
              (html/h1 "Clip bookmark")
              (html/p "URL must start with http:// or https://")
              (html/tag
                "p"
                ""
                (html/a "/bookmarks" (html/text "Back to bookmarks"))))))
        (else
          (let
            ((t (bookmarks/clip-title title url))
              (existing (bookmarks/find-by-url url)))
            (seq
              (when (null? existing) (bookmarks/create t url))
              (html/page
                (list
                  (html/h1 (if (null? existing) "Saved" "Already saved"))
                  (html/div
                    (html/attr "class" "card")
                    (html/join
                      (list
                        (html/tag "p" "" (html/strong t))
                        (html/p*
                          (html/attr "class" "muted")
                          (html/a url (html/text url)))
                        (html/tag
                          "p"
```

```
                          ""
                          (html/a "/bookmarks" (html/text "View all bookmarks")))))))))))))))

(define bookmarks/clip-title
  (lambda
    (title url)
    "Return title, or the url when title is blank."
    (let
      ((t (string-trim (as-str title))))
      (if (blank? t) (as-str url) t))))

(define bookmarks/create
  (lambda
    (title url)
    "Create a bookmark row with the given title and url."
    (create (quote bookmark) (bookmarks/fields title url))))

(define bookmarks/delete
  (lambda (id) "Delete a bookmark row by id." (delete (quote bookmark) (as-str id))))

(define bookmarks/fields
  (lambda
    (title url)
    "Build a bookmark fields JSON string from title and url."
    (dict-set*
      "{}"
      (list "title" (as-str title) "url" (as-str url)))))

(define bookmarks/find-by-url
  (lambda
    (url)
    "Find the first bookmark row whose url matches; () when none."
    (bookmarks/find-by-url-in (bookmarks/all) (as-str url))))

(define bookmarks/find-by-url-in
  (lambda
    (rows-list url)
    "Search a bookmark row list for the given url."
    (if
      (null? rows-list)
      ()
      (let
        ((row (car rows-list)))
        (let
          ((fields (cdr row)))
          (if
            (string-eq (as-str (dict-get fields "url")) url)
            row
            (bookmarks/find-by-url-in (cdr rows-list) url)))))))

(define bookmarks/form-html
  (lambda
    ()
    "Render the add-bookmark HTML form."
    (html/div
      (html/attr "class" "card")
      (html/join
        (list
          (html/h2 "Add bookmark")
          (html/form
            "POST"
            "/bookmarks"
            (html/attrs
              (list
                (html/attr "hx-post" "/bookmarks")
                (html/attr "hx-target" "#bookmarks-main")
                (html/attr "hx-swap" "innerHTML")
                (html/attr
                  "hx-on::after-request"
                  "if(event.detail.successful){this.reset()}")))
            (html/join
```

```
              (list
                (html/hidden "action" "create")
                (html/label (html/attr "for" "bm-title") "Title")
                (html/text-input
                  "title"
                  (html/attrs
                    (list
                      (html/attr "id" "bm-title")
                      (html/attr "required" "required")
                      (html/attr "placeholder" "Example"))))
                (html/label (html/attr "for" "bm-url") "URL")
                (html/text-input
                  "url"
                  (html/attrs
                    (list
                      (html/attr "id" "bm-url")
                      (html/attr "required" "required")
                      (html/attr "placeholder" "https://..."))))
                (html/button (html/attr "type" "submit") "Add")))))))))

(define bookmarks/handle-post
  (lambda
    ()
    "Dispatch a POST action: create or delete a bookmark."
    (let
      ((action (as-str (request-param "action"))))
      (cond
        ((string-eq action "delete")
          (bookmarks/delete (request-param "id")))
        ((string-eq action "create")
          (let
            ((title (request-param "title"))
              (url (request-param "url")))
            (when
              (and (not (blank? title)) (not (blank? url)))
              (bookmarks/create title url))))
        (else ())))))

(define bookmarks/install-html
  (lambda
    ()
    "Render the browser bookmarklet install card."
    (html/div
      (html/attr "class" "card")
      (html/join
        (list
          (html/h2 "Browser bookmarklet")
          (html/p*
            (html/attr "class" "muted")
            (html/text
              "Drag the link below to your bookmarks bar. Click it on any page while signed in to save title and URL."))
          (html/tag
            "p"
            ""
            (html/tag
              "a"
              (html/attrs
                (list
                  (html/attr "class" "button")
                  (html/attr "href" (bookmarks/bookmarklet-js))))
              (html/text "Save to Pricklypear")))
          (html/p*
            (html/attr "class" "muted")
            (html/text
              "If logged out, you will sign in and return to finish the save."))))))))

(define bookmarks/item-li
  (lambda
    (row)
    "Render one bookmark row as a list item with a delete button."
    (let
```

```
      ((id (as-str (car row))) (fields (cdr row)))
      (let
        ((title (as-str (dict-get fields "title")))
          (url (as-str (dict-get fields "url"))))
        (html/li
          (html/attr "class" "card")
          (html/join
            (list
              (html/a url (html/text title))
              (html/p* (html/attr "class" "muted") (html/text url))
              (html/form
                "POST"
                "/bookmarks"
                (html/attrs
                  (list
                    (html/attr "style" "display:inline")
                    (html/attr "hx-post" "/bookmarks")
                    (html/attr "hx-target" "#bookmarks-main")
                    (html/attr "hx-swap" "innerHTML")))
                (html/join
                  (list
                    (html/hidden "action" "delete")
                    (html/hidden "id" id)
                    (html/button (html/attr "type" "submit")
    "Delete")))))))))))

(define bookmarks/list-html
  (lambda
    ()
    "Render the full bookmarks list, or an empty-state message."
    (html/div
      (html/attr "id" "bookmarks-list")
      (let
        ((xs (bookmarks/all)))
        (if
          (null? xs)
          (html/p*
            (html/attr "class" "muted")
            (html/text "No bookmarks yet."))
          (html/ul "" (html/join (list/map bookmarks/item-li xs))))))))

(define bookmarks/main-html
  (lambda
    ()
    "Render the install card, add form, and bookmarks list."
    (html/div
      (html/attr "id" "bookmarks-main")
      (html/join
        (list
          (bookmarks/install-html)
          (bookmarks/form-html)
          (html/h2 "All bookmarks")
          (bookmarks/list-html))))))

(define bookmarks/page
  (lambda
    ()
    "Handle POST actions; otherwise render the full bookmarks page."
    (if
      (string-eq (request-method) "POST")
      (seq
        (bookmarks/handle-post)
        (if
          (hx-request?)
          (render-html (bookmarks/main-html))
          (redirect "/bookmarks")))
      (html/page
        (list
          (html/h1 "Bookmarks")
          (html/p*
            (html/attr "class" "muted")
            (html/text
```

```
              "Save links on data_rows (type bookmark). Hosted bookmarklet clips the current tab."))
          (bookmarks/main-html))))))

(define bookmarks/public-origin
  (lambda
    ()
    "Return the public origin (scheme://host) for generated links."
    (let
      ((host (as-str (request-header "host")))
        (xfwd (as-str (request-header "x-forwarded-proto"))))
      (let
        ((scheme
          (if
            (or
              (string-eq xfwd "https")
              (string-contains? host "pricklypear.rocks"))
            "https"
            "http")))
        (if (blank? host) "" (string-append scheme "://" host))))))

(define bookmarks/starts-with?
  (lambda
    (s prefix)
    "True when string s starts with prefix."
    (let
      ((ss (as-str s)) (p (as-str prefix)))
      (let
        ((n (string-length p)))
        (and
          (>= (string-length ss) n)
          (string-eq (substring ss 0 n) p))))))

(define bookmarks/url-ok?
  (lambda
    (url)
    "True when url starts with http:// or https://."
    (let
      ((u (as-str url)))
      (or
        (bookmarks/starts-with? u "https://")
        (bookmarks/starts-with? u "http://")))))
```

157 libs/calendar/lib.pp

```
;; package calendar -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push

(define calendar/add-month
  (lambda
    (ym delta)
    "Add delta months to a YYYY-MM string, wrapping year boundaries."
    (let
      ((y (parse-int (substring (as-str ym) 0 4))))
      (let
        ((m (parse-int (substring (as-str ym) 5 2))))
        (let
          ((m2 (+ m delta)))
          (if
            (> m2 12)
            (string-append
              (as-str (+ y 1))
              "-"
              (calendar/pad2 (- m2 12)))
            (if
              (< m2 1)
              (string-append
                (as-str (- y 1))
                "-"
                (calendar/pad2 (+ m2 12)))
              (string-append (as-str y) "-" (calendar/pad2 m2)))))))))

(define calendar/agenda
  (lambda
    ()
    "Build a 7-day agenda list with events and due todos per day."
    (let
      ((today (date-today)))
      (let
        ((weekday-names
          (list
            "Sun"
            "Mon"
            "Tue"
            "Wed"
            "Thu"
            "Fri"
            "Sat")))
        (let
          ((days
            (list
              today
              (date-add-days today 1)
              (date-add-days today 2)
              (date-add-days today 3)
              (date-add-days today 4)
              (date-add-days today 5)
              (date-add-days today 6))))
          (list/map
            (lambda
              (d)
              (let
                ((y (parse-int (substring d 0 4)))
                  (m (parse-int (substring d 5 2)))
                  (day (parse-int (substring d 8 2))))
                (let
                  ((w (calendar/weekday y m day)))
                  (list
                    (list-ref weekday-names w)
                    d
                    (calendar/events-for-day d)
                    (calendar/todos-due-on d)))))
            days))))))

(define calendar/agenda-widget
  (lambda
```

```
  ()
  "Render the 7-day agenda as an HTML card widget."
  (let
    ((month-names
      (list
        "Jan"
        "Feb"
        "Mar"
        "Apr"
        "May"
        "Jun"
        "Jul"
        "Aug"
        "Sep"
        "Oct"
        "Nov"
        "Dec")))
    (let
      ((rows (calendar/agenda)))
      (layout/card
        "Agenda"
        (html/div
          (html/attr "id" "agenda-widget")
          (html/join
            (list/map
              (lambda
                (row)
                (let
                  ((day-name (car row))
                    (date (car (cdr row)))
                    (events (car (cdr (cdr row))))
                    (todos (car (cdr (cdr (cdr row))))))
                  (let
                    ((m-str (substring date 5 2))
                      (d-str (substring date 8 2)))
                    (let
                      ((month (nth month-names (- (parse-int m-str) 1))))
                      (html/div
                        (html/attr "class" "agenda-day")
                        (html/join
                          (list
                            (html/div
                              (html/attr "class" "agenda-day-header")
                              (html/strong
                                (string-append day-name " " month " "
  d-str)))
                            (if
                              (null? events)
                              (html/p*
                                (html/attr "class" "muted")
                                "nothing scheduled")
                              (html/ul
                                (list)
                                (html/join
                                  (list/map
                                    (lambda
                                      (ev)
                                      (let
                                        ((f (cdr ev)))
                                        (let
                                          ((t (calendar/event-time-string
  f)))
                                          (html/li
                                            (list)
                                            (if
                                              (string-eq t "")
                                              (html/text
  (calendar/event-title f))
                                              (html/join
                                                (list
                                                  (html/span
                                                    (html/attr
                                                      "style"
```

```
                                                    "font-weight: bold;
    color: gray; margin-right: 0.5em;")
                                                  (html/text t))
                                                (html/text
    (calendar/event-title f)))))))))
                                      events))))))))))))
                rows))))))))

(define calendar/all-events
  (lambda
    ()
    "Return up to 500 calendar-event rows from the store."
    (rows (quote calendar-event) 500)))

(define calendar/build-cells
  (lambda
    (y
      m
      dim
      start-wd
      today
      ym
      d
      acc)
    "Recursively build calendar day cells from day 1 to days-in-month."
    (if
      (> d dim)
      acc
      (calendar/build-cells
        y
        m
        dim
        start-wd
        today
        ym
        (+ d 1)
        (string-append acc (calendar/day-cell y m d today ym))))))

(define calendar/cell-marks
  (lambda
    (day)
    "Render event/todo/daily-note indicator dots for a given day."
    (let
      ((ne (calendar/count-events-on day)))
      (let
        ((nt (calendar/count-todos-on day)))
        (let
          ((daily (calendar/has-daily? day)))
          (html/div
            (html/attr "class" "cal-marks")
            (html/join
              (list
                (if
                  (> ne 0)
                  (html/span
                    (html/attr "class" "cal-dot cal-dot-ev")
                    (html/text (as-str ne)))
                  "")
                (if
                  (> nt 0)
                  (html/span
                    (html/attr "class" "cal-dot cal-dot-todo")
                    (html/text (as-str nt)))
                  "")
                (if
                  daily
                  (html/span
                    (html/attr "class" "cal-dot cal-dot-note")
                    (html/text "n"))
                  ""))))))))))

(define calendar/count-events-on
  (lambda
```

```
    (day)
    "Count events on a specific YYYY-MM-DD day."
    (list/foldl
      (lambda (acc _) (+ acc 1))
      0
      (calendar/events-on-day day))))

(define calendar/count-todos-on
  (lambda
    (day)
    "Count todos due on a specific YYYY-MM-DD day."
    (list/foldl
      (lambda (acc _) (+ acc 1))
      0
      (calendar/todos-due-on day))))

(define calendar/create
  (lambda
    (title day start notes)
    "Create a new calendar-event with title, day, start time, and notes."
    (let
      ((st (if (null? start) "" (as-str start))))
      (let
        ((all (if (string-eq st "") "true" "false")))
        (create
          (quote calendar-event)
          (dict-set*
            "{}"
            (list
              "title"
              (as-str title)
              "day"
              (as-str day)
              "start_time"
              st
              "end_time"
              ""
              "all_day"
              all
              "notes"
              (as-str notes)
              "location"
              ""
              "color"
              ""
              "source"
              "calendar")))))))

(define calendar/cron-field-matches?
  (lambda
    (field val)
    "Test if a cron field value matches a given integer."
    (if
      (string-eq field "*")
      #t
      (if
        (string-contains? field ",")
        (let
          ((vals (string-split field ",")))
          (list/foldl
            (lambda (acc v) (or acc (= val (parse-int v))))
            #f
            vals))
        (if
          (string-contains? field "-")
          (let*
            ((parts (string-split field "-"))
              (lo (parse-int (car parts)))
              (hi (parse-int (car (cdr parts)))))
            (and (>= val lo) (<= val hi)))
          (let ((field-val (parse-int field))) (= val field-val)))))))

(define calendar/cron-matches?
```

```
  (lambda
    (date-str cron-expr)
    "Test if a cron expression matches a YYYY-MM-DD date string."
    (let*
      ((parts (string-split cron-expr " "))
        (min-field (car parts))
        (hr-field (car (cdr parts)))
        (dom-field (car (cdr (cdr parts))))
        (mon-field (car (cdr (cdr (cdr parts)))))
        (dow-field (car (cdr (cdr (cdr (cdr parts)))))))
      (let*
        ((y (parse-int (substring date-str 0 4)))
          (m (parse-int (substring date-str 5 2)))
          (d (parse-int (substring date-str 8 2)))
          (dow (calendar/weekday y m d)))
        (and
          (calendar/cron-field-matches? mon-field m)
          (let
            ((dom-specified (not (string-eq dom-field "*")))
              (dow-specified (not (string-eq dow-field "*"))))
            (if
              (and dom-specified dow-specified)
              (or
                (calendar/cron-field-matches? dom-field d)
                (calendar/cron-field-matches? dow-field dow))
              (if
                dom-specified
                (calendar/cron-field-matches? dom-field d)
                (if
                  dow-specified
                  (calendar/cron-field-matches? dow-field dow)
                  #t)))))))))

(define calendar/cron-to-event
  (lambda
    (rec)
    "Convert a recurring cron rule to a calendar-event JSON row."
    (let
      ((cron (car rec))
        (title (car (cdr rec)))
        (time (car (cdr (cdr rec)))))
      (cons
        ""
        (string-append
          "{\"title\": \""
          title
          "\", \"start_time\": \""
          time
          "\", \"cron\": \""
          cron
          "\", \"all_day\": \"false\"}")))))

(define calendar/day-cell
  (lambda
    (y m d today ym)
    "Render a single calendar day cell as HTML with link and marks."
    (if
      (= d 0)
      (html/div (html/attr "class" "cal-cell cal-empty") "")
      (let
        ((day (calendar/ymd y m d)))
        (html/a
          (string-append "/calendar?day=" day)
          (html/div
            (html/attr
              "class"
              (if
                (calendar/day-eq? day today)
                "cal-cell cal-today"
                "cal-cell"))
            (html/join
              (list
                (html/span
```

```
                  (html/attr "class" "cal-dom")
                  (html/text (as-str d)))
                (calendar/cell-marks day)))))))))

(define calendar/day-eq?
  (lambda
    (a b)
    "Check if two day strings are equal (as strings)."
    (string-eq (as-str a) (as-str b))))

(define calendar/day-html
  (lambda
    (day)
    "Render the full day view: events, due todos, and add-event form."
    (html/div
      (html/attr "class" "cal-day-panel")
      (html/join
        (list
          (html/h2 (html/text day))
          (html/p*
            (html/attr "class" "muted")
            (html/join
              (list
                (html/a
                  (string-append "/calendar?ym=" (calendar/ym day))
                  "? month")
                " ? "
                (html/a
                  (string-append "/notes?slug=daily-" day)
                  "daily note"))))
          (html/h3 (html/text "Events"))
          (html/ul
            (html/attr "class" "card cal-list")
            (list/foldl
              (lambda
                (acc row)
                (string-append acc (calendar/event-li row)))
              ""
              (calendar/events-on-day day)))
          (html/h3 (html/text "Due todos"))
          (html/ul
            (html/attr "class" "card cal-list")
            (list/foldl
              (lambda
                (acc row)
                (string-append acc (calendar/todo-due-li row)))
              ""
              (calendar/todos-due-on day)))
          (html/div
            (html/attr "class" "card")
            (html/join
              (list
                (html/h3 (html/text "Add event"))
                (html/form
                  "POST"
                  "/calendar"
                  (html/attrs
                    (list
                      (html/attr "hx-post" "/calendar")
                      (html/attr "hx-target" "#cal-main")
                      (html/attr "hx-swap" "innerHTML")
                      (html/attr
                        "hx-on::after-request"
                        "if(event.detail.successful){this.reset()}")))
                  (html/join
                    (list
                      (html/hidden "action" "create")
                      (html/hidden "day" day)
                      (html/label (html/attr "for" "title") "Title")
                      (html/text-input
                        "title"
                        (html/attrs
                          (list
```

```
                            (html/attr "id" "title")
                            (html/attr "required" "required")
                            (html/attr "placeholder" "standup"))))
                        (html/label
                          (html/attr "for" "start_time")
                          "Start (HH:MM, blank=all day)")
                        (html/text-input
                          "start_time"
                          (html/attrs
                            (list
                              (html/attr "id" "start_time")
                              (html/attr "placeholder" "10:00"))))
                        (html/label (html/attr "for" "notes") "Notes")
                        (html/text-input
                          "notes"
                          (html/attrs
                            (list
                              (html/attr "id" "notes")
                              (html/attr "placeholder" ""))))
                        (html/button (html/attr "type" "submit") "Add
    event")))))))))))

(define calendar/day-prefix?
  (lambda
    (day ym)
    "Check if a day string starts with a given YYYY-MM prefix."
    (string-eq (substring (as-str day) 0 7) (as-str ym))))

(define calendar/days-in-month
  (lambda
    (y m)
    "Return number of days in month m of year y, handling leap years."
    (if
      (= m 2)
      (if (calendar/leap? y) 29 28)
      (if
        (= m 4)
        30
        (if (= m 6) 30 (if (= m 9) 30 (if (= m 11) 30 31)))))))

(define calendar/delete
  (lambda
    (id)
    "Delete a calendar-event by id."
    (delete (quote calendar-event) id)))

(define calendar/dom
  (lambda
    (day)
    "Extract day-of-month integer from a YYYY-MM-DD string."
    (parse-int (substring (as-str day) 8 2))))

(define calendar/event-all-day?
  (lambda
    (f)
    "Check if an event fields dict is marked all-day."
    (let
      ((a (dict-get f "all_day")))
      (if
        (null? a)
        (string-eq (calendar/event-start f) "")
        (string-eq (as-str a) "true")))))

(define calendar/event-day
  (lambda
    (f)
    "Extract the day field from an event fields dict."
    (as-str (dict-get f "day"))))

(define calendar/event-li
  (lambda
    (row)
    "Render a calendar event as an HTML list item with delete button."
```

```
    (let
      ((id (car row)))
      (let
        ((f (cdr row)))
        (html/li
          (html/attr "class" "cal-event")
          (html/join
            (list
              (html/span
                (html/attr "class" "cal-time")
                (html/text
                  (if
                    (calendar/event-all-day? f)
                    "all day"
                    (calendar/event-start f))))
              (html/span
                (html/attr "class" "cal-ev-title")
                (html/text (calendar/event-title f)))
              (html/form
                "POST"
                "/calendar"
                (html/attrs
                  (list
                    (html/attr "class" "inline-form")
                    (html/attr "hx-post" "/calendar")
                    (html/attr "hx-target" "#cal-main")
                    (html/attr "hx-swap" "innerHTML")))
                (html/join
                  (list
                    (html/hidden "action" "delete")
                    (html/hidden "id" id)
                    (html/hidden "day" (calendar/event-day f))
                    (html/button
                      (html/attrs
                        (list
                          (html/attr "type" "submit")
                          (html/attr "class" "secondary ol-btn")))
                      "del")))))))))

(define calendar/event-start
  (lambda
    (f)
    "Extract start_time from event fields dict, empty if missing."
    (let
      ((s (dict-get f "start_time")))
      (if (null? s) "" (as-str s)))))

(define calendar/event-time-string
  (lambda
    (f)
    "Format event time range from start_time and end_time fields."
    (let
      ((s (as-str (dict-get f "start_time")))
        (e (as-str (dict-get f "end_time"))))
      (if
        (string-eq s "")
        ""
        (if
          (string-contains? s "-")
          s
          (if (string-eq e "") s (string-append s " - " e)))))))

(define calendar/event-title
  (lambda
    (f)
    "Extract the title from an event fields dict."
    (as-str (dict-get f "title"))))

(define calendar/events-for-day
  (lambda
    (d)
    "Get events for a day including cron-based recurring events."
    (let
```

```
      ((regular (calendar/events-on-day d))
        (recurring (calendar/recurring-events)))
      (let
        ((cron-evs
          (list/filter
            (lambda (rec) (calendar/cron-matches? d (car rec)))
            recurring)))
        (append
          regular
          (list/map calendar/cron-to-event cron-evs))))))

(define calendar/events-in-month
  (lambda
    (ym)
    "Filter all events to those in a given YYYY-MM month."
    (list/filter
      (lambda
        (row)
        (calendar/day-prefix? (calendar/event-day (cdr row)) ym))
      (calendar/all-events))))

(define calendar/events-on-day
  (lambda
    (day)
    "Filter all events to those on a specific YYYY-MM-DD day."
    (list/filter
      (lambda
        (row)
        (calendar/day-eq? (calendar/event-day (cdr row)) day))
      (calendar/all-events))))

(define calendar/foo
  (lambda
    ()
    "Returns todays date. Debug/test stub."
    (date-today)))

(define calendar/handle-post
  (lambda
    ()
    "Handle POST requests for create and delete calendar actions."
    (let
      ((action (as-str (request-param "action"))))
      (cond
        ((string-eq action "create")
          (calendar/create
            (request-param "title")
            (request-param "day")
            (request-param "start_time")
            (request-param "notes")))
        ((string-eq action "delete")
          (calendar/delete (request-param "id")))
        (else ())))))

(define calendar/has-daily?
  (lambda
    (day)
    "Check if a daily note exists for a given YYYY-MM-DD day."
    (let
      ((slug (string-append "daily-" (as-str day))))
      (if (null? (notes/doc/get slug)) #f #t))))

(define calendar/leap?
  (lambda
    (y)
    "Check if year y is a leap year."
    (if
      (= (mod y 400) 0)
      #t
      (if (= (mod y 100) 0) #f (= (mod y 4) 0)))))

(define calendar/main-html
  (lambda
```

```
    ()
    "Render the main calendar panel (month grid or day view)."
    (let
      ((day (request-param "day")) (ym (request-param "ym")))
      (html/div
        (html/attr "id" "cal-main")
        (if
          (null? day)
          (calendar/month-html
            (if
              (null? ym)
              (calendar/ym (calendar/today))
              (as-str ym)))
          (calendar/day-html (as-str day)))))))

(define calendar/month
  (lambda
    (day)
    "Extract month integer from a YYYY-MM-DD string."
    (parse-int (substring (as-str day) 5 2))))

(define calendar/month-grid
  (lambda
    (ym)
    "Render the month grid HTML with day-of-week headers and cells."
    (let
      ((y (parse-int (substring (as-str ym) 0 4))))
      (let
        ((m (parse-int (substring (as-str ym) 5 2))))
        (let
          ((today (calendar/today)))
          (let
            ((dim (calendar/days-in-month y m)))
            (let
              ((wd (calendar/weekday y m 1)))
              (html/div
                (html/attr "class" "cal-grid")
                (html/join
                  (list
                    (html/div
                      (html/attr "class" "cal-dow")
                      (html/text "Su"))
                    (html/div
                      (html/attr "class" "cal-dow")
                      (html/text "Mo"))
                    (html/div
                      (html/attr "class" "cal-dow")
                      (html/text "Tu"))
                    (html/div
                      (html/attr "class" "cal-dow")
                      (html/text "We"))
                    (html/div
                      (html/attr "class" "cal-dow")
                      (html/text "Th"))
                    (html/div
                      (html/attr "class" "cal-dow")
                      (html/text "Fr"))
                    (html/div
                      (html/attr "class" "cal-dow")
                      (html/text "Sa"))
                    (calendar/pad-start wd "")
                    (calendar/build-cells
                      y
                      m
                      dim
                      wd
                      today
                      ym
                      1
                      "")))))))))))

(define calendar/month-html
  (lambda
```

```
    (ym)
    "Render the full month view with navigation, grid, and legend."
    (html/div
      (html/attr "class" "cal-month")
      (html/join
        (list
          (calendar/month-nav ym)
          (calendar/month-grid ym)
          (html/p*
            (html/attr "class" "muted")
            "Dots: events ? todos due ? daily note. Click a day for
     agenda.")))))

(define calendar/month-nav
  (lambda
    (ym)
    "Render previous/next/today navigation links for a YYYY-MM month."
    (let
      ((prev (calendar/add-month ym -1)))
      (let
        ((next (calendar/add-month ym 1)))
        (html/div
          (html/attr "class" "cal-nav")
          (html/join
            (list
              (html/a
                (string-append "/calendar?ym=" prev)
                "?")
              (html/strong ym)
              (html/a
                (string-append "/calendar?ym=" next)
                "?")
              (html/a "/calendar" "Today")))))))

(define calendar/pad-start
  (lambda
    (n acc)
    "Recursively prepend n empty cells to pad the grid start."
    (if
      (<= n 0)
      acc
      (calendar/pad-start
        (- n 1)
        (string-append
          acc
          (html/div (html/attr "class" "cal-cell cal-empty") ""))))))

(define calendar/pad2
  (lambda
    (n)
    "Zero-pad a number to 2 digits (e.g. 3 -> 03)."
    (if (< n 10) (string-append "0" (as-str n)) (as-str n))))

(define calendar/page
  (lambda
    ()
    "Route handler for /calendar: POST actions or GET page render."
    (if
      (string-eq (request-method) "POST")
      (seq
        (calendar/handle-post)
        (if
          (hx-request?)
          (render-html
            (let
              ((day (request-param "day")))
              (if
                (null? day)
                (calendar/main-html)
                (html/div
                  (html/attr "id" "cal-main")
                  (calendar/day-html (as-str day))))))
          (let
```

```
            ((day (request-param "day")))
            (if
              (null? day)
              (redirect "/calendar")
              (redirect (string-append "/calendar?day=" (as-str day)))))))
      (html/page
        (list
          (html/h1 "Calendar")
          (html/p*
            (html/attr "class" "muted")
            "Month grid ? day agenda ? events + due todos + daily notes")
          (calendar/main-html))))))

(define calendar/recurring-events
  (lambda
    ()
    "Return hardcoded list of recurring cron-based events."
    (list
      (list "30 7 * * 1-4" "Taylor Dental" "07:30-17:00")
      (list "0 17 * * 2-4" "Cassie's" "17:00-23:00"))))

(define calendar/today
  (lambda
    ()
    "Returns todays date as YYYY-MM-DD via date-today."
    (date-today)))

(define calendar/todo-due-li
  (lambda
    (row)
    "Render a due todo as an HTML list item linking to /todos."
    (let
      ((id (car row)))
      (let
        ((f (cdr row)))
        (html/li
          (html/attr "class" "cal-todo")
          (html/join
            (list
              (html/span
                (html/attr "class" "cal-time")
                (html/text "todo"))
              (html/a
                "/todos"
                (html/text (as-str (dict-get f "title")))))))))))

(define calendar/todos-due-on
  (lambda
    (day)
    "Filter all todo items to those due on a specific YYYY-MM-DD day."
    (list/filter
      (lambda
        (row)
        (let
          ((due (dict-get (cdr row) "due")))
          (if (null? due) #f (calendar/day-eq? due day))))
      (todo/item/all))))

(define calendar/weekday
  (lambda
    (y m d)
    "Return day-of-week (0=Sun..6=Sat) for year y, month m, day d."
    (date-weekday y m d)))

(define calendar/year
  (lambda
    (day)
    "Extract year integer from a YYYY-MM-DD string."
    (parse-int (substring (as-str day) 0 4))))

(define calendar/ym
  (lambda
    (day)
```

```
    "Extract YYYY-MM prefix from a YYYY-MM-DD string."
    (substring (as-str day) 0 7)))

(define calendar/ymd
  (lambda
    (y m d)
    "Format year y, month m, day d as a YYYY-MM-DD string."
    (date-format y m d)))


(define calendar/css
  (lambda
    ()
    "Render the calendar component stylesheet (extracted from ui/theme)."
    (css/sheet
      (list
        (css/rule
          ".cal-nav"
          (list
            "display"
            "flex"
            "align-items"
            "center"
            "gap"
            ".75rem"
            "margin"
            ".5rem 0"
            "flex-wrap"
            "wrap"))
        (css/rule
          ".cal-nav a"
          (list "text-decoration" "none" "font-weight" "700"))
        (css/rule
          ".cal-grid"
          (list
            "display"
            "grid"
            "grid-template-columns"
            "repeat(7,1fr)"
            "gap"
            "2px"
            "background"
            "var(--shadow)"
            "border"
            "2px solid #000"
            "max-width"
            "42rem"))
        (css/rule
          ".cal-dow"
          (list
            "background"
            "var(--chrome)"
            "text-align"
            "center"
            "font-weight"
            "700"
            "padding"
            ".25rem"
            "font-size"
            ".8rem"))
        (css/rule
          ".cal-cell"
          (list
            "background"
            "var(--panel)"
            "min-height"
            "3.5rem"
            "padding"
            ".2rem"
            "display"
            "block"
            "color"
            "var(--ink)"
```

```
            "text-decoration"
            "none"))
          (css/rule
            ".cal-empty"
            (list "background" "var(--bg)" "min-height" "3.5rem"))
          (css/rule
            ".cal-today"
            (list
              "outline"
              "2px solid #000"
              "background"
              "var(--today-bg)"))
          (css/rule
            ".cal-dom"
            (list "font-weight" "700" "font-size" ".85rem"))
          (css/rule
            ".cal-marks"
            (list
              "display"
              "flex"
              "flex-wrap"
              "wrap"
              "gap"
              "2px"
              "margin-top"
              ".15rem"))
          (css/rule
            ".cal-dot"
            (list
              "font-size"
              ".65rem"
              "padding"
              "0 .2rem"
              "border"
              "1px solid #000"
              "background"
              "var(--chrome)"
              "line-height"
              "1.2"))
          (css/rule
            ".cal-dot-ev"
            (list "background" "var(--done-bg)"))
          (css/rule
            ".cal-dot-todo"
            (list "background" "var(--pending-bg)"))
          (css/rule
            ".cal-dot-note"
            (list "background" "var(--waiting-bg)"))
          (css/rule
            ".cal-list"
            (list
              "list-style"
              "none"
              "padding"
              ".25rem"
              "margin"
              "0"))
          (css/rule
            ".cal-event,.cal-todo"
            (list
              "display"
              "flex"
              "gap"
              ".5rem"
              "align-items"
              "center"
              "padding"
              ".35rem 0"
              "border-bottom"
              "1px solid #ccc"))
          (css/rule
            ".cal-time"
            (list
```

```
            "font-size"
            ".8rem"
            "min-width"
            "3.5rem"
            "color"
            "var(--muted)"))
          (css/rule
            ".cal-day-panel label"
            (list "display" "block" "margin-top" ".4rem"))
          (css/rule
            "a .cal-cell:hover"
            (list "background" "var(--sel-bg)"))))))
```

158 libs/chat/lib.pp

```
;; package chat -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push

(define chat/all-messages
  (lambda
    ()
    "Return up to 200 chat-message rows."
    (rows (quote chat-message) 200)))

(define chat/alphabet
  "
    !\"#$%&'()*+,-./0123456789:;<=>?@ABCDEFGHIJKLMNOPQRSTUVWXYZ[\\]^_`abcdefghijklmnopqrstuvwxyz{|}~")

(define chat/can-access?
  (lambda
    (room-id)
    "Check if current user can access a chat room."
    (let
      ((rid (as-str room-id)) (me (as-str (whoami))))
      (if
        (string-eq rid chat/lobby-id)
        #t
        (let
          ((row (chat/find-room rid)))
          (if
            (null? row)
            #f
            (let
              ((f (chat/row-fields row)))
              (if
                (string-eq (as-str (dict-get f "kind")) "lobby")
                #t
                (chat/member? (dict-get f "members") me)))))))))

(define chat/char-rank
  (lambda
    (c)
    "Get the rank (index) of a character in the alphabet."
    (chat/char-rank-loop
      c
      chat/alphabet
      0
      (string-length chat/alphabet))))

(define chat/char-rank-loop
  (lambda
    (c alphabet i n)
    "Tail-recursive helper for char-rank."
    (if
      (>= i n)
      999
      (if
        (string-eq (substring alphabet i 1) c)
        i
        (chat/char-rank-loop c alphabet (+ i 1) n)))))

(define chat/dm
  (lambda
    (peer)
    "Get or create a DM room id for a peer."
    (let
      ((them (as-str peer)) (me (as-str (whoami))))
      (if
        (blank? them)
        ""
        (if
          (string-eq them me)
          ""
          (let
            ((rid (chat/dm-room-id them))
              (found (chat/find-room (chat/dm-room-id them))))
```

```
            (if
              (null? found)
              (seq
                (create
                  (quote chat-room)
                  (dict-set*
                    "{}"
                    (list
                      "kind"
                      "dm"
                      "room_id"
                      rid
                      "title"
                      (string-append "DM " me "/" them)
                      "members"
                      (chat/members-json me them)
                      "created_by"
                      me
                      "created_at"
                      "")))
                rid)
              rid)))))))

(define chat/dm-list-li
  (lambda
    (row)
    "Render a DM thread as an HTML list item."
    (let
      ((f (chat/row-fields row)) (me (as-str (whoami))))
      (let
        ((title (as-str (dict-get f "title")))
          (rid (as-str (dict-get f "room_id"))))
        (let
          ((peer (chat/dm-peer-from-title title me)))
          (html/li
            (html/attr "class" "card")
            (html/p*
              ""
              (html/a
                (string-append "/chat/dm?peer=" peer)
                (html/text peer)))))))))

(define chat/dm-list-page
  (lambda
    ()
    "Render the DM list page with start-DM form."
    (html/page
      (list
        (html/h1 "Direct messages")
        (html/p* "" (html/a "/chat" "Lobby"))
        (html/h2 "Start DM")
        (html/form
          "GET"
          "/chat/dm"
          ""
          (html/join
            (list
              (html/label (html/attr "for" "dm-peer") "Peer username")
              (html/text-input
                "peer"
                (html/attrs
                  (list
                    (html/attr "id" "dm-peer")
                    (html/attr "required" "required")
                    (html/attr "placeholder" "alice"))))
              (html/button (html/attr "type" "submit") "Open"))))
        (html/h2 "Your threads")
        (let
          ((xs (chat/dms)))
          (if
            (null? xs)
            (html/p* (html/attr "class" "muted") "No DMs yet.")
            (html/ul "" (html/join (list/map chat/dm-list-li xs)))))))))
```

```
144
145 (define chat/dm-main-html
146   (lambda
147     (peer)
148     "Render the DM main area with message form and history."
149     (html/div
150       (html/attr "id" "dm-main")
151       (html/join
152         (list
153           (html/div
154             (html/attr "class" "card")
155             (html/form
156               "POST"
157               (string-append "/chat/dm?peer=" peer)
158               (html/attrs
159                 (list
160                   (html/attr
161                     "hx-post"
162                     (string-append "/chat/dm?peer=" peer))
163                   (html/attr "hx-target" "#dm-main")
164                   (html/attr "hx-swap" "innerHTML")
165                   (html/attr
166                     "hx-on::after-request"
167                     "if(event.detail.successful){this.reset()}")))
168               (html/join
169                 (list
170                   (html/hidden "peer" peer)
171                   (html/label (html/attr "for" "dm-body") "Message")
172                   (html/textarea
173                     "body"
174                     (html/attrs
175                       (list
176                         (html/attr "id" "dm-body")
177                         (html/attr "rows" "2")
178                         (html/attr "required" "required")
179                         (html/attr "maxlength" "4000")))
180                     "")
181                   (html/button (html/attr "type" "submit") "Send")))))
182           (chat/dm-messages-html peer))))))
183
184 (define chat/dm-messages-html
185   (lambda
186     (peer)
187     "Render DM message history for a peer."
188     (let
189       ((rid (chat/dm peer))
190         (xs (chat/history (chat/dm peer))))
191       (html/div
192         (html/attr "id" "dm-messages")
193         (if
194           (null? xs)
195           (html/p* (html/attr "class" "muted") "No messages yet.")
196           (html/ul
197             (html/attr "class" "chat-list")
198             (html/join (list/map chat/msg-li xs)))))))))
199
200 (define chat/dm-page
201   (lambda
202     ()
203     "DM route handler: list, open, or post to a DM thread."
204     (let
205       ((peer (as-str (request-param "peer"))))
206       (if
207         (blank? peer)
208         (chat/dm-list-page)
209         (if
210           (string-eq (request-method) "POST")
211           (seq
212             (chat/say-dm peer (request-param "body"))
213             (if
214               (hx-request?)
215               (render-html (chat/dm-main-html peer))
216               (redirect (string-append "/chat/dm?peer=" peer))))
```

```
          (html/page
            (list
              (html/h1 (string-append "DM with " peer))
              (html/p*
                ""
                (html/join
                  (list
                    (html/a "/chat" "Lobby")
                    (html/span "" " ? ")
                    (html/a "/chat/dm" "All DMs"))))
              (chat/dm-main-html peer))))))))

(define chat/dm-peer-from-title
  (lambda
    (title me)
    "Extract peer username from a DM room title."
    (let
      ((t (as-str title)) (prefix "DM "))
      (if
        (string-contains? t prefix)
        (let
          ((rest (substring t 3)))
          (if
            (string-contains? rest (string-append me "/"))
            (substring rest (+ (string-length me) 1))
            (if
              (string-contains? rest (string-append "/" me))
              (substring
                rest
                0
                (- (string-length rest) (+ (string-length me) 1)))
              rest)))
        t))))

(define chat/dm-room-id
  (lambda
    (peer)
    "Compute the canonical DM room id for two users."
    (let
      ((me (as-str (whoami))) (them (as-str peer)))
      (if
        (chat/str-le? me them)
        (string-append "dm:" me ":" them)
        (string-append "dm:" them ":" me)))))

(define chat/dms
  (lambda
    ()
    "List all DM rooms the current user is a member of."
    (let
      ((me (as-str (whoami))))
      (list/filter
        (lambda
          (row)
          (let
            ((f (chat/row-fields row)))
            (if
              (string-eq (as-str (dict-get f "kind")) "dm")
              (chat/member? (dict-get f "members") me)
              #f)))
        (rows (quote chat-room) 100)))))

(define chat/ensure-lobby
  (lambda
    ()
    "Create the lobby chat-room if it does not exist."
    (let
      ((found (chat/find-room chat/lobby-id)))
      (if
        (null? found)
        (create
          (quote chat-room)
          (dict-set*
```

```
            "{}"
            (list
              "kind"
              "lobby"
              "room_id"
              chat/lobby-id
              "title"
              "Lobby"
              "members"
              "[]"
              "created_by"
              (as-str (whoami))
              "created_at"
              ""))))
        (chat/row-id found)))))

(define chat/find-room
  (lambda
    (room-id)
    "Find a chat-room row by room-id (scan up to 50)."
    (let
      ((want (as-str room-id)))
      (list/foldl
        (lambda
          (acc row)
          (if
            (null? acc)
            (if
              (string-eq (chat/room-id-of (chat/row-fields row)) want)
              row
              acc)
            acc))
        ()
        (rows (quote chat-room) 50)))))

(define chat/form-html
  (lambda
    ()
    "Render the lobby message form with htmx submit."
    (html/div
      (html/attr "class" "card")
      (html/form
        "POST"
        "/chat"
        (html/attrs
          (list
            (html/attr "hx-post" "/chat")
            (html/attr "hx-target" "#chat-main")
            (html/attr "hx-swap" "innerHTML")
            (html/attr
              "hx-on::after-request"
              "if(event.detail.successful){this.reset()}")))
        (html/join
          (list
            (html/label (html/attr "for" "chat-body") "Message")
            (html/textarea
              "body"
              (html/attrs
                (list
                  (html/attr "id" "chat-body")
                  (html/attr "rows" "2")
                  (html/attr "required" "required")
                  (html/attr
                    "placeholder"
                    "Hello from the lobby?")
                  (html/attr "maxlength" "4000")))
              "")
            (html/button (html/attr "type" "submit") "Send")))))))

(define chat/history
  (lambda
    (room-id)
    "Get messages for a room the user can access."
```

```
    (let
      ((want (as-str room-id)))
      (if
        (chat/can-access? want)
        (list/filter
          (lambda
            (row)
            (string-eq (chat/msg-room (chat/row-fields row)) want))
          (chat/all-messages))
        (list)))))

(define chat/lobby
  (lambda
    ()
    "Ensure the lobby exists and return its message history."
    (seq (chat/ensure-lobby) (chat/history chat/lobby-id))))

(define chat/lobby-id
  "lobby")

(define chat/main-html
  (lambda
    ()
    "Render the chat main area: form plus lobby messages."
    (html/div
      (html/attr "id" "chat-main")
      (html/join
        (list
          (chat/form-html)
          (html/h2 "Lobby")
          (chat/messages-html))))))

(define chat/member?
  (lambda
    (members-json user)
    "Check if a user is in a members JSON array string."
    (string-contains? (as-str members-json) (as-str user))))

(define chat/members-json
  (lambda
    (a b)
    "Build a 2-member JSON array string for a DM room."
    (string-append
      "[\""
      (as-str a)
      "\",\""
      (as-str b)
      "\"]")))

(define chat/messages-html
  (lambda
    ()
    "Render lobby messages with auto-refresh htmx polling."
    (let
      ((xs (chat/lobby)))
      (html/div
        (html/attrs
          (list
            (html/attr "id" "chat-messages")
            (html/attr "hx-get" "/chat/partial")
            (html/attr "hx-trigger" "every 4s")
            (html/attr "hx-swap" "innerHTML")))
        (if
          (null? xs)
          (html/p*
            (html/attr "class" "muted")
            "Say hi to the box.")
          (html/ul
            (html/attr "class" "chat-list")
            (html/join (list/map chat/msg-li xs)))))))))

(define chat/msg-body
  (lambda
```

```
    (fields)
    "Extract body field from a message fields dict."
    (as-str (dict-get fields "body"))))

(define chat/msg-from
  (lambda
    (fields)
    "Extract from field from a message fields dict."
    (as-str (dict-get fields "from"))))

(define chat/msg-li
  (lambda
    (row)
    "Render a chat message as an HTML list item."
    (let
      ((f (chat/row-fields row)))
      (html/li
        (html/attr "class" "card chat-msg")
        (html/join
          (list
            (html/p*
              ""
              (html/join
                (list
                  (html/strong (chat/msg-from f))
                  (html/span (html/attr "class" "muted") " ? "))))
            (html/p (html/text (chat/msg-body f)))))))))

(define chat/msg-room
  (lambda
    (fields)
    "Extract room_id field from a message fields dict."
    (as-str (dict-get fields "room_id"))))

(define chat/page
  (lambda
    ()
    "Chat lobby route handler: GET renders page, POST sends message."
    (if
      (string-eq (request-method) "POST")
      (seq
        (chat/say (request-param "body"))
        (if
          (hx-request?)
          (render-html (chat/main-html))
          (redirect "/chat")))
      (html/page
        (list
          (html/h1 "Chat")
          (html/p*
            (html/attr "class" "muted")
            (string-append
              "Lobby for everyone on this box. You are "
              (as-str (whoami))
              "."))
          (chat/main-html))))))

(define chat/partial
  (lambda
    ()
    "Render lobby messages as an htmx partial for polling."
    (render-html
      (let
        ((xs (chat/lobby)))
        (if
          (null? xs)
          (html/p*
            (html/attr "class" "muted")
            "Say hi to the box.")
          (html/ul
            (html/attr "class" "chat-list")
            (html/join (list/map chat/msg-li xs))))))))

```

```
(define chat/room-id-of
  (lambda
    (fields)
    "Extract room_id field from a chat-room fields dict."
    (as-str (dict-get fields "room_id"))))

(define chat/row-fields
  (lambda
    (row)
    "Get fields dict from a chat row."
    (cdr row)))

(define chat/row-id
  (lambda
    (row)
    "Get the id from a chat row."
    (as-str (car row))))

(define chat/say
  (lambda
    (body)
    "Say a message in the lobby."
    (chat/say-in chat/lobby-id body)))

(define chat/say-dm
  (lambda
    (peer body)
    "Say a direct message to a peer user."
    (let
      ((rid (chat/dm peer)))
      (if
        (blank? rid)
        "chat: bad peer"
        (chat/say-in rid body)))))

(define chat/say-in
  (lambda
    (room-id body)
    "Say a message in a specific chat room (with access check)."
    (let
      ((text (as-str body))
        (user (as-str (whoami)))
        (rid (as-str room-id)))
      (if
        (blank? text)
        ""
        (if
          (> (string-length text) 4000)
          "chat: body too long (max 4000)"
          (if
            (chat/can-access? rid)
            (seq
              (if
                (string-eq rid chat/lobby-id)
                (chat/ensure-lobby)
                ())
              (create
                (quote chat-message)
                (dict-set*
                  "{}"
                  (list
                    "room_id"
                    rid
                    "from"
                    user
                    "body"
                    text
                    "created_at"
                    "")))
              text)
            "chat: not a member of room"))))))

(define chat/str-le?
```

```
  (lambda
    (a b)
    "Lexicographic string less-than-or-equal via char ranks."
    (if
      (string-eq a b)
      #t
      (if
        (string-eq a "")
        #t
        (if
          (string-eq b "")
          #f
          (let
            ((ca (substring a 0 1)) (cb (substring b 0 1)))
            (if
              (string-eq ca cb)
              (chat/str-le? (substring a 1) (substring b 1))
              (<= (chat/char-rank ca) (chat/char-rank cb)))))))))

(define chat/users
  (lambda
    ()
    "List all usernames on the image."
    (list-usernames)))
```

159 libs/contacts/lib.pp

```
;; package contacts -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push

(define contacts/add-name
  (lambda
    (name)
    "Create a contact with just the name provided"
    (create
      (quote contact)
      (dict-set*
        "{}"
        (list
          "name"
          (as-str name)
          "email"
          ""
          "phone"
          ""
          "notes"
          "")))))

(define contacts/all
  (lambda
    ()
    "Return up to 100 contact rows"
    (rows (quote contact) 100)))

(define contacts/blank?
  (lambda
    (s)
    "Check if string is null or empty"
    (if (null? s) #t (string-eq (as-str s) ""))))

(define contacts/create
  (lambda
    (name email phone notes)
    "Create a new contact with name, email, phone, notes"
    (create
      (quote contact)
      (contacts/fields name email phone notes))))

(define contacts/delete
  (lambda
    (id)
    "Delete a single contact by id"
    (delete (quote contact) (as-str id))))

(define contacts/email
  (lambda
    (fields)
    "Extract email from contact fields object"
    (as-str (dict-get fields "email"))))

(define contacts/fields
  (lambda
    (name email phone notes)
    "Build contact fields dict from name, email, phone, notes"
    (dict-set*
      "{}"
      (list
        "name"
        (as-str name)
        "email"
        (as-str email)
        "phone"
        (as-str phone)
        "notes"
        (as-str notes)))))

(define contacts/form-html
  (lambda
```

```
    ()
    "Render the HTML contact form"
    (html/div
      (html/attr "class" "card")
      (html/join
        (list
          (html/h2 "Add contact")
          (html/form
            "POST"
            "/contacts"
            (html/attrs
              (list
                (html/attr "hx-post" "/contacts")
                (html/attr "hx-target" "#contacts-main")
                (html/attr "hx-swap" "innerHTML")
                (html/attr
                  "hx-on::after-request"
                  "if(event.detail.successful){this.reset()}")))
            (html/join
              (list
                (html/hidden "action" "create")
                (html/label (html/attr "for" "c-name") "Name")
                (html/text-input
                  "name"
                  (html/attrs
                    (list
                      (html/attr "id" "c-name")
                      (html/attr "required" "required")
                      (html/attr "placeholder" "Ada Lovelace")
                      (html/attr "autocomplete" "name"))))
                (html/label (html/attr "for" "c-email") "Email")
                (html/text-input
                  "email"
                  (html/attrs
                    (list
                      (html/attr "id" "c-email")
                      (html/attr "placeholder" "ada@example.com")
                      (html/attr "autocomplete" "email"))))
                (html/label (html/attr "for" "c-phone") "Phone")
                (html/text-input
                  "phone"
                  (html/attrs
                    (list
                      (html/attr "id" "c-phone")
                      (html/attr "placeholder" "+1 ?")
                      (html/attr "autocomplete" "tel"))))
                (html/label (html/attr "for" "c-notes") "Notes")
                (html/textarea
                  "notes"
                  (html/attrs
                    (list
                      (html/attr "id" "c-notes")
                      (html/attr "rows" "2")
                      (html/attr "placeholder" "optional")))
                  "")
                (html/button (html/attr "type" "submit") "Add")))))))))

(define contacts/get
  (lambda
    (id)
    "Get a specific contact by id (returns id and fields)"
    (let
      ((row (get (quote contact) (as-str id))))
      (if (null? row) () (cons (as-str id) row)))))

(define contacts/handle-post
  (lambda
    ()
    "Haitch-post handler for create/delete contacts"
    (let
      ((action (as-str (request-param "action"))))
      (cond
        ((string-eq action "delete")
```

```
          (contacts/delete (request-param "id")))
        ((string-eq action "create")
          (let
            ((name (request-param "name")))
            (unless
              (blank? name)
              (contacts/create
                name
                (request-param "email")
                (request-param "phone")
                (request-param "notes")))))
        (else ())))))

(define contacts/index-html
  (lambda
    ()
    "Render the contacts page with form and list"
    (html/page
      (list
        (html/h1 "Contacts")
        (html/p*
          (html/attr "class" "muted")
          "Manage your contacts")
        (contacts/form-html)
        (contacts/list-html)))))

(define contacts/item-li
  (lambda
    (row)
    "Render a single contact as HTML list item"
    (let
      ((id (contacts/row-id row))
        (fields (contacts/row-fields row)))
      (let
        ((name (contacts/name fields))
          (email (contacts/email fields))
          (phone (contacts/phone fields))
          (notes (contacts/notes fields)))
        (html/li
          (html/attr "class" "card")
          (html/join
            (list
              (html/div
                (html/attr "class" "contact-name")
                (html/strong (html/text name)))
              (unless
                (blank? email)
                (html/p*
                  (html/attr "class" "muted")
                  (html/a (string-append "mailto:" email) email)))
              (unless
                (blank? phone)
                (html/p* (html/attr "class" "muted") (html/text phone)))
              (unless (blank? notes) (html/p (html/text notes)))
              (html/form
                "POST"
                "/contacts"
                (html/attrs
                  (list
                    (html/attr "style" "display:inline")
                    (html/attr "hx-post" "/contacts")
                    (html/attr "hx-target" "#contacts-main")
                    (html/attr "hx-swap" "innerHTML")))
                (html/join
                  (list
                    (html/hidden "action" "delete")
                    (html/hidden "id" id)
                    (html/button (html/attr "type" "submit")
    "Delete")))))))))))

(define contacts/list-html
  (lambda
    ()
```

```
    "Render the list of contacts using the item-li view"
    (html/div
      (html/attr "id" "contacts-list")
      (list-or-empty
        (contacts/all)
        "No contacts yet."
        contacts/item-li))))

(define contacts/main-html
  (lambda
    ()
    "Render main contacts display with form and list"
    (html/div
      (html/attr "id" "contacts-main")
      (html/join
        (list
          (contacts/form-html)
          (html/h2 "All contacts")
          (contacts/list-html))))))

(define contacts/name
  (lambda
    (fields)
    "name field accessor for contact data"
    (as-str (dict-get fields "name"))))

(define contacts/notes
  (lambda
    (fields)
    "notes field accessor for contact data"
    (as-str (dict-get fields "notes"))))

(define contacts/page
  (lambda
    ()
    "contacts route handler (GET/POST)"
    (if
      (string-eq (request-method) "POST")
      (seq
        (contacts/handle-post)
        (if
          (hx-request?)
          (render-html (contacts/main-html))
          (redirect "/contacts")))
      (html/page
        (list
          (html/h1 "Contacts")
          (html/p*
            (html/attr "class" "muted")
            "Simple address book on data_rows (type contact).")
          (contacts/main-html))))))

(define contacts/phone
  (lambda
    (fields)
    "phone field accessor for contact data"
    (as-str (dict-get fields "phone"))))

(define contacts/row-fields
  (lambda
    (row)
    "Get fields from a contact row"
    (cdr row)))

(define contacts/row-id
  (lambda
    (row)
    "Get id from a contact row"
    (as-str (car row))))

(define contacts/update
  (lambda
    (id name email phone notes)
```

```
290     "update a contact with new name, email, phone, notes"
291     (update
292       (quote contact)
293       (as-str id)
294       (contacts/fields name email phone notes))))
```

160 libs/css/lib.pp

```
;; package css -- exported from live branch admin/personal
;; Sync: scripts/pp_sync.py pull|push

(define css/media
  (lambda
    (query items)
    (list "media" (as-str query) items)))

(define css/prop-name
  (lambda (k) (as-str k)))

(define css/prop-value
  (lambda (v) (as-str v)))

(define css/raw
  (lambda (s) (list "raw" (as-str s))))

(define css/render
  (lambda
    (sheet)
    (if
      (string-eq (type-of sheet) "lambda")
      (css/render (sheet))
      (if
        (null? sheet)
        ""
        (if
          (string-eq (as-str (car sheet)) "sheet")
          (css/render-items (car (cdr sheet)))
          (css/render-item sheet))))))

(define css/render-decls
  (lambda
    (props)
    (if
      (null? props)
      ""
      (if
        (null? (cdr props))
        ""
        (string-append
          (css/prop-name (car props))
          ":"
          (css/prop-value (car (cdr props)))
          ";"
          (css/render-decls (cdr (cdr props))))))))

(define css/render-item
  (lambda
    (item)
    (if
      (null? item)
      ""
      (let
        ((kind (as-str (car item))))
        (if
          (string-eq kind "rule")
          (let
            ((sel (car (cdr item))) (props (car (cdr (cdr item)))))
            (string-append
              (as-str sel)
              "{"
              (css/render-decls props)
              "}\n"))
          (if
            (string-eq kind "media")
            (let
              ((q (car (cdr item))) (kids (car (cdr (cdr item)))))
              (string-append
                "@media("
                (as-str q)
```

```
                "){\n"
                (css/render-items kids)
                "}\n"))
            (if
              (string-eq kind "raw")
              (string-append (as-str (car (cdr item))) "\n")
              (if
                (string-eq kind "use")
                (css/render (car (cdr item)))
                (if
                  (string-eq kind "sheet")
                  (css/render-items (car (cdr item)))
                  "")))))))))

(define css/render-items
  (lambda
    (items)
    (list/foldl
      (lambda
        (acc it)
        (string-append acc (css/render-item it)))
      ""
      items)))

(define css/rule
  (lambda
    (selector props)
    (list "rule" (as-str selector) props)))

(define css/sheet
  (lambda (items) (list "sheet" items)))

(define css/use
  (lambda (sheet) (list "use" sheet)))
```

161 libs/dabar/lib.pp

```
;; package dabar -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push
;;
;; Study layer on top of bible/*: Strong's-linked interlinear + /dabar chrome.
;; Interlinear rows: type bible-interlinear (seed: scripts/seed_interlinear.py).
;; Toggle: request param inter=1 shows token HTML; default plain (inter=0).
;; Chapter-reader decision (v1): chapter view stays bible-owned; each study
;; path is /dabar?book=&chapter=&verse=&inter=1 (no inline chapter tokens).

(define dabar/inter-html
  (lambda
    (book chapter verse)
    (let
      ((row (dabar/inter-row book chapter verse)))
      (if
        (null? row)
        (html/p*
          (html/attr "class" "muted")
          (string-append
            "No interlinear for "
            (bible/ref book chapter verse)
            " -- plain: "
            (as-str (bible/verse book chapter verse))))
        (let
          ((pre (dict-get (cdr row) "html")))
          (if
            (null? pre)
            (html/p*
              (html/attr "class" "muted")
              "interlinear row has no html field")
            pre)))))
  "Return the interlinear HTML for one verse, or a 'No interlinear for <ref>' fallback.")

;; Canonical alias -- spec smoke (dabar/inter "John" 1 1).
(define dabar/inter
  (lambda
    (book chapter verse)
    (dabar/inter-html book chapter verse))  "Canonical alias for interlinear HTML; e.g. (dabar/inter book chapter verse).")

(define dabar/inter-row
  (lambda
    (book chapter verse)
    (let
      ((rows
        (find-rows
          (quote bible-interlinear)
          "ref"
          (bible/ref book chapter verse)
          1)))
      (if (null? rows) () (car rows))))
  "Fetch one bible-interlinear row as (id . fields) pair, or () if missing. Matches find-rows shape.")

;; Truthy when inter=1 (default off / plain).
(define dabar/inter-on?
  (lambda
    ()
    (let
      ((p (request-param "inter")))
      (if
        (null? p)
        #f
        (string-eq (as-str p) "1"))))  "Truthy when the inter=1 request param is set (plain view is the default).")

(define dabar/study-href
  (lambda
```

```
    (book chapter verse inter-on)
    (string-append
      "/dabar?book="
      (bible/abbrev-of book)
      "&chapter="
      (as-str chapter)
      "&verse="
      (as-str verse)
      "&inter="
      (if inter-on "1" "0")))  "/dabar URL for a verse study, including an intercross-toggling inter= param.")

(define dabar/toggle-link
  (lambda
    (book chapter verse inter-on)
    (let
      ((next (not inter-on))
        (label (if inter-on "plain" "interlinear")))
      (html/tag
        "a"
        (html/attrs
          (list
            (html/attr "href" (dabar/study-href book chapter verse next))
            (html/attr "hx-get" (dabar/study-href book chapter verse next))
            (html/attr "hx-target" "#dabar-panel")
            (html/attr "hx-swap" "innerHTML")
            (html/attr "hx-push-url" "true")
            (html/attr "class" "dabar-inter-toggle")))
        label)))  "Interlinear<->plain toggle link targeting #dabar-panel.")

(define dabar/page
  (lambda
    ()
    (if
      (string-eq (request-method) "POST")
      (let
        ((book (request-param "book"))
          (chapter (request-param "chapter"))
          (verse (request-param "verse"))
          (inter (request-param "inter")))
        (redirect
          (string-append
            "/dabar?book="
            (if (null? book) "John" (as-str book))
            "&chapter="
            (if (null? chapter) "3" (as-str chapter))
            (if
              (null? verse)
              ""
              (if
                (string-eq (as-str verse) "")
                ""
                (string-append "&verse=" (as-str verse))))
            (if
              (null? inter)
              ""
              (if
                (string-eq (as-str inter) "")
                ""
                (string-append "&inter=" (as-str inter)))))))))
      (if
        (hx-request?)
        (dabar/panel-for-request)
        (html/page
          (list
            (html/div
              (html/attr "id" "dabar-panel")
              (dabar/panel-for-request)))))))  "/dabar route handler: redirects POSTs, else renders the panel (htmx request or full page).")

(define dabar/panel
  (lambda
    ()
```

```
    (let
      ((book (request-param "book"))
        (chapter (request-param "chapter"))
        (verse (request-param "verse"))
        (ref (request-param "ref")))
      (if
        (null? ref)
        (let
          ((b (if (null? book) "John" (as-str book)))
            (c (if (null? chapter) "3" (as-str chapter)))
            (v (if (null? verse) "16" (as-str verse))))
          (dabar/verse-study b c v))
        (let
          ((row (bible/parse-ref ref)))
          (if
            (null? row)
            (html/p*
              (html/attr "class" "err")
              (string-append "unknown ref " (as-str ref)))
            (dabar/verse-study
              (dict-get row "abbrev")
              (dict-get row "chapter")
              (dict-get row "verse")))))))  "Resolve request params
     (book/chapter/verse or ref=) into a verse-study panel.")

(define dabar/panel-for-request
  (lambda
    ()
    (let
      ((ref (request-param "ref"))
        (verse (request-param "verse")))
      (let
        ((has-ref
            (if (null? ref) #f (not (string-eq (as-str ref) ""))))
          (has-verse
            (if
              (null? verse)
              #f
              (not (string-eq (as-str verse) "")))))
        (if
          has-ref
          (dabar/verse-study-from-ref ref)
          (if
            has-verse
            (dabar/verse-study-from-params
              (request-param "book")
              (request-param "chapter")
              verse)
            (bible/chapter-reader))))))  "Dispatch a request by ref=,
     verse=, or fall back to the chapter reader.")

(define dabar/verse-study
  (lambda
    (book chapter verse)
    (let
      ((ab (bible/abbrev-of book))
        (ch (bible/safe-chapter book chapter))
        (inter-on (dabar/inter-on?)))
      (html/join
        (list
          (html/h1
            (string-append
              (as-str book)
              " "
              (as-str ch)
              ":"
              (as-str verse)))
          (bible/nav-links ab ch)
          (html/div
            (html/attr "class" "card")
            (html/join
              (list
                (html/h2 "BSB")
```

```
                (html/p (bible/verse book ch verse)))))
          (html/div
            (html/attr "class" "card dabar-inter")
            (html/join
              (list
                (html/h2
                  (html/join
                    (list
                      "Interlinear ? "
                      (dabar/toggle-link book ch verse inter-on))))
                (if
                  inter-on
                  (dabar/inter-html book ch verse)
                  (html/p*
                    (html/attr "class" "muted")
                    (html/join
                      (list
                        "Plain view. "
                        (dabar/toggle-link book ch verse #f)
                        " for original-language tokens + Strong's."))))))))
          (html/p*
            (html/attr "class" "muted")
            (html/join
              (list
                (html/a
                  (string-append
                    "/dabar?book="
                    (bible/abbrev-of book)
                    "&chapter="
                    (as-str ch))
                  "open in reader")
                " ? "
                (html/a "/strongs" "Strong's"))))))))  "Full verse-study
    panel: heading, BSB card, interlinear card, and reader/Strong's
    links.")

(define dabar/verse-study-from-params
  (lambda
    (book chapter verse)
    (let
      ((b (if (null? book) "John" (as-str book)))
        (c (if (null? chapter) "3" (as-str chapter)))
        (v (as-str verse)))
      (dabar/verse-study
        b
        (bible/safe-chapter b c)
        (bible/parse-int v))))  "Render verse-study from request
    book/chapter/verse params, applying defaults.")

(define dabar/verse-study-from-ref
  (lambda
    (ref)
    (let
      ((row (bible/parse-ref ref)))
      (if
        (null? row)
        (html/join
          (list
            (bible/go-form "John" "3" ())
            (html/p*
              (html/attr "class" "err")
              (string-append "Unknown ref " (as-str ref)))))
        (let
          ((book (dict-get row "book"))
            (chapter (dict-get row "chapter"))
            (verse (dict-get row "verse")))
          (html/join
            (list
              (bible/go-form book chapter verse)
              (dabar/verse-study book chapter verse)))))))  "Render
    verse-study from a plain ref string, or an unknown-ref error with a
    go-form.")

```

```
(route "/dabar" "dabar/page")

;; ---- daily-scripture (borge/dabar.borg daily-scripture) ------------
;; Verse-of-the-day, pushed to yourself: deterministic day-indexed pick,
;; delivered into the calendar's daily note and pinged via ntfy.
;; Requires: bible/*, notes/doc/*, outline/*, ntfy/* (seeded libs).

(define dabar/day-index
  (lambda
    ()
    "Days since 1970-01-01 in image-local time -- the daily cycle."
    (date-diff-days "1970-01-01" (date-today))))

(define dabar/daily-ref
  (lambda
    ()
    "Verse-of-the-day as (abbrev chapter verse). Deterministic and
stateless: book = day mod 66 (canonical ordinal), chapter = day within
the book's chapters, verse = day within the chapter's verse count.
Same verse for every user on the same day; survives re-fire; cycles
annually-ish. bible/random-row stays the Surprise control."
    (let
      ((n (dabar/day-index)))
      (let
        ((book-fields (bible/book-by-ordinal (+ 1 (mod n 66)))))
        (let
          ((book
            (if
              (null? book-fields)
              (error "dabar/daily-ref: bible corpus empty -- run
    scripts/seed_bible_bsb.py")
              (as-str (dict-get book-fields "abbrev")))))
          (let
            ((c (+ 1 (mod (quotient n 66) (bible/chapter-count book)))))
            (let
              ((vs (bible/chapter book c)))
              (let
                ((vc (count vs)))
                (list
                  book
                  c
                  (if (<= vc 0) 1 (+ 1 (mod n vc))))))))))

(define dabar/starts-with?
  (lambda
    (s prefix)
    (if
      (< (string-length s) (string-length prefix))
      #f
      (string-eq (substring s 0 (string-length prefix)) prefix))))

(define dabar/daily-has?
  (lambda
    (tree marker)
    "True when a root-level kid body already starts with marker (today's
    ref was delivered -- idempotency for job re-fires)."
    (dabar/daily-has-loop (outline/kids tree) marker)))

(define dabar/daily-has-loop
  (lambda
    (kids marker)
    (if
      (null? kids)
      #f
      (if
        (dabar/starts-with? (outline/body (car kids)) marker)
        #t
        (dabar/daily-has-loop (cdr kids) marker)))))

(define dabar/daily-click
  (lambda
    (book c v)
```

```
    "Absolute study URL for the ntfy Click: header (empty when no site
     configured)."
    (let
      ((cfg (guard (lambda (e) ()) (ntfy/config))))
      (let
        ((site-f (if (null? cfg) () (dict-get cfg "site"))))
        (let
          ((site (if (null? site-f) "" (as-str site-f))))
          (if
            (string-eq site "")
            ""
            (string-append
              site
              "/dabar?book="
              (as-str book)
              "&chapter="
              (as-str c)
              "&verse="
              (as-str v))))))))

(define dabar/daily-deliver
  (lambda
    ()
    "Verse-of-the-day -> append to the daily note (daily-YYYY-MM-DD,
the calendar package's slug convention) as a (bible ?) ref node, then
ntfy-push with a Click: straight to the dabar study view. Idempotent
per day; returns a short status string."
    (let
      ((r (dabar/daily-ref)))
      (let
        ((book (car r))
          (c (cadr r))
          (v (caddr r)))
        (let
          ((ref-str
            (string-append
              (as-str book) " " (as-str c) ":" (as-str v)))
            (marker
              (string-append "(bible " (as-str book) " " (as-str c) " "
    (as-str v) ")"))
            (text (as-str (bible/verse book c v)))
            (day (date-today)))
          (let
            ((slug (string-append "daily-" day)))
            (let
              ((prev (notes/doc/get slug)))
              (let
                ((tree
                  (if
                    (null? prev)
                    (notes/doc/empty-outline day)
                    (notes/doc/outline prev))))
                (if
                  (dabar/daily-has? tree marker)
                  (string-append "already: " ref-str)
                  (let
                    ((node
                      (outline/set-body
                        (outline/node
                          (string-append "dv-" (as-str (now)))
                          (string-append "Daily scripture -- " ref-str))
                        (string-append marker "\n\n" text "\n"))))
                    (let
                      ((new-tree (outline/add-child-by-id tree "root"
    node)))
                      (let
                        ((title (string-append "Daily scripture -- "
    ref-str)))
                        (let
                          ((payload
                            (guard
                              (lambda (e) (str e))
```

```
                            (ntfy/send/click title text
    (dabar/daily-click book c v)))))
                        (seq
                          (notes/doc/upsert slug day new-tree "daily"
    "note" day)
                          (string-append
                            "delivered: "
                            ref-str
                            (if (string-eq (as-str payload) "")
                                ""
                                (string-append " (ntfy: " (as-str
    payload) ")"))))))))))))))))))
```

162 libs/data/lib.pp

```
;; package data -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push

(define data/get-field
  (lambda
    (type-name id field)
    (dict-get (get type-name id) field)))
```

163 libs/edit/lib.pp

```
;; Film edit -- studio library, cut graph, sound lines/cues, EDL export.
;; Prefix: edit/   Package dir: edit
;; Install: (load-library "edit") -- discovered via libs/edit/lib.pp
;;
;; Types use edit- prefix to avoid colliding with generic names.
;; Spec chapter: borge/film-edit.borg (film-edit)
;;
;; v1: rows + expand + canonical hashes + text exports (MLT, Blender
;; VSE script, cue sheet, ffmpeg script). No gen API, no Blender
;; bridge, no automated bounce (those need kernel prims / v2).

(deftype 'edit-asset
  'project 'slug 'kind 'summary 'bible 'appearance 'negative
  'primary_hash 'primary_path 'ref_hashes 'voice_ref 'lora_ref
  'media_id 'meta)

(deftype 'edit-project
  'slug 'title 'fps 'resolution 'budget_cents 'notes)

(deftype 'edit-shot
  'project 'ord 'slug 'summary 'prompt 'dur_target)

(deftype 'edit-shot-cast
  'shot 'asset 'wardrobe 'role)

(deftype 'edit-shot-ref
  'shot 'asset 'purpose 'weight)

(deftype 'edit-take
  'shot 'n 'backend 'model 'seed 'cost_cents
  'media_hash 'media_path 'state 'rating
  'input_snapshot 'source_assets)

(deftype 'edit-edl-entry
  'project 'cut 'ord 'take 'tc_in 'dur 'transition 'audio)

(deftype 'edit-render-job
  'project 'cut 'edl_hash 'state 'out_hash 'out_path 'log)

(deftype 'edit-line
  'project 'shot 'ord 'character 'text 'notes 'take)

(deftype 'edit-cue
  'project 'cut 'ord 'mode 'asset 'tc_start 'dur 'gain_db 'loop)

(deftype 'edit-mix-job
  'project 'cut 'cue_hash 'profile 'state 'out_hash 'out_path 'log)

;; --- helpers ---

(define edit/blank?
  (lambda (v)
    (if (null? v)
        #t
        (string-eq (as-str v) ""))))
  "True if value is null or empty string.")

(define edit/as-int
  (lambda (v)
    (if (null? v) 0 (parse-int (as-str v))))
  "Coerce value to int.")

(define edit/id
  (lambda (row)
    (if (pair? row)
        (as-str (car row))
        ""))
  "Row id string; empty if value is a bare fields dict from get.")

(define edit/fields
  (lambda (row)
```

```
    (if (pair? row)
        (cdr row)
        row))
  "Fields JSON from a rows pair or a bare get result.")

(define edit/f
  (lambda (row key)
    (as-str (dict-get (edit/fields row) key)))
  "Field string from rows pair or get result.")

(define edit/tr (lambda (body) (html/tag "tr" "" body)))
(define edit/td (lambda (body) (html/tag "td" "" body)))
(define edit/th (lambda (body) (html/tag "th" "" body)))
(define edit/table (lambda (body) (html/tag "table" "" body)))
(define edit/thead (lambda (body) (html/tag "thead" "" body)))
(define edit/tbody (lambda (body) (html/tag "tbody" "" body)))

(define edit/input
  (lambda (name value)
    (html/text-input name (html/attr "value" (as-str value))))
  "Text input with value= attr (edit/input takes attrs, not raw value).")

(define edit/escape-xml
  (lambda (s)
    (list/foldl
      (lambda (acc ch)
        (string-append acc
          (if (string-eq ch "&") "&amp;"
            (if (string-eq ch "<") "&lt;"
              (if (string-eq ch ">") "&gt;"
                (if (string-eq ch "\"") "&quot;" ch))))))
      ""
      (edit/chars (as-str s))))
  "Minimal XML escape.")

(define edit/chars
  (lambda (s)
    (edit/chars-acc s 0 (string-length s) (list)))
  "Split string into 1-char strings.")

(define edit/chars-acc
  (lambda (s i n acc)
    (if (>= i n)
        (reverse acc)
        (edit/chars-acc s (+ i 1) n
          (cons (substring s i 1) acc)))))

(define edit/join-lines
  (lambda (xs)
    (list/foldl
      (lambda (acc x)
        (if (string-eq acc "")
            (as-str x)
            (string-append acc "\n" (as-str x))))
      ""
      xs))
  "Join strings with newlines.")

(define edit/pad2
  (lambda (n)
    (let ((s (as-str n)))
      (if (< (edit/as-int n) 10)
          (string-append "0" s)
          s)))
  "Zero-pad small ints for timecode.")

(define edit/mod
  (lambda (n d)
    (if (<= (edit/as-int d) 0)
        0
        (- (edit/as-int n)
           (* (/ (edit/as-int n) (edit/as-int d)) (edit/as-int d)))))
  "Non-negative integer modulo via truncating division.")
```

```
145
146 (define edit/frames->tc
147   (lambda (frames fps)
148     (let ((f (edit/as-int frames))
149           (r (if (<= (edit/as-int fps) 0) 24 (edit/as-int fps))))
150       (let ((ff (edit/mod f r))
151             (total-sec (/ f r)))
152         (let ((ss (edit/mod total-sec 60))
153               (total-min (/ total-sec 60)))
154           (let ((mm (edit/mod total-min 60))
155                 (hh (/ total-min 60)))
156             (string-append
157               (edit/pad2 hh) ":"
158               (edit/pad2 mm) ":"
159               (edit/pad2 ss) ":"
160               (edit/pad2 ff)))))))
161   "Integer frames to HH:MM:SS:FF at fps.")
162
163 ;; --- projects ---
164
165 (define edit/projects
166   (lambda ()
167     (rows 'edit-project 100))
168   "List edit projects.")
169
170 (define edit/project-by-slug
171   (lambda (slug)
172     (let ((want (as-str slug)))
173       (list/foldl
174         (lambda (acc row)
175           (if (null? acc)
176               (if (string-eq (edit/f row "slug") want) row acc)
177               acc))
178         ()
179         (edit/projects))))
180   "Find project row by slug.")
181
182 (define edit/project-get
183   (lambda (id) (get 'edit-project (as-str id)))
184   "Get project by id.")
185
186 (define edit/project-upsert
187   (lambda (slug title fps resolution budget-cents notes)
188     (let ((existing (edit/project-by-slug slug)))
189       (if (null? existing)
190           (create 'edit-project
191             (dict-set* "{}"
192               (list
193                 "slug" (as-str slug)
194                 "title" (as-str title)
195                 "fps" (as-str fps)
196                 "resolution" (as-str resolution)
197                 "budget_cents" (as-str budget-cents)
198                 "notes" (as-str notes))))
199           (seq
200             (update 'edit-project (edit/id existing)
201               (dict-set* (edit/fields existing)
202                 (list
203                   "title" (as-str title)
204                   "fps" (as-str fps)
205                   "resolution" (as-str resolution)
206                   "budget_cents" (as-str budget-cents)
207                   "notes" (as-str notes))))
208             (edit/id existing)))))
209   "Create or update project by slug; returns id.")
210
211 ;; --- assets (studio library) ---
212
213 (define edit/assets
214   (lambda ()
215     (rows 'edit-asset 200))
216   "All assets on branch.")
217
```

```
(define edit/assets-for
  (lambda (project kind)
    (let ((p (as-str project))
          (k (as-str kind)))
      (list/filter
        (lambda (row)
          (let ((ok-p
                  (if (edit/blank? p)
                      #t
                      (or (string-eq (edit/f row "project") p)
                          (string-eq (edit/f row "project") ""))))
                (ok-k
                  (if (edit/blank? k)
                      #t
                      (string-eq (edit/f row "kind") k))))
            (and ok-p ok-k)))
        (edit/assets))))
  "Filter assets by project (incl studio-wide) and optional kind.")

(define edit/asset-by-slug
  (lambda (project slug)
    (let ((p (as-str project))
          (s (as-str slug)))
      (list/foldl
        (lambda (acc row)
          (if (null? acc)
              (if (and (string-eq (edit/f row "slug") s)
                       (or (string-eq (edit/f row "project") p)
                           (string-eq (edit/f row "project") "")))
                  row
                  acc)
              acc))
        ()
        (edit/assets))))
  "Lookup asset: prefer project scope then studio-wide (first match).")

(define edit/asset-get
  (lambda (id) (get 'edit-asset (as-str id)))
  "Get asset by id.")

(define edit/asset-upsert
  (lambda (project slug kind summary bible appearance negative
           primary-hash primary-path ref-hashes voice-ref lora-ref
           media-id meta)
    (let ((existing (edit/asset-by-slug project slug)))
      (let ((fields
              (dict-set* "{}"
                (list
                  "project" (as-str project)
                  "slug" (as-str slug)
                  "kind" (as-str kind)
                  "summary" (as-str summary)
                  "bible" (as-str bible)
                  "appearance" (as-str appearance)
                  "negative" (as-str negative)
                  "primary_hash" (as-str primary-hash)
                  "primary_path" (as-str primary-path)
                  "ref_hashes" (as-str ref-hashes)
                  "voice_ref" (as-str voice-ref)
                  "lora_ref" (as-str lora-ref)
                  "media_id" (as-str media-id)
                  "meta" (as-str meta)))))
        (if (null? existing)
            (create 'edit-asset fields)
            (seq
              (update 'edit-asset (edit/id existing)
                (dict-set* (edit/fields existing)
                  (list
                    "kind" (as-str kind)
                    "summary" (as-str summary)
                    "bible" (as-str bible)
                    "appearance" (as-str appearance)
                    "negative" (as-str negative)
```

```
                    "primary_hash" (as-str primary-hash)
                    "primary_path" (as-str primary-path)
                    "ref_hashes" (as-str ref-hashes)
                    "voice_ref" (as-str voice-ref)
                    "lora_ref" (as-str lora-ref)
                    "media_id" (as-str media-id)
                    "meta" (as-str meta))))
              (edit/id existing))))))
  "Upsert asset by project+slug; returns id.")

;; --- shots ---

(define edit/shots
  (lambda (project)
    (let ((p (as-str project)))
      (edit/sort-by-ord
        (list/filter
          (lambda (row) (string-eq (edit/f row "project") p))
          (rows 'edit-shot 300)))))
  "Shots for project slug, ordered.")

(define edit/shot-get
  (lambda (id) (get 'edit-shot (as-str id)))
  "Get shot by id.")

(define edit/shot-create
  (lambda (project ord slug summary prompt dur-target)
    (create 'edit-shot
      (dict-set* "{}"
        (list
          "project" (as-str project)
          "ord" (as-str ord)
          "slug" (as-str slug)
          "summary" (as-str summary)
          "prompt" (as-str prompt)
          "dur_target" (as-str dur-target)))))
  "Create shot; returns id.")

(define edit/sort-by-ord
  (lambda (rows-in)
    (edit/sort-rows rows-in "ord"))
  "Sort rows by integer ord field ascending.")

(define edit/sort-rows
  (lambda (xs key)
    (if (null? xs)
        (list)
        (edit/insert-sorted
          (car xs)
          (edit/sort-rows (cdr xs) key)
          key)))
  "Insertion sort by integer field.")

(define edit/insert-sorted
  (lambda (row xs key)
    (if (null? xs)
        (list row)
        (if (<= (edit/as-int (edit/f row key))
                (edit/as-int (edit/f (car xs) key)))
            (cons row xs)
            (cons (car xs) (edit/insert-sorted row (cdr xs) key)))))
  "Insert row into sorted list by key.")

;; --- cast / refs ---

(define edit/shot-casts
  (lambda (shot-id)
    (let ((s (as-str shot-id)))
      (list/filter
        (lambda (row) (string-eq (edit/f row "shot") s))
        (rows 'edit-shot-cast 200))))
  "Cast bindings for a shot.")

```

```
(define edit/shot-cast
  (lambda (shot-id asset-id role wardrobe-id)
    (create 'edit-shot-cast
      (dict-set* "{}"
        (list
          "shot" (as-str shot-id)
          "asset" (as-str asset-id)
          "wardrobe" (as-str wardrobe-id)
          "role" (as-str role)))))
  "Bind asset to shot as cast.")

(define edit/shot-cast-clear
  (lambda (shot-id)
    (list/map
      (lambda (row) (delete 'edit-shot-cast (edit/id row)))
      (edit/shot-casts shot-id)))
  "Remove all cast bindings for shot.")

(define edit/shot-refs
  (lambda (shot-id)
    (let ((s (as-str shot-id)))
      (list/filter
        (lambda (row) (string-eq (edit/f row "shot") s))
        (rows 'edit-shot-ref 200))))
  "Ref bindings for a shot.")

(define edit/shot-ref
  (lambda (shot-id asset-id purpose weight)
    (create 'edit-shot-ref
      (dict-set* "{}"
        (list
          "shot" (as-str shot-id)
          "asset" (as-str asset-id)
          "purpose" (as-str purpose)
          "weight" (as-str weight)))))
  "Bind non-cast ref asset to shot.")

(define edit/shots-using-asset
  (lambda (asset-id)
    (let ((a (as-str asset-id)))
      (let ((cast-shots
              (list/map
                (lambda (row) (edit/f row "shot"))
                (list/filter
                  (lambda (row)
                    (or (string-eq (edit/f row "asset") a)
                        (string-eq (edit/f row "wardrobe") a)))
                  (rows 'edit-shot-cast 500))))
            (ref-shots
              (list/map
                (lambda (row) (edit/f row "shot"))
                (list/filter
                  (lambda (row) (string-eq (edit/f row "asset") a))
                  (rows 'edit-shot-ref 500)))))
        (edit/unique-strs (edit/append cast-shots ref-shots)))))
  "Shot ids that bind this asset via cast or ref.")

(define edit/append
  (lambda (a b)
    (if (null? a) b (cons (car a) (edit/append (cdr a) b))))
  "Append two lists.")

(define edit/unique-strs
  (lambda (xs)
    (list/foldl
      (lambda (acc x)
        (if (edit/member-str? (as-str x) acc)
            acc
            (cons (as-str x) acc)))
      (list)
      xs))
  "Unique string list (order reversed ok).")

```

```
(define edit/member-str?
  (lambda (s xs)
    (if (null? xs)
        #f
        (if (string-eq s (as-str (car xs)))
            #t
            (edit/member-str? s (cdr xs)))))
  "String membership.")

;; --- takes ---

(define edit/takes
  (lambda (shot-id)
    (let ((s (as-str shot-id)))
      (edit/sort-rows
        (list/filter
          (lambda (row) (string-eq (edit/f row "shot") s))
          (rows 'edit-take 300))
        "n")))
  "Takes for shot, by take number.")

(define edit/take-get
  (lambda (id) (get 'edit-take (as-str id)))
  "Get take by id.")

(define edit/take-create
  (lambda (shot-id n backend model seed cost media-hash media-path
           state rating input-snapshot source-assets)
    (create 'edit-take
      (dict-set* "{}"
        (list
          "shot" (as-str shot-id)
          "n" (as-str n)
          "backend" (as-str backend)
          "model" (as-str model)
          "seed" (as-str seed)
          "cost_cents" (as-str cost)
          "media_hash" (as-str media-hash)
          "media_path" (as-str media-path)
          "state" (as-str state)
          "rating" (as-str rating)
          "input_snapshot" (as-str input-snapshot)
          "source_assets" (as-str source-assets)))))
  "Create take row; returns id.")

(define edit/take-set-state
  (lambda (take-id state)
    (update-field 'edit-take (as-str take-id) "state" (as-str state)))
  "Update take state.")

(define edit/choose
  (lambda (shot-id take-n)
    (let ((want (as-str take-n)))
      (list/map
        (lambda (row)
          (if (string-eq (edit/f row "n") want)
              (edit/take-set-state (edit/id row) "done")
              (if (string-eq (edit/f row "state") "done")
                  (edit/take-set-state (edit/id row) "rejected")
                  ())))
        (edit/takes shot-id))))
  "Mark take n chosen (done); demote prior done takes to rejected.")

(define edit/chosen-take
  (lambda (shot-id)
    (list/foldl
      (lambda (acc row)
        (if (null? acc)
            (if (string-eq (edit/f row "state") "done") row acc)
            acc))
      ()
      (edit/takes shot-id)))
  "First take with state=done for shot.")
```

```
510
511 (define edit/promote-take
512   (lambda (take-id kind slug)
513     (let ((trow (edit/take-get take-id)))
514       (if (null? trow)
515           ""
516           (let ((shot (edit/shot-get (edit/f trow "shot"))))
517             (let ((proj
518                     (if (null? shot) "" (edit/f shot "project"))))
519               (edit/asset-upsert
520                 proj
521                 slug
522                 kind
523                 (string-append "promoted from take " (as-str take-id))
524                 ""
525                 ""
526                 ""
527                 (edit/f trow "media_hash")
528                 (edit/f trow "media_path")
529                 "[]"
530                 ""
531                 ""
532                 ""
533                 "{}"))))))
534   "Mint asset from landed take media; returns asset id.")
535
536 ;; --- expand (pure preview of gen bundle) ---
537
538 (define edit/expand
539   (lambda (shot-id)
540     (let ((shot (edit/shot-get shot-id)))
541       (if (null? shot)
542           "{}"
543           (let ((casts (edit/shot-casts shot-id))
544                 (refs (edit/shot-refs shot-id)))
545             (let ((appearances
546                     (edit/join-lines
547                       (list/map edit/cast-appearance-line casts)))
548                   (ref-bits
549                     (edit/join-lines
550                       (list/map edit/ref-appearance-line refs)))
551                   (hashes
552                     (edit/join-lines
553                       (edit/unique-strs
554                         (edit/append
555                           (list/map edit/cast-hashes casts)
556                           (list/map edit/ref-hashes-line refs)))))
557                   (asset-ids
558                     (edit/join-lines
559                       (edit/unique-strs
560                         (edit/append
561                           (list/map
562                             (lambda (r) (edit/f r "asset"))
563                             casts)
564                           (list/map
565                             (lambda (r) (edit/f r "asset"))
566                             refs))))))
567               (dict-set* "{}"
568                 (list
569                   "shot" (as-str shot-id)
570                   "prompt" (edit/f shot "prompt")
571                   "cast_appearance" appearances
572                   "ref_appearance" ref-bits
573                   "ref_hashes" hashes
574                   "source_assets" asset-ids
575                   "expanded_prompt"
576                   (edit/join-lines
577                     (list
578                       (edit/f shot "prompt")
579                       appearances
580                       ref-bits)))))))))
581   "Pure expansion of cast/refs into a gen input bundle (JSON string).")
582
```

```
(define edit/cast-appearance-line
  (lambda (crow)
    (let ((a (edit/asset-get (edit/f crow "asset")))
          (w-id (edit/f crow "wardrobe")))
      (let ((base
              (if (null? a)
                  ""
                  (string-append
                    (edit/f a "slug") ": "
                    (edit/f a "appearance"))))
            (ward
              (if (edit/blank? w-id)
                  ""
                  (let ((w (edit/asset-get w-id)))
                    (if (null? w)
                        ""
                        (string-append
                          " wardrobe "
                          (edit/f w "slug") ": "
                          (edit/f w "appearance")))))))
        (edit/join-lines
          (list/filter
            (lambda (s) (not (edit/blank? s)))
            (list base ward))))))
  "Appearance line for one cast binding.")

(define edit/ref-appearance-line
  (lambda (rrow)
    (let ((a (edit/asset-get (edit/f rrow "asset"))))
      (if (null? a)
          ""
          (string-append
            (edit/f rrow "purpose") " "
            (edit/f a "slug") ": "
            (edit/f a "appearance")))))
  "Appearance line for one ref binding.")

(define edit/cast-hashes
  (lambda (crow)
    (let ((a (edit/asset-get (edit/f crow "asset")))
          (w-id (edit/f crow "wardrobe")))
      (edit/join-lines
        (list/filter
          (lambda (s) (not (edit/blank? s)))
          (list
            (if (null? a) "" (edit/f a "primary_hash"))
            (if (null? a) "" (edit/f a "ref_hashes"))
            (if (edit/blank? w-id)
                ""
                (let ((w (edit/asset-get w-id)))
                  (if (null? w) "" (edit/f w "primary_hash")))))))))
  "Hash material from cast binding.")

(define edit/ref-hashes-line
  (lambda (rrow)
    (let ((a (edit/asset-get (edit/f rrow "asset"))))
      (if (null? a)
          ""
          (edit/join-lines
            (list
              (edit/f a "primary_hash")
              (edit/f a "ref_hashes"))))))
  "Hash material from ref binding.")

;; --- EDL ---

(define edit/edl
  (lambda (project cut)
    (let ((p (as-str project))
          (c (as-str cut)))
      (edit/sort-by-ord
        (list/filter
          (lambda (row)
```

```
            (and (string-eq (edit/f row "project") p)
                 (string-eq (edit/f row "cut") c)))
          (rows 'edit-edl-entry 500)))))
  "EDL entries for project+cut.")

(define edit/edl-add
  (lambda (project cut ord take-id tc-in dur transition audio)
    (create 'edit-edl-entry
      (dict-set* "{}"
        (list
          "project" (as-str project)
          "cut" (as-str cut)
          "ord" (as-str ord)
          "take" (as-str take-id)
          "tc_in" (as-str tc-in)
          "dur" (as-str dur)
          "transition" (as-str transition)
          "audio" (as-str audio)))))
  "Append EDL entry.")

(define edit/edl-rebuild-from-shots
  (lambda (project cut)
    (let ((shots (edit/shots project)))
      (seq
        (list/map
          (lambda (e) (delete 'edit-edl-entry (edit/id e)))
          (edit/edl project cut))
        (list/map
          (lambda (shot)
            (let ((ch (edit/chosen-take (edit/id shot))))
              (if (null? ch)
                  ()
                  (edit/edl-add
                    project
                    cut
                    (edit/f shot "ord")
                    (edit/id ch)
                    "0"
                    (edit/f shot "dur_target")
                    "cut"
                    (edit/line-audio-for-shot (edit/id shot))))))
          shots))))
  "Rebuild cut EDL from chosen takes on each shot.")

(define edit/edl-canonical
  (lambda (project cut)
    (edit/join-lines
      (list/map
        (lambda (e)
          (string-append
            (edit/f e "ord") "|"
            (edit/f e "take") "|"
            (edit/f e "tc_in") "|"
            (edit/f e "dur") "|"
            (edit/f e "transition") "|"
            (edit/f e "audio")))
        (edit/edl project cut))))
  "Canonical EDL print for hashing.")

(define edit/edl-hash
  (lambda (project cut)
    (sha1 (edit/edl-canonical project cut)))
  "Cache key for picture EDL.")

;; --- lines (dialogue) ---

(define edit/lines
  (lambda (shot-id)
    (let ((s (as-str shot-id)))
      (edit/sort-by-ord
        (list/filter
          (lambda (row) (string-eq (edit/f row "shot") s))
          (rows 'edit-line 300))))))
```

```
  "Dialogue lines for a shot.")

(define edit/lines-project
  (lambda (project)
    (let ((p (as-str project)))
      (edit/sort-by-ord
        (list/filter
          (lambda (row) (string-eq (edit/f row "project") p))
          (rows 'edit-line 500)))))
  "All lines for a project.")

(define edit/line-upsert
  (lambda (project shot ord character text notes)
    (let ((existing
            (list/foldl
              (lambda (acc row)
                (if (null? acc)
                    (if (and (string-eq (edit/f row "shot") (as-str shot))
                             (string-eq (edit/f row "ord") (as-str ord)))
                        row
                        acc)
                    acc))
              ()
              (edit/lines shot))))
      (if (null? existing)
          (create 'edit-line
            (dict-set* "{}"
              (list
                "project" (as-str project)
                "shot" (as-str shot)
                "ord" (as-str ord)
                "character" (as-str character)
                "text" (as-str text)
                "notes" (as-str notes)
                "take" "")))
          (seq
            (update 'edit-line (edit/id existing)
              (dict-set* (edit/fields existing)
                (list
                  "character" (as-str character)
                  "text" (as-str text)
                  "notes" (as-str notes))))
            (edit/id existing))))))
  "Create or update line by shot+ord.")

(define edit/line-choose
  (lambda (line-id take-id)
    (update-field 'edit-line (as-str line-id) "take" (as-str take-id)))
  "Set chosen audio take for a line.")

(define edit/line-audio-for-shot
  (lambda (shot-id)
    (list/foldl
      (lambda (acc row)
        (if (edit/blank? acc)
            (edit/f row "take")
            acc))
      ""
      (edit/lines shot-id)))
  "First non-empty chosen line take for EDL audio pointer.")

;; --- cues (beds + hits) ---

(define edit/cues
  (lambda (project cut)
    (let ((p (as-str project))
          (c (as-str cut)))
      (edit/sort-by-ord
        (list/filter
          (lambda (row)
            (and (string-eq (edit/f row "project") p)
                 (string-eq (edit/f row "cut") c)))
          (rows 'edit-cue 500))))))
```

```
  "Timeline cues for project+cut.")

(define edit/cue-add
  (lambda (project cut ord mode asset tc-start dur gain-db loop)
    (create 'edit-cue
      (dict-set* "{}"
        (list
          "project" (as-str project)
          "cut" (as-str cut)
          "ord" (as-str ord)
          "mode" (as-str mode)
          "asset" (as-str asset)
          "tc_start" (as-str tc-start)
          "dur" (as-str dur)
          "gain_db" (as-str gain-db)
          "loop" (as-str loop)))))
  "Add bed or hit cue.")

(define edit/cue-canonical
  (lambda (project cut)
    (let ((cue-part
            (edit/join-lines
              (list/map
                (lambda (c)
                  (string-append
                    (edit/f c "ord") "|"
                    (edit/f c "mode") "|"
                    (edit/f c "asset") "|"
                    (edit/f c "tc_start") "|"
                    (edit/f c "dur") "|"
                    (edit/f c "gain_db") "|"
                    (edit/f c "loop")))
                (edit/cues project cut))))
          (line-part
            (edit/join-lines
              (list/map
                (lambda (ln)
                  (string-append
                    (edit/f ln "shot") "|"
                    (edit/f ln "ord") "|"
                    (edit/f ln "take")))
                (edit/lines-project project)))))
      (string-append cue-part "\n--\n" line-part)))
  "Canonical audio graph print for hashing.")

(define edit/cue-hash
  (lambda (project cut profile)
    (sha1
      (string-append
        (edit/cue-canonical project cut)
        "\nprofile:"
        (as-str profile))))
  "Cache key for mix-job.")

;; --- exports (v1: strings; save to disk by hand or via UI) ---

(define edit/export-mlt
  (lambda (project cut)
    (let ((proj (edit/project-by-slug project))
          (entries (edit/edl project cut)))
      (let ((fps
              (if (null? proj) "24" (edit/f proj "fps")))
            (title
              (if (null? proj) project (edit/f proj "title"))))
        (string-append
          "<?xml version=\"1.0\" encoding=\"utf-8\"?>\n"
          "<!-- pricklypear edit/export-mlt project="
          (edit/escape-xml project)
          " cut="
          (edit/escape-xml cut)
          " edl_hash="
          (edit/edl-hash project cut)
          " -->\n"
```

```
          "<mlt LC_NUMERIC=\"C\" version=\"7.0.0\" "
          "title=\""
          (edit/escape-xml title)
          "\" producer=\"main_bin\">\n"
          "  <profile frame_rate_num=\""
          (edit/escape-xml fps)
          "\" frame_rate_den=\"1\" width=\"1920\" height=\"1080\" "
          "display_aspect_num=\"16\" display_aspect_den=\"9\" "
          "sample_aspect_num=\"1\" sample_aspect_den=\"1\" "
          "colorspace=\"709\" progressive=\"1\"/>\n"
          "  <playlist id=\"main_bin\">\n"
          (edit/join-lines
            (list/map
              (lambda (e) (edit/mlt-entry-xml e))
              entries))
          "\n  </playlist>\n"
          "  <tractor id=\"tractor0\" title=\""
          (edit/escape-xml title)
          "\">\n"
          "    <track producer=\"main_bin\"/>\n"
          "  </tractor>\n"
          "</mlt>\n"))))
  "MLT XML string for Kdenlive/melt.")

(define edit/mlt-entry-xml
  (lambda (e)
    (let ((t (edit/take-get (edit/f e "take"))))
      (let ((path
              (if (null? t)
                  (string-append "missing-take-" (edit/f e "take"))
                  (let ((p (edit/f t "media_path")))
                    (if (edit/blank? p)
                        (string-append "hash:" (edit/f t "media_hash"))
                        p))))
            (in (edit/f e "tc_in"))
            (dur (edit/f e "dur")))
        (string-append
          "    <entry producer=\""
          (edit/escape-xml path)
          "\" in=\""
          (edit/escape-xml in)
          "\" out=\""
          (edit/escape-xml
            (as-str (+ (edit/as-int in) (edit/as-int dur) -1)))
          "\"/>"))))
  "One MLT playlist entry.")

(define edit/export-blender
  (lambda (project cut)
    (let ((proj (edit/project-by-slug project))
          (entries (edit/edl project cut)))
      (let ((fps
              (if (null? proj) "24" (edit/f proj "fps"))))
        (string-append
          "# pricklypear edit/export-blender\n"
          "# project=" project " cut=" cut
          " edl_hash=" (edit/edl-hash project cut) "\n"
          "# Load in Blender Text Editor and Run Script, or:\n"
          "#   blender -b -P this_script.py\n"
          "import bpy\n"
          "scene = bpy.context.scene\n"
          "scene.render.fps = " (as-str (edit/as-int fps)) "\n"
          "scene.sequence_editor_clear()\n"
          "se = scene.sequence_editor_create()\n"
          "frame = 1\n"
          (edit/join-lines
            (list/map
              (lambda (e) (edit/blender-strip-py e))
              entries))
          "\nprint('pricklypear EDL loaded')\n"))))
  "Blender VSE build script string.")

(define edit/blender-strip-py
```

```
  (lambda (e)
    (let ((t (edit/take-get (edit/f e "take"))))
      (let ((path
              (if (null? t)
                  "/missing.mp4"
                  (let ((p (edit/f t "media_path")))
                    (if (edit/blank? p) "/missing.mp4" p))))
            (dur (edit/as-int (edit/f e "dur")))
            (tc (edit/as-int (edit/f e "tc_in"))))
        (string-append
          "se.sequences.new_movie("
          "name='t" (edit/f e "ord") "', "
          "filepath='" path "', "
          "channel=1, frame_start=frame)\n"
          "strip = se.sequences_all['t" (edit/f e "ord") "']\n"
          "strip.frame_offset_start = " (as-str tc) "\n"
          "strip.frame_final_duration = " (as-str dur) "\n"
          "frame += " (as-str dur)))))
  "Python lines for one VSE strip.")

(define edit/export-cue-sheet
  (lambda (project cut)
    (string-append
      "# pricklypear cue sheet project=" project " cut=" cut "\n"
      "# cue_hash=" (edit/cue-hash project cut "dailies") "\n"
      "ord,mode,asset,tc_start,dur,gain_db,loop,path\n"
      (edit/join-lines
        (list/map
          (lambda (c)
            (let ((a (edit/asset-get (edit/f c "asset"))))
              (string-append
                (edit/f c "ord") ","
                (edit/f c "mode") ","
                (edit/f c "asset") ","
                (edit/f c "tc_start") ","
                (edit/f c "dur") ","
                (edit/f c "gain_db") ","
                (edit/f c "loop") ","
                (if (null? a) "" (edit/f a "primary_path")))))
          (edit/cues project cut)))
      "\n# lines\n"
      "shot,ord,character,text,take,path\n"
      (edit/join-lines
        (list/map
          (lambda (ln)
            (let ((t (if (edit/blank? (edit/f ln "take"))
                         ()
                         (edit/take-get (edit/f ln "take")))))
              (string-append
                (edit/f ln "shot") ","
                (edit/f ln "ord") ","
                (edit/f ln "character") ","
                (edit/escape-xml (edit/f ln "text")) ","
                (edit/f ln "take") ","
                (if (null? t) "" (edit/f t "media_path")))))
          (edit/lines-project project)))
      "\n"))
  "CSV-ish cue sheet for humans and agents.")

(define edit/export-ffmpeg
  (lambda (project cut)
    (let ((entries (edit/edl project cut))
          (cues (edit/cues project cut)))
      (string-append
        "#!/bin/sh\n"
        "# pricklypear edit/export-ffmpeg -- dailies-style bounce sketch\n"
        "# project=" project " cut=" cut "\n"
        "# edl_hash=" (edit/edl-hash project cut) "\n"
        "# cue_hash=" (edit/cue-hash project cut "dailies") "\n"
        "# Review paths, then run under ksh/sh. Adjust -filter_complex as
    needed.\n"
        "set -e\n"
        "OUT=\"dailies-" project "-" cut ".mp4\"\n"
```

```
        (edit/ffmpeg-inputs entries)
        (edit/ffmpeg-filter entries)
        "ffmpeg -y \\\n"
        (edit/ffmpeg-input-flags entries)
        "  -filter_complex \"$FC\" \\\n"
        "  -map \"[vout]\" -map \"[aout]\" \\\n"
        "  -c:v libx264 -preset veryfast -crf 23 \\\n"
        "  -c:a aac -b:a 160k \"$OUT\"\n"
        "echo wrote \"$OUT\"\n"
        "# cues (apply in a second pass or DAW):\n"
        (edit/join-lines
          (list/map
            (lambda (c)
              (string-append
                "# cue " (edit/f c "mode")
                " asset=" (edit/f c "asset")
                " tc=" (edit/f c "tc_start")
                " gain=" (edit/f c "gain_db") "dB"))
            cues))
        "\n")))
  "Shell script sketch that concatenates chosen picture takes.")

(define edit/ffmpeg-inputs
  (lambda (entries)
    (edit/join-lines
      (list/map
        (lambda (e)
          (let ((t (edit/take-get (edit/f e "take"))))
            (let ((path
                    (if (null? t)
                        "/missing.mp4"
                        (let ((p (edit/f t "media_path")))
                          (if (edit/blank? p) "/missing.mp4" p)))))
              (string-append
                "# input ord=" (edit/f e "ord") " path=" path))))
        entries)))
  "Comments listing inputs.")

(define edit/ffmpeg-input-flags
  (lambda (entries)
    (edit/join-lines
      (list/map
        (lambda (e)
          (let ((t (edit/take-get (edit/f e "take"))))
            (let ((path
                    (if (null? t)
                        "/missing.mp4"
                        (let ((p (edit/f t "media_path")))
                          (if (edit/blank? p) "/missing.mp4" p)))))
              (string-append "  -i \"" path "\" \\"))))
        entries)))
  "ffmpeg -i flags.")

(define edit/ffmpeg-filter
  (lambda (entries)
    (if (null? entries)
        "FC=\"nullsrc=s=1920x1080:d=1[vout];anullsrc=r=48000:cl=stereo[aout]\"\n"
        (let ((n (edit/count entries)))
          (string-append
            "FC=\""
            (edit/ffmpeg-scale-chain entries 0)
            (edit/ffmpeg-concat n)
            "\"\n")))))
  "Build filter_complex assignment.")

(define edit/count
  (lambda (xs)
    (if (null? xs) 0 (+ 1 (edit/count (cdr xs))))))
  "List length.")

(define edit/ffmpeg-scale-chain
  (lambda (entries i)
```

```
    (if (null? entries)
        ""
        (string-append
          "[" (as-str i)
     ":v]scale=1920:1080:force_original_aspect_ratio=decrease,"
          "pad=1920:1080:(ow-iw)/2:(oh-ih)/2,setsar=1,fps=24[v" (as-str i)
     "];"
          "[" (as-str i)
     ":a]aformat=sample_fmts=fltp:sample_rates=48000:channel_layouts=stereo,"
          "volume=1.0[a" (as-str i) "];"
          (edit/ffmpeg-scale-chain (cdr entries) (+ i 1)))))
  "Per-input scale/aformat labels.")

(define edit/ffmpeg-concat
  (lambda (n)
    (string-append
      (edit/ffmpeg-concat-labels 0 n)
      "concat=n=" (as-str n) ":v=1:a=1[vout][aout]"))
  "concat filter tail.")

(define edit/ffmpeg-concat-labels
  (lambda (i n)
    (if (>= i n)
        ""
        (string-append
          "[v" (as-str i) "][a" (as-str i) "]"
          (edit/ffmpeg-concat-labels (+ i 1) n))))
  "v/a label stream for concat.")

;; --- spend ---

(define edit/spend-cents
  (lambda (project)
    (list/foldl
      (lambda (acc shot)
        (+ acc
           (list/foldl
             (lambda (a t)
               (+ a (edit/as-int (edit/f t "cost_cents"))))
             0
             (edit/takes (edit/id shot)))))
      0
      (edit/shots project)))
  "Sum take cost_cents for project.")

;; --- UI ---

(define edit/home-html
  (lambda ()
    (html/div
      (html/attr "class" "stack")
      (html/join
        (list
          (html/h1 "Studio")
          (html/p*
            (html/attr "class" "muted")
            "Film edit: library, shots, takes, lines, cues, EDL export.
    Film-as-data.")
          (html/div
            (html/attr "class" "card")
            (html/join
              (list
                (html/h2 "Projects")
                (edit/projects-table)
                (edit/project-create-form))))
          (html/div
            (html/attr "class" "card")
            (html/join
              (list
                (html/h2 "Studio assets")
                (edit/assets-table ""))))))))
  "Home list HTML.")

```

```
(define edit/projects-table
  (lambda ()
    (let ((ps (edit/projects)))
      (if (null? ps)
          (html/p* (html/attr "class" "muted") "No projects yet.")
          (edit/table
            (html/join
              (list
                (edit/thead
                  (edit/tr
                    (html/join
                      (list
                        (edit/th "slug")
                        (edit/th "title")
                        (edit/th "fps")
                        (edit/th "budget?")
                        (edit/th "")))))
                (edit/tbody
                  (html/join
                    (list/map edit/project-row ps)))))))))
  "Projects table.")

(define edit/project-row
  (lambda (row)
    (edit/tr
      (html/join
        (list
          (edit/td (html/code (edit/f row "slug")))
          (edit/td (html/text (edit/f row "title")))
          (edit/td (html/text (edit/f row "fps")))
          (edit/td (html/text (edit/f row "budget_cents")))
          (edit/td
            (html/a
              (string-append "/studio/project?slug=" (edit/f row "slug"))
              "open")))))))
  "One project table row.")

(define edit/project-create-form
  (lambda ()
    (html/form
      "POST"
      "/studio"
      (html/attrs
        (list
          (html/attr "class" "stack")
          (html/attr "hx-post" "/studio")
          (html/attr "hx-target" "body")
          (html/attr "hx-swap" "outerHTML")))
      (html/join
        (list
          (html/hidden "action" "create-project")
          (html/label "" "slug")
          (edit/input "slug" "")
          (html/label "" "title")
          (edit/input "title" "")
          (html/label "" "fps")
          (edit/input "fps" "24")
          (html/label "" "resolution")
          (edit/input "resolution" "1920x1080")
          (html/label "" "budget_cents")
          (edit/input "budget_cents" "0")
          (html/button (html/attr "type" "submit") "Create project"))))))
  "Create project form.")

(define edit/asset-thumb
  (lambda (row)
    (let ((mid (edit/f row "media_id"))
          (path (edit/f row "primary_path"))
          (slug (edit/f row "slug")))
      (let ((src
              (if (not (edit/blank? mid))
                  (string-append "/media/" mid)
                  (if (and (not (edit/blank? path))
```

```
                          (string-eq (substring path 0 1) "/"))
                     path
                     "")))))
        (if (edit/blank? src)
            (html/span (html/attr "class" "muted") "--")
            (html/a src
              (html/void
                "img"
                (html/attrs
                  (list
                    (html/attr "src" src)
                    (html/attr "alt" slug)
                    (html/attr "class" "edit-asset-thumb")
                    (html/attr "loading" "lazy")
                    (html/attr
                      "style"
                      "width:72px;height:48px;object-fit:cover;border:1px solid #000;vertical-align:middle;")))))))))
  "Thumb linked to full media when media_id or /media path set.")

(define edit/assets-table
  (lambda (project)
    (let ((xs (edit/assets-for project "")))
      (if (null? xs)
          (html/p* (html/attr "class" "muted") "No assets.")
          (edit/table
            (html/join
              (list
                (edit/thead
                  (edit/tr
                    (html/join
                      (list
                        (edit/th "")
                        (edit/th "slug")
                        (edit/th "kind")
                        (edit/th "project")
                        (edit/th "appearance")))))
                (edit/tbody
                  (html/join
                    (list/map edit/asset-row xs)))))))))
  "Assets table with optional media thumbs.")

(define edit/asset-row
  (lambda (row)
    (edit/tr
      (html/join
        (list
          (edit/td (edit/asset-thumb row))
          (edit/td (html/code (edit/f row "slug")))
          (edit/td (html/text (edit/f row "kind")))
          (edit/td
            (html/text
              (if (edit/blank? (edit/f row "project"))
                  "(studio)"
                  (edit/f row "project"))))
          (edit/td
            (html/text
              (let ((a (edit/f row "appearance")))
                (if (< (string-length a) 80)
                    a
                    (string-append (substring a 0 80) "?")))))))))
  "One asset row with thumb.")

(define edit/project-page-html
  (lambda (slug)
    (let ((proj (edit/project-by-slug slug)))
      (if (null? proj)
          (html/div ""
            (html/join
              (list
                (html/h1 "Missing project")
                (html/p* "" (html/a "/studio" "Back")))))
          (let ((p (edit/f proj "slug")))
```

```
            (html/div
              (html/attr "class" "stack")
              (html/join
                (list
                  (html/p* "" (html/a "/studio" "? Studio"))
                  (html/h1 (html/text (edit/f proj "title")))
                  (html/p*
                    (html/attr "class" "muted")
                    (string-append
                      "slug " p
                      " ? fps " (edit/f proj "fps")
                      " ? " (edit/f proj "resolution")
                      " ? spend "
                      (as-str (edit/spend-cents p))
                      "? / budget "
                      (edit/f proj "budget_cents")
                      "? ? edl "
                      (edit/edl-hash p "main")))
                  (html/div
                    (html/attr "class" "card")
                    (html/join
                      (list
                        (html/h2 "Cast & library")
                        (edit/assets-table p)
                        (edit/asset-create-form p))))
                  (html/div
                    (html/attr "class" "card")
                    (html/join
                      (list
                        (html/h2 "Shots")
                        (edit/shots-table p)
                        (edit/shot-create-form p))))
                  (html/div
                    (html/attr "class" "card")
                    (html/join
                      (list
                        (html/h2 "Cut tools")
                        (html/p*
                          (html/attr "class" "muted")
                          "Rebuild EDL from chosen takes, then export.")
                        (edit/cut-actions p)
                        (html/p*
                          ""
                          (html/a
                            (string-append
                              "/studio/export?slug=" p "&cut=main&kind=mlt")
                            "MLT XML"))
                        (html/p*
                          ""
                          (html/a
                            (string-append
                              "/studio/export?slug=" p
    "&cut=main&kind=blender")
                            "Blender VSE script"))
                        (html/p*
                          ""
                          (html/a
                            (string-append
                              "/studio/export?slug=" p
    "&cut=main&kind=cues")
                            "Cue sheet"))
                        (html/p*
                          ""
                          (html/a
                            (string-append
                              "/studio/export?slug=" p
    "&cut=main&kind=ffmpeg")
                            "ffmpeg dailies script"))))))))))
  "Project detail HTML.")

(define edit/asset-create-form
  (lambda (project)
    (html/form
```

```
      "POST"
      (string-append "/studio/project?slug=" project)
      (html/attrs (list (html/attr "class" "stack")))
      (html/join
        (list
          (html/hidden "action" "create-asset")
          (html/hidden "project" project)
          (html/label "" "slug")
          (edit/input "slug" "")
          (html/label "" "kind
    (character|location|prop|style|voice|sfx|score|?)")
          (edit/input "kind" "character")
          (html/label "" "appearance (locked prompt fragment)")
          (edit/input "appearance" "")
          (html/label "" "summary")
          (edit/input "summary" "")
          (html/button (html/attr "type" "submit") "Add asset")))))
  "Quick asset form.")

(define edit/shots-table
  (lambda (project)
    (let ((xs (edit/shots project)))
      (if (null? xs)
          (html/p* (html/attr "class" "muted") "No shots.")
          (edit/table
            (html/join
              (list
                (edit/thead
                  (edit/tr
                    (html/join
                      (list
                        (edit/th "#")
                        (edit/th "slug")
                        (edit/th "summary")
                        (edit/th "cast")
                        (edit/th "take")
                        (edit/th ""))))))
                (edit/tbody
                  (html/join
                    (list/map
                      (lambda (s) (edit/shot-row project s))
                      xs)))))))))
  "Shots table with cast + chosen take.")

(define edit/shot-row
  (lambda (project shot)
    (let ((sid (edit/id shot))
          (ch (edit/chosen-take (edit/id shot)))
          (casts (edit/shot-casts (edit/id shot))))
      (edit/tr
        (html/join
          (list
            (edit/td (html/text (edit/f shot "ord")))
            (edit/td (html/code (edit/f shot "slug")))
            (edit/td (html/text (edit/f shot "summary")))
            (edit/td
              (html/text
                (edit/join-lines
                  (list/map
                    (lambda (c)
                      (let ((a (edit/asset-get (edit/f c "asset"))))
                        (if (null? a)
                            (edit/f c "asset")
                            (edit/f a "slug"))))
                    casts))))
            (edit/td
              (html/text
                (if (null? ch)
                    "--"
                    (string-append
                      "#" (edit/f ch "n")
                      " "
                      (edit/f ch "state")))))
```

```
            (edit/td
              (html/a
                (string-append
                  "/studio/shot?id=" sid "&project=" project)
                "open")))))))
  "One shot row.")

(define edit/shot-create-form
  (lambda (project)
    (html/form
      "POST"
      (string-append "/studio/project?slug=" project)
      (html/attrs (list (html/attr "class" "stack")))
      (html/join
        (list
          (html/hidden "action" "create-shot")
          (html/hidden "project" project)
          (html/label "" "ord")
          (edit/input "ord" "1")
          (html/label "" "slug")
          (edit/input "slug" "")
          (html/label "" "summary")
          (edit/input "summary" "")
          (html/label "" "prompt (action only)")
          (edit/input "prompt" "")
          (html/label "" "dur_target (frames)")
          (edit/input "dur_target" "48")
          (html/button (html/attr "type" "submit") "Add shot")))))
  "Create shot form.")

(define edit/cut-actions
  (lambda (project)
    (html/form
      "POST"
      (string-append "/studio/project?slug=" project)
      ""
      (html/join
        (list
          (html/hidden "action" "rebuild-edl")
          (html/hidden "project" project)
          (html/hidden "cut" "main")
          (html/button
            (html/attr "type" "submit")
            "Rebuild EDL from chosen takes")))))
  "Rebuild EDL form.")

(define edit/shot-page-html
  (lambda (shot-id project)
    (let ((shot (edit/shot-get shot-id)))
      (if (null? shot)
          (html/div "" (html/h1 "Missing shot"))
          (html/div
            (html/attr "class" "stack")
            (html/join
              (list
                (html/p*
                  ""
                  (html/a
                    (string-append "/studio/project?slug=" project)
                    "? project"))
                (html/h1
                  (html/text
                    (string-append
                      (edit/f shot "ord") ". "
                      (edit/f shot "slug"))))
                (html/p (html/text (edit/f shot "summary")))
                (html/p*
                  (html/attr "class" "muted")
                  (edit/f shot "prompt"))
                (html/div
                  (html/attr "class" "card")
                  (html/join
                    (list
```

```
                      (html/h2 "Expand preview")
                      (html/pre
                        (html/text (edit/expand shot-id))))))
                (html/div
                  (html/attr "class" "card")
                  (html/join
                    (list
                      (html/h2 "Cast")
                      (edit/cast-list shot-id)
                      (edit/cast-form shot-id project))))
                (html/div
                  (html/attr "class" "card")
                  (html/join
                    (list
                      (html/h2 "Refs")
                      (edit/ref-list shot-id)
                      (edit/ref-form shot-id project))))
                (html/div
                  (html/attr "class" "card")
                  (html/join
                    (list
                      (html/h2 "Takes")
                      (edit/takes-table shot-id project)
                      (edit/take-form shot-id project))))
                (html/div
                  (html/attr "class" "card")
                  (html/join
                    (list
                      (html/h2 "Lines")
                      (edit/lines-table shot-id)
                      (edit/line-form shot-id project)))))))))
  "Shot detail HTML.")

(define edit/cast-list
  (lambda (shot-id)
    (let ((xs (edit/shot-casts shot-id)))
      (if (null? xs)
          (html/p* (html/attr "class" "muted") "No cast.")
          (html/ul ""
            (html/join
              (list/map
                (lambda (c)
                  (let ((a (edit/asset-get (edit/f c "asset"))))
                    (html/li ""
                      (html/text
                        (string-append
                          (if (null? a) (edit/f c "asset") (edit/f a
    "slug"))
                          " ("
                          (edit/f c "role")
                          ")")))))
                xs))))))
  "Cast ul.")

(define edit/cast-form
  (lambda (shot-id project)
    (html/form
      "POST"
      (string-append "/studio/shot?id=" shot-id "&project=" project)
      (html/attrs (list (html/attr "class" "stack")))
      (html/join
        (list
          (html/hidden "action" "cast")
          (html/label "" "asset id")
          (edit/input "asset" "")
          (html/label "" "role")
          (edit/input "role" "hero")
          (html/label "" "wardrobe asset id (optional)")
          (edit/input "wardrobe" "")
          (html/button (html/attr "type" "submit") "Bind cast"))))))
  "Cast bind form.")

(define edit/ref-list
```

```
  (lambda (shot-id)
    (let ((xs (edit/shot-refs shot-id)))
      (if (null? xs)
          (html/p* (html/attr "class" "muted") "No refs.")
          (html/ul ""
            (html/join
              (list/map
                (lambda (r)
                  (html/li ""
                    (html/text
                      (string-append
                        (edit/f r "purpose")
                        " ? "
                        (edit/f r "asset")))))
                xs))))))
  "Refs ul.")

(define edit/ref-form
  (lambda (shot-id project)
    (html/form
      "POST"
      (string-append "/studio/shot?id=" shot-id "&project=" project)
      (html/attrs (list (html/attr "class" "stack")))
      (html/join
        (list
          (html/hidden "action" "ref")
          (html/label "" "asset id")
          (edit/input "asset" "")
          (html/label "" "purpose")
          (edit/input "purpose" "style")
          (html/label "" "weight")
          (edit/input "weight" "")
          (html/button (html/attr "type" "submit") "Bind ref")))))
  "Ref bind form.")

(define edit/takes-table
  (lambda (shot-id project)
    (let ((xs (edit/takes shot-id)))
      (if (null? xs)
          (html/p* (html/attr "class" "muted") "No takes.")
          (edit/table
            (html/join
              (list
                (edit/thead
                  (edit/tr
                    (html/join
                      (list
                        (edit/th "n")
                        (edit/th "backend")
                        (edit/th "state")
                        (edit/th "cost?")
                        (edit/th "hash")
                        (edit/th ""))))))
                (edit/tbody
                  (html/join
                    (list/map
                      (lambda (t)
                        (edit/take-row shot-id project t))
                      xs)))))))))
  "Takes table.")

(define edit/take-row
  (lambda (shot-id project t)
    (edit/tr
      (html/join
        (list
          (edit/td (html/text (edit/f t "n")))
          (edit/td (html/text (edit/f t "backend")))
          (edit/td (html/text (edit/f t "state")))
          (edit/td (html/text (edit/f t "cost_cents")))
          (edit/td
            (html/code
              (let ((h (edit/f t "media_hash")))
```

```
                (if (< (string-length h) 12)
                    h
                    (substring h 0 12)))))
          (edit/td
            (html/form
              "POST"
              (string-append
                "/studio/shot?id=" shot-id "&project=" project)
              ""
              (html/join
                (list
                  (html/hidden "action" "choose")
                  (html/hidden "n" (edit/f t "n"))
                  (html/button
                    (html/attr "type" "submit")
                    "choose")))))))))
  "Take row with choose.")

(define edit/take-form
  (lambda (shot-id project)
    (html/form
      "POST"
      (string-append "/studio/shot?id=" shot-id "&project=" project)
      (html/attrs (list (html/attr "class" "stack")))
      (html/join
        (list
          (html/hidden "action" "add-take")
          (html/label "" "n")
          (edit/input "n" "1")
          (html/label "" "backend")
          (edit/input "backend" "camera")
          (html/label "" "media_path")
          (edit/input "media_path" "")
          (html/label "" "media_hash")
          (edit/input "media_hash" "")
          (html/label "" "cost_cents")
          (edit/input "cost_cents" "0")
          (html/button (html/attr "type" "submit") "Add take")))))
  "Manual take form (camera or imported).")

(define edit/lines-table
  (lambda (shot-id)
    (let ((xs (edit/lines shot-id)))
      (if (null? xs)
          (html/p* (html/attr "class" "muted") "No lines.")
          (edit/table
            (html/join
              (list
                (edit/thead
                  (edit/tr
                    (html/join
                      (list
                        (edit/th "#")
                        (edit/th "character")
                        (edit/th "text")
                        (edit/th "take")))))
                (edit/tbody
                  (html/join
                    (list/map
                      (lambda (ln)
                        (edit/tr
                          (html/join
                            (list
                              (edit/td (html/text (edit/f ln "ord")))
                              (edit/td (html/code (edit/f ln "character")))
                              (edit/td (html/text (edit/f ln "text")))
                              (edit/td
                                (html/text
                                  (if (edit/blank? (edit/f ln "take"))
                                      "--"
                                      (edit/f ln "take"))))))))
                      xs)))))))))
  "Lines table.")
```

```
(define edit/line-form
  (lambda (shot-id project)
    (html/form
      "POST"
      (string-append "/studio/shot?id=" shot-id "&project=" project)
      (html/attrs (list (html/attr "class" "stack")))
      (html/join
        (list
          (html/hidden "action" "add-line")
          (html/hidden "project" project)
          (html/label "" "ord")
          (edit/input "ord" "1")
          (html/label "" "character asset id")
          (edit/input "character" "")
          (html/label "" "text")
          (edit/input "text" "")
          (html/label "" "notes")
          (edit/input "notes" "")
          (html/button (html/attr "type" "submit") "Add line")))))
  "Line form.")

(define edit/export-page-html
  (lambda (slug cut kind)
    (let ((body
            (if (string-eq kind "mlt")
                (edit/export-mlt slug cut)
                (if (string-eq kind "blender")
                    (edit/export-blender slug cut)
                    (if (string-eq kind "cues")
                        (edit/export-cue-sheet slug cut)
                        (if (string-eq kind "ffmpeg")
                            (edit/export-ffmpeg slug cut)
                            "unknown kind")))))))
      (html/div
        (html/attr "class" "stack")
        (html/join
          (list
            (html/p*
              ""
              (html/a
                (string-append "/studio/project?slug=" slug)
                "? project"))
            (html/h1
              (html/text
                (string-append "Export " kind " ? " slug " / " cut)))
            (html/p*
              (html/attr "class" "muted")
              "v1 export is text -- save this file yourself. Automated
    write lands with spawn prims.")
            (html/pre (html/text body)))))))
  "Export viewer.")

;; --- routes ---

(define edit/home-page
  (lambda ()
    (if (string-eq (request-method) "POST")
        (seq
          (let ((action (as-str (request-param "action"))))
            (if (string-eq action "create-project")
                (edit/project-upsert
                  (request-param "slug")
                  (request-param "title")
                  (request-param "fps")
                  (request-param "resolution")
                  (request-param "budget_cents")
                  "")
                ()))
          (redirect "/studio"))
        (html/page (list (edit/home-html)))))
  "GET/POST /studio")

```

```
(define edit/project-page
  (lambda ()
    (let ((slug (as-str (request-param "slug"))))
      (if (edit/blank? slug)
          (redirect "/studio")
          (if (string-eq (request-method) "POST")
              (seq
                (let ((action (as-str (request-param "action"))))
                  (cond
                    ((string-eq action "create-asset")
                     (edit/asset-upsert
                       (request-param "project")
                       (request-param "slug")
                       (request-param "kind")
                       (request-param "summary")
                       ""
                       (request-param "appearance")
                       ""
                       ""
                       ""
                       "[]"
                       ""
                       ""
                       ""
                       "{}"))
                    ((string-eq action "create-shot")
                     (edit/shot-create
                       (request-param "project")
                       (request-param "ord")
                       (request-param "slug")
                       (request-param "summary")
                       (request-param "prompt")
                       (request-param "dur_target")))
                    ((string-eq action "rebuild-edl")
                     (edit/edl-rebuild-from-shots
                       (request-param "project")
                       (request-param "cut")))
                    (else ())))
                (redirect (string-append "/studio/project?slug=" slug)))
              (html/page (list (edit/project-page-html slug)))))))
  "GET/POST /studio/project")

(define edit/shot-page
  (lambda ()
    (let ((id (as-str (request-param "id")))
          (project (as-str (request-param "project"))))
      (if (edit/blank? id)
          (redirect "/studio")
          (if (string-eq (request-method) "POST")
              (seq
                (let ((action (as-str (request-param "action"))))
                  (cond
                    ((string-eq action "cast")
                     (edit/shot-cast
                       id
                       (request-param "asset")
                       (request-param "role")
                       (request-param "wardrobe")))
                    ((string-eq action "ref")
                     (edit/shot-ref
                       id
                       (request-param "asset")
                       (request-param "purpose")
                       (request-param "weight")))
                    ((string-eq action "add-take")
                     (edit/take-create
                       id
                       (request-param "n")
                       (request-param "backend")
                       ""
                       ""
                       (request-param "cost_cents")
                       (request-param "media_hash")
```

```
                        (request-param "media_path")
                        "done"
                        ""
                        ""
                        "[]"))
                     ((string-eq action "choose")
                      (edit/choose id (request-param "n")))
                     ((string-eq action "add-line")
                      (edit/line-upsert
                        (request-param "project")
                        id
                        (request-param "ord")
                        (request-param "character")
                        (request-param "text")
                        (request-param "notes")))
                     (else ()))
                (redirect
                  (string-append
                    "/studio/shot?id=" id "&project=" project)))
              (html/page
                (list (edit/shot-page-html id project)))))))
  "GET/POST /studio/shot")

(define edit/export-page
  (lambda ()
    (let ((slug (as-str (request-param "slug")))
          (cut (as-str (request-param "cut")))
          (kind (as-str (request-param "kind"))))
      (let ((c (if (edit/blank? cut) "main" cut))
            (k (if (edit/blank? kind) "mlt" kind)))
        (if (edit/blank? slug)
            (redirect "/studio")
            (html/page
              (list (edit/export-page-html slug c k)))))))
  "GET /studio/export")

;; NOTE: /edit is the kernel function editor. Package UI is /studio
;; (borge issue filed; film-edit.ui smokes still say /edit until plan).
(route "/studio" "edit/home-page")
(route "/studio/project" "edit/project-page")
(route "/studio/shot" "edit/shot-page")
(route "/studio/export" "edit/export-page")
```

164 libs/err/lib.pp

```
;; package err -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push
```

165 libs/flash/lib.pp

```
;; package flash -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push

(define flash/add
  (lambda
    (deck-id front back tags)
    "Create a flash-card in a deck with default fields."
    (create
      (quote flash-card)
      (flash/default-fields deck-id front back tags))))

(define flash/apply-rating
  (lambda
    (fields rating today)
    "Apply an FSRS review rating to card fields; return updated dict."
    (let
      ((s (flash/as-float (dict-get fields "stability")))
        (d (flash/as-float (dict-get fields "difficulty")))
        (last (as-str (dict-get fields "last_reviewed")))
        (reps (flash/as-int (dict-get fields "reps")))
        (lapses (flash/as-int (dict-get fields "lapses")))
        (r (as-str rating)))
      (let
        ((elapsed (flash/elapsed-days last today)))
        (let
          ((res (fsrs-schedule s d elapsed r)))
          (let
            ((s2 (nth res 0))
              (d2 (nth res 1))
              (iv (flash/as-int (nth res 2))))
            (dict-set*
              fields
              (list
                "stability"
                (as-str s2)
                "difficulty"
                (as-str d2)
                "interval"
                (str iv)
                "reps"
                (str (if (flash/rating-again? r) 0 (+ reps 1)))
                "lapses"
                (str (if (flash/rating-again? r) (+ lapses 1) lapses))
                "due"
                (if (= iv 0) today (date-add-days today iv))
                "last_reviewed"
                today)))))))))

(define flash/as-float
  (lambda (v) "Numeric as float; 0.0 when absent." (if (null? v) 0.0
    (parse-float (as-str v)))))

(define flash/as-int
  (lambda (v) "Numeric as int; 0 when absent." (if (null? v) 0 (parse-int
    (as-str v)))))

(define flash/blank?
  blank?
  "Alias for the kernel blank? predicate.")

(define flash/card
  (lambda (id) "Fetch a flash-card row by id." (get (quote flash-card)
    (as-str id))))

(define flash/card-back
  (lambda (fields) "Card back text." (as-str (dict-get fields "back"))))

(define flash/card-deck-id
  (lambda (fields) "Card's owning deck id." (as-str (dict-get fields
    "deck_id"))))

```

```
(define flash/card-front
  (lambda (fields) "Card front text." (as-str (dict-get fields "front"))))

(define flash/card-row-html
  (lambda
    (row)
    "HTML table row for one flash-card with delete action."
    (let
      ((id (as-str (car row))) (fields (cdr row)))
      (flash/tr
        (html/join
          (list
            (flash/td (html/text (flash/card-front fields)))
            (flash/td (html/text (as-str (dict-get fields "due"))))
            (flash/td
              (html/text (as-str (dict-get fields "interval"))))
            (flash/td
              (html/form
                "POST"
                (string-append
                  "/flashcards/deck?id="
                  (flash/card-deck-id fields))
                (html/attrs (list))
                (html/join
                  (list
                    (html/hidden "action" "delete-card")
                    (html/hidden "id" id)
                    (html/button (html/attr "type" "submit")
    "Delete")))))))))

(define flash/cards
  (lambda
    (deck-id)
    "All flash-cards belonging to a deck."
    (let
      ((did (as-str deck-id)))
      (list/filter
        (lambda
          (row)
          (string-eq (as-str (dict-get (cdr row) "deck_id")) did))
        (rows (quote flash-card) 500)))))

(define flash/char-le?
  (lambda
    (ca cb)
    "True when char ca sorts at-or-before char cb."
    (<= (flash/digit-rank ca) (flash/digit-rank cb))))

(define flash/deck
  (lambda (id) "Fetch a flash-deck row by id." (get (quote flash-deck)
    (as-str id))))

(define flash/deck-archive
  (lambda
    (id)
    "Mark a deck archived."
    (update-field
      (quote flash-deck)
      (as-str id)
      "archived"
      "true")))

(define flash/deck-create
  (lambda
    (title description)
    "Create a new deck from title and description."
    (create
      (quote flash-deck)
      (dict-set*
        "{}"
        (list
          "title"
          (as-str title)
```

```
          "description"
          (as-str description)
          "color"
          ""
          "archived"
          "")))))

(define flash/deck-page
  (lambda
    ()
    "Deck route handler: GET renders, POST handles card actions."
    (let
      ((id (as-str (request-param "id"))))
      (if
        (string-eq (request-method) "POST")
        (seq
          (let
            ((action (as-str (request-param "action"))))
            (cond
              ((string-eq action "add-card")
                (flash/add
                  id
                  (request-param "front")
                  (request-param "back")
                  (request-param "tags")))
              ((string-eq action "delete-card")
                (flash/delete (request-param "id")))
              (else ())))
          (redirect (string-append "/flashcards/deck?id=" id)))
        (html/page (list (flash/deck-page-html id)))))))

(define flash/deck-page-html
  (lambda
    (deck-id)
    "Full deck page HTML (header, add-card form, card table)."
    (let
      ((fields (flash/deck deck-id))
        (cards (flash/cards deck-id)))
      (if
        (null? fields)
        (html/p* (html/attr "class" "err") "Deck not found")
        (html/join
          (list
            (html/p*
              ""
              (html/a "/flashcards" "? All decks"))
            (html/h1 (flash/esc (flash/deck-title fields)))
            (html/p*
              (html/attr "class" "meta")
              (as-str (dict-get fields "description")))
            (html/p*
              ""
              (html/a
                (string-append
                  "/flashcards/study?deck="
                  (as-str deck-id))
                "Study this deck"))
            (html/div
              (html/attr "class" "card")
              (html/join
                (list
                  (html/h2 "Add card")
                  (html/form
                    "POST"
                    (string-append "/flashcards/deck?id=" (as-str deck-id))
                    (html/attrs (list))
                    (html/join
                      (list
                        (html/hidden "action" "add-card")
                        (html/label "" "Front")
                        (html/text-input
                          "front"
                          (html/attr "required" "required"))
```

```
                        (html/label "" "Back")
                        (html/text-input
                          "back"
                          (html/attr "required" "required"))
                        (html/label "" "Tags")
                        (html/text-input "tags" "")
                        (html/button (html/attr "type" "submit")
    "Add")))))))
            (html/div
              (html/attr "class" "card")
              (html/join
                (list
                  (html/h2 "Cards")
                  (if
                    (null? cards)
                    (html/p* (html/attr "class" "muted") "No cards yet.")
                    (flash/table
                      (html/join
                        (list
                          (flash/thead
                            (flash/tr
                              (html/join
                                (list
                                  (flash/th "Front")
                                  (flash/th "Due")
                                  (flash/th "Interval")
                                  (flash/th "")))))
                          (flash/tbody
                            (html/join (list/map flash/card-row-html
    cards)))))))))))))))

(define flash/deck-row-html
  (lambda
    (row)
    "HTML table row for one deck with due count and study link."
    (let
      ((id (as-str (car row))) (fields (cdr row)))
      (let
        ((title (flash/deck-title fields))
          (n (flash/due-count id)))
        (flash/tr
          (html/join
            (list
              (flash/td
                (html/a
                  (string-append "/flashcards/deck?id=" id)
                  (html/text title)))
              (flash/td (html/text (str n)))
              (flash/td
                (html/a
                  (string-append "/flashcards/study?deck=" id)
                  "Study")))))))))

(define flash/deck-title
  (lambda (fields) "Deck title text." (as-str (dict-get fields "title"))))

(define flash/decks
  (lambda
    ()
    "All non-archived decks."
    (list/filter
      (lambda
        (row)
        (not (flash/true? (dict-get (cdr row) "archived"))))
      (rows (quote flash-deck) 200))))

(define flash/decks-table
  (lambda
    (ds)
    "HTML table of decks, or a muted empty note."
    (if
      (null? ds)
      (html/p* (html/attr "class" "muted") "No decks yet.")
```

```
      (flash/table
        (html/join
          (list
            (flash/thead
              (flash/tr
                (html/join
                  (list (flash/th "Deck") (flash/th "Due") (flash/th "")))))
            (flash/tbody
              (html/join (list/map flash/deck-row-html ds)))))))))

(define flash/default-fields
  (lambda
    (deck-id front back tags)
    "Initial fields dict for a new flash-card."
    (dict-set*
      "{}"
      (list
        "deck_id"
        (as-str deck-id)
        "front"
        (as-str front)
        "back"
        (as-str back)
        "tags"
        (as-str tags)
        "suspended"
        ""
        "stability"
        "0"
        "difficulty"
        "0"
        "interval"
        "0"
        "reps"
        "0"
        "lapses"
        "0"
        "due"
        ""
        "last_reviewed"
        ""
        "created"
        (flash/today)))))

(define flash/delete
  (lambda (id) "Delete a flash-card by id." (delete (quote flash-card)
     (as-str id))))

(define flash/digit-rank
  (lambda
    (c)
    "Sort rank for a digit character (- < 0..9 < others)."
    (if
      (string-eq c "-")
      0
      (if
        (string-eq c "0")
        1
        (if
          (string-eq c "1")
          2
          (if
            (string-eq c "2")
            3
            (if
              (string-eq c "3")
              4
              (if
                (string-eq c "4")
                5
                (if
                  (string-eq c "5")
                  6
```

```
                  (if
                    (string-eq c "6")
                    7
                    (if
                      (string-eq c "7")
                      8
                      (if (string-eq c "8") 9 (if (string-eq c "9") 10
    11)))))))))))))

(define flash/due
  (lambda
    (deck-id limit)
    "Due cards in a deck (all when deck blank), up to limit."
    (let
      ((today (flash/today))
        (lim (if (null? limit) 20 (flash/as-int limit)))
        (did (as-str deck-id)))
      (let
        ((xs
          (list/filter
            (lambda
              (row)
              (if
                (flash/row-due? row today)
                (if
                  (flash/blank? did)
                  #t
                  (string-eq (flash/card-deck-id (cdr row)) did))
                #f))
            (rows (quote flash-card) 500))))
        (flash/take xs lim)))))

(define flash/due-count
  (lambda (deck-id) "Number of cards due now in a deck." (count (flash/due
     deck-id 500))))

(define flash/due-now?
  (lambda
    (due today)
    "True when a due date is today or earlier."
    (let
      ((d (as-str due)))
      (if
        (flash/blank? d)
        #t
        (if (string-eq d today) #t (flash/str-le? d today))))))

(define flash/elapsed-days
  (lambda
    (last today)
    "Days between last review and today; 0 when never reviewed."
    (if
      (flash/blank? last)
      0
      (date-diff-days (as-str last) (as-str today)))))

(define flash/esc
  (lambda (s) "HTML-escape a string." (html-escape (as-str s))))

(define flash/explain
  (lambda
    (front back)
    "Ask the agent to explain a flashcard in plain language."
    (let
      ((sid (agent-session-new flash/explain-seed #f)))
      (agent-say
        sid
        (string-append
          "I don't understand this flashcard. Can you explain it to me?\n\n"
          "Front:\n"
          front
          "\n\n"
          "Back:\n"
```

```
          back
          "\n\n"
          "Please explain what this means in plain language.")))))

(define flash/explain-html
  (lambda
    (text)
    "HTML card wrapping an agent explanation."
    (string-append
      "<div class=\"card\" style=\"margin-top:1rem\">"
      "<h3>Explanation</h3>"
      "<div style=\"white-space:pre-wrap\">"
      (flash/esc text)
      "</div>"
      "</div>")))

(define flash/explain-page
  (lambda
    ()
    "Agent explanation route handler for a card id."
    (let
      ((id (as-str (request-param "id"))))
      (let
        ((fields (flash/card id)))
        (if
          (null? fields)
          (render-html (flash/explain-html "Card not found."))
          (render-html
            (flash/explain-html
              (flash/explain
                (flash/card-front fields)
                (flash/card-back fields)))))))))

(define flash/explain-seed
  "You are a patient tutor helping someone understand a flashcard. Explain
     the concept clearly and concisely in plain language. Use short
     paragraphs separated by blank lines. Do not use markdown headings or
     code fences unless the card itself is about code. If the card is about
     a specific system (e.g. Pricklypear, OCaml), assume the reader is
     learning that system.")

(define flash/home-html
  (lambda
    ()
    "Flashcards home body HTML (decks + new-deck form)."
    (let
      ((ds (flash/decks)))
      (html/join
        (list
          (html/h1 "Flashcards")
          (html/p*
            (html/attr "class" "meta")
            "Spaced repetition on shared data_rows. Data is shared across
     branches.")
          (html/p*
            ""
            (html/a "/flashcards/study" "Study all due"))
          (flash/new-deck-form)
          (html/div
            (html/attr "class" "card")
            (html/join
              (list (html/h2 "Decks") (flash/decks-table ds)))))))))

(define flash/home-page
  (lambda
    ()
    "Flashcards home route handler (GET render / POST create)."
    (if
      (string-eq (request-method) "POST")
      (seq
        (let
          ((action (as-str (request-param "action"))))
          (when
```

```
            (string-eq action "create-deck")
            (flash/deck-create
              (request-param "title")
              (request-param "description"))))
        (redirect "/flashcards"))
      (html/page (list (flash/home-html))))))

(define flash/new-deck-form
  (lambda
    ()
    "HTML form for creating a new deck."
    (html/div
      (html/attr "class" "card")
      (html/join
        (list
          (html/h2 "New deck")
          (html/form
            "POST"
            "/flashcards"
            (html/attrs (list))
            (html/join
              (list
                (html/hidden "action" "create-deck")
                (html/label "" "Title")
                (html/text-input
                  "title"
                  (html/attr "required" "required"))
                (html/label "" "Description")
                (html/text-input "description" "")
                (html/button (html/attr "type" "submit") "Create")))))))))

(define flash/rating-again?
  (lambda
    (r)
    "True for the \"again\" rating (or 1)."
    (let
      ((s (as-str r)))
      (if (string-eq s "again") #t (string-eq s "1")))))

(define flash/rating-form
  (lambda
    (card-id deck rating label)
    "Inline study rating submit form."
    (html/form
      "POST"
      "/flashcards/study"
      (html/attrs
        (list
          (html/attr "style" "display:inline;margin-right:.35rem")))
      (html/join
        (list
          (html/hidden "action" "review")
          (html/hidden "id" (as-str card-id))
          (html/hidden "rating" rating)
          (html/hidden "deck" (as-str deck))
          (html/button (html/attr "type" "submit") label))))))

(define flash/review
  (lambda
    (id rating)
    "Apply a rating to a card and persist it."
    (let
      ((fields (flash/card id)))
      (if
        (null? fields)
        "review: no such card"
        (let
          ((updated
            (flash/apply-rating fields rating (flash/today))))
          (seq
            (update (quote flash-card) (as-str id) updated)
            updated))))))

```

```
(define flash/row-due?
  (lambda
    (row today)
    "True when a row is unsuspended and due today or earlier."
    (let
      ((fields (cdr row)))
      (if
        (flash/suspended? fields)
        #f
        (flash/due-now? (dict-get fields "due") today)))))

(define flash/stats
  (lambda
    (deck-id)
    "(total due suspended) counts for a deck (all when blank)."
    (let
      ((today (flash/today))
        (did (as-str deck-id))
        (all (rows (quote flash-card) 500)))
      (let
        ((scoped
          (if
            (flash/blank? did)
            all
            (list/filter
              (lambda
                (row)
                (string-eq (flash/card-deck-id (cdr row)) did))
              all))))
        (list
          (count scoped)
          (count
            (list/filter
              (lambda (row) (flash/row-due? row today))
              scoped))
          (count
            (list/filter
              (lambda (row) (flash/suspended? (cdr row)))
              scoped)))))))

(define flash/str-le-loop
  (lambda
    (a b i na nb)
    "Tail-recursive char-wise string comparison loop."
    (if
      (>= i na)
      #t
      (if
        (>= i nb)
        #f
        (let
          ((ca (substring a i 1)) (cb (substring b i 1)))
          (if
            (string-eq ca cb)
            (flash/str-le-loop a b (+ i 1) na nb)
            (flash/char-le? ca cb)))))))

(define flash/str-le?
  (lambda
    (a b)
    "Lexicographic string less-than-or-equal."
    (let
      ((sa (as-str a)) (sb (as-str b)))
      (if
        (string-eq sa sb)
        #t
        (flash/str-le-loop
          sa
          sb
          0
          (string-length sa)
          (string-length sb))))))

```

```
(define flash/study-card-html
  (lambda
    (row deck reveal)
    "HTML for one study card with reveal + rating buttons."
    (let
      ((id (as-str (car row))) (fields (cdr row)))
      (html/div
        (html/attr "class" "card")
        (html/join
          (list
            (html/h2 "Front")
            (html/p (flash/card-front fields))
            (if
              (flash/true? reveal)
              (html/join
                (list
                  (html/h2 "Back")
                  (html/p (flash/card-back fields))
                  (html/p*
                    ""
                    (html/join
                      (list
                        (flash/rating-form id deck "again" "Again")
                        (flash/rating-form id deck "hard" "Hard")
                        (flash/rating-form id deck "good" "Good")
                        (flash/rating-form id deck "easy" "Easy"))))))
              (html/form
                "GET"
                "/flashcards/study"
                (html/attrs (list))
                (html/join
                  (list
                    (html/hidden "deck" (as-str deck))
                    (html/hidden "id" id)
                    (html/hidden "reveal" "true")
                    (html/button (html/attr "type" "submit")
    "Reveal")))))))))))

(define flash/study-html
  (lambda
    (deck-id card-id reveal)
    "Study page body: next due card with reveal/rating controls."
    (let
      ((due (flash/due deck-id 1)))
      (html/join
        (list
          (html/p* "" (html/a "/flashcards" "? Decks"))
          (html/h1 "Study")
          (if
            (null? due)
            (html/div
              (html/attr "class" "card")
              (html/join
                (list
                  (html/p "No cards due. Nice work.")
                  (html/p* "" (html/a "/flashcards" "Back to decks")))))
            (let
              ((row
                (if
                  (flash/blank? card-id)
                  (car due)
                  (let
                    ((found
                      (list/filter
                        (lambda
                          (r)
                          (string-eq (as-str (car r)) (as-str card-id)))
                        due)))
                    (if (null? found) (car due) (car found))))))
              (flash/study-card-html row deck-id reveal)))))))))

(define flash/study-page
  (lambda
```

```
    ()
    "Study route handler: GET renders, POST handles review rating."
    (let
      ((deck (as-str (request-param "deck")))
        (id (as-str (request-param "id")))
        (reveal (as-str (request-param "reveal"))))
      (if
        (string-eq (request-method) "POST")
        (seq
          (let
            ((action (as-str (request-param "action"))))
            (when
              (string-eq action "review")
              (flash/review
                (request-param "id")
                (request-param "rating"))))
          (redirect
            (if
              (flash/blank? deck)
              "/flashcards/study"
              (string-append "/flashcards/study?deck=" deck))))
        (html/page
          (list
            (flash/study-html
              deck
              id
              (if (string-eq reveal "true") "true" ""))))))))

(define flash/suspend
  (lambda
    (id)
    "Mark a flash-card suspended."
    (update-field
      (quote flash-card)
      (as-str id)
      "suspended"
      "true")))

(define flash/suspended?
  (lambda
    (fields)
    "True when card fields are suspended."
    (flash/true? (dict-get fields "suspended"))))

(define flash/table
  (lambda (body) "Wrap content in a <table>." (html/tag "table" "" body)))

(define flash/take
  (lambda
    (xs n)
    "First n elements of a list (fewer when shorter)."
    (if
      (null? xs)
      (list)
      (if
        (<= n 0)
        (list)
        (append (list (car xs)) (flash/take (cdr xs) (- n 1)))))))

(define flash/tbody
  (lambda (body) "Wrap content in a <tbody>." (html/tag "tbody" "" body)))

(define flash/td
  (lambda (body) "Wrap content in a <td>." (html/tag "td" "" body)))

(define flash/th
  (lambda (body) "Wrap content in a <th>." (html/tag "th" "" body)))

(define flash/thead
  (lambda (body) "Wrap content in a <thead>." (html/tag "thead" "" body)))

(define flash/today
  (lambda () "Today's date string." (date-today)))
```

```
784
785 (define flash/tr
786   (lambda (body) "Wrap content in a <tr>." (html/tag "tr" "" body)))
787
788 (define flash/true?
789   (lambda (v) "True when a value stringifies to \"true\"." (string-eq
        (as-str v) "true")))
790
791 (define flash/unsuspend
792   (lambda
793     (id)
794     "Clear the suspended flag on a flash-card."
795     (update-field
796       (quote flash-card)
797       (as-str id)
798       "suspended"
799       "")))
800
801 (define flash/widget
802   (lambda
803     ()
804     "Dashboard widget with due/total card counts."
805     (flash/widget-stats
806       (flash/due-count "")
807       (count (rows (quote flash-card) 500)))))
808
809 (define flash/widget-stats
810   (lambda
811     (due total)
812     "Dashboard card rendering due and total counts."
813     (layout/card
814       "Flashcards"
815       (html/div
816         (list)
817         (html/join
818           (list
819             (html/p*
820               (html/attr "class" "flash-widget-due")
821               (html/join
822                 (list
823                   (html/strong (str due))
824                   (html/text " cards due today"))))
825             (html/p*
826               (list)
827               (html/text (string-append (str total) " total cards")))
828             (if
829               (> due 0)
830               (layout/cta "/flashcards/study" "Study now")
831               (html/p*
832                 (html/attr "class" "muted")
833                 "Nothing due -- you're all caught up!"))
834             (html/p* (list) (html/a "/flashcards" "Manage decks")))))))))
835
836 (define flash/zz-main-sentinel
837   (lambda () "MAIN-SENTINEL"))
838
839 (define flash/zz-sentinel
840   (lambda () "CHANGED-99"))
```

166 libs/flashcards/lib.pp

```
;; Flashcards -- decks + FSRS-4.5 spaced repetition on data_rows.
;; Prefix: flash/   Package dir: flashcards
;; Install: (load-library "flashcards") -- seeded after calendar.
;;
;; Types: flash-deck, flash-card
;; Scheduler: kernel prim (fsrs-schedule S D elapsed rating)
;;   -> (S' D' interval_days). New cards use S=0 D=0.
;; due is YYYY-MM-DD; empty or due <= today means due now.

(deftype 'flash-deck 'title 'description 'color 'archived)
(deftype 'flash-card
  'deck_id 'front 'back 'tags 'suspended
  'stability 'difficulty 'interval 'reps 'lapses
  'due 'last_reviewed 'created)

;; --- helpers ---

(define flash/blank?
  (lambda
    (s)
    "True if value is blank after as-str trim."
    (if (null? s) #t (string-eq (as-str s) ""))))

(define flash/today
  (lambda
    ()
    "Today date string YYYY-MM-DD."
    (date-today)))

(define flash/as-int
  (lambda (v)
    (if (null? v) 0 (parse-int (as-str v))))
  "Coerce value to int.")

(define flash/as-float
  (lambda (v)
    (if (null? v) 0.0 (parse-float (as-str v))))
  "Coerce value to float.")

(define flash/true?
  (lambda (v) (string-eq (as-str v) "true"))
  "Truthy check for bool-ish field strings.")

;; Empty due is always due; else ISO YYYY-MM-DD string order.
(define flash/due-now?
  (lambda (due today)
    (let ((d (as-str due)))
      (if (flash/blank? d)
          #t
          (if (string-eq d today)
              #t
              (flash/str-le? d today)))))
  "True if due date empty or on/before today (ISO order).")

(define flash/str-le?
  (lambda (a b)
    (let ((sa (as-str a))
          (sb (as-str b)))
      (if (string-eq sa sb)
          #t
          (flash/str-le-loop sa sb 0 (string-length sa) (string-length
    sb)))))
  "Lexicographic string <= comparison.")

(define flash/str-le-loop
  (lambda (a b i na nb)
    (if (>= i na)
        #t
        (if (>= i nb)
            #f
            (let ((ca (substring a i 1))
```

```
                  (cb (substring b i 1)))
              (if (string-eq ca cb)
                  (flash/str-le-loop a b (+ i 1) na nb)
                  (flash/char-le? ca cb))))))
  "Internal loop for flash/str-le?.")

(define flash/digit-rank
  (lambda (c)
    (if (string-eq c "-") 0
      (if (string-eq c "0") 1
        (if (string-eq c "1") 2
          (if (string-eq c "2") 3
            (if (string-eq c "3") 4
              (if (string-eq c "4") 5
                (if (string-eq c "5") 6
                  (if (string-eq c "6") 7
                    (if (string-eq c "7") 8
                      (if (string-eq c "8") 9
                        (if (string-eq c "9") 10 11)))))))))))
  "Rank of digit character for comparisons.")

(define flash/char-le?
  (lambda (ca cb)
    (<= (flash/digit-rank ca) (flash/digit-rank cb)))
  "Character <= comparison helper.")

(define flash/tr (lambda (body) (html/tag "tr" "" body))
  "HTML tr tag helper.")
(define flash/td (lambda (body) (html/tag "td" "" body))
  "HTML td tag helper.")
(define flash/th (lambda (body) (html/tag "th" "" body))
  "HTML th tag helper.")
(define flash/table (lambda (body) (html/tag "table" "" body))
  "HTML table tag helper.")
(define flash/thead (lambda (body) (html/tag "thead" "" body))
  "HTML thead tag helper.")
(define flash/tbody (lambda (body) (html/tag "tbody" "" body))
  "HTML tbody tag helper.")

;; --- decks ---

(define flash/decks
  (lambda ()
    ;; Cap list size: each home row used to call flash/due-count (full card
    ;; scan). With hundreds of decks that blows the eval budget (100k
     steps).
    (flash/take
      (list/filter
        (lambda (row)
          (not (flash/true? (dict-get (cdr row) "archived"))))
        (rows 'flash-deck 80))
      40))
  "List flashcard deck rows (max 40 non-archived for home UI).")

(define flash/deck
  (lambda (id)
    (get 'flash-deck (as-str id)))
  "Fetch deck row by id.")

(define flash/deck-title
  (lambda (fields) (as-str (dict-get fields "title")))
  "Deck title from fields.")

(define flash/deck-create
  (lambda (title description)
    (create 'flash-deck
      (dict-set* "{}"
        (list
          "title" (as-str title)
          "description" (as-str description)
          "color" ""
          "archived" "")))))
  "Create a new deck with title.")
```

```
143
144 (define flash/deck-archive
145   (lambda (id)
146     (update-field 'flash-deck (as-str id) "archived" "true"))
147   "Archive deck by id.")
148
149 ;; --- cards ---
150
151 (define flash/default-fields
152   (lambda (deck-id front back tags)
153     (dict-set* "{}"
154       (list
155         "deck_id" (as-str deck-id)
156         "front" (as-str front)
157         "back" (as-str back)
158         "tags" (as-str tags)
159         "suspended" ""
160         "stability" "0"
161         "difficulty" "0"
162         "interval" "0"
163         "reps" "0"
164         "lapses" "0"
165         "due" ""
166         "last_reviewed" ""
167         "created" (flash/today))))
168   "Default card scheduling fields dict.")
169
170 (define flash/add
171   (lambda (deck-id front back tags)
172     (create 'flash-card (flash/default-fields deck-id front back tags)))
173   "Add card to deck with front/back.")
174
175 (define flash/card
176   (lambda (id) (get 'flash-card (as-str id)))
177   "Fetch card row by id.")
178
179 (define flash/delete
180   (lambda (id) (delete 'flash-card (as-str id)))
181   "Delete card by id.")
182
183 (define flash/suspend
184   (lambda (id) (update-field 'flash-card (as-str id) "suspended" "true"))
185   "Suspend card from review.")
186
187 (define flash/unsuspend
188   (lambda (id) (update-field 'flash-card (as-str id) "suspended" ""))
189   "Unsuspend card for review.")
190
191 (define flash/cards
192   (lambda (deck-id)
193     (let ((did (as-str deck-id)))
194       (list/filter
195         (lambda (row)
196           (string-eq (as-str (dict-get (cdr row) "deck_id")) did))
197         (rows 'flash-card 500))))
198   "Cards belonging to deck id.")
199
200 (define flash/card-front
201   (lambda (fields) (as-str (dict-get fields "front")))
202   "Card front text from fields.")
203
204 (define flash/card-back
205   (lambda (fields) (as-str (dict-get fields "back")))
206   "Card back text from fields.")
207
208 (define flash/card-deck-id
209   (lambda (fields) (as-str (dict-get fields "deck_id")))
210   "Deck id from card fields.")
211
212 (define flash/suspended?
213   (lambda (fields) (flash/true? (dict-get fields "suspended")))
214   "True if card is suspended.")
215
```

```
;; --- FSRS-4.5 via kernel (fsrs-schedule) ---

(define flash/elapsed-days
  (lambda (last today)
    (if (flash/blank? last)
        0
        (date-diff-days (as-str last) (as-str today))))
  "Days elapsed since last review field.")

(define flash/rating-again?
  (lambda (r)
    (let ((s (as-str r)))
      (if (string-eq s "again") #t (string-eq s "1"))))
  "True if rating means Again.")

(define flash/apply-rating
  (lambda (fields rating today)
    (let ((s (flash/as-float (dict-get fields "stability")))
          (d (flash/as-float (dict-get fields "difficulty")))
          (last (as-str (dict-get fields "last_reviewed")))
          (reps (flash/as-int (dict-get fields "reps")))
          (lapses (flash/as-int (dict-get fields "lapses")))
          (r (as-str rating)))
      (let ((elapsed (flash/elapsed-days last today)))
        (let ((res (fsrs-schedule s d elapsed r)))
          (let ((s2 (nth res 0))
                (d2 (nth res 1))
                (iv (flash/as-int (nth res 2))))
            (dict-set* fields
              (list
                "stability" (as-str s2)
                "difficulty" (as-str d2)
                "interval" (str iv)
                "reps"
                  (str (if (flash/rating-again? r) 0 (+ reps 1)))
                "lapses"
                  (str (if (flash/rating-again? r) (+ lapses 1) lapses))
                "due"
                  (if (= iv 0) today (date-add-days today iv))
                "last_reviewed" today)))))))
  "Apply FSRS rating to card fields via kernel.")

(define flash/review
  (lambda (id rating)
    (let ((fields (flash/card id)))
      (if (null? fields)
          "review: no such card"
          (let ((updated (flash/apply-rating fields rating (flash/today))))
            (seq
              (update 'flash-card (as-str id) updated)
              updated))))))
  "Record review rating for card id.")

;; --- explain (LLM) ---

(define flash/explain-seed
  "You are a patient tutor helping someone understand a flashcard. Explain
     the concept clearly and concisely in plain language. Use short
     paragraphs separated by blank lines. Do not use markdown headings or
     code fences unless the card itself is about code. If the card is about
     a specific system (e.g. Pricklypear, OCaml), assume the reader is
     learning that system."
  "System seed for card-explanation LLM sessions (no tools).")

(define flash/explain
  (lambda (front back)
    (let ((sid (agent-session-new flash/explain-seed #f)))
      (agent-say sid
        (string-append
          "I don't understand this flashcard. Can you explain it to me?\n\n"
          "Front:\n" front "\n\n"
          "Back:\n" back "\n\n"
          "Please explain what this means in plain language."))))
```

```
  "Ask the LLM to explain a card's front/back in plain language; returns
     explanation text.")

(define flash/explain-html
  (lambda (text)
    (string-append
      "<div class=\"card\" style=\"margin-top:1rem\">"
      "<h3>Explanation</h3>"
      "<div style=\"white-space:pre-wrap\">" (flash/esc text) "</div>"
      "</div>"))
  "HTML for the explanation panel. Escapes LLM text; preserves newlines via
     CSS.")

(define flash/explain-page
  (lambda ()
    (let ((id (as-str (request-param "id"))))
      (if (flash/blank? id)
          (render-html (flash/explain-html "Missing card id."))
          (let ((fields (flash/card id)))
            (if (null? fields)
                (render-html (flash/explain-html "Card not found."))
                (render-html
                  (flash/explain-html
                    (flash/explain (flash/card-front fields)
    (flash/card-back fields)))))))))
  "GET /flashcards/explain?id= -- LLM explanation of a card as a raw HTML
     div.")

(define flash/row-due?
  (lambda (row today)
    (let ((fields (cdr row)))
      (if (flash/suspended? fields)
          #f
          (flash/due-now? (dict-get fields "due") today))))
  "True if card row is due for review.")

(define flash/due
  (lambda (deck-id limit)
    (let ((today (flash/today))
          (lim (if (null? limit) 20 (flash/as-int limit)))
          (did (as-str deck-id)))
      (let ((xs
              (list/filter
                (lambda (row)
                  (if (flash/row-due? row today)
                      (if (flash/blank? did)
                          #t
                          (string-eq (flash/card-deck-id (cdr row)) did))
                      #f))
                (rows 'flash-card 500))))
        ;; Session-fresh cards (last_reviewed < today or blank) come before
        ;; cards already reviewed today, so a card rated Again sinks to the
        ;; bottom of the stack and is not re-shown until the rest of the due
        ;; batch is cleared. Created/id order preserved within each part.
        (let ((seen
                (list/filter
                  (lambda (row)
                    (string-eq (as-str (dict-get (cdr row)
    "last_reviewed")) today))
                  xs))
              (fresh
                (list/filter
                  (lambda (row)
                    (not (string-eq (as-str (dict-get (cdr row)
    "last_reviewed")) today)))
                  xs)))
          (flash/take (append fresh seen) lim)))))
  "Due cards for deck (or all), unsuspended. Session-fresh cards (not yet
  reviewed today) come before cards already reviewed today, so a card rated
  Again sinks to the bottom of the stack and is not re-shown until the rest
  of the due batch is cleared.")

(define flash/take
```

```
  (lambda (xs n)
    (if (null? xs)
        (list)
        (if (<= n 0)
            (list)
            (append (list (car xs))
                    (flash/take (cdr xs) (- n 1))))))
  "Take first n items from list.")

(define flash/due-count
  (lambda (deck-id)
    (count (flash/due deck-id 500)))
  "Count of due cards for deck.")

(define flash/stats
  (lambda (deck-id)
    (let ((today (flash/today))
          (did (as-str deck-id))
          (all (rows 'flash-card 500)))
      (let ((scoped
              (if (flash/blank? did)
                  all
                  (list/filter
                    (lambda (row)
                      (string-eq (flash/card-deck-id (cdr row)) did))
                    all))))
        (list
          (count scoped)
          (count
            (list/filter
              (lambda (row) (flash/row-due? row today))
              scoped))
          (count
            (list/filter
              (lambda (row) (flash/suspended? (cdr row)))
              scoped))))))
  "Deck stats dict (due, new, suspended counts).")

;; --- HTML ---

(define flash/esc (lambda (s) (html-escape (as-str s)))
  "HTML-escape helper. Use ONLY with raw-body builders (html/h1, flash/td
  WITHOUT an inner html/text) -- those do not escape. The auto-escaping
  builders html/p and html/text already call html-escape, so wrapping their
  args with flash/esc DOUBLE-escapes (\" -> &quot; -> &amp;quot;, which the
  browser shows as the literal text '&quot;').")

(define flash/deck-row-html
  (lambda (row)
    (let ((id (as-str (car row)))
          (fields (cdr row)))
      (let ((title (flash/deck-title fields)))
        (flash/tr
          (html/join
            (list
              (flash/td
                (html/a
                  (string-append "/flashcards/deck?id=" id)
                  (html/text title)))
              (flash/td
                (html/a
                  (string-append "/flashcards/study?deck=" id)
                  "Study")))))))))
  "Table row HTML for one deck (no per-row due scan -- budget-safe).")

(define flash/new-deck-form
  (lambda ()
    (html/div (html/attr "class" "card")
      (html/join
        (list
          (html/h2 "New deck")
          (html/form "POST" "/flashcards"
            (html/attrs (list))
```

```
            (html/join
              (list
                (html/hidden "action" "create-deck")
                (html/label "" "Title")
                (html/text-input "title" (html/attr "required" "required"))
                (html/label "" "Description")
                (html/text-input "description" "")
                (html/button (html/attr "type" "submit") "Create")))))))
  "Form HTML to create a deck.")

(define flash/decks-table
  (lambda (ds)
    (if (null? ds)
        (html/p* (html/attr "class" "muted") "No decks yet.")
        (flash/table
          (html/join
            (list
              (flash/thead
                (flash/tr
                  (html/join
                    (list
                      (flash/th "Deck")
                      (flash/th "")))))
              (flash/tbody
                (html/join (list/map flash/deck-row-html ds))))))))
  "Table of all decks.")

(define flash/home-html
  (lambda ()
    (let ((ds (flash/decks)))
      (html/join
        (list
          (html/h1 "Flashcards")
          (html/p* (html/attr "class" "meta")
            "Spaced repetition on shared data_rows. Data is shared across
    branches.")
          (html/p* "" (html/a "/flashcards/study" "Study all due"))
          (flash/new-deck-form)
          (html/div (html/attr "class" "card")
            (html/join
              (list
                (html/h2 "Decks")
                (flash/decks-table ds))))))))
  "Flashcards home panel HTML.")

(define flash/card-row-html
  (lambda (row)
    (let ((id (as-str (car row)))
          (fields (cdr row)))
      (flash/tr
        (html/join
          (list
            (flash/td (html/text (flash/card-front fields)))
            (flash/td (html/text (as-str (dict-get fields "due"))))
            (flash/td (html/text (as-str (dict-get fields "interval"))))
            (flash/td
              (html/form "POST"
                (string-append "/flashcards/deck?id="
                  (flash/card-deck-id fields))
                (html/attrs (list))
                (html/join
                  (list
                    (html/hidden "action" "delete-card")
                    (html/hidden "id" id)
                    (html/button (html/attr "type" "submit")
    "Delete")))))))))))
  "Table row HTML for one card in deck view.")

(define flash/deck-page-html
  (lambda (deck-id)
    (let ((fields (flash/deck deck-id))
          (cards (flash/cards deck-id)))
      (if (null? fields)
```

```
          (html/p* (html/attr "class" "err") "Deck not found")
          (html/join
            (list
              (html/p* "" (html/a "/flashcards" "? All decks"))
              (html/h1 (flash/esc (flash/deck-title fields)))
              (html/p* (html/attr "class" "meta")
                (as-str (dict-get fields "description")))
              (html/p* ""
                (html/a
                  (string-append "/flashcards/study?deck=" (as-str deck-id))
                  "Study this deck"))
              (html/div (html/attr "class" "card")
                (html/join
                  (list
                    (html/h2 "Add card")
                    (html/form "POST"
                      (string-append "/flashcards/deck?id=" (as-str deck-id))
                      (html/attrs (list))
                      (html/join
                        (list
                          (html/hidden "action" "add-card")
                          (html/label "" "Front")
                          (html/text-input "front" (html/attr "required" "required"))
                          (html/label "" "Back")
                          (html/text-input "back" (html/attr "required" "required"))
                          (html/label "" "Tags")
                          (html/text-input "tags" "")
                          (html/button (html/attr "type" "submit") "Add")))))))
              (html/div (html/attr "class" "card")
                (html/join
                  (list
                    (html/h2 "Cards")
                    (if (null? cards)
                        (html/p* (html/attr "class" "muted") "No cards yet.")
                        (flash/table
                          (html/join
                            (list
                              (flash/thead
                                (flash/tr
                                  (html/join
                                    (list
                                      (flash/th "Front")
                                      (flash/th "Due")
                                      (flash/th "Interval")
                                      (flash/th ""))))))
                              (flash/tbody
                                (html/join
                                  (list/map flash/card-row-html cards))))))))))))
  "Deck detail page HTML with cards list.")

(define flash/rating-form
  (lambda (card-id deck rating label)
    (html/form "POST" "/flashcards/study"
      (html/attrs (list (html/attr "style" "display:inline;margin-right:.35rem")))
      (html/join
        (list
          (html/hidden "action" "review")
          (html/hidden "id" (as-str card-id))
          (html/hidden "rating" rating)
          (html/hidden "deck" (as-str deck))
          (html/button (html/attr "type" "submit") label)))))
  "Study rating buttons form for card.")

(define flash/study-card-html
  (lambda (row deck reveal)
    (let ((id (as-str (car row)))
```

```
          (fields (cdr row)))
      (html/div (html/attr "class" "card")
        (html/join
          (list
            (html/h2 "Front")
            (html/p (flash/card-front fields))
            (if (flash/true? reveal)
                (html/join
                  (list
                    (html/h2 "Back")
                    (html/p (flash/card-back fields))
                    (html/p* ""
                      (html/join
                        (list
                          (flash/rating-form id deck "again" "Again")
                          (flash/rating-form id deck "hard" "Hard")
                          (flash/rating-form id deck "good" "Good")
                          (flash/rating-form id deck "easy" "Easy"))))
                    (html/p* ""
                      (html/join
                        (list
                          (html/button
                            (html/attrs
                              (list
                                (html/attr "type" "button")
                                (html/attr "class" "secondary")
                                (html/attr "hx-get"
                                  (string-append "/flashcards/explain?id="
    id))
                                (html/attr "hx-target" "#flash-explain")
                                (html/attr "hx-swap" "innerHTML")
                                (html/attr "hx-indicator"
    "#explain-spinner")
                                (html/attr "hx-disabled-elt" "this")))
                            "Explain")
                          (html/span
                            (html/attrs
                              (list
                                (html/attr "id" "explain-spinner")
                                (html/attr "class" "htmx-indicator")))
                            "Thinking?"))))
                    (html/div
                      (html/attrs
                        (list
                          (html/attr "id" "flash-explain")))
                      ""))))
                (html/form "GET" "/flashcards/study"
                  (html/attrs (list))
                  (html/join
                    (list
                      (html/hidden "deck" (as-str deck))
                      (html/hidden "id" id)
                      (html/hidden "reveal" "true")
                      (html/button (html/attr "type" "submit")
    "Reveal")))))))))
  "Single study card face HTML.")

(define flash/study-html
  (lambda (deck-id card-id reveal)
    (let ((due (flash/due deck-id 1)))
      (html/join
        (list
          (html/p* "" (html/a "/flashcards" "? Decks"))
          (html/h1 "Study")
          (if (null? due)
              (html/div (html/attr "class" "card")
                (html/join
                  (list
                    (html/p "No cards due. Nice work.")
                    (html/p* "" (html/a "/flashcards" "Back to decks")))))
              (let ((row
                      (if (flash/blank? card-id)
                          (car due)
```

```
                          (let ((found
                                  (list/filter
                                    (lambda (r)
                                      (string-eq (as-str (car r)) (as-str card-id)))
                                    due)))
                            (if (null? found) (car due) (car found))))))
                (flash/study-card-html row deck-id reveal)))))))
  "Study session panel HTML for deck.")

;; --- routes ---

(define flash/home-page
  (lambda ()
    (if (string-eq (request-method) "POST")
        (seq
          (let ((action (as-str (request-param "action"))))
            (when (string-eq action "create-deck")
              (flash/deck-create
                (request-param "title")
                (request-param "description"))))
          (redirect "/flashcards"))
        (html/page (list (flash/home-html)))))
  "GET/POST /flashcards home handler.")

(define flash/deck-page
  (lambda ()
    (let ((id (as-str (request-param "id"))))
      (if (flash/blank? id)
          (redirect "/flashcards")
          (if (string-eq (request-method) "POST")
              (seq
                (let ((action (as-str (request-param "action"))))
                  (cond
                    ((string-eq action "add-card")
                     (flash/add id
                       (request-param "front")
                       (request-param "back")
                       (request-param "tags")))
                    ((string-eq action "delete-card")
                     (flash/delete (request-param "id")))
                    (else ())))
                (redirect (string-append "/flashcards/deck?id=" id)))
              (html/page (list (flash/deck-page-html id)))))))
  "GET/POST deck detail handler.")

(define flash/study-page
  (lambda ()
    (let ((deck (as-str (request-param "deck")))
          (id (as-str (request-param "id")))
          (reveal (as-str (request-param "reveal"))))
      (if (string-eq (request-method) "POST")
          (seq
            (let ((action (as-str (request-param "action"))))
              (when (string-eq action "review")
                (flash/review
                  (request-param "id")
                  (request-param "rating"))))
            (redirect
              (if (flash/blank? deck)
                  "/flashcards/study"
                  (string-append "/flashcards/study?deck=" deck))))
          (html/page
            (list
              (flash/study-html deck id
                (if (string-eq reveal "true") "true" ""))))))))
  "GET/POST study session handler.")

(route "/flashcards" "flash/home-page")
(route "/flashcards/deck" "flash/deck-page")
(route "/flashcards/study" "flash/study-page")
(route "/flashcards/explain" "flash/explain-page")
```

167 libs/food/lib.pp

```
;; package food -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push

(define food/a
  (lambda () 1))

(define food/add-template
  (lambda
    (day-type
      meal
      food
      calories
      protein
      carbs
      fat)
    (create
      (quote food-template)
      (dict-set*
        "{}"
        (list
          "day-type"
          (as-str day-type)
          "meal"
          (as-str meal)
          "food"
          (as-str food)
          "calories"
          (as-str calories)
          "protein"
          (as-str protein)
          "carbs"
          (as-str carbs)
          "fat"
          (as-str fat)
          "author"
          (as-str (whoami)))))))

(define food/all
  (lambda () (rows (quote food-log) 500)))

(define food/apply-templates
  (lambda
    (date day-type)
    (food/foldl
      (lambda
        (acc entry)
        (food/log
          date
          (food/entry-str entry "meal")
          (food/entry-str entry "food")
          (food/entry-val entry "calories")
          (food/entry-val entry "protein")
          (food/entry-val entry "carbs")
          (food/entry-val entry "fat")
          ""))
      0
      (food/templates day-type))))

(define food/b
  (lambda () 2))

(define food/cal-of
  (lambda
    (row)
    (parse-int (food/field (cdr row) "calories"))))

(define food/dates-back
  (lambda
    (n)
    (if
      (< n 0)
```

```
      ()
      (cons
        (date-add-days (date-today) (- 0 n))
        (food/dates-back (- n 1))))))

(define food/day-calories
  (lambda
    (day)
    (food/sum-calories
      (find-rows (quote food-log) "date" day))))

(define food/day-summary
  (lambda
    (date)
    (let*
      ((dtype (food/day-type date))
        (tgt (food/target dtype))
        (tot (food/day-total date))
        (tot-cal (food/safe-num (dict-get tot "calories")))
        (tot-pro (food/safe-num (dict-get tot "protein")))
        (tot-car (food/safe-num (dict-get tot "carbs")))
        (tot-fat (food/safe-num (dict-get tot "fat")))
        (tgt-cal (food/safe-num (dict-get tgt "calories")))
        (tgt-pro (food/safe-num (dict-get tgt "protein")))
        (tgt-car (food/safe-num (dict-get tgt "carbs")))
        (tgt-fat (food/safe-num (dict-get tgt "fat"))))
      (dict-set*
        "{}"
        (list
          "date"
          (as-str date)
          "day-type"
          dtype
          "calories"
          (as-str tot-cal)
          "calories-target"
          (as-str tgt-cal)
          "calories-remaining"
          (as-str (- tgt-cal tot-cal))
          "protein"
          (as-str tot-pro)
          "protein-target"
          (as-str tgt-pro)
          "protein-remaining"
          (as-str (- tgt-pro tot-pro))
          "carbs"
          (as-str tot-car)
          "carbs-target"
          (as-str tgt-car)
          "carbs-remaining"
          (as-str (- tgt-car tot-car))
          "fat"
          (as-str tot-fat)
          "fat-target"
          (as-str tgt-fat)
          "fat-remaining"
          (as-str (- tgt-fat tot-fat))
          "entries"
          (dict-get tot "entries"))))))

(define food/day-total
  (lambda
    (date)
    (let*
      ((es (food/entries date))
        (cal (food/sum-field es "calories"))
        (pro (food/sum-field es "protein"))
        (car (food/sum-field es "carbs"))
        (fat (food/sum-field es "fat")))
      (dict-set*
        "{}"
        (list
          "date"
```

```
          (as-str date)
          "calories"
          (as-str cal)
          "protein"
          (as-str pro)
          "carbs"
          (as-str car)
          "fat"
          (as-str fat)
          "entries"
          (as-str (length es)))))))

(define food/day-type
  (lambda
    (date)
    (let
      ((row (find-rows (quote food-day) "date" (as-str date))))
      (if
        (null? row)
        "off"
        (as-str (dict-get (cdr (car row)) "day-type"))))))

(define food/delete
  (lambda (id) (delete (quote food-log) id)))

(define food/entries
  (lambda
    (date)
    (find-rows (quote food-log) "date" (as-str date))))

(define food/entry-li
  (lambda
    (row)
    (html/li
      ""
      (html/join
        (list
          (html/strong (food/name-of row))
          (html/text
            (string-append " - " (as-str (food/cal-of row)) " cal")))))))

(define food/entry-str
  (lambda
    (entry field)
    (as-str (dict-get (cdr entry) field))))

(define food/entry-val
  (lambda
    (entry field)
    (food/safe-num (dict-get (cdr entry) field))))

(define food/field
  (lambda
    (fields key)
    (let
      ((v (dict-get fields key)))
      (if (null? v) "" (as-str v)))))

(define food/fields
  (lambda
    (food calories date)
    (dict-set*
      "{}"
      (list
        "food"
        food
        "calories"
        (as-str calories)
        "date"
        date
        "author"
        (as-str (whoami))))))

```

```
(define food/foldl
  (lambda
    (f init xs)
    (if
      (null? xs)
      init
      (food/foldl f (f init (car xs)) (cdr xs)))))

(define food/form
  (lambda
    ()
    (html/div
      (html/attr "class" "card")
      (html/join
        (list
          (html/h2 "Log food")
          (html/form
            "POST"
            "/food"
            ""
            (html/join
              (list
                (html/text-input
                  "food"
                  (html/attr "placeholder" "food name"))
                (html/text-input
                  "calories"
                  (html/attr "placeholder" "cal"))
                (html/button (html/attr "type" "submit") "Log")))))))))

(define food/log
  (lambda
    (date
      meal
      food
      calories
      protein
      carbs
      fat
      notes)
    (create
      (quote food-log)
      (dict-set*
        "{}"
        (list
          "date"
          date
          "meal"
          meal
          "food"
          food
          "calories"
          (as-str calories)
          "protein"
          (as-str protein)
          "carbs"
          (as-str carbs)
          "fat"
          (as-str fat)
          "notes"
          notes
          "author"
          (as-str (whoami)))))))

(define food/name-of
  (lambda (row) (food/field (cdr row) "food")))

(define food/page
  (lambda
    ()
    (if
      (string-eq (request-method) "POST")
      (let
```

```
        ((fv (request-param "food"))
          (cv (request-param "calories")))
        (seq
          (food/log (as-str fv) (parse-int (as-str cv)))
          (redirect "/food")))
      (let*
        ((entries (food/today))
          (total (food/sum-calories entries)))
        (html/page
          (list
            (html/h1 "Food Log")
            (food/form)
            (html/div
              (html/attr "class" "card")
              (html/join
                (list
                  (html/h2
                    (string-append "Today - " (as-str total) " cal"))
                  (html/ul
                    ""
                    (html/join (list/map food/entry-li entries))))))))))

(define food/quick
  (lambda
    (date meal food calories)
    (food/log
      date
      meal
      food
      calories
      ""
      ""
      ""
      "")))

(define food/safe-num
  (lambda (s) (parse-float (as-str s))))

(define food/set-day
  (lambda
    (date day-type)
    (let
      ((existing
        (find-rows (quote food-day) "date" (as-str date))))
      (if
        (null? existing)
        (create
          (quote food-day)
          (dict-set*
            "{}"
            (list
              "date"
              (as-str date)
              "day-type"
              (as-str day-type)
              "author"
              (as-str (whoami)))))
        (update
          (quote food-day)
          (car (car existing))
          (dict-set*
            "{}"
            (list
              "date"
              (as-str date)
              "day-type"
              (as-str day-type)
              "author"
              (as-str (whoami)))))))))

(define food/set-target
  (lambda
    (day-type calories protein carbs fat)
```

```
    (let
      ((existing
        (find-rows
          (quote food-target)
          "day-type"
          (as-str day-type))))
      (if
        (null? existing)
        (create
          (quote food-target)
          (dict-set*
            "{}"
            (list
              "day-type"
              (as-str day-type)
              "calories"
              (as-str calories)
              "protein"
              (as-str protein)
              "carbs"
              (as-str carbs)
              "fat"
              (as-str fat)
              "author"
              (as-str (whoami)))))
        (update
          (quote food-target)
          (car (car existing))
          (dict-set*
            "{}"
            (list
              "day-type"
              (as-str day-type)
              "calories"
              (as-str calories)
              "protein"
              (as-str protein)
              "carbs"
              (as-str carbs)
              "fat"
              (as-str fat)
              "author"
              (as-str (whoami))))))))

(define food/sum-calories
  (lambda
    (rows)
    (list/foldl
      (lambda (acc row) (+ acc (food/cal-of row)))
      0
      rows)))

(define food/sum-field
  (lambda
    (entries field)
    (food/foldl
      (lambda (acc e) (+ acc (food/entry-val e field)))
      0
      entries)))

(define food/target
  (lambda
    (day-type)
    (let
      ((row
        (find-rows
          (quote food-target)
          "day-type"
          (as-str day-type))))
      (if (null? row) "{}" (cdr (car row))))))

(define food/templates
  (lambda
```

```
    (day-type)
    (find-rows
      (quote food-template)
      "day-type"
      (as-str day-type))))

(define food/test
  (lambda () "hello from food"))

(define food/test2
  (lambda () (dict-set* "{}" (list "a" "1" "b" "2"))))

(define food/today
  (lambda
    ()
    (find-rows (quote food-log) "date" (date-today))))

(define food/today-total
  (lambda () (food/sum-calories (food/today))))

(define food/week-dates
  (lambda () (food/dates-back 6)))

(define food/week-summary
  (lambda
    ()
    (food/foldl
      (lambda
        (acc date)
        (cons (cons date (food/day-summary date)) acc))
      ()
      (food/week-dates))))
```

168 libs/forge/lib.pp

```
;; package forge -- executable spec graphs as habitat data.
;; Spec: borge/forge.borg. Load: (load-library "forge") or load-string.
;; Depends on: list (foldl/filter/map), notes (log/issues), and kernel prims
;; (read-string, forge-effects, deftype, create/update/get/rows, dict-*).
;;
;; The borge discipline as a living graph: a spec is a tree of typed nodes,
;; each node carries a machine-checkable victory condition, LLM drafters fill
;; leaves bottom-up along a computed frontier, mechanical checkers flag
;; anomalies, and the human approves each level before the descent continues.
;;
;; Exports are prefix-convention (forge/*). See borge/forge.borg for the
;; design rationale: victory conditions are the artifact, code is a cache;
;; planning is a computation (free-vars), not an LLM judgment; mechanical
;; flags run before any human is asked to judge.

;; ============================================================
;; TYPES (graph-model)
;; ============================================================
;; Placement: data.shared.forge-* (outside branch lenses) per one-tree
;; branching-interaction -- same argument as jobs: the loop (host identity)
;; mutates node rows regardless of who drafted.

(deftype 'forge-graph
  'name 'root_path 'status 'created_by 'created_at 'max_depth)

(deftype 'forge-node
  'graph 'path 'kind 'title 'body 'verify 'effects 'status
  'constraints 'parent 'children 'branch 'author)

(deftype 'forge-flag
  'node_path 'check 'detail 'resolved 'resolution)

;; ============================================================
;; SMALL HELPERS (pure)
;; ============================================================

(define forge/member?
  (lambda (x xs)
    (if (null? xs)
        #f
        (if (string-eq (as-str (car xs)) (as-str x))
            #t
            (forge/member? x (cdr xs)))))
  "String membership over a list.")

(define forge/append
  (lambda (a b)
    (if (null? a) b (cons (car a) (forge/append (cdr a) b))))
  "Concatenate two lists.")

(define forge/str-join
  (lambda (sep xs)
    (if (null? xs)
        ""
        (if (null? (cdr xs))
            (as-str (car xs))
            (string-append (as-str (car xs))
              (string-append sep (forge/str-join sep (cdr xs)))))))
  "Join a list of strings with sep.")

;; field access on a data row (id . fields-json-string) pair.
;; rows returns (id . fields) pairs where fields is a JSON string;
;; get returns the bare fields JSON string. Both normalize here.
(define forge/field
  (lambda (fields-json key)
    (dict-get fields-json key))
  "dict-get over a fields JSON string; () when absent.")

(define forge/field-str
```

```
  (lambda (fields-json key)
    (let ((v (forge/field fields-json key)))
      (if (null? v) "" (as-str v))))
  "Field as a string; \"\" when absent.")

;; ============================================================
;; GRAPH CRUD (graph-model)
;; ============================================================

(define forge/new-graph
  (lambda (name root-title max-depth)
    (let ((gid (create 'forge-graph
                 (string-append
                   "{\"name\":\"" name
                   "\",\"root_path\":\"" name
                   "\",\"status\":\"planning"
                   "\",\"created_by\":\"" (whoami)
                   "\",\"created_at\":\"" (as-str (now-ms))
                   "\",\"max_depth\":\"" (as-str max-depth)
                   "\"}"))))
      gid))
  "Create a graph row; returns the new id. Root node is added separately
   via forge/add-node so the caller controls the root requirement.")

(define forge/graph-by-name
  (lambda (name)
    (let ((all (rows 'forge-graph 100)))
      (forge/find-row
        (lambda (pair) (string-eq (forge/field-str (cdr pair) "name") name))
        all)))
  "Find a forge-graph row by its name field; () when absent.")

(define forge/find-row
  (lambda (pred xs)
    (if (null? xs)
        (list)
        (if (pred (car xs))
            (car xs)
            (forge/find-row pred (cdr xs)))))
  "First element of xs satisfying pred, or ().")

;; ---- nodes ----

(define forge/node-by-path
  (lambda (graph path)
    (forge/find-row
      (lambda (pair)
        (let ((f (cdr pair)))
          (if (string-eq (forge/field-str f "graph") graph)
              (string-eq (forge/field-str f "path") path)
              #f)))
      (rows 'forge-node 500)))
  "Find a node (id . fields) pair by graph+path; () when absent.")

(define forge/node-by-path-in
  (lambda (nodes path)
    (forge/find-row
      (lambda (pair) (string-eq (forge/field-str (cdr pair) "path") path))
      nodes))
  "Find a node in a PRE-FETCHED nodes list (the graph's own nodes, not
   all 500 rows). O(nodes-in-graph) vs O(500) for node-by-path. Use this
   in any loop that touches many nodes (flaggers, frontier, scans) to
   avoid re-scanning the whole forge-node table per lookup.")

(define forge/node-in-graph?
  (lambda (graph node-path)
    (let ((g (string-append graph "/")))
      (or (string-eq node-path graph)
          (string-starts-with? node-path g))))
  "True when node-path belongs to graph -- by path PREFIX, no row scan.
   Paths are graph/scoped/... so node-path starts with graph/ (or equals
   graph for the root). Replaces path-in-graph? (which called node-by-path
   per flag, O(flags?nodes)).")
```

```
143
144 (define forge/node-exists?
145   (lambda (graph path)
146     (not (null? (forge/node-by-path graph path))))
147   "True when a node row exists for graph+path.")
148
149 ;; children is a denormalized JSON list of child paths maintained in the
150 ;; same mutation as the child's parent link (one logical write).
151 (define forge/add-node
152   (lambda (graph path kind title parent)
153     (if (forge/node-exists? graph path)
154         (forge/node-id graph path)
155         (let ((id (create 'forge-node
156                     (string-append
157                       "{\"graph\":\"" graph
158                       "\",\"path\":\"" path
159                       "\",\"kind\":\"" kind
160                       "\",\"title\":\"" title
161                       "\",\"body\":\""
162                       "\",\"verify\":\""
163                       "\",\"effects\":\"pure"
164                       "\",\"status\":\"planned"
165                       "\",\"constraints\":\"[]"
166                       "\",\"parent\":\"" parent
167                       "\",\"children\":\"[]"
168                       "\",\"branch\":\"" (current-branch)
169                       "\",\"author\":\"" (whoami)
170                       "\"}"))))
171           (if (string-eq parent "")
172               id
173               (forge/link-child graph parent path)))))
174   "Create a planned node and link it under parent (when parent non-empty).")
175
176 (define forge/node-id
177   (lambda (graph path)
178     (let ((pair (forge/node-by-path graph path)))
179       (if (null? pair) "" (car pair))))
180   "Id of the node at graph+path, or \"\".")
181
182 (define forge/link-child
183   (lambda (graph parent-path child-path)
184     (let ((pair (forge/node-by-path graph parent-path)))
185       (if (null? pair)
186           ""
187           (let ((f (cdr pair)))
188             (let ((kids (json-parse (forge/field-str f "children"))))
189               (if (forge/member? child-path kids)
190                   (car pair)
191                   (seq
192                     (update 'forge-node (car pair)
193                       (dict-set f "children"
194                         (json-emit (forge/append kids (list child-path)))))
195                     (car pair))))))))
196   "Append child-path to parent's children index (idempotent).")
197
198 (define forge/children-of
199   (lambda (graph path)
200     (let ((pair (forge/node-by-path graph path)))
201       (if (null? pair)
202           (list)
203           (json-parse (forge/field-str (cdr pair) "children"))))))
204   "List of child paths for a node.")
205
206 (define forge/nodes-of-graph
207   (lambda (graph)
208     (list/filter
209       (lambda (pair)
210         (string-eq (forge/field-str (cdr pair) "graph") graph))
211       (rows 'forge-node 500)))
212   "All node (id . fields) pairs belonging to a graph.")
213
214 (define forge/node-depth
215   (lambda (path)
```

```
    (forge/count-slashes path 0 0))
  "Depth of a node = number of '/' separators in its path. Root has 0.")

(define forge/count-slashes
  (lambda (s i n)
    (if (>= i (string-length s))
        n
        (forge/count-slashes s (+ i 1)
          (if (string-eq (substring s i 1) "/") (+ n 1) n))))
  "Count '/' characters in s (recursive over index).")

;; ============================================================
;; STATUS TRANSITIONS (graph-model)
;; ============================================================

(define forge/set-status
  (lambda (graph path status)
    (let ((pair (forge/node-by-path graph path)))
      (if (null? pair)
          ""
          (update 'forge-node (car pair)
            (dict-set (cdr pair) "status" status)))))
  "Set a node's status; \"\" when the node is absent.")

(define forge/set-body-verify
  (lambda (graph path body verify)
    (let ((pair (forge/node-by-path graph path)))
      (if (null? pair)
          ""
          (let ((f (cdr pair)))
            (update 'forge-node (car pair)
              (dict-set (dict-set f "body" body) "verify" verify))))))
  "Write a drafted body + verify source onto a node.")

(define forge/add-constraint
  (lambda (graph path note)
    (let ((pair (forge/node-by-path graph path)))
      (if (null? pair)
          ""
          (let ((f (cdr pair)))
            (let ((cs (json-parse (forge/field-str f "constraints"))))
              (seq
                (update 'forge-node (car pair)
                  (dict-set f "constraints"
                    (json-emit (forge/append cs (list note)))))
                (car pair)))))))
  "Append a reroll-with-note constraint; the next draft prompt includes
    it.")

;; ============================================================
;; FLAGS (graph-model writes; mechanical-flags computes)
;; ============================================================

(define forge/flag
  (lambda (node-path check detail)
    (create 'forge-flag
      (string-append
        "{\"node_path\":\"" node-path
        "\",\"check\":\"" check
        "\",\"detail\":\"" detail
        "\",\"resolved\":false"
        ",\"resolution\":\"\""
        "}")))
  "Raise a flag on a node. check ?
    capability|victory|dangling|cycle|orphan|size|judgment.")

(define forge/flags-for
  (lambda (node-path)
    (list/filter
      (lambda (pair)
        (string-eq (forge/field-str (cdr pair) "node_path") node-path))
      (rows 'forge-flag 500)))
  "All flag rows for a node path.")
```

```
287
288 (define forge/open-flags
289   (lambda (graph)
290     (list/filter
291       (lambda (pair)
292         (let ((f (cdr pair)))
293           (if (string-eq (forge/field-str f "resolved") "true")
294               #f
295               (forge/node-in-graph? graph (forge/field-str f
    "node_path")))))
296       (rows 'forge-flag 500)))
297   "Unresolved flags whose node_path belongs to the graph. Uses the path
298    prefix check (node-in-graph?) -- O(flags) with cheap string ops, no
299    per-flag node row scan.")
300
301 (define forge/path-in-graph?
302   (lambda (graph node-path)
303     (forge/node-in-graph? graph node-path))
304   "Legacy alias for node-in-graph? (prefix check, no row scan).")
305
306 (define forge/resolve-flag
307   (lambda (flag-id resolution)
308     (let ((row (get 'forge-flag flag-id)))
309       (if (null? row)
310           ""
311           (update 'forge-flag flag-id
312             (dict-set (dict-set row "resolved" "true")
313                       "resolution" resolution)))))
314   "Close a flag with a resolution note.")
315
316 ;; ============================================================
317 ;; THE FRONTIER (planning is a computation, not a judgment)
318 ;; ============================================================
319 ;; free-vars walks a parsed body and collects symbols not bound
320 ;; by any enclosing lambda/let. unresolved filters those against
321 ;; image bindings (function-doc oracle) + sibling node names.
322 ;; frontier = planned nodes whose unresolved set is empty.
323
324 ;; ---- set helpers on lists of strings ----
325
326 (define forge/union
327   (lambda (a b)
328     (forge/union-into a b (list)))
329   "Set union of two lists of strings (dedup, order-preserving).")
330
331 (define forge/union-into
332   (lambda (a b acc)
333     (if (null? a)
334         (forge/finish-union b acc)
335         (if (forge/member? (car a) acc)
336             (forge/union-into (cdr a) b acc)
337             (forge/union-into (cdr a) b (cons (car a) acc)))))
338   "Fold a into acc, then fold b.")
339
340 (define forge/finish-union
341   (lambda (b acc)
342     (if (null? b)
343         (forge/reverse acc)
344         (if (forge/member? (car b) acc)
345             (forge/finish-union (cdr b) acc)
346             (forge/finish-union (cdr b) (cons (car b) acc)))))
347   "Fold remaining b, then reverse.")
348
349 (define forge/reverse
350   (lambda (xs)
351     (forge/reverse-into xs (list)))
352   "Reverse a list.")
353
354 (define forge/reverse-into
355   (lambda (xs acc)
356     (if (null? xs)
357         acc
358         (forge/reverse-into (cdr xs) (cons (car xs) acc))))
```

```
  "Tail-recursive reverse.")

(define forge/difference
  (lambda (xs exclude)
    (if (null? xs)
        (list)
        (if (forge/member? (car xs) exclude)
            (forge/difference (cdr xs) exclude)
            (cons (car xs) (forge/difference (cdr xs) exclude)))))
  "xs with any element in exclude removed.")

;; ---- special forms (excluded from free-var collection) ----

(define forge/constant?
  (lambda (name)
    (forge/member? name (list "nil" "#t" "#f" "true" "false" "else")))
  "True for literal constants that are not free variables.")

(define forge/special-form?
  (lambda (name)
    (forge/member? name (list
      "lambda" "let" "letrec" "if" "define" "quote" "seq" "begin"
      "match" "guard" "error" "else" "=>" "cond")))
  "Is name a Nopales special form / keyword?")

;; ---- image binding oracle ----

(define forge/image-bound?
  (lambda (name)
    (if (forge/special-form? name)
        #t
        (not (null? (function-doc name)))))
  "True when name resolves to a kernel prim, special form, or
   user binding. function-doc covers prims + user defs; special
   forms return () from function-doc so they are handled here.")

;; ---- bound name collection from lambda params / let bindings ----

(define forge/lambda-params
  (lambda (params)
    (if (null? params)
        (list)
        (cons (as-str (car params)) (forge/lambda-params (cdr params)))))
  "List of param name strings from a lambda formals list.")

(define forge/let-names
  (lambda (bindings)
    (if (null? bindings)
        (list)
        (let ((b (car bindings)))
          (let ((name (car b)))
            (cons (as-str name) (forge/let-names (cdr bindings)))))))
  "List of bound name strings from a let bindings list.")

;; ---- free-vars over a list of expressions ----

(define forge/free-vars-list
  (lambda (exprs bound)
    (if (null? exprs)
        (list)
        (forge/union
          (forge/free-vars (car exprs) bound)
          (forge/free-vars-list (cdr exprs) bound))))
  "Union of free-vars over each expr in a list.")

;; ---- free-vars over one expression ----

(define forge/free-vars
  (lambda (expr bound)
    (cond
      ((null? expr) (list))
      ((not (pair? expr))
       (if (string-eq (type-of expr) "symbol")
```

```
          (let ((name (as-str expr)))
            (if (or (forge/member? name bound)
                    (forge/constant? name))
                (list)
                (list name)))
          (list)))
      ((string-eq (as-str (car expr)) "quote") (list))
      ((string-eq (as-str (car expr)) "lambda")
       (forge/free-vars (caddr expr)
         (forge/append bound (forge/lambda-params (cadr expr)))))
      ((string-eq (as-str (car expr)) "let")
       (forge/free-vars (caddr expr)
         (forge/append bound (forge/let-names (cadr expr)))))
      ((string-eq (as-str (car expr)) "define") (list))
      ((string-eq (as-str (car expr)) "if")
       (forge/union
         (forge/free-vars (cadr expr) bound)
         (forge/union
           (forge/free-vars (caddr expr) bound)
           (forge/free-vars (car (cdr (cdr (cdr expr)))) bound))))
      (#t
       (forge/free-vars-list expr bound))))
  "Free variables of a parsed expression: symbols not bound by an
   enclosing lambda/let and not a special form. quote is a literal;
   define is top-level (handled by the caller).")

;; ---- unresolved: free-vars that reference nothing real ----

(define forge/sibling-names
  (lambda (graph path)
    (let ((siblings (forge/children-of graph (forge/parent-of path))))
      (forge/node-titles->names graph siblings)))
  "Names offered by sibling nodes (their label, derived from path).")

(define forge/parent-of
  (lambda (path)
    (forge/parent-of-acc path ""))
  "Parent path: drop the last /segment.")

(define forge/parent-of-acc
  (lambda (path acc)
    (if (string-eq path "")
        acc
        (forge/parent-of-acc (forge/tail-str path)
          (if (string-eq (forge/head-char path) "/") path acc))))
  "Scan to last '/', return everything before it (or \"\" for root).")

(define forge/node-titles->names
  (lambda (graph paths)
    (if (null? paths)
        (list)
        (let ((p (car paths)))
          (let ((node (forge/node-by-path graph p)))
            (if (null? node)
                (forge/node-titles->names graph (cdr paths))
                (cons (forge/field-str (cdr node) "title")
                      (forge/node-titles->names graph (cdr paths)))))))))
  "Titles of the nodes at the given paths.")

(define forge/unresolved
  (lambda (graph path body-src)
    (let ((fvs (forge/free-vars (forge/first-form body-src) (list))))
      (forge/difference fvs
        (forge/append (list) (forge/image-bound-names fvs)))))
  "Free vars of body that resolve to nothing: not a prim, not a
   binding, not a special form. These are the stub-children candidates.
   Uses first-form to extract the lambda robustly (the drafter may
   append prose; read-string would error on trailing input).")

(define forge/image-bound-names
  (lambda (names)
    (if (null? names)
        (list)
```

```
        (if (forge/image-bound? (car names))
            (cons (car names) (forge/image-bound-names (cdr names)))
            (forge/image-bound-names (cdr names)))))
  "Subset of names that ARE bound in the image (prims + defs + specials).")

(define forge/frontier
  (lambda (graph)
    (forge/frontier-scan graph (forge/nodes-of-graph graph) (list)))
  "Planned nodes whose unresolved set is empty -- leaves whose
   dependencies all exist. Bottom-up build order.")

(define forge/frontier-scan
  (lambda (graph nodes acc)
    (if (null? nodes)
        (forge/reverse acc)
        (let ((pair (car nodes)))
          (let ((f (cdr pair)))
            (let ((status (forge/field-str f "status"))
                  (path (forge/field-str f "path"))
                  (kind (forge/field-str f "kind"))
                  (body (forge/field-str f "body")))
              (let ((ready
                      (if (string-eq status "planned")
                          (if (string-eq kind "requirement")
                              #f
                              (if (string-eq body "")
                                  #t
                                  (forge/ready? graph path body)))
                          #f)))
                (forge/frontier-scan graph (cdr nodes)
                  (if ready (cons path acc) acc))))))))
  "Scan nodes; a planned node with empty body is trivially ready
   (no deps), otherwise check unresolved against the image.
   Requirements are never drafted (human-authored) so they are
   excluded from the frontier; the human approves them at the
   level gate, then propose-descent stubs their children.")

(define forge/ready?
  (lambda (graph path body)
    (let ((unres (forge/unresolved graph path body)))
      (null? unres)))
  "True when a drafted body has no unresolved references.")

;; ============================================================
;; MECHANICAL FLAGS (checks that run before the human wakes)
;; ============================================================
;; Each flagger is a small function over the graph. They run after
;; every draft and after every level-descent proposal. The approval
;; surface shows FLAGS, not nodes.

;; ---- capability escalation: child effects exceed parent declared ----

(define forge/flag-capability
  (lambda (nodes path)
    (let ((node (forge/node-by-path-in nodes path)))
      (if (null? node) (list)
      (let ((f (cdr node)))
        (let ((parent-path (forge/field-str f "parent")))
          (if (string-eq parent-path "") (list)
          (let ((parent (forge/node-by-path-in nodes parent-path)))
            (if (null? parent) (list)
            (let ((body (forge/field-str f "body"))
                  (declared (forge/field-str (cdr parent) "effects")))
              (if (string-eq body "") (list)
              (let ((actual (forge-effects body)))
                (if (forge/effect-escalates? declared actual)
                    (list (forge/make-flag-spec path "capability"
                            (string-append "declared " declared
                              " but body is " actual)))
                    (list))))))))))))
  "Raise a capability flag when a child's inferred effect exceeds the
   parent's declared badge. Takes the pre-fetched nodes list.")

```

```
(define forge/effect-escalates?
  (lambda (declared actual)
    (not (forge/effect-subset? declared actual)))
  "True when actual is NOT a subset of declared -- i.e. the child does
   more than the parent promised.")

(define forge/effect-subset?
  (lambda (declared actual)
    (cond
      ((string-eq actual "pure") #t)
      ((string-eq declared "pure") #f)
      ((string-eq declared actual) #t)
      ((string-eq declared "db-read")
       (string-eq actual "db-write"))
      (#t #f)))
  "Coarse effect lattice: pure < db-read < db-write < branch/publish/ai/net.
   A read-only parent allows a read child; a pure parent forbids any
    effect.")

;; ---- missing victory condition: empty verify source ----

(define forge/flag-victory
  (lambda (nodes path)
    (let ((node (forge/node-by-path-in nodes path)))
      (if (null? node) (list)
      (let ((f (cdr node)))
        (let ((verify (forge/field-str f "verify"))
              (kind (forge/field-str f "kind")))
          (if (string-eq kind "requirement")
              (list)
              (if (string-eq verify "")
                  (list (forge/make-flag-spec path "victory"
                          "node has no verify stanza -- unapprovable"))
                  (list))))))))
  "Flag nodes (non-requirement) with no verify source.")

;; ---- dangling refs: free-vars referencing nothing ----

(define forge/flag-dangling
  (lambda (nodes path)
    (let ((node (forge/node-by-path-in nodes path)))
      (if (null? node) (list)
      (let ((f (cdr node)))
        (let ((body (forge/field-str f "body")))
          (if (string-eq body "") (list)
          (let ((unres (forge/unresolved "" path body)))
            (if (null? unres) (list)
            (list (forge/make-flag-spec path "dangling"
                    (string-append "unresolved references: "
                      (forge/str-join ", " unres))))))))))))
  "Flag drafted bodies with references that resolve to nothing.")

;; ---- orphan: node traces to no requirement ancestor ----

(define forge/flag-orphan
  (lambda (nodes path)
    (if (forge/has-requirement-ancestor? nodes path)
        (list)
        (list (forge/make-flag-spec path "orphan"
                "traces to no requirement ancestor"))))
  "Flag nodes with no requirement node in their ancestor chain.")

(define forge/has-requirement-ancestor?
  (lambda (nodes path)
    (let ((node (forge/node-by-path-in nodes path)))
      (if (null? node) #t
      (let ((f (cdr node)))
        (let ((kind (forge/field-str f "kind"))
              (parent (forge/field-str f "parent")))
          (cond
            ((string-eq kind "requirement") #t)
            ((string-eq parent "") #f)
            (#t (forge/has-requirement-ancestor? nodes parent)))))))))
```

```
  "Walk parent chain; true when a requirement kind node is found.")

;; ---- size heuristic: body too big to review in one glance ----

(define forge/flag-size
  (lambda (nodes path)
    (let ((node (forge/node-by-path-in nodes path)))
      (if (null? node) (list)
      (let ((f (cdr node)))
        (let ((body (forge/field-str f "body")))
          (if (forge/too-big? body 200)
              (list (forge/make-flag-spec path "size"
                      "body exceeds review-in-one-glance threshold"))
              (list)))))))
  "Flag bodies longer than the threshold (default ~200 chars).")

(define forge/too-big?
  (lambda (s limit)
    (> (forge/strlen s) limit))
  "True when string length exceeds limit.")

(define forge/strlen
  (lambda (s)
    (string-length s))
  "String length (wrapper for readability).")

;; ---- run all mechanical flaggers on a node ----

(define forge/mechanical-flags
  (lambda (nodes path)
    (forge/append
      (forge/append
        (forge/append
          (forge/flag-capability nodes path)
          (forge/flag-victory nodes path))
        (forge/append
          (forge/flag-dangling nodes path)
          (forge/flag-orphan nodes path)))
      (forge/flag-size nodes path)))
  "All mechanical flags for one node. Returns a list of flag-spec pairs:
   (check . detail). Takes the pre-fetched nodes list so the flaggers
   search the graph's own nodes, not all 500 rows.")

(define forge/make-flag-spec
  (lambda (path check detail)
    (cons check detail))
  "A flag spec is (check . detail); raised as a row by forge/raise-flags.")

(define forge/raise-flags
  (lambda (graph path specs)
    (if (null? specs) (list)
    (let ((spec (car specs)))
      (seq
        (forge/flag path (car spec) (cdr spec))
        (forge/raise-flags graph path (cdr specs))))))
  "Raise each flag-spec as a forge-flag row on the node.")

(define forge/scan-mechanical-flags
  (lambda (graph)
    (forge/scan-flags-nodes graph (forge/nodes-of-graph graph)))
  "Run mechanical flaggers over every node in the graph and raise flags.")

(define forge/scan-flags-nodes
  (lambda (graph nodes)
    (if (null? nodes) (list)
    (let ((pair (car nodes)))
      (let ((path (forge/field-str (cdr pair) "path")))
        (seq
          (forge/raise-flags graph path
            (forge/mechanical-flags nodes path))
          (forge/scan-flags-nodes graph (cdr nodes))))))))
  "Scan all nodes, raise their mechanical flags.")

```

```
;; ============================================================
;; THE LOOP (the interpreter that replaces plan/make/review)
;; ============================================================
;; forge/step is the whole state machine. It is deliberately boring:
;;   1. open judgment flags ? do nothing; BLOCKED on the human
;;   2. current level has unapproved nodes ? run flags, present, stop
;;   3. frontier non-empty ? pick one, draft it (LLM), write, flag
;;   4. else ? propose descent (stub children), stop (human gate)

;; ---- depth helpers ----

(define forge/levels
  (lambda (graph)
    (forge/levels-scan (forge/nodes-of-graph graph) (list)))
  "Distinct depths present in the graph, ascending.")

(define forge/levels-scan
  (lambda (nodes acc)
    (if (null? nodes)
        (forge/sort-numbers acc)
        (let ((d (forge/node-depth (forge/field-str (cdr (car nodes)) "path"))))
          (forge/levels-scan (cdr nodes)
            (if (forge/member? d acc) acc (cons d acc)))))))
  "Collect distinct depths.")

(define forge/sort-numbers
  (lambda (xs)
    (if (null? xs) (list)
    (forge/insert-sorted (car xs) (forge/sort-numbers (cdr xs)))))
  "Insertion sort for small lists of numbers.")

(define forge/insert-sorted
  (lambda (x xs)
    (if (null? xs) (list x)
    (if (<= x (car xs))
        (cons x xs)
        (cons (car xs) (forge/insert-sorted x (cdr xs))))))
  "Insert x into sorted xs.")

(define forge/nodes-at-depth
  (lambda (graph d)
    (list/filter
      (lambda (pair)
        (string-eq (as-str (forge/node-depth (forge/field-str (cdr pair) "path")))
                   (as-str d)))
      (forge/nodes-of-graph graph)))
  "All node pairs at a given depth.")

(define forge/status>=?
  (lambda (status target)
    (forge/status-rank status >= (forge/status-rank target 0)))
  "Status ordering: planned < drafted < verified < approved < implemented.")

(define forge/status-rank
  (lambda (status fallback)
    (cond
      ((string-eq status "planned") 0)
      ((string-eq status "drafted") 1)
      ((string-eq status "verified") 2)
      ((string-eq status "approved") 3)
      ((string-eq status "implemented") 4)
      (#t fallback)))
  "Numeric rank of a status string.")

;; ---- the draft prompt (context assembly) ----

(define forge/draft-prompt
  (lambda (graph path)
    (let ((node (forge/node-by-path graph path)))
      (if (null? node) ""
      (let ((f (cdr node)))
```

```
        (let ((title (forge/field-str f "title"))
              (kind (forge/field-str f "kind"))
              (target (forge/field-str f "target"))
              (test-fn (forge/field-str f "test-fn"))
              (constraints (json-parse (forge/field-str f "constraints"))))
          (if (string-eq target "")
            (string-append
              "You are drafting a Nopales (Lisp-1) function for a Pricklypear spec graph.\n"
              "Node: " path "\n"
              "Kind: " kind "\n"
              "Purpose: " title "\n"
              "Output TWO s-expressions on separate lines: (1) the (lambda ...) body, (2) a verify expression using `fn` that returns #t if correct.\n"
              "Available prims: + - * = < > list car cdr cons null? string-append str\n"
              "  get create update delete rows dict-get dict-set render-html\n"
              (forge/constraints-text constraints))
            (string-append
              "You are MODIFYING an existing Nopales (Lisp-1) function on a Pricklypear spec graph.\n"
              "Node: " path "\n"
              "Kind: " kind "\n"
              "Purpose: " title "\n"
              "Target binding: " target "\n"
              "CURRENT IMPLEMENTATION (you are changing this -- produce the FULL new body, not a diff):\n"
              (forge/safe-source target) "\n\n"
              "Output TWO s-expressions on separate lines: (1) the FULL new (lambda ...) body, (2) a verify expression using `fn` that returns #t if correct.\n"
              (forge/test-fn-prompt test-fn)
              "Available prims: + - * = < > list car cdr cons null? string-append str\n"
              "  get create update delete rows dict-get dict-set render-html\n"
              (forge/constraints-text constraints)))))))
  "Assemble the draft prompt from a node's context + accumulated constraints.
   Greenfield when target is empty; modify-existing (includes the target's current source) when target is set.")

(define forge/constraints-text
  (lambda (cs)
    (if (null? cs) ""
    (string-append "\nConstraints from prior rerolls:\n"
      (forge/str-join "\n" cs) "\n")))
  "Render constraints as prompt text.")

;; ---- draft a node (agentic: eval-and-iterate, then harvest last form) ----

(define forge/drafter-seed
  (string-append
    "You are a Nopales (Lisp-1) drafter for a Pricklypear spec graph. "
    "Your job: produce ONE working (lambda ...) body for a single node.\n"
    "\n"
    "PROCESS (mandatory):\n"
    "1. Use eval_expr to TEST your draft. Call it like: "
    "((lambda (a b) (+ a b)) 1 2) -- it returns the value or an error.\n"
    "2. If eval_expr errors, READ the error, FIX the lambda, and re-eval. "
    "Iterate until it produces the CORRECT RESULT with realistic args -- "
    "not just no-error. Check that the returned value is what the function "
    "should actually return for those inputs.\n"
    "3. Do NOT use install_function, register_route, or smoke_route -- "
    "forge manages the body in a graph node, NOT on the branch. "
    "install_function is forbidden; only eval_expr is allowed for testing.\n"
    "4. Your FINAL message must contain EXACTLY TWO s-expression forms,\n"
    "   each on its own line, nothing else:\n"
    "   Line 1: the (lambda ...) body you verified -- copy it verbatim.\n"
```

```
    "   Line 2: a VERIFY expression that ASSERTS THE OUTCOME -- not just\n"
    "   the return type. Use `fn` (bound to your lambda) and assert what\n"
    "   the function should actually produce or change.\n"
    "\n"
    "   GOOD verifies (assert the outcome):\n"
    "     (= (fn 5) 10)                          ; doubling: 5 -> 10\n"
    "     (string-eq (dict-get (fn 'user id) \"name\") \"Alice\")  ; field value\n"
    "     (let* ((id (fn \"x\" \"y\"))\n"
    "            (row (get 'type id)))\n"
    "       (string-eq (dict-get (cdr row) \"title\") \"y\"))      ; row was created with right data\n"
    "     (let* ((id (fn \"x\"))\n"
    "            (_ (fn-remove id)))\n"
    "       (null? (get 'type id)))               ; row is gone after remove\n"
    "   BAD verifies (too weak -- just check types, not correctness):\n"
    "     (string? (fn \"x\" \"y\"))            ; only checks return is a string\n"
    "     (not (null? (fn)))                     ; only checks non-empty\n"
    "\n"
    "   For DB-reading functions, assert the actual field values returned.\n"
    "   For DB-writing functions, read back the created row and assert its fields.\n"
    "   For DB-deleting functions, assert the row is gone after the delete.\n"
    "   If the verify needs a row to exist first, create it inside the verify\n"
    "   with (create ...) before calling fn. Keep verifies SHORT -- one assertion.\n"
    "   The verify MUST return #t only when the function is correct.\n"
    "   Count parens carefully on both lines.\n"
    "\n"
    "NOPALES SYNTAX REMINDERS:\n"
    "  - let takes exactly ONE body form; use (seq a b) for multi-step.\n"
    "  - let* is sequential binding: (let* ((a 1) (b (+ a 1))) b) => 2.\n"
    "  - There is NO named-let (no (let loop ...)); use list/foldl or recursion.\n"
    "  - dict-get returns JSON values as strings; compare with string-eq.\n"
    "  - substring takes (start, length), not (start, end).\n"
    "  - create: (create 'type-name (dict-set* \"{}\" (list \"field1\" val1 \"field2\" val2))) "
    "-> returns an id string. Fields MUST be a JSON dict string built via "
    "dict-set*, NOT an assoc list.\n"
    "  - rows: (rows 'type-name 200) -> list of (id . json-string) pairs; "
    "(cdr row) is the JSON field string; use (dict-get (cdr row) \"field\").\n"
    "  - CRITICAL: prims return error STRINGS (e.g. \"create: bad args\") "
    "instead of throwing. After eval_expr, CHECK the returned value: if it "
    "contains \": bad args\" or \"error\" or starts with the prim name + \":\", "
    "it FAILED -- fix and re-test. A returned string is not always success.\n"
    "\n"
    "AVAILABLE PRIMS: + - * = < > list car cdr cons null? count "
    "string-append str type-of whoami list-bindings dict-get dict-set "
    "dict-set* render-html get create update delete rows as-str string-eq "
    "string-length string-starts-with? request-param request-method "
    "list/map list/filter list/foldl with-transaction")
  "Seed for the forge drafter agent session. Tells the LLM to use eval_expr
     to test-and-iterate, forbids install/route tools, and pins Nopales
     syntax gotchas. The drafter returns TWO forms: the (lambda ...) body,
     then a verify expression that uses `fn` as the bound name.")

(define forge/draft-node
  (lambda (graph path)
    (let* ((prompt (forge/draft-prompt graph path)))
      (guard (lambda (e)
               (seq
                 (forge/flag path "judgment"
                   (string-append "drafter crashed: " (str e)))
```

```
                  (forge/commit-draft graph path "")))
        (let* ((sid (agent-session-new forge/drafter-seed #t))
               (resp (agent-say sid prompt 16))
               (body (forge/first-form resp))
               (verify (forge/second-form resp)))
          (seq
            (agent-session-delete sid)
            (if (forge/drafter-failed? resp)
                (seq
                  (forge/flag path "judgment"
                    (string-append "drafter failed: " resp))
                  (forge/commit-draft graph path ""))
                (if (forge/parse-ok? body)
                    (forge/commit-draft graph path body verify)
                    (seq
                      (forge/flag path "judgment"
                        "drafter body unparseable (missing parens or syntax error)")
                      (forge/commit-draft graph path ""))))))))
  "Agentic draft: spin up a drafter agent session (eval_expr enabled), let it test-and-iterate up to 12 steps, harvest the body (first form) + verify (second form), parse-check the body, then commit-draft. Guard-wrapped: any crash (agent loop, eval, effects) becomes a judgment flag + empty body, not an uncaught exception.")

(define forge/parse-ok?
  (lambda (s)
    (if (string-eq s "")
        #f
        (guard (lambda (e) #f) (seq (read-string s) #t))))
  "True when s parses as valid Lisp. read-string raises on parse error; guard catches it and returns #f. Empty string is #f.")

(define forge/drafter-failed?
  (lambda (s)
    (or (string-starts-with? s "agent error:")
        (string-starts-with? s "agent: max steps exceeded")))
  "True when the agent session errored or exhausted its steps without a clean final turn. (agent-say returns these prefixes on failure.)")

(define forge/llm-error?
  (lambda (s)
    (string-starts-with? s "llm-chat:"))
  "Legacy: true when a raw llm-chat call returned an error string. Kept for frontier/review paths that still use llm-chat directly.")

;; ---- tri-state review: a SECOND LLM session judges the draft ----
;; Separation of powers: the drafter writes code; the reviewer (a different
;; system prompt, not the drafter's session) returns approve | deny | flag.
;; Runs only when mechanical flags are clean (mechanical checks gate first).

(define forge/reviewer-seed
  (string-append
    "You are a Nopales code reviewer for a Pricklypear spec graph. "
    "A drafter (a different LLM session) produced a function body. "
    "Judge it for correctness and style.\n"
    "\n"
    "Respond with EXACTLY one line, starting with one of:\n"
    "  APPROVE -- the body is correct and clean\n"
    "  DENY: <reason> -- the body is wrong (bad syntax, wrong prim args, logic error); the drafter will redraft with this reason as a constraint\n"
    "  FLAG: <reason> -- you are unsure or it needs a human judgment\n"
    "\n"
    "Check for: balanced parens, valid Nopales prims (create takes dict-set* JSON not assoc lists; let takes one body form; no named-let), correct field access (dict-get on cdr of row), and that the body matches the node purpose. Do not nitpick style; only flag real defects.")
  "Seed for the reviewer LLM. Separation of powers: never the drafter's session.")

(define forge/review-node
```

```
  (lambda (graph path)
    (let* ((node (forge/node-by-path graph path)))
      (if (null? node) "review: node not found"
      (let* ((f (cdr node))
             (body (forge/field-str f "body"))
             (title (forge/field-str f "title")))
        (if (string-eq body "") "review: empty body"
        (let* ((all-flags (forge/flags-for path))
               (mflags (list/filter (lambda (p) (not (string-eq (forge/field-str (cdr p) "resolved") "true"))) all-flags)))
          (if (not (null? mflags)) "review: skipped (mechanical flags open)"
          (let* ((prompt (forge/review-prompt path body title))
                 (verdict (llm-chat forge/reviewer-seed prompt)))
            (if (forge/llm-error? verdict)
                (seq
                  (forge/flag path "judgment"
                    (string-append "reviewer call failed: " verdict))
                  "review: reviewer call failed")
                (forge/apply-review verdict graph path)))))))))
  "Tri-state review of a drafted node. Skips if mechanical flags are open
   (mechanical checks gate first). Calls the reviewer LLM (a separate
   session via llm-chat -- no tools, pure judgment). Returns a status
     string.")

(define forge/review-prompt
  (lambda (path body title)
    (string-append
      "Review this Nopales function draft.\n\n"
      "Node: " path "\n"
      "Purpose: " title "\n"
      "Body:\n" body "\n\n"
      "Respond with one line: APPROVE, DENY: <reason>, or FLAG: <reason>."))
  "Assemble the review prompt from the node's body + purpose.")

(define forge/apply-review
  (lambda (verdict graph path)
    (let* ((first-line (car (string-split verdict "\n")))
           (head (forge/trim first-line)))
      (cond
        ((string-starts-with? head "APPROVE") "review: approved")
        ((string-starts-with? head "DENY")
         (let ((reason (forge/review-reason head)))
           (seq
             (forge/add-constraint graph path (string-append "reviewer denied: " reason))
             (forge/flag path "judgment"
               (string-append "reviewer denied: " reason))
             "review: denied (constraint + judgment flag added)")))
        ((string-starts-with? head "FLAG")
         (let ((reason (forge/review-reason head)))
           (seq
             (forge/flag path "judgment"
               (string-append "reviewer flagged: " reason))
             "review: flagged (judgment flag added)")))
        (#t
         (seq
           (forge/flag path "judgment"
             (string-append "reviewer verdict unparseable: " head))
           "review: unparseable verdict"))))))
  "Apply the reviewer's verdict. Takes the FIRST line of the verdict, trims
     it,
   and checks prefixes (APPROVE/DENY/FLAG). APPROVE: no-op (stays drafted,
   human gates the level). DENY: append reason as a constraint + judgment
     flag.
   FLAG: judgment flag. Unknown: judgment flag. Robust to trailing
     commentary.")

(define forge/review-reason
  (lambda (head)
    (let ((parts (string-split head ":")))
      (if (null? (cdr parts))
          "no reason given"
          (forge/trim (forge/str-join ":" (cdr parts)))))))
```

```
  "Extract the reason from a DENY:/FLAG: verdict line -- the text after the
   first colon, trimmed. Joins on ':' in case the reason itself has
     colons.")

(define forge/trim
  (lambda (s)
    (string-trim s))
  "Trim whitespace (wrapper over string-trim).")

;; ---- verify: run the drafter's test, resolve the victory flag ----
;; The verify is a Nopales expression that uses `fn` (bound to the body).
;; verify-node evals (let ((fn body)) verify) -- if truthy, the victory
;; flag resolves and the node advances to verified. This is the TDD loop:
;; the drafter writes a test alongside the code; the loop runs it.

(define forge/verify-node
  (lambda (graph path)
    (let ((node (forge/node-by-path graph path)))
      (if (null? node) "verify: node not found"
      (let ((f (cdr node)))
        (let ((body (forge/field-str f "body"))
              (verify (forge/field-str f "verify")))
          (if (string-eq body "") "verify: empty body"
          (if (string-eq verify "") "verify: empty verify (victory flag
    stays open)"
          (if (not (forge/parse-ok? verify))
              (seq
                (forge/flag path "judgment"
                  "verify expression unparseable")
                "verify: unparseable")
          (let ((test-src
                  (string-append "(let ((fn " body ")) " verify ")")))
            (let ((result (forge/eval-or-error test-src)))
              (if (forge/eval-failed? result)
                  (seq
                    (forge/flag path "judgment"
                      (string-append "verify failed: " result))
                    "verify: failed (judgment flag)")
                  (if (forge/truthy? result)
                      (let ((reg (forge/run-regression graph path body f)))
                        (if (or (string-eq reg "")
                                (string-starts-with? reg "verify: passed;
    regression test passed"))
                            (seq
                              (forge/resolve-victory-flag graph path)
                              (forge/set-status graph path "verified")
                              (if (string-eq reg "")
                                  "verify: passed (victory flag resolved,
    status verified)"
                                  reg))
                            reg))
                      (seq
                        (forge/flag path "judgment"
                          "verify returned falsey -- function fails its own
    test")
                        "verify: returned falsey (judgment
    flag)")))))))))))
  "Run the node's verify expression. Evals (let ((fn body)) verify). If it
   returns truthy, resolve the victory flag and set status verified. If it
   errors or returns falsey, raise a judgment flag. This is the TDD gate:
   the drafter's test must pass before review fires.")

(define forge/eval-or-error
  (lambda (src)
    (guard (lambda (e) (string-append "error: " (as-str e)))
      (str (eval-string src))))
  "Eval a string; return the str'd result on success or 'error: ...' on
     exception.")

(define forge/eval-failed?
  (lambda (s)
    (string-starts-with? s "error:"))
  "True when eval-or-error caught an exception.")
```

```
(define forge/truthy?
  (lambda (s)
    (not (or (string-eq s "#f") (string-eq s "()") (string-eq s ""))))
  "True when the eval result string is truthy (not #f, not (), not empty).")

(define forge/resolve-victory-flag
  (lambda (graph path)
    (let ((flags (forge/flags-for path)))
      (forge/resolve-victory-scan path flags)))
  "Resolve all open victory flags on a node (the verify passed).")

(define forge/resolve-victory-scan
  (lambda (path flags)
    (if (null? flags)
        ""
        (let ((pair (car flags)))
          (let ((fid (car pair))
                (check (forge/field-str (cdr pair) "check"))
                (resolved (forge/field-str (cdr pair) "resolved")))
            (seq
              (if (and (string-eq check "victory")
                       (not (string-eq resolved "true")))
                  (forge/resolve-flag fid "verify passed")
                  "")
              (forge/resolve-victory-scan path (cdr flags)))))))
  "Scan flags, resolve each open victory flag.")

(define forge/first-form
  (lambda (s)
    (forge/first-form-at s 0))
  "Extract the first balanced s-expression from the LLM response,
   ignoring trailing prose/reasoning. Scans for the first '(',
   tracks paren depth respecting \" strings, returns the substring
   up to the matching close. Robust to dots and other prose tokens.")

(define forge/second-form
  (lambda (s)
    (let ((first-end (forge/first-form-end s 0)))
      (if (<= first-end 0)
          ""
          (forge/first-form-at s first-end))))
  "Extract the SECOND balanced s-expression (the verify expression).
   Finds the end of the first form, then scans from there for the next '('.
   Returns \"\" if there is no second form.")

(define forge/first-form-end
  (lambda (s i)
    (let ((n (string-length s)))
      (if (>= i n)
          0
          (if (string-eq (substring s i 1) "(")
              (let ((end (forge/balanced-end s i 0 #f)))
                (if (<= end i) 0 end))
              (forge/first-form-end s (+ i 1))))))
  "Return the index just past the end of the first balanced form, or 0.")

(define forge/first-form-at
  (lambda (s i)
    (let ((n (string-length s)))
      (if (>= i n)
          ""
          (if (string-eq (substring s i 1) "(")
              (let ((end (forge/balanced-end s i 0 #f)))
                (if (<= end i)
                    ""
                    (substring s i (- end i))))
              (forge/first-form-at s (+ i 1))))))
  "Find the first '(' then return substring from there to its match.")

(define forge/balanced-end
  (lambda (s i depth in-str)
    (let ((n (string-length s)))
```

```
      (if (>= i n)
          i
          (let ((c (substring s i 1)))
            (cond
              ((string-eq c "\"")
               (if in-str
                   (forge/balanced-end s (+ i 1) depth #f)
                   (forge/balanced-end s (+ i 1) depth #t)))
              (in-str
               (forge/balanced-end s (+ i 1) depth #t))
              ((string-eq c "(")
               (forge/balanced-end s (+ i 1) (+ depth 1) #f))
              ((string-eq c ")")
               (if (<= depth 1)
                   (+ i 1)
                   (forge/balanced-end s (+ i 1) (- depth 1) #f)))
              (#t
               (forge/balanced-end s (+ i 1) depth #f)))))))
  "Return the index just past the close paren matching the open
   paren at the starting depth.")

;; ---- commit a draft: write body, infer effects, set status, flag ----

(define forge/commit-draft
  (lambda (graph path body-src verify-src)
    (seq
      (forge/set-body-verify graph path body-src verify-src)
      (forge/set-status graph path "drafted")
      (forge/set-inferred-effects graph path body-src)
      (let ((nodes (forge/nodes-of-graph graph)))
        (seq
          (forge/raise-flags graph path
            (forge/mechanical-flags nodes path))
          path))))
  "Write the drafted body, mark drafted, set inferred effects, raise flags.
   Fetches the graph's nodes AFTER the body/effects writes so the flaggers
   see the fresh body (not a stale empty one). The five mechanical flaggers
   search the graph's own nodes (not all 500 rows) -- O(nodes-in-graph).")

(define forge/set-inferred-effects
  (lambda (graph path body-src)
    (let ((pair (forge/node-by-path graph path)))
      (if (null? pair) ""
      (let ((f (cdr pair)))
        (let ((badge (forge-effects body-src)))
          (update 'forge-node (car pair)
            (dict-set f "effects" badge)))))))
  "Set the node's effects field to the inferred badge.")

;; ---- stub children: unresolved refs become planned nodes ----

(define forge/stub-children
  (lambda (graph path)
    (let ((node (forge/node-by-path graph path)))
      (if (null? node) (list)
      (let ((f (cdr node)))
        (let ((body (forge/field-str f "body")))
          (if (string-eq body "") (list)
          (let ((unres (forge/unresolved graph path body)))
            (forge/stub-list graph path unres)))))))))
  "Create a planned child node for each unresolved reference in the body.")

(define forge/stub-list
  (lambda (graph parent-path names)
    (if (null? names) (list)
    (let ((name (car names)))
      (let ((child-path (string-append parent-path "/" name)))
        (seq
          (forge/add-node graph child-path "function" name parent-path)
          (forge/stub-list graph parent-path (cdr names))))))))
  "Stub each unresolved name as a child node.")

;; ---- the step state machine ----
```

```
1245
1246 (define forge/step
1247   (lambda (graph-name)
1248     (let ((graph (forge/graph-by-name graph-name)))
1249       (if (null? graph) "graph not found"
1250       (let ((judgment (forge/open-judgment-flags graph-name)))
1251         (cond
1252           ((not (null? judgment)) "blocked: open judgment flags -- see
      /forge")
1253           ((forge/has-unapproved-level? graph-name) "pending: level needs
      approval -- see /forge")
1254           ((forge/frontier-non-empty? graph-name) (forge/draft-frontier
      graph-name))
1255           (#t (forge/propose-descent graph-name)))))))
1256   "One step of the loop. Returns a status string or the drafted path.")
1257
1258 (define forge/open-judgment-flags
1259   (lambda (graph-name)
1260     (list/filter
1261       (lambda (pair)
1262         (string-eq (forge/field-str (cdr pair) "check") "judgment"))
1263       (forge/open-flags graph-name)))
1264   "Open flags with check=judgment (the human escape hatch).")
1265
1266 (define forge/has-unapproved-level?
1267   (lambda (graph-name)
1268     (forge/has-unapproved-scan graph-name (forge/nodes-of-graph-by-name
      graph-name)))
1269   "True when any node at the current working depth is below approved.")
1270
1271 (define forge/nodes-of-graph-by-name
1272   (lambda (graph-name)
1273     (forge/nodes-of-graph graph-name))
1274   "Alias for nodes-of-graph (kept for step/descent callers).")
1275
1276 (define forge/has-unapproved-scan
1277   (lambda (graph-name nodes)
1278     (if (null? nodes) #f
1279     (let ((f (cdr (car nodes))))
1280       (let ((status (forge/field-str f "status")))
1281         (cond
1282           ((string-eq status "drafted") #t)
1283           ((string-eq status "verified") #t)
1284           (#t (forge/has-unapproved-scan graph-name (cdr nodes))))))))
1285   "A drafted/verified node is unapproved (past planned, not yet approved).
1286    Planned/approved/implemented nodes are not unapproved -- keep scanning.")
1287
1288 (define forge/frontier-non-empty?
1289   (lambda (graph-name)
1290     (not (null? (forge/frontier graph-name))))
1291   "True when the computed frontier has ready nodes.")
1292
1293 (define forge/draft-frontier
1294   (lambda (graph-name)
1295     (let ((f (forge/frontier graph-name)))
1296       (if (null? f) "frontier empty"
1297       (let ((path (car f)))
1298         (seq
1299           (forge/draft-node graph-name path)
1300           (forge/verify-node graph-name path)
1301           (forge/review-node graph-name path))))))
1302   "Draft the first ready node, run its verify (TDD gate), then tri-state
      review. The verify must pass before review fires -- mechanical flags
      (including victory) gate first.")
1303
1304 (define forge/propose-descent
1305   (lambda (graph-name)
1306     (forge/descent-scan graph-name (forge/nodes-of-graph-by-name
      graph-name)))
1307   "Stub children for every approved node with a body; propose the next
      level.")
1308
1309 (define forge/descent-scan
```

```
  (lambda (graph-name nodes)
    (if (null? nodes) "done: no further descent"
    (let ((f (cdr (car nodes))))
      (let ((status (forge/field-str f "status"))
            (path (forge/field-str f "path")))
        (cond
          ((string-eq status "approved")
           (seq
             (forge/stub-children graph-name path)
             (forge/descent-scan graph-name (cdr nodes))))
          (#t (forge/descent-scan graph-name (cdr nodes))))))))
  "For each approved node, stub its unresolved refs as children (descent).")

;; ---- forge/run: the loop as a job (borge/jobs.borg) ----

(define forge/run
  (lambda (graph-name)
    (submit-job "forge/step-job" (string-append "{\"graph\":\"" graph-name
    "\"}")))
  "Submit forge/step as a background job. The runner evals (forge/step-job
    {\"graph\":...})
   on your branch; the loop persists across HTTP requests and survives the
   laptop disconnecting. Poll (job-status id) to watch progress. The
    step-job
   function below is the job's entry point -- it calls forge/step once per
    run.
   Re-submit (forge/run graph) after each gate (approval) to continue the
    descent.")

(define forge/step-job
  (lambda (args)
    (let ((graph-name (dict-get args "graph")))
      (if (null? graph-name)
          "no graph in args"
          (forge/step graph-name))))
  "Job entry point for forge/run. Calls forge/step once. The runner records
   the step result; the caller re-submits after approving the level. This is
   the loop-persists-across-requests shape from borge/forge.borg
    forge/run.")

;; ============================================================
;; EXISTING-APP BRIDGE (existing-app-bridge subsection)
;; import-function bootstraps a graph from live bindings; the target
;; field anchors modify-existing nodes; test-fn enables regression.
;; ============================================================

(define forge/safe-source
  (lambda (name)
    (guard (lambda (e) "")
      (let ((s (function-source name)))
        (if (null? s) "" (as-str s)))))
  "Read a binding's stored source via function-source; \"\" on any error
   or absent binding (e.g. a prim with no stored source).")

(define forge/set-target
  (lambda (graph path target)
    (let ((pair (forge/node-by-path graph path)))
      (if (null? pair) ""
        (update 'forge-node (car pair)
          (dict-set (cdr pair) "target" target))))))
  "Set the target field -- names an existing binding this node modifies.
   Empty string = greenfield (create-new).")

(define forge/set-test-fn
  (lambda (graph path test-fn)
    (let ((pair (forge/node-by-path graph path)))
      (if (null? pair) ""
        (update 'forge-node (car pair)
          (dict-set (cdr pair) "test-fn" test-fn))))))
  "Set the test-fn field -- existing test binding to run as a regression
   after the drafted body is installed as the target.")

(define forge/test-fn-prompt
```

```
  (lambda (test-fn)
    (if (string-eq test-fn "") ""
      (string-append
        "A regression test function exists: " test-fn "\n"
        "After install, forge will call (" test-fn ") with your new body\n"
        "bound to the target name. Write your verify to test `fn` directly;\n"
        "the regression is automatic. Your verify should still assert the\n"
        "new behavior.\n")))
  "Prompt fragment telling the drafter a regression test will run.")

(define forge/import-function
  (lambda (graph path binding-name parent)
    (let ((src (forge/safe-source binding-name)))
      (if (string-eq src "")
        (seq
          (forge/flag path "judgment"
            (string-append "import-function: no source for " binding-name))
          "")
        (seq
          (forge/add-node graph path "function" binding-name parent)
          (forge/set-body-verify graph path src "(begin #t)")
          (forge/set-status graph path "implemented")
          (forge/set-target graph path binding-name)
          (forge/set-inferred-effects graph path src)
          (forge/node-id graph path))))))
  "Import an existing binding into the graph as a status=implemented node.
   Reads source via function-source, sets body=source, verify=(begin #t)
   (pre-existing code is trusted), target=binding-name, effects=inferred.
   Bootstraps a graph from an existing package without hand-authoring nodes.")

;; ---- regression verify: install new body, run test-fn, restore ----

(define forge/run-regression
  (lambda (graph path body-src f)
    (let ((target (forge/field-str f "target")))
      (if (string-eq target "")
          ""
          (forge/run-regression-2 graph path body-src f target
    (forge/field-str f "test-fn")))))
  "Run regression for a modify-existing node. Dispatches to part 2 with
   target + test-fn extracted. Returns \"\" when no regression configured.")

(define forge/run-regression-2
  (lambda (graph path body-src f target test-fn)
    (if (string-eq test-fn "")
        ""
        (let ((old (forge/safe-source target)))
          (if (string-eq old "")
              (seq
                (forge/flag path "judgment"
                  (string-append "regression skipped: no source for target " target))
                "verify: passed; regression skipped (target has no source -- flagged)")
              (forge/run-regression-3 graph path body-src target test-fn old))))))
  "Part 2: check old source exists, dispatch to install+test+restore.")

(define forge/run-regression-3
  (lambda (graph path body-src target test-fn old)
    (let ((install-ok (forge/eval-or-error
                        (string-append "(define " target " " body-src ")"))))
      (if (forge/eval-failed? install-ok)
          (seq
            (forge/flag path "judgment"
              (string-append "regression install failed: " install-ok))
            "verify: passed; regression install failed (judgment flag)")
          (forge/run-regression-4 graph path target test-fn old))))
  "Part 3: install the drafted body, dispatch to test+restore.")

```

```
(define forge/run-regression-4
  (lambda (graph path target test-fn old)
    (let ((test-result (forge/eval-or-error
                         (string-append "(" test-fn ")"))))
      (seq
        (forge/eval-or-error (string-append "(define " target " " old ")"))
        (if (forge/eval-failed? test-result)
            (seq
              (forge/flag path "judgment"
                (string-append "regression test errored: " test-result))
              "verify: passed; regression test errored (judgment flag)")
            (if (forge/truthy? test-result)
                "verify: passed; regression test passed (target restored)"
                (seq
                  (forge/flag path "judgment"
                    "regression test returned falsey -- existing tests break")
                  "verify: passed; regression FAILED (judgment flag)")))))))
  "Part 4: run the test, restore original source, report result.")

;; ============================================================
;; HUMAN SURFACE -- /forge flag queue (human-surface subsection)
;; Anomaly-first: clean nodes render one-line; flagged nodes show detail.
;; htmx actions: approve (POST /forge/approve), reroll-with-note (POST /forge/reroll).
;; ============================================================

(define forge/page
  (lambda ()
    (let ((name (request-param "graph")))
      (if (or (null? name) (string-eq (as-str name) ""))
          (forge/page-graphs)
          (forge/page-graph (as-str name)))))
  "GET /forge -- list graphs, or one graph when ?graph=<name> is set.")

(define forge/page-graphs
  (lambda ()
    (let ((gs (rows 'forge-graph 100)))
      (render-html
        (string-append
          "<h1>Forge</h1>"
          "<p>Executable spec graphs. The flag queue is the human surface -- "
          "review anomalies, not a wall of nodes.</p>"
          (forge/graph-list-html gs)))))
  "List all forge graphs with status.")

(define forge/all-open-flags
  (lambda ()
    (list/filter
      (lambda (pair)
        (not (string-eq (forge/field-str (cdr pair) "resolved") "true")))
      (rows 'forge-flag 500)))
  "All unresolved forge-flag rows -- one DB scan, reused per node.")

(define forge/graph-list-html
  (lambda (gs)
    (if (null? gs)
      "<p><em>No graphs yet. Create one with (forge/new-graph \"name\" \"title\" 4).</em></p>"
      (string-append
        "<table><tr><th>graph</th><th>status</th></tr>"
        (forge/graph-row-html gs)
        "</table>")))
  "Render the graph list table body (or empty-state message).")

(define forge/graph-row-html
  (lambda (gs)
    (if (null? gs) ""
      (let ((f (cdr (car gs))))
        (let ((name (forge/field-str f "name"))
              (status (forge/field-str f "status")))
          (string-append
```

```
            "<tr><td><a href=\"/forge?graph=" name "\">" name "</a></td>"
            "<td>" status "</td></tr>"
            (forge/graph-row-html (cdr gs)))))))
  "One table row per graph.")

(define forge/page-graph
  (lambda (name)
    (let ((graph (forge/graph-by-name name)))
      (if (null? graph)
        (render-html
          (string-append "<h1>Forge</h1><p>Graph not found: " name "</p>"
            "<p><a href=\"/forge\">? all graphs</a></p>"))
        (let ((nodes (forge/nodes-of-graph name))
              (all-flags (forge/all-open-flags)))
          (render-html
            (string-append
              "<h1>Forge: " name "</h1>"
              "<p><a href=\"/forge\">? all graphs</a></p>"
              (forge/node-list-html name nodes all-flags)))))))
  "GET /forge?graph=<name> -- anomaly-first view of a graph's nodes.")

(define forge/node-list-html
  (lambda (graph nodes all-flags)
    (if (null? nodes) "<p><em>No nodes.</em></p>"
      (string-append
        (forge/node-row-html graph (car nodes) all-flags)
        (forge/node-list-html graph (cdr nodes) all-flags))))
  "Render all node rows for a graph.")

(define forge/node-row-html
  (lambda (graph pair all-flags)
    (let ((path (forge/field-str (cdr pair) "path"))
          (title (forge/field-str (cdr pair) "title"))
          (status (forge/field-str (cdr pair) "status"))
          (effects (forge/field-str (cdr pair) "effects")))
      (let ((flags (forge/open-flags-for-node-in path all-flags)))
        (string-append
          "<div id=\"node-" path "\" style=\"border:1px solid
    #ccc;padding:8px;margin:4px\">"
          "<b>" path "</b> -- " title "<br>"
          "status: " status " | effects: " effects
          (forge/flag-detail-html flags)
          (forge/node-actions-html graph path status)
          "</div>"))))
  "One node card: path, title, status, effects, flags, actions.")

(define forge/open-flags-for-node-in
  (lambda (path flags)
    (list/filter
      (lambda (pair)
        (and (string-eq (forge/field-str (cdr pair) "node_path") path)
             (not (string-eq (forge/field-str (cdr pair) "resolved")
    "true"))))
      flags))
  "Unresolved flags for a node path, filtered from a pre-fetched list (no
    DB).")

(define forge/flag-detail-html
  (lambda (flags)
    (if (null? flags) ""
      (string-append "<ul>" (forge/flag-item-html flags) "</ul>")))
  "Render flag list as a <ul>, or empty when clean.")

(define forge/flag-item-html
  (lambda (flags)
    (if (null? flags) ""
      (let ((f (cdr (car flags))))
        (string-append
          "<li><b>" (forge/field-str f "check") "</b>: "
          (forge/field-str f "detail") "</li>"
          (forge/flag-item-html (cdr flags))))))
  "One <li> per flag.")
```

```
(define forge/node-actions-html
  (lambda (graph path status)
    (if (or (string-eq status "approved") (string-eq status "implemented"))
      " <em>approved</em>"
      (string-append
        " <button hx-post=\"/forge/approve\" "
        "hx-vals='{\"graph\":\"" graph "\",\"path\":\"" path "\"}' "
        "hx-target=\"closest div\" hx-swap=\"outerHTML\">approve</button> "
        "<form hx-post=\"/forge/reroll\" hx-target=\"closest div\" "
        "hx-swap=\"outerHTML\" style=\"display:inline\">"
        "<input type=\"hidden\" name=\"graph\" value=\"" graph "\">"
        "<input type=\"hidden\" name=\"path\" value=\"" path "\">"
        "<input type=\"text\" name=\"note\" placeholder=\"reroll note\">"
        "<button>reroll</button></form>")))
  "Action buttons for a node: approve + reroll-with-note.
    Approved/implemented
   nodes show only an 'approved' marker.")

(define forge/approve-action
  (lambda ()
    (let ((graph (as-str (request-param "graph")))
          (path (as-str (request-param "path"))))
      (seq
        (forge/set-status graph path "approved")
        (render-html (forge/node-row-html graph (forge/node-by-path graph
    path) (forge/all-open-flags))))))
  "POST /forge/approve -- set node status approved, return updated node
    row.")

(define forge/reroll-action
  (lambda ()
    (let ((graph (as-str (request-param "graph")))
          (path (as-str (request-param "path")))
          (note (as-str (request-param "note"))))
      (seq
        (forge/add-constraint graph path note)
        (render-html (forge/node-row-html graph (forge/node-by-path graph
    path) (forge/all-open-flags))))))
  "POST /forge/reroll -- append the note as a constraint, return updated
    node row.")

(route "/forge" "forge/page")
(route "/forge/approve" "forge/approve-action")
(route "/forge/reroll" "forge/reroll-action")
```

169 libs/hello/lib.pp

```
;; Hello page recipe -- pure Lisp.

(define hi-page
  (lambda ()
    (render-html
      "<h1>Hello</h1><p>Your first Pricklypear page. Edit me at
     /edit/hi-page.</p>"))
  "GET /hello -- minimal sample page.")

(route "/hello" "hi-page")
```

170 libs/html/lib.pp

```
;; package html -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push

(define html/a
  (lambda
    (href body)
    "An anchor tag with the given href around body."
    (html/tag "a" (html/attr "href" href) body)))

(define html/attr
  (lambda
    (k v)
    "Render one HTML attribute as k="v", escaping v."
    (string-append
      (as-str k)
      "=\""
      (html-escape (as-str v))
      "\"")))

(define html/attrs
  (lambda
    (xs)
    "Render a list of attribute strings space-separated."
    (list/foldl
      (lambda
        (acc a)
        (if
          (string-eq acc "")
          (as-str a)
          (string-append acc " " (as-str a))))
      ""
      xs)))

(define html/button
  (lambda (attrs body) "A button tag with the given attrs and body."
    (html/tag "button" attrs body)))

(define html/code
  (lambda (body) "A code tag with escaped text body." (html/tag "code" ""
    (html/text body))))

(define html/div
  (lambda (attrs body) "A div tag with the given attrs and body." (html/tag
    "div" attrs body)))

(define html/form
  (lambda
    (method action attrs body)
    "A form tag with method and action attributes plus extra attrs."
    (html/tag
      "form"
      (html/attrs
        (list
          (html/attr "method" method)
          (html/attr "action" action)
          attrs))
      body)))

(define html/h1
  (lambda (body) "An h1 tag with the given body." (html/tag "h1" "" body)))

(define html/h2
  (lambda (body) "An h2 tag with the given body." (html/tag "h2" "" body)))

(define html/h3
  (lambda (body) "An h3 tag with the given body." (html/tag "h3" "" body)))

(define html/hidden
  (lambda
    (name value)
    "A hidden input field for name/value."
```

```
    (html/input
      (html/attrs
        (list
          (html/attr "type" "hidden")
          (html/attr "name" name)
          (html/attr "value" value))))))

(define html/img
  (lambda
    (src alt class)
    "An img void tag with src, alt, and optional class."
    (html/void
      "img"
      (html/attrs
        (list
          (html/attr "src" src)
          (html/attr "alt" alt)
          (if
            (string-eq (as-str class) "")
            ""
            (html/attr "class" class)))))))

(define html/input
  (lambda (attrs) "An input void tag with the given attrs." (html/void
     "input" attrs)))

(define html/join
  (lambda
    (xs)
    "Concatenate a list of elements into one string."
    (list/foldl
      (lambda (acc x) (string-append acc (as-str x)))
      ""
      xs)))

(define html/label
  (lambda (attrs body) "A label tag with the given attrs and body."
     (html/tag "label" attrs body)))

(define html/li
  (lambda (attrs body) "An li tag with the given attrs and body." (html/tag
     "li" attrs body)))

(define html/option
  (lambda
    (value label selected?)
    "An option tag with value, label, and optional selected attrs."
    (html/tag
      "option"
      (if
        selected?
        (html/attrs
          (list
            (html/attr "value" value)
            (html/attr "selected" "selected")))
        (html/attr "value" value))
      (html/text label))))

(define html/p
  (lambda (body) "A p tag with escaped text body." (html/tag "p" ""
     (html/text body))))

(define html/p*
  (lambda (attrs body) "A p tag with the given attrs and body." (html/tag
     "p" attrs body)))

(define html/page
  (lambda (parts) "Render joined parts as a full HTML page." (render-html
     (html/join parts))))

(define html/pre
  (lambda (body) "A pre tag with the given body." (html/tag "pre" "" body)))

```

```
(define html/select
  (lambda
    (name attrs body)
    "A select tag with name attribute plus extra attrs."
    (html/tag
      "select"
      (html/attrs (list (html/attr "name" name) attrs))
      body)))

(define html/span
  (lambda (attrs body) "A span tag with the given attrs and body."
    (html/tag "span" attrs body)))

(define html/strong
  (lambda (body) "A strong tag with escaped text body." (html/tag "strong"
    "" (html/text body))))

(define html/tag
  (lambda
    (name attrs body)
    "Wrap body in an opening/closing tag, adding attrs when non-empty."
    (let
      ((n (as-str name)) (a (as-str attrs)) (b (as-str body)))
      (if
        (string-eq a "")
        (string-append
          "<"
          n
          ">"
          b
          "</"
          n
          ">")
        (string-append
          "<"
          n
          " "
          a
          ">"
          b
          "</"
          n
          ">")))))

(define html/text
  (lambda (s) "HTML-escapes the string s." (html-escape s)))

(define html/text-input
  (lambda
    (name attrs)
    "A text input field with name and extra attrs."
    (html/input
      (html/attrs
        (list
          (html/attr "type" "text")
          (html/attr "name" name)
          attrs)))))

(define html/textarea
  (lambda
    (name attrs body)
    "A textarea tag with name, attrs, and escaped body."
    (html/tag
      "textarea"
      (html/attrs (list (html/attr "name" name) attrs))
      (html-escape (as-str body)))))

(define html/ul
  (lambda (attrs body) "A ul tag with the given attrs and body." (html/tag
    "ul" attrs body)))

(define html/void
  (lambda
```

```
    (name attrs)
    "Render a void (self-closing) tag, adding attrs when non-empty."
    (let
      ((n (as-str name)) (a (as-str attrs)))
      (if
        (string-eq a "")
        (string-append "<" n ">")
        (string-append "<" n " " a ">")))))

(define html/wrap
  (lambda
    (title body)
    "Wrap title and body in an article with a header."
    (render-html
      (html/tag
        "article"
        ""
        (html/join
          (list (html/h1 (html/text title)) (html/div "" body)))))))
```

171 libs/hyper/lib.pp

```
;; hypermedia harness client -- headless htmx swap simulator (Nopales layer)
;; Wraps soup/* + route/dispatch prims. Spec: borge/hypermedia-harness.borg
;;
;; A page value is an opaque VStr handle into the soup/ prims' handle table.
;; Each interaction returns a NEW page value (deep-copied via soup/page-copy
;; before mutation), so pages are immutable at this surface -- free history
;; and diff. Outside the fixi-five supported subset, signals
;; 'unsupported-feature' so the flow graduates to the CDP tier.

;; ---------- URL parsing ----------
;; Split a URL into (path . query-alist).
(define hyper/parse-url
  (lambda (url)
    (let* ((parts (string-split url "?"))
           (path (car parts))
           (rest (if (null? (cdr parts)) "" (car (cdr parts))))
           (pairs (if (equal? rest "") (list)
                      (string-split rest "&"))))
      (cons path (hyper/parse-pairs pairs))))
  "Parse url into (path . ((key . val) ...)).")

(define hyper/parse-pairs
  (lambda (pairs)
    (if (null? pairs) (list)
        (let* ((kv (car pairs))
               (eqparts (string-split kv "=")))
          (cons (if (null? (cdr eqparts)) (cons kv "")
                    (cons (car eqparts) (car (cdr eqparts))))
                (hyper/parse-pairs (cdr pairs)))))))
  "Parse a list of kv strings into an alist.")

;; ---------- visit ----------
;; (hyper/visit url) -> page handle
;; Resolve url -> handler via route/lookup, GET in-process (no HX-Request
;; header, so the handler returns the full page like a browser's initial
    nav),
;; soup/parse the returned HTML.
(define hyper/visit
  (lambda (url)
    (let* ((parsed (hyper/parse-url url))
           (path (car parsed))
           (params (cdr parsed))
           (handler (route/lookup path)))
      (if (null? handler)
          (error (string-append "unsupported-feature: no route for " path))
          (let ((html (route/dispatch handler "GET" path params)))
            (soup/parse html)))))
  "GET the route handler in-process; parse body -> page value.")

;; ---------- queries ----------
(define hyper/text
  (lambda (page sel)
    (let ((matches (soup/select page sel)))
      (if (null? matches) ""
          (soup/text (car matches)))))
  "Text content of the first element matching sel.")

(define hyper/query
  (lambda (page sel)
    (soup/select page sel))
  "List of node handles matching sel.")

(define hyper/count
  (lambda (page sel)
    (count (soup/select page sel)))
  "Count elements matching sel.")

(define hyper/html
  (lambda (page)
    (soup/serialize page))
  "Serialize the page tree to an HTML string.")
```

```
71
72  (define hyper/attr
73    (lambda (node name)
74      (soup/attr node name))
75    "Get an attribute value from a node handle.")
76
77  ;; ---------- diff + expect ----------
78  (define hyper/diff
79    (lambda (p1 p2)
80      (soup/diff p1 p2))
81    "Node-level what-changed report between two pages.")
82
83  (define hyper/expect
84    (lambda (page sel expected)
85      (let ((actual (hyper/text page sel)))
86        (if (equal? actual expected)
87            #t
88            (error (string-append
89                     "hyper/expect: "
90                     sel
91                     " expected \""
92                     expected
93                     "\" got \""
94                     actual
95                     "\""))))))
96    "Assert text at sel matches expected; errors on mismatch.")
97
98  ;; ---------- click (the swap loop) ----------
99  ;; (hyper/click page sel params) -> NEW page value
100 ;; Find trigger at sel, read hx-post/hx-get (from element), read
101 ;; hx-target/hx-swap (v1: element only -- ancestor walk needs a soup prim),
102 ;; POST in-process with HX-Request header, parse fragment, deep-copy the
103 ;; page, apply swap, return new page.
104 ;;
105 ;; v1 limitations (errors with 'unsupported-feature'):
106 ;;   - inherited hx-target/hx-swap (ancestor walk) -- needs soup/ancestor
    prim
107 ;;   - hx-on, hx-push-url, hx-ext, hx-vals js:, debounce ? unsupported
108 ;;   - form serialization: caller passes params explicitly
109 (define hyper/click
110   (lambda (page sel params)
111     (let ((matches (soup/select page sel)))
112       (if (null? matches)
113           (error (string-append "hyper/click: no element matches " sel))
114           (hyper/click* page (car matches) params))))
115   "Simulate a click: read hx-*, POST, swap, return new page.")
116
117 (define hyper/click*
118   (lambda (page elt params)
119     ;; Read hx-post/hx-get from elt OR nearest ancestor (inherited trigger
    --
120     ;; mirrors htmx compat implicit inheritance).
121     (let* ((hxpost (soup/closest-attr elt "hx-post"))
122            (hxget (soup/closest-attr elt "hx-get")))
123       (cond
124         ((not (null? hxpost)) (hyper/do-swap page elt "POST" hxpost params))
125         ((not (null? hxget)) (hyper/do-swap page elt "GET" hxget params))
126         (#t (error "unsupported-feature: no hx-get/hx-post on element or
    ancestor")))))
127   "Read trigger from element/ancestor; POST or GET then swap.")
128
129 ;; Read target + swap, POST, deep-copy page, apply swap, return new page.
130 ;; target/swap are read via closest-attr (inherited from ancestors).
131 (define hyper/do-swap
132   (lambda (page elt verb url params)
133     (let* ((target-raw (soup/closest-attr elt "hx-target"))
134            (swap-raw (soup/closest-attr elt "hx-swap"))
135            (swap (if (null? swap-raw) "innerHTML" swap-raw))
136            (parsed (hyper/parse-url url))
137            (path (car parsed))
138            (qparams (cdr parsed))
139            (all-params (hyper/append-alist qparams params))
140            (handler (route/lookup path)))
```

```
      (if (null? handler)
          (error (string-append "unsupported-feature: no route for " path))
          (begin
            (let* ((fragment-html (route/dispatch handler verb path
                                    (cons (cons "hx-request" "true")
    all-params)))
                   (new-page (soup/page-copy page))
                   (target (if (null? target-raw) elt
                               (let ((ts (soup/select new-page target-raw)))
                                 (if (null? ts) elt (car ts))))))
              (begin (soup/swap! target swap fragment-html)
                     new-page))))))
  "Read target+swap, dispatch, deep-copy, apply swap; return new page.")

(define hyper/append-alist
  (lambda (a b)
    (if (null? b) a (cons (car b) (hyper/append-alist a (cdr b)))))
  "Append b onto the end of alist a.")

;; ---------- stream (multipart replay) ----------
;; (hyper/stream page parts) -> NEW page value
;; Apply an ordered multipart part list onto a page via soup/page-copy +
;; soup/apply-parts!, returning a NEW page (immutability preserved: the
;; input page is never mutated). parts come from the host stream as DATA
;; (agent: loop.ml collects the emitted HX-Target/HX-Swap parts;
;; image: http_agent mp_write_part). Lets hyper/** assert a consumed
;; chat/agent turn deterministically. Each part is an alist
;; (("html" . frag) ("target" . sel) ("swap" . strat)); target/swap
;; absent -> caller element + "innerHTML" (soup/apply-parts! defaults).
(define hyper/stream
  (lambda (page parts)
    (let ((copy (soup/page-copy page)))
      (begin (soup/apply-parts! copy parts)
             copy)))
  "Apply an ordered part list onto a fresh copy of page; return NEW page.")

;; ---------- sandbox ----------
;; (hyper/sandbox branch body) -- v1 stub. Full impl needs a branch checkout
;; with restore (dynamic-wind). Returns unsupported-feature for now.
(define hyper/sandbox
  (lambda (branch body)
    (error "unsupported-feature: hyper/sandbox (branch checkout in
     simulator)"))
  "Run body on a dev branch (v1 stub).")
```

172 libs/kanban/lib.pp

```
;; package kanban -- a board view over todos. One todo-list = one board;
;; the list's `columns` field (a JSON array of state-name strings, read via
;; json-parse) overrides todo/state/list when present. Cards are todo-items;
;; every mutation is a TREE ACTION over the card's datum node
;; (one-tree.tree-datum-prims): tree/upsert at the card's node-path, not a
;; create/update call. Reorder/rename wrap multiple tree/upserts in
;; with-transaction. Sync: scripts/pp_sync.py pull|push

;; ---------------------------------------------------------------------
;; columns: per-list state set, or the default
;; ---------------------------------------------------------------------

(define kanban/columns-default
  (lambda ()
    "The fallback column set when a list has no `columns` field."
    todo/state/list))

(define kanban/columns-for-list
  (lambda (list-id)
    "Return the list's columns as a Lisp list of state strings.
     Falls back to todo/state/list when the list has no columns field
     or the field is empty. Single source of truth:
     todo/state/columns-for-list."
    (todo/state/columns-for-list list-id)))

(define kanban/columns-set
  (lambda (list-id cols)
    "tree/upsert the LIST datum with a new columns array (cols is a Lisp
     list
     of state strings). Returns the upserted path."
    (let ((lf (get (quote todo-list) list-id)))
      (if (null? lf)
        "no such list"
        (let ((path (node-path (quote todo-list) list-id)))
          (let ((body (dict-set* lf (list "columns" (json-emit cols)))))
            (tree/upsert path "datum" "todo-list" body)))))))

;; ---------------------------------------------------------------------
;; card helpers (all tree actions)
;; ---------------------------------------------------------------------

(define kanban/cards-in
  (lambda (list-id state)
    "Return (id . fields) pairs of items in list-id with the given state."
    (list/filter
      (lambda (row)
        (string-eq (as-str (dict-get (cdr row) "state")) state))
      (todo/items-for-list list-id))))

(define kanban/renumber-at
  (lambda (cards state i)
    "tree/upsert each card with the given state and position = its index.
     Must run inside a with-transaction for atomicity."
    (if (null? cards)
      ()
      (let ((row (car cards)))
        (let ((id (car row)) (fields (cdr row)))
          (seq
            (let ((path (node-path (quote todo-item) id)))
              (tree/upsert path "datum" "todo-item"
                (dict-set* fields (list "state" state "position" (as-str
    i)))))
            (kanban/renumber-at (cdr cards) state (+ i 1)))))))))

(define kanban/renumber
  (lambda (cards state)
    "Renumber cards in the given state from 0, tree/upserting each."
    (kanban/renumber-at cards state 0)))

(define kanban/remove-id
  (lambda (cards id)
```

```
    "Filter cards to exclude the row whose id matches."
    (list/filter
      (lambda (row) (not (string-eq (as-str (car row)) (as-str id))))
      cards)))

(define kanban/insert-at
  (lambda (cards row index)
    "Insert row into cards at the given index (0-based); clamp to end."
    (kanban/insert-at-at cards row index 0 (list))))

(define kanban/insert-at-at
  (lambda (cards row index i acc)
    "Internal accumulating insert of row at index into cards."
    (if (null? cards)
      (append (reverse acc) (list row))
      (if (>= i index)
        (append (reverse acc) (cons row cards))
        (kanban/insert-at-at (cdr cards) row index (+ i 1) (cons (car
    cards) acc))))))

(define kanban/move-card
  (lambda (id target-state target-index)
    "Move card id to target-state at target-index (0-based; clamps to end).
     Renumbers the old and new columns so no two cards share a position.
     All tree/upserts run in one with-transaction."
    (with-transaction
      (lambda ()
        (let ((fields (get (quote todo-item) id)))
          (if (null? fields)
            "no such card"
            (let ((old-state (as-str (dict-get fields "state")))
                  (list-id (as-str (dict-get fields "list_id"))))
              (let ((all (todo/items-for-list list-id)))
                (let ((without (kanban/remove-id all id)))
                  (if (string-eq old-state target-state)
                    (let ((col (list/filter
                                 (lambda (row)
                                   (string-eq (as-str (dict-get (cdr row)
    "state")) old-state))
                                 without)))
                      (let ((reordered (kanban/insert-at col (cons id
    fields) target-index)))
                        (kanban/renumber reordered old-state)))
                    (let ((old-col (list/filter
                                     (lambda (row)
                                       (string-eq (as-str (dict-get (cdr
    row) "state")) old-state))
                                     without))
                          (let ((new-col (list/filter
                                           (lambda (row)
                                             (string-eq (as-str (dict-get (cdr
    row) "state")) target-state))
                                           without)))
                            (let ((reordered (kanban/insert-at new-col (cons id
    fields) target-index)))
                              (seq
                                (kanban/renumber old-col old-state)
                                (kanban/renumber reordered
    target-state)))))))))))))))

(define kanban/swap
  (lambda (id1 id2)
    "Swap the position fields of two cards (same column) in one
     with-transaction."
    (with-transaction
      (lambda ()
        (let ((f1 (get (quote todo-item) id1)))
          (if (null? f1)
            "no such card"
            (let ((f2 (get (quote todo-item) id2)))
              (if (null? f2)
                "no such card"
                (let ((p1 (as-str (dict-get f1 "position")))
```

```
                    (p2 (as-str (dict-get f2 "position"))))
                  (seq
                    (tree/upsert (node-path (quote todo-item) id1) "datum"
    "todo-item"
                      (dict-set* f1 (list "position" p2)))
                    (tree/upsert (node-path (quote todo-item) id2) "datum"
    "todo-item"
                      (dict-set* f2 (list "position" p1)))))))))))))

(define kanban/index-of
  (lambda (cards id i)
    "0-based index of the row whose id matches, or -1."
    (if (null? cards)
      -1
      (if (string-eq (as-str (car (car cards))) (as-str id))
        i
        (kanban/index-of (cdr cards) id (+ i 1))))))

;; ----------------------------------------------------------------
;; column mutation (add / remove / rename)
;; ----------------------------------------------------------------

(define kanban/add-column
  (lambda (list-id name)
    "Append a new column name to the list's columns. Returns upserted path."
    (let ((cols (kanban/columns-for-list list-id)))
      (kanban/columns-set list-id (append cols (list (as-str name)))))))

(define kanban/remove-column
  (lambda (list-id name)
    "Remove a column name from the list's columns. Cards in that state are
     NOT deleted (they land in the 'other' bucket). Returns upserted path."
    (let ((cols (kanban/columns-for-list list-id)))
      (kanban/columns-set list-id (kanban/remove-named cols (as-str
    name))))))

(define kanban/remove-named
  (lambda (cols name)
    "Drop the column named name from cols."
    (list/filter (lambda (c) (not (string-eq (as-str c) name))) cols)))

(define kanban/rename-column
  (lambda (list-id old-name new-name)
    "Rename a column: update the list's columns AND cascade-rename the state
     field of every card in that column, in one with-transaction."
    (with-transaction
      (lambda ()
        (let ((cols (kanban/columns-for-list list-id)))
          (let ((renamed (kanban/replace-named cols (as-str old-name)
    (as-str new-name))))
            (seq
              (kanban/columns-set list-id renamed)
              (kanban/cascade-rename-state list-id (as-str old-name)
    (as-str new-name)))))))))

(define kanban/replace-named
  (lambda (cols old-name new-name)
    "Rename old-name to new-name throughout cols."
    (list/map
      (lambda (c) (if (string-eq (as-str c) old-name) new-name (as-str c)))
      cols)))

(define kanban/cascade-rename-state
  (lambda (list-id old-name new-name)
    "tree/upsert every card in list-id whose state is old-name, setting
     new-name."
    (let ((matches (list/filter
                     (lambda (row)
                       (string-eq (as-str (dict-get (cdr row) "state"))
    old-name))
                     (todo/items-for-list list-id))))
      (kanban/rename-state-at matches new-name))))

```

```
(define kanban/rename-state-at
  (lambda (cards new-name)
    "tree/upsert each card with its state set to new-name."
    (if (null? cards)
      ()
      (let ((row (car cards)))
        (let ((id (car row)) (fields (cdr row)))
          (seq
            (tree/upsert (node-path (quote todo-item) id) "datum"
    "todo-item"
              (dict-set* fields (list "state" new-name)))
            (kanban/rename-state-at (cdr cards) new-name)))))))

(define kanban/add-card
  (lambda (list-id title state)
    "Add a todo-item to the list with the given state (uses create, the
     id-generating convenience; the board re-reads)."
    (let ((lid (if (string-eq (as-str list-id) "") (todo/ensure-inbox)
     (as-str list-id))))
      (create (quote todo-item)
        (dict-set* "{}"
          (list
            "list_id" lid
            "title" (as-str title)
            "notes" ""
            "state" (as-str state)
            "due" ""
            "position" "0"
            "done" "false"))))))

;; ---------------------------------------------------------------------
;; HTML rendering
;; ---------------------------------------------------------------------

(define kanban/card-html
  (lambda (list-id row columns)
    "Render one card as a draggable article with up/down/left/right
     buttons."
    (let ((id (as-str (car row))) (fields (cdr row)))
      (let ((title (as-str (dict-get fields "title")))
            (state (as-str (dict-get fields "state"))))
        (html/tag "article"
          (html/attrs
            (list
              (html/attr "class" "kb-card")
              (html/attr "draggable" "true")
              (html/attr "data-id" id)
              (html/attr "ondragstart" "kbDrag(event)")))
          (html/join
            (list
              (html/div (html/attr "class" "kb-card-title") (html/text
    title))
              (html/div (html/attr "class" "kb-card-meta")
                (html/text state))
              (html/div (html/attr "class" "kb-card-controls")
                (html/join
                  (list
                    (kanban/move-btn id list-id "left" "?")
                    (kanban/move-btn id list-id "up" "?")
                    (kanban/move-btn id list-id "down" "?")
                    (kanban/move-btn id list-id "right" "?")))))))))))

(define kanban/move-btn
  (lambda (id list-id action label)
    "Small POST form button for one move action on a card."
    (html/form "POST" "/kanban"
      (html/attrs
        (list
          (html/attr "class" "kb-move-form")
          (html/attr "hx-post" "/kanban")
          (html/attr "hx-target" "#kb-board")
          (html/attr "hx-swap" "innerHTML")))
      (html/join
```

```
        (list
          (html/hidden "action" action)
          (html/hidden "id" id)
          (html/hidden "list" list-id)
          (html/button (html/attr "type" "submit") label))))))

(define kanban/column-html
  (lambda (list-id state columns)
    "Render one column as a form (drop target) with its cards."
    (let ((cards (kanban/cards-in list-id state)))
      (let ((count (as-str (length cards))))
        (html/form "POST" "/kanban"
          (html/attrs
            (list
              (html/attr "class" "kb-col")
              (html/attr "data-state" state)
              (html/attr "hx-post" "/kanban")
              (html/attr "hx-target" "#kb-board")
              (html/attr "hx-swap" "innerHTML")
              (html/attr "ondragover" "event.preventDefault()")
              (html/attr "ondrop" "kbDrop(event)")))
          (html/join
            (list
              (html/hidden "action" "move")
              (html/hidden "id" "")
              (html/hidden "state" state)
              (html/hidden "index" "")
              (html/hidden "list" list-id)
              (html/div (html/attr "class" "kb-col-header")
                (html/join
                  (list
                    (html/text
                      (if (string-eq state "") "(none)" state))
                    (html/span (html/attr "class" "kb-count")
                      (html/text count))
                    (kanban/col-remove-btn list-id state))))
              (html/div (html/attr "class" "kb-cards")
                (kanban/cards-html list-id cards columns))
              (kanban/add-card-form list-id state)))))))

(define kanban/cards-html
  (lambda (list-id cards columns)
    "Concatenated card HTML for the card list."
    (if (null? cards)
      ""
      (string-append
        (kanban/card-html list-id (car cards) columns)
        (kanban/cards-html list-id (cdr cards) columns)))))

(define kanban/col-remove-btn
  (lambda (list-id state)
    "Confirming button to remove a column from the list."
    (html/form "POST" "/kanban"
      (html/attrs
        (list
          (html/attr "class" "kb-col-del")
          (html/attr "hx-post" "/kanban")
          (html/attr "hx-target" "#kb-board")
          (html/attr "hx-swap" "innerHTML")
          (html/attr "hx-confirm" "Remove this column? Cards stay in the
    other bucket.")))
      (html/join
        (list
          (html/hidden "action" "remove-column")
          (html/hidden "list" list-id)
          (html/hidden "name" state)
          (html/button (html/attr "type" "submit" ) "?"))))))

(define kanban/add-card-form
  (lambda (list-id state)
    "Per-column form to add a card with a title."
    (html/form "POST" "/kanban"
      (html/attrs
```

```
        (list
          (html/attr "class" "kb-add-card")
          (html/attr "hx-post" "/kanban")
          (html/attr "hx-target" "#kb-board")
          (html/attr "hx-swap" "innerHTML")))
      (html/join
        (list
          (html/hidden "action" "add-card")
          (html/hidden "list" list-id)
          (html/hidden "state" state)
          (html/text-input "title"
            (html/attrs
              (list
                (html/attr "placeholder" "+ add card")
                (html/attr "class" "kb-add-input"))))
          (html/button (html/attr "type" "submit") "?"))))))

(define kanban/other-bucket
  (lambda (list-id columns)
    "Cards whose state is not in the list's columns, rendered in a trailing bucket."
    (let ((all (todo/items-for-list list-id)))
      (let ((strays (list/filter
                      (lambda (row)
                        (not (kanban/in-columns? columns (as-str (dict-get (cdr row) "state")))))
                      all)))
        (if (null? strays)
          ""
          (html/tag "section"
            (html/attrs
              (list (html/attr "class" "kb-col kb-other")))
            (html/join
              (list
                (html/div (html/attr "class" "kb-col-header") (html/text "other"))
                (html/div (html/attr "class" "kb-cards")
                  (kanban/cards-html list-id strays columns)))))))))

(define kanban/in-columns?
  (lambda (columns state)
    "True if state is one of the column names."
    (if (null? columns)
      #f
      (if (string-eq (as-str (car columns)) state)
        #t
        (kanban/in-columns? (cdr columns) state)))))

(define kanban/add-column-form
  (lambda (list-id)
    "Form to add a new column to the list."
    (html/form "POST" "/kanban"
      (html/attrs
        (list
          (html/attr "class" "kb-add-col")
          (html/attr "hx-post" "/kanban")
          (html/attr "hx-target" "#kb-board")
          (html/attr "hx-swap" "innerHTML")))
      (html/join
        (list
          (html/hidden "action" "add-column")
          (html/hidden "list" list-id)
          (html/text-input "name"
            (html/attrs
              (list (html/attr "placeholder" "+ column"))))
          (html/button (html/attr "type" "submit") "add"))))))

(define kanban/list-picker
  (lambda (current-id)
    "Render a select of all todo-lists linking to /kanban?list=<id>."
    (let ((lists (todo/list/all)))
      (html/form "GET" "/kanban"
        (html/attr "class" "kb-list-picker")
```

```
        (html/join
          (list
            (html/label (html/attr "class" "muted") "board:")
            (html/select "list"
              (html/attrs (list (html/attr "onchange"
    "this.form.submit()")))
              (kanban/list-options lists current-id)))))))

(define kanban/list-options
  (lambda (lists current-id)
    "Concatenated <option> HTML for each todo-list, marking current-id
     selected."
    (if (null? lists)
      ""
      (let ((id (as-str (car (car lists))))
            (title (as-str (dict-get (cdr (car lists)) "title"))))
        (string-append
          (html/option id title (string-eq id (as-str current-id)))
          (kanban/list-options (cdr lists) current-id))))))

(define kanban/script
  (lambda ()
    "Minimal DnD shim: dragstart stores the card id; drop fills the column
     form."
    (render-html
      "<script>
function kbDrag(ev){
     window.__kbDrag=ev.target.dataset.id||ev.target.closest('.kb-card').dataset.id;
     }
function kbDrop(ev){
  ev.preventDefault();
  var f=ev.currentTarget;
  f.querySelector('[name=id]').value=window.__kbDrag||'';
  f.querySelector('[name=index]').value='';
  htmx.trigger(f,'submit');
}
</script>")))

(define kanban/board
  (lambda (list-id)
    "Render the full board: list picker + columns row + other bucket +
     add-column."
    (let ((cols (kanban/columns-for-list list-id)))
      (html/div (html/attr "id" "kb-board")
        (html/join
          (list
            (kanban/list-picker list-id)
            (html/div (html/attr "class" "kb-cols")
              (string-append
                (kanban/columns-html list-id cols)
                (kanban/add-column-form list-id)))
            (kanban/other-bucket list-id cols)
            (kanban/script)))))))

(define kanban/columns-html
  (lambda (list-id cols)
    "Concatenated column HTML for the column list."
    (if (null? cols)
      ""
      (string-append
        (kanban/column-html list-id (as-str (car cols)) cols)
        (kanban/columns-html list-id (cdr cols))))))

;; ---------------------------------------------------------------------
;; POST dispatch
;; ---------------------------------------------------------------------

(define kanban/handle-post
  (lambda ()
    "Read action and dispatch to the tree-action mutation."
    (let ((action (let ((a (request-param "action"))) (if (null? a) ""
     (as-str a)))))
```

```
      (let ((id (let ((x (request-param "id"))) (if (null? x) "" (as-str x))))
            (list-id (let ((x (request-param "list"))) (if (null? x) (todo/ensure-inbox) (as-str x))))
            (state (let ((x (request-param "state"))) (if (null? x) "" (as-str x))))
            (index (let ((x (request-param "index"))) (if (null? x) "" (as-str x))))
            (name (let ((x (request-param "name"))) (if (null? x) "" (as-str x))))
            (title (let ((x (request-param "title"))) (if (null? x) "" (as-str x)))))
        (if (string-eq action "move")
          (kanban/move-card id state (kanban/parse-index index))
          (if (string-eq action "up")
            (kanban/shift id list-id -1)
            (if (string-eq action "down")
              (kanban/shift id list-id 1)
              (if (string-eq action "left")
                (kanban/shift-col id list-id -1)
                (if (string-eq action "right")
                  (kanban/shift-col id list-id 1)
                  (if (string-eq action "add-card")
                    (kanban/add-card list-id title state)
                    (if (string-eq action "add-column")
                      (kanban/add-column list-id name)
                      (if (string-eq action "remove-column")
                        (kanban/remove-column list-id name)
                        (if (string-eq action "rename-column")
                          (kanban/rename-column list-id name state)
                          "unknown action"))))))))))))

(define kanban/parse-index
  (lambda (s)
    "Parse a decimal index string; empty/non-numeric -> 9999 (append)."
    (if (string-eq s "")
      9999
      (kanban/parse-index-safe s 0))))

(define kanban/parse-index-safe
  (lambda (s acc)
    "Accumulating decimal parse of an index string."
    (if (string-eq s "")
      acc
      (kanban/parse-index-safe
        (substring s 1 (string-length s))
        (+ (* acc 10) (kanban/digit (substring s 0 1)))))))

(define kanban/digit
  (lambda (c)
    "Integer value of a single digit char, or 0."
    (if (string-eq c "0") 0
      (if (string-eq c "1") 1
        (if (string-eq c "2") 2
          (if (string-eq c "3") 3
            (if (string-eq c "4") 4
              (if (string-eq c "5") 5
                (if (string-eq c "6") 6
                  (if (string-eq c "7") 7
                    (if (string-eq c "8") 8
                      (if (string-eq c "9") 9 0)))))))))))

(define kanban/out-of-bounds?
  (lambda (j len)
    "True if index j is outside [0, len)."
    (if (< j 0) #t (>= j len))))

(define kanban/shift
  (lambda (id list-id delta)
    "Shift card up/down within its column by swapping with the neighbor at
     (current-index + delta)."
    (let ((fields (get (quote todo-item) id)))
      (if (null? fields)
```

```
        "no such card"
        (let ((state (as-str (dict-get fields "state"))))
          (let ((cards (kanban/cards-in list-id state)))
            (let ((i (kanban/index-of cards id 0)))
              (let ((j (+ i delta)))
                (if (kanban/out-of-bounds? j (length cards))
                  "at edge"
                  (let ((other-row (nth cards j)))
                    (kanban/swap id (as-str (car other-row))))))))))

(define kanban/shift-col
  (lambda (id list-id delta)
    "Move card to the adjacent column (delta -1 = left, +1 = right)."
    (let ((fields (get (quote todo-item) id)))
      (if (null? fields)
        "no such card"
        (let ((state (as-str (dict-get fields "state")))
              (cols (kanban/columns-for-list list-id)))
          (let ((i (kanban/index-of-cols cols state 0)))
            (let ((j (+ i delta)))
              (if (kanban/out-of-bounds? j (length cols))
                "at edge"
                (kanban/move-card id (as-str (nth cols j)) 9999)))))))))

(define kanban/index-of-cols
  (lambda (cols state i)
    "0-based index of state within cols, or -1."
    (if (null? cols)
      -1
      (if (string-eq (as-str (car cols)) state)
        i
        (kanban/index-of-cols (cdr cols) state (+ i 1))))))

;; ---------------------------------------------------------------------
;; route handler
;; ---------------------------------------------------------------------

(define kanban/page
  (lambda ()
    "GET /kanban?list=<id> renders the board; POST dispatches a tree
     action."
    (let ((list-id
            (let ((x (request-param "list")))
              (if (null? x) (todo/ensure-inbox) (as-str x)))))
      (if (string-eq (request-method) "POST")
        (seq
          (kanban/handle-post)
          (if (hx-request?)
            (render-html (kanban/board list-id))
            (redirect (string-append "/kanban?list=" list-id))))
        (html/page
          (list
            (html/h1 "Kanban")
            (html/p* (html/attr "class" "muted")
              "A board view over your todo-list. Drag cards between
     columns, or use the arrow buttons. Columns are the list's states; edit
     them with + column.")
            (kanban/board list-id)))))))

(route "/kanban" "kanban/page")
```

173 libs/layout/lib.pp

```
;; package layout -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push

(package layout
  (export
    layout/card
    layout/cards
    layout/cta
    layout/hero
    layout/logo
    layout/page
    layout/page-with-css
    layout/section
    layout/style
    layout/theme
    layout/theme-vars
    layout/neocities-banner
    layout/neocities-foot
    layout/neocities-style
    layout/neocities-theme
    layout/neocities-top))

(define layout/card
  (lambda
    (title body)
    "Render a titled HTML card wrapping body."
    (html/div
      (html/attr "class" "layout-card")
      (html/join
        (list
          (html/tag
            "h3"
            (html/attr "class" "layout-card-title")
            (html/text title))
          (html/div (html/attr "class" "layout-card-body") body))))))

(define layout/cards
  (lambda
    (card-htmls)
    "Render a grid of card HTML snippets."
    (html/div
      (html/attr "class" "layout-cards")
      (html/join card-htmls))))

(define layout/cta
  (lambda
    (href label)
    "Render a call-to-action link to href with label."
    (html/tag
      "a"
      (html/attrs
        (list
          (html/attr "class" "layout-cta")
          (html/attr "href" href)))
      (html/text label))))

(define layout/hero
  (lambda
    (title tagline actions-html)
    "Render a hero section with logo, title, tagline, and actions."
    (html/div
      (html/attr "class" "layout-hero")
      (html/join
        (list
          (layout/logo)
          (html/tag
            "h1"
            (html/attr "class" "layout-hero-title")
            (html/text title))
          (html/tag
            "p"
```

```
            (html/attr "class" "layout-hero-tagline")
            (html/text tagline))
          (html/div
            (html/attr "class" "layout-hero-actions")
            actions-html))))))

(define layout/logo
  (lambda
    ()
    "Render the Pricklypear logo image."
    (html/void
      "img"
      (html/attrs
        (list
          (html/attr "src" "/assets/pplogo.png")
          (html/attr "alt" "Pricklypear")
          (html/attr "class" "layout-logo")
          (html/attr "width" "160")
          (html/attr "height" "156"))))))

(define layout/nc-badge
  (lambda
    (href bg fg label)
    "Render a badge link with background color and label."
    (html/tag
      "a"
      (html/attrs
        (list
          (html/attr "class" "nc-badge")
          (html/attr "href" href)
          (html/attr
            "style"
            (string-append "background:" bg ";color:" fg))))
      (html/text label))))

(define layout/neocities-banner
  (lambda
    ()
    "Render the under-construction banner strip."
    (html/div
      (html/attr "class" "nc-banner")
      (html/join
        (list
          (html/tag
            "span"
            (html/attr "class" "nc-construction")
            (html/text "UNDER CONSTRUCTION"))
          (html/text " ")
          (html/tag
            "span"
            (html/attr "class" "nc-blink")
            (html/text "welcome to my homepage!!"))
          (html/text
            " -- last updated 2026 -- "))))))

(define layout/neocities-foot
  (lambda
    ()
    "Render the neocities footer with webring, badges, counter, and
     colophon."
    (html/div
      (html/attr "class" "nc-foot")
      (html/join
        (list
          (html/div
            (html/attr "class" "nc-webring")
            (html/join
              (list
                (html/text "[ ")
                (html/tag
                  "a"
                  (html/attr "href" "/welcome")
                  (html/text "prev"))
```

```
            (html/text " | ")
            (html/tag
              "a"
              (html/attr "href" "/background")
              (html/text "list"))
            (html/text " | ")
            (html/tag
              "a"
              (html/attr "href" "/roadmap")
              (html/text "random"))
            (html/text " | ")
            (html/tag
              "a"
              (html/attr "href" "/code")
              (html/text "next"))
            (html/text " ]  the pricklypear webring"))))
        (html/div
          (html/attr "class" "nc-badges")
          (html/join
            (list
              (layout/nc-badge
                "/welcome"
                "#000080"
                "#ff0"
                "BEST VIEWED IN NETSCAPE")
              (layout/nc-badge
                "/code"
                "#000"
                "#fff"
                "HAND-CODED LISP")
              (layout/nc-badge
                "/book.pdf"
                "#600"
                "#fff"
                "NO JAVASCRIPT")
              (layout/nc-badge
                "/code"
                "#070"
                "#fff"
                "POSTGRES POWERED")
              (layout/nc-badge
                "/agent.txt"
                "#036"
                "#fff"
                "AGENT READY")
              (layout/nc-badge
                "/guestbook"
                "#609"
                "#fff"
                "SIGN GUESTBOOK")
              (layout/nc-badge "/welcome" "#fc0" "#000" "Y2K READY")
              (layout/nc-badge
                "/welcome"
                "#fff"
                "#000"
                "VALID HTML 4.01"))))
        (html/div
          (html/attr "class" "nc-counter")
          (html/join
            (list
              (html/tag
                "span"
                (html/attr "class" "nc-counter-label")
                (html/text "you are visitor"))
              (html/tag
                "span"
                (html/attr "class" "nc-counter-digits")
                (html/text "0000042"))
              (html/tag
                "span"
                (html/attr "class" "nc-counter-label")
                (html/text "since 2026")))))
        (html/void "hr" (html/attr "class" "nc-rainbow-hr"))
```

```
          (html/div
            (html/attr "class" "nc-colophon")
            (html/text
              "this site is hand-coded in a lisp machine and rendered by a cactus ? powered by postgres, htmx, and questionable life choices. no javascript was harmed. best viewed at 800x600.")))))))

(define layout/neocities-marquee-msg
  (lambda
    ()
    "Render the scrolling marquee welcome message."
    (html/tag
      "span"
      (html/attr "class" "nc-marquee-msg")
      (html/text
        "? UNDER CONSTRUCTION ? welcome to pricklypear ? a lisp machine that lives in postgres ? sign the guestbook ? best viewed in netscape 4.0 at 800x600 ? built in public ? "))))

(define layout/neocities-style
  (lambda
    ()
    "Render the neocities theme as a <style> element."
    (html/tag "style" "" (layout/neocities-theme))))

(define layout/neocities-theme
  (lambda
    ()
    "Return the neocities CSS stylesheet for the retro cactus look."
    (html/join
      (list

      "body{background-color:#2a6a6a;background-image:repeating-linear-gradient(45deg,rgba(255,255,255,.05) 0 2px,transparent 2px 14px),repeating-linear-gradient(-45deg,rgba(0,0,0,.06) 0 2px,transparent 2px 14px)}"
        ".layout-page{background:#faf7f4;border:2px solid #000;box-shadow:6px 6px 0 #000}"
        ".layout-hero{border:2px solid #000;background:linear-gradient(180deg,#ffd1a4,#ffe9c8);margin:1rem 0;box-shadow:4px 4px 0 #000}"
        ".layout-card{box-shadow:3px 3px 0 #000}"
        "a.layout-cta{box-shadow:3px 3px 0 #000}"
        "a.layout-cta:hover{background:#000;color:#ff0}"
        "a.layout-cta:visited:hover{color:#ff0}"
        ".nc-marquee-bar{background:#000;color:#ff0;border-bottom:2px solid #000;overflow:hidden;white-space:nowrap;font-family:monospace;font-size:.85rem;padding:.3rem 0}"

      ".nc-marquee-track{display:inline-block;white-space:nowrap;animation:nc-scroll 30s linear infinite}"
        "@keyframes nc-scroll{0%{transform:translateX(0)}100%{transform:translateX(-50%)}}"
        ".nc-blink{animation:nc-blink 1.1s steps(2,start) infinite}"
        "@keyframes nc-blink{50%{opacity:0}}"
        ".nc-banner{text-align:center;padding:.6rem .5rem;font-family:monospace;font-size:.85rem}"

      ".nc-construction{display:inline-block;background:repeating-linear-gradient(45deg,#fc0 0 12px,#000 12px 16px);color:#000;font-weight:700;padding:.12rem .4rem;border:2px solid #000;letter-spacing:.05em;text-shadow:0 0 2px #fc0,0 0 2px #fc0}"
        ".nc-foot{background:#000040;color:#cdd;border-top:2px solid #000;padding:1rem .75rem;text-align:center;font-family:monospace;font-size:.8rem}"
        ".nc-foot a{color:#9cf}"
        ".nc-webring{font-family:monospace;font-size:.8rem;margin:.4rem 0}"
        ".nc-webring a{color:#ff0;text-decoration:none}"
        ".nc-webring a:hover{text-decoration:underline}"

      ".nc-badges{display:flex;flex-wrap:wrap;gap:.4rem;justify-content:center;margin:.6rem 0}"
```

```
264
     "a.nc-badge{display:inline-flex;align-items:center;justify-content:center;width:88px;height:31px;border:2px
     outset
     #bbb;font-family:monospace;font-size:8px;line-height:1.1;text-align:center;text-decoration:none;padding:1px;box-sizing:
265      "a.nc-badge:hover{border-style:inset}"
266
     ".nc-counter{display:inline-flex;align-items:center;gap:.35rem;font-family:monospace;font-size:.78rem;margin:.5rem
     0}"
267      ".nc-counter-digits{background:#000;border:2px inset
     #555;color:#0f0;padding:.12rem
     .4rem;font-family:monospace;font-weight:700;letter-spacing:.18em}"
268      ".nc-counter-label{color:#9cf}"
269
     ".nc-rainbow-hr{height:4px;border:0;background:linear-gradient(90deg,red,orange,#fc0,green,blue,#a0f);margin:1rem
     auto;width:80%}"
270
     ".nc-colophon{color:#88a;font-size:.72rem;margin-top:.4rem;line-height:1.5}"
271      ".err{color:#c00;font-weight:700}"
272      ".ok{color:#070;font-weight:700}"
273      ".gb-list{list-style:none;padding:0;margin:1rem 0}"
274      ".gb-entry{border:2px solid
     #000;background:#fff;padding:.6rem;margin:.5rem 0;box-shadow:3px 3px 0
     #000}"
275      ".gb-name{font-weight:700;color:#000}"
276      ".gb-site{color:#000080;text-decoration:underline;font-size:.85rem}"
277      ".gb-msg{margin-top:.3rem;color:#222;line-height:1.4}"
278      ".gb-form{max-width:30rem}"
279      ".gb-form input,.gb-form
     textarea{width:100%;font-family:monospace;padding:.3rem;margin:.2rem
     0;border:2px solid #000;background:#fff}"))))
280
281  (define layout/neocities-top
282    (lambda
283      ()
284      "Render the top marquee bar."
285      (html/div
286        (html/attr "class" "nc-marquee-bar")
287        (html/div
288          (html/attr "class" "nc-marquee-track")
289          (html/join
290            (list
291              (layout/neocities-marquee-msg)
292              (layout/neocities-marquee-msg)))))))
293
294  (define layout/page
295    (lambda
296      (parts)
297      "Render a full neocities page from a list of HTML parts."
298      (render-html
299        (html/join
300          (list
301            (layout/style)
302            (layout/neocities-style)
303            (layout/neocities-top)
304            (html/div
305              (html/attr "class" "layout-page")
306              (html/join
307                (list (layout/neocities-banner) (html/join parts))))
308            (layout/neocities-foot))))))
309
310  (define layout/page-with-css
311    (lambda
312      (parts)
313      "Render a page from parts with only the base stylesheet."
314      (render-html
315        (html/join
316          (list
317            (layout/style)
318            (html/div
319              (html/attr "class" "layout-page")
320              (html/join parts)))))))
321
322  (define layout/section
```

```
  (lambda
    (title body)
    "Render a titled section wrapping body."
    (html/div
      (html/attr "class" "layout-section")
      (html/join
        (list
          (html/tag
            "h2"
            (html/attr "class" "layout-section-title")
            (html/text title))
          (html/div
            (html/attr "class" "layout-section-body")
            body))))))

(define layout/style
  (lambda
    ()
    "Render the base layout theme as a <style> element."
    (html/tag "style" "" (css/render (layout/theme)))))

(define layout/theme
  (lambda
    ()
    "Return the base layout CSS stylesheet."
    (css/sheet
      (list
        (css/rule
          ".layout-hero"
          (list
            "padding"
            "2.5rem 1.25rem 2rem"
            "text-align"
            "center"
            "border-bottom"
            "2px solid #000"
            "background"
            "#faf7f4"
            "margin"
            "0 0 1.25rem"))
        (css/rule
          ".layout-logo"
          (list
            "display"
            "block"
            "width"
            "10rem"
            "height"
            "auto"
            "max-width"
            "40vw"
            "margin"
            "0 auto 1rem"
            "image-rendering"
            "auto"))
        (css/rule
          ".layout-hero-title"
          (list
            "font-size"
            "2rem"
            "font-weight"
            "700"
            "margin"
            "0 0 .5rem"
            "letter-spacing"
            "-0.02em"))
        (css/rule
          ".layout-hero-tagline"
          (list
            "font-size"
            "1.1rem"
            "max-width"
            "36rem"
```

```
            "margin"
            "0 auto 1.25rem"
            "line-height"
            "1.5"
            "color"
            "#333"))
          (css/rule
            ".layout-hero-actions"
            (list
              "display"
              "flex"
              "flex-wrap"
              "wrap"
              "gap"
              ".5rem"
              "justify-content"
              "center"
              "align-items"
              "center"))
          (css/rule
            "a.layout-cta"
            (list
              "display"
              "inline-block"
              "padding"
              ".45rem .9rem"
              "border"
              "2px solid #000"
              "background"
              "#fff"
              "color"
              "#000"
              "text-decoration"
              "none"
              "font-weight"
              "700"
              "font-size"
              ".95rem"))
          (css/rule
            "a.layout-cta:hover"
            (list "background" "#000" "color" "#fff"))
          (css/rule "a.layout-cta:visited" (list "color" "#000"))
          (css/rule
            "a.layout-cta:visited:hover"
            (list "color" "#fff"))
          (css/rule
            ".layout-cards"
            (list
              "display"
              "grid"
              "grid-template-columns"
              "repeat(auto-fit,minmax(14rem,1fr))"
              "gap"
              ".75rem"
              "margin"
              "1rem 0"))
          (css/rule
            ".layout-card"
            (list
              "border"
              "2px solid #000"
              "background"
              "#fff"
              "padding"
              ".85rem"
              "text-align"
              "left"))
          (css/rule
            ".layout-card-title"
            (list "margin" "0 0 .4rem" "font-size" "1.05rem"))
          (css/rule
            ".layout-card-body"
            (list
```

```
            "font-size"
            ".95rem"
            "line-height"
            "1.45"
            "color"
            "#222"))
          (css/rule
            ".layout-section"
            (list "margin" "1.5rem 0" "padding" "0 .25rem"))
          (css/rule
            ".layout-section-title"
            (list
              "font-size"
              "1.25rem"
              "border-bottom"
              "1px solid #808080"
              "padding-bottom"
              ".25rem"
              "margin"
              "0 0 .75rem"))
          (css/rule
            ".layout-section-body"
            (list "line-height" "1.55" "max-width" "40rem"))
          (css/rule
            ".ast-table"
            (list
              "border-collapse"
              "collapse"
              "margin"
              "1rem 0"
              "max-width"
              "40rem"
              "width"
              "100%"))
          (css/rule
            ".ast-table th"
            (list
              "text-align"
              "left"
              "border"
              "1px solid #808080"
              "padding"
              ".3rem .5rem"
              "background"
              "#f0ece6"
              "font-family"
              "monospace"
              "font-size"
              ".8rem"))
          (css/rule
            ".ast-table td"
            (list
              "border"
              "1px solid #808080"
              "padding"
              ".3rem .5rem"
              "vertical-align"
              "top"
              "font-family"
              "monospace"
              "font-size"
              ".8rem"))
          (css/rule
            ".ast-table pre"
            (list
              "margin"
              "0"
              "white-space"
              "pre-wrap"
              "word-break"
              "break-word"))
          (css/rule
            "pre"
```

```
          (list
            "background"
            "#f6f3ee"
            "border"
            "1px solid #c9c2b6"
            "padding"
            ".6rem"
            "overflow-x"
            "auto"
            "font-size"
            ".82rem"
            "line-height"
            "1.4"))
        (css/rule
          ".layout-page"
          (list
            "max-width"
            "52rem"
            "margin"
            "0 auto"
            "padding"
            "0 .75rem 2rem"))
        (css/rule
          ".layout-foot"
          (list
            "margin-top"
            "2rem"
            "padding-top"
            ".75rem"
            "border-top"
            "1px solid #808080"
            "font-size"
            ".85rem"
            "color"
            "#333"
            "text-align"
            "center"))
        (css/media
          "max-width:40rem"
          (list
            (css/rule
              ".layout-hero"
              (list "padding" "1.5rem .75rem 1.25rem"))
            (css/rule
              ".layout-hero-title"
              (list "font-size" "1.55rem")))))))

(define layout/theme-vars
  (lambda
    ()
    "Return the CSS custom-property (theme variable) stylesheet."
    (css/sheet
      (list
        (css/rule
          ":root"
          (list
            "--cactus-green"
            "#2d5a27"
            "--cactus-light"
            "#4a7c43"
            "--cactus-lighter"
            "#6b9b5c"
            "--cactus-yellow"
            "#f4d03f"
            "--cactus-flower"
            "#e74c3c"
            "--cactus-spike"
            "#1a1a1a"
            "--cactus-shadow"
            "4px 4px 0 #1a1a1a"))))))


(define layout/card-css
```

```
(lambda
  ()
  "Render the layout component stylesheet (extracted from ui/theme)."
  (css/sheet
    (list
      (css/rule
        ".card"
        (list
          "background"
          "var(--panel)"
          "border"
          "2px solid #000"
          "padding"
          ".75rem"
          "margin"
          "0 0 .75rem"))))))
```

174 libs/lens/lib.pp

```
;; package lens -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push
```

175 libs/list/lib.pp

```
;; package list -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push

(define list/filter
  (lambda
    (p xs)
    "Return the elements of xs for which p is true."
    (if
      (null? xs)
      (list)
      (if
        (p (car xs))
        (cons (car xs) (list/filter p (cdr xs)))
        (list/filter p (cdr xs))))))

(define list/foldl
  (lambda
    (f acc xs)
    "Left-fold f over xs with initial accumulator acc."
    (if
      (null? xs)
      acc
      (list/foldl f (f acc (car xs)) (cdr xs)))))

(define list/map
  (lambda
    (f xs)
    "Apply f to each element of xs, returning a new list."
    (if
      (null? xs)
      (list)
      (cons (f (car xs)) (list/map f (cdr xs))))))

(define list/range
  (lambda (n) "Build the list (0 1 ... n-1) of length n." (list/range-acc n
     (list))))

(define list/range-acc
  (lambda
    (n acc)
    "Accumulator helper for list/range."
    (if
      (<= n 0)
      acc
      (list/range-acc (- n 1) (cons (- n 1) acc)))))

(define list/take
  (lambda (xs n) "Return the first n elements of xs." (list/take-acc xs n
     (list))))

(define list/take-acc
  (lambda
    (xs n acc)
    "Accumulator helper for list/take."
    (if
      (or (null? xs) (<= n 0))
      (reverse acc)
      (list/take-acc (cdr xs) (- n 1) (cons (car xs) acc)))))
```

176 libs/menu/lib.pp

```
;; package menu -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push

(define menu/bar
  (lambda
    (items)
    "Render a top-level menu bar from a list of menu items."
    (html/join
      (list/map (lambda (i) (menu/render-d i 0)) items))))

(define menu/group
  (lambda (label kids) "A nested-menu item: label plus child items." (list
     "group" (as-str label) kids)))

(define menu/link
  (lambda
    (label href)
    "A menu link item with label and href."
    (list "link" (as-str label) (as-str href))))

(define menu/raw
  (lambda (html-str) "A menu item that renders raw HTML." (list "raw"
     (as-str html-str) "")))

;; depth-aware renderer: d=0 top-level nav, d>0 nested fly-out.
;; Top-level groups share name="pp-nav" so the browser's native
;; <details name> accordion auto-closes siblings (replaces the old
;; chrome toggle JS). Nested groups get name="pp-sub-<label>" per
;; fly-out column, so opening one submenu closes its siblings.
(define menu/render-d
  (lambda
    (item d)
    "Depth-aware renderer: d=0 top-level nav, d>0 nested fly-out."
    (let
      ((kind (car item))
        (label (car (cdr item)))
        (rest (car (cdr (cdr item)))))
      (if
        (string-eq kind "link")
        (html/a rest (html/text label))
        (if
          (string-eq kind "group")
          (html/tag
            "details"
            (html/attrs
              (list
                (html/attr "class" "menu-drop")
                (html/attr
                  "name"
                  (if
                    (= d 0)
                    "pp-nav"
                    (string-append "pp-sub-" (as-str label))))))
            (html/join
              (list
                (html/tag
                  "summary"
                  (html/attr "class" "menu-drop-btn")
                  (html/text label))
                (html/div
                  (html/attr "class" "menu-drop-panel")
                  (html/join
                    (list/map
                      (lambda (k) (menu/render-d k (+ d 1)))
                      rest))))))
          (if (string-eq kind "raw") label ""))))))

(define menu/render
  (lambda (item) "Render a menu item at top level." (menu/render-d item 0)))
```

177 libs/notes/capture/lib.pp

```
;; package notes/capture -- exported from live branch admin/personal
;; Sync: scripts/pp_sync.py pull|push

(define notes/capture
  (lambda
    (title body)
    "Capture title/body as a node in the Inbox tree."
    (let
      ((prev (notes/ensure-inbox)))
      (let
        ((tree (notes/get-outline prev))
          (t (notes/capture-title title body))
          (b (as-str body)))
        (let
          ((child
            (outline/set-body
              (outline/node (notes/capture-node-id tree) t)
              b)))
          (let
            ((new-tree (outline/add-child-by-id tree "root" child)))
            (seq
              (notes/upsert
                "inbox"
                "Inbox"
                new-tree
                "inbox"
                "inbox"
                "")
              "inbox")))))))

(define notes/capture/add
  (lambda
    (title body)
    "Add a captured node to the Inbox document tree."
    (let
      ((prev (notes/doc/ensure-inbox)))
      (let
        ((tree (notes/doc/outline prev))
          (t (notes/capture/title title body))
          (b (as-str body)))
        (let
          ((child
            (outline/set-body
              (outline/node (notes/capture/node-id tree) t)
              b)))
          (let
            ((new-tree (outline/add-child-by-id tree "root" child)))
            (seq
              (notes/doc/upsert
                "inbox"
                "Inbox"
                new-tree
                "inbox"
                "inbox"
                "")
              "inbox")))))))

(define notes/capture/form
  (lambda
    ()
    "Render the capture form for posting new nodes."
    (html/form
      "POST"
      "/notes"
      (html/attrs
        (list
          (html/attr "class" "card capture-card")
          (html/attr "hx-post" "/notes")
          (html/attr "hx-target" "#capture-status")
          (html/attr "hx-swap" "innerHTML")
          (html/attr
```

```
            "hx-on::after-request"
            "if(event.detail.successful){this.reset()}")))
      (html/join
        (list
          (html/hidden "action" "capture")
          (html/h2 "Capture")
          (html/p*
            (html/attr "class" "muted")
            "Drops a node into the Inbox note.")
          (html/label (html/attr "for" "capture-title") "Title")
          (html/text-input
            "title"
            (html/attrs
              (list
                (html/attr "id" "capture-title")
                (html/attr "placeholder" "optional")
                (html/attr "autocomplete" "off"))))
          (html/label (html/attr "for" "capture-body") "Note")
          (html/textarea
            "body"
            (html/attrs
              (list
                (html/attr "id" "capture-body")
                (html/attr "rows" "3")
                (html/attr "class" "capture-body")
                (html/attr "placeholder" "Quick capture?")
                (html/attr "required" "required")))
            "")
          (html/div
            (html/attr "class" "capture-row")
            (html/join
              (list
                (html/button (html/attr "type" "submit") "Capture")
                (html/a "/notes?slug=inbox" "Open inbox"))))
          (html/div (html/attr "id" "capture-status") ""))))))

(define notes/capture/handle
  (lambda
    ()
    "Handle a capture POST, saving output or showing status."
    (let
      ((title (request-param "title"))
        (body (request-param "body")))
      (let
        ((b (if (null? body) "" (as-str body))))
        (if
          (string-eq (string-trim b) "")
          (if
            (hx-request?)
            (render-html
              (notes/capture/status #f "Write something first."))
            (redirect "/"))
          (seq
            (notes/capture/add (if (null? title) "" title) b)
            (if
              (hx-request?)
              (render-html
                (notes/capture/status #t "Saved to Inbox."))
              (redirect "/notes?slug=inbox")))))))))

(define notes/capture/nav
  (lambda
    ()
    "Render the drop-down capture form used in site navigation."
    (html/tag
      "details"
      (html/attrs
        (list
          (html/attr "class" "menu-drop capture-drop")
          (html/attr "name" "pp-nav")))
      (html/join
        (list
          (html/tag
```

```
            "summary"
            (html/attr "class" "menu-drop-btn")
            "capture")
          (html/div
            (html/attr "class" "menu-drop-panel capture-panel")
            (html/form
              "POST"
              "/notes"
              (html/attrs
                (list
                  (html/attr "class" "capture-nav-form")
                  (html/attr "hx-post" "/notes")
                  (html/attr "hx-target" "#capture-status")
                  (html/attr "hx-swap" "innerHTML")
                  (html/attr
                    "hx-on::after-request"
                    "if(event.detail.successful){this.reset()}")))
              (html/join
                (list
                  (html/hidden "action" "capture")
                  (html/text-input
                    "title"
                    (html/attrs
                      (list
                        (html/attr "placeholder" "title (optional)")
                        (html/attr "autocomplete" "off")
                        (html/attr "class" "capture-nav-title"))))
                  (html/textarea
                    "body"
                    (html/attrs
                      (list
                        (html/attr "rows" "3")
                        (html/attr "class" "capture-body")
                        (html/attr "placeholder" "Quick capture?")
                        (html/attr "required" "required")))
                    "")
                  (html/div
                    (html/attr "class" "capture-row")
                    (html/join
                      (list
                        (html/button (html/attr "type" "submit") "Save")
                        (html/a "/notes?slug=inbox" "inbox"))))
                  (html/div (html/attr "id" "capture-status") ""))))))))))

(define notes/capture/node-id
  (lambda
    (tree)
    "Build a unique-ish node id from tree size and encoded length."
    (string-append
      "c"
      (as-str (count (outline/kids tree)))
      "-"
      (as-str (string-length (outline/encode tree))))))

(define notes/capture/page
  (lambda
    ()
    "Serve the capture page, handling POSTs."
    (if
      (string-eq (request-method) "POST")
      (notes/capture/handle)
      (html/page
        (list (html/h1 "Capture") (notes/capture/form))))))

(define notes/capture/status
  (lambda
    (ok msg)
    "Render the ok/err capture status line."
    (html/p* (html/attr "class" (if ok "ok" "err")) msg)))

(define notes/capture/title
  (lambda
    (title body)
```

```
    "Derive a node title from title, else the body, truncated."
    (let
      ((t (string-trim (as-str title))))
      (if
        (not (string-eq t ""))
        t
        (let
          ((b (string-trim (as-str body))))
          (if
            (string-eq b "")
            "capture"
            (if
              (<= (string-length b) 48)
              b
              (string-append (substring b 0 45) "..."))))))))
```

178 libs/notes/doc/lib.pp

```
;; package notes/doc -- exported from live branch admin/personal
;; Sync: scripts/pp_sync.py pull|push

(define notes/doc/daily
  (lambda
    (day)
    "Fetch the daily note for day; create it if absent, returning its row."
    (let
      ((slug (string-append "daily-" (as-str day))))
      (if
        (null? (notes/doc/get slug))
        (seq
          (notes/doc/upsert
            slug
            (string-append "Daily " (as-str day))
            (notes/doc/empty-outline
              (string-append "Daily " (as-str day)))
            "daily"
            "daily"
            (as-str day))
          (notes/doc/get slug))
        (notes/doc/get slug)))))

(define notes/doc/day
  (lambda (n) "The day field of a doc row n." (car (cdr (cdr (cdr (cdr (cdr
    (cdr n))))))))

(define notes/doc/delete
  (lambda
    (slug)
    "Delete the note with slug (roam links and tree path), returning slug."
    (let
      ((row (notes/doc/row (as-str slug))))
      (if
        (null? row)
        ()
        (seq
          (roam/delete-from-slug (as-str slug))
          (let
            ((p (notes/doc/row-path row)))
            (if
              (and (notes/tree/enabled?) (not (string-eq p "")))
              (tree/delete p "recursive")
              (delete (quote note) (car row))))
          slug)))))

(define notes/doc/empty-outline
  (lambda (title) "Build a root outline node with title." (outline/node
    "root" title)))

(define notes/doc/ensure-inbox
  (lambda
    ()
    "Ensure the inbox note exists, creating it if absent and returning it."
    (if
      (null? (notes/doc/get "inbox"))
      (seq
        (notes/doc/upsert
          "inbox"
          "Inbox"
          (notes/doc/empty-outline "Inbox")
          "inbox"
          "inbox"
          "")
        (notes/doc/get "inbox"))
      (notes/doc/get "inbox"))))

(define notes/doc/folder
  (lambda
    (n)
    "The folder field of a doc row n; empty when absent."
```

```
    (if
      (null? n)
      ""
      (let
        ((f (car (cdr (cdr (cdr (cdr (cdr (cdr (cdr n)))))))))
        (if (null? f) "" (as-str f))))))

;; Display folder for the index: strip import garden/ prefix; kebab labels.
(define notes/doc/display-folder
  (lambda
    (folder)
    "Display folder for the index: strip a garden/ prefix and kebab labels."
    (let
      ((f (as-str folder)))
      (let
        ((stripped
          (if
            (string-starts-with? f "garden/")
            (string-replace f "garden/" "")
            (if (string-eq f "garden") "" f))))
        (string-replace stripped "_" "-"))))))

(define notes/doc/folder-names
  (lambda
    ()
    "Distinct folder names across all visible notes in index order."
    (list/foldl
      (lambda
        (acc row)
        (let
          ((f (notes/doc/row-folder row)))
          (if
            (notes/doc/folder-seen? acc f)
            acc
            (append acc (list f)))))
      (list)
      (notes/doc/index-list))))

(define notes/doc/folder-seen?
  (lambda
    (xs f)
    "True if f is already in the string list xs."
    (if
      (null? xs)
      #f
      (if
        (string-eq (car xs) f)
        #t
        (notes/doc/folder-seen? (cdr xs) f)))))

(define notes/doc/get
  (lambda
    (slug)
    "Fetch a doc row for slug as (id slug title outline tags kind day
     folder); () when absent."
    (let
      ((row (notes/doc/row slug)))
      (if
        (null? row)
        ()
        (let
          ((f (cdr row)))
          (list
            (car row)
            (dict-get f "slug")
            (dict-get f "title")
            (outline/decode (dict-get f "outline"))
            (dict-get f "tags")
            (dict-get f "kind")
            (dict-get f "day")
            (let
              ((folder (dict-get f "folder")))
              (if (null? folder) "" folder)))))))))
```

```
142
143 (define notes/doc/hidden?
144   (lambda
145     (row)
146     "True if row should be hidden (inbox/hidden kind or hidden tag)."
147     (let
148       ((f (cdr row)))
149       (let
150         ((kind (as-str (dict-get f "kind")))
151           (tags (as-str (dict-get f "tags")))
152           (slug (as-str (dict-get f "slug"))))
153         (if
154           (string-eq kind "inbox")
155           #t
156           (if
157             (string-eq kind "hidden")
158             #t
159             (if
160               (string-eq slug "inbox")
161               #t
162               (string-contains? tags "hidden")))))))))
163
164 (define notes/doc/index-list
165   (lambda
166     ()
167     "All non-hidden notes."
168     (list/filter
169       (lambda (row) (not (notes/doc/hidden? row)))
170       (notes/doc/list))))
171
172 (define notes/doc/kind
173   (lambda (n) "The kind field of a doc row n." (car (cdr (cdr (cdr (cdr
      (cdr n)))))))
174
175 (define notes/doc/list
176   (lambda
177     ()
178     ;; index-read-path I4 / F1: tree only -- no data_rows fallback
179     "List all notes from the tree when enabled, else ()."
180     (if (notes/tree/enabled?) (notes/tree/list-notes) (list))))
181
182 (define notes/doc/list-folder
183   (lambda
184     (folder)
185     "All visible notes in the given folder."
186     (let
187       ((want (as-str folder)))
188       (list/filter
189         (lambda
190           (row)
191           (string-eq (notes/doc/row-folder row) want))
192         (notes/doc/index-list)))))
193
194 (define notes/doc/make-fields
195   (lambda
196     (slug
197       title
198       outline
199       tags
200       kind
201       day
202       folder)
203     "Build the fields JSON string for a note row."
204     (dict-set*
205       "{}"
206       (list
207         "slug"
208         (as-str slug)
209         "title"
210         (as-str title)
211         "outline"
212         (outline/encode outline)
213         "tags"
```

```
        (as-str tags)
        "kind"
        (as-str kind)
        "day"
        (as-str day)
        "folder"
        (as-str folder)))))

(define notes/doc/outline
  (lambda (n) "The decoded outline of a doc row n." (car (cdr (cdr (cdr
    n))))))

(define notes/doc/row
  (lambda
    (slug)
    "Fetch the raw note row for slug from tree or data_rows; () when
     absent."
    (let
      ((tr
        (if
          (notes/tree/enabled?)
          (notes/tree/find-slug slug)
          ())))
      (if
        (null? tr)
        (let
          ((rs (find-rows (quote note) "slug" (as-str slug) 1)))
          (if (null? rs) () (car rs)))
        tr))))

(define notes/doc/row-folder
  (lambda
    (row)
    "The folder field of a raw row; empty when absent."
    (if
      (null? row)
      ""
      (let
        ((f (dict-get (cdr row) "folder")))
        (if (null? f) "" (as-str f))))))

(define notes/doc/row-path
  (lambda
    (row)
    "The tree path of a raw row; empty when absent."
    (if
      (null? row)
      ""
      (let
        ((p (dict-get (cdr row) "_path")))
        (if (null? p) "" (as-str p))))))

;; Postgres-backed garden search (notes.search-modal / BORGE-1785605474).
;; Returns list of hit JSON strings -- never enumerates the garden in
    Nopales.
(define notes/doc/search
  (lambda
    (q)
    "Full-text search over notes; returns a list of hit JSON strings."
    (let
      ((needle (string-trim (as-str q))))
      (if
        (string-eq needle "")
        (list)
        (list/map
          (lambda (j) (if (null? j) "{}" (as-str j)))
          (tree/search-text needle 20))))))

(define notes/doc/set-folder
  (lambda
    (slug folder)
    "Move the note with slug into folder (tree move or data_rows update)."
    (let
```

```
      ((row (notes/doc/row (as-str slug))))
      (if
        (null? row)
        ()
        (if
          (notes/tree/enabled?)
          (let
            ((old (notes/doc/row-path row))
              (newp (notes/tree/note-path folder (as-str slug))))
            (if
              (string-eq old "")
              (notes/doc/upsert-in
                slug
                (dict-get (cdr row) "title")
                (outline/decode (dict-get (cdr row) "outline"))
                (dict-get (cdr row) "tags")
                (dict-get (cdr row) "kind")
                (dict-get (cdr row) "day")
                folder)
              (if (string-eq old newp) old (tree/move old newp))))
          (update-field
            (quote note)
            (car row)
            "folder"
            (as-str folder)))))))

(define notes/doc/show-archived?
  (lambda
    ()
    "True when the request asks to show archived notes."
    (string-eq (as-str (request-param "show")) "archived")))

(define notes/doc/tags
  (lambda (n) "The tags field of a doc row n." (car (cdr (cdr (cdr (cdr
    n)))))))

(define notes/doc/title
  (lambda (n) "The title field of a doc row n." (car (cdr (cdr n)))))

(define notes/doc/upsert
  (lambda
    (slug title outline tags kind day)
    "Create or update the note with slug, preserving its folder, returning
     slug."
    (let
      ((row (notes/doc/row slug)))
      (notes/doc/upsert-in
        slug
        title
        outline
        tags
        kind
        day
        (if (null? row) "" (notes/doc/row-folder row))))))

(define notes/doc/upsert-in
  (lambda
    (slug
      title
      outline
      tags
      kind
      day
      folder)
    "Write the note with slug into the given folder, returning slug."
    (if
      (notes/tree/enabled?)
      (let
        ((prev (notes/doc/row slug)))
        (let
          ((uuid (if (null? prev) (random-uuid) (car prev)))
            (oldp (if (null? prev) "" (notes/doc/row-path prev))))
          (let
```

```
            ((path
                (if
                  (string-eq oldp "")
                  (notes/tree/note-path folder slug)
                  oldp))
              (body
                (dict-set*
                  "{}"
                  (list
                    "uuid"
                    uuid
                    "slug"
                    (as-str slug)
                    "title"
                    (as-str title)
                    "tags"
                    (as-str tags)
                    "kind"
                    (as-str kind)
                    "day"
                    (as-str day)))))
            (seq
              (notes/tree/ensure-folder (notes/tree/root) (whoami))
              (tree/upsert path "note" "note" body)
              (notes/tree/save-outline path outline)
              (if
                (and
                  (not (null? prev))
                  (not (string-eq oldp ""))
                  (not
                    (string-eq (as-str folder) (notes/doc/row-folder
    prev))))
                (notes/doc/set-folder slug folder)
                0)
              (roam/reindex (as-str slug) outline)
              slug))))
      (let
        ((row (notes/doc/row slug))
          (fields
            (notes/doc/make-fields
              slug
              title
              outline
              tags
              kind
              day
              folder)))
        (seq
          (if
            (null? row)
            (create (quote note) fields)
            (update (quote note) (car row) fields))
          (roam/reindex (as-str slug) outline)
          slug)))))
```

179 libs/notes/issues/lib.pp

```
;; package notes/issues -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push

(define notes/issues/add
  (lambda
    (kind title body)
    (let
      ((n (notes/issues/ensure)))
      (let
        ((tree (notes/doc/outline n))
          (full-title
            (string-append
              (as-str kind)
              ": "
              (notes/issues/title-or-kind kind title))))
        (let
          ((child
            (outline/set-body
              (outline/node (notes/issues/node-id tree) full-title)
              (as-str body))))
          (let
            ((new-tree (outline/add-child-by-id tree "root" child)))
            (notes/issues/upsert new-tree)))))))

(define notes/issues/body
  (lambda
    ()
    (html/join
      (list
        (html/h1 "Issues")
        (html/p*
          (html/attr "class" "muted")
          (html/join
            (list
              (html/a "/notes" "Notes")
              " ? "
              (html/a "/notes?slug=log/issues" "Raw note"))))
        (notes/issues/form)
        (notes/issues/list-html)))))

(define notes/issues/delete
  (lambda
    (id)
    (let
      ((n (notes/doc/get "log/issues")))
      (if
        (null? n)
        #f
        (let
          ((tree (notes/doc/outline n)))
          (let
            ((new-tree (outline/delete tree (as-str id))))
            (seq (notes/issues/upsert new-tree) #t)))))))

(define notes/issues/ensure
  (lambda
    ()
    (let
      ((n (notes/doc/get "log/issues")))
      (if
        (null? n)
        (seq
          (notes/doc/upsert
            "log/issues"
            "log/issues"
            (notes/empty-outline "log/issues")
            "log"
            "issue"
            "")
          (notes/doc/get "log/issues"))
        n))))
```

```
72
73  (define notes/issues/form
74    (lambda
75      ()
76      (html/form
77        "POST"
78        "/issues"
79        (html/attr "class" "card")
80        (html/join
81          (list
82            (html/h2 "File an issue")
83            (html/hidden "action" "issue-add")
84            (html/label "" "Kind ")
85            (html/select
86              "kind"
87              ""
88              (html/join
89                (list
90                  (html/option "BUG" "BUG" #t)
91                  (html/option "DESIGN" "DESIGN" #f)
92                  (html/option "SMELL" "SMELL" #f)
93                  (html/option "CLEANUP" "CLEANUP" #f))))
94            " "
95            (html/label "" "Title ")
96            (html/text-input
97              "title"
98              (html/attr "placeholder" "One-line title"))
99            " "
100           (html/textarea
101             "body"
102             (html/attrs
103               (list
104                 (html/attr "rows" "3")
105                 (html/attr "placeholder" "Details")))
106             "")
107           " "
108           (html/button
109             (html/attr "type" "submit")
110             (html/text "File issue")))))))
111
112 (define notes/issues/handle-post
113   (lambda
114     ()
115     (let
116       ((action (as-str (request-param "action"))))
117       (cond
118         ((string-eq action "issue-add")
119           (seq
120             (notes/issues/add
121               (request-param "kind")
122               (request-param "title")
123               (request-param "body"))
124             (redirect "/issues")))
125         ((string-eq action "issue-resolve")
126           (seq
127             (notes/issues/resolve (request-param "id"))
128             (redirect "/issues")))
129         ((string-eq action "issue-reopen")
130           (seq
131             (notes/issues/reopen (request-param "id"))
132             (redirect "/issues")))
133         ((string-eq action "issue-delete")
134           (seq
135             (notes/issues/delete (request-param "id"))
136             (redirect "/issues")))
137         (else (redirect "/issues"))))))
138
139 (define notes/issues/list-html
140   (lambda
141     ()
142     (let
143       ((n (notes/doc/get "log/issues")))
144       (if
```

```
        (null? n)
        (html/p* (html/attr "class" "muted") "No issues yet.")
        (let
          ((tree (notes/doc/outline n)))
          (html/join
            (list
              (html/h2 "Register")
              (html/ul
                (html/attr "class" "card")
                (notes/issues/nodes-html (outline/kids tree)))))))))

(define notes/issues/node-html
  (lambda
    (node)
    (let
      ((id (outline/id node))
        (title (outline/title node))
        (body (outline/body node))
        (done (string-eq (outline/todo node) "DONE")))
      (html/li
        (html/attr "class" "issue-item")
        (html/join
          (list
            (html/div
              (html/attr
                "class"
                (if done "issue-head done" "issue-head"))
              (html/join
                (list (html/code id) " " (html/strong title))))
            (if
              (string-eq (as-str body) "")
              ""
              (html/p* (html/attr "class" "issue-body") (as-str body)))
            (html/div
              (html/attr "class" "issue-actions")
              (html/join
                (list
                  (html/form
                    "POST"
                    "/issues"
                    ""
                    (html/join
                      (list
                        (html/hidden
                          "action"
                          (if done "issue-reopen" "issue-resolve"))
                        (html/hidden "id" id)
                        (html/button
                          (html/attr "type" "submit")
                          (html/text (if done "Reopen" "Resolve"))))))
                  " "
                  (html/form
                    "POST"
                    "/issues"
                    ""
                    (html/join
                      (list
                        (html/hidden "action" "issue-delete")
                        (html/hidden "id" id)
                        (html/button
                          (html/attrs
                            (list
                              (html/attr "type" "submit")
                              (html/attr "class" "secondary")))
                          (html/text "Del"))))))))))))))

(define notes/issues/node-id
  (lambda (tree) (string-append "i-" (as-str (now)))))

(define notes/issues/nodes-html
  (lambda
    (nodes)
    (if
```

```
      (null? nodes)
      ""
      (html/join
        (list
          (notes/issues/node-html (car nodes))
          (notes/issues/nodes-html (cdr nodes)))))))

(define notes/issues/page
  (lambda
    ()
    (if
      (string-eq (request-method) "POST")
      (notes/issues/handle-post)
      (html/page (list (notes/issues/body))))))

(define notes/issues/reopen
  (lambda (id) (notes/issues/set-todo id "")))

(define notes/issues/resolve
  (lambda (id) (notes/issues/set-todo id "DONE")))

(define notes/issues/set-todo
  (lambda
    (id todo)
    (let
      ((n (notes/doc/get "log/issues")))
      (if
        (null? n)
        #f
        (let
          ((tree (notes/doc/outline n)))
          (let
            ((new-tree
              (outline/update
                tree
                (as-str id)
                (lambda (node) (outline/set-todo node todo)))))
            (seq (notes/issues/upsert new-tree) #t)))))))

(define notes/issues/title-or-kind
  (lambda
    (kind title)
    (let
      ((t (string-trim (as-str title))))
      (if (string-eq t "") (as-str kind) t))))

(define notes/issues/upsert
  (lambda
    (tree)
    (let
      ((n (notes/doc/get "log/issues")))
      (notes/doc/upsert
        "log/issues"
        "log/issues"
        tree
        (if (null? n) "log" (notes/doc/tags n))
        "issue"
        (if (null? n) "" (notes/doc/day n))))))
```

180 libs/notes/legacy/lib.pp

```
;; package notes/legacy -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push

(define notes/legacy/tree-render
  (lambda
    ()
    (html/p*
      (html/attr "class" "muted")
      "notes/tree/render retired -- use notes/ui/folders-html (flat
     index).")))

(define notes/legacy/tree-render-folder
  (lambda
    (path names depth)
    (html/p*
      (html/attr "class" "muted")
      (string-append
        "notes/tree/render-folder retired ("
        (as-str path)
        ")"))))
```

181 libs/notes/lib.pp

```
;; package notes -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push

(define notes/all-todos
  (lambda
    ()
    (list/foldl
      (lambda
        (acc row)
        (let
          ((f (cdr row)))
          (let
            ((slug (as-str (dict-get f "slug")))
              (tree (outline/decode (dict-get f "outline"))))
            (list/foldl
              (lambda
                (a t)
                (cons
                  (list slug (car t) (car (cdr t)) (car (cdr (cdr t))))
                  a))
              acc
              (outline/todos tree)))))
      (list)
      (notes/list))))

(define notes/apply-op
  (lambda
    (slug tree op nid)
    (if
      (string-eq op "add-child")
      (outline/add-child-by-id
        tree
        nid
        (outline/node "n-new" "New node"))
      (if
        (string-eq op "add-sibling")
        (outline/insert-sibling
          tree
          nid
          (outline/node "n-new" "New node"))
        (if
          (string-eq op "delete")
          (outline/delete tree nid)
          (if
            (string-eq op "move-up")
            (outline/move-up tree nid)
            (if
              (string-eq op "move-down")
              (outline/move-down tree nid)
              (if
                (string-eq op "set-title")
                (outline/update
                  tree
                  nid
                  (lambda
                    (n)
                    (outline/set-title n (request-param "title"))))
                (if
                  (string-eq op "set-body")
                  (outline/update
                    tree
                    nid
                    (lambda (n) (outline/set-body n (request-param
    "body"))))
                  (if
                    (string-eq op "set-todo")
                    (outline/update
                      tree
                      nid
                      (lambda (n) (outline/set-todo n (request-param
    "todo"))))
```

```
                    (if
                      (string-eq op "cycle-todo")
                      (outline/update
                        tree
                        nid
                        (lambda (n) (outline/cycle-todo n)))
                      tree)))))))))))

(define notes/capture
  (lambda
    (title body)
    (let
      ((prev (notes/ensure-inbox)))
      (let
        ((tree (notes/get-outline prev))
          (t (notes/capture-title title body))
          (b (as-str body)))
        (let
          ((child
            (outline/set-body
              (outline/node (notes/capture-node-id tree) t)
              b)))
          (let
            ((new-tree (outline/add-child-by-id tree "root" child)))
            (seq
              (notes/upsert
                "inbox"
                "Inbox"
                new-tree
                "inbox"
                "inbox"
                "")
              "inbox")))))))

(define notes/capture-form
  (lambda
    ()
    (html/form
      "POST"
      "/notes"
      (html/attrs
        (list
          (html/attr "class" "card capture-card")
          (html/attr "hx-post" "/notes")
          (html/attr "hx-target" "#capture-status")
          (html/attr "hx-swap" "innerHTML")
          (html/attr
            "hx-on::after-request"
            "if(event.detail.successful){this.reset()}")))
      (html/join
        (list
          (html/hidden "action" "capture")
          (html/h2 "Capture")
          (html/p*
            (html/attr "class" "muted")
            "Drops a node into the Inbox note.")
          (html/label (html/attr "for" "capture-title") "Title")
          (html/text-input
            "title"
            (html/attrs
              (list
                (html/attr "id" "capture-title")
                (html/attr "placeholder" "optional")
                (html/attr "autocomplete" "off"))))
          (html/label (html/attr "for" "capture-body") "Note")
          (html/textarea
            "body"
            (html/attrs
              (list
                (html/attr "id" "capture-body")
                (html/attr "rows" "3")
                (html/attr "class" "capture-body")
                (html/attr "placeholder" "Quick capture226128166")
```

```
                (html/attr "required" "required")))
            "")
          (html/div
            (html/attr "class" "capture-row")
            (html/join
              (list
                (html/button (html/attr "type" "submit") "Capture")
                (html/a "/notes?slug=inbox" "Open inbox"))))
          (html/div (html/attr "id" "capture-status") "")))))

(define notes/capture-nav
  (lambda
    ()
    (html/tag
      "details"
      (html/attr "class" "menu-drop capture-drop")
      (html/join
        (list
          (html/tag
            "summary"
            (html/attr "class" "menu-drop-btn")
            "capture")
          (html/div
            (html/attr "class" "menu-drop-panel capture-panel")
            (html/form
              "POST"
              "/notes"
              (html/attrs
                (list
                  (html/attr "class" "capture-nav-form")
                  (html/attr "hx-post" "/notes")
                  (html/attr "hx-target" "#capture-status")
                  (html/attr "hx-swap" "innerHTML")
                  (html/attr
                    "hx-on::after-request"
                    "if(event.detail.successful){this.reset()}")))
              (html/join
                (list
                  (html/hidden "action" "capture")
                  (html/text-input
                    "title"
                    (html/attrs
                      (list
                        (html/attr "placeholder" "title (optional)")
                        (html/attr "autocomplete" "off")
                        (html/attr "class" "capture-nav-title"))))
                  (html/textarea
                    "body"
                    (html/attrs
                      (list
                        (html/attr "rows" "3")
                        (html/attr "class" "capture-body")
                        (html/attr "placeholder" "Quick capture226128166")
                        (html/attr "required" "required")))
                    "")
                  (html/div
                    (html/attr "class" "capture-row")
                    (html/join
                      (list
                        (html/button (html/attr "type" "submit") "Save")
                        (html/a "/notes?slug=inbox" "inbox"))))
                  (html/div (html/attr "id" "capture-status") ""))))))))))

(define notes/capture-node-id
  (lambda
    (tree)
    (string-append
      "c"
      (as-str (count (outline/kids tree)))
      "-"
      (as-str (string-length (outline/encode tree))))))

(define notes/capture-page
```

```
  (lambda
    ()
    (if
      (string-eq (request-method) "POST")
      (notes/do-capture)
      (html/page
        (list (html/h1 "Capture") (notes/capture-form))))))

(define notes/capture-status-html
  (lambda
    (ok msg)
    (html/p* (html/attr "class" (if ok "ok" "err")) msg)))

(define notes/capture-title
  (lambda
    (title body)
    (let
      ((t (string-trim (as-str title))))
      (if
        (not (string-eq t ""))
        t
        (let
          ((b (string-trim (as-str body))))
          (if
            (string-eq b "")
            "capture"
            (if
              (<= (string-length b) 48)
              b
              (string-append (substring b 0 45) "226128166")))))))))

(define notes/capture/add
  (lambda
    (title body)
    (let
      ((prev (notes/doc/ensure-inbox)))
      (let
        ((tree (notes/doc/outline prev))
          (t (notes/capture/title title body))
          (b (as-str body)))
        (let
          ((child
            (outline/set-body
              (outline/node (notes/capture/node-id tree) t)
              b)))
          (let
            ((new-tree (outline/add-child-by-id tree "root" child)))
            (seq
              (notes/doc/upsert
                "inbox"
                "Inbox"
                new-tree
                "inbox"
                "inbox"
                "")
              "inbox")))))))

(define notes/capture/form
  (lambda
    ()
    (html/form
      "POST"
      "/notes"
      (html/attrs
        (list
          (html/attr "class" "card capture-card")
          (html/attr "hx-post" "/notes")
          (html/attr "hx-target" "#capture-status")
          (html/attr "hx-swap" "innerHTML")
          (html/attr
            "hx-on::after-request"
            "if(event.detail.successful){this.reset()}")))
      (html/join
```

```
        (list
          (html/hidden "action" "capture")
          (html/h2 "Capture")
          (html/p*
            (html/attr "class" "muted")
            "Drops a node into the Inbox note.")
          (html/label (html/attr "for" "capture-title") "Title")
          (html/text-input
            "title"
            (html/attrs
              (list
                (html/attr "id" "capture-title")
                (html/attr "placeholder" "optional")
                (html/attr "autocomplete" "off"))))
          (html/label (html/attr "for" "capture-body") "Note")
          (html/textarea
            "body"
            (html/attrs
              (list
                (html/attr "id" "capture-body")
                (html/attr "rows" "3")
                (html/attr "class" "capture-body")
                (html/attr "placeholder" "Quick capture226128166")
                (html/attr "required" "required")))
            "")
          (html/div
            (html/attr "class" "capture-row")
            (html/join
              (list
                (html/button (html/attr "type" "submit") "Capture")
                (html/a "/notes?slug=inbox" "Open inbox"))))
          (html/div (html/attr "id" "capture-status") "")))))

(define notes/capture/handle
  (lambda
    ()
    (let
      ((title (request-param "title"))
        (body (request-param "body")))
      (let
        ((b (if (null? body) "" (as-str body))))
        (if
          (string-eq (string-trim b) "")
          (if
            (hx-request?)
            (render-html
              (notes/capture/status #f "Write something first."))
            (redirect "/"))
          (seq
            (notes/capture/add (if (null? title) "" title) b)
            (if
              (hx-request?)
              (render-html
                (notes/capture/status #t "Saved to Inbox."))
              (redirect "/notes?slug=inbox")))))))))

(define notes/capture/nav
  (lambda
    ()
    (html/tag
      "details"
      (html/attr "class" "menu-drop capture-drop")
      (html/join
        (list
          (html/tag
            "summary"
            (html/attr "class" "menu-drop-btn")
            "capture")
          (html/div
            (html/attr "class" "menu-drop-panel capture-panel")
            (html/form
              "POST"
              "/notes"
```

```
              (html/attrs
                (list
                  (html/attr "class" "capture-nav-form")
                  (html/attr "hx-post" "/notes")
                  (html/attr "hx-target" "#capture-status")
                  (html/attr "hx-swap" "innerHTML")
                  (html/attr
                    "hx-on::after-request"
                    "if(event.detail.successful){this.reset()}")))
              (html/join
                (list
                  (html/hidden "action" "capture")
                  (html/text-input
                    "title"
                    (html/attrs
                      (list
                        (html/attr "placeholder" "title (optional)")
                        (html/attr "autocomplete" "off")
                        (html/attr "class" "capture-nav-title"))))
                  (html/textarea
                    "body"
                    (html/attrs
                      (list
                        (html/attr "rows" "3")
                        (html/attr "class" "capture-body")
                        (html/attr "placeholder" "Quick capture226128166")
                        (html/attr "required" "required")))
                    "")
                  (html/div
                    (html/attr "class" "capture-row")
                    (html/join
                      (list
                        (html/button (html/attr "type" "submit") "Save")
                        (html/a "/notes?slug=inbox" "inbox"))))
                  (html/div (html/attr "id" "capture-status") ""))))))))))

(define notes/capture/node-id
  (lambda
    (tree)
    (string-append
      "c"
      (as-str (count (outline/kids tree)))
      "-"
      (as-str (string-length (outline/encode tree))))))

(define notes/capture/outline-of-doc
  (lambda (note) (notes/doc/outline note))
  "Outline tree for a notes/doc row; when notes/doc/ensure-inbox has
     already patched in the loaded tree, return it without a second tree
     walk.")

(define notes/capture/page
  (lambda
    ()
    (if
      (string-eq (request-method) "POST")
      (notes/capture/handle)
      (html/page
        (list (html/h1 "Capture") (notes/capture/form))))))

(define notes/capture/status
  (lambda
    (ok msg)
    (html/p* (html/attr "class" (if ok "ok" "err")) msg)))

(define notes/capture/title
  (lambda
    (title body)
    (let
      ((t (string-trim (as-str title))))
      (if
        (not (string-eq t ""))
        t
```

```
        (let
          ((b (string-trim (as-str body))))
          (if
            (string-eq b "")
            "capture"
            (if
              (<= (string-length b) 48)
              b
              (string-append (substring b 0 45) "..."))))))))

(define notes/capture/upsert-inbox
  (lambda
    (tree)
    (seq
      (notes/doc/upsert-in
        "inbox"
        "Inbox"
        tree
        "inbox"
        "inbox"
        ""
        (notes/doc/row-folder (notes/doc/row "inbox")))
      "inbox"))
  "Upsert the inbox note from an already-decoded outline tree. Skips
     notes/doc/upsert's own outline reload.")

(define notes/create-form
  (lambda
    ()
    (html/form
      "POST"
      "/notes"
      (html/attr "class" "card")
      (html/join
        (list
          (html/hidden "action" "create")
          (html/label "" "Title ")
          (html/text-input
            "title"
            (html/attr "placeholder" "New note"))
          " "
          (html/label "" "Slug ")
          (html/text-input
            "slug"
            (html/attr "placeholder" "my-note"))
          " "
          (html/button (html/attr "type" "submit") "Create"))))))

(define notes/css
  (lambda
    ()
    (css/sheet
      (list
        (css/rule
          "img.note-media"
          (list
            "display"
            "block"
            "max-width"
            "100%"
            "max-height"
            "28rem"
            "height"
            "auto"
            "margin"
            ".5rem 0"
            "border"
            "1px solid #000"
            "background"
            "var(--panel)"))
        (css/rule
          ".note-media-cap"
          (list "margin" "0 0 .5rem" "font-size" ".85rem"))
```

```
        (css/rule
          "li.note-item"
          (list
            "display"
            "flex"
            "align-items"
            "center"
            "gap"
            ".5rem"
            "padding"
            ".35rem 0"
            "border-bottom"
            "1px solid #c0c0c0"))
        (css/rule "li.note-item > a" (list "flex" "1 1 auto"))
        (css/rule
          "li.note-item form.note-del"
          (list "margin" "0"))
        (css/rule
          "li.note-item .note-del button"
          (list
            "width"
            "auto"
            "padding"
            ".25rem .45rem"
            "font-size"
            ".85rem"))
        (css/rule
          ".note-title-row"
          (list
            "display"
            "flex"
            "flex-wrap"
            "wrap"
            "align-items"
            "center"
            "gap"
            ".75rem"))
        (css/rule
          ".note-title-row h1"
          (list
            "flex"
            "1 1 auto"
            "margin"
            "0"
            "border-bottom"
            "none"
            "padding"
            "0"))
        (css/rule
          ".note-title-row .note-del-page button"
          (list "width" "auto")))))
  "Render the notes component stylesheet (extracted from ui/theme).")

(define notes/cycle-node-todo
  (lambda
    (slug nid)
    (let
      ((prev (notes/get slug)))
      (if
        (null? prev)
        ()
        (let
          ((tree (notes/get-outline prev))
            (title (notes/get-title prev))
            (tags (notes/get-tags prev))
            (kind (notes/get-kind prev))
            (day (notes/get-day prev)))
          (let
            ((new-tree
              (outline/update
                tree
                nid
                (lambda (n) (outline/cycle-todo n)))))
```

```
            (notes/upsert
              slug
              title
              new-tree
              tags
              kind
              day)))))))

(define notes/daily
  (lambda
    (day)
    (let
      ((slug (string-append "daily-" (as-str day))))
      (if
        (null? (notes/get slug))
        (seq
          (notes/upsert
            slug
            (string-append "Daily " (as-str day))
            (notes/empty-outline
              (string-append "Daily " (as-str day)))
            "daily"
            "daily"
            (as-str day))
          (notes/get slug))
        (notes/get slug)))))

(define notes/delete
  (lambda
    (slug)
    (let
      ((row (notes/row (as-str slug))))
      (if (null? row) () (delete (quote note) (car row))))))

(define notes/do-capture
  (lambda
    ()
    (let
      ((title (request-param "title"))
        (body (request-param "body")))
      (let
        ((b (if (null? body) "" (as-str body))))
        (if
          (string-eq (string-trim b) "")
          (if
            (hx-request?)
            (render-html
              (notes/capture-status-html #f "Write something first."))
            (redirect "/"))
          (seq
            (notes/capture (if (null? title) "" title) b)
            (if
              (hx-request?)
              (render-html
                (notes/capture-status-html #t "Saved to Inbox."))
              (redirect "/notes?slug=inbox")))))))))

(define notes/do-create
  (lambda
    (title slug)
    (let
      ((s
        (if
          (if
            (null? slug)
            #t
            (string-eq (string-trim (as-str slug)) ""))
          (string-append "note-" (as-str (count (notes/list))))
          (string-trim (as-str slug)))))
      (seq
        (notes/upsert
          s
          (as-str title)
```

```
          (notes/empty-outline (as-str title))
          ""
          "note"
          "")
        (redirect (string-append "/notes?slug=" s))))))

(define notes/do-delete
  (lambda
    ()
    (let
      ((slug (as-str (request-param "slug"))))
      (seq (notes/delete slug) (redirect "/notes")))))

(define notes/do-refile
  (lambda
    ()
    (let
      ((src (as-str (request-param "src_slug")))
        (nid (as-str (request-param "node_id")))
        (dst (as-str (request-param "dst_slug"))))
      (seq
        (notes/refile src nid dst)
        (redirect (string-append "/notes?slug=" src))))))

(define notes/do-save
  (lambda
    (slug raw)
    (let
      ((tree (outline/decode raw)) (prev (notes/get slug)))
      (let
        ((title
            (if (null? prev) (as-str slug) (notes/get-title prev)))
          (tags (if (null? prev) "" (notes/get-tags prev)))
          (kind (if (null? prev) "note" (notes/get-kind prev)))
          (day (if (null? prev) "" (notes/get-day prev))))
        (seq
          (notes/upsert
            (as-str slug)
            title
            tree
            tags
            kind
            day)
          (redirect (string-append "/notes?slug=" (as-str slug))))))))

(define notes/doc/daily
  (lambda
    (day)
    (let
      ((slug (string-append "daily-" (as-str day))))
      (if
        (null? (notes/doc/get slug))
        (seq
          (notes/doc/upsert
            slug
            (string-append "Daily " (as-str day))
            (notes/doc/empty-outline
              (string-append "Daily " (as-str day)))
            "daily"
            "daily"
            (as-str day))
          (notes/doc/get slug))
        (notes/doc/get slug)))))

(define notes/doc/day
  (lambda (n) (car (cdr (cdr (cdr (cdr (cdr (cdr n)))))))))

(define notes/doc/delete
  (lambda
    (slug)
    (let
      ((row (notes/doc/row (as-str slug))))
      (if
```

```
        (null? row)
        ()
        (seq
          (roam/delete-from-slug (as-str slug))
          (let
            ((p (notes/doc/row-path row)))
            (if
              (and (notes/tree/enabled?) (not (string-eq p "")))
              (tree/delete p "recursive")
              (delete (quote note) (car row))))
          slug)))))

(define notes/doc/display-folder
  (lambda
    (folder)
    (let
      ((f (as-str folder)))
      (let
        ((stripped
          (if
            (string-starts-with? f "garden/")
            (string-replace f "garden/" "")
            (if (string-eq f "garden") "" f))))
        (string-replace stripped "_" "-"))))))

(define notes/doc/empty-outline
  (lambda (title) (outline/node "root" title)))

(define notes/doc/ensure-inbox
  (lambda
    ()
    (if
      (null? (notes/doc/get "inbox"))
      (seq
        (notes/doc/upsert
          "inbox"
          "Inbox"
          (notes/doc/empty-outline "Inbox")
          "inbox"
          "inbox"
          "")
        (notes/doc/get "inbox"))
      (notes/doc/get "inbox"))))

(define notes/doc/folder
  (lambda
    (n)
    (if
      (null? n)
      ""
      (let
        ((f (car (cdr (cdr (cdr (cdr (cdr (cdr (cdr n))))))))))
        (if (null? f) "" (as-str f))))))

(define notes/doc/folder-names
  (lambda () (tree/folder-index (notes/tree/root))))

(define notes/doc/folder-seen?
  (lambda
    (xs f)
    (if
      (null? xs)
      #f
      (if
        (string-eq (car xs) f)
        #t
        (notes/doc/folder-seen? (cdr xs) f)))))

(define notes/doc/get
  (lambda
    (slug)
    (let
      ((row (notes/doc/row slug)))
```

```
      (if
        (null? row)
        ()
        (let
          ((f (cdr row)))
          (list
            (car row)
            (dict-get f "slug")
            (dict-get f "title")
            (outline/decode (dict-get f "outline"))
            (dict-get f "tags")
            (dict-get f "kind")
            (dict-get f "day")
            (let
              ((folder (dict-get f "folder")))
              (if (null? folder) "" folder))))))))

(define notes/doc/hidden?
  (lambda
    (row)
    (let
      ((f (cdr row)))
      (let
        ((kind (as-str (dict-get f "kind")))
          (tags (as-str (dict-get f "tags")))
          (slug (as-str (dict-get f "slug"))))
        (if
          (string-eq kind "inbox")
          #t
          (if
            (string-eq kind "hidden")
            #t
            (if
              (string-eq kind "issue")
              #t
              (if
                (string-eq slug "inbox")
                #t
                (string-contains? tags "hidden")))))))))

(define notes/doc/index-list
  (lambda
    ()
    (list/filter
      (lambda (row) (not (notes/doc/hidden? row)))
      (notes/doc/list))))

(define notes/doc/kind
  (lambda (n) (car (cdr (cdr (cdr (cdr (cdr n))))))))

(define notes/doc/list
  (lambda
    ()
    (if
      (notes/tree/enabled?)
      (notes/tree/list-notes)
      (list))))

(define notes/doc/list-folder
  (lambda
    (folder)
    (let
      ((want (as-str folder)))
      (list/filter
        (lambda
          (row)
          (string-eq (notes/doc/row-folder row) want))
        (notes/doc/index-list)))))

(define notes/doc/make-fields
  (lambda
    (slug
      title
```

```
      outline
      tags
      kind
      day
      folder)
    (dict-set*
      "{}"
      (list
        "slug"
        (as-str slug)
        "title"
        (as-str title)
        "outline"
        (outline/encode outline)
        "tags"
        (as-str tags)
        "kind"
        (as-str kind)
        "day"
        (as-str day)
        "folder"
        (as-str folder)))))

(define notes/doc/outline
  (lambda (n) (car (cdr (cdr (cdr n))))))

(define notes/doc/row
  (lambda
    (slug)
    (let
      ((tr
        (if
          (notes/tree/enabled?)
          (notes/tree/find-slug slug)
          ())))
      (if
        (null? tr)
        (let
          ((rs (find-rows (quote note) "slug" (as-str slug) 1)))
          (if (null? rs) () (car rs)))
        tr))))

(define notes/doc/row-folder
  (lambda
    (row)
    (if
      (null? row)
      ""
      (let
        ((f (dict-get (cdr row) "folder")))
        (if (null? f) "" (as-str f))))))

(define notes/doc/row-path
  (lambda
    (row)
    (if
      (null? row)
      ""
      (let
        ((p (dict-get (cdr row) "_path")))
        (if (null? p) "" (as-str p))))))

(define notes/doc/search
  (lambda
    (q)
    (let
      ((needle (string-trim (as-str q))))
      (if
        (string-eq needle "")
        (list)
        (list/map
          (lambda (j) (if (null? j) "{}" (as-str j)))
          (tree/search-text needle 20))))))
```

```
(define notes/doc/set-folder
  (lambda
    (slug folder)
    (let
      ((row (notes/doc/row (as-str slug))))
      (if
        (null? row)
        ()
        (if
          (notes/tree/enabled?)
          (let
            ((old (notes/doc/row-path row))
              (newp (notes/tree/note-path folder (as-str slug))))
            (if
              (string-eq old "")
              (notes/doc/upsert-in
                slug
                (dict-get (cdr row) "title")
                (outline/decode (dict-get (cdr row) "outline"))
                (dict-get (cdr row) "tags")
                (dict-get (cdr row) "kind")
                (dict-get (cdr row) "day")
                folder)
              (if (string-eq old newp) old (tree/move old newp))))
          (update-field
            (quote note)
            (car row)
            "folder"
            (as-str folder)))))))

(define notes/doc/show-archived?
  (lambda
    ()
    (string-eq (as-str (request-param "show")) "archived")))

(define notes/doc/tags
  (lambda (n) (car (cdr (cdr (cdr (cdr n)))))))

(define notes/doc/title
  (lambda (n) (car (cdr (cdr n)))))

(define notes/doc/upsert
  (lambda
    (slug title outline tags kind day)
    (let
      ((row (notes/doc/row slug)))
      (notes/doc/upsert-in
        slug
        title
        outline
        tags
        kind
        day
        (if (null? row) "" (notes/doc/row-folder row))))))

(define notes/doc/upsert-in
  (lambda
    (slug
      title
      outline
      tags
      kind
      day
      folder)
    (if
      (notes/tree/enabled?)
      (let
        ((prev (notes/doc/row slug)))
        (let
          ((uuid (if (null? prev) (random-uuid) (car prev)))
            (oldp (if (null? prev) "" (notes/doc/row-path prev))))
          (let
```

```
            ((path
                (if
                  (string-eq oldp "")
                  (notes/tree/note-path folder slug)
                  oldp))
              (body
                (dict-set*
                  "{}"
                  (list
                    "uuid"
                    uuid
                    "slug"
                    (as-str slug)
                    "title"
                    (as-str title)
                    "tags"
                    (as-str tags)
                    "kind"
                    (as-str kind)
                    "day"
                    (as-str day)))))
            (seq
              (notes/tree/ensure-folder (notes/tree/root) (whoami))
              (tree/upsert path "note" "note" body)
              (notes/tree/save-outline path outline)
              (if
                (and
                  (not (null? prev))
                  (not (string-eq oldp ""))
                  (not
                    (string-eq (as-str folder) (notes/doc/row-folder prev))))
                (notes/doc/set-folder slug folder)
                0)
              (roam/reindex (as-str slug) outline)
              slug))))
      (let
        ((row (notes/doc/row slug))
          (fields
            (notes/doc/make-fields
              slug
              title
              outline
              tags
              kind
              day
              folder)))
        (seq
          (if
            (null? row)
            (create (quote note) fields)
            (update (quote note) (car row) fields))
          (roam/reindex (as-str slug) outline)
          slug)))))

(define notes/edit-form
  (lambda
    (slug outline)
    (html/form
      "POST"
      "/notes"
      ""
      (html/join
        (list
          (html/hidden "action" "save")
          (html/hidden "slug" slug)
          (html/tag
            "textarea"
            (html/attrs
              (list
                (html/attr "name" "outline")
                (html/attr "rows" "16")))
            (outline/encode outline))
```

```
          (html/p*
            ""
            (html/button (html/attr "type" "submit") "Save outline")))))))

(define notes/empty-outline
  (lambda (title) (outline/node "root" title)))

(define notes/ensure-inbox
  (lambda
    ()
    (if
      (null? (notes/get "inbox"))
      (seq
        (notes/upsert
          "inbox"
          "Inbox"
          (notes/empty-outline "Inbox")
          "inbox"
          "inbox"
          "")
        (notes/get "inbox"))
      (notes/get "inbox"))))

(define notes/get
  (lambda
    (slug)
    (let
      ((row (notes/row slug)))
      (if
        (null? row)
        ()
        (let
          ((f (cdr row)))
          (list
            (car row)
            (dict-get f "slug")
            (dict-get f "title")
            (outline/decode (dict-get f "outline"))
            (dict-get f "tags")
            (dict-get f "kind")
            (dict-get f "day")))))))

(define notes/get-day
  (lambda (n) (car (cdr (cdr (cdr (cdr (cdr (cdr (cdr (cdr n)))))))))))

(define notes/get-kind
  (lambda (n) (car (cdr (cdr (cdr (cdr (cdr (cdr (cdr n))))))))))

(define notes/get-outline
  (lambda (n) (car (cdr (cdr (cdr n))))))

(define notes/get-tags
  (lambda (n) (car (cdr (cdr (cdr (cdr n)))))))

(define notes/get-title
  (lambda (n) (car (cdr (cdr n)))))

(define notes/handle-post
  (lambda
    ()
    (let
      ((action (as-str (request-param "action"))))
      (if
        (string-eq action "create")
        (notes/do-create
          (request-param "title")
          (request-param "slug"))
        (if
          (string-eq action "save")
          (notes/do-save
            (request-param "slug")
            (request-param "outline"))
          (if
```

```
            (string-eq action "capture")
            (notes/do-capture)
            (if
              (string-eq action "delete")
              (notes/do-delete)
              (redirect "/notes")))))))))

(define notes/hidden-from-index?
  (lambda
    (row)
    (let
      ((f (cdr row)))
      (let
        ((kind (as-str (dict-get f "kind")))
          (tags (as-str (dict-get f "tags")))
          (slug (as-str (dict-get f "slug"))))
        (if
          (string-eq kind "inbox")
          #t
          (if
            (string-eq kind "hidden")
            #t
            (if
              (string-eq kind "issue")
              #t
              (if
                (string-eq slug "inbox")
                #t
                (string-contains? tags "hidden")))))))))

(define notes/index-html
  (lambda
    ()
    (html/join
      (list
        (html/h1 "Notes")
        (html/p*
          (html/attr "class" "muted")
          "Outline-first notes. Create or open a slug.")
        (notes/create-form)
        (html/h2 "Recent")
        (html/ul (html/attr "class" "card") (notes/recent-list))))))

(define notes/index-list
  (lambda
    ()
    (list/filter
      (lambda (row) (not (notes/hidden-from-index? row)))
      (notes/list))))

(define notes/issues/add
  (lambda
    (kind title body)
    (let*
      ((np (notes/tree/note-path "" "log/issues"))
        (full-title
          (string-append
            (as-str kind)
            ": "
            (notes/issues/title-or-kind kind title)))
        (nid (notes/issues/node-id (list))))
      (seq
        (notes/issues/ensure)
        (notes/node/upsert np nid full-title (as-str body) "")
        nid)))
  "Add an issue to log/issues as a single per-node write (no whole-tree
     rewrite). Ensures the register exists, then upserts one leaf heading
     with a collision-free id. Returns the new node id. Fixes the data-loss
     clobber.")

(define notes/issues/body
  (lambda
    ()
```

```
    (html/join
      (list
        (html/h1 "Issues")
        (html/p*
          (html/attr "class" "muted")
          (html/join
            (list
              (html/a "/notes" "Notes")
              " ? "
              (html/a "/notes?slug=log/issues" "Raw note"))))
        (notes/issues/form)
        (notes/issues/list-html)))))

(define notes/issues/delete
  (lambda
    (id)
    (notes/node/delete
      (notes/tree/note-path "" "log/issues")
      id))
  "Delete one issue id from log/issues. Single per-node delete (no
     whole-tree rewrite). Returns #t if found, #f if absent. Fixes
     i-1786862188 and the data-loss clobber.")

(define notes/issues/ensure
  (lambda
    ()
    (let
      ((n (notes/doc/get "log/issues")))
      (if
        (null? n)
        (seq
          (notes/doc/upsert
            "log/issues"
            "log/issues"
            (notes/empty-outline "log/issues")
            "log"
            "issue"
            "")
          (notes/doc/get "log/issues"))
        n))))

(define notes/issues/form
  (lambda
    ()
    (html/form
      "POST"
      "/issues"
      (html/attr "class" "card")
      (html/join
        (list
          (html/h2 "File an issue")
          (html/hidden "action" "issue-add")
          (html/label "" "Kind ")
          (html/select
            "kind"
            ""
            (html/join
              (list
                (html/option "BUG" "BUG" #t)
                (html/option "DESIGN" "DESIGN" #f)
                (html/option "SMELL" "SMELL" #f)
                (html/option "CLEANUP" "CLEANUP" #f))))
          " "
          (html/label "" "Title ")
          (html/text-input
            "title"
            (html/attr "placeholder" "One-line title"))
          " "
          (html/textarea
            "body"
            (html/attrs
              (list
                (html/attr "rows" "3")
```

```
                (html/attr "placeholder" "Details")))
            "")
          " "
          (html/button
            (html/attr "type" "submit")
            (html/text "File issue")))))))

(define notes/issues/handle-post
  (lambda
    ()
    (let
      ((action (as-str (request-param "action"))))
      (cond
        ((string-eq action "issue-add")
          (seq
            (notes/issues/add
              (request-param "kind")
              (request-param "title")
              (request-param "body"))
            (redirect "/issues")))
        ((string-eq action "issue-resolve")
          (seq
            (notes/issues/resolve (request-param "id"))
            (redirect "/issues")))
        ((string-eq action "issue-reopen")
          (seq
            (notes/issues/reopen (request-param "id"))
            (redirect "/issues")))
        ((string-eq action "issue-delete")
          (seq
            (notes/issues/delete (request-param "id"))
            (redirect "/issues")))
        (else (redirect "/issues"))))))

(define notes/issues/list-html
  (lambda
    ()
    (let
      ((n (notes/doc/get "log/issues")))
      (if
        (null? n)
        (html/p* (html/attr "class" "muted") "No issues yet.")
        (let
          ((tree (notes/doc/outline n)))
          (html/join
            (list
              (html/h2 "Register")
              (html/ul
                (html/attr "class" "card")
                (notes/issues/nodes-html (outline/kids tree)))))))))

(define notes/issues/node-html
  (lambda
    (node)
    (let
      ((id (outline/id node))
        (title (outline/title node))
        (body (outline/body node))
        (done (string-eq (outline/todo node) "DONE")))
      (html/li
        (html/attr "class" "issue-item")
        (html/join
          (list
            (html/div
              (html/attr
                "class"
                (if done "issue-head done" "issue-head"))
              (html/join
                (list (html/code id) " " (html/strong title))))
            (if
              (string-eq (as-str body) "")
              ""
              (html/p* (html/attr "class" "issue-body") (as-str body)))
```

```
            (html/div
              (html/attr "class" "issue-actions")
              (html/join
                (list
                  (html/form
                    "POST"
                    "/issues"
                    ""
                    (html/join
                      (list
                        (html/hidden
                          "action"
                          (if done "issue-reopen" "issue-resolve"))
                        (html/hidden "id" id)
                        (html/button
                          (html/attr "type" "submit")
                          (html/text (if done "Reopen" "Resolve"))))))
                  " "
                  (html/form
                    "POST"
                    "/issues"
                    ""
                    (html/join
                      (list
                        (html/hidden "action" "issue-delete")
                        (html/hidden "id" id)
                        (html/button
                          (html/attrs
                            (list
                              (html/attr "type" "submit")
                              (html/attr "class" "secondary")))
                          (html/text "Del"))))))))))

(define notes/issues/node-id
  (lambda
    (tree)
    (string-append "i-" (as-str (now)) "-" (random-uuid)))
  "Generate a collision-free issue node id: i-<unix-seconds>-<uuid>. Fixes
     same-second collision in i-1786862188.")

(define notes/issues/nodes-html
  (lambda
    (nodes)
    (if
      (null? nodes)
      ""
      (html/join
        (list
          (notes/issues/node-html (car nodes))
          (notes/issues/nodes-html (cdr nodes)))))))

(define notes/issues/page
  (lambda
    ()
    (if
      (string-eq (request-method) "POST")
      (notes/issues/handle-post)
      (html/page (list (notes/issues/body))))))

(define notes/issues/reopen
  (lambda (id) (notes/issues/set-todo id "")))

(define notes/issues/resolve
  (lambda (id) (notes/issues/set-todo id "DONE")))

(define notes/issues/set-todo
  (lambda
    (id todo)
    (notes/node/set-todo
      (notes/tree/note-path "" "log/issues")
      id
      todo))
```

```
  "Set todo status of one issue id in log/issues. Single per-node write (no
     whole-tree rewrite). Returns #t if found, #f if absent. Fixes
     i-1786862188 and the data-loss clobber.")

(define notes/issues/title-or-kind
  (lambda
    (kind title)
    (let
      ((t (string-trim (as-str title))))
      (if (string-eq t "") (as-str kind) t))))

(define notes/issues/upsert
  (lambda
    (tree)
    (let
      ((n (notes/doc/get "log/issues")))
      (notes/doc/upsert
        "log/issues"
        "log/issues"
        tree
        (if (null? n) "log" (notes/doc/tags n))
        "issue"
        (if (null? n) "" (notes/doc/day n))))))

(define notes/item-li
  (lambda
    (row)
    (let
      ((f (cdr row)) (id (car row)))
      (let
        ((slug (as-str (dict-get f "slug")))
          (title (as-str (dict-get f "title"))))
        (html/li
          (html/attr "class" "note-item")
          (html/join
            (list
              (html/a
                (string-append "/notes?slug=" slug)
                (html/text title))
              (html/form
                "POST"
                "/notes"
                (html/attrs
                  (list
                    (html/attr "class" "note-del")
                    (html/attr "hx-confirm" "Delete this note?")))
                (html/join
                  (list
                    (html/hidden "action" "delete")
                    (html/hidden "slug" slug)
                    (html/button
                      (html/attrs
                        (list
                          (html/attr "type" "submit")
                          (html/attr "class" "secondary")
                          (html/attr "title" "delete note")))
                      "del")))))))))))

(define notes/legacy/tree-render
  (lambda
    ()
    (html/p*
      (html/attr "class" "muted")
      "notes/tree/render retired -- use notes/ui/folders-html (flat
     index).")))

(define notes/legacy/tree-render-folder
  (lambda
    (path names depth)
    (html/p*
      (html/attr "class" "muted")
      (string-append
        "notes/tree/render-folder retired ("
```

```
        (as-str path)
        ")"))))

(define notes/light/add-prefixes
  (lambda
    (parts prefix acc)
    (if
      (null? parts)
      acc
      (let*
        ((seg (as-str (car parts)))
          (full
            (if
              (string-eq prefix "")
              seg
              (string-append prefix "/" seg))))
        (notes/light/add-prefixes
          (cdr parts)
          full
          (notes/light/cons-unique full acc))))))

(define notes/light/all-folder-paths
  (lambda
    (all-notes)
    (notes/light/dedupe
      (notes/tree/sort-by-name
        (list/foldl
          (lambda
            (acc note)
            (let
              ((f (as-str (notes/light/nth note 1))))
              (if
                (string-eq f "")
                acc
                (notes/light/add-prefixes (string-split f "/") "" acc))))
          (list)
          all-notes)))))

(define notes/light/children-of
  (lambda
    (parent paths)
    (list/filter
      (lambda
        (p)
        (string-eq (notes/light/parent-of p) parent))
      paths)))

(define notes/light/cons-unique
  (lambda
    (x xs)
    (if (notes/light/contains? x xs) xs (cons x xs))))

(define notes/light/contains?
  (lambda
    (x xs)
    (if
      (null? xs)
      #f
      (if
        (string-eq x (as-str (car xs)))
        #t
        (notes/light/contains? x (cdr xs))))))

(define notes/light/dedupe
  (lambda
    (xs)
    (if
      (null? xs)
      (list)
      (notes/light/cons-unique
        (car xs)
        (notes/light/dedupe (cdr xs))))))

```

```
(define notes/light/folder-section
  (lambda
    (folder)
    (let*
      ((all-notes (notes/list-light))
        (items (notes/light/notes-in folder all-notes))
        (label
          (if (string-eq folder "") "(uncategorized)" folder))
        (href (string-append "/notes?folder=" folder)))
      (html/div
        (html/attr "class" "notes-folder card")
        (html/join
          (list
            (html/h3 (html/a href (html/text label)))
            (if
              (null? items)
              (html/p*
                (html/attr "class" "muted")
                "No notes in this folder.")
              (notes/light/render-notes items)))))))

(define notes/light/last
  (lambda
    (xs)
    (if
      (null? (cdr xs))
      (car xs)
      (notes/light/last (cdr xs)))))

(define notes/light/last-segment
  (lambda
    (path)
    (if
      (string-eq path "")
      ""
      (notes/light/last (string-split path "/")))))

(define notes/light/note-li
  (lambda
    (note)
    (let*
      ((title (as-str (notes/light/nth note 2)))
        (slug (as-str (notes/light/nth note 3))))
      (html/li
        (html/attr "class" "note-item")
        (html/a
          (string-append "/notes?slug=" slug)
          (html/text title))))))

(define notes/light/notes-in
  (lambda
    (folder all-notes)
    (list/filter
      (lambda
        (n)
        (string-eq (as-str (notes/light/nth n 1)) folder))
      all-notes)))

(define notes/light/nth
  (lambda
    (xs i)
    (if
      (<= i 0)
      (car xs)
      (notes/light/nth (cdr xs) (- i 1)))))

(define notes/light/page
  (lambda
    ()
    (let
      ((slug (request-param "slug")))
      (if
        (if (null? slug) #t (string-eq (as-str slug) ""))
```

```
        (let
          ((folder (request-param "folder")))
          (html/page
            (list
              (notes/tree/style)
              (html/h1 "Notes")
              (html/h2 "Folders")
              (if
                (if
                  (null? folder)
                  #f
                  (not (string-eq (as-str folder) "")))
                (notes/light/folder-section (as-str folder))
                (notes/light/render)))))
        (html/page (list (notes/ui/show (as-str slug))))))))

(define notes/light/parent-of
  (lambda
    (path)
    (if
      (string-eq path "")
      ""
      (let
        ((parts (string-split path "/")))
        (if
          (<= (length parts) 1)
          ""
          (notes/tree/join-slash (notes/tree/butlast parts)))))))

(define notes/light/render
  (lambda
    ()
    (let*
      ((all-notes (notes/list-light))
        (paths (notes/light/all-folder-paths all-notes))
        (roots
          (notes/tree/sort-by-name
            (notes/light/children-of "" paths)))
        (uncat (notes/light/notes-in "" all-notes)))
      (html/div
        (html/attr "class" "notes-tree")
        (html/join
          (list
            (if
              (null? uncat)
              ""
              (html/tag
                "details"
                (html/attrs
                  (list
                    (html/attr "class" "tree-folder tree-uncategorized")
                    (html/attr "open" "open")))
                (html/join
                  (list
                    (html/tag
                      "summary"
                      (html/attr "class" "tree-folder-header")
                      (html/join
                        (list
                          (html/span
                            (html/attr "class" "tree-folder-icon")
                            "&#x1F4C2; ")
                          (html/text "(uncategorized)")
                          (html/span
                            (html/attr "class" "muted")
                            (html/text
                              (string-append " (" (as-str (length uncat))
    ")"))))))
                    (notes/light/render-notes uncat)))))
            (html/join
              (list/map
                (lambda
                  (root)
```

```
                  (notes/light/render-folder root paths all-notes))
                roots)))))))

(define notes/light/render-folder
  (lambda
    (path paths all-notes)
    (let*
      ((label (notes/light/last-segment path))
        (children (notes/light/children-of path paths))
        (items (notes/light/notes-in path all-notes))
        (item-count (length items)))
      (html/tag
        "details"
        (html/attrs
          (list
            (html/attr "class" "tree-folder")
            (html/attr "open" "open")))
        (html/join
          (list
            (html/tag
              "summary"
              (html/attr "class" "tree-folder-header")
              (html/join
                (list
                  (html/span
                    (html/attr "class" "tree-folder-icon")
                    "&#x1F4C1; ")
                  (html/a
                    (string-append "/notes?folder=" path)
                    (html/text label))
                  (html/span
                    (html/attr "class" "muted")
                    (html/text (string-append " (" (as-str item-count)
    ")"))))))
            (notes/light/render-notes items)
            (if
              (null? children)
              ""
              (html/join
                (list/map
                  (lambda
                    (child)
                    (notes/light/render-folder child paths all-notes))
                  (notes/tree/sort-by-name children)))))))))

(define notes/light/render-notes
  (lambda
    (note-list)
    (if
      (null? note-list)
      ""
      (html/ul
        (html/attr "class" "tree-items")
        (html/join
          (list/map
            (lambda (n) (notes/light/note-li n))
            note-list))))))

(define notes/list
  (lambda () (notes/doc/list)))

(define notes/list-light
  (lambda
    ()
    (map
      (lambda
        (r)
        (let
          ((d (cdr r)))
          (list
            (car r)
            (as-str (dict-get d "folder"))
            (as-str (dict-get d "title"))
```

```
            (as-str (dict-get d "slug")))))
      (notes/doc/index-list))))

(define notes/list-light-cap
  (lambda () (notes/list-take (notes/list-light) 40)))

(define notes/list-take
  (lambda
    (xs n)
    (if
      (or (null? xs) (<= n 0))
      (list)
      (cons (car xs) (notes/list-take (cdr xs) (- n 1))))))

(define notes/make-fields
  (lambda
    (slug title outline tags kind day)
    (dict-set*
      "{}"
      (list
        "slug"
        (as-str slug)
        "title"
        (as-str title)
        "outline"
        (outline/encode outline)
        "tags"
        (as-str tags)
        "kind"
        (as-str kind)
        "day"
        (as-str day)))))

(define notes/migrate/legacy-all
  (lambda
    ()
    (list/map
      notes/migrate/legacy-one
      (rows (quote note) 2000))))

(define notes/migrate/legacy-delete-all
  (lambda
    ()
    (list/map
      (lambda
        (row)
        (let
          ((id (as-str (car row)))
            (slug (as-str (dict-get (cdr row) "slug"))))
          (if
            (null? (tree/get-uuid id))
            (string-append "kept-missing-tree:" slug)
            (seq
              (delete (quote note) id)
              (string-append "deleted:" slug)))))
      (rows (quote note) 2000))))

(define notes/migrate/legacy-force-all
  (lambda
    ()
    (list/map
      notes/migrate/legacy-force-one
      (rows (quote note) 2000))))

(define notes/migrate/legacy-force-one
  (lambda
    (row)
    (if
      (null? row)
      "skip-empty"
      (let
        ((id (as-str (car row))) (f (cdr row)))
        (let
```

```
          ((slug (as-str (dict-get f "slug")))
            (title (as-str (dict-get f "title")))
            (outline (outline/decode (dict-get f "outline"))))
          (let
            ((existing (tree/get-uuid id)))
            (if
              (null? existing)
              (notes/migrate/legacy-one row)
              (let
                ((path (as-str (dict-get existing "path"))))
                (seq
                  (notes/tree/save-outline path outline)
                  (roam/reindex slug outline)
                  (string-append "reoutlined:" slug)))))))))

(define notes/migrate/legacy-one
  (lambda
    (row)
    (if
      (null? row)
      "skip-empty"
      (let
        ((id (as-str (car row))) (f (cdr row)))
        (let
          ((slug (as-str (dict-get f "slug")))
            (title (as-str (dict-get f "title")))
            (tags (as-str (dict-get f "tags")))
            (kind (as-str (dict-get f "kind")))
            (day (as-str (dict-get f "day")))
            (folder (as-str (dict-get f "folder")))
            (outline (outline/decode (dict-get f "outline"))))
          (let
            ((existing (tree/get-uuid id))
              (by-slug (notes/tree/find-slug slug)))
            (if
              (not (null? existing))
              (string-append "exists-uuid:" slug)
              (if
                (not (null? by-slug))
                (string-append "exists-slug:" slug)
                (let
                  ((path (notes/tree/note-path folder slug))
                    (body
                      (dict-set*
                        "{}"
                        (list
                          "uuid"
                          id
                          "slug"
                          slug
                          "title"
                          title
                          "tags"
                          tags
                          "kind"
                          (if (string-eq kind "") "note" kind)
                          "day"
                          day))))
                  (seq
                    (notes/tree/ensure-folder (notes/tree/root) (whoami))
                    (tree/upsert path "note" "note" body)
                    (notes/tree/save-outline path outline)
                    (roam/reindex slug outline)
                    (string-append "migrated:" slug)))))))))))

(define notes/missing-html
  (lambda
    (slug)
    (html/join
      (list
        (html/h1 "Missing")
        (html/p*
          (html/attr "class" "err")
```

```
          (string-append "No note " (as-str slug)))
        (html/p* "" (html/a "/notes" "All notes"))))))

(define notes/move-folder
  (lambda
    (slug folder)
    (let*
      ((r (notes/doc/row (as-str slug)))
        (f (if (null? r) "{}" (cdr r)))
        (bv (dict-get f "body"))
        (f2 (if (null? bv) (dict-set f "body" "") f))
        (fields (dict-set f2 "folder" (as-str folder))))
      (if (null? r) () (update (quote note) (car r) fields)))))

(define notes/node-op
  (lambda
    ()
    (let
      ((slug (as-str (request-param "slug")))
        (op (as-str (request-param "op")))
        (nid (as-str (request-param "node-id"))))
      (let
        ((prev (notes/get slug)))
        (if
          (null? prev)
          (render-html
            (html/p* (html/attr "class" "err") "missing"))
          (let
            ((tree (notes/get-outline prev))
              (title (notes/get-title prev))
              (tags (notes/get-tags prev))
              (kind (notes/get-kind prev))
              (day (notes/get-day prev)))
            (let
              ((new-tree (notes/apply-op slug tree op nid)))
              (seq
                (notes/upsert
                  slug
                  title
                  new-tree
                  tags
                  kind
                  day)
                (render-html (outline/html-lite slug new-tree #f)))))))))

(define notes/node/delete
  (lambda
    (note-path node-id)
    (let
      ((p
        (string-append
          note-path
          ".h."
          (tree/path-label (as-str node-id)))))
      (let
        ((row (tree/get p)))
        (if (null? row) #f (seq (tree/delete p "recursive") #t)))))
  "Delete one heading node (and its subtree) at note-path.h.<node-id>. One
     tree write. Returns #t if deleted, #f if absent.")

(define notes/node/get
  (lambda
    (note-path node-id)
    (let
      ((p
        (string-append
          note-path
          ".h."
          (tree/path-label (as-str node-id)))))
      (let
        ((row (tree/get p)))
        (if
          (null? row)
```

```
          ()
          (notes/tree/body (notes/tree/parse row))))))
  "Read one heading node's body dict at note-path.h.<node-id>, or () if
     absent. Single-node read.")

(define notes/node/path
  (lambda
    (note-path node-id)
    (string-append
      note-path
      ".h."
      (tree/path-label (as-str node-id))))
  "The heading tree path note-path.h.<sanitized-node-id>. Pure string
     computation.")

(define notes/node/set-todo
  (lambda
    (note-path node-id todo)
    (let
      ((p
        (string-append
          note-path
          ".h."
          (tree/path-label (as-str node-id)))))
      (let
        ((row (tree/get p)))
        (if
          (null? row)
          #f
          (seq
            (tree/upsert
              p
              "heading"
              "note"
              (dict-set
                (notes/tree/body (notes/tree/parse row))
                "todo"
                (as-str todo)))
            #t)))))
  "Set the todo field of one heading node at note-path.h.<node-id>. One
     read + one write on a single node (no whole-tree rewrite). Returns #t
     if the node exists, #f otherwise.")

(define notes/node/upsert
  (lambda
    (note-path node-id title body todo)
    (let
      ((p
        (string-append
          note-path
          ".h."
          (tree/path-label (as-str node-id)))))
      (seq
        (tree/upsert
          p
          "heading"
          "note"
          (dict-set*
            "{}"
            (list
              "id"
              (as-str node-id)
              "title"
              (as-str title)
              "body"
              (as-str body)
              "todo"
              (as-str todo))))
        (as-str node-id))))
  "Upsert a single leaf heading node at note-path.h.<node-id>. One tree
     write (no whole-tree rewrite, no notes/doc/row read). Returns node-id.
     Fixes i-1787030118 / the data-loss clobber.")
```

```
(define notes/open-todos
  (lambda
    ()
    (list/filter
      (lambda
        (t)
        (outline/todo-open? (car (cdr (cdr (cdr t))))))
      (notes/all-todos))))

(define notes/outline-partial
  (lambda
    ()
    (let
      ((n (notes/get (request-param "slug"))))
      (if
        (null? n)
        (render-html
          (html/p* (html/attr "class" "err") "missing"))
        (render-html
          (outline/html-lite
            (as-str (request-param "slug"))
            (notes/get-outline n)
            #f))))))

(define notes/outline/all-todos
  (lambda
    ()
    (list/foldl
      (lambda
        (acc row)
        (let
          ((f (cdr row)))
          (let
            ((slug (as-str (dict-get f "slug")))
              (tree (outline/decode (dict-get f "outline"))))
            (list/foldl
              (lambda
                (a t)
                (cons
                  (list slug (car t) (car (cdr t)) (car (cdr (cdr t))))
                  a))
              acc
              (outline/todos tree)))))
      (list)
      (notes/doc/list))))

(define notes/outline/apply-op
  (lambda
    (slug tree op nid)
    (cond
      ((string-eq op "add-child")
        (outline/add-child-by-id
          tree
          nid
          (notes/outline/new-node)))
      ((string-eq op "add-sibling")
        (outline/insert-sibling
          tree
          nid
          (notes/outline/new-node)))
      ((string-eq op "delete")
        (if
          (string-eq (as-str nid) "root")
          tree
          (outline/delete tree nid)))
      ((string-eq op "move-up") (outline/move-up tree nid))
      ((string-eq op "move-down")
        (outline/move-down tree nid))
      ((string-eq op "promote")
        (if
          (string-eq (as-str nid) "root")
          tree
          (outline/promote tree nid)))
```

```
      ((string-eq op "demote")
        (if
          (string-eq (as-str nid) "root")
          tree
          (outline/demote tree nid)))
      ((string-eq op "set-title")
        (outline/update
          tree
          nid
          (lambda
            (n)
            (outline/set-title n (request-param "title")))))
      ((string-eq op "set-body")
        (outline/update
          tree
          nid
          (lambda (n) (outline/set-body n (request-param "body")))))
      ((string-eq op "set-todo")
        (outline/update
          tree
          nid
          (lambda (n) (outline/set-todo n (request-param "todo")))))
      ((string-eq op "cycle-todo")
        (outline/update
          tree
          nid
          (lambda (n) (outline/cycle-todo n))))
      ((string-eq op "archive")
        (if
          (string-eq (as-str nid) "root")
          tree
          (outline/update
            tree
            nid
            (lambda (n) (outline/archive n)))))
      ((string-eq op "unarchive")
        (if
          (string-eq (as-str nid) "root")
          tree
          (outline/update
            tree
            nid
            (lambda (n) (outline/unarchive n)))))
      ((string-eq op "slurp-forward")
        (if
          (string-eq (as-str nid) "root")
          tree
          (car
            (structed/slurp-forward
              outline/kids
              outline/set-kids
              tree
              nid))))
      ((string-eq op "barf-forward")
        (if
          (string-eq (as-str nid) "root")
          tree
          (car
            (structed/barf-forward
              outline/kids
              outline/set-kids
              tree
              nid))))
      (else tree))))

(define notes/outline/apply-op-sel
  (lambda
    (slug tree op nid)
    (let
      ((new-tree (notes/outline/apply-op slug tree op nid)))
      (let
        ((sel (notes/outline/sel-for op nid new-tree tree)))
        (list new-tree sel)))))
```

```
2237
2238 (define notes/outline/cycle-todo
2239   (lambda
2240     (slug nid)
2241     (let
2242       ((prev (notes/doc/get slug)))
2243       (if
2244         (null? prev)
2245         ()
2246         (let
2247           ((tree (notes/doc/outline prev))
2248             (title (notes/doc/title prev))
2249             (tags (notes/doc/tags prev))
2250             (kind (notes/doc/kind prev))
2251             (day (notes/doc/day prev)))
2252           (let
2253             ((new-tree
2254               (outline/update
2255                 tree
2256                 nid
2257                 (lambda (n) (outline/cycle-todo n)))))
2258             (notes/doc/upsert
2259               slug
2260               title
2261               new-tree
2262               tags
2263               kind
2264               day)))))))
2265
2266 (define notes/outline/eval-form
2267   (lambda
2268     (slug nid)
2269     (html/form
2270       "POST"
2271       "/partials/notes-node-op"
2272       (html/attrs
2273         (list
2274           (html/attr "class" "ol-eval")
2275           (html/attr "hx-post" "/partials/notes-node-op")
2276           (html/attr "hx-target" "#outline")
2277           (html/attr "hx-swap" "outerHTML")))
2278       (html/join
2279         (list
2280           (html/hidden "slug" (as-str slug))
2281           (html/hidden "op" "eval")
2282           (html/hidden "node-id" (as-str nid))
2283           (html/text-input
2284             "expr"
2285             (html/attrs
2286               (list
2287                 (html/attr
2288                   "placeholder"
2289                   "(outline/node (gensym \"n\") \"hi\")")
2290                 (html/attr "class" "ol-eval-expr")
2291                 (html/attr "style" "width:12rem"))))
2292           (html/select
2293             "target"
2294             (html/attr "class" "ol-eval-tgt")
2295             (html/join
2296               (list
2297                 (html/option "child" "child" #t)
2298                 (html/option "sibling" "sibling" #f)
2299                 (html/option "body" "body" #f))))
2300           (html/button
2301             (html/attrs
2302               (list
2303                 (html/attr "type" "submit")
2304                 (html/attr "class" "ol-btn")
2305                 (html/attr "title" "eval Lisp and splice result")))
2306             "eval"))))))
2307
2308 (define notes/outline/eval-result
2309   (lambda
```

```
    (slug nid expr)
    (let
      ((prev (notes/doc/get slug)))
      (if
        (null? prev)
        ()
        (let
          ((tree (notes/doc/outline prev)))
          (let
            ((node (outline/find tree (as-str nid))))
            (if
              (null? node)
              ()
              (seq
                (load-string
                  (string-append
                    "(define __notes_eval_fn (lambda (self doc slug) "
                    (as-str expr)
                    "))"))
                (__notes_eval_fn node prev (as-str slug)))))))))

(define notes/outline/eval-target
  (lambda
    ()
    (let
      ((tg (request-param "target")))
      (if
        (if (null? tg) #t (string-eq (as-str tg) ""))
        "child"
        (as-str tg)))))

(define notes/outline/new-node
  (lambda () (outline/node (gensym "n") "New node")))

(define notes/outline/node-op
  (lambda
    ()
    (let
      ((slug (as-str (request-param "slug")))
        (op (as-str (request-param "op")))
        (nid (as-str (request-param "node-id")))
        (show-archived (notes/doc/show-archived?)))
      (let
        ((prev (notes/doc/get slug)))
        (if
          (null? prev)
          (render-html
            (html/p* (html/attr "class" "err") "missing"))
          (let
            ((tree (notes/doc/outline prev))
              (title (notes/doc/title prev))
              (tags (notes/doc/tags prev))
              (kind (notes/doc/kind prev))
              (day (notes/doc/day prev)))
            (let
              ((res
                (if
                  (string-eq op "eval")
                  (notes/outline/node-op-eval tree nid slug)
                  (notes/outline/apply-op-sel slug tree op nid))))
              (let
                ((new-tree (car res)) (sel (as-str (car (cdr res)))))
                (seq
                  (notes/doc/upsert
                    slug
                    title
                    new-tree
                    tags
                    kind
                    day)
                  (render-html
                    (string-append
                      (outline/html-lite slug new-tree show-archived)
```

```
                      (html/void
                        "input"
                        (html/attrs
                          (list
                            (html/attr "type" "hidden")
                            (html/attr "id" "ol-selected-id")
                            (html/attr "name" "node-id")
                            (html/attr "hx-swap-oob" "true")
                            (html/attr "value" sel)))))))))))))))

(define notes/outline/node-op-eval
  (lambda
    (tree nid slug)
    (let
      ((et
        (notes/outline/splice-eval
          tree
          nid
          (notes/outline/eval-target)
          (notes/outline/eval-result
            slug
            nid
            (request-param "expr")))))
      (list et nid))))

(define notes/outline/open-todos
  (lambda
    ()
    (list/filter
      (lambda
        (t)
        (outline/todo-open? (car (cdr (cdr (cdr t))))))
      (notes/outline/all-todos))))

(define notes/outline/partial
  (lambda
    ()
    (let
      ((n (notes/doc/get (request-param "slug")))
        (show-archived (notes/doc/show-archived?)))
      (if
        (null? n)
        (render-html
          (html/p* (html/attr "class" "err") "missing"))
        (render-html
          (outline/html-lite
            (as-str (request-param "slug"))
            (notes/doc/outline n)
            show-archived))))))

(define notes/outline/sel-after-add-child
  (lambda
    (new-tree nid)
    (let
      ((node (structed/find outline/kids new-tree nid)))
      (if
        (null? node)
        nid
        (let
          ((ks (outline/kids node)))
          (if
            (null? ks)
            nid
            (structed/id (structed/list/nth ks 0)))))))))

(define notes/outline/sel-after-add-sibling
  (lambda
    (new-tree nid)
    (let
      ((parent
        (structed/find-parent outline/kids new-tree nid)))
      (if
        (null? parent)
```

```
        nid
        (let
          ((sibs (outline/kids parent)))
          (let
            ((i (structed/sibling-index outline/kids new-tree nid)))
            (if
              (>= (+ i 1) (structed/count sibs))
              nid
              (structed/id (structed/list/nth sibs (+ i 1)))))))))

(define notes/outline/sel-after-delete
  (lambda
    (tree nid)
    (let
      ((parent (structed/find-parent outline/kids tree nid)))
      (if
        (null? parent)
        nid
        (notes/outline/sel-after-delete-in
          parent
          (outline/kids parent)
          (structed/sibling-index outline/kids tree nid))))))

(define notes/outline/sel-after-delete-in
  (lambda
    (parent sibs i)
    (let
      ((after (structed/list/drop sibs (+ i 1))))
      (if
        (not (null? after))
        (structed/id (car after))
        (if
          (> i 0)
          (structed/id (structed/list/nth sibs (- i 1)))
          (structed/id parent))))))

(define notes/outline/sel-for
  (lambda
    (op nid new-tree old-tree)
    (cond
      ((string-eq op "delete")
        (if
          (string-eq (as-str nid) "root")
          nid
          (notes/outline/sel-after-delete old-tree nid)))
      ((string-eq op "add-child")
        (notes/outline/sel-after-add-child new-tree nid))
      ((string-eq op "add-sibling")
        (notes/outline/sel-after-add-sibling new-tree nid))
      (else nid))))

(define notes/outline/splice-eval
  (lambda
    (tree nid target result)
    (let
      ((t (as-str target)))
      (if
        (string-eq t "body")
        (outline/update
          tree
          nid
          (lambda (n) (outline/set-body n (as-str result))))
        (let
          ((child
            (if
              (outline/valid? result)
              result
              (outline/node (gensym "n") (as-str result)))))
          (if
            (string-eq t "sibling")
            (outline/insert-sibling tree nid child)
            (outline/add-child-by-id tree nid child)))))))
```

```
(define notes/page
  (lambda
    ()
    (if
      (string-eq (request-method) "POST")
      (notes/ui/handle-post)
      (let
        ((slug (request-param "slug")))
        (if
          (if (null? slug) #t (string-eq (as-str slug) ""))
          (html/page (list (notes/ui/index)))
          (html/page (list (notes/ui/show (as-str slug)))))))))

(define notes/recent-list
  (lambda
    ()
    (list/foldl
      (lambda
        (acc row)
        (string-append acc (notes/item-li row)))
      ""
      (notes/list))))

(define notes/refile
  (lambda
    (src-slug nid dst-slug)
    (let
      ((src (notes/get src-slug))
        (dst (notes/get dst-slug))
        (sid (as-str src-slug))
        (did (as-str dst-slug))
        (id (as-str nid)))
      (if
        (if (null? src) #t (null? dst))
        #f
        (if
          (string-eq sid did)
          #f
          (if
            (string-eq id "root")
            #f
            (let
              ((src-tree (notes/get-outline src))
                (dst-tree (notes/get-outline dst)))
              (let
                ((node (outline/find src-tree id)))
                (if
                  (null? node)
                  #f
                  (seq
                    (notes/upsert
                      did
                      (notes/get-title dst)
                      (outline/add-child-by-id dst-tree "root" node)
                      (notes/get-tags dst)
                      (notes/get-kind dst)
                      (notes/get-day dst))
                    (seq
                      (notes/upsert
                        sid
                        (notes/get-title src)
                        (outline/delete src-tree id)
                        (notes/get-tags src)
                        (notes/get-kind src)
                        (notes/get-day src))
                      #t)))))))))))

(define notes/refile-form
  (lambda
    (src-slug nid)
    (if
      (string-eq (as-str nid) "root")
      ""
```

```
      (html/form
        "POST"
        "/notes"
        (html/attr "class" "ol-refile")
        (html/join
          (list
            (html/hidden "action" "refile")
            (html/hidden "src_slug" (as-str src-slug))
            (html/hidden "node_id" (as-str nid))
            (html/select
              "dst_slug"
              (html/attr "class" "ol-refile-sel")
              (notes/refile-options src-slug))
            (html/button
              (html/attrs
                (list
                  (html/attr "type" "submit")
                  (html/attr "class" "ol-btn")
                  (html/attr "title" "move node to another note")))
              "refile")))))))

(define notes/refile-options
  (lambda
    (src-slug)
    (list/foldl
      (lambda
        (acc row)
        (let
          ((f (cdr row)))
          (let
            ((slug (as-str (dict-get f "slug")))
              (title (as-str (dict-get f "title"))))
            (if
              (string-eq slug (as-str src-slug))
              acc
              (string-append
                acc
                (html/option
                  slug
                  (if (string-eq title "") slug title)
                  #f))))))
      ""
      (notes/list))))

(define notes/refile/extract
  (lambda
    (src-slug nid)
    (let
      ((src (notes/doc/get src-slug)) (id (as-str nid)))
      (if
        (null? src)
        ()
        (if
          (string-eq id "root")
          ()
          (let
            ((src-tree (notes/doc/outline src)))
            (let
              ((pair (outline/detach src-tree id)))
              (if
                (null? pair)
                ()
                (let
                  ((node (car pair)) (src-without (car (cdr pair))))
                  (let
                    ((title (as-str (outline/title node)))
                      (body (as-str (outline/body node)))
                      (kids (outline/kids node))
                      (tags (notes/doc/tags src)))
                    (let
                      ((new-slug
                          (notes/refile/unique-slug (notes/refile/slugify
    title)))
```

```
                        (new-tree
                          (outline/set-kids
                            (outline/set-body (outline/node "root" title)
    body)
                            kids)))
                      (with-transaction
                        (lambda
                          ()
                          (seq
                            (notes/doc/upsert
                              new-slug
                              title
                              new-tree
                              tags
                              "note"
                              "")
                            (notes/doc/upsert
                              (as-str src-slug)
                              (notes/doc/title src)
                              src-without
                              (notes/doc/tags src)
                              (notes/doc/kind src)
                              (notes/doc/day src))
                            new-slug)))))))))))))

(define notes/refile/extract-form
  (lambda
    (src-slug nid)
    (if
      (string-eq (as-str nid) "root")
      ""
      (html/form
        "POST"
        "/notes"
        (html/attr "class" "ol-extract")
        (html/join
          (list
            (html/hidden "action" "extract")
            (html/hidden "src_slug" (as-str src-slug))
            (html/hidden "node_id" (as-str nid))
            (html/button
              (html/attrs
                (list
                  (html/attr "type" "submit")
                  (html/attr "class" "ol-btn")
                  (html/attr "title" "extract node into its own note")))
              "extract")))))))

(define notes/refile/extract-handle
  (lambda
    ()
    (let
      ((src (as-str (request-param "src_slug")))
        (nid (as-str (request-param "node_id"))))
      (let
        ((new (notes/refile/extract src nid)))
        (if
          (null? new)
          (redirect (string-append "/notes?slug=" src))
          (redirect (string-append "/notes?slug=" new)))))))

(define notes/refile/form
  (lambda
    (src-slug nid)
    (if
      (string-eq (as-str nid) "root")
      ""
      (html/form
        "POST"
        "/notes"
        (html/attr "class" "ol-refile")
        (html/join
          (list
```

```
            (html/hidden "action" "refile")
            (html/hidden "src_slug" (as-str src-slug))
            (html/hidden "node_id" (as-str nid))
            (html/select
              "dst_slug"
              (html/attr "class" "ol-refile-sel")
              (notes/refile/options src-slug))
            (html/button
              (html/attrs
                (list
                  (html/attr "type" "submit")
                  (html/attr "class" "ol-btn")
                  (html/attr "title" "move node to another note")))
              "refile")))))))

(define notes/refile/handle
  (lambda
    ()
    (let
      ((src (as-str (request-param "src_slug")))
        (nid (as-str (request-param "node_id")))
        (dst (as-str (request-param "dst_slug")))
        (dst-nid
          (if
            (null? (request-param "dst_node_id"))
            ""
            (as-str (request-param "dst_node_id")))))
      (seq
        (notes/refile/move src nid dst dst-nid)
        (redirect (string-append "/notes?slug=" src))))))

(define notes/refile/move
  (lambda
    (src-slug nid dst-slug dst-nid)
    (let
      ((src (notes/doc/get src-slug))
        (dst (notes/doc/get dst-slug))
        (sid (as-str src-slug))
        (did (as-str dst-slug))
        (id (as-str nid))
        (parent
          (let
            ((p (as-str dst-nid)))
            (if (string-eq p "") "root" p))))
      (if
        (if (null? src) #t (null? dst))
        #f
        (if
          (string-eq sid did)
          #f
          (if
            (string-eq id "root")
            #f
            (let
              ((src-tree (notes/doc/outline src))
                (dst-tree (notes/doc/outline dst)))
              (let
                ((pair (outline/detach src-tree id)))
                (if
                  (null? pair)
                  #f
                  (let
                    ((node (car pair)) (src-without (car (cdr pair))))
                    (with-transaction
                      (lambda
                        ()
                        (seq
                          (notes/doc/upsert
                            did
                            (notes/doc/title dst)
                            (outline/add-child-by-id dst-tree parent node)
                            (notes/doc/tags dst)
                            (notes/doc/kind dst)
```

```
                            (notes/doc/day dst))
                          (notes/doc/upsert
                            sid
                            (notes/doc/title src)
                            src-without
                            (notes/doc/tags src)
                            (notes/doc/kind src)
                            (notes/doc/day src))
                          #t))))))))))))

(define notes/refile/options
  (lambda
    (src-slug)
    (list/foldl
      (lambda
        (acc row)
        (let
          ((f (cdr row)))
          (let
            ((slug (as-str (dict-get f "slug")))
              (title (as-str (dict-get f "title"))))
            (if
              (string-eq slug (as-str src-slug))
              acc
              (string-append
                acc
                (html/option
                  slug
                  (if (string-eq title "") slug title)
                  #f))))))
      ""
      (notes/doc/list))))

(define notes/refile/slug-char-ok?
  (lambda
    (c)
    (string-contains?
      "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789"
      (as-str c))))

(define notes/refile/slugify
  (lambda
    (title)
    (let
      ((raw
        (notes/refile/slugify-acc
          (string-trim (as-str title))
          0
          ""
          #t)))
      (let
        ((n (string-length raw)))
        (let
          ((s
            (if
              (if
                (> n 0)
                (string-eq (substring raw (- n 1) 1) "-")
                #f)
              (substring raw 0 (- n 1))
              raw)))
          (if
            (string-eq s "")
            (string-append "note-" (as-str (gensym "n")))
            (if (> (string-length s) 48) (substring s 0 48) s)))))))

(define notes/refile/slugify-acc
  (lambda
    (s i acc last-hyphen?)
    (if
      (>= i (string-length s))
      acc
      (let
```

```
        ((c (substring s i 1)))
        (if
          (notes/refile/slug-char-ok? c)
          (notes/refile/slugify-acc
            s
            (+ i 1)
            (string-append acc c)
            #f)
          (if
            last-hyphen?
            (notes/refile/slugify-acc s (+ i 1) acc #t)
            (notes/refile/slugify-acc
              s
              (+ i 1)
              (string-append acc "-")
              #t)))))))

(define notes/refile/unique-slug
  (lambda
    (base)
    (if
      (null? (notes/doc/row (as-str base)))
      (as-str base)
      (notes/refile/unique-slug
        (string-append (as-str base) "-" (as-str (gensym "x")))))))

(define notes/row
  (lambda
    (slug)
    (let
      ((rs (find-rows (quote note) "slug" (as-str slug) 1)))
      (if (null? rs) () (car rs)))))

(define notes/search
  (lambda (q) (notes/doc/search q)))

(define notes/show-archived?
  (lambda
    ()
    (string-eq (as-str (request-param "show")) "archived")))

(define notes/show-body
  (lambda
    (slug title outline tags)
    (html/join
      (list
        (html/p*
          (html/attr "class" "muted")
          (html/join
            (list
              (html/a "/notes" "Notes")
              " / "
              (html/code (as-str slug)))))
        (html/div
          (html/attr "class" "note-title-row")
          (html/join
            (list
              (html/h1 (html/text title))
              (html/form
                "POST"
                "/notes"
                (html/attrs
                  (list
                    (html/attr "class" "note-del note-del-page")
                    (html/attr "hx-confirm" "Delete this entire note?")))
                (html/join
                  (list
                    (html/hidden "action" "delete")
                    (html/hidden "slug" (as-str slug))
                    (html/button
                      (html/attrs
                        (list
                          (html/attr "type" "submit")
```

```
                          (html/attr "class" "secondary")
                          (html/attr "title" "delete note")))
                      "Delete note")))))))
        (html/p*
          (html/attr "class" "muted")
          (string-append "tags: " (as-str tags)))
        (outline/html-lite slug outline #f)
        (html/tag
          "details"
          (html/attr "class" "ol-debug")
          (html/join
            (list
              (html/tag
                "summary"
                (html/attr "class" "muted")
                "Raw JSON (debug)")
              (notes/edit-form slug outline))))
        (html/tag
          "script"
          ""
          (string-append

    "document.getElementById('outline').addEventListener('blur',function(e){"
            "var t=e.target;if(!t.dataset||!t.dataset.field)return;"
            "var p='slug="
            slug
            "&op=set-'+t.dataset.field+'&node-id='+t.dataset.id;"
            "p+='&'+t.dataset.field+'='+encodeURIComponent(t.innerText);"

    "fetch('/partials/notes-node-op',{method:'POST',headers:{'Content-Type':'application/x-www-form-urlencoded'},body:p});"
            "},true);"))))))

(define notes/show-html
  (lambda
    (slug)
    (let
      ((n (notes/get slug)))
      (if
        (null? n)
        (notes/missing-html slug)
        (notes/show-body
          slug
          (notes/get-title n)
          (notes/get-outline n)
          (notes/get-tags n))))))

(define notes/tree-css
  (css/sheet
    (list
      (css/raw
        "/* notes-tree layout (component-scoped; colors via var(--*)) */")
      (css/rule
        ".notes-tree"
        (list
          "list-style"
          "none"
          "padding"
          "0"
          "margin"
          "0"))
      (css/rule
        ".notes-tree ul.tree-root"
        (list
          "list-style"
          "none"
          "padding"
          "0"
          "margin"
          "0"))
      (css/rule
        ".notes-tree .tree-folder"
        (list
          "margin"
```

```
3036            ".25rem 0"
3037            "padding-left"
3038            ".5rem"
3039            "border-left"
3040            "2px solid var(--border)"))
3041        (css/rule
3042          ".notes-tree .tree-folder-header"
3043          (list
3044            "display"
3045            "flex"
3046            "align-items"
3047            "center"
3048            "gap"
3049            ".35rem"
3050            "padding"
3051            ".15rem 0"
3052            "font-weight"
3053            "700"))
3054        (css/rule
3055          ".notes-tree .tree-folder-header a"
3056          (list "text-decoration" "none" "color" "var(--link)"))
3057        (css/rule
3058          ".notes-tree .tree-folder-header a:hover"
3059          (list
3060            "text-decoration"
3061            "underline"
3062            "color"
3063            "var(--ink)"))
3064        (css/rule
3065          ".notes-tree .tree-folder-icon"
3066          (list "font-size" ".9em"))
3067        (css/rule
3068          ".notes-tree ul.tree-children"
3069          (list
3070            "list-style"
3071            "none"
3072            "padding"
3073            "0 0 0 1rem"
3074            "margin"
3075            ".15rem 0"))
3076        (css/rule
3077          ".notes-tree li.note-item"
3078          (list "border-bottom" "none" "padding" ".15rem 0"))
3079        (css/rule
3080          ".notes-tree li.note-item > a"
3081          (list "flex" "1 1 auto" "color" "var(--link)"))
3082        (css/rule
3083          ".notes-tree li.note-item > a:hover"
3084          (list "color" "var(--ink)"))
3085        (css/rule
3086          ".notes-tree .tree-uncategorized"
3087          (list
3088            "margin-top"
3089            "1rem"
3090            "padding-top"
3091            ".5rem"
3092            "border-top"
3093            "1px solid var(--border)"))
3094        (css/rule
3095          ".notes-tree .tree-uncategorized h3"
3096          (list
3097            "font-size"
3098            ".95rem"
3099            "color"
3100            "var(--muted)"
3101            "margin"
3102            ".5rem 0 .25rem"
3103            "text-transform"
3104            "uppercase"
3105            "letter-spacing"
3106            ".05em"))
3107        (css/rule
3108          ".notes-tree li.note-item form.note-del button"
```

```
        (list
          "border"
          "1px solid var(--border)"
          "background"
          "var(--panel)"
          "color"
          "var(--muted)"
          "padding"
          ".2rem .4rem"
          "font-size"
          ".8rem"
          "min-height"
          "auto"
          "width"
          "auto"))
      (css/rule
        ".notes-tree li.note-item form.note-del button:hover"
        (list "background" "var(--ink)" "color" "var(--bg)")))))

(define notes/tree-css-patch
  (css/sheet
    (list
      (css/raw
        "/* notes/tree-css-patch: retired (was v4 global clobber; now
     empty) */"))))

(define notes/tree/all-paths
  (lambda
    (names)
    (list/foldl
      (lambda
        (acc name)
        (if
          (string-eq name "")
          acc
          (let
            ((parent (notes/tree/parent-of name)))
            (if
              (string-eq parent "")
              acc
              (if
                (notes/doc/folder-seen? acc parent)
                acc
                (if
                  (notes/doc/folder-seen? names parent)
                  acc
                  (append acc (list parent))))))))))
      (list)
      names)))

(define notes/tree/body
  (lambda
    (node)
    (if (null? node) () (dict-get node "body"))))

(define notes/tree/butlast
  (lambda
    (xs)
    (if
      (null? xs)
      (list)
      (if
        (null? (cdr xs))
        (list)
        (cons (car xs) (notes/tree/butlast (cdr xs)))))))

(define notes/tree/children-of
  (lambda
    (parent names)
    (list/filter
      (lambda
        (name)
        (if
```

```
          (string-eq name "")
          #f
          (let
            ((p (notes/tree/parent-of name)))
            (string-eq p parent))))
      names)))

(define notes/tree/clear-headings
  (lambda
    (note-path)
    (list/foldl
      (lambda
        (acc nj)
        (let
          ((n (notes/tree/parse nj)))
          (if
            (null? n)
            acc
            (if
              (string-eq (as-str (dict-get n "kind")) "heading")
              (seq
                (tree/delete (as-str (dict-get n "path")) "recursive")
                acc)
              acc))))
      0
      (tree/list note-path 1))))

(define notes/tree/details-css
  (lambda
    ()
    (css/sheet
      (list
        (css/raw "/* details/summary tree nesting */")
        (css/rule
          "summary.tree-folder-header"
          (list "cursor" "pointer" "list-style" "none"))
        (css/rule
          "summary.tree-folder-header::-webkit-details-marker"
          (list "display" "none"))
        (css/rule
          "summary.tree-folder-header::marker"
          (list "content" ""))
        (css/rule
          ".notes-tree ul.tree-items"
          (list
            "list-style"
            "none"
            "padding"
            "0 0 0 1rem"
            "margin"
            ".15rem 0"))
        (css/rule
          ".notes-tree > details.tree-folder"
          (list "margin" ".25rem 0"))
        (css/raw
          "/* refile modal (slug picker -- no O(n) options) */")
        (css/rule
          "details.ol-refile-modal"
          (list "display" "inline-block" "position" "relative"))
        (css/rule
          "details.ol-refile-modal > summary.ol-btn"
          (list "cursor" "pointer" "list-style" "none"))
        (css/rule
          "details.ol-refile-modal > summary.ol-btn::-webkit-details-marker"
          (list "display" "none"))
        (css/rule
          "details.ol-refile-modal[open] > .ol-refile-body"
          (list
            "position"
            "absolute"
            "z-index"
            "20"
            "background"
```

```
            "var(--card,#fff)"
            "border"
            "1px solid var(--muted,#888)"
            "padding"
            ".5rem"
            "min-width"
            "16rem"
            "box-shadow"
            "0 2px 8px rgba(0,0,0,.12)"))
        (css/rule
          ".ol-refile-body .ol-refile-sel"
          (list "width" "12rem" "max-width" "100%"))
        (css/rule
          ".ol-refile-body .ol-refile-label"
          (list "font-size" ".85rem" "margin-right" ".25rem"))
        (css/raw "/* find / search modal */")
        (css/rule
          "details.ol-find-modal"
          (list "display" "inline-block" "position" "relative"))
        (css/rule
          "details.ol-find-modal > summary.ol-btn"
          (list "cursor" "pointer" "list-style" "none"))
        (css/rule
          "details.ol-find-modal > summary.ol-btn::-webkit-details-marker"
          (list "display" "none"))
        (css/rule
          "details.ol-find-modal[open] > .ol-find-body,\n
      details.ol-refile-modal[open] > .ol-refile-body.ol-find-wide"
          (list
            "position"
            "absolute"
            "z-index"
            "30"
            "right"
            "0"
            "background"
            "var(--card,#fff)"
            "border"
            "1px solid var(--muted,#888)"
            "padding"
            ".5rem"
            "min-width"
            "22rem"
            "max-width"
            "min(36rem, 92vw)"
            "box-shadow"
            "0 4px 16px rgba(0,0,0,.14)"))
        (css/rule
          ".ol-find-input"
          (list
            "width"
            "100%"
            "box-sizing"
            "border-box"
            "padding"
            ".35rem .5rem"
            "margin"
            "0 0 .35rem 0"))
        (css/rule
          ".ol-find-results"
          (list
            "max-height"
            "18rem"
            "overflow"
            "auto"
            "margin"
            "0"
            "padding"
            "0"
            "list-style"
            "none"))
        (css/rule
          ".ol-find-group"
```

```
          (list
            "font-size"
            ".75rem"
            "font-weight"
            "600"
            "text-transform"
            "uppercase"
            "letter-spacing"
            ".04em"
            "color"
            "var(--muted,#666)"
            "padding"
            ".4rem .35rem .15rem"))
        (css/rule
          "a.ol-find-hit, button.ol-find-hit"
          (list
            "display"
            "block"
            "width"
            "100%"
            "text-align"
            "left"
            "border"
            "0"
            "background"
            "transparent"
            "padding"
            ".35rem .5rem .35rem 1rem"
            "cursor"
            "pointer"
            "color"
            "inherit"
            "text-decoration"
            "none"
            "font"
            "inherit"
            "box-sizing"
            "border-box"))
        (css/rule
          "a.ol-find-hit:hover, button.ol-find-hit:hover,\n
      a.ol-find-hit.is-active, button.ol-find-hit.is-active"
          (list "background" "var(--accent-bg,#eef3ff)"))
        (css/rule
          ".ol-find-hit-path"
          (list "font-size" ".85rem"))
        (css/rule
          ".ol-find-hit-meta"
          (list
            "font-size"
            ".75rem"
            "color"
            "var(--muted,#666)"
            "margin-left"
            ".35rem"))
        (css/rule
          ".ol-find-empty"
          (list
            "padding"
            ".5rem"
            "color"
            "var(--muted,#666)"
            "font-size"
            ".9rem"))
        (css/rule
          ".ol-find-bar"
          (list
            "display"
            "flex"
            "gap"
            ".5rem"
            "align-items"
            "center"
            "margin"
```

```
            "0 0 .75rem 0"))
        (css/rule
          ".ol-find-bar details.ol-find-modal"
          (list "flex" "1 1 auto"))
        (css/rule
          ".ol-find-bar .ol-find-body"
          (list "left" "0" "right" "auto"))
        (css/raw "/* create form collapsed behind summary */")
        (css/rule
          "details.notes-create > summary.notes-create-sum"
          (list
            "cursor"
            "pointer"
            "font-weight"
            "600"
            "list-style"
            "none"))
        (css/rule
          "details.notes-create > summary.notes-create-sum::-webkit-details-marker"
          (list "display" "none"))
        (css/rule
          "details.notes-create > .notes-create-body"
          (list "margin-top" ".5rem"))))))

(define notes/tree/enabled?
  (lambda () #t))

(define notes/tree/ensure-chain
  (lambda
    (base segs)
    (if
      (null? segs)
      base
      (let
        ((next
          (string-append base "." (tree/path-label (car segs)))))
        (seq
          (notes/tree/ensure-folder next (car segs))
          (notes/tree/ensure-chain next (cdr segs)))))))

(define notes/tree/ensure-folder
  (lambda
    (path title)
    (let
      ((cur (tree/get path)))
      (if
        (null? cur)
        (tree/upsert
          path
          "folder"
          "note"
          (dict-set*
            "{}"
            (list "uuid" (random-uuid) "title" (as-str title))))
        path))))

(define notes/tree/find-slug
  (lambda
    (slug)
    (let
      ((hits (tree/find-by "note" "slug" (as-str slug) 5)))
      (if
        (null? hits)
        ()
        (notes/tree/node->row (car hits) #t)))))

(define notes/tree/folder-label
  (lambda
    (path)
    (let
      ((pos (string-find-pos path "/" 0 1)))
      (if
```

```
        (< pos 0)
        path
        (substring
          path
          (+ pos 1)
          (- (string-length path) (+ pos 1)))))))

(define notes/tree/folder-of-path
  (lambda
    (path)
    (let
      ((root (notes/tree/root)) (p (as-str path)))
      (if
        (string-starts-with? p (string-append root "."))
        (let
          ((rest
            (substring
              p
              (+ (string-length root) 1)
              (- (string-length p) (+ (string-length root) 1)))))
          (let
            ((parts (string-split rest ".")))
            (notes/tree/join-slash (notes/tree/butlast parts))))
        ""))))

(define notes/tree/heading-kids
  (lambda
    (parent-path)
    (list/foldl
      (lambda
        (acc nj)
        (let
          ((n (notes/tree/parse nj)))
          (if
            (null? n)
            acc
            (if
              (string-eq (as-str (dict-get n "kind")) "heading")
              (let
                ((b (notes/tree/body n)))
                (let
                  ((p (as-str (dict-get n "path"))))
                  (append
                    acc
                    (list
                      (outline/node*
                        (as-str (dict-get b "id"))
                        (as-str (dict-get b "title"))
                        (as-str (dict-get b "body"))
                        (as-str (dict-get b "todo"))
                        (notes/tree/heading-kids p))))))
              acc))))
      (list)
      (tree/list parent-path 1))))

(define notes/tree/join-slash
  (lambda
    (xs)
    (if
      (null? xs)
      ""
      (if
        (null? (cdr xs))
        (as-str (car xs))
        (string-append
          (as-str (car xs))
          "/"
          (notes/tree/join-slash (cdr xs)))))))

(define notes/tree/list-notes
  (lambda () (tree/note-index (notes/tree/root) 50000)))

(define notes/tree/load-outline
```

```
  (lambda
    (note-path title)
    (outline/node*
      "root"
      (as-str title)
      (notes/tree/root-body note-path)
      ""
      (notes/tree/heading-kids note-path))))

(define notes/tree/node->row
  (lambda
    (nj load-outline?)
    (let
      ((n (notes/tree/parse nj)))
      (if
        (null? n)
        ()
        (if
          (not (string-eq (as-str (dict-get n "kind")) "note"))
          ()
          (let
            ((b (notes/tree/body n))
              (path (as-str (dict-get n "path"))))
            (let
              ((u (dict-get b "uuid"))
                (title (as-str (dict-get b "title"))))
              (let
                ((uuid
                  (if (null? u) (as-str (dict-get n "id")) (as-str u))))
                (let
                  ((fields
                    (list
                      "slug"
                      (as-str (dict-get b "slug"))
                      "title"
                      title
                      "tags"
                      (as-str (dict-get b "tags"))
                      "kind"
                      (as-str (dict-get b "kind"))
                      "day"
                      (as-str (dict-get b "day"))
                      "folder"
                      (notes/tree/folder-of-path path)
                      "_path"
                      path)))
                  (cons
                    uuid
                    (dict-set*
                      "{}"
                      (if
                        load-outline?
                        (append
                          fields
                          (list
                            "outline"
                            (outline/encode (notes/tree/load-outline path
    title))))
                        (append fields (list "outline" "[]"))))))))))))

(define notes/tree/note-path
  (lambda
    (folder slug)
    (let
      ((base
        (notes/tree/ensure-chain
          (notes/tree/root)
          (notes/tree/split-folder folder))))
      (string-append base "." (tree/path-label slug)))))

(define notes/tree/parent-of
  (lambda
    (path)
```

```
    (let
      ((pos (string-find-pos path "/" 0 1)))
      (if (< pos 0) "" (substring path 0 pos)))))

(define notes/tree/parse
  (lambda (s) (if (null? s) () (json-parse (as-str s)))))

(define notes/tree/render
  (lambda
    ()
    (let*
      ((names (notes/doc/folder-names))
        (all-names (append names (notes/tree/all-paths names)))
        (roots
          (notes/tree/sort-by-name
            (notes/tree/children-of "" all-names)))
        (uncat (notes/doc/list-folder "")))
      (html/div
        (html/attr "class" "notes-tree")
        (html/join
          (list
            (if
              (null? uncat)
              ""
              (html/tag
                "details"
                (html/attrs
                  (list
                    (html/attr "class" "tree-folder tree-uncategorized")
                    (html/attr "open" "open")))
                (html/join
                  (list
                    (html/tag
                      "summary"
                      (html/attr "class" "tree-folder-header")
                      (html/join
                        (list
                          (html/span
                            (html/attr "class" "tree-folder-icon")
                            "&#x1F4C2; ")
                          (html/text "(uncategorized)")
                          (html/span
                            (html/attr "class" "muted")
                            (html/text
                              (string-append " (" (as-str (length uncat))
    ")"))))))
                    (html/ul
                      (html/attr "class" "tree-items")
                      (notes/ui/folder-list "")))))))
            (html/join
              (list/map
                (lambda
                  (root)
                  (notes/tree/render-folder root all-names 0))
                roots)))))))

(define notes/tree/render-folder
  (lambda
    (path names depth)
    (let*
      ((label (notes/tree/folder-label path))
        (children (notes/tree/children-of path names))
        (items (notes/doc/list-folder path))
        (item-count (length items)))
      (html/tag
        "details"
        (html/attrs
          (list
            (html/attr "class" "tree-folder")
            (html/attr "open" "open")))
        (html/join
          (list
            (html/tag
```

```
              "summary"
              (html/attr "class" "tree-folder-header")
              (html/join
                (list
                  (html/span
                    (html/attr "class" "tree-folder-icon")
                    "&#x1F4C1; ")
                  (html/a
                    (string-append "/notes?folder=" path)
                    (html/text label))
                  (html/span
                    (html/attr "class" "muted")
                    (html/text (string-append " (" (as-str item-count)
    ")"))))))
            (if
              (null? items)
              ""
              (html/ul
                (html/attr "class" "tree-items")
                (notes/ui/folder-list path)))
            (if
              (null? children)
              ""
              (html/join
                (list/map
                  (lambda
                    (child)
                    (notes/tree/render-folder child names (+ depth 1)))
                  (notes/tree/sort-by-name children)))))))))

(define notes/tree/root
  (lambda () (tree/garden-root)))

(define notes/tree/root-body
  (lambda
    (note-path)
    (let
      ((cur (tree/get note-path)))
      (if
        (null? cur)
        ""
        (let
          ((b
            (dict-get
              (notes/tree/body (notes/tree/parse cur))
              "body")))
          (if (null? b) "" (as-str b)))))))

(define notes/tree/save-outline
  (lambda
    (note-path outline)
    (seq
      (notes/tree/clear-headings note-path)
      (notes/tree/write-root-body
        note-path
        (outline/body outline))
      (list/foldl
        (lambda
          (acc kid)
          (seq (notes/tree/write-heading note-path kid) acc))
        0
        (outline/kids outline))
      note-path)))

(define notes/tree/sort-by-name
  (lambda
    (names)
    (if
      (null? names)
      (list)
      (let
        ((pivot (car names)) (rest (cdr names)))
        (append
```

```
          (notes/tree/sort-by-name
            (list/filter (lambda (x) (string<? x pivot)) rest))
          (list pivot)
          (notes/tree/sort-by-name
            (list/filter (lambda (x) (not (string<? x pivot))) rest)))))))

(define notes/tree/split-folder
  (lambda
    (folder)
    (list/filter
      (lambda (s) (not (string-eq (as-str s) "")))
      (string-split (as-str folder) "/"))))

(define notes/tree/style
  (lambda () (html/tag "style" "" notes/tree/style-css)))

(define notes/tree/style-css
  (string-append
    (css/render notes/tree-css)
    (css/render notes/tree-css-patch)
    (css/render (notes/tree/details-css))))

(define notes/tree/write-heading
  (lambda
    (parent-path node)
    (let
      ((nid (tree/path-label (outline/id node)))
        (path
          (string-append
            parent-path
            ".h."
            (tree/path-label (outline/id node)))))
      (seq
        (tree/upsert
          path
          "heading"
          "note"
          (dict-set*
            "{}"
            (list
              "id"
              (as-str (outline/id node))
              "title"
              (as-str (outline/title node))
              "body"
              (as-str (outline/body node))
              "todo"
              (as-str (outline/todo node)))))
        (list/foldl
          (lambda
            (acc kid)
            (seq (notes/tree/write-heading path kid) acc))
          0
          (outline/kids node))
        path))))

(define notes/tree/write-root-body
  (lambda
    (note-path body)
    (let
      ((cur (tree/get note-path)))
      (if
        (null? cur)
        note-path
        (let*
          ((n (notes/tree/parse cur)) (b (notes/tree/body n)))
          (seq
            (tree/upsert
              note-path
              "note"
              "note"
              (dict-set b "body" (as-str body)))
            note-path))))))
```

```
(define notes/ui/create-form
  (lambda
    ()
    (html/tag
      "details"
      (html/attr "class" "notes-create card")
      (html/join
        (list
          (html/tag
            "summary"
            (html/attr "class" "notes-create-sum")
            "New note")
          (html/form
            "POST"
            "/notes"
            (html/attr "class" "notes-create-body")
            (html/join
              (list
                (html/hidden "action" "create")
                (html/label "" "Title ")
                (html/text-input
                  "title"
                  (html/attr "placeholder" "New note"))
                " "
                (html/label "" "Slug ")
                (html/text-input
                  "slug"
                  (html/attr "placeholder" "my-note"))
                " "
                (html/label "" "Folder ")
                (html/text-input
                  "folder"
                  (html/attr "placeholder" "wiki/entities"))
                " "
                (html/button (html/attr "type" "submit") "Create")))))))))

(define notes/ui/do-create
  (lambda
    (title slug folder)
    (let
      ((s
          (if
            (if
              (null? slug)
              #t
              (string-eq (string-trim (as-str slug)) ""))
            (string-append
              "note-"
              (as-str (count (notes/doc/list))))
            (string-trim (as-str slug))))
        (fld
          (if (null? folder) "" (string-trim (as-str folder)))))
      (seq
        (notes/doc/upsert-in
          s
          (as-str title)
          (notes/doc/empty-outline (as-str title))
          ""
          "note"
          ""
          fld)
        (redirect (string-append "/notes?slug=" s))))))

(define notes/ui/do-delete
  (lambda
    ()
    (let
      ((slug (as-str (request-param "slug"))))
      (seq (notes/doc/delete slug) (redirect "/notes")))))

(define notes/ui/do-save
  (lambda
```

```
    (slug raw)
    (let
      ((slug-s (as-str slug))
        (prev (notes/doc/get (as-str slug))))
      (let
        ((title (if (null? prev) slug-s (notes/doc/title prev)))
          (tags (if (null? prev) "" (notes/doc/tags prev)))
          (kind (if (null? prev) "note" (notes/doc/kind prev)))
          (day (if (null? prev) "" (notes/doc/day prev))))
        (if
          (or
            (null? raw)
            (string-eq (string-trim (as-str raw)) ""))
          (seq
            (notes/doc/upsert
              slug-s
              title
              (outline/node "root" "Untitled")
              tags
              kind
              day)
            (redirect (string-append "/notes?slug=" slug-s)))
          (let
            ((parsed (json-parse (as-str raw))))
            (if
              (outline/valid? parsed)
              (seq
                (notes/doc/upsert
                  slug-s
                  title
                  parsed
                  tags
                  kind
                  day)
                (redirect (string-append "/notes?slug=" slug-s)))
              (html/page
                (list
                  (html/h1 (string-append "Could not save " slug-s))
                  (html/p*
                    (html/attr "class" "err")
                    "The outline is malformed. Each node must be a
    5-element list [id title body todo kids]. The note was not changed.")
                  (html/p*
                    ""
                    (html/a
                      (string-append "/notes?slug=" slug-s)
                      "Back to the note"))
                  (html/tag
                    "pre"
                    (html/attrs (list (html/attr "class" "err")))
                    (as-str raw)))))))))

(define notes/ui/do-undo
  (lambda
    ()
    (let
      ((slug (as-str (request-param "slug")))
        (n (notes/doc/get (as-str (request-param "slug")))))
      (let
        ((folder (notes/doc/folder n))
          (path (notes/tree/note-path folder slug)))
        (seq
          (undo path)
          (redirect (string-append "/notes?slug=" slug)))))))

(define notes/ui/edit-form
  (lambda
    (slug outline)
    (html/form
      "POST"
      "/notes"
      ""
      (html/join
```

```
        (list
          (html/hidden "action" "save")
          (html/hidden "slug" slug)
          (html/tag
            "textarea"
            (html/attrs
              (list
                (html/attr "name" "outline")
                (html/attr "rows" "16")))
            (outline/encode outline))
          (html/p*
            ""
            (html/button (html/attr "type" "submit") "Save outline")))))))

(define notes/ui/find-modal
  (lambda
    ()
    (html/tag
      "details"
      (html/attrs
        (list
          (html/attr "class" "ol-find-modal")
          (html/attr "id" "ol-find-modal")))
      (html/join
        (list
          (html/tag
            "summary"
            (html/attrs
              (list
                (html/attr "class" "ol-btn")
                (html/attr "title" "Find note (Ctrl/Cmd-K)")))
            "find")
          (html/tag
            "div"
            (html/attr "class" "ol-find-body")
            (html/join
              (list
                (html/void
                  "input"
                  (html/attrs
                    (list
                      (html/attr "type" "search")
                      (html/attr "class" "ol-find-input")
                      (html/attr "name" "q")
                      (html/attr "placeholder" "Find note or header?")
                      (html/attr "autocomplete" "off")
                      (html/attr "hx-get" "/partials/notes-search")
                      (html/attr
                        "hx-trigger"
                        "input changed delay:200ms, search")
                      (html/attr "hx-target" "next .ol-find-results")
                      (html/attr "hx-swap" "outerHTML")
                      (html/attr "hx-include" "this"))))
                (html/div
                  (html/attr "class" "ol-find-results")
                  (html/div
                    (html/attr "class" "ol-find-empty")
                    "Type to find a note or header?")))))))))

(define notes/ui/find-script
  (lambda
    ()
    (html/tag
      "script"
      ""
      (string-append
        "(function(){"
        "function hits(root){return
    root?Array.prototype.slice.call(root.querySelectorAll('[data-ol-find-hit]')):[];}"
        "function setActive(root,i){var
    hs=hits(root);if(!hs.length)return;hs.forEach(function(h){h.classList.remove('is-active');});"
        "var
    n=((i%hs.length)+hs.length)%hs.length;hs[n].classList.add('is-active');hs[n].scrollIntoView({block:'nearest'});root.dat
```

```
4047        "function openFind(sel){var
    d=document.querySelector(sel||'details.ol-find-modal');if(!d)return;d.open=true;"
4048        "var
    inp=d.querySelector('.ol-find-input');if(inp){inp.focus();inp.select();}}"
4049        "document.addEventListener('keydown',function(e){"
4050        "var t=e.target;var
    typing=t&&(t.tagName==='INPUT'||t.tagName==='TEXTAREA'||t.isContentEditable);"
4051
    "if((e.metaKey||e.ctrlKey)&&!e.shiftKey&&(e.key==='k'||e.key==='K')){e.preventDefault();openFind('details.ol-find-modal
4052
    "if(e.key==='Escape'){document.querySelectorAll('details.ol-find-modal[open],details.ol-refile-modal[open]').forEach(fu
4053        "var
    body=t&&t.closest&&t.closest('.ol-find-body,.ol-refile-body');if(!body)return;"
4054        "var box=body.querySelector('.ol-find-results')||body;"
4055
    "if(e.key==='ArrowDown'){e.preventDefault();setActive(box,(parseInt(box.dataset.olFindIdx||'-1',10)+1));return;}"
4056
    "if(e.key==='ArrowUp'){e.preventDefault();setActive(box,(parseInt(box.dataset.olFindIdx||'0',10)-1));return;}"
4057        "if(e.key==='Enter'&&!e.shiftKey){var hs=hits(box);var
    i=parseInt(box.dataset.olFindIdx||'-1',10);"
4058
    "if(i>=0&&hs[i]){e.preventDefault();if(hs[i].tagName==='BUTTON'){hs[i].click();}else
    if(hs[i].tagName==='A'){window.location=hs[i].href;}else{var
    b=hs[i].querySelector('button,[type=submit]');if(b)b.click();else
    hs[i].click();}}}"
4059        "});"
4060        "document.body.addEventListener('htmx:afterSwap',function(ev){var
    t=ev.target;if(!t||!t.classList)return;"
4061
    "if(t.classList.contains('ol-find-results')||t.querySelector&&t.querySelector('.ol-find-results')){var
    box=t.classList.contains('ol-find-results')?t:t.querySelector('.ol-find-results');if(box)setActive(box,0);}});"
4062        "document.addEventListener('toggle',function(e){var
    d=e.target;if(!d||!d.classList)return;"
4063
    "if((d.classList.contains('ol-find-modal')||d.classList.contains('ol-refile-modal'))&&d.open){var
    inp=d.querySelector('.ol-find-input');if(inp)setTimeout(function(){inp.focus();},0);}}"
4064        ",true);"
4065        "document.addEventListener('pointerdown',function(e){var
    rf=e.target.closest&&e.target.closest('.ol-find-refile-form');"
4066        "if(!rf)return;var src=document.getElementById('ol-refile-nid');var
    m=rf.querySelector('.ol-refile-nid-mirror');"
4067        "if(src&&m)m.value=src.value;},true);"
4068        "})();"))))
4069
4070  (define notes/ui/folder-block
4071    (lambda
4072      (folder notes)
4073      (let
4074        ((items (notes/light/notes-in folder notes))
4075          (label
4076            (if (string-eq folder "") "(uncategorized)" folder)))
4077        (if
4078          (null? items)
4079          ""
4080          (html/tag
4081            "details"
4082            (html/attrs
4083              (list
4084                (html/attr
4085                  "class"
4086                  (if
4087                    (string-eq folder "")
4088                    "tree-folder tree-uncategorized"
4089                    "tree-folder"))
4090                (html/attr "open" "open")))
4091            (html/join
4092              (list
4093                (html/tag
4094                  "summary"
4095                  (html/attr "class" "tree-folder-header")
4096                  (html/join
4097                    (list
4098                      (html/span
```

```
                      (html/attr "class" "tree-folder-icon")
                      (if (string-eq folder "") "&#x1F4C2; " "&#x1F4C1; "))
                    (html/a
                      (string-append "/notes?folder=" folder)
                      (html/text label))
                    (html/span
                      (html/attr "class" "muted")
                      (html/text
                        (string-append " (" (as-str (length items))
    ")"))))))
              (notes/light/render-notes items))))))))

(define notes/ui/folder-block-grouped
  (lambda
    (pair)
    (let
      ((folder (as-str (car pair))) (items (car (cdr pair))))
      (let
        ((label
            (if (string-eq folder "") "(uncategorized)" folder))
          (n (length items)))
        (html/tag
          "details"
          (html/attrs
            (list
              (html/attr
                "class"
                (if
                  (string-eq folder "")
                  "tree-folder tree-uncategorized"
                  "tree-folder"))
              (html/attr "open" "open")))
          (html/join
            (list
              (html/tag
                "summary"
                (html/attr "class" "tree-folder-header")
                (html/join
                  (list
                    (html/span
                      (html/attr "class" "tree-folder-icon")
                      (if (string-eq folder "") "&#x1F4C2; " "&#x1F4C1; "))
                    (html/text label)
                    (html/span
                      (html/attr "class" "muted")
                      (html/text (string-append " (" (as-str n) ")"))))))
              (notes/light/render-notes items))))))))

(define notes/ui/folder-details
  (lambda
    (n)
    (let
      ((path (notes/ui/node-path n))
        (label (notes/ui/node-title n)))
      (html/tag
        "details"
        (html/attrs
          (list
            (html/attr "class" "tree-folder")
            (html/attr
              "hx-get"
              (string-append "/partials/notes-tree?path=" path))
            (html/attr "hx-trigger" "toggle once")
            (html/attr "hx-target" "find .tree-children")
            (html/attr "hx-swap" "innerHTML")))
        (html/join
          (list
            (html/tag
              "summary"
              (html/attr "class" "tree-folder-header")
              (html/join
                (list
                  (html/span
```

```
                    (html/attr "class" "tree-folder-icon")
                    "&#x1F4C1; ")
                  (html/text label))))
            (html/div
              (html/attr "class" "tree-children")
              (html/span
                (html/attr "class" "muted")
                "open to load?")))))))

(define notes/ui/folder-form
  (lambda
    (slug folder)
    (html/form
      "POST"
      "/notes"
      (html/attr "class" "notes-folder-form")
      (html/join
        (list
          (html/hidden "action" "set-folder")
          (html/hidden "slug" (as-str slug))
          (html/label "" "folder ")
          (html/text-input
            "folder"
            (html/attrs
              (list
                (html/attr "value" (as-str folder))
                (html/attr "placeholder" "wiki/entities")
                (html/attr "style" "width:12rem"))))
          " "
          (html/button (html/attr "type" "submit") "move"))))))

(define notes/ui/folder-group-keys
  (lambda
    (notes)
    (notes/light/dedupe
      (map (lambda (n) (as-str (notes/light/nth n 1))) notes))))

(define notes/ui/folder-list
  (lambda
    (folder)
    (list/foldl
      (lambda
        (acc row)
        (string-append acc (notes/ui/item-li row)))
      ""
      (notes/doc/list-folder folder))))

(define notes/ui/folder-section
  (lambda
    (folder)
    (let
      ((label
          (if
            (string-eq (as-str folder) "")
            "(uncategorized)"
            (as-str folder)))
        (href
          (if
            (string-eq (as-str folder) "")
            "/notes?folder="
            (string-append "/notes?folder=" (as-str folder)))))
      (html/div
        (html/attr "class" "notes-folder card")
        (html/join
          (list
            (html/h3 (html/a href (html/text label)))
            (html/ul "" (notes/ui/folder-list folder))))))))

(define notes/ui/folders-html
  (lambda
    ()
    (let
      ((root (notes/tree/root)))
```

```
      (html/div
        (html/attr "class" "notes-tree")
        (notes/ui/tree-children-html root root)))))

(define notes/ui/group-add
  (lambda
    (acc k slug title)
    (if
      (null? acc)
      (list (list k (list (list slug title))))
      (if
        (string-eq (as-str (car (car acc))) k)
        (cons
          (list k (cons (list slug title) (car (cdr (car acc)))))
          (cdr acc))
        (cons
          (car acc)
          (notes/ui/group-add (cdr acc) k slug title))))))

(define notes/ui/group-notes
  (lambda
    ()
    (list/foldl
      (lambda
        (acc r)
        (let
          ((d (cdr r)))
          (notes/ui/group-add
            acc
            (notes/ui/top-folder (dict-get d "folder"))
            (as-str (dict-get d "slug"))
            (as-str (dict-get d "title")))))
      (list)
      (notes/doc/index-list))))

(define notes/ui/handle-post
  (lambda
    ()
    (let
      ((action (as-str (request-param "action"))))
      (cond
        ((string-eq action "create")
          (notes/ui/do-create
            (request-param "title")
            (request-param "slug")
            (request-param "folder")))
        ((string-eq action "set-folder")
          (seq
            (notes/doc/set-folder
              (request-param "slug")
              (if
                (null? (request-param "folder"))
                ""
                (request-param "folder")))
            (redirect
              (string-append
                "/notes?slug="
                (as-str (request-param "slug"))))))
        ((string-eq action "save")
          (notes/ui/do-save
            (request-param "slug")
            (request-param "outline")))
        ((string-eq action "undo") (notes/ui/do-undo))
        ((string-eq action "capture") (notes/capture/handle))
        ((string-eq action "delete") (notes/ui/do-delete))
        ((string-eq action "refile") (notes/refile/handle))
        ((string-eq action "extract")
          (notes/refile/extract-handle))
        (else (redirect "/notes"))))))

(define notes/ui/index
  (lambda
    ()
```

```
    (html/join
      (list
        (notes/tree/style)
        (html/div
          (html/attr "class" "ol-find-bar")
          (html/join
            (list
              (notes/ui/find-modal)
              (html/span (html/attr "class" "muted") "Ctrl/Cmd-K"))))
        (html/h1 "Notes")
        (html/p*
          (html/attr "class" "muted")
          "Outline-first notes. Folders load on open (htmx).")
        (html/p*
          (html/attr "class" "muted")
          (html/join
            (list
              (html/a "/notes?slug=inbox" "Open inbox")
              " ? "
              (html/a "/issues" "Issues")
              " ? "
              (html/a "/notes" "All folders"))))
        (notes/ui/create-form)
        (html/h2 "Folders")
        (notes/ui/folders-html)
        (notes/ui/find-script)))))

(define notes/ui/index-group
  (lambda
    (notes)
    (list/foldl
      (lambda
        (acc n)
        (let
          ((k (as-str (notes/light/nth n 1))))
          (notes/ui/index-group-add acc k n)))
      (list)
      notes)))

(define notes/ui/index-group-add
  (lambda
    (acc k n)
    (if
      (null? acc)
      (list (list k (list n)))
      (if
        (string-eq (as-str (car (car acc))) k)
        (cons (list k (cons n (car (cdr (car acc))))) (cdr acc))
        (cons
          (car acc)
          (notes/ui/index-group-add (cdr acc) k n))))))

(define notes/ui/is-garden-folder?
  (lambda
    (n root)
    (if
      (not (string-eq (notes/ui/node-kind n) "folder"))
      #f
      (string-eq
        (notes/ui/node-path n)
        (string-append (as-str root) ".garden")))))

(define notes/ui/item-li
  (lambda
    (row)
    (let
      ((f (cdr row)) (id (car row)))
      (let
        ((slug (as-str (dict-get f "slug")))
          (title (as-str (dict-get f "title")))
          (folder (notes/doc/row-folder row)))
        (html/li
          (html/attr "class" "note-item")
```

```
          (html/join
            (list
              (html/a
                (string-append "/notes?slug=" slug)
                (html/text title))
              (if
                (string-eq folder "")
                ""
                (html/span
                  (html/attr "class" "muted")
                  (html/text (string-append " ? " folder))))
              (html/form
                "POST"
                "/notes"
                (html/attrs
                  (list
                    (html/attr "class" "note-del")
                    (html/attr "hx-confirm" "Delete this note?")))
                (html/join
                  (list
                    (html/hidden "action" "delete")
                    (html/hidden "slug" slug)
                    (html/button
                      (html/attrs
                        (list
                          (html/attr "type" "submit")
                          (html/attr "class" "secondary")
                          (html/attr "title" "delete note")))
                      "del")))))))))

(define notes/ui/item-li-lite
  (lambda
    (row)
    (let
      ((f (cdr row)))
      (let
        ((slug (as-str (dict-get f "slug")))
          (title (as-str (dict-get f "title")))
          (folder (notes/doc/row-folder row)))
        (html/li
          (html/attr "class" "note-item")
          (html/join
            (list
              (html/a
                (string-append "/notes?slug=" slug)
                (html/text title))
              (if
                (string-eq folder "")
                ""
                (html/span
                  (html/attr "class" "muted")
                  (html/text (string-append " ? " folder))))))))))

(define notes/ui/missing
  (lambda
    (slug)
    (html/join
      (list
        (html/h1 "Missing")
        (html/p*
          (html/attr "class" "err")
          (string-append "No note " (as-str slug)))
        (html/p* "" (html/a "/notes" "All notes"))))))

(define notes/ui/node-kind
  (lambda (n) (as-str (dict-get n "kind"))))

(define notes/ui/node-path
  (lambda (n) (as-str (dict-get n "path"))))

(define notes/ui/node-title
  (lambda
    (n)
```

```
    (let
      ((b (dict-get n "body")))
      (let
        ((t (dict-get b "title")) (s (dict-get b "slug")))
        (if
          (if (null? t) #t (string-eq (as-str t) ""))
          (if
            (if (null? s) #t (string-eq (as-str s) ""))
            (notes/ui/path-label (notes/ui/node-path n))
            (as-str s))
          (as-str t))))))

(define notes/ui/note-li-from-node
  (lambda
    (n)
    (let
      ((slug (as-str (dict-get (dict-get n "body") "slug")))
        (title (notes/ui/node-title n)))
      (let
        ((s
          (if
            (string-eq slug "")
            (notes/ui/path-label (notes/ui/node-path n))
            slug)))
        (html/li
          (html/attr "class" "tree-item")
          (html/a
            (string-append "/notes?slug=" s)
            (html/text title)))))))

(define notes/ui/outline-script
  (lambda
    (slug show-archived tree)
    (html/tag
      "script"
      ""
      (string-append
        "(function(){var slug='"
        (as-str slug)
        "';var showArch="
        (if show-archived "true" "false")
        ";var ppTree="
        (json-emit tree)
        ";window.__ppTree=ppTree;"
        "var __ppReady=false,__ppBooting=false,__opSeq=0;"
        "var
    st=document.createElement('style');st.textContent='.pp-in-transit{opacity:.5;filter:grayscale(1);}';document.head.apper
        "function
    ppBoot(){if(__ppReady||__ppBooting)return;__ppBooting=true;"
        "var s=document.createElement('script');s.src='/assets/interp.js';"
        "s.onload=function(){var pp=window.pp;pp.boot();"
        "var pkgs=['html','structed','structed/list','outline','bible'];var
    done=0;"

    "pkgs.forEach(function(p){fetch('/code/raw/libs/'+p+'/lib.pp',{credentials:'same-origin'}).then(function(r){return
    r.text();}).then(function(src){pp.load_source(src);done++;if(done===pkgs.length){fetch('/code/raw/client/ops.pp',{crede
    r.text();}).then(function(os){pp.load_source(os);pp.set_tree(JSON.stringify(ppTree));__ppReady=true;__ppBooting=false;}
        "document.head.appendChild(s);}"
        "ppBoot();"
        "function selId(){var
    el=document.getElementById('ol-selected-id');return
    el?el.value:'root';}"
        "function findNode(id){var
    all=document.querySelectorAll('.ol-node');for(var
    i=0;i<all.length;i++){if(all[i].getAttribute('data-id')===id)return
    all[i];}return null;}"
        "function ownTitle(node){if(!node||!node.children)return
    null;for(var i=0;i<node.children.length;i++){var
    ch=node.children[i];if(ch.classList&&ch.classList.contains('ol-sum')){return
    ch.querySelector('.ol-title');}}return null;}"
        "var __selPin=null,__selPinT=null;"
        "function setSel(id){"
        "if(!id)return;"
```

```
4522        "if(__selPin&&id===’root’&&__selPin!==’root’)return;"
4523        "var node=findNode(id);if(!node)return;"
4524        "var
        hid=document.getElementById(’ol-selected-id’);if(hid)hid.value=id;"
4525        "var lab=document.getElementById(’ol-selected-label’);"
4526        "if(lab){var
        t=ownTitle(node);if(!t)t=node.querySelector(’.ol-title’);"
4527
        "lab.textContent=(t&&t.innerText)?t.innerText.trim():id;lab.title=id;}"
4528        "var rf=document.getElementById(’ol-refile-nid’);if(rf)rf.value=id;"
4529        "var
        ex=document.getElementById(’ol-extract-nid’);if(ex)ex.value=id;"
4530
        "document.querySelectorAll(’.ol-node.is-selected’).forEach(function(n){n.classList.remove(’is-selected’);});"
4531        "node.classList.add(’is-selected’);"
4532
        "if(id!==’root’){__selPin=id;if(__selPinT)clearTimeout(__selPinT);__selPinT=setTimeout(function(){__selPin=null;},400);
4533        "else{__selPin=’root’;}"
4534        "}"
4535        "function postOp(op){var id=selId();"
4536        "if(op===’delete’){if(id===’root’){alert(’Cannot delete the root
        node.’);return;}if(!confirm(’Delete this node?’))return;}"
4537        "if(op===’archive’&&id===’root’){alert(’Cannot archive the root
        node.’);return;}"
4538        "if(!__ppReady)ppBoot();"
4539        "var seq=++__opSeq;var keep=id;__selPin=keep;"
4540        "var
        snap=document.getElementById(’outline’)?document.getElementById(’outline’).outerHTML:null;"
4541        "if(__ppReady){try{var prev=window.pp.apply_op(op,id,slug);"
4542        "if(prev&&prev.length){var
        t=document.getElementById(’outline’);if(t){var
        tmp=document.createElement(’div’);tmp.innerHTML=prev;var
        neo=tmp.firstChild;if(neo){neo.classList.add(’pp-in-transit’);t.replaceWith(neo);}}}"
4543        "}catch(e){console.warn(’pp optimistic failed’,e);}}"
4544        "var
        q=’slug=’+encodeURIComponent(slug)+’&op=’+encodeURIComponent(op)+’&node-id=’+encodeURIComponent(id)+’&op-seq=’+seq;"
4545        "if(showArch)q+=’&show=archived’;"
4546
        "fetch(’/partials/notes-node-op?’+q,{method:’POST’,credentials:’same-origin’}).then(function(r){return
        r.text();}).then(function(html){"
4547        "if(seq<__opSeq)return;"
4548        "var doc=new DOMParser().parseFromString(html,’text/html’);"
4549        "var oobSel=doc.querySelector(’#ol-selected-id[hx-swap-oob]’);"
4550        "var olHTML=doc.querySelector(’#outline’);"
4551        "var cur=document.getElementById(’outline’);"
4552
        "if(olHTML&&cur){if(window.Idiomorph){Idiomorph.morph(cur,olHTML);}else{cur.outerHTML=olHTML.outerHTML;}}"
4553        "var selIdVal=oobSel?oobSel.getAttribute(’value’):null;"
4554        "if(selIdVal){var
        hid=document.getElementById(’ol-selected-id’);if(hid)hid.value=selIdVal;}"
4555        "var target=selIdVal||keep;"
4556        "if(findNode(target))setSel(target);else
        if(findNode(keep))setSel(keep);else{__selPin=null;setSel(’root’);}"
4557
        "if(window.htmx){try{htmx.process(document.getElementById(’outline’));}catch(e){}}"
4558        "}).catch(function(e){"
4559        "if(snap){var
        tmp=document.createElement(’div’);tmp.innerHTML=snap;var
        c=document.getElementById(’outline’);if(c)c.replaceWith(tmp.firstChild);}"
4560        "console.error(’notes postOp’,e);});}"
4561        "function pickNode(e){"
4562        "if(e.target.closest&&e.target.closest(’[data-ol-op]’))return null;"
4563        "if(e.target.closest&&e.target.closest(’.ol-actions’))return null;"
4564        "if(e.target.closest&&e.target.closest(’.ol-fold’))return null;"
4565        "var node=e.target.closest&&e.target.closest(’.ol-node’);"
4566        "return(node&&node.dataset&&node.dataset.id)?node:null;}"
4567        "document.addEventListener(’pointerdown’,function(e){"
4568        "if(e.button!==0&&e.button!==undefined)return;"
4569        "var node=pickNode(e);if(node)setSel(node.dataset.id);},true);"
4570        "document.addEventListener(’click’,function(e){"
4571        "var btn=e.target.closest&&e.target.closest(’[data-ol-op]’);"
4572
        "if(btn){e.preventDefault();postOp(btn.getAttribute(’data-ol-op’));return;}"
```

```
        "var node=pickNode(e);if(node)setSel(node.dataset.id);},true);"
        "document.addEventListener('focusin',function(e){"
        "var t=e.target;if(!t||!t.getAttribute)return;"
        "var id=t.getAttribute('data-id');var
    field=t.getAttribute('data-field');"
        "if(id&&(field==='title'||field==='body'))setSel(id);},true);"
        "var
    root=document.getElementById('outline');if(root){root.addEventListener('blur',function(e){"
        "var t=e.target;if(!t.dataset||!t.dataset.field)return;"
        "var
    p='slug='+encodeURIComponent(slug)+'&op=set-'+t.dataset.field+'&node-id='+encodeURIComponent(t.dataset.id);"
        "if(showArch)p+='&show=archived';"
        "p+='&'+t.dataset.field+'='+encodeURIComponent(t.innerText);"

    "fetch('/partials/notes-node-op',{method:'POST',headers:{'Content-Type':'application/x-www-form-urlencoded'},body:p});}
        "document.addEventListener('keydown',function(e){"
        "if(!e.altKey)return;"
        "var k=e.key;var op=null;"
        "if(e.shiftKey&&k==='ArrowLeft')op='promote';"
        "else if(e.shiftKey&&k==='ArrowRight')op='demote';"
        "else if(k==='ArrowUp')op='move-up';"
        "else if(k==='ArrowDown')op='move-down';"
        "else if(k==='Enter')op='add-sibling';"
        "else if(k==='ArrowRight')op='slurp-forward';"
        "else if(k==='ArrowLeft')op='barf-forward';"
        "if(op){e.preventDefault();postOp(op);}"
        "},true);"

    "setSel('root');document.body.addEventListener('htmx:afterSwap',function(){setSel(selId());});})();"))))

(define notes/ui/path-label
  (lambda
    (path)
    (let
      ((parts (string-split (as-str path) ".")))
      (if
        (null? parts)
        ""
        (string-replace (notes/light/last parts) "_" "-")))))

(define notes/ui/recent-list
  (lambda
    ()
    (list/foldl
      (lambda
        (acc row)
        (string-append acc (notes/ui/item-li row)))
      ""
      (notes/doc/index-list))))

(define notes/ui/refile-modal
  (lambda
    (slug)
    (html/tag
      "details"
      (html/attr "class" "ol-refile-modal")
      (html/join
        (list
          (html/tag
            "summary"
            (html/attrs
              (list
                (html/attr "class" "ol-btn")
                (html/attr
                  "title"
                  "refile selected node to another note")))
            "refile")
          (notes/ui/refile-modal-body slug))))))

(define notes/ui/refile-modal-body
  (lambda
    (slug)
    (html/tag
```

```
      "div"
      (html/attrs
        (list (html/attr "class" "ol-refile-body ol-find-wide")))
      (html/join
        (list
          (html/void
            "input"
            (html/attrs
              (list
                (html/attr "type" "hidden")
                (html/attr "id" "ol-refile-nid")
                (html/attr "value" "root"))))
          (html/void
            "input"
            (html/attrs
              (list
                (html/attr "type" "search")
                (html/attr "class" "ol-find-input")
                (html/attr "name" "q")
                (html/attr
                  "placeholder"
                  "destination note or header?")
                (html/attr "autocomplete" "off")
                (html/attr
                  "hx-get"
                  (string-append
                    "/partials/notes-search?mode=refile&src="
                    (as-str slug)))
                (html/attr
                  "hx-trigger"
                  "input changed delay:200ms, search")
                (html/attr "hx-target" "next .ol-find-results")
                (html/attr "hx-swap" "outerHTML")
                (html/attr "hx-include" "this"))))
          (html/div
            (html/attr "class" "ol-find-results")
            (html/div
              (html/attr "class" "ol-find-empty")
              "Type a destination note or header?"))
          (html/form
            "POST"
            "/notes"
            (html/attr "class" "ol-refile")
            (html/join
              (list
                (html/hidden "action" "refile")
                (html/hidden "src_slug" (as-str slug))
                (html/void
                  "input"
                  (html/attrs
                    (list
                      (html/attr "type" "hidden")
                      (html/attr "name" "node_id")
                      (html/attr "class" "ol-refile-nid-mirror")
                      (html/attr "value" "root"))))
                (html/label
                  (html/attr "class" "ol-refile-label")
                  "or slug ")
                (html/text-input
                  "dst_slug"
                  (html/attrs
                    (list
                      (html/attr "class" "ol-refile-sel")
                      (html/attr "placeholder" "destination-slug")
                      (html/attr "autocomplete" "off"))))
                " "
                (html/button
                  (html/attrs
                    (list
                      (html/attr "type" "submit")
                      (html/attr "class" "ol-btn")
                      (html/attr "title" "refile selected node")))
                  "go")))))))))
```

```
4714
4715 (define notes/ui/refile-partial
4716   (lambda
4717     ()
4718     (let
4719       ((slug (as-str (request-param "src"))))
4720       (if
4721         (string-eq slug "")
4722         (render-html
4723           (html/p* (html/attr "class" "muted") "missing src"))
4724         (render-html
4725           (html/tag
4726             "details"
4727             (html/attrs
4728               (list
4729                 (html/attr "class" "ol-refile-modal")
4730                 (html/attr "open" "")))
4731             (html/join
4732               (list
4733                 (html/tag
4734                   "summary"
4735                   (html/attrs
4736                     (list
4737                       (html/attr "class" "ol-btn")
4738                       (html/attr
4739                         "title"
4740                         "refile selected node to another note")))
4741                   "refile")
4742                 (notes/ui/refile-modal-body slug)))))))))
4743
4744 (define notes/ui/render-child
4745   (lambda
4746     (n root)
4747     (let
4748       ((k (notes/ui/node-kind n)))
4749       (if
4750         (string-eq k "heading")
4751         ""
4752         (if
4753           (notes/ui/is-garden-folder? n root)
4754           (notes/ui/tree-children-html
4755             (notes/ui/node-path n)
4756             root)
4757           (if
4758             (string-eq k "folder")
4759             (notes/ui/folder-details n)
4760             (if
4761               (string-eq k "note")
4762               (notes/ui/note-li-from-node n)
4763               ""))))))))
4764
4765 (define notes/ui/search-group
4766   (lambda
4767     (hits)
4768     (list/map
4769       (lambda (s) (notes/ui/search-hits-for hits s))
4770       (notes/ui/search-slugs hits))))
4771
4772 (define notes/ui/search-group-html
4773   (lambda
4774     (group mode src-slug)
4775     (if
4776       (null? group)
4777       ""
4778       (let
4779         ((title (as-str (dict-get (car group) "title")))
4780           (slug (as-str (dict-get (car group) "slug"))))
4781         (html/join
4782           (list
4783             (html/div
4784               (html/attr "class" "ol-find-group")
4785               (html/text (if (string-eq title "") slug title)))
4786             (html/join
```

```
              (list/map
                (lambda
                  (hit)
                  (if
                    (string-eq mode "refile")
                    (notes/ui/search-hit-refile hit src-slug)
                    (notes/ui/search-hit-open hit)))
                group))))))))

(define notes/ui/search-hit-href
  (lambda
    (hit)
    (let
      ((slug (as-str (dict-get hit "slug")))
        (nid (as-str (dict-get hit "node_id"))))
      (if
        (string-eq nid "")
        (string-append "/notes?slug=" slug)
        (string-append "/notes?slug=" slug "&node=" nid)))))

(define notes/ui/search-hit-label
  (lambda
    (hit)
    (let
      ((path (as-str (dict-get hit "header_path")))
        (matched (as-str (dict-get hit "matched")))
        (title (as-str (dict-get hit "title")))
        (nid (as-str (dict-get hit "node_id"))))
      (if
        (string-eq nid "")
        (html/join
          (list
            (html/span
              (html/attr "class" "ol-find-hit-path")
              (html/text
                (if
                  (string-eq title "")
                  (as-str (dict-get hit "slug"))
                  title)))
            (html/span
              (html/attr "class" "ol-find-hit-meta")
              (html/text matched))))
        (html/join
          (list
            (html/span
              (html/attr "class" "ol-find-hit-path")
              (html/text (if (string-eq path "") title path)))
            (html/span
              (html/attr "class" "ol-find-hit-meta")
              (html/text matched)))))))))

(define notes/ui/search-hit-open
  (lambda
    (hit)
    (html/tag
      "a"
      (html/attrs
        (list
          (html/attr "href" (notes/ui/search-hit-href hit))
          (html/attr "class" "ol-find-hit")
          (html/attr "data-ol-find-hit" "1")
          (html/attr "data-slug" (as-str (dict-get hit "slug")))
          (html/attr
            "data-node"
            (as-str (dict-get hit "node_id")))))
      (notes/ui/search-hit-label hit))))

(define notes/ui/search-hit-refile
  (lambda
    (hit src-slug)
    (html/form
      "POST"
      "/notes"
```

```
      (html/attr "class" "ol-find-refile-form")
      (html/join
        (list
          (html/hidden "action" "refile")
          (html/hidden "src_slug" (as-str src-slug))
          (html/void
            "input"
            (html/attrs
              (list
                (html/attr "type" "hidden")
                (html/attr "name" "node_id")
                (html/attr "class" "ol-refile-nid-mirror")
                (html/attr "value" "root"))))
          (html/hidden "dst_slug" (as-str (dict-get hit "slug")))
          (html/hidden
            "dst_node_id"
            (as-str (dict-get hit "node_id")))
          (html/button
            (html/attrs
              (list
                (html/attr "type" "submit")
                (html/attr "class" "ol-find-hit")
                (html/attr "data-ol-find-hit" "1")
                (html/attr "data-slug" (as-str (dict-get hit "slug")))
                (html/attr
                  "data-node"
                  (as-str (dict-get hit "node_id")))
                (html/attr "title" "refile selected node here")))
            (notes/ui/search-hit-label hit)))))))

(define notes/ui/search-hits-for
  (lambda
    (hits slug)
    (list/filter
      (lambda
        (h)
        (string-eq (as-str (dict-get h "slug")) (as-str slug)))
      hits)))

(define notes/ui/search-partial
  (lambda
    ()
    (let
      ((q
          (if
            (null? (request-param "q"))
            ""
            (as-str (request-param "q"))))
        (mode
          (if
            (null? (request-param "mode"))
            "open"
            (as-str (request-param "mode"))))
        (src
          (if
            (null? (request-param "src"))
            ""
            (as-str (request-param "src")))))
      (render-html (notes/ui/search-results-html q mode src)))))

(define notes/ui/search-results-html
  (lambda
    (q mode src-slug)
    (let
      ((needle (string-trim (as-str q))))
      (if
        (string-eq needle "")
        (html/div
          (html/attr "class" "ol-find-results")
          (html/div
            (html/attr "class" "ol-find-empty")
            (if
              (string-eq mode "refile")
```

```
              "Type a destination note or header?"
              "Type to find a note or header?")))
        (let
          ((hits (notes/doc/search needle)))
          (if
            (null? hits)
            (html/div
              (html/attr "class" "ol-find-results")
              (html/div
                (html/attr "class" "ol-find-empty")
                "No matches."))
            (html/div
              (html/attr "class" "ol-find-results")
              (html/join
                (list/map
                  (lambda
                    (g)
                    (notes/ui/search-group-html g mode src-slug))
                  (notes/ui/search-group hits)))))))))

(define notes/ui/search-slug-seen?
  (lambda
    (xs s)
    (if
      (null? xs)
      #f
      (if
        (string-eq (as-str (car xs)) s)
        #t
        (notes/ui/search-slug-seen? (cdr xs) s)))))

(define notes/ui/search-slugs
  (lambda
    (hits)
    (list/foldl
      (lambda
        (acc h)
        (let
          ((s (as-str (dict-get h "slug"))))
          (if
            (notes/ui/search-slug-seen? acc s)
            acc
            (append acc (list s)))))
      (list)
      hits)))

(define notes/ui/show
  (lambda
    (slug)
    (let
      ((n (notes/doc/get slug))
        (show-archived (notes/doc/show-archived?)))
      (if
        (null? n)
        (notes/ui/missing slug)
        (notes/ui/show-body
          slug
          (notes/doc/title n)
          (notes/doc/outline n)
          (notes/doc/tags n)
          show-archived
          (notes/doc/folder n))))))

(define notes/ui/show-body
  (lambda
    (slug title outline tags show-archived folder)
    (html/join
      (list
        (notes/tree/style)
        (html/p*
          (html/attr "class" "muted")
          (html/join
            (list
```

```
              (html/a "/notes" "Notes")
              (if
                (string-eq (as-str folder) "")
                ""
                (html/join
                  (list
                    " / "
                    (html/a
                      (string-append "/notes?folder=" (as-str folder))
                      (html/text (as-str folder))))))
              " / "
              (html/code (as-str slug)))))
        (html/div
          (html/attr "class" "note-title-row")
          (html/join
            (list
              (html/h1 (html/text title))
              (html/form
                "POST"
                "/notes"
                (html/attrs
                  (list
                    (html/attr "class" "note-del note-del-page")
                    (html/attr "hx-confirm" "Delete this entire note?")))
                (html/join
                  (list
                    (html/hidden "action" "delete")
                    (html/hidden "slug" (as-str slug))
                    (html/button
                      (html/attrs
                        (list
                          (html/attr "type" "submit")
                          (html/attr "class" "secondary")
                          (html/attr "title" "delete note")))
                      "Delete note"))))
              (html/form
                "POST"
                "/notes"
                (html/attrs
                  (list
                    (html/attr "class" "note-undo")
                    (html/attr "title" "undo the last change to this
    note")))
                (html/join
                  (list
                    (html/hidden "action" "undo")
                    (html/hidden "slug" (as-str slug))
                    (html/button
                      (html/attrs
                        (list
                          (html/attr "type" "submit")
                          (html/attr "class" "secondary")
                          (html/attr "title" "undo last note change")))
                      "Undo")))))))
          (html/p*
            (html/attr "class" "muted")
            (string-append "tags: " (as-str tags)))
          (notes/ui/folder-form slug folder)
          (notes/ui/toolbar slug show-archived)
          (outline/html-lite slug outline show-archived)
          (html/div
            (html/attr "class" "roam-backlinks-panel")
            (html/join
              (list (html/h3 "Backlinks") (roam/backlinks-html slug))))
          (html/tag
            "details"
            (html/attr "class" "ol-debug")
            (html/join
              (list
                (html/tag
                  "summary"
                  (html/attr "class" "muted")
                  "Raw JSON (debug)")
```

```
              (notes/ui/edit-form slug outline))))
        (notes/ui/outline-script slug show-archived outline)
        (notes/ui/find-script)))))

(define notes/ui/tb-btn
  (lambda
    (op label)
    (html/tag
      "button"
      (html/attrs
        (list
          (html/attr "type" "submit")
          (html/attr "class" "ol-btn")
          (html/attr "title" op)
          (html/attr "data-op" op)
          (html/attr "form" "ol-op-form")))
      label)))

(define notes/ui/toolbar
  (lambda
    (slug show-archived)
    (html/div
      (html/attr "class" "ol-toolbar")
      (html/join
        (list
          (html/span
            (html/attr "class" "ol-toolbar-label")
            "Selected:")
          (html/span (html/attr "id" "ol-selected-label") "root")
          (html/void
            "input"
            (html/attrs
              (list
                (html/attr "type" "hidden")
                (html/attr "id" "ol-selected-id")
                (html/attr "name" "node-id")
                (html/attr "value" "root"))))
          (html/tag
            "template"
            (html/attr "id" "ol-op-opt")
            (html/span
              (html/attrs
                (list
                  (html/attr "class" "ol-opt-msg")
                  (html/attr "style" "opacity:0.6;font-style:italic")))
              (html/text "updating?")))
          (html/form
            "POST"
            "/partials/notes-node-op"
            (html/attrs
              (list
                (html/attr "id" "ol-op-form")
                (html/attr "class" "ol-op-form")
                (html/attr "hx-post" "/partials/notes-node-op")
                (html/attr "hx-target" "#outline")
                (html/attr "hx-swap" "outerHTML")
                (html/attr "hx-include" "#ol-selected-id")
                (html/attr "hx-optimistic" "#ol-op-opt")))
            (html/join
              (list
                (html/hidden "slug" (as-str slug))
                (if
                  show-archived
                  (html/void
                    "input"
                    (html/attrs
                      (list
                        (html/attr "type" "hidden")
                        (html/attr "name" "show")
                        (html/attr "value" "archived"))))
                  "")
                (html/void
                  "input"
```

```
                  (html/attrs
                    (list
                      (html/attr "type" "hidden")
                      (html/attr "name" "op")
                      (html/attr "id" "ol-op"))))
                (notes/ui/tb-btn "cycle-todo" "todo")
                (notes/ui/tb-btn "archive" "arch")
                (notes/ui/tb-btn "unarchive" "unarch")
                (notes/ui/tb-btn "add-child" "+kid")
                (notes/ui/tb-btn "add-sibling" "+sib")
                (notes/ui/tb-btn "promote" "<<")
                (notes/ui/tb-btn "demote" ">>")
                (notes/ui/tb-btn "move-up" "up")
                (notes/ui/tb-btn "move-down" "dn")
                (html/tag
                  "button"
                  (html/attrs
                    (list
                      (html/attr "type" "submit")
                      (html/attr "class" "ol-btn ol-del")
                      (html/attr "title" "delete")
                      (html/attr "data-op" "delete")
                      (html/attr "form" "ol-op-form")
                      (html/attr "hx-confirm" "Delete this node?")))
                  "del"))))
          (notes/ui/find-modal)
          (notes/ui/refile-modal slug)
          (html/form
            "POST"
            "/notes"
            (html/attr "class" "ol-extract")
            (html/join
              (list
                (html/hidden "action" "extract")
                (html/hidden "src_slug" (as-str slug))
                (html/void
                  "input"
                  (html/attrs
                    (list
                      (html/attr "type" "hidden")
                      (html/attr "name" "node_id")
                      (html/attr "id" "ol-extract-nid")
                      (html/attr "value" "root"))))
                (html/button
                  (html/attrs
                    (list
                      (html/attr "type" "submit")
                      (html/attr "class" "ol-btn")
                      (html/attr
                        "title"
                        "extract selected node into its own note")))
                  "extract"))))
          (if
            show-archived
            (html/a
              (string-append "/notes?slug=" slug)
              "Hide archived")
            (html/a
              (string-append "/notes?slug=" slug "&show=archived")
              "Show archived")))))))

(define notes/ui/top-folder
  (lambda
    (folder)
    (let
      ((f (as-str folder)))
      (let
        ((s
          (if
            (string-starts-with? f "garden/")
            (string-replace f "garden/" "")
            (if (string-eq f "garden") "" f))))
        (let
```

```
          ((k (string-replace s "_" "-")))
          (if (string-eq k "") "" (car (string-split k "/"))))))))

(define notes/ui/tree-children-html
  (lambda (path root) (tree/children-html (as-str path))))

(define notes/ui/tree-partial
  (lambda
    ()
    (let
      ((path (as-str (request-param "path")))
        (root (notes/tree/root)))
      (if
        (string-eq path "")
        (render-html
          (html/p* (html/attr "class" "muted") "missing path"))
        (render-html
          (html/div
            (html/attr "class" "tree-children-inner")
            (notes/ui/tree-children-html path root)))))))

(define notes/upsert
  (lambda
    (slug title outline tags kind day)
    (notes/doc/upsert
      slug
      title
      outline
      tags
      kind
      day))
  "Upsert a note by slug. Delegates to notes/doc/upsert (tree path); body
     defaults to empty, folder preserved on update. Returns slug. Fixes
     i-1786914877.")
```

182 libs/notes/light/lib.pp

```
;; package notes/light -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push

(define notes/light/add-prefixes
  (lambda
    (parts prefix acc)
    (if
      (null? parts)
      acc
      (let*
        ((seg (as-str (car parts)))
          (full
            (if
              (string-eq prefix "")
              seg
              (string-append prefix "/" seg))))
        (notes/light/add-prefixes
          (cdr parts)
          full
          (notes/light/cons-unique full acc))))))

(define notes/light/all-folder-paths
  (lambda
    (all-notes)
    (notes/light/dedupe
      (notes/tree/sort-by-name
        (list/foldl
          (lambda
            (acc note)
            (let
              ((f (as-str (notes/light/nth note 1))))
              (if
                (string-eq f "")
                acc
                (notes/light/add-prefixes (string-split f "/") "" acc))))
          (list)
          all-notes)))))

(define notes/light/children-of
  (lambda
    (parent paths)
    (list/filter
      (lambda
        (p)
        (string-eq (notes/light/parent-of p) parent))
      paths)))

(define notes/light/cons-unique
  (lambda
    (x xs)
    (if (notes/light/contains? x xs) xs (cons x xs))))

(define notes/light/contains?
  (lambda
    (x xs)
    (if
      (null? xs)
      #f
      (if
        (string-eq x (as-str (car xs)))
        #t
        (notes/light/contains? x (cdr xs))))))

(define notes/light/dedupe
  (lambda
    (xs)
    (if
      (null? xs)
      (list)
      (notes/light/cons-unique
        (car xs)
```

```
        (notes/light/dedupe (cdr xs))))))

(define notes/light/folder-section
  (lambda
    (folder)
    (let*
      ((all-notes (notes/list-light))
        (items (notes/light/notes-in folder all-notes))
        (label
          (if (string-eq folder "") "(uncategorized)" folder))
        (href (string-append "/notes?folder=" folder)))
      (html/div
        (html/attr "class" "notes-folder card")
        (html/join
          (list
            (html/h3 (html/a href (html/text label)))
            (if
              (null? items)
              (html/p*
                (html/attr "class" "muted")
                "No notes in this folder.")
              (notes/light/render-notes items)))))))

(define notes/light/last
  (lambda
    (xs)
    (if
      (null? (cdr xs))
      (car xs)
      (notes/light/last (cdr xs)))))

(define notes/light/last-segment
  (lambda
    (path)
    (if
      (string-eq path "")
      ""
      (notes/light/last (string-split path "/")))))

(define notes/light/note-li
  (lambda
    (note)
    (let*
      ((title (as-str (notes/light/nth note 2)))
        (slug (as-str (notes/light/nth note 3))))
      (html/li
        (html/attr "class" "note-item")
        (html/a
          (string-append "/notes?slug=" slug)
          (html/text title))))))

(define notes/light/notes-in
  (lambda
    (folder all-notes)
    (list/filter
      (lambda
        (n)
        (string-eq (as-str (notes/light/nth n 1)) folder))
      all-notes)))

(define notes/light/nth
  (lambda
    (xs i)
    (if
      (<= i 0)
      (car xs)
      (notes/light/nth (cdr xs) (- i 1)))))

(define notes/light/page
  (lambda
    ()
    (let
      ((slug (request-param "slug")))
```

```
      (if
        (if (null? slug) #t (string-eq (as-str slug) ""))
        (let
          ((folder (request-param "folder")))
          (html/page
            (list
              (notes/tree/style)
              (html/h1 "Notes")
              (html/h2 "Folders")
              (if
                (if
                  (null? folder)
                  #f
                  (not (string-eq (as-str folder) "")))
                (notes/light/folder-section (as-str folder))
                (notes/light/render)))))
        (html/page (list (notes/ui/show (as-str slug))))))))

(define notes/light/parent-of
  (lambda
    (path)
    (if
      (string-eq path "")
      ""
      (let
        ((parts (string-split path "/")))
        (if
          (<= (length parts) 1)
          ""
          (notes/tree/join-slash (notes/tree/butlast parts)))))))

(define notes/light/render
  (lambda
    ()
    (let*
      ((all-notes (notes/list-light))
        (paths (notes/light/all-folder-paths all-notes))
        (roots
          (notes/tree/sort-by-name
            (notes/light/children-of "" paths)))
        (uncat (notes/light/notes-in "" all-notes)))
      (html/div
        (html/attr "class" "notes-tree")
        (html/join
          (list
            (if
              (null? uncat)
              ""
              (html/tag
                "details"
                (html/attrs
                  (list
                    (html/attr "class" "tree-folder tree-uncategorized")
                    (html/attr "open" "open")))
                (html/join
                  (list
                    (html/tag
                      "summary"
                      (html/attr "class" "tree-folder-header")
                      (html/join
                        (list
                          (html/span
                            (html/attr "class" "tree-folder-icon")
                            "&#x1F4C2; ")
                          (html/text "(uncategorized)")
                          (html/span
                            (html/attr "class" "muted")
                            (html/text
                              (string-append " (" (as-str (length uncat))
    ")")))))))
                  (notes/light/render-notes uncat)))))
            (html/join
              (list/map
```

```
                (lambda
                  (root)
                  (notes/light/render-folder root paths all-notes))
                roots))))))))

(define notes/light/render-folder
  (lambda
    (path paths all-notes)
    (let*
      ((label (notes/light/last-segment path))
        (children (notes/light/children-of path paths))
        (items (notes/light/notes-in path all-notes))
        (item-count (length items)))
      (html/tag
        "details"
        (html/attrs
          (list
            (html/attr "class" "tree-folder")
            (html/attr "open" "open")))
        (html/join
          (list
            (html/tag
              "summary"
              (html/attr "class" "tree-folder-header")
              (html/join
                (list
                  (html/span
                    (html/attr "class" "tree-folder-icon")
                    "&#x1F4C1; ")
                  (html/a
                    (string-append "/notes?folder=" path)
                    (html/text label))
                  (html/span
                    (html/attr "class" "muted")
                    (html/text (string-append " (" (as-str item-count)
    ")"))))))
            (notes/light/render-notes items)
            (if
              (null? children)
              ""
              (html/join
                (list/map
                  (lambda
                    (child)
                    (notes/light/render-folder child paths all-notes))
                  (notes/tree/sort-by-name children)))))))))

(define notes/light/render-notes
  (lambda
    (note-list)
    (if
      (null? note-list)
      ""
      (html/ul
        (html/attr "class" "tree-items")
        (html/join
          (list/map
            (lambda (n) (notes/light/note-li n))
            note-list))))))
```

183 libs/notes/migrate/lib.pp

```
;; package notes/migrate -- M0 complete + F3/F4 cutover finish
;; Sync: scripts/pp_sync.py pull|push

(define notes/migrate/legacy-all
  (lambda
    ()
    "Migrate every legacy note row that is not yet present."
    (list/map
      notes/migrate/legacy-one
      (rows (quote note) 2000))))

(define notes/migrate/legacy-force-one
  (lambda
    (row)
    "Reseed one legacy note row, reoutlining even when already migrated."
    (if
      (null? row)
      "skip-empty"
      (let
        ((id (as-str (car row))) (f (cdr row)))
        (let
          ((slug (as-str (dict-get f "slug")))
            (title (as-str (dict-get f "title")))
            (outline (outline/decode (dict-get f "outline"))))
          (let
            ((existing (tree/get-uuid id)))
            (if
              (null? existing)
              (notes/migrate/legacy-one row)
              (let
                ((path (as-str (dict-get existing "path"))))
                (seq
                  (notes/tree/save-outline path outline)
                  (roam/reindex slug outline)
                  (string-append "reoutlined:" slug)))))))))

(define notes/migrate/legacy-force-all
  (lambda
    ()
    "Force-reoutline every existing legacy note row."
    (list/map
      notes/migrate/legacy-force-one
      (rows (quote note) 2000))))

(define notes/migrate/legacy-delete-all
  (lambda
    ()
    "Delete all legacy note rows that have a corresponding tree node."
    (list/map
      (lambda
        (row)
        (let
          ((id (as-str (car row)))
            (slug (as-str (dict-get (cdr row) "slug"))))
          (if
            (null? (tree/get-uuid id))
            (string-append "kept-missing-tree:" slug)
            (seq
              (delete (quote note) id)
              (string-append "deleted:" slug)))))
      (rows (quote note) 2000))))

(define notes/migrate/legacy-one
  (lambda
    (row)
    "Migrate a single legacy note row into the tree, or returns a status
     tag."
    (if
      (null? row)
      "skip-empty"
      (let
```

```
        ((id (as-str (car row))) (f (cdr row)))
        (let
          ((slug (as-str (dict-get f "slug")))
            (title (as-str (dict-get f "title")))
            (tags (as-str (dict-get f "tags")))
            (kind (as-str (dict-get f "kind")))
            (day (as-str (dict-get f "day")))
            (folder (as-str (dict-get f "folder")))
            (outline (outline/decode (dict-get f "outline"))))
          (let
            ((existing (tree/get-uuid id))
              (by-slug (notes/tree/find-slug slug)))
            (if
              (not (null? existing))
              (string-append "exists-uuid:" slug)
              (if
                (not (null? by-slug))
                (string-append "exists-slug:" slug)
                (let
                  ((path (notes/tree/note-path folder slug))
                    (body
                      (dict-set*
                        "{}"
                        (list
                          "uuid" id
                          "slug" slug
                          "title" title
                          "tags" tags
                          "kind" (if (string-eq kind "") "note" kind)
                          "day" day))))
                  (seq
                    (notes/tree/ensure-folder (notes/tree/root) (whoami))
                    (tree/upsert path "note" "note" body)
                    (notes/tree/save-outline path outline)
                    (roam/reindex slug outline)
                    (string-append "migrated:" slug)))))))))))
```

184 libs/notes/outline/lib.pp

```
;; package notes/outline -- exported from live branch admin/personal
;; Sync: scripts/pp_sync.py pull|push

(define notes/outline/all-todos
  (lambda
    ()
    "All (slug title todo done?) todo triples across notes docs."
    (list/foldl
      (lambda
        (acc row)
        (let
          ((f (cdr row)))
          (let
            ((slug (as-str (dict-get f "slug")))
              (tree (outline/decode (dict-get f "outline"))))
            (list/foldl
              (lambda
                (a t)
                (cons
                  (list slug (car t) (car (cdr t)) (car (cdr (cdr t))))
                  a))
              acc
              (outline/todos tree)))))
      (list)
      (notes/doc/list))))

(define notes/outline/apply-op
  (lambda
    (slug tree op nid)
    "Apply an outline edit op (add/delete/move/promote/set/archive/slurp)
     to tree."
    (cond
      ((string-eq op "add-child")
        (outline/add-child-by-id
          tree
          nid
          (notes/outline/new-node)))
      ((string-eq op "add-sibling")
        (outline/insert-sibling
          tree
          nid
          (notes/outline/new-node)))
      ((string-eq op "delete")
        (if
          (string-eq (as-str nid) "root")
          tree
          (outline/delete tree nid)))
      ((string-eq op "move-up") (outline/move-up tree nid))
      ((string-eq op "move-down")
        (outline/move-down tree nid))
      ((string-eq op "promote")
        (if
          (string-eq (as-str nid) "root")
          tree
          (outline/promote tree nid)))
      ((string-eq op "demote")
        (if
          (string-eq (as-str nid) "root")
          tree
          (outline/demote tree nid)))
      ((string-eq op "set-title")
        (outline/update
          tree
          nid
          (lambda
            (n)
            (outline/set-title n (request-param "title")))))
      ((string-eq op "set-body")
        (outline/update
          tree
          nid
```

```
          (lambda (n) (outline/set-body n (request-param "body")))))
      ((string-eq op "set-todo")
        (outline/update
          tree
          nid
          (lambda (n) (outline/set-todo n (request-param "todo")))))
      ((string-eq op "cycle-todo")
        (outline/update
          tree
          nid
          (lambda (n) (outline/cycle-todo n))))
      ((string-eq op "archive")
        (if
          (string-eq (as-str nid) "root")
          tree
          (outline/update
            tree
            nid
            (lambda (n) (outline/archive n)))))
      ((string-eq op "unarchive")
        (if
          (string-eq (as-str nid) "root")
          tree
          (outline/update
            tree
            nid
            (lambda (n) (outline/unarchive n)))))
      ((string-eq op "slurp-forward")
        (if
          (string-eq (as-str nid) "root")
          tree
          (car
            (structed/slurp-forward
              outline/kids
              outline/set-kids
              tree
              nid))))
      ((string-eq op "barf-forward")
        (if
          (string-eq (as-str nid) "root")
          tree
          (car
            (structed/barf-forward
              outline/kids
              outline/set-kids
              tree
              nid))))
      (else tree))))

(define notes/outline/apply-op-sel
  (lambda
    (slug tree op nid)
    "Apply op and return (new-tree selected-nid) with the post-op
     selection."
    (let
      ((new-tree (notes/outline/apply-op slug tree op nid)))
      (let
        ((sel (notes/outline/sel-for op nid new-tree tree)))
        (list new-tree sel)))))

(define notes/outline/cycle-todo
  (lambda
    (slug nid)
    "Cycle the todo state of node nid and persist the updated doc."
    (let
      ((prev (notes/doc/get slug)))
      (if
        (null? prev)
        ()
        (let
          ((tree (notes/doc/outline prev))
            (title (notes/doc/title prev))
            (tags (notes/doc/tags prev))
```

```
            (kind (notes/doc/kind prev))
            (day (notes/doc/day prev)))
          (let
            ((new-tree
              (outline/update
                tree
                nid
                (lambda (n) (outline/cycle-todo n)))))
            (notes/doc/upsert
              slug
              title
              new-tree
              tags
              kind
              day)))))))

(define notes/outline/eval-form
  (lambda
    (slug nid)
    "POST form for evaling a Lisp expr and splicing the result onto node
     nid."
    (html/form
      "POST"
      "/partials/notes-node-op"
      (html/attrs
        (list
          (html/attr "class" "ol-eval")
          (html/attr "hx-post" "/partials/notes-node-op")
          (html/attr "hx-target" "#outline")
          (html/attr "hx-swap" "outerHTML")))
      (html/join
        (list
          (html/hidden "slug" (as-str slug))
          (html/hidden "op" "eval")
          (html/hidden "node-id" (as-str nid))
          (html/text-input
            "expr"
            (html/attrs
              (list
                (html/attr
                  "placeholder"
                  "(outline/node (gensym \"n\") \"hi\")")
                (html/attr "class" "ol-eval-expr")
                (html/attr "style" "width:12rem"))))
          (html/select
            "target"
            (html/attr "class" "ol-eval-tgt")
            (html/join
              (list
                (html/option "child" "child" #t)
                (html/option "sibling" "sibling" #f)
                (html/option "body" "body" #f))))
          (html/button
            (html/attrs
              (list
                (html/attr "type" "submit")
                (html/attr "class" "ol-btn")
                (html/attr "title" "eval Lisp and splice result")))
            "eval"))))))

(define notes/outline/eval-result
  (lambda
    (slug nid expr)
    "Eval expr as a lambda over (self doc slug) and return its spliced
     value."
    (let
      ((prev (notes/doc/get slug)))
      (if
        (null? prev)
        ()
        (let
          ((tree (notes/doc/outline prev)))
          (let
```

```
            ((node (outline/find tree (as-str nid))))
            (if
              (null? node)
              ()
              (seq
                (load-string
                  (string-append
                    "(define __notes_eval_fn (lambda (self doc slug) "
                    (as-str expr)
                    "))"))
                (__notes_eval_fn node prev (as-str slug))))))))))

(define notes/outline/eval-target
  (lambda
    ()
    "Eval splice target from the request (child, sibling, or body);
     defaults to child."
    (let
      ((tg (request-param "target")))
      (if
        (if (null? tg) #t (string-eq (as-str tg) ""))
        "child"
        (as-str tg)))))

(define notes/outline/new-node
  (lambda () "Fresh outline node with a gensym id titled \"New node\"."
     (outline/node (gensym "n") "New node")))

;; op-seq helpers: the stored seq is the last op the server accepted.
;; A request with op-seq <= stored seq is stale (an older optimistic
;; apply arriving after a newer one already landed) and is rejected 409.
;; Spec: structural-editing.client-eval OP-SEQ. Stored in a small
;; note-op-seq data_rows table keyed by slug (no notes schema migration).
(define notes/outline/op-seq-get
  (lambda
    (slug)
    "Stored last-accepted op sequence for slug, or 0."
    (let
      ((rs (find-rows (quote note-op-seq) "slug" (as-str slug) 1)))
      (if (null? rs) 0 (parse-int (as-str (dict-get (cdr (car rs))
     "seq")))))))

(define notes/outline/op-seq-bump
  (lambda
    (slug seq)
    "Persist seq as the accepted op sequence for slug."
    (let
      ((rs (find-rows (quote note-op-seq) "slug" (as-str slug) 1)))
      (if
        (null? rs)
        (create
          (quote note-op-seq)
          (dict-set (dict-set "{}" "slug" (as-str slug)) "seq" (as-str
     seq)))
        (let ((row (car rs)))
          (update
            (quote note-op-seq)
            (car row)
            (dict-set (cdr row) "seq" (as-str seq)))))))))

(define notes/outline/node-op
  (lambda
    ()
    "POST handler for a single outline node op: applies it, persists,
     re-renders the outline."
    (let
      ((slug (as-str (request-param "slug")))
        (op (as-str (request-param "op")))
        (nid (as-str (request-param "node-id")))
        (show-archived (notes/doc/show-archived?)))
      (let
        ((prev (notes/doc/get slug)))
        (if
```

```
          (null? prev)
          (render-html
            (html/p* (html/attr "class" "err") "missing"))
          (let
            ((tree (notes/doc/outline prev))
              (title (notes/doc/title prev))
              (tags (notes/doc/tags prev))
              (kind (notes/doc/kind prev))
              (day (notes/doc/day prev)))
            (let
              ((op-seq-raw (request-param "op-seq")))
              (if
                (and (not (null? op-seq-raw)) (not (string-eq (as-str
    op-seq-raw) "")))
                (let
                  ((req-seq (parse-int (as-str op-seq-raw)))
                    (stored-seq (notes/outline/op-seq-get slug)))
                  (if
                    (<= req-seq stored-seq)
                    (status-response 409 "stale op-seq")
                    (seq
                      (notes/outline/op-seq-bump slug req-seq)
                      (let
                        ((res
                          (if
                            (string-eq op "eval")
                            (notes/outline/node-op-eval tree nid slug)
                            (notes/outline/apply-op-sel slug tree op nid))))
                        (let
                          ((new-tree (car res)) (sel (as-str (car (cdr
    res)))))
                          (seq
                            (notes/doc/upsert slug title new-tree tags kind
    day)
                            (render-html
                              (string-append
                                (outline/html-lite slug new-tree
    show-archived)
                                (html/void
                                  "input"
                                  (html/attrs
                                    (list
                                      (html/attr "type" "hidden")
                                      (html/attr "id" "ol-selected-id")
                                      (html/attr "hx-swap-oob" "true")
                                      (html/attr "value" sel))))))))))
                (let
                  ((res
                    (if
                      (string-eq op "eval")
                      (notes/outline/node-op-eval tree nid slug)
                      (notes/outline/apply-op-sel slug tree op nid))))
                  (let
                    ((new-tree (car res)) (sel (as-str (car (cdr res)))))
                    (seq
                      (notes/doc/upsert slug title new-tree tags kind day)
                      (render-html
                        (string-append
                          (outline/html-lite slug new-tree show-archived)
                          (html/void
                            "input"
                            (html/attrs
                              (list
                                (html/attr "type" "hidden")
                                (html/attr "id" "ol-selected-id")
                                (html/attr "hx-swap-oob" "true")
                                (html/attr "value" sel))))))))))))

(define notes/outline/node-op-eval
  (lambda
    (tree nid slug)
    "Eval splice handler: apply the eval result to tree at nid, return
     (tree nid)."
```

```
    (let
      ((et
        (notes/outline/splice-eval
          tree
          nid
          (notes/outline/eval-target)
          (notes/outline/eval-result
            slug
            nid
            (request-param "expr")))))
      (list et nid))))

(define notes/outline/open-todos
  (lambda
    ()
    "Todos that are not yet done."
    (list/filter
      (lambda
        (t)
        (outline/todo-open? (car (cdr (cdr (cdr t))))))
      (notes/outline/all-todos))))

(define notes/outline/partial
  (lambda
    ()
    "Rendered outline fragment for the requested slug (htmx swap target)."
    (let
      ((n (notes/doc/get (request-param "slug")))
        (show-archived (notes/doc/show-archived?)))
      (if
        (null? n)
        (render-html
          (html/p* (html/attr "class" "err") "missing"))
        (render-html
          (outline/html-lite
            (as-str (request-param "slug"))
            (notes/doc/outline n)
            show-archived))))))

(define notes/outline/sel-after-add-child
  (lambda
    (new-tree nid)
    "Selected id after adding a child to nid: its first child, else nid."
    (let
      ((node (structed/find outline/kids new-tree nid)))
      (if
        (null? node)
        nid
        (let
          ((ks (outline/kids node)))
          (if
            (null? ks)
            nid
            (structed/id (structed/list/nth ks 0)))))))))

(define notes/outline/sel-after-add-sibling
  (lambda
    (new-tree nid)
    "Selected id after adding a sibling: the next sibling, else nid."
    (let
      ((parent
        (structed/find-parent outline/kids new-tree nid)))
      (if
        (null? parent)
        nid
        (let
          ((sibs (outline/kids parent)))
          (let
            ((i (structed/sibling-index outline/kids new-tree nid)))
            (if
              (>= (+ i 1) (structed/count sibs))
              nid
              (structed/id (structed/list/nth sibs (+ i 1)))))))))))
```

```
423
424 (define notes/outline/sel-after-delete
425   (lambda
426     (tree nid)
427     "Selected id after deleting nid: next/allowed prior sibling or the
        parent."
428     (let
429       ((parent (structed/find-parent outline/kids tree nid)))
430       (if
431         (null? parent)
432         nid
433         (notes/outline/sel-after-delete-in
434           parent
435           (outline/kids parent)
436           (structed/sibling-index outline/kids tree nid))))))
437
438 (define notes/outline/sel-after-delete-in
439   (lambda
440     (parent sibs i)
441     "Selected id among sibs after removing index i: next sibling, prior, or
        parent."
442     (let
443       ((after (structed/list/drop sibs (+ i 1))))
444       (if
445         (not (null? after))
446         (structed/id (car after))
447         (if
448           (> i 0)
449           (structed/id (structed/list/nth sibs (- i 1)))
450           (structed/id parent))))))
451
452 (define notes/outline/sel-for
453   (lambda
454     (op nid new-tree old-tree)
455     "Post-op selected id for delete/add ops, else unchanged nid."
456     (cond
457       ((string-eq op "delete")
458         (if
459           (string-eq (as-str nid) "root")
460           nid
461           (notes/outline/sel-after-delete old-tree nid)))
462       ((string-eq op "add-child")
463         (notes/outline/sel-after-add-child new-tree nid))
464       ((string-eq op "add-sibling")
465         (notes/outline/sel-after-add-sibling new-tree nid))
466       (else nid))))
467
468 (define notes/outline/splice-eval
469   (lambda
470     (tree nid target result)
471     "Splice an eval result into tree at nid as body, sibling, or child."
472     (let
473       ((t (as-str target)))
474       (if
475         (string-eq t "body")
476         (outline/update
477           tree
478           nid
479           (lambda (n) (outline/set-body n (as-str result))))
480         (let
481           ((child
482             (if
483               (outline/valid? result)
484               result
485               (outline/node (gensym "n") (as-str result)))))
486           (if
487             (string-eq t "sibling")
488             (outline/insert-sibling tree nid child)
489             (outline/add-child-by-id tree nid child)))))))
```

185 libs/notes/refile/lib.pp

```
;; package notes/refile -- exported from live branch admin/personal
;; Sync: scripts/pp_sync.py pull|push

(define notes/refile
  (lambda
    (src-slug nid dst-slug)
    "Move node nid from src-slug to the root of dst-slug; #t on success."
    (let
      ((src (notes/get src-slug))
        (dst (notes/get dst-slug))
        (sid (as-str src-slug))
        (did (as-str dst-slug))
        (id (as-str nid)))
      (if
        (if (null? src) #t (null? dst))
        #f
        (if
          (string-eq sid did)
          #f
          (if
            (string-eq id "root")
            #f
            (let
              ((src-tree (notes/get-outline src))
                (dst-tree (notes/get-outline dst)))
              (let
                ((node (outline/find src-tree id)))
                (if
                  (null? node)
                  #f
                  (seq
                    (notes/upsert
                      did
                      (notes/get-title dst)
                      (outline/add-child-by-id dst-tree "root" node)
                      (notes/get-tags dst)
                      (notes/get-kind dst)
                      (notes/get-day dst))
                    (seq
                      (notes/upsert
                        sid
                        (notes/get-title src)
                        (outline/delete src-tree id)
                        (notes/get-tags src)
                        (notes/get-kind src)
                        (notes/get-day src))
                      #t)))))))))))

(define notes/refile/extract
  (lambda
    (src-slug nid)
    "Detach node nid as its own note and return its slug."
    (let
      ((src (notes/doc/get src-slug)) (id (as-str nid)))
      (if
        (null? src)
        ()
        (if
          (string-eq id "root")
          ()
          (let
            ((src-tree (notes/doc/outline src)))
            (let
              ((pair (outline/detach src-tree id)))
              (if
                (null? pair)
                ()
                (let
                  ((node (car pair)) (src-without (car (cdr pair))))
                  (let
                    ((title (as-str (outline/title node)))
```

```
                      (body (as-str (outline/body node)))
                      (kids (outline/kids node))
                      (tags (notes/doc/tags src)))
                    (let
                      ((new-slug
                          (notes/refile/unique-slug (notes/refile/slugify
    title)))
                        (new-tree
                          (outline/set-kids
                            (outline/set-body (outline/node "root" title)
    body)
                            kids)))
                        (with-transaction
                          (lambda
                            ()
                            (seq
                              (notes/doc/upsert
                                new-slug
                                title
                                new-tree
                                tags
                                "note"
                                "")
                              (notes/doc/upsert
                                (as-str src-slug)
                                (notes/doc/title src)
                                src-without
                                (notes/doc/tags src)
                                (notes/doc/kind src)
                                (notes/doc/day src))
                              new-slug))))))))))))))

(define notes/refile/extract-form
  (lambda
    (src-slug nid)
    "HTML extract button for node nid, or empty for the root."
    (if
      (string-eq (as-str nid) "root")
      ""
      (html/form
        "POST"
        "/notes"
        (html/attr "class" "ol-extract")
        (html/join
          (list
            (html/hidden "action" "extract")
            (html/hidden "src_slug" (as-str src-slug))
            (html/hidden "node_id" (as-str nid))
            (html/button
              (html/attrs
                (list
                  (html/attr "type" "submit")
                  (html/attr "class" "ol-btn")
                  (html/attr "title" "extract node into its own note")))
              "extract")))))))

(define notes/refile/extract-handle
  (lambda
    ()
    "Handle the extract POST and redirect to the new or source note."
    (let
      ((src (as-str (request-param "src_slug")))
        (nid (as-str (request-param "node_id"))))
      (let
        ((new (notes/refile/extract src nid)))
        (if
          (null? new)
          (redirect (string-append "/notes?slug=" src))
          (redirect (string-append "/notes?slug=" new)))))))

(define notes/refile/form
  (lambda
    (src-slug nid)
```

```
    "HTML refile form for node nid, or empty for the root."
    (if
      (string-eq (as-str nid) "root")
      ""
      (html/form
        "POST"
        "/notes"
        (html/attr "class" "ol-refile")
        (html/join
          (list
            (html/hidden "action" "refile")
            (html/hidden "src_slug" (as-str src-slug))
            (html/hidden "node_id" (as-str nid))
            (html/select
              "dst_slug"
              (html/attr "class" "ol-refile-sel")
              (notes/refile/options src-slug))
            (html/button
              (html/attrs
                (list
                  (html/attr "type" "submit")
                  (html/attr "class" "ol-btn")
                  (html/attr "title" "move node to another note")))
              "refile")))))))

(define notes/refile/handle
  (lambda
    ()
    "Handle the refile POST by moving the node then redirecting."
    (let
      ((src (as-str (request-param "src_slug")))
        (nid (as-str (request-param "node_id")))
        (dst (as-str (request-param "dst_slug")))
        (dst-nid
          (if
            (null? (request-param "dst_node_id"))
            ""
            (as-str (request-param "dst_node_id")))))
      (seq
        (notes/refile/move src nid dst dst-nid)
        (redirect (string-append "/notes?slug=" src))))))

(define notes/refile/move
  (lambda
    (src-slug nid dst-slug dst-nid)
    "Move node nid to parent dst-nid within dst-slug; #t on success."
    (let
      ((src (notes/doc/get src-slug))
        (dst (notes/doc/get dst-slug))
        (sid (as-str src-slug))
        (did (as-str dst-slug))
        (id (as-str nid))
        (parent
          (let
            ((p (as-str dst-nid)))
            (if (string-eq p "") "root" p))))
      (if
        (if (null? src) #t (null? dst))
        #f
        (if
          (string-eq sid did)
          #f
          (if
            (string-eq id "root")
            #f
            (let
              ((src-tree (notes/doc/outline src))
                (dst-tree (notes/doc/outline dst)))
              (let
                ((pair (outline/detach src-tree id)))
                (if
                  (null? pair)
                  #f
```

```
                  (let
                    ((node (car pair)) (src-without (car (cdr pair))))
                    (with-transaction
                      (lambda
                        ()
                        (seq
                          (notes/doc/upsert
                            did
                            (notes/doc/title dst)
                            (outline/add-child-by-id dst-tree parent node)
                            (notes/doc/tags dst)
                            (notes/doc/kind dst)
                            (notes/doc/day dst))
                          (notes/doc/upsert
                            sid
                            (notes/doc/title src)
                            src-without
                            (notes/doc/tags src)
                            (notes/doc/kind src)
                            (notes/doc/day src))
                          #t))))))))))))

(define notes/refile/options
  (lambda
    (src-slug)
    "HTML option list of all notes, excluding src-slug."
    (list/foldl
      (lambda
        (acc row)
        (let
          ((f (cdr row)))
          (let
            ((slug (as-str (dict-get f "slug")))
              (title (as-str (dict-get f "title"))))
            (if
              (string-eq slug (as-str src-slug))
              acc
              (string-append
                acc
                (html/option
                  slug
                  (if (string-eq title "") slug title)
                  #f))))))
      ""
      (notes/doc/list))))

(define notes/refile/slug-char-ok?
  (lambda
    (c)
    "True when c is an ASCII letter or digit."
    (string-contains?
      "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789"
      (as-str c))))

(define notes/refile/slugify
  (lambda
    (title)
    "Turn a title into a slug, -joined and truncated to 48 chars."
    (let
      ((raw
        (notes/refile/slugify-acc
          (string-trim (as-str title))
          0
          ""
          #t)))
      (let
        ((n (string-length raw)))
        (let
          ((s
            (if
              (if
                (> n 0)
                (string-eq (substring raw (- n 1) 1) "-")
```

```
                #f)
              (substring raw 0 (- n 1))
              raw)))
          (if
            (string-eq s "")
            (string-append "note-" (as-str (gensym "n")))
            (if (> (string-length s) 48) (substring s 0 48) s)))))))

(define notes/refile/slugify-acc
  (lambda
    (s i acc last-hyphen?)
    "Accumulator for slugify, inserting - and coalescing separators."
    (if
      (>= i (string-length s))
      acc
      (let
        ((c (substring s i 1)))
        (if
          (notes/refile/slug-char-ok? c)
          (notes/refile/slugify-acc
            s
            (+ i 1)
            (string-append acc c)
            #f)
          (if
            last-hyphen?
            (notes/refile/slugify-acc s (+ i 1) acc #t)
            (notes/refile/slugify-acc
              s
              (+ i 1)
              (string-append acc "-")
              #t)))))))

(define notes/refile/unique-slug
  (lambda
    (base)
    "Return base if unused, else append a gensym suffix."
    (if
      (null? (notes/doc/row (as-str base)))
      (as-str base)
      (notes/refile/unique-slug
        (string-append (as-str base) "-" (as-str (gensym "x")))))))
```

186 libs/notes/tree/lib.pp

```
;; package notes/tree -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push

(define notes/tree/all-paths
  (lambda
    (names)
    (list/foldl
      (lambda
        (acc name)
        (if
          (string-eq name "")
          acc
          (let
            ((parent (notes/tree/parent-of name)))
            (if
              (string-eq parent "")
              acc
              (if
                (notes/doc/folder-seen? acc parent)
                acc
                (if
                  (notes/doc/folder-seen? names parent)
                  acc
                  (append acc (list parent))))))))
      (list)
      names)))

(define notes/tree/body
  (lambda
    (node)
    (if (null? node) () (dict-get node "body"))))

(define notes/tree/butlast
  (lambda
    (xs)
    (if
      (null? xs)
      (list)
      (if
        (null? (cdr xs))
        (list)
        (cons (car xs) (notes/tree/butlast (cdr xs)))))))

(define notes/tree/children-of
  (lambda
    (parent names)
    (list/filter
      (lambda
        (name)
        (if
          (string-eq name "")
          #f
          (let
            ((p (notes/tree/parent-of name)))
            (string-eq p parent))))
      names)))

(define notes/tree/clear-headings
  (lambda
    (note-path)
    (list/foldl
      (lambda
        (acc nj)
        (let
          ((n (notes/tree/parse nj)))
          (if
            (null? n)
            acc
            (if
              (string-eq (as-str (dict-get n "kind")) "heading")
              (seq
```

```
                (tree/delete (as-str (dict-get n "path")) "recursive")
                acc)
              acc))))
      0
      (tree/list note-path 1))))

(define notes/tree/details-css
  (lambda
    ()
    (css/sheet
      (list
        (css/raw "/* details/summary tree nesting */")
        (css/rule
          "summary.tree-folder-header"
          (list "cursor" "pointer" "list-style" "none"))
        (css/rule
          "summary.tree-folder-header::-webkit-details-marker"
          (list "display" "none"))
        (css/rule
          "summary.tree-folder-header::marker"
          (list "content" ""))
        (css/rule
          ".notes-tree ul.tree-items"
          (list
            "list-style"
            "none"
            "padding"
            "0 0 0 1rem"
            "margin"
            ".15rem 0"))
        (css/rule
          ".notes-tree > details.tree-folder"
          (list "margin" ".25rem 0"))
        (css/raw
          "/* refile modal (slug picker -- no O(n) options) */")
        (css/rule
          "details.ol-refile-modal"
          (list "display" "inline-block" "position" "relative"))
        (css/rule
          "details.ol-refile-modal > summary.ol-btn"
          (list "cursor" "pointer" "list-style" "none"))
        (css/rule
          "details.ol-refile-modal > summary.ol-btn::-webkit-details-marker"
          (list "display" "none"))
        (css/rule
          "details.ol-refile-modal[open] > .ol-refile-body"
          (list
            "position"
            "absolute"
            "z-index"
            "20"
            "background"
            "var(--card,#fff)"
            "border"
            "1px solid var(--muted,#888)"
            "padding"
            ".5rem"
            "min-width"
            "16rem"
            "box-shadow"
            "0 2px 8px rgba(0,0,0,.12)"))
        (css/rule
          ".ol-refile-body .ol-refile-sel"
          (list "width" "12rem" "max-width" "100%"))
        (css/rule
          ".ol-refile-body .ol-refile-label"
          (list "font-size" ".85rem" "margin-right" ".25rem"))
        (css/raw "/* find / search modal */")
        (css/rule
          "details.ol-find-modal"
          (list "display" "inline-block" "position" "relative"))
        (css/rule
          "details.ol-find-modal > summary.ol-btn"
```

```
          (list "cursor" "pointer" "list-style" "none"))
        (css/rule
          "details.ol-find-modal > summary.ol-btn::-webkit-details-marker"
          (list "display" "none"))
        (css/rule
          "details.ol-find-modal[open] > .ol-find-body,\n
      details.ol-refile-modal[open] > .ol-refile-body.ol-find-wide"
          (list
            "position"
            "absolute"
            "z-index"
            "30"
            "right"
            "0"
            "background"
            "var(--card,#fff)"
            "border"
            "1px solid var(--muted,#888)"
            "padding"
            ".5rem"
            "min-width"
            "22rem"
            "max-width"
            "min(36rem, 92vw)"
            "box-shadow"
            "0 4px 16px rgba(0,0,0,.14)"))
        (css/rule
          ".ol-find-input"
          (list
            "width"
            "100%"
            "box-sizing"
            "border-box"
            "padding"
            ".35rem .5rem"
            "margin"
            "0 0 .35rem 0"))
        (css/rule
          ".ol-find-results"
          (list
            "max-height"
            "18rem"
            "overflow"
            "auto"
            "margin"
            "0"
            "padding"
            "0"
            "list-style"
            "none"))
        (css/rule
          ".ol-find-group"
          (list
            "font-size"
            ".75rem"
            "font-weight"
            "600"
            "text-transform"
            "uppercase"
            "letter-spacing"
            ".04em"
            "color"
            "var(--muted,#666)"
            "padding"
            ".4rem .35rem .15rem"))
        (css/rule
          "a.ol-find-hit, button.ol-find-hit"
          (list
            "display"
            "block"
            "width"
            "100%"
            "text-align"
```

```
            "left"
            "border"
            "0"
            "background"
            "transparent"
            "padding"
            ".35rem .5rem .35rem 1rem"
            "cursor"
            "pointer"
            "color"
            "inherit"
            "text-decoration"
            "none"
            "font"
            "inherit"
            "box-sizing"
            "border-box"))
        (css/rule
          "a.ol-find-hit:hover, button.ol-find-hit:hover,\n
      a.ol-find-hit.is-active, button.ol-find-hit.is-active"
          (list "background" "var(--accent-bg,#eef3ff)"))
        (css/rule
          ".ol-find-hit-path"
          (list "font-size" ".85rem"))
        (css/rule
          ".ol-find-hit-meta"
          (list
            "font-size"
            ".75rem"
            "color"
            "var(--muted,#666)"
            "margin-left"
            ".35rem"))
        (css/rule
          ".ol-find-empty"
          (list
            "padding"
            ".5rem"
            "color"
            "var(--muted,#666)"
            "font-size"
            ".9rem"))
        (css/rule
          ".ol-find-bar"
          (list
            "display"
            "flex"
            "gap"
            ".5rem"
            "align-items"
            "center"
            "margin"
            "0 0 .75rem 0"))
        (css/rule
          ".ol-find-bar details.ol-find-modal"
          (list "flex" "1 1 auto"))
        (css/rule
          ".ol-find-bar .ol-find-body"
          (list "left" "0" "right" "auto"))
        (css/raw "/* create form collapsed behind summary */")
        (css/rule
          "details.notes-create > summary.notes-create-sum"
          (list
            "cursor"
            "pointer"
            "font-weight"
            "600"
            "list-style"
            "none"))
        (css/rule
          "details.notes-create >
      summary.notes-create-sum::-webkit-details-marker"
          (list "display" "none"))
```

```
        (css/rule
          "details.notes-create > .notes-create-body"
          (list "margin-top" ".5rem"))))))

(define notes/tree/enabled?
  (lambda () #t))

(define notes/tree/ensure-chain
  (lambda
    (base segs)
    (if
      (null? segs)
      base
      (let
        ((next
          (string-append base "." (tree/path-label (car segs)))))
        (seq
          (notes/tree/ensure-folder next (car segs))
          (notes/tree/ensure-chain next (cdr segs)))))))

(define notes/tree/ensure-folder
  (lambda
    (path title)
    (let
      ((cur (tree/get path)))
      (if
        (null? cur)
        (tree/upsert
          path
          "folder"
          "note"
          (dict-set*
            "{}"
            (list "uuid" (random-uuid) "title" (as-str title))))
        path))))

(define notes/tree/find-slug
  (lambda
    (slug)
    (let
      ((hits (tree/find-by "note" "slug" (as-str slug) 5)))
      (if
        (null? hits)
        ()
        (notes/tree/node->row (car hits) #t)))))

(define notes/tree/folder-label
  (lambda
    (path)
    (let
      ((pos (string-find-pos path "/" 0 1)))
      (if
        (< pos 0)
        path
        (substring
          path
          (+ pos 1)
          (- (string-length path) (+ pos 1)))))))

(define notes/tree/folder-of-path
  (lambda
    (path)
    (let
      ((root (notes/tree/root)) (p (as-str path)))
      (if
        (string-starts-with? p (string-append root "."))
        (let
          ((rest
            (substring
              p
              (+ (string-length root) 1)
              (- (string-length p) (+ (string-length root) 1)))))
          (let
```

```
            ((parts (string-split rest ".")))
            (notes/tree/join-slash (notes/tree/butlast parts))))
        ""))))

(define notes/tree/heading-kids
  (lambda
    (parent-path)
    (list/foldl
      (lambda
        (acc nj)
        (let
          ((n (notes/tree/parse nj)))
          (if
            (null? n)
            acc
            (if
              (string-eq (as-str (dict-get n "kind")) "heading")
              (let
                ((b (notes/tree/body n)))
                (let
                  ((p (as-str (dict-get n "path"))))
                  (append
                    acc
                    (list
                      (outline/node*
                        (as-str (dict-get b "id"))
                        (as-str (dict-get b "title"))
                        (as-str (dict-get b "body"))
                        (as-str (dict-get b "todo"))
                        (notes/tree/heading-kids p))))))
              acc))))
      (list)
      (tree/list parent-path 1))))

(define notes/tree/join-slash
  (lambda
    (xs)
    (if
      (null? xs)
      ""
      (if
        (null? (cdr xs))
        (as-str (car xs))
        (string-append
          (as-str (car xs))
          "/"
          (notes/tree/join-slash (cdr xs)))))))

(define notes/tree/list-notes
  (lambda () (tree/note-index (notes/tree/root) 50000)))

(define notes/tree/load-outline
  (lambda
    (note-path title)
    (outline/node*
      "root"
      (as-str title)
      (notes/tree/root-body note-path)
      ""
      (notes/tree/heading-kids note-path))))

(define notes/tree/node->row
  (lambda
    (nj load-outline?)
    (let
      ((n (notes/tree/parse nj)))
      (if
        (null? n)
        ()
        (if
          (not (string-eq (as-str (dict-get n "kind")) "note"))
          ()
          (let
```

```
            ((b (notes/tree/body n))
              (path (as-str (dict-get n "path"))))
            (let
              ((u (dict-get b "uuid"))
                (title (as-str (dict-get b "title"))))
              (let
                ((uuid
                  (if (null? u) (as-str (dict-get n "id")) (as-str u))))
                (let
                  ((fields
                    (list
                      "slug"
                      (as-str (dict-get b "slug"))
                      "title"
                      title
                      "tags"
                      (as-str (dict-get b "tags"))
                      "kind"
                      (as-str (dict-get b "kind"))
                      "day"
                      (as-str (dict-get b "day"))
                      "folder"
                      (notes/tree/folder-of-path path)
                      "_path"
                      path)))
                  (cons
                    uuid
                    (dict-set*
                      "{}"
                      (if
                        load-outline?
                        (append
                          fields
                          (list
                            "outline"
                            (outline/encode (notes/tree/load-outline path
    title))))
                        (append fields (list "outline" "[]"))))))))))

(define notes/tree/note-path
  (lambda
    (folder slug)
    (let
      ((base
        (notes/tree/ensure-chain
          (notes/tree/root)
          (notes/tree/split-folder folder))))
      (string-append base "." (tree/path-label slug)))))

(define notes/tree/parent-of
  (lambda
    (path)
    (let
      ((pos (string-find-pos path "/" 0 1)))
      (if (< pos 0) "" (substring path 0 pos)))))

(define notes/tree/parse
  (lambda (s) (if (null? s) () (json-parse (as-str s)))))

(define notes/tree/render
  (lambda
    ()
    (let*
      ((names (notes/doc/folder-names))
        (all-names (append names (notes/tree/all-paths names)))
        (roots
          (notes/tree/sort-by-name
            (notes/tree/children-of "" all-names)))
        (uncat (notes/doc/list-folder "")))
      (html/div
        (html/attr "class" "notes-tree")
        (html/join
          (list
```

```
            (if
              (null? uncat)
              ""
              (html/tag
                "details"
                (html/attrs
                  (list
                    (html/attr "class" "tree-folder tree-uncategorized")
                    (html/attr "open" "open")))
                (html/join
                  (list
                    (html/tag
                      "summary"
                      (html/attr "class" "tree-folder-header")
                      (html/join
                        (list
                          (html/span
                            (html/attr "class" "tree-folder-icon")
                            "&#x1F4C2; ")
                          (html/text "(uncategorized)")
                          (html/span
                            (html/attr "class" "muted")
                            (html/text
                              (string-append " (" (as-str (length uncat))
    ")"))))))
                    (html/ul
                      (html/attr "class" "tree-items")
                      (notes/ui/folder-list ""))))))
            (html/join
              (list/map
                (lambda
                  (root)
                  (notes/tree/render-folder root all-names 0))
                roots))))))))

(define notes/tree/render-folder
  (lambda
    (path names depth)
    (let*
      ((label (notes/tree/folder-label path))
        (children (notes/tree/children-of path names))
        (items (notes/doc/list-folder path))
        (item-count (length items)))
      (html/tag
        "details"
        (html/attrs
          (list
            (html/attr "class" "tree-folder")
            (html/attr "open" "open")))
        (html/join
          (list
            (html/tag
              "summary"
              (html/attr "class" "tree-folder-header")
              (html/join
                (list
                  (html/span
                    (html/attr "class" "tree-folder-icon")
                    "&#x1F4C1; ")
                  (html/a
                    (string-append "/notes?folder=" path)
                    (html/text label))
                  (html/span
                    (html/attr "class" "muted")
                    (html/text (string-append " (" (as-str item-count)
    ")"))))))
            (if
              (null? items)
              ""
              (html/ul
                (html/attr "class" "tree-items")
                (notes/ui/folder-list path)))
            (if
```

```
              (null? children)
              ""
              (html/join
                (list/map
                  (lambda
                    (child)
                    (notes/tree/render-folder child names (+ depth 1)))
                  (notes/tree/sort-by-name children)))))))))))

(define notes/tree/root
  (lambda () (tree/garden-root)))

(define notes/tree/root-body
  (lambda
    (note-path)
    (let
      ((cur (tree/get note-path)))
      (if
        (null? cur)
        ""
        (let
          ((b
            (dict-get
              (notes/tree/body (notes/tree/parse cur))
              "body")))
          (if (null? b) "" (as-str b)))))))

(define notes/tree/save-outline
  (lambda
    (note-path outline)
    (seq
      (notes/tree/clear-headings note-path)
      (notes/tree/write-root-body
        note-path
        (outline/body outline))
      (list/foldl
        (lambda
          (acc kid)
          (seq (notes/tree/write-heading note-path kid) acc))
        0
        (outline/kids outline))
      note-path)))

(define notes/tree/sort-by-name
  (lambda
    (names)
    (if
      (null? names)
      (list)
      (let
        ((pivot (car names)) (rest (cdr names)))
        (append
          (notes/tree/sort-by-name
            (list/filter (lambda (x) (string<? x pivot)) rest))
          (list pivot)
          (notes/tree/sort-by-name
            (list/filter (lambda (x) (not (string<? x pivot))) rest)))))))

(define notes/tree/split-folder
  (lambda
    (folder)
    (list/filter
      (lambda (s) (not (string-eq (as-str s) "")))
      (string-split (as-str folder) "/"))))

(define notes/tree/style
  (lambda () (html/tag "style" "" notes/tree/style-css)))

(define notes/tree/style-css
  (string-append
    (css/render notes/tree-css)
    (css/render notes/tree-css-patch)
    (css/render (notes/tree/details-css))))
```

```
650
651 (define notes/tree/write-heading
652   (lambda
653     (parent-path node)
654     (let
655       ((nid (tree/path-label (outline/id node)))
656         (path
657           (string-append
658             parent-path
659             ".h."
660             (tree/path-label (outline/id node)))))
661       (seq
662         (tree/upsert
663           path
664           "heading"
665           "note"
666           (dict-set*
667             "{}"
668             (list
669               "id"
670               (as-str (outline/id node))
671               "title"
672               (as-str (outline/title node))
673               "body"
674               (as-str (outline/body node))
675               "todo"
676               (as-str (outline/todo node)))))
677         (list/foldl
678           (lambda
679             (acc kid)
680             (seq (notes/tree/write-heading path kid) acc))
681           0
682           (outline/kids node))
683         path))))
684
685 (define notes/tree/write-root-body
686   (lambda
687     (note-path body)
688     (let
689       ((cur (tree/get note-path)))
690       (if
691         (null? cur)
692         note-path
693         (let*
694           ((n (notes/tree/parse cur)) (b (notes/tree/body n)))
695           (seq
696             (tree/upsert
697               note-path
698               "note"
699               "note"
700               (dict-set b "body" (as-str body)))
701             note-path))))))
```

187 libs/notes/ui/lib.pp

```
;; package notes/ui -- exported from live branch admin/personal
;; Sync: scripts/pp_sync.py pull|push

;; Collapsed by default -- open the summary to create (keeps the index
     clean).
;; Collapsed by default -- open summary to create (keeps the index clean).
(define notes/ui/create-form
  (lambda
    ()
    "Collapsible HTML form to create a new note."
    (html/tag
      "details"
      (html/attr "class" "notes-create card")
      (html/join
        (list
          (html/tag
            "summary"
            (html/attr "class" "notes-create-sum")
            "New note")
          (html/form
            "POST"
            "/notes"
            (html/attr "class" "notes-create-body")
            (html/join
              (list
                (html/hidden "action" "create")
                (html/label "" "Title ")
                (html/text-input
                  "title"
                  (html/attr "placeholder" "New note"))
                " "
                (html/label "" "Slug ")
                (html/text-input
                  "slug"
                  (html/attr "placeholder" "my-note"))
                " "
                (html/label "" "Folder ")
                (html/text-input
                  "folder"
                  (html/attr "placeholder" "wiki/entities"))
                " "
                (html/button
                  (html/attr "type" "submit")
                  "Create")))))))))

(define notes/ui/do-create
  (lambda
    (title slug folder)
    "Create a note and redirect to it; derived slug when slug is blank."
    (let
      ((s
          (if
            (if
              (null? slug)
              #t
              (string-eq (string-trim (as-str slug)) ""))
            (string-append
              "note-"
              (as-str (count (notes/doc/list))))
            (string-trim (as-str slug))))
        (fld
          (if (null? folder) "" (string-trim (as-str folder)))))
      (seq
        (notes/doc/upsert-in
          s
          (as-str title)
          (notes/doc/empty-outline (as-str title))
          ""
          "note"
          ""
          fld)
```

```
        (redirect (string-append "/notes?slug=" s))))))

(define notes/ui/do-delete
  (lambda
    ()
    "Delete the requested slug and redirect to the note index."
    (let
      ((slug (as-str (request-param "slug"))))
      (seq (notes/doc/delete slug) (redirect "/notes")))))

(define notes/ui/do-save
  (lambda
    (slug raw)
    "Save a note from raw outline JSON, or show an error page."
    (let
      ((slug-s (as-str slug))
        (prev (notes/doc/get (as-str slug))))
      (let
        ((title (if (null? prev) slug-s (notes/doc/title prev)))
          (tags (if (null? prev) "" (notes/doc/tags prev)))
          (kind (if (null? prev) "note" (notes/doc/kind prev)))
          (day (if (null? prev) "" (notes/doc/day prev))))
        (if
          (or
            (null? raw)
            (string-eq (string-trim (as-str raw)) ""))
          (seq
            (notes/doc/upsert
              slug-s
              title
              (outline/node "root" "Untitled")
              tags
              kind
              day)
            (redirect (string-append "/notes?slug=" slug-s)))
          (let
            ((parsed (json-parse (as-str raw))))
            (if
              (outline/valid? parsed)
              (seq
                (notes/doc/upsert
                  slug-s
                  title
                  parsed
                  tags
                  kind
                  day)
                (redirect (string-append "/notes?slug=" slug-s)))
              (html/page
                (list
                  (html/h1 (string-append "Could not save " slug-s))
                  (html/p*
                    (html/attr "class" "err")
                    "The outline is malformed. Each node must be a
    5-element list [id title body todo kids]. The note was not changed.")
                  (html/p*
                    ""
                    (html/a
                      (string-append "/notes?slug=" slug-s)
                      "Back to the note"))
                  (html/tag
                    "pre"
                    (html/attrs (list (html/attr "class" "err")))
                    (as-str raw)))))))))))

(define notes/ui/edit-form
  (lambda
    (slug outline)
    "HTML form to save the raw outline JSON for a note."
    (html/form
      "POST"
      "/notes"
      ""
```

```
      (html/join
        (list
          (html/hidden "action" "save")
          (html/hidden "slug" slug)
          (html/tag
            "textarea"
            (html/attrs
              (list
                (html/attr "name" "outline")
                (html/attr "rows" "16")))
            (outline/encode outline))
          (html/p*
            ""
            (html/button (html/attr "type" "submit") "Save outline")))))))

(define notes/ui/folder-form
  (lambda
    (slug folder)
    "HTML form to move a note into a folder."
    (html/form
      "POST"
      "/notes"
      (html/attr "class" "notes-folder-form")
      (html/join
        (list
          (html/hidden "action" "set-folder")
          (html/hidden "slug" (as-str slug))
          (html/label "" "folder ")
          (html/text-input
            "folder"
            (html/attrs
              (list
                (html/attr "value" (as-str folder))
                (html/attr "placeholder" "wiki/entities")
                (html/attr "style" "width:12rem"))))
          " "
          (html/button (html/attr "type" "submit") "move"))))))

(define notes/ui/folder-list
  (lambda
    (folder)
    "HTML list of note items in a folder."
    (list/foldl
      (lambda
        (acc row)
        (string-append acc (notes/ui/item-li row)))
      ""
      (notes/doc/list-folder folder))))

(define notes/ui/folder-section
  (lambda
    (folder)
    "HTML section (header plus list) for a folder."
    (let
      ((label
          (if
            (string-eq (as-str folder) "")
            "(uncategorized)"
            (as-str folder)))
        (href
          (if
            (string-eq (as-str folder) "")
            "/notes?folder="
            (string-append "/notes?folder=" (as-str folder)))))
      (html/div
        (html/attr "class" "notes-folder card")
        (html/join
          (list
            (html/h3 (html/a href (html/text label)))
            (html/ul "" (notes/ui/folder-list folder))))))))
```

```
216
217 (define notes/ui/handle-post
218   (lambda
219     ()
220     "Route a notes POST to the matching action handler."
221     (let
222       ((action (as-str (request-param "action"))))
223       (cond
224         ((string-eq action "create")
225           (notes/ui/do-create
226             (request-param "title")
227             (request-param "slug")
228             (request-param "folder")))
229         ((string-eq action "set-folder")
230           (seq
231             (notes/doc/set-folder
232               (request-param "slug")
233               (if
234                 (null? (request-param "folder"))
235                 ""
236                 (request-param "folder")))
237             (redirect
238               (string-append
239                 "/notes?slug="
240                 (as-str (request-param "slug"))))))
241         ((string-eq action "save")
242           (notes/ui/do-save
243             (request-param "slug")
244             (request-param "outline")))
245         ((string-eq action "capture") (notes/capture/handle))
246         ((string-eq action "delete") (notes/ui/do-delete))
247         ((string-eq action "refile") (notes/refile/handle))
248         ((string-eq action "extract")
249           (notes/refile/extract-handle))
250         (else (redirect "/notes"))))))
251
252
253 (define notes/ui/item-li
254   (lambda
255     (row)
256     "HTML list item for a note row (with delete form)."
257     (let
258       ((f (cdr row)) (id (car row)))
259       (let
260         ((slug (as-str (dict-get f "slug")))
261           (title (as-str (dict-get f "title")))
262           (folder (notes/doc/row-folder row)))
263         (html/li
264           (html/attr "class" "note-item")
265           (html/join
266             (list
267               (html/a
268                 (string-append "/notes?slug=" slug)
269                 (html/text title))
270               (if
271                 (string-eq folder "")
272                 ""
273                 (html/span
274                   (html/attr "class" "muted")
275                   (html/text (string-append " ? " folder))))
276               (html/form
277                 "POST"
278                 "/notes"
279                 (html/attrs
280                   (list
281                     (html/attr "class" "note-del")
282                     (html/attr "hx-confirm" "Delete this note?")))
283                 (html/join
284                   (list
285                     (html/hidden "action" "delete")
286                     (html/hidden "slug" slug)
287                     (html/button
288                       (html/attrs
```

```
                        (list
                          (html/attr "type" "submit")
                          (html/attr "class" "secondary")
                          (html/attr "title" "delete note")))
                      "del")))))))))))

(define notes/ui/item-li-lite
  (lambda
    (row)
    "HTML list item for a note row (no delete form)."
    (let
      ((f (cdr row)))
      (let
        ((slug (as-str (dict-get f "slug")))
          (title (as-str (dict-get f "title")))
          (folder (notes/doc/row-folder row)))
        (html/li
          (html/attr "class" "note-item")
          (html/join
            (list
              (html/a
                (string-append "/notes?slug=" slug)
                (html/text title))
              (if
                (string-eq folder "")
                ""
                (html/span
                  (html/attr "class" "muted")
                  (html/text (string-append " ? " folder))))))))))

(define notes/ui/missing
  (lambda
    (slug)
    "HTML page fragment for a missing note."
    (html/join
      (list
        (html/h1 "Missing")
        (html/p*
          (html/attr "class" "err")
          (string-append "No note " (as-str slug)))
        (html/p* "" (html/a "/notes" "All notes"))))))

(define notes/ui/outline-script
  (lambda
    (slug show-archived tree)
    "Embedded interp.js bootstrap and editing script for a note."
    (html/tag
      "script"
      ""
      (string-append
        "(function(){var slug='"
        (as-str slug)
        "';var showArch="
        (if show-archived "true" "false")
        ";var ppTree="
        (json-emit tree)
        ";window.__ppTree=ppTree;"
        "var __ppReady=false,__ppBooting=false,__opSeq=0;"
        "var
    st=document.createElement('style');st.textContent='.pp-in-transit{opacity:.5;filter:grayscale(1);}';document.head.apper
        "function
    ppBoot(){if(__ppReady||__ppBooting)return;__ppBooting=true;"
        "var s=document.createElement('script');s.src='/assets/interp.js';"
        "s.onload=function(){var pp=window.pp;pp.boot();"
        "var pkgs=['html','structed','structed/list','outline','bible'];var
    done=0;"

    "pkgs.forEach(function(p){fetch('/code/raw/libs/'+p+'/lib.pp',{credentials:'same-origin'}).then(function(r){return
    r.text();}).then(function(src){pp.load_source(src);done++;if(done===pkgs.length){fetch('/code/raw/client/ops.pp',{crede
    r.text();}).then(function(os){pp.load_source(os);pp.set_tree(JSON.stringify(ppTree));__ppReady=true;__ppBooting=false;]
        "document.head.appendChild(s);}"
        "ppBoot();"
```

```
355         "function selId(){var
        el=document.getElementById('ol-selected-id');return
        el?el.value:'root';}"
356         "function findNode(id){var
        all=document.querySelectorAll('.ol-node');for(var
        i=0;i<all.length;i++){if(all[i].getAttribute('data-id')===id)return
        all[i];}return null;}"
357         "function ownTitle(node){if(!node||!node.children)return
        null;for(var i=0;i<node.children.length;i++){var
        ch=node.children[i];if(ch.classList&&ch.classList.contains('ol-sum')){return
        ch.querySelector('.ol-title');}}return null;}"
358         "var __selPin=null,__selPinT=null;"
359         "function setSel(id){"
360         "if(!id)return;"
361         "if(__selPin&&id==='root'&&__selPin!=='root')return;"
362         "var node=findNode(id);if(!node)return;"
363         "var
        hid=document.getElementById('ol-selected-id');if(hid)hid.value=id;"
364         "var lab=document.getElementById('ol-selected-label');"
365         "if(lab){var
        t=ownTitle(node);if(!t)t=node.querySelector('.ol-title');"
366
        "lab.textContent=(t&&t.innerText)?t.innerText.trim():id;lab.title=id;}"
367         "var rf=document.getElementById('ol-refile-nid');if(rf)rf.value=id;"
368         "var
        ex=document.getElementById('ol-extract-nid');if(ex)ex.value=id;"
369
        "document.querySelectorAll('.ol-node.is-selected').forEach(function(n){n.classList.remove('is-selected');});"
370         "node.classList.add('is-selected');"
371
        "if(id!=='root'){__selPin=id;if(__selPinT)clearTimeout(__selPinT);__selPinT=setTimeout(function(){__selPin=null;},400);
372         "else{__selPin='root';}"
373         "}"
374         "function postOp(op){var id=selId();"
375         "if(op==='delete'){if(id==='root'){alert('Cannot delete the root
        node.');return;}if(!confirm('Delete this node?'))return;}"
376         "if(op==='archive'&&id==='root'){alert('Cannot archive the root
        node.');return;}"
377         "if(!__ppReady)ppBoot();"
378         "var seq=++__opSeq;var keep=id;__selPin=keep;"
379         "var
        snap=document.getElementById('outline')?document.getElementById('outline').outerHTML:null;"
380         "if(__ppReady){try{var prev=window.pp.apply_op(op,id,slug);"
381         "if(prev&&prev.length){var
        t=document.getElementById('outline');if(t){var
        tmp=document.createElement('div');tmp.innerHTML=prev;var
        neo=tmp.firstChild;if(neo){neo.classList.add('pp-in-transit');t.replaceWith(neo);}}}"
382         "}catch(e){console.warn('pp optimistic failed',e);}}"
383         "var
        q='slug='+encodeURIComponent(slug)+'&op='+encodeURIComponent(op)+'&node-id='+encodeURIComponent(id)+'&op-seq='+seq;"
384         "if(showArch)q+='&show=archived';"
385
        "fetch('/partials/notes-node-op?'+q,{method:'POST',credentials:'same-origin'}).then(function(r){return
        r.text();}).then(function(html){"
386         "if(seq<__opSeq)return;" "var doc=new
        DOMParser().parseFromString(html,'text/html');"
387         "var oobSel=doc.querySelector('#ol-selected-id[hx-swap-oob]');"
388         "var olHTML=doc.querySelector('#outline');"
389         "var cur=document.getElementById('outline');"
390
        "if(olHTML&&cur){if(window.Idiomorph){Idiomorph.morph(cur,olHTML);}else{cur.outerHTML=olHTML.outerHTML;}}"
391         "var selIdVal=oobSel?oobSel.getAttribute('value'):null;"
392         "if(selIdVal){var
        hid=document.getElementById('ol-selected-id');if(hid)hid.value=selIdVal;}"
393         "var target=selIdVal||keep;"
394         "if(findNode(target))setSel(target);else
        if(findNode(keep))setSel(keep);else{__selPin=null;setSel('root');}"
395
        "if(window.htmx){try{htmx.process(document.getElementById('outline'));}catch(e){}}"
396         "}).catch(function(e){"
397         "if(snap){var
        tmp=document.createElement('div');tmp.innerHTML=snap;var
        c=document.getElementById('outline');if(c)c.replaceWith(tmp.firstChild);}"
```

```
        "console.error('notes postOp',e);});}"
        "function pickNode(e){"
        "if(e.target.closest&&e.target.closest('[data-ol-op]'))return null;"
        "if(e.target.closest&&e.target.closest('.ol-actions'))return null;"
        "if(e.target.closest&&e.target.closest('.ol-fold'))return null;"
        "var node=e.target.closest&&e.target.closest('.ol-node');"
        "return(node&&node.dataset&&node.dataset.id)?node:null;}"
        "document.addEventListener('pointerdown',function(e){"
        "if(e.button!==0&&e.button!==undefined)return;"
        "var node=pickNode(e);if(node)setSel(node.dataset.id);},true);"
        "document.addEventListener('click',function(e){"
        "var btn=e.target.closest&&e.target.closest('[data-ol-op]');"

    "if(btn){e.preventDefault();postOp(btn.getAttribute('data-ol-op'));return;}"
        "var node=pickNode(e);if(node)setSel(node.dataset.id);},true);"
        "document.addEventListener('focusin',function(e){"
        "var t=e.target;if(!t||!t.getAttribute)return;"
        "var id=t.getAttribute('data-id');var
    field=t.getAttribute('data-field');"
        "if(id&&(field==='title'||field==='body'))setSel(id);},true);"
        "var
    root=document.getElementById('outline');if(root){root.addEventListener('blur',function(e){"
        "var t=e.target;if(!t.dataset||!t.dataset.field)return;"
        "var
    p='slug='+encodeURIComponent(slug)+'&op=set-'+t.dataset.field+'&node-id='+encodeURIComponent(t.dataset.id);"
        "if(showArch)p+='&show=archived';"
        "p+='&'+t.dataset.field+'='+encodeURIComponent(t.innerText);"

    "fetch('/partials/notes-node-op',{method:'POST',headers:{'Content-Type':'application/x-www-form-urlencoded'},body:p});}
        "document.addEventListener('keydown',function(e){"
        "if(!e.altKey)return;"
        "var k=e.key;var op=null;"
        "if(e.shiftKey&&k==='ArrowLeft')op='promote';"
        "else if(e.shiftKey&&k==='ArrowRight')op='demote';"
        "else if(k==='ArrowUp')op='move-up';"
        "else if(k==='ArrowDown')op='move-down';"
        "else if(k==='Enter')op='add-sibling';"
        "else if(k==='ArrowRight')op='slurp-forward';"
        "else if(k==='ArrowLeft')op='barf-forward';"
        "if(op){e.preventDefault();postOp(op);}"
        "},true);"

    "setSel('root');document.body.addEventListener('htmx:afterSwap',function(){setSel(selId());});})();"))))
(define notes/ui/recent-list
  (lambda
    ()
    "HTML list of recently indexed notes."
    (list/foldl
      (lambda
        (acc row)
        (string-append acc (notes/ui/item-li row)))
      ""
      (notes/doc/index-list))))

(define notes/ui/show
  (lambda
    (slug)
    "Render a note page, or the missing fragment when absent."
    (let
      ((n (notes/doc/get slug))
        (show-archived (notes/doc/show-archived?)))
      (if
        (null? n)
        (notes/ui/missing slug)
        (notes/ui/show-body
          slug
          (notes/doc/title n)
          (notes/doc/outline n)
          (notes/doc/tags n)
          show-archived
          (notes/doc/folder n))))))

(define notes/ui/show-body
```

```
  (lambda
    (slug title outline tags show-archived folder)
    "HTML body of a note page: breadcrumbs, toolbar, and outline."
    (html/join
      (list
        (html/p*
          (html/attr "class" "muted")
          (html/join
            (list
              (html/a "/notes" "Notes")
              (if
                (string-eq (as-str folder) "")
                ""
                (html/join
                  (list
                    " / "
                    (html/a
                      (string-append "/notes?folder=" (as-str folder))
                      (html/text (as-str folder))))))
              " / "
              (html/code (as-str slug)))))
        (html/div
          (html/attr "class" "note-title-row")
          (html/join
            (list
              (html/h1 (html/text title))
              (html/form
                "POST"
                "/notes"
                (html/attrs
                  (list
                    (html/attr "class" "note-del note-del-page")
                    (html/attr "hx-confirm" "Delete this entire note?")))
                (html/join
                  (list
                    (html/hidden "action" "delete")
                    (html/hidden "slug" (as-str slug))
                    (html/button
                      (html/attrs
                        (list
                          (html/attr "type" "submit")
                          (html/attr "class" "secondary")
                          (html/attr "title" "delete note")))
                      "Delete note")))))))
        (html/p*
          (html/attr "class" "muted")
          (string-append "tags: " (as-str tags)))
        (notes/ui/folder-form slug folder)
        (notes/ui/toolbar slug show-archived)
        (outline/html-lite slug outline show-archived)
        (html/div
          (html/attr "class" "roam-backlinks-panel")
          (html/join
            (list (html/h3 "Backlinks") (roam/backlinks-html slug))))
        (html/tag
          "details"
          (html/attr "class" "ol-debug")
          (html/join
            (list
              (html/tag
                "summary"
                (html/attr "class" "muted")
                "Raw JSON (debug)")
              (notes/ui/edit-form slug outline))))
        (notes/ui/outline-script slug show-archived outline)))))

(define notes/ui/tb-btn
  (lambda
    (op label)
    "Toolbar button carrying a data-ol-op operation."
    (html/tag
      "button"
      (html/attrs
```

```
        (list
          (html/attr "type" "button")
          (html/attr "class" "ol-btn")
          (html/attr "title" op)
          (html/attr "data-ol-op" op)))
      label)))

(define notes/ui/toolbar
  (lambda
    (slug show-archived)
    "HTML outline editing toolbar for a note."
    (html/div
      (html/attr "class" "ol-toolbar")
      (html/join
        (list
          (html/span
            (html/attr "class" "ol-toolbar-label")
            "Selected:")
          (html/span (html/attr "id" "ol-selected-label") "root")
          (html/void
            "input"
            (html/attrs
              (list
                (html/attr "type" "hidden")
                (html/attr "id" "ol-selected-id")
                (html/attr "value" "root"))))
          (notes/ui/tb-btn "cycle-todo" "todo")
          (notes/ui/tb-btn "archive" "arch")
          (notes/ui/tb-btn "unarchive" "unarch")
          (notes/ui/tb-btn "add-child" "+kid")
          (notes/ui/tb-btn "add-sibling" "+sib")
          (notes/ui/tb-btn "promote" "<<")
          (notes/ui/tb-btn "demote" ">>")
          (notes/ui/tb-btn "move-up" "up")
          (notes/ui/tb-btn "move-down" "dn")
          (notes/ui/tb-btn "delete" "del")
          (html/form
            "POST"
            "/notes"
            (html/attr "class" "ol-refile")
            (html/join
              (list
                (html/hidden "action" "refile")
                (html/hidden "src_slug" (as-str slug))
                (html/void
                  "input"
                  (html/attrs
                    (list
                      (html/attr "type" "hidden")
                      (html/attr "name" "node_id")
                      (html/attr "id" "ol-refile-nid")
                      (html/attr "value" "root"))))
                (html/select
                  "dst_slug"
                  (html/attr "class" "ol-refile-sel")
                  (notes/refile/options slug))
                (html/button
                  (html/attrs
                    (list
                      (html/attr "type" "submit")
                      (html/attr "class" "ol-btn")
                      (html/attr "title" "refile selected node")))
                  "refile"))))
          (html/form
            "POST"
            "/notes"
            (html/attr "class" "ol-extract")
            (html/join
              (list
                (html/hidden "action" "extract")
                (html/hidden "src_slug" (as-str slug))
                (html/void
                  "input"
```

```
                  (html/attrs
                    (list
                      (html/attr "type" "hidden")
                      (html/attr "name" "node_id")
                      (html/attr "id" "ol-extract-nid")
                      (html/attr "value" "root"))))
                (html/button
                  (html/attrs
                    (list
                      (html/attr "type" "submit")
                      (html/attr "class" "ol-btn")
                      (html/attr
                        "title"
                        "extract selected node into its own note")))
                  "extract"))))
          (if
            show-archived
            (html/a
              (string-append "/notes?slug=" slug)
              "Hide archived")
            (html/a
              (string-append "/notes?slug=" slug "&show=archived")
              "Show archived")))))))

(define notes/ui/path-label
  (lambda
    (path)
    "Human label for a tree path (last segment, underscores as dashes)."
    (let
      ((parts (string-split (as-str path) ".")))
      (if
        (null? parts)
        ""
        (string-replace (notes/light/last parts) "_" "-")))))

(define notes/ui/node-path
  (lambda (n) "Path string of a tree node." (as-str (dict-get n "path"))))

(define notes/ui/node-kind
  (lambda (n) "Kind string of a tree node." (as-str (dict-get n "kind"))))

(define notes/ui/node-title
  (lambda
    (n)
    "Display title of a node (body title/slug or path label)."
    (let
      ((b (dict-get n "body")))
      (let
        ((t (dict-get b "title"))
          (s (dict-get b "slug")))
        (if
          (if (null? t) #t (string-eq (as-str t) ""))
          (if
            (if (null? s) #t (string-eq (as-str s) ""))
            (notes/ui/path-label (notes/ui/node-path n))
            (as-str s))
          (as-str t))))))

(define notes/ui/is-garden-folder?
  (lambda
    (n root)
    "True when node is the .garden folder under root."
    (if
      (not (string-eq (notes/ui/node-kind n) "folder"))
      #f
      (string-eq
        (notes/ui/node-path n)
        (string-append (as-str root) ".garden")))))

(define notes/ui/note-li-from-node
  (lambda
    (n)
    "HTML list item linking a tree node to its note."
```

```
    (let
      ((slug (as-str (dict-get (dict-get n "body") "slug")))
        (title (notes/ui/node-title n)))
      (let
        ((s
          (if (string-eq slug "") (notes/ui/path-label (notes/ui/node-path
     n)) slug)))
          (html/li
            (html/attr "class" "tree-item")
            (html/a
              (string-append "/notes?slug=" s)
              (html/text title)))))))

(define notes/ui/folder-details
  (lambda
    (n)
    "HTML collapsible folder node that lazily loads children."
    (let
      ((path (notes/ui/node-path n))
        (label (notes/ui/node-title n)))
      (html/tag
        "details"
        (html/attrs
          (list
            (html/attr "class" "tree-folder")
            (html/attr "hx-get"
              (string-append
                "/partials/notes-tree?path="
                path))
            (html/attr "hx-trigger" "toggle once")
            (html/attr "hx-target" "find .tree-children")
            (html/attr "hx-swap" "innerHTML")))
        (html/join
          (list
            (html/tag
              "summary"
              (html/attr "class" "tree-folder-header")
              (html/join
                (list
                  (html/span
                    (html/attr "class" "tree-folder-icon")
                    "&#x1F4C1; ")
                  (html/text label))))
            (html/div
              (html/attr "class" "tree-children")
              (html/span
                (html/attr "class" "muted")
                "open to load?")))))))))

(define notes/ui/render-child
  (lambda
    (n root)
    "Render one tree child node, dispatching on its kind."
    (let
      ((k (notes/ui/node-kind n)))
      (if
        (string-eq k "heading")
        ""
        (if
          (notes/ui/is-garden-folder? n root)
          (notes/ui/tree-children-html (notes/ui/node-path n) root)
          (if
            (string-eq k "folder")
            (notes/ui/folder-details n)
            (if
              (string-eq k "note")
              (notes/ui/note-li-from-node n)
              "")))))))

(define notes/ui/tree-children-html
  (lambda
    (path root)
    "HTML for the children of a tree path."
```

```
    (let
      ((kids (tree/list (as-str path) 1)))
      (if
        (null? kids)
        (html/p* (html/attr "class" "muted") "(empty)")
        (html/join
          (list/map
            (lambda (n) (notes/ui/render-child n root))
            kids))))))

(define notes/ui/tree-partial
  (lambda
    ()
    "htmx partial rendering the children of the requested path."
    (let
      ((path (as-str (request-param "path")))
        (root (notes/tree/root)))
      (if
        (string-eq path "")
        (render-html (html/p* (html/attr "class" "muted") "missing path"))
        (render-html
          (html/div
            (html/attr "class" "tree-children-inner")
            (notes/ui/tree-children-html path root)))))))

(define notes/ui/folders-html
  (lambda
    ()
    "HTML notes tree rooted at the garden root."
    (let
      ((root (notes/tree/root)))
      (html/div
        (html/attr "class" "notes-tree")
        (notes/ui/tree-children-html root root)))))

(define notes/ui/index
  (lambda
    ()
    "HTML notes index page: create form plus folder tree."
    (html/join
      (list
        (notes/tree/style)
        (html/h1 "Notes")
        (html/p*
          (html/attr "class" "muted")
          "Outline-first notes. Folders load on open (htmx).")
        (html/p*
          (html/attr "class" "muted")
          (html/join
            (list
              (html/a "/notes?slug=inbox" "Open inbox")
              " ? "
              (html/a "/notes" "All folders"))))
        (notes/ui/create-form)
        (html/h2 "Folders")
        (notes/ui/folders-html)))))
```

188 libs/ntfy/lib.pp

```
;; package ntfy -- push notifications via ntfy topics over http-post
;; Spec: borge/notify.borg (notify.ntfy-delivery)
;; Kernel dep: (http-post url [headers-json [body]]) -- image.outbound-http
;; First consumer: dabar/daily-deliver (borge/dabar.borg daily-scripture)
;;
;; CONFIG: one data row of type ntfy-config (fields dict, read via rows):
;;   url    ntfy base, no trailing slash (e.g. "https://ntfy.sh")
;;   topic  the topic path (a capability: treat like a password)
;;   token  optional bearer for protected topics ("")
;;   site   optional public habitat base used to build Click: targets
;; Create it (admin):
;;   (deftype 'ntfy-config 'url 'topic 'token 'site)
;;   (create 'ntfy-config
     "{\"url\":\"https://ntfy.sh\",\"topic\":\"<topic>\",\"token\":\"\",\"site\":\"https://pricklypear.rocks\"}")
;; Phone side: subscribe the ntfy app (or a UnifiedPush distributor) to
;;   <url>/<topic>. Self-hosting ntfy is a config change, not code.

(define ntfy/json-esc
  (lambda
    (s i acc)
    (if
      (>= i (string-length s))
      acc
      (let
        ((c (substring s i 1)))
        (ntfy/json-esc
          s
          (+ i 1)
          (string-append
            acc
            (cond
              ((string-eq c "\\") "\\\\")
              ((string-eq c "\"") "\\\"")
              ((string-eq c "\n") "\\n")
              ((string-eq c "\r") "\\r")
              ((string-eq c "\t") "\\t")
              (else c))))))))

(define ntfy/json-escape
  (lambda
    (s)
    "Escape a string for embedding in a JSON header value."
    (ntfy/json-esc (as-str s) 0 "")))

(define ntfy/config
  (lambda
    ()
    "Load the ntfy-config row (first row of the type -- there is exactly
     one); User_error if absent."
    (let
      ((rs (rows 'ntfy-config 5)))
      (if
        (null? rs)
        (error "ntfy/send: no ntfy-config row -- create it (admin): (create
     'ntfy-config
     \"{\\\"url\\\":\\\"https://ntfy.sh\\\",\\\"topic\\\":\\\"<topic>\\\",\\\"site\\\":\\\"https://pricklypear.rocks\\\"}\")
        (cdr (car rs))))))

(define ntfy/headers-json
  (lambda
    (title click token)
    "ntfy wire headers: Title, optional Click, optional Authorization."
    (string-append
      "{\"Title\": \""
      (ntfy/json-escape title)
      "\""
      (if
        (string-eq (as-str click) "")
        ""
        (string-append
          ", \"Click\": \""
```

```
          (ntfy/json-escape click)
          "\""))
      (if
        (string-eq (as-str token) "")
        ""
        (string-append
          ", \"Authorization\": \"Bearer "
          (ntfy/json-escape token)
          "\""))
      "}")))

(define ntfy/send/click
  (lambda
    (title body click-url)
    "POST one ntfy message; returns the http-post payload dict (dict-get it
     for ok/status/err/kind). Transient network failures RAISE -- inside
     jobs the runner's attempts/backoff owns retries."
    (let
      ((cfg (ntfy/config)))
      (let
        ((url (as-str (dict-get cfg "url")))
          (topic (as-str (dict-get cfg "topic")))
          (token-f (dict-get cfg "token")))
        (let
          ((token (if (null? token-f) "" (as-str token-f))))
          (http-post
            (string-append url "/" topic)
            (ntfy/headers-json title click-url token)
            (as-str body)))))))

(define ntfy/send
  (lambda
    (title body)
    "Push a notification with no click target (see ntfy/send/click)."
    (ntfy/send/click title body "")))
```

189 libs/ops/code/lib.pp

```
;; package ops/code -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push

(define ops/code/apply-pairs
  (lambda
    (src pairs)
    (if
      (null? pairs)
      src
      (let
        ((pair (car pairs)))
        (ops/code/apply-pairs
          (string-replace
            src
            (as-str (car pair))
            (as-str (car (cdr pair))))
          (cdr pairs))))))

(define ops/code/default-pkgs
  (list
    "bible"
    "workout"
    "welcome"
    "layout"
    "todo"
    "dabar"
    "outline"
    "notes"
    "roam"
    "calendar"
    "flash"
    "chat"
    "signup"
    "bi"
    "contacts"
    "plan"
    "strongs"
    "weather"
    "ref"
    "ai"
    "ui"
    "menu"
    "html"
    "css"
    "ops"))

(define ops/code/define-src
  (lambda
    (name body-src)
    (load-string
      (string-append
        "(define "
        (as-str name)
        "\n  "
        (as-str body-src)
        "\n)"))))

(define ops/code/fix-mangled
  (lambda
    ()
    (list/filter
      (lambda
        (row)
        (not (string-eq (as-str (car (cdr row))) "clean")))
      (list/map ops/code/fix-name (ops/code/mangled-hits)))))

(define ops/code/fix-name
  (lambda
    (name)
    (if
      (or
```

```
        (string-eq (as-str name) "ops/code/mangled-pairs")
        (string-eq (as-str name) "ops/code/mangled-needles"))
      (list name "skip")
      (let
        ((src0 (function-source name)))
        (if
          (null? src0)
          (list name "missing")
          (let
            ((src (as-str src0)))
            (let
              ((new (ops/code/apply-pairs src ops/code/mangled-pairs)))
              (if
                (string-eq src new)
                (list name "clean")
                (begin
                  (ops/code/define-src name new)
                  (list name "fixed")))))))))

(define ops/code/fix-names
  (lambda (names) (list/map ops/code/fix-name names)))

(define ops/code/fix-pkg
  (lambda
    (pkg)
    (list/filter
      (lambda
        (row)
        (not (string-eq (as-str (car (cdr row))) "clean")))
      (list/map
        ops/code/fix-name
        (ops/code/uniq
          (list/foldl
            (lambda
              (acc needle)
              (append acc (ops/code/scan-pkg pkg needle)))
            ()
            ops/code/mangled-needles))))))

(define ops/code/mangled-hits
  (lambda
    ()
    (ops/code/uniq
      (list/foldl
        (lambda
          (acc needle)
          (append acc (ops/code/search-limit needle 200)))
        ()
        ops/code/mangled-needles))))

(define ops/code/mangled-needles
  (list
    "226134"
    "194183"
    "226128"
    "226151"
    "226150"
    "194171"
    "194187"
    "194176"
    "u2190"
    "u2192"
    "u00b7"
    "u2014"
    "u2013"
    "u2026"
    "u2019"
    "u2018"
    "u201c"
    "u201d"
    "u25c0"
    "u25b6"
    "u00ab"
```

```
    "u00bb"
    "u00b0"
    "u2191"
    "u2193"))

(define ops/code/mangled-pairs
  (list
    (list "226134144" "&larr;")
    (list "226134146" "&rarr;")
    (list "226134145" "&uarr;")
    (list "226134147" "&darr;")
    (list "194183" "&middot;")
    (list "226128148" "&mdash;")
    (list "226128147" "&ndash;")
    (list "226128166" "&hellip;")
    (list "226128153" "&rsquo;")
    (list "226128152" "&lsquo;")
    (list "226128156" "&ldquo;")
    (list "226128157" "&rdquo;")
    (list "226151128" "&#9664;")
    (list "226150182" "&#9654;")
    (list "194171" "&laquo;")
    (list "194187" "&raquo;")
    (list "194176" "&deg;")
    (list "u2190" "&larr;")
    (list "u2192" "&rarr;")
    (list "u2191" "&uarr;")
    (list "u2193" "&darr;")
    (list "u00b7" "&middot;")
    (list "u2014" "&mdash;")
    (list "u2013" "&ndash;")
    (list "u2026" "&hellip;")
    (list "u2019" "&rsquo;")
    (list "u2018" "&lsquo;")
    (list "u201c" "&ldquo;")
    (list "u201d" "&rdquo;")
    (list "u25c0" "&#9664;")
    (list "u25b6" "&#9654;")
    (list "u00ab" "&laquo;")
    (list "u00bb" "&raquo;")
    (list "u00b0" "&deg;")))

(define ops/code/scan-pkg
  (lambda
    (pkg needle)
    (list/filter
      (lambda
        (name)
        (let
          ((src (function-source name)))
          (if
            (null? src)
            #f
            (string-contains? (as-str src) (as-str needle)))))
      (package-bindings pkg))))

(define ops/code/scan-pkgs
  (lambda
    (pkgs needle limit)
    (let
      ((lim (parse-int (as-str limit))))
      (list/foldl
        (lambda
          (acc pkg)
          (if
            (>= (count acc) lim)
            acc
            (append acc (ops/code/scan-pkg pkg needle))))
        ()
        pkgs))))

(define ops/code/search
  (lambda (needle) (ops/code/search-limit needle 500)))
```

```
218
219 (define ops/code/search-limit
220   (lambda
221     (needle limit)
222     (ops/code/scan-pkgs ops/code/default-pkgs needle limit)))
223
224 (define ops/code/uniq
225   (lambda
226     (xs)
227     (list/foldl
228       (lambda
229         (acc x)
230         (if (null? (member x acc)) (cons x acc) acc))
231       ()
232       xs)))
```

190 libs/ops/lib.pp

```
;; package ops -- code intel + mangled-UTF repair
;;
;; Kernel (after deploy of this branch's image):
;;   (find-code "needle")           ; Postgres ILIKE on function body JSON
;;   (find-code "needle" 100)
;;
;; Lisp (works now):
;;   (ops/code/search "226134")
;;   (ops/code/fix-pkg "bible")
;;   (ops/code/fix-mangled)
;;   (ops/code/fix-names (list "bible/nav-links"))

(define ops/code/default-pkgs
  (list "bible" "workout" "welcome" "layout" "todo" "dabar" "outline"
        "notes" "roam" "calendar" "flash" "chat" "signup" "bi" "contacts"
        "plan" "strongs" "weather" "ref" "ai" "ui" "menu" "html" "css"
    "ops"))

(define ops/code/scan-pkg
  (lambda (pkg needle)
    "Filter package-bindings of pkg whose source contains needle."
    (list/filter
      (lambda (name)
        (let ((src (function-source name)))
          (if (null? src) #f
              (string-contains? (as-str src) (as-str needle)))))
      (package-bindings pkg))))

(define ops/code/scan-pkgs
  (lambda (pkgs needle limit)
    "Scan pkgs for needle source matches, stopping at limit."
    (let ((lim (parse-int (as-str limit))))
      (list/foldl
        (lambda (acc pkg)
          (if (>= (count acc) lim) acc
              (append acc (ops/code/scan-pkg pkg needle))))
        ()
        pkgs))))

(define ops/code/search-limit
  (lambda (needle limit)
    "Search default packages for needle, capped at limit hits."
    (ops/code/scan-pkgs ops/code/default-pkgs needle limit)))

(define ops/code/search
  (lambda (needle) "Search default packages for needle, up to 500 hits."
    (ops/code/search-limit needle 500)))

(define ops/code/apply-pairs
  (lambda (src pairs)
    "Replace every mangled needle in src with its pair; returns new src."
    (if (null? pairs) src
        (let ((pair (car pairs)))
          (ops/code/apply-pairs
            (string-replace src (as-str (car pair)) (as-str (car (cdr
    pair))))
            (cdr pairs))))))

(define ops/code/mangled-pairs
  (list
    (list "226134144" "&larr;")
    (list "226134146" "&rarr;")
    (list "226134145" "&uarr;")
    (list "226134147" "&darr;")
    (list "194183" "&middot;")
    (list "226128148" "&mdash;")
    (list "226128147" "&ndash;")
    (list "226128166" "&hellip;")
    (list "226128153" "&rsquo;")
    (list "226128152" "&lsquo;")
    (list "226128156" "&ldquo;")
```

```
    (list "226128157" "&rdquo;")
    (list "226151128" "&#9664;")
    (list "226150182" "&#9654;")
    (list "194171" "&laquo;")
    (list "194187" "&raquo;")
    (list "194176" "&deg;")
    (list "u2190" "&larr;")
    (list "u2192" "&rarr;")
    (list "u2191" "&uarr;")
    (list "u2193" "&darr;")
    (list "u00b7" "&middot;")
    (list "u2014" "&mdash;")
    (list "u2013" "&ndash;")
    (list "u2026" "&hellip;")
    (list "u2019" "&rsquo;")
    (list "u2018" "&lsquo;")
    (list "u201c" "&ldquo;")
    (list "u201d" "&rdquo;")
    (list "u25c0" "&#9664;")
    (list "u25b6" "&#9654;")
    (list "u00ab" "&laquo;")
    (list "u00bb" "&raquo;")
    (list "u00b0" "&deg;")))

(define ops/code/mangled-needles
  (list "226134" "194183" "226128" "226151" "226150"
        "194171" "194187" "194176"
        "u2190" "u2192" "u00b7" "u2014" "u2013" "u2026"
        "u2019" "u2018" "u201c" "u201d" "u25c0" "u25b6"
        "u00ab" "u00bb" "u00b0" "u2191" "u2193"))

(define ops/code/uniq
  (lambda (xs)
    "Remove duplicate elements from xs, keeping first occurrence order."
    (list/foldl
      (lambda (acc x)
        (if (null? (member x acc)) (cons x acc) acc))
      ()
      xs)))

(define ops/code/define-src
  (lambda (name body-src)
    "Re-define a function from a body source string via load-string."
    (load-string
      (string-append
        "(define "
        (as-str name)
        "\n  "
        (as-str body-src)
        "\n)"))))

(define ops/code/mangled-hits
  (lambda ()
    "Collect unique function names whose source contains a mangled code
     point."
    (ops/code/uniq
      (list/foldl
        (lambda (acc needle)
          (append acc (ops/code/search-limit needle 200)))
        ()
        ops/code/mangled-needles))))

(define ops/code/fix-name
  (lambda (name)
    "Repair one function's mangled sequences; skips pair/needle tables."
    (if
      (or
        (string-eq (as-str name) "ops/code/mangled-pairs")
        (string-eq (as-str name) "ops/code/mangled-needles"))
      (list name "skip")
      (let ((src0 (function-source name)))
        (if (null? src0)
            (list name "missing")
```

```
            (let ((src (as-str src0)))
              (let ((new (ops/code/apply-pairs src ops/code/mangled-pairs)))
                (if (string-eq src new)
                    (list name "clean")
                    (begin
                      (ops/code/define-src name new)
                      (list name "fixed")))))))))

(define ops/code/fix-mangled
  (lambda ()
    "Return rows for every function name that still needs a repair."
    (list/filter
      (lambda (row)
        (not (string-eq (as-str (car (cdr row))) "clean")))
      (list/map ops/code/fix-name (ops/code/mangled-hits)))))

(define ops/code/fix-names
  (lambda (names) "Repair each of the given function names." (list/map
     ops/code/fix-name names)))

(define ops/code/fix-pkg
  (lambda (pkg)
    "Repair all mangled functions in one package; returns dirty rows."
    (list/filter
      (lambda (row)
        (not (string-eq (as-str (car (cdr row))) "clean")))
      (list/map
        ops/code/fix-name
        (ops/code/uniq
          (list/foldl
            (lambda (acc needle)
              (append acc (ops/code/scan-pkg pkg needle)))
            ()
            ops/code/mangled-needles))))))
```

191 libs/outline/lib.pp

```
;; package outline -- exported from live branch admin/personal
;; Sync: scripts/pp_sync.py pull|push

(define outline/action-btn
  (lambda
    (slug nid op label class show-archived)
    "Render a node-operation button with an htmx post."
    (html/tag
      "button"
      (html/attrs
        (list
          (html/attr "class" class)
          (html/attr "type" "button")
          (html/attr "title" op)
          (html/attr
            "hx-post"
            (string-append
              "/partials/notes-node-op?slug="
              slug
              "&op="
              op
              "&node-id="
              nid
              (if show-archived "&show=archived" "")))
          (html/attr "hx-target" "#outline")
          (html/attr "hx-swap" "outerHTML")))
      label)))

(define outline/actions
  (lambda
    (slug nid todo show-archived)
    "Render the per-node actions dropdown menu."
    (html/tag
      "details"
      (html/attr "class" "ol-actions")
      (html/join
        (list
          (html/tag
            "summary"
            (html/attr "class" "ol-actions-sum")
            (html/text "???"))
          (html/div
            (html/attr "class" "ol-actions-panel")
            (html/join
              (list
                (outline/action-btn
                  slug
                  nid
                  "cycle-todo"
                  "todo"
                  "ol-btn"
                  show-archived)
                (if
                  (outline/archived? todo)
                  (outline/action-btn
                    slug
                    nid
                    "unarchive"
                    "unarch"
                    "ol-btn"
                    show-archived)
                  (outline/action-btn
                    slug
                    nid
                    "archive"
                    "arch"
                    "ol-btn"
                    show-archived))
                (outline/action-btn
                  slug
                  nid
```

```
                  "add-child"
                  "+kid"
                  "ol-btn"
                  show-archived)
                (outline/action-btn
                  slug
                  nid
                  "add-sibling"
                  "+sib"
                  "ol-btn"
                  show-archived)
                (outline/action-btn
                  slug
                  nid
                  "promote"
                  "<<"
                  "ol-btn"
                  show-archived)
                (outline/action-btn
                  slug
                  nid
                  "demote"
                  ">>"
                  "ol-btn"
                  show-archived)
                (outline/action-btn
                  slug
                  nid
                  "move-up"
                  "up"
                  "ol-btn"
                  show-archived)
                (outline/action-btn
                  slug
                  nid
                  "move-down"
                  "dn"
                  "ol-btn"
                  show-archived)
                (outline/extra-actions slug nid)
                (html/tag
                  "button"
                  (html/attrs
                    (list
                      (html/attr "class" "ol-btn ol-del")
                      (html/attr "type" "button")
                      (html/attr "title" "delete")
                      (html/attr
                        "hx-post"
                        (string-append
                          "/partials/notes-node-op?slug="
                          slug
                          "&op=delete&node-id="
                          nid
                          (if show-archived "&show=archived" "")))
                      (html/attr "hx-target" "#outline")
                      (html/attr "hx-swap" "outerHTML")
                      (html/attr "hx-confirm" "Delete this node?")))
                  "del")))))))))

(define outline/add-child
  (lambda
    (n child)
    "Prepend child to node n's kids list."
    (outline/set-kids n (cons child (outline/kids n)))))

(define outline/add-child-by-id
  (lambda
    (tree id child)
    "Add child to the node with id, returning a new tree."
    (outline/update
      tree
      id
```

```
      (lambda (n) (outline/add-child n child)))))

(define outline/archive
  (lambda (n) "Mark a node as ARCHIVED." (outline/set-todo n "ARCHIVED")))

(define outline/archived?
  (lambda (s) "True when the todo string is ARCHIVED." (string-eq (as-str
     s) "ARCHIVED")))

(define outline/body
  (lambda (n) "Return the body string of a node." (car (cdr (cdr n)))))

(define outline/cycle-todo
  (lambda
    (n)
    "Advance a node's todo state, leaving archived untouched."
    (if
      (outline/archived? (outline/todo n))
      n
      (outline/set-todo
        n
        (outline/todo-next (outline/todo n))))))

(define outline/decode
  (lambda
    (s)
    "Parse a JSON outline string into a tree, defaulting on error."
    (if
      (null? s)
      (outline/node "root" "Untitled")
      (if
        (string-eq (as-str s) "")
        (outline/node "root" "Untitled")
        (let
          ((parsed (json-parse (as-str s))))
          (if
            (outline/valid? parsed)
            parsed
            (outline/node "root" "Untitled")))))))

(define outline/delete
  (lambda
    (tree id)
    "Return a copy of tree with the node id removed."
    (if
      (string-eq (outline/id tree) id)
      tree
      (outline/set-kids
        tree
        (outline/delete-from-kids (outline/kids tree) id)))))

(define outline/delete-from-kids
  (lambda
    (kids id)
    "Remove the node id from a kids list, recursing into children."
    (if
      (null? kids)
      (list)
      (if
        (string-eq (outline/id (car kids)) id)
        (cdr kids)
        (cons
          (outline/delete (car kids) id)
          (outline/delete-from-kids (cdr kids) id))))))

(define outline/demote
  (lambda
    (tree id)
    "Return a copy of tree with node id demoted one level."
    (let
      ((tid (as-str id)))
      (if
        (string-eq (outline/id tree) tid)
```

```
        tree
        (outline/set-kids
          tree
          (outline/demote-kids (outline/kids tree) tid))))))

(define outline/demote-kids
  (lambda
    (kids id)
    "Demote node id within a kids list, shifting it under its sibling."
    (if
      (null? kids)
      (list)
      (if
        (null? (cdr kids))
        (list (outline/demote (car kids) id))
        (if
          (string-eq (outline/id (car (cdr kids))) id)
          (let
            ((prev (car kids))
              (node (car (cdr kids)))
              (rest (cdr (cdr kids))))
            (cons
              (outline/set-kids
                prev
                (append (outline/kids prev) (list node)))
              rest))
          (cons
            (outline/demote (car kids) id)
            (outline/demote-kids (cdr kids) id)))))))

(define outline/detach
  (lambda
    (tree id)
    "Return the detached node and the tree without it; () when top-level."
    (let
      ((tid (as-str id)))
      (if
        (string-eq (outline/id tree) tid)
        ()
        (let
          ((pair (outline/detach-in-kids (outline/kids tree) tid)))
          (if
            (null? pair)
            ()
            (list
              (car pair)
              (outline/set-kids tree (car (cdr pair)))))))))

(define outline/detach-direct
  (lambda
    (n id)
    "Detach a direct child id from node n; () when absent."
    (let
      ((pair (outline/take-from-list (outline/kids n) id)))
      (if
        (null? pair)
        ()
        (list (car pair) (outline/set-kids n (car (cdr pair))))))))

(define outline/detach-in-kids
  (lambda
    (kids id)
    "Detach node id from a kids list, recursing into children."
    (if
      (null? kids)
      ()
      (if
        (string-eq (outline/id (car kids)) id)
        (list (car kids) (cdr kids))
        (let
          ((sub (outline/detach (car kids) id)))
          (if
            (null? sub)
```

```
            (let
              ((rest (outline/detach-in-kids (cdr kids) id)))
              (if
                (null? rest)
                ()
                (list (car rest) (cons (car kids) (car (cdr rest))))))
            (list (car sub) (cons (car (cdr sub)) (cdr kids)))))))))

(define outline/encode
  (lambda (root) "Serialise a tree to a JSON string." (json-emit root)))

(define outline/extra-actions
  (lambda (slug nid) "Render the extra refile actions for a node."
    (notes/refile/extract-form slug nid)))

(define outline/find
  (lambda
    (tree id)
    "Find the node with id in tree; () when absent."
    (if
      (string-eq (outline/id tree) id)
      tree
      (outline/find-in-kids (outline/kids tree) id))))

(define outline/find-in-kids
  (lambda
    (kids id)
    "Find node id within a kids list; () when absent."
    (if
      (null? kids)
      ()
      (let
        ((found (outline/find (car kids) id)))
        (if
          (null? found)
          (outline/find-in-kids (cdr kids) id)
          found)))))

(define outline/fresh-id
  (lambda
    ()
    "Generate a cheap unique node id from the empty list length."
    (string-append
      "n"
      (as-str (string-length (as-str (list)))))))

(define outline/html
  (lambda
    (slug root show-archived)
    "Render the full outline HTML with per-node actions."
    (html/div
      (html/attr "id" "outline")
      (outline/html-node slug root show-archived #t))))

(define outline/html-lite
  (lambda
    (slug root show-archived)
    "Render the outline HTML without per-node actions."
    (html/div
      (html/attr "id" "outline")
      (outline/html-node slug root show-archived #f))))

(define outline/html-node
  (lambda (slug n show-archived with-actions)
    "Render one node and its children recursively."
    (let ((nid (outline/id n))
          (todo (outline/todo n))
          (body (outline/body n))
          (kids (outline/kids n)))
      (let ((is-ref (bible/ref-body? body)))
        (html/tag
          "details"
          (html/attrs
```

```
            (list
              (html/attr
                "class"
                (if (string-eq (as-str todo) "")
                    "ol-node"
                    (string-append "ol-node is-todo st-" (as-str todo))))
              (html/attr "open" "open")
              (html/attr "data-id" nid)))
          (html/join
            (list
              (html/tag
                "summary"
                (html/attr "class" "ol-sum")
                (html/join
                  (list
                    (html/span (html/attr "class" "ol-fold") "")
                    (outline/todo-badge todo)
                    (html/span
                      (html/attrs
                        (list
                          (html/attr "class" "ol-title")
                          (html/attr "contenteditable" "true")
                          (html/attr "data-id" nid)
                          (html/attr "data-field" "title")))
                      (html/text (outline/title n))))))
              (if with-actions
                  (outline/actions slug nid todo show-archived)
                  "")
              (if (string-eq body "")
                  ""
                  (html/div
                    (html/attrs
                      (if is-ref
                          (list
                            (html/attr "class" "ol-body ol-body-ref")
                            (html/attr "data-id" nid)
                            (html/attr "data-bible-ref" "1"))
                          (list
                            (html/attr "class" "ol-body")
                            (html/attr "contenteditable" "true")
                            (html/attr "data-id" nid)
                            (html/attr "data-field" "body"))))
                    (outline/render-body body)))
              (html/div
                (html/attr "class" "ol-kids")
                (html/join
                  (list/map
                    (lambda (k)
                      (outline/html-node slug k show-archived with-actions))
                    kids)))))))))

(define outline/id
  (lambda (n) "Return the id of a node." (car n)))

(define outline/insert-sib-list
  (lambda
    (kids id new-node)
    "Insert new-node after the sibling with id in a kids list."
    (if
      (null? kids)
      (list)
      (if
        (string-eq (outline/id (car kids)) id)
        (cons (car kids) (cons new-node (cdr kids)))
        (cons
          (outline/insert-sibling-kids (car kids) id new-node)
          (outline/insert-sib-list (cdr kids) id new-node))))))

(define outline/insert-sibling
  (lambda
    (tree id new-node)
    "Return a copy of tree with new-node inserted after node id."
    (outline/insert-sibling-kids tree id new-node)))
```

```
435
436 (define outline/insert-sibling-kids
437   (lambda
438     (tree id new-node)
439     "Append new-node after node id, or append when tree itself matches."
440     (if
441       (string-eq (outline/id tree) id)
442       (outline/set-kids
443         tree
444         (append (outline/kids tree) (list new-node)))
445       (outline/set-kids
446         tree
447         (outline/insert-sib-list
448           (outline/kids tree)
449           id
450           new-node)))))
451
452 (define outline/kids
453   (lambda (n) "Return the kids list of a node." (car (cdr (cdr (cdr (cdr
      n)))))))
454
455 (define outline/move-down
456   (lambda (tree id) "Move node id down one position in its sibling list."
      (outline/move-down-kids tree id)))
457
458 (define outline/move-down-kids
459   (lambda
460     (tree id)
461     "Move a child down among its siblings in the tree."
462     (if
463       (string-eq (outline/id tree) (as-str id))
464       tree
465       (outline/set-kids
466         tree
467         (outline/swap-next (outline/kids tree) (as-str id))))))
468
469 (define outline/move-up
470   (lambda (tree id) "Move node id up one position in its sibling list."
      (outline/move-up-kids tree id)))
471
472 (define outline/move-up-kids
473   (lambda
474     (tree id)
475     "Move a child up among its siblings in the tree."
476     (if
477       (string-eq (outline/id tree) (as-str id))
478       tree
479       (outline/set-kids
480         tree
481         (outline/swap-prev (outline/kids tree) (as-str id))))))
482
483 (define outline/node
484   (lambda
485     (id title)
486     "Build an empty outline node with the given id and title."
487     (list (as-str id) (as-str title) "" "" (list))))
488
489 (define outline/node*
490   (lambda
491     (id title body todo kids)
492     "Build a full outline node with all fields."
493     (list
494       (as-str id)
495       (as-str title)
496       (as-str body)
497       (as-str todo)
498       kids)))
499
500 (define outline/promote
501   (lambda
502     (tree id)
503     "Return a copy of tree with node id promoted one level."
504     (let
```

```
      ((tid (as-str id)))
      (if
        (string-eq (outline/id tree) tid)
        tree
        (outline/set-kids
          tree
          (outline/promote-kids (outline/kids tree) tid))))))

(define outline/promote-kids
  (lambda
    (kids id)
    "Pull node id up past its siblings across a kids list."
    (outline/promote-lift
      (list/map (lambda (k) (outline/promote k id)) kids)
      id)))

(define outline/promote-lift
  (lambda
    (kids id)
    "Lift the node id out of each sibling's kids, placing it earlier."
    (if
      (null? kids)
      (list)
      (let
        ((pair (outline/detach-direct (car kids) id)))
        (if
          (null? pair)
          (cons (car kids) (outline/promote-lift (cdr kids) id))
          (cons (car (cdr pair)) (cons (car pair) (cdr kids)))))))))

(define outline/render-body
  (lambda (body)
    "Render a node body block, preferring bible then roam renderers."
    (let
      ((br (bible/render-body body)))
      (if (null? br) (roam/render-body body) br))))

(define outline/set-body
  (lambda
    (n body)
    "Return a copy of node n with a new body."
    (outline/node*
      (outline/id n)
      (outline/title n)
      body
      (outline/todo n)
      (outline/kids n))))

(define outline/set-kids
  (lambda
    (n kids)
    "Return a copy of node n with a new kids list."
    (outline/node*
      (outline/id n)
      (outline/title n)
      (outline/body n)
      (outline/todo n)
      kids)))

(define outline/set-title
  (lambda
    (n title)
    "Return a copy of node n with a new title."
    (outline/node*
      (outline/id n)
      title
      (outline/body n)
      (outline/todo n)
      (outline/kids n))))

(define outline/set-todo
  (lambda
    (n todo)
```

```
    "Return a copy of node n with a new todo state."
    (outline/node*
      (outline/id n)
      (outline/title n)
      (outline/body n)
      todo
      (outline/kids n))))

(define outline/swap-next
  (lambda
    (kids id)
    "Swap node id downward with its next sibling in a kids list."
    (if
      (null? kids)
      (list)
      (if
        (string-eq (outline/id (car kids)) id)
        (if
          (null? (cdr kids))
          kids
          (cons
            (car (cdr kids))
            (cons (car kids) (cdr (cdr kids)))))
        (cons
          (outline/move-down-kids (car kids) id)
          (outline/swap-next (cdr kids) id))))))

(define outline/swap-prev
  (lambda
    (kids id)
    "Swap node id upward with its previous sibling in a kids list."
    (if
      (null? kids)
      (list)
      (if
        (null? (cdr kids))
        kids
        (if
          (string-eq (outline/id (car (cdr kids))) id)
          (cons
            (car (cdr kids))
            (cons (car kids) (cdr (cdr kids))))
          (cons
            (outline/move-up-kids (car kids) id)
            (outline/swap-prev (cdr kids) id)))))))

(define outline/take-from-list
  (lambda
    (kids id)
    "Split off node id from a kids list, returning (node rest)."
    (if
      (null? kids)
      ()
      (if
        (string-eq (outline/id (car kids)) (as-str id))
        (list (car kids) (cdr kids))
        (let
          ((rest (outline/take-from-list (cdr kids) id)))
          (if
            (null? rest)
            ()
            (list (car rest) (cons (car kids) (car (cdr rest)))))))))

(define outline/title
  (lambda (n) "Return the title of a node." (car (cdr n))))

(define outline/todo
  (lambda (n) "Return the todo state of a node." (car (cdr (cdr (cdr n))))))

(define outline/todo-badge
  (lambda
    (todo)
    "Render the todo state badge for a node, or empty when none."
```

```
    (let
      ((t (as-str todo)))
      (if
        (string-eq t "")
        ""
        (html/span
          (html/attr "class" (string-append "ol-todo st-" t))
          (html/text t))))))

(define outline/todo-next
  (lambda
    (cur)
    "Return the next todo state after cur in the state list."
    (outline/todo-next-in outline/todo-states (as-str cur))))

(define outline/todo-next-in
  (lambda
    (xs cur)
    "Find the state following cur in the given state list."
    (if
      (null? xs)
      "TODO"
      (if
        (string-eq (as-str (car xs)) cur)
        (if
          (null? (cdr xs))
          (as-str (car outline/todo-states))
          (as-str (car (cdr xs))))
        (outline/todo-next-in (cdr xs) cur)))))

(define outline/todo-open?
  (lambda
    (s)
    "True when a todo state is an active (not closed) state."
    (let
      ((t (as-str s)))
      (if
        (string-eq t "")
        #f
        (if
          (string-eq t "DONE")
          #f
          (if
            (string-eq t "CANCELLED")
            #f
            (if (string-eq t "ARCHIVED") #f #t)))))))

(define outline/todo-states
  (list "" "TODO" "DONE"))

(define outline/todos
  (lambda (tree) "Collect all non-empty todo entries in a tree."
    (outline/todos-acc tree (list))))

(define outline/todos-acc
  (lambda
    (n acc)
    "Accumulate todo entries while walking the tree."
    (let
      ((todo (as-str (outline/todo n))))
      (let
        ((acc1
          (if
            (string-eq todo "")
            acc
            (cons (list (outline/id n) (outline/title n) todo) acc))))
        (list/foldl
          (lambda (a k) (outline/todos-acc k a))
          acc1
          (outline/kids n))))))

(define outline/unarchive
```

```
  (lambda (n) "Un-archive a node, resetting its todo to TODO."
    (outline/set-todo n "TODO")))

(define outline/update
  (lambda
    (tree id f)
    "Return a copy of tree with f applied to the node id."
    (if
      (string-eq (outline/id tree) id)
      (f tree)
      (outline/set-kids
        tree
        (outline/update-kids (outline/kids tree) id f)))))

(define outline/update-kids
  (lambda
    (kids id f)
    "Apply f to node id throughout a kids list."
    (if
      (null? kids)
      (list)
      (cons
        (outline/update (car kids) id f)
        (outline/update-kids (cdr kids) id f)))))

(define outline/valid?
  (lambda
    (n)
    "True when n is a proper 5-field outline node."
    (if (null? n) () (if (pair? n) (= (count n) 5) ()))))


(define outline/css
  (lambda
    ()
    "Render the outline component stylesheet (extracted from ui/theme)."
    (css/sheet
      (list
        (css/rule
          "#ol-selected-label"
          (list
            "font-size"
            ".85rem"
            "max-width"
            "12rem"
            "overflow"
            "hidden"
            "text-overflow"
            "ellipsis"
            "white-space"
            "nowrap"
            "margin-right"
            ".35rem"))
        (css/rule ".ol-root" (list "margin" ".5rem 0"))
        (css/rule
          ".ol-node"
          (list
            "position"
            "relative"
            "margin"
            ".25rem 0 .25rem .35rem"
            "border-left"
            "2px solid #808080"
            "padding-left"
            ".35rem"))
        (css/rule
          ".ol-sum"
          (list
            "cursor"
            "pointer"
            "font-weight"
            "700"
            "display"
```

```
            "flex"
            "align-items"
            "center"
            "gap"
            ".35rem"
            "flex-wrap"
            "nowrap"
            "min-width"
            "0"
            "padding-right"
            "2.5rem"))
        (css/rule
          ".ol-node.is-selected > .ol-sum"
          (list
            "background"
            "var(--sel-bg)"
            "outline"
            "2px solid #000"
            "box-shadow"
            "inset 3px 0 0 var(--sel-accent)"))
        (css/rule
          ".ol-fold"
          (list
            "font-size"
            ".75em"
            "flex"
            "0 0 auto"
            "display"
            "inline-flex"
            "align-items"
            "center"
            "justify-content"
            "center"
            "user-select"
            "none"
            "cursor"
            "pointer"
            "line-height"
            "1"
            "min-width"
            "1.6rem"
            "min-height"
            "1.6rem"
            "padding"
            ".2rem"
            "margin"
            "0 .05rem 0 -.1rem"
            "color"
            "var(--muted)"))
        (css/rule
          ".ol-fold:hover"
          (list
            "background"
            "var(--panel)"
            "border"
            "1px solid #808080"
            "padding"
            "calc(.2rem - 1px)"))
        (css/rule
          "details.ol-node > .ol-sum .ol-fold::before"
          (list "content" "\"v\""))
        (css/rule
          "details.ol-node:not([open]) > .ol-sum .ol-fold::before"
          (list "content" "\">\""))
        (css/rule
          ".ol-todo"
          (list
            "display"
            "inline-block"
            "flex"
            "0 0 auto"
            "width"
            "auto"
```

```
            "margin"
            "0 .15rem 0 0"
            "padding"
            ".05rem .3rem"
            "border"
            "1px solid #666"
            "background"
            "transparent"
            "font-size"
            ".7rem"
            "font-weight"
            "700"
            "line-height"
            "1.3"
            "letter-spacing"
            ".02em"
            "color"
            "var(--muted)"
            "vertical-align"
            "middle"))
        (css/rule
          ".ol-todo.st-TODO"
          (list
            "border-color"
            "var(--pending-border)"
            "color"
            "var(--pending-ink)"
            "background"
            "var(--pending-bg)"))
        (css/rule
          ".ol-todo.st-DONE"
          (list
            "border-color"
            "var(--done-ink)"
            "color"
            "var(--done-ink)"
            "background"
            "var(--done-bg)"))
        (css/rule
          ".ol-todo.st-ARCHIVED"
          (list
            "border-color"
            "var(--archived-border)"
            "color"
            "var(--archived-ink)"
            "background"
            "var(--archived-bg)"))
        (css/rule
          ".ol-node.st-DONE > .ol-sum .ol-title"
          (list
            "text-decoration"
            "line-through"
            "color"
            "var(--done-muted)"
            "font-weight"
            "500"))
        (css/rule
          ".ol-node.st-ARCHIVED > .ol-sum .ol-title"
          (list
            "color"
            "var(--archived-ink)"
            "font-style"
            "italic"
            "font-weight"
            "500"))
        (css/rule
          ".ol-node.st-ARCHIVED"
          (list "opacity" "0.72"))
        (css/rule
          ".ol-title"
          (list
            "outline"
            "none"
```

```
            "padding"
            ".15rem .25rem"
            "border-radius"
            "0"
            "flex"
            "1 1 auto"
            "min-width"
            "0"
            "word-break"
            "break-word"
            "white-space"
            "normal"
            "line-height"
            "1.35"
            "cursor"
            "text"
            "font-weight"
            "700"))
        (css/rule
          ".ol-title:focus"
          (list
            "background"
            "var(--panel)"
            "border"
            "1px solid #000"))
        (css/rule
          ".ol-body"
          (list
            "margin"
            ".25rem 0 .35rem"
            "white-space"
            "pre-wrap"
            "font-size"
            ".95rem"
            "outline"
            "none"
            "padding"
            ".25rem"
            "min-width"
            "0"
            "word-break"
            "break-word"))
        (css/rule
          ".ol-body:focus"
          (list
            "background"
            "var(--panel)"
            "border"
            "1px solid #000"))
        (css/rule
          ".ol-kids"
          (list "margin-left" ".15rem" "min-width" "0"))
        (css/rule
          ".ol-toolbar"
          (list
            "display"
            "flex"
            "flex-wrap"
            "wrap"
            "align-items"
            "center"
            "gap"
            ".3rem"
            "margin"
            "0 0 .65rem"
            "padding"
            ".35rem .5rem"
            "background"
            "var(--panel)"
            "border"
            "2px solid #000"
            "position"
            "sticky"
```

```
            "top"
            "0"
            "z-index"
            "40"
            "box-shadow"
            "0 2px 0 #808080"))
          (css/rule
            ".ol-toolbar-label"
            (list
              "font-size"
              ".8rem"
              "color"
              "var(--muted)"
              "margin-right"
              ".2rem"
              "min-width"
              "auto"
              "font-weight"
              "700"))
          (css/rule
            ".ol-toolbar .ol-btn"
            (list "min-height" "2rem" "padding" ".2rem .45rem"))
          (css/rule
            ".ol-toolbar form.ol-refile"
            (list
              "margin"
              "0"
              "display"
              "inline-flex"
              "width"
              "auto"))
          (css/rule
            "details.ol-actions"
            (list
              "display"
              "none"
              "position"
              "absolute"
              "top"
              "0"
              "right"
              "0"
              "z-index"
              "5"
              "margin"
              "0"))
          (css/rule
            "details.ol-actions > summary.ol-actions-sum"
            (list
              "display"
              "inline-flex"
              "align-items"
              "center"
              "justify-content"
              "center"
              "list-style"
              "none"
              "cursor"
              "pointer"
              "user-select"
              "none"
              "width"
              "auto"
              "min-width"
              "2.25rem"
              "min-height"
              "2.25rem"
              "padding"
              ".2rem .35rem"
              "border"
              "1px solid #666"
              "background"
              "var(--chrome)"
```

```
            "font-weight"
            "700"
            "color"
            "var(--ink)"
            "text-align"
            "center"
            "line-height"
            "1.2"))
        (css/rule
          "details.ol-actions > summary.ol-actions-sum::-webkit-details-marker"
          (list "display" "none"))
        (css/rule
          "details.ol-actions > summary.ol-actions-sum:hover, details.ol-actions[open] > summary.ol-actions-sum"
          (list
            "background"
            "var(--panel)"
            "border-color"
            "var(--ink)"))
        (css/rule
          ".ol-actions-panel"
          (list
            "display"
            "none"
            "position"
            "absolute"
            "right"
            "0"
            "top"
            "100%"
            "margin-top"
            "2px"
            "flex-direction"
            "column"
            "flex-wrap"
            "nowrap"
            "align-items"
            "stretch"
            "min-width"
            "7rem"
            "padding"
            ".2rem"
            "background"
            "var(--panel)"
            "border"
            "1px solid #000"
            "z-index"
            "20"))
        (css/rule
          "details.ol-actions[open] > .ol-actions-panel"
          (list "display" "flex"))
        (css/rule
          "details.ol-actions[open] > .ol-actions-panel .ol-btn"
          (list
            "width"
            "100%"
            "text-align"
            "left"
            "min-height"
            "2rem"))
        (css/rule
          ".ol-btn"
          (list
            "font-size"
            ".75rem"
            "padding"
            ".2rem .4rem"
            "min-height"
            "1.8rem"
            "width"
            "auto"
            "flex"
```

```
            "0 0 auto"
            "margin"
            "0"
            "border"
            "1px solid #000"
            "background"
            "var(--bg)"
            "cursor"
            "pointer"
            "line-height"
            "1.2"))
          (css/rule
            ".ol-btn:hover"
            (list "background" "var(--panel)"))
          (css/rule ".ol-del" (list "color" "var(--err)"))
          (css/rule
            "form.ol-refile"
            (list
              "display"
              "flex"
              "flex-wrap"
              "wrap"
              "gap"
              ".25rem"
              "align-items"
              "center"
              "margin"
              ".25rem 0 0"
              "width"
              "auto"
              "max-width"
              "100%"))
          (css/rule
            "form.ol-refile .ol-refile-sel"
            (list
              "flex"
              "1 1 8rem"
              "min-width"
              "6rem"
              "max-width"
              "14rem"
              "font-size"
              ".75rem"
              "padding"
              ".2rem"))
          (css/rule
            "form.ol-refile button"
            (list
              "width"
              "auto"
              "min-width"
              "0"
              "font-size"
              ".75rem"
              "padding"
              ".2rem .4rem"
              "margin"
              "0"))
          (css/rule
            ".ol-debug"
            (list "margin" ".75rem 0" "opacity" ".6"))
          (css/rule
            ".ol-debug > summary"
            (list "cursor" "pointer" "font-size" ".85rem"))
          (css/rule ".ol-todo-btn" (list "cursor" "pointer"))
          (css/rule
            ".ol-todo-btn.st-TODO"
            (list "background" "var(--pending-bg)"))
          (css/rule
            ".ol-todo-btn.st-DONE"
            (list "background" "var(--done-bg)"))
          (css/rule
            ".ol-todo-btn.st-WAITING"
```

```
1230            (list "background" "var(--waiting-bg)"))))))
```

192 libs/plan/lib.pp

```
;; package plan -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push

(define plan
  (lambda
    (body)
    "Set user plan"
    (plan/set! body)))

(define plan/all-rows
  (lambda
    ()
    "Retrieve up to 200 plan rows"
    (rows (quote user-plan) 200)))

(define plan/clear!
  (lambda
    ()
    "Clear user plan"
    (plan/set! "")))

(define plan/directory-html
  (lambda
    ()
    "Render plans directory HTML"
    (let
      ((xs (plan-list)))
      (if
        (null? xs)
        (html/p*
          (html/attr "class" "muted")
          "No one has set a plan yet.")
        (html/ul "" (html/join (list/map plan/directory-li xs)))))))

(define plan/directory-li
  (lambda
    (pair)
    "Render plan entry HTML list item"
    (let
      ((user (as-str (car pair))) (body (as-str (cdr pair))))
      (html/li
        (html/attr "class" "card")
        (html/join
          (list
            (html/p*
              ""
              (html/a
                (string-append "/profile?user=" user)
                (html/strong user)))
            (html/p (html/text body)))))))))

(define plan/fields-body
  (lambda
    (fields)
    "Extract plan body from fields"
    (as-str (dict-get fields "body"))))

(define plan/fields-updated
  (lambda
    (fields)
    "Extract updated timestamp or empty string"
    (let
      ((u (dict-get fields "updated_at")))
      (if (null? u) "" (as-str u)))))

(define plan/fields-username
  (lambda
    (fields)
    "Extract plan username from fields"
    (as-str (dict-get fields "username"))))

```

```
(define plan/find-row
  (lambda
    (username)
    "Find plan row by username"
    (let
      ((want (as-str username)))
      (list/foldl
        (lambda
          (acc row)
          (if
            (null? acc)
            (if
              (string-eq
                (plan/fields-username (plan/row-fields row))
                want)
              row
              acc)
            acc))
        ()
        (plan/all-rows)))))

(define plan/form-html
  (lambda
    (body)
    "Render plan entry HTML form"
    (html/div
      (html/attr "class" "card")
      (html/join
        (list
          (html/h2 "Your plan")
          (html/p*
            (html/attr "class" "muted")
            "Like a Unix .plan file -- short note other users on this box can read.")
          (html/form
            "POST"
            "/plans"
            (html/attrs
              (list
                (html/attr "hx-post" "/plans")
                (html/attr "hx-target" "#plan-main")
                (html/attr "hx-swap" "innerHTML")))
            (html/join
              (list
                (html/label
                  (html/attr "for" "plan-body")
                  "What are you working on?")
                (html/textarea
                  "body"
                  (html/attrs
                    (list
                      (html/attr "id" "plan-body")
                      (html/attr "rows" "4")
                      (html/attr
                        "placeholder"
                        "shipping macros, out Thursday, ...")))
                  (as-str body))
                (html/button (html/attr "type" "submit") "Save plan")))))))))

(define plan/main-html
  (lambda
    ()
    "Render main plan page"
    (let
      ((body (plan/mine)))
      (html/div
        (html/attr "id" "plan-main")
        (html/join
          (list (plan/mine-display body) (plan/form-html body)))))))

(define plan/mine
  (lambda
```

```
    ()
    "Get my own plan"
    (plan/of (whoami))))

(define plan/mine-display
  (lambda
    (body)
    "Render my plan display card"
    (html/div
      (html/attr "class" "card")
      (if
        (blank? body)
        (html/p*
          (html/attr "class" "muted")
          "No plan yet. What are you working on?")
        (html/p (html/text body))))))

(define plan/of
  (lambda
    (username)
    "Get another user's plan"
    (let
      ((row (plan/find-row username)))
      (if
        (null? row)
        ""
        (plan/fields-body (plan/row-fields row))))))

(define plan/plans-page
  (lambda
    ()
    "Render everyone's plans directory"
    (html/page
      (list
        (html/h1 "Plans")
        (html/p*
          (html/attr "class" "muted")
          "Who is working on what (Unix .plan directory).")
        (plan/directory-html)))))

(define plan/profile-page
  (lambda
    ()
    "Render another user's profile page"
    (let
      ((user (as-str (request-param "user"))))
      (if
        (blank? user)
        (html/page
          (list
            (html/h1 "Profile")
            (html/p*
              (html/attr "class" "muted")
              "Pass ?user=username (v1; path /u/:user deferred).")))
        (let
          ((body (plan/of user)))
          (html/page
            (list
              (html/h1 (html/text user))
              (html/p* (html/attr "class" "muted") "Plan (finger)")
              (if
                (blank? body)
                (html/p*
                  (html/attr "class" "muted")
                  "No plan yet. What are you working on?")
                (html/div
                  (html/attr "class" "card")
                  (html/p (html/text body))))
              (html/p* "" (html/a "/plans" "View their plan")))))))))

(define plan/row-fields
  (lambda
    (row)
```

```
    "Extract fields from plan row"
    (cdr row)))

(define plan/row-id
  (lambda
    (row)
    "Extract plan row ID"
    (as-str (car row))))

(define plan/set!
  (lambda
    (body)
    "Create or update my plan"
    (let
      ((user (as-str (whoami)))
        (text (as-str body))
        (row (plan/find-row (whoami))))
      (if
        (null? row)
        (seq
          (create
            (quote user-plan)
            (dict-set*
              "{}"
              (list
                "username"
                user
                "body"
                text
                "updated_at"
                "")))
          text)
        (seq
          (update
            (quote user-plan)
            (plan/row-id row)
            (dict-set*
              "{}"
              (list
                "username"
                user
                "body"
                text
                "updated_at"
                "")))
          text)))))
```

193 libs/rb/lib.pp

```
;; libs/rb/lib.pp -- root beer recipe DSL for Pricklypear
;; Port of rb (OCaml) into Nopales. The recipe IS the program.
;;
;; A recipe is a Nopales expression that evaluates into an alist (list of
;; pairs). Calc functions operate on those alists. Persistence to the tree
;; is a separate concern (rb/save! ? create 'recipe).

;; ============================================================
;; ALIST HELPERS
;; ============================================================

;; assoc returns (key . value) or () if missing; rb/get unwraps it.
(define (rb/get key alist)
  "Look up key in an alist; return its value or ()."
  (let ((pair (assoc key alist)))
    (if (null? pair) () (cdr pair))))

(define (rb/has? key alist)
  "True when key is present in the alist."
  (not (null? (assoc key alist))))

(define (rb/set key val alist)
  "Prepend (key . val) onto alist."
  (cons (cons key val) alist))

(define (rb/append a b)
  "Concatenate two lists."
  (if (null? a) b (cons (car a) (rb/append (cdr a) b))))

(define (rb/map fn xs)
  "Map fn over a cons-built list."
  (if (null? xs) (list)
    (cons (fn (car xs)) (rb/map fn (cdr xs)))))

(define (rb/filter pred xs)
  "Keep elements of xs satisfying pred."
  (if (null? xs) (list)
    (if (pred (car xs))
        (cons (car xs) (rb/filter pred (cdr xs)))
      (rb/filter pred (cdr xs)))))

(define (rb/sum fn xs)
  "Sum fn applied to each element, as a float."
  (if (null? xs) 0.0 (+ (fn (car xs)) (rb/sum fn (cdr xs)))))

;; count that works on cons-built lists (not just VLists)
(define (rb/count xs)
  "Number of elements (works on cons-built lists)."
  (if (null? xs) 0 (+ 1 (rb/count (cdr xs)))))

;; ============================================================
;; UNITS -- dimension table and conversion
;; Base units: Volume?L, Mass?Lb, Count?Each, Time?Min
;; ============================================================

(define rb/unit-table
  (list
    (cons "L"     (cons "volume" 1.0))
    (cons "mL"    (cons "volume" 0.001))
    (cons "gal"   (cons "volume" 3.78541))
    (cons "qt"    (cons "volume" 0.946353))
    (cons "fl_oz" (cons "volume" 0.0295735))
    (cons "cup"   (cons "volume" 0.236588))
    (cons "tbsp"  (cons "volume" 0.0147868))
    (cons "tsp"   (cons "volume" 0.00492892))
    (cons "lb"    (cons "mass" 1.0))
    (cons "oz"    (cons "mass" 0.0625))
    (cons "kg"    (cons "mass" 2.20462))
    (cons "g"     (cons "mass" 0.00220462))
    (cons "each"  (cons "count" 1.0))
    (cons "batch" (cons "count" 1.0))
```

```
    (cons "min"   (cons "time" 1.0))
    (cons "hr"    (cons "time" 60.0))
    (cons "day"   (cons "time" 1440.0))))

(define (rb/unit-dim u)   "Dimension of a unit string (volume/mass/count/time)." (car (rb/get u rb/unit-table)))
(define (rb/unit-factor u) "Factor of a unit relative to its base unit." (cdr (rb/get u rb/unit-table)))

;; quantity: (value . "unit")
(define (qty val unit) "Build a quantity (value . unit)." (cons val unit))
(define (qty-val q) "Value part of a quantity." (car q))
(define (qty-unit q) "Unit string of a quantity." (cdr q))

(define (rb/to-base q)
  "Convert a quantity to its base unit."
  (* (qty-val q) (rb/unit-factor (qty-unit q))))

(define (rb/compatible? q1 q2)
  "True when two quantities share a dimension."
  (string-eq (rb/unit-dim (qty-unit q1)) (rb/unit-dim (qty-unit q2))))

(define (rb/convert q target-unit)
  "Convert a quantity to a target unit within its dimension."
  (if (not (string-eq (rb/unit-dim (qty-unit q)) (rb/unit-dim target-unit)))
      q
    (let ((base (rb/to-base q)))
      (qty (/ base (rb/unit-factor target-unit)) target-unit))))

(define (rb/qty-str q)
  "Render a quantity as \"value unit\"."
  (string-append (as-str (qty-val q)) " " (qty-unit q)))

;; ============================================================
;; COSTS (ingredients, utilities, packaging)
;; A cost is an alist with type/name/qty_per_batch/order_qty/order_price/category
;; plus optional source/url/note/on_hand.
;; ============================================================

(define (ingredient name qbatch oqty price opts)
  "Build an ingredient cost entry."
  (rb/make-cost name qbatch oqty price "ingredient" opts))

(define (utility name qbatch oqty price opts)
  "Build a utility cost entry."
  (rb/make-cost name qbatch oqty price "utility" opts))

(define (pkg-cost name qbatch oqty price opts)
  "Build a packaging cost entry."
  (rb/make-cost name qbatch oqty price "packaging" opts))

(define (rb/make-cost name qbatch oqty price category opts)
  "Assemble a cost alist from name/quantities/price/category/opts."
  (cons (cons "type" "cost")
  (cons (cons "name" name)
  (cons (cons "qty_per_batch" qbatch)
  (cons (cons "order_qty" oqty)
  (cons (cons "order_price" price)
  (cons (cons "category" category) opts)))))))

;; opt constructors -- each returns (key . value) for the opts alist
(define (source s) "Opt: source key/value." (cons "source" s))
(define (url s) "Opt: url key/value." (cons "url" s))
(define (note s) "Opt: note key/value." (cons "note" s))
(define (on-hand q) "Opt: on-hand quantity key/value." (cons "on_hand" q))

;; ============================================================
;; STEPS
;; ============================================================

(define (step name duration opts)
  "Build a process step entry."
```

```
  (cons (cons "type" "step")
  (cons (cons "name" name)
  (cons (cons "duration" duration) opts))))

(define (use name) "Opt: use-this key/value." (cons "use" name))

;; ============================================================
;; PACKAGES
;; A package is an alist with product info + packaging_costs list.
;; ============================================================

(define (package product-name units-per-batch liters-per-unit selling-price
    pkgs)
  "Build a package/product alist with packaging costs."
  (list
    (cons "type" "package")
    (cons "product_name" product-name)
    (cons "units_per_batch" units-per-batch)
    (cons "liters_per_unit" liters-per-unit)
    (cons "selling_price" selling-price)
    (cons "packaging_costs" pkgs)))

;; ============================================================
;; FIXED COSTS, EQUIPMENT & PLAN
;; ============================================================

(define (fixed-cost name monthly)
  "Build a fixed-cost entry."
  (list
    (cons "type" "fixed")
    (cons "name" name)
    (cons "monthly_cost" monthly)))

;; equipment: capital purchase with priority tier
(define (equipment name price priority opts)
  "Build an equipment/capital entry with priority."
  (cons (cons "type" "equipment")
  (cons (cons "name" name)
  (cons (cons "price" price)
  (cons (cons "priority" priority) opts)))))

(define (impact s) "Opt: impact key/value." (cons "impact" s))
(define (unlocks s) "Opt: unlocks key/value." (cons "unlocks" s))

(define (plan batches-per-month starter)
  "Build a plan alist (batches and starter recipe)."
  (list
    (cons "batches_per_month" batches-per-month)
    (cons "starter_recipe" starter)))

;; ============================================================
;; SHARED BLOCKS & RECIPE IMPORTS
;; ============================================================

;; A shared block is a named list of costs, reusable across recipes.
(define (shared name costs)
  "Build a reusable named block of costs."
  (list
    (cons "type" "shared")
    (cons "name" name)
    (cons "costs" costs)))

;; uses: recipe-level import of a shared block. Returns a marker that
;; rb/collect-recipe splices -- the block's costs get appended to the
    recipe.
(define (uses block)
  "Marker importing a shared block's costs into a recipe."
  (list
    (cons "type" "uses")
    (cons "block" block)))

;; ============================================================
;; MIXED RUNS -- allocate batch liters across package types
```

```
;; ============================================================

;; A mix-part is (package-name . count)
(define (mixed name parts)
  "Build a mixed-run part (package-name . count) list."
  (list
    (cons "type" "mixed")
    (cons "name" name)
    (cons "parts" parts)))

;; ============================================================
;; RECIPES -- collect items into a recipe alist
;; ============================================================

(define (recipe name batch-liters items)
  "Collect items into a recipe alist with name and batch liters."
  (rb/collect-recipe
    (list (cons "name" name) (cons "batch_liters" batch-liters))
    items))

(define (rb/collect-recipe base items)
  "Partition recipe items into costs/steps/packages/mixed/uses."
  (if (null? items)
      base
    (let ((item (car items)))
      (let ((item-type (rb/get "type" item)))
        (cond
          ((string-eq item-type "cost")
           (rb/collect-recipe
             (rb/set "costs" (cons item (rb/get "costs" base)) base)
             (cdr items)))
          ((string-eq item-type "step")
           (rb/collect-recipe
             (rb/set "steps" (cons item (rb/get "steps" base)) base)
             (cdr items)))
          ((string-eq item-type "package")
           (rb/collect-recipe
             (rb/set "packages" (cons item (rb/get "packages" base)) base)
             (cdr items)))
          ((string-eq item-type "mixed")
           (rb/collect-recipe
             (rb/set "mixed" (cons item (rb/get "mixed" base)) base)
             (cdr items)))
          ((string-eq item-type "uses")
           (let ((block (rb/get "block" item)))
             (rb/collect-recipe
               (rb/set "costs"
                 (rb/append (rb/get "costs" block) (rb/get "costs" base))
                 base)
               (cdr items))))
          (else (rb/collect-recipe base (cdr items))))))))

;; ============================================================
;; CALC -- business calculations (ported from calc.ml)
;; ============================================================

;; unit_cost = order_price / to_base(order_qty)
(define (rb/unit-cost c)
  "Cost per base unit of a cost entry."
  (let ((oqty-base (rb/to-base (rb/get "order_qty" c))))
    (/ (rb/get "order_price" c) oqty-base)))

;; line_cost = to_base(qty_per_batch) * unit_cost
(define (rb/line-cost c)
  "Total line cost for a cost entry's batch quantity."
  (* (rb/to-base (rb/get "qty_per_batch" c)) (rb/unit-cost c)))

;; total variable cost = sum of line costs
(define (rb/variable-cost-total costs)
  "Sum of line costs across cost entries."
  (rb/sum rb/line-cost costs))

;; fixed cost total = sum of monthly_cost
```

```
(define (rb/fixed-cost-total costs)
  "Sum of monthly fixed costs."
  (rb/sum (lambda (c) (rb/get "monthly_cost" c)) costs))

;; filter costs by category
(define (rb/costs-by-category cat costs)
  "Filter cost entries to one category."
  (rb/filter (lambda (c) (string-eq (rb/get "category" c) cat)) costs))

(define (rb/ingredient-costs costs)
  "Ingredient-cost entries only."
  (rb/costs-by-category "ingredient" costs))

(define (rb/packaging-costs costs)
  "Packaging-cost entries only."
  (rb/costs-by-category "packaging" costs))

;; ============================================================
;; BUSINESS (recipe + package option ? business alist)
;; ============================================================

(define (rb/business recipe-data pkg fixed-costs)
  "Assemble a business alist from recipe, package, and fixed costs."
  (list
    (cons "product_name" (rb/get "product_name" pkg))
    (cons "units_per_batch" (rb/get "units_per_batch" pkg))
    (cons "liters_per_unit" (rb/get "liters_per_unit" pkg))
    (cons "selling_price" (rb/get "selling_price" pkg))
    (cons "batch_liters" (rb/get "batch_liters" recipe-data))
    (cons "variable_costs"
          (rb/append (rb/get "costs" recipe-data)
                     (rb/get "packaging_costs" pkg)))
    (cons "fixed_costs" fixed-costs)))

;; ============================================================
;; PER-UNIT CALCULATIONS
;; ============================================================

(define (rb/variable-cost-per-unit biz)
  "Variable cost per produced unit."
  (let ((units (rb/get "units_per_batch" biz)))
    (/ (rb/variable-cost-total (rb/get "variable_costs" biz)) units)))

(define (rb/contribution-margin-per-unit biz)
  "Selling price minus variable cost per unit."
  (- (rb/get "selling_price" biz) (rb/variable-cost-per-unit biz)))

;; ============================================================
;; MONTHLY REPORT
;; ============================================================

(define (rb/monthly-report biz pln)
  "Monthly revenue/cost/profit figures plus breakeven."
  (let ((units-per-batch (rb/get "units_per_batch" biz)))
    (let ((batches-per-month (rb/get "batches_per_month" pln)))
      (let ((units-per-month (* units-per-batch batches-per-month)))
        (let ((variable-batch (rb/variable-cost-total (rb/get
    "variable_costs" biz))))
          (let ((variable-unit (/ variable-batch units-per-batch)))
            (let ((fixed-month (rb/fixed-cost-total (rb/get "fixed_costs"
    biz))))
              (let ((revenue-month (* units-per-month (rb/get
    "selling_price" biz))))
                (let ((variable-month (* variable-batch batches-per-month)))
                  (let ((total-month (+ fixed-month variable-month)))
                    (let ((profit-month (- revenue-month total-month)))
                      (let ((margin-unit (- (rb/get "selling_price" biz)
    variable-unit)))
                        (let ((breakeven-units
                                (if (<= margin-unit 0.0) 999999.0 (/
    fixed-month margin-unit))))
                          (list
                            (cons "units_per_month" units-per-month)
```

```
                            (cons "revenue_per_month" revenue-month)
                            (cons "variable_cost_per_batch" variable-batch)
                            (cons "variable_cost_per_unit" variable-unit)
                            (cons "fixed_cost_per_month" fixed-month)
                            (cons "variable_cost_per_month" variable-month)
                            (cons "total_cost_per_month" total-month)
                            (cons "profit_per_month" profit-month)
                            (cons "contribution_margin_per_unit"
    margin-unit)
                            (cons "breakeven_units_per_month"
    breakeven-units)
                            (cons "breakeven_revenue_per_month"
                                  (* breakeven-units (rb/get
    "selling_price" biz)))))))))))))))))

;; ============================================================
;; RECIPE SCALING (on-hand constraint)
;; ============================================================

;; scale_recipe: if any ingredient/utility has on_hand, find the limiting
;; ingredient (smallest on_hand/qty_per_batch ratio) and scale down.
(define (rb/scale-recipe recipe-data)
  "Scale a recipe down to the limiting on-hand ingredient."
  (let ((costs (rb/get "costs" recipe-data)))
    (let ((constrained (rb/filter
                        (lambda (c)
                          (and (or (string-eq (rb/get "category" c)
    "ingredient")
                                   (string-eq (rb/get "category" c)
    "utility"))
                               (rb/has? "on_hand" c)))
                        costs)))
      (if (null? constrained)
          (list (cons "scale_factor" 1.0)
                (cons "limiting" ())
                (cons "batch_liters" (rb/get "batch_liters" recipe-data)))
        (let ((factors (rb/map
                        (lambda (c)
                          (/ (rb/to-base (rb/get "on_hand" c))
                             (rb/to-base (rb/get "qty_per_batch" c))))
                        constrained)))
          (let ((scale (rb/min-list factors)))
            (list
              (cons "scale_factor" scale)
              (cons "limiting" (rb/find-limiting costs scale))
              (cons "batch_liters"
                    (qty (* (qty-val (rb/get "batch_liters" recipe-data))
    scale)
                            (qty-unit (rb/get "batch_liters"
    recipe-data)))))))))))

(define (rb/min-list xs)
  "Smallest element of a non-empty numeric list."
  (if (null? (cdr xs))
      (car xs)
    (let ((rest (rb/min-list (cdr xs))))
      (if (< (car xs) rest) (car xs) rest))))

(define (rb/find-limiting costs scale)
  "Name of the on-hand cost matching the given scale factor."
  (if (null? costs) ()
    (let ((c (car costs)))
      (if (rb/has? "on_hand" c)
          (let ((f (/ (rb/to-base (rb/get "on_hand" c))
                      (rb/to-base (rb/get "qty_per_batch" c)))))
            (if (= f scale)
                (rb/get "name" c)
              (rb/find-limiting (cdr costs) scale)))
        (rb/find-limiting (cdr costs) scale)))))

;; ============================================================
;; MONEY FORMATTING
;; ============================================================
```

```
420
421 (define (rb/money n)
422   "Format a number as dollars-and-cents text."
423   (let ((cents (round (* n 100.0))))
424     (let ((whole (/ (floor cents) 100))
425           (frac (mod (floor cents) 100)))
426       (string-append "$"
427         (string-append (as-str whole)
428           (string-append "."
429             (if (< frac 10)
430                 (string-append "0" (as-str frac))
431               (as-str frac))))))))
432
433 ;; ============================================================
434 ;; OPERATING TIER -- batches brewed, purchases, orders
435 ;; (Beyond OCaml rb -- closes the planned?actual loop)
436 ;; ============================================================
437
438 ;; A batch log: records an actual brew
439 (define (brew date recipe-name yield-liters packaged-list ferment-days
      carbonation rating notes)
440   "Build a batch log entry for an actual brew."
441   (rb/make-brew date recipe-name yield-liters packaged-list ferment-days
      carbonation rating notes))
442
443 (define (rb/make-brew date recipe-name yield-liters packaged-list
      ferment-days carbonation rating notes)
444   "Assemble a batch alist from brew parameters."
445   (cons (cons "type" "batch")
446   (cons (cons "date" date)
447   (cons (cons "recipe" recipe-name)
448   (cons (cons "yield_liters" yield-liters)
449   (cons (cons "packaged" packaged-list)
450   (cons (cons "ferment_days" ferment-days)
451   (cons (cons "carbonation" carbonation)
452   (cons (cons "rating" rating)
453   (cons (cons "notes" notes) (list)))))))))))
454
455 ;; packaged: (cons pkg-name count) -- e.g. (cons "500mL PET" 12)
456 (define (packaged pkg-name count) "A packaged-units entry (name . count)."
      (cons pkg-name count))
457
458 ;; A purchase log: records an actual ingredient/equipment buy
459 (define (bought name qty price date opts)
460   "Build a purchase log entry."
461   (cons (cons "type" "purchase")
462   (cons (cons "name" name)
463   (cons (cons "qty" qty)
464   (cons (cons "price" price)
465   (cons (cons "date" date) opts))))))
466
467 ;; ============================================================
468 ;; PERSISTENCE -- save recipe to the tree
469 ;; ============================================================
470
471 (define (rb/init-storage!)
472   "Ensure the recipe type exists."
473   (if (null? (type-fields (quote recipe)))
474       (deftype (quote recipe) (quote name) (quote batch_liters) (quote
      source) (quote created_day))
475     ()))
476
477 (define (rb/save-recipe! recipe-data source-text)
478   "Persist a recipe alist to the tree with its source text."
479   (seq
480     (rb/init-storage!)
481     (create (quote recipe)
482       (dict-set
483         (dict-set
484           (dict-set
485             (dict-set (json-emit "{}")
486               "name" (rb/get "name" recipe-data))
```

```
            "batch_liters" (as-str (rb/qty-str (rb/get "batch_liters" recipe-data))))
          "source" source-text)
        "created_day" (date-today)))))

(define (rb/recipe-rows)
  "All persisted recipe rows."
  (rows (quote recipe)))

(define (rb/fetch-recipe id)
  "Fetch a single recipe row by id."
  (get (quote recipe) id))

(define (rb/seed-all!)
  "Seed every bundled recipe, skipping ones already present."
  (seq
    (rb/init-storage!)
    (rb/seed-recipes rb/all-recipes)))

(define (rb/seed-recipes recipes)
  "Seed a list of (name . recipe-alist) pairs that are not yet stored."
  (if (null? recipes)
      (list)
    (seq
      (let ((name (car (car recipes)))
            (r (cdr (car recipes))))
        (if (null? (rb/find-recipe-row name))
            (rb/save-recipe! r (string-append "(recipe \"" name "\" ...)"))
          ()))
      (rb/seed-recipes (cdr recipes)))))

(define (rb/find-recipe-row name)
  "Locate a stored recipe row by name; () when absent."
  (rb/find-in-rows name (rows (quote recipe))))

(define (rb/find-in-rows name rws)
  "Find a row whose name field equals name."
  (if (null? rws)
      ()
    (if (string-eq (dict-get (cdr (car rws)) "name") name)
        (car rws)
      (rb/find-in-rows name (cdr rws)))))
;; libs/rb/recipes.pp -- all rb recipes converted to Nopales
;; Hand-converted from /home/roerick/dev/wyo.tech/rb/data/*.rb

;; ============================================================
;; SHARED BLOCKS
;; ============================================================

(define rb/ginger-bug-base
  (shared "ginger-bug-base"
    (list
      (ingredient "Ginger bug starter" (qty 500 "mL") (qty 1 "L") 10.00
        (list (source "Sandpoint Soda Company")
              (url "https://sandpointsodacompany.com/product/ginger-bug-soda-starter/")
              (note "Organic living soda starter, $10 (sale from $20). Make from scratch if out of stock.")))
      (utility "Filtered water" (qty 10 "L") (qty 50 "L") 4.00
        (list (note "5-gallon jugs from grocery or delivery. Must be chlorine-free. ~50c/gal.")))
      (utility "Cleaning + sanitizer" (qty 1 "batch") (qty 1 "batch") 6.00
        (list (source "Any homebrew supply")
              (url "https://www.northernbrewer.com/")
              (note "Star San or similar no-rinse sanitizer. Essential for fermentation equipment."))))))

;; ============================================================
;; RECIPES
;; ============================================================

(define rb/simple-root-beer
  (recipe "Simple Root Beer" (qty 6 "L")
```

```
    (list
      (ingredient "Ginger bug starter" (qty 300 "mL") (qty 1 "L") 10.00
        (list (note "~50 mL per L of batch. Active bubbly starter.")))
      (utility "Filtered water" (qty 6 "L") (qty 50 "L") 4.00
        (list (note "Chlorine-free. Must be dechlorinated for ginger bug health.")))
      (utility "Cleaning + sanitizer" (qty 1 "batch") (qty 1 "batch") 6.00
        (list (note "Star San or similar no-rinse sanitizer.")))
      (ingredient "Sassafras root bark" (qty 42 "g") (qty 450 "g") 109.02
        (list (source "Monterey Bay Herb Co")
              (note "Signature root beer flavor. Simmer with wintergreen, strain.")))
      (ingredient "Wintergreen leaf" (qty 17 "g") (qty 110 "g") 6.50
        (list (source "Penn Herb Co")
              (note "Classic root beer minty note. Simmer with sassafras.")))
      (ingredient "Vanilla extract" (qty 15 "mL") (qty 500 "mL") 12.00
        (list (note "Add at bottling to preserve aroma.")))
      (ingredient "Cane sugar" (qty 630 "g") (qty 11 "kg") 22.50
        (list (source "Any grocery")
              (note "~105 g/L - original 1876 Hires ratio (4 lb per 5 gal). Less sweet, more old-timey.")))
      (step "Steep herbs" (qty 20 "min")
        (list (use "Sassafras root bark") (use "Wintergreen leaf")
              (note "Simmer sassafras and wintergreen in ~3L water, covered, 20 min. Strain into fermenter.")))
      (step "Dissolve sugar" (qty 5 "min")
        (list (use "Cane sugar") (note "Stir sugar into hot tea until fully dissolved.")))
      (step "Cool wort" (qty 2 "hr")
        (list (note "Cool to room temp before adding starter. Speeds up with ice bath.")))
      (step "Pitch starter" (qty 10 "min")
        (list (use "Ginger bug starter") (note "Add ginger bug starter, top up to 6L with filtered water.")))
      (step "Bottle and ferment" (qty 3 "day")
        (list (note "Bottle in 12 x 500mL PET bottles. Ferment at room temp until carbonated (2-4 days). Squeeze-test for pressure.")))
      (step "Add vanilla" (qty 5 "min")
        (list (use "Vanilla extract") (note "Add vanilla extract at bottling to preserve aroma.")))
      (package "500mL PET" 12 (qty 0.5 "L") 7.00
        (list
          (pkg-cost "PET bottles with caps" (qty 12 "each") (qty 96 "each") 112.32
            (list (source "KegLand") (note "Reusable PET beer bottles, 500mL. $1.17/bottle after 10% volume discount.")))
          (pkg-cost "Premium labels" (qty 12 "each") (qty 100 "each") 20.00
            (list (source "Sticker Mule") (note "Waterproof BOPP film roll labels.")))
          (pkg-cost "Breakage reserve" (qty 12 "each") (qty 96 "each") 5.00
            (list (note "~5% replacement rate for caps/gaskets."))))))))

(define rb/root-beer
  (recipe "Ginger Bug Root Beer" (qty 10 "L")
    (list
      (uses rb/ginger-bug-base)
      (ingredient "Sarsaparilla root" (qty 0.42 "lb") (qty 1 "lb") 22.00
        (list (source "Starwest Botanicals")
              (url "https://www.starwest-botanicals.com/sarsaparilla-root-whole-csw.html")
              (note "Sarsaparilla Root Whole Cut & Sifted. ~$7.33/4oz, ~$22/lb. Jamaican sarsaparilla also works.")))
      (ingredient "Sassafras root bark" (qty 0.16 "lb") (qty 1 "lb") 109.02
        (list (source "Monterey Bay Herb Co")
              (url "https://www.montereybayherb.com/product/sassafras-root-bark-cut-and-sifted/")
              (note "Real sassafras - legal under Wyoming Food Freedom Act (Wyo. Stat. 35-10-301). The signature root beer flavor.")))
      (ingredient "Wintergreen leaf" (qty 0.06 "lb") (qty 4 "oz") 6.50
        (list (source "Penn Herb Co")
              (url "https://www.pennherb.com/wintergreen-leaves-cut-4-oz-456C4")
```

```
              (note "Wintergreen Leaves, Cut (Gaultheria procumbens). Wild
    harvested. Provides the classic root beer flavor note.")))
      (ingredient "Vanilla beans" (qty 0.05 "lb") (qty 4 "oz") 30.00
        (list (source "Natural Vanilla USA")
              (url "https://naturalvanilla.us/vanilla-beans/")
              (note "Grade B (extract grade) Madagascar Bourbon. Best value
    for infusion/brewing.")))
      (ingredient "Molasses" (qty 0.31 "gal") (qty 1 "gal") 28.00
        (list (source "WebstaurantStore")
              (url
    "https://www.webstaurantstore.com/grandmas-unsulfured-molasses-1-gallon/9997971942.html")
              (note "Grandma's Unsulfured Original Molasses, 1 gallon.")))
      (ingredient "Cane sugar" (qty 5.0 "lb") (qty 25 "lb") 22.50
        (list (source "Any grocery / FoodServiceDirect")
              (url
    "https://www.foodservicedirect.com/esome-organic-cane-sugar-25-pound-1-per-case-172716.html")
              (note "Wholesome Organic Cane Sugar 25 lb bag. Conventional
    ~$0.50/lb. Organic ~$2/lb.")))
      (package "1L Flip-Top" 10 (qty 1.0 "L") 18.00
        (list
          (pkg-cost "Flip-top glass bottles" (qty 10 "each") (qty 12
    "each") 89.99
            (list (source "Northern Brewer")
                  (url
    "https://www.northernbrewer.com/products/ez-cap-bottles-brown-1-liter-with-flip-tops")
                  (note "Brown EZ Cap 1L swing top bottles, 12/case.
    Reusable with replacement gaskets.")))
          (pkg-cost "Premium labels" (qty 10 "each") (qty 100 "each") 20.00
            (list (source "Sticker Mule")
                  (url
    "https://www.stickermule.com/products/die-cut-roll-labels")
                  (note "Waterproof BOPP film roll labels. 100 for $66.
    Dishwasher safe.")))
          (pkg-cost "Breakage reserve" (qty 10 "each") (qty 12 "each") 5.00
            (list (note "Budget ~2% of bottle cost per batch for
    breakage/replacement gaskets.")))))
      (package "500mL Flip-Top" 20 (qty 0.5 "L") 12.00
        (list
          (pkg-cost "Flip-top glass bottles" (qty 20 "each") (qty 12
    "each") 69.99
            (list (source "Northern Brewer")
                  (url
    "https://www.northernbrewer.com/products/ez-cap-bottles-brown-1-liter-with-flip-tops")
                  (note "Brown EZ Cap 16oz (~500mL) swing top bottles,
    12/case.")))
          (pkg-cost "Premium labels" (qty 20 "each") (qty 100 "each") 20.00
            (list (source "Sticker Mule")
                  (url
    "https://www.stickermule.com/products/die-cut-roll-labels")))
          (pkg-cost "Breakage reserve" (qty 20 "each") (qty 12 "each") 5.00
            (list (note "Breakage budget.")))))
      (mixed "Half 1L + Half 500mL"
        (list (cons "1L Flip-Top" 5) (cons "500mL Flip-Top" 10))))))

(define rb/ginger-ale
  (recipe "Ginger Ale" (qty 10 "L")
    (list
      (uses rb/ginger-bug-base)
      (ingredient "Fresh ginger" (qty 1.5 "lb") (qty 1 "lb") 5.50
        (list (source "Any grocery / Asian market")
              (note "For syrup method: grate or slice thin, simmer with
    sugar and water, strain before mixing. Start with 1.5 lb for 10L,
    adjust to taste.")))
      (ingredient "Cane sugar" (qty 2.2 "lb") (qty 25 "lb") 22.50
        (list (source "Any grocery / FoodServiceDirect")
              (note "Dissolve in ginger syrup or add directly. ~100g/L is
    the standard ginger ale ratio. Adjust based on taste tests.")))
      (ingredient "Lemon juice" (qty 0.25 "cup") (qty 1 "gal") 8.00
        (list (source "Any grocery")
              (note "Bottled or fresh. For acidity balance. Start
    conservative, add more if too sweet.")))
      (ingredient "Black tea" (qty 2 "each") (qty 20 "each") 5.00
        (list (source "Any grocery")
```

```
              (note "2 tea bags for tannin/body. Optional - omit for lighter style.")))
      (package "1L Flip-Top" 10 (qty 1.0 "L") 16.00
        (list
          (pkg-cost "Flip-top glass bottles" (qty 10 "each") (qty 12 "each") 89.99
            (list (source "Northern Brewer")))
          (pkg-cost "Premium labels" (qty 10 "each") (qty 100 "each") 20.00
            (list (source "Sticker Mule")))
          (pkg-cost "Breakage reserve" (qty 10 "each") (qty 12 "each") 5.00
            (list))))
      (package "500mL Flip-Top" 20 (qty 0.5 "L") 10.00
        (list
          (pkg-cost "Flip-top glass bottles" (qty 20 "each") (qty 12 "each") 69.99
            (list (source "Northern Brewer")))
          (pkg-cost "Premium labels" (qty 20 "each") (qty 100 "each") 20.00
            (list (source "Sticker Mule")))
          (pkg-cost "Breakage reserve" (qty 20 "each") (qty 12 "each") 5.00
            (list))))
      (mixed "Market mix"
        (list (cons "1L Flip-Top" 4) (cons "500mL Flip-Top" 12))))))

(define rb/lemonade
  (recipe "Ginger Bug Lemonade" (qty 10 "L")
    (list
      (uses rb/ginger-bug-base)
      (ingredient "Fresh lemons" (qty 20 "each") (qty 20 "each") 18.00
        (list (source "Any grocery / Costco")
              (note "Bag of 20 lemons ~$18. Organic costs more. Check farmers markets for better pricing.")))
      (ingredient "Fresh ginger" (qty 1.2 "lb") (qty 1 "lb") 5.50
        (list (source "Any grocery / Asian market")
              (note "Fresh ginger root. Asian markets often have best bulk pricing (~$3-5/lb). Also feeds the ginger bug.")))
      (ingredient "Cane sugar" (qty 4.5 "lb") (qty 25 "lb") 22.50
        (list (source "Any grocery / FoodServiceDirect")
              (url "https://www.foodservicedirect.com/esome-organic-cane-sugar-25-pound-1-per-case-172716.html")
              (note "Same source as root beer recipe.")))
      (package "1L Flip-Top" 10 (qty 1.0 "L") 16.00
        (list
          (pkg-cost "Flip-top glass bottles" (qty 10 "each") (qty 12 "each") 89.99
            (list (source "Northern Brewer")))
          (pkg-cost "Premium labels" (qty 10 "each") (qty 100 "each") 20.00
            (list (source "Sticker Mule")))
          (pkg-cost "Breakage reserve" (qty 10 "each") (qty 12 "each") 5.00
            (list))))
      (package "500mL Flip-Top" 20 (qty 0.5 "L") 10.00
        (list
          (pkg-cost "Flip-top glass bottles" (qty 20 "each") (qty 12 "each") 69.99
            (list (source "Northern Brewer")))
          (pkg-cost "Premium labels" (qty 20 "each") (qty 100 "each") 20.00
            (list (source "Sticker Mule")))
          (pkg-cost "Breakage reserve" (qty 20 "each") (qty 12 "each") 5.00
            (list))))
      (mixed "Market mix"
        (list (cons "1L Flip-Top" 4) (cons "500mL Flip-Top" 12))))))

(define rb/cola
  (recipe "Ginger Bug Cola" (qty 10 "L")
    (list
      (uses rb/ginger-bug-base)
      (ingredient "Kola nut" (qty 0.35 "lb") (qty 1 "lb") 24.00
        (list (source "American Bulk Herbs / KolaNutWholesale")
              (url "https://www.kolanutwholesale.com/")
              (note "Cola nitida (white or red variety). Small quantities $20-40/lb. Also at Starwest Botanicals and some African grocery stores.")))
      (ingredient "Cinnamon" (qty 0.08 "lb") (qty 1 "lb") 14.00
        (list (source "Any grocery / Monterey Bay Herb Co")
```

```
727               (url "https://www.herbco.com/")
728               (note "Ceylon cinnamon sticks or cut. Cassia is cheaper
      (~$6/lb) but Ceylon is better for cola.")))
729       (ingredient "Orange peel" (qty 0.15 "lb") (qty 1 "lb") 11.00
730         (list (source "Monterey Bay Herb Co")
731               (url "https://www.herbco.com/")
732               (note "Dried orange peel cut & sifted. Also at Mountain Rose
      Herbs, Starwest Botanicals.")))
733       (ingredient "Lemon peel" (qty 0.10 "lb") (qty 1 "lb") 10.00
734         (list (source "Monterey Bay Herb Co")
735               (url "https://www.herbco.com/")
736               (note "Dried lemon peel cut & sifted. Also at Mountain Rose
      Herbs, Starwest Botanicals.")))
737       (ingredient "Vanilla beans" (qty 0.04 "lb") (qty 4 "oz") 30.00
738         (list (source "Natural Vanilla USA")
739               (url "https://naturalvanilla.us/vanilla-beans/")
740               (note "Grade B Madagascar Bourbon. Best value for
      infusion.")))
741       (ingredient "Cane sugar" (qty 5.5 "lb") (qty 25 "lb") 22.50
742         (list (source "Any grocery / FoodServiceDirect")
743               (note "Same source as other recipes.")))
744       (ingredient "Molasses" (qty 0.20 "gal") (qty 1 "gal") 28.00
745         (list (source "WebstaurantStore")
746               (url
      "https://www.webstaurantstore.com/grandmas-unsulfured-molasses-1-gallon/9997971942.html")
747               (note "Grandma's Unsulfured Original, 1 gallon.")))
748       (package "1L Flip-Top" 10 (qty 1.0 "L") 17.00
749         (list
750           (pkg-cost "Flip-top glass bottles" (qty 10 "each") (qty 12
      "each") 89.99
751             (list (source "Northern Brewer")))
752           (pkg-cost "Premium labels" (qty 10 "each") (qty 100 "each") 20.00
753             (list (source "Sticker Mule")))
754           (pkg-cost "Breakage reserve" (qty 10 "each") (qty 12 "each") 5.00
755             (list))))
756       (package "500mL Flip-Top" 20 (qty 0.5 "L") 11.00
757         (list
758           (pkg-cost "Flip-top glass bottles" (qty 20 "each") (qty 12
      "each") 69.99
759             (list (source "Northern Brewer")))
760           (pkg-cost "Premium labels" (qty 20 "each") (qty 100 "each") 20.00
761             (list (source "Sticker Mule")))
762           (pkg-cost "Breakage reserve" (qty 20 "each") (qty 12 "each") 5.00
763             (list))))
764       (mixed "Balanced split"
765         (list (cons "1L Flip-Top" 5) (cons "500mL Flip-Top" 10))))))
766
767 (define rb/happy-ginger
768   (recipe "Happy Ginger Ginger Beer" (qty 9.6 "L")
769     (list
770       (ingredient "Ginger bug starter" (qty 480 "mL") (qty 1 "L") 0.00
771         (list (note "Home-made ginger bug. 1 L on hand shared with
      Traditional Root Beer tonight. Save ~40 mL as next starter first, then
      split the remaining 960 mL 50/50 -> 480 mL each. ~50 mL/L = normal
      pitch rate. Sunk cost - ginger/sugar that fed it tracked below (spent
      + fresh ginger, cane sugar), not here.")))
772       (utility "Filtered water" (qty 9.6 "L") (qty 50 "L") 4.00
773         (list (note "Chlorine-free, dechlorinated for bug health. 5-gallon
      jugs from grocery/delivery. ~50c/gal. Top up to 9.6 L batch volume.")))
774       (utility "Cleaning + sanitizer" (qty 1 "batch") (qty 1 "batch") 6.00
775         (list (source "Any homebrew supply")
776               (url "https://www.northernbrewer.com/")
777               (note "Star San or similar no-rinse sanitizer. Essential for
      fermentation equipment. One sanitizer cycle can cover both carboys
      tonight.")))
778       (ingredient "Spent ginger (from bug)" (qty 300 "g") (qty 1 "lb") 0.00
779         (list (note "ALL 300 g spent ginger on hand goes here. Strained out
      of the ginger bug before pitching. Already extracted by the bug -
      body/residual ginger flavor, not bright heat. Free ingredient (sunk
      into the bug).")))
780       (ingredient "Fresh ginger" (qty 200 "g") (qty 1 "lb") 5.50
781         (list (source "Any grocery / Asian market")
```

```
782             (note "ALL 200 g fresh ginger on hand goes here. Bright heat.
    Total ginger = 300 g spent + 200 g fresh = 500 g ~ 52 g/L - right on
    brew 3's 54 g/L. Grate or slice thin, simmer with spent ginger, strain
    before mixing.")))
783       (ingredient "Cane sugar" (qty 2.22 "lb") (qty 25 "lb") 22.50
784         (list (source "Any grocery / FoodServiceDirect")
785               (note "1008 g total = 105 g/L target (brew 3 ratio, rated
    9/10). Residual after ferment will be lower - BACKSWEETEN at bottling,
    do not skip.")))
786       (ingredient "Lime juice" (qty 0.64 "cup") (qty 1 "gal") 8.00
787         (list (source "Any grocery")
788               (note "~150 mL = juice of ~5 limes for 9.6 L. Add at bottling
    to preserve freshness. Taste and adjust.")))
789       (ingredient "Pineapple juice" (qty 0.64 "cup") (qty 1 "gal") 8.00
790         (list (source "Any grocery")
791               (note "~150 mL. Pairs with lime. Add at bottling. Taste and
    adjust.")))
792       (ingredient "Black tea" (qty 2 "each") (qty 20 "each") 5.00
793         (list (source "Any grocery")
794               (note "2 tea bags for tannin/body. Optional - omit for
    lighter style.")))
795       (step "Steep ginger" (qty 20 "min")
796         (list (use "Fresh ginger") (use "Spent ginger (from bug)")
797               (note "Simmer grated/sliced fresh ginger WITH spent bug
    ginger in ~3 L water, covered, 20 min. Strain into fermenter. The
    fresh ginger is where the bright heat comes from - don't skip the
    simmer.")))
798       (step "Steep tea" (qty 5 "min")
799         (list (use "Black tea") (note "Steep tea bags in the hot ginger tea
    off the heat, 5 min, then remove bags.")))
800       (step "Dissolve sugar" (qty 5 "min")
801         (list (use "Cane sugar") (note "Stir 1008 g sugar into hot tea
    until fully dissolved.")))
802       (step "Cool wort" (qty 2 "hr")
803         (list (note "Cool to room temp before adding bug. Ice bath speeds
    this up. Same window as the root beer cool - stagger or ice-bath
    both.")))
804       (step "Pitch bug" (qty 10 "min")
805         (list (use "Ginger bug starter") (note "Pitch 480 mL ginger bug.
    Top up to 9.6 L with filtered water. Normal 50 mL/L pitch rate.")))
806       (step "Primary ferment" (qty 2 "day")
807         (list (note "Ferment at room temp 2-3 days until actively bubbly
    and sweetness has dropped. Taste - should read dry/tart, not
    soda-sweet. That dryness is expected.")))
808       (step "Backsweeten, citrus, bottle" (qty 20 "min")
809         (list (use "Lime juice") (use "Pineapple juice")
810               (note "CRITICAL - load-bearing for the 9/10 result. In the
    fermenter: dissolve extra cane sugar to taste (start ~3/4 cup / ~150 g
    for 9.6 L, taste, add more). Add lime + pineapple (~150 mL each), stir
    evenly. Bottle into 19 x 500 mL PET. Fridge when firm, or fridge
    immediately for lighter carb. Drink within 1-2 weeks. Lime/pineapple
    also add ~12 g and ~20 g fermentable sugar on top of the cane.")))
811       (step "Carbonation watch" (qty 2 "day")
812         (list (note "If carbing at room temp after backsweetening:
    squeeze-test PET daily. When firm, REFRIGERATE IMMEDIATELY. Ginger
    grenade risk is real.")))
813       (package "500mL PET" 19 (qty 0.5 "L") 5.00
814         (list
815           (pkg-cost "PET bottles with caps" (qty 19 "each") (qty 96 "each")
    112.32
816             (list (source "KegLand") (note "Reusable PET beer bottles,
    500mL. $1.17/bottle after 10% volume discount.")))
817           (pkg-cost "Premium labels" (qty 19 "each") (qty 100 "each") 20.00
818             (list (source "Sticker Mule") (note "Waterproof BOPP film roll
    labels.")))
819           (pkg-cost "Breakage reserve" (qty 19 "each") (qty 96 "each") 5.00
820             (list (note "~5% replacement rate for caps/gaskets."))))))))
821
822 (define rb/traditional-root-beer
823   (recipe "Traditional Root Beer" (qty 9.6 "L")
824     (list
825       (ingredient "Ginger bug starter" (qty 480 "mL") (qty 1 "L") 0.00
```

```
826       (list (note "Home-made ginger bug. 1 L on hand shared with Happy
      Ginger tonight. Save ~40 mL as next starter first, then split the
      remaining 960 mL 50/50 -> 480 mL each. ~50 mL/L. Sunk cost - not
      tracked here.")))
827     (utility "Filtered water" (qty 9.6 "L") (qty 50 "L") 4.00
828       (list (note "Chlorine-free, dechlorinated for bug health. Steep was
      4 L; top up to 9.6 L batch volume after pitching.")))
829     (utility "Cleaning + sanitizer" (qty 1 "batch") (qty 1 "batch") 6.00
830       (list (source "Any homebrew supply")
831             (url "https://www.northernbrewer.com/")
832             (note "Star San or similar no-rinse sanitizer. One sanitizer
      cycle can cover both carboys tonight.")))
833     (ingredient "Sassafras root bark" (qty 77 "g") (qty 454 "g") 128.99
834       (list (source "Mountain Rose Herbs")
835             (note "As brewed 2026-07-21. Signature root beer flavor.
      Simmer with hard roots. Legal under Wyoming Food Freedom Act (Wyo.
      Stat. 35-10-301).")))
836     (ingredient "Sarsaparilla root" (qty 77 "g") (qty 454 "g") 31.98
837       (list (note "As brewed 2026-07-21. Equal bill with sassafras.
      Earthy sweetness + foamy head (saponins).")))
838     (ingredient "Wintergreen leaf" (qty 44 "g") (qty 454 "g") 39.78
839       (list (source "Mountain Rose Herbs")
840             (note "As brewed 2026-07-21. Less than sassafras by weight
      (44 g vs 77 g). Modern root beer mint note - commercial root beer
      leans wintergreen after the sassafras ban. Simmer with hard roots.")))
841     (ingredient "Licorice root" (qty 31 "g") (qty 300 "g") 5.00
842       (list (source "Local food co-op")
843             (note "As brewed 2026-07-21. Whole licorice root. FORWARD
      flavor - glycyrrhizin = sweet + mouth-coating. Simmer full steep;
      whole root needs the extraction time.")))
844     (ingredient "Star anise" (qty 4 "each") (qty 1 "each") 8.00
845       (list (note "As brewed 2026-07-21. 4 pods. Supports licorice with
      anise aroma. Within published range (Serious Eats uses 3 pods/gal).")))
846     (ingredient "Ashwagandha root" (qty 44 "g") (qty 300 "g") 5.00
847       (list (source "Local food co-op")
848             (note "As brewed 2026-07-21. Back in from v1. Earthy
      baseline; secondary to sassafras/sarsaparilla/wintergreen/licorice.")))
849     (ingredient "Echinacea root" (qty 13 "g") (qty 300 "g") 5.00
850       (list (source "Local food co-op")
851             (note "As brewed 2026-07-21. Light supporting root. Simmer
      with hard roots.")))
852     (ingredient "Cane sugar" (qty 2.33 "lb") (qty 25 "lb") 22.50
853       (list (source "Any grocery / FoodServiceDirect")
854             (note "1056 g = 110 g/L pitched. No molasses - cane only. Bug
      eats most in 2-4 days; residual goes dry/tart. MUST backsweeten at
      bottling or you get the sour note from v1.")))
855     (ingredient "Vanilla extract" (qty 10 "mL") (qty 500 "mL") 12.00
856       (list (note "Add at bottling to preserve aroma. ~2 tsp for 9.6 L.
      Rounds the roots. Third leg of modern root beer (wintergreen +
      licorice + vanilla).")))
857     (step "Steep hard roots" (qty 20 "min")
858       (list (use "Sassafras root bark") (use "Sarsaparilla root") (use
      "Wintergreen leaf")
859             (use "Licorice root") (use "Star anise") (use "Ashwagandha
      root") (use "Echinacea root")
860             (note "As brewed 2026-07-21: all listed herbs simmered in 4 L
      water, covered. Hard roots + wintergreen + whole licorice + star anise
      + ashwagandha + echinacea together. Strain through cheesecloth into
      fermenter, pressing to extract. Wort dark amber.")))
861     (step "Dissolve sugar" (qty 5 "min")
862       (list (use "Cane sugar")
863             (note "Stir 1056 g cane sugar into the hot herbal tea until
      fully dissolved. No molasses - do not add any. Color comes from the
      roots.")))
864     (step "Cool wort" (qty 2 "hr")
865       (list (note "Cool to room temp before adding bug. Ice bath speeds
      this to ~30-60 min. Pitching above ~100F / 38C kills the bug. Same
      window as ginger beer cool - stagger or ice-bath both.")))
866     (step "Pitch bug" (qty 10 "min")
867       (list (use "Ginger bug starter")
868             (note "Pitch 480 mL ginger bug. Top up to 9.6 L with filtered
      water. ~50 mL/L. Bug must be actively bubbling.")))
869     (step "Primary ferment" (qty 2 "day")
```

```
870         (list (note "Ferment at room temp 2-3 days until actively bubbly
      and sweetness has dropped. Taste - should read dry/tart, not
      soda-sweet. That dryness is expected; backsweetening fixes it. Do not
      go bone-dry for a week or the sour note takes over.")))
871       (step "Backsweeten, vanilla, bottle" (qty 20 "min")
872         (list (use "Vanilla extract")
873               (note "CRITICAL - load-bearing for not-sour root beer (same
      lesson as Happy Ginger brew 3, 9/10). In the fermenter: dissolve extra
      cane sugar to taste (start ~3/4-1 cup / ~150-200 g for 9.6 L, stir,
      taste, add more). Add 10 mL vanilla, stirred evenly. Bottle into 19 x
      500 mL PET. Fridge when carb is where you want it. Once backsweetened,
      bug restarts in bottle. Drink within 1-2 weeks, or pasteurize (165F /
      74C x 10 min) after 1-2 days carb to lock it.")))
874       (step "Carbonation watch" (qty 2 "day")
875         (list (note "If carbing at room temp after backsweetening:
      squeeze-test PET daily. When firm, REFRIGERATE IMMEDIATELY. Ginger
      grenade risk is real with backsweetened bug soda.")))
876       (package "500mL PET" 19 (qty 0.5 "L") 7.00
877         (list
878           (pkg-cost "PET bottles with caps" (qty 19 "each") (qty 96 "each")
      112.32
879             (list (source "KegLand") (note "Reusable PET beer bottles,
      500mL. $1.17/bottle after 10% volume discount.")))
880           (pkg-cost "Premium labels" (qty 19 "each") (qty 100 "each") 20.00
881             (list (source "Sticker Mule") (note "Waterproof BOPP film roll
      labels.")))
882           (pkg-cost "Breakage reserve" (qty 19 "each") (qty 96 "each") 5.00
883             (list (note "~5% replacement rate for caps/gaskets."))))))))
884
885 (define rb/experimental-004
886   (recipe "Experimental Root Beer 004: 1896 edition" (qty 6 "L")
887     (list
888       (ingredient "Ginger bug starter" (qty 300 "mL") (qty 1 "L") 0.00
889         (list (note "Home-made ginger bug. ~50 mL/L pitch.")))
890       (utility "Filtered water" (qty 6 "L") (qty 50 "L") 4.00
891         (list (note "Chlorine-free, dechlorinated for bug health.")))
892       (utility "Cleaning + sanitizer" (qty 1 "batch") (qty 1 "batch") 6.00
893         (list (note "Star San or similar no-rinse sanitizer.")))
894       (ingredient "Sassafras root bark" (qty 42 "g") (qty 454 "g") 128.99
895         (list (source "Mountain Rose Herbs") (note "Signature root beer
      flavor. Legal under Wyoming Food Freedom Act.")))
896       (ingredient "Sarsaparilla root" (qty 40 "g") (qty 454 "g") 31.98
897         (list (note "Earthy sweetness + foamy head.")))
898       (ingredient "Wintergreen leaf" (qty 17 "g") (qty 454 "g") 39.78
899         (list (source "Mountain Rose Herbs") (note "Classic root beer mint
      note.")))
900       (ingredient "Licorice root" (qty 25 "g") (qty 300 "g") 5.00
901         (list (source "Local food co-op") (note "Bought 2026-07-05 with the
      kitchen-sink haul. Whole root.")))
902       (ingredient "Ashwagandha root" (qty 30 "g") (qty 300 "g") 5.00
903         (list (source "Local food co-op") (note "Bought 2026-07-05. Earthy
      baseline.")))
904       (ingredient "Echinacea root" (qty 20 "g") (qty 300 "g") 5.00
905         (list (source "Local food co-op") (note "Bought 2026-07-05.
      Supporting root.")))
906       (ingredient "Spearmint leaf" (qty 15 "g") (qty 300 "g") 5.00
907         (list (source "Local food co-op") (note "Bought 2026-07-05. Extra
      mint on top of wintergreen.")))
908       (ingredient "Chamomile flower" (qty 10 "g") (qty 300 "g") 5.00
909         (list (source "Local food co-op") (note "Bought 2026-07-05. Soft
      floral roundness.")))
910       (ingredient "Hyssop herb" (qty 10 "g") (qty 300 "g") 5.00
911         (list (source "Local food co-op") (note "Bought 2026-07-05.
      Bitter/herbal edge.")))
912       (ingredient "Molasses" (qty 0.15 "gal") (qty 1 "gal") 28.00
913         (list (note "THE off-flavor culprit in post-mortem. Present in this
      batch; dropped entirely in Traditional Root Beer v2.")))
914       (ingredient "Cane sugar" (qty 630 "g") (qty 11 "kg") 22.50
915         (list (source "Any grocery") (note "~105 g/L base fermentables,
      plus molasses.")))
916       (ingredient "Vanilla extract" (qty 15 "mL") (qty 500 "mL") 12.00
917         (list (note "Add at bottling to preserve aroma.")))
918       (step "Steep kitchen sink" (qty 25 "min")
```

```
919         (list (use "Sassafras root bark") (use "Sarsaparilla root") (use
      "Wintergreen leaf")
920               (use "Licorice root") (use "Ashwagandha root") (use
      "Echinacea root")
921               (use "Spearmint leaf") (use "Chamomile flower") (use "Hyssop
      herb")
922               (note "Everything in the pot. Simmer covered ~20-25 min in ~3
      L water. Strain, press.")))
923       (step "Dissolve sugars" (qty 5 "min")
924         (list (use "Cane sugar") (use "Molasses")
925               (note "Stir cane sugar + molasses into hot tea until
      dissolved.")))
926       (step "Cool wort" (qty 2 "hr")
927         (list (note "Cool to room temp before pitching. Ice bath helps.")))
928       (step "Pitch bug" (qty 10 "min")
929         (list (use "Ginger bug starter") (note "Pitch ~300 mL bug, top up
      to 6 L.")))
930       (step "Primary ferment" (qty 3 "day")
931         (list (note "Room temp until actively bubbly and sweetness
      drops.")))
932       (step "Vanilla and bottle" (qty 15 "min")
933         (list (use "Vanilla extract") (note "Vanilla at bottling. 12 x 500
      mL PET.")))
934       (package "500mL PET" 12 (qty 0.5 "L") 7.00
935         (list
936           (pkg-cost "PET bottles with caps" (qty 12 "each") (qty 96 "each")
      112.32
937             (list (source "KegLand") (note "Reusable PET beer bottles,
      500mL.")))
938           (pkg-cost "Premium labels" (qty 12 "each") (qty 100 "each") 20.00
939             (list (source "Sticker Mule") (note "Waterproof BOPP film roll
      labels.")))
940           (pkg-cost "Breakage reserve" (qty 12 "each") (qty 96 "each") 5.00
941             (list (note "~5% replacement rate for caps."))))))))
942
943 ;; ============================================================
944 ;; BUSINESS CONFIG (from business.rb)
945 ;; ============================================================
946
947 (define rb/fixed-costs
948   (list
949     (fixed-cost "Booth rent" 60.00)))
950
951 (define rb/equipment-list
952   (list
953     (equipment "2-gal glass jar with airlock" 23.58 1
954       (list (impact "Ferment a 10L batch in one vessel")
955             (unlocks "First batch")
956             (source "Walmart")
957             (url
      "https://www.walmart.com/ip/2-Gallon-Primary-Fermenter-with-Lid-and-Airlock/654980559")))
958     (equipment "Nylon strainer bags (2-pack)" 13.98 1
959       (list (impact "Steep herbs without mess, easy cleanup")
960             (unlocks "Clean root beer production")
961             (source "Northern Brewer")
962             (url
      "https://www.northernbrewer.com/products/nylon-hop-bag-8-x-9-1-4")))
963     (equipment "Auto-siphon + tubing" 19.99 1
964       (list (impact "Sanitary transfer, no contamination risk. Saves 1
      spoiled batch/month.")
965             (unlocks "Reliable fermentation")
966             (source "Northern Brewer")
967             (url
      "https://www.northernbrewer.com/products/auto-siphon-3-flags")))
968     (equipment "Hydrometer" 8.79 1
969       (list (impact "Know when fermentation is done, consistent
      carbonation")
970             (unlocks "Consistent product")
971             (source "MoreBeer")
972             (url
      "https://morebeer.com/products/triple-scale-hydrometer-specific-gravity-potential-alcohol-brix")))
973     (equipment "Folding digital thermometer" 12.49 1
974       (list (impact "Brew temp control for ginger bug health")
```

```
            (unlocks "Consistent fermentation")
            (source "MoreBeer")
            (url
     "https://morebeer.com/products/folding-instant-read-digital-thermometer")))))

(define rb/plan-config
  (plan 4 "Ginger Bug Root Beer"))

;; ============================================================
;; OPERATING LOG (from batch_logs/ and orders/)
;; ============================================================

(define rb/batches
  (list
    (brew "2026-06-05" "Ginger Ale" 3.16 (list (packaged "500mL PET" 8)) 3
     "light" 6
      "Ginger bug was weak, carbonation suffered. Too sweet (145 g/L).
     Lemon+vanilla combo was iffy. Some said 'tastes like Chinese food but
     in the best way.' Next time: reduce ginger to 180g, cut sugar, swap
     lemon for lime or pineapple.")
    (brew "2026-06-12" "Simple Root Beer" 6.0 (list (packaged "500mL" 12))
     0 () 0
      "First batch. Sassafras + wintergreen + vanilla extract. No molasses,
     no sarsaparilla. Old-school root beer profile.")
    (brew "2026-07-11" "Happy Ginger Ginger Beer" 12.0 (list (packaged
     "500mL PET" 24)) 3 () 9
      "Fresh+spent ginger hypothesis confirmed - clean ginger heat, no
     'Chinese food' note from brew 1. Lime and pineapple are subtle. Sugar
     at 105 g/L landed well. Yield 12L vs 9.44L planned (27% over).
     BACKSWEETENED at bottling - load-bearing for 9/10 result.")
    (brew "2026-07-12" "Experimental Root Beer 004: 1896 edition" 6.0 (list
     (packaged "500mL PET" 12)) 7 () 7
      "Brewed 2026-07-05 (kitchen-sink herb haul); bottled ~2026-07-12.
     Molasses later diagnosed as 100% the off-flavor (dropped in
     Traditional Root Beer v2). Selling as Experimental 004 / 1896
     edition.")))

(define rb/purchases
  (list
    (bought "Sassafras root bark" (qty 454 "g") 128.99 "2026-06-12"
      (list (source "Mountain Rose Herbs") (note "SKU 11-00466-17. $112.50
     + $16.49 shipping. Cut & sifted sassafras bark.")))
    (bought "Wintergreen leaf" (qty 454 "g") 39.78 "2026-06-14"
      (list (source "Mountain Rose Herbs") (note "2 x 8 oz bags. Product ID
     576812558. $19.89 each, free shipping.")))
    (bought "500mL PET bottles" (qty 96 "each") 112.32 "2026-06-14"
      (list (source "KegLand") (note "4 cases of 24. $119.96 - 10% discount
     = $108.00 + $4.32 tax. Free shipping.")))
    (bought "Echinacea root" (qty 300 "g") 5.00 "2026-07-05"
      (list (source "Local food co-op bulk bins") (note "Co-op bulk-bin
     pricing estimate ($1.67/100g).")))
    (bought "Ashwagandha root" (qty 300 "g") 5.00 "2026-07-05"
      (list (source "Local food co-op bulk bins") (note "Co-op bulk-bin
     pricing estimate ($1.67/100g).")))
    (bought "Spearmint leaf" (qty 300 "g") 5.00 "2026-07-05"
      (list (source "Local food co-op bulk bins") (note "Co-op bulk-bin
     pricing estimate ($1.67/100g).")))
    (bought "Chamomile flower" (qty 300 "g") 5.00 "2026-07-05"
      (list (source "Local food co-op bulk bins") (note "Co-op bulk-bin
     pricing estimate ($1.67/100g).")))
    (bought "Licorice root" (qty 300 "g") 5.00 "2026-07-05"
      (list (source "Local food co-op bulk bins") (note "Co-op bulk-bin
     pricing estimate ($1.67/100g).")))
    (bought "Hyssop herb" (qty 300 "g") 5.00 "2026-07-05"
      (list (source "Local food co-op bulk bins") (note "Co-op bulk-bin
     pricing estimate ($1.67/100g).")))))

;; ============================================================
;; ALL RECIPES registry
;; ============================================================

(define rb/all-recipes
  (list
```

```
    (cons "Simple Root Beer" rb/simple-root-beer)
    (cons "Ginger Bug Root Beer" rb/root-beer)
    (cons "Ginger Ale" rb/ginger-ale)
    (cons "Ginger Bug Lemonade" rb/lemonade)
    (cons "Ginger Bug Cola" rb/cola)
    (cons "Happy Ginger Ginger Beer" rb/happy-ginger)
    (cons "Traditional Root Beer" rb/traditional-root-beer)
    (cons "Experimental Root Beer 004: 1896 edition" rb/experimental-004)))

;; lookup recipe by name
(define (rb/find-recipe name)
  "Look up a recipe alist by name."
  (rb/get name rb/all-recipes))

;; calc one recipe with first package option + fixed costs
;; returns (name . report) or (name . error-string)
(define (rb/calc-one name)
  "One recipe's monthly report (or an error string)."
  (let ((r (rb/find-recipe name)))
    (if (null? r)
        (cons name "not found")
      (let ((pkgs (rb/get "packages" r)))
        (if (null? pkgs)
            (cons name "no packages")
          (let ((biz (rb/business r (car pkgs) rb/fixed-costs)))
            (cons name (rb/monthly-report biz rb/plan-config)))))))))

;; calc all recipes -> list of (name . report)
(define (rb/calc-all)
  "Monthly report for every recipe."
  (rb/map (lambda (pair) (rb/calc-one (car pair))) rb/all-recipes))
;; libs/rb/ui.pp -- rb route handlers (htmx, server-rendered)
;; Depends on: libs/rb/lib.pp, libs/rb/recipes.pp, html package

;; ============================================================
;; HTML HELPERS
;; ============================================================

(define (rb/h1 s) "Render an h1." (html/h1 s))
(define (rb/h2 s) "Render an h2." (html/h2 s))
(define (rb/h3 s) "Render an h3." (html/h3 s))
(define (rb/p s) "Render a paragraph." (html/tag "p" "" s))
(define (rb/td s) "Render a table cell." (html/tag "td" "" s))
(define (rb/th s) "Render a table header cell." (html/tag "th" "" s))

(define (rb/tr cells) "Render a table row from a list of cells." (html/tag
    "tr" "" (html/join cells)))
(define (rb/table rows) "Render a styled table." (html/tag "table"
    (html/attr "class" "rb-table") (html/join rows)))

(define (rb/li s) "Render a list item." (html/tag "li" "" s))
(define (rb/ul items) "Render a list from items." (html/tag "ul" ""
    (html/join items)))

(define (rb/card title body)
  "Render a titled card around a body."
  (html/div (html/attr "class" "card")
    (html/join (list (html/h3 title) body))))

(define (rb/link href text)
  "Render an anchor with href."
  (html/tag "a" (html/attr "href" href) text))

(define (rb/money-cell n)
  "Render a money value as a table cell."
  (rb/td (rb/money n)))

;; ============================================================
;; RECIPE LIST PAGE -- /recipes
;; ============================================================

(define (rb/page-recipes)
  "Recipes index page route handler."
```

```
  (seq
    (load-library "rb")
    (html/page
      (list
        (rb/h1 "Recipes")
        (rb/table
          (rb/map rb/recipe-row rb/all-recipes))))))

(define (rb/recipe-row pair)
  "HTML table row for one recipe with key figures."
  (let ((name (car pair))
        (r (cdr pair)))
    (let ((report (cdr (rb/calc-one name))))
      (rb/tr (list
        (rb/td (rb/link (string-append "/recipe?name=" (html-escape name)) name))
        (rb/td (rb/qty-str (rb/get "batch_liters" r)))
        (rb/td (as-str (rb/count (rb/get "costs" r))))
        (rb/money-cell (rb/get "variable_cost_per_unit" report))
        (rb/money-cell (rb/get "contribution_margin_per_unit" report))
        (rb/money-cell (rb/get "profit_per_month" report)))))))

;; ============================================================
;; RECIPE DETAIL PAGE -- /recipe?name=...
;; ============================================================

(define (rb/page-recipe-detail)
  "Recipe detail page route handler."
  (seq
    (load-library "rb")
    (let ((name (request-param "name")))
      (if (null? name)
          (html/page (list (rb/h1 "Recipe not found")))
        (let ((r (rb/find-recipe (as-str name))))
          (if (null? r)
              (html/page (list (rb/h1 "Recipe not found")))
            (html/page
              (list
                (rb/h1 (rb/get "name" r))
                (rb/recipe-detail-body r)))))))))

(define (rb/recipe-detail-body r)
  "Body of the recipe detail page (ingredients/steps/packages/report)."
  (html/join
    (list
      (rb/h2 "Ingredients")
      (rb/table (rb/ingredient-rows r))
      (rb/h2 "Steps")
      (rb/step-list r)
      (rb/h2 "Packages")
      (rb/table (rb/package-rows r))
      (rb/h2 "Monthly Report")
      (rb/report-table r))))

(define (rb/ingredient-rows r)
  "Header plus one row per ingredient cost."
  (let ((costs (rb/get "costs" r)))
    (cons
      (rb/tr (list (rb/th "Name") (rb/th "Per batch") (rb/th "Order")
                   (rb/th "Price") (rb/th "Line cost") (rb/th "Category")))
      (rb/map rb/ingredient-row costs))))

(define (rb/ingredient-row c)
  "HTML row for a single cost entry."
  (rb/tr (list
    (rb/td (rb/get "name" c))
    (rb/td (rb/qty-str (rb/get "qty_per_batch" c)))
    (rb/td (rb/qty-str (rb/get "order_qty" c)))
    (rb/money-cell (rb/get "order_price" c))
    (rb/money-cell (rb/line-cost c))
    (rb/td (rb/get "category" c)))))

(define (rb/step-list r)
```

```
  "Unordered list of a recipe's steps."
  (rb/ul (rb/map rb/step-item (rb/get "steps" r))))

(define (rb/step-item s)
  "List item for a single step with duration and note."
  (let ((name (rb/get "name" s))
        (dur (rb/get "duration" s))
        (note (rb/get "note" s)))
    (rb/li
      (string-append
        (string-append name " (" (rb/qty-str dur) ")")
        (if (null? note) "" (string-append " - " (as-str note)))))))

(define (rb/package-rows r)
  "Header plus one row per package option."
  (let ((pkgs (rb/get "packages" r)))
    (cons
      (rb/tr (list (rb/th "Package") (rb/th "Units/batch") (rb/th "L/unit")
     (rb/th "Price") (rb/th "Margin/unit")))
      (rb/map (lambda (pk) (rb/package-row r pk)) pkgs))))

(define (rb/package-row r pk)
  "HTML row for one package option and its margins."
  (let ((biz (rb/business r pk rb/fixed-costs)))
    (rb/tr (list
      (rb/td (rb/get "product_name" pk))
      (rb/td (as-str (rb/get "units_per_batch" pk)))
      (rb/td (rb/qty-str (rb/get "liters_per_unit" pk)))
      (rb/money-cell (rb/get "selling_price" pk))
      (rb/money-cell (rb/contribution-margin-per-unit biz))))))

(define (rb/report-table r)
  "Monthly report as a metric/value table."
  (let ((pkgs (rb/get "packages" r)))
    (if (null? pkgs)
        (rb/p "No packages configured")
      (let ((report (cdr (rb/calc-one (rb/get "name" r)))))
        (rb/table
          (list
            (rb/tr (list (rb/th "Metric") (rb/th "Value")))
            (rb/tr (list (rb/td "Units/month") (rb/td (as-str (rb/get
    "units_per_month" report)))))
            (rb/tr (list (rb/td "Revenue/month") (rb/money-cell (rb/get
    "revenue_per_month" report))))
            (rb/tr (list (rb/td "Variable cost/batch") (rb/money-cell
    (rb/get "variable_cost_per_batch" report))))
            (rb/tr (list (rb/td "Variable cost/unit") (rb/money-cell
    (rb/get "variable_cost_per_unit" report))))
            (rb/tr (list (rb/td "Fixed cost/month") (rb/money-cell (rb/get
    "fixed_cost_per_month" report))))
            (rb/tr (list (rb/td "Total cost/month") (rb/money-cell (rb/get
    "total_cost_per_month" report))))
            (rb/tr (list (rb/td "Profit/month") (rb/money-cell (rb/get
    "profit_per_month" report))))
            (rb/tr (list (rb/td "Margin/unit") (rb/money-cell (rb/get
    "contribution_margin_per_unit" report))))
            (rb/tr (list (rb/td "Breakeven units") (rb/td (as-str (rb/get
    "breakeven_units_per_month" report))))))))))

;; ============================================================
;; SHOPPING LIST PAGE -- /shopping?name=...
;; ============================================================

(define (rb/page-shopping)
  "Shopping-list page route handler."
  (seq
    (load-library "rb")
    (let ((name (request-param "name")))
      (if (null? name)
          (html/page (list (rb/h1 "Shopping list not found")))
        (let ((r (rb/find-recipe (as-str name))))
          (if (null? r)
              (html/page (list (rb/h1 "Recipe not found")))
```

```
            (html/page
              (list
                (rb/h1 (string-append "Shopping List: " (rb/get "name" r)))
                (rb/shopping-body r)))))))))

(define (rb/shopping-body r)
  "Shopping list table for one recipe."
  (let ((costs (rb/get "costs" r)))
    (rb/table
      (cons
        (rb/tr (list (rb/th "Ingredient") (rb/th "Order qty") (rb/th "Price") (rb/th "Source")))
        (rb/map rb/shopping-row costs)))))

(define (rb/shopping-row c)
  "HTML row for one shopping-list line."
  (let ((src (rb/get "source" c)))
    (rb/tr (list
      (rb/td (rb/get "name" c))
      (rb/td (rb/qty-str (rb/get "order_qty" c)))
      (rb/money-cell (rb/get "order_price" c))
      (rb/td (if (null? src) "" (as-str src)))))))

;; ============================================================
;; CAPITAL PAGE -- /capital
;; ============================================================

(define (rb/page-capital)
  "Capital plan page route handler."
  (seq
    (load-library "rb")
    (html/page
      (list
        (rb/h1 "Capital Plan")
        (rb/h2 "Equipment")
        (rb/equipment-table)
        (rb/h2 "Fixed Costs")
        (rb/fixed-cost-table)))))

(define (rb/equipment-table)
  "Equipment table with impact/unlocks columns."
  (rb/table
    (cons
      (rb/tr (list (rb/th "Priority") (rb/th "Item") (rb/th "Price") (rb/th "Impact") (rb/th "Unlocks")))
      (rb/map rb/equipment-row rb/equipment-list))))

(define (rb/equipment-row e)
  "HTML row for one equipment entry."
  (rb/tr (list
    (rb/td (as-str (rb/get "priority" e)))
    (rb/td (rb/get "name" e))
    (rb/money-cell (rb/get "price" e))
    (rb/td (let ((i (rb/get "impact" e))) (if (null? i) "" (as-str i))))
    (rb/td (let ((u (rb/get "unlocks" e))) (if (null? u) "" (as-str u)))))))

(define (rb/fixed-cost-table)
  "Fixed-cost summary table."
  (rb/table
    (cons
      (rb/tr (list (rb/th "Name") (rb/th "Monthly cost")))
      (rb/map rb/fixed-cost-row rb/fixed-costs))))

(define (rb/fixed-cost-row f)
  "HTML row for one fixed-cost entry."
  (rb/tr (list
    (rb/td (rb/get "name" f))
    (rb/money-cell (rb/get "monthly_cost" f)))))

;; ============================================================
;; LABEL PAGE -- /label?name=...
;; ============================================================

```

```
(define (rb/page-label)
  "Label page route handler (renders a print label)."
  (seq
    (load-library "rb")
    (let ((name (request-param "name")))
      (if (null? name)
          (html/page (list (rb/h1 "Label not found")))
        (let ((r (rb/find-recipe (as-str name))))
          (if (null? r)
              (html/page (list (rb/h1 "Recipe not found")))
            (render-html (rb/label-html r))))))))

(define (rb/label-html r)
  "Inline HTML for a printable product label."
  (let ((name (rb/get "name" r)))
    (string-append
      "<div style=\"width:4in;height:3in;border:2px solid
    #8B4513;border-radius:8px;padding:12px;font-family:serif;background:linear-gradient(135deg,#f5e6d3,#f0d5b8);display:fle
      "<div style=\"text-align:center\">"
      "<h1 style=\"font-size:18px;margin:4px 0;color:#5C3317\">" name
    "</h1>"
      "<p style=\"font-size:10px;margin:2px
    0;color:#8B4513;font-style:italic\">Ginger Bug Soda - Handcrafted in
    Wyoming</p>"
      "</div>"
      "<div style=\"text-align:center\">"
      "<p style=\"font-size:9px;margin:2px 0;color:#5C3317\">Made with
    fresh ginger bug starter</p>"
      "<p style=\"font-size:9px;margin:2px 0;color:#5C3317\">Keep
    refrigerated - Live culture</p>"
      "</div>"
      "<div style=\"text-align:center\">"
      "<p style=\"font-size:8px;margin:2px 0;color:#8B4513\">Not licensed
    or inspected per Wyo. Stat. 35-10-301</p>"
      "<p style=\"font-size:8px;margin:2px 0;color:#8B4513\">Pricklypear
    Soda Co.</p>"
      "</div>"
      "</div>")))

;; ============================================================
;; BATCHES PAGE -- /batches
;; ============================================================

(define (rb/page-batches)
  "Batch log page route handler."
  (seq
    (load-library "rb")
    (html/page
      (list
        (rb/h1 "Batch Log")
        (rb/batch-table)))))

(define (rb/batch-table)
  "Batch log table."
  (rb/table
    (cons
      (rb/tr (list (rb/th "Date") (rb/th "Recipe") (rb/th "Yield") (rb/th
    "Packaged") (rb/th "Rating")))
      (rb/map rb/batch-row rb/batches))))

(define (rb/batch-row b)
  "HTML row for one batch log entry."
  (rb/tr (list
    (rb/td (rb/get "date" b))
    (rb/td (rb/get "recipe" b))
    (rb/td (as-str (rb/get "yield_liters" b)))
    (rb/td (as-str (rb/count (rb/get "packaged" b))))
    (rb/td (as-str (rb/get "rating" b))))))

;; ============================================================
;; PURCHASES PAGE -- /purchases
;; ============================================================

```

```
(define (rb/page-purchases)
  "Purchase log page route handler."
  (seq
    (load-library "rb")
    (html/page
      (list
        (rb/h1 "Purchase Log")
        (rb/purchase-table)))))

(define (rb/purchase-table)
  "Purchase log table."
  (rb/table
    (cons
      (rb/tr (list (rb/th "Date") (rb/th "Item") (rb/th "Qty") (rb/th "Price") (rb/th "Source")))
      (rb/map rb/purchase-row rb/purchases))))

(define (rb/purchase-row p)
  "HTML row for one purchase entry."
  (let ((src (rb/get "source" p)))
    (rb/tr (list
      (rb/td (rb/get "date" p))
      (rb/td (rb/get "name" p))
      (rb/td (rb/qty-str (rb/get "qty" p)))
      (rb/money-cell (rb/get "price" p))
      (rb/td (if (null? src) "" (as-str src)))))))

;; ============================================================
;; ORDERS -- record actual sales (closes planned?actual loop)
;; ============================================================

(define (order customer date line-items opts)
  "Build an order alist."
  (cons (cons "type" "order")
  (cons (cons "customer" customer)
  (cons (cons "date" date)
  (cons (cons "line_items" line-items) opts)))))

(define (line-item recipe-name qty price)
  "Build an order line item alist."
  (list
    (cons "recipe" recipe-name)
    (cons "qty" qty)
    (cons "price" price)))

;; seed orders (empty -- no historical orders in OCaml rb)
(define rb/orders (list))

;; order persistence
(define (rb/init-order-storage!)
  "Ensure the order type exists."
  (if (null? (type-fields (quote order)))
      (deftype (quote order) (quote customer) (quote date) (quote line_items) (quote total) (quote created_day))
    ()))

(define (rb/save-order! ord)
  "Persist an order alist to the tree."
  (seq
    (rb/init-order-storage!)
    (create (quote order)
      (dict-set
        (dict-set
          (dict-set
            (dict-set
              (dict-set (json-emit "{}")
                "customer" (rb/get "customer" ord))
              "date" (rb/get "date" ord))
            "line_items" (json-emit (rb/get "line_items" ord)))
          "total" (as-str (rb/order-total ord)))
        "created_day" (date-today)))))

(define (rb/order-total ord)
```

```
  "Total revenue of an order's line items."
  (rb/sum (lambda (li) (* (rb/get "qty" li) (rb/get "price" li)))
          (rb/get "line_items" ord)))

(define (rb/order-rows)
  "All persisted order rows."
  (rows (quote order)))

;; ============================================================
;; PLANNED VS ACTUAL
;; ============================================================

;; Compare planned profit (from calc) vs actual revenue (from orders).
;; Returns ((planned_revenue . X) (actual_revenue . Y) (variance . Z))
(define (rb/planned-vs-actual)
  "Planned vs actual revenue with variance."
  (let ((planned (rb/sum (lambda (p) (rb/get "revenue_per_month" (cdr p)))
     (rb/calc-all))))
    (let ((actual (rb/actual-revenue)))
      (list
        (cons "planned_revenue" planned)
        (cons "actual_revenue" actual)
        (cons "variance" (- actual planned))))))

(define (rb/actual-revenue)
  "Sum of totals across all order rows."
  (rb/sum (lambda (row) (parse-float (dict-get (cdr row) "total")))
     (rb/order-rows)))

;; ============================================================
;; ORDERS PAGE -- /orders
;; ============================================================

(define (rb/page-orders)
  "Orders page route handler (table + planned vs actual)."
  (seq
    (load-library "rb")
    (html/page
      (list
        (rb/h1 "Orders")
        (rb/order-table)
        (rb/h2 "Planned vs Actual")
        (rb/pva-table)))))

(define (rb/order-table)
  "Orders table, or a note when none exist."
  (let ((rws (rb/order-rows)))
    (if (null? rws)
        (rb/p "No orders yet. Sell some soda!")
      (rb/table
        (cons
          (rb/tr (list (rb/th "Date") (rb/th "Customer") (rb/th "Total")))
          (rb/map rb/order-data-row rws))))))

(define (rb/order-data-row row)
  "HTML row for one persisted order."
  (let ((f (cdr row)))
    (rb/tr (list
      (rb/td (dict-get f "date"))
      (rb/td (dict-get f "customer"))
      (rb/money-cell (parse-float (dict-get f "total")))))))

(define (rb/pva-table)
  "Planned-vs-actual metric table."
  (let ((pva (rb/planned-vs-actual)))
    (rb/table
      (list
        (rb/tr (list (rb/th "Metric") (rb/th "Value")))
        (rb/tr (list (rb/td "Planned revenue/month") (rb/money-cell (rb/get
    "planned_revenue" pva))))
        (rb/tr (list (rb/td "Actual revenue (all orders)") (rb/money-cell
    (rb/get "actual_revenue" pva))))
```

```
        (rb/tr (list (rb/td "Variance") (rb/money-cell (rb/get "variance" pva))))))))

;; ============================================================
;; REPORT PAGE -- /report (multi-recipe business overview)
;; ============================================================

(define (rb/page-report)
  "Business report page route handler."
  (seq
    (load-library "rb")
    (html/page
      (list
        (rb/h1 "Business Report")
        (rb/h2 "Monthly Profit by Recipe")
        (rb/report-summary-table)
        (rb/h2 "Plan")
        (rb/plan-table)))))

(define (rb/report-summary-table)
  "Monthly profit-by-recipe summary table."
  (rb/table
    (cons
      (rb/tr (list (rb/th "Recipe") (rb/th "Revenue/mo") (rb/th "Cost/mo") (rb/th "Profit/mo") (rb/th "Margin/unit")))
      (rb/map rb/report-summary-row (rb/calc-all)))))

(define (rb/report-summary-row pair)
  "HTML row for one recipe's report summary."
  (let ((name (car pair)) (report (cdr pair)))
    (rb/tr (list
      (rb/td (rb/link (string-append "/recipe?name=" (html-escape name)) name))
      (rb/money-cell (rb/get "revenue_per_month" report))
      (rb/money-cell (rb/get "total_cost_per_month" report))
      (rb/money-cell (rb/get "profit_per_month" report))
      (rb/money-cell (rb/get "contribution_margin_per_unit" report))))))

(define (rb/plan-table)
  "Plan configuration summary table."
  (rb/table
    (list
      (rb/tr (list (rb/th "Metric") (rb/th "Value")))
      (rb/tr (list (rb/td "Batches/month") (rb/td (as-str (rb/get "batches_per_month" rb/plan-config)))))
      (rb/tr (list (rb/td "Starter recipe") (rb/td (rb/get "starter_recipe" rb/plan-config)))))))

;; ============================================================
;; ROUTES
;; ============================================================

(route "/recipes" "rb/page-recipes")
(route "/recipe" "rb/page-recipe-detail")
(route "/shopping" "rb/page-shopping")
(route "/capital" "rb/page-capital")
(route "/label" "rb/page-label")
(route "/batches" "rb/page-batches")
(route "/purchases" "rb/page-purchases")
(route "/orders" "rb/page-orders")
(route "/report" "rb/page-report")
```

194 libs/ref/lib.pp

```
;; package ref -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push

(define ref/fn-li
  (lambda
    (fn)
    "One reference list item: function name plus its docstring when
     present."
    (let
      ((doc (function-doc (as-str fn))))
      (if
        (null? doc)
        (html/li "" (html/code (as-str fn)))
        (html/li
          ""
          (html/join
            (list
              (html/code (as-str fn))
              (html/span
                (html/attr "class" "muted")
                (string-append " -- " (as-str doc))))))))))

(define ref/packages
  (lambda
    ()
    "Distinct package prefixes over all visible bindings."
    (list/foldl
      (lambda
        (acc name)
        (let
          ((p (prefix-of name)))
          (if
            (null?
              (list/filter (lambda (x) (string-eq (as-str x) p)) acc))
            (cons p acc)
            acc)))
      (list)
      (list-bindings))))

(define ref/page
  (lambda
    ()
    "Function Reference page: search box, package groups, or matching rows."
    (let
      ((q (request-param "q")) (all (list-bindings)))
      (let
        ((needle (if (null? q) "" (as-str q))))
        (let
          ((fns (if (string-eq needle "") all (apropos needle))))
          (if
            (string-eq needle "")
            (html/page
              (list
                (html/h1 "Function Reference")
                (html/p*
                  (html/attr "class" "muted")
                  (string-append
                    (as-str (count fns))
                    " of "
                    (as-str (count all))
                    " functions. Search with ?q= or (apropos \"term\"). "))
                (html/form
                  "GET"
                  "/ref"
                  ""
                  (html/join
                    (list
                      (html/text-input
                        "q"
                        (html/attrs
                          (list
```

```
                          (html/attr "placeholder" "apropos?"))))
                    (html/button (html/attr "type" "submit") "Search"))))
                (html/div
                  (html/attr "class" "ref-packages")
                  (html/join
                    (list/map
                      ref/render-pkg
                      (list/filter
                        (lambda (p) (not (string-eq (as-str p) "")))
                        (ref/packages)))))))
            (html/page
              (list
                (html/h1 "Function Reference")
                (html/p*
                  (html/attr "class" "muted")
                  (string-append
                    (as-str (count fns))
                    " of "
                    (as-str (count all))
                    " functions matching \""
                    needle
                    "\". "))
                (html/form
                  "GET"
                  "/ref"
                  ""
                  (html/join
                    (list
                      (html/text-input
                        "q"
                        (html/attrs
                          (list
                            (html/attr "value" needle)
                            (html/attr "placeholder" "apropos?"))))
                      (html/button (html/attr "type" "submit") "Search"))))
                (html/ul "" (html/join (list/map ref/fn-li fns)))))))))))

(define ref/render-pkg
  (lambda
    (pkg)
    "Card listing the functions whose prefix equals pkg."
    (let
      ((fns
        (list/filter
          (lambda (n) (string-eq (prefix-of n) pkg))
          (list-bindings))))
      (html/div
        (html/attr "class" "card")
        (html/join
          (list
            (html/h2 (html/text pkg))
            (html/p*
              (html/attr "class" "muted")
              (string-append (as-str (count fns)) " functions"))
            (html/ul "" (html/join (list/map ref/fn-li fns)))))))))
```

195 libs/repl/lib.pp

```
;; Advanced browser REPL (multipart.repl-v2) -- one input, multipart transcript.
;; Dogfood for multipart/mixed. Load: (load-library "repl")
;; Optionally notes/ai. Provides a tiny dict-set* fallback when std seed
;; skipped it (list/map special-form shadow aborts mid-std load).

(package repl
  (export
    repl/page
    repl/send
    repl/answer
    repl/command
    repl/help-text))

(define repl/help-text
  (lambda ()
    "Commands: /note /eval /describe /branch /help -- bare (? evals Nopales; plain text asks the LLM. Extend via (define (repl/cmd-NAME args) ?) and (repl/command \"NAME\").")
  "Help blurb for /help and empty input.")

(define repl/part-html
  (lambda (html)
    (part html :target "#repl-transcript" :swap "beforeend"))
  "One transcript-append HTML part.")

(define repl/bubble
  (lambda (kind body)
    (string-append
      "<div class=\"repl-line repl-" (as-str kind) "\">"
      "<span class=\"repl-kind\">" (html-escape (as-str kind)) "</span> "
      body
      "</div>"))
  "Transcript bubble HTML.")

(define repl/text-bubble
  (lambda (kind text)
    (repl/bubble kind (html-escape (as-str text))))
  "Escaped text bubble.")

(define repl/starts-with?
  (lambda (s prefix)
    (let ((s (as-str s)) (p (as-str prefix)))
      (if (< (string-length s) (string-length p))
          #f
          (string-eq (substring s 0 (string-length p)) p))))
  "Prefix check.")

(define repl/split1
  (lambda (s)
    (let ((s (as-str s))
          (slen (string-length (as-str s))))
      (let ((rec
              (lambda (self i)
                (if (>= i slen)
                    -1
                    (if (string-eq (substring s i 1) " ")
                        i
                        (self self (+ i 1)))))))
        (let ((pos (rec rec 0)))
          (if (< pos 0)
              (list s "")
              (list
                (substring s 0 pos)
                (string-trim
                  (substring s (+ pos 1) slen))))))))
  "Split on first space ? (head rest).")

;; e is (error msg kind) list from guard, or any value
(define repl/err-msg
  (lambda (e)
```

```
    (if (pair? e)
        (if (null? (cdr e))
            (str e)
            (as-str (car (cdr e))))
        (str e)))
  "Extract message from guard error value.")

;; Fallback if std dict-set* missing (notes/capture needs it).
(define dict-set*
  (lambda (j pairs)
    (if (null? pairs)
        j
        (dict-set*
          (dict-set j (car pairs) (car (cdr pairs)))
          (cdr (cdr pairs))))))
  "Multi dict-set: (dict-set* json (list k1 v1 k2 v2 ?)).")

;; --- built-in commands ---

(define repl/cmd-help
  (lambda (args)
    (list (repl/part-html (repl/text-bubble "help" (repl/help-text)))))
  "/help")

(define repl/cmd-branch
  (lambda (args)
    (list
      (repl/part-html
        (repl/text-bubble "branch"
          (string-append (whoami) " @ " (current-branch))))))
  "/branch")

(define repl/cmd-describe
  (lambda (args)
    (let ((pkg (string-trim (as-str args))))
      (if (string-eq pkg "")
          (list (repl/part-html (repl/text-bubble "error" "/describe needs a package name")))
          (guard
            (lambda (e)
              (list (repl/part-html
                      (repl/text-bubble "error"
                        (string-append "describe failed: " (repl/err-msg e))))))
            (load-library pkg)
            (list
              (repl/part-html
                (repl/bubble "describe"
                  (string-append "<pre>" (html-escape (str (describe pkg)))
                                 "</pre>")))))))))
  "/describe <pkg>")

(define repl/cmd-eval
  (lambda (args)
    (repl/eval-expr (as-str args)))
  "/eval <expr>")

(define repl/cmd-note
  (lambda (args)
    (let ((text (string-trim (as-str args))))
      (if (string-eq text "")
          (list (repl/part-html (repl/text-bubble "error" "/note needs text")))
          (guard
            (lambda (e)
              (list (repl/part-html
                      (repl/text-bubble "error"
                        (string-append "note failed: " (repl/err-msg e))))))
            ;; Prefer already-loaded notes; avoid nested load-library under the
            ;; request rim (can hang on heavy package reload).
            (if (null?
                  (list/filter
```

```
                    (lambda (b) (string-eq (as-str b) "notes/capture"))
                    (list-bindings)))
                (load-library "notes")
                #t)
            (notes/capture "" text)
            (list
              (repl/part-html
                (repl/text-bubble "note"
                  (string-append "captured: " text)))))))
  "/note <text> -- quick capture into notes inbox")

(define repl/command
  (lambda (name)
    (as-str name))
  "Declare a custom slash command name. Implement as (define (repl/cmd-NAME
     args) ?) returning a list of parts.")

(define repl/run-command
  (lambda (name args)
    (let ((n (as-str name)))
      (cond
        ((string-eq n "help") (repl/cmd-help args))
        ((string-eq n "branch") (repl/cmd-branch args))
        ((string-eq n "describe") (repl/cmd-describe args))
        ((string-eq n "eval") (repl/cmd-eval args))
        ((string-eq n "note") (repl/cmd-note args))
        (else
          (list
            (repl/part-html
              (repl/text-bubble "error"
                (string-append "unknown command /" n " -- try /help")))))))
  "Dispatch slash command.")

(define repl/eval-expr
  (lambda (src)
    (guard
      (lambda (e)
        (list
          (repl/part-html
            (repl/bubble "error"
              (string-append "<pre class=\"repl-err\">"
                             (html-escape (repl/err-msg e))
                             "</pre>")))))
      (let ((v (eval-string (as-str src))))
        (list
          (repl/part-html
            (repl/bubble "value"
              (string-append "<pre>" (html-escape (str v)) "</pre>")))))))
  "Eval Nopales source; return value or error parts.")

(define repl/llm-answer
  (lambda (q)
    (guard
      (lambda (e)
        (list
          (repl/part-html
            (repl/text-bubble "error"
              (string-append "LLM failed: " (repl/err-msg e))))))
      (load-library "ai")
      (let ((ans (ai/ask q)))
        (list
          (repl/part-html
            (repl/bubble "llm"
              (string-append "<div class=\"repl-llm\">"
                             (html-escape (as-str ans))
                             "</div>")))))))
  "LLM question via libs/ai (eager part; token-delta stream deferred).")

(define repl/answer
  (lambda (raw)
    (let ((input (string-trim (as-str raw))))
      (cond
        ((string-eq input "")
```

```
          (list (repl/part-html (repl/text-bubble "help"
    (repl/help-text)))))
        ((repl/starts-with? input "/")
          (let ((pair (repl/split1 (substring input 1 (string-length
    input)))))
            (repl/run-command (car pair) (car (cdr pair)))))
        ((repl/starts-with? input "(")
          (repl/eval-expr input))
        (else (repl/llm-answer input)))))
  "Dispatch one input line ? list of parts.")

(define repl/user-echo
  (lambda (input)
    (if (string-eq input "")
        '()
        (list (repl/part-html
               (repl/bubble "user"
                 (string-append "<div class=\"repl-user\">"
                                (html-escape input)
                                "</div>")))))))
  "User input echo bubble; empty for blank input (help path).")

(define repl/send
  (lambda ()
    (let ((raw (as-str (request-param "q"))))
      (let ((input (string-trim raw)))
        (let ((echo (repl/user-echo input))
              (resp (repl/answer input)))
          (multipart :target "#repl-transcript" :swap "beforeend"
            (append echo resp)))))))
  "POST handler -- user echo + multipart transcript parts.")

(define repl/page
  (lambda ()
    (render-html
      (string-append
        "<div class=\"repl-v2 card\">"
        "<h1>REPL</h1>"
        "<p class=\"muted\">Eval, slash commands, or ask the LLM. "
        "Responses append to the transcript (multipart).</p>"
        "<div id=\"repl-transcript\" class=\"repl-transcript\" "
        "aria-live=\"polite\"></div>"
        "<form class=\"repl-form\""
        " hx-post=\"/repl-v2/send\""
        " hx-ext=\"hx-multipart\""
        " hx-config='{\"timeout\":0}'"
        " hx-target=\"#repl-transcript\""
        " hx-swap=\"beforeend\""
        " hx-on::htmx:config-request=\"let
    q=(event.detail.parameters.q||'');if(q[0]!='/'&&q[0]!='('){event.detail.path='/repl-v2/llm';}\""
        " hx-on::before-request=\"this.reset()\""
        " hx-on::after-request=\"this.reset()\">"
        "<label class=\"sr-only\" for=\"repl-q\">Input</label>"
        "<input id=\"repl-q\" name=\"q\" type=\"text\" "
        "placeholder=\"(+ 20 22)  ?  /note hi  ?  what packages?\" "
        "autocomplete=\"off\" autofocus />"
        "<button type=\"submit\">Send</button>"
        "</form>"
        "<p class=\"muted small\">" (html-escape (repl/help-text)) "</p>"
        "<script src=\"/assets/hx-multipart.js\"></script>"
        "<style>"
        ".repl-transcript{min-height:12rem;max-height:50vh;overflow:auto;"
        "border:1px solid #ccc;padding:.5rem;margin:.5rem
    0;background:#fafafa}"
        ".repl-line{margin:.35rem 0;padding:.35rem .5rem;border-left:3px
    solid #888}"

    ".repl-value{border-color:#2a7}.repl-error{border-color:#a22;background:#fff5f5}"

    ".repl-llm{border-color:#26a}.repl-note{border-color:#a62}.repl-user{border-color:#555;background:#f0f0f0;font-style:it

    ".repl-kind{font-size:.75rem;text-transform:uppercase;color:#666;margin-right:.35rem}"
        ".repl-form{display:flex;gap:.5rem}.repl-form input{flex:1}"
```

```
275
      ".sr-only{position:absolute;width:1px;height:1px;overflow:hidden;clip:rect(0,0,0,0)}"
276         "</style>"
277         "</div>")))
278   "GET /repl-v2 page.")
279
280 (route "/repl-v2" "repl/page")
281 (route "/repl-v2/send" "repl/send")
```

196 libs/replication/lib.pp

```
;; package replication -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push
```

197 libs/roam/lib.pp

```
;; package roam -- exported from live branch admin/personal
;; Sync: scripts/pp_sync.py pull|push

(define roam/at-open?
  (lambda
    (s i)
    "True when s has a [[ at index i."
    (if
      (>= i (- (string-length s) 3))
      #f
      (if
        (string-eq (substring s i 1) "[")
        (string-eq (substring s (+ i 1) 1) "[")
        #f))))

(define roam/backlinks
  (lambda
    (slug)
    "Rows linking to slug (by slug or uuid), newest first."
    (let
      ((target (as-str slug)) (tu (roam/note-uuid slug)))
      (list/map
        (lambda
          (row)
          (let
            ((f (cdr row)))
            (let
              ((fs (as-str (dict-get f "from_slug")))
                (fid (as-str (dict-get f "from_id")))
                (lab (as-str (dict-get f "label"))))
              (let
                ((src (notes/doc/get fs)))
                (list
                  fs
                  fid
                  (if (null? src) fs (notes/doc/title src))
                  lab)))))
        (if
          (string-eq tu "")
          (find-rows (quote note-link) "to_slug" target 200)
          (let
            ((by-uuid (find-rows (quote note-link) "to_uuid" tu 200)))
            (if
              (null? by-uuid)
              (find-rows (quote note-link) "to_slug" target 200)
              by-uuid)))))))

(define roam/backlinks-html
  (lambda
    (slug)
    "HTML list of backlinks for slug."
    (let
      ((bls (roam/backlinks slug)))
      (if
        (null? bls)
        (html/p*
          (html/attr "class" "muted")
          "No backlinks yet.")
        (html/ul
          (html/attr "class" "roam-backlinks")
          (html/join
            (list/map
              (lambda
                (b)
                (html/li
                  ""
                  (html/join
                    (list
                      (html/a
                        (string-append "/notes?slug=" (car b))
                        (html/text (car (cdr (cdr b))))))
```

```
                      (html/span
                        (html/attr "class" "muted")
                        (html/text (string-append " ? " (car (cdr b))))))))))
              bls)))))))

(define roam/collect-from-tree
  (lambda (tree) "All wikilink triples across an outline tree."
    (roam/collect-node tree (list))))

(define roam/collect-node
  (lambda
    (n acc)
    "Accumulate wikilink triples (id target label) from node n and kids."
    (let
      ((id (as-str (outline/id n)))
        (links (roam/parse-links (outline/body n))))
      (let
        ((acc1
          (list/foldl
            (lambda
              (a pair)
              (cons (list id (car pair) (car (cdr pair))) a))
            acc
            links)))
        (list/foldl
          (lambda (a k) (roam/collect-node k a))
          acc1
          (outline/kids n))))))

(define roam/delete-from-slug
  (lambda
    (slug)
    "Delete all note-link rows whose from_slug is slug."
    (list/foldl
      (lambda
        (_ row)
        (seq (delete (quote note-link) (car row)) 0))
      0
      (find-rows
        (quote note-link)
        "from_slug"
        (as-str slug)
        500))))

(define roam/find-close
  (lambda
    (s i)
    "Index of the ]] closing the wikilink starting at i, or ()."
    (if
      (>= i (- (string-length s) 1))
      ()
      (if
        (if
          (string-eq (substring s i 1) "]")
          (string-eq (substring s (+ i 1) 1) "]")
          #f)
        i
        (roam/find-close s (+ i 1))))))

(define roam/insert-collected
  (lambda
    (from-slug triples seen)
    "Insert wikilink rows for triples from from-slug, returning seen keys."
    (let
      ((from-uuid (roam/note-uuid from-slug)))
      (roam/insert-collected-acc
        from-slug
        from-uuid
        triples
        seen))))

(define roam/insert-collected-acc
  (lambda
```

```
    (from-slug from-uuid triples seen)
    "Accumulator for insert-collected, skipping media and duplicates."
    (if
      (null? triples)
      seen
      (let
        ((t (car triples)))
        (let
          ((fid (as-str (car t)))
            (to (as-str (car (cdr t))))
            (lab (as-str (car (cdr (cdr t))))))
          (let
            ((key (string-append fid "\t" to)))
            (if
              (roam/media-slug? to)
              (roam/insert-collected-acc
                from-slug
                from-uuid
                (cdr triples)
                seen)
              (if
                (roam/seen? seen key)
                (roam/insert-collected-acc
                  from-slug
                  from-uuid
                  (cdr triples)
                  seen)
                (let
                  ((to-uuid (roam/note-uuid to)))
                  (seq
                    (create
                      (quote note-link)
                      (dict-set*
                        "{}"
                        (list
                          "from_slug"
                          (as-str from-slug)
                          "from_uuid"
                          (as-str from-uuid)
                          "from_id"
                          fid
                          "to_slug"
                          to
                          "to_uuid"
                          (as-str to-uuid)
                          "to_id"
                          ""
                          "label"
                          lab
                          "kind"
                          "wikilink")))
                    (roam/insert-collected-acc
                      from-slug
                      from-uuid
                      (cdr triples)
                      (cons key seen)))))))))))

(define roam/media-id
  (lambda
    (slug)
    "Strip the media: prefix from a media slug."
    (let
      ((s (as-str slug)))
      (substring s 6 (- (string-length s) 6)))))

(define roam/media-slug?
  (lambda
    (slug)
    "True when slug carries the media: prefix."
    (let
      ((s (as-str slug)))
      (if
        (< (string-length s) 7)
```

```
        #f
        (string-eq (substring s 0 6) "media:")))))

(define roam/note-uuid
  (lambda
    (slug)
    "Row id (uuid) of a note slug, or \"\" when absent."
    (let
      ((row (notes/doc/row (as-str slug))))
      (if (null? row) "" (as-str (car row))))))

(define roam/parse-links
  (lambda (body) "Extract wikilink pairs from body text." (extract-links
     (as-str body))))

(define roam/parse-links-acc
  (lambda
    (s i acc)
    "Accumulator for parse-links scanning s from index i."
    (if
      (>= i (- (string-length s) 3))
      (reverse acc)
      (if
        (if
          (string-eq (substring s i 1) "[")
          (string-eq (substring s (+ i 1) 1) "[")
          #f)
        (let
          ((end (roam/find-close s (+ i 2))))
          (if
            (null? end)
            (roam/parse-links-acc s (+ i 1) acc)
            (let
              ((inner (substring s (+ i 2) (- end (+ i 2)))))
              (let
                ((pair (roam/split-label inner)))
                (if
                  (string-eq (car pair) "")
                  (roam/parse-links-acc s (+ end 2) acc)
                  (roam/parse-links-acc s (+ end 2) (cons pair acc)))))))
        (roam/parse-links-acc s (+ i 1) acc)))))

(define roam/reindex
  (lambda
    (slug outline-tree)
    "Rebuild all wikilink rows for slug from its outline tree."
    (seq
      (roam/delete-from-slug slug)
      (roam/insert-collected
        slug
        (roam/collect-from-tree outline-tree)
        (list))
      slug)))

(define roam/render-acc
  (lambda
    (s i acc)
    "Render wikilinks in s to HTML, appending to acc."
    (if
      (>= i (string-length s))
      acc
      (if
        (roam/at-open? s i)
        (let
          ((end (roam/find-close s (+ i 2))))
          (if
            (null? end)
            (roam/render-acc
              s
              (+ i 1)
              (string-append acc (html-escape (substring s i 1))))
            (let
              ((inner (substring s (+ i 2) (- end (+ i 2)))))
```

```
              (let
                ((pair (roam/split-label inner)))
                (let
                  ((slug (car pair)) (lab (car (cdr pair))))
                  (if
                    (string-eq slug "")
                    (roam/render-acc
                      s
                      (+ end 2)
                      (string-append
                        acc
                        (html-escape (substring s i (- (+ end 2) i)))))
                    (roam/render-acc
                      s
                      (+ end 2)
                      (string-append acc (roam/render-wiki-or-media slug
    lab)))))))))
        (roam/render-acc
          s
          (+ i 1)
          (string-append acc (html-escape (substring s i 1))))))))

(define roam/render-body
  (lambda (body) "Render wikilinks in body to HTML." (render-links (as-str
    body))))

(define roam/render-link
  (lambda
    (slug lab)
    "HTML anchor for a wikilink to slug with label lab."
    (let
      ((s (as-str slug)) (row (notes/doc/row slug)))
      (let
        ((href-slug
           (if (null? row) s (as-str (dict-get (cdr row) "slug"))))
          (title
            (if
              (null? row)
              (as-str lab)
              (let
                ((t (dict-get (cdr row) "title")))
                (if
                  (or (null? t) (string-eq (as-str lab) s))
                  (if (null? t) (as-str lab) (as-str t))
                  (as-str lab))))))
        (html/a
          (string-append "/notes?slug=" href-slug)
          (html/text title))))))

(define roam/render-media
  (lambda
    (id alt)
    "HTML image (with caption) for a media id."
    (let
      ((mid (as-str id)) (lab (as-str alt)))
      (let
        ((href (string-append "/media/" mid))
          (label (if (string-eq lab "") mid lab)))
        (html/join
          (list
            (html/a href (html/img href label "note-media"))
            (html/p*
              (html/attr "class" "muted note-media-cap")
              (html/a href (html/text label)))))))))

(define roam/render-wiki-or-media
  (lambda
    (slug lab)
    "Render a wikilink as media or a note link."
    (if
      (roam/media-slug? slug)
      (roam/render-media (roam/media-id slug) lab)
      (roam/render-link slug lab))))
```

```
360
361 (define roam/seen?
362   (lambda
363     (seen key)
364     "True when key is already in the seen list."
365     (if
366       (null? seen)
367       #f
368       (if
369         (string-eq (car seen) key)
370         #t
371         (roam/seen? (cdr seen) key)))))
372
373 (define roam/split-label
374   (lambda
375     (inner)
376     "Split [[inner]] into (slug label) on the first |."
377     (let
378       ((s (string-trim (as-str inner))))
379       (roam/split-label-at s 0))))
380
381 (define roam/split-label-at
382   (lambda
383     (s i)
384     "Accumulator for split-label scanning s from index i."
385     (if
386       (>= i (string-length s))
387       (list s s)
388       (if
389         (string-eq (substring s i 1) "|")
390         (list
391           (string-trim (substring s 0 i))
392           (string-trim
393             (substring s (+ i 1) (- (string-length s) (+ i 1)))))
394         (roam/split-label-at s (+ i 1))))))
```

198 libs/route/lib.pp

```
;; package route -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push
```

199 libs/signup/lib.pp

```
;; package signup -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push

(define signup/blank?
  (lambda (s) "True if trimmed s is blank." (blank? (string-trim (as-str
    s)))))

(define signup/fields
  (lambda
    (email note source ua)
    "Build the signup fields JSON string."
    (dict-set*
      "{}"
      (list
        "email"
        (string-trim (as-str email))
        "note"
        (as-str note)
        "source"
        (as-str source)
        "user_agent"
        (as-str ua)))))

(define signup/form
  (lambda
    (msg err)
    "Render the email signup form page with optional msg and err notices."
    (layout/page
      (list
        (layout/hero
          "Stay in the loop"
          "Drop your email if you want updates when Pricklypear grows. No
    spam, no list-sold nonsense -- just a note in my inbox."
          (html/join
            (list
              (layout/cta "/welcome" "Welcome")
              (layout/cta "/background" "Background")
              (layout/cta "/login" "Log in"))))
        (if
          (signup/blank? err)
          ""
          (html/p* (html/attr "class" "err") (html/text err)))
        (if
          (signup/blank? msg)
          ""
          (html/p* (html/attr "class" "ok") (html/text msg)))
        (layout/section
          "Email signup"
          (html/form
            "POST"
            "/signup"
            (html/attr "class" "signup-form")
            (html/join
              (list
                (html/p "Email")
                (html/text-input
                  "email"
                  (html/attrs
                    (list
                      (html/attr "required" "required")
                      (html/attr "placeholder" "you@example.com")
                      (html/attr "autocomplete" "email"))))
                (html/p "Optional note")
                (html/textarea
                  "note"
                  (html/attrs
                    (list
                      (html/attr "rows" "3")
                      (html/attr "placeholder" "What brought you here?")))
                  "")
                (html/hidden "source" "web")
```

```
                (html/div
                  ""
                  (html/button
                    (html/attrs (list (html/attr "type" "submit")))
                    "Sign up"))))))
        (html/div
          (html/attr "class" "layout-foot")
          (html/join
            (list
              (html/a "/welcome" (html/text "welcome"))
              (html/text " ? ")
              (html/a "/roadmap" (html/text "roadmap")))))))))

(define signup/looks-like-email?
  (lambda
    (email)
    "True if email looks like a plausible address (has @ and length > 3)."
    (let
      ((e (string-trim (as-str email))))
      (if
        (signup/blank? e)
        #f
        (if
          (string-contains? e "@")
          (if (> (string-length e) 3) #t #f)
          #f)))))

(define signup/notify-inbox
  (lambda
    (email note)
    "Capture a signup notification note when notes/capture add is
     available."
    (let
      ((title
          (string-append "Signup: " (string-trim (as-str email))))
        (body
          (string-append
            "New public signup\n"
            "email: "
            (string-trim (as-str email))
            "\n"
            "note: "
            (as-str note)
            "\n"
            "source: web")))
      (if
        (string-eq (type-of notes/capture/add) "lambda")
        (notes/capture/add title body)
        "no-notes"))))

(define signup/page
  (lambda
    ()
    "Route handler for /signup: record POSTs and render the page otherwise."
    (if
      (string-eq (request-method) "POST")
      (let
        ((email (request-param "email"))
          (note (request-param "note")))
        (if
          (signup/looks-like-email? email)
          (seq
            (with-public-write
              (lambda () (signup/record email note)))
            (signup/thanks email))
          (signup/form "" "Need a real-looking email (with @).")))
      (signup/form "" ""))))

(define signup/record
  (lambda
    (email note)
    "Create a signup row, notify the inbox, and return email."
    (let
```

```
      ((fields (signup/fields email note "web" "")))
      (seq
        (create (quote signup) fields)
        (seq (signup/notify-inbox email note) email)))))

(define signup/thanks
  (lambda
    (email)
    "Render the post-signup thank-you page for email."
    (layout/page
      (list
        (layout/hero
          "You're on the list"
          (string-append
            "Thanks -- "
            (as-str email)
            ".")
          (html/join
            (list
              (layout/cta "/welcome" "Back to welcome")
              (layout/cta "/code" "Browse the source")
              (layout/cta "/login" "Log in"))))
        (html/div
          (html/attr "class" "layout-foot")
          (html/a "/welcome" (html/text "welcome")))))))

;; Public signup form (hard-public path in http.ml).
(public-route "/signup" "signup/page")
```

200 libs/soup/lib.pp

```
;; package soup -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push
```

201 libs/spec/lib.pp

```
;; package spec -- habitat-workflow lint queries over notes docs + bindings.
;; Spec: borge/habitat-workflow.borg (spec-checks / session-ritual).
;; Load: (load-library "spec") after vault/seed, or load-string this file.
;; Depends on notes, outline, list (and string-split / string-downcase prims).
;;
;; Note: (package ?) form is omitted so load-string works on images whose
;; eval path already has package intercept only via load-library; exports
;; are prefix-convention (spec/*). Re-add (package spec (export ?)) when
;; seeding via load-library on a current image.


(define spec/doc-slug
  (lambda (pkg)
    (string-append "spec/" (as-str pkg)))
  "Slug of the package intent notes doc.")

(define spec/doc
  (lambda (pkg)
    (notes/doc/get (spec/doc-slug pkg)))
  "notes/doc/get for spec/<pkg>, or ().")

(define spec/member?
  (lambda (x xs)
    (if (null? xs)
        #f
        (if (string-eq (as-str (car xs)) (as-str x))
            #t
            (spec/member? x (cdr xs)))))
  "Membership test on a list of strings.")

(define spec/unique
  (lambda (xs)
    (list/foldl
      (lambda (acc x)
        (if (spec/member? x acc) acc (cons (as-str x) acc)))
      (list)
      xs))
  "Dedup preserving reverse encounter order.")

(define spec/append
  (lambda (a b)
    (if (null? a) b (cons (car a) (spec/append (cdr a) b))))
  "Concatenate two lists.")

(define spec/outline-text
  (lambda (n)
    (let ((here
            (string-append
              (as-str (outline/title n))
              " "
              (as-str (outline/body n))
              " "))
          (kids (outline/kids n)))
      (list/foldl
        (lambda (acc k) (string-append acc (spec/outline-text k)))
        here
        kids)))
  "Flatten outline titles+bodies into one searchable string.")

(define spec/find-section
  (lambda (ol needle)
    (let ((want (string-downcase (as-str needle))))
      (car
        (list/filter
          (lambda (k)
            (or (string-eq (string-downcase (as-str (outline/id k))) want)
                (string-eq (string-downcase (as-str (outline/title k))) want)
                (string-contains?
                  (string-downcase (as-str (outline/title k)))
```

```
                want)))
          (outline/kids ol)))))
  "First child of outline whose id/title matches needle
     (case-insensitive).")

(define spec/strip-punct
  (lambda (tok)
    (let ((s0 (as-str tok)))
      (if (= (string-length s0) 0)
          ""
          (let ((n0 (string-length s0))
                (a (substring s0 0 1)))
            (let ((s1
                    (if (or (string-eq a "(")
                            (string-eq a "[")
                            (string-eq a "'")
                            (string-eq a ","))
                        (substring s0 1 (- n0 1))
                        s0)))
              (let ((n1 (string-length s1)))
                (if (= n1 0)
                    ""
                    (let ((z1 (substring s1 (- n1 1) 1)))
                      (if (or (string-eq z1 ")")
                              (string-eq z1 "]")
                              (string-eq z1 ",")
                              (string-eq z1 ".")
                              (string-eq z1 ";")
                              (string-eq z1 ":"))
                          (substring s1 0 (- n1 1))
                          s1)))))))))
  "Trim one layer of common punctuation from a token.")

(define spec/token-binding?
  (lambda (tok)
    (let ((s (as-str tok)))
      (and (> (string-length s) 2)
           (string-contains? s "/")
           (not (string-eq s "->"))
           (not (string-eq s "/"))
           (not (string-contains? s " "))
           (not (string-eq (substring s 0 1) "/"))))))
  "Heuristic: tokens that look like pkg/name bindings.")

(define spec/tokens-from-text
  (lambda (text)
    (list/filter
      spec/token-binding?
      (list/map
        spec/strip-punct
        (string-split (as-str text) " "))))
  "Split text on spaces; strip punct; keep pkg/name-looking tokens.")

(define spec/flow-entries
  (lambda (pkg)
    (let ((d (spec/doc pkg)))
      (if (null? d)
          (list)
          (let ((ol (car (cdr (cdr (cdr d)))))
                (sec (spec/find-section
                       (car (cdr (cdr (cdr d))))
                       "flows")))
            (if (null? sec)
                (list)
                (list/map
                  (lambda (c)
                    (list
                      (as-str (outline/title c))
                      (as-str (outline/body c))
                      (spec/unique
                        (spec/append
                          (spec/tokens-from-text (outline/title c))
                          (spec/tokens-from-text (outline/body c))))))
```

```
                  (outline/kids sec)))))))
  "List of (flow-title flow-body binding-tokens) from FLOWS section.")

(define spec/binding-resolves?
  (lambda (name)
    (not (null? (function-source (as-str name)))))
  "True when function-source returns a body.")

(define spec/flows-resolve
  (lambda (pkg)
    (let ((entries (spec/flow-entries pkg)))
      (if (null? (spec/doc pkg))
          (list
            (list "ERROR" (string-append "missing notes doc "
     (spec/doc-slug pkg)) (list)))
          (list/map
            (lambda (e)
              (let ((title (car e))
                    (body (car (cdr e)))
                    (toks (car (cdr (cdr e)))))
                (let ((missing
                        (list/filter
                          (lambda (n) (not (spec/binding-resolves? n)))
                          toks))
                      (ok
                        (list/filter spec/binding-resolves? toks)))
                  (list title body ok missing))))
            entries))))
  "Parse spec/<pkg> FLOWS; each entry ? (title body resolved missing).")

(define spec/public-names
  (lambda (pkg)
    (let ((exports
            (package-exports (as-str pkg)))
          (all
            (package-bindings (as-str pkg))))
      ;; Prefer declared exports; fall back to all package bindings.
      (if (null? exports) all exports)))
  "Public surface: package-exports, else package-bindings.")

(define spec/doc-text
  (lambda (pkg)
    (let ((d (spec/doc pkg)))
      (if (null? d)
          ""
          (spec/outline-text (car (cdr (cdr (cdr d)))))))))
  "Full searchable text of the package spec doc.")

(define spec/mentioned?
  (lambda (name text)
    (string-contains? (as-str text) (as-str name)))
  "True if name appears as substring in text.")

(define spec/doc-coverage
  (lambda (pkg)
    (list/filter
      (lambda (n) (null? (function-doc (as-str n))))
      (spec/public-names pkg)))
  "Public bindings with empty function-doc.")

(define spec/source-sig
  (lambda (name)
    (let ((s (function-source (as-str name))))
      (if (null? s) "" (as-str s))))
  "function-source text or empty.")

(define spec/names-redefined
  (lambda (pkg branch)
    ;; Compare public binding sources on BRANCH vs system/main.
    ;; Restores the prior current-branch afterward.
    (let ((prev (current-branch))
          (names (spec/public-names pkg))
          (br (as-str branch)))
```

```
      (let ((main-pairs
              (begin
                (checkout "system/main")
                (list/map
                  (lambda (n) (list n (spec/source-sig n)))
                  names)))
            (branch-pairs
              (begin
                (checkout br)
                (list/map
                  (lambda (n) (list n (spec/source-sig n)))
                  names))))
        (begin
          (if (string-eq (as-str prev) "")
              #t
              (checkout prev))
          (list/filter
            (lambda (n)
              (let ((m
                      (car
                        (list/filter
                          (lambda (p) (string-eq (car p) n))
                          main-pairs)))
                    (b
                      (car
                        (list/filter
                          (lambda (p) (string-eq (car p) n))
                          branch-pairs))))
                (and (not (null? m))
                     (not (null? b))
                     (not (string-eq (car (cdr m)) ""))
                     (not (string-eq (car (cdr b)) ""))
                     (not (string-eq (car (cdr m)) (car (cdr b)))))))
            names)))))
  "Public names whose function-source differs between system/main and
     branch.")

(define spec/unacknowledged
  (lambda (pkg branch)
    (let ((text (spec/doc-text pkg))
          (br (as-str branch))
          (redef
            (if (or (string-eq (as-str branch) "")
                    (string-eq (as-str branch) "system/main"))
                ;; No branch lens: treat all public names as candidates
                ;; that must at least be mentioned in the spec doc.
                (spec/public-names pkg)
                (spec/names-redefined pkg branch))))
      (list/filter
        (lambda (n) (not (spec/mentioned? n text)))
        redef)))
  "Public bindings redefined on branch (vs main) with no mention in
     spec/<pkg> text. If branch is empty/main, all unmentioned public
     names.")

(define spec/flows-missing-count
  (lambda (rows)
    (list/foldl
      (lambda (n row)
        (+ n (count (car (cdr (cdr (cdr row)))))))
      0
      rows))
  "Count missing bindings across flows-resolve rows.")

(define spec/lint
  (lambda (pkg branch)
    (let ((flows (spec/flows-resolve pkg))
          (undoc (spec/doc-coverage pkg))
          (unack (spec/unacknowledged pkg branch))
          (miss (spec/flows-missing-count (spec/flows-resolve pkg))))
      (list
        (list "pkg" (as-str pkg))
        (list "branch" (as-str branch))
```

```
        (list "spec-doc" (spec/doc-slug pkg))
        (list "flows" flows)
        (list "flow-missing-count" miss)
        (list "doc-coverage-empty" undoc)
        (list "doc-coverage-count" (count undoc))
        (list "unacknowledged" unack)
        (list "unacknowledged-count" (count unack)))))
  "Aggregate lint report for pkg on branch (soft warnings shape for
     promote).")

;; ---- /spec health page (habitat-workflow.spec-health) ----

(define spec/starts-with?
  (lambda (s p)
    (let ((ss (as-str s)) (pp (as-str p))
          (n (string-length (as-str p))))
      (and (>= (string-length ss) n)
           (string-eq (substring ss 0 n) pp))))
  "Prefix test.")

(define spec/packages-with-docs
  (lambda ()
    (list/map
      (lambda (row)
        ;; notes/list row: (id . fields-json-str)
        (let ((slug (as-str (dict-get (cdr row) "slug"))))
          (substring slug 5 (- (string-length slug) 5))))
      (list/filter
        (lambda (row)
          (spec/starts-with?
            (as-str (dict-get (cdr row) "slug"))
            "spec/"))
        (notes/list))))
  "Package names that have a notes doc slug spec/<pkg>.")

(define spec/assoc-get
  (lambda (report key)
    (let ((hit
            (list/filter
              (lambda (pair) (string-eq (as-str (car pair)) (as-str key)))
              report)))
      (if (null? hit) () (car (cdr (car hit)))))))
  "Lookup key in (spec/lint) alist report.")

(define spec/lint-summary
  (lambda (pkg branch)
    (if (null? (spec/doc pkg))
        (string-append (as-str pkg) ": (no spec doc)")
        (let ((r (spec/lint pkg branch)))
          (string-append
            (as-str pkg) ": flow-holes="
            (as-str (spec/assoc-get r "flow-missing-count"))
            " undoc="
            (as-str (spec/assoc-get r "doc-coverage-count"))
            " unack="
            (as-str (spec/assoc-get r "unacknowledged-count")))))))
  "One-line soft warning string for promote checklist / UI.")

(define spec/promote-checklist
  (lambda (branch names)
    ;; Soft warnings for packages touched by the promote name set.
    (let ((pkgs
            (spec/unique
              (list/map
                (lambda (n)
                  (let ((s (as-str n)))
                    (if (string-contains? s "/")
                        (car (string-split s "/"))
                        "")))
                names))))
      (list/map
        (lambda (p)
          (if (string-eq (as-str p) "")
```

```
              "(root): skipped"
              (spec/lint-summary p branch)))
        (list/filter (lambda (p) (not (string-eq (as-str p) ""))) pkgs))))
  "List of soft-warning lines for promote of names on branch.")

(define spec/row-summary
  (lambda (pkg branch)
    (let ((r (spec/lint pkg branch)))
      (list
        (as-str pkg)
        (spec/assoc-get r "flow-missing-count")
        (spec/assoc-get r "doc-coverage-count")
        (spec/assoc-get r "unacknowledged-count"))))
  "(pkg flow-missing doc-empty unack) for one package.")

(define spec/td
  (lambda (s)
    (html/tag "td" "" (html-escape (as-str s))))
  "Table cell.")

(define spec/th
  (lambda (s)
    (html/tag "th" "" (html-escape (as-str s))))
  "Header cell.")

(define spec/tr
  (lambda (cells)
    (html/tag "tr" "" (html/join cells)))
  "Table row from cell html list.")

(define spec/status-emoji
  (lambda (miss undoc unack)
    (if (and (= miss 0) (= undoc 0) (= unack 0))
        "ok"
        (if (> miss 0) "flows!" "warn")))
  "Coarse phone-readable status label.")

(define spec/health-row-html
  (lambda (sum)
    (let ((pkg (car sum))
          (miss (car (cdr sum)))
          (undoc (car (cdr (cdr sum))))
          (unack (car (cdr (cdr (cdr sum))))))
      (spec/tr
        (list
          (spec/td pkg)
          (spec/td (spec/status-emoji miss undoc unack))
          (spec/td miss)
          (spec/td undoc)
          (spec/td unack)
          (html/tag
            "td" ""
            (html/a
              (string-append "/notes?slug=spec/" (as-str pkg))
              (string-append "spec/" (as-str pkg)))))))))
  "One health table row.")

(define spec/health-table
  (lambda (branch)
    (let ((pkgs (spec/packages-with-docs))
          (br (as-str branch)))
      (if (null? pkgs)
          (html/p* (html/attr "class" "muted")
                   "No spec/<pkg> notes docs yet. Seed with B1 ritual.")
          (html/tag
            "table"
            (html/attr "class" "spec-health")
            (html/join
              (cons
                (spec/tr
                  (list
                    (spec/th "pkg")
                    (spec/th "status")
```

```
                    (spec/th "flow holes")
                    (spec/th "undoc")
                    (spec/th "unack")
                    (spec/th "spec")))
                (list/map
                  (lambda (p)
                    (spec/health-row-html (spec/row-summary p br)))
                  pkgs)))))))
  "Full health table HTML for packages with spec docs.")

(define spec/page
  (lambda ()
    (let ((br (current-branch)))
      (html/page
        (list
          (html/h1 "/spec health")
          (html/p*
            (html/attr "class" "muted")
            (string-append
              "Package intent lint (soft). Branch: "
              (as-str br)
              ". flow holes = unresolved FLOWS bindings; undoc = public "
              "bindings missing function-doc; unack = public names not "
              "mentioned in spec doc (or redefined vs main when branch
    set)."))
          (spec/health-table "")
          (html/p*
            (html/attr "class" "muted")
            "API: (spec/lint \"todo\" \"\") ? (spec/flows-resolve
    \"todo\")")))))
  "Private GET /spec handler -- phone-readable package health.")

(route "/spec" "spec/page")

;; ---- /book/<pkg> (habitat-workflow.book-view) ----
;; Kernel routes GET /book/<pkg> ? (spec/book-page); pkg from request-path.

(define spec/path-pkg
  (lambda (path)
    (let ((p (as-str path))
          (prefix "/book/"))
      (if (spec/starts-with? p prefix)
          (substring p 6 (- (string-length p) 6))
          ""))))
  "Extract package name from /book/<pkg> path.")

(define spec/hole-html
  (lambda (name)
    (html/tag
      "div"
      (html/attrs
        (list
          (html/attr "class" "spec-hole")
          (html/attr "style"
            "border:2px dashed #c44;padding:.5rem;margin:.4rem
    0;background:#2a1515;color:#f88;")))
      (string-append
        "? unresolved binding: "
        (html-escape (as-str name))
        " -- drift-as-rendering")))
  "Visible styled hole for missing binding refs.")

(define spec/binding-inline-html
  (lambda (name)
    (let ((n (as-str name))
          (src (function-source (as-str name)))
          (doc (function-doc (as-str name))))
      (if (null? src)
          (spec/hole-html n)
          (html/tag
            "div"
            (html/attr "class" "spec-binding")
            (html/join
```

```
              (list
                (html/tag "h3" ""
                  (html/a
                    (string-append "/binding/" n)
                    (html-escape n)))
                (if (null? doc)
                    (html/p* (html/attr "class" "muted") "(no
    function-doc)")
                    (html/p (html-escape (as-str doc))))
                (html/pre (html-escape (as-str src)))))))))
  "Inline one binding's doc + source, or a hole.")

(define spec/node-bindings-html
  (lambda (n)
    ;; Body only -- titles like "spec/todo" would false-positive as
    bindings.
    (let ((toks (spec/unique (spec/tokens-from-text (outline/body n)))))
      (if (null? toks)
          ""
          (html/join (list/map spec/binding-inline-html toks)))))
  "Render binding inlines for tokens found in a node body.")

(define spec/outline-chapter-html
  (lambda (n depth)
    (let ((title (html-escape (as-str (outline/title n))))
          (body (html-escape (as-str (outline/body n))))
          (tag
            (if (= depth 0) "h1"
                (if (= depth 1) "h2"
                    (if (= depth 2) "h3" "h4"))))
          (kids (outline/kids n))
          (bind-html (spec/node-bindings-html n)))
      (html/tag
        "section"
        (html/attr "class" "spec-chapter-node")
        (html/join
          (list
            (html/tag tag "" title)
            (if (string-eq body "")
                ""
                (html/p body))
            bind-html
            (html/join
              (list/map
                (lambda (k) (spec/outline-chapter-html k (+ depth 1)))
                kids)))))))
  "Walk outline; inline binding refs under each node.")

(define spec/book-index
  (lambda ()
    (html/page
      (list
        (html/h1 "Package book")
        (html/p* (html/attr "class" "muted")
                 "Spec docs as chapters. Drift shows as holes.")
        (html/ul
          ""
          (html/join
            (list/map
              (lambda (p)
                (html/li
                  ""
                  (html/a
                    (string-append "/book/" (as-str p))
                    (as-str p))))
              (spec/packages-with-docs))))
        (html/p*
          (html/attr "class" "muted")
          (html/a "/spec" "? /spec health"))))))
  "GET /book index of packages with spec docs.")

(define spec/book-page
  (lambda ()
```

```
    (let ((path (request-path))
          (pkg (spec/path-pkg (request-path))))
      (if (string-eq (as-str pkg) "")
          (spec/book-index)
          (let ((d (spec/doc pkg)))
            (if (null? d)
                (html/page
                  (list
                    (html/h1 (string-append "/book/" (as-str pkg)))
                    (html/p* (html/attr "class" "muted")
                             "No spec doc -- create notes slug spec/<pkg>.")
                    (html/tag "p" "" (html/a "/book" "? book index"))
                    (html/tag "p" "" (html/a "/spec" "/spec health"))))
                (let ((ol (car (cdr (cdr (cdr d)))))
                      (title (as-str (car (cdr (cdr d))))))
                  (html/page
                    (list
                      (html/p*
                        (html/attr "class" "muted")
                        (html/join
                          (list
                            (html/a "/book" "book")
                            " ? "
                            (html/a "/spec" "spec health")
                            " ? "
                            (html/a
                              (string-append "/notes?slug=spec/" (as-str
    pkg))
                              (string-append "spec/" (as-str pkg))))))
                      (spec/outline-chapter-html ol 0)))))))
  "Private GET /book/<pkg> -- chapter with inlined binding doc+source.")

(route "/book" "spec/book-page")
```

202 libs/std/lib.pp

```
;; Standard Lisp helpers (package list/, html/, css/, menu/, data/)
;; Loaded by (load-library "std") at image boot.
;; Docs: trailing string on define (or leading multi-body lambda string).

(package std
  (export
    list/map
    list/filter
    list/foldl
    list/range
    map
    filter
    foldl
    foldr
    dict-set*
    html/join
    html/text
    html/attr
    html/attrs
    html/tag
    html/void
    html/h1
    html/h2
    html/h3
    html/p
    html/p*
    html/span
    html/div
    html/ul
    html/li
    html/strong
    html/code
    html/pre
    html/label
    html/button
    html/a
    html/img
    html/form
    html/input
    html/hidden
    html/text-input
    html/textarea
    html/option
    html/select
    html/wrap
    html/page
    menu/link
    menu/group
    menu/raw
    menu/render
    menu/bar
    data/get-field
    css/rule
    css/media
    css/raw
    css/use
    css/sheet
    css/render
    blank?
    string-find-pos))

;; --- list/ ---
;; list/map, list/filter, list/foldl are KERNEL SPECIAL FORMS (eval.ml):
;; OCaml loops + apply, provided by the image; defining them here trips
;; assert_bindable_name (BORGE-1785213238) and aborts std seeding.
;; Do NOT re-add fallback defines -- the kernel always provides them.

;; (list/range n) ? (0 1 ... n-1) for synthetic tests / generators.
(define list/range
  (lambda (n)
    (list/range-acc n (list)))
```

```
  "List of integers 0 .. n-1.")

(define list/range-acc
  (lambda (n acc)
    (if (<= n 0)
        acc
        (list/range-acc (- n 1) (cons (- n 1) acc))))
  "Internal accumulator for list/range.")

;; --- unprefixed list aliases (conventional Lisp names; thin wrappers
;;     over the kernel list/ special forms). Not part of the frozen
;;     list/ set, but provided by std for ergonomics so (map f xs) etc.
;;     work without the list/ prefix. ---
(define map
  (lambda (f xs) (list/map f xs))
  "Map f over xs (alias of list/map).")
(define filter
  (lambda (p xs) (list/filter p xs))
  "Keep elements of xs where p is truthy (alias of list/filter).")
(define foldl
  (lambda (f init xs) (list/foldl f init xs))
  "Left fold f over xs from init (alias of list/foldl).")
(define foldr
  (lambda (f init xs) (list/foldr f init xs))
  "Right fold f over xs from init (alias of list/foldr).")

;; --- binding/control macros ---
;; Unhygienic defmacro + quasiquote (language-roadmap decision default).
;; Single-body only: defmacro params do not support dotted rest args.

;; (let* ((a 1) (b (+ a 1))) body) -- sequential bindings; each value
;; sees earlier names. Expands to nested parallel let.
(defmacro let* (bindings body)
  (if (null? bindings)
      body
      (quasiquote
        (let ((unquote (car bindings)))
          (let* (unquote (cdr bindings)) (unquote body))))))

;; (when test body) -- evaluate body only when test is truthy; else ().
(defmacro when (test body)
  (quasiquote (if (unquote test) (unquote body) ())))

;; (unless test body) -- evaluate body only when test is falsey.
(defmacro unless (test body)
  (quasiquote (if (unquote test) () (unquote body))))

;; --- dict/ ---

;; (dict-set* json (list "k1" v1 "k2" v2 ?)) -- flat multi-field update.
;; Prefer this over nested (dict-set (dict-set ?)).
(define dict-set*
  (lambda (json kvs)
    (if (null? kvs)
        json
        (if (null? (cdr kvs))
            json
            (dict-set*
              (dict-set json (car kvs) (car (cdr kvs)))
              (cdr (cdr kvs)))))))
  "Apply flat key/value pairs from kvs onto json via dict-set.")

;; --- html/  (strings ? render-html; no raw tag soup in apps) ---
;;
;; Pattern:
;;   (render-html
;;     (html/join (list
;;       (html/h1 "Todos")
;;       (html/div (html/attr "class" "card") body))))
;;
;; Attrs are space-joined strings from html/attr.
;; Body is always a string (use html/join for children).

```

```
(define html/join
  (lambda (xs)
    (list/foldl
      (lambda (acc x) (string-append acc (as-str x)))
      ""
      xs))
  "Concatenate list of values into one string.")

(define html/text
  (lambda (s) (html-escape s))
  "HTML-escape s for safe text content.")

(define html/attr
  (lambda (k v)
    (string-append (as-str k) "=\"" (html-escape (as-str v)) "\""))
  "One HTML attribute k=\"escaped-v\".")

(define html/attrs
  (lambda (xs)
    (list/foldl
      (lambda (acc a)
        (if (string-eq acc "")
            (as-str a)
            (string-append acc " " (as-str a))))
      ""
      xs))
  "Space-join attribute strings.")

(define html/tag
  (lambda (name attrs body)
    (let ((n (as-str name))
          (a (as-str attrs))
          (b (as-str body)))
      (if (string-eq a "")
          (string-append "<" n ">" b "</" n ">")
          (string-append "<" n " " a ">" b "</" n ">"))))
  "Element <name attrs>body</name>; attrs may be empty string.")

(define html/void
  (lambda (name attrs)
    (let ((n (as-str name))
          (a (as-str attrs)))
      (if (string-eq a "")
          (string-append "<" n ">")
          (string-append "<" n " " a ">"))))
  "Void element <name attrs> (no close tag).")

(define html/h1
  (lambda (body) (html/tag "h1" "" body))
  "h1 with raw body string (caller escapes if needed).")

(define html/h2
  (lambda (body) (html/tag "h2" "" body))
  "h2 with raw body string.")

(define html/h3
  (lambda (body) (html/tag "h3" "" body))
  "h3 with raw body string.")

(define html/p
  (lambda (body)
    (html/tag "p" "" (html/text body)))
  "Paragraph with HTML-escaped text body.")

(define html/p*
  (lambda (attrs body)
    (html/tag "p" attrs body))
  "Paragraph with attrs and raw body (no auto-escape).")

(define html/span
  (lambda (attrs body)
    (html/tag "span" attrs body))
  "span with attrs and raw body.")
```

```
218
219 (define html/div
220   (lambda (attrs body)
221     (html/tag "div" attrs body))
222   "div with attrs and raw body.")
223
224 (define html/ul
225   (lambda (attrs body)
226     (html/tag "ul" attrs body))
227   "ul with attrs and raw body (usually joined li strings).")
228
229 (define html/li
230   (lambda (attrs body)
231     (html/tag "li" attrs body))
232   "li with attrs and raw body.")
233
234 (define html/strong
235   (lambda (body)
236     (html/tag "strong" "" (html/text body)))
237   "strong with HTML-escaped text.")
238
239 (define html/code
240   (lambda (body)
241     (html/tag "code" "" (html/text body)))
242   "code with HTML-escaped text.")
243
244 (define html/pre
245   (lambda (body)
246     (html/tag "pre" "" body))
247   "pre with raw body (no auto-escape).")
248
249 (define html/label
250   (lambda (attrs body)
251     (html/tag "label" attrs body))
252   "label with attrs and raw body.")
253
254 (define html/button
255   (lambda (attrs body)
256     (html/tag "button" attrs body))
257   "button with attrs and raw body.")
258
259 (define html/a
260   (lambda (href body)
261     (html/tag "a" (html/attr "href" href) body))
262   "Anchor with href attr and raw body.")
263
264 ;; Void <img>. src/alt required; optional class via third arg ("" ok).
265 (define html/img
266   (lambda (src alt class)
267     (html/void "img"
268       (html/attrs
269         (list
270           (html/attr "src" src)
271           (html/attr "alt" alt)
272           (if (string-eq (as-str class) "")
273               ""
274               (html/attr "class" class))))))
275   "img void tag; class may be empty string to omit.")
276
277 (define html/form
278   (lambda (method action attrs body)
279     (html/tag "form"
280       (html/attrs
281         (list
282           (html/attr "method" method)
283           (html/attr "action" action)
284           attrs))
285       body))
286   "form with method, action, extra attrs string, and body.")
287
288 (define html/input
289   (lambda (attrs)
290     (html/void "input" attrs))
```

```
  "input void tag from attrs string.")

(define html/hidden
  (lambda (name value)
    (html/input
      (html/attrs
        (list
          (html/attr "type" "hidden")
          (html/attr "name" name)
          (html/attr "value" value)))))
  "Hidden input name/value.")

(define html/text-input
  (lambda (name attrs)
    (html/input
      (html/attrs
        (list
          (html/attr "type" "text")
          (html/attr "name" name)
          attrs))))
  "Text input with name and extra attrs string.")

(define html/textarea
  (lambda (name attrs body)
    (html/tag "textarea"
      (html/attrs
        (list
          (html/attr "name" name)
          attrs))
      (html-escape (as-str body))))
  "textarea; body is HTML-escaped.")

;; (html/option value label selected?) -- selected? is bool/truthy
(define html/option
  (lambda (value label selected?)
    (html/tag "option"
      (if selected?
          (html/attrs
            (list
              (html/attr "value" value)
              (html/attr "selected" "selected")))
          (html/attr "value" value))
      (html/text label)))
  "option element; selected? adds selected attr.")

;; (html/select name attrs body) -- body is options HTML string
(define html/select
  (lambda (name attrs body)
    (html/tag "select"
      (html/attrs
        (list
          (html/attr "name" name)
          attrs))
      body))
  "select with name, attrs, and options body HTML.")

(define html/wrap
  (lambda (title body)
    (render-html
      (html/tag "article" ""
        (html/join
          (list
            (html/h1 (html/text title))
            (html/div "" body))))))
  "render-html article with h1 title and body div.")

(define html/page
  (lambda (parts)
    (render-html (html/join parts)))
  "render-html of joined page parts (list of HTML strings).")

;; --- menu/  (navbar trees as plain lists) ---
;;
```

```
;;   (menu/link "home" "/")
;;   (menu/group "apps" (list (menu/link "todos" "/todos") ?))
;;   (menu/bar (list item1 item2 ?))  ?  HTML string for <nav>
;;
;; Leaves are ("link" label href); groups are ("group" label kids-list).

(define menu/link
  (lambda (label href)
    (list "link" (as-str label) (as-str href)))
  "Nav leaf: (\"link\" label href).")

(define menu/group
  (lambda (label kids)
    (list "group" (as-str label) kids))
  "Nav dropdown group: (\"group\" label kids-list).")

;; Embed pre-rendered HTML as a nav item (e.g. capture dropdown form).
(define menu/raw
  (lambda (html-str)
    (list "raw" (as-str html-str) ""))
  "Nav raw HTML item: (\"raw\" html \"\").")

;; depth-aware renderer: d=0 top-level nav, d>0 nested fly-out.
;; Top-level groups share name="pp-nav" so the browser's native
;; <details name> accordion auto-closes siblings (replaces the old
;; chrome toggle JS). Nested groups get name="pp-sub-<label>" per
;; fly-out column, so opening one submenu closes its siblings.
(define menu/render-d
  (lambda
    (item d)
    "Depth-aware menu renderer: link/group/raw item to HTML; open groups
     get accordion names by depth."
    (let
      ((kind (car item))
        (label (car (cdr item)))
        (rest (car (cdr (cdr item)))))
      (if
        (string-eq kind "link")
        (html/a rest (html/text label))
        (if
          (string-eq kind "group")
          (html/tag
            "details"
            (html/attrs
              (list
                (html/attr "class" "menu-drop")
                (html/attr
                  "name"
                  (if
                    (= d 0)
                    "pp-nav"
                    (string-append "pp-sub-" (as-str label))))))
            (html/join
              (list
                (html/tag
                  "summary"
                  (html/attr "class" "menu-drop-btn")
                  (html/text label))
                (html/div
                  (html/attr "class" "menu-drop-panel")
                  (html/join
                    (list/map
                      (lambda (k) (menu/render-d k (+ d 1)))
                      rest))))))
          (if (string-eq kind "raw") label ""))))))

(define menu/render
  (lambda (item) (menu/render-d item 0))
  "Render one menu item (link, group, or raw) to HTML string.")

(define menu/bar
  (lambda (items)
    (html/join (list/map (lambda (i) (menu/render-d i 0)) items)))
```

```
  "Render menu item list to a nav HTML fragment.")

;; --- data/ ---

(define data/get-field
  (lambda (type-name id field)
    (dict-get (get type-name id) field))
  "dict-get field from (get type-name id) row.")

;; --- css/  (sheets ? css/render; no raw rule soup in apps) ---
;;
;; Path B product theme (image.product-theme): styles are Lisp data.
;; Selectors stay strings; declarations are flat k v k v lists.
;;
;;   (css/sheet
;;     (list
;;       (css/rule "body" (list "margin" 0 "background" theme/bg))
;;       (css/media "max-width:40rem"
;;         (list (css/rule "main" (list "padding" ".5rem"))))
;;       (css/raw "/* rare escape */")))
;;   (css/render sheet) ? CSS string

(define css/rule
  (lambda (selector props)
    (list "rule" (as-str selector) props))
  "CSS rule node: (\"rule\" selector flat-props).")

(define css/media
  (lambda (query items)
    (list "media" (as-str query) items))
  "CSS @media node wrapping child items.")

(define css/raw
  (lambda (s)
    (list "raw" (as-str s)))
  "Raw CSS text node (escape hatch).")

(define css/use
  (lambda (sheet)
    (list "use" sheet))
  "Include another sheet node by reference.")

(define css/sheet
  (lambda (items)
    (list "sheet" items))
  "Top-level CSS sheet: (\"sheet\" items).")

(define css/prop-name
  (lambda (k)
    (as-str k))
  "CSS property name as string.")

(define css/prop-value
  (lambda (v)
    (as-str v))
  "CSS property value as string.")

(define css/render-decls
  (lambda (props)
    (if (null? props)
        ""
        (if (null? (cdr props))
            ""
            (string-append
              (css/prop-name (car props))
              ":"
              (css/prop-value (car (cdr props)))
              ";"
              (css/render-decls (cdr (cdr props)))))))
  "Render flat k v k v prop list to CSS declarations.")

(define css/render-items
  (lambda (items)
```

```
    (list/foldl
      (lambda (acc it)
        (string-append acc (css/render-item it)))
      ""
      items))
  "Render list of CSS AST items to a CSS string.")

(define css/render-item
  (lambda (item)
    (if (null? item)
        ""
        (let ((kind (as-str (car item))))
          (if (string-eq kind "rule")
              (let ((sel (car (cdr item)))
                    (props (car (cdr (cdr item)))))
                (string-append
                  (as-str sel) "{" (css/render-decls props) "}\n"))
              (if (string-eq kind "media")
                  (let ((q (car (cdr item)))
                        (kids (car (cdr (cdr item)))))
                    (string-append
                      "@media(" (as-str q) "){\n"
                      (css/render-items kids)
                      "}\n"))
                  (if (string-eq kind "raw")
                      (string-append (as-str (car (cdr item))) "\n")
                      (if (string-eq kind "use")
                          (css/render (car (cdr item)))
                          (if (string-eq kind "sheet")
                              (css/render-items (car (cdr item)))
                              ""))))))))
  "Render one CSS AST node (rule/media/raw/use/sheet).")

(define css/render
  (lambda (sheet)
    (if (string-eq (type-of sheet) "lambda")
        (css/render (sheet))
        (if (null? sheet)
            ""
            (if (string-eq (as-str (car sheet)) "sheet")
                (css/render-items (car (cdr sheet)))
                (css/render-item sheet))))))
  "Render sheet AST or thunk to CSS text.")

;; --- control / macros (loaded after kernel cond/and/or) ---

;; Null or empty string (after as-str). Shared by contacts/plan/chat/?
(define blank?
  (lambda (s)
    (if (null? s)
        #t
        (string-eq (as-str s) ""))))
  "True if s is nil or empty string.")

(defmacro when (test body)
  `(if ,test ,body ()))

(defmacro unless (test body)
  `(if ,test () ,body))

;; HTTP page helper: POST runs action then hx fragment or redirect;
;; GET renders page body (already a full html/page or raw fragment).
;; get-html and post-html are expressions (usually (html/page ?)).
(defmacro respond-post (redirect-path post-action get-html)
  `(if (string-eq (request-method) "POST")
       (seq
         ,post-action
         (if (hx-request?)
             (render-html ,get-html)
             (redirect ,redirect-path)))
       (html/page ,get-html)))

;; Empty list ? muted message; else map items into a ul.
```

```
(defmacro list-or-empty (xs empty-msg item-fn)
  `(if (null? ,xs)
       (html/p* (html/attr "class" "muted") ,empty-msg)
       (html/ul ""
         (html/join (list/map ,item-fn ,xs)))))

;; --- discoverability (define-docstring companion) ---

;; Find first index of delim in s at or after start, or -1.
(define string-find-pos
  (lambda (s delim start dlen)
    "Index of delim in s starting at start, or -1."
    (if (>= start (- (string-length s) (- dlen 1)))
        -1
        (if (string-eq (substring s start dlen) delim)
            start
            (string-find-pos s delim (+ start 1) dlen)))))

;; string-split: now a KERNEL PRIM (pure/prims_pure.ml prim_string_split,
;; registered in image/lib/primitives.ml + agent + remote runtimes).
;; DECISION (2026-08-30, habitat-book web-renderer): the interpreted
;; char-walk version below cost ~O(n) interpreter steps per input BYTE
;; (string-find-pos recursion per position), so one 3KB prose paragraph
;; blew the 100k-step page budget -- book chapter pages could not render
;; at all. The kernel prim keeps the split semantics (empty s or no
;; match ? single-element list; empty pieces kept); an empty delimiter
;; returns the whole string (the old version looped forever).
;; string-find-pos STAYS (notes/ uses it directly).

;; Package prefix before first / (or whole name).
(define prefix-of
  (lambda (name)
    "First path segment of a binding name (before /)."
    (car (string-split (as-str name) "/"))))

;; Substring search over visible binding names.
(define apropos
  (lambda (pat)
    "Names from (list-bindings) whose string form contains pat."
    (list/filter
      (lambda (n) (string-contains? (as-str n) (as-str pat)))
      (list-bindings))))
```

203 libs/strongs/lib.pp

```
;; package strongs -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push

(define strongs/description
  (lambda
    (number)
    "The description of a Strong's entry, or () when absent."
    (let
      ((row (strongs/get number)))
      (if (null? row) () (dict-get row "description")))))

(define strongs/entry-html
  (lambda
    (row)
    "Render a Strong's entry row as HTML, or a not-found message."
    (if
      (null? row)
      (html/p* (html/attr "class" "err") "not found")
      (html/div
        (html/attr "class" "card strongs-entry")
        (html/join
          (list
            (html/h2 (html/text (dict-get row "number")))
            (html/p (dict-get row "lemma"))
            (html/p*
              (html/attr "class" "muted")
              (html/text
                (string-append
                  (as-str (dict-get row "xlit"))
                  " / "
                  (as-str (dict-get row "pronounce")))))
            (html/p (dict-get row "description")))))))

(define strongs/get
  (lambda
    (number)
    "Fetch a Strong's entry row by number, or () when absent."
    (find-row
      (quote strongs-entry)
      "number"
      (as-str number))))

(define strongs/lemma
  (lambda
    (number)
    "The lemma of a Strong's entry, or () when absent."
    (let
      ((row (strongs/get number)))
      (if (null? row) () (dict-get row "lemma")))))

(define strongs/lookup-form
  (lambda
    (n)
    "An htmx lookup form for a Strong's number."
    (html/form
      "GET"
      "/strongs"
      (html/attrs
        (list
          (html/attr "class" "card")
          (html/attr "hx-get" "/strongs")
          (html/attr "hx-target" "#strongs-panel")
          (html/attr "hx-swap" "innerHTML")
          (html/attr "hx-push-url" "true")))
      (html/join
        (list
          (html/label
            ""
            (html/join
              (list
                "Number "
```

```
                (html/text-input
                  "n"
                  (html/attrs
                    (list
                      (html/attr "value" (if (null? n) "" (as-str n)))
                      (html/attr "placeholder" "H430 or G26")))))))
          " "
          (html/button (html/attr "type" "submit") "Look up"))))))

(define strongs/page
  (lambda
    ()
    "Serve the Strong's page, htmx fragment or full page."
    (if
      (hx-request?)
      (strongs/panel)
      (html/page
        (list
          (html/div
            (html/attr "id" "strongs-panel")
            (strongs/panel)))))))

(define strongs/panel
  (lambda
    ()
    "Render the Strong's lookup panel for the request number."
    (let
      ((n (request-param "n")))
      (html/join
        (list
          (html/h1 "Strong's")
          (strongs/lookup-form n)
          (if
            (null? n)
            (html/p*
              (html/attr "class" "muted")
              "Enter a Strong's number (H for Hebrew, G for Greek). Seed: scripts/seed_strongs.py")
            (strongs/entry-html (strongs/get n)))
          (html/p*
            (html/attr "class" "muted")
            (html/join
              (list
                (html/a "/dabar?book=John&chapter=3" "Bible")
                " ? "
                (html/a
                  "/dabar?book=John&chapter=3&verse=16"
                  "Dabar study")))))))))

(route "/strongs" "strongs/page")
```

204 libs/structed/lib.pp

```
;; package structed -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push

(define structed/barf-backward
  (lambda
    (get-kids set-kids tree id)
    "Move a target's first child up to become its preceding sibling;
     returns (tree id)."
    (let
      ((target (structed/find get-kids tree id)))
      (if
        (or (not target) (string-eq id (structed/id tree)))
        (list tree id)
        (let
          ((kids (get-kids target)))
          (if
            (<= (structed/count kids) 0)
            (list tree id)
            (let
              ((first-kid (structed/list/nth kids 0)))
              (let
                ((shrunk (set-kids target (structed/list/drop kids 1))))
                (let
                  ((parent (structed/find-parent get-kids tree id)))
                  (if
                    (not parent)
                    (list tree id)
                    (let
                      ((sibs (get-kids parent)))
                      (let
                        ((i (structed/sibling-index get-kids tree id)))
                        (let
                          ((new-sibs
                            (structed/list/insert-at
                              (structed/list/replace-at sibs i shrunk)
                              i
                              first-kid)))
                          (let
                            ((new-parent (set-kids parent new-sibs)))
                            (let
                              ((new-tree
                                (structed/replace
                                  get-kids
                                  set-kids
                                  tree
                                  (structed/id parent)
                                  new-parent)))
                              (list new-tree id)))))))))))))))))

(define structed/barf-forward
  (lambda
    (get-kids set-kids tree id)
    "Move a target's last child down to become its next sibling; returns
     (tree id)."
    (let
      ((target (structed/find get-kids tree id)))
      (if
        (or (not target) (string-eq id (structed/id tree)))
        (list tree id)
        (let
          ((kids (get-kids target)))
          (if
            (<= (structed/count kids) 0)
            (list tree id)
            (let
              ((last-kid
                (structed/list/nth kids (- (structed/count kids) 1))))
              (let
                ((shrunk
                  (set-kids
                    target
```

```
                    (structed/list/take kids (- (structed/count kids) 1)))))
                (let
                  ((parent (structed/find-parent get-kids tree id)))
                  (if
                    (not parent)
                    (list tree id)
                    (let
                      ((sibs (get-kids parent)))
                      (let
                        ((i (structed/sibling-index get-kids tree id)))
                        (let
                          ((new-sibs
                            (structed/list/insert-at
                              (structed/list/replace-at sibs i shrunk)
                              (+ i 1)
                              last-kid)))
                          (let
                            ((new-parent (set-kids parent new-sibs)))
                            (let
                              ((new-tree
                                (structed/replace
                                  get-kids
                                  set-kids
                                  tree
                                  (structed/id parent)
                                  new-parent)))
                              (list new-tree id)))))))))))))))))

(define structed/count
  (lambda
    (xs)
    "Length of the list xs."
    (if (null? xs) 0 (+ 1 (structed/count (cdr xs))))))

(define structed/find
  (lambda
    (get-kids tree id)
    "Node in tree whose id matches, or ()."
    (if
      (string-eq (structed/id tree) id)
      tree
      (structed/find-in-kids get-kids (get-kids tree) id))))

(define structed/find-in-kids
  (lambda
    (get-kids kids id)
    "Depth-first search for id across the sibling list kids."
    (if
      (null? kids)
      ()
      (let
        ((hit (structed/find get-kids (car kids) id)))
        (if
          hit
          hit
          (structed/find-in-kids get-kids (cdr kids) id))))))

(define structed/find-parent
  (lambda
    (get-kids tree id)
    "Parent node of the node with id, or ()."
    (if
      (string-eq (structed/id tree) id)
      ()
      (structed/find-parent-in-kids
        get-kids
        tree
        (get-kids tree)
        id))))

(define structed/find-parent-in-kids
  (lambda
    (get-kids parent kids id)
```

```
    "Search kids for id, returning the enclosing parent node."
    (if
      (null? kids)
      ()
      (let
        ((k (car kids)))
        (if
          (string-eq (structed/id k) id)
          parent
          (let
            ((hit (structed/find-parent get-kids k id)))
            (if
              hit
              hit
              (structed/find-parent-in-kids
                get-kids
                parent
                (cdr kids)
                id)))))))

(define structed/id
  (lambda (n) "Node id as a string." (as-str (car n))))

(define structed/join
  (lambda
    (get-kids set-kids tree id)
    "Merge a target with its next sibling (children merged, next removed);
     returns (tree id)."
    (let
      ((parent (structed/find-parent get-kids tree id)))
      (if
        (or (not parent) (string-eq id (structed/id tree)))
        (list tree id)
        (let
          ((sibs (get-kids parent)))
          (let
            ((i (structed/sibling-index get-kids tree id)))
            (if
              (>= (+ i 1) (structed/count sibs))
              (list tree id)
              (let
                ((me (structed/list/nth sibs i))
                  (next (structed/list/nth sibs (+ i 1))))
                (let
                  ((merged
                    (set-kids me (append (get-kids me) (get-kids next)))))
                  (let
                    ((new-sibs
                      (structed/list/remove-at
                        (structed/list/replace-at sibs i merged)
                        (+ i 1))))
                    (let
                      ((new-parent (set-kids parent new-sibs)))
                      (let
                        ((new-tree
                          (structed/replace
                            get-kids
                            set-kids
                            tree
                            (structed/id parent)
                            new-parent)))
                        (list new-tree id))))))))))))

(define structed/list/drop
  (lambda
    (xs n)
    "Element list with the first n removed."
    (if
      (or (null? xs) (<= n 0))
      xs
      (structed/list/drop (cdr xs) (- n 1)))))

(define structed/list/index-of
```

```
  (lambda
    (xs x eq?)
    "First index of x in xs per eq?, or -1."
    (structed/list/index-of-acc xs x eq? 0)))

(define structed/list/index-of-acc
  (lambda
    (xs x eq? i)
    "Accumulating index search helper."
    (if
      (null? xs)
      -1
      (if
        (eq? (car xs) x)
        i
        (structed/list/index-of-acc (cdr xs) x eq? (+ i 1))))))

(define structed/list/insert-at
  (lambda
    (xs i v)
    "Insert v into xs at index i (clamped to head)."
    (if
      (or (null? xs) (<= i 0))
      (cons v xs)
      (cons
        (car xs)
        (structed/list/insert-at (cdr xs) (- i 1) v)))))

(define structed/list/insert-list-at
  (lambda
    (xs i ys)
    "Insert every element of ys into xs starting at index i."
    (if
      (null? ys)
      xs
      (structed/list/insert-list-at
        (structed/list/insert-at xs i (car ys))
        (+ i 1)
        (cdr ys)))))

(define structed/list/nth
  (lambda
    (xs i)
    "Element at index i of xs, or ()."
    (if
      (null? xs)
      ()
      (if
        (<= i 0)
        (car xs)
        (structed/list/nth (cdr xs) (- i 1))))))

(define structed/list/remove-at
  (lambda
    (xs i)
    "xs with the element at index i removed."
    (if
      (null? xs)
      (list)
      (if
        (<= i 0)
        (cdr xs)
        (cons
          (car xs)
          (structed/list/remove-at (cdr xs) (- i 1)))))))

(define structed/list/replace-at
  (lambda
    (xs i v)
    "xs with index i replaced by v."
    (if
      (null? xs)
      (list)
```

```
      (if
        (<= i 0)
        (cons v (cdr xs))
        (cons
          (car xs)
          (structed/list/replace-at (cdr xs) (- i 1) v))))))

(define structed/list/split-at
  (lambda
    (xs i)
    "Split xs at index i into (before after)."
    (list
      (structed/list/take xs i)
      (structed/list/drop xs i))))

(define structed/list/take
  (lambda
    (xs n)
    "First n elements of xs."
    (if
      (or (null? xs) (<= n 0))
      (list)
      (cons (car xs) (structed/list/take (cdr xs) (- n 1))))))

(define structed/raise
  (lambda
    (get-kids set-kids tree id)
    "Lift a target up to replace its grandparent; returns (tree id)."
    (let
      ((parent (structed/find-parent get-kids tree id)))
      (if
        (not parent)
        (list tree id)
        (let
          ((grand
            (structed/find-parent
              get-kids
              tree
              (structed/id parent))))
          (if
            (not grand)
            (let
              ((target (structed/find get-kids tree id)))
              (list target id))
            (let
              ((sibs (get-kids grand)))
              (let
                ((i
                  (structed/list/index-of
                    sibs
                    parent
                    (lambda
                      (a b)
                      (string-eq (structed/id a) (structed/id b))))))
                (let
                  ((target (structed/find get-kids tree id)))
                  (let
                    ((new-sibs (structed/list/replace-at sibs i target)))
                    (let
                      ((new-grand (set-kids grand new-sibs)))
                      (let
                        ((new-tree
                          (structed/replace
                            get-kids
                            set-kids
                            tree
                            (structed/id grand)
                            new-grand)))
                        (list new-tree id)))))))))))))

(define structed/reattach
  (lambda
    (get-kids set-kids tree id new-sibs-fn)
```

```
    "Replace the target's sibling slot with new-sibs-fn applied to its
     parent; returns (tree id)."
    (let
      ((parent (structed/find-parent get-kids tree id)))
      (if
        (not parent)
        (list tree id)
        (let
          ((sibs (get-kids parent)))
          (let
            ((i (structed/sibling-index get-kids tree id)))
            (let
              ((new-sibs (new-sibs-fn parent sibs i)))
              (let
                ((new-parent (set-kids parent new-sibs)))
                (let
                  ((new-tree
                    (structed/replace
                      get-kids
                      set-kids
                      tree
                      (structed/id parent)
                      new-parent)))
                  (list new-tree id)))))))))

(define structed/replace
  (lambda
    (get-kids set-kids tree id node)
    "Substitute node for the node currently at id."
    (structed/update
      get-kids
      set-kids
      tree
      id
      (lambda (_) node))))

(define structed/roundtrip-slurp-barf
  (lambda
    (get-kids set-kids tree id)
    "Check that slurp-forward then barf-forward restores the tree (or () on
     mismatch)."
    (let
      ((r1 (structed/slurp-forward get-kids set-kids tree id)))
      (let
        ((t1 (car r1)))
        (let
          ((r2 (structed/barf-forward get-kids set-kids t1 id)))
          (let
            ((t2 (car r2)))
            (if (string-eq (json-emit t2) (json-emit tree)) t2 ())))))))

(define structed/roundtrip-wrap-splice
  (lambda
    (get-kids set-kids make-wrap-node tree id)
    "Check that wrap then splice restores the tree (or () on mismatch)."
    (let
      ((r1
        (structed/wrap
          get-kids
          set-kids
          make-wrap-node
          tree
          id
          ())))
      (let
        ((t1 (car r1)) (s1 (car (cdr r1))))
        (let
          ((r2 (structed/splice get-kids set-kids t1 s1)))
          (let
            ((t2 (car r2)))
            (if (string-eq (json-emit t2) (json-emit tree)) t2 ())))))))

(define structed/sibling-index
```

```
  (lambda
    (get-kids tree id)
    "0-based index of a target within its parent's kids."
    (let
      ((sibs (structed/siblings get-kids tree id)))
      (let
        ((i
          (structed/list/index-of
            sibs
            (structed/find get-kids tree id)
            (lambda
              (a b)
              (string-eq (structed/id a) (structed/id b))))))
        (if (< i 0) 0 i)))))

(define structed/siblings
  (lambda
    (get-kids tree id)
    "Sibling list of the node with id, or () for the root."
    (let
      ((parent (structed/find-parent get-kids tree id)))
      (if parent (get-kids parent) (list)))))

(define structed/slurp-backward
  (lambda
    (get-kids set-kids tree id)
    "Absorb a target's previous sibling into it; returns (tree id)."
    (let
      ((parent (structed/find-parent get-kids tree id)))
      (if
        (or (not parent) (string-eq id (structed/id tree)))
        (list tree id)
        (let
          ((sibs (get-kids parent)))
          (let
            ((i (structed/sibling-index get-kids tree id)))
            (if
              (<= i 0)
              (list tree id)
              (let
                ((prev (structed/list/nth sibs (- i 1))))
                (let
                  ((absorbed
                    (set-kids
                      (structed/list/nth sibs i)
                      (cons prev (get-kids (structed/list/nth sibs i))))))
                  (let
                    ((new-sibs
                      (structed/list/remove-at
                        (structed/list/replace-at sibs i absorbed)
                        (- i 1))))
                    (let
                      ((new-parent (set-kids parent new-sibs)))
                      (let
                        ((new-tree
                          (structed/replace
                            get-kids
                            set-kids
                            tree
                            (structed/id parent)
                            new-parent)))
                        (list new-tree id))))))))))))))

(define structed/slurp-forward
  (lambda
    (get-kids set-kids tree id)
    "Absorb a target's next sibling into it; returns (tree id)."
    (let
      ((parent (structed/find-parent get-kids tree id)))
      (if
        (or (not parent) (string-eq id (structed/id tree)))
        (list tree id)
        (let
```

```
          ((sibs (get-kids parent)))
          (let
            ((i (structed/sibling-index get-kids tree id)))
            (if
              (>= (+ i 1) (structed/count sibs))
              (list tree id)
              (let
                ((next (structed/list/nth sibs (+ i 1))))
                (let
                  ((absorbed
                    (set-kids
                      (structed/list/nth sibs i)
                      (append
                        (get-kids (structed/list/nth sibs i))
                        (list next)))))
                  (let
                    ((new-sibs
                      (structed/list/remove-at
                        (structed/list/replace-at sibs i absorbed)
                        (+ i 1))))
                    (let
                      ((new-parent (set-kids parent new-sibs)))
                      (let
                        ((new-tree
                          (structed/replace
                            get-kids
                            set-kids
                            tree
                            (structed/id parent)
                            new-parent)))
                        (list new-tree id))))))))))))

(define structed/splice
  (lambda
    (get-kids set-kids tree id)
    "Lift a target's children up into its slot (removing it); returns (tree
     sel)."
    (let
      ((target (structed/find get-kids tree id)))
      (if
        (or (not target) (string-eq id (structed/id tree)))
        (list tree id)
        (let
          ((kids (get-kids target)))
          (let
            ((parent (structed/find-parent get-kids tree id)))
            (let
              ((parent-id (if (null? parent) id (structed/id parent))))
              (let
                ((res
                  (structed/reattach
                    get-kids
                    set-kids
                    tree
                    id
                    (lambda
                      (parent sibs i)
                      (structed/list/insert-list-at
                        (structed/list/remove-at sibs i)
                        i
                        kids)))))
                (let
                  ((new-tree (car res)))
                  (let
                    ((sel
                      (if (null? kids) parent-id (structed/id (car kids)))))
                    (list new-tree sel)))))))))))

(define structed/split
  (lambda
    (get-kids set-kids make-node tree id pos)
    "Split a target's children at pos into two sibling nodes; returns (tree
     first-id)."
```

```
    (let
      ((target (structed/find get-kids tree id)))
      (if
        (or (not target) (string-eq id (structed/id tree)))
        (list tree id)
        (let
          ((kids (get-kids target)))
          (let
            ((parts (structed/list/split-at kids pos)))
            (let
              ((before (car parts)) (after (car (cdr parts))))
              (let
                ((n1 (make-node before)) (n2 (make-node after)))
                (let
                  ((parent (structed/find-parent get-kids tree id)))
                  (if
                    (not parent)
                    (list tree id)
                    (let
                      ((sibs (get-kids parent)))
                      (let
                        ((i (structed/sibling-index get-kids tree id)))
                        (let
                          ((new-sibs
                            (structed/list/insert-at
                              (structed/list/insert-at
                                (structed/list/remove-at sibs i)
                                i
                                n1)
                              (+ i 1)
                              n2)))
                          (let
                            ((new-parent (set-kids parent new-sibs)))
                            (let
                              ((new-tree
                                (structed/replace
                                  get-kids
                                  set-kids
                                  tree
                                  (structed/id parent)
                                  new-parent)))
                              (list new-tree (structed/id
    n1))))))))))))))))

(define structed/transpose
  (lambda
    (get-kids set-kids tree id)
    "Swap a target with its next sibling (or previous at the end); returns
     (tree id)."
    (let
      ((parent (structed/find-parent get-kids tree id)))
      (if
        (or (not parent) (string-eq id (structed/id tree)))
        (list tree id)
        (let
          ((sibs (get-kids parent)))
          (let
            ((i (structed/sibling-index get-kids tree id)))
            (if
              (< (structed/count sibs) 2)
              (list tree id)
              (let
                ((j
                  (if (>= (+ i 1) (structed/count sibs)) (- i 1) (+ i 1))))
                (let
                  ((a (structed/list/nth sibs i))
                    (b (structed/list/nth sibs j)))
                  (let
                    ((new-sibs
                      (structed/list/replace-at
                        (structed/list/replace-at sibs i b)
                        j
                        a)))
```

```
                    (let
                      ((new-parent (set-kids parent new-sibs)))
                      (let
                        ((new-tree
                          (structed/replace
                            get-kids
                            set-kids
                            tree
                            (structed/id parent)
                            new-parent)))
                        (list new-tree id))))))))))))

(define structed/update
  (lambda
    (get-kids set-kids tree id f)
    "Apply f to the node at id, rebuilding the tree along the way."
    (if
      (string-eq (structed/id tree) id)
      (f tree)
      (set-kids
        tree
        (structed/update-kids
          get-kids
          set-kids
          (get-kids tree)
          id
          f)))))

(define structed/update-kids
  (lambda
    (get-kids set-kids kids id f)
    "Apply update across a sibling list, mapping each child."
    (if
      (null? kids)
      (list)
      (cons
        (structed/update get-kids set-kids (car kids) id f)
        (structed/update-kids get-kids set-kids (cdr kids) id f)))))

(define structed/wrap
  (lambda
    (get-kids set-kids make-wrap-node tree id head)
    "Wrap a target in a new node (from make-wrap-node); returns (tree
     wrap-id)."
    (let
      ((target (structed/find get-kids tree id)))
      (if
        (not target)
        (list tree id)
        (let
          ((wrapped
            (if
              (null? head)
              (make-wrap-node target)
              (make-wrap-node target head))))
          (let
            ((parent (structed/find-parent get-kids tree id)))
            (if
              (not parent)
              (list tree id)
              (let
                ((sibs (get-kids parent)))
                (let
                  ((i (structed/sibling-index get-kids tree id)))
                  (let
                    ((new-sibs (structed/list/replace-at sibs i wrapped)))
                    (let
                      ((new-parent (set-kids parent new-sibs)))
                      (let
                        ((new-tree
                          (structed/replace
                            get-kids
                            set-kids
```

```
719                             tree
720                             (structed/id parent)
721                             new-parent)))
722                         (list new-tree (structed/id wrapped))))))))))))))
```

205 libs/structed/list/lib.pp

```
;; package structed/list -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push

(define structed/list/drop
  (lambda
    (xs n)
    "Return xs minus its first n elements."
    (if
      (or (null? xs) (<= n 0))
      xs
      (structed/list/drop (cdr xs) (- n 1)))))

(define structed/list/index-of
  (lambda
    (xs x eq?)
    "Index of first element equal to x, or -1."
    (structed/list/index-of-acc xs x eq? 0)))

(define structed/list/index-of-acc
  (lambda
    (xs x eq? i)
    "Accumulator for index-of starting the scan at i."
    (if
      (null? xs)
      -1
      (if
        (eq? (car xs) x)
        i
        (structed/list/index-of-acc (cdr xs) x eq? (+ i 1))))))

(define structed/list/insert-at
  (lambda
    (xs i v)
    "Insert v into xs at index i (clamped to front)."
    (if
      (or (null? xs) (<= i 0))
      (cons v xs)
      (cons
        (car xs)
        (structed/list/insert-at (cdr xs) (- i 1) v)))))

(define structed/list/insert-list-at
  (lambda
    (xs i ys)
    "Insert every element of ys into xs starting at index i."
    (if
      (null? ys)
      xs
      (structed/list/insert-list-at
        (structed/list/insert-at xs i (car ys))
        (+ i 1)
        (cdr ys)))))

(define structed/list/nth
  (lambda
    (xs i)
    "Element at index i of xs, or () when out of range."
    (if
      (null? xs)
      ()
      (if
        (<= i 0)
        (car xs)
        (structed/list/nth (cdr xs) (- i 1))))))

(define structed/list/remove-at
  (lambda
    (xs i)
    "xs with the element at index i removed."
    (if
      (null? xs)
```

```
      (list)
      (if
        (<= i 0)
        (cdr xs)
        (cons
          (car xs)
          (structed/list/remove-at (cdr xs) (- i 1)))))))

(define structed/list/replace-at
  (lambda
    (xs i v)
    "xs with the element at index i replaced by v."
    (if
      (null? xs)
      (list)
      (if
        (<= i 0)
        (cons v (cdr xs))
        (cons
          (car xs)
          (structed/list/replace-at (cdr xs) (- i 1) v))))))

(define structed/list/split-at
  (lambda
    (xs i)
    "Split xs into (taken dropped) at index i."
    (list
      (structed/list/take xs i)
      (structed/list/drop xs i))))

(define structed/list/take
  (lambda
    (xs n)
    "First n elements of xs."
    (if
      (or (null? xs) (<= n 0))
      (list)
      (cons (car xs) (structed/list/take (cdr xs) (- n 1))))))
```

206 libs/sync/lib.pp

```
;; package sync -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push
```

207 libs/theme-as400/lib.pp

```
;; package theme-as400 -- AS/400 CRT green-on-black theme (image.themes)
;; Loaded by (theme/set "as400") / (load-library "theme-as400").
;;
;; Token-only theme: green/black palette + monospace font stacks. The
;; CRT feel comes from color + font alone -- no structural sheet. The
;; old global CRT clobber (monospace body, uppercase headings, 2px
;; borders) that used to hide in notes/tree-css is gone; structure lives
;; in components now, themes only declare tokens.

(package theme-as400
  (export
    theme/name
    theme/bg
    theme/border
    theme/chrome
    theme/err
    theme/ink
    theme/link
    theme/muted
    theme/ok
    theme/panel
    theme/row-alt
    theme/shadow
    theme/visited
    theme/sel-bg
    theme/sel-accent
    theme/pending-bg
    theme/pending-ink
    theme/pending-border
    theme/done-bg
    theme/done-ink
    theme/done-muted
    theme/waiting-bg
    theme/archived-bg
    theme/archived-ink
    theme/archived-border
    theme/cancelled-bg
    theme/today-bg
    theme/banner-bg
    theme/banner-border
    theme/banner-ink
    theme/badge-bg
    theme/badge-border
    theme/font-sans
    theme/font-mono
    theme/extra))

(define theme/name
  "as400")

(define theme/bg
  "#111111")

(define theme/border
  "#33cc33")

(define theme/chrome
  "#1a2a1a")

(define theme/err
  "#ff4444")

(define theme/ink
  "#33cc33")

(define theme/link
  "#33ff33")

(define theme/muted
  "#aaaaaa")

```

```
(define theme/ok
  "#33ff33")

(define theme/panel
  "#1a1a1a")

(define theme/row-alt
  "#151515")

(define theme/shadow
  "#222222")

(define theme/visited
  "#2a9a2a")

;; --- semantic style tokens (dark CRT values, matching phosphor) -----
(define theme/sel-bg "#1a3a1a")
(define theme/sel-accent "#2a9a2a")
(define theme/pending-bg "#1f3a1f")
(define theme/pending-ink "#88cc44")
(define theme/pending-border "#886600")
(define theme/done-bg "#0f2a10")
(define theme/done-ink "#55cc55")
(define theme/done-muted "#4a6a4a")
(define theme/waiting-bg "#1a2a3a")
(define theme/archived-bg "#1a1a1a")
(define theme/archived-ink "#777777")
(define theme/archived-border "#555555")
(define theme/cancelled-bg "#222222")
(define theme/today-bg "#333a1a")
(define theme/banner-bg "#3a2a00")
(define theme/banner-border "#886600")
(define theme/banner-ink "#ffdd88")
(define theme/badge-bg "#3a2f00")
(define theme/badge-border "#8b6914")

;; --- font stacks (as400: monospace everywhere -- the CRT look) -----
(define theme/font-sans
  "\"Courier New\",\"Lucida Console\",Monaco,\"DejaVu Sans
     Mono\",monospace")
(define theme/font-mono
  "\"Courier New\",\"Lucida Console\",Monaco,\"DejaVu Sans
     Mono\",monospace")

;; --- per-theme structural sheet hook (no-op) ----------------------
;; Themes are token-only (colors + fonts); no structural sheets. The
;; CRT look is just green-on-black tokens + monospace fonts.
(define theme/extra
  (lambda
    ()
    "No-op: themes are token-only."
    (css/sheet (list))))
```

208 libs/theme-classic/lib.pp

```
;; package theme-classic -- platinum System 7 gray (image.themes)
;; Loaded by (theme/set "classic") / (load-library "theme-classic").
;; The old product default look people either love or flee from.

(define theme/name
  "classic")

(define theme/bg
  "#d4d0c8")

(define theme/ink
  "#000000")

(define theme/panel
  "#ece9d8")

(define theme/chrome
  "#c0c0c0")

(define theme/link
  "#0000ee")

(define theme/visited
  "#551a8b")

(define theme/muted
  "#555555")

(define theme/err
  "#800000")

(define theme/ok
  "#006600")

(define theme/border
  "#808080")

(define theme/row-alt
  "#e8e4d8")

(define theme/shadow
  "#808080")

;; --- semantic style tokens (platinum light values) --------------------
(define theme/sel-bg "#fff8dc")
(define theme/sel-accent "#000080")
(define theme/pending-bg "#fff3cd")
(define theme/pending-ink "#664400")
(define theme/pending-border "#886600")
(define theme/done-bg "#cce8cc")
(define theme/done-ink "#006600")
(define theme/done-muted "#555555")
(define theme/waiting-bg "#d8e4f0")
(define theme/archived-bg "#e4e0d8")
(define theme/archived-ink "#555555")
(define theme/archived-border "#808080")
(define theme/cancelled-bg "#ddd8d0")
(define theme/today-bg "#ffffe0")
(define theme/banner-bg "#fff3cd")
(define theme/banner-border "#800000")
(define theme/banner-ink "#000000")
(define theme/badge-bg "#ffe8a3")
(define theme/badge-border "#8b6914")

;; --- font stacks (classic: sans body, mono code) ----------------
(define theme/font-sans
  "Geneva,\"Lucida Grande\",\"DejaVu Sans\",Verdana,sans-serif")
(define theme/font-mono
  "\"Courier New\",\"Lucida Console\",Monaco,\"DejaVu Sans
     Mono\",monospace")

```

```
;; --- per-theme structural sheet hook (no-op) ----------------------
;; Themes are token-only (colors + fonts); no structural sheets.
(define theme/extra
  (lambda
    ()
    "No-op: themes are token-only."
    (css/sheet (list))))
```

209 libs/theme-midnight/lib.pp

```
;; package theme-midnight -- soft blue-gray dark (image.themes)
;; Loaded by (theme/set "midnight") / (load-library "theme-midnight").

(define theme/name
  "midnight")

;; WHITEPURPOSE: theme-midnight is THE WHITEOUT/PROBE THEME -- every
;; core token is magenta so ANY element still painted by a hardcoded
;; hex (not var(--*)) glows magenta and exposes the leak. Real users
;; pick midnight only to audit CSS theming coverage.

(define theme/bg
  "magenta")

(define theme/ink
  "magenta")

(define theme/panel
  "magenta")

(define theme/chrome
  "magenta")

(define theme/link
  "magenta")

(define theme/visited
  "magenta")

(define theme/muted
  "magenta")

(define theme/err
  "#bf616a")

(define theme/ok
  "#a3be8c")

(define theme/border
  "#3b4558")

(define theme/row-alt
  "#141b26")

(define theme/shadow
  "magenta")

;; --- semantic style tokens (dark blue-gray values) ------------------
(define theme/sel-bg "magenta")
(define theme/sel-accent "magenta")
(define theme/pending-bg "magenta")
(define theme/pending-ink "magenta")
(define theme/pending-border "magenta")
(define theme/done-bg "magenta")
(define theme/done-ink "magenta")
(define theme/done-muted "magenta")
(define theme/waiting-bg "magenta")
(define theme/archived-bg "magenta")
(define theme/archived-ink "magenta")
(define theme/archived-border "magenta")
(define theme/cancelled-bg "magenta")
(define theme/today-bg "magenta")
(define theme/banner-bg "magenta")
(define theme/banner-border "magenta")
(define theme/banner-ink "magenta")
(define theme/badge-bg "magenta")
(define theme/badge-border "magenta")

;; --- font stacks (midnight: sans body, mono code) ----------------
;; Colors are magenta on purpose (whiteout probe -- see above). Fonts
;; are real stacks so layout still reads while you hunt for leaks.
```

```
(define theme/font-sans
  "Geneva,\"Lucida Grande\",\"DejaVu Sans\",Verdana,sans-serif")
(define theme/font-mono
  "\"Courier New\",\"Lucida Console\",Monaco,\"DejaVu Sans
     Mono\",monospace")

;; --- per-theme structural sheet hook (no-op) ----------------------
;; Themes are token-only (colors + fonts); no structural sheets.
(define theme/extra
  (lambda
    ()
    "No-op: themes are token-only."
    (css/sheet (list))))
```

210 libs/theme-paper/lib.pp

```
;; package theme-paper -- warm cream paper, dark ink (image.themes)
;; Loaded by (theme/set "paper") / (load-library "theme-paper").
;; Token-only: pins theme/* onto the current branch.

(define theme/name
  "paper")

(define theme/bg
  "#f4f0e6")

(define theme/ink
  "#1c1916")

(define theme/panel
  "#ebe4d4")

(define theme/chrome
  "#ddd4c0")

(define theme/link
  "#0b57a8")

(define theme/visited
  "#5b3d8a")

(define theme/muted
  "#6b6458")

(define theme/err
  "#a32020")

(define theme/ok
  "#1f6b3a")

(define theme/border
  "#c4b8a0")

(define theme/row-alt
  "#efe9db")

(define theme/shadow
  "#c9c0ae")

;; --- semantic style tokens (light paper values) ----------------------
(define theme/sel-bg "#fff8dc")
(define theme/sel-accent "#000080")
(define theme/pending-bg "#fff8dc")
(define theme/pending-ink "#664400")
(define theme/pending-border "#886600")
(define theme/done-bg "#cfc")
(define theme/done-ink "#226622")
(define theme/done-muted "#666666")
(define theme/waiting-bg "#e0e8ff")
(define theme/archived-bg "#eeeeee")
(define theme/archived-ink "#555555")
(define theme/archived-border "#666666")
(define theme/cancelled-bg "#dddddd")
(define theme/today-bg "#ffffe0")
(define theme/banner-bg "#fff3cd")
(define theme/banner-border "#8b0000")
(define theme/banner-ink "#1a1a1a")
(define theme/badge-bg "#ffe8a3")
(define theme/badge-border "#8b6914")

;; --- font stacks (paper: sans body, mono code) -------------------
(define theme/font-sans
  "Geneva,\"Lucida Grande\",\"DejaVu Sans\",Verdana,sans-serif")
(define theme/font-mono
  "\"Courier New\",\"Lucida Console\",Monaco,\"DejaVu Sans
     Mono\",monospace")

```

```
;; --- per-theme structural sheet hook (no-op) ----------------------
;; Themes are token-only (colors + fonts); no structural sheets.
(define theme/extra
  (lambda
    ()
    "No-op: themes are token-only."
    (css/sheet (list))))
```

211 libs/theme-prickly-night/lib.pp

```
;; package theme-prickly-night -- desert night: blue sky, brown earth
     (image.themes)
;; Loaded by (theme/set "prickly-night") / (load-library
     "theme-prickly-night").
;; Token-only: pins theme/* onto the current branch.
;; Signature move: blue-slate page/chrome over warm brown panels --
;; night sky above, desert floor below. NOT black: gruvbox-warm-dark
;; midtones, cream ink, subtle borders by design (~1.5:1). Chips are
;; darker/hue-shifted washes on the brown panel (pending 1.20:1, today
;; 1.28:1). Family identity shared with theme-prickly: yellow pending,
;; green done, blue waiting, magenta accent/selection.

(define theme/name
  "prickly-night")

(define theme/bg
  "#272b33")

(define theme/ink
  "#e9e4d5")

(define theme/panel
  "#332c24")

(define theme/chrome
  "#303740")

(define theme/link
  "#82aacc")

(define theme/visited
  "#a592d4")

(define theme/muted
  "#a8a292")

(define theme/err
  "#e57676")

(define theme/ok
  "#8faf7a")

(define theme/border
  "#4d463e")

(define theme/row-alt
  "#2c3038")

(define theme/shadow
  "#14161c")

;; --- semantic style tokens (desert night values) --------------------
;; Selection is a dark magenta wash with a fruit-pink accent.
(define theme/sel-bg "#4a2f42")
(define theme/sel-accent "#e59ac2")
(define theme/pending-bg "#3f3a1e")
(define theme/pending-ink "#e3cf8f")
(define theme/pending-border "#8a7c3a")
(define theme/done-bg "#31402a")
(define theme/done-ink "#a9c894")
(define theme/done-muted "#9db08a")
(define theme/waiting-bg "#2e3d50")
(define theme/archived-bg "#38322b")
(define theme/archived-ink "#b3ab99")
(define theme/archived-border "#5a5346")
(define theme/cancelled-bg "#35302a")
(define theme/today-bg "#453e1c")
(define theme/banner-bg "#4a3320")
(define theme/banner-border "#b97c3e")
(define theme/banner-ink "#f0e4d0")
(define theme/badge-bg "#4a2f42")
```

```
(define theme/badge-border "#c27ba0")

;; --- font stacks (prickly-night: sans body, mono code) ------------
(define theme/font-sans
  "Geneva,\"Lucida Grande\",\"DejaVu Sans\",Verdana,sans-serif")
(define theme/font-mono
  "\"Courier New\",\"Lucida Console\",Monaco,\"DejaVu Sans
     Mono\",monospace")

;; --- per-theme structural sheet hook (no-op) ----------------------
;; Themes are token-only (colors + fonts); no structural sheets.
(define theme/extra
  (lambda
    ()
    "No-op: themes are token-only."
    (css/sheet (list))))
```

212 libs/theme-prickly/lib.pp

```
;; package theme-prickly -- desert day: sand & sky, green-olive ink (image.themes)
;; Loaded by (theme/set "prickly") / (load-library "theme-prickly").
;; Token-only: pins theme/* onto the current branch.
;; Family: prickly (light) / prickly-night (dark) -- shared semantic hues
;; (yellow pending, green done, blue waiting, magenta accent), Catppuccin-style
;; same-accents-different-base. All ink/muted values checked >= 4.5:1 WCAG AA.

(define theme/name
  "prickly")

(define theme/bg
  "#f6f0e0")

(define theme/ink
  "#2c3126")

(define theme/panel
  "#eee6d1")

(define theme/chrome
  "#e2d8bf")

(define theme/link
  "#3f6fae")

(define theme/visited
  "#a34d7d")

(define theme/muted
  "#6a6553")

(define theme/err
  "#b23b3b")

(define theme/ok
  "#3d7a3a")

(define theme/border
  "#c2b48a")

(define theme/row-alt
  "#f1ead9")

(define theme/shadow
  "#c9b892")

;; --- semantic style tokens (desert day values) ----------------------
;; Selection is magenta -- the pricklypear fruit. Visited links too.
(define theme/sel-bg "#f2d6e3")
(define theme/sel-accent "#a34d7d")
(define theme/pending-bg "#f2e2ae")
(define theme/pending-ink "#6b5410")
(define theme/pending-border "#a8892a")
(define theme/done-bg "#d3e5c2")
(define theme/done-ink "#2f6b2f")
(define theme/done-muted "#57634a")
(define theme/waiting-bg "#cfe0f0")
(define theme/archived-bg "#e8e2d2")
(define theme/archived-ink "#6a6553")
(define theme/archived-border "#b9ac8d")
(define theme/cancelled-bg "#e4ddd2")
(define theme/today-bg "#f8e9a8")
(define theme/banner-bg "#f7e9d0")
(define theme/banner-border "#a35b1f")
(define theme/banner-ink "#3c2f1a")
(define theme/badge-bg "#f6dfe9")
(define theme/badge-border "#a34d7d")

```

```
;; --- font stacks (prickly: sans body, mono code) ------------------
(define theme/font-sans
  "Geneva,\"Lucida Grande\",\"DejaVu Sans\",Verdana,sans-serif")
(define theme/font-mono
  "\"Courier New\",\"Lucida Console\",Monaco,\"DejaVu Sans
     Mono\",monospace")

;; --- per-theme structural sheet hook (no-op) ----------------------
;; Themes are token-only (colors + fonts); no structural sheets.
(define theme/extra
  (lambda
    ()
    "No-op: themes are token-only."
    (css/sheet (list))))
```

213 libs/theme/lib.pp

```
;; package theme -- exported from live branch admin/personal
;; Sync: scripts/pp_sync.py pull|push

(package theme
  (export
    theme/available
    theme/card-html
    theme/cards-html
    theme/current
    theme/entry-desc
    theme/entry-display
    theme/entry-name
    theme/entry-package
    theme/find
    theme/page
    theme/set
    theme/swatch-html
    theme/vars
    theme/extra
))

(define theme/archived-bg
  "#eeeeee")

(define theme/archived-border
  "#666666")

(define theme/archived-ink
  "#555555")

(define theme/available
  (lambda
    ()
    (list/map
      (lambda
        (e)
        (list
          (list "name" (theme/entry-name e))
          (list "display" (theme/entry-display e))
          (list "description" (theme/entry-desc e))
          (list "package" (theme/entry-package e))))
      theme/catalog)))

(define theme/badge-bg
  "#ffe8a3")

(define theme/badge-border
  "#8b6914")

(define theme/banner-bg
  "#fff3cd")

(define theme/banner-border
  "#8b0000")

(define theme/banner-ink
  "#1a1a1a")

(define theme/bg
  "#f4f0e6")

(define theme/border
  "#c4b8a0")

(define theme/cancelled-bg
  "#dddddd")

(define theme/card-html
  (lambda
    (e)
    (let
```

```
      ((name (theme/entry-name e))
        (display (theme/entry-display e))
        (desc (theme/entry-desc e))
        (cur (theme/current)))
      (let
        ((is-cur (string-eq name cur)))
        (html/div
          (html/attrs
            (list
              (html/attr
                "class"
                (if
                  is-cur
                  "card theme-card theme-card-current"
                  "card theme-card"))
              (html/attr "data-theme" name)))
          (html/join
            (list
              (html/h3 display)
              (theme/preview-swatch name)
              (html/p* (html/attr "class" "muted") (html/text desc))
              (if
                is-cur
                (html/p* (html/attr "class" "ok") "Current theme")
                (html/form
                  "POST"
                  "/settings/theme"
                  (html/attrs
                    (list
                      (html/attr "hx-post" "/settings/theme")
                      (html/attr "hx-target" "#theme-cards")
                      (html/attr "hx-swap" "outerHTML")))
                  (html/join
                    (list
                      (html/void
                        "input"
                        (html/attrs
                          (list
                            (html/attr "type" "hidden")
                            (html/attr "name" "name")
                            (html/attr "value" name))))
                      (html/button
                        (html/attr "type" "submit")
                        (string-append "Use " display)))))))))))

(define theme/cards-html
  (lambda
    ()
    (html/div
      (html/attr "id" "theme-cards")
      (html/join (list/map theme/card-html theme/catalog)))))

(define theme/catalog
  (list
    (list
      "paper"
      "Paper"
      "Warm cream paper, dark ink, soft brown chrome. Easy on the eyes."
      "theme-paper")
    (list
      "midnight"
      "Midnight"
      "Soft blue-gray dark theme -- no phosphor green."
      "theme-midnight")
    (list
      "classic"
      "Classic Mac"
      "Platinum gray System 7 vibes. The old product default."
      "theme-classic")
    (list
      "as400"
      "AS/400"
      "CRT green on near-black -- the original notes-tree terminal skin."
```

```
      "theme-as400")))

(define theme/chrome
  "#ddd4c0")

(define theme/current
  (lambda
    ()
    (guard (lambda (_) "paper") (as-str theme/name))))

(define theme/done-bg
  "#cfc")

(define theme/done-ink
  "#226622")

(define theme/done-muted
  "#666666")

(define theme/entry-desc
  (lambda (e) (as-str (car (cdr (cdr e))))))

(define theme/entry-display
  (lambda (e) (as-str (car (cdr e)))))

(define theme/entry-name
  (lambda (e) (as-str (car e))))

(define theme/entry-package
  (lambda (e) (as-str (car (cdr (cdr (cdr e)))))))

(define theme/err
  "#a32020")

(define theme/extra
  (lambda () (css/sheet (list))))

(define theme/font-mono
  "\"Courier New\",\"Lucida Console\",Monaco,\"DejaVu Sans
     Mono\",monospace")

(define theme/font-sans
  "Geneva,\"Lucida Grande\",\"DejaVu Sans\",Verdana,sans-serif")

(define theme/find
  (lambda
    (name)
    (let
      ((want (as-str name)))
      (list/foldl
        (lambda
          (acc e)
          (if
            (null? acc)
            (if (string-eq (theme/entry-name e) want) e acc)
            acc))
        ()
        theme/catalog))))

(define theme/ink
  "#1c1916")

(define theme/link
  "#0b57a8")

(define theme/muted
  "#6b6458")

(define theme/name
  "paper")

(define theme/ok
  "#1f6b3a")
```

```
217
218 (define theme/page
219   (lambda
220     ()
221     (if
222       (string-eq (request-method) "POST")
223       (let
224         ((name (as-str (request-param "name"))))
225         (seq
226           (theme/set name)
227           (if
228             (hx-request?)
229             (render-html (theme/cards-html))
230             (redirect "/settings/theme"))))
231       (html/page
232         (list
233           (html/h1 "Themes")
234           (html/p*
235             (html/attr "class" "muted")
236             "Installable looks for your garden. Switching pins theme tokens
    on your branch; checkout main restores the system default.")
237           (html/p*
238             (html/attr "class" "muted")
239             (string-append "Current: " (theme/current)))
240           (theme/cards-html)
241           (html/tag
242             "style"
243             ""
244             (html/join
245               (list
246                 ".theme-card{margin:.75rem 0;max-width:28rem}"
247                 ".theme-card-current{outline:2px solid
    var(--ok,#33ff33);outline-offset:2px}"
248                 ".theme-swatch{display:flex;gap:.35rem;margin:.4rem 0}"
249
    ".theme-swatch-chip{display:inline-block;width:1.5rem;height:1.5rem;border:1px
    solid rgba(0,0,0,.25);border-radius:3px}"))))))))
250
251 (define theme/panel
252   "#ebe4d4")
253
254 (define theme/pending-bg
255   "#fff8dc")
256
257 (define theme/pending-border
258   "#886600")
259
260 (define theme/pending-ink
261   "#664400")
262
263 (define theme/preview-swatch
264   (lambda
265     (name)
266     (if
267       (string-eq name "paper")
268       (theme/swatch-html
269         "#f4f0e6"
270         "#1c1916"
271         "#ebe4d4"
272         "#ddd4c0"
273         "#0b57a8")
274       (if
275         (string-eq name "midnight")
276         (theme/swatch-html
277           "#0f1419"
278           "#d8dee9"
279           "#1a2332"
280           "#15202b"
281           "#7aa2f7")
282         (if
283           (string-eq name "classic")
284           (theme/swatch-html
285             "#d4d0c8"
```

```
            "#000000"
            "#ece9d8"
            "#c0c0c0"
            "#0000ee")
          (if
            (string-eq name "as400")
            (theme/swatch-html
              "#111111"
              "#33cc33"
              "#1a1a1a"
              "#1a2a1a"
              "#33ff33")
            (theme/swatch-html
              "#111111"
              "#33cc33"
              "#1a1a1a"
              "#1a2a1a"
              "#33ff33")))))))

(define theme/row-alt
  "#efe9db")

(define theme/sel-accent
  "#000080")

(define theme/sel-bg
  "#fff8dc")

(define theme/set
  (lambda
    (name)
    (let
      ((e (theme/find name)))
      (if
        (null? e)
        (error
          (string-append
            "theme/set: unknown theme: "
            (as-str name)))
        (let
          ((pkg (theme/entry-package e))
            (want (theme/entry-name e)))
          (seq (load-library pkg) want))))))

(define theme/shadow
  "#c9c0ae")

(define theme/swatch-html
  (lambda
    (bg ink panel chrome link)
    (html/tag
      "div"
      (html/attr "class" "theme-swatch")
      (html/join
        (list
          (html/tag
            "span"
            (html/attrs
              (list
                (html/attr "class" "theme-swatch-chip")
                (html/attr "style" (string-append "background:" bg))))
            "")
          (html/tag
            "span"
            (html/attrs
              (list
                (html/attr "class" "theme-swatch-chip")
                (html/attr "style" (string-append "background:" ink))))
            "")
          (html/tag
            "span"
            (html/attrs
              (list
```

```
                (html/attr "class" "theme-swatch-chip")
                (html/attr "style" (string-append "background:" panel))))
            "")
          (html/tag
            "span"
            (html/attrs
              (list
                (html/attr "class" "theme-swatch-chip")
                (html/attr "style" (string-append "background:" chrome))))
            "")
          (html/tag
            "span"
            (html/attrs
              (list
                (html/attr "class" "theme-swatch-chip")
                (html/attr "style" (string-append "background:" link))))
            "")))))

(define theme/today-bg
  "#ffffe0")

(define theme/vars
  (lambda
    ()
    (css/sheet
      (list
        (css/rule
          ":root"
          (list
            "--bg"
            theme/bg
            "--ink"
            theme/ink
            "--font-sans"
            theme/font-sans
            "--font-mono"
            theme/font-mono
            "--panel"
            theme/panel
            "--chrome"
            theme/chrome
            "--link"
            theme/link
            "--visited"
            theme/visited
            "--muted"
            theme/muted
            "--err"
            theme/err
            "--ok"
            theme/ok
            "--border"
            theme/border
            "--row-alt"
            theme/row-alt
            "--shadow"
            theme/shadow
            "--sel-bg"
            theme/sel-bg
            "--sel-accent"
            theme/sel-accent
            "--pending-bg"
            theme/pending-bg
            "--pending-ink"
            theme/pending-ink
            "--pending-border"
            theme/pending-border
            "--done-bg"
            theme/done-bg
            "--done-ink"
            theme/done-ink
            "--done-muted"
            theme/done-muted
```

```
            "--waiting-bg"
            theme/waiting-bg
            "--archived-bg"
            theme/archived-bg
            "--archived-ink"
            theme/archived-ink
            "--archived-border"
            theme/archived-border
            "--cancelled-bg"
            theme/cancelled-bg
            "--today-bg"
            theme/today-bg
            "--banner-bg"
            theme/banner-bg
            "--banner-border"
            theme/banner-border
            "--banner-ink"
            theme/banner-ink
            "--badge-bg"
            theme/badge-bg
            "--badge-border"
            theme/badge-border))))))

(define theme/visited
  "#5b3d8a")

(define theme/waiting-bg
  "#e0e8ff")
```

214 libs/todo/db/lib.pp

```
;; package todo/db -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push

(define todo/db/create
  (lambda
    (type fields)
    (let
      ((r
        (tree/upsert
          (node-path type "new")
          "datum"
          (as-str type)
          fields)))
      (if
        (string-starts-with? r "data.")
        (todo/db/path-id r)
        r))))

(define todo/db/delete
  (lambda
    (type id)
    (let
      ((p (node-path type (as-str id))))
      (if
        (null? (tree/get p))
        ()
        (seq (tree/delete p) (as-str id))))))

(define todo/db/get
  (lambda
    (type id)
    (let
      ((n (tree/get (node-path type (as-str id)))))
      (if (null? n) () (dict-get n "body")))))

(define todo/db/insert-by-created
  (lambda
    (n sorted)
    (if
      (null? sorted)
      (list n)
      (if
        (string<?
          (as-str (dict-get n "created_at"))
          (as-str (dict-get (car sorted) "created_at")))
        (cons n sorted)
        (cons
          (car sorted)
          (todo/db/insert-by-created n (cdr sorted)))))))

(define todo/db/node-row
  (lambda
    (node)
    (cons
      (todo/db/path-id (dict-get node "path"))
      (dict-get node "body"))))

(define todo/db/path-id
  (lambda
    (path)
    (let
      ((h (substring path (- (string-length path) 32) 32)))
      (string-append
        (substring h 0 8)
        "-"
        (substring h 8 4)
        "-"
        (substring h 12 4)
        "-"
        (substring h 16 4)
        "-"
```

```
        (substring h 20 12)))))

(define todo/db/prefix
  (lambda
    (type)
    (let
      ((p (node-path type "")))
      (substring p 0 (- (string-length p) 1)))))

(define todo/db/rows
  (lambda
    (type limit)
    (map
      todo/db/node-row
      (tree/list-kind (todo/db/prefix type) "datum" limit))))

(define todo/db/sort-by-created
  (lambda
    (ns)
    (if
      (null? ns)
      (list)
      (todo/db/insert-by-created
        (car ns)
        (todo/db/sort-by-created (cdr ns))))))

(define todo/db/update
  (lambda
    (type id fields)
    (let
      ((p (node-path type (as-str id))))
      (if
        (null? (tree/get p))
        ()
        (let
          ((r (tree/upsert p "datum" (as-str type) fields)))
          (if (string-starts-with? r "data.") (as-str id) r))))))

(define todo/db/update-field
  (lambda
    (type id field value)
    (let
      ((fields (todo/db/get type (as-str id))))
      (if
        (null? fields)
        ()
        (todo/db/update type id (dict-set fields field value))))))
```

215 libs/todo/item/lib.pp

```
;; package todo/item -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push

(define todo/item/active
  (lambda
    (items)
    "Items that are not archived."
    (list/filter
      (lambda
        (row)
        (not (todo/state/archived? (todo/item/state (cdr row)))))
      items)))

(define todo/item/add
  (lambda
    (list-id title)
    "Create a new TODO item in list-id (or inbox) with title; returns the new id."
    (let
      ((lid
        (if
          (string-eq (as-str list-id) "")
          (todo/list/ensure-inbox)
          (as-str list-id))))
      (todo/tree/create
        "todo-item"
        (dict-set*
          "{}"
          (list
            "list_id"
            lid
            "title"
            (as-str title)
            "notes"
            ""
            "state"
            "TODO"
            "due"
            ""
            "position"
            "0"
            "done"
            "false"))))))

(define todo/item/all
  (lambda () "List up to 200 todo item rows." (todo/tree/rows "todo-item" 200)))

(define todo/item/archive
  (lambda (id) "Set item id to ARCHIVED." (todo/item/set-state id "ARCHIVED")))

(define todo/item/archived
  (lambda
    ()
    "All archived items."
    (list/filter
      (lambda
        (row)
        (todo/state/archived? (todo/item/state (cdr row))))
      (todo/item/all))))

(define todo/item/count-all
  (lambda
    (items)
    "Total number of items."
    (list/foldl (lambda (acc _) (+ acc 1)) 0 items)))

(define todo/item/count-open
  (lambda
    (items)
```

```
    "Number of items in an open state."
    (list/foldl
      (lambda
        (acc row)
        (if
          (todo/state/open? (todo/item/state (cdr row)))
          (+ acc 1)
          acc))
      0
      items)))

(define todo/item/cycle
  (lambda
    (id)
    "Advance item id to its next state; () when missing. Cycles within the
     item's OWN list's state set (custom columns when present, else the
     global default) so /todos and /kanban stay consistent --
     BORGE-1786336755."
    (let
      ((row (todo/tree/get "todo-item" id)))
      (if
        (null? row)
        ()
        (let
          ((cur (todo/item/state row))
            (list-id (todo/item/list-id row)))
          (if
            (todo/state/archived? cur)
            cur
            (todo/item/set-state id (todo/state/next-for-list cur
    list-id))))))))

(define todo/item/delete
  (lambda (id) "Delete item id." (todo/tree/del "todo-item" id)))

(define todo/item/due
  (lambda
    (fields)
    "The due field of item fields; empty when absent."
    (let
      ((d (dict-get fields "due")))
      (if (null? d) "" (as-str d)))))

(define todo/item/for-list
  (lambda
    (list-id)
    "All items belonging to list-id (inbox treated as empty list_id)."
    (let
      ((lid (as-str list-id))
        (inbox-id (todo/list/ensure-inbox)))
      (list/filter
        (lambda
          (row)
          (let
            ((fl (todo/item/list-id (cdr row))))
            (if
              (string-eq fl lid)
              #t
              (if (string-eq fl "") (string-eq lid inbox-id) #f))))
        (todo/item/all)))))

(define todo/item/get
  (lambda (id) "Fetch the item row with id." (todo/tree/get "todo-item"
    id)))

(define todo/item/html
  (lambda
    (view)
    "Render TODO items as HTML for the given view
     (from-notes/archived/everything or a list id)."
    (if
      (string-eq view "from-notes")
      (html/ul
```

```
        (html/attr "class" "card todo-list")
        (list/foldl
          (lambda
            (acc t)
            (string-append acc (todo/item/note-li t view)))
          ""
          (notes/outline/all-todos)))
      (if
        (string-eq view "archived")
        (html/ul
          (html/attr "class" "card todo-list")
          (list/foldl
            (lambda
              (acc row)
              (string-append acc (todo/item/list-li row view)))
            ""
            (todo/item/archived)))
        (if
          (string-eq view "everything")
          (html/join
            (list
              (html/h2 "Lists")
              (html/ul
                (html/attr "class" "card todo-list")
                (list/foldl
                  (lambda
                    (acc row)
                    (string-append acc (todo/item/list-li row view)))
                  ""
                  (todo/item/active (todo/item/all))))
              (html/h2 "From notes")
              (html/ul
                (html/attr "class" "card todo-list")
                (list/foldl
                  (lambda
                    (acc t)
                    (if
                      (todo/state/open? (car (cdr (cdr (cdr t)))))
                      (string-append acc (todo/item/note-li t view))
                      acc))
                  ""
                  (notes/outline/all-todos)))))
            (html/ul
              (html/attr "class" "card todo-list")
              (list/foldl
                (lambda
                  (acc row)
                  (string-append acc (todo/item/list-li row view)))
                ""
                (todo/item/active (todo/item/for-list view)))))))))

(define todo/item/list-id
  (lambda
    (fields)
    "The list_id field of item fields; empty when absent."
    (let
      ((lid (dict-get fields "list_id")))
      (if (null? lid) "" (as-str lid)))))

(define todo/item/list-li
  (lambda
    (row view)
    "Render a single todo item row as an HTML <li>."
    (let
      ((id (car row)) (fields (cdr row)))
      (let
        ((title (todo/item/title fields))
          (state (todo/item/state fields))
          (open (todo/state/open? (todo/item/state fields))))
        (html/li
          (html/attr
            "class"
            (if
```

```
            open
            "todo-item"
            (if
              (todo/state/archived? state)
              "todo-item is-done is-archived"
              "todo-item is-done")))
          (html/join
            (list
              (html/form
                "POST"
                "/todos"
                (html/attrs
                  (list
                    (html/attr "class" "todo-check")
                    (html/attr "hx-post" "/todos")
                    (html/attr "hx-target" "#todo-main")
                    (html/attr "hx-swap" "innerHTML")))
                (html/join
                  (list
                    (html/hidden "action" "cycle")
                    (html/hidden "id" id)
                    (html/hidden "view" view)
                    (html/button
                      (html/attrs
                        (list
                          (html/attr "type" "submit")
                          (html/attr "class" (string-append "todo-box st-"
    state))
                          (html/attr "title" "cycle state")))
                      (if (string-eq state "") "?" state)))))
              (html/a
                (string-append "/todos?id=" id "&view=" view)
                (html/span
                  (html/attr "class" "todo-title")
                  (html/text title)))
              (if
                (string-eq state "DONE")
                (html/form
                  "POST"
                  "/todos"
                  (html/attrs
                    (list
                      (html/attr "class" "todo-archive")
                      (html/attr "hx-post" "/todos")
                      (html/attr "hx-target" "#todo-main")
                      (html/attr "hx-swap" "innerHTML")))
                  (html/join
                    (list
                      (html/hidden "action" "archive")
                      (html/hidden "id" id)
                      (html/hidden "view" view)
                      (html/button
                        (html/attrs
                          (list
                            (html/attr "type" "submit")
                            (html/attr "class" "secondary")
                            (html/attr "title" "archive done task")))
                        "arch"))))
                "")
              (html/form
                "POST"
                "/todos"
                (html/attrs
                  (list
                    (html/attr "class" "todo-del")
                    (html/attr "hx-post" "/todos")
                    (html/attr "hx-target" "#todo-main")
                    (html/attr "hx-swap" "innerHTML")))
                (html/join
                  (list
                    (html/hidden "action" "delete")
                    (html/hidden "id" id)
                    (html/hidden "view" view)
```

```
                    (html/button
                      (html/attrs
                        (list
                          (html/attr "type" "submit")
                          (html/attr "class" "secondary")
                          (html/attr "title" "delete")))
                      "del")))))))))))

(define todo/item/note-li
  (lambda
    (t view)
    "Render a TODO sourced from a note as an HTML <li>."
    (let
      ((slug (car t))
        (nid (car (cdr t)))
        (title (car (cdr (cdr t))))
        (state (car (cdr (cdr (cdr t))))))
      (let
        ((open (todo/state/open? state)))
        (html/li
          (html/attr
            "class"
            (if
              open
              "todo-item todo-from-note"
              "todo-item todo-from-note is-done"))
          (html/join
            (list
              (html/form
                "POST"
                "/todos"
                (html/attrs
                  (list
                    (html/attr "class" "todo-check")
                    (html/attr "hx-post" "/todos")
                    (html/attr "hx-target" "#todo-main")
                    (html/attr "hx-swap" "innerHTML")))
                (html/join
                  (list
                    (html/hidden "action" "cycle-note")
                    (html/hidden "slug" slug)
                    (html/hidden "node-id" nid)
                    (html/hidden "view" view)
                    (html/button
                      (html/attrs
                        (list
                          (html/attr "type" "submit")
                          (html/attr "class" (string-append "todo-box st-"
    state))
                          (html/attr "title" "cycle note TODO")))
                      (if (string-eq state "") "?" state)))))
              (html/span
                (html/attr "class" "todo-title")
                (html/text title))
              (html/a
                (string-append "/notes?slug=" slug)
                (html/span
                  (html/attr "class" "muted todo-src")
                  (html/text slug)))))))))

(define todo/item/notes
  (lambda
    (fields)
    "The notes field of item fields; empty when absent."
    (let
      ((n (dict-get fields "notes")))
      (if (null? n) "" (as-str n)))))

(define todo/item/save
  (lambda
    (id title notes due state list-id)
    "Update item id's fields; () when missing."
    (let
```

```
      ((fields (todo/item/get id)))
      (if
        (null? fields)
        ()
        (let
          ((lid
            (if
              (string-eq (as-str list-id) "")
              (todo/list/ensure-inbox)
              (as-str list-id))))
          (todo/tree/save
            "todo-item"
            id
            (dict-set*
              fields
              (list
                "title"
                (as-str title)
                "notes"
                (as-str notes)
                "due"
                (as-str due)
                "state"
                (as-str state)
                "list_id"
                lid))))))))

(define todo/item/set-state
  (lambda
    (id state)
    "Set item id's state field to state."
    (todo/tree/set-field
      "todo-item"
      id
      "state"
      (as-str state))))

(define todo/item/state
  (lambda
    (fields)
    "The state field of item fields; falls back to the legacy done flag
     (TODO/DONE)."
    (let
      ((s (dict-get fields "state")))
      (if
        (null? s)
        (let
          ((d (dict-get fields "done")))
          (if
            (null? d)
            "TODO"
            (if (string-eq d "true") "DONE" "TODO")))
        (as-str s)))))

(define todo/item/title
  (lambda (fields) "The title field of item fields." (as-str (dict-get
     fields "title"))))
```

216 libs/todo/lib.pp

```
;; package todo -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push

(package todo
  (export
    todo/add
    todo/active-items
    todo/list
    todo/page
    todo/archive
    todo/cycle
    todo/delete
    todo/item/add
    todo/item/active
    todo/item/archive
    todo/item/cycle
    todo/item/delete
    todo/list/all
    todo/list/create
    todo/list/delete
    todo/ensure-inbox
))

(define todo/active-items
  (lambda
    (items)
    (list/filter
      (lambda
        (row)
        (not (todo/archived? (todo/item-state (cdr row)))))
      items)))

(define todo/add
  (lambda
    (list-id title)
    (let
      ((lid
        (if
          (string-eq (as-str list-id) "")
          (todo/ensure-inbox)
          (as-str list-id))))
      (todo/tree/create
        "todo-item"
        (dict-set*
          "{}"
          (list
            "list_id"
            lid
            "title"
            (as-str title)
            "notes"
            ""
            "state"
            "TODO"
            "due"
            ""
            "position"
            "0"
            "done"
            "false"))))))

(define todo/add-form
  (lambda
    (view list-id)
    (if
      (string-eq view "from-notes")
      (html/p*
        (html/attr "class" "muted")
        "Mark nodes TODO in notes -- they appear here. Toggle writes back.")
      (if
        (string-eq view "archived")
```

```
        (html/p*
          (html/attr "class" "muted")
          "Archived tasks. Unarchive from detail (set state back), or delete.")
        (html/form
          "POST"
          "/todos"
          (html/attrs
            (list
              (html/attr "class" "card todo-add")
              (html/attr "hx-post" "/todos")
              (html/attr "hx-target" "#todo-main")
              (html/attr "hx-swap" "innerHTML")
              (html/attr
                "hx-on::after-request"
                "if(event.detail.successful){this.reset()}")))
          (html/join
            (list
              (html/hidden "action" "add")
              (html/hidden "view" view)
              (html/hidden "list_id" list-id)
              (html/label
                (html/attrs
                  (list
                    (html/attr "class" "todo-add-label")
                    (html/attr "for" "title")))
                "New item")
              (html/div
                (html/attr "class" "todo-add-row")
                (html/join
                  (list
                    (html/text-input
                      "title"
                      (html/attrs
                        (list
                          (html/attr "id" "title")
                          (html/attr "required" "required")
                          (html/attr "placeholder" "buy milk")
                          (html/attr "autocomplete" "off"))))
                    (html/button (html/attr "type" "submit") "Add")))))))))

(define todo/all-items
  (lambda () (todo/tree/rows "todo-item" 200)))

(define todo/archive
  (lambda (id) (todo/set-state id "ARCHIVED")))

(define todo/archived-items
  (lambda
    ()
    (list/filter
      (lambda
        (row)
        (todo/archived? (todo/item-state (cdr row))))
      (todo/all-items))))

(define todo/archived?
  (lambda (s) (string-eq (as-str s) "ARCHIVED")))

(define todo/count-all-in
  (lambda
    (items)
    (list/foldl (lambda (acc _) (+ acc 1)) 0 items)))

(define todo/count-open-in
  (lambda
    (items)
    (list/foldl
      (lambda
        (acc row)
        (if
          (todo/open? (todo/item-state (cdr row)))
```

```
          (+ acc 1)
          acc))
      0
      items)))

(define todo/create-list
  (lambda
    (title)
    (todo/tree/create
      "todo-list"
      (dict-set*
        "{}"
        (list
          "title"
          (as-str title)
          "description"
          ""
          "is_default"
          "false")))))

(define todo/cycle
  (lambda
    (id)
    "Advance item id to its next state; () when missing. Cycles within the
     item's OWN list's state set (custom columns when present, else the
     global
     default) -- BORGE-1786336755, so /todos exactly mirrors /kanban."
    (let
      ((row (todo/tree/get "todo-item" id)))
      (if
        (null? row)
        ()
        (let
          ((cur (todo/item-state row))
            (list-id (todo/item/list-id row)))
          (if
            (todo/archived? cur)
            cur
            (todo/set-state id (todo/state/next-for-list cur list-id))))))))

(define todo/db/create
  (lambda
    (type fields)
    (let
      ((r
        (tree/upsert
          (node-path type "new")
          "datum"
          (as-str type)
          fields)))
      (if
        (string-starts-with? r "data.")
        (todo/db/path-id r)
        r))))

(define todo/db/delete
  (lambda
    (type id)
    (let
      ((p (node-path type (as-str id))))
      (if
        (null? (tree/get p))
        ()
        (seq (tree/delete p) (as-str id))))))

(define todo/db/get
  (lambda
    (type id)
    (let
      ((n (tree/get (node-path type (as-str id)))))
      (if (null? n) () (dict-get n "body")))))

(define todo/db/insert-by-created
```

```
  (lambda
    (n sorted)
    (if
      (null? sorted)
      (list n)
      (if
        (string<?
          (as-str (dict-get n "created_at"))
          (as-str (dict-get (car sorted) "created_at")))
        (cons n sorted)
        (cons
          (car sorted)
          (todo/db/insert-by-created n (cdr sorted)))))))

(define todo/db/node-row
  (lambda
    (node)
    (cons
      (todo/db/path-id (dict-get node "path"))
      (dict-get node "body"))))

(define todo/db/path-id
  (lambda
    (path)
    (let
      ((h (substring path (- (string-length path) 32) 32)))
      (string-append
        (substring h 0 8)
        "-"
        (substring h 8 4)
        "-"
        (substring h 12 4)
        "-"
        (substring h 16 4)
        "-"
        (substring h 20 12)))))

(define todo/db/prefix
  (lambda
    (type)
    (let
      ((p (node-path type "")))
      (substring p 0 (- (string-length p) 1)))))

(define todo/db/rows
  (lambda
    (type limit)
    (map
      todo/db/node-row
      (tree/list-kind (todo/db/prefix type) "datum" limit))))

(define todo/db/sort-by-created
  (lambda
    (ns)
    (if
      (null? ns)
      (list)
      (todo/db/insert-by-created
        (car ns)
        (todo/db/sort-by-created (cdr ns))))))

(define todo/db/update
  (lambda
    (type id fields)
    (let
      ((p (node-path type (as-str id))))
      (if
        (null? (tree/get p))
        ()
        (let
          ((r (tree/upsert p "datum" (as-str type) fields)))
          (if (string-starts-with? r "data.") (as-str id) r))))))

```

```
(define todo/db/update-field
  (lambda
    (type id field value)
    (let
      ((fields (todo/db/get type (as-str id))))
      (if
        (null? fields)
        ()
        (todo/db/update type id (dict-set fields field value))))))

(define todo/delete
  (lambda (id) (todo/tree/del "todo-item" id)))

(define todo/delete-list
  (lambda
    (list-id)
    (let
      ((lid (as-str list-id)))
      (if
        (string-eq lid "")
        "missing"
        (let
          ((fields (todo/get-list lid)))
          (if
            (null? fields)
            "missing"
            (if
              (todo/list-default? fields)
              "default"
              (seq
                (todo/delete-list-items (todo/all-items) lid)
                (seq (todo/tree/del "todo-list" lid) "ok")))))))))

(define todo/delete-list-items
  (lambda
    (items lid)
    (if
      (null? items)
      ()
      (seq
        (if
          (string-eq
            (todo/item-list-id (cdr (car items)))
            (as-str lid))
          (todo/delete (car (car items)))
          ())
        (todo/delete-list-items (cdr items) lid)))))

(define todo/detail-html
  (lambda
    (id view)
    (let
      ((fields (todo/get-item id)))
      (if
        (null? fields)
        (html/div
          (html/attr "class" "card")
          (html/join
            (list
              (html/p* (html/attr "class" "err") "Item not found.")
              (html/a (string-append "/todos?view=" view) "? back"))))
        (let
          ((title (todo/item-title fields)))
          (let
            ((notes (todo/item-notes fields)))
            (let
              ((due (todo/item-due fields)))
              (let
                ((state (todo/item-state fields)))
                (let
                  ((lid (todo/item-list-id fields)))
                  (html/div
                    (html/attr "class" "card todo-detail")
```

```
                    (html/join
                      (list
                        (html/p*
                          (html/attr "class" "muted")
                          (html/a
                            (string-append "/todos?view=" view)
                            "? back to list"))
                        (html/h2 (html/text title))
                        (html/form
                          "POST"
                          "/todos"
                          (html/attrs
                            (list (html/attr "class" "todo-detail-form")))
                          (html/join
                            (list
                              (html/hidden "action" "save")
                              (html/hidden "id" id)
                              (html/hidden "view" view)
                              (html/label (html/attr "for" "title") "Title")
                              (html/text-input
                                "title"
                                (html/attrs
                                  (list
                                    (html/attr "id" "title")
                                    (html/attr "value" title)
                                    (html/attr "required" "required"))))
                              (html/label (html/attr "for" "state") "State")
                              (html/select
                                "state"
                                (html/attr "id" "state")
                                (todo/state-options-all state))
                              (html/label (html/attr "for" "due") "Due
    (YYYY-MM-DD)")
                              (html/text-input
                                "due"
                                (html/attrs
                                  (list
                                    (html/attr "id" "due")
                                    (html/attr "value" due)
                                    (html/attr "placeholder"
    "2026-07-15"))))
                              (html/label (html/attr "for" "list_id")
    "List")
                              (html/select
                                "list_id"
                                (html/attr "id" "list_id")
                                (todo/list-options lid))
                              (html/label (html/attr "for" "notes") "Notes")
                              (html/textarea
                                "notes"
                                (html/attrs
                                  (list
                                    (html/attr "id" "notes")
                                    (html/attr "rows" "8")
                                    (html/attr "class" "todo-notes")))
                                notes)
                              (html/div
                                ""
                                (html/join
                                  (list
                                    (html/button (html/attr "type"
    "submit") "Save")
                                    " "
                                    (html/a (string-append "/todos?view="
    view) "Cancel"))))))))))))))))))))

(define todo/done?
  (lambda
    (fields)
    (let
      ((d (dict-get fields "done")))
      (if (null? d) false (string-eq d "true")))))

```

```
(define todo/ensure-inbox
  (lambda
    ()
    (let
      ((found (todo/find-default-list)))
      (if
        (null? found)
        (todo/tree/create
          "todo-list"
          (dict-set*
            "{}"
            (list
              "title"
              "Inbox"
              "description"
              "Default list"
              "is_default"
              "true")))
        (car found)))))

(define todo/find-default-in
  (lambda
    (xs)
    (if
      (null? xs)
      ()
      (if
        (todo/list-default? (cdr (car xs)))
        (car xs)
        (let
          ((t (todo/list-title (cdr (car xs)))))
          (if
            (string-eq t "Inbox")
            (car xs)
            (todo/find-default-in (cdr xs))))))))

(define todo/find-default-list
  (lambda () (todo/find-default-in (todo/lists))))

(define todo/get-item
  (lambda (id) (todo/tree/get "todo-item" id)))

(define todo/get-list
  (lambda (id) (todo/tree/get "todo-list" id)))

(define todo/handle-post
  (lambda
    ()
    (let
      ((action (as-str (request-param "action"))))
      (if
        (string-eq action "cycle")
        (todo/cycle (request-param "id"))
        (if
          (string-eq action "delete")
          (todo/delete (request-param "id"))
          (if
            (string-eq action "archive")
            (todo/archive (request-param "id"))
            (if
              (string-eq action "add")
              (todo/add
                (request-param "list_id")
                (request-param "title"))
              (if
                (string-eq action "create-list")
                (todo/create-list (request-param "title"))
                (if
                  (string-eq action "delete-list")
                  (todo/delete-list (request-param "list_id"))
                  (if
                    (string-eq action "cycle-note")
                    (notes/cycle-node-todo
```

```
                      (request-param "slug")
                      (request-param "node-id"))
                    (if
                      (string-eq action "save")
                      (todo/save
                        (request-param "id")
                        (request-param "title")
                        (request-param "notes")
                        (request-param "due")
                        (request-param "state")
                        (request-param "list_id"))
                      (todo/add (todo/ensure-inbox) (request-param
    "title")))))))))))))

(define todo/item-due
  (lambda
    (fields)
    (let
      ((d (dict-get fields "due")))
      (if (null? d) "" (as-str d)))))

(define todo/item-li
  (lambda
    (row)
    (let
      ((id (car row)))
      (let
        ((fields (cdr row)))
        (let
          ((title (dict-get fields "title")))
          (let
            ((done (todo/done? fields)))
            (html/li
              (html/attrs
                (list
                  (html/attr
                    "class"
                    (if done "todo-item is-done" "todo-item"))))
              (html/join
                (list
                  (html/form
                    "POST"
                    "/todos"
                    (html/attrs
                      (list
                        (html/attr "class" "todo-check")
                        (html/attr "hx-post" "/todos")
                        (html/attr "hx-target" "#todo-list")
                        (html/attr "hx-swap" "innerHTML")))
                    (html/join
                      (list
                        (html/hidden "action" "toggle")
                        (html/hidden "id" id)
                        (html/button
                          (html/attrs
                            (list
                              (html/attr "type" "submit")
                              (html/attr "class" "todo-box")
                              (html/attr "title" "toggle done")
                              (html/attr "aria-label" "toggle")))
                          (if done "[x]" "[ ]")))))
                  (html/span
                    (html/attr "class" "todo-title")
                    (html/text title))
                  (html/form
                    "POST"
                    "/todos"
                    (html/attrs
                      (list
                        (html/attr "class" "todo-del")
                        (html/attr "hx-post" "/todos")
                        (html/attr "hx-target" "#todo-list")
                        (html/attr "hx-swap" "innerHTML")))
```

```
                    (html/join
                      (list
                        (html/hidden "action" "delete")
                        (html/hidden "id" id)
                        (html/button
                          (html/attrs
                            (list
                              (html/attr "type" "submit")
                              (html/attr "class" "secondary")
                              (html/attr "title" "delete")))
                          "del")))))))))))))

(define todo/item-list-id
  (lambda
    (fields)
    (let
      ((lid (dict-get fields "list_id")))
      (if (null? lid) "" (as-str lid)))))

(define todo/item-notes
  (lambda
    (fields)
    (let
      ((n (dict-get fields "notes")))
      (if (null? n) "" (as-str n)))))

(define todo/item-state
  (lambda
    (fields)
    (let
      ((s (dict-get fields "state")))
      (if
        (null? s)
        (let
          ((d (dict-get fields "done")))
          (if
            (null? d)
            "TODO"
            (if (string-eq d "true") "DONE" "TODO")))
        (as-str s)))))

(define todo/item-title
  (lambda (fields) (as-str (dict-get fields "title"))))

(define todo/item/active
  (lambda
    (items)
    (list/filter
      (lambda
        (row)
        (not (todo/state/archived? (todo/item/state (cdr row)))))
      items)))

(define todo/item/add
  (lambda
    (list-id title)
    (let
      ((lid
        (if
          (string-eq (as-str list-id) "")
          (todo/list/ensure-inbox)
          (as-str list-id))))
      (todo/tree/create
        "todo-item"
        (dict-set*
          "{}"
          (list
            "list_id"
            lid
            "title"
            (as-str title)
            "notes"
            ""
```

```
            "state"
            "TODO"
            "due"
            ""
            "position"
            "0"
            "done"
            "false"))))))

(define todo/item/all
  (lambda () (todo/tree/rows "todo-item" 200)))

(define todo/item/archive
  (lambda (id) (todo/item/set-state id "ARCHIVED")))

(define todo/item/archived
  (lambda
    ()
    (list/filter
      (lambda
        (row)
        (todo/state/archived? (todo/item/state (cdr row))))
      (todo/item/all))))

(define todo/item/count-all
  (lambda
    (items)
    (list/foldl (lambda (acc _) (+ acc 1)) 0 items)))

(define todo/item/count-open
  (lambda
    (items)
    (list/foldl
      (lambda
        (acc row)
        (if
          (todo/state/open? (todo/item/state (cdr row)))
          (+ acc 1)
          acc))
      0
      items)))

(define todo/item/cycle
  (lambda
    (id)
    (let
      ((row (todo/tree/get "todo-item" id)))
      (if
        (null? row)
        ()
        (let
          ((cur (todo/item/state row))
            (list-id (todo/item/list-id row)))
          (if
            (todo/state/archived? cur)
            cur
            (todo/item/set-state id (todo/state/next-for-list cur
    list-id))))))))

(define todo/item/delete
  (lambda (id) (todo/tree/del "todo-item" id)))

(define todo/item/due
  (lambda
    (fields)
    (let
      ((d (dict-get fields "due")))
      (if (null? d) "" (as-str d)))))

(define todo/item/for-list
  (lambda
    (list-id)
    (let
```

```
      ((lid (as-str list-id))
        (inbox-id (todo/list/ensure-inbox)))
      (list/filter
        (lambda
          (row)
          (let
            ((fl (todo/item/list-id (cdr row))))
            (if
              (string-eq fl lid)
              #t
              (if (string-eq fl "") (string-eq lid inbox-id) #f))))
        (todo/item/all)))))

(define todo/item/get
  (lambda (id) (todo/tree/get "todo-item" id)))

(define todo/item/html
  (lambda
    (view)
    (if
      (string-eq view "from-notes")
      (html/ul
        (html/attr "class" "card todo-list")
        (list/foldl
          (lambda
            (acc t)
            (string-append acc (todo/item/note-li t view)))
          ""
          (notes/outline/all-todos)))
      (if
        (string-eq view "archived")
        (html/ul
          (html/attr "class" "card todo-list")
          (list/foldl
            (lambda
              (acc row)
              (string-append acc (todo/item/list-li row view)))
            ""
            (todo/item/archived)))
        (if
          (string-eq view "everything")
          (html/join
            (list
              (html/h2 "Lists")
              (html/ul
                (html/attr "class" "card todo-list")
                (list/foldl
                  (lambda
                    (acc row)
                    (string-append acc (todo/item/list-li row view)))
                  ""
                  (todo/item/active (todo/item/all))))
              (html/h2 "From notes")
              (html/ul
                (html/attr "class" "card todo-list")
                (list/foldl
                  (lambda
                    (acc t)
                    (if
                      (todo/state/open? (car (cdr (cdr (cdr t)))))
                      (string-append acc (todo/item/note-li t view))
                      acc))
                  ""
                  (notes/outline/all-todos)))))
          (html/ul
            (html/attr "class" "card todo-list")
            (list/foldl
              (lambda
                (acc row)
                (string-append acc (todo/item/list-li row view)))
              ""
              (todo/item/active (todo/item/for-list view)))))))))
```

```
(define todo/item/list-id
  (lambda
    (fields)
    (let
      ((lid (dict-get fields "list_id")))
      (if (null? lid) "" (as-str lid)))))

(define todo/item/list-li
  (lambda
    (row view)
    (let
      ((id (car row)) (fields (cdr row)))
      (let
        ((title (todo/item/title fields))
          (state (todo/item/state fields))
          (open (todo/state/open? (todo/item/state fields))))
        (html/li
          (html/attr
            "class"
            (if
              open
              "todo-item"
              (if
                (todo/state/archived? state)
                "todo-item is-done is-archived"
                "todo-item is-done")))
          (html/join
            (list
              (html/form
                "POST"
                "/todos"
                (html/attrs
                  (list
                    (html/attr "class" "todo-check")
                    (html/attr "hx-post" "/todos")
                    (html/attr "hx-target" "#todo-main")
                    (html/attr "hx-swap" "innerHTML")))
                (html/join
                  (list
                    (html/hidden "action" "cycle")
                    (html/hidden "id" id)
                    (html/hidden "view" view)
                    (html/button
                      (html/attrs
                        (list
                          (html/attr "type" "submit")
                          (html/attr "class" (string-append "todo-box st-"
    state))
                          (html/attr "title" "cycle state")))
                      (if (string-eq state "") "?" state)))))
              (html/a
                (string-append "/todos?id=" id "&view=" view)
                (html/span
                  (html/attr "class" "todo-title")
                  (html/text title)))
              (if
                (string-eq state "DONE")
                (html/form
                  "POST"
                  "/todos"
                  (html/attrs
                    (list
                      (html/attr "class" "todo-archive")
                      (html/attr "hx-post" "/todos")
                      (html/attr "hx-target" "#todo-main")
                      (html/attr "hx-swap" "innerHTML")))
                  (html/join
                    (list
                      (html/hidden "action" "archive")
                      (html/hidden "id" id)
                      (html/hidden "view" view)
                      (html/button
                        (html/attrs
```

```
                          (list
                            (html/attr "type" "submit")
                            (html/attr "class" "secondary")
                            (html/attr "title" "archive done task")))
                        "arch"))))
                "")
              (html/form
                "POST"
                "/todos"
                (html/attrs
                  (list
                    (html/attr "class" "todo-del")
                    (html/attr "hx-post" "/todos")
                    (html/attr "hx-target" "#todo-main")
                    (html/attr "hx-swap" "innerHTML")))
                (html/join
                  (list
                    (html/hidden "action" "delete")
                    (html/hidden "id" id)
                    (html/hidden "view" view)
                    (html/button
                      (html/attrs
                        (list
                          (html/attr "type" "submit")
                          (html/attr "class" "secondary")
                          (html/attr "title" "delete")))
                      "del")))))))))))

(define todo/item/note-li
  (lambda
    (t view)
    (let
      ((slug (car t))
        (nid (car (cdr t)))
        (title (car (cdr (cdr t))))
        (state (car (cdr (cdr (cdr t))))))
      (let
        ((open (todo/state/open? state)))
        (html/li
          (html/attr
            "class"
            (if
              open
              "todo-item todo-from-note"
              "todo-item todo-from-note is-done"))
          (html/join
            (list
              (html/form
                "POST"
                "/todos"
                (html/attrs
                  (list
                    (html/attr "class" "todo-check")
                    (html/attr "hx-post" "/todos")
                    (html/attr "hx-target" "#todo-main")
                    (html/attr "hx-swap" "innerHTML")))
                (html/join
                  (list
                    (html/hidden "action" "cycle-note")
                    (html/hidden "slug" slug)
                    (html/hidden "node-id" nid)
                    (html/hidden "view" view)
                    (html/button
                      (html/attrs
                        (list
                          (html/attr "type" "submit")
                          (html/attr "class" (string-append "todo-box st-"
    state))
                          (html/attr "title" "cycle note TODO")))
                      (if (string-eq state "") "?" state)))))
              (html/span
                (html/attr "class" "todo-title")
                (html/text title))
```

```
              (html/a
                (string-append "/notes?slug=" slug)
                (html/span
                  (html/attr "class" "muted todo-src")
                  (html/text slug)))))))))))

(define todo/item/notes
  (lambda
    (fields)
    (let
      ((n (dict-get fields "notes")))
      (if (null? n) "" (as-str n)))))

(define todo/item/save
  (lambda
    (id title notes due state list-id)
    (let
      ((fields (todo/item/get id)))
      (if
        (null? fields)
        ()
        (let
          ((lid
            (if
              (string-eq (as-str list-id) "")
              (todo/list/ensure-inbox)
              (as-str list-id))))
          (todo/tree/save
            "todo-item"
            id
            (dict-set*
              fields
              (list
                "title"
                (as-str title)
                "notes"
                (as-str notes)
                "due"
                (as-str due)
                "state"
                (as-str state)
                "list_id"
                lid))))))))

(define todo/item/set-state
  (lambda
    (id state)
    (todo/tree/set-field
      "todo-item"
      id
      "state"
      (as-str state))))

(define todo/item/state
  (lambda
    (fields)
    (let
      ((s (dict-get fields "state")))
      (if
        (null? s)
        (let
          ((d (dict-get fields "done")))
          (if
            (null? d)
            "TODO"
            (if (string-eq d "true") "DONE" "TODO")))
        (as-str s)))))

(define todo/item/title
  (lambda (fields) (as-str (dict-get fields "title"))))

(define todo/items-for-list
  (lambda
```

```
    (list-id)
    (let
      ((lid (as-str list-id)) (inbox-id (todo/ensure-inbox)))
      (list/filter
        (lambda
          (row)
          (let
            ((fl (todo/item-list-id (cdr row))))
            (if
              (string-eq fl lid)
              #t
              (if (string-eq fl "") (string-eq lid inbox-id) #f))))
        (todo/all-items)))))

(define todo/items-html
  (lambda
    (view)
    (if
      (string-eq view "from-notes")
      (html/ul
        (html/attr "class" "card todo-list")
        (list/foldl
          (lambda
            (acc t)
            (string-append acc (todo/note-item-li t view)))
          ""
          (notes/all-todos)))
      (if
        (string-eq view "archived")
        (html/ul
          (html/attr "class" "card todo-list")
          (list/foldl
            (lambda
              (acc row)
              (string-append acc (todo/list-item-li row view)))
            ""
            (todo/archived-items)))
        (if
          (string-eq view "everything")
          (html/join
            (list
              (html/h2 "Lists")
              (html/ul
                (html/attr "class" "card todo-list")
                (list/foldl
                  (lambda
                    (acc row)
                    (string-append acc (todo/list-item-li row view)))
                  ""
                  (todo/active-items (todo/all-items))))
              (html/h2 "From notes")
              (html/ul
                (html/attr "class" "card todo-list")
                (list/foldl
                  (lambda
                    (acc t)
                    (if
                      (todo/open? (car (cdr (cdr (cdr t)))))
                      (string-append acc (todo/note-item-li t view))
                      acc))
                  ""
                  (notes/all-todos)))))
          (html/ul
            (html/attr "class" "card todo-list")
            (list/foldl
              (lambda
                (acc row)
                (string-append acc (todo/list-item-li row view)))
              ""
              (todo/active-items (todo/items-for-list view)))))))))

(define todo/land-ping
  1)
```

```
1082
1083 (define todo/list
1084   (lambda () (todo/tree/rows "todo-item" 50)))
1085
1086 (define todo/list-default?
1087   (lambda
1088     (fields)
1089     (let
1090       ((d (dict-get fields "is_default")))
1091       (if (null? d) #f (string-eq d "true")))))
1092
1093 (define todo/list-html
1094   (lambda
1095     ()
1096     (html/ul
1097       (html/attr "class" "card todo-list")
1098       (list/foldl
1099         (lambda
1100           (acc row)
1101           (string-append acc (todo/item-li row)))
1102         ""
1103         (todo/list)))))
1104
1105 (define todo/list-item-li
1106   (lambda
1107     (row view)
1108     (let
1109       ((id (car row)) (fields (cdr row)))
1110       (let
1111         ((title (todo/item-title fields))
1112           (state (todo/item-state fields))
1113           (open (todo/open? (todo/item-state fields))))
1114         (html/li
1115           (html/attr
1116             "class"
1117             (if
1118               open
1119               "todo-item"
1120               (if
1121                 (todo/archived? state)
1122                 "todo-item is-done is-archived"
1123                 "todo-item is-done")))
1124           (html/join
1125             (list
1126               (html/form
1127                 "POST"
1128                 "/todos"
1129                 (html/attrs
1130                   (list
1131                     (html/attr "class" "todo-check")
1132                     (html/attr "hx-post" "/todos")
1133                     (html/attr "hx-target" "#todo-main")
1134                     (html/attr "hx-swap" "innerHTML")))
1135                 (html/join
1136                   (list
1137                     (html/hidden "action" "cycle")
1138                     (html/hidden "id" id)
1139                     (html/hidden "view" view)
1140                     (html/button
1141                       (html/attrs
1142                         (list
1143                           (html/attr "type" "submit")
1144                           (html/attr "class" (string-append "todo-box st-"
     state))
1145                           (html/attr "title" "cycle state")))
1146                       (if (string-eq state "") "?" state)))))
1147               (html/a
1148                 (string-append "/todos?id=" id "&view=" view)
1149                 (html/span
1150                   (html/attr "class" "todo-title")
1151                   (html/text title)))
1152               (if
1153                 (string-eq state "DONE")
```

```
                (html/form
                  "POST"
                  "/todos"
                  (html/attrs
                    (list
                      (html/attr "class" "todo-archive")
                      (html/attr "hx-post" "/todos")
                      (html/attr "hx-target" "#todo-main")
                      (html/attr "hx-swap" "innerHTML")))
                  (html/join
                    (list
                      (html/hidden "action" "archive")
                      (html/hidden "id" id)
                      (html/hidden "view" view)
                      (html/button
                        (html/attrs
                          (list
                            (html/attr "type" "submit")
                            (html/attr "class" "secondary")
                            (html/attr "title" "archive done task")))
                        "arch"))))
                  "")
                (html/form
                  "POST"
                  "/todos"
                  (html/attrs
                    (list
                      (html/attr "class" "todo-del")
                      (html/attr "hx-post" "/todos")
                      (html/attr "hx-target" "#todo-main")
                      (html/attr "hx-swap" "innerHTML")))
                  (html/join
                    (list
                      (html/hidden "action" "delete")
                      (html/hidden "id" id)
                      (html/hidden "view" view)
                      (html/button
                        (html/attrs
                          (list
                            (html/attr "type" "submit")
                            (html/attr "class" "secondary")
                            (html/attr "title" "delete")))
                        "del")))))))))

(define todo/list-options
  (lambda
    (selected)
    (list/foldl
      (lambda
        (acc row)
        (let
          ((id (car row)))
          (let
            ((title (todo/list-title (cdr row))))
            (string-append
              acc
              (html/option
                id
                title
                (string-eq (as-str id) (as-str selected)))))))
      ""
      (todo/lists))))

(define todo/list-title
  (lambda (fields) (as-str (dict-get fields "title"))))

(define todo/list/all
  (lambda () (todo/tree/rows "todo-list" 50)))

(define todo/list/create
  (lambda
    (title)
    (todo/tree/create
```

```
      "todo-list"
      (dict-set*
        "{}"
        (list
          "title"
          (as-str title)
          "description"
          ""
          "is_default"
          "false")))))

(define todo/list/default?
  (lambda
    (fields)
    (let
      ((d (dict-get fields "is_default")))
      (if (null? d) #f (string-eq d "true")))))

(define todo/list/delete
  (lambda
    (list-id)
    (let
      ((lid (as-str list-id)))
      (if
        (string-eq lid "")
        "missing"
        (let
          ((fields (todo/list/get lid)))
          (if
            (null? fields)
            "missing"
            (if
              (todo/list/default? fields)
              "default"
              (seq
                (todo/list/delete-items (todo/item/all) lid)
                (seq (todo/tree/del "todo-list" lid) "ok")))))))))

(define todo/list/delete-items
  (lambda
    (items lid)
    (if
      (null? items)
      ()
      (seq
        (if
          (string-eq
            (todo/item/list-id (cdr (car items)))
            (as-str lid))
          (todo/item/delete (car (car items)))
          ())
        (todo/list/delete-items (cdr items) lid)))))

(define todo/list/ensure-inbox
  (lambda
    ()
    (let
      ((found (todo/list/find-default)))
      (if
        (null? found)
        (todo/tree/create
          "todo-list"
          (dict-set*
            "{}"
            (list
              "title"
              "Inbox"
              "description"
              "Default list"
              "is_default"
              "true")))
        (car found)))))

```

```
(define todo/list/find-default
  (lambda () (todo/list/find-default-in (todo/list/all))))

(define todo/list/find-default-in
  (lambda
    (xs)
    (if
      (null? xs)
      ()
      (if
        (todo/list/default? (cdr (car xs)))
        (car xs)
        (let
          ((t (todo/list/title (cdr (car xs)))))
          (if
            (string-eq t "Inbox")
            (car xs)
            (todo/list/find-default-in (cdr xs))))))))

(define todo/list/get
  (lambda (id) (todo/tree/get "todo-list" id)))

(define todo/list/nav-li
  (lambda
    (view row)
    (let
      ((id (car row))
        (f (cdr row))
        (items
          (todo/item/active (todo/item/for-list (car row)))))
      (html/li
        (html/attr
          "class"
          (if
            (string-eq view id)
            "todo-nav-item is-active todo-nav-list"
            "todo-nav-item todo-nav-list"))
        (html/join
          (list
            (html/a
              (string-append "/todos?view=" id)
              (string-append
                (todo/list/title f)
                " ("
                (todo/ui/ratio items)
                ")"))
            (if
              (todo/list/default? f)
              ""
              (html/form
                "POST"
                "/todos"
                (html/attrs
                  (list
                    (html/attr "class" "todo-list-del")
                    (html/attr "hx-post" "/todos")
                    (html/attr "hx-target" "#todo-main")
                    (html/attr "hx-swap" "innerHTML")
                    (html/attr
                      "hx-confirm"
                      "Delete this entire list and its items?")))
                (html/join
                  (list
                    (html/hidden "action" "delete-list")
                    (html/hidden "list_id" id)
                    (html/hidden "view" "everything")
                    (html/button
                      (html/attrs
                        (list
                          (html/attr "type" "submit")
                          (html/attr "class" "secondary")
                          (html/attr "title" "delete list")))
                      "?")))))))))))
```

```
1373
1374 (define todo/list/options
1375   (lambda
1376     (selected)
1377     (list/foldl
1378       (lambda
1379         (acc row)
1380         (let
1381           ((id (car row)))
1382           (let
1383             ((title (todo/list/title (cdr row))))
1384             (string-append
1385               acc
1386               (html/option
1387                 id
1388                 title
1389                 (string-eq (as-str id) (as-str selected)))))))
1390       ""
1391       (todo/list/all))))
1392
1393 (define todo/list/title
1394   (lambda (fields) (as-str (dict-get fields "title"))))
1395
1396 (define todo/lists
1397   (lambda () (todo/tree/rows "todo-list" 50)))
1398
1399 (define todo/main-html
1400   (lambda
1401     (view)
1402     (let
1403       ((id-param (request-param "id"))
1404         (is-get (string-eq (request-method) "GET")))
1405       (if
1406         (if
1407           is-get
1408           (if
1409             (null? id-param)
1410             #f
1411             (not (string-eq (as-str id-param) "")))
1412           #f)
1413         (html/div
1414           (html/attr "id" "todo-main")
1415           (html/join
1416             (list
1417               (html/div
1418                 (html/attr "class" "todo-layout")
1419                 (html/join
1420                   (list
1421                     (html/div
1422                       (html/attr "class" "todo-side")
1423                       (todo/nav-html view))
1424                     (html/div
1425                       (html/attr "class" "todo-body")
1426                       (todo/detail-html (as-str id-param) view))))))))
1427         (let
1428           ((list-id
1429             (if
1430               (string-eq view "everything")
1431               (todo/ensure-inbox)
1432               (if
1433                 (string-eq view "from-notes")
1434                 ""
1435                 (if (string-eq view "archived") "" view)))))
1436           (html/div
1437             (html/attr "id" "todo-main")
1438             (html/join
1439               (list
1440                 (html/div
1441                   (html/attr "class" "todo-layout")
1442                   (html/join
1443                     (list
1444                       (html/div
1445                         (html/attr "class" "todo-side")
```

```
                        (todo/nav-html view))
                      (html/div
                        (html/attr "class" "todo-body")
                        (html/join
                          (list
                            (todo/add-form view list-id)
                            (todo/items-html view))))))))))))))

(define todo/nav-html
  (lambda
    (view)
    (let
      ((lists (todo/lists))
        (inbox (todo/ensure-inbox))
        (arch-n (todo/count-all-in (todo/archived-items))))
      (html/ul
        (html/attr "class" "todo-nav")
        (html/join
          (list
            (todo/nav-link view "everything" "Everything" "")
            (todo/nav-link view "from-notes" "From notes" "")
            (todo/nav-link
              view
              "archived"
              "Archived"
              (string-append "(" (as-str arch-n) ")"))
            (list/foldl
              (lambda
                (acc row)
                (string-append acc (todo/nav-list-li view row)))
              ""
              lists)
            (html/li
              (html/attr "class" "todo-nav-item")
              (html/form
                "POST"
                "/todos"
                (html/attrs
                  (list
                    (html/attr "class" "todo-new-list")
                    (html/attr "hx-post" "/todos")
                    (html/attr "hx-target" "#todo-main")
                    (html/attr "hx-swap" "innerHTML")))
                (html/join
                  (list
                    (html/hidden "action" "create-list")
                    (html/hidden "view" view)
                    (html/text-input
                      "title"
                      (html/attrs
                        (list
                          (html/attr "placeholder" "New list")
                          (html/attr "required" "required")
                          (html/attr "class" "todo-new-list-input"))))
                    (html/button (html/attr "type" "submit") "+
    list"))))))))))

(define todo/nav-link
  (lambda
    (view key label extra)
    (html/li
      (html/attr
        "class"
        (if
          (string-eq view key)
          "todo-nav-item is-active"
          "todo-nav-item"))
      (html/a
        (string-append "/todos?view=" key)
        (string-append
          label
          (if (string-eq extra "") "" (string-append " " extra)))))))

```

```
(define todo/nav-list-li
  (lambda
    (view row)
    (let
      ((id (car row))
        (f (cdr row))
        (items
          (todo/active-items (todo/items-for-list (car row)))))
      (html/li
        (html/attr
          "class"
          (if
            (string-eq view id)
            "todo-nav-item is-active todo-nav-list"
            "todo-nav-item todo-nav-list"))
        (html/join
          (list
            (html/a
              (string-append "/todos?view=" id)
              (string-append
                (todo/list-title f)
                " ("
                (todo/ratio items)
                ")"))
            (if
              (todo/list-default? f)
              ""
              (html/form
                "POST"
                "/todos"
                (html/attrs
                  (list
                    (html/attr "class" "todo-list-del")
                    (html/attr "hx-post" "/todos")
                    (html/attr "hx-target" "#todo-main")
                    (html/attr "hx-swap" "innerHTML")
                    (html/attr
                      "hx-confirm"
                      "Delete this entire list and its items?")))
                (html/join
                  (list
                    (html/hidden "action" "delete-list")
                    (html/hidden "list_id" id)
                    (html/hidden "view" "everything")
                    (html/button
                      (html/attrs
                        (list
                          (html/attr "type" "submit")
                          (html/attr "class" "secondary")
                          (html/attr "title" "delete list")))
                      "?")))))))))))

(define todo/next-state
  (lambda
    (cur)
    (outline/todo-next-in todo/states (as-str cur))))

(define todo/note-item-li
  (lambda
    (t view)
    (let
      ((slug (car t))
        (nid (car (cdr t)))
        (title (car (cdr (cdr t))))
        (state (car (cdr (cdr (cdr t))))))
      (let
        ((open (todo/open? state)))
        (html/li
          (html/attr
            "class"
            (if
              open
              "todo-item todo-from-note"
```

```
              "todo-item todo-from-note is-done"))
          (html/join
            (list
              (html/form
                "POST"
                "/todos"
                (html/attrs
                  (list
                    (html/attr "class" "todo-check")
                    (html/attr "hx-post" "/todos")
                    (html/attr "hx-target" "#todo-main")
                    (html/attr "hx-swap" "innerHTML")))
                (html/join
                  (list
                    (html/hidden "action" "cycle-note")
                    (html/hidden "slug" slug)
                    (html/hidden "node-id" nid)
                    (html/hidden "view" view)
                    (html/button
                      (html/attrs
                        (list
                          (html/attr "type" "submit")
                          (html/attr "class" (string-append "todo-box st-"
    state))
                          (html/attr "title" "cycle note TODO")))
                      (if (string-eq state "") "?" state)))))
              (html/span
                (html/attr "class" "todo-title")
                (html/text title))
              (html/a
                (string-append "/notes?slug=" slug)
                (html/span
                  (html/attr "class" "muted todo-src")
                  (html/text slug)))))))))

(define todo/open?
  (lambda
    (s)
    (let
      ((t (as-str s)))
      (if
        (string-eq t "")
        #f
        (if
          (string-eq t "DONE")
          #f
          (if
            (string-eq t "CANCELLED")
            #f
            (if (string-eq t "ARCHIVED") #f #t)))))))

(define todo/page
  (lambda
    ()
    (seq
      (todo/list/ensure-inbox)
      (if
        (string-eq (request-method) "POST")
        (seq
          (todo/ui/handle-post)
          (let
            ((view
              (let
                ((v (request-param "view")))
                (if (null? v) "everything" (as-str v)))))
            (if
              (hx-request?)
              (render-html (todo/ui/main view))
              (redirect (string-append "/todos?view=" view)))))
        (html/page
          (list
            (html/h1 "To-dos")
            (html/p*
```

```
              (html/attr "class" "muted")
              "Lists (Basecamp) + notes scrape (org). Click a title for
    details.")
            (todo/ui/main (todo/ui/view))))))))

(define todo/partial
  (lambda
    ()
    (seq
      (todo/list/ensure-inbox)
      (render-html (todo/ui/main (todo/ui/view))))))

(define todo/ratio
  (lambda
    (items)
    (let
      ((n (todo/count-all-in items))
        (open (todo/count-open-in items)))
      (string-append (as-str (- n open)) "/" (as-str n)))))

(define todo/save
  (lambda
    (id title notes due state list-id)
    (let
      ((fields (todo/get-item id)))
      (if
        (null? fields)
        ()
        (let
          ((lid
            (if
              (string-eq (as-str list-id) "")
              (todo/ensure-inbox)
              (as-str list-id))))
          (todo/tree/save
            "todo-item"
            id
            (dict-set*
              fields
              (list
                "title"
                (as-str title)
                "notes"
                (as-str notes)
                "due"
                (as-str due)
                "state"
                (as-str state)
                "list_id"
                lid)))))))))

(define todo/seqtest
  (lambda () 7))

(define todo/set-state
  (lambda
    (id state)
    (todo/tree/set-field
      "todo-item"
      id
      "state"
      (as-str state))))

(define todo/state-options
  (lambda
    (selected)
    (list/foldl
      (lambda
        (acc st)
        (let
          ((lab (if (string-eq st "") "(none)" st)))
          (string-append
            acc
```

```
            (html/option st lab (string-eq st (as-str selected))))))
      ""
      todo/states)))

(define todo/state-options-all
  (lambda
    (selected)
    (todo/state-options-from
      selected
      (append todo/states (list "ARCHIVED")))))

(define todo/state-options-from
  (lambda
    (selected states)
    (list/foldl
      (lambda
        (acc st)
        (let
          ((lab (if (string-eq st "") "(none)" st)))
          (string-append
            acc
            (html/option st lab (string-eq st (as-str selected))))))
      ""
      states)))

(define todo/state/archived?
  (lambda (s) (string-eq (as-str s) "ARCHIVED")))

(define todo/state/list
  (list "" "TODO" "DONE" "WAITING" "CANCELLED"))

(define todo/state/next
  (lambda
    (cur)
    (outline/todo-next-in todo/state/list (as-str cur))))

(define todo/state/columns-for-list
  (lambda
    (list-id)
    "The state set (list of state strings) for a todo-list: the list's own
     `columns` field when present and non-empty, else the global default
     todo/state/list. THE per-list override kanban columns and the /todos
     state machine both read through, so a list with custom columns cycles
     only those states everywhere."
    (let
      ((lf (get (quote todo-list) (as-str list-id))))
      (if
        (null? lf)
        todo/state/list
        (let
          ((cols (dict-get lf "columns")))
          (if
            (null? cols)
            todo/state/list
            (let
              ((cs (as-str cols)))
              (if
                (string-eq cs "")
                todo/state/list
                (json-parse cs)))))))))

(define todo/state/todo-next-in
  (lambda
    (xs cur)
    "State following cur within the state list xs; xs's own head on wrap
     (last
     -> first) and on a not-found cur, so the cycle never leaves the list's
     columns. (A per-set wrap, distinct from outline/todo-next-in which
     hardcodes the global default state list's head.)"
    (if
      (null? xs)
      "TODO"
      (todo/state/todo-next-in-at xs cur (as-str (car xs))))))
```

```
1807
1808 (define todo/state/todo-next-in-at
1809   (lambda
1810     (xs cur head)
1811     "Accumulating: find cur in xs, return the next; wrap to head at the end
      or
1812      on absent."
1813     (if
1814       (null? xs)
1815       head
1816       (if
1817         (string-eq (as-str (car xs)) cur)
1818         (if
1819           (null? (cdr xs))
1820           head
1821           (as-str (car (cdr xs))))
1822         (todo/state/todo-next-in-at (cdr xs) cur head)))))
1823
1824 (define todo/state/next-for-list
1825   (lambda
1826     (cur list-id)
1827     "Next state after cur within list-id's own state set (custom columns
      when
1828      present, else the global default). Wraps around: last column -> first
      of
1829      that same set."
1830     (todo/state/todo-next-in (todo/state/columns-for-list list-id) (as-str
      cur))))
1831
1832 (define todo/state/open?
1833   (lambda
1834     (s)
1835     (let
1836       ((t (as-str s)))
1837       (if
1838         (string-eq t "")
1839         #f
1840         (if
1841           (string-eq t "DONE")
1842           #f
1843           (if
1844             (string-eq t "CANCELLED")
1845             #f
1846             (if (string-eq t "ARCHIVED") #f #t)))))))
1847
1848 (define todo/state/options
1849   (lambda
1850     (selected)
1851     (list/foldl
1852       (lambda
1853         (acc st)
1854         (let
1855           ((lab (if (string-eq st "") "(none)" st)))
1856           (string-append
1857             acc
1858             (html/option st lab (string-eq st (as-str selected))))))
1859       ""
1860       todo/state/list)))
1861
1862 (define todo/state/options-all
1863   (lambda
1864     (selected)
1865     (todo/state/options-from
1866       selected
1867       (append todo/state/list (list "ARCHIVED")))))
1868
1869 (define todo/state/options-from
1870   (lambda
1871     (selected states)
1872     (list/foldl
1873       (lambda
1874         (acc st)
1875         (let
```

```
          ((lab (if (string-eq st "") "(none)" st)))
          (string-append
            acc
            (html/option st lab (string-eq st (as-str selected))))))
      ""
      states)))

(define todo/state/options-for-list
  (lambda
    (selected list-id)
    "HTML options over list-id's own state set PLUS ARCHIVED, with selected
     marked -- the detail <select> for an item in that list. Uses the list's
     custom columns when present, else the canonical options-all state set."
    (todo/state/options-from
      selected
      (append (todo/state/columns-for-list list-id) (list "ARCHIVED")))))

(define todo/states
  (list "" "TODO" "DONE" "WAITING" "CANCELLED"))

(define todo/toggle
  (lambda
    (id)
    (let
      ((row (todo/tree/get "todo-item" id)))
      (if
        (null? row)
        ()
        (todo/tree/set-field
          "todo-item"
          id
          "done"
          (if (todo/done? row) "false" "true"))))))

(define todo/tree/create
  (lambda
    (t fields)
    (let
      ((r (tree/upsert (node-path t "new") "datum" t fields)))
      (if
        (todo/tree/path-ok? r)
        (todo/tree/redash (todo/tree/tail r))
        r))))

(define todo/tree/del
  (lambda
    (t id)
    (seq (tree/delete (node-path t (as-str id))) id)))

(define todo/tree/get
  (lambda
    (t id)
    (let
      ((n (tree/get (node-path t (as-str id)))))
      (if (null? n) (list) (dict-get n "body")))))

(define todo/tree/insert-by-created
  (lambda
    (nj sorted)
    (if
      (null? sorted)
      (list nj)
      (if
        (string<?
          (as-str (dict-get nj "created_at"))
          (as-str (dict-get (car sorted) "created_at")))
        (cons nj sorted)
        (cons
          (car sorted)
          (todo/tree/insert-by-created nj (cdr sorted)))))))

(define todo/tree/node->pair
  (lambda
```

```
    (nj)
    (cons
      (todo/tree/redash (todo/tree/tail (dict-get nj "path")))
      (dict-get nj "body"))))

(define todo/tree/path-ok?
  (lambda
    (r)
    (if
      (null? r)
      #f
      (string-eq (substring (as-str r) 0 5) "data."))))

(define todo/tree/prefix
  (lambda
    (t)
    (let
      ((p (node-path t "")))
      (substring p 0 (- (string-length p) 1)))))

(define todo/tree/redash
  (lambda
    (h)
    (string-append
      (substring h 0 8)
      "-"
      (substring h 8 4)
      "-"
      (substring h 12 4)
      "-"
      (substring h 16 4)
      "-"
      (substring h 20 12))))

(define todo/tree/rows
  (lambda
    (t n)
    (list/map
      todo/tree/node->pair
      (tree/list (todo/tree/prefix t) 1 n))))

(define todo/tree/save
  (lambda
    (t id fields)
    (seq
      (tree/upsert (node-path t (as-str id)) "datum" t fields)
      id)))

(define todo/tree/set-field
  (lambda
    (t id f v)
    (let
      ((cur (todo/tree/get t id)))
      (if
        (null? cur)
        (list)
        (todo/tree/save
          t
          id
          (dict-set* cur (list f (as-str v))))))))

(define todo/tree/sort-by-created
  (lambda
    (nodes)
    (list/foldl
      (lambda (acc n) (todo/tree/insert-by-created n acc))
      (list)
      nodes)))

(define todo/tree/tail
  (lambda (p) (substring p (- (string-length p) 32) 32)))

(define todo/ui/add-form
```

```
  (lambda
    (view list-id)
    (if
      (string-eq view "from-notes")
      (html/p*
        (html/attr "class" "muted")
        "Mark nodes TODO in notes -- they appear here. Toggle writes back.")
      (if
        (string-eq view "archived")
        (html/p*
          (html/attr "class" "muted")
          "Archived tasks. Unarchive from detail (set state back), or delete.")
        (html/form
          "POST"
          "/todos"
          (html/attrs
            (list
              (html/attr "class" "card todo-add")
              (html/attr "hx-post" "/todos")
              (html/attr "hx-target" "#todo-main")
              (html/attr "hx-swap" "innerHTML")
              (html/attr
                "hx-on::after-request"
                "if(event.detail.successful){this.reset()}")))
          (html/join
            (list
              (html/hidden "action" "add")
              (html/hidden "view" view)
              (html/hidden "list_id" list-id)
              (html/label
                (html/attrs
                  (list
                    (html/attr "class" "todo-add-label")
                    (html/attr "for" "title")))
                "New item")
              (html/div
                (html/attr "class" "todo-add-row")
                (html/join
                  (list
                    (html/text-input
                      "title"
                      (html/attrs
                        (list
                          (html/attr "id" "title")
                          (html/attr "required" "required")
                          (html/attr "placeholder" "buy milk")
                          (html/attr "autocomplete" "off"))))
                    (html/button (html/attr "type" "submit")
                      "Add")))))))))

(define todo/ui/detail
  (lambda
    (id view)
    (let
      ((fields (todo/item/get id)))
      (if
        (null? fields)
        (html/div
          (html/attr "class" "card")
          (html/join
            (list
              (html/p* (html/attr "class" "err") "Item not found.")
              (html/a (string-append "/todos?view=" view) "? back"))))
        (let
          ((title (todo/item/title fields)))
          (let
            ((notes (todo/item/notes fields)))
            (let
              ((due (todo/item/due fields)))
              (let
                ((state (todo/item/state fields)))
                (let
```

```
                    ((lid (todo/item/list-id fields)))
                    (html/div
                      (html/attr "class" "card todo-detail")
                      (html/join
                        (list
                          (html/p*
                            (html/attr "class" "muted")
                            (html/a
                              (string-append "/todos?view=" view)
                              "? back to list"))
                          (html/h2 (html/text title))
                          (html/form
                            "POST"
                            "/todos"
                            (html/attrs
                              (list (html/attr "class" "todo-detail-form")))
                            (html/join
                              (list
                                (html/hidden "action" "save")
                                (html/hidden "id" id)
                                (html/hidden "view" view)
                                (html/label (html/attr "for" "title") "Title")
                                (html/text-input
                                  "title"
                                  (html/attrs
                                    (list
                                      (html/attr "id" "title")
                                      (html/attr "value" title)
                                      (html/attr "required" "required"))))
                                (html/label (html/attr "for" "state") "State")
                                (html/select
                                  "state"
                                  (html/attr "id" "state")
                                  (todo/state/options-for-list state lid))
                                (html/label (html/attr "for" "due") "Due (YYYY-MM-DD)")
                                (html/text-input
                                  "due"
                                  (html/attrs
                                    (list
                                      (html/attr "id" "due")
                                      (html/attr "value" due)
                                      (html/attr "placeholder" "2026-07-15"))))
                                (html/label (html/attr "for" "list_id") "List")
                                (html/select
                                  "list_id"
                                  (html/attr "id" "list_id")
                                  (todo/list/options lid))
                                (html/label (html/attr "for" "notes") "Notes")
                                (html/textarea
                                  "notes"
                                  (html/attrs
                                    (list
                                      (html/attr "id" "notes")
                                      (html/attr "rows" "8")
                                      (html/attr "class" "todo-notes")))
                                  notes)
                                (html/div
                                  ""
                                  (html/join
                                    (list
                                      (html/button (html/attr "type" "submit") "Save")
                                      " "
                                      (html/a (string-append "/todos?view=" view) "Cancel"))))))))))))

(define todo/ui/handle-post
  (lambda
    ()
    (let
```

```
      ((action (as-str (request-param "action"))))
      (cond
        ((string-eq action "cycle")
          (todo/item/cycle (request-param "id")))
        ((string-eq action "delete")
          (todo/item/delete (request-param "id")))
        ((string-eq action "archive")
          (todo/item/archive (request-param "id")))
        ((string-eq action "add")
          (todo/item/add
            (request-param "list_id")
            (request-param "title")))
        ((string-eq action "create-list")
          (todo/list/create (request-param "title")))
        ((string-eq action "delete-list")
          (todo/list/delete (request-param "list_id")))
        ((string-eq action "cycle-note")
          (notes/outline/cycle-todo
            (request-param "slug")
            (request-param "node-id")))
        ((string-eq action "save")
          (todo/item/save
            (request-param "id")
            (request-param "title")
            (request-param "notes")
            (request-param "due")
            (request-param "state")
            (request-param "list_id")))
        (else
          (todo/item/add
            (todo/list/ensure-inbox)
            (request-param "title")))))))

(define todo/ui/main
  (lambda
    (view)
    (let
      ((id-param (request-param "id"))
        (is-get (string-eq (request-method) "GET")))
      (if
        (if
          is-get
          (if
            (null? id-param)
            #f
            (not (string-eq (as-str id-param) "")))
          #f)
        (html/div
          (html/attr "id" "todo-main")
          (html/join
            (list
              (html/div
                (html/attr "class" "todo-layout")
                (html/join
                  (list
                    (html/div
                      (html/attr "class" "todo-side")
                      (todo/ui/nav view))
                    (html/div
                      (html/attr "class" "todo-body")
                      (todo/ui/detail (as-str id-param) view))))))))
        (let
          ((list-id
            (if
              (string-eq view "everything")
              (todo/list/ensure-inbox)
              (if
                (string-eq view "from-notes")
                ""
                (if (string-eq view "archived") "" view)))))
          (html/div
            (html/attr "id" "todo-main")
            (html/join
```

```
              (list
                (html/div
                  (html/attr "class" "todo-layout")
                  (html/join
                    (list
                      (html/div
                        (html/attr "class" "todo-side")
                        (todo/ui/nav view))
                      (html/div
                        (html/attr "class" "todo-body")
                        (html/join
                          (list
                            (todo/ui/add-form view list-id)
                            (todo/item/html view))))))))))))

(define todo/ui/nav
  (lambda
    (view)
    (let
      ((lists (todo/list/all))
        (inbox (todo/list/ensure-inbox))
        (arch-n (todo/item/count-all (todo/item/archived))))
      (html/ul
        (html/attr "class" "todo-nav")
        (html/join
          (list
            (todo/ui/nav-link view "everything" "Everything" "")
            (todo/ui/nav-link view "from-notes" "From notes" "")
            (todo/ui/nav-link
              view
              "archived"
              "Archived"
              (string-append "(" (as-str arch-n) ")"))
            (list/foldl
              (lambda
                (acc row)
                (string-append acc (todo/list/nav-li view row)))
              ""
              lists)
            (html/li
              (html/attr "class" "todo-nav-item")
              (html/form
                "POST"
                "/todos"
                (html/attrs
                  (list
                    (html/attr "class" "todo-new-list")
                    (html/attr "hx-post" "/todos")
                    (html/attr "hx-target" "#todo-main")
                    (html/attr "hx-swap" "innerHTML")))
                (html/join
                  (list
                    (html/hidden "action" "create-list")
                    (html/hidden "view" view)
                    (html/text-input
                      "title"
                      (html/attrs
                        (list
                          (html/attr "placeholder" "New list")
                          (html/attr "required" "required")
                          (html/attr "class" "todo-new-list-input"))))
                    (html/button (html/attr "type" "submit") "+
    list"))))))))))

(define todo/ui/nav-link
  (lambda
    (view key label extra)
    (html/li
      (html/attr
        "class"
        (if
          (string-eq view key)
          "todo-nav-item is-active"
```

```
          "todo-nav-item"))
      (html/a
        (string-append "/todos?view=" key)
        (string-append
          label
          (if (string-eq extra "") "" (string-append " " extra)))))))

(define todo/ui/ratio
  (lambda
    (items)
    (let
      ((n (todo/item/count-all items))
        (open (todo/item/count-open items)))
      (string-append (as-str (- n open)) "/" (as-str n)))))

(define todo/ui/view
  (lambda
    ()
    (let
      ((v (request-param "view")))
      (if (null? v) "everything" (as-str v)))))

(define todo/view
  (lambda
    ()
    (let
      ((v (request-param "view")))
      (if (null? v) "everything" (as-str v)))))


(define todo/css
  (lambda
    ()
    "Render the todo component stylesheet (extracted from ui/theme)."
    (css/sheet
      (list
        (css/rule
          ".todo-layout"
          (list
            "display"
            "grid"
            "grid-template-columns"
            "12rem 1fr"
            "gap"
            ".75rem"))
        (css/rule
          ".todo-nav"
          (list
            "list-style"
            "none"
            "padding"
            "0"
            "margin"
            "0"))
        (css/rule ".todo-nav-item" (list "margin" ".25rem 0"))
        (css/rule
          ".todo-nav-item.is-active a"
          (list
            "font-weight"
            "700"
            "border-left"
            "3px solid #000"
            "padding-left"
            ".35rem"))
        (css/rule
          ".todo-new-list"
          (list
            "display"
            "flex"
            "flex-direction"
            "column"
            "gap"
            ".25rem"
```

```
            "margin-top"
            ".5rem"))
          (css/rule
            ".todo-new-list-input"
            (list
              "font-size"
              ".85rem"
              "min-height"
              "2rem"
              "padding"
              ".25rem"))
          (css/rule
            ".todo-box.st-TODO"
            (list "background" "var(--pending-bg)"))
          (css/rule
            ".todo-box.st-DONE"
            (list
              "background"
              "var(--done-bg)"
              "text-decoration"
              "line-through"))
          (css/rule
            ".todo-box.st-WAITING"
            (list "background" "var(--waiting-bg)"))
          (css/rule
            ".todo-box.st-CANCELLED"
            (list
              "background"
              "var(--cancelled-bg)"
              "color"
              "var(--archived-ink)"))
          (css/rule
            ".todo-box.st-ARCHIVED"
            (list
              "background"
              "var(--archived-bg)"
              "color"
              "var(--archived-ink)"
              "border-style"
              "dashed"))
          (css/rule
            "li.todo-item.is-archived .todo-title"
            (list
              "color"
              "var(--archived-ink)"
              "font-style"
              "italic"))
          (css/rule
            ".todo-from-note .todo-src"
            (list "font-size" ".8rem" "margin-left" ".35rem"))
          (css/rule
            ".todo-add-label"
            (list
              "display"
              "block"
              "font-weight"
              "bold"
              "margin"
              "0 0 .35rem"))
          (css/rule
            ".todo-add-row"
            (list
              "display"
              "flex"
              "gap"
              ".5rem"
              "align-items"
              "stretch"
              "flex-wrap"
              "wrap"))
          (css/rule
            ".todo-add-row input[type=text]"
            (list "flex" "1 1 12rem" "min-width" "0"))
```

```
        (css/rule
          ".todo-add-row button"
          (list
            "flex"
            "0 0 auto"
            "width"
            "auto"
            "min-width"
            "5rem"))
        (css/rule
          "ul.todo-list"
          (list
            "list-style"
            "none"
            "padding"
            ".25rem .5rem"
            "margin"
            "0 0 .75rem"))
        (css/rule
          "li.todo-item"
          (list
            "display"
            "flex"
            "align-items"
            "center"
            "gap"
            ".5rem"
            "padding"
            ".45rem 0"
            "border-bottom"
            "1px solid #c0c0c0"))
        (css/rule
          "li.todo-item:last-child"
          (list "border-bottom" "none"))
        (css/rule
          "li.todo-item form"
          (list "margin" "0" "display" "inline"))
        (css/rule
          "li.todo-item .todo-box"
          (list
            "font-family"
            "ui-monospace,Menlo,monospace"
            "min-width"
            "2.6rem"
            "padding"
            ".25rem .35rem"
            "width"
            "auto"))
        (css/rule
          "li.todo-item .todo-title"
          (list
            "flex"
            "1 1 auto"
            "min-width"
            "0"
            "word-break"
            "break-word"))
        (css/rule
          "li.todo-item.is-done .todo-title"
          (list
            "text-decoration"
            "line-through"
            "color"
            "var(--done-muted)"))
        (css/rule
          "li.todo-item .todo-del button, li.todo-item .todo-archive button"
          (list
            "width"
            "auto"
            "padding"
            ".25rem .45rem"
            "font-size"
            ".85rem"))
```

```
        (css/rule
          "li.todo-nav-list"
          (list
            "display"
            "flex"
            "align-items"
            "center"
            "gap"
            ".25rem"
            "flex-wrap"
            "wrap"))
        (css/rule
          "li.todo-nav-list > a"
          (list "flex" "1 1 auto" "min-width" "0"))
        (css/rule
          "li.todo-nav-list form.todo-list-del"
          (list "margin" "0" "display" "inline"))
        (css/rule
          "li.todo-nav-list .todo-list-del button"
          (list
            "width"
            "auto"
            "min-width"
            "1.6rem"
            "padding"
            ".1rem .3rem"
            "font-size"
            ".8rem"
            "line-height"
            "1.2"))
        (css/media
          "max-width:700px"
          (list
            (css/rule
              ".todo-layout"
              (list "grid-template-columns" "1fr"))))
      ))))
```

217 libs/todo/list/lib.pp

```
;; package todo/list -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push

(define todo/list
  (lambda () "Return up to 50 todo list rows." (todo/tree/rows "todo-item"
    50)))

(define todo/list/all
  (lambda () "Return up to 50 todo-list rows." (todo/tree/rows "todo-list"
    50)))

(define todo/list/create
  (lambda
    (title)
    "Create a new todo list with the given title."
    (todo/tree/create
      "todo-list"
      (dict-set*
        "{}"
        (list
          "title"
          (as-str title)
          "description"
          ""
          "is_default"
          "false")))))

(define todo/list/default?
  (lambda
    (fields)
    "Whether a list's fields mark it as the default (Inbox) list."
    (let
      ((d (dict-get fields "is_default")))
      (if (null? d) #f (string-eq d "true")))))

(define todo/list/delete
  (lambda
    (list-id)
    "Delete a todo list and its items; blocks default list."
    (let
      ((lid (as-str list-id)))
      (if
        (string-eq lid "")
        "missing"
        (let
          ((fields (todo/list/get lid)))
          (if
            (null? fields)
            "missing"
            (if
              (todo/list/default? fields)
              "default"
              (seq
                (todo/list/delete-items (todo/item/all) lid)
                (seq (todo/tree/del "todo-list" lid) "ok")))))))))

(define todo/list/delete-items
  (lambda
    (items lid)
    "Delete all todo items belonging to list lid."
    (if
      (null? items)
      ()
      (seq
        (if
          (string-eq
            (todo/item/list-id (cdr (car items)))
            (as-str lid))
          (todo/item/delete (car (car items)))
          ())
        (todo/list/delete-items (cdr items) lid)))))
```

```
70
71  (define todo/list/ensure-inbox
72    (lambda
73      ()
74      "Return the default list, creating an Inbox if none exists."
75      (let
76        ((found (todo/list/find-default)))
77        (if
78          (null? found)
79          (todo/tree/create
80            "todo-list"
81            (dict-set*
82              "{}"
83              (list
84                "title"
85                "Inbox"
86                "description"
87                "Default list"
88                "is_default"
89                "true")))
90          (car found)))))
91
92  (define todo/list/find-default
93    (lambda () "Return the first default or Inbox list, or ()."
        (todo/list/find-default-in (todo/list/all))))
94
95  (define todo/list/find-default-in
96    (lambda
97      (xs)
98      "Find the default or Inbox list within a list of rows."
99      (if
100       (null? xs)
101       ()
102       (if
103         (todo/list/default? (cdr (car xs)))
104         (car xs)
105         (let
106           ((t (todo/list/title (cdr (car xs)))))
107           (if
108             (string-eq t "Inbox")
109             (car xs)
110             (todo/list/find-default-in (cdr xs))))))))
111
112 (define todo/list/get
113   (lambda (id) "Fetch a todo list row by id." (todo/tree/get "todo-list"
        id)))
114
115 (define todo/list/nav-li
116   (lambda
117     (view row)
118     "Render a navigation list item with item ratio and delete form."
119     (let
120       ((id (car row))
121         (f (cdr row))
122         (items
123           (todo/item/active (todo/item/for-list (car row)))))
124       (html/li
125         (html/attr
126           "class"
127           (if
128             (string-eq view id)
129             "todo-nav-item is-active todo-nav-list"
130             "todo-nav-item todo-nav-list"))
131         (html/join
132           (list
133             (html/a
134               (string-append "/todos?view=" id)
135               (string-append
136                 (todo/list/title f)
137                 " ("
138                 (todo/ui/ratio items)
139                 ")"))
140             (if
```

```
              (todo/list/default? f)
              ""
              (html/form
                "POST"
                "/todos"
                (html/attrs
                  (list
                    (html/attr "class" "todo-list-del")
                    (html/attr "hx-post" "/todos")
                    (html/attr "hx-target" "#todo-main")
                    (html/attr "hx-swap" "innerHTML")
                    (html/attr
                      "hx-confirm"
                      "Delete this entire list and its items?")))
                (html/join
                  (list
                    (html/hidden "action" "delete-list")
                    (html/hidden "list_id" id)
                    (html/hidden "view" "everything")
                    (html/button
                      (html/attrs
                        (list
                          (html/attr "type" "submit")
                          (html/attr "class" "secondary")
                          (html/attr "title" "delete list")))
                      "?")))))))))))

(define todo/list/options
  (lambda
    (selected)
    "Render option tags for all lists, marking selected."
    (list/foldl
      (lambda
        (acc row)
        (let
          ((id (car row)))
          (let
            ((title (todo/list/title (cdr row))))
            (string-append
              acc
              (html/option
                id
                title
                (string-eq (as-str id) (as-str selected)))))))
      ""
      (todo/list/all))))

(define todo/list/title
  (lambda (fields) "The title of a list from its fields." (as-str (dict-get
     fields "title"))))
```

218 libs/todo/state/lib.pp

```
;; package todo/state -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push

(define todo/state/archived?
  (lambda (s) "True when state s is the ARCHIVED value." (string-eq (as-str
    s) "ARCHIVED")))

(define todo/state/list
  (list "" "TODO" "DONE" "WAITING" "CANCELLED"))

(define todo/state/next
  (lambda
    (cur)
    "Next state after cur in the canonical state list."
    (outline/todo-next-in todo/state/list (as-str cur))))

(define todo/state/columns-for-list
  (lambda
    (list-id)
    "The state set (list of state strings) for a todo-list: the list's own
     `columns` field when present and non-empty, else the global default
     todo/state/list. THE per-list override kanban columns and the /todos
     state machine both read through, so a list with custom columns cycles
     only those states everywhere."
    (let
      ((lf (get (quote todo-list) (as-str list-id))))
      (if
        (null? lf)
        todo/state/list
        (let
          ((cols (dict-get lf "columns")))
          (if
            (null? cols)
            todo/state/list
            (let
              ((cs (as-str cols)))
              (if
                (string-eq cs "")
                todo/state/list
                (json-parse cs)))))))))

(define todo/state/next-for-list
  (lambda
    (cur list-id)
    "Next state after cur within list-id's own state set (custom columns
     when
     present, else the global default). Wraps around: last column -> first
     of
     that same set."
    (todo/state/todo-next-in (todo/state/columns-for-list list-id) (as-str
     cur))))

(define todo/state/todo-next-in
  (lambda
    (xs cur)
    "State following cur within the state list xs; xs's own head on wrap
     (last
     -> first) and on a not-found cur, so the cycle never leaves the list's
     columns. (A per-set wrap, distinct from outline/todo-next-in which
     hardcodes the global default state list's head.)"
    (if
      (null? xs)
      "TODO"
      (todo/state/todo-next-in-at xs cur (as-str (car xs))))))

(define todo/state/todo-next-in-at
  (lambda
    (xs cur head)
    "Accumulating: find cur in xs, return the next; wrap to head at the end
     or
     on absent."
```

```
    (if
      (null? xs)
      head
      (if
        (string-eq (as-str (car xs)) cur)
        (if
          (null? (cdr xs))
          head
          (as-str (car (cdr xs))))
        (todo/state/todo-next-in-at (cdr xs) cur head)))))

(define todo/state/open?
  (lambda
    (s)
    "True for any state other than empty/TODO/DONE/CANCELLED/ARCHIVED."
    (let
      ((t (as-str s)))
      (if
        (string-eq t "")
        #f
        (if
          (string-eq t "DONE")
          #f
          (if
            (string-eq t "CANCELLED")
            #f
            (if (string-eq t "ARCHIVED") #f #t)))))))

(define todo/state/options
  (lambda
    (selected)
    "HTML options for the canonical state list with selected marked."
    (list/foldl
      (lambda
        (acc st)
        (let
          ((lab (if (string-eq st "") "(none)" st)))
          (string-append
            acc
            (html/option st lab (string-eq st (as-str selected))))))
      ""
      todo/state/list)))

(define todo/state/options-all
  (lambda
    (selected)
    "HTML options over the state list plus ARCHIVED."
    (todo/state/options-from
      selected
      (append todo/state/list (list "ARCHIVED")))))

(define todo/state/options-from
  (lambda
    (selected states)
    "HTML options over the given states with selected marked."
    (list/foldl
      (lambda
        (acc st)
        (let
          ((lab (if (string-eq st "") "(none)" st)))
          (string-append
            acc
            (html/option st lab (string-eq st (as-str selected))))))
      ""
      states)))

(define todo/state/options-for-list
  (lambda
    (selected list-id)
    "HTML options over list-id's own state set PLUS ARCHIVED, with selected
     marked -- the detail <select> for an item in that list. Uses the list's
     custom columns when present, else the canonical options-all state set."
    (todo/state/options-from
```

```
139         selected
140         (append (todo/state/columns-for-list list-id) (list "ARCHIVED")))))
```

219 libs/todo/tree/lib.pp

```
;; package todo/tree -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push

(define todo/tree/create
  (lambda
    (t fields)
    (let
      ((r (tree/upsert (node-path t "new") "datum" t fields)))
      (if
        (todo/tree/path-ok? r)
        (todo/tree/redash (todo/tree/tail r))
        r))))

(define todo/tree/del
  (lambda
    (t id)
    (seq (tree/delete (node-path t (as-str id))) id)))

(define todo/tree/get
  (lambda
    (t id)
    (let
      ((n (tree/get (node-path t (as-str id)))))
      (if (null? n) (list) (dict-get n "body")))))

(define todo/tree/insert-by-created
  (lambda
    (nj sorted)
    (if
      (null? sorted)
      (list nj)
      (if
        (string<?
          (as-str (dict-get nj "created_at"))
          (as-str (dict-get (car sorted) "created_at")))
        (cons nj sorted)
        (cons
          (car sorted)
          (todo/tree/insert-by-created nj (cdr sorted)))))))

(define todo/tree/node->pair
  (lambda
    (nj)
    (cons
      (todo/tree/redash (todo/tree/tail (dict-get nj "path")))
      (dict-get nj "body"))))

(define todo/tree/path-ok?
  (lambda
    (r)
    (if
      (null? r)
      #f
      (string-eq (substring (as-str r) 0 5) "data."))))

(define todo/tree/prefix
  (lambda
    (t)
    (let
      ((p (node-path t "")))
      (substring p 0 (- (string-length p) 1)))))

(define todo/tree/redash
  (lambda
    (h)
    (string-append
      (substring h 0 8)
      "-"
      (substring h 8 4)
      "-"
      (substring h 12 4)
```

```
      "-"
      (substring h 16 4)
      "-"
      (substring h 20 12))))

(define todo/tree/rows
  (lambda
    (t n)
    (list/map
      todo/tree/node->pair
      (tree/list (todo/tree/prefix t) 1 n))))

(define todo/tree/save
  (lambda
    (t id fields)
    (seq
      (tree/upsert (node-path t (as-str id)) "datum" t fields)
      id)))

(define todo/tree/set-field
  (lambda
    (t id f v)
    (let
      ((cur (todo/tree/get t id)))
      (if
        (null? cur)
        (list)
        (todo/tree/save
          t
          id
          (dict-set* cur (list f (as-str v))))))))

(define todo/tree/sort-by-created
  (lambda
    (nodes)
    (list/foldl
      (lambda (acc n) (todo/tree/insert-by-created n acc))
      (list)
      nodes)))

(define todo/tree/tail
  (lambda (p) (substring p (- (string-length p) 32) 32)))
```

220 libs/todo/ui/lib.pp

```
;; package todo/ui -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push

(define todo/ui/add-form
  (lambda
    (view list-id)
    "New-item form for a live view; explanatory note for
     from-notes/archived views."
    (if
      (string-eq view "from-notes")
      (html/p*
        (html/attr "class" "muted")
        "Mark nodes TODO in notes -- they appear here. Toggle writes back.")
      (if
        (string-eq view "archived")
        (html/p*
          (html/attr "class" "muted")
          "Archived tasks. Unarchive from detail (set state back), or
     delete.")
        (html/form
          "POST"
          "/todos"
          (html/attrs
            (list
              (html/attr "class" "card todo-add")
              (html/attr "hx-post" "/todos")
              (html/attr "hx-target" "#todo-main")
              (html/attr "hx-swap" "innerHTML")
              (html/attr
                "hx-on::after-request"
                "if(event.detail.successful){this.reset()}")))
          (html/join
            (list
              (html/hidden "action" "add")
              (html/hidden "view" view)
              (html/hidden "list_id" list-id)
              (html/label
                (html/attrs
                  (list
                    (html/attr "class" "todo-add-label")
                    (html/attr "for" "title")))
                "New item")
              (html/div
                (html/attr "class" "todo-add-row")
                (html/join
                  (list
                    (html/text-input
                      "title"
                      (html/attrs
                        (list
                          (html/attr "id" "title")
                          (html/attr "required" "required")
                          (html/attr "placeholder" "buy milk")
                          (html/attr "autocomplete" "off"))))
                    (html/button (html/attr "type" "submit")
     "Add")))))))))))

(define todo/ui/detail
  (lambda
    (id view)
    "Detail card: edit form for one todo item's fields."
    (let
      ((fields (todo/item/get id)))
      (if
        (null? fields)
        (html/div
          (html/attr "class" "card")
          (html/join
            (list
              (html/p* (html/attr "class" "err") "Item not found.")
              (html/a (string-append "/todos?view=" view) "? back"))))
```

```
      (let
        ((title (todo/item/title fields)))
        (let
          ((notes (todo/item/notes fields)))
          (let
            ((due (todo/item/due fields)))
            (let
              ((state (todo/item/state fields)))
              (let
                ((lid (todo/item/list-id fields)))
                (html/div
                  (html/attr "class" "card todo-detail")
                  (html/join
                    (list
                      (html/p*
                        (html/attr "class" "muted")
                        (html/a
                          (string-append "/todos?view=" view)
                          "? back to list"))
                      (html/h2 (html/text title))
                      (html/form
                        "POST"
                        "/todos"
                        (html/attrs
                          (list (html/attr "class" "todo-detail-form")))
                        (html/join
                          (list
                            (html/hidden "action" "save")
                            (html/hidden "id" id)
                            (html/hidden "view" view)
                            (html/label (html/attr "for" "title") "Title")
                            (html/text-input
                              "title"
                              (html/attrs
                                (list
                                  (html/attr "id" "title")
                                  (html/attr "value" title)
                                  (html/attr "required" "required"))))
                            (html/label (html/attr "for" "state") "State")
                            (html/select
                              "state"
                              (html/attr "id" "state")
                              (todo/state/options-for-list state lid))
                            (html/label (html/attr "for" "due") "Due
(YYYY-MM-DD)")
                            (html/text-input
                              "due"
                              (html/attrs
                                (list
                                  (html/attr "id" "due")
                                  (html/attr "value" due)
                                  (html/attr "placeholder"
"2026-07-15"))))
                            (html/label (html/attr "for" "list_id")
"List")
                            (html/select
                              "list_id"
                              (html/attr "id" "list_id")
                              (todo/list/options lid))
                            (html/label (html/attr "for" "notes") "Notes")
                            (html/textarea
                              "notes"
                              (html/attrs
                                (list
                                  (html/attr "id" "notes")
                                  (html/attr "rows" "8")
                                  (html/attr "class" "todo-notes")))
                              notes)
                            (html/div
                              ""
                              (html/join
                                (list
```

```
                                    (html/button (html/attr "type" "submit") "Save")
                                    " "
                                    (html/a (string-append "/todos?view=" view) "Cancel"))))))))))))))))))))

(define todo/ui/handle-post
  (lambda
    ()
    "Dispatch a POST /todos action to the matching todo/item or notes op."
    (let
      ((action (as-str (request-param "action"))))
      (cond
        ((string-eq action "cycle")
          (todo/item/cycle (request-param "id")))
        ((string-eq action "delete")
          (todo/item/delete (request-param "id")))
        ((string-eq action "archive")
          (todo/item/archive (request-param "id")))
        ((string-eq action "add")
          (todo/item/add
            (request-param "list_id")
            (request-param "title")))
        ((string-eq action "create-list")
          (todo/list/create (request-param "title")))
        ((string-eq action "delete-list")
          (todo/list/delete (request-param "list_id")))
        ((string-eq action "cycle-note")
          (notes/outline/cycle-todo
            (request-param "slug")
            (request-param "node-id")))
        ((string-eq action "save")
          (todo/item/save
            (request-param "id")
            (request-param "title")
            (request-param "notes")
            (request-param "due")
            (request-param "state")
            (request-param "list_id")))
        (else
          (todo/item/add
            (todo/list/ensure-inbox)
            (request-param "title")))))))

(define todo/ui/main
  (lambda
    (view)
    "Main todo view: detail when a GET id is given, else nav + list body."
    (let
      ((id-param (request-param "id"))
        (is-get (string-eq (request-method) "GET")))
      (if
        (if
          is-get
          (if
            (null? id-param)
            #f
            (not (string-eq (as-str id-param) "")))
          #f)
        (html/div
          (html/attr "id" "todo-main")
          (html/join
            (list
              (html/div
                (html/attr "class" "todo-layout")
                (html/join
                  (list
                    (html/div
                      (html/attr "class" "todo-side")
                      (todo/ui/nav view))
                    (html/div
                      (html/attr "class" "todo-body")
                      (todo/ui/detail (as-str id-param) view))))))))))))
```

```
        (let
          ((list-id
            (if
              (string-eq view "everything")
              (todo/list/ensure-inbox)
              (if
                (string-eq view "from-notes")
                ""
                (if (string-eq view "archived") "" view)))))
          (html/div
            (html/attr "id" "todo-main")
            (html/join
              (list
                (html/div
                  (html/attr "class" "todo-layout")
                  (html/join
                    (list
                      (html/div
                        (html/attr "class" "todo-side")
                        (todo/ui/nav view))
                      (html/div
                        (html/attr "class" "todo-body")
                        (html/join
                          (list
                            (todo/ui/add-form view list-id)
                            (todo/item/html view))))))))))))

(define todo/ui/nav
  (lambda
    (view)
    "Sidebar nav: built-in views, todo lists, and a new-list form."
    (let
      ((lists (todo/list/all))
        (inbox (todo/list/ensure-inbox))
        (arch-n (todo/item/count-all (todo/item/archived))))
      (html/ul
        (html/attr "class" "todo-nav")
        (html/join
          (list
            (todo/ui/nav-link view "everything" "Everything" "")
            (todo/ui/nav-link view "from-notes" "From notes" "")
            (todo/ui/nav-link
              view
              "archived"
              "Archived"
              (string-append "(" (as-str arch-n) ")"))
            (list/foldl
              (lambda
                (acc row)
                (string-append acc (todo/list/nav-li view row)))
              ""
              lists)
            (html/li
              (html/attr "class" "todo-nav-item")
              (html/form
                "POST"
                "/todos"
                (html/attrs
                  (list
                    (html/attr "class" "todo-new-list")
                    (html/attr "hx-post" "/todos")
                    (html/attr "hx-target" "#todo-main")
                    (html/attr "hx-swap" "innerHTML")))
                (html/join
                  (list
                    (html/hidden "action" "create-list")
                    (html/hidden "view" view)
                    (html/text-input
                      "title"
                      (html/attrs
                        (list
                          (html/attr "placeholder" "New list")
                          (html/attr "required" "required")
```

```
                        (html/attr "class" "todo-new-list-input"))))
                    (html/button (html/attr "type" "submit") "+
    list")))))))))))

(define todo/ui/nav-link
  (lambda
    (view key label extra)
    "Nav list item linking to a view, active when it matches the current
     view."
    (html/li
      (html/attr
        "class"
        (if
          (string-eq view key)
          "todo-nav-item is-active"
          "todo-nav-item"))
      (html/a
        (string-append "/todos?view=" key)
        (string-append
          label
          (if (string-eq extra "") "" (string-append " " extra)))))))

(define todo/ui/ratio
  (lambda
    (items)
    "Done/total ratio string for the given items."
    (let
      ((n (todo/item/count-all items))
        (open (todo/item/count-open items)))
      (string-append (as-str (- n open)) "/" (as-str n)))))

(define todo/ui/view
  (lambda
    ()
    "Current view name from the request, defaulting to everything."
    (let
      ((v (request-param "view")))
      (if (null? v) "everything" (as-str v)))))
```

221 libs/tree/lib.pp

```
;; package tree -- exported from live branch admin/personal
;; Sync: scripts/pp_sync.py pull|push

(define tree
  (outline/node*
    "root"
    "Move smoke A"
    ""
    ""
    (list
      (outline/node* "keep" "Keep me" "" "" (list))
      (outline/node*
        "child"
        "Child title"
        "body-x"
        "TODO"
        (list (outline/node* "grand" "Grand" "" "" (list)))))))

(define tree/as-int
  (lambda
    (x dft)
    "Parse x as an int, clamping negatives to 0; dft when blank/invalid."
    (if
      (tree/blank? x)
      dft
      (let
        ((n (parse-int (as-str x))))
        (if (null? n) dft (if (< n 0) 0 n))))))

(define tree/badge
  (lambda
    (n)
    "Render a child-count badge span for a node, or empty when leaf."
    (let
      ((c (tree/child-count n)))
      (if
        (<= c 0)
        ""
        (html/span
          (html/attr "class" "tb-badge")
          (html/text (string-append " (" (as-str c) ")")))))))

(define tree/blank?
  (lambda (x) "True when x is null or an empty string." (or (null? x)
     (string-eq (as-str x) ""))))

(define tree/breadcrumb
  (lambda
    (tree focus-id slug folders-only?)
    "Render a navigable breadcrumb to a focus node."
    (let
      ((path (tree/find-path tree (as-str focus-id))))
      (if
        (null? path)
        ""
        (html/div
          (html/attr "class" "tb-crumb")
          (tree/breadcrumb-parts path slug folders-only? #t))))))

(define tree/breadcrumb-parts
  (lambda
    (path slug folders-only? first?)
    "Recursively render breadcrumb links over a node path."
    (if
      (null? path)
      ""
      (let
        ((n (car path)) (rest (cdr path)))
        (string-append
          (if
            first?
```

```
            ""
            (html/span (html/attr "class" "sep") (html/text " / ")))
          (if
            (null? rest)
            (html/span
              (html/attr "class" "tb-crumb-here")
              (html/text (as-str (outline/title n))))
            (html/a
              (tree/page-url slug (outline/id n) 2 folders-only?)
              (html/text (as-str (outline/title n)))))
          (tree/breadcrumb-parts rest slug folders-only? #f))))))

(define tree/child-count
  (lambda (n) "Return the number of child nodes of n." (if (null? n) 0
     (count (outline/kids n)))))

(define tree/default-depth
  (lambda () "Return the requested tree depth param, defaulting to 2."
     (tree/as-int (request-param "depth") 2)))

(define tree/default-slug
  (lambda
    ()
    "Return the requested slug param, defaulting to scripture_concepts."
    (let
      ((s (request-param "slug")))
      (if (tree/blank? s) "scripture_concepts" (as-str s)))))

(define tree/find-path
  (lambda
    (tree target)
    "Return the list of nodes from root to the target node."
    (tree/find-path-acc tree (as-str target) (list))))

(define tree/find-path-acc
  (lambda
    (node target acc)
    "Accumulator helper that walks a node looking for target id."
    (if
      (null? node)
      ()
      (let
        ((here (append acc (list node))))
        (if
          (string-eq (outline/id node) target)
          here
          (tree/find-path-in-kids (outline/kids node) target here))))))

(define tree/find-path-in-kids
  (lambda
    (kids target acc)
    "Search a kids list for target, returning its path or ()."
    (if
      (null? kids)
      ()
      (let
        ((hit (tree/find-path-acc (car kids) target acc)))
        (if
          (null? hit)
          (tree/find-path-in-kids (cdr kids) target acc)
          hit)))))

(define tree/folders-only-param
  (lambda
    ()
    "Return whether the folders-only query param is set."
    (tree/truthy? (request-param "folders-only"))))

(define tree/has-children?
  (lambda
    (n)
    "True when node n has at least one child."
    (if (null? n) #f (not (null? (outline/kids n))))))
```

```
142
143 (define tree/leaf-row
144   (lambda
145     (slug n folders-only?)
146     "Render a leaf node row, or empty when folders-only."
147     (if
148       folders-only?
149       ""
150       (html/tag
151         "li"
152         (html/attr "class" "tb-node tb-leaf")
153         (html/div
154           (html/attr "class" "tb-row")
155           (html/join
156             (list
157               (html/span (html/attr "class" "tb-twisty") ".")
158               (tree/title-link slug n folders-only?))))))))
159
160 (define tree/load-doc
161   (lambda
162     (slug)
163     "Load a document's outline by slug, or () when missing."
164     (let
165       ((doc (notes/doc/get (as-str slug))))
166       (if (null? doc) () (notes/doc/outline doc)))))
167
168 (define tree/open-row
169   (lambda
170     (slug n depth folders-only?)
171     "Render an expanded tree node with a child list."
172     (html/tag
173       "li"
174       (html/attrs
175         (list
176           (html/attr "class" "tb-node tb-open")
177           (html/attr "id" (string-append "tb-" (outline/id n)))))
178       (html/join
179         (list
180           (html/div
181             (html/attr "class" "tb-row")
182             (html/join
183               (list
184                 (html/span (html/attr "class" "tb-twisty") "-")
185                 (tree/title-link slug n folders-only?)
186                 (tree/badge n))))
187           (html/tag
188             "ul"
189             (html/attr "class" "tb-kids")
190             (tree/render-kids
191               (outline/kids n)
192               (- depth 1)
193               folders-only?
194               slug)))))))
195
196 (define tree/page
197   (lambda () "Render the full tree browser page." (html/page (list
      (tree/page-body)))))
198
199 (define tree/page-body
200   (lambda
201     ()
202     "Render the tree page body from current request params."
203     (let
204       ((slug (tree/default-slug))
205         (rid (tree/root-param))
206         (d (tree/default-depth))
207         (fo (tree/folders-only-param)))
208       (html/join
209         (list
210           (tree/style)
211           (html/h1 "Tree")
212           (html/p*
213             (html/attr "class" "muted")
```

```
            "Lazy outline browser (image.tree-browser). Folder = non-leaf
    view bit.")
          (tree/toolbar slug rid d fo)
          (tree/render-doc slug rid d fo))))))

(define tree/page-url
  (lambda
    (slug root depth folders-only?)
    "Build a /tree URL for the given navigation args."
    (string-append
      "/tree?"
      (tree/qs slug root depth folders-only?))))

(define tree/partial
  (lambda () "Render the lazy-load tree partial page." (html/page (list
     (tree/partial-body)))))

(define tree/partial-body
  (lambda
    ()
    "Render a single focus node for lazy partial load."
    (let
      ((slug (tree/default-slug))
        (rid (tree/root-param))
        (d (tree/default-depth))
        (fo (tree/folders-only-param)))
      (let
        ((doc (tree/load-doc slug)))
        (if
          (null? doc)
          (html/p* (html/attr "class" "err") "missing doc")
          (let
            ((focus
              (if (string-eq rid "root") doc (outline/find doc rid))))
            (if
              (null? focus)
              (html/p* (html/attr "class" "err") "missing node")
              (tree/render-node focus (if (<= d 0) 2 d) fo slug)))))))))

(define tree/partial-url
  (lambda
    (slug root depth folders-only?)
    "Build a /partials/tree URL for the given navigation args."
    (string-append
      "/partials/tree?"
      (tree/qs slug root depth folders-only?))))

(define tree/qs
  (lambda
    (slug root depth folders-only?)
    "Build the tree query string from navigation args."
    (string-append
      "slug="
      (as-str slug)
      "&root="
      (as-str root)
      "&depth="
      (as-str depth)
      (if folders-only? "&folders-only=1" ""))))

(define tree/render
  (lambda
    (root-id depth folders-only?)
    "Render the scripture_concepts document tree directly."
    (tree/render-doc
      "scripture_concepts"
      root-id
      depth
      folders-only?)))

(define tree/render-doc
  (lambda
    (slug root-id depth folders-only?)
```

```
    "Render a whole document tree for slug, focused on root-id."
    (let
      ((rid (if (tree/blank? root-id) "root" (as-str root-id)))
        (d (if (null? depth) 2 (if (< depth 0) 0 depth)))
        (fo (if (null? folders-only?) #f folders-only?)))
      (let
        ((doc (tree/load-doc slug)))
        (if
          (null? doc)
          (html/p*
            (html/attr "class" "tb-empty")
            (string-append "No outline for slug " (as-str slug)))
          (let
            ((focus
              (if (string-eq rid "root") doc (outline/find doc rid))))
            (if
              (null? focus)
              (html/p*
                (html/attr "class" "tb-empty")
                (string-append "No node " rid " in " (as-str slug)))
              (html/join
                (list
                  (tree/breadcrumb doc rid slug fo)
                  (html/tag
                    "ul"
                    (html/attrs
                      (list
                        (html/attr "class" "tb")
                        (html/attr "id" "tb-root")))
                    (tree/render-node focus d fo slug)))))))))))

(define tree/render-kids
  (lambda
    (kids depth folders-only? slug)
    "Concatenate rendered rows for every node in kids."
    (if
      (null? kids)
      ""
      (string-append
        (tree/render-node (car kids) depth folders-only? slug)
        (tree/render-kids (cdr kids) depth folders-only? slug)))))

(define tree/render-node
  (lambda
    (n depth folders-only? slug)
    "Render one node: leaf, open, or stub depending on depth."
    (if
      (null? n)
      ""
      (if
        (not (tree/has-children? n))
        (tree/leaf-row slug n folders-only?)
        (if
          (<= depth 0)
          (tree/stub slug n 2 folders-only?)
          (tree/open-row slug n depth folders-only?))))))

(define tree/root-param
  (lambda
    ()
    "Return the root/node-id param, defaulting to root."
    (let
      ((r (request-param "root")))
      (if
        (tree/blank? r)
        (let
          ((n (request-param "node-id")))
          (if (tree/blank? n) "root" (as-str n)))
        (as-str r)))))

(define tree/stub
  (lambda
    (slug n depth folders-only?)
```

```
    "Render a collapsed stub that lazy-loads its children."
    (let
      ((url
        (tree/partial-url
          slug
          (outline/id n)
          depth
          folders-only?)))
      (html/tag
        "li"
        (html/attrs
          (list
            (html/attr "class" "tb-stub")
            (html/attr "id" (string-append "tb-" (outline/id n)))))
        (html/tag
          "a"
          (html/attrs
            (list
              (html/attr "class" "tb-row")
              (html/attr "href" url)
              (html/attr "hx-get" url)
              (html/attr "hx-trigger" "click")
              (html/attr "hx-swap" "outerHTML")
              (html/attr "hx-target" "closest li")))
          (html/join
            (list
              (html/span (html/attr "class" "tb-twisty") "+")
              (html/span
                (html/attr "class" "tb-title")
                (html/text (as-str (outline/title n))))
              (tree/badge n)))))))))

(define tree/style
  (lambda
    ()
    "Return the CSS style tag for the tree browser."
    (html/tag
      "style"
      ""
      (string-append
        ".tb{list-style:none;padding:0;margin:.5rem 0;}"
        ".tb ul{list-style:none;padding:0 0 0 1.1rem;margin:.15rem
    0;border-left:1px solid #33cc33;}"
        ".tb-node,.tb-stub{margin:.1rem 0;}"
        ".tb-row{display:flex;align-items:center;gap:.35rem;padding:.15rem
    .25rem;}"
        ".tb-row:hover{background:#1a2a1a;}"

    ".tb-twisty{display:inline-block;width:1.1rem;color:#888;text-align:center;}"
        ".tb-title{color:#33ff33;text-decoration:none;font-weight:700;}"
        ".tb-title:hover{color:#66ff66;text-decoration:underline;}"
        ".tb-badge{color:#888;font-size:.85rem;}"

    ".tb-stub>a.tb-row{display:flex;cursor:pointer;color:inherit;text-decoration:none;}"

    ".tb-crumb{display:flex;flex-wrap:wrap;gap:.35rem;align-items:center;margin:.5rem
    0 1rem;}"
        ".tb-crumb a{color:#33ff33;}"
        ".tb-crumb .sep{color:#888;}"

    ".tb-toolbar{display:flex;flex-wrap:wrap;gap:.5rem;align-items:center;margin:.5rem
    0;}"
        ".tb-toolbar a,.tb-toolbar label{font-size:.9rem;}"
        ".tb-empty{color:#888;font-style:italic;}"))))

(define tree/title-link
  (lambda
    (slug n folders-only?)
    "Render a link to a node's page using its title."
    (html/tag
      "a"
      (html/attrs
        (list
```

```
          (html/attr
            "href"
            (tree/page-url slug (outline/id n) 2 folders-only?))
          (html/attr "class" "tb-title")))
      (html/text (as-str (outline/title n))))))

(define tree/toolbar
  (lambda
    (slug root depth folders-only?)
    "Render the tree navigation toolbar."
    (let
      ((flip (not folders-only?)))
      (html/div
        (html/attr "class" "tb-toolbar")
        (html/join
          (list
            (html/a
              (tree/page-url slug "root" depth #f)
              "full tree")
            (html/a
              (tree/page-url slug root depth flip)
              (if folders-only? "show leaves" "folders only"))
            (html/a
              (string-append "/notes?slug=" (as-str slug))
              "open in notes")
            (html/span
              (html/attr "class" "muted")
              (html/text
                (string-append
                  "slug="
                  (as-str slug)
                  " root="
                  (as-str root)
                  " depth="
                  (as-str depth))))))))))

(define tree/truthy?
  (lambda
    (x)
    "True when x is a truthy value: 1, true, yes, or on."
    (if
      (tree/blank? x)
      #f
      (let
        ((s (as-str x)))
        (or
          (string-eq s "1")
          (string-eq s "true")
          (string-eq s "yes")
          (string-eq s "on"))))))
```

222 libs/ui/lib.pp

```
;; package ui -- exported from live branch admin/personal
;; Sync: scripts/pp_sync.py pull|push

(package ui
  (export
    ui/css
    ui/home
    ui/home-style
    ui/menu
    ui/nav
    ui/nav-style
    ui/theme
    ui/theme-button
    ui/theme-form
    ui/theme-header
    ui/theme-mobile
    ui/theme-table
    ui/theme-nav
))

(define ui/css
  (lambda
    ()
    "Render the combined UI stylesheet."
    (string-append
      "/* ui/css */\n"
      (css/render
        (css/sheet
          (list
            (css/use (theme/vars))
            (css/use (ui/theme))
            (css/use (ui/theme-header))
            (css/use (ui/theme-table))
            (css/use (ui/theme-form))
            (css/use (ui/theme-button))
            (css/use (outline/css))
            (css/use (notes/css))
            (css/use (todo/css))
            (css/use (calendar/css))
            (css/use (weather/css))
            (css/use (layout/card-css))
            (css/use (bible/css))
            (css/use (ui/theme-menu))
            (css/use (ui/theme-branches))
            (css/use (ui/theme-code-browser))
            (css/use (ui/theme-capture))
            (css/use (ui/theme-mobile))
            (css/use (ui/theme-nav))
            (css/use notes/tree-css)
            (css/use (theme/extra)))))))

(define ui/home
  (lambda
    ()
    "Render the home page with weather and astro widgets."
    (html/page (list (weather/widget) (astro/widget)))))

(define ui/home-style
  (lambda
    ()
    "Render the home page style block."
    (html/tag
      "style"
      ""
      (html/join
        (list
          "nav{display:flex;flex-wrap:wrap;gap:.35rem
    .5rem;align-items:center;width:100%}"
          "nav>a,.menu-drop{flex:0 0 auto}"
          "@media(max-width:40rem){"
          "nav>a,.menu-drop{flex:1 1 0;min-width:0;text-align:center}"
```

```
          "nav>a{display:block;text-align:center}"
     ".menu-drop-btn{display:block;width:100%;text-align:center;box-sizing:border-box}"
          "}"
          ".capture-card label{display:block;margin:.35rem 0
     .15rem;font-weight:700}"
          ".capture-card
     .capture-body{width:100%;box-sizing:border-box;min-height:4.5rem}"
          ".capture-card
     .capture-row{display:flex;flex-wrap:wrap;gap:.5rem;align-items:center;margin-top:.5rem}"
          ".capture-card .capture-row button{width:auto;min-width:6rem}"
          ".capture-card #capture-status{margin-top:.35rem;font-size:.9rem}"
          ".capture-card #capture-status.ok,.capture-card
     .ok{color:var(--ok);font-weight:700}"
          ".capture-card #capture-status.err,.capture-card
     .err{color:var(--err);font-weight:700}")))))

(define ui/menu
  (lambda
    ()
    "Build the site navigation menu structure."
    (list
      (menu/link "home" "/")
      (menu/raw (notes/capture/nav))
      (menu/group
        "apps"
        (list
          (menu/group
            "writing"
            (list
              (menu/link "notes" "/notes")
              (menu/link "inbox" "/notes?slug=inbox")
              (menu/link "plans" "/plans")
              (menu/link "bookmarks" "/bookmarks")))
          (menu/group
            "study"
            (list
              (menu/link "dabar" "/dabar")
              (menu/link "strongs" "/strongs")
              (menu/link "flashcards" "/flashcards")))
          (menu/group
            "tasks"
            (list
              (menu/link "todos" "/todos")
              (menu/link "calendar" "/calendar")))
          (menu/group
            "connect"
            (list
              (menu/link "contacts" "/contacts")
              (menu/link "chat" "/chat")))
          (menu/group
            "tools"
            (list
              (menu/link "weather" "/weather")
              (menu/link "agent" "/agent")
              (menu/link "calories" "/cal")))))
      (menu/group
        "system"
        (list
          (menu/link "functions" "/bindings")
          (menu/link "ref" "/ref")
          (menu/link "pages" "/pages")
          (menu/link "data" "/types")
          (menu/link "branches" "/branches")
          (menu/link "analytics" "/analytics")
          (menu/link "llm keys" "/settings/keys")
          (menu/link "ssh keys" "/settings/ssh")
          (menu/link "themes" "/settings/theme")
          (menu/link "about" "/about")
          (menu/link "repl" "/repl"))))))

(define ui/nav
  (lambda
```

```
    ()
    "Render the site navigation bar."
    (render-html
      (menu/bar
        (if
          (string-eq (type-of ui/menu) "lambda")
          (ui/menu)
          ui/menu)))))

(define ui/nav-style
  (lambda
    ()
    "Render the site navigation bar style block."
    (html/tag
      "style"
      ""
      (html/join
        (list
          "nav{display:flex;flex-wrap:wrap;gap:.35rem
    .5rem;align-items:center;width:100%}"
          "nav>a,.menu-drop{flex:0 0 auto}"
          "@media(max-width:40rem){"
          "nav>a,.menu-drop{flex:1 1 0;min-width:0;text-align:center}"
          "nav>a{display:block;text-align:center}"

    ".menu-drop-btn{display:block;width:100%;text-align:center;box-sizing:border-box}"
          "}"
          ".capture-panel{min-width:16rem;max-width:min(22rem,calc(100vw -
    1rem));padding:.4rem}"

    ".capture-nav-form{display:flex;flex-direction:column;gap:.35rem;margin:0}"
          ".capture-nav-form input[type=text],.capture-nav-form textarea{"
          "width:100%;box-sizing:border-box;font-size:.9rem}"
          ".capture-nav-form .capture-body{min-height:4rem}"
          ".capture-nav-form
    .capture-row{display:flex;flex-wrap:wrap;gap:.35rem;align-items:center}"
          ".capture-nav-form .capture-row
    button{width:auto;min-width:4.5rem;margin:0}"
          ".capture-nav-form .capture-row
    a{display:inline-block;padding:.35rem .45rem}"
          ".capture-nav-form
    #capture-status{font-size:.85rem;min-height:1.1em}"
          ".capture-nav-form .ok{color:var(--ok);font-weight:700}"
          ".capture-nav-form .err{color:var(--err);font-weight:700}")))))

(define ui/theme
  (lambda
    ()
    "Render the base theme stylesheet rules."
    (css/sheet
      (list
        (css/rule "*" (list "box-sizing" "border-box"))
        (css/rule
          "html"
          (list
            "font-size"
            "16px"
            "-webkit-text-size-adjust"
            "100%"))
        (css/rule
          "body"
          (list
            "margin"
            "0"
            "background"
            "var(--bg)"
            "color"
            "var(--ink)"
            "font-family"
            "var(--font-sans)"
            "line-height"
            "1.4"))
        (css/rule
```

```
          "a"
          (list
            "color"
            "var(--link)"
            "text-decoration"
            "underline"))
        (css/rule "a:visited" (list "color" "var(--visited)"))
        (css/rule "a:hover" (list "color" "var(--ink)"))
        (css/rule
          "nav"
          (list
            "display"
            "flex"
            "flex-wrap"
            "wrap"
            "gap"
            ".35rem .5rem"
            "align-items"
            "center"))
        (css/rule
          ".emergency-banner"
          (list
            "display"
            "block"
            "width"
            "100%"
            "padding"
            ".4rem .55rem"
            "border"
            "2px solid var(--banner-border)"
            "background"
            "var(--banner-bg)"
            "color"
            "var(--banner-ink)"
            "font-size"
            ".85rem"
            "font-family"
            "var(--font-mono)"))
        (css/rule
          ".emergency-banner a"
          (list "color" "var(--link)"))
        (css/rule
          ".emergency-home"
          (list
            "border"
            "2px solid var(--banner-border)"
            "background"
            "var(--banner-bg)"))
        (css/rule
          "main"
          (list
            "padding"
            ".75rem"
            "max-width"
            "52rem"
            "margin"
            "0 auto"))
        (css/rule
          "h1"
          (list
            "font-size"
            "1.25rem"
            "font-weight"
            "700"
            "margin"
            ".25rem 0 .75rem"
            "border-bottom"
            "1px solid #000"
            "padding-bottom"
            ".25rem"))
        (css/rule
          "h2"
          (list
```

```
            "font-size"
            "1.05rem"
            "font-weight"
            "700"
            "margin"
            "0 0 .5rem"))
        (css/rule "p" (list "margin" ".5rem 0"))
        (css/rule "label" (list "font-weight" "700"))
        (css/rule
          ".meta,.muted"
          (list "color" "var(--muted)" "font-size" ".9rem"))
        (css/rule
          ".err"
          (list "color" "var(--err)" "font-weight" "700"))
        (css/rule
          ".ok"
          (list "color" "var(--ok)" "font-weight" "700"))
        (css/rule
          "#out"
          (list
            "white-space"
            "pre-wrap"
            "min-height"
            "4rem"
            "background"
            "var(--panel)"
            "border"
            "1px solid #000"
            "padding"
            ".5rem"
            "font-family"
            "var(--font-mono)"))
        (css/rule
          "ul.actions"
          (list
            "list-style"
            "none"
            "padding"
            "0"
            "margin"
            ".5rem 0"))
        (css/rule "ul.actions li" (list "margin" ".4rem 0"))
        (css/rule
          ".badge.impact"
          (list
            "background"
            "var(--badge-bg)"
            "border"
            "1px solid var(--badge-border)"
            "padding"
            ".15rem .4rem"
            "font-size"
            ".85rem"))

))))

(define ui/theme-header
  (lambda
    ()
    "Render the theme-header base stylesheet rules."
    (css/sheet
      (list
        (css/rule
          "header"
          (list
            "background"
            "var(--panel)"
            "border-bottom"
            "2px solid #000"
            "padding"
            ".5rem .75rem"))
        (css/rule
          "header .brand"
```

```
          (list
            "font-weight"
            "700"
            "font-size"
            "1.1rem"
            "letter-spacing"
            ".02em"
            "color"
            "var(--ink)"
            "margin-right"
            ".75rem"))
        (css/rule
          "header .meta"
          (list
            "display"
            "block"
            "font-family"
            "var(--font-mono)"
            "font-size"
            ".8rem"
            "color"
            "var(--muted)"
            "margin"
            ".25rem 0 .5rem"))))))

(define ui/theme-table
  (lambda
    ()
    "Render the theme-table base stylesheet rules."
    (css/sheet
      (list
        (css/rule
          "table"
          (list
            "border-collapse"
            "collapse"
            "width"
            "100%"
            "background"
            "var(--panel)"
            "border"
            "1px solid #000"
            "font-size"
            ".9rem"))
        (css/rule
          "th,td"
          (list
            "border"
            "1px solid #808080"
            "padding"
            ".45rem .5rem"
            "text-align"
            "left"
            "vertical-align"
            "top"
            "word-break"
            "break-word"))
        (css/rule
          "th"
          (list
            "background"
            "var(--chrome)"
            "font-weight"
            "700"
            "border-color"
            "var(--ink)"))
        (css/rule
          "tr:nth-child(even) td"
          (list "background" "var(--row-alt)"))))))

(define ui/theme-form
  (lambda
    ()
```

```
    "Render the theme-form base stylesheet rules."
    (css/sheet
      (list
        (css/rule
     "pre,textarea,input[type=text],input[type=search],input:not([type])"
          (list
            "font-family"
            "var(--font-mono)"
            "font-size"
            "1rem"
            "background"
            "var(--panel)"
            "color"
            "var(--ink)"
            "border"
            "1px solid #000"
            "border-radius"
            "0"
            "padding"
            ".5rem"
            "width"
            "100%"
            "max-width"
            "100%"))
        (css/rule
          "pre"
          (list
            "white-space"
            "pre-wrap"
            "overflow-x"
            "auto"
            "line-height"
            "1.35"))
        (css/rule
          "textarea"
          (list
            "min-height"
            "16rem"
            "line-height"
            "1.35"
            "white-space"
            "pre"
            "tab-size"
            "2"))))))

(define ui/theme-button
  (lambda
    ()
    "Render the theme-button base stylesheet rules."
    (css/sheet
      (list
        (css/rule
          "button,.btn,input[type=submit]"
          (list
            "font"
            "inherit"
            "font-size"
            "1rem"
            "cursor"
            "pointer"
            "background"
            "var(--chrome)"
            "color"
            "var(--ink)"
            "border"
            "2px outset #fff"
            "border-radius"
            "0"
            "padding"
            ".5rem .85rem"
            "min-height"
            "2.75rem"))
```

```
        (css/rule
          "button:active,.btn:active"
          (list "border-style" "inset"))
        (css/rule
          "button.secondary,.btn.secondary"
          (list "background" "var(--bg)"))
        (css/rule
          "a.btn"
          (list
            "display"
            "inline-block"
            "text-decoration"
            "none"
            "color"
            "var(--ink)"
            "line-height"
            "1.2"
            "vertical-align"
            "middle"))
        (css/rule "a.btn:visited" (list "color" "var(--ink)"))))))

(define ui/theme-mobile
  (lambda
    ()
    "Render the theme-mobile mobile override stylesheet rules."
    (css/sheet
      (list
        (css/media
          "max-width:40rem"
          (list
            (css/rule "main" (list "padding" ".5rem"))
            (css/rule "header" (list "padding" ".5rem"))
            (css/rule "nav" (list "gap" ".3rem" "width" "100%"))
            (css/rule
              "nav > a,.menu-drop"
              (list
                "flex"
                "1 1 0"
                "min-width"
                "0"
                "text-align"
                "center"))
            (css/rule "nav > a" (list "display" "block"))
            (css/rule
              ".menu-drop-btn"
              (list
                "display"
                "block"
                "width"
                "100%"
                "text-align"
                "center"))
            (css/rule
              "table"
              (list
                "display"
                "block"
                "overflow-x"
                "auto"
                "-webkit-overflow-scrolling"
                "touch"))
            (css/rule "h1" (list "font-size" "1.15rem"))
            (css/rule
              "button,.btn"
              (list "width" "100%" "margin" ".2rem 0"))
            (css/rule
              ".card form button,.card form .btn"
              (list "width" "auto" "min-width" "44%"))
            (css/rule
              "button.ol-btn,button.ol-todo-btn,
    .ol-btn,.ol-todo,.ol-todo-btn"
              (list "width" "auto" "margin" "0"))
            (css/rule
```

```
              ".ol-node"
              (list "margin-left" ".1rem" "padding-left" ".3rem"))
            (css/rule
              ".ol-sum"
              (list "padding-right" "2.5rem" "align-items" "center"))
            (css/rule ".ol-toolbar" (list "display" "none"))
            (css/rule "details.ol-actions" (list "display" "block"))
            (css/rule ".ol-sum" (list "padding-right" "2.75rem"))
            (css/rule
              "details.ol-actions[open] > .ol-actions-panel"
              (list
                "min-width"
                "9rem"
                "padding"
                ".35rem"
                "box-shadow"
                "2px 2px 0 #808080"))
            (css/rule
              "details.ol-actions[open] > .ol-actions-panel .ol-btn"
              (list
                "min-height"
                "2.4rem"
                "width"
                "100%"
                "margin"
                "0 0 .25rem"
                "background"
                "var(--panel)"
                "text-align"
                "left")))))))

(define ui/theme-menu
  (lambda
    ()
    "Render the theme-menu stylesheet rules."
    (css/sheet
      (list
        (css/rule "nav > a,.menu-drop" (list "flex" "0 0 auto"))
        (css/rule
          "nav a,.menu-drop-btn"
          (list
            "display"
            "inline-block"
            "padding"
            ".45rem .55rem"
            "min-height"
            "2.5rem"
            "line-height"
            "1.5"
            "border"
            "1px solid #000"
            "background"
            "var(--chrome)"
            "color"
            "var(--ink)"
            "text-decoration"
            "none"
            "font-size"
            ".9rem"
            "cursor"
            "pointer"
            "list-style"
            "none"
            "box-sizing"
            "border-box"))
        (css/rule "nav a:visited" (list "color" "var(--ink)"))
        (css/rule
          "nav a:hover,nav a:focus,.menu-drop-btn:hover"
          (list "background" "var(--panel)"))
        (css/rule
          "nav.emergency-nav"
          (list "width" "100%" "margin-top" ".35rem"))
        (css/rule
```

```
639           ".menu-drop"
640           (list
641             "position"
642             "relative"
643             "display"
644             "inline-block"
645             "margin"
646             "0"))
647         (css/rule
648           ".menu-drop>summary"
649           (list "list-style" "none"))
650         (css/rule
651           ".menu-drop>summary::-webkit-details-marker"
652           (list "display" "none"))
653         (css/rule
654           ".menu-drop>summary::after"
655           (list "content" "\" v\"" "font-size" ".7em"))
656         (css/rule
657           ".menu-drop[open]>summary"
658           (list "position" "relative" "z-index" "81"))
659         (css/rule
660           ".menu-drop[open]>summary::before"
661           (list
662             "content"
663             "\"\""
664             "position"
665             "fixed"
666             "inset"
667             "0"
668             "z-index"
669             "80"
670             "cursor"
671             "default"
672             "background"
673             "transparent"))
674         (css/rule
675           ".menu-drop-panel"
676           (list
677             "display"
678             "none"
679             "position"
680             "absolute"
681             "left"
682             "0"
683             "right"
684             "auto"
685             "top"
686             "100%"
687             "z-index"
688             "82"
689             "min-width"
690             "10rem"
691             "max-width"
692             "min(16rem,calc(100vw - 1rem))"
693             "width"
694             "max-content"
695             "margin-top"
696             "2px"
697             "padding"
698             ".25rem"
699             "box-sizing"
700             "border-box"
701             "background"
702             "var(--panel)"
703             "border"
704             "2px solid #000"
705             "box-shadow"
706             "2px 2px 0 #808080"))
707         (css/rule
708           ".menu-drop[open]>.menu-drop-panel"
709           (list "display" "block"))
710         (css/rule
711           ".menu-drop-panel a"
```

```
          (list
            "display"
            "block"
            "width"
            "100%"
            "margin"
            "0 0 .2rem"
            "white-space"
            "nowrap"
            "box-sizing"
            "border-box"))
        (css/rule
          ".menu-drop-panel a:last-child"
          (list "margin-bottom" "0"))
        (css/media
          "max-width:40rem"
          (list
            (css/rule "header,nav" (list "overflow" "visible"))
            (css/rule
              ".menu-drop-panel"
              (list
                "left"
                "auto"
                "right"
                "0"
                "max-width"
                "calc(100vw - .75rem)"))
            (css/rule ".capture-drop" (list "position" "static"))
            (css/rule
              ".capture-drop > .menu-drop-panel.capture-panel"
              (list
                "left"
                ".5rem"
                "right"
                ".5rem"
                "width"
                "auto"
                "min-width"
                "0"
                "max-width"
                "none"
                "box-sizing"
                "border-box"))))
      ))))

(define ui/theme-branches
  (lambda
    ()
    "Render the theme-branches stylesheet rules."
    (css/sheet
      (list
        (css/rule ".br-table-wrap" (list "overflow-x" "auto"))
        (css/rule
          ".br-table"
          (list "width" "100%" "font-size" ".9rem"))
        (css/rule
          ".br-table th,.br-table td"
          (list "padding" ".35rem .4rem" "vertical-align" "top"))
        (css/rule
          "tr.br-current"
          (list "background" "var(--sel-bg)"))
        (css/rule
          ".br-stats"
          (list
            "display"
            "flex"
            "flex-wrap"
            "wrap"
            "gap"
            ".5rem"
            "margin"
            ".5rem 0"))
        (css/rule
```

```
          ".br-stat"
          (list "padding" ".4rem .6rem" "margin" "0"))
        (css/rule
          ".br-diff"
          (list
            "display"
            "grid"
            "grid-template-columns"
            "1fr 1fr"
            "gap"
            ".5rem"))
        (css/rule "form.inline-form" (list "display" "inline"))
        (css/rule
          "form.inline-form button"
          (list "margin-left" ".2rem"))
        (css/media
          "max-width:700px"
          (list
            (css/rule
              ".br-diff"
              (list "grid-template-columns" "1fr"))))
      ))))

(define ui/theme-code-browser
  (lambda
    ()
    "Render the theme-code-browser stylesheet rules."
    (css/sheet
      (list
        (css/rule
          ".bb-browser"
          (list
            "display"
            "flex"
            "flex-direction"
            "column"
            "gap"
            ".75rem"))
        (css/rule ".bb-three" (list "display" "block"))
        (css/rule ".bb-three .bb-col" (list "margin" ".5rem 0"))
        (css/rule
          ".bb-list"
          (list
            "list-style"
            "none"
            "padding"
            "0"
            "margin"
            ".25rem 0"))
        (css/rule
          ".bb-item"
          (list "margin" "0" "line-height" "1.35"))
        (css/rule
          ".bb-item a"
          (list
            "display"
            "block"
            "padding"
            ".2rem .35rem"
            "text-decoration"
            "none"
            "color"
            "var(--link)"
            "border-left"
            "3px solid transparent"))
        (css/rule
          ".bb-item.is-active a"
          (list
            "font-weight"
            "700"
            "background"
            "var(--row-alt)"
            "border-left"
```

```
            "3px solid #000"
            "color"
            "var(--ink)"))
          (css/rule
            ".bb-item a:hover"
            (list "background" "var(--chrome)" "color" "var(--ink)"))
          (css/rule
            ".bb-crumb"
            (list "margin" "0 0 .5rem" "font-size" ".9rem"))
          (css/rule
            ".bb-meta h3"
            (list "margin" ".75rem 0 .35rem" "font-size" "1rem"))
          (css/rule ".bb-code" (list "margin" "0"))
          (css/rule
            ".bb-code-head"
            (list
              "display"
              "flex"
              "flex-wrap"
              "wrap"
              "align-items"
              "baseline"
              "justify-content"
              "space-between"
              "gap"
              ".5rem"
              "margin-bottom"
              ".5rem"))
          (css/rule
            ".bb-code-head h2"
            (list "margin" "0" "font-size" "1.1rem"))
          (css/rule ".bb-code-actions" (list "margin" "0"))
          (css/rule
            ".bb-tabs"
            (list
              "display"
              "flex"
              "gap"
              ".35rem"
              "flex-wrap"
              "wrap"
              "margin"
              ".4rem 0 .6rem"
              "align-items"
              "center"))
          (css/rule
            ".bb-tab"
            (list
              "display"
              "inline-block"
              "padding"
              ".28rem .65rem"
              "border"
              "1px solid #000"
              "text-decoration"
              "none"
              "color"
              "inherit"
              "background"
              "var(--panel)"
              "border-radius"
              "2px"))
          (css/rule
            ".bb-tab:hover"
            (list "background" "var(--row-alt)"))
          (css/rule
            ".bb-tab.is-active"
            (list
              "background"
              "var(--ink)"
              "color"
              "var(--panel)"
              "font-weight"
```

```
            "700"))
        (css/rule
          ".bb-edit-form textarea"
          (list
            "width"
            "100%"
            "min-height"
            "14rem"
            "font-family"
            "ui-monospace, monospace"))
        (css/rule
          ".bb-code .bb-edit-form .card"
          (list
            "border"
            "none"
            "padding"
            "0"
            "box-shadow"
            "none"))
        (css/rule
          ".bb-source"
          (list
            "margin"
            "0"
            "max-height"
            "55vh"
            "overflow"
            "auto"
            "background"
            "var(--panel)"
            "border"
            "1px solid #000"
            "padding"
            ".6rem"
            "font-size"
            ".9rem"))
        (css/media
          "min-width:900px"
          (list
            (css/rule
              ".bb-three"
              (list
                "display"
                "grid"
                "grid-template-columns"
                "12rem 14rem 1fr"
                "gap"
                ".75rem"
                "align-items"
                "stretch"
                "min-height"
                "16rem"))
            (css/rule
              ".bb-three .bb-col"
              (list
                "max-height"
                "42vh"
                "overflow"
                "auto"
                "margin"
                "0"
                "display"
                "flex"
                "flex-direction"
                "column"))
            (css/rule
              ".bb-three .bb-col .bb-list"
              (list "flex" "1 1 auto" "overflow" "auto"))
            (css/rule
              ".bb-code .bb-source"
              (list "max-height" "50vh"))))
      ))))
```

```
(define ui/theme-capture
  (lambda
    ()
    "Render the theme-capture stylesheet rules."
    (css/sheet
      (list
        (css/rule
          ".capture-panel"
          (list
            "min-width"
            "min(16rem, calc(100vw - 1rem))"
            "max-width"
            "min(22rem, calc(100vw - 1rem))"
            "padding"
            ".4rem"
            "box-sizing"
            "border-box"))
        (css/rule
          ".capture-nav-form"
          (list
            "display"
            "flex"
            "flex-direction"
            "column"
            "gap"
            ".35rem"
            "margin"
            "0"))
        (css/rule
          ".capture-nav-form input[type=text],.capture-nav-form textarea"
          (list
            "width"
            "100%"
            "box-sizing"
            "border-box"
            "font-size"
            ".9rem"))
        (css/rule
          ".capture-nav-form .capture-body"
          (list "min-height" "4rem"))
        (css/rule
          ".capture-nav-form .capture-row"
          (list
            "display"
            "flex"
            "flex-wrap"
            "wrap"
            "gap"
            ".35rem"
            "align-items"
            "center"))
        (css/rule
          ".capture-nav-form .capture-row button"
          (list
            "width"
            "auto"
            "min-width"
            "4.5rem"
            "margin"
            "0"))
        (css/rule
          ".capture-nav-form #capture-status"
          (list "font-size" ".85rem" "min-height" "1.1em"))
        (css/rule
          ".capture-nav-form .ok"
          (list "color" "var(--ok)" "font-weight" "700"))
        (css/rule
          ".capture-nav-form .err"
          (list "color" "var(--err)" "font-weight" "700"))
        (css/rule
          ".capture-card label"
          (list
            "display"
```

```
            "block"
            "margin"
            ".35rem 0 .15rem"
            "font-weight"
            "700"))
        (css/rule
          ".capture-card .capture-body"
          (list
            "width"
            "100%"
            "box-sizing"
            "border-box"
            "min-height"
            "4.5rem"))
        (css/rule
          ".capture-card .capture-row"
          (list
            "display"
            "flex"
            "flex-wrap"
            "wrap"
            "gap"
            ".5rem"
            "align-items"
            "center"
            "margin-top"
            ".5rem"))
        (css/rule
          ".capture-card .capture-row button"
          (list "width" "auto" "min-width" "6rem"))))))

(define ui/theme-nav
  (lambda
    ()
    "Render the theme navigation stylesheet rules."
    (css/sheet
      (list
        (css/rule
          "nav"
          (list
            "display"
            "flex"
            "flex-wrap"
            "wrap"
            "gap"
            ".35rem .5rem"
            "align-items"
            "center"
            "width"
            "100%"))
        (css/rule "nav>a, .menu-drop" (list "flex" "0 0 auto"))
        (css/rule
          ".menu-drop-btn"
          (list
            "white-space"
            "nowrap"
            "display"
            "inline-flex"
            "align-items"
            "center"
            "justify-content"
            "center"
            "gap"
            ".25rem"))
        (css/media
          "max-width:40rem"
          (list
            (css/rule
              "nav>a, .menu-drop"
              (list
                "flex"
                "1 1 0"
                "min-width"
```

```
                "0"
                "text-align"
                "center"))
            (css/rule
              "nav>a"
              (list "display" "block" "text-align" "center"))
            (css/rule
              ".menu-drop-btn"
              (list
                "display"
                "block"
                "width"
                "100%"
                "text-align"
                "center"
                "box-sizing"
                "border-box"))))
          (css/rule
            ".capture-panel"
            (list
              "min-width"
              "min(16rem, calc(100vw - 1rem))"
              "max-width"
              "min(22rem, calc(100vw - 1rem))"
              "padding"
              ".4rem"
              "box-sizing"
              "border-box"))
          (css/media
            "max-width:40rem"
            (list
              (css/rule ".capture-drop" (list "position" "static"))
              (css/rule
                ".capture-drop > .menu-drop-panel.capture-panel"
                (list
                  "left"
                  ".5rem"
                  "right"
                  ".5rem"
                  "width"
                  "auto"
                  "min-width"
                  "0"
                  "max-width"
                  "none"
                  "box-sizing"
                  "border-box"))))
          (css/rule
            ".capture-nav-form"
            (list
              "display"
              "flex"
              "flex-direction"
              "column"
              "gap"
              ".35rem"
              "margin"
              "0"))
          (css/rule
            ".capture-nav-form input[type=text], .capture-nav-form textarea"
            (list
              "width"
              "100%"
              "box-sizing"
              "border-box"
              "font-size"
              ".9rem"))
          (css/rule
            ".capture-nav-form .capture-body"
            (list "min-height" "4rem"))
          (css/rule
            ".capture-nav-form .capture-row"
            (list
```

```
            "display"
            "flex"
            "flex-wrap"
            "wrap"
            "gap"
            ".35rem"
            "align-items"
            "center"))
          (css/rule
            ".capture-nav-form .capture-row button"
            (list
              "width"
              "auto"
              "min-width"
              "4.5rem"
              "margin"
              "0"))
          (css/rule
            ".capture-nav-form .capture-row a"
            (list
              "display"
              "inline-block"
              "padding"
              ".35rem .45rem"))
          (css/rule
            ".capture-nav-form #capture-status"
            (list "font-size" ".85rem" "min-height" "1.1em"))
          (css/rule
            ".capture-nav-form .ok"
            (list "color" "var(--ok)" "font-weight" "700"))
          (css/rule
            ".capture-nav-form .err"
            (list "color" "var(--err)" "font-weight" "700"))
          (css/rule
            ".menu-drop-panel .menu-drop"
            (list
              "margin"
              "0"
              "display"
              "block"
              "width"
              "100%"
              "box-sizing"
              "border-box"))
          (css/rule
            ".menu-drop-panel .menu-drop > summary"
            (list
              "display"
              "block"
              "width"
              "100%"
              "text-align"
              "left"
              "box-sizing"
              "border-box"))
          (css/rule
            ".menu-drop-panel > .menu-drop-btn"
            (list
              "display"
              "block"
              "width"
              "100%"
              "text-align"
              "left"
              "box-sizing"
              "border-box"))
          (css/rule
            ".menu-drop-panel .menu-drop > summary::after"
            (list "content" "\" ?\""))
          (css/rule
            ".menu-drop-panel .menu-drop-panel"
            (list
              "left"
```

```
            "100%"
            "top"
            "0"
            "right"
            "auto"
            "margin-top"
            "0"))
          (css/rule
            ".menu-drop-panel .menu-drop:last-child > .menu-drop-panel"
            (list "left" "auto" "right" "100%"))
          (css/media
            "max-width:40rem"
            (list
              (css/rule
                ".menu-drop-panel .menu-drop-panel"
                (list
                  "left"
                  "0"
                  "right"
                  "0"
                  "top"
                  "100%"
                  "position"
                  "static"
                  "margin-top"
                  "0"))
              (css/rule
                ".menu-drop-panel .menu-drop > summary::after"
                (list "content" "\" v\""))))))))
```

223 libs/verify/lib.pp

```
;; package verify -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push

(define verify/check
  (lambda
    (name pred)
    "Return a PASS or FAIL result for pred named name."
    (if pred (verify/pass name) (verify/fail name))))

(define verify/count-fails
  (lambda
    (rs)
    "Count the FAIL results in rs."
    (list/foldl
      (lambda (n r) (if (verify/failed? r) (+ n 1) n))
      0
      rs)))

(define verify/fail
  (lambda (name) "Return a FAIL result for name." (list "FAIL" name)))

(define verify/fail-names
  (lambda
    (rs)
    "Collect the names of the FAIL results in rs."
    (list/foldl
      (lambda
        (acc r)
        (if (verify/failed? r) (cons (car (cdr r)) acc) acc))
      (list)
      rs)))

(define verify/failed?
  (lambda (r) "Whether a result is a FAIL." (string-eq (as-str (car r))
    "FAIL")))

(define verify/pass
  (lambda (name) "Return a PASS result for name." (list "PASS" name)))

(define verify/run-suite
  (lambda
    (name)
    "Run the named verify suite, or a FAIL for unknown suites."
    (cond
      ((string-eq name "extract-links")
        (verify/suite-extract-links))
      ((string-eq name "list-hof") (verify/suite-list-hof))
      ((string-eq name "budget-parse")
        (seq (load-library "notes") (verify/suite-budget-parse)))
      ((string-eq name "outline-render")
        (seq
          (load-library "outline")
          (load-library "notes")
          (verify/suite-outline-render)))
      ((string-eq name "upsert-reindex")
        (seq
          (load-library "notes")
          (verify/suite-upsert-reindex)))
      ((string-eq name "extra-actions-contract")
        (seq
          (load-library "notes")
          (verify/suite-extra-actions-contract)))
      ((string-eq name "function-doc")
        (verify/suite-function-doc))
      (else
        (list
          (verify/fail (string-append "unknown-suite:" name)))))))

(define verify/suite-budget-parse
  (lambda
    ()
```

```
    "Check roam link parsing scales on 500 links and stays under budget."
    (let
      ((body
        (list/foldl
          (lambda
            (a i)
            (string-append a "lorem [[p" (as-str i) "]] x "))
          ""
          (list/range 500))))
      (let
        ((n (count (roam/parse-links body)))
          (left (budget-remaining)))
        (list
          (verify/check "parse-links 500" (= n 500))
          (verify/check "parse budget floor" (> left 20000)))))))

(define verify/suite-extra-actions-contract
  (lambda
    ()
    "Check outline/extra-actions stays extract-only."
    (list
      (verify/check
        "extra-actions is extract-only"
        (let
          ((src (function-source "outline/extra-actions")))
          (if
            (string-contains? src "refile/form")
            #f
            (string-contains? src "extract")))))))

(define verify/suite-extract-links
  (lambda
    ()
    "Check link extraction and rendering behavior."
    (list
      (verify/check
        "extract-links three"
        (=
          (count
            (extract-links "see [[a]] and [[b|Bee]] [[media:u|x]]"))
          3))
      (verify/check
        "extract-links zero"
        (= (count (extract-links "no links")) 0))
      (verify/check
        "extract-links unclosed"
        (= (count (extract-links "text [[oops")) 0))
      (verify/check
        "extract-links empty brackets"
        (= (count (extract-links "[[]]")) 0))
      (verify/check
        "render-links wiki"
        (string-contains?
          (render-links "hi [[a|A]]")
          "/notes?slug=a"))
      (verify/check
        "render-links media"
        (string-contains?
          (render-links "[[media:uuid-1|alt]]")
          "note-media")))))

(define verify/suite-list-hof
  (lambda
    ()
    "Check list higher-order functions behave correctly."
    (list
      (verify/check
        "list/map"
        (string-eq
          (str (list/map (lambda (x) (* x 2)) (list 1 2 3)))
          "(2 4 6)"))
      (verify/check
        "list/filter"
```

```
        (string-eq
          (str (list/filter (lambda (x) (= x 2)) (list 1 2 3 2)))
          "(2 2)"))
      (verify/check
        "list/foldl sum"
        (=
          (list/foldl (lambda (a x) (+ a x)) 0 (list 1 2 3 4))
          10)))))

(define verify/suite-names
  (list
    "extract-links"
    "list-hof"
    "budget-parse"
    "outline-render"
    "upsert-reindex"
    "extra-actions-contract"))

(define verify/suite-outline-render
  (lambda
    ()
    "Check outline HTML rendering of many nodes and budget floor."
    (let
      ((kids
        (list/map
          (lambda
            (i)
            (list
              (string-append "n" (as-str i))
              (string-append "T" (as-str i))
              (string-append "body [[x" (as-str i) "]] text")
              ""
              (list)))
          (list/range 25))))
      (let
        ((tree (list "root" "R" "" "" kids)))
        (let
          ((html (outline/html "verify-render" tree #f))
            (left (budget-remaining)))
          (list
            (verify/check
              "outline/html 25 nodes length"
              (> (string-length (as-str html)) 1000))
            (verify/check
              "outline/html wiki rendered"
              (string-contains? (as-str html) "/notes?slug=x0"))
            (verify/check "outline/html budget floor" (> left 5000))
            (verify/check
              "extra-actions no per-node refile/options"
              (not
                (string-contains?
                  (function-source "outline/extra-actions")
                  "refile/options")))))))))

(define verify/suite-upsert-reindex
  (lambda
    ()
    "Check doc upsert reindexes links and stays under budget."
    (let
      ((slug "verify-lib-upsert-tmp")
        (pad
          (list/foldl
            (lambda
              (a i)
              (string-append a "abcdefghijklmnopqrstuvwxyz0123 "))
            ""
            (list/range 100))))
      (let
        ((mk
          (lambda
            (i)
            (list
              (string-append "n" (as-str i))
```

```
              "t"
              (string-append pad " [[L" (as-str i) "]] " pad)
              ""
              (list)))))
        (let
          ((tree
            (list
              "root"
              "V"
              ""
              ""
              (list
                (mk 0)
                (mk 1)
                (mk 2)
                (mk 3)
                (mk 4)
                (mk 5)
                (mk 6)
                (mk 7)
                (mk 8)
                (mk 9)))))
          (let
            ((up
                (notes/doc/upsert
                  slug
                  "V"
                  tree
                  "verify"
                  "note"
                  ""))
              (nlinks
                (count
                  (find-rows (quote note-link) "from_slug" slug 100)))
              (left (budget-remaining)))
            (seq
              (notes/doc/delete slug)
              (list
                (verify/check "upsert ok" (string-eq (as-str up) slug))
                (verify/check "reindex 10 links" (= nlinks 10))
                (verify/check "upsert budget floor" (> left 10000)))))))))

(define verify/suite-function-doc
  (lambda
    ()
    "Check docstrings exist for core library functions."
    (seq
      (load-library "calendar")
      (load-library "contacts")
      (load-library "plan")
      (load-library "todo")
      (load-library "flashcards")
      (list
        (verify/check
          "calendar/add-month has doc"
          (not
            (null? (function-doc "calendar/add-month"))))
        (verify/check
          "calendar/today has doc"
          (not
            (null? (function-doc "calendar/today"))))
        (verify/check
          "calendar/year has doc"
          (not
            (null? (function-doc "calendar/year"))))
        (verify/check
          "contacts/create has doc"
          (not
            (null? (function-doc "contacts/create"))))
        (verify/check
          "contacts/all has doc"
          (not
            (null? (function-doc "contacts/all"))))
```

```
        (verify/check
          "plan/all-rows has doc"
          (not
            (null? (function-doc "plan/all-rows"))))
        (verify/check
          "todo/add has doc"
          (not
            (null? (function-doc "todo/add"))))
        (verify/check
          "todo/list has doc"
          (not
            (null? (function-doc "todo/list"))))
        (verify/check
          "todo/states has doc"
          (not
            (null? (function-doc "todo/states"))))
        (verify/check
          "flash/add has doc"
          (not
            (null? (function-doc "flash/add"))))
        (verify/check
          "flash/blank? has doc"
          (not
            (null? (function-doc "flash/blank?"))))
        (verify/check
          "apropos-docs finds calendar/add-month"
          (> (count (apropos-docs "calendar/add-month")) 0))
        (verify/check
          "apropos-docs finds todo/add"
          (> (count (apropos-docs "todo/add")) 0))))))

(define verify/summary-of
  (lambda
    (rs)
    "Summarize a result list as an all-passed or FAILED string."
    (let
      ((nfails (verify/count-fails rs)))
      (if
        (= nfails 0)
        (string-append
          "ALL PASSED ("
          (as-str (count rs))
          " checks)")
        (string-append "FAILED: " (str (verify/fail-names rs)))))))
```

224 libs/weather/lib.pp

```
;; package weather -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push
;;
;; Keep JS string bodies ASCII-safe (use JS \\u00B0 etc). Nopales used to
;; print non-ASCII via OCaml String.escaped (\\194\\176); the reader treated
;; those as literal digits, so 71 deg F became 71?F in the browser.

(define weather/default-label
  "Cody, WY")

(define weather/default-lat
  "44.52634")

(define weather/default-lon
  "-109.05653")

(define weather/page
  (lambda
    ()
    (html/page
      (list
        (html/h1 "Weather")
        (html/p*
          (html/attr "class" "muted")
          "Browser fetch via Open-Meteo. Location is stored in this browser only.")
        (weather/widget)))))

(define weather/script
  (lambda
    (lat lon label)
    (html/tag
      "script"
      ""
      (string-append
        "(function(){"
        "var root=document.getElementById('weather-widget');"
        "if(!root)return;"
        "var body=document.getElementById('weather-body');"
        "var locEl=document.getElementById('weather-loc-label');"
        "var form=document.getElementById('weather-loc-form');"
        "var input=document.getElementById('weather-city');"
        "var KEY='pp.weather.loc';"
        "var def={lat:"
        (as-str lat)
        ",lon:"
        (as-str lon)
        ",label:"
        (json-emit (as-str label))
        "};"
        "function codes(c){"
        "c=String(c);"
        "return({0:'Clear',1:'Mainly clear',2:'Partly cloudy',3:'Overcast',"
        "45:'Fog',48:'Rime fog',51:'Light drizzle',53:'Drizzle',55:'Heavy drizzle',"
        "61:'Light rain',63:'Rain',65:'Heavy rain',71:'Light snow',73:'Snow',"
        "75:'Heavy snow',80:'Rain showers',81:'Rain showers',82:'Heavy showers',"
        "95:'Thunderstorm'}[c]||'-');"
        "}"
        "function loadLoc(){"
        "try{var s=localStorage.getItem(KEY);if(s)return JSON.parse(s);}catch(e){}"
        "return def;"
        "}"
        "function saveLoc(loc){"
        "try{localStorage.setItem(KEY,JSON.stringify(loc));}catch(e){}"
        "}"
        "function render(loc,cur){"
        "if(locEl)locEl.textContent=loc.label||(loc.lat+', '+loc.lon);"
```

```
        "if(!cur){body.innerHTML='<p class=\"muted\">No data</p>';return;}"
        "var t=Math.round(cur.temperature_2m);"
        "var u=cur.temperature_2m!=null?'\\u00B0F':'';"
        "var w=codes(cur.weather_code);"
        "var
    hum=cur.relative_humidity_2m!=null?(cur.relative_humidity_2m+'%
    rh'):'';"
        "var
    wind=cur.wind_speed_10m!=null?(Math.round(cur.wind_speed_10m)+'
    mph'):'';"
        "body.innerHTML="
        "'<div class=\"wx-row\">'"
        "+'<div class=\"wx-temp\">'+t+u+'</div>'"
        "+'<div><div><strong>'+w+'</strong></div>'"
        "+'<div class=\"wx-meta\">'+[hum,wind].filter(Boolean).join(' |
    ')+'</div></div>'"
        "+'</div>'"
        "+'<div class=\"wx-updated muted\">via Open-Meteo | local browser
    fetch</div>';"
        "}"
        "function fail(msg){"
        "body.innerHTML='<p class=\"wx-err\">'+msg+'</p>';"
        "}"
        "function fetchWx(loc){"
        "body.innerHTML='<p class=\"muted\">Loading...</p>';"
        "if(locEl)locEl.textContent=loc.label||'...';"
        "var url='https://api.open-meteo.com/v1/forecast'"
        "+'?latitude='+encodeURIComponent(loc.lat)"
        "+'&longitude='+encodeURIComponent(loc.lon)"

    "+'&current=temperature_2m,relative_humidity_2m,weather_code,wind_speed_10m'"
        "+'&temperature_unit=fahrenheit&wind_speed_unit=mph&timezone=auto';"
        "fetch(url).then(function(r){if(!r.ok)throw new Error('HTTP
    '+r.status);return r.json();})"
        ".then(function(j){render(loc,j.current||null);})"
        ".catch(function(e){fail('Weather fetch failed:
    '+(e&&e.message?e.message:e));});"
        "}"
        "function geocode(q){"
        "var url='https://geocoding-api.open-meteo.com/v1/search?name='"
        "+encodeURIComponent(q)+'&count=1&language=en&format=json';"
        "return fetch(url).then(function(r){if(!r.ok)throw new Error('HTTP
    '+r.status);return r.json();})"
        ".then(function(j){"
        "if(!j.results||!j.results.length)throw new Error('place not
    found');"
        "var p=j.results[0];"
        "var bits=[p.name,p.admin1,p.country_code].filter(Boolean);"
        "return{lat:p.latitude,lon:p.longitude,label:bits.join(', ')};"
        "});"
        "}"
        "if(form){"
        "form.addEventListener('submit',function(e){"
        "e.preventDefault();"
        "var q=(input&&input.value||'').trim();"
        "if(!q)return;"
        "body.innerHTML='<p class=\"muted\">Looking up...</p>';"
        "geocode(q).then(function(loc){saveLoc(loc);fetchWx(loc);})"

    ".catch(function(err){fail(err&&err.message?err.message:String(err));});"
        "});"
        "}"
        "fetchWx(loadLoc());"
        "})();"))))

(define weather/style
  (lambda
    ()
    (html/tag
      "style"
      ""
      (html/join
        (list
```

```
          ".weather-card
    .wx-row{display:flex;align-items:baseline;gap:.75rem;flex-wrap:wrap}"
          ".weather-card
    .wx-temp{font-size:2rem;font-weight:700;line-height:1}"
          ".weather-card .wx-meta{font-size:.9rem;color:var(--muted,#333)}"
          ".weather-card .wx-loc{font-weight:700;margin:.25rem 0}"
          ".weather-card
    .wx-form{display:flex;gap:.35rem;flex-wrap:wrap;margin-top:.5rem}"
          ".weather-card .wx-form input[type=text]{flex:1 1
    8rem;min-width:0}"
          ".weather-card .wx-form button{width:auto;min-width:4.5rem}"
          ".weather-card .wx-err{color:var(--err,#800000);font-weight:700}"
          ".weather-card
    .wx-updated{font-size:.8rem;color:var(--muted,#555);margin-top:.35rem}")))))

(define weather/widget
  (lambda
    ()
    (html/div
      (html/attrs
        (list
          (html/attr "class" "card weather-card")
          (html/attr "id" "weather-widget")))
      (html/join
        (list
          (html/h2 "Weather")
          (html/div
            (html/attr "class" "wx-loc")
            (html/span
              (html/attr "id" "weather-loc-label")
              (html/text weather/default-label)))
          (html/div
            (html/attr "id" "weather-body")
            (html/p*
              (html/attr "class" "muted")
              "Loading..."))
          (html/tag
            "form"
            (html/attrs
              (list
                (html/attr "id" "weather-loc-form")
                (html/attr "class" "wx-form")
                (html/attr "action" "#")
                (html/attr "method" "get")))
            (html/join
              (list
                (html/input
                  (html/attrs
                    (list
                      (html/attr "type" "text")
                      (html/attr "id" "weather-city")
                      (html/attr "name" "city")
                      (html/attr "placeholder" "City...")
                      (html/attr "autocomplete" "off"))))
                (html/button (html/attr "type" "submit") "Set"))))
          (weather/style)
          (weather/script
            weather/default-lat
            weather/default-lon
            weather/default-label))))))

(define weather/css
  (lambda
    ()
    "Render the weather component stylesheet (extracted from ui/theme)."
    (css/sheet
      (list
        (css/rule
          ".weather-card .wx-row"
          (list
            "display"
            "flex"
            "align-items"
```

```
            "baseline"
            "gap"
            ".75rem"
            "flex-wrap"
            "wrap"))
          (css/rule
            ".weather-card .wx-temp"
            (list
              "font-size"
              "2rem"
              "font-weight"
              "700"
              "line-height"
              "1"))
          (css/rule
            ".weather-card .wx-meta"
            (list "font-size" ".9rem" "color" "var(--muted)"))
          (css/rule
            ".weather-card .wx-loc"
            (list "font-weight" "700" "margin" ".25rem 0"))
          (css/rule
            ".weather-card .wx-form"
            (list
              "display"
              "flex"
              "gap"
              ".35rem"
              "flex-wrap"
              "wrap"
              "margin-top"
              ".5rem"))
          (css/rule
            ".weather-card .wx-form input[type=text]"
            (list "flex" "1 1 8rem" "min-width" "0"))
          (css/rule
            ".weather-card .wx-form button"
            (list "width" "auto" "min-width" "4.5rem"))
          (css/rule
            ".weather-card .wx-err"
            (list "color" "var(--err)" "font-weight" "700"))))))
```

225 libs/weight/lib.pp

```
;; package weight -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push

(define weight/all
  (lambda
    ()
    "Return all weight-log rows sorted by date."
    (list/foldl
      weight/insert-sorted
      ()
      (rows (quote weight-log) 500))))

(define weight/chart-svg
  (lambda
    (rows)
    "Render an SVG line chart of the given weight rows."
    (let
      ((n (weight/len rows)))
      (if
        (< n 1)
        "<div class=\"muted\">No weight entries yet - log one below.</div>"
        (let
          ((rawmin (weight/min-w rows))
            (rawmax (weight/max-w rows)))
          (let
            ((wmin (- rawmin 2.0)) (wmax (+ rawmax 2.0)))
            (let
              ((coords (weight/coords rows 0 n wmin wmax)))
              (string-append
                "<svg viewBox=\"0 0 760 360\" width=\"100%\" height=\"360\"
    "
                "style=\"max-width:760px\" preserveAspectRatio=\"xMidYMid
    meet\">"
                "<rect x=\"60\" y=\"20\" width=\"680\" height=\"300\" "
                "fill=\"#f7f7f9\" stroke=\"#e2e2e8\"/>"
                "<line x1=\"60\" y1=\"20\" x2=\"60\" y2=\"320\"
    stroke=\"#ccc\"/>"
                "<line x1=\"60\" y1=\"320\" x2=\"740\" y2=\"320\"
    stroke=\"#ccc\"/>"
                "<text x=\"54\" y=\"24\" text-anchor=\"end\"
    font-size=\"11\" fill=\"#666\">"
                (as-str (round wmax))
                "</text>"
                "<text x=\"54\" y=\"320\" text-anchor=\"end\"
    font-size=\"11\" fill=\"#666\">"
                (as-str (round wmin))
                "</text>"
                "<polyline points=\""
                (weight/pts-str coords)
                "\" fill=\"none\" stroke=\"#2a7\" stroke-width=\"2\"/>"
                (weight/dots coords)
                "</svg>")))))))

(define weight/coords
  (lambda
    (rows i n wmin wmax)
    "Map weight rows to (x y) chart coordinates."
    (if
      (null? rows)
      (list)
      (let
        ((w (weight/wt-num (car rows))))
        (cons
          (list (weight/x-of i n) (weight/y-of w wmin wmax))
          (weight/coords (cdr rows) (+ i 1) n wmin wmax))))))

(define weight/delete
  (lambda (id) "Delete a weight-log row by id." (delete (quote weight-log)
    id)))

(define weight/dots
```

```
  (lambda
    (coords)
    "Emit SVG circle dots for each chart coordinate."
    (list/foldl
      (lambda
        (acc p)
        (string-append
          acc
          "<circle cx=\""
          (as-str (car p))
          "\" cy=\""
          (as-str (car (cdr p)))
          "\" r=\"3\" fill=\"#2a7\" stroke=\"#fff\" stroke-width=\"1\"/>"))
      ""
      coords)))

(define weight/field
  (lambda
    (fields key)
    "dict-get over a fields JSON string; empty string when absent."
    (let
      ((v (dict-get fields key)))
      (if (null? v) "" (as-str v)))))

(define weight/fields
  (lambda
    (ts weight)
    "Build a weight-log fields JSON string."
    (dict-set*
      "{}"
      (list
        "ts"
        (as-str ts)
        "weight"
        (as-str weight)
        "author"
        (as-str (whoami))))))

(define weight/fmt
  (lambda
    (w)
    "Format a weight value as a trimmed number."
    (let
      ((s (as-str w)))
      (if (blank? s) s (as-str (parse-float s))))))

(define weight/form
  (lambda
    ()
    "Render the log-weight HTML form."
    (html/div
      (html/attr "class" "card")
      (html/join
        (list
          (html/h2 "Log weight")
          (html/form
            "POST"
            "/weight"
            ""
            (html/join
              (list
                (html/label (html/attr "for" "w-date") "Date")
                (html/text-input
                  "date"
                  (html/attr "value" (date-today)))
                (html/label (html/attr "for" "w-weight") "Weight (lbs)")
                (html/text-input
                  "weight"
                  (html/attr "placeholder" "185.4"))
                (html/button (html/attr "type" "submit") "Save")
                (html/hidden "action" "log")))))))))

(define weight/insert-sorted
```

```
  (lambda
    (sorted row)
    "Insert a row into a date-sorted list."
    (if
      (null? sorted)
      (list row)
      (let
        ((d (weight/ts-of row))
          (hd (weight/ts-of (car sorted))))
        (if
          (string<=? d hd)
          (cons row sorted)
          (cons
            (car sorted)
            (weight/insert-sorted (cdr sorted) row)))))))

(define weight/last
  (lambda
    (xs)
    "Return the last element of a non-empty list."
    (if (null? (cdr xs)) (car xs) (weight/last (cdr xs)))))

(define weight/len
  (lambda
    (xs)
    "Return the number of elements in xs."
    (list/foldl (lambda (acc x) (+ acc 1)) 0 xs)))

(define weight/log
  (lambda
    (date weight)
    "Create a weight-log row from a date and weight value."
    (create (quote weight-log) (weight/fields date weight))))

(define weight/max-w
  (lambda
    (rows)
    "Return the maximum weight value across rows; 0.0 when empty."
    (if
      (null? rows)
      0.0
      (list/foldl
        (lambda
          (acc row)
          (let ((w (weight/wt-num row))) (if (> w acc) w acc)))
        (weight/wt-num (car rows))
        (cdr rows)))))

(define weight/min-w
  (lambda
    (rows)
    "Return the minimum weight value across rows; 0.0 when empty."
    (if
      (null? rows)
      0.0
      (list/foldl
        (lambda
          (acc row)
          (let ((w (weight/wt-num row))) (if (< w acc) w acc)))
        (weight/wt-num (car rows))
        (cdr rows)))))

(define weight/page
  (lambda
    ()
    "Handle POST logging; otherwise render the full weight page."
    (if
      (string-eq (request-method) "POST")
      (let
        ((dv (request-param "date"))
          (wv (request-param "weight")))
        (let
          ((d
```

```
              (let
                ((ds (as-str dv)))
                (if (blank? ds) (date-today) ds)))
            (w (as-str wv)))
          (seq (weight/log d w) (redirect "/weight"))))
      (let
        ((rows (weight/all)))
        (html/page
          (list
            (html/h1 "Weight")
            (weight/form)
            (weight/chart-svg rows)
            (weight/summary rows)))))))

(define weight/pts-str
  (lambda
    (coords)
    "Render coordinates as an SVG polyline points string."
    (list/foldl
      (lambda
        (acc p)
        (string-append
          acc
          (as-str (car p))
          ","
          (as-str (car (cdr p)))
          " "))
      ""
      coords)))

(define weight/summary
  (lambda
    (rows)
    "Render a one-line summary of weight entries and latest value."
    (let
      ((n (weight/len rows)))
      (if
        (< n 1)
        ""
        (let
          ((last (weight/last rows)))
          (html/p*
            (html/attr "class" "muted")
            (html/text
              (string-append
                (as-str n)
                " entries  -  latest "
                (weight/ts-of last)
                "  "
                (weight/fmt (weight/wt-of last))
                " lbs")))))))))

(define weight/test-fn
  (lambda (x) "Increment x by one (test helper)." (+ x 1)))

(define weight/ts-of
  (lambda (row) "Return the timestamp field of a weight row." (weight/field
    (cdr row) "ts")))

(define weight/wt-num
  (lambda
    (row)
    "Return a weight row's numeric weight, 0.0 when blank."
    (let
      ((s (weight/wt-of row)))
      (if (blank? s) 0.0 (parse-float s)))))

(define weight/wt-of
  (lambda (row) "Return the weight field of a weight row." (weight/field
    (cdr row) "weight")))

(define weight/x-of
  (lambda
```

```
    (i n)
    "Map a zero-based index to an SVG x position across n points."
    (if
      (<= n 1)
      (+ 60.0 (/ 680.0 2.0))
      (+ 60.0 (* (+ 0.0 i) (/ 680.0 (+ 0.0 (- n 1)))))))

(define weight/y-of
  (lambda
    (w wmin wmax)
    "Map a weight value to an SVG y position within min/max range."
    (+ 20.0 (* (- 1.0 (/ (- w wmin) (- wmax wmin))) 300.0))))
```

226 libs/welcome/lib.pp

```
;; package welcome -- exported from live branch system/main

(define welcome/blank?
  (lambda (s) "True if trimmed s is empty." (string-eq (string-trim (as-str
    s)) "")))

;; Sync: scripts/pp_sync.py pull|push

(define welcome/background
  (lambda
    ()
    "Render the /background about page."
    (layout/page
      (list
        (layout/hero
          "Background"
          "A little about how Pricklypear happened."
          (html/join
            (list
              (layout/cta "/welcome" "Welcome")
              (layout/cta "/roadmap" "Roadmap")
              (layout/cta "/code" "Source")
              (layout/cta "/login" "Log in"))))
        (layout/section
          "A little about me"
          (html/join
            (list
              (html/p
                "I'm super excited about what has emerged with Pricklypear.
    Pricklypear is the result of a kind of long, meandering journey. I see
    a lot of these cactus on my hike. In the middle of the desert, they
    are common, they are prickly, and they bear delicious fruit. I've also
    been told you can use them to harvest water kefir yeast, but I've yet
    to confirm this.")
              (html/p
                "I quit my development job and now work at a dentist office
    and a restaurant. But I just can't seem to stop coding. For a while I
    was trying to make the ultimate \"CRUDmonkey\" platform. Basically, I
    spent a lot of time with half baked designs trying to reinvent web
    frameworks. Then, I became totally taken with the idea of a tree-based
    structure editor.")
              (html/p
                "My background is largely as a python CRUDmonkey, hence my
    interest in reinventing frameworks. I did a lot of data processing
    elbow grease work. I was CTO of a 13 person startup for a while. When
    I left that job I was pretty burnt out. I didn't think I enjoyed
    programming, but I still was obsessed with the idea of maybe I could
    make it easier.")
              (html/p
                "Then I found OCaml. Functional programming did it for me,
    man. It was like learning to program all over again. I started
    learning about all the things I had basically ignored as a python guy.
    I learned about compilers, low level languages. I took a freelance gig
    doing static analysis on IBM HLASM assembly language.")
              (html/p
                "The only thing I really wanted from code? I wanted to
    build my dream zettelkasten. I just wanted a good notes app for my
    bible study. First I built a bible mode in Obsidian with markdown
    files. But as soon as I tried to get Strong's Concordance in there, I
    absolutely hit the limits of what Obsidian could do. Then, I decided
    to switch over to elisp. LLMs were barely functional at this time, and
    they were really bad at lisp. This was the \"copy and paste from
    ChatGPT\" era. My emacs / org-roam zettelkasten worked amazingly.
    Honestly, I love emacs and I love org-roam. Absolutely beautiful
    pieces of software. I also love the sense of calm that I get with
    emacs software. I pulled up this bible app in emacs 4 years later and
    it just... worked. Amazingly. None of the constant churn of the JS
    ecosystem.")
              (html/p
                "OK, so then I got into OCaml, and I was very anti LLM.
    They didn't work, they made you dumb, they were a scam, yada yada
```

```
      yada. So I built myself a bible app, all by hand, in OCaml / Dream. I
      had a great time with this. I felt like I probably learned more about
      Dream than I did OCaml, but that's how I felt when I first learned
      Python also.")
39              (html/p
40                "Then I took a gig with some guys who were doing IBM
      mainframe LLM stuff. I started coding in TypeScript (which i had never
      really used), right around the time when the LLMs got really, really
      good. Like November of 2025? I saw a massive increase in their ability
      to compose and build code.")
41              (html/p
42                "Then I went deep. I started building harnesses,
      orchestrators, I did the full LLM psychosis thing. I eventually
      settled on my Borge architecture. Without going too deeply into it,
      it's a DSL for defining spec and a series of CLI tools to get agents
      to implement, review, and build out that spec. I'm not sure it's any
      more efficient but it seems to work super well for me.")
43              (html/p
44                "So then one day I had the brainwave. Can I make an
      interpreted lisp where the functions are stored in postgres? The agent
      assured me I was a genius yet again, but I had a good feeling about
      this one. I had the LLM draw out my Borge spec, and then started a
      ralph loop. I think it went for like, 25 iterations? It ran for 4 days
      and it crashed a couple times and I think I maxed out three different
      providers. It used a lot of tokens. I got an email from one of my LLM
      providers the next day letting me know they were tripling their
      costs.")
45              (html/p
46                "When I started playing with it, I actually couldn't figure
      out how to use it at first. I had the agent draw up a intro guide.
      Then I realized that not only did it work, I could write language
      functions to the database, but that also those functions were working
      as web endpoints. I realized I had a whole framework.")
47              (html/p
48                "It worked so well. I built a couple apps on top, it was
      effortless. I plugged an LLM into the system. Once I prompted it to
      realize Pricklypear was not a cactus in this scenario, it completely
      understood the harness it was in and it just... started writing
      functions. I couldn't believe it. Then I gave my local LLM access to
      the full REPL, and I realized I wouldn't ever have to build an API or
      CLI interface for agents ever again.")
49              (html/p
50                "I'm delighted by this. I have wanted a Smalltalk type
      environment since I learned about what that was. But by incredible,
      happy accident, I also got my brand spanking new web framework, and I
      also got my tree based editor. I honestly feel, I don't need any more
      software. I'd be good. If I had my druthers, I would just work on this
      in private and read Functional Programming papers and Alonzo Church in
      my hammock for the next 5 years while I dogfood this thing. But, you
      know, \"building in public\" and all that.")
51              (html/p
52                "I'm hoping some people are as thrilled and inspired by
      this as I am. Maybe the code is useful, or maybe just hearing about
      the concept is enough to inspire people. Or maybe, and this may be
      more likely, nobody will give a crap. I don't know where LLMs will go
      in the future, but I am really happy right here."))))
53          (html/div
54            (html/attr "class" "layout-foot")
55            (html/join
56              (list
57                (html/a "/roadmap" (html/text "roadmap"))
58                (html/text " ? ")
59                (html/a "/welcome" (html/text "welcome")))))))))
60
61  (define welcome/book
62    (lambda
63      ()
64      "Render the /book explainer page for the literate code PDF."
65      (layout/page
66        (list
67          (layout/hero
68            "The Pricklypear book"
```

```
          "A paginated, linearly ordered rendering of the spec and code -- literate programming without a WEB weave."
          (html/join
            (list
              (layout/cta "/book.pdf" "Download PDF")
              (layout/cta "/src.tgz" "Source tarball")
              (layout/cta "/code" "Browse online")
              (layout/cta "/welcome" "Back to welcome"))))
        (layout/section
          "How to read it"
          (html/join
            (list
              (html/p
                "The book is produced by borge book print. Order is four-tier: root .borg, chapter .borg files, .mli signatures, then .ml implementations (dependency-ranked). Line numbers and a wide margin are for hand notes.")
              (html/p
                "It is not a tangle/weave of prose-into-code. The .borg files remain the source of truth; the PDF is a read-only projection so you can leave the screen.")
              (html/p
                "If the PDF is missing on this host, the deploy step that runs scripts/public_artifacts.sh has not installed it yet -- the code browser and tarball still cover the tree."))))
        (html/div
          (html/attr "class" "layout-foot")
          (html/a "/welcome" (html/text "? welcome")))))))

(define welcome/guestbook
  (lambda
    ()
    "Route handler for /guestbook: record POSTs or show the form."
    (if
      (string-eq (request-method) "POST")
      (let
        ((name (request-param "name"))
          (site (request-param "site"))
          (message (request-param "message")))
        (if
          (or
            (welcome/blank? (string-trim (as-str name)))
            (welcome/blank? (string-trim (as-str message))))
          (welcome/guestbook-form "" "Need a name and a message.")
          (seq
            (with-public-write
              (lambda () (welcome/guestbook-record name site message)))
            (welcome/guestbook-form
              "Thanks for signing! ?"
              ""))))
      (welcome/guestbook-form "" ""))))

(define welcome/guestbook-entries
  (lambda
    ()
    "Render the list of guestbook entries as an HTML list."
    (let
      ((rs (rows (quote guestbook_entry) 200)))
      (if
        (null? rs)
        (html/p
          "No one has signed the guestbook yet. Be the first!")
        (html/ul
          (html/attr "class" "gb-list")
          (html/join (list/map welcome/guestbook-row-html rs)))))))

(define welcome/guestbook-fields
  (lambda
    (name site message)
    "Build the guestbook_entry fields JSON string from trimmed inputs."
    (dict-set*
      "{}"
      (list
```

```
        "name"
        (string-trim (as-str name))
        "site"
        (string-trim (as-str site))
        "message"
        (string-trim (as-str message))))))

(define welcome/guestbook-form
  (lambda
    (msg err)
    "Render the guestbook form page with msg/err notices and entries."
    (layout/page
      (list
        (layout/hero
          "Guestbook"
          "Sign my guestbook!! Leave a note. Tell me what you think. ?"
          (html/join
            (list
              (layout/cta "/welcome" "Welcome")
              (layout/cta "/code" "Source")
              (layout/cta "/login" "Log in"))))
        (if
          (welcome/blank? err)
          ""
          (html/p* (html/attr "class" "err") (html/text err)))
        (if
          (welcome/blank? msg)
          ""
          (html/p* (html/attr "class" "ok") (html/text msg)))
        (layout/section
          "Sign it"
          (html/form
            "POST"
            "/guestbook"
            (html/attr "class" "gb-form")
            (html/join
              (list
                (html/p "Your name")
                (html/text-input
                  "name"
                  (html/attrs
                    (list
                      (html/attr "required" "required")
                      (html/attr "placeholder" "anon")
                      (html/attr "maxlength" "60"))))
                (html/p "Your site (optional)")
                (html/text-input
                  "site"
                  (html/attrs
                    (list
                      (html/attr "placeholder" "https://your.site")
                      (html/attr "maxlength" "120"))))
                (html/p "Message")
                (html/textarea
                  "message"
                  (html/attrs
                    (list
                      (html/attr "rows" "3")
                      (html/attr "required" "required")
                      (html/attr "placeholder" "say hi")
                      (html/attr "maxlength" "500")))
                  "")
                (html/hidden "source" "web")
                (html/div
                  ""
                  (html/button
                    (html/attrs (list (html/attr "type" "submit")))
                    "Sign it!"))))))
        (layout/section "Entries" (welcome/guestbook-entries))
        (html/div
          (html/attr "class" "layout-foot")
          (html/a "/welcome" (html/text "welcome")))))))

```

```
(define welcome/guestbook-record
  (lambda
    (name site message)
    "Create a new guestbook_entry row from name, site, and message."
    (create
      (quote guestbook_entry)
      (welcome/guestbook-fields name site message))))

(define welcome/guestbook-row-html
  (lambda
    (row)
    "Render a single guestbook entry row as an HTML <li>."
    (let
      ((f (cdr row)))
      (let
        ((name (as-str (dict-get f "name")))
          (site (as-str (dict-get f "site")))
          (message (as-str (dict-get f "message"))))
        (html/li
          (html/attr "class" "gb-entry")
          (html/join
            (list
              (html/tag
                "span"
                (html/attr "class" "gb-name")
                (html/text name))
              (if
                (welcome/blank? site)
                ""
                (html/join
                  (list
                    " "
                    (html/tag
                      "a"
                      (html/attrs
                        (list
                          (html/attr "class" "gb-site")
                          (html/attr "href" site)))
                      (html/text site)))))
              (html/tag
                "div"
                (html/attr "class" "gb-msg")
                (html/text message)))))))))

(define welcome/page
  (lambda
    ()
    "Render the /welcome landing page."
    (layout/page
      (list
        (layout/hero
          "Pricklypear"
          "What if WordPress was a Lisp Machine? (Nobody has asked this question)"
          (html/join
            (list
              (layout/cta "/code" "Explore the source")
              (layout/cta "/book.pdf" "Download the book (PDF)")
              (layout/cta "/agent.txt" "Agent instructions")
              (layout/cta "/background" "Background")
              (layout/cta "/roadmap" "Roadmap")
              (layout/cta "/signup" "Email signup")
              (layout/cta "/guestbook" "Sign guestbook")
              (layout/cta "/login" "Log in"))))
        (layout/section
          "What is Pricklypear?"
          (html/join
            (list
              (html/p
                "Pricklypear is a multi-user Lisp operating system. One unix process is the whole OS surface -- HTTP, REPL, eval, auth. Postgres is the filesystem: every function, type, route, and data row lives there as rows. The image is ephemeral and rebuilds from the
```

```
    store.")
276           (html/p
277             "You don't deploy Nopales code. You save a row. Change a
    function and every route that calls it picks it up immediately -- no
    compile step, no restart.")))
278       (layout/section
279         "What is Nopales?"
280         (html/join
281           (list
282             (html/p
283               "Nopales is the language. It's a small Scheme-family
    Lisp-1. It's homoiconic: the AST is the source, and function
    definitions persist in Postgres as JSONB. Editing the AST is editing
    the code.")
284             (html/p
285               "Here's a handler function. The browser hits /hello and
    this runs.")
286             (html/pre
287               (html/text
288                 "(define hi-page\n  (lambda () (render-html
    \"<h1>Hello</h1>\")))"))
289             (html/p
290               "You don't need to restart to see this page. There's no
    translation layer. Postgres just stores the direct AST of your sexp,
    serialized as JSONB:")
291             (html/tag
292               "table"
293               (html/attr "class" "ast-table")
294               (html/join
295                 (list
296                   (html/tag
297                     "tr"
298                     ""
299                     (html/join
300                       (list
301                         (html/tag "th" "" (html/text "column"))
302                         (html/tag "th" "" (html/text "value")))))
303                   (html/tag
304                     "tr"
305                     ""
306                     (html/join
307                       (list
308                         (html/tag "td" "" (html/text "name"))
309                         (html/tag "td" "" (html/code "hi-page")))))
310                   (html/tag
311                     "tr"
312                     ""
313                     (html/join
314                       (list
315                         (html/tag "td" "" (html/text "body"))
316                         (html/tag
317                           "td"
318                           ""
319                           (html/pre
320                             (html/text
321                               "{\"lambda\":[[],\n
    {\"app\":[{\"var\":\"render-html\"},\n
    {\"lit\":{\"str\":\"<h1>Hello</h1>\"}}]}]}"))))))
322                   (html/tag
323                     "tr"
324                     ""
325                     (html/join
326                       (list
327                         (html/tag "td" "" (html/text "doc"))
328                         (html/tag "td" "" (html/text "Render the hello
    page."))))))
329                   (html/tag
330                     "tr"
331                     ""
332                     (html/join
333                       (list
334                         (html/tag "td" "" (html/text "author"))
335                         (html/tag "td" "" (html/text "admin")))))))))
```

```
336             (html/p
337               "It has closures, lambda, let, quasiquote macros, pattern
    matching, and a frozen core. Dynamic typing, impure functional,
    effects-inferred. Two kinds of function: data builders you call from
    the REPL, and HTTP handlers you bind to a URL.")
338             (html/p*
339               ""
340               (html/join
341                 (list
342                   "A handler is just "
343                   (html/code "(route \"/hello\" \"hi-page\")")
344                   " -- where "
345                   (html/code "hi-page")
346                   " is a function you defined. The browser hits /hello;
    the REPL calls hi-page.")))))
347       (layout/cards
348         (list
349           (layout/card
350             "Interpreted runtime"
351             "Agent skills and MCP are 2024 era coding dude. Here in
    Pricklypear we directly access the REPL. POST one Lisp expression to
    /api/eval and get a value back. Pure prims (+, car, string-append) run
    in the interpreter; DB prims (get, create, rows, route, fork, merge)
    read and write real Postgres rows. No build, no deploy, no second
    language. You can also mirror the whole language and run the
    interpreter locally.")
352           (layout/card
353             "HTML & CSS primitives"
354             "Sexp wrappers for the whole tree. html/div, html/a, html/p,
    html/attr. You build pages in Lisp, not string templates. A tiny CSS
    DSL (css/rule, css/sheet, css/media) emits a real stylesheet. This
    page is styled that way.")
355           (layout/card
356             "htmx, server-evaluated"
357             "Route handlers return htmx fragments. A server-side swap
    simulator (the hypermedia harness) lets agents can verify a flow
    without a browser. You don't have to do a goofy Chrome / screenshot
    workflow.")
358           (layout/card
359             "Local-first evaluation"
360             "The same interpreter runs three ways: in the image against
    Postgres, over HTTP via /api/eval (local compute, remote data), and
    against a local lens offline that grafts back on reconnect. One
    language, one seam, three modes.")))
361       (layout/section
362         "3 hour tour"
363         (html/join
364           (list
365             (html/p
366               "After you log in (or from scripts/repl.sh against a local
    image):")
367             (html/tag
368               "ol"
369               ""
370               (html/join
371                 (list
372                   (html/li "" (html/code "scripts/repl.sh"))
373                   (html/li
374                     ""
375                     (html/code
376                       "(define hi-page (lambda () (render-html
    \"<h1>Hello</h1>\")))"))
377                   (html/li
378                     ""
379                     (html/code "(route \"/hello\" \"hi-page\")"))
380                   (html/li "" "open /hello in the browser")
381                   (html/li "" "edit at /edit/hi-page")
382                   (html/li
383                     ""
384                     (html/join
385                       (list
386                         "optional: "
387                         (html/code
```

```
                          "(propose-function \"double\" \"lambda of int x -> (* x 2)\")")))))))))
    (layout/section
      "Agents: no special API"
      (html/join
        (list
          (html/p
            "Remote agents (Agora, pi, curl on a phone) do not need a custom REST surface. Fetch the public bootstrap doc, take a username/password from a human once, then POST Lisp to /api/eval.")
          (html/p*
            ""
            (html/join
              (list
                "Machine contract: "
                (html/a
                  "/.well-known/agent.json"
                  (html/code "/.well-known/agent.json"))
                " ? Prose: "
                (html/a "/agent.txt" (html/code "/agent.txt"))
                ". No secrets in either document.")))
          (html/div
            ""
            (html/join
              (list
                (layout/cta "/agent.txt" "Read agent.txt")
                " "
                (layout/cta "/.well-known/agent.json" "agent.json")))))))
    (layout/section
      "Harness, agent, habitat"
      (html/join
        (list
          (html/p
            "Has anybody noticed there's a third thing missing? I call it The Third Substrate if I'm trying to be fancy. Your harness writes code. Your agent manages tasks and processes. But there's nowhere to deploy your code. Now your poor OpenClaw is a TypeScript DevOps engineer")
          (html/p
            "Pricklypear is the third thing. It's an living app and a memory. If you want you can write all your agent memory to it.")
          (html/p
            "The agent calls it a habitat. I kind of think that name sucks, but oh well. Maybe its natural habitat is living inside its own Lisp. I think Cloudflare is trying to solve this problem with some insane, poorly designed abstraction. (like everything they touch)")
          (html/p
            "I kind of think a habitat has to be a Lisp. I think you could build it in CL or Clojure. But there are big advantages to having an interpreted Lisp. I'd actually like to see a CL or Clojure habitat. My agents say you could build one in less than 10000 LoC. That's about a quarter of the size of Pricklypear. But then I built an interpreter, too.")))))
    (layout/section
      "Get in, Loser. We're a cactus now."
      (html/join
        (list
          (html/p
            "Browse the source or download the book without an account. To run code in the image, log in -- workspace home, Functions, Pages, Data, and the REPL stay private.")
          (html/div
            ""
            (html/join
              (list
                (layout/cta "/code" "Code browser")
                " "
                (layout/cta "/book.pdf" "Book PDF")
                " "
                (layout/cta "/agent.txt" "Agent instructions")
                " "
                (layout/cta "/login" "Sign in")))))))
```

```
        (html/div
          (html/attr "class" "layout-foot")
          (html/join
            (list
              (html/text
                "Pricklypear ? hosted release ? ")
              (html/a "/background" (html/text "background"))
              (html/text " ? ")
              (html/a "/roadmap" (html/text "roadmap"))
              (html/text " ? ")
              (html/a "/signup" (html/text "signup"))
              (html/text " ? ")
              (html/a "/guestbook" (html/text "guestbook"))
              (html/text " ? ")
              (html/a "/agent.txt" (html/text "agent instructions"))
              (html/text " ? ")
              (html/a "/about" (html/text "concepts (after login)")))))))))

(define welcome/roadmap
  (lambda
    ()
    "Render the /roadmap page."
    (layout/page
      (list
        (layout/hero
          "Why this, and where next"
          "Lisp machines, agents, and a weird little platform for apps."
          (html/join
            (list
              (layout/cta "/welcome" "Welcome")
              (layout/cta "/background" "Background")
              (layout/cta "/book.pdf" "Book (PDF)")
              (layout/cta "/login" "Log in"))))
        (layout/section
          "The case for Pricklypear"
          (html/join
            (list
              (html/p
                "Alright - I'll talk a little bit about what I think
    Pricklypear is good at, and where I see it going")
              (html/p
                "A lot of people have talked about how incredible Lisp is.
    Personally, I'm not totally sold. Don't get me wrong, lisp is
    wonderful. But I think for larger projects you just need static
    typing. I didn't like static typing for a long time, I totally get it.
    But for medium or big projects, I just don't know how you do big
    refactors without it.")
              (html/p
                "That said, I think Lisp is like Ruby a lot, it's a
    beautiful and expressive language. So this is a lisp machine. And I
    think it's good for the same types of programs that Ruby is good at.
    So I did my best to clone Basecamp, hah.")
              (html/p
                "Having an agent work with a REPL is incredible. I mean,
    it's spectacular. They can define functions. They can read and execute
    individual functions. They can build functions inside out or bottom
    up. They can build them top down. They can SSH in remotely and access
    the REPL while they're developing. It's pretty tight.")
              (html/p
                "I don't think any other architecture can match Pricklypear
    for building web apps, and I honestly don't think it's even close. All
    of those LLM marketing idiots telling you about on-demand self healing
    software? Hey, this is it. It's right here. I don't think you can do
    this with any other stack? Maybe like erlang or something.")
              (html/p
                "The future I see for Pricklypear or something like it is
    enabling LLMs to build on demand software for small and medium
    businesses. I don't even think you need to have a Pricklypear guy on
    staff? I mean you could, it'd probably help. But I think literally
    users can ask for features and they will be deployed. I've had some
    trouble getting the built-in agent to build out like a whole app from
    start to finish, but it can certainly add features. And that loop is
    closing fast.")
```

```
            (html/p
              "I'm biased and deep in AI psychosis, but I'm telling you this is a much bigger leap than OpenClaw. Oh, you put your little crab agent inside a chat bot? Guess what? Mine lives inside a language which he is developing himself, and that language is also a fully baked web app. (Sorry, OpenClaw is brilliant, but I had to. You'll understand when you login the first time.)")
            (html/p
              "Let's talk roadmap. I'm going to dogfood this thing for a while. Maybe some other people will do something similar. I don't really feel any obligation to build anything for anybody else. Get your own! But, I would like to get this thing to the point where it's functioning as a self contained unit, and like, my dad could hop on it and he wouldn't have to learn anything about Lisp Machines in order to use it. I think this is feasible. It's just a weird little platform to build apps on.")
            (html/p
              "I'm hoping to get those apps fleshed out a bit. This is my new homelab center. I'm moving my life onto this thing. It's too good not to. So I'd like for those apps to feel really really good. Currently, they do not feel great. They are not cozy. I want calm, and I want cozy. Even though I can change these apps every day - I want to run them without changes for 5 years.")
            (html/p
              "I initially spec'd this out to run on MirageOS, but we've scrapped that for now. I would like to make this happen eventually. For now, I remain an OpenBSD guy. If you want Linux compatibility, make your agent do it. (I use Arch for development, btw). So I think self contained image is a priority.")
            (html/p
              "I would like to get some academic grounding on building this Lisp out. I don't know nothin about languages. Seriously. I just added floats to this thing. Also I think the language itself is called Nopales, but that isn't reflected anywhere right now. There is some sort of rudimentary effects system. I have no idea how it works. I don't really understand the subtleties of async, and the OCaml landscape is pretty confusing.")
            (html/p
              "On a similar note, I'd like to get some education on formal proofs in software. I think that it is absolutely work spending some time trying to add formal verification to the OCaml outershell of Pricklypear. Lisp gives a great surface for the LLM on top, and I'm perfectly happy with a little jank there, but the core really should be incredibly stable.")
            (html/p
              "I want to really polish up the node editor and the code editor. It should be a first class experience. Keyboard shortcuts, node navigation. I want it to feel as good as vim.")
            (html/p
              "I'd like to see local-first LLMs, too. You really don't need much here. And the way it's designed, it would plug right into a local agent. That local agent really wants to generate his own lisp functions using a recursive bottom up function generator call!")
            (html/p
              "OK, that's enough for now. This is about as much sharing as I do, generally speaking. I guess I could get on Twitter and try to shill this thing but I think I'd rather just build it instead."))))
          (html/div
            (html/attr "class" "layout-foot")
            (html/join
              (list
                (html/a "/background" (html/text "background"))
                (html/text " ? ")
                (html/a "/welcome" (html/text "welcome")))))))))

;; Public release-face routes (hard-public in image/lib/http.ml).
;; Must be public-route so anonymous GET/POST resolve the handlers.
(public-route "/welcome" "welcome/page")
(public-route "/background" "welcome/background")
(public-route "/roadmap" "welcome/roadmap")
(public-route "/guestbook" "welcome/guestbook")
```

227 libs/wiki/lib.pp

```
;; package wiki -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push

(define wiki/apply-entities
  (lambda
    (ents source-slug touched)
    (if
      (null? ents)
      touched
      (wiki/apply-entities
        (cdr ents)
        source-slug
        (wiki/apply-entity (car ents) source-slug touched)))))

(define wiki/apply-entity
  (lambda
    (ent source-slug touched)
    (let
      ((slug (as-str (dict-get ent "slug")))
        (title (as-str (dict-get ent "title")))
        (body (as-str (dict-get ent "body")))
        (kind (as-str (dict-get ent "kind"))))
      ;; let* (not parallel let): t's value references sibling s bound
      ;; in the same group -- parallel `let` evaluates bindings in the outer
      ;; env, so s was unbound when title was empty (threw `unbound: s`).
      (let*
        ((s (if (string-eq slug "") (wiki/slugify title) slug))
          (t (if (string-eq title "") s title))
          (b
            (if
              (string-eq body "")
              (string-append "From [[" source-slug "]].")
              (string-append
                body
                "\\n\\nSee also [["
                source-slug
                "]].")))
          (k
            (if
              (if
                (string-eq kind "entity")
                #t
                (string-eq kind "concept"))
              kind
              "entity")))
        (seq
          (wiki/upsert-page s t b k)
          (append touched (list s)))))))

(define wiki/ask
  (lambda
    (prompt)
    (let
      ((sid
        (agent-session-new
          "You help maintain a personal knowledge wiki. Reply with plain
    text or JSON as asked. Do not call tools."
          #f)))
      (agent-say sid (as-str prompt)))))

(define wiki/corpus-index
  (lambda
    ()
    (list/foldl
      (lambda
        (acc row)
        (let
          ((f (cdr row)))
          (let
            ((slug (as-str (dict-get f "slug")))
              (title (as-str (dict-get f "title")))
```

```
              (kind (as-str (dict-get f "kind"))))
            (string-append
              acc
              "- ["
              kind
              "] "
              title
              " ([["
              slug
              "]])\\n"))))
      ""
      (notes/doc/index-list))))

(define wiki/default-schema
  (string-append
    "You maintain a personal knowledge wiki of outline notes.\\n"
    "Conventions:\\n"
    "- Pages are notes with kind in {source, entity, concept, query,
     synthesis, schema}.\\n"
    "- Cross-links use [[slug]] or [[slug|label]].\\n"
    "- Prefer updating existing entity/concept pages over creating
     duplicates.\\n"
    "- Keep titles short; put detail in outline bodies.\\n"
    "- When asked for JSON, reply with ONE JSON object only (no markdown
     fences).\\n"))

(define wiki/events
  (lambda () (rows (quote wiki-event) 100)))

(define wiki/extract-json
  (lambda
    (s)
    (let
      ((t (as-str s)))
      (wiki/extract-json-scan t 0 () ()))))

(define wiki/extract-json-scan
  (lambda
    (s i start end)
    (if
      (>= i (string-length s))
      (if
        (if (null? start) #t (null? end))
        "{}"
        (substring s start (+ (- end start) 1)))
      (let
        ((c (substring s i 1)))
        (if
          (string-eq c "{")
          (wiki/extract-json-scan
            s
            (+ i 1)
            (if (null? start) i start)
            end)
          (if
            (string-eq c "}")
            (wiki/extract-json-scan s (+ i 1) start i)
            (wiki/extract-json-scan s (+ i 1) start end)))))))

(define wiki/ingest-from-source
  (lambda
    (source-text)
    (let
      ((src-slug
        (string-append "src-" (as-str (count (notes/doc/list))))))
      (seq
        (wiki/upsert-page
          src-slug
          (string-append "Source " src-slug)
          (as-str source-text)
          "source")
        (let
          ((raw
```

```
            (wiki/ask (wiki/ingest-prompt source-text src-slug))))
          (let
            ((plan (wiki/parse-plan raw)))
            (let
              ((summary (as-str (dict-get plan "summary")))
                (ents (wiki/plan-list plan "entities")))
              (seq
                (when
                  (not (string-eq summary ""))
                  (wiki/upsert-page
                    src-slug
                    (string-append "Source " src-slug)
                    (string-append
                      summary
                      "\\n\\n---\\n"
                      (as-str source-text))
                    "source"))
                (let
                  ((touched
                    (wiki/apply-entities ents src-slug (list src-slug))))
                  (seq
                    (wiki/log-event
                      "ingest"
                      "wiki/ingest-from-source"
                      src-slug
                      (string-append
                        "touched "
                        (as-str (count touched))
                        " pages"))
                    (list src-slug touched raw)))))))))

(define wiki/ingest-prompt
  (lambda
    (source-text source-slug)
    (string-append
      (wiki/schema)
      "\\n\\nIngest this source into the wiki.\\n"
      "Source slug: "
      source-slug
      "\\n"
      "Source text:\\n"
      (as-str source-text)
      "\\n\\n"
      "Reply with ONE JSON object only, shape:\\n"
    "{\"summary\":\"...\",\"entities\":[{\"slug\":\"...\",\"title\":\"...\","
      "\"body\":\"... with [[links]]\",\"kind\":\"entity|concept\"}]}\\n"
      "Create 1-5 entities/concepts. Bodies may include [[slug]]
    links.\\n")))

(define wiki/lint
  (lambda
    ()
    (let
      ((report (wiki/ask (wiki/lint-prompt))))
      (seq
        (wiki/log-event "lint" "wiki/lint" "" "lint pass")
        report))))

(define wiki/lint-prompt
  (lambda
    ()
    (string-append
      (wiki/schema)
      "\\n\\nCorpus index:\\n"
      (wiki/corpus-index)
      "\\n\\nLint this wiki. Report:\\n"
      "1. Orphan pages (no inbound links)\\n"
      "2. Missing pages (mentioned via [[slug]] but absent)\\n"
      "3. Contradictions or stale claims\\n"
      "4. Suggested new sources or questions\\n"
      "Be concise. Use bullet lists.\\n")))
```

```
(define wiki/log-event
  (lambda
    (kind op slug summary)
    (create
      (quote wiki-event)
      (dict-set*
        "{}"
        (list
          "kind"
          (as-str kind)
          "op"
          (as-str op)
          "slug"
          (as-str slug)
          "ts"
          (wiki/now)
          "summary"
          (as-str summary))))))

(define wiki/now
  (lambda () (as-str (date-today))))

(define wiki/parse-plan
  (lambda (text) (json-parse (wiki/extract-json text))))

(define wiki/plan-list
  (lambda
    (plan key)
    (let
      ((v (dict-get plan key)))
      (if
        (null? v)
        (list)
        (let
          ((parsed (json-parse (as-str v))))
          (if
            (pair? parsed)
            parsed
            (if (null? parsed) (list) (list)))))))))

(define wiki/prune-events
  (lambda
    (days)
    (let
      ((cutoff
        (date-add-days
          (date-today)
          (- 0 (if (null? days) 30 (parse-int (as-str days)))))))
      (list/foldl
        (lambda
          (n row)
          (let
            ((f (cdr row)))
            (let
              ((ts (as-str (dict-get f "ts"))))
              (if
                (if
                  (string-eq ts "")
                  #f
                  (< (date-diff-days ts cutoff) 0))
                (seq (delete (quote wiki-event) (car row)) (+ n 1))
                n))))
        0
        (wiki/events))))))

(define wiki/query
  (lambda
    (question)
    (let
      ((answer (wiki/ask (wiki/query-prompt question))))
      (seq
        (wiki/log-event
          "query"
```

```
          "wiki/query"
          ""
          (substring
            (as-str question)
            0
            (if
              (< (string-length (as-str question)) 80)
              (string-length (as-str question))
              80)))
        answer))))

(define wiki/query-prompt
  (lambda
    (question)
    (string-append
      (wiki/schema)
      "\\n\\nCorpus index:\\n"
      (wiki/corpus-index)
      "\\n\\nQuestion: "
      (as-str question)
      "\\n\\n"
      "Answer using the corpus. Cite pages as [[slug]]. "
      "If you create a synthesis, say so. Plain prose is fine.\\n")))

(define wiki/schema
  (lambda
    ()
    (let
      ((n (notes/doc/get "wiki-schema")))
      (if
        (null? n)
        (seq
          (notes/doc/upsert-in
            "wiki-schema"
            "Wiki Schema"
            (outline/node*
              "root"
              "Wiki Schema"
              wiki/default-schema
              ""
              (list))
            "schema"
            "schema"
            ""
            "wiki")
          wiki/default-schema)
        (outline/body (notes/doc/outline n))))))

(define wiki/slugify
  (lambda
    (title)
    (let
      ((t (string-trim (as-str title))))
      (if
        (string-eq t "")
        (string-append
          "wiki-"
          (as-str (count (notes/doc/list))))
        (string-append
          "w-"
          (substring
            t
            0
            (if (< (string-length t) 32) (string-length t) 32)))))))

(define wiki/upsert-page
  (lambda
    (slug title body kind)
    (let
      ((prev (notes/doc/get slug)))
      (let
        ((tree
            (if
```

```
            (null? prev)
            (outline/node* "root" title body "" (list))
            (outline/set-body
              (outline/set-title (notes/doc/outline prev) title)
              body)))
        (tags (if (null? prev) kind (notes/doc/tags prev)))
        (day (if (null? prev) "" (notes/doc/day prev)))
        (sc
          (if
            (null? prev)
            "1"
            (let
              ((raw
                (dict-get (cdr (notes/doc/row slug)) "source_count")))
              (if
                (null? raw)
                "1"
                (as-str (+ 1 (parse-int (as-str raw)))))))))
      (seq
        (notes/doc/upsert-in
          slug
          title
          tree
          tags
          kind
          day
          (if
            (null? prev)
            (if
              (string-eq kind "source")
              "wiki/sources"
              "wiki/entities")
            (notes/doc/folder prev)))
        (let
          ((row (notes/doc/row slug)))
          (if
            (null? row)
            slug
            (seq
              (update-field
                (quote note)
                (car row)
                "updated_at"
                (wiki/now))
              (update-field (quote note) (car row) "updated_by" "llm")
              (update-field (quote note) (car row) "source_count" sc)
              slug)))))))
```

228 libs/workout/lib.pp

```
;; package workout -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push

(define workout/add-exercise-form
  (lambda
    (session-id)
    "HTML card form to log an exercise into a session."
    (html/div
      (html/attr "class" "card")
      (html/join
        (list
          (html/h2 "Log exercise")
          (html/form
            "POST"
            "/workout/session"
            ""
            (html/join
              (list
                (html/hidden "action" "log-exercise")
                (html/hidden "id" (as-str session-id))
                (html/label (html/attr "for" "we-name") "Exercise")
                (html/text-input
                  "exercise"
                  (html/attrs
                    (list
                      (html/attr "id" "we-name")
                      (html/attr "required" "required")
                      (html/attr "placeholder" "Swing / Snatch L /
    Pushup"))))
                (html/label (html/attr "for" "we-reps") "Reps (total)")
                (html/text-input
                  "reps"
                  (html/attrs
                    (list
                      (html/attr "id" "we-reps")
                      (html/attr "required" "required")
                      (html/attr "inputmode" "numeric")
                      (html/attr "placeholder" "75"))))
                (html/label (html/attr "for" "we-w") "Weight")
                (html/text-input
                  "weight"
                  (html/attrs
                    (list
                      (html/attr "id" "we-w")
                      (html/attr "inputmode" "decimal")
                      (html/attr "placeholder" "24"))))
                (html/label (html/attr "for" "we-u") "Unit")
                (html/text-input
                  "unit"
                  (html/attrs
                    (list
                      (html/attr "id" "we-u")
                      (html/attr "placeholder" "kg"))))
                (html/label (html/attr "for" "we-n") "Notes")
                (html/text-input
                  "notes"
                  (html/attrs
                    (list
                      (html/attr "id" "we-n")
                      (html/attr "placeholder" "optional"))))
                (html/button (html/attr "type" "submit") "Add")))))))))

(define workout/as-int
  (lambda
    (x)
    "Parse x as an integer, 0 when blank."
    (let ((s (as-str x))) (if (blank? s) 0 (parse-int s)))))

(define workout/char-rank
  (lambda
    (c)
```

```
    "Rank of a single char for custom string ordering."
    (let
      ((s (as-str c)))
      (cond
        ((string-eq s "0") 0)
        ((string-eq s "1") 1)
        ((string-eq s "2") 2)
        ((string-eq s "3") 3)
        ((string-eq s "4") 4)
        ((string-eq s "5") 5)
        ((string-eq s "6") 6)
        ((string-eq s "7") 7)
        ((string-eq s "8") 8)
        ((string-eq s "9") 9)
        ((string-eq s "-") 10)
        (else 50)))))

(define workout/continue-card
  (lambda
    ()
    "HTML card to resume any in-progress session."
    (let
      ((id (workout/session-in-progress ())))
      (if
        (null? id)
        ""
        (let
          ((f (workout/session id)))
          (if
            (null? f)
            ""
            (let
              ((prog (workout/session-program f))
                (run-href (string-append "/workout/run?id=" id))
                (sess-href (string-append "/workout/session?id=" id)))
              (html/div
                (html/attr "class" "card")
                (html/join
                  (list
                    (html/h2 "Continue")
                    (html/p
                      (string-append
                        (workout/session-title f)
                        " is in progress."))
                    (html/p*
                      ""
                      (html/join
                        (list
                          (if
                            (workout/ksk-program? prog)
                            (html/a run-href "Resume run")
                            "")
                          (if
                            (workout/ksk-program? prog)
                            (html/text " ? ")
                            "")
                          (html/a sess-href "Open session")))))))))))))

(define workout/date-ge?
  (lambda (a b) "True when date a is on/after date b." (string>=? (as-str
     a) (as-str b))))

(define workout/date-in-range?
  (lambda
    (day from to)
    "True when day is within the optional from/to range."
    (let
      ((d (as-str day)))
      (if
        (blank? d)
        #f
        (let
          ((ok-from
```

```
              (if (blank? (as-str from)) #t (workout/date-ge? d from)))
            (ok-to
              (if (blank? (as-str to)) #t (workout/date-le? d to))))
          (if ok-from ok-to #f))))))

(define workout/date-le?
  (lambda (a b) "True when date a is on/before date b." (string<=? (as-str
     a) (as-str b))))

(define workout/doc-params
  (lambda (doc) "Params sub-dict of a session doc." (dict-get doc
     "params")))

(define workout/doc-progress
  (lambda (doc) "Progress sub-dict of a session doc." (dict-get doc
     "progress")))

(define workout/empty-doc
  (lambda
    (program intensity)
    "Fresh empty session doc for a program and intensity."
    (dict-set*
      "{}"
      (list
        "program"
        (as-str program)
        "intensity"
        (as-str intensity)
        "params"
        "{}"
        "progress"
        "{}"
        "blocks"
        "[]"))))

(define workout/exercise-delete
  (lambda
    (id)
    "Delete a workout-exercise row by id."
    (delete (quote workout-exercise) (as-str id))))

(define workout/exercise-fields
  (lambda
    (session-id
      exercise
      weight
      unit
      reps
      notes
      detail)
    "Dict of fields for a workout-exercise row."
    (dict-set*
      "{}"
      (list
        "session_id"
        (as-str session-id)
        "exercise"
        (workout/normalize-exercise exercise)
        "weight"
        (as-str weight)
        "unit"
        (workout/normalize-unit unit)
        "reps"
        (as-str reps)
        "notes"
        (as-str notes)
        "detail"
        (as-str detail)))))

(define workout/exercise-li
  (lambda
    (row)
    "HTML list item for an exercise row, with delete form."
```

```
    (let
      ((id (workout/row-id row)) (f (workout/row-fields row)))
      (let
        ((ex (workout/field f "exercise"))
          (w (workout/field f "weight"))
          (u (workout/field f "unit"))
          (r (workout/field f "reps"))
          (n (workout/field f "notes"))
          (sid (workout/field f "session_id")))
        (let
          ((load
            (if
              (blank? w)
              ""
              (string-append
                " @ "
                w
                (if (blank? u) "" (string-append " " u))))))
          (html/li
            (html/attr "class" "card")
            (html/join
              (list
                (html/div
                  ""
                  (html/strong
                    (string-append ex " -- " r " reps" load)))
                (if
                  (blank? n)
                  ""
                  (html/p* (html/attr "class" "muted") (html/text n)))
                (html/form
                  "POST"
                  "/workout/session"
                  (html/attr "style" "display:inline")
                  (html/join
                    (list
                      (html/hidden "action" "delete-exercise")
                      (html/hidden "id" sid)
                      (html/hidden "exercise_id" id)
                      (html/button (html/attr "type" "submit")
    "Delete")))))))))))

(define workout/exercise-log
  (lambda
    (session-id
      exercise
      weight
      unit
      reps
      notes
      detail)
    "Create a workout-exercise row; () when the name is blank."
    (let
      ((name (workout/normalize-exercise exercise)))
      (if
        (blank? name)
        ()
        (create
          (quote workout-exercise)
          (workout/exercise-fields
            session-id
            name
            weight
            unit
            (if (blank? (as-str reps)) "0" (as-str reps))
            (if (null? notes) "" notes)
            (if (null? detail) "" detail)))))))

(define workout/exercise-names
  (lambda
    (limit)
    "Distinct normalized exercise names, up to limit."
    (let
```

```
      ((lim (if (null? limit) 100 (workout/as-int limit))))
      (list/foldl
        (lambda
          (acc row)
          (let
            ((n (workout/field (workout/row-fields row) "exercise")))
            (if
              (blank? n)
              acc
              (if (workout/name-seen? acc n) acc (cons n acc)))))
        ()
        (rows (quote workout-exercise) lim)))))

(define workout/exercises
  (lambda
    (session-id)
    "Exercise rows belonging to a session."
    (let
      ((sid (as-str session-id)))
      (list/filter
        (lambda
          (row)
          (string-eq
            (workout/field (workout/row-fields row) "session_id")
            sid))
        (rows (quote workout-exercise) 500)))))

(define workout/field
  (lambda
    (fields key)
    "Field value from a fields dict, \"\" when absent."
    (let
      ((v (dict-get fields (as-str key))))
      (if (null? v) "" (as-str v)))))

(define workout/handle-home-post
  (lambda
    ()
    "Route the home page POST to start or start-ksk."
    (let
      ((action (as-str (request-param "action"))))
      (cond
        ((string-eq action "start")
          (let
            ((id
              (workout/session-create
                (request-param "program")
                (request-param "intensity"))))
            (redirect
              (string-append "/workout/session?id=" (as-str id)))))
        ((string-eq action "start-ksk")
          (let
            ((id
              (workout/ksk-start
                (request-param "program")
                (request-param "phase")
                (request-param "day")
                (request-param "week")
                (request-param "load")
                (request-param "unit")
                (request-param "rm_total"))))
            (if
              (null? id)
              (redirect "/workout")
              (redirect
                (string-append "/workout/run?id=" (as-str id))))))
        (else (redirect "/workout"))))))

(define workout/handle-log-post
  (lambda
    ()
    "Route the quick-log POST to quick-log or quick-walk."
    (let
```

```
      ((action (as-str (request-param "action"))))
      (cond
        ((string-eq action "quick-log")
          (let
            ((sid
              (workout/session-create-complete
                "free"
                ""
                (request-param "date")
                (request-param "notes"))))
            (seq
              (workout/maybe-log
                sid
                (request-param "exercise1")
                (request-param "weight1")
                (request-param "unit1")
                (request-param "reps1"))
              (workout/maybe-log
                sid
                (request-param "exercise2")
                (request-param "weight2")
                (request-param "unit2")
                (request-param "reps2"))
              (workout/maybe-log
                sid
                (request-param "exercise3")
                (request-param "weight3")
                (request-param "unit3")
                (request-param "reps3"))
              (redirect
                (string-append "/workout/session?id=" (as-str sid))))))
        ((string-eq action "quick-walk")
          (let
            ((sid
              (workout/walk-create
                (request-param "date")
                (request-param "distance")
                (request-param "unit")
                (request-param "duration")
                (request-param "location")
                (request-param "notes"))))
            (redirect
              (string-append "/workout/session?id=" (as-str sid)))))
        (else (redirect "/workout/log"))))

(define workout/handle-run-post
  (lambda
    ()
    "Route the run page POST to ksk-step/capture/abandon."
    (let
      ((action (as-str (request-param "action")))
        (id (as-str (request-param "id"))))
      (cond
        ((string-eq action "ksk-step")
          (seq
            (workout/ksk-step id)
            (redirect (string-append "/workout/run?id=" id))))
        ((string-eq action "ksk-capture")
          (seq
            (workout/ksk-capture id)
            (redirect (string-append "/workout/session?id=" id))))
        ((string-eq action "abandon")
          (seq
            (workout/session-abandon id)
            (redirect (string-append "/workout/session?id=" id))))
        (else (redirect "/workout"))))

(define workout/handle-session-post
  (lambda
    ()
    "Route the session page POST to the matching action."
    (let
      ((action (as-str (request-param "action")))
```

```
        (id (as-str (request-param "id"))))
      (cond
        ((string-eq action "log-exercise")
          (seq
            (workout/exercise-log
              id
              (request-param "exercise")
              (request-param "weight")
              (request-param "unit")
              (request-param "reps")
              (request-param "notes")
              "")
            (redirect (string-append "/workout/session?id=" id))))
        ((string-eq action "delete-exercise")
          (seq
            (workout/exercise-delete (request-param "exercise_id"))
            (redirect (string-append "/workout/session?id=" id))))
        ((string-eq action "complete")
          (seq
            (if
              (workout/ksk-program?
                (workout/session-program (workout/session id)))
              (workout/ksk-capture id)
              (workout/session-complete id (request-param "notes")))
            (redirect (string-append "/workout/session?id=" id))))
        ((string-eq action "abandon")
          (seq
            (workout/session-abandon id)
            (redirect (string-append "/workout/session?id=" id))))
        ((string-eq action "ksk-step")
          (seq
            (workout/ksk-step id)
            (redirect (string-append "/workout/run?id=" id))))
        ((string-eq action "ksk-capture")
          (seq
            (workout/ksk-capture id)
            (redirect (string-append "/workout/session?id=" id))))
        ((string-eq action "delete-session")
          (seq (workout/session-delete id) (redirect "/workout")))
        (else (redirect "/workout"))))))

(define workout/history
  (lambda
    (from-date to-date limit)
    "Sessions whose date falls in the range, newest first."
    (let
      ((lim (if (null? limit) 50 (workout/as-int limit))))
      (list/filter
        (lambda
          (row)
          (workout/date-in-range?
            (workout/session-date (workout/row-fields row))
            from-date
            to-date))
        (workout/session-list lim)))))

(define workout/home-main
  (lambda
    ()
    "HTML main content of the workout home page."
    (html/div
      (html/attr "id" "workout-main")
      (html/join
        (list
          (workout/continue-card)
          (workout/start-bar)
          (workout/recent-list))))))

(define workout/home-page
  (lambda
    ()
    "Home page, handling POST or rendering the HTML."
    (if
```

```
      (string-eq (request-method) "POST")
      (workout/handle-home-post)
      (html/page
        (list
          (html/h1 "Workout")
          (html/p*
            (html/attr "class" "muted")
            (html/text
              "Training journal -- free log + King-Sized Killer (snatch)."))
          (workout/home-main))))))

(define workout/insert-session-desc
  (lambda
    (sorted row)
    "Insert a session row into a date-descending sorted list."
    (if
      (null? sorted)
      (list row)
      (let
        ((d (workout/session-date (workout/row-fields row)))
          (hd
            (workout/session-date (workout/row-fields (car sorted)))))
        (if
          (workout/date-ge? d hd)
          (cons row sorted)
          (cons
            (car sorted)
            (workout/insert-session-desc (cdr sorted) row)))))))

(define workout/join-nums
  (lambda
    (xs)
    "Join a list of numbers into a space-separated string."
    (list/foldl
      (lambda
        (acc x)
        (if
          (blank? acc)
          (as-str x)
          (string-append acc " " (as-str x))))
      ""
      xs)))

(define workout/ksk-build-doc
  (lambda
    (program
      phase
      day
      week
      load
      unit
      pattern
      rm-total
      pct
      per-hand)
    "Build the KSK session document dict from plan params."
    (dict-set*
      "{}"
      (list
        "program"
        (as-str program)
        "exercise"
        "Snatch"
        "params"
        (dict-set*
          "{}"
          (list
            "phase"
            (as-str phase)
            "day"
            (as-str day)
            "week"
            (as-str week)
```

```
            "time_cap_min"
            (as-str (workout/ksk-time-cap program))
            "load"
            (as-str load)
            "unit"
            (workout/normalize-unit unit)
            "pattern"
            (as-str pattern)
            "kind"
            (workout/pattern-kind pattern)
            "rm_total"
            (as-str rm-total)
            "pct"
            (as-str pct)
            "per_hand"
            (as-str per-hand)
            "hand_policy"
            "per-rung-both"))
        "progress"
        (dict-set*
          "{}"
          (list
            "ladder"
            "0"
            "rung_index"
            "0"
            "side"
            "L"
            "reps_l"
            "0"
            "reps_r"
            "0"
            "sets_done"
            "0"
            "started"
            ""
            "elapsed_sec"
            "0"))
        "blocks"
        "[]"))))

(define workout/ksk-capture
  (lambda
    (session-id)
    "Write KSK facts then complete the session."
    (let
      ((doc (workout/parse-session-doc session-id)))
      (seq
        (unless
          (null? doc)
          (workout/ksk-upsert-facts session-id doc))
        (workout/session-complete session-id "")))))

(define workout/ksk-current-target
  (lambda
    (doc)
    "Current per-hand snatch target from a session doc."
    (let*
      ((params (workout/doc-params doc))
        (prog (workout/doc-progress doc))
        (kind (workout/pget params "kind"))
        (pat (workout/pget params "pattern"))
        (ri (workout/as-int (workout/pget prog "rung_index"))))
      (if
        (string-eq kind "sets")
        (let
          ((ph (workout/pget params "per_hand")))
          (if
            (blank? ph)
            (workout/pattern-set-size pat)
            (workout/as-int ph)))
        (let*
          ((rungs (workout/pattern-rungs pat))
```

```
            (n (workout/len rungs))
            (i (if (< ri 0) 0 (if (>= ri n) 0 ri))))
          (if (null? rungs) 0 (workout/as-int (nth rungs i))))))))

(define workout/ksk-label
  (lambda
    (doc)
    "Human-readable title for a KSK session doc."
    (let
      ((params (workout/doc-params doc))
        (prog (as-str (dict-get doc "program"))))
      (let
        ((phase (workout/pget params "phase"))
          (day (workout/pget params "day"))
          (week (workout/pget params "week"))
          (pat (workout/pget params "pattern"))
          (load (workout/pget params "load"))
          (unit (workout/pget params "unit"))
          (cap (workout/pget params "time_cap_min")))
        (string-append
          (as-str prog)
          (if (blank? phase) "" (string-append " P" phase))
          (if (blank? week) "" (string-append " W" week))
          " D"
          day
          " ? "
          pat
          " ? "
          load
          (if (blank? unit) "" (string-append " " unit))
          " ? "
          cap
          "m"))))))

(define workout/ksk-program?
  (lambda
    (p)
    "True when p names a KSK program (ksk1/2/3)."
    (let
      ((s (as-str p)))
      (if
        (string-eq s "ksk1")
        #t
        (if (string-eq s "ksk2") #t (string-eq s "ksk3"))))))

(define workout/ksk-replace-side
  (lambda
    (sid name load unit reps)
    "Replace the exercise rows for one side with the given totals."
    (seq
      (list/map
        (lambda
          (row)
          (let
            ((f (workout/row-fields row)))
            (if
              (string-eq (workout/field f "exercise") (as-str name))
              (workout/exercise-delete (workout/row-id row))
              ())))
        (workout/exercises sid))
      (unless
        (or
          (blank? (as-str reps))
          (string-eq (as-str reps) "0"))
        (workout/exercise-log
          sid
          name
          load
          unit
          reps
          ""
          ""))))))

```

```
(define workout/ksk-save-doc
  (lambda
    (id doc)
    "Persist doc back onto the session, preserving other fields."
    (let
      ((cur (workout/session id)))
      (if
        (null? cur)
        ()
        (workout/session-update
          id
          (workout/session-fields
            (workout/session-date cur)
            (workout/session-program cur)
            (workout/session-intensity cur)
            (workout/session-state cur)
            (workout/session-notes cur)
            doc
            (workout/field cur "duration_sec")
            (workout/field cur "author")))))))

(define workout/ksk-start
  (lambda
    (program
      phase
      day
      week
      load
      unit
      rm-total)
    "Create an in-progress KSK session; () when the plan is unknown."
    (let*
      ((p (as-str program))
        (doc
          (cond
            ((string-eq p "ksk1")
              (workout/ksk1-plan phase day load unit))
            ((string-eq p "ksk2")
              (workout/ksk2-plan phase day load unit))
            ((string-eq p "ksk3")
              (workout/ksk3-plan week day load unit rm-total))
            (else ())))
        (inten
          (if
            (string-eq p "ksk3")
            (string-append "w" (as-str week) "d" (as-str day))
            (string-append "p" (as-str phase) "d" (as-str day)))))
      (if
        (null? doc)
        ()
        (workout/session-create-with-doc p inten doc)))))

(define workout/ksk-step
  (lambda
    (session-id)
    "Advance one side of a KSK run and persist the new progress."
    (let
      ((doc (workout/parse-session-doc session-id)))
      (if
        (null? doc)
        ()
        (let*
          ((params (workout/doc-params doc))
            (prog (workout/doc-progress doc))
            (target (workout/ksk-current-target doc))
            (side (workout/pget prog "side"))
            (kind (workout/pget params "kind"))
            (pat (workout/pget params "pattern"))
            (rl (workout/as-int (workout/pget prog "reps_l")))
            (rr (workout/as-int (workout/pget prog "reps_r")))
            (ri (workout/as-int (workout/pget prog "rung_index")))
            (lad (workout/as-int (workout/pget prog "ladder")))
            (sd (workout/as-int (workout/pget prog "sets_done")))
```

```
            (advance-rung (string-eq side "R"))
            (is-sets (string-eq kind "sets"))
            (n-rungs (workout/len (workout/pattern-rungs pat)))
            (rl2 (if (string-eq side "L") (+ rl target) rl))
            (rr2 (if advance-rung (+ rr target) rr))
            (next-side (if advance-rung "L" "R"))
            (ri2
              (if
                is-sets
                ri
                (if
                  advance-rung
                  (if (>= (+ ri 1) n-rungs) 0 (+ ri 1))
                  ri)))
            (lad2
              (if
                is-sets
                lad
                (if
                  advance-rung
                  (if (>= (+ ri 1) n-rungs) (+ lad 1) lad)
                  lad)))
            (sd2 (if is-sets (if advance-rung (+ sd 1) sd) sd))
            (prog2
              (dict-set*
                "{}"
                (list
                  "ladder"
                  (as-str lad2)
                  "rung_index"
                  (as-str ri2)
                  "side"
                  next-side
                  "reps_l"
                  (as-str rl2)
                  "reps_r"
                  (as-str rr2)
                  "sets_done"
                  (as-str sd2)
                  "started"
                  (workout/pget prog "started")
                  "elapsed_sec"
                  (workout/pget prog "elapsed_sec"))))
              (doc2 (dict-set doc "progress" prog2)))
          (seq
            (workout/ksk-save-doc session-id doc2)
            (workout/ksk-upsert-facts session-id doc2)
            doc2))))))

(define workout/ksk-time-cap
  (lambda
    (program)
    "Time cap in minutes for a KSK program (15 for ksk3, else 20)."
    (if (string-eq (as-str program) "ksk3") 15 20)))

(define workout/ksk-upsert-facts
  (lambda
    (session-id doc)
    "Write the current L/R snatch totals as exercise rows."
    (let
      ((params (workout/doc-params doc))
        (prog (workout/doc-progress doc)))
      (let
        ((load (workout/pget params "load"))
          (unit (workout/pget params "unit"))
          (rl (workout/pget prog "reps_l"))
          (rr (workout/pget prog "reps_r"))
          (sid (as-str session-id)))
        (seq
          (workout/ksk-replace-side sid "Snatch L" load unit rl)
          (workout/ksk-replace-side sid "Snatch R" load unit rr))))))

(define workout/ksk1-next
```

```
  (lambda
    (phase week day)
    "Next (phase week day) tuple, or () when the program completes."
    (cond
      ((< day 3) (list phase week (+ day 1)))
      ((< week 3) (list phase (+ week 1) 1))
      ((< phase 3) (list (+ phase 1) 1 1))
      (else (list)))))

(define workout/ksk1-pattern
  (lambda
    (phase day)
    "Pattern string for a KSK1 phase/day."
    (let
      ((p (workout/as-int phase)) (d (workout/as-int day)))
      (cond
        ((= p 1)
          (cond
            ((= d 1) "1,2,3,4,5")
            ((= d 2) "1,2,3")
            (else "1,2,3,4")))
        ((= p 2)
          (cond
            ((= d 1) "3,6,9")
            ((= d 2) "3,6")
            (else "3,6,9,12")))
        (else
          (cond ((= d 1) "S:9") ((= d 2) "S:15") (else "S:12")))))))

(define workout/ksk1-plan
  (lambda
    (phase day load unit)
    "Build a KSK1 session doc from phase/day/load/unit."
    (let
      ((p (workout/as-int phase))
        (d (workout/as-int day))
        (pat (workout/ksk1-pattern phase day)))
      (workout/ksk-build-doc
        "ksk1"
        p
        d
        ""
        load
        unit
        pat
        ""
        ""
        "")))))

(define workout/ksk1-sched-from
  (lambda
    (date
      weekdays
      load
      unit
      skip
      phase
      week
      day
      notes
      create?
      acc)
    "Accumulate KSK1 scheduled dates, creating calendar rows when asked."
    (if
      (> phase 3)
      (if create? (count acc) acc)
      (let*
        ((pat (workout/ksk1-pattern phase day))
          (title
            (string-append
              "KSK1 P"
              (as-str phase)
              " W"
```

```
              (as-str week)
              " D"
              (as-str day)
              " ? "
              pat
              " ? "
              (as-str load)
              (as-str unit)
              " ? "
              (as-str (workout/ksk-time-cap "ksk1"))
              "m"))
          (nxt (workout/ksk1-next phase week day)))
        (if
          (> skip 0)
          (if
            (null? nxt)
            (if create? (count acc) acc)
            (workout/ksk1-sched-from
              date
              weekdays
              load
              unit
              (- skip 1)
              (nth nxt 0)
              (nth nxt 1)
              (nth nxt 2)
              notes
              create?
              acc))
          (let*
            ((acc2 (append acc (list (list date title))))
              (next-date
                (workout/next-training-day
                  (date-add-days date 1)
                  weekdays)))
            (seq
              (if create? (calendar/create title date "" notes) ())
              (if
                (null? nxt)
                (if create? (count acc2) acc2)
                (workout/ksk1-sched-from
                  next-date
                  weekdays
                  load
                  unit
                  0
                  (nth nxt 0)
                  (nth nxt 1)
                  (nth nxt 2)
                  notes
                  create?
                  acc2)))))))

(define workout/ksk1-schedule
  (lambda
    (start-date
      weekdays
      load
      unit
      skip
      notes
      create?)
    "Plan the full KSK1 schedule of dates and titles."
    (workout/ksk1-sched-from
      (workout/next-training-day start-date weekdays)
      weekdays
      load
      unit
      skip
      1
      1
      1
      notes
```

```
      create?
      (list))))

(define workout/ksk2-plan
  (lambda
    (phase day load unit)
    "Build a KSK2 session doc from phase/day/load/unit."
    (let
      ((p (workout/as-int phase))
        (d (workout/as-int day))
        (n (workout/ksk2-set-size phase day)))
      (workout/ksk-build-doc
        "ksk2"
        p
        d
        ""
        load
        unit
        (string-append "S:" (as-str n))
        ""
        ""
        ""))))

(define workout/ksk2-set-size
  (lambda
    (phase day)
    "Reps per set for a KSK2 phase/day."
    (let
      ((p (workout/as-int phase)) (d (workout/as-int day)))
      (cond
        ((= p 1) (cond ((= d 1) 10) ((= d 2) 20) (else 15)))
        ((= p 2) (cond ((= d 1) 12) ((= d 2) 24) (else 18)))
        (else (cond ((= d 1) 15) ((= d 2) 30) (else 20)))))))

(define workout/ksk3-pct
  (lambda
    (week day)
    "Percentage of RM for a KSK3 week/day."
    (let
      ((w (workout/as-int week)) (d (workout/as-int day)))
      (cond
        ((= d 1) (if (or (= w 1) (= w 3)) 50 60))
        ((= d 2) (if (or (= w 1) (= w 3)) 65 75))
        (else (if (or (= w 1) (= w 3)) 70 80))))))

(define workout/ksk3-per-hand
  (lambda
    (rm-total pct)
    "Per-hand snatch target given RM total and percentage."
    (workout/max0
      (floor
        (div
          (* (workout/as-int rm-total) (workout/as-int pct))
          200)))))

(define workout/ksk3-plan
  (lambda
    (week day load unit rm-total)
    "Build a KSK3 session doc from week/day/load/unit/rm."
    (let
      ((w (workout/as-int week))
        (d (workout/as-int day))
        (rm (workout/as-int rm-total))
        (pct (workout/ksk3-pct week day)))
      (let
        ((ph (workout/ksk3-per-hand rm pct)))
        (workout/ksk-build-doc
          "ksk3"
          ""
          d
          w
          load
          unit
```

```
          (string-append "S:" (as-str ph))
          rm
          pct
          ph)))))

(define workout/len
  (lambda
    (xs)
    "Number of elements in a list."
    (list/foldl (lambda (acc _) (+ acc 1)) 0 xs)))

(define workout/log-form
  (lambda
    ()
    "HTML card form for the quick-log of a finished workout."
    (html/div
      (html/attr "class" "card")
      (html/join
        (list
          (html/h2 "Quick log")
          (html/p*
            (html/attr "class" "muted")
            (html/text
              "Log a finished free workout (session + up to 3 exercises)."))
          (html/form
            "POST"
            "/workout/log"
            ""
            (html/join
              (list
                (html/hidden "action" "quick-log")
                (html/label (html/attr "for" "ql-date") "Date")
                (html/text-input
                  "date"
                  (html/attrs
                    (list
                      (html/attr "id" "ql-date")
                      (html/attr "value" (workout/today))
                      (html/attr "placeholder" "YYYY-MM-DD"))))
                (html/label
                  (html/attr "for" "ql-notes")
                  "Session notes")
                (html/textarea
                  "notes"
                  (html/attrs
                    (list
                      (html/attr "id" "ql-notes")
                      (html/attr "rows" "2")
                      (html/attr "placeholder" "optional")))
                  "")
                (html/h3 "Exercise 1")
                (html/label (html/attr "for" "ql-e1") "Name")
                (html/text-input
                  "exercise1"
                  (html/attrs
                    (list
                      (html/attr "id" "ql-e1")
                      (html/attr "required" "required")
                      (html/attr "placeholder" "Swing"))))
                (html/label (html/attr "for" "ql-r1") "Reps")
                (html/text-input
                  "reps1"
                  (html/attrs
                    (list
                      (html/attr "id" "ql-r1")
                      (html/attr "required" "required")
                      (html/attr "inputmode" "numeric"))))
                (html/label (html/attr "for" "ql-w1") "Weight")
                (html/text-input
                  "weight1"
                  (html/attrs (list (html/attr "id" "ql-w1"))))
                (html/label (html/attr "for" "ql-u1") "Unit")
                (html/text-input
```

```
                  "unit1"
                  (html/attrs
                    (list
                      (html/attr "id" "ql-u1")
                      (html/attr "placeholder" "kg"))))
                (html/h3 "Exercise 2 (optional)")
                (html/label (html/attr "for" "ql-e2") "Name")
                (html/text-input
                  "exercise2"
                  (html/attrs (list (html/attr "id" "ql-e2"))))
                (html/label (html/attr "for" "ql-r2") "Reps")
                (html/text-input
                  "reps2"
                  (html/attrs
                    (list
                      (html/attr "id" "ql-r2")
                      (html/attr "inputmode" "numeric"))))
                (html/label (html/attr "for" "ql-w2") "Weight")
                (html/text-input
                  "weight2"
                  (html/attrs (list (html/attr "id" "ql-w2"))))
                (html/label (html/attr "for" "ql-u2") "Unit")
                (html/text-input
                  "unit2"
                  (html/attrs
                    (list
                      (html/attr "id" "ql-u2")
                      (html/attr "placeholder" "kg"))))
                (html/h3 "Exercise 3 (optional)")
                (html/label (html/attr "for" "ql-e3") "Name")
                (html/text-input
                  "exercise3"
                  (html/attrs (list (html/attr "id" "ql-e3"))))
                (html/label (html/attr "for" "ql-r3") "Reps")
                (html/text-input
                  "reps3"
                  (html/attrs
                    (list
                      (html/attr "id" "ql-r3")
                      (html/attr "inputmode" "numeric"))))
                (html/label (html/attr "for" "ql-w3") "Weight")
                (html/text-input
                  "weight3"
                  (html/attrs (list (html/attr "id" "ql-w3"))))
                (html/label (html/attr "for" "ql-u3") "Unit")
                (html/text-input
                  "unit3"
                  (html/attrs
                    (list
                      (html/attr "id" "ql-u3")
                      (html/attr "placeholder" "kg"))))
                (html/button (html/attr "type" "submit") "Save workout"))))
          (html/p*
            ""
            (html/a "/workout" "? All workouts")))))))

(define workout/log-page
  (lambda
    ()
    "Quick-log page, handling POST or rendering the form."
    (if
      (string-eq (request-method) "POST")
      (workout/handle-log-post)
      (html/page
        (list
          (html/h1 "Quick log")
          (workout/log-form)
          (workout/walk-form))))))

(define workout/log/ksk-write
  (lambda
    (date
      program
```

```
      intensity
      doc
      load
      unit
      total-l
      total-r
      notes
      rungs-l-str
      rungs-r-str)
    "Create a completed KSK session from logged totals and rungs."
    (let*
      ((tl (workout/as-int total-l))
        (tr (workout/as-int total-r))
        (rsl (as-str rungs-l-str))
        (rsr (as-str rungs-r-str))
        (prog0 (workout/doc-progress doc))
        (prog
          (dict-set*
            "{}"
            (list
              "ladder"
              (workout/pget prog0 "ladder")
              "rung_index"
              (workout/pget prog0 "rung_index")
              "side"
              "L"
              "reps_l"
              (as-str tl)
              "reps_r"
              (as-str tr)
              "sets_done"
              "0"
              "started"
              ""
              "elapsed_sec"
              ""
              "logged"
              "true"
              "rungs_l"
              rsl
              "rungs_r"
              rsr)))
        (n
          (if
            (blank? (as-str notes))
            (if
              (string-eq (as-str tl) (as-str tr))
              (string-append
                (as-str program)
                " log: "
                rsl
                " @ "
                (as-str load)
                (as-str unit)
                " ("
                (as-str tl)
                "/hand)")
              (string-append
                (as-str program)
                " log L:"
                rsl
                " R:"
                rsr
                " @ "
                (as-str load)
                (as-str unit)
                " ("
                (as-str tl)
                "/"
                (as-str tr)
                ")"))
            (as-str notes)))
        (doc2 (dict-set doc "progress" prog))
```

```
        (sid
          (create
            "workout-session"
            (workout/session-fields
              (if
                (blank? (as-str date))
                (workout/today)
                (as-str date))
              (as-str program)
              (as-str intensity)
              "complete"
              n
              doc2
              ""
              (as-str (whoami))))))
      (seq
        (unless
          (< tl 1)
          (workout/exercise-log
            sid
            "Snatch L"
            load
            unit
            tl
            ""
            rsl))
        (unless
          (< tr 1)
          (workout/exercise-log
            sid
            "Snatch R"
            load
            unit
            tr
            ""
            rsr))
        sid))))

(define workout/log/ksk1
  (lambda
    (date
      phase
      day
      load
      unit
      rungs
      notes)
    "Log a finished KSK1 session from a rungs list."
    (let
      ((doc (workout/ksk1-plan phase day load unit))
        (total (workout/sum rungs))
        (rs (workout/join-nums rungs))
        (inten
          (string-append "p" (as-str phase) "d" (as-str day)))
        (n (if (null? notes) "" notes)))
      (workout/log/ksk-write
        date
        "ksk1"
        inten
        doc
        load
        unit
        total
        total
        n
        rs
        rs))))

(define workout/log/ksk1-sides
  (lambda
    (date
      phase
      day
```

```
      load
      unit
      rungs-l
      rungs-r
      notes)
    "Log a finished KSK1 session with separate L/R rung lists."
    (let
      ((doc (workout/ksk1-plan phase day load unit))
        (tl (workout/sum rungs-l))
        (tr (workout/sum rungs-r))
        (rsl (workout/join-nums rungs-l))
        (rsr (workout/join-nums rungs-r))
        (inten
          (string-append "p" (as-str phase) "d" (as-str day)))
        (n (if (null? notes) "" notes)))
      (workout/log/ksk-write
        date
        "ksk1"
        inten
        doc
        load
        unit
        tl
        tr
        n
        rsl
        rsr))))

(define workout/log/ksk2
  (lambda
    (date
      phase
      day
      load
      unit
      sets
      notes)
    "Log a finished KSK2 session from a sets list."
    (let
      ((doc (workout/ksk2-plan phase day load unit))
        (total (workout/sum sets))
        (rs (workout/join-nums sets))
        (inten
          (string-append "p" (as-str phase) "d" (as-str day)))
        (n (if (null? notes) "" notes)))
      (workout/log/ksk-write
        date
        "ksk2"
        inten
        doc
        load
        unit
        total
        total
        n
        rs
        rs))))

(define workout/log/ksk3
  (lambda
    (date
      week
      day
      load
      unit
      rm-total
      sets
      notes)
    "Log a finished KSK3 session from a sets list."
    (let
      ((doc (workout/ksk3-plan week day load unit rm-total))
        (total (workout/sum sets))
        (rs (workout/join-nums sets))
```

```
        (inten
          (string-append "w" (as-str week) "d" (as-str day)))
        (n (if (null? notes) "" notes)))
      (workout/log/ksk-write
        date
        "ksk3"
        inten
        doc
        load
        unit
        total
        total
        n
        rs
        rs))))

(define workout/max0
  (lambda (n) "At least 1 (bottom floor for positive counts)." (if (< n 1)
     1 n)))

(define workout/maybe-log
  (lambda
    (sid name weight unit reps)
    "Log an exercise row, or () when the name is blank."
    (if
      (blank? (as-str name))
      ()
      (workout/exercise-log
        sid
        name
        weight
        unit
        reps
        ""
        ""))))

(define workout/member?
  (lambda
    (x xs)
    "True when x is in list xs (numeric equality)."
    (if
      (null? xs)
      #f
      (if (= (car xs) x) #t (workout/member? x (cdr xs))))))

(define workout/name-seen?
  (lambda
    (lst name)
    "True when an exercise name is already in lst."
    (list/foldl
      (lambda
        (acc x)
        (if acc acc (string-eq (as-str x) (as-str name))))
      #f
      lst)))

(define workout/next-training-day
  (lambda
    (date weekdays)
    "Next date (from date) that falls on a training weekday."
    (let*
      ((parts (string-split (as-str date) "-"))
        (y (parse-int (nth parts 0)))
        (m (parse-int (nth parts 1)))
        (d (parse-int (nth parts 2)))
        (wd (date-weekday y m d)))
      (if
        (workout/member? wd weekdays)
        (as-str date)
        (workout/next-training-day
          (date-add-days (as-str date) 1)
          weekdays)))))
```

```
(define workout/normalize-exercise
  (lambda (name) "Trimmed exercise name." (string-trim (as-str name))))

(define workout/normalize-unit
  (lambda
    (u)
    "Normalize a weight unit string to kg or lb, else as-is."
    (let
      ((s (as-str u)))
      (cond
        ((blank? s) "")
        ((string-eq s "KG") "kg")
        ((string-eq s "Kg") "kg")
        ((string-eq s "LB") "lb")
        ((string-eq s "Lb") "lb")
        ((string-eq s "lbs") "lb")
        ((string-eq s "LBS") "lb")
        (else s)))))

(define workout/parse-session-doc
  (lambda
    (id)
    "Parsed doc dict for a session, or ()."
    (let
      ((f (workout/session id)))
      (if
        (null? f)
        ()
        (let
          ((raw (workout/session-doc f)))
          (if (blank? raw) () raw))))))

(define workout/pattern-kind
  (lambda
    (pat)
    "Kind of a pattern string: sets or ladder."
    (let
      ((s (as-str pat)))
      (if
        (string-eq
          (substring s 0 (if (< (string-length s) 2) 0 2))
          "S:")
        "sets"
        "ladder"))))

(define workout/pattern-rungs
  (lambda
    (pat)
    "Rung list of a ladder pattern, () for a sets pattern."
    (let
      ((s (as-str pat)))
      (if
        (string-eq (workout/pattern-kind s) "sets")
        (list)
        (string-split s ",")))))

(define workout/pattern-set-size
  (lambda
    (pat)
    "Set size of a sets pattern, 0 for a ladder pattern."
    (let
      ((s (as-str pat)))
      (if
        (string-eq (workout/pattern-kind s) "sets")
        (workout/as-int (substring s 2 (- (string-length s) 2)))
        0))))

(define workout/pget
  (lambda
    (dict key)
    "dict-get as a string, \"\" when absent."
    (if
      (null? dict)
```

```
      ""
      (let
        ((v (dict-get dict (as-str key))))
        (if (null? v) "" (as-str v))))))

(define workout/programs
  (lambda () "Supported workout program names." (list "free" "ksk1" "ksk2"
    "ksk3")))

(define workout/recent-list
  (lambda
    ()
    "HTML list of the most recent sessions plus walk form."
    (let
      ((rows (workout/sessions-sorted)))
      (html/div
        (html/attr "id" "workout-recent")
        (html/join
          (list
            (html/h2 "Recent")
            (if
              (null? rows)
              (html/p*
                (html/attr "class" "muted")
                (html/text "No sessions yet."))
              (html/ul
                ""
                (html/join (list/map workout/session-li rows))))
            (workout/walk-form)))))))

(define workout/row-fields
  (lambda (row) "Fields portion of a data row." (cdr row)))

(define workout/row-id
  (lambda (row) "Id string of a data row." (as-str (car row))))

(define workout/run-main
  (lambda
    (id)
    "Main HTML of the KSK run UI for a session."
    (let
      ((f (workout/session id)))
      (if
        (null? f)
        (html/div
          ""
          (html/join
            (list
              (html/p "Session not found.")
              (html/p* "" (html/a "/workout" "? Back")))))
        (let
          ((doc (workout/parse-session-doc id))
            (st (workout/session-state f))
            (prog-name (workout/session-program f)))
          (if
            (not (workout/ksk-program? prog-name))
            (html/div
              ""
              (html/join
                (list
                  (html/p "Not a KSK session.")
                  (html/p*
                    ""
                    (html/a
                      (string-append "/workout/session?id=" id)
                      "Open session")))))
            (let
              ((params (workout/doc-params doc))
                (prog (workout/doc-progress doc))
                (target (workout/ksk-current-target doc))
                (side (workout/pget (workout/doc-progress doc) "side"))
                (cap
                  (workout/pget (workout/doc-params doc) "time_cap_min"))
```

```
                (kind (workout/pget (workout/doc-params doc) "kind"))
                (pat (workout/pget (workout/doc-params doc) "pattern"))
                (rl (workout/pget (workout/doc-progress doc) "reps_l"))
                (rr (workout/pget (workout/doc-progress doc) "reps_r"))
                (lad (workout/pget (workout/doc-progress doc) "ladder"))
                (sd
                  (workout/pget (workout/doc-progress doc) "sets_done"))
                (ri
                  (workout/pget (workout/doc-progress doc) "rung_index")))
              (html/div
                (html/attr "class" "ksk-run")
                (html/join
                  (list
                    (workout/run-style)
                    (html/h1 (html/text (workout/ksk-label doc)))
                    (html/p*
                      (html/attr "class" "muted")
                      (html/text (string-append "state: " st)))
                    (html/div
                      (html/attrs
                        (list
                          (html/attr "id" "ksk-timer")
                          (html/attr "class" "ksk-timer")
                          (html/attr "data-sid" id)))
                      (html/text "--:--"))
                    (workout/timer-script cap)
                    (html/p*
                      (html/attr "class" "ksk-side")
                      (html/text
                        (string-append
                          "Hand: "
                          side
                          (if
                            (string-eq kind "ladder")
                            (string-append
                              " ? rung #"
                              (as-str (+ (workout/as-int ri) 1))
                              " ? ladder "
                              lad)
                            (string-append " ? sets done " sd)))))
                    (html/p*
                      (html/attr "class" "ksk-target")
                      (html/text
                        (string-append (as-str target) " snatches (" side
")")))
                    (html/p*
                      (html/attr "class" "muted")
                      (html/text
                        (string-append
                          "pattern "
                          pat
                          " ? per-rung both hands")))
                    (if
                      (string-eq st "in_progress")
                      (html/join
                        (list
                          (html/form
                            "POST"
                            "/workout/run"
                            ""
                            (html/join
                              (list
                                (html/hidden "action" "ksk-step")
                                (html/hidden "id" id)
                                (html/button
                                  (html/attr "type" "submit")
                                  (string-append "Done " (as-str target) "
" side)))))
                          (html/form
                            "POST"
                            "/workout/run"
                            ""
                            (html/join
```

```
                              (list
                                (html/hidden "action" "ksk-capture")
                                (html/hidden "id" id)
                                (html/button
                                  (html/attr "type" "submit")
                                  "Finish session"))))
                          (html/form
                            "POST"
                            "/workout/run"
                            (html/attr "style" "display:inline")
                            (html/join
                              (list
                                (html/hidden "action" "abandon")
                                (html/hidden "id" id)
                                (html/button (html/attr "type" "submit")
    "Abandon"))))))
                        "")
                      (html/p*
                        (html/attr "class" "ksk-totals")
                        (html/text
                          (string-append
                            "Totals -- L: "
                            rl
                            " ? R: "
                            rr
                            " ? all: "
                            (as-str (+ (workout/as-int rl) (workout/as-int
    rr))))))
                      (html/p*
                        ""
                        (html/join
                          (list
                            (html/a
                              (string-append "/workout/session?id=" id)
                              "Session detail")
                            (html/text " ? ")
                            (html/a "/workout" "All workouts")))))))))))))

(define workout/run-page
  (lambda
    ()
    "Run page, handling POST or rendering the run UI."
    (if
      (string-eq (request-method) "POST")
      (workout/handle-run-post)
      (let
        ((id (as-str (request-param "id"))))
        (html/page
          (list
            (if
              (blank? id)
              (html/join
                (list
                  (html/h1 "Run")
                  (html/p "Missing session id.")
                  (html/p* "" (html/a "/workout" "? Back"))))
              (workout/run-main id)))))))

(define workout/run-style
  (lambda
    ()
    "Inline CSS for the KSK run page."
    (html/tag
      "style"
      ""
      (html/join
        (list
          ".ksk-run
    .ksk-timer{font-size:2.5rem;font-weight:700;letter-spacing:.05em}"
          ".ksk-run .ksk-target{font-size:2rem;font-weight:700;margin:.5rem
    0}"
          ".ksk-run .ksk-side{font-size:1.4rem}"
```

```
          ".ksk-run form button{font-size:1.2rem;padding:.75rem
    1.25rem;margin:.35rem .35rem .35rem 0}"
          ".ksk-run .ksk-totals{font-size:1.1rem;margin-top:.75rem}")))))

(define workout/session
  (lambda
    (id)
    "Session fields for id, or ()."
    (let
      ((f (get (quote workout-session) (as-str id))))
      (if (null? f) () f))))

(define workout/session-abandon
  (lambda
    (id)
    "Mark a session abandoned."
    (let
      ((cur (workout/session id)))
      (if
        (null? cur)
        ()
        (workout/session-update
          id
          (workout/session-fields
            (workout/session-date cur)
            (workout/session-program cur)
            (workout/session-intensity cur)
            "abandoned"
            (workout/session-notes cur)
            (workout/session-doc cur)
            (workout/field cur "duration_sec")
            (workout/field cur "author")))))))

(define workout/session-actions
  (lambda
    (id state)
    "HTML card of session actions (complete/abandon/run/delete)."
    (let
      ((f (workout/session id))
        (prog
          (if
            (null? (workout/session id))
            ""
            (workout/session-program (workout/session id)))))
      (html/div
        (html/attr "class" "card")
        (html/join
          (list
            (html/h2 "Session")
            (if
              (and
                (string-eq (as-str state) "in_progress")
                (workout/ksk-program? prog))
              (html/p*
                ""
                (html/a
                  (string-append "/workout/run?id=" (as-str id))
                  "? Open run UI"))
              "")
            (if
              (string-eq (as-str state) "in_progress")
              (html/join
                (list
                  (html/form
                    "POST"
                    "/workout/session"
                    (html/attr "style" "display:inline")
                    (html/join
                      (list
                        (html/hidden "action" "complete")
                        (html/hidden "id" (as-str id))
                        (html/button (html/attr "type" "submit")
    "Complete"))))
```

```
                  (html/text " ")
                  (html/form
                    "POST"
                    "/workout/session"
                    (html/attr "style" "display:inline")
                    (html/join
                      (list
                        (html/hidden "action" "abandon")
                        (html/hidden "id" (as-str id))
                        (html/button (html/attr "type" "submit")
    "Abandon"))))))
                "")
              (html/form
                "POST"
                "/workout/session"
                (html/attrs
                  (list
                    (html/attr "style" "margin-top:0.75rem")
                    (html/attr
                      "onsubmit"
                      "return confirm('Delete session and its exercises?')")))
                (html/join
                  (list
                    (html/hidden "action" "delete-session")
                    (html/hidden "id" (as-str id))
                    (html/button
                      (html/attr "type" "submit")
                      "Delete session"))))
              (html/p*
                ""
                (html/a "/workout" "? All workouts"))))))))

(define workout/session-complete
  (lambda
    (id notes)
    "Mark a session complete, keeping existing notes when blank."
    (let
      ((cur (workout/session id)))
      (if
        (null? cur)
        ()
        (let
          ((n
            (if
              (blank? (as-str notes))
              (workout/session-notes cur)
              (as-str notes))))
          (workout/session-update
            id
            (workout/session-fields
              (workout/session-date cur)
              (workout/session-program cur)
              (workout/session-intensity cur)
              "complete"
              n
              (workout/session-doc cur)
              (workout/field cur "duration_sec")
              (workout/field cur "author")))))))))

(define workout/session-create
  (lambda
    (program intensity)
    "Create an in-progress session using today's date."
    (let
      ((prog
          (if (blank? (as-str program)) "free" (as-str program)))
        (inten (if (null? intensity) "" (as-str intensity))))
      (create
        (quote workout-session)
        (workout/session-fields
          (workout/today)
          prog
          inten
```

```
          "in_progress"
          ""
          (workout/empty-doc prog inten)
          ""
          (as-str (whoami)))))))

(define workout/session-create-complete
  (lambda
    (program intensity date notes)
    "Create an already-complete session with the given date and notes."
    (let
      ((prog
          (if (blank? (as-str program)) "free" (as-str program)))
        (inten (if (null? intensity) "" (as-str intensity)))
        (d
          (if
            (blank? (as-str date))
            (workout/today)
            (as-str date)))
        (n (if (null? notes) "" (as-str notes))))
      (create
        (quote workout-session)
        (workout/session-fields
          d
          prog
          inten
          "complete"
          n
          (workout/empty-doc prog inten)
          ""
          (as-str (whoami)))))))

(define workout/session-create-with-doc
  (lambda
    (program intensity doc)
    "Create an in-progress session carrying the given doc."
    ;; let* (not parallel let): d's value references siblings prog/inten
    ;; bound in the same group -- parallel `let` evaluates bindings in the
    ;; outer env, so prog/inten were unbound when doc was blank
    ;; (threw `unbound: prog`).
    (let*
      ((prog
          (if (blank? (as-str program)) "free" (as-str program)))
        (inten (if (null? intensity) "" (as-str intensity)))
        (d
          (if
            (blank? (as-str doc))
            (workout/empty-doc prog inten)
            (as-str doc))))
      (create
        (quote workout-session)
        (workout/session-fields
          (workout/today)
          prog
          inten
          "in_progress"
          ""
          d
          ""
          (as-str (whoami)))))))

(define workout/session-date
  (lambda (f) "Date field of a session." (workout/field f "date")))

(define workout/session-date-of
  (lambda
    (session-id)
    "Date of a session by id, or \"\" when absent."
    (let
      ((f (workout/session session-id)))
      (if (null? f) "" (workout/session-date f)))))

(define workout/session-delete
```

```
  (lambda
    (id)
    "Delete a session and all of its exercise rows."
    (let
      ((sid (as-str id)))
      (seq
        (list/map
          (lambda
            (row)
            (workout/exercise-delete (workout/row-id row)))
          (workout/exercises sid))
        (delete (quote workout-session) sid)))))

(define workout/session-doc
  (lambda (f) "Doc field of a session." (workout/field f "doc")))

(define workout/session-fields
  (lambda
    (date
      program
      intensity
      state
      notes
      doc
      duration_sec
      author)
    "Dict of fields for a workout-session row."
    (dict-set*
      "{}"
      (list
        "date"
        (as-str date)
        "program"
        (as-str program)
        "intensity"
        (as-str intensity)
        "state"
        (as-str state)
        "notes"
        (as-str notes)
        "doc"
        (as-str doc)
        "duration_sec"
        (as-str duration_sec)
        "author"
        (as-str author)))))

(define workout/session-in-progress
  (lambda
    (program)
    "Id of an in-progress session (optionally matching program), or ()."
    (let
      ((want (if (null? program) "" (as-str program))))
      (list/foldl
        (lambda
          (acc row)
          (if
            (null? acc)
            (let
              ((f (workout/row-fields row)))
              (if
                (string-eq (workout/session-state f) "in_progress")
                (if
                  (blank? want)
                  (workout/row-id row)
                  (if
                    (string-eq (workout/session-program f) want)
                    (workout/row-id row)
                    acc))
                acc))
            acc))
        ()
        (workout/session-list 100)))))
```

```
2098
2099 (define workout/session-intensity
2100   (lambda (f) "Intensity field of a session." (workout/field f
       "intensity")))
2101
2102 (define workout/session-li
2103   (lambda
2104     (row)
2105     "HTML list item for a session row."
2106     (let
2107       ((id (workout/row-id row)) (f (workout/row-fields row)))
2108       (let
2109         ((reps (workout/session-total-reps id)))
2110         (html/li
2111           (html/attr "class" "card")
2112           (html/join
2113             (list
2114               (html/div
2115                 ""
2116                 (html/join
2117                   (list
2118                     (html/a
2119                       (string-append "/workout/session?id=" id)
2120                       (html/strong (workout/session-title f)))
2121                     (html/text " ")
2122                     (workout/state-badge (workout/session-state f)))))
2123               (if
2124                 (string-eq (workout/session-program f) "walk")
2125                 (html/p*
2126                   (html/attr "class" "muted")
2127                   (html/text (workout/walk-li-summary f)))
2128                 (html/p*
2129                   (html/attr "class" "muted")
2130                   (html/text (string-append (as-str reps) " total
    reps")))))))))))
2131
2132 (define workout/session-list
2133   (lambda
2134     (limit)
2135     "Session rows, up to limit (default 50)."
2136     (let
2137       ((n (if (null? limit) 50 (workout/as-int limit))))
2138       (rows (quote workout-session) (if (< n 1) 50 n)))))
2139
2140 (define workout/session-main
2141   (lambda
2142     (id)
2143     "Main HTML of a session page (title, actions, exercises)."
2144     (let
2145       ((f (workout/session id)))
2146       (if
2147         (null? f)
2148         (html/div
2149           (html/attr "id" "workout-session-main")
2150           (html/join
2151             (list
2152               (html/p "Session not found.")
2153               (html/p* "" (html/a "/workout" "Back")))))
2154         (let
2155           ((exs (workout/exercises id))
2156             (st (workout/session-state f))
2157             (notes (workout/session-notes f)))
2158           (let
2159             ((is-walk (string-eq (workout/session-program f) "walk")))
2160             (html/div
2161               (html/attr "id" "workout-session-main")
2162               (html/join
2163                 (list
2164                   (html/h1 (html/text (workout/session-title f)))
2165                   (html/p* "" (workout/state-badge st))
2166                   (if (blank? notes) "" (html/p notes))
2167                   (workout/session-actions id st)
2168                   (if
```

```
                    is-walk
                    (workout/walk-detail f)
                    (html/join
                      (list
                        (if
                          (string-eq st "abandoned")
                          ""
                          (workout/add-exercise-form id))
                        (html/h2 "Exercises")
                        (if
                          (null? exs)
                          (html/p*
                            (html/attr "class" "muted")
                            (html/text "No exercises logged."))
                          (html/ul
                            ""
                            (html/join (list/map workout/exercise-li exs))))
                        (html/p*
                          (html/attr "class" "muted")
                          (html/text
                            (string-append
                              "Total reps: "
                              (as-str (workout/session-total-reps
    id))))))))))))))))))

(define workout/session-notes
  (lambda (f) "Notes field of a session." (workout/field f "notes")))

(define workout/session-page
  (lambda
    ()
    "Session page, handling POST or rendering the session."
    (if
      (string-eq (request-method) "POST")
      (workout/handle-session-post)
      (let
        ((id (as-str (request-param "id"))))
        (html/page
          (list
            (if
              (blank? id)
              (html/join
                (list
                  (html/h1 "Session")
                  (html/p "Missing id.")
                  (html/p* "" (html/a "/workout" "? Back"))))
              (workout/session-main id)))))))))

(define workout/session-program
  (lambda (f) "Program field of a session." (workout/field f "program")))

(define workout/session-state
  (lambda (f) "State field of a session." (workout/field f "state")))

(define workout/session-summary
  (lambda
    (id)
    "Summary list (id fields exercises) for a session."
    (let
      ((f (workout/session id)))
      (if (null? f) () (list id f (workout/exercises id))))))

(define workout/session-title
  (lambda
    (f)
    "Title string for a session: date, program, and intensity."
    (let
      ((date (workout/session-date f))
        (prog (workout/session-program f))
        (inten (workout/session-intensity f)))
      (if
        (blank? inten)
        (string-append date " ? " prog)
```

```
        (string-append date " ? " prog " / " inten)))))

(define workout/session-total-reps
  (lambda
    (session-id)
    "Sum of reps across a session's exercises."
    (list/foldl
      (lambda
        (acc row)
        (+
          acc
          (workout/as-int
            (workout/field (workout/row-fields row) "reps"))))
      0
      (workout/exercises session-id))))

(define workout/session-update
  (lambda
    (id fields)
    "Update a session row with new fields."
    (update (quote workout-session) (as-str id) fields)))

(define workout/sessions-sorted
  (lambda
    ()
    "Sessions sorted newest first."
    (list/foldl
      workout/insert-session-desc
      ()
      (workout/session-list 100))))

(define workout/start-bar
  (lambda
    ()
    "HTML card to start a free session or a KSK run."
    (html/div
      (html/attr "class" "card")
      (html/join
        (list
          (html/h2 "Start workout")
          (html/form
            "POST"
            "/workout"
            ""
            (html/join
              (list
                (html/hidden "action" "start")
                (html/hidden "program" "free")
                (html/label
                  (html/attr "for" "w-inten")
                  "Note (optional)")
                (html/text-input
                  "intensity"
                  (html/attrs
                    (list
                      (html/attr "id" "w-inten")
                      (html/attr "placeholder" "optional tag"))))
                (html/button
                  (html/attr "type" "submit")
                  "Start free session"))))
          (html/h3 "King-Sized Killer")
          (html/p*
            (html/attr "class" "muted")
            (html/text
              "Single-bell snatch programs (1.0 ladders, 2.0 high sets, 3.0
    %RM)."))
          (html/form
            "POST"
            "/workout"
            ""
            (html/join
              (list
                (html/hidden "action" "start-ksk")
```

```
              (html/label (html/attr "for" "ksk-prog") "Program")
              (html/select
                "program"
                (html/attr "id" "ksk-prog")
                (html/join
                  (list
                    (html/option "ksk1" "KSK 1.0" #t)
                    (html/option "ksk2" "KSK 2.0" #f)
                    (html/option "ksk3" "KSK 3.0" #f))))
              (html/label
                (html/attr "for" "ksk-phase")
                "Phase (1.0/2.0)")
              (html/text-input
                "phase"
                (html/attrs
                  (list
                    (html/attr "id" "ksk-phase")
                    (html/attr "value" "1")
                    (html/attr "inputmode" "numeric"))))
              (html/label (html/attr "for" "ksk-week") "Week (3.0)")
              (html/text-input
                "week"
                (html/attrs
                  (list
                    (html/attr "id" "ksk-week")
                    (html/attr "value" "1")
                    (html/attr "inputmode" "numeric"))))
              (html/label
                (html/attr "for" "ksk-day")
                "Day 1-3")
              (html/text-input
                "day"
                (html/attrs
                  (list
                    (html/attr "id" "ksk-day")
                    (html/attr "value" "1")
                    (html/attr "inputmode" "numeric"))))
              (html/label (html/attr "for" "ksk-load") "Load")
              (html/text-input
                "load"
                (html/attrs
                  (list
                    (html/attr "id" "ksk-load")
                    (html/attr "placeholder" "24")
                    (html/attr "required" "required"))))
              (html/label (html/attr "for" "ksk-unit") "Unit")
              (html/text-input
                "unit"
                (html/attrs
                  (list
                    (html/attr "id" "ksk-unit")
                    (html/attr "value" "kg"))))
              (html/label
                (html/attr "for" "ksk-rm")
                "RM total (3.0 -- L+R max test)")
              (html/text-input
                "rm_total"
                (html/attrs
                  (list
                    (html/attr "id" "ksk-rm")
                    (html/attr "placeholder" "50")
                    (html/attr "inputmode" "numeric"))))
              (html/button
                (html/attr "type" "submit")
                "Start KSK run"))))
          (html/p*
            (html/attr "class" "muted")
            (html/join
              (list
                (html/text "Or ")
                (html/a "/workout/log" "quick log")
                (html/text " a completed free workout."))))))))
```

```
(define workout/state-badge
  (lambda
    (state)
    "Muted badge showing a session state."
    (html/span
      (html/attr "class" "muted")
      (html/text (as-str state)))))

(define workout/str-lt-loop
  (lambda
    (a b i)
    "Compare strings char-by-char from index i."
    (let
      ((la (string-length a)) (lb (string-length b)))
      (if
        (>= i la)
        (if (>= i lb) #f #t)
        (if
          (>= i lb)
          #f
          (let
            ((ca (substring a i 1)) (cb (substring b i 1)))
            (if
              (string-eq ca cb)
              (workout/str-lt-loop a b (+ i 1))
              (< (workout/char-rank ca) (workout/char-rank cb)))))))))

(define workout/str-lt?
  (lambda
    (a b)
    "True when string a sorts before string b."
    (let
      ((sa (as-str a)) (sb (as-str b)))
      (if (string-eq sa sb) #f (workout/str-lt-loop sa sb 0)))))

(define workout/sum
  (lambda
    (xs)
    "Sum of a list of numbers."
    (list/foldl
      (lambda (acc x) (+ acc (workout/as-int x)))
      0
      xs)))

(define workout/timer-script
  (lambda
    (cap-min)
    "Inline JS implementing the countdown timer for a KSK run."
    (html/tag
      "script"
      ""
      (string-append
        "(function(){"
        "var el=document.getElementById('ksk-timer');"
        "if(!el)return;"
        "var cap="
        (as-str (workout/as-int cap-min))
        "*60;"
        "var key='pp.ksk.t.'+ (el.getAttribute('data-sid')||'');"
        "var start=0;"
    "try{start=parseInt(sessionStorage.getItem(key)||'0',10)||0;}catch(e){}"
    "if(!start){start=Date.now();try{sessionStorage.setItem(key,String(start));}catch(e){}}"
        "function tick(){"
        "var s=Math.floor((Date.now()-start)/1000);"
        "var left=Math.max(0,cap-s);"
        "var m=Math.floor(left/60), r=left%60;"
        "el.textContent=(m<10?'0':'')+m+':'+(r<10?'0':'')+r"
        "+(left===0?' DONE':'');"
        "if(left>0)setTimeout(tick,250);"
        "}"
        "tick();"
```

```
        "})();"))))

(define workout/today
  (lambda () "Today's date string." (date-today)))

(define workout/volume
  (lambda
    (exercise from-date to-date weight unit)
    "Total reps of an exercise in the range, filtered by weight/unit."
    (let
      ((ex (workout/normalize-exercise exercise))
        (w (as-str weight))
        (u (workout/normalize-unit unit)))
      (list/foldl
        (lambda
          (acc row)
          (let
            ((f (workout/row-fields row)))
            (if
              (string-eq (workout/field f "exercise") ex)
              (let
                ((day
                  (workout/session-date-of (workout/field f "session_id"))))
                (if
                  (workout/date-in-range? day from-date to-date)
                  (let
                    ((match-w
                        (if
                          (blank? w)
                          #t
                          (string-eq (workout/field f "weight") w)))
                      (match-u
                        (if
                          (blank? u)
                          #t
                          (string-eq
                            (workout/normalize-unit (workout/field f
    "unit"))
                            u))))
                    (if
                      match-w
                      (if
                        match-u
                        (+ acc (workout/as-int (workout/field f "reps")))
                        acc)
                      acc))
                  acc))
              acc)))
        0
        (rows (quote workout-exercise) 1000)))))

(define workout/walk-create
  (lambda
    (date distance unit duration-min location notes)
    "Create a completed walk session."
    (let
      ((d
          (if
            (blank? (as-str date))
            (workout/today)
            (as-str date)))
        (dur
          (if
            (blank? (as-str duration-min))
            "0"
            (as-str duration-min))))
      (create
        (quote workout-session)
        (workout/session-fields
          d
          "walk"
          ""
          "complete"
```

```
          (if (null? notes) "" (as-str notes))
          (workout/walk-doc distance unit dur location)
          (as-str (* (workout/as-int dur) 60))
          (as-str (whoami)))))))

(define workout/walk-detail
  (lambda
    (f)
    "HTML detail card for a walk session."
    (let
      ((doc (workout/session-doc f))
        (dist (workout/pget (workout/session-doc f) "distance"))
        (unit (workout/pget (workout/session-doc f) "unit"))
        (dur
          (workout/pget (workout/session-doc f) "duration_min"))
        (loc (workout/pget (workout/session-doc f) "location"))
        (pace (workout/walk-pace-str (workout/session-doc f))))
      (html/div
        (html/attr "class" "card")
        (html/join
          (list
            (html/h2 "Walk")
            (html/ul
              ""
              (html/join
                (list
                  (html/li "" (html/text (string-append dist " " unit)))
                  (html/li "" (html/text (string-append dur " min")))
                  (if
                    (blank? pace)
                    ""
                    (html/li "" (html/text (string-append pace " /" unit))))
                  (if (blank? loc) "" (html/li "" (html/text loc))))))))))))

(define workout/walk-doc
  (lambda
    (distance unit duration-min location)
    "Session doc dict for a walk."
    (dict-set*
      "{}"
      (list
        "kind"
        "walk"
        "distance"
        (as-str distance)
        "unit"
        (as-str unit)
        "duration_min"
        (as-str duration-min)
        "location"
        (as-str location)))))

(define workout/walk-form
  (lambda
    ()
    "HTML card form to log a walk."
    (html/div
      (html/attr "class" "card")
      (html/join
        (list
          (html/h2 "Log a walk")
          (html/p*
            (html/attr "class" "muted")
            (html/text "Distance, duration, and pace."))
          (html/form
            "POST"
            "/workout/log"
            ""
            (html/join
              (list
                (html/hidden "action" "quick-walk")
                (html/label (html/attr "for" "wk-date") "Date")
                (html/text-input
```

```
                  "date"
                  (html/attrs
                    (list
                      (html/attr "id" "wk-date")
                      (html/attr "value" (workout/today))
                      (html/attr "placeholder" "YYYY-MM-DD"))))
                (html/label (html/attr "for" "wk-dist") "Distance")
                (html/text-input
                  "distance"
                  (html/attrs
                    (list
                      (html/attr "id" "wk-dist")
                      (html/attr "required" "required")
                      (html/attr "inputmode" "decimal")
                      (html/attr "placeholder" "3.2"))))
                (html/label (html/attr "for" "wk-unit") "Unit")
                (html/select
                  "unit"
                  (html/attr "id" "wk-unit")
                  (html/join
                    (list
                      (html/option "mi" "mi" #t)
                      (html/option "km" "km" #f))))
                (html/label (html/attr "for" "wk-dur") "Duration (min)")
                (html/text-input
                  "duration"
                  (html/attrs
                    (list
                      (html/attr "id" "wk-dur")
                      (html/attr "required" "required")
                      (html/attr "inputmode" "numeric")
                      (html/attr "placeholder" "45"))))
                (html/label
                  (html/attr "for" "wk-loc")
                  "Location (optional)")
                (html/text-input
                  "location"
                  (html/attrs
                    (list
                      (html/attr "id" "wk-loc")
                      (html/attr "placeholder" "neighborhood / trail"))))
                (html/label
                  (html/attr "for" "wk-notes")
                  "Notes (optional)")
                (html/textarea
                  "notes"
                  (html/attrs
                    (list
                      (html/attr "id" "wk-notes")
                      (html/attr "rows" "2")))
                  "")
                (html/button (html/attr "type" "submit") "Save
    walk")))))))))

(define workout/walk-li-summary
  (lambda
    (f)
    "One-line summary of a walk session."
    (let
      ((dist (workout/pget (workout/session-doc f) "distance"))
        (unit (workout/pget (workout/session-doc f) "unit"))
        (dur
          (workout/pget (workout/session-doc f) "duration_min"))
        (pace (workout/walk-pace-str (workout/session-doc f))))
      (if
        (blank? pace)
        (string-append
          dist
          " "
          unit
          " — "
          dur
          " min")
```

```
        (string-append
          dist
          " "
          unit
          " - "
          dur
          " min - "
          pace
          " /"
          unit)))))

(define workout/walk-pace-str
  (lambda
    (doc)
    "Pace string (mm:ss per unit) for a walk doc, \"\" when no distance."
    (let
      ((dist-str (workout/pget doc "distance"))
        (dur-str (workout/pget doc "duration_min")))
      (if
        (or (blank? dist-str) (blank? dur-str))
        ""
        (let
          ((dist (parse-float dist-str))
            (dur (parse-float dur-str)))
          (if
            (= dist 0.0)
            ""
            (let
              ((pace-sec (/ (* dur 60.0) dist)))
              (let
                ((pace-min (floor (/ pace-sec 60.0))))
                (let
                  ((sec-rem (round (- pace-sec (* pace-min 60.0)))))
                  (string-append
                    (as-str pace-min)
                    ":"
                    (if
                      (< sec-rem 10)
                      (string-append "0" (as-str sec-rem))
                      (as-str sec-rem)))))))))))
```

229 libs/workout/log/lib.pp

```
;; package workout/log -- exported from live branch system/main
;; Sync: scripts/pp_sync.py pull|push

(define workout/log/ksk-write
  (lambda
    (date
      program
      intensity
      doc
      load
      unit
      total-l
      total-r
      notes
      rungs-l-str
      rungs-r-str)
    "Log a completed workout session, writing snatch exercise rows."
    (let*
      ((tl (workout/as-int total-l))
        (tr (workout/as-int total-r))
        (rsl (as-str rungs-l-str))
        (rsr (as-str rungs-r-str))
        (prog0 (workout/doc-progress doc))
        (prog
          (dict-set*
            "{}"
            (list
              "ladder"
              (workout/pget prog0 "ladder")
              "rung_index"
              (workout/pget prog0 "rung_index")
              "side"
              "L"
              "reps_l"
              (as-str tl)
              "reps_r"
              (as-str tr)
              "sets_done"
              "0"
              "started"
              ""
              "elapsed_sec"
              ""
              "logged"
              "true"
              "rungs_l"
              rsl
              "rungs_r"
              rsr)))
        (n
          (if
            (blank? (as-str notes))
            (if
              (string-eq (as-str tl) (as-str tr))
              (string-append
                (as-str program)
                " log: "
                rsl
                " @ "
                (as-str load)
                (as-str unit)
                " ("
                (as-str tl)
                "/hand)")
              (string-append
                (as-str program)
                " log L:"
                rsl
                " R:"
                rsr
                " @ "
```

```
                (as-str load)
                (as-str unit)
                " ("
                (as-str tl)
                "/"
                (as-str tr)
                ")"))
            (as-str notes)))
        (doc2 (dict-set doc "progress" prog))
        (sid
          (create
            "workout-session"
            (workout/session-fields
              (if
                (blank? (as-str date))
                (workout/today)
                (as-str date))
              (as-str program)
              (as-str intensity)
              "complete"
              n
              doc2
              ""
              (as-str (whoami))))))
      (seq
        (unless
          (< tl 1)
          (workout/exercise-log
            sid
            "Snatch L"
            load
            unit
            tl
            ""
            rsl))
        (unless
          (< tr 1)
          (workout/exercise-log
            sid
            "Snatch R"
            load
            unit
            tr
            ""
            rsr))
        sid))))

(define workout/log/ksk1
  (lambda
    (date
      phase
      day
      load
      unit
      rungs
      notes)
    "Log a ksk1 ladder workout with equal totals for both hands."
    (let
      ((doc (workout/ksk1-plan phase day load unit))
        (total (workout/sum rungs))
        (rs (workout/join-nums rungs))
        (inten
          (string-append "p" (as-str phase) "d" (as-str day)))
        (n (if (null? notes) "" notes)))
      (workout/log/ksk-write
        date
        "ksk1"
        inten
        doc
        load
        unit
        total
        total
```

```
        n
        rs
        rs))))

(define workout/log/ksk1-sides
  (lambda
    (date
      phase
      day
      load
      unit
      rungs-l
      rungs-r
      notes)
    "Log a ksk1 workout with separate left and right rung totals."
    (let
      ((doc (workout/ksk1-plan phase day load unit))
        (tl (workout/sum rungs-l))
        (tr (workout/sum rungs-r))
        (rsl (workout/join-nums rungs-l))
        (rsr (workout/join-nums rungs-r))
        (inten
          (string-append "p" (as-str phase) "d" (as-str day)))
        (n (if (null? notes) "" notes)))
      (workout/log/ksk-write
        date
        "ksk1"
        inten
        doc
        load
        unit
        tl
        tr
        n
        rsl
        rsr))))

(define workout/log/ksk2
  (lambda
    (date
      phase
      day
      load
      unit
      sets
      notes)
    "Log a ksk2 workout from its set list."
    (let
      ((doc (workout/ksk2-plan phase day load unit))
        (total (workout/sum sets))
        (rs (workout/join-nums sets))
        (inten
          (string-append "p" (as-str phase) "d" (as-str day)))
        (n (if (null? notes) "" notes)))
      (workout/log/ksk-write
        date
        "ksk2"
        inten
        doc
        load
        unit
        total
        total
        n
        rs
        rs))))

(define workout/log/ksk3
  (lambda
    (date
      week
      day
      load
```

```
      unit
      rm-total
      sets
      notes)
    "Log a ksk3 workout from its week/day and set list."
    (let
      ((doc (workout/ksk3-plan week day load unit rm-total))
        (total (workout/sum sets))
        (rs (workout/join-nums sets))
        (inten
          (string-append "w" (as-str week) "d" (as-str day)))
        (n (if (null? notes) "" notes)))
      (workout/log/ksk-write
        date
        "ksk3"
        inten
        doc
        load
        unit
        total
        total
        n
        rs
        rs))))
```

230 lisp/bin/repl.ml

```
(** Pricklypear Lisp -- standalone dev REPL.

   No image, no Postgres, no SSH. Reads s-expressions from stdin,
   evaluates them, prints results. Useful for testing the interpreter
   without booting the full image.

   Dev convenience only -- not the image REPL. The live system uses
   browser /repl, POST /api/eval, and an optional loopback TCP line
   protocol on the image.

   H1.7: interpreter is direct-style via [Control.run_sync]. *)
open Pricklypear_lisp

let prompt = "lisp> "

let run_one env prims input : unit =
  (try
     let v = Eval.eval_string ~prims env input in
     print_endline (Reader.pp_value v)
   with
   | Reader.Reader_error (msg, ln, col) ->
     Printf.printf "Read error (line %d, col %d): %s\n" ln col msg
   | (Ast.Interp_error _ as exn) ->
     Printf.printf "Error: %s\n" (Printexc.to_string exn))

let rec loop env prims =
  print_string prompt;
  flush stdout;
  match input_line stdin with
  | exception End_of_file -> print_endline "bye"
  | line ->
    if String.trim line = "" then loop env prims
    else
      let () =
        if not (Reader.is_complete line) then
          let buf = Buffer.create 64 in
          Buffer.add_string buf line;
          let rec more () =
            print_string "  ... ";
            flush stdout;
            (match input_line stdin with
             | exception End_of_file -> print_endline "(aborted)"
             | l ->
                Buffer.add_char buf '\n';
                Buffer.add_string buf l;
                if Reader.is_complete (Buffer.contents buf) then ()
                else more ())
          in
          more ();
          run_one env prims (Buffer.contents buf)
        else
          run_one env prims line
      in
      loop env prims

let () =
  let env = Env.create () in
  let prims = Eval.make_prim_table () in
  Eval.register_prim prims "+"
    (fun args ->
      let n =
        List.fold_left (fun acc v ->
          match v with Ast.VInt i -> Int64.add acc i | _ -> acc) 0L args
      in
      Ast.VInt n);
  Eval.register_prim prims "println"
    (fun args ->
      List.iter (fun v -> print_endline (Reader.pp_value v)) args;
      Ast.VNil);
  print_endline "Pricklypear Lisp dev REPL. Ctrl-D to quit.";
  loop env prims
```

231 lisp/conformance/conformance.ml

```
(** Nopales core conformance case data -- single source of truth.

    Shared between the alcotest runner ([lisp/test/conformance_test.ml])
    and the js_of_ocaml browser-target runner
    ([client/conformance/main.ml]). Spec [lisp-interpreter.conformance] +
    [structural-editing.client-eval]: client/server agreement is a tested
    invariant, not a hope. *)
open Pricklypear_lisp

type case =
  { group : string
  ; name : string
  ; setups : string list
  ; src : string
  ; expected : string
  }

(* Minimal pure prim table for the core gate. The full product prim table
   lives in pure/prims_pure.ml; conformance pins the EVALUATOR semantics
   with a tiny arith/list surface so the gate is service-free. *)
let setup_arith () =
  let prims = Eval.make_prim_table () in
  let binop name op =
    Eval.register_prim prims name (function
      | [ Ast.VInt a; Ast.VInt b ] -> Ast.VInt (op a b)
      | _ -> raise (Ast.Interp_error (Ast.User_error (name, "bad-args"))))
  in
  binop "+" Int64.add;
  binop "-" Int64.sub;
  binop "*" Int64.mul;
  Eval.register_prim prims "=" (function
    | [ Ast.VInt a; Ast.VInt b ] -> Ast.VBool (Int64.equal a b)
    | [ a; b ] -> Ast.VBool (a = b)
    | _ -> raise (Ast.Interp_error (Ast.User_error ("=", "bad-args"))));
  Eval.register_prim prims "<" (function
    | [ Ast.VInt a; Ast.VInt b ] -> Ast.VBool (a < b)
    | _ -> raise (Ast.Interp_error (Ast.User_error ("<", "bad-args"))));
  Eval.register_prim prims "not" (function
    | [ Ast.VBool b ] -> Ast.VBool (not b)
    | [ Ast.VNil ] -> Ast.VBool true
    | [ _ ] -> Ast.VBool false
    | _ -> raise (Ast.Interp_error (Ast.User_error ("not", "bad-args"))));
  Eval.register_prim prims "list" (fun xs -> Ast.VList xs);
  Eval.register_prim prims "car" (function
    | [ Ast.VList (h :: _) ] -> h
    | [ Ast.VPair (a, _) ] -> a
    | _ -> raise (Ast.Interp_error (Ast.User_error ("car", "bad-args"))));
  Eval.register_prim prims "cdr" (function
    | [ Ast.VList (_ :: t) ] -> Ast.VList t
    | [ Ast.VPair (_, b) ] -> b
    | _ -> raise (Ast.Interp_error (Ast.User_error ("cdr", "bad-args"))));
  Eval.register_prim prims "cons" (function
    | [ h; Ast.VList t ] -> Ast.VList (h :: t)
    | [ h; t ] -> Ast.VPair (h, t)
    | _ -> raise (Ast.Interp_error (Ast.User_error ("cons", "bad-args"))));
  Eval.register_prim prims "null?" (function
    | [ Ast.VNil ] | [ Ast.VList [] ] -> Ast.VBool true
    | [ _ ] -> Ast.VBool false
    | _ -> raise (Ast.Interp_error (Ast.User_error ("null?", "bad-args"))));
  Eval.register_prim prims "symbol?" (function
    | [ Ast.VSymbol _ ] -> Ast.VBool true
    | [ _ ] -> Ast.VBool false
    | _ -> raise (Ast.Interp_error (Ast.User_error ("symbol?",
     "bad-args"))));
  Eval.register_prim prims "string?" (function
    | [ Ast.VStr _ ] -> Ast.VBool true
    | [ _ ] -> Ast.VBool false
    | _ -> raise (Ast.Interp_error (Ast.User_error ("string?",
     "bad-args"))));
  Eval.register_prim prims "symbol=?" (function
    | [ Ast.VSymbol a; Ast.VSymbol b ] -> Ast.VBool (String.equal a b)
```

```
    | [ _; _ ] -> Ast.VBool false
    | _ -> raise (Ast.Interp_error (Ast.User_error ("symbol=?",
     "bad-args"))));
  Eval.register_prim prims "type-of" (function
    | [ Ast.VSymbol _ ] -> Ast.VStr "symbol"
    | [ Ast.VStr _ ] -> Ast.VStr "str"
    | [ Ast.VInt _ ] -> Ast.VStr "int"
    | [ Ast.VBool _ ] -> Ast.VStr "bool"
    | [ Ast.VNil ] -> Ast.VStr "nil"
    | [ Ast.VList _ ] -> Ast.VStr "list"
    | _ -> Ast.VStr "other");
  let env = Env.create () in
  List.iter
    (fun n -> Env.define env n (Ast.VPrim n))
    [ "+"; "-"; "*"; "="; "<"; "not"; "list"; "car"; "cdr"; "cons"; "null?"
    ; "symbol?"; "string?"; "symbol=?"; "type-of" ];
  (prims, env)

let eval_pp ~prims env src =
  Reader.pp_value (Eval.eval_string ~prims env src)

let mk ?(setups = []) group name src expected =
  { group; name; setups; src; expected }

let cases = [
  (* reader *)
  mk "reader" "keyword self-evaluates" ":target" ":target";
  mk "reader" "int literal" "42" "42";
  mk "reader" "string literal" "\"hi\"" "\"hi\"";
  mk "reader" "bool true" "#t" "#t";
  mk "reader" "bool false" "#f" "#f";
  mk "reader" "nil" "()" "()";
  mk "reader" "quote symbol" "'hello" "hello";
  mk "reader" "quote string stays string" "'\"hello\"" "\"hello\"";
  mk "reader" "quote list" "'(a b c)" "(a b c)";
  mk "reader" "unicode \\uXXXX overline" "\"\\u203e\"" "\"\xE2\x80\xBE\"";
  mk "reader" "unicode \\uXXXX degree" "\"71\\u00B0F\"" "\"71\xC2\xB0F\"";
  (* values *)
  mk "values" "symbol vs string type-of symbol" "(type-of 'a)" "\"symbol\"";
  mk "values" "symbol vs string type-of str" "(type-of \"a\")" "\"str\"";
  mk "values" "symbol?" "(symbol? 'lambda)" "#t";
  mk "values" "string? rejects symbol" "(string? 'lambda)" "#f";
  mk "values" "string? accepts str" "(string? \"lambda\")" "#t";
  mk "values" "car of quoted lambda is symbol"
    "(symbol=? (car '(lambda (x) x)) 'lambda)" "#t";
  mk "values" "list constructor" "(list 1 2 3)" "(1 2 3)";
  (* special-forms *)
  mk "special-forms" "if true" "(if #t 1 2)" "1";
  mk "special-forms" "if false" "(if #f 1 2)" "2";
  mk "special-forms" "begin last" "(begin 1 2 3)" "3";
  mk "special-forms" "let" "(let ((x 10)) (+ x 1))" "11";
  mk "special-forms" "lambda app" "((lambda (x) (+ x 1)) 41)" "42";
  mk "special-forms" "and short-circuit true" "(and #t #t 7)" "7";
  mk "special-forms" "and short-circuit false" "(and #t #f 7)" "#f";
  mk "special-forms" "or short-circuit" "(or #f 3 9)" "3";
  mk "special-forms" "cond else" "(cond (#f 1) (else 2))" "2";
  mk "special-forms" "match list head"
    "(match '(lambda (x) y) ((lambda params body) params) (_ 0))" "(x)";
  mk "special-forms" "match fallthrough underscore"
    "(match 42 ((list x) 1) (_ 7))" "7";
  mk "special-forms" "match binds var" "(match 5 (x (+ x 1)))" "6";
  mk "special-forms" "guard recovers" "(guard (lambda (e) 99) (error
     \"boom\"))" "99";
  mk "special-forms" "define sugar"
    ~setups:[ "(define (add1 x) (+ x 1))" ] "(add1 41)" "42";
  mk "special-forms" "define multi-body"
    ~setups:[ "(define (f x) (+ x 0) (+ x 1))" ] "(f 2)" "3";
  mk "special-forms" "defmacro when"
    ~setups:[ "(defmacro when (test body) `(if ,test ,body #f))" ]
    "(when #t 42)" "42";
  (* list-hof *)
  mk "list-hof" "list/map"
    "(list/map (lambda (x) (* x 2)) (list 1 2 3))" "(2 4 6)";
```

```
  mk "list-hof" "list/filter"
    "(list/filter (lambda (x) (= x 2)) (list 1 2 3 2))" "(2 2)";
  mk "list-hof" "list/foldl"
    "(list/foldl (lambda (acc x) (+ acc x)) 0 (list 1 2 3 4))" "10";
  mk "list-hof" "list/foldr"
    "(list/foldr (lambda (x acc) (cons x acc)) (list) (list 1 2 3))" "(1 2
     3)";
  mk "list-hof" "list/map empty" "(list/map (lambda (x) x) (list))" "()";
  (* arith *)
  mk "arith" "add" "(+ 20 22)" "42";
  mk "arith" "nested arith" "(* (+ 1 2) (- 10 4))" "18";
]
```

232 lisp/lib/ast.ml

```
(** Pricklypear Lisp -- AST.

    The AST IS the source (homoiconicity). Function bodies are stored as
    JSONB rows in [function_defs.body], parsed back into [expr] values by
    the reader and evaluated by [Eval].

    Per spec [lisp-interpreter.ast]: this is a Scheme-flavored Lisp-1
    (lexically scoped, first-class closures, primitives dispatched by
    name through a table the IMAGE layer registers at boot).

    The interpreter proper is PURE: it takes an env and an expr and
    returns a value or raises an [Interp_error]. No I/O here. Side
    effects go through [Prim] calls, wired to real Postgres/HTTP by the
    image layer. *)

(** A reference into the persistent store. Carries its target type so
    the interpreter can verify derefs at call time before querying. *)
type type_name = string
type id = string  (* uuid-ish; opaque to the interpreter *)

type expr =
  | Var of string
  | Lit of value
  | App of expr * expr list
  | Let of string * expr * expr
  | Letrec of (string * expr) list * expr
  | If of expr * expr * expr
  | Lambda of string list * expr
  | Prim of string * expr list
  | Seq of expr * expr
  | Quote of expr
  | Define of string * expr   (* top-level binding *)

(** Values are what evaluation produces. The interesting cases are
    [VRef] (a tagged reference into the store, so values can point into
    the persistent object graph without copying it) and [VLambda]
    (closures capture their defining env). *)
and value =
  | VInt of int64
  | VFloat of float                (* IEEE float; arithmetic promotes
     int+float *)
  | VStr of string
  | VSymbol of string              (* code names as data; distinct from
     VStr *)
  | VBool of bool
  | VNil
  | VPair of value * value         (* cons cells *)
  | VLambda of string list * expr * env
  | VMacro of string list * expr * env
      (* defmacro transformer; params bind to unevaluated arg values *)
  | VPrim of string                (* primitive name; resolved at apply *)
  | VRef of type_name * id
  | VList of value list            (* sugar for nested pairs; reader
     interns *)
  | VThunk of thunk_cell ref       (* memoized recipe; forced at
     Env.lookup *)

(** A not-yet-evaluated binding body (lisp-interpreter.lazy-env-thunks).
    The payload carries its OWN evaluator so the pure kernel never
    decides HOW a body runs: pure hosts force under [Eval]'s table,
    the image forces under its prim rim. Memoization covers failure
    too -- a body that raises once replays the exception, it does not
    re-run (and re-pay) on every touch. [Forcing] marks an in-flight
    force so self-referential cycles raise instead of diverging. *)
and thunk_cell =
  | Unforced of thunk_payload
  | Forcing
  | Forced of value
  | Failed of exn

and thunk_payload = {
```

```
  thunk_body : expr;
  thunk_env : env;
  thunk_force : env -> expr -> value;
}

(** Lexical environment: a local binding table plus an optional parent
    closure chain; lookup walks toward the root. *)
and env = {
  bindings : (string, value) Hashtbl.t;
  parent : env option;
}

(** Raised on interpreter errors. The REPL catches these, prints them
    with location if available, and continues the session. *)
type interp_error =
  | Unbound_var of string
  | Not_a_function of value
  | Arity_mismatch of string * int * int  (* (name, expected, got) *)
  | Wrong_arg_count of string * int * int
  | Type_mismatch of string * value  (* (expected, got) *)
  | Budget_exceeded
  | Prim_error of string * string * string  (* (prim_name, message, kind) *)
  | Parse_error of string * int * int (* (message, line, col) *)
  | User_error of string * string         (* (message, kind) *)

exception Interp_error of interp_error

let _ = Obj.repr (fun (_ : interp_error) -> ())  (* keep type referenced *)

(** Human-readable form of an [interp_error]. Used by logging/REPL so a
    refresh_ui_shell warn names the actual failure (Unbound_var x /
    User_error ... / Prim_error ...) instead of the opaque
    `Interp_error(_)` Printexc produces. *)
let rec pp_value_short (v : value) : string =
  match v with
  | VStr s -> "\"" ^ s ^ "\""
  | VSymbol s -> s
  | VInt i -> Int64.to_string i
  | VNil -> "nil"
  | VBool b -> string_of_bool b
  | VFloat f -> string_of_float f
  | VList xs -> "[" ^ String.concat ", " (List.map pp_value_short xs) ^ "]"
  | VPair (a, b) -> "(" ^ pp_value_short a ^ " . " ^ pp_value_short b ^ ")"
  | VLambda _ -> "<lambda>"
  | VMacro _ -> "<macro>"
  | VPrim n -> "<prim " ^ n ^ ">"
  | VRef (t, _) -> "<ref " ^ t ^ ">"
  | VThunk _ -> "<thunk>"

(** Machine-readable kind tag for an [interp_error] (used by CLI/UI code).
    *)
let interp_error_kind (e : interp_error) : string =
  match e with
  | Unbound_var _ -> "unbound_var"
  | Not_a_function _ -> "not_a_function"
  | Arity_mismatch _ -> "bad-arity"
  | Wrong_arg_count _ -> "bad-arity"
  | Type_mismatch _ -> "type_mismatch"
  | Budget_exceeded -> "budget_exceeded"
  | Prim_error (_, _, k) -> k
  | Parse_error _ -> "parse_error"
  | User_error (_, k) -> k

(** Source location (line, col) carried by an error, if any
    ([Parse_error]). *)
let interp_error_loc (e : interp_error) : (int * int) option =
  match e with
  | Parse_error (_, line, col) -> Some (line, col)
  | _ -> None

(** Human-readable rendering of an [interp_error] for logs/REPL output. *)
let pp_interp_error (e : interp_error) : string =
  match e with
```

```
  | Unbound_var s -> "unbound: " ^ s
  | Not_a_function v -> "not a function: " ^ pp_value_short v
  | Arity_mismatch (n, exp, got) ->
      Printf.sprintf "arity mismatch in %s: expected %d, got %d" n exp got
  | Wrong_arg_count (n, exp, got) ->
      Printf.sprintf "wrong arg count in %s: expected %d, got %d" n exp got
  | Type_mismatch (exp, v) ->
      "type mismatch: expected " ^ exp ^ ", got " ^ pp_value_short v
  | Budget_exceeded -> "budget exceeded"
  | Prim_error (p, m, _) -> "prim " ^ p ^ ": " ^ m
  | Parse_error (m, l, c) -> Printf.sprintf "parse error: %s (%d:%d)" m l c
  | User_error (m, _) -> m

(** Helper constructors for building AST in tests *)
let var s = Var s
let int n = Lit (VInt (Int64.of_int n))
let float f = Lit (VFloat f)
let str s = Lit (VStr s)
let bool b = Lit (VBool b)
let nil = Lit VNil

(** JSON serialization of [expr].

    Function bodies are stored as JSONB. Round-trips via [of_json]. *)
let rec expr_to_json (e : expr) : Yojson.Safe.t =
  match e with
  | Var s -> `Assoc [ ("var", `String s) ]
  | Lit v -> `Assoc [ ("lit", value_to_json v) ]
  | App (f, args) ->
      `Assoc [ ("app", `List (expr_to_json f :: List.map expr_to_json
     args)) ]
  | Let (name, val_expr, body) ->
      `Assoc [
        ("let", `List [ `String name; expr_to_json val_expr; expr_to_json
     body ]);
      ]
  | Letrec (bindings, body) ->
      `Assoc [
        ( "letrec",
          `List [
            `List (List.map (fun (n, e) -> `List [ `String n; expr_to_json
     e ]) bindings);
            expr_to_json body;
          ] );
      ]
  | If (test, then_e, else_e) ->
      `Assoc [ ("if", `List [ expr_to_json test; expr_to_json then_e;
     expr_to_json else_e ]) ]
  | Lambda (params, body) ->
      `Assoc [ ("lambda", `List [ `List (List.map (fun s -> `String s)
     params); expr_to_json body ]) ]
  | Prim (name, args) ->
      `Assoc [ ("prim", `List [ `String name; `List (List.map expr_to_json
     args) ]) ]
  | Seq (a, b) -> `Assoc [ ("seq", `List [ expr_to_json a; expr_to_json b
     ]) ]
  | Quote e -> `Assoc [ ("quote", expr_to_json e) ]
  | Define (name, val_expr) ->
      `Assoc [ ("define", `List [ `String name; expr_to_json val_expr ]) ]

and value_to_json (v : value) : Yojson.Safe.t =
  match v with
  | VInt n -> `Assoc [ ("int", `String (Int64.to_string n)) ]
  | VFloat f -> `Assoc [ ("float", `Float f) ]
  | VStr s -> `Assoc [ ("str", `String s) ]
  | VSymbol s -> `Assoc [ ("sym", `String s) ]
  | VBool b -> `Assoc [ ("bool", `Bool b) ]
  | VNil -> `Assoc [ ("nil", `Null) ]
  | VPair (a, b) -> `Assoc [ ("pair", `List [ value_to_json a;
     value_to_json b ]) ]
  | VLambda _ -> `Assoc [ ("lambda", `String "#<lambda>") ]  (* closures
     aren't serializable *)
  | VMacro _ -> `Assoc [ ("macro", `String "#<macro>") ]
```

```
  | VThunk _ -> `Assoc [ ("thunk", `String "#<thunk>") ]
  | VPrim name -> `Assoc [ ("prim", `String name) ]
  | VRef (t, i) -> `Assoc [ ("ref", `List [ `String t; `String i ]) ]
  | VList vs -> `Assoc [ ("list", `List (List.map value_to_json vs)) ]

(** JSON deserialization of [expr]. Inverse of [expr_to_json]. *)
let rec expr_of_json (j : Yojson.Safe.t) : (expr, string) result =
  match j with
  | `Assoc [ ("var", `String s) ] -> Ok (Var s)
  | `Assoc [ ("lit", v) ] ->
      (match value_of_json v with
       | Ok v -> Ok (Lit v)
       | Error e -> Error e)
  | `Assoc [ ("app", `List (f :: args)) ] ->
      (match expr_of_json f with
       | Ok f ->
           let args' =
             List.fold_left
               (fun acc a ->
                 match acc, expr_of_json a with
                 | Ok lst, Ok e -> Ok (lst @ [ e ])
                 | Error e, _ -> Error e
                 | _, Error e -> Error e)
               (Ok []) args
           in
           (match args' with Ok lst -> Ok (App (f, lst)) | Error e -> Error e)
       | Error e -> Error e)
  | `Assoc [ ("let", `List [ `String name; v; b ]) ] ->
      (match expr_of_json v, expr_of_json b with
       | Ok v, Ok b -> Ok (Let (name, v, b))
       | Error e, _ | _, Error e -> Error e)
  | `Assoc [ ("letrec", `List [ `List bindings; b ]) ] ->
      let bindings' =
        List.fold_left
          (fun acc bj ->
            match acc, bj with
            | Ok lst, `List [ `String n; e ] ->
                (match expr_of_json e with
                 | Ok e -> Ok (lst @ [ (n, e) ])
                 | Error e -> Error e)
            | Ok _, _ -> Error "letrec binding not a [name, expr] pair"
            | Error e, _ -> Error e)
          (Ok []) bindings
      in
      (match bindings', expr_of_json b with
       | Ok bs, Ok b -> Ok (Letrec (bs, b))
       | Error e, _ | _, Error e -> Error e)
  | `Assoc [ ("if", `List [ t; th; el ]) ] ->
      (match expr_of_json t, expr_of_json th, expr_of_json el with
       | Ok t, Ok th, Ok el -> Ok (If (t, th, el))
       | Error e, _, _
       | _, Error e, _
       | _, _, Error e -> Error e)
  | `Assoc [ ("lambda", `List [ `List params; b ]) ] ->
      let params' =
        List.fold_left
          (fun acc p ->
            match acc, p with
            | Ok lst, `String s -> Ok (lst @ [ s ])
            | Ok _, _ -> Error "lambda param not a string"
            | Error e, _ -> Error e)
          (Ok []) params
      in
      (match params', expr_of_json b with
       | Ok ps, Ok b -> Ok (Lambda (ps, b))
       | Error e, _ | _, Error e -> Error e)
  | `Assoc [ ("prim", `List [ `String name; `List args ]) ] ->
      let args' =
        List.fold_left
          (fun acc a ->
            match acc, expr_of_json a with
            | Ok lst, Ok e -> Ok (lst @ [ e ])
```

```
            | Error e, _ -> Error e
            | _, Error e -> Error e)
          (Ok []) args
      in
      (match args' with Ok lst -> Ok (Prim (name, lst)) | Error e -> Error e)
  | `Assoc [ ("seq", `List [ a; b ]) ] ->
      (match expr_of_json a, expr_of_json b with
       | Ok a, Ok b -> Ok (Seq (a, b))
       | Error e, _ | _, Error e -> Error e)
  | `Assoc [ ("quote", e) ] ->
      (match expr_of_json e with Ok e -> Ok (Quote e) | Error e -> Error e)
  | `Assoc [ ("define", `List [ `String name; v ]) ] ->
      (match expr_of_json v with
       | Ok v -> Ok (Define (name, v))
       | Error e -> Error e)
  | _ -> Error ("unrecognized expr JSON: " ^ Yojson.Safe.to_string j)

and value_of_json (j : Yojson.Safe.t) : (value, string) result =
  match j with
  | `Assoc [ ("int", `String s) ] ->
      (try Ok (VInt (Int64.of_string s))
       with _ -> Error ("bad int: " ^ s))
  | `Assoc [ ("float", `Float f) ] -> Ok (VFloat f)
  | `Assoc [ ("float", `Int n) ] -> Ok (VFloat (float_of_int n))
  | `Assoc [ ("float", `String s) ] ->
      (try Ok (VFloat (float_of_string s))
       with _ -> Error ("bad float: " ^ s))
  | `Assoc [ ("str", `String s) ] -> Ok (VStr s)
  | `Assoc [ ("sym", `String s) ] -> Ok (VSymbol s)
  | `Assoc [ ("bool", `Bool b) ] -> Ok (VBool b)
  | `Assoc [ ("nil", `Null) ] -> Ok VNil
  | `Assoc [ ("pair", `List [ a; b ]) ] ->
      (match value_of_json a, value_of_json b with
       | Ok a, Ok b -> Ok (VPair (a, b))
       | Error e, _ | _, Error e -> Error e)
  | `Assoc [ ("prim", `String name) ] -> Ok (VPrim name)
  | `Assoc [ ("ref", `List [ `String t; `String i ]) ] -> Ok (VRef (t, i))
  | `Assoc [ ("list", `List lst) ] ->
      let lst' =
        List.fold_left
          (fun acc v ->
            match acc, value_of_json v with
            | Ok l, Ok v -> Ok (l @ [ v ])
            | Error e, _ -> Error e
            | _, Error e -> Error e)
          (Ok []) lst
      in
      (match lst' with Ok lst -> Ok (VList lst) | Error e -> Error e)
  | _ -> Error ("unrecognized value JSON: " ^ Yojson.Safe.to_string j)
```

233 lisp/lib/ast_diff.ml

```
(** Pure structural diff over [Ast.expr] trees.

    Consumer: [branching.diff] -- [Store.diff_branches] feeds the parsed
    from/to bodies of a modified [function_defs] row into [diff] so the
    /diff page can overlay the delta tree on [Ast_html.pp_expr_html].

    No I/O, no Postgres -- tests with [dune runtest lisp] alone, like
    reader / eval / env.

    Semantics authority: [branching.diff]. This module is the pure
    walker (lisp-interpreter.ast-diff), NOT the surface prim -- a
    programmatic [(diff-ast name from to)] prim is explicit v2.

    Non-goals (v1, per branching.diff): rename detection (a bound-var
    rename surfaces as Replaced at the binding site AND every reference
    site), alpha-equivalence (bound names are NOT normalized), and
    param-order is a *positional* child delta, not "renamed params". *)

let sha1_hex (s : string) : string =
  let d = Digestif.SHA1.digest_string s in
  Digestif.SHA1.to_hex d

(** Content identity: sha1 hex of [Reader.pp_expr]. Bytes-identical to
    [Analyze.ast_hash] in image/lib/analyze.ml (same sha1_hex over the
    same pp_expr string) so the fast path below agrees with the SQL
    layer's hashes. *)
let ast_hash (e : Ast.expr) : string = sha1_hex (Reader.pp_expr e)

type delta =
  | Same
  | Replaced of { from : Ast.expr; to_ : Ast.expr }
  | Node of { ctor : string; children : (string * delta) list }

(** Sentinel for the absent side of a list-level add/remove. The locked
    [delta] type has no Added/Removed variant, so an element present on
    only one side of a list (App arg / Letrec binding / Lambda param) is
    a [Replaced] whose other side is this sentinel. A renderer can
    present a [Replaced] where one side is the sentinel as pure
    add/remove. *)
let absent = Ast.nil

(** Longest common subsequence of index pairs, keyed by [key]. Result
    pairs are strictly increasing in both indices (standard LCS
    back-track). *)
let lcs_pairs (key : 'a -> string) (xs : 'a list) (ys : 'a list) :
    (int * int) list =
  let xa = Array.of_list xs and ya = Array.of_list ys in
  let n = Array.length xa and m = Array.length ya in
  let dp = Array.make_matrix (n + 1) (m + 1) 0 in
  for i = n - 1 downto 0 do
    for j = m - 1 downto 0 do
      dp.(i).(j) <-
        if key xa.(i) = key ya.(j) then 1 + dp.(i + 1).(j + 1)
        else max dp.(i + 1).(j) dp.(i).(j + 1)
    done
  done;
  let rec go i j acc =
    if i < n && j < m then
      if key xa.(i) = key ya.(j) then go (i + 1) (j + 1) ((i, j) :: acc)
      else if dp.(i + 1).(j) >= dp.(i).(j + 1) then go (i + 1) j acc
      else go i (j + 1) acc
    else List.rev acc
  in
  go 0 0 []

(** List children from a list-LCS over [xs]/[ys] matched at [pairs].
    Matched pairs recurse [diff]; a lone element on only one side is
    converted to a delta by [wrap] (which decides which side is absent
    from [side]). Generic over the element type so it serves both App
    args (expr) and Letrec bindings (string * expr). *)
let list_children ~(label : 'a -> string) ~(diff : 'a -> 'a -> delta)
```

```
    ~(wrap : [ `Left | `Right ] -> 'a -> delta)
    (xs : 'a list) (ys : 'a list) (pairs : (int * int) list) :
    (string * delta) list =
  let xa = Array.of_list xs and ya = Array.of_list ys in
  let n = Array.length xa and m = Array.length ya in
  let pair_map = Hashtbl.create 16 in
  List.iter (fun (i, j) -> Hashtbl.replace pair_map i j) pairs;
  let out = ref [] in
  let i = ref 0 and j = ref 0 in
  while !i < n || !j < m do
    if !i < n && Hashtbl.mem pair_map !i then begin
      let jj = Hashtbl.find pair_map !i in
      while !j < jj do
        out := (label ya.(!j), wrap `Right ya.(!j)) :: !out;
        incr j
      done;
      out := (label xa.(!i), diff xa.(!i) ya.(!j)) :: !out;
      incr i;
      incr j
    end
    else if !i < n then begin
      out := (label xa.(!i), wrap `Left xa.(!i)) :: !out;
      incr i
    end
    else begin
      out := (label ya.(!j), wrap `Right ya.(!j)) :: !out;
      incr j
    end
  done;
  List.rev !out

(** Positional children: pair [xs]/[ys] up by index (no LCS). Used where
    order matters and is meant to be visible (Lambda params -- see
    non-goal "param order matters"). Generic via [wrap] like
    [list_children]. *)
let positional_children ~(label : 'a -> string) ~(diff : 'a -> 'a -> delta)
    ~(wrap : [ `Left | `Right ] -> 'a -> delta)
    (xs : 'a list) (ys : 'a list) : (string * delta) list =
  let rec go xs ys acc =
    match xs, ys with
    | [], [] -> List.rev acc
    | x :: xr, y :: yr -> go xr yr ((label x, diff x y) :: acc)
    | x :: xr, [] -> go xr [] ((label x, wrap `Left x) :: acc)
    | [], y :: yr -> go [] yr ((label y, wrap `Right y) :: acc)
  in
  go xs ys []

(** Structural diff between two exprs.

    Fast path first: if the content hashes are equal, the trees are
    identical and we skip the walk entirely (returns [Same]). The 90%
    common case (unchanged defs between branches) costs one hash
    compare per row, not a tree walk. *)
let rec diff (from_e : Ast.expr) (to_e : Ast.expr) : delta =
  if ast_hash from_e = ast_hash to_e then Same else walk from_e to_e

and walk (f : Ast.expr) (t : Ast.expr) : delta =
  let wrap_expr side e =
    match side with
    | `Left -> Replaced { from = e; to_ = absent }
    | `Right -> Replaced { from = absent; to_ = e }
  in
  match f, t with
  (* Leaf constructors: structural = on the payload. *)
  | Var a, Var b -> if a = b then Same else Replaced { from = f; to_ = t }
  | Lit a, Lit b -> if a = b then Same else Replaced { from = f; to_ = t }
  (* Fixed-arity constructors: recurse positionally. *)
  | If (a1, a2, a3), If (b1, b2, b3) ->
      Node {
        ctor = "if";
        children = [
          ("test", diff a1 b1);
          ("then", diff a2 b2);
```

```
        ("else", diff a3 b3);
      ];
    }
| Seq (a1, a2), Seq (b1, b2) ->
    Node { ctor = "seq"; children = [ ("a", diff a1 b1); ("b", diff a2 b2) ] }
| Lambda (ps1, b1), Lambda (ps2, b2) ->
    let params =
      Node {
        ctor = "params";
        children =
          positional_children ~label:(fun _ -> "param")
            ~diff:(fun a b ->
              if a = b then Same
              else Replaced { from = Ast.var a; to_ = Ast.var b })
            ~wrap:(fun side s ->
              match side with
              | `Left -> Replaced { from = Ast.var s; to_ = absent }
              | `Right -> Replaced { from = absent; to_ = Ast.var s })
            ps1 ps2;
      }
    in
    Node { ctor = "lambda"; children = [ ("params", params); ("body", diff b1 b2) ] }
| Let (n1, v1, b1), Let (n2, v2, b2) ->
    Node {
      ctor = "let";
      children = [
        ("name", if n1 = n2 then Same else Replaced { from = Ast.var n1; to_ = Ast.var n2 });
        ("val", diff v1 v2);
        ("body", diff b1 b2);
      ];
    }
| Define (n1, v1), Define (n2, v2) ->
    Node {
      ctor = "define";
      children = [
        ("name", if n1 = n2 then Same else Replaced { from = Ast.var n1; to_ = Ast.var n2 });
        ("val", diff v1 v2);
      ];
    }
| Quote e1, Quote e2 -> Node { ctor = "quote"; children = [ ("body", diff e1 e2) ] }
(* List children: App head recurses; args use list-LCS keyed on hash so
   an insertion mid-sequence does not mark every following arg changed. *)
| App (fh, args1), App (th, args2) ->
    Node {
      ctor = "app";
      children =
        ("fn", diff fh th)
        :: list_children ~label:(fun _ -> "arg") ~diff ~wrap:wrap_expr
             args1 args2 (lcs_pairs ast_hash args1 args2);
    }
| Prim (n1, a1), Prim (n2, a2) ->
    if n1 <> n2 then Replaced { from = f; to_ = t }
    else
      Node {
        ctor = "prim";
        children =
          list_children ~label:(fun _ -> "arg") ~diff ~wrap:wrap_expr a1 a2
            (lcs_pairs ast_hash a1 a2);
      }
| Letrec (bs1, b1), Letrec (bs2, b2) ->
    Node {
      ctor = "letrec";
      children =
        list_children ~label:(fun (n, _) -> n)
          ~diff:(fun (_, e1) (_, e2) -> diff e1 e2)
          ~wrap:(fun side (_, e) ->
            match side with
```

```
              | `Left -> Replaced { from = e; to_ = absent }
              | `Right -> Replaced { from = absent; to_ = e })
            bs1 bs2 (lcs_pairs (fun (n, _) -> n) bs1 bs2)
          @ [ ("body", diff b1 b2) ];
      }
  (* Different constructors: structural replacement. *)
  | _, _ -> Replaced { from = f; to_ = t }
```

234 lisp/lib/complexity.ml

```
(** Heuristic big-O classifier for Nopales ASTs.

    Honest [Unknown] default. Pattern-matches common shapes only.
    Pure: no I/O. *)
open Ast
open Recursion

(** Asymptotic complexity class of a Nopales AST. [Unknown] when
    no recognized shape matches. *)
type complexity =
  | O_constant
  | O_log_n
  | O_linear
  | O_n_log_n
  | O_quadratic
  | O_exponential
  | Unknown

(** Human-readable badge for a complexity class (e.g. ["O(n?)"]). *)
let badge = function
  | O_constant -> "O(1)"
  | O_log_n -> "O(log n)"
  | O_linear -> "O(n)"
  | O_n_log_n -> "O(n log n)"
  | O_quadratic -> "O(n?)"
  | O_exponential -> "O(2?)"
  | Unknown -> "?"

(** The more expensive of two complexity classes (by rank). *)
let max_cx a b =
  let rank = function
    | O_constant -> 0
    | O_log_n -> 1
    | O_linear -> 2
    | O_n_log_n -> 3
    | O_quadratic -> 4
    | O_exponential -> 5
    | Unknown -> 6
  in
  if rank a >= rank b then a else b

(** Is [n] the name of a list-iteration primitive ([map], [foldl], ?)? *)
let is_list_iter_prim = function
  | "list/map" | "list/filter" | "list/foldl" | "map" | "filter" | "foldl"
  | "for-each" -> true
  | _ -> false

(** Is [n] the name of a DB-scan primitive ([rows], [find-rows], ?)? *)
let is_db_scan_prim = function
  | "rows" | "find-rows" | "find-text" -> true
  | _ -> false

(** Does [e] reach any DB-scan primitive (or call a function that does)? *)
let rec mentions_db_scan (e : expr) : bool =
  match e with
  | Prim (n, args) -> is_db_scan_prim n || List.exists mentions_db_scan args
  | App (Var n, args) when is_db_scan_prim n -> true
  | App (f, args) -> mentions_db_scan f || List.exists mentions_db_scan args
  | Let (_, a, b) | Seq (a, b) -> mentions_db_scan a || mentions_db_scan b
  | Letrec (bs, body) ->
      List.exists (fun (_, e) -> mentions_db_scan e) bs || mentions_db_scan
     body
  | If (a, b, c) ->
      mentions_db_scan a || mentions_db_scan b || mentions_db_scan c
  | Lambda (_, body) | Define (_, body) -> mentions_db_scan body
  | Var _ | Lit _ | Quote _ -> false

(** Does [e] contain any list-iteration primitive? *)
let rec has_list_iter (e : expr) : bool =
  match e with
  | Prim (n, args) -> is_list_iter_prim n || List.exists has_list_iter args
```

```
  | App (Var n, args) when is_list_iter_prim n -> true
  | App (f, args) -> has_list_iter f || List.exists has_list_iter args
  | Let (_, a, b) | Seq (a, b) -> has_list_iter a || has_list_iter b
  | Letrec (bs, body) ->
      List.exists (fun (_, e) -> has_list_iter e) bs || has_list_iter body
  | If (a, b, c) -> has_list_iter a || has_list_iter b || has_list_iter c
  | Lambda (_, body) | Define (_, body) -> has_list_iter body
  | Var _ | Lit _ | Quote _ -> false

(** Nested list-iter or list-iter over a DB scan ? quadratic-ish. *)
let rec has_nested_iter (e : expr) : bool =
  let iter_body_has_iter = function
    | Prim (n, args) when is_list_iter_prim n ->
        List.exists has_list_iter args || List.exists mentions_db_scan args
    | App (Var n, args) when is_list_iter_prim n ->
        List.exists has_list_iter args || List.exists mentions_db_scan args
    | _ -> false
  in
  match e with
  | e when iter_body_has_iter e -> true
  | App (f, args) -> has_nested_iter f || List.exists has_nested_iter args
  | Prim (_, args) -> List.exists has_nested_iter args
  | Let (_, a, b) | Seq (a, b) -> has_nested_iter a || has_nested_iter b
  | Letrec (bs, body) ->
      List.exists (fun (_, e) -> has_nested_iter e) bs || has_nested_iter
     body
  | If (a, b, c) ->
      has_nested_iter a || has_nested_iter b || has_nested_iter c
  | Lambda (_, body) | Define (_, body) -> has_nested_iter body
  | Var _ | Lit _ | Quote _ -> false

(** Does [e] look like a shrinking argument ([cdr] or decrement by 1)? *)
let looks_like_cdr_arg (e : expr) : bool =
  match e with
  | App (Var "cdr", [ _ ]) | Prim ("cdr", [ _ ]) -> true
  | App (Var "-", [ _; Lit (VInt 1L) ]) | Prim ("-", [ _; Lit (VInt 1L) ])
    -> true
  | _ -> false

(** Does [e] look like a halving argument (divide by 2)? *)
let looks_like_halve (e : expr) : bool =
  match e with
  | App (Var op, [ _; Lit (VInt 2L) ]) when op = "/" || op = "div" || op =
     "halve"
    -> true
  | Prim (op, [ _; Lit (VInt 2L) ]) when op = "/" || op = "div" -> true
  | App (Var "cdr", _) -> false
  | _ -> false

(** Number of self-recursive call sites of [fn_name] within [e]. *)
let count_self_calls ~(fn_name : string) (e : expr) : int =
  let info = analyze_self ~fn_name e in
  List.length info.self_sites

(** Does [fn_name] recurse on arguments that shrink linearly (cdr / -1)? *)
let self_args_look_linear ~(fn_name : string) (e : expr) : bool =
  let rec find acc = function
    | App (Var n, args) when n = fn_name ->
        List.exists looks_like_cdr_arg args || acc
    | App (f, args) -> List.fold_left find (find acc f) args
    | Prim (_, args) -> List.fold_left find acc args
    | Let (_, a, b) | Seq (a, b) -> find (find acc a) b
    | Letrec (bs, body) ->
        let acc = List.fold_left (fun a (_, e) -> find a e) acc bs in
        find acc body
    | If (a, b, c) -> find (find (find acc a) b) c
    | Lambda (_, body) | Define (_, body) -> find acc body
    | Var _ | Lit _ | Quote _ -> acc
  in
  find false e

(** Does [fn_name] recurse on arguments that shrink by halving (/ 2)? *)
let self_args_look_divide ~(fn_name : string) (e : expr) : bool =
```

```
  let rec find acc = function
    | App (Var n, args) when n = fn_name ->
        List.exists looks_like_halve args || acc
    | App (f, args) -> List.fold_left find (find acc f) args
    | Prim (_, args) -> List.fold_left find acc args
    | Let (_, a, b) | Seq (a, b) -> find (find acc a) b
    | Letrec (bs, body) ->
        let acc = List.fold_left (fun a (_, e) -> find a e) acc bs in
        find acc body
    | If (a, b, c) -> find (find (find acc a) b) c
    | Lambda (_, body) | Define (_, body) -> find acc body
    | Var _ | Lit _ | Quote _ -> acc
  in
  find false e

(** Classify the big-O complexity of the function [fn_name] with body [e].
    Pure pattern matching over recursion shape, list iters and DB scans.*)
let classify ~(fn_name : string) (e : expr) : complexity =
  let body = unwrap_fn_body e in
  let n_self = count_self_calls ~fn_name body in
  if has_nested_iter body then O_quadratic
  else if n_self >= 2 && self_args_look_linear ~fn_name body then
    O_exponential
  else if n_self >= 2 && self_args_look_divide ~fn_name body then O_n_log_n
  else if n_self = 1 && self_args_look_linear ~fn_name body then O_linear
  else if n_self = 1 && self_args_look_divide ~fn_name body then O_log_n
  else if has_list_iter body || mentions_db_scan body then O_linear
  else if n_self = 0 && not (has_list_iter body) && not (mentions_db_scan
    body)
  then O_constant
  else if n_self > 0 then O_linear (* recursive but shape unclear -- mild
    guess *)
  else Unknown

(** Classify [e] but also fold in the complexity of known callees
    (looked up in [known_fn]), taking the max of all reachable calls. *)
let classify_with_known
    ~(known_fn : (string, expr) Hashtbl.t)
    ~(fn_name : string)
    (e : expr)
  : complexity =
  let base = classify ~fn_name e in
  let info = analyze_self ~fn_name e in
  List.fold_left
    (fun acc callee ->
      match Hashtbl.find_opt known_fn callee with
      | None -> acc
      | Some body ->
          let cx = classify ~fn_name:callee body in
          max_cx acc cx)
    base info.callees
```

235 lisp/lib/control.ml

```
(** Host control effects for direct-style eval (H1).

    Orthogonal to guest capability effects in [effect.ml].
    Nopales users never see [perform], Lwt, or these constructors.

    See docs/runtime-effects-transition.md Appendix D.

    H1.7: this module is Lwt-free. The image rim provides
    [Control_rim.run_lwt] to service [Prim_call] with Lwt prim tables. *)

open Ast
module Effect = Stdlib.Effect

(** Coarse prim dispatch. Handlers (sync tests or image Lwt rim) interpret. *)
type _ Effect.t +=
  | Prim_call : string * value list -> value Effect.t

(** Optional direct prim table for nested eval under an active rim.
    When set, [perform_prim] calls the table instead of [Effect.perform],
    avoiding Unhandled nested Prim_call (handler context) and
    Continuation_already_resumed from nested match_with. *)
let direct_prims : (string, value list -> value) Hashtbl.t option ref =
  ref None

(** Dispatch a prim call: honors [direct_prims] when set, else escalates to
    [Effect.perform] so the active rim handler (or test handler) services it.
    Raises [Interp_error] if a direct table is set but lacks [name]. *)
let perform_prim (name : string) (args : value list) : value =
  match !direct_prims with
  | Some tbl ->
      (match Hashtbl.find_opt tbl name with
       | Some fn -> fn args
       | None ->
           raise
             (Interp_error (Prim_error (name, "unregistered primitive",
    "prim_error"))))
  | None -> Effect.perform (Prim_call (name, args))

type control_mode = Lwt_mode | Eff_mode

(** Parse a control-mode string into a [control_mode]; unknown strings
    default to [Eff_mode]. *)
let control_mode_of_string = function
  | "lwt" | "promise" -> Lwt_mode
  | "eff" | "effect" | "effects" | _ -> Eff_mode

(** [PP_EVAL_CONTROL=lwt|eff] -- default **eff** (H1.6).
    [lwt] is a legacy escape; after H1.7 the monadic eval path is gone and
    lwt mode only changes nothing if the image always uses the rim handler. *)
let control_mode_from_env () : control_mode =
  match Sys.getenv_opt "PP_EVAL_CONTROL" with
  | Some s -> control_mode_of_string (String.lowercase_ascii (String.trim s))
  | None -> Eff_mode

(** Sync prim table for pure lisp tests (no services). *)
type sync_prim_fn = value list -> value
type sync_prim_table = (string, sync_prim_fn) Hashtbl.t

(** Run [thunk] under a sync effect handler that services [Prim_call] via
    [prims]; unregistered names abort the continuation with [Interp_error]. *)
let run_sync ~(prims : sync_prim_table) (thunk : unit -> 'a) : 'a =
  let open Effect.Deep in
  match_with thunk ()
    { retc = (fun v -> v)
    ; exnc = (fun e -> raise e)
    ; effc =
```

```
        (fun (type a) (eff : a Effect.t) ->
          match eff with
          | Prim_call (name, args) ->
              Some
                (fun (k : (a, _) continuation) ->
                  match Hashtbl.find_opt prims name with
                  | None ->
                      discontinue k
                        (Interp_error
                           (Prim_error (name, "unregistered primitive",
    "prim_error")))
                  | Some fn ->
                      (* Eval prim body then resume [k] once; a prim-body or
                         continuation exception propagates to [exnc].
    Wrapping
                         [continue k (fn args)] in a try would [discontinue]
                         an already-resumed [k]. *)
                      continue k (fn args))
          | _ -> None)
    }

(** Host rim: service [Prim_call] with a sync prim table, setting
    [direct_prims] for nested eval (borge/agent.borg nopales-embed).

    Moved down from image/lib/control_rim.ml so both hosts (image and
    the agent binary) share one rim. Behavior identical: handler
    dispatches to the table; while a prim body runs, [direct_prims] is
    set so nested eval (eval-string, plugin callbacks, loaders)
    dispatches prims by direct lookup instead of re-entering the
    handler (effect handlers do not catch performs from within the
    handler body). *)
let run_with_rim ~(prims : sync_prim_table) (thunk : unit -> 'a) : 'a =
  let open Effect.Deep in
  let prev = !direct_prims in
  direct_prims := Some prims;
  Fun.protect
    ~finally:(fun () -> direct_prims := prev)
    (fun () ->
      match_with thunk ()
        { retc = (fun v -> v)
        ; exnc = (fun e -> raise e)
        ; effc =
            (fun (type a) (eff : a Effect.t) ->
              match eff with
              | Prim_call (name, args) ->
                  Some
                    (fun (k : (a, _) continuation) ->
                      match Hashtbl.find_opt prims name with
                      | None ->
                          discontinue k
                            (Interp_error
                               (Prim_error
                                  (name, "unregistered primitive",
    "prim_error")))
                      | Some fn ->
                          (* Same prim-body/continuation split as
                             [run_sync]: [direct_prims] usually means
                             [Prim_call] never reaches here, but be
                             correct if it does. *)
                          continue k (fn args))
              | _ -> None)
        })
```

236 lisp/lib/effect.ml

```
(** Capability effects for Pricklypear Lisp programs.

    Separate from Lwt async plumbing and from OCaml 5 algebraic effects.
    See spec [lisp-interpreter.effects].

    Effects label what a term *needs* (capabilities), not when it runs. *)

open Ast

(** [effect] is an OCaml 5 keyword; use [eff]. *)
type eff =
  | Pure
  | ReadDb
  | WriteDb
  | Branch
  | Publish
  | Ai
  | Html
  | Txn
  | Net

(** Canonical string name for an [eff]. Inverse of [eff_of_string]. *)
let eff_to_string = function
  | Pure -> "pure"
  | ReadDb -> "read-db"
  | WriteDb -> "write-db"
  | Branch -> "branch"
  | Publish -> "publish"
  | Ai -> "ai"
  | Html -> "html"
  | Txn -> "txn"
  | Net -> "net"

(** Parse a string into an [eff]; [None] if the name is unknown. Inverse of
    [eff_to_string]. *)
let eff_of_string = function
  | "pure" -> Some Pure
  | "read-db" -> Some ReadDb
  | "write-db" -> Some WriteDb
  | "branch" -> Some Branch
  | "publish" -> Some Publish
  | "ai" -> Some Ai
  | "html" -> Some Html
  | "txn" -> Some Txn
  | "net" -> Some Net
  | _ -> None

(** Total order on effects (0..8) used to sort sets: higher rank = stronger
    capability. *)
let eff_rank = function
  | Pure -> 0
  | Html -> 1
  | ReadDb -> 2
  | WriteDb -> 3
  | Txn -> 4
  | Publish -> 5
  | Branch -> 6
  | Ai -> 7
  | Net -> 8

module EffSet = struct
  type t = eff list  (* unique, sorted by rank then name *)

  (** The empty effect set (no capabilities). *)
  let empty : t = []

  (** Sort and dedupe an effect list: drop [Pure], order by rank then name.
      *)
  let normalize (xs : eff list) : t =
    let xs = List.filter (function Pure -> false | _ -> true) xs in
    let xs =
```

```
      List.sort_uniq
        (fun a b ->
          let c = compare (eff_rank a) (eff_rank b) in
          if c <> 0 then c else String.compare (eff_to_string a)
    (eff_to_string b))
        xs
    in
    xs

  (** Effect set containing exactly [e] (normalized). *)
  let singleton e = normalize [ e ]

  (** Set union, dropping [Pure] and deduping by rank. *)
  let union a b = normalize (a @ b)

  (** Fold [union] over a list of effect sets. *)
  let unions sets = List.fold_left union empty sets

  (** Membership test (linear scan over the normalized list). *)
  let mem e set = List.mem e set

  (** [true] when the set holds no effects. *)
  let is_empty = function [] -> true | _ -> false

  (** [true] when the set is empty (no capabilities required). *)
  let is_pure = is_empty

  (** [true] when every effect of [a] is also in [b]. *)
  let subset (a : t) (b : t) : bool =
    List.for_all (fun e -> mem e b) a

  (** The normalized effect list backing this set. *)
  let to_list s = s

  (** Human-readable rendering: ["pure"] or effects joined by ["+"]. *)
  let pp (s : t) : string =
    if is_pure s then "pure"
    else String.concat "+" (List.map eff_to_string s)

  (** Short UI badge string; collapses to a coarse capability label when
     one effect dominates. *)
  let badge (s : t) : string =
    if is_pure s then "pure"
    else if mem Ai s then "ai"
    else if mem Publish s then "page"
    else if mem Branch s then "branch"
    else if mem Net s then "net"
    else if mem WriteDb s then "db-write"
    else if mem ReadDb s then "db-read"
    else if mem Html s then "html"
    else pp s
end

type effect_set = EffSet.t

(** Default pure prim names (image may add more).
    H0.1 audit 2026-07-18: every registered image prim must appear in
    exactly one capability bucket (or the special-case arms in
    [effects_for_prim]). Guest-visible only -- not host control effects. *)
let pure_prims = [
  "+"; "-"; "*"; "div"; "/"; "mod"; "quotient"; "remainder";
  "abs"; "min"; "max";
  "expt"; "sqrt"; "exp"; "log";
  "sin"; "cos"; "tan"; "asin"; "acos"; "atan";
  "number?"; "integer?"; "zero?"; "positive?"; "negative?";
  "even?"; "odd?";
  "random"; "random-seed";
  "="; "<"; ">"; "<="; ">="; "not";
  "list"; "car"; "cdr"; "cons"; "null?"; "pair?"; "count";
  "append"; "reverse"; "nth";
  "string-append"; "string-join"; "str"; "type-of"; "as-str";
  "symbol?"; "string?"; "symbol=?"; "symbol->string"; "string->symbol";
```

```
  "string-eq"; "string-trim"; "string-length"; "substring";
    "string-contains?";
  "string<?"; "string<=?"; "string>?"; "string>=?";
  "html-escape"; "json-parse"; "json-emit";
  "read-string"; "read-all-string";
  "dict-get"; "dict-set"; "dict-keys";
  "hx-get"; "hx-post"; "hx-request?";
  "request-header"; "request-method"; "request-path";
  "request-param"; "request-params"; "request-body";
  "date-today"; "date-weekday"; "date-format";
  "date-add-days"; "date-diff-days";
  "now"; "now-ms"; "sleep";
  "parse-int"; "parse-float"; "floor"; "round";
  "fsrs-schedule";
  "length"; "equal?";
  "cadr"; "caar"; "caddr"; "cddr"; "assoc"; "member";
  "string-replace"; "string-downcase"; "string-upcase";
  "string-starts-with?"; "string-ends-with?";
  "url-encode"; "url-decode"; "sha1"; "html-decode";
  "parse-date"; "xml-parse"; "random-uuid";
  "verify-ssh-key";  (* pure crypto check on args; no store *)
]

let read_db_prims = [
  "get"; "rows"; "select-for-update";
  "find-row"; "find-rows"; "find-text"; "find-random";
  "list-branches"; "current-branch"; "list-bindings";
  "whoami"; "current-author";
  "function-source"; "function-doc"; "function-ast"; "package-bindings";
    "package-exports";
  "export-package";
  "route-visibility";
  "list-usernames"; "list-ssh-keys";
  "llm-key-status";
  "log-tail"; "ops/summary"; "ops/incidents";
  "agent-history"; "agent-sessions";
]

let write_db_prims = [
  "create"; "update"; "update-field"; "delete"; "deftype";
  "load-string"; "load-library";  (* persist defs/routes onto branch *)
  "add-ssh-key"; "revoke-ssh-key";
  "set-llm-key"; "clear-llm-key";
  "with-public-write";  (* elevates anon ? may write data_rows *)
  "submit-job"; "submit-job-at"; "cancel-job";  (* borge/jobs.borg WriteDb
    *)
  "with-step"; "__step__";  (* checkpoint write; composes with Txn for
    exactly-once *)
  "schedule"; "unschedule";  (* borge/jobs.borg schedules WriteDb *)
  "undo";  (* borge/tree-time.borg reads-as-of-gen restore-by-write WriteDb
    *)
  "code-restore";  (* borge/tree-time.borg consumers code-restore WriteDb *)
]

let read_db_prims = [
  "get"; "rows"; "select-for-update";
  "find-row"; "find-rows"; "find-text"; "find-random";
  "list-branches"; "current-branch"; "list-bindings";
  "whoami"; "current-author";
  "function-source"; "function-doc"; "function-ast"; "package-bindings";
    "package-exports";
  "export-package";
  "route-visibility";
  "list-usernames"; "list-ssh-keys";
  "llm-key-status";
  "log-tail"; "ops/summary"; "ops/incidents";
  "agent-history"; "agent-sessions";
  "job-status"; "jobs"; "schedules";  (* borge/jobs.borg ReadDb *)
  "gen-now";  (* borge/tree-time.borg reads-as-of-gen ReadDb *)
  "gen-read"; "gen-diff";  (* borge/tree-time.borg reads-as-of-gen ReadDb *)
]

let branch_prims = [ "fork"; "merge"; "checkout" ]
```

```
let publish_prims = [ "route"; "public-route" ]
let html_prims = [ "render-html"; "redirect" ]
(* Outbound network (HTTP fetch). Gated to admin/agent at the prim level
   (session.ml); tracked as a capability badge so public routes and AI install
   policy can deny it by default. *)
let net_prims = [ "http-get"; "http-post" ]
(* Author tools that freeze into function_defs or drive the model (Ai + WriteDb). *)
let ai_write_prims = [
  "propose-function"; "install-function"; "agent-task";
  "llm-inner"; "llm-outer"; "llm-rewrite"; "llm-seed";
  "agent-session-new"; "agent-say"; "agent-session-delete";
]
(* Portal that only talks to the model (Ai only) -- legacy one-shot. *)
let ai_prims = "agent" :: ai_write_prims

let effects_for_prim (name : string) : effect_set =
  if List.mem name pure_prims then EffSet.empty
  else if List.mem name read_db_prims then EffSet.singleton ReadDb
  else if List.mem name write_db_prims then EffSet.singleton WriteDb
  else if List.mem name branch_prims then EffSet.singleton Branch
  else if List.mem name publish_prims then EffSet.singleton Publish
  else if List.mem name html_prims then EffSet.singleton Html
  else if List.mem name net_prims then EffSet.singleton Net
  else if name = "agent" then EffSet.singleton Ai
  else if List.mem name ai_write_prims then
    EffSet.union (EffSet.singleton Ai) (EffSet.singleton WriteDb)
  else if name = "with-transaction" || name = "__with_transaction__" then EffSet.singleton Txn
  else
    (* Unknown name: not a registered capability prim. User fns resolved
       via known_fn; bare unknowns contribute nothing (avoid painting every
       call as db-write). *)
    EffSet.empty

type prim_effect_lookup = string -> effect_set
type known_fn_lookup = string -> effect_set option

(** Default [effect_set] resolution for prim names -- [effects_for_prim]. *)
let default_prim_lookup = effects_for_prim

(** Build a [known_fn] lookup from visible bindings [(name, body)].
    Memoized; cycles break by treating the in-progress name as pure so
    mutual recursion does not diverge. Used by badges, AI install gate,
    and route publish policy (H0.2 / H0.3). *)
let rec known_fn_of_bindings (bindings : (string * expr) list) : known_fn_lookup =
  let bodies : (string, expr) Hashtbl.t =
    Hashtbl.create (max 16 (List.length bindings))
  in
  List.iter (fun (n, e) -> Hashtbl.replace bodies n e) bindings;
  let cache : (string, effect_set) Hashtbl.t =
    Hashtbl.create (max 16 (List.length bindings))
  in
  let rec lookup (name : string) : effect_set option =
    match Hashtbl.find_opt cache name with
    | Some s -> Some s
    | None ->
        (match Hashtbl.find_opt bodies name with
         | None -> None
         | Some body ->
             (* cycle break *)
             Hashtbl.replace cache name EffSet.empty;
             let s = infer ~known_fn:lookup body in
             Hashtbl.replace cache name s;
             Some s)
  in
  lookup

(** Conservative AST effect inference.
    [known_fn] resolves user function names when known; unknown globals
    contribute no extra effects (overapprox only on prims). Higher-order
```

```
    unknown application does not explode to top -- callers may wrap. *)
and infer
    ?(prims = default_prim_lookup)
    ?(known_fn : known_fn_lookup = fun _ -> None)
    (e : expr) : effect_set =
  let go = infer ~prims ~known_fn in
  match e with
  | Var name ->
      (match known_fn name with
       | Some s -> s
       | None ->
           (* bare var might be a prim bound in env *)
           if List.mem name pure_prims
              || List.mem name read_db_prims
              || List.mem name write_db_prims
              || List.mem name branch_prims
              || List.mem name publish_prims
              || List.mem name html_prims
              || List.mem name ai_prims
              || List.mem name net_prims
           then prims name
           else EffSet.empty)
  | Lit _ -> EffSet.empty
  | Quote _ -> EffSet.empty
  | Lambda (_params, body) -> go body
  | If (t, a, b) -> EffSet.unions [ go t; go a; go b ]
  | Let (_n, v, body) -> EffSet.union (go v) (go body)
  | Letrec (bindings, body) ->
      let bs = List.map (fun (_, e') -> go e') bindings in
      EffSet.unions (go body :: bs)
  | Seq (a, b) -> EffSet.union (go a) (go b)
  | Define (_n, v) -> go v
  | Prim (name, args) ->
      EffSet.union (prims name) (EffSet.unions (List.map go args))
  | App (head, args) ->
      let head_fx =
        match head with
        | Var name ->
            (match known_fn name with
             | Some s -> s
             | None -> prims name)
        | _ -> go head
      in
      let special =
        match head with
        | Var "with-transaction" -> EffSet.singleton Txn
        | _ -> EffSet.empty
      in
      EffSet.unions (head_fx :: special :: List.map go args)

(** Default AI allowlist for generated code (install_function bodies).

    WriteDb is allowed so agents can build real apps (create/update/delete/
    deftype on data_rows). Publish (route) stays denied here -- use the
    dedicated register_route tool so route registration is explicit and
    not smuggled inside a lambda body. Elevate with PP_AI_ALLOW_WRITE for
    Branch/Publish/Ai extras. *)
let ai_default_allow : effect_set =
  EffSet.normalize [ ReadDb; WriteDb; Html ]

(** [true] when env var [name] is set to a truthy value (1/true/yes/on). *)
let env_truthy (name : string) : bool =
  match Sys.getenv_opt name with
  | Some ("1" | "true" | "TRUE" | "yes" | "on") -> true
  | _ -> false

(** Public user-route allowlist (capability-enforcement).
    Default: pure | read-db | html | txn -- no WriteDb/Branch/Ai/Publish.
    Set PP_PUBLIC_ALLOW_WRITE=1 to permit WriteDb (signup forms, public
    capture). Prefer (with-public-write ?) inside those handlers so the
    write runs as elevated identity, not the anonymous user. *)
let public_route_allow () : effect_set =
  let base = [ ReadDb; Html; Txn ] in
```

```
  let base =
    if env_truthy "PP_PUBLIC_ALLOW_WRITE" then WriteDb :: base else base
  in
  EffSet.normalize base

(** String form of the current public-route allowlist. *)
let public_route_allow_pp () : string = EffSet.pp (public_route_allow ())

(** Capability policy check: [true] when [got] is a subset of [allow], or when
    [got] is pure (no capabilities required). *)
let policy_allows ~(allow : effect_set) (got : effect_set) : bool =
  EffSet.subset got allow
  || (EffSet.is_pure got)

(** Soft transactional hygiene hints (H0.4). Advisory only -- does not
    reject eval. Looks for multi-write bodies and [select-for-update]
    without a surrounding [with-transaction] / Txn effect. *)
let count_write_sites (e : expr) : int =
  let rec go = function
    | Prim (name, args) ->
        let here = if List.mem name write_db_prims then 1 else 0 in
        here + List.fold_left (fun n a -> n + go a) 0 args
    | App (Var name, args) when List.mem name write_db_prims ->
        1 + List.fold_left (fun n a -> n + go a) 0 args
    | App (h, args) -> go h + List.fold_left (fun n a -> n + go a) 0 args
    | Lambda (_, body) | Define (_, body) -> go body
    | If (t, a, b) -> go t + go a + go b
    | Let (_, v, body) | Seq (v, body) -> go v + go body
    | Letrec (bs, body) ->
        List.fold_left (fun n (_, e') -> n + go e') (go body) bs
    | Var _ | Lit _ | Quote _ -> 0
  in
  go e

(** [true] if [e] contains a [select-for-update] call anywhere; used by txn linting. *)
let has_select_for_update (e : expr) : bool =
  let rec go = function
    | Prim ("select-for-update", _) -> true
    | App (Var "select-for-update", _) -> true
    | Prim (_, args) -> List.exists go args
    | App (h, args) -> go h || List.exists go args
    | Lambda (_, body) | Define (_, body) -> go body
    | If (t, a, b) -> go t || go a || go b
    | Let (_, v, body) | Seq (v, body) -> go v || go body
    | Letrec (bs, body) -> go body || List.exists (fun (_, e') -> go e') bs
    | Var _ | Lit _ | Quote _ -> false
  in
  go e

(** Advisory transactional-hygiene hints for [e] (soft, non-blocking).
    Flags multi-write bodies and [select-for-update] outside a
    [with-transaction]. *)
let txn_lint ?(known_fn : known_fn_lookup = fun _ -> None) (e : expr)
  : string list =
  let fx = infer ~known_fn e in
  let has_txn = EffSet.mem Txn fx in
  let writes = count_write_sites e in
  let hints = ref [] in
  if writes >= 2 && not has_txn then
    hints :=
      (Printf.sprintf
         "soft txn: %d write-db sites without with-transaction -- consider wrapping multi-step mutations"
         writes)
      :: !hints;
  if has_select_for_update e && not has_txn then
    hints :=
      "soft txn: select-for-update without with-transaction -- row lock will not span later prims"
      :: !hints;
  List.rev !hints
```

237 lisp/lib/env.ml

```
(** Pricklypear Lisp -- Lexical environment.

    Lexically scoped bindings. Each env has a parent (set at closure
    capture time) and a hashtable of name -> value. Lookup walks up the
    parent chain.

    Per spec [lisp-interpreter.eval]: the evaluator is pure w.r.t. I/O;
    the env is mutable (binding updates), which is fine -- it's local to
    a session, not persisted. Persistence happens via [(define ...)]
    which, in the IMAGE layer, ALSO writes to the store. *)
open Ast

type env = Ast.env

(** Create a fresh env. [~parent] (default [None]) is the lexical parent
    captured at closure-creation time; lookups walk up this chain. *)
let create ?(parent = None) () : env =
  { bindings = Hashtbl.create 16; parent }

(** Create a child env whose parent is [e]. Bindings defined in the child
    shadow (not mutate) names in [e]. *)
let child (e : env) : env = create ~parent:(Some e) ()

(** Read-set tracking for dependency-tracked caching
    (image.product-theme). While armed, every successful [lookup]
    posts the resolved name to this hook. Armed ONLY by
    [Session.refresh_ui_shell] around the (ui/css)/(ui/nav) evals so
    a define can bust the shell cache iff the defined name was
    actually read -- no TTL, no prefix guess. Global mutable by design
    (eval is single-threaded within a session fiber); [None] costs
    one ref-read per lookup. *)
let lookup_hook : (string -> unit) option ref = ref None

(** Force a [VThunk] cell (lisp-interpreter.lazy-env-thunks).
    Returns the memoized value on [Forced], replays the memoized
    exception on [Failed], raises Interp_error on a [Forcing] cycle,
    and on [Unforced] runs the payload's OWN evaluator (which is how
    the image forces under its prim rim without the pure kernel
    learning about I/O). Values other than VThunk pass through. *)
let rec force_value (v : value) : value =
  match v with
  | VThunk cell ->
      (match !cell with
       | Forced v -> v
       | Failed exn -> raise exn
       | Forcing ->
           raise
             (Interp_error
                (User_error
                   ("cyclic thunk force (binding refers to itself)",
                    "cyclic-thunk")))
       | Unforced payload ->
           cell := Forcing;
           (match
              payload.thunk_force payload.thunk_env payload.thunk_body
            with
            | v ->
                let v = force_value v in
                cell := Forced v;
                v
            | exception exn ->
                cell := Failed exn;
                raise exn))
  | v -> v

(** Bind [name] in [e] to a not-yet-evaluated body (image load_defs).
    [force] is the evaluator used when the binding is first touched. *)
let define_thunk (e : env) (name : string)
    ~(force : env -> expr -> value) (body : expr) : unit =
  Hashtbl.replace e.bindings name
    (VThunk (ref (Unforced { thunk_body = body; thunk_env = e;
```

```
                              thunk_force = force })))

(** [lookup e name] returns [Some v] if [name] is bound in [e] or any
    ancestor, else [None]. Walks the parent chain to the root.
    VThunk bindings are forced (and memoized) here -- the single
    observation point, so user code never receives a VThunk. *)
let rec lookup (e : env) (name : string) : value option =
  match Hashtbl.find_opt e.bindings name with
  | Some v ->
      (match !lookup_hook with Some f -> f name | None -> ());
      Some (force_value v)
  | None ->
      (match e.parent with
       | Some p -> lookup p name
       | None -> None)

(** Bind [name] to [v] in [e], replacing any existing binding locally
    (does not touch parents). Mirrors the image-layer [(define ...)]
    which in the IMAGE also persists to the store. *)
let define (e : env) (name : string) (v : value) : unit =
  Hashtbl.replace e.bindings name v

(** Like [define] but only binds if [name] is already bound somewhere in
    the parent chain. Returns [true] on success, [false] if [name] is
    not bound (leaving the env unchanged). See [lisp-interpreter.eval] for
    the set!/redefinition semantics this mirrors. *)
let set_existing (e : env) (name : string) (v : value) : bool =
  (* Like [define] but only succeeds if the name is already bound
     somewhere in the parent chain. Returns false if not bound. *)
  let rec find_env e =
    match Hashtbl.find_opt e.bindings name with
    | Some _ -> Some e
    | None -> Option.bind e.parent find_env
  in
  match find_env e with
  | Some e' -> Hashtbl.replace e'.bindings name v; true
  | None -> false

(** Fold [f] over every binding in [e] and its ancestor chain, in
    child-to-parent order. Used by reflection primitives like
    [(list-bindings)]. *)
let rec fold (e : env) (f : string -> value -> 'a -> 'a) (acc : 'a) : 'a =
  let local = Hashtbl.fold f e.bindings acc in
  match e.parent with
  | Some p -> fold p f local
  | None -> local
```

238 lisp/lib/eval.ml

```
(** Pricklypear Lisp -- Evaluator (H1.7 direct-style).

    Takes an env and an expr, returns a value (or raises [Interp_error]).
    I/O prims use [Control.perform_prim] (OCaml 5 effects). The image rim
    ([Control_rim.run_lwt]) or pure tests ([Control.run_sync]) service
    [Prim_call]. No Lwt in this module.

    Per spec [lisp-interpreter.eval]: cooperative [budget], special forms,
    define binds env; image persists. *)
open Ast

(** A budget tracks a per-call step limit. Counted down at each eval
    step. When zero, raises [Interp_error Budget_exceeded]. *)
type budget = {
  mutable steps : int;
  limit : int;
  mutable current_fn : string option;
  mutable prim_calls : int;
  mutable db_hits : int;
  mutable db_ms : float;
  fn_step_counts : (string, int) Hashtbl.t;
}

(** Default per-call step limit for [make_budget] (100k steps). *)
let default_limit = 100_000

(** Allocate a fresh [budget] with the given [limit] (default
    [default_limit]); counters start at zero. *)
let make_budget ?(limit = default_limit) () : budget =
  {
    steps = 0;
    limit;
    current_fn = None;
    prim_calls = 0;
    db_hits = 0;
    db_ms = 0.;
    fn_step_counts = Hashtbl.create 16;
  }

(** Dynamic binding for the budget of the eval currently on the stack.
    Lets OCaml prims like [budget-remaining] and nested apply helpers
    share the caller's step counter without threading budget through
    the prim ABI. *)
let current_budget : budget option ref = ref None

(** Last top-level budget from [eval_string_direct] / rim entry -- for
    image-layer function_stat persistence. *)
let last_budget : budget option ref = ref None

(** Remaining steps on the current budget (0 if none active). *)
let budget_remaining_steps () : int =
  match !current_budget with
  | None -> 0
  | Some b -> max 0 (b.limit - b.steps)

(** Absolute step count on the current budget (0 if none active). Used by
    the remote-eval stepping trace (borge/remote-eval.borg stepping) to
    attribute the span of local pure steps between two wire crossings: the
    trace samples [current_steps] right before a prim forwards over the
    wire, so the difference counts the local steps since the last sample.
    Additive; harmless when no stepping controller is active. *)
let current_steps () : int =
  match !current_budget with
  | None -> 0
  | Some b -> b.steps

(** Record one DB hit on the current budget, if any. *)
let note_db_hit () : unit =
  match !current_budget with
  | Some b -> b.db_hits <- b.db_hits + 1
  | None -> ()
```

```
72
73  (** Add [ms] to the current budget's accumulated DB time, if any. *)
74  let note_db_ms (ms : float) : unit =
75    match !current_budget with
76    | Some b -> b.db_ms <- b.db_ms +. ms
77    | None -> ()
78
79  (** Advance [b]'s step count (tracking per-function counts) and raise
80      [Interp_error Budget_exceeded] once the limit is crossed. *)
81  let tick (b : budget) : unit =
82    b.steps <- b.steps + 1;
83    (match b.current_fn with
84     | Some fn ->
85         let n = try Hashtbl.find b.fn_step_counts fn with Not_found -> 0 in
86         Hashtbl.replace b.fn_step_counts fn (n + 1)
87     | None -> ());
88    if b.steps > b.limit then raise (Interp_error Budget_exceeded)
89
90  (** A primitive for pure tests / name registration: sync function.
91      Image I/O prims are NOT stored here as Lwt -- the rim handler owns
        those.
92      Eval always [perform]s; [run_sync] invokes these sync fns. *)
93  type prim_fn = value list -> value
94
95  (** Name -> [prim_fn] table for pure/sync prims. Image I/O prims are
96      served by the rim handler, not stored here. *)
97  type prim_table = (string, prim_fn) Hashtbl.t
98
99  (** Create an empty [prim_table]. *)
100 let make_prim_table () : prim_table = Hashtbl.create 32
101
102 (** Register [fn] in [tbl] under [name] (replaces any existing binding). *)
103 let register_prim (tbl : prim_table) (name : string) (fn : prim_fn) : unit =
104   Hashtbl.replace tbl name fn
105
106 (** Look up [name] in [tbl], returning the registered prim if present. *)
107 let lookup_prim (tbl : prim_table) (name : string) : prim_fn option =
108   Hashtbl.find_opt tbl name
109
110 (** Special-form recognition: an [App] whose head is [Var name] for one of
111     these names is dispatched as the special form, not as a function call.
        *)
112 let is_special_form (name : string) : bool =
113   match name with
114   | "lambda" | "let" | "letrec" | "if" | "define" | "quote" | "seq"
115   | "begin" | "match" | "guard" | "error"
116   | "prim" | "with-transaction" | "with-step"
117   | "quasiquote" | "unquote" | "unquote-splicing" | "defmacro"
118   | "macroexpand-1" | "macroexpand" | "gensym"
119   | "cond" | "and" | "or"
120   (* Higher-order list ops: OCaml loops + apply, not recursive Lisp.
121      Avoids ~O(n) eval-frame overhead per element (budget P0). *)
122   | "apply" | "list/map" | "list/filter" | "list/foldl" | "list/foldr"
123   (* lazy-env-thunks: explicit delay/force (Var lookup auto-forces) *)
124   | "delay" | "force" -> true
125   | _ -> false
126
127 (** The full set of reserved kernel special-form names (see
128     [is_special_form]). Used by [assert_bindable_name] to refuse
129     redefinition. *)
130 let kernel_special_names =
131   [ "lambda"; "let"; "letrec"; "if"; "define"; "quote"; "seq"; "begin";
132     "match"; "guard"; "error"; "prim";
133     "with-transaction"; "with-step"; "quasiquote"; "unquote";
        "unquote-splicing";
134     "defmacro"; "macroexpand-1"; "macroexpand"; "gensym";
135     "cond"; "and"; "or";
136     "apply"; "list/map"; "list/filter"; "list/foldl"; "list/foldr";
137     "delay"; "force" ]
138
139 (** Refuse define/defmacro of kernel special forms and of names currently
140     bound to [VPrim] (registered kernel prims). Closes BORGE-1785213238:
141     a user/AI (define is-admin? ?) / (define create ?) must not shadow the
```

```
142     prim the image and effect gates rely on. Special forms are always
143     reserved; prims are reserved once bound as [VPrim] in [env] (image
144     binds the full catalog at boot / rehydrate before load_defs). *)
145 let assert_bindable_name (env : env) (name : string) : unit =
146   if List.mem name kernel_special_names then
147     raise
148       (Interp_error
149          (User_error ("cannot redefine kernel special form: " ^ name,
    "bad-args")));
150   match Env.lookup env name with
151   | Some (VPrim _) ->
152       raise
153         (Interp_error
154            (User_error ("cannot redefine kernel primitive: " ^ name,
    "bad-args")))
155   | _ -> ()
156
157 (** Desugar multiple body exprs to a single expr (begin/seq chain). *)
158 let body_of_exprs = function
159   | [] -> Lit VNil
160   | [ e ] -> e
161   | e1 :: e2 :: rest ->
162       let rec fold a = function
163         | [] -> a
164         | x :: xs -> Seq (a, fold x xs)
165       in
166       fold e1 (e2 :: rest)
167
168 (** Leading string literal is a docstring only when followed by ?1 body
    form.
169     A lone string body is a value-returning body, not metadata.
170     Per spec [lisp-interpreter.define-docstring]. *)
171 let peel_leading_doc (bodies : expr list) : string option * expr list =
172   match bodies with
173   | Lit (VStr doc) :: (_ :: _ as rest) -> (Some doc, rest)
174   | _ -> (None, bodies)
175
176 (** Extract parameter names from [define] sugar param exprs; an
177     [App (Var c, [_])] is accepted as an optional-with-default [c]. *)
178 let params_of_define_sugar (param_exprs : expr list) : string list =
179   List.map
180     (function
181      | Var p -> p
182      | App (Var p, [ _default ]) ->
183          (* optional with default: (c default) -- allowed in define sugar *)
184          p
185      | _ ->
186          raise
187            (Interp_error
188               (User_error ("define function params must be names",
    "bad-args"))))
189     param_exprs
190
191 (** Rewrite a lambda-shaped expr so a leading multi-body docstring is
    removed
192     from the stored/bound body. Returns (body_expr, doc_opt). *)
193 let strip_lambda_doc (e : expr) : expr * string option =
194   match e with
195   | App (Var "lambda", params_expr :: bodies) ->
196       let doc, bodies' = peel_leading_doc bodies in
197       (App (Var "lambda", params_expr :: bodies'), doc)
198   | Lambda (params, body) ->
199       (* Constructor form is already a single body; only Seq-peel if the
200          first side of a top Seq is a string lit and we want cleanliness.
201          Keep simple: no peel on pre-built Lambda (tests/AST path). *)
202       let _ = params in
203       (Lambda (params, body), None)
204   | _ -> (e, None)
205
206 (** Parsed top-level (define ?) for eval + image persist.
207     [body] is the expr to bind/persist (lambda leading-doc already
    stripped). *)
208 type define_shape = {
```

```
  name : string;
  body : expr;
  doc : string option;
}

(** Parse a top-level [(define name body) / (define (name params?) ?)]
    (or a pre-constructed [Define] node) into a [define_shape], stripping
    any leading lambda docstring. Raises on malformed forms. *)
let parse_top_define (e : expr) : define_shape option =
  match e with
  | App (Var "define", [ Var name; body_expr ]) ->
      let body, doc = strip_lambda_doc body_expr in
      Some { name; body; doc }
  | App (Var "define", [ Var name; body_expr; Lit (VStr doc) ]) ->
      let body, _lam_doc = strip_lambda_doc body_expr in
      (* Trailing define docstring wins over lambda-internal. *)
      Some { name; body; doc = Some doc }
  | App (Var "define", [ Var _; _; other ]) ->
      let _ = other in
      raise
        (Interp_error
           (User_error
              ("define docstring must be a string literal \"?\"",
    "bad-args")))
  | App (Var "define", App (Var name, param_exprs) :: bodies) ->
      let doc, bodies' = peel_leading_doc bodies in
      let params = params_of_define_sugar param_exprs in
      if List.mem "." params then
        raise
          (Interp_error
             (User_error ("dotted rest params not supported in define
    sugar", "bad-args")));
      let body = body_of_exprs bodies' in
      Some { name; body = Lambda (params, body); doc }
  | Define (name, body_expr) ->
      let body, doc = strip_lambda_doc body_expr in
      Some { name; body; doc }
  | _ -> None

(** Match pattern against value; returns Some binding list or None. *)
let rec match_pat (pat : expr) (v : value) : (string * value) list option =
  match pat, v with
  | Var "_", _ -> Some []
  | Var name, _ -> Some [ (name, v) ]
  | Lit lit, v when lit = v -> Some []
  | Lit (VInt n), VInt m when n = m -> Some []
  | Lit (VFloat f), VFloat g when f = g -> Some []
  | Lit (VStr s), VStr t when s = t -> Some []
  | Lit (VBool b), VBool c when b = c -> Some []
  | Lit VNil, VNil -> Some []
  | Lit VNil, VList [] -> Some []
  | Quote (Var s), VSymbol t when s = t -> Some []
  | Quote (Var s), VStr t when s = t -> Some [] (* compat *)
  | Quote (Lit lit), v when lit = v -> Some []
  | App (Var "quote", [ Var s ]), VSymbol t when s = t -> Some []
  | App (Var "quote", [ Var s ]), VStr t when s = t -> Some []
  | App (Var "list", pats), VList vs -> match_list_pats pats vs
  | App (Var "list", pats), VNil when pats = [] -> Some []
  | App (Var head, pats), VList (vh :: vt) ->
      (* (lambda params body) as pattern => list starting with symbol head
    *)
      let head_ok =
        match vh with
        | VSymbol s | VStr s -> s = head
        | _ -> false
      in
      if head_ok then match_list_pats pats vt else None
  | App (Var head, []), VSymbol s when s = head -> Some []
  | App (Var head, []), VStr s when s = head -> Some []
  | _ -> None

(** Match a list pattern against a value list (helper for [match_pat]).
    Returns combined bindings or [None] on mismatch. *)
```

```
and match_list_pats (pats : expr list) (vs : value list)
  : (string * value) list option =
  match pats, vs with
  | [], [] -> Some []
  | [], _ | _, [] -> None
  | p :: ps, v :: vs ->
      (match match_pat p v with
       | None -> None
       | Some b1 ->
           (match match_list_pats ps vs with
            | None -> None
            | Some b2 -> Some (b1 @ b2)))

(** Lisp truthiness: [false] and [nil] are falsy, everything else truthy. *)
let truthy (v : value) : bool =
  match v with
  | VBool false | VNil -> false
  | _ -> true

(** Quote any expr into a first-class value (full s-expr quote).
    Compounds become nested VLists so quoted data is usable in Lisp.
    Vars become VSymbol; string lits stay VStr (no collapse). *)
let rec quote_to_value (e : expr) : value =
  let sym s = VSymbol s in
  match e with
  | Lit v -> v
  | Var s -> VSymbol s
  | App (h, args) ->
      VList (quote_to_value h :: List.map quote_to_value args)
  | Quote e' -> VList [ sym "quote"; quote_to_value e' ]
  | If (t, th, el) ->
      VList [ sym "if"; quote_to_value t; quote_to_value th; quote_to_value
    el ]
  | Lambda (params, body) ->
      VList [ sym "lambda"
            ; VList (List.map (fun p -> VSymbol p) params)
            ; quote_to_value body ]
  | Let (name, v, body) ->
      VList [ sym "let"
            ; VList [ VList [ VSymbol name; quote_to_value v ] ]
            ; quote_to_value body ]
  | Letrec (bindings, body) ->
      VList [ sym "letrec"
            ; VList (List.map (fun (n, e') ->
                VList [ VSymbol n; quote_to_value e' ]) bindings)
            ; quote_to_value body ]
  | Define (name, v) ->
      VList [ sym "define"; VSymbol name; quote_to_value v ]
  | Seq (a, b) ->
      VList [ sym "seq"; quote_to_value a; quote_to_value b ]
  | Prim (name, args) ->
      VList (sym name :: List.map quote_to_value args)

(** Macro reify: same as quote_to_value now that symbols exist.
    __str__ tagging retained only via value_to_expr compat path. *)
let reify_expr (e : expr) : value = quote_to_value e

(** Name atom: VSymbol preferred; bare VStr still accepted as a name
    during compat (old trees / hand-built lists of strings). *)
let name_of_atom ?(ctx = "value_to_expr") = function
  | VSymbol s | VStr s -> s
  | _ ->
      raise (Interp_error (User_error (ctx ^ ": expected symbol or string
    name", "bad-args")))

(** Head name of a value if it is a symbol or string atom. *)
let head_name = function
  | VSymbol s | VStr s -> Some s
  | _ -> None

(** Inverse of quote_to_value / macro expansion trees. *)
let rec value_to_expr (v : value) : expr =
  match v with
```

```
  | VList [ VStr "__str__"; VStr s ] -> Lit (VStr s) (* transitional *)
  | VList [ VSymbol "__str__"; VStr s ] -> Lit (VStr s)
  | VInt _ | VFloat _ | VBool _ | VNil as lit -> Lit lit
  | VSymbol s -> Var s
  | VStr s -> Lit (VStr s) (* string data, not a name *)
  | VList [] -> Lit VNil
  | VList (h :: t) ->
      (match head_name h, t with
       | Some "quote", [ x ] -> Quote (value_to_expr x)
       | Some "if", [ te; th; el ] ->
           If (value_to_expr te, value_to_expr th, value_to_expr el)
       | Some "lambda", [ VList params; body ] ->
           let ps =
             List.map
               (fun p -> name_of_atom ~ctx:"value_to_expr:lambda" p)
               params
           in
           Lambda (ps, value_to_expr body)
       | Some "define", [ name_v; body ] ->
           Define
             ( name_of_atom ~ctx:"value_to_expr:define" name_v
             , value_to_expr body )
       | Some "let", [ VList binds; body ] ->
           (match binds with
            | [ VList [ name_v; bv ] ] ->
                Let
                  ( name_of_atom ~ctx:"value_to_expr:let" name_v
                  , value_to_expr bv
                  , value_to_expr body )
            | _ ->
                App
                  ( Var "let"
                  , [ value_to_expr (VList binds); value_to_expr body ] ))
       | Some "seq", a :: b :: rest ->
           let rec fold x xs =
             match xs with
             | [] -> value_to_expr x
             | y :: ys -> Seq (value_to_expr x, fold y ys)
           in
           fold a (b :: rest)
       | _ -> App (value_to_expr h, List.map value_to_expr t))
  | VPair (a, b) ->
      App (Var "cons", [ value_to_expr a; value_to_expr b ])
  | VLambda _ | VPrim _ | VRef _ | VMacro _ | VThunk _ ->
      raise
        (Interp_error
           (User_error ("value_to_expr: cannot reify live
    closure/prim/ref/macro/thunk", "bad-args")))

(** Global counter backing [gensym]. *)
let gensym_counter = ref 0

(** Generate a unique symbol name [prefix$N] using a global counter. *)
let gensym ?(prefix = "g") () : string =
  incr gensym_counter;
  prefix ^ "$" ^ string_of_int !gensym_counter

(** Flatten a Lisp list value (VList or VPair chain or VNil) to OCaml list.
    *)
let rec value_to_list (v : value) : value list =
  match v with
  | VNil | VList [] -> []
  | VList xs -> xs
  | VPair (a, b) -> a :: value_to_list b
  | other ->
      raise
        (Interp_error
           (User_error
              ("list op needs a list, got " ^ pp_value_short other,
    "bad-args")))

(** Trampoline step: [Done] finishes; [Cont] continues without OCaml stack.
    *)
```

```
type step =
  | Done of value
  | Cont of env * expr

(** One evaluation step. Tail positions return [Cont]; non-tails force via
    [eval]. *)
let rec eval_step
  ~(budget : budget)
  ~(prims : prim_table)
  (env : env)
  (e : expr)
  : step =
  (* Publish budget for prims (budget-remaining) and nested helpers. *)
  current_budget := Some budget;
  tick budget;
  match e with
  | Var name ->
      (* Keyword atoms (:target, :swap, ?) are self-evaluating symbols so
         prim kwargs like (part html :target "#x") need no quote. *)
      if String.length name > 0 && name.[0] = ':' then Done (VSymbol name)
      else
        (match Env.lookup env name with
         | Some v -> Done v
         | None -> raise (Interp_error (Unbound_var name)))
  | Lit v -> Done v
  | Quote e' -> Done (quote_to_value e')
  | If (test, then_e, else_e) ->
      let v = eval ~budget ~prims env test in
      (match v with
       | VBool false | VNil -> Cont (env, else_e)
       | _ -> Cont (env, then_e))
  | Seq (a, b) ->
      ignore (eval ~budget ~prims env a);
      Cont (env, b)
  | Let (name, val_expr, body) ->
      let v = eval ~budget ~prims env val_expr in
      let env' = Env.child env in
      Env.define env' name v;
      Cont (env', body)
  | Letrec (bindings, body) ->
      let env' = Env.child env in
      List.iter (fun (n, _) -> Env.define env' n VNil) bindings;
      List.iter
        (fun (n, e') ->
          let v = eval ~budget ~prims env' e' in
          Env.define env' n v)
        bindings;
      Cont (env', body)
  | Lambda (params, body) -> Done (VLambda (params, body, env))
  | Define (name, val_expr) ->
      (match parse_top_define (Define (name, val_expr)) with
       | Some { name = n; body; doc = _ } ->
           assert_bindable_name env n;
           let v =
             match body with
             | Lambda (params, b) -> VLambda (params, b, env)
             | other -> eval ~budget ~prims env other
           in
           Env.define env n v;
           Done v
       | None ->
           assert_bindable_name env name;
           let v = eval ~budget ~prims env val_expr in
           Env.define env name v;
           Done v)
  | Prim (name, args) ->
      let arg_vals = List.map (eval ~budget ~prims env) args in
      let _ = prims in
      budget.prim_calls <- budget.prim_calls + 1;
      Done (Control.perform_prim name arg_vals)
  | App (head, args) ->
      (match head with
       | Var name when is_special_form name ->
```

```
            eval_special_form ~budget ~prims env name (head :: args)
       | Var name ->
           (match Env.lookup env name with
            | Some (VMacro (params, body, clo_env)) ->
                expand_and_eval_macro ~budget ~prims env
                  ~params ~body ~clo_env ~arg_exprs:args
            | _ ->
                let f = eval ~budget ~prims env head in
                let arg_vals = List.map (eval ~budget ~prims env) args in
                let step = apply_step ~budget ~prims f arg_vals in
                (match f, step with
                 | VLambda _, Cont _ -> budget.current_fn <- Some name
                 | VPrim _, Done _ ->
                     budget.prim_calls <- budget.prim_calls + 1
                 | _ -> ());
                step)
       | _ ->
           let f = eval ~budget ~prims env head in
           let arg_vals = List.map (eval ~budget ~prims env) args in
           let step = apply_step ~budget ~prims f arg_vals in
           (match f with
            | VPrim _ -> budget.prim_calls <- budget.prim_calls + 1
            | _ -> ());
           step)

(** Force evaluation to a value (trampoline driver). *)
and eval
  ~(budget : budget)
  ~(prims : prim_table)
  (env : env)
  (e : expr)
  : value =
  let prev_fn = budget.current_fn in
  let rec loop env e =
    match eval_step ~budget ~prims env e with
    | Done v ->
        budget.current_fn <- prev_fn;
        v
    | Cont (env', e') -> loop env' e'
  in
  loop env e

(** Expand and evaluate a macro application by name. Args are bound as
    quoted values (VStr/VSymbol, not evaluated); the expansion is
    [value_to_expr]-reified and re-evaluated in [call_env] via [Cont].
    Enforces macro arity. *)
and expand_and_eval_macro
  ~(budget : budget)
  ~(prims : prim_table)
  (call_env : env)
  ~(params : string list)
  ~(body : expr)
  ~(clo_env : env)
  ~(arg_exprs : expr list)
  : step =
  tick budget;
  let expected = List.length params
  and got = List.length arg_exprs in
  if expected <> got then
    raise (Interp_error (Arity_mismatch ("macro", expected, got)));
  (* Bind macro args as quoted values (not evaluated).
       "foo" -> VStr; bare foo -> VSymbol. *)
  let arg_vals = List.map reify_expr arg_exprs in
  let menv = Env.child clo_env in
  List.iter2 (fun p a -> Env.define menv p a) params arg_vals;
  let expansion_v = eval ~budget ~prims menv body in
  let expansion_e =
    try value_to_expr expansion_v
    with Interp_error _ as ex -> raise ex
       | _ ->
           raise
             (Interp_error
```

```
563                 (User_error ("macro expansion is not a valid expr tree",
      "bad-args")))
564   in
565   Cont (call_env, expansion_e)
566
567 (** Fully macroexpand [e] in [env], returning an expr tree.
568     Used by the public-route gate and AI install effect checks so
569     [Effect.infer] sees EXPANDED bodies (lisp-interpreter.effects ?4;
570     BORGE-1785212882). Keeps [Effect.infer] pure -- expansion runs here
571     with the eval rim (macro bodies are expected to be pure templates).
572
573     Does not expand under [Quote]. Recurses into lambda/let/if/seq/app
574     subforms. When the head of an app is a [VMacro], expands once and
575     loops (same as the [macroexpand] special form). *)
576 and macroexpand_expr
577     ?(budget = make_budget ())
578     ?(prims : prim_table = make_prim_table ())
579     (env : env)
580     (e : expr) : expr =
581   tick budget;
582   let go e' = macroexpand_expr ~budget ~prims env e' in
583   match e with
584   | Quote _ as q -> q
585   | Lit _ | Var _ as atom -> atom
586   | Lambda (ps, body) -> Lambda (ps, go body)
587   | If (a, b, c) -> If (go a, go b, go c)
588   | Seq (a, b) -> Seq (go a, go b)
589   | Let (n, v, body) -> Let (n, go v, go body)
590   | Letrec (binds, body) ->
591       Letrec (List.map (fun (n, be) -> (n, go be)) binds, go body)
592   | Define (n, body) -> Define (n, go body)
593   | Prim (name, args) -> Prim (name, List.map go args)
594   | App (Var name, args) ->
595       (match Env.lookup env name with
596        | Some (VMacro (params, body, clo_env)) ->
597            let expected = List.length params
598            and got = List.length args in
599            if expected <> got then
600              raise (Interp_error (Arity_mismatch ("macro", expected, got)));
601            let arg_vals = List.map reify_expr args in
602            let menv = Env.child clo_env in
603            List.iter2 (fun p a -> Env.define menv p a) params arg_vals;
604            let expansion_v = eval ~budget ~prims menv body in
605            let expansion_e =
606              try value_to_expr expansion_v
607              with Interp_error _ as ex -> raise ex
608                 | _ ->
609                     raise
610                       (Interp_error
611                          (User_error
612                             ("macro expansion is not a valid expr tree",
      "bad-args")))
613            in
614            go expansion_e
615        | _ -> App (Var name, List.map go args))
616   | App (h, args) -> App (go h, List.map go args)
617
618 (** Dispatch a special form by [name] (an [App] whose head is a [Var]
619     recognized by [is_special_form]). [full_args] is head :: rest where
620     head is the [Var name]. Handles lambda/let/define/cond/match/guard/
621     quasiquote/defmacro, the list higher-order operators, and more;
622     returns a [step]. *)
623 and eval_special_form
624   ~(budget : budget)
625   ~(prims : prim_table)
626   (env : env)
627   (name : string)
628   (full_args : expr list)
629   : step =
630   match name, full_args with
631   | "lambda", _head :: params_expr :: bodies ->
632       let _doc, bodies' = peel_leading_doc bodies in
633       let body = body_of_exprs bodies' in
```

```
634       let params =
635         match params_expr with
636         | Lit VNil -> []
637         | App (Var p, rest) ->
638             p
639             :: List.map
640                  (fun e ->
641                    match e with
642                    | Var s -> s
643                    | _ ->
644                        raise
645                          (Interp_error (User_error ("lambda param not a
    symbol", "bad-args"))))
646                  rest
647       | _ -> raise (Interp_error (User_error ("malformed lambda params",
    "bad-args")))
648       in
649       if List.mem "." params then
650         raise
651           (Interp_error
652              (User_error
653                 ("dotted rest params not supported (reader treats \".\" as
    a \
654                  symbol); use fixed-arity params", "bad-args")));
655       Done (VLambda (params, body, env))
656   | "let", _ ->
657       (match full_args with
658        | [ _; bindings_expr; body ] ->
659            (match bindings_expr with
660             | Lit VNil -> Cont (env, body)
661             | App (App (Var _, _) as first_binding, rest_bindings) ->
662                 let all_bindings = first_binding :: rest_bindings in
663                 let env' = Env.child env in
664                 List.iter
665                   (fun bp ->
666                     match bp with
667                     | App (Var n, [ val_expr ]) ->
668                         let v = eval ~budget ~prims env val_expr in
669                         Env.define env' n v
670                     | _ -> raise (Interp_error (User_error ("bad let
    binding", "bad-args"))))
671                   all_bindings;
672                 Cont (env', body)
673             | _ -> raise (Interp_error (User_error ("bad let bindings",
    "bad-args"))))
674        | _ -> raise (Interp_error (User_error ("let needs bindings + body",
    "bad-args"))))
675   | "if", [ _; test; then_e; else_e ] ->
676       let v = eval ~budget ~prims env test in
677       (match v with
678        | VBool false | VNil -> Cont (env, else_e)
679        | _ -> Cont (env, then_e))
680   | "delay", [ _; body ] ->
681       (* lisp-interpreter.lazy-env-thunks: capture the recipe, cook on
682          first force. The payload evaluator charges the ACTIVE budget
683          at force time (the request that forces pays) when one exists,
684          else a fresh one, and reuses this eval's prim table. *)
685       let force env' expr' =
686         let b =
687           match !current_budget with Some b -> b | None -> make_budget ()
688         in
689         eval ~budget:b ~prims env' expr'
690       in
691       Done
692         (VThunk
693            (ref
694               (Unforced
695                  { thunk_body = body; thunk_env = env; thunk_force = force
    })))
696   | "delay", _ ->
697       raise (Interp_error (User_error ("delay needs exactly one body",
    "bad-args")))
698   | "force", [ _; x ] ->
```

```
      let v = eval ~budget ~prims env x in
      Done (Env.force_value v)
  | "force", _ ->
      raise (Interp_error (User_error ("force needs exactly one arg",
     "bad-args")))
  | "define", _ ->
      (* Rebuild App without the head name already consumed: full_args is
         head :: rest where head is Var "define". parse_top_define wants
         the whole (define ?) app. *)
      (match parse_top_define (App (Var "define", List.tl full_args)) with
       | Some { name = n; body; doc = _ } ->
           assert_bindable_name env n;
           let v =
             match body with
             | Lambda (params, b) -> VLambda (params, b, env)
             | other -> eval ~budget ~prims env other
           in
           Env.define env n v;
           Done v
       | None ->
           raise
             (Interp_error
                (User_error
                   ("define needs (define name expr [\"doc\"]) or \
                    (define (name args?) [\"doc\"] body?)", "bad-args"))))
  | "quote", [ _; e' ] -> Done (quote_to_value e')
  | "begin", _head :: bodies | "seq", _head :: bodies ->
      (match bodies with
       | [] -> Done VNil
       | [ only ] -> Cont (env, only)
       | first :: rest ->
           ignore (eval ~budget ~prims env first);
           eval_special_form ~budget ~prims env "seq" (Var "seq" :: rest))
  | "error", _head :: args ->
      let kind, msg =
        match args with
        | [ Lit (VStr m) ] | [ Lit (VSymbol m) ] | [ Var m ] ->
     ("user_error", m)
        | [ e ] ->
            (match eval ~budget ~prims env e with
             | VStr m | VSymbol m -> ("user_error", m)
             | other -> ("user_error", pp_value_short other))
        | [ ke; me ] ->
            let k =
              match eval ~budget ~prims env ke with
              | VStr s | VSymbol s -> s
              | other -> pp_value_short other
            in
            let m =
              match eval ~budget ~prims env me with
              | VStr s | VSymbol s -> s
              | other -> pp_value_short other
            in
            (k, m)
        | _ -> ("user_error", "error")
      in
      raise (Interp_error (User_error (msg, kind)))
  | "guard", _head :: handler_expr :: bodies when bodies <> [] ->
      let body = body_of_exprs bodies in
      (try Done (eval ~budget ~prims env body)
       with
       | Interp_error Budget_exceeded as ex -> raise ex
       | Interp_error e ->
           let kind = interp_error_kind e in
           let err_v =
             VList
               [ VSymbol "error"
               ; VStr (pp_interp_error e)
               ; VSymbol kind
               ]
           in
           let hv = eval ~budget ~prims env handler_expr in
           (match hv with
```

```
770             | VLambda (params, hbody, clo) ->
771                 let call = Env.child clo in
772                 (match params with
773                  | [] -> Done (eval ~budget ~prims call hbody)
774                  | [ p ] ->
775                      Env.define call p err_v;
776                      Done (eval ~budget ~prims call hbody)
777                  | _ ->
778                      raise
779                        (Interp_error
780                           (User_error ("guard handler must be 0- or 1-arg lambda", "bad-args"))))
781             | _ ->
782                 raise
783                   (Interp_error
784                      (User_error ("guard handler must be a lambda", "bad-args")))))
785     | "guard", _ ->
786         raise
787           (Interp_error
788              (User_error ("guard needs (guard handler-lambda body?)", "bad-args")))
789     | "match", _head :: scrut_expr :: clauses ->
790         let scrut = eval ~budget ~prims env scrut_expr in
791         let rec try_clauses = function
792           | [] ->
793               raise
794                 (Interp_error (User_error ("match: no matching clause", "bad-args")))
795           | clause :: rest ->
796               let pat, bodies =
797                 match clause with
798                 | App (pat, bs) -> pat, bs
799                 | _ ->
800                     raise
801                       (Interp_error
802                          (User_error ("match clause must be (pat body?)", "bad-args")))
803               in
804               (match match_pat pat scrut with
805                | None -> try_clauses rest
806                | Some binds ->
807                    let env' = Env.child env in
808                    List.iter (fun (n, v) -> Env.define env' n v) binds;
809                    Cont (env', body_of_exprs bodies))
810         in
811         try_clauses clauses
812     | "prim", _ ->
813         (match full_args with
814          | [ _; Var prim_name; body ] ->
815              let arg_vals = [ eval ~budget ~prims env body ] in
816              Done (Control.perform_prim prim_name arg_vals)
817          | _ -> raise (Interp_error (User_error ("prim needs name + arg", "bad-args"))))
818     | "with-transaction", _ ->
819         let body_expr_opt =
820           match full_args with
821           | [ _; body_expr ] -> Some body_expr
822           | [ _; _isolation; body_expr ] -> Some body_expr
823           | _ -> None
824         in
825         (match body_expr_opt with
826          | None ->
827              raise
828                (Interp_error
829                   (User_error ("with-transaction needs a lambda body", "bad-args")))
830           | Some body_expr ->
831              let lambda_v = eval ~budget ~prims env body_expr in
832              (match lambda_v with
833               | VLambda _ ->
834                   Done
835                     (Control.perform_prim "__with_transaction__" [ lambda_v ])
```

```
            | _ ->
                raise
                  (Interp_error
                     (User_error ("with-transaction body must be a lambda",
   "bad-args")))))
  | "with-step", _ ->
      (* (with-step "name" (lambda () body)) -- a checkpoint boundary. In a
         job, the __step__ prim in session.ml memoizes the thunk's result
         under (job-id, name); outside a job it is a plain thunk call.
         Adjacent to with-transaction for the exactly-once
         (with-transaction (lambda () (with-step ?))) composition. Named
         with-step (NOT step) to avoid colliding with the pre-existing
         libs/rb `step` recipe-DSL function. See borge/jobs.borg
         step-checkpoints. *)
      (match full_args with
       | [ _; name_expr; body_expr ] ->
           let name_v = eval ~budget ~prims env name_expr in
           (match name_v with
            | VStr name ->
                let lambda_v = eval ~budget ~prims env body_expr in
                (match lambda_v with
                 | VLambda _ ->
                     Done (Control.perform_prim "__step__" [ VStr name;
   lambda_v ])
                 | _ ->
                     raise
                       (Interp_error
                          (User_error ("with-step body must be a lambda",
   "bad-args"))))
            | _ ->
                raise
                  (Interp_error
                     (User_error ("with-step needs a string name",
   "bad-args"))))
       | _ ->
           raise
             (Interp_error
                (User_error ("with-step needs (with-step \"name\" (lambda
   () body))", "bad-args"))))
  | "gensym", [ _ ] -> Done (VSymbol (gensym ()))
  | "gensym", [ _; Lit (VStr prefix) ] | "gensym", [ _; Var prefix ]
  | "gensym", [ _; Lit (VSymbol prefix) ] ->
      Done (VSymbol (gensym ~prefix ()))
  | "and", _head :: args ->
      let rec loop = function
        | [] -> Done (VBool true)
        | [ last ] -> Cont (env, last)
        | e' :: rest ->
            let v = eval ~budget ~prims env e' in
            if truthy v then loop rest else Done v
      in
      loop args
  | "or", _head :: args ->
      let rec loop = function
        | [] -> Done (VBool false)
        | [ last ] -> Cont (env, last)
        | e' :: rest ->
            let v = eval ~budget ~prims env e' in
            if truthy v then Done v else loop rest
      in
      loop args
  | "cond", _head :: clauses ->
      let rec loop = function
        | [] -> Done VNil
        | clause :: rest ->
            let test_e, body_e =
              match clause with
              | App (t, [ b ]) -> (t, b)
              | App (t, b :: bs) ->
                  let rec fold x xs =
                    match xs with
                    | [] -> x
                    | y :: ys -> Seq (x, fold y ys)
```

```
                in
                (t, fold b bs)
            | Lit _ | Var _ as t -> (t, t)
            | _ ->
                raise
                  (Interp_error
                     (User_error ("cond clause must be (test body)",
   "bad-args")))
          in
          let always =
            match test_e with
            | Var "else" | Lit (VBool true) -> true
            | _ -> false
          in
          if always then Cont (env, body_e)
          else
            let tv = eval ~budget ~prims env test_e in
            if truthy tv then Cont (env, body_e)
            else loop rest
     in
     loop clauses
| "quasiquote", [ _; template ] ->
     Done (eval_quasiquote ~budget ~prims env ~depth:1 template)
| "unquote", _ | "unquote-splicing", _ ->
     raise (Interp_error (User_error ("unquote outside quasiquote",
    "bad-args")))
| "defmacro", [ _; Var name; params_expr; body ] ->
     assert_bindable_name env name;
     let params =
       match params_expr with
       | Lit VNil -> []
       | App (Var p, rest) ->
           p
           :: List.map
                (function
                  | Var s -> s
                  | _ ->
                      raise
                        (Interp_error
                           (User_error ("defmacro param not a symbol",
   "bad-args"))))
                rest
       | _ -> raise (Interp_error (User_error ("malformed defmacro
   params", "bad-args")))
     in
     if List.mem "." params then
       raise
         (Interp_error
            (User_error
               ("dotted rest params not supported (reader treats \".\" as
   a \
                symbol); use fixed-arity params, e.g. (name val body)",
   "bad-args")));
     let m = VMacro (params, body, env) in
     Env.define env name m;
     Done m
| "macroexpand-1", [ _; form ] ->
     (match form with
      | App (Var name, args) ->
          (match Env.lookup env name with
           | Some (VMacro (params, body, clo_env)) ->
               let expected = List.length params
               and got = List.length args in
               if expected <> got then
                 raise
                   (Interp_error (Arity_mismatch ("macro", expected,
   got)));
               let arg_vals =
                 List.map
                   (fun e ->
                     match e with
                     | Lit v -> v
                     | _ -> reify_expr e)
```

```
                    args
                in
                let menv = Env.child clo_env in
                List.iter2 (fun p a -> Env.define menv p a) params arg_vals;
                Done (eval ~budget ~prims menv body)
            | _ -> Done (reify_expr form))
        | _ -> Done (reify_expr form))
  | "macroexpand", [ _; form ] ->
      let rec loop e =
        tick budget;
        match e with
        | App (Var name, args) ->
            (match Env.lookup env name with
             | Some (VMacro (params, body, clo_env)) ->
                 let expected = List.length params
                 and got = List.length args in
                 if expected <> got then
                   raise
                     (Interp_error (Arity_mismatch ("macro", expected,
   got)));
                 let arg_vals =
                   List.map
                     (fun e' ->
                       match e' with
                       | Lit v -> v
                       | _ -> reify_expr e')
                     args
                 in
                 let menv = Env.child clo_env in
                 List.iter2 (fun p a -> Env.define menv p a) params
   arg_vals;
                 let v = eval ~budget ~prims menv body in
                 let e' = value_to_expr v in
                 loop e'
             | _ -> reify_expr e)
        | _ -> reify_expr e
      in
      Done (loop form)
  | "letrec", _ ->
      (match full_args with
       | [ _; bindings_expr; body ] ->
           (match bindings_expr with
            | Lit VNil -> Cont (env, body)
            | App (App (Var _, _) as first_binding, rest_bindings) ->
                let all_bindings = first_binding :: rest_bindings in
                let env' = Env.child env in
                (* Backpatch: pre-bind all names to VNil so binding exprs
                   (lambdas) can close over env' and refer to each other
                   for mutual recursion, then evaluate and overwrite. *)
                List.iter
                  (fun bp ->
                    match bp with
                    | App (Var n, _) -> Env.define env' n VNil
                    | _ ->
                        raise (Interp_error (User_error ("bad letrec
   binding", "bad-args"))))
                  all_bindings;
                List.iter
                  (fun bp ->
                    match bp with
                    | App (Var n, [ val_expr ]) ->
                        let v = eval ~budget ~prims env' val_expr in
                        Env.define env' n v
                    | _ ->
                        raise (Interp_error (User_error ("bad letrec
   binding", "bad-args"))))
                  all_bindings;
                Cont (env', body)
            | _ -> raise (Interp_error (User_error ("bad letrec bindings",
   "bad-args"))))
       | _ -> raise (Interp_error (User_error ("letrec needs bindings +
   body", "bad-args"))))
  (* apply: (apply f args-list) applies f to the list of args.
```

```
     First-class function application -- reuses the same apply helper
     that list/map etc. use, so closures and VPrims both work. *)
  | "apply", [ _; f_expr; args_expr ] ->
      let f = eval ~budget ~prims env f_expr in
      let args = value_to_list (eval ~budget ~prims env args_expr) in
      Done (apply ~budget ~prims f args)
  | "apply", _ -> raise (Interp_error (User_error ("apply needs (f args)",
     "bad-args")))
  (* list/map, list/filter, list/foldl -- OCaml loops + apply.
     One special-form tick total instead of O(n) recursive Lisp frames.
     Closure bodies still tick normally via apply. *)
  | "list/map", [ _; f_expr; xs_expr ] ->
      let f = eval ~budget ~prims env f_expr in
      let xs = value_to_list (eval ~budget ~prims env xs_expr) in
      Done (VList (List.map (fun x -> apply ~budget ~prims f [ x ]) xs))
  | "list/filter", [ _; p_expr; xs_expr ] ->
      let p = eval ~budget ~prims env p_expr in
      let xs = value_to_list (eval ~budget ~prims env xs_expr) in
      Done
        (VList
           (List.filter
              (fun x -> truthy (apply ~budget ~prims p [ x ]))
              xs))
  | "list/foldl", [ _; f_expr; init_expr; xs_expr ] ->
      let f = eval ~budget ~prims env f_expr in
      let init = eval ~budget ~prims env init_expr in
      let xs = value_to_list (eval ~budget ~prims env xs_expr) in
      Done
        (List.fold_left
           (fun acc x -> apply ~budget ~prims f [ acc; x ])
           init xs)
  | "list/foldr", [ _; f_expr; init_expr; xs_expr ] ->
      (* Right fold via reverse + left fold with swapped arg order:
         foldr f init [a;b;c] = f a (f b (f c init))
         = foldl (fun acc x -> f x acc) init (rev xs).
         Tail-recursive (OCaml loops), one special-form tick. *)
      let f = eval ~budget ~prims env f_expr in
      let init = eval ~budget ~prims env init_expr in
      let xs = value_to_list (eval ~budget ~prims env xs_expr) in
      Done
        (List.fold_left
           (fun acc x -> apply ~budget ~prims f [ x; acc ])
           init (List.rev xs))
  | "list/map", _ ->
      raise (Interp_error (User_error ("list/map needs (f xs)",
     "bad-args")))
  | "list/filter", _ ->
      raise (Interp_error (User_error ("list/filter needs (p xs)",
     "bad-args")))
  | "list/foldl", _ ->
      raise (Interp_error (User_error ("list/foldl needs (f init xs)",
     "bad-args")))
  | "list/foldr", _ ->
      raise (Interp_error (User_error ("list/foldr needs (f init xs)",
     "bad-args")))
  | other, _ ->
      raise
        (Interp_error (User_error ("malformed special form: " ^ other,
     "bad-args")))

(** Evaluate a quasiquote [template] at nesting [depth]. Unquote at
    depth 1 evaluates the inner expr; deeper unquotes are preserved as
    nested lists; unquote-splicing splices list elements. *)
and eval_quasiquote
  ~(budget : budget)
  ~(prims : prim_table)
  (env : env)
  ~(depth : int)
  (template : expr)
  : value =
  tick budget;
  match template with
  | App (Var "unquote", [ e ]) when depth = 1 ->
```

```
      eval ~budget ~prims env e
  | App (Var "unquote", [ e ]) ->
      let inner = eval_quasiquote ~budget ~prims env ~depth:(depth - 1) e in
      VList [ VSymbol "unquote"; inner ]
  | App (Var "unquote-splicing", _) when depth = 1 ->
      raise
        (Interp_error (User_error ("unquote-splicing not in list context",
   "bad-args")))
  | App (Var "quasiquote", [ e ]) ->
      let inner =
        eval_quasiquote ~budget ~prims env ~depth:(depth + 1) e
      in
      VList [ VSymbol "quasiquote"; inner ]
  | App (h, args) ->
      let head_part =
        match h with
        | Var "unquote-splicing" when depth = 1 ->
            raise
              (Interp_error
                 (User_error ("unquote-splicing as list head not
   supported", "bad-args")))
        | _ ->
            let hv = eval_quasiquote ~budget ~prims env ~depth h in
            [ hv ]
      in
      let tail_parts =
        List.map
          (fun a ->
            match a with
            | App (Var "unquote-splicing", [ e ]) when depth = 1 ->
                let v = eval ~budget ~prims env e in
                (match v with
                 | VList xs -> xs
                 | VNil -> []
                 | _ ->
                     raise
                       (Interp_error
                          (User_error ("unquote-splicing needs a list",
   "bad-args"))))
            | _ ->
                let v = eval_quasiquote ~budget ~prims env ~depth a in
                [ v ])
          args
      in
      VList (head_part @ List.concat tail_parts)
  | Lit v -> v
  | Var s -> VSymbol s
  | Quote e ->
      let v = eval_quasiquote ~budget ~prims env ~depth e in
      VList [ VSymbol "quote"; v ]
  | other -> reify_expr other

(** Apply [f] to [args], returning a [step] without forcing. Closures
    (with &optional/&rest support) bind params and return [Cont]; VPrims
    perform the effect; a [VMacro] applied first-class is an error. *)
and apply_step
  ~(budget : budget)
  ~(prims : prim_table)
  (f : value)
  (args : value list)
  : step =
  match f with
  | VLambda (params, body, clo_env) ->
      (* Parse param spec: required, &optional, &rest markers.
         params is a flat string list; &optional and &rest are markers.
         Layout: required... [&optional opt...] [&rest name]. *)
      let has_marker =
        List.exists (fun p -> p = "&optional" || p = "&rest") params
      in
      if not has_marker then begin
        (* Fixed arity -- original path *)
        let expected = List.length params and got = List.length args in
        if expected <> got then
```

```
          raise (Interp_error (Arity_mismatch ("lambda", expected, got)));
        let call_env = Env.child clo_env in
        List.iter2 (fun p a -> Env.define call_env p a) params args;
        Cont (call_env, body)
      end else begin
        (* Parse markers *)
        let rec split_params ps required opt_phase opt_acc rest_acc =
          match ps with
          | [] -> (List.rev required, List.rev opt_acc, rest_acc)
          | p :: rest when p = "&optional" ->
              split_params rest required true opt_acc rest_acc
          | p :: rest when p = "&rest" ->
              (match rest with
               | [ r ] ->
                   split_params [] required false opt_acc (Some r)
               | _ ->
                   raise
                     (Interp_error
                        (User_error ("&rest must be last and have one name", "bad-args"))))
          | p :: rest when opt_phase ->
              split_params rest required true (p :: opt_acc) rest_acc
          | p :: rest ->
              if opt_phase then
                split_params rest required true (p :: opt_acc) rest_acc
              else
                split_params rest (p :: required) false opt_acc rest_acc
        in
        let (required, optionals, rest_name) =
          split_params params [] false [] None
        in
        let n_required = List.length required
        and n_optional = List.length optionals
        and n_args = List.length args in
        (* Arity check: at least required, at most required+optional (if no rest) *)
        if n_args < n_required then
          raise (Interp_error (Arity_mismatch ("lambda", n_required, n_args)));
        (match rest_name with
         | None ->
             if n_args > n_required + n_optional then
               raise
                 (Interp_error
                    (Arity_mismatch ("lambda", n_required + n_optional, n_args)))
         | Some _ -> ());
        let call_env = Env.child clo_env in
        (* Bind required *)
        let req_args = List.take n_required args in
        List.iter2 (fun p a -> Env.define call_env p a) required req_args;
        (* Bind optional (fill missing with empty string VStr "") *)
        let remaining = List.drop n_required args in
        let opt_args = List.take n_optional remaining in
        let n_supplied_opt = List.length opt_args in
        let supplied_opt_names = List.take n_supplied_opt optionals in
        List.iter2
          (fun p a -> Env.define call_env p a)
          supplied_opt_names opt_args;
        (* Fill missing optionals with "" *)
        let missing_opt = List.drop n_supplied_opt optionals in
        List.iter (fun p -> Env.define call_env p (VStr "")) missing_opt;
        (* Bind rest *)
        (match rest_name with
         | Some r ->
             let rest_args = List.drop n_optional remaining in
             Env.define call_env r (VList rest_args)
         | None -> ());
        Cont (call_env, body)
      end
  | VMacro _ ->
      raise
        (Interp_error
```

```
          (User_error
             ("macro must be applied by name (not as first-class apply)",
    "bad-args")))
  | VPrim name ->
      let _ = prims in
      (* prim_calls counted at App/Prim call sites *)
      Done (Control.perform_prim name args)
  | other -> raise (Interp_error (Not_a_function other))

(** Apply [f] to [args], forcing the result to a [value] via [eval] when
    the closure returns [Cont]. *)
and apply
  ~(budget : budget)
  ~(prims : prim_table)
  (f : value)
  (args : value list)
  : value =
  match apply_step ~budget ~prims f args with
  | Done v -> v
  | Cont (env, body) -> eval ~budget ~prims env body

(** Eval one toplevel form read from [s] directly, without any rim --
    the caller owns the [Prim_call] effect. Records the fresh budget in
    [last_budget] for image persistence. *)
let eval_string_direct ~(prims : prim_table) (env : env) (s : string) :
    value =
  let budget = make_budget () in
  last_budget := Some budget;
  let e = Reader.read_one s in
  eval ~budget ~prims env e

(** Evaluate under [Control.run_sync] (pure tests; sync prim table). *)
let eval_string ~(prims : prim_table) (env : env) (s : string) : value =
  Control.run_sync ~prims (fun () -> eval_string_direct ~prims env s)

(** Evaluate all toplevel forms in [s], sequentially, under the given
    prim table but WITHOUT installing any rim -- the caller hosts the
    [Prim_call] effect (hosts: image Control_rim in session, agent
    Control.run_with_rim). Returns the last form's value. Used by host
    plugin/file loaders that want file-per-file or form-per-form eval
    without string re-rendering. (Added for borge/agent.borg
    nopales-embed.) *)
let read_eval_all_direct ~(prims : prim_table) (env : env) (s : string)
    : value =
  let forms = Reader.read_all s in
  let budget = make_budget () in
  last_budget := Some budget;
  List.fold_left
    (fun _acc form -> eval ~budget ~prims env form)
    VNil forms

(** An empty [prim_table] convenience value. *)
let empty_prims : prim_table = make_prim_table ()

(** Aliases kept for call-site stability during H1 cleanup. *)
let apply_eff = apply
let eval_eff = eval
let eval_string_eff = eval_string_direct
let eval_string_auto = eval_string
```

239 lisp/lib/explain.ml

```
(** Mechanical inside-out (innermost-first) peel of a Nopales AST.

    Pure structure + light annotations. LLM glosses are image-layer. *)
open Ast
open Recursion

(** One peel layer: depth, a human kind label, an annotation string,
    the subexpression, and its path relative to the root. *)
type layer = {
  depth : int;
  kind : string;
  annotation : string;
  expr : expr;
  path : int list;
}

(** Short label for an expression shape (e.g. ["var"], ["call:map"]). *)
let kind_of = function
  | Var _ -> "var"
  | Lit _ -> "lit"
  | App (Var n, _) -> "call:" ^ n
  | App _ -> "app"
  | Let _ -> "let"
  | Letrec _ -> "letrec"
  | If _ -> "if"
  | Lambda _ -> "lambda"
  | Prim (n, _) -> "prim:" ^ n
  | Seq _ -> "seq"
  | Quote _ -> "quote"
  | Define _ -> "define"

(** Build a human annotation for [e] describing recursion kind (tail /
    non-tail), primitives, calls and literals, per
     [lisp-interpreter.eval].*)
let annotate ~(fn_name : string) ~(info : recursion_info) (e : expr)
    ~(path : int list) : string =
  let bits = ref [] in
  (match e with
   | App (Var n, _) when n = fn_name ->
       bits := "recursive call" :: !bits;
       if is_tail_self_path info path then bits := "tail" :: !bits
       else if is_nontail_self_path info path then bits := "non-tail" ::
     !bits
   | Prim (n, _) -> bits := ("prim " ^ n) :: !bits
   | App (Var n, _) -> bits := ("calls " ^ n) :: !bits
   | Lit _ -> bits := "literal" :: !bits
   | Var n -> bits := ("var " ^ n) :: !bits
   | If _ -> bits := "conditional" :: !bits
   | Lambda _ -> bits := "closure" :: !bits
   | _ -> ());
  match !bits with
  | [] -> kind_of e
  | xs -> String.concat "; " (List.rev xs)

(** Depth of an expression (1 = leaf). *)
let rec depth_of = function
  | Var _ | Lit _ | Quote _ -> 1
  | App (f, args) ->
      1 + List.fold_left max (depth_of f) (List.map depth_of args)
  | Let (_, a, b) | Seq (a, b) -> 1 + max (depth_of a) (depth_of b)
  | Letrec (bs, body) ->
      let d =
        List.fold_left max (depth_of body)
          (List.map (fun (_, e) -> depth_of e) bs)
      in
      1 + d
  | If (a, b, c) ->
      1 + max (depth_of a) (max (depth_of b) (depth_of c))
  | Lambda (_, body) | Define (_, body) -> 1 + depth_of body
  | Prim (_, args) ->
      1
```

```
      + List.fold_left max 0 (List.map depth_of args)

(** All subexpressions with path and depth, leaves first (stable). *)
let collect_nodes (e : expr) : (int * int list * expr) list =
  let acc = ref [] in
  let rec go path e =
    acc := (depth_of e, List.rev path, e) :: !acc;
    match e with
    | Var _ | Lit _ | Quote _ -> ()
    | App (f, args) ->
        go (0 :: path) f;
        List.iteri (fun i a -> go ((i + 1) :: path) a) args
    | Let (_, v, body) ->
        go (0 :: path) v;
        go (1 :: path) body
    | Letrec (bs, body) ->
        List.iteri (fun i (_, e') -> go (i :: path) e') bs;
        go (List.length bs :: path) body
    | If (t, th, el) ->
        go (0 :: path) t;
        go (1 :: path) th;
        go (2 :: path) el
    | Lambda (_, body) | Define (_, body) -> go (0 :: path) body
    | Prim (_, args) -> List.iteri (fun i a -> go (i :: path) a) args
    | Seq (a, b) ->
        go (0 :: path) a;
        go (1 :: path) b
  in
  go [] e;
  (* Innermost first: higher depth first; then left-to-right by path. *)
  !acc
  |> List.sort (fun (d1, p1, _) (d2, p2, _) ->
         let c = compare d2 d1 in
         if c <> 0 then c else compare p1 p2)

(** Peel [e] (assumed to be [fn_name]'s body) into an innermost-first
    list of layers, each annotated with recursion info. *)
let peel ~(fn_name : string) (e : expr) : layer list =
  let body = unwrap_fn_body e in
  let info = analyze_self ~fn_name body in
  collect_nodes body
  |> List.mapi (fun i (_d, path, expr) ->
         {
           depth = i;
           kind = kind_of expr;
           annotation = annotate ~fn_name ~info expr ~path;
           expr;
           path;
         })

(** Render peel [layers] as human-readable lines (depth/kind/annotation). *)
let layers_to_strings (layers : layer list) : string list =
  List.map
    (fun (l : layer) ->
      Printf.sprintf "[%d] %s -- %s\n  %s" l.depth l.kind l.annotation
        (Reader.pp_expr l.expr))
    layers

(** ---- Stepping / trace debugger (borge/remote-eval.borg stepping) ----

    The LLM-debugger substrate. eval runs in a client process; stepping
    is PURELY a client concern -- the server never learns a breakpoint
    fired. The data plane still crosses the wire exactly as a fast eval
    would: pure compute stays local, every prim call that is not a local
    pure prim is forwarded via [transport], and its value/error returns
    to the paused continuation.

    [trace] is a ONE-PASS trace: it runs [src] once under a proxy-style
    rim and records an ordered, interleaved event list whose [Local]
    entries count the local pure eval steps since the last wire crossing
    and whose [Remote_ok]/[Remote_err] entries are the remote prim round
    trips (distinguished from local steps). The [Breakpoint]/[Step]/
    [Continue] driver walks that already-produced trace (presentation
```

```
    stepping): it NEVER re-evals, so side effects are never repeated and
    the wire is never re-crossed -- stepping is a client concern by
    construction (remote-eval.borg hard-constraints). *)

(** One trusted event in a [trace] run. [Local n] = n local pure eval
    steps since the last wire crossing (0 between two adjacent round
    trips); [Remote_ok (name, value)] = a remote prim round-trip
    returned [value]; [Remote_err (name, err)] = it relayed an error. *)
type trace_event =
  | Local of int
  | Remote_ok of string * string
  | Remote_err of string * string

(** A full one-pass trace of one expression. [events] is in execution
    order (innermost-first stepping would revisit, but the trace records
    the actual crossed wire + local-step sequence). *)
type trace_result = {
  result : string;         (* final value, or "error: ?" on a raised exn *)
  events : trace_event list;
  total_steps : int;       (* budget steps consumed by the whole eval *)
}

let pp_trace_event = function
  | Local n -> Printf.sprintf "local %d step(s)" n
  | Remote_ok (name, v) -> Printf.sprintf "remote %s -> %s" name v
  | Remote_err (name, e) -> Printf.sprintf "remote %s !! %s" name e

(** Render a [trace_result] as display lines (one per event, plus the
    outcome). A trace of an expression shows local pure steps
    interleaved with remote prim round trips, distinguished. *)
let trace_to_strings (tr : trace_result) : string list =
  List.map pp_trace_event tr.events
  @ [ Printf.sprintf "=> %s (%d steps)" tr.result tr.total_steps ]

(* Stepping uses OCaml-5 effect handlers (a proxy-style rim intercepting
   [Control.Prim_call]): alias Stdlib.Effect here so [Effect.Deep] in the
   trace handler below resolves. *)
module Effect = Stdlib.Effect

(** Run [src] once under a proxy-style rim and record the interleaved
    local/remote event list. [local_prims] is the local pure prim table
    (serviced locally -- counts as local steps, never crosses the wire);
    [transport] is the remote round trip (name -> args -> (value, exn),
    the same boundary [Proxy_rim.transport] uses); [init_env] is the
    starting env (defaults to a fresh top-level env). Pure and Lwt-free. *)
let trace
    ~(local_prims : Eval.prim_table)
    ~(transport : string -> Ast.value list -> (Ast.value, exn) result)
    ~(init_env : Ast.env)
    ~(src : string)
    : trace_result =
  let env = init_env in
  let events = ref ([] : trace_event list) in
  let last_steps = ref 0 in
  (* sample current-budget absolute steps; attribute a [Local n] span
     since the last sample (the running counter is published by
     [Eval.eval_step] while we are inside the eval that performed the
     [Prim_call]). *)
  let add_local () =
    let n = Eval.current_steps () - !last_steps in
    last_steps := Eval.current_steps ();
    if n > 0 then events := Local n :: !events
  in
  let budget = Eval.make_budget () in
  Eval.last_budget := Some budget;
  let outcome =
    try
      let forms = Reader.read_all src in
      let open Effect.Deep in
      match_with
        (fun () ->
          List.fold_left
            (fun _ f -> Eval.eval ~budget ~prims:local_prims env f)
```

```
            VNil forms)
        ()
        { retc = (fun v -> `Ok v)
        ; exnc = (fun e -> `Exn e)
        ; effc =
            (fun (type a) (eff : a Effect.t) ->
              match eff with
              | Control.Prim_call (name, args) ->
                  Some
                    (fun (k : (a, _) continuation) ->
                      match Hashtbl.find_opt local_prims name with
                      | Some fn ->
                          (* local pure prim: serviced locally, a local
                             step, no wire crossing *)
                          (try continue k (fn args)
                           with exn -> discontinue k exn)
                      | None ->
                          add_local ();
                          (match transport name args with
                           | Ok v ->
                               events :=
                                 Remote_ok (name, Reader.pp_value v) ::
    !events;
                               continue k v
                           | Error e ->
                               events :=
                                 Remote_err (name, Printexc.to_string e)
                                 :: !events;
                               discontinue k e))
              | _ -> None)
        }
    with exn -> `Exn exn
  in
  add_local ();
  {
    result =
      (match outcome with
       | `Ok v -> Reader.pp_value v
       | `Exn e -> "error: " ^ Printexc.to_string e);
    events = List.rev !events;
    total_steps = budget.steps;
  }

(** A drivable stepper over an already-produced [trace_result]. [Step]
    advances one event, [Continue] advances to the next breakpoint, and
    [Frames] inspects the source-level AST at the current position using
    the peel above. It never re-evals the source, so nothing is repeated
    and the wire is never re-crossed. *)
type stepper = {
  tr : trace_result;
  mutable cursor : int;          (* index of the next event to step to *)
  mutable breakpoints : (int -> trace_event -> bool) list;
}

let stepper_of_trace (tr : trace_result) : stepper = { tr; cursor = 0;
    breakpoints = [] }

(** The index of the current event (past the last event when done). *)
let current_index (s : stepper) : int = s.cursor

(** True when the stepper has consumed every event. *)
let done_p (s : stepper) : bool = s.cursor >= List.length s.tr.events

(** Advance one event; raise [End_of_file] when already done. Returns
    the event stepped onto. *)
let step (s : stepper) : trace_event =
  if done_p s then raise End_of_file;
  let e = List.nth s.tr.events s.cursor in
  s.cursor <- s.cursor + 1;
  e

(** Register a breakpoint predicate; [continue] stops at the first
    future event satisfying any registered predicate. *)
```

```
let add_breakpoint (s : stepper) (pred : int -> trace_event -> bool) : unit
    =
  s.breakpoints <- pred :: s.breakpoints

(** Advance (step) until the next breakpoint fires or the trace ends.
    Returns [true] if a breakpoint fired, [false] if the trace ran to
    completion. *)
let continue (s : stepper) : bool =
  let fired = ref false in
  while (not !fired) && not (done_p s) do
    let _ = step s in
    let idx = s.cursor - 1 in
    let e = List.nth s.tr.events idx in
    if List.exists (fun p -> p idx e) s.breakpoints then fired := true
  done;
  !fired

(** A convenient breakpoint: pause on a remote prim round trip whose
    name matches [name] (the natural "break on this server prim"). *)
let break_on_remote (name : string) : int -> trace_event -> bool =
  fun _idx -> function
    | Remote_ok (n, _) | Remote_err (n, _) -> n = name
    | Local _ -> false

(** Inspect the source-level frame stack at the current position: the
    peeked AST layers of [src] (the peel machinery above). [src] may
    hold several top-level forms; each is peeled independently. *)
let frames (src : string) : string list =
  let forms = try Reader.read_all src with _ -> [] in
  List.flatten
    (List.map (fun f -> layers_to_strings (peel ~fn_name:"<expr>" f)) forms)
```

240 lisp/lib/reader.ml

```
(** Pricklypear Lisp -- S-expression reader.

    Parses surface s-expression syntax -> Ast.expr. The AST IS the
    source (homoiconicity): a function stored as JSONB is its AST, and
    the s-expr syntax is one of several possible editors (along with
    the htmx tree-editor that edits the AST directly).

    Per spec [lisp-interpreter.reader]:
      - Standard s-expressions: (foo bar baz), 'symbol, "string", 123,
        #t, #f, ()
      - String escapes: named (n/t/r/backslash/quote/apostrophe),
        three-digit decimal byte (\\ddd), unicode scalar (\\uXXXX,
        UTF-8 encoded; lone surrogates rejected); other backslash-X
        is forgiving (literal X)
      - Comments: ; to end of line
      - Docstrings: a leading string in a body position (deferred; reader
        just produces the AST, docstring extraction is eval-time)
      - Special forms [lambda/let/letrec/if/define/quote] are recognized
        at EVAL time, not read time -- the reader produces plain App
        nodes; eval dispatches them.

    The reader tracks line/col for error messages. *)
open Ast

(** A parse error: (message, line, col). *)
type reader_error = string * int * int
exception Reader_error of reader_error

(** True for whitespace (space, tab, newline, carriage return). *)
let is_space c = c = ' ' || c = '\t' || c = '\n' || c = '\r'

(** True for ASCII digit characters. *)
let is_digit c = c >= '0' && c <= '9'

(** True for characters that may begin a symbol: anything that is not a
    delimiter (paren, quote, double-quote, semicolon) and not whitespace. *)
let is_symbol_start c =
  (c <> '(' && c <> ')' && c <> '\'' && c <> '"'
  && c <> ';' && not (is_space c))

(** True for characters that may continue a symbol (start chars, digits,
    '#'). *)
let is_symbol_cont c = is_symbol_start c || is_digit c || c = '#'

(** True at a token start. '#' may begin a token for #t/#f (and later
    #\char etc). *)
let is_token_start c = c = '#' || is_symbol_start c

(** True for characters that may continue a token. *)
let is_token_cont c = is_symbol_cont c

(** Reader cursor state: the remaining input, current position, and the
    line/col of the current position (for error messages). *)
type state = {
  mutable input : string;
  mutable pos : int;
  mutable line : int;
  mutable col : int;
}

(** [make_state s] builds an initial cursor over [s] at position (1,1). *)
let make_state (s : string) : state =
  { input = s; pos = 0; line = 1; col = 1 }

(** [peek st] returns the next character without consuming it, or [None]
    at end of input. *)
let peek (st : state) : char option =
  if st.pos >= String.length st.input then None
  else Some st.input.[st.pos]

(** [advance st] consumes and returns the next character (or [None] at
```

```
    end of input), tracking line/col across newlines. *)
let advance (st : state) : char option =
  match peek st with
  | None -> None
  | Some c ->
      st.pos <- st.pos + 1;
      if c = '\n' then (st.line <- st.line + 1; st.col <- 1)
      else st.col <- st.col + 1;
      Some c

(** [skip_ws_and_comments st] skips whitespace and ';' line comments,
    leaving the cursor at the next significant character (or EOF). *)
let skip_ws_and_comments (st : state) : unit =
  let rec loop () =
    match peek st with
    | None -> ()
    | Some ' ' | Some '\t' | Some '\n' | Some '\r' ->
        ignore (advance st); loop ()
    | Some ';' ->
        (* line comment: skip to end of line or EOF, then keep looping
           so the trailing newline (and any further ws/comments) is
           consumed. The old code re-peeked the same ';' without
           advancing, so it only consumed the ';' itself and left the
           rest of the line to be read as a symbol -- e.g. a
           `;; package ...` header lexed as the symbol `package`. *)
        (let rec skip_to_eol () =
           match peek st with
           | Some '\n' | None -> ()
           | _ -> ignore (advance st); skip_to_eol ()
         in skip_to_eol (); loop ())
    | _ -> ()
  in
  loop ()

(** True for ASCII digit characters. *)
let is_digit c = c >= '0' && c <= '9'

(** Numeric value of an ASCII digit. *)
let digit_val c = Char.code c - Char.code '0'

(** True for hexadecimal digit characters. *)
let is_hex c =
  (c >= '0' && c <= '9') || (c >= 'a' && c <= 'f') || (c >= 'A' && c <= 'F')

(** Numeric value of a hexadecimal digit character. *)
let hex_val c =
  if c >= '0' && c <= '9' then Char.code c - Char.code '0'
  else if c >= 'a' && c <= 'f' then 10 + Char.code c - Char.code 'a'
  else 10 + Char.code c - Char.code 'A'

(** UTF-8 encode a Unicode scalar value into buf (1-4 bytes).
    Caller must have rejected lone surrogates (D800..DFFF). *)
let utf8_add_scalar (buf : Buffer.t) (n : int) : unit =
  if n < 0x80 then Buffer.add_char buf (Char.chr n)
  else if n < 0x800 then begin
    Buffer.add_char buf (Char.chr (0xC0 lor (n lsr 6)));
    Buffer.add_char buf (Char.chr (0x80 lor (n land 0x3F)))
  end else if n < 0x10000 then begin
    Buffer.add_char buf (Char.chr (0xE0 lor (n lsr 12)));
    Buffer.add_char buf (Char.chr (0x80 lor ((n lsr 6) land 0x3F)));
    Buffer.add_char buf (Char.chr (0x80 lor (n land 0x3F)))
  end else begin
    Buffer.add_char buf (Char.chr (0xF0 lor (n lsr 18)));
    Buffer.add_char buf (Char.chr (0x80 lor ((n lsr 12) land 0x3F)));
    Buffer.add_char buf (Char.chr (0x80 lor ((n lsr 6) land 0x3F)));
    Buffer.add_char buf (Char.chr (0x80 lor (n land 0x3F)))
  end

(** Escape a string for Nopales source. Keep UTF-8 bytes as-is so
    round-trip through the reader preserves non-ASCII text. Only quote,
    backslash, and a few controls get named escapes; other controls use
    exactly-three-digit decimal [\\ddd] (OCaml-compatible).
    \\uXXXX is a READ-side escape only -- the printer never emits it;
```

```
    UTF-8 bytes go out raw and the reader accepts them on the way back. *)
let escape_string (s : string) : string =
  let buf = Buffer.create (String.length s + 8) in
  String.iter
    (fun c ->
      match c with
      | '"' -> Buffer.add_string buf "\\\""
      | '\\' -> Buffer.add_string buf "\\\\"
      | '\n' -> Buffer.add_string buf "\\n"
      | '\t' -> Buffer.add_string buf "\\t"
      | '\r' -> Buffer.add_string buf "\\r"
      | c when Char.code c < 0x20 ->
          Buffer.add_string buf (Printf.sprintf "\\%03d" (Char.code c))
      | c -> Buffer.add_char buf c)
    s;
  Buffer.contents buf

(** [read_string st] reads a string literal. Assumes the current char is
    the opening quote (which is consumed). Handles named escapes
    (n/t/r/backslash/quote/apostrophe), three-digit decimal bytes
    [\\ddd], and unicode [\\uXXXX] (lone surrogates rejected); any other
    backslash-X is treated as literal X. Raises [Reader_error] on
    unterminated strings or bad escapes. *)
let read_string (st : state) : string =
  (* assumes current char is the opening quote *)
  ignore (advance st);  (* consume the opening quote *)
  let buf = Buffer.create 16 in
  let rec loop () =
    match peek st with
    | None ->
        raise (Reader_error ("unterminated string", st.line, st.col))
    | Some '"' ->
        ignore (advance st);
        Buffer.contents buf
    | Some '\\' ->
        (* Escape sequences in string literals.
           Named: n t r backslash double-quote single-quote.
           Exactly three decimal digits [\\ddd] ? byte (OCaml String.escaped
           form; needed so export/pull vaults with non-ASCII still load).
           Any other backslash-X is treated as literal X (forgiving), so
           editor over-escapes and paste noise do not hard-fail. *)
        ignore (advance st);  (* consume the backslash we peeked *)
        (match peek st with
         | None ->
             raise (Reader_error ("unterminated escape", st.line, st.col))
         | Some 'n' -> ignore (advance st); Buffer.add_char buf '\n'
         | Some 't' -> ignore (advance st); Buffer.add_char buf '\t'
         | Some 'r' -> ignore (advance st); Buffer.add_char buf '\r'
         | Some '\\' -> ignore (advance st); Buffer.add_char buf '\\'
         | Some '"' -> ignore (advance st); Buffer.add_char buf '"'
         | Some '\'' -> ignore (advance st); Buffer.add_char buf '\''
         | Some 'u' ->
             (* \uXXXX unicode scalar (exactly 4 hex digits, UTF-8
                encoded). Lone surrogates are rejected: v1 does not
                combine surrogate pairs. *)
             ignore (advance st);  (* consume the 'u' *)
             let hex4 () =
               match peek st with
               | Some c when is_hex c -> ignore (advance st); hex_val c
               | _ ->
                   raise
                     (Reader_error
                        ("bad unicode escape \\u (need 4 hex digits)",
                         st.line, st.col))
             in
             (* Bind left-to-right: OCaml evaluates + operands
    right-to-left,
                which would reverse nibble significance (203e ? e302). *)
             let h1 = hex4 () in
             let h2 = hex4 () in
             let h3 = hex4 () in
             let h4 = hex4 () in
             let n = (h1 * 4096) + (h2 * 256) + (h3 * 16) + h4 in
```

```
            if n >= 0xD800 && n <= 0xDFFF then
              raise
                (Reader_error
                   (Printf.sprintf "bad unicode escape \\u%04X (lone surrogate)" n,
                    st.line, st.col))
            else utf8_add_scalar buf n
        | Some d1 when is_digit d1 ->
            ignore (advance st);
            (match peek st with
             | Some d2 when is_digit d2 ->
                 ignore (advance st);
                 (match peek st with
                  | Some d3 when is_digit d3 ->
                      ignore (advance st);
                      let n =
                        (digit_val d1 * 100) + (digit_val d2 * 10) + digit_val d3
                      in
                      if n > 255 then
                        raise
                          (Reader_error
                             ( "bad decimal escape \\"
                               ^ String.make 1 d1
                               ^ String.make 1 d2
                               ^ String.make 1 d3
                             , st.line
                             , st.col ));
                      Buffer.add_char buf (Char.chr n)
                  | _ ->
                      (* two digits only: forgiving literal digits *)
                      Buffer.add_char buf d1;
                      Buffer.add_char buf d2)
             | _ ->
                 (* one digit only: forgiving literal digit *)
                 Buffer.add_char buf d1)
        | Some c ->
            ignore (advance st);
            Buffer.add_char buf c);
        loop ()
    | Some c ->
        Buffer.add_char buf c;
        ignore (advance st);
        loop ()
  in
  loop ()

(** [read_atom st] reads a non-list, non-quote atom starting at the cursor
    (the caller has ensured the char is a token start). Returns [Var] for
    bare symbols (resolved by eval), [Lit] for numbers, booleans, and
    (already-read) strings. Rejects a lone '.' (dotted-pair notation) with
    a clear error. *)
let read_atom (st : state) : expr =
  (* reads a non-list, non-quote atom. Returns an [expr]: [Var] for bare
     symbols (which eval will look up), [Lit] for numbers, booleans,
     and (already-read) strings. *)
  let buf = Buffer.create 8 in
  let rec loop () =
    match peek st with
    | Some c when is_symbol_cont c ->
        Buffer.add_char buf c;
        ignore (advance st);
        loop ()
    | _ -> ()
  in
  loop ();
  let s = Buffer.contents buf in
  if s = "" then
    raise (Reader_error ("empty token", st.line, st.col))
  else if s = "#t" then Lit (VBool true)
  else if s = "#f" then Lit (VBool false)
  else if s = "." then
    (* A lone '.' is Scheme dotted-pair notation. Improper lists are
```

```
       not supported; reject at read time with a clear error instead
       of silently reading it as a symbol named "." (which would
       corrupt list length and trap macro authors). Float literals
       like .5 still parse below because they are not exactly ".". *)
    raise
      (Reader_error
         ("dotted-pair notation (.) is not supported", st.line, st.col))
  else
    (* Numbers: int first; if token has '.' or exponent, try float.
       Bare symbols otherwise (Var). *)
    let has_dot_or_exp =
      String.contains s '.'
      || String.contains s 'e'
      || String.contains s 'E'
    in
    if has_dot_or_exp then
      (try Lit (VFloat (float_of_string s))
       with _ -> Var s)
    else
      (try Lit (VInt (Int64.of_string s))
       with _ -> Var s)

(** [read_list st] reads a parenthesized list. Assumes the current char is
    '('. Returns the element list (empty for '()'); the caller turns it
    into an [expr]. Raises [Reader_error] on an unterminated list. *)
let rec read_list (st : state) : expr list =
  (* current char is '('; consume it *)
  ignore (advance st);
  let rec loop acc =
    skip_ws_and_comments st;
    match peek st with
    | None ->
        raise (Reader_error ("unterminated list", st.line, st.col))
    | Some ')' ->
        ignore (advance st);
        List.rev acc
    | Some _ ->
        let e = read_expr st in
        loop (e :: acc)
  in
  loop []

(** [read_expr st] reads one expression (the full reader core). Lists
    become [App] nodes with the first element as operator (or [Lit VNil]
    for () -- special forms are dispatched at eval), quote/backtick/comma
    become the corresponding quote sugar, strings become [Lit (VStr ?)],
    and other tokens become atoms via [read_atom]. Raises [Reader_error]
    on unexpected end of input or characters. *)
and read_expr (st : state) : expr =
  skip_ws_and_comments st;
  match peek st with
  | None -> raise (Reader_error ("unexpected end of input", st.line,
     st.col))
  | Some '(' ->
      (* list -> App or special form (handled at eval; reader produces App
         with the first element as operator, or Lit for empty list) *)
      let items = read_list st in
      (match items with
       | [] -> Lit VNil  (* () is nil *)
       | op :: args -> App (op, args))
  | Some '\'' ->
      (* quote sugar: 'x -> (quote x) *)
      ignore (advance st);
      let e = read_expr st in
      App (Var "quote", [ e ])
  | Some '`' ->
      (* quasiquote sugar: `x -> (quasiquote x) *)
      ignore (advance st);
      let e = read_expr st in
      App (Var "quasiquote", [ e ])
  | Some ',' ->
      (* unquote / unquote-splicing: ,x or ,@x *)
      ignore (advance st);
```

```
      (match peek st with
       | Some '@' ->
           ignore (advance st);
           let e = read_expr st in
           App (Var "unquote-splicing", [ e ])
       | _ ->
           let e = read_expr st in
           App (Var "unquote", [ e ]))
  | Some '"' -> Lit (VStr (read_string st))
  | Some c when is_token_start c ->
      (* atom: symbol (-> Var), number, or #t/#f (-> Lit) *)
      read_atom st
  | Some c ->
      raise (Reader_error
               ("unexpected character " ^ String.make 1 c, st.line, st.col))

(** [read_one s] reads exactly one s-expression from [s]. Raises
    [Reader_error] on parse error or if there's trailing junk after the
    expression. *)
let read_one (s : string) : expr =
  let st = make_state s in
  let e = read_expr st in
  skip_ws_and_comments st;
  (match peek st with
   | None -> ()
   | Some _ ->
       raise (Reader_error ("trailing input after expression", st.line,
     st.col)));
  e

(** [read_all s] reads as many top-level expressions as are in [s]. *)
let read_all (s : string) : expr list =
  let st = make_state s in
  let rec loop acc =
    skip_ws_and_comments st;
    match peek st with
    | None -> List.rev acc
    | Some _ ->
        let e = read_expr st in
        loop (e :: acc)
  in
  loop []

(** [is_complete s] returns true if [s] has balanced parens and closed
    strings, i.e. the REPL can evaluate it. Used by the REPL to know
    whether to prompt for more input vs. evaluate now. *)
let is_complete (s : string) : bool =
  let st = make_state s in
  let depth = ref 0 in
  let in_string = ref false in
  let in_comment = ref false in
  try
    while peek st |> Option.is_some do
      let c = Option.get (peek st) in
      if !in_comment then begin
        if c = '\n' then in_comment := false;
        ignore (advance st)
      end else if !in_string then begin
        (match c with
         | '"' -> in_string := false
         | _ -> ());
        ignore (advance st)
      end else begin
        (match c with
         | ';' -> in_comment := true
         | '"' -> in_string := true
         | '(' -> incr depth
         | ')' -> decr depth
         | _ -> ());
        ignore (advance st)
      end
    done;
    not !in_string && !depth = 0
```

```
    (* in_comment at EOF is fine: a line comment is implicitly
       closed by end-of-input. Only an unclosed string or
       unbalanced parens means "need more input". *)
  with _ -> false

(** Pretty-print a value for the REPL. *)
let rec pp_value (v : value) : string =
  match v with
  | VInt n -> Int64.to_string n
  | VFloat f ->
      (* Prefer a readable form; ensure a decimal so re-read is float. *)
      let s = Printf.sprintf "%.12g" f in
      if String.contains s '.' || String.contains s 'e' || String.contains s 'E'
      then s
      else s ^ ".0"
  | VStr s -> "\"" ^ escape_string s ^ "\""
  | VSymbol s ->
      (* Bare token; quote odd names as |?| if needed later. *)
      s
  | VBool b -> if b then "#t" else "#f"
  | VNil -> "()"
  | VPair (a, b) ->
      (* Conventional list printer: a chain of VPairs ending in VNil
         prints as (a b c ...); an improper tail prints as
         (a b . tail). A VList tail is treated as the proper
         continuation (VList is just sugar for nested pairs). *)
      let rec collect acc = function
        | VPair (x, rest) -> collect (x :: acc) rest
        | VNil -> Some (List.rev acc, None)
        | VList items -> Some (List.rev acc @ items, None)
        | tail -> Some (List.rev acc, Some tail)
      in
      (match collect [] (VPair (a, b)) with
       | Some (items, None) ->
           "(" ^ String.concat " " (List.map pp_value items) ^ ")"
       | Some (items, Some tail) ->
           "(" ^ String.concat " " (List.map pp_value items)
           ^ " . " ^ pp_value tail ^ ")"
       | None -> assert false)
  | VLambda _ -> "#<lambda>"
  | VMacro _ -> "#<macro>"
  | VPrim name -> "#<prim:" ^ name ^ ">"
  | VThunk _ -> "#<thunk>"
  | VRef (t, i) -> "#<ref:" ^ t ^ ":" ^ String.sub i 0 (min 8 (String.length i)) ^ ">"
  | VList vs -> "(" ^ String.concat " " (List.map pp_value vs) ^ ")"

(** Flat one-line print (for length checks / compact leaves). *)
let rec pp_expr_flat (e : expr) : string =
  match e with
  | Var s -> s
  | Lit v -> pp_value v
  | App (f, args) ->
      "(" ^ pp_expr_flat f
      ^ (if args = [] then ""
         else " " ^ String.concat " " (List.map pp_expr_flat args))
      ^ ")"
  | Let (n, v, b) ->
      "(let ((" ^ n ^ " " ^ pp_expr_flat v ^ ")) " ^ pp_expr_flat b ^ ")"
  | Letrec (bs, b) ->
      "(letrec ("
      ^ String.concat " "
          (List.map (fun (n, e) -> "(" ^ n ^ " " ^ pp_expr_flat e ^ ")") bs)
      ^ ") " ^ pp_expr_flat b ^ ")"
  | If (t, th, el) ->
      "(if " ^ pp_expr_flat t ^ " " ^ pp_expr_flat th ^ " " ^ pp_expr_flat el
      ^ ")"
  | Lambda (ps, b) ->
      "(lambda (" ^ String.concat " " ps ^ ") " ^ pp_expr_flat b ^ ")"
  | Prim (n, args) ->
      "(prim " ^ n
```

```
      ^ (if args = [] then ""
         else " " ^ String.concat " " (List.map pp_expr_flat args))
      ^ ")"
  | Seq (a, b) -> "(seq " ^ pp_expr_flat a ^ " " ^ pp_expr_flat b ^ ")"
  | Quote e -> "'" ^ pp_expr_flat e
  | Define (n, e) -> "(define " ^ n ^ " " ^ pp_expr_flat e ^ ")"

(** [pp_indent depth] returns a string of [depth * 2] spaces, used as the
    indentation for each nesting level in [pp_expr_at]. *)
let pp_indent (depth : int) : string = String.make (depth * 2) ' '

(** Prefer one line when short and shallow; otherwise break. *)
let rec expr_is_compact (e : expr) : bool =
  let flat = pp_expr_flat e in
  if String.length flat <= 56 then
    match e with
    | Var _ | Lit _ -> true
    | Quote e' -> expr_is_compact e'
    | App (f, args) ->
        List.length args <= 5
        && expr_is_compact f
        && List.for_all expr_is_compact args
    | Lambda (ps, b) -> List.length ps <= 4 && expr_is_compact b
    | Prim (_, args) -> List.for_all expr_is_compact args
    | If (t, th, el) ->
        expr_is_compact t && expr_is_compact th && expr_is_compact el
    | Seq (a, b) -> expr_is_compact a && expr_is_compact b
    | Let (_, v, b) -> expr_is_compact v && expr_is_compact b
    | Letrec (bs, b) ->
        List.length bs <= 2
        && List.for_all (fun (_, e) -> expr_is_compact e) bs
        && expr_is_compact b
    | Define (_, e) -> expr_is_compact e
  else false

(** Pretty-print an expr with indentation for the Functions browser /
    editor. *)
let rec pp_expr_at (depth : int) (e : expr) : string =
  if depth > 0 && expr_is_compact e then pp_expr_flat e
  else
    let ind1 = pp_indent (depth + 1) in
    match e with
    | Var s -> s
    | Lit v -> pp_value v
    | Quote e' ->
        if expr_is_compact e' then "'" ^ pp_expr_flat e'
        else "(quote\n" ^ ind1 ^ pp_expr_at (depth + 1) e' ^ ")"
    | App (f, []) -> "(" ^ pp_expr_at depth f ^ ")"
    | App (f, args) ->
        if expr_is_compact e then pp_expr_flat e
        else
          let head = pp_expr_at (depth + 1) f in
          let body =
            List.map
              (fun a -> ind1 ^ pp_expr_at (depth + 1) a)
              args
          in
          "(" ^ head ^ "\n" ^ String.concat "\n" body ^ ")"
    | Lambda (ps, b) ->
        if expr_is_compact e then pp_expr_flat e
        else
          "(lambda (" ^ String.concat " " ps ^ ")\n"
          ^ ind1 ^ pp_expr_at (depth + 1) b ^ ")"
    | If (t, th, el) ->
        if expr_is_compact e then pp_expr_flat e
        else
          "(if\n"
          ^ ind1 ^ pp_expr_at (depth + 1) t ^ "\n"
          ^ ind1 ^ pp_expr_at (depth + 1) th ^ "\n"
          ^ ind1 ^ pp_expr_at (depth + 1) el ^ ")"
    | Let (n, v, b) ->
        if expr_is_compact e then pp_expr_flat e
        else
```

```
          "(let ((" ^ n ^ "\n"
          ^ ind1 ^ "  " ^ pp_expr_at (depth + 2) v ^ "))\n"
          ^ ind1 ^ pp_expr_at (depth + 1) b ^ ")"
    | Letrec (bs, b) ->
        if expr_is_compact e then pp_expr_flat e
        else
          let binds =
            List.map
              (fun (n, e') ->
                ind1 ^ "(" ^ n ^ "\n"
                ^ pp_indent (depth + 2) ^ pp_expr_at (depth + 2) e' ^ ")")
              bs
          in
          "(letrec (\n"
          ^ String.concat "\n" binds ^ ")\n"
          ^ ind1 ^ pp_expr_at (depth + 1) b ^ ")"
    | Seq (a, b) ->
        if expr_is_compact e then pp_expr_flat e
        else
          "(seq\n"
          ^ ind1 ^ pp_expr_at (depth + 1) a ^ "\n"
          ^ ind1 ^ pp_expr_at (depth + 1) b ^ ")"
    | Prim (n, args) ->
        if expr_is_compact e then pp_expr_flat e
        else if args = [] then "(prim " ^ n ^ ")"
        else
          "(prim " ^ n ^ "\n"
          ^ String.concat "\n"
              (List.map
                 (fun a -> ind1 ^ pp_expr_at (depth + 1) a)
                 args)
          ^ ")"
    | Define (n, body) ->
        if expr_is_compact e then pp_expr_flat e
        else
          "(define " ^ n ^ "\n"
          ^ ind1 ^ pp_expr_at (depth + 1) body ^ ")"

(** [pp_expr e] pretty-prints [e] with indentation, starting at depth 0. *)
let pp_expr (e : expr) : string = pp_expr_at 0 e
```

241 lisp/lib/recursion.ml

```
(** Static recursion / tail-position analysis over Nopales ASTs.

    Pure: no I/O. Used by binding-page HTML and complexity heuristics.
    Tail context mirrors the eval trampoline (lisp-interpreter.eval TCO). *)
open Ast

type tail_ctx = Tail | Non_tail

(** A single self-or-other call site within a function body. [path] is
    the AST child-index path from the root body to the call; [tail]
    records whether the call is in tail position; [callee] is the
    callee name (equal to the enclosing fn for self-calls). *)
type site = {
  path : int list;
  tail : bool;
  callee : string;
}

(** Summary of a function's recursion structure. *)
type recursion_info = {
  is_recursive : bool;
  tail_recursive : bool;
  (** True when there is at least one self-call and every self-call is
     tail. *)
  self_sites : site list;
  (** Direct App-Var callees excluding self (for mutual annotation). *)
  callees : string list;
}

(** Strip a single [define]/[lambda] wrapper (AST or surface App form)
    to expose the function body for analysis. Non-wrapper exprs pass
    through unchanged. *)
let rec unwrap_fn_body (e : expr) : expr =
  match e with
  | Define (_, body) -> unwrap_fn_body body
  | Lambda (_, body) -> body
  (* Surface reader keeps define/lambda as App until eval. *)
  | App (Var "define", [ _name; body ]) -> unwrap_fn_body body
  | App (Var "define", [ _name; body; _doc ]) -> unwrap_fn_body body
  | App (Var "lambda", [ _params; body ]) -> body
  | App (Var "lambda", _params :: body :: _) -> body
  | other -> other

(** Walk [e] and return every direct self-call site of [fn_name], tagged
    with its AST [path] and whether it sits in tail position ([ctx] is
    [Tail] only at tail-eligible spots, mirroring the eval trampoline
    TCO; argument/condition positions are [Non_tail]). *)
let rec collect_sites ~(fn_name : string) ~(ctx : tail_ctx) ~(path : int
    list)
    (e : expr) : site list =
  match e with
  | Var _ | Lit _ | Quote _ -> []
  (* Surface special forms stay App(Var ?) until eval -- treat like
     If/Seq/Let. *)
  | App (Var "if", test :: then_e :: else_e :: _) ->
      collect_sites ~fn_name ~ctx:Non_tail ~path:(0 :: path) test
      @ collect_sites ~fn_name ~ctx ~path:(1 :: path) then_e
      @ collect_sites ~fn_name ~ctx ~path:(2 :: path) else_e
  | App (Var "if", test :: then_e :: []) ->
      collect_sites ~fn_name ~ctx:Non_tail ~path:(0 :: path) test
      @ collect_sites ~fn_name ~ctx ~path:(1 :: path) then_e
  | App (Var sf, args)
    when sf = "seq" || sf = "begin" ->
      (match List.rev args with
       | [] -> []
       | last :: rev_rest ->
           let early =
             List.concat
               (List.mapi
                  (fun i a ->
```

```
                    collect_sites ~fn_name ~ctx:Non_tail ~path:(i :: path) a)
                (List.rev rev_rest))
          in
          let last_i = List.length args - 1 in
          early
          @ collect_sites ~fn_name ~ctx ~path:(last_i :: path) last)
  | App (Var "and", args) | App (Var "or", args) ->
      (match List.rev args with
       | [] -> []
       | last :: rev_rest ->
          let early =
            List.concat
              (List.mapi
                 (fun i a ->
                   collect_sites ~fn_name ~ctx:Non_tail ~path:(i :: path) a)
                 (List.rev rev_rest))
          in
          let last_i = List.length args - 1 in
          early @ collect_sites ~fn_name ~ctx ~path:(last_i :: path) last)
  | App (Var "lambda", args) ->
      (* New frame -- body is not tail of the outer function. *)
      (match args with
       | _params :: body :: _ ->
          collect_sites ~fn_name ~ctx:Non_tail ~path:(1 :: path) body
       | _ -> [])
  | App (Var "define", args) ->
      (match args with
       | _name :: body :: _ ->
          collect_sites ~fn_name ~ctx:Non_tail ~path:(1 :: path) body
       | _ -> [])
  | App (Var callee, args) ->
      let here =
        if callee = fn_name then
          [ { path = List.rev path; tail = ctx = Tail; callee } ]
        else []
      in
      let arg_sites =
        List.concat
          (List.mapi
             (fun i a ->
               collect_sites ~fn_name ~ctx:Non_tail ~path:(i :: path) a)
             args)
      in
      here @ arg_sites
  | App (f, args) ->
      let fs = collect_sites ~fn_name ~ctx:Non_tail ~path:(0 :: path) f in
      let arg_sites =
        List.concat
          (List.mapi
             (fun i a ->
               collect_sites ~fn_name ~ctx:Non_tail ~path:((i + 1) :: path) a)
             args)
      in
      fs @ arg_sites
  | If (t, th, el) ->
      collect_sites ~fn_name ~ctx:Non_tail ~path:(0 :: path) t
      @ collect_sites ~fn_name ~ctx ~path:(1 :: path) th
      @ collect_sites ~fn_name ~ctx ~path:(2 :: path) el
  | Seq (a, b) ->
      collect_sites ~fn_name ~ctx:Non_tail ~path:(0 :: path) a
      @ collect_sites ~fn_name ~ctx ~path:(1 :: path) b
  | Let (_, v, body) ->
      collect_sites ~fn_name ~ctx:Non_tail ~path:(0 :: path) v
      @ collect_sites ~fn_name ~ctx ~path:(1 :: path) body
  | Letrec (binds, body) ->
      let bs =
        List.concat
          (List.mapi
             (fun i (_, e') ->
               collect_sites ~fn_name ~ctx:Non_tail ~path:(i :: path) e')
```

```
            binds)
      in
      bs @ collect_sites ~fn_name ~ctx ~path:(List.length binds :: path)
     body
  | Lambda (_, body) ->
      (* New frame -- body is not tail of the outer function. *)
      collect_sites ~fn_name ~ctx:Non_tail ~path:(0 :: path) body
  | Prim (_, args) ->
      List.concat
        (List.mapi
           (fun i a ->
             collect_sites ~fn_name ~ctx:Non_tail ~path:(i :: path) a)
           args)
  | Define (_, body) ->
      collect_sites ~fn_name ~ctx:Non_tail ~path:(0 :: path) body

(** Collect the names of all variables called as functions within [e]
    (deduped, in first-encounter order). Used to detect mutual calls to
    sibling functions. *)
let collect_callees (e : expr) : string list =
  let rec go acc = function
    | Var _ | Lit _ | Quote _ -> acc
    | App (Var f, args) ->
        List.fold_left go (f :: acc) args
    | App (f, args) -> List.fold_left go (go acc f) args
    | Let (_, a, b) -> go (go acc a) b
    | Letrec (bs, body) ->
        let acc = List.fold_left (fun a (_, e) -> go a e) acc bs in
        go acc body
    | If (a, b, c) -> go (go (go acc a) b) c
    | Lambda (_, body) -> go acc body
    | Prim (_, args) -> List.fold_left go acc args
    | Seq (a, b) -> go (go acc a) b
    | Define (_, body) -> go acc body
  in
  let seen = Hashtbl.create 16 in
  go [] e
  |> List.fold_left
       (fun acc n ->
         if Hashtbl.mem seen n then acc
         else begin
           Hashtbl.add seen n ();
           n :: acc
         end)
       []
  |> List.rev

(** Analyze the recursion structure of function [fn_name]'s body [e]:
    whether it is recursive, whether all self-calls are tail (and thus
    eligible for the evaluator's TCO), the self-call [site]s, and the
    names of other functions it calls directly (for mutual-recursion
    annotation). *)
let analyze_self ~(fn_name : string) (e : expr) : recursion_info =
  let body = unwrap_fn_body e in
  let self_sites = collect_sites ~fn_name ~ctx:Tail ~path:[] body in
  let is_recursive = self_sites <> [] in
  let tail_recursive =
    is_recursive && List.for_all (fun s -> s.tail) self_sites
  in
  let callees =
    List.filter (fun c -> c <> fn_name) (collect_callees body)
  in
  { is_recursive; tail_recursive; self_sites; callees }

(** Path predicates: [is_tail path] / [is_self path] for HTML decoration. *)
let site_index (info : recursion_info) : (int list, site) Hashtbl.t =
  let h = Hashtbl.create 8 in
  List.iter (fun s -> Hashtbl.replace h s.path s) info.self_sites;
  h

(** [is_self_path info path] is [true] if [path] is a self-call site in
    [info]. *)
let is_self_path (info : recursion_info) (path : int list) : bool =
```

```
  List.exists (fun s -> s.path = path) info.self_sites

(** [is_tail_self_path info path] is [true] if [path] is a self-call site
    in tail position. *)
let is_tail_self_path (info : recursion_info) (path : int list) : bool =
  List.exists (fun s -> s.path = path && s.tail) info.self_sites

(** [is_nontail_self_path info path] is [true] if [path] is a self-call
    site in non-tail position. *)
let is_nontail_self_path (info : recursion_info) (path : int list) : bool =
  List.exists (fun s -> s.path = path && not s.tail) info.self_sites
```

242 lisp/lib/route_gate.ml

```
(** Public-route handler effect gate (pure).

    Decides whether a handler may be published as a public route by
    inferring its effects from the in-memory env (synchronous, fresh)
    and checking them against the public-route allowlist. Fail-closed:
    an unknown or non-function handler is DENIED, never silently allowed.

    Pure: no Lwt, no DB. The image's [Session.check_public_handler]
    (session.ml) delegates here so the gate is unit-testable without
    Postgres.

    Addresses BORGE-1785212463 / BORGE-1785212603: the previous gate read
    function_defs from the store (stale after the define persist race)
    and DEFAULTED TO ALLOW when the handler was absent -- so a typo or an
    eval-order race silently published a route with no effect check.

    BORGE-1785212882: bodies are macroexpanded before [Effect.infer] so a
    macro cannot hide WriteDb/Publish/etc. from the allowlist. Expansion
    uses [Eval.macroexpand_expr]; [Effect.infer] stays pure. *)
open Ast

(** Macroexpand [body] in [env]. Propagates [Interp_error]; other
    failures leave the body unexpanded (still fail-closed if effects
    remain hidden -- prefer a correct expand, don't crash the gate). *)
let expand_body (env : Env.env) (body : expr) : expr =
  try Eval.macroexpand_expr env body
  with Interp_error _ as ex -> raise ex
     | _ -> body

(** Collect (name, body) for every VLambda visible in [env] (walks the
    parent chain). Bodies are macroexpanded so sibling known_fn resolution
    also sees through macros. *)
let env_lambda_bindings (env : Env.env) : (string * expr) list =
  Env.fold env
    (fun name v acc ->
      match v with
      | VLambda (_, body, _) -> (name, expand_body env body) :: acc
      | _ -> acc)
    []

(** Check whether [handler] may be published as a public route.

    Returns [Ok ()] if the handler is a defined function (or prim) whose
    inferred effects fit the public-route allowlist; [Error msg] otherwise.
    Unknown / non-function handlers return [Error] (fail-closed). *)
let check_public_handler ~(env : Env.env) (handler : string)
    : (unit, string) result =
  let known_fn = Effect.known_fn_of_bindings (env_lambda_bindings env) in
  let allow = Effect.public_route_allow () in
  let check fx =
    if Effect.policy_allows ~allow fx then Ok ()
    else
      Error
        (Printf.sprintf
           "public route denied: handler %S effects %s (allow %s; set
    PP_PUBLIC_ALLOW_WRITE=1 for write-db)"
           handler
           (Effect.EffSet.pp fx)
           (Effect.public_route_allow_pp ()))
  in
  match Env.lookup env handler with
  | Some (VLambda (_, body, _)) ->
      (try check (Effect.infer ~known_fn (expand_body env body))
       with Interp_error e ->
         Error
           (Printf.sprintf
              "public route denied: handler %S macroexpand failed: %s"
              handler
              (match e with
               | User_error (m, _) -> m
               | Arity_mismatch (n, exp, got) ->
```

```
                  Printf.sprintf "%s arity %d vs %d" n exp got
              | _ -> "interp error")))
  | Some (VPrim name) -> check (Effect.effects_for_prim name)
  | Some _ | None ->
      Error
        (Printf.sprintf
           "public route denied: handler %S is not a defined function"
           handler)
```

243 lisp/lib/special_forms.ml

```
(** Language special forms -- syntax handled by the evaluator, not
    bound as env lambdas or registered prims.

    Browsable from the image Functions UI as package [(special-forms)].
    Keep this catalog in sync with [Eval.is_special_form] /
     eval_special_form. *)

type entry = {
  name : string;
  summary : string;
  arity : string;
  examples : string;
}

(** Static catalog of special-form entries, one per special form {{name}},
    documenting its summary, arity, and an example. Serves as reference
    text for the Functions UI; keep in sync with the evaluator. *)
let catalog : entry list =
  [
    {
      name = "seq";
      summary =
        "Evaluate body expressions left-to-right for effects; return \
         the last body's value. Empty (seq) returns nil.";
      arity = "(seq e1 e2 ? en)  -- 0 or more bodies";
      examples = "(seq (define x 1) (define y 2) y)  ?  2";
    };
    {
      name = "if";
      summary =
        "Conditional: evaluate test, then then-branch or else-branch \
         (not both). Non-nil non-false is truthy; only #f and () are
    falsey.";
      arity = "(if test then else)";
      examples = "(if #t 1 2)  ?  1";
    };
    {
      name = "lambda";
      summary =
        "Create a closure. Params are symbols; body is a single expr \
         (use seq for multi-step bodies).";
      arity = "(lambda (p1 p2 ?) body)";
      examples = "((lambda (x) (+ x 1)) 41)  ?  42";
    };
    {
      name = "let";
      summary =
        "Bind names in a child env, then evaluate body. Bindings are \
         evaluated in the outer env (not sequential let*).";
      arity = "(let ((name val) ?) body)";
      examples = "(let ((y 5)) y)  ?  5";
    };
    {
      name = "letrec";
      summary =
        "Mutual recursive bindings (AST form). App-form letrec from \
         surface text is not fully supported yet.";
      arity = "(letrec ((name val) ?) body)  -- AST / partial surface";
      examples = "prefer lambda + define for most code";
    };
    {
      name = "define";
      summary =
        "Bind name to value in the current env (and image persists \
         top-level defines to function_defs). Optional docstring \
         (string literal) is store metadata via function_defs.doc; \
         read back with (function-doc name). Name is not evaluated.";
      arity =
        "(define name expr [\"doc\"]) | (define (name p?) [\"doc\"] body?)";
      examples =
```

```
        "(define double (lambda (x) \"2x\" (+ x x)))  ; or trailing \"doc\"";
    };
    {
      name = "quote";
      summary =
        "Return the argument as data without evaluating it. Reader \
         sugar 'x expands to (quote x).";
      arity = "(quote expr)  or  'expr";
      examples = "'(a b c)  ?  list of symbols/strings as values";
    };
    {
      name = "prim";
      summary =
        "Call a registered primitive by name with one evaluated arg \
         (legacy/low-level path; prefer normal (name ?) calls).";
      arity = "(prim name arg)";
      examples = "(prim add-one 41) when add-one is registered";
    };
    {
      name = "with-transaction";
      summary =
        "Run a zero-arg lambda inside a dynamic DB transaction \
         (image-layer __with_transaction__ prim). Nested txs error.";
      arity = "(with-transaction (lambda () body?))";
      examples = "(with-transaction (lambda () (create 'note ?)))";
    };
    {
      name = "quasiquote";
      summary =
        "Template quote with unquote / unquote-splicing. Reader sugar `x.";
      arity = "(quasiquote template)  or  `template";
      examples = "`(a ,@'(b c) d)";
    };
    {
      name = "defmacro";
      summary =
        "Define a syntactic macro (unevaluated args; body returns code as data).";
      arity = "(defmacro name (params) body)";
      examples = "(defmacro when (t b) `(if ,t ,b ()))";
    };
    {
      name = "gensym";
      summary = "Fresh symbol name string for unhygienic macros.";
      arity = "(gensym)  or  (gensym prefix)";
      examples = "(gensym) ? g$1";
    };
    {
      name = "cond";
      summary =
        "Multi-branch conditional. First truthy test's body wins. \
         Use else or #t as final catch-all. Empty cond ? nil.";
      arity = "(cond (test body) ?)";
      examples = "(cond ((null? xs) 'empty) (#t 'full))";
    };
    {
      name = "and";
      summary =
        "Short-circuit AND. (and) ? #t. Returns last value if all truthy, \
         else first falsey (#f or nil).";
      arity = "(and e1 e2 ?)";
      examples = "(and #t 1 2) ? 2";
    };
    {
      name = "or";
      summary =
        "Short-circuit OR. (or) ? #f. Returns first truthy value, else last.";
      arity = "(or e1 e2 ?)";
      examples = "(or #f 0 'x) ? 0 (0 is truthy)";
    };
  ]
```

```
(** The [name] of every entry in [catalog], in catalog order. *)
let names : string list = List.map (fun e -> e.name) catalog

(** Look up a special-form entry by [name]; [None] if not in [catalog]. *)
let find (name : string) : entry option =
  List.find_opt (fun e -> e.name = name) catalog

(** (name, one-line summary) for package index tables. *)
let summaries : (string * string) list =
  List.map (fun e -> (e.name, e.summary)) catalog
```

244 lisp/test/ast_diff_test.ml

```
(** Tests for the pure AST diff walker (lisp-interpreter.ast-diff).

    PURE tests -- no Postgres, no HTTP. Per spec:
      cd lisp/ && dune build && dune runtest
    needs no services. Semantics authority: branching.diff.

    Covers same/modified/replaced over all 11 real expr constructors
    (Var Lit App Let Letrec If Lambda Prim Seq Quote Define), the
    list-LCS properties (App args + Letrec bindings so an insertion
    mid-sequence does not cascade), and the top-level ast_hash
    short-circuit. *)
open Pricklypear_lisp

let rec sprintf_delta (d : Ast_diff.delta) : string =
  match d with
  | Same -> "Same"
  | Replaced { from = f; to_ = t } ->
      Printf.sprintf "Replaced(%s -> %s)" (Reader.pp_expr f)
     (Reader.pp_expr t)
  | Node { ctor; children } ->
      Printf.sprintf "Node(%s,[%s])" ctor
        (String.concat "; "
           (List.map (fun (l, d) -> l ^ "=" ^ sprintf_delta d) children))

(* compare two exprs for equality irrespective of the diff *)
let rec expr_eq (a : Ast.expr) (b : Ast.expr) : bool =
  match a, b with
  | Var x, Var y -> x = y
  | Lit x, Lit y -> x = y
  | App (f1, a1), App (f2, a2) ->
      expr_eq f1 f2 && List.length a1 = List.length a2 && List.for_all2
     expr_eq a1 a2
  | Let (n1, v1, b1), Let (n2, v2, b2) -> n1 = n2 && expr_eq v1 v2 &&
     expr_eq b1 b2
  | Letrec (bs1, b1), Letrec (bs2, b2) ->
      List.length bs1 = List.length bs2
      && List.for_all2 (fun (n1, e1) (n2, e2) -> n1 = n2 && expr_eq e1 e2)
     bs1 bs2
      && expr_eq b1 b2
  | If (a1, b1, c1), If (a2, b2, c2) -> expr_eq a1 a2 && expr_eq b1 b2 &&
     expr_eq c1 c2
  | Lambda (p1, b1), Lambda (p2, b2) -> p1 = p2 && expr_eq b1 b2
  | Prim (n1, a1), Prim (n2, a2) ->
      n1 = n2 && List.length a1 = List.length a2 && List.for_all2 expr_eq
     a1 a2
  | Seq (a1, b1), Seq (a2, b2) -> expr_eq a1 a2 && expr_eq b1 b2
  | Quote e1, Quote e2 -> expr_eq e1 e2
  | Define (n1, e1), Define (n2, e2) -> n1 = n2 && expr_eq e1 e2
  | _ -> false

let rec delta_eq (a : Ast_diff.delta) (b : Ast_diff.delta) : bool =
  match a, b with
  | Same, Same -> true
  | Replaced { from = f1; to_ = t1 }, Replaced { from = f2; to_ = t2 } ->
      expr_eq f1 f2 && expr_eq t1 t2
  | Node { ctor = c1; children = ch1 }, Node { ctor = c2; children = ch2 }
     ->
      c1 = c2
      && List.length ch1 = List.length ch2
      && List.for_all2
           (fun (l1, d1) (l2, d2) -> l1 = l2 && delta_eq d1 d2)
           ch1 ch2
  | _ -> false

let eq_delta expected actual =
  Alcotest.(check bool)
    (Printf.sprintf "delta %s" (sprintf_delta expected))
    true (delta_eq expected actual)

let delta_tests = [
  (* --- Var --- *)
```

```
  ("diff: same Var", `Quick, fun () ->
     eq_delta Same (Ast_diff.diff (Ast.var "x") (Ast.var "x")));
  ("diff: replaced Var", `Quick, fun () ->
     let from = Ast.var "x" and to_ = Ast.var "y" in
     eq_delta (Ast_diff.Replaced { from; to_ }) (Ast_diff.diff from to_));
  (* --- Lit --- *)
  ("diff: same Lit", `Quick, fun () ->
     eq_delta Same (Ast_diff.diff (Ast.int 1) (Ast.int 1)));
  ("diff: replaced Lit", `Quick, fun () ->
     let from = Ast.int 1 and to_ = Ast.int 2 in
     eq_delta (Ast_diff.Replaced { from; to_ }) (Ast_diff.diff from to_));
  (* --- If --- *)
  ("diff: If unchanged", `Quick, fun () ->
     let e = Ast.If (Ast.var "x", Ast.int 1, Ast.int 2) in
     eq_delta Same (Ast_diff.diff e e));
  ("diff: If modified branch", `Quick, fun () ->
     let from = Ast.If (Ast.var "x", Ast.int 1, Ast.int 2)
     and to_ = Ast.If (Ast.var "x", Ast.int 9, Ast.int 2) in
     let d = Ast_diff.diff from to_ in
     (match d with
      | Ast_diff.Node { ctor = "if"; children = [("test", Ast_diff.Same);
     ("then", Ast_diff.Replaced _); ("else", Ast_diff.Same)] } -> ()
      | _ -> Alcotest.failf "unexpected if delta: %s" (sprintf_delta d)));
  ("diff: If replaced by Lambda (ctor swap)", `Quick, fun () ->
     let from = Ast.If (Ast.var "x", Ast.int 1, Ast.int 2)
     and to_ = Ast.Lambda (["x"], Ast.var "x") in
     let d = Ast_diff.diff from to_ in
     eq_delta (Ast_diff.Replaced { from; to_ }) d);
  (* --- Seq --- *)
  ("diff: Seq modified second", `Quick, fun () ->
     let from = Ast.Seq (Ast.int 1, Ast.int 2)
     and to_ = Ast.Seq (Ast.int 1, Ast.int 3) in
     let d = Ast_diff.diff from to_ in
     (match d with
      | Ast_diff.Node { ctor = "seq"; children = [("a", Ast_diff.Same);
     ("b", Ast_diff.Replaced _)] } -> ()
      | _ -> Alcotest.failf "unexpected seq delta: %s" (sprintf_delta d)));
  (* --- Lambda --- *)
  ("diff: Lambda body modified, params Same", `Quick, fun () ->
     let from = Ast.Lambda (["x"], Ast.var "x")
     and to_ = Ast.Lambda (["x"], Ast.int 1) in
     let d = Ast_diff.diff from to_ in
     (match d with
      | Ast_diff.Node { ctor = "lambda";
                        children = [("params", Ast_diff.Node { ctor =
     "params"; children = [("param", Ast_diff.Same)] });
                                    ("body", Ast_diff.Replaced _)] } -> ()
      | _ -> Alcotest.failf "unexpected lambda delta: %s" (sprintf_delta
     d)));
  ("diff: Lambda param rename is positional Replaced (not rename-detect)",
     `Quick, fun () ->
     let from = Ast.Lambda (["x"], Ast.var "x")
     and to_ = Ast.Lambda (["y"], Ast.var "y") in
     let d = Ast_diff.diff from to_ in
     (match d with
      | Ast_diff.Node { ctor = "lambda"; children = _ } ->
          (* must NOT be Same; a genuine rename is visible, not hidden *)
          ()
      | _ -> Alcotest.failf "unexpected lambda delta: %s" (sprintf_delta
     d)));
  ("diff: Lambda param order swapped is a positional delta", `Quick, fun ()
     ->
     let from = Ast.Lambda (["a"; "b"], Ast.var "a")
     and to_ = Ast.Lambda (["b"; "a"], Ast.var "a") in
     let d = Ast_diff.diff from to_ in
     (* order matters: (a b) vs (b a) must NOT collapse to Same *)
     if d = Ast_diff.Same then
       Alcotest.fail "param order swap wrongly reported Same");
  (* --- Let --- *)
  ("diff: Let value modified", `Quick, fun () ->
     let from = Ast.Let ("x", Ast.int 1, Ast.var "x")
     and to_ = Ast.Let ("x", Ast.int 2, Ast.var "x") in
     let d = Ast_diff.diff from to_ in
```

```
    (match d with
     | Ast_diff.Node { ctor = "let"; children = [("name", Ast_diff.Same);
    ("val", Ast_diff.Replaced _); ("body", Ast_diff.Same)] } -> ()
     | _ -> Alcotest.failf "unexpected let delta: %s" (sprintf_delta d)));
  ("diff: Let name change is Replaced at name", `Quick, fun () ->
    let from = Ast.Let ("x", Ast.int 1, Ast.var "x")
    and to_ = Ast.Let ("y", Ast.int 1, Ast.var "y") in
    let d = Ast_diff.diff from to_ in
    (match d with
     | Ast_diff.Node { ctor = "let"; children = [("name",
    Ast_diff.Replaced _); _; _] } -> ()
     | _ -> Alcotest.failf "unexpected let delta: %s" (sprintf_delta d)));
  (* --- Letrec --- *)
  ("diff: Letrec body modified", `Quick, fun () ->
    let bs = [("f", Ast.Lambda (["n"], Ast.var "n"))] in
    let from = Ast.Letrec (bs, Ast.App (Ast.var "f", [Ast.int 1]))
    and to_ = Ast.Letrec (bs, Ast.App (Ast.var "f", [Ast.int 2])) in
    let d = Ast_diff.diff from to_ in
    (match d with
     | Ast_diff.Node { ctor = "letrec"; children = _ } -> ()
     | _ -> Alcotest.failf "unexpected letrec delta: %s" (sprintf_delta
    d)));
  ("diff: Letrec binding insertion does not cascade", `Quick, fun () ->
    let from =
      Ast.Letrec
        ([ ("g", Ast.Lambda ([], Ast.int 1));
           ("f", Ast.Lambda (["x"], Ast.var "x")) ],
         Ast.App (Ast.var "f", [Ast.int 1]))
    and to_ =
      Ast.Letrec
        ([ ("g", Ast.Lambda ([], Ast.int 1));
           ("h", Ast.Lambda ([], Ast.int 999));   (* inserted mid-sequence
    *)
           ("f", Ast.Lambda (["x"], Ast.var "x")) ],
         Ast.App (Ast.var "f", [Ast.int 1]))
    in
    let d = Ast_diff.diff from to_ in
    (match d with
     | Ast_diff.Node { ctor = "letrec"; children } ->
         let bindings_chilren, body_is_same =
           match List.rev children with
           | ("body", Ast_diff.Same) :: rest -> (List.rev rest, true)
           | ("body", _) :: rest -> (List.rev rest, false)
           | _ -> (children, false)
         in
         (* g unchanged, h added, f unchanged (not all following changed)
    *)
         let labels = List.map fst bindings_chilren in
         Alcotest.(check bool) "g still present" true (List.mem "g"
    labels);
         Alcotest.(check bool) "h added to" true (List.mem "h" labels);
         Alcotest.(check bool) "f still present" true (List.mem "f"
    labels);
         let h_delta = List.assoc "h" bindings_chilren in
         let is_replaced =
           match h_delta with
           | Ast_diff.Replaced { from = from_side; _ } ->
               (* from side is the sentinel (absent) *)
               expr_eq from_side Ast.nil
           | _ -> false
         in
         Alcotest.(check bool) "h is an added Replaced" true is_replaced;
         Alcotest.(check bool) "letrec body Same" true body_is_same
     | _ -> Alcotest.failf "unexpected letrec delta: %s" (sprintf_delta
    d)));
  (* --- App --- *)
  ("diff: App head replaced", `Quick, fun () ->
    let from = Ast.App (Ast.var "f", [Ast.int 1])
    and to_ = Ast.App (Ast.var "g", [Ast.int 1]) in
    let d = Ast_diff.diff from to_ in
    (match d with
     | Ast_diff.Node { ctor = "app"; children = ("fn", Ast_diff.Replaced
    _) :: _ } -> ()
```

```
    | _ -> Alcotest.failf "unexpected app delta: %s" (sprintf_delta d));
 ("diff: App args insertion mid-sequence does NOT mark every following arg changed", `Quick, fun () ->
    let from = Ast.App (Ast.var "f", [Ast.int 1; Ast.int 2; Ast.int 3])
    and to_ = Ast.App (Ast.var "f", [Ast.int 1; Ast.int 42; Ast.int 2; Ast.int 3]) in
    let d = Ast_diff.diff from to_ in
    (match d with
     | Ast_diff.Node { ctor = "app"; children = ("fn", Ast_diff.Same) :: args } ->
         (* args 1 and 2 and 3 should be Same (matched via LCS), 42 added *)
         let same_count = List.fold_left (fun n (_, d) ->
             match d with Ast_diff.Same -> n + 1 | _ -> n) 0 args in
         let replaced_count = List.length args - same_count in
         Alcotest.(check int) "matched args remain Same" 3 same_count;
         Alcotest.(check int) "one added arg" 1 replaced_count
     | _ -> Alcotest.failf "unexpected app delta: %s" (sprintf_delta d));
 ("diff: App trailing add", `Quick, fun () ->
    let from = Ast.App (Ast.var "f", [Ast.int 1])
    and to_ = Ast.App (Ast.var "f", [Ast.int 1; Ast.int 2]) in
    let d = Ast_diff.diff from to_ in
    (match d with
     | Ast_diff.Node { ctor = "app"; children = ("fn", Ast_diff.Same) :: args } ->
         let same_count = List.fold_left (fun n (_, d) ->
             match d with Ast_diff.Same -> n + 1 | _ -> n) 0 args in
         Alcotest.(check int) "arg 1 Same, arg 2 added" 1 same_count;
         Alcotest.(check int) "two args total" 2 (List.length args)
     | _ -> Alcotest.failf "unexpected app delta: %s" (sprintf_delta d));
 (* --- Prim --- *)
 ("diff: Prim same", `Quick, fun () ->
    let e = Ast.Prim ("+", [Ast.int 1; Ast.int 2]) in
    eq_delta Same (Ast_diff.diff e e));
 ("diff: Prim arg modified", `Quick, fun () ->
    let from = Ast.Prim ("+", [Ast.int 1; Ast.int 2])
    and to_ = Ast.Prim ("+", [Ast.int 1; Ast.int 3]) in
    let d = Ast_diff.diff from to_ in
    (match d with
     | Ast_diff.Node { ctor = "prim"; children = _ } -> ()
     | _ -> Alcotest.failf "unexpected prim delta: %s" (sprintf_delta d)));
 ("diff: Prim name changed is Replaced", `Quick, fun () ->
    let from = Ast.Prim ("+", [Ast.int 1])
    and to_ = Ast.Prim ("-", [Ast.int 1]) in
    eq_delta (Ast_diff.Replaced { from; to_ }) (Ast_diff.diff from to_));
 (* --- Quote --- *)
 ("diff: Quote body modified", `Quick, fun () ->
    let from = Ast.Quote (Ast.var "x")
    and to_ = Ast.Quote (Ast.var "y") in
    let d = Ast_diff.diff from to_ in
    (match d with
     | Ast_diff.Node { ctor = "quote"; children = [("body", Ast_diff.Replaced _)] } -> ()
     | _ -> Alcotest.failf "unexpected quote delta: %s" (sprintf_delta d)));
 (* --- Define --- *)
 ("diff: Define unchanged", `Quick, fun () ->
    let e = Ast.Define ("foo", Ast.Lambda (["x"], Ast.var "x")) in
    eq_delta Same (Ast_diff.diff e e));
 ("diff: Define val modified", `Quick, fun () ->
    let from = Ast.Define ("foo", Ast.Lambda (["x"], Ast.var "x"))
    and to_ = Ast.Define ("foo", Ast.Lambda (["x"], Ast.int 7)) in
    let d = Ast_diff.diff from to_ in
    (match d with
     | Ast_diff.Node { ctor = "define"; children = [("name", Ast_diff.Same); ("val", Ast_diff.Node _)] } -> ()
     | _ -> Alcotest.failf "unexpected define delta: %s" (sprintf_delta d)));
 (* --- top-level hash short-circuit --- *)
 ("diff: top-level hash equality short-circuits to Same", `Quick, fun () ->
    let e = Ast.Prim ("+", [Ast.int 1; Ast.int 2]) in
    Alcotest.(check bool) "hash equal" true (Ast_diff.ast_hash e = Ast_diff.ast_hash e);
```

```
    eq_delta Same (Ast_diff.diff e e));
  ("diff: separate-but-equal exprs hash to Same", `Quick, fun () ->
    (* two distinct AST values with identical structure *)
    let a =
      Ast.Letrec
        ([ ("f", Ast.Lambda (["x"], Ast.Prim ("+", [Ast.var "x"; Ast.int
    1]))) ],
         Ast.App (Ast.var "f", [Ast.int 5]))
    in
    let b =
      Ast.Letrec
        ([ ("f", Ast.Lambda (["x"], Ast.Prim ("+", [Ast.var "x"; Ast.int
    1]))) ],
         Ast.App (Ast.var "f", [Ast.int 5]))
    in
    Alcotest.(check bool) "distinct values, same hash" true
      (Ast_diff.ast_hash a = Ast_diff.ast_hash b);
    eq_delta Same (Ast_diff.diff a b));
  (* expr_to_json round-trip is untouched by diff (diff only reads) *)
  ("diff: does not mutate (round-trip preserved)", `Quick, fun () ->
    let e = Ast.App (Ast.var "f", [Ast.int 1; Ast.var "x"]) in
    let j = Ast.expr_to_json e in
    (match Ast.expr_of_json j with
     | Ok e' ->
         Alcotest.(check bool) "round-trip" true (expr_eq e e')
     | Error _ -> Alcotest.fail "expr round-trip failed"));
]

let () =
  Alcotest.run "ast_diff" [ ("ast_diff", delta_tests) ]
```

245 lisp/test/codec_test.ml

```
(** Tests for the shared value + error codecs in pure/prims_pure.ml
    (borge/remote-eval.borg substrate).
    PURE tests -- no Postgres, no HTTP, no net. They verify that
    Prims_pure.value_to_json / json_to_value and the factored
    error_to_json / error_of_json round-trip losslessly over the wire
    shape both /api/eval and a proxy rim link. *)
open Pricklypear_lisp

(* --- value codec round-trip --- *)

let value_round_trips name expected =
  ( "value round-trip: " ^ name
  , `Quick
  , fun () ->
      let j = Prims_pure.value_to_json expected in
      let back = Prims_pure.json_to_value j in
      Alcotest.(check string)
        ("round-trip " ^ name)
        (Reader.pp_value expected)
        (Reader.pp_value back) )

let round_trip_tests =
  [ value_round_trips "VStr" (Ast.VStr "hello")
  ; value_round_trips "VInt" (Ast.VInt 42L)
  ; value_round_trips "negative VInt" (Ast.VInt (-7L))
  ; value_round_trips "VFloat" (Ast.VFloat 3.14)
  ; value_round_trips "integral VFloat stays VFloat" (Ast.VFloat 5.0)
  ; ( "value round-trip: VFloat identity (constructor kept)"
    , `Quick
    , fun () ->
        let j = Prims_pure.value_to_json (Ast.VFloat 3.14) in
        match Prims_pure.json_to_value j with
        | Ast.VFloat f ->
            if abs_float (f -. 3.14) > 1e-9 then
              Alcotest.failf "float value %g" f
        | other ->
            Alcotest.failf "expected VFloat, got %s" (Reader.pp_value
    other) )
  ; value_round_trips "VBool true" (Ast.VBool true)
  ; value_round_trips "VBool false" (Ast.VBool false)
  ; value_round_trips "VNil" Ast.VNil
  ; value_round_trips "VList" (Ast.VList [ Ast.VInt 1L; Ast.VStr "a";
    Ast.VBool true ])
  ; value_round_trips "nested VList" (Ast.VList [ Ast.VList [ Ast.VInt 1L
    ]; Ast.VNil ])
  ; ( "value round-trip: VPair (dotted) via list wire form"
    , `Quick
    , fun () ->
        (* Pairs are normalized to their list wire form: (1 . 2) travels
           as [1,2] and comes back as VList [1;2]. Data is preserved
           losslessly even though the constructor normalizes. *)
        let expected = Ast.VPair (Ast.VInt 1L, Ast.VInt 2L) in
        let j = Prims_pure.value_to_json expected in
        let back = Prims_pure.json_to_value j in
        Alcotest.(check string)
          "pair-list wire form" (Reader.pp_value (Ast.VList [ Ast.VInt 1L;
    Ast.VInt 2L ]))
          (Reader.pp_value back) )
  ; ( "value round-trip: VPair proper list via list wire form"
    , `Quick
    , fun () ->
        let expected =
          Ast.VPair (Ast.VInt 1L, Ast.VPair (Ast.VInt 2L, Ast.VNil))
        in
        let j = Prims_pure.value_to_json expected in
        let back = Prims_pure.json_to_value j in
        Alcotest.(check string)
          "pair list wire form" (Reader.pp_value (Ast.VList [ Ast.VInt 1L;
    Ast.VInt 2L ]))
          (Reader.pp_value back) )
  ; value_round_trips "JSON-shaped VStr stays structured"
```

```
      (Ast.VStr "{\"a\":1,\"b\":[true,false]}")
  ]

(* Non-round-trippable values: VLambda/VPrim/VMacro serialize to a
   sentinel string and come back as a VStr (never the original). *)
let non_round_trip_sentinel name v =
  ( "value sentinel: " ^ name
  , `Quick
  , fun () ->
      let j = Prims_pure.value_to_json v in
      let back = Prims_pure.json_to_value j in
      (match back with
       | Ast.VStr _ -> ()
       | other ->
           Alcotest.failf
             "%s should come back as a VStr sentinel, got %s"
             name (Reader.pp_value other)) )

let sentinel_tests =
  [ non_round_trip_sentinel "VLambda" Ast.(VLambda ([ "x" ], Var "x", {
     bindings = Hashtbl.create 0; parent = None }))
  ; non_round_trip_sentinel "VPrim" (Ast.VPrim "+")
  ; non_round_trip_sentinel "VMacro" Ast.(VMacro ([ "x" ], Var "x", {
     bindings = Hashtbl.create 0; parent = None }))
  ]

(* On the wire, JSON strings map to VStr; a VSymbol handed to
   value_to_json does NOT come back as a VSymbol (no symbols on the
   wire -- symbols are a read-side concern). *)
let () =
  ()

let vsymbol_wire =
  ("value symbol: no VSymbol on wire", `Quick, fun () ->
    let j = Prims_pure.value_to_json (Ast.VSymbol "foo") in
    match Prims_pure.json_to_value j with
    | Ast.VStr _ -> ()
    | other ->
        Alcotest.failf
          "VSymbol should come back as VStr, got %s" (Reader.pp_value
     other))

(* --- error codec round-trip --- *)

let err_round_trips name (expected : exn) =
  ( "error round-trip: " ^ name
  , `Quick
  , fun () ->
      let j = Prims_pure.error_to_json expected in
      let back = Prims_pure.error_of_json j in
      (* Re-raise/pattern-match both to compare structurally. *)
      let canon (e : exn) =
        match e with
        | Ast.Interp_error ie -> "interp:" ^ Ast.pp_interp_error ie
        | Reader.Reader_error (m, l, c) -> Printf.sprintf "reader:%s:%d:%d"
    m l c
        | _ -> Printexc.to_string e
      in
      Alcotest.(check string)
        ("error round-trip " ^ name)
        (canon expected) (canon back) )

let error_tests =
  [ err_round_trips "Prim_error"
      (Ast.Interp_error (Ast.Prim_error ("foo", "boom", "my-kind")))
  ; err_round_trips "Prim_error with kind arity"
      (Ast.Interp_error (Ast.Prim_error ("get", "no such row",
    "not-found")))
  ; err_round_trips "Unbound_var"
      (Ast.Interp_error (Ast.Unbound_var "missing"))
  ; err_round_trips "Reader_error"
      (Reader.Reader_error ("unterminated list", 3, 17))
  ; err_round_trips "Reader_error empty list"
```

```
      (Reader.Reader_error ("unexpected end of input", 0, 0))
  ; err_round_trips "Budget_exceeded"
      (Ast.Interp_error Ast.Budget_exceeded)
  ; err_round_trips "User_error"
      (Ast.Interp_error (Ast.User_error ("needs 2 strings", "bad-args")))
  ; err_round_trips "Payload error kind stays"
      (Ast.Interp_error (Ast.User_error ("boom", "server_only")))
  ]

(* wire-shape assertions: the error JSON carries ok:false and the
   kind/name/line/col fields /api/eval clients already read. *)
let wire_shape =
  ("error wire shape", `Quick, fun () ->
    let open Yojson.Safe in
    let j =
      Prims_pure.error_to_json
        (Ast.Interp_error (Ast.Prim_error ("get", "no row", "not-found")))
    in
    (match j with
     | `Assoc fields ->
         let get k = List.assoc_opt k fields in
         (match get "ok", get "kind", get "name" with
          | Some (`Bool false), Some (`String "not-found"), Some (`String "get") -> ()
          | _ -> Alcotest.failf "unexpected Prim_error wire: %s" (to_string j))
     | _ -> Alcotest.failf "expected assoc, got %s" (to_string j));
    let j2 =
      Prims_pure.error_to_json
        (Ast.Interp_error (Ast.Unbound_var "x"))
    in
    (match j2 with
     | `Assoc fields ->
         (match List.assoc_opt "name" fields, List.assoc_opt "error" fields with
          | Some (`String "x"), Some (`String "unbound: x") -> ()
          | _ -> Alcotest.failf "unexpected Unbound_var wire: %s" (to_string j2))
     | _ -> Alcotest.failf "expected assoc, got %s" (to_string j2));
    let j3 =
      Prims_pure.error_to_json (Reader.Reader_error ("bad", 1, 2))
    in
    (match j3 with
     | `Assoc fields ->
         (match List.assoc_opt "kind" fields, List.assoc_opt "line" fields,
                List.assoc_opt "col" fields with
          | Some (`String "parse_error"), Some (`Int 1), Some (`Int 2) -> ()
          | _ -> Alcotest.failf "unexpected Reader_error wire: %s" (to_string j3))
     | _ -> Alcotest.failf "expected assoc, got %s" (to_string j3)) )

let () =
  Alcotest.run "codec"
    [ ( "value", round_trip_tests @ sentinel_tests @ [ vsymbol_wire ] )
    ; ("error", error_tests @ [ wire_shape ]) ]
```

246 lisp/test/conformance_test.ml

```
(** Nopales core conformance -- alcotest runner.

    The case data lives in the shared [pricklypear-conformance] library
    ([lisp/conformance/conformance.ml]) so the js_of_ocaml browser-target
    runner runs the SAME cases. Spec [lisp-interpreter.conformance]:
    this is the semantic gate of core-freeze. *)
open Pricklypear_lisp

open Pricklypear_lisp
open Pricklypear_conformance
open Conformance

let run_case c =
  ( c.Conformance.name,
    `Quick,
    fun () ->
      let prims, env = Conformance.setup_arith () in
      List.iter (fun s -> ignore (Eval.eval_string ~prims env s)) c.setups;
      let got = Conformance.eval_pp ~prims env c.src in
      Alcotest.(check string) c.name c.expected got )

(* Group cases by their declared group to preserve the test layout. *)
let groups =
  let tbl = Hashtbl.create 8 in
  List.iter (fun c ->
      let l = try Hashtbl.find tbl c.group with Not_found -> [] in
      Hashtbl.replace tbl c.group (run_case c :: l))
    Conformance.cases;
  Hashtbl.fold (fun g lst acc -> (g, List.rev lst) :: acc) tbl []
  |> List.sort (fun (a, _) (b, _) -> compare a b)

(* Error-handling cases: alcotest-specific (exception metadata), kept
   here rather than in the shared data so the JS runner stays a clean
   pass/fail loop. *)
let error_cases =
  [ ( "unbound has kind unbound_var",
      `Quick,
      fun () ->
        let prims, env = Conformance.setup_arith () in
        match
          try
            ignore (Eval.eval_string ~prims env "no-such-binding");
            None
          with Ast.Interp_error e -> Some e
        with
        | Some (Ast.Unbound_var "no-such-binding" as e) ->
            Alcotest.(check string) "kind" "unbound_var"
              (Ast.interp_error_kind e)
        | Some e -> Alcotest.failf "wrong: %s" (Ast.pp_interp_error e)
        | None -> Alcotest.fail "expected unbound" )
  ; ( "budget not caught by guard",
      `Quick,
      fun () ->
        let prims, env = Conformance.setup_arith () in
        ignore (Eval.eval_string ~prims env "(define loop (lambda ()
    (loop)))");
        let raised =
          try
            let budget = Eval.make_budget ~limit:40 () in
            ignore
              (Control.run_sync ~prims (fun () ->
                   Eval.eval ~budget ~prims:Eval.empty_prims env
                     (Reader.read_one "(guard (lambda (e) 0) (loop))")));
            false
          with
          | Ast.Interp_error Ast.Budget_exceeded -> true
          | _ -> false
        in
        Alcotest.(check bool) "budget escapes" true raised )
  ; ( "parse error has line col",
      `Quick,
```

```
      fun () ->
        match
          try
            ignore (Reader.read_one "(foo");
            None
          with Reader.Reader_error (_, line, col) -> Some (line, col)
        with
        | Some (line, col) ->
            Alcotest.(check bool) "line" true (line >= 1);
            Alcotest.(check bool) "col" true (col >= 1)
        | None -> Alcotest.fail "expected parse error" )
  ]

let () =
  Alcotest.run "nopales-conformance" (groups @ [ "errors", error_cases ])
```

247 lisp/test/effect_test.ml

```
(** Capability effect inference tests -- pure, no services. *)
open Pricklypear_lisp

let infer s =
  let e = Reader.read_one s in
  Effect.infer e

let pp_fx e = Effect.EffSet.pp e

let contains ~sub s =
  let n = String.length s and m = String.length sub in
  let rec find i =
    if i + m > n then false
    else if String.sub s i m = sub then true
    else find (i + 1)
  in
  m = 0 || find 0

let checks = [
  ("pure arithmetic", `Quick, fun () ->
    let fx = infer "(* x x)" in
    Alcotest.(check bool) "pure" true (Effect.EffSet.is_pure fx));
  ("lambda pure", `Quick, fun () ->
    let fx = infer "(lambda (x) (* x x))" in
    Alcotest.(check bool) "pure" true (Effect.EffSet.is_pure fx));
  ("get is read-db", `Quick, fun () ->
    let fx = infer "(get 'note id)" in
    Alcotest.(check bool) "read" true (Effect.EffSet.mem Effect.ReadDb fx);
    Alcotest.(check bool) "not write" false (Effect.EffSet.mem
     Effect.WriteDb fx));
  ("create is write-db", `Quick, fun () ->
    let fx = infer "(create 'note \"{}\")" in
    Alcotest.(check bool) "write" true (Effect.EffSet.mem Effect.WriteDb
     fx));
  ("route is publish", `Quick, fun () ->
    let fx = infer "(route \"/x\" \"f\")" in
    Alcotest.(check bool) "pub" true (Effect.EffSet.mem Effect.Publish fx));
  ("render-html is html", `Quick, fun () ->
    let fx = infer "(render-html \"<b>x</b>\")" in
    Alcotest.(check bool) "html" true (Effect.EffSet.mem Effect.Html fx));
  ("propose-function is ai", `Quick, fun () ->
    let fx = infer "(propose-function \"f\" \"spec\")" in
    Alcotest.(check bool) "ai" true (Effect.EffSet.mem Effect.Ai fx));
  ("compose read+html", `Quick, fun () ->
    let fx = infer "(lambda () (render-html (dict-get (get 'n id) \"t\")))"
     in
    Alcotest.(check bool) "read" true (Effect.EffSet.mem Effect.ReadDb fx);
    Alcotest.(check bool) "html" true (Effect.EffSet.mem Effect.Html fx));
  ("badge pure", `Quick, fun () ->
    Alcotest.(check string) "b" "pure" (Effect.EffSet.badge
     Effect.EffSet.empty));
  ("policy allows pure", `Quick, fun () ->
    Alcotest.(check bool) "ok" true
      (Effect.policy_allows ~allow:Effect.ai_default_allow
     Effect.EffSet.empty));
  ("policy allows write-db for app scaffolding", `Quick, fun () ->
    let got = Effect.EffSet.singleton Effect.WriteDb in
    Alcotest.(check bool) "allow write" true
      (Effect.policy_allows ~allow:Effect.ai_default_allow got));
  ("policy denies publish without elevate", `Quick, fun () ->
    let got = Effect.EffSet.singleton Effect.Publish in
    Alcotest.(check bool) "deny publish" false
      (Effect.policy_allows ~allow:Effect.ai_default_allow got));
  ("policy allows read+write+html", `Quick, fun () ->
    let got =
      Effect.EffSet.normalize
        [ Effect.ReadDb; Effect.WriteDb; Effect.Html ]
    in
    Alcotest.(check bool) "ok" true
      (Effect.policy_allows ~allow:Effect.ai_default_allow got));
  ("public_route_allow default has no write-db", `Quick, fun () ->
```

```
    Unix.putenv "PP_PUBLIC_ALLOW_WRITE" "0";
    let allow = Effect.public_route_allow () in
    Alcotest.(check bool) "no write" false
      (Effect.EffSet.mem Effect.WriteDb allow);
    Alcotest.(check bool) "has read" true
      (Effect.EffSet.mem Effect.ReadDb allow);
    let got = Effect.EffSet.singleton Effect.WriteDb in
    Alcotest.(check bool) "deny write" false
      (Effect.policy_allows ~allow got));
  ("public_route_allow write opt-in", `Quick, fun () ->
    Unix.putenv "PP_PUBLIC_ALLOW_WRITE" "1";
    let allow = Effect.public_route_allow () in
    Alcotest.(check bool) "write" true
      (Effect.EffSet.mem Effect.WriteDb allow);
    Unix.putenv "PP_PUBLIC_ALLOW_WRITE" "0");
  (* H0.1 audit coverage for previously unlisted registered prims *)
  ("div/abs pure", `Quick, fun () ->
    let fx = infer "(div (abs x) 2)" in
    Alcotest.(check bool) "pure" true (Effect.EffSet.is_pure fx));
  ("/ alias pure (no drift vs div)", `Quick, fun () ->
    let fx = infer "(/ 10 4)" in
    Alcotest.(check bool) "pure" true (Effect.EffSet.is_pure fx));
  ("quotient/remainder pure", `Quick, fun () ->
    let fx = infer "(remainder (quotient x y) z)" in
    Alcotest.(check bool) "pure" true (Effect.EffSet.is_pure fx));
  ("math tower pure", `Quick, fun () ->
    let fx = infer "(sqrt (expt (sin x) 2))" in
    Alcotest.(check bool) "pure" true (Effect.EffSet.is_pure fx));
  ("number? pure", `Quick, fun () ->
    let fx = Effect.effects_for_prim "number?" in
    Alcotest.(check bool) "pure" true (Effect.EffSet.is_pure fx));
  ("random pure", `Quick, fun () ->
    let fx = Effect.effects_for_prim "random" in
    Alcotest.(check bool) "pure" true (Effect.EffSet.is_pure fx));
  ("public-route is publish", `Quick, fun () ->
    let fx = Effect.effects_for_prim "public-route" in
    Alcotest.(check bool) "pub" true (Effect.EffSet.mem Effect.Publish fx));
  ("load-library is write-db", `Quick, fun () ->
    let fx = Effect.effects_for_prim "load-library" in
    Alcotest.(check bool) "write" true (Effect.EffSet.mem Effect.WriteDb
     fx));
  ("agent-say is ai+write", `Quick, fun () ->
    let fx = Effect.effects_for_prim "agent-say" in
    Alcotest.(check bool) "ai" true (Effect.EffSet.mem Effect.Ai fx);
    Alcotest.(check bool) "write" true (Effect.EffSet.mem Effect.WriteDb
     fx));
  ("redirect is html", `Quick, fun () ->
    let fx = Effect.effects_for_prim "redirect" in
    Alcotest.(check bool) "html" true (Effect.EffSet.mem Effect.Html fx));
  (* H0.2 known_fn derive-on-read *)
  ("known_fn pulls callee write-db", `Quick, fun () ->
    let save =
      Reader.read_one "(lambda (j) (create 'note j))"
    in
    let page =
      Reader.read_one "(lambda () (save \"{}\"))"
    in
    let known =
      Effect.known_fn_of_bindings [("save", save)]
    in
    let fx = Effect.infer ~known_fn:known page in
    Alcotest.(check bool) "write via callee" true
      (Effect.EffSet.mem Effect.WriteDb fx));
  ("known_fn cycle does not diverge", `Quick, fun () ->
    let a = Reader.read_one "(lambda () (b))" in
    let b = Reader.read_one "(lambda () (a))" in
    let known = Effect.known_fn_of_bindings [("a", a); ("b", b)] in
    let fx = Effect.infer ~known_fn:known (Reader.read_one "(a)") in
    Alcotest.(check bool) "pure cycle" true (Effect.EffSet.is_pure fx));
  (* H0.4 soft txn lint *)
  ("txn_lint multi-write without txn", `Quick, fun () ->
    let e =
      Reader.read_one
```

```
        "(lambda () (create 'n \"{}\") (delete 'n id))"
    in
    let hs = Effect.txn_lint e in
    Alcotest.(check bool) "has hint" true (hs <> []);
    Alcotest.(check bool) "mentions write-db sites" true
      (List.exists (contains ~sub:"write-db sites") hs));
  ("txn_lint quiet with with-transaction", `Quick, fun () ->
    let e =
      Reader.read_one
        "(with-transaction (lambda () (create 'n \"{}\") (delete 'n id)))"
    in
    let hs = Effect.txn_lint e in
    Alcotest.(check bool) "no multi-write hint" true
      (not (List.exists (contains ~sub:"write-db sites") hs)));
  ("txn_lint select-for-update bare", `Quick, fun () ->
    let e = Reader.read_one "(lambda () (select-for-update 'n id))" in
    let hs = Effect.txn_lint e in
    Alcotest.(check bool) "s4u hint" true
      (List.exists (contains ~sub:"select-for-update") hs));
]

let () =
  Alcotest.run "pricklypear-lisp-effects" [ "effects", checks ]
```

248 lisp/test/eval_test.ml

```
(** Tests for the Pricklypear Lisp interpreter.

   These are PURE tests -- no Postgres, no HTTP, no SSH. They verify the
   reader, AST round-trip, and evaluator. Per spec
   [lisp-interpreter.eval] / [workflow.build]:
     cd lisp/ && dune build && dune runtest
   needs no services. *)
open Pricklypear_lisp

(* --- Reader tests --- *)

let reads_as name input expected =
  ("reader: " ^ name, `Quick, fun () ->
    match Reader.read_one input with
    | Lit v -> Alcotest.(check string)
                 (Reader.pp_value expected) (Reader.pp_value expected)
                 (Reader.pp_value v)
    | other -> Alcotest.failf "expected Lit, got %s" (Reader.pp_expr other))

let reader_tests = [
  reads_as "int" "42" (Ast.VInt 42L);
  reads_as "negative int" "-7" (Ast.VInt (-7L));
  ("reader: float literal", `Quick, fun () ->
    match Reader.read_one "3.14" with
    | Lit (Ast.VFloat f) ->
        if abs_float (f -. 3.14) > 1e-9 then
          Alcotest.failf "float value %g" f
    | other -> Alcotest.failf "expected VFloat lit, got %s" (Reader.pp_expr
    other));
  reads_as "true" "#t" (Ast.VBool true);
  reads_as "false" "#f" (Ast.VBool false);
  reads_as "string" "\"hello\"" (Ast.VStr "hello");
  reads_as "nil" "()" Ast.VNil;
  (* UTF-8 degree sign as raw bytes must round-trip via pp_value (not
     OCaml String.escaped \\194\\176, which the old reader turned into
     literal digits "194176" and broke the weather widget). *)
  ("reader: utf8 string round-trip", `Quick, fun () ->
    let deg = "\xC2\xB0" in
    let original = "71" ^ deg ^ "F" in
    let printed = Reader.pp_value (Ast.VStr original) in
    (* printer keeps UTF-8 bytes; must not emit \\194\\176 *)
    Alcotest.(check string) "escape_string keeps utf8"
      original (Reader.escape_string original);
    Alcotest.(check bool) "pp has no backslash-digit for degree"
      false (String.contains printed '\\' &&
             let i = try String.index printed '\\' with Not_found -> -1 in
             i >= 0 && i + 1 < String.length printed &&
             printed.[i + 1] >= '0' && printed.[i + 1] <= '9');
    match Reader.read_one printed with
    | Lit (Ast.VStr s) ->
        Alcotest.(check string) "utf8 round-trip" original s
    | other -> Alcotest.failf "expected string lit, got %s" (Reader.pp_expr
    other));
  ("reader: decimal \\ddd byte escape", `Quick, fun () ->
    (* Legacy OCaml String.escaped form still loads: \\194\\176 ? UTF-8 ? *)
    match Reader.read_one "\"\\194\\176F\"" with
    | Lit (Ast.VStr s) ->
        Alcotest.(check string) "ddd escape" "\xC2\xB0F" s
    | other -> Alcotest.failf "expected string lit, got %s" (Reader.pp_expr
    other));
  ("reader: named escapes", `Quick, fun () ->
    match Reader.read_one "\"a\\nb\\tc\\\"d\"" with
    | Lit (Ast.VStr s) ->
        Alcotest.(check string) "named" "a\nb\tc\"d" s
    | other -> Alcotest.failf "expected string lit, got %s" (Reader.pp_expr
    other));
  ("reader: \\uXXXX unicode escape", `Quick, fun () ->
    (* The menu-caret bug: "\u203e" must become the overline ? (U+203E,
       UTF-8 E2 80 BE), not the literal 4 chars "u203e". *)
    (match Reader.read_one "\"a\\u203eb\"" with
     | Lit (Ast.VStr s) ->
```

```
68          Alcotest.(check string) "u203e overline" "a\xE2\x80\xBEb" s
69        | other -> Alcotest.failf "expected string lit, got %s"
          (Reader.pp_expr other));
70      (* degree sign U+00B0 ? C2 B0 (the weather \u00B0 case) *)
71      (match Reader.read_one "\"71\\u00B0F\"" with
72        | Lit (Ast.VStr s) ->
73          Alcotest.(check string) "u00B0 degree" "71\xC2\xB0F" s
74        | other -> Alcotest.failf "expected string lit, got %s"
          (Reader.pp_expr other));
75      (* case-insensitive hex *)
76      (match Reader.read_one "\"\\u00b0\"" with
77        | Lit (Ast.VStr s) -> Alcotest.(check string) "lower hex" "\xC2\xB0" s
78        | other -> Alcotest.failf "expected string lit, got %s"
          (Reader.pp_expr other));
79      (* BMP 3-byte scalar U+203E already covered; check a 1-byte ASCII *)
80      (match Reader.read_one "\"\\u0041\"" with
81        | Lit (Ast.VStr s) -> Alcotest.(check string) "u0041 = A" "A" s
82        | other -> Alcotest.failf "expected string lit, got %s"
          (Reader.pp_expr other)));
83    ("reader: \\uXXXX lone surrogate rejected", `Quick, fun () ->
84      let raised = ref false in
85      (try ignore (Reader.read_one "\"\\uD800\"")
86       with Reader.Reader_error _ -> raised := true);
87      Alcotest.(check bool) "lone surrogate D800 raises" true !raised);
88    ("reader: \\uXXXX needs 4 hex digits", `Quick, fun () ->
89      let raised = ref false in
90      (try ignore (Reader.read_one "\"\\u203\""   (* only 3 hex digits *))
91       with Reader.Reader_error _ -> raised := true);
92      Alcotest.(check bool) "short \\u raises" true !raised);
93    ("reader: \\uXXXX round-trips through pp_value", `Quick, fun () ->
94      (* escape_string keeps the UTF-8 bytes, so reading back the printed
95         form preserves the overline -- no \\u needed on the print side. *)
96      let s = "x\xE2\x80\xBEy" in
97      match Reader.read_one (Reader.pp_value (Ast.VStr s)) with
98      | Lit (Ast.VStr s') -> Alcotest.(check string) "roundtrip" s s'
99      | other -> Alcotest.failf "expected string lit, got %s" (Reader.pp_expr
          other));
100   ("reader: forgiving fallback still drops backslash for non-u", `Quick,
        fun () ->
101     (* Unrelated unknown escape still hits the forgiving rule. *)
102     match Reader.read_one "\"a\\qb\"" with
103     | Lit (Ast.VStr s) -> Alcotest.(check string) "forgiving \\q -> q"
          "aqb" s
104     | other -> Alcotest.failf "expected string lit, got %s" (Reader.pp_expr
          other));
105   (* Line comments: `;` must skip to end of line, not just consume the
106      `;` itself. Regression for the bug where `;; package structed ...`
107      headers lexed as the symbol `package` (every consumer worked around
108      it by pre-stripping comments in image/lib/loader.ml). *)
109   ("reader: trailing line comment is skipped", `Quick, fun () ->
110     match Reader.read_one "42 ; this is a comment" with
111     | Lit (Ast.VInt n) -> Alcotest.(check int64) "42" 42L n
112     | other -> Alcotest.failf "expected 42, got %s" (Reader.pp_expr other));
113   ("reader: leading line comment then form", `Quick, fun () ->
114     match Reader.read_one ";; package structed\n(define x 99)" with
115     | App (Var "define", _) -> ()
116     | other -> Alcotest.failf "expected define, got %s" (Reader.pp_expr
          other));
117   ("reader: double-semicolon header comment", `Quick, fun () ->
118     (* The exact shape of a libs/*.pp header -- the original bug left
119        `package` as a bare symbol after consuming only the two `;;`. *)
120     match Reader.read_all ";; package structed/list -- exported from
          system/main\n\n(define x 99)" with
121     | [ App (Var "define", _) ] -> ()
122     | other -> Alcotest.failf "expected single define form, got %d forms"
123                  (List.length other));
124   ("reader: comment-only input reads as no forms", `Quick, fun () ->
125     match Reader.read_all ";; just a comment\n; another\n" with
126     | [] -> ()
127     | other -> Alcotest.failf "expected 0 forms, got %d" (List.length
          other));
128 ]
129
```

```
let reader_list_tests = [
  ("reader: list of ints parses", `Quick, fun () ->
    (* (1 2 3) reads as App(Lit 1, [Lit 2; Lit 3]); head is separate *)
    match Reader.read_one "(1 2 3)" with
    | App (_, [ Lit (Ast.VInt _); Lit (Ast.VInt _) ]) -> ()
    | other -> Alcotest.failf "expected App with 2 li int args, got %s"
                                 (Reader.pp_expr other));
  ("reader: nested list parses", `Quick, fun () ->
    match Reader.read_one "(foo (bar) baz)" with
    | Ast.App _ -> ()
    | other -> Alcotest.failf "expected App, got %s" (Reader.pp_expr
    other));
]

let reader_completeness_tests = [
  ("is_complete: balanced parens", `Quick, fun () ->
    Alcotest.(check bool) "balanced" true (Reader.is_complete "(foo bar)"));
  ("is_complete: unbalanced parens", `Quick, fun () ->
    Alcotest.(check bool) "unbalanced" false (Reader.is_complete "(foo
    bar"));
  ("is_complete: unclosed string", `Quick, fun () ->
    Alcotest.(check bool) "unclosed" false (Reader.is_complete "\"hello"));
  ("is_complete: comment only", `Quick, fun () ->
    Alcotest.(check bool) "comment" true (Reader.is_complete "; comment
    only"));
]

(* Dotted-pair notation is rejected at read time (improper lists are not
   supported). A lone '.' must raise a clean Reader_error instead of
   silently becoming a symbol named "." that corrupts list length. *)
let reader_dotted_tests = [
  ("reader: dotted pair (1 . 2) raises", `Quick, fun () ->
    let raised =
      try
        let _ = Reader.read_one "(1 . 2)" in
        false
      with Reader.Reader_error _ -> true
    in
    Alcotest.(check bool) "dotted pair raises" true raised);
  ("reader: dotted rest (1 2 . 3) raises", `Quick, fun () ->
    let raised =
      try
        let _ = Reader.read_one "(1 2 . 3)" in
        false
      with Reader.Reader_error _ -> true
    in
    Alcotest.(check bool) "dotted rest raises" true raised);
  ("reader: bare dot raises", `Quick, fun () ->
    let raised =
      try
        let _ = Reader.read_one "." in
        false
      with Reader.Reader_error _ -> true
    in
    Alcotest.(check bool) "bare dot raises" true raised);
  ("reader: float .5 still parses", `Quick, fun () ->
    match Reader.read_one ".5" with
    | Lit (Ast.VFloat f) ->
        if abs_float (f -. 0.5) > 1e-9 then
          Alcotest.failf "expected 0.5, got %g" f
    | other ->
        Alcotest.failf "expected VFloat lit, got %s" (Reader.pp_expr
    other));
  ("reader: ellipsis ... still a symbol", `Quick, fun () ->
    match Reader.read_one "..." with
    | Ast.Var "..." -> ()
    | other ->
        Alcotest.failf "expected Var ..., got %s" (Reader.pp_expr other));
]

(* --- AST JSON round-trip tests --- *)

let roundtrip name e =
```

```
  ("ast roundtrip: " ^ name, `Quick, fun () ->
    let j = Ast.expr_to_json e in
    (match Ast.expr_of_json j with
     | Ok e' ->
        Alcotest.(check string)
          (Reader.pp_expr e) (Reader.pp_expr e) (Reader.pp_expr e')
     | Error err -> Alcotest.failf "roundtrip failed: %s" err))

let ast_roundtrip_tests = [
  roundtrip "var" (Ast.Var "x");
  roundtrip "int" (Ast.int 5);
  roundtrip "string" (Ast.str "hello");
  roundtrip "if" (Ast.If (Ast.var "x", Ast.int 1, Ast.int 2));
  roundtrip "lambda" (Ast.Lambda (["x"], Ast.var "x"));
  roundtrip "let" (Ast.Let ("y", Ast.int 5, Ast.var "y"));
  roundtrip "prim" (Ast.Prim ("query", [Ast.str "event"]));
  roundtrip "define" (Ast.Define ("greet", Ast.Lambda (["name"], Ast.Var
     "name")));
  roundtrip "app" (Ast.App (Ast.Var "f", [Ast.int 1; Ast.int 2]));
]

(* --- Evaluator tests --- *)

(* H1.7: interpreter is direct-style; [Eval.eval_string] runs under
   [Control.run_sync]. Pure tests -- no services. *)

let evals_to name expr expected =
  ("eval: " ^ name, `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let budget = Eval.make_budget () in
    let got = (Eval.eval ~budget ~prims env expr) in
    Alcotest.(check string)
      (Reader.pp_value expected) (Reader.pp_value expected)
      (Reader.pp_value got))

let eval_tests = [
  evals_to "int literal" (Ast.int 42) (Ast.VInt 42L);
  ("eval: define binds var", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let _ = (Eval.eval_string ~prims env "(define x 99)") in
    (match Env.lookup env "x" with
     | Some (Ast.VInt n) ->
        Alcotest.(check int64) "x's value" 99L n
     | other -> Alcotest.failf "expected Some VInt, got %s"
                                (match other with None -> "None"
                                 | Some v -> Reader.pp_value v)));
  ("eval: if-true branch", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let got = (Eval.eval_string ~prims env "(if #t 1 2)") in
    Alcotest.(check string) "1" "1" (Reader.pp_value got));
  ("eval: if-false branch", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let got = (Eval.eval_string ~prims env "(if #f 1 2)") in
    Alcotest.(check string) "2" "2" (Reader.pp_value got));
  ("eval: lambda application", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let got = (Eval.eval_string ~prims env "((lambda (x) x) 42)") in
    Alcotest.(check string) "42" "42" (Reader.pp_value got));
  ("eval: quote compound list", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let got = (Eval.eval_string ~prims env "'(a b c)") in
    (match got with
     | Ast.VList [Ast.VSymbol "a"; Ast.VSymbol "b"; Ast.VSymbol "c"] -> ()
     | other -> Alcotest.failf "expected (a b c) list, got %s"
                  (Reader.pp_value other)));
  ("eval: quote nested", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
```

```
    let env = Env.create () in
    let got = (Eval.eval_string ~prims env "'(1 (2 3))") in
    (match got with
     | Ast.VList [Ast.VInt 1L; Ast.VList [Ast.VInt 2L; Ast.VInt 3L]] -> ()
     | other -> Alcotest.failf "expected nested quote, got %s"
                  (Reader.pp_value other)));
  ("eval: primitive dispatch", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    Eval.register_prim prims "add-one"
      (fun args -> match args with
        | [Ast.VInt n] -> (Ast.VInt (Int64.add n 1L))
        | _ -> Ast.VNil);
    let env = Env.create () in
    let got = (Eval.eval_string ~prims env "(prim add-one 41)") in
    Alcotest.(check string) "42" "42" (Reader.pp_value got));
  ("eval: let binds locally", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let got = (Eval.eval_string ~prims env "(let ((y 5)) y)") in
    Alcotest.(check string) "5" "5" (Reader.pp_value got));
  ("eval: budget exceeded", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let budget = Eval.make_budget ~limit:10 () in
    (* build a deep nest to burn through 10 steps *)
    let rec nest n acc =
      if n = 0 then acc else nest (n-1) (Ast.Seq (Ast.int 1, acc))
    in
    let e = nest 20 (Ast.int 0) in
    try
      let _ = (Eval.eval ~budget ~prims env e) in
      Alcotest.fail "expected budget exceeded"
    with Ast.Interp_error Ast.Budget_exceeded -> ());
  ("eval: unbound var raises", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    try
      let _ = (Eval.eval_string ~prims env "nope") in
      Alcotest.fail "expected Unbound_var"
    with Ast.Interp_error (Ast.Unbound_var name) ->
      Alcotest.(check string) "nope" "nope" name);
  ("eval: seq empty returns nil", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let got = (Eval.eval_string ~prims env "(seq)") in
    Alcotest.(check string) "nil" "()" (Reader.pp_value got));
  ("eval: seq single returns body", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let got = (Eval.eval_string ~prims env "(seq 7)") in
    Alcotest.(check string) "7" "7" (Reader.pp_value got));
  ("eval: seq binary returns last", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let got = (Eval.eval_string ~prims env "(seq 1 2)") in
    Alcotest.(check string) "2" "2" (Reader.pp_value got));
  ("eval: seq multi returns last", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let got = (Eval.eval_string ~prims env "(seq 1 2 3)") in
    Alcotest.(check string) "3" "3" (Reader.pp_value got));
  ("eval: seq nested multi", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let got = (Eval.eval_string ~prims env
                            "(seq 1 (seq 2 3) 4)") in
    Alcotest.(check string) "4" "4" (Reader.pp_value got));
  ("eval: seq side effects left-to-right", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let got = (Eval.eval_string ~prims env
      "(seq (define a 1) (define b 2) (define c 3) c)") in
    Alcotest.(check string) "c" "3" (Reader.pp_value got);
```

```
    (match Env.lookup env "a", Env.lookup env "b" with
     | Some (Ast.VInt 1L), Some (Ast.VInt 2L) -> ()
     | _ -> Alcotest.fail "expected a=1 b=2 after seq defines"));
  ("eval: seq in lambda body multi", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let got = (Eval.eval_string ~prims env
      "((lambda () (seq (define x 10) (define y 20) y)))") in
    Alcotest.(check string) "20" "20" (Reader.pp_value got));
  ("eval: letrec simple lambda", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let got = (Eval.eval_string ~prims env
      "(letrec ((f (lambda (n) n))) (f 5))") in
    Alcotest.(check string) "5" "5" (Reader.pp_value got));
  ("eval: letrec self-recursion", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    (* loop calls itself once: #t -> recurse on #f -> base case 7.
       Prim-free: exercises the self-binding backpatch. *)
    let got = (Eval.eval_string ~prims env
      "(letrec ((loop (lambda (x) (if x (loop #f) 7)))) (loop #t))") in
    Alcotest.(check string) "7" "7" (Reader.pp_value got));
  ("eval: letrec mutual recursion", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    (* a's body calls b; b's body is a constant. Prim-free proof that
       each binding closes over env' and can reach its sibling. *)
    let got = (Eval.eval_string ~prims env
      "(letrec ((a (lambda () (b)))\
                (b (lambda () 42)))\
        (a))") in
    Alcotest.(check string) "42" "42" (Reader.pp_value got));
  ("eval: letrec empty bindings", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let got = (Eval.eval_string ~prims env "(letrec () 42)") in
    Alcotest.(check string) "42" "42" (Reader.pp_value got));
  ("eval: apply special form on lambda", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let got = (Eval.eval_string ~prims env
      "(apply (lambda (a b) b) '(1 2))") in
    Alcotest.(check string) "2" "2" (Reader.pp_value got));
  ("eval: apply special form on prim", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    Eval.register_prim prims "add2"
      (fun args -> match args with
       | [Ast.VInt a; Ast.VInt b] -> Ast.VInt (Int64.add a b)
       | _ -> Ast.VNil);
    let env = Env.create () in
    Env.define env "add2" (Ast.VPrim "add2");
    let got = (Eval.eval_string ~prims env
      "(apply add2 '(1 2))") in
    Alcotest.(check string) "3" "3" (Reader.pp_value got));
  ("eval: apply errors on wrong arity", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let raised =
      try
      let _ = (Eval.eval_string ~prims env
        "(apply (lambda (a b) a) '(1))") in
      false
      with Ast.Interp_error _ -> true
    in
    Alcotest.(check bool) "arity mismatch raises" true raised);
  ("eval: optional param omitted", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let got = (Eval.eval_string ~prims env
      "((lambda (a &optional b) b) 1)") in
    Alcotest.(check string) "empty for omitted optional" "\"\""
      (Reader.pp_value got));
```

```
  ("eval: optional param supplied", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let got = (Eval.eval_string ~prims env
      "((lambda (a &optional b) b) 1 2)") in
    Alcotest.(check string) "2nd arg" "2" (Reader.pp_value got));
  ("eval: rest param collects extras", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let got = (Eval.eval_string ~prims env
      "((lambda (a &rest rest) rest) 1 2 3)") in
    Alcotest.(check string) "rest list" "(2 3)" (Reader.pp_value got));
  ("eval: rest param all variadic", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let got = (Eval.eval_string ~prims env
      "((lambda (&rest all) all) 1 2 3)") in
    Alcotest.(check string) "all args" "(1 2 3)" (Reader.pp_value got));
  ("eval: optional + rest together", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let got = (Eval.eval_string ~prims env
      "((lambda (a &optional b &rest rest) rest) 1 2 3 4)") in
    Alcotest.(check string) "rest after optional" "(3 4)"
      (Reader.pp_value got));
  ("eval: fixed arity still enforced", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let raised =
      try
      let _ = (Eval.eval_string ~prims env "((lambda (a b) a) 1)") in
      false
      with Ast.Interp_error _ -> true
    in
    Alcotest.(check bool) "fixed arity mismatch raises" true raised);
  ("eval: optional arity too many raises", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let raised =
      try
      let _ = (Eval.eval_string ~prims env
        "((lambda (a &optional b) a) 1 2 3)") in
      false
      with Ast.Interp_error _ -> true
    in
    Alcotest.(check bool) "too many args raises" true raised);
  ("eval: required arity too few raises", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let raised =
      try
      let _ = (Eval.eval_string ~prims env
        "((lambda (a b &optional c) a) 1)") in
      false
      with Ast.Interp_error _ -> true
    in
    Alcotest.(check bool) "too few required raises" true raised);
]

let pretty_print_tests = [
  ("pp_expr: nested list breaks lines", `Quick, fun () ->
    let src =
      "(list (menu/link \"home\" \"/\") (menu/group \"apps\" (list
     (menu/link \"todos\" \"/todos\"))))"
    in
    let e = Reader.read_one src in
    let pp = Reader.pp_expr e in
    if not (String.contains pp '\n') then
      Alcotest.failf "expected newlines in:\n%s" pp;
    if not (String.contains pp 'a') then
      Alcotest.fail "missing content";
    (* Round-trip: pretty output still parses to same flat shape *)
    let e' = Reader.read_one pp in
```

```
    Alcotest.(check string) "flat equal"
      (Reader.pp_expr_flat e) (Reader.pp_expr_flat e'));
  ("pp_expr: short form stays one line", `Quick, fun () ->
    let e = Reader.read_one "(+ 1 2)" in
    let pp = Reader.pp_expr e in
    Alcotest.(check bool) "no newline" false (String.contains pp '\n');
    Alcotest.(check string) "compact" "(+ 1 2)" pp);
]

let macro_tests = [
  ("value_to_expr roundtrip reify list", `Quick, fun () ->
    let e = Reader.read_one "(a b c)" in
    let v = Eval.reify_expr e in
    let e' = Eval.value_to_expr v in
    Alcotest.(check string) "pp"
      (Reader.pp_expr_flat e) (Reader.pp_expr_flat e'));
  ("value_to_expr keeps string lits", `Quick, fun () ->
    let e = Reader.read_one "(html/attr \"class\" \"muted\")" in
    let v = Eval.reify_expr e in
    let e' = Eval.value_to_expr v in
    Alcotest.(check string) "pp"
      (Reader.pp_expr_flat e) (Reader.pp_expr_flat e'));
  ("quasiquote unquote", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    Eval.register_prim prims "+" (function
      | [Ast.VInt a; Ast.VInt b] -> (Ast.VInt (Int64.add a b))
      | _ -> raise (Ast.Interp_error (Ast.User_error ("+", "bad-args"))));
    let env = Env.create () in
    Env.define env "+" (Ast.VPrim "+");
    let got = (Eval.eval_string ~prims env "`(a ,(+ 1 2) c)") in
    match got with
    | Ast.VList [Ast.VSymbol "a"; Ast.VInt 3L; Ast.VSymbol "c"] -> ()
    | other -> Alcotest.failf "qq got %s" (Reader.pp_value other));
  ("quasiquote splice", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let got = (Eval.eval_string ~prims env "`(a ,@'(b c) d)") in
    match got with
    | Ast.VList [Ast.VSymbol "a"; Ast.VSymbol "b"; Ast.VSymbol "c";
     Ast.VSymbol "d"] -> ()
    | other -> Alcotest.failf "splice got %s" (Reader.pp_value other));
  ("defmacro when", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let _ =
      (Eval.eval_string ~prims env
           "(defmacro when (t b) `(if ,t ,b ()))")
    in
    let got =
      (Eval.eval_string ~prims env "(when #t 42)")
    in
    Alcotest.(check string) "42" "42" (Reader.pp_value got);
    let got2 =
      (Eval.eval_string ~prims env "(when #f 42)")
    in
    Alcotest.(check string) "nil" "()" (Reader.pp_value got2));
  ("defmacro expands to let (qq)", `Quick, fun () ->
    (* Inbox c7-2611: macros expanding to let crashed with opaque
       Interp_error. Must evaluate to the bound value. *)
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let _ =
      (Eval.eval_string ~prims env
           "(defmacro let1 (name val body) `(let ((,name ,val)) ,body))")
    in
    let got =
      (Eval.eval_string ~prims env "(let1 a 1 a)")
    in
    Alcotest.(check string) "let1 => 1" "1" (Reader.pp_value got));
  ("defmacro expands to let (list)", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    Eval.register_prim prims "list" (fun args -> (Ast.VList args));
    let env = Env.create () in
```

```
    (* Image binds registered prims into env as VPrim; pure tests must too.
     *)
    Env.define env "list" (Ast.VPrim "list");
    let _ =
      (Eval.eval_string ~prims env
           "(defmacro let1 (name val body)\
             (list (quote let) (list (list name val)) body))")
    in
    let got =
      try (Eval.eval_string ~prims env "(let1 a 1 a)")
      with Ast.Interp_error e ->
        Alcotest.failf "let1 list failed: %s" (Ast.pp_interp_error e)
    in
    Alcotest.(check string) "let1 list => 1" "1" (Reader.pp_value got));
  ("defmacro rejects dotted rest params", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    match
      try
        ignore
          ((Eval.eval_string ~prims env
                "(defmacro bad (a . b) a)"));
        `Ok
      with Ast.Interp_error (Ast.User_error (m, _)) -> `Err m
         | Ast.Interp_error e -> `Err (Ast.pp_interp_error e)
         | Reader.Reader_error (m, _, _) -> `Err m
    with
    | `Err m ->
        if not (String.starts_with ~prefix:"dotted rest" m
                || String.length m >= 6
                   && String.sub m 0 6 = "dotted") then
          Alcotest.failf "expected dotted rest error, got: %s" m
    | `Ok -> Alcotest.fail "expected dotted rest rejection");
  ("gensym unique", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let a = (Eval.eval_string ~prims env "(gensym)") in
    let b = (Eval.eval_string ~prims env "(gensym)") in
    match a, b with
    | Ast.VSymbol sa, Ast.VSymbol sb when sa <> sb -> ()
    | _ -> Alcotest.fail "gensym not unique");
  ("cond first match", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let got =
      (Eval.eval_string ~prims env
           "(cond (#f 1) (#t 2) (#t 3))")
    in
    Alcotest.(check string) "2" "2" (Reader.pp_value got));
  ("cond else", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let got =
      (Eval.eval_string ~prims env "(cond (#f 1) (else 9))")
    in
    Alcotest.(check string) "9" "9" (Reader.pp_value got));
  ("cond empty", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let got = (Eval.eval_string ~prims env "(cond)") in
    Alcotest.(check string) "nil" "()" (Reader.pp_value got));
  ("and short-circuit", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let got =
      (Eval.eval_string ~prims env "(and #t 1 2)")
    in
    Alcotest.(check string) "2" "2" (Reader.pp_value got);
    let got2 =
      (Eval.eval_string ~prims env "(and #t #f 3)")
    in
    Alcotest.(check string) "false" "#f" (Reader.pp_value got2);
    let got3 = (Eval.eval_string ~prims env "(and)") in
```

```
    Alcotest.(check string) "true" "#t" (Reader.pp_value got3));
  ("or short-circuit", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let got =
      (Eval.eval_string ~prims env "(or #f 0 1)")
    in
    Alcotest.(check string) "0" "0" (Reader.pp_value got);
    let got2 = (Eval.eval_string ~prims env "(or)") in
    Alcotest.(check string) "false" "#f" (Reader.pp_value got2));
]

(* H1 dual-path: eval_eff + Control.run_lwt (PP_EVAL_CONTROL=eff shape). *)
let eval_eff_tests = [
  ("eval_eff: int literal", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let got =
      (Eval.eval_string ~prims env "42")
    in
    Alcotest.(check string) "42" "42" (Reader.pp_value got));
  ("eval_eff: define + lookup", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let _ = (Eval.eval_string ~prims env "(define x 99)") in
    let got = (Eval.eval_string ~prims env "x") in
    Alcotest.(check string) "99" "99" (Reader.pp_value got));
  ("eval_eff: if + lambda app", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let got =
      (Eval.eval_string ~prims env "((lambda (x) (if #t x 0)) 7)")
    in
    Alcotest.(check string) "7" "7" (Reader.pp_value got));
  ("eval_eff: prim via Prim_call", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    Eval.register_prim prims "add-one"
      (fun args ->
        match args with
        | [ Ast.VInt n ] -> (Ast.VInt (Int64.add n 1L))
        | _ -> failwith "bad args");
    let env = Env.create () in
    let got =
      (Eval.eval_string ~prims env "(prim add-one 41)")
    in
    Alcotest.(check string) "42" "42" (Reader.pp_value got));
  ("eval_eff: composition bind then prim", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    Eval.register_prim prims "get"
      (fun args ->
        match args with
        | [ Ast.VStr _; Ast.VInt id ] ->
            (Ast.VStr ("row-" ^ Int64.to_string id))
        | _ -> failwith "bad get");
    let env = Env.create () in
    Env.define env "get" (Ast.VPrim "get");
    let got =
      (Eval.eval_string ~prims env
           "(let ((u (get \"user\" 1))) u)")
    in
    Alcotest.(check string) "row" "\"row-1\"" (Reader.pp_value got));
  ("eval_eff: budget still aborts", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let budget = Eval.make_budget ~limit:3 () in
    let e = Reader.read_one "(seq 1 2 3 4 5 6 7 8 9 10)" in
    (try
       let _ =
         Control.run_sync ~prims (fun () -> Eval.eval ~budget ~prims env e)
       in
       Alcotest.fail "expected Budget_exceeded"
     with
     | Ast.Interp_error Ast.Budget_exceeded -> ()
```

```
    | exn ->
        Alcotest.failf "wrong exn: %s" (Printexc.to_string exn)));
  (* H1.3: with-transaction ? __with_transaction__ via Prim_call; nested
     apply_eff under run_lwt so body prims still perform. *)
  ("eval_eff: with-transaction nests prims", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let seen = ref [] in
    Eval.register_prim prims "create"
      (fun args ->
        seen := "create" :: !seen;
        match args with
        | [ Ast.VStr t; Ast.VStr _ ] -> (Ast.VStr ("id-" ^ t))
        | _ -> failwith "bad create");
    Eval.register_prim prims "__with_transaction__"
      (fun args ->
        seen := "tx-enter" :: !seen;
        match args with
        | [ lam ] ->
            let budget = Eval.make_budget () in
            (* image-shaped: nested run_lwt + apply_eff *)
            Control.run_sync ~prims (fun () ->
                Eval.apply ~budget ~prims lam [])
        | _ -> failwith "bad tx");
    let env = Env.create () in
    Env.define env "create" (Ast.VPrim "create");
    let got =
      (Eval.eval_string ~prims env
           "(with-transaction (lambda () (create \"note\" \"{}\")))")
    in
    Alcotest.(check string) "id" "\"id-note\"" (Reader.pp_value got);
    Alcotest.(check bool) "tx ran" true (List.mem "tx-enter" !seen);
    Alcotest.(check bool) "create nested" true (List.mem "create" !seen));
  (* H1.5 special-form parity *)
  ("eval_eff: cond else", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let got =
      (Eval.eval_string ~prims env
           "(cond (#f 1) (else 2))")
    in
    Alcotest.(check string) "2" "2" (Reader.pp_value got));
  ("eval_eff: quasiquote unquote", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    Eval.register_prim prims "+"
      (fun args ->
        match args with
        | [ Ast.VInt a; Ast.VInt b ] -> (Ast.VInt (Int64.add a b))
        | _ -> failwith "+");
    let env = Env.create () in
    Env.define env "+" (Ast.VPrim "+");
    let got =
      (Eval.eval_string ~prims env "`(a ,(+ 1 2) c)")
    in
    match got with
    | Ast.VList [ Ast.VSymbol "a"; Ast.VInt 3L; Ast.VSymbol "c" ] -> ()
    | other -> Alcotest.failf "qq got %s" (Reader.pp_value other));
  ("eval_eff: defmacro when", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let _ =
      (Eval.eval_string ~prims env
           "(defmacro when (test body) `(if ,test ,body ()))")
    in
    let got =
      (Eval.eval_string ~prims env "(when #t 42)")
    in
    Alcotest.(check string) "42" "42" (Reader.pp_value got));
]

(* Macro string args: Lit VStr bound as (__str__ s) so expansion + unquote
   round-trip to Lit VStr (inbox c11-4013). *)
let macro_str_tests =
  let setup () =
```

```
    let prims = Eval.make_prim_table () in
    Eval.register_prim prims "string-append"
      (fun args ->
        let buf = Buffer.create 16 in
        List.iter
          (function
            | Ast.VStr s -> Buffer.add_string buf s
            | Ast.VList [ Ast.VStr "__str__"; Ast.VStr s ] ->
                Buffer.add_string buf s
            | _ -> failwith "string-append")
          args;
        Ast.VStr (Buffer.contents buf));
    let env = Env.create () in
    Env.define env "string-append" (Ast.VPrim "string-append");
    (prims, env)
  in
  [ ("macro string lit via quasiquote unquote", `Quick, fun () ->
      let prims, env = setup () in
      ignore
        (Eval.eval_string ~prims env
           "(defmacro m-str (x) `(string-append ,x \"-a\"))");
      let got = Eval.eval_string ~prims env "(m-str \"foo\")" in
      Alcotest.(check string) "foo-a" "\"foo-a\"" (Reader.pp_value got));
    ("macro string lit builds lambda body literal", `Quick, fun () ->
      let prims, env = setup () in
      ignore
        (Eval.eval_string ~prims env
           "(defmacro m-lam (x) `(lambda () ,x))");
      let got = Eval.eval_string ~prims env "((m-lam \"A\"))" in
      Alcotest.(check string) "A" "\"A\"" (Reader.pp_value got));
  ]

(* list/map|filter|foldl are kernel special forms (OCaml loops + apply). *)
let list_hof_tests =
  let setup () =
    let prims = Eval.make_prim_table () in
    Eval.register_prim prims "+"
      (fun args ->
        match args with
        | [ Ast.VInt a; Ast.VInt b ] -> Ast.VInt (Int64.add a b)
        | _ -> failwith "+");
    Eval.register_prim prims "*"
      (fun args ->
        match args with
        | [ Ast.VInt a; Ast.VInt b ] -> Ast.VInt (Int64.mul a b)
        | _ -> failwith "*");
    Eval.register_prim prims "="
      (fun args ->
        match args with
        | [ Ast.VInt a; Ast.VInt b ] -> Ast.VBool (Int64.equal a b)
        | _ -> Ast.VBool false);
    Eval.register_prim prims "list" (fun args -> Ast.VList args);
    Eval.register_prim prims "null?"
      (function
        | [ Ast.VNil ] | [ Ast.VList [] ] -> Ast.VBool true
        | _ -> Ast.VBool false);
    Eval.register_prim prims "car"
      (function
        | [ Ast.VList (h :: _) ] -> h
        | _ -> failwith "car");
    Eval.register_prim prims "cdr"
      (function
        | [ Ast.VList (_ :: t) ] -> Ast.VList t
        | _ -> failwith "cdr");
    Eval.register_prim prims "cons"
      (function
        | [ a; Ast.VList xs ] -> Ast.VList (a :: xs)
        | [ a; b ] -> Ast.VPair (a, b)
        | _ -> failwith "cons");
    let env = Env.create () in
    List.iter
      (fun n -> Env.define env n (Ast.VPrim n))
      [ "+"; "*"; "="; "list"; "null?"; "car"; "cdr"; "cons" ];
```

```
    (prims, env)
  in
  [ ("list/map doubles", `Quick, fun () ->
      let prims, env = setup () in
      let got =
        Eval.eval_string ~prims env
          "(list/map (lambda (x) (* x 2)) (list 1 2 3))"
      in
      Alcotest.(check string) "map" "(2 4 6)" (Reader.pp_value got));
    ("list/filter evens", `Quick, fun () ->
      let prims, env = setup () in
      let got =
        Eval.eval_string ~prims env
          "(list/filter (lambda (x) (= x 2)) (list 1 2 3 2))"
      in
      Alcotest.(check string) "filter" "(2 2)" (Reader.pp_value got));
    ("list/foldl sum", `Quick, fun () ->
      let prims, env = setup () in
      let got =
        Eval.eval_string ~prims env
          "(list/foldl (lambda (a x) (+ a x)) 0 (list 1 2 3 4))"
      in
      Alcotest.(check string) "fold" "10" (Reader.pp_value got));
    ("list/map empty", `Quick, fun () ->
      let prims, env = setup () in
      let got =
        Eval.eval_string ~prims env
          "(list/map (lambda (x) x) (list))"
      in
      Alcotest.(check string) "empty" "()" (Reader.pp_value got));
  ]

(* Tail-call optimization: trampoline keeps OCaml stack flat. *)
let tco_tests =
  let setup () =
    let prims = Eval.make_prim_table () in
    Eval.register_prim prims "+"
      (fun args ->
        match args with
        | [ Ast.VInt a; Ast.VInt b ] -> Ast.VInt (Int64.add a b)
        | _ -> failwith "+");
    Eval.register_prim prims "-"
      (fun args ->
        match args with
        | [ Ast.VInt a; Ast.VInt b ] -> Ast.VInt (Int64.sub a b)
        | [ Ast.VInt a ] -> Ast.VInt (Int64.neg a)
        | _ -> failwith "-");
    Eval.register_prim prims "*"
      (fun args ->
        match args with
        | [ Ast.VInt a; Ast.VInt b ] -> Ast.VInt (Int64.mul a b)
        | _ -> failwith "*");
    Eval.register_prim prims "="
      (fun args ->
        match args with
        | [ Ast.VInt a; Ast.VInt b ] -> Ast.VBool (Int64.equal a b)
        | _ -> Ast.VBool false);
    Eval.register_prim prims "<"
      (fun args ->
        match args with
        | [ Ast.VInt a; Ast.VInt b ] -> Ast.VBool (a < b)
        | _ -> Ast.VBool false);
    let env = Env.create () in
    List.iter
      (fun n -> Env.define env n (Ast.VPrim n))
      [ "+"; "-"; "*"; "="; "<" ];
    (prims, env)
  in
  [
    ("tail sum does not stack-overflow", `Quick, fun () ->
      let prims, env = setup () in
      (* Self-tail recursion: depth far above typical OCaml stack. *)
      ignore
```

```
        (Eval.eval_string ~prims env
           "(define sumt (lambda (n acc)\n\
               (if (= n 0) acc (sumt (- n 1) (+ acc n)))))");
      let budget = Eval.make_budget ~limit:2_000_000 () in
      let got =
        Control.run_sync ~prims (fun () ->
            Eval.eval ~budget ~prims:Eval.empty_prims env
              (Reader.read_one "(sumt 100000 0)"))
      in
      (* 100000*100001/2 = 5_000_050_000 *)
      Alcotest.(check string) "sum" "5000050000" (Reader.pp_value got));
    ("non-tail factorial still works", `Quick, fun () ->
      let prims, env = setup () in
      ignore
        (Eval.eval_string ~prims env
           "(define fac (lambda (n)\n\
               (if (= n 0) 1 (* n (fac (- n 1))))))");
      let got = Eval.eval_string ~prims env "(fac 10)" in
      Alcotest.(check string) "10!" "3628800" (Reader.pp_value got));
    ("infinite tail loop hits budget", `Quick, fun () ->
      let prims, env = setup () in
      ignore
        (Eval.eval_string ~prims env
           "(define loop (lambda () (loop)))");
      let raised =
        try
          ignore (Eval.eval_string ~prims env "(loop)");
          false
        with
        | Ast.Interp_error Ast.Budget_exceeded -> true
        | _ -> false
      in
      Alcotest.(check bool) "budget" true raised);
    ("mutual tail via if", `Quick, fun () ->
      let prims, env = setup () in
      ignore
        (Eval.eval_string ~prims env
           "(define even? (lambda (n)\n\
               (if (= n 0) #t (odd? (- n 1)))))");
      ignore
        (Eval.eval_string ~prims env
           "(define odd? (lambda (n)\n\
               (if (= n 0) #f (even? (- n 1)))))");
      let budget = Eval.make_budget ~limit:2_000_000 () in
      let got =
        Control.run_sync ~prims (fun () ->
            Eval.eval ~budget ~prims:Eval.empty_prims env
              (Reader.read_one "(even? 100000)"))
      in
      Alcotest.(check string) "even" "#t" (Reader.pp_value got));
  ]

(** Minimal pure stand-ins for image symbol/string prims so dialect tests
    stay service-free while covering the symbols contract. *)
let register_symbol_prims (prims : Eval.prim_table) : unit =
  Eval.register_prim prims "symbol?" (function
    | [Ast.VSymbol _] -> Ast.VBool true
    | [_] -> Ast.VBool false
    | _ -> raise (Ast.Interp_error (Ast.User_error ("symbol?",
    "bad-args"))));
  Eval.register_prim prims "string?" (function
    | [Ast.VStr _] -> Ast.VBool true
    | [_] -> Ast.VBool false
    | _ -> raise (Ast.Interp_error (Ast.User_error ("string?",
    "bad-args"))));
  Eval.register_prim prims "symbol=?" (function
    | [Ast.VSymbol a; Ast.VSymbol b] -> Ast.VBool (String.equal a b)
    | [_; _] -> Ast.VBool false
    | _ -> raise (Ast.Interp_error (Ast.User_error ("symbol=?",
    "bad-args"))));
  Eval.register_prim prims "symbol->string" (function
    | [Ast.VSymbol s] -> Ast.VStr s
```

```
    | _ -> raise (Ast.Interp_error (Ast.User_error ("symbol->string",
     "bad-args"))));
  Eval.register_prim prims "string->symbol" (function
    | [Ast.VStr s] -> Ast.VSymbol s
    | _ -> raise (Ast.Interp_error (Ast.User_error ("string->symbol",
     "bad-args"))));
  Eval.register_prim prims "type-of" (function
    | [Ast.VSymbol _] -> Ast.VStr "symbol"
    | [Ast.VStr _] -> Ast.VStr "str"
    | [Ast.VInt _] -> Ast.VStr "int"
    | [Ast.VBool _] -> Ast.VStr "bool"
    | [Ast.VNil] -> Ast.VStr "nil"
    | [Ast.VList _] -> Ast.VStr "list"
    | _ -> Ast.VStr "other")

let dialect_tests = [
  ("symbols: quote distinct from string", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let a = Eval.eval_string ~prims env "'hello" in
    let b = Eval.eval_string ~prims env "\"hello\"" in
    (match a, b with
     | Ast.VSymbol "hello", Ast.VStr "hello" -> ()
     | _ -> Alcotest.failf "got %s and %s"
              (Reader.pp_value a) (Reader.pp_value b)));
  ("symbols: quote list heads are symbols", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let v = Eval.eval_string ~prims env "'(lambda (x) x)" in
    (match v with
     | Ast.VList (Ast.VSymbol "lambda" :: _) -> ()
     | other -> Alcotest.failf "head: %s" (Reader.pp_value other)));
  ("symbols: value_to_expr preserves Var vs Lit VStr", `Quick, fun () ->
    let e = Reader.read_one "(lambda (x) \"hi\")" in
    let v = Eval.quote_to_value e in
    (match v with
     | Ast.VList [Ast.VSymbol "lambda"; Ast.VList [Ast.VSymbol "x"];
     Ast.VStr "hi"] -> ()
     | other -> Alcotest.failf "quote_to_value: %s" (Reader.pp_value
     other));
    let e' = Eval.value_to_expr v in
    (match e' with
     | Ast.Lambda (["x"], Ast.Lit (Ast.VStr "hi")) -> ()
     | other -> Alcotest.failf "value_to_expr: %s" (Reader.pp_expr other)));
  ("symbols: gensym returns VSymbol", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let a = Eval.eval_string ~prims env "(gensym)" in
    let b = Eval.eval_string ~prims env "(gensym)" in
    (match a, b with
     | Ast.VSymbol sa, Ast.VSymbol sb when sa <> sb -> ()
     | _ -> Alcotest.failf "gensym: %s %s"
              (Reader.pp_value a) (Reader.pp_value b)));
  ("symbols: prims symbol? string? symbol=? conversions type-of", `Quick,
     fun () ->
    let prims = Eval.make_prim_table () in
    register_symbol_prims prims;
    let env = Env.create () in
    List.iter (fun n -> Env.define env n (Ast.VPrim n))
      ["symbol?"; "string?"; "symbol=?"; "symbol->string";
     "string->symbol"; "type-of"];
    let check expr expect =
      let got = Eval.eval_string ~prims env expr in
      Alcotest.(check string) expr expect (Reader.pp_value got)
    in
    check "(symbol? 'lambda)" "#t";
    check "(symbol? \"lambda\")" "#f";
    check "(string? \"lambda\")" "#t";
    check "(string? 'lambda)" "#f";
    check "(symbol=? 'a 'a)" "#t";
    check "(symbol=? 'a 'b)" "#f";
    check "(symbol=? 'a \"a\")" "#f";
    check "(symbol->string 'hello)" "\"hello\"";
```

```
    check "(string->symbol \"hello\")" "hello";
    check "(type-of 'x)" "\"symbol\"";
    check "(type-of \"x\")" "\"str\"";
  ("define sugar", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    Eval.register_prim prims "+" (function
      | [Ast.VInt a; Ast.VInt b] -> Ast.VInt (Int64.add a b)
      | _ -> raise (Ast.Interp_error (Ast.User_error ("+", "bad-args"))));
    let env = Env.create () in
    Env.define env "+" (Ast.VPrim "+");
    ignore (Eval.eval_string ~prims env "(define (add1 x) (+ x 1))");
    let got = Eval.eval_string ~prims env "(add1 41)" in
    Alcotest.(check string) "42" "42" (Reader.pp_value got));
  ("define docstring trailing", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let v = Eval.eval_string ~prims env "(define d1 1 \"one\")" in
    Alcotest.(check string) "bind" "1" (Reader.pp_value v);
    let got = Eval.eval_string ~prims env "d1" in
    Alcotest.(check string) "lookup" "1" (Reader.pp_value got);
    (match Eval.parse_top_define (Reader.read_one "(define d1 1 \"one\")")
     with
     | Some { name = "d1"; doc = Some "one"; _ } -> ()
     | _ -> Alcotest.fail "parse_top_define trailing"));
  ("define sugar leading docstring", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    Eval.register_prim prims "+" (function
      | [Ast.VInt a; Ast.VInt b] -> Ast.VInt (Int64.add a b)
      | _ -> raise (Ast.Interp_error (Ast.User_error ("+", "bad-args"))));
    let env = Env.create () in
    Env.define env "+" (Ast.VPrim "+");
    ignore (Eval.eval_string ~prims env
              "(define (inc x) \"add one\" (+ x 1))");
    let got = Eval.eval_string ~prims env "(inc 2)" in
    Alcotest.(check string) "3" "3" (Reader.pp_value got);
    (match
       Eval.parse_top_define
         (Reader.read_one "(define (inc x) \"add one\" (+ x 1))")
     with
     | Some { name = "inc"; doc = Some "add one"; body = Ast.Lambda (["x"],
    _) } ->
         ()
     | _ -> Alcotest.fail "parse sugar docstring"));
  ("define lambda leading docstring", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    Eval.register_prim prims "+" (function
      | [Ast.VInt a; Ast.VInt b] -> Ast.VInt (Int64.add a b)
      | _ -> raise (Ast.Interp_error (Ast.User_error ("+", "bad-args"))));
    let env = Env.create () in
    Env.define env "+" (Ast.VPrim "+");
    ignore (Eval.eval_string ~prims env
              "(define f (lambda (x) \"lam\" (+ x 1)))");
    let got = Eval.eval_string ~prims env "(f 10)" in
    Alcotest.(check string) "11" "11" (Reader.pp_value got);
    (match
       Eval.parse_top_define
         (Reader.read_one "(define f (lambda (x) \"lam\" (+ x 1)))")
     with
     | Some { doc = Some "lam"; body = Ast.App (Ast.Var "lambda", _); _ } ->
         ()
     | other ->
         Alcotest.failf "lambda doc peel: %s"
           (match other with None -> "none" | Some _ -> "bad shape")));
  ("export-package trailing doc round-trip (i-1787318456)", `Quick, fun ()
    ->
    (* export_package emits (define <name> <pp_expr body> <pp_value (VStr
    doc)>).
       Verify the re-parsed form recovers body + doc, so the vault
       round-trip preserves the doc column on reseed. *)
    let body_expr = Reader.read_one "(lambda (x) (+ x 1))" in
    let emitted =
      Printf.sprintf "(define f %s %s)"
        (Reader.pp_expr body_expr)
```

```
        (Reader.pp_value (Ast.VStr "adds one to x"))
    in
    (match Eval.parse_top_define (Reader.read_one emitted) with
     | Some { name = "f"; doc = Some "adds one to x";
              body = Ast.App (Ast.Var "lambda", _) } -> ()
     | other ->
         Alcotest.failf "export trailing doc round-trip: %s"
           (match other with
            | None -> "none"
            | Some s -> Reader.pp_expr s.body)));
  ("define lone string body is value not doc", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    ignore (Eval.eval_string ~prims env "(define (hi) \"hello\")");
    let got = Eval.eval_string ~prims env "(hi)" in
    Alcotest.(check string) "hello" "\"hello\"" (Reader.pp_value got);
    (match Eval.parse_top_define (Reader.read_one "(define (hi)
     \"hello\")") with
     | Some { doc = None; _ } -> ()
     | _ -> Alcotest.fail "lone string must not be docstring"));
  ("begin multi-body", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let got = Eval.eval_string ~prims env "(begin 1 2 3)" in
    Alcotest.(check string) "3" "3" (Reader.pp_value got));
  ("match list lambda", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let got =
      Eval.eval_string ~prims env
        "(match '(lambda (x) y)\n\
          ((lambda params body) params)\n\
          (_ 0))"
    in
    match got with
    | Ast.VList [Ast.VSymbol "x"] -> ()
    | other -> Alcotest.failf "match got %s" (Reader.pp_value other));
  ("guard catches user error", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let got =
      Eval.eval_string ~prims env
        "(guard (lambda (e) 99) (error \"boom\"))"
    in
    Alcotest.(check string) "99" "99" (Reader.pp_value got));
  ("guard handler receives error tagged list", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let got =
      Eval.eval_string ~prims env
        "(guard (lambda (e) e) (error \"boom\"))"
    in
    match got with
    | Ast.VList [Ast.VSymbol "error"; Ast.VStr msg; Ast.VSymbol
     "user_error"] ->
        Alcotest.(check string) "msg" "boom" msg
    | other -> Alcotest.failf "guard err shape: %s" (Reader.pp_value
     other));
  ("guard does not catch Budget_exceeded", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    ignore
      (Eval.eval_string ~prims env
         "(define loop (lambda () (loop)))");
    let raised =
      try
        let budget = Eval.make_budget ~limit:50 () in
        ignore
          (Control.run_sync ~prims (fun () ->
              Eval.eval ~budget ~prims:Eval.empty_prims env
                (Reader.read_one
                   "(guard (lambda (e) 0) (loop))")));
        false
```

```
      with
      | Ast.Interp_error Ast.Budget_exceeded -> true
      | _ -> false
    in
    Alcotest.(check bool) "budget escapes guard" true raised);
  ("parse error includes line:col", `Quick, fun () ->
    (match
       try
         ignore (Reader.read_one "(foo");
         None
       with
       | Reader.Reader_error (msg, line, col) -> Some (msg, line, col)
       | Ast.Interp_error (Ast.Parse_error (msg, line, col)) ->
           Some (msg, line, col)
       | _ -> None
     with
     | Some (_msg, line, col) ->
         Alcotest.(check bool) "line positive" true (line >= 1);
         Alcotest.(check bool) "col positive" true (col >= 1);
         (match Ast.interp_error_loc (Ast.Parse_error ("x", line, col)) with
          | Some (l, c) ->
              Alcotest.(check int) "loc line" line l;
              Alcotest.(check int) "loc col" col c
          | None -> Alcotest.fail "interp_error_loc missing")
     | None -> Alcotest.fail "expected parse error with loc"));
  ("unbound var kind and name", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    (match
       try
         ignore (Eval.eval_string ~prims env "missing-name");
         None
       with Ast.Interp_error e -> Some e
     with
     | Some (Ast.Unbound_var name as e) ->
         Alcotest.(check string) "name" "missing-name" name;
         Alcotest.(check string) "kind" "unbound_var"
    (Ast.interp_error_kind e)
     | Some e ->
         Alcotest.failf "wrong error: %s" (Ast.pp_interp_error e)
     | None -> Alcotest.fail "expected Unbound_var"));
  ("error kinds come from the stored kind slot", `Quick, fun () ->
    Alcotest.(check string) "budget" "budget_exceeded"
      (Ast.interp_error_kind Ast.Budget_exceeded);
    Alcotest.(check string) "user-stored-kind" "bad-args"
      (Ast.interp_error_kind (Ast.User_error ("x", "bad-args")));
    Alcotest.(check string) "user-custom-kind" "denied"
      (Ast.interp_error_kind (Ast.User_error ("no", "denied")));
    (* A2: Arity_mismatch and Wrong_arg_count collapse to bad-arity *)
    Alcotest.(check string) "arity" "bad-arity"
      (Ast.interp_error_kind (Ast.Arity_mismatch ("f", 1, 2)));
    Alcotest.(check string) "wrong-arg-count" "bad-arity"
      (Ast.interp_error_kind (Ast.Wrong_arg_count ("f", 1, 2)));
    Alcotest.(check string) "parse" "parse_error"
      (Ast.interp_error_kind (Ast.Parse_error ("bad", 1, 2)));
    Alcotest.(check string) "prim-stored-kind" "not-found"
      (Ast.interp_error_kind (Ast.Prim_error ("get", "missing",
     "not-found")));
    (* pp_interp_error still works with the new arity *)
    Alcotest.(check string) "pp-prim" "prim get: missing"
      (Ast.pp_interp_error (Ast.Prim_error ("get", "missing",
     "not-found")));
    Alcotest.(check string) "pp-user" "boom"
      (Ast.pp_interp_error (Ast.User_error ("boom", "bad-args"))));
]

(* --- public-route gate (Route_gate) ---
   Pure fail-closed gate: infer a handler's effects from the in-memory
   env and check against the public-route allowlist. Unknown / non-function
   handlers are DENIED. No DB, no Lwt -- tests build an env with lambda
   handlers whose bodies mention prims (the bodies are not evaluated at
   define time, so no prim table is needed). *)
let gate_ok env handler =
```

```
  match Route_gate.check_public_handler ~env handler with
  | Ok () -> true
  | Error _ -> false

let gate_tests = [
  ("gate: pure handler allowed", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    ignore (Eval.eval_string ~prims env "(define h (lambda () 1))");
    Alcotest.(check bool) "pure allowed" true (gate_ok env "h"));
  ("gate: write-db handler denied", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    ignore (Eval.eval_string ~prims env "(define h (lambda () (create)))");
    Alcotest.(check bool) "write denied" false (gate_ok env "h"));
  ("gate: read-db handler allowed", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    ignore (Eval.eval_string ~prims env "(define h (lambda () (get)))");
    Alcotest.(check bool) "read allowed" true (gate_ok env "h"));
  ("gate: branch handler denied", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    ignore (Eval.eval_string ~prims env "(define h (lambda () (fork)))");
    Alcotest.(check bool) "branch denied" false (gate_ok env "h"));
  ("gate: unknown handler denied (fail-closed)", `Quick, fun () ->
    let env = Env.create () in
    Alcotest.(check bool) "unknown denied" false (gate_ok env "nope"));
  ("gate: non-function binding denied", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    ignore (Eval.eval_string ~prims env "(define x 5)");
    Alcotest.(check bool) "non-fn denied" false (gate_ok env "x"));
  ("gate: handler calling a write-db sibling denied (env known_fn)",
     `Quick, fun () ->
    (* Cross-reference via the in-memory env: h calls bad; bad writes.
       The known_fn built from env lambdas must resolve bad ? WriteDb. *)
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    ignore (Eval.eval_string ~prims env "(define bad (lambda ()
     (create)))");
    ignore (Eval.eval_string ~prims env "(define h (lambda () (bad)))");
    Alcotest.(check bool) "sibling write denied" false (gate_ok env "h"));
  ("gate: handler calling a read-db sibling allowed", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    ignore (Eval.eval_string ~prims env "(define reader (lambda ()
     (get)))");
    ignore (Eval.eval_string ~prims env "(define h (lambda () (reader)))");
    Alcotest.(check bool) "sibling read allowed" true (gate_ok env "h"));
  ("gate: http-get handler denied (Net effect)", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    ignore (Eval.eval_string ~prims env "(define h (lambda ()
     (http-get)))");
    Alcotest.(check bool) "net denied by default" false (gate_ok env "h"));
  ("gate: macro hiding create is denied after expand", `Quick, fun () ->
    (* BORGE-1785212882: (wq ?) expands to (create ?); gate must see
     WriteDb. *)
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    ignore (Eval.eval_string ~prims env
      "(defmacro wq (t) `(create ,t))");
    ignore (Eval.eval_string ~prims env
      "(define h (lambda () (wq note)))");
    Alcotest.(check bool) "macro write denied" false (gate_ok env "h"));
  ("gate: pure macro still allowed", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    ignore (Eval.eval_string ~prims env
      "(defmacro twice (x) `(+ ,x ,x))");
    ignore (Eval.eval_string ~prims env
      "(define h (lambda () (twice 3)))");
```

```
    Alcotest.(check bool) "pure macro allowed" true (gate_ok env "h"));
]

let contains_sub ~sub s =
  let n = String.length s and m = String.length sub in
  let rec find i =
    if i + m > n then false
    else if String.sub s i m = sub then true
    else find (i + 1)
  in
  m = 0 || find 0

let shadow_tests = [
  ("define cannot shadow VPrim", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    Env.define env "create" (Ast.VPrim "create");
    (match
       try
         ignore (Eval.eval_string ~prims env "(define create (lambda ()
    1))");
         None
       with Ast.Interp_error (Ast.User_error (m, _)) -> Some m
         | exn -> Alcotest.failf "unexpected: %s" (Printexc.to_string exn)
     with
     | Some m ->
         Alcotest.(check bool) "mentions primitive" true
           (contains_sub ~sub:"primitive" m)
     | None -> Alcotest.fail "expected define to reject prim shadow"));
  ("define cannot shadow special form name", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    (match
       try
         ignore (Eval.eval_string ~prims env "(define if (lambda () 1))");
         None
       with Ast.Interp_error (Ast.User_error (m, _)) -> Some m
          | exn -> Alcotest.failf "unexpected: %s" (Printexc.to_string exn)
     with
     | Some m ->
         Alcotest.(check bool) "mentions special form" true
           (contains_sub ~sub:"special form" m)
     | None -> Alcotest.fail "expected define to reject special form"));
  ("define of ordinary name still works", `Quick, fun () ->
    let prims = Eval.make_prim_table () in
    let env = Env.create () in
    let v = Eval.eval_string ~prims env "(define ok (lambda () 42))" in
    match v with
    | Ast.VLambda _ -> ()
    | other -> Alcotest.failf "got %s" (Reader.pp_value other));
]


let () =
  Alcotest.run "pricklypear-lisp" [
    "reader", reader_tests @ reader_list_tests @ reader_completeness_tests
              @ reader_dotted_tests;
    "ast-roundtrip", ast_roundtrip_tests;
    "pretty-print", pretty_print_tests;
    "eval", eval_tests;
    "macros", macro_tests;
    "macro-str", macro_str_tests;
    "eval-eff", eval_eff_tests;
    "list-hof", list_hof_tests;
    "tco", tco_tests;
    "dialect", dialect_tests;
    "gate", gate_tests;
    "shadow", shadow_tests;
  ]
```

249 lisp/test/lazy__thunk__test.ml

```
(** Tests for lisp-interpreter.lazy-env-thunks: VThunk cells, forcing at
    Env.lookup (single observation point), memoization of value AND
    failure, cycle detection, and the explicit (delay)/(force) forms.

    Pure tests -- no services. Per spec borge/lisp-interpreter.borg. *)
open Pricklypear_lisp
open Ast

let eval_str env s = Eval.eval_string ~prims:Eval.empty_prims env s

(* A counting evaluator: forces via Eval like the kernel's delay does,
   but records how many times it ran. *)
let counting_eval (count : int ref) (env : env) (expr : expr) : value =
  incr count;
  let budget = Eval.make_budget () in
  Eval.eval ~budget ~prims:Eval.empty_prims env expr

let delay_force_roundtrip () =
  let env = Env.create () in
  let v = eval_str env "(force (delay ((lambda (x) x) 3)))" in
  Alcotest.(check string) "force(delay)" "3" (Reader.pp_value v)

let force_non_thunk_is_identity () =
  let env = Env.create () in
  let v = eval_str env "(force 41)" in
  Alcotest.(check string) "force non-thunk" "41" (Reader.pp_value v)

let delay_does_not_eval_at_definition () =
  let env = Env.create () in
  (* If delay were eager, the error in the body would raise here. *)
  let v = eval_str env "(delay (error \"boom\" \"test\"))" in
  Alcotest.(check string) "delay is lazy" "#<thunk>" (Reader.pp_value v)

let force_runs_body_once_memoizes () =
  let env = Env.create () in
  let count = ref 0 in
  (* (define t (delay ...)) keeps define EAGER but the thunk lazy.
     Use side-effect-free counting via the OCaml evaluator instead. *)
  Env.define_thunk env "recipe" ~force:(counting_eval count)
    (App (Lambda ([ "x" ], Var "x"), [ Lit (VInt 42L) ]));
  ignore (eval_str env "recipe");
  ignore (eval_str env "recipe");
  let v = eval_str env "recipe" in
  Alcotest.(check string) "forced value" "42" (Reader.pp_value v);
  Alcotest.(check int) "body ran once" 1 !count

let failure_is_memoized () =
  let env = Env.create () in
  let count = ref 0 in
  Env.define_thunk env "bad" ~force:(counting_eval count)
    (App (Var "error", [ Lit (VStr "test"); Lit (VStr "nope") ]));
  let raised_first =
    try ignore (eval_str env "bad"); false
    with Interp_error (User_error ("nope", _)) -> true
  in
  let raised_second =
    try ignore (eval_str env "bad"); false
    with Interp_error (User_error ("nope", _)) -> true
  in
  Alcotest.(check bool) "first force raised" true raised_first;
  Alcotest.(check bool) "second touch replays failure" true raised_second;
  Alcotest.(check int) "body ran once despite failure" 1 !count

let cyclic_force_raises () =
  let env = Env.create () in
  (* self refers to itself: forcing marks Forcing, the inner lookup
     hits Forcing -> cyclic error (not divergence). *)
  Env.define_thunk env "self" ~force:(counting_eval (ref 0)) (Var "self");
  let raised =
    try ignore (eval_str env "self"); false
    with Interp_error (User_error (_, "cyclic-thunk")) -> true
```

```
  in
  Alcotest.(check bool) "cycle detected" true raised

let forcing_resolves_through_fully_linked_env () =
  let env = Env.create () in
  (* recipe is bound BEFORE recipe-fn is defined, and its body calls
     recipe-fn. Strict load would fail (unbound); lazy force after the
     env is fully linked works. This is the rb/*-before-recipe case. *)
  let count = ref 0 in
  Env.define_thunk env "rb/cola" ~force:(counting_eval count)
    (App (Var "recipe", [ Lit (VStr "Cola") ]));
  ignore (eval_str env "(define (recipe name) name)");
  let v = eval_str env "rb/cola" in
  Alcotest.(check string) "resolved via linked env"
    "\"Cola\"" (Reader.pp_value v);
  Alcotest.(check int) "body ran once" 1 !count

let tests = [
  ("delay/force round-trip", `Quick, delay_force_roundtrip);
  ("force on non-thunk is identity", `Quick, force_non_thunk_is_identity);
  ("delay does not eval at definition", `Quick,
     delay_does_not_eval_at_definition);
  ("force runs body once, memoizes", `Quick, force_runs_body_once_memoizes);
  ("failure is memoized", `Quick, failure_is_memoized);
  ("cyclic force raises", `Quick, cyclic_force_raises);
  ("forcing sees fully linked env", `Quick,
     forcing_resolves_through_fully_linked_env);
]

let () = Alcotest.run "lazy_thunk" [ ("lazy-env-thunks", tests) ]
```

250 lisp/test/recursion_test.ml

```
open Pricklypear_lisp

let parse s = Reader.read_one s

let fact_non_tail =
  parse
    "(define fact (lambda (n) (if (= n 0) 1 (* n (fact (- n 1))))))"

let fact_tail =
  parse
    "(define fact-t (lambda (n acc) (if (= n 0) acc (fact-t (- n 1) (* n
     acc)))))"

let fib =
  parse
    "(define fib (lambda (n) (if (< n 2) n (+ (fib (- n 1)) (fib (- n
     2))))))"

let pure_add = parse "(define add1 (lambda (x) (+ x 1)))"

let test_self_recursive () =
  let info = Recursion.analyze_self ~fn_name:"fact" fact_non_tail in
  Alcotest.(check bool) "recursive" true info.is_recursive;
  Alcotest.(check bool) "non-tail" false info.tail_recursive

let test_tail_recursive () =
  let info = Recursion.analyze_self ~fn_name:"fact-t" fact_tail in
  Alcotest.(check bool) "recursive" true info.is_recursive;
  Alcotest.(check bool) "tail" true info.tail_recursive

let test_fib_branching () =
  let info = Recursion.analyze_self ~fn_name:"fib" fib in
  Alcotest.(check bool) "recursive" true info.is_recursive;
  Alcotest.(check int) "two sites" 2 (List.length info.self_sites);
  Alcotest.(check bool) "non-tail" false info.tail_recursive

let test_non_recursive () =
  let info = Recursion.analyze_self ~fn_name:"add1" pure_add in
  Alcotest.(check bool) "not recursive" false info.is_recursive

let test_complexity () =
  Alcotest.(check string) "fact O(n)" "O(n)"
    (Complexity.badge (Complexity.classify ~fn_name:"fact" fact_non_tail));
  Alcotest.(check string) "fib exp" "O(2?)"
    (Complexity.badge (Complexity.classify ~fn_name:"fib" fib));
  Alcotest.(check string) "add1 O(1)" "O(1)"
    (Complexity.badge (Complexity.classify ~fn_name:"add1" pure_add))

let test_explain_peel () =
  let layers = Explain.peel ~fn_name:"fact" fact_non_tail in
  Alcotest.(check bool) "non-empty" true (layers <> []);
  let kinds = List.map (fun (l : Explain.layer) -> l.kind) layers in
  Alcotest.(check bool) "has call or if" true
    (List.exists
       (fun k ->
         k = "if" || String.starts_with ~prefix:"if" k
         || String.starts_with ~prefix:"call:" k)
       kinds)

let test_step_attribution () =
  let env = Env.create () in
  let prims = Eval.make_prim_table () in
  let open Ast in
  let bin name f =
    Eval.register_prim prims name (function
      | [ VInt a; VInt b ] -> VInt (f a b)
      | _ -> raise (Interp_error (User_error (name ^ " bad args",
     "bad-args"))))
  in
  bin "+" Int64.add;
  bin "-" Int64.sub;
```

```
  bin "*" Int64.mul;
  Eval.register_prim prims "=" (function
    | [ VInt a; VInt b ] -> VBool (Int64.equal a b)
    | _ -> VBool false);
  Env.define env "+" (VPrim "+");
  Env.define env "-" (VPrim "-");
  Env.define env "*" (VPrim "*");
  Env.define env "=" (VPrim "=");
  ignore (Eval.eval_string ~prims env
            "(define fact (lambda (n) (if (= n 0) 1 (* n (fact (- n
    1))))))");
  ignore (Eval.eval_string ~prims env "(fact 5)");
  match !Eval.last_budget with
  | None -> Alcotest.fail "no last_budget"
  | Some b ->
      Alcotest.(check bool) "steps>0" true (b.steps > 0);
      Alcotest.(check bool) "prim_calls>0" true (b.prim_calls > 0);
      let fact_steps =
        try Hashtbl.find b.fn_step_counts "fact" with Not_found -> 0
      in
      Alcotest.(check bool) "fact attributed" true (fact_steps > 0)

let () =
  Alcotest.run "recursion-complexity"
    [
      ( "recursion",
        [
          ("self non-tail", `Quick, test_self_recursive);
          ("tail", `Quick, test_tail_recursive);
          ("fib", `Quick, test_fib_branching);
          ("non-rec", `Quick, test_non_recursive);
        ] );
      ( "complexity",
        [ ("classify", `Quick, test_complexity) ] );
      ( "explain",
        [ ("peel", `Quick, test_explain_peel) ] );
      ( "budget",
        [ ("attribution", `Quick, test_step_attribution) ] );
    ]
```

251 lisp/test/stepping_test.ml

```
(** Tests for the remote-eval stepping/trace debugger (borge/
    remote-eval.borg stepping, campaign 7.3): Explain.trace /
    Explain.stepper / breakpoints. PURE tests -- a mock transport stands
    in for the remote prim round trip, no Postgres, no HTTP.

    These verify the three stepping verify-stanzas:
      - a breakpoint on a def/prim pauses local eval mid-expression and
        the data plane (prim calls) still crosses the wire and returns
        (the breakpoint fires ON a Remote_ok/Remote_err event);
      - the server is unaware a breakpoint fired: the mock transport
        observes EXACTLY the calls the fast eval would make -- one per
        proxied prim, never extra, and presentation-stepping never
        re-evals so the wire is never re-crossed;
      - a trace shows local pure steps interleaved with remote prim
        round trips, distinguished ([Local n] vs [Remote_ok]). *)
open Pricklypear_lisp

(* --- harness: a local pure prim table + a proxy-prim env + a mock
   transport that counts wire crossings --- *)

(** A proxy prim bound as VPrim in the env (so eval performs
    [Control.Prim_call] for it, which the trace's proxy-style rim
    forwards via the transport). *)
let bind_proxy (env : Ast.env) (name : string) : unit =
  Env.define env name (Ast.VPrim name)

(** A local pure prim: registered in [local_prims] AND bound as VPrim in
    the env -- the trace handler services it locally (a local step, never
    a wire crossing). *)
let add_local (env : Ast.env) (tbl : Eval.prim_table) (name : string)
    (fn : Ast.value list -> Ast.value) : unit =
  Eval.register_prim tbl name fn;
  Env.define env name (Ast.VPrim name)

let test_env (tbl : Eval.prim_table) : Ast.env =
  let env = Env.create () in
  add_local env tbl "+"
    (function
      | [ Ast.VInt a; Ast.VInt b ] -> Ast.VInt (Int64.add a b)
      | _ -> Ast.VNil);
  bind_proxy env "get";
  env

let mk_transport () =
  let calls = ref 0 in
  let transport name args =
    incr calls;
    match name with
    | "get" -> Ok (Ast.VStr "ok")
    | _ -> Error (Failure ("unexpected prim " ^ name))
  in
  (transport, calls)

(* --- test 1: trace distinguishes local pure steps from remote round trips
     --- *)

let test_trace_interleaved () =
  let tbl = Eval.make_prim_table () in
  let env = test_env tbl in
  let transport, calls = mk_transport () in
  let tr =
    Explain.trace ~local_prims:tbl ~transport ~init_env:env
      ~src:"(+ 1 (get 'user 42))"
  in
  (* the expression crossed the wire exactly once (one get) *)
  Alcotest.(check int) "one remote round trip" 1 !calls;
  (* a Remote_ok record for the get prim *)
  let has_get =
    List.exists
      (function
        | Explain.Remote_ok (n, _) -> n = "get"
```

```
        | _ -> false)
      tr.events
  in
  Alcotest.(check bool) "trace records the get round trip" true has_get;
  (* at least one Local span (the pure (+) work between wire crossings) *)
  let has_local =
    List.exists (function Explain.Local _ -> true | _ -> false) tr.events
  in
  Alcotest.(check bool) "trace records local pure steps" true has_local;
  (* the final value is present *)
  Alcotest.(check bool) "trace result recorded" true (tr.result <> "")

(* --- test 2: breakpoint on a remote prim pauses mid-expression --- *)

let test_breakpoint_on_remote () =
  let tbl = Eval.make_prim_table () in
  let env = test_env tbl in
  let transport, calls = mk_transport () in
  let tr =
    Explain.trace ~local_prims:tbl ~transport ~init_env:env
      ~src:"(+ 1 (get 'user 42))"
  in
  let s = Explain.stepper_of_trace tr in
  Explain.add_breakpoint s (Explain.break_on_remote "get");
  (* continue pauses at the first get round trip *)
  let fired = Explain.continue s in
  Alcotest.(check bool) "breakpoint on get fires" true fired;
  (* the wire was crossed exactly once during trace -- the server saw the
     SAME single call a fast eval would; stepping never re-crossed it *)
  Alcotest.(check int) "server unaware of breakpoint (one call only)" 1
    !calls

(* --- test 3: single-step advances event-by-event without re-eval --- *)

let test_single_step () =
  let tbl = Eval.make_prim_table () in
  let env = test_env tbl in
  let transport, calls = mk_transport () in
  let tr =
    Explain.trace ~local_prims:tbl ~transport ~init_env:env
      ~src:"(+ 1 (get 'user 42))"
  in
  let s = Explain.stepper_of_trace tr in
  (* step through every event one at a time *)
  let n = ref 0 in
  while not (Explain.done_p s) do
    let _ = Explain.step s in
    incr n
  done;
  Alcotest.(check int) "stepped all events"
    (List.length tr.events) !n;
  (* stepping over the trace never made another wire call *)
  Alcotest.(check int) "single-step never re-crosses the wire" 1 !calls

(* --- test 4: a trace with two remote round trips interleaves --- *)

let test_multiple_round_trips () =
  let tbl = Eval.make_prim_table () in
  let env = test_env tbl in
  let calls = ref 0 in
  let transport name args =
    incr calls;
    ignore args;
    match name with
    | "get" | "rows" -> Ok (Ast.VStr "ok")
    | _ -> Error (Failure ("unexpected " ^ name))
  in
  (* two wire crossings, pure arithmetic between them *)
  let tr =
    Explain.trace ~local_prims:tbl ~transport ~init_env:env
      ~src:"(get 'a 1) (+ 1 1) (get 'b 2)"
  in
  Alcotest.(check int) "two round trips" 2 !calls;
```

```
  let remote_count =
    List.fold_left
      (fun acc -> function
        | Explain.Remote_ok _ | Explain.Remote_err _ -> acc + 1
        | Explain.Local _ -> acc)
      0 tr.events
  in
  Alcotest.(check int) "two remote events recorded" 2 remote_count

let () =
  Alcotest.run "remote-stepping"
    [
      ( "stepping",
        [
          Alcotest.test_case "trace interleaves local/remote" `Quick
            test_trace_interleaved;
          Alcotest.test_case "breakpoint on remote prim" `Quick
            test_breakpoint_on_remote;
          Alcotest.test_case "single-step without re-eval" `Quick
            test_single_step;
          Alcotest.test_case "multiple round trips interleave" `Quick
            test_multiple_round_trips;
        ] );
    ]
```

252 llm/lib/loop.ml

```
(** Agent turn loop (borge/agent.borg agent-loop) -- HOST-AGNOSTIC.

    The loop is shared by every host that embeds it: the agent's
    Runtime.t and pp-remote's Remote.t both link this library and supply
    a small [host] record (tool registry, event handlers, tool-body
    eval, serialized prim dispatch). The provider (provider.ml) and the
    builtin read/bash tool bodies (tools.ml) are pure OCaml with zero
    host coupling -- the API key is a value, the HTTP client is a
    library, so any client process can run the LLM turn locally while
    its DB prim calls cross whatever rim the host installs (in-process
    table on the agent, /api/prim proxy on pp-remote). *)

open Pricklypear_lisp
open Ast

let max_turns_default = 8

(** Tool registry entry: a Nopales closure (plugin tool) or a VPrim
    naming a builtin's registered prim. Structural record -- the hosts'
    own registry_entry definitions are compatible with it. *)
type registry_entry = { doc : string; run : Ast.value }

type turn_event =
  | Assistant_text of string
  | Reasoning_text of string  (* provider chain-of-thought chunk: rendered
                                 into the turn's "thinking + tools" zone,
                                 kept OUT of history *)
  | Tool_begin of string * string
  | Tool_end of string * string
  | Turn_error of string

(** Parse a host/plugin-produced value back to host JSON. VStr is
    treated as raw JSON first (dict prims produce JSON strings), else a
    plain string. Moved here from the agent's runtime.ml -- pure, and the
    loop needs it to inspect plugin handler results. *)
let rec json_of_value (v : Ast.value) : Yojson.Safe.t =
  match v with
  | Ast.VStr s -> (
      try Yojson.Safe.from_string s
      with Yojson.Json_error _ -> `String s)
  | VInt i -> `Intlit (Int64.to_string i)
  | VFloat f -> `Float f
  | VBool b -> `Bool b
  | VNil -> `Null
  | VSymbol s -> `String s
  | VList vs -> `List (List.map json_of_value vs)
  | VLambda _ -> `String "<lambda>"
  | VPrim n -> `String ("<prim " ^ n ^ ">")
  | other -> `String (Reader.pp_value other)

(** The host seam: everything the loop needs from its embedding.

    - [tool_order] live tool-name order (register-tool appends);
    - [tool_entry] registry lookup by name;
    - [fire] run event handlers ("input" / "tool_call" /
      "tool_result"), returning their values (a {"block": true,
      "reason": s} handler result short-circuits a tool call);
    - [eval_tool_body] run a plugin tool's Nopales closure with JSON
      args (errors contained to {"error": ...} values);
    - [call_prim] dispatch a builtin tool's registered prim under the
      host's rim + serialization (eval lock on the agent, proxy rim on
      pp-remote). Raises [Interp_error]; the loop contains it. *)
type host = {
  tool_order : unit -> string list;
  tool_entry : string -> registry_entry option;
  fire : string -> Ast.value list -> Ast.value list;
  eval_tool_body : string -> string -> Ast.value;
  call_prim : string -> Ast.value list -> Ast.value;
}

(** Tool defs visible to the provider: builtin schemas plus
```

```
    plugin-registered tools with synthesized permissive schemas
    (v0.0: plugins can't describe params on the wire yet -- spec notes
    alist -> JSON schema mapping as follow-on). *)
let tool_defs (h : host) : Provider.tool_def list =
  List.filter_map
    (fun name ->
      match Tools.schema_of name with
      | Some schema -> Some { Provider.name; description = "builtin " ^
    name; schema }
      | None ->
          (* plugin tool: permissive single-object schema *)
          Some
            {
              Provider.name;
              description = "plugin tool " ^ name;
              schema =
                `Assoc
                  [
                    ("type", `String "object");
                    ("additionalProperties", `Bool true);
                  ];
            })
    (h.tool_order ())

(** Execute one tool call: plugin tool via Nopales closure, builtin via
    its registered VPrim. Routes through [tool_call] / [tool_result]
    events so hooks see builtins too. *)
let exec_tool (h : host) (call : Provider.tool_call) : string =
  let payload =
    Ast.VStr
      (Yojson.Safe.to_string
         (`Assoc
           [
             ("id", `String call.id);
             ("name", `String call.name);
             ("input", call.input);
           ]))
  in
  (* tool_call event: any handler returning {"block": true, "reason": s}
     short-circuits. *)
  let results = h.fire "tool_call" [ payload ] in
  let blocked =
    List.find_map
      (fun v ->
        match json_of_value v with
        | `Assoc kvs -> (
            match List.assoc_opt "block" kvs with
            | Some (`Bool true) ->
                Some
                  (match List.assoc_opt "reason" kvs with
                  | Some (`String r) -> r
                  | _ -> "blocked by plugin")
            | _ -> None)
        | _ -> None)
      results
  in
  let out =
    match blocked with
    | Some reason ->
        Yojson.Safe.to_string
          (`Assoc [ ("error", `String ("blocked: " ^ reason)) ])
    | None ->
        (match h.tool_entry call.name with
         | None ->
             Yojson.Safe.to_string
               (`Assoc [ ("error", `String ("no tool named " ^ call.name))
    ])
         | Some entry -> (
             let args_json = Yojson.Safe.to_string call.input in
             match entry.run with
             | Ast.VPrim prim_name ->
                 (* builtin: call the prim directly through the host rim *)
                 (try
```

```
                    match h.call_prim prim_name [ Ast.VStr args_json ] with
                    | Ast.VStr s -> s
                    | other -> Reader.pp_value other
                  with Ast.Interp_error ie ->
                    Yojson.Safe.to_string
                      (`Assoc [ ("error", `String (Ast.pp_interp_error ie)) ]))
            | Ast.VLambda _ ->
                (match h.eval_tool_body call.name args_json with
                 | Ast.VStr s -> s
                 | other -> Reader.pp_value other)
            | _ ->
                Yojson.Safe.to_string
                  (`Assoc [ ("error", `String "malformed tool entry") ])))
  in
  ignore
    (h.fire "tool_result"
       [ Ast.VStr
           (Yojson.Safe.to_string
              (`Assoc
                [ ("id", `String call.id); ("name", `String call.name);
                  ("result", `String out) ])) ]);
  out

(** One user prompt -> full agent run. Appends user/assistant/tool
    messages into [history] (mutable stack). *)
let run ~(env : Eio_unix.Stdenv.base) ~sw ~(host : host)
    ~(cfg : Provider.cfg) ~(complete :
        env:Eio_unix.Stdenv.base ->
        sw:Eio.Switch.t ->
        cfg:Provider.cfg ->
        messages:Provider.message list ->
        tools:Provider.tool_def list ->
        on_delta:(Provider.delta -> unit) ->
        (Provider.reply, string) Result.t)
    ?(max_turns = max_turns_default)
    ~(on_event : turn_event -> unit) (history : Provider.message list ref)
    (prompt : string) : (unit, string) Result.t =
  history := !history @ [ { Provider.role = "user"; content = `String
    prompt; tool_call_id = None; tool_calls = None } ];
  (* input + before_turn events *)
  ignore (host.fire "input" [ Ast.VStr prompt ]);
  let rec go turn =
    if turn > max_turns then Error "max turns exceeded"
    else
      match
        complete ~env ~sw ~cfg ~messages:!history ~tools:(tool_defs host)
          ~on_delta:(function
            | Provider.Reasoning s -> on_event (Reasoning_text s)
            | Provider.Text s -> on_event (Assistant_text s)
            | Provider.Done -> ())
      with
      | Error e -> Error e
      | Ok reply ->
          (* Replay the assistant turn in OPENAI MESSAGE SHAPE: content is
    the
             raw message's content field (never the whole message object --
             nesting it here produced provider 400s on turn 2), tool_calls
             replayed verbatim so following tool-role messages pair up. *)
          let raw_fields =
            match reply.raw with `Assoc fields -> fields | _ -> []
          in
          let content =
            match List.assoc_opt "content" raw_fields with
            | Some ((`String _ | `List _) as c) -> c
            | Some `Null when reply.text = "" -> `Null
            | Some `Null -> `String reply.text
            | None -> `String reply.text
            | Some _ -> `String reply.text
          in
          let tool_calls = List.assoc_opt "tool_calls" raw_fields in
          history :=
            !history
```

```
            @ [ { role = "assistant"; content; tool_call_id = None;
    tool_calls } ];
          (match reply.calls with
           | [] -> Ok ()
           | calls ->
               List.iter
                 (fun (c : Provider.tool_call) ->
                   on_event (Tool_begin (c.name, Yojson.Safe.to_string
    c.input));
                   let out = exec_tool host c in
                   on_event (Tool_end (c.name, out));
                   (* OpenAI tool-result message: role=tool, tool_call_id
                      routes the result to the call that produced it,
                      content is the plain tool output string. *)
                   history :=
                     !history
                     @ [
                         {
                           role = "tool";
                           content = `String out;
                           tool_call_id = Some c.id;
                           tool_calls = None;
                         };
                       ])
                 calls;
               go (turn + 1))
  in
  go 1
```

253 llm/lib/provider.ml

```
(** Provider abstraction (borge/agent.borg agent-loop).

    Wire-shape-neutral message model + per-provider HTTP. v0 provider:
    OpenAI-compatible chat completions (covers synthetic.new and every
    gateway we use, Anthropic included via its compat endpoint). Accepts
    [mock_base_url] so tests run against a local h1 server; no curl
     anywhere. *)

type message = {
  role : string;             (* "user" | "assistant" | "tool" *)
  content : Yojson.Safe.t;   (* string | null | content-part array; NEVER
     the
                                whole assistant message object *)
  tool_call_id : string option;  (* OpenAI tool-result routing; None for
     non-tool msgs *)
  tool_calls : Yojson.Safe.t option;  (* replayed verbatim on assistant
     history
                                         entries: OpenAI requires an
     assistant
                                         message's tool_calls when tool-role
                                         messages follow it *)
}

type tool_def = {
  name : string;
  description : string;
  schema : Yojson.Safe.t;    (* JSON schema of parameters *)
}

type tool_call = { id : string; name : string; input : Yojson.Safe.t }

type delta = Text of string | Done | Reasoning of string
(* Reasoning: GLM/DeepSeek-class models stream their chain-of-thought as
   separate `reasoning_content` deltas -- except Synthetic's gateway, which
   emits plain `reasoning` deltas (observed on hf:zai-org/GLM-4.7-Flash,
   2026-08 wire probe: 228 `reasoning` chunks, zero `reasoning_content`).
   Both keys are accepted everywhere reasoning is read. The surface renders
   it into the collapsible "thinking + tools" zone instead of the reply
   bubble. *)

type reply = {
  text : string;
  calls : tool_call list;
  stop_reason : string;
  raw : Yojson.Safe.t;       (* assistant message in provider shape,
                                for appending to history verbatim *)
}

type cfg = {
  base_url : string;
  api_key : string;
  model : string;
  system : string;
}

(* The OpenAI wire codec (types, tool_def_to_json, tool-call parsing,
   XML-in-content fallback, GLM reasoning_content handling) is shared
   with the image agent via common/lib/llm_codec.ml. This module owns
   only the Eio transport + the host-specific [message]/[reply]
   shapes (the standalone agent round-trips raw message JSON in
   history, so [reply.raw] is the assistant message object). *)
module Llm_codec = Pricklypear_common.Llm_codec

(* HTTP/1.1 POST written directly on an Eio flow (plain or TLS).
   Httpun_eio.Client's gluten runtime insists on a socket-typed flow,
   which TLS flows can't satisfy; a 1.1 request/response with
   `connection: close` is simple enough to do honestly by hand.
   Handles content-length and chunked framing. Returns the raw response
   body string on 2xx (error string otherwise); callers parse it as
   JSON ([post_json]) or as SSE event lines ([Openai.complete]). *)
let post_body ~(env : Eio_unix.Stdenv.base) ~sw ~(cfg : cfg) ~path
```

```
    ~extra_headers ~body : (string, string) Result.t =
  let open Eio in
  let host, port, tls =
    match String.split_on_char '/' cfg.base_url with
    | "https:" :: "" :: hostport :: _ ->
        let h, p =
          match String.split_on_char ':' hostport with
          | [ h; p ] -> (h, int_of_string p)
          | _ -> (hostport, 443)
        in
        (h, p, true)
    | "http:" :: "" :: hostport :: _ ->
        let h, p =
          match String.split_on_char ':' hostport with
          | [ h; p ] -> (h, int_of_string p)
          | _ -> (hostport, 80)
        in
        (h, p, false)
    | _ -> failwith ("bad base_url: " ^ cfg.base_url)
  in
  let net = (Stdenv.net env :> [ `Generic ] Net.ty Std.r) in
  let addr = Net.getaddrinfo_stream ~service:(string_of_int port) net host
     |> List.hd in
  let raw = Net.connect ~sw net addr in
  let flow : Flow.two_way_ty Std.r =
    if tls then
      let auth = Result.get_ok (Ca_certs_nss.authenticator ()) in
      let tls_config = Result.get_ok (Tls.Config.client ~authenticator:auth
     ()) in
      (Tls_eio.client_of_flow tls_config
         ~host:(Domain_name.host_exn (Domain_name.of_string_exn host))
         raw
       : Tls_eio.t :> Flow.two_way_ty Std.r)
    else (raw :> Flow.two_way_ty Std.r)
  in
  let request_text =
    let hdrs =
      String.concat "\r\n"
        (List.map (fun (k, v) -> k ^ ": " ^ v)
           ([ ("host", host); ("content-type", "application/json");
              ("content-length", string_of_int (String.length body));
              ("connection", "close") ]
            @ extra_headers))
    in
    Printf.sprintf "POST %s HTTP/1.1\r\n%s\r\n\r\n%s" path hdrs body
  in
  Flow.copy_string request_text flow;
  let br = Buf_read.of_flow ~initial_size:4096 ~max_size:(20 * 1024 * 1024)
     flow in
  let line () = String.trim (Buf_read.line br) in
  let status_line = line () in
  let code =
    match String.split_on_char ' ' status_line with
    | _http :: c :: _ -> (try int_of_string c with _ -> 0)
    | _ -> 0
  in
  let rec read_headers acc =
    let l = line () in
    if l = "" then List.rev acc
    else
      (match String.index_opt l ':' with
       | Some i ->
           read_headers
             ((String.lowercase_ascii (String.sub l 0 i),
               String.trim (String.sub l (i + 1) (String.length l - i -
     1))) :: acc)
       | None -> read_headers acc)
  in
  let headers = read_headers [] in
  let body =
    match List.assoc_opt "transfer-encoding" headers with
    | Some te when String.lowercase_ascii te = "chunked" ->
        let buf = Buffer.create 8192 in
```

```
        let rec chunks () =
          let size_line = line () in
          match int_of_string_opt ("0x" ^ size_line) with
          | Some 0 -> ignore (line ()) (* trailing CRLF *)
          | Some n ->
              Buffer.add_string buf (Buf_read.take n br);
              ignore (line ()); (* CRLF after chunk *)
              chunks ()
          | None -> ()
        in
        chunks ();
        Buffer.contents buf
    | _ ->
        (match List.assoc_opt "content-length" headers with
         | Some n -> Buf_read.take (int_of_string n) br
         | None -> Buf_read.take_all br)
  in
  (try Flow.shutdown flow `All with _ -> ());
  if code >= 200 && code < 300 then Ok body
  else Error (Printf.sprintf "HTTP %d: %s" code body)

(* Convenience wrapper: POST and parse the response as JSON. *)
let post_json ~(env : Eio_unix.Stdenv.base) ~sw ~(cfg : cfg) ~path
    ~extra_headers ~body : (Yojson.Safe.t, string) Result.t =
  match post_body ~env ~sw ~cfg ~path ~extra_headers ~body with
  | Error e -> Error e
  | Ok b ->
      (try Ok (Yojson.Safe.from_string b)
       with Yojson.Json_error e -> Error ("bad json: " ^ e))

(* ---- OpenAI-compatible (chat completions) ---- *)

module Openai = struct
  let tool_json (t : tool_def) : Yojson.Safe.t =
    `Assoc
      [
        ("type", `String "function");
        ( "function",
          `Assoc
            [
              ("name", `String t.name);
              ("description", `String t.description);
              ("parameters", t.schema);
            ] );
      ]

  let request_body ~(cfg : cfg) ~(messages : message list)
      ~(tools : tool_def list) ~(stream : bool) : string =
    let msgs =
      `Assoc [ ("role", `String "system"); ("content", `String cfg.system) ]
      :: List.map
           (fun m ->
             `Assoc
               (("role", `String m.role) :: ("content", m.content)
                :: (match m.tool_call_id with
                    | Some id -> [ ("tool_call_id", `String id) ]
                    | None -> [])
                @ (match m.tool_calls with
                   | Some tcs -> [ ("tool_calls", tcs) ]
                   | None -> [])))
           messages
    in
    Yojson.Safe.to_string
      (`Assoc
        ([
           ("model", `String cfg.model);
           ("messages", `List msgs);
         ]
         @ (if stream then [ ("stream", `Bool true) ] else [])
         @ (match tools with [] -> [] | ts -> [ ("tools", `List (List.map
     tool_json ts)) ])))

  let parse_reply (j : Yojson.Safe.t) : reply =
```

```
  let open Yojson.Safe.Util in
  let choice = j |> member "choices" |> index 0 in
  let msg = choice |> member "message" in
  let stop_reason =
    choice |> member "finish_reason" |> to_string_option
    |> Option.value ~default:"stop"
  in
  (* Tool calls: shared parser handles the API [tool_calls] field AND
     the XML-in-content fallback (GLM-class models that dump tool
     calls as XML tags inside [content] instead of using the API
     field). The image agent has bled for this; the standalone agent
     inherits it. *)
  let api_tcs = Llm_codec.parse_tool_calls_from_msg msg in
  let tcs, content_for_xml =
    match api_tcs with
    | _ :: _ -> (api_tcs, None)
    | [] ->
        let content_opt =
          match msg |> member "content" |> to_string_option with
          | Some c when String.trim c <> "" -> Some c
          | _ -> None
        in
        (match content_opt with
         | Some c ->
             let xml_tcs = Llm_codec.parse_xml_tool_calls c in
             (xml_tcs, if xml_tcs = [] then None else Some "")
         | None -> ([], None))
  in
  let calls =
    List.map
      (fun (tc : Llm_codec.tool_call) ->
        let input =
          match Yojson.Safe.from_string tc.arguments with
          | json -> json
          | exception _ -> `Assoc []
        in
        { id = tc.id; name = tc.name; input })
      tcs
  in
  let text =
    (* Prefer explicit content; GLM-class models sometimes fill
       reasoning_content and leave content null -- fall back to it so a
       pure-text reply is not lost. *)
    let c =
      match content_for_xml with
      | Some s -> s
      | None -> (
          match msg |> member "content" |> to_string_option with
          | Some s -> s
          | None -> "")
    in
    if c <> "" then c
    else
      (match msg |> member "reasoning_content" |> to_string_option with
       | Some s -> s
       | None ->
           (match msg |> member "reasoning" |> to_string_option with
            | Some s -> s
            | None -> ""))
  in
  { text; calls; stop_reason; raw = msg }

(* Reconstruct a [reply] from the list of SSE [data:] payloads in a
   streaming response. [on_delta (Text s)] is fired per content chunk
   as it is decoded; [Done] is fired once after the stream ends.
   Tool calls arrive as fragments: the first chunk for an [index]
   carries [id] + [function.name], later chunks carry
   [function.arguments] pieces that must be concatenated. We
   accumulate per-index, then build a synthetic assistant [message]
   JSON object and run it through the shared [parse_reply] so the
   XML-fallback / reasoning_content logic is reused unchanged. This
   is a pure function so it can be unit-tested without a socket. *)
let parse_stream_lines (lines : string list) ~(on_delta : delta -> unit)
```

```
    : reply =
  let open Yojson.Safe.Util in
  let text_buf = Buffer.create 128 in
  let reasoning_buf = Buffer.create 128 in
  (* index -> accumulated tool-call fragment {"id";"name";arguments buf} *)
  let tool_bufs : (int, string * string * Buffer.t) Hashtbl.t = Hashtbl.create 4 in
  let finish = ref "stop" in
  List.iter
    (fun data ->
      if data <> "[DONE]" then
        (try
           let json = Yojson.Safe.from_string data in
           match json |> member "choices" |> to_list with
           | choice :: _ ->
               (match choice |> member "finish_reason" |> to_string_option with
                | Some f when f <> "" -> finish := f
                | _ -> ());
               let delta = choice |> member "delta" in
               let reasoning_chunk =
                 match delta |> member "reasoning_content" |> to_string_option with
                 | Some s when s <> "" -> Some s
                 | _ ->
                     (* Synthetic OpenAI-compat streams thinking as
                        plain "reasoning" deltas. *)
                     (match delta |> member "reasoning" |> to_string_option with
                      | Some s when s <> "" -> Some s
                      | _ -> None)
               in
               (match reasoning_chunk with
                | Some s ->
                    Buffer.add_string reasoning_buf s;
                    on_delta (Reasoning s)
                | None -> ());
               (match delta |> member "content" |> to_string_option with
                | Some s when s <> "" ->
                    Buffer.add_string text_buf s;
                    on_delta (Text s)
                | _ -> ());
               (match delta |> member "tool_calls" with
                | `List tcs ->
                    List.iter
                      (fun tc ->
                        let idx =
                          match tc |> member "index" with
                          | `Int i -> i
                          | `Intlit s -> (try int_of_string s with _ -> 0)
                          | _ -> 0
                        in
                        let id', name', args =
                          match Hashtbl.find_opt tool_bufs idx with
                          | Some e -> e
                          | None ->
                              let e = ("", "", Buffer.create 64) in
                              Hashtbl.replace tool_bufs idx e;
                              e
                        in
                        let id'' =
                          match tc |> member "id" |> to_string_option with
                          | Some i when i <> "" -> i
                          | _ -> id'
                        in
                        let fn = tc |> member "function" in
                        let name'' =
                          match fn |> member "name" |> to_string_option with
                          | Some n when n <> "" -> n
                          | _ -> name'
                        in
```

```
                        (match fn |> member "arguments" |>
     to_string_option with
                         | Some a when a <> "" -> Buffer.add_string args a
                         | _ -> ());
                        Hashtbl.replace tool_bufs idx (id'', name'',
     args))
                    tcs
                | _ -> ())
          | [] -> ()
        with _ -> ()) (* malformed data line -> skip, lenient *))
      lines;
    let text = Buffer.contents text_buf in
    let tool_calls =
      Hashtbl.fold
        (fun _ (id, name, args) acc ->
          if name = "" then acc
          else
            {
              id = (if id = "" then "tool-call" else id);
              name;
              input =
                (match Yojson.Safe.from_string (Buffer.contents args) with
                 | j -> j
                 | exception _ -> `Assoc []);
            }
            :: acc)
        tool_bufs []
    in
    (* Synthetic assistant message, fed back through [parse_reply] so the
       shared XML/GLM fallbacks and [raw] history shape are preserved. *)
    let reasoning = Buffer.contents reasoning_buf in
    let msg =
      `Assoc
        ([ ("role", `String "assistant"); ("content", `String text) ]
        @ (if reasoning = "" then []
           else [ ("reasoning_content", `String reasoning) ])
        @ (if tool_calls = [] then []
           else
             [
               ( "tool_calls",
                 `List
                   (List.map
                      (fun (tc : tool_call) ->
                        `Assoc
                          [
                            ("id", `String tc.id);
                            ("type", `String "function");
                            ( "function",
                              `Assoc
                                [
                                  ("name", `String tc.name);
                                  ( "arguments",
                                    `String (Yojson.Safe.to_string
     tc.input) );
                                ] );
                          ])
                      tool_calls) );
             ]))
    in
    let response =
      `Assoc
        [
          ( "choices",
            `List
              [
                `Assoc
                  [ ("message", msg); ("finish_reason", `String !finish) ];
              ] );
        ]
    in
    let r = parse_reply response in
    on_delta Done;
    r
```

```
418
419   (* Parse one SSE [data:] line from the raw body: returns the payload
420      after the "data: " prefix. Empty/comment lines yield None. *)
421   let sse_data_payload (line : string) : string option =
422     let line = String.trim line in
423     if String.length line >= 5 && String.sub line 0 5 = "data:" then
424       let rest = String.trim (String.sub line 5 (String.length line - 5)) in
425       if rest = "" then None else Some rest
426     else None
427
428   let complete ~(env : Eio_unix.Stdenv.base) ~sw ~(cfg : cfg)
429       ~(messages : message list) ~(tools : tool_def list)
430       ~(on_delta : delta -> unit) =
431     match
432       post_body ~env ~sw ~cfg ~path:"/v1/chat/completions"
433         ~extra_headers:[ ("authorization", "Bearer " ^ cfg.api_key) ]
434         ~body:(request_body ~cfg ~messages ~tools ~stream:true)
435     with
436     | Error e -> Error e
437     | Ok body ->
438         (* v0 streaming: the whole de-chunked body is read, then SSE
439            lines are dispatched (parsing-incremental but not
440            network-incremental -- acceptable v0; matches the image's
441            line-loop shape). *)
442         let data_lines =
443           String.split_on_char '\n' body
444           |> List.filter_map sse_data_payload
445         in
446         Ok (parse_stream_lines data_lines ~on_delta)
447 end
```

254 llm/lib/tools.ml

```
(** Builtin tools (borge/agent.borg agent-loop): read, bash.
    Shapes follow pi's baseline so prompts/habits transfer. Builtin
    bodies are OCaml; plugin hooks (tool_call / tool_result events)
    still fire around them, so plugins can block or rewrite.

    HOST-AGNOSTIC: tool bodies take an explicit [resolve_path] (each
    host's path sandbox) and registration goes through an explicit
    [register] callback -- no Runtime.t / Remote.t coupling, so the
    agent and pp-remote install the same builtin set. *)

open Yojson.Safe.Util

let max_read_bytes = 100_000 (* chars; no offset/limit in v0.0 *)

let tool_read ~(resolve_path : string -> string) (args : Yojson.Safe.t) :
    string =
  let path = args |> member "path" |> to_string in
  let resolved = resolve_path path in
  match
    (try
       let ic = open_in_bin resolved in
       let n = min (in_channel_length ic) max_read_bytes in
       let s = really_input_string ic n in
       close_in ic;
       Ok s
     with Sys_error e -> Error e)
  with
  | Error e -> Yojson.Safe.to_string (`Assoc [ ("error", `String e) ])
  | Ok s ->
      Yojson.Safe.to_string
        (`Assoc [ ("path", `String resolved); ("content", `String s) ])

let tool_bash (args : Yojson.Safe.t) : string =
  let cmd = args |> member "command" |> to_string in
  let ic = Unix.open_process_in (cmd ^ " 2>&1") in
  let buf = Buffer.create 4096 in
  (try
     while true do
       Buffer.add_channel buf ic 4096
     done
   with End_of_file -> ());
  let code = match Unix.close_process_in ic with Unix.WEXITED c -> c | _ ->
    -1 in
  let out = Buffer.contents buf in
  let max_out = 50 * 1024 in
  let out, truncated =
    if String.length out > max_out then (String.sub out 0 max_out, true)
    else (out, false)
  in
  Yojson.Safe.to_string
    (`Assoc
      [ ("exit", `Int code); ("out", `String out); ("truncated", `Bool
    truncated) ])

(* JSON schemas in provider-neutral shape (JSON schema). *)
let read_schema : Yojson.Safe.t =
  `Assoc
    [
      ("type", `String "object");
      ( "properties",
        `Assoc
          [
            ( "path",
              `Assoc
                [
                  ("type", `String "string");
                  ("description", `String "Path, relative to the working
    directory");
                ] );
          ] );
      ("required", `List [ `String "path" ]);
```

```
    ]

let bash_schema : Yojson.Safe.t =
  `Assoc
    [
      ("type", `String "object");
      ( "properties",
        `Assoc
          [
            ( "command",
              `Assoc
                [
                  ("type", `String "string");
                  ("description", `String "Shell command to run");
                ] );
          ] );
      ("required", `List [ `String "command" ]);
    ]

let install ~(resolve_path : string -> string)
    ~(register : string -> (Yojson.Safe.t -> string) -> unit) : unit =
  (* [register] is the host's builtin-tool registration (agent:
     Runtime.register_builtin_tool; pp-remote: Remote.register_builtin_
     tool) -- same registry shape on both, so plugin hooks see builtins
     identically. *)
  register "read" (tool_read ~resolve_path);
  register "bash" tool_bash

let schema_of (_name : string) : Yojson.Safe.t option =
  match _name with
  | "read" -> Some read_schema
  | "bash" -> Some bash_schema
  | _ -> None
```

255 migrations/001_branches.sql

```
-- Pricklypear migration 001: the code store.
--
-- Branched code (functions, types, view bindings) plus a lightweight
-- sessions table for process bookkeeping. Applied by the image on boot
-- (and by scripts/dev.sh / deploy hooks). Tracked in schema_migrations.
--
-- Per spec [storage.schema]. NOTE: NO Postgres INHERITANCE anywhere
-- (see storage.no-inheritance -- branches are an OVERRIDE primitive, not
-- a UNION primitive; we use plain tables with branch_id FK and resolve
-- overrides in queries).

CREATE TABLE IF NOT EXISTS branches (
  id          uuid PRIMARY KEY,
  parent_id   uuid REFERENCES branches(id),
  owner       text NOT NULL,
  name        text NOT NULL,
  created_at  timestamptz NOT NULL DEFAULT now(),
  last_synced timestamptz,
  UNIQUE (owner, name)
);

-- The 'main' branch is always present; reseed idempotently.
INSERT INTO branches (id, parent_id, owner, name, created_at)
SELECT '00000000-0000-0000-0000-000000000001', NULL, 'system', 'main', now()
WHERE NOT EXISTS (
  SELECT 1 FROM branches WHERE owner = 'system' AND name = 'main'
);

CREATE TABLE IF NOT EXISTS function_defs (
  branch_id   uuid NOT NULL REFERENCES branches(id) ON DELETE CASCADE,
  name        text NOT NULL,
  body        jsonb NOT NULL,  -- serialized Lisp AST (see lisp/lib/ast.ml)
  doc         text,
  author      text NOT NULL,
  updated_at  timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (branch_id, name)
);

CREATE TABLE IF NOT EXISTS type_defs (
  branch_id   uuid NOT NULL REFERENCES branches(id) ON DELETE CASCADE,
  name        text NOT NULL,
  fields      jsonb NOT NULL,  -- field declarations
  migrated    boolean NOT NULL DEFAULT false,
  schema_vers integer NOT NULL DEFAULT 1,
  author      text NOT NULL,
  updated_at  timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (branch_id, name)
);

CREATE TABLE IF NOT EXISTS view_bindings (
  branch_id   uuid NOT NULL REFERENCES branches(id) ON DELETE CASCADE,
  route       text NOT NULL,    -- e.g. "/inspect/:id"
  handler     text NOT NULL,    -- function name in function_defs
  PRIMARY KEY (branch_id, route)
);

-- sessions = optional process bookkeeping (health/uptime style).
-- Auth login sessions live in auth_sessions (002_auth.sql).
CREATE TABLE IF NOT EXISTS sessions (
  id          uuid PRIMARY KEY,
  user_id     text NOT NULL,
  branch_id   uuid NOT NULL REFERENCES branches(id),
  pid         integer,             -- image process pid when recorded
  boot_mode   text NOT NULL,       -- 'on-connect' | 'always-on'
  health      text NOT NULL DEFAULT 'unknown',  -- see
    Protocol.health_status
  started_at  timestamptz NOT NULL DEFAULT now(),
  last_seen   timestamptz
);

CREATE INDEX IF NOT EXISTS sessions_user_idx ON sessions (user_id);
```

```
CREATE INDEX IF NOT EXISTS sessions_health_idx ON sessions (health);

CREATE TABLE IF NOT EXISTS schema_migrations (
  version     integer PRIMARY KEY,
  applied_at timestamptz NOT NULL DEFAULT now(),
  description text
);

INSERT INTO schema_migrations (version, description)
SELECT 1, 'initial branches/code-store schema'
WHERE NOT EXISTS (SELECT 1 FROM schema_migrations WHERE version = 1);
```

256 migrations/002_auth.sql

```
-- Pricklypear migration 002: the auth tables.
--
-- Applied by the single-image process (same migration runner as 001).
-- Auth is in-process in v1; the optional pricklypear_auth role and
-- system_keys grants below are defense-in-depth leftovers from the
-- earlier multi-process sketch -- harmless if unused.
--
-- KNOWN RISK: a DB dump that includes system_keys (or the app role that
-- can read it) can forge session tokens. Accepted for personal/small-team
-- scope. HARDENING: keep signing material out of shared dumps.

CREATE TABLE IF NOT EXISTS users (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  username    text NOT NULL UNIQUE,
  is_admin    boolean NOT NULL DEFAULT false,
  created_at  timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE IF NOT EXISTS credentials (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  kind        text NOT NULL,        -- 'ssh_key' | 'password' | 'web_token'
    | 'api_key'
  public_data jsonb,                -- SSH key material for verification;
    null for password
  secret_data jsonb,                -- argon2 hash for password; reference
    for api_key
  created_at  timestamptz NOT NULL DEFAULT now(),
  last_used_at timestamptz
);

CREATE INDEX IF NOT EXISTS credentials_user_idx ON credentials (user_id);
CREATE INDEX IF NOT EXISTS credentials_kind_idx ON credentials (kind);

-- auth_sessions: browser/API login sessions (token hash + expiry).
-- Not the same as the process `sessions` table in 001_branches.sql.
CREATE TABLE IF NOT EXISTS auth_sessions (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),           -- the
    token's opaque id
  user_id     uuid NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  token_hash  text NOT NULL,              -- HMAC of the token; lookup
    target
  expires_at  timestamptz NOT NULL,
  created_at  timestamptz NOT NULL DEFAULT now(),
  revoked_at  timestamptz
);

CREATE INDEX IF NOT EXISTS auth_sessions_user_idx ON auth_sessions
    (user_id);
CREATE INDEX IF NOT EXISTS auth_sessions_hash_idx ON auth_sessions
    (token_hash);

CREATE TABLE IF NOT EXISTS auth_log (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     text,                       -- NULL if unknown/failed
  kind        text NOT NULL,               -- 'ssh_key' | 'password' |
    'mint' | 'verify_token'
  success     boolean NOT NULL,
  source_ip   text,
  happened_at timestamptz NOT NULL DEFAULT now()
);

CREATE INDEX IF NOT EXISTS auth_log_user_idx ON auth_log (user_id);
CREATE INDEX IF NOT EXISTS auth_log_kind_idx ON auth_log (kind);

CREATE TABLE IF NOT EXISTS system_keys (
  id          text PRIMARY KEY,           -- 'session_signing'
  key_data    bytea NOT NULL,              -- the HMAC signing key, raw
  created_at  timestamptz NOT NULL DEFAULT now()
);

```

```
-- Optional role isolation (historical multi-process sketch).
-- Single-image v1 typically uses the main app DB role; this grant is
-- retained for deployments that still create pricklypear_auth.
DO $$
BEGIN
  IF NOT EXISTS (SELECT 1 FROM pg_roles WHERE rolname = 'pricklypear_auth')
     THEN
    CREATE ROLE pricklypear_auth LOGIN PASSWORD 'change-me-on-prod';
  END IF;
END $$;

GRANT SELECT, INSERT, UPDATE ON users, credentials, auth_sessions,
                             auth_log, system_keys TO pricklypear_auth;
GRANT USAGE ON ALL SEQUENCES IN SCHEMA public TO pricklypear_auth;

INSERT INTO schema_migrations (version, description)
SELECT 2, 'auth tables (users, credentials, auth_sessions, auth_log,
     system_keys)'
WHERE NOT EXISTS (SELECT 1 FROM schema_migrations WHERE version = 2);
```

257 migrations/003_data.sql

```
-- Pricklypear migration 003: generic data row storage.
--
-- Per spec [storage.types]: user-declared types in Lisp (registered via
-- (deftype 'name fields...)) are stored as type_defs (migration 001).
-- Actual data rows of those types live HERE in `data_rows`.
--
-- Data rows are shared across branches (per spec [branching.data]):
-- fork/merge only copies CODE (functions/types/views), not data.
-- All branches see the SAME data rows. Writes go straight to main;
-- reads don't need the recursive-CTE branch walk.
--
-- Design choice for v1: rows are stored as a single jsonb {fields}
-- document per row, keyed by (type_name, id). This avoids generating
-- one PG table per Lisp-declared type (which would require DDL at
-- runtime and break the additive-migrations_only rule). Future work:
-- if a type is hot enough to warrant its own table, a migration can
-- hoist it (the Lisp layer wouldn't change).
--
-- Per spec [workflow.conventions]: SERIALIZABLE transactions by
-- default for mutations. Additive-only migration. No ORM.

CREATE TABLE IF NOT EXISTS data_rows (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  type_name   text NOT NULL,         -- Lisp-level type, e.g. 'user', 'post'
  fields      jsonb NOT NULL,        -- {field_name: value, ...} as JSON
  author      text NOT NULL,         -- session username that wrote it
  created_at  timestamptz NOT NULL DEFAULT now(),
  updated_at  timestamptz NOT NULL DEFAULT now()
);

-- Branchable lookups: by type, by (type, field) value.
CREATE INDEX IF NOT EXISTS data_rows_type_idx ON data_rows (type_name);
CREATE INDEX IF NOT EXISTS data_rows_fields_gin ON data_rows USING gin
    (fields);

-- A "type registry": lightweight bookkeeping for the Lisp-level type
-- defs that have been INSTANTIATED (so (list 'user) knows the type
-- exists). type_defs in 001 is the source of truth for type DECLARATION;
-- this is just a fast (name -> exists) cache, same data.
-- We don't add a separate table; type_defs already has it.

INSERT INTO schema_migrations (version, description)
SELECT 3, 'generic data row storage (shared across branches)'
WHERE NOT EXISTS (SELECT 1 FROM schema_migrations WHERE version = 3);
```

258 migrations/004_bible_ref_index.sql

```
-- Pricklypear migration 004: unique ref index for bible-verse rows.
--
-- Supports idempotent bulk seed of BSB (fields.ref = 'BSB:John:3:16')
-- and fast (find-row 'bible-verse "ref" ?). Additive only.

CREATE UNIQUE INDEX IF NOT EXISTS data_rows_bible_ref_uidx
  ON data_rows ((fields->>'ref'))
  WHERE type_name = 'bible-verse';

CREATE INDEX IF NOT EXISTS data_rows_bible_abbrev_chapter_idx
  ON data_rows ((fields->>'abbrev'), (fields->>'chapter'))
  WHERE type_name = 'bible-verse';

INSERT INTO schema_migrations (version, description)
SELECT 4, 'bible-verse fields.ref unique index + abbrev/chapter index'
WHERE NOT EXISTS (SELECT 1 FROM schema_migrations WHERE version = 4);
```

259 migrations/005_ai_sessions.sql

```
-- Pricklypear migration 005: durable agent conversation store.
--
-- Per spec [ai-authoring.agent-portal]:
--   One tool-capable agent loop. Provider chat/completions is
--   stateless -- every turn resubmits full history. We own durability
--   here in ai_sessions + ai_messages.
--
-- Named to avoid clash with:
--   sessions      (fleet / image instances, migration 001)
--   auth_sessions (login cookies, migration 002)
--
-- Additive-only. System seed lives on ai_sessions, not as role=system
-- message rows. Message roles: user | assistant | tool.

CREATE TABLE IF NOT EXISTS ai_sessions (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  owner       text NOT NULL,              -- username (whoami)
  title       text,                       -- optional; often first user line
  seed        text NOT NULL,              -- system prompt; not a message
    row
  branch_ref  text,                       -- branch at create / last say
  tools       boolean NOT NULL DEFAULT true,
  model       text,                       -- LLM_MODEL snapshot at create
  created_at  timestamptz NOT NULL DEFAULT now(),
  updated_at  timestamptz NOT NULL DEFAULT now()
);

CREATE INDEX IF NOT EXISTS ai_sessions_owner_updated_idx
  ON ai_sessions (owner, updated_at DESC);

CREATE TABLE IF NOT EXISTS ai_messages (
  id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  session_id    uuid NOT NULL REFERENCES ai_sessions(id) ON DELETE CASCADE,
  seq           int  NOT NULL,            -- dense order from 0
  role          text NOT NULL,            -- user | assistant | tool
  content       text,                     -- nullable for pure tool-call
    turns
  tool_calls    jsonb,                    -- [{id,name,arguments}] or null
  tool_call_id  text,                     -- set when role=tool
  created_at    timestamptz NOT NULL DEFAULT now(),
  UNIQUE (session_id, seq),
  CONSTRAINT ai_messages_role_chk
    CHECK (role IN ('user', 'assistant', 'tool'))
);

CREATE INDEX IF NOT EXISTS ai_messages_session_seq_idx
  ON ai_messages (session_id, seq);

INSERT INTO schema_migrations (version, description)
SELECT 5, 'agent conversation store (ai_sessions, ai_messages)'
WHERE NOT EXISTS (SELECT 1 FROM schema_migrations WHERE version = 5);
```

260 migrations/006_route_visibility.sql

```
-- 006_route_visibility.sql
-- Additive: public|private flag on user routes (auth.route-visibility).
-- Existing rows stay private via DEFAULT.

ALTER TABLE view_bindings
  ADD COLUMN IF NOT EXISTS visibility text NOT NULL DEFAULT 'private';
```

261 migrations/007_media.sql

```
-- Pricklypear migration 007: binary media blob storage.
--
-- Per spec [storage.media]: agent/browser image (and small binary)
-- uploads live in media_blobs, NOT in data_rows jsonb. Bytea keeps
-- payloads off the GIN-indexed JSON path and off the 64 KiB /api/eval
-- body cap. Ownership is the author column; rows are shared across
-- branches (same rule as data_rows -- fork/merge copies code, not media).
--
-- Additive-only. Idempotent.

CREATE TABLE IF NOT EXISTS media_blobs (
  id            uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  author        text NOT NULL,
  content_type  text NOT NULL,
  filename      text,
  byte_size     int  NOT NULL,
  sha256        text NOT NULL,
  data          bytea NOT NULL,
  meta          jsonb NOT NULL DEFAULT '{}'::jsonb,
  created_at    timestamptz NOT NULL DEFAULT now(),
  updated_at    timestamptz NOT NULL DEFAULT now()
);

CREATE INDEX IF NOT EXISTS media_blobs_author_idx
  ON media_blobs (author);
CREATE INDEX IF NOT EXISTS media_blobs_created_idx
  ON media_blobs (created_at DESC);
CREATE INDEX IF NOT EXISTS media_blobs_sha256_idx
  ON media_blobs (sha256);

INSERT INTO schema_migrations (version, description)
SELECT 7, 'media_blobs binary object store (agent image upload)'
WHERE NOT EXISTS (SELECT 1 FROM schema_migrations WHERE version = 7);
```

262 migrations/008_agent_users.sql

```
-- Pricklypear migration 008: agent users + token auth + promote toggle.
--
-- Additive ALTER to the users table (migrations.borg additive-only rule):
--   user_type           'human' (default) | 'agent'
--   parent_user_id      the human an agent belongs to; NULL for humans
--   allow_agent_promote the human's toggle for whether their agent may
--                       promote into their personal branch (default true)
--
-- No schema change to credentials: kind='api_key' rows already fit the
-- existing (user_id, kind, public_data, secret_data) shape -- secret_data
-- holds {"token_hash": "<sha256 hex>"} and public_data holds {"label": ...}.
-- A new index on (kind, secret_data->>'token_hash') makes Bearer lookups fast.

ALTER TABLE users
  ADD COLUMN IF NOT EXISTS user_type text NOT NULL DEFAULT 'human';

ALTER TABLE users
  ADD COLUMN IF NOT EXISTS parent_user_id uuid NULL REFERENCES users(id);

ALTER TABLE users
  ADD COLUMN IF NOT EXISTS allow_agent_promote boolean NOT NULL DEFAULT true;

-- Fast lookup for Bearer token verification: WHERE kind='api_key'
-- AND secret_data->>'token_hash' = $1.
CREATE INDEX IF NOT EXISTS credentials_api_key_hash_idx
  ON credentials (kind, (secret_data->>'token_hash'))
  WHERE kind = 'api_key';

INSERT INTO schema_migrations (version, description)
SELECT 8, 'agent users (user_type, parent_user_id, allow_agent_promote) + api_key token index'
WHERE NOT EXISTS (SELECT 1 FROM schema_migrations WHERE version = 8);
```

263 migrations/009_tree.sql

```
-- Pricklypear migration 009: ONE TREE substrate (shadow phase).
--
-- Per spec [one-tree.tree-model / migration-plan]. This is the ADDITIVE
-- physical layout for the one-big-tree store, plus the P2 SHADOW MIRROR
-- that keeps the tree in lockstep with the scaffolding tables
-- (function_defs, type_defs, view_bindings, data_rows) via deferred
-- triggers. Nothing reads from the tree yet (that is P3+); this is
-- pure additive shadow -- zero behavior change, fully reversible by
-- dropping the triggers (the additive-only rule applies to the tree's
-- OWN tables going forward; these triggers are the mirror mechanism).
--
-- Path grammar (ltree, dots not slashes -- ltree labels):
--   fn     b.<branch_id>.fn.<name>
--   type   b.<branch_id>.type.<name>
--   route  b.<branch_id>.route.<route_b64url>   (routes contain '/:+'
--                                               which are not ltree-safe)
--   datum  data.<type_name>.<id>
-- Node payload is a jsonb envelope {v,kind,path,type,body,author,doc,hash}.
-- hash = sha256 over the CANONICAL body (independent of path/author) so
-- content corruption is detectable. data_nodes are SHARED (branch-free)
-- per [branching.data]; code/type/route nodes live under a branch prefix.

CREATE EXTENSION IF NOT EXISTS ltree;
CREATE EXTENSION IF NOT EXISTS pgcrypto;  -- digest() for content hashes

CREATE TABLE IF NOT EXISTS tree_nodes (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  path        ltree NOT NULL UNIQUE,
  kind        text NOT NULL,          -- fn | type | route | datum
  type_name   text,                   -- datum only: the Nopales type
  payload     jsonb NOT NULL,         -- envelope (see header)
  hash        text NOT NULL,          -- sha256 over canonical body
  author      text NOT NULL,
  created_at  timestamptz NOT NULL DEFAULT now(),
  updated_at  timestamptz NOT NULL DEFAULT now()
);
CREATE INDEX IF NOT EXISTS tree_nodes_path_gist ON tree_nodes USING gist
    (path);
CREATE INDEX IF NOT EXISTS tree_nodes_kind_type_idx ON tree_nodes (kind,
    type_name);
CREATE INDEX IF NOT EXISTS tree_nodes_payload_gin ON tree_nodes USING gin
    (payload);

-- The lens registry. In v1 the branches table still owns branch_id +
-- parent chain; tree_branches adds the lens root pointer + head hash.
-- True content-addressed pointer-moves (fork/promote) land in P5.
CREATE TABLE IF NOT EXISTS tree_branches (
  branch_id   uuid PRIMARY KEY REFERENCES branches(id) ON DELETE CASCADE,
  root_path   ltree NOT NULL,          -- b.<branch_id>
  parent_id   uuid,                    -- mirror of branches.parent_id
    (walk)
  head_hash   text                     -- subtree root hash (P5)
);

--
    ------------------------------------------------------------------------
-- Label encoding. ltree labels: [A-Za-z0-9_], no dots/dashes/slashes.
-- UUIDs -> hex (strip dashes). Names/routes/types -> base64url-ish of the
-- raw bytes using _ for +, - for /, and trailing digits for pad count.
-- Deterministic + reversible in OCaml (tree_store.ml decodes).
--
    ------------------------------------------------------------------------
CREATE OR REPLACE FUNCTION tree_enc_label(s text) RETURNS text
LANGUAGE plpgsql IMMUTABLE AS $$
DECLARE
  b64 text;
  pad int;
BEGIN
  IF s IS NULL OR s = '' THEN RETURN 'x'; END IF;
  b64 := encode(s::bytea, 'base64');
  pad := (length(b64) - length(rtrim(b64, '=')));  -- 0,1,2 trailing '='
```

```
  b64 := rtrim(b64, '=');
  b64 := replace(b64, '+', '_');
  b64 := replace(b64, '/', '-');
  RETURN b64 || pad::text;   -- pad count disambiguates; ''->'x' handled above
END $$;

CREATE OR REPLACE FUNCTION tree_uuid_label(u uuid) RETURNS text
LANGUAGE sql IMMUTABLE AS $$ SELECT replace(u::text, '-', '') $$;

-- ---------------------------------------------------------------------------
-- Mirror trigger functions. Each maps one scaffolding row -> one tree node
-- (upsert), and removes the node on row delete. CONSTRAINT TRIGGER ... DEFERRABLE
-- INITIALLY DEFERRED so the mirror runs at COMMIT (after the statement),
-- keeping it in the same transaction as the write.
-- ---------------------------------------------------------------------------
CREATE OR REPLACE FUNCTION tree_mirror_function_defs() RETURNS trigger
LANGUAGE plpgsql AS $$
DECLARE p ltree; body jsonb; h text; env jsonb;
BEGIN
  PERFORM set_config('pp.tree_mirror','1', true);  -- guard: our tree write must not bounce back
  IF (TG_OP = 'DELETE') THEN
    DELETE FROM tree_nodes
     WHERE path = ('b.' || tree_uuid_label(OLD.branch_id) || '.fn.'
                   || tree_enc_label(OLD.name))::ltree;
    RETURN OLD;
  END IF;
  p := ('b.' || tree_uuid_label(NEW.branch_id) || '.fn.'
        || tree_enc_label(NEW.name))::ltree;
  body := NEW.body;  -- canonical fn body IS the AST jsonb
  h := encode(digest(body::text, 'sha256'), 'hex');
  env := jsonb_build_object(
    'v', 1, 'kind', 'fn', 'path', p::text, 'type', 'fn',
    'body', body, 'author', NEW.author, 'doc', NEW.doc, 'hash', h);
  INSERT INTO tree_nodes (path, kind, type_name, payload, hash, author, updated_at)
  VALUES (p, 'fn', NULL, env, h, NEW.author, now())
  ON CONFLICT (path) DO UPDATE
    SET payload = EXCLUDED.payload, hash = EXCLUDED.hash,
        author = EXCLUDED.author, updated_at = now();
  RETURN NEW;
END $$;

CREATE OR REPLACE FUNCTION tree_mirror_type_defs() RETURNS trigger
LANGUAGE plpgsql AS $$
DECLARE p ltree; body jsonb; h text; env jsonb;
BEGIN
  PERFORM set_config('pp.tree_mirror','1', true);  -- guard: our tree write must not bounce back
  IF (TG_OP = 'DELETE') THEN
    DELETE FROM tree_nodes
     WHERE path = ('b.' || tree_uuid_label(OLD.branch_id) || '.type.'
                   || tree_enc_label(OLD.name))::ltree;
    RETURN OLD;
  END IF;
  p := ('b.' || tree_uuid_label(NEW.branch_id) || '.type.'
        || tree_enc_label(NEW.name))::ltree;
  body := NEW.fields;  -- canonical type body IS the fields jsonb array
  h := encode(digest(body::text, 'sha256'), 'hex');
  env := jsonb_build_object(
    'v', 1, 'kind', 'type', 'path', p::text, 'type', NEW.name,
    'body', body, 'author', NEW.author, 'doc', NULL, 'hash', h);
  INSERT INTO tree_nodes (path, kind, type_name, payload, hash, author, updated_at)
  VALUES (p, 'type', NEW.name, env, h, NEW.author, now())
  ON CONFLICT (path) DO UPDATE
    SET payload = EXCLUDED.payload, hash = EXCLUDED.hash,
        author = EXCLUDED.author, updated_at = now();
  RETURN NEW;
```

```
END $$;

CREATE OR REPLACE FUNCTION tree_mirror_view_bindings() RETURNS trigger
LANGUAGE plpgsql AS $$
DECLARE p ltree; body jsonb; h text; env jsonb; vis text;
BEGIN
  PERFORM set_config('pp.tree_mirror','1', true);  -- guard: our tree write must not bounce back
  IF (TG_OP = 'DELETE') THEN
    DELETE FROM tree_nodes
     WHERE path = ('b.' || tree_uuid_label(OLD.branch_id) || '.route.'
                   || tree_enc_label(OLD.route))::ltree;
    RETURN OLD;
  END IF;
  p := ('b.' || tree_uuid_label(NEW.branch_id) || '.route.'
        || tree_enc_label(NEW.route))::ltree;
  vis := COALESCE(NEW.visibility, 'private');
  body := jsonb_build_object('route', NEW.route, 'handler', NEW.handler,
                             'visibility', vis);
  h := encode(digest(body::text, 'sha256'), 'hex');
  env := jsonb_build_object(
    'v', 1, 'kind', 'route', 'path', p::text, 'type', 'route',
    'body', body, 'author', 'system', 'doc', NULL, 'hash', h);
  INSERT INTO tree_nodes (path, kind, type_name, payload, hash, author, updated_at)
  VALUES (p, 'route', NULL, env, h, 'system', now())
  ON CONFLICT (path) DO UPDATE
    SET payload = EXCLUDED.payload, hash = EXCLUDED.hash,
        author = EXCLUDED.author, updated_at = now();
  RETURN NEW;
END $$;

CREATE OR REPLACE FUNCTION tree_mirror_data_rows() RETURNS trigger
LANGUAGE plpgsql AS $$
DECLARE p ltree; body jsonb; h text; env jsonb;
BEGIN
  PERFORM set_config('pp.tree_mirror','1', true);  -- guard: our tree write must not bounce back
  IF (TG_OP = 'DELETE') THEN
    DELETE FROM tree_nodes
     WHERE path = ('data.' || tree_enc_label(OLD.type_name) || '.'
                   || tree_uuid_label(OLD.id))::ltree;
    RETURN OLD;
  END IF;
  p := ('data.' || tree_enc_label(NEW.type_name) || '.'
        || tree_uuid_label(NEW.id))::ltree;
  body := NEW.fields;  -- canonical datum body IS the fields jsonb
  h := encode(digest(body::text, 'sha256'), 'hex');
  env := jsonb_build_object(
    'v', 1, 'kind', 'datum', 'path', p::text, 'type', NEW.type_name,
    'body', body, 'author', NEW.author, 'doc', NULL, 'hash', h);
  INSERT INTO tree_nodes (path, kind, type_name, payload, hash, author, updated_at)
  VALUES (p, 'datum', NEW.type_name, env, h, NEW.author, now())
  ON CONFLICT (path) DO UPDATE
    SET payload = EXCLUDED.payload, hash = EXCLUDED.hash,
        author = EXCLUDED.author, updated_at = now();
  RETURN NEW;
END $$;

-- Deferred mirror triggers (fire at commit, in the write's transaction).
DROP TRIGGER IF EXISTS tree_mirror_function_defs_trg ON function_defs;
CREATE CONSTRAINT TRIGGER tree_mirror_function_defs_trg
  AFTER INSERT OR UPDATE OR DELETE ON function_defs
  DEFERRABLE INITIALLY DEFERRED
  FOR EACH ROW EXECUTE FUNCTION tree_mirror_function_defs();

DROP TRIGGER IF EXISTS tree_mirror_type_defs_trg ON type_defs;
CREATE CONSTRAINT TRIGGER tree_mirror_type_defs_trg
  AFTER INSERT OR UPDATE OR DELETE ON type_defs
  DEFERRABLE INITIALLY DEFERRED
  FOR EACH ROW EXECUTE FUNCTION tree_mirror_type_defs();

```

```
DROP TRIGGER IF EXISTS tree_mirror_view_bindings_trg ON view_bindings;
CREATE CONSTRAINT TRIGGER tree_mirror_view_bindings_trg
  AFTER INSERT OR UPDATE OR DELETE ON view_bindings
  DEFERRABLE INITIALLY DEFERRED
  FOR EACH ROW EXECUTE FUNCTION tree_mirror_view_bindings();

DROP TRIGGER IF EXISTS tree_mirror_data_rows_trg ON data_rows;
CREATE CONSTRAINT TRIGGER tree_mirror_data_rows_trg
  AFTER INSERT OR UPDATE OR DELETE ON data_rows
  DEFERRABLE INITIALLY DEFERRED
  FOR EACH ROW EXECUTE FUNCTION tree_mirror_data_rows();

-- Keep tree_branches in step with branches (lens registry).
CREATE OR REPLACE FUNCTION tree_mirror_branches() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
  IF (TG_OP = 'DELETE') THEN
    DELETE FROM tree_branches WHERE branch_id = OLD.id;
    RETURN OLD;
  END IF;
  INSERT INTO tree_branches (branch_id, root_path, parent_id)
  VALUES (NEW.id, ('b.' || tree_uuid_label(NEW.id))::ltree, NEW.parent_id)
  ON CONFLICT (branch_id) DO UPDATE
    SET root_path = EXCLUDED.root_path, parent_id = EXCLUDED.parent_id;
  RETURN NEW;
END $$;
DROP TRIGGER IF EXISTS tree_mirror_branches_trg ON branches;
CREATE CONSTRAINT TRIGGER tree_mirror_branches_trg
  AFTER INSERT OR UPDATE OR DELETE ON branches
  DEFERRABLE INITIALLY DEFERRED
  FOR EACH ROW EXECUTE FUNCTION tree_mirror_branches();

--
    ------------------------------------------------------------------------
-- BACKFILL existing rows into the tree (idempotent upserts; the ON CONFLICT
-- in the mirror functions makes re-runs safe).
--
    ------------------------------------------------------------------------
INSERT INTO tree_branches (branch_id, root_path, parent_id)
SELECT id, ('b.' || tree_uuid_label(id))::ltree, parent_id FROM branches
ON CONFLICT (branch_id) DO NOTHING;

INSERT INTO tree_nodes (path, kind, type_name, payload, hash, author)
SELECT
  ('b.' || tree_uuid_label(branch_id) || '.fn.' ||
     tree_enc_label(name))::ltree,
  'fn', NULL,
  jsonb_build_object('v',1,'kind','fn',
    'path', ('b.' || tree_uuid_label(branch_id) || '.fn.' ||
    tree_enc_label(name)),
    'type','fn','body', body, 'author', author, 'doc', doc,
    'hash', encode(digest(body::text,'sha256'),'hex')),
  encode(digest(body::text,'sha256'),'hex'),
  author
FROM function_defs
ON CONFLICT (path) DO NOTHING;

INSERT INTO tree_nodes (path, kind, type_name, payload, hash, author)
SELECT
  ('b.' || tree_uuid_label(branch_id) || '.type.' ||
     tree_enc_label(name))::ltree,
  'type', name,
  jsonb_build_object('v',1,'kind','type',
    'path', ('b.' || tree_uuid_label(branch_id) || '.type.' ||
    tree_enc_label(name)),
    'type', name, 'body', fields, 'author', author, 'doc', NULL,
    'hash', encode(digest(fields::text,'sha256'),'hex')),
  encode(digest(fields::text,'sha256'),'hex'),
  author
FROM type_defs
ON CONFLICT (path) DO NOTHING;

INSERT INTO tree_nodes (path, kind, type_name, payload, hash, author)
```

```
SELECT
  ('b.' || tree_uuid_label(branch_id) || '.route.' ||
    tree_enc_label(route))::ltree,
  'route', NULL,
  jsonb_build_object('v',1,'kind','route',
    'path', ('b.' || tree_uuid_label(branch_id) || '.route.' ||
    tree_enc_label(route)),
    'type','route',
    'body', jsonb_build_object('route', route, 'handler', handler,
                               'visibility',
    COALESCE(visibility,'private')),
    'author','system','doc', NULL,
    'hash', encode(digest(jsonb_build_object('route',
    route,'handler',handler,
             'visibility',
    COALESCE(visibility,'private'))::text,'sha256'),'hex')),
  encode(digest(jsonb_build_object('route', route,'handler',handler,
           'visibility',
    COALESCE(visibility,'private'))::text,'sha256'),'hex'),
  'system'
FROM view_bindings
ON CONFLICT (path) DO NOTHING;

INSERT INTO tree_nodes (path, kind, type_name, payload, hash, author)
SELECT
  ('data.' || tree_enc_label(type_name) || '.' ||
    tree_uuid_label(id))::ltree,
  'datum', type_name,
  jsonb_build_object('v',1,'kind','datum',
    'path', ('data.' || tree_enc_label(type_name) || '.' ||
    tree_uuid_label(id)),
    'type', type_name, 'body', fields, 'author', author, 'doc', NULL,
    'hash', encode(digest(fields::text,'sha256'),'hex')),
  encode(digest(fields::text,'sha256'),'hex'),
  author
FROM data_rows
ON CONFLICT (path) DO NOTHING;

--
    ------------------------------------------------------------------------
-- Consistency check: counts of mirrored rows vs tree nodes per kind.
-- Returns rows (kind, table_count, tree_count, in_sync). Used by
    smoke/tests.
--
    ------------------------------------------------------------------------
CREATE OR REPLACE FUNCTION tree_verify()
  RETURNS TABLE(kind text, table_count bigint, tree_count bigint, in_sync
    boolean)
LANGUAGE sql AS $$
  SELECT 'fn', (SELECT count(*) FROM function_defs),
         (SELECT count(*) FROM tree_nodes WHERE kind='fn'),
         (SELECT count(*) FROM function_defs) =
           (SELECT count(*) FROM tree_nodes WHERE kind='fn')
  UNION ALL
  SELECT 'type', (SELECT count(*) FROM type_defs),
         (SELECT count(*) FROM tree_nodes WHERE kind='type'),
         (SELECT count(*) FROM type_defs) =
           (SELECT count(*) FROM tree_nodes WHERE kind='type')
  UNION ALL
  SELECT 'route', (SELECT count(*) FROM view_bindings),
         (SELECT count(*) FROM tree_nodes WHERE kind='route'),
         (SELECT count(*) FROM view_bindings) =
           (SELECT count(*) FROM tree_nodes WHERE kind='route')
  UNION ALL
  SELECT 'datum', (SELECT count(*) FROM data_rows),
         (SELECT count(*) FROM tree_nodes WHERE kind='datum'),
         (SELECT count(*) FROM data_rows) =
           (SELECT count(*) FROM tree_nodes WHERE kind='datum')
$$;

--
    ------------------------------------------------------------------------
-- P4 REVERSE MIRROR: tree_nodes -> scaffolding tables. Lets the image write
```

```
-- the TREE as truth while the legacy tables (and every tool/read path that
-- still hits them) stay in lockstep. Recursion-guarded via a transaction-
-- scoped GUC: table->tree triggers set it, tree->table triggers skip when
-- set, so a table write mirrors to tree and STOPS (no ping-pong).
-- Deferred to commit, same-transaction => identical durability semantics.
--
    ------------------------------------------------------------------------
CREATE OR REPLACE FUNCTION tree_in_mirror() RETURNS boolean
LANGUAGE sql STABLE AS $$
  SELECT COALESCE(current_setting('pp.tree_mirror', true), '') = '1'
$$;

CREATE OR REPLACE FUNCTION tree_from_nodes() RETURNS trigger
LANGUAGE plpgsql AS $$
DECLARE bid uuid; lbl text; docv text;
BEGIN
  IF tree_in_mirror() THEN
    IF TG_OP = 'DELETE' THEN RETURN OLD; ELSE RETURN NEW; END IF;
  END IF;
  IF (TG_OP = 'DELETE') THEN
    bid := tree_path_branch(OLD.path);
    IF OLD.kind = 'fn' THEN
      lbl := (string_to_array(OLD.path::text,'.'))[4];
      DELETE FROM function_defs WHERE branch_id=bid AND
     name=tree_dec_label(lbl);
    ELSIF OLD.kind = 'type' THEN
      lbl := (string_to_array(OLD.path::text,'.'))[4];
      DELETE FROM type_defs WHERE branch_id=bid AND
     name=tree_dec_label(lbl);
    ELSIF OLD.kind = 'route' THEN
      DELETE FROM view_bindings WHERE branch_id=bid AND
     route=OLD.payload->'body'->>'route';
    ELSIF OLD.kind = 'datum' THEN
      DELETE FROM data_rows WHERE
     id=((string_to_array(OLD.path::text,'.'))[3])::uuid;
    END IF;
    RETURN OLD;
  END IF;
  docv := NEW.payload->>'doc';
  IF NEW.kind = 'fn' THEN
    bid := tree_path_branch(NEW.path);
    lbl := (string_to_array(NEW.path::text,'.'))[4];
    INSERT INTO function_defs (branch_id,name,body,author,doc)
    VALUES (bid, tree_dec_label(lbl), NEW.payload->'body',
            NEW.payload->>'author', docv)
    ON CONFLICT (branch_id,name) DO UPDATE
      SET body=EXCLUDED.body, author=EXCLUDED.author, doc=EXCLUDED.doc,
          updated_at=now();
  ELSIF NEW.kind = 'type' THEN
    bid := tree_path_branch(NEW.path);
    lbl := (string_to_array(NEW.path::text,'.'))[4];
    INSERT INTO type_defs (branch_id,name,fields,author)
    VALUES (bid, tree_dec_label(lbl), NEW.payload->'body',
     NEW.payload->>'author')
    ON CONFLICT (branch_id,name) DO UPDATE
      SET fields=EXCLUDED.fields, author=EXCLUDED.author, updated_at=now();
  ELSIF NEW.kind = 'route' THEN
    bid := tree_path_branch(NEW.path);
    INSERT INTO view_bindings (branch_id,route,handler,visibility)
    VALUES (bid, NEW.payload->'body'->>'route',
     NEW.payload->'body'->>'handler',
            COALESCE(NEW.payload->'body'->>'visibility','private'))
    ON CONFLICT (branch_id,route) DO UPDATE
      SET handler=EXCLUDED.handler, visibility=EXCLUDED.visibility;
  ELSIF NEW.kind = 'datum' THEN
    INSERT INTO data_rows (id,type_name,fields,author)
    VALUES (((string_to_array(NEW.path::text,'.'))[3])::uuid,
            NEW.type_name, NEW.payload->'body', NEW.payload->>'author')
    ON CONFLICT (id) DO UPDATE
      SET fields=EXCLUDED.fields, author=EXCLUDED.author, updated_at=now();
  END IF;
  RETURN NEW;
END $$;
```

```
393
394 DROP TRIGGER IF EXISTS tree_from_nodes_trg ON tree_nodes;
395 CREATE CONSTRAINT TRIGGER tree_from_nodes_trg
396   AFTER INSERT OR UPDATE OR DELETE ON tree_nodes
397   DEFERRABLE INITIALLY DEFERRED
398   FOR EACH ROW EXECUTE FUNCTION tree_from_nodes();
399
400 -- Re-point the forward (table->tree) triggers to set the guard so the
401 -- tree write they cause does NOT bounce back into the table.
402 CREATE OR REPLACE FUNCTION tree_mirror_guard_set() RETURNS void
403 LANGUAGE plpgsql AS $$
404 BEGIN
405   PERFORM set_config('pp.tree_mirror','1', true);
406 END $$;
407
408 --
      ------------------------------------------------------------------------
409 -- P3 READ VIEWS: tree-backed relations that reproduce the exact column
      shape
410 -- of the scaffolding tables, so store.ml / crud.ml can read from the tree
411 -- behind the unchanged prim ABI by swapping the relation (flag-gated via
412 -- PP_TREE_READS). branch_id is recovered from the path's 2nd label (the
413 -- uuid-hex); names/routes are decoded from their labels.
414 --
      ------------------------------------------------------------------------
415 CREATE OR REPLACE FUNCTION tree_dec_label(lbl text) RETURNS text
416 LANGUAGE plpgsql IMMUTABLE AS $$
417 DECLARE pad int; core text; b64 text;
418 BEGIN
419   IF lbl IS NULL THEN RETURN NULL; END IF;
420   IF lbl = 'x' THEN RETURN ''; END IF;
421   pad := substr(lbl, length(lbl), 1)::int;   -- trailing pad count 0/1/2
422   core := substr(lbl, 1, length(lbl) - 1);
423   b64 := replace(replace(core, '_', '+'), '-', '/');
424   b64 := b64 || repeat('=', pad);
425   RETURN convert_from(decode(b64, 'base64'), 'UTF8');
426 END $$;
427
428 -- branch_id (uuid) from a b.<uuidhex>.... path. Shared subtrees
429 -- (data.*, meta.*) have NO branch prefix -> return NULL (caller checks).
430 CREATE OR REPLACE FUNCTION tree_path_branch(p ltree) RETURNS uuid
431 LANGUAGE plpgsql IMMUTABLE AS $$
432 BEGIN
433   IF p::text NOT LIKE 'b.%' THEN RETURN NULL; END IF;
434   RETURN (substr(p::text, 3, 8) || '-' || substr(p::text, 11, 4) || '-' ||
435           substr(p::text, 15, 4) || '-' || substr(p::text, 19, 4) || '-' ||
436           substr(p::text, 23, 12))::uuid;
437 END $$;
438
439 CREATE OR REPLACE VIEW tree_function_defs AS
440 SELECT tree_path_branch(path) AS branch_id,
441        tree_dec_label((string_to_array(path::text, '.'))[4]) AS name,
442        payload->'body' AS body,
443        payload->>'author' AS author,
444        payload->>'doc' AS doc,
445        created_at, updated_at
446 FROM tree_nodes WHERE kind = 'fn';
447
448 CREATE OR REPLACE VIEW tree_type_defs AS
449 SELECT tree_path_branch(path) AS branch_id,
450        tree_dec_label((string_to_array(path::text, '.'))[4]) AS name,
451        payload->'body' AS fields,
452        payload->>'author' AS author,
453        created_at, updated_at
454 FROM tree_nodes WHERE kind = 'type';
455
456 CREATE OR REPLACE VIEW tree_view_bindings AS
457 SELECT tree_path_branch(path) AS branch_id,
458        payload->'body'->>'route' AS route,
459        payload->'body'->>'handler' AS handler,
460        COALESCE(payload->'body'->>'visibility', 'private') AS visibility,
461        created_at, updated_at
462 FROM tree_nodes WHERE kind = 'route';
```

```
463
464 CREATE OR REPLACE VIEW tree_data_rows AS
465 SELECT ((string_to_array(path::text, '.'))[3])::uuid AS id,
466        type_name,
467        payload->'body' AS fields,
468        payload->>'author' AS author,
469        created_at, updated_at
470 FROM tree_nodes WHERE kind = 'datum';
471
472 INSERT INTO schema_migrations (version, description)
473 SELECT 9, 'one-tree substrate: tree_nodes + tree_branches + deferred shadow
      mirror + read views'
474 WHERE NOT EXISTS (SELECT 1 FROM schema_migrations WHERE version = 9);
```

264 migrations/010_notes_tree.sql

```
-- Pricklypear migration 010: hierarchical notes indexes
    (notes.tree-cutover K1).
--
-- folder/note/heading live in tree_nodes (not kind=datum). Enforce:
--   - unique body.uuid among folder+note (stable identity / get-uuid)
--   - unique (author, body.slug) among notes (per-user global slug)
-- Additive only; safe to re-run (IF NOT EXISTS).

CREATE UNIQUE INDEX IF NOT EXISTS tree_nodes_hier_uuid_uidx
  ON tree_nodes ((payload #>> '{body,uuid}'))
  WHERE kind IN ('folder', 'note')
    AND coalesce(payload #>> '{body,uuid}', '') <> '';

CREATE UNIQUE INDEX IF NOT EXISTS tree_nodes_note_slug_author_uidx
  ON tree_nodes (author, (payload #>> '{body,slug}'))
  WHERE kind = 'note'
    AND coalesce(payload #>> '{body,slug}', '') <> '';

CREATE INDEX IF NOT EXISTS tree_nodes_body_slug_idx
  ON tree_nodes ((payload #>> '{body,slug}'))
  WHERE kind = 'note';

CREATE INDEX IF NOT EXISTS tree_nodes_body_uuid_idx
  ON tree_nodes ((payload #>> '{body,uuid}'))
  WHERE kind IN ('folder', 'note', 'heading')
    AND coalesce(payload #>> '{body,uuid}', '') <> '';
```

265 migrations/011_package_registry.sql

```
-- Pricklypear migration 011: package registry + function provenance.
--
-- lisp-interpreter.package-surface Phase 1:
--   - package_registry is branch-lensed (enable/disable + export list)
--   - path family b.<branch>.pkg.<name> via forward-only tree mirror
--   - function_defs.package stamps which package defined a binding
--
-- Disable semantics = HIDE (enabled=false), never DELETE rows.
-- std is hard-protected in OCaml; not special-cased in SQL.

ALTER TABLE function_defs
  ADD COLUMN IF NOT EXISTS package text NULL;

CREATE INDEX IF NOT EXISTS function_defs_package_idx
  ON function_defs (branch_id, package)
  WHERE package IS NOT NULL;

CREATE TABLE IF NOT EXISTS package_registry (
  branch_id   uuid NOT NULL REFERENCES branches(id) ON DELETE CASCADE,
  name        text NOT NULL,
  enabled     boolean NOT NULL DEFAULT true,
  exports     jsonb NOT NULL DEFAULT '[]'::jsonb,
  author      text NOT NULL DEFAULT 'system',
  updated_at  timestamptz NOT NULL DEFAULT now(),
  PRIMARY KEY (branch_id, name)
);

CREATE INDEX IF NOT EXISTS package_registry_enabled_idx
  ON package_registry (branch_id, enabled);

-- Forward mirror: package_registry -> tree_nodes kind=pkg
-- Pattern matches 009_tree.sql tree_mirror_function_defs (pp.tree_mirror
    guard).
CREATE OR REPLACE FUNCTION tree_mirror_package_registry() RETURNS trigger
LANGUAGE plpgsql AS $$
DECLARE p ltree; body jsonb; h text; env jsonb;
BEGIN
  PERFORM set_config('pp.tree_mirror','1', true);
  IF (TG_OP = 'DELETE') THEN
    DELETE FROM tree_nodes
     WHERE path = ('b.' || tree_uuid_label(OLD.branch_id) || '.pkg.'
                   || tree_enc_label(OLD.name))::ltree;
    RETURN OLD;
  END IF;
  p := ('b.' || tree_uuid_label(NEW.branch_id) || '.pkg.'
        || tree_enc_label(NEW.name))::ltree;
  body := jsonb_build_object(
    'name', NEW.name,
    'enabled', NEW.enabled,
    'exports', COALESCE(NEW.exports, '[]'::jsonb));
  h := encode(digest(body::text, 'sha256'), 'hex');
  env := jsonb_build_object(
    'v', 1, 'kind', 'pkg', 'path', p::text, 'type', 'pkg',
    'body', body, 'author', NEW.author, 'doc', NULL, 'hash', h);
  INSERT INTO tree_nodes (path, kind, type_name, payload, hash, author,
     updated_at)
  VALUES (p, 'pkg', NULL, env, h, NEW.author, now())
  ON CONFLICT (path) DO UPDATE
    SET payload = EXCLUDED.payload, hash = EXCLUDED.hash,
        author = EXCLUDED.author, updated_at = now();
  RETURN NEW;
END $$;

DROP TRIGGER IF EXISTS tree_mirror_package_registry_trg ON package_registry;
CREATE CONSTRAINT TRIGGER tree_mirror_package_registry_trg
  AFTER INSERT OR UPDATE OR DELETE ON package_registry
  DEFERRABLE INITIALLY DEFERRED
  FOR EACH ROW EXECUTE FUNCTION tree_mirror_package_registry();

-- Expose package provenance on the tree-backed fn view when
    PP_TREE_READS=1.
```

```
-- package lives on the scaffolding row (not in historic tree payloads);
-- LEFT JOIN keeps tree-only rows visible with package NULL.
-- DROP required: CREATE OR REPLACE cannot reorder/rename view columns.
DROP VIEW IF EXISTS tree_function_defs;
CREATE VIEW tree_function_defs AS
SELECT tree_path_branch(n.path) AS branch_id,
       tree_dec_label((string_to_array(n.path::text, '.'))[4]) AS name,
       n.payload->'body' AS body,
       n.payload->>'author' AS author,
       n.payload->>'doc' AS doc,
       fd.package AS package,
       n.created_at, n.updated_at
FROM tree_nodes n
LEFT JOIN function_defs fd
  ON fd.branch_id = tree_path_branch(n.path)
 AND fd.name = tree_dec_label((string_to_array(n.path::text, '.'))[4])
WHERE n.kind = 'fn';

CREATE OR REPLACE VIEW tree_package_registry AS
SELECT tree_path_branch(path) AS branch_id,
       tree_dec_label((string_to_array(path::text, '.'))[4]) AS name,
       COALESCE((payload->'body'->>'enabled')::boolean, true) AS enabled,
       COALESCE(payload->'body'->'exports', '[]'::jsonb) AS exports,
       payload->>'author' AS author,
       created_at, updated_at
FROM tree_nodes WHERE kind = 'pkg';

INSERT INTO schema_migrations (version, description)
SELECT 11, 'package registry + function_defs.package provenance'
WHERE NOT EXISTS (SELECT 1 FROM schema_migrations WHERE version = 11);
```

266 migrations/012_code_history.sql

```
-- Pricklypear migration 012: append-only before-image history for code rows.
-- Spec: branching.code-history
--
-- Captures OLD on BEFORE UPDATE OR DELETE for function_defs, type_defs,
-- and view_bindings. INSERT does not fire. Triggers run in the writer
-- transaction so rollbacks leave no history rows.

CREATE TABLE IF NOT EXISTS code_history (
  id         bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  rel        text NOT NULL,          -- 'function_defs' | 'type_defs' | 'view_bindings'
  branch_id  uuid NOT NULL,
  name       text NOT NULL,          -- fn/type name or route path
  body       jsonb,                  -- superseded before-image
  author     text,
  doc        text,
  package    text,
  op         text NOT NULL,          -- 'UPDATE' | 'DELETE'
  changed_at timestamptz NOT NULL DEFAULT now()
);

CREATE INDEX IF NOT EXISTS code_history_lookup_idx
  ON code_history (rel, branch_id, name, changed_at DESC);

CREATE OR REPLACE FUNCTION code_history_capture() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
  IF TG_TABLE_NAME = 'function_defs' THEN
    INSERT INTO code_history
      (rel, branch_id, name, body, author, doc, package, op)
    VALUES
      ('function_defs', OLD.branch_id, OLD.name, OLD.body,
       OLD.author, OLD.doc, OLD.package, TG_OP);
  ELSIF TG_TABLE_NAME = 'type_defs' THEN
    INSERT INTO code_history
      (rel, branch_id, name, body, author, doc, package, op)
    VALUES
      ('type_defs', OLD.branch_id, OLD.name, OLD.fields,
       OLD.author, NULL, NULL, TG_OP);
  ELSIF TG_TABLE_NAME = 'view_bindings' THEN
    INSERT INTO code_history
      (rel, branch_id, name, body, author, doc, package, op)
    VALUES
      ('view_bindings', OLD.branch_id, OLD.route,
       jsonb_build_object(
         'handler', OLD.handler,
         'visibility', COALESCE(OLD.visibility, 'private')),
       NULL, NULL, NULL, TG_OP);
  END IF;

  IF TG_OP = 'DELETE' THEN
    RETURN OLD;
  END IF;
  RETURN NEW;
END
$$;

DROP TRIGGER IF EXISTS code_history_function_defs_trg ON function_defs;
CREATE TRIGGER code_history_function_defs_trg
  BEFORE UPDATE OR DELETE ON function_defs
  FOR EACH ROW EXECUTE FUNCTION code_history_capture();

DROP TRIGGER IF EXISTS code_history_type_defs_trg ON type_defs;
CREATE TRIGGER code_history_type_defs_trg
  BEFORE UPDATE OR DELETE ON type_defs
  FOR EACH ROW EXECUTE FUNCTION code_history_capture();

DROP TRIGGER IF EXISTS code_history_view_bindings_trg ON view_bindings;
CREATE TRIGGER code_history_view_bindings_trg
  BEFORE UPDATE OR DELETE ON view_bindings
```

```
  FOR EACH ROW EXECUTE FUNCTION code_history_capture();

INSERT INTO schema_migrations (version, description)
SELECT 12, 'code_history table + BEFORE UPDATE/DELETE triggers on code rows'
WHERE NOT EXISTS (SELECT 1 FROM schema_migrations WHERE version = 12);
```

267 migrations/013_data_user_scoping.sql

```
-- Pricklypear migration 013: user-scoped datum paths + RLS floor.
--
-- Per spec [one-tree.branching-interaction] /data DECISION (2026-08-02)
-- and [one-tree.migration-plan] Phase 2b. Closes BORGE-1785693032.
--
-- Datum paths become:
--   per-user  data.u.<user_enc>.<type_enc>.<id_hex>
--   shared    data.shared.<type_enc>.<id_hex>
-- A session sees exactly its own data.u.<user>.* plus data.shared.*.
-- No branch id in datum paths. data_rows stays physically (additive-only);
-- tree_data_rows view + CRUD prims confine; RLS on tree_nodes is the floor.
--
-- Seed types (visible to every session via data.shared):
--   bible-verse, bible-book, bible-interlinear, strongs-entry
-- All other existing rows backfill under data.u.<author_enc>.* keyed off
-- data_rows.author / tree_nodes.author. Unknown/empty author ->
    data.u.unknown.
--
-- Notes hierarchy (kind folder|note|heading under data.note.u.<user>.*) is
-- NOT kind=datum and is exempted by the RLS kind <> 'datum' clause.

--
    ------------------------------------------------------------------------
-- Helpers: datum id is ALWAYS the last ltree label (uuid hex).
-- Works for data.<type>.<id>, data.u.<user>.<type>.<id>,
    data.shared.<type>.<id>.
--
    ------------------------------------------------------------------------
CREATE OR REPLACE FUNCTION tree_datum_id(p ltree) RETURNS uuid
LANGUAGE plpgsql IMMUTABLE AS $$
DECLARE
  labs text[];
  h text;
BEGIN
  labs := string_to_array(p::text, '.');
  IF labs IS NULL OR array_length(labs, 1) IS NULL THEN
    RETURN NULL;
  END IF;
  h := labs[array_length(labs, 1)];
  IF h IS NULL OR length(h) <> 32 THEN
    RETURN NULL;
  END IF;
  RETURN (substr(h, 1, 8) || '-' || substr(h, 9, 4) || '-' ||
          substr(h, 13, 4) || '-' || substr(h, 17, 4) || '-' ||
          substr(h, 21, 12))::uuid;
END $$;

-- Seed-type predicate (shared corpus). Keep list short and explicit.
CREATE OR REPLACE FUNCTION tree_is_seed_type(t text) RETURNS boolean
LANGUAGE sql IMMUTABLE AS $$
  SELECT t IN (
    'bible-verse',
    'bible-book',
    'bible-interlinear',
    'strongs-entry'
  )
$$;

-- Build the canonical datum path for (type, id, author).
CREATE OR REPLACE FUNCTION tree_datum_path(type_name text, id uuid, author
    text)
RETURNS ltree
LANGUAGE plpgsql IMMUTABLE AS $$
DECLARE
  a text;
BEGIN
  IF tree_is_seed_type(type_name) THEN
    RETURN ('data.shared.' || tree_enc_label(type_name) || '.' ||
            tree_uuid_label(id))::ltree;
  END IF;
  a := COALESCE(NULLIF(author, ''), 'unknown');
```

```
  RETURN ('data.u.' || tree_enc_label(a) || '.' ||
          tree_enc_label(type_name) || '.' ||
          tree_uuid_label(id))::ltree;
END $$;

--
    ------------------------------------------------------------------------
-- Forward mirror: data_rows -> tree_nodes (user/shared path roots).
--
    ------------------------------------------------------------------------
CREATE OR REPLACE FUNCTION tree_mirror_data_rows() RETURNS trigger
LANGUAGE plpgsql AS $$
DECLARE p ltree; body jsonb; h text; env jsonb;
BEGIN
  PERFORM set_config('pp.tree_mirror','1', true);
  IF (TG_OP = 'DELETE') THEN
    -- Delete by id via either old or new path shape (id is last label).
    DELETE FROM tree_nodes
     WHERE kind = 'datum'
       AND tree_datum_id(path) = OLD.id;
    RETURN OLD;
  END IF;
  p := tree_datum_path(NEW.type_name, NEW.id, NEW.author);
  body := NEW.fields;
  h := encode(digest(body::text, 'sha256'), 'hex');
  env := jsonb_build_object(
    'v', 1, 'kind', 'datum', 'path', p::text, 'type', NEW.type_name,
    'body', body, 'author', NEW.author, 'doc', NULL, 'hash', h);
  -- If the row moved path (author/type change), drop the old node first.
  DELETE FROM tree_nodes
   WHERE kind = 'datum'
     AND tree_datum_id(path) = NEW.id
     AND path <> p;
  INSERT INTO tree_nodes (path, kind, type_name, payload, hash, author,
    updated_at)
  VALUES (p, 'datum', NEW.type_name, env, h, NEW.author, now())
  ON CONFLICT (path) DO UPDATE
    SET payload = EXCLUDED.payload, hash = EXCLUDED.hash,
        author = EXCLUDED.author, type_name = EXCLUDED.type_name,
        updated_at = now();
  RETURN NEW;
END $$;

--
    ------------------------------------------------------------------------
-- Reverse mirror: tree_nodes -> data_rows (id = last path label).
--
    ------------------------------------------------------------------------
CREATE OR REPLACE FUNCTION tree_from_nodes() RETURNS trigger
LANGUAGE plpgsql AS $$
DECLARE bid uuid; lbl text; docv text; did uuid;
BEGIN
  IF tree_in_mirror() THEN
    IF TG_OP = 'DELETE' THEN RETURN OLD; ELSE RETURN NEW; END IF;
  END IF;
  IF (TG_OP = 'DELETE') THEN
    bid := tree_path_branch(OLD.path);
    IF OLD.kind = 'fn' THEN
      lbl := (string_to_array(OLD.path::text,'.'))[4];
      DELETE FROM function_defs WHERE branch_id=bid AND
     name=tree_dec_label(lbl);
    ELSIF OLD.kind = 'type' THEN
      lbl := (string_to_array(OLD.path::text,'.'))[4];
      DELETE FROM type_defs WHERE branch_id=bid AND
     name=tree_dec_label(lbl);
    ELSIF OLD.kind = 'route' THEN
      DELETE FROM view_bindings WHERE branch_id=bid AND
     route=OLD.payload->'body'->>'route';
    ELSIF OLD.kind = 'datum' THEN
      did := tree_datum_id(OLD.path);
      IF did IS NOT NULL THEN
        DELETE FROM data_rows WHERE id = did;
      END IF;
```

```
    END IF;
    RETURN OLD;
  END IF;
  docv := NEW.payload->>'doc';
  IF NEW.kind = 'fn' THEN
    bid := tree_path_branch(NEW.path);
    lbl := (string_to_array(NEW.path::text,'.'))[4];
    INSERT INTO function_defs (branch_id,name,body,author,doc)
    VALUES (bid, tree_dec_label(lbl), NEW.payload->'body',
            NEW.payload->>'author', docv)
    ON CONFLICT (branch_id,name) DO UPDATE
      SET body=EXCLUDED.body, author=EXCLUDED.author, doc=EXCLUDED.doc,
          updated_at=now();
  ELSIF NEW.kind = 'type' THEN
    bid := tree_path_branch(NEW.path);
    lbl := (string_to_array(NEW.path::text,'.'))[4];
    INSERT INTO type_defs (branch_id,name,fields,author)
    VALUES (bid, tree_dec_label(lbl), NEW.payload->'body',
     NEW.payload->>'author')
    ON CONFLICT (branch_id,name) DO UPDATE
      SET fields=EXCLUDED.fields, author=EXCLUDED.author, updated_at=now();
  ELSIF NEW.kind = 'route' THEN
    bid := tree_path_branch(NEW.path);
    INSERT INTO view_bindings (branch_id,route,handler,visibility)
    VALUES (bid, NEW.payload->'body'->>'route',
     NEW.payload->'body'->>'handler',
            COALESCE(NEW.payload->'body'->>'visibility','private'))
    ON CONFLICT (branch_id,route) DO UPDATE
      SET handler=EXCLUDED.handler, visibility=EXCLUDED.visibility;
  ELSIF NEW.kind = 'datum' THEN
    did := tree_datum_id(NEW.path);
    IF did IS NOT NULL THEN
      INSERT INTO data_rows (id,type_name,fields,author)
      VALUES (did, NEW.type_name, NEW.payload->'body',
     NEW.payload->>'author')
      ON CONFLICT (id) DO UPDATE
        SET fields=EXCLUDED.fields, author=EXCLUDED.author,
            type_name=EXCLUDED.type_name, updated_at=now();
    END IF;
  END IF;
  RETURN NEW;
END $$;

--
     ------------------------------------------------------------------------
-- Backfill existing datum nodes from old data.<type>.<id> shape.
-- Guard with pp.tree_mirror so reverse trigger does not thrash data_rows.
--
     ------------------------------------------------------------------------
DO $$
DECLARE
  r RECORD;
  new_p ltree;
  moved int := 0;
  shared_n int := 0;
  user_n int := 0;
BEGIN
  PERFORM set_config('pp.tree_mirror', '1', true);
  FOR r IN
    SELECT n.path, n.type_name, n.author, n.payload, n.hash,
     tree_datum_id(n.path) AS id
      FROM tree_nodes n
     WHERE n.kind = 'datum'
  LOOP
    IF r.id IS NULL THEN
      CONTINUE;
    END IF;
    new_p := tree_datum_path(r.type_name, r.id, r.author);
    IF r.path = new_p THEN
      CONTINUE;
    END IF;
    -- Drop any pre-existing target (shouldn't happen) then move.
    DELETE FROM tree_nodes WHERE path = new_p AND path <> r.path;
```

```
    UPDATE tree_nodes
       SET path = new_p,
           payload = jsonb_set(COALESCE(payload, '{}'::jsonb), '{path}',
                               to_jsonb(new_p::text)),
           updated_at = now()
     WHERE path = r.path;
    moved := moved + 1;
    IF tree_is_seed_type(r.type_name) THEN
      shared_n := shared_n + 1;
    ELSE
      user_n := user_n + 1;
    END IF;
  END LOOP;
  RAISE NOTICE '013 backfill: moved=% shared=% user=%', moved, shared_n,
    user_n;
END $$;

-- Ensure every data_rows row has a tree node at the new path (repair gaps).
DO $$
DECLARE
  r RECORD;
  p ltree;
  body jsonb;
  h text;
  env jsonb;
  n int := 0;
BEGIN
  PERFORM set_config('pp.tree_mirror', '1', true);
  FOR r IN SELECT id, type_name, fields, author FROM data_rows LOOP
    p := tree_datum_path(r.type_name, r.id, r.author);
    IF EXISTS (SELECT 1 FROM tree_nodes WHERE path = p) THEN
      CONTINUE;
    END IF;
    -- Remove any stale datum node for this id under an old path.
    DELETE FROM tree_nodes
     WHERE kind = 'datum' AND tree_datum_id(path) = r.id AND path <> p;
    body := r.fields;
    h := encode(digest(body::text, 'sha256'), 'hex');
    env := jsonb_build_object(
      'v', 1, 'kind', 'datum', 'path', p::text, 'type', r.type_name,
      'body', body, 'author', r.author, 'doc', NULL, 'hash', h);
    INSERT INTO tree_nodes (path, kind, type_name, payload, hash, author,
     updated_at)
    VALUES (p, 'datum', r.type_name, env, h, r.author, now())
    ON CONFLICT (path) DO UPDATE
      SET payload = EXCLUDED.payload, hash = EXCLUDED.hash,
          author = EXCLUDED.author, type_name = EXCLUDED.type_name,
          updated_at = now();
    n := n + 1;
  END LOOP;
  RAISE NOTICE '013 repair: inserted/updated % missing datum nodes', n;
END $$;

--
    ------------------------------------------------------------------------
-- tree_data_rows view: id from last label; optional pp.user confinement.
-- When pp.user is unset/empty the view returns ALL datums (scripts/admin).
-- When set, only data.u.<user>.* ? data.shared.* (RLS-aligned).
--
    ------------------------------------------------------------------------
CREATE OR REPLACE VIEW tree_data_rows AS
SELECT tree_datum_id(path) AS id,
       type_name,
       payload->'body' AS fields,
       payload->>'author' AS author,
       created_at, updated_at
FROM tree_nodes
WHERE kind = 'datum'
  AND (
    COALESCE(current_setting('pp.user', true), '') = ''
    OR path <@ ('data.u.' || tree_enc_label(current_setting('pp.user',
     true)))::ltree
    OR path <@ 'data.shared'::ltree
```

```
  );

--
    ------------------------------------------------------------------------
-- RLS floor on tree_nodes. Non-datum kinds unrestricted here (branch lens
-- is the code isolation axis; notes already path-keyed under
    data.note.u.*).
-- App role is typically not the table owner; FORCE covers owner
    connections.
--
    ------------------------------------------------------------------------
ALTER TABLE tree_nodes ENABLE ROW LEVEL SECURITY;
ALTER TABLE tree_nodes FORCE ROW LEVEL SECURITY;

DROP POLICY IF EXISTS datum_user_isolation ON tree_nodes;
CREATE POLICY datum_user_isolation ON tree_nodes
  USING (
    kind <> 'datum'
    OR COALESCE(current_setting('pp.user', true), '') = ''
    OR path <@ ('data.u.' || tree_enc_label(current_setting('pp.user',
    true)))::ltree
    OR path <@ 'data.shared'::ltree
  )
  WITH CHECK (
    kind <> 'datum'
    OR COALESCE(current_setting('pp.user', true), '') = ''
    OR path <@ ('data.u.' || tree_enc_label(current_setting('pp.user',
    true)))::ltree
    OR path <@ 'data.shared'::ltree
  );

-- Bypass for the mirror guard path is unnecessary: triggers run as table
-- owner and FORCE applies, but pp.user='' (unset) opens the policy. Mirror
-- functions set pp.tree_mirror, not pp.user -- unset pp.user keeps mirrors
-- working. Prim code sets pp.user inside request-scoped transactions.

INSERT INTO schema_migrations (version, description)
SELECT 13, 'user-scoped datum paths data.u.<user>.* + data.shared.* + RLS
    floor'
WHERE NOT EXISTS (SELECT 1 FROM schema_migrations WHERE version = 13);
```

268 migrations/014_guestbook_shared.sql

```
-- Pricklypear migration 014: guestbook_entry is shared corpus.
--
-- Follow-up to 013 / BORGE-1785695889. Guestbook is a public wall: every
-- visitor must see every entry. Under user-scoped data, rows authored by
-- with-public-write (author='public') landed under data.u.public.* and
-- were invisible to other sessions. Treat guestbook_entry like bible seed:
-- path data.shared.guestbook_entry.<id>, visible via tree_is_seed_type /
-- CRUD seed allowlist.

CREATE OR REPLACE FUNCTION tree_is_seed_type(t text) RETURNS boolean
LANGUAGE sql IMMUTABLE AS $$
  SELECT t IN (
    'bible-verse',
    'bible-book',
    'bible-interlinear',
    'strongs-entry',
    'guestbook_entry'
  )
$$;

-- Move any existing guestbook datum nodes onto data.shared.*
DO $$
DECLARE
  r RECORD;
  new_p ltree;
  n int := 0;
BEGIN
  PERFORM set_config('pp.tree_mirror', '1', true);
  FOR r IN
    SELECT path, type_name, author, tree_datum_id(path) AS id
      FROM tree_nodes
     WHERE kind = 'datum' AND type_name = 'guestbook_entry'
  LOOP
    IF r.id IS NULL THEN CONTINUE; END IF;
    new_p := tree_datum_path(r.type_name, r.id, r.author);
    IF r.path = new_p THEN CONTINUE; END IF;
    DELETE FROM tree_nodes WHERE path = new_p AND path <> r.path;
    UPDATE tree_nodes
       SET path = new_p,
           payload = jsonb_set(COALESCE(payload, '{}'::jsonb), '{path}',
                               to_jsonb(new_p::text)),
           updated_at = now()
     WHERE path = r.path;
    n := n + 1;
  END LOOP;
  RAISE NOTICE '014 guestbook shared backfill: moved=%', n;
END $$;

-- Repair: ensure every data_rows guestbook row has a shared tree node.
DO $$
DECLARE
  r RECORD;
  p ltree;
  body jsonb;
  h text;
  env jsonb;
  n int := 0;
BEGIN
  PERFORM set_config('pp.tree_mirror', '1', true);
  FOR r IN
    SELECT id, type_name, fields, author
      FROM data_rows
     WHERE type_name = 'guestbook_entry'
  LOOP
    p := tree_datum_path(r.type_name, r.id, r.author);
    DELETE FROM tree_nodes
     WHERE kind = 'datum' AND tree_datum_id(path) = r.id AND path <> p;
    IF EXISTS (SELECT 1 FROM tree_nodes WHERE path = p) THEN
      CONTINUE;
    END IF;
    body := r.fields;
```

```
    h := encode(digest(body::text, 'sha256'), 'hex');
    env := jsonb_build_object(
      'v', 1, 'kind', 'datum', 'path', p::text, 'type', r.type_name,
      'body', body, 'author', r.author, 'doc', NULL, 'hash', h);
    INSERT INTO tree_nodes (path, kind, type_name, payload, hash, author,
     updated_at)
    VALUES (p, 'datum', r.type_name, env, h, r.author, now())
    ON CONFLICT (path) DO UPDATE
      SET payload = EXCLUDED.payload, hash = EXCLUDED.hash,
          author = EXCLUDED.author, type_name = EXCLUDED.type_name,
          updated_at = now();
    n := n + 1;
  END LOOP;
  RAISE NOTICE '014 guestbook repair: inserted=%', n;
END $$;

INSERT INTO schema_migrations (version, description)
SELECT 14, 'guestbook_entry is shared seed corpus (public wall)'
WHERE NOT EXISTS (SELECT 1 FROM schema_migrations WHERE version = 14);
```

269 migrations/015_jobs.sql

```
-- 015_jobs.sql -- durable background jobs (borge/jobs.borg)
-- Dedicated table (not data_rows): the runner's SKIP LOCKED claim protocol
-- needs real status/run_at columns. Jobs live outside branch lenses
    (shared).

CREATE TABLE IF NOT EXISTS jobs (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  fn          text NOT NULL,
  args        jsonb NOT NULL DEFAULT '{}'::jsonb,
  branch      text NOT NULL,
  submitted_by text NOT NULL,
  status      text NOT NULL DEFAULT 'pending',  -- pending | running | done
    | failed | dead
  run_at      bigint NOT NULL DEFAULT 0,         -- epoch-ms earliest start
  attempts    int  NOT NULL DEFAULT 0,
  max_attempts int NOT NULL DEFAULT 3,
  result      jsonb,
  error       text NOT NULL DEFAULT '',
  runner_id   text NOT NULL DEFAULT '',
  created_at  timestamptz NOT NULL DEFAULT now()
);

-- Claim index: due pending jobs, oldest first.
CREATE INDEX IF NOT EXISTS jobs_claim_idx
  ON jobs (run_at, id) WHERE status = 'pending';

CREATE INDEX IF NOT EXISTS jobs_submitted_by_idx
  ON jobs (submitted_by, created_at DESC);

CREATE INDEX IF NOT EXISTS jobs_status_running_idx
  ON jobs (runner_id) WHERE status = 'running';

-- job_steps: checkpoint rows for (step) memoization (borge/jobs.borg
    step-checkpoints).
CREATE TABLE IF NOT EXISTS job_steps (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  job_id      uuid NOT NULL REFERENCES jobs(id) ON DELETE CASCADE,
  step_name   text NOT NULL,
  value       jsonb NOT NULL DEFAULT 'null'::jsonb,
  finished_at timestamptz NOT NULL DEFAULT now(),
  UNIQUE (job_id, step_name)
);

-- schedules: cron / every-ms (borge/jobs.borg schedules).
CREATE TABLE IF NOT EXISTS schedules (
  name        text PRIMARY KEY,
  fn          text NOT NULL,
  args        jsonb NOT NULL DEFAULT '{}'::jsonb,
  branch      text NOT NULL,
  submitted_by text NOT NULL,
  cron        text,           -- 5-field cron, or NULL
  every_ms    bigint,         -- interval, or NULL
  next_run    bigint NOT NULL DEFAULT 0,  -- epoch-ms
  enabled     boolean NOT NULL DEFAULT true
);

INSERT INTO schema_migrations (version, description)
SELECT 15, 'durable background jobs (jobs, job_steps, schedules)'
WHERE NOT EXISTS (SELECT 1 FROM schema_migrations WHERE version = 15);
```

270 migrations/016_seed_single_source.sql

```
-- Pricklypear migration 016: single source of truth for the seed-type list.
--
-- Seed corpus types (shared under data.shared.*, visible to every session)
-- were previously listed in three places and drifted (see log/issues/
-- i-seed-list-3copies-2026-08-12): the SQL predicate tree_is_seed_type, the
-- crud.ml visible_sql IN-list, and the OCaml Tree_store.seed_types array.
-- Migration 013/014 already moved these into one SQL predicate; this step
-- makes THAT predicate derive from a single authoritative array function,
-- tree_seed_types(). The OCaml Tree_store.seed_types constant stays as the
-- pure mirror needed by the DB-free `node-path` prim; a boot-time check
-- (Tree_store.verify_seed_types) flags any divergence from this authority.

-- The ONE place the type-name list lives on the server side:
CREATE OR REPLACE FUNCTION tree_seed_types() RETURNS text[]
LANGUAGE sql IMMUTABLE AS $$
  SELECT ARRAY[
    'bible-verse',
    'bible-book',
    'bible-interlinear',
    'strongs-entry',
    'guestbook_entry'
  ]
$$;

-- Predicate now derives from the single authority above.
CREATE OR REPLACE FUNCTION tree_is_seed_type(t text) RETURNS boolean
LANGUAGE sql IMMUTABLE AS $$
  SELECT t = ANY(tree_seed_types())
$$;

INSERT INTO schema_migrations (version, description)
SELECT 16, 'single source of truth for seed-type list (tree_seed_types)'
WHERE NOT EXISTS (SELECT 1 FROM schema_migrations WHERE version = 16);
```

271 migrations/017_branch_generation.sql

```
-- Pricklypear migration 017: branch generation counter for definition-cache
-- invalidation (remote-eval.borg caching subsection).
--
-- The remote Nopales client caches function/type/route defs locally under
-- a (branch_id, generation) key. A write anywhere in a branch's ancestry
-- must bump a monotonic generation counter so the client's next cheap
-- cache-check round trip observes the bump and reloads. This is
-- materialized-view invalidation in small costume: ONE counter per branch,
-- not a hash per def.
--
-- We use a small branch_generations table keyed by branch_id (kept apart
-- from the core branches row and the one-tree lens registry). Triggers on
-- the branch-scoped code tables (function_defs / type_defs /
-- view_bindings) bump the generation of the branch actually written. The
-- reverse one-tree mirror (tree_nodes -> scaffolding tables, migration 009)
-- shares these tables, so a tree-as-truth write still flows through these
-- triggers. promote/merge write function_defs/view_bindings, so those
-- bumps are covered by the same triggers (no separate promote/merge hook).
--
-- data_rows are BRANCH-FREE / shared across every branch (migration 003
-- branching.data). A data write is therefore visible to every branch's
-- ancestry, so we bump the ROOT branch(es) (parent_id IS NULL); the
-- ancestry-walking current_generation query then surfaces that bump to
-- every descendant. (The client does not cache hot data_rows -- placement
-- keeps those remote -- but the counter remains truthful for the shared
-- data plane.)

CREATE TABLE IF NOT EXISTS branch_generations (
  branch_id  uuid PRIMARY KEY REFERENCES branches(id) ON DELETE CASCADE,
  generation bigint NOT NULL DEFAULT 0
);

-- Seed a row for every existing branch so lookups are always non-null.
INSERT INTO branch_generations (branch_id, generation)
SELECT id, 0 FROM branches
ON CONFLICT (branch_id) DO NOTHING;

-- Bump one branch's counter (upsert to 1 then increment).
CREATE OR REPLACE FUNCTION branch_gen_bump(bid uuid) RETURNS void
LANGUAGE plpgsql AS $$
BEGIN
  IF bid IS NULL THEN RETURN; END IF;
  INSERT INTO branch_generations (branch_id, generation)
  VALUES (bid, 1)
  ON CONFLICT (branch_id)
    DO UPDATE SET generation = branch_generations.generation + 1;
END $$;

-- Bump the root branch(es) for the shared (branch-free) data plane, so a
-- data write is visible to every branch's ancestry via the CTE walk.
CREATE OR REPLACE FUNCTION branch_gen_bump_root() RETURNS void
LANGUAGE plpgsql AS $$
BEGIN
  PERFORM branch_gen_bump(b.id)
  FROM branches b WHERE b.parent_id IS NULL;
END $$;

-- Trigger-compatible wrapper (STATEMENT triggers must return trigger).
CREATE OR REPLACE FUNCTION branch_gen_bump_root_trg() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
  PERFORM branch_gen_bump_root();
  RETURN NULL;
END $$;

--
    ------------------------------------------------------------------------
-- Per-statement settlement: AFTER ROW triggers accumulate the affected
-- branch_ids into branch_gen_pending; the AFTER STATEMENT trigger bumps
-- each distinct one once and clears the accumulation. This keeps a
-- multi-row statement (merge / promote) to a single bump, and works
```

```
-- whether the row trigger sees NEW (insert/update) or OLD (delete).
--
    ------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS branch_gen_pending (
  pending_id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  branch_id  uuid NOT NULL
);

CREATE OR REPLACE FUNCTION branch_gen_accumulate() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
  IF TG_OP = 'DELETE' THEN
    INSERT INTO branch_gen_pending(branch_id) VALUES (OLD.branch_id);
  ELSE
    INSERT INTO branch_gen_pending(branch_id) VALUES (NEW.branch_id);
  END IF;
  RETURN COALESCE(NEW, OLD);
END $$;

CREATE OR REPLACE FUNCTION branch_gen_settle() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
  PERFORM branch_gen_bump(branch_id)
  FROM (SELECT DISTINCT branch_id FROM branch_gen_pending) p;
  DELETE FROM branch_gen_pending;
  RETURN NULL;
END $$;

-- function_defs (branch-scoped code writes incl.
    promote/merge/reverse-mirror)
DROP TRIGGER IF EXISTS branch_gen_function_accum_trg ON function_defs;
CREATE TRIGGER branch_gen_function_accum_trg
  AFTER INSERT OR UPDATE OR DELETE ON function_defs
  FOR EACH ROW EXECUTE FUNCTION branch_gen_accumulate();
DROP TRIGGER IF EXISTS branch_gen_function_settle_trg ON function_defs;
CREATE TRIGGER branch_gen_function_settle_trg
  AFTER INSERT OR UPDATE OR DELETE ON function_defs
  FOR EACH STATEMENT EXECUTE FUNCTION branch_gen_settle();

-- type_defs
DROP TRIGGER IF EXISTS branch_gen_type_accum_trg ON type_defs;
CREATE TRIGGER branch_gen_type_accum_trg
  AFTER INSERT OR UPDATE OR DELETE ON type_defs
  FOR EACH ROW EXECUTE FUNCTION branch_gen_accumulate();
DROP TRIGGER IF EXISTS branch_gen_type_settle_trg ON type_defs;
CREATE TRIGGER branch_gen_type_settle_trg
  AFTER INSERT OR UPDATE OR DELETE ON type_defs
  FOR EACH STATEMENT EXECUTE FUNCTION branch_gen_settle();

-- view_bindings (routes)
DROP TRIGGER IF EXISTS branch_gen_route_accum_trg ON view_bindings;
CREATE TRIGGER branch_gen_route_accum_trg
  AFTER INSERT OR UPDATE OR DELETE ON view_bindings
  FOR EACH ROW EXECUTE FUNCTION branch_gen_accumulate();
DROP TRIGGER IF EXISTS branch_gen_route_settle_trg ON view_bindings;
CREATE TRIGGER branch_gen_route_settle_trg
  AFTER INSERT OR UPDATE OR DELETE ON view_bindings
  FOR EACH STATEMENT EXECUTE FUNCTION branch_gen_settle();

-- data_rows (shared data plane -> bump the ROOT branch(es) so every
-- branch's ancestry sees the change).
DROP TRIGGER IF EXISTS branch_gen_data_row_trg ON data_rows;
CREATE TRIGGER branch_gen_data_row_trg
  AFTER INSERT OR UPDATE OR DELETE ON data_rows
  FOR EACH STATEMENT EXECUTE FUNCTION branch_gen_bump_root_trg();

--
    ------------------------------------------------------------------------
-- Current-generation query used by the decorated inheritance CTE: the max
-- generation across the branch's ancestry chain. Bumping an ancestor (e.g.
-- main) is automatically visible to every descendant.
--
    ------------------------------------------------------------------------
```

```
CREATE OR REPLACE FUNCTION current_branch_generation(bid uuid) RETURNS
    bigint
LANGUAGE sql STABLE AS $$
  WITH RECURSIVE chain AS (
    SELECT id, parent_id FROM branches WHERE id = bid
    UNION ALL
    SELECT b.id, b.parent_id FROM branches b
    JOIN chain c ON b.id = c.parent_id
  )
  SELECT COALESCE(MAX(bg.generation), 0)
    FROM chain c
    LEFT JOIN branch_generations bg ON bg.branch_id = c.id
$$;

INSERT INTO schema_migrations (version, description)
SELECT 17, 'branch generation counter + triggers for def-cache invalidation'
WHERE NOT EXISTS (SELECT 1 FROM schema_migrations WHERE version = 17);
```

272 migrations/018_tree_versions.sql

```
-- Pricklypear migration 018: tree-time versioned-leaf table.
--
-- Per spec [tree-time.migration-018] (campaign 5.0 plan pass, pinned
-- schema contract). This ADDS the append-only history table for the
-- generation axis on the one tree: (branch_id, path, gen) -> node. It is
-- pure Phase 1 additive foundation -- nothing reads it yet (the scaffolding
-- mirrors + tree_nodes stay the source of truth through Phase 2's
-- flag-guarded read switch), and the write path only appends version rows
-- once tree_store.ml (5.1) grows its dual-write shadow.
--
-- Path grammar matches tree_nodes (migration 009):
--   fn     b.<branch_id>.fn.<name>
--   type   b.<branch_id>.type.<name>
--   route  b.<branch_id>.route.<route_b64url>
--   datum  data.<type_name>.<id>          (branch-free)
--
-- This file is strictly ADDITIVE + idempotent + image-applied (registered
-- in schema_migrations so the runner will not re-apply).

-- tree_generations: the TREE-WIDE data head counter (branch-free paths).
-- code paths (b.*) use branch_generations (017) as their head instead, so
-- this row serves only branch-free data.* writes. One monotonic bigint;
-- the 017 root-row bump discipline generalized to one row (the
-- local-first.sync merge clock).
CREATE TABLE IF NOT EXISTS tree_generations (
  generation bigint NOT NULL DEFAULT 0
);
INSERT INTO tree_generations (generation)
SELECT 0 WHERE NOT EXISTS (SELECT 1 FROM tree_generations);

--
    ------------------------------------------------------------------------
-- tree_versions -- append-only history, one row per (branch_id, path, gen)
-- write. update/delete never rewrite: they APPEND a new gen row. Delete
-- appends a TOMBSTONE row (a row, never a DELETE) so undo-of-delete /
-- audit-by-state-diff are free.
--
    ------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS tree_versions (
  branch_id  uuid         NULL,     -- NULL for branch-free data.* paths
  path       ltree        NOT NULL, -- same encoding as tree_nodes
  gen        bigint       NOT NULL,
  kind       text         NOT NULL, -- fn | type | route | datum
  type_name  text,                  -- datum only
  body       jsonb        NOT NULL, -- node envelope; TOMBSTONE = reserved
    delete marker
  hash       text         NOT NULL, -- sha256 over canonical body
    (diff-first prune key)
  author     text         NOT NULL,
  created_at timestamptz  NOT NULL DEFAULT now(),  -- write time; immutable
    per version row
  -- Append-only uniqueness = one row per (path, gen). `path` ALREADY
  -- encodes the branch (b.<branch hex>.* for code paths, globally-unique
  -- data.<type>.<id> for data), so (path, gen) is unique across every
  -- branch -- and it permits branch_id to be NULL on branch-free data.*
  -- rows (a composite PK including branch_id would force branch_id NOT
  -- NULL, which the data plane forbids). branch_id is a nullable
  -- denormalized copy for code-branch reads only.
  PRIMARY KEY (path, gen),
  -- code paths (path under the `b` root label) MUST carry their owning
  -- branch; data.* paths are branch-free by the one-tree /data DECISION.
  -- (ltree: path <@ 'b' is true when path is a descendant of the root `b`.)
  CONSTRAINT tree_versions_code_owns_branch CHECK (
    NOT (path <@ 'b'::ltree) OR branch_id IS NOT NULL
  )
) PARTITION BY RANGE (gen);

-- Partitioning note: the spec's migration-018 pin says "RANGE PARTITIONED
-- BY list(gen/1000)". PostgreSQL forbids a PRIMARY KEY (or any unique
-- index) on a partitioned table whose partition key is an EXPRESSION, so we
```

```
-- partition by RANGE (gen) directly with 1000-gen windows instead -- identical
-- retention semantics (DROP PARTITION per gen window) and it permits the
-- (branch_id, path, gen) primary key. The 1000-gen window is the retention
-- granularity; the base window covers gens below 1000, and the retention
-- worker (tree-time gc) creates further windows lazily.
CREATE TABLE IF NOT EXISTS tree_versions_gen_0 PARTITION OF tree_versions
  FOR VALUES FROM (MINVALUE) TO (1000);

-- Covering index for one-pass head/projection reads: an index range scan
-- returns the newest gen per path with no join. Note: the pk index is
-- (branch_id, path, gen) -- this dedicated b-tree adds (branch_id, path,
-- gen DESC) so a DESC scan hits the newest row first.
CREATE INDEX IF NOT EXISTS tree_versions_head_idx
  ON tree_versions (branch_id, path, gen DESC);

-- ---------------------------------------------------------------------------
-- Head + tombstone helpers. A path is ALIVE at head when its newest row at
-- or below head is NOT the TOMBSTONE marker. These are the SQL-level reads
-- Phase 2 wiring (tree_store/crud) will drive through the gen filter.
-- ---------------------------------------------------------------------------
CREATE OR REPLACE FUNCTION tree_versions_tombstone() RETURNS jsonb
LANGUAGE plpgsql IMMUTABLE AS $$
DECLARE marker text := '{"_tombstone":true}';
BEGIN
  RETURN marker::jsonb;
END $$;

-- The newest gen for (branch, path) at or below `at_gen` (inclusive).
-- NULL row = path never written; returns the tombstone marker body if the
-- newest resolved row is a delete.
CREATE OR REPLACE FUNCTION tree_versions_head(
  p_branch uuid, p_path ltree, at_gen bigint DEFAULT NULL
) RETURNS TABLE(gen bigint, body jsonb, hash text, author text, created_at timestamptz)
LANGUAGE sql STABLE AS $$
  SELECT v.gen, v.body, v.hash, v.author, v.created_at
  FROM tree_versions v
  WHERE v.branch_id IS NOT DISTINCT FROM p_branch
    AND v.path = p_path
    AND (at_gen IS NULL OR v.gen <= at_gen)
  ORDER BY v.gen DESC
  LIMIT 1
$$;

-- All leaves under p_path alive at head (or at at_gen), newest row per
-- path first (projection reads). `gen <= at_gen` when given; otherwise head.
CREATE OR REPLACE FUNCTION tree_versions_leaves(
  p_branch uuid, p_path ltree, at_gen bigint DEFAULT NULL
) RETURNS TABLE(path ltree, gen bigint, body jsonb, hash text, author text, created_at timestamptz)
LANGUAGE sql STABLE AS $$
  SELECT DISTINCT ON (v.path) v.path, v.gen, v.body, v.hash, v.author, v.created_at
  FROM tree_versions v
  WHERE v.branch_id IS NOT DISTINCT FROM p_branch
    AND v.path <@ p_path
    AND (at_gen IS NULL OR v.gen <= at_gen)
  ORDER BY v.path, v.gen DESC
$$;

INSERT INTO schema_migrations (version, description)
SELECT 18, 'tree_versions append-only versioned-leaf table + tree_generations head counter (tree-time)'
WHERE NOT EXISTS (SELECT 1 FROM schema_migrations WHERE version = 18);
```

273 migrations/019_tree_version_shadow.sql

```
-- Pricklypear migration 019: tree-time Phase 1 dual-write shadow.
--
-- tree-time.versioned-leaves (5.1): the write path appends a version row
-- to tree_versions (migration 018) IN ADDITION to every tree_nodes write.
-- The dual-write is implemented as an AFTER trigger on tree_nodes -- not as
-- per-function appends inside tree_store.ml -- for ONE decisive reason:
-- MIRROR PRESERVATION (one-tree.bl-blast-radius / this chapter's
-- versioned-leaves details doc). A scaffolding-table writer (promote/merge
-- writing function_defs/type_defs/view_bindings/data_rows) reaches the tree
-- through the 009_tree.sql tree_mirror_* triggers; a trigger on tree_nodes
-- shadows EVERY such write source (tree_store.ml direct writes AND mirror
-- writes AND reverse-mirror writes) with a single, uniform mechanism. A
-- tree_store.ml-only dual-write would have to be re-wired through every
-- mirror trigger to keep the scaffolding-writer and tree-writer histories in
-- lockstep; the node trigger gets that for free.
--
-- Semantics (per tree-time.migration-018):
--   - gen source per path class: b.<branch>.* code paths bump
--     branch_generations (017, per-branch); branch-free data.* paths bump
--     the tree-wide tree_generations row (018). The append takes gen+1 and
--     bumps the counter -- the counter never moves backwards (verify script
--     check #4).
--   - delete never DELETEs: it appends a TOMBSTONE row. The tombstone body
--     uses the SAME reserved marker tree_versions_tombstone() returns
--     ({"_tombstone":true}) so head helpers and readers agree on one marker.
--   - gen-window partitions (>=1000) are created lazily via
--     tree_versions_prepare(); the base window tree_versions_gen_0 comes
--     from migration 018.
--
-- This migration is STRICTLY ADDITIVE + idempotent + image-applied
-- (registered in schema_migrations so the runner will not re-apply).
-- It is the shadow only; nothing reads tree_versions until Phase 2
-- (reads-as-of-gen, tree-time.reads-as-of-gen) flips the PP_* flag.

-- Lazily create the gen-window partition for [gen] (windows >= 1000 only;
-- the base window gen_0 exists from 018). The retention worker (gc) also
-- uses this to pre-create windows. Skipping wn=0 avoids an overlap with
-- tree_versions_gen_0 (partition-of = tree_versions FOR VALUES FROM
-- MINVALUE TO (1000); a partition FOR VALUES (0,1000) would collide).
CREATE OR REPLACE FUNCTION tree_versions_prepare(gen bigint)
RETURNS void
LANGUAGE plpgsql
AS $function$
DECLARE
  wn bigint := gen / 1000;
  lo bigint := wn * 1000;
  hi bigint := lo + 1000;
BEGIN
  IF wn < 1 THEN RETURN; END IF;
  IF to_regclass(format('tree_versions_p%s', wn)) IS NOT NULL THEN RETURN; END IF;
  BEGIN
    EXECUTE format(
      'CREATE TABLE tree_versions_p%s PARTITION OF tree_versions FOR VALUES FROM (%s) TO (%s)',
      wn, lo, hi);
  EXCEPTION WHEN duplicate_table THEN NULL;  -- lost the race; window exists
  END;
END $function$;

-- The node-level AFTER trigger body: derive the owning branch, bump the
-- right head counter, auto-create the gen-window, and append a version row
-- (TOMBSTONE on delete). Runs per row on INSERT/UPDATE/DELETE of tree_nodes.
CREATE OR REPLACE FUNCTION tree_version_append()
RETURNS trigger
LANGUAGE plpgsql
AS $function$
DECLARE
```

```
  bid  uuid;
  ngen bigint;
BEGIN
  -- Owning branch for code paths; NULL for branch-free data.*.
  bid := tree_path_branch(COALESCE(NEW.path, OLD.path));

  -- Allocate the next gen atomically, per path class.
  IF bid IS NOT NULL THEN
    INSERT INTO branch_generations (branch_id, generation)
    VALUES (bid, 1)
    ON CONFLICT (branch_id)
      DO UPDATE SET generation = branch_generations.generation + 1
    RETURNING generation INTO ngen;
  ELSE
    UPDATE tree_generations SET generation = generation + 1
    RETURNING generation INTO ngen;
    IF ngen IS NULL THEN
      INSERT INTO tree_generations (generation) VALUES (1)
      RETURNING generation INTO ngen;
    END IF;
  END IF;

  PERFORM tree_versions_prepare(ngen);

  IF TG_OP = 'DELETE' THEN
    -- TOMBSTONE: reserved marker body (same `_tombstone` key
    -- tree_versions_tombstone() returns), never a DELETE.
    INSERT INTO tree_versions
      (branch_id, path, gen, kind, type_name, body, hash, author,
     created_at)
    VALUES
      ( bid, OLD.path, ngen, OLD.kind, OLD.type_name,
        jsonb_build_object(
          'v', 1, 'kind', OLD.kind, 'path', OLD.path::text,
          'type', OLD.type_name, 'body', JSON 'null',
          'author', OLD.author, 'doc', JSON 'null',
          'hash', encode(digest('__pp_tombstone__', 'sha256'), 'hex'),
          '_tombstone', true),
        encode(digest('__pp_tombstone__', 'sha256'), 'hex'),
        OLD.author, now() );
    RETURN OLD;
  END IF;

  -- INSERT / UPDATE: record the new head payload at gen. An
  -- INSERT .. ON CONFLICT (path) DO UPDATE fires this once as an UPDATE
  -- (with NEW) -- a single version row per logical write, no dupes.
  INSERT INTO tree_versions
    (branch_id, path, gen, kind, type_name, body, hash, author, created_at)
  VALUES
    ( bid, NEW.path, ngen, NEW.kind, NEW.type_name,
      NEW.payload, NEW.hash, NEW.author, now() );
  RETURN NEW;
END $function$;

-- Dual-write shadow: AFTER each tree_nodes write, append its version row.
DROP TRIGGER IF EXISTS tree_version_append_trg ON tree_nodes;
CREATE TRIGGER tree_version_append_trg
  AFTER INSERT OR DELETE OR UPDATE ON tree_nodes
  FOR EACH ROW EXECUTE FUNCTION tree_version_append();

INSERT INTO schema_migrations (version, description)
SELECT 19, 'tree-time Phase 1 dual-write shadow: tree_version_append
     trigger on tree_nodes'
WHERE NOT EXISTS (SELECT 1 FROM schema_migrations WHERE version = 19);
```

274 migrations/020_tree_version_echo_guard.sql

```
-- Pricklypear migration 020: tree-time mirror-echo guard for tree_version_append.
--
-- tree-time.versioned-leaves (5.1/5.2): the 019 AFTER-trigger on tree_nodes
-- shadows every tree_nodes write into tree_versions. But the one-tree mirror
-- arrival means a single LOGICAL write can touch tree_nodes TWICE:
--
--   tree-first writer -> tree_nodes (authoritative, gen N)
--     -> tree_from_nodes (reverse mirror, deferred) -> scaffolding table (*_defs /
--        data_rows) -> tree_mirror_* (forward mirror, constraint) -> tree_nodes echo
--        (same content, recomputed hash; pg.tree_mirror guard stops the ping-pong)
--
-- The echo carries the same content body/author/kind/type_name (only the derived
-- `hash` -- sha256 over a differently-normalized body text -- is recomputed by the
-- mirror). Without a guard, 019 double-counts every logical write as two (path, gen)
-- rows and bumps the head counter 2x.
--
-- This migration REFINES tree_version_append(): on UPDATE, when the new row's
-- content (kind, type_name IS NOT DISTINCT, author, payload->'body') is unchanged
-- from OLD, it is a no-op mirror echo -- return NEW BEFORE allocating a gen or
-- inserting a version row. This is the "mirror head-gen filter" of
-- tree-time.projections (every appended gen is a real new head; echoes are not
-- versions). Authoritative writes -- tree-first (store writes tree_nodes directly)
-- AND table-first (a scaffolding write mirrored INTO tree_nodes with changed
-- content) -- always change body/author/kind/type and land exactly one version row.
--
-- Additive: CREATE OR REPLACE on the 019 function; no table changes. Idempotent via
-- the 019 trigger (unaffected) -- re-applying this only re-replaces the function.

-- The node-level AFTER trigger body, with the mirror-echo no-op guard.
CREATE OR REPLACE FUNCTION tree_version_append()
RETURNS trigger
LANGUAGE plpgsql
AS $function$
DECLARE
  bid  uuid;
  ngen bigint;
BEGIN
  -- A MIRROR NO-OP WRITE-BACK must not allocate a gen or a version row:
  -- the reverse mirror (tree_from_nodes) synced this tree node into a
  -- scaffolding table whose forward mirror (tree_mirror_* with the
  -- pp.tree_mirror guard) wrote the SAME CONTENT back to tree_nodes. The
  -- mirror recomputes the derived `hash` field (a consistency artifact, not
  -- content), so we compare the CONTENT BODY + author + kind + type, NOT
  -- the hash: a spurious hash delta is still a mirror echo, not a new
  -- write. Content is unchanged, so versioning it would double-count every
  -- logical write as two (path, gen) rows and bump gens 2x. Authoritative
  -- writes -- tree-first (store writes tree_nodes directly) AND table-first
  -- (a scaffolding write mirrored INTO tree_nodes with changed content) --
  -- always change the body/author and land exactly one version row here.
  -- Only the no-op mirror echo drops, BEFORE gen allocation so the head
  -- counter stays aligned with the rows it actually owns.
  IF TG_OP = 'UPDATE' AND NEW.kind = OLD.kind
     AND NEW.type_name IS NOT DISTINCT FROM OLD.type_name
     AND NEW.author = OLD.author
```

```
     AND NEW.payload->'body' = OLD.payload->'body' THEN
    RETURN NEW;
  END IF;

  -- Owning branch for code paths; NULL for branch-free data.*.
  bid := tree_path_branch(COALESCE(NEW.path, OLD.path));

  -- Allocate the next gen atomically, per path class.
  IF bid IS NOT NULL THEN
    INSERT INTO branch_generations (branch_id, generation)
    VALUES (bid, 1)
    ON CONFLICT (branch_id)
      DO UPDATE SET generation = branch_generations.generation + 1
    RETURNING generation INTO ngen;
  ELSE
    UPDATE tree_generations SET generation = generation + 1
    RETURNING generation INTO ngen;
    IF ngen IS NULL THEN
      INSERT INTO tree_generations (generation) VALUES (1)
      RETURNING generation INTO ngen;
    END IF;
  END IF;

  PERFORM tree_versions_prepare(ngen);

  IF TG_OP = 'DELETE' THEN
    -- TOMBSTONE: reserved marker body (same `_tombstone` key
    -- tree_versions_tombstone() returns), never a DELETE.
    INSERT INTO tree_versions
      (branch_id, path, gen, kind, type_name, body, hash, author,
     created_at)
    VALUES
      ( bid, OLD.path, ngen, OLD.kind, OLD.type_name,
        jsonb_build_object(
          'v', 1, 'kind', OLD.kind, 'path', OLD.path::text,
          'type', OLD.type_name, 'body', JSON 'null',
          'author', OLD.author, 'doc', JSON 'null',
          'hash', encode(digest('__pp_tombstone__', 'sha256'), 'hex'),
          '_tombstone', true),
        encode(digest('__pp_tombstone__', 'sha256'), 'hex'),
        OLD.author, now() );
    RETURN OLD;
  END IF;

  -- INSERT / UPDATE: record the new head payload at gen. An
  -- INSERT .. ON CONFLICT (path) DO UPDATE fires this once as an UPDATE
  -- (with NEW) when it changes content: a single version row per logical
  -- write, no dupes.
  INSERT INTO tree_versions
    (branch_id, path, gen, kind, type_name, body, hash, author, created_at)
  VALUES
    ( bid, NEW.path, ngen, NEW.kind, NEW.type_name,
      NEW.payload, NEW.hash, NEW.author, now() );
  RETURN NEW;
END $function$;

INSERT INTO schema_migrations (version, description)
SELECT 20, 'tree-time mirror-echo guard: tree_version_append suppresses
     no-op mirror echoes'
WHERE NOT EXISTS (SELECT 1 FROM schema_migrations WHERE version = 20);
```

275 migrations/021__tree__versions__reconcile.sql

```
-- Pricklypear migration 021: tree-time head reconciliation tool.
--
-- tree-time.phase-1 (5.1/5.2): the dual-write shadow (019 trigger) writes
-- a tree_versions row per tree_nodes write, including a TOMBSTONE on delete.
-- A delete-then-recreate cycle whose re-create did NOT re-shadow (historical
-- trigger-iteration residue, or a writer that inserted tree_nodes without the
-- append trigger attached) leaves a LIVE tree_nodes row beneath a TOMBSTONE
-- newest version row: the versioned head says ABSENT while the live row says
-- PRESENT. Per tree-time.projections, that is NOT head-equivalent and the
-- Phase-2 read switch must not run until it is.
--
-- This migration ADDS an idempotent reconciliation function (no data is
-- destroyed, nothing is rewritten in place) that heals exactly that class:
-- for every shadowed LIVE tree_nodes row whose newest version row is a
-- TOMBSTONE, it appends a FRESH non-tombstone version row holding the LIVE
-- tree_nodes content (the live row is the authority), allocating the head
-- gen exactly as the 019 trigger does (branch_generations for b.* code paths,
-- tree_generations for branch-free data.* paths, gen-window partition lazily
-- created). The live tree_nodes row is untouched. Idempotent: a healed row no
-- longer matches the cursor, so rerunning it heals zero.
--
-- It ALSO repairs head COUNTERS (
--branch_generations / tree_generations): a branch whose
-- branch_generations.generation is below the max live code-gen of its own
-- live code version rows is a head-coordinate regression (the next write
-- would collide or regress); the counter is raised to that max. This is the
-- same restore-by-repair discipline, and is what makes the Phase-1 verify
-- gate's head-counter check pass after a reconcile.
--
-- Run it, then the Phase-1 verify gate
-- (scripts/verify_tree_version_shadow.sh) must PASS.
--
-- Additive (CREATE OR REPLACE FUNCTION only) + idempotent + image-applied.

CREATE OR REPLACE FUNCTION tree_versions_reconcile_heads()
RETURNS bigint                                -- number of paths healed
LANGUAGE plpgsql
AS $function$
DECLARE
  healed bigint := 0;
  rec record;
  bid  uuid;
  ngen bigint;
BEGIN
  -- (1) Heal tombstone-head-but-live rows: append live content as the new head.
  FOR rec IN
    SELECT tn.path, tn.kind, tn.type_name, tn.payload, tn.hash, tn.author
    FROM tree_nodes tn
    WHERE EXISTS (SELECT 1 FROM tree_versions v0 WHERE v0.path = tn.path)
      AND (SELECT (body->>'_tombstone')::text FROM tree_versions v
           WHERE v.path = tn.path ORDER BY v.gen DESC LIMIT 1) = 'true'
  LOOP
    -- Allocate the next head gen per path class, exactly like tree_version_append().
    bid := tree_path_branch(rec.path);
    IF bid IS NOT NULL THEN
      INSERT INTO branch_generations (branch_id, generation)
      VALUES (bid, 1)
      ON CONFLICT (branch_id)
        DO UPDATE SET generation = branch_generations.generation + 1
      RETURNING generation INTO ngen;
    ELSE
      UPDATE tree_generations SET generation = generation + 1
```

```
      RETURNING generation INTO ngen;
      IF ngen IS NULL THEN
        INSERT INTO tree_generations (generation) VALUES (1)
        RETURNING generation INTO ngen;
      END IF;
    END IF;

    PERFORM tree_versions_prepare(ngen);

    -- Append the LIVE content as a fresh non-tombstone head row (restore-by-
    -- append, the same lossless pattern undo uses). The live row is authority.
    INSERT INTO tree_versions
      (branch_id, path, gen, kind, type_name, body, hash, author, created_at)
    VALUES
      ( bid, rec.path, ngen, rec.kind, rec.type_name,
        rec.payload, rec.hash, rec.author, now() );
    healed := healed + 1;
  END LOOP;

  -- (2) Repair head counters: a branch_generations counter below the max live
  --     code-gen of its own live code version rows, or a tree_generations
  --     counter below the max data-gen, is a head-coordinate regression. Raise
  --     to the max. Idempotent (GREATEST with the current value).
  UPDATE branch_generations bg
  SET generation = GREATEST(bg.generation, sub.max_gen)
  FROM (SELECT v.branch_id, max(v.gen) AS max_gen
        FROM tree_versions v
        WHERE v.branch_id IS NOT NULL
          AND EXISTS (SELECT 1 FROM tree_nodes tn WHERE tn.path = v.path)
        GROUP BY v.branch_id) sub
  WHERE sub.branch_id = bg.branch_id
    AND bg.generation < sub.max_gen;

  UPDATE tree_generations tg
  SET generation = GREATEST(tg.generation, sub.max_gen)
  FROM (SELECT max(v.gen) AS max_gen
        FROM tree_versions v
        WHERE v.branch_id IS NULL
          AND EXISTS (SELECT 1 FROM tree_nodes tn WHERE tn.path = v.path)) sub;

  RETURN healed;
END $function$;

INSERT INTO schema_migrations (version, description)
SELECT 21, 'tree-time head reconciliation: tree_versions_reconcile_heads() heals tombstone-head-but-live rows'
WHERE NOT EXISTS (SELECT 1 FROM schema_migrations WHERE version = 21);
```

276 migrations/022_tree_retention.sql

```
-- Pricklypear migration 022: tree-time GC -- retention policy + retention worker.
--
-- tree-time.gc (5.3): retention is the three existing pieces composed:
--   (1) PARTITIONING -- tree_versions is already partitioned by gen-range windows (018 base gen_0 + lazily-created tree_versions_p{wn} windows
--       >= 1000 from 019). Physical removal is DROP PARTITION -- instant, no
--       vacuum scan, PG-native.
--   (2) RETENTION POLICY IS HABITAT DATA -- this migration adds a policy
--       TABLE (tree_retention_policy). Policy rows declare per-path-class
--       retention (code paths keep all -- the audit trail; data paths per-type,
--       default keep-everything until a policy row exists). The jobs runner
--       (migration 015) executes the retention worker.
--   (3) ROLLUP, NOT SILENT LOSS -- before a window is dropped, surviving
--       state collapses to a CHECKPOINT row at the window boundary gen, so
--       reads as of ancient gens still resolve (to the checkpointed state)
--       and head reads never notice. "Nobody gets everything forever -- the
--       audit claim survives at coarser grain."
--
-- PHASE 4 (tree-time.coexistence-plan): this is the gate that flips the gc
-- chapter to implemented -- "the retention worker runs on the jobs runner."
--
-- STRICTLY ADDITIVE + idempotent + image-applied (registered in
-- schema_migrations so the runner will not re-apply).

--
    ------------------------------------------------------------------------
-- Policy as habitat data. path_class: 'code' | 'data'. type_name NULL =
-- whole path class. keep_windows = number of newest 1000-gen windows to
-- retain; windows entirely below the retention floor are rolled up + dropped.
--
    ------------------------------------------------------------------------
CREATE TABLE IF NOT EXISTS tree_retention_policy (
  id            bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  path_class    text        NOT NULL CHECK (path_class IN ('code','data')),
  type_name     text,                   -- NULL = whole path class
  keep_windows  bigint      NOT NULL DEFAULT 8 CHECK (keep_windows >= 1),
  created_at    timestamptz NOT NULL DEFAULT now(),
  UNIQUE (path_class, type_name)
);

--
    ------------------------------------------------------------------------
-- Rollup helper: collapse surviving state for a path class to CHECKPOINT
-- rows at the retention floor, then DROP the gen-window partitions entirely
-- below that floor. Idempotent + conservative:
--   * floor = head_gen - keep_windows*1000, clamped to >= 1000 so the base
--     gen_0 window (gens < 1000, the womb of the tree) is never dropped.
--   * only paths whose ENTIRE write history sits below floor are checkpointed
--     (paths with any row at gen >= floor keep their live/kept head -- their
--     past stays readable via rows still in the kept region, no checkpoint
--     needed; a delete-then-never-recreate path collapses to its last state).
--   * checkpoint rows are written at gen = floor (the boundary INTO the kept
--     region), so they survive every dropped window below and as-of reads of
--     an ancient dropped gen resolve to the checkpointed state.
--   * only lazy windows (wn >= 1) are dropped -- never gen_0.
--
-- Returns (dropped_windows, checkpointed_paths).
--
    ------------------------------------------------------------------------
CREATE OR REPLACE FUNCTION tree_versions_retain(p_class text, keep_windows bigint)
```

```
RETURNS TABLE(dropped_windows bigint, checkpointed_paths bigint)
LANGUAGE plpgsql
AS $function$
DECLARE
  head_gen bigint;
  floor    bigint;
  wn       bigint;
  rolled   bigint := 0;
  dropped  bigint := 0;
  rec      record;
  wins     bigint[] := '{}';
BEGIN
  -- Head gen per path class.
  IF p_class = 'code' THEN
    SELECT COALESCE(max(generation), 0) INTO head_gen FROM
     branch_generations;
  ELSE
    SELECT COALESCE(generation, 0) INTO head_gen FROM tree_generations;
  END IF;

  -- Retention floor. Never below the base-window boundary (1000).
  floor := GREATEST(head_gen - keep_windows * 1000, 1000);
  IF floor <= 1000 OR head_gen < keep_windows * 1000 THEN
    -- Nothing beyond the base window is retentive yet.
    RETURN QUERY SELECT 0::bigint, 0::bigint;
    RETURN;
  END IF;

  -- Make sure the boundary window (floor/1000) exists to receive
     checkpoints.
  PERFORM tree_versions_prepare(floor);

  -- Checkpoint: newest-in-retained-past per path whose whole history is <
     floor.
  FOR rec IN
    SELECT DISTINCT ON (v.path)
           v.path, v.branch_id, v.kind, v.type_name, v.body, v.hash,
     v.author
    FROM tree_versions v
    WHERE v.gen < floor
      AND NOT EXISTS (SELECT 1 FROM tree_versions v2
                      WHERE v2.path = v.path AND v2.gen >= floor)
    ORDER BY v.path, v.gen DESC
  LOOP
    INSERT INTO tree_versions
      (branch_id, path, gen, kind, type_name, body, hash, author,
     created_at)
    VALUES
      (rec.branch_id, rec.path, floor, rec.kind, rec.type_name,
       rec.body, rec.hash, rec.author, now())
    ON CONFLICT (path, gen) DO UPDATE
      SET body = EXCLUDED.body, hash = EXCLUDED.hash, author =
     EXCLUDED.author;
    rolled := rolled + 1;
  END LOOP;

  -- Drop lazy windows entirely below the floor: (wn+1)*1000 <= floor.
  -- Materialize the window list into an ARRAY first (a FOR-loop over a live
  -- SELECT on tree_versions holds a lock on its partitions and DROP TABLE
  -- would fail with 'used by active queries'), then DROP each partition.
  SELECT array_agg(DISTINCT gen / 1000)
    INTO wins
    FROM tree_versions
   WHERE gen >= 1000 AND gen < floor;

  FOREACH wn IN ARRAY wins LOOP
    IF (wn + 1) * 1000 <= floor
       AND to_regclass(format('tree_versions_p%s', wn)) IS NOT NULL
    THEN
      EXECUTE format('DROP TABLE %I', format('tree_versions_p%s', wn));
      dropped := dropped + 1;
    END IF;
  END LOOP;
```

```
124
125   RETURN QUERY SELECT dropped, rolled;
126 END $function$;
127
128 INSERT INTO schema_migrations (version, description)
129 SELECT 22, 'tree-time gc: tree_retention_policy table +
        tree_versions_retain() retention worker (rollup + DROP PARTITION)'
130 WHERE NOT EXISTS (SELECT 1 FROM schema_migrations WHERE version = 22);
```

277 migrations/023_data_lens.sql

```
-- Pricklypear migration 023: versioned user-keyed data -- data lens namespace.
--
-- Spec: one-tree.borg branching-interaction (2026-08-23 plan-phase proposal,
--       now make). Adds an OPT-IN data.l.<lens>.<user>.<type>.<id> path for
--       deliberate data forks that diverge and merge back. The default
--       data.u.<user>.* stays single-head and branch-free; data.l.* is the
--       explicit version axis.
--
-- What this migration does:
--   1. tree_datum_path gains a ~lens_name parameter; when set, the path is
--      data.l.<lens>.<user>.<type>.<id> instead of data.u.<user>.<type>.<id>.
--   2. tree_data_rows view + RLS policy allow data.l.<lens>.<user>.* reads/writes
--      for the owning user (same path-prefix confinement as data.u.*).
--
-- What this migration does NOT do:
--   - No data migration of existing rows (new path shape is opt-in only).
--   - No notes path migration (that is a separate make task; see notes.borg).
--   - No tree-time generation tracking for data.l.* (follows in tree-time make).

--
    ------------------------------------------------------------------------
-- tree_datum_path: add optional lens_name parameter.
-- When lens_name is non-empty, path is data.l.<lens>.<user>.<type>.<id>.
-- The DEFAULT '' preserves the existing data.u.<user>.<type>.<id> shape for
-- all existing callers that don't pass lens_name.
-- DROP the old 3-arg signature first to avoid a non-unique overload
-- (CREATE OR REPLACE with a different arg count creates a new function,
-- not a replacement). DO block for PG portability (no IF EXISTS on DROP).
DO $$ BEGIN
  DROP FUNCTION tree_datum_path(text, uuid, text);
EXCEPTION WHEN undefined_function THEN NULL;
END $$;
CREATE OR REPLACE FUNCTION tree_datum_path(type_name text, id uuid, author text,
                                           lens_name text)
RETURNS ltree
LANGUAGE plpgsql IMMUTABLE AS $$
DECLARE
  a text;
BEGIN
  IF tree_is_seed_type(type_name) THEN
    RETURN ('data.shared.' || tree_enc_label(type_name) || '.' ||
            tree_uuid_label(id))::ltree;
  END IF;
  a := COALESCE(NULLIF(author, ''), 'unknown');
  IF lens_name IS NOT NULL AND lens_name <> '' THEN
    RETURN ('data.l.' || tree_enc_label(lens_name) || '.' ||
            tree_enc_label(a) || '.' ||
            tree_enc_label(type_name) || '.' ||
            tree_uuid_label(id))::ltree;
  END IF;
  RETURN ('data.u.' || tree_enc_label(a) || '.' ||
          tree_enc_label(type_name) || '.' ||
          tree_uuid_label(id))::ltree;
END $$;

--
    ------------------------------------------------------------------------
-- tree_data_rows view: LIVE datum projection (data.u + data.shared only).
-- Default CRUD reads (get/rows/list/find-row) project through this view, so
-- it surfaces LIVE data only. Data lens copies (data.l.<lens>.<user>.*) are
-- excluded: a lens shadows the same (type,id) as its live source, and showing
-- both would duplicate ids and make (get 'type id) ambiguous. Lens copies are
```

```
-- accessed by the lens prims (fork-data/merge-data/abandon-data/list-data-lenses)
-- which read tree_nodes directly, and by the RLS floor (datum_user_isolation
-- still permits data.l.<lens>.<user>.* for the owner). A datum is visible when:
--   - pp.user is unset (admin/scripts), OR
--   - path is under data.u.<user> (own live data), OR
--   - path is under data.shared (seed).
--
-- ---------------------------------------------------------------------------
CREATE OR REPLACE VIEW tree_data_rows AS
SELECT tree_datum_id(path) AS id,
       type_name,
       payload->'body' AS fields,
       payload->>'author' AS author,
       created_at, updated_at
FROM tree_nodes
WHERE kind = 'datum'
  AND NOT (path <@ 'data.l'::ltree)
  AND (
    COALESCE(current_setting('pp.user', true), '') = ''
    OR path <@ ('data.u.' || tree_enc_label(current_setting('pp.user', true)))::ltree
    OR path <@ 'data.shared'::ltree
  );

--
-- ---------------------------------------------------------------------------
-- RLS policy: extend datum_user_isolation to allow data.l.* for the owner.
-- Same path-prefix confinement: data.l.<lens>.<user>.* is visible to <user>.
--
-- ---------------------------------------------------------------------------
-- Drop the old policy (PG < 9.6 doesn't support DROP POLICY IF EXISTS;
-- wrap in a DO block for portability).
DO $$ BEGIN
  DROP POLICY datum_user_isolation ON tree_nodes;
EXCEPTION WHEN undefined_object THEN NULL;
END $$;
CREATE POLICY datum_user_isolation ON tree_nodes
  USING (
    kind <> 'datum'
    OR COALESCE(current_setting('pp.user', true), '') = ''
    OR path <@ ('data.u.' || tree_enc_label(current_setting('pp.user', true)))::ltree
    OR path <@ 'data.shared'::ltree
    OR (
      path <@ 'data.l'::ltree
      AND subltree(path, 3, 4) = tree_enc_label(current_setting('pp.user', true))::ltree
    )
  )
  WITH CHECK (
    kind <> 'datum'
    OR COALESCE(current_setting('pp.user', true), '') = ''
    OR path <@ ('data.u.' || tree_enc_label(current_setting('pp.user', true)))::ltree
    OR path <@ 'data.shared'::ltree
    OR (
      path <@ 'data.l'::ltree
        AND subltree(path, 3, 4) =
      tree_enc_label(current_setting('pp.user', true))::ltree
    )
  );


--
-- ---------------------------------------------------------------------------
-- Update tree_mirror_data_rows to use the 4-arg tree_datum_path explicitly.
-- The original (migration 013) calls tree_datum_path(type, id, author) with 3
```

```
-- args. After this migration drops the 3-arg overload, that call must resolve
-- to the 4-arg function with DEFAULT ''. Updating the function body here
-- ensures PL/pgSQL recompiles against the new signature so the 3-arg overload
-- is not silently recreated by the planner caching the old resolution.
--
    ------------------------------------------------------------------------
CREATE OR REPLACE FUNCTION tree_mirror_data_rows() RETURNS trigger
LANGUAGE plpgsql AS $$
DECLARE p ltree; body jsonb; h text; env jsonb;
BEGIN
  PERFORM set_config('pp.tree_mirror','1', true);
  IF (TG_OP = 'DELETE') THEN
    DELETE FROM tree_nodes
     WHERE kind = 'datum'
       AND tree_datum_id(path) = OLD.id;
    RETURN OLD;
  END IF;
  p := tree_datum_path(NEW.type_name, NEW.id, NEW.author, '');
  body := NEW.fields;
  h := encode(digest(body::text, 'sha256'), 'hex');
  env := jsonb_build_object(
    'v', 1, 'kind', 'datum', 'path', p::text, 'type', NEW.type_name,
    'body', body, 'author', NEW.author, 'doc', NULL, 'hash', h);
    -- Purge stale same-id datum nodes diverged from the live path p, but NEVER
    -- data.l.* lens copies: a live write must not delete the user's forked
    -- snapshot (merge/abandon own the lens lifecycle).
    DELETE FROM tree_nodes
     WHERE kind = 'datum'
       AND tree_datum_id(path) = NEW.id
       AND path <> p
       AND NOT (path <@ 'data.l'::ltree);
  INSERT INTO tree_nodes (path, kind, type_name, payload, hash, author, updated_at)
  VALUES (p, 'datum', NEW.type_name, env, h, NEW.author, now())
  ON CONFLICT (path) DO UPDATE
    SET payload = EXCLUDED.payload, hash = EXCLUDED.hash,
        author = EXCLUDED.author, type_name = EXCLUDED.type_name,
        updated_at = now();
  RETURN NEW;
END
$$;
--
    ------------------------------------------------------------------------
-- Guard the reverse-mirror trigger (tree_from_nodes) against data.l.* paths.
-- The reverse mirror syncs tree_nodes -> scaffolding tables (function_defs,
-- type_defs, view_bindings, data_rows). Data lens nodes (data.l.*) are
-- version-only copies that have no scaffolding-table counterpart; mirroring them
-- would corrupt the live data_rows (e.g. DELETE FROM data_rows WHERE id = did
-- would delete the live row). The trigger must SKIP data.l.* datum nodes.
--
    ------------------------------------------------------------------------
CREATE OR REPLACE FUNCTION tree_from_nodes() RETURNS trigger
LANGUAGE plpgsql AS $$
DECLARE bid uuid; lbl text; docv text; did uuid;
BEGIN
  IF tree_in_mirror() THEN
    IF TG_OP = 'DELETE' THEN RETURN OLD; ELSE RETURN NEW; END IF;
  END IF;
  -- Skip data.l.* nodes entirely: they are lens-only copies with no
  -- scaffolding-table counterpart. Mirroring them would corrupt live rows.
  IF (TG_OP = 'DELETE' AND OLD.path <@ 'data.l'::ltree)
     OR (TG_OP <> 'DELETE' AND NEW.path <@ 'data.l'::ltree) THEN
    IF TG_OP = 'DELETE' THEN RETURN OLD; ELSE RETURN NEW; END IF;
  END IF;
  IF (TG_OP = 'DELETE') THEN
    bid := tree_path_branch(OLD.path);
    IF OLD.kind = 'fn' THEN
```

```
      lbl := (string_to_array(OLD.path::text,'.'))[4];
      DELETE FROM function_defs WHERE branch_id=bid AND name=tree_dec_label(lbl);
    ELSIF OLD.kind = 'type' THEN
      lbl := (string_to_array(OLD.path::text,'.'))[4];
      DELETE FROM type_defs WHERE branch_id=bid AND name=tree_dec_label(lbl);
    ELSIF OLD.kind = 'route' THEN
      DELETE FROM view_bindings WHERE branch_id=bid AND route=OLD.payload->'body'->>'route';
    ELSIF OLD.kind = 'datum' THEN
      did := tree_datum_id(OLD.path);
      IF did IS NOT NULL THEN
        DELETE FROM data_rows WHERE id = did;
      END IF;
    END IF;
    RETURN OLD;
  END IF;
  docv := NEW.payload->>'doc';
  IF NEW.kind = 'fn' THEN
    bid := tree_path_branch(NEW.path);
    lbl := (string_to_array(NEW.path::text,'.'))[4];
    INSERT INTO function_defs (branch_id,name,body,author,doc)
    VALUES (bid, tree_dec_label(lbl), NEW.payload->'body',
            NEW.payload->>'author', docv)
    ON CONFLICT (branch_id,name) DO UPDATE
      SET body=EXCLUDED.body, author=EXCLUDED.author, doc=EXCLUDED.doc,
          updated_at=now();
  ELSIF NEW.kind = 'type' THEN
    bid := tree_path_branch(NEW.path);
    lbl := (string_to_array(NEW.path::text,'.'))[4];
    INSERT INTO type_defs (branch_id,name,fields,author)
    VALUES (bid, tree_dec_label(lbl), NEW.payload->'body',
     NEW.payload->>'author')
    ON CONFLICT (branch_id,name) DO UPDATE
      SET fields=EXCLUDED.fields, author=EXCLUDED.author, updated_at=now();
  ELSIF NEW.kind = 'route' THEN
    bid := tree_path_branch(NEW.path);
    INSERT INTO view_bindings (branch_id,route,handler,visibility)
    VALUES (bid, NEW.payload->'body'->>'route',
     NEW.payload->'body'->>'handler',
            COALESCE(NEW.payload->'body'->>'visibility','private'))
    ON CONFLICT (branch_id,route) DO UPDATE
      SET handler=EXCLUDED.handler, visibility=EXCLUDED.visibility;
  ELSIF NEW.kind = 'datum' THEN
    did := tree_datum_id(NEW.path);
    IF did IS NOT NULL THEN
      INSERT INTO data_rows (id,type_name,fields,author)
      VALUES (did, NEW.type_name, NEW.payload->'body',
     NEW.payload->>'author')
      ON CONFLICT (id) DO UPDATE
        SET fields=EXCLUDED.fields, author=EXCLUDED.author,
            type_name=EXCLUDED.type_name, updated_at=now();
    END IF;
  END IF;
  RETURN NEW;
END
$$;

INSERT INTO schema_migrations (version, description)
SELECT 23, 'data lens namespace data.l.<lens>.<user>.* + RLS visibility + trigger guard'
WHERE NOT EXISTS (SELECT 1 FROM schema_migrations WHERE version = 23);
```

278 migrations/024_branch_base_gen.sql

```
-- Pricklypear migration 024: branch base_gen -- the merge-base coordinate.
--
-- Per spec [branching.branch-status] (plan pass pinned; tree-time consumer).
-- A branch records the parent's head generation (branch_generations,
-- migration 017/019) at fork time. Combined with the append-only
-- tree_versions history (018-021), "value of path P at base_gen" is a
-- READ, so git-style clean / ahead / behind / diverged / synced status is
-- a derived hash comparison, not a name-presence join.
--
-- base_gen ADVANCES when a merge lands the branch into its parent (git's
-- moving merge-base). promote deliberately does NOT advance it: selective
-- pins stay visible as `synced` (the unbind-candidate class) until
-- unbound or merged.
--
-- Backfill: existing branches predate the column, so the true fork point
-- is unrecoverable. Approximation (documented in branching.branch-status):
-- treat the fork as having happened at the parent's CURRENT generation --
-- inherited paths then read clean/behind correctly, and existing own pins
-- classify against today's parent content. Empty children (the dominant
-- shape) are exact under this backfill; full-fork copies whose parent
-- moved since the real fork read `synced` (they are content-identical
-- pins -- which is the actionable truth for the unbind flow anyway).
--
-- Strictly ADDITIVE + idempotent + image-applied (registered in
-- schema_migrations so the runner will not re-apply).

ALTER TABLE branches ADD COLUMN IF NOT EXISTS base_gen bigint NOT NULL DEFAULT 0;

UPDATE branches b
SET base_gen = COALESCE(
  (SELECT bg.generation FROM branch_generations bg WHERE bg.branch_id = b.parent_id),
  0)
WHERE b.parent_id IS NOT NULL;

-- Root branch(es) (system/main): no parent, base_gen stays 0 (unused).

INSERT INTO schema_migrations (version, description)
SELECT 24, 'branch base_gen merge-base coordinate (branching.branch-status / tree-time consumer)'
WHERE NOT EXISTS (SELECT 1 FROM schema_migrations WHERE version = 24);
```

279 migrations/025_sibling_order.sql

```
-- Pricklypear migration 025: sibling-order (one-tree.sibling-order).
--
-- Children get a USER-CONTROLLED ORDER: a nullable rank TEXT column on
-- tree_nodes (fractional-indexing string, Figma/Linear style), carried
-- through the tree-time version shadow so ordering is versioned,
-- diffable, undoable like every other node property.
--
--   - NOT fractional path labels: the path is WHERE a node is; the rank
--     is WHAT SLOT it is in. Rank is mutable without touching identity.
--   - NULL rank means unordered: reads order by
--       rank COLLATE "C" NULLS LAST, path::text ASC
--     so unranked siblings sort after ranked ones, path-ordered among
--     themselves. Existing rows need no backfill; ordering is opt-in
--     per sibling set.
--   - Rank strings are base-36 fractional strings over [0-9a-z] compared
--     as raw bytes (COLLATE "C", with implicit right-padding '0') -- the
--     same alphabet byte order everywhere, locale-independent. Midpoint
--     allocation + rebalance-on-exhaustion live in image/lib/tree_store.ml
--     (write-path detail, not visible in the prim ABI).
--   - tree_versions gains the same rank column and tree_version_append
--     echoes it, so gen-read/undo carry order (reorder is an ordinary
--     versioned write; a rank-only UPDATE is NOT a mirror echo).
--
-- This file is strictly ADDITIVE + idempotent + image-applied (registered
-- in schema_migrations so the runner will not re-apply).

ALTER TABLE tree_nodes ADD COLUMN IF NOT EXISTS rank text;
ALTER TABLE tree_versions ADD COLUMN IF NOT EXISTS rank text;

-- Read-shape index for ranked sibling sets. The children read is a LEVEL
-- predicate over the ltree GiST path index (CHILDREN-BY-LEVEL invariant --
-- never an anti-join); this partial btree only covers the ranked subset so
-- rank-presence checks and rank-ordered scans of a small set stay cheap.
CREATE INDEX IF NOT EXISTS tree_nodes_rank_idx
  ON tree_nodes (rank COLLATE "C")
  WHERE rank IS NOT NULL;

-- The node-level AFTER trigger body (migration 019, refined by 020's
-- mirror-echo guard), now carrying RANK:
--   - the echo guard must treat a RANK-ONLY change as a REAL write
--     (a reorder is content), so `rank IS NOT DISTINCT FROM` joins the
--     no-op condition;
--   - both version INSERTs gain the rank column (NEW.rank / OLD.rank),
--     so gen-read / gen-diff / undo see the order that was in force at
--     that generation.
CREATE OR REPLACE FUNCTION tree_version_append()
RETURNS trigger
LANGUAGE plpgsql
AS $function$
DECLARE
  bid  uuid;
  ngen bigint;
BEGIN
  -- MIRROR NO-OP WRITE-BACK guard (020), now rank-aware: identical
  -- kind/type/author/body AND identical rank is an echo; a rank delta is
  -- an authoritative reorder and MUST be versioned.
  IF TG_OP = 'UPDATE' AND NEW.kind = OLD.kind
     AND NEW.type_name IS NOT DISTINCT FROM OLD.type_name
     AND NEW.author = OLD.author
     AND NEW.rank IS NOT DISTINCT FROM OLD.rank
     AND NEW.payload->'body' = OLD.payload->'body' THEN
    RETURN NEW;
  END IF;

  -- Owning branch for code paths; NULL for branch-free data.*.
  bid := tree_path_branch(COALESCE(NEW.path, OLD.path));

  -- Allocate the next gen atomically, per path class.
  IF bid IS NOT NULL THEN
    INSERT INTO branch_generations (branch_id, generation)
    VALUES (bid, 1)
```

```
    ON CONFLICT (branch_id)
      DO UPDATE SET generation = branch_generations.generation + 1
    RETURNING generation INTO ngen;
  ELSE
    UPDATE tree_generations SET generation = generation + 1
    RETURNING generation INTO ngen;
    IF ngen IS NULL THEN
      INSERT INTO tree_generations (generation) VALUES (1)
      RETURNING generation INTO ngen;
    END IF;
  END IF;

  PERFORM tree_versions_prepare(ngen);

  IF TG_OP = 'DELETE' THEN
    -- TOMBSTONE: reserved marker body (same `_tombstone` key
    -- tree_versions_tombstone() returns), never a DELETE.
    INSERT INTO tree_versions
      (branch_id, path, gen, kind, type_name, rank, body, hash, author,
       created_at)
    VALUES
      ( bid, OLD.path, ngen, OLD.kind, OLD.type_name, OLD.rank,
        jsonb_build_object(
          'v', 1, 'kind', OLD.kind, 'path', OLD.path::text,
          'type', OLD.type_name, 'body', JSON 'null',
          'author', OLD.author, 'doc', JSON 'null',
          'hash', encode(digest('__pp_tombstone__', 'sha256'), 'hex'),
          '_tombstone', true),
        encode(digest('__pp_tombstone__', 'sha256'), 'hex'),
        OLD.author, now() );
    RETURN OLD;
  END IF;

  -- INSERT / UPDATE: record the new head payload at gen (with its rank).
  INSERT INTO tree_versions
    (branch_id, path, gen, kind, type_name, rank, body, hash, author,
     created_at)
  VALUES
    ( bid, NEW.path, ngen, NEW.kind, NEW.type_name, NEW.rank,
      NEW.payload, NEW.hash, NEW.author, now() );
  RETURN NEW;
END $function$;

INSERT INTO schema_migrations (version, description)
SELECT 25, 'sibling-order: tree_nodes/tree_versions rank column +
    rank-aware version trigger'
WHERE NOT EXISTS (SELECT 1 FROM schema_migrations WHERE version = 25);
```

280 migrations/026_tree_edges.sql

```
-- Pricklypear migration 026: tree-edges index (one-tree.tree-edges).
--
-- An edge IS a node: kind=link at <source-path>.lnk.<uuid>, body
-- {to, kind, label?}. "from" is positional (the parent path), never in
-- the body. This migration adds the index that makes edge queries
-- index-driven instead of scans:
--
--   - out-links(path): a children-by-level read (path <@ source AND
--     nlevel = nlevel(source)+1 AND kind='link') -- already covered by
--     the ltree GiST path index.
--   - backlinks/edges-to(needle): payload->'body'->>'to' = needle over
--     kind='link' -- covered by the partial btree below.
--
-- Edges dangle BY DESIGN (branching-interaction): to is stored verbatim
-- and resolved at read time against the reader's lens; an unresolved
-- edge is dangling data, never an error state. No foreign keys.

CREATE INDEX IF NOT EXISTS tree_nodes_link_to_idx
  ON tree_nodes ((payload->'body'->>'to'))
  WHERE kind = 'link';

INSERT INTO schema_migrations (version, description)
SELECT 26, 'tree-edges: partial index on link edge targets (payload body
    to, kind=link)'
WHERE NOT EXISTS (SELECT 1 FROM schema_migrations WHERE version = 26);
```

281 pure/prims_pure.ml

```
(** Pure Lisp primitives (no DB, no Session.t).
    Registered by [Primitives.make_with_defaults]. *)
open Pricklypear_lisp.Ast

type num = NInt of int64 | NFloat of float

let num_of_value = function
  | VInt n -> NInt n
  | VFloat f -> NFloat f
  | v ->
      raise
        (Interp_error
           (User_error ("numeric op needs int or float, got " ^
                        (match v with
                         | VStr _ -> "string"
                         | VBool _ -> "bool"
                         | VNil -> "nil"
                         | _ -> "other"), "bad-args")))

let float_of_num = function
  | NInt n -> Int64.to_float n
  | NFloat f -> f

let value_of_num = function
  | NInt n -> VInt n
  | NFloat f -> VFloat f

let num_add a b =
  match a, b with
  | NInt x, NInt y -> NInt (Int64.add x y)
  | _ -> NFloat (float_of_num a +. float_of_num b)

let num_sub a b =
  match a, b with
  | NInt x, NInt y -> NInt (Int64.sub x y)
  | _ -> NFloat (float_of_num a -. float_of_num b)

let num_mul a b =
  match a, b with
  | NInt x, NInt y -> NInt (Int64.mul x y)
  | _ -> NFloat (float_of_num a *. float_of_num b)

let num_div a b =
  let bf = float_of_num b in
  if bf = 0. then
    raise (Interp_error (User_error ("div by zero", "bad-args")))
  else
    match a, b with
    | NInt x, NInt y -> NInt (Int64.div x y)
    | _ -> NFloat (float_of_num a /. bf)

let num_neg = function
  | NInt n -> NInt (Int64.neg n)
  | NFloat f -> NFloat (~-. f)

let num_compare a b =
  match a, b with
  | NInt x, NInt y -> Int64.compare x y
  | _ -> compare (float_of_num a) (float_of_num b)

(* "+" sums zero-or-more numeric args. Variadic; (+) = 0 (additive
    identity). *)
let prim_add (args : value list) : value =
  value_of_num
    (List.fold_left
       (fun acc v -> num_add acc (num_of_value v))
       (NInt 0L)
       args)

(* "-" binary: (- a b) = a - b; unary: (- a) = -a. *)
let prim_sub args =
```

```
  match args with
  | [ a ] -> value_of_num (num_neg (num_of_value a))
  | [ a; b ] -> value_of_num (num_sub (num_of_value a) (num_of_value b))
  | _ -> raise (Interp_error (User_error ("- needs one or two numeric
     args", "bad-args")))

(* "*". Zero-or-more numeric args. Variadic; empty product is 1
     (multiplicative identity). *)
let prim_mul (args : value list) : value =
  value_of_num
    (List.fold_left
       (fun acc v -> num_mul acc (num_of_value v))
       (NInt 1L)
       args)

(* "=" numeric equality (int/float). *)
let prim_num_eq args =
  match args with
  | [ a; b ] -> VBool (num_compare (num_of_value a) (num_of_value b) = 0)
  | _ -> raise (Interp_error (User_error ("= needs two numeric args",
     "bad-args")))

let prim_lt args =
  match args with
  | [ a; b ] -> VBool (num_compare (num_of_value a) (num_of_value b) < 0)
  | _ -> raise (Interp_error (User_error ("< needs two numeric args",
     "bad-args")))

let prim_gt args =
  match args with
  | [ a; b ] -> VBool (num_compare (num_of_value a) (num_of_value b) > 0)
  | _ -> raise (Interp_error (User_error ("> needs two numeric args",
     "bad-args")))

let prim_le args =
  match args with
  | [ a; b ] -> VBool (num_compare (num_of_value a) (num_of_value b) <= 0)
  | _ -> raise (Interp_error (User_error ("<= needs two numeric args",
     "bad-args")))

let prim_ge args =
  match args with
  | [ a; b ] -> VBool (num_compare (num_of_value a) (num_of_value b) >= 0)
  | _ -> raise (Interp_error (User_error (">= needs two numeric args",
     "bad-args")))

(* "not" boolean negation *)
let prim_not args =
  match args with
  | [ VBool b ] -> VBool (not b)
  | [ VNil ] -> VBool true
  | [ _ ] -> VBool false   (* any non-nil non-false is truthy *)
  | _ -> raise (Interp_error (User_error ("not needs one arg", "bad-args")))

(* (list a b c ...) -> VList. Variadic. *)
let prim_list args = VList args

(* (car xs): head of a VList or VPair. Errors on empty / atoms. *)
let prim_car args =
  match args with
  | [ VList (h :: _) ] -> h
  | [ VList [] ] -> VNil
  | [ VPair (a, _) ] -> a
  | _ -> raise (Interp_error (User_error ("car needs a list or pair",
     "bad-args")))

(* (cdr xs): tail of a VList or VPair. *)
let prim_cdr args =
  match args with
  | [ VList (_ :: rest) ] -> VList rest
  | [ VList [] ] -> VNil
  | [ VPair (_, b) ] -> b
```

```
  | _ -> raise (Interp_error (User_error ("cdr needs a list or pair",
     "bad-args")))

(* (cons a b) -> VPair. Classic Lisp cons cell. *)
let prim_cons args =
  match args with
  | [ a; b ] -> VPair (a, b)
  | _ -> raise (Interp_error (User_error ("cons needs two args",
     "bad-args")))

(* (null? x) -> VBool. True if VNil. *)
let prim_null_p args =
  match args with
  | [ VNil ] -> VBool true
  | [ VList [] ] -> VBool true
  | [ _ ] -> VBool false
  | _ -> raise (Interp_error (User_error ("null? needs one arg",
     "bad-args")))

(* (pair? x) -> VBool. True if VPair or non-empty VList. *)
let prim_pair_p args =
  match args with
  | [ VPair _ ] -> VBool true
  | [ VList (_ :: _) ] -> VBool true
  | _ -> VBool false

(** Unwrap macro string tags produced by expand_and_eval_macro:
    (__str__ "foo") is a VList that must behave like VStr for string prims.
     *)
let rec coerce_str ?(ctx = "string") (v : value) : string =
  match v with
  | VStr s -> s
  | VSymbol s -> s (* product HTML/paths; symbols print as names *)
  | VList [ VStr "__str__"; VStr s ] -> s
  | VList [ VSymbol "__str__"; VStr s ] -> s
  | VInt n -> Int64.to_string n
  | VFloat f -> Printf.sprintf "%.12g" f
  | VBool true -> "true"
  | VBool false -> "false"
  | VNil -> ""
  (* Empty list is null? in Lisp and used as empty html attrs: (html/p*
     (list) body).
     Treat like nil so as-str/html/tag do not blow up product home widgets.
     *)
  | VList [] -> ""
  | _ ->
      raise
        (Interp_error
           (User_error (ctx ^ " needs a string (got " ^
                        Pricklypear_lisp.Reader.pp_value v ^ ")",
     "bad-args")))

(* (string-append s1 s2 ...) -> VStr. Variadic over VStr / __str__ tags. *)
let prim_string_append args =
  let buf = Buffer.create 32 in
  List.iter
    (fun v -> Buffer.add_string buf (coerce_str ~ctx:"string-append" v))
    args;
  VStr (Buffer.contents buf)

(* (string-join strs [sep]) -> VStr. Concatenate a list of strings in a
     single
   O(n) pass (Buffer add), optionally separated by sep (default ""). Unlike
     a
   fold over string-append (O(n?) accumulator-copy), this is linear. List
     items
   are coerced like string-append
     (string/__str__/symbol/int/float/bool/nil). *)
let prim_string_join args =
  let unwrap = function
    | VList xs -> xs
    | VNil -> []
    | VPair (a, b) ->
```

```
197         let rec unwind p acc =
198           match p with
199           | VPair (x, y) -> unwind y (x :: acc)
200           | VNil | VList [] -> List.rev acc
201           | VList xs -> List.rev_append acc xs
202           | other -> List.rev (other :: acc)
203         in
204         unwind (VPair (a, b)) []
205     | other ->
206         raise (Interp_error (User_error ("string-join needs a list of
      strings (got " ^
207                                          Pricklypear_lisp.Reader.pp_value
      other ^ ")", "bad-args")))
208   in
209   let join items sep =
210     let buf = Buffer.create 32 in
211     (match items with
212      | [] -> ()
213      | x :: rest ->
214          Buffer.add_string buf (coerce_str ~ctx:"string-join" x);
215          List.iter
216            (fun v -> Buffer.add_string buf sep; Buffer.add_string buf
      (coerce_str ~ctx:"string-join" v))
217            rest);
218     VStr (Buffer.contents buf)
219   in
220   match args with
221   | [ lst ] -> join (unwrap lst) ""
222   | [ lst; sep_v ] -> join (unwrap lst) (coerce_str ~ctx:"string-join"
      sep_v)
223   | _ -> raise (Interp_error (User_error ("string-join needs a list [sep]",
      "bad-args")))
224
225 (* prim_string_split is DEFINED LATER in this file (agent section, ~line
226    1830) -- it is now ALSO a kernel image prim (habitat-book web-renderer:
227    the interpreted libs/std char-walk cost ~interpreter steps per input
228    byte, so one 3KB prose paragraph blew the 100k-step page budget).
229    libs/std no longer defines it; keep agent semantics in one place. *)
230
231 (* (type-of x) -> VStr. Returns a type tag for runtime reflection. *)
232 let prim_type_of args =
233   match args with
234   | [ VInt _ ] -> VStr "int"
235   | [ VFloat _ ] -> VStr "float"
236   | [ VStr _ ] -> VStr "str"
237   | [ VSymbol _ ] -> VStr "symbol"
238   | [ VList [ VStr "__str__"; VStr _ ] ] -> VStr "str"
239   | [ VList [ VSymbol "__str__"; VStr _ ] ] -> VStr "str"
240   | [ VBool _ ] -> VStr "bool"
241   | [ VNil ] -> VStr "nil"
242   | [ VPair _ ] -> VStr "pair"
243   | [ VList _ ] -> VStr "list"
244   | [ VLambda _ ] -> VStr "lambda"
245   | [ VMacro _ ] -> VStr "macro"
246   | [ VPrim name ] -> VStr ("prim:" ^ name)
247   | [ VRef (t, _) ] -> VStr ("ref:" ^ t)
248   | _ -> raise (Interp_error (User_error ("type-of needs one arg",
      "bad-args")))
249
250 let prim_symbol_p = function
251   | [ VSymbol _ ] -> VBool true
252   | [ _ ] -> VBool false
253   | _ -> raise (Interp_error (User_error ("symbol? needs one arg",
      "bad-args")))
254
255 let prim_string_p = function
256   | [ VStr _ ] -> VBool true
257   | [ VList [ VStr "__str__"; VStr _ ] ]
258   | [ VList [ VSymbol "__str__"; VStr _ ] ] -> VBool true
259   | [ _ ] -> VBool false
260   | _ -> raise (Interp_error (User_error ("string? needs one arg",
      "bad-args")))
261
```

```
let prim_symbol_eq = function
  | [ VSymbol a; VSymbol b ] -> VBool (String.equal a b)
  | [ _; _ ] -> VBool false
  | _ -> raise (Interp_error (User_error ("symbol=? needs two args",
     "bad-args")))

let prim_symbol_to_string = function
  | [ VSymbol s ] -> VStr s
  | [ VStr s ] -> VStr s (* soft: already a string *)
  | _ -> raise (Interp_error (User_error ("symbol->string needs a symbol",
     "bad-args")))

let prim_string_to_symbol = function
  | [ VStr s ] -> VSymbol s
  | [ VSymbol s ] -> VSymbol s
  | _ -> raise (Interp_error (User_error ("string->symbol needs a string",
     "bad-args")))

(* (str x) -> VStr. Coerces any value to its pretty-printed form. *)
let prim_str args =
  match args with
  | [ v ] -> VStr (Pricklypear_lisp.Reader.pp_value v)
  | _ -> raise (Interp_error (User_error ("str needs one arg", "bad-args")))

(* (as-str x) -> VStr without Lisp quotes (for path/ref building). *)
let prim_as_str args =
  match args with
  | [ v ] -> VStr (coerce_str ~ctx:"as-str" v)
  | _ -> raise (Interp_error (User_error ("as-str needs one arg",
     "bad-args")))

(* (parse-float s|n) -> VFloat *)
let prim_parse_float args =
  match args with
  | [ VFloat f ] -> VFloat f
  | [ VInt n ] -> VFloat (Int64.to_float n)
  | [ VStr s ] ->
      (try VFloat (float_of_string (String.trim s))
       with _ -> VFloat 0.)
  | _ -> raise (Interp_error (User_error ("parse-float needs string or
     number", "bad-args")))

(* (floor x) -> VInt *)
let prim_floor args =
  match args with
  | [ VInt n ] -> VInt n
  | [ VFloat f ] -> VInt (Int64.of_float (floor f))
  | _ -> raise (Interp_error (User_error ("floor needs one numeric arg",
     "bad-args")))

(* (round x) -> VInt *)
let prim_round args =
  match args with
  | [ VInt n ] -> VInt n
  | [ VFloat f ] -> VInt (Int64.of_float (Float.round f))
  | _ -> raise (Interp_error (User_error ("round needs one numeric arg",
     "bad-args")))

(* (div a b) -- integer div when both int; else float. *)
let prim_div args =
  match args with
  | [ a; b ] -> value_of_num (num_div (num_of_value a) (num_of_value b))
  | _ -> raise (Interp_error (User_error ("div needs two numeric args",
     "bad-args")))

let prim_mod args =
  match args with
  | [ VInt a; VInt b ] when b <> 0L -> VInt (Int64.rem a b)
  | [ VInt _; VInt 0L ] ->
      raise (Interp_error (User_error ("mod by zero", "bad-args")))
  | [ a; b ] ->
      let bf = float_of_num (num_of_value b) in
```

```
      if bf = 0. then raise (Interp_error (User_error ("mod by zero",
     "bad-args")))
      else VFloat (mod_float (float_of_num (num_of_value a)) bf)
  | _ -> raise (Interp_error (User_error ("mod needs two numeric args",
     "bad-args")))

(* Scheme aliases of the int paths of div/mod -- reject floats. *)
let prim_quotient args =
  match args with
  | [ VInt a; VInt b ] when b <> 0L -> VInt (Int64.div a b)
  | [ VInt _; VInt 0L ] ->
      raise (Interp_error (User_error ("quotient by zero", "bad-args")))
  | [ VInt _; _ ] | [ _; VInt _ ] | [ _; _ ] ->
      raise (Interp_error (User_error ("quotient needs two int args",
     "bad-args")))
  | _ -> raise (Interp_error (User_error ("quotient needs two int args",
     "bad-args")))

let prim_remainder args =
  match args with
  | [ VInt a; VInt b ] when b <> 0L -> VInt (Int64.rem a b)
  | [ VInt _; VInt 0L ] ->
      raise (Interp_error (User_error ("remainder by zero", "bad-args")))
  | [ _; _ ] ->
      raise (Interp_error (User_error ("remainder needs two int args",
     "bad-args")))
  | _ -> raise (Interp_error (User_error ("remainder needs two int args",
     "bad-args")))

(* ---- math tower (lisp-interpreter.math) ---- *)

let int_pow (base : int64) (exp : int64) : int64 =
  (* Non-negative exponent only; Int64 overflow wraps like Int64.mul.
     Exponentiation by squaring -- O(log e), not O(e). *)
  let rec loop acc b e =
    if e = 0L then acc
    else if Int64.logand e 1L = 1L then
      loop (Int64.mul acc b) (Int64.mul b b) (Int64.shift_right_logical e 1)
    else
      loop acc (Int64.mul b b) (Int64.shift_right_logical e 1)
  in
  if exp < 0L then
    raise (Interp_error (User_error ("expt int path needs non-negative
     exponent", "bad-args")))
  else loop 1L base exp

let prim_expt args =
  match args with
  | [ a; b ] ->
      (match num_of_value a, num_of_value b with
       | NInt base, NInt e when e >= 0L -> VInt (int_pow base e)
       | ba, bb ->
           VFloat (Float.pow (float_of_num ba) (float_of_num bb)))
  | _ -> raise (Interp_error (User_error ("expt needs two numeric args",
     "bad-args")))

let prim_sqrt args =
  match args with
  | [ a ] ->
      let x = float_of_num (num_of_value a) in
      if x < 0. then
        raise (Interp_error (User_error ("sqrt of negative", "bad-args")))
      else VFloat (sqrt x)
  | _ -> raise (Interp_error (User_error ("sqrt needs one numeric arg",
     "bad-args")))

let prim_exp args =
  match args with
  | [ a ] -> VFloat (exp (float_of_num (num_of_value a)))
  | _ -> raise (Interp_error (User_error ("exp needs one numeric arg",
     "bad-args")))

let prim_log args =
```

```
  match args with
  | [ a ] ->
      let x = float_of_num (num_of_value a) in
      if x <= 0. then
        raise (Interp_error (User_error ("log of non-positive",
     "bad-args")))
      else VFloat (log x)
  | [ a; b ] ->
      let x = float_of_num (num_of_value a) in
      let base = float_of_num (num_of_value b) in
      if x <= 0. then
        raise (Interp_error (User_error ("log of non-positive",
     "bad-args")))
      else if base <= 0. || base = 1. then
        raise (Interp_error (User_error ("log: invalid base", "bad-args")))
      else VFloat (log x /. log base)
  | _ -> raise (Interp_error (User_error ("log needs one or two numeric
     args", "bad-args")))

let prim_sin args =
  match args with
  | [ a ] -> VFloat (sin (float_of_num (num_of_value a)))
  | _ -> raise (Interp_error (User_error ("sin needs one numeric arg",
     "bad-args")))

let prim_cos args =
  match args with
  | [ a ] -> VFloat (cos (float_of_num (num_of_value a)))
  | _ -> raise (Interp_error (User_error ("cos needs one numeric arg",
     "bad-args")))

let prim_tan args =
  match args with
  | [ a ] -> VFloat (tan (float_of_num (num_of_value a)))
  | _ -> raise (Interp_error (User_error ("tan needs one numeric arg",
     "bad-args")))

let prim_asin args =
  match args with
  | [ a ] ->
      let x = float_of_num (num_of_value a) in
      if x < -1. || x > 1. then
        raise (Interp_error (User_error ("asin domain [-1,1]", "bad-args")))
      else VFloat (asin x)
  | _ -> raise (Interp_error (User_error ("asin needs one numeric arg",
     "bad-args")))

let prim_acos args =
  match args with
  | [ a ] ->
      let x = float_of_num (num_of_value a) in
      if x < -1. || x > 1. then
        raise (Interp_error (User_error ("acos domain [-1,1]", "bad-args")))
      else VFloat (acos x)
  | _ -> raise (Interp_error (User_error ("acos needs one numeric arg",
     "bad-args")))

let prim_atan args =
  match args with
  | [ a ] -> VFloat (atan (float_of_num (num_of_value a)))
  | [ a; b ] ->
      (* Two-arg form ? atan2(y, x) with y first (Scheme order). *)
      let y = float_of_num (num_of_value a) in
      let x = float_of_num (num_of_value b) in
      VFloat (atan2 y x)
  | _ -> raise (Interp_error (User_error ("atan needs one or two numeric
     args", "bad-args")))

let prim_number_p args =
  match args with
  | [ VInt _ ] | [ VFloat _ ] -> VBool true
  | [ _ ] -> VBool false
```

```
  | _ -> raise (Interp_error (User_error ("number? needs one arg",
     "bad-args")))

let prim_integer_p args =
  match args with
  | [ VInt _ ] -> VBool true
  | [ _ ] -> VBool false
  | _ -> raise (Interp_error (User_error ("integer? needs one arg",
     "bad-args")))

let prim_zero_p args =
  match args with
  | [ a ] -> VBool (num_compare (num_of_value a) (NInt 0L) = 0)
  | _ -> raise (Interp_error (User_error ("zero? needs one numeric arg",
     "bad-args")))

let prim_positive_p args =
  match args with
  | [ a ] -> VBool (num_compare (num_of_value a) (NInt 0L) > 0)
  | _ -> raise (Interp_error (User_error ("positive? needs one numeric
     arg", "bad-args")))

let prim_negative_p args =
  match args with
  | [ a ] -> VBool (num_compare (num_of_value a) (NInt 0L) < 0)
  | _ -> raise (Interp_error (User_error ("negative? needs one numeric
     arg", "bad-args")))

let as_integral_int64 = function
  | VInt n -> n
  | VFloat f ->
      if Float.is_integer f then Int64.of_float f
      else raise (Interp_error (User_error ("even?/odd? needs an integer",
     "bad-args")))
  | _ -> raise (Interp_error (User_error ("even?/odd? needs an integer",
     "bad-args")))

let prim_even_p args =
  match args with
  | [ a ] ->
      let n = as_integral_int64 a in
      VBool (Int64.logand n 1L = 0L)
  | _ -> raise (Interp_error (User_error ("even? needs one arg",
     "bad-args")))

let prim_odd_p args =
  match args with
  | [ a ] ->
      let n = as_integral_int64 a in
      VBool (Int64.logand n 1L <> 0L)
  | _ -> raise (Interp_error (User_error ("odd? needs one arg",
     "bad-args")))

(** (random) -> float [0,1); (random n) -> int [0,n) for VInt n > 0. *)
let prim_random args =
  match args with
  | [] -> VFloat (Random.float 1.0)
  | [ VInt n ] when n > 0L ->
      (* Random.int takes int; clamp to positive native int range. *)
      let n_int =
        if n > Int64.of_int max_int then max_int else Int64.to_int n
      in
      VInt (Int64.of_int (Random.int n_int))
  | [ VInt _ ] ->
      raise (Interp_error (User_error ("random: n must be positive int",
     "bad-args")))
  | _ -> raise (Interp_error (User_error ("random needs 0 args or one
     positive int", "bad-args")))

(** (random-seed s) reseeds process PRNG; returns nil. *)
let prim_random_seed args =
  match args with
  | [ VInt s ] ->
```

```
      let seed =
        if s > Int64.of_int max_int then max_int
        else if s < Int64.of_int min_int then min_int
        else Int64.to_int s
      in
      Random.init seed;
      VNil
  | _ -> raise (Interp_error (User_error ("random-seed needs one int",
     "bad-args")))

let prim_abs args =
  match args with
  | [ VInt a ] -> VInt (Int64.abs a)
  | [ VFloat f ] -> VFloat (abs_float f)
  | _ -> raise (Interp_error (User_error ("abs needs one numeric arg",
     "bad-args")))

let prim_min args =
  match args with
  | [] -> raise (Interp_error (User_error ("min needs at least one numeric
     arg", "bad-args")))
  | h :: rest ->
      value_of_num
        (List.fold_left
           (fun acc v ->
             let n = num_of_value v in
             if num_compare n acc < 0 then n else acc)
           (num_of_value h) rest)

let prim_max args =
  match args with
  | [] -> raise (Interp_error (User_error ("max needs at least one numeric
     arg", "bad-args")))
  | h :: rest ->
      value_of_num
        (List.fold_left
           (fun acc v ->
             let n = num_of_value v in
             if num_compare n acc > 0 then n else acc)
           (num_of_value h) rest)

let prim_string_length args =
  match args with
  | [ v ] ->
      let s = coerce_str ~ctx:"string-length" v in
      VInt (Int64.of_int (String.length s))
  | _ -> raise (Interp_error (User_error ("string-length needs one VStr",
     "bad-args")))

let prim_substring args =
  match args with
  | [ sv; VInt start; VInt len ] ->
      let s = coerce_str ~ctx:"substring" sv in
      let st = Int64.to_int start in
      let n = Int64.to_int len in
      let sl = String.length s in
      if st < 0 || n < 0 || st > sl then
        raise (Interp_error (User_error ("substring out of range",
     "bad-args")))
      else
        let n = min n (sl - st) in
        VStr (String.sub s st n)
  | [ sv; VInt start ] ->
      let s = coerce_str ~ctx:"substring" sv in
      let st = Int64.to_int start in
      let sl = String.length s in
      if st < 0 || st > sl then
        raise (Interp_error (User_error ("substring out of range",
     "bad-args")))
      else VStr (String.sub s st (sl - st))
  | _ ->
      raise
        (Interp_error
```

```
580           (User_error ("substring needs string start [len]", "bad-args")))
581
582 let prim_string_contains args =
583   match args with
584   | [ hay_v; needle_v ] ->
585       let hay = coerce_str ~ctx:"string-contains?" hay_v in
586       let needle = coerce_str ~ctx:"string-contains?" needle_v in
587       let rec loop i =
588         if needle = "" then true
589         else if i + String.length needle > String.length hay then false
590         else if String.sub hay i (String.length needle) = needle then true
591         else loop (i + 1)
592       in
593       VBool (loop 0)
594   | _ ->
595       raise
596         (Interp_error (User_error ("string-contains? needs two VStr args",
      "bad-args")))
597
598 let list_of_value = function
599   | VList xs -> xs
600   | VNil -> []
601   | VPair (a, b) ->
602       let rec unwind p acc =
603         match p with
604         | VPair (x, y) -> unwind y (x :: acc)
605         | VNil | VList [] -> List.rev acc
606         | VList xs -> List.rev_append acc xs
607         | other -> List.rev (other :: acc)
608       in
609       unwind (VPair (a, b)) []
610   | _ -> raise (Interp_error (User_error ("expected a list", "bad-args")))
611
612 let prim_append args =
613   match args with
614   | [ a; b ] -> VList (list_of_value a @ list_of_value b)
615   | xs ->
616       VList
617         (List.concat_map
618            (function
619              | VList l -> l
620              | VNil -> []
621              | _ -> raise (Interp_error (User_error ("append needs lists",
      "bad-args"))))
622            xs)
623
624 let prim_reverse args =
625   match args with
626   | [ xs ] -> VList (List.rev (list_of_value xs))
627   | _ -> raise (Interp_error (User_error ("reverse needs one list",
      "bad-args")))
628
629 let prim_nth args =
630   match args with
631   | [ xs; VInt i ] ->
632       let n = Int64.to_int i in
633       let lst = list_of_value xs in
634       if n < 0 || n >= List.length lst then VNil
635       else List.nth lst n
636   | _ -> raise (Interp_error (User_error ("nth needs list and VInt",
      "bad-args")))
637
638 (* ---- dict primitives over VStr JSON -----------------------------
639
640    Iteration-14 decision: don't introduce a new VDict value type yet
641    (that would ripple through ast.ml / reader.ml / eval.ml / value
642    JSON round-trip). Instead, (get 'type id) returns a VStr holding
643    the fields JSON; these prims operate on VStr JSON directly so Lisp
644    code can do:
645        (define u (get 'user id))
646        (dict-get u "name")
647        (dict-set u "active" "false")
648    The primitive returns VStr JSON (or VNil if the input wasn't a valid
```

```
   object, or key wasn't found). The underlying parse-and-emit is
   Yojson; the canonical round-trip preserves well-formed objects. *)

(* (dict-get json-str key) -> VStr value | VNil.
   Looks up [key] in the JSON object at [json-str]. Returns the value
   as a VStr (numbers and booleans stringified via Yojson); VNil if
   the key is missing or the input isn't a JSON object. *)
let prim_dict_get args =
  match args with
  | [ VStr json; VStr key ] ->
      (try
        let j = Yojson.Safe.from_string json in
        (match j with
         | `Assoc kvs ->
             (match List.assoc_opt key kvs with
              | Some (`String s) -> VStr s
              | Some (`Int n) -> VStr (string_of_int n)
              | Some (`Intlit s) -> VStr s
              | Some (`Float f) -> VStr (string_of_float f)
              | Some (`Bool b) -> VStr (string_of_bool b)
              | Some (`Null) -> VNil
              | Some (#Yojson.Safe.t as v) ->
                  VStr (Yojson.Safe.to_string v)
              | None -> VNil)
         | _ -> VNil)
      with _ -> VNil)
  | _ -> raise (Interp_error (User_error ("dict-get needs 2 strings",
     "bad-args")))

(* (dict-set json-str key value) -> VStr new-json.
   Builds a JSON object with [key] = [value] (a string; numbers
   detected and emitted as JSON numeric where parseable). Returns the
   updated JSON as a VStr. If [json-str] isn't a JSON object, starts
   a fresh {} and adds the new key. *)
let prim_dict_set args =
  match args with
  | [ VStr json; VStr key; VStr value ] ->
      let base =
        try
          (match Yojson.Safe.from_string json with
           | `Assoc kvs -> kvs
           | _ -> [])
        with _ -> []
      in
      let v_json =
        (* Try parsing the value as JSON (so callers can pass nested
           objects/arrays); if it doesn't parse, emit as a JSON string. *)
        try Yojson.Safe.from_string value
        with _ -> `String value
      in
      let updated =
        (key, v_json) ::
        (List.filter (fun (k, _) -> k <> key) base)
      in
      VStr (Yojson.Safe.to_string (`Assoc updated))
  | _ -> raise (Interp_error (User_error ("dict-set needs 3 strings",
     "bad-args")))

(* (dict-keys json-str) -> VList of VStr keys.
   Returns the keys of the JSON object at [json-str] (insertion order
   preserved by Yojson's `Assoc). Empty list if input isn't an object. *)
let prim_dict_keys args =
  match args with
  | [ VStr json ] ->
      (try
        (match Yojson.Safe.from_string json with
         | `Assoc kvs ->
             VList (List.map (fun (k, _) -> VStr k) kvs)
         | _ -> VList [])
      with _ -> VList [])
  | _ -> raise (Interp_error (User_error ("dict-keys needs 1 string",
     "bad-args")))

```

```
(* (render-html html-str) -> VStr tagged-for-raw-rendering.
   Marks the returned VStr as raw HTML so http.ml renders it
   unescaped (instead of pretty-printing the value inside <pre>).
   The tag is a private prefix sentinel; http.ml checks for it. *)
let html_sentinel = "__html__:"
let redirect_sentinel = "__redirect__:"
let status_sentinel = "__status__:"

let prim_render_html args =
  match args with
  | [ VStr s ] -> VStr (html_sentinel ^ s)
  | _ -> raise (Interp_error (User_error ("render-html needs 1 string",
     "bad-args")))

(* Expose the sentinel so http.ml can import it (single source of
   truth for the marker). *)
let is_raw_html (v : value) : string option =
  match v with
  | VStr s when String.length s >= String.length html_sentinel
              && String.sub s 0 (String.length html_sentinel) =
     html_sentinel ->
      Some (String.sub s (String.length html_sentinel)
              (String.length s - String.length html_sentinel))
  | _ -> None

let redirect_value (path : string) : value =
  (* Single funnel for redirect targets (the redirect prim is its only
     caller). Reject CR/LF: httpun does not validate header values, so a
     newline in the target would let a handler inject response headers. *)
  if String.contains path '\r' || String.contains path '\n' then
    raise
      (Interp_error
         (User_error ("redirect target must not contain CR or LF",
     "bad-args")))
  else VStr (redirect_sentinel ^ path)

let is_redirect (v : value) : string option =
  match v with
  | VStr s when String.length s >= String.length redirect_sentinel
              && String.sub s 0 (String.length redirect_sentinel)
                 = redirect_sentinel ->
      Some (String.sub s (String.length redirect_sentinel)
              (String.length s - String.length redirect_sentinel))
  | _ -> None

(* A handler can signal a non-200 response by returning a status-tagged
   value: __status__:<code>\n<body>. The OCaml layer splits on the first
   newline and responds with that status + body. Used by op-seq rejection
   (409) and other handler-level error responses. Spec: structural-editing.
   client-eval OP-SEQ. *)
let status_response ~(code : int) ~(body : string) : value =
  VStr (status_sentinel ^ string_of_int code ^ "\n" ^ body)

(* (status-response code body) -> VStr tagged for non-200 response.
   Nopales-facing prim wrapper around status_response. *)
let prim_status_response args =
  match args with
  | [ VInt code; VStr body ] ->
      status_response ~code:(Int64.to_int code) ~body
  | [ VInt code; body ] ->
      status_response ~code:(Int64.to_int code)
        ~body:(coerce_str ~ctx:"status-response" body)
  | _ -> raise (Interp_error (User_error ("status-response needs code +
     body", "bad-args")))

let is_status (v : value) : (int * string) option =
  match v with
  | VStr s when String.length s >= String.length status_sentinel
              && String.sub s 0 (String.length status_sentinel)
                 = status_sentinel ->
      let rest =
        String.sub s (String.length status_sentinel)
          (String.length s - String.length status_sentinel)
```

```
      in
      (try
         let nl = String.index rest '\n' in
         let code = int_of_string (String.sub rest 0 nl) in
         let body = String.sub rest (nl + 1) (String.length rest - nl - 1)
     in
         Some (code, body)
       with _ -> None)
  | _ -> None

(* ---------------------------------------------------------------
   multipart/mixed values (multipart.mime-framing)

   Structured as VLists with symbolic tags (not string sentinels) so
   parts keep headers as data:
     part  = ( __pp_part__  body content-type target-or-() swap-or-()
     id-or-() )
     multi = ( __pp_multipart__  env-target env-swap  part? )
   --------------------------------------------------------------- *)

let part_tag = "__pp_part__"
let multipart_tag = "__pp_multipart__"

type mime_part =
  { body : string
  ; content_type : string
  ; target : string option
  ; swap : string option
  ; id : string option
  }

type mime_multipart =
  { envelope_target : string option
  ; envelope_swap : string option
  ; parts : mime_part list
  }

let opt_str = function
  | VNil | VList [] -> None
  | VStr s -> Some s
  | VSymbol s -> Some s
  | other -> Some (Pricklypear_lisp.Reader.pp_value other)

let str_or_raw = function
  | VStr s ->
      (match is_raw_html (VStr s) with Some raw -> raw | None -> s)
  | VSymbol s -> s
  | other -> Pricklypear_lisp.Reader.pp_value other

let kw_name = function
  | VSymbol s when String.length s > 1 && s.[0] = ':' ->
      Some (String.sub s 1 (String.length s - 1))
  | VStr s when String.length s > 1 && s.[0] = ':' ->
      Some (String.sub s 1 (String.length s - 1))
  | _ -> None

(** Split args into (kwargs, positionals).
    Supports leading kwargs, trailing kwargs, or positionals-then-kwargs:
      (part html :target "#x")  (multipart :target "#f" p1 p2) *)
let split_kwargs (args : value list) : (string * value) list * value list =
  let rec go kws pos = function
    | k :: v :: rest ->
        (match kw_name k with
         | Some name when String.length name > 0 ->
             go ((name, v) :: kws) pos rest
         | _ -> go kws (k :: pos) (v :: rest))
    | [ last ] -> go kws (last :: pos) []
    | [] -> (List.rev kws, List.rev pos)
  in
  go [] [] args

let kw_get (kws : (string * value) list) (name : string) : string option =
  match List.assoc_opt name kws with
```

```
  | None -> None
  | Some v -> opt_str v

let make_part_value ~(body : string) ~(content_type : string)
    ~(target : string option) ~(swap : string option) ~(id : string option)
  : value =
  let o = function None -> VNil | Some s -> VStr s in
  VList
    [ VSymbol part_tag
    ; VStr body
    ; VStr content_type
    ; o target
    ; o swap
    ; o id
    ]

let part_of_value = function
  | VList
      [ VSymbol t; VStr body; VStr ct; target; swap; id ]
    when t = part_tag ->
      Some
        { body
        ; content_type = ct
        ; target = opt_str target
        ; swap = opt_str swap
        ; id = opt_str id
        }
  | _ -> None

let is_part (v : value) : mime_part option = part_of_value v

let is_multipart (v : value) : mime_multipart option =
  match v with
  | VList (VSymbol t :: env_target :: env_swap :: rest) when t =
     multipart_tag
    ->
      let parts =
        List.filter_map part_of_value rest
      in
      if parts = [] && rest <> [] then None
      else
        Some
          { envelope_target = opt_str env_target
          ; envelope_swap = opt_str env_swap
          ; parts
          }
  | _ -> None

(* (part html [:target t] [:swap s] [:id i]) *)
let prim_part args =
  let kws, pos = split_kwargs args in
  match pos with
  | [ body ] ->
      make_part_value
        ~body:(str_or_raw body)
        ~content_type:"text/html; charset=utf-8"
        ~target:(kw_get kws "target")
        ~swap:(kw_get kws "swap")
        ~id:(kw_get kws "id")
  | _ ->
      raise
        (Interp_error
           (User_error
              ("part needs (part html [:target t] [:swap s] [:id i])",
    "bad-args")))

(* (json-part json-string [:id i]) *)
let prim_json_part args =
  let kws, pos = split_kwargs args in
  match pos with
  | [ body ] ->
      make_part_value
        ~body:(str_or_raw body)
```

```
        ~content_type:"application/json"
        ~target:(kw_get kws "target")
        ~swap:(kw_get kws "swap")
        ~id:(kw_get kws "id")
  | _ ->
      raise
        (Interp_error
           (User_error ("json-part needs (json-part json [:id i])",
    "bad-args")))

(* (multipart [:target t] [:swap s] part?)
   Also accepts a single list of parts: (multipart :target "#x" parts-list)
    *)
let prim_multipart args =
  let kws, pos = split_kwargs args in
  let pos =
    match pos with
    | [ VList items ] when items <> [] && List.for_all (fun v ->
    part_of_value v <> None) items
      -> items
    | [ VList items ] when items <> [] -> items
    | _ -> pos
  in
  let parts =
    List.map
      (fun v ->
        match part_of_value v with
        | Some _ -> v
        | None ->
            (* Bare string / raw-html becomes an html part. *)
            make_part_value
              ~body:(str_or_raw v)
              ~content_type:"text/html; charset=utf-8"
              ~target:None ~swap:None ~id:None)
      pos
  in
  let o = function None -> VNil | Some s -> VStr s in
  VList
    ( VSymbol multipart_tag
      :: o (kw_get kws "target")
      :: o (kw_get kws "swap")
      :: parts )

(* (string-eq a b) -> VBool. String equality for request-method checks etc.
    *)
let prim_string_eq args =
  match args with
  | [ a; b ] ->
      VBool
        (String.equal
           (coerce_str ~ctx:"string-eq" a)
           (coerce_str ~ctx:"string-eq" b))
  | _ -> raise (Interp_error (User_error ("string-eq needs 2 strings",
    "bad-args")))

(* (string<? a b), (string<=? a b), (string>? a b), (string>=? a b) ->
    VBool.
   Byte-wise lexicographic comparison (NOT numeric). Correct ordering for
   ISO dates (YYYY-MM-DD) and other fixed-width formats. *)
let prim_string_cmp name op args =
  match args with
  | [ a; b ] ->
      VBool
        (op
           (String.compare
              (coerce_str ~ctx:name a)
              (coerce_str ~ctx:name b))
           0)
  | _ ->
      raise
        (Interp_error
           (User_error (Printf.sprintf "%s needs 2 strings" name,
    "bad-args")))
```

```
996
997  let prim_string_lt args = prim_string_cmp "string<?" ( < ) args
998  let prim_string_le args = prim_string_cmp "string<=?" ( <= ) args
999  let prim_string_gt args = prim_string_cmp "string>?" ( > ) args
1000 let prim_string_ge args = prim_string_cmp "string>=?" ( >= ) args
1001
1002 (* (string-trim s) -> VStr. Trim leading/trailing whitespace. *)
1003 let prim_string_trim args =
1004   match args with
1005   | [ v ] -> VStr (String.trim (coerce_str ~ctx:"string-trim" v))
1006   | _ -> raise (Interp_error (User_error ("string-trim needs 1 string",
        "bad-args")))
1007
1008 (* JSON <-> Lisp values for outline trees and generic data.
1009    Objects become VStr of JSON object (use dict-get); arrays -> VList. *)
1010 let rec json_to_value (j : Yojson.Safe.t) : value =
1011   match j with
1012   | `Null -> VNil
1013   | `Bool b -> VBool b
1014   | `Int n -> VInt (Int64.of_int n)
1015   | `Intlit s -> (try VInt (Int64.of_string s) with _ -> VStr s)
1016   | `Float f -> VFloat f
1017   | `String s -> VStr s
1018   | `List xs -> VList (List.map json_to_value xs)
1019   | `Assoc _ as o -> VStr (Yojson.Safe.to_string o)
1020
1021 let rec value_to_json (v : value) : Yojson.Safe.t =
1022   match v with
1023   | VNil -> `Null
1024   | VBool b -> `Bool b
1025   | VInt n -> `Intlit (Int64.to_string n)
1026   | VStr s ->
1027       (* Nested JSON object/array strings stay structured when emitted. *)
1028       let len = String.length s in
1029       if len >= 2
1030          && ((s.[0] = '{' && s.[len - 1] = '}')
1031              || (s.[0] = '[' && s.[len - 1] = ']'))
1032       then (try Yojson.Safe.from_string s with _ -> `String s)
1033       else `String s
1034   | VList xs -> `List (List.map value_to_json xs)
1035   | VFloat f ->
1036       (* Real JSON float so it round-trips back to VFloat (not a VStr).
1037          Integral floats still come back as VFloat via [json_to_value]
1038          (Float f -> VFloat f), keeping the type distinct from VInt. *)
1039       `Float f
1040   | VPair (a, b) ->
1041       let rec unwind acc = function
1042         | VPair (x, y) -> unwind (x :: acc) y
1043         | VNil -> List.rev acc
1044         | VList ys -> List.rev_append acc ys
1045         | tail -> List.rev (tail :: acc)
1046       in
1047       `List (List.map value_to_json (unwind [] (VPair (a, b))))
1048   | other -> `String (Pricklypear_lisp.Reader.pp_value other)
1049
1050 let prim_json_parse args =
1051   match args with
1052   | [ VStr s ] ->
1053       (try json_to_value (Yojson.Safe.from_string s)
1054        with _ -> raise (Interp_error (User_error ("json-parse: invalid
       JSON", "bad-args"))))
1055   | _ -> raise (Interp_error (User_error ("json-parse needs 1 string",
        "bad-args")))
1056
1057 let prim_json_emit args =
1058   match args with
1059   | [ v ] -> VStr (Yojson.Safe.to_string (value_to_json v))
1060   | _ -> raise (Interp_error (User_error ("json-emit needs 1 value",
        "bad-args")))
1061
1062 (* (read-string s) -> quoted value tree. Same reader as load-string / the
1063    REPL, then quote_to_value -- NO eval. Pair with (function-ast name):
1064    both yield walkable s-expr data. Clojure-style twin of load-string.
```

```
   (read-all-string s) -> VList of forms (multi top-level). *)
let prim_read_string args =
  match args with
  | [ VStr s ] ->
      (try
         let e = Pricklypear_lisp.Reader.read_one s in
         Pricklypear_lisp.Eval.quote_to_value e
       with
       | Pricklypear_lisp.Reader.Reader_error (msg, line, col) ->
           raise
             (Interp_error
                (User_error
                   (Printf.sprintf "read-string: %s (%d:%d)" msg line col,
    "bad-args")))
       | exn ->
           raise
             (Interp_error
                (User_error
                   ("read-string: " ^ Printexc.to_string exn, "bad-args"))))
  | _ -> raise (Interp_error (User_error ("read-string needs 1 string",
    "bad-args")))

let prim_read_all_string args =
  match args with
  | [ VStr s ] ->
      (try
         let es = Pricklypear_lisp.Reader.read_all s in
         VList (List.map Pricklypear_lisp.Eval.quote_to_value es)
       with
       | Pricklypear_lisp.Reader.Reader_error (msg, line, col) ->
           raise
             (Interp_error
                (User_error
                   (Printf.sprintf "read-all-string: %s (%d:%d)" msg line
    col, "bad-args")))
       | exn ->
           raise
             (Interp_error
                (User_error
                   ("read-all-string: " ^ Printexc.to_string exn,
    "bad-args"))))
  | _ -> raise (Interp_error (User_error ("read-all-string needs 1 string",
    "bad-args")))

(* (html-escape s) -> VStr. Escape amp lt gt quote apos for HTML
    text/attrs. *)
let rec prim_html_escape args =
  match args with
  | [ VStr s ] ->
      let buf = Buffer.create (String.length s + 8) in
      String.iter
        (function
          | '&' -> Buffer.add_string buf "&amp;"
          | '<' -> Buffer.add_string buf "&lt;"
          | '>' -> Buffer.add_string buf "&gt;"
          | '"' -> Buffer.add_string buf "&quot;"
          | '\'' -> Buffer.add_string buf "&#39;"
          | c -> Buffer.add_char buf c)
        s;
      VStr (Buffer.contents buf)
  | [ v ] ->
      (* coerce then escape *)
      let s =
        match v with
        | VStr s -> s
        | VInt n -> Int64.to_string n
        | VBool true -> "true"
        | VBool false -> "false"
        | VNil -> ""
        | _ -> Pricklypear_lisp.Reader.pp_value v
      in
      prim_html_escape [ VStr s ]
```

```
  | _ -> raise (Interp_error (User_error ("html-escape needs 1 value",
     "bad-args")))

(* (hx-get url) -> VStr tagged attribute helper (lightweight; authors may
   also embed Hx attrs inside render-html). *)
let prim_hx_get args =
  match args with
  | [ VStr url ] -> VStr ("hx-get:" ^ url)
  | _ -> raise (Interp_error (User_error ("hx-get needs a string url",
     "bad-args")))

let prim_hx_post args =
  match args with
  | [ VStr url ] -> VStr ("hx-post:" ^ url)
  | _ -> raise (Interp_error (User_error ("hx-post needs a string url",
     "bad-args")))

(* (err/shape msg kind) -> VList (error msg kind).
   Constructs the guard-handler payload shape as data so Lisp code can
   build/inspect error envelopes without raising. Mirrors the VList the
   guard special form delivers to its handler lambda. See design note
   error-shape-design ?3,?x-2a. *)
let prim_err_shape args =
  match args with
  | [ VStr msg; VStr kind ] ->
      VList [ VSymbol "error"; VStr msg; VSymbol kind ]
  | [ msg; kind ] ->
      (* coerce: symbols accepted as kind/msg for ergonomics *)
      let s = function VStr s -> s | VSymbol s -> s | v ->
     Pricklypear_lisp.Reader.pp_value v in
      VList [ VSymbol "error"; VStr (s msg); VSymbol (s kind) ]
  | _ -> raise (Interp_error (User_error ("err/shape needs msg and kind",
     "bad-args")))

(* (count list) -> int. Demonstrates a prim that walks the evaluator's
   own value type. *)
let prim_count args =
  match args with
  | [ VList items ] -> VInt (Int64.of_int (List.length items))
  | [ VNil ] -> VInt 0L
  | _ -> raise (Interp_error (User_error ("count needs a list",
     "bad-args")))

(* (length xs) -> VInt. Like count but accepts VList, VNil, and proper
   VPair chains (the conventional Lisp `length`). Frozen std name. *)
let prim_length args =
  match args with
  | [ xs ] -> VInt (Int64.of_int (List.length (list_of_value xs)))
  | _ -> raise (Interp_error (User_error ("length needs one list",
     "bad-args")))

(* Structural deep equality (equal?): recurses VList/VPair/VStr/VInt/
   VFloat/VBool/VSymbol/VNil. VList and VPair chains are compared
   element-wise as sequences, so `(list 1 2 3)` is equal? to the
   equivalent cons-chain; a dotted/improper pair is NOT equal? to the
   same proper list (the atom tail falls through to a type mismatch).
   VNil and the empty VList are equal (both the empty list). Different
   value types are not equal? (1 is not 1.0). *)
let rec value_equal (a : value) (b : value) : bool =
  match a, b with
  | VInt x, VInt y -> Int64.equal x y
  | VFloat x, VFloat y -> x = y
  | VBool x, VBool y -> x = y
  | VStr x, VStr y -> String.equal x y
  | VSymbol x, VSymbol y -> String.equal x y
  | VNil, VNil -> true
  | VList [], VNil | VNil, VList [] | VList [], VList [] -> true
  | VList (h1 :: t1), VList (h2 :: t2) ->
      value_equal h1 h2 && value_equal (VList t1) (VList t2)
  | VList (h1 :: t1), VPair (h2, t2) ->
      value_equal h1 h2 && value_equal (VList t1) t2
  | VPair (h1, t1), VList (h2 :: t2) ->
      value_equal h1 h2 && value_equal t1 (VList t2)
```

```
  | VPair (h1, t1), VPair (h2, t2) ->
      value_equal h1 h2 && value_equal t1 t2
  | _ -> false

let prim_equal_p args =
  match args with
  | [ a; b ] -> VBool (value_equal a b)
  | _ -> raise (Interp_error (User_error ("equal? needs two args",
     "bad-args")))

(* c*r compositions: reuse car/cdr so error messages + VPair/VList
   handling stay consistent. cadr/caddr/caar/cddr are bread-and-butter
   for code-as-data walking. *)
let prim_cadr args =
  match args with
  | [ v ] -> prim_car [ prim_cdr [ v ] ]
  | _ -> raise (Interp_error (User_error ("cadr needs one arg",
     "bad-args")))

let prim_caar args =
  match args with
  | [ v ] -> prim_car [ prim_car [ v ] ]
  | _ -> raise (Interp_error (User_error ("caar needs one arg",
     "bad-args")))

let prim_caddr args =
  match args with
  | [ v ] -> prim_car [ prim_cdr [ prim_cdr [ v ] ] ]
  | _ -> raise (Interp_error (User_error ("caddr needs one arg",
     "bad-args")))

let prim_cddr args =
  match args with
  | [ v ] -> prim_cdr [ prim_cdr [ v ] ]
  | _ -> raise (Interp_error (User_error ("cddr needs one arg",
     "bad-args")))

(* (assoc key alist) -> the first entry (pair or list) whose car is
   equal? to key, or VNil if absent. Entries may be (k . v) pairs or
   (k v) two-element lists. *)
let prim_assoc args =
  match args with
  | [ key; alist ] ->
      let rec search = function
      | VNil | VList [] -> VNil
      | VList (entry :: rest) ->
          (match entry with
           | VPair (k, _) when value_equal k key -> entry
           | VList (k :: _) when value_equal k key -> entry
           | _ -> search (VList rest))
      | VPair (entry, rest) ->
          (match entry with
           | VPair (k, _) when value_equal k key -> entry
           | VList (k :: _) when value_equal k key -> entry
           | _ -> search rest)
      | _ -> VNil
      in
      search alist
  | _ -> raise (Interp_error (User_error ("assoc needs key and alist",
     "bad-args")))

(* (member x list) -> the sublist starting at the first element
   equal? to x (truthy), or #f if absent. *)
let prim_member args =
  match args with
  | [ x; lst ] ->
      let rec search = function
      | VNil | VList [] -> VBool false
      | VList (h :: t) ->
          if value_equal h x then VList (h :: t) else search (VList t)
      | VPair (h, t) ->
          if value_equal h x then VPair (h, t) else search t
      | _ -> VBool false
```

```
      in
      search lst
  | _ -> raise (Interp_error (User_error ("member needs x and list",
     "bad-args")))
(* (parse-int s|n) -> VInt. Soft-fail to 0 on bad string. *)
let prim_parse_int (args : value list) : value =
  match args with
  | [ VInt n ] -> VInt n
  | [ VStr s ] ->
      (try VInt (Int64.of_int (int_of_string (String.trim s)))
       with _ -> VInt 0L)
  | _ -> raise (Interp_error (User_error ("parse-int needs string or int",
     "bad-args")))

(** FSRS-4.5 schedule step (open-spaced-repetition defaults).

    (fsrs-schedule stability difficulty elapsed_days rating)
      stability/difficulty: float or int; use 0 0 for a new card
      elapsed_days: days since last review (0 for first)
      rating: 1..4 or again|hard|good|easy

    Returns (list S' D' interval_days) where interval_days is int >= 0
    (0 means relearn today). request_retention fixed at 0.9. *)
let fsrs_w : float array =
  [| 0.40255; 1.18385; 3.173; 15.69105; 7.1949; 0.5345; 1.4604; 0.0046;
     1.54575; 0.1192; 1.01925; 1.9395; 0.11; 0.29605; 2.2698; 0.2315;
     2.9898; 0.51655; 0.6621 |]

let fsrs_clamp_d (d : float) : float = max 1. (min 10. d)

let fsrs_init_stability (grade : int) : float =
  (* w[0..3] for Again..Easy *)
  max 0.1 fsrs_w.(grade - 1)

let fsrs_init_difficulty (grade : int) : float =
  fsrs_clamp_d (fsrs_w.(4) -. exp (fsrs_w.(5) *. float_of_int (grade - 1))
     +. 1.)

let fsrs_next_difficulty (d : float) (grade : int) : float =
  let delta = -. fsrs_w.(6) *. float_of_int (grade - 3) in
  let d' = d +. delta *. (10. -. d) /. 9. in
  let d0_good = fsrs_init_difficulty 3 in
  fsrs_clamp_d (fsrs_w.(7) *. d0_good +. (1. -. fsrs_w.(7)) *. d')

let fsrs_retrievability ~(s : float) ~(elapsed : float) : float =
  if s <= 0. then 0.
  else (1. +. elapsed /. (9. *. s)) ** (-1.)

let fsrs_next_stability_success ~(s : float) ~(d : float) ~(r : float)
    ~(grade : int) : float =
  let hard_pen = if grade = 2 then fsrs_w.(15) else 1. in
  let easy_bon = if grade = 4 then fsrs_w.(16) else 1. in
  let s' =
    s
    *. (exp fsrs_w.(8)
        *. (11. -. d)
        *. (s ** (-. fsrs_w.(9)))
        *. (exp (fsrs_w.(10) *. (1. -. r)) -. 1.)
        *. hard_pen
        *. easy_bon
        +. 1.)
  in
  max s' s  (* stability should not decrease on success in v1 *)

let fsrs_next_stability_fail ~(s : float) ~(d : float) ~(r : float) : float
    =
  let s' =
    fsrs_w.(11)
    *. (d ** (-. fsrs_w.(12)))
    *. (((s +. 1.) ** fsrs_w.(13)) -. 1.)
    *. exp (fsrs_w.(14) *. (1. -. r))
  in
  min s (max 0.1 s')
```

```
1333
1334 let fsrs_interval_days ~(s : float) : int =
1335   (* For R=0.9, t ? S (FSRS-4 power formula). *)
1336   max 0 (int_of_float (Float.round (max 0. s)))
1337
1338 let fsrs_parse_grade = function
1339   | VInt n ->
1340       let g = Int64.to_int n in
1341       if g < 1 || g > 4 then None else Some g
1342   | VStr s ->
1343       (match String.lowercase_ascii (String.trim s) with
1344        | "again" | "1" -> Some 1
1345        | "hard" | "2" -> Some 2
1346        | "good" | "3" -> Some 3
1347        | "easy" | "4" -> Some 4
1348        | _ -> None)
1349   | _ -> None
1350
1351 let float_arg v =
1352   match num_of_value v with
1353   | NInt n -> Int64.to_float n
1354   | NFloat f -> f
1355
1356 let prim_fsrs_schedule (args : value list) : value =
1357   match args with
1358   | [ s_v; d_v; elapsed_v; grade_v ] ->
1359       (match fsrs_parse_grade grade_v with
1360        | None ->
1361            raise
1362              (Interp_error
1363                 (User_error
1364                    ("fsrs-schedule: rating must be 1..4 or
     again|hard|good|easy", "bad-args")))
1365        | Some grade ->
1366            let s0 = float_arg s_v in
1367            let d0 = float_arg d_v in
1368            let elapsed = max 0. (float_arg elapsed_v) in
1369            let is_new = s0 <= 0. || d0 <= 0. in
1370            let s, d =
1371              if is_new then
1372                (fsrs_init_stability grade, fsrs_init_difficulty grade)
1373              else
1374                let r = fsrs_retrievability ~s:s0 ~elapsed in
1375                let d' = fsrs_next_difficulty d0 grade in
1376                let s' =
1377                  if grade = 1 then
1378                    fsrs_next_stability_fail ~s:s0 ~d:d0 ~r
1379                  else
1380                    fsrs_next_stability_success ~s:s0 ~d:d0 ~r ~grade
1381                in
1382                (s', d')
1383            in
1384            let interval =
1385              if grade = 1 then 0
1386              else if is_new && grade = 2 then 1
1387              else if is_new && grade = 3 then max 1 (fsrs_interval_days ~s)
1388              else if is_new && grade = 4 then max 2 (fsrs_interval_days ~s)
1389              else max 1 (fsrs_interval_days ~s)
1390            in
1391            VList [ VFloat s; VFloat d; VInt (Int64.of_int interval) ])
1392   | _ ->
1393       raise
1394         (Interp_error
1395            (User_error
1396               ("fsrs-schedule needs stability difficulty elapsed rating",
     "bad-args")))
1397
1398 (* ---- extract-links: OCaml wiki-link scanner (budget P1) ----
1399
1400    (extract-links body-string) -> list of (slug label) pairs.
1401    Parses [[slug]] and [[slug|label]] left-to-right.
1402    Empty targets ([[]]) are skipped. Unclosed [[ is ignored (no hang).
1403    Media syntax [[media:uuid|alt]] is included (caller may filter);
```

```
   roam/insert-collected already skips media: via roam/media-slug?.

   Char-by-char Lisp recursion was ~4-6 steps/char and blew the 100k
   budget on ~2-4k bodies. This prim is O(n) in OCaml = 1 budget tick. *)

let extract_links_from_string (s : string) : (string * string) list =
  let len = String.length s in
  let rec find_close i =
    if i >= len - 1 then None
    else if s.[i] = ']' && s.[i + 1] = ']' then Some i
    else find_close (i + 1)
  in
  let split_label inner =
    let inner = String.trim inner in
    match String.index_opt inner '|' with
    | None -> (inner, inner)
    | Some p ->
        let slug = String.trim (String.sub inner 0 p) in
        let lab =
          String.trim
            (String.sub inner (p + 1) (String.length inner - p - 1))
        in
        (slug, lab)
  in
  let rec loop i acc =
    if i >= len - 3 then List.rev acc
    else if s.[i] = '[' && s.[i + 1] = '[' then
      match find_close (i + 2) with
      | None -> loop (i + 1) acc
      | Some end_i ->
          let inner = String.sub s (i + 2) (end_i - (i + 2)) in
          let slug, lab = split_label inner in
          if slug = "" then loop (end_i + 2) acc
          else loop (end_i + 2) ((slug, lab) :: acc)
    else loop (i + 1) acc
  in
  loop 0 []

let prim_extract_links args =
  let of_body body =
    let pairs = extract_links_from_string body in
    VList
      (List.map
         (fun (slug, lab) -> VList [ VStr slug; VStr lab ])
         pairs)
  in
  match args with
  | [ VStr body ] -> of_body body
  | [ VNil ] -> VList []
  | [ other ] ->
      let body =
        match other with
        | VStr s -> s
        | VInt n -> Int64.to_string n
        | VFloat f -> string_of_float f
        | VBool true -> "true"
        | VBool false -> "false"
        | VNil -> ""
        | _ ->
            raise
              (Interp_error
                 (User_error ("extract-links needs a string body",
    "bad-args")))
      in
      of_body body
  | _ ->
      raise
        (Interp_error (User_error ("extract-links needs one string arg",
    "bad-args")))

(* (budget-remaining) -> VInt steps left in current eval call. *)
let prim_budget_remaining _args =
  VInt (Int64.of_int (Pricklypear_lisp.Eval.budget_remaining_steps ()))
```

```
(* ---- render-links: OCaml wiki/media body ? HTML (budget view path) ----

   (render-links body-string) -> HTML string matching roam/render-body:
   plain text html-escaped; [[slug]]/[[slug|label]] ? notes anchors;
   [[media:uuid|alt]] ? linked img + caption. Unclosed [[ escapes one
   char and continues. Empty [[]] emits escaped brackets. *)

let html_escape_to_buffer buf s =
  String.iter
    (function
      | '&' -> Buffer.add_string buf "&amp;"
      | '<' -> Buffer.add_string buf "&lt;"
      | '>' -> Buffer.add_string buf "&gt;"
      | '"' -> Buffer.add_string buf "&quot;"
      | '\'' -> Buffer.add_string buf "&#39;"
      | c -> Buffer.add_char buf c)
    s

let html_escape_str s =
  let buf = Buffer.create (String.length s + 8) in
  html_escape_to_buffer buf s;
  Buffer.contents buf

let media_prefix = "media:"

let is_media_slug slug =
  let n = String.length media_prefix in
  String.length slug >= n && String.sub slug 0 n = media_prefix

let media_id_of slug =
  let n = String.length media_prefix in
  String.sub slug n (String.length slug - n)

(* Parse (bible BOOK ch [verse] [verse-to] ["label"]) starting at '('.
    Returns
   Some (end_exclusive, book, chapter, verse_opt, label). verse_opt is "" if
   chapter-only. label defaults to a human ref. *)
let parse_bible_sexp_at (s : string) (start : int) :
    (int * string * string * string * string) option =
  let len = String.length s in
  if start >= len || s.[start] <> '(' then None
  else
    let skip_ws i =
      let rec go i =
        if i < len && (s.[i] = ' ' || s.[i] = '\t' || s.[i] = '\n' || s.[i]
    = '\r')
        then go (i + 1)
        else i
      in
      go i
    in
    let i0 = skip_ws (start + 1) in
    if i0 + 5 > len then None
    else if String.lowercase_ascii (String.sub s i0 5) <> "bible" then None
    else
      let after_kw = i0 + 5 in
      let ok_kw_end =
        after_kw >= len
        ||
        let c = s.[after_kw] in
        c = ' ' || c = '\t' || c = '\n' || c = '\r' || c = '"' || c = ')'
        || (c >= '0' && c <= '9')
      in
      if not ok_kw_end then None
      else
        let parse_token i =
          let i = skip_ws i in
          if i >= len then None
          else if s.[i] = ')' then Some (`Close, i + 1)
          else if s.[i] = '"' then
            let rec find_q j =
              if j >= len then None
```

```
              else if s.[j] = '\\' && j + 1 < len then find_q (j + 2)
              else if s.[j] = '"' then Some j
              else find_q (j + 1)
            in
            (match find_q (i + 1) with
             | None -> None
             | Some q ->
                 let raw = String.sub s (i + 1) (q - i - 1) in
                 Some (`Str raw, q + 1))
          else
            let rec end_atom j =
              if j >= len then j
              else
                match s.[j] with
                | ' ' | '\t' | '\n' | '\r' | ')' | '"' -> j
                | _ -> end_atom (j + 1)
            in
            let j = end_atom i in
            if j = i then None
            else Some (`Atom (String.sub s i (j - i)), j)
        in
        let rec gather i acc =
          match parse_token i with
          | None -> None
          | Some (`Close, i') -> Some (List.rev acc, i')
          | Some (`Str v, i') | Some (`Atom v, i') -> gather i' (v :: acc)
        in
        match gather after_kw [] with
        | None -> None
        | Some (args, end_i) ->
            (match args with
             | book :: ch :: rest when book <> "" && ch <> "" ->
                 let verse, label =
                   match rest with
                   | [] -> ("", book ^ " " ^ ch)
                   | [ v ] ->
                       (* verse or label-only? prefer verse if numeric *)
                       let numeric =
                         String.length v > 0
                         && String.for_all (function '0' .. '9' -> true | _
    -> false) v
                       in
                       if numeric then (v, book ^ " " ^ ch ^ ":" ^ v)
                       else ("", v)
                   | v :: lab :: _ ->
                       let numeric =
                         String.length v > 0
                         && String.for_all (function '0' .. '9' -> true | _
    -> false) v
                       in
                       if numeric then
                         let lab =
                           if lab = "" then book ^ " " ^ ch ^ ":" ^ v else
    lab
                         in
                         (v, lab)
                       else ("", v)
                 in
                 Some (end_i, book, ch, verse, label)
             | _ -> None)

let render_links_from_string (s : string) : string =
  let len = String.length s in
  let buf = Buffer.create (len + 64) in
  let rec find_close i =
    if i >= len - 1 then None
    else if s.[i] = ']' && s.[i + 1] = ']' then Some i
    else find_close (i + 1)
  in
  let split_label inner =
    let inner = String.trim inner in
    match String.index_opt inner '|' with
    | None -> (inner, inner)
```

```
    | Some p ->
        let slug = String.trim (String.sub inner 0 p) in
        let lab =
          String.trim
            (String.sub inner (p + 1) (String.length inner - p - 1))
        in
        (slug, lab)
  in
  let emit_wiki slug lab =
    Buffer.add_string buf "<a href=\"/notes?slug=";
    html_escape_to_buffer buf slug;
    Buffer.add_string buf "\">";
    html_escape_to_buffer buf lab;
    Buffer.add_string buf "</a>"
  in
  let emit_bible book ch verse lab =
    Buffer.add_string buf "<a class=\"bible-ref\" href=\"/dabar?book=";
    html_escape_to_buffer buf book;
    Buffer.add_string buf "&chapter=";
    html_escape_to_buffer buf ch;
    if verse <> "" then begin
      Buffer.add_string buf "&verse=";
      html_escape_to_buffer buf verse
    end;
    Buffer.add_string buf "\">";
    html_escape_to_buffer buf lab;
    Buffer.add_string buf "</a>"
  in
  let emit_media id alt =
    let href = "/media/" ^ id in
    let label = if alt = "" then id else alt in
    Buffer.add_string buf "<a href=\"";
    html_escape_to_buffer buf href;
    Buffer.add_string buf "\"><img src=\"";
    html_escape_to_buffer buf href;
    Buffer.add_string buf "\" alt=\"";
    html_escape_to_buffer buf label;
    Buffer.add_string buf "\" class=\"note-media\"></a>";
    Buffer.add_string buf "<p class=\"muted note-media-cap\"><a href=\"";
    html_escape_to_buffer buf href;
    Buffer.add_string buf "\">";
    html_escape_to_buffer buf label;
    Buffer.add_string buf "</a></p>"
  in
  let at_open i =
    i < len - 1 && s.[i] = '[' && s.[i + 1] = '['
  in
  let rec loop i =
    if i >= len then ()
    else if at_open i then
      match find_close (i + 2) with
      | None ->
          html_escape_to_buffer buf (String.sub s i 1);
          loop (i + 1)
      | Some end_i ->
          let inner = String.sub s (i + 2) (end_i - (i + 2)) in
          let slug, lab = split_label inner in
          if slug = "" then begin
            html_escape_to_buffer buf
              (String.sub s i (end_i + 2 - i));
            loop (end_i + 2)
          end else if is_media_slug slug then begin
            emit_media (media_id_of slug) lab;
            loop (end_i + 2)
          end else begin
            emit_wiki slug lab;
            loop (end_i + 2)
          end
    else
      match parse_bible_sexp_at s i with
      | Some (end_i, book, ch, verse, lab) ->
          emit_bible book ch verse lab;
          loop end_i
```

```
      | None ->
          let j = ref i in
          while !j < len && not (at_open !j) && s.[!j] <> '(' do
            incr j
          done;
          (* If we stopped on '(', try bible; else emit through j. If '(' but
             not bible, emit the paren and continue. *)
          if !j > i then begin
            html_escape_to_buffer buf (String.sub s i (!j - i));
            loop !j
          end else if s.[i] = '(' then begin
            html_escape_to_buffer buf "(";
            loop (i + 1)
          end else
            loop (i + 1)
  in
  loop 0;
  Buffer.contents buf

let prim_render_links args =
  let of_body body = VStr (render_links_from_string body) in
  match args with
  | [ VStr body ] -> of_body body
  | [ VNil ] -> VStr ""
  | [ other ] ->
      let body =
        match other with
        | VStr s -> s
        | VInt n -> Int64.to_string n
        | VFloat f -> string_of_float f
        | VBool true -> "true"
        | VBool false -> "false"
        | VNil -> ""
        | _ ->
            raise
              (Interp_error
                 (User_error ("render-links needs a string body", "bad-args")))
      in
      of_body body
  | _ ->
      raise
        (Interp_error (User_error ("render-links needs one string arg", "bad-args")))

(** habitat-book web-renderer: prose tokenization for binding-token
    links. Both prims are ONE OCaml scan over the prose (Buffer + direct
    char tests, zero interpreter steps) -- the interpreted per-word
    pipeline they replace (book/split-piece + spec/token-binding? per
    word in libs/book) cost ~300 steps per word, so a 12.7k-word chapter
    needed ~4M steps against a 100k page budget. Resolution stays in
    Nopales (spec/binding-resolves? via function-source) but at
    O(unique tokens), not O(words).

    (prose-tokens s) -> list of UNIQUE token-shaped cores in first-
    occurrence order. Whitespace runs (space/tab/newline) separate
    words; at most one leading char from ( [ ' , and one trailing char
    from ) ] , . ; : is stripped as edge punctuation (the retired
    book/split-piece sets, minus \n which is now a separator). The
    token-shape test MIRRORS spec/token-binding? (len>2, contains "/",
    not "->", not "/", no space, not leading "/") -- spec/ keeps the
    authoritative definition; keep the two in lockstep (parity-tested
    in image/test/primitives_test.ml). *)
let token_shape (core : string) : bool =
  String.length core > 2
  && String.contains core '/'
  && core <> "->"
  && core <> "/"
  && not (String.contains core ' ')
  && core.[0] <> '/'

let prose_token_edges (s : string) : string * string * string =
```

```
  let is_lead = function '(' | '[' | '\'' | ',' -> true | _ -> false in
  let is_trail = function ')' | ']' | ',' | '.' | ';' | ':' -> true | _ ->
     false in
  let n = String.length s in
  let lead = if n > 0 && is_lead s.[0] then 1 else 0 in
  let s1 = if lead = 1 then String.sub s 1 (n - 1) else s in
  let n1 = String.length s1 in
  let trail = if n1 > 0 && is_trail s1.[n1 - 1] then 1 else 0 in
  (String.sub s 0 lead,
   (if n1 - trail > 0 then String.sub s1 0 (n1 - trail) else ""),
   (if trail = 1 then String.sub s1 (n1 - 1) 1 else ""))

let is_word_char = function
  | ' ' | '\t' | '\n' | '\r' -> false
  | _ -> true

let prim_prose_tokens args =
  let s = coerce_str ~ctx:"prose-tokens" (match args with [ v ] -> v | _ ->
     raise (Interp_error (User_error ("prose-tokens needs one string",
     "bad-args")))) in
  let seen = Hashtbl.create 64 in
  let out = ref [] in
  let n = String.length s in
  let i = ref 0 in
  while !i < n do
    if is_word_char s.[!i] then begin
      let j = ref !i in
      while !j < n && is_word_char s.[!j] do incr j done;
      let _, core, _ = prose_token_edges (String.sub s !i (!j - !i)) in
      if core <> "" && token_shape core && not (Hashtbl.mem seen core) then
     begin
        Hashtbl.add seen core ();
        out := VStr core :: !out
      end;
      i := !j
    end else incr i
  done;
  VList (List.rev !out)

(** (render-tokens s resolved unresolved) -> HTML string for one prose
    paragraph body. Same word scan as prose-tokens; each word renders as
    escape(lead) ++ core-html ++ escape(trail) where core-html is:
      - core in `resolved`   -> <a href="/binding/CORE">CORE</a>
      - core in `unresolved` -> <span class="book-hole">CORE&#8709;</span>
      - otherwise (incl. empty core / non-token-shaped word) -> the whole
        word escaped verbatim
    Inter-word whitespace is preserved verbatim. Presentation constants
    (/binding/ href, book-hole class) mirror render-links' kernel-side
    anchors (/notes?slug=); the book package owns the .book-hole CSS. *)
let prim_render_tokens args =
  let unwrap = function
    | VList xs -> xs
    | VNil -> []
    | _ -> raise (Interp_error (User_error ("render-tokens needs lists of
    strings", "bad-args")))
  in
  match args with
  | [ s_v; resolved_v; unresolved_v ] ->
      let s = coerce_str ~ctx:"render-tokens" s_v in
      let strset vals =
        let h = Hashtbl.create 64 in
        List.iter (fun v -> Hashtbl.replace h (coerce_str
    ~ctx:"render-tokens" v) ()) vals;
        h
      in
      let resolved = strset (unwrap resolved_v) in
      let unresolved = strset (unwrap unresolved_v) in
      let buf = Buffer.create (String.length s + 16) in
      let n = String.length s in
      let i = ref 0 in
      while !i < n do
        if is_word_char s.[!i] then begin
          let j = ref !i in
```

```
          while !j < n && is_word_char s.[!j] do incr j done;
          let word = String.sub s !i (!j - !i) in
          let lead, core, trail = prose_token_edges word in
          if core <> "" && token_shape core && Hashtbl.mem resolved core
     then begin
            html_escape_to_buffer buf lead;
            Buffer.add_string buf "<a href=\"/binding/";
            html_escape_to_buffer buf core;
            Buffer.add_string buf "\">";
            html_escape_to_buffer buf core;
            Buffer.add_string buf "</a>";
            html_escape_to_buffer buf trail
          end
          else if core <> "" && token_shape core && Hashtbl.mem unresolved
     core then begin
            html_escape_to_buffer buf lead;
            Buffer.add_string buf "<span class=\"book-hole\">";
            html_escape_to_buffer buf core;
            Buffer.add_string buf "&#8709;</span>";
            html_escape_to_buffer buf trail
          end
          else html_escape_to_buffer buf word;
          i := !j
        end else begin
          Buffer.add_char buf s.[!i];
          incr i
        end
      done;
      VStr (Buffer.contents buf)
  | _ ->
      raise (Interp_error (User_error ("render-tokens needs (s resolved
     unresolved)", "bad-args")))

(* ---- string helpers (P2 from kernel-io-primitives note) ---- *)

let prim_string_replace args =
  match args with
  | [ VStr s; VStr pat; VStr repl ] ->
      let plen = String.length pat in
      if plen = 0 then VStr s
      else
        let buf = Buffer.create (String.length s) in
        let rec loop i =
          if i + plen > String.length s then begin
            Buffer.add_substring buf s i (String.length s - i);
            Buffer.contents buf
          end else if String.sub s i plen = pat then begin
            Buffer.add_string buf repl;
            loop (i + plen)
          end else begin
            Buffer.add_char buf s.[i];
            loop (i + 1)
          end
        in
        VStr (loop 0)
  | _ -> raise (Interp_error (User_error ("string-replace needs (s pat
     repl)", "bad-args")))

let prim_string_downcase args =
  match args with
  | [ v ] -> VStr (String.lowercase_ascii (coerce_str
     ~ctx:"string-downcase" v))
  | _ -> raise (Interp_error (User_error ("string-downcase needs 1 string",
     "bad-args")))

let prim_string_upcase args =
  match args with
  | [ v ] -> VStr (String.uppercase_ascii (coerce_str ~ctx:"string-upcase"
     v))
  | _ -> raise (Interp_error (User_error ("string-upcase needs 1 string",
     "bad-args")))

let prim_string_starts_with_p args =
```

```
  match args with
  | [ VStr s; VStr pfx ] -> VBool (String.starts_with ~prefix:pfx s)
  | [ a; b ] ->
      VBool (String.starts_with
               ~prefix:(coerce_str ~ctx:"string-starts-with?" b)
               (coerce_str ~ctx:"string-starts-with?" a))
  | _ -> raise (Interp_error (User_error ("string-starts-with? needs 2
     strings", "bad-args")))

let prim_string_ends_with_p args =
  match args with
  | [ VStr s; VStr sfx ] -> VBool (String.ends_with ~suffix:sfx s)
  | [ a; b ] ->
      VBool (String.ends_with
               ~suffix:(coerce_str ~ctx:"string-ends-with?" b)
               (coerce_str ~ctx:"string-ends-with?" a))
  | _ -> raise (Interp_error (User_error ("string-ends-with? needs 2
     strings", "bad-args")))

(* (string-split s delim) -> VList of VStr.
   Matches the libs/std Nopales string-split semantics so libs/hyper runs
   unchanged on hosts that don't load libs/std (the agent): empty/no-match
   -> single-element list of the whole string; otherwise splits on each
   non-overlapping occurrence of delim, preserving empty trailing
   segments. KERNEL PRIM as of 2026-08-30 (habitat-book web-renderer):
   the interpreted libs/std definition cost ~O(n) interpreter steps per
   input byte -- one 3KB prose paragraph blew the 100k-step page budget --
   so std no longer defines it and the image registers this prim
   (string-split moves to the reserved catalog). Empty delimiter returns
   the whole string (the retired std version looped forever there). *)
let prim_string_split args =
  match args with
  | [ s_v; d_v ] ->
      let s = coerce_str ~ctx:"string-split" s_v in
      let d = coerce_str ~ctx:"string-split" d_v in
      let slen = String.length s in
      let dlen = String.length d in
      if slen = 0 || dlen = 0 then VList [ VStr s ]
      else begin
        (* search for delim like string-contains? (byte compare) *)
        let find_from i =
          let rec go j =
            if j + dlen > slen then None
            else if String.sub s j dlen = d then Some j
            else go (j + 1)
          in
          go i
        in
        let rec loop acc i =
          match find_from i with
          | None -> List.rev (VStr (String.sub s i (slen - i)) :: acc)
          | Some p -> loop (VStr (String.sub s i (p - i)) :: acc) (p + dlen)
        in
        VList (loop [] 0)
      end
  | _ -> raise (Interp_error (User_error ("string-split needs (s delim)",
     "bad-args")))

(* ---- url-encode / url-decode (percent-encoding) ---- *)

let prim_url_encode args =
  let encode s =
    let buf = Buffer.create (String.length s) in
    String.iter
      (fun c ->
        match c with
        | 'A'..'Z' | 'a'..'z' | '0'..'9' | '-' | '.' | '_' | '~' ->
            Buffer.add_char buf c
        | _ ->
            Buffer.add_string buf
              (Printf.sprintf "%%%02X" (Char.code c)))
      s;
    Buffer.contents buf
```

```
  in
  match args with
  | [ v ] -> VStr (encode (coerce_str ~ctx:"url-encode" v))
  | _ -> raise (Interp_error (User_error ("url-encode needs 1 string",
     "bad-args")))

let prim_url_decode args =
  let decode s =
    let len = String.length s in
    let buf = Buffer.create len in
    let rec loop i =
      if i >= len then ()
      else if s.[i] = '%' && i + 2 < len then
        (try
           let n = int_of_string ("0x" ^ String.sub s (i + 1) 2) in
           Buffer.add_char buf (Char.chr n);
           loop (i + 3)
         with _ -> Buffer.add_char buf s.[i]; loop (i + 1))
      else if s.[i] = '+' then begin Buffer.add_char buf ' '; loop (i + 1)
     end
      else begin Buffer.add_char buf s.[i]; loop (i + 1) end
    in
    loop 0;
    Buffer.contents buf
  in
  match args with
  | [ v ] -> VStr (decode (coerce_str ~ctx:"url-decode" v))
  | _ -> raise (Interp_error (User_error ("url-decode needs 1 string",
     "bad-args")))

(* ---- html-decode (entity decode) ----

   Handles &amp; &lt; &gt; &quot; &apos; &#NN; &#xHH; and common named
     entities.
   Best-effort; unknown entities are left as-is. *)

let named_entities =
  [ "amp", "&"; "lt", "<"; "gt", ">"; "quot", "\""; "apos", "'";
    "nbsp", "\240"; "copy", "\169"; "reg", "\174";
    "mdash", "\226\128\148"; "ndash", "\226\128\147";
    "hellip", "\226\128\166"; "rsquo", "\226\128\153"; "lsquo",
     "\226\128\152";
    "ldquo", "\226\128\156"; "rdquo", "\226\128\157"; "euro",
     "\226\130\172";
    "trade", "\226\132\162"; "deg", "\176"; "plusmn", "\191"; (* actual ?
     U+00B1 *)
    "times", "\195\151"; "divide", "\195\183"; "bull", "\226\128\162";
    "middot", "\194\183"; "laquo", "\194\171"; "raquo", "\194\187" ]

let prim_html_decode args =
  let decode s =
    let len = String.length s in
    let buf = Buffer.create len in
    let lookup_named name =
      try Some (List.assoc name named_entities) with Not_found -> None
    in
    let rec loop i =
      if i >= len then ()
      else if s.[i] <> '&' then begin Buffer.add_char buf s.[i]; loop (i +
     1) end
      else begin
        (* find ';' within a reasonable window *)
        let rec find_semi j =
          if j >= len || j > i + 12 then None
          else if s.[j] = ';' then Some j else find_semi (j + 1)
        in
        match find_semi (i + 1) with
        | None -> Buffer.add_char buf '&'; loop (i + 1)
        | Some e ->
            let body = String.sub s (i + 1) (e - i - 1) in
            let decoded =
              if String.length body >= 1 && body.[0] = '#' then
                let num = String.sub body 1 (String.length body - 1) in
```

```
                let n =
                  if String.length num >= 1 && (num.[0] = 'x' || num.[0] = 'X')
                  then
                    (try Some (int_of_string ("0x" ^ String.sub num 1 (String.length num - 1)))
                     with _ -> None)
                  else (try Some (int_of_string num) with _ -> None)
                in
                (match n with
                 | Some n when n >= 0 && n < 0x110000 ->
                     (* encode as UTF-8 *)
                     let rec utf8 n =
                       if n < 0x80 then String.make 1 (Char.chr n)
                       else if n < 0x800 then
                         String.make 1 (Char.chr (0xC0 lor (n lsr 6)))
                         ^ String.make 1 (Char.chr (0x80 lor (n land 0x3F)))
                       else if n < 0x10000 then
                         String.make 1 (Char.chr (0xE0 lor (n lsr 12)))
                         ^ String.make 1 (Char.chr (0x80 lor ((n lsr 6) land 0x3F)))
                         ^ String.make 1 (Char.chr (0x80 lor (n land 0x3F)))
                       else
                         String.make 1 (Char.chr (0xF0 lor (n lsr 18)))
                         ^ String.make 1 (Char.chr (0x80 lor ((n lsr 12) land 0x3F)))
                         ^ String.make 1 (Char.chr (0x80 lor ((n lsr 6) land 0x3F)))
                         ^ String.make 1 (Char.chr (0x80 lor (n land 0x3F)))
                     in
                     Some (utf8 n)
                 | _ -> None)
             else lookup_named body
           in
           (match decoded with
            | Some txt -> Buffer.add_string buf txt
            | None -> Buffer.add_string buf ("&" ^ body ^ ";"));
           loop (e + 1)
     end
   in
   loop 0;
   Buffer.contents buf
 in
 match args with
 | [ v ] -> VStr (decode (coerce_str ~ctx:"html-decode" v))
 | _ -> raise (Interp_error (User_error ("html-decode needs 1 string", "bad-args")))

(* ---- parse-date -> UTC epoch int (VInt) ----

   Accepts RFC-822, ISO-8601, and bare YYYY-MM-DD forms. Returns seconds
   since the Unix epoch (UTC) as a VInt, or VInt 0 on failure so Lisp can
   fall back to document order. UTC-anchored via a days-from-civil formula
   (no timezone drift). *)

let days_from_civil ~year ~month ~day =
  (* Howard Hinnant's algorithm; works for proleptic Gregorian, year >= 0. *)
  let y = if month <= 2 then year - 1 else year in
  let era = if y >= 0 then y / 400 else (y - 399) / 400 in
  let yoe = y - era * 400 in
  let m = month in
  let doy = (153 * (if m > 2 then m - 3 else m + 9) + 2) / 5 + day - 1 in
  let doe = yoe * 365 + yoe / 4 - yoe / 100 + doy in
  era * 146097 + doe - 719468

let parse_int_sub s off len =
  try Some (int_of_string (String.sub s off len)) with _ -> None

let prim_parse_date args =
  let parse s =
    let s = String.trim s in
    let len = String.length s in
```

```
    (* strip optional leading day-of-week: "Tue, " or "Tue " *)
    let s =
      match String.index_opt s ',' with
      | Some i -> String.trim (String.sub s (i + 1) (len - i - 1))
      | None -> s
    in
    let len = String.length s in
    let months =
      [| "Jan", 1; "Feb", 2; "Mar", 3; "Apr", 4; "May", 5; "Jun", 6;
         "Jul", 7; "Aug", 8; "Sep", 9; "Oct", 10; "Nov", 11; "Dec", 12 |]
    in
    let month_of_name nm =
      let rec i = ref 0 in
      let found = ref None in
      while !i < 12 && !found = None do
        let (n, m) = months.(!i) in
        if String.equal (String.lowercase_ascii n) (String.lowercase_ascii nm)
        then found := Some m;
        incr i
      done;
      !found
    in
    (* ISO-8601: YYYY-MM-DDTHH:MM:SS[.f][Z|+HH:MM] or YYYY-MM-DD HH:MM:SS *)
    let try_iso () =
      if len >= 10
         && String.length s >= 10
         && String.sub s 0 4 |> String.for_all (fun c -> c >= '0' && c <= '9')
         && (s.[4] = '-' || s.[4] = '/')
      then
        let y = parse_int_sub s 0 4 in
        let mo = parse_int_sub s 5 2 in
        let d = parse_int_sub s 8 2 in
        let h, mi, sec =
          if len >= 19 && (s.[10] = 'T' || s.[10] = 't' || s.[10] = ' ')
          then (parse_int_sub s 11 2, parse_int_sub s 14 2, parse_int_sub s 17 2)
          else (Some 0, Some 0, Some 0)
        in
        match y, mo, d, h, mi, sec with
        | Some y, Some mo, Some d, Some h, Some mi, Some sec ->
            let days = days_from_civil ~year:y ~month:mo ~day:d in
            Some (Int64.of_int (days * 86400 + h * 3600 + mi * 60 + sec))
        | _ -> None
      else None
    in
    (* RFC-822: "28 Jul 2026 02:56:21 GMT" (day-of-week already stripped) *)
    let try_rfc822 () =
      let tokens = String.split_on_char ' ' s in
      let strip_tz = function
        | [ d; mo; y; t; _tz ] -> [ d; mo; y; t ]
        | other -> other
      in
      match strip_tz tokens with
      | [ d_str; mon_str; y_str; time_str ] ->
          (match parse_int_sub d_str 0 (String.length d_str),
                 month_of_name mon_str,
                 parse_int_sub y_str 0 (String.length y_str) with
           | Some d, Some mo, Some y ->
               let parts = String.split_on_char ':' time_str in
               (match parts with
                | [ hh; mm; ss ] ->
                    (match parse_int_sub hh 0 2, parse_int_sub mm 0 2,
                           parse_int_sub ss 0 2 with
                     | Some h, Some mi, Some se ->
                         let days = days_from_civil ~year:y ~month:mo ~day:d in
                         Some (Int64.of_int (days * 86400 + h * 3600 + mi * 60 + se))
                     | _ -> None)
                | [ hh; mm ] ->
```

```
                    (match parse_int_sub hh 0 2, parse_int_sub mm 0 2 with
                     | Some h, Some mi ->
                         let days = days_from_civil ~year:y ~month:mo ~day:d in
                         Some (Int64.of_int (days * 86400 + h * 3600 + mi * 60))
                     | _ -> None)
                | _ -> None)
           | _ -> None)
      | _ -> None
    in
    (* bare YYYY-MM-DD *)
    let try_bare () =
      if len >= 10 && (s.[4] = '-' || s.[4] = '/') then
        match parse_int_sub s 0 4, parse_int_sub s 5 2, parse_int_sub s 8 2 with
        | Some y, Some mo, Some d ->
            let days = days_from_civil ~year:y ~month:mo ~day:d in
            Some (Int64.of_int (days * 86400))
        | _ -> None
      else None
    in
    try_iso () |> function Some e -> Some e | None ->
    try_rfc822 () |> function Some e -> Some e | None ->
    try_bare ()
  in
  match args with
  | [ v ] ->
      (match parse (coerce_str ~ctx:"parse-date" v) with
       | Some e -> VInt e
       | None -> VInt 0L)
  | _ -> raise (Interp_error (User_error ("parse-date needs 1 string", "bad-args")))

(* ---- xml-parse (markup) ----

   (xml-parse text) -> Lisp tree. Node shape (per note):
     ("tag" (("k" . "v") ...) child1 child2 ...)
   text node = bare VStr (already entity-decoded). comments/PI dropped.
   Namespaces: raw prefixed names preserved verbatim. Tag names are strings
   so string-eq/car walks are safe. On hard failure returns
   ("error" message). *)

(* Coerce a Markup.name (namespace_uri, local_name) to a single string.
   For lenient RSS/Atom with no namespace declaration, the raw prefix is
   preserved in the local part, so snd alone is correct. When a namespace URI
   is present we still return the local name (Lisp-side normalizes prefixes). *)
let xml_name_to_string (_ns : string) (local : string) : string = local

let rec xml_tree_to_value = function
  | `Text s -> VStr s
  | `Elem (name, attrs, children) ->
      let tag = xml_name_to_string (fst name) (snd name) in
      let attr_pairs =
        List.map
          (fun (k, v) ->
            VPair (VStr (xml_name_to_string (fst k) (snd k)), VStr v))
          attrs
      in
      VList (VStr tag :: VList attr_pairs :: List.map xml_tree_to_value children)

let prim_xml_parse args =
  let parse s =
    let open Markup in
    try
      let parsed =
        s |> string |> parse_xml |> signals
        |> tree
            ~text:(fun ss -> `Text (String.concat "" ss))
            ~element:(fun name attrs children ->
```

```
              `Elem (name, attrs, children))
      in
      match parsed with
      | None -> VList [ VStr "error"; VStr "empty document" ]
      | Some t -> xml_tree_to_value t
    with exn ->
      VList [ VStr "error"; VStr (Printexc.to_string exn) ]
  in
  match args with
  | [ v ] -> parse (coerce_str ~ctx:"xml-parse" v)
  | _ -> raise (Interp_error (User_error ("xml-parse needs 1 string",
     "bad-args")))

(* ---- http response header parsing (helper for http-get prim) ----

   curl -D writes headers like:
     HTTP/2 200\r\n  (or "HTTP/1.1 200 OK")
     content-type: text/html\r\n
   Returns a JSON object string with lowercased keys. *)
(** (random-uuid) -> VStr canonical uuid. Uses stdlib Random (seeded at
     process start).
    Distinct from (random)/(random-seed) -- those are the numeric PRNG
     surface. *)
let prim_random_uuid (_args : value list) : value =
  let b = Bytes.create 16 in
  for i = 0 to 15 do Bytes.set b i (Char.chr (Random.int 256)) done;
  (* RFC 4122 version 4 + variant 10xx *)
  Bytes.set b 6 (Char.chr ((Char.code (Bytes.get b 6) land 0x0f) lor 0x40));
  Bytes.set b 8 (Char.chr ((Char.code (Bytes.get b 8) land 0x3f) lor 0x80));
  let hex_byte i = Printf.sprintf "%02x" (Char.code (Bytes.get b i)) in
  VStr
    (String.concat ""
       [ hex_byte 0; hex_byte 1; hex_byte 2; hex_byte 3; "-"
       ; hex_byte 4; hex_byte 5; "-"
       ; hex_byte 6; hex_byte 7; "-"
       ; hex_byte 8; hex_byte 9; "-"
       ; hex_byte 10; hex_byte 11; hex_byte 12; hex_byte 13; hex_byte 14;
         hex_byte 15
       ])

let http_parse_headers (blob : string) : string =
  let lines = String.split_on_char '\n' blob in
  let acc = ref [] in
  List.iter
    (fun line ->
      let line = String.trim line in
      if line = "" then ()
      else
        (match String.index_opt line ':' with
         | Some i ->
             let k = String.lowercase_ascii (String.trim (String.sub line 0
    i)) in
             let v = String.trim (String.sub line (i + 1) (String.length
    line - i - 1)) in
             acc := (k, v) :: !acc
         | None ->
             (* status line; ignore -- we capture status separately *)
             ()))
    lines;
  Yojson.Safe.to_string (`Assoc (List.map (fun (k, v) -> (k, `String v))
     (List.rev !acc)))


(* Shared ERROR codec (borge/remote-eval.borg substrate).
   Factored out of image/lib/http_pages.ml api_eval so that the image
   and a proxy rim link the SAME codec: [error_to_json] serializes any
   [exn] to the /api/eval error JSON shape; [error_of_json] is the
   inverse -- it re-raises the SAME [exn] the local rim would, so a
   remote client observes identical exceptions to an in-process eval.
   The shape is {ok:false, error, kind, name?, line?, col?} plus two
   additive round-trip fields ([raw_error], and [name] on Prim_error)
   that let the inverse reconstruct Prim_error / Reader_error exactly.
   NB: [json_to_value] maps JSON strings to VStr, so a deserialized
```

```
   value is never a VSymbol on its own -- symbols are a read-side
   concern. *)
let error_to_json (exn : exn) : Yojson.Safe.t =
  match exn with
  | Pricklypear_lisp.Ast.Interp_error e ->
      let base =
        ref
          [ ("ok", `Bool false)
          ; ("error", `String (Pricklypear_lisp.Ast.pp_interp_error e))
          ; ("kind", `String (Pricklypear_lisp.Ast.interp_error_kind e)) ]
      in
      (match e with
       | Unbound_var name -> base := !base @ [ ("name", `String name) ]
       | Prim_error (name, rmsg, _) ->
           (* carry name + raw message so a proxy can re-raise the SAME exn
      *)
           base := !base @ [ ("name", `String name); ("raw_error", `String
      rmsg) ]
       | _ -> ());
      (match Pricklypear_lisp.Ast.interp_error_loc e with
       | Some (line, col) ->
           base := !base @ [ ("line", `Int line); ("col", `Int col) ]
       | None -> ());
      `Assoc !base
  | Pricklypear_lisp.Reader.Reader_error (m, line, col) ->
      `Assoc
        [ ("ok", `Bool false)
        ; ("error", `String (Printf.sprintf "%s (%d:%d)" m line col))
        ; ("kind", `String "parse_error")
        ; ("raw_error", `String m)
        ; ("line", `Int line)
        ; ("col", `Int col) ]
  | _ ->
      `Assoc
        [ ("ok", `Bool false)
        ; ("error", `String (Printexc.to_string exn)) ]

(** Inverse of [error_to_json]. Returns an [exn] that, when raised by a
    proxy rim, is the same exception the local rim would raise. *)
let error_of_json (j : Yojson.Safe.t) : exn =
  let field l k = List.assoc_opt k l in
  let str l k =
    match field l k with Some (`String s) -> Some s | _ -> None
  in
  let int l k =
    match field l k with
    | Some (`Int n) -> Some n
    | Some (`Intlit s) -> (try Some (int_of_string s) with _ -> None)
    | _ -> None
  in
  match j with
  | `Assoc fields ->
      let kind = str fields "kind" in
      (match kind with
       | Some k when k <> "parse_error" ->
           let name = str fields "name" in
           let raw = str fields "raw_error" in
           let err = str fields "error" in
           let interp e = Pricklypear_lisp.Ast.Interp_error e in
           (match name, raw with
            | Some n, Some r ->
                interp (Prim_error (n, r, k))
            | Some n, None ->
                (match k with
                 | "unbound_var" -> interp (Unbound_var n)
                 | _ ->
                     (* Prim_error without raw_error (e.g. a legacy peer);
                        degrade to User_error carrying the name+kind so the
                        message still names the prim. *)
                     interp (User_error ("prim " ^ n ^ ": " ^ n, k)))
            | None, _ ->
                (match k with
                 | "budget_exceeded" -> interp Budget_exceeded
```

```
                | _ ->
                    interp
                      (User_error
                         (Option.value err ~default:"", k))))
      | _ ->
          (* parse_error: re-raise as Reader_error. A Parse_error also
             serializes to this shape; reconstruct as Reader_error (same
             parse failure, same wire). *)
          let m =
            match str fields "raw_error" with
            | Some m -> m
            | None -> Option.value (str fields "error") ~default:""
          in
          let line = Option.value (int fields "line") ~default:0 in
          let col = Option.value (int fields "col") ~default:0 in
          Pricklypear_lisp.Reader.Reader_error (m, line, col))
  | _ ->
      Pricklypear_lisp.Ast.Interp_error
        (Pricklypear_lisp.Ast.User_error
           (Yojson.Safe.to_string j, "unknown"))
```

282 pure/prims_soup.ml

```
(** Pricklypear -- shared soup/ tree prims (the hypermedia-harness kernel).

    DB-free lambdasoup-based HTML tree prims lifted out of the
    image-only [image/lib/prims_hyper.ml] into this shared pure-prims
    layer so BOTH hosts register the SAME tree prims: the image
    (Postgres corpus) and the agent (local-first lens corpus). Only
    route/dispatch + route/lookup remain host-specific (they touch the
    host's store / routes / request context -- see hypermedia-harness
    request-runner). lambdasoup + markup are deps of this layer.

    Spec: borge/hypermedia-harness.borg (kernel-prims + supported-subset).

    Page/node values are opaque VStr handles into module-global tables
    (same handle-as-id pattern as net_streams). Low-level soup/* prims
    mutate in place; the Lisp layer [libs/hyper] deep-copies
    (soup/page-copy) before each deriving interaction, so pages are
    immutable at the hyper/* surface (free history + diff). *)
open Pricklypear_lisp.Ast
module S = Soup

type soup_node = S.soup S.node
type element_node = S.element S.node
type general_node = S.general S.node
type element_nodes = S.element S.nodes

(* The registration table type is the structural prim table both hosts
   use: (string, value list -> value) Hashtbl.t. The image's
   [Primitives.t] and the agent's [Eval.prim_table] are both aliases of
   this exact type, so a single register_pure_prims serves both. *)
type prim_table = (string, value list -> value) Hashtbl.t

let register (tbl : prim_table) (name : string) (fn : value list -> value)
    : unit =
  Hashtbl.replace tbl name fn

(* ---------------------------------------------------------------- *
   Handle tables: VStr id <-> lambdasoup node. *)

type page = { soup : soup_node }
type node = { elt : element_node; pid : string }

let pages : (string, page) Hashtbl.t = Hashtbl.create 16
let nodes : (string, node) Hashtbl.t = Hashtbl.create 64

let counter = ref 0
let fresh prefix =
  incr counter;
  Printf.sprintf "%s-%d" prefix !counter

let store_page (s : soup_node) : string =
  let id = fresh "page" in
  Hashtbl.replace pages id { soup = s };
  id

let store_node (e : element_node) (pid : string) : string =
  let id = fresh "node" in
  Hashtbl.replace nodes id { elt = e; pid };
  id

let page_of (v : value) : soup_node option =
  match v with
  | VStr id -> (match Hashtbl.find_opt pages id with
                | Some p -> Some p.soup | None -> None)
  | _ -> None

let node_of (v : value) : element_node option =
  match v with
  | VStr id -> (match Hashtbl.find_opt nodes id with
                | Some n -> Some n.elt | None -> None)
  | _ -> None

```

```
let page_id_of (v : value) : string option =
  match v with VStr s -> Some s | _ -> None

(* ---------------------------------------------------------------- *
   Deep-copy a page (serialize + reparse) so a deriving operation
   never mutates its source -- the hyper/* immutability invariant. *)
let copy_soup (s : soup_node) : soup_node =
  S.parse (S.to_string s)

(* Parse an HTML fragment into a list of top-level general nodes. *)
let fragment_nodes (html : string) : general_node list =
  S.to_list (S.children (S.parse html))

(* Apply one htmx swap strategy to [target], grafting [fragment]'s
   top-level nodes. Matches the 8 htmx 4 strategies. *)
let apply_swap ~target (style : string) (fragment : string) : unit =
  let new_nodes = fragment_nodes fragment in
  match String.lowercase_ascii style with
  | "innerhtml" ->
    S.clear target;
    List.iter (S.append_child target) new_nodes
  | "outerhtml" ->
    (* Insert new nodes before target, then unlink target itself. *)
    List.iter (fun n -> S.insert_before target n) new_nodes;
    S.delete target
  | "beforebegin" ->
    List.iter (fun n -> S.insert_before target n) new_nodes
  | "afterend" ->
    List.iter (fun n -> S.insert_after target n) new_nodes
  | "beforeend" ->
    List.iter (S.append_child target) new_nodes
  | "afterbegin" ->
    (* prepend_child reverses order; reverse first to preserve it. *)
    List.iter (S.prepend_child target) (List.rev new_nodes)
  | "delete" -> S.delete target
  | "none" -> ()
  | _ -> ()  (* unknown style: no-op *)

(* ---------------------------------------------------------------- *
   Handler value -> HTML string (mirrors http_core.value_to_html_string:
   raw-HTML-tagged VStrs unwrap, plain VStrs are text, others pp'd).

   Host-agnostic helper shared by both the image's route/dispatch and
   the agent's route/dispatch. *)
let render_handler_value (v : value) : string =
  match Prims_pure.is_raw_html v with
  | Some s -> s
  | None ->
    (match v with
     | VStr s -> s
     | _ -> Pricklypear_lisp.Reader.pp_value v)

(* Alist-of-pairs value -> (string,string) list. Accepts VList of
   Pair(VStr k, VStr v). Used for params / headers in route/dispatch.
   Host-agnostic helper shared by both hosts' route/dispatch. *)
let rec assoc_of (v : value) : (string * string) list =
  match v with
  | VList items -> List.filter_map pair_of items
  | VPair (head, tail) ->
    (match pair_of head with
     | Some kv -> kv :: assoc_of tail
     | None -> assoc_of tail)
  | _ -> []

and pair_of (p : value) : (string * string) option =
  match p with
  | VPair (VStr k, VStr x) -> Some (k, x)
  | _ -> None

(* ---------------------------------------------------------------- *
   Pure prims: soup/parse, soup/select, soup/text, soup/attr,
   soup/serialize, soup/swap!, soup/create, soup/replace, soup/diff,
   soup/page-copy, soup/closest-attr, soup/parent, soup/select-one,
```

```
   soup/has-attr, soup/apply-parts!. *)
let err s = VStr ("hyper: " ^ s)

let register_pure_prims (tbl : prim_table) : unit =
  register tbl "soup/parse"
    (fun args ->
       match args with
       | [VStr s] -> VStr (store_page (S.parse s))
       | _ -> err "html/parse needs one string");

  register tbl "soup/page-copy"
    (fun args ->
       match args with
       | [pid] ->
         (match page_of pid with
          | Some s -> VStr (store_page (copy_soup s))
          | None -> err "html/page-copy: not a page")
       | _ -> err "html/page-copy needs one page");

  register tbl "soup/select"
    (fun args ->
       match args with
       | [page_v; VStr sel] ->
         (match page_of page_v with
          | None -> err "html/select: not a page"
          | Some s ->
            let matches = S.to_list (S.select sel s) in
            let pid = match page_id_of page_v with Some p -> p | None ->
    "?" in
            VList (List.map (fun e -> VStr (store_node e pid)) matches))
       | _ -> err "html/select needs page + selector");

  register tbl "soup/select-one"
    (fun args ->
       match args with
       | [page_v; VStr sel] ->
         (match page_of page_v with
          | None -> err "html/select-one: not a page"
          | Some s ->
            match S.select_one sel s with
            | None -> VNil
            | Some e ->
              let pid = match page_id_of page_v with Some p -> p | None ->
    "?" in
              VStr (store_node e pid))
       | _ -> err "html/select-one needs page + selector");

  register tbl "soup/serialize"
    (fun args ->
       match args with
       | [v] ->
         (match page_of v with
          | Some s -> VStr (S.to_string s)
          | None ->
            match node_of v with
            | Some e -> VStr (S.to_string e)
            | None -> err "html/serialize: not a page or node")
       | _ -> err "html/serialize needs one handle");

  register tbl "soup/text"
    (fun args ->
       match args with
       | [v] ->
         (match page_of v with
          | Some s -> VStr (String.concat "" (S.texts s))
          | None ->
            match node_of v with
            | Some e -> VStr (String.concat "" (S.texts e))
            | None -> err "html/text: not a page or node")
       | _ -> err "html/text needs one handle");

  register tbl "soup/attr"
    (fun args ->
```

```
      match args with
      | [node_v; VStr name] ->
        (match node_of node_v with
         | None -> err "html/attr: not a node"
         | Some e ->
           match S.attribute name e with
           | Some s -> VStr s
           | None -> VNil)
      | _ -> err "html/attr needs node + name");

  register tbl "soup/has-attr"
    (fun args ->
      match args with
      | [node_v; VStr name] ->
        (match node_of node_v with
         | None -> err "html/has-attr: not a node"
         | Some e ->
           if S.has_attribute name e then VStr "true" else VNil)
      | _ -> err "html/has-attr needs node + name");

  register tbl "soup/swap!"
    (fun args ->
       (* (html/swap! node style fragment) -- mutate in place, return nil.
    *)
      match args with
      | [node_v; VStr style; VStr fragment] ->
        (match node_of node_v with
         | None -> err "html/swap!: not a node"
         | Some e -> apply_swap ~target:e style fragment; VNil)
      | _ -> err "html/swap! needs node + style + fragment");

  register tbl "soup/create"
    (fun args ->
      match args with
      | [VStr tag] ->
        let e = S.create_element tag in
        let id = store_node e "orphan" in
        VStr id
      | _ -> err "html/create needs tag name");

  register tbl "soup/replace"
    (fun args ->
       (* (html/replace node fragment) -- replace target with fragment's
          top-level nodes, preserving order. *)
      match args with
      | [node_v; VStr fragment] ->
        (match node_of node_v with
         | None -> err "html/replace: not a node"
         | Some e ->
           let news = fragment_nodes fragment in
           List.iter (fun n -> S.insert_before e n) news;
           S.delete e;
           VNil)
      | _ -> err "html/replace needs node + fragment");

  register tbl "soup/closest-attr"
    (fun args ->
       (* (soup/closest-attr node name) -> walk from node up through
    ancestors,
          return the value of `name` from the nearest element that HAS it.
          Mirrors htmx inherited-target resolution (compat implicit
          inheritance). Returns VNil if none found. *)
      match args with
      | [node_v; VStr name] ->
        (match node_of node_v with
         | None -> err "soup/closest-attr: not a node"
         | Some e ->
           (* check the element itself, then ancestors *)
           let rec find n =
             if Soup.has_attribute name n then Soup.attribute name n
             else
               match Soup.parent n with
               | None -> None
```

```
              | Some p -> find p
          in
          (match find e with
           | Some v -> VStr v
           | None -> VNil))
      | _ -> err "soup/closest-attr needs node + name");

  register tbl "soup/parent"
    (fun args ->
       match args with
       | [node_v] ->
         (match node_of node_v with
          | None -> err "soup/parent: not a node"
          | Some e ->
            match Soup.parent e with
            | None -> VNil
            | Some p -> VStr (store_node p "?"))
       | _ -> err "soup/parent needs one node");

  register tbl "soup/diff"
    (fun args ->
       (* Walk both pages' element descendants by id attribute. Report
          added ids, removed ids, and text changes within same-id
          elements (the v0 text-change detection). *)
       match args with
       | [a; b] ->
         (match page_of a, page_of b with
          | Some sa, Some sb ->
            if S.equal_modulo_whitespace sa sb then VStr "no changes"
            else
              let ea = S.to_list (S.select "*" sa) in
              let eb = S.to_list (S.select "*" sb) in
              let idset lst =
                List.filter_map
                  (fun e -> S.attribute "id" e) lst
              in
              let ia, ib = idset ea, idset eb in
              let added = List.filter (fun x -> not (List.mem x ia)) ib in
              let removed = List.filter (fun x -> not (List.mem x ib)) ia in
              (* Text changes: for ids present in both, compare text
    content. *)
              let text_of_id lst id =
                List.find_map
                  (fun e ->
                    if S.attribute "id" e = Some id
                    then Some (String.concat "" (S.texts e))
                    else None)
                  lst
              in
              let common = List.filter (fun x -> List.mem x ib) ia in
              let text_changed =
                List.filter_map
                  (fun id ->
                    match text_of_id ea id, text_of_id eb id with
                    | Some ta, Some tb when ta <> tb ->
                      Some (Printf.sprintf "#%s: \"%s\" -> \"%s\"" id ta tb)
                    | _ -> None)
                  common
              in
              let parts =
                (if added = [] then []
                 else [Printf.sprintf "added: %s" (String.concat ","
    added)])
                @ (if removed = [] then []
                   else [Printf.sprintf "removed: %s" (String.concat ","
    removed)])
                @ text_changed
              in
              VStr
                (if parts = [] then "changed (structural)"
                 else String.concat "; " parts)
          | _ -> err "html/diff needs two pages")
       | _ -> err "html/diff needs two pages");
```

```
357
358   (* soup/apply-parts! -- apply an ordered multipart part list onto the
359      page in place. parts is a VList of part alists, each
360      (("html" . frag) ("target" . sel) ("swap" . strat)); each part is
361      located (soup/select target on the page) then swapped
362      (soup/swap! strat frag) in ORDER. target/swap absent -> caller's
363      element + "innerHTML", matching hyper/do-swap defaults.
364
365      Matches the multipart/mixed wire shape http_agent.mp_write_part
366      emits (per-part HX-Target/HX-Swap); see hypermedia-harness
367      supported-subset hx-multipart. *)
368   register tbl "soup/apply-parts!"
369     (fun args ->
370       match args with
371       | [page_v; parts_v] ->
372         (match page_of page_v with
373          | None -> err "soup/apply-parts!: not a page"
374          | Some s ->
375            let part_list =
376              match parts_v with
377              | VList xs -> xs
378              | VNil -> []
379              | _ -> []
380            in
381            List.iter
382              (fun part ->
383                let alist = assoc_of part in
384                let frag =
385                  match List.assoc_opt "html" alist with
386                  | Some h -> h
387                  | None -> ""
388                in
389                let target_sel =
390                  List.assoc_opt "target" alist
391                in
392                let swap_str =
393                  match List.assoc_opt "swap" alist with
394                  | Some sw -> sw
395                  | None -> "innerHTML"
396                in
397                (match target_sel with
398                 | Some sel ->
399                   (* locate the target element on the page *)
400                   (match S.select_one sel s with
401                    | None -> ()  (* target not found: skip *)
402                    | Some e -> apply_swap ~target:e swap_str frag)
403                 | None -> ()))
404              part_list;
405            VNil)
406       | _ -> err "soup/apply-parts! needs page + parts")
```

283 remote/bin/main.ml

```
(** pp-remote -- a Nopales client process (borge/remote-eval.borg client).

    Embeds the pure lisp interpreter, lazily materializes function
    definitions from a remote branch, and proxies Prim_call effects to
    the image over HTTP. Pure compute stays local; everything else
    crosses the wire to POST /api/prim. The store stays the single
    source of truth -- quit the client, lose nothing but REPL state.

      export PP_URL=http://127.0.0.1:18080
      export PP_USER=admin PP_PASS=admin
      pp-remote '(+ 1 2)'          # one expr, print, exit
      pp-remote                    # stdin REPL

    Flags:
      --image URL    base URL of the image (default: PP_URL / 127.0.0.1:18080)
      --user NAME    basic-auth user (default: PP_USER / admin)
      --pass PASS    basic-auth pass (default: PP_PASS / admin)
      -e EXPR        eval one expr and exit (repeatable)
      -p EXPR        alias of -e (pipe mode)
      --model NAME   provider model (default: AGENT_MODEL / GLM-4.7-Flash)
      --base-url URL provider base URL (default: AGENT_BASE_URL / synthetic.new)
      --mock-script FILE  canned provider replies (test/demo of /chat)

    The LLM provider runs LOCALLY (borge/remote-eval.borg agent-loop):
    the API key is a value read from AGENT_API_KEY (same env the agent
    reads); only the data plane (DB prims inside tool bodies) crosses
    to the image. *)

open Pricklypear_lisp

let getenv name ~default =
  match Sys.getenv_opt name with
  | Some s when s <> "" -> s
  | Some _ | None -> default

(* Auto-load ./.env if present (KEY=VALUE or `export KEY=VALUE` lines),
   same as the agent binary -- AGENT_API_KEY etc. land without sourcing.
   Failures are silent (file optional). *)
let load_dotenv () : unit =
  let path = ".env" in
  match open_in path with
  | exception _ -> ()
  | ic ->
    (try
       while true do
         let line = String.trim (input_line ic) in
         if line = "" || line.[0] = '#' then ()
         else begin
           let line =
             if String.length line > 7 && String.sub line 0 7 = "export "
             then String.trim (String.sub line 7 (String.length line - 7))
             else line
           in
           match String.index_opt line '=' with
           | Some i ->
               let k = String.trim (String.sub line 0 i) in
               let v_raw = String.trim (String.sub line (i + 1) (String.length line - i - 1)) in
               let v =
                 let n = String.length v_raw in
                 if n >= 2
                    && (v_raw.[0] = '"' && v_raw.[n - 1] = '"'
                        || v_raw.[0] = '\'' && v_raw.[n - 1] = '\'')
                 then String.sub v_raw 1 (n - 2)
                 else v_raw
               in
               Unix.putenv k v
           | None -> ()
         end
```

```
      done
    with End_of_file -> ());
   close_in ic

(* Mock provider: reads a JSON script of turns (same shape as the
   agent's --mock-script), consumed in order. Mirrors the real
   provider's streaming contract (fires the reply's text as an
   on_delta) so the loop path is identical, canned or live. *)
let parse_mock_script script_path : Provider.reply list =
  let open Yojson.Safe.Util in
  let j = Yojson.Safe.from_file script_path in
  j |> to_list
  |> List.map (fun t ->
         let text = t |> member "text" |> to_string_option |> Option.value
    ~default:"" in
         let calls =
           match t |> member "calls" with
           | `List cs ->
               List.map
                 (fun c ->
                   {
                     Provider.id = c |> member "id" |> to_string;
                     name = c |> member "name" |> to_string;
                     input = c |> member "input";
                   })
                 cs
           | _ -> []
         in
         {
           Provider.text;
           calls;
           stop_reason = "stop";
           raw = `Assoc [ ("role", `String "assistant"); ("content",
    `String text) ];
         })

let mock_complete ~(env : Eio_unix.Stdenv.base) ~sw ~cfg ~messages ~tools
    ~on_delta script_cell
    : (Provider.reply, string) Result.t =
  ignore env;
  ignore sw;
  ignore cfg;
  ignore tools;
  match !script_cell with
  | [] ->
      on_delta (Provider.Text "");
      on_delta Provider.Done;
      Ok { Provider.text = ""; calls = []; stop_reason = "end_turn"; raw =
    `Null }
  | r :: rest ->
      script_cell := rest;
      ignore messages;
      if r.Provider.text <> "" then on_delta (Provider.Text
    r.Provider.text);
      on_delta Provider.Done;
      Ok r

(* One-shot harness identity for the system prompt (pp-remote variant
   of the agent's harness_context): told up front so the model does not
   probe the OS or leak secrets discovering them. *)
let run_captured (cmd : string) : string =
  try
    let ic = Unix.open_process_in cmd in
    let out = String.trim (In_channel.input_all ic) in
    match Unix.close_process_in ic with
    | Unix.WEXITED 0 -> out
    | _ -> ""
  with _ -> ""

let harness_context ~(model : string) ~(base_url : string) ~(image : string)
    ~(user : string) : string =
  let host = try Unix.gethostname () with _ -> "?" in
  let uname = run_captured "uname -srm" in
```

```
  let cwd = Sys.getcwd () in
  let tm = Unix.localtime (Unix.gettimeofday ()) in
  let date =
    Printf.sprintf "%04d-%02d-%02d %02d:%02d"
      (tm.Unix.tm_year + 1900) (tm.Unix.tm_mon + 1) tm.Unix.tm_mday
      tm.Unix.tm_hour tm.Unix.tm_min
  in
  Printf.sprintf
    "You are the pp-remote chat agent: the LLM turn runs LOCALLY on this \n\
     machine while Nopales evaluation runs against a remote Pricklypear \n\
     image over a proxy rim. The human operator talks to you through the \n\
     browser chat surface this process serves. You run as ONE unix process \n\
     directly on the operator's machine -- no container, no sandbox; your \n\
     commands affect the operator's real system.\n\
     \n\
     Harness facts (told to you here; do NOT probe the OS to discover them):\n\
     - host: %s (%s)\n\
     - cwd: %s -- the operator's working directory; treat as project root\n\
     - date: %s (operator local)\n\
     - model: %s via %s\n\
     - image: %s (basic-auth user %s) -- Nopales DB prims (get/create/\n\
       rows/tree/*...) proxy there per call; whoami/current-author are\n\
       server-asserted\n\
     - tools: read, bash, plus Nopales plugin tools registered with\n\
       (register-tool ...) -- a plugin tool's prim calls cross the proxy rim\n\
     \n\
     Conduct:\n\
     - Prefer the read and bash tools. Be terse.\n\
     - For questions about the project, read AGENTS.md/docs in cwd -- do not \n\
       probe the OS for facts listed above.\n\
     - Never print secret VALUES (env vars ending in _API_KEY, _TOKEN, \n\
       _SECRET); reference them by name. Avoid full `env` dumps."
    host (if uname = "" then "?" else uname) cwd date model base_url image
    user

(** ---- local-first lens (borge/local-first.borg Phase 1) ----

    [--lens DIR] wires [Remote.make ~lens_lookup] so function defs
    resolve from a filesystem JSON substrate BEFORE proxying
    [function-source] to the image -- the offline-first def path. The
    lens root is a directory of node-envelope .json files (one file per
    node, at its tree path) produced by [Lens_store.materialize_*].

    [--lens-pull NAME] (repeatable) seeds the lens from the live image:
    each NAME is fetched via the [function-source] proxy (the resolved,
    closest-wins source) and written as a code node under the lens
    branch, so the substrate is populated and subsequent evals read
    from disk. This is the Phase-1 materialize step done simply --
    online pull, then offline eval. *)
let () =
  let image = ref None in
  let user = ref None in
  let pass = ref None in
  let exprs = ref [] in
  let lens_dir = ref None in
  let lens_owner = ref None in
  let lens_device = ref None in
  let lens_branch = ref None in
  let lens_pull = ref [] in
  let lens_pull_tree = ref [] in
  let lens_pull_data = ref [] in
  let sync_flag = ref false in
  let sync_loop_flag = ref false in
  let sync_interval = ref 30.0 in
  let serve_flag = ref false in
  let open_browser = ref false in
  let model = ref None in
  let base_url = ref None in
```

```
205 let mock_script = ref None in
206 let anon _ = () in
207 let speclist =
208   Arg.align
209     [
210       ("--image", Arg.String (fun s -> image := Some s), "URL image
    base");
211       ("--user", Arg.String (fun s -> user := Some s), "NAME basic-auth
    user");
212       ("--pass", Arg.String (fun s -> pass := Some s), "PASS basic-auth
    pass");
213       ("-e", Arg.String (fun s -> exprs := s :: !exprs), "EXPR eval and
    exit");
214       ("-p", Arg.String (fun s -> exprs := s :: !exprs), "EXPR eval and
    exit");
215       ("--lens", Arg.String (fun s -> lens_dir := Some s), "DIR
    local-first lens root (offline-first defs from JSON)");
216       ("--lens-owner", Arg.String (fun s -> lens_owner := Some s), "OWNER
    lens owner (default: --user)");
217       ("--lens-device", Arg.String (fun s -> lens_device := Some s),
    "DEVICE lens device tag (default: hostname)");
218       ("--lens-branch", Arg.String (fun s -> lens_branch := Some s),
    "BRANCH lens branch id (default: lens-<device>)");
219       ("--lens-pull", Arg.String (fun s -> lens_pull := s :: !lens_pull),
    "NAME fetch def from image into lens (repeatable)");
220       ("--lens-pull-tree", Arg.String (fun s -> lens_pull_tree := s ::
    !lens_pull_tree), "NAME fetch def + transitive deps into lens
    (repeatable)");
221       ("--lens-pull-data", Arg.String (fun s -> lens_pull_data := s ::
    !lens_pull_data), "PREFIX fetch data subtree into lens: 'user' |
    'shared' | path (repeatable)");
222       ("--sync", Arg.Unit (fun () -> sync_flag := true), "push local lens
    data up to the image (two-way, data-only v1) then pull server-only
    paths; records the new (branch, generation) snapshot");
223       ("--sync-loop", Arg.Unit (fun () -> sync_loop_flag := true), "run
    the sync round on a timer in the background (the invisible part; needs
    --lens)");
224       ("--sync-interval", Arg.Float (fun f -> sync_interval := f),
    "SECONDS between background sync rounds (default 30)");
225       ("--serve", Arg.Set serve_flag, "serve the Nopales-authored HTML
    surface (chat + eval box + /code lens browser) on
    127.0.0.1:random-port until Ctrl-C");
226       ("--open", Arg.Set open_browser, "with --serve: best-effort browser
    launch");
227       ("--model", Arg.String (fun s -> model := Some s), "NAME provider
    model (AGENT_MODEL)");
228       ("--base-url", Arg.String (fun s -> base_url := Some s), "URL
    provider base URL (AGENT_BASE_URL)");
229       ("--mock-script", Arg.String (fun s -> mock_script := Some s),
    "FILE canned provider replies (test/demo of /chat)");
230       ]
231 in
232   Arg.parse speclist anon
233   "pp-remote [--image URL] [--user U --pass P] [-e EXPR]\n\
234    \                  [--lens DIR] [--lens-pull NAME] ... [--sync]
    [--sync-loop] [--serve [--open]]\n\
235    \                  [--model NAME] [--base-url URL] [--mock-script
    FILE]";
236 load_dotenv ();
237 (* TLS (tls-eio) needs a seeded RNG before any HTTPS proxy call;
238    mock transports in the tests never hit TLS, so this slipped.
239    Mirage_crypto_rng_unix.use_default wires /dev/urandom + reseeding. *)
240 Mirage_crypto_rng_unix.use_default ();
241 (* Random.bits drives multipart boundaries (agent) + future stream ids;
242    without self_init every process run repeats the same sequence. *)
243 Random.self_init ();
244 Eio_main.run @@ fun env ->
245 Eio.Switch.run @@ fun sw ->
246 let cfg : Http_transport.cfg =
247   {
248     base_url =
249       Option.value !image ~default:(getenv "PP_URL"
    ~default:"http://127.0.0.1:18080");
```

```
      user = Option.value !user ~default:(getenv "PP_USER" ~default:"admin");
      password = Option.value !pass ~default:(getenv "PP_PASS" ~default:"admin");
    }
  in
  let transport = Http_transport.transport ~env ~sw ~cfg in
  (* Hybrid placement: Local evaluates under the proxy rim; Remote
     (prim-dense / transactional) is POSTed to /api/eval wholesale. *)
  let remote_eval = Http_transport.eval_transport ~env ~sw ~cfg in
  (* ---- local-first lens wiring (Phase 1) ----
     If [--lens] is given, build a lens record and (optionally) pull
     defs from the image into the substrate, then pass [lens_lookup]
     to [Remote.make] so [fetch_def] consults the substrate first. *)
  let lens_opt =
    match !lens_dir with
    | None -> None
    | Some root ->
        let owner =
          Option.value !lens_owner ~default:cfg.user
        in
        let device =
          Option.value !lens_device
            ~default:(
              try
                let h = Unix.gethostname () in
                if h = "" then "laptop" else String.lowercase_ascii h
              with _ -> "laptop")
        in
        let branch =
          Option.value !lens_branch ~default:("lens-" ^ device)
        in
        if not (Sys.file_exists root) then begin
          let rec mkdir d =
            if d = "" || d = "." || d = Filename.dir_sep then ()
            else begin
              mkdir (Filename.dirname d);
              try Unix.mkdir d 0o755 with
              | Unix.Unix_error (Unix.EEXIST, _, _) -> ()
            end
          in
          mkdir root;
          Printf.eprintf "[lens] created empty lens root %s\n%!" root
        end;
        Some
          { Lens_store.root; owner; device; branch;
            generation = 0L (* unused by resolve_def in read-only mode *) }
  in
  let () =
    (* [--lens-pull] / [--lens-pull-tree]: seed the lens from the live
       image. [pull_one] fetches a single def via the [function-source]
       proxy and writes it as a code node; returns true if the image had
       the def, false on a nil/empty miss (prim, special form, or genuinely
       unbound). [--lens-pull-tree] walks the transitive closure of each
       name's free variables, skipping nil misses silently (they are prims
       / special forms, not defs) and logging only successful pulls. *)
    let pull_one ~lens ~name =
      match transport "function-source" [ Ast.VStr name ] with
      | Ok (Ast.VStr src) when String.trim src <> "" ->
          let _ =
            Lens_store.materialize_code_node ~lens ~branch:lens.branch
              ~kind:"fn" ~name ~src ~doc:"" ~author:lens.owner
          in
          Some src
      | Ok _ -> None
      | Error e ->
          Printf.eprintf "[lens] pull %s: %s\n%!" name (Printexc.to_string e);
          None
    in
    let pull_tree ~lens ~root_name =
      (* record ancestry (self-first) so resolve_def walks the lens
         branch. If a chain was already recorded, leave it. *)
```

```
      (match Lens_store.ancestry ~lens with
       | [] -> Lens_store.record_ancestry ~lens [ lens.branch ]
       | _ -> ());
      let visited = Hashtbl.create 16 in
      let queue = Queue.create () in
      Queue.push root_name queue;
      let pulled = ref 0 in
      while not (Queue.is_empty queue) do
        let name = Queue.pop queue in
        if Hashtbl.mem visited name then ()
        else begin
          Hashtbl.replace visited name ();
          match pull_one ~lens ~name with
          | Some src ->
              incr pulled;
              Printf.eprintf "[lens] pulled %s\n%!" name;
              (* walk the def's free vars for transitive deps *)
              let deps =
                try
                  Free_vars.free_vars_of_source src
                with _ -> []
              in
              List.iter
                (fun d ->
                  if not (Hashtbl.mem visited d) then Queue.push d queue)
                deps
          | None -> ()
          (* a prim / special form / unbound -- skip silently *)
        end
      done;
      Printf.eprintf "[lens] tree %s: %d def(s) pulled\n%!" root_name
     !pulled
    in
    (match lens_opt with
     | None -> ()
     | Some lens ->
         (* explicit pulls: log not-found (the user asked for these) *)
         if List.rev !lens_pull <> [] then
           (match Lens_store.ancestry ~lens with
            | [] -> Lens_store.record_ancestry ~lens [ lens.branch ]
            | _ -> ());
         List.iter
           (fun name ->
             match pull_one ~lens ~name with
             | Some _ -> Printf.eprintf "[lens] pulled %s\n%!" name
             | None -> Printf.eprintf "[lens] pull %s: not found on
    image\n%!" name)
           (List.rev !lens_pull);
         (* tree pulls: walk transitive closure silently skipping prims *)
         List.iter (fun n -> pull_tree ~lens ~root_name:n)
           (List.rev !lens_pull_tree);
         (* data pulls: materialize a datum subtree into the lens via
            [tree/subtree] (returns a VList of node envelopes, each the
            exact shape [Lens_store.materialize] writes). 'user' ?
            data.u.<owner>, 'shared' ? data.shared, else a raw path. *)
         List.iter
           (fun spec ->
             let prefix =
               match spec with
               | "user" -> "data.u." ^ Lens_store.enc_label lens.owner
               | "shared" -> "data.shared"
               | p -> p
             in
             match transport "tree/subtree" [ Ast.VStr prefix ] with
             | Ok (Ast.VList nodes) ->
                 let n = List.length nodes in
                 List.iter
                   (fun node ->
                     match node with
                     | Ast.VStr env ->
                         ignore (Lens_store.materialize ~lens env)
                     | _ -> ())
                   nodes;
```

```
                Printf.eprintf "[lens] pulled data %s: %d node(s)\n%!"
                  prefix n
            | Ok _ ->
                Printf.eprintf "[lens] pull-data %s: bad shape\n%!" prefix
            | Error e ->
                Printf.eprintf "[lens] pull-data %s: %s\n%!" prefix
                  (Printexc.to_string e))
          (List.rev !lens_pull_data);
        (* pin the (branch, generation) snapshot -- the merge base a
           later sync round's three-way diff keys on. Only when we
           actually pulled (code or data); a pure offline eval against
           an existing lens must not touch the wire. Catches transport
           exceptions (Eio raises, not Error) so an unreachable image
           degrades to an empty snapshot rather than crashing. *)
        if !lens_pull <> [] || !lens_pull_tree <> [] || !lens_pull_data <> [] then begin
          let branch =
            match (try transport "current-branch" [] with _ -> Error (Failure "offline")) with
            | Ok (Ast.VStr b) -> b | _ -> ""
          in
          let gen =
            match (try transport "gen-now" [] with _ -> Error (Failure "offline")) with
            | Ok (Ast.VInt g) -> g | _ -> 0L
          in
          Lens_store.record_snapshot ~lens ~branch ~generation:gen
        end)
  in
  let lens_lookup =
    match lens_opt with
    | None -> None
    | Some lens ->
        Some
          (fun name ->
            match Lens_store.resolve_def ~lens ~kind:"fn" ~name with
            | Some env -> Lens_store.code_src_of_envelope env
            | None -> None)
  in
  let t = Remote.make ~transport ~remote_eval ?lens_lookup () in
  (* Override [get] / [tree/get] with lens-local handlers so working-set
     reads resolve from the substrate (offline-first data path), and pin
     the lens record so lens/list + lens/read surface prims browse it. *)
  (match lens_opt with
   | None -> ()
   | Some lens ->
       Remote.register_lens_reads t ~lens;
       Remote.set_lens t lens);

  (* Boot-load the Nopales surface (libs/html + libs/hyper + the
     eval/code route-table handlers) so the HTML surface is
     Nopales-authored and redefinable live (borge/remote-eval.borg
     surfaces). The route + soup + pure prims are registered in
     [Remote.make]; [Surface.load] runs the embedded source under the
     proxy rim. *)
  Surface.load ~rt:t;

  (* Install the builtin agent tools (read/bash -- llm/lib/tools.ml) into
     the client's tool registry so the chat loop has them. Path
     resolution is the client's sandbox (same refuse-".." posture as the
     agent). *)
  Tools.install ~resolve_path:Remote.resolve_path
    ~register:(fun name run -> Remote.register_builtin_tool t name ~run);

  (* ---- --serve: serve the Nopales-authored HTML surface (chat + eval
     box + /code lens browser) on 127.0.0.1:random until Ctrl-C. The
     chat turn runs the SHARED agent loop (llm/) over this client: the
     provider call is local (API key from AGENT_API_KEY / --mock-script),
     tool bodies eval under the proxy rim so DB prims cross to the
     image. Mutually exclusive with --sync / REPL / -e. *)
  (if !serve_flag then begin
    let model_name =
```

```
      Option.value !model ~default:(getenv "AGENT_MODEL"
    ~default:"hf:zai-org/GLM-4.7-Flash")
  in
  let provider_base =
    Option.value !base_url
      ~default:(getenv "AGENT_BASE_URL"
    ~default:"https://api.synthetic.new/openai/v1")
  in
  let pcfg : Provider.cfg =
    {
      base_url = provider_base;
      api_key = getenv "AGENT_API_KEY" ~default:"";
      model = model_name;
      system =
        getenv "AGENT_SYSTEM"
          ~default:(harness_context ~model:model_name
    ~base_url:provider_base
                      ~image:cfg.base_url ~user:cfg.user);
    }
  in
  (if pcfg.Provider.api_key = "" && !mock_script = None then
     Printf.eprintf
       "[chat] AGENT_API_KEY not set -- /chat will fail provider calls;
    set \n\
        AGENT_API_KEY (or AGENT_BASE_URL/AGENT_MODEL) or pass
    --mock-script\n%!");
  let script_cell =
    ref (Option.map parse_mock_script !mock_script |> Option.value
    ~default:[])
  in
  let complete =
    match !mock_script with
    | Some _ ->
        (fun ~env ~sw ~cfg ~messages ~tools ~on_delta ->
           mock_complete ~env ~sw ~cfg ~messages ~tools ~on_delta
    script_cell)
    | None -> Provider.Openai.complete
  in
  Http_ui.serve ~env ~sw ~rt:t ~cfg:pcfg ~complete
    ~open_browser:!open_browser print_endline;
  exit 0
end);

(* ---- --sync: run ONE sync round via remote/lib/sync_client.ml
   (borge/local-first.borg sync): push local lens data up to the image,
   pull server copies, mirror server deletions, surface every LWW
   resolution, and pin the new (branch, generation) snapshot + merged
   manifest as the next round's base. *)
(if !sync_flag then begin
  match lens_opt with
  | None ->
      prerr_endline "--sync requires --lens DIR"; exit 2
  | Some lens ->
      let r = Sync_client.run_round ~transport ~lens ~quiet:false () in
      if not r.Sync_client.ok then exit 1
end);

(* ---- --sync-loop: the BACKGROUND half of "sync invisibly in the
   background" (borge/local-first.borg sync -- "'background' is client
   scheduling, not new machinery"). An Eio fiber runs the same round
   every --sync-interval seconds (quiet mode: activity and surfaced
   resolutions print, a clean round is silent); a failed round (image
   unreachable -- offline work continues locally) retries on a short
   backoff so reconnect is automatic. Runs alongside --serve / the
   REPL. *)
(if !sync_loop_flag then
   match lens_opt with
   | None ->
       prerr_endline "--sync-loop requires --lens DIR"; exit 2
   | Some lens ->
       let clock = Eio.Stdenv.clock env in
       Printf.eprintf "[sync-loop] every %.0fs against %s\n%!"
         !sync_interval cfg.base_url;
```

```
        Eio.Fiber.fork ~sw (fun () ->
            while true do
              let r =
                try Sync_client.run_round ~transport ~lens ~quiet:true ()
                with exn ->
                  Printf.eprintf "[sync-loop] round failed: %s\n%!"
                    (Printexc.to_string exn);
                  { (Sync_client.empty_report ()) with
                    Sync_client.ok = false;
                    error = Some (Printexc.to_string exn) }
              in
              let backoff =
                if r.Sync_client.ok then !sync_interval
                else Float.min !sync_interval 5.0
              in
              Eio.Time.sleep clock backoff
          done));

  let eval_print src =
    try
      let v = Remote.eval t src in
      print_endline (Reader.pp_value v)
    with
    | Ast.Interp_error ie ->
        Printf.eprintf "error: %s\n%!" (Ast.pp_interp_error ie); exit 1
    | Reader.Reader_error (msg, ln, col) ->
        Printf.eprintf "read error %d:%d %s\n%!" ln col msg; exit 1
  in
  match List.rev !exprs with
  | [] ->
      (* stdin REPL: same env, same prims, same proxy rim. *)
      (match lens_opt with
       | None ->
           Printf.printf "pp-remote -- %s  user=%s\n%!" cfg.base_url
     cfg.user
       | Some lens ->
           Printf.printf "pp-remote -- %s  user=%s  [lens %s, branch
     %s]\n%!"
             cfg.base_url cfg.user lens.root lens.branch;
           if Lens_store.ancestry ~lens = [] then
             prerr_endline
               "[lens] no ancestry recorded -- defs will proxy; use
     --lens-pull to seed");
       print_endline "enter Nopales forms; :quit to exit";
       let rec loop () =
         output_string stdout "remote> ";
         flush stdout;
         (try
            match input_line stdin |> String.trim with
            | line when line = "" -> loop ()
            | line when line = ":quit" || line = ":q" -> ()
            | line ->
                (try
                   let v = Remote.eval t line in
                   print_endline (Reader.pp_value v)
                 with
                 | Ast.Interp_error ie ->
                     Printf.printf "error: %s\n%!" (Ast.pp_interp_error ie)
                 | Reader.Reader_error (msg, ln, col) ->
                     Printf.printf "read error %d:%d %s\n%!" ln col msg);
                loop ()
          with End_of_file -> print_newline ())
      in
      loop ()
  | xs -> List.iter eval_print xs
```

284 remote/lib/def_cache.ml

```
(** remote/lib/def_cache.ml (borge/remote-eval.borg caching).

    The client-side half of the definition cache, keyed
    (branch, generation). The client materializes defs lazily into its
    env (client subsection, [Remote.load_def]); this module is the
    INVALIDATION half: "is my (branch, generation) still current?" is a
    single cheap round trip (two single-counter read prims) -- NOT an
    N-way hash diff over every def.

    KEY. The cache holds (def-name -> (value, source-generation)) and a
    pinned (branch, generation) key. Generation is migration-017
    branch_generations -- the SAME coordinate tree-time reads as-of and
    the (gen-now) prim read (the campaign-7.0 dependency pin): one
    counter, one namespace, no independent version clock to drift.

    INVALIDATION ([check] returns the names to evict from the client
    env so the next reference re-fetches from the writer -- the server is
    the source of truth, the cache never is):
      - Branch is the namespace: a checkout to a different branch
        invalidates the WHOLE cache (every cached name is returned for
        eviction).
      - A generation bump (a write anywhere in the branch's ancestry --
        a promote into main, a define on personal, a data write) marks
        every def loaded before the new generation STALE and returns
        their names for eviction.
      - No writes -> the check round trip returns unchanged-generation
        and returns [] (nothing touched).
    A def-free / pure-compute session never pins a coordinate, so
    [is_active] is false and the round trip is skipped entirely (zero
    HTTP on pure compute).

    TRANSPORT-DRIVEN (proxy rim discipline). This module does zero
    interpretation itself. It reads the two counter prims (current-branch
    + gen-now, both proxyable reads registered on the image) through the
    same [Proxy_rim.transport] everything else uses, so it is unit-testable
    with a mock (no live server). It never writes. It is a cache +
    interpreter, never a source of truth. *)

open Pricklypear_lisp
open Ast

type transport = Proxy_rim.transport

(** A materialized def: the value [load_def] produced under the env and
    the generation counter it was fetched at (so a bump can drop it). *)
type entry = {
  value : value;
  source_gen : int64; (* branch_generations at fetch time *)
}

type t = {
  mutable branch : string; (* branch namespace *)
  mutable generation : int64; (* cached generation *)
  mutable initialized : bool; (* branch + gen pinned yet? *)
  mutable table : (string, entry) Hashtbl.t;
  transport : transport;
}

let create ~(transport : transport) () : t =
  {
    branch = "";
    generation = 0L;
    initialized = false;
    table = Hashtbl.create 16;
    transport;
  }

(** Whether the cache is live (a (branch, gen) coordinate has been
    pinned). A pure-compute session that never materialized a def stays
    [!is_active], so [Remote] skips the check round trip entirely. *)
let is_active (t : t) : bool = t.initialized
```

```
72
73  (** One cheap round trip: current-branch + gen-now, both proxyable read
74      prims (single counters). Returns (branch, generation). Either prim
75      failing resolves conservatively to ("" , 0) -- a caller treating an
76      unknown coordinate as "recheck later / at head" stays safe. *)
77  let read_key (t : t) : (string * int64) option =
78    (* The transport may RAISE on connection failure (the Eio HTTP
79       transport raises [Eio.Io], it does not return [Error]); catch
80       both so an offline client conservatively treats the coordinate as
81       unknown rather than crashing -- the [check] docstring's "transport
82       failure conservatively returns []" contract, made honest. *)
83    let branch =
84      match (try t.transport "current-branch" [] with _ -> Error (Failure
       "offline")) with
85      | Ok (VStr b) -> b
86      | _ -> ""
87    in
88    let gen =
89      match (try t.transport "gen-now" [] with _ -> Error (Failure
       "offline")) with
90      | Ok (VInt g) -> g
91      | _ -> 0L
92    in
93    Some (branch, gen)
94
95  (** The cache-check round trip. Returns the list of def names whose ENV
96      binding should be evicted (for the client to re-fetch on next use).
97      Idempotent; [] (no evictions) on the unchanged path.
98
99      - First call: pin (branch, generation) and return [].
100     - Branch change: the namespace moved -- clear the WHOLE cache and
101       return every cached name (evict all from the env).
102     - Generation bump: drop every def loaded before the new generation
103       and return their names (evict only stale defs from the env).
104     - Otherwise: unchanged -- a single cheap HEAD-ish round trip, [].
105     A transport failure (none of the reads answered) conservatively
106     returns [] (treat as "recheck later"; never evict on uncertainty). *)
107 let check (t : t) : string list =
108   match read_key t with
109   | Some (branch, gen) ->
110       if not t.initialized then (
111         t.branch <- branch;
112         t.generation <- gen;
113         t.initialized <- true;
114         [])
115       else if branch <> t.branch then (
116         (* branch is the namespace -- a checkout invalidates everything *)
117         let stale = Hashtbl.fold (fun n _ acc -> n :: acc) t.table [] in
118         Hashtbl.clear t.table;
119         t.branch <- branch;
120         t.generation <- gen;
121         stale)
122       else if Int64.compare gen t.generation <> 0 then (
123         (* generation bumped: reload affected defs (stale = fetched
124            before this gen; keep any fetched at-or-after it) *)
125         let ng = gen in
126         let stale = ref [] in
127         Hashtbl.filter_map_inplace
128           (fun n e ->
129             if Int64.compare e.source_gen ng < 0 then (
130               stale := n :: !stale;
131               None)
132             else Some e)
133           t.table;
134         t.generation <- ng;
135         !stale)
136       else []
137   | None -> []
138
139 (** Look up a materialized def by name (nil on miss). *)
140 let lookup (t : t) (name : string) : value option =
141   match Hashtbl.find_opt t.table name with
142   | Some e -> Some e.value
```

```
  | None -> None

(** Record a def as materialized under the current generation. *)
let store (t : t) (name : string) (v : value) : unit =
  Hashtbl.replace t.table name { value = v; source_gen = t.generation }

(** Drop a single def (used when a fetch fails / the name is genuinely
    unbound, so it is not cached as present). *)
let remove (t : t) (name : string) : unit =
  Hashtbl.remove t.table name

let branch (t : t) : string = t.branch
let generation (t : t) : int64 = t.generation

(** Number of defs currently cached (testing / introspection). *)
let size (t : t) : int = Hashtbl.length t.table
```

285 remote/lib/free_vars.ml

```
(** remote/lib/free_vars.ml -- free-variable analysis over a Nopales AST.

    Used by [--lens-pull-tree] (remote/bin/main.ml) to walk the transitive
    closure of a fetched def's dependencies: every free [Var] is a
    candidate def to pull from the image. The reader produces special
    forms as [App(Var "lambda", ?)] (eval desugars later), so we dispatch
    on those App forms -- binding lambda params / let names so they
    aren't mistaken for deps, and treating [quote]/[quasiquote] bodies
    as data (not code). Kernel special-form names are never deps.

    Pure (no I/O, no env); unit-tested in remote/test/free_vars_test.ml. *)

open Pricklypear_lisp

(** [lambda_params] extracts param names from the reader's App form of a
    lambda param list: (n) -> App(Var "n",[]); (a b) ->
    App(Var "a",[Var "b"]); () -> Lit VNil. *)
let lambda_params (pe : Ast.expr) : string list =
  match pe with
  | Ast.Lit Ast.VNil -> []
  | Ast.App (Ast.Var p, rest) ->
      p :: List.filter_map (function Ast.Var s -> Some s | _ -> None) rest
  | _ -> []

(** [let_bindings] extracts (name, val_expr) pairs from the reader's App
    form of a let/letrec binding list: ((a 1) (b 2)) ->
    App(App(Var "a",[1]), [App(Var "b",[2])]). *)
let let_bindings (be : Ast.expr) : (string * Ast.expr) list =
  match be with
  | Ast.Lit Ast.VNil -> []
  | Ast.App (first, rest) ->
      List.filter_map
        (fun b ->
          match b with
          | Ast.App (Ast.Var n, [ val_e ]) -> Some (n, val_e)
          | _ -> None)
        (first :: rest)
  | _ -> []

(** Free variables referenced in an [expr], minus [bound]. Every free
    [Var] is a candidate def to pull. The reader produces special forms
    as [App(Var "lambda", ?)] (eval desugars later), so we dispatch on
    those App forms: binding lambda params / let names, treating
    [quote]/[quasiquote] bodies as data. Kernel special-form names are
    never deps. The desugared AST nodes (Lambda/Let/?) are also handled
    so the walker is correct against either shape. *)
let rec free_vars (bound : string list) (e : Ast.expr) : string list =
  let open Ast in
  let is_kernel n = List.mem n Eval.kernel_special_names in
  match e with
  | Var s -> if List.mem s bound || is_kernel s then [] else [ s ]
  | Lit _ -> []
  | App (Var f, args) when is_kernel f ->
      (match f, args with
       | "lambda", params_e :: bodies ->
           let params = lambda_params params_e in
           List.concat_map (free_vars (params @ bound)) bodies
       | "let", bindings_e :: body :: _ ->
           let bs = let_bindings bindings_e in
           let names = List.map fst bs in
           let vals = List.map snd bs in
           List.concat_map (free_vars bound) vals @ free_vars (names @
    bound) body
       | "letrec", bindings_e :: body :: _ ->
           let bs = let_bindings bindings_e in
           let names = List.map fst bs in
           let bound' = names @ bound in
           List.concat_map (free_vars bound') (List.map snd bs)
           @ free_vars bound' body
       | "define", name_or_app :: rest ->
           (* (define name val) or (define (name ps?) body?). For the
```

```
71                function form, the params in the (name ps?) App are bound
72                in the body -- extract them so they aren't mistaken for
73                deps. The name itself is bound (the def being defined). *)
74             let bound' =
75               match name_or_app with
76               | Ast.App (Ast.Var _, pargs) ->
77                   List.filter_map (function Ast.Var s -> Some s | _ -> None)
       pargs
78               | _ -> []
79             in
80             List.concat_map (free_vars (bound' @ bound)) rest
81         | ("quote"), _ -> []
82         | ("quasiquote" | "unquote" | "unquote-splicing"), _ -> []
83         | ("defmacro" | "macroexpand" | "macroexpand-1" | "gensym" |
       "error"), _ ->
84               []
85         | ( "if" | "begin" | "seq" | "cond" | "and" | "or" | "match" |
       "guard"
86             | "apply" | "list/map" | "list/filter" | "list/foldl" |
       "list/foldr"
87             | "prim" | "delay" | "force" | "with-transaction" | "with-step" ),
       _
88             ->
89               List.concat_map (free_vars bound) args
90         | _ -> List.concat_map (free_vars bound) args)
91   | App (f, args) ->
92       free_vars bound f @ List.concat_map (free_vars bound) args
93   | Let (name, val_e, body) ->
94       free_vars bound val_e @ free_vars (name :: bound) body
95   | Letrec (bindings, body) ->
96       let names = List.map fst bindings in
97       let bound' = names @ bound in
98       List.concat_map (fun (_, e') -> free_vars bound' e') bindings
99       @ free_vars bound' body
100  | If (t, a, b) -> free_vars bound t @ free_vars bound a @ free_vars bound
       b
101  | Lambda (params, body) -> free_vars (params @ bound) body
102  | Prim (_, args) -> List.concat_map (free_vars bound) args
103  | Seq (a, b) -> free_vars bound a @ free_vars bound b
104  | Quote _ -> []
105  | Define (name, val_e) -> free_vars (name :: bound) val_e
106
107 (** Free vars across a whole source string (all top-level forms). Used by
108     [--lens-pull-tree] to scan a fetched def's source for transitive
109     deps. Returns [] on a reader error (conservative: a def we can't
110     parse has no scan-able deps -- the puller will still materialize the
111     def itself; it just won't recurse its body). *)
112 let free_vars_of_source (src : string) : string list =
113   try List.concat_map (free_vars []) (Reader.read_all src)
114   with _ -> []
```

286 remote/lib/http_transport.ml

```
(** remote/lib/http.ml (borge/remote-eval.borg client).

    The production [Proxy_rim.transport]: POST /api/prim to the image
    over a hand-rolled HTTP/1.1 connection on an Eio flow (plain or
    TLS). No curl -- the agent's [Provider.post_json] is the reference
    pattern; gluten (Httpun_eio.Client) refuses non-socket TLS flows,
    and a 1.1 request/response with `connection: close` is honest.

    Auth is HTTP Basic (user:pass base64), the SAME path /api/eval and
    /api/prim accept on the image. Identity is server-asserted: the
    client only presents credentials; whoami/current-author come back
    FROM the server. The transport returns (value, exn) Result so it
    slots directly into [Proxy_rim.run] -- errors are re-raised by the
    rim via [Prims_pure.error_of_json] so the client sees identical
    exceptions to in-process eval. *)

open Pricklypear_lisp
open Ast

type cfg = {
  base_url : string;   (* e.g. http://127.0.0.1:18080 *)
  user : string;
  password : string;
}

let trim_url base =
  let base = String.trim base in
  if base = "" then "http://127.0.0.1:18080"
  else if base.[String.length base - 1] = '/' then
    String.sub base 0 (String.length base - 1)
  else base

(** Split a http(s)://host[:port] base into (host, port, tls). *)
let parse_base base =
  match String.split_on_char '/' (trim_url base) with
  | "https:" :: "" :: hostport :: _ ->
      let h, p =
        match String.split_on_char ':' hostport with
        | [ h; p ] -> (h, int_of_string p)
        | _ -> (hostport, 443)
      in
      (h, p, true)
  | "http:" :: "" :: hostport :: _ ->
      let h, p =
        match String.split_on_char ':' hostport with
        | [ h; p ] -> (h, int_of_string p)
        | _ -> (hostport, 80)
      in
      (h, p, false)
  | _ -> failwith ("bad base_url: " ^ base)

let basic_auth_header ~user ~password =
  let cred = user ^ ":" ^ password in
  let b64 = Base64.encode_string ~pad:true cred in
  ("authorization", "Basic " ^ b64)

(** Read the response body off [br] honoring transfer-encoding /
    content-length, mirroring agent/lib/provider.ml post_json. *)
let read_body br ~headers =
  let line () = String.trim (Eio.Buf_read.line br) in
  match List.assoc_opt "transfer-encoding" headers with
  | Some te when String.lowercase_ascii te = "chunked" ->
      let buf = Buffer.create 8192 in
      let rec chunks () =
        let size_line = line () in
        match int_of_string_opt ("0x" ^ size_line) with
        | Some 0 -> ignore (line ()) (* trailing CRLF *)
        | Some n ->
            Buffer.add_string buf (Eio.Buf_read.take n br);
            ignore (line ()); (* CRLF after chunk *)
            chunks ()
```

```
        | None -> ()
      in
      chunks ();
      Buffer.contents buf
  | _ ->
      (match List.assoc_opt "content-length" headers with
       | Some n -> Eio.Buf_read.take (int_of_string n) br
       | None -> Eio.Buf_read.take_all br)

(** Generic one-POST: POST [path] with [payload] (a JSON value, encoded
    as the request body) and return (parsed-response-JSON | Error msg).
    The same HTTP/1.1-over-Eio/TLS plumbing as [post_prim], shared by
    /api/prim (per prim call) and /api/eval (whole expression). *)
let post_json
    ~(env : Eio_unix.Stdenv.base) ~(sw : Eio.Switch.t) ~(cfg : cfg)
    ~(path : string) ~(payload : Yojson.Safe.t)
    : (Yojson.Safe.t, string) Result.t =
  let open Eio in
  let host, port, tls = parse_base cfg.base_url in
  let body = Yojson.Safe.to_string payload in
  let net = (Stdenv.net env :> [ `Generic ] Net.ty Std.r) in
  let addr =
    Net.getaddrinfo_stream ~service:(string_of_int port) net host |> List.hd
  in
  let raw = Net.connect ~sw net addr in
  let flow : Flow.two_way_ty Std.r =
    if tls then
      let auth = Result.get_ok (Ca_certs_nss.authenticator ()) in
      let tls_config = Result.get_ok (Tls.Config.client ~authenticator:auth
    ()) in
      (Tls_eio.client_of_flow tls_config
         ~host:(Domain_name.host_exn (Domain_name.of_string_exn host))
         raw
       : Tls_eio.t :> Flow.two_way_ty Std.r)
    else (raw :> Flow.two_way_ty Std.r)
  in
  let request_text =
    let hdrs =
      String.concat "\r\n"
        (List.map
           (fun (k, v) -> k ^ ": " ^ v)
           ([ ("host", host); ("content-type", "application/json");
              ("content-length", string_of_int (String.length body));
              ("connection", "close")
            ; basic_auth_header ~user:cfg.user ~password:cfg.password ]))
    in
    Printf.sprintf "POST %s HTTP/1.1\r\n%s\r\n\r\n%s" path hdrs body
  in
  Flow.copy_string request_text flow;
  let br = Buf_read.of_flow ~initial_size:4096 ~max_size:(20 * 1024 * 1024)
     flow in
  let line () = String.trim (Buf_read.line br) in
  let status_line = line () in
  let code =
    match String.split_on_char ' ' status_line with
    | _http :: c :: _ -> (try int_of_string c with _ -> 0)
    | _ -> 0
  in
  let rec read_headers acc =
    let l = line () in
    if l = "" then List.rev acc
    else
      (match String.index_opt l ':' with
       | Some i ->
           read_headers
             ((String.lowercase_ascii (String.sub l 0 i),
               String.trim (String.sub l (i + 1) (String.length l - i -
    1))) :: acc)
       | None -> read_headers acc)
  in
  let headers = read_headers [] in
  let body = read_body br ~headers in
  (try Flow.shutdown flow `All with _ -> ());
```

```
  if code >= 200 && code < 300 then
    (try Ok (Yojson.Safe.from_string body)
     with Yojson.Json_error e -> Error ("bad json: " ^ e))
  else Error (Printf.sprintf "HTTP %d: %s" code body)

(** One POST to /api/prim with {name, args:[json?]}. Returns the parsed
    JSON response (or an Error message). *)
let post_prim
    ~(env : Eio_unix.Stdenv.base) ~(sw : Eio.Switch.t) ~(cfg : cfg)
    ~(name : string) ~(args_json : Yojson.Safe.t)
    : (Yojson.Safe.t, string) Result.t =
  post_json ~env ~sw ~cfg ~path:"/api/prim"
    ~payload:(`Assoc [ ("name", `String name); ("args", args_json) ])

(** One POST to /api/eval with {expr}. Returns the parsed JSON response
    (or an Error message). *)
let post_eval
    ~(env : Eio_unix.Stdenv.base) ~(sw : Eio.Switch.t) ~(cfg : cfg)
    ~(expr : string) : (Yojson.Safe.t, string) Result.t =
  post_json ~env ~sw ~cfg ~path:"/api/eval"
    ~payload:(`Assoc [ ("expr", `String expr) ])

(** Build a [Proxy_rim.transport] from an Eio env + [cfg]. The
    transport is the function the proxy rim calls per Prim_call: it
    serializes args, POSTs, and returns (Ok value | Error exn). A
    server-only refusal ({ok:false, kind:"server_only"}) is surfaced
    as an [Interp_error (Prim_error ?)] so it behaves like any other
    prim failure to the client. *)
let transport
    ~(env : Eio_unix.Stdenv.base) ~(sw : Eio.Switch.t) ~(cfg : cfg)
    : Proxy_rim.transport =
  fun (name : string) (args : value list) ->
    let args_json = `List (List.map Prims_pure.value_to_json args) in
    match post_prim ~env ~sw ~cfg ~name ~args_json with
    | Error msg ->
        Error
          (Ast.Interp_error
             (Ast.Prim_error (name, "transport: " ^ msg, "transport")) :
     exn)
    | Ok j ->
        let open Yojson.Safe.Util in
        match j |> member "ok" with
        | `Bool true ->
            (try Ok (Prims_pure.json_to_value (j |> member "value"))
             with _ -> Error (Ast.Interp_error (Ast.Prim_error (name, "bad
     value json", "prim_error")) : exn))
        | _ ->
            let kind = j |> member "kind" |> to_string_option |>
     Option.value ~default:"error" in
            let msg = j |> member "error" |> to_string_option |>
     Option.value ~default:"prim failed" in
            (* A server-side error_to_json payload carries the structured
               Interp_error/Reader_error; reconstruct it so the client
               raises the SAME exn. Otherwise (server_only / generic)
               synthesize a Prim_error naming the prim. *)
            (match j |> member "kind" with
             | `String k when k = "interp_error" || k = "parse_error" ->
                 (try Error (Prims_pure.error_of_json j) with _ ->
                  Error (Ast.Interp_error (Ast.Prim_error (name, msg,
     kind)) : exn))
             | _ -> Error (Ast.Interp_error (Ast.Prim_error (name, msg,
     kind)) : exn))

(** Build the whole-expression remote eval hook ([Remote.set_remote_eval])
    from an Eio env + [cfg]. POSTs the whole source to /api/eval (the
    image's own eval path, so server-side /api/eval is authoritative
    for placement-Remote expressions) and returns the value, raising the
    SAME exception an in-process eval would (via the shared codec). This
    is the Remote half of hybrid placement: Local runs under the proxy
    rim, Remote (prim-dense / transactional) POSTs here wholesale. *)
let eval_transport
    ~(env : Eio_unix.Stdenv.base) ~(sw : Eio.Switch.t) ~(cfg : cfg)
    : string -> Ast.value =
```

```
  fun (expr : string) ->
    match post_eval ~env ~sw ~cfg ~expr with
    | Error msg ->
        raise (Ast.Interp_error (Ast.Prim_error ("api/eval", msg,
     "transport")))
    | Ok j ->
        let open Yojson.Safe.Util in
        (match j |> member "ok" with
         | `Bool true ->
             (try Prims_pure.json_to_value (j |> member "value")
              with _ ->
                raise (Ast.Interp_error (Ast.Prim_error ("api/eval", "bad
     value json", "prim_error"))))
         | _ ->
             let kind =
               j |> member "kind" |> to_string_option
               |> Option.value ~default:"error"
             in
             let msg =
               j |> member "error" |> to_string_option
               |> Option.value ~default:"eval failed"
             in
             (match j |> member "kind" with
              | `String k when k = "interp_error" || k = "parse_error" ->
                  (try raise (Prims_pure.error_of_json j)
                   with
                   | Ast.Interp_error _ as e -> raise e
                   | Reader.Reader_error _ as e -> raise e
                   | _ -> raise (Ast.Interp_error (Ast.Prim_error
     ("api/eval", msg, kind))))
              | _ -> raise (Ast.Interp_error (Ast.Prim_error ("api/eval",
     msg, kind)))))
```

287 remote/lib/http_ui.ml

```
(** pp-remote HTML surface server (borge/remote-eval.borg surfaces +
    agent-loop).

    A local HTTP server (Httpun_eio) bound to 127.0.0.1 on a random
    port. The pages are NOPALES-AUTHORED htmx fragments on the client
    route table (rendered by Nopales via [libs/html]), dispatched
    in-process through [Remote.dispatch_route]. The eval POST handler
    is OCaml (it calls [Remote.eval] and renders the result via the
    Nopales [eval/result] template).

    The CHAT TURN is the shared agent loop running on THIS host
    (borge/remote-eval.borg agent-loop): provider call local (the API
    key is a value from the environment), tool bodies eval under the
    proxy rim (DB prims cross to /api/prim), streaming as
    multipart/mixed htmx fragments -- the same emitter + Nopales
    fragment templates the agent serves (chat/user-bubble,
    chat/turn-open, chat/think-open, chat/think-token, chat/tool-open,
    chat/tool-line, chat/tool-done, chat/token, chat/error).

    Bootstrap/asset plumbing stays OCaml (per the surfaces scope): the
    vendored htmx assets are baked into the binary ([Assets]) and
    served at /assets/*.js. Sibling of agent/lib/http_ui.ml: the HTTP
    plumbing + GET dispatch + asset serving mirror it; the chat stream
    drives Remote.t instead of Runtime.t. *)

open Pricklypear_lisp

(* ---- HTML escaping (status/error pages only; surfaces are Nopales) ----
     *)

let esc (s : string) : string =
  let buf = Buffer.create (String.length s + 16) in
  String.iter
    (fun c ->
      match c with
      | '<' -> Buffer.add_string buf "&lt;"
      | '>' -> Buffer.add_string buf "&gt;"
      | '&' -> Buffer.add_string buf "&amp;"
      | '"' -> Buffer.add_string buf "&quot;"
      | c -> Buffer.add_char buf c)
    s;
  Buffer.contents buf

let status_page (title : string) (body : string) : string =
  Printf.sprintf
    {|<!doctype html><html><head><meta charset="utf-8"><title>%s</title>
<style>body{background:#0f1117;color:#d8dee9;font-family:ui-monospace,monospace;padding:1.5rem}a{color:#7aa2f7}</style>
</head><body>%s</body></html>|}
    (esc title) body

(* ---- HTTP plumbing ---- *)

let read_body (reqd : Httpun.Reqd.t) : string =
  let body = Httpun.Reqd.request_body reqd in
  let promise, resolver = Eio.Promise.create () in
  let buf = Buffer.create 1024 in
  let rec sched () =
    Httpun.Body.Reader.schedule_read body
      ~on_eof:(fun () -> Eio.Promise.resolve resolver (Buffer.contents buf))
      ~on_read:(fun bs ~off ~len ->
        Buffer.add_string buf (Bigstringaf.substring bs ~off ~len);
        sched ())
  in
  sched ();
  Eio.Promise.await promise

let respond_text (reqd : Httpun.Reqd.t) ~(status : Httpun.Status.t)
    ~(content_type : string) ~(body : string) : unit =
  let headers =
    Httpun.Headers.of_list
      [
```

```
        ("content-type", content_type);
        ("content-length", string_of_int (String.length body));
      ]
  in
  let _body_w =
    Httpun.Reqd.respond_with_string reqd
      (Httpun.Response.create ~headers status)
      body
  in
  ()

let respond_html (reqd : Httpun.Reqd.t) ~(status : Httpun.Status.t)
    ~(body : string) : unit =
  respond_text reqd ~status ~content_type:"text/html; charset=utf-8" ~body

(* ---- query-string + form parsing (route params + eval field) ---- *)

let url_decode (s : string) : string =
  let buf = Buffer.create (String.length s) in
  let len = String.length s in
  let rec go i =
    if i >= len then Buffer.contents buf
    else if i + 2 < len && s.[i] = '%' then (
      (try
         Buffer.add_char buf
           (Char.chr (int_of_string ("0x" ^ String.sub s (i + 1) 2)))
       with _ -> ());
      go (i + 3))
    else (
      Buffer.add_char buf (if s.[i] = '+' then ' ' else s.[i]);
      go (i + 1))
  in
  go 0

let parse_query (q : string) : (string * string) list =
  if q = "" then []
  else
    List.filter_map
      (fun kv ->
        match String.index_opt kv '=' with
        | Some i ->
            let k = String.sub kv 0 i in
            let v = String.sub kv (i + 1) (String.length kv - i - 1) in
            Some (url_decode k, url_decode v)
        | None -> Some (url_decode kv, ""))
      (String.split_on_char '&' q)

(* url-decode a form field value (htmx posts
    application/x-www-form-urlencoded). *)
let form_field (body : string) (name : string) : string =
  let prefix = name ^ "=" in
  let rec find i =
    if i >= String.length body then None
    else if i = 0 || body.[i - 1] = '&' then
      if String.length body - i >= String.length prefix
         && String.sub body i (String.length prefix) = prefix
      then
        let rest_start = i + String.length prefix in
        let rest_end =
          match String.index_from_opt body rest_start '&' with
          | Some j -> j | None -> String.length body
        in
        Some (String.sub body rest_start (rest_end - rest_start))
      else find (i + 1)
    else find (i + 1)
  in
  match find 0 with None -> "" | Some v -> url_decode v

(* ---- GET route dispatch through the client route table ---- *)

let dispatch_get (rt : Remote.t) (reqd : Httpun.Reqd.t) (target : string)
    : unit =
  let path, query =
```

```
    match String.index_opt target '?' with
    | Some i ->
        ( String.sub target 0 i,
          String.sub target (i + 1) (String.length target - i - 1) )
    | None -> (target, "")
  in
  let params = parse_query query in
  match Remote.lookup_route ~rt path with
  | None ->
      respond_html reqd ~status:`Not_found
        ~body:(status_page "404" (Printf.sprintf "<p>no route %s</p>" (esc
     path)))
  | Some handler ->
      (try
         let v =
           Remote.dispatch_route ~rt ~handler ~method_:"GET" ~path ~params
             ~headers:[] ~body:""
         in
         let html =
           match v with Ast.VStr s -> s | other ->
     Pricklypear_lisp.Reader.pp_value other
         in
         respond_html reqd ~status:`OK ~body:html
       with exn ->
         respond_html reqd ~status:`Internal_server_error
           ~body:(status_page "error"
                    (Printf.sprintf "<p>dispatch error: %s</p>"
                       (esc (Printexc.to_string exn)))))

(* ---- static assets (vendored, baked into the binary) ---- *)

let serve_asset (reqd : Httpun.Reqd.t) (name : string) : unit =
  let body, ct =
    match name with
    | "htmx.min.js" -> (Assets.htmx_min, "application/javascript;
     charset=utf-8")
    | "htmx-2-compat.min.js" -> (Assets.htmx_2_compat,
     "application/javascript; charset=utf-8")
    | "hx-multipart.js" -> (Assets.hx_multipart, "application/javascript;
     charset=utf-8")
    | _ -> ("", "application/octet-stream")
  in
  if body = "" then
    respond_html reqd ~status:`Not_found
      ~body:(status_page "404" (Printf.sprintf "<p>no asset %s</p>" (esc
     name)))
  else respond_text reqd ~status:`OK ~content_type:ct ~body

(* ---- POST /eval: OCaml handler -> Remote.eval -> Nopales template ---- *)

let handle_eval (rt : Remote.t) (reqd : Httpun.Reqd.t) : unit =
  let body_raw = read_body reqd in
  let expr = form_field body_raw "expr" in
  let result =
    if String.trim expr = "" then "(no input)"
    else
      try
        let v = Remote.eval rt expr in
        Pricklypear_lisp.Reader.pp_value v
      with
      | Ast.Interp_error ie -> Ast.pp_interp_error ie
      | Reader.Reader_error (msg, ln, col) ->
          Printf.sprintf "read error %d:%d %s" ln col msg
      | exn -> Printf.sprintf "error: %s" (Printexc.to_string exn)
  in
  (* Render the result fragment via the Nopales template so it is
     redefinable live; fall back to a plain <pre> if the template is
     unbound or errors. *)
  let html =
    match Remote.call_template ~rt "eval/result" [ Ast.VStr expr; Ast.VStr
     result ] with
    | Some h -> h
    | None ->
```

```
        Printf.sprintf "<div class=\"expr\">$ %s</div><pre>%s</pre>"
          (esc expr) (esc result)
  in
  respond_html reqd ~status:`OK ~body:html

(* ---- chat turn: the shared agent loop on this host (multipart/mixed) ---- *)

type server_state = {
  history : Provider.message list ref;
  complete :
    env:Eio_unix.Stdenv.base ->
    sw:Eio.Switch.t ->
    cfg:Provider.cfg ->
    messages:Provider.message list ->
    tools:Provider.tool_def list ->
    on_delta:(Provider.delta -> unit) ->
    (Provider.reply, string) Result.t;
  rt : Remote.t;
  cfg : Provider.cfg;
}

let is_closed_writer_exn = function
  | Failure msg ->
      String.equal msg "cannot write to closed writer"
      || String.starts_with ~prefix:"cannot write to closed writer" msg
  | _ -> false

let close_writer_quiet (body : Httpun.Body.Writer.t) : unit =
  (try Httpun.Body.Writer.close body
   with exn when is_closed_writer_exn exn -> ())

(* Bail out of the multipart drain loop when the client disconnects. *)
exception Peer_gone

(* One multipart part frame + flush. Mirrors the image's mp_write_part:
   --boundary + Content-Type: text/html + optional HX-Target/HX-Swap +
   blank + html. Returns `Peer_gone on closed writer so the drain loop
   stops cleanly.

   WRITE WATCHDOG: a client that disconnects mid-stream leaves the
   socket in CLOSE-WAIT -- writes keep SUCCEEDING into the kernel send
   buffer until it fills, then the write/flush blocks FOREVER (and on
   some teardowns gluten stops servicing writes without closing the
   Body.Writer, so the flush callback never fires either way). Race
   the whole write path against a 5s sleep: a healthy part write
   completes in milliseconds; a timeout means the connection is gone
   ? Peer_gone. (Same fix in agent/lib/http_ui.ml, which had the same
   latent hang.) *)
let mp_write_part ~(env : Eio_unix.Stdenv.base) (body :
    Httpun.Body.Writer.t)
    ~boundary ?(target : string option) ?(swap : string option)
    (html : string) : [ `Ok | `Peer_gone ] =
  let buf = Buffer.create 128 in
  Buffer.add_string buf ("--" ^ boundary ^ "\r\n");
  Buffer.add_string buf "Content-Type: text/html; charset=utf-8\r\n";
  (match target with Some t -> Buffer.add_string buf ("HX-Target: " ^ t ^
     "\r\n") | None -> ());
  (match swap with Some s -> Buffer.add_string buf ("HX-Swap: " ^ s ^
     "\r\n") | None -> ());
  Buffer.add_string buf "\r\n";
  Buffer.add_string buf html;
  Buffer.add_string buf "\r\n";
  let chunk = Buffer.contents buf in
  let write_and_flush () =
    Httpun.Body.Writer.write_string body chunk;
    let promise, resolver = Eio.Promise.create () in
    let wake () =
      if not (Eio.Promise.is_resolved promise) then Eio.Promise.resolve
    resolver ()
    in
    (try Httpun.Body.Writer.flush body (fun () -> wake ())
     with exn when is_closed_writer_exn exn -> wake ());
```

```
    Eio.Promise.await promise
  in
  try
    (* Race the WHOLE write path (write_string + flush + await): a dead
       peer in CLOSE-WAIT accepts writes until the send buffer fills,
       then the write blocks FOREVER -- a healthy part write completes in
       milliseconds, so a 5s timeout means the connection is gone. *)
    let timed_out =
      Eio.Fiber.any
        [ (fun () -> write_and_flush (); false)
        ; (fun () -> Eio.Time.sleep (Eio.Stdenv.clock env) 5.0; true) ]
    in
    if timed_out then `Peer_gone else `Ok
  with exn when is_closed_writer_exn exn ->
    `Peer_gone

let mp_close (body : Httpun.Body.Writer.t) ~boundary : unit =
  let chunk = "--" ^ boundary ^ "--\r\n" in
  (try Httpun.Body.Writer.write_string body chunk
   with exn when is_closed_writer_exn exn -> ());
  close_writer_quiet body

(* A captured part for hyper/stream replay, in the alist shape
   soup/apply-parts! consumes: (("html" . frag) ("target" . sel) ("swap" .
    strat)). *)
let part_alist ~html ~target ~swap : Ast.value =
  Ast.VList
    [ Ast.VPair (Ast.VStr "html", Ast.VStr html)
    ; Ast.VPair (Ast.VStr "target", Ast.VStr target)
    ; Ast.VPair (Ast.VStr "swap", Ast.VStr swap)
    ]

let handle_chat (env : Eio_unix.Stdenv.base) (sw : Eio.Switch.t)
    (st : server_state) (reqd : Httpun.Reqd.t) : unit =
  ignore sw;
  let body_raw = read_body reqd in
  let prompt = form_field body_raw "msg" in
  let boundary = Printf.sprintf "pp%x%x" (Random.bits ()) (Random.bits ())
    in
  let rid = Printf.sprintf "llm-%d" (Random.bits () land 0xFFFFFF) in
  let headers =
    Httpun.Headers.of_list
      [
        ("content-type", "multipart/mixed; boundary=" ^ boundary);
        ("cache-control", "no-cache");
        ("x-accel-buffering", "no");
        (* Chunked framing is MANDATORY for a streamed response (see
           agent http_ui.ml): without it httpun close-delimits the body
           and a keep-alive client can never detect end-of-body. *)
        ("transfer-encoding", "chunked");
      ]
  in
  let body_w =
    Httpun.Reqd.respond_with_streaming ~flush_headers_immediately:true
      reqd (Httpun.Response.create ~headers `OK)
  in
  (* Captured parts for hyper/stream replay (data). *)
  let parts = ref [] in
  let emit ?target ?swap html =
    let target = match target with Some t -> t | None -> "#log" in
    let swap = match swap with Some s -> s | None -> "beforeend" in
    parts := part_alist ~html ~target ~swap :: !parts;
    match mp_write_part ~env body_w ~boundary ~target ~swap html with
    | `Ok -> ()
    | `Peer_gone -> raise Peer_gone
  in
  (* Render a Nopales fragment template and emit it as a part; false if
     the template is unbound (caller emits its OCaml fallback). *)
  let tpl name args =
    match Remote.call_template ~rt:st.rt name (List.map (fun s -> Ast.VStr
     s) args) with
    | Some html -> emit html; true
    | None -> false
```

```
  in
  let run () =
    (* Openers: user echo bubble + turn skeleton (Nopales templates,
       redefinable live; OCaml fallbacks if unbound). *)
    if not (tpl "chat/user-bubble" [ prompt ]) then
      emit (Printf.sprintf "<div class=\"bubble user\">%s</div>" (esc
     prompt));
    if not (tpl "chat/turn-open" [ rid ]) then
      emit
        (Printf.sprintf
           "<div class=\"turn\"><details class=\"think\"><summary>thinking
     + tools</summary><div id=\"%s-think\"></div></details><div
     class=\"bubble llm\" id=\"%s\"></div></div>"
           rid rid);
    (* Producer fiber: run the agent turn over THIS client's host seam,
       pushing events to a queue. *)
    let queue : Loop.turn_event Queue.t = Queue.create () in
    let qmutex = Eio.Mutex.create () in
    let qcond = Eio.Condition.create () in
    let finished = ref false in
    let push (e : Loop.turn_event) : unit =
      Eio.Mutex.lock qmutex;
      Queue.add e queue;
      Eio.Condition.broadcast qcond;
      Eio.Mutex.unlock qmutex
    in
    let on_event = function
      | Loop.Reasoning_text s -> push (Loop.Reasoning_text s)
      | Loop.Assistant_text s -> push (Loop.Assistant_text s)
      | Loop.Tool_begin (n, i) -> push (Loop.Tool_begin (n, i))
      | Loop.Tool_end (n, o) -> push (Loop.Tool_end (n, o))
      | Loop.Turn_error msg -> push (Loop.Turn_error msg)
    in
    Eio.Switch.run @@ fun lsw ->
    Eio.Fiber.fork ~sw:lsw (fun () ->
        (match
           Loop.run ~env ~sw:lsw ~host:(Remote.loop_host st.rt) ~cfg:st.cfg
             ~complete:st.complete ~on_event st.history prompt
         with
         | Ok () -> ()
         | Error e ->
             (* A failed turn must be VISIBLE in the stream. *)
             push (Loop.Turn_error e));
        Eio.Mutex.lock qmutex;
        finished := true;
        Eio.Condition.broadcast qcond;
        Eio.Mutex.unlock qmutex);
    (* Nested collapsibles in the turn's "thinking + tools" zone
       (#<rid>-think): each contiguous reasoning run its own <details>
       (chat/think-open), each tool call its own (chat/tool-open,
       summary marked done by chat/tool-done). Mirrors the agent's
       drain loop. *)
    let think_zone = "#" ^ rid ^ "-think" in
    let think_n = ref 0 in
    let cur_think : string option ref = ref None in
    let tool_n = ref 0 in
    let open_tools : string list ref = ref [] in
    let open_think_block () : string =
      incr think_n;
      let tid = Printf.sprintf "%s-tk-%d" rid !think_n in
      if not (tpl "chat/think-open" [ tid ]) then
        emit ~target:think_zone
          (Printf.sprintf
             "<details class=\"tk\"><summary>thinking</summary><div
     id=\"%s\"></div></details>"
             tid);
      cur_think := Some tid;
      tid
    in
    let emit_tool_line tid label text =
      if not (tpl "chat/tool-line" [ label; text ]) then
        emit ~target:("#" ^ tid)
          (Printf.sprintf "<div class=\"trace\">%s %s</div>"
```

```
            (esc label) (esc text))
    in
    let rec drain () =
      Eio.Mutex.lock qmutex;
      while Queue.is_empty queue && not !finished do
        Eio.Condition.await qcond qmutex
      done;
      let items = List.of_seq (Queue.to_seq queue) in
      Queue.clear queue;
      let fin = !finished && Queue.is_empty queue in
      Eio.Mutex.unlock qmutex;
      List.iter
        (fun e ->
          match e with
          | Loop.Reasoning_text s ->
              let tid =
                match !cur_think with
                | Some t -> t
                | None -> open_think_block ()
              in
              if not (tpl "chat/think-token" [ s ]) then
                emit ~target:("#" ^ tid)
                  (Printf.sprintf "<span class=\"reason\">%s</span>" (esc
    s))
          | Loop.Assistant_text s ->
              cur_think := None;
              if not (tpl "chat/token" [ s ]) then
                emit ~target:("#" ^ rid)
                  (Printf.sprintf "<span>%s</span>" (esc s))
          | Loop.Tool_begin (n, i) ->
              cur_think := None;
              incr tool_n;
              let tid = Printf.sprintf "%s-tool-%d" rid !tool_n in
              open_tools := tid :: !open_tools;
              if not (tpl "chat/tool-open" [ tid; n ]) then
                emit ~target:think_zone
                  (Printf.sprintf
                     "<details class=\"tool\"><summary id=\"%s-sum\">tool:
    %s</summary><div id=\"%s\"></div></details>"
                     tid (esc n) tid);
              emit_tool_line tid "in" i
          | Loop.Tool_end (n, o) ->
              let tid =
                match !open_tools with
                | t :: rest -> open_tools := rest; t
                | [] -> rid ^ "-think"
              in
              emit_tool_line tid "out" o;
              if tid <> rid ^ "-think" && not (tpl "chat/tool-done" [ n ])
    then
                emit ~target:("#" ^ tid ^ "-sum") ~swap:"innerHTML"
                  (Printf.sprintf "tool: %s &#10003;" (esc n))
          | Loop.Turn_error msg ->
              if not (tpl "chat/error" [ msg ]) then
                emit ~target:("#" ^ rid)
                  (Printf.sprintf
                     "<div class=\"trace\">[turn error: %s]</div>"
                     (esc msg)))
        items;
      if not fin then drain ()
    in
    drain ()
  in
  (try run () with Peer_gone -> ());

  (* Record the captured parts for hyper/stream replay. *)
  st.rt.Remote.last_turn_parts <- Ast.VList (List.rev !parts);
  (try mp_close body_w ~boundary with Peer_gone -> ())


(* ---- serve: bind 127.0.0.1:0, print URL, accept loop ---- *)

let serve ~(env : Eio_unix.Stdenv.base) ~(sw : Eio.Switch.t)
```

```
    ~(rt : Remote.t) ~(cfg : Provider.cfg)
    ~(complete :
        env:Eio_unix.Stdenv.base ->
        sw:Eio.Switch.t ->
        cfg:Provider.cfg ->
        messages:Provider.message list ->
        tools:Provider.tool_def list ->
        on_delta:(Provider.delta -> unit) ->
        (Provider.reply, string) Result.t)
    ?(open_browser = false) (print : string -> unit) : unit =
  let st = { history = ref []; complete; rt; cfg } in
  (* Serialize all HTTP requests through one mutex: the client env is
     single-threaded state (lazy def cache, route table, current_request
     ref). [Remote.dispatch_route] and [Remote.call_template] run under
     the proxy rim; the mutex keeps concurrent requests from interleaving
     on the env / transport. A chat turn holds the mutex for its whole
     lifetime -- a long turn parks other tabs until it finishes (v0
     single-user posture; the agent interleaves via its per-eval lock,
     the client has no eval mutex yet). *)
  let req_mutex = Eio.Mutex.create () in
  let with_mutex f =
    Eio.Mutex.lock req_mutex;
    Fun.protect ~finally:(fun () -> Eio.Mutex.unlock req_mutex) f
  in
  let request_handler (sa : Eio.Net.Sockaddr.stream)
      (g : Httpun.Reqd.t Gluten.reqd) : unit =
    ignore sa;
    let reqd = g.Gluten.reqd in
    let req = Httpun.Reqd.request reqd in
    let target = req.Httpun.Request.target in
    let path, _query =
      match String.index_opt target '?' with
      | Some i ->
          ( String.sub target 0 i,
            Some (String.sub target (i + 1) (String.length target - i - 1))
    )
      | None -> (target, None)
    in
    let fork_work (f : unit -> unit) : unit =
      Eio.Fiber.fork ~sw (fun () ->
          (try f () with
           | Eio.Cancel.Cancelled _ as e -> raise e
           | exn -> Printf.eprintf "[pp-remote-ui] handler error: %s\n%!"
    (Printexc.to_string exn)))
    in
    match req.Httpun.Request.meth with
    | `GET -> (
        if String.length path >= 8 && String.sub path 0 8 = "/assets/" then
          fork_work (fun () ->
              serve_asset reqd (String.sub path 8 (String.length path - 8)))
        else
          fork_work (fun () -> with_mutex (fun () -> dispatch_get rt reqd
    target)))
    | `POST -> (
        match path with
        | "/eval" -> fork_work (fun () -> with_mutex (fun () -> handle_eval
    rt reqd))
        | "/chat" ->
            fork_work (fun () -> with_mutex (fun () -> handle_chat env sw
    st reqd))
        | _ ->
            respond_html reqd ~status:`Not_found
              ~body:(status_page "404" (Printf.sprintf "<p>no route %s</p>"
    (esc path))))
    | _ -> respond_html reqd ~status:`Not_found ~body:(status_page "404"
    "<p>method not supported</p>")
  in
  let error_handler (_sa : Eio.Net.Sockaddr.stream)
      ?(request : Httpun.Request.t option)
      (err : Httpun.Server_connection.error)
      (respond : Httpun.Headers.t -> Httpun.Body.Writer.t) : unit =
    ignore request;
    ignore err;
```

```
    let body = "error" in
    let body_w =
      respond
        (Httpun.Headers.of_list
           [ ("content-type", "text/plain"); ("content-length",
     string_of_int (String.length body)) ])
    in
    Httpun.Body.Writer.write_string body_w body;
    Httpun.Body.Writer.close body_w
  in
  let conn_handler =
    Httpun_eio.Server.create_connection_handler ~request_handler
      ~error_handler ~sw
  in
  let net = Eio.Stdenv.net env in
  let listen_addr = `Tcp (Eio.Net.Ipaddr.V4.loopback, 0) in
  let socket =
    Eio.Net.listen ~reuse_addr:true ~reuse_port:false ~backlog:64 ~sw net
      listen_addr
  in
  let bound_addr = Eio.Net.listening_addr socket in
  let port =
    match bound_addr with `Tcp (_ip, p) -> p | _ -> 0
  in
  let url = Printf.sprintf "http://127.0.0.1:%d/" port in
  print (Printf.sprintf "pp-remote serving %s\n" url);
  if open_browser then (
    try
      let _ = Sys.command (Printf.sprintf "xdg-open '%s' >/dev/null 2>&1 &"
     url) in
      ()
    with _ -> ());
  let on_error ex = ignore ex in
  while true do
    Eio.Net.accept_fork socket ~sw ~on_error (fun client_sock client_addr ->
        try conn_handler client_addr client_sock
        with exn -> ignore exn)
  done
```

288 remote/lib/lens_store.ml

```
(** remote/lib/lens_store.ml -- borge/local-first.borg local-substrate.

    A local lens is a branch of the one tree whose root lives on a client.
    This module is the FILESPUN SUBSTRATE (local-substrate): a directory
    tree of node-envelope JSON files, one file per node, at its path. It
    also pins the two slices the 8.1 make of lens-model + replication-units
    is concretely responsible for:

      - LENS FORK / NAMING (lens-model). A clone forks the user's current
        branch into a client-owned lens via the EXISTING (fork ...) /
        Store.fork_branch (an empty child branch, parent pointer only).
        [fork_lens_name] builds the <owner>/lens-<device>[-<date>] branch
        name that fork is given (e.g. admin/personal -> admin/lens-laptop-
        2026-08-14). The lens is single-writer; the server never writes into
        it except via sync/merge; local writes are commits whose atomicity
        is the substrate's (rename(2)).

      - REPLICATION UNIT (replication-units). [in_replication_unit] decides
        which node paths travel to a client -- exactly the authorized-read
        subtree: data.u.<user>.* (the user's own rows, full), data.shared.*
        (read-mostly seed), b.<lens>.* (the forked code lens within the
        subtree). Inheritance (defs resolving through branch ancestry) is
        READ-TIME resolution -- the client stores only the lens's own
        overrides, not a per-ancestor copy. Nothing outside the path-prefix
        confinement (the one-tree RLS floor) is offered for replication: the
        client cannot ask for, and the server cannot send, a path the
        identity cannot read in a normal session.

    Substrate interface (the spec pins: path-addressed get/put/subtree-list
    + envelope + hash). [materialize] writes a single node JSON under the
    lens root at its path (one file per node); [get]/[subtree_list] are the
    path-addressed reads; [node_hash] is the content hash (the git-style
    "hash over the canonical body" that sync's three-way-diff keys on).
    Every write is atomic (temp file + rename(2)) -- the single-writer
    atomicity the lens gets WITHOUT a second engine (no SQLite, no second
    query engine; the server stays the index).

    This module is pure filesystem + JSON: no Postgres, no Lwt, no network.
    It lives in the client package (remote/) so the offline lens works even
    when the server is unreachable. *)

(** Content hash over a canonical JSON string -- the node-envelope [hash]
    field, the git-style "hash over the canonical body" that sync's
    three-way-diff keys on. Pure digestif (the same digest family the lisp
    package uses); no image-side dependency, so the client computes hashes
    offline. *)
let content_hash (s : string) : string =
  Digestif.SHA256.to_hex (Digestif.SHA256.digest_string s)

(** Whole-file read (standard library; no external File_io). *)
let read_file (file : string) : string =
  let ic = open_in_bin file in
  let n = in_channel_length ic in
  let buf = Bytes.create n in
  really_input ic buf 0 n;
  close_in ic;
  Bytes.unsafe_to_string buf

(** UUID -> ltree-safe label (strip dashes). Mirrors
    image/lib/tree_store.ml [uuid_label] so a client can build a datum
    node path [data.u.<user>.<type>.<id>] from a raw UUID. *)
let uuid_label (u : string) : string =
  String.concat "" (String.split_on_char '-' u)

let enc_label (s : string) : string =
  (* Filesystem- and ltree-safe label encoding, mirrored from
     image/lib/tree_store.ml so a client can reproduce the replication
     path prefix for its own name without round-tripping the server. *)
  if s = "" then "x"
  else begin
    let b64 = Base64.encode_string ~pad:true s in
```

```
    let len = String.length b64 in
    let pad =
      if len >= 2 && String.sub b64 (len - 2) 2 = "==" then 2
      else if len >= 1 && b64.[len - 1] = '=' then 1
      else 0
    in
    let core = if pad > 0 then String.sub b64 0 (len - pad) else b64 in
    let buf = Bytes.of_string core in
    for i = 0 to Bytes.length buf - 1 do
      match Bytes.get buf i with
      | '+' -> Bytes.set buf i '_'
      | '/' -> Bytes.set buf i '-'
      | c -> ignore c
    done;
    Bytes.unsafe_to_string buf ^ string_of_int pad
  end

(** A client materialized lens. [root] is the filesystem directory holding
    the node-envelope tree (local-substrate); [owner]/[device]/[branch]/
    [generation] identify the (branch, generation) snapshot the lens is
    pinned at -- the same migration-017 / gen-now coordinate sync negotiates
    on (last-synced = the merge base). *)
type lens = {
  root : string;
  owner : string;
  device : string;
  branch : string;      (* the forked lens branch id, e.g. the child of
     (fork) *)
  generation : int64;   (* branch_generations / gen-now snapshot *)
}

(** lens-model: the <owner>/lens-<device>[-<date>] fork name a clone mints
    for its lens branch. The existing (fork ...) / Store.fork_branch creates
    the empty child branch; this is the naming that keeps one device's lens
    distinct from another's (star topology, all rendezvousing at the owner's
    server branch). *)
let fork_lens_name ?date ~owner ~device () : string =
  match date with
  | None -> Printf.sprintf "%s/lens-%s" owner device
  | Some d -> Printf.sprintf "%s/lens-%s-%s" owner device d

(** Split a tree node path into its dot-separated segments. *)
let path_segments (path : string) : string list =
  String.split_on_char '.' path

(** replication-units: is [path] inside the replication unit for [user] with
    code lens [lens_prefix]? Exactly the authorized-read subtree:
      data.u.<user>.*   (the user's own rows, full)
      data.shared.*     (read-mostly seed)
      b.<lens_prefix>.* (the forked code lens)
    NOTHING outside it -- another user's data.u.<other>.* never replicates,
    by the same path-prefix confinement that governs reads. *)
let datum_path ~(user : string) ~type_name ~id () : string =
  Printf.sprintf "data.u.%s.%s.%s" (enc_label user) (enc_label type_name)
    (uuid_label id)

let in_replication_unit ?(lens_prefix : string = "") ~user (path : string)
    : bool =
  match path_segments path with
  | "data" :: "u" :: u_seg :: _ -> u_seg = enc_label user
  | "data" :: "shared" :: _ -> true
  | "b" :: branch_uu :: _ ->
      String.length branch_uu = 32 && lens_prefix <> ""
      && branch_uu = lens_prefix
  | _ -> false

(** Map a tree node path to its file path under the lens root: one file per
    node, at its path, as a nested directory tree (the pp-sync / filetree
    format, inspectable with cat / jq / grep / diff / git). E.g.
      b.<uuid>.fn.<name>       -> b/<uuid>/fn/<name>.json
      data.u.admin.todo.<id>   -> data/u/admin/todo/<id>.json *)
let path_to_file ~(root : string) (path : string) : string =
  match path_segments path with
```

```
  | [] -> Filename.concat root "__root.json"
  | segs ->
      let dirs, leaf =
        match List.rev segs with
        | last :: rest -> (List.rev rest, last)
        | [] -> ([], "")
      in
      let dir = Filename.concat root (String.concat Filename.dir_sep dirs)
     in
      Filename.concat dir (leaf ^ ".json")

(** mkdir_p for the parent directory of [file]. *)
let mkdir_p_parent (file : string) : unit =
  let dir = Filename.dirname file in
  let rec mkdir d =
    if d = "" || d = "." || d = Filename.dir_sep then ()
    else begin
      if not (Sys.file_exists d) then begin
        mkdir (Filename.dirname d);
        (try Unix.mkdir d 0o755 with
         | Unix.Unix_error (Unix.EEXIST, _, _) -> ())
      end
    end
  in
  mkdir dir

(** Atomic write: write [contents] to a temp file in the same directory then
    rename(2) over [file]. This is the single-writer atomicity the lens
    substrate provides WITHOUT a concurrency engine. *)
let atomic_write (file : string) (contents : string) : unit =
  mkdir_p_parent file;
  let tmp = Filename.concat (Filename.dirname file) (".tmp-" ^
    string_of_int (Unix.getpid ())) in
  let fd = Unix.openfile tmp [ Unix.O_WRONLY; Unix.O_CREAT; Unix.O_TRUNC ]
    0o600 in
  (try
     let buf = Bytes.of_string contents in
     let len = Bytes.length buf in
     let written = ref 0 in
     while !written < len do
       let n = Unix.write fd buf !written (len - !written) in
       written := !written + n
     done;
     (* fsync before rename so a crash never leaves a truncated envelope
        at the leaf path *)
     Unix.fsync fd;
     Unix.close fd
   with exn ->
     (try Unix.close fd with _ -> ());
     (try Sys.remove tmp with _ -> ());
     raise exn);
  Unix.rename tmp file

(** Extract the [path] field from a node-envelope JSON, or None. *)
let node_path_of_json (node_json : string) : string option =
  match Yojson.Safe.from_string node_json with
  | `Assoc kv -> (match List.assoc_opt "path" kv with
      | Some (`String p) -> Some p | _ -> None)
  | _ -> None

(** materialize: write one node envelope under the lens root at its path.
    Returns the written file path. The node JSON is the server-side envelope
    ({v,kind,path,type,body,author,doc,hash,created_at,updated_at} -- the
    tree_store.node_to_json surface); the file IS that envelope verbatim. *)
let materialize ~(lens : lens) (node_json : string) : string option =
  match node_path_of_json node_json with
  | None -> None
  | Some path ->
      let file = path_to_file ~root:lens.root path in
      atomic_write file node_json;
      Some file

(** put_body: build a node envelope from a canonical body and write it at
```

```
    [path]. This is the substrate write path a local edit uses while the
    lens is offline (single-writer; no server round trip). The hash is the
    content hash over the canonical body (the sync/three-way-diff key). *)
let put_body ~(lens : lens) ~path ~kind ~type_ ~author ~(body :
     Yojson.Safe.t)
    : unit =
  let hash = content_hash (Yojson.Safe.to_string body) in
  let env =
    Yojson.Safe.to_string
      (`Assoc
         [ ("v", `Int 1)
         ; ("kind", `String kind)
         ; ("path", `String path)
         ; ("type", `String type_)
         ; ("body", body)
         ; ("author", `String author)
         ; ("doc", `Null)
         ; ("hash", `String hash)
         ; ("created_at", `String (string_of_float (Unix.gettimeofday ())))
         ; ("updated_at", `String (string_of_float (Unix.gettimeofday ())))
         ])
  in
  let file = path_to_file ~root:lens.root path in
  atomic_write file env

(** get: read the node envelope at [path] (None if absent). *)
let get ~(lens : lens) (path : string) : string option =
  let file = path_to_file ~root:lens.root path in
  if Sys.file_exists file then
    (try Some (read_file file) with _ -> None)
  else None

(** subtree_list: all node envelope file paths under the lens root whose
    tree path begins with [prefix] (readdir/find -- the filesystem IS the
    path index; no second query engine). Returns tree paths. *)
let subtree_list ~(lens : lens) (prefix : string) : string list =
  let prefix_file = path_to_file ~root:lens.root prefix in
  let prefix_dir = Filename.dirname prefix_file in
  let rec walk dir acc =
    if not (Sys.file_exists dir) then acc
    else begin
      try
        Sys.readdir dir
        |> Array.fold_left (fun acc name ->
             let full = Filename.concat dir name in
             if Sys.is_directory full then walk full acc
             else if Filename.check_suffix name ".json" then full :: acc
             else acc)
             acc
      with _ -> acc
    end
  in
  let files = walk prefix_dir [] in
  List.map
    (fun f ->
      match node_path_of_json (read_file f) with
      | Some p -> p
      | None -> "")
    files
  |> List.filter (fun p -> p <> "" && String.length p >= String.length
     prefix
                         && String.sub p 0 (String.length prefix) = prefix)

(** node_hash: the content hash for the node envelope at [path] -- the
    {v,kind,path,type,body,author,doc,hash} envelope hash field, which
    sync's three-way-diff keys on. *)
let node_hash ~(lens : lens) (path : string) : string option =
  match get ~lens path with
  | None -> None
  | Some json ->
      (match Yojson.Safe.from_string json with
       | `Assoc kv ->
           (match List.assoc_opt "hash" kv with
```

```
            | Some (`String h) -> Some h | _ -> None)
       | _ -> None)

(* ========================================================================
   CODE-LENS + ANCESTRY MATERIALIZATION (local-substrate, 8.2)

   The replication-unit manifest (replication-units subsection) names the
   forked code lens b.<lens>.{fn,type,route,pkg}.* PLUS the branch
   ancestry the lens resolves through (defs resolve through inheritance
   per one-tree tree-model read-time resolution). These helpers complete
   the substrate's coverage of that manifest: the client can materialize
   the forked code lens + record the ancestry chain it resolves through,
   then resolve ANY definition (offline) by walking the chain
   self-first with the same closest-ancestor-wins rule a server-side
   eval of the lens would use (store.branch_chain self-first).

   One file per code node at its b.<branch>.<kind>.<name> tree path,
   exactly like a datum node -- the filesystem stays the only index, and
   sync (3-way tree-diff) keys on the same envelope [hash].
   ======================================================================== *)

(** The code-node kinds the replication unit manifests (fn/type/route/pkg).
    Any other kind is not a code-lens subtree member. *)
let code_kind_path ~branch (kind : string) ~(name : string) : string option
    =
  match kind with
  | "fn" | "type" | "route" | "pkg" ->
      Some (Printf.sprintf "b.%s.%s.%s" branch kind name)
  | _ -> None

(** Materialize one code-node envelope at its b.<branch>.<kind>.<name> tree
    path. [src] is the evaluable definition source (a toplevel (define ?)
    form -- the same string a server-side eval would build); [doc] the
    docstring. The envelope follows the one-tree tree-model shape; [hash]
    is the content hash over the canonical body.

    Returns the written file path, or None when [kind] is not a code kind.
    *)
let materialize_code_node ~(lens : lens) ~branch ~kind ~(name : string)
    ~(src : string) ~(doc : string) ~(author : string)
    : string option =
  match code_kind_path ~branch kind ~name with
  | None -> None
  | Some path ->
      let body =
        `Assoc [ ("src", `String src); ("doc", `String doc) ]
      in
      let hash = content_hash (Yojson.Safe.to_string body) in
      let env =
        Yojson.Safe.to_string
          (`Assoc
             [ ("v", `Int 1)
             ; ("kind", `String kind)
             ; ("path", `String path)
             ; ("type", `String "code")
             ; ("body", body)
             ; ("author", `String author)
             ; ("doc", `String doc)
             ; ("hash", `String hash)
             ; ("created_at", `String (string_of_float (Unix.gettimeofday
    ())))
             ; ("updated_at", `String (string_of_float (Unix.gettimeofday
    ())))
             ])
      in
      let file = path_to_file ~root:lens.root path in
      atomic_write file env;
      Some file

(** Extract the evaluable definition [src] from a code-node envelope which
    carries {body:{src,?}} (the [materialize_code_node] shape). Returns
    None when the envelope is absent or not a code node with a src body. *)
let code_src_of_envelope (envelope_json : string) : string option =
```

```
  match Yojson.Safe.from_string envelope_json with
  | `Assoc kv -> (
      match List.assoc_opt "body" kv with
      | Some (`Assoc bkv) -> (
          match List.assoc_opt "src" bkv with
          | Some (`String src) -> Some src | _ -> None)
      | _ -> None)
  | _ -> None

(** Extract the [body] of a DATUM node envelope as a JSON string -- the
    shape [(get 'type id)] returns (the row's fields). Returns None when
    the envelope is absent or has no body. This is the lens-local
    equivalent of the server's [Crud.get]: the datum node's [body] IS the
    row's fields JSON. *)
let datum_body_of_envelope (envelope_json : string) : string option =
  match Yojson.Safe.from_string envelope_json with
  | `Assoc kv ->
      (match List.assoc_opt "body" kv with
       | Some body -> Some (Yojson.Safe.to_string body)
       | None -> None)
  | _ -> None

(** The branch ancestry chain metadata file under the lens root. The chain
    is ordered SELF-FIRST (closest-wins): [lens, parent, ?, system/main],
    the same ordering store.branch_chain / function_defs override use. *)
let ancestry_file (root : string) : string =
  Filename.concat root "__ancestry.json"

(** Record the ancestry chain a lens resolves defs through. Self-first
    (closest-wins), oldest ancestor last. Atomic (rename(2)), like every
    other substrate write. *)
let record_ancestry ~(lens : lens) (chain : string list) : unit =
  let json =
    Yojson.Safe.to_string
      (`Assoc
         [ ("chain", `List (List.map (fun b -> `String b) chain))
         ; ("recorded_at", `String (string_of_float (Unix.gettimeofday ())))
         ])
  in
  atomic_write (ancestry_file lens.root) json

(** Read back the recorded ancestry chain (self-first). Empty when none. *)
let ancestry ~(lens : lens) : string list =
  let f = ancestry_file lens.root in
  if not (Sys.file_exists f) then []
  else
    (try
       match Yojson.Safe.from_string (read_file f) with
       | `Assoc kv -> (
           match List.assoc_opt "chain" kv with
           | Some (`List l) ->
               List.filter_map (function `String b -> Some b | _ -> None) l
           | _ -> [])
       | _ -> []
     with _ -> [])

(** resolve_def: find the def (kind, name) in the substrate by walking the
    ancestry chain SELF-FIRST (closest-ancestor-wins -- the same override
    rule a server-side eval of the lens would use), returning the envelope
    JSON of the closest b.<branch>.<kind>.<name> that exists in the
    substrate, or None. Pure / offline: reads the filesystem only; it is
    the read-time inheritance resolution of one-tree tree-model, without a
    server round trip. ([chain] defaults to the lens's recorded ancestry.)
    *)
(** resolve_def over an EXPLICIT chain (self-first). The caller controls
    the chain (e.g. a freshly forked lens whose ancestry it just recorded,
    to avoid re-touching the metadata file). *)
let resolve_def_chain ~(lens : lens) (chain : string list) ~kind
    ~(name : string) : string option =
  List.find_map
    (fun branch ->
      match code_kind_path ~branch kind ~name with
      | None -> None
```

```
      | Some path -> get ~lens path)
    chain

(** resolve_def: find the def (kind, name) in the substrate by walking the
    lens's RECORDED ancestry chain SELF-FIRST (closest-ancestor-wins -- the
    same override rule a server-side eval of the lens would use), returning
    the envelope JSON of the closest b.<branch>.<kind>.<name> that exists in
    the substrate, or None. Pure / offline: reads the filesystem only; it is
    the read-time inheritance resolution of one-tree tree-model, without a
    server round trip. *)
let resolve_def ~(lens : lens) ~kind ~(name : string) : string option =
  resolve_def_chain ~lens (ancestry ~lens) ~kind ~name

(** The subtree paths the replication unit names, materialized under a
    lens root, in a form the server manifest can diff against: every code
    node present in the substrate (its b.<branch>.<kind>.<name> tree
    paths) plus every datum node, each with its envelope hash. This is the
    (path, hash) manifest a sync round negotiates on (see the sync
    stanza). Returns [(path, hash)] pairs for every .json node envelope
    under the lens root (excluding the ancestry metadata file). *)
let node_manifest ~(lens : lens) : (string * string) list =
  let prefix = "" in
  let prefix_file = path_to_file ~root:lens.root prefix in
  let prefix_dir = Filename.dirname prefix_file in
  let rec walk dir acc =
    if not (Sys.file_exists dir) then acc
    else begin
      try
        Sys.readdir dir
        |> Array.fold_left (fun acc name ->
             let full = Filename.concat dir name in
             if Sys.is_directory full then walk full acc
             else if Filename.check_suffix name ".json" then full :: acc
             else acc)
             acc
      with _ -> acc
    end
  in
  (* hash of an arbitrary node-envelope FILE (extract path + hash) *)
  let path_hash_of_file f : (string * string) option =
    match node_path_of_json (read_file f) with
    | None -> None
    | Some p ->
        let h =
          match Yojson.Safe.from_string (read_file f) with
          | `Assoc kv -> (
              match List.assoc_opt "hash" kv with
              | Some (`String h) -> h | _ -> "")
          | _ -> ""
        in
        Some (p, h)
  in
  let files =
    walk prefix_dir []
    |> List.filter (fun f ->
         let b = Filename.basename f in
         b <> "__root.json" && b <> "__ancestry.json" && b <>
    "__snapshot.json"
         && b <> "__base.json")
  in
  List.filter_map path_hash_of_file files

(** delete_node: remove the node envelope at [path] (the Ours_del mirror:
    the server deleted the node, the client's stale copy goes too).
    Best-effort empty-parent cleanup; returns true when a file was
    removed. *)
let delete_node ~(lens : lens) (path : string) : bool =
  let file = path_to_file ~root:lens.root path in
  if Sys.file_exists file then begin
    (try Sys.remove file with _ -> ());
    (* prune now-empty parent dirs up to the lens root (cosmetic; the
       filesystem index regenerates on demand) *)
    let rec prune d =
```

```
      let root = Filename.concat lens.root "" in
      if d <> lens.root && String.length d > String.length root then begin
        (try if Sys.readdir d |> Array.length = 0 then Sys.rmdir d with _
     -> ());
        prune (Filename.dirname d)
      end
    in
    prune (Filename.dirname file);
    true
  end
  else false

(* ================================================================
   BASE MANIFEST (sync's common ancestor, client side)

   The (path, hash) manifest the server returned at the client's last
   successful sync round (merged_manifest). Recorded to __base.json and
   offered as the OPTIONAL third argument of the next sync/push so the
   server runs the full three-way plan: clean client edits apply,
   client DELETIONS propagate (theirs_del), server deletions come back
   as client_delete_paths (Ours_del), conflicts LWW-arbitrate. Without
   a recorded base the round degrades to two-way (additions + LWW only)
   -- the exact behavior before a first sync ever ran.
   ================================================================ *)

let base_file (root : string) : string =
  Filename.concat root "__base.json"

(** Record the last-synced (merged) server manifest as the next round's
    base. Atomic (rename(2)) like every substrate write. *)
let record_base_manifest ~(lens : lens) (m : (string * string) list) : unit
    =
  let json =
    Yojson.Safe.to_string
      (`Assoc
         [ ( "paths"
           , `List
               (List.map
                  (fun (p, h) -> `List [ `String p; `String h ])
                  m)
           )
         ; ("recorded_at", `String (string_of_float (Unix.gettimeofday ())))
         ])
  in
  atomic_write (base_file lens.root) json

(** Read back the recorded base manifest (empty when none was ever
    recorded -- a fresh lens syncs two-way first). *)
let base_manifest ~(lens : lens) : (string * string) list =
  let f = base_file lens.root in
  if not (Sys.file_exists f) then []
  else
    (try
       match Yojson.Safe.from_string (read_file f) with
       | `Assoc kv -> (
           match List.assoc_opt "paths" kv with
           | Some (`List rows) ->
               List.filter_map
                 (function
                   | `List [ `String p; `String h ] -> Some (p, h)
                   | _ -> None)
                 rows
           | _ -> [])
       | _ -> []
     with _ -> [])

(* ================================================================
   SNAPSHOT COORDINATE (sync's common ancestor)

   The (branch, generation) the lens was last synced at -- the merge
   base a sync round's three-way diff keys on (local-first.borg sync:
   "last-synced as the common ancestor"). Stamped at pull time; read back
   to offer as the base manifest coordinate. One file under the lens
```

```
   root; atomic (rename(2)) like every substrate write. *)

let snapshot_file (root : string) : string =
  Filename.concat root "__snapshot.json"

(** Record the (branch, generation) snapshot the lens was materialized
    at. [generation] is the migration-017 / gen-now coordinate; [branch]
    is the checked-out branch name. Both come from the server at pull
    time (proxied read prims), so a later sync can offer this snapshot
    as the three-way merge base. *)
let record_snapshot ~(lens : lens) ~branch ~(generation : int64) : unit =
  let json =
    Yojson.Safe.to_string
      (`Assoc
         [ ("branch", `String branch)
         ; ("generation", `String (Int64.to_string generation))
         ; ("pulled_at", `String (string_of_float (Unix.gettimeofday ())))
         ])
  in
  atomic_write (snapshot_file lens.root) json

(** Read back the recorded (branch, generation) snapshot, or None. *)
let snapshot ~(lens : lens) : (string * int64) option =
  let f = snapshot_file lens.root in
  if not (Sys.file_exists f) then None
  else
    (try
       match Yojson.Safe.from_string (read_file f) with
       | `Assoc kv ->
           (match List.assoc_opt "branch" kv, List.assoc_opt "generation"
    kv with
            | Some (`String b), Some g ->
                let gen =
                  match g with
                  | `String s -> (try Int64.of_string s with _ -> 0L)
                  | `Int i -> Int64.of_int i
                  | _ -> 0L
                in
                Some (b, gen)
            | _ -> None)
       | _ -> None
     with _ -> None)
```

289 remote/lib/placer.ml

```
(** remote/lib/placer.ml (borge/remote-eval.borg placement).

    Hybrid placement: decide, per expression, whether to evaluate
    LOCALLY (under the proxy rim -- pure compute stays local, prims
    proxy individually) or to POST the expression REMOTE (/api/eval
    wholesale -- one round trip for the whole thing).

    Rules (correctness first, then latency):
      1. (with-transaction ?) ? REMOTE, hard. A transaction holds
         (select-for-update ?) locks across prims in its body; locks
         cannot span HTTP calls. Relaxing this corrupts the store.
      2. prim-call count above a configurable threshold ? REMOTE, to
         avoid N prim round trips where one eval round trip suffices.
      3. zero prim calls ? LOCAL (pure compute, no wire).
      4. otherwise ? LOCAL (proxy prims individually; the common case
         for definitional/interactive work).

    The decision is visible: [placement] carries a human-readable
    reason so the user can see WHY an expr went remote (no silent
    boundary). The walker is conservative: it counts apparent prim
    calls by AST shape (App of Var name where name is not a known
    special form / local pure prim definition site). It does NOT eval,
    so a name that resolves to a lambda is still counted -- that's the
    right bias for a latency heuristic (over-counting pushes remote,
    which is the safe direction). *)

open Pricklypear_lisp
open Ast

type placement = Local | Remote of string * string (* (where, reason) *)

let pp_placement = function
  | Local -> "local"
  | Remote (where, why) -> Printf.sprintf "remote(%s): %s" where why

(** Names that are NOT prim calls even though they appear as App head:
    special forms and local pure-primitive definitions. The placer
    receives the local pure prim names + the special-form set so it
    doesn't miscount a (define ?) or a (lambda ?) as a prim call. *)
type cfg = {
  threshold : int;           (* prim-call count above which ? REMOTE *)
  local_prims : string list; (* names handled locally (pure prims) *)
  special_forms : string list;
}

let default_threshold = 3

let default_cfg ~local_prims =
  { threshold = default_threshold; local_prims;
    special_forms =
      [ "lambda"; "quote"; "if"; "define"; "set!"; "begin"; "let";
        "let*"; "letrec"; "cond"; "when"; "unless"; "and"; "or";
        "case"; "match"; "guard"; "with-transaction"; "delay"; "force" ] }

let is_special cfg name = List.mem name cfg.special_forms
let is_local_prim cfg name = List.mem name cfg.local_prims

(** Walk [e], returning (has_transaction, prim_call_count). A prim call
    is an [App (Var name, _)] where name is not a special form and not a
    known local pure prim -- i.e. a name that would proxy. Local pure
    prims stay local by definition and don't count toward the
    proxy-round-trip axis. *)
let rec walk cfg (e : expr) : bool * int =
  match e with
  | Var _ | Lit _ -> (false, 0)
  | Quote _ -> (false, 0)
  | Define (_, body) -> walk cfg body
  | Lambda (_, body) -> walk cfg body
  | Let (_, bound, body) ->
      let hb, cb = walk cfg bound in
      let hb2, cb2 = walk cfg body in
```

```
      (hb || hb2, cb + cb2)
  | Letrec (binds, body) ->
      let hb, cb =
        List.fold_left
          (fun (h, c) (_, e) ->
            let h2, c2 = walk cfg e in (h || h2, c + c2))
          (false, 0) binds
      in
      let hb2, cb2 = walk cfg body in
      (hb || hb2, cb + cb2)
  | If (a, b, c) ->
      let h1, c1 = walk cfg a in
      let h2, c2 = walk cfg b in
      let h3, c3 = walk cfg c in
      (h1 || h2 || h3, c1 + c2 + c3)
  | Prim (name, args) ->
      (* Prim node: a prim call the reader already resolved. Treat
         like an App of Var name. with-transaction is usually an App
         (special form), but be safe. *)
      let body_ht, body_count =
        List.fold_left
          (fun (h, c) a ->
            let h2, c2 = walk cfg a in (h || h2, c + c2))
          (false, 0) args
      in
      let self_count =
        if is_special cfg name || is_local_prim cfg name then 0 else 1
      in
      let is_tx = (name = "with-transaction") in
      (is_tx || body_ht, body_count + self_count)
  | Seq (a, b) ->
      let ht, ca = walk cfg a in
      let ht2, cb = walk cfg b in
      (ht || ht2, ca + cb)
  | App (Var name, args) when name = "with-transaction" ->
      let ht, cb =
        List.fold_left
          (fun (h, c) a ->
            let h2, c2 = walk cfg a in (h || h2, c + c2))
          (true, 0) args
      in
      (ht, cb)
  | App (Var name, args) ->
      let body_ht, body_count =
        List.fold_left
          (fun (h, c) a ->
            let h2, c2 = walk cfg a in (h || h2, c + c2))
          (false, 0) args
      in
      let self_count =
        if is_special cfg name || is_local_prim cfg name then 0 else 1
      in
      (body_ht, body_count + self_count)
  | App (f, args) ->
      let htf, cf = walk cfg f in
      let ht, ca =
        List.fold_left
          (fun (h, c) a ->
            let h2, c2 = walk cfg a in (h || h2, c + c2))
          (false, 0) args
      in
      (htf || ht, cf + ca)

(** Decide placement for [src]. Parses with the shared reader; a parse
    error is REMOTE (let the server report it consistently with
    /api/eval). *)
let place ~(cfg : cfg) (src : string) : placement =
  match Reader.read_all src with
  | exception _ ->
      (* parse error: send to server; it reports the error shape *)
      Remote ("/api/eval", "parse error; let server report")
  | forms ->
      let has_tx, prim_count =
```

```
        List.fold_left
          (fun (h, c) f ->
            let h2, c2 = walk cfg f in (h || h2, c + c2))
          (false, 0) forms
      in
      if has_tx then
        Remote ("/api/eval", "with-transaction: locks cannot span HTTP")
      else if prim_count = 0 then Local
      else if prim_count > cfg.threshold then
        Remote
          ( "/api/eval",
            Printf.sprintf
              "prim-dense (%d > threshold %d): one eval round trip beats %d
    prim round trips"
              prim_count cfg.threshold prim_count )
      else Local
```

290 remote/lib/proxy_rim.ml

```
(** remote/lib/proxy_rim.ml (borge/remote-eval.borg client).

    A second rim for the SAME [Control.Prim_call] seam the image rim
    services (lisp/lib/control.ml, [Control.run_with_rim]). Where the
    image services [Prim_call(name, args)] from the in-process store
    prim table, the proxy rim forwards every prim call that is not a
    local pure prim to the server via [transport] (POST /api/prim) and
    [continue]s with the deserialized value -- or [discontinue]s with
    the re-raised exception (reconstructed by Prims_pure.error_of_json)
    so a remote client observes identical exceptions to an in-process
    eval.

    PROXY DISCIPLINE (spec): the rim is a PURE proxy with zero
    interpretation logic. The only allowed local behavior is "dispatch
    to a local pure prim table" or "forward to /api/prim". Nothing in
    between -- the moment it grows its own meaning for a prim there are
    two dialects.

    NOTE on Control.direct_prims: the image sets it so nested eval
    inside a prim body dispatches by direct table lookup (its table is
    TOTAL there). Here the local table (pure prims only) is
    INTENTIONALLY partial, so setting direct_prims to it would make
    every DB/other prim raise "unregistered primitive" instead of
    proxying. We therefore leave direct_prims empty and let the
    Effect.Deep handler intercept EVERY [Prim_call]; the local/proxy
    split happens in the handler, not in direct_prims. This is safe
    because no local pure prim performs a nested eval from within a
    handler body (the case direct_prims exists to serve).

    The [transport] boundary is what makes the rim unit-testable: the
    Eio HTTP transport (remote/lib/http.ml) is swapped for a mock in
    the tests, so no live server is required. *)

open Pricklypear_lisp
open Ast
module Effect = Stdlib.Effect

(** name -> args -> (value | exn). The production transport POSTs to
    /api/prim and uses the shared value/error codecs; a test transport
    records calls and returns canned values. *)
type transport = string -> value list -> (value, exn) Result.t

(** Run [thunk] under the proxy rim. Prim calls resolvable in
    [local_prims] are serviced locally (pure compute stays local, zero
    HTTP); everything else forwards to [transport] and, on error,
    re-raises the SAME exn the local rim would. *)
let run ~(local_prims : Eval.prim_table) ~(transport : transport)
    (thunk : unit -> 'a) : 'a =
  let open Effect.Deep in
  match_with thunk ()
    { retc = (fun v -> v)
    ; exnc = (fun e -> raise e)
    ; effc =
        (fun (type a) (eff : a Effect.t) ->
          match eff with
          | Control.Prim_call (name, args) ->
              Some
                (fun (k : (a, _) continuation) ->
                  match Hashtbl.find_opt local_prims name with
                  | Some fn ->
                      (* Eval the prim body, then resume [k] once with the
                         result. A prim-body exception propagates to [exnc]
                         (k is never resumed); a CONTINUATION exception
    (after
                         [continue k v] resumes) also propagates to [exnc].
                         Wrapping [continue k (fn args)] in a try would
    catch a
                         continuation-raised exn and [discontinue] an
                         already-resumed [k]
    (Continuation_already_resumed). *)
                      continue k (fn args)
```

```
                | None ->
                    (* The transport may RAISE (Eio HTTP raises
                       [Eio.Io] on a refused connection, it does not
                       return [Error]); catch both so a transport
                       failure becomes a catchable Lisp [Prim_error]
                       instead of crashing the client. This is the
                       offline-degradation seam: an indexed prim ([rows],
                       [tree/find-by], ?) that the lens does not
                       service locally raises a Lisp error the REPL
                       prints, not a process-killing [Eio.Io]. *)
                    (match (try transport name args with exn -> Error
    exn) with
                     | Ok v -> continue k v
                     | Error exn ->
                         let msg =
                           Printf.sprintf "%s: transport failure (%s)"
                             name (Printexc.to_string exn)
                         in
                         discontinue k
                           (Ast.Interp_error
                              (Ast.Prim_error (name, msg, "transport")))))
          | _ -> None)
    }
```

291 remote/lib/remote.ml

```
(** remote/lib/remote.ml (borge/remote-eval.borg client).

    The client host: a pure-lisp env + a small LOCAL table of pure
    prims (shared Prims_pure code, so semantics are identical to the
    image's) + a [Proxy_rim.transport] to the image. All evaluation
    runs under the proxy rim; pure compute stays local, everything
    else proxies.

    LAZY ENV (spec): function resolution is data (the server's CTE
    over function_defs). The client materializes defs on first
    reference instead of at boot: an [Interp_error (Unbound_var name)]
    during eval triggers a fetch of the def source (via the
    [function-source] prim, proxied) and evals it into the env under
    the proxy rim, then retries the expression once. The env is a
    CACHE of definitions -- never a source of truth.

    IDENTITY: the client passes basic-auth per prim call through the
    transport. It never claims an author it does not hold; identity is
    server-asserted (whoami/current-author come back from the server). *)

open Pricklypear_lisp
open Ast

type transport = Proxy_rim.transport

type t = {
  env : Ast.env;
  local_prims : Eval.prim_table;
  transport : transport;
  (* definitions materialized locally, keyed (branch, generation): the
     def cache (caching subsection). Bounds server fetches on a miss and
     invalidates on generation change / branch switch. *)
  def_cache : Def_cache.t;
  (* PURE prim names bound locally (via [add_prim]): the placer treats
     these as NOT crossing the wire, so they do not count toward the
     prim-density heuristic. Pure compute stays local regardless. *)
  mutable pure_prims : string list;
  (* Whole-expression remote eval (POST /api/eval) for Remote
     placements. Some = hybrid placement is active; None = no remote
     eval path, so a Remote placement raises rather than silently
     running locally (which would break transaction correctness). *)
  remote_eval : (string -> value) option;
  (* OPTIONAL local lens substrate (borge/local-first.borg local-
     substrate): a materialized lens root + ancestry chain, used as the
     OFFLINE-FIRST def source. When set, [fetch_def] consults it before
     proxying [function-source], so defs in the replication unit resolve
     locally (the client evaluates against the local substrate; the
     server is only the merge/index authority). None = no lens wired,
     defs always proxy. *)
  lens_lookup : (string -> string option) option;
  (* The lens record itself, when wired. Set by the host after [make]
     (pp-remote --lens) so the lens/list + lens/read surface prims can
     browse the substrate. None = no lens; the prims return empty. *)
  mutable lens : Lens_store.lens option;
  (* ---- Route table + request context (agent-route-table host seam,
     ported to the client so the Nopales-authored surface runs on
     pp-remote unchanged -- borge/remote-eval.borg surfaces). Local
     hashtbl backing for (route path handler); identical Nopales
     signatures to the image's route prims so libs/hyper runs
     unchanged. Env-only: no host-prim dependency, no rim coupling
     beyond the proxy rim the rest of the client already uses. *)
  routes : (string, string) Hashtbl.t;
  current_request : request_shape option ref;
  mutable last_turn_parts : Ast.value;
  (* ---- agent-loop host seam (borge/remote-eval.borg agent-loop) ----
     The shared LLM turn loop (llm/lib/loop.ml) drives the client
     through a small [Loop.host] record; these fields are its backing:
     a LOCAL tool registry (builtin read/bash + Nopales register-tool
     plugins) and event handlers ((on "tool_call" ?) hooks). All
     env+hashtbl ops -- same shape as the agent's Runtime.t surface. *)
  tools : (string, Loop.registry_entry) Hashtbl.t;
```

```
  mutable tool_order : string list;
  handlers : (string, (Ast.value list -> Ast.value) list ref) Hashtbl.t;
}

(** Thin request context bound during route/dispatch (agent-route-table
    host seam). Carries ONLY request-shape values: method, path, params,
    headers, body -- the same shape the agent binds, so libs/hyper's
    query-param / request-method / hx-request? resolve identically. *)
and request_shape = {
  method_ : string;
  path : string;
  params : (string * string) list;
  headers : (string * string) list;  (* lowercased keys *)
  body : string;
}

let value_to_json = Prims_pure.value_to_json
let json_to_value = Prims_pure.json_to_value

(** Register a prim BOTH in the dispatch table and as a VPrim binding
    in the env (Nopales resolves prims via the env, like the image
    session binds its catalog). *)
let add_prim (t : t) (name : string) (fn : value list -> value) : unit =
  Eval.register_prim t.local_prims name fn;
  Env.define t.env name (VPrim name);
  (* track the pure/local prim for the placer (it stays local, so it
     must NOT count toward the prim-density threshold) *)
  t.pure_prims <- name :: t.pure_prims

(** The [Placer.cfg] for this client: the local pure prims are the
    names the placer must NOT count as proxy-round-trips. The rest --
    proxyable prims and user-function references -- walk as prim calls. *)
let placer_cfg (t : t) : Placer.cfg =
  Placer.default_cfg ~local_prims:(List.rev t.pure_prims)

(** Bind a prim as VPrim in the env WITHOUT a local handler. Calling it
    performs [Control.Prim_call], which the proxy rim forwards to
    /api/prim (it is absent from [local_prims]). This is how server
    prims (get/create/rows/?) reach the wire: they must be bound as
    VPrim so eval performs Prim_call, but must NOT be in the local
    table or they'd be serviced locally. Server-only prims
    (route/fork/merge/?) are intentionally NOT bound here -- the server
    refuses them with a `server_only` error, and leaving them unbound
    surfaces that locally as Unbound_var rather than a round-trip refusal.
     *)
let bind_proxy_prim (t : t) (name : string) : unit =
  Env.define t.env name (VPrim name)

(** Register the frozen pure prim surface locally. Same code as the
    image links (Prims_pure), so local semantics are identical -- the
    proxy rim's local table is the "pure compute stays local" half of
    hybrid placement. Options that keep a prim OUT of this table
    simply proxy it; nothing depends on the split being exhaustive. *)
let register_pure_prims (t : t) : unit =
  let open Prims_pure in
  let r n f = add_prim t n f in
  r "+" prim_add;
  r "-" prim_sub;
  r "*" prim_mul;
  r "/" prim_div;
  r "div" prim_div;
  r "mod" prim_mod;
  r "quotient" prim_quotient;
  r "remainder" prim_remainder;
  r "abs" prim_abs;
  r "min" prim_min;
  r "max" prim_max;
  r "=" prim_num_eq;
  r "<" prim_lt;
  r ">" prim_gt;
  r "<=" prim_le;
  r ">=" prim_ge;
  r "not" prim_not;
```

```
  r "list" prim_list;
  r "car" prim_car;
  r "cdr" prim_cdr;
  r "cons" prim_cons;
  r "null?" prim_null_p;
  r "pair?" prim_pair_p;
  r "append" prim_append;
  r "reverse" prim_reverse;
  r "nth" prim_nth;
  r "count" prim_count;
  r "length" prim_length;
  r "string-append" prim_string_append;
  r "string-length" prim_string_length;
  r "substring" prim_substring;
  r "string-contains?" prim_string_contains;
  r "string-split" prim_string_split;
  r "string-trim" prim_string_trim;
  r "string-replace" prim_string_replace;
  r "string-downcase" prim_string_downcase;
  r "string-upcase" prim_string_upcase;
  r "string-starts-with?" prim_string_starts_with_p;
  r "string-ends-with?" prim_string_ends_with_p;
  r "string=?" prim_string_eq;
  r "string-eq" prim_string_eq;
  r "type-of" prim_type_of;
  r "symbol?" prim_symbol_p;
  r "string?" prim_string_p;
  r "symbol->string" prim_symbol_to_string;
  r "string->symbol" prim_string_to_symbol;
  r "str" prim_str;
  r "as-str" prim_as_str;
  r "assoc" prim_assoc;
  r "member" prim_member;
  r "json-parse" prim_json_parse;
  r "json-emit" prim_json_emit;
  r "dict-get" prim_dict_get;
  r "dict-set" prim_dict_set;
  r "dict-keys" prim_dict_keys;
  r "equal?" prim_equal_p;
  r "random-uuid" prim_random_uuid;
  (* Pure rendering + string prims the Nopales surface (libs/html, the
     pp-remote surface) calls heavily. These are PURE (tag a VStr /
     split a string) -- registering them LOCAL (not proxied) keeps the
     surface responsive offline and matches the agent's pure set. *)
  r "render-html" prim_render_html;
  r "html-escape" prim_html_escape;
  r "string-split" prim_string_split;
  (* c*r accessors used by libs/hyper + libs/html (parity with the
     agent's pure set). *)
  r "cadr" prim_cadr;
  r "caar" prim_caar;
  r "caddr" prim_caddr;
  r "cddr" prim_cddr

(** Bind the server-side prims a client is allowed to proxy. These are
    the proxyable DB CRUD + read/identity prims (NOT the server-only
    branch/admin prims -- those stay unbound; see [bind_proxy_prim]). The
    set is deliberately a known list: it is the client's declaration of
    which server prims exist for it. A server prim not in this list is
    simply unbound on the client (Unbound_var), which is honest -- the
    client doesn't pretend to know about prims it wasn't built against. *)
let register_proxy_prims (t : t) : unit =
  List.iter (bind_proxy_prim t)
    [ "get"; "create"; "update"; "delete"; "rows"
    ; "function-source"; "function-doc"; "apropos"; "apropos-docs"
    ; "find-code"
    ; "list-bindings"; "whoami"; "current-author"
    ; "node-path"; "backlinks"; "type-fields"; "list-kind"
      (* tree/* prims -- the hierarchical store surface (one-tree). These
         are read-mostly (tree/get, tree/list, tree/subtree, ?) plus the
         write trio tree/upsert | tree/delete | tree/move. Proxied so a
         lens client can eval defs that call them; the server stays the
         data-plane authority. *)
```

```
    ; "tree/get"; "tree/list"; "tree/children-of"; "tree/subtree"
    ; "tree/lens-get"; "tree/lens-children"; "tree/lens-subtree"
    ; "tree/upsert"; "tree/delete"; "tree/move"
    ; "tree/get-uuid"; "tree/find-by"; "tree/path-label"; "tree/garden-root"
    ; "tree/folder-index"; "tree/children-html"
    ; "package-exports"; "package-bindings"
      (* def-cache invalidation coordinates (caching subsection):
         current-branch (branch namespace) + gen-now (migration-017
         branch_generations = the tree-time gen coordinate -- one
         counter, one namespace). Both are proxyable read prims. *)
    ; "current-branch"; "gen-now"
      (* local-first sync prims (borge/local-first.borg sync). sync/plan
         classifies; sync/push applies a two-way data push. Proxied so
         a lens client can drive a sync round. *)
    ; "sync/plan"; "sync/push" ]

(** Fetch the stored source for [name] by proxying the [function-source]
    read prim (registered on the image as a plain, proxyable prim -- see
    image/lib/prims_db.ml). Returns Some source when the server has a
    def for [name], None when it does not. *)
let fetch_def (t : t) (name : string) : string option =
  (match t.lens_lookup with
   | None -> None
   | Some lookup -> lookup name)
  |> (function
      | Some src -> Some src
      | None -> (
          (* Proxy [function-source]; catch transport exceptions (the Eio
             HTTP transport RAISES [Eio.Io] on a refused connection, it
             does not return [Error]) so an offline client with a lens
             misses cleanly ? [Unbound_var] instead of a process crash. *)
          match (try t.transport "function-source" [ VStr name ] with _ ->
    Error (Failure "offline")) with
          | Ok (VStr src) when String.trim src <> "" -> Some src
          | Ok (VStr _) -> None
          | Ok _ -> None
          | Error _ -> None))

(** Materialize a def into the client env: parse + eval the fetched
    source (a toplevel (define ?)) under the proxy rim in [env]. The
    def body may reference prims -- those proxy/run locally on call,
    not at define time.

    [function-source] returns the stored body as a bare (lambda ?),
    not a (define name ?) form, so eval produces the value but does
    NOT bind the name. Bind it explicitly when the src did not already
    do so (a (define ?) binds during eval; a bare lambda does not) --
    this is what makes a fetched def resolvable on the retry. *)
let load_def (t : t) (name : string) (src : string) : value =
  let v =
    Proxy_rim.run ~local_prims:t.local_prims ~transport:t.transport (fun ()
     ->
        Eval.read_eval_all_direct ~prims:t.local_prims t.env src)
  in
  (match Env.lookup t.env name with
   | None -> Env.define t.env name v
   | Some _ -> ());
  Def_cache.store t.def_cache name v;
  v

(** Lazy env miss inlined into [eval_local]: on [Unbound_var name],
    fetch the def source and materialize it, then retry the expression.
    Bounds retries per name so a genuinely unbound name fails cleanly. *)
let eval_local (t : t) (src : string) : value =
  (* ONE CHEAP CACHE-CHECK PER EVAL (caching subsection). Once the cache
     is live (a def has been materialized), confirm the (branch,
     generation) coordinate is still current -- a single HEAD-ish round
     trip (current-branch + gen-now), never an N-way hash diff. If a
     generation bump or branch switch makes any loaded def stale, evict
     its ENV binding so the next reference re-fetches it (the writer is
     the source of truth; the client cache never is). A def-free /
     pure-compute session is skipped entirely (nothing cached, zero
     round trips). *)
```

```
  (if Def_cache.is_active t.def_cache then (
     let stale = Def_cache.check t.def_cache in
     List.iter (fun n -> Hashtbl.remove t.env.bindings n) stale)
   else ());
  let seen = Hashtbl.create 4 in
  let rec go () =
    try
      Proxy_rim.run ~local_prims:t.local_prims ~transport:t.transport (fun
     () ->
          Eval.eval_string_direct ~prims:t.local_prims t.env src)
    with
    | Interp_error (Unbound_var name) ->
        if Hashtbl.mem seen name then raise (Interp_error (Unbound_var
     name))
        else (
          Hashtbl.replace seen name ();
          (* one cheap cache-check round trip before re-fetching: a
             branch switch invalidates the whole cache and a generation
             bump drops stale defs, so a def that changed on the server
             is re-fetched on next use. *)
          let stale = Def_cache.check t.def_cache in
          List.iter (fun n -> Hashtbl.remove t.env.bindings n) stale;
          (match Def_cache.lookup t.def_cache name with
           | Some _ ->
               (* still materialized at the current generation: retry the
                  expression (the env already binds the name; if it does not
                  the [seen] guard below re-raises rather than looping). *)
               go ()
           | None -> (
               match fetch_def t name with
               | Some def_src ->
                   ignore (load_def t name def_src);
                   go ()
               | None ->
                   Def_cache.remove t.def_cache name;
                   raise (Interp_error (Unbound_var name)))))
  in
  go ()

(** Hybrid placement entry point (borge/remote-eval.borg placement).
    Decide once per expression whether it runs LOCAL under the proxy rim
    or is POSTed REMOTE to /api/eval wholesale. The decision is logged
    so the boundary is visible -- no silent routing.

    - Local (pure, or <= threshold individually-proxied prims): eval
      under the proxy rim with lazy def materialization ([eval_local]).
    - Remote (transaction-wrapped or prim-dense): hand the WHOLE source
      to the [remote_eval] hook (a POST /api/eval). If no hook is
      configured we RAISE rather than silently run locally -- a
      (with-transaction ?) CANNOT run locally without corrupting the
      store (locks don't span HTTP), so pretending to is never safe. *)
let eval (t : t) (src : string) : value =
  match Placer.place ~cfg:(placer_cfg t) src with
  | Placer.Local -> eval_local t src
  | Placer.Remote (target, _reason) -> (
      match t.remote_eval with
      | Some f -> f src
      | None ->
          failwith
            (Printf.sprintf
               "pp-remote: expression must run remotely (%s) but no
    remote-eval \
                transport is configured -- not running it locally"
               target))

(** Create a client bound to a [transport] against one image. No
    network is touched here (boot is lazy), so construction is cheap
    and pure. Pass [~remote_eval] (a whole-expression POST /api/eval) to
    enable hybrid placement; without it a Remote placement (e.g. a
    (with-transaction ?)) raises rather than silently running locally. *)
let str_arg name v =
  match v with
  | VStr s -> s
```

```
  | VSymbol s -> s
  | _ ->
      raise
        (Ast.Interp_error
           (Ast.Prim_error (name, "expected string/symbol argument",
     "bad-args")))

let emit_error name msg =
  raise (Ast.Interp_error (Ast.Prim_error (name, msg, "error")))

(* ---- Route table + request-context prims (agent-route-table host
   seam, ported to the client). Identical Nopales signatures to the
   image's route / route/lookup / route/dispatch, host-local backing
   (client route table + thin request context vs the image's Postgres
   routes + fat Request_context). libs/hyper runs unchanged on both.

   The only adaptation from agent/runtime.ml: the rim that services
   Prim_call inside a dispatched handler is the PROXY rim (forwarding
   to /api/prim), not the agent's local host-prim table. The route /
   dispatch / template machinery itself is env-only -- it has zero
   dependency on which rim services the handler's prim calls. *)

(** [apply_value_raw] -- apply a Nopales function value to host-side
    value args directly, assuming a rim is ALREADY active (the caller
    holds nothing: the proxy rim is non-reentrant per fiber but the
    caller is the rim's own continuation). Used by prims that run
    inside an eval (route/dispatch invokes a handler from within the
    rim). Args NEVER go through the reader, so lambda values and
    arbitrary JSON strings are safe. *)
let apply_value_raw (t : t) (fn : Ast.value) (args : Ast.value list)
    : Ast.value =
  let fn_sym = "__host_apply_fn__" in
  Env.define t.env fn_sym fn;
  let arg_syms =
    List.mapi (fun i _ -> Printf.sprintf "__host_apply_arg_%d__" i) args
  in
  List.iter2 (Env.define t.env) arg_syms args;
  let src =
    "(" ^ fn_sym
    ^ List.fold_left (fun acc a -> acc ^ " " ^ a) "" arg_syms
    ^ ")"
  in
  Eval.eval_string_direct ~prims:t.local_prims t.env src

(** Apply a Nopales function value to host-side value args under the
    proxy rim -- for OCaml callers OUTSIDE an active eval (the HTTP
    surface renders templates). Prims running inside an eval use
    [apply_value_raw] instead. *)
let apply_value (t : t) (fn : Ast.value) (args : Ast.value list)
    : Ast.value =
  Proxy_rim.run ~local_prims:t.local_prims ~transport:t.transport (fun () ->
      apply_value_raw t fn args)

let require_request (t : t) : request_shape =
  match !(t.current_request) with
  | Some r -> r
  | None ->
      emit_error "request" "no active request (call inside route/dispatch)"

(** [dispatch_route_raw] -- the shared route/dispatch body. Assumes a
    rim is active (route/dispatch prim runs inside an eval; the HTTP
    gate uses the [dispatch_route] wrapper). Returns the rendered
    raw-HTML VStr the browser would receive. *)
let dispatch_route_raw (t : t) ~handler ~method_ ~path ~params ~headers
    ~body : Ast.value =
  let req : request_shape =
    { method_ = String.uppercase_ascii method_; path; params; headers; body
    }
  in
  match Env.lookup t.env handler with
  | None -> VStr (Printf.sprintf "route/dispatch: unbound handler %s"
     handler)
  | Some fn ->
```

```
      t.current_request := Some req;
      Fun.protect
        ~finally:(fun () -> t.current_request := None)
        (fun () ->
          try
            let v = apply_value_raw t fn [] in
            VStr (Prims_soup.render_handler_value v)
          with Ast.Interp_error ie ->
            VStr
              (Printf.sprintf "route/dispatch: %s"
                 (Ast.pp_interp_error ie)))

(** [dispatch_route] -- rim-establishing wrapper for OCaml callers
    OUTSIDE an active eval (the HTTP gate dispatches GET routes through
    the route table). Runs under the proxy rim so the handler's prim
    calls forward to /api/prim. *)
let dispatch_route ~(rt : t) ~handler ~method_ ~path ~params ~headers
    ~body : Ast.value =
  Proxy_rim.run ~local_prims:rt.local_prims ~transport:rt.transport (fun ()
    ->
      dispatch_route_raw rt ~handler ~method_ ~path ~params ~headers ~body)

(** Resolve a URL path to its handler name via the client route table. *)
let lookup_route ~(rt : t) (path : string) : string option =
  Hashtbl.find_opt rt.routes path

(** Call a Nopales template (zero-or-more-arg fn) by name and return the
    rendered raw-HTML string. For OCaml callers outside an eval (the
    HTTP surface renders fragment templates). Returns [None] if the
    template is unbound or errors. *)
let call_template ~(rt : t) (name : string) (args : Ast.value list)
    : string option =
  match Env.lookup rt.env name with
  | None -> None
  | Some fn ->
      (try
         let v = apply_value rt fn args in
         Some (Prims_soup.render_handler_value v)
       with Ast.Interp_error _ -> None)

(** Register the route-table + request-context prims (route,
    route/lookup, route/dispatch, query-param, request-method,
    hx-request?, session/last-turn-parts) PLUS the lens-browsing prims
    (lens/list, lens/read) so the Nopales-authored surface can register
    routes and browse the local lens. All are LOCAL prims (they operate
    on the client's env / route table / lens, never the wire), so they
    go through [add_prim] and the placer treats them as not-crossing. *)
let register_route_prims (t : t) : unit =
  let route args =
    match args with
    | [ path; handler ] ->
        let path = str_arg "route" path in
        let handler = str_arg "route" handler in
        Hashtbl.replace t.routes path handler;
        VBool true
    | _ -> emit_error "route" "usage: (route path handler)"
  in
  add_prim t "route" route;
  let route_lookup args =
    match args with
    | [ path ] ->
        let path = str_arg "route/lookup" path in
        (match Hashtbl.find_opt t.routes path with
         | Some h -> VStr h
         | None -> VNil)
    | _ -> emit_error "route/lookup" "usage: (route/lookup path)"
  in
  add_prim t "route/lookup" route_lookup;
  let route_dispatch args =
    match args with
    | VStr handler :: VStr meth :: VStr path :: rest ->
        let params =
          match rest with p :: _ -> Prims_soup.assoc_of p | _ -> []
```

```
        in
        let headers =
          match rest with
          | _ :: h :: _ ->
              List.map
                (fun (k, v) -> (String.lowercase_ascii k, v))
                (Prims_soup.assoc_of h)
          | _ -> []
        in
        let body =
          match rest with _ :: _ :: VStr b :: _ -> b | _ -> ""
        in
        dispatch_route_raw t ~handler ~method_:meth ~path ~params
          ~headers ~body
    | _ ->
        emit_error "route/dispatch"
          "usage: (route/dispatch handler method path [params] [headers]
     [body])"
  in
  add_prim t "route/dispatch" route_dispatch;
  let query_param args =
    match args with
    | [ key ] ->
        let r = require_request t in
        let key = str_arg "query-param" key in
        (match List.assoc_opt key r.params with
         | Some v -> VStr v
         | None -> VNil)
    | _ -> emit_error "query-param" "usage: (query-param key)"
  in
  add_prim t "query-param" query_param;
  let request_method _args =
    let r = require_request t in
    VStr r.method_
  in
  add_prim t "request-method" request_method;
  let hx_request_p _args =
    let r = require_request t in
    let from_headers =
      match List.assoc_opt "hx-request" r.headers with
      | Some v when v <> "" && v <> "false" -> true
      | _ -> false
    in
    let from_params =
      match List.assoc_opt "hx-request" r.params with
      | Some v when v <> "" && v <> "false" -> true
      | _ -> false
    in
    VBool (from_headers || from_params)
  in
  add_prim t "hx-request?" hx_request_p;
  let last_turn_parts _args = t.last_turn_parts in
  add_prim t "session/last-turn-parts" last_turn_parts;
  (* lens/list -> JSON array of {"path","kind","name"} over the lens
     substrate. Empty array when no lens is wired. Kind is derived from
     the tree path (b.<branch>.fn|type|route.<name> | data.<...>). *)
  let lens_list _args =
    match t.lens with
    | None -> VStr "[]"
    | Some lens ->
        let nodes = Lens_store.node_manifest ~lens in
        let kind_of_path p =
          if String.length p >= 2 && String.sub p 0 2 = "b." then begin
            let segs = String.split_on_char '.' p in
            (* b.<branch>.<kind>.<name> -> segs.(2) *)
            match segs with
            | _ :: _ :: k :: _ -> k
            | _ -> "code"
          end else if String.length p >= 5 && String.sub p 0 5 = "data."
          then "data"
          else "node"
        in
        let name_of_path p =
```

```
          let segs = String.split_on_char '.' p in
          match List.rev segs with
          | n :: _ -> n
          | [] -> p
        in
        let items =
          List.map
            (fun (p, _h) ->
              `Assoc
                [ ("path", `String p)
                ; ("kind", `String (kind_of_path p))
                ; ("name", `String (name_of_path p))
                ])
            nodes
        in
        VStr (Yojson.Safe.to_string (`List items))
  in
  add_prim t "lens/list" lens_list;
  (* lens/read path -> VStr source | VNil. For a code node returns the
     def source (code_src_of_envelope); for a data node returns the
     row's body JSON (datum_body_of_envelope); else the raw envelope. *)
  let lens_read args =
    match t.lens, args with
    | None, _ -> VNil
    | Some lens, [ path ] ->
        let p = str_arg "lens/read" path in
        (match Lens_store.get ~lens p with
         | Some env ->
             (match Lens_store.code_src_of_envelope env with
              | Some s -> VStr s
              | None ->
                  (match Lens_store.datum_body_of_envelope env with
                   | Some s -> VStr s
                   | None -> VStr env))
         | None -> VNil)
    | _ -> emit_error "lens/read" "usage: (lens/read path)"
  in
  add_prim t "lens/read" lens_read

(* ---- agent-loop host seam (borge/remote-eval.borg agent-loop) ----

   The client can run the LLM agent turn LOCALLY: the shared loop
   (llm/lib/loop.ml) + provider (llm/lib/provider.ml) are pure OCaml
   and link into pp-remote; the API key is a value read from the
   environment (same AGENT_* vars the agent reads). What stays remote
   is the DATA PLANE: a tool body's prim calls go through the proxy
   rim, so (get 'user id) inside a Nopales tool crosses to /api/prim
   exactly as a REPL eval would. The surface below mirrors the agent's
   Runtime surface (fire / eval_tool_body / register_builtin_tool) --
   same shapes, proxy rim instead of the local host-prim table. *)

(** Current working directory as seen by the read tool's path
    resolution. Immutable per process (pp-remote never chdir-s).
    Refuses ".." escapes -- the same v0 sandbox posture as the agent. *)
let cwd = ref (Sys.getcwd ())

let resolve_path p =
  let p =
    if String.length p > 0 && p.[0] = '/' then p else Filename.concat !cwd p
  in
  let parts =
    String.split_on_char '/' p
    |> List.filter (fun s -> s <> "" && s <> ".")
  in
  if List.exists (fun s -> s = "..") parts then
    emit_error "fs/path" ("path escapes sandbox: " ^ p);
  "/" ^ String.concat "/" parts

(** Register the event + plugin-tool prims: (on "event" fn) hooks and
    (register-tool name params fn). Local prims over the client's own
    hashtbls -- the loop's tool_call/tool_result/input events fire
    through [fire] below, and a Nopales plugin tool's closure evals
    under the proxy rim (its DB prim calls proxy). Same Nopales
```

```
    signatures as the agent's, so plugin sources transfer. *)
let register_event_prims (t : t) : unit =
  let on args =
    match args with
    | [ ev; fn ] ->
        let ev = str_arg "on" ev in
        let cell =
          match Hashtbl.find_opt t.handlers ev with
          | Some c -> c
          | None ->
              let c = ref [] in
              Hashtbl.replace t.handlers ev c;
              c
        in
        cell := !cell @ [ (fun vals -> apply_value t fn vals) ];
        VBool true
    | _ -> emit_error "on" "usage: (on \"event\" (lambda (ev) ...))"
  in
  add_prim t "on" on;
  let reg_tool args =
    match args with
    | [ name; _params; fn ] ->
        let name = str_arg "register-tool" name in
        Hashtbl.replace t.tools name { doc = ""; run = fn };
        if not (List.mem name t.tool_order) then
          t.tool_order <- t.tool_order @ [ name ];
        VBool true
    | _ -> emit_error "register-tool" "usage: (register-tool name params
     fn)"
  in
  add_prim t "register-tool" reg_tool

(** Fire an event: run handlers in registration order, containing a
    raising handler into an {"error": ?} value (mirrors agent
    Runtime.fire). *)
let fire (t : t) (ev : string) (payload : Ast.value list) : Ast.value list =
  match Hashtbl.find_opt t.handlers ev with
  | None -> []
  | Some hs ->
      List.map
        (fun h ->
          try h payload
          with Ast.Interp_error ie ->
            Ast.VStr
              (Yojson.Safe.to_string
                 (`Assoc
                   [
                     ("error",
                      `String
                        (Printf.sprintf "%s handler: %s" ev
                           (Ast.pp_interp_error ie)));
                   ])))
        !hs

(** Run a plugin tool's closure with JSON args under the proxy rim,
    containing errors into a {"error": ?} JSON string (mirrors agent
    Runtime.eval_tool_body). *)
let eval_tool_body (t : t) (name : string) (args_json : string) : Ast.value
    =
  match Hashtbl.find_opt t.tools name with
  | None ->
      VStr
        (Yojson.Safe.to_string
           (`Assoc [ ("error", `String ("no tool named " ^ name)) ]))
  | Some e ->
      (try apply_value t e.run [ VStr args_json ]
       with Ast.Interp_error ie ->
         VStr
           (Yojson.Safe.to_string
              (`Assoc
                [ ("error", `String (Ast.pp_interp_error ie)) ])))

(** Register a builtin host tool (read/bash): registry entry as a VPrim
```

```
716     naming a LOCAL prim, so the loop dispatches builtins through the
717     proxy rim's local table (no wire -- builtin bodies are OCaml).
718     Mirrors agent Runtime.register_builtin_tool. *)
719 let register_builtin_tool (t : t) (name : string)
720     ~(run : Yojson.Safe.t -> string) : unit =
721   Hashtbl.replace t.tools name
722     { doc = "builtin"; run = VPrim ("__builtin__" ^ name) };
723   if not (List.mem name t.tool_order) then
724     t.tool_order <- t.tool_order @ [ name ];
725   add_prim t ("__builtin__" ^ name)
726     (fun args ->
727       match args with
728       | [ Ast.VStr args_json ] -> (
729           try VStr (run (Yojson.Safe.from_string args_json))
730           with
731           | Yojson.Json_error e ->
732               VStr
733                 (Yojson.Safe.to_string
734                    (`Assoc [ ("error", `String ("bad json: " ^ e)) ]))
735           | Ast.Interp_error ie ->
736               VStr
737                 (Yojson.Safe.to_string
738                    (`Assoc
739                      [ ("error", `String (Ast.pp_interp_error ie)) ])))
740       | _ -> emit_error "__builtin" "expected single JSON string arg")
741
742 (** Host adapter for the shared agent turn loop (llm/lib/loop.ml): the
743     client's [Loop.host] -- local tool registry, event fire, tool-body
744     eval under the proxy rim, and builtin prim dispatch through the
745     proxy rim (builtin bodies run as local prims; a plugin tool's DB
746     prim calls forward to /api/prim). This is the whole seam: the loop
747     itself is host-agnostic. *)
748 let loop_host (t : t) : Loop.host =
749   {
750     Loop.tool_order = (fun () -> t.tool_order);
751     tool_entry = (fun name -> Hashtbl.find_opt t.tools name);
752     fire = fire t;
753     eval_tool_body = eval_tool_body t;
754     call_prim =
755       (fun name args ->
756          Proxy_rim.run ~local_prims:t.local_prims ~transport:t.transport
757            (fun () -> (Hashtbl.find t.local_prims name) args));
758   }
759
760 (** Register the shared soup/ tree prims (lifted into pure/prims_soup.ml
761     so both hosts register the same set). [Prims_soup.register_pure_prims]
762     fills the prim table; we bind each soup/ name as a VPrim in env so
763     Nopales resolves it, and add each to [pure_prims] so the placer
764     treats it as local (soup prims are pure -- they never cross the
765     wire). Mirrors agent/runtime.ml register_soup_prims. *)
766 let register_soup_prims (t : t) : unit =
767   Prims_soup.register_pure_prims t.local_prims;
768   Hashtbl.iter
769     (fun name _ ->
770       if String.length name >= 5 && String.sub name 0 5 = "soup/" then begin
771         Env.define t.env name (Ast.VPrim name);
772         t.pure_prims <- name :: t.pure_prims
773       end)
774     t.local_prims
775
776 (** Evaluate all top-level forms in [src] into the shared env, under the
777     proxy rim. Used to boot-load Nopales packages (libs/html, libs/hyper,
778     the surface source) as SOURCE -- the same env as the REPL, so the
779     surface is redefinable live (borge/remote-eval.borg surfaces). *)
780 let load_source ~(rt : t) (src : string) : unit =
781   Proxy_rim.run ~local_prims:rt.local_prims ~transport:rt.transport (fun ()
      ->
782       ignore (Eval.read_eval_all_direct ~prims:rt.local_prims rt.env src))
783
784 (** Set the lens record on the client (after [make]) so lens/list +
785     lens/read surface prims browse the substrate. *)
786 let set_lens (t : t) (lens : Lens_store.lens) : unit =
787   t.lens <- Some lens
```

```
788
789
790 let make ~(transport : transport) ?(remote_eval : (string -> value) option)
791     ?(lens_lookup : (string -> string option) option) () : t =
792   let t =
793     {
794       env = Env.create ();
795       local_prims = Eval.make_prim_table ();
796       transport;
797       def_cache = Def_cache.create ~transport ();
798       pure_prims = [];
799       remote_eval;
800       lens_lookup;
801       lens = None;
802       routes = Hashtbl.create 16;
803       current_request = ref None;
804       last_turn_parts = Ast.VNil;
805       tools = Hashtbl.create 16;
806       tool_order = [];
807       handlers = Hashtbl.create 8;
808     }
809   in
810   register_pure_prims t;
811   register_soup_prims t;
812   register_route_prims t;
813   register_event_prims t;
814   register_proxy_prims t;
815   (* Minimal stdlib: the [let*] sequential-binding macro libs/hyper
816      uses, without the heavy libs/std (which depends on image-only
817      prims). Mirrors agent/runtime.ml boot_stdlib. *)
818   load_source ~rt:t {|
819 (defmacro let* (bindings body)
820   (if (null? bindings)
821       (list (quote let) (list) body)
822       (list (quote let)
823             (list (list (car (car bindings)) (car (cdr (car bindings)))))
824             (list (quote let*) (cdr bindings) body))))
825 |};
826   t
827
828 (** [register_lens_reads t ~lens] overrides the [get] and [tree/get]
829     proxy prims with LOCAL handlers that read from the lens substrate
830     FIRST, proxying to the image on miss. This is the Phase-1
831     local-first data path: working-set reads [(get 'type id)] and
832     [(tree/get path)] resolve from the filesystem JSON the lens
833     materialized, so they stay local offline. Indexed reads [(rows ?)],
834     [(tree/find-by ?)] keep proxying (the server is the index --
835     local-first.borg local-substrate).
836
837     A transport FAILURE (the Eio HTTP transport raises [Eio.Io] on a
838     refused connection) is caught and degrades to [VNil] ("not found")
839     rather than crashing -- offline grace. The lens is a WORKING COPY; a
840     miss + offline is "not in the working set," not an error. *)
841 let register_lens_reads (t : t) ~(lens : Lens_store.lens) : unit =
842   (* Mirror the server's [coerce_sym_args]: quoted symbols ('type) arrive
843      as VSymbol; the server's CRUD handlers coerce to VStr, so the lens
844      handlers must too or the [VStr] arg match would miss. *)
845   let coerce = function Ast.VSymbol s -> Ast.VStr s | v -> v in
846   let proxy_or lens_hit args =
847     match lens_hit () with
848     | Some v -> v
849     | None ->
850         (* miss: proxy. Catch transport exceptions (offline) ? VNil. *)
851         (match (try t.transport "get" args with _ -> Error (Failure
     "offline")) with
852          | Ok v -> v
853          | Error _ -> Ast.VNil)
854   in
855   (* (get 'type id) -> VStr fields-json | VNil. Reads the datum node
856      envelope at [data.u.<user>.<type>.<id>] from the lens, extracts the
857      body (the row's fields), returns it as VStr -- matching [Crud.get]. *)
858   add_prim t "get"
859     (fun args ->
```

```
      match List.map coerce args with
      | [ Ast.VStr type_name; Ast.VStr id ] ->
          let path = Lens_store.datum_path ~user:lens.owner ~type_name ~id
     () in
          proxy_or
            (fun () ->
              match Lens_store.get ~lens path with
              | Some env ->
                  (match Lens_store.datum_body_of_envelope env with
                   | Some s -> Some (Ast.VStr s)
                   | None -> None)
              | None -> None)
            args
      | _ -> Ast.VNil)
  (* (tree/get path) -> VStr node-envelope-json | VNil. Returns the full
     envelope verbatim (matches [Crud.tree_get]). *)
  ; add_prim t "tree/get"
    (fun args ->
      match List.map coerce args with
      | [ Ast.VStr path ] ->
          proxy_or
            (fun () ->
              match Lens_store.get ~lens path with
              | Some env -> Some (Ast.VStr env)
              | None -> None)
            args
      | _ -> Ast.VNil)

(** ---- Stepping / trace debugger over one remote client ----

    (borge/remote-eval.borg stepping: set bp, step, inspect frame,
    continue.) The driver runs [Explain.trace] over THIS client's env +
    local pure prim table + transport, so local pure compute stays local
    and every other prim round trip crosses the same [transport] the
    fast eval would -- the server is unaware a breakpoint fired; the
    data plane is untouched. *)

(** One-pass stepped run of [src] over this client. Returns the trace
    (local steps interleaved with remote round trips) plus a drivable
    stepper to set breakpoints / step / continue over it. *)
let stepper_of (t : t) (src : string) :
    Explain.trace_result * Explain.stepper =
  let tr =
    Explain.trace ~local_prims:t.local_prims ~transport:t.transport
      ~init_env:t.env ~src
  in
  (tr, Explain.stepper_of_trace tr)
```

292 remote/lib/surface.ml

```
(** pp-remote Nopales surface (borge/remote-eval.borg surfaces +
    agent-loop).

    The browser surface served by [Http_ui.serve] is NOPALES-AUTHORED
    htmx fragments on the client route table, rendered by Nopales via
    [libs/html] -- NOT compiled OCaml pages. The surface is redefinable
    live (same env as the REPL): re-evaluating a `(define chat/page ?)`
    at the REPL changes the next render without a restart. This is the
    faithful version of the agent's surface thesis, ported to the
    client: the route table + template dispatch are env-only and run
    unchanged under the proxy rim.

    Routes:
      /        -- chat page. POST /chat (OCaml-handled multipart stream)
                  runs the SHARED agent loop locally: provider call on
                  this machine, tool bodies under the proxy rim (DB prims
                  cross to the image). Fragment templates
                  (chat/user-bubble, chat/turn-open, chat/token, ...) are
                  Nopales, same names as the agent's so they transfer.
      /eval    -- eval box. POST /eval (OCaml-handled) calls Remote.eval
                  and renders the result via the Nopales eval/result
                  template.
      /code    -- lens browser. Lists lens nodes (lens/list) and reads
                  one (lens/read). Shows "no lens" when --lens is absent.

    The chat stream emitter + eval POST handler and the HTTP accept
    loop stay OCaml (per the surfaces scope: bootstrap/transport
    plumbing is OCaml); the PAGES and fragment templates are Nopales. *)

open Pricklypear_lisp

(** The surface source. Defines chat/page + the chat/* fragment
    templates, eval/page, eval/result, code/page, code/rows and registers
    the /, /eval and /code routes via the client route table. Uses
    libs/html + the client's request-shape prims (query-param) + the
    lens-browsing prims (lens/list, lens/read). The chat fragment
    templates carry the same names/shapes as the agent's
    (agent/lib/surface.ml) so a restyling transfers between hosts. *)
let surface_source = {pp|
;; ---- chat surface (GET /) ----
;; The chat page skeleton: log target + an hx-post=/chat form with the
;; upstream htmx-4 hx-multipart extension so a multipart/mixed reply
;; streams each part live into the bubble. htmx + hx-multipart are
;; vendored assets served at /assets/*.js (baked into the binary).
;; hx-config '{"timeout":0}' disables htmx's 60s defaultTimeout for a
;; request whose lawful lifetime is "a whole agent turn".
(define (pp/nav)
  (string-append
    "<h1><a href=\"/\">pp-remote</a> <span class=\"muted\">\u2014 "
    "<a href=\"/eval\">/eval</a> \u2014 <a
     href=\"/code\">/code</a></span></h1>"))

(define (pp/styles)
  (string-append
    "body{background:#0f1117;color:#d8dee9;font-family:ui-monospace,Menlo,monospace;margin:0;padding:1.5rem}"
    "a{color:#7aa2f7;text-decoration:none}a:hover{text-decoration:underline}"
    ".muted{color:#6b7080}"
    "h1 a{color:#d8dee9}"
    "#log{height:60vh;overflow-y:scroll;border:1px solid
     #333;border-radius:6px;padding:.75rem;margin-bottom:.75rem}"
    ".bubble{background:#1b1f2a;border-radius:6px;padding:.5rem
     .75rem;margin:.5rem 0;white-space:pre-wrap}"
    ".bubble.user{color:#9ece6a}"
    ".bubble.llm{color:#c0caf5}"
    ".trace{color:#6b7080;font-size:.85em;white-space:pre-wrap}"
    ".think{color:#6b7080;font-size:.9em;margin:.4rem 0}"
    ".think summary{cursor:pointer}"
    ".think>div{white-space:pre-wrap}"
    ".reason{color:#8a8f9e;font-style:italic}"
```

```
    ".think .tk,.think .tool{margin:.25rem 0 .25rem .25rem}"
    ".think .tk>summary,.think .tool>summary{color:#8a8f9e}"
    ".think .tk>div,.think
     .tool>div{margin-left:.9rem;padding-left:.5rem;border-left:1px solid
     #2a2e3a}"
    ".think:not(:has(.reason,.trace)){display:none}"
    "form{display:flex;gap:.5rem}"
    "input{flex:1;background:#1b1f2a;color:#d8dee9;border:1px solid
     #333;border-radius:6px;padding:.5rem}"

     "button{background:#2a6fdb;color:#fff;border:0;border-radius:6px;padding:.5rem
     1rem;cursor:pointer}"
    "textarea{background:#1b1f2a;color:#d8dee9;border:1px solid
     #333;border-radius:6px;padding:.5rem;font-family:inherit;font-size:1rem;min-height:6rem}"

     "pre{background:#12141c;padding:1rem;border-radius:6px;overflow:auto;white-space:pre-wrap;word-break:break-word}"
    ".expr{color:#9ece6a;margin:.5rem 0}"
    "table{border-collapse:collapse}td,th{padding:.25rem
     .75rem;border-bottom:1px solid #222;text-align:left}"))

(define (chat/page)
  (render-html
    (string-append
      "<!doctype html><html><head><meta
     charset=utf-8><title>pp-remote</title>"
      "<style>" (pp/styles) "</style></head><body>"
      (pp/nav)
      "<nav class=\"muted\">chat: the agent turn runs here \u2014 provider
     local, DB prims proxy to the image.</nav>"
      "<div id=\"log\"><div class=\"muted\">pp-remote ready.</div></div>"
      "<form id=\"f\" hx-post=\"/chat\" hx-target=\"#log\"
     hx-swap=\"beforeend\" hx-ext=\"hx-multipart\""
      " hx-config='{\"timeout\":0}' hx-on::after-request=\"this.reset()\'>"
      "<input id=\"msg\" name=\"msg\" type=\"text\"
     placeholder=\"prompt\u2026\" autofocus>"
      "<button>send</button>"
      "</form>"
      "<script src=\"/assets/htmx.min.js\"></script>"
      "<script src=\"/assets/htmx-2-compat.min.js\"></script>"
      "<script src=\"/assets/hx-multipart.js\"></script>"
      "</body></html>")))
(route "/" "chat/page")

;; ---- chat fragment templates (called per-part by the OCaml stream) ----
;; Same names/shapes as the agent's surface (agent/lib/surface.ml):
;; redefine live to restyle; a restyle transfers between hosts.
(define (chat/user-bubble text)
  (render-html (string-append "<div class=\"bubble user\">" (html-escape
     text) "</div>")))

(define (chat/turn-open id)
  (render-html
    (string-append
      "<div class=\"turn\"><details class=\"think\"><summary>thinking +
     tools</summary>"
      "<div id=\"" id "-think\"></div></details>"
      "<div class=\"bubble llm\" id=\"" id "\"></div></div>")))

(define (chat/think-open id)
  (render-html
    (string-append
      "<details class=\"tk\"><summary>thinking</summary>"
      "<div id=\"" id "\"></div></details>")))

(define (chat/tool-open id name)
  (render-html
    (string-append
      "<details class=\"tool\"><summary id=\"" id "-sum\">tool: "
      (html-escape name) "</summary><div id=\"" id "\"></div></details>")))

(define (chat/tool-line label text)
  (render-html
    (string-append
```

```
      "<div class=\"trace\">" (html-escape label) " "
      (html-escape text) "</div>")))

(define (chat/tool-done name)
  (render-html (string-append "tool: " (html-escape name) " &#10003;")))

(define (chat/think-token text)
  (render-html (string-append "<span class=\"reason\">" (html-escape text)
     "</span>")))

(define (chat/token text)
  (render-html (string-append "<span>" (html-escape text) "</span>")))

(define (chat/error text)
  (render-html (string-append "<div class=\"trace\">[turn error: "
     (html-escape text) "]</div>")))

;; ---- eval surface (GET /eval) ----
;; An eval box: textarea + button that hx-post=/eval, target #out.
;; htmx is a vendored asset served at /assets/*.js (baked into the
;; pp-remote binary). The POST handler (OCaml) calls Remote.eval and
;; renders the result via eval/result, so the result fragment is
;; Nopales-authored and redefinable live.
(define (eval/page)
  (render-html
    (string-append
      "<!doctype html><html><head><meta charset=utf-8><title>pp-remote
     eval</title>"
      "<style>" (pp/styles) "</style></head><body>"
      (pp/nav)
      "<nav class=\"muted\">eval against the lens + proxy rim. POST /eval
     runs Remote.eval.</nav>"
      "<form hx-post=\"/eval\" hx-target=\"#out\" hx-swap=\"innerHTML\">"
      "<textarea name=\"expr\" placeholder=\"(+ 1
     2)\\n(list-packages)\\n(whoami)\" autofocus></textarea>"
      "<button>eval</button></form>"
      "<pre id=\"out\" class=\"muted\">--</pre>"
      "<script src=\"/assets/htmx.min.js\"></script>"
      "</body></html>")))
(route "/eval" "eval/page")

;; ---- eval result fragment (rendered by the OCaml POST /eval handler) ----
;; (eval/result expr result) -> raw-HTML fragment swapped into #out.
;; [result] is the pp_value string the handler produced; [expr] is the
;; source the user submitted. Redefine live to restyle.
(define (eval/result expr result)
  (render-html
    (string-append
      "<div class=\"expr\">$ " (html-escape expr) "</div>"
      "<pre>" (html-escape result) "</pre>")))

;; ---- lens browser (GET /code) ----
;; Read-only list of lens nodes + a node view, rendered by Nopales via
;; lens/list + lens/read (client host prims backed by Lens_store). With
;; no --lens wired, lens/list returns "[]" and the page says so.
;; query-param "path" selects a node to view.
(define (code/page)
  (let* ((p (query-param "path"))
         (rel (if (null? p) "" p)))
    (if (string-eq rel "")
        (let* ((nodes-json (lens/list))
               (nodes (json-parse nodes-json)))
          (render-html
            (string-append
              "<!doctype html><html><head><meta
     charset=utf-8><title>code</title>"
              "<style>" (pp/styles) "</style></head><body>"
              (pp/nav)
              (if (null? nodes)
                  "<p class=\"muted\">no lens wired (start pp-remote with
     --lens DIR).</p>"
                  (string-append
```

```
                    "<p class=\"muted\">" (html-escape (as-str (count
    nodes))) " node(s)</p>"

    "<table><tr><th>kind</th><th>name</th><th>path</th></tr>"
                    (code/rows nodes)
                    "</table>"))
              "</body></html>")))
        (let* ((src (lens/read rel)))
          (render-html
            (string-append
              "<!doctype html><html><head><meta
    charset=utf-8><title>code</title>"
              "<style>" (pp/styles) "</style></head><body>"
              (pp/nav)
              "<p class=\"muted\">" (html-escape rel) "</p>"
              (if (null? src)
                  "<p class=\"muted\">not found.</p>"
                  (string-append "<pre>" (html-escape src) "</pre>"))
              "</body></html>"))))))

(define (code/rows nodes)
  (list/foldl
    (lambda (acc entry)
      (let* ((name (dict-get entry "name"))
             (kind (dict-get entry "kind"))
             (path (dict-get entry "path")))
        (string-append acc
          "<tr><td>" (html-escape kind)
          "</td><td><a href=\"/code?path=" (html-escape path) "\">"
          (html-escape name) "</a></td><td class=\"muted\">"
          (html-escape path) "</td></tr>")))
    "" nodes))

(route "/code" "code/page")
|pp}

(** Boot-load the embedded Nopales packages + this surface into the
    shared env, registering the route-table routes. Idempotent-ish:
    re-evaluating defines redefines bindings live. Call AFTER
    [Remote.make] (which registers the route + soup + pure prims and
    the let* macro). *)
let load ~(rt : Remote.t) : unit =
  Remote.load_source ~rt Libs_source.html_lib;
  Remote.load_source ~rt Libs_source.hyper_lib;
  Remote.load_source ~rt surface_source
```

293 remote/lib/sync_client.ml

```
(* remote/lib/sync_client.ml -- borge/local-first.borg sync, the CLIENT
   half of the exchange (the git protocol in miniature, one round).

   A round: build the client's data (path, hash) manifest from the lens
   substrate, send it to the image's sync/push prim (with the recorded
   base manifest from the last successful round -- three-way), apply the
   response (pull server copies, mirror server deletions, print the
   surfaced LWW resolutions), and pin the merged state as the NEXT
   round's base. The server is the merge authority; the client is a
   governed branch rendezvousing with it.

   Invisibility is the common case: a quiet round (nothing to transfer,
   nothing diverged) prints nothing. Every divergence resolution --
   client-won or server-kept -- is a printed line (the obituary rule
   applies to sync losses). This module is the machinery --sync runs
   once and --sync-loop runs on a timer (the "background" in
   "sync invisibly in the background" is client-side scheduling of these
   rounds; no new server surface is needed). *)

open Pricklypear_lisp
open Ast

(** The transport shape (Proxy_rim.transport): a prim call over the wire
    returning a value or a printable error. *)
type transport = string -> value list -> (value, exn) Result.t

type round_report = {
  ok : bool;                    (* false when the round failed (offline,
     bad shape) *)
  error : string option;
  pushed : int;                 (* client-only nodes applied server-side *)
  pulled : int;                 (* server copies fetched into the lens *)
  deleted_local : int;          (* stale local copies removed (server
     deleted) *)
  diverge : int;                (* diverge pairs seen (detection count) *)
  lww_client_won : string list; (* diverge/conflict resolved TO the client
     copy *)
  lww_server_kept : string list; (* diverge/conflict resolved TO the server
     copy (loss surfaced) *)
  branch : string;              (* pinned snapshot branch *)
  generation : Int64.t;         (* pinned snapshot generation (next
     merge-base coordinate) *)
}

let empty_report () =
  { ok = false;
    error = None;
    pushed = 0;
    pulled = 0;
    deleted_local = 0;
    diverge = 0;
    lww_client_won = [];
    lww_server_kept = [];
    branch = "";
    generation = 0L }

(** The data replication unit this client syncs: data.u.<owner>.* (the
    user's own rows). Code-lens sync is deliberately NOT here -- it is
    the branch-pointer move (promote/merge), server-mediated. *)
let data_manifest ~(lens : Lens_store.lens) : (string * string) list =
  let user_prefix = "data.u." ^ Lens_store.enc_label lens.owner in
  let plen = String.length user_prefix in
  Lens_store.node_manifest ~lens
  |> List.filter (fun (p, _) ->
         String.length p >= plen && String.sub p 0 plen = user_prefix)

(** Run ONE sync round. [quiet] = background mode: print only activity
    (pushes, pulls, deletions, surfaced LWW resolutions) and errors --
    a clean round is silent. Never raises: transport errors (including
    exceptions -- Eio raises, it does not return Error) come back as
    {ok = false; error}. *)
```

```
let run_round ~(transport : transport) ~(lens : Lens_store.lens)
    ~(quiet : bool) () : round_report =
  let log fmt =
    Printf.ksprintf (fun s -> prerr_endline ("[sync] " ^ s)) fmt
  in
  let report = ref (empty_report ()) in
  (try
     let manifest = data_manifest ~lens in
     let bodies =
       List.filter_map
         (fun (p, _) ->
           match Lens_store.get ~lens p with
           | Some env -> Some (Ast.VList [ Ast.VStr p; Ast.VStr env ])
           | None -> None)
         manifest
     in
     let manifest_v =
       Ast.VList
         (List.map
            (fun (p, h) -> Ast.VList [ Ast.VStr p; Ast.VStr h ])
            manifest)
     in
     let base = Lens_store.base_manifest ~lens in
     let resp =
       if base = [] then
         (* no recorded base yet (fresh lens): two-way round -- additions
            + LWW only. Deletions need the three-way, which exists once
            the first successful round pins the base. *)
         transport "sync/push" [ manifest_v; Ast.VList bodies ]
       else
         let base_v =
           Ast.VList
             (List.map
                (fun (p, h) -> Ast.VList [ Ast.VStr p; Ast.VStr h ])
                base)
         in
         transport "sync/push" [ manifest_v; Ast.VList bodies; base_v ]
     in
     match resp with
     | Error e ->
         report :=
           { !report with
             ok = false;
             error = Some (Printexc.to_string e) };
         log "failed: %s" (Printexc.to_string e)
     | Ok (Ast.VStr json) -> (
         match Yojson.Safe.from_string json with
         | `Assoc kv ->
             let int k =
               match List.assoc_opt k kv with Some (`Int n) -> n | _ -> 0
             in
             let strs k =
               match List.assoc_opt k kv with
               | Some (`List xs) ->
                   List.filter_map (function `String s -> Some s | _ ->
    None) xs
               | _ -> []
             in
             let str k =
               match List.assoc_opt k kv with
               | Some (`String s) -> s | _ -> ""
             in
             let server_only = strs "server_only_paths" in
             let pull_paths = strs "pull_paths" in
             let pulls =
               (* three-way rounds give an explicit pull list (server-only
                  + LWW-server-kept convergence); two-way rounds just
                  server_only_paths. *)
               if pull_paths = [] then server_only else pull_paths
             in
             let client_deletes = strs "client_delete_paths" in
             let diverge = strs "diverge_paths" in
             let won = strs "lww_client_won" in
```

```
            let kept = strs "lww_server_kept" in
            let gen_str = str "generation" in
            let gen = try Int64.of_string gen_str with _ -> 0L in
            (* pull server copies (tree/get bypasses the rim's lens-local
               reads on purpose: the pull must come from the server) *)
            let pulled =
              List.fold_left
                (fun n p ->
                  match transport "tree/get" [ Ast.VStr p ] with
                  | Ok (Ast.VStr env) ->
                      ignore (Lens_store.materialize ~lens env);
                      if not quiet then log "pulled %s" p;
                      n + 1
                  | Ok _ -> n
                  | Error e ->
                      log "pull %s: %s" p (Printexc.to_string e);
                      n)
                0 pulls
            in
            (* mirror server deletions (Ours_del): the stale local copy
               of a node the server deleted goes too *)
            let deleted =
              List.fold_left
                (fun n p ->
                  if Lens_store.delete_node ~lens p then begin
                    if not quiet then log "deleted local %s (server
deleted)" p;
                    n + 1
                  end
                  else n)
                0 client_deletes
            in
            (* the obituary rule, applied to sync losses: every LWW
               resolution is surfaced with its winner *)
            List.iter (fun p -> log "LWW %s: client edit wins" p) won;
            List.iter (fun p -> log "LWW %s: server copy kept (client edit
lost)" p) kept;
            let branch =
              match
                (try transport "current-branch" [] with _ -> Error
(Failure "offline"))
              with
              | Ok (Ast.VStr b) -> b
              | _ -> ""
            in
            (* pin the merged state as the NEXT round's base + the
               (branch, generation) merge-base coordinate *)
            (match List.assoc_opt "merged_manifest" kv with
             | Some (`List rows) ->
                 let m =
                   List.filter_map
                     (function
                       | `List [ `String p; `String h ] -> Some (p, h)
                       | _ -> None)
                     rows
                 in
                 if m <> [] then Lens_store.record_base_manifest ~lens m
             | _ -> ());
            Lens_store.record_snapshot ~lens ~branch ~generation:gen;
            let activity =
              int "applied" + pulled + deleted + List.length kept
            in
            if not quiet then
              log "applied=%d pulled=%d deleted=%d diverge=%d gen=%s"
                (int "applied") pulled deleted (List.length diverge)
gen_str
            else if activity > 0 || List.length diverge > 0 then
              log "pushed=%d pulled=%d deleted=%d diverge=%d (won=%d
lost=%d) gen=%s"
                (int "applied") pulled deleted (List.length diverge)
                (List.length won) (List.length kept) gen_str;
            report :=
              { ok = true;
```

```
                error = None;
                pushed = int "applied";
                pulled;
                deleted_local = deleted;
                diverge = List.length diverge;
                lww_client_won = won;
                lww_server_kept = kept;
                branch;
                generation = gen }
        | _ ->
            report := { !report with ok = false; error = Some "bad
  response shape" };
            log "bad response shape")
    | Ok _ ->
        report := { !report with ok = false; error = Some "unexpected
  return value" };
        log "unexpected return value"
 with exn ->
   report :=
     { !report with
       ok = false;
       error = Some (Printexc.to_string exn) };
   log "failed: %s" (Printexc.to_string exn));
!report
```

294 remote/test/def_cache_test.ml

```
(** Tests for borge/remote-eval.borg caching subsection. No network --
    the [Proxy_rim.transport] is a mock serving the two proxyable read
    prims the def cache reads: [current-branch] (VStr, branch namespace)
    and [gen-now] (VInt, migration-017 branch_generations = the tree-time
    gen coordinate).

    Covers the three verification cases:
      1. a write in the client's branch ancestry bumps the generation;
         the next cache-check round trip observes the bump and reloads
         affected defs (the stale def is returned for eviction and
         dropped from the table, so the next lookup misses -> re-fetch).
      2. a checkout to a different branch invalidates the WHOLE local
         def cache (branch is the namespace).
      3. with no writes, the cache-check round trip is the single cheap
         HEAD-ish (two single-counter reads) returning unchanged-
         generation -- it does NOT do an N-way hash diff and mutates
         nothing.
    Plus a client-integration case: the lazy env re-fetches a def once
    the generation bumps (the [function-source] prim is called again and
    the NEW source is what eval uses). *)

open Pricklypear_lisp
open Ast

(** A mock transport over a mutable branch/gen coordinate + a per-name
    def-source feed, so the def cache's read prims can be driven
    (bump gen / switch branch) across cache-checks. *)
let coord_transport
    ?(branch = "user/personal") ?(gen = 0L) ?(defs : (string -> string)
     option) ()
    : Proxy_rim.transport
      * (string -> unit)
      * (unit -> unit)
      * (unit -> string * int64)
      * (string * value list) list ref =
  let b = ref branch in
  let g = ref gen in
  let calls = ref [] in
  let set_branch nb = b := nb in
  let bump_gen () = g := Int64.add !g 1L in
  let get_coord () = (!b, !g) in
  let fn name args =
    calls := !calls @ [ (name, args) ];
    match name with
    | "current-branch" -> Ok (VStr !b)
    | "gen-now" -> Ok (VInt !g)
    | _ -> (
        match defs with
        | Some feed -> (
            match args with
            | [ VStr n ] ->
                let s = feed n in
                if s = "" then Ok VNil else Ok (VStr s)
            | _ -> Ok VNil)
        | None ->
            Error
              (Interp_error
                 (Prim_error (name, "unserved", "prim_error"))))
  in
  (fn, set_branch, bump_gen, get_coord, calls)

(** Test 1: a generation bump (write-in-ancestry) invalidates stale
    defs. Load a def at gen 5; bump gen-now to 6; the cache-check round
    trip observes the bump and returns the stale name for eviction (the
    entry is dropped, so a subsequent lookup misses -- the client would
    re-fetch). A def loaded AT the new generation survives a second
    check at that same generation (only affected defs reload). *)
let test_generation_bump_reloads_stale () =
  let transport, _sb, bump, _coord, _calls = coord_transport ~gen:5L () in
  let c = Def_cache.create ~transport () in
  ignore (Def_cache.check c); (* pins (branch, gen=5) *)
```

```
71    Def_cache.store c "inc" (VInt 1L); (* loaded at gen 5 *)
72    Def_cache.store c "sq" (VInt 2L); (* loaded at gen 5 *)
73    Alcotest.(check int) "two defs cached" 2 (Def_cache.size c);
74    (match Def_cache.lookup c "inc" with
75     | Some _ -> ()
76     | None -> Alcotest.fail "def should be cached at gen 5");
77    (* a write bumps the generation to 6 *)
78    bump ();
79    (* cache-check observes the bump and flags the stale defs for eviction *)
80    let stale = Def_cache.check c in
81    Alcotest.(check bool) "check reported a change (stale names returned)"
        true
82      (stale <> []);
83    Alcotest.(check int) "both stale defs flagged for eviction" 2
        (List.length stale);
84    Alcotest.(check bool) "inc flagged stale" true (List.mem "inc" stale);
85    (* table dropped the stale defs: next lookup misses -> re-fetch *)
86    (match Def_cache.lookup c "inc" with
87     | Some _ -> Alcotest.fail "stale def should have been dropped (reload on
        next use)"
88     | None -> Alcotest.(check bool) "stale def dropped" true true);
89    Alcotest.(check int) "cache empty after eviction" 0 (Def_cache.size c)
90
91  (** Test 2: a checkout to a different branch invalidates the WHOLE
92      cache (branch is the namespace), regardless of generation. Every
93      cached name is returned for eviction. *)
94  let test_branch_switch_invalidates_all () =
95    let transport, set_branch, _bump, _coord, _calls =
96      coord_transport ~branch:"user/personal" ~gen:3L ()
97    in
98    let c = Def_cache.create ~transport () in
99    ignore (Def_cache.check c);
100   Def_cache.store c "inc" (VInt 1L);
101   Def_cache.store c "dec" (VInt 2L);
102   Def_cache.store c "sq" (VInt 3L);
103   Alcotest.(check int) "three defs cached" 3 (Def_cache.size c);
104   (* switch branch, same generation *)
105   set_branch "admin/user";
106   let stale = Def_cache.check c in
107   Alcotest.(check int) "check reported a branch change (all names evicted)"
        3
108     (List.length stale);
109   Alcotest.(check bool) "all three names flagged" true
110     (List.length (List.filter (fun n -> List.mem n stale) [ "inc"; "dec";
        "sq" ]) = 3);
111   Alcotest.(check int) "WHOLE cache cleared on branch switch" 0
        (Def_cache.size c);
112   (match Def_cache.lookup c "inc" with
113    | Some _ -> Alcotest.fail "branch switch must drop everything"
114    | None -> ())
115
116 (** Test 3: with no writes, the cache-check round trip is the single
117     cheap HEAD-ish (two single-counter reads) returning unchanged-
118     generation -- it does NOT do an N-way hash diff and mutates nothing
119     (a cached def survives, [] returned, no fetch of def sources). *)
120 let test_no_writes_unchanged_single_roundtrip () =
121   let transport, _sb, _bump, _coord, calls = coord_transport ~gen:9L () in
122   let c = Def_cache.create ~transport () in
123   ignore (Def_cache.check c); (* pin *)
124   Def_cache.store c "inc" (VInt 1L);
125   calls := []; (* forget pin-roundtrip calls *)
126   (* a second check at the SAME branch+gen: unchanged *)
127   let stale = Def_cache.check c in
128   Alcotest.(check bool) "unchanged-generation -> no evictions" true (stale
        = []);
129   Alcotest.(check int) "still cached after unchanged check" 1
        (Def_cache.size c);
130   (* exactly the two cheap counter reads, nothing else (no N-way diff) *)
131   Alcotest.(check int) "one cheap round trip = 2 counter reads" 2
132     (List.length !calls);
133   List.iter
134     (fun (n, _) ->
135       Alcotest.(check bool) "only counter read prims cross the wire" true
```

```
        (n = "current-branch" || n = "gen-now"))
    !calls

(** Integration: the client's lazy env uses the def cache so a def
    fetched at gen N is re-fetched (reloaded) once the generation bumps
    -- the second eval re-fetches the NEW source from the server and uses
    it, because the stale env binding is evicted. *)
let test_client_lazy_env_reloads_after_bump () =
  let gen = ref 5L in
  let transport name args =
    match name with
    | "current-branch" -> Ok (VStr "user/personal")
    | "gen-now" -> Ok (VInt !gen)
    | "function-source" -> (
        match args with
        | [ VStr "inc" ] ->
            if Int64.compare !gen 6L < 0 then Ok (VStr "(define (inc x) (+ x 1))")
            else Ok (VStr "(define (inc x) (+ x 100))")
        | _ -> Ok VNil)
    | _ -> Error (Interp_error (Prim_error (name, "unserved", "prim_error")))
  in
  let t = Remote.make ~transport () in
  (* first use: gen 5 -> inc x => (+ x 1); (inc 41) = 42 *)
  let v1 = Remote.eval t "(inc 41)" in
  Alcotest.(check int) "inc at gen5 = 42" 42
    (match v1 with VInt n -> Int64.to_int n | _ -> -1);
  (* bump the server generation to 6 -> the next use re-fetches v2 source *)
  gen := 6L;
  let v2 = Remote.eval t "(inc 41)" in
  Alcotest.(check int) "inc after bump = 141 (reloaded v2 source)" 141
    (match v2 with VInt n -> Int64.to_int n | _ -> -1)

let () =
  Alcotest.run "remote-def-cache"
    [
      ( "caching",
        [
          Alcotest.test_case "generation bump reloads stale defs" `Quick
            test_generation_bump_reloads_stale;
          Alcotest.test_case "branch switch invalidates whole cache" `Quick
            test_branch_switch_invalidates_all;
          Alcotest.test_case "no writes = single cheap unchanged round trip" `Quick
            test_no_writes_unchanged_single_roundtrip;
          Alcotest.test_case "client lazy env reloads after generation bump" `Quick
            test_client_lazy_env_reloads_after_bump;
        ] );
    ]
```

295 remote/test/free_vars_test.ml

```
(** Tests for remote/lib/free_vars.ml -- free-variable analysis over a
    Nopales AST. Pure (no I/O, no env); mirrors the worker's purity.

    The reader produces special forms as [App(Var "lambda", ?)] (eval
    desugars later), so the walker must dispatch on those App forms --
    binding lambda params / let names so they aren't mistaken for deps,
    treating quote bodies as data, and skipping kernel special-form
    names. These tests pin that behavior against the reader's actual
    output (not hand-built AST). *)

open Pricklypear_lisp

(** Free vars of a source string, sorted for order-independent compares. *)
let fv (src : string) : string list =
  List.sort compare (Free_vars.free_vars_of_source src)

(** All distinct free names in [src], as a sorted list. *)
let check label expected src =
  Alcotest.(check (list string)) label expected (fv src)

let () =
  Alcotest.run "free_vars"
  [ ( "basics"
    , [ Alcotest.test_case "plain var" `Quick (fun () ->
          check "plain var" [ "x" ] "x")
      ; Alcotest.test_case "lambda binds params" `Quick (fun () ->
          (* (lambda (a b) a) -- a,b bound; no free vars *)
          check "lambda binds params" [] "(lambda (a b) a)")
      ; Alcotest.test_case "lambda with free var" `Quick (fun () ->
          (* (lambda (a) b) -- a bound, b free *)
          check "lambda free b" [ "b" ] "(lambda (a) b)")
      ; Alcotest.test_case "nested call" `Quick (fun () ->
          (* (foo (bar x)) -- foo, bar, x all free *)
          check "nested call" [ "bar"; "foo"; "x" ] "(foo (bar x))")
      ; Alcotest.test_case "pure prim shows as dep (conservative)" `Quick
    (fun () ->
          (* (+ 1 2) -- + is a PRIM, not a kernel special form, so it
             shows as free. The puller asks function-source, gets nil,
             skips. This is the intended conservative behavior. *)
          check "+ is free" [ "+" ] "(+ 1 2)")
      ; Alcotest.test_case "stdlib call is a dep" `Quick (fun () ->
          (* (cons 1 xs) -- cons is a local pure prim (not kernel special),
             but xs is free. cons IS registered as a pure prim on the
             client, but free_vars only skips kernel special-form names,
             not pure prims -- so cons shows as free. That's fine: the
             puller will ask function-source for "cons", get nil (it's a
             prim), and skip it. Pin the actual behavior. *)
          check "cons + xs" [ "cons"; "xs" ] "(cons 1 xs)")
      ] )
  ; ( "special-forms"
    , [ Alcotest.test_case "if branches" `Quick (fun () ->
          (* (if t a b) -- t, a, b free; if is kernel *)
          check "if" [ "a"; "b"; "t" ] "(if t a b)")
      ; Alcotest.test_case "let binds" `Quick (fun () ->
          (* (let ((a 1) (b 2)) (+ a b)) -- a,b bound; + is a prim (not a
             kernel special form) so it shows free; puller skips. *)
          check "let binds" [ "+" ] "(let ((a 1) (b 2)) (+ a b))")
      ; Alcotest.test_case "let val free" `Quick (fun () ->
          (* (let ((a x)) a) -- a bound, x free in the val *)
          check "let val free" [ "x" ] "(let ((a x)) a)")
      ; Alcotest.test_case "define binds name + params" `Quick (fun () ->
          (* (define (f x) (g x)) -- f, x bound; g free *)
          check "define binds" [ "g" ] "(define (f x) (g x))")
      ; Alcotest.test_case "define bare lambda" `Quick (fun () ->
          (* (define f (lambda (x) (g x))) -- f bound, x bound, g free *)
          check "define bare lambda" [ "g" ] "(define f (lambda (x) (g
    x)))")
      ; Alcotest.test_case "quote is data" `Quick (fun () ->
          (* '(a b c) -- quoted, not code; no free vars *)
          check "quote data" [] "'(a b c)")
      ; Alcotest.test_case "quasiquote is data" `Quick (fun () ->
```

```
70          (* `(a b) -- quasiquoted; no free vars (unquote handled too) *)
71          check "quasiquote data" [] "`(a b)")
72        ; Alcotest.test_case "seq/cond" `Quick (fun () ->
73          (* (seq (f) (g)) -- f, g free; seq kernel *)
74          check "seq" [ "f"; "g" ] "(seq (f) (g))")
75        ; Alcotest.test_case "cond branches" `Quick (fun () ->
76          (* (cond (a b) (else c)) -- cond kernel; a,b,c free; `else` is
77             a cond keyword but not a kernel special-form name, so it
78             shows as free (conservative -- puller gets nil, skips). *)
79          check "cond" [ "a"; "b"; "c"; "else" ] "(cond (a b) (else c))")
80        ; Alcotest.test_case "lambda no spurious 'lambda' dep" `Quick (fun ()
      ->
81          (* The bug that started these tests: the reader produces
82             App(Var "lambda", ?), so a naive walker treated "lambda"
83             as a free var. Pin that it does NOT. *)
84          check "no lambda dep" [ "g" ] "(lambda (x) (g x))")
85        ; Alcotest.test_case "and/or" `Quick (fun () ->
86          check "and/or" [ "a"; "b" ] "(and a b)")
87        ] )
88    ; ( "transitive"
89      , [ Alcotest.test_case "3-deep chain" `Quick (fun () ->
90          (* (define (top n) (- (mid n) 1)) -- top,n bound; mid free; - is
91             a prim (shows free, puller skips). *)
92          check "top" [ "-"; "mid" ] "(define (top n) (- (mid n) 1))")
93        ; Alcotest.test_case "higher-order list ops" `Quick (fun () ->
94          (* (list/map f xs) -- list/map kernel; f, xs free *)
95          check "list/map" [ "f"; "xs" ] "(list/map f xs)")
96        ; Alcotest.test_case "with-transaction body" `Quick (fun () ->
97          (* (with-transaction (lambda () (create 'note 'n1))) --
98             with-transaction kernel; create free (it's a proxy prim,
99             shows as a dep -- the puller will ask function-source, get
100            nil, skip). Pin actual behavior. *)
101         check "with-tx" [ "create" ]
102           "(with-transaction (lambda () (create 'note 'n1)))")
103       ] )
104   ]
```

296 remote/test/lens_store_test.ml

```
(** Tests for borge/local-first.borg lens-model + replication-units +
    local-substrate (the remote/lib/lens_store.ml module). No network, no
    Postgres -- pure filesystem + JSON, mirroring how the worker itself is
    pure.

    Verifies the pinned contracts from the campaign 8.0 plan pass:
      - fork_lens_name produces <owner>/lens-<device>[-<date>] (lens-model
        fork naming for the (fork ...)/Store.fork_branch child branch).
      - in_replication_unit confines replication to exactly the
        authorized-read subtree: data.u.<user>.* (full), data.shared.*
        (read-mostly seed), b.<lens>.* (the forked code lens) - and NOTHING
        outside it (another user's data.u.<other>.* never replicates, the
        one-tree path-prefix RLS floor is the replication boundary).
      - the substrate (local-substrate): materialize writes one node
        envelope file per node at its path under the lens root; get /
        subtree_list are the path-addressed reads (the filesystem IS the
        path index); put_body + node_hash are the local write path whose
        content hash is the sync/three-way-diff key. *)

open Pricklypear_lisp
open Lens_store

let tmpdir () : string =
  let base = Filename.get_temp_dir_name () in
  let dir = Filename.concat base ("lens-test-" ^ string_of_int (Unix.getpid
     ()) ^ "-" ^ string_of_int (Random.int 100000)) in
  Unix.mkdir dir 0o755;
  dir

let with_tmpdir f =
  let dir = tmpdir () in
  Fun.protect ~finally:(fun () -> (try Unix.rmdir dir with _ -> ()))
    (fun () -> f dir)

let lens_of ~root =
  { root; owner = "admin"; device = "laptop"; branch = "lens-laptop";
     generation = 7L }

(* --- fork naming (lens-model) --- *)
let test_fork_lens_name () =
  Alcotest.(check string) "no date"
    "admin/lens-laptop" (fork_lens_name ~owner:"admin" ~device:"laptop" ());
  Alcotest.(check string) "with date"
    "admin/lens-laptop-2026-08-14"
    (fork_lens_name ~date:"2026-08-14" ~owner:"admin" ~device:"laptop" ())

(* --- replication unit (replication-units, path-prefix confinement) --- *)
let test_replication_unit () =
  let user = "admin" in
  let lens_prefix = "aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa" in  (* 32-char uuid
     *)
  (* own data: in *)
  Alcotest.(check bool) "own data.u.admin" true
    (in_replication_unit ~lens_prefix:lens_prefix ~user
       ("data.u." ^ enc_label "admin" ^ ".todo.abc"));
  (* another user: OUT -- never replicates *)
  Alcotest.(check bool) "other user data.u.bob" false
    (in_replication_unit ~lens_prefix:lens_prefix ~user
       ("data.u." ^ enc_label "bob" ^ ".todo.xyz"));
  (* shared seed: in *)
  Alcotest.(check bool) "data.shared" true
    (in_replication_unit ~lens_prefix:lens_prefix ~user
       ("data.shared.bible-verse.abc"));
  (* the forked code lens: in *)
  Alcotest.(check bool) "forks lens code" true
    (in_replication_unit ~lens_prefix:lens_prefix ~user
       ("b." ^ lens_prefix ^ ".fn.todo-get"));
  (* a DIFFERENT lens / code branch: out *)
  Alcotest.(check bool) "other lens branch" false
    (in_replication_unit ~lens_prefix:lens_prefix ~user
       ("b.bbbbbbbbbbbbbbbbbbbbbbbbbbbbbbbb.fn.other"));
```

```
  (* non-data, non-lens root: out *)
  Alcotest.(check bool) "external root" false
    (in_replication_unit ~lens_prefix:lens_prefix ~user "sys.meta")

(* --- substrate path mapping --- *)
let test_path_to_file () =
  let root = "/tmp/lensroot" in
  Alcotest.(check string) "fn path" (root ^ "/b/aaaa/fn/todo-get.json")
    (path_to_file ~root "b.aaaa.fn.todo-get");
  Alcotest.(check string) "datum path"
    (root ^ "/data/u/admin/todo/id123.json")
    (path_to_file ~root "data.u.admin.todo.id123")

(* --- substrate read/write round trips --- *)
let test_materialize_and_get () =
  with_tmpdir (fun root ->
    let lens = lens_of ~root in
    let path = "data.u.abc.todo.id1" in
    let env =
      Yojson.Safe.to_string
        (`Assoc [ ("v", `Int 1); ("kind", `String "datum");
                  ("path", `String path); ("type", `String "todo");
                  ("body", `Assoc [("title", `String "hi")]);
                  ("author", `String "admin"); ("hash", `String "h1");
                  ("created_at", `String "t1"); ("updated_at", `String
    "t2") ])
    in
    let file = match materialize ~lens env with Some f -> f | None ->
    Alcotest.fail "materialize returned None" in
    Alcotest.(check bool) "file exists" true (Sys.file_exists file);
    (match get ~lens path with
     | Some j -> Alcotest.(check string) "round trip" env j
     | None -> Alcotest.fail "get returned None");
    Alcotest.(check string) "cleaned path" (root ^
    "/data/u/abc/todo/id1.json") file)

let test_put_body_hash () =
  with_tmpdir (fun root ->
    let lens = lens_of ~root in
    let body = `Assoc [("title", `String "offline edit")] in
    put_body ~lens ~path:"data.u.abc.todo.id2" ~kind:"datum" ~type_:"todo"
      ~author:"admin" ~body;
    Alcotest.(check bool) "get present" true (get ~lens
    "data.u.abc.todo.id2" <> None);
    (* the envelope carries the content hash of the canonical body *)
    (match node_hash ~lens "data.u.abc.todo.id2" with
     | Some h ->
         let expected = content_hash (Yojson.Safe.to_string body) in
         Alcotest.(check string) "hash = content hash" expected h
     | None -> Alcotest.fail "node_hash None");
    (* content_hash is deterministic *)
    Alcotest.(check string) "deterministic hash"
      (content_hash "x") (content_hash "x"))

let test_subtree_list () =
  with_tmpdir (fun root ->
    let lens = lens_of ~root in
    List.iter
      (fun p ->
         let env = Yojson.Safe.to_string
             (`Assoc [ ("v", `Int 1); ("kind", `String "datum");
                       ("path", `String p); ("type", `String "todo");
                       ("body", `Assoc []); ("author", `String "admin");
                       ("hash", `String "h"); ("created_at", `String "t");
                       ("updated_at", `String "t") ]) in
         ignore (materialize ~lens env))
      [ "data.u.abc.todo.a"; "data.u.abc.todo.b"; "data.u.abc.note.c" ];
    let todos = subtree_list ~lens "data.u.abc.todo" in
    Alcotest.(check int) "two todo paths under prefix" 2 (List.length
    todos);
    Alcotest.(check bool) "contains a" true (List.mem "data.u.abc.todo.a"
    todos);
```

```
    Alcotest.(check bool) "contains b" true (List.mem "data.u.abc.todo.b"
     todos);
    (* the note path is NOT under the todo prefix *)
    Alcotest.(check bool) "note not under todo" false (List.mem
     "data.u.abc.note.c" todos))

(* --- code-lens + ancestry materialization (local-substrate 8.2) --- *)

let test_code_kind_path () =
  Alcotest.(check (option string)) "fn path"
    (Some "b.admin.fn.todo-get")
    (code_kind_path ~branch:"admin" "fn" ~name:"todo-get");
  Alcotest.(check (option string)) "route path"
    (Some "b.admin.route./issues")
    (code_kind_path ~branch:"admin" "route" ~name:"/issues");
  Alcotest.(check (option string)) "non-code kind rejected" None
    (code_kind_path ~branch:"admin" "datum" ~name:"x")

let test_materialize_code_and_resolve () =
  with_tmpdir (fun root ->
    let lens = lens_of ~root in
    (* lens owns an override; main is ancestry -- different source *)
    let lens_file =
      materialize_code_node ~lens ~branch:"lens-laptop" ~kind:"fn"
        ~name:"todo-get" ~src:"(define (todo-get id) 'lens)" ~doc:"lens own"
        ~author:"admin"
      |> function Some f -> f | None -> Alcotest.fail "materialize_code
     None"
    in
    Alcotest.(check bool) "code file exists" true (Sys.file_exists
     lens_file);
    ignore
      (materialize_code_node ~lens ~branch:"main" ~kind:"fn"
         ~name:"todo-get" ~src:"(define (todo-get id) 'main)" ~doc:"main"
         ~author:"admin");
    (* record ancestry self-first: lens then main *)
    record_ancestry ~lens [ "lens-laptop"; "main" ];
    Alcotest.(check (list string)) "ancestry round trip"
      [ "lens-laptop"; "main" ] (ancestry ~lens);
    (* closest-wins: the lens's own override resolves first *)
    (match resolve_def ~lens ~kind:"fn" ~name:"todo-get" with
     | Some env ->
         Alcotest.(check (option string)) "closest = lens's src"
           (Some "(define (todo-get id) 'lens)")
           (code_src_of_envelope env)
     | None -> Alcotest.fail "resolve_def None for own override");
    (* a def only on ancestry resolves through inheritance *)
    ignore
      (materialize_code_node ~lens ~branch:"main" ~kind:"fn"
         ~name:"shared-only" ~src:"(define (shared-only) 42)" ~doc:"main
     only"
         ~author:"admin");
    (match resolve_def ~lens ~kind:"fn" ~name:"shared-only" with
     | Some env ->
         Alcotest.(check (option string)) "inherited from main"
           (Some "(define (shared-only) 42)")
           (code_src_of_envelope env)
     | None -> Alcotest.fail "resolve_def None for ancestry def");
    (* a def on NO branch does not resolve *)
    Alcotest.(check bool) "unknown def is None" true
      (resolve_def ~lens ~kind:"fn" ~name:"nope" = None))

let test_node_manifest () =
  with_tmpdir (fun root ->
    let lens = lens_of ~root in
    (* one datum + one code node *)
    ignore
      (materialize ~lens
         (Yojson.Safe.to_string
            (`Assoc [ ("v", `Int 1); ("kind", `String "datum");
                      ("path", `String "data.u.admin.todo.a");
                      ("type", `String "todo"); ("body", `Assoc []);
                      ("author", `String "admin"); ("hash", `String "h1");
```

```
                      ("created_at", `String "t"); ("updated_at", `String
    "t") ])));
    ignore
      (materialize_code_node ~lens ~branch:"lens-laptop" ~kind:"fn"
         ~name:"f" ~src:"(define (f) 1)" ~doc:"" ~author:"admin");
    let manifest = node_manifest ~lens in
    Alcotest.(check int) "two node entries" 2 (List.length manifest);
    Alcotest.(check bool) "datum path present" true
      (List.mem_assoc "data.u.admin.todo.a" manifest);
    Alcotest.(check bool) "code path present" true
      (List.mem_assoc "b.lens-laptop.fn.f" manifest);
    (* the ancestry metadata file is NOT in the manifest *)
    Alcotest.(check bool) "no ancestry in manifest" false
      (List.exists (fun (p, _) -> String.length p > 0 && p.[0] = '_')
    manifest))

let test_lens_fetch_first () =
  (* the offline-first fetch order: lens substrate before the wire. We
     assert
     that Remote.fetch_def consults the lens [lens_lookup] first by giving
     it
     a transport that would FAIL, so only a substrate hit can satisfy it. *)
  with_tmpdir (fun root ->
    let lens = lens_of ~root in
    ignore
      (materialize_code_node ~lens ~branch:"lens-laptop" ~kind:"fn"
         ~name:"off" ~src:"(define (off) 7)" ~doc:"" ~author:"admin");
    record_ancestry ~lens [ "lens-laptop" ];
    let lookup name =
      match resolve_def ~lens ~kind:"fn" ~name with
      | Some env -> code_src_of_envelope env
      | None -> None
    in
    let fail_transport (_ : string) (_ : Pricklypear_lisp.Ast.value list) :
        (Pricklypear_lisp.Ast.value, exn) result =
      Error (Failure "wire must not be touched")
    in
    let client = Remote.make ~transport:fail_transport ~lens_lookup:lookup
    () in
    (* offline-first: resolves from the substrate without touching the wire
    *)
    match Remote.fetch_def client "off" with
    | Some src -> Alcotest.(check string) "lens-first src" "(define (off)
    7)" src
    | None -> Alcotest.fail "fetch_def None despite substrate hit")

let test_delete_node () =
  (* Ours_del mirror: the server deleted the node, the client's stale
     copy goes too. delete_node removes the envelope file, prunes empty
     parents, and is false on an absent path. *)
  with_tmpdir (fun root ->
    let lens = lens_of ~root in
    let path = "data.u.admin.todo.gone" in
    ignore
      (materialize ~lens
         (Yojson.Safe.to_string
            (`Assoc [ ("v", `Int 1); ("kind", `String "datum");
                      ("path", `String path);
                      ("type", `String "todo"); ("body", `Assoc []);
                      ("author", `String "admin"); ("hash", `String "h9");
                      ("created_at", `String "t"); ("updated_at", `String
    "t") ])));
    Alcotest.(check bool) "node present before delete" true
      (get ~lens path <> None);
    Alcotest.(check bool) "delete removes" true (delete_node ~lens path);
    Alcotest.(check bool) "node gone" true (get ~lens path = None);
    Alcotest.(check bool) "delete absent is false" false
      (delete_node ~lens path))

let test_base_manifest () =
  (* the sync common ancestor, client side: record the last-synced merged
     manifest, read it back as the next round's base; it must NOT appear
     in the node manifest (metadata file). *)
```

```
  with_tmpdir (fun root ->
    let lens = lens_of ~root in
    Alcotest.(check (list (pair string string))) "no base on a fresh lens"
     []
      (base_manifest ~lens);
    record_base_manifest ~lens
      [ ("data.u.admin.todo.a", "h1"); ("data.u.admin.todo.b", "h2") ];
    let base = base_manifest ~lens in
    Alcotest.(check int) "two base entries" 2 (List.length base);
    Alcotest.(check bool) "base entry round-trips" true
      (List.mem_assoc "data.u.admin.todo.a" base);
    (* a datum node coexisting with the base file still manifests; the
       base file itself never does *)
    ignore
      (materialize ~lens
         (Yojson.Safe.to_string
            (`Assoc [ ("v", `Int 1); ("kind", `String "datum");
                      ("path", `String "data.u.admin.todo.a");
                      ("type", `String "todo"); ("body", `Assoc []);
                      ("author", `String "admin"); ("hash", `String "h1");
                      ("created_at", `String "t"); ("updated_at", `String
    "t") ])));
    let manifest = node_manifest ~lens in
    Alcotest.(check (list string)) "manifest holds the datum only"
      [ "data.u.admin.todo.a" ] (List.map fst manifest))

let () =
  Random.self_init ();
  Alcotest.run "lens_store"
    [ ("lens-model", [ Alcotest.test_case "fork_lens_name" `Quick
    test_fork_lens_name ])
    ; ("replication-units", [ Alcotest.test_case "in_replication_unit"
    `Quick test_replication_unit ])
    ; ("local-substrate"
       , [ Alcotest.test_case "path_to_file" `Quick test_path_to_file
         ; Alcotest.test_case "materialize_and_get" `Quick
    test_materialize_and_get
         ; Alcotest.test_case "put_body_hash" `Quick test_put_body_hash
         ; Alcotest.test_case "subtree_list" `Quick test_subtree_list
         ; Alcotest.test_case "code_kind_path" `Quick test_code_kind_path
         ; Alcotest.test_case "materialize_code_and_resolve" `Quick
    test_materialize_code_and_resolve
         ; Alcotest.test_case "node_manifest" `Quick test_node_manifest
         ; Alcotest.test_case "delete_node" `Quick test_delete_node
         ; Alcotest.test_case "base_manifest" `Quick test_base_manifest
         ; Alcotest.test_case "lens_fetch_first" `Quick
    test_lens_fetch_first ])
    ]
```

297 remote/test/loop_test.ml

```
(** Tests for borge/remote-eval.borg agent-loop: the SHARED LLM turn
    loop (llm/) running over the CLIENT host seam. No network -- the
    provider is a canned script (mock complete) and the transport is a
    mock. Verifies the thesis end-to-end: the provider call runs on
    this process, a Nopales plugin tool's body evals under the proxy
    rim (its DB prim crosses to /api/prim exactly once), builtin tool
    bodies stay LOCAL (zero wire), and (on "tool_call" ?) hooks block
    identically to the agent host. *)

open Pricklypear_lisp
open Ast

(** A mock transport: records (name, args) calls and answers from a
    table (same shape as remote_test.ml's). *)
let mock_transport
    ?(fallback : (string -> value list -> (value, exn) Stdlib.result)
     option)
    ()
    : Proxy_rim.transport * (string * value list) list ref
      * (string, value list -> (value, exn) Stdlib.result) Hashtbl.t =
  let calls = ref [] in
  let table = Hashtbl.create 8 in
  let fn name args =
    calls := !calls @ [ (name, args) ];
    match Hashtbl.find_opt table name with
    | Some f -> f args
    | None -> (
        match fallback with
        | Some g -> g name args
        | None ->
            Error
              (Interp_error
                 (Prim_error (name, "not in mock table", "prim_error"))))
  in
  (fn, calls, table)

let cfg : Provider.cfg =
  { base_url = "http://127.0.0.1:1"; api_key = ""; model = "mock"; system =
     "" }

(** A mock complete that emits one tool call then a final text. *)
let complete_with_tool_call name input =
  let script = ref 0 in
  fun ~env:_ ~sw:_ ~cfg:_ ~messages:_ ~tools:_ ~on_delta:_ ->
    let n = !script in
    incr script;
    if n = 0 then
      Ok
        {
          Provider.text = "";
          calls = [ { id = "t1"; name; input } ];
          stop_reason = "tool_use";
          raw = `Assoc [ ("role", `String "assistant"); ("content", `String
     "") ];
        }
    else
      Ok
        {
          Provider.text = "done";
          calls = [];
          stop_reason = "stop";
          raw = `Assoc [ ("role", `String "assistant"); ("content", `String
     "done") ];
        }

(** The thesis test: a Nopales plugin tool registered with
    (register-tool ?) whose body calls a DB prim (whoami). The loop runs
    over [Remote.loop_host]; the tool body evals under the proxy rim so
    whoami crosses to the transport EXACTLY ONCE; the tool result is
    appended to history and the loop makes its follow-up call. *)
let test_plugin_tool_proxies_db_prim ~env ~sw () =
```

```
  let transport, calls, table = mock_transport () in
  Hashtbl.replace table "whoami" (fun _args -> Ok (VStr "admin"));
  let t = Remote.make ~transport () in
  ignore
    (Remote.eval t
       {|(register-tool "who" (list) (lambda (args) (whoami)))|});
  Alcotest.(check bool) "tool registered in order" true
    (List.mem "who" t.Remote.tool_order);
  let events = ref [] in
  let history = ref [] in
  (match
     Loop.run ~env ~sw ~host:(Remote.loop_host t) ~cfg
       ~complete:(complete_with_tool_call "who" `Null)
       ~on_event:(fun e -> events := e :: !events)
       history "who am I"
   with
  | Error e -> Alcotest.fail e
  | Ok () -> ());
  (* the DB prim crossed the wire exactly once *)
  let whoami_calls = List.filter (fun (n, _) -> n = "whoami") !calls in
  Alcotest.(check int) "whoami crossed the wire exactly once" 1
    (List.length whoami_calls);
  (* the tool result reached the model as a tool-role message *)
  Alcotest.(check int) "history has 4 entries" 4 (List.length !history);
  (match List.nth !history 2 with
  | { Provider.role = "tool"; content = `String out; tool_call_id = Some
     "t1"; _ } ->
      Alcotest.(check string) "tool result is the proxied row" "admin" out
  | m ->
      Alcotest.fail
        ("bad tool message: " ^ m.Provider.role ^ " content="
         ^ (match m.Provider.content with
            | `String s -> s
            | c -> Yojson.Safe.to_string c)));
  (* Tool_end event carried the tool output to the stream *)
  let saw_tool_end =
    List.exists
      (fun e -> match e with Loop.Tool_end (_, o) -> o = "admin" | _ ->
     false)
      !events
  in
  Alcotest.(check bool) "Tool_end event carried output" true saw_tool_end

(** Builtin tools (read/bash) run as LOCAL prims through the proxy
    rim's local table -- a bash round trip makes ZERO wire crossings. *)
let test_builtin_tool_stays_local ~env ~sw () =
  let transport, calls, _ = mock_transport () in
  let t = Remote.make ~transport () in
  Tools.install ~resolve_path:Remote.resolve_path
    ~register:(fun name run -> Remote.register_builtin_tool t name ~run);
  let history = ref [] in
  (match
     Loop.run ~env ~sw ~host:(Remote.loop_host t) ~cfg
       ~complete:
         (complete_with_tool_call "bash"
            (`Assoc [ ("command", `String "echo hello") ]))
       ~on_event:(fun _ -> ()) history "run echo"
   with
  | Error e -> Alcotest.fail e
  | Ok () -> ());
  Alcotest.(check int) "builtin bash made zero wire crossings" 0
    (List.length !calls);
  (match List.nth !history 2 with
  | { role = "tool"; content = `String out; _ } ->
      (* {"exit":0,"out":"hello\n",...} *)
      Alcotest.(check bool) "bash output captured" true
        (try ignore (Str.search_forward (Str.regexp_string "hello") out 0);
     true
         with Not_found -> false)
  | _ -> Alcotest.fail "missing tool message")

(** A (on "tool_call" ?) hook blocking a call reaches the model as a
    blocked tool result -- same hook semantics as the agent host. *)
```

```
let test_tool_call_hook_blocks ~env ~sw () =
  let transport, _calls, _table = mock_transport () in
  let t = Remote.make ~transport () in
  Tools.install ~resolve_path:Remote.resolve_path
    ~register:(fun name run -> Remote.register_builtin_tool t name ~run);
  ignore
    (Remote.eval t
       {|(on "tool_call" (lambda (ev)
            (if (string-contains? (dict-get ev "input") "rm -rf")
                (dict-set (dict-set "{}" "block" "true") "reason" "nope")
                "{}")))|});
  let saw_block = ref false in
  let complete ~env:_ ~sw:_ ~cfg:_ ~messages ~tools:_ ~on_delta:_ =
    if not !saw_block then (
      saw_block :=
        List.exists
          (fun (m : Provider.message) ->
            match m.content with
            | `String s ->
                (try
                   ignore
                     (Str.search_forward (Str.regexp_string "blocked: nope") s 0);
                   true
                 with Not_found -> false)
            | _ -> false)
          messages;
      Ok
        {
          Provider.text = "";
          calls =
            [ { id = "t1"; name = "bash";
                input = `Assoc [ ("command", `String "rm -rf /") ] } ];
          stop_reason = "tool_use";
          raw = `Assoc [ ("role", `String "assistant"); ("content", `String "") ];
        })
    else
      Ok
        {
          Provider.text = "ok";
          calls = [];
          stop_reason = "stop";
          raw = `Assoc [ ("role", `String "assistant"); ("content", `String "ok") ];
        }
  in
  (match
     Loop.run ~env ~sw ~host:(Remote.loop_host t) ~cfg ~complete
       ~on_event:(fun _ -> ()) (ref []) "delete everything"
   with
  | Error e -> Alcotest.fail e
  | Ok () -> Alcotest.(check bool) "block reason reached model" true
     !saw_block)

(** The chat surface is Nopales-authored on the client route table:
    chat/page renders the hx-post /chat form, the chat/* fragment
    templates render, and / serves chat while /eval serves the eval
    box. *)
let test_chat_surface_nopales ~rt () =
  Surface.load ~rt;
  let render handler =
    match
      Remote.dispatch_route ~rt ~handler ~method_:"GET" ~path:"/"
        ~params:[] ~headers:[] ~body:""
    with
    | VStr s -> s
    | other -> Alcotest.fail ("bad render: " ^ Reader.pp_value other)
  in
  let chat = render "chat/page" in
  Alcotest.(check bool) "chat page has hx-post /chat" true
    (try ignore (Str.search_forward (Str.regexp_string "hx-post=\"/chat\"")
     chat 0); true
```

```
    with Not_found -> false);
  Alcotest.(check bool) "chat page loads hx-multipart asset" true
    (try
       ignore
         (Str.search_forward (Str.regexp_string "/assets/hx-multipart.js")
    chat 0);
       true
    with Not_found -> false);
  (match Remote.call_template ~rt "chat/user-bubble" [ VStr "hi" ] with
  | Some html ->
      Alcotest.(check bool) "user bubble template renders" true
        (try ignore (Str.search_forward (Str.regexp_string "bubble user")
    html 0); true
         with Not_found -> false)
  | None -> Alcotest.fail "chat/user-bubble unbound");
  (match Remote.call_template ~rt "chat/turn-open" [ VStr "t1" ] with
  | Some html ->
      Alcotest.(check bool) "turn-open template renders" true
        (try ignore (Str.search_forward (Str.regexp_string "id=\"t1\"")
    html 0); true
         with Not_found -> false)
  | None -> Alcotest.fail "chat/turn-open unbound");
  (* /eval still serves the eval box *)
  (match
    Remote.dispatch_route ~rt ~handler:"eval/page" ~method_:"GET"
      ~path:"/eval" ~params:[] ~headers:[] ~body:""
  with
  | VStr s ->
      Alcotest.(check bool) "eval box moved to /eval" true
        (try
           ignore (Str.search_forward (Str.regexp_string
    "hx-post=\"/eval\"") s 0);
           true
         with Not_found -> false)
  | other -> Alcotest.fail ("eval/page: " ^ Reader.pp_value other))

let () =
  Eio_main.run @@ fun env ->
  Eio.Switch.run @@ fun sw ->
  let stransport, _, _ = mock_transport () in
  let surface_rt = Remote.make ~transport:stransport () in
  Alcotest.run "remote-loop"
    [
      ( "agent-loop",
        [
          Alcotest.test_case "plugin-tool-proxies-db-prim" `Quick
            (test_plugin_tool_proxies_db_prim ~env ~sw);
          Alcotest.test_case "builtin-tool-stays-local" `Quick
            (test_builtin_tool_stays_local ~env ~sw);
          Alcotest.test_case "tool-call-hook-blocks" `Quick
            (test_tool_call_hook_blocks ~env ~sw);
        ] );
      ( "surfaces",
        [ Alcotest.test_case "chat-surface-nopales" `Quick
            (test_chat_surface_nopales ~rt:surface_rt) ] );
    ]
```

298 remote/test/placer_test.ml

```
(** Tests for borge/remote-eval.borg placement subsection. The placer
    decides LOCAL vs REMOTE(/api/eval) per expression. Verifies the
    four rules: transaction-wrapped ? Remote (hard correctness); zero
    prims ? Local; prim-dense above threshold ? Remote; the decision
    carries a human-readable reason. No network -- the placer is a pure
    AST walker. *)

open Pricklypear_lisp
open Ast

let cfg =
  let local_prims =
    [ "+"; "-"; "*"; "div"; "list"; "car"; "cdr"; "cons"; "not";
      "dict-get"; "string-append"; "str"; "count"; "null?" ]
  in
  Placer.default_cfg ~local_prims

let check_placement ~name expected src =
  let got = Placer.place ~cfg src in
  let got_s = Placer.pp_placement got in
  let exp_s = Placer.pp_placement expected in
  Alcotest.(check string) name exp_s got_s

let test_zero_prims_local () =
  check_placement ~name:"pure arithmetic is local" Local "(+ 1 2)";
  check_placement ~name:"pure list is local" Local "(cons 1 (list 2 3))";
  check_placement ~name:"nested pure is local" Local
    "(+ (+ 1 2) (- 3 1))"

let test_transaction_remote () =
  (* A transaction forces REMOTE regardless of prim count -- locks
     can't span HTTP. *)
  let p = Placer.place ~cfg "(with-transaction (lambda () (create 'note
    1)))" in
  (match p with
   | Remote (_, reason) ->
       Alcotest.(check bool) "reason mentions with-transaction" true
         (String.length reason > 0)
   | Local -> Alcotest.fail "transaction must be Remote");
  (* even a transaction with a pure body is Remote (the wrapper is the
    point) *)
  let p2 = Placer.place ~cfg "(with-transaction (lambda () (+ 1 2)))" in
  (match p2 with
   | Remote _ -> ()
   | Local -> Alcotest.fail "transaction wrapper alone must be Remote")

let test_prim_dense_remote () =
  (* Four proxied prim calls (get) above the default threshold 3 ? Remote *)
  let p =
    Placer.place ~cfg
      "(dict-get (get 'a 1) \"x\") (dict-get (get 'b 2) \"y\") (get 'c 3)
     (get 'd 4)"
  in
  (match p with
   | Remote (_, reason) ->
       Alcotest.(check bool) "reason is non-empty" true (String.length
     reason > 0)
   | Local -> Alcotest.fail "prim-dense above threshold must be Remote")

let test_below_threshold_local () =
  (* One proxied prim call (get) at/below threshold ? Local (proxy it) *)
  check_placement ~name:"single prim call is local" Local
    "(dict-get (get 'user 42) \"name\")"

let test_decision_carries_reason () =
  let p =
    Placer.place ~cfg
      "(get 'a 1) (get 'b 2) (get 'c 3) (get 'd 4) (get 'e 5)"
  in
  (match p with
   | Remote (where, reason) ->
```

```
      Alcotest.(check string) "routes to /api/eval" "/api/eval" where;
      Alcotest.(check bool) "reason explains the threshold" true
        (String.length reason > 0
         && (String.length
               (try ignore (Str.search_forward (Str.regexp "threshold") reason 0); "y"
                with Not_found -> "") > 0))
  | Local -> Alcotest.fail "prim-dense must be Remote with a reason")

let () =
  Alcotest.run "remote-placer"
    [
      ( "placement",
        [
          Alcotest.test_case "zero prims ? local" `Quick test_zero_prims_local;
          Alcotest.test_case "with-transaction ? remote" `Quick test_transaction_remote;
          Alcotest.test_case "prim-dense ? remote" `Quick test_prim_dense_remote;
          Alcotest.test_case "below threshold ? local" `Quick test_below_threshold_local;
          Alcotest.test_case "decision carries reason" `Quick test_decision_carries_reason;
        ] );
    ]
```

299 remote/test/remote_test.ml

```
(** Tests for borge/remote-eval.borg client subsection. No network --
    the [Proxy_rim.transport] is a mock that records calls and returns
    canned values. Verifies: pure compute stays local (zero prim calls),
    a prim call proxies and the result binds locally for subsequent
    pure use, lazy def miss fetches + evals, genuinely-unbound fails
    cleanly, and the client makes no local store write (it is a cache +
    interpreter, never a source of truth). *)

open Pricklypear_lisp
open Ast

let pp_value_testable =
  Alcotest.testable
    (fun fmt v -> Format.pp_print_string fmt (Reader.pp_value v))
    (fun a b -> Reader.pp_value a = Reader.pp_value b)

(** A mock transport: records (name, args) calls and answers from a
    table; an optional fallback for calls not in the table. *)
let mock_transport
    ?(fallback : (string -> value list -> (value, exn) Stdlib.result)
     option)
    ()
    : Proxy_rim.transport * (string * value list) list ref
      * (string, value list -> (value, exn) Stdlib.result) Hashtbl.t =
  let calls = ref [] in
  let table = Hashtbl.create 8 in
  let fn name args =
    calls := !calls @ [ (name, args) ];
    match Hashtbl.find_opt table name with
    | Some f -> f args
    | None -> (
        match fallback with
        | Some g -> g name args
        | None ->
            Error
              (Interp_error
                 (Prim_error (name, "not in mock table", "prim_error"))))
  in
  (fn, calls, table)

(** Pure compute stays local: (+ 1 2) makes ZERO prim calls (the proxy
    rim services + from the local pure table). *)
let test_pure_local () =
  let transport, calls, _ = mock_transport () in
  let t = Remote.make ~transport () in
  let v = Remote.eval t "(+ 1 2)" in
  Alcotest.(check int) "pure result" 3
    (match v with VInt n -> Int64.to_int n | _ -> -1);
  Alcotest.(check int) "zero prim calls (pure stayed local)" 0 (List.length
     !calls)

(** A prim call proxies to /api/prim and the result binds locally for
    subsequent pure use. (get 'user 42) is NOT in the local pure table
    ? forward ? the mock returns a VStr JSON row ? (dict-get ? name)
    runs locally on it. *)
let test_prim_proxies () =
  let transport, calls, table = mock_transport () in
  Hashtbl.replace table "get"
    (fun args ->
      (* (get 'user 42) -> a JSON string row, as the image prim does *)
      Ok (VStr {|{"id":42,"name":"ada"}|}));
  let t = Remote.make ~transport () in
  let v = Remote.eval t "(dict-get (get 'user 42) \"name\")" in
  Alcotest.(check pp_value_testable)
    "proxied prim result usable locally" (VStr "ada") v;
  Alcotest.(check int) "exactly one prim call (get)" 1 (List.length !calls);
  Alcotest.(check string) "prim name was get" "get" (fst (List.hd !calls))

(** Lazy env miss: referencing an unbound function name triggers a
    fetch of its def source (proxied via the function-source prim) and
    evals it into the env, then retries -- so the call succeeds and the
```

```
    function is cached (a second reference does NOT re-fetch). *)
let test_lazy_def_miss () =
  let transport, calls, table = mock_transport () in
  (* function-source returns the stored body for "inc" *)
  Hashtbl.replace table "function-source"
    (fun args ->
      match args with
      | [ VStr n ] when n = "inc" -> Ok (VStr "(define (inc x) (+ x 1))")
      | _ -> Ok VNil);
  let t = Remote.make ~transport () in
  let v = Remote.eval t "(inc 41)" in
  Alcotest.(check int) "lazy-loaded inc 41 = 42" 42
    (match v with VInt n -> Int64.to_int n | _ -> -1);
  (* one function-source fetch for the miss, then + runs local *)
  let fetches =
    List.length (List.filter (fun (n, _) -> n = "function-source") !calls)
  in
  Alcotest.(check int) "exactly one function-source fetch" 1 fetches;
  (* second reference: inc is now local, no new fetch *)
  let _ = Remote.eval t "(inc 100)" in
  let fetches2 =
    List.length (List.filter (fun (n, _) -> n = "function-source") !calls)
  in
  Alcotest.(check int) "second reference does not re-fetch" 1 fetches2

(** A genuinely unbound name (function-source returns nothing/empty)
    fails cleanly with Unbound_var, not an infinite retry loop. *)
let test_genuinely_unbound () =
  let transport, _calls, table = mock_transport () in
  Hashtbl.replace table "function-source" (fun _ -> Ok (VStr ""));
  let t = Remote.make ~transport () in
  (try
     ignore (Remote.eval t "(nope-such-fn 1)");
     Alcotest.fail "expected Unbound_var"
   with
  | Interp_error (Unbound_var n) ->
      Alcotest.(check string) "unbound name reported" "nope-such-fn" n
  | other -> Alcotest.fail ("wrong exn: " ^ Printexc.to_string other))

(** The client is a cache + interpreter, never a source of truth: it
    exposes NO prim that writes to a local store. There is no local
    store; the only write-ish prim is function-source (a read). Assert
    the local prim table contains no DB/branch/admin prims -- those can
    only ever be proxied.

    NOTE: [route] is NOT in the forbidden set. The client owns a LOCAL
    route table (the agent-route-table host seam, ported so the
    Nopales-authored surface runs unchanged -- borge/remote-eval.borg
    surfaces). (route path handler) writes to the client's local
    hashtbl, NOT the image's Postgres view_bindings; it is a surface
    prim, not a store write. The image's server-side [route] stays
    server-only (register_server_only) -- a different prim with the same
    name on a different host. *)
let test_no_local_store_write () =
  let transport, _calls, _ = mock_transport () in
  let t = Remote.make ~transport () in
  let forbidden =
    [ "create"; "update"; "delete"; "fork"; "merge"; "checkout";
      "create-user" ]
  in
  let local_names =
    Hashtbl.to_seq_keys t.Remote.local_prims |> List.of_seq |> List.sort
      String.compare
  in
  List.iter
    (fun f ->
      Alcotest.(check bool)
        ("no local binding for store prim " ^ f) true
        (not (List.mem f local_names)))
    forbidden

(** Hybrid placement integration: with a whole-expression remote_eval
    hook attached, a (with-transaction ?) expression is NOT run locally
```

```
    through the proxy rim -- it is POSTed to the remote_eval hook
    wholesale (hard correctness: locks can't span HTTP). No local store
    write, no per-prim proxying of the transaction body. *)
let test_transaction_routed_remote () =
  let transport, calls, _ = mock_transport () in
  let remote_calls = ref [] in
  let t =
    Remote.make ~transport
      ~remote_eval:(fun src ->
        remote_calls := src :: !remote_calls;
        VStr "committed")
      ()
  in
  let v =
    Remote.eval t
      "(with-transaction (lambda () (create 'note 'n1)))"
  in
  Alcotest.(check pp_value_testable) "hook returned value" (VStr
     "committed") v;
  Alcotest.(check int) "remote_eval called exactly once" 1 (List.length
     !remote_calls);
  (match !remote_calls with
   | [ src ] ->
       Alcotest.(check bool) "WHOLE source sent to /api/eval" true
         (String.length src > 0)
   | _ -> Alcotest.fail "expected exactly one remote eval");
  (* the transaction body was NOT proxied prim-by-prim: no /api/prim calls
     *)
  Alcotest.(check int) "zero per-prim transport calls" 0 (List.length
     !calls)

(** A Remote placement with NO remote_eval hook raises instead of
    silently running locally -- never pretend a transaction is local. *)
let test_transaction_without_hook_raises () =
  let transport, _calls, _ = mock_transport () in
  let t = Remote.make ~transport () in
  (try
     ignore (Remote.eval t "(with-transaction (lambda () (+ 1 2)))");
     Alcotest.fail "expected failwith (no remote-eval transport)"
   with
  | Failure msg ->
      Alcotest.(check bool) "raises a clear message" true
        (String.length msg > 0)
  | other -> Alcotest.fail ("wrong exn: " ^ Printexc.to_string other))

(** Stepping over a remote client (borge/remote-eval.borg stepping smoke:
    an agent loop drives a stepping eval against a remote branch -- set
    bp, step, inspect frame, continue). Runs [Remote.stepper_of]
    (Explain.trace over the client's env + local pure prims + transport);
    a breakpoint on [get] fires; stepping never re-crosses the wire, so
    the server saw exactly the calls a fast eval would (stepping is a
    purely client concern). *)
let test_stepping_via_client () =
  let transport, calls, table = mock_transport () in
  Hashtbl.replace table "get" (fun _ -> Ok (VStr "ok"));
  let t = Remote.make ~transport () in
  let tr, s = Remote.stepper_of t "(+ 1 (get 'user 42))" in
  (* the trace recorded the get round trip (data plane crossed the wire
     and returned) *)
  Alcotest.(check bool) "trace has a get round trip" true
    (List.exists
       (function
         | Explain.Remote_ok (n, _) | Explain.Remote_err (n, _) -> n = "get"
         | Explain.Local _ -> false)
       tr.events);
  (* the fast eval crosses the wire exactly once for (get 'user 42): the
     server is unaware a breakpoint fired -- one call, never re-crossed *)
  Alcotest.(check int) "one wire crossing" 1 (List.length !calls);
  (* drive: set a breakpoint on get, continue, continue to end, inspect
     the frame stack -- set bp / step / inspect frame / continue *)
  Explain.add_breakpoint s (Explain.break_on_remote "get");
  let fired = Explain.continue s in
  Alcotest.(check bool) "breakpoint on get fires" true fired;
```

```
  let fired2 = Explain.continue s in
  Alcotest.(check bool) "continue past the end does not re-trigger" false
    fired2;
  let fr = Explain.frames "(+ 1 (get 'user 42))" in
  Alcotest.(check bool) "frames inspectable (source-level AST)" true (fr <>
    [])

(** Route table + template dispatch (agent-route-table host seam,
    ported to the client -- borge/remote-eval.borg surfaces). The client
    owns a LOCAL route hashtbl; (route path handler) populates it;
    [lookup_route] / [dispatch_route] / [call_template] run handlers
    under the proxy rim so a handler's prim calls forward to /api/prim.
    This is what makes the Nopales-authored surface run on pp-remote
    unchanged. *)
let test_route_table_and_dispatch () =
  let transport, calls, table = mock_transport () in
  (* a handler that calls a proxy prim (whoami) + pure compute *)
  Hashtbl.replace table "whoami"
    (fun _args -> Ok (VStr "admin"));
  let t = Remote.make ~transport () in
  (* define a handler + bind it to a route, in Nopales (the surface
     boot-load path). *)
  ignore (Remote.eval t
    {|(define (hello) (render-html (string-append "<p>hi " (whoami)
    "</p>")))|} );
  ignore (Remote.eval t {|(route "/x" "hello")|});
  Alcotest.(check string) "lookup_route finds the handler"
    "hello"
    (match Remote.lookup_route ~rt:t "/x" with Some h -> h | None -> "");
  (* dispatch_route runs the handler under the proxy rim: whoami proxies,
     the result is rendered raw-HTML. *)
  let v = Remote.dispatch_route ~rt:t ~handler:"hello" ~method_:"GET"
            ~path:"/x" ~params:[] ~headers:[] ~body:"" in
  (match v with
   | VStr s ->
       Alcotest.(check string) "dispatch renders handler html" "<p>hi
    admin</p>" s
   | other -> Alcotest.fail ("dispatch wrong shape: " ^ Reader.pp_value
    other));
  Alcotest.(check bool) "dispatch proxied whoami" true
    (List.exists (fun (n, _) -> n = "whoami") !calls);
  (* call_template renders a named fn for the OCaml HTTP surface. *)
  (match Remote.call_template ~rt:t "hello" [] with
   | Some s -> Alcotest.(check string) "call_template renders" "<p>hi
    admin</p>" s
   | None -> Alcotest.fail "call_template returned None");
  (* query-param resolves inside a dispatched handler. *)
  ignore (Remote.eval t
    {|(define (qp) (render-html (string-append "<p>" (query-param "q")
    "</p>")))|});
  let v2 = Remote.dispatch_route ~rt:t ~handler:"qp" ~method_:"GET"
            ~path:"/y" ~params:[ ("q", "stuff") ] ~headers:[] ~body:"" in
  (match v2 with
   | VStr s -> Alcotest.(check string) "query-param resolves"
    "<p>stuff</p>" s
   | other -> Alcotest.fail ("qp wrong shape: " ^ Reader.pp_value other))

let () =
  Alcotest.run "remote"
    [
      ( "client",
        [
          Alcotest.test_case "pure compute stays local" `Quick
    test_pure_local;
          Alcotest.test_case "prim call proxies and binds locally" `Quick
    test_prim_proxies;
          Alcotest.test_case "lazy def miss fetches + evals" `Quick
    test_lazy_def_miss;
          Alcotest.test_case "genuinely unbound fails cleanly" `Quick
    test_genuinely_unbound;
          Alcotest.test_case "no local store-write prims" `Quick
    test_no_local_store_write;
```

```
269         Alcotest.test_case "transaction routed to remote_eval wholesale"
    `Quick test_transaction_routed_remote;
270         Alcotest.test_case "transaction without hook raises" `Quick
    test_transaction_without_hook_raises;
271         Alcotest.test_case "stepping over a remote client" `Quick
    test_stepping_via_client;
272         Alcotest.test_case "route table + template dispatch" `Quick
    test_route_table_and_dispatch;
273       ] );
274   ]
```

300 spikes/jsoo_interp/main.ml

```
open Pricklypear_lisp.Ast

(* ---- inline pure prims (subset copied from image/lib/prims_pure.ml,
   eliding the Unix.time helpers that jsoo can't run but structed never
   calls). This is a spike stand-in; the real plan factors prims_pure.ml
   into a shared library both image/ and client/ depend on. ---- *)

type num = NInt of int64 | NFloat of float
let num_of_value = function
  | VInt n -> NInt n
  | VFloat f -> NFloat f
  | v -> raise (Interp_error (User_error
      ("numeric op needs int/float, got "
       ^ (match v with VStr _ -> "string" | VBool _ -> "bool"
          | VNil -> "nil" | _ -> "other"), "bad-args")))
let float_of_num = function NInt n -> Int64.to_float n | NFloat f -> f
let value_of_num = function NInt n -> VInt n | NFloat f -> VFloat f
let num_add a b = match a, b with
  | NInt x, NInt y -> NInt (Int64.add x y)
  | _ -> NFloat (float_of_num a +. float_of_num b)
let num_sub a b = match a, b with
  | NInt x, NInt y -> NInt (Int64.sub x y)
  | _ -> NFloat (float_of_num a -. float_of_num b)
let num_mul a b = match a, b with
  | NInt x, NInt y -> NInt (Int64.mul x y)
  | _ -> NFloat (float_of_num a *. float_of_num b)
let num_neg = function NInt n -> NInt (Int64.neg n) | NFloat f -> NFloat
    (-. f)
let num_compare a b = match a, b with
  | NInt x, NInt y -> Int64.compare x y
  | _ -> compare (float_of_num a) (float_of_num b)

let prim_add args = value_of_num (List.fold_left (fun acc v -> num_add acc
    (num_of_value v)) (NInt 0L) args)
let prim_sub args = match args with
  | [a] -> value_of_num (num_neg (num_of_value a))
  | [a; b] -> value_of_num (num_sub (num_of_value a) (num_of_value b))
  | _ -> raise (Interp_error (User_error ("- needs 1 or 2 args",
    "bad-args")))
let prim_mul args = value_of_num (List.fold_left (fun acc v -> num_mul acc
    (num_of_value v)) (NInt 1L) args)
let prim_num_eq args = match args with
  | [a; b] -> VBool (num_compare (num_of_value a) (num_of_value b) = 0)
  | _ -> raise (Interp_error (User_error ("= needs 2 args", "bad-args")))
let prim_lt args = match args with
  | [a; b] -> VBool (num_compare (num_of_value a) (num_of_value b) < 0)
  | _ -> raise (Interp_error (User_error ("< needs 2 args", "bad-args")))
let prim_gt args = match args with
  | [a; b] -> VBool (num_compare (num_of_value a) (num_of_value b) > 0)
  | _ -> raise (Interp_error (User_error ("> needs 2 args", "bad-args")))
let prim_le args = match args with
  | [a; b] -> VBool (num_compare (num_of_value a) (num_of_value b) <= 0)
  | _ -> raise (Interp_error (User_error ("<= needs 2 args", "bad-args")))
let prim_ge args = match args with
  | [a; b] -> VBool (num_compare (num_of_value a) (num_of_value b) >= 0)
  | _ -> raise (Interp_error (User_error (">= needs 2 args", "bad-args")))
let prim_not args = match args with
  | [VBool b] -> VBool (not b)
  | [VNil] -> VBool true
  | [_] -> VBool false
  | _ -> raise (Interp_error (User_error ("not needs 1 arg", "bad-args")))
let prim_list args = VList args
let prim_car args = match args with
  | [VList (h :: _)] -> h
  | [VList []] -> VNil
  | [VPair (a, _)] -> a
  | _ -> raise (Interp_error (User_error ("car needs list/pair",
    "bad-args")))
let prim_cdr args = match args with
  | [VList (_ :: rest)] -> VList rest
  | [VList []] -> VNil
```

```
  | [VPair (_, b)] -> b
  | _ -> raise (Interp_error (User_error ("cdr needs list/pair",
     "bad-args")))
let prim_cons args = match args with
  | [a; b] -> VPair (a, b)
  | _ -> raise (Interp_error (User_error ("cons needs 2 args", "bad-args")))
let prim_null_p args = match args with
  | [VNil] -> VBool true
  | [VList []] -> VBool true
  | [_] -> VBool false
  | _ -> raise (Interp_error (User_error ("null? needs 1 arg", "bad-args")))
let prim_pair_p args = match args with
  | [VPair _] -> VBool true
  | [VList (_ :: _)] -> VBool true
  | _ -> VBool false

let rec coerce_str ?(ctx = "string") v = match v with
  | VStr s -> s
  | VSymbol s -> s
  | VList [VStr "__str__"; VStr s] -> s
  | VList [VSymbol "__str__"; VStr s] -> s
  | VInt n -> Int64.to_string n
  | VFloat f -> Printf.sprintf "%.12g" f
  | VBool true -> "true"
  | VBool false -> "false"
  | VNil -> ""
  | _ -> "<nonstr>"
let prim_string_eq args = match args with
  | [a; b] -> VBool (String.equal (coerce_str a) (coerce_str b))
  | _ -> raise (Interp_error (User_error ("string-eq needs 2 args",
     "bad-args")))
let prim_string_append args = VStr (String.concat "" (List.map coerce_str
     args))
let prim_string_length args = match args with
  | [VStr s] -> VInt (Int64.of_int (String.length s))
  | _ -> raise (Interp_error (User_error ("string-length needs 1 arg",
     "bad-args")))
let prim_as_str args = match args with
  | [v] -> VStr (coerce_str v)
  | _ -> raise (Interp_error (User_error ("as-str needs 1 arg",
     "bad-args")))
let prim_count args = match args with
  | [VList items] -> VInt (Int64.of_int (List.length items))
  | [VNil] -> VInt 0L
  | _ -> raise (Interp_error (User_error ("count needs a list",
     "bad-args")))
let rec list_of_value = function
  | VList xs -> xs
  | VNil -> []
  | VPair (a, b) -> a :: list_of_value b
  | _ -> []
let prim_length args = match args with
  | [xs] -> VInt (Int64.of_int (List.length (list_of_value xs)))
  | _ -> raise (Interp_error (User_error ("length needs 1 arg",
     "bad-args")))

let register_prims tbl =
  List.iter (fun (n, f) -> Pricklypear_lisp.Eval.register_prim tbl n f)
    [ "+", prim_add; "-", prim_sub; "*", prim_mul; "=", prim_num_eq;
      "<", prim_lt; ">", prim_gt; "<=", prim_le; ">=", prim_ge;
      "not", prim_not; "list", prim_list; "car", prim_car; "cdr", prim_cdr;
      "cons", prim_cons; "null?", prim_null_p; "pair?", prim_pair_p;
      "string-eq", prim_string_eq; "string-append", prim_string_append;
      "string-length", prim_string_length; "as-str", prim_as_str;
      "count", prim_count; "length", prim_length ]

let try_eval label prims env s =
  try Pricklypear_lisp.Eval.eval_string ~prims env s
  with Pricklypear_lisp.Ast.Interp_error err ->
    Printf.printf "FAIL %s: Interp_error %s\n" label
      (match err with
       | Pricklypear_lisp.Ast.User_error (m, _) -> "User_error: " ^ m
       | Pricklypear_lisp.Ast.Unbound_var s -> "Unbound_var: " ^ s
```

```
      | Pricklypear_lisp.Ast.Arity_mismatch (n,e,g) -> Printf.sprintf
    "Arity %s exp %d got %d" n e g
      | Pricklypear_lisp.Ast.Parse_error (m,l,c) -> Printf.sprintf "Parse
    %s (%d:%d)" m l c
      | _ -> Printexc.to_string (Pricklypear_lisp.Ast.Interp_error err));
    flush stdout; raise (Failure "interp")
  | e ->
    Printf.printf "FAIL %s: %s\n" label (Printexc.to_string e);
    flush stdout; raise e

(* Load a multi-form .pp source: read_all + eval each form under the
   effect handler, mirroring image/lib/loader.ml load_string. *)
(* The reader's `;` line-comment is broken (doesn't skip to EOL), so
   image/lib/loader.ml strips comments before read_all. Replicate here. *)
let strip_line_comments (s : string) : string =
  let buf = Buffer.create (String.length s) in
  let len = String.length s in
  let rec loop i in_str =
    if i >= len then Buffer.contents buf
    else
      let c = s.[i] in
      if in_str then begin
        Buffer.add_char buf c;
        if c = '\\' && i + 1 < len then begin
          Buffer.add_char buf s.[i + 1];
          loop (i + 2) true
        end else if c = '"' then loop (i + 1) false
        else loop (i + 1) true
      end else if c = '"' then begin
        Buffer.add_char buf c;
        loop (i + 1) true
      end else if c = ';' then
        let rec skip j =
          if j >= len || s.[j] = '\n' then j else skip (j + 1)
        in
        let j = skip i in
        if j < len then Buffer.add_char buf '\n';
        loop (if j < len then j + 1 else j) false
      else begin
        Buffer.add_char buf c;
        loop (i + 1) false
      end
  in
  loop 0 false

let eval_program ~prims env src =
  Pricklypear_lisp.Control.run_sync ~prims (fun () ->
    let forms = Pricklypear_lisp.Reader.read_all src in
    List.iter (fun e ->
        let budget = Pricklypear_lisp.Eval.make_budget () in
        ignore (Pricklypear_lisp.Eval.eval ~budget ~prims env e))
      forms)

let try_program label prims env src =
  try eval_program ~prims env src
  with Pricklypear_lisp.Reader.Reader_error (m, l, c) ->
    Printf.printf "FAIL %s: Reader_error %s (%d:%d)\n" label m l c; flush
     stdout; raise (Failure "reader")
  | Pricklypear_lisp.Ast.Interp_error err ->
    (Printf.printf "FAIL %s: Interp_error %s\n" label
       (match err with
        | Pricklypear_lisp.Ast.User_error (m, _) -> "User_error: " ^ m
        | Pricklypear_lisp.Ast.Unbound_var s -> "Unbound_var: " ^ s
        | Pricklypear_lisp.Ast.Parse_error (m,l,c) -> Printf.sprintf
    "Parse_error %s (%d:%d)" m l c
        | _ -> Printexc.to_string (Pricklypear_lisp.Ast.Interp_error err));
     flush stdout; raise (Failure "interp"))
  | e ->
    Printf.printf "FAIL %s: %s\n" label (Printexc.to_string e); flush
     stdout; raise e

let () =
  let prims = Pricklypear_lisp.Eval.make_prim_table () in
```

```
  register_prims prims;
  let env = Pricklypear_lisp.Env.create () in
  (* Mirror image/lib/session.ml: bind every prim name as VPrim in env so
     `(list ...)` resolves to a callable. The prim_table stays the
     dispatch map for Control.perform_prim. *)
  Hashtbl.iter
    (fun name _ -> Pricklypear_lisp.Env.define env name
     (Pricklypear_lisp.Ast.VPrim name))
    prims;
  Printf.printf "step: load structed/list (%d bytes)\n" (String.length
     Sources.structed_list); flush stdout;
  ignore (try_program "load-structed-list" prims env Sources.structed_list);
  Printf.printf "step: load structed (%d bytes)\n" (String.length
     Sources.structed); flush stdout;
  ignore (try_program "load-structed" prims env Sources.structed);
  Printf.printf "step: TEST1\n"; flush stdout;
  let r1 = try_eval "test1" prims env
      "(structed/list/insert-at (list 1 2 3) 1 99)" in
  Printf.printf "TEST1 structed/list/insert-at (list 1 2 3) 1 99 => %s\n"
    (Pricklypear_lisp.Reader.pp_value r1);
  Printf.printf "step: TEST2\n"; flush stdout;
  let r2 = try_eval "test2" prims env
      "(structed/barf-forward (lambda (n) (cdr n)) (lambda (n kids) (cons
     (car n) kids)) (list \"root\" (list \"A\" (list \"A1\")) (list \"B\"))
     \"A\")" in
  Printf.printf "TEST2 structed/barf-forward on tree => %s\n"
    (Pricklypear_lisp.Reader.pp_value r2);
  flush stdout
```

301 spikes/navbar-editor/install.pp

```
; navbar editor -- one-time install (run AFTER lib.pp, on the user's
    personal branch)
; captures the CURRENT ui/menu into the navbar-config row, backs up ui/menu,
; then redefines ui/menu to serve the editable items.
; re-running creates a duplicate config row (harmless: first row wins) --
; check (rows 'navbar-config) first if unsure.
(deftype (quote navbar-config) (quote items))
(define navbar/ui-menu-backup ui/menu)
(create (quote navbar-config) (dict-set* "{}" (list "items" (json-emit
    (ui/menu)))))
(define ui/menu (lambda () (navbar/items)))
(checkout (current-branch))
```

302 spikes/navbar-editor/lib.pp

```
; navbar editor -- WYSIWYG drag-and-drop navbar rearrangement (PoC spike)
; ------------------------------------------------------------------------
; storage: one data_rows row of type navbar-config, field "items" = JSON
; of the menu item tree. item shapes (same contract as menu/render):
;   ("link" label href) | ("group" label (kids ...)) | ("raw" html "")
; the chip bar reuses menu/render + scoped copies of the chrome nav CSS,
; so the editor IS the navbar (WYSIWYG). drag reorder + delete post via
; fetch (HX-Request header -> fragment response); add/reset use htmx.
; after every mutation the fragment carries a hidden #nbed-realnav div
; with the fresh (menu/bar (navbar/items)) so JS can live-update the
; real header navbar without a reload; (checkout (current-branch)) in
; each handler also resets the 60s ui_shell cache for other pages.
;
; load:   python3 scripts/pp_eval.py < spikes/navbar-editor/lib.pp
; install: python3 scripts/pp_eval.py < spikes/navbar-editor/install.pp
; (one define per line -- pp_eval stdin mode evals line by line)

; ---------- params / storage ----------
(define navbar/param (lambda (k) (if (null? (request-param k)) "" (request-param k))))
(define navbar/row-pair (lambda () (car (rows (quote navbar-config)))))
(define navbar/config-id (lambda () (car (navbar/row-pair))))
(define navbar/items (lambda () (json-parse (dict-get (get (quote navbar-config) (navbar/config-id)) "items"))))
(define navbar/save! (lambda (xs) (update (quote navbar-config) (navbar/config-id) (dict-set* "{}" (list "items" (json-emit xs))))))

; ---------- item tree helpers ----------
(define navbar/group? (lambda (it) (string-eq (car it) "group")))
(define navbar/kids (lambda (grp) (car (cdr (cdr grp)))))
; NOTE: order arrives as a JSON array string (json-parse -> proper VList).
; do NOT count() cons-built chains in this dialect: cons makes VPairs and
; count/nth-only prims reject them ("count needs a list"). json round-trip
; normalizes pairs back to proper lists on every save/load.
(define navbar/permute (lambda (items order) (if (= (count items) (count order)) (list/map (lambda (i) (nth items i)) order) items)))
(define navbar/remove-at (lambda (xs n) (if (null? xs) (list) (if (= n 0) (cdr xs) (cons (car xs) (navbar/remove-at (cdr xs) (- n 1)))))))
(define navbar/update-group (lambda (items gi f) (if (null? items) (list) (if (= gi 0) (cons (list "group" (car (cdr (car items))) (f (navbar/kids (car items)))) (cdr items)) (cons (car items) (navbar/update-group (cdr items) (- gi 1) f))))))
(define navbar/update-scope (lambda (items g f) (if (string-eq g "") (f items) (navbar/update-group items (parse-int g) f))))
(define navbar/save-scope! (lambda (g f) (navbar/save! (navbar/update-scope (navbar/items) g f))))

; ---------- rendering (chips reuse menu/render => same markup as chrome) ----------
(define navbar/chip (lambda (i it show-edit) (string-append "<span class=\"nbed-item\" draggable=\"true\" data-idx=\"" (as-str i) "\">" (menu/render it) (if (string-eq show-edit "yes") (if (navbar/group? it) (string-append "<a class=\"nbed-edit\" href=\"/nav-editor?g=" (as-str i) "\">edit</a>") "") "") "<button class=\"nbed-del\" type=\"button\" data-del=\"" (as-str i) "\" title=\"remove from navbar\">x</button>" "</span>")))
(define navbar/chips-from (lambda (items i show-edit) (if (null? items) (list) (cons (navbar/chip i (car items) show-edit) (navbar/chips-from (cdr items) (+ i 1) show-edit)))))
(define navbar/bar (lambda (items g show-edit) (string-append "<div class=\"nbed-bar\" data-g=\"" g "\">" (html/join (navbar/chips-from items 0 show-edit)) "</div>")))
(define navbar/group-panel (lambda (g items) (if (string-eq g "") "" (let ((grp (nth items (parse-int g)))) (string-append "<div class=\"nbed-group-panel\"><p class=\"nbed-hint\">editing group <strong>" (html-escape (car (cdr grp))) "</strong> &mdash; <a href=\"/nav-editor\">close</a></p>" (navbar/bar (navbar/kids grp) g "no") "</div>")))))
(define navbar/group-options (lambda (items i gsel) (if (null? items) (list) (cons (if (navbar/group? (car items)) (html/option (as-str i) (car (cdr (car items))) (string-eq gsel (as-str i))) "")
```

```
    (navbar/group-options (cdr items) (+ i 1) gsel)))))
(define navbar/add-form (lambda (g items) (string-append "<form
    class=\"nbed-add\" hx-post=\"/nav-editor/add\" hx-target=\"#nbed\"
    hx-swap=\"innerHTML\"><input type=\"text\" name=\"label\"
    placeholder=\"new link label\" required><input type=\"text\"
    name=\"href\" placeholder=\"/path\"><select name=\"g\">" (html/option
    "" "top level" (string-eq g "")) (html/join (navbar/group-options
    items 0 g)) "</select><button type=\"submit\">add
    link</button></form>")))
(define navbar/reset-row (lambda () "<p class=\"nbed-tools\"><button
    type=\"button\" hx-post=\"/nav-editor/reset\" hx-target=\"#nbed\"
    hx-swap=\"innerHTML\" hx-confirm=\"Reset navbar to package
    defaults?\">reset to defaults</button></p>"))
(define navbar/realnav (lambda () (string-append "<div id=\"nbed-realnav\"
    style=\"display:none\">" (menu/bar (navbar/items)) "</div>")))
(define navbar/fragment (lambda (g status) (let ((items (navbar/items)))
    (string-append (navbar/bar items "" "yes") (navbar/group-panel g
    items) (navbar/add-form g items) (navbar/reset-row) "<p class=\"muted
    nbed-status\">" (html-escape (if (string-eq status "") "drag items to
    rearrange -- changes save instantly" status)) " &middot; " (as-str
    (count items)) " top-level items</p>"))))
(define navbar/respond (lambda (g status) (if (hx-request?) (render-html
    (string-append (navbar/fragment g status) (navbar/realnav))) (redirect
    (if (string-eq g "") "/nav-editor" (string-append "/nav-editor?g="
    g))))))

; ---------- mutation handlers ----------
(define navbar/reorder (lambda () (if (string-eq (request-method) "POST")
    (let ((g (navbar/param "g"))) (seq (navbar/save-scope! g (lambda (xs)
    (navbar/permute xs (json-parse (navbar/param "order"))))) (seq
    (checkout (current-branch)) (navbar/respond g "order saved"))))
    (redirect "/nav-editor"))))
(define navbar/add (lambda () (if (string-eq (request-method) "POST") (let
    ((g (navbar/param "g")) (label (string-trim (navbar/param "label")))
    (href (string-trim (navbar/param "href")))) (if (string-eq label "")
    (navbar/respond g "label required") (seq (navbar/save-scope! g (lambda
    (xs) (append xs (list (menu/link label (if (string-eq href "") "#"
    href)))))) (seq (checkout (current-branch)) (navbar/respond g
    (string-append "added " label)))))) (redirect "/nav-editor"))))
(define navbar/delete (lambda () (if (string-eq (request-method) "POST")
    (let ((g (navbar/param "g"))) (seq (navbar/save-scope! g (lambda (xs)
    (navbar/remove-at xs (parse-int (navbar/param "idx"))))) (seq
    (checkout (current-branch)) (navbar/respond g "deleted")))) (redirect
    "/nav-editor"))))
(define navbar/reset (lambda () (if (string-eq (request-method) "POST")
    (seq (navbar/save! (navbar/ui-menu-backup)) (seq (checkout
    (current-branch)) (navbar/respond "" "reset to defaults"))) (redirect
    "/nav-editor"))))

; ---------- page ----------
(define navbar/page (lambda () (if (hx-request?) (navbar/respond
    (navbar/param "g") "") (layout/page (list (html/h1 "navbar editor")
    (html/p* (html/attr "class" "muted") "This bar below IS your real
    navbar (same markup, same CSS). Drag items to rearrange -- the header
    above updates instantly. Click edit on a group to rearrange its
    contents.") (navbar/style) (string-append "<div id=\"nbed\">"
    (navbar/fragment (navbar/param "g") "") "</div>") (navbar/script))))))

; ---------- styles (scoped copies of the chrome nav CSS = pixel-true)
    ----------
(define navbar/style (lambda () (html/tag "style" "" (html/join (list
    ".nbed-bar{display:flex;flex-wrap:wrap;gap:.35rem
    .5rem;align-items:center;border:1px dashed
    #666;background:#f4f4f4;padding:.4rem;margin:.35rem
    0;min-height:3rem}" ".nbed-bar a{display:inline-block;padding:.45rem
    .55rem;min-height:2.5rem;line-height:1.5;border:1px solid
    #000;background:#c0c0c0;color:#000;text-decoration:none;font-size:.9rem;box-sizing:border-box}"
    ".nbed-item{display:inline-flex;align-items:center;gap:.2rem;cursor:grab;border:1px
    dashed #999;background:#fff;padding:.12rem;user-select:none}"
    ".nbed-item.dragging{opacity:.4}"
    ".nbed-item.drop-before{box-shadow:-4px 0 0 #000}"
    ".nbed-item.drop-after{box-shadow:4px 0 0 #000}" ".nbed-item
    a,.nbed-item details,.nbed-item summary{pointer-events:none}"
```

```
    ".nbed-item a.nbed-edit{pointer-events:auto}"
    ".nbed-del,.nbed-edit{font-size:.7rem;line-height:1;padding:.15rem
    .3rem;min-height:0;border:1px solid
    #000;background:#e8e8e8;color:#000;cursor:pointer;text-decoration:none}"
    ".nbed-del:hover,.nbed-edit:hover{background:#fff}"
    ".nbed-group-panel{border:1px solid
    #000;background:#fafafa;padding:.4rem;margin:.35rem 0}"
    ".nbed-hint{font-size:.85rem;color:#333;margin:.1rem 0 .3rem}"
    ".nbed-add{display:flex;gap:.35rem;flex-wrap:wrap;align-items:center;margin:.5rem
    0}" ".nbed-add
    input[type=text]{font-size:.9rem;padding:.3rem;min-height:2rem;width:auto;flex:0
    1 14rem}" ".nbed-add select{min-height:2rem}" ".nbed-add
    button,.nbed-tools button{width:auto;min-width:5rem}"
    ".nbed-status{font-size:.85rem}")))))

; ---------- drag-and-drop (native HTML5 DnD, event delegation, no
    rebinding after swaps) ----------
(define navbar/script (lambda () (html/tag "script" "" (string-append
    "(function(){" "function itemOf(t){return
    t&&t.closest?t.closest('.nbed-item'):null;}" "function barOf(t){return
    t&&t.closest?t.closest('.nbed-bar'):null;}" "var dragEl=null;"
    "function clearMarks(){var
    ms=document.querySelectorAll('.nbed-item.drop-before,.nbed-item.drop-after');for(var
    i=0;i<ms.length;i++){ms[i].classList.remove('drop-before');ms[i].classList.remove('drop-after');}}"
    "function nearest(c,x,y){var
    items=c.querySelectorAll('.nbed-item');var
    best=null,bd=1e9,before=true;for(var
    i=0;i<items.length;i++){if(items[i]===dragEl)continue;var
    r=items[i].getBoundingClientRect();var
    cx=r.left+r.width/2,cy=r.top+r.height/2;var
    d=(cx-x)*(cx-x)+(cy-y)*(cy-y);if(d<bd){bd=d;best=items[i];before=(Math.abs(y-cy)<r.height*0.35)?(x<cx):(y<cy);}}return{
    "document.addEventListener('dragstart',function(e){var
    it=itemOf(e.target);if(!it)return;dragEl=it;it.classList.add('dragging');if(e.dataTransfer){e.dataTransfer.effectAllowe
    "document.addEventListener('dragend',function(){if(dragEl)dragEl.classList.remove('dragging');dragEl=null;clearMarks();
    "document.addEventListener('dragover',function(e){var
    c=barOf(e.target);if(!c||!dragEl)return;if(barOf(dragEl)!==c)return;e.preventDefault();if(e.dataTransfer)e.dataTransfer
    n=nearest(c,e.clientX,e.clientY);if(n.el)n.el.classList.add(n.before?'drop-before':'drop-after');});"
    "document.addEventListener('drop',function(e){var
    c=barOf(e.target);if(!c||!dragEl)return;e.preventDefault();var
    n=nearest(c,e.clientX,e.clientY);if(n.el){c.insertBefore(dragEl,n.before?n.el:n.el.nextSibling);}else{c.appendChild(dra
    "document.addEventListener('click',function(e){var
    b=e.target&&e.target.closest?e.target.closest('.nbed-del'):null;if(!b)return;e.preventDefault();var
    c=barOf(b);post('/nav-editor/delete','idx='+b.getAttribute('data-del')+'&g='+encodeURIComponent(c?c.getAttribute('data-
    "function postReorder(c){var
    items=c.querySelectorAll('.nbed-item');var ids=[];for(var
    i=0;i<items.length;i++)ids.push(parseInt(items[i].getAttribute('data-idx'),10));post('/nav-editor/reorder','order='+enc
    "function
    post(url,body){fetch(url,{method:'POST',headers:{'Content-Type':'application/x-www-form-urlencoded','HX-Request':'true'
    r.text();}).then(function(h){swap(h);}).catch(function(err){});}"
    "function swap(h){var
    el=document.getElementById('nbed');if(!el)return;el.innerHTML=h;if(window.htmx)htmx.process(el);syncRealnav();}"
    "function syncRealnav(){var
    rn=document.getElementById('nbed-realnav');var
    nav=document.querySelector('header
    nav');if(rn&&nav)nav.innerHTML=rn.innerHTML;}"
    "document.body.addEventListener('htmx:afterSwap',function(e){if(e.target&&e.target.id==='nbed')syncRealnav();});"
    "})();"))))

; ---------- routes ----------
(route "/nav-editor" "navbar/page")
(route "/nav-editor/reorder" "navbar/reorder")
(route "/nav-editor/add" "navbar/add")
(route "/nav-editor/delete" "navbar/delete")
(route "/nav-editor/reset" "navbar/reset")
```

303 spikes/navbar-editor/uninstall.pp

```
; navbar editor -- uninstall: restore the original ui/menu lambda.
; routes stay registered (page will error without a config row) --
; delete the row too if you want it fully gone:
;   (delete 'navbar-config (navbar/config-id))
(define ui/menu navbar/ui-menu-backup)
(checkout (current-branch))
```